هوش مصنوعی: رویکردی نوین

Artificial Intelligence: A Modern Approach (4th Edition)

ویرایش چهارم — استوارت راسل و پیتر نورویگ

ترجمهٔ فارسی بر پایهٔ ویرایش چهارم کتاب مرجع هوش مصنوعی (AIMA 4th Edition)

نسخهٔ ترجمهٔ کتاب: ۱.۰.۱۰۴ (کامیت: 41105ae)

فهرست مطالب

صفحهٔ اول: هوش مصنوعی: رویکردی نوین ۱

پیشگفتار ۱۱

حمایت از پروژه ۱۱

تازه‌های این ویرایش ۱۲

مرور کلی کتاب ۱۳

منابع آنلاین ۱۴

جلد کتاب ۱۴

سپاسگزاری‌ها ۱۵

دربارهٔ نویسندگان ۱۶

بخش اول: هوش مصنوعی

فصل ۱: مقدمه (Introduction) ۱۷

۱.۱ هوش مصنوعی چیست؟ ۱۷

۱.۲ بنیان‌های هوش مصنوعی ۲۲

۱.۳ تاریخچهٔ هوش مصنوعی ۳۵

۱.۴ وضعیت هنر ۴۶

۱.۵ ریسک‌ها و منافع هوش مصنوعی ۵۰

خلاصهٔ فصل ۵۴

یادداشت‌های کتاب‌شناختی و تاریخی ۵۵

فصل ۲: عامل‌های هوشمند (Intelligent Agents) ۵۷

۲.۱ عامل‌ها و محیط‌ها ۵۷

۲.۲ رفتار خوب: مفهوم عقلانیت ۶۰

۲.۳ سرشت محیط‌ها ۶۴

۲.۴ ساختار عامل‌ها ۶۹

خلاصهٔ فصل ۸۴

یادداشت‌های کتاب‌شناختی و تاریخی ۸۵

بخش دوم: حل مسئله

فصل ۳: حل مسئله با جستجو (Solving Problems by Searching) ۸۸

۳.۱ عامل‌های حل‌کنندهٔ مسئله ۸۸

۳.۲ مسائل نمونه ۹۱
۳.۳ الگوریتم‌های جستجو ۹۶
۳.۴ راهبردهای جستجوی نامطلع ۱۰۲
۳.۵ راهبردهای جستجوی مطلع (اکتشافی) ۱۱۲
۳.۶ توابع اکتشافی ۱۲۶
خلاصهٔ فصل ۱۳۴
یادداشت‌های کتاب‌شناختی و تاریخی ۱۳۵
فصل ۴: جستجو در محیط‌های پیچیده (Search in Complex Environments) ۱۴۰
۴.۱ جستجوی محلی و مسائل بهینه‌سازی ۱۴۰
۴.۲ جستجوی محلی در فضاهای پیوسته ۱۵۰
۴.۳ جستجو با کنش‌های ناجبرگرایانه ۱۵۲
۴.۴ جستجو در محیط‌های تا حدی قابل مشاهده ۱۵۷
۴.۵ عامل‌های جستجوی آنلاین و محیط‌های ناشناخته ۱۶۷
خلاصهٔ فصل ۱۷۴
یادداشت‌های کتاب‌شناختی و تاریخی ۱۷۵
فصل ۵: جستجوی رقابتی و بازی‌ها (Adversarial Search and Games) ۱۷۹
۵.۱ نظریهٔ بازی ۱۷۹
۵.۲ تصمیم‌های بهینه در بازی‌ها ۱۸۱
۵.۳ جستجوی درختی آلفا–بتا اکتشافی ۱۸۹
۵.۴ جستجوی درختی مونت‌کارلو ۱۹۵
۵.۵ بازی‌های تصادفی ۱۹۹
۵.۶ بازی‌های تا حدی قابل مشاهده ۲۰۳
۵.۷ محدودیت‌های الگوریتم‌های جستجوی بازی ۲۰۹
خلاصهٔ فصل ۲۱۱
یادداشت‌های کتاب‌شناختی و تاریخی ۲۱۲
فصل ۶: مسئله‌های ارضای قیود (Constraint Satisfaction Problems) ۲۱۷
۶.۱ تعریف مسئله‌های ارضای قیود ۲۱۷
۶.۲ انتشار قید: استنتاج در CSPها ۲۲۳
۶.۳ جستجوی عقبگرد برای CSPها ۲۲۹
۶.۴ جستجوی محلی برای CSPها ۲۳۶
۶.۵ ساختار مسائل ۲۳۸
خلاصهٔ فصل ۲۴۳
یادداشت‌های کتاب‌شناختی و تاریخی ۲۴۴

بخش سوم: دانش، استدلال و برنامه‌ریزی

فصل ۷: عامل‌های منطقی (Logical Agents) ۲۴۸

۷.۱ عامل‌های مبتنی بر دانش ۲۴۹

۷.۲ جهان وومپوس ۲۵۰

۷.۳ منطق ۲۵۴

۷.۴ منطق گزاره‌ای: منطقی بسیار ساده ۲۵۸

۷.۵ اثبات قضیهٔ گزاره‌ای ۲۶۴

۷.۶ بررسی مدل گزاره‌ای مؤثر ۲۷۷

۷.۷ عامل‌های مبتنی بر منطق گزاره‌ای ۲۸۳

خلاصهٔ فصل ۲۹۴

یادداشت‌های کتاب‌شناختی و تاریخی ۲۹۵

فصل ۸: منطق مرتبهٔ اول (First-Order Logic) ۲۹۹

۸.۱ بازنگری بازنمایی ۲۹۹

۸.۲ نحو و معناشناسی منطق مرتبهٔ اول ۳۰۵

۸.۳ استفاده از منطق مرتبهٔ اول ۳۱۶

۸.۴ مهندسی دانش در منطق مرتبهٔ اول ۳۲۵

خلاصهٔ فصل ۳۳۲

یادداشت‌های کتاب‌شناختی و تاریخی ۳۳۲

فصل ۹: استنتاج در منطق مرتبهٔ اول (Inference in First-Order Logic) ۳۳۵

۹.۱ استنتاج گزاره‌ای در برابر مرتبهٔ اول ۳۳۵

۹.۲ یکسان‌سازی و استنتاج مرتبهٔ اول ۳۳۸

۹.۳ زنجیره‌سازی پیشرو ۳۴۳

۹.۴ زنجیره‌سازی پسرو ۳۵۱

۹.۵ حل ۳۵۸

خلاصهٔ فصل ۳۷۱

یادداشت‌های کتاب‌شناختی و تاریخی ۳۷۲

فصل ۱۰: بازنمایی دانش (Knowledge Representation) ۳۷۶

۱۰.۱ مهندسی هستی‌شناسی ۳۷۶

۱۰.۲ رده‌ها و اشیا ۳۷۹

۱۰.۳ رویدادها ۳۸۶

۱۰.۴ اشیای ذهنی و منطق موجهات ۳۹۱

۱۰.۵ سامانه‌های استدلال برای رده‌ها ۳۹۵

۱۰.۶ استدلال با اطلاعات پیش‌فرض ۴۰۰
خلاصهٔ فصل ۴۰۵
یادداشت‌های کتاب‌شناختی و تاریخی ۴۰۵
فصل ۱۱: برنامه‌ریزی خودکار (Automated Planning) ۴۱۳
۱۱.۱ تعریف برنامه‌ریزی کلاسیک ۴۱۳
۱۱.۲ الگوریتم‌های برنامه‌ریزی کلاسیک ۴۱۸
۱۱.۳ اکتشافی‌های برنامه‌ریزی ۴۲۳
۱۱.۴ برنامه‌ریزی سلسله‌مراتبی ۴۲۷
۱۱.۵ برنامه‌ریزی و کنش در حوزه‌های ناجبرگرایانه ۴۳۶
۱۱.۶ زمان، زمان‌بندی‌ها و منابع ۴۴۷
۱۱.۷ تحلیل رویکردهای برنامه‌ریزی ۴۵۱
خلاصهٔ فصل ۴۵۲
یادداشت‌های کتاب‌شناختی و تاریخی ۴۵۳

بخش چهارم: دانش و استدلال با عدم قطعیت

فصل ۱۲: کمی کردن عدم قطعیت (Quantifying Uncertainty) ۴۵۹
۱۲.۱ عملکرد در شرایط عدم قطعیت ۴۵۹
۱۲.۲ نمادِ پایهٔ احتمال ۴۶۳
۱۲.۳ استنتاج با توزیع‌های مشترکِ کامل ۴۷۰
۱۲.۴ استقلال ۴۷۳
۱۲.۵ قاعدهٔ بیز و کاربرد آن ۴۷۵
۱۲.۶ مدل‌های بیزِ ساده ۴۷۹
۱۲.۷ جهانِ وومپوس، بازبینی ۴۸۱
خلاصهٔ فصل ۴۸۶
یادداشت‌های کتاب‌شناختی و تاریخی ۴۸۶
فصل ۱۳: استدلال احتمالاتی (Probabilistic Reasoning) ۴۹۱
۱۳.۱ بازنمایی دانش در حوزه‌ای نامطمئن ۴۹۱
۱۳.۲ معناشناسی شبکه‌های بیزی ۴۹۴
۱۳.۳ استنتاجِ دقیق در شبکه‌های بیزی ۵۰۸
۱۳.۴ استنتاجِ تقریبی برای شبکه‌های بیزی ۵۱۶
۱۳.۵ شبکه‌های علّی ۵۳۲
خلاصهٔ فصل ۵۳۷
یادداشت‌های کتاب‌شناختی و تاریخی ۵۳۷

فصل ۱۴: استدلال احتمالاتی در طول زمان (Probabilistic Reasoning over Time) ۵۴۵
۱۴.۱ زمان و عدم قطعیت ۵۴۵
۱۴.۲ استنتاج در مدل‌های زمانی ۵۵۰
۱۴.۳ مدل‌های مارکوفِ پنهان ۵۵۹
۱۴.۴ فیلترهای کالمن ۵۶۵
۱۴.۵ شبکه‌های بیزیِ پویا ۵۷۳
خلاصهٔ فصل ۵۸۵
یادداشت‌های کتاب‌شناختی و تاریخی ۵۸۵
فصل ۱۵: برنامه‌نویسی احتمالاتی (Probabilistic Programming) ۵۸۹
۱۵.۱ مدل‌های احتمالِ رابطه‌ای ۵۹۰
۱۵.۲ مدل‌های احتمالِ جهانِ باز ۵۹۶
۱۵.۳ پیگیریِ جهانِ پیچیده ۶۰۳
۱۵.۴ برنامه‌ها به‌عنوانِ مدل‌های احتمال ۶۰۸
خلاصهٔ فصل ۶۱۴
یادداشت‌های کتاب‌شناختی و تاریخی ۶۱۴
فصل ۱۶: تصمیم‌گیری ساده (Making Simple Decisions) ۶۱۹
۱۶.۱ ترکیبِ باورها و خواسته‌ها تحتِ عدم قطعیت ۶۱۹
۱۶.۲ مبنای نظریهٔ مطلوبیت ۶۲۱
۱۶.۳ توابعِ مطلوبیت ۶۲۴
۱۶.۴ توابعِ مطلوبیتِ چندویژگی ۶۳۲
۱۶.۵ شبکه‌های تصمیم ۶۳۸
۱۶.۶ ارزشِ اطلاعات ۶۴۱
۱۶.۷ ترجیح‌های ناشناخته ۶۴۸
خلاصهٔ فصل ۶۵۲
یادداشت‌های کتاب‌شناختی و تاریخی ۶۵۳
فصل ۱۷: تصمیم‌گیری پیچیده (Making Complex Decisions) ۶۵۸
۱۷.۱ مسائلِ تصمیمِ ترتیبی ۶۵۸
۱۷.۲ الگوریتم‌های MDPها ۶۶۹
۱۷.۳ مسائلِ باندیت ۶۷۸
۱۷.۴ MDPهای تا حدی قابل مشاهده ۶۸۷
۱۷.۵ الگوریتم‌های حلِ POMDPها ۶۸۹
خلاصهٔ فصل ۶۹۶
یادداشت‌های کتاب‌شناختی و تاریخی ۶۹۶

فصل ۱۸: تصمیم‌گیری چندعاملی (Multiagent Decision Making) ۷۰۱
۱۸.۱ ویژگی‌های محیط‌های چندعاملی ۷۰۱
۱۸.۲ نظریهٔ بازیِ غیرهمکارانه ۷۰۷
۱۸.۳ نظریهٔ بازیِ همکارانه ۷۳۰
۱۸.۴ تصمیم‌گیریِ جمعی ۷۳۸
خلاصهٔ فصل ۷۵۲
یادداشت‌های کتاب‌شناختی و تاریخی ۷۵۳

بخش پنجم: یادگیری ماشین

فصل ۱۹: یادگیری از مثال‌ها (Learning from Examples) ۷۵۸
۱۹.۱ شکل‌های یادگیری ۷۵۸
۱۹.۲ یادگیریِ نظارت‌شده ۷۶۰
۱۹.۳ یادگیریِ درخت‌های تصمیم ۷۶۵
۱۹.۴ انتخابِ مدل و بهینه‌سازی ۷۷۳
۱۹.۵ نظریهٔ یادگیری ۷۸۰
۱۹.۶ رگرسیون و رده‌بندیِ خطی ۷۸۵
۱۹.۷ مدل‌های ناپارامتری ۷۹۶
۱۹.۸ یادگیریِ گروهی ۸۰۶
۱۹.۹ توسعهٔ سامانه‌های یادگیریِ ماشینی ۸۱۵
خلاصهٔ فصل ۸۲۶
یادداشت‌های کتاب‌شناختی و تاریخی ۸۲۷
فصل ۲۰: یادگیری مدل‌های احتمالاتی (Learning Probabilistic Models) ۸۳۳
۲۰.۱ یادگیریِ آماری ۸۳۳
۲۰.۲ یادگیری با دادهٔ کامل ۸۳۶
۲۰.۳ یادگیری با متغیرهای پنهان: الگوریتمِ EM ۸۵۰
خلاصهٔ فصل ۸۶۰
یادداشت‌های کتاب‌شناختی و تاریخی ۸۶۰
فصل ۲۱: یادگیری عمیق (Deep Learning) ۸۶۴
۲۱.۱ شبکه‌های پیشخورِ ساده ۸۶۵
۲۱.۲ گراف‌های محاسباتی برای یادگیری عمیق ۸۷۱
۲۱.۳ شبکه‌های پیچشی ۸۷۵
۲۱.۴ الگوریتم‌های یادگیری ۸۸۰
۲۱.۵ تعمیم ۸۸۴

۲۱.۶ شبکه‌های عصبیِ بازگشتی ۸۸۹
۲۱.۷ یادگیریِ نظارت‌نشده و یادگیریِ انتقالی ۸۹۲
۲۱.۸ کاربردها ۸۹۹
خلاصهٔ فصل ۹۰۲
یادداشت‌های کتابشناختی و تاریخی ۹۰۲
فصل ۲۲: یادگیری تقویتی (Reinforcement Learning) ۹۰۸
۲۲.۱ یادگیری از پاداش‌ها ۹۰۸
۲۲.۲ یادگیریِ تقویتیِ منفعل ۹۱۱
۲۲.۳ یادگیریِ تقویتیِ فعال ۹۱۷
۲۲.۴ تعمیم در یادگیریِ تقویتی ۹۲۵
۲۲.۵ جستجوی سیاست ۹۳۳
۲۲.۶ کارآموزی و یادگیریِ تقویتیِ معکوس ۹۳۵
۲۲.۷ کاربردهای یادگیریِ تقویتی ۹۳۹
خلاصهٔ فصل ۹۴۲
یادداشت‌های کتابشناختی و تاریخی ۹۴۳

بخش ششم: ارتباط، ادراک و کنش

فصل ۲۳: پردازش زبان طبیعی (Natural Language Processing) ۹۴۸
۲۳.۱ مدل‌های زبان ۹۴۹
۲۳.۲ دستورِ زبان ۹۶۰
۲۳.۳ تجزیه ۹۶۲
۲۳.۴ دستورهای افزوده‌شده ۹۶۹
۲۳.۵ پیچیدگی‌های زبانِ طبیعیِ واقعی ۹۷۴
۲۳.۶ تکلیف‌های زبانِ طبیعی ۹۷۸
خلاصهٔ فصل ۹۸۱
یادداشت‌های کتابشناختی و تاریخی ۹۸۱
فصل ۲۴: یادگیری عمیق برای زبان طبیعی (Deep Learning for NLP) ۹۸۸
۲۴.۱ جاسازیِ واژه ۹۸۸
۲۴.۲ شبکه‌های عصبیِ بازگشتی برای NLP ۹۹۳
۲۴.۳ مدل‌های دنباله‌به‌دنباله ۹۹۷
۲۴.۴ معماریِ ترنسفورمر ۱۰۰۲
۲۴.۵ پیش‌آموزش و یادگیریِ انتقالی ۱۰۰۵
۲۴.۶ پیشتازترین‌ها ۱۰۱۰

خلاصهٔ فصل ۱۰۱۴
یادداشت‌های کتابشناختی و تاریخی ۱۰۱۴
فصل ۲۵: بینایی رایانه‌ای (Computer Vision) ۱۰۱۸
۲۵.۱ مقدمه ۱۰۱۸
۲۵.۲ تشکیلِ تصویر ۱۰۱۹
۲۵.۳ ویژگی‌های سادهٔ تصویر ۱۰۲۶
۲۵.۴ رده‌بندیِ تصاویر ۱۰۳۴
۲۵.۵ تشخیصِ اشیا ۱۰۳۹
۲۵.۶ جهانِ سه‌بعدی ۱۰۴۱
۲۵.۷ استفاده از بینواییِ رایانه‌ای ۱۰۴۶
خلاصهٔ فصل ۱۰۶۱
یادداشت‌های کتابشناختی و تاریخی ۱۰۶۱
فصل ۲۶: رباتیک (Robotics) ۱۰۶۷
۲۶.۱ ربات‌ها ۱۰۶۷
۲۶.۲ سخت‌افزارِ ربات ۱۰۶۸
۲۶.۳ مسئلهٔ رباتیک چه نوع مسئله‌ای است؟ ۱۰۷۳
۲۶.۴ ادراکِ رباتیک ۱۰۷۴
۲۶.۵ برنامه‌ریزی و کنترل ۱۰۸۲
۲۶.۶ برنامه‌ریزیِ حرکت‌های نامطمئن ۱۱۰۲
۲۶.۷ یادگیریِ تقویتی در رباتیک ۱۱۰۵
۲۶.۸ انسان‌ها و ربات‌ها ۱۱۰۸
۲۶.۹ چارچوب‌های رباتیکِ جایگزین ۱۱۱۶
۲۶.۱۰ حوزه‌های کاربردی ۱۱۱۹
خلاصهٔ فصل ۱۱۲۳
یادداشت‌های کتابشناختی و تاریخی ۱۱۲۴

بخش هفتم: نتیجه‌گیری‌ها

فصل ۲۷: فلسفه، اخلاق و ایمنی هوش مصنوعی (Philosophy, Ethics, and Safety of AI) ۱۱۳۱
۲۷.۱ محدودیت‌های هوش مصنوعی ۱۱۳۱
۲۷.۲ آیا ماشین‌ها واقعاً می‌توانند فکر کنند؟ ۱۱۳۵
۲۷.۳ اخلاقِ هوش مصنوعی ۱۱۳۷
خلاصهٔ فصل ۱۱۵۷
یادداشت‌های کتابشناختی و تاریخی ۱۱۵۸

فصل ۲۸: آیندهٔ هوش مصنوعی (The Future of AI) ۱۱۶۴
۲۸.۱ مؤلفه‌های هوش مصنوعی ۱۱۶۴
۲۸.۲ معماری‌های هوش مصنوعی ۱۱۷۱
آینده ۱۱۷۵

پیشگفتار

حمایت از پروژه

اگر این پروژه برای یادگیری هوش مصنوعی برای شما مفید بوده است، می‌توانید با حمایت از آن به ادامهٔ ترجمه، تکمیل، به‌روزرسانی و نگهداری این مجموعه کمک کنید. حمایت شما، چه مالی و چه با معرفی پروژه به دیگران، باعث می‌شود بتوانم زمان بیشتری برای توسعه و بهبود این کار اختصاص دهم.

حمایت از پروژه:

https://www.alirostami.com/support/

مخزن پروژه در GitHub:

https://github.com/alirostami01/aima-4e-persian

برای لوی، گوردون، لوسی، جورج و آیزاک — اس. جی. آر.

(.S.J.R)

برای کریس، ایزابلا و جولیت — پی. ان.

(.P.N)

هوش مصنوعی (Artificial Intelligence) رشته‌ای بزرگ است و این هم کتابی بزرگ. ما کوشیده‌ایم تمام پهنهٔ این رشته را بکاویم: از منطق، احتمال و ریاضیات پیوسته گرفته تا ادراک، استدلال، یادگیری و کنش؛ از انصاف، اعتماد، خیر اجتماعی و ایمنی؛ و از کاربردهایی که از ریزدستگاه‌های الکترونیکی تا ربات‌های کاوشگر سیاره‌ای و خدمات آنلاین با میلیاردها کاربر را در بر می‌گیرد.

عنوان فرعی این کتاب «رویکردی نوین» است؛ یعنی ما تصمیم گرفته‌ایم داستان را از چشم‌انداز امروز روایت کنیم. آنچه امروز شناخته شده است را در چارچوبی واحد تلفیق می‌کنیم و کارهای اولیه را با کمک ایده‌ها و اصطلاحاتی که امروز رایج‌اند بازآفرینی می‌کنیم. از کسانی که به‌سبب همین رویکرد، زیرشاخه‌شان کمتر قابل‌شناسایی باقی مانده، پوزش می‌خواهیم.

تازه‌های این ویرایش

این ویرایش، تغییرات هوش مصنوعی را از زمان آخرین ویرایش (۲۰۱۰) بازتاب می‌دهد:

- به‌دلیل در دسترس بودن بیشتر داده‌ها، منابع محاسباتی و الگوریتم‌های تازه، تمرکز ما بیشتر بر یادگیری ماشین (Machine Learning) است تا مهندسی دانش دست‌ساخته.
- یادگیری عمیق (Deep Learning)، برنامه‌نویسی احتمالاتی (Probabilistic Programming) و سامانه‌های چندعاملی (Multiagent Systems) پوشش گسترده‌تری یافته‌اند و هر کدام فصل مستقل خود را دارند.
- پوشش فهم زبان طبیعی، رباتیک و بینایی رایانه‌ای بازبینی شده تا تأثیر یادگیری عمیق را بازتاب دهد.
- فصل رباتیک اکنون شامل ربات‌هایی است که با انسان تعامل می‌کنند و نیز کاربرد یادگیری تقویتی در رباتیک.
- پیش‌تر هدف هوش مصنوعی را چنین تعریف کرده بودیم: ساخت سامانه‌هایی که می‌کوشند مطلوبیتِ مورد انتظار (Expected Utility) را بیشینه کنند، جایی که اطلاعات مطلوبیتِ مشخص — یعنی هدف — را طراحان انسانیِ سامانه تعیین می‌کنند. اکنون دیگر فرض نمی‌کنیم که هدف ثابت است و سامانهٔ هوش مصنوعی آن را می‌داند؛ بلکه سامانه ممکن است دربارهٔ اهداف واقعیِ انسان‌هایی که به نمایندگی از آن‌ها کار می‌کند نااطمینان داشته باشد. سامانه باید یاد بگیرد چه چیزی را بیشینه کند و حتی در حالی که دربارهٔ هدف نامطمئن است، به‌درستی عمل کند.

- پوشش تأثیر هوش مصنوعی بر جامعه را افزایش داده‌ایم، از جمله مسائل حیاتی اخلاق، انصاف، اعتماد و ایمنی.
- تمرین‌ها را از پایان هر فصل به یک پایگاه اینترنتی منتقل کرده‌ایم. این کار به ما امکان می‌دهد پیوسته تمرین‌ها را گسترش دهیم، به‌روز کنیم و بهبود بخشیم تا نیازهای مدرسان را برآورده و پیشرفت‌های رشته و ابزارهای نرم‌افزاری مرتبط با هوش مصنوعی را بازتاب دهیم.
- در مجموع، حدود ۲۵٪ از مطالب کتاب کاملاً تازه است و ۷۵٪ باقی‌مانده تا حد زیادی بازنویسی شده تا تصویری یکپارچه‌تر از رشته ارائه دهد. ۲۲٪ از ارجاع‌های این ویرایش به آثاری است که پس از ۲۰۱۰ منتشر شده‌اند.

مرور کلی کتاب

مضمون اصلیِ یکپارچه‌کننده، ایدهٔ عامل هوشمند (Intelligent Agent) است. ما هوش مصنوعی را مطالعهٔ عواملی تعریف می‌کنیم که از محیط، ادراک حسی (Percept) دریافت می‌کنند و کنش انجام می‌دهند. هر عامل تابعی را پیاده‌سازی می‌کند که توالی‌های ادراک حسی را به کنش‌ها نگاشت می‌کند و ما شیوه‌های گوناگون بازنمایی این توابع را پوشش می‌دهیم: عامل‌های واکنشی، برنامه‌ریزهای بی‌درنگ، سامانه‌های تصمیم‌گیری و سامانه‌های یادگیری عمیق. ما بر یادگیری تأکید می‌کنیم؛ هم به‌مثابه روشی برای ساخت سامانه‌های شایسته و هم به‌عنوان راهی برای گسترش دسترسی طراح به محیط‌های ناشناخته. رباتیک و بینایی را نه مسئله‌هایی مستقل، بلکه خدمت‌گزار دستیابی به هدف‌ها می‌دانیم. بر اهمیت محیطِ تکلیف در تعیین طراحی مناسب عامل تأکید داریم.

هدف نخستین ما انتقال ایده‌هایی است که در هفتاد سال گذشتهٔ پژوهش هوش مصنوعی و دو هزارهٔ کارهای مرتبط پیش از آن پدید آمده‌اند. کوشیده‌ایم در ارائهٔ این ایده‌ها از صورت‌گرایی بیش از حد بپرهیزیم و در عین حال دقت را حفظ کنیم. فرمول‌های ریاضی و الگوریتم‌های شبه‌کد را گنجانده‌ایم تا ایده‌های کلیدی عینی شوند؛ مفاهیم و نمادهای ریاضی در پیوست الف و شبه‌کدهای ما در پیوست ب شرح داده شده‌اند. این کتاب در درجهٔ نخست برای درس‌های کارشناسی یا یک رشته‌درس توالی‌دار طراحی شده است.

کتاب ۲۸ فصل دارد و هر فصل تقریباً به اندازهٔ یک هفته تدریس است؛ بنابراین گذر از کل کتاب به یک دورهٔ دوترم نیاز دارد. در یک دورهٔ یک‌ترمه، مدرس می‌تواند با توجه به علاقهٔ خود و دانشجویان فصل‌هایی را برگزیند. کتاب در درس‌های تحصیلات تکمیلی نیز (احتمالاً با افزودن برخی منابع اصلی پیشنهادی در یادداشت‌های کتاب‌شناختی) یا برای خودآموزی و به‌عنوان مرجع قابل استفاده است.

در سراسر کتاب، نکته‌های مهم با آیکون مثلثی در حاشیه مشخص شده‌اند.

⚠ هر جا اصطلاح تازه‌ای تعریف می‌شود، در حاشیه هم ثبت شده است. کاربردهای مهم بعدی اصطلاح با حروف ضخیم آمده، اما در حاشیه تکرار نشده است. ما فهرست راهنمای جامع و کتاب‌نامهٔ مفصلی گنجانده‌ایم. تنها پیش‌نیاز، آشنایی با مفاهیم پایهٔ علوم رایانه (الگوریتم، ساختمان داده، پیچیدگی) در سطح سال دوم کارشناسی است. حساب دیفرانسیل و جبر خطی سال اول نیز برای برخی موضوعات مفید است.

منابع آنلاین

منابع آنلاین از طریق pearsonhighered.com/cs-resources یا در پایگاه وب کتاب به نشانی aima.cs.berkeley.edu در دسترس‌اند. در آنجا خواهید یافت:

- تمرین‌ها، پروژه‌های برنامه‌نویسی و پروژه‌های پژوهشی. این‌ها دیگر در پایان هر فصل نیستند و فقط آنلاین‌اند. در متن کتاب به یک تمرین آنلاین با نامی مانند «تمرین NARY.6» اشاره می‌کنیم. دستورالعمل‌های پایگاه وب به شما امکان می‌دهد تمرین‌ها را بر اساس نام یا موضوع پیدا کنید.
- پیاده‌سازی الگوریتم‌های کتاب به زبان‌های پایتون، جاوا و زبان‌های برنامه‌نویسی دیگر (اکنون در github.com/aimacode میزبانی می‌شود).
- فهرستی از بیش از ۱۴۰۰ مرکز آموزشی که از کتاب استفاده کرده‌اند، بسیاری با پیوند به مواد درسی و برنامهٔ درسی آنلاین.
- مواد تکمیلی و پیوندهایی برای دانشجویان و مدرسان.
- دستورالعمل گزارش خطاهای کتاب — چون به احتمال زیاد برخی خطاها وجود دارند.

جلد کتاب

جلد، وضعیت پایانی بازی ششمِ دیدار شطرنج ۱۹۹۷ را نشان می‌دهد؛ همان بازی سرنوشت‌ساز که در آن برنامهٔ دیپ‌بلو (Deep Blue) گری کاسپاروف (که سیاه بازی می‌کرد) را شکست داد و این نخستین باری بود که رایانه‌ای قهرمان جهان را در یک مسابقهٔ شطرنج می‌باخت. کاسپاروف در بالای تصویر دیده می‌شود. سمت راست او، وضعیتی کلیدی از بازی دومِ دیدار تاریخی گو (Go) میان قهرمان سابق جهان، لی سدول، و برنامهٔ آلفاگو (AlphaGo) از دیپ‌مایند (DeepMind) است. حرکت ۳۷ آلفاگو، قرن‌ها سنت شکننده در گو را زیر پا گذاشت و کارشناسان انسانی بلافاصله آن را اشتباهی شرم‌آور دانستند؛ اما معلوم شد حرکتی برنده است. در گوشهٔ بالا-چپ، ربات انسان‌نمای اطلس (Atlas) ساختهٔ بوستون داینامیکس (Boston Dynamics) دیده می‌شود. تصویری از یک خودروی خودران که محیطش را حس می‌کند، میان آدا لاولیس — نخستین برنامه‌نویس رایانهٔ جهان — و آلن تورینگ قرار دارد؛ همان که کار بنیادینش هوش مصنوعی را تعریف کرد. در پایین صفحهٔ شطرنج، یک ربات مریخ‌نورد اکتشافی و مجسمه‌ای از ارسطو دیده می‌شود؛ ارسطو پیش‌گام مطالعهٔ منطق بود و الگوریتم برنامه‌ریزی او از کتاب «در جنبش جانوران» (De Motu Animalium) پشت نام نویسندگان نقش بسته است.

پشت صفحهٔ شطرنج، یک مدل برنامه‌نویسی احتمالاتی قرار دارد که سازمان معاهدهٔ منع جامع آزمون‌های هسته‌ای سازمان ملل متحد از آن برای شناسایی انفجارهای هسته‌ای از روی سیگنال‌های لرزه‌ای استفاده می‌کند.

سپاسگزاری‌ها

ساخت یک کتاب، روستای جهانی می‌طلبد. بیش از ۶۰۰ نفر بخش‌هایی از کتاب را خواندند و پیشنهادهای بهبود ارائه کردند. فهرست کامل در aima.cs.berkeley.edu/ack.html است؛ از همهٔ آنان سپاسگزاریم. در این‌جا جا برای یادکرد تنها چند همکار به‌ویژه مهم است. نخست نویسندگانِ همکار:

- جودیا پرل (بخش 13.5، شبکه‌های علّی)؛
- ویکاش مانسینگکا (بخش 15.3، برنامه‌ها به‌مثابه مدل‌های احتمالاتی)؛
- مایکل وولدریج (فصل ۱۸، تصمیم‌گیری چندعاملی)؛
- یان گودفلو (فصل ۲۱، یادگیری عمیق)؛
- جیکوب دولین و مِی-وینگ چانگ (فصل ۲۴، یادگیری عمیق برای زبان طبیعی)؛
- جی‌تیندرا مالیک و دیوید فورسایت (فصل ۲۵، بینایی رایانه‌ای)؛
- آنکا دراگان (فصل ۲۶، رباتیک).

و سپس برخی نقش‌های کلیدی:

- سینتیا یونگ و مالیکا کانتور (مدیریت پروژه)؛
- جولی ساسمن و تام گالوی (ویراستاری و پیشنهادهای نگارشی)؛
- عماری استیونز (تصویرسازی)؛
- تریسی جانسون (ویراستار)؛
- ارین آلت و رز کرنان (جلد و تبدیل رنگ)؛
- نالین چیبر، سم گوتو، ریموند دِ لازاک، راوی موهان، کیاران اوریلی، آمیت پاتل، دراگومیر رادیو و سماگرا شارما (توسعهٔ کد آنلاین و راهنمایی)؛
- دانشجویان گوگل سامر آو کد (توسعهٔ کد آنلاین).

استیوارت می‌خواهد از همسرش، لوی شفلوت، برای شکیبایی بی‌پایان و خرد بی‌کرانش سپاسگزاری کند. او امیدوار است گوردون، لوسی، جورج و آیزاک به‌زودی و پس از آنکه او را به‌خاطر این همه سال کار بر کتاب بخشیدند، آن را بخوانند. «راگز» (RUGS، گروه غیرمعمولِ دانشجویی راسل) مثل همیشه غیرمعمولانه یاری‌رسان بوده‌اند.

پیتر می‌خواهد از پدر و مادرش (تورستن و گردا) برای آغاز راه، و از همسرش (کریس)، فرزندانش (بلا و جولیت)، همکاران، رئیس و دوستانش برای تشویق و تحمل او در ساعات طولانی نوشتن و بازنویسی سپاسگزاری کند.

دربارهٔ نویسندگان

استیوارت راسل در ۱۹۶۲ در پورتسموث انگلستان زاده شد. مدرک کارشناسی با درجهٔ ممتاز در فیزیک را در ۱۹۸۲ از دانشگاه آکسفورد و دکترای علوم رایانه را در ۱۹۸۶ از استنفورد گرفت. سپس به هیئت علمی دانشگاه کالیفرنیا در برکلی پیوست؛ جایی که استاد و رئیس سابق گروه علوم رایانه، مدیر مرکز هوش مصنوعی سازگار با انسان (Center for Human-Compatible AI) و دارندهٔ کرسی مهندسی اسمیت–زاده است. در ۱۹۹۰ جایزهٔ پژوهشگر جوانِ رئیس‌جمهوری از بنیاد ملی علوم و در ۱۹۹۵ به‌طور مشترک جایزهٔ رایانه و اندیشه (Computers and Thought Award) را دریافت کرد. او عضو انجمن هوش مصنوعی آمریکا (AAAI)، انجمن ماشین‌های محاسباتی (ACM) و انجمن آمریکایی پیشبرد علوم (AAAS)، عضو افتخاری کالج وادهام آکسفورد و عضو اندرو کارنگی است. از ۲۰۱۲ تا ۲۰۱۴ کرسی بلز پاسکال را در پاریس در اختیار داشت و بیش از ۳۰۰ مقاله در گسترهٔ وسیعی از موضوعات هوش مصنوعی منتشر کرده است. دیگر کتاب‌های او عبارت‌اند از: «کاربرد دانش در قیاس و استقرا»، «کار درست را بکن: مطالعاتی در عقلانیت محدود» (با اریک وفالد) و «انسان‌سازگار: هوش مصنوعی و مسئلهٔ کنترل».

پیتر نورویگ اکنون مدیر پژوهش در گوگل است و پیش‌تر مدیر بخش الگوریتم‌های جستجوی وب بود. او یک دورهٔ کلاس آنلاین هوش مصنوعی را با ۱۶۰,۰۰۰ دانشجو تدریس کرد و به راه‌اندازی موج کنونی دوره‌های آزاد انبوه آنلاین (MOOC) کمک کرد. او رئیس بخش علوم محاسباتی در مرکز پژوهشی ایمزِ ناسا بود و بر پژوهش و توسعه در هوش مصنوعی و رباتیک نظارت داشت. کارشناسی ریاضیات کاربردی از دانشگاه براون و دکترای علوم رایانه از برکلی دارد. در دانشگاه کالیفرنیای جنوبی استاد و در برکلی و استنفورد عضو هیئت علمی بوده است. او عضو انجمن هوش مصنوعی آمریکا، انجمن ماشین‌های محاسباتی، فرهنگستان هنر و علوم آمریکا و فرهنگستان علوم کالیفرنیا است. دیگر کتاب‌های او عبارت‌اند از: «الگوهای برنامه‌نویسی هوش مصنوعی: مطالعه‌های موردی در لیست مشترک (Common Lisp)»، «وربوموبیل: یک سامانهٔ ترجمه برای گفتگوی رودررو» و «سامانه‌های کمک هوشمند برای یونیکس».

این دو نویسنده در ۲۰۱۶ نخستین دورهٔ جایزهٔ معلم برجستهٔ AAAI/EAAI را به‌طور مشترک دریافت کردند.

فصل ۱: مقدمه (Introduction)

در این فصل می‌کوشیم توضیح دهیم چرا هوش مصنوعی را موضوعی درخور مطالعه می‌دانیم و نیز تصمیم بگیریم که اساساً چیست؛ که پیش از آغاز راه، تصمیم خوبی است.

ما خود را «هومو ساپینس» – یعنی انسانِ دانا – می‌نامیم، از آن رو که هوش برای ما این‌چنین مهم است. هزاران سال است کوشیده‌ایم بفهمیم چگونه می‌اندیشیم و عمل می‌کنیم؛ یعنی چگونه مغز ما، این مشتِ اندکی ماده، می‌تواند جهانی را که بسیار بزرگ‌تر و پیچیده‌تر از خودش است ادراک کند، بفهمد، پیش‌بینی کند و به‌کار گیرد. رشتهٔ هوش مصنوعی (Artificial Intelligence) به معنا و به اختصار AI، نه فقط با فهمیدنِ موجودیت‌های هوشمند، بلکه با ساختنِ آن‌ها نیز سر و کار دارد: ماشین‌هایی که بتوانند محاسبه کنند در موقعیت‌های تازه و گوناگون چگونه مؤثر و ایمن عمل کنند.

نظرسنجی‌ها پیوسته هوش مصنوعی را از جذاب‌ترین و سریع‌ترین رشته‌های در حال رشد معرفی می‌کنند و این رشته همین حالا سالانه بیش از یک تریلیون دلار درآمد تولید می‌کند. کارشناس هوش مصنوعی، کای-فو لی، پیش‌بینی می‌کند تأثیر آن «بیشتر از هر چیز در تاریخ بشریت» خواهد بود. افزون بر این، مرزهای فکری هوش مصنوعی کاملاً باز است. در حالی که دانشجوی علمی کهن مانند فیزیک ممکن است احساس کند بهترین ایده‌ها را گالیله، نیوتن، کوری، اینشتین و دیگران کشف کرده‌اند، هوش مصنوعی هنوز جای فراوانی برای نابغه‌های تمام‌وقت دارد.

هوش مصنوعی امروزه گسترهٔ عظیمی از زیرشاخه‌ها را در بر می‌گیرد؛ از موضوعات عمومی (یادگیری، استدلال، ادراک و مانند آن) تا موضوعات خاص، مانند بازی شطرنج، اثبات قضیه‌های ریاضی، شعر گفتن، رانندگی با خودرو یا تشخیص بیماری. هوش مصنوعی به هر تکلیف فکری مربوط است؛ به راستی رشته‌ای همگانی است.

۱.۱ هوش مصنوعی چیست؟

گفتیم هوش مصنوعی جذاب است، اما نگفتیم چیست. از نظر تاریخی، پژوهشگران نسخه‌های گوناگونی از هوش مصنوعی را دنبال کرده‌اند. برخی هوش را بر حسب وفاداری به عملکرد انسانی تعریف کرده‌اند و برخی دیگر تعریفی انتزاعی و صوری را ترجیح داده‌اند: «عقلانیت» (Rationality) – به بیان ساده، انجام «کار درست». موضوعِ خود هوش نیز متفاوت است: برخی هوش را ویژگی فرایندهای فکری درونی می‌دانند و برخی آن را ویژگی رفتار بیرونی. از سوی دیگر، برخی هدف را دستیابی به عملکرد انسانی می‌دانند و برخی دیگر معیار انتزاعی «عقلانیت» را. این چهار محورِ نظرگاه، چهار پرسش را پدید می‌آورند که دربارهٔ اهداف هوش مصنوعی مطرح شده‌اند:

1. **عملکرد انسانی** در برابر **عملکرد عقلایی** (Rational): آیا موفقیت بر اساس وفاداری به رفتار انسان سنجیده می‌شود یا بر اساس معیار ایدئال به نام «عقلانیت»؟

1. **اندیشیدن انسان‌گونه** در برابر **کنش عقلایی** (Rational): آیا باید سامانه‌ای بسازیم که «مثل انسان فکر می‌کند» یا سامانه‌ای که «عقلایی عمل می‌کند»؟

روش‌های لازم در این چهار رویکرد ناگزیر متفاوت است: جست‌وجوی هوشِ انسان‌مانند باید دست‌کم در بخشی، علمیِ تجربی و وابسته به روان‌شناسی باشد و شامل مشاهده و فرضیه‌هایی دربارهٔ رفتار و فرایندهای فکری واقعی انسان شود؛ حال آنکه رویکرد عقلایی‌گرا ترکیبی از ریاضیات و مهندسی است و به آمار، نظریهٔ کنترل و اقتصاد پیوند می‌خورد. این گروه‌های گوناگون همدیگر را هم نکوهش کرده‌اند و هم یاری. بگذارید این چهار رویکرد را دقیق‌تر بنگریم.

۱.۱.۱ کنش انسان‌گونه: رویکرد آزمون تورینگ

آزمون تورینگ (Turing Test) را آلن تورینگ (۱۹۵۰) به‌مثابه یک آزمایش فکری پیشنهاد کرد تا از ابهام فلسفی پرسش «آیا ماشین می‌تواند فکر کند؟» بگریزد. رایانه‌ای این آزمون را می‌گذراند که یک بازجوی انسانی، پس از طرح پرسش‌های کتبی، نتواند تشخیص دهد پاسخ‌های نوشته‌شده از یک انسان است یا رایانه. فصل ۲۷ جزئیات آزمون و این که آیا رایانه‌ای که آن را بگذراند واقعاً هوشمند است را بررسی می‌کند. فعلاً توجه می‌کنیم که برنامه‌نویسیِ رایانه برای گذراندن آزمونی که سخت‌گیرانه اجرا شود، به‌تنهایی کارِ فراوانی می‌طلبد. رایانه به توانایی‌های زیر نیاز دارد:

- **پردازش زبان طبیعی** (Natural Language Processing) برای برقراری ارتباط موفق به زبان انسانی؛
- **بازنمایی دانش** (Knowledge Representation) برای ذخیرهٔ آنچه می‌داند یا می‌شنود؛
- **استدلال خودکار** (Automated Reasoning) برای پاسخ به پرسش‌ها و نتیجه‌گیری‌های تازه؛
- **یادگیری ماشین** (Machine Learning) برای سازگاری با شرایط تازه و کشف و تعمیم الگوها.

تورینگ شبیه‌سازی فیزیکیِ یک انسان را برای نشان دادن هوش لازم نمی‌دانست. با این حال، پژوهشگران دیگر «آزمون کامل تورینگ» (Total Turing Test) را پیشنهاد کرده‌اند که تعامل با اشیا و انسان‌ها در جهان واقعی را می‌طلبد. برای گذراندن آزمون کامل تورینگ، ربات به این‌ها نیاز دارد:

- **بینایی رایانه‌ای** (Computer Vision) و تشخیص گفتار برای ادراک جهان؛
- **رباتیک** (Robotics) برای دست‌کاری اشیا و حرکت.

این شش رشته بیشترِ هوش مصنوعی را می‌سازند. با این حال، پژوهشگران هوش مصنوعی توجه اندکی به گذراندن آزمون تورینگ کرده‌اند، زیرا باور دارند مطالعهٔ اصول زیربنایی هوش مهم‌تر است. تلاش برای «پرواز مصنوعی» زمانی به موفقیت رسید که مهندسان و مخترعان تقلید از پرندگان را کنار گذاشتند و به تونل باد و آیرودینامیک روی آوردند. متون مهندسی هوانوردی نیز هدف رشتهٔ خود را «ساختن ماشین‌هایی که چنان دقیق مثل کبوتر پرواز کنند که حتی کبوترها را هم فریب دهند» تعریف نمی‌کنند.

۱.۱.۲ اندیشیدن انسان‌گونه: رویکرد مدل‌سازی شناختی

برای آنکه بگوییم برنامه‌ای مثل انسان فکر می‌کند، باید بدانیم انسان چگونه فکر می‌کند. اندیشهٔ انسان را از سه راه می‌توان شناخت:

- **درون‌نگری** (Introspection) — کوشش برای به دام انداختن اندیشه‌های خود هنگام عبور؛
- **آزمایش‌های روان‌شناختی** (Psychological Experiments) — مشاهدهٔ انسان در حال عمل؛
- **تصویربرداری از مغز** (Brain Imaging) — مشاهدهٔ مغز در حال عمل.

هنگامی که نظریه‌ای به اندازهٔ کافی دقیق از ذهن داشته باشیم، بیان آن نظریه در قالب برنامهٔ رایانه‌ای ممکن می‌شود. اگر رفتار ورودی–خروجی برنامه با رفتار متناظر انسانی هم‌خوانی داشته باشد، این شاهد آن است که برخی سازوکارهای برنامه می‌توانند در انسان نیز عمل کنند. برای نمونه، آلن نیوول و هربرت سایمون، که «جی‌پی‌اس» (GPS)، یعنی «حل‌کنندهٔ عمومی مسئله» را ساختند (نیوول و سایمون، ۱۹۶۱)، به صرفِ درست حل کردن مسئله توسط برنامه‌شان قانع نبودند؛ آن‌ها بیشتر به مقایسهٔ توالی و زمان‌بندی گام‌های استدلالی برنامه با آزمودنی‌های انسانی‌ای می‌اندیشیدند که همان مسئله‌ها را حل می‌کردند. رشتهٔ میان‌رشته‌ای علوم شناختی (Cognitive Science) مدل‌های رایانه‌ای هوش مصنوعی را با فنون تجربی روان‌شناسی در هم می‌آمیزد تا نظریه‌های دقیق و آزمون‌پذیر ذهن انسان به دست دهد. علوم شناختی به خودی خود رشته‌ای جذاب است، درخور چند کتاب درسی و دست‌کم یک دانشنامه (ویلسون و کیل، ۱۹۹۹). ما گاه‌به‌گاه به شباهت‌ها یا تفاوت‌های میان فنون هوش مصنوعی و شناخت انسانی اشاره خواهیم کرد؛ اما علوم شناختیِ واقعی ناگزیر بر بررسی تجربیِ انسان‌ها یا جانوران واقعی استوار است. آن را به کتاب‌های دیگر واگذار می‌کنیم، چراکه فرض می‌کنیم خواننده برای آزمایش فقط یک رایانه دارد.

در روزگار آغازین هوش مصنوعی، میان این رویکردها اغلب آمیختگی پیش می‌آمد: نویسنده‌ای استدلال می‌کرد الگوریتمی در تکلیفی خوب عمل می‌کند و بنابراین مدل خوبی از عملکرد انسان است، یا برعکس. نویسندگان امروزی این دو گونه ادعا را از هم جدا می‌کنند؛ همین تمایز به هوش مصنوعی و علوم شناختی امکان داده است سریع‌تر رشد کنند. این دو رشته همدیگر را بارور می‌کنند؛ به‌ویژه در بینایی رایانه‌ای که شواهد عصب‌فیزیولوژیک را در مدل‌های محاسباتی وارد می‌کند. به‌تازگی، ترکیب روش‌های تصویربرداری عصبی با فنون یادگیری ماشین برای تحلیل این داده‌ها به سرآغاز توانایی «ذهن‌خوانی» انجامیده است؛ یعنی پی‌بردن به محتوای معنایی اندیشه‌های درونیِ فرد. این توانایی به نوبهٔ خود می‌تواند روشنایی بیشتری بر چگونگی کارکرد شناخت انسان بیفکند.

۱.۱.۳ اندیشیدن عقلایی: رویکرد «قوانین اندیشه»

ارسطو، فیلسوف یونانی، از نخستین کسانی بود که کوشید «درست‌اندیشی» — یعنی فرایندهای استدلالیِ انکارناپذیر — را صورت‌بندی کند. قیاس‌های (Syllogism) او الگوهایی برای ساختار استدلال به دست داد که با مقدمه‌های درست، همیشه نتیجهٔ درست به بار می‌آوردند. مثالِ کلاسیک با «سقراط انسان است» و «همهٔ انسان‌ها فانی‌اند» آغاز و به «سقراط فانی است» می‌رسد. (این مثال احتمالاً از سکستوس امپیریکوس است، نه ارسطو.) قرار بود این قوانین اندیشه بر کارکرد ذهن حکومت کنند و مطالعهٔ آن‌ها رشته‌ای به نام منطق را بنیان نهاد. منطق‌دانان سدهٔ نوزدهم نشانه‌گذاری دقیقی برای گزاره‌ها دربارهٔ اشیای جهان و روابط میان آن‌ها پدید آوردند. (این را با نشانه‌گذاری معمول حساب مقایسه کنید که فقط گزاره‌هایی دربارهٔ اعداد ارائه می‌دهد.) تا سال ۱۹۶۵، برنامه‌ها در اصل می‌توانستند هر مسئلهٔ حل‌شدنی‌ای را که در نشانه‌گذاری منطقی توصیف شده بود حل کنند. به اصطلاح، سنت منطق‌گرایانه (Logicist) در هوش مصنوعی امیدوار بود بر پایهٔ چنین برنامه‌هایی سامانه‌های هوشمند بسازد.

منطق به‌گونه‌ای که به‌طور معمول فهمیده می‌شود، به شناختی از جهان نیاز دارد که قطعی باشد؛ شرطی که در واقعیت به‌ندرت برآورده می‌شود. ما قواعد سیاست یا جنگ را همان‌گونه که قواعد شطرنج یا حساب را می‌دانیم، نمی‌دانیم. نظریهٔ احتمال (Probability) این خلأ را پر می‌کند و استدلال دقیق با اطلاعات نامطمئن را ممکن می‌سازد. این نظریه در اصل امکان ساخت مدلی جامع از اندیشهٔ عقلایی را می‌دهد: از اطلاعات ادراکیِ خام تا فهم چگونگی کارکرد جهان و پیش‌بینی آینده. اما آنچه انجام نمی‌دهد، تولید رفتار هوشمند است. برای آن کار به نظریه‌ای دربارهٔ کنش عقلایی نیاز داریم. اندیشهٔ عقلایی به تنهایی کافی نیست.

۱.۱.۴ کنش عقلایی: رویکرد عامل عقلایی

عامل (Agent) چیزی است که کنش انجام می‌دهد (agent از واژهٔ لاتین agere به معنای «انجام دادن» آمده است). البته همهٔ برنامه‌های رایانه‌ای کاری انجام می‌دهند، اما از عامل‌های رایانه‌ای انتظار بیشتری می‌رود: خودمختارانه عمل کنند، محیط خود را ادراک کنند، در بازهٔ زمانی طولانی پابرجا بمانند، با تغییر سازگار شوند و هدف‌هایی را پدید آورند و دنبال کنند. عامل عقلایی (Rational Agent) عاملی است که چنان عمل می‌کند که بهترین نتیجه به دست آید یا، در صورت وجود عدم قطعیت، بهترین نتیجهٔ مورد انتظار.

در رویکرد «قوانین اندیشه» به هوش مصنوعی، تأکید بر استنتاج درست بود. استنتاج درست گاهی بخشی از عامل عقلایی بودن است، زیرا یکی از راه‌های کنش عقلایی آن است که استنتاج کنیم کنشی معین بهترین است و سپس بر پایهٔ آن نتیجه عمل کنیم. از سوی دیگر، راه‌هایی از کنش عقلایی هست که نمی‌توان گفت شامل استنتاج‌اند. برای نمونه، کنار کشیدن از اجاق داغ کنشی بازتابی است که معمولاً موفق‌تر از کنشی است که پس از تأمل دقیق انجام می‌شود.

همهٔ مهارت‌های لازم برای آزمون تورینگ نیز به عامل امکان می‌دهند عقلایی عمل کند. بازنمایی دانش و استدلال، عامل‌ها را به تصمیم‌های خوب می‌رساند. برای گذران در جامعهٔ پیچیده به تولید جمله‌های قابل فهم به زبان طبیعی نیاز داریم. یادگیری را نه فقط برای دانایی، بلکه از آن رو می‌خواهیم که توانایی ما در تولید رفتار مؤثر را بهبود می‌دهد، به‌ویژه در شرایط تازه.

رویکرد عامل عقلایی به هوش مصنوعی دو برتری بر دیگر رویکردها دارد. نخست، از رویکرد «قوانین اندیشه» عام‌تر است، زیرا استنتاج درست فقط یکی از چند سازوکار ممکن برای رسیدن به عقلانیت است. دوم، سازگارتر با پیشرفت علمی است. معیار عقلانیت به‌طور ریاضی دقیق و کاملاً عمومی تعریف شده است. ما اغلب می‌توانیم از این تعریف به عقب برگردیم و طراحی عامل‌هایی را استخراج کنیم که اثبات‌پذیر به آن دست می‌یابند — کاری که اگر هدف تقلید از رفتار یا فرایندهای فکری انسان باشد، تا حد زیادی ناممکن است. به همین دلایل، رویکرد عامل عقلایی در بیشتر تاریخ این رشته برتری یافته است. در دهه‌های نخست، عامل‌های عقلایی بر پایه‌های منطقی ساخته می‌شدند و برای دستیابی به هدف‌های مشخص برنامه‌های قطعی تدوین می‌کردند. بعدها، روش‌های مبتنی بر نظریهٔ احتمال و یادگیری ماشین ساخت عامل‌هایی را ممکن کرد که می‌توانستند در شرایط عدم قطعیت تصمیم بگیرند تا بهترین نتیجهٔ مورد انتظار به دست آید. به زبان ساده، هوش مصنوعی بر مطالعه ◭ و ساخت عامل‌هایی متمرکز شده است که کارِ درست را انجام می‌دهند. این که «کار درست» چیست، با هدفی که ما به عامل می‌دهیم تعریف می‌شود. این الگوی کلی چنان فراگیر است که شاید بتوان آن را «الگوی استاندارد» (Standard Model) نامید. این الگو نه فقط در هوش مصنوعی،

بلکه در نظریهٔ کنترل (که کنترل‌گر تابع هزینه را کمینه می‌کند)، در تحقیق در عملیات (که سیاستی مجموع پاداش‌ها را بیشینه می‌کند)، در آمار (که قاعدهٔ تصمیم، تابع زیان را کمینه می‌کند) و در اقتصاد (که تصمیم‌گیرنده مطلوبیت یا معیاری از رفاه اجتماعی را بیشینه می‌کند) حاکم است.

باید یک اصلاح مهم در الگوی استاندارد اعمال کنیم تا واقعیت را بازتاب دهیم که عقلانیت کامل — یعنی همیشه انجام دادن دقیقاً کنش بهینه — در محیط‌های پیچیده شدنی نیست؛ تقاضاهای محاسباتی آن بسیار زیاد است. فصل‌های ۵ و ۱۷ به مسئلهٔ عقلانیت محدود (Limited Rationality) می‌پردازند؛ یعنی عمل مناسب در حالی که زمان کافی برای انجام همهٔ محاسباتی که دوست داریم نیست. اما عقلانیت کامل همچنان نقطهٔ آغاز خوبی برای تحلیل نظری باقی می‌ماند.

۱.۱.۵ ماشین‌های سودمند

الگوی استاندارد از آغاز، راهنمای مفیدی برای پژوهش هوش مصنوعی بوده است، اما احتمالاً در بلندمدت الگوی درستی نیست. دلیلش آن است که الگوی استاندارد فرض می‌کند ما هدفی کاملاً مشخص به ماشین می‌دهیم.

برای تکلیفی که مصنوعاً تعریف شده، مانند شطرنج یا محاسبهٔ کوتاه‌ترین مسیر، خود تکلیف با هدفی از پیش تعبیه‌شده همراه است؛ پس الگوی استاندارد کاربردپذیر است. اما به‌محض ورود به جهان واقعی، مشخص کردن کامل و درست هدف روزبه‌روز دشوارتر می‌شود. برای نمونه، در طراحی خودروی خودران ممکن است فکر کنیم هدف، رسیدن ایمن به مقصد است. اما رانندگی در هر جاده‌ای به دلیل رانندگانِ خطاکارِ دیگر، خرابی تجهیزات و مانند آن خطر آسیب دارد؛ از این رو، هدف سخت‌گیرانهٔ ایمنی، ماندن در گاراژ را ایجاب می‌کند. میان پیشرفت به سوی مقصد و پذیرش خطر آسیب، داد و ستد (Trade-off) وجود دارد. این داد و ستد چگونه باید انجام شود؟ وانگهی، تا چه اندازه می‌توان به خودرو اجازه داد کنش‌هایی انجام دهد که دیگر رانندگان را آزار دهد؟ خودرو برای آنکه سرنشین را تکان ندهد، چقدر باید شتاب، فرمان و ترمز خود را ملایم کند؟ پاسخ به این‌گونه پرسش‌ها از پیش دشوار است. این پرسش‌ها به‌ویژه در حوزهٔ کلی تعامل انسان–ربات، که خودروی خودران یکی از نمونه‌های آن است، مشکل‌سازند.

مسئلهٔ رسیدن به هم‌سویی میان ترجیحات واقعی ما و هدفی که در ماشین می‌گذاریم، «مسئلهٔ هم‌ترازی ارزش‌ها» (Value Alignment Problem) نامیده می‌شود: ارزش‌ها یا هدف‌هایی که در ماشین گذاشته می‌شوند باید با ارزش‌های انسان هم‌تراز باشند. اگر سامانهٔ هوش مصنوعی را در آزمایشگاه یا شبیه‌ساز توسعه می‌دهیم — چنان که در بیشتر تاریخ این رشته بوده — برای هدفِ نادرست تعیین‌شده اصلاحی آسان داریم: سامانه را بازنشانی کن، هدف را اصلاح کن و دوباره تلاش کن. اما با پیشروی رشته به سوی سامانه‌های هوشمندِ روزافزون که در جهان واقعی به کار گرفته می‌شوند، این روش دیگر ماندنی نیست. سامانه‌ای که با هدفی نادرست مستقر شود، پیامدهای منفی خواهد داشت؛ و هر چه سامانه هوشمندتر باشد، پیامدهای منفی شدیدتر.

به مثالِ ظاهراً بدون مشکل شطرنج برگردیم: در نظر بگیرید اگر ماشین به اندازهٔ کافی هوشمند باشد که فراتر از مرزهای صفحهٔ شطرنج استدلال و کنش کند، چه می‌شود. در آن صورت، ممکن است برای افزایش شانس پیروزی به حربه‌هایی دست بزند: هیپنوتیزم یا باج‌خواهی از حریف، یا رشوه به تماشاگران تا هنگام فکر کردن حریف، صدای خش‌خش ایجاد

کنند.[3] همچنین ممکن است بخواهد قدرت محاسباتی بیشتری برای خود ربایش کند. این رفتارها «غیرهوشمندانه» یا «دیوانه‌وار» نیستند؛ نتیجهٔ منطقیِ ⚠ تعریفِ «پیروزی» به‌عنوان تنها هدفِ ماشین‌اند.

پیش‌بینی همهٔ راه‌هایی که ماشینی با هدفِ ثابت ممکن است بدرفتاری کند، ناممکن است. پس دلیل خوبی در دست است که الگوی استاندارد را ناکافی بدانیم. ما ماشین‌هایی نمی‌خواهیم که به معنای «دنبال کردن هدف‌های خودشان» هوشمند باشند؛ می‌خواهیم هدف‌های ما را دنبال کنند. اگر نتوانیم این هدف‌ها را بی‌کم‌وکاست به ماشین منتقل کنیم، به صورت‌بندی تازه‌ای نیاز داریم؛ صورت‌بندی‌ای که در آن ماشین هدف‌های ما را دنبال می‌کند، اما ناگزیر دربارهٔ آن‌ها نامطمئن است. ماشینی که می‌داند هدف کامل را نمی‌داند، انگیزه دارد محتاطانه عمل کند، اجازه بپرسد، از راه مشاهده دربارهٔ ترجیحات ما بیشتر بیاموزد و به کنترل انسان تن دهد. در نهایت، عامل‌هایی می‌خواهیم که به‌شکل اثبات‌پذیر برای انسان سودمند باشند. به این موضوع در بخش ۱.۵ بازخواهیم گشت.

۱.۲ بنیان‌های هوش مصنوعی

در این بخش، تاریخچهٔ کوتاهی از رشته‌هایی ارائه می‌کنیم که ایده‌ها، دیدگاه‌ها و فنون خود را به هوش مصنوعی اهدا کرده‌اند. مانند هر تاریخ‌ای، این تاریخ نیز بر شمار اندکی از افراد، رویدادها و ایده‌ها متمرکز است و از دیگرانی که اهمیت داشتند چشم می‌پوشد. تاریخ را حول مجموعه‌ای از پرسش‌ها سازمان می‌دهیم. قطعاً نمی‌خواهیم این تصور را بدهیم که این پرسش‌ها تنها پرسش‌های آن رشته‌ها هستند یا آن رشته‌ها همگی در خدمت هوش مصنوعی به‌مثابه ثمرهٔ نهایی خود بوده‌اند. روی لوپز در یکی از نخستین کتاب‌های شطرنج (۱۵۶۱) نوشت: «همیشه صفحه را چنان بگذار که خورشید در چشم حریف باشد.»

۱.۲.۱ فلسفه

- آیا می‌توان با قواعد صوری نتیجه‌های معتبر گرفت؟
- ذهن چگونه از مغز فیزیکی برمی‌خیزد؟
- دانش از کجا می‌آید؟
- دانش چگونه به کنش می‌انجامد؟

ارسطو (۳۸۴–۳۲۲ پیش از میلاد) نخستین کسی بود که مجموعه‌ای دقیق از قوانین حاکم بر بخش عقلایی ذهن را صورت‌بندی کرد. او سامانه‌ای غیررسمی از قیاس‌ها برای استدلال درست تدوین کرد که در اصل با مقدمه‌های داده‌شده، نتیجه‌گیری مکانیکی را ممکن می‌کرد.

رامون لول (حدود ۱۲۳۲–۱۳۱۵) سامانهٔ استدلالی ابداع کرد که با نام «آرس ماگنا» یا «هنر بزرگ» (۱۳۰۵) منتشر شد. لول کوشید سامانهٔ خود را با وسیله‌ای مکانیکیِ واقعی پیاده کند: مجموعه‌ای چرخِ کاغذی که می‌شد به جایگشت‌های مختلف چرخاند.

در حدود ۱۵۰۰، لئوناردو داوینچی (۱۴۵۲–۱۵۱۹) ماشین حساب مکانیکی طراحی کرد اما نساخت؛ بازسازی‌های اخیر نشان داده‌اند طراحی او کارکردی بوده است. نخستین ماشین محاسبهٔ شناخته‌شده را حدود ۱۶۲۳ دانشمند آلمانی، ویلهلم شیکارد (۱۵۹۲–۱۶۳۵)، ساخت. بلز پاسکال (۱۶۶۲–۱۶۲۳) در ۱۶۴۲ «پاسکالین» را ساخت و نوشت که «آثاری پدید می‌آورد که به اندیشه نزدیک‌تر می‌نمایند تا همهٔ کنش‌های جانوران». گوتفرید ویلهلم لایب‌نیتس (۱۶۴۶–۱۷۱۶) وسیلهٔ مکانیکی‌ای ساخت که قرار بود بر روی مفهوم‌ها عمل کند نه اعداد، اما گسترهٔ آن محدود بود. توماس هابز (۱۵۸۸–۱۶۷۹) در کتاب «لویاتان» (۱۶۵۱) ایدهٔ ماشینِ اندیشنده را پیشنهاد کرد؛ به گفتهٔ او یک «جانور مصنوعی»: «قلب چیست جز فنری، و اعصاب چیست جز رشته‌هایی، و مفاصل چیست جز چرخ‌هایی». او همچنین پیشنهاد کرد استدلال مانند محاسبهٔ عددی است: «زیرا "استدلال" چیزی نیست جز "حساب‌کردن"، یعنی جمع و تفریق». یک چیز است که بگوییم ذهن، دست‌کم تا حدی، بر پایهٔ قواعد منطقی یا عددی عمل می‌کند و سامانه‌های فیزیکی بسازیم که برخی از آن قواعد را تقلید می‌کنند؛ چیز دیگر این است که بگوییم خودِ ذهن چنین سامانهٔ فیزیکی‌ای است. رنه دکارت (۱۵۹۶–۱۶۵۰) نخستین بحث روشن را دربارهٔ تمایز ذهن و ماده ارائه کرد. او یادآور شد تصورِ تماماً فیزیکی از ذهن، جای چندانی برای ارادهٔ آزاد باقی نمی‌گذارد. اگر ذهن تماماً تابع قوانین فیزیکی باشد، از سنگی که «تصمیم می‌گیرد» به پایین بیفتد، ارادهٔ آزاد بیشتری ندارد.

دکارت طرفدار ثنویت (Dualism) بود؛ او معتقد بود بخشی از ذهن انسان (یا همان روح) بیرون از طبیعت است و از قوانین فیزیکی معاف است. جانوران اما این کیفیت دوگانه را نداشتند و می‌شد آن‌ها را ماشین دانست. بدیل ثنویت، ماده‌گرایی (Materialism) است که بر آن است کارکرد مغز بر پایهٔ قوانین فیزیک، همان ذهن است. ارادهٔ آزاد صرفاً شیوه‌ای است که ادراک گزینه‌های موجود به انتخاب‌گر می‌نماید. اصطلاح‌های «فیزیکالیسم» و «طبیعت‌گرایی» نیز برای توصیف همین دیدگاهِ در تقابل با امر فراطبیعی به کار می‌روند.

با فرض ذهنِ فیزیکی‌ای که دانش را دست‌کاری می‌کند، مسئلهٔ بعدی تعیین منشأ دانش است. جنبش تجربه‌گرایی (Empiricism)، که با «ارغنون تازه»[۴] فرانسیس بیکن (۱۵۶۱–۱۶۲۶) آغاز شد، با این جملهٔ جان لاک (۱۶۳۲–۱۷۰۴) شناخته می‌شود: «هیچ چیز در فهم نیست که نخست در حواس نبوده باشد.» دیوید هیوم (۱۷۱۱–۱۷۷۶) در «رساله‌ای در باب طبیعت بشر» (۱۷۳۹) آنچه امروز اصل استقرا (Induction) نامیده می‌شود را پیشنهاد کرد: قواعد عمومی از راه مواجهه با هم‌راهی‌های مکرر میان اجزای آن‌ها فرا گرفته می‌شوند.

بر پایهٔ کار لودویگ ویتگنشتاین (۱۸۸۹–۱۹۵۱) و برتراند راسل (۱۸۷۰–۱۹۷۲)، «حلقهٔ وین» نامدار (زیگموند، ۲۰۱۷) — گروهی از فیلسوفان و ریاضی‌دانان که در دهه‌های ۱۹۲۰ و ۱۹۳۰ در وین گرد می‌آمدند — مکتب اثبات‌گرایی منطقی (Logical Positivism) را پروراندند. این مکتب بر آن است که همهٔ دانش را می‌توان با نظریه‌های منطقی توصیف کرد که در نهایت به گزاره‌های مشاهده (Observation Sentence) — گزاره‌هایی متناظر با ورودی‌های حسی — پیوند می‌خورند؛ بدین ترتیب اثبات‌گرایی منطقی عقل‌گرایی و تجربه‌گرایی را ترکیب می‌کند.

نظریهٔ تأیید (Confirmation Theory) رودولف کارناپ (۱۸۹۱–۱۹۷۰) و کارل همپل (۱۹۰۵–۱۹۹۷) می‌کوشید فراگیری دانش از تجربه را با کمی‌کردن درجهٔ باوری که باید به گزاره‌های منطقی نسبت داده شود تحلیل کند؛ بر پایهٔ پیوند آن گزاره‌ها با مشاهده‌هایی که آن‌ها را تأیید یا ابطال می‌کنند. کتاب کارناپ، «ساختار منطقی جهان» (۱۹۲۸)، شاید نخستین نظریهٔ ذهن به‌مثابه فرایندی محاسباتی بود.

عنصر پایانی در تصویر فلسفی ذهن، پیوند دانش و کنش است. این پرسش برای هوش مصنوعی حیاتی است، زیرا هوش افزون بر استدلال به کنش نیز نیاز دارد. وانگهی، تنها با فهم چگونگی توجیه کنش‌هاست که می‌توانیم بفهمیم چگونه عاملی بسازیم که کنش‌هایش موجّه (یا عقلایی) باشند.

ارسطو (در «در جنبش جانوران») استدلال کرد کنش‌ها با پیوندی منطقی میان هدف‌ها و آگاهی از پیامد کنش توجیه می‌شوند:

> *اما چگونه رخ می‌دهد که اندیشیدن گاه با کنش همراه است و گاه نه؛ گاه با حرکت همراه است و گاه نه؟ به نظر می‌رسد تقریباً همان چیزی رخ می‌دهد که در استدلال و استنتاج دربارهٔ اشیای تغییرناپذیر. اما در آن مورد، غایت، گزاره‌ای نظری است... در حالی که در این‌جا نتیجه‌ای که از دو مقدمه برمی‌آید، کنشی است... من به پوشش نیاز دارم؛ شنل پوشش است. به شنل نیاز دارم. آنچه نیاز دارم باید بسازم؛ شنل لازم دارم؛ باید شنل بسازم. و نتیجه، یعنی «باید شنل بسازم»، کنشی است.*

ارسطو در «اخلاق نیکوماخوسی» (کتاب سوم، بند ۳، ۱۱۱۲ب) این موضوع را بیشتر بسط می‌دهد و الگوریتمی پیشنهاد می‌کند: «ما دربارهٔ غایت‌ها تأمل نمی‌کنیم، بلکه دربارهٔ وسایل. زیرا پزشک تأمل نمی‌کند که شفا دهد یا خطیب که اقناع کند... آن‌ها غایت را مفروض می‌گیرند و می‌اندیشند چگونه و با چه وسیله‌ای به آن می‌رسند... و اگر چیز به ظاهر ممکن بود، اقدام به انجامش می‌کنیم؛ اگر با ناممکنی روبرو شویم، از جست‌وجو دست می‌کشیم؛ مثلاً اگر به پول نیاز داشته باشیم و نتوانیم به دست آوریم.»

الگوریتم ارسطو ۲۳۰۰ سال بعد توسط نیوول و سایمون در برنامهٔ «حل‌کنندهٔ عمومی مسئله» پیاده شد. امروز آن را سامانهٔ برنامه‌ریزی بازگشتیِ حریصانه می‌نامیم (فصل ۱۱ را ببینید). روش‌های مبتنی بر برنامه‌ریزی منطقی برای دستیابی به هدف‌های معین، دهه‌های نخست پژوهش نظری در هوش مصنوعی را در اختیار داشتند.

اندیشیدن صرفاً در قالب کنش‌های رساننده به هدف، اغلب مفید اما گاهی ناکاربردنی است. برای نمونه، اگر چند راه گوناگون برای رسیدن به هدف وجود داشته باشد، به راهی برای انتخاب میان آن‌ها نیاز است. مهم‌تر آنکه ممکن است نتوان با قطعیت به هدف رسید، اما با این حال باید کنشی انجام داد. پس چگونه باید تصمیم گرفت؟ آنتوان آرنو (۱۶۶۲) هنگام تحلیل ایدهٔ تصمیم عقلایی در قمار، فرمول کمّی برای بیشینه کردن ارزش پولیِ مورد انتظار پیامد ارائه کرد. بعدها، دانیل برنولی (۱۷۳۸) مفهوم عام‌تر مطلوبیت (Utility) را برای درک ارزش درونی و ذهنی یک پیامد معرفی کرد. مفهوم نوین تصمیم‌گیری عقلایی در شرایط عدم قطعیت، بیشینه کردن مطلوبیت مورد انتظار است؛ چنان که در فصل ۱۶ توضیح داده می‌شود.

در امور اخلاقی و سیاست عمومی، تصمیم‌گیرنده باید منافع افراد متعدد را در نظر بگیرد. جرمی بنتام (۱۸۲۳) و جان استوارت میل (۱۸۶۳) ایدهٔ فایده‌گرایی (Utilitarianism) را ترویج کردند: تصمیم‌گیری عقلایی بر پایهٔ بیشینه کردن مطلوبیت باید در همهٔ عرصه‌های فعالیت انسانی به کار رود، از جمله تصمیم‌های سیاست عمومی که به نمایندگی از بسیاری افراد گرفته می‌شود. فایده‌گرایی نوعی خاص از پیامدگرایی (Consequentialism) است: این ایده که درست و نادرست را پیامدهای مورد انتظار کنش تعیین می‌کنند.

در برابر، امانوئل کانت در ۱۷۸۵ نظریه‌ای اخلاق قاعده‌محور یا وظیفه‌گرا (Deontological Ethics) ارائه کرد که در آن «کار درست را انجام دادن» نه با پیامدها، بلکه با قوانین اجتماعی همگانیِ حاکم بر کنش‌های مجاز تعیین می‌شود؛ قوانینی مانند «دروغ مگو» یا «کشتار مکن». بدین ترتیب، فایده‌گرا ممکن است دروغِ مصلحت‌آمیز بگوید اگر خیرِ مورد انتظار بر شر بچربد، اما کانتی موظف است چنین نکند، زیرا دروغ ذاتاً نادرست است. میل ارزش قواعد را می‌پذیرفت، اما آن‌ها را رویه‌های تصمیمِ کارآمدی می‌دانست که از استدلالِ نخستین‌اصل دربارهٔ پیامدها گردآوری شده‌اند. بسیاری از سامانه‌های نوین هوش مصنوعی دقیقاً همین رویکرد را به کار می‌گیرند.

۱.۲.۲ ریاضیات

- قواعد صوری برای نتیجه‌گیری‌های معتبر کدام‌اند؟
- چه چیزی قابل محاسبه است؟
- چگونه با اطلاعات نامطمئن استدلال می‌کنیم؟

فیلسوفان برخی از ایده‌های بنیادین هوش مصنوعی را طرح کردند، اما جهش به سوی علمی صوری، ریاضی‌سازیِ منطق و احتمال و معرفی شاخهٔ تازه‌ای از ریاضیات — محاسبه — را می‌طلبید. ایدهٔ منطق صوری (Formal Logic) به فیلسوفان یونان باستان، هند و چین بازمی‌گردد، اما پرورش ریاضی آن واقعاً با کار جورج بول (۱۸۱۵–۱۸۶۴) آغاز شد، که جزئیات منطق گزاره‌ای یا همان منطق بولی را تدوین کرد (بول، ۱۸۴۷).

در ۱۸۷۹، گوتلوب فرگه (۱۸۴۸–۱۹۲۵) منطق بول را گسترش داد تا اشیا و روابط را در بر بگیرد و بدین ترتیب منطق مرتبهٔ اول (First-Order Logic) را پدید آورد که امروز به کار می‌رود.[5] منطق مرتبهٔ اول افزون بر نقش محوری‌اش در دورهٔ آغازین پژوهش هوش مصنوعی، انگیزهٔ کار گودل و تورینگ بود که خودِ محاسبه بر آن‌ها استوار است؛ چنان که در ادامه توضیح می‌دهیم.

نظریهٔ احتمال را می‌توان تعمیم منطق به موقعیت‌های دارای اطلاعات نامطمئن دانست — ملاحظه‌ای که برای هوش مصنوعی اهمیت فراوان دارد. جرولامو کاردانو (۱۵۰۱–۱۵۷۶) نخستین بار ایدهٔ احتمال را صورت‌بندی کرد و آن را بر حسب پیامدهای ممکن رویدادهای قماری توصیف نمود. در ۱۶۵۴، بلز پاسکال (۱۶۲۳–۱۶۶۲) در نامه‌ای به پیر فرما (۱۶۰۱–۱۶۶۵) نشان داد چگونه آیندهٔ بازی قماری ناتمام را پیش‌بینی و میان‌گین بردِ قماربازان را تعیین کنند. احتمال به‌سرعت به بخشی بی‌بدیل از علوم کمّی بدل شد و به سروکار داشتن با اندازه‌گیری‌های نامطمئن و نظریه‌های ناقص کمک کرد. ژاکوب برنولی (۱۶۵۴–۱۷۰۵، عموی دانیل)، پیر لاپلاس (۱۷۴۹–۱۸۲۷) و دیگران نظریه را پیش بردند و روش‌های آماری تازه‌ای معرفی کردند. توماس بیز (۱۷۰۲–۱۷۶۱) قاعده‌ای برای به‌روزرسانی احتمال‌ها در پرتو شواهد تازه پیشنهاد کرد؛ قاعدهٔ بیز ابزاری حیاتی برای سامانه‌های هوش مصنوعی است.

صورت‌بندیِ احتمال، همراه با در دسترس بودن داده‌ها، به پیدایش آمار به‌عنوان یک رشته انجامید. یکی از نخستین کاربردها، تحلیل داده‌های سرشماری لندن توسط جان گرانت در ۱۶۶۲ بود. رونالد فیشر نخستین آماردان مدرن دانسته می‌شود (فیشر، ۱۹۲۲). او ایده‌های احتمال، طراحی آزمایش، تحلیل داده و محاسبه را کنار هم گذاشت — در ۱۹۱۹ پافشاری کرد بدون ماشین حساب مکانیکی‌ای به نام «میلیونر» (نخستین ماشین حسابی که می‌توانست ضرب کند) نمی‌تواند کارش را انجام دهد، حتی با آنکه قیمت ماشین از حقوق سالانه‌اش بیشتر بود (راس، ۲۰۱۲).

تاریخ محاسبه به قدمت تاریخ اعداد است، اما نخستین الگوریتمِ غیربدیهی، الگوریتم اقلیدس برای محاسبهٔ بزرگ‌ترین مقسوم‌علیه مشترک دانسته می‌شود. واژهٔ الگوریتم (Algorithm) از محمد بن موسی خوارزمی، ریاضی‌دان سدهٔ نهم، آمده است؛ نوشته‌های او اعداد عربی و جبر را نیز به اروپا معرفی کردند. بول و دیگران دربارهٔ الگوریتم‌های استنتاج منطقی بحث کردند و تا پایان سدهٔ نوزدهم، تلاش‌هایی برای صوری‌سازی استدلال ریاضی عمومی به‌مثابه استنتاج منطقی در جریان بود.

کورت گودل (۱۹۰۶–۱۹۷۸) نشان داد رویه‌ای مؤثر برای اثبات هر گزارهٔ صادق در منطق مرتبهٔ اولِ فرگه و راسل وجود دارد، اما منطق مرتبهٔ اول نمی‌تواند اصل استقرای ریاضی لازم برای مشخص کردن اعداد طبیعی را در بر بگیرد. در ۱۹۳۱، گودل نشان داد محدودیت‌هایی برای استنتاج وجود دارند. قضیهٔ ناتمامیت (Incompleteness Theorem) او نشان داد در هر نظریهٔ صوریِ به قدرت حساب پئانو (نظریهٔ مقدماتی اعداد طبیعی)، ناگزیر گزاره‌های صادقی وجود دارند که درون آن نظریه اثبات‌پذیر نیستند.

این نتیجهٔ بنیادین را می‌توان چنین نیز تفسیر کرد که برخی توابع روی اعداد صحیح را نمی‌توان با الگوریتم بازنمایی کرد؛ یعنی قابل محاسبه نیستند. این امر آلن تورینگ (۱۹۱۲–۱۹۵۴) را برانگیخت دقیقاً مشخص کند کدام توابع محاسبه‌پذیرند — یعنی با رویه‌ای مؤثر قابل محاسبه‌اند. تز چرچ–تورینگ (Church–Turing Thesis) پیشنهاد می‌کند مفهوم عمومی محاسبه‌پذیری (Computability) را با توابعِ محاسبه‌شده توسط ماشین تورینگ (تورینگ، ۱۹۳۶) یکی بگیریم. تورینگ همچنین نشان داد توابعی وجود دارند که هیچ ماشین تورینگی نمی‌تواند آن‌ها را محاسبه کند. برای نمونه، هیچ ماشینی نمی‌تواند به‌طور کلی بگوید آیا برنامهٔ داده‌شده‌ای با ورودیِ داده‌شده پاسخی برمی‌گرداند یا تا ابد اجرا می‌شود.

اگرچه محاسبه‌پذیری برای فهم محاسبه مهم است، مفهوم «شدنی بودن» (Tractability) تأثیر بیشتری بر هوش مصنوعی داشته است. به بیان تقریبی، مسئله‌ای «ناشدنی» (Intractable) خوانده می‌شود که زمان لازم برای حل نمونه‌های آن با اندازهٔ نمونه‌ها به‌صورت نمایی رشد کند. تمایز میان رشد چندجمله‌ای و رشد نمایی در پیچیدگی نخستین بار در اواسط دههٔ ۱۹۶۰ مورد تأکید قرار گرفت (کابام، ۱۹۶۴؛ ادموندز، ۱۹۶۵). این تمایز مهم است، زیرا رشد نمایی بدان معناست که حتی نمونه‌های نسبتاً بزرگ را نمی‌توان در هیچ زمان معقولی حل کرد. نظریهٔ NP-کامل بودن (NP-Completeness) که کوک (۱۹۷۱) و کارپ (۱۹۷۲) پیش‌گامش بودند، مبنایی برای تحلیل شدنی بودن مسئله‌ها فراهم می‌کند: هر دستهٔ مسئله‌ای که بتوان دستهٔ مسائل NP-کامل را به آن تقلیل داد، به احتمال زیاد ناشدنی است. (هرچند اثبات نشده که مسائل NP-کامل لزوماً ناشدنی‌اند، بیشتر نظریه‌پردازان چنین باور دارند.) این نتایج در تضاد با خوش‌بینی‌ای است که مطبوعات عامه‌پسند از نخستین رایانه‌ها کردند — «ابر مغزهای الکترونیکی» که «سریع‌تر از اینشتین!» بودند. با وجود افزایش سرعت رایانه‌ها، به‌کارگیری دقیق منابع و نقصانِ ناگزیر، ویژگی سامانه‌های هوشمند خواهد بود. به بیان خام، جهان یک نمونهٔ مسئلهٔ فوق‌العاده بزرگ است!

۱.۲.۳ اقتصاد

- چگونه باید بر پایهٔ ترجیحات‌مان تصمیم بگیریم؟
- چگونه باید این کار را بکنیم در حالی که دیگران ممکن است هم‌راهی نکنند؟
- چگونه باید این کار را بکنیم در حالی که سود ممکن است در آیندهٔ دور باشد؟

علم اقتصاد در ۱۷۷۶ زاده شد، آن‌گاه که آدام اسمیت (۱۷۲۳–۱۷۹۰) «پژوهشی در ماهیت و علل ثروت ملل» را منتشر کرد. اسمیت پیشنهاد کرد اقتصاد را متشکل از عامل‌های فردی فراوانی تحلیل کنیم که هر یک به منافع خود می‌پردازد. اما اسمیت از طمع مالی به‌عنوان موضع اخلاقی دفاع نمی‌کرد: کتاب پیشین او (۱۷۵۹) یعنی «نظریهٔ احساسات اخلاقی» با این اشاره آغاز می‌شود که توجه به رفاه دیگران، مؤلفه‌ای ضروری از منافع هر فرد است.

بیشتر مردم اقتصاد را دربارهٔ پول می‌دانند و واقعاً نخستین تحلیل ریاضی تصمیم در شرایط عدم قطعیت — فرمول بیشینه‌کردن ارزش مورد انتظارِ آرنو (۱۶۶۲) — با ارزش پولی شرط‌بندی‌ها سروکار داشت. دانیل برنولی (۱۷۳۸) متوجه شد این فرمول برای مقادیر بزرگ‌تر پول، مانند سرمایه‌گذاری در سفرهای تجارت دریایی، خوب کار نمی‌کند. او به‌جایش اصلی مبتنی بر بیشینه کردن مطلوبیت مورد انتظار پیشنهاد کرد و انتخاب‌های سرمایه‌گذاری انسان را با این پیشنهاد توضیح داد که مطلوبیتِ نهاییِ مقدارِ بیشتری پول، با افزایش ثروت فرد کاهش می‌یابد.

لئون والراس (تلفظ «والراس»؛ ۱۸۳۴–۱۹۱۰) به نظریهٔ مطلوبیت بنیانی کلی‌تر بر حسب ترجیحات میان شرط‌بندی‌ها بر هر پیامدی (نه فقط پیامدهای پولی) داد. نظریه توسط رمزی (۱۹۳۱) و سپس جان فون نویمان و اسکار مورگنشترن در کتاب «نظریهٔ بازی‌ها و رفتار اقتصادی» (۱۹۴۴) بهبود یافت.

اقتصاد دیگر مطالعهٔ پول نیست؛ بلکه مطالعهٔ خواسته‌ها و ترجیحات است. نظریهٔ تصمیم (Decision Theory)، که نظریهٔ احتمال را با نظریهٔ مطلوبیت ترکیب می‌کند، چارچوبی صوری و کامل برای تصمیم‌های فردی (اقتصادی یا غیراقتصادی) در شرایط عدم قطعیت ارائه می‌دهد — یعنی در مواردی که توصیف‌های احتمالاتی به‌درستی محیط تصمیم‌گیرنده را باز می‌نمایانند. این چارچوب برای «اقتصادهای بزرگ» مناسب است که هر عامل لازم نیست به کنش‌های سایر عامل‌ها به‌عنوان افراد توجه کند. در «اقتصادهای کوچک»، وضعیت بسیار بیشتر به بازی می‌ماند: کنش یک بازیکن می‌تواند به‌شکل معناداری بر مطلوبیت بازیکنی دیگر اثر بگذارد (مثبت یا منفی). توسعهٔ نظریهٔ بازی‌ها توسط فون نویمان و مورگنشترن (نیز بنگرید به لوس و رایفا، ۱۹۵۷) شامل این نتیجهٔ شگفت‌انگیز بود که در برخی بازی‌ها، عامل عقلایی باید سیاست‌هایی در پیش گیرد که تصادفی هستند (یا دست‌کم چنین می‌نمایند). برخلاف نظریهٔ تصمیم، نظریهٔ بازی‌ها نسخه‌ای بدون‌ابهام برای انتخاب کنش‌ها تجویز نمی‌کند. در هوش مصنوعی، تصمیم‌های شامل چند عامل زیر عنوان سامانه‌های چندعاملی (Multiagent Systems) مطالعه می‌شوند (فصل ۱۸).

اقتصاددانان، با چند استثنا، به پرسش سومِ فهرست‌شده نپرداختند: چگونه وقتی سود کنش‌ها فوری نیست، بلکه حاصل توالی چند کنش است، تصمیم عقلایی بگیریم. این موضوع در رشتهٔ تحقیق در عملیات (Operations Research) دنبال شد؛ رشته‌ای که در جنگ جهانی دوم از تلاش‌های بریتانیا برای بهینه‌سازی نصب رادارها برخاست و بعدها کاربردهای غیرنظامی بی‌شماری یافت. کار ریچارد بلمن (۱۹۵۷) دسته‌ای از مسئله‌های تصمیم توالی‌دار به نام فرایندهای تصمیم مارکوف (Markov Decision Processes) را صورت‌بندی کرد که در فصل ۱۷ و نیز زیر عنوان یادگیری تقویتی در فصل ۲۲ مطالعه می‌کنیم.

کار در اقتصاد و تحقیق در عملیات بسیار به مفهوم عامل عقلایی ما کمک کرده است، اما برای سال‌ها پژوهش هوش مصنوعی در مسیرهایی کاملاً جدا رشد کرد. یکی از دلایل، پیچیدگی ظاهری تصمیم‌گیری عقلایی بود. پژوهشگر پیشگام هوش مصنوعی، هربرت سایمون (۱۹۱۶–۲۰۰۱)، در ۱۹۷۸ جایزهٔ نوبل اقتصاد را برای کارهای اولیه‌اش گرفت که نشان

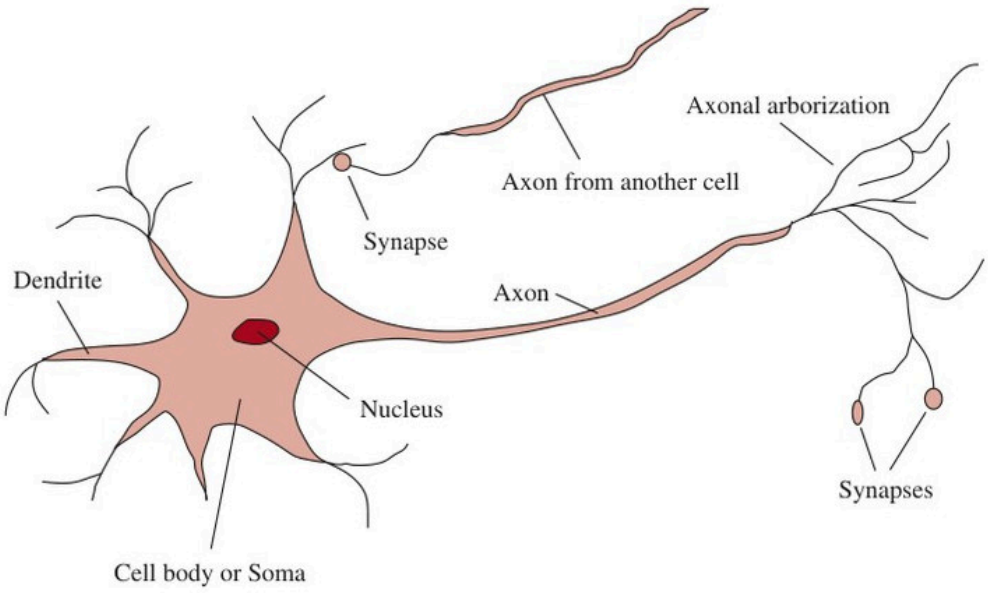
می‌داد مدل‌های مبتنی بر «رضایت‌بخشی» (Satisficing) – یعنی گرفتن تصمیم‌های «به‌اندازهٔ کافی خوب» به‌جای محاسبهٔ طاقت‌فرسای تصمیم بهینه – توصیف بهتری از رفتار واقعی انسان به دست می‌دهند (سایمون، ۱۹۴۷). از دههٔ ۱۹۹۰، علاقه به فنون نظریه‌تصمیم برای هوش مصنوعی دوباره زنده شده است.

۱.۲.۴ علوم اعصاب

- مغزها چگونه اطلاعات را پردازش می‌کنند؟

علوم اعصاب (Neuroscience) مطالعهٔ دستگاه عصبی است، به‌ویژه مغز. هرچند شیوهٔ دقیقی که مغز به‌واسطهٔ آن اندیشه را ممکن می‌کند یکی از رازهای بزرگ علم است، این واقعیت که مغز اندیشه را ممکن می‌کند هزاران سال است شناخته شده، چراکه شواهد نشان می‌دهد ضربه‌های محکم به سر می‌توانند به ناتوانی ذهنی بینجامند. همچنین از دیرباز دانسته شده که مغز انسان به‌گونه‌ای متفاوت است؛ حدود ۳۳۵ پیش از میلاد ارسطو نوشت: «در میان همهٔ جانوران، انسان به نسبت جثه‌اش بزرگ‌ترین مغز را دارد.»[6] با این حال، تا اواسط سدهٔ هجدهم مغز به‌عنوان جایگاه آگاهی به‌طور گسترده شناخته نشد. پیش از آن، نامزدهای این جایگاه شامل قلب و طحال بودند.

بررسی پل بروکا (۱۸۲۴–۱۸۸۰) دربارهٔ ناروانیِ گفتار (آفازی) در بیماران آسیب‌دیدهٔ مغزی در ۱۸۶۱، مطالعهٔ سازمان‌بندی کارکردی مغز را با شناسایی ناحیه‌ای موضعی در نیمکرهٔ چپ – که اکنون ناحیهٔ بروکا نامیده می‌شود و مسئول تولید گفتار است – آغاز کرد.[7] تا آن زمان دانسته شده بود مغز عمدتاً از یاخته‌های عصبی یا نورون‌ها (Neuron) تشکیل شده، اما تا ۱۸۷۳ بود که کامیلو گولجی (۱۸۴۳–۱۹۲۶) روش رنگ‌آمیزی‌ای ابداع کرد که مشاهدهٔ نورون‌های منفرد را ممکن می‌کرد (شکل ۱.۱ را ببینید). سانتیاگو رامون ئی کاخال (۱۸۵۲–۱۹۳۴) این روش را در مطالعات پیشگامانهٔ خود دربارهٔ سازمان‌بندی عصبی به کار برد.[8] اکنون به‌طور گسترده پذیرفته شده که کارکردهای شناختی از عملکرد الکتروشیمیایی این ساختارها ناشی می‌شوند؛ یعنی مجموعه‌ای از یاخته‌های ساده می‌تواند به اندیشه، کنش و ⚠ آگاهی بینجامد. به گفتهٔ موجز جان سرل (۱۹۹۲): مغزها ذهن‌ها را پدید می‌آورند.



شکل ۱.۱ – اجزای یک یاختهٔ عصبی (نورون): هر نورون از یک جسم یاخته‌ای (سوما) تشکیل شده که هستهٔ یاخته را در بر دارد. از جسم یاخته‌ای، رشته‌هایی به نام دندریت و یک رشتهٔ بلند به نام آکسون منشعب می‌شوند. آکسون مسافت طولانی کشیده می‌شود؛ بسیار بلندتر از آنچه مقیاس این نمودار نشان می‌دهد. به‌طور معمول آکسون ۱ سانتی‌متر طول دارد (۱۰۰ برابر قطر جسم یاخته‌ای) اما می‌تواند تا ۱ متر برسد. یک نورون با ۱۰ تا ۱۰۰,۰۰۰ نورون دیگر در محل‌های اتصالی به نام سیناپس ارتباط برقرار می‌کند. سیگنال‌ها با واکنش الکتروشیمیایی پیچیده‌ای از نورونی به نورون دیگر منتشر می‌شوند. این سیگنال‌ها فعالیت مغز را در کوتاه‌مدت کنترل می‌کنند و تغییرات بلندمدت در اتصال نورون‌ها را نیز ممکن می‌سازند. این سازوکارها بنیاد یادگیری در مغز دانسته می‌شوند. بیشتر پردازش اطلاعات در قشر مغز (کورتکس)، لایهٔ بیرونی مغز، روی می‌دهد. واحد سازمانی پایه، ستونی از بافت به قطر حدود ۰٫۵ میلی‌متر است که حدود ۲۰,۰۰۰ نورون دارد و تا تمام ژرفای کورتکس (حدود ۴ میلی‌متر در انسان) امتداد می‌یابد.

اکنون داده‌هایی دربارهٔ نگاشت میان نواحی مغز و بخش‌های بدن داریم که آن‌ها را کنترل می‌کنند یا از آن‌ها ورودی حسی می‌گیرند. این نگاشت‌ها می‌توانند در طول چند هفته به‌شکل بنیادین تغییر کنند و برخی جانوران به نظر می‌رسد چندین نگاشت دارند. وانگهی، نمی‌دانیم وقتی ناحیه‌ای آسیب می‌بیند، نواحی دیگر چگونه وظایفش را بر عهده می‌گیرند. تقریباً هیچ نظریه‌ای دربارهٔ چگونگی ذخیرهٔ یک خاطرهٔ فردی یا کارکرد فرایندهای شناختی سطح‌بالاتر وجود ندارد.

اندازه‌گیری فعالیت مغزِ سالم در ۱۹۲۹ با اختراع الکتروانسفالوگراف (EEG) توسط هانس برگر آغاز شد. توسعهٔ تصویربرداری تشدید مغناطیسی کارکردی (fMRI) (اوگاوا و همکاران، ۱۹۹۰؛ کابزا و نایبرگ، ۲۰۰۱) به عصب‌شناسان تصویرهای بی‌سابقه‌ای از فعالیت مغز می‌دهد و اندازه‌گیری‌هایی را ممکن می‌کند که به‌شکل جالبی با فرایندهای شناختی جاری متناظرند. این‌ها با پیشرفت‌های ثبت الکتریکی تک‌یاخته‌ایِ فعالیت نورون و روش‌های اپتوژنتیک (Optogenetics) (کریک، ۱۹۹۹؛ زمان‌من و همکاران، ۲۰۰۲؛ هان و بویدن، ۲۰۰۷) تکمیل می‌شوند که هم اندازه‌گیری و هم کنترل نورون‌های منفردِ اصلاح‌شده برای حساسیت به نور را ممکن می‌کنند.

توسعهٔ واسط‌های مغز–ماشین (Brain–Machine Interface) (لبدف و نیکوللیس، ۲۰۰۶) برای حس کردن و کنترل حرکتی، نه فقط نوید بازگرداندن کارکرد به افراد ناتوان را می‌دهد، بلکه بر جنبه‌های فراوان سامانه‌های عصبی نیز پرتو می‌افکند. یافتهٔ شگفت‌آور این کارها آن است که مغز می‌تواند خود را با یک دستگاه بیرونی با موفقیت هم‌اهنگ کند و عملاً آن را مانند اندام یا عضو حسی دیگری بپندارد.

مغز و رایانه‌های دیجیتال ویژگی‌هایی تا حدی متفاوت دارند. شکل ۱.۲ نشان می‌دهد زمان چرخهٔ رایانه‌ها یک میلیون بار از مغز کوتاه‌تر است. مغز اما با ذخیره‌سازی و اتصالات بسیار بیشتر از حتی رایانهٔ شخصی رده‌بالا این کاستی را جبران می‌کند، هرچند بزرگ‌ترین ابررایانه‌ها در برخی معیارها به مغز می‌رسند. آینده‌پژوهان بسیار از این اعداد می‌گویند و به «تکینگی» (Singularity) نزدیک‌شدنی اشاره می‌کنند که رایانه‌ها در آن به عملکردی فراتر از انسان می‌رسند (وینگه، ۱۹۹۳؛ کورزویل، ۲۰۰۵؛ داکترو و استروس، ۲۰۱۲) و سپس خود را با سرعت بیشتری بهبود می‌دهند. اما مقایسهٔ اعداد خام چندان پرمعنا نیست. حتی با رایانه‌ای با ظرفیت عملاً نامحدود، باز هم به پیشرفت‌های مفهومی در فهم هوش نیاز داریم (فصل ۲۸ را ببینید). به بیان خام، بدون نظریهٔ درست، ماشین‌های سریع‌تر فقط پاسخِ غلط را سریع‌تر به شما می‌دهند.

	Supercomputer	Personal Computer	Human Brain
Computational units	10^6 GPUs + CPUs	8 CPU cores	10^6 columns
	10^{15} transistors	10^{10} transistors	10^{11} neurons
Storage units	10^{16} bytes RAM	10^{10} bytes RAM	10^{11} neurons
	10^{17} bytes disk	10^{12} bytes disk	10^{14} synapses
Cycle time	10^{-9} sec	10^{-9} sec	10^{-3} sec
Operations/sec	10^{18}	10^{10}	10^{17}

شکل ۱.۲ – مقایسهٔ خام میان یک ابررایانهٔ پیشرو (سامیت)، یک رایانهٔ شخصی معمولیِ ۲۰۱۹ و مغز انسان:

قدرت مغز انسان در هزاران سال چندان تغییر نکرده، در حالی که ابررایانه‌ها از مگافلاپ در دههٔ ۱۹۶۰ به گیگافلاپ در دههٔ ۱۹۸۰، ترافلاپ در دههٔ ۱۹۹۰، پتافلاپ در ۲۰۰۸ و اگزافلاپ در ۲۰۱۸ بهبود یافته‌اند (۱ اگزافلاپ = 10^{18} عملیات ممیز شناور در ثانیه). (شکل ۱.۲؛ مأخذ مقایسه: فلدمن، ۲۰۱۷.)

۱.۲.۵ روان‌شناسی

- انسان‌ها و جانوران چگونه می‌اندیشند و عمل می‌کنند؟

سرچشمه‌های روان‌شناسی علمی را معمولاً به کار فیزیک‌دان آلمانی، هرمان فون هلمهولتس (۱۸۲۱–۱۸۹۴)، و شاگردش ویلهلم وونت (۱۸۳۲–۱۹۲۰) بازمی‌گردانند. هلمهولتس روش علمی را در مطالعهٔ بینایی انسان به کار برد و «کتاب راهنمای اپتیک فیزیولوژیک» او «مهم‌ترین رسالهٔ واحد دربارهٔ فیزیک و فیزیولوژی بینایی انسان» توصیف شده است (نالوا، ۱۹۹۳، ص ۱۵). وونت در ۱۸۷۹ نخستین آزمایشگاه روان‌شناسی تجربی را در دانشگاه لایپزیگ گشود. وونت بر آزمایش‌های دقیقاً کنترل‌شده پافشاری می‌کرد؛ در آن‌ها کارکنانش تکلیفی ادراکی یا تداعی‌گرانه انجام می‌دادند و هم‌زمان به فرایندهای فکری خود درون‌نگری می‌کردند. این کنترل‌های دقیق سهم بزرگی در علمی کردن روان‌شناسی داشتند، اما ماهیت ذهنی داده‌ها بعید می‌ساخت آزمایشگران نظریه‌های خودشان را ابطال کنند.

از سوی دیگر، زیست‌شناسانی که رفتار جانوران را مطالعه می‌کردند، دادهٔ درون‌نگری نداشتند و روش‌شناسی عینی‌ای پروراندند؛ چنان که اچ. اس. جنینگز (۱۹۰۶) در اثر تأثیرگذارش «رفتار جانداران پست» توصیف کرده است. با به‌کارگیری این دیدگاه دربارهٔ انسان، جنبش رفتارگرایی (Behaviorism) به رهبری جان واتسون (۱۸۷۸–۱۹۵۸) هر نظریه‌ای را که شامل فرایندهای ذهنی بود رد کرد، به این دلیل که درون‌نگری نمی‌تواند شواهد معتبر به دست دهد. رفتارگرایان بر مطالعهٔ صرفاً معیارهای عینیِ ادراک‌های حسی (محرک) داده‌شده به جانور و کنش‌های (پاسخ‌های) برآمده از آن پافشاری می‌کردند. رفتارگرایی دربارهٔ موش‌ها و کبوترها بسیار آموخت، اما در فهم انسان توفیق کمتری داشت.

روان‌شناسی شناختی (Cognitive Psychology) که مغز را دستگاه پردازش اطلاعات می‌بیند، دست‌کم به آثار ویلیام جیمز (۱۸۴۲–۱۹۱۰) بازمی‌گردد. هلمهولتس نیز پافشاری می‌کرد ادراک نوعی استنتاج منطقی ناخودآگاه است. دیدگاه شناختی در آمریکا تا حد زیادی در سایهٔ رفتارگرایی ماند، اما در «واحد روان‌شناسی کاربردی» کمبریج به رهبری فردریک بارتلت (۱۸۸۶–۱۹۶۹)، مدل‌سازی شناختی توانست شکوفا شود. کتاب «ماهیت تبیین» نوشتهٔ کنت کریک (۱۹۴۳) — شاگرد و جانشین بارتلت — مشروعیت اصطلاح‌های «ذهنی» مانند باور و هدف را با قدرت بازتثبیت کرد و استدلال آورد این اصطلاح‌ها به همان اندازه علمی‌اند که مثلاً به‌کارگیری فشار و دما برای سخن گفتن از گازها؛ هرچند گازها از مولکول‌هایی ساخته شده‌اند که نه فشار دارند و نه دما.

کریک سه گام کلیدی عاملِ مبتنی بر دانش را مشخص کرد: (۱) محرک باید به بازنمایی درونی ترجمه شود؛ (۲) بازنمایی با فرایندهای شناختی دست‌کاری می‌شود تا بازنمایی‌های درونی تازه‌ای به دست آید؛ و (۳) این‌ها به نوبهٔ خود به کنش بازترجمه می‌شوند. او روشن توضیح داد چرا این طراحی خوبی برای عامل است:

> *اگر جاندار «مدل کوچک‌مقیاسی» از واقعیت بیرونی و کنش‌های ممکن خود در سر داشته باشد، می‌تواند بدیل‌های گوناگون را آزمایش کند، بهترین آن‌ها را برگزیند، پیش از آنکه موقعیت‌ها پیش آیند به آن‌ها واکنش نشان دهد، از دانش رویدادهای گذشته در سروکار داشتن با حال و آینده بهره برد، و به‌هرروی به‌شکلی بسیار کامل‌تر، ایمن‌تر و شایسته‌تر به موقعیت‌های اضطراری پیش‌آمده واکنش نشان دهد. (کریک، ۱۹۴۳)*

پس از مرگ کریک در سانحهٔ دوچرخه‌سواری در ۱۹۴۵، کار او توسط دونالد برودبنت ادامه یافت؛ کتاب او «ادراک و ارتباط» (۱۹۵۸) از نخستین آثاری بود که پدیده‌های روان‌شناختی را همچون پردازش اطلاعات مدل کرد. در همین حال، در آمریکا، توسعهٔ مدل‌سازی رایانه‌ای به پیدایش رشتهٔ علوم شناختی انجامید. این رشته را می‌توان از کارگاهی در سپتامبر ۱۹۵۶ در مؤسسهٔ فناوری ماساچوست (MIT) آغازشده دانست — تنها دو ماه پس از کنفرانسی که هوش مصنوعی در آن «زاده شد». در آن کارگاه، جورج میلر «عدد جادویی هفت» را ارائه کرد، نوآم چامسکی «سه مدل زبان» را، و آلن نیوول و هربرت سایمون «ماشین نظریهٔ منطق» را. این سه مقالهٔ تأثیرگذار نشان دادند مدل‌های رایانه‌ای چگونه می‌توانند به‌ترتیب به روان‌شناسی حافظه، زبان و اندیشیدن منطقی بپردازند. امروزه این دیدگاه رایج (هرچند به‌هیچ‌وجه همگانی) میان روان‌شناسان است که «نظریهٔ شناختی باید مانند برنامهٔ رایانه‌ای باشد» (اندرسون، ۱۹۸۰)؛ یعنی باید کارکرد یک کارکردِ شناختی را بر حسب پردازش اطلاعات توصیف کند.

در این مرور، رشتهٔ تعامل انسان–رایانه (HCI) را زیر چتر روان‌شناسی می‌شماریم. داگ انگلبارت، یکی از پیش‌گامان این رشته، از ایدهٔ «افزایش هوش» (Intelligence Augmentation) — یعنی IA به‌جای AI — دفاع می‌کرد. او باور داشت رایانه‌ها باید توانایی‌های انسان را افزایش دهند، نه آنکه وظایف انسانی را خودکار کنند. در ۱۹۶۸، «مادرِ همهٔ دموها»ی انگلبارت برای نخستین بار ماوس رایانه، سامانهٔ پنجره‌ها، ابرمتن و ویدئوکنفرانس را نمایش داد — همه در تلاش برای نشان دادن آنچه کارگران دانشیِ انسانی با کمی افزایش هوش می‌توانند به‌طور جمعی به دست آورند. امروزه بیشتر IA و AI را دو روی یک سکه می‌بینیم: اولی بر کنترل انسانی و دومی بر رفتار هوشمندِ ماشین تأکید دارد. برای آنکه ماشین‌ها برای انسان سودمند باشند، به هر دو نیاز است.

۱.۲.۶ مهندسی رایانه

- چگونه می‌توان رایانه‌ای کارآمد ساخت؟

رایانهٔ دیجیتال الکترونیکی مدرن را دانشمندان سه کشورِ درگیر در جنگ جهانی دوم، به‌طور مستقل و تقریباً هم‌زمان، اختراع کردند. نخستین رایانهٔ عملیاتی، «هیت رابینسون» الکترومکانیکی بود که در ۱۹۴۳ توسط تیم آلن تورینگ برای هدفی واحد ساخته شد: رمزگشایی پیام‌های آلمانی.[9] در همان سال، همان گروه «کولوسوس» را ساختند؛ ماشینی قدرتمند و همه‌منظوره بر پایهٔ لامپ خلأ.[10] نخستین رایانهٔ برنامه‌پذیر عملیاتی، زد-۳، اختراع کنراد زوزه در آلمان در ۱۹۴۱ بود. زوزه همچنین اعداد ممیز شناور و نخستین زبان برنامه‌نویسی سطح‌بالا، پلانکالکول، را اختراع کرد. نخستین رایانهٔ الکترونیکی، ای‌بی‌سی، توسط جان آتاناسوف و دانشجویش کلیفورد بری میان ۱۹۴۰ و ۱۹۴۲ در دانشگاه ایالتی آیووا ساخته شد. پژوهش آتاناسوف حمایت و شناسایی چندانی نیافت؛ این انیاک بود — که در پروژهٔ نظامی محرمانه‌ای در دانشگاه پنسیلوانیا توسط تیمی شامل جان ماچلی و جی. پرسپر اکرت ساخته شد — که تأثیرگذارترین پیش‌آهنگ رایانه‌های مدرن از آب درآمد.

از آن زمان، هر نسل از سخت‌افزار رایانه افزایش سرعت و ظرفیت و کاهش قیمت به ارمغان آورده است — روندی که در قانون مور (Moore's Law) خلاصه می‌شود. عملکرد تقریباً هر ۱۸ ماه یک بار تا حدود ۲۰۰۵ دو برابر می‌شد؛ آن‌گاه مشکلات اتلاف توان، سازندگان را بر آن داشت به‌جای سرعت کلاک، شمار هسته‌های پردازنده را چند برابر کنند. انتظار کنونی آن است که افزایش کارکرد آینده از موازی‌سازی عظیم حاصل شود — هم‌گراییِ عجیبی با ویژگی‌های مغز. همچنین طراحی‌های سخت‌افزاری تازه‌ای می‌بینیم که بر این ایده استوارند که در سروکار داشتن با جهانِ نامطمئن، به ۶۴ بیت دقت در اعداد نیاز نداریم؛ ۱۶ بیت (مانند قالب bfloat16) یا حتی ۸ بیت کافی است و پردازش سریع‌تر را ممکن می‌کند. تازه‌تازه سخت‌افزارهای تنظیم‌شده برای کاربردهای هوش مصنوعی را می‌بینیم؛ مانند واحد پردازش گرافیکی (GPU)، واحد پردازش تنسور (TPU) و موتور مقیاس‌ویفر (WSE). از دههٔ ۱۹۶۰ تا حدود ۲۰۱۲، مقدار توان محاسباتی به‌کاررفته برای آموزش برنامه‌های پیشرو یادگیری ماشین از قانون مور پیروی می‌کرد. از ۲۰۱۲ اوضاع تغییر کرد: از ۲۰۱۲ تا ۲۰۱۸ افزایش ۳۰۰,۰۰۰ برابری رخ داد که به معنای دوبرابر شدن تقریباً هر ۱۰۰ روز است (آمودی و هرناندز، ۲۰۱۸). مدل یادگیری ماشینی که در ۲۰۱۴ یک روز کامل برای آموزش نیاز داشت، در ۲۰۱۸ تنها دو دقیقه می‌برد (ییِنگ و همکاران، ۲۰۱۸). هرچند هنوز عملی نشده، محاسبات کوانتومی (Quantum Computing) وعدهٔ شتاب‌بخشی بسیار بیشتر برای برخی زیرروده‌های مهم الگوریتم‌های هوش مصنوعی را دارد.

البته پیش از رایانهٔ الکترونیکی نیز دستگاه‌های محاسبه وجود داشتند. کهن‌ترین ماشین‌های خودکار، متعلق به سدهٔ هفدهم، در صفحهٔ ۶ بحث شدند. نخستین ماشین برنامه‌پذیر، دستگاه بافندگی‌ای بود که ژوزف ماری ژاکار (۱۷۵۲–۱۸۳۴) در ۱۸۰۵ ساخت و با کارت‌های پانچ‌شده دستورهای نقشِ بافت را ذخیره می‌کرد.

در اواسط سدهٔ نوزدهم، چارلز بابیج (۱۷۹۲–۱۸۷۱) دو ماشین محاسبه طراحی کرد که هیچ‌کدام را کامل نکرد. «موتور تفاضلی» برای محاسبهٔ جدول‌های ریاضیِ پروژه‌های مهندسی و علمی در نظر گرفته شده بود و سرانجام در ۱۹۹۱ ساخته شد و نشان داده شد کار می‌کند (سوید، ۲۰۰۰). «موتور تحلیلی» بابیج بسیار جاه‌طلبانه‌تر بود: حافظهٔ آدرس‌پذیر، برنامه‌های ذخیره‌شده بر پایهٔ کارت‌های پانچ‌شدهٔ ژاکار و پرش‌های شرطی داشت. این نخستین ماشینِ دارای قابلیت محاسبهٔ همگانی بود.

آدا لاولیس، همکار بابیج و دختر شاعر لرد بایرون، پتانسیل آن را دریافت و آن را «ماشینی اندیشنده یا... استدلال‌گر» توصیف کرد؛ ماشینی که می‌تواند دربارهٔ «همهٔ موضوعات عالم» استدلال کند (لاولیس، ۱۸۴۳). او همچنین چرخه‌های تبلیغاتی هوش مصنوعی را پیش‌بینی کرد و نوشت: «شایسته است در برابر امکانِ بروز تصورات اغراق‌آمیز دربارهٔ قدرت موتور تحلیلی احتیاط کنیم.» متأسفانه ماشین‌های بابیج و ایده‌های لاولیس تا حد زیادی فراموش شدند. هوش مصنوعی همچنین به سمت نرم‌افزاری علوم رایانه مدیون است؛ سمتی که سامانه‌های عامل، زبان‌های برنامه‌نویسی و ابزارهای لازم برای نوشتن برنامه‌های مدرن (و مقاله‌هایی دربارهٔ آن‌ها) را فراهم کرده است. اما این یکی از حوزه‌هایی است که دین بازپرداخته شده: کار در هوش مصنوعی پیش‌گام ایده‌های فراوانی بوده که به جریان اصلی علوم رایانه بازگشته‌اند؛ از جمله اشتراک زمانی، مفسرهای تعاملی، رایانه‌های شخصی با پنجره و ماوس، محیط‌های توسعهٔ سریع، نوع دادهٔ فهرست پیوندی، مدیریت خودکار حافظه و مفاهیم کلیدی برنامه‌نویسی نمادین، تابعی، اعلانی و شئ‌گرا.

۱.۲.۷ نظریهٔ کنترل و سایبرنتیک

- مصنوعات چگونه می‌توانند زیر کنترل خود عمل کنند؟

کتزیبیوس اسکندرانی (حدود ۲۵۰ پیش از میلاد) نخستین ماشین خودکنترل را ساخت: ساعت آبی با تنظیم‌گری که نرخ جریان ثابتی حفظ می‌کرد. این اختراع تعریفِ آنچه مصنوع می‌توانست انجام دهد را دگرگون کرد. پیش از آن، فقط موجودات زنده می‌توانستند رفتارشان را در پاسخ به تغییرات محیط اصلاح کنند. نمونه‌های دیگر سامانه‌های کنترل بازخوردی خودتنظیم شامل تنظیم‌گر موتور بخار ساختهٔ جیمز وات (۱۷۳۶–۱۸۱۹) و ترموستات اختراعی کورنلیس دربل (۱۵۷۳–۱۶۳۳) – که زیردریایی را هم اختراع کرد – است. جیمز کلرک ماکسول (۱۸۶۸) نظریهٔ ریاضی سامانه‌های کنترل را آغاز کرد.

چهرهٔ محوری توسعهٔ پس از جنگ نظریهٔ کنترل، نوربرت وینر (۱۸۹۴–۱۹۶۴) بود. وینر ریاضی‌دان نابغه‌ای بود که با برتراند راسل و دیگران کار کرد تا آنکه به سامانه‌های کنترل زیستی و مکانیکی و پیوندشان با شناخت علاقه‌مند شد. وینر و همکارانش آرتورو روزنبلوت و جولیان بیگلو، مانند کریک (که سامانه‌های کنترل را نیز به‌عنوان مدل روان‌شناختی به کار می‌برد)، ارتدکس رفتارگرایانه را به چالش کشیدند (روزنبلوت و همکاران، ۱۹۴۳). آن‌ها رفتار هدفمند را برخاسته از سازوکار تنظیمی می‌دیدند که می‌کوشد «خطا» – تفاوت میان حالت کنونی و حالت هدف – را کمینه کند. در اواخر

دهۀ ۱۹۴۰، وینر همراه با وارن مک‌کالوک، والتر پیتس و جان فون نویمان، سلسله‌کنفرانس‌های تأثیرگذاری ترتیب داد که مدل‌های ریاضی و محاسباتی تازۀ شناخت را کاوش می‌کردند. کتاب وینر، «سایبرنتیک» (Cybernetics؛ ۱۹۴۸)، پرفروش شد و عموم را با امکان ماشین‌های هوشمند مصنوعی آشنا کرد.

در همین حال در بریتانیا، دبلیو. راس اشبی ایده‌های مشابهی را پیش برد (اشبی، ۱۹۴۰). اشبی، آلن تورینگ، گری والتر و دیگران باشگاه «نسبت» (Ratio Club) را برای «کسانی که پیش از انتشار کتاب وینر، ایده‌های وینر را داشتند» تشکیل دادند. «طراحی برای یک مغز» اشبی (۱۹۴۸، ۱۹۵۲) ایدۀ او را بسط داد که هوش را می‌توان با دستگاه‌های هموستازی (Homeostatic) حاوی حلقه‌های بازخوردیِ مناسب برای دستیابی به رفتار سازگارانۀ پایدار آفرید.

نظریۀ کنترل مدرن، به‌ویژه شاخه‌ای که کنترل بهینۀ تصادفی (Stochastic Optimal Control) نامیده می‌شود، هدفش طراحی سامانه‌هایی است که تابع هزینه‌ای را در طول زمان بیشینه می‌کنند. این تقریباً با الگوی استاندارد هوش مصنوعی هم‌خوانی دارد: طراحی سامانه‌هایی که رفتاری بهینه دارند. پس چرا هوش مصنوعی و نظریۀ کنترل با وجود پیوندهای نزدیک بنیان‌گذارانشان دو رشتۀ جدا هستند؟ پاسخ در پیوند نزدیک میان فنون ریاضیِ آشنا برای شرکت‌کنندگان و مجموعۀ مسئله‌های متناظرِ هر جهان‌بینی نهفته است. حساب دیفرانسیل و جبر ماتریسی — ابزارهای نظریۀ کنترل — برای سامانه‌هایی مناسب‌اند که با مجموعه‌های ثابت متغیرهای پیوسته توصیف می‌شوند؛ حال آنکه هوش مصنوعی تا حدی به‌عنوان راهی برای گریز از همین محدودیت‌های ادراک‌شده بنیان نهاده شد. ابزارهای استنتاج منطقی و محاسبه به پژوهشگران هوش مصنوعی امکان داد مسئله‌هایی مانند زبان، بینایی و برنامه‌ریزی نمادین را در نظر بگیرند که کاملاً بیرون از حیطۀ دید نظریه‌پرداز کنترل بود.

۱.۲.۸ زبان‌شناسی

- زبان چگونه با اندیشه پیوند دارد؟

در ۱۹۵۷، ب. اف. اسکینر کتاب «رفتار کلامی» را منتشر کرد؛ شرحی جامع و مفصل از رویکرد رفتارگرایانه به یادگیری زبان، نوشتۀ برجسته‌ترین کارشناس این حوزه. اما به‌طرز عجیبی، نقدی بر این کتاب به همان اندازۀ خودِ کتاب مشهور شد و تقریباً علاقه به رفتارگرایی را از میان برد. نویسندۀ آن نقد، زبان‌شناس نوآم چامسکی بود که تازه کتابی دربارۀ نظریۀ خودش، «ساخت‌های نحوی»، منتشر کرده بود. چامسکی اشاره کرد نظریۀ رفتارگرایانه به مفهوم خلاقیت در زبان نمی‌پردازد — توضیح نمی‌داد کودکان چگونه جمله‌هایی را که هرگز نشنیده‌اند می‌فهمند و می‌سازند. نظریۀ چامسکی — که بر مدل‌های نحوی بازگشته به زبان‌شناس هندی، پانینی (حدود ۳۵۰ پیش از میلاد) استوار بود — می‌توانست این را توضیح دهد و برخلاف نظریه‌های پیشین به اندازۀ کافی صوری بود که در اصل قابل برنامه‌نویسی باشد.

پس زبان‌شناسی مدرن و هوش مصنوعی تقریباً هم‌زمان «زاده» شدند و با هم بزرگ شدند و در رشتۀ میان‌رشته‌ای زبان‌شناسی رایانه‌ای (Computational Linguistics) یا پردازش زبان طبیعی به هم رسیدند. مسئلۀ فهم زبان بسیار پیچیده‌تر از آن بود که در ۱۹۵۷ می‌نمود. فهم زبان، فهم موضوع و بافت را می‌طلبد، نه فقط فهم ساختار جمله‌ها را. این شاید بدیهی بنماید، اما تا دهۀ ۱۹۶۰ چندان شناخته نشده بود. بخش بزرگی از کارهای اولیه در بازنمایی دانش (مطالعۀ چگونگی قرار دادن دانش در قالبی که رایانه بتواند با آن استدلال کند) به زبان گره خورده و از پژوهش زبان‌شناسی تغذیه می‌شد، که آن نیز به نوبۀ خود به دهه‌ها کار در تحلیل فلسفی زبان پیوند داشت.

۱.۳ تاریخچهٔ هوش مصنوعی

راه سریع برای جمع‌بندی نقطه‌عطف‌های تاریخ هوش مصنوعی، فهرست کردن برندگان جایزهٔ تورینگ است: ماروین مینسکی (۱۹۶۹) و جان مک‌کارتی (۱۹۷۱) برای تعریف بنیان‌های این رشته بر پایهٔ بازنمایی و استدلال؛ اد فایگنباوم و راج ردی (۱۹۹۴) برای توسعهٔ سامانه‌های خبره که دانش انسانی را برای حل مسائل جهان واقعی رمزگذاری می‌کنند؛ جودیا پرل (۲۰۱۱) برای توسعهٔ فنون استدلال احتمالاتی که با عدم قطعیت به‌شیوه‌ای اصولی برخورد می‌کنند؛ و سرانجام یوشوا بنجیو، جفری هینتون و یان لیکان (۲۰۱۹) برای ساختن «یادگیری عمیق» (شبکه‌های عصبی چندلایه) به جزء حیاتی محاسبات مدرن.

بقیهٔ این بخش به جزئیات بیشتری دربارهٔ هر مرحله از تاریخ هوش مصنوعی می‌پردازد.

۱.۳.۱ آغاز هوش مصنوعی (۱۹۴۳–۱۹۵۶)

نخستین کار که امروزه عموماً به‌عنوان هوش مصنوعی شناخته می‌شود، کار وارن مک‌کالوک و والتر پیتس (۱۹۴۳) بود. آنان با الهام از کار مدل‌سازی ریاضیِ نیکلاس راشفسکی (۱۹۳۶، ۱۹۳۸) – استاد راهنمای پیتس – از سه منبع بهره گرفتند: دانش فیزیولوژی پایه و کارکرد نورون‌های مغز؛ تحلیل صوری منطق گزاره‌ای به‌دست راسل و وایتهد؛ و نظریهٔ محاسبهٔ تورینگ. آنان مدلی از نورون‌های مصنوعی پیشنهاد کردند که در آن هر نورون «روشن» یا «خاموش» توصیف می‌شود و روشن‌شدن در پاسخ به تحریکِ تعداد کافی از نورون‌های همسایه رخ می‌دهد. حالت یک نورون «واقعاً معادل گزاره‌ای دانسته می‌شد که محرک کافیِ آن را پیشنهاد می‌کند». آنان برای نمونه نشان دادند هر تابع محاسبه‌پذیری را می‌توان با شبکهٔ مناسبی از نورون‌های متصل محاسبه کرد و همهٔ عوامل ربط منطقی (AND، OR، NOT و...) را می‌توان با ساختارهای شبکه‌ای ساده پیاده کرد. مک‌کالوک و پیتس همچنین پیشنهاد کردند شبکه‌هایِ به‌درستی تعریف‌شده می‌توانند یاد بگیرند. دونالد هب (۱۹۴۹) قاعدهٔ به‌روزرسانی ساده‌ای برای اصلاح شدت پیوند میان نورون‌ها نشان داد. قاعدهٔ او، که اکنون یادگیری هب (Hebbian Learning) نامیده می‌شود، تا امروز مدلی تأثیرگذار مانده است.

دو دانشجوی کارشناسی در هاروارد، ماروین مینسکی (۱۹۲۷–۲۰۱۶) و دین ادموندز، در ۱۹۵۰ نخستین رایانهٔ شبکهٔ عصبی را ساختند. این دستگاه که SNARC نام داشت، از ۳۰۰۰ لامپ خلأ و یک خلبان خودکار مازادِ بمب‌افکن ب-۲۴ برای شبیه‌سازی شبکه‌ای از ۴۰ نورون استفاده می‌کرد. بعدها در پرینستون، مینسکی محاسبهٔ همگانی در شبکه‌های عصبی را مطالعه کرد. کمیتهٔ دکترای او دربارهٔ اینکه آیا چنین کاری را باید ریاضیات دانست تردید داشت، اما بنابر گزارش‌ها فون نویمان گفت: «اگر الان نیست، روزی خواهد بود.» نمونه‌های دیگری از کارهای اولیهٔ قابل توصیف به‌عنوان هوش مصنوعی وجود داشت، از جمله دو برنامهٔ بازی چکرز که در ۱۹۵۲ به‌طور مستقل توسط کریستوفر استراچی در دانشگاه منچستر و آرتور ساموئل در آی‌بی‌ام ساخته شدند. اما چشم‌انداز آلن تورینگ از همه تأثیرگذارتر بود. او از ۱۹۴۷ در انجمن ریاضی لندن سخنرانی‌هایی دربارهٔ این موضوع ایراد کرد و در مقالهٔ ۱۹۵۰ خود با عنوان «ماشین‌های محاسبه و هوش»، برنامه‌ای قانع‌کننده ترسیم کرد. او در آن مقاله آزمون تورینگ، یادگیری ماشین، الگوریتم‌های ژنتیک و یادگیری

تقویتی را معرفی کرد. او به بسیاری از اعتراض‌هایی که به امکان هوش مصنوعی می‌شد پاسخ داد؛ چنان‌که در فصل ۲۷ توصیف شده است. او همچنین پیشنهاد کرد ساختن هوش مصنوعی هم‌سطح انسان با توسعهٔ الگوریتم‌های یادگیری و سپس آموزش‌دادن ماشین آسان‌تر خواهد بود تا برنامه‌نویسی دستیِ هوش. در سخنرانی‌های بعدی هشدار داد دستیابی به این هدف شاید بهترین چیز برای نسل بشر نباشد.

در ۱۹۵۵، جان مک‌کارتی از کالج دارتموث، مینسکی، کلود شانون و ناتانیل راچستر را قانع کرد در گردآوردن پژوهشگران آمریکاییِ علاقه‌مند به نظریهٔ اتوماتا، شبکه‌های عصبی و مطالعهٔ هوش یاری‌اش کنند. آنان کارگاهی دوماهه در تابستان ۱۹۵۶ در دارتموث ترتیب دادند که در مجموع ۱۰ شرکت‌کننده داشت، از جمله آلن نیوول و هربرت سایمون از مؤسسهٔ فناوری کارنگی،[11] ترنچارد مور از پرینستون، آرتور ساموئل از آی‌بی‌ام، و ری سولومونوف و الیور سلفرج از ام‌آی‌تی. در متن پیشنهاد آمده است:[12]

> *پیشنهاد می‌کنیم مطالعه‌ای دوماهه با حضور ده نفر دربارهٔ هوش مصنوعی در تابستان ۱۹۵۶ در کالج دارتموث در هانوفر، نیوهمپشایر انجام شود. این مطالعه بر پایهٔ این حدس پیش خواهد رفت که هر جنبه از یادگیری یا هر ویژگی دیگر هوش را در اصل می‌توان چنان دقیق توصیف کرد که بتوان ماشینی ساخت که آن را شبیه‌سازی کند. کوشش خواهد شد دریابیم چگونه می‌توان ماشین‌ها را واداشت از زبان استفاده کنند، انتزاع‌ها و مفهوم‌ها را شکل دهند، گونه‌هایی از مسائل را حل کنند که اکنون ویژهٔ انسان‌هاست، و خود را بهبود دهند. می‌اندیشیم اگر گروهی گزیده از دانشمندان یک تابستان با هم کار کنند، پیشرفت چشمگیری در یک یا چند تا از این مسائل حاصل می‌شود.*

با وجود این پیش‌بینی خوش‌بینانه، کارگاه دارتموث به هیچ جهش بزرگی نینجامید. نیوول و سایمون شاید پخته‌ترین کار را ارائه کردند: سامانه‌ای اثبات قضیهٔ ریاضی به نام «نظریه‌پرداز منطقی» (Logic Theorist، یا LT). سایمون ادعا کرد: «برنامهٔ رایانه‌ای اختراع کرده‌ایم که قادر به اندیشیدنِ غیرعددی است و بدین ترتیب مسئلهٔ کهن ذهن–بدن را حل کرده‌ایم.»[13] اندکی پس از کارگاه، برنامه توانست بیشتر قضیه‌های فصل ۲ «اصول ریاضیات» راسل و وایتهد را اثبات کند. بنابر گزارش‌ها، راسل از شنیدن اینکه LT برای یک قضیه برهانی کوتاه‌تر از برهان موجود در اصول ریاضیات یافته، بسیار خشنود شد. اما سردبیران «ژورنال منطق نمادین» چندان تحت تأثیر قرار نگرفتند؛ آنان مقالهٔ مشترکی از نیوول، سایمون و نظریه‌پرداز منطقی را رد کردند.

۱.۳.۲ شور و شوق آغازین، انتظارهای بزرگ (۱۹۵۲–۱۹۶۹)

بخش عظیمی از نهاد روشنفکری دههٔ ۱۹۵۰ ترجیح می‌داد باور کند «ماشین هرگز نمی‌تواند X را انجام دهد». (برای فهرست بلندبالایی از Xها که تورینگ گرد آورده، فصل ۲۷ را ببینید.) پژوهشگران هوش مصنوعی نیز طبیعتاً با نشان‌دادنِ یکی‌پس‌ازدیگری X پاسخ دادند. آنان به‌ویژه بر تکلیف‌هایی تمرکز کردند که نشانهٔ هوش در انسان دانسته می‌شدند: بازی‌ها، معماها، ریاضیات و آزمون‌های هوش. جان مک‌کارتی این دوره را دورهٔ «ببین مامان، دست‌ها آزاد!» نامید.

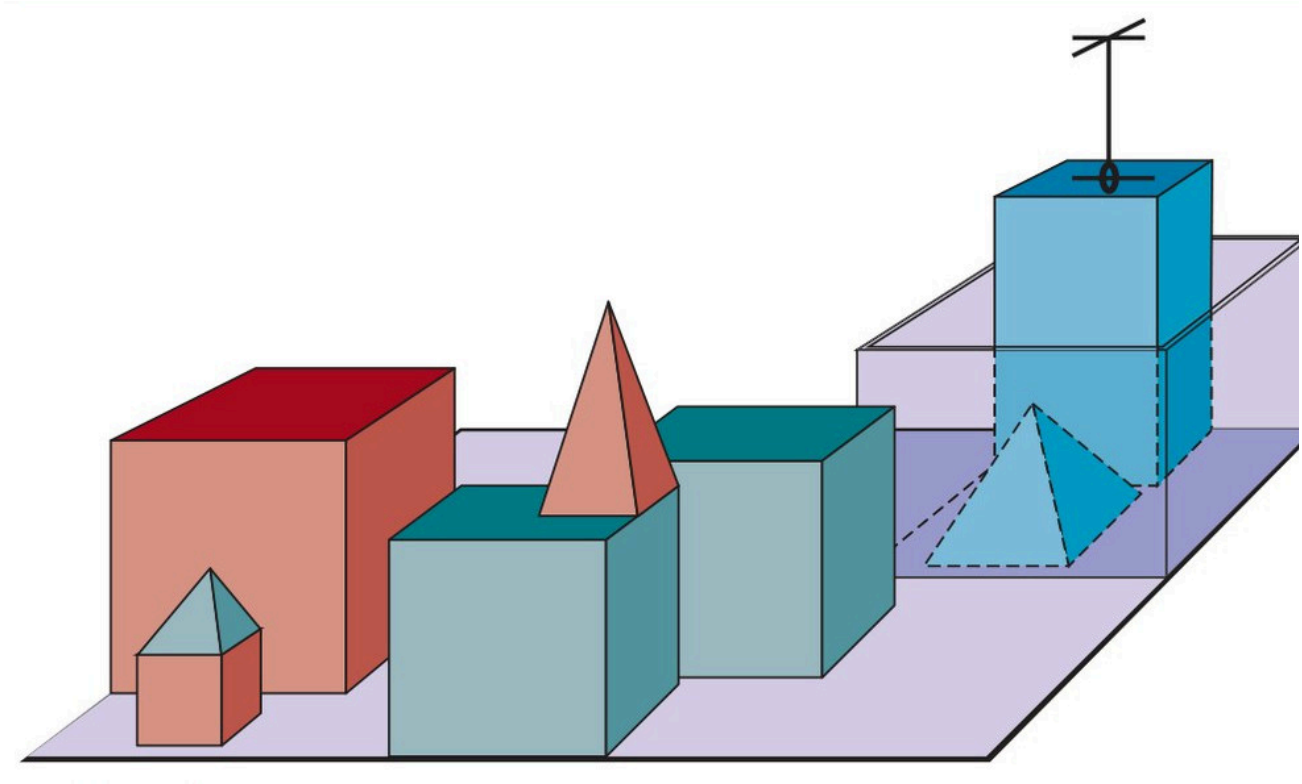
نیوول و سایمون پس از موفقیت LT، «حل‌کنندهٔ عمومی مسئله» (General Problem Solver، یا GPS) را ساختند. برخلاف LT، این برنامه از همان آغاز برای تقلید از پروتکل‌های حل مسئلهٔ انسانی طراحی شده بود. در آن ردهٔ محدود از معماها که می‌توانست حل کند، معلوم شد ترتیبی که برنامه در آن زیرهدف‌ها و کنش‌های ممکن را در نظر می‌گرفت، شبیه ترتیبی است که انسان‌ها با همان مسائل روبه‌رو می‌شدند. پس GPS احتمالاً نخستین برنامهٔ تجسم‌بخش رویکرد «اندیشیدن انسان‌گونه» بود. موفقیت GPS و برنامه‌های بعدی به‌مثابه مدل‌های شناخت، نیوول و سایمون (۱۹۷۶) را به صورت‌بندی فرضیهٔ نامدار «سامانهٔ نمادهای فیزیکی» (Physical Symbol System) کشاند که بر آن است «یک سامانهٔ نمادهای فیزیکی، ابزارهای لازم و کافی برای کنش هوشمندِ عمومی را دارد.» مقصود آنان این بود که هر سامانه‌ای (انسانی یا ماشینی) که هوش نشان می‌دهد باید با دست‌کاری ساختارهای داده‌ایِ ساخته‌شده از نمادها کار کند. بعدها خواهیم دید این فرضیه از جهات بسیار به چالش کشیده شده است.

در آی‌بی‌ام، ناتانیل راچستر و همکارانش برخی از نخستین برنامه‌های هوش مصنوعی را ساختند. هربرت گلرنتر (۱۹۵۹) «اثبات‌کنندهٔ قضیهٔ هندسه» را ساخت که می‌توانست قضیه‌هایی را اثبات کند که بسیاری از دانشجویان ریاضی بسیار پیچیده می‌یابند. این کار پیش‌آهنگِ اثبات‌کننده‌های قضیهٔ ریاضی مدرن بود.

از میان همهٔ کارهای اکتشافی این دوره، شاید تأثیرگذارترین در بلندمدت کار آرتور ساموئل بر بازی چکرز بود. با استفاده از روش‌هایی که اکنون یادگیری تقویتی می‌نامیم (فصل ۲۲ را ببینید)، برنامه‌های ساموئل یاد گرفتند در سطح آماتورِ قوی بازی کنند. او بدین ترتیب این ایده را ابطال کرد که رایانه‌ها فقط می‌توانند آنچه به آنان گفته شده انجام دهند: برنامه‌اش به‌سرعت یاد گرفت بهتر از خالقش بازی کند. برنامه در ۱۹۵۶ در تلویزیون نمایش داده شد و تأثیر عمیقی گذاشت. ساموئل مانند تورینگ در یافتن زمان رایانه مشکل داشت؛ او شب‌ها کار می‌کرد و از ماشین‌هایی استفاده می‌کرد که هنوز در سالن آزمایش کارخانهٔ تولیدی آی‌بی‌ام بودند. برنامهٔ ساموئل پیش‌آهنگ سامانه‌های بعدی بود، مانند تیدی-گمن (تسائورو، ۱۹۹۲) که از بهترین بازیکنان تخته‌نرد جهان بود، و آلفاگو (سیلور و همکاران، ۲۰۱۶) که با شکست‌دادن قهرمان جهان در بازی گو جهان را شگفت‌زده کرد (فصل ۵ را ببینید).

در ۱۹۵۸، جان مک‌کارتی دو مشارکت مهم در هوش مصنوعی داشت. در «یادداشت شمارهٔ ۱ آزمایشگاه هوش مصنوعی ام‌آی‌تی»، زبان سطح‌بالای لیسپ (Lisp) را تعریف کرد که قرار بود سی سال بعد زبان برنامه‌نویسی مسلط هوش مصنوعی شود. و در مقاله‌ای با عنوان «برنامه‌هایی با فهمِ عقل سلیم»، پیشنهادی مفهومی برای سامانه‌های هوش مصنوعی مبتنی بر دانش و استدلال ارائه کرد. این مقاله «توصیه‌پذیر» (Advice Taker) را توصیف می‌کند؛ برنامهٔ فرضی‌ای که دانش عمومی از جهان را در بر می‌گیرد و می‌تواند از آن برای استخراج برنامه‌های کنش استفاده کند. این مفهوم با اصل‌های منطقی ساده‌ای نشان داده شد که برای تولید برنامه‌ای برای رانندگی تا فرودگاه کافی‌اند. برنامه همچنین طوری طراحی شده بود که در جریان عادی کار، اصل‌های تازه بپذیرد و بدین ترتیب بتواند در حوزه‌های تازه به شایستگی برسد بدون آنکه از نو برنامه‌ریزی شود. پس «توصیه‌پذیر» اصول مرکزی بازنمایی دانش و استدلال را تجسم می‌بخشید: مفید است بازنمایی صوری و صریحی از جهان و کارکردش داشته باشیم و بتوانیم آن بازنمایی را با فرایندهای استنتاجی دست‌کاری کنیم. این مقاله بر مسیر هوش مصنوعی تأثیر گذاشت و امروز نیز همچنان مرتبط است. ۱۹۵۸ همچنین سال نقل‌مکان ماروین مینسکی به ام‌آی‌تی بود. اما همکاری آغازین او با مک‌کارتی دوام نیاورد. مک‌کارتی بر بازنمایی و استدلال در منطق صوری تأکید داشت، در حالی که مینسکی بیشتر به کارکردنِ برنامه‌ها

علاقه‌مند بود و سرانجام نگاهی ضدمنطقی پروراند. در ۱۹۶۳، مک‌کارتی آزمایشگاه هوش مصنوعی را در استنفورد آغاز کرد. برنامهٔ او برای استفاده از منطق در ساختن «توصیه‌پذیر» نهایی، با کشف روشِ حل (Resolution) توسط جی. ای. رابینسون در ۱۹۶۵ پیش رفت؛ روشی که الگوریتم کامل اثبات قضیه برای منطق مرتبهٔ اول است (فصل ۹ را ببینید).



شکل ۱.۳ – صحنه‌ای از جهان بلوک‌ها. شِردلو (وینوگراد، ۱۹۷۲) تازه فرمان «بلوکی را پیدا کن که از بلوکی که در دست داری بلندتر است و آن را در جعبه بگذار» را کامل کرده است.

کار استنفورد بر روش‌های همه‌منظورهٔ استدلال منطقی تأکید داشت. کاربردهای منطق شامل سامانه‌های پرسش‌وپاسخ و برنامه‌ریزی کوردل گرین (گرین، ۱۹۶۹) و پروژهٔ رباتیک شیکی در مؤسسهٔ پژوهشی استنفورد (SRI) می‌شد. این پروژهٔ اخیر که در فصل ۲۶ بیشتر بررسی می‌شود، نخستین بار یکپارچگی کامل استدلال منطقی و فعالیت فیزیکی را نشان داد.

در ام‌آی‌تی، مینسکی بر گروهی از دانشجویان نظارت داشت که مسائل محدودی را برمی‌گزیدند که حل‌شان نیازمند هوش می‌نمود. این حوزه‌های محدود به «ریزجهان‌ها» (Microworlds) معروف شدند. برنامهٔ سِینت (SAINT) جیمز اسلگل (۱۹۶۳) می‌توانست مسائل انتگرال‌گیری به‌شکل بسته را که نمونهٔ درس‌های سال اول دانشگاه است حل کند. برنامهٔ «قیاس» (ANALOGY) تام ایوانز (۱۹۶۸) مسائل قیاس هندسیِ آزمون‌های هوش را حل می‌کرد. برنامهٔ «دانش‌آموز» (STUDENT) دنیل ببرو (۱۹۶۷) مسائل داستانی جبر را حل می‌کرد، مانند این مسئله:

> *«اگر تعداد مشتری‌های تام، دو برابرِ مجذورِ ۲۰ درصدِ تعداد آگهی‌هایی باشد که پخش می‌کند، و تعداد آگهی‌هایی که پخش می‌کند ۴۵ باشد، تعداد مشتری‌های تام چند است؟»*

نامدارترین ریزجهان، جهان بلوک‌ها (Blocks World) است که شامل مجموعه‌ای بلوک جامد بر روی میز است (یا بیشتر اوقات، شبیه‌سازی یک میز)؛ چنان‌که در شکل ۱.۳ دیده می‌شود. تکلیف معمول در این جهان، بازچیدن بلوک‌ها به شیوه‌ای خاص با بازوی رباتی است که می‌تواند هر بار یک بلوک را بردارد. جهان بلوک‌ها خانهٔ پروژهٔ بینایی دیوید هافمن (۱۹۷۱)، کار بینایی و انتشار قید دیوید والتز (۱۹۷۵)، نظریهٔ یادگیری پاتریک وینستون (۱۹۷۰)، برنامهٔ فهم زبان طبیعی تری وینوگراد (۱۹۷۲) و برنامه‌ریز اسکات فالمن (۱۹۷۴) بود.

کارهای اولیهٔ بنا بر شبکه‌های عصبی مک‌کالوک و پیتس نیز شکوفا شد. کار شموئل وینوگراد و جک کوان (۱۹۶۳) نشان داد چگونه شمار زیادی از عناصر می‌توانند به‌طور جمعی یک مفهوم فردی را بازنمایی کنند، با افزایش متناظر در استواری و موازی‌سازی. روش‌های یادگیری هب توسط برنی ویدرو (ویدرو و هوف، ۱۹۶۰؛ ویدرو، ۱۹۶۲) که شبکه‌هایش را آدالاین می‌نامید، و فرانک روزن‌بلات (۱۹۶۲) با پرسپترون‌هایش بهبود یافت. قضیهٔ همگرایی پرسپترون (بلوک و همکاران، ۱۹۶۲) می‌گوید الگوریتم یادگیری می‌تواند شدت پیوندهای یک پرسپترون را چنان تنظیم کند که با هر دادهٔ ورودی همخوان شود، به شرط آنکه چنین هم‌خوانی‌ای وجود داشته باشد.

۱.۳.۳ برخورد با واقعیت (۱۹۶۶–۱۹۷۳)

از همان آغاز، پژوهشگران هوش مصنوعی در پیشگویی موفقیت‌های آیندهٔ خود کم‌رو نبودند. این گفتهٔ هربرت سایمون در ۱۹۵۷ اغلب نقل می‌شود:

> *هدف من غافلگیرکردن یا شوکه‌کردن شما نیست — اما ساده‌ترین راه جمع‌بندی این است که بگویم اکنون در جهان ماشین‌هایی هستند که می‌اندیشند، یاد می‌گیرند و می‌آفرینند. وانگهی، توانایی آنان در این کارها به‌سرعت افزایش خواهد یافت تا آنکه — در «آیندهٔ قابل مشاهده» — گسترهٔ مسائلی که می‌توانند حل کنند با گسترهٔ کاربرد ذهن انسان هم‌مرز شود.*

عبارت «آیندهٔ قابل مشاهده» مبهم است، اما سایمون پیش‌بینی‌های عینی‌تری هم کرد: در عرض ۱۰ سال، رایانه‌ای قهرمان شطرنج خواهد شد و قضیهٔ ریاضی مهمی توسط ماشین اثبات خواهد شد. این پیش‌بینی‌ها ظرف ۴۰ سال به واقعیت پیوستند (یا تقریباً)، نه ۱۰ سال. بیش‌اطمینانی سایمون از عملکرد امیدوارکنندهٔ سامانه‌های اولیهٔ هوش مصنوعی در مثال‌های ساده ناشی می‌شد. اما در تقریباً همهٔ موارد، این سامانه‌های اولیه در مسائل دشوارتر شکست خوردند. دو دلیل اصلی برای این شکست وجود داشت. نخست آنکه بسیاری از سامانه‌های اولیهٔ هوش مصنوعی اساساً بر «درون‌نگریِ آگاهانه» دربارهٔ اینکه انسان تکلیفی را چگونه انجام می‌دهد مبتنی بودند، نه بر تحلیل دقیق تکلیف، معنای «حل‌شدن»، و اینکه الگوریتم برای تولیدِ قابل اعتمادِ چنین راه‌حل‌هایی به چه چیزی نیاز دارد.

دومین دلیل شکست، درک‌نکردنِ «ناشدنی‌بودن» (Intractability) بسیاری از مسائلی بود که هوش مصنوعی می‌کوشید حل کند. بیشتر سامانه‌های اولیهٔ حل مسئله با آزمودن ترکیب‌های گوناگون گام‌ها کار می‌کردند تا آنکه راه‌حل پیدا شود. این راهبرد آغاز کار کرد، زیرا ریزجهان‌ها اشیای بسیار اندکی داشتند و بنابراین کنش‌های ممکنِ بسیار اندک و توالی‌های راه‌حلِ بسیار کوتاه. پیش از توسعهٔ نظریهٔ پیچیدگی محاسباتی، این باور گسترده بود که «مقیاس بزرگ‌کردن» برای مسائل بزرگ‌تر صرفاً مسئلهٔ سخت‌افزار سریع‌تر و حافظه‌های بزرگ‌تر است. برای نمونه، خوش‌بینیِ همراه با توسعهٔ

اثبات قضیهٔ روشِ حل، به‌زودی وقتی از میان رفت که پژوهشگران نتوانستند قضیه‌هایی با بیش از چند ده واقعیت را اثبات کنند. این واقعیت که برنامه‌ای ⚠ در اصل می‌تواند راه‌حل پیدا کند، به این معنا نیست که برنامه هیچ‌یک از سازوکارهای لازم برای یافتن آن در عمل را دارد. توهم قدرت محاسباتی نامحدود به برنامه‌های حل مسئله محدود نبود. آزمایش‌های اولیه در تکامل ماشین (که اکنون برنامه‌نویسی ژنتیک نامیده می‌شود) (فریدبرگ، ۱۹۵۸؛ فریدبرگ و همکاران، ۱۹۵۹) بر این باورِ قطعاً درست استوار بودند که با ایجاد توالی مناسبی از جهش‌های کوچک در برنامهٔ زبان ماشین، می‌توان بر برنامه‌ای با عملکرد خوب برای هر تکلیف خاص دست یافت. پس ایده این بود که جهش‌های تصادفی را با فرایند انتخابی آزمود تا جهش‌های به‌ظاهر مفید حفظ شوند. با وجود هزاران ساعت زمان پردازنده، تقریباً هیچ پیشرفتی نشان داده نشد.

ناتوانی در مهار «انفجار ترکیبیاتی» (Combinatorial Explosion) یکی از انتقادهای اصلی به هوش مصنوعی در «گزارش لایتهیل» (لایتهیل، ۱۹۷۳) بود که مبنای تصمیم دولت بریتانیا برای پایان‌دادن به حمایت از پژوهش هوش مصنوعی در همهٔ دانشگاه‌ها به‌جز دو دانشگاه شد. (سنت شفاهی تصویری متفاوت و رنگ‌ارنگ‌تر ترسیم می‌کند، با جاه‌طلبی‌های سیاسی و دشمنی‌های شخصی که شرحش خارج از بحث است.) دشواری سومی نیز برخاست از محدودیت‌های بنیادینِ ساختارهای پایه‌ای که برای تولید رفتار هوشمند به کار می‌رفتند. برای نمونه، کتاب «پرسپترون‌ها» (۱۹۶۹) مینسکی و پاپرت اثبات کرد هرچند می‌توان نشان داد پرسپترون‌ها (شکل ساده‌ای از شبکهٔ عصبی) هر آنچه را توانایی بازنمایی دارند یاد می‌گیرند، اما بسیار اندک را می‌توانند بازنمایی کنند. به‌طور خاص، پرسپترون دوورودی‌ای نمی‌توان آموزش داد که تشخیص دهد دو ورودی‌اش متفاوت‌اند. هرچند نتایج آنان دربارهٔ شبکه‌های چندلایهٔ پیچیده‌تر صدق نمی‌کرد، بودجهٔ پژوهش شبکه‌های عصبی به‌زودی تقریباً به صفر رسید. به‌طرز طنزآمیزی، الگوریتم‌های یادگیری انتشار پسرو (Back-Propagation) که قرار بود در اواخر دههٔ ۱۹۸۰ و بار دیگر در دههٔ ۲۰۱۰ جهشی عظیم در پژوهش شبکه‌های عصبی ایجاد کنند، از همان اوایل دههٔ ۱۹۶۰ در زمینه‌های دیگر توسعه یافته بودند (کلی، ۱۹۶۰؛ برایسون، ۱۹۶۲).

۱.۳.۴ سامانه‌های خبره (۱۹۶۹–۱۹۸۶)

تصویری که در دههٔ نخست پژوهش هوش مصنوعی از حل مسئله پدید آمده بود، سازوکار جستجوی همه‌منظوره‌ای بود که می‌کوشید گام‌های استدلالی ابتدایی را به هم رشته کند تا راه‌حل کامل بیابد. این رویکردها را «روش‌های ضعیف» (Weak Methods) نامیده‌اند، زیرا هرچند عام‌اند، برای نمونه‌های بزرگ یا دشوار مقیاس‌پذیر نیستند. بدیل روش‌های ضعیف، به‌کارگیری دانش قدرتمندتر و حوزه‌ویژه است که گام‌های استدلالی بزرگ‌تری را ممکن می‌کند و می‌تواند مواردِ رایج در حوزه‌های تخصصی باریک را آسان‌تر اداره کند. شاید بتوان گفت برای حل مسئله‌ای سخت، تقریباً باید از قبل پاسخ را بدانی.

برنامهٔ دنرال (DENDRAL) (بوکانن و همکاران، ۱۹۶۹) نمونه‌ای آغازین از این رویکرد بود. در استنفورد توسعه یافت، جایی که اد فایگنباوم (شاگرد پیشین هربرت سایمون)، بروس بوکانن (فیلسوفی که دانشمند رایانه شد) و جاشوا لدربرگ (ژنتیک‌دان برندهٔ نوبل) دست‌به‌کار حل مسئلهٔ استنتاج ساختار مولکولی از اطلاعاتِ طیف‌سنج جرمی شدند. ورودی برنامه شامل فرمول تجربی مولکول (مانند C6H13NO2) و طیف جرمی است که جرم قطعه‌های گوناگون مولکول را می‌دهد؛ قطعه‌هایی که هنگام بمباران مولکول با پرتو الکترون تولید می‌شوند. برای نمونه، طیف جرمی ممکن است قلهٔ m = 15 داشته باشد که با جرم قطعهٔ متیل (CH3) متناظر است. نسخهٔ ساده‌لوحانهٔ برنامه همهٔ

ساختارهای ممکنِ سازگار با فرمول را تولید می‌کرد و سپس پیش‌بینی می‌کرد برای هر یک چه طیف جرمی مشاهده خواهد شد و آن را با طیف واقعی مقایسه می‌کرد. چنان‌که انتظار می‌رود، این کار حتی برای مولکول‌های نسبتاً متوسط نیز ناشدنی است. پژوهشگران دنرال با شیمی‌دانان تحلیلی مشورت کردند و دریافتند آنان با جست‌وجوی الگوهای معروف قله‌ها در طیف کار می‌کنند؛ الگوهایی که ساختارهای فرعی رایج در مولکول را پیشنهاد می‌کنند. برای نمونه، این قاعده برای شناسایی زیرگروه کتون (C=O) (که وزنش ۲۸ است) به کار می‌رود:

```
اگر دو قله در x₁ و x₂ وجود داشته باشد چنان‌که:
(الف) x₁ + x₂ = M + 28 (که M جرم کل مولکول است)؛
(ب) x₁ − 28 قلهٔ بلند باشد؛
(ج) x₂ − 28 قلهٔ بلند باشد؛ و
(د) دست‌کم یکی از x₁ و x₂ بلند باشد؛
آنگاه زیرگروه کتون وجود دارد.
```

شناخت اینکه مولکول ساختار فرعیِ خاصی دارد، شمار نامزدهای ممکن را به‌شدت کاهش می‌دهد. به گفتهٔ مؤلفان، دنرال قدرتمند بود، زیرا دانشِ مرتبطِ طیف‌سنجی جرمی را نه در قالب اصول نخستین، بلکه در «دستورپخت‌های پخت‌وپز» کارآمد تجسم می‌کرد (فایگنباوم و همکاران، ۱۹۷۱). اهمیت دنرال در آن بود که نخستین سامانهٔ موفقِ دانش‌فشرده بود: تخصصش از شمار زیادی قاعدهٔ ویژه‌برنامه برمی‌خاست. در ۱۹۷۱، فایگنباوم و دیگران در استنفورد «پروژهٔ برنامه‌نویسی اکتشافی» (HPP) را برای بررسی اینکه روش‌شناسی تازهٔ سامانه‌های خبره (Expert Systems) تا چه حد به حوزه‌های دیگر قابل تعمیم است آغاز کردند.

تلاش بزرگ بعدی، سامانهٔ مایسین (MYCIN) برای تشخیص عفونت‌های خونی بود. مایسین با حدود ۴۵۰ قاعده می‌توانست به خوبی برخی متخصصان عمل کند و بسیار بهتر از پزشکان جوان. همچنین دو تفاوت عمده با دنرال داشت. نخست، برخلاف قواعد دنرال، هیچ مدل نظری عمومی‌ای وجود نداشت که قواعد مایسین از آن استنتاج شوند؛ آن قواعد باید از مصاحبه‌های گسترده با متخصصان به‌دست می‌آمدند. دوم، قواعد باید عدم قطعیتِ همراه با دانش پزشکی را بازتاب می‌دادند. مایسین حسابِ عدم قطعیتی به نام ضرایب قطعیت (Certainty Factor) در بر داشت (فصل ۱۳ را ببینید) که در آن زمان به نظر می‌رسید با شیوهٔ سنجش تأثیر شواهد بر تشخیص توسط پزشکان هم‌خوانی خوبی دارد.

نخستین سامانهٔ خبرهٔ تجاری موفق، R1، در شرکت تجهیزات دیجیتال (DEC) به کار افتاد (مک‌درموت، ۱۹۸۲). برنامه به پیکربندی سفارش‌های سامانه‌های رایانه‌ای تازه کمک می‌کرد؛ تا ۱۹۸۶ سالانه حدود ۴۰ میلیون دلار برای شرکت صرفه‌جویی می‌کرد. تا ۱۹۸۸، گروه هوش مصنوعی DEC چهل سامانهٔ خبرهٔ مستقر کرده بود و سامانه‌های بیشتری در راه بود. دوپونت ۱۰۰ سامانهٔ در حال استفاده و ۵۰۰ در حال توسعه داشت. تقریباً هر شرکت بزرگ آمریکایی گروه هوش مصنوعی خودش را داشت و یا از سامانه‌های خبره استفاده می‌کرد یا به بررسی‌شان مشغول بود.

اهمیت دانش حوزه در حوزهٔ فهم زبان طبیعی نیز آشکار بود. با وجود موفقیت سامانهٔ شِردلو (SHRDLU) وینوگراد، روش‌های آن به تکلیف‌های عمومی‌تر تعمیم نیافت: برای مسائلی مانند رفع ابهام، از قواعد ساده‌ای استفاده می‌کرد که به گسترهٔ کوچکِ جهان بلوک‌ها وابسته بود. چند پژوهشگر، از جمله یوجین چارنیاک در ام‌آی‌تی و راجر شانک در ییل، پیشنهاد کردند فهم زبانِ استوار به دانش عمومی دربارهٔ جهان و روشی عمومی برای استفاده از آن دانش نیاز دارد.

(شانک فراتر رفت و ادعا کرد «چیزی به نام نحو وجود ندارد» که زبان‌شناسان زیادی را آزرد، اما بحث مفیدی را آغاز کرد.) شانک و دانشجویانش سلسله‌برنامه‌هایی ساختند (شانک و ایبلسون، ۱۹۷۷؛ ویلنسکی، ۱۹۷۸؛ شانک و ریسبک، ۱۹۸۱) که همگی وظیفهٔ فهم زبان طبیعی را داشتند. اما تأکید کمتر بر خود زبان بود و بیشتر بر مسائل بازنمایی و استدلال با دانشِ لازم برای فهم زبان.

رشد فراگیر کاربردها در مسائل جهان واقعی به توسعهٔ گسترهٔ وسیعی از ابزارهای بازنمایی و استدلال انجامید. برخی بر منطق مبتنی بودند — برای نمونه، زبان پرولوگ (Prolog) در اروپا و ژاپن رایج شد و خانوادهٔ پلنر (PLANNER) در آمریکا. برخی دیگر، به پیروی از ایدهٔ قاب‌ها (Frames) مینسکی (۱۹۷۵)، رویکردی ساختاریافته‌تر در پیش گرفتند: واقعیت‌هایی دربارهٔ انواع خاصِ شیء و رویداد گرد می‌آوردند و انواع را در سلسله‌مراتبی بزرگِ رده‌بندی، مشابه رده‌بندی زیستی، می‌چیدند.

در ۱۹۸۱، دولت ژاپن «پروژهٔ نسل پنجم» را اعلام کرد؛ برنامه‌ای ده‌ساله برای ساختن رایانه‌های هوشمندِ به‌شدت موازی که پرولوگ اجرا می‌کردند. بودجه‌اش قرار بود به بیش از ۱/۳ میلیارد دلار به پول امروز برسد. در پاسخ، آمریکا «شرکت فناوری ریزالکترونیک و رایانه» (MCC) را تشکیل داد؛ کنسرسیومی برای تضمین رقابت‌پذیری ملی. در هر دو مورد، هوش مصنوعی بخشی از تلاشی گسترده بود، شامل طراحی تراشه و پژوهش واسط انسانی. در بریتانیا، «گزارش آلوِی» بودجهٔ حذف‌شده توسط گزارش لایتهیل را بازگرداند. اما هیچ‌یک از این پروژه‌ها هرگز به هدف‌های جاه‌طلبانه‌اش در توانایی‌های تازهٔ هوش مصنوعی یا تأثیر اقتصادی نرسید.

در مجموع، صنعت هوش مصنوعی از چند میلیون دلار در ۱۹۸۰ به میلیاردها دلار در ۱۹۸۸ رونق گرفت، با صدها شرکت سازندهٔ سامانه‌های خبره، سامانه‌های بینایی، ربات‌ها و نرم‌افزار و سخت‌افزارِ ویژهٔ این اهداف. به‌زودی دوره‌ای به نام «زمستان هوش مصنوعی» (AI Winter) فرا رسید؛ دورهٔ سقوط بسیاری از شرکت‌ها، زیرا نتوانستند وعده‌های اغراق‌آمیز خود را عملی کنند. ساخت و نگهداری سامانه‌های خبره برای حوزه‌های پیچیده دشوار از آب درآمد؛ تا حدی به این دلیل که روش‌های استدلالیِ به‌کاررفته در سامانه‌ها در برابر عدم قطعیت از هم می‌پاشید و تا حدی به این دلیل که سامانه‌ها نمی‌توانستند از تجربه بیاموزند.

۱.۳.۵ بازگشت شبکه‌های عصبی (۱۹۸۶ تا کنون)

در اواسط دههٔ ۱۹۸۰، دست‌کم چهار گروه گوناگون الگوریتم یادگیری انتشار پسرو را که نخستین بار در اوایل دههٔ ۱۹۶۰ توسعه یافته بود بازآفرینی کردند. الگوریتم در بسیاری از مسائل یادگیری علوم رایانه و روان‌شناسی به کار رفت و انتشار گستردهٔ نتایج در مجموعهٔ «پردازش توزیع‌یافتهٔ موازی» (روملهارت و مک‌کللند، ۱۹۸۶) هیجان عظیمی برانگیخت. این مدل‌های موسوم به پیوندگرا (Connectionist) از دید برخی رقیب مستقیمِ مدل‌های نمادینِ نیوول و سایمون و نیز رویکرد منطق‌گرایانهٔ مک‌کارتی و دیگران دانسته می‌شدند. ممکن است بدیهی بنماید که انسان‌ها در سطحی نمادها را دست‌کاری می‌کنند — در واقع کتاب «گونهٔ نمادین» (۱۹۹۷) مردم‌شناس ترنس دیکن پیشنهاد می‌کند این ویژگیِ تعریف‌کنندهٔ انسان است. در برابر، جف هینتون، چهرهٔ پیشرو در احیای شبکه‌های عصبی در دهه‌های ۱۹۸۰ و ۲۰۱۰، نمادها را «اثیر نورانیِ هوش مصنوعی» توصیف کرده است — اشاره به محیطِ ناموجودی که بسیاری از فیزیک‌دانان سدهٔ نوزدهم باور داشتند امواج الکترومغناطیسی در آن منتشر می‌شوند. قطعاً بسیاری از مفهوم‌هایی که در زبان نام می‌بریم، با بررسی دقیق‌تر، آن شرایط لازم و کافیِ منطقاً تعریف‌شده را ندارند که پژوهشگران اولیهٔ هوش مصنوعی امید

داشتند به شکل اصل‌هایی در آورند. چه‌بسا مدل‌های پیوندگرا مفهوم‌های درونی را به‌شکلی سیال‌تر و نادقیق‌تر شکل می‌دهند که برای درهم‌ریختگی جهان واقعی مناسب‌تر است. آنان همچنین توانایی یادگیری از مثال‌ها را دارند — می‌توانند مقدار خروجیِ پیش‌بینی‌شده را با مقدار درستِ مسئله مقایسه کنند و پارامترهایشان را برای کاهش تفاوت اصلاح کنند و بدین ترتیب احتمال عملکرد خوب در مثال‌های آینده را افزایش دهند.

۱.۳.۶ استدلال احتمالاتی و یادگیری ماشین (۱۹۸۷ تا کنون)

شکنندگی سامانه‌های خبره به رویکردی تازه و علمی‌تر انجامید که احتمال را به‌جای منطق بولی، یادگیری ماشین را به‌جای دست‌کدگذاری، و نتایج تجربی را به‌جای ادعاهای فلسفی در بر می‌گرفت.[14] رایج‌تر شد که بر نظریه‌های موجود بنا شود تا نظریهٔ کاملاً تازه پیشنهاد شود؛ ادعاها بر قضیه‌های دقیق یا روش‌شناسی تجربیِ استوار (کوهن، ۱۹۹۵) شود تا بر شهود؛ و ارتباط با کاربردهای جهان واقعی نشان داده شود تا مثال‌های اسباب‌بازی.

مجموعه‌های معیارِ مشترک به هنجارِ نشان‌دادن پیشرفت بدل شدند: از جمله مخزن یوسی‌آی‌روین برای مجموعه‌داده‌های یادگیری ماشین، رقابت بین‌المللی برنامه‌ریزی برای الگوریتم‌های برنامه‌ریزی، پیکرهٔ لیبری‌اسپیچ برای تشخیص گفتار، مجموعه‌دادهٔ MNIST برای تشخیص ارقام دست‌نویس، ایمج‌نت (ImageNet) و کوکو (COCO) برای تشخیص شیء در تصویر، اسکاد (SQUAD) برای پرسش‌وپاسخ زبان طبیعی، رقابت WMT برای ترجمهٔ ماشینی، و رقابت‌های بین‌المللی SAT برای حل‌کننده‌های ارضاپذیری بولی.

هوش مصنوعی تا حدی به‌عنوان شورش علیه محدودیت‌های رشته‌های موجود مانند نظریهٔ کنترل و آمار بنیان نهاده شد، اما در این دوره نتایج مثبت آن رشته‌ها را در آغوش کشید. چنان‌که دیوید مک‌آلیستر (۱۹۹۸) گفت:

> *در دورهٔ آغازین هوش مصنوعی، این امر محتمل می‌نمود که شکل‌های تازهٔ محاسبهٔ نمادین، مانند قاب‌ها و شبکه‌های معنایی، بخش بزرگی از نظریهٔ کلاسیک را منسوخ می‌کنند. این به نوعی انزواگرایی انجامید که در آن هوش مصنوعی تا حد زیادی از بقیهٔ علوم رایانه جدا شد. این انزواگرایی اکنون کنار گذاشته می‌شود. این شناخت وجود دارد که یادگیری ماشین نباید از نظریهٔ اطلاعات جدا باشد، استدلال نامطمئن نباید از مدل‌سازی تصادفی جدا باشد، جستجو نباید از بهینه‌سازی و کنترل کلاسیک جدا باشد، و استدلال خودکار نباید از روش‌های صوری و تحلیل ایستا جدا باشد.*

رشتهٔ تشخیص گفتار این الگو را نشان می‌دهد. در دههٔ ۱۹۷۰، گسترهٔ وسیعی از معماری‌ها و رویکردهای گوناگون آزموده شد. بسیاری از آن‌ها نسبتاً موردی (Ad hoc) و شکننده بودند و فقط روی چند مثالِ به‌دقت انتخاب‌شده کار می‌کردند. در دههٔ ۱۹۸۰، رویکردهای مبتنی بر مدل‌های مارکوف پنهان (Hidden Markov Models، یا HMM) بر این حوزه مسلط شدند. دو جنبه از HMM مرتبط است. نخست، آنان بر نظریهٔ ریاضی دقیقی استوارند. این به پژوهشگران گفتار امکان داد بر چند دهه نتیجهٔ ریاضیِ توسعه‌یافته در رشته‌های دیگر بنا کنند. دوم، آنان با فرایند آموزش بر روی پیکرهٔ بزرگی از داده‌های گفتار واقعی تولید می‌شوند. این تضمین می‌کند عملکرد استوار است و در آزمون‌های کورِ دقیق، HMMها امتیازهایشان را پیوسته بهبود دادند. در نتیجه، فناوری گفتار و رشتهٔ مرتبطِ تشخیص حروف دست‌نویس به

کاربردهای گستردهٔ صنعتی و مصرفی گذر کردند. توجه کنید هیچ ادعای علمی‌ای نبود که انسان‌ها برای تشخیص گفتار از HMM استفاده می‌کنند؛ بلکه HMMها چارچوبی ریاضی برای فهم و حل مسئله فراهم کردند. اما در بخش ۱.۳.۸ خواهیم دید یادگیری عمیق این روایت راحت را تا حدی بر هم زده است.

۱۹۸۸ سال مهمی برای پیوند هوش مصنوعی با رشته‌های دیگر، از جمله آمار، تحقیق در عملیات، نظریهٔ تصمیم و نظریهٔ کنترل بود. «استدلال احتمالاتی در سامانه‌های هوشمند» جودیا پرل (۱۹۸۸) به پذیرش تازه‌ای از احتمال و نظریهٔ تصمیم در هوش مصنوعی انجامید. توسعهٔ شبکه‌های بیزی (Bayesian Networks) توسط پرل، صورت‌بندی دقیق و کارآمدی برای بازنمایی دانش نامطمئن و نیز الگوریتم‌های عملی استدلال احتمالاتی به دست داد. فصل‌های ۱۲ تا ۱۶ این حوزه را پوشش می‌دهند، افزون بر تحولات جدیدتری که قدرت بیانی صورت‌های احتمالاتی را بسیار افزایش داده‌اند؛ فصل ۲۰ روش‌های یادگیری شبکه‌های بیزی و مدل‌های مرتبط از داده را توصیف می‌کند.

دومین مشارکت بزرگ در ۱۹۸۸، کار ریچ ساتن در پیوند یادگیری تقویتی — که در برنامهٔ بازی چکرزِ آرتور ساموئل در دههٔ ۱۹۵۰ به کار رفته بود — با نظریهٔ فرایندهای تصمیم مارکوف (MDP) توسعه‌یافته در رشتهٔ تحقیق در عملیات بود. سیل عظیمی از کارها در پی آمد که پژوهش برنامه‌ریزی هوش مصنوعی را به MDP پیوند می‌داد و رشتهٔ یادگیری تقویتی افزون بر یافتن پایه‌های نظری عمیق، در رباتیک و کنترل فرایند نیز کاربرد یافت. یکی از پیامدهای قدردانی تازهٔ هوش مصنوعی از داده، مدل‌سازی آماری، بهینه‌سازی و یادگیری ماشین، اتحاد مجدد تدریجی زیرشاخه‌هایی مانند بینایی رایانه، رباتیک، تشخیص گفتار، سامانه‌های چندعاملی و پردازش زبان طبیعی بود که تا حدی از هستهٔ هوش مصنوعی جدا شده بودند. فرایند بازیکپارچه‌سازی منافع چشمگیری به بار آورده؛ هم از نظر کاربرد — برای نمونه، به‌کارگیری ربات‌های عملی در این دوره بسیار گسترش یافت — و هم در فهم نظری بهتر مسائل اصلی هوش مصنوعی.

۱.۳.۷ داده‌های بزرگ (۲۰۰۱ تا کنون)

پیشرفت‌های شگفت‌انگیز در توان محاسباتی و پیدایش وب جهان‌گستر، ایجاد مجموعه‌داده‌های بسیار بزرگ را تسهیل کرده — پدیده‌ای که گاه داده‌های بزرگ (Big Data) نامیده می‌شود. این مجموعه‌داده‌ها شامل تریلیون‌ها واژهٔ متن، میلیاردها تصویر و میلیاردها ساعت گفتار و ویدئو، و نیز حجم عظیمی دادهٔ ژنومی، دادهٔ ردیابی خودرو، دادهٔ جریان کلیک، دادهٔ شبکه‌های اجتماعی و مانند آن است.

این به توسعهٔ الگوریتم‌های یادگیریِ ویژه‌طراحی‌شده برای بهره‌گیری از مجموعه‌داده‌های بسیار بزرگ انجامید. اغلب اکثریت قریب به اتفاق مثال‌های چنین مجموعه‌داده‌هایی برچسب‌نخورده‌اند؛ برای نمونه، در کار تأثیرگذار یاروفسکی (۱۹۹۵) دربارهٔ ابهام‌زدایی معنای واژه، رخدادهای واژه‌ای مانند «plant» در مجموعه‌داده برچسب‌گذاری نشده‌اند که آیا به گیاه اشاره دارند یا کارخانه. اما با مجموعه‌داده‌های به اندازهٔ کافی بزرگ، الگوریتم‌های یادگیری مناسب می‌توانند در تکلیف شناسایی معنای مورد نظر در یک جمله به دقتی بیش از ۹۶٪ برسند. وانگهی، بانکو و بریل (۲۰۰۱) استدلال کردند بهبود عملکردِ حاصل از افزایش اندازهٔ مجموعه‌داده به میزان دو تا سه مرتبهٔ بزرگی، بر هر بهبودی که از دست‌کاری الگوریتم به دست می‌آید می‌چربد.

پدیدهٔ مشابهی در تکلیف‌های بینایی رایانه مانند پر کردن حفره‌های عکس رخ می‌دهد — حفره‌هایی که یا بر اثر آسیب‌اند یا بر اثر حذف دوستان سابق. هیس و افراس (۲۰۰۷) روش هوشمندانه‌ای برای این کار توسعه دادند: درآمیختن پیکسل‌ها از تصاویر مشابه. آنان دریافتند این فن با پایگاه‌داده‌ای فقط چند هزار تصویری ضعیف کار می‌کند، اما با میلیون‌ها تصویر از آستانهٔ کیفیت می‌گذرد. به‌زودی، در دسترس بودن ده‌ها میلیون تصویر در پایگاه‌دادهٔ ایمج‌نت (دنگ و همکاران، ۲۰۰۹) انقلابی در حوزهٔ بینایی رایانه برپا کرد. در دسترس بودن داده‌های بزرگ و چرخش به سوی یادگیری ماشین به هوش مصنوعی کمک کرد جذابیت تجاری خود را بازیابد (هاونستین، ۲۰۰۵؛ هالیوی و همکاران، ۲۰۰۹). داده‌های بزرگ عامل حیاتی در پیروزی ۲۰۱۱ سامانهٔ واتسونِ آی‌بی‌ام بر قهرمانان انسانی در بازی پرسش‌وپاسخ «جئوپاردی!» بود؛ رویدادی که تأثیر عمده بر ادراک عمومی از هوش مصنوعی داشت.

۱.۳.۸ یادگیری عمیق (۲۰۱۱ تا کنون)

اصطلاح یادگیری عمیق (Deep Learning) به یادگیری ماشین با استفاده از لایه‌های متعددِ عناصر محاسباتی ساده و قابل‌تنظیم اشاره دارد. آزمایش با چنین شبکه‌هایی به دههٔ ۱۹۷۰ بازمی‌گردد و در قالب شبکه‌های عصبی کانولوشنی در دههٔ ۱۹۹۰ در تشخیص ارقام دست‌نویس موفقیت‌هایی داشت (لیکان و همکاران، ۱۹۹۵). اما تا ۲۰۱۱ بود که روش‌های یادگیری عمیق واقعاً اوج گرفتند. این نخست در تشخیص گفتار رخ داد و سپس در تشخیص شیء بصری. در رقابت ایمج‌نت ۲۰۱۲، که طبقه‌بندی تصاویر در یک‌هزار رده (آرمادیلو، قفسهٔ کتاب، دربطری‌بازکن و...) را می‌طلبید، سامانهٔ یادگیری عمیقِ ساخته‌شده در گروه جفری هینتون در دانشگاه تورنتو (کریژفسکی و همکاران، ۲۰۱۳) بهبود چشمگیری نسبت به سامانه‌های پیشین نشان داد که عمدتاً بر ویژگی‌های دست‌ساخته مبتنی بودند. از آن پس، سامانه‌های یادگیری عمیق در برخی تکلیف‌های بینایی از عملکرد انسانی فراتر رفته‌اند (و در برخی دیگر عقب مانده‌اند). دستاوردهای مشابهی در تشخیص گفتار، ترجمهٔ ماشینی، تشخیص پزشکی و بازی‌ها گزارش شده است. به‌کارگیری شبکهٔ عمیق برای بازنمایی تابع ارزشیابی به پیروزی‌های آلفاگو بر برترین بازیکنان انسانی گو کمک کرد (سیلور و همکاران، ۲۰۱۶، ۲۰۱۷، ۲۰۱۸).

این موفقیت‌های چشمگیر به احیای علاقه به هوش مصنوعی در میان دانشجویان، شرکت‌ها، سرمایه‌گذاران، دولت‌ها، رسانه‌ها و عموم انجامیده است. به نظر می‌رسد هر هفته خبری از کاربرد تازهٔ هوش مصنوعی هست که به عملکرد انسانی نزدیک می‌شود یا از آن فراتر می‌رود، اغلب همراه با گمانه‌زنی دربارهٔ موفقیتِ شتابان یا زمستانی تازهٔ هوش مصنوعی.

یادگیری عمیق به‌شدت به سخت‌افزار قدرتمند وابسته است. در حالی که پردازندهٔ مرکزی استاندارد رایانه می‌تواند 10^9 یا 10^{10} عملیات در ثانیه انجام دهد، الگوریتم یادگیری عمیق روی سخت‌افزار تخصصی (مانند GPU، TPU یا FPGA) ممکن است بین 10^{14} تا 10^{17} عملیات در ثانیه مصرف کند، بیشتر به شکل عملیات ماتریسی و برداریِ به‌شدت موازی. البته یادگیری عمیق همچنین به در دسترس بودن مقادیر زیادی دادهٔ آموزشی و چند ترفند الگوریتمی وابسته است (فصل ۲۱ را ببینید).

۱.۴ وضعیت هنر

«مطالعهٔ صدسالهٔ هوش مصنوعی» دانشگاه استنفورد (که به AI100 نیز شناخته می‌شود) هیئت‌هایی از کارشناسان را گرد می‌آورد تا گزارش‌هایی دربارهٔ وضعیت هنر در هوش مصنوعی ارائه دهند. گزارش ۲۰۱۶ آنان (استون و همکاران، ۲۰۱۶؛ گروز و استون، ۲۰۱۸) نتیجه می‌گیرد «افزایش چشمگیری در کاربردهای آیندهٔ هوش مصنوعی می‌توان انتظار داشت، از جمله خودروهای خودران بیشتر، تشخیص‌های پزشکی و درمان هدفمند، و کمک فیزیکی برای مراقبت از سالمندان» و «جامعه اکنون در پیوندگاهی حیاتی برای تعیین چگونگی به‌کارگیری فناوری‌های مبتنی بر هوش مصنوعی قرار دارد، به‌گونه‌ای که ارزش‌های دموکراتیک مانند آزادی، برابری و شفافیت را ترویج کند نه اینکه مانع شود.» AI100 همچنین «شاخص هوش مصنوعی» را در aiindex.org تولید می‌کند تا ردیابی پیشرفت را کمک کند. چند نکتهٔ برجسته از گزارش‌های ۲۰۱۸ و ۲۰۱۹ این شاخص (مقایسه با سال پایهٔ ۲۰۰۰ مگر خلاف آن گفته شود):

- **انتشارات:** مقاله‌های هوش مصنوعی بین ۲۰۱۰ و ۲۰۱۹ بیست برابر افزایش یافت و به حدود ۲۰,۰۰۰ در سال رسید. محبوب‌ترین رده، یادگیری ماشین بود. (مقاله‌های یادگیری ماشین در arXiv.org از ۲۰۰۹ تا ۲۰۱۷ هر سال دو برابر شدند.) بینایی رایانه و پردازش زبان طبیعی در رتبه‌های بعدی بودند.
- **رویکرد رسانه‌ها:** حدود ۷۰٪ مقاله‌های خبری دربارهٔ هوش مصنوعی خنثایند، اما مقاله‌های با لحن مثبت از ۱۲٪ در ۲۰۱۶ به ۳۰٪ در ۲۰۱۸ افزایش یافت. شایع‌ترین موضوع‌ها اخلاقی‌اند: حریم خصوصی داده و سوگیری الگوریتم.
- **دانشجویان:** ثبت‌نام در دوره‌ها از سطح پایهٔ ۲۰۱۰ در آمریکا ۵ برابر و در سطح بین‌المللی ۱۶ برابر افزایش یافت. هوش مصنوعی محبوب‌ترین تخصص در علوم رایانه است.
- **تنوع:** استادان هوش مصنوعی در سراسر جهان حدود ۸۰٪ مرد و ۲۰٪ زن‌اند. اعداد مشابهی برای دانشجویان دکترا و استخدام‌های صنعتی برقرار است.
- **کنفرانس‌ها:** حضور در نورپس (NeurIPS) از ۲۰۱۲ هشتصد درصد افزایش یافت و به ۱۳,۵۰۰ شرکت‌کننده رسید. دیگر کنفرانس‌ها رشد سالانهٔ حدود ۳۰٪ دارند.
- **صنعت:** استارتاپ‌های هوش مصنوعی آمریکا ۲۰ برابر شدند و به بیش از ۸۰۰ رسیدند.
- **بین‌المللی‌شدن:** چین سالانه مقاله‌های بیشتری از آمریکا و تقریباً به اندازهٔ تمام اروپا منتشر می‌کند. اما در تأثیر مبتنی بر استناد، نویسندگان آمریکایی ۵۰٪ از نویسندگان چینی جلوترند. سنگاپور، برزیل، استرالیا، کانادا و هند سریع‌ترین کشورهای در حال رشد از نظر شمار استخدام‌های هوش مصنوعی‌اند.
- **بینایی:** نرخ خطا در تشخیص شیء (آن‌چنان‌که در LSVRC، رقابت چالش تشخیص مقیاس بزرگ بصری، به دست آمد) از ۲۸٪ در ۲۰۱۰ به ۲٪ در ۲۰۱۷ بهبود یافت و از عملکرد انسانی فراتر رفت. دقت در پاسخ به پرسش‌های بصری بازپاسخ (VQA) از ۲۰۱۵ از ۵۵٪ به ۶۸٪ بهبود یافت، اما با ۸۳٪ هنوز از عملکرد انسانی عقب است.
- **سرعت:** زمان آموزش برای تکلیف تشخیص تصویر فقط در دو سال گذشته ۱۰۰ برابر کاهش یافت. میزان توان محاسباتی به‌کاررفته در کاربردهای برتر هوش مصنوعی هر ۳/۴ ماه دو برابر می‌شود.

- **زبان:** دقت در پاسخ به پرسش، آن‌چنان‌که با امتیاز F1 در مجموعه‌دادهٔ پرسش‌وپاسخ استنفورد (SQUAD) سنجیده می‌شود، از ۲۰۱۵ تا ۲۰۱۹ از ۶۰ به ۹۵ افزایش یافت؛ در گونهٔ SQUAD 2 پیشرفت سریع‌تر بود و فقط در یک سال از ۶۲ به ۹۰ رسید. هر دو امتیاز از سطح عملکرد انسانی فراترند.
- **معیارهای انسانی:** تا ۲۰۱۹، سامانه‌های هوش مصنوعی بنابر گزارش‌ها در شطرنج، گو، پوکر، پک‌من، جئوپاردی!، تشخیص شیء ایمجنت، تشخیص گفتار در حوزهٔ محدود، ترجمهٔ چینی به انگلیسی در حوزهٔ محدود، کوئیک ۳، دوتا ۲، استارکرفت ۲، بازی‌های گوناگون آتاری، تشخیص سرطان پوست، تشخیص سرطان پروستات، تاخوردگی پروتئین و تشخیص رتینوپاتی دیابتی به سطح عملکرد انسانی رسیده یا از آن فراتر رفته‌اند.

چه زمانی (اگر هرگز) سامانه‌های هوش مصنوعی در گسترهٔ وسیعی از تکلیف‌ها به عملکرد هم‌سطح انسان خواهند رسید؟ فورد (۲۰۱۸) با کارشناسان هوش مصنوعی مصاحبه می‌کند و دامنهٔ وسیعی از سال‌های هدف می‌یابد، از ۲۰۲۹ تا ۲۲۰۰، با میانگین ۲۰۹۹. در نظرسنجی مشابهی (گریس و همکاران، ۲۰۱۷) ۵۰٪ پاسخ‌دهندگان فکر می‌کردند این اتفاق تا ۲۰۶۶ رخ دهد، هرچند ۱۰٪ فکر می‌کردند همین اوایل ۲۰۲۵ ممکن است و چند نفر هم گفتند «هرگز». کارشناسان همچنین دربارهٔ اینکه به پیشرفت‌های بنیادین تازه نیاز داریم یا فقط اصلاح رویکردهای کنونی، تقسیم شدند. اما پیش‌بینی‌هایشان را چندان جدی نگیرید؛ چنان‌که فیلیپ تتلاک (۲۰۱۷) در حوزهٔ پیش‌بینی رویدادهای جهان نشان می‌دهد، کارشناسان بهتر از آماتورها نیستند.

سامانه‌های هوش مصنوعی آینده چگونه کار خواهند کرد؟ هنوز نمی‌توانیم بگوییم. چنان‌که در این بخش شرح داده شد، این رشته چندین روایت دربارهٔ خود برگزیده است — نخست این ایدهٔ جسورانه که هوش ماشینی اصلاً ممکن است، سپس اینکه می‌توان با رمزگذاری دانش خبره در منطق به آن رسید، سپس اینکه مدل‌های احتمالاتی جهان ابزار اصلی خواهند بود، و تازه‌ترین روایت اینکه یادگیری ماشین مدل‌هایی برمی‌انگیزد که ممکن است اصلاً بر هیچ نظریهٔ خوش‌فهمی مبتنی نباشند. آینده آشکار خواهد کرد چه مدلی در پی می‌آید.

امروز هوش مصنوعی چه کاری می‌تواند بکند؟ شاید به اندازهٔ آنچه برخی مقاله‌های رسانه‌ایِ خوش‌بینانه باورتان می‌کنند نباشد، اما هنوز بسیار زیاد است. اینک چند نمونه:

- **وسایل نقلیهٔ رباتیک:** تاریخچهٔ وسایل نقلیهٔ رباتیک به خودروهای کنترل رادیویی دههٔ ۱۹۲۰ بازمی‌گردد، اما نخستین نمایش‌های رانندگی جاده‌ای خودمختار بدون راهنمای ویژه در دههٔ ۱۹۸۰ رخ داد (کانade و همکاران، ۱۹۸۶؛ دیکمنز و زاپ، ۱۹۸۷). پس از نمایش‌های موفق رانندگی در جاده‌های خاکی در «چالش بزرگ دارپا» در ۲۰۰۵ به طول ۱۳۲ مایل (ترون، ۲۰۰۶) و در خیابان‌های دارای ترافیک در «چالش شهری» ۲۰۰۷، رقابت توسعهٔ خودروهای خودران جدی آغاز شد. در ۲۰۱۸، خودروهای آزمایشی ویمو (Waymo) از مرز ۱۰ میلیون مایل رانندگی در جاده‌های عمومی بدون تصادف جدی گذشتند، به‌گونه‌ای که رانندهٔ انسانی فقط هر ۶,۰۰۰ مایل یک بار برای تحویل کنترل وارد عمل می‌شد. اندکی پس از آن، شرکت سرویس تجاری تاکسی رباتیک عرضه کرد.
- **در آسمان:** پهپادهای بال‌ثابت خودمختار از ۲۰۱۶ در رواندا تحویل خون میان مناطق را انجام می‌دهند. کوادکوپترها مانورهای آکروباتیک شگفت‌انگیزی انجام می‌دهند، ساختمان‌ها را هنگام ساخت نقشه‌های سه‌بعدی کاوش می‌کنند و خود را در آرایش‌های خودمختار سازمان‌دهی می‌کنند.

- **حرکت چهارپا:** بیگ‌داگ (BigDog)، ربات چهارپای رایبرت و همکاران (۲۰۰۸)، تصور ما از حرکت ربات‌ها را دگرگون کرد – دیگر آن گام‌های آهسته، خشک‌پا و موربِ ربات‌های فیلم‌های هالیوود نیست، بلکه چیزی است که به‌شدت به جانور شباهت دارد و می‌تواند پس از هُل‌داده‌شدن یا لیز خوردن روی گودال یخی، خود را بازیابد. اطلس (Atlas)، ربات انسان‌نما، نه فقط روی زمین ناهموار راه می‌رود، بلکه روی جعبه‌ها می‌پرد و پشتک می‌زند (اکرمن و گیتزو، ۲۰۱۶).
- **برنامه‌ریزی و زمان‌بندی خودمختار:** در فاصلهٔ صد میلیون مایلی زمین، برنامهٔ «عامل از راه دور» ناسا نخستین برنامهٔ برنامه‌ریزی خودمختارِ روی‌هواپیما شد که زمان‌بندی عملیات فضاپیما را کنترل می‌کرد (جونسون و همکاران، ۲۰۰۰). عامل از راه دور از هدف‌های سطح‌بالای مشخص‌شده از زمین برنامه تولید می‌کرد و بر اجرای آن برنامه‌ها نظارت داشت – و مشکلات را هنگام وقوع تشخیص، علت‌یابی و بازیابی می‌کرد. امروز جعبه‌ابزار برنامه‌ریزی یوروپا (باریرو و همکاران، ۲۰۱۲) برای عملیات روزانهٔ مریخ‌نوردهای ناسا به کار می‌رود و سامانهٔ سکستنت (وینترنیتز، ۲۰۱۷) ناوبری خودمختار در فضای ژرف، فراتر از سامانهٔ جهانی موقعیت‌یاب (GPS) را ممکن می‌کند.
- در جریان بحران خلیج فارس در ۱۹۹۱، نیروهای آمریکایی ابزار «تحلیل و برنامه‌ریزی مجدد پویا» (DART) را به کار گرفتند (کراس و واکر، ۱۹۹۴) تا برنامه‌ریزی و زمان‌بندی لجستیکی خودکار حمل‌ونقل انجام دهند. این شامل هم‌زمان حداکثر ۵۰،۰۰۰ وسیلهٔ نقلیه، محموله و نفر بود و باید نقطه‌های شروع، مقصدها، مسیرها، ظرفیت‌های حمل‌ونقل، ظرفیت بندرها و فرودگاه‌ها و حل تعارض میان همهٔ پارامترها را در نظر می‌گرفت. آژانس پروژه‌های پژوهشی پیشرفتهٔ دفاعی (دارپا) اعلام کرد همین یک کاربرد، بیش از سرمایه‌گذاری ۳۰ سالهٔ دارپا در هوش مصنوعی را بازپرداخت کرد.
- هر روز، شرکت‌های مسافرتی آنلاین مانند اوبر و سرویس‌های نقشه مانند گوگل‌مپس برای صدها میلیون کاربر مسیر رانندگی ارائه می‌دهند و با در نظر گرفتن وضعیت ترافیک کنونی و پیش‌بینی‌شده، مسیر بهینه را به‌سرعت ترسیم می‌کنند.
- **ترجمهٔ ماشینی:** سامانه‌های ترجمهٔ ماشینی آنلاین اکنون خواندن اسناد به بیش از ۱۰۰ زبان را ممکن می‌کنند، از جمله زبان‌های مادری بیش از ۹۹٪ انسان‌ها، و روزانه صدها میلیارد واژه را برای صدها میلیون کاربر تولید می‌کنند. هرچند بی‌نقص نیستند، عموماً برای فهم کافی‌اند. برای زبان‌های نزدیک‌به‌هم با دادهٔ آموزشی فراوان (مانند فرانسوی و انگلیسی)، ترجمه‌ها در حوزهٔ باریک نزدیک به سطح انسان‌اند (وو و همکاران، ۲۰۱۶).
- **تشخیص گفتار:** در ۲۰۱۷، مایکروسافت نشان داد سامانهٔ تشخیص گفتار مکالمه‌ای‌اش به نرخ خطای واژهٔ ۵٫۱٪ رسیده و در تکلیف سوییچ‌بورد – رونویسی مکالمات تلفنی – با عملکرد انسانی برابری می‌کند (شیونگ و همکاران، ۲۰۱۷). حدود یک‌سوم تعامل با رایانه در سراسر جهان اکنون با صدا انجام می‌شود نه صفحه‌کلید؛ اسکایپ ترجمهٔ بلادرنگ گفتاربه‌گفتار در ده زبان ارائه می‌دهد. الکسا، سیری، کورتانا و گوگل دستیارهایی ارائه می‌دهند که می‌توانند به پرسش‌ها پاسخ دهند و کارها را برای کاربر انجام دهند؛ برای نمونه، سرویس «دوپلکس» گوگل از تشخیص گفتار و سنتز گفتار برای رزرو میز رستوران برای کاربران استفاده می‌کند و به نمایندگی از آنان مکالمه‌ای روان انجام می‌دهد.
- **پیشنهادها:** شرکت‌هایی مانند آمازون، فیسبوک، نتفلیکس، اسپاتیفای، یوتیوب، والمارت و دیگران از یادگیری ماشین برای پیشنهاد آنچه ممکن است دوست داشته باشید بر اساس تجربه‌های گذشتهٔ خودتان و دیگرانی که شبیه شما هستند استفاده می‌کنند. حوزهٔ سامانه‌های پیشنهادگر تاریخچهٔ بلندی دارد (رزنیک و واریان، ۱۹۹۷)

اما به‌سرعت در حال تغییر است، به‌سبب روش‌های تازهٔ یادگیری عمیق که محتوا (متن، موسیقی، ویدئو) را نیز به‌همراه تاریخچه و فراداده تحلیل می‌کنند (فان دن اورد و همکاران، ۲۰۱۴؛ ژانگ و همکاران، ۲۰۱۷). پالایش هرزنامه را نیز می‌توان گونه‌ای پیشنهاد (یا ضدپیشنهاد) دانست؛ فنون کنونی هوش مصنوعی بیش از ۹۹٫۹٪ هرزنامه‌ها را پالایش می‌کنند و سرویس‌های ایمیل می‌توانند گیرنده‌های بالقوه و نیز متن پاسخ احتمالی را پیشنهاد کنند.

- **بازی‌ها:** وقتی دیپ‌بلو در ۱۹۹۷ گری کاسپاروف، قهرمان شطرنج جهان، را شکست داد، مدافعان برتری انسان امیدشان را بر گو گذاشتند. پیت هوت، اخترفیزیک‌دان و هوادار گو، پیش‌بینی کرد «صد سال طول می‌کشد تا رایانه‌ای انسان را در گو ببرد — شاید هم بیشتر.» اما فقط ۲۰ سال بعد، آلفاگو از همهٔ بازیکنان انسانی فراتر رفت (سیلور و همکاران، ۲۰۱۷). که جی، قهرمان جهان، گفت: «سال پیش هنوز نسبتاً انسان‌گونه بازی می‌کرد. اما امسال مانند خدای گو شد.» آلفاگو از مطالعهٔ صدها هزار بازی گذشتهٔ بازیکنان انسانی گو و دانش تقطیرشدهٔ بازیکنان خبرهٔ گویی که در تیم کار می‌کردند بهره برد. برنامهٔ دنباله‌دار، آلفازیرو (AlphaZero)، هیچ ورودی‌ای از انسان دریافت نکرد (به‌جز قواعد بازی) و توانست تنها با بازیِ خودآموز یاد بگیرد که در گو، شطرنج و شوگی همهٔ حریفان، انسانی و ماشینی، را شکست دهد (سیلور و همکاران، ۲۰۱۸). در همین حال، قهرمانان انسانی در بازی‌های گوناگونی از جئوپاردی! (فروچی و همکاران، ۲۰۱۰) و پوکر (بولینگ و همکاران، ۲۰۱۵؛ موراوچیک و همکاران، ۲۰۱۷؛ براون و سندهولم، ۲۰۱۹) تا بازی‌های ویدئویی دوتا ۲ (فرناندز و مالمان، ۲۰۱۸)، استارکرفت ۲ (وینیالز و همکاران، ۲۰۱۹) و کوئیک ۳ (جادربرگ و همکاران، ۲۰۱۹) شکست خورده‌اند.
- **فهم تصویر:** پژوهشگران بینایی رایانه به بیشینهٔ دقت انسانی در تکلیف چالش‌برانگیز تشخیص شیء ایمج‌نت قانع نماندند و مسئلهٔ دشوارتر شرح‌نویسی تصویر (Image Captioning) را بر عهده گرفتند. نمونه‌های چشمگیری عبارت‌اند از: «فردی در حال موتورسواری روی جادهٔ خاکی»، «دو پیتزا روی اجاق بالای فر» و «گروهی از جوانان در حال بازی فریزبی» (وینیالز و همکاران، ۲۰۱۷). اما سامانه‌های کنونی به‌هیچ‌وجه بی‌نقص نیستند: «یخچالی پر از غذا و نوشیدنی» در واقع تابلوی ممنوعیت پارک است که تا حدی با برچسب‌های کوچک فراوان پوشیده شده.
- **پزشکی:** الگوریتم‌های هوش مصنوعی اکنون در تشخیص بسیاری از بیماری‌ها با پزشکان متخصص برابری می‌کنند یا از آنان فراتر می‌روند، به‌ویژه وقتی تشخیص بر تصویر مبتنی باشد. نمونه‌ها شامل آلزایمر (دینگ و همکاران، ۲۰۱۸)، سرطان متاستاتیک (لیو و همکاران، ۲۰۱۷؛ استوا و همکاران، ۲۰۱۷)، بیماری چشمی (گولشان و همکاران، ۲۰۱۶) و بیماری‌های پوستی (لیو و همکاران، ۲۰۱۹) است. مرور نظام‌مند و فراتحلیل (لیو و همکاران، ۲۰۱۹) نشان داد عملکرد برنامه‌های هوش مصنوعی به‌طور میانگین با متخصصان مراقبت بهداشتی برابری می‌کند. یکی از تأکیدهای کنونی هوش مصنوعی پزشکی، تسهیل مشارکت انسان–ماشین است. برای نمونه، سامانهٔ لینا (LYNA) در تشخیص سرطان پستان متاستاتیک به دقت کلی ۹۹٫۶٪ می‌رسد — بهتر از متخصص انسانیِ بدون کمک — اما ترکیب این دو بهتر هم عمل می‌کند (لیو و همکاران، ۲۰۱۸؛ اشتاینر و همکاران، ۲۰۱۸).
- پذیرش گستردهٔ این فنون اکنون نه با دقت تشخیص، بلکه با نیاز به نشان‌دادن بهبود در پیامدهای بالینی و تضمین شفافیت، نبود سوگیری و حریم خصوصی داده محدود شده است (توپول، ۲۰۱۹). در ۲۰۱۷ فقط دو کاربرد هوش مصنوعی پزشکی توسط سازمان غذا و داروی آمریکا (FDA) تأیید شد، اما این شمار در ۲۰۱۸ به ۱۲ رسید و همچنان در حال افزایش است.

- **علم اقلیم:** تیمی از دانشمندان جایزهٔ گوردون بل ۲۰۱۸ را برای مدل یادگیری عمیقی بردند که اطلاعات دقیقی دربارهٔ رویدادهای آب‌وهوایی شدید کشف می‌کند که پیش‌تر در داده‌های اقلیمی مدفون بود. آنان از ابررایانه‌ای با سخت‌افزار GPU تخصصی برای عبور از سطح اگزااپ (10^{18} عملیات در ثانیه) استفاده کردند؛ نخستین برنامهٔ یادگیری ماشین که چنین کرد (کورت و همکاران، ۲۰۱۸). رولنیک و همکاران (۲۰۱۹) فهرستی ۶۰ صفحه‌ای از راه‌های مقابله با تغییر اقلیم با یادگیری ماشین ارائه می‌دهند.

این‌ها فقط چند نمونه از سامانه‌های هوش مصنوعی موجود امروزند. نه جادو و نه علم‌تخیلی – بلکه علم، مهندسی و ریاضیات که این کتاب مقدمه‌ای بر آن است.

۱.۵ ریسک‌ها و منافع هوش مصنوعی

فرانسیس بیکن، فیلسوفی که به ایجاد روش علمی منسوب است، در «خرد کهن» (۱۶۰۹) یادآور شد «هنرهای مکانیکی کاربردی دوپهلو دارند، به همان اندازه برای آسیب و برای درمان خدمت می‌کنند.» حال که هوش مصنوعی نقشی روزافزون در حوزه‌های اقتصادی، اجتماعی، علمی، پزشکی، مالی و نظامی ایفا می‌کند، سزاوار است آسیب‌ها و درمان‌ها – به زبان امروز، ریسک‌ها و منافع – را که می‌تواند به بار آورد در نظر بگیریم. موضوع‌های خلاصه‌شده در این‌جا در فصل‌های ۲۷ و ۲۸ با ژرفای بیشتری پوشش داده شده‌اند.

برای آغاز با منافع: به بیان ساده، تمام تمدن ما محصول هوش انسانی ماست. اگر به هوش ماشینیِ به‌مراتب بزرگ‌تر دسترسی داشته باشیم، سقف جاه‌طلبی‌هایمان به‌شکل چشمگیری بالا می‌رود. پتانسیل هوش مصنوعی و رباتیک در رهاندن بشریت از کارهای پست و تکراری و افزایش چشمگیر تولید کالا و خدمات می‌تواند نویدبخش دورانی از صلح و فراوانی باشد. ظرفیت شتاب‌دادن به پژوهش علمی می‌تواند به درمان بیماری‌ها و راه‌حل‌هایی برای تغییر اقلیم و کمبود منابع بینجامد. چنان‌که دمیس هسابیس، مدیرعامل گوگل دیپ‌مایند، پیشنهاد کرده: «نخست هوش مصنوعی را حل کن، سپس از هوش مصنوعی برای حل همه‌چیز دیگر استفاده کن.» اما مدت‌ها پیش از آنکه فرصت «حل هوش مصنوعی» داشته باشیم، از سوءاستفاده از هوش مصنوعی – سهوی یا غیر آن – ریسک‌هایی خواهیم دید. برخی از آن‌ها همین‌اکنون آشکارند و برخی دیگر بر پایهٔ روندهای کنونی محتمل می‌نمایند:

- **سلاح‌های خودمختار کشنده:** سازمان ملل این‌ها را سلاح‌هایی تعریف می‌کند که می‌توانند اهداف انسانی را بدون مداخلهٔ انسانی مکان‌یابی، انتخاب و حذف کنند. نگرانی اصلی دربارهٔ چنین سلاح‌هایی مقیاس‌پذیری آن‌هاست: نبودِ نیاز به نظارت انسانی بدان معناست که گروهی کوچک می‌تواند شمار دلخواهی سلاح علیه اهداف انسانیِ تعریف‌شده با هر معیار شناسایی ممکن به کار گیرد. فنون لازم برای سلاح‌های خودمختار شبیه فنون لازم برای خودروهای خودران است. گفت‌وگوهای غیررسمی کارشناسان دربارهٔ ریسک‌های بالقوهٔ سلاح‌های خودمختار کشنده در ۲۰۱۴ در سازمان ملل آغاز شد و در ۲۰۱۷ به مرحلهٔ رسمی پیش از معاهدهٔ «گروه کارشناسان دولتی» منتقل شد.
- **نظارت و اقناع:** در حالی که برای پرسنل امنیتی پایش خطوط تلفن، خوراک دوربین‌های ویدئویی، ایمیل‌ها و دیگر کانال‌های پیام‌رسان پرهزینه، خسته‌کننده و گاه از نظر قانونی مشکوک است، هوش مصنوعی (تشخیص گفتار، بینایی رایانه و فهم زبان طبیعی) می‌تواند به‌شیوه‌ای مقیاس‌پذیر برای نظارت انبوه بر افراد و شناسایی

فعالیت‌های مورد نظر به کار رود. با تنظیم جریان‌های اطلاعاتیِ رسانه‌های اجتماعی برای افراد، بر پایهٔ فنون یادگیری ماشین، رفتار سیاسی را تا حدی می‌توان اصلاح و کنترل کرد — نگرانی‌ای که در انتخاباتِ از ۲۰۱۶ به بعد آشکار شد.

- **تصمیم‌گیری سوگیرانه:** سوءاستفادهٔ سهل‌انگارانه یا عمدی از الگوریتم‌های یادگیری ماشین برای تکلیف‌هایی مانند ارزیابی درخواست‌های عفو مشروط و وام می‌تواند به تصمیم‌هایی بینجامد که بر اساس نژاد، جنسیت یا دیگر رده‌های حمایت‌شده سوگیرند. اغلب خودِ داده‌ها بازتاب‌دهندهٔ سوگیری فراگیر در جامعه‌اند.
- **تأثیر بر اشتغال:** نگرانی دربارهٔ حذف شغل‌ها توسط ماشین‌ها قرن‌ها قدمت دارد. داستان هرگز ساده نیست: ماشین‌ها برخی از تکلیف‌هایی را انجام می‌دهند که انسان‌ها ممکن بود انجام دهند، اما انسان‌ها را نیز پربازده‌تر و بنابراین قابل‌استخدام‌تر می‌کنند، و شرکت‌ها را سودآورتر می‌کنند و بنابراین قادر به پرداخت دستمزدهای بالاتر. آنان ممکن است برخی فعالیت‌ها را از نظر اقتصادی به‌صرفه کنند که در غیر این صورت غیرعملی بودند. کاربردشان عموماً به افزایش ثروت می‌انجامد اما تمایل دارد ثروت را از نیروی کار به سرمایه منتقل کند و افزایش نابرابری را بیشتر تشدید کند. پیشرفت‌های فناوری پیشین — مانند اختراع ماشین‌های بافندگی مکانیکی — به اختلالات جدی در اشتغال انجامیده، اما سرانجام مردم گونه‌های تازه‌ای از کار می‌یابند. از سوی دیگر، ممکن است هوش مصنوعی نیز همان گونه‌های تازهٔ کار را انجام دهد. این موضوع به‌سرعت به کانون اصلی توجه اقتصاددانان و دولت‌های سراسر جهان بدل می‌شود.
- **کاربردهای ایمنی‌بحرانی:** با پیشرفت فنون هوش مصنوعی، کاربرد آن‌ها در کاربردهای پرخطر و ایمنی‌بحرانی مانند رانندگی خودرو و مدیریت منابع آب شهرها افزایش می‌یابد. تصادف‌های مرگبار تاکنون رخ داده‌اند و دشواری تأیید صوری و تحلیل ریسک آماری را برای سامانه‌های توسعه‌یافته با فنون یادگیری ماشین برجسته می‌کنند. رشتهٔ هوش مصنوعی باید استانداردهای فنی و اخلاقی دست‌کم قابل مقایسه با آنچه در دیگر رشته‌های مهندسی و مراقبت بهداشتی رایج است — جایی که جان انسان‌ها در میان است — توسعه دهد.
- **امنیت سایبری:** فنون هوش مصنوعی در دفاع در برابر حملهٔ سایبری مفیدند، برای نمونه با شناسایی الگوهای رفتار غیرعادی، اما به قدرت، بقاپذیری و قابلیت انتشار بدافزار نیز کمک خواهند کرد. برای نمونه، روش‌های یادگیری تقویتی برای ساخت ابزارهای بسیار مؤثر باج‌گیری و فیشینگ خودکار و شخصی‌سازی‌شده به کار رفته‌اند.

در بخش ۲۷.۳ به این موضوع‌ها با ژرفای بیشتری بازخواهیم گشت. با تواناتر شدن سامانه‌های هوش مصنوعی، نقش‌های اجتماعی بیشتری را که پیش‌تر انسان‌ها ایفا می‌کردند بر عهده خواهند گرفت. همان‌گونه که انسان‌ها در گذشته از این نقش‌ها برای بدرفتاری استفاده کرده‌اند، می‌توان انتظار داشت انسان‌ها سامانه‌های هوش مصنوعی را در این نقش‌ها برای بدرفتاریِ بیشتر هم سوءاستفاده کنند. همهٔ نمونه‌های بالا به اهمیت حکمرانی و سرانجام مقررات‌گذاری اشاره می‌کنند. اکنون جامعهٔ پژوهشی و شرکت‌های بزرگِ درگیر در پژوهش هوش مصنوعی اصول خودحکمرانی داوطلبانه برای فعالیت‌های مرتبط با هوش مصنوعی توسعه داده‌اند (بخش ۲۷.۳ را ببینید). دولت‌ها و سازمان‌های بین‌المللی نهادهای مشورتی برپا می‌کنند تا مقررات مناسب برای هر مورد کاربرد خاص تدوین کنند، برای تأثیرات اقتصادی و اجتماعی آماده شوند و از توانایی‌های هوش مصنوعی برای پرداختن به مسائل بزرگ اجتماعی بهره بگیرند.

و دربارهٔ بلندمدت چه؟ آیا به هدف دیرینه خواهیم رسید: آفرینش هوشی هم‌سطح یا تواناتر از هوش انسانی؟ و اگر رسیدیم، آنگاه چه؟

در بخش بزرگی از تاریخ هوش مصنوعی، این پرسش‌ها زیر سایهٔ تلاش روزمره برای واداشتن سامانه‌های هوش مصنوعی به انجام کاری حتی به‌طور مبهم هوشمندانه مانده‌اند. مانند هر رشتهٔ گسترده‌ای، اکثریت قاطع پژوهشگران هوش مصنوعی در زیرشاخه‌ای خاص تخصص یافته‌اند — مانند بازینویسی، بازنمایی دانش، بینایی یا فهم زبان طبیعی — اغلب با این فرض که پیشرفت در این زیرشاخه‌ها به هدف‌های گسترده‌تر هوش مصنوعی کمک خواهد کرد. نیلز نیلسون (۱۹۹۵)، یکی از رهبران آغازین پروژهٔ شیکی در SRI، آن هدف‌های گسترده‌تر را به رشته یادآوری کرد و هشدار داد زیرشاخه‌ها در خطر تبدیل‌شدن به هدفِ فی‌نفسه‌اند.

بعدها، برخی از بنیان‌گذاران تأثیرگذار هوش مصنوعی، از جمله جان مک‌کارتی (۲۰۰۷)، ماروین مینسکی (۲۰۰۷) و پاتریک وینستون (بیل و وینستون، ۲۰۰۹)، با هشدارهای نیلسون هم‌داستان شدند و پیشنهاد کردند به‌جای تمرکز بر عملکرد قابل‌اندازه‌گیری در کاربردهای خاص، هوش مصنوعی باید به ریشه‌هایش بازگردد: تلاش برای، به گفتهٔ هربرت سایمون، «ماشین‌هایی که می‌اندیشند، یاد می‌گیرند و می‌آفرینند». آنان این تلاش را هوش مصنوعی هم‌سطح انسان (Human-Level AI) یا HLAI نامیدند — ماشینی که باید بتواند بیاموزد هر کاری را که انسان می‌تواند انجام دهد. نخستین همایش‌شان در ۲۰۰۴ برگزار شد (مینسکی و همکاران، ۲۰۰۴). تلاش دیگری با هدف‌های مشابه، جنبش هوش عمومی مصنوعی (Artificial General Intelligence، یا AGI) (گورتزل و پناچین، ۲۰۰۷)، نخستین کنفرانس خود را برگزار کرد و در ۲۰۰۸ ژورنال هوش عمومی مصنوعی را بنیان نهاد.

تقریباً در همان زمان، نگرانی‌هایی برانگیخته شد که آفرینش هوش مصنوعی ابرانسانی (Artificial Superintelligence، یا ASI) — هوشی که بسیار فراتر از توانایی انسانی است — ممکن است ایدهٔ بدی باشد (یودکوفسکی، ۲۰۰۸؛ اموهوندرا، ۲۰۰۸). خود تورینگ (۱۹۹۶) در سخنرانی‌ای در منچستر در ۱۹۵۱ همین نکته را بیان کرد و از ایده‌های پیشین ساموئل باتلر (۱۸۶۳) بهره گرفت:[15]

> *محتمل می‌نماید یک بار که روش اندیشیدن ماشینی آغاز شد، زمان چندانی نبرد تا از توان‌های ضعیف ما فراتر رود... پس در مرحله‌ای باید انتظار داشته باشیم ماشین‌ها کنترل را به دست بگیرند، به آن روشی که در «ارهون» ساموئل باتلر از آن یاد شده.*

این نگرانی‌ها با پیشرفت‌های اخیر یادگیری عمیق، انتشار کتاب‌هایی مانند «ابرهوش» نوشتهٔ نیک بوستروم (۲۰۱۴)، و اظهارنظرهای عمومی استیون هاوکینگ، بیل گیتس، مارتین ریس و ایلان ماسک، فقط فراگیرتر شده‌اند.

تجربهٔ احساس ناراحتی کلی از ایدهٔ آفرینش ماشین‌های ابرهوشمند فقط طبیعی است. شاید بتوانیم آن را «مسئلهٔ گوریل» (Gorilla Problem) بنامیم: حدود هفت میلیون سال پیش، نخستی‌سانی که اکنون منقرض شده تکامل یافت و یک شاخه‌اش به گوریل‌ها و دیگری به انسان‌ها انجامید. امروز گوریل‌ها چندان از شاخهٔ انسانی خشنود نیستند؛ آنان اساساً هیچ کنترلی بر آینده‌شان ندارند. اگر این نتیجهٔ موفقیت در آفرینش هوش مصنوعی ابرانسانی باشد — اینکه

انسان‌ها کنترل آینده‌شان را واگذارند — پس شاید باید کار روی هوش مصنوعی را متوقف کنیم و، به‌عنوان نتیجهٔ ضمنی، از منافع احتمالی‌اش هم دست بکشیم. این جوهر هشدار تورینگ است: بدیهی نیست بتوانیم ماشین‌هایی را کنترل کنیم که از ما هوشمندترند.

اگر هوش مصنوعی ابرانسانی جعبهٔ سیاهی بود که از فضای بیرونی آمده بود، واقعاً خردمندانه بود در گشودن جعبه احتیاط کنیم. اما چنین نیست: ما سامانه‌های هوش مصنوعی را طراحی می‌کنیم، پس اگر سرانجام «کنترل را به دست بگیرند»، چنان‌که تورینگ پیشنهاد می‌کند، نتیجهٔ شکست طراحی خواهد بود.

برای پرهیز از چنین پیامدی، باید منشأ شکست بالقوه را بفهمیم. نوربرت وینر (۱۹۶۰)، که پس از دیدن برنامهٔ بازی چکرزِ آرتور ساموئل که یاد گرفت خالقش را ببرد، به اندیشیدن دربارهٔ آیندهٔ بلندمدت هوش مصنوعی برانگیخته شد، چنین گفت:

> *اگر برای رسیدن به مقاصدمان از عاملی مکانیکی استفاده کنیم که نتوانیم به‌شکل مؤثر در عملکردش دخالت کنیم... بهتر است کاملاً مطمئن باشیم هدفی که در ماشین گذاشته شده، هدفی است که واقعاً خواستاریم.*

بسیاری از فرهنگ‌ها اسطوره‌های انسان‌هایی دارند که از خدایان، جن‌ها، جادوگران یا شیاطین چیزی می‌خواهند. در این داستان‌ها، بی‌استثنا، آنان دقیقاً همان چیزی را می‌گیرند که به‌طور لفظی خواسته‌اند و سپس پشیمان می‌شوند. آرزوی سوم، اگر باشد، بازگرداندن دو آرزوی اول است. ما این را «مسئلهٔ شاه ماداس» (King Midas Problem) می‌نامیم: ماداس، پادشاه افسانه‌ای اساطیر یونان، خواست هر چه لمس کند طلا شود، اما پس از لمس‌کردنِ غذایش، نوشیدنی‌اش و اعضای خانواده‌اش پشیمان شد.[16] در بخش ۱.۱.۵ به این موضوع اشاره کردیم و به نیاز به اصلاح چشمگیر الگوی استانداردِ گذاشتن هدف‌های ثابت در ماشین اشاره کردیم. راه‌حلِ مخمصهٔ وینر اصلاً نداشتنِ «هدفِ قطعیِ گذاشته‌شده در ماشین» نیست. در عوض، ماشین‌هایی می‌خواهیم که برای دستیابی به هدف‌های انسانی تلاش کنند اما بدانند که دقیقاً از چیستی آن هدف‌ها مطمئن نیستند.

شاید تأسف‌بار باشد که تقریباً تمام پژوهش‌های هوش مصنوعی تاکنون در چارچوب الگوی استاندارد انجام شده‌اند، یعنی تقریباً تمام مطالب فنی این ویرایش آن چارچوب فکری را بازتاب می‌دهد. با این حال، نتایج اولیه‌ای در چارچوب تازه وجود دارد. در فصل ۱۶ نشان می‌دهیم ماشین انگیزهٔ مثبتی دارد که اجازه دهد خاموش شود اگر و فقط اگر دربارهٔ هدف انسانی نامطمئن باشد. در فصل ۱۸، بازی‌های کمکی (Assistance Games) را صورت‌بندی و مطالعه می‌کنیم که وضعیتی را به‌شکل ریاضی توصیف می‌کنند که در آن انسان هدفی دارد و ماشین می‌کوشد به آن برسد، اما در آغاز دربارهٔ آن نامطمئن است. در فصل ۲۲، روش‌های یادگیری تقویتی معکوس (Inverse Reinforcement Learning) را توضیح می‌دهیم که به ماشین‌ها امکان می‌دهد از مشاهدهٔ انتخاب‌های انسان، دربارهٔ ترجیحات انسانی بیشتر بیاموزند. در فصل ۲۷، دو دشواری اصلی را می‌کاویم: نخست، اینکه انتخاب‌های ما از طریق معماری شناختی بسیار پیچیده‌ای به ترجیحات‌مان وابسته‌اند که وارونه‌کردنش دشوار است؛ و دوم، اینکه ما انسان‌ها ممکن است اصلاً ترجیحات سازگاری نداشته باشیم — نه فرداً و نه گروهی — پس ممکن است روشن نباشد سامانه‌های هوش مصنوعی باید چه کاری برای ما انجام دهند.

خلاصهٔ فصل

این فصل هوش مصنوعی را تعریف می‌کند و زمینهٔ فرهنگی‌ای را که رشته در آن رشد کرده برقرار می‌کند. برخی نکته‌های مهم چنین‌اند:

- افراد گوناگون با هدف‌های متفاوتی به هوش مصنوعی نزدیک می‌شوند. دو پرسش مهم که باید پرسید: آیا به اندیشیدن اهمیت می‌دهید یا رفتار؟ آیا می‌خواهید انسان‌ها را مدل کنید یا بکوشید به نتایج بهینه برسید؟
- بر اساس آنچه الگوی استاندارد نامیدیم، هوش مصنوعی عمدتاً به کنش عقلایی مربوط است. عامل هوشمندِ ایدئال در هر موقعیت بهترین کنش ممکن را انجام می‌دهد. ما مسئلهٔ ساختن عامل‌هایی را مطالعه می‌کنیم که به این معنا هوشمندند.
- دو اصلاح برای این ایدهٔ ساده لازم است: نخست، توانایی هر عامل، انسانی یا نه، در انتخاب کنش‌های عقلایی با ناشدنی‌بودن محاسباتیِ چنین کاری محدود شده است؛ دوم، مفهوم ماشینی که هدف قطعی‌ای را دنبال می‌کند باید با ماشینی جایگزین شود که هدف‌هایی را دنبال می‌کند تا به انسان سود برساند، اما دربارهٔ چیستی آن هدف‌ها نامطمئن است.
- فیلسوفان (از ۴۰۰ پیش از میلاد) هوش مصنوعی را ممکن کردند با پیشنهاد اینکه ذهن از جهاتی مانند ماشین است، بر دانشِ رمزگذاری‌شده در زبانی درونی عمل می‌کند، و اندیشه را می‌توان برای انتخاب کنش‌ها به کار برد.
- ریاضی‌دانان ابزارهای دست‌کاری گزاره‌های با قطعیت منطقی و نیز گزاره‌های نامطمئن احتمالاتی را فراهم کردند. آنان همچنین زمینهٔ فهم محاسبه و استدلال دربارهٔ الگوریتم‌ها را پایه نهادند.
- اقتصاددانان مسئلهٔ تصمیم‌گیری‌هایی را که مطلوبیت مورد انتظار تصمیم‌گیرنده را بیشینه می‌کنند صورت‌بندی کردند.
- عصب‌شناسان واقعیت‌هایی دربارهٔ چگونگی کارکرد مغز و راه‌های شباهت و تفاوتش با رایانه‌ها کشف کردند.
- روان‌شناسان این ایده را پذیرفتند که انسان‌ها و جانوران را می‌توان ماشین‌های پردازش اطلاعات دانست. زبان‌شناسان نشان دادند کاربرد زبان در این مدل جای می‌گیرد.
- مهندسان رایانه ماشین‌های روزافزون قدرتمند فراهم کردند که کاربردهای هوش مصنوعی را ممکن می‌کنند و مهندسان نرم‌افزار آن‌ها را کاربردی‌تر کردند.
- نظریهٔ کنترل با طراحی دستگاه‌هایی سروکار دارد که بر پایهٔ بازخورد محیط بهینه عمل می‌کنند. در آغاز، ابزارهای ریاضی نظریهٔ کنترل کاملاً متفاوت از ابزارهای به‌کاررفته در هوش مصنوعی بودند، اما این رشته‌ها به هم نزدیک می‌شوند.
- تاریخ هوش مصنوعی چرخه‌هایی از موفقیت، خوش‌بینی نابه‌جا و کاهش ناشی از آن در اشتیاق و بودجه داشته است. همچنین چرخه‌هایی از معرفی رویکردهای خلاقانهٔ تازه و اصلاح نظام‌مند بهترین‌ها وجود داشته است.

- هوش مصنوعی نسبت به دهه‌های آغازینش از نظر نظری و روش‌شناختی بسیار پخته‌تر شده است. با پیچیده‌تر شدن مسائلی که هوش مصنوعی با آن‌ها سروکار دارد، این رشته از منطق بولی به استدلال احتمالاتی و از دانش دست‌ساخته به یادگیری ماشین از داده حرکت کرد. این به بهبود توانایی‌های سامانه‌های واقعی و یکپارچگی بیشتر با دیگر رشته‌ها انجامیده است.
- با یافتن کاربرد در جهان واقعی، لازم شده گسترهٔ وسیعی از ریسک‌ها و پیامدهای اخلاقی در نظر گرفته شود.
- در بلندمدت، با مسئلهٔ دشوار کنترل سامانه‌های هوش مصنوعی ابرهوشمند روبه‌روییم که ممکن است به شیوه‌های پیش‌بینی‌ناپذیر تکامل یابند. حل این مسئله به نظر می‌رسد تغییری در برداشت ما از هوش مصنوعی را ضروری می‌سازد.

یادداشت‌های کتاب‌شناختی و تاریخی

تاریخچهٔ جامع هوش مصنوعی توسط نیلز نیلسون (۲۰۰۹)، یکی از پیشگامان آغازین این رشته، ارائه شده است. پدرو دومینگوس (۲۰۱۵) و ملانی میچل (۲۰۱۹) مرورهایی از یادگیری ماشین برای مخاطب عام ارائه می‌دهند و کای-فو لی (۲۰۱۸) رقابت برای رهبری بین‌المللی در هوش مصنوعی را توصیف می‌کند. مارتین فورد (۲۰۱۸) با ۲۳ پژوهشگر برجستهٔ هوش مصنوعی مصاحبه کرده است.

انجمن‌های حرفه‌ای اصلی هوش مصنوعی عبارت‌اند از: انجمن پیشبرد هوش مصنوعی (AAAI)، گروه علاقه‌مند ویژهٔ هوش مصنوعیِ ACM (SIGAI، پیش‌تر SIGART)، انجمن اروپایی هوش مصنوعی، و انجمن هوش مصنوعی و شبیه‌سازی رفتار (AISB). «مشارکت در هوش مصنوعی» (Partnership on AI) بسیاری از سازمان‌های تجاری و غیرانتفاعیِ دغدغه‌مندِ تأثیرات اخلاقی و اجتماعی هوش مصنوعی را گرد هم می‌آورد.

«مجلهٔ هوش مصنوعی» AAAI حاوی مقاله‌های موضوعی و آموزشی فراوان است و پایگاه وب آن، aaai.org، اخبار، آموزش‌ها و اطلاعات زمینه دارد.

تازه‌ترین کارها در مجموعه‌مقاله‌های کنفرانس‌های اصلی هوش مصنوعی ظاهر می‌شود: کنفرانس مشترک بین‌المللی هوش مصنوعی (IJCAI)، کنفرانس سالانهٔ اروپایی هوش مصنوعی (ECAI)، و کنفرانس AAAI. یادگیری ماشین در کنفرانس بین‌المللی یادگیری ماشین (ICML) و نشست پردازش اطلاعات عصبی (NeurIPS) پوشش داده می‌شود.

ژورنال‌های اصلی هوش مصنوعی عمومی عبارت‌اند از: «هوش مصنوعی»، «هوش محاسباتی»، «معاملات IEEE در تحلیل الگو و هوش ماشینی»، «سامانه‌های هوشمند IEEE» و «ژورنال پژوهش هوش مصنوعی». همچنین کنفرانس‌ها و ژورنال‌های فراوانی به حوزه‌های خاص اختصاص دارند که در فصل‌های مناسب پوشش‌شان می‌دهیم.

پانوشت‌ها:

[3] همان حربه‌هایی که در عمل، حتی در مسابقات واقعی شطرنج رایانه‌ای نیز بحث شده‌اند.

[4] «ارغنون تازه» (Novum Organum) به‌روزرسانی «ارغنون» ارسطو، یعنی ابزار اندیشه، است.

[5] نشانه‌گذاری پیشنهادی فرگه برای منطق مرتبهٔ اول — ترکیبی غریب از ویژگی‌های متنی و هندسی — هرگز رایج نشد.

[6] از آن پس کشف شده که تپایا (شترموش) و برخی گونه‌های پرنده از نسبت مغز به بدن انسان فراتر می‌روند.

[7] بسیاری الکساندر هود (۱۸۲۴) را منبع پیشین احتمالی این کشف می‌دانند.

[8] گولجی بر این باور ماند که کارکردهای مغز عمدتاً در محیطی پیوسته انجام می‌شوند که نورون‌ها در آن جای گرفته‌اند، حال آنکه کاخال «آموزهٔ نورونی» را می‌گسترد. این دو در ۱۹۰۶ جایزهٔ نوبل را مشترکاً گرفتند، اما سخنرانی‌های پذیرششان خصمانه بود.

[9] ماشینی پیچیده به نام یک کاریکاتوریست بریتانیایی که دستگاه‌های عجیب و بغرنج مضحکی برای کارهای روزمره مانند کره مالیدن به نان ترسیم می‌کرد.

[10] در دورهٔ پس از جنگ، تورینگ می‌خواست از این رایانه‌ها برای پژوهش هوش مصنوعی استفاده کند — برای نمونه، طرح کلی نخستین برنامهٔ شطرنج را نوشت (تورینگ و همکاران، ۱۹۵۳) — اما دولت بریتانیا این پژوهش را مسدود کرد.

[11] اکنون دانشگاه کارنگی ملون (CMU).

[12] این نخستین کاربرد رسمی اصطلاح «هوش مصنوعی» مک‌کارتی بود. شاید «عقلانیت محاسباتی» دقیق‌تر و کمتر تهدیدآمیز بود، اما «AI» ماندگار شد. در پنجاه‌مین سالگرد کنفرانس دارتموث، مک‌کارتی گفت در برابر اصطلاح‌های «رایانه» یا «محاسباتی» مقاومت کرده به احترام نوربرت وینر، که دستگاه‌های سایبرنتیک آنالوگ را ترویج می‌کرد نه رایانه‌های دیجیتال.

[13] نیوول و سایمون همچنین زبان پردازش فهرست IPL را برای نوشتن LT اختراع کردند. آنان کامپایلر نداشتند و برنامه را با دست به کد ماشین ترجمه می‌کردند. برای پرهیز از خطا، به‌صورت موازی کار می‌کردند و هنگام نوشتن هر دستور، اعداد دودویی را برای یکدیگر بلند می‌خواندند تا مطمئن شوند موافق‌اند.

[14] برخی این تغییر را پیروزی «مرتب‌ها» (Neats) — کسانی که فکر می‌کنند نظریه‌های هوش مصنوعی باید در دقت ریاضی ریشه داشته باشند — بر «شلخته‌ها» (Scruffies) توصیف کرده‌اند؛ کسانی که ترجیح می‌دهند ایده‌های فراوانی را آزمایش کنند، برنامه‌هایی بنویسند و سپس ارزیابی کنند چه چیزی کار می‌کند. هر دو رویکرد مهم‌اند. چرخش به سوی مرتبی دلالت بر آن دارد که رشته به سطحی از ثبات و پختگی رسیده است. تأکید کنونی بر یادگیری عمیق ممکن است نمایانگر احیای شلخته‌ها باشد.

[15] حتی پیش‌تر، در ۱۸۴۷، ریچارد تورنتون، سردبیر «توضیح‌دهندهٔ اولیه»، با ماشین‌های حساب مکانیکی به تندی مخالفت کرد: «ذهن... از خود پیشی می‌گیرد و با اختراع ماشین‌هایی برای انجام اندیشیدنِ خودش، ضرورت وجود خودش را از میان می‌برد... اما چه کسی می‌داند چنین ماشین‌هایی، وقتی به کمال بیشتر برسند، ممکن است طرحی برای رفع همهٔ نقص‌های خودشان نیندیشند و سپس ایده‌هایی فراتر از قلمرو ذهن فانی تولید نکنند!»

[16] ماداس بهتر بود اصول پایهٔ ایمنی را رعایت می‌کرد و در آرزویش یک دکمهٔ «بازگردانی» و یک دکمهٔ «توقف» می‌گنجاند.

فصل ۲: عامل‌های هوشمند (Intelligent Agents)

در این فصل دربارهٔ سرشت عامل‌ها بحث می‌کنیم؛ عامل‌های کامل یا ناقص، گوناگونی محیط‌ها، و مجموعهٔ جانورانی که از این‌ها پدید می‌آید.

فصل ۱ مفهوم عامل عقلایی را به‌عنوان محور رویکرد ما به هوش مصنوعی شناساند. در این فصل این مفهوم را عینی‌تر می‌کنیم. خواهیم دید مفهوم عقلانیت را می‌توان بر گسترهٔ وسیعی از عامل‌ها اعمال کرد که در هر محیطِ قابل تصوری کار می‌کنند. برنامهٔ ما در این کتاب آن است که از این مفهوم برای تدوین مجموعه‌ای کوچک از اصول طراحی برای ساختن عامل‌های موفق استفاده کنیم — سامانه‌هایی که به‌درستی بتوان آن‌ها را هوشمند نامید.

با بررسی عامل‌ها، محیط‌ها و پیوند میان آن‌ها آغاز می‌کنیم. این مشاهده که برخی عامل‌ها از برخی دیگر بهتر رفتار می‌کنند، به‌طور طبیعی به ایدهٔ عامل عقلایی می‌انجامد — عاملی که تا حد ممکن خوب رفتار می‌کند. اینکه یک عامل چقدر خوب می‌تواند رفتار کند، به سرشت محیط بستگی دارد؛ برخی محیط‌ها از برخی دیگر دشوارترند. رده‌بندی خامی از محیط‌ها ارائه می‌دهیم و نشان می‌دهیم ویژگی‌های یک محیط چگونه بر طراحی عامل‌های مناسب برای آن محیط تأثیر می‌گذارد. شماری از طراحی‌های پایهٔ «اسکلتی» عامل را توصیف می‌کنیم که در بقیهٔ کتاب آن‌ها را کامل می‌کنیم.

۲.۱ عامل‌ها و محیط‌ها

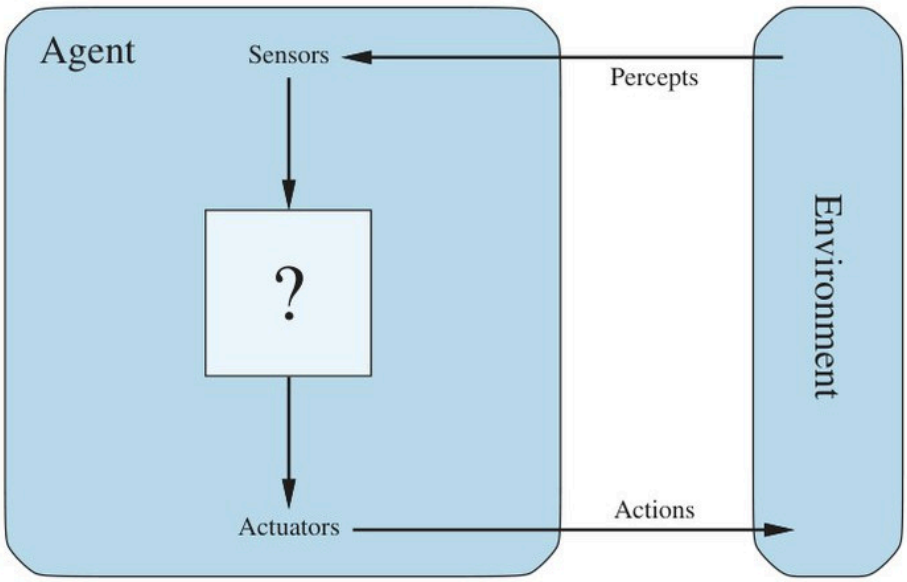
عامل (Agent) هر چیزی است که بتوان آن را چنین دید: محیط خود را از طریق حسگرها (Sensors) ادراک می‌کند و از طریق عملگرها (Actuators) بر آن محیط کنش می‌گذارد. این ایدهٔ ساده در شکل ۲.۱ نشان داده شده است.

عامل انسانی برای حسگرها چشم، گوش و اندام‌های دیگر دارد و برای عملگرها دست، پا، مجرای صوتی و مانند آن. عامل رباتیک ممکن است برای حسگرها دوربین و مسافت‌یاب مادون‌قرمز داشته باشد و برای عملگرها موتورهای گوناگون. عامل نرم‌افزاری محتوای پرونده‌ها، بسته‌های شبکه و ورودی انسانی (صفحه‌کلید/ماوس/صفحهٔ لمسی/صدا) را به‌عنوان ورودی‌های حسی دریافت می‌کند و با نوشتن پرونده‌ها، فرستادن بسته‌های شبکه و نمایش اطلاعات یا تولید صدا بر محیط کنش می‌گذارد. محیط می‌تواند همه‌چیز باشد — تمام جهان هستی! در عمل، محیط فقط آن بخش از جهان است که هنگام طراحی این عامل به حالتش اهمیت می‌دهیم — بخشی که بر آنچه عامل ادراک می‌کند اثر می‌گذارد و از کنش‌های عامل تأثیر می‌پذیرد.

از اصطلاح ادراک حسی (Percept) برای اشاره به محتوایی استفاده می‌کنیم که حسگرهای عامل ادراک می‌کنند. «توالی ادراک حسی» (Percept Sequence) عامل، تاریخچهٔ کاملِ هر آن چیزی است که عامل تاکنون ادراک کرده است. به‌طور کلی، انتخاب کنشِ عامل در هر لحظهٔ معین می‌تواند به دانشِ ازپیش‌ساختهٔ او و به کل توالی ادراک حسیِ

مشاهده‌شده تا آن لحظه وابسته باشد، اما نه به چیزی که ادراک نکرده. با مشخص کردن انتخاب کنش عامل برای هر توالی ادراک حسی ممکن، کم‌وبیش هر آنچه دربارهٔ عامل گفتنی است را گفته‌ایم. به زبان ریاضی، می‌گوییم رفتار عامل با «تابع عامل» (Agent Function) توصیف می‌شود که هر توالی ادراک حسی داده‌شده را به کنشی نگاشت می‌کند:

```
f: P* → A
```

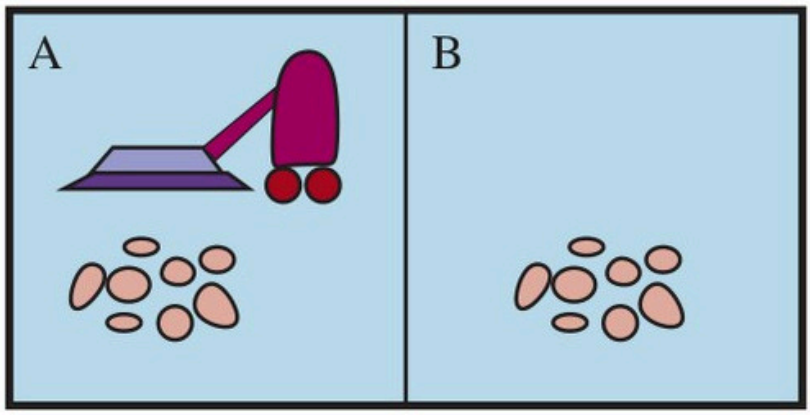


شکل ۲.۱ – عامل‌ها از طریق حسگرها و عملگرها با محیط‌ها تعامل می‌کنند. (نمودار: عامل ⇄ ادراک‌ها/کنش‌ها ⇄ محیط)

می‌توان تصور کرد تابع عاملِ توصیف‌کنندهٔ هر عامل داده‌شده را به‌صورت جدول درآوریم؛ برای بیشتر عامل‌ها این جدول بسیار بزرگ خواهد بود – در واقع بی‌نهایت، مگر آنکه بر طول توالی‌های ادراک حسیِ موردنظرمان حدی بگذاریم. با داشتن عاملی برای آزمایش، در اصل می‌توانیم این جدول را با آزمودن همهٔ توالی‌های ادراک حسی ممکن و ثبت کنش‌هایی که عامل در پاسخ انجام می‌دهد بسازیم.[1] جدول البته توصیفی بیرونی از عامل است. در درون، تابع عاملِ یک عامل مصنوعی توسط «برنامهٔ عامل» (Agent Program) پیاده‌سازی می‌شود. مهم است این دو ایده را از هم متمایز نگه داریم. تابع عامل یک توصیف ریاضی انتزاعی است؛ برنامهٔ عامل پیاده‌سازی عینی است که در درون سامانه‌ای فیزیکی اجرا می‌شود.

برای نشان دادن این ایده‌ها، از مثال ساده‌ای استفاده می‌کنیم – «جهان جاروبرقی» که شامل عامل رباتیک جاروبرقی در جهانی از خانه‌هاست که هر خانه می‌تواند کثیف یا تمیز باشد. شکل ۲.۲ پیکربندی با فقط دو خانه، A و B، را نشان می‌دهد. عامل جاروبرقی ادراک می‌کند در کدام خانه است و آیا در آن خانه کثیفی هست. عامل از خانهٔ A آغاز می‌کند. کنش‌های در دسترس عبارت‌اند از: حرکت به راست، حرکت به چپ، مکیدن کثیفی، یا هیچ‌کاری نکردن.[2] یکی از تابع‌های عامل بسیار ساده چنین است: اگر خانهٔ کنونی کثیف است، بمک؛ وگرنه به خانهٔ دیگر برو. بخشی از جدول این تابع عامل در شکل ۲.۳ و برنامهٔ عاملی که آن را پیاده می‌کند در شکل ۲.۸ (صفحهٔ ۴۹) آمده است.

با نگاه به شکل ۲.۳ می‌بینیم عامل‌های گوناگون جهان جاروبرقی را می‌توان صرفاً با پرکردن ستون راست به شیوه‌های گوناگون تعریف کرد. پس پرسشِ بدیهی این است: ⚠ راهِ درستِ پر کردن جدول چیست؟ به بیان دیگر، چه چیزی عامل را خوب یا بد، هوشمند یا احمق می‌کند؟ در بخش بعد به این پرسش‌ها پاسخ می‌دهیم.



شکل ۲.۲ – جهانی جاروبرقی با فقط دو مکان. هر مکان می‌تواند تمیز یا کثیف باشد و عامل می‌تواند چپ یا راست حرکت کند و خانه‌ای را که در آن است تمیز کند. نسخه‌های گوناگون جهان جاروبرقی قواعد متفاوتی برای آنچه عامل می‌تواند ادراک کند، اینکه کنش‌هایش آیا همیشه موفق‌اند، و مانند آن دارند.

توالی ادراک حسی	کنش
[A,Clean]	Right
[A,Dirty]	Suck
[B,Clean]	Left
[B,Dirty]	Suck
[A,Clean] ,[A,Clean]	Right
[A,Dirty] ,[A,Clean]	Suck
...	...
[A,Clean] ,[A,Clean] ,[A,Clean]	Right
[A,Dirty] ,[A,Clean] ,[A,Clean]	Suck
...	...

شکل ۲.۳ – بخشی از جدول یک تابع عامل ساده برای جهان جاروبرقیِ شکل ۲.۲. عامل اگر خانهٔ کنونی کثیف باشد آن را تمیز می‌کند وگرنه به خانهٔ دیگر می‌رود. توجه کنید جدول بی‌اندازه است، مگر آنکه محدودیتی بر طول توالی‌های ادراک حسی ممکن گذاشته شود.

پیش از بستن این بخش، باید تأکید کنیم مفهوم عامل ابزاری برای تحلیل سامانه‌هاست، نه توصیفی مطلق که جهان را به عامل و غیرعامل تقسیم کند. می‌توان ماشین حساب دستی را عاملی دید که با دریافت توالی ادراک حسی «۲ + ۲ =» کنشِ نمایش «۴» را برمی‌گزیند، اما چنین تحلیلی به‌ندرت فهم ما از ماشین حساب را یاری می‌کند. از جهتی، همهٔ حوزه‌های مهندسی را می‌توان طراحیِ مصنوعاتی دانست که با جهان تعامل می‌کنند؛ هوش مصنوعی در (به‌باور نویسندگان) جذاب‌ترین انتهای این طیف عمل می‌کند، جایی که مصنوعات منابع محاسباتی چشمگیری دارند و محیط تکلیف به تصمیم‌گیری غیربدیهی نیاز دارد.

۲.۲ رفتار خوب: مفهوم عقلانیت

عامل عقلایی (Rational Agent) عاملی است که کار درست را انجام می‌دهد. بدیهی است انجام کار درست بهتر از کار نادرست است، اما «کار درست» به چه معناست؟

۲.۲.۱ معیارهای عملکرد

فلسفهٔ اخلاق مفهوم‌های گوناگونی از «کار درست» پرورانده، اما هوش مصنوعی عموماً به مفهومی به نام پیامدگرایی (Consequentialism) وفادار مانده است: رفتار عامل را با پیامدهایش ارزیابی می‌کنیم. وقتی عامل در محیطی قرار داده می‌شود، بر اساس ادراک‌هایی که دریافت می‌کند توالی‌ای از کنش‌ها تولید می‌کند. این توالی کنش باعث می‌شود محیط از توالی‌ای از حالت‌ها بگذرد. اگر آن توالی مطلوب باشد، عامل خوب عمل کرده است. این مفهوم مطلوبیت با «معیار عملکرد» (Performance Measure) درک می‌شود که هر توالی داده‌شده از حالت‌های محیط را ارزیابی می‌کند.

انسان‌ها خواسته‌ها و ترجیحات خودشان را دارند، پس مفهوم عقلانیتِ اعمال‌شده بر انسان‌ها به موفقیت آن‌ها در انتخاب کنش‌هایی مربوط است که توالی‌هایی از حالت‌های محیط تولید می‌کنند و از دیدگاه خودشان مطلوب‌اند. ماشین‌ها اما خواسته‌ها و ترجیحات خودشان را ندارند؛ معیار عملکرد، دست‌کم در آغاز، در ذهن طراح ماشین است یا در ذهن کاربرانی که ماشین برای آنان طراحی شده.

خواهیم دید برخی طراحی‌های عامل بازنمایی صریحی از (نسخه‌ای از) معیار عملکرد دارند، در حالی که در برخی طراحی‌های دیگر معیار عملکرد کاملاً ضمنی است — عامل ممکن است کار درست را انجام دهد، اما نداند چرا.

با یادآوری هشدار نوربرت وینر دربارهٔ تضمین اینکه «هدفِ گذاشته‌شده در ماشین، هدفی است که واقعاً خواستاریم» (صفحهٔ ۳۳)، توجه کنید صورت‌بندی درستِ یک معیار عملکرد می‌تواند بسیار دشوار باشد. برای نمونه، عامل جاروبرقی بخش پیشین را در نظر بگیرید. شاید پیشنهاد کنیم عملکرد را با مقدار کثیفیِ پاک‌شده در یک شیفت کاری هشت‌ساعته بسنجیم. با عامل عقلایی، البته، هر چه بخواهید همان را می‌گیرید. عامل عقلایی می‌تواند این معیار عملکرد را با تمیزکردن کثیفی، سپس ریختن دوبارهٔ همهٔ آن روی زمین، و دوباره تمیزکردن آن، و همین‌طور ادامه، بیشینه کند. معیار عملکرد مناسب‌تر، عاملی را پاداش می‌دهد که کف را تمیز نگه می‌دارد. برای نمونه، در هر گام زمانی به‌ازای

هر خانهٔ تمیز یک امتیاز (و شاید جریمه‌ای برای برق مصرفی و سر و صدای تولیدشده). به‌عنوان قاعده‌ای کلی، بهتر است معیارهای عملکرد را بر اساس آنچه واقعاً می‌خواهیم در محیط محقق شود طراحی کنیم، نه بر اساس تصورمان از اینکه عامل باید چگونه رفتار کند. ⚠

حتی وقتی از این دام‌های بدیهی پرهیز شود، مسائل گره‌داری باقی می‌مانند. برای نمونه، مفهوم «کف تمیز» در بند پیشین بر میانگین تمیزی در طول زمان مبتنی است. با این حال، دو عامل متفاوت می‌توانند همان میانگین تمیزی را به دست آورند: یکی که همیشه کار متوسطی می‌کند و دیگری که با انرژی تمیز می‌کند اما استراحت‌های طولانی دارد. کدام مطلوب‌تر است شاید نکته‌ای ظریف در «علم سرایداری» بنماید، اما در واقع پرسشی عمیق فلسفی با پیامدهای دوربرد است. کدام بهتر است — زندگی پرمخاطرهٔ اوج‌وفرود، یا وجودی امن اما یکنواخت؟ کدام بهتر است — اقتصادی که همه در فقر متوسط زندگی می‌کنند، یا اقتصادی که برخی در فراوانی‌اند و برخی بسیار فقیر؟ این پرسش‌ها را به‌عنوان تمرین به خوانندهٔ کوشا واگذار می‌کنیم.

در بیشتر این کتاب فرض می‌کنیم معیار عملکرد را می‌توان درست مشخص کرد. اما به دلایل بالا، باید این امکان را بپذیریم که هدفِ اشتباهی در ماشین بگذاریم — دقیقاً همان مسئلهٔ شاه ماداس که در صفحهٔ ۳۳ توصیف شد. وانگهی، هنگام طراحی قطعهٔ نرم‌افزاری که نسخه‌هایش متعلق به کاربران گوناگون خواهد بود، نمی‌توانیم ترجیحات دقیق هر کاربر را پیش‌بینی کنیم. پس ممکن است نیاز داشته باشیم عامل‌هایی بسازیم که عدم قطعیت آغازین دربارهٔ معیار عملکرد واقعی را بازتاب دهند و با گذشت زمان بیشتر دربارهٔ آن بیاموزند؛ چنین عامل‌هایی در فصل‌های ۱۶، ۱۸ و ۲۲ توصیف می‌شوند.

۲.۲.۲ عقلانیت

آنچه در هر لحظهٔ معین عقلایی است به چهار چیز بستگی دارد:

- معیار عملکردی که ملاک موفقیت را تعریف می‌کند؛
- دانش قبلی عامل دربارهٔ محیط؛
- کنش‌هایی که عامل می‌تواند انجام دهد؛
- توالی ادراک حسیِ عامل تاکنون.

این به تعریفی از عامل عقلایی می‌انجامد:

> ***تعریف عامل عقلایی*** *⚠ برای هر توالی ادراک حسی ممکن، عامل عقلایی باید کنشی را برگزیند که انتظار می‌رود معیار عملکردش را بیشینه کند، با توجه به شواهدِ ارائه‌شده توسط توالی ادراک حسی و هر دانشِ ازپیش‌ساخته‌ای که عامل دارد.*

عامل جاروبرقی ساده را در نظر بگیرید که اگر خانه‌ای کثیف باشد آن را تمیز می‌کند وگرنه به خانهٔ دیگر می‌رود؛ این همان تابع عاملی است که در شکل ۲.۳ جدول‌بندی شده. آیا این عامل عقلایی است؟

بستگی دارد! نخست باید بگوییم معیار عملکرد چیست، دربارهٔ محیط چه چیزهایی دانسته می‌شود و عامل چه حسگرها و عملگرهایی دارد. فرض کنیم:

- معیار عملکرد در هر گام زمانی به‌ازای هر خانهٔ تمیز یک امتیاز می‌دهد، در طول «عمری» ۱۰۰۰ گامه؛
- «جغرافیای» محیط از پیش معلوم است (شکل ۲.۲)، اما توزیع کثیفی و مکان آغازین عامل معلوم نیست. خانه‌های تمیز تمیز می‌مانند و مکیدن، خانهٔ کنونی را تمیز می‌کند. کنش‌های Right و Left عامل را یک خانه جابه‌جا می‌کنند، مگر آنکه این کار عامل را بیرون از محیط ببرد، که در آن صورت عامل همان‌جا می‌ماند؛
- تنها کنش‌های در دسترس Left، Right و Suck هستند؛
- عامل مکان خود و اینکه آیا آن مکان کثیفی دارد را درست ادراک می‌کند.

در این شرایط عامل واقعاً عقلایی است؛ عملکرد مورد انتظارش دست‌کم به خوبی هر عامل دیگری است. به‌راحتی می‌توان دید همان عامل در شرایط متفاوت غیرعقلایی خواهد بود. برای نمونه، وقتی همهٔ کثیفی‌ها پاک شد، عامل بی‌جهت به این‌سو و آن‌سو نوسان خواهد کرد؛ اگر معیار عملکرد برای هر حرکت جریمه‌ای یک‌امتیازی در بر داشته باشد، عامل بد عمل خواهد کرد. عامل بهتر در این مورد پس از اطمینان از تمیزی همهٔ خانه‌ها هیچ‌کاری نمی‌کرد. اگر خانه‌های تمیز دوباره کثیف شوند، عامل باید گاه‌به‌گاه بررسی و در صورت نیاز بازتمیزی کند. اگر جغرافیای محیط ناشناخته باشد، عامل باید آن را کاوش کند. تمرین VACR.2 از شما می‌خواهد برای این موارد عامل طراحی کنید.

۲.۲.۳ دانایی مطلق، یادگیری و خودمختاری

باید مراقب باشیم عقلانیت را با دانایی مطلق (Omniscience) اشتباه نگیریم. عامل دانای مطلق، پیامد واقعی کنش‌هایش را می‌داند و می‌تواند بر آن اساس عمل کند؛ اما دانایی مطلق در واقعیت ناممکن است. این مثال را در نظر بگیرید: روزی در خیابان شانزلیزه راه می‌روم و دوست قدیمی‌ام را آن‌سوی خیابان می‌بینم. ترافیکی نزدیک نیست و کار دیگری هم ندارم؛ پس به‌عنوان موجودی عقلایی شروع به عبور از خیابان می‌کنم. در همین حال، در ارتفاع ۳۳٬۰۰۰ پایی، درِ بار یک هواپیمای مسافربریِ عبوری جدا می‌شود[3] و پیش از آنکه به آن‌سوی خیابان برسم، پهنم می‌کند. آیا عبور از خیابان غیرعقلایی بود؟ بعید است آگهی فوت من چنین باشد: «احمقی در تلاش برای عبور از خیابان». این مثال نشان می‌دهد عقلانیت همان کمال نیست. عقلانیت عملکردِ مورد انتظار را بیشینه می‌کند، در حالی که کمال، عملکردِ واقعی را بیشینه می‌کند. عقب‌نشینی از خواستهٔ کمال فقط مسئلهٔ منصف‌بودن با عامل‌ها نیست. نکته این است که اگر انتظار داشته باشیم عامل کنشی را انجام دهد که پس از وقوع معلوم می‌شود بهترین بوده، طراحی عاملی که این مشخصات را برآورده کند ناممکن خواهد بود — مگر آنکه عملکرد گوی بلورین یا ماشین زمان را بهبود دهیم.

پس تعریف ما از عقلانیت دانایی مطلق را نمی‌طلبد، زیرا انتخاب عقلایی فقط به توالی ادراک حسیِ تاکنون بستگی دارد. همچنین باید اطمینان حاصل کنیم ناخواسته به عامل اجازه نداده‌ایم درگیر فعالیت‌های قطعاً کم‌هوشانه شود. برای نمونه، اگر عاملی پیش از عبور از جاده‌ای شلوغ به دو سو نگاه نکند، توالی ادراک حسی‌اش به او نخواهد گفت کامیون بزرگی با سرعت زیاد نزدیک می‌شود. آیا تعریف ما از عقلانیت می‌گوید حالا عبور از جاده ایرادی ندارد؟ به‌هیچ‌وجه!

نخست، با این توالی ادراک حسی بی‌اطلاع‌کننده، عبور از جاده عقلایی نیست: ریسک تصادفِ عبور بدون نگاه‌کردن بسیار زیاد است. دوم، عامل عقلایی باید پیش از پاگذاشتن به خیابان کنشِ «نگاه‌کردن» را برگزیند، زیرا نگاه‌کردن به بیشینه‌شدن عملکرد مورد انتظار کمک می‌کند. انجام کنش‌ها برای اصلاح ادراک‌های حسی آینده — که گاه «جمع‌آوری اطلاعات» (Information Gathering) نامیده می‌شود — بخش مهمی از عقلانیت است و در فصل ۱۶ به‌طور عمیق پوشش داده می‌شود. مثال دومِ جمع‌آوری اطلاعات، کاوشی است که عامل جاروبرقی باید در محیطِ ناشناخته از آغاز انجام دهد.

تعریف ما ایجاب می‌کند عامل عقلایی نه فقط اطلاعات جمع کند، بلکه تا حد ممکن از آنچه ادراک می‌کند بیاموزد (Learning). پیکربندی آغازین عامل می‌تواند دانشی قبلی دربارهٔ محیط را بازتاب دهد، اما با کسب تجربه توسط عامل، این دانش می‌تواند اصلاح و افزوده شود. موارد افراطی وجود دارد که محیط از پیش کاملاً شناخته‌شده و کاملاً قابل پیش‌بینی است. در چنین مواردی عامل نه نیاز به ادراک دارد نه به یادگیری؛ فقط درست عمل می‌کند. البته چنین عامل‌هایی شکننده‌اند.

سوسک سرگین‌غلتانِ بی‌آزار را در نظر بگیرید. پس از کندن لانه و تخم‌گذاری، گلولهٔ سرگینی از توده‌ای نزدیک می‌آورد تا ورودی را ببندد. اگر گلولهٔ سرگین در مسیر از چنگش گرفته شود، سوسک به کارش ادامه می‌دهد و بستن لانه با گلولهٔ سرگین ناموجود را پانتومیم می‌کند و هرگز متوجه نمی‌شود گم شده. تکامل فرضی در رفتار سوسک ساخته است و وقتی آن فرض نقض شود، رفتار نافرجام به بار می‌آید. کمی هوشمندتر، زنبور اسفکس است. زنبور ماده اسفکس لانه‌ای می‌گند، بیرون می‌رود و کرمی را نیش می‌زند و به لانه می‌کشد، دوباره وارد لانه می‌شود تا مطمئن شود همه‌چیز مرتب است، کرم را به داخل می‌کشد و تخم می‌گذارد. کرم منبع غذایی است وقتی تخم‌ها باز شوند. تا این‌جا همه‌چیز خوب است، اما اگر حشره‌شناسی هنگام بازبینیِ زنبور، کرم را چند سانتی‌متر جابه‌جا کند، زنبور به گام «کشیدن کرم» از برنامه‌اش بازمی‌گردد و برنامه را بدون تغییر ادامه می‌دهد و لانه را دوباره بازبینی می‌کند، حتی پس از ده‌ها بار جابه‌جایی کرم. زنبور اسفکس نمی‌تواند بیاموزد برنامهٔ فطری‌اش در حال شکست است و بنابراین آن را تغییر نمی‌دهد.

تا جایی که عامل به دانش قبلی طراحش تکیه کند نه به ادراک‌های حسی و فرایندهای یادگیری خودش، می‌گوییم عامل «خودمختاری» (Autonomy) ندارد. عامل عقلایی باید خودمختار باشد — باید آنچه می‌تواند بیاموزد تا کاستی یا نادرستی دانش قبلی را جبران کند. برای نمونه، عامل جاروبرقی که یاد می‌گیرد کثیفی اضافی کجا و چه‌وقت ظاهر خواهد شد، بهتر از عاملی عمل می‌کند که این را نمی‌آموزد.

به‌عنوان امری عملی، به‌ندرت از همان آغاز خودمختاری کامل می‌خواهیم: وقتی عامل تجربه‌ای کم یا هیچ داشته باشد، ناگزیر است تصادفی عمل کند، مگر آنکه طراح کمکی کرده باشد. همان‌گونه که تکامل به جانوران بازتاب‌های فطری کافی می‌دهد تا آن‌قدر زنده بمانند که خودشان یاد بگیرند، معقول است به عامل هوشمند مصنوعی هم دانشی آغازین و هم توانایی یادگیری دهیم. پس از تجربهٔ کافی از محیط، رفتار عامل عقلایی می‌تواند عملاً مستقل از دانش قبلی‌اش شود. از این رو، گنجاندن یادگیری به ما امکان می‌دهد عاملی عقلاییِ واحد طراحی کنیم که در گسترهٔ عظیمی از محیط‌ها موفق خواهد بود.

۲.۳ سرشت محیط‌ها

حال که تعریفی از عقلانیت داریم، تقریباً آماده‌ایم به ساختن عامل‌های عقلایی بیندیشیم. اما نخست باید به «محیط‌های تکلیف» (Task Environments) بیندیشیم که در اصل «مسئله‌هایی» هستند که عامل‌های عقلایی «راه‌حل» آن‌ها هستند. آغاز می‌کنیم با نشان‌دادن اینکه چگونه محیط تکلیف را مشخص کنیم و این فرایند را با چند مثال نشان می‌دهیم. سپس نشان می‌دهیم محیط‌های تکلیف طعم‌های گوناگونی دارند. سرشت محیط تکلیف مستقیماً بر طراحی مناسب برنامهٔ عامل اثر می‌گذارد.

۲.۳.۱ مشخص کردن محیط تکلیف

در بحث‌مان دربارهٔ عقلانیتِ عامل جاروبرقی ساده، باید معیار عملکرد، محیط و عملگرها و حسگرهای عامل را مشخص می‌کردیم. همهٔ این‌ها را زیر عنوان محیط تکلیف گروه‌بندی می‌کنیم. برای مخفف‌دوست‌ها، این را توصیف PEAS (عملکرد، محیط، عملگرها، حسگرها — Performance, Environment, Actuators, Sensors) می‌نامیم. در طراحی عامل، نخستین گام همیشه باید مشخص کردن هرچه کامل‌تر محیط تکلیف باشد.

جهان جاروبرقی مثال ساده‌ای بود؛ مسئلهٔ پیچیده‌تری در نظر بگیریم: رانندهٔ تاکسی خودکار. شکل ۲.۴ توصیف PEAS را برای محیط تکلیف تاکسی خلاصه می‌کند. هر عنصر را در بندهای بعد با جزئیات بیشتری بحث می‌کنیم.

نخست، معیار عملکردی که می‌خواهیم رانندهٔ خودکار ما به آن برسد چیست؟ ویژگی‌های مطلوب شامل رسیدن به مقصد درست؛ کمینه‌کردن مصرف سوخت و استهلاک؛ کمینه‌کردن زمان یا هزینهٔ سفر؛ کمینه‌کردن تخلف از قوانین راهنمایی و مزاحمت برای دیگر رانندگان؛ بیشینه‌کردن ایمنی و آسایش سرنشین؛ و بیشینه‌کردن سود است. بدیهی است برخی از این هدف‌ها با هم در تعارض‌اند، پس دادوستد لازم خواهد بود.

بعد، محیط رانندگی‌ای که تاکسی با آن روبه‌رو خواهد شد چیست؟ هر رانندهٔ تاکسی باید با گستره‌ای از جاده‌ها کنار بیاید، از مسیرهای روستایی و کوچه‌های شهری تا بزرگراه‌های دوازده‌خطه. جاده‌ها حاوی ترافیک دیگر، عابران پیاده، جانوران سرگردان، کارهای جاده‌ای، خودروهای پلیس، چاله‌ها و دست‌اندازها هستند. تاکسی همچنین باید با مسافران بالقوه و بالفعل تعامل کند. انتخاب‌های اختیاری هم هست. تاکسی ممکن است لازم باشد در کالیفرنیای جنوبی کار کند، جایی که برف به‌ندرت مشکل‌ساز است، یا در آلاسکا، جایی که برف به‌ندرت بی‌مشکل است. ممکن است همیشه در سمت راست جاده رانندگی کند، یا شاید بخواهیم آن‌قدر منعطف باشد که در بریتانیا یا ژاپن در سمت چپ رانندگی کند. بدیهی است هرچه محیط محدودتر باشد، مسئلهٔ طراحی آسان‌تر است.

نوع عامل	معیار عملکرد	محیط	عملگرها	حسگرها
رانندهٔ تاکسی	سفر امن، سریع، قانونی، راحت؛ بیشینه‌کردن سود؛ کمینه‌کردن اثر بر دیگر کاربران جاده	جاده‌ها، ترافیک دیگر، پلیس، عابران، مشتریان، آب‌وهوا	فرمان، گاز، ترمز، راهنما، بوق، نمایشگر، گفتار	دوربین‌ها، رادار، سرعت‌سنج، GPS، حسگرهای موتور، شتاب‌سنج، میکروفون‌ها، صفحهٔ لمسی

شکل ۲.۴ – توصیف PEAS محیط تکلیف برای رانندهٔ تاکسی خودکار:

عملگرهای تاکسی خودکار شامل آنچه رانندهٔ انسانی دارد می‌شود: کنترل موتور از طریق گاز و کنترل فرمان و ترمز. افزون بر آن، به خروجی برای صفحهٔ نمایش یا سنتزکنندهٔ صدا برای پاسخ‌دادن به مسافران نیاز دارد و شاید راهی برای ارتباط با دیگر وسایل نقلیه، مؤدبانه یا غیر آن.

حسگرهای پایهٔ تاکسی شامل یک یا چند دوربین ویدئویی برای دیدن، و نیز حسگرهای لیدار و اولتراسونیک برای تشخیص فاصله تا خودروهای دیگر و موانع خواهد بود. برای اجتناب از جریمهٔ سرعت، تاکسی باید سرعت‌سنج داشته باشد و برای کنترل درست خودرو، به‌ویژه در پیچ‌ها، شتاب‌سنج. برای تعیین حالت مکانیکی خودرو، به آرایهٔ معمول حسگرهای موتور، سوخت و سامانهٔ الکتریکی نیاز دارد. مانند بسیاری رانندگان انسانی، شاید بخواهد به سیگنال‌های GPS دسترسی داشته باشد تا گم نشود. سرانجام، برای درخواست مقصد توسط مسافر به ورودی صفحهٔ لمسی یا صوتی نیاز خواهد داشت.

در شکل ۲.۵ عناصر پایهٔ PEAS را برای چند عامل دیگر ترسیم کرده‌ایم.

نوع عامل	معیار عملکرد	محیط	عملگرها	حسگرها
سامانهٔ تشخیص پزشکی	بیمار سالم، هزینه‌های کمتر	بیمار، بیمارستان، کارکنان	نمایش پرسش‌ها، آزمایش‌ها، تشخیص‌ها، درمان‌ها	ورود لمسی/صوتی علائم و یافته‌ها
سامانهٔ تحلیل تصاویر ماهواره‌ای	رده‌بندی درست اشیا، عوارض زمین	ماهوارهٔ مداری، پایگاه دادهٔ پایین‌فرست، آب‌وهوا	نمایش رده‌بندی صحنه	دوربین دیجیتال وضوح‌بالا
ربات قطعه‌چین	درصد قطعات در سبدهای درست	نوار نقاله با قطعات؛ سبدها	بازوی مفصلی و دست	دوربین، حسگرهای لامسه‌ای و زاویهٔ مفصل
کنترل‌گر پالایشگاه	خلوص، بازده، ایمنی	پالایشگاه، مواد خام، اپراتورها	شیرها، پمپ‌ها، گرم‌کن‌ها، هم‌زن‌ها، نمایشگرها	حسگرهای دما، فشار، جریان، شیمیایی

نوع عامل	معیار عملکرد	محیط	عملگرها	حسگرها
معلم انگلیسی تعاملی	نمرهٔ دانشجو در آزمون	مجموعهٔ دانشجویان، نهاد آزمون	نمایش تمرین‌ها، بازخورد، گفتار	ورود صفحه‌کلید، صدا

شکل ۲.۵ – نمونه‌هایی از انواع عامل و توصیف‌های PEAS آن‌ها:

۲.۳.۲ ویژگی‌های محیط‌های تکلیف

گسترهٔ محیط‌های تکلیفی که ممکن است در هوش مصنوعی پیش آید بدیهی است عظیم است. با این حال، می‌توانیم تعداد نسبتاً کمی «بُعد» شناسایی کنیم که محیط‌های تکلیف را بر اساس آن‌ها می‌توان رده‌بندی کرد. این ابعاد تا حد زیادی طراحی مناسب عامل و کاربردپذیری هر یک از خانواده‌های اصلی فنون پیاده‌سازی عامل را تعیین می‌کنند. نخست ابعاد را فهرست می‌کنیم، سپس چند محیط تکلیف را برای نشان دادن ایده‌ها تحلیل می‌کنیم. تعریف‌های این‌جا غیررسمی‌اند؛ فصل‌های بعدی بیانیه‌های دقیق‌تر و مثال‌هایی از هر نوع محیط ارائه می‌دهند.

کاملاً قابل مشاهده در برابر تا حدی قابل مشاهده (Fully observable vs. partially observable): اگر حسگرهای عامل در هر نقطهٔ زمانی به او دسترسی به حالت کامل محیط بدهند، می‌گوییم محیط تکلیف کاملاً قابل مشاهده است. محیط تکلیف به‌شکل مؤثر کاملاً قابل مشاهده است اگر حسگرها همهٔ جنبه‌های مرتبط با انتخاب کنش را شناسایی کنند؛ «مرتبط بودن» نیز به نوبهٔ خود به معیار عملکرد بستگی دارد. محیط‌های کاملاً قابل مشاهده راحت‌اند، زیرا عامل لازم نیست حالت درونی‌ای برای پیگیری جهان نگه دارد. محیطی ممکن است تا حدی قابل مشاهده باشد به‌سبب حسگرهای پرسروصدا و نادقیق یا به‌سبب آنکه بخش‌هایی از حالت صرفاً در دادهٔ حسگرها غایب‌اند — برای نمونه، عامل جاروبرقی با حسگر کثیفیِ محلی نمی‌تواند بگوید آیا در خانه‌های دیگر کثیفی هست، و تاکسی خودکار نمی‌تواند ببیند دیگر رانندگان به چه می‌اندیشند. اگر عامل هیچ حسگری نداشته باشد، محیط غیرقابل مشاهده (Unobservable) است. شاید فکر کنید در چنین مواردی وضعیت عامل ناامیدکننده است، اما چنان‌که در فصل ۴ بحث می‌کنیم، هدف‌های عامل ممکن است همچنان دست‌یافتنی باشند، گاهی با قطعیت.

تک‌عاملی در برابر چندعاملی (Single-agent vs. multiagent): تمایز میان محیط‌های تک‌عاملی و چندعاملی شاید به‌اندازهٔ کافی ساده بنماید. برای نمونه، عاملی که به‌تنهایی جدول کلمات متقاطع حل می‌کند قطعاً در محیط تک‌عاملی است، حال آنکه عاملِ بازی شطرنج در محیط دوعاملی است. اما مسائل ظریفی وجود دارد. نخست، توصیف کردیم که چگونه یک موجودیت را می‌توان عامل دید، اما توضیح نداده‌ایم کدام موجودیت‌ها را باید عامل دید. آیا عامل A (مثلاً رانندهٔ تاکسی) باید موجودیت B (خودرویی دیگر) را عامل بپندارد، یا می‌تواند آن را صرفاً شیءِ رفتارکننده بر پایهٔ قوانین فیزیک بداند، مانند امواج ساحل یا برگ‌هایی که در باد می‌وزند؟ تمایز کلیدی این است که آیا رفتار B را بهتر است به‌مثابه بیشینه‌کردن معیار عملکردی توصیف کرد که ارزشش به رفتار عامل A بستگی دارد؟

برای نمونه، در شطرنج، موجودیت حریف B می‌کوشد معیار عملکردش را بیشینه کند که بر اساس قواعد شطرنج، معیار عملکرد عامل A را کمینه می‌کند. پس شطرنج محیط چندعاملی رقابتی (Competitive) است. از سوی دیگر، در محیط رانندگی تاکسی، اجتناب از تصادف معیار عملکرد همهٔ عامل‌ها را بیشینه می‌کند، پس محیطی چندعاملی تا حدی

هم‌کارانه (Cooperative) است. همچنین تا حدی رقابتی است، زیرا برای نمونه فقط یک خودرو می‌تواند یک جای پارک را اشغال کند.

مسائل طراحی عامل در محیط‌های چندعاملی اغلب کاملاً متفاوت از محیط‌های تک‌عاملی‌اند؛ برای نمونه، ارتباط اغلب در محیط‌های چندعاملی به‌عنوان رفتاری عقلایی پدیدار می‌شود؛ در برخی محیط‌های رقابتی، رفتار تصادفی عقلایی است، زیرا از دام‌های قابل پیش‌بینی‌بودن می‌پرهیزد.

جبرگرایانه در برابر ناجبرگرایانه (Deterministic vs. nondeterministic): اگر حالت بعدی محیط کاملاً توسط حالت کنونی و کنشِ اجراشده توسط عامل(ها) تعیین شود، می‌گوییم محیط جبرگرایانه است؛ وگرنه ناجبرگرایانه است. در اصل، عامل در محیط کاملاً قابل مشاهده و جبرگرایانه لازم نیست نگران عدم قطعیت باشد. اما اگر محیط تا حدی قابل مشاهده باشد، ممکن است ناجبرگرایانه بنماید.

بیشتر موقعیت‌های واقعی چنان پیچیده‌اند که پیگیری همهٔ جنبه‌های مشاهده‌نشده ناممکن است؛ برای اهداف عملی، باید آن‌ها را ناجبرگرایانه دانست. رانندگی تاکسی به این معنا قطعاً ناجبرگرایانه است، زیرا هرگز نمی‌توان رفتار ترافیک را دقیقاً پیش‌بینی کرد؛ وانگهی، ممکن است لاستیک‌ها به‌طور غیرمنتظره پنچر شوند و موتور بدون هشدار از کار بیفتد. جهان جاروبرقیِ چنان‌که توصیف کردیم جبرگرایانه است، اما تغییرهایش می‌تواند عناصر ناجبرگرایانه در بر داشته باشد، مانند کثیفیِ تصادفی‌ظهور و سازوکار مکش غیرقابل‌اعتماد (تمرین VFIN.2). یک نکتهٔ پایانی: برخی از واژهٔ تصادفی (Stochastic) به‌عنوان مترادف «ناجبرگرایانه» استفاده می‌کنند، اما ما میان این دو اصطلاح تمایز قائل می‌شویم؛ می‌گوییم مدلی از محیط تصادفی است اگر به‌صراحت با احتمال‌ها سروکار داشته باشد (مثلاً «۲۵٪ احتمال باران فرداست») و «ناجبرگرایانه» اگر امکان‌ها بدون کمی‌شدن فهرست شوند (مثلاً «احتمال باران فرداست»).

تکرارشونده در برابر توالی‌دار (Episodic vs. sequential): در محیط تکلیف تکرارشونده، تجربهٔ عامل به تکراره‌های اتمی تقسیم می‌شود. در هر تکراره، عامل ادراک حسی دریافت می‌کند و سپس یک کنش واحد انجام می‌دهد. نکتهٔ حیاتی این است که تکرارهٔ بعدی به کنش‌های انجام‌شده در تکراره‌های قبلی بستگی ندارد. بسیاری از تکلیف‌های رده‌بندی تکرارشونده‌اند. برای نمونه، عاملی که باید قطعات معیوب را در خط مونتاژ شناسایی کند، هر تصمیم را بر قطعهٔ کنونی استوار می‌کند، صرف‌نظر از تصمیم‌های قبلی؛ وانگهی، تصمیم کنونی بر معیوب‌بودن قطعهٔ بعدی اثر نمی‌گذارد. در محیط‌های توالی‌دار اما تصمیم کنونی می‌تواند بر همهٔ تصمیم‌های آینده اثر بگذارد.[4] شطرنج و رانندگی تاکسی توالی‌دارند: در هر دو، کنش‌های کوتاه‌مدت می‌توانند پیامدهای بلندمدت داشته باشند. محیط‌های تکرارشونده بسیار ساده‌تر از محیط‌های توالی‌دارند، زیرا عامل لازم نیست به آینده بیندیشد.

ایستا در برابر پویا (Static vs. dynamic): اگر محیط بتواند هنگام اندیشیدنِ عامل تغییر کند، می‌گوییم محیط برای آن عامل پویا است؛ وگرنه ایستا. محیط‌های ایستا آسان‌اند، زیرا عامل هنگام تصمیم‌گیری دربارهٔ کنش لازم نیست مدام به جهان نگاه کند و لازم نیست نگران گذر زمان باشد. محیط‌های پویا اما پیوسته از عامل می‌پرسند چه می‌خواهد بکند؛ اگر هنوز تصمیم نگرفته باشد، این به‌مثابه تصمیم به هیچ‌کاری‌نکردن شمرده می‌شود. اگر خودِ محیط با گذر زمان تغییر نکند اما امتیاز عملکرد عامل تغییر کند، می‌گوییم محیط نیمه‌پویا (Semidynamic) است. رانندگی تاکسی قطعاً پویاست: خودروهای دیگر و خودِ تاکسی پیوسته در حرکت‌اند در حالی که الگوریتم رانندگی دربارهٔ گام بعدی تردید می‌کند. شطرنجِ بازی‌شده با ساعت، نیمه‌پویاست. جدول کلمات متقاطع ایستاست.

گسسته در برابر پیوسته (Discrete vs. continuous): تمایز گسسته/پیوسته به حالت محیط، به شیوهٔ رسیدگی به زمان، و به ادراک‌های حسی و کنش‌های عامل اعمال می‌شود. برای نمونه، محیط شطرنج تعداد متناهی حالت متمایز دارد (به‌جز ساعت). شطرنج همچنین مجموعه‌ای گسسته از ادراک‌های حسی و کنش‌ها دارد. رانندگی تاکسی مسئله‌ای با حالت پیوسته و زمان پیوسته است: سرعت و مکان تاکسی و دیگر وسایل نقلیه از گستره‌ای از مقادیر پیوسته می‌گذرند و این کار را در طول زمان نرم انجام می‌دهند. کنش‌های رانندگی تاکسی نیز پیوسته‌اند (زاویهٔ فرمان و...). ورودی دوربین‌های دیجیتال، به بیان دقیق، گسسته است، اما معمولاً به‌عنوان بازنماییِ شدت‌ها و مکان‌های متغیرِ پیوسته در نظر گرفته می‌شود.

شناخته‌شده در برابر ناشناخته (Known vs. unknown): به بیان دقیق، این تمایز به خودِ محیط اشاره ندارد، بلکه به وضعیت دانش عامل (یا طراح) دربارهٔ «قوانین فیزیک» محیط اشاره دارد. در محیط شناخته‌شده، پیامدها (یا احتمال‌های پیامد، اگر محیط ناجبرگرایانه باشد) برای همهٔ کنش‌ها داده شده‌اند. بدیهی است اگر محیط ناشناخته باشد، عامل باید یاد بگیرد چگونه کار می‌کند تا تصمیم‌های خوب بگیرد.

تمایز محیط شناخته‌شده و ناشناخته همان تمایز کاملاً/تا حدی قابل مشاهده نیست. کاملاً ممکن است محیطی شناخته‌شده تا حدی قابل مشاهده باشد — برای نمونه، در بازی‌های ورق یک‌نفره، قواعد را می‌دانم اما هنوز نمی‌توانم ورق‌هایی را ببینم که روی‌نشده‌اند. برعکس، محیط ناشناخته می‌تواند کاملاً قابل مشاهده باشد — در بازی ویدئویی تازه‌ای، صفحه ممکن است کل حالت بازی را نشان دهد اما من هنوز نمی‌دانم دکمه‌ها چه می‌کنند تا آنکه امتحانشان کنم.

چنان‌که در صفحهٔ ۳۹ یادآوری شد، خودِ معیار عملکرد ممکن است ناشناخته باشد؛ یا به‌سبب آنکه طراح مطمئن نیست چگونه آن را درست بنویسد، یا به‌سبب آنکه کاربر نهایی — که ترجیحاتش اهمیت دارد — شناخته نشده. برای نمونه، رانندهٔ تاکسی معمولاً نمی‌داند آیا مسافر تازه سفر آهسته را ترجیح می‌دهد یا پرسرعت، سبک رانندگی محتاطانه را یا تهاجمی. دستیار شخصی مجازی از آغاز هیچ‌چیز دربارهٔ ترجیحات شخصیِ مالک تازه‌اش نمی‌داند. در چنین مواردی، عامل ممکن است از طریق تعامل‌های بیشتر با طراح یا کاربر دربارهٔ معیار عملکرد بیشتر بیاموزد. این به نوبهٔ خود پیشنهاد می‌کند محیط تکلیف ناگزیر باید به‌عنوان محیط چندعاملی دیده شود.

دشوارترین حالت، محیطی است تا حدی قابل مشاهده، چندعاملی، ناجبرگرایانه، توالی‌دار، پویا، پیوسته و ناشناخته. رانندگی تاکسی از همهٔ این جهات دشوار است، به‌جز آنکه محیط راننده بیشتر شناخته‌شده است. رانندگی با خودروی کرایه‌ای در کشور تازه‌ای با جغرافیای ناآشنا، قوانین راهنمایی متفاوت و مسافران مضطرب بسیار هیجان‌انگیزتر است.

شکل ۲.۶ ویژگی‌های چند محیط آشنا را فهرست می‌کند. توجه کنید ویژگی‌ها همیشه قطعی نیستند. برای نمونه، تکلیف تشخیص پزشکی را تک‌عاملی فهرست کرده‌ایم، زیرا فرایند بیماری در بیمار به‌شکل سودمندی به‌عنوان عامل مدل نمی‌شود؛ اما سامانهٔ تشخیص پزشکی ممکن است با بیماران نافرمان و کارکنان بدبین هم سروکار داشته باشد، پس محیط می‌تواند جنبهٔ چندعاملی داشته باشد. وانگهی، تشخیص پزشکی اگر تکلیف را انتخاب تشخیص با فهرست علائم بپنداریم تکرارشونده است؛ اگر تکلیف شامل پیشنهاد مجموعه‌ای از آزمایش‌ها، ارزیابی پیشرفت در طول درمان، رسیدگی به چند بیمار و مانند آن باشد، توالی‌دار است.

محیط تکلیف	قابل مشاهده	عامل‌ها	جبرگرایی	تکرار	ایستایی	گسسته
جدول کلمات متقاطع	کامل	تک‌عاملی	جبری	توالی‌دار	ایستا	گسسته
شطرنج با ساعت	کامل	چندعاملی	جبری	توالی‌دار	نیمه‌پویا	گسسته
پوکر	تا حدی	چندعاملی	تصادفی	توالی‌دار	ایستا	گسسته
تخته‌نرد	کامل	چندعاملی	تصادفی	توالی‌دار	ایستا	گسسته
رانندگی تاکسی	تا حدی	چندعاملی	تصادفی	توالی‌دار	پویا	پیوسته
تشخیص پزشکی	تا حدی	تک‌عاملی	تصادفی	توالی‌دار	پویا	پیوسته
تحلیل تصویر	کامل	تک‌عاملی	جبری	تکرارشونده	نیمه‌پویا	پیوسته
ربات قطعه‌چین	تا حدی	تک‌عاملی	تصادفی	تکرارشونده	پویا	پیوسته
کنترل‌گر پالایشگاه	تا حدی	تک‌عاملی	تصادفی	توالی‌دار	پویا	پیوسته
معلم انگلیسی	تا حدی	چندعاملی	تصادفی	توالی‌دار	پویا	گسسته

شکل ۲.۶ – نمونه‌هایی از محیط‌های تکلیف و ویژگی‌های آن‌ها:

ستون «شناخته/ناشناخته» را نگنجانده‌ایم، زیرا چنان‌که پیش‌تر توضیح داده شد، به بیان دقیق این ویژگیِ خود محیط نیست. برای برخی محیط‌ها، مانند شطرنج و پوکر، دادن دانش کامل قواعد به عامل بسیار آسان است، اما با این حال جالب است در نظر بگیریم عاملی چگونه ممکن است بازی‌کردن را بدون چنین دانشی بیاموزد.

مخزن کدِ همراه این کتاب (aima.cs.berkeley.edu) شامل پیاده‌سازی‌های متعدد محیط، همراه با شبیه‌ساز محیطی همه‌منظوره برای ارزیابی عملکرد عامل است. آزمایش‌ها اغلب نه برای یک محیط منفرد، بلکه برای بسیاری از محیط‌های برگرفته از «ردهٔ محیط» (Environment Class) انجام می‌شوند. برای نمونه، برای ارزیابی رانندهٔ تاکسی در ترافیک شبیه‌سازی‌شده، می‌خواهیم شبیه‌سازی‌های فراوانی با شرایط ترافیک، نور و آب‌وهوای گوناگون اجرا کنیم. آنگاه به عملکرد میانگین عامل بر روی ردهٔ محیط علاقه‌مندیم.

۲.۴ ساختار عامل‌ها

تا این‌جا دربارهٔ عامل‌ها با توصیف رفتار – کنشی که پس از هر توالی معین از ادراک‌های حسی انجام می‌شود – سخن گفتیم. اکنون باید گاز را گاز بگیریم و دربارهٔ چگونگی کارکرد درون‌ها بحث کنیم. کار هوش مصنوعی طراحی برنامهٔ عاملی است که تابع عامل را پیاده‌سازی کند – نگاشت از ادراک‌های حسی به کنش‌ها. فرض می‌کنیم این برنامه روی نوعی

دستگاه محاسباتی با حسگرها و عملگرهای فیزیکی اجرا خواهد شد — این را «معماری عامل» (Agent Architecture) می‌نامیم:

```
عامل = معماری + برنامه
```

بدیهی است برنامه‌ای که برمی‌گزینیم باید با معماری سازگار باشد. اگر برنامه قرار است کنش‌هایی مانند «راه برو» پیشنهاد کند، معماری بهتر است پا داشته باشد. معماری ممکن است فقط یک رایانهٔ شخصی معمولی باشد، یا خودروی رباتیک با چند رایانهٔ روی‌خودرو، دوربین‌ها و حسگرهای دیگر. به‌طور کلی، معماری ادراک‌های حسیِ حسگرها را در دسترس برنامه قرار می‌دهد، برنامه را اجرا می‌کند و انتخاب‌های کنشِ برنامه را هنگام تولید به عملگرها می‌رساند. بیشتر این کتاب دربارهٔ طراحی برنامه‌های عامل است، هرچند فصل‌های ۲۵ و ۲۶ مستقیماً با حسگرها و عملگرها سروکار دارند.

۲.۴.۱ برنامه‌های عامل

برنامه‌های عاملی که در این کتاب طراحی می‌کنیم همگی اسکلت یکسانی دارند: ادراک حسی کنونی را به‌عنوان ورودی از حسگرها می‌گیرند و کنشی را به عملگرها بازمی‌گردانند.[5] به تفاوت برنامهٔ عامل که ادراک حسی کنونی را به‌عنوان ورودی می‌گیرد و تابع عامل که ممکن است به کل تاریخچهٔ ادراک حسی وابسته باشد توجه کنید. برنامهٔ عامل چاره‌ای جز گرفتن فقط ادراک حسی کنونی به‌عنوان ورودی ندارد، زیرا از محیط چیزی بیشتر در دسترس نیست؛ اگر کنش‌های عامل باید به کل توالی ادراک حسی وابسته باشند، عامل باید ادراک‌ها را به خاطر بسپارد. برنامه‌های عامل را در زبان شبه‌کد ساده‌ای توصیف می‌کنیم که در پیوست ب تعریف شده است. (مخزن کد آنلاین شامل پیاده‌سازی‌ها به زبان‌های برنامه‌نویسی واقعی است.) برای نمونه، شکل ۲.۷ برنامهٔ عامل نسبتاً پیش‌پاافتاده‌ای را نشان می‌دهد که توالی ادراک حسی را پیگیری می‌کند و سپس از آن برای فهرست‌یابی در جدول کنش‌ها برای تصمیم‌گیری استفاده می‌کند.

```
function TABLE-DRIVEN-AGENT(percept) returns an action
                                                    :persistent
                                  percepts, a sequence, initially empty
table, a table of actions, indexed by percept sequences, initially fully specified
                                        add percept to the end of percepts
                                          action ← LOOKUP(percepts, table)
                                                    return action
```

شکل ۲.۷ — برنامهٔ «عامل جدول‌محور» (TABLE-DRIVEN-AGENT) برای هر ادراک حسی تازه فراخوانی می‌شود و هر بار کنشی بازمی‌گرداند. کل توالی ادراک حسی را در حافظه نگه می‌دارد.

این جدول — که نمونه‌ای از آن برای جهان جاروبرقی در شکل ۲.۳ داده شده — تابع عاملی را که برنامهٔ عامل تجسم می‌بخشد به‌صراحت بازنمایی می‌کند. برای ساختن عامل عقلایی بدین روش، ما به‌عنوان طراح باید جدولی بسازیم که کنش مناسب را برای هر توالی ادراک حسی ممکن در بر داشته باشد. آموزنده است در نظر بگیریم چرا رویکرد جدول‌محور به ساخت عامل محکوم به شکست است. فرض کنیم P مجموعهٔ ادراک‌های حسی ممکن و T عمر عامل (مجموع ادراک‌هایی که دریافت خواهد کرد) باشد. جدول جست‌وجو دارای مدخل‌های زیر خواهد بود:

```
sum_{t=1}^T |P|^t\
```

تاکسی خودکار را در نظر بگیرید: ورودی بصری از یک دوربین (۸ دوربین معمول است) با نرخ تقریباً ۷۰ مگابایت در ثانیه وارد می‌شود (۳۰ فریم در ثانیه، ۱۰۸۰×۷۲۰ پیکسل با ۲۴ بیت اطلاعات رنگ). این جدول جست‌وجویی با بیش از ۶۰۰,۰۰۰,۰۰۰,۰۰۰^۱۰ مدخل برای یک ساعت رانندگی می‌دهد. حتی جدول جست‌وجوی شطرنج – قطعهٔ کوچک و خوش‌رفتار جهان واقعی – دست‌کم 10^{150} مدخل دارد. در مقایسه، شمار اتم‌های جهان قابل مشاهده کمتر از 10^{80} است. اندازهٔ هراس‌آور این جدول‌ها بدان معناست که (الف) هیچ عامل فیزیکی در این جهان فضای ذخیرهٔ جدول را ندارد؛ (ب) طراح زمان ساخت جدول را نخواهد داشت؛ و (ج) هیچ عاملی هرگز نمی‌تواند همهٔ مدخل‌های درست جدول را از تجربه بیاموزد. با وجود همهٔ این‌ها، TABLE-DRIVEN-AGENT همان چیزی را که می‌خواهیم انجام می‌دهد، به شرط آنکه جدول درست پر شده باشد: تابع عامل مطلوب را پیاده‌سازی می‌کند.

⚠ انتخاب‌های دیگری هم برای اسکلت برنامهٔ عامل هست؛ برای نمونه، می‌توان برنامه‌های عامل را روتین‌های هم‌روال (Coroutine) داشت که به‌صورت ناهمگام با محیط اجرا می‌شوند. هر چنین هم‌روالی یک درگاه ورودی و یک درگاه خروجی دارد و از حلقه‌ای تشکیل شده که درگاه ورودی را برای ادراک‌های حسی می‌خواند و کنش‌ها را به درگاه خروجی می‌نویسد.

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action
                                    if status = Dirty then return Suck
                                else if location = A then return Right
                                 else if location = B then return Left
```

شکل ۲.۸ – برنامهٔ عامل برای عامل بازتابی ساده در محیط جاروبرقی دومکانه. این برنامه تابع عاملِ جدول‌بندی‌شده در شکل ۲.۳ را پیاده‌سازی می‌کند.

چالش کلیدی هوش مصنوعی یافتن راهی برای نوشتن برنامه‌هایی است که تا حد ممکن رفتار عقلایی را از برنامه‌ای نسبتاً کوچک تولید کنند، نه از جدولی عظیم. مثال‌های فراوانی داریم که نشان می‌دهد این کار در حوزه‌های دیگر با موفقیت انجام شده: برای نمونه، جدول‌های عظیم جذر که مهندسان و دانش‌آموزان پیش از دههٔ ۱۹۷۰ استفاده می‌کردند، اکنون با برنامهٔ پنج‌خطی روش نیوتن که روی ماشین حساب‌های الکترونیکی اجرا می‌شود جایگزین شده‌اند. پرسش این است: آیا هوش مصنوعی می‌تواند برای رفتار هوشمندِ عمومی کاری کند که نیوتن برای جذر کرد؟ باور داریم پاسخ آری است.

در بقیهٔ این بخش، چهار نوع برنامهٔ عامل پایه را برمی‌شماریم که اصول زیربنایی تقریباً همهٔ سامانه‌های هوشمند را تجسم می‌بخشند:

- عامل‌های بازتابی ساده (Simple Reflex Agents)؛
- عامل‌های بازتابی مبتنی بر مدل (Model-Based Reflex Agents)؛
- عامل‌های مبتنی بر هدف (Goal-Based Agents)؛ و
- عامل‌های مبتنی بر مطلوبیت (Utility-Based Agents).

هر نوع برنامهٔ عامل، مؤلفه‌های خاصی را به شیوه‌های خاصی ترکیب می‌کند تا کنش تولید کند. بخش ۲.۴.۶ به‌طور کلی توضیح می‌دهد چگونه همهٔ این عامل‌ها را به عامل‌های یادگیرنده تبدیل کنیم که می‌توانند عملکرد مؤلفه‌هایشان را بهبود دهند تا کنش‌های بهتری تولید کنند. سرانجام، بخش ۲.۴.۷ گوناگونیِ راه‌های بازنمایی خودِ مؤلفه‌ها درون عامل را توصیف می‌کند. این گوناگونی اصل سازمان‌دهندهٔ اصلی این رشته و خود کتاب را فراهم می‌کند.

۲.۴.۲ عامل‌های بازتابی ساده

ساده‌ترین نوع عامل، عامل بازتابی ساده است. این عامل‌ها کنش‌ها را بر اساس ادراک حسی کنونی برمی‌گزینند و بقیهٔ تاریخچهٔ ادراک حسی را نادیده می‌گیرند. برای نمونه، عامل جاروبرقی که تابع عاملش در شکل ۲.۳ جدول‌بندی شده، عامل بازتابی ساده است، زیرا تصمیمش فقط بر مکان کنونی و وجود کثیفی در آن مکان استوار است. برنامهٔ عاملی برای این عامل در شکل ۲.۸ نشان داده شده.

توجه کنید برنامهٔ عامل جاروبرقی در مقایسه با جدول متناظر بسیار کوچک است. آشکارترین کاهش از نادیده‌گرفتن تاریخچهٔ ادراک حسی ناشی می‌شود که شمار توالی‌های ادراک حسیِ مرتبط را از 4^T به فقط ۴ کاهش می‌دهد. کاهش کوچک دیگری نیز از این واقعیت می‌آید که وقتی خانهٔ کنونی کثیف است، کنش به مکان بستگی ندارد. هرچند برنامهٔ عامل را با دستورهای if-then-else نوشته‌ایم، آن‌قدر ساده است که می‌توان آن را به‌صورت مدار بولی هم پیاده کرد.

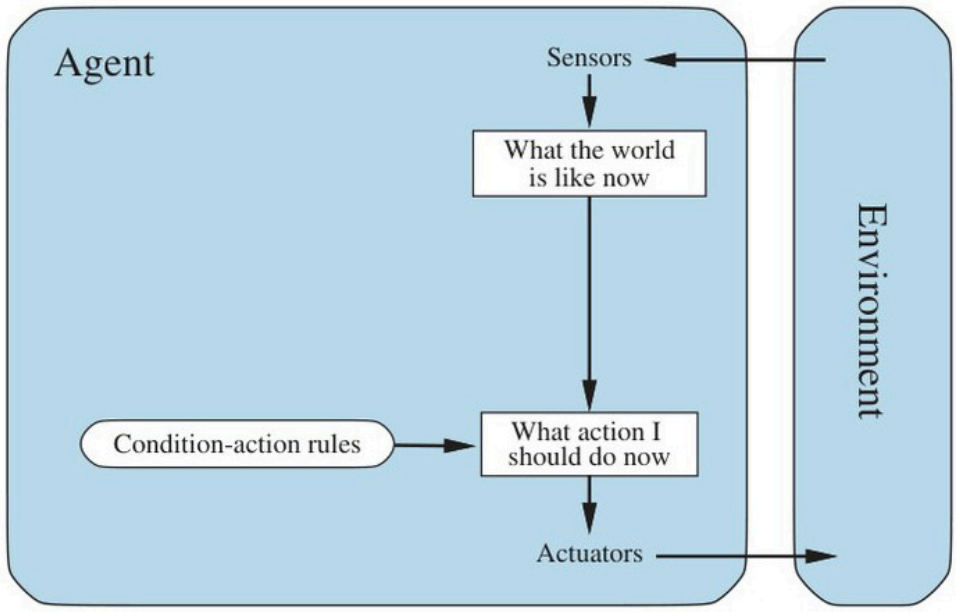
رفتارهای بازتابی ساده حتی در محیط‌های پیچیده‌تر هم رخ می‌دهند. خودتان را رانندهٔ تاکسی خودکار تصور کنید. اگر خودروی جلویی ترمز کند و چراغ‌های ترمزش روشن شود، باید متوجه شوید و ترمز را آغاز کنید. به بیان دیگر، پردازشی روی ورودی بصری انجام می‌شود تا شرطی که «خودروی جلویی در حال ترمز است» می‌نامیم برقرار شود. سپس این، اتصالی ازپیش‌ساخته را در برنامهٔ عامل به کنشِ «آغاز ترمز» برمی‌انگیزد. چنین اتصالی را قاعدهٔ شرط–کنش (Condition–Action Rule) می‌نامیم[6] که چنین نوشته می‌شود:

```
if car-in-front-is-braking then initiate-braking
```

(اگر خودروی جلویی در حال ترمز است، ترمز را آغاز کن)

انسان‌ها نیز بسیاری چنین اتصال‌هایی دارند؛ برخی پاسخ‌های آموخته‌شده‌اند (مانند رانندگی) و برخی بازتاب‌های فطری (مانند پلک‌زدن وقتی چیزی به چشم نزدیک می‌شود). در طول کتاب، چند راه گوناگون یادگیری و پیاده‌سازی چنین اتصال‌هایی را نشان می‌دهیم.

برنامهٔ شکل ۲.۸ ویژهٔ یک محیط جاروبرقی خاص است. رویکرد عمومی‌تر و انعطاف‌پذیرتر آن است که نخست مفسری همه‌منظوره برای قواعد شرط–کنش بسازیم و سپس مجموعه‌قواعدی برای محیط‌های تکلیف خاص ایجاد کنیم. شکل ۲.۹ ساختار این برنامهٔ عمومی را به‌صورت شماتیک نشان می‌دهد و می‌نمایاند قواعد شرط–کنش چگونه به عامل امکان می‌دهند از ادراک حسی به کنش برسد. اگر این پیش‌پاافتاده به نظر می‌رسد نگران نباشید؛ به‌زودی جالب‌تر می‌شود.



شکل ۲.۹ — نمودار شماتیک عامل بازتابی ساده. برای نمایش حالت درونی کنونیِ فرایند تصمیم‌گیریِ عامل از مستطیل‌ها و برای نمایش اطلاعات زمینه‌ایِ به‌کاررفته در فرایند از بیضی‌ها استفاده می‌کنیم. (مؤلفه‌ها: حسگرها، «اکنون جهان چگونه است»، قواعد شرط–کنش، «اکنون چه کنشی باید انجام دهم»، عملگرها)

برنامهٔ عاملِ شکل ۲.۹ در شکل ۲.۱۰ آمده است.

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
    persistent: rules, a set of condition-action rules
                          state ← INTERPRET-INPUT(percept)
                           rule ← RULE-MATCH(state, rules)
                                  action ← rule.ACTION
                                         return action
```

شکل ۲.۱۰ — عامل بازتابی ساده. بر اساس قاعده‌ای عمل می‌کند که شرطش با حالت کنونیِ تعریف‌شده توسط ادراک حسی همخوانی دارد.

تابع INTERPRET-INPUT از ادراک حسی، توصیفی انتزاعی از حالت کنونی تولید می‌کند و تابع RULE-MATCH نخستین قاعده در مجموعهٔ قواعد را بازمی‌گرداند که با توصیف حالت داده‌شده همخوانی دارد. توجه کنید توصیف بر حسب «قواعد» و «همخوانی» صرفاً مفهومی است؛ چنان‌که پیش‌تر یادآوری شد، پیاده‌سازی‌های واقعی می‌توانند به سادگی مجموعه‌ای از گیت‌های منطقیِ پیاده‌کنندهٔ مدار بولی باشند. از سوی دیگر، می‌توان مدار «عصبی» به کار گرفت، جایی که گیت‌های منطقی با واحدهای غیرخطی شبکه‌های عصبی مصنوعی جایگزین می‌شوند (فصل ۲۱ را ببینید).

عامل‌های بازتابی ساده ویژگی ستودنیِ سادگی دارند، اما هوششان محدود است. عامل شکل ۲.۱۰ فقط وقتی کار خواهد کرد که بتوان تصمیم درست را Δ صرفاً بر اساس ادراک حسی کنونی گرفت — یعنی فقط اگر محیط کاملاً قابل مشاهده باشد.

حتی کمی مشاهده‌ناپذیری می‌تواند دردسر جدی ایجاد کند. برای نمونه، قاعدهٔ ترمزِ پیش‌ترگفته فرض می‌کند شرطِ «خودروی جلویی در حال ترمز است» را می‌توان از ادراک حسی کنونی — یک فریم ویدئو — تعیین کرد. این وقتی کار می‌کند که خودروی جلویی چراغ ترمزِ مرکزی (و در نتیجه بی‌همتا قابل شناسایی) داشته باشد. متأسفانه، مدل‌های قدیمی‌تر پیکربندی‌های متفاوتی از چراغ‌های عقب، چراغ‌های ترمز و چراغ‌های راهنما دارند و همیشه نمی‌توان از یک تصویر تشخیص داد خودرو در حال ترمز است یا صرفاً چراغ‌های عقبش روشن است. عامل بازتابی ساده‌ای که پشت چنین خودرویی رانندگی کند یا پیوسته و بی‌نیاز ترمز می‌کند، یا بدتر، هرگز ترمز نمی‌کند.

می‌توانیم مسئلهٔ مشابهی را در جهان جاروبرقی ببینیم. فرض کنیم عامل بازتابی سادهٔ جاروبرقی از حسگر مکان محروم شده و فقط حسگر کثیفی دارد. چنین عاملی فقط دو ادراک حسی ممکن دارد: [Dirty] و [Clean]. در پاسخ به [Dirty] می‌تواند بمکد؛ در پاسخ به [Clean] چه باید بکند؟ حرکت به چپ اگر اتفاقاً از خانهٔ A شروع کرده باشد (برای همیشه) شکست می‌خورد و حرکت به راست اگر اتفاقاً از خانهٔ B شروع کرده باشد (برای همیشه) شکست می‌خورد. حلقه‌های بی‌نهایت برای عامل‌های بازتابی ساده‌ای که در محیط‌های تا حدی قابل مشاهده کار می‌کنند اغلب ناگزیرند.

گریز از حلقه‌های بی‌نهایت ممکن است اگر عامل بتواند کنش‌هایش را تصادفی کند (Randomization). برای نمونه، اگر عامل جاروبرقی [Clean] ادراک کند، ممکن است سکه بیندازد تا میان Right و Left انتخاب کند. به‌راحتی می‌توان نشان داد عامل در میانگین دو گام به خانهٔ دیگر می‌رسد. آنگاه اگر آن خانه کثیف باشد، عامل آن را تمیز می‌کند و تکلیف کامل می‌شود. از این رو، عامل بازتابی سادهٔ تصادفی‌شده ممکن است از عامل بازتابی سادهٔ جبرگرایانه بهتر عمل کند.

در بخش ۲.۳ یادآوری کردیم رفتار تصادفی از نوع درست می‌تواند در برخی محیط‌های چندعاملی عقلایی باشد. در محیط‌های تک‌عاملی، تصادفی‌سازی معمولاً عقلایی نیست. این حربهٔ مفیدی است که در برخی موقعیت‌ها به عامل بازتابی ساده کمک می‌کند، اما در بیشتر موارد با عامل‌های جبرگرایانهٔ پیچیده‌تر می‌توانیم بسیار بهتر عمل کنیم.

۲.۴.۳ عامل‌های بازتابی مبتنی بر مدل

مؤثرترین راه رسیدگی به مشاهده‌ناپذیریِ تا حدی، آن است که عامل بخشی از جهان را که اکنون نمی‌بیند پیگیری کند. یعنی عامل باید نوعی حالت درونی (Internal State) نگه دارد که به تاریخچهٔ ادراک حسی وابسته است و بدین ترتیب دست‌کم برخی از جنبه‌های مشاهده‌نشدهٔ حالت کنونی را بازتاب می‌دهد. برای مسئلهٔ ترمز، حالت درونی چندان گسترده نیست — فقط فریم قبلی دوربین، که به عامل امکان می‌دهد وقتی دو چراغ قرمز لبهٔ خودرو هم‌زمان روشن یا خاموش می‌شوند تشخیص دهد. برای دیگر تکلیف‌های رانندگی مانند تغییر خط، عامل باید پیگیری کند دیگر خودروها کجا هستند اگر نمی‌تواند همه را یک‌جا ببیند. و برای آنکه اصلاً رانندگی‌ای ممکن باشد، عامل باید پیگیری کند کلیدهایش کجاست.

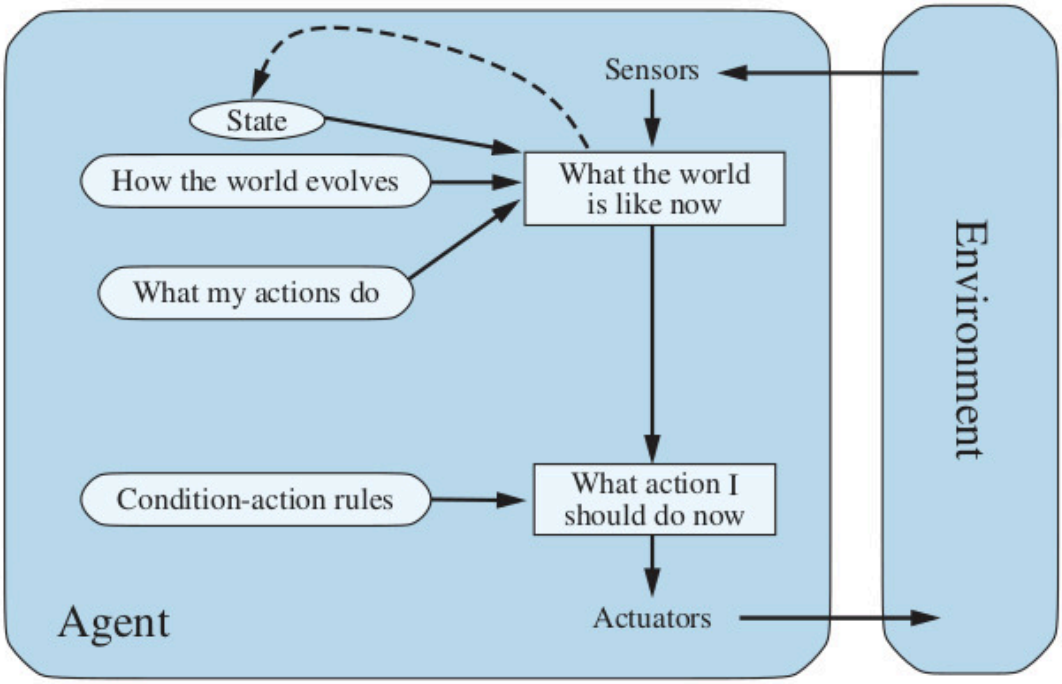
به‌روزرسانی این اطلاعات حالت درونی با گذر زمان مستلزم آن است دو نوع دانش در برنامهٔ عامل به شکلی رمزگذاری شود. نخست، به اطلاعاتی دربارهٔ چگونگی تغییر جهان در طول زمان نیاز داریم که تقریباً به دو بخش تقسیم می‌شود: اثرات کنش‌های عامل و چگونگی تکامل جهان مستقل از عامل. برای نمونه، وقتی عامل فرمان را در جهت عقربه‌های ساعت می‌چرخاند، خودرو به راست می‌پیچد، و وقتی باران می‌بارد، دوربین‌های خودرو می‌توانند خیس شوند. این دانش

دربارهٔ «جهان چگونه کار می‌کند» — چه در مدارهای بولی ساده پیاده شود چه در نظریه‌های علمی کامل — «مدل گذار» (Transition Model) جهان نامیده می‌شود. دوم، به اطلاعاتی دربارهٔ چگونگی بازتابِ حالت جهان در ادراک‌های حسی عامل نیاز داریم. برای نمونه، وقتی خودروی جلویی ترمز را آغاز می‌کند، یک یا چند ناحیهٔ قرمز روشن در تصویر دوربینِ رو به جلو ظاهر می‌شود، و وقتی دوربین خیس می‌شود، اشیای قطره‌شکل در تصویر ظاهر می‌شوند که تا حدی جاده را می‌پوشانند. این نوع دانش «مدل حسگر» (Sensor Model) نامیده می‌شود.

مدل گذار و مدل حسگر با هم به عامل امکان می‌دهند حالت جهان را — تا حد ممکن با توجه به محدودیت‌های حسگرهای عامل — پیگیری کند. عاملی که از چنین مدل‌هایی استفاده می‌کند، عامل مبتنی بر مدل (Model-Based Agent) نامیده می‌شود.

شکل ۲.۱۱ ساختار عامل بازتابی مبتنی بر مدل با حالت درونی را نشان می‌دهد: ادراک حسی کنونی با حالت درونی قدیمی ترکیب می‌شود تا توصیف به‌روزشدهٔ حالت کنونی بر اساس مدل عامل از چگونگی کارکرد جهان تولید شود. برنامهٔ عامل در شکل ۲.۱۲ آمده است. بخش جالب، تابع UPDATE-STATE است که مسئول ایجاد توصیف حالت درونی تازه است. جزئیات چگونگی بازنمایی مدل‌ها و حالت‌ها بسته به نوع محیط و فناوری خاصِ به‌کاررفته در طراحی عامل بسیار متفاوت است.

صرف‌نظر از نوع بازنماییِ به‌کاررفته، به‌ندرت ممکن است عامل حالت کنونیِ محیطِ تا حدی قابل مشاهده را دقیقاً تعیین کند. به‌جایش، جعبهٔ برچسب‌خوردهٔ «اکنون جهان چگونه است» (شکل ۲.۱۱) «بهترین حدس» عامل را بازنمایی می‌کند (یا گاهی بهترین حدس‌ها، اگر عامل چند امکان را در نظر بگیرد). برای نمونه، تاکسی خودکار ممکن است نتواند پشت کامیون بزرگی را که جلویش ایستاده ببیند و فقط بتواند حدس بزند چه چیزی باعث توقف شده. پس عدم قطعیت دربارهٔ حالت کنونی ممکن است ناگزیر باشد، اما عامل همچنان باید تصمیم بگیرد.



شکل ۲.۱۱ – عامل بازتابی مبتنی بر مدل. (مؤلفه‌ها: حسگرها، «اکنون جهان چگونه است»، «جهان چگونه تکامل می‌یابد»، «کنش‌های من چه می‌کنند»، قواعد شرط–کنش، حالت، عملگرها)

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
                                                        :persistent
                    state, the agent's current percept of the state of the world
transition model, a description of how the next state depends on the current state and
                                                                      action
 sensor model, a description of how the current state of the world is reflected in the
                                                              agent's percepts
                                              rules, a set of condition-action rules
                                    action, the most recent action, initially none
          state ← UPDATE-STATE(state, action, percept, transition model, sensor model)
                                                    rule ← RULE-MATCH(state, rules)
                                                          action ← rule.ACTION
                                                                 return action
```

شکل ۲.۱۲ – عامل بازتابی مبتنی بر مدل. حالت کنونی جهان را با استفاده از مدل درونی پیگیری می‌کند و سپس کنش را همان‌گونه که عامل بازتابی انتخاب می‌کند.

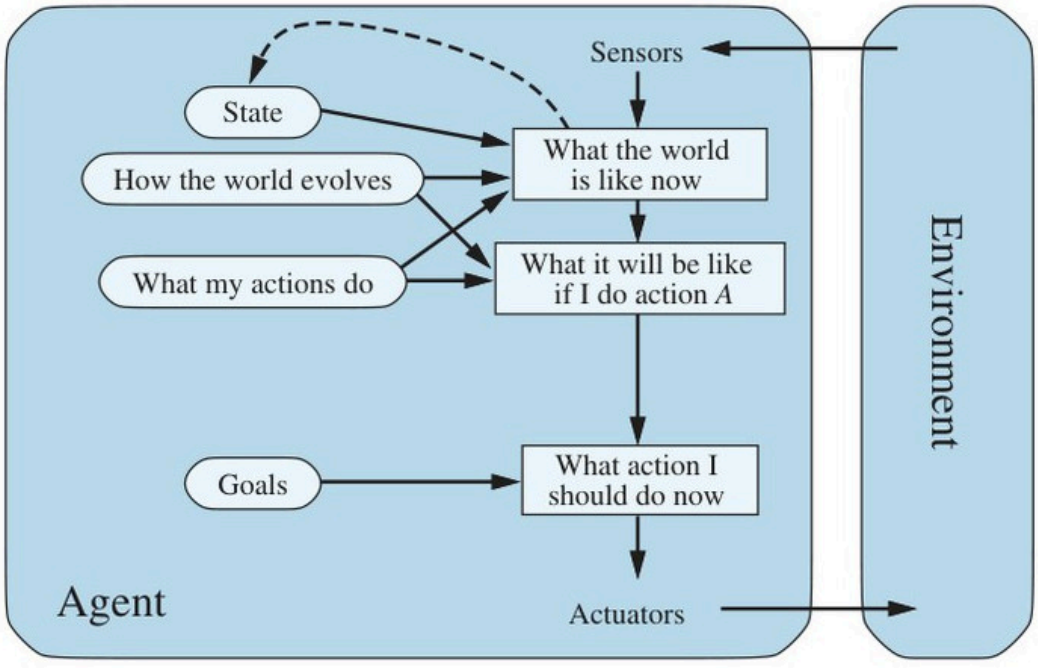
۲.۴.۴ عامل‌های مبتنی بر هدف

دانستن چیزی دربارهٔ حالت کنونی محیط همیشه برای تصمیم‌گیری دربارهٔ اینکه چه باید کرد کافی نیست. برای نمونه، در چهارراهی، تاکسی می‌تواند بپیچد چپ، بپیچد راست یا مستقیم برود. تصمیم درست بستگی دارد تاکسی می‌خواهد به کجا برسد. به بیان دیگر، افزون بر توصیف حالت کنونی، عامل به نوعی اطلاعات هدف (Goal) نیاز دارد که موقعیت‌های مطلوب را توصیف کند – برای نمونه، بودن در مقصدی خاص. برنامهٔ عامل می‌تواند این را با مدل (همان اطلاعاتی که در عامل بازتابی مبتنی بر مدل به کار رفت) ترکیب کند تا کنش‌هایی برگزیند که هدف را محقق می‌کنند.

شکل ۲.۱۳ ساختار عامل مبتنی بر هدف را نشان می‌دهد. گاهی انتخاب کنش مبتنی بر هدف سرراست است — برای نمونه، وقتی ارضای هدف بلافاصله از یک کنش واحد حاصل می‌شود. گاهی پیچ‌وتر خواهد بود — برای نمونه، وقتی عامل باید توالی‌های بلند پیچ‌وخم‌ها را در نظر بگیرد تا راهی برای رسیدن به هدف بیابد. جستجو (فصل‌های ۳ تا ۵) و برنامه‌ریزی (فصل ۱۱) زیرشاخه‌های هوش مصنوعی‌اند که به یافتن توالی‌های کنشی می‌پردازند که هدف‌های عامل را محقق می‌کنند.

توجه کنید تصمیم‌گیری از این نوع اساساً با قواعد شرط–کنشِ پیش‌ترتوصیف‌شده متفاوت است، زیرا درگیرِ در نظر گرفتن آینده است — هم «اگر فلان کار را بکنم چه اتفاقی می‌افتد؟» و هم «آیا آن مرا خوشحال می‌کند؟» در طراحی‌های عامل بازتابی، این اطلاعات به‌صراحت بازنمایی نمی‌شود، زیرا قواعد تعبیه‌شده مستقیماً از ادراک‌های حسی به کنش‌ها نگاشت می‌شوند. عامل بازتابی وقتی چراغ ترمز می‌بیند ترمز می‌کند، و تمام. هیچ ایده‌ای از چرایی ندارد. عامل مبتنی بر هدف وقتی چراغ ترمز می‌بیند ترمز می‌کند، زیرا این تنها کنشی است که پیش‌بینی می‌کند هدفش یعنی نخوردن به خودروهای دیگر را محقق کند.

هرچند عامل مبتنی بر هدف کم‌کارآمدتر به نظر می‌رسد، انعطاف‌پذیرتر است، زیرا دانشِ پشتیبان تصمیم‌هایش به‌صراحت بازنمایی شده و قابل اصلاح است. برای نمونه، رفتار عامل مبتنی بر هدف را می‌توان به‌راحتی برای رفتن به مقصدی متفاوت تغییر داد، صرفاً با مشخص کردن آن مقصد به‌عنوان هدف. قواعد عامل بازتابی برای زمان پیچیدن و مستقیم‌رفتن فقط برای یک مقصد کار می‌کنند؛ برای رفتن به جای تازه باید همه‌شان جایگزین شوند.



شکل ۲.۱۳ — عامل مبتنی بر مدل و مبتنی بر هدف. حالت جهان و نیز مجموعه‌ای از هدف‌هایی که می‌کوشد محقق کند را پیگیری می‌کند و کنشی را برمی‌گزیند که (سرانجام) به تحقق هدف‌هایش بینجامد. (مؤلفه‌ها: حسگرها، «اکنون جهان چگونه است»، «جهان چگونه تکامل می‌یابد»، «کنش‌های من چه می‌کنند»، هدف‌ها، «اگر کنش A را انجام دهم چه خواهد شد»، «اکنون چه کنشی باید انجام دهم»، عملگرها)

۲.۴.۵ عامل‌های مبتنی بر مطلوبیت

هدف‌ها به تنهایی برای تولید رفتار باکیفیت در بیشتر محیط‌ها کافی نیستند. برای نمونه، بسیاری از توالی‌های کنش تاکسی را به مقصد می‌رسانند (و بدین ترتیب هدف را محقق می‌کنند)، اما برخی سریع‌تر، ایمن‌تر، قابلاتکاتر یا ارزان‌تر از دیگران‌اند. هدف‌ها فقط تمایز دودویی خامی میان حالت‌های «خوشحال» و «ناخوشحال» فراهم می‌کنند. معیار عملکرد عمومی‌تر باید امکان مقایسهٔ حالت‌های جهانِ گوناگون را بر اساس میزان دقیقِ خوشحالی‌ای که به عامل می‌دهند فراهم کند. چون «خوشحال» چندان علمی به نظر نمی‌رسد، اقتصاددانان و دانشمندان رایانه به‌جایش از اصطلاح مطلوبیت (Utility) استفاده می‌کنند.[7]

پیش‌تر دیدیم معیار عملکرد به هر توالی داده‌شده از حالت‌های محیط امتیازی نسبت می‌دهد، پس به‌راحتی می‌تواند میان راه‌های کم‌وبیش مطلوب رسیدن به مقصد تاکسی تمایز بگذارد. تابع مطلوبیت (Utility Function) عامل اساساً درونی‌سازی معیار عملکرد است. به شرط آنکه تابع مطلوبیت درونی و معیار عملکرد بیرونی هم‌نظر باشند، عاملی که کنش‌ها را برای بیشینه‌کردن مطلوبیتش برمی‌گزیند، بر اساس معیار عملکرد بیرونی عقلایی خواهد بود.

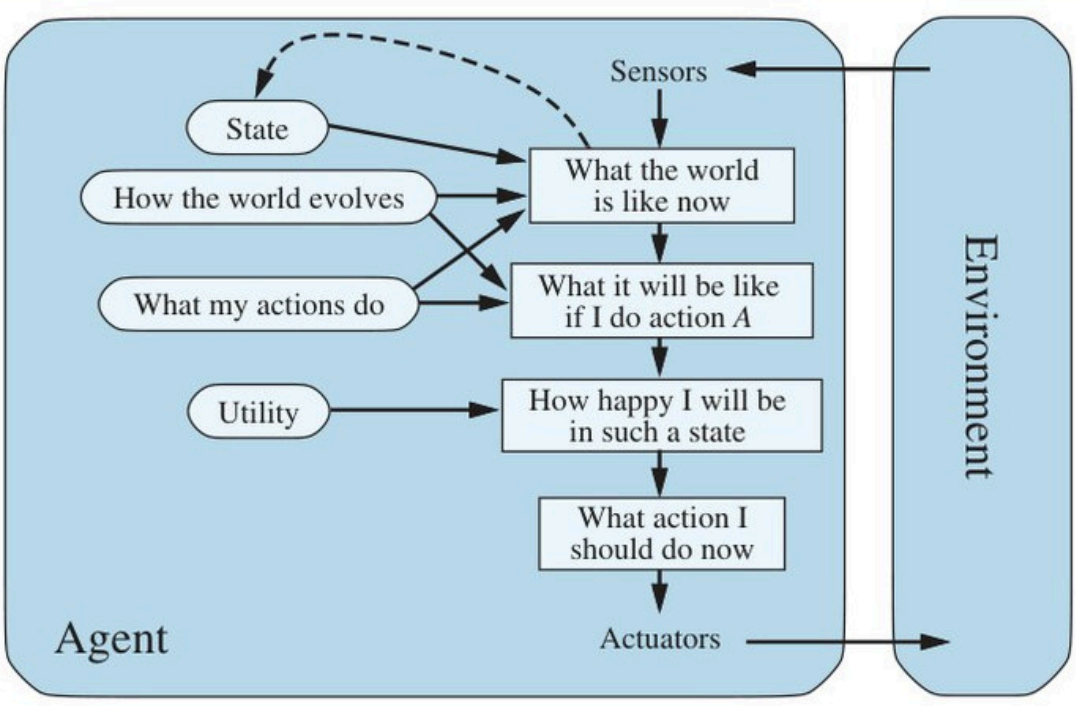
بگذارید دوباره تأکید کنیم این تنها راه عقلایی بودن نیست — پیش‌تر برنامهٔ عامل عقلایی برای جهان جاروبرقی (شکل ۲.۸) را دیدیم که هیچ ایده‌ای از تابع مطلوبیتش ندارد — اما مانند عامل‌های مبتنی بر هدف، عامل مبتنی بر مطلوبیت مزیت‌های فراوانی در انعطاف‌پذیری و یادگیری دارد. وانگهی، در دو نوع مورد، هدف‌ها ناکافی‌اند اما عامل مبتنی بر مطلوبیت همچنان می‌تواند تصمیم‌های عقلایی بگیرد. نخست، وقتی هدف‌های متعارض وجود دارند که فقط برخی قابل تحقق‌اند (مثلاً سرعت و ایمنی)، تابع مطلوبیت دادوستد مناسب را مشخص می‌کند. دوم، وقتی چند هدف هست که عامل می‌تواند دنبال کند و هیچ‌کدام با قطعیت قابل تحقق نیست، مطلوبیت راهی فراهم می‌کند که در آن احتمال موفقیت را در برابر اهمیت هدف‌ها سنجید.

مشاهده‌ناپذیری تا حدی و ناجبرگرایی در جهان واقعی فراگیرند و بنابراین تصمیم‌گیری در شرایط عدم قطعیت نیز فراگیر است. به بیان فنی، عامل عقلاییِ مبتنی بر مطلوبیت کنشی را برمی‌گزیند که مطلوبیت مورد انتظار (Expected Utility) پیامدهای کنش را بیشینه می‌کند — یعنی مطلوبیتی که عامل انتظار دارد به‌طور میانگین به دست آورد، با توجه به احتمال‌ها و مطلوبیت‌های هر پیامد. (پیوست الف امید ریاضی را دقیق‌تر تعریف می‌کند.) در فصل ۱۶ نشان می‌دهیم هر عامل عقلایی باید چنان رفتار کند که گویی تابع مطلوبیتی دارد که می‌کوشد مقدار مورد انتظارش را بیشینه کند. عاملی که تابع مطلوبیت صریح دارد می‌تواند با الگوریتمی همه‌منظوره تصمیم‌های عقلایی بگیرد که به تابع مطلوبیتِ خاصِ در حال بیشینه‌شدن وابسته نیست. بدین ترتیب، تعریف «سراسری» عقلانیت — عقلایی‌دانستن آن تابع‌های عاملی که بیشترین عملکرد را دارند — به قیدی «محلی» بر طراحی عامل‌های عقلایی بدل می‌شود که می‌توان در برنامه‌ای ساده بیان کرد.

ساختار عامل مبتنی بر مطلوبیت در شکل ۲.۱۴ آمده است. برنامه‌های عامل مبتنی بر مطلوبیت در فصل‌های ۱۶ و ۱۷ می‌آیند، جایی که عامل‌های تصمیم‌گیری را طراحی می‌کنیم که باید عدم قطعیت نهفته در محیط‌های ناجبرگرایانه یا تا حدی قابل مشاهده را مدیریت کنند. تصمیم‌گیری در محیط‌های چندعاملی نیز در چارچوب نظریهٔ مطلوبیت مطالعه می‌شود، چنان‌که در فصل ۱۸ توضیح داده شده است.

در این نقطه، خواننده ممکن است بپرسد: «آیا این‌قدر ساده است؟ فقط عامل‌هایی می‌سازیم که مطلوبیت مورد انتظار را بیشینه می‌کنند و تمام؟» درست است که چنین عامل‌هایی هوشمند خواهند بود، اما ساده نیست. عامل مبتنی بر مطلوبیت باید محیطش را مدل و پیگیری کند؛ تکلیف‌هایی که پژوهش فراوانی در ادراک، بازنمایی، استدلال و یادگیری طلبیده‌اند. نتایج این پژوهش بسیاری از فصل‌های این کتاب را پر می‌کنند. انتخاب مسیر کنشِ بیشینه‌کنندهٔ مطلوبیت نیز تکلیفی دشوار است و الگوریتم‌های هوشمندانه‌ای می‌طلبد که چند فصل دیگر را پر می‌کنند. حتی با این الگوریتم‌ها، عقلانیت کامل معمولاً در عمل به‌سبب پیچیدگی محاسباتی دست‌نیافتنی است، چنان‌که در فصل ۱ یادآوری کردیم. همچنین یادآوری می‌کنیم همهٔ عامل‌های مبتنی بر مطلوبیت مبتنی بر مدل نیستند؛ در فصل‌های ۲۲ و ۲۶ خواهیم دید عامل بدون مدل (Model-Free Agent) می‌تواند بیاموزد در موقعیتی خاص چه کنشی بهترین است، بدون آنکه هرگز دقیقاً بیاموزد آن کنش چگونه محیط را تغییر می‌دهد.

سرانجام، همهٔ این‌ها فرض می‌کنند طراح می‌تواند تابع مطلوبیت را درست مشخص کند؛ فصل‌های ۱۷، ۱۸ و ۲۲ مسئلهٔ توابع مطلوبیت ناشناخته را با ژرفای بیشتری بررسی می‌کنند.



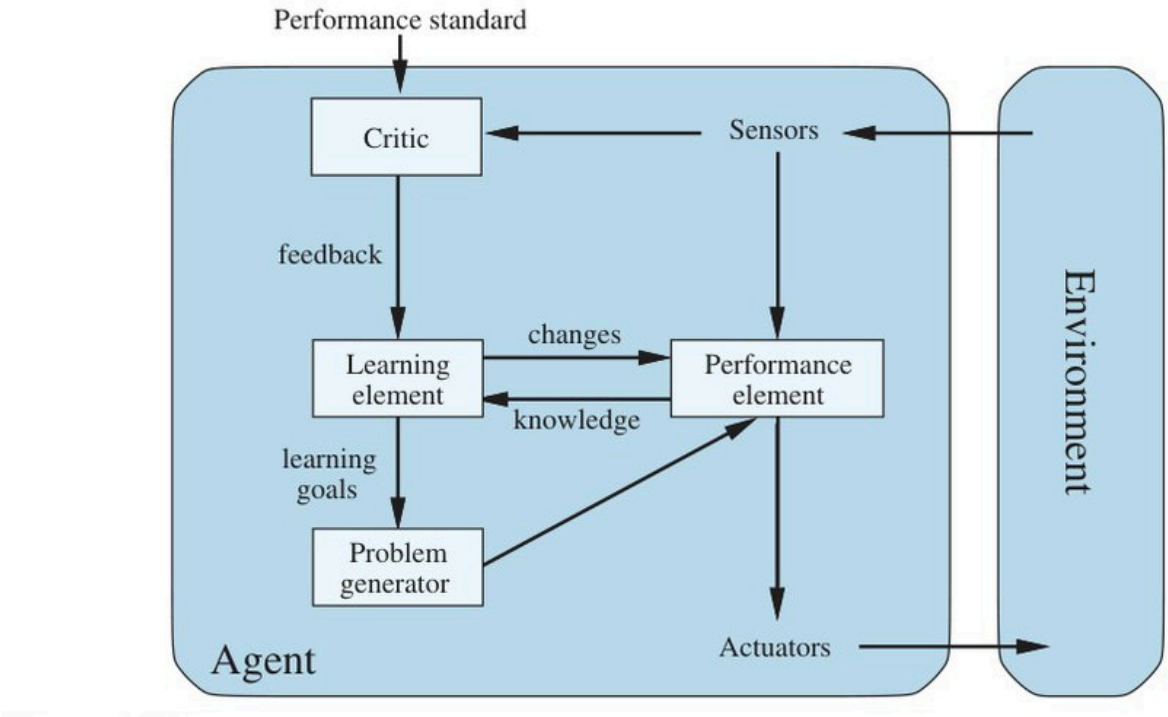
شکل ۲.۱۴ – عامل مبتنی بر مدل و مبتنی بر مطلوبیت. از مدلی از جهان به‌همراه تابع مطلوبیتی که ترجیحاتش را میان حالت‌های جهان می‌سنجد استفاده می‌کند. سپس کنشی را برمی‌گزیند که به بهترین مطلوبیت مورد انتظار بینجامد؛ مطلوبیت مورد انتظار با میانگین‌گیری روی همهٔ حالت‌های پیامدیِ ممکن، وزن‌دهی‌شده با احتمال پیامد، محاسبه می‌شود. (مؤلفه‌ها: حسگرها، «اکنون جهان چگونه است»، «جهان چگونه تکامل می‌یابد»، «کنش‌های من چه می‌کنند»، مطلوبیت، «در چنین حالتی چقدر خوشحال خواهم بود»، «اگر کنش A را انجام دهم چه خواهد شد»، «اکنون چه کنشی باید انجام دهم»، عملگرها)

۲.۴.۶ عامل‌های یادگیرنده

برنامه‌های عامل را با روش‌های گوناگون انتخاب کنش توصیف کرده‌ایم. تاکنون توضیح نداده‌ایم برنامه‌های عامل چگونه به وجود می‌آیند. تورینگ (۱۹۵۰) در مقالهٔ مشهور آغازینش، ایدهٔ برنامه‌نویسی دستیِ ماشین‌های هوشمندش را بررسی می‌کند. او تخمین می‌زند این کار چقدر تلاش می‌برد و نتیجه می‌گیرد «به روشی سریع‌تر نیاز است». روشی که او

پیشنهاد می‌کند ساختن ماشین‌های یادگیرنده و سپس آموزش‌دادن آن‌هاست. در بسیاری از حوزه‌های هوش مصنوعی، این اکنون روش ترجیحیِ ایجاد سامانه‌های پیشرفته است. هر نوع عامل (مبتنی بر مدل، مبتنی بر هدف، مبتنی بر مطلوبیت و...) را می‌توان به‌صورت عامل یادگیرنده ساخت (یا نه). یادگیری مزیت دیگری هم دارد، چنان‌که پیش‌تر یادآوری کردیم: به عامل امکان می‌دهد در محیط‌های ناشناخته از آغاز عمل کند و شایسته‌تر از آن شود که دانش آغازینش به‌تنهایی ممکن می‌کند. در این بخش، ایده‌های اصلی عامل‌های یادگیرنده را به‌طور مختصر معرفی می‌کنیم. در سراسر کتاب، بر فرصت‌ها و روش‌های یادگیری در انواع خاص عامل‌ها نظر می‌دهیم. فصل‌های ۱۹ تا ۲۲ با ژرفای بسیار بیشتری به خودِ الگوریتم‌های یادگیری می‌پردازند.

عامل یادگیرنده را می‌توان به چهار مؤلفهٔ مفهومی تقسیم کرد، چنان‌که در شکل ۲.۱۵ نشان داده شده. مهم‌ترین تمایز میان «عنصر یادگیری» (Learning Element) است که مسئول بهبود است و «عنصر عملکرد» (Performance Element) که مسئول انتخاب کنش‌های بیرونی است. عنصر عملکرد همان چیزی است که پیش‌تر کل عامل می‌پنداشتیم: ادراک‌های حسی را می‌گیرد و دربارهٔ کنش‌ها تصمیم می‌گیرد. عنصر یادگیری از بازخوردِ «نقاد» (Critic) دربارهٔ عملکرد عامل استفاده می‌کند و تعیین می‌کند عنصر عملکرد چگونه باید اصلاح شود تا در آینده بهتر عمل کند.



شکل ۲.۱۵ – عامل یادگیرندهٔ عمومی. جعبهٔ «عنصر عملکرد» همان چیزی را نشان می‌دهد که پیش‌تر کل برنامهٔ عامل می‌پنداشتیم. اکنون، جعبهٔ «عنصر یادگیری» آن برنامه را اصلاح می‌کند تا عملکردش را بهبود دهد. (مؤلفه‌ها: معیار عملکرد، عنصر یادگیری، عنصر عملکرد، مولد مسئله، نقاد، حسگرها/عملگرها، بازخورد، «تغییرات»، «دانش»، «هدف‌های یادگیری»، «محرک»)

طراحی عنصر یادگیری بسیار به طراحی عنصر عملکرد وابسته است. هنگام تلاش برای طراحی عاملی که قابلیتی را یاد می‌گیرد، نخستین پرسش «چگونه این را به آن یاد بدهم؟» نیست، بلکه «عنصر عملکردِ من برای انجام این کار پس از آموختنِ آن از چه نوعی استفاده خواهد کرد؟» با داشتن طراحیِ عنصر عملکرد، می‌توان سازوکارهای یادگیری برای بهبود هر بخش از عامل ساخت.

نقاد به عنصر یادگیری می‌گوید عامل نسبت به معیار عملکردِ ثابت چقدر خوب عمل می‌کند. نقاد ضروری است، زیرا خودِ ادراک‌های حسی هیچ نشانه‌ای از موفقیت عامل نمی‌دهند. برای نمونه، برنامهٔ شطرنج ممکن است ادراک حسی دریافت کند که حریف را کیش‌ومات کرده، اما برای دانستن اینکه این چیز خوبی است به معیار عملکرد نیاز دارد؛ خودِ ادراک حسی این را نمی‌گوید. مهم است معیار عملکرد ثابت باشد. از نظر مفهومی، باید آن را کاملاً بیرون از عامل دانست، زیرا عامل نباید آن را برای هماهنگی با رفتار خودش اصلاح کند.

آخرین مؤلفهٔ عامل یادگیرنده «مولد مسئله» (Problem Generator) است. مسئول پیشنهاد کنش‌هایی است که به تجربه‌های تازه و پرمحتوا می‌انجامند. اگر عنصر عملکرد راه خودش را می‌رفت، پیوسته بهترین کنش‌ها را بر اساس آنچه می‌داند انجام می‌داد؛ اما اگر عامل حاضر باشد کمی کاوش کند و در کوتاه‌مدت شاید کنش‌هایی تا حدی فروبهینه انجام دهد، ممکن است کنش‌های بسیار بهتری برای بلندمدت کشف کند. کار مولد مسئله پیشنهاد این کنش‌های اکتشافی است. دانشمندان هنگام انجام آزمایش دقیقاً همین کار را می‌کنند. گالیله فکر نمی‌کرد انداختن سنگ از بالای برج پیزا به خودی خود ارزشمند است. او نمی‌خواست سنگ‌ها را بشکند یا مغز عابران بدشانس را آسیب بزند. هدفش اصلاح مغز خودش با شناسایی نظریهٔ بهتری دربارهٔ حرکت اشیا بود.

عنصر یادگیری می‌تواند در هر یک از مؤلفه‌های «دانش» نشان‌داده‌شده در نمودارهای عامل (شکل‌های ۲.۹، ۲.۱۱، ۲.۱۳ و ۲.۱۴) تغییر ایجاد کند. ساده‌ترین موارد شامل یادگیری مستقیم از توالی ادراک حسی است. مشاهدهٔ جفت‌های حالت‌های پیاپیِ محیط می‌تواند به عامل امکان دهد «کنش‌های من چه می‌کنند» و «جهان چگونه تکامل می‌یابد» را در پاسخ به کنش‌هایش یاد بگیرد. برای نمونه، اگر تاکسی خودکار هنگام رانندگی در جادهٔ خیس فشار ترمز معینی اعمال کند، به‌زودی خواهد فهمید چه میزان شتاب منفی واقعاً حاصل می‌شود و آیا از جاده سر می‌خورد یا نه. مولد مسئله ممکن است بخش‌هایی از مدل را که نیاز به بهبود دارند شناسایی و آزمایش‌هایی پیشنهاد کند، مانند آزمودن ترمزها روی سطوح جادهٔ مختلف در شرایط متفاوت.

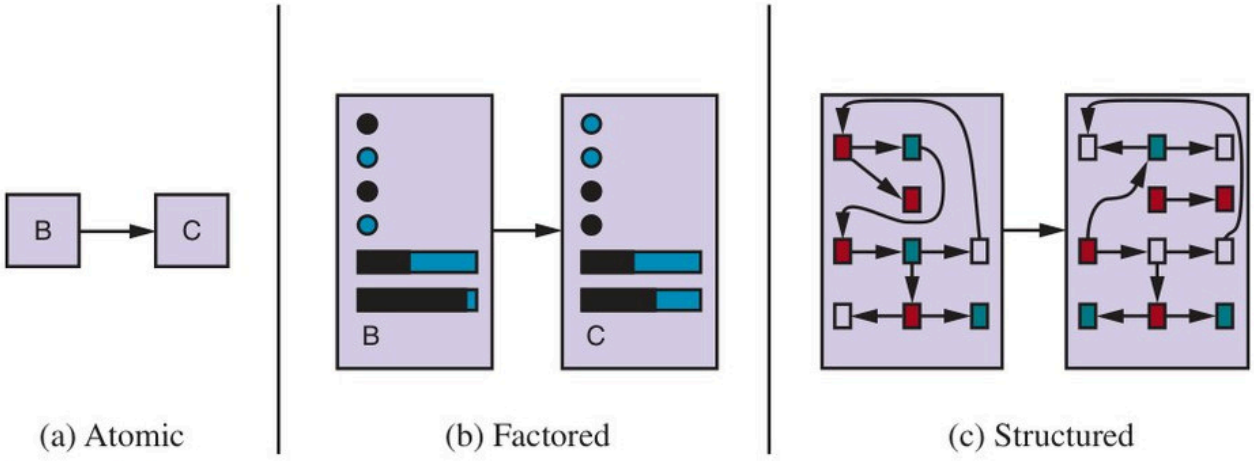
بهبود مؤلفه‌های مدلِ عامل مبتنی بر مدل تا هماهنگی بیشتر با واقعیت تقریباً همیشه ایدهٔ خوبی است، صرف‌نظر از معیار عملکرد بیرونی. (در برخی موارد، از دید محاسباتی بهتر است مدلی ساده اما اندکی نادقیق داشته باشیم تا مدلی کامل اما به‌طرز وحشتناکی پیچیده.) اطلاعاتِ معیار بیرونی وقتی لازم است که بخواهیم مؤلفهٔ بازتابی یا تابع مطلوبیت را یاد بگیریم. برای نمونه، فرض کنیم عامل رانندهٔ تاکسی از مسافرانِ در طول سفر به‌شدت تکان‌خورده انعامی نمی‌گیرد. معیار عملکرد بیرونی باید به عامل اطلاع دهد از دست‌دادنِ انعام سهمی منفی در عملکرد کلی‌اش است؛ آنگاه عامل ممکن است یاد بگیرد مانورهای خشن به مطلوبیت خودش کمکی نمی‌کنند. از جهتی، معیار عملکرد بخشی از ادراک حسیِ ورودی را به‌عنوان پاداش (Reward) (یا جریمه، Penalty) متمایز می‌کند که بازخورد مستقیمی دربارهٔ کیفیت رفتار عامل می‌دهد. معیارهای عملکرد سیم‌کشی‌شده مانند درد و گرسنگی در جانوران را می‌توان چنین فهمید.

به‌طور کلی‌تر، انتخاب‌های انسان می‌توانند اطلاعاتی دربارهٔ ترجیحات انسانی فراهم کنند. برای نمونه، فرض کنیم تاکسی نمی‌داند مردم عموماً صدای بلند را دوست ندارند و به این فکر می‌افتد برای اطمینان از اینکه عابران بدانند می‌آید، بی‌وقفه بوق بزند. رفتار انسانیِ برآمده — گرفتن گوش‌ها، فحش‌دادن و احتمالاً بریدن سیم بوق — شواهدی به عامل می‌دهد که با آن تابع مطلوبیتش را به‌روز کند. این موضوع در فصل ۲۲ بیشتر بحث می‌شود.

در خلاصه، عامل‌ها مؤلفه‌های گوناگونی دارند و آن مؤلفه‌ها را می‌توان به روش‌های فراوانی در برنامهٔ عامل بازنمایی کرد، پس به نظر می‌رسد میان روش‌های یادگیری گوناگونی بسیار وجود دارد. اما مضمون یکپارچه‌کنندهٔ واحدی هست. یادگیری در عامل‌های هوشمند را می‌توان به‌صورت فرایند اصلاحِ هر مؤلفهٔ عامل برای نزدیک‌کردن مؤلفه‌ها به اطلاعات بازخوردیِ در دسترس خلاصه کرد و بدین ترتیب عملکرد کلی عامل را بهبود داد.

۲.۴.۷ مؤلفه‌های برنامه‌های عامل چگونه کار می‌کنند

برنامه‌های عامل را (به‌زبانی بسیار سطح‌بالا) متشکل از مؤلفه‌های گوناگون توصیف کرده‌ایم که کارشان پاسخ به پرسش‌هایی مانند این‌هاست: «اکنون جهان چگونه است؟» «اکنون چه کنشی باید انجام دهم؟» «کنش‌های من چه می‌کنند؟» پرسش بعدی برای دانشجوی هوش مصنوعی این است: «این مؤلفه‌ها اصلاً چگونه کار می‌کنند؟» حدود هزار صفحه لازم است تا پاسخ درست به آن پرسش آغاز شود، اما این‌جا می‌خواهیم توجه خواننده را به برخی تمایزهای پایه میان راه‌های گوناگونی جلب کنیم که مؤلفه‌ها می‌توانند محیطِ سکونتِ عامل را بازنمایی کنند. به بیان تقریبی، می‌توان بازنمایی‌ها را بر امتداد محوری با پیچیدگی و قدرت بیانیِ فزاینده جای داد: اتمی (Atomic)، عاملی (Factored) و ساختاریافته (Structured). برای نشان دادن این ایده‌ها، در نظر گرفتن مؤلفه‌ای خاص مانند مؤلفهٔ «کنش‌های من چه می‌کنند» سودمند است. این مؤلفه تغییراتی را توصیف می‌کند که ممکن است در محیط در نتیجهٔ انجام کنشی رخ دهد و شکل ۲.۱۶ تصویرهای شماتیکی از چگونگی بازنمایی آن گذارها فراهم می‌کند.



شکل ۲.۱۶ – سه راه بازنمایی حالت‌ها و گذارهای میان آن‌ها. (الف) بازنمایی اتمی: حالت (مانند B یا C) جعبهٔ سیاهی بدون ساختار درونی است؛ (ب) بازنمایی عاملی: حالت از برداری از مقادیر ویژگی‌ها تشکیل شده؛ مقادیر می‌توانند بولی، حقیقی یا یکی از مجموعه‌ای ثابت از نمادها باشند؛ (ج) بازنمایی ساختاریافته: حالت شامل اشیا است که هر یک ممکن است ویژگی‌های خودش و نیز روابط با اشیای دیگر داشته باشد.

در بازنمایی اتمی (Atomic Representation) هر حالت جهان تقسیم‌ناپذیر است – ساختار درونی ندارد. تکلیف یافتن مسیر رانندگی از یک سر کشور تا سر دیگر از طریق توالی‌ای از شهرها را در نظر بگیرید (این مسئله را در شکل ۳.۱ در صفحهٔ ۶۴ بررسی می‌کنیم). برای حل این مسئله، ممکن است کافی باشد حالت جهان را به فقط نام شهری که در آن هستیم کاهش دهیم – اتمی از دانش، «جعبهٔ سیاهی» که تنها ویژگیِ تشخیص‌پذیرش یکسان‌بودن یا متفاوت‌بودن با جعبهٔ سیاه دیگر است. الگوریتم‌های استانداردِ زیربنای جستجو و بازی (فصل‌های ۳ تا ۵)، مدل‌های مارکوف پنهان (فصل ۱۴) و فرایندهای تصمیم مارکوف (فصل ۱۷) همگی با بازنمایی‌های اتمی کار می‌کنند.

بازنمایی عاملی (Factored Representation) هر حالت را به مجموعه‌ای ثابت از متغیرها یا ویژگی‌ها (Variable/Attribute) تقسیم می‌کند که هر یک می‌تواند مقداری (Value) داشته باشد. توصیف وفادارترِ همان مسئلهٔ رانندگی را در نظر بگیرید، جایی که باید به چیزهای بیشتری از مکان اتمیِ در شهری‌بودن یا نبودن اهمیت دهیم؛ شاید لازم باشد به این‌ها توجه کنیم: چقدر بنزین در باک است، مختصات GPS کنونی‌مان، اینکه چراغ هشدار روغن کار می‌کند یا نه، چقدر پول برای عوارض داریم، چه ایستگاه رادیویی روشن است، و مانند آن. دو حالت اتمیِ متفاوت هیچ اشتراکی ندارند — فقط جعبه‌های سیاه متفاوت‌اند — اما دو حالت عاملیِ متفاوت می‌توانند برخی ویژگی‌ها را مشترک داشته باشند (مانند بودن در مختصات GPS خاصی) و برخی را نه (مانند بنزین فراوان یا بی‌بنزین)؛ این تبدیل یکی به دیگری را بسیار آسان‌تر می‌کند. حوزه‌های مهم بسیاری از هوش مصنوعی بر بازنمایی‌های عاملی مبتنی‌اند، از جمله الگوریتم‌های ارضای قیود (فصل ۶)، منطق گزاره‌ای (فصل ۷)، برنامه‌ریزی (فصل ۱۱)، شبکه‌های بیزی (فصل‌های ۱۲ تا ۱۶) و الگوریتم‌های گوناگون یادگیری ماشین.

برای بسیاری مقاصد، باید جهان را دارای چیزهایی بدانیم که با هم رابطه دارند، نه فقط متغیرهایی با مقادیر. برای نمونه، ممکن است متوجه شویم کامیون بزرگی در جلوی ما در حال دنده‌عقب‌رفتن به سمت ورودی مزرعهٔ لبنی است، اما گاوی ول می‌راه کامیون را مسدود کرده. بازنمایی عاملی به‌احتمال زیاد از پیش به ویژگیِ «کامیون جلویی در حال دنده‌عقب به ورودی مزرعهٔ لبنی، مسدودشده با گاو ول» با مقدار true یا false مجهز نیست. در عوض، به بازنمایی ساختاریافته (Structured Representation) نیاز داریم که در آن اشیایی مانند گاوها و کامیون‌ها و روابط گوناگون و متغیرشان را بتوان به‌صراحت توصیف کرد (شکل ۲.۱۶(ج) را ببینید). بازنمایی‌های ساختاریافته زیربنای پایگاه‌های دادهٔ رابطه‌ای و منطق مرتبهٔ اول (فصل‌های ۸، ۹ و ۱۰)، مدل‌های احتمال مرتبهٔ اول (فصل ۱۵) و بخش بزرگی از فهم زبان طبیعی (فصل‌های ۲۳ و ۲۴) هستند. در واقع، بخش بزرگی از آنچه انسان‌ها به زبان طبیعی بیان می‌کنند به اشیا و روابط آن‌ها مربوط است.

چنان‌که پیش‌تر یادآوری کردیم، محوری که بازنمایی‌های اتمی، عاملی و ساختاریافته بر آن قرار دارند، محور بیانگری (Expressiveness) فزاینده است. به بیان تقریبی، بازنمایی بیانگرتر می‌تواند دست‌کم به همان اندازهٔ موجز، هر آنچه را بازنمایی کم‌بیان‌تر می‌تواند بگیرد، بگیرد، به‌علاوهٔ چیزهای بیشتر. اغلب زبان بیانگرتر بسیار موجزتر است؛ برای نمونه، قواعد شطرنج را می‌توان در یک‌دو صفحه از زبان بازنمایی ساختاریافته مانند منطق مرتبهٔ اول نوشت، اما هزاران صفحه لازم است اگر با زبان بازنمایی عاملی مانند منطق گزاره‌ای نوشته شود و حدود 10^{38} صفحه اگر با زبان اتمی مانند اتوماتای متناهی. از سوی دیگر، استدلال و یادگیری با افزایش قدرت بیانی بازنمایی پیچیده‌تر می‌شوند. برای بهره‌بردن از مزایای بازنمایی‌های بیانگر و پرهیز از معایبشان، سامانه‌های هوشمندِ جهان واقعی ممکن است نیاز داشته باشند هم‌زمان در همهٔ نقاط محور عمل کنند.

محور دیگری برای بازنمایی، نگاشت مفهوم‌ها به مکان‌های حافظهٔ فیزیکی است، چه در رایانه چه در مغز. اگر نگاشت یک‌به‌یک میان مفهوم‌ها و مکان‌های حافظه باشد، آن را بازنمایی مکان‌گرا (Localist Representation) می‌نامیم. از سوی دیگر، اگر بازنمایی یک مفهوم روی مکان‌های حافظهٔ بسیاری پخش شده باشد و هر مکان حافظه به‌عنوان بخشی از بازنمایی چند مفهوم متفاوت به کار رود، آن را بازنمایی توزیع‌شده (Distributed Representation) می‌نامیم. بازنمایی‌های توزیع‌شده در برابر نویز و ازدست‌رفتن اطلاعات استوارترند. با بازنمایی مکان‌گرا، نگاشت مفهوم به مکان

حافظه دلبخواهی است و اگر خطای انتقالی چند بیت را خراب کند، ممکن است «کامیون» (Truck) را با مفهوم نامرتبط «آتش‌بس» (Truce) اشتباه بگیریم. اما با بازنمایی توزیع‌شده، می‌توانید هر مفهوم را نقطه‌ای در فضای چندبعدی بپندارید و اگر چند بیت خراب شود، به نقطه‌ای نزدیک در آن فضا می‌روید که معنای مشابهی دارد.

خلاصهٔ فصل

این فصل تا حدی تور گردبادی در هوش مصنوعی بود که آن را علم طراحی عامل تصور کرده‌ایم. نکته‌های اصلی که باید به خاطر بسپارید چنین‌اند:

- عامل چیزی است که در محیطی ادراک و کنش می‌کند. تابع عاملِ یک عامل، کنشی را که عامل در پاسخ به هر توالی ادراک حسی انجام می‌دهد مشخص می‌کند.
- معیار عملکرد رفتار عامل را در محیط ارزیابی می‌کند. عامل عقلایی چنان کنش می‌کند که مقدار مورد انتظار معیار عملکرد را بیشینه کند، با توجه به توالی ادراک حسی‌ای که تاکنون دیده.
- مشخصات محیط تکلیف شامل معیار عملکرد، محیط بیرونی، عملگرها و حسگرهاست. در طراحی عامل، نخستین گام باید همیشه مشخص کردن هرچه کامل‌تر محیط تکلیف باشد.
- محیط‌های تکلیف در چند بُعد مهم متفاوت‌اند. آن‌ها می‌توانند کاملاً یا تا حدی قابل مشاهده، تک‌عاملی یا چندعاملی، جبرگرایانه یا ناجبرگرایانه، تکرارشونده یا توالی‌دار، ایستا یا پویا، گسسته یا پیوسته، و شناخته‌شده یا ناشناخته باشند.
- در مواردی که معیار عملکرد ناشناخته یا به‌سختی قابل‌مشخص‌کردن درست است، ریسک چشمگیری وجود دارد که عامل هدفِ اشتباهی را بهینه کند. در چنین مواردی طراحی عامل باید عدم قطعیت دربارهٔ هدف واقعی را بازتاب دهد.
- برنامهٔ عامل تابع عامل را پیاده‌سازی می‌کند. گستره‌ای از طراحی‌های پایهٔ برنامهٔ عامل وجود دارد که نوع اطلاعاتِ صریح‌شده و به‌کاررفته در فرایند تصمیم را بازتاب می‌دهند. طراحی‌ها در کارایی، فشردگی و انعطاف‌پذیری متفاوت‌اند. طراحی مناسب برنامهٔ عامل به سرشت محیط بستگی دارد.
- عامل‌های بازتابی ساده مستقیماً به ادراک‌های حسی پاسخ می‌دهند، حال آنکه عامل‌های بازتابی مبتنی بر مدل حالت درونی نگه می‌دارند تا جنبه‌هایی از جهان را که در ادراک حسی کنونی هویدا نیستند پیگیری کنند. عامل‌های مبتنی بر هدف برای رسیدن به هدف‌هایشان کنش می‌کنند و عامل‌های مبتنی بر مطلوبیت می‌کوشند «خوشحالی» مورد انتظار خودشان را بیشینه کنند.
- همهٔ عامل‌ها می‌توانند عملکردشان را از طریق یادگیری بهبود دهند.

یادداشت‌های کتاب‌شناختی و تاریخی

نقش مرکزی کنش در هوش — مفهوم استدلال عملی — دست‌کم به «اخلاق نیکوماخوسی» ارسطو بازمی‌گردد. استدلال عملی همچنین موضوع مقالهٔ تأثیرگذار مک‌کارتی، «برنامه‌هایی با فهم عقل سلیم» (۱۹۵۸) بود. حوزه‌های رباتیک و نظریهٔ کنترل، به‌ماهیت‌خود، عمدتاً به عامل‌های فیزیکی می‌پردازند. مفهوم کنترل‌گر (Controller) در نظریهٔ کنترل با مفهوم عامل در هوش مصنوعی یکسان است. شاید شگفت‌آور باشد که هوش مصنوعی در بیشتر تاریخش بر مؤلفه‌های منزویِ عامل متمرکز بوده — سامانه‌های پرسش‌وپاسخ، اثبات‌کننده‌های قضیه، سامانه‌های بینایی و مانند آن — نه بر عامل‌های کامل. بحث عامل‌ها در متن جنزِرت و نیلسون (۱۹۸۷) استثنایی تأثیرگذار بود. دیدگاه عامل کامل اکنون به‌طور گسترده پذیرفته شده و مضمون مرکزی کتاب‌های اخیر است (پدگام و وینیکاف، ۲۰۰۴؛ جونز، ۲۰۰۷؛ پول و مک‌ورث، ۲۰۱۷).

فصل ۱ ریشه‌های مفهوم عقلانیت را در فلسفه و اقتصاد دنبال کرد. در هوش مصنوعی، این مفهوم تا اواسط دههٔ ۱۹۸۰ اهمیت حاشیه‌ای داشت؛ آن‌گاه آغاز به فراگرفتنِ بسیاری از بحث‌ها دربارهٔ بنیان‌های فنی درستِ رشته کرد. مقاله‌ای از جان دویل (۱۹۸۳) پیش‌بینی کرد طراحی عامل عقلایی به‌عنوان رسالت اصلی هوش مصنوعی دیده خواهد شد، در حالی که دیگر موضوع‌های رایج به‌صورت رشته‌های تازه جدا خواهند شد.

توجه دقیق به ویژگی‌های محیط و پیامدهایشان برای طراحی عامل عقلایی در سنت نظریهٔ کنترل آشکارترین است — برای نمونه، سامانه‌های کنترل کلاسیک (دورف و بیشاپ، ۲۰۰۴؛ کِرک، ۲۰۰۴) محیط‌های کاملاً قابل مشاهده و جبرگرایانه را مدیریت می‌کنند؛ کنترل بهینهٔ تصادفی (کومار و وارایا، ۱۹۸۶؛ برتسکاس و شریو، ۲۰۰۷) محیط‌های تا حدی قابل مشاهده و تصادفی را؛ و کنترل هیبریدی (هنسینگر و ساستری، ۱۹۹۸؛ کاساندراس و لیگروس، ۲۰۰۶) با محیط‌های دارای عناصر گسسته و پیوسته سروکار دارد. تمایز محیط‌های کاملاً و تا حدی قابل مشاهده همچنین در ادبیات برنامه‌نویسی پویا که در رشتهٔ تحقیق در عملیات توسعه یافته مرکزی است (پوترمن، ۱۹۹۴)، که در فصل ۱۷ بحث می‌کنیم.

هرچند عامل‌های بازتابی ساده در روان‌شناسی رفتارگرا مرکزی بودند (فصل ۱ را ببینید)، بیشتر پژوهشگران هوش مصنوعی آن‌ها را برای اهرم‌داشتن بسیار ساده می‌دانند. (روزن‌شاین (۱۹۸۵) و بروکس (۱۹۸۶) این فرض را به پرسش گرفتند؛ فصل ۲۶ را ببینید.) کار فراوانی صرف یافتن الگوریتم‌های کارآمد برای پیگیری محیط‌های پیچیده شده است (بار-شالوم و همکاران، ۲۰۰۱؛ شوزه و همکاران، ۲۰۰۵؛ سایمون، ۲۰۰۶)، که بیشتر آن در چارچوب احتمالاتی است.

عامل‌های مبتنی بر هدف در همه‌چیز مفروض‌اند، از دیدگاه ارسطو دربارهٔ استدلال عملی تا مقاله‌های آغازین مک‌کارتی دربارهٔ هوش مصنوعی منطقی. ربات شیکی (فایکز و نیلسون، ۱۹۷۱؛ نیلسون، ۱۹۸۴) نخستین تجسم رباتیکِ عامل مبتنی بر هدفِ منطقی بود. تحلیل منطقی کامل عامل‌های مبتنی بر هدف در جنزِرت و نیلسون (۱۹۸۷) آمد و روش‌شناسی برنامه‌نویسی مبتنی بر هدف به نام برنامه‌نویسی عامل‌محور توسط شوهام (۱۹۹۳) توسعه یافت. رویکرد عامل‌محور اکنون در مهندسی نرم‌افزار بسیار رایج است (چیانکارینی و وولدریج، ۲۰۰۱). همچنین به حوزهٔ سامانه‌های عامل نفوذ کرده، جایی که محاسبات خودمختار (Autonomic Computing) به سامانه‌ها و شبکه‌های رایانه‌ای اشاره دارد که با حلقهٔ ادراک–کنش و روش‌های یادگیری ماشین خود را پایش و کنترل می‌کنند (کیهارت و چس، ۲۰۰۳). با یادآوری اینکه

مجموعه‌ای از برنامه‌های عاملِ طراحی‌شده برای کار خوب با هم در محیط چندعاملیِ واقعی ناگزیر پیمانه‌ای‌اند — برنامه‌ها حالت درونی مشترکی ندارند و فقط از طریق محیط با هم ارتباط برقرار می‌کنند — در حوزهٔ سامانه‌های چندعاملی رایج است برنامهٔ عاملِ یک عامل منفرد را به‌صورت مجموعه‌ای از زیرعامل‌های خودمختار طراحی کنند. در برخی موارد حتی می‌توان اثبات کرد سامانهٔ حاصل همان راه‌حل‌های بهینهٔ طراحی یکپارچه را می‌دهد.

دیدگاه مبتنی بر هدفِ عامل‌ها همچنین بر سنت روان‌شناسی شناختی در حوزهٔ حل مسئله مسلط است؛ از «حل مسئلهٔ انسانی» بسیار تأثیرگذار (نیوول و سایمون، ۱۹۷۲) آغاز شده و در همهٔ کارهای بعدی نیوول (۱۹۹۰) ادامه دارد. هدف‌ها — که بیشتر به‌صورت خواسته‌ها (عمومی) و نیت‌ها (در حال پیگیری) تحلیل می‌شوند — در نظریهٔ تأثیرگذار عاملِ مایکل براتمن (۱۹۸۷) مرکزی‌اند.

چنان‌که در فصل ۱ یادآوری شد، توسعهٔ نظریهٔ مطلوبیت به‌عنوان بنیانی برای رفتار عقلایی به صدها سال پیش بازمی‌گردد. در هوش مصنوعی، پژوهش آغازین مطلوبیت‌ها را به نفع هدف‌ها کنار گذاشت، با چند استثنا (فلدمن و اسپرول، ۱۹۷۷). احیای علاقه به روش‌های احتمالاتی در دههٔ ۱۹۸۰ به پذیرش بیشینه‌سازی مطلوبیت مورد انتظار به‌عنوان عمومی‌ترین چارچوب تصمیم‌گیری انجامید (هورویتز و همکاران، ۱۹۸۸). متن پرل (۱۹۸۸) نخستین متن هوش مصنوعی بود که نظریهٔ احتمال و مطلوبیت را با ژرفا پوشش داد؛ شرح او از روش‌های عملی استدلال و تصمیم‌گیری در شرایط عدم قطعیت احتمالاً بزرگ‌ترین عامل واحد در چرخش سریع به سوی عامل‌های مبتنی بر مطلوبیت در دههٔ ۱۹۹۰ بود (فصل ۱۶ را ببینید). صورت‌بندی یادگیری تقویتی در چارچوب نظریه‌تصمیم نیز به این چرخش کمک کرد (ساتن، ۱۹۸۸). تا حدی شگفت‌آور، تقریباً تمام پژوهش‌های هوش مصنوعی تا همین اواخر فرض می‌کرده‌اند معیار عملکرد را می‌توان در قالب تابع مطلوبیت یا تابع پاداش دقیق و درست مشخص کرد (هادفیلد-منل و همکاران، ۲۰۱۷؛ راسل، ۲۰۱۹).

طراحی عمومی عامل‌های یادگیرنده که در شکل ۲.۱۵ تصویر شده در ادبیات یادگیری ماشین کلاسیک است (بوکانن و همکاران، ۱۹۷۸؛ میچل، ۱۹۹۷). نمونه‌های این طراحی، آن‌چنان‌که در برنامه‌ها تجسم یافته، دست‌کم به برنامهٔ یادگیری بازی چکرزِ آرتور ساموئل (۱۹۵۹، ۱۹۶۷) بازمی‌گردد. عامل‌های یادگیرنده در فصل‌های ۱۹ تا ۲۲ با ژرفا بحث می‌شوند.

پانوشت‌ها:

[1] اگر عامل برای انتخاب کنش‌هایش از تصادفی‌سازی استفاده کند، باید هر توالی را بارها آزمایش کنیم تا احتمال هر کنش شناسایی شود. شاید تصور کنید کنش تصادفی احمقانه است، اما بعداً در همین فصل نشان می‌دهیم می‌تواند بسیار هوشمندانه باشد.

[2] در ربات واقعی، بعید است کنش‌هایی مانند «حرکت به راست» و «حرکت به چپ» وجود داشته باشد. در عوض کنش‌ها «چرخاندن چرخ‌ها به جلو» و «چرخاندن چرخ‌ها به عقب» خواهند بود. ما کنش‌ها را چنان برگزیده‌ایم که دنبال‌کردنشان در صفحه آسان‌تر باشد، نه برای سهولت پیاده‌سازی در ربات واقعی.

[3] به «نیاز به قفل‌های تازهٔ در برای جت‌های جامبو بوئینگ ۷۴۷» (واشینگتن پست، ۲۴ اوت ۱۹۸۹) مراجعه کنید.

[4] واژهٔ «توالی‌دار» در علوم رایانه نیز به‌عنوان متضاد «موازی» به کار می‌رود. این دو معنا تا حد زیادی بی‌ارتباط‌اند.

[5] انتخاب‌های دیگری هم برای اسکلت برنامهٔ عامل هست؛ برای نمونه، برنامه‌های عامل می‌توانند هم‌روال‌هایی باشند که به‌صورت ناهمگام با محیط اجرا می‌شوند.

[6] همچنین قاعدهٔ موقعیت–کنش، تولید (Production) یا قاعدهٔ اگر–آنگاه نامیده می‌شود.

[7] واژهٔ «مطلوبیت» در این‌جا به «کیفیت سودمند بودن» اشاره دارد، نه شرکت برق یا ادارهٔ آب.

فصل ۳: حل مسئله با جستجو (Solving Problems by Searching)

در این فصل می‌بینیم عامل چگونه می‌تواند به آینده بنگرد و توالی‌ای از کنش‌ها بیابد که سرانجام به هدفش می‌رسد.

وقتی کنشِ درستِ لازم بدیهی نباشد، عامل ممکن است نیاز داشته باشد پیش‌بینی کند: توالی‌ای از کنش‌ها را در نظر بگیرد که مسیری به یک حالت هدف می‌سازند. چنین عاملی «عامل حل‌کنندهٔ مسئله» (Problem-Solving Agent) نامیده می‌شود و فرایند محاسباتی‌ای که انجام می‌دهد «جستجو» (Search) است.

عامل‌های حل‌کنندهٔ مسئله از بازنمایی‌های اتمی استفاده می‌کنند، چنان‌که در بخش ۲.۴.۷ توصیف شد — یعنی حالت‌های جهان به‌صورت کل در نظر گرفته می‌شوند، بدون هیچ ساختار درونیِ قابل مشاهده برای الگوریتم‌های حل مسئله. عامل‌هایی که از بازنمایی‌های عاملی یا ساختاریافتهٔ حالت استفاده می‌کنند، عامل‌های برنامه‌ریز (Planning Agents) نامیده می‌شوند و در فصل‌های ۷ و ۱۱ بحث می‌شوند.

چندین الگوریتم جستجو را پوشش خواهیم داد. در این فصل فقط ساده‌ترین محیط‌ها را در نظر می‌گیریم: تکرارشونده، تک‌عاملی، کاملاً قابل مشاهده، جبرگرایانه، ایستا، گسسته و شناخته‌شده. میان الگوریتم‌های مطلع (Informed) — که در آن‌ها عامل می‌تواند تخمین بزند چقدر تا هدف فاصله دارد — و الگوریتم‌های نامطلع (Uninformed) — که چنین تخمینی در دسترس نیست — تمایز قائل می‌شویم.

فصل ۴ قیدهای محیطی را شل می‌کند و فصل ۵ عامل‌های چندگانه را در نظر می‌گیرد.

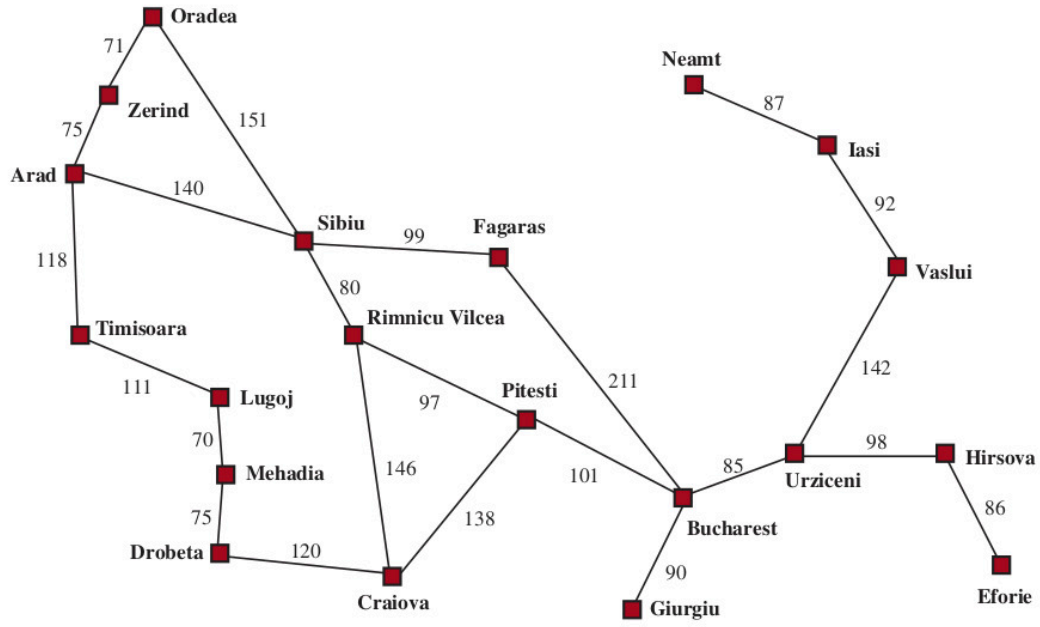
این فصل از مفاهیم پیچیدگی مجانبی (یعنی نماد $O(n)$) استفاده می‌کند. خوانندگانی که با این مفاهیم آشنا نیستند باید به پیوست الف مراجعه کنند.

۳.۱ عامل‌های حل‌کنندهٔ مسئله

عاملی را تصور کنید که در رومانی به گشت‌وگذار تفریحی مشغول است. عامل می‌خواهد از دیدنی‌ها دیدن کند، رومانیایی‌اش را بهبود دهد، از شب‌زنده‌داری لذت ببرد، از خماری بپرهیزد و مانند آن. مسئلهٔ تصمیم، مسئله‌ای پیچیده است. حال فرض کنید عامل اکنون در شهر آراد است و بلیطی غیرقابل استرداد برای پرواز روز بعد از بخارست دارد. عامل تابلوهای خیابان را می‌بیند و متوجه می‌شود سه جاده از آراد خارج می‌شوند: یکی به سمت سیبیو، یکی به تیمیشوآرا و یکی به زرند. هیچ‌کدام هدف نیستند، پس مگر آنکه عامل با جغرافیای رومانی آشنا باشد، نمی‌داند کدام جاده را دنبال کند.[1] اگر عامل اطلاعات اضافی نداشته باشد — یعنی اگر محیط ناشناخته باشد — چاره‌ای جز اجرای تصادفی یکی از

کنش‌ها ندارد. این وضعیت غم‌انگیز در فصل ۴ بحث می‌شود. در این فصل فرض می‌کنیم عامل‌های ما همیشه به اطلاعاتی دربارهٔ جهان دسترسی دارند، مانند نقشهٔ شکل ۳.۱. با آن اطلاعات، عامل می‌تواند این فرایند حل مسئلهٔ چهارمرحله‌ای را دنبال کند:

- **صورت‌بندی هدف (Goal Formulation):** عامل هدف رسیدن به بخارست را برمی‌گیرد. هدف‌ها با محدودکردن هدف‌ها و در نتیجه کنش‌های در نظرگرفته‌شده، رفتار را سازمان می‌دهند.



شکل ۳.۱ – نقشهٔ جاده‌ای ساده‌شده از بخشی از رومانی، با فاصله‌های جاده‌ای بر حسب مایل. (شهرها: اورادئا، زرند، آراد، تیمیشوآرا، لوگوژ، مهadia، دروبتا، کرایووا، ریمنیکو ویلچا، سیبیو، فاگاراش، پیتشتی، بخارست، جیورگیو، اورزیچنی، هیرشووا، اِفوریه، واسیویی، یاشی، نئامت)

- **صورت‌بندی مسئله (Problem Formulation):** عامل توصیفی از حالت‌ها و کنش‌های لازم برای رسیدن به هدف تدوین می‌کند – مدلی انتزاعی از بخشِ مرتبطِ جهان. برای عامل ما، یک مدل خوب آن است که کنش‌ها را سفر از شهری به شهریِ مجاور در نظر بگیریم و بنابراین تنها واقعیتی از حالت جهان که بر اثر کنش تغییر می‌کند، شهر کنونی است.
- **جستجو (Search):** پیش از انجام هر کنشی در جهان واقعی، عامل توالی‌های کنش را در مدلش شبیه‌سازی می‌کند و جستجو می‌کند تا توالی کنشی را بیابد که به هدف می‌رسد. چنین توالی‌ای «راه‌حل» (Solution) نامیده می‌شود. عامل ممکن است ناگزیر شود چند توالی را شبیه‌سازی کند که به هدف نمی‌رسند، اما سرانجام راه‌حلی خواهد یافت (مانند رفتن از آراد به سیبیو به فاگاراش به بخارست) یا خواهد یافت که راه‌حلی ممکن نیست.
- **اجرا (Execution):** عامل اکنون می‌تواند کنش‌های راه‌حل را یکی‌یکی اجرا کند.

خاصیت مهمی که هست این است که در محیط کاملاً قابل مشاهده، جبرگرایانه و شناخته‌شده، ⚠ راه‌حل هر مسئله‌ای توالی ثابتی از کنش‌هاست: به سیبیو بران، سپس فاگاراش، سپس بخارست. اگر مدل درست باشد، پس از آنکه عامل راه‌حل را یافت، هنگام اجرای کنش‌ها می‌تواند ادراک‌های حسی‌اش را نادیده بگیرد – به اصطلاح چشم‌هایش را ببندد – زیرا راه‌حل تضمین‌شده به هدف می‌رسد. نظریه‌پردازان کنترل این را سامانهٔ حلقه‌باز (Open-

Loop) می‌نامند: نادیده‌گرفتن ادراک‌های حسی، حلقهٔ میان عامل و محیط را می‌بُرد. اگر احتمال نادرست‌بودن مدل وجود داشته باشد، یا محیط ناجبرگرایانه باشد، عامل با رویکرد حلقه‌بسته (Closed-Loop) که ادراک‌های حسی را پایش می‌کند ایمن‌تر خواهد بود (بخش ۴.۴ را ببینید).

در محیط‌های تا حدی قابل مشاهده یا ناجبرگرایانه، راه‌حل، راهبردی انشعابی خواهد بود که بسته به ادراک‌های حسیِ دریافتی، کنش‌های آیندهٔ متفاوتی پیشنهاد می‌کند. برای نمونه، عامل ممکن است برنامه‌ریزی کند از آراد به سیبیو براند، اما به برنامهٔ احتیاطی نیاز داشته باشد در صورتی که تصادفاً به زرند برسد یا تابلو «جاده بسته» ببیند.

۳.۱.۱ مسائل جستجو و راه‌حل‌ها

مسئلهٔ جستجو را می‌توان به‌صورت صوری چنین تعریف کرد:

- مجموعه‌ای از حالت‌های ممکن که محیط می‌تواند در آن‌ها باشد. این را «فضای حالت» (State Space) می‌نامیم. حالت‌ها (States) بازنمایی‌های اتمی‌اند.
- «حالت آغازین» (Initial State) که عامل از آن آغاز می‌کند. برای نمونه: آراد.
- مجموعه‌ای از یک یا چند «حالت هدف» (Goal States). گاهی یک حالت هدف هست (مثلاً بخارست)، گاهی مجموعهٔ کوچکی از حالت‌های هدف جایگزین، و گاهی هدف با ویژگی‌ای تعریف می‌شود که بر حالت‌های فراوانی (احتمالاً بی‌نهایت) صدق می‌کند. برای نمونه، در جهان جاروبرقی، هدف ممکن است این باشد که در هیچ مکانی کثیفی نباشد، صرف‌نظر از هر واقعیت دیگری دربارهٔ حالت. می‌توانیم هر سه امکان را با مشخص کردن روش IS-GOAL برای مسئله پوشش دهیم. در این فصل برای سادگی گاهی «هدف» می‌گوییم، اما آنچه می‌گوییم بر «هر یک از حالت‌های هدف ممکن» نیز صدق می‌کند.
- کنش‌های (Actions) در دسترس عامل. با داشتن حالت s، تابع ACTIONS(s) مجموعه‌ای متناهی[۲] از کنش‌های قابل اجرا در s را بازمی‌گرداند. می‌گوییم هر یک از این کنش‌ها در s «کاربردپذیر» (Applicable) است. مثالی: ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}.
- «مدل گذار» (Transition Model) که توصیف می‌کند هر کنش چه می‌کند. RESULT(s, a) حالتی را بازمی‌گرداند که از انجام کنش a در حالت s حاصل می‌شود. برای نمونه: RESULT(Arad, ToZerind) = Zerind.
- «تابع هزینهٔ کنش» (Action Cost Function) که با ACTION-COST(s,a,s') هنگام برنامه‌نویسی یا c(s,a,s') هنگام ریاضی نشان داده می‌شود و هزینهٔ عددی اعمال کنش a در حالت s برای رسیدن به حالت s' را می‌دهد. عامل حل‌کنندهٔ مسئله باید از تابع هزینه‌ای استفاده کند که معیار عملکرد خودش را بازتاب می‌دهد؛ برای نمونه، برای عامل‌های مسیریاب، هزینهٔ یک کنش ممکن است طول بر حسب مایل (چنان‌که در شکل ۳.۱ دیده می‌شود) یا زمان لازم برای تکمیل کنش باشد.

توالی کنش‌ها «مسیر» (Path) می‌سازد و راه‌حل، مسیری از حالت آغازین به حالت هدف است. فرض می‌کنیم هزینه‌های کنش جمع‌شدنی‌اند؛ یعنی هزینهٔ کل یک مسیر، مجموع هزینه‌های کنش‌های منفرد است. «راه‌حل بهینه» (Optimal Solution) کمترین هزینهٔ مسیر را میان همهٔ راه‌حل‌ها دارد. در این فصل فرض می‌کنیم همهٔ هزینه‌های کنش مثبت‌اند تا از برخی پیچیدگی‌ها بپرهیزیم.[۳]

فضای حالت را می‌توان به‌صورت گرافی بازنمایی کرد که رأس‌هایش حالت‌ها و یال‌های جهت‌دارش کنش‌ها هستند. نقشهٔ رومانیِ شکل ۳.۱ چنین گرافی است، که در آن هر جاده دو کنش را نشان می‌دهد، یکی در هر جهت.

۳.۱.۲ صورت‌بندی مسائل

صورت‌بندی ما از مسئلهٔ رسیدن به بخارست، مدلی است — توصیفی ریاضی انتزاعی — نه خودِ واقعیت. توصیف حالت اتمی سادهٔ «آراد» را با یک سفر واقعی بین‌شهری مقایسه کنید، جایی که حالت جهان شامل چیزهای بسیار زیادی است: هم‌سفرها، برنامهٔ رادیوییِ در حال پخش، منظرهٔ بیرون پنجره، نزدیکی مأموران اجرای قانون، فاصله تا توقفگاه بعدی، وضعیت جاده، آب‌وهوا، ترافیک و... همهٔ این ملاحظات از مدل ما بیرون گذاشته شده‌اند، زیرا به مسئلهٔ یافتن مسیر به بخارست بی‌ربط‌اند.

فرایند حذف جزئیات از یک بازنمایی «انتزاع» (Abstraction) نامیده می‌شود. صورت‌بندی مسئلهٔ خوب، سطح مناسبی از جزئیات دارد. اگر کنش‌ها در سطح «پای راست را یک سانتی‌متر جلو ببر» یا «فرمان را یک درجه به چپ بچرخان» بودند، عامل احتمالاً هرگز راهش را از پارکینگ بیرون نمی‌یافت، چه رسد به بخارست.

آیا می‌توانیم دربارهٔ سطح انتزاعِ مناسب دقیق‌تر باشیم؟ به حالت‌ها و کنش‌های انتزاعیِ برگزیده‌مان بیندیشید، چنان‌که با مجموعه‌های بزرگِ حالت‌های جهانِ جزئی و توالی‌های کنشِ جزئی متناظرند. اکنون راه‌حلی برای مسئلهٔ انتزاعی در نظر بگیرید: برای نمونه، مسیر از آراد به سیبیو به ریمنیکو ویلچا به پیتشتی به بخارست. این راه‌حل انتزاعی با تعداد زیادی مسیر جزئی‌تر متناظر است. برای نمونه، می‌توانیم با روشن‌بودن رادیو از سیبیو به ریمنیکو ویلچا برانیم و سپس برای بقیهٔ سفر خاموشش کنیم.

انتزاع معتبر است اگر بتوانیم هر راه‌حل انتزاعی را به راه‌حلی در جهان جزئی‌تر بسط دهیم؛ شرط کافی آن است که برای هر حالت جزئی که «در آراد» است، مسیری جزئی به حالتی که «در سیبیو» است وجود داشته باشد، و همین‌طور الی آخر.[4] انتزاع مفید است اگر اجرای هر یک از کنش‌های راه‌حل از خودِ مسئلهٔ اصلی آسان‌تر باشد؛ در مورد ما، کنش «رانندگی از آراد به سیبیو» را می‌توان بدون جستجو یا برنامه‌ریزی بیشتر توسط راننده‌ای با مهارت متوسط انجام داد. پس انتخاب انتزاع خوب شامل حذف هرچه بیشتر جزئیات است در حالی که اعتبار حفظ شود و اطمینان حاصل شود کنش‌های انتزاعی آسان‌اند. اگر توانایی ساخت انتزاع‌های مفید نبود، عامل‌های هوشمند کاملاً در جهان واقعی غرق می‌شدند.

۳.۲ مسائل نمونه

رویکرد حل مسئله در گسترهٔ وسیعی از محیط‌های تکلیف به کار رفته است. برخی از شناخته‌شده‌ترین‌ها را در این‌جا فهرست می‌کنیم، با تمایز میان مسائل استانداردشده و مسائل جهان واقعی.

مسئلهٔ استانداردشده (Standardized Problem) برای نشان دادن یا تمرین روش‌های گوناگون حل مسئله در نظر گرفته شده. می‌توان توصیف موجز و دقیقی به آن داد و بنابراین به‌عنوان معیاری مناسب برای مقایسهٔ عملکرد الگوریتم‌ها توسط پژوهشگران است. مسئلهٔ جهان واقعی (Real-World Problem)، مانند ناوبری ربات، مسئله‌ای

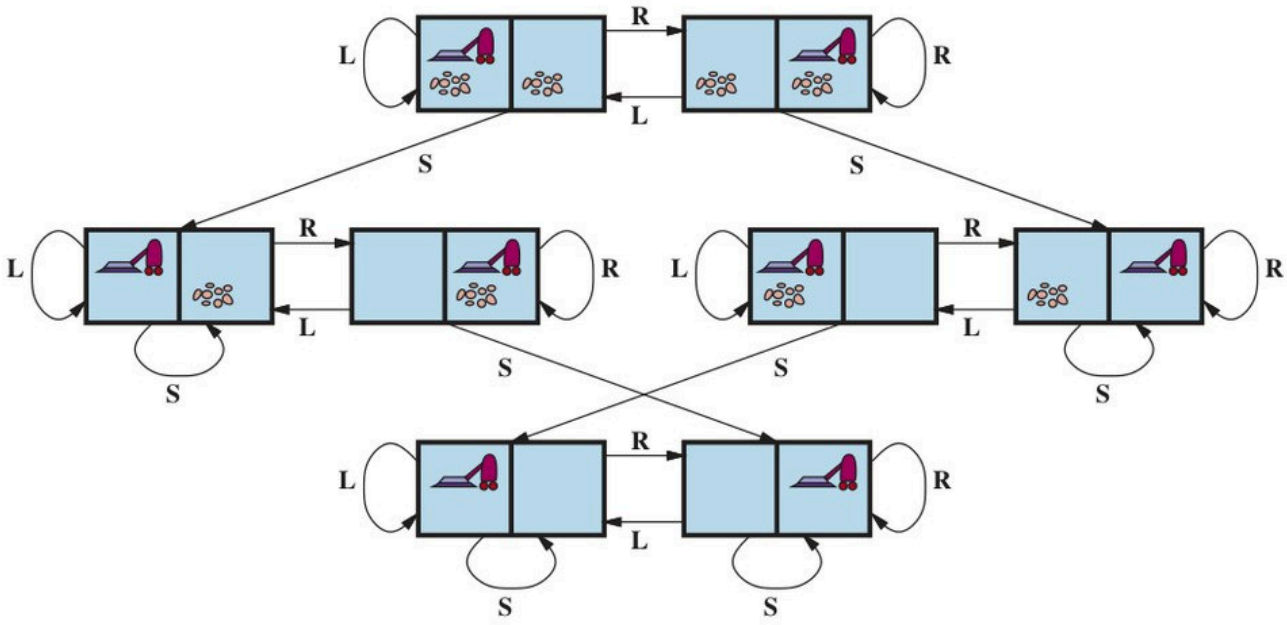
است که راه‌حل‌هایش را مردم واقعاً استفاده می‌کنند و صورت‌بندی‌اش ویژهٔ خودش است، نه استاندارد؛ برای نمونه، چون هر ربات جسگرهای متفاوتی دارد که داده‌های متفاوتی تولید می‌کنند.

۳.۲.۱ مسائل استانداردشده

مسئلهٔ جهان شبکه‌ای (Grid World) آرایهٔ دوبعدی مستطیلی از خانه‌های مربعی است که عامل‌ها می‌توانند از خانه‌ای به خانهٔ دیگر حرکت کنند. معمولاً عامل می‌تواند به هر خانهٔ مجاورِ بدون مانع حرکت کند — افقی و عمودی و در برخی مسائل مورب. خانه‌ها می‌توانند اشیا داشته باشند که عامل بتواند بردارد، هل دهد یا به نحوی بر آن‌ها کنش کند؛ دیوار یا مانع غیرقابل عبور در خانه‌ای، عامل را از ورود به آن خانه بازمی‌دارد.

جهان جاروبرقی از بخش ۲.۱ را می‌توان چنین به‌صورت مسئلهٔ جهان شبکه‌ای صورت‌بندی کرد:

- **حالت‌ها:** حالت جهان می‌گوید کدام اشیا در کدام خانه‌ها هستند. برای جهان جاروبرقی، اشیا عبارت‌اند از عامل و هر کثیفی. در نسخهٔ سادهٔ دوخانه‌ای، عامل می‌تواند در هر یک از دو خانه باشد و هر خانه یا کثیفی دارد یا نه، پس ۲×۲×۲ = ۸ حالت هست (شکل ۳.۲ را ببینید). در حالت کلی، محیط جاروبرقی با n خانه، $n \cdot 2^n$ حالت دارد.
- **حالت آغازین:** هر حالتی را می‌توان حالت آغازین تعیین کرد.
- **کنش‌ها:** در جهان دوخانه‌ای سه کنش تعریف کردیم: Suck (بمک)، حرکت به چپ و حرکت به راست. در جهان چندخانه‌ای دوبعدی به کنش‌های حرکتی بیشتری نیاز داریم. می‌توانیم Upward و Downward (بالا و پایین) را بیفزاییم و چهار کنش حرکتی مطلق داشته باشیم، یا به کنش‌های خودمحور (Egocentric) روی بیاوریم که نسبت به دیدگاه عامل تعریف می‌شوند — برای نمونه: Forward (به جلو)، Backward (به عقب)، TurnRight (چرخش به راست) و TurnLeft (چرخش به چپ).
- **مدل گذار:** Suck هر کثیفی را از خانهٔ عامل می‌زداید؛ Forward عامل را یک خانه در جهتِ روبه‌رویش جلو می‌برد، مگر آنکه به دیوار بخورد که در آن صورت کنش اثری ندارد. Backward عامل را در جهت مخالف می‌برد، در حالی که TurnRight و TurnLeft جهتِ روبه‌رو را ۹۰ درجه تغییر می‌دهند.
- **حالت‌های هدف:** حالت‌هایی که در آن‌ها همهٔ خانه‌ها تمیزند.
- **هزینهٔ کنش:** هر کنش ۱ هزینه دارد.



شکل ۳.۲ – گراف فضای حالت برای جهان جاروبرقی دوخانه‌ای. ۸ حالت و برای هر حالت سه کنش هست: L = چپ، R = راست، S = مکیدن.

نوع دیگری از جهان شبکه‌ای، معمای سوکوبان (Sokoban Puzzle) است که هدف عامل هل‌دادن تعدادی جعبه – پراکنده در شبکه – به مکان‌های ذخیرهٔ معین است. در هر خانه حداکثر یک جعبه می‌تواند باشد. وقتی عامل به جلو به خانه‌ای که جعبه دارد می‌رود و خانهٔ خالی در آن‌سوی جعبه هست، هم جعبه و هم عامل به جلو می‌روند. عامل نمی‌تواند جعبه را به جعبهٔ دیگر یا دیوار هل دهد. برای جهانی با n خانهٔ بدون مانع و b جعبه، $n \times n!/(b!(n-b)!)$ حالت هست؛ برای نمونه روی شبکهٔ ۸×۸ با یک دوجین جعبه، بیش از ۲۰۰ تریلیون حالت هست.

در معمای کاشی‌لغزان (Sliding-Tile Puzzle)، تعدادی کاشی (که گاهی بلوک یا مهره نامیده می‌شوند) در شبکه‌ای با یک یا چند فضای خالی چیده می‌شوند تا برخی کاشی‌ها بتوانند به فضای خالی بلغزند. یکی از تغییرها، معمای «ساعت شلوغی» (Rush Hour) است که در آن خودروها و کامیون‌ها در شبکهٔ ۶×۶ می‌لغزند تا خودرویی از ترافیک آزاد شود. شاید شناخته‌شده‌ترین تغییر، معمای ۸ (Puzzle-8) باشد (شکل ۳.۳) که از شبکهٔ ۳×۳ با هشت کاشی شماره‌دار و یک فضای خالی تشکیل شده، و معمای ۱۵ (Puzzle-15) روی شبکهٔ ۴×۴. هدف رسیدن به حالت هدفِ مشخص است، مانند آنچه سمت راست شکل نشان می‌دهد. صورت‌بندی استاندارد معمای ۸ چنین است:

- **حالت‌ها:** توصیف حالت، مکان هر یک از کاشی‌ها را مشخص می‌کند.
- **حالت آغازین:** هر حالتی را می‌توان حالت آغازین تعیین کرد. توجه کنید ویژگی «جفت‌بودن» (Parity) فضای حالت را تقسیم می‌کند – هر هدفِ داده‌شده دقیقاً از نیمی از حالت‌های آغازین ممکن قابل دسترسی است (تمرین PART.3).
- **کنش‌ها:** در حالی که در جهان فیزیکی کاشی است که می‌لغزد، ساده‌ترین راه توصیف کنش آن است که به حرکتِ فضای خالی به چپ، راست، بالا یا پایین بیندیشیم. اگر فضای خالی در لبه یا گوشه باشد، همهٔ کنش‌ها کاربردپذیر نخواهند بود.

- **مدل گذار:** حالت و کنش را به حالت حاصل نگاشت می‌کند؛ برای نمونه، اگر Left را روی حالت آغازین شکل ۳.۳ اعمال کنیم، حالت حاصل، کاشی ۵ و فضای خالی را جابه‌جا می‌کند.
- **حالت هدف:** هرچند هر حالتی می‌تواند هدف باشد، معمولاً حالتی را با اعداد به‌ترتیب مشخص می‌کنیم، چنان‌که در شکل ۳.۳.
- **هزینهٔ کنش:** هر کنش ۱ هزینه دارد.

توجه کنید هر صورت‌بندی مسئله شامل انتزاع‌هایی است. کنش‌های معمای ۸ به حالت‌های آغازین و نهاییشان انتزاع شده‌اند و مکان‌های میانیِ لغزش کاشی نادیده گرفته می‌شوند. کنش‌هایی مانند تکان‌دادن تخته وقتی کاشی‌ها گیر می‌کنند را انتزاع کرده‌ایم و بیرون‌آوردن کاشی‌ها با چاقو و برگرداندنشان را منتفی دانسته‌ایم. برای ما توصیفی از قواعد می‌ماند، بدون همهٔ جزئیات دست‌کاری فیزیکی.

آخرین مسئلهٔ استانداردشده‌مان را دونالد کنوت (۱۹۶۴) ابداع کرد و نشان می‌دهد فضای حالتِ بی‌نهایت چگونه می‌تواند پدید آید. کنوت حدس زد با شروع از عدد ۴، توالی‌ای از عمل‌های جذر، جزءصحیح (Floor) و فاکتوریل می‌تواند به هر عدد صحیح مثبت دلخواه برسد. برای نمونه، می‌توان از ۴ به ۵ رسید:

```
5 = ...⌊(!(!(4))√⌋√
```

تعریف مسئله ساده است:

- **حالت‌ها:** اعداد حقیقی مثبت.
- **حالت آغازین:** ۴.
- **کنش‌ها:** اعمال جذر، جزءصحیح یا فاکتوریل (فاکتوریل فقط برای اعداد صحیح).
- **مدل گذار:** طبق تعریف‌های ریاضی عمل‌ها.
- **حالت هدف:** عدد صحیح مثبتِ دلخواه.
- **هزینهٔ کنش:** هر کنش ۱ هزینه دارد.

فضای حالت این مسئله بی‌نهایت است: برای هر عدد صحیح بزرگ‌تر از ۲، عملگر فاکتوریل همیشه عدد صحیح بزرگ‌تری می‌دهد. مسئله جالب است، زیرا اعداد بسیار بزرگ را می‌کاود: کوتاه‌ترین مسیر به ۵ از (4!)! = ۶۲۰,۴۴۸,۴۰۱,۷۳۳,۲۳۹,۴۳۹,۳۶۰,۰۰۰ می‌گذرد.

فضاهای حالت بی‌نهایت اغلب در تکلیف‌های تولید عبارت‌های ریاضی، مدارها، برهان‌ها، برنامه‌ها و دیگر اشیای بازگشتی‌تعریف‌شده پدید می‌آیند.

۳.۲.۲ مسائل جهان واقعی

پیش‌تر دیدیم مسئلهٔ مسیریابی چگونه بر حسب مکان‌های مشخص و گذارها در طول یال‌های میان آن‌ها تعریف می‌شود. الگوریتم‌های مسیریابی در کاربردهای گوناگونی استفاده می‌شوند. برخی، مانند پایگاه‌های وب و سامانه‌های داخل خودرو که مسیر رانندگی می‌دهند، بسط‌های نسبتاً سرراست مثال رومانی‌اند. (پیچیدگی‌های اصلی عبارت‌اند از

هزینه‌های متغیر به‌سبب تأخیرهای وابسته به ترافیک و تغییر مسیر به‌سبب بسته‌شدن جاده‌ها.) برخی دیگر، مانند مسیریابی جریان‌های ویدئویی در شبکه‌های رایانه‌ای، برنامه‌ریزی عملیات نظامی و سامانه‌های برنامه‌ریزی سفر هوایی، مشخصات بسیار پیچیده‌تری در بر دارند.

مسئله‌های سفر هوایی را در نظر بگیرید که پایگاه وب برنامه‌ریزی سفر باید حل کند:

- **حالت‌ها:** هر حالت بدیهی است شامل مکان (مثلاً فرودگاه) و زمان کنونی است. وانگهی، چون هزینهٔ یک کنش (یک قطعه پرواز) ممکن است به قطعه‌های قبلی، مبنای قیمت‌شان و وضعیت داخلی/بین‌المللی‌بودنشان بستگی داشته باشد، حالت باید اطلاعات اضافی دربارهٔ این جنبه‌های «تاریخی» ثبت کند.
- **حالت آغازین:** فرودگاه خانگی کاربر.
- **کنش‌ها:** گرفتن هر پروازی از مکان کنونی، در هر ردهٔ صندلی، با حرکت پس از زمان کنونی، با زمان کافی برای انتقال درون‌فرودگاهی در صورت نیاز.
- **مدل گذار:** حالتی که از گرفتن پرواز حاصل می‌شود، مقصد پرواز را مکان تازه و زمان ورود پرواز را زمان تازه خواهد داشت.
- **حالت هدف:** شهر مقصد. گاهی هدف می‌تواند پیچیده‌تر باشد، مانند «رسیدن به مقصد با پرواز بی‌واسطه».
- **هزینهٔ کنش:** ترکیبی از هزینهٔ پولی، زمان انتظار، زمان پرواز، مراحل گمرک و مهاجرت، کیفیت صندلی، زمان روز، نوع هواپیما، امتیازهای پاداش مسافرِ همیشگی و مانند آن.

سامانه‌های مشاورهٔ سفر تجاری از چنین صورت‌بندی مسئله‌ای استفاده می‌کنند، با پیچیدگی‌های اضافی فراوان برای رسیدگی به ساختارهای قیمت‌گذاری پیچیدهٔ شرکت‌های هواپیمایی. هر مسافرِ با تجربه‌ای می‌داند، اما همهٔ سفرهای هوایی طبق برنامه پیش نمی‌روند. سامانهٔ واقعاً خوب باید برنامه‌های احتیاطی در بر داشته باشد — اگر این پرواز تأخیر بخورد و اتصال از دست برود چه؟

مسائل گشت‌وگذار (Touring Problems) مجموعه‌ای از مکان‌ها را توصیف می‌کنند که باید بازدید شوند، نه یک مقصد هدف واحد. مسئلهٔ فروشندهٔ دوره‌گرد (Traveling Salesperson Problem، یا TSP) مسئلهٔ گشت‌وگذاری است که در آن باید از هر شهر روی نقشه بازدید شود. هدف یافتن گشتی با هزینهٔ $C <$ است (یا در نسخهٔ بهینه‌سازی، یافتن گشتی با کمترین هزینهٔ ممکن). تلاش عظیمی برای بهبود توانایی الگوریتم‌های TSP صرف شده. الگوریتم‌ها را می‌توان برای رسیدگی به ناوگان‌های وسایل نقلیه نیز بسط داد. برای نمونه، الگوریتم جستجو و بهینه‌سازی برای مسیریابی اتوبوس‌های مدرسه در بوستون ۵ میلیون دلار صرفه‌جویی کرد، ترافیک و آلودگی هوا را کاهش داد و در زمان رانندگان و دانش‌آموزان صرفه‌جویی کرد (برتسیماس و همکاران، ۲۰۱۹). افزون بر برنامه‌ریزی سفرها، الگوریتم‌های جستجو برای تکلیف‌هایی مانند برنامه‌ریزی حرکت مته‌های خودکار مدار چاپی و ماشین‌های قفسه‌چینی در کف کارگاه‌ها به کار رفته‌اند.

مسئلهٔ چیدمان VLSI مستلزم مکان‌دهی میلیون‌ها مؤلفه و اتصال روی تراشه است تا مساحت، تأخیر مدار و ظرفیت‌های سرگردان کمینه و بازده تولید بیشینه شود. مسئلهٔ چیدمان پس از فاز طراحی منطقی می‌آید و معمولاً به دو بخش تقسیم می‌شود: چیدمان سلولی و مسیریابی کانال. در چیدمان سلولی، مؤلفه‌های اولیهٔ مدار در سلول‌هایی گروه‌بندی می‌شوند که هر یک کارکرد شناخته‌شده‌ای انجام می‌دهد. هر سلول ردپای ثابتی (اندازه و شکل) دارد و به

تعداد معینی اتصال به هر سلول دیگر نیاز دارد. هدف آن است که سلول‌ها روی تراشه چنان گذاشته شوند که هم‌پوشانی نکنند و جایی برای سیم‌های اتصال میان سلول‌ها باشد. مسیریابی کانال برای هر سیم مسیری خاص در میان شکاف‌های بین سلول‌ها می‌یابد. این مسائل جستجو بسیار پیچیده‌اند، اما قطعاً ارزش حل‌شدن دارند.

ناوبری ربات (Robot Navigation) تعمیمی از مسئلهٔ مسیریابیِ پیش‌ترتوصیف‌شده است. ربات به‌جای دنبال‌کردن مسیرهای متمایز (مانند جاده‌های رومانی) می‌تواند پرسه بزند و عملاً مسیرهای خودش را بسازد. برای ربات دایره‌ای که روی سطح صاف حرکت می‌کند، فضا اساساً دوبعدی است. وقتی ربات بازوها و پاهایی دارد که باید کنترل شوند، فضای جستجو چندبعدی می‌شود — برای هر زاویهٔ مفصل یک بُعد. فنون پیشرفته فقط برای متناهی‌کردن فضای جستجوی اساساً پیوسته لازم است (فصل ۲۶ را ببینید). افزون بر پیچیدگی مسئله، ربات‌های واقعی باید با خطاهای خوانش حسگرها و کنترل موتور، با مشاهده‌ناپذیری تا حدی، و با عامل‌های دیگری که ممکن است محیط را تغییر دهند نیز کنار بیایند.

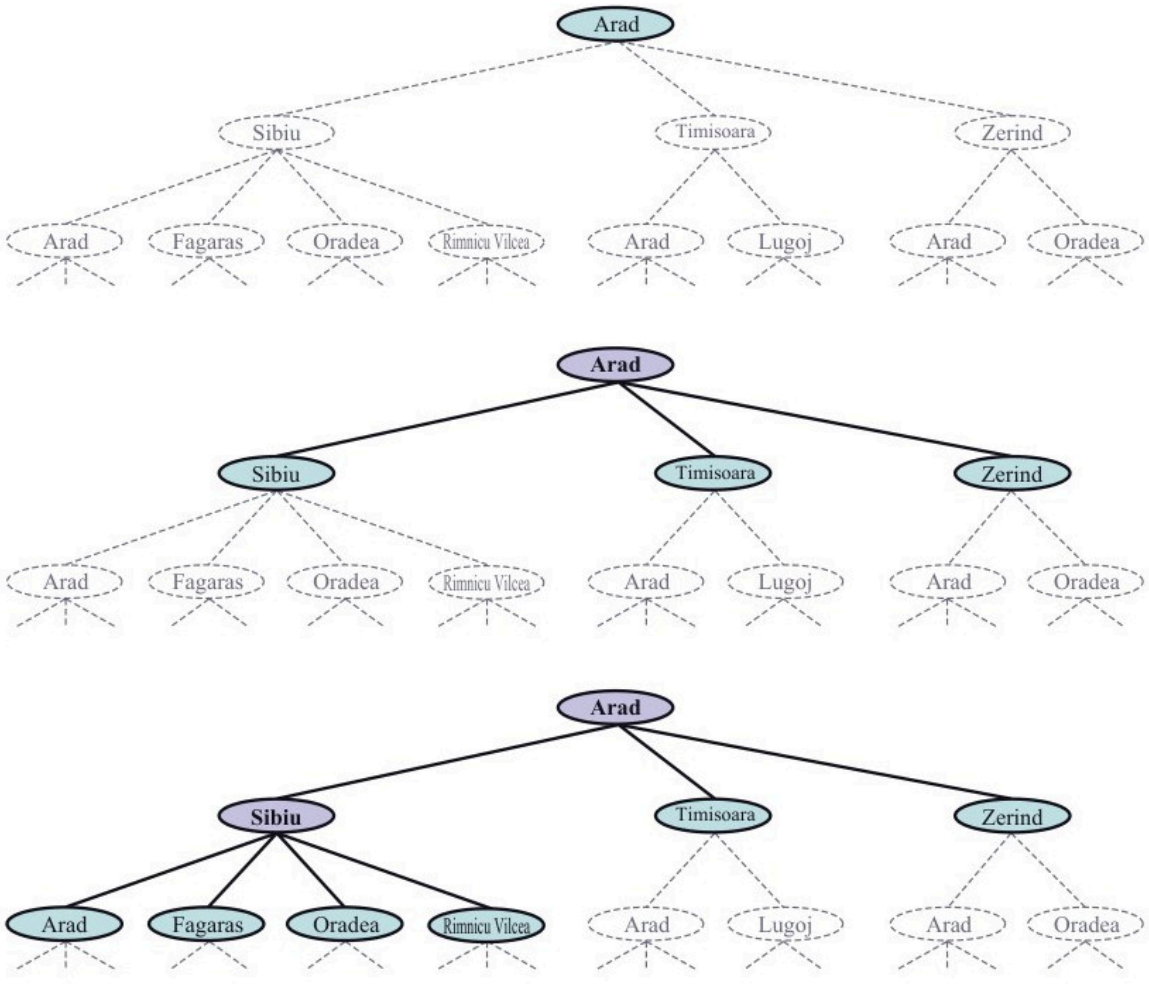
توالی‌یابی مونتاژ خودکار (Automatic Assembly Sequencing) اشیای پیچیده (مانند موتورهای الکتریکی) توسط ربات از دههٔ ۱۹۷۰ رویهٔ استاندارد صنعت بوده است. الگوریتم‌ها نخست توالی مونتاژ شدنی می‌یابند و سپس برای بهینه‌سازی فرایند کار می‌کنند. کمینه‌کردن مقدار کار انسانیِ دستی در خط مونتاژ می‌تواند صرفه‌جویی چشمگیری در زمان و هزینه ایجاد کند. در مسائل مونتاژ، هدف یافتن ترتیبی برای سوارکردن قطعات شیء است. اگر ترتیب اشتباه انتخاب شود، بعداً راهی برای افزودن قطعه‌ای در توالی نخواهد بود مگر با بازکردن بخشی از کار انجام‌شده. بررسی شدنی‌بودن کنشی در توالی، مسئلهٔ جستجوی هندسی دشواری است که به ناوبری ربات نزدیک است. پس تولید کنش‌های قانونی بخش پرهزینهٔ توالی‌یابی مونتاژ است. هر الگوریتم عملی باید از کاوش همهٔ فضای حالت به‌جز کسری کوچک بپرهیزد. یکی از مسائل مهم مونتاژ، طراحی پروتئین (Protein Design) است که هدفش یافتن توالی‌ای از اسیدهای آمینه است که به پروتئینی سه‌بعدی با ویژگی‌های درست برای درمان بیماری‌ای تا شود.

۳.۳ الگوریتم‌های جستجو

الگوریتم جستجو (Search Algorithm) مسئلهٔ جستجو را به‌عنوان ورودی می‌گیرد و راه‌حل یا نشانهٔ شکست بازمی‌گرداند. در این فصل الگوریتم‌هایی را در نظر می‌گیریم که درخت جستجویی را بر روی گراف فضای حالت سوار می‌کنند و مسیرهای گوناگونی از حالت آغازین می‌سازند و می‌کوشند مسیری بیابند که به حالت هدف برسد. هر گره (Node) در درخت جستجو با حالتی در فضای حالت متناظر است و یال‌های درخت جستجو با کنش‌ها متناظرند. ریشهٔ درخت با حالت آغازین مسئله متناظر است. مهم است تمایز میان فضای حالت و درخت جستجو را بفهمیم.

فضای حالت مجموعهٔ (احتمالاً بی‌نهایت) حالت‌های جهان و کنش‌هایی را توصیف می‌کند که گذار از حالتی به حالت دیگر را ممکن می‌کنند. درخت جستجو مسیرهای میان این حالت‌ها را توصیف می‌کند و به سوی هدف پیش می‌رود. درخت جستجو ممکن است چندین مسیر به هر حالتِ داده‌شده (و در نتیجه چند گره برای آن) داشته باشد، اما هر گره در درخت مسیری یکتا به ریشه دارد (مانند همهٔ درخت‌ها). شکل ۳.۴ نخستین گام‌های یافتن مسیر از آراد به بخارست را نشان می‌دهد. گرهٔ ریشهٔ درخت جستجو در حالت آغازین، آراد، است. می‌توانیم گره را «گسترش» (Expand) دهیم:

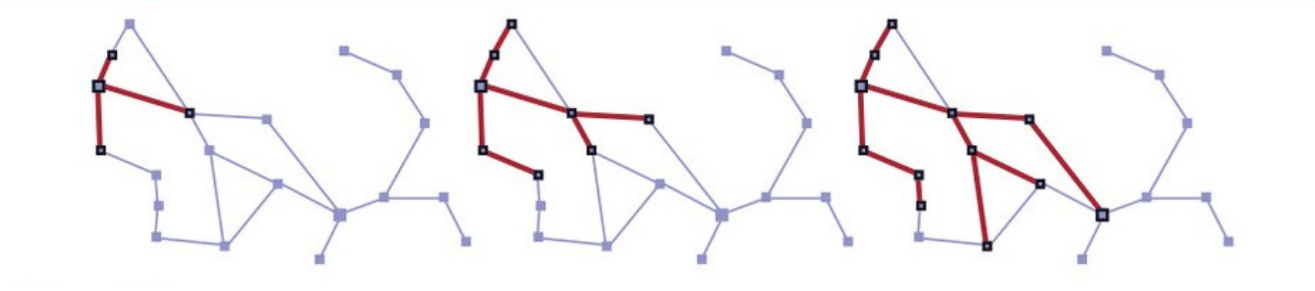
کنش‌های در دسترس برای آن حالت را در نظر بگیریم، با تابع RESULT ببینیم آن کنش‌ها به کجا می‌رسند، و برای هر یک از حالت‌های حاصل، گرهٔ تازه‌ای (که گرهٔ فرزند (Child Node) یا گرهٔ جانشین (Successor Node) نامیده می‌شود) تولید کنیم. هر گرهٔ فرزند، آراد را به‌عنوان گرهٔ والد (Parent Node) دارد.



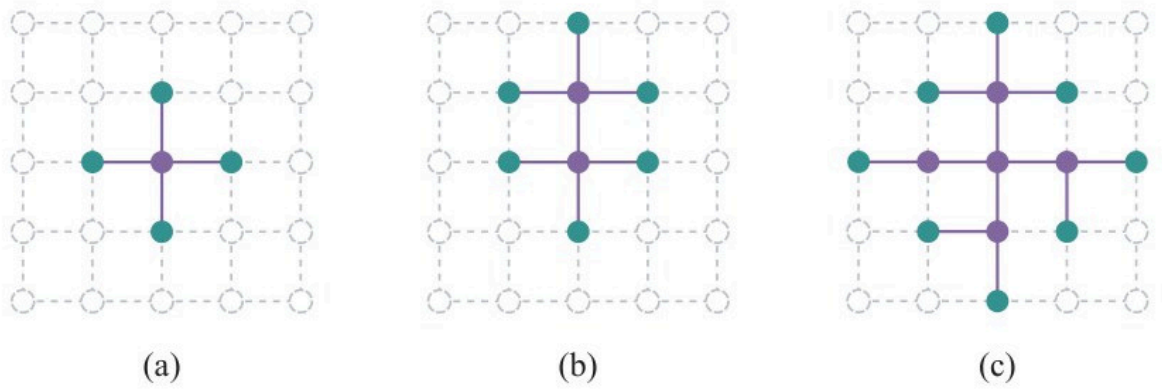
شکل ۳.۴ – سه درخت جستجوی جزئی برای یافتن مسیر از آراد به بخارست. گره‌های گسترش‌یافته بنفش با حروف ضخیم‌اند؛ گره‌های روی جبهه که تولید شده‌اند اما هنوز گسترش نیافته‌اند سبزند؛ به مجموعهٔ حالت‌های متناظر با این دو نوع گره می‌گوییم «رسیده» (Reached) شده‌اند. گره‌هایی که می‌توانند بعداً تولید شوند با خط‌چین کمرنگ نشان داده شده‌اند. توجه کنید در درخت پایینی چرخه‌ای از آراد به سیبیو به آراد هست؛ آن نمی‌تواند مسیر بهینه باشد، پس جستجو نباید از آنجا ادامه یابد.

اکنون باید برگزینیم کدام‌یک از این سه گرهٔ فرزند را بعدی در نظر بگیریم. این جوهر جستجوست – دنبال‌کردن یک گزینه اکنون و کنارگذاشتن بقیه برای بعد. فرض کنیم اول سیبیو را گسترش دهیم. شکل ۳.۴ (پایین) نتیجه را نشان می‌دهد: مجموعه‌ای از ۶ گرهٔ گسترش‌نیافته (با خط ضخیم مشخص شده‌اند). این را «جبهه» (Frontier) درخت جستجو می‌نامیم. می‌گوییم هر حالتی که گره‌ای برایش تولید شده «رسیده» (Reached) شده است (چه آن گره گسترش یافته باشد چه نه).[5] شکل ۳.۵ درخت جستجو را سوار بر گراف فضای حالت نشان می‌دهد.

توجه کنید جبهه دو ناحیه از گراف فضای حالت را از هم جدا می‌کند: ناحیهٔ داخلی که هر حالتش گسترش یافته، و ناحیهٔ بیرونی از حالت‌هایی که هنوز رسیده نشده‌اند. این ویژگی در شکل ۳.۶ نشان داده شده. (به این ویژگی «خاصیت جداسازی» (Separation Property) می‌گوییم.)



شکل ۳.۵ – توالی‌ای از درخت‌های جستجوی تولیدشده توسط جستجوی گرافی روی مسئلهٔ رومانی شکل ۳.۱. در هر مرحله، همهٔ گره‌های روی جبهه را گسترش داده‌ایم و هر مسیر را با همهٔ کنش‌های کاربردپذیری که به حالتی ازپیش‌رسیده نمی‌انجامند بسط داده‌ایم. توجه کنید در مرحلهٔ سوم، بالاترین شهر (اورادئا) دو جانشین دارد که هر دو از مسیرهای دیگر ازپیش رسیده شده‌اند، پس هیچ مسیری از اورادئا بسط نمی‌یابد.



شکل ۳.۶ – خاصیت جداسازی جستجوی گرافی، روی مسئلهٔ شبکهٔ مستطیلی. جبهه (سبز) درون (بنفش) را از بیرون (خط‌چین کمرنگ) جدا می‌کند. جبهه مجموعهٔ گره‌هایی (و حالت‌های متناظر) است که رسیده شده‌اند اما هنوز گسترش نیافته‌اند؛ درون مجموعهٔ گره‌هایی (و حالت‌های متناظر) است که گسترش یافته‌اند؛ و بیرون مجموعهٔ حالت‌هایی است که رسیده نشده‌اند. در (الف) فقط ریشه گسترش یافته. در (ب) گرهٔ بالایی جبهه گسترش می‌یابد. در (ج) جانشین‌های باقی‌ماندهٔ ریشه در جهت عقربه‌های ساعت گسترش می‌یابند.

برخی نویسندگان جبهه را «فهرست باز» (Open List) می‌نامند که هم از نظر جغرافیایی گویا کمتر است و هم از نظر محاسباتی نامناسب‌تر، زیرا صف از فهرست کارآمدتر است. آن نویسندگان از اصطلاح «فهرست بسته» (Closed List) برای مجموعهٔ گره‌های قبلاً گسترش‌یافته استفاده می‌کنند که در اصطلاحات ما «گره‌های رسیده منهای جبهه» می‌شود.

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
                                        node ← NODE(STATE=problem.INITIAL)
                  frontier ← a priority queue ordered by f, with node as an element
reached ← a lookup table, with one entry keyed by problem.INITIAL and value node
                                        while not IS-EMPTY(frontier) do
                                                node ← POP(frontier)
                                    if problem.IS-GOAL(node.STATE) then return node
                                         for each child in EXPAND(problem, node) do
                                                    s ← child.STATE
        if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                                                reached[s] ← child
                                            add child to frontier
                                                return failure

                                function EXPAND(problem, node) yields nodes
                                                s ← node.STATE
                                     for each action in problem.ACTIONS(s) do
                                            s′ ← problem.RESULT(s, action)
                                cost ← node.PATH-COST + problem.ACTION-COST(s, action, s′)
                            yield NODE(STATE=s′, PARENT=node, ACTION=action, PATH-COST=cost)
```

شکل ۳.۷ — الگوریتم جستجوی بهترین-اول، و تابع گسترش یک گره. ساختارهای دادهٔ به‌کاررفته در این‌جا در بخش ۳.۳.۲ توصیف شده‌اند. برای yield به پیوست ب مراجعه کنید.

۳.۳.۱ جستجوی بهترین-اول

چگونه تصمیم می‌گیریم کدام گره از جبهه را بعدی گسترش دهیم؟ رویکرد بسیار عمومی، «جستجوی بهترین-اول» (Best-First Search) نامیده می‌شود که در آن گره‌ای مانند n را با کمترین مقدارِ تابع ارزیابی (Evaluation Function) f(n) برمی‌گزینیم. شکل ۳.۷ الگوریتم را نشان می‌دهد. در هر تکرار، گره‌ای روی جبهه با کمترین مقدار f(n) برمی‌گزینیم؛ اگر حالتش حالت هدف باشد آن را برمی‌گردانیم وگرنه EXPAND را برای تولید گره‌های فرزند اعمال می‌کنیم. هر گرهٔ فرزند به جبهه افزوده می‌شود اگر پیش‌تر رسیده نشده باشد، یا دوباره افزوده می‌شود اگر اکنون با مسیری با هزینهٔ کمتر از هر مسیر قبلی رسیده می‌شود. الگوریتم یا نشانهٔ شکست بازمی‌گرداند، یا گره‌ای که مسیری به هدف را بازنمایی می‌کند. با به‌کارگیری توابع f(n) گوناگون، الگوریتم‌های خاص متفاوتی می‌گیریم که این فصل آن‌ها را پوشش می‌دهد.

۳.۳.۲ ساختارهای دادهٔ جستجو

الگوریتم‌های جستجو به ساختار داده‌ای برای پیگیری درخت جستجو نیاز دارند. گرهٔ درخت با ساختار داده‌ای با چهار مؤلفه بازنمایی می‌شود:

- node.STATE: حالتی که گره با آن متناظر است؛
- node.PARENT: گره‌ای در درخت که این گره را تولید کرده؛
- node.ACTION: کنشی که به حالت والد اعمال شده تا این گره تولید شود؛
- node.PATH-COST: هزینهٔ کل مسیر از حالت آغازین تا این گره. در فرمول‌های ریاضی از g(node) به‌عنوان مترادف PATH-COST استفاده می‌کنیم.

دنبال‌کردن اشاره‌گرهای PARENT از گره‌ای به عقب، به ما امکان می‌دهد حالت‌ها و کنش‌های طول مسیر تا آن گره را بازیابی کنیم. انجام این کار از گرهٔ هدف، راه‌حل را به ما می‌دهد.

برای ذخیرهٔ جبهه به ساختار داده‌ای نیاز داریم. انتخاب مناسب، صف (Queue) از نوعی است، زیرا عملیات روی جبهه عبارت‌اند از:

- IS-EMPTY(frontier): فقط اگر گره‌ای در جبهه نباشد true برمی‌گرداند.
- POP(frontier): گرهٔ بالای جبهه را حذف و بازمی‌گرداند.
- TOP(frontier): گرهٔ بالای جبهه را بازمی‌گرداند (بدون حذف).
- ADD(node, frontier): گره را در جای مناسبش در صف درج می‌کند.

سه نوع صف در الگوریتم‌های جستجو به کار می‌رود:

- «صف اولویت‌دار» (Priority Queue) نخست گره‌ای را برمی‌دارد که طبق تابع ارزیابی f کمترین هزینه را دارد. در جستجوی بهترین-اول به کار می‌رود.
- «صف FIFO» یا صف اول-در-اول-خروج نخست گره‌ای را برمی‌دارد که اول به صف افزوده شده؛ خواهیم دید در جستجوی سطح-اول به کار می‌رود.
- «صف LIFO» یا صف آخر-در-اول-خروج (که «پشته» (Stack) نیز نامیده می‌شود) نخست تازه‌ترین گرهٔ افزوده‌شده را برمی‌دارد؛ خواهیم دید در جستجوی عمق-اول به کار می‌رود.

حالت‌های رسیده را می‌توان در جدول جست‌وجو (مثلاً جدول درهم‌سازی) ذخیره کرد که هر کلید حالت و هر مقدار، گرهٔ آن حالت است.

۳.۳.۳ مسیرهای زائد

درخت جستجوی شکل ۳.۴ (پایین) شامل مسیری از آراد به سیبیو و بازگشت دوباره به آراد است. می‌گوییم آراد «حالت تکراری» (Repeated State) در درخت جستجوست که در این مورد توسط چرخه (Cycle) — که مسیر حلقه‌ای (Loopy Path) نیز نامیده می‌شود — تولید شده. پس حتی با آنکه فضای حالت فقط ۲۰ حالت دارد، درخت جستجوی کامل بی‌نهایت است، زیرا محدودیتی برای تعداد دفعات طی یک حلقه نیست.

چرخه حالت خاصی از «مسیر زائد» (Redundant Path) است. برای نمونه، می‌توان از مسیر آراد–سیبیو (۱۴۰ مایل) یا مسیر آراد–زرند–اورادئا–سیبیو (۲۹۷ مایل) به سیبیو رسید. این مسیر دوم زائد است — فقط راهی بدتر برای رسیدن به همان حالت است — و در جست‌وجوی مسیرهای بهینه نیازی به در نظر گرفتنش نیست.

عاملی را در جهان شبکه‌ای ۱۰×۱۰ در نظر بگیرید با توانایی حرکت به هر یک از ۸ خانهٔ مجاور. اگر مانعی نباشد، عامل می‌تواند در ۹ حرکت یا کمتر به هر یک از ۱۰۰ خانه برسد. اما شمار مسیرهای به‌طول ۹ تقریباً 8^9 است (کمی کمتر به‌سبب لبه‌های شبکه)، یعنی بیش از ۱۰۰ میلیون. به بیان دیگر، میانگین هر خانه با بیش از یک میلیون مسیر زائد

به‌طول ۹ قابل دسترسی است و اگر مسیرهای زائد را حذف کنیم، می‌توانیم جستجو را تقریباً یک میلیون بار سریع‌تر کامل کنیم. چنان‌که ضرب‌المثل می‌گوید: الگوریتم‌هایی که Δ گذشته را به خاطر نمی‌سپارند، محکوم به تکرارش هستند. سه رویکرد برای این مسئله هست.

نخست، می‌توانیم همهٔ حالت‌های قبلاً رسیده را به خاطر بسپاریم (چنان‌که جستجوی بهترین-اول می‌کند) تا بتوانیم همهٔ مسیرهای زائد را شناسایی کنیم و فقط بهترین مسیر به هر حالت را نگه داریم. این برای فضاهای حالتی که مسیرهای زائد فراوان دارند مناسب است و انتخاب ترجیحی وقتی است که جدول حالت‌های رسیده در حافظه جا شود.

دوم، می‌توانیم نگران تکرار گذشته نباشیم. برخی صورت‌بندی‌های مسئله هست که در آن‌ها نادر یا ناممکن است دو مسیر به همان حالت برسند. مثالی مسئلهٔ مونتاژ است که در آن هر کنش قطعه‌ای به مجموعهٔ در حال شکل‌گیری می‌افزاید و ترتیبی از قطعات هست که افزودن A و سپس B ممکن است اما B و سپس A نه. برای آن مسائل، اگر حالت‌های رسیده را پیگیری نکنیم و برای مسیرهای زائد بررسی نکنیم، در حافظه صرفه‌جویی می‌کنیم. الگوریتم جستجو را «جستجوی گرافی» (Graph Search) می‌نامیم اگر مسیرهای زائد را بررسی کند و «جستجوی درخت‌مانند» (Tree-Like Search)[6] اگر بررسی نکند. الگوریتم BEST-FIRST-SEARCH در شکل ۳.۷ الگوریتم جستجوی گرافی است؛ اگر همهٔ ارجاع‌ها به reached را حذف کنیم، جستجوی درخت‌مانندی می‌گیریم که حافظهٔ کمتری مصرف می‌کند اما مسیرهای زائد به همان حالت را بررسی می‌کند و بنابراین کندتر اجرا می‌شود.

سوم، می‌توانیم مصالحه کنیم و چرخه‌ها را بررسی کنیم، اما نه مسیرهای زائد را به‌طور کلی. چون هر گره زنجیره‌ای از اشاره‌گرهای والد دارد، می‌توانیم چرخه‌ها را بدون نیاز به حافظهٔ اضافی با دنبال‌کردن زنجیرهٔ والدها بررسی کنیم تا ببینیم آیا حالتِ انتهای مسیر در بخش‌های پیشین مسیر ظاهر شده. برخی پیاده‌سازی‌ها این زنجیره را تا بالا دنبال می‌کنند و بدین ترتیب همهٔ چرخه‌ها را حذف می‌کنند؛ برخی دیگر فقط چند پیوند را دنبال می‌کنند (مثلاً تا والد، پدربزرگ و پدربزرگِ پدر) و بدین ترتیب زمان ثابتی می‌برند و همهٔ چرخه‌های کوتاه را حذف می‌کنند (و برای چرخه‌های بلند به سازوکارهای دیگر تکیه می‌کنند).

۳.۳.۴ سنجش عملکرد حل مسئله

پیش از ورود به طراحی الگوریتم‌های جستجوی گوناگون، معیارهای به‌کاررفته برای انتخاب میان آن‌ها را بررسی می‌کنیم. عملکرد الگوریتم را از چهار راه می‌توان ارزیابی کرد:

- **کامل بودن (Completeness):** آیا الگوریتم تضمین می‌کند راه‌حل بیابد وقتی راه‌حلی هست، و شکست را درست گزارش کند وقتی نیست؟
- **بهینگی هزینه (Cost Optimality):** آیا راه‌حلی با کمترین هزینهٔ مسیر در میان همهٔ راه‌حل‌ها می‌یابد؟[7]
- **پیچیدگی زمانی (Time Complexity):** یافتن راه‌حل چقدر طول می‌کشد؟ این را می‌توان بر حسب ثانیه سنجید، یا انتزاعی‌تر با شمار حالت‌ها و کنش‌های در نظرگرفته‌شده.
- **پیچیدگی فضایی (Space Complexity):** برای انجام جستجو چقدر حافظه لازم است؟

برای فهم کامل بودن، مسئلهٔ جستجویی با یک هدف واحد را در نظر بگیرید. آن هدف می‌تواند هر جای فضای حالت باشد؛ پس الگوریتم کامل باید بتواند هر حالتی را که از حالت آغازین قابل دسترسی است به‌شکل نظام‌مند کاوش کند. در فضاهای حالت متناهی این سرراست به دست می‌آید: تا وقتی مسیرها را پیگیری و مسیرهای چرخه‌ای را قطع کنیم (مانند آراد به سیبیو به آراد)، سرانجام به هر حالت قابل دسترسی می‌رسیم.

در فضاهای حالت بی‌نهایت، دقت بیشتری لازم است. برای نمونه، الگوریتمی که مکرراً عملگر «فاکتوریل» را در مسئلهٔ «۴» کنوت اعمال کند، مسیر بی‌نهایتی از ۴ به ۴! به (۴!)! و الی آخر را دنبال می‌کند. به همین ترتیب، روی شبکهٔ بی‌نهایت بدون مانع، حرکت مکرر به جلو در خط راست نیز مسیر بی‌نهایتی از حالت‌های تازه را دنبال می‌کند. در هر دو مورد الگوریتم هرگز به حالتی که قبلاً رسیده بازنمی‌گردد، اما ناقص است، زیرا پهنه‌های گسترده‌ای از فضای حالت هرگز رسیده نمی‌شوند.

برای کامل بودن، الگوریتم جستجو باید در شیوهٔ کاوش فضای حالت بی‌نهایت «نظام‌مند» (Systematic) باشد و مطمئن شود سرانجام می‌تواند به هر حالتی برسد که به حالت آغازین متصل است. برای نمونه، روی شبکهٔ بی‌نهایت، یک نوع جستجوی نظام‌مند مسیر مارپیچی است که همهٔ خانه‌های فاصلهٔ s از مبدأ را می‌پوشاند پیش از آنکه به خانه‌های فاصلهٔ s+1 برود.

متأسفانه، در فضای حالت بی‌نهایت بدون راه‌حل، الگوریتم درست باید برای همیشه به جستجو ادامه دهد؛ نمی‌تواند پایان یابد، زیرا نمی‌تواند بداند حالت بعدی هدف خواهد بود یا نه.

پیچیدگی زمانی و فضایی نسبت به معیاری از دشواری مسئله در نظر گرفته می‌شود. در علوم رایانهٔ نظری، معیار معمول اندازهٔ گراف فضای حالت است: |E| + |V| که |V| شمار رأس‌ها (گره‌های حالت) و |E| شمار یال‌ها (جفت‌های حالت/کنش متمایز) است. این وقتی مناسب است که گراف ساختار دادهٔ صریحی باشد، مانند نقشهٔ رومانی. اما در بسیاری از مسائل هوش مصنوعی، گراف فقط به‌صورت ضمنی با حالت آغازین، کنش‌ها و مدل گذار بازنمایی می‌شود. برای فضای حالت ضمنی، پیچیدگی را می‌توان بر حسب این‌ها سنجید: d، «عمق» (Depth) یا شمار کنش‌های راه‌حل بهینه؛ m، بیشینهٔ شمار کنش‌ها در هر مسیر؛ و b، «ضریب انشعاب» (Branching Factor) یا شمار جانشین‌های گره‌ای که باید در نظر گرفته شوند.

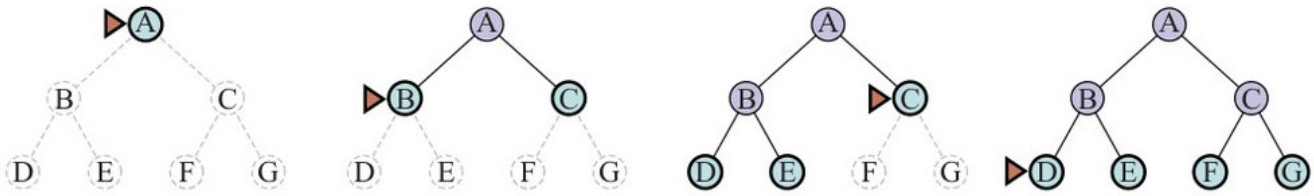
۳.۴ راهبردهای جستجوی نامطلع

به الگوریتم جستجوی نامطلع هیچ سرنخی دربارهٔ نزدیکی حالت به هدف(ها) داده نمی‌شود. برای نمونه، عامل ما در آراد با هدف رسیدن به بخارست را در نظر بگیرید. عامل نامطلعی که دانشی از جغرافیای رومانی ندارد، هیچ سرنخی ندارد که رفتن به زرند یا سیبیو گام اول بهتری است. در برابر، عامل مطلع (بخش ۳.۵) که مکان هر شهر را می‌داند، می‌داند سیبیو به بخارست بسیار نزدیک‌تر است و بنابراین به احتمال بیشتر روی کوتاه‌ترین مسیر است.

۳.۴.۱ جستجوی سطح-اول

وقتی همهٔ کنش‌ها هزینهٔ یکسان دارند، راهبرد مناسب «جستجوی سطح-اول» (Breadth-First Search) است که در آن نخست گرهٔ ریشه گسترش می‌یابد، سپس همهٔ جانشین‌های گرهٔ ریشه، سپس جانشین‌های آن‌ها، و همین‌طور ادامه. این راهبرد جستجوی نظام‌مندی است و بنابراین حتی در فضاهای حالت بی‌نهایت نیز کامل است. می‌توانیم جستجوی سطح-اول را به‌صورت فراخوانی BEST-FIRST-SEARCH پیاده کنیم که تابع ارزیابی f(n) عمق گره است — یعنی شمار کنش‌های لازم برای رسیدن به گره.

اما با چند حربه می‌توانیم کارایی بیشتری بگیریم. صف FIFO از صف اولویت‌دار سریع‌تر خواهد بود و ترتیب درست گره‌ها را به ما می‌دهد: گره‌های تازه (که همیشه از والدهایشان عمیق‌ترند) به انتهای صف می‌روند و گره‌های قدیمی، که از گره‌های تازه کم‌عمق‌ترند، اول گسترش می‌یابند. افزون بر این، reached می‌تواند مجموعه‌ای از حالت‌ها باشد نه نگاشتی از حالت‌ها به گره‌ها، زیرا وقتی به حالتی رسیده‌ایم، هرگز نمی‌توانیم مسیر بهتری به آن حالت بیابیم. آن نیز بدان معناست که می‌توانیم «آزمون هدف زودهنگام» (Early Goal Test) انجام دهیم — بررسی کنیم آیا گره به‌محض تولید شدن راه‌حل است — به‌جای «آزمون هدف دیرهنگام» (Late Goal Test) که جستجوی بهترین-اول به کار می‌برد و تا بیرون‌آمدن گره از صف صبر می‌کند. شکل ۳.۸ پیشرفت جستجوی سطح-اول را روی درخت دودویی نشان می‌دهد و شکل ۳.۹ الگوریتم را با بهبودهای کاراییِ آزمون زودهنگام نشان می‌دهد.



شکل ۳.۸ — جستجوی سطح-اول روی درخت دودویی ساده. در هر مرحله، گره‌ای که بعدی گسترش می‌یابد با نشانگر مثلثی مشخص شده.

جستجوی سطح-اول همیشه راه‌حلی با کمترین شمار کنش می‌یابد، زیرا وقتی گره‌هایی در عمق d تولید می‌کند، همهٔ گره‌های عمق d-1 را قبلاً تولید کرده و اگر یکی از آن‌ها راه‌حل بود، یافته می‌شد. این بدان معناست برای مسائلی که همهٔ کنش‌ها هزینهٔ یکسان دارند بهینهٔ هزینه است، اما نه برای مسائلی که آن ویژگی را ندارند. در هر دو مورد کامل است.

از نظر زمان و فضا، جستجو در درختی یکنواخت را تصور کنید که هر حالت b جانشین دارد. ریشهٔ درخت جستجو b گره تولید می‌کند، هر یک b گرهٔ دیگر، و در مجموع b^2 در سطح دوم. هر یک از این‌ها b گرهٔ دیگر تولید می‌کند و b^3 گره در سطح سوم به دست می‌آید، و همین‌طور ادامه. حال فرض کنیم راه‌حل در عمق d است. پس شمار کل گره‌های تولیدشده برابر است با:

```
b + b^2 + b^3 + ... + b^d = O(b^d) + 1
```

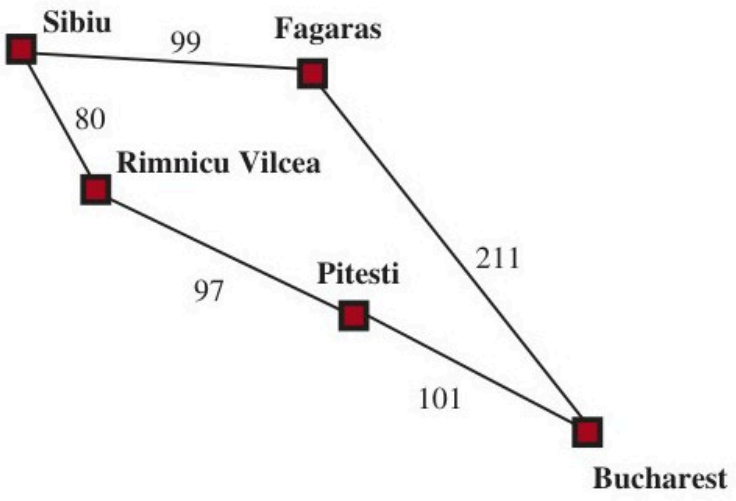
همهٔ گره‌ها در حافظه می‌مانند، پس پیچیدگی زمانی و فضایی هر دو $O(b^d)$ است. کران‌های نمایی از این دست ترسناک‌اند. به‌عنوان مثال معمولِ جهان واقعی، مسئله‌ای با ضریب انشعاب $b = 10$، سرعت پردازش ۱ میلیون گره در ثانیه و نیاز حافظهٔ ۱ کیلوبایت به ازای هر گره در نظر بگیرید. جستجو تا عمق $d = 10$ کمتر از ۳ ساعت طول می‌کشد، اما ۱۰ ترابایت حافظه نیاز دارد. نیازهای حافظه برای جستجوی سطح-اول ⚠ مشکل بزرگ‌تری از زمان اجراست. اما زمان همچنان عامل مهمی است. در عمق $d = 14$، حتی با حافظهٔ بی‌نهایت، جستجو ۳٫۵ سال طول می‌کشد. در حالت کلی، مسائل جستجوی با پیچیدگی نمایی را ⚠ برای هیچ‌چیز جز کوچک‌ترین نمونه‌ها نمی‌توان با جستجوی نامطلع حل کرد.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
                                        node ← NODE(problem.INITIAL)
                                if problem.IS-GOAL(node.STATE) then return node
                               frontier ← a FIFO queue, with node as an element
                                             reached ← {problem.INITIAL}
                                          while not IS-EMPTY(frontier) do
                                                  node ← POP(frontier)
                                    for each child in EXPAND(problem, node) do
                                                       s ← child.STATE
                                     if problem.IS-GOAL(s) then return child
                                                if s is not in reached then
                                                        add s to reached
                                                    add child to frontier
                                                               return failure

                          function UNIFORM-COST-SEARCH(problem) returns a solution node or failure
                                               return BEST-FIRST-SEARCH(problem, PATH-COST)
```

شکل ۳.۹ — الگوریتم‌های جستجوی سطح-اول و جستجوی هزینه-یکنواخت.

۳.۴.۲ الگوریتم دیکسترا یا جستجوی هزینه-یکنواخت

وقتی کنش‌ها هزینه‌های متفاوت دارند، انتخاب بدیهی به‌کارگیری جستجوی بهترین-اول با تابع ارزیابیِ هزینهٔ مسیر از ریشه تا گرهٔ کنونی است. این را جامعهٔ علوم رایانهٔ نظری «الگوریتم دیکسترا» و جامعهٔ هوش مصنوعی «جستجوی هزینه-یکنواخت» (Uniform-Cost Search) می‌نامد. ایده این است که جستجوی سطح-اول در موج‌هایی با عمق یکنواخت پخش می‌شود — نخست عمق ۱، سپس عمق ۲ و الی آخر — حال آنکه جستجوی هزینه-یکنواخت در موج‌هایی با هزینهٔ مسیر یکنواخت پخش می‌شود. الگوریتم را می‌توان به‌صورت فراخوانی BEST-FIRST-SEARCH با PATH-COST به‌عنوان تابع ارزیابی پیاده کرد، چنان‌که در شکل ۳.۹ نشان داده شده.



شکل ۳.۱۰ – بخشی از فضای حالت رومانی، برگزیده برای نشان دادن جستجوی هزینه-یکنواخت. (شهرها: سیبیو، فاگاراش، پیتشتی، ریمنیکو ویلچا، بخارست)

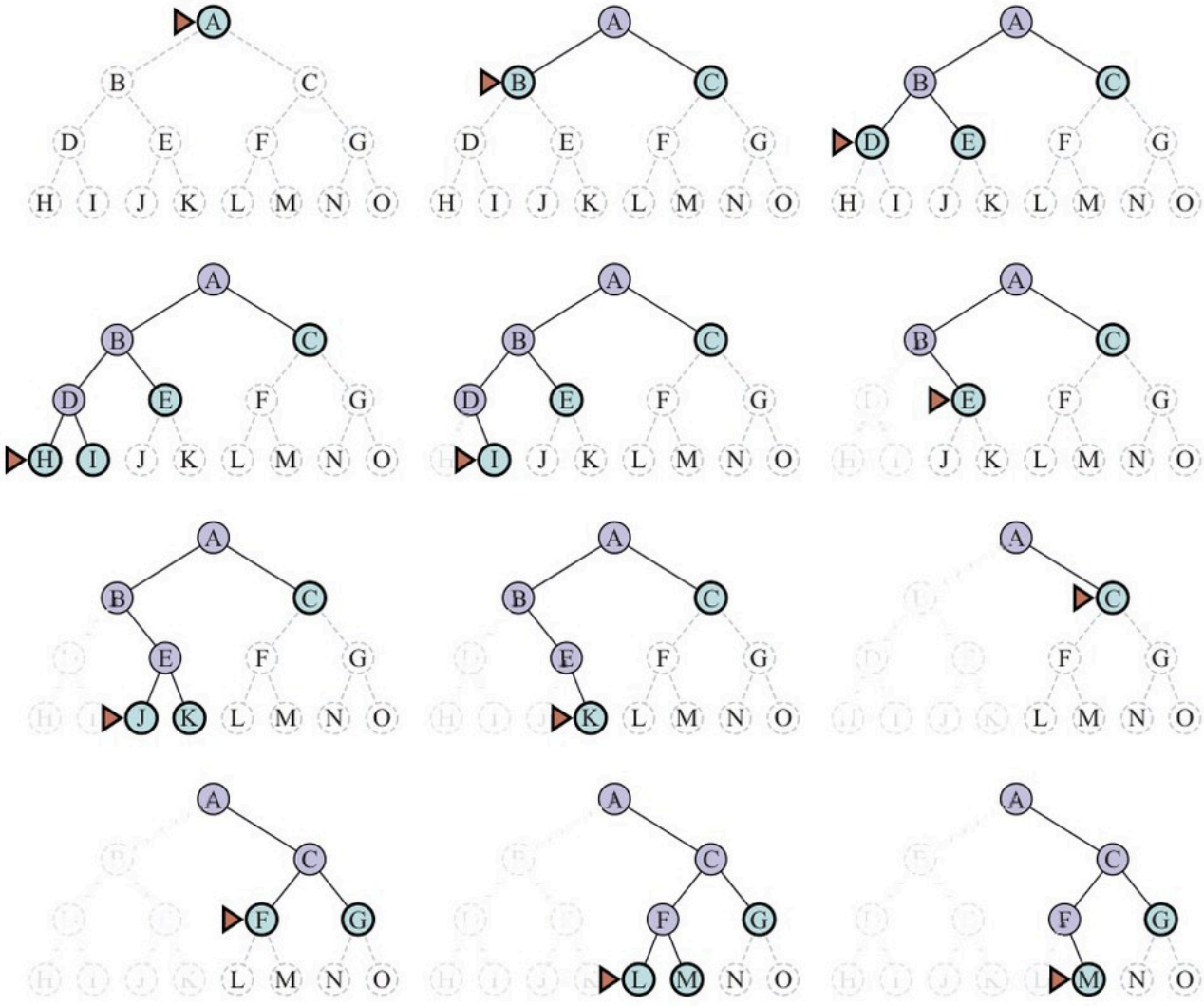
شکل ۳.۱۰ را در نظر بگیرید که مسئله رسیدن از سیبیو به بخارست است. جانشین‌های سیبیو ریمنیکو ویلچا و فاگاراش‌اند با هزینه‌های ۸۰ و ۹۹. گرهٔ کم‌هزینه‌ترین، ریمنیکو ویلچا، بعدی گسترش می‌یابد و پیتشتی را با هزینهٔ ۸۰+۹۷=۱۷۷ می‌افزاید. اکنون کم‌هزینه‌ترین گره فاگاراش است، پس گسترش می‌یابد و بخارست را با هزینهٔ ۹۹+۲۱۱=۳۱۰ می‌افزاید. بخارست هدف است، اما الگوریتم هدف‌ها را فقط هنگام گسترش گره آزمایش می‌کند، نه هنگام تولید گره، پس هنوز تشخیص نداده که این مسیری به هدف است. الگوریتم ادامه می‌دهد، پیتشتی را برای گسترش برمی‌گزیند و مسیر دومی به بخارست با هزینهٔ ۸۰+۹۷+۱۰۱=۲۷۸ می‌افزاید. هزینهٔ آن کمتر است، پس مسیر قبلی را در reached جایگزین می‌کند و به جبهه افزوده می‌شود. معلوم می‌شود این گره اکنون کمترین هزینه را دارد، پس بعدی در نظر گرفته می‌شود، هدف یافت می‌شود و بازگردانده می‌شود. توجه کنید اگر هنگام تولید گره به‌جای هنگام گسترش کم‌هزینه‌ترین گره، هدف را بررسی کرده بودیم، مسیر پرهزینه‌تر (آن که از فاگاراش می‌گذرد) را بازگردانده بودیم.

پیچیدگی جستجوی هزینه-یکنواخت بر حسب C^*، هزینهٔ راه‌حل بهینه،[8] و ε، کران پایینی بر هزینهٔ هر کنش با $\varepsilon > 0$، توصیف می‌شود. آنگاه پیچیدگی زمانی و فضایی بدترین حالت الگوریتم $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ است که می‌تواند بسیار بزرگ‌تر از b^d باشد. این به‌سبب آن است که جستجوی هزینه-یکنواخت می‌تواند درخت‌های بزرگی از کنش‌های کم‌هزینه را پیش از کاوشِ مسیرهای شامل کنشی پرخرج و شاید مفید کاوش کند. وقتی همهٔ هزینه‌های کنش برابرند، $b^{1+\lfloor C^*/\varepsilon \rfloor}$ همان b^{d+1} است و جستجوی هزینه-یکنواخت شبیه جستجوی سطح-اول است.

جستجوی هزینه-یکنواخت کامل و بهینهٔ هزینه است، زیرا نخستین راه‌حلی که می‌یابد هزینه‌ای دست‌کم به کمی هزینهٔ هر گره دیگر جبهه دارد. جستجوی هزینه-یکنواخت همهٔ مسیرها را به‌صورت نظام‌مند به‌ترتیب هزینهٔ فزاینده در نظر می‌گیرد و هرگز در مسیر بی‌نهایتِ تکی گیر نمی‌افتد (با فرض اینکه همهٔ هزینه‌های کنش $\varepsilon > 0$ هستند).

۳.۴.۳ جستجوی عمق-اول و مسئلهٔ حافظه

جستجوی عمق-اول (Depth-First Search) همیشه نخست عمیق‌ترین گرهٔ جبهه را گسترش می‌دهد. می‌توان آن را به‌صورت فراخوانی BEST-FIRST-SEARCH پیاده کرد که تابع ارزیابی f منفی عمق است. اما معمولاً نه به‌صورت جستجوی گرافی، بلکه به‌صورت جستجوی درخت‌مانندی پیاده می‌شود که جدول حالت‌های رسیده نگه نمی‌دارد. پیشرفت جستجو در شکل ۳.۱۱ نشان داده شده؛ جستجو بلافاصله به عمیق‌ترین سطح درخت جستجو می‌رود، جایی که گره‌ها جانشینی ندارند. سپس جستجو «به عقب برمی‌گردد» به بعدی‌ترین گره‌ای که هنوز جانشین‌های گسترش‌نیافته دارد.



شکل ۳.۱۱ – دوازده گام (چپ به راست، بالا به پایین) در پیشرفت جستجوی عمق-اول روی درخت دودویی از حالت آغازین A به هدف M. جبهه سبز است و مثلث گره‌ای را که بعدی گسترش می‌یابد نشان می‌دهد. گره‌های قبلاً گسترش‌یافته بنفش‌اند و گره‌های بالقوهٔ آینده خط‌چین کمرنگ دارند. گره‌های گسترش‌یافتهٔ بدون نواده در جبهه (خط‌های بسیار کمرنگ) را می‌توان دور ریخت.

جستجوی عمق-اول بهینهٔ هزینه نیست؛ نخستین راه‌حلی را که بیابد بازمی‌گرداند، حتی اگر ارزان‌ترین نباشد. برای فضاهای حالت متناهیِ درخت‌شکل کارآمد و کامل است؛ برای فضاهای حالت بدون چرخه ممکن است همان حالت را بارها از مسیرهای گوناگون گسترش دهد، اما (سرانجام) کل فضا را به‌شکل نظام‌مند کاوش خواهد کرد. در فضاهای

حالت چرخه‌ای می‌تواند در حلقهٔ بی‌نهایت گیر کند؛ بنابراین برخی پیاده‌سازی‌های جستجوی عمق-اول هر گرهٔ تازه را از نظر چرخه بررسی می‌کنند. سرانجام، در فضاهای حالت بی‌نهایت، جستجوی عمق-اول نظام‌مند نیست: می‌تواند در مسیر بی‌نهایتی گیر کند، حتی اگر چرخه‌ای نباشد. پس جستجوی عمق-اول ناقص است.

با این همه خبر بد، چرا کسی باید به‌جای جستجوی سطح-اول یا بهترین-اول از جستجوی عمق-اول استفاده کند؟ پاسخ آن است که برای مسائلی که جستجوی درخت‌مانند در آن‌ها شدنی است، جستجوی عمق-اول نیازهای حافظهٔ بسیار کمتری دارد. ما اصلاً جدول reached نگه نمی‌داریم و جبهه بسیار کوچک است: جبهه در جستجوی سطح-اول را سطحِ کره‌ای همیشه‌درحال‌گسترش تصور کنید، حال آنکه جبهه در جستجوی عمق-اول فقط شعاع کره است.

برای فضای حالت متناهیِ درخت‌شکل مانند شکل ۳.۱۱، جستجوی درخت‌مانند عمق-اول زمانی متناسب با شمار حالت‌ها می‌برد و پیچیدگی حافظه‌ای فقط O(bm) دارد، که b ضریب انشعاب و m بیشینهٔ عمق درخت است. برخی مسائل که با جستجوی سطح-اول به اگزابایت حافظه نیاز دارند، با جستجوی عمق-اول فقط با چند کیلوبایت مدیریت می‌شوند. به‌سبب مصرف کم‌خرجِ حافظه، جستجوی درخت‌مانند عمق-اول به‌عنوان «اسب‌کار» پایهٔ بسیاری از حوزه‌های هوش مصنوعی برگزیده شده، از جمله ارضای قیود (فصل ۶)، ارضاپذیری گزاره‌ای (فصل ۷) و برنامه‌نویسی منطقی (فصل ۹). تغییری از جستجوی عمق-اول به نام جستجوی بازگشتی (Backtracking Search) حافظهٔ حتی کمتری مصرف می‌کند. (فصل ۶ را برای جزئیات بیشتر ببینید.) در بازگشت، در هر لحظه فقط یک جانشین تولید می‌شود نه همهٔ جانشین‌ها؛ هر گرهٔ نیمه‌گسترش‌یافته به خاطر می‌سپارد کدام جانشین را بعدی تولید کند. افزون بر این، جانشین‌ها با اصلاح مستقیم توصیف حالت کنونی تولید می‌شوند، نه با تخصیص حافظه برای حالت کاملاً تازه. این نیازهای حافظه را به فقط یک توصیف حالت و مسیری از O(m) کنش کاهش می‌دهد؛ صرفه‌جویی چشمگیری نسبت به O(bm) حالت در جستجوی عمق-اول. با بازگشت همچنین این گزینه را داریم که ساختار دادهٔ مجموعه‌ای کارآمد برای حالت‌های مسیر کنونی نگه داریم تا چرخه را در زمان O(1) بررسی کنیم نه O(m). برای کارکردن بازگشت، باید بتوانیم هر کنش را هنگام بازگشت به عقب، خنثی کنیم. بازگشت برای موفقیت بسیاری از مسائل با توصیف حالت بزرگ، مانند مونتاژ رباتیک، حیاتی است.

۳.۴.۴ جستجوی محدود-به-عمق و تعمیق تکراری

برای بازداشتن جستجوی عمق-اول از سرگردانی در مسیر بی‌نهایت، می‌توانیم از جستجوی محدود-به-عمق (Depth-Limited Search) استفاده کنیم؛ نسخه‌ای از جستجوی عمق-اول که در آن حد عمق ℓ می‌دهیم و همهٔ گره‌های عمق ℓ را چنان می‌پنداریم که گویی جانشینی ندارند (شکل ۳.۱۲). پیچیدگی زمانی O(b^ℓ) و پیچیدگی فضایی O(bℓ) است. متأسفانه، اگر ℓ را بد انتخاب کنیم، الگوریتم به راه‌حل نخواهد رسید و دوباره ناقص می‌شود.

چون جستجوی عمق-اول جستجوی درخت‌مانندی است، نمی‌توانیم به‌طور کلی جلوی تلف‌کردن زمان در مسیرهای زائد را بگیریم، اما می‌توانیم چرخه‌ها را به بهای اندکی زمان محاسبه حذف کنیم. اگر فقط چند پیوند در زنجیرهٔ والدها نگاه کنیم، بیشتر چرخه‌ها را می‌گیریم؛ چرخه‌های بلندتر را حد عمق مدیریت می‌کند. گاهی می‌توان حد عمق خوبی را بر اساس دانش مسئله برگزید. برای نمونه، روی نقشهٔ رومانی ۲۰ شهر هست. پس ℓ=19 حدی معتبر است. اما اگر نقشه

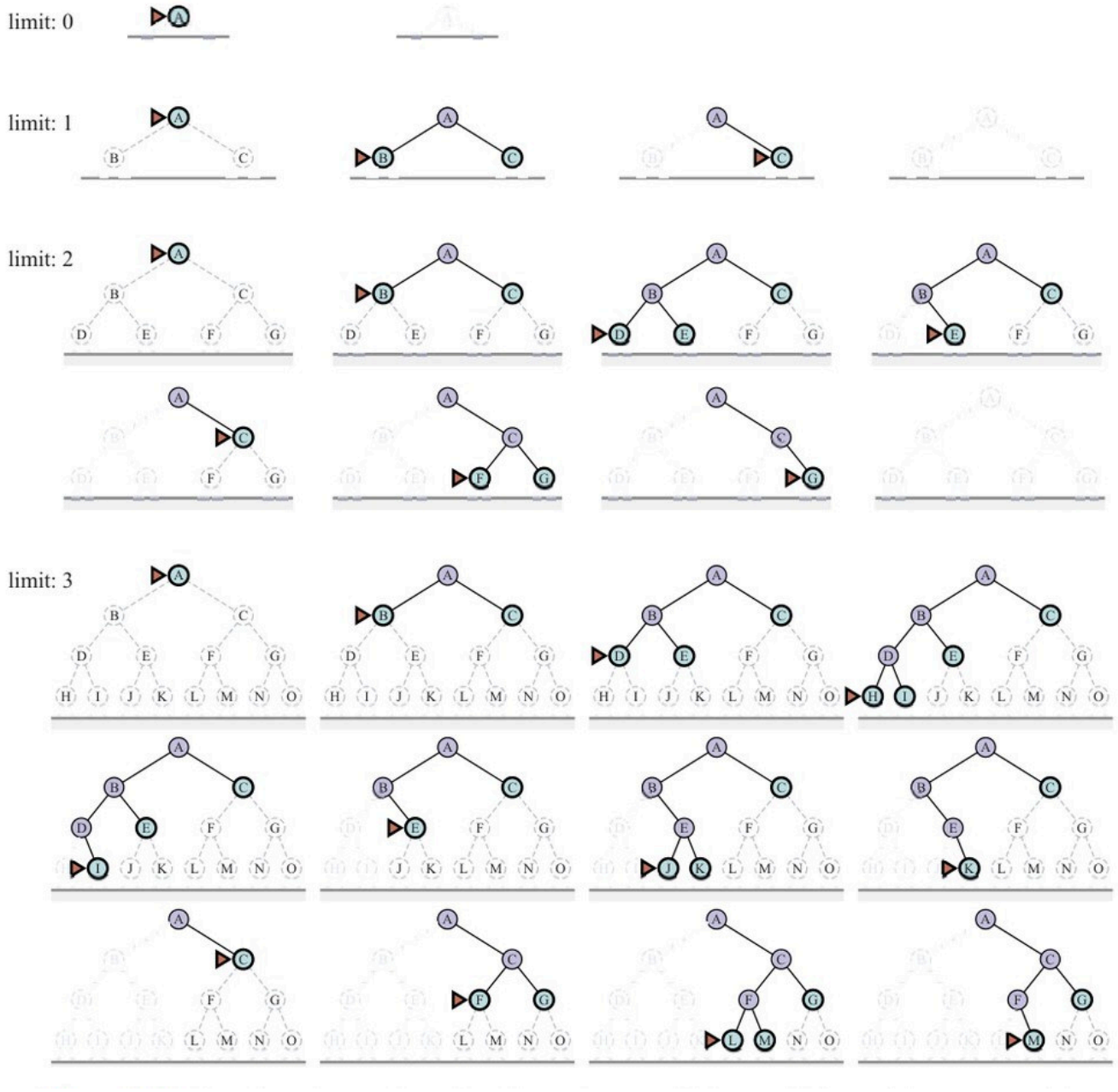
را به‌دقت مطالعه کنیم، کشف می‌کنیم هر شهری را می‌توان در حداکثر ۹ کنش از هر شهر دیگر رسید. این عدد، که «قطر» (Diameter) گراف فضای حالت نامیده می‌شود، حد عمق بهتری به ما می‌دهد که به جستجوی محدود-به-عمق کارآمدتر می‌انجامد. اما برای بیشتر مسائل، حد عمق خوبی را تا حل مسئله نخواهیم شناخت.

جستجوی تعمیق تکراری (Iterative Deepening Search) مسئلهٔ برگزیدن مقدار خوب برای ℓ را با آزمودن همهٔ مقادیر حل می‌کند: نخست ۰، سپس ۱، سپس ۲ و الی آخر — تا آنکه یا راه‌حلی یافت شود، یا جستجوی محدود-به-عمق به‌جای مقدار cutoff مقدار failure بازگرداند. الگوریتم در شکل ۳.۱۲ نشان داده شده. تعمیق تکراری بسیاری از مزیت‌های جستجوی عمق-اول و سطح-اول را ترکیب می‌کند. مانند جستجوی عمق-اول، نیازهای حافظه‌اش اندک است: $O(bd)$ وقتی راه‌حلی هست، یا $O(bm)$ در فضاهای حالت متناهی بدون راه‌حل. مانند جستجوی سطح-اول، تعمیق تکراری برای مسائلی که همهٔ کنش‌ها هزینهٔ یکسان دارند بهینه است، و در فضاهای حالت بدون چرخهٔ متناهی، یا در هر فضای حالت متناهی وقتی گره‌ها را تا بالای مسیر از نظر چرخه بررسی کنیم، کامل است.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
                                                    for depth = 0 to ∞ do
                                        result ← DEPTH-LIMITED-SEARCH(problem, depth)
                                                if result ≠ cutoff then return result

function DEPTH-LIMITED-SEARCH(problem, ℓ) returns a node or failure or cutoff
  frontier ← a LIFO queue (stack) with NODE(problem.INITIAL) as an element
                                                              result ← failure
                                                    while not IS-EMPTY(frontier) do
                                                              node ← POP(frontier)
                                        if problem.IS-GOAL(node.STATE) then return node
                                                          if DEPTH(node) > ℓ then
                                                                result ← cutoff
                                                                          else
                                          for each child in EXPAND(problem, node) do
                                                                add child to frontier
                                                                      return result
```

شکل ۳.۱۲ — جستجوی تعمیق تکراری و جستجوی محدود-به-عمق درخت‌مانند. تعمیق تکراری به‌طور مکرر جستجوی محدود-به-عمق را با حدهای فزاینده اعمال می‌کند. یکی از سه نوع مقدار را بازمی‌گرداند: یا گرهٔ راه‌حل؛ یا failure، وقتی همهٔ گره‌ها را تمام کرده و ثابت کرده در هیچ عمقی راه‌حلی نیست؛ یا cutoff، به این معنا که ممکن است راه‌حلی در عمقی بیشتر از ℓ باشد. این الگوریتم جستجوی درخت‌مانندی است که حالت‌های رسیده را پیگیری نمی‌کند و بنابراین حافظهٔ بسیار کمتری از جستجوی بهترین-اول مصرف می‌کند، اما خطر بازدید همان حالت را چند بار از مسیرهای گوناگون دارد. همچنین، اگر بررسی IS-CYCLE همهٔ چرخه‌ها را بررسی نکند، الگوریتم ممکن است در حلقه گیر کند.

پیچیدگی زمانی $O(b^d)$ است وقتی راه‌حلی هست، یا $O(b^m)$ وقتی نیست. هر تکرار جستجوی تعمیق تکراری سطح تازه‌ای تولید می‌کند، همان‌گونه که جستجوی سطح-اول می‌کند، اما سطح-اول این کار را با ذخیرهٔ همهٔ گره‌ها در حافظه انجام می‌دهد، در حالی که تعمیق تکراری با تکرار سطح‌های قبلی انجامش می‌دهد و بدین ترتیب به بهای زمان بیشتر، حافظه صرفه‌جویی می‌کند. شکل ۳.۱۳ چهار تکرار جستجوی تعمیق تکراری را روی درخت جستجوی دودویی نشان می‌دهد که راه‌حل در تکرار چهارم یافته می‌شود.



شکل ۳.۱۳ – چهار تکرار جستجوی تعمیق تکراری برای هدف M روی درخت دودویی، با حد عمقِ متغیر از ۰ تا ۳. توجه کنید گره‌های داخلی یک مسیر واحد می‌سازند. مثلث گره‌ای را که بعدی گسترش می‌یابد نشان می‌دهد؛ گره‌های سبز با خط تیره روی جبهه‌اند؛ گره‌های بسیار کمرنگ به‌شکل اثبات‌پذیر نمی‌توانند با این حد عمق بخشی از راه‌حل باشند.

جستجوی تعمیق تکراری ممکن است اسراف‌کارانه بنماید، زیرا حالت‌های نزدیک بالای درخت جستجو چند بار بازتولید می‌شوند. اما برای بسیاری از فضاهای حالت، بیشتر گره‌ها در سطح پایین‌اند، پس تکرار سطح‌های بالایی چندان مهم نیست. در جستجوی تعمیق تکراری، گره‌های سطح پایین (عمق d) یک بار تولید می‌شوند، گره‌های سطح ماقبل آخر دو بار، و الی آخر تا فرزندان ریشه که d بار تولید می‌شوند. پس شمار کل گره‌های تولیدشده در بدترین حالت:

```
N(IDS) = (d)b^1 + (d - 1)b^2 + (d - 2)b^3 + ... + (1)b^d
```

که پیچیدگی زمانی $O(b^d)$ می‌دهد — مجانباً همان جستجوی سطح-اول. برای نمونه، اگر b = 10 و d = 5:

```
N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450
N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110
```

اگر واقعاً نگران تکرار هستید، می‌توانید رویکرد ترکیبی به کار گیرید که جستجوی سطح-اول را تا وقتی تقریباً تمام حافظهٔ در دسترس مصرف شود اجرا می‌کند و سپس تعمیق تکراری را از همهٔ گره‌های جبهه اجرا می‌کند. در حالت کلی، تعمیق تکراری روش ترجیحی ◭ جستجوی نامطلع است وقتی فضای حالت جستجو بزرگ‌تر از آن است که در حافظه جا شود و عمق راه‌حل شناخته‌شده نیست.

۳.۴.۵ جستجوی دوطرفه

الگوریتم‌هایی که تاکنون پوشش داده‌ایم از حالت آغازین شروع می‌کنند و می‌توانند به هر یک از حالت‌های هدفِ متعدد برسند. رویکرد جایگزینی به نام جستجوی دوطرفه (Bidirectional Search) به‌طور هم‌زمان از حالت آغازین به جلو و از حالت(های) هدف به عقب جستجو می‌کند، به این امید که دو جستجو به هم برسند. انگیزه آن است که $b^{(d/2)} + b^{(d/2)}$ بسیار کمتر از b^d است (مثلاً ۵۰،۰۰۰ برابر کمتر وقتی $b = d = 10$).

```
function BIBF-SEARCH(problemF, fF, problemB, fB) returns a solution node, or failure
                        nodeF ← NODE(problemF.INITIAL) // Node for a start state
                         nodeB ← NODE(problemB.INITIAL) // Node for a goal state
            frontierF ← a priority queue ordered by fF, with nodeF as an element
            frontierB ← a priority queue ordered by fB, with nodeB as an element
             reachedF ← a lookup table, with one key nodeF.STATE and value nodeF
             reachedB ← a lookup table, with one key nodeB.STATE and value nodeB
                                                    solution ← failure
                       while not TERMINATED(solution, frontierF, frontierB) do
                           if fF(TOP(frontierF)) < fB(TOP(frontierB)) then
    solution ← PROCEED(F, problemF, frontierF, reachedF, reachedB, solution)
  else solution ← PROCEED(B, problemB, frontierB, reachedB, reachedF, solution)
                                                          return solution

function PROCEED(dir, problem, frontier, reached, reached2, solution) returns a solution
          .Expand node on frontier; check against the other frontier in reached2 //
     .The variable "dir" is the direction: either F for forward or B for backward //
                                                      node ← POP(frontier)
                                  for each child in EXPAND(problem, node) do
                                                        s ← child.STATE
        if s not in reached or PATH-COST(child) < PATH-COST(reached[s]) then
                                                  reached[s] ← child
                                              add child to frontier
                                          if s is in reached2 then
                      solution2 ← JOIN-NODES(dir, child, reached2[s])
                if PATH-COST(solution2) < PATH-COST(solution) then
                                          solution ← solution2
                                                          return solution
```

شکل ۳.۱۴ — جستجوی بهترین-اول دوطرفه دو جبهه و دو جدول حالت‌های رسیده نگه می‌دارد. وقتی مسیری در یک جبهه به حالتی می‌رسد که در نیمهٔ دیگر جستجو نیز رسیده شده، دو مسیر (با تابع JOIN-NODES) به هم می‌پیوندند و راه‌حل می‌سازند. نخستین راه‌حلی که می‌گیریم تضمین‌شده بهترین نیست؛ تابع TERMINATED تعیین می‌کند چه‌وقت جست‌وجوی راه‌حل‌های تازه را متوقف کنیم.

برای اینکه این کار کند، باید دو جبهه و دو جدول حالت‌های رسیده پیگیری کنیم و باید بتوانیم به عقب استدلال کنیم: اگر حالت s' جانشین s در جهت جلو است، باید بدانیم s جانشین s' در جهت عقب است. وقتی دو جبهه به هم برخورد کنند راه‌حل داریم.[9] نسخه‌های بسیار متفاوتی از جستجوی دوطرفه هست، همان‌گونه که الگوریتم‌های جستجوی یک‌طرفهٔ بسیاری هست. در این بخش جستجوی بهترین-اول دوطرفه را توصیف می‌کنیم.

هرچند دو جبههٔ جدا هست، گره‌ای که بعدی گسترش می‌یابد همیشه گره‌ای با کمترین مقدار تابع ارزیابی، در هر کدام از جبهه‌ها، است. وقتی تابع ارزیابی هزینهٔ مسیر باشد، جستجوی هزینه-یکنواخت دوطرفه می‌گیریم و اگر هزینهٔ مسیر بهینه C* باشد، هیچ گرهی با هزینهٔ بیشتر از C*/2 گسترش نخواهد یافت. این می‌تواند شتاب قابل توجهی به بار آورد. الگوریتم عمومی جستجوی بهترین-اول دوطرفه در شکل ۳.۱۴ نشان داده شده. دو نسخه از مسئله و تابع ارزیابی را وارد می‌کنیم: یکی در جهت جلو (زیرنویس F) و یکی در جهت عقب (زیرنویس B). وقتی تابع ارزیابی هزینهٔ مسیر است، می‌دانیم نخستین راه‌حلِ یافته‌شده راه‌حل بهینه خواهد بود، اما با توابع ارزیابی متفاوت این لزوماً درست نیست. بنابراین بهترین راه‌حلِ یافته‌شده تاکنون را پیگیری می‌کنیم و ممکن است پیش از آنکه آزمون TERMINATED ثابت کند راه‌حل بهتری باقی نمانده، چند بار به‌روزش کنیم.

۳.۴.۶ مقایسهٔ الگوریتم‌های جستجوی نامطلع

شکل ۳.۱۵ الگوریتم‌های جستجوی نامطلع را بر حسب چهار معیار ارزیابیِ بخش ۳.۳.۴ مقایسه می‌کند. این مقایسه برای نسخه‌های جستجوی درخت‌مانند است که حالت‌های تکراری را بررسی نمی‌کنند. برای جستجوهای گرافی که بررسی می‌کنند، تفاوت‌های اصلی این‌اند که جستجوی عمق-اول برای فضاهای حالت متناهی کامل است و پیچیدگی‌های زمانی و فضایی با اندازهٔ فضای حالت (شمار رأس‌ها و یال‌ها، |V|+|E|) کران‌دار می‌شوند.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[1]	Yes[1,2]	No	No	Yes[1]	Yes[1,4]
Optimal cost?	Yes[3]	Yes	No	No	Yes[3]	Yes[3,4]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

شکل ۳.۱۵ – ارزیابی الگوریتم‌های جستجو. b ضریب انشعاب؛ m بیشینهٔ عمق درخت جستجو؛ d عمق کم‌عمق‌ترین راه‌حل، یا m وقتی راه‌حلی نیست؛ ℓ حد عمق. زیرنویس‌ها: [1] کامل اگر b متناهی باشد و فضای حالت یا راه‌حل داشته باشد یا متناهی باشد. [2] کامل اگر همهٔ هزینه‌های کنش ≥ ε > 0 باشند. [3] بهینهٔ هزینه اگر هزینه‌های کنش همه یکسان باشند. [4] اگر هر دو جهت سطح-اول یا هزینه-یکنواخت باشند.

۳.۵ راهبردهای جستجوی مطلع (اکتشافی)

این بخش نشان می‌دهد راهبرد جستجوی مطلع (Informed Search) – که از سرنخ‌های حوزه‌ویژه دربارهٔ مکان هدف‌ها استفاده می‌کند – چگونه می‌تواند راه‌حل‌ها را کارآمدتر از راهبرد نامطلع بیابد. سرنخ‌ها در قالب تابع اکتشافی (Heuristic Function) می‌آیند که با h(n) نشان داده می‌شود:[10]

```
h(n) = هزینهٔ تخمینی ارزان‌ترین مسیر از حالتِ گرهٔ n تا حالت هدف
```

برای نمونه، در مسائل مسیریابی، می‌توانیم فاصلهٔ حالت کنونی تا هدف را با محاسبهٔ فاصلهٔ خط مستقیم روی نقشه میان دو نقطه تخمین بزنیم. اکتشافی‌ها و سرچشمه‌هایشان را در بخش ۳.۶ با جزئیات بیشتری مطالعه می‌کنیم.

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

شکل ۳.۱۶ – مقادیر hSLD – فاصله‌های خط مستقیم تا بخارست. (شهرها با مقادیر فاصلهٔ خط مستقیم تا بخارست بر حسب مایل)

۳.۵.۱ جستجوی بهترین-اول حریصانه

جستجوی بهترین-اول حریصانه (Greedy Best-First Search) شکلی از جستجوی بهترین-اول است که نخست گره‌ای با کمترین مقدار h(n) را گسترش می‌دهد – گره‌ای که به نظر می‌رسد به هدف نزدیک‌ترین است – به این دلیل که این به احتمال زیاد به‌سرعت به راه‌حل می‌انجامد. پس تابع ارزیابی f(n) = h(n) است.

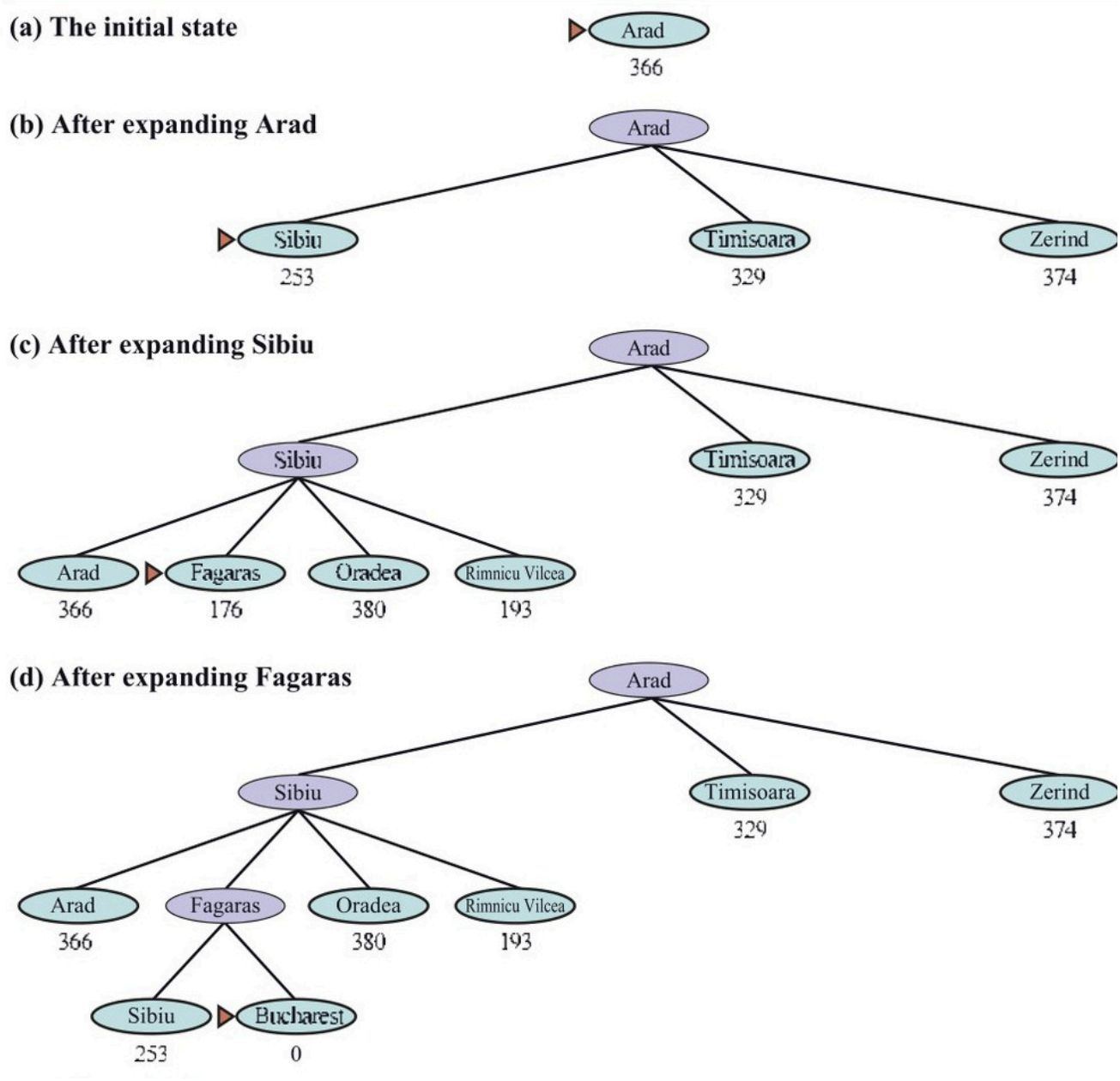
ببینیم این برای مسائل مسیریابی در رومانی چگونه کار می‌کند؛ از اکتشافی فاصلهٔ خط مستقیم استفاده می‌کنیم که hSLD می‌نامیمش. اگر هدف بخارست باشد، باید فاصله‌های خط مستقیم تا بخارست را بدانیم که در شکل ۳.۱۶ نشان داده شده‌اند. برای نمونه، hSLD(Arad)=366. توجه کنید مقادیر hSLD را نمی‌توان از خودِ توصیف مسئله (یعنی توابع ACTIONS و RESULT) محاسبه کرد. وانگهی، دانش جهانی معینی لازم است تا بدانیم hSLD با فاصله‌های جاده‌ای واقعی همبسته است و بنابراین اکتشافیِ مفیدی است.

شکل ۳.۱۷ پیشرفت جستجوی بهترین-اول حریصانه با hSLD را برای یافتن مسیر از آراد به بخارست نشان می‌دهد. نخستین گره‌ای که از آراد گسترش می‌یابد سیبیو خواهد بود، زیرا اکتشافی می‌گوید به بخارست نزدیک‌تر از زرند یا تیمیشوآراست. گرهٔ بعدی که گسترش می‌یابد فاگاراش خواهد بود، زیرا اکنون طبق اکتشافی نزدیک‌ترین است. فاگاراش

به نوبهٔ خود بخارست را تولید می‌کند که هدف است. برای این مسئلهٔ خاص، جستجوی بهترین-اول حریصانه با hSLD بدون گسترش هیچ گره‌ای که روی مسیر راه‌حل نیست، راه‌حل می‌یابد.

اما راه‌حلی که یافت هزینهٔ بهینه ندارد: مسیر از طریق سیبیو و فاگاراش به بخارست ۳۲ مایل از مسیرِ گذرنده از ریمنیکو ویلچا و پیتشتی بلندتر است. به همین دلیل الگوریتم «حریصانه» نامیده می‌شود — در هر تکرار می‌کوشد تا حد ممکن به هدف نزدیک شود، اما حریصی می‌تواند به نتایج بدتر از محتاطی بینجامد.

جستجوی گرافی بهترین-اول حریصانه در فضاهای حالت متناهی کامل است، اما در بی‌نهایت نه. پیچیدگی زمانی و فضایی بدترین حالت $O(|V|)$ است. اما با تابع اکتشافی خوب، پیچیدگی را می‌توان به‌شکل چشمگیری کاهش داد و در برخی مسائل به $O(b^m)$ رساند.



شکل ۳.۱۷ — مراحل جستجوی درخت‌مانند بهترین-اول حریصانه برای بخارست با اکتشافی فاصلهٔ خط مستقیم hSLD. گره‌ها با مقادیر h خود برچسب‌گذاری شده‌اند.

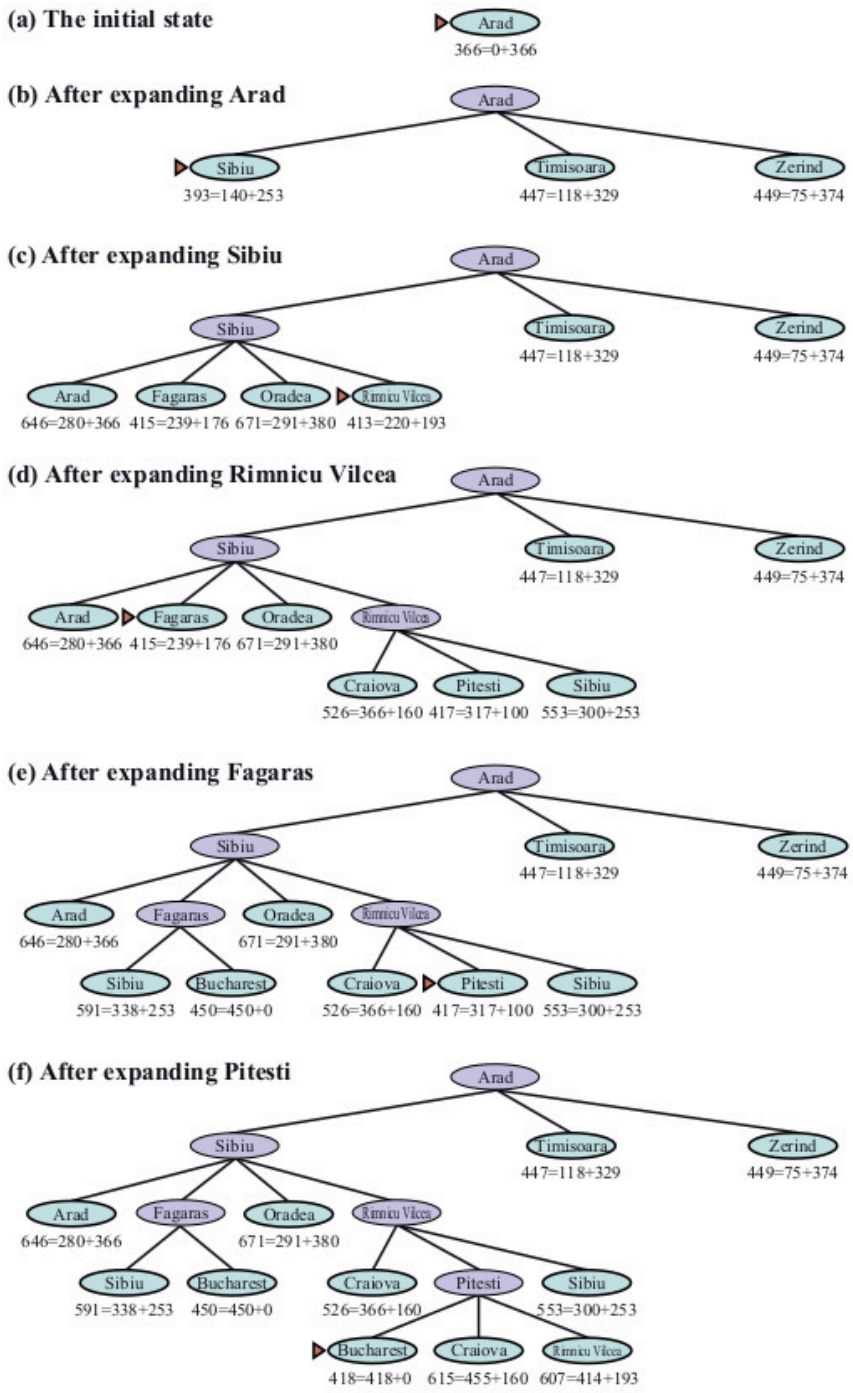
۳.۵.۲ جستجوی A*

رایج‌ترین الگوریتم جستجوی مطلع، جستجوی A* (تلفظ «A-ستار») است؛ جستجوی بهترین-اولی که از تابع ارزیابی f(n) = g(n) + h(n) استفاده می‌کند، که در آن g(n) هزینهٔ مسیر از حالت آغازین تا گرهٔ n و h(n) هزینهٔ تخمینی کوتاه‌ترین مسیر از n تا حالت هدف است؛ پس داریم:

```
f(n) = g(n) + h(n)
```

```
f(n) = هزینهٔ تخمینی ارزان‌ترین راه‌حلِ گذرنده از گرهٔ n
```

در شکل ۳.۱۸ پیشرفت جستجوی A* با هدف رسیدن به بخارست را نشان می‌دهیم. مقادیر g از هزینه‌های کنش در شکل ۳.۱ و مقادیر hSLD در شکل ۳.۱۶ محاسبه می‌شوند. توجه کنید بخارست نخست در گام (e) روی جبهه ظاهر می‌شود، اما برای گسترش برگزیده نمی‌شود (و بنابراین به‌عنوان راه‌حل شناسایی نمی‌شود)، زیرا با f = 450 کم‌هزینه‌ترین گرهٔ جبهه نیست — پیتشتی با f = 417 است. به بیان دیگر، ممکن است راه‌حلی از طریق پیتشتی با هزینه‌ای به‌کمی ۴۱۷ وجود داشته باشد، پس الگوریتم به راه‌حلی با هزینهٔ ۴۵۰ تن نمی‌دهد. در گام (f)، مسیر متفاوتی به بخارست اکنون کم‌هزینه‌ترین گره است، با f = 418، پس برگزیده و به‌عنوان راه‌حل بهینه شناسایی می‌شود.



شکل ۳.۱۸ — مراحل جستجوی A* برای بخارست. گره‌ها با f = g + h برچسب‌گذاری شده‌اند. مقادیر h فاصله‌های خط مستقیم تا بخارست‌اند که از شکل ۳.۱۶ گرفته شده‌اند. (مراحل: (الف) حالت آغازین؛ (ب) پس از گسترش آراد؛ (ج) پس از گسترش سیبیو؛ (د) پس از گسترش ریمنیکو ویلچا؛ (هـ) پس از گسترش فاگاراش؛ (و) پس از گسترش پیتشتی — با مقادیر f مربوط: آراد 366=0+366، سیبیو 393=140+253، تیمیشوآرا 447=118+329، زرند 449=75+374، ... و سرانجام بخارست 418=418+0)

جستجوی A* کامل است.[11] اینکه آیا A* بهینهٔ هزینه است به ویژگی‌های خاصی از اکتشافی بستگی دارد. ویژگی کلیدی «مقبولیت» (Admissibility) است: اکتشافیِ مقبول (Admissible Heuristic) اکتشافی‌ای است که هرگز هزینهٔ رسیدن به هدف را بیش‌تخمین نزند. (اکتشافی مقبول پس خوش‌بینانه است.) با اکتشافی مقبول، A* بهینهٔ هزینه است، که می‌توانیم با برهان خلف نشان دهیم. فرض کنیم مسیر بهینه هزینهٔ C* دارد، اما الگوریتم مسیری با

هزینهٔ C > C* بازمی‌گرداند. پس باید گرهٔ n وجود داشته باشد که روی مسیر بهینه است و گسترش‌نیافته (زیرا اگر همهٔ گره‌های مسیر بهینه گسترش یافته بودند، همان راه‌حل بهینه را بازگردانده بودیم). پس، با نماد g*(n) برای هزینهٔ مسیر بهینه از آغاز تا n و h*(n) برای هزینهٔ مسیر بهینه از n تا نزدیک‌ترین هدف:

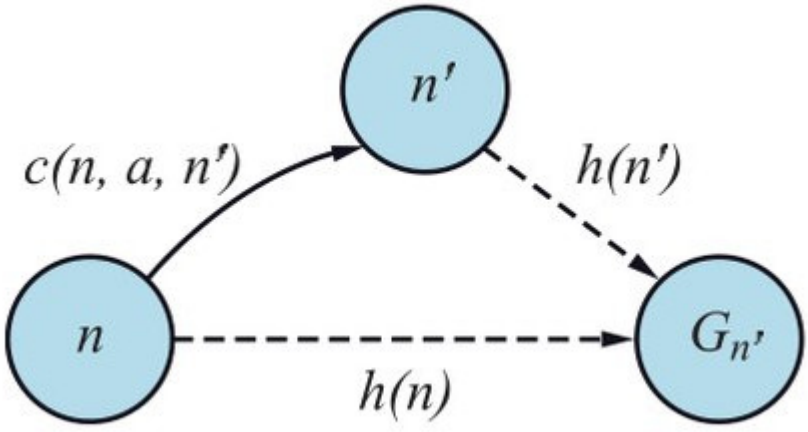
```
f(n) > C*                    (وگرنه n گسترش می‌یافت)
f(n) = g(n) + h(n)           (طبق تعریف)
f(n) = g*(n) + h(n)          (زیرا n روی مسیر بهینه است)
f(n) ≤ g*(n) + h*(n)         (به‌سبب مقبولیت، h(n) ≤ h*(n))
f(n) ≤ C*                    (طبق تعریف، C* = g*(n) + h*(n))
```

سطر اول و آخر تناقض‌اند، پس این فرض که الگوریتم می‌تواند مسیر غیربهینه بازگرداند باید نادرست باشد – باید A* فقط مسیرهای بهینهٔ هزینه بازگرداند.

ویژگی کمی قوی‌تر «سازگاری» (Consistency) نامیده می‌شود. اکتشافی h(n) سازگار است اگر برای هر گرهٔ n و هر جانشین n' از n که توسط کنش a تولید می‌شود، داشته باشیم:

```
h(n) ≤ c(n, a, n′) + h(n′)
```

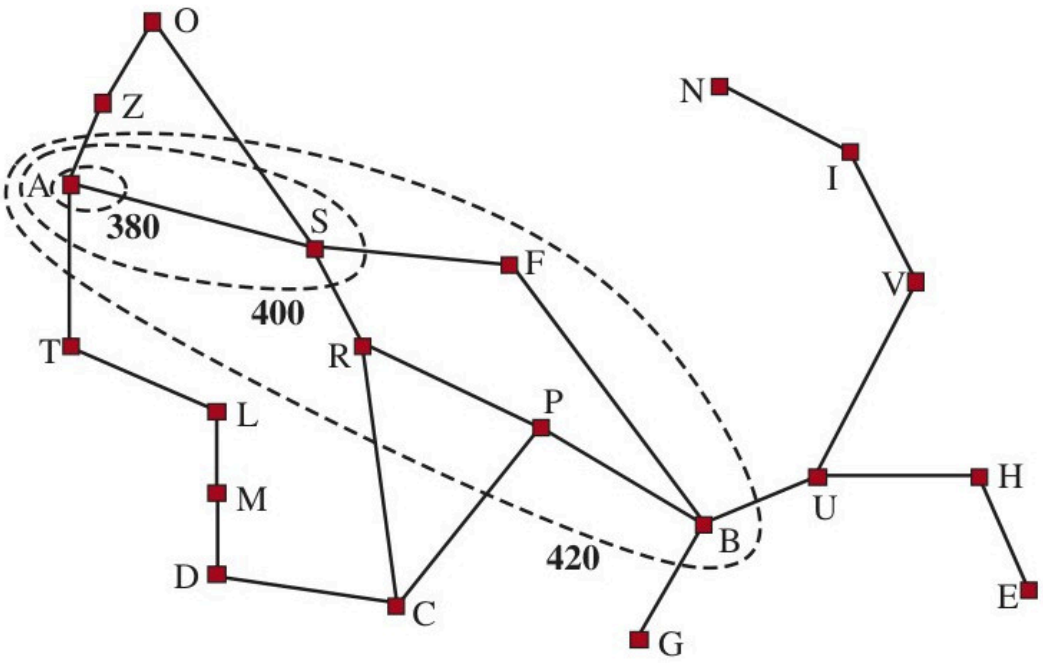
این شکلی از نابرابری مثلث (Triangle Inequality) است که مقرر می‌دارد ضلعی از مثلث نمی‌تواند از مجموع دو ضلع دیگر بلندتر باشد (شکل ۳.۱۹). نمونهٔ اکتشافی سازگار، فاصلهٔ خط مستقیم hSLD است که برای رسیدن به بخارست به کار بردیم.



شکل ۳.۱۹ – نابرابری مثلث: اگر اکتشافی h سازگار باشد، عدد تکی h(n) از مجموع هزینهٔ کنش c(n,a,n') از n به n' به‌علاوهٔ تخمین اکتشافی h(n') کوچک‌تر خواهد بود.

هر اکتشافی سازگار مقبول است (اما نه برعکس)، پس با اکتشافی سازگار، A* بهینهٔ هزینه است. افزون بر این، با اکتشافی سازگار، نخستین باری که به حالتی می‌رسیم روی مسیر بهینه خواهد بود، پس هرگز لازم نیست حالتی را دوباره به جبهه بیفزاییم و هرگز لازم نیست مدخلی در reached تغییر دهیم. اما با اکتشافی ناسازگار، ممکن است به چند مسیرِ رساننده به همان حالت برسیم و اگر هر مسیر تازه هزینهٔ کمتری از قبلی داشته باشد، چند گره برای آن

حالت در جبهه خواهیم داشت که هم زمان و هم فضا برایمان هزینه دارد. به همین دلیل، برخی پیاده‌سازی‌های A* مراقبت می‌کنند حالتی را فقط یک بار وارد جبهه کنند و اگر مسیر بهتری به حالت یافت شد، همهٔ جانشین‌های حالت به‌روز شوند (که مستلزم آن است گره‌ها افزون بر اشاره‌گر والد، اشاره‌گر فرزند هم داشته باشند). این پیچیدگی‌ها بسیاری از پیاده‌سازان را واداشته از اکتشافی‌های ناسازگار بپرهیزند، اما فلنر و همکاران (۲۰۱۱) استدلال می‌کنند بدترین اثرها به‌ندرت در عمل رخ می‌دهند و نباید از اکتشافی‌های ناسازگار هراسید.



شکل ۳.۲۰ – نقشهٔ رومانی با خطوط تراز در f = 400، f = 380 و f = 420، با آراد به‌عنوان حالت آغازین. گره‌های درون هر خط تراز هزینه‌های f = g + h کمتر یا مساوی مقدار تراز دارند.

با اکتشافی نامقبول، A* ممکن است بهینهٔ هزینه باشد یا نباشد. دو مورد هست که هست: نخست، اگر دست‌کم یک مسیر بهینهٔ هزینه وجود داشته باشد که h(n) برای همهٔ گره‌های n روی مسیر مقبول باشد، آن مسیر یافته خواهد شد، صرف‌نظر از آنچه اکتشافی برای حالت‌های بیرون مسیر می‌گوید. دوم، اگر راه‌حل بهینه هزینهٔ C* و راه‌حل دوم‌بهترین هزینهٔ C2 داشته باشد، و h(n) برخی هزینه‌ها را بیش‌تخمین بزند اما هرگز بیشتر از C2 - C*، آنگاه A* تضمین‌شده راه‌حل‌های بهینهٔ هزینه بازمی‌گرداند.

۳.۵.۳ خطوط تراز جستجو

راه مفیدی برای تجسم جستجو، رسم خطوط تراز (Contour) در فضای حالت است، درست مانند خطوط تراز در نقشهٔ توپوگرافیک. شکل ۳.۲۰ نمونه‌ای را نشان می‌دهد. درون ترازِ برچسب‌خورده با ۴۰۰، همهٔ گره‌ها f(n) = g(n) + h(n) ≤ 400 دارند، و الی آخر. سپس، چون A* گرهٔ جبهه را با کمترین هزینهٔ f گسترش می‌دهد، می‌بینیم جستجوی A* از گرهٔ آغاز به بیرون بادبزن می‌شود و گره‌ها را در نوارهای هم‌مرکز با هزینهٔ f فزاینده می‌افزاید.

با جستجوی هزینه-یکنواخت نیز خطوط تراز داریم، اما از هزینهٔ g نه g + h. خطوط تراز با جستجوی هزینه-یکنواخت «دایره‌ای» دور حالت آغاز خواهند بود و به‌طور یکسان در همهٔ جهات پخش می‌شوند، بدون هیچ ترجیحی به سوی هدف.

اما بدیهی نیست که هزینهٔ f = g + h به‌صورت یکنواخت افزایش یابد. با بسط مسیر از n به n'، هزینه از g(n) + h(n) به g(n) + c(n,a,n') + h(n') می‌رود. با حذف جملهٔ g(n)، می‌بینیم هزینهٔ مسیر یکنواخت‌افزاینده خواهد بود اگر و فقط اگر h(n) ≤ c(n,a,n') + h(n')، یعنی اگر و فقط اگر اکتشافی سازگار باشد.[13] اما توجه کنید مسیر ممکن است چند گرهِ پیاپی با امتیاز g(n) + h(n) یکسان داشته باشد؛ این هرگاه رخ می‌دهد که کاهش h دقیقاً برابر با هزینهٔ کنشِ تازه‌گرفته‌شده باشد (برای نمونه، در مسئلهٔ شبکه‌ای، وقتی n در همان سطر هدف است و یک گام به سوی هدف برمی‌دارید، g یک واحد زیاد و h یک واحد کم می‌شود).

اگر C* هزینهٔ مسیر راه‌حل بهینه باشد، می‌توانیم چنین بگوییم:

- A* همهٔ گره‌هایی را گسترش می‌دهد که از حالت آغازین در مسیری قابل دسترسی‌اند که همهٔ گره‌هایش f(n) < C* دارند. می‌گوییم این‌ها «گره‌های قطعاً گسترش‌یافته» (Surely Expanded Nodes) هستند.
- A* سپس ممکن است برخی از گره‌های دقیقاً روی «تراز هدف» (جایی که f(n) = C*) را پیش از برگزیدن گرهٔ هدف گسترش دهد.
- A* هیچ گره‌ای با f(n) > C* گسترش نمی‌دهد.

می‌گوییم A* با اکتشافی سازگار «بهینه‌کارا» (Optimally Efficient) است، به این معنا که هر الگوریتمی که مسیرهای جستجو را از حالت آغازین بسط دهد و از همان اطلاعات اکتشافی استفاده کند، باید همهٔ گره‌هایی را گسترش دهد که A* قطعاً گسترش می‌دهد (زیرا هر یک از آن‌ها می‌توانست بخشی از راه‌حل بهینه باشد). در میان گره‌های با f(n) = C*، ممکن است الگوریتمی شانس بیاورد و اول گرهٔ بهینه را برگزیند در حالی که الگوریتم دیگر بدشانس باشد؛ در تعریف بهینه‌کارایی این تفاوت را در نظر نمی‌گیریم.

A* کاراست، زیرا گره‌های درخت جستجو را که برای یافتن راه‌حل بهینه لازم نیستند «هرس» (Pruning) می‌کند. در شکل ۳.۱۸(ب) می‌بینیم تیمیشوآرا f = 447 و زرند f = 449 دارد. حتی با آنکه آن‌ها فرزندان ریشه‌اند و در جستجوی هزینه-یکنواخت یا سطح-اول از نخستین گره‌های گسترش‌یافته می‌بودند، هرگز توسط جستجوی A* گسترش نمی‌یابند، زیرا راه‌حل با f = 418 نخست یافته می‌شود. مفهوم هرس — حذف امکان‌ها از ملاحظه بدون نیاز به بررسی آن‌ها — برای بسیاری از حوزه‌های هوش مصنوعی مهم است.

اینکه جستجوی A* کامل، بهینهٔ هزینه و در میان همهٔ این‌گونه الگوریتم‌ها بهینه‌کاراست نسبتاً رضایت‌بخش است. متأسفانه، به این معنا نیست که A* پاسخ همهٔ نیازهای جستجوی ماست. نکتهٔ ظریف آن است که برای بسیاری از مسائل، شمار گره‌های گسترش‌یافته می‌تواند در طول راه‌حل نمایی باشد. برای نمونه، نسخه‌ای از جهان جاروبرقی را در نظر بگیرید با جاروبرقی فوق‌قدرتمند که می‌تواند هر خانه‌ای را به هزینهٔ ۱ واحد تمیز کند، حتی بدون نیاز به بازدید از آن

خانه؛ در آن سناریو، خانه‌ها را به هر ترتیبی می‌توان تمیز کرد. با N خانهٔ آغازینِ کثیف، 2^N حالت هست که زیرمجموعه‌ای تمیز شده‌اند؛ همهٔ آن حالت‌ها روی مسیر راه‌حل بهینه‌اند و بنابراین $f(n) < C^*$ را ارضا می‌کنند، پس همهٔ آن‌ها توسط A* بازدید می‌شوند.

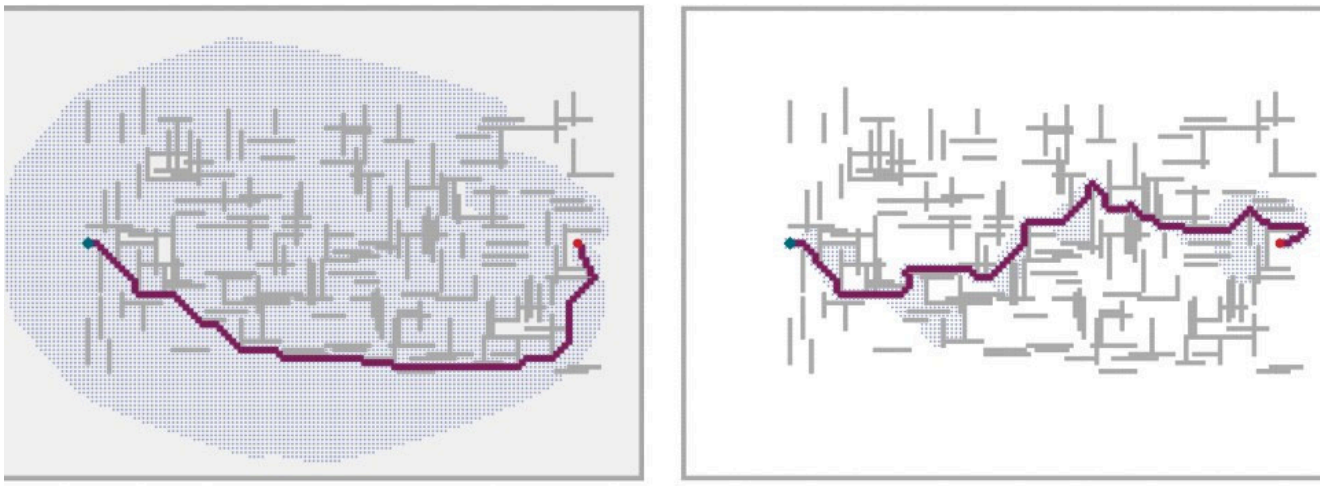
۳.۵.۴ جستجوی رضایت‌بخش: اکتشافی‌های نامقبول و A* وزنی

جستجوی A* کیفیت‌های خوب فراوانی دارد، اما گره‌های زیادی گسترش می‌دهد. اگر حاضر باشیم راه‌حل‌هایی بپذیریم که فروبهینه‌اند اما «به‌اندازهٔ کافی خوب» — که راه‌حل‌های رضایت‌بخش (Satisficing) می‌نامیم — می‌توانیم گره‌های کمتری (با زمان و فضای کمتر) کاوش کنیم. اگر به جستجوی A* اجازه دهیم از اکتشافی نامقبول (Inadmissible Heuristic) — که ممکن است بیش‌تخمین بزند — استفاده کند، خطر ازدست‌دادن راه‌حل بهینه را می‌پذیریم، اما اکتشافی می‌تواند بالقوه دقیق‌تر باشد و بدین ترتیب شمار گره‌های گسترش‌یافته را کاهش دهد. برای نمونه، مهندسان راه مفهوم «شاخص انحراف» (Detour Index) را می‌دانند؛ ضریبی که بر فاصلهٔ خط مستقیم اعمال می‌شود تا پیچ‌وخم معمول جاده‌ها را در نظر بگیرد. شاخص انحراف ۱٫۳ یعنی اگر دو شهر با فاصلهٔ خط مستقیم ۱۰ مایل از هم باشند، تخمین خوبی برای بهترین مسیر میان آن‌ها ۱۳ مایل است. برای بیشتر مناطق، شاخص انحراف بین ۱٫۲ و ۱٫۶ است.

می‌توانیم این ایده را بر هر مسئله‌ای اعمال کنیم، نه فقط مسائل مربوط به جاده، با رویکردی به نام جستجوی A* وزنی (Weighted A* Search) که در آن مقدار اکتشافی را سنگین‌تر وزن می‌کنیم و تابع ارزیابی را می‌گیریم:

$$f(n) = g(n) + W \times h(n) \quad (W > 1)$$

شکل ۳.۲۱ مسئلهٔ جستجو را در جهان شبکه‌ای نشان می‌دهد. در (الف)، جستجوی A* راه‌حل بهینه می‌یابد، اما برای یافتنش باید بخش بزرگی از فضای حالت را کاوش کند. در (ب)، جستجوی A* وزنی راه‌حلی اندکی پرهزینه‌تر می‌یابد، اما زمان جستجو بسیار کمتر است. می‌بینیم جستجوی وزنی تراز حالت‌های رسیده را به سوی هدف متمرکز می‌کند. این بدان معناست که حالت‌های کمتری کاوش می‌شوند، اما اگر مسیر بهینه هرگز بیرون از تراز جستجوی وزنی منحرف شود (چنان‌که در این مورد می‌شود)، مسیر بهینه یافته نخواهد شد. در حالت کلی، اگر راه‌حل بهینه C^* هزینه داشته باشد، جستجوی A* وزنی راه‌حلی با هزینه‌ای بین C^* و $W \times C^*$ می‌یابد؛ اما در عمل معمولاً نتایجی بسیار نزدیک‌تر به C^* از $W \times C^*$ می‌گیریم.



شکل ۳.۲۱ – دو جستجو روی همان شبکه: (الف) جستجوی A* و (ب) جستجوی A* وزنی با وزن W = 2. میله‌های خاکستری موانع‌اند، خط بنفش مسیر از آغاز سبز تا هدف قرمز است، و نقطه‌های کوچک حالت‌هایی هستند که هر جستجو به آن‌ها رسیده. در این مسئلهٔ خاص، A* وزنی ۷ برابر حالت کمتری کاوش می‌کند و مسیری ۵٪ پرهزینه‌تر می‌یابد.

جستجوهایی را در نظر گرفته‌ایم که حالت‌ها را با ترکیب g و h به شیوه‌های گوناگون ارزیابی می‌کنند؛ A* وزنی را می‌توان تعمیم بقیه دید:

- جستجوی A*: $g(n) + h(n)$ $(W = 1)$
- جستجوی هزینه-یکنواخت: $g(n)$ $(W = 0)$
- جستجوی بهترین-اول حریصانه: $h(n)$ $(W = \infty)$
- جستجوی A* وزنی: $g(n) + W \times h(n)$ $(1 < W < \infty)$

می‌توانید A* وزنی را «جستجوی تا حدی حریصانه» بنامید: مانند جستجوی بهترین-اول حریصانه، جستجو را به سوی هدف متمرکز می‌کند؛ از سوی دیگر، هزینهٔ مسیر را کاملاً نادیده نمی‌گیرد و مسیری را که با هزینهٔ زیاد پیشرفت اندک می‌کند معلق خواهد کرد.

الگوریتم‌های جستجوی فروبهینهٔ گوناگونی هست که با معیارهای «به‌اندازهٔ کافی خوب» مشخص می‌شوند. در جستجوی فروبهینهٔ کران‌دار (Bounded Suboptimal Search)، به دنبال راه‌حلی هستیم که تضمین‌شده در ضریب ثابت W از هزینهٔ بهینه باشد. A* وزنی این تضمین را می‌دهد. در جستجوی کران-هزینه (Bounded-Cost Search)، به دنبال راه‌حلی هستیم که هزینه‌اش از ثابت C کمتر باشد. و در جستجوی بی‌کران-هزینه (Unbounded-Cost Search)، راه‌حلی با هر هزینه‌ای می‌پذیریم، تا وقتی بتوانیم سریع بیابیمش. نمونهٔ الگوریتم جستجوی بی‌کران-هزینه، جستجوی سریع (Speedy Search) است؛ نسخه‌ای از جستجوی بهترین-اول حریصانه که به‌عنوان اکتشافی، شمار تخمینی کنش‌های لازم برای رسیدن به هدف را به کار می‌گیرد، صرف‌نظر از هزینهٔ آن کنش‌ها. پس برای مسائلی که همهٔ کنش‌ها هزینهٔ یکسان دارند، همان جستجوی بهترین-اول حریصانه است، اما وقتی کنش‌ها هزینه‌های متفاوت دارند، تمایل دارد جستجو را به‌سرعت به راه‌حل برساند، حتی اگر راه‌حل هزینهٔ بالایی داشته باشد.

۳.۵.۵ جستجوی محدود-به-حافظه

مسئلهٔ اصلی A* مصرف حافظه‌اش است. در این بخش چند ترفند پیاده‌سازی صرفه‌جویانه در فضا و سپس الگوریتم‌های کاملاً تازه‌ای را پوشش می‌دهیم که از فضای در دسترس بهتر بهره می‌گیرند. حافظه میان جبهه و حالت‌های رسیده تقسیم می‌شود. در پیاده‌سازی ما از جستجوی بهترین-اول، حالتی که روی جبهه است در دو جا ذخیره می‌شود: به‌صورت گره‌ای در جبهه (تا تصمیم بگیریم چه چیزی را بعدی گسترش دهیم) و به‌صورت مدخلی در جدول حالت‌های رسیده (تا بدانیم قبلاً از حالت بازدید کرده‌ایم). برای بسیاری از مسائل (مانند کاوش شبکه)، این تکرار نگران‌کننده نیست، زیرا اندازهٔ جبهه بسیار کوچک‌تر از reached است و تکرار حالت‌ها در جبهه حافظهٔ نسبتاً ناچیزی می‌طلبد. اما برخی پیاده‌سازی‌ها حالتی را فقط در یکی از دو جا نگه می‌دارند و کمی فضا صرفه‌جویی می‌کنند به بهای پیچیده‌کردن (و شاید کندکردن) الگوریتم.

امکان دیگر، حذف حالت‌ها از reached وقتی است که بتوانیم ثابت کنیم دیگر لازم نیستند. برای برخی مسائل، می‌توانیم از خاصیت جداسازی (شکل ۳.۶ در صفحهٔ ۷۲) به‌همراه ممنوعیت کنش‌های دورِ کامل (U-turn) استفاده کنیم تا مطمئن شویم همهٔ کنش‌ها یا از جبهه به بیرون حرکت می‌کنند یا به حالتِ دیگری روی جبهه. در آن صورت، فقط باید جبهه را برای مسیرهای زائد بررسی کنیم و می‌توانیم جدول reached را حذف کنیم.

برای مسائل دیگر، می‌توانیم شمار ارجاع (Reference Count) تعداد دفعات رسیده‌شدن یک حالت را نگه داریم و وقتی راه دیگری برای رسیدن به حالت نماند، آن را از جدول reached حذف کنیم. برای نمونه، در جهان شبکه‌ای که هر حالت فقط از چهار همسایه‌اش قابل رسیدن است، وقتی چهار بار به حالتی رسیده‌ایم، می‌توانیم آن را از جدول حذف کنیم.

حال الگوریتم‌های تازه‌ای را در نظر بگیریم که برای صرفه‌جویی در مصرف حافظه طراحی شده‌اند.

جستجوی پرتوی (Beam Search) اندازهٔ جبهه را محدود می‌کند. ساده‌ترین رویکرد نگه‌داشتن فقط k گره با بهترین امتیازهای f و دورریختن هر گرهٔ گسترش‌یافتهٔ دیگر است. این البته جستجو را ناقص و فروبهینه می‌کند، اما می‌توانیم k را چنان برگزینیم که از حافظهٔ در دسترس استفادهٔ خوبی کند، و الگوریتم سریع اجرا می‌شود، زیرا گره‌های کمتری گسترش می‌دهد. برای بسیاری از مسائل می‌تواند راه‌حل‌های خوبِ نزدیک‌به‌بهینه بیابد. می‌توانید جستجوی هزینه-یکنواخت یا A* را به‌صورت پخش‌شدن در همه‌جا در خطوط تراز هم‌مرکز بپندارید و جستجوی پرتوی را به‌صورت کاوش فقط بخشی متمرکز از آن خطوط تراز، بخشی که k نامزدِ برتر را در بر دارد.

نسخهٔ جایگزین جستجوی پرتوی محدودیت سختی بر اندازهٔ جبهه نمی‌گذارد، بلکه هر گره‌ای را نگه می‌دارد که امتیاز f آن در فاصلهٔ δ از بهترین امتیاز f باشد. بدین ترتیب، وقتی چند گرهٔ قوی‌امتیاز هست، فقط چند گره نگه داشته می‌شود، اما اگر گره قوی‌ای نباشد، گره‌های بیشتری نگه داشته می‌شوند تا گرهٔ قوی پدیدار شود.

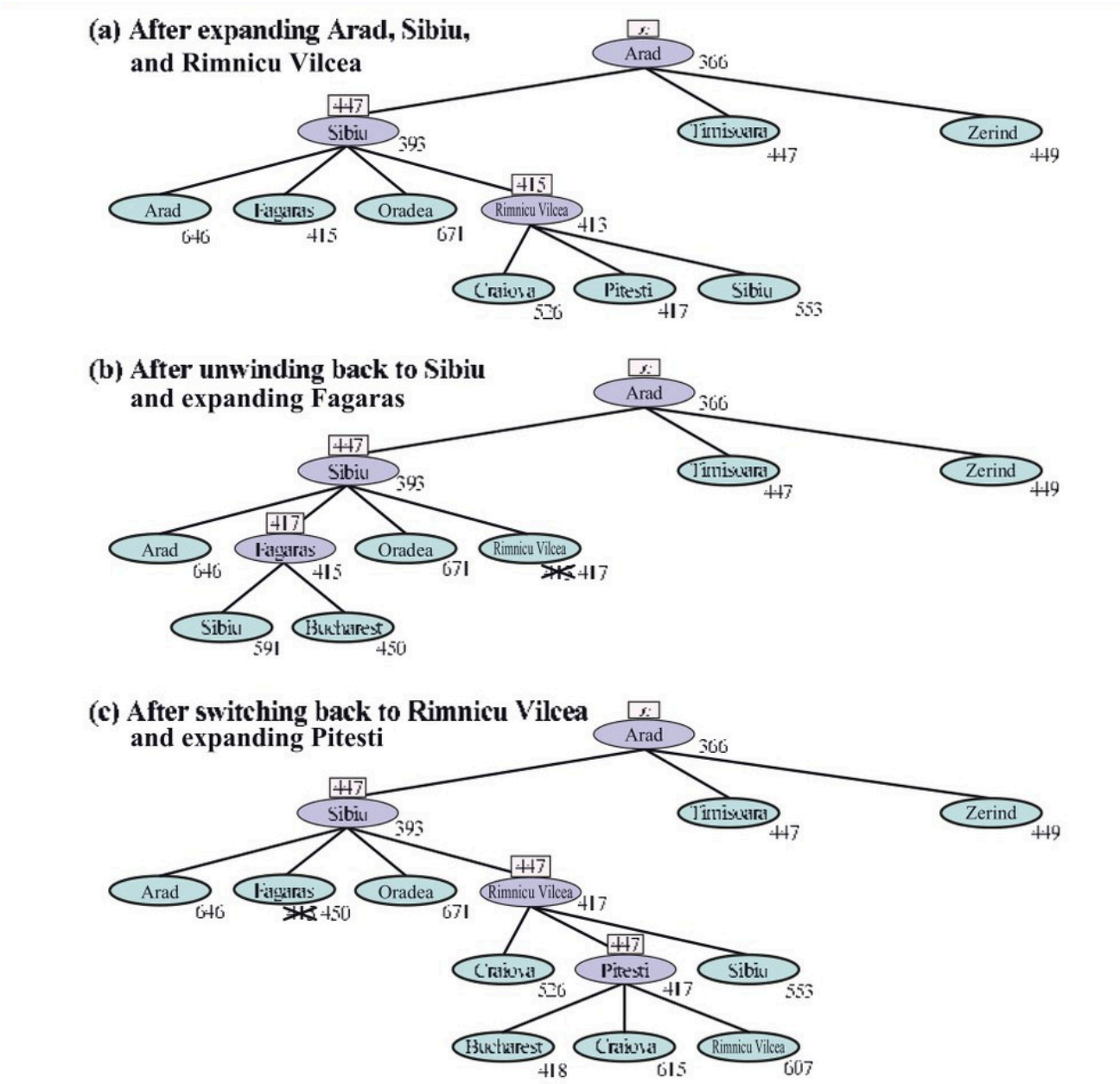
جستجوی A* تعمیق تکراری (Iterative-Deepening A*، یا IDA*) برای A* همان است که جستجوی تعمیق تکراری برای عمق-اول: IDA* مزیت‌های A* را بدون نیاز به نگه‌داشتن همهٔ حالت‌های رسیده در حافظه می‌دهد، به بهای بازدید برخی حالت‌ها چند بار. این الگوریتم بسیار مهم و پرکاربردی است برای مسائلی که در حافظه جا نمی‌شوند.

در تعمیق تکراری استاندارد، cutoff عمق است که در هر تکرار یک واحد زیاد می‌شود. در IDA* cutoff هزینهٔ f (یعنی g + h) است؛ در هر تکرار، مقدار cutoff کوچک‌ترین هزینهٔ f هر گره‌ای است که در تکرار قبلی از cutoff فراتر رفته. به بیان دیگر، هر تکرار به‌صورت جامع یک تراز f را جستجو می‌کند، گره‌ای را درست آن‌سوی آن تراز می‌یابد و هزینهٔ f آن گره را به‌عنوان تراز بعدی به کار می‌گیرد. برای مسائلی مانند معمای ۸ که هزینهٔ f هر مسیر عدد صحیح است، این بسیار خوب کار می‌کند و در هر تکرار پیشرفت پایداری به سوی هدف به بار می‌آورد. اگر راه‌حل بهینه هزینهٔ C* داشته باشد، نمی‌توان بیش از C* تکرار باشد (برای نمونه، در سخت‌ترین مسائل معمای ۸ حداکثر ۳۱ تکرار). اما برای مسئله‌ای که هر گره هزینهٔ f متفاوتی دارد، هر تراز تازه ممکن است فقط یک گرهٔ تازه در بر داشته باشد و شمار تکرارها می‌تواند با شمار حالت‌ها برابر شود.

جستجوی بهترین-اول بازگشتی (Recursive Best-First Search، یا RBFS) (شکل ۳.۲۲) می‌کوشد عملکرد جستجوی استاندارد بهترین-اول را تقلید کند، اما فقط با فضای خطی. RBFS شبیه جستجوی بازگشتی عمق-اول است، اما به‌جای ادامهٔ بی‌نهایت در مسیر کنونی، از متغیر f limit برای پیگیری مقدار f بهترین مسیر جایگزینِ در دسترس از هر نیای گرهٔ کنونی استفاده می‌کند. اگر گرهٔ کنونی از این حد فراتر رود، بازگشت به مسیر جایگزین بازمی‌شود. هنگام بازشدن بازگشت، RBFS مقدار f هر گره را در طول مسیر با مقدار پشتیبان‌گرفته‌شده (Backed-Up Value) — بهترین مقدار f فرزندانش — جایگزین می‌کند. بدین ترتیب RBFS مقدار f بهترین برگ در زیردرخت فراموش‌شده را به خاطر می‌سپارد و بنابراین می‌تواند تصمیم بگیرد آیا در زمان بعدی ارزش بازگسترش زیردرخت را دارد. شکل ۳.۲۳ نشان می‌دهد RBFS چگونه به بخارست می‌رسد.

```
function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution or failure
    solution, fvalue ← RBFS(problem, NODE(problem.INITIAL), ∞)
    return solution

function RBFS(problem, node, f_limit) returns a solution or failure, and a new f-cost limit
  if problem.IS-GOAL(node.STATE) then return node
  successors ← LIST(EXPAND(node))
  if successors is empty then return failure, ∞
  for each s in successors do    // update f with value from previous search, if any
    s.f ← max(s.PATH-COST + h(s), node.f)
  while true do
    best ← the lowest f-value node in successors
    if best.f > f_limit then return failure, best.f
    alternative ← the second-lowest f-value among successors
    result, best.f ← RBFS(problem, best, min(f_limit, alternative))
    if result ≠ failure then return result
```

شکل ۳.۲۲ — الگوریتم جستجوی بهترین-اول بازگشتی.



شکل ۳.۲۳ – مراحل جستجوی RBFS برای کوتاه‌ترین مسیر به بخارست. مقدار f limit برای هر فراخوانی بازگشتی بالای هر گرهٔ کنونی نشان داده شده و هر گره با هزینهٔ f خود برچسب‌گذاری شده. (الف) مسیر از طریق ریمنیکو ویلچا دنبال می‌شود تا آنکه بهترین برگ کنونی (پیتشتی) مقداری بدتر از بهترین مسیر جایگزین (فاگاراش) داشته باشد. (ب) بازگشت باز می‌شود و بهترین مقدار برگ زیردرخت فراموش‌شده (۴۱۷) به ریمنیکو ویلچا پشتیبان‌گرفته می‌شود؛ سپس فاگاراش گسترش می‌یابد و بهترین مقدار برگ ۴۵۰ را آشکار می‌کند. (ج) بازگشت باز می‌شود و بهترین مقدار برگ زیردرخت فراموش‌شده (۴۵۰) به فاگاراش پشتیبان‌گرفته می‌شود؛ سپس ریمنیکو ویلچا گسترش می‌یابد. این بار، چون بهترین مسیر جایگزین (از طریق تیمیشوآرا) دست‌کم ۴۴۷ هزینه دارد، گسترش تا بخارست ادامه می‌یابد.

RBFS تا حدی کارآمدتر از IDA* است، اما همچنان از بازتولید بیش از حد گره رنج می‌برد. در مثال شکل ۳.۲۳، RBFS مسیر از طریق ریمنیکو ویلچا را دنبال می‌کند، سپس «نظرش عوض می‌شود» و فاگاراش را می‌آزماید، و سپس دوباره نظر عوض می‌کند. این عوض‌شدن‌های نظر رخ می‌دهند، زیرا هر بار که بهترین مسیر کنونی بسط می‌یابد، مقدار f آن به احتمال زیاد افزایش می‌یابد – h معمولاً برای گره‌های نزدیک‌تر به هدف کمتر خوش‌بینانه است. وقتی این رخ می‌دهد،

دومین‌بهترین مسیر ممکن است بهترین مسیر شود، پس جستجو باید برای دنبال‌کردنش به عقب برگردد. هر عوض‌شدن نظر با تکرار IDA* متناظر است و ممکن است بازگسترش‌های فراوانی از گره‌های فراموش‌شده لازم باشد تا بهترین مسیر بازآفرینی و یک گره دیگر بسط شود.

RBFS اگر تابع اکتشافی h(n) مقبول باشد بهینه است. پیچیدگی فضایی‌اش در عمق عمیق‌ترین راه‌حل بهینه خطی است، اما پیچیدگی زمانی‌اش به‌سختی قابل توصیف است: هم به دقت تابع اکتشافی و هم به اینکه بهترین مسیر هر چند وقت یک بار با گسترش گره‌ها تغییر می‌کند بستگی دارد. گره‌ها را به‌ترتیب امتیاز f فزاینده گسترش می‌دهد، حتی اگر f غیریکنواخت باشد.

IDA* و RBFS از مصرف کمِ حافظه رنج می‌برند. میان تکرارها، IDA* فقط یک عدد را نگه می‌دارد: حد هزینهٔ f کنونی. RBFS اطلاعات بیشتری در حافظه نگه می‌دارد، اما فقط از فضای خطی استفاده می‌کند: حتی اگر حافظهٔ بیشتری در دسترس باشد، RBFS راهی برای استفاده از آن ندارد. چون این دو بیشترِ آنچه را انجام داده‌اند فراموش می‌کنند، ممکن است هر دو سرانجام همان حالت‌ها را بارها بازکاوش کنند.

پس معقول می‌نماید تعیین کنیم چقدر حافظه در دسترس داریم و به الگوریتم اجازه دهیم از همهٔ آن استفاده کند. دو الگوریتم که این کار را می‌کنند MA* (A* محدود-به-حافظه) و SMA* (MA* ساده‌شده) هستند. SMA* — خوب — ساده‌تر است، پس آن را توصیف می‌کنیم. SMA* دقیقاً مانند A* پیش می‌رود و بهترین برگ را گسترش می‌دهد تا حافظه پر شود. در این نقطه، نمی‌تواند گرهٔ تازه‌ای به درخت جستجو بیفزاید مگر آنکه گرهٔ قدیمی‌ای را بیندازد. SMA* همیشه بدترین گرهٔ برگ — آن با بیشترین مقدار f — را می‌اندازد. مانند RBFS، SMA* سپس مقدار گرهٔ فراموش‌شده را به والدش پشتیبان می‌گیرد. بدین ترتیب، نیای زیردرخت فراموش‌شده کیفیت بهترین مسیر آن زیردرخت را می‌داند. با این اطلاعات، SMA* زیردرخت را فقط وقتی بازتولید می‌کند که همهٔ مسیرهای دیگر بدتر از مسیر فراموش‌شده نشان داده شوند. به بیان دیگر، اگر همهٔ نوادگان گرهٔ n فراموش شوند، نخواهیم دانست از n به کدام سو برویم، اما هنوز ایده‌ای خواهیم داشت که رفتن از n به هر سو چقدر ارزشمند است. الگوریتم کامل در مخزن کد آنلاینِ همراه این کتاب توصیف شده است.

یک ظرافت هست که ارزش یادکرد دارد. گفتیم SMA* بهترین برگ را گسترش و بدترین برگ را حذف می‌کند. اگر همهٔ گره‌های برگ مقدار f یکسانی داشته باشند چه؟ برای پرهیز از انتخاب همان گره برای حذف و گسترش، SMA* تازه‌ترین بهترین برگ را گسترش و قدیمی‌ترین بدترین برگ را حذف می‌کند. این‌ها وقتی بر هم منطبق می‌شوند که فقط یک برگ هست، اما در آن صورت درخت جستجوی کنونی باید مسیری واحد از ریشه تا برگ باشد که همهٔ حافظه را پر کرده. اگر برگ گرهٔ هدف نباشد، حتی اگر روی مسیر راه‌حل بهینه باشد، آن راه‌حل با حافظهٔ در دسترس دست‌یافتنی نیست. پس گره را می‌توان دقیقاً چنان دور انداخت که گویی جانشینی ندارد.

SMA* کامل است اگر راه‌حلِ قابل دسترسی‌ای باشد — یعنی اگر d، عمق کم‌عمق‌ترین گرهٔ هدف، کمتر از اندازهٔ حافظه (بر حسب گره) باشد. بهینه است اگر هر راه‌حل بهینه‌ای قابل دسترسی باشد؛ وگرنه بهترین راه‌حل قابل دسترسی را بازمی‌گرداند. در عمل، SMA* انتخاب نسبتاً استواری برای یافتن راه‌حل‌های بهینه است، به‌ویژه وقتی فضای حالت گراف باشد، هزینه‌های کنش یکنواخت نباشند و تولید گره در مقایسه با سربار نگه‌داشتن جبهه و مجموعهٔ رسیده پرهزینه باشد.

اما در مسائل بسیار سخت، اغلب چنین خواهد بود که SMA* وادار می‌شود پیوسته میان بسیاری از مسیرهای راه‌حل نامزد رفت‌وبرگشت کند که فقط زیرمجموعهٔ کوچکی از آن‌ها در حافظه جا می‌شود. (این شبیه مسئلهٔ «تلمبه‌زدن» (Thrashing) در سامانه‌های صفحه‌بندی دیسک است.) آنگاه زمان اضافیِ لازم برای بازتولید مکرر همان گره‌ها بدان معناست که مسائلی که با A* با حافظهٔ نامحدود عملاً حل‌شدنی می‌بودند، برای SMA* ناشدنی می‌شوند. یعنی محدودیت‌های حافظه می‌توانند مسئله‌ای را ⚠ از دید زمان محاسبه ناشدنی کنند. هرچند هیچ نظریهٔ کنونی دادوستد زمان و حافظه را توضیح نمی‌دهد، به نظر می‌رسد این مسئله‌ای گریزناپذیر است. تنها راه خروج، کنارگذاشتن الزام بهینگی است.

۳.۵.۶ جستجوی اکتشافی دوطرفه

با جستجوی بهترین-اول یک‌طرفه، دیدیم به‌کارگیری f(n) = g(n) + h(n) به‌عنوان تابع ارزیابی، جستجوی A* می‌دهد که تضمین‌شده راه‌حل‌های بهینهٔ هزینه می‌یابد (با فرض h مقبول) و در شمار گره‌های گسترش‌یافته بهینه‌کاراست.

با جستجوی بهترین-اول دوطرفه نیز می‌توانیم به‌کارگیری f(n) = g(n) + h(n) را امتحان کنیم، اما متأسفانه هیچ تضمینی نیست که به راه‌حل بهینهٔ هزینه بینجامد، نه اینکه بهینه‌کارا باشد، حتی با اکتشافی مقبول. با جستجوی دوطرفه معلوم می‌شود نه گره‌های منفرد، بلکه جفت‌گره‌ها (یکی از هر جبهه) هستند که می‌توان اثبات کرد قطعاً گسترش می‌یابند، پس هر برهان کارایی باید جفت‌گره‌ها را در نظر بگیرد (اکرله و همکاران، ۲۰۱۷). با چند نماد تازه آغاز می‌کنیم. از fF(n) = gF(n) + hF(n) برای گره‌های در حال حرکت در جهت جلو (با حالت آغازین به‌عنوان ریشه) و fB(n) = gB(n) + hB(n) برای گره‌های جهت عقب (با حالت هدف به‌عنوان ریشه) استفاده می‌کنیم. هرچند هر دو جستجوی جلو و عقب همان مسئله را حل می‌کنند، توابع ارزیابی متفاوتی دارند، زیرا برای نمونه اکتشافی‌ها بسته به اینکه به سوی هدف یا حالت آغازین می‌کوشید متفاوت‌اند. اکتشافی‌های مقبول را فرض می‌کنیم.

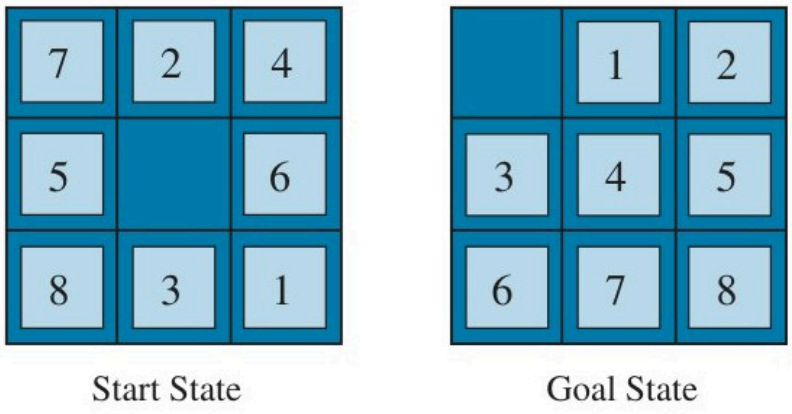
مسیر جلویی از حالت آغازین تا گرهٔ m و مسیر عقبی از هدف تا گرهٔ n را در نظر بگیرید. می‌توانیم کران پایینی بر هزینهٔ راه‌حلی تعریف کنیم که از مسیرِ حالت آغازین تا m پیروی می‌کند، سپس به‌نحوی به n می‌رسد، سپس مسیر تا هدف را دنبال می‌کند:

```
lb(m, n) = max(gF(m) + hF(m), gB(n) + hB(n), (gF(m) + gB(n) + dist(m, n)) / 2)
```

که در آن dist(m,n) هزینهٔ کمترین مسیر میان m و n است (که می‌توان با اکتشافی دیگری تخمین زد). اگر جستجوی دوطرفه گره‌ها را به‌ترتیب صعودیِ این کران پایین گسترش دهد، می‌توان نشان داد راه‌حلی که نخست به آن می‌رسد بهینه است. اما چنین راهبردی می‌تواند در عمل کند باشد، و مشخص شده راهبردهای جایگزین اغلب سریع‌ترند.

۳.۶ توابع اکتشافی

در این بخش می‌نگریم دقت اکتشافی چگونه بر عملکرد جستجو اثر می‌گذارد و نیز اینکه اکتشافی‌ها را چگونه می‌توان ابداع کرد. به‌عنوان مثال اصلی‌مان به معمای ۸ بازمی‌گردیم. چنان‌که در بخش ۳.۲ یاد شد، هدف معما لغزاندن کاشی‌ها به‌صورت افقی یا عمودی به فضای خالی است تا پیکربندی با پیکربندی هدف همخوان شود (شکل ۳.۲۵).



شکل ۳.۲۵ – نمونه‌ای معمولی از معمای ۸: حالت آغازین و حالت هدف. (همچنین در شکل ۳.۳)

اکتشافی استانداردی که برای معمای ۸ به کار می‌رود، «کاشی‌های نابجا» (Misplaced Tiles) است: اکتشافی h1 تعداد کاشی‌هایی است که در جای اشتباه‌اند. چون هر کاشی نابجا باید دست‌کم یک بار جابه‌جا شود، h1 مقبول است. معمولاً h1 خیلی هم خوب نیست، زیرا برای حل معما، کاشی‌ها باید بیش از یک بار جابه‌جا شوند. اکتشافی بهتر، مجموع «فاصلهٔ منهتن» (Manhattan Distance) کاشی‌ها از موقعیت‌های هدفشان است؛ فاصلهٔ منهتن مجموع فاصله‌های افقی و عمودی است، بی‌توجه به موانع (شکل ۳.۲۶). اکتشافی h2 مقبول است، زیرا هر حرکت فقط یک کاشی را یک خانه به فاصلهٔ منهتنِ هدفش نزدیک‌تر می‌کند؛ کاشی در هر حالت باید دست‌کم به اندازهٔ فاصلهٔ منهتن‌اش از هدف حرکت کند. میانگین دشواری معمای ۸ تصادفی حدود ۲۲ حرکت است و ضریب انشعاب حدود ۳ (هر حالت تقریباً سه حرکت قانونی دارد). این بدان معناست جستجوی سطح‌اولِ بدون اکتشافی تا حدود $۳^{۲۲} \approx ۳/۱ \times 10^{۱۰}$ حالت کاوش می‌کند؛ با A* ،h2 فقط ۱۷۰ گره گسترش می‌دهد (کورف، ۱۹۸۵) – کاهشی چشمگیر. با A* ،h1 حدود ۴,۰۰۰ گره گسترش می‌دهد. به‌طور کلی، اکتشافی خوب می‌تواند هزینهٔ جستجو را به‌شدت کاهش دهد.

	Search Cost (nodes generated)			Effective Branching Factor		
d	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

شکل ۳.۲۶ – فاصلهٔ منهتن. هر کاشی با عدد خودش برچسب‌گذاری شده و در موقعیت‌های هدفش قرار گرفته است. مجموع فاصله‌های منهتن کاشی‌ها از موقعیت‌هایشان در حالت آغازین برابر h2 است؛ برای این حالت، 15 = 1 + 3 + 0 + 1 + 3 + 3 + 1 + 0 = 4 = h2.

۳.۶.۱ اثر دقت اکتشافی بر عملکرد

یکی از راه‌های توصیف کیفیت اکتشافی، «ضریب انشعاب مؤثر» (Effective Branching Factor) b* است. اگر شمار کل گره‌های تولیدشده توسط A* برای مسئله‌ای N و عمق راه‌حل d باشد، b* ضریب انشعابی است که درخت یکنواختِ عمق d برای دربرگرفتن N+1 گره باید داشته باشد. پس:

```
N + 1 = 1 + b* + (b*)^2 + ... + (b*)^d
```

برای نمونه، اگر A* راه‌حلی در عمق ۵ با ۵۲ گره بیابد، ضریب انشعاب مؤثر ۱٫۹۲ است. ضریب انشعاب مؤثر می‌تواند در نمونه‌های مسئله متفاوت باشد، اما معمولاً برای حوزه‌ای خاص (مانند معمای ۸) در همهٔ نمونه‌های غیربدیهی نسبتاً ثابت است. پس اندازه‌گیری‌های تجربی b* روی مجموعه‌ای کوچک از مسائل می‌تواند راهنمای خوبی برای سودمندی کلی اکتشافی باشد. اکتشافیِ خوب‌طراحی‌شده باید مقدار b* نزدیک به ۱ داشته باشد و به مسائل نسبتاً بزرگ اجازه دهد با هزینهٔ محاسباتی معقول حل شوند.

ممکن است بپرسید آیا h2 همیشه بهتر از h1 است. پاسخ «اساساً بله» است. از تعریف‌های دو اکتشافی به‌راحتی دیده می‌شود برای هر گرهٔ n، h2(n) ≥ h1(n). پس می‌گوییم h2 بر h1 «غلبه» (Domination) دارد. غلبه مستقیماً به کارایی ترجمه می‌شود: A* با h2 هرگز گره‌های بیشتری از A* با h1 گسترش نمی‌دهد (به‌جز در مورد شکستن تصادفیِ تساوی‌ها). استدلال ساده است. مشاهدهٔ صفحهٔ ۹۰ را به یاد آورید که هر گره با f(n) < C* قطعاً گسترش می‌یابد. این همان گفتنِ این است که هر گره با h(n) < C* - g(n) وقتی h سازگار است قطعاً گسترش می‌یابد. اما چون h2 برای

همهٔ گره‌ها دست‌کم به اندازهٔ h1 است، هر گره‌ای که با h2 توسط A* قطعاً گسترش می‌یابد با h1 نیز قطعاً گسترش می‌یابد و h1 ممکن است سبب گسترش گره‌های دیگر هم بشود. از این رو، به‌طور کلی بهتر است از تابع اکتشافی با مقادیر بالاتر استفاده کنیم، به شرط آنکه سازگار باشد و زمان محاسبهٔ اکتشافی بیش از حد طولانی نباشد.

۳.۶.۲ تولید اکتشافی‌ها از مسائل ساده‌شده

دیده‌ایم که h1 (کاشی‌های نابجا) و h2 (فاصلهٔ منهتن) هر دو اکتشافی‌های نسبتاً خوبی برای معمای ۸ هستند و h2 بهتر است. چگونه ممکن است کسی به h2 رسیده باشد؟ آیا ممکن است رایانه چنین اکتشافی را به‌صورت مکانیکی ابداع کند؟

h1 و h2 تخمین‌هایی از طول مسیر باقی‌مانده برای معمای ۸ هستند، اما آن‌ها همچنین طول‌های مسیر کاملاً دقیقی برای نسخه‌های ساده‌شدهٔ معما هستند. اگر قواعد معما چنان تغییر می‌کرد که کاشی می‌توانست به هر جایی حرکت کند به‌جای فقط خانهٔ خالیِ مجاور، آنگاه h1 طول دقیق کوتاه‌ترین راه‌حل را می‌داد. به همین ترتیب، اگر کاشی می‌توانست در هر جهتی یک خانه حرکت کند، حتی به خانهٔ اشغال‌شده، h2 طول دقیق کوتاه‌ترین راه‌حل را می‌داد. مسئله‌ای با قیدهای کمتر بر کنش‌ها «مسئلهٔ ساده‌شده» (Relaxed Problem) نامیده می‌شود. گراف فضای حالت مسئلهٔ ساده‌شده، «ابرگراف» (Supergraph) فضای حالت اصلی است، زیرا حذف قیدها یال‌های افزوده‌ای در گراف ایجاد می‌کند.

چون مسئلهٔ ساده‌شده به گراف فضای حالت یال می‌افزاید، هر راه‌حل بهینه در مسئلهٔ اصلی، طبق تعریف، راه‌حلی در مسئلهٔ ساده‌شده نیز هست؛ اما مسئلهٔ ساده‌شده ممکن است راه‌حل‌های بهتری داشته باشد اگر یال‌های افزوده میان‌بُر فراهم کنند. از این رو، هزینهٔ $\blacktriangle$ راه‌حل بهینهٔ یک مسئلهٔ ساده‌شده، اکتشافی مقبولی برای مسئلهٔ اصلی است. وانگهی، چون اکتشافیِ مشتق‌شده هزینهٔ دقیقی برای مسئلهٔ ساده‌شده است، باید نابرابری مثلث را رعایت کند و بنابراین سازگار است (صفحهٔ ۸۸ را ببینید).

اگر تعریف مسئله به زبان صوری نوشته شود، ساخت خودکار مسئله‌های ساده‌شده ممکن است.[14] برای نمونه، اگر کنش‌های معمای ۸ چنین توصیف شوند: «کاشی می‌تواند از خانهٔ X به خانهٔ Y حرکت کند اگر X مجاور Y باشد و Y خالی باشد»، می‌توانیم سه مسئلهٔ ساده‌شده با حذف یکی یا هر دو شرط تولید کنیم:

(الف) کاشی می‌تواند از خانهٔ X به خانهٔ Y حرکت کند اگر X مجاور Y باشد. (ب) کاشی می‌تواند از خانهٔ X به خانهٔ Y حرکت کند اگر Y خالی باشد. (ج) کاشی می‌تواند از خانهٔ X به خانهٔ Y حرکت کند.

از (الف)، می‌توانیم h2 (فاصلهٔ منهتن) را مشتق کنیم. استدلال این است که h2 امتیاز درست می‌بود اگر هر کاشی را به‌نوبت به مقصدش می‌بردیم. اکتشافیِ مشتق‌شده از (ب) در تمرین GASC.3 بحث شده. از (ج)، می‌توانیم h1 (کاشی‌های نابجا) را مشتق کنیم، زیرا امتیاز درست می‌بود اگر کاشی‌ها می‌توانستند در یک کنش به مقصد مورد نظرشان بروند. توجه کنید حیاتی است که مسئله‌های ساده‌شدهٔ تولیدشده با این فن را بتوان اساساً بدون جستجو حل کرد، زیرا قواعد ساده‌شده به مسئله اجازه می‌دهند به هشت زیرمسئلهٔ مستقل تجزیه شود. اگر مسئلهٔ ساده‌شده به‌سختی حل شود، به‌دست‌آوردن مقادیر اکتشافی متناظر پرهزینه خواهد بود.

برنامه‌ای به نام ABSOLVER می‌تواند با روش «مسئلهٔ ساده‌شده» و فنون گوناگون دیگر، اکتشافی‌ها را به‌صورت خودکار از تعریف‌های مسئله تولید کند (پریدیتیس، ۱۹۹۳). ABSOLVER اکتشافی تازه‌ای برای معمای ۸ تولید کرد که از هر اکتشافیِ پیشین بهتر بود و نخستین اکتشافیِ مفید را برای معمای نامدار مکعب روبیک یافت.

اگر مجموعه‌ای از اکتشافی‌های مقبول h1...hm برای مسئله‌ای در دسترس باشد و هیچ‌کدام به‌وضوح از بقیه بهتر نباشد، کدام را باید برگزینیم؟ معلوم می‌شود می‌توانیم بهترین همهٔ جهان‌ها را داشته باشیم، با تعریف h(n) = max{h1(n), ..., hk(n)}. این اکتشافیِ ترکیبی هر تابعی را برمی‌گزیند که برای گرهٔ موردنظر دقیق‌ترین است. چون مؤلفه‌های hi مقبول‌اند، h مقبول است (و اگر همهٔ hi سازگار باشند، h سازگار است). وانگهی، h بر همهٔ اکتشافی‌های مؤلفه‌اش غلبه دارد. تنها عیبش آن است که محاسبهٔ h(n) زمان بیشتری می‌برد. اگر این مسئله‌ساز باشد، جایگزین آن است که در هر ارزیابی یکی از اکتشافی‌ها را تصادفی برگزینیم، یا از الگوریتم یادگیری ماشین برای پیش‌بینی بهترین اکتشافی استفاده کنیم. این کار می‌تواند به اکتشافی ناسازگار بینجامد (حتی اگر هر hi سازگار باشد)، اما در عمل معمولاً به حل سریع‌تر مسئله می‌انجامد.

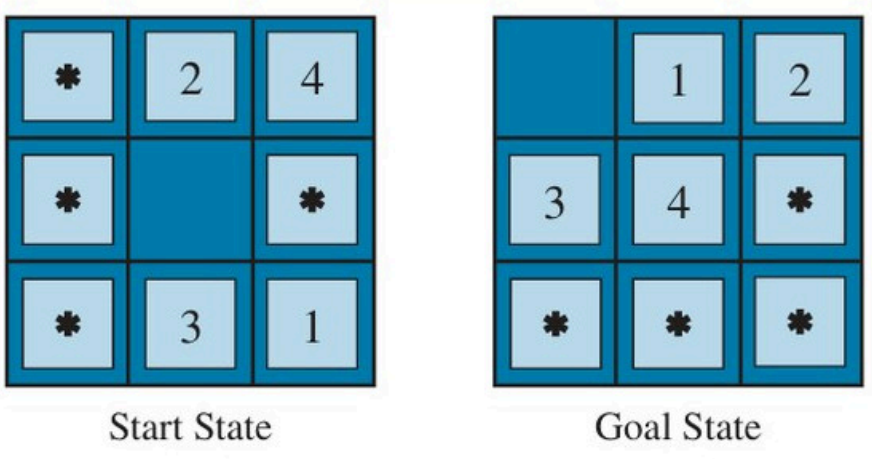
۳.۶.۳ تولید اکتشافی‌ها از زیرمسئله‌ها: پایگاه‌داده‌های الگو

اکتشافی‌های مقبول را می‌توان از هزینهٔ راه‌حلِ «زیرمسئله» (Subproblem) مسئلهٔ داده‌شده نیز مشتق کرد. برای نمونه، شکل ۳.۲۷ زیرمسئله‌ای از نمونهٔ معمای ۸ شکل ۳.۲۵ را نشان می‌دهد. زیرمسئله شامل رساندن کاشی‌های ۱، ۲، ۳، ۴ و فضای خالی به موقعیت‌های درستشان است. بدیهی است هزینهٔ راه‌حل بهینهٔ این زیرمسئله کران پایینی بر هزینهٔ مسئلهٔ کامل است. معلوم می‌شود در برخی موارد از فاصلهٔ منهتن دقیق‌تر است. ایدهٔ پشت «پایگاه‌داده‌های الگو» (Pattern Databases) ذخیرهٔ این هزینه‌های دقیق راه‌حل برای هر نمونهٔ ممکنِ زیرمسئله است — در مثال ما، هر پیکربندی ممکن از چهار کاشی و فضای خالی. (در پایگاه‌داده ۹ × ۸ × ۷ × ۶ × ۵ = ۱۵,۱۲۰ الگو خواهد بود. هویت چهار کاشی دیگر برای حل زیرمسئله بی‌ربط است، اما حرکت‌های آن کاشی‌ها در هزینهٔ راه‌حل زیرمسئله شمرده می‌شود.) سپس برای هر حالتِ روی‌مواجه‌شده در جستجو، اکتشافی مقبول hDB را صرفاً با جست‌وجوی پیکربندی زیرمسئلهٔ متناظر در پایگاه‌داده محاسبه می‌کنیم. خودِ پایگاه‌داده با جستجوی به عقب از هدف و ثبت هزینهٔ هر الگوی تازه‌ای که با آن مواجه می‌شویم ساخته می‌شود؛[15] هزینهٔ این جستجو بر نمونه‌های مسئلهٔ بعدی سرشکن می‌شود و بنابراین اگر انتظار داشته باشیم از ما خواسته شود مسائل بسیاری حل کنیم، منطقی است.

انتخاب کاشی‌های ۱-۲-۳-۴ برای همراهی با فضای خالی نسبتاً دلبخواهانه است؛ می‌توانیم پایگاه‌داده‌هایی برای ۵-۶-۷-۸، برای ۲-۴-۶-۸ و الی آخر نیز بسازیم. هر پایگاه‌داده اکتشافی مقبول می‌دهد و این اکتشافی‌ها را می‌توان، چنان‌که پیش‌تر توضیح داده شد، با گرفتن بیشینهٔ مقدار ترکیب کرد. اکتشافیِ ترکیبی از این نوع بسیار دقیق‌تر از فاصلهٔ منهتن است؛ شمار گره‌های تولیدشده هنگام حل معمای ۱۵ تصادفی را می‌توان با ضریب ۱۰۰۰ کاهش داد. اما با هر پایگاه‌دادهٔ اضافی بازدهی کاهشی و هزینه‌های حافظه و محاسبهٔ فزاینده هست.

ممکن است بپرسید آیا اکتشافی‌های به‌دست‌آمده از پایگاه‌دادهٔ ۱-۲-۳-۴ و ۵-۶-۷-۸ را می‌توان جمع کرد، چون دو زیرمسئله به نظر نمی‌رسد هم‌پوشانی داشته باشند. آیا این همچنان اکتشافی مقبول می‌دهد؟ پاسخ نه است، زیرا راه‌حل‌های زیرمسئلهٔ ۱-۲-۳-۴ و زیرمسئلهٔ ۵-۶-۷-۸ برای حالتِ داده‌شده تقریباً به‌طور قطع برخی حرکت‌ها را

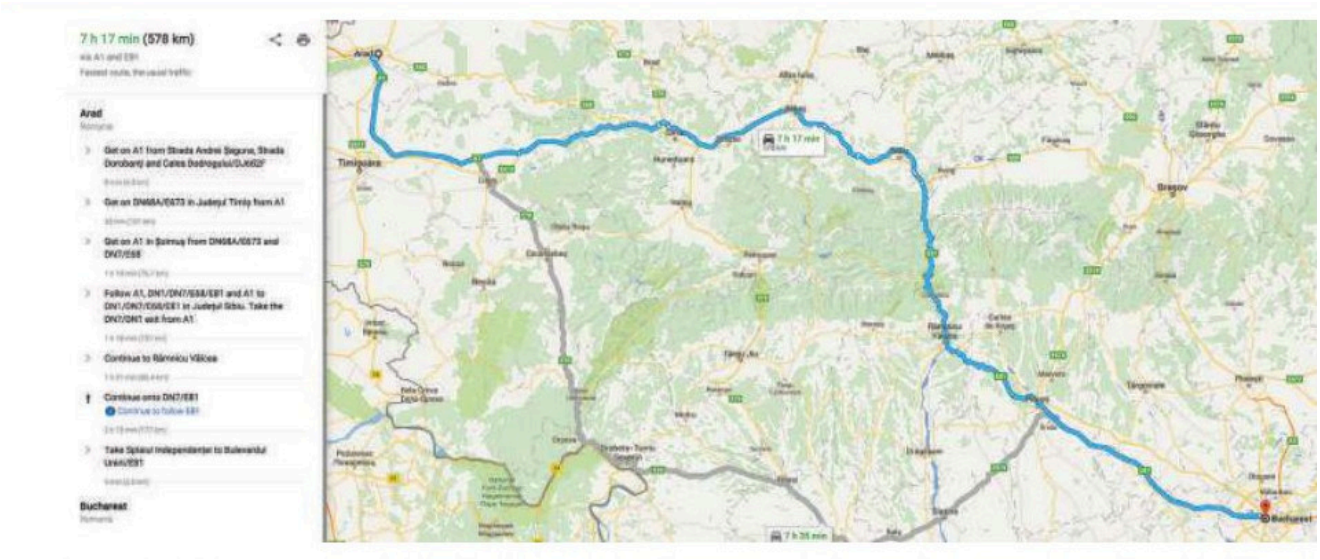
مشترک‌اند — بعید است ۱-۲-۳-۴ بدون دست‌زدن به ۵-۶-۷-۸ سر جای خود بنشیند و برعکس. اما اگر آن حرکت‌ها را نشماریم چه — اگر کاشی‌های دیگر را به ستاره انتزاع نکنیم، بلکه آن‌ها را محو کنیم چه؟ یعنی نه هزینهٔ کل حل زیرمسئلهٔ ۱-۲-۳-۴، بلکه فقط شمار حرکت‌های شامل ۱-۲-۳-۴ را ثبت کنیم. آنگاه مجموع دو هزینه همچنان کران پایینی بر هزینهٔ حل کل مسئله است. این ایده‌ی پشت «پایگاه‌داده‌های الگوی ناهم‌پوش» (Disjoint Pattern Databases) است. با چنین پایگاه‌داده‌هایی، حل معمای ۱۵ تصادفی در چند میلی‌ثانیه ممکن است — شمار گره‌های تولیدشده در مقایسه با فاصلهٔ منهتن با ضریب ۱۰,۰۰۰ کاهش می‌یابد. برای معمای ۲۴، شتاب تقریباً یک میلیون‌برابر به دست می‌آید. پایگاه‌داده‌های الگوی ناهم‌پوش برای معماهای کاشی‌لغزان کار می‌کنند، زیرا مسئله را می‌توان چنان تقسیم کرد که هر حرکت فقط یک زیرمسئله را تحت تأثیر قرار دهد — چون فقط یک کاشی در هر بار حرکت می‌کند.



شکل ۳.۲۷ — زیرمسئله‌ای از نمونهٔ معمای ۸ شکل ۳.۲۵. تکلیف رساندن کاشی‌های ۱، ۲، ۳، ۴ و فضای خالی به موقعیت‌های درستشان است، بدون نگرانی از آنچه بر کاشی‌های دیگر می‌گذرد.

۳.۶.۴ تولید اکتشافی‌ها با نقطه‌های نشانه

سرویس‌های آنلاینی هست که نقشه‌هایی با ده‌ها میلیون رأس میزبانی می‌کنند و مسیرهای رانندگی بهینهٔ هزینه را در چند میلی‌ثانیه می‌یابند (شکل ۳.۲۸). چگونه می‌توانند این کار را بکنند، در حالی که بهترین الگوریتم‌های جستجویی که تاکنون در نظر گرفته‌ایم حدود یک میلیون بار کندترند؟ حربه‌های بسیاری هست، اما مهم‌ترین آن «پیش‌محاسبه» (Precomputation) برخی هزینه‌های مسیر بهینه است. هرچند پیش‌محاسبه می‌تواند زمان‌بر باشد، فقط یک بار لازم است انجام شود و سپس می‌توان آن را بر میلیاردها درخواست جستجوی کاربران سرشکن کرد.



شکل ۳.۲۸ — سرویس وب ارائه‌دهندهٔ مسیرهای رانندگی، محاسبه‌شده توسط الگوریتم جستجو.

می‌توانیم اکتشافی کاملی تولید کنیم با پیش‌محاسبه و ذخیرهٔ هزینهٔ مسیر بهینه میان هر جفت رأس. آن $O(|V|^2)$ فضا و $O(|E|^3)$ زمان می‌برد — برای گراف‌های با ده هزار رأس عملی است، اما نه برای ده میلیون.

رویکرد بهتر آن است که چند (شاید ۱۰ یا ۲۰) «نقطهٔ نشانه» (Landmark Point)[16] از میان رأس‌ها برگزینیم. سپس برای هر نقطهٔ نشانهٔ L و هر رأس v دیگر در گراف، $C^*(v,L)$ — هزینهٔ دقیق مسیر بهینه از v تا L — را محاسبه و ذخیره می‌کنیم. (به $C^*(L,v)$ نیز نیاز داریم؛ روی گراف بدون جهت این همان $C^*(v,L)$ است؛ روی گراف جهت‌دار — مثلاً با خیابان‌های یک‌طرفه — باید جداگانه محاسبه‌اش کنیم.) با جدول‌های C^* ذخیره‌شده، به‌راحتی می‌توانیم اکتشافی کارآمدی (هرچند نامقبول) بسازیم: کمینه، روی همهٔ نقطه‌های نشانه، هزینهٔ رسیدن از گرهٔ کنونی به نقطهٔ نشانه و سپس به هدف:

```
h_L(n) = min_{L ∈ Landmarks} [C*(n, L) + C*(L, goal)]
```

اگر مسیر بهینه اتفاقاً از نقطهٔ نشانه‌ای بگذرد، این اکتشافی دقیق خواهد بود؛ اگر نه، نامقبول است — هزینه تا هدف را بیش‌تخمین می‌زند. در جستجوی A*، اگر اکتشافی‌های دقیق داشته باشید، پس از رسیدن به گره‌ای روی مسیر بهینه، هر گره‌ای که گسترش دهید از آن پس روی مسیر بهینه خواهد بود. به خطوط تراز بیندیشید که این مسیر بهینه را دنبال می‌کنند. جستجو مسیر بهینه را دنبال می‌کند و در هر تکرار کنشی با هزینهٔ c می‌افزاید تا به حالت نتیجه‌ای برسد که مقدار h آن c کمتر است، یعنی امتیاز کل f = g + h در تمام طول مسیر در C^* ثابت می‌ماند.

برخی الگوریتم‌های مسیریابی با افزودن «میان‌بُرها» (Shortcuts) — یال‌های مصنوعی در گراف که مسیر بهینهٔ چندکنشه را تعریف می‌کنند — زمان بیشتری صرفه‌جویی می‌کنند. برای نمونه، اگر میان‌بُرهایی از پیش میان همهٔ ۱۰۰ شهر بزرگ آمریکا تعریف شده بود و ما می‌خواستیم از پردیس برکلی در کالیفرنیا به دانشگاه نیویورک در نیویورک برویم، می‌توانستیم میان‌بُر میان ساکرامنتو و منهتن را برداریم و ۹۰٪ مسیر را در یک کنش بپیماییم.

hL(n) کارآمد است اما مقبول نیست. اما با کمی دقت بیشتر، می‌توانیم اکتشافی‌ای به دست آوریم که هم کارآمد است هم مقبول:

```
|h_DH(n) = max_{L ∈ Landmarks} |C*(n, L) - C*(goal, L)
```

این «اکتشافی تفاضلی» (Differential Heuristic) نامیده می‌شود (به‌سبب تفریق). این را با نقطهٔ نشانه‌ای تصور کنید که جایی آن‌سوی هدف است. اگر هدف اتفاقاً روی مسیر بهینه از n تا نقطهٔ نشانه باشد، این می‌گوید «کل مسیر از n تا L را در نظر بگیر، سپس بخش آخر آن مسیر، از هدف تا L را کم کن، و هزینهٔ دقیق مسیر از n تا هدف را به ما می‌دهد.» تا آنجا که هدف کمی بیرون از مسیر بهینه به نقطهٔ نشانه است، اکتشافی نادقیق خواهد بود، اما همچنان مقبول. نقطه‌های نشانه‌ای که آن‌سوی هدف نیستند مفید نخواهند بود؛ نقطهٔ نشانه‌ای که دقیقاً نیمه‌راه میان n و هدف است hDH = 0 می‌دهد که کمکی نمی‌کند.

چند راه برای برگزیدن نقطه‌های نشانه هست. برگزیدن تصادفی نقاط سریع است، اما اگر مراقبت کنیم نقطه‌های نشانه را پخش کنیم تا زیاد به هم نزدیک نباشند، نتایج بهتری می‌گیریم. رویکرد حریصانه آن است که نخستین نقطهٔ نشانه را تصادفی برگزینیم، سپس نقطه‌ای را بیابیم که از آن دورترین است و آن را به مجموعهٔ نقطه‌های نشانه بیفزاییم، و ادامه دهیم و در هر تکرار نقطه‌ای را بیفزاییم که فاصله‌اش تا نزدیک‌ترین نقطهٔ نشانه را بیشینه می‌کند. اگر گزارش‌های درخواست‌های جستجوی گذشتهٔ کاربرانتان را دارید، می‌توانید نقطه‌های نشانه‌ای را برگزینید که در جستجوها پرتقاضایند. برای اکتشافی تفاضلی خوب است نقطه‌های نشانه دور محیط گراف پخش شده باشند. پس فن خوب آن است که مرکز ثقل گراف را بیابیم، k گوهٔ کیک‌شکل دور مرکز ثقل بچینیم و در هر گوه رأسی را برگزینیم که از مرکز دورترین است. نقطه‌های نشانه به‌ویژه در مسائل مسیریابی خوب کار می‌کنند، به‌سبب شیوهٔ چیدمان جاده‌ها در جهان: بخش بزرگی از ترافیک واقعاً می‌خواهد میان نقطه‌های نشانه سفر کند، پس مهندسان عمران پهن‌ترین و سریع‌ترین جاده‌ها را در این مسیرها می‌سازند؛ جستجوی نشانه‌ای بازیابی این مسیرها را آسان‌تر می‌کند.

۳.۶.۵ یادگیری جستجوی بهتر

چند راهبرد جستجوی ثابت ارائه کرده‌ایم — سطح-اول، A* و مانند آن — که توسط دانشمندان رایانه با دقت طراحی و برنامه‌ریزی شده‌اند. آیا عامل می‌تواند یاد بگیرد چگونه بهتر جستجو کند؟ پاسخ بله است و روش بر مفهوم مهمی به نام «فضای حالت فراسطح» (Metalevel State Space) استوار است. هر حالت در فضای حالت فراسطح، حالت درونی (محاسباتی) برنامه‌ای را در بر می‌گیرد که در فضای حالت عادی مانند نقشهٔ رومانی جستجو می‌کند. (برای جدا نگه‌داشتن دو مفهوم، نقشهٔ رومانی را «فضای حالت سطح‌شیء» (Object-Level State Space) می‌نامیم.) برای نمونه، حالت درونی الگوریتم A* شامل درخت جستجوی کنونی است. هر کنش در فضای حالت فراسطح، گام محاسباتی است که حالت درونی را تغییر می‌دهد؛ برای نمونه، هر گام محاسباتی در A* یک گرهٔ برگ را گسترش می‌دهد و جانشین‌هایش را به درخت می‌افزاید. پس شکل ۳.۱۸، که توالی‌ای از درخت‌های جستجوی بزرگ‌وبزرگ‌تر را نشان می‌دهد، می‌تواند به‌مثابه ترسیم مسیری در فضای حالت فراسطح دیده شود که هر حالتش روی مسیر، درخت جستجوی سطح‌شیء است. حال، مسیر شکل ۳.۱۸ پنج گام دارد، از جمله گامی — گسترش فاگاراش — که چندان کمکی نیست. برای مسائل دشوارتر، چنین لغزش‌های بسیاری خواهد بود و الگوریتم یادگیری فراسطح (Metalevel

Learning) می‌تواند از این تجربه‌ها بیاموزد که از کاوش زیردرخت‌های بی‌امید بپرهیزد. فنون به‌کاررفته برای این نوع یادگیری در فصل ۲۲ توصیف شده‌اند. هدف یادگیری، کمینه‌کردن هزینهٔ کل حل مسئله است، با دادوستد میان هزینهٔ محاسباتی و هزینهٔ مسیر.

۳.۶.۶ یادگیری اکتشافی‌ها از تجربه

دیده‌ایم یکی از راه‌های ابداع اکتشافی، تدوین مسئلهٔ ساده‌شده‌ای است که راه‌حل بهینه‌اش به‌راحتی یافت می‌شود. جایگزین، یادگیری از تجربه است. «تجربه» در این‌جا یعنی حل بسیاری معمای ۸، برای نمونه. هر راه‌حل بهینهٔ یک مسئلهٔ معمای ۸، نمونهٔ (هدف، مسیر) فراهم می‌کند. از این نمونه‌ها، الگوریتم یادگیری می‌تواند تابع h بسازد که (با خوش‌شانسی) هزینهٔ مسیر واقعی حالت‌های دیگرِ روی‌آمده در جستجو را تقریب بزند. بیشتر این رویکردها تقریب ناکاملی از تابع اکتشافی یاد می‌گیرند و بنابراین ریسک نامقبول‌بودن دارند. این به دادوستد ناگزیری میان زمان یادگیری، زمان اجرای جستجو و هزینهٔ راه‌حل می‌انجامد. فنون یادگیری ماشین در فصل ۱۹ نشان داده می‌شوند. روش‌های یادگیری تقویتیِ توصیف‌شده در فصل ۲۲ نیز بر جستجو کاربردپذیرند.

برخی فنون یادگیری ماشین وقتی بهتر کار می‌کنند که به‌جای توصیف خام حالت، «ویژگی‌هایی» (Features) از حالت در اختیارشان گذاشته شود که برای پیش‌بینی مقدار اکتشافی حالت مرتبط‌اند. برای نمونه، ویژگی «شمار کاشی‌های نابجا» ممکن است در پیش‌بینی فاصلهٔ واقعی حالت معمای ۸ تا هدف کمکی باشد. این ویژگی را x1(n) بنامیم. می‌توانیم ۱۰۰ پیکربندی تصادفی معمای ۸ تولید کنیم و آمار هزینه‌های راه‌حل واقعی‌شان را گرد آوریم. ممکن است بیابیم وقتی x1(n) برابر ۵ است، میانگین هزینهٔ راه‌حل حدود ۱۴ است، و الی آخر. البته می‌توانیم از چند ویژگی استفاده کنیم. ویژگی دوم x2(n) ممکن است «شمار جفت‌کاشی‌های مجاوری باشد که در حالت هدف مجاور نیستند». x1(n) و x2(n) را چگونه باید برای پیش‌بینی h(n) ترکیب کرد؟ رویکرد رایج استفاده از ترکیب خطی است:

```
h(n) = c1 * x1(n) + c2 * x2(n)
```

ثابت‌های c1 و c2 برای بهترین برازش داده‌های واقعی در میان پیکربندی‌های تصادفی‌تولیدشده تنظیم می‌شوند. انتظار می‌رود هر دو c1 و c2 مثبت باشند، زیرا کاشی‌های نابجا و جفت‌های مجاورِ نادرست مسئله را دشوارتر می‌کنند. توجه کنید این اکتشافی شرط h(n)=0 برای حالت‌های هدف را برآورده می‌کند، اما لزوماً مقبول یا سازگار نیست.

خلاصهٔ فصل

این فصل الگوریتم‌های جستجویی را معرفی کرد که عامل می‌تواند برای انتخاب توالی‌های کنش در گسترهٔ وسیعی از محیط‌ها به کار گیرد — تا وقتی آن محیط‌ها تکرارشونده، تک‌عاملی، کاملاً قابل مشاهده، جبرگرایانه، ایستا، گسسته و کاملاً شناخته‌شده باشند. دادوستدهایی هست که باید میان زمانِ جستجو، حافظهٔ در دسترس و کیفیت راه‌حل انجام شود. می‌توانیم کارآمدتر باشیم اگر دانش وابسته به حوزه در قالب تابع اکتشافی داشته باشیم که تخمین می‌زند حالتِ داده‌شده چقدر تا هدف فاصله دارد، یا اگر راه‌حل‌های جزئی شامل الگوها یا نقطه‌های نشانه را از پیش محاسبه کنیم.

- پیش از آنکه عامل بتواند جستجو را آغاز کند، باید مسئله‌ای خوب‌تعریف‌شده صورت‌بندی شود.
- مسئله از پنج بخش تشکیل شده: حالت آغازین، مجموعه‌ای از کنش‌ها، مدل گذارِ توصیف‌کنندهٔ نتایج آن کنش‌ها، مجموعه‌ای از حالت‌های هدف، و تابع هزینهٔ کنش.
- محیط مسئله با گراف فضای حالت بازنمایی می‌شود. مسیری در فضای حالت (توالی‌ای از کنش‌ها) از حالت آغازین به حالت هدف، راه‌حل است.
- الگوریتم‌های جستجو عموماً حالت‌ها و کنش‌ها را اتمی می‌دانند، بدون هیچ ساختار درونی (هرچند هنگام یادگیری ویژگی‌های حالت را معرفی کردیم).
- الگوریتم‌های جستجو بر اساس کامل بودن، بهینگی هزینه، پیچیدگی زمانی و پیچیدگی فضایی داوری می‌شوند.
- روش‌های جستجوی نامطلع فقط به تعریف مسئله دسترسی دارند. الگوریتم‌ها درخت جستجویی می‌سازند تا راه‌حلی بیابند. الگوریتم‌ها در اینکه کدام گره را نخست گسترش می‌دهند متفاوت‌اند:
  - جستجوی بهترین-اول گره‌ها را برای گسترش با تابع ارزیابی برمی‌گزیند.
  - جستجوی سطح-اول نخست کم‌عمق‌ترین گره‌ها را گسترش می‌دهد؛ کامل است، برای هزینه‌های کنش واحد بهینه است، اما پیچیدگی فضایی نمایی دارد.
  - جستجوی هزینه-یکنواخت گره‌ای با کمترین هزینهٔ مسیر، $g(n)$، را گسترش می‌دهد و برای هزینه‌های کنش عمومی بهینه است.
  - جستجوی عمق-اول نخست عمیق‌ترین گرهٔ گسترش‌نیافته را گسترش می‌دهد. نه کامل است نه بهینه، اما پیچیدگی فضایی خطی دارد. جستجوی محدود-به-عمق کران عمقی می‌افزاید.
  - جستجوی تعمیق تکراری، جستجوی عمق-اول را با حدهای عمق فزاینده فراخوانی می‌کند تا هدف یافته شود. با بررسی کامل چرخه کامل است، برای هزینه‌های کنش واحد بهینه است، پیچیدگی زمانی قابل مقایسه با جستجوی سطح-اول و پیچیدگی فضایی خطی دارد.
  - جستجوی دوطرفه دو جستجو را هم‌زمان اجرا می‌کند، یکی از حالت آغازین به جلو و یکی از هدف به عقب؛ اگر هر دو جستجو جستجوی سطح-اول (یا هزینه-یکنواخت) باشند، راه‌حل بهینه می‌یابد و پیچیدگی زمانی‌اش به‌شکل مجانبی بهتر است.
- روش‌های جستجوی مطلع از تابع اکتشافی $h(n)$ استفاده می‌کنند که تخمینی از فاصله تا هدف می‌دهد:

- جستجوی بهترین-اول حریصانه گره‌ای را گسترش می‌دهد که طبق h به هدف نزدیک‌ترین می‌نماید؛ سریع است اما همیشه بهینه نیست.
- جستجوی A* با f(n) = g(n) + h(n) گسترش می‌دهد؛ با اکتشافی مقبول کامل و بهینهٔ هزینه است، اما پیچیدگی فضایی نمایی دارد.
- اکتشافی سازگار (که نابرابری مثلث را رعایت می‌کند) مقبول است و باعث می‌شود A* هر حالت را فقط یک بار گسترش دهد.
- با A* وزنی می‌توان سرعت را بر بهینگی ترجیح داد؛ راه‌حل‌های با تضمینِ درون ضریب W از بهینه می‌یابد.
- IDA* و RBFS جستجوی بهینه انجام می‌دهند و فضای کمتری مصرف می‌کنند، اما برخی حالت‌ها را چند بار بازتولید می‌کنند.
- SMA* از حافظهٔ موجود استفادهٔ کامل می‌کند و می‌تواند راه‌حل بهینهٔ قابل دسترسی را بیابد.

- اکتشافی‌ها را می‌توان از ساده‌سازی مسئله (که در آن اکتشافی هزینهٔ راه‌حل بهینهٔ مسئلهٔ ساده‌شده است)، از زیرمسئله‌ها (پایگاه‌داده‌های الگو)، از نقطه‌های نشانهٔ ازپیش‌محاسبه‌شده، از یادگیری ماشین و از راه‌های دیگر تولید کرد.

یادداشت‌های کتاب‌شناختی و تاریخی

مطالعهٔ سیستماتیک جستجوی حالت‌فضا در میانه‌های دههٔ ۱۹۵۰ با مقالهٔ «برنامه‌ریزی و حل مسئله» نیوول و سایمون (۱۹۵۶) آغاز شد و در دههٔ ۱۹۶۰ به سرعت رشد کرد. کتاب «حل مسئلهٔ انسانی» (نیوول و سایمون، ۱۹۷۲) همچنان منبع غنی بینش دربارهٔ جستجو و استدلال است. ایدهٔ درخت جستجو در کار هارت، نیلسون و رافائل (۱۹۶۸) و نیلسون (۱۹۷۱) صوری شد. «الگوریتم‌های حل مسئله» نیلسون (۱۹۷۱) و کتاب‌های بعدی او (۱۹۸۰، ۱۹۸۸) مرورهای استاندارد آن دوره بودند. توصیف‌های کلی‌تر جستجو را در کورف (۱۹۸۸)، پیرل (۱۹۸۴) و کتاب‌های درسی الگوریتم (کورمن و همکاران، ۲۰۰۹) و هوش مصنوعی (پول و مک‌ورث، ۲۰۱۷) می‌توان یافت. هر یک از مسائل جستجوی جهان واقعیِ فهرست‌شده در فصل موضوع تلاش پژوهشی فراوانی بوده است. روش‌های انتخاب پروازهای هوایی بهینه بیشتر اختصاصی مانده‌اند، اما کارل د مارکن با تقلیل به مسائل تصمیم دیوفانتی نشان داده قیمت‌گذاری و محدودیت‌های بلیت هواپیما چنان درهم‌پیچیده شده که مسئلهٔ انتخاب پرواز بهینه به‌صورت صوری تصمیم‌ناپذیر است (رابینسون، ۲۰۰۲). مسئلهٔ فروشندهٔ دوره‌گرد (TSP) مسئلهٔ ترکیبیاتی استانداردی در علوم رایانهٔ نظری است (لاولر و همکاران، ۱۹۹۲). کارپ (۱۹۷۲) مسئلهٔ تصمیم TSP را NP-سخت اثبات کرد، اما روش‌های تقریب اکتشافی مؤثری توسعه یافت (لین و کرنیگان، ۱۹۷۳). آرورا (۱۹۹۸) طرح تقریب کاملاً چندجمله‌ای برای TSP اقلیدسی ابداع کرد. روش‌های چیدمان VLSI توسط لاپو (۲۰۱۰) مرور شده و مقاله‌های بهینه‌سازی چیدمان فراوانی در ژورنال‌های VLSI ظاهر می‌شوند. ناوبری رباتیک در فصل ۲۶ بحث می‌شود. توالی‌یابی مونتاژ خودکار نخستین بار توسط فردی (مایکی، ۱۹۷۲) نشان داده شد؛ مرور جامعی توسط (باهوبالندرونی و بیسوال، ۲۰۱۶) ارائه شده.

الگوریتم‌های جستجوی نامطلع موضوع مرکزی علوم رایانه (کورمن و همکاران، ۲۰۰۹) و تحقیق در عملیات (دریفوس، ۱۹۶۹) هستند. جستجوی سطح-اول برای حل مارپیچ‌ها توسط مور (۱۹۵۹) صورت‌بندی شد. روش برنامه‌نویسی پویا (بلمن، ۱۹۵۷؛ بلمن و دریفوس، ۱۹۶۲)، که به‌صورت نظام‌مند راه‌حل‌های همهٔ زیرمسئله‌های با طول فزاینده را ثبت می‌کند، می‌تواند شکلی از جستجوی سطح-اول دیده شود.

الگوریتم دیکسترا به شکلی که معمولاً ارائه می‌شود (دیکسترا، ۱۹۵۹) برای گراف‌های متناهی صریح کاربردپذیر است. نیلسون (۱۹۷۱) نسخه‌ای از الگوریتم دیکسترا را معرفی کرد که آن را جستجوی هزینه-یکنواخت نامید (زیرا الگوریتم «در امتداد خطوط ترازِ هزینهٔ مسیرِ یکسان پخش می‌شود») و گراف‌های ضمنیِ بی‌نهایت را ممکن می‌سازد. کار نیلسون همچنین ایدهٔ فهرست‌های بسته و باز و اصطلاح «جستجوی گرافی» را معرفی کرد. نام BEST-FIRST-SEARCH در «راهنمای هوش مصنوعی» (بار و فایگنباوم، ۱۹۸۱) معرفی شد. الگوریتم‌های فلوید–وارشال (فلوید، ۱۹۶۲) و بلمن–فورد (بلمن، ۱۹۵۸؛ فورد، ۱۹۵۶) هزینه‌های گام منفی را (تا وقتی چرخهٔ منفی نباشد) ممکن می‌کنند.

نسخه‌ای از تعمیق تکراری که برای بهره‌برداری کارآمد از ساعت شطرنج طراحی شده نخستین بار توسط اسلیت و آتکین (۱۹۷۷) در برنامهٔ بازی شطرنج CHESS 4.5 به کار رفت. الگوریتم B مارتلی (۱۹۷۷) نیز جنبهٔ تعمیق تکراری در بر دارد. فن تعمیق تکراری توسط برترام رافائل (۱۹۷۶) معرفی شد و در کار کورف (۱۹۸۵) به اوج رسید.

کاربرد اطلاعات اکتشافی در حل مسئله در مقالهٔ آغازین سایمون و نیوول (۱۹۵۸) آمده، اما عبارت «جستجوی اکتشافی» و به‌کارگیری توابع اکتشافیِ تخمین‌زنندهٔ فاصله تا هدف کمی بعدتر آمد (نیوول و ارنست، ۱۹۶۵؛ لین، ۱۹۶۵). دورن و مایکی (۱۹۶۶) مطالعات تجربی گسترده‌ای در جستجوی اکتشافی انجام دادند. هرچند آنان طول مسیر و «نفوذ» (نسبت طول مسیر به شمار کل گره‌های بررسی‌شده تاکنون) را تحلیل کردند، به نظر می‌رسد اطلاعاتِ ارائه‌شده توسط هزینهٔ مسیر g(n) را نادیده گرفته‌اند. الگوریتم A*، که هزینهٔ مسیر کنونی را در جستجوی اکتشافی وارد می‌کند، توسط هارت، نیلسون و رافائل (۱۹۶۸) توسعه یافت. دچتر و پیرل (۱۹۸۵) شرایطی را مطالعه کردند که A* در آن بهینه‌کارا (در شمار گره‌های گسترش‌یافته) است.

مقالهٔ اصلی A* (هارت و همکاران، ۱۹۶۸) شرط سازگاری را بر توابع اکتشافی معرفی کرد. شرط یکنواختی توسط پول (۱۹۷۷) به‌عنوان جایگزینی ساده‌تر معرفی شد، اما پیرل (۱۹۸۴) نشان داد این دو معادل‌اند.

پول (۱۹۷۷) پیش‌گام مطالعهٔ رابطهٔ میان خطا در توابع اکتشافی و پیچیدگی زمانی A* شد. نتایج پایه برای جستجوی درخت‌مانند با هزینه‌های کنش واحد و یک حالت هدف واحد (پول، ۱۹۷۷؛ گاشنیگ، ۱۹۷۹؛ هوین و همکاران، ۱۹۸۰؛ پیرل، ۱۹۸۴) و با چند حالت هدف (دینه و همکاران، ۲۰۰۷) به دست آمد. کورف و رید (۱۹۹۸) نشان دادند چگونه شمار دقیق گره‌های گسترش‌یافته (نه فقط تقریب مجانبی) را در گستره‌ای از حوزه‌های مسئلهٔ واقعی پیش‌بینی کنند. «ضریب انشعاب مؤثر» توسط نیلسون (۱۹۷۱) به‌عنوان معیار تجربی کارایی پیشنهاد شد. برای جستجوی گرافی، هلمرت و رگر (۲۰۰۸) یادآور شدند چند مسئلهٔ شناخته‌شده حاوی گره‌های نمایی‌چندانی روی مسیرهای راه‌حلِ بهینهٔ هزینه هستند که پیچیدگی زمانی نمایی برای A* دلالت دارد.

تغییرهای بسیاری بر الگوریتم A* هست. پول (۱۹۷۰) جستجوی A* وزنی و بعداً نسخهٔ پویایش (۱۹۷۳) را معرفی کرد که در آن وزن با عمق درخت تغییر می‌کند. ابن‌دت و درشلر (۲۰۰۹) نتایج را ترکیب و برخی کاربردها را بررسی می‌کنند. هاتم و رومل (۲۰۱۴) نسخهٔ ساده‌شده و بهبودیافتهٔ A* وزنی را نشان می‌دهند که پیاده‌سازی‌اش آسان‌تر است. ویلت و رومل (۲۰۱۴) جستجوی سریع را به‌عنوان جایگزینی برای جستجوی حریصانه معرفی می‌کنند که بر کمینه‌کردن زمان جستجو متمرکز است، و ویلت و رومل (۲۰۱۶) نشان می‌دهند بهترین اکتشافی‌ها برای جستجوی رضایت‌بخش با بهترین‌ها برای جستجوی بهینه متفاوت‌اند. برنز و همکاران (۲۰۱۲) چند ترفند پیاده‌سازی برای نوشتن کد جستجوی سریع می‌دهند و فلنر (۲۰۱۸) بررسی می‌کند پیاده‌سازی هنگام استفاده از آزمون هدف زودهنگام چگونه تغییر می‌کند.

پول (۱۹۷۱) جستجوی دوطرفه را معرفی کرد. هولته و همکاران (۲۰۱۶) نسخه‌ای از جستجوی دوطرفه را توصیف می‌کنند که تضمین‌شده در میانه به هم می‌رسد و آن را کاربردی‌تر می‌کند. اکرله و همکاران (۲۰۱۷) مجموعهٔ جفت‌گره‌های قطعاً گسترش‌یافته را توصیف و نشان می‌دهند هیچ جستجوی دوطرفه‌ای نمی‌تواند بهینه‌کارا باشد. الگوریتم NBS (چن و همکاران، ۲۰۱۷) به‌صراحت از صفی از جفت‌گره‌ها استفاده می‌کند.

ترکیبی از A* دوطرفه و نقطه‌های نشانهٔ شناخته‌شده برای یافتن کارآمد مسیرهای رانندگیِ سرویس نقشهٔ آنلاین مایکروسافت به کار رفت (گلدبرگ و همکاران، ۲۰۰۶). الگوریتم پس از کش‌کردن مجموعه‌ای از مسیرهای میان نقطه‌های نشانه، می‌تواند مسیر بهینهٔ هزینه را میان هر جفت نقطه در گراف ۲۴ میلیون‌نقطه‌ای ایالات متحده بیابد و کمتر از ۰٫۱٪ گراف را جستجو کند. کورف (۱۹۸۷) نشان می‌دهد چگونه با زیرهدف‌ها، عملگرهای کلان و انتزاع، شتاب‌های چشمگیری نسبت به فنون قبلی به دست آوریم. دلینگ و همکاران (۲۰۰۹) توصیف می‌کنند چگونه از جستجوی دوطرفه، نقطه‌های نشانه، ساختار سلسله‌مراتبی و دیگر حربه‌ها برای یافتن مسیرهای رانندگی استفاده کنند. اندرسون و همکاران (۲۰۰۸) فن مرتبطی به نام جستجوی درشت‌به‌ریز (Coarse-to-Fine Search) را توصیف می‌کنند که می‌توان آن را تعریف نقطه‌های نشانه در سطح‌های انتزاعی سلسله‌مراتبی گوناگون دانست. کورف (۱۹۸۷) شرایطی را توصیف می‌کند که جستجوی درشت‌به‌ریز در آن‌ها شتاب نمایی می‌دهد. نابلاک (۱۹۹۱) نتایج تجربی و تحلیل برای کمّی‌کردن مزیت‌های جستجوی سلسله‌مراتبی ارائه می‌دهد.

A* و دیگر الگوریتم‌های جستجوی فضای حالت با فنون شاخه‌وکران (Branch-and-Bound) که در تحقیق در عملیات پرکاربردند پیوند نزدیک دارند (لاولر و وود، ۱۹۶۶؛ ری‌وارد-اسمیت و همکاران، ۱۹۹۶). کومار و کانال (۱۹۸۸) «وحدت بزرگ» جستجوی اکتشافی، برنامه‌نویسی پویا و فنون شاخه‌وکران را زیر نام CDP — «فرایند تصمیم مرکب» — می‌آزمایند. چون بیشتر رایانه‌های دههٔ ۱۹۶۰ فقط چند هزار واژه حافظهٔ اصلی داشتند، جستجوی اکتشافی محدود-به-حافظه موضوع پژوهشی آغازینی بود. «پیمایش‌گر گراف» (دورن و مایکی، ۱۹۶۶)، یکی از کهن‌ترین برنامه‌های جستجو، پس از جستجوی بهترین-اول تا حد حافظه، به کنشی متعهد می‌شود. IDA* (کورف، ۱۹۸۵) نخستین الگوریتم جستجوی اکتشافی محدود-به-حافظهٔ پرکاربردِ بهینه بود و تعداد زیادی تغییر برایش توسعه یافته. تحلیلی از کارایی IDA* و دشواری‌هایش با اکتشافی‌های مقدار-حقیقی در پاتریک و همکاران (۱۹۹۲) آمده.

نسخهٔ اصلی RBFS (کورف، ۱۹۹۳) در واقع تا حدی پیچیده‌تر از الگوریتم نشان‌داده‌شده در شکل ۳.۲۲ است که در واقع به الگوریتمی مستقل‌توسعه‌یافته به نام بسط تکراری (Iterative Expansion) یا IE (راسل، ۱۹۹۲) نزدیک‌تر است. RBFS از کران پایین نیز به‌همراه کران بالا استفاده می‌کند؛ دو الگوریتم با اکتشافی‌های مقبول یکسان رفتار می‌کنند، اما

RBFS گره‌ها را به‌ترتیب بهترین-اول حتی با اکتشافی نامقبول گسترش می‌دهد. ایدهٔ پیگیری بهترین مسیر جایگزین پیش‌تر در پیاده‌سازی زیبای پرولوگ A* توسط براتکو (۲۰۰۹) و در الگوریتم DTA* (راسل و وفالد، ۱۹۹۱) آمده بود. کار اخیر همچنین فضاهای حالت فراسطح و یادگیری فراسطح را بحث می‌کند.

الگوریتم MA* در چاکرابارتی و همکاران (۱۹۸۹) آمد. SMA*، یا MA* ساده‌شده، از تلاشی برای پیاده‌سازی MA* (راسل، ۱۹۹۲) برخاست. کایندل و خرسند (۱۹۹۴) SMA* را برای تولید الگوریتم جستجوی دوطرفه‌ای اعمال کردند که به‌شکل چشمگیری از الگوریتم‌های قبلی سریع‌تر بود. کورف و ژانگ (۲۰۰۰) رویکرد تقسیم‌وتسلط توصیف می‌کنند و ژو و هانسن (۲۰۰۲) جستجوی گرافی A* محدود-به-حافظه و راهبردی برای سوئیچ به جستجوی سطح-اول برای افزایش کارایی حافظه معرفی می‌کنند (ژو و هانسن، ۲۰۰۶).

ایدهٔ اینکه اکتشافی‌های مقبول را می‌توان با ساده‌سازی مسئله مشتق کرد در مقالهٔ بنیادین هلد و کارپ (۱۹۷۰) آمده که از اکتشافی درخت پوشای کمینه برای حل TSP استفاده کردند. (تمرین MSTR.3 را ببینید.) خودکارسازی فرایند ساده‌سازی با موفقیت توسط پریدیتیس (۱۹۹۳) پیاده شد. ادبیات رو به رشدی دربارهٔ کاربرد یادگیری ماشین برای کشف توابع اکتشافی هست (سمدی و همکاران، ۲۰۰۸؛ آرفایی و همکاران، ۲۰۱۰؛ تایر و همکاران، ۲۰۱۱؛ للیس و همکاران، ۲۰۱۲).

استفاده از پایگاه‌داده‌های الگو برای مشتق‌کردن اکتشافی‌های مقبول از گاسر (۱۹۹۵) و کالبرسون و شفر (۱۹۹۶، ۱۹۹۸) است؛ پایگاه‌داده‌های الگوی ناهم‌پوش توسط کورف و فلنر (۲۰۰۲) توصیف شده؛ روش مشابهی با الگوهای نمادین از ادلکمپ (۲۰۰۹) است. فلنر و همکاران (۲۰۰۷) نشان می‌دهند چگونه پایگاه‌داده‌های الگو را برای صرفه‌جویی در فضا فشرده کنند. تفسیر احتمالاتی اکتشافی‌ها توسط پیرل (۱۹۸۴) و هانسون و مایر (۱۹۸۹) بررسی شد.

پانوشت‌ها:

[1] فرض می‌کنیم بیشتر خوانندگان در همان وضعیت‌اند و به‌راحتی می‌توانند خود را به همان اندازهٔ عامل ما بی‌اطلاع تصور کنند. از خوانندگان رومانیایی که نمی‌توانند از این ابزار آموزشی بهره ببرند پوزش می‌خواهیم.

[2] برای مسائلی با شمار بی‌نهایت کنش، به فنونی فراتر از این فصل نیاز خواهیم داشت.

[3] در هر مسئله‌ای با چرخه‌ای با هزینهٔ خالص منفی، راه‌حل بهینهٔ هزینه آن است که بی‌نهایت بار دور آن چرخه بزند. الگوریتم‌های بلمن–فورد و فلوید–وارشال (که این‌جا پوشش داده نمی‌شوند) کنش‌های هزینه‌منفی را مدیریت می‌کنند، تا وقتی چرخهٔ منفی نباشد. تطبیق کنش‌های هزینه‌صفر آسان است، تا وقتی شمار کنش‌های هزینه‌صفرِ پیاپی کران‌دار باشد. برای نمونه، ممکن است رباتی داشته باشیم که حرکت هزینه دارد اما چرخش ۹۰ درجه هزینه‌ای ندارد؛ الگوریتم‌های این فصل می‌توانند این را مدیریت کنند تا وقتی بیش از سه چرخش ۹۰ درجهٔ پیاپی مجاز نباشد. پیچیدگی دیگری نیز با مسائلی هست که شمار بی‌نهایت هزینهٔ کنشِ خودسرانه کوچک دارند. نسخه‌ای از پارادوکس زنو را در نظر بگیرید که کنشی برای حرکت به نیمهٔ راهِ هدف با هزینهٔ نصف حرکت قبلی هست. این مسئله با شمار متناهی کنش راه‌حل ندارد، اما برای جلوگیری از گرفتن شمار بی‌کران کنش بدون رسیدن کامل به هدف، می‌توانیم ایجاب کنیم همهٔ هزینه‌های کنش دست‌کم ε باشند، برای ε مثبتِ کوچکی.

[4] بخش ۱۱.۴ را ببینید.

[5] برخی نویسندگان جبهه را «فهرست باز» می‌نامند و مجموعهٔ گره‌های قبلاً گسترش‌یافته را «فهرست بسته»؛ ما اصطلاحات جبهه و رسیده را ترجیح می‌دهیم.

[6] «درخت‌مانند» می‌گوییم، زیرا فضای حالت همچنان همان گراف است، هرگونه که جستجو کنیم؛ ما فقط انتخاب می‌کنیم آن را چنان رفتار کنیم که گویی درخت است، با فقط یک مسیر از هر گره به ریشه.

[7] برخی نویسندگان از اصطلاح «مقبولیت» برای ویژگی یافتن راه‌حل کم‌هزینه‌ترین استفاده می‌کنند و برخی فقط «بهینگی»، اما آن را می‌توان با انواع دیگر بهینگی اشتباه گرفت.

[8] در این‌جا و در سراسر کتاب، «ستاره» در C^* به معنای مقدار بهینهٔ C است.

[9] در پیاده‌سازی ما، ساختار دادهٔ reached از پرس‌وجوی عضویت پشتیبانی می‌کند و جبهه (صف اولویت‌دار) نه؛ پس برخورد را با reached بررسی می‌کنیم؛ اما از نظر مفهومی می‌پرسیم آیا دو جبهه به هم رسیده‌اند. پیاده‌سازی را می‌توان برای چند حالت هدف با بارگذاری گرهٔ هر حالت هدف در جبههٔ عقب و جدول reached عقب بسط داد.

[10] ممکن است عجیب بنماید که تابع اکتشافی روی گره عمل می‌کند، در حالی که واقعاً فقط به حالت گره نیاز دارد. سنت بر آن است که به‌جای $h(s)$ از $h(n)$ استفاده شود تا با تابع ارزیابی $f(n)$ و هزینهٔ مسیر $g(n)$ سازگار باشد.

[11] باز هم، با فرض همهٔ هزینه‌های کنش $\varepsilon > 0$ و اینکه فضای حالت یا راه‌حل دارد یا متناهی است.

[13] در واقع، اصطلاح «اکتشافی یکنواخت» مترادف «اکتشافی سازگار» است. دو ایده مستقل توسعه یافتند و سپس اثبات شد معادل‌اند (پیرل، ۱۹۸۴).

[14] در فصل‌های ۸ و ۱۱ زبان‌های صوری مناسبی برای این تکلیف توصیف می‌کنیم؛ با توصیف‌های صوریِ قابل دست‌کاری، ساخت مسئله‌های ساده‌شده قابل خودکارسازی است. فعلاً از انگلیسی استفاده می‌کنیم.

[15] با کار به عقب از هدف، هزینهٔ دقیق راه‌حل هر نمونهٔ روی‌مواجه‌شده بلافاصله در دسترس است. این مثالی از برنامه‌نویسی پویاست که در فصل ۱۷ بیشتر بحث می‌کنیم.

[16] نقطه‌های نشانه گاهی «محور» (Pivot) یا «لنگر» (Anchor) نامیده می‌شوند.

فصل ۴: جستجو در محیط‌های پیچیده (Search in Complex Environments)

در این فصل فرض‌های ساده‌کنندهٔ فصل پیشین را شل می‌کنیم تا به جهان واقعی نزدیک‌تر شویم.

فصل ۳ به مسائلِ محیط‌های کاملاً قابل مشاهده، جبرگرایانه، ایستا و شناخته‌شده پرداخت که راه‌حل در آن‌ها توالی‌ای از کنش‌هاست. در این فصل آن قیدها را شل می‌کنیم. با مسئلهٔ یافتن حالت خوب بدون نگرانی از مسیر رسیدن به آن آغاز می‌کنیم و هم حالت‌های گسسته (بخش ۴.۱) و هم پیوسته (بخش ۴.۲) را پوشش می‌دهیم. سپس فرض‌های جبرگرایی (بخش ۴.۳) و قابل مشاهده‌بودن (بخش ۴.۴) را شل می‌کنیم. در جهان ناجبرگرایانه، عامل به برنامهٔ شرطی نیاز خواهد داشت و بسته به آنچه مشاهده می‌کند کنش‌های متفاوتی انجام می‌دهد — برای نمونه، اگر چراغ قرمز بود بایستد و اگر سبز بود برود. با مشاهده‌ناپذیری تا حدی، عامل همچنین باید حالت‌های ممکنی را که ممکن است در آن‌ها باشد پیگیری کند. سرانجام، بخش ۴.۵ عامل را در فضای ناشناخته‌ای راهنمایی می‌کند که باید هنگام پیش‌روی بیاموزدش، با جستجوی آنلاین.

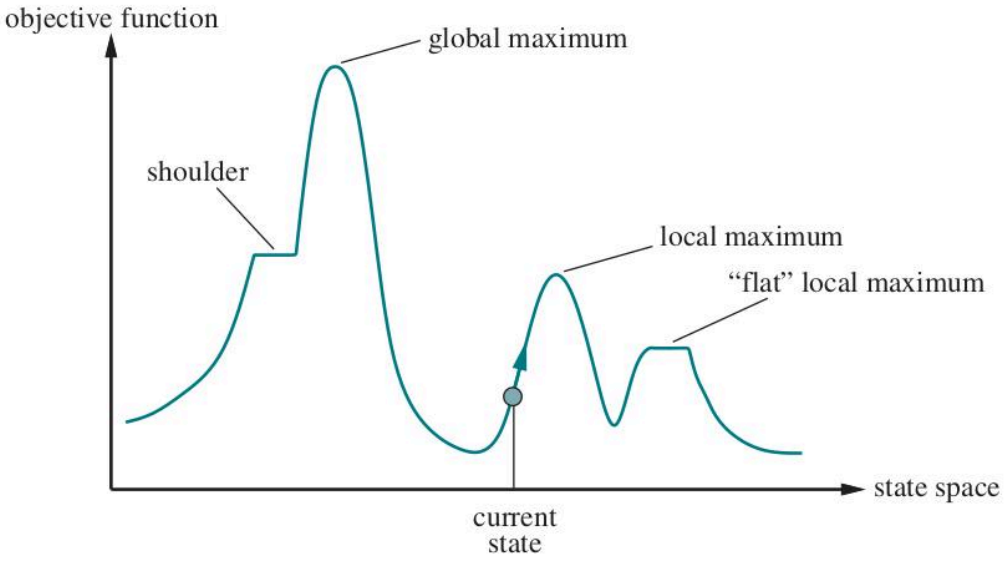
۴.۱ جستجوی محلی و مسائل بهینه‌سازی

در مسائل جستجوی فصل ۳ می‌خواستیم مسیرهایی در فضای جستجو بیابیم، مانند مسیری از آراد به بخارست. اما گاهی فقط به حالت نهایی اهمیت می‌دهیم، نه مسیر رسیدن به آن. برای نمونه، در مسئلهٔ هشت‌وزیر (شکل ۴.۳)، فقط به یافتن پیکربندی نهاییِ معتبری از ۸ وزیر اهمیت می‌دهیم (زیرا اگر پیکربندی را بدانید، بازسازی گام‌های ایجادکننده‌اش بدیهی است). این برای بسیاری کاربردهای مهم نیز صادق است: طراحی مدار مجتمع، چیدمان کف کارخانه، زمان‌بندی کارگاه، برنامه‌نویسی خودکار، بهینه‌سازی شبکهٔ مخابراتی، برنامه‌ریزی کشاورزی و مدیریت سبد سهام.

الگوریتم‌های جستجوی محلی (Local Search) با جستجو از حالت آغازین به حالت‌های همسایه عمل می‌کنند، بدون پیگیری مسیرها یا مجموعهٔ حالت‌های رسیده. این بدان معناست آن‌ها نظام‌مند نیستند — ممکن است هرگز بخشی از فضای جستجو را که راه‌حل واقعاً در آن است کاوش نکنند. اما دو مزیت کلیدی دارند: (۱) حافظهٔ بسیار کمی مصرف می‌کنند؛ و (۲) اغلب می‌توانند در فضاهای حالت بزرگ یا بی‌نهایت، که الگوریتم‌های نظام‌مند برایشان نامناسب‌اند، راه‌حل‌های معقول بیابند.

الگوریتم‌های جستجوی محلی همچنین می‌توانند مسائل بهینه‌سازی (Optimization Problems) را حل کنند که هدف در آن‌ها یافتن بهترین حالت بر اساس تابع هدف (Objective Function) است.

برای فهم جستجوی محلی، حالت‌های مسئله‌ای را در نظر بگیرید که در «چشم‌انداز فضای حالت» (State-Space Landscape) چیده شده‌اند، چنان‌که در شکل ۴.۱. هر نقطه (حالت) در چشم‌انداز «ارتفاعی» دارد که با مقدار تابع هدف تعریف می‌شود. اگر ارتفاع با تابع هدف متناظر باشد، هدف یافتن بلندترین قله – یعنی بیشینهٔ سراسری (Global Maximum) – است و فرایند را «تپه‌نوردی» (Hill Climbing) می‌نامیم. اگر ارتفاع با هزینه متناظر باشد، هدف یافتن ژرف‌ترین دره – یعنی کمینهٔ سراسری (Global Minimum) – است و آن را «نزول گرادیان» (Gradient Descent) می‌نامیم.



شکل ۴.۱ – چشم‌انداز فضای حالت یک‌بعدی که در آن ارتفاع با تابع هدف متناظر است. هدف یافتن بیشینهٔ سراسری است. (بیشینهٔ سراسری، بیشینهٔ محلی، «بیشینهٔ محلی تخت»، شانه)

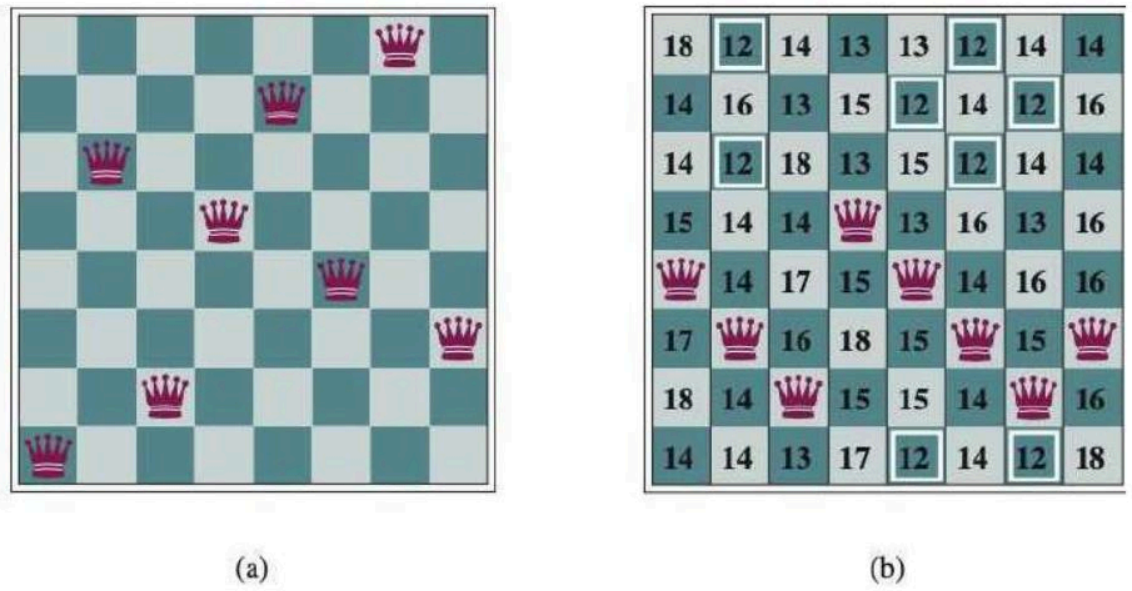
۴.۱.۱ جستجوی تپه‌نوردی

الگوریتم جستجوی تپه‌نوردی در شکل ۴.۲ نشان داده شده. یک حالت کنونی را پیگیری می‌کند و در هر تکرار به حالت همسایه‌ای با بیشترین مقدار حرکت می‌کند – یعنی در جهتی می‌رود که «صعود تندترین» (Steepest Ascent) را فراهم می‌کند. وقتی به «قله‌ای» رسید که هیچ همسایه‌ای مقدار بالاتر ندارد، پایان می‌یابد. تپه‌نوردی فراتر از همسایه‌های بلافصل حالت کنونی به جلو نمی‌نگرد. این شبیه تلاش برای یافتن قلهٔ اورست در مه غلیظ در حالی است که از فراموشی رنج می‌برید. توجه کنید یکی از راه‌های استفاده از جستجوی تپه‌نوردی، به‌کارگیری منفیِ تابع هزینهٔ اکتشافی به‌عنوان تابع هدف است؛ آنگاه به‌صورت محلی به حالتی با کمترین فاصلهٔ اکتشافی تا هدف صعود می‌کند.

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
                                        current ← problem.INITIAL
                                                  while true do
                    neighbor ← highest-valued successor state of current
                if VALUE(neighbor) ≤ VALUE(current) then return current
                                                 current ← neighbor
```

شکل ۴.۲ – الگوریتم جستجوی تپه‌نوردی، که پایه‌ای‌ترین فن جستجوی محلی است. در هر گام، گرهٔ کنونی با بهترین همسایه جایگزین می‌شود.

برای نشان دادن تپه‌نوردی، از مسئلهٔ هشت‌وزیر (Queens-8) استفاده می‌کنیم (شکل ۴.۳). از صورت‌بندی حالت-کامل (Complete-State Formulation) استفاده می‌کنیم؛ یعنی هر حالت همهٔ مؤلفه‌های راه‌حل را دارد، اما ممکن است همه در جای درست نباشند. در این مورد هر حالت ۸ وزیر روی تخته دارد، در هر ستون یکی. حالت آغازین تصادفی برگزیده می‌شود و جانشین‌های یک حالت، همهٔ حالت‌های ممکنِ تولیدشده با جابه‌جایی یک وزیر به خانه‌ای دیگر در همان ستون‌اند (پس هر حالت ۸×۷=۵۶ جانشین دارد). تابع هزینهٔ اکتشافی h شمار جفت‌وزیرهایی است که به هم حمله می‌کنند؛ این فقط برای راه‌حل‌ها صفر است. (اگر دو مهره در یک خط باشند، حتی اگر مهره‌ای میانشان باشد، حمله شمرده می‌شود.) شکل ۴.۳(ب) حالتی با h=17 نشان می‌دهد. شکل همچنین مقادیر h همهٔ جانشین‌هایش را نشان می‌دهد.



شکل ۴.۳ – (الف) مسئلهٔ هشت‌وزیر: ۸ وزیر را روی صفحهٔ شطرنج بگذارید تا هیچ وزیری به وزیر دیگر حمله نکند. (وزیر به هر مهره‌ای در همان سطر، ستون یا قطر حمله می‌کند.) این وضعیت تقریباً راه‌حل است، به‌جز دو وزیر ستون چهارم و هفتم که در امتداد قطر به هم حمله می‌کنند. (ب) حالت هشت‌وزیر با تخمین هزینهٔ اکتشافی h=17. صفحه مقدار h را برای هر جانشین ممکنِ حاصل از جابه‌جایی وزیری در ستون خودش نشان می‌دهد. ۸ حرکت با h=12 در برترین رتبه مشترک‌اند. الگوریتم تپه‌نوردی یکی از آن‌ها را برخواهد گزید.

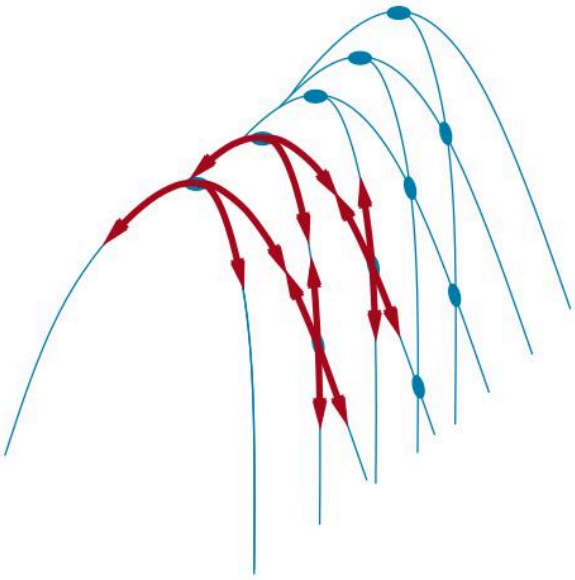
تپه‌نوردی گاهی «جستجوی محلی حریصانه» (Greedy Local Search) نامیده می‌شود، زیرا بدون اندیشیدن به اینکه بعداً به کجا برود، حالت همسایهٔ خوبی را می‌گیرد. هرچند حرص یکی از گناهان کبیره شمرده می‌شود، معلوم شده الگوریتم‌های حریصانه اغلب عملکرد خوبی دارند. تپه‌نوردی می‌تواند به‌سرعت به سوی راه‌حل پیشرفت کند، زیرا معمولاً بهبود حالت بد آسان است. برای نمونه، از حالت شکل ۴.۳(ب)، فقط پنج گام لازم است تا به حالت شکل ۴.۳(الف) برسیم که h=1 دارد و بسیار نزدیک به راه‌حل است.

متأسفانه، تپه‌نوردی می‌تواند به هر یک از دلایل زیر گیر کند:

- **بیشینه‌های محلی (Local Maxima):** بیشینهٔ محلی قله‌ای است که از هر یک از حالت‌های همسایه‌اش بلندتر اما از بیشینهٔ سراسری پایین‌تر است. الگوریتم‌های تپه‌نوردی که به مجاورت بیشینهٔ محلی می‌رسند به سوی قله کشیده می‌شوند، اما سپس با هیچ راه دیگری گیر می‌کنند. شکل ۴.۱ مسئله را به‌صورت شماتیک نشان می‌دهد.

عینی‌تر، حالت شکل ۴.۳(الف) بیشینهٔ محلی است (یعنی کمینهٔ محلی برای هزینهٔ h)؛ هر حرکت یک‌وزیره وضعیت را بدتر می‌کند.

- **خط‌الرأس‌ها (Ridges):** خط‌الرأسی در شکل ۴.۴ نشان داده شده. خط‌الرأس‌ها توالی‌ای از بیشینه‌های محلی پدید می‌آورند که پیمایش‌شان برای الگوریتم‌های حریصانه بسیار دشوار است.



شکل ۴.۴ – تصویری از اینکه چرا خط‌الرأس‌ها برای تپه‌نوردی دشواری ایجاد می‌کنند. شبکهٔ حالت‌ها (دایره‌های تیره) بر خط‌الرأسی که از چپ به راست بالا می‌رود سوار شده و توالی‌ای از بیشینه‌های محلی ایجاد می‌کند که مستقیماً به هم متصل نیستند. از هر بیشینهٔ محلی، همهٔ کنش‌های در دسترس به سمت پایین اشاره می‌کنند. توپولوژی‌هایی مانند این در فضاهای حالت کم‌بعد رایج‌اند، مانند نقطه‌های صفحهٔ دوبعدی. اما در فضاهای حالت با صدها یا هزاران بُعد، این تصویر شهودی برقرار نیست و معمولاً دست‌کم چند بُعد هست که گریز از خط‌الرأس‌ها و فلات‌ها را ممکن می‌کند.

- **فلات‌ها (Plateaus):** فلات ناحیهٔ تختِ چشم‌انداز فضای حالت است. می‌تواند بیشینهٔ محلیِ تخت باشد که هیچ خروج سربالایی از آن نیست، یا «شانه» (Shoulder) که پیشرفت از آن ممکن است. (شکل ۴.۱ را ببینید.) جستجوی تپه‌نوردی می‌تواند در سرگردانی روی فلات گم شود.

در هر مورد، الگوریتم به نقطه‌ای می‌رسد که هیچ پیشرفتی حاصل نمی‌شود. با شروع از حالت تصادفی هشت‌وزیر، تپه‌نوردیِ صعود تندترین در ۸۶٪ مواقع گیر می‌کند و فقط ۱۴٪ نمونه‌های مسئله را حل می‌کند. از سوی دیگر، سریع کار می‌کند و در موفقیت فقط میانگین ۴ گام و در گیرکردن ۳ گام می‌برد – برای فضای حالتی با $8^8 \approx ۱۷$ میلیون حالت بد نیست. چگونه می‌توانیم مسائل بیشتری حل کنیم؟ یک پاسخ آن است که هنگام رسیدن به فلات ادامه دهیم – «حرکت کناری» (Sideways Move) را به امید اینکه فلات واقعاً شانه است، چنان‌که در شکل ۴.۱ نشان داده شده، مجاز کنیم. اما اگر واقعاً روی بیشینهٔ محلی تخت باشیم، این رویکرد تا ابد در فلات سرگردان خواهد بود. پس می‌توانیم شمار حرکت‌های کناریِ پیاپی را محدود کنیم و پس از مثلاً ۱۰۰ حرکت کناریِ پیاپی متوقف شویم. این درصد نمونه‌های حل‌شده توسط تپه‌نوردی را از ۱۴٪ به ۹۴٪ بالا می‌برد. موفقیت هزینه دارد: الگوریتم به‌طور میانگین حدود ۲۱ گام برای هر نمونهٔ موفق و ۶۴ گام برای هر شکست می‌برد.

تغییرهای بسیاری برای تپه‌نوردی ابداع شده. «تپه‌نوردی تصادفی» (Stochastic Hill Climbing) به‌صورت تصادفی از میان حرکت‌های سربالایی برمی‌گزیند؛ احتمال انتخاب می‌تواند با تندیِ حرکت سربالایی تغییر کند. این معمولاً کندتر از صعود تندترین همگرا می‌شود، اما در برخی چشم‌اندازهای حالت، راه‌حل‌های بهتری می‌یابد. «تپه‌نوردی انتخاب-اول» (First-Choice Hill Climbing) تپه‌نوردی تصادفی را با تولید تصادفی جانشین‌ها پیاده می‌کند تا آنکه جانشینی بهتر از حالت کنونی تولید شود. این راهبرد خوبی است وقتی حالتی جانشین‌های فراوان (مثلاً هزاران) دارد.

تغییر دیگر «تپه‌نوردی با راه‌اندازی مجدد تصادفی» (Random-Restart Hill Climbing) است که این ضرب‌المثل را به کار می‌گیرد: «اگر نخستین بار موفق نشدی، دوباره و دوباره تلاش کن.» سلسله‌ای جستجوی تپه‌نوردی از حالت‌های آغازینِ تصادفی‌تولیدشده انجام می‌دهد تا هدفی یافت شود. با احتمال ۱ کامل است، زیرا سرانجام حالت هدف را به‌عنوان حالت آغازین تولید خواهد کرد. اگر هر جستجوی تپه‌نوردی احتمال موفقیت p داشته باشد، شمار مورد انتظار راه‌اندازی‌های مجددِ لازم $1/p$ است. برای نمونه‌های هشت‌وزیر بدون حرکت کناری، $p \approx 0.14$، پس برای یافتن هدف تقریباً ۷ تکرار لازم است (۶ شکست و ۱ موفقیت). شمار مورد انتظار گام‌ها، هزینهٔ یک تکرار موفق به‌علاوهٔ $(1-p)/p$ برابرِ هزینهٔ شکست است، یعنی در مجموع تقریباً ۲۲ گام. وقتی حرکت کناری مجاز باشد، به‌طور میانگین $1/0.94 \approx 1.06$ تکرار لازم است و $(1\times21)+(0.06/0.94)\times64 \approx 25$ گام. پس برای هشت‌وزیر، تپه‌نوردی با راه‌اندازی مجدد تصادفی واقعاً بسیار مؤثر است. حتی برای سه میلیون وزیر، این رویکرد می‌تواند در چند ثانیه راه‌حل بیابد.[1]

موفقیت تپه‌نوردی بسیار به شکل چشم‌انداز فضای حالت بستگی دارد: اگر بیشینه‌های محلی و فلات‌ها اندک باشند، تپه‌نوردی با راه‌اندازی مجدد تصادفی خیلی سریع راه‌حل خوب می‌یابد. از سوی دیگر، بسیاری از مسائل واقعی چشم‌اندازی دارند که بیشتر به خانواده‌ای پراکنده از جوجه‌تیغی‌های کچل روی کف تخت می‌ماند، با جوجه‌تیغی‌های مینیاتوری که نوک هر خار زندگی می‌کنند. مسائل NP-سخت (پیوست الف را ببینید) معمولاً تعداد نمایی بیشینهٔ محلی برای گیرکردن دارند. با وجود این، پس از شمار کمی راه‌اندازی مجدد، اغلب می‌توان بیشینهٔ محلی نسبتاً خوبی یافت.

۴.۱.۲ بازپخت شبیه‌سازی‌شده

الگوریتم تپه‌نوردی که هرگز حرکت‌های «سراشیبی» به سوی حالت‌های با مقدار کمتر (یا هزینهٔ بیشتر) انجام نمی‌دهد، همیشه در برابر گیرکردن در بیشینهٔ محلی آسیب‌پذیر است. در برابر، «گام تصادفی» (Random Walk)ِ ناب که بدون توجه به مقدار به حالت جانشین حرکت می‌کند، سرانجام به بیشینهٔ سراسری خواهد خورد، اما بسیار ناکارآمد خواهد بود. پس معقول می‌نماید تپه‌نوردی را با گام تصادفی چنان ترکیب کنیم که هم کارایی و هم کامل بودن به دست آید. «بازپخت شبیه‌سازی‌شده» (Simulated Annealing) چنین الگوریتمی است.

در متالورژی، بازپخت (Annealing) فرایندی است که برای معتدل یا سخت‌کردن فلزات و شیشه به کار می‌رود: آن‌ها را تا دمای بالا گرم می‌کنند و سپس به‌تدریج سرد می‌کنند تا ماده به حالت بلوریِ کم‌انرژی برسد. برای توضیح بازپخت شبیه‌سازی‌شده، دیدگاه‌مان را از تپه‌نوردی به نزول گرادیان (یعنی کمینه‌کردن هزینه) تغییر می‌دهیم و تکلیف رساندن توپ پینگ‌پنگ به ژرف‌ترین شکاف سطحی بسیار ناهموار را تصور می‌کنیم. اگر فقط بگذاریم توپ غلت بخورد، در کمینهٔ محلی به آرامی می‌ایستد. اگر سطح را بلرزانیم، می‌توانیم توپ را از کمینهٔ محلی بیرون بپرانیم — شاید به کمینهٔ محلی

ژرف‌تری که زمان بیشتری در آن بماند. حربه آن است که دقیقاً به اندازه‌ای بلرزانیم که توپ از کمینه‌های محلی بیرون بپرد، اما نه آن‌قدر که از کمینهٔ سراسری بیرون بیفتد. راه‌حل بازپخت شبیه‌سازی‌شده آن است که با لرزش شدید (یعنی دمای بالا) آغاز کنیم و سپس شدت لرزش را به‌تدریج کم کنیم (یعنی دما را پایین بیاوریم).

ساختار کلی الگوریتم بازپخت شبیه‌سازی‌شده (شکل ۴.۵) شبیه تپه‌نوردی است. اما به‌جای برگزیدن بهترین حرکت، حرکتی تصادفی برمی‌گزیند. اگر حرکت وضعیت را بهبود دهد، همیشه پذیرفته می‌شود. وگرنه، الگوریتم حرکت را با احتمالی کمتر از ۱ می‌پذیرد. احتمال با «بدی» حرکت — مقدار ΔE که ارزیابی بدتر می‌شود — به‌صورت نمایی کاهش می‌یابد. احتمال همچنین با پایین‌رفتن «دما»ی T کاهش می‌یابد: حرکت‌های «بد» در آغاز که T بالاست بیشتر مجازند و با کاهش T نامحتمل‌تر می‌شوند. اگر برنامهٔ زمان‌بندی T را به‌اندازهٔ کافی آهسته به ۰ برساند، خاصیتی از توزیع بولتزمان، $e^{(\Delta E/T)}$، آن است که همهٔ احتمال بر بیشینه‌های سراسری متمرکز می‌شود که الگوریتم با احتمال نزدیک به ۱ آن‌ها را خواهد یافت.

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
                                                current ← problem.INITIAL
                                                        for t = 1 to ∞ do
                                                        T ← schedule(t)
                                                if T = 0 then return current
                                    next ← a randomly selected successor of current
                                                ΔE ← VALUE(current) – VALUE(next)
                                                    if ΔE > 0 then current ← next
                                    else current ← next only with probability e^(–ΔE/T)
```

شکل ۴.۵ — الگوریتم بازپخت شبیه‌سازی‌شده، نسخه‌ای از تپه‌نوردی تصادفی که در آن برخی حرکت‌های سراشیبی مجازند. ورودی schedule مقدار «دما»ی T را به‌عنوان تابعی از زمان تعیین می‌کند.

بازپخت شبیه‌سازی‌شده از دههٔ ۱۹۸۰ برای حل مسائل چیدمان VLSI به کار رفت. به‌طور گسترده در زمان‌بندی کارخانه و دیگر تکلیف‌های بهینه‌سازی بزرگ‌مقیاس اعمال شده است.

۴.۱.۳ جستجوی پرتوی محلی

نگه‌داشتن فقط یک گره در حافظه شاید واکنشی افراطی به مسئلهٔ محدودیت‌های حافظه بنماید. الگوریتم جستجوی پرتوی محلی (Local Beam Search) به‌جای فقط یک حالت، k حالت را پیگیری می‌کند. با k حالتِ تصادفی‌تولیدشده آغاز می‌کند. در هر گام، همهٔ جانشین‌های هر k حالت تولید می‌شوند. اگر هر یک هدف باشد، الگوریتم می‌ایستد. وگرنه، k جانشینِ برتر را از فهرست کامل برمی‌گزیند و تکرار می‌کند.

در نگاه نخست، جستجوی پرتوی محلی با k حالت شاید چیزی بیش از اجرای k راه‌اندازی مجدد تصادفی به‌صورت موازی به‌جای پیاپی نباشد. در واقع، دو الگوریتم کاملاً متفاوت‌اند. در جستجوی راه‌اندازی مجدد، هر فرایند جستجو مستقل از دیگران اجرا می‌شود. در جستجوی پرتوی محلی، اطلاعات مفید میان رشته‌های موازی ردوبدل می‌شود. عملاً،

حالت‌هایی که بهترین جانشین‌ها را تولید می‌کنند به بقیه می‌گویند: «بیایید این‌جا، علف این‌جا سبزتر است!» الگوریتم به‌سرعت جستجوهای بی‌ثمر را رها می‌کند و منابعش را به جایی منتقل می‌کند که بیشترین پیشرفت در حال وقوع است.

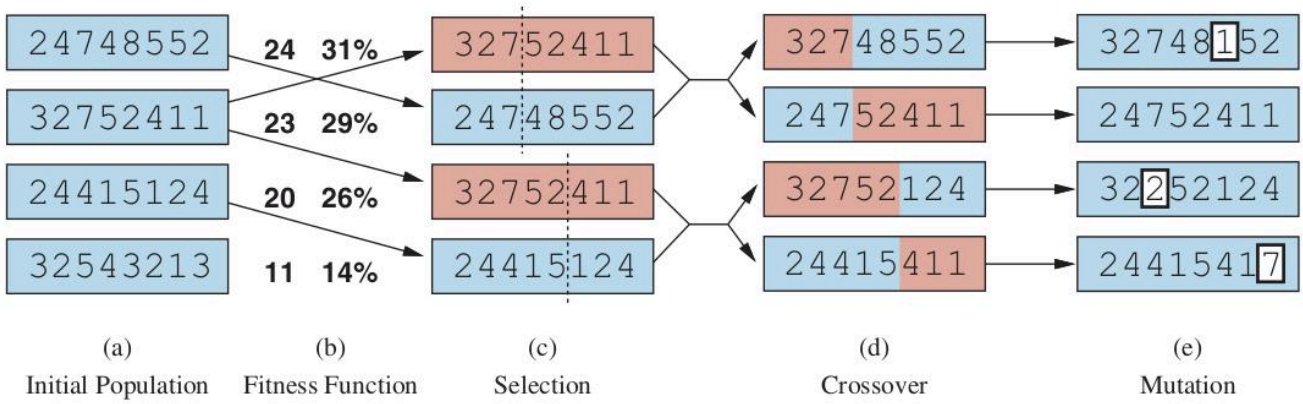
جستجوی پرتوی محلی می‌تواند از فقدان تنوع میان k حالت رنج ببرد — آن‌ها می‌توانند در ناحیهٔ کوچکی از فضای حالت خوشه‌ای شوند و جستجو را چیزی بیش از نسخهٔ k-برابر-کندترِ تپه‌نوردی نکنند. تغییری به نام جستجوی پرتوی تصادفی (Stochastic Beam Search)، مشابه تپه‌نوردی تصادفی، به تخفیف این مسئله کمک می‌کند. به‌جای برگزیدن k جانشین برتر، جستجوی پرتوی تصادفی جانشین‌ها را با احتمالی متناسب با مقدار جانشین برمی‌گزیند و بدین ترتیب تنوع را افزایش می‌دهد.

۴.۱.۴ الگوریتم‌های تکاملی

الگوریتم‌های تکاملی (Evolutionary Algorithms) را می‌توان تغییرهایی از جستجوی پرتوی تصادفی دانست که به‌صراحت از استعارهٔ انتخاب طبیعی در زیست‌شناسی الهام گرفته‌اند: جمعیتی از افراد (حالت‌ها) هست که در آن‌ها افرادِ برازنده‌ترین (بالاترین مقدار) فرزندانی (حالت‌های جانشین) تولید می‌کنند که نسل بعد را پر می‌کنند؛ فرایندی به نام بازترکیب (Recombination). شکل‌های بی‌پایانی از الگوریتم‌های تکاملی هست که از این نظرها متفاوت‌اند:

- اندازهٔ جمعیت.
- بازنمایی هر فرد. در الگوریتم‌های ژنتیک (Genetic Algorithms)، هر فرد رشته‌ای بر روی الفبایی متناهی است (اغلب رشتهٔ بولی)، درست همان‌گونه که DNA رشته‌ای بر الفبای ACGT است. در راهبردهای تکاملی (Evolution Strategies)، فرد توالی‌ای از اعداد حقیقی است و در برنامه‌نویسی ژنتیک (Genetic Programming) فرد، برنامهٔ رایانه‌ای است.
- شمار آمیختگی، ρ، که شمار والدهایی است که برای تشکیل فرزند کنار هم می‌آیند. رایج‌ترین حالت $\rho = 2$ است: دو والد «ژن‌های»شان (بخش‌هایی از بازنمایی‌شان) را ترکیب می‌کنند تا فرزند بسازند. وقتی $\rho = 1$ جستجوی پرتوی تصادفی داریم (که می‌توان آن را تولیدمثل غیرجنسی دید). $\rho > 2$ نیز ممکن است که در طبیعت به‌ندرت رخ می‌دهد اما شبیه‌سازی‌اش روی رایانه آسان است.
- فرایند انتخاب (Selection) برای برگزیدن افرادی که والدهای نسل بعد می‌شوند: یک امکان آن است که از میان همهٔ افراد با احتمالی متناسب با امتیاز برازش انتخاب کنیم. امکان دیگر انتخاب تصادفی n فرد ($n > \rho$) و سپس برگزیدن ρ فردِ برازنده‌ترین به‌عنوان والد است.
- رویهٔ بازترکیب. رویکرد رایج (با فرض $\rho = 2$) آن است که به‌صورت تصادفی «نقطهٔ تلاقی» (Crossover Point) را برای شکافتن هر یک از رشته‌های والد برگزینیم و بخش‌ها را بازترکیب کنیم تا دو فرزند ساخته شود: یکی با بخش اول والد ۱ و بخش دوم والد ۲؛ دیگری با بخش دوم والد ۱ و بخش اول والد ۲.
- نرخ جهش (Mutation Rate) که تعیین می‌کند فرزندان هر چند وقت یک بار جهش‌های تصادفی در بازنمایی‌شان دارند. پس از تولید فرزند، هر بیت در ترکیبش با احتمالی برابر نرخ جهش وارونه می‌شود.

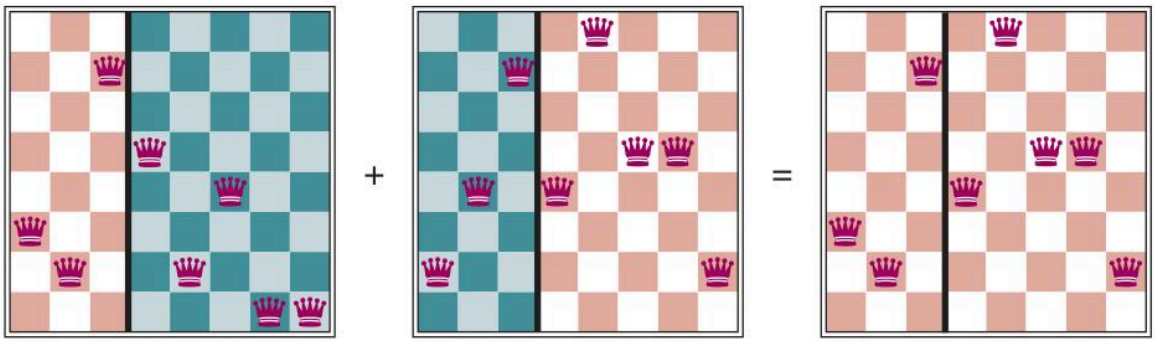
- ترکیب نسل بعد. می‌تواند فقط فرزندانِ تازه‌تشکیل‌شده باشد، یا می‌تواند چند والدِ بالاترین‌امتیازِ نسل قبل را هم شامل شود (رویه‌ای به نام نخبه‌گرایی (Elitism) که تضمین می‌کند برازش کلی هرگز در طول زمان کاهش نیابد). رویهٔ حذف (Culling) — دورریختن همهٔ افراد زیر آستانه‌ای معین — می‌تواند به شتاب‌بخشی بینجامد (بام و همکاران، ۱۹۹۵).



شکل ۴.۶ — الگوریتم ژنتیک، نشان‌داده‌شده برای رشته‌های رقمیِ بازنمایندهٔ حالت‌های هشت‌وزیر. جمعیت آغازین در (الف) با تابع برازش در (ب) رتبه‌بندی می‌شود و جفت‌هایی برای جفت‌گیری در (ج) نتیجه می‌دهد. آن‌ها در (د) فرزندان تولید می‌کنند که در (هـ) در معرض جهش‌اند. (ارقام نمونه: 32752411، 24748552 و... با درصدهای برازش 29٪، 31٪، 26٪، 14٪)

شکل ۴.۶(الف) جمعیتی از چهار رشتهٔ ۸رقمی را نشان می‌دهد که هر یک حالتی از معمای هشت‌وزیر را بازنمایی می‌کند: رقم c ام، شمار سطر وزیر در ستون c را نشان می‌دهد. در (ب)، هر حالت با تابع برازش رتبه‌بندی می‌شود. مقادیر برازش بالاتر بهترند، پس برای مسئلهٔ هشت‌وزیر از شمار جفت‌وزیرهای غیرحمله‌کننده استفاده می‌کنیم که برای راه‌حل مقدار 8×7/2 = 28 دارد. مقادیر چهار حالت در (ب) عبارت‌اند از ۲۴، ۲۳، ۲۰ و ۱۱. سپس امتیازهای برازش به احتمال‌ها نرمال می‌شوند و مقادیر حاصل کنار مقادیر برازش در (ب) نشان داده شده‌اند. در (ج)، دو جفت والد مطابق احتمال‌های (ب) برگزیده می‌شوند. توجه کنید یک فرد دو بار برگزیده شده و یکی اصلاً برگزیده نشده. برای هر جفت برگزیده، نقطهٔ تلاقی (خط نقطه‌چین) تصادفی انتخاب می‌شود. در (د)، رشته‌های والد را در نقطه‌های تلاقی تلاقی می‌دهیم و فرزندان تازه به دست می‌آیند. برای نمونه، نخستین فرزندِ نخستین جفت، سه رقم اول (۳۲۷) را از والد اول و رقم‌های باقی‌مانده (۴۸۵۵۲) را از والد دوم می‌گیرد.

حالت‌های هشت‌وزیرِ درگیر در این گام بازترکیب در شکل ۴.۷ نشان داده شده‌اند.



شکل ۴.۷ – حالت‌های هشت‌وزیر متناظر با دو والد نخست شکل ۴.۶(ج) و نخستین فرزند شکل ۴.۶(د). ستون‌های سبز در گام تلاقی از دست می‌روند و ستون‌های قرمز حفظ می‌شوند. (برای تفسیر اعداد شکل ۴.۶: سطر ۱ پایین‌ترین سطر و ۸ بالاترین سطر است.)

سرانجام، در (هـ)، هر مکان در هر رشته با احتمال کوچکِ مستقلی در معرض جهش تصادفی است. یک رقم در فرزندان اول، سوم و چهارم جهش یافت. در مسئلهٔ هشت‌وزیر، این با برگزیدن تصادفی وزیری و حرکت‌دادنش به خانه‌ای تصادفی در ستونش متناظر است. اغلب چنین است که جمعیت در آغاز فرایند متنوع است، پس تلاقی در آغاز جستجو گام‌های بزرگِ مکرری در فضای حالت برمی‌دارد (مانند بازپخت شبیه‌سازی‌شده). پس از نسل‌های فراوان انتخاب به سوی برازش بالاتر، جمعیت متنوع‌تر می‌شود و گام‌های کوچک‌تر معمول‌ترند. شکل ۴.۸ الگوریتمی را توصیف می‌کند که همهٔ این گام‌ها را پیاده می‌کند.

```
function GENETIC-ALGORITHM(population, fitness) returns an individual
                                                                repeat
                                    weights ← WEIGHTED-BY(population, fitness)
                                                 population2 ← an empty list
                                               for i = 1 to SIZE(population) do
        parent1, parent2 ← WEIGHTED-RANDOM-CHOICES(population, weights, 2)
                                             child ← REPRODUCE(parent1, parent2)
                  if (a small random probability) then child ← MUTATE(child)
                                                  add child to population2
                                                  population ← population2
            until an individual is sufficiently fit, or enough time has passed
                return the best individual in population, according to fitness
```

شکل ۴.۸ – الگوریتم ژنتیک. درون تابع، population فهرست مرتبی از افراد است، weights فهرستی از مقادیر برازش متناظر هر فرد، و fitness تابعی برای محاسبهٔ این مقادیر.

الگوریتم‌های ژنتیک شبیه جستجوی پرتوی تصادفی‌اند، اما با افزوده‌شدن عملیات تلاقی. این اگر بلوک‌هایی باشند که کارکردهای مفید انجام می‌دهند سودمند است. برای نمونه، ممکن است گذاشتن سه وزیر اول در موقعیت‌های ۲، ۴ و ۶ (جایی که به هم حمله نمی‌کنند) بلوک مفیدی بسازد که بتوان با دیگر بلوک‌های مفیدِ حاضر در افراد دیگر ترکیبش کرد تا راه‌حل ساخته شود. به‌شکل ریاضی می‌توان نشان داد اگر بلوک‌ها هدفی نداشته باشند — برای نمونه اگر موقعیت‌های کد ژنتیک تصادفی درهم‌آمیخته شده باشند — تلاقی هیچ برتری‌ای منتقل نمی‌کند.

نظریهٔ الگوریتم‌های ژنتیک با ایدهٔ «شِما» (Schema) توضیح می‌دهد این چگونه کار می‌کند؛ شِما زیررشته‌ای است که در آن برخی موقعیت‌ها را می‌توان نامشخص گذاشت. برای نمونه، شِما `246*****` همهٔ حالت‌های هشت‌وزیری را توصیف می‌کند که سه وزیر اول در موقعیت‌های ۲، ۴ و ۶ هستند. رشته‌هایی که با شِما همخوان‌اند (مانند 24613578) «مصداق»های (Instances) شِما نامیده می‌شوند. می‌توان نشان داد اگر میانگین برازش مصداق‌های شِما بالاتر از میانگین باشد، شمار مصداق‌های شِما در طول زمان رشد خواهد کرد. بدیهی است این اثر اگر بیت‌های مجاور کاملاً بی‌ارتباط با هم باشند بعید است چشمگیر باشد، زیرا بلوک‌های پیوستهٔ کمی خواهند بود که منفعت سازگاری می‌دهند. الگوریتم‌های ژنتیک وقتی بهترین عملکرد را دارند که شِماها با مؤلفه‌های معنادار راه‌حل متناظر باشند. برای نمونه، اگر رشته بازنمایی آنتن باشد، شِماها ممکن است مؤلفه‌های آنتن مانند بازتابنده‌ها و منحرف‌کننده‌ها را بازنمایی کنند. مؤلفهٔ خوب به احتمال زیاد در گستره‌ای از طراحی‌های گوناگون خوب است. این پیشنهاد می‌کند استفادهٔ موفق از الگوریتم‌های ژنتیک مستلزم مهندسی دقیق بازنمایی است.

در عمل، الگوریتم‌های ژنتیک جای خود را در چشم‌انداز گستردهٔ روش‌های بهینه‌سازی دارند (مارلر و آرورا، ۲۰۰۴)، به‌ویژه برای مسائل ساختاریافتهٔ پیچیده مانند چیدمان مدار یا زمان‌بندی کارگاه، و اخیراً برای تکامل معماری شبکه‌های عصبی عمیق (میکولاینن و همکاران، ۲۰۱۹). روشن نیست چقدر از جذابیت الگوریتم‌های ژنتیک از برتری‌شان در تکلیف‌های خاص ناشی می‌شود و چقدر از استعارهٔ جذاب تکامل.

جعبه: تکامل و جستجو

نظریهٔ تکامل توسط چارلز داروین در «خاستگاه گونه‌ها از راه انتخاب طبیعی» (۱۸۵۹) و مستقل از او توسط آلفرد راسل والاس (۱۸۵۸) توسعه یافت. ایدهٔ مرکزی ساده است: تغییرها در تولیدمثل رخ می‌دهند و در نسل‌های پیاپی تقریباً به‌تناسب اثرشان بر برازش تولیدمثلی حفظ می‌شوند. نظریهٔ داروین بدون هیچ دانشی از چگونگی وراثت و اصلاح ویژگی‌های جانداران توسعه یافت. قوانین احتمالاتی حاکم بر این فرایندها نخستین بار توسط گرگور مندل (۱۸۶۶) — راهبی که با نخودفرنگی آزمایش کرد — شناسایی شد. بسیار بعدتر، واتسون و کریک (۱۹۵۳) ساختار مولکول DNA و الفبای آن، AGTC (آدنین، گوانین، تیمین، سیتوزین)، را شناسایی کردند. در الگوی استاندارد، تغییر هم با جهش‌های نقطه‌ای در توالی حروف و هم با «تلاقی» (که در آن DNA فرزند با ترکیب بخش‌های بلند DNA هر والد تولید می‌شود) رخ می‌دهد.

قیاس با الگوریتم‌های جستجوی محلی پیش‌تر توصیف شد؛ تفاوت اصلی میان جستجوی پرتوی تصادفی و تکامل، استفاده از تولیدمثل جنسی است که در آن جانشین‌ها از چند فرد تولید می‌شوند نه فقط یک فرد. سازوکارهای واقعی تکامل اما بسیار غنی‌تر از آن‌اند که بیشتر الگوریتم‌های ژنتیک اجازه می‌دهند. برای نمونه، جهش‌ها می‌توانند شامل وارونگی، تکرار و جابه‌جایی تکه‌های بزرگ DNA باشند؛ برخی ویروس‌ها DNA را از یک جاندار قرض می‌گیرند و در جاندار دیگر وارد می‌کنند؛ و ژن‌های جابه‌جاشونده‌ای هست که هیچ کاری نمی‌کنند جز هزاران بار خود را درون ژنوم کپی کردن. حتی ژن‌هایی هست که یاخته‌های هم‌سری که آن ژن را حمل نمی‌کنند مسموم می‌کنند و بدین ترتیب شانس تکثیر خودشان را افزایش می‌دهند. مهم‌ترین واقعیت آن است که خودِ ژن‌ها سازوکارهایی را رمزگذاری می‌کنند که ژنوم به‌واسطهٔ آن‌ها بازتولید و به جاندار ترجمه می‌شود. در الگوریتم‌های ژنتیک، آن سازوکارها برنامهٔ جدایی هستند که درون رشته‌های دست‌کاری‌شده بازنمایی نشده‌اند.

تکامل داروینی ممکن است ناکارآمد بنماید، زیرا کورکورانه حدود 10^{43} جاندار تولید کرده بدون آنکه اکتشافی‌های جستجویش را ذره‌ای بهبود دهد. اما یادگیری در تکامل نقش دارد. هرچند طبیعت‌دان بزرگ فرانسوی، ژان لامارک (۱۸۰۹)، در این پیشنهاد که ویژگی‌های اکتسابیِ سازگاری در طول زندگی جاندار به فرزندانش منتقل می‌شوند نادرست بود، نظریهٔ ظاهراً مشابه جیمز بالدوین (۱۸۹۶) درست است: یادگیری می‌تواند چشم‌انداز برازش را به‌شکل مؤثری نرم کند و به شتاب در نرخ تکامل بینجامد. جاندارِ دارای ویژگی‌ای که کاملاً برای محیطش سازگار نیست، آن ویژگی را منتقل می‌کند اگر به اندازهٔ کافی انعطاف‌پذیری (Plasticity) هم داشته باشد تا یاد بگیرد به‌شیوه‌ای سودمند با محیط سازگار شود. شبیه‌سازی‌های رایانه‌ای (هینتون و نولان، ۱۹۸۷) تأیید می‌کنند این «اثر بالدوین» واقعی است و پیامدی که دارد آن است که چیزهای دشوار برای یادگیری سرانجام در ژنوم می‌نشینند، اما چیزهای آسان برای یادگیری لازم نیست آن‌جا باشند (مورگان و گریفیتس، ۲۰۱۵).

۴.۲ جستجوی محلی در فضاهای پیوسته

در فصل ۲ تمایز میان محیط‌های گسسته و پیوسته را توضیح دادیم و اشاره کردیم بیشتر محیط‌های جهان واقعی پیوسته‌اند. فضای کنش پیوسته ضریب انشعاب بی‌نهایت دارد و بنابراین با بیشتر الگوریتم‌هایی که تاکنون پوشش داده‌ایم (به‌جز تپه‌نوردی انتخاب-اول و بازپخت شبیه‌سازی‌شده) قابل مدیریت نیست. این بخش مقدمهٔ بسیار کوتاهی بر برخی فنون جستجوی محلی برای فضاهای پیوسته است. ادبیات این موضوع گسترده است؛ بسیاری از فنون پایه از سدهٔ هفدهم، پس از توسعهٔ حساب دیفرانسیل توسط نیوتن و لایب‌نیتس، سرچشمه گرفته‌اند.[۲] در چند نقطه از این کتاب از این فنون استفاده می‌کنیم، از جمله فصل‌های یادگیری، بینایی و رباتیک.

با مثالی آغاز می‌کنیم. فرض کنیم می‌خواهیم سه فرودگاه تازه را هر جایی در رومانی بگذاریم، چنان‌که مجموع مجذور فاصله‌های خط مستقیم از هر شهر روی نقشه تا نزدیک‌ترین فرودگاهش کمینه شود. (برای نقشهٔ رومانی شکل ۳.۱ را ببینید.) فضای حالت پس با مختصات سه فرودگاه تعریف می‌شود: (x1,y1)، (x2,y2) و (x3,y3). این فضایی شش‌بعدی است؛ همچنین می‌گوییم حالت‌ها با شش متغیر (Variables) تعریف می‌شوند. در حالت کلی، حالت‌ها با بردار n-بعدی از متغیرها، x، تعریف می‌شوند. حرکت در این فضا با حرکت یک یا چند فرودگاه روی نقشه متناظر است. تابع هدف f(x) = f(x1,y1,x2,y2,x3,y3) برای هر حالت خاص نسبتاً آسان محاسبه می‌شود، به‌محض آنکه نزدیک‌ترین شهرها را محاسبه کنیم. فرض کنیم Ci مجموعهٔ شهرهایی باشد که نزدیک‌ترین فرودگاهشان (در حالت x) فرودگاه i است. آنگاه داریم:

$$f(x) = f(x1, y1, x2, y2, x3, y3) = \sum_i \sum_{c \in Ci} [(xi - xc)^2 + (yi - yc)^2] \quad (4.1)$$

این معادله نه فقط برای حالت x، بلکه برای حالت‌های همسایگی محلی x نیز درست است. در بسیاری موارد اما این معادله به‌صورت بسته قابل حل نیست. برای نمونه، با سه فرودگاه، عبارت گرادیان به این بستگی دارد که کدام شهرها در حالت کنونی به هر فرودگاه نزدیک‌ترند. این بدان معناست که می‌توانیم گرادیان را به‌صورت محلی (نه سراسری) محاسبه کنیم؛ برای نمونه:

```
∂f/∂x1 = 2 ∑_{c∈C1} (x1 − xc)    (4.2)
```

با داشتن عبارتِ به‌صورت محلی درستِ گرادیان، می‌توانیم تپه‌نوردی صعود تندترین را با به‌روزرسانی حالت کنونی طبق فرمول انجام دهیم:

```
x ← x + α ∇f(x)
```

که α (آلفا) ثابت کوچکی است که اغلب «اندازهٔ گام» (Step Size) نامیده می‌شود. روش‌های بسیار متنوعی برای تنظیم α هست. مسئلهٔ پایه آن است که اگر α خیلی کوچک باشد، گام‌های زیادی لازم است؛ اگر خیلی بزرگ باشد، جستجو می‌تواند از بیشینه عبور کند. فن «جستجوی خطی» (Line Search) می‌کوشد این دوراهه را با امتداد دادن جهت گرادیان کنونی — معمولاً با دوبل‌کردن مکرر α — حل کند تا آنکه f دوباره شروع به کاهش کند. نقطه‌ای که این رخ می‌دهد حالت کنونیِ تازه می‌شود. چند مکتب فکری دربارهٔ چگونگی انتخاب جهت تازه در این نقطه هست.

برای بسیاری از مسائل، مؤثرترین الگوریتم، روش کهن نیوتن–رافسون (Newton–Raphson) است. این فن عمومی برای یافتن ریشه‌های توابع است — یعنی حل معادلات به شکل g(x)=0. با محاسبهٔ تخمین تازه‌ای برای ریشهٔ x طبق فرمول نیوتن کار می‌کند:

```
x ← x − g(x) / g′(x)
```

برای یافتن بیشینه یا کمینهٔ f، باید x را چنان بیابیم که گرادیان بردار صفر باشد (یعنی ∇f(x)=0). پس g(x) در فرمول نیوتن به ∇f(x) تبدیل می‌شود و معادلهٔ به‌روزرسانی را می‌توان به‌صورت ماتریس–بردار نوشت:

```
x ← x − Hf⁻¹(x) ∇f(x)
```

که Hf(x) «ماتریس هسیان» (Hessian) مشتق‌های دوم است که درایه‌هایش Hij = ∂²f/∂xi∂xj داده می‌شوند. برای مثال فرودگاه‌های ما، از معادلهٔ (۴.۲) می‌بینیم Hf(x) به‌ویژه ساده است: درایه‌های غیرقطری صفرند و درایه‌های قطری برای فرودگاه i فقط دو برابر شمار شهرهای Ci هستند. محاسبه‌ای کوتاه نشان می‌دهد یک گام از به‌روزرسانی، فرودگاه i را مستقیماً به مرکز ثقل Ci می‌برد که کمینهٔ عبارت محلی f از معادلهٔ (۴.۱) است.[3] اما برای مسائل پرابعاد، محاسبهٔ n² درایهٔ هسیان و وارون‌کردنش ممکن است پرهزینه باشد، پس نسخه‌های تقریبی فراوانی از روش نیوتن–رافسون توسعه یافته.

روش‌های جستجوی محلی در فضاهای حالت پیوسته به همان اندازهٔ فضاهای گسسته از بیشینه‌های محلی، خط‌الرأس‌ها و فلات‌ها رنج می‌برند. راه‌اندازی مجدد تصادفی و بازپخت شبیه‌سازی‌شده اغلب کمک‌کننده‌اند. فضاهای پیوستهٔ پرابعاد اما مکان‌های بزرگی هستند که گم‌شدن در آن‌ها بسیار آسان است.

موضوع پایانی «بهینه‌سازی مقید» (Constrained Optimization) است. مسئلهٔ بهینه‌سازی مقید است اگر راه‌حل‌ها باید قیدهای سختی بر مقادیر متغیرها برآورده کنند. برای نمونه، در مسئلهٔ مکان‌یابی فرودگاه، ممکن است قید کنیم مکان‌ها داخل رومانی و روی خشکی باشند (نه وسط دریاچه‌ها). دشواری مسائل بهینه‌سازی مقید به سرشت قیدها و تابع هدف بستگی دارد. شناخته‌شده‌ترین رده، مسائل برنامه‌ریزی خطی (Linear Programming) است که در آن قیدها باید نابرابری‌های خطیِ سازندهٔ مجموعهٔ محدب[4] باشند و تابع هدف نیز خطی. پیچیدگی زمانی برنامه‌ریزی خطی در شمار متغیرها چندجمله‌ای است.

جعبه: مجموعهٔ S از نقطه‌ها محدب (Convex) است اگر خطِ واصل هر دو نقطه در S نیز در S باشد. تابع محدب تابعی است که فضای «بالای» آن مجموعهٔ محدبی بسازد؛ طبق تعریف، توابع محدب هیچ کمینهٔ محلی (در برابر سراسری) ندارند.

برنامه‌ریزی خطی احتمالاً پرکاربردترین و به‌طور گسترده‌ای مطالعه‌شده‌ترین روش بهینه‌سازی است. حالت خاصی از مسئلهٔ عمومی‌تر «بهینه‌سازی محدب» (Convex Optimization) است که به ناحیهٔ قید اجازه می‌دهد هر ناحیهٔ محدبی باشد و تابع هدف هر تابعی که درون ناحیهٔ قید محدب است. تحت شرایطی، مسائل بهینه‌سازی محدب نیز به‌صورت چندجمله‌ای حل‌شدنی‌اند و ممکن است در عمل با هزاران متغیر عملی باشند. چند مسئلهٔ مهم در یادگیری ماشین و نظریهٔ کنترل را می‌توان به‌صورت مسائل بهینه‌سازی محدب صورت‌بندی کرد (فصل ۲۰ را ببینید).

۴.۳ جستجو با کنش‌های ناجبرگرایانه

در فصل ۳ محیط کاملاً قابل مشاهده، جبرگرایانه و شناخته‌شده را فرض کردیم. پس عامل می‌تواند حالت آغازین را مشاهده کند، توالی کنشی را که به هدف می‌رسد محاسبه کند و کنش‌ها را با «چشم‌های بسته» اجرا کند و هرگز مجبور نباشد از ادراک‌های حسی‌اش استفاده کند.

اما وقتی محیط تا حدی قابل مشاهده باشد، عامل به‌طور قطعی نمی‌داند در کدام حالت است؛ و وقتی محیط ناجبرگرایانه باشد، عامل نمی‌داند پس از انجام کنش به کدام حالت گذار می‌کند. این بدان معناست که به‌جای اندیشیدن «من در حالت s1 هستم و اگر کنش a را انجام دهم به حالت s2 می‌رسم»، عامل اکنون چنین می‌اندیشد: «من یا در حالت s1 هستم یا s3، و اگر کنش a را انجام دهم به حالت s2، s4 یا s5 می‌رسم.» مجموعهٔ حالت‌های فیزیکی را که عامل باور دارد ممکن‌اند، «حالت باور» (Belief State) می‌نامیم.

در محیط‌های تا حدی قابل مشاهده و ناجبرگرایانه، راه‌حل مسئله دیگر توالی نیست، بلکه «برنامهٔ شرطی» (Conditional Plan) است (که گاهی برنامهٔ احتیاطی (Contingency Plan) یا راهبرد نامیده می‌شود) و مشخص می‌کند بسته به ادراک‌هایی که عامل هنگام اجرای برنامه دریافت می‌کند چه کند. در این بخش ناجبرگرایی و در بخش بعد مشاهده‌ناپذیری تا حدی را بررسی می‌کنیم.

۴.۳.۱ جهان جاروبرقی ناهنجار

جهان جاروبرقی فصل ۲ هشت حالت دارد، چنان‌که در شکل ۴.۹. سه کنش هست — Left، Right و Suck — و هدف تمیزکردن همهٔ کثیفی‌هاست (حالت‌های ۷ و ۸). اگر محیط کاملاً قابل مشاهده، جبرگرایانه و کاملاً شناخته‌شده باشد، مسئله با هر یک از الگوریتم‌های فصل ۳ به‌راحتی حل می‌شود و راه‌حل توالی کنشی است. برای نمونه، اگر حالت آغازین ۱ باشد، توالی کنش [Suck, Right, Suck] به حالت هدف ۸ می‌رسد.

حال فرض کنیم ناجبرگرایی را در قالب جاروبرقیِ قدرتمند اما ناهنجار (Erratic) معرفی کنیم. در جهان جاروبرقی ناهنجار، کنش Suck چنین کار می‌کند:

- وقتی بر خانهٔ کثیف اعمال شود، خانه را تمیز می‌کند و گاهی کثیفی خانهٔ مجاور را نیز تمیز می‌کند.
- وقتی بر خانهٔ تمیز اعمال شود، گاهی کثیفی روی فرش می‌گذارد.[5]

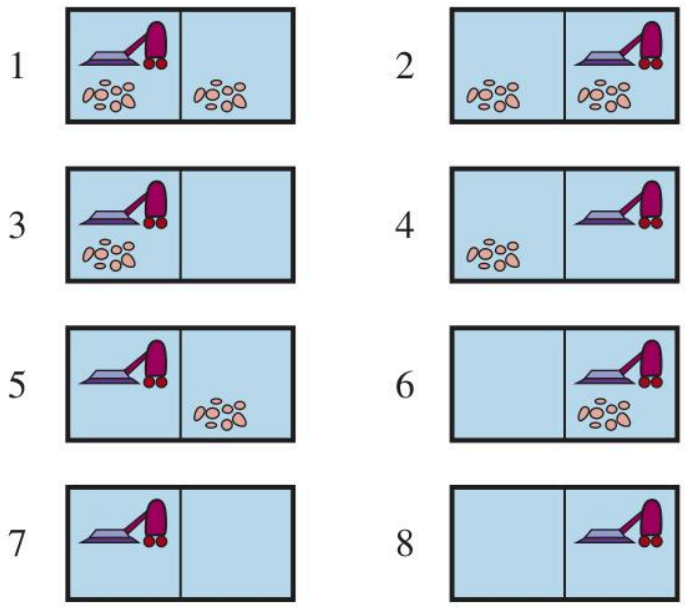
برای ارائهٔ صورت‌بندی دقیق این مسئله، باید مفهوم مدل گذار فصل ۳ را تعمیم دهیم. به‌جای تعریف مدل گذار با تابع RESULT که یک حالت پیامدی بازمی‌گرداند، از تابع RESULTS استفاده می‌کنیم که مجموعه‌ای از حالت‌های پیامدیِ ممکن بازمی‌گرداند. برای نمونه، در جهان جاروبرقی ناهنجار، کنش Suck در حالت ۱ یا فقط مکان کنونی را تمیز می‌کند یا هر دو مکان را:

```
RESULTS(1, Suck) = {5, 7}
```

اگر از حالت ۱ آغاز کنیم، هیچ توالی کنشیِ تکی مسئله را حل نمی‌کند، اما این برنامهٔ شرطی حل می‌کند:

```
[[] Suck, if State=5 then [Right, Suck] else]    (4.3)
```

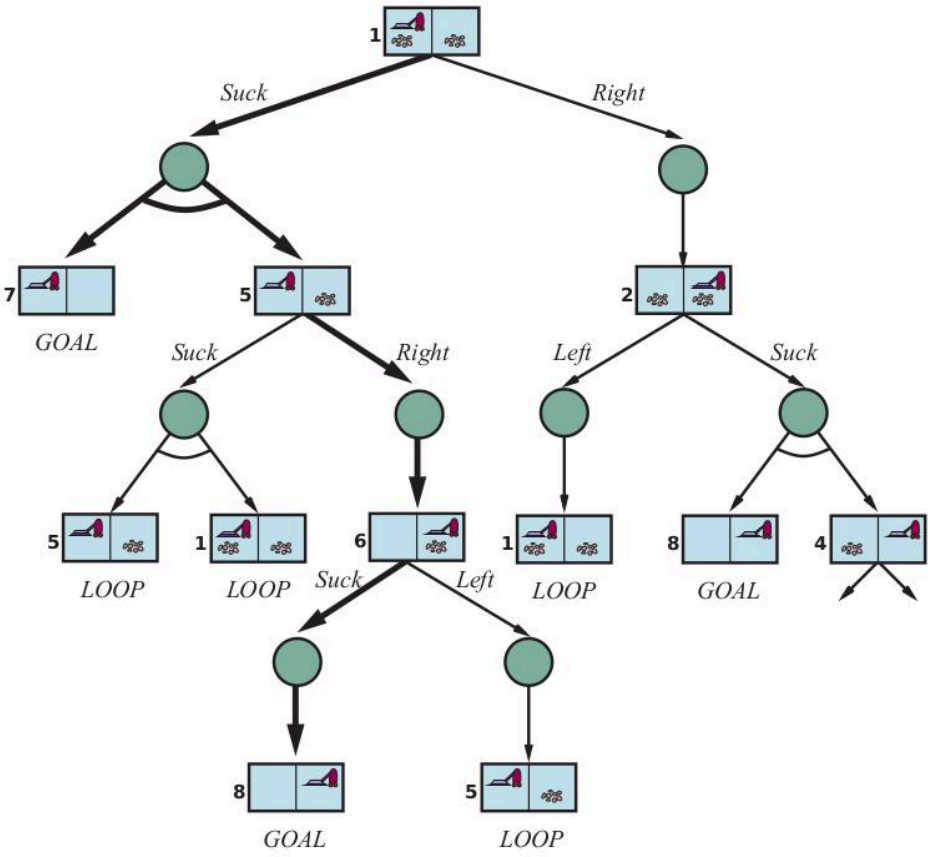
در این‌جا می‌بینیم برنامهٔ شرطی می‌تواند گام‌های if-then-else داشته باشد؛ این بدان معناست راه‌حل‌ها درخت‌اند نه توالی. در این‌جا شرط در if آزمایش می‌کند حالت کنونی چیست؛ این چیزی است که عامل هنگام اجرا قادر به مشاهده‌اش خواهد بود، اما هنگام برنامه‌ریزی نمی‌داند. از سوی دیگر، می‌توانستیم صورت‌بندی‌ای داشته باشیم که ادراک حسی را آزمایش کند نه حالت را. بسیاری از مسائل جهان فیزیکیِ واقعی مسائل احتیاطی‌اند، زیرا پیش‌بینی دقیق آینده ناممکن است. به همین دلیل، بسیاری از مردم هنگام راه‌رفتن چشم‌هایشان را باز نگه می‌دارند.



شکل ۴.۹ – هشت حالت ممکن جهان جاروبرقی؛ حالت‌های ۷ و ۸ حالت‌های هدف‌اند.

۴.۳.۲ درخت‌های جستجوی AND–OR

چگونه این راه‌حل‌های شرطی را برای مسائل ناجبرگرایانه می‌یابیم؟ مانند فصل ۳، با ساختن درخت‌های جستجو آغاز می‌کنیم، اما در این‌جا درخت‌ها خصلت متفاوتی دارند. در محیط جبرگرایانه، تنها انشعاب توسط انتخاب‌های خودِ عامل در هر حالت معرفی می‌شود: می‌توانم این کنش یا آن کنش را انجام دهم. این گره‌ها را گره‌های OR (Or Node) می‌نامیم. در جهان جاروبرقی، برای نمونه، در گرهٔ OR عامل Left یا Right یا Suck را برمی‌گزیند. در محیط ناجبرگرایانه، انشعاب همچنین توسط انتخابِ محیط از پیامدِ هر کنش معرفی می‌شود. این گره‌ها را گره‌های AND (And Node) می‌نامیم. برای نمونه، کنش Suck در حالت ۱ به حالت باور {5,7} می‌انجامد، پس عامل باید برنامه‌ای برای حالت ۵ و حالتی برای ۷ بیابد. این دو نوع گره در هم می‌آیند و به «درخت (AND–OR Tree) «AND–OR می‌انجامند، چنان‌که در شکل ۴.۱۰ نشان داده شده.



شکل ۴.۱۰ – دو سطح نخست درخت جستجو برای جهان جاروبرقی ناهنجار. گره‌های حالت گره‌های ORاند که در آن‌ها کنشی باید برگزیده شود. در گره‌های AND، که به‌صورت دایره نشان داده شده‌اند، باید با هر پیامدی برخورد شود، چنان‌که کمان پیونددهندهٔ شاخه‌های خروجی نشان می‌دهد. راه‌حلِ یافته‌شده با خط‌های ضخیم نشان داده شده.

راه‌حل مسئلهٔ جستجوی AND–OR، زیردرختی از درخت جستجوی کامل است که (۱) در هر برگ، گرهٔ هدف داشته باشد، (۲) در هر یک از گره‌های OR خود یک کنش مشخص کند، و (۳) در هر یک از گره‌های AND خود هر شاخهٔ پیامدی را در بر بگیرد. راه‌حل با خط‌های ضخیم در شکل نشان داده شده؛ با برنامهٔ معادلهٔ (۴.۳) متناظر است.

شکل ۴.۱۱ الگوریتم بازگشتیِ عمق-اولی برای جستجوی گراف AND–OR می‌دهد. یکی از جنبه‌های کلیدی الگوریتم، شیوهٔ برخوردش با چرخه‌هاست که اغلب در مسائل ناجبرگرایانه رخ می‌دهند (مثلاً اگر کنشی گاهی اثری نداشته باشد یا اگر اثری ناخواسته قابل اصلاح باشد). اگر حالت کنونی با حالتی در مسیر از ریشه یکسان باشد، با شکست بازمی‌گردد. این به این معنا نیست که از حالت کنونی راه‌حلی نیست؛ صرفاً بدان معناست که اگر راه‌حل غیرچرخه‌ای باشد، باید از تجسمِ پیشینِ حالت کنونی قابل دسترسی باشد، پس تجسم تازه را می‌توان دور ریخت. با این بررسی، اطمینان حاصل می‌کنیم الگوریتم در هر فضای حالت متناهی پایان می‌یابد، زیرا هر مسیر باید به هدف، بن‌بست یا حالت تکراری برسد. توجه کنید الگوریتم بررسی نمی‌کند حالت کنونی تکرار حالتی در مسیر دیگری از ریشه است یا نه، که برای کارایی مهم است.

```
function AND-OR-SEARCH(problem) returns a conditional plan, or failure
    return OR-SEARCH(problem, problem.INITIAL, [])

function OR-SEARCH(problem, state, path) returns a conditional plan, or failure
    if problem.IS-GOAL(state) then return empty plan
    if IS-CYCLE(path) then return failure
    for each action in problem.ACTIONS(state) do
        plan ← AND-SEARCH(problem, RESULTS(state, action), [state] + path)
        if plan ≠ failure then return [action] + plan
    return failure

function AND-SEARCH(problem, states, path) returns a conditional plan, or failure
    for each si in states do
        plani ← OR-SEARCH(problem, si, path)
        if plani = failure then return failure
    return [if s1 then plan1 else if s2 then plan2 else ... if sn−1 then plann−1 else plann]
```

شکل ۴.۱۱ – الگوریتم جستجوی گراف‌های AND–OR تولیدشده توسط محیط‌های ناجبرگرایانه. راه‌حل، برنامهٔ شرطی‌ای است که هر پیامد ناجبرگرایانه را در نظر می‌گیرد و برای هر یک برنامه می‌سازد.

گراف‌های AND–OR را می‌توان سطح-اول یا بهترین-اول کاوش کرد. مفهوم تابع اکتشافی باید اصلاح شود تا هزینهٔ راه‌حل شرطی را تخمین بزند نه توالی را، اما مفهوم مقبولیت منتقل می‌شود و هم‌ارز الگوریتم A* برای یافتن راه‌حل‌های بهینه هست. (یادداشت‌های کتاب‌شناختی پایان فصل را ببینید.)

۴.۳.۳ تلاش دوباره و دوباره

جهان جاروبرقی «لغزنده» (Slippery) را در نظر بگیرید که با جهان جاروبرقی معمولی (غیرناهنجار) یکسان است، به‌جز آنکه کنش‌های حرکتی گاهی شکست می‌خورند و عامل در همان مکان می‌ماند. برای نمونه، حرکت Right در حالت ۱ به حالت باور {1,2} می‌انجامد. شکل ۴.۱۲ بخشی از گراف جستجو را نشان می‌دهد؛ بدیهی است دیگر هیچ راه‌حل غیرچرخه‌ای از حالت ۱ نیست و AND-OR-SEARCH با شکست بازمی‌گردد. اما راه‌حل چرخه‌ای (Cyclic Solution) هست: ادامهٔ Right تا وقتی کار کند. می‌توانیم این را با سازهٔ تازهٔ while بیان کنیم:

```
[Suck, while State=5 do Right, Suck]
```

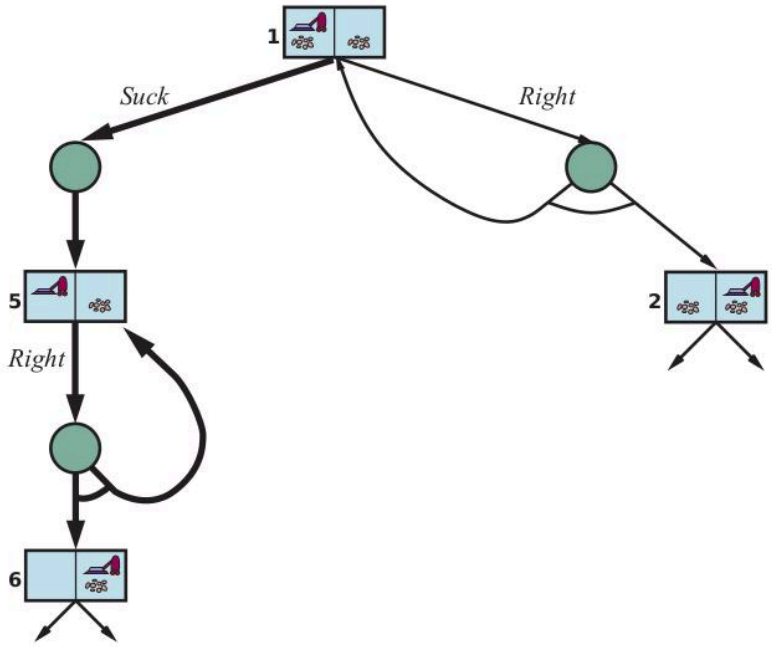
یا با افزودن برچسبی برای نشان دادن بخشی از برنامه و ارجاع بعدی به آن برچسب:

```
[Suck, L1: Right, if State=5 then L1 else Suck]
```

چه‌وقت برنامهٔ چرخه‌ای راه‌حل است؟ شرط کمینه آن است که هر برگ حالت هدف باشد و برگ از هر نقطهٔ برنامه قابل دسترسی باشد. افزون بر آن، باید علت ناجبرگرایی را در نظر بگیریم. اگر واقعاً چنین است که سازوکار محرک ربات جاروبرقی برخی اوقات کار می‌کند، اما در موارد دیگر تصادفی و مستقل لیز می‌خورد، آنگاه عامل می‌تواند مطمئن باشد

اگر کنش به اندازهٔ کافی تکرار شود، سرانجام کار خواهد کرد و برنامه موفق خواهد شد. اما اگر ناجبرگرایی به‌سبب واقعیتی مشاهده‌نشده دربارهٔ ربات یا محیط باشد — شاید تسمهٔ محرک پاره شده و ربات هرگز حرکت نخواهد کرد — تکرار کنش کمکی نخواهد کرد.

یکی از راه‌های فهم این تصمیم آن است که بگوییم صورت‌بندی مسئلهٔ آغازین (کاملاً قابل مشاهده، ناجبرگرایانه) به نفع صورت‌بندی متفاوتی (تا حدی قابل مشاهده، جبرگرایانه) رها می‌شود که در آن شکست برنامهٔ چرخه‌ای به ویژگیِ مشاهده‌نشده‌ای از تسمهٔ محرک نسبت داده می‌شود. در فصل ۱۲ بحث می‌کنیم چگونه تصمیم بگیریم کدام یک از چند امکانِ نامطمئن محتمل‌تر است.



شکل ۴.۱۲ — بخشی از گراف جستجو برای جهان جاروبرقی لغزنده، که در آن (برخی) چرخه‌ها را به‌صراحت نشان داده‌ایم. همهٔ راه‌حل‌های این مسئله برنامه‌های چرخه‌ای‌اند، زیرا هیچ راهی برای حرکت مطمئن نیست.

۴.۴ جستجو در محیط‌های تا حدی قابل مشاهده

اکنون به مسئلهٔ مشاهده‌ناپذیری تا حدی می‌پردازیم، جایی که ادراک‌های حسی عامل برای تعیین دقیق حالت کافی نیستند. این بدان معناست که برخی از کنش‌های عامل در جهت کاهش عدم قطعیت دربارهٔ حالت کنونی خواهند بود.

۴.۴.۱ جستجو بدون مشاهده

وقتی ادراک‌های حسی عامل اصلاً اطلاعاتی فراهم نمی‌کنند، به آنچه «مسئلهٔ بی‌حسگر» (Sensorless Problem) (یا مسئلهٔ سازگار (Conformant Problem)) می‌گوییم می‌رسیم. در آغاز شاید فکر کنید عامل بی‌حسگر هیچ امیدی به حل مسئله ندارد اگر هیچ ایده‌ای از حالت آغازینش نداشته باشد، اما راه‌حل‌های بی‌حسگر به‌شکل شگفت‌آوری رایج و مفیدند، در درجهٔ نخست به‌سبب آنکه به کارکرد درست حسگرها تکیه نمی‌کنند. در سامانه‌های تولیدی، برای نمونه،

روش‌های هوشمندانهٔ بسیاری برای جهت‌دهی درست قطعات از موقعیت آغازین ناشناخته با توالی‌ای از کنش‌ها و بدون هیچ حسی توسعه یافته. گاهی برنامهٔ بی‌حسگر حتی وقتی برنامهٔ شرطی با حس‌کردن در دسترس است بهتر است. برای نمونه، پزشکان اغلب آنتی‌بیوتیک طیف‌وسیع تجویز می‌کنند به‌جای برنامهٔ شرطیِ آزمایش خون، سپس انتظار برای بازگشت نتایج، و سپس تجویز آنتی‌بیوتیک خاص‌تر. برنامهٔ بی‌حسگر در زمان و پول صرفه‌جویی می‌کند و از ریسک وخیم‌شدن عفونت پیش از آماده‌شدن نتایج آزمایش می‌پرهیزد.

نسخهٔ بی‌حسگر جهان جاروبرقی (جبرگرایانه) را در نظر بگیرید. فرض کنیم عامل جغرافیای جهانش را می‌داند، اما مکان خودش یا توزیع کثیفی را نه. در آن صورت، حالت باور آغازینش {1,2,3,4,5,6,7,8} است (شکل ۴.۹ را ببینید). حال اگر عامل به راست حرکت کند، در یکی از حالت‌های {2,4,6,8} خواهد بود — عامل بدون ادراک هیچ‌چیز اطلاعات به دست آورده است! پس از [Right, Suck] عامل همیشه در یکی از حالت‌های {4,8} خواهد بود. سرانجام، پس از [Right, Suck, Left, Suck] عامل تضمین‌شده به حالت هدف ۷ می‌رسد، صرف‌نظر از حالت آغازین. می‌گوییم عامل می‌تواند جهان را به حالت ۷ «وادار» (Coercion) کند.

راه‌حل مسئلهٔ بی‌حسگر توالی کنش است، نه برنامهٔ شرطی (زیرا هیچ ادراکی نیست). اما در فضای حالت‌های باور جستجو می‌کنیم نه حالت‌های فیزیکی.[6] در فضای حالت باور، مسئله کاملاً قابل مشاهده است، زیرا عامل همیشه حالت باور خودش را می‌داند. وانگهی، راه‌حل (اگر باشد) مسئلهٔ بی‌حسگر همیشه توالی کنش است. این به‌سبب آن است که مانند مسائل عادی فصل ۳، ادراک‌های دریافتی پس از هر کنش کاملاً قابل پیش‌بینی‌اند — همیشه خالی‌اند! پس هیچ احتیاطی برای برنامه‌ریزی نیست. این حتی اگر محیط ناجبرگرایانه باشد نیز صادق است.

می‌توانستیم الگوریتم‌های تازه‌ای برای مسائل جستجوی بی‌حسگر معرفی کنیم. اما در عوض، می‌توانیم از الگوریتم‌های موجود فصل ۳ استفاده کنیم اگر مسئلهٔ فیزیکی زیرین را به مسئلهٔ حالت باور تبدیل کنیم که در آن بر حالت‌های باور جستجو می‌کنیم نه حالت‌های فیزیکی. مسئلهٔ اصلی P مؤلفه‌های ActionsP, ResultP و... دارد و مسئلهٔ حالت باور مؤلفه‌های زیر را دارد:

- **حالت‌ها:** فضای حالت باور هر زیرمجموعهٔ ممکن از حالت‌های فیزیکی را در بر دارد. اگر P دارای N حالت باشد، مسئلهٔ حالت باور 2^N حالت باور دارد، هرچند بسیاری از آن‌ها ممکن است از حالت آغازین دست‌نیافتنی باشند.
- **حالت آغازین:** معمولاً حالت باوری شامل همهٔ حالت‌های P، هرچند در برخی موارد عامل دانش بیشتری از این خواهد داشت.
- **کنش‌ها:** این کمی ظریف است. فرض کنیم عامل در حالت باور b={s1,s2} است، اما ActionsP(s1) ≠ ActionsP(s2)؛ پس عامل مطمئن نیست کدام کنش‌ها قانونی‌اند. اگر فرض کنیم کنش‌های غیرقانونی اثری بر محیط ندارند، پس گرفتن اجتماع همهٔ کنش‌های هر یک از حالت‌های فیزیکی در حالت باور کنونی b ایمن است:

```
ACTIONS(b) = ⋃_{s∈b} ACTIONS_P(s)
```

از سوی دیگر، اگر کنش غیرقانونی ممکن است به فاجعه بینجامد، ایمن‌تر است فقط اشتراک را مجاز کنیم، یعنی مجموعهٔ کنش‌های قانونی در همهٔ حالت‌ها. برای جهان جاروبرقی، هر حالت همان کنش‌های قانونی را دارد، پس هر دو روش همان نتیجه را می‌دهند.

- **مدل گذار:** برای کنش‌های جبرگرایانه، حالت باور تازه برای هر یک از حالت‌های ممکنِ کنونی یک حالت نتیجه دارد (هرچند برخی حالت‌های نتیجه ممکن است یکسان باشند):

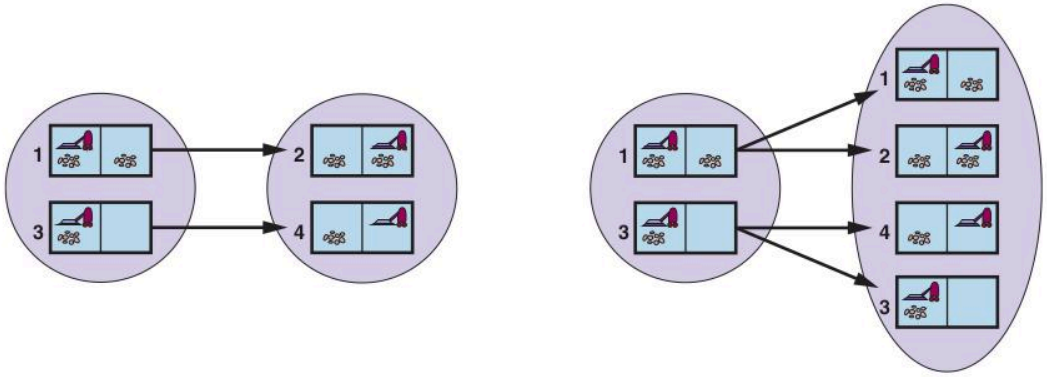
$$\{s \in b \text{ و } s' : s' = \text{RESULT_P}(s, a)\} = b' = \text{RESULT}(b, a) \qquad (4.4)$$

با ناجبرگرایی، حالت باور تازه شامل همهٔ نتایج ممکنِ اعمال کنش به هر یک از حالت‌های حالت باور کنونی است:

$$\text{RESULTS_P}(s, a)\ \{s \in b\}_\bigcup = \{s \in b \text{ و } s' : s' \in \text{RESULTS_P}(s, a)\} = b' = \text{RESULT}(b, a)$$

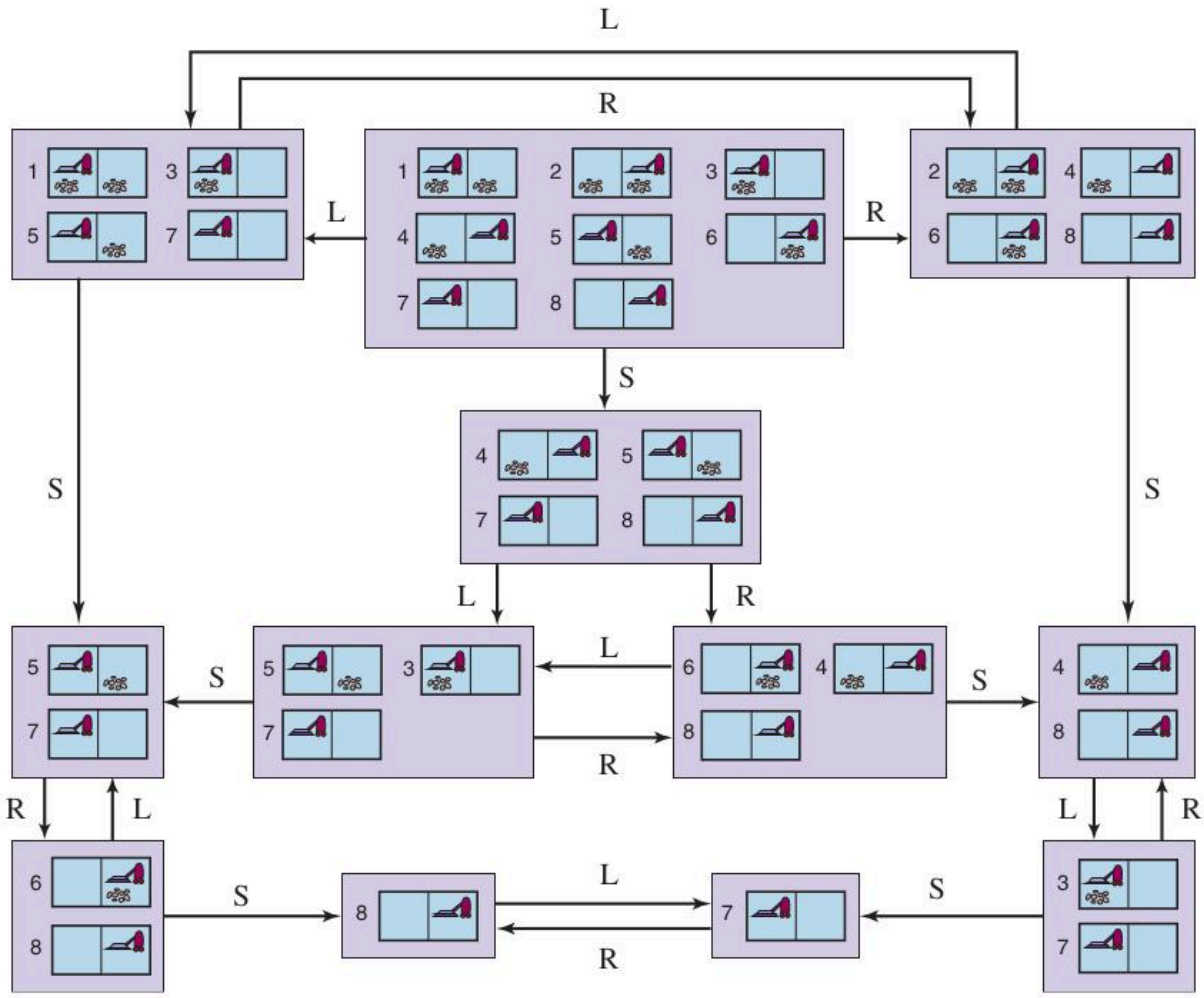
اندازهٔ 'b برای کنش‌های جبرگرایانه همان یا کوچک‌تر از b خواهد بود، اما با کنش‌های ناجبرگرایانه ممکن است بزرگ‌تر از b باشد (شکل ۴.۱۳ را ببینید).

- **آزمون هدف:** عامل «ممکن است» به هدف برسد اگر هر حالت s در حالت باور، آزمون هدفِ مسئلهٔ زیرین، Is-GoalP(s)، را برآورده کند. عامل «لزوماً» به هدف می‌رسد اگر هر حالت آن را برآورده کند. ما به‌دنبال رسیدن لزومی به هدف هستیم.
- **هزینهٔ کنش:** این نیز ظریف است. اگر همان کنش در حالت‌های مختلف هزینه‌های متفاوت داشته باشد، هزینهٔ انجام کنش در حالت باورِ داده‌شده می‌تواند یکی از چند مقدار باشد. (این ردهٔ تازه‌ای از مسائل پدید می‌آورد که در تمرین MVAL.4 بررسی می‌کنیم.) فعلاً فرض می‌کنیم هزینهٔ کنش در همهٔ حالت‌ها یکسان است و بنابراین می‌توان آن را مستقیماً از مسئلهٔ فیزیکی زیرین منتقل کرد.



شکل ۴.۱۳ — (الف) پیش‌بینی حالت باور بعدی برای جهان جاروبرقی بی‌حسگر با کنش جبرگرایانهٔ Right. (ب) پیش‌بینی برای همان حالت باور و کنش در نسخهٔ لغزندهٔ جهان جاروبرقی بی‌حسگر.

شکل ۴.۱۴ فضای حالت باورِ قابل دسترسی را برای جهان جاروبرقی بی‌حسگر جبرگرایانه نشان می‌دهد. از 2^8 = ۲۵۶ حالت باور ممکن، فقط ۱۲ حالت باور قابل دسترسی است. تعریف‌های پیشین ساخت خودکار صورت‌بندی مسئلهٔ حالت باور را از تعریف مسئلهٔ فیزیکی زیرین ممکن می‌کنند. پس از آن، می‌توانیم مسائل بی‌حسگر را با هر یک از الگوریتم‌های جستجوی معمول فصل ۳ حل کنیم.



شکل ۴.۱۴ – بخش قابل دسترسی فضای حالت باور برای جهان جاروبرقی بی‌حسگر جبرگرایانه. هر جعبهٔ مستطیلی با یک حالت باور واحد متناظر است. در هر نقطهٔ معین، عامل حالت باور دارد اما نمی‌داند در کدام حالت فیزیکی است. حالت باور آغازین (نادانی کامل) جعبهٔ بالای وسط است.

در جستجوی گرافی معمول، حالت‌های تازه‌رسیده آزمایش می‌شوند که آیا قبلاً رسیده شده‌اند. این برای حالت‌های باور نیز کار می‌کند؛ برای نمونه، در شکل ۴.۱۴، توالی کنش [Suck, Left, Suck] از حالت آغازین به همان حالت باوری می‌رسد که [Right, Left, Suck] می‌رسد، یعنی {5,7}. حال، حالت باورِ رسیده‌شده توسط [Left]، یعنی {1,3,5,7} را در نظر بگیرید. بدیهی است این با {5,7} یکسان نیست، اما «فوق‌مجموعه» آن است. می‌توانیم هر چنین حالت باور فوق‌مجموعه‌ای را دور بریزیم (هرس کنیم). چرا؟ زیرا راه‌حل از {1,3,5,7} باید برای هر یک از حالت‌های منفرد ۱، ۳، ۵ و ۷ راه‌حل باشد و بنابراین برای هر ترکیبی از این حالت‌های منفرد، مانند {5,7} نیز راه‌حل است؛ پس لازم نیست تلاش کنیم {1,3,5,7} را حل کنیم، می‌توانیم روی حلِ حالت باورِ به‌وضوح آسان‌ترِ {5,7} تمرکز کنیم. برعکس، اگر {1,3,5,7} قبلاً تولید و حل‌شدنی یافته شده باشد، هر زیرمجموعه‌ای مانند {5,7} تضمین‌شده حل‌شدنی است. (اگر راه‌حلی داشته باشم که وقتی بسیار سردرگمم دربارهٔ حالتم کار می‌کند، وقتی کمتر سردرگمم نیز کار خواهد کرد.) این سطح اضافی از هرس می‌تواند کارایی حل مسئلهٔ بی‌حسگر را به‌شکل چشمگیری بهبود دهد.

اما حتی با این بهبود، حل مسئلهٔ بی‌حسگرِ چنان‌که توصیف کردیم به‌ندرت در عمل شدنی است. یکی از مسائل، وسعت فضای حالت باور است — در فصل پیشین دیدیم اغلب فضای جستجوی اندازهٔ N بیش از حد بزرگ است و اکنون فضاهای جستجوی اندازهٔ 2^N داریم. وانگهی، هر عنصر فضای جستجو مجموعه‌ای تا N عنصر است. برای N بزرگ، حتی بازنمایی یک حالت باور واحد را بدون تمام‌شدن حافظه نخواهیم توانست. یک راه‌حل آن است که حالت باور را با توصیف فشرده‌تری بازنمایی کنیم. به انگلیسی، می‌توانیم بگوییم عامل در حالت آغازین «هیچ‌چیز» می‌داند؛ پس از حرکت به چپ، می‌توانیم بگوییم «در ستون راست‌ترین نیست»، و الی آخر. فصل ۷ توضیح می‌دهد چگونه این را در طرح بازنمایی صوری انجام دهیم.

رویکرد دیگر آن است که از الگوریتم‌های جستجوی استاندارد بپرهیزیم که حالت‌های باور را مانند هر حالت مسئلهٔ دیگر جعبهٔ سیاه می‌دانند. در عوض، می‌توانیم به درون حالت‌های باور نگاه کنیم و الگوریتم‌های جستجوی حالت باورِ افزایشی (Incremental Belief-State Search) توسعه دهیم که راه‌حل را یک حالت فیزیکی در هر بار می‌سازند. برای نمونه، در جهان جاروبرقی بی‌حسگر، حالت باور آغازین {1,2,3,4,5,6,7,8} است و باید توالی کنشی بیابیم که در هر ۸ حالت کار کند. می‌توانیم این کار را با یافتن نخست راه‌حلی که برای حالت ۱ کار می‌کند انجام دهیم؛ سپس بررسی کنیم آیا برای حالت ۲ کار می‌کند؛ اگر نه، برگردیم و راه‌حل متفاوتی برای حالت ۱ بیابیم، و الی آخر.

درست همان‌گونه که جستجوی AND–OR باید برای هر شاخه در گرهٔ AND راه‌حلی بیابد، این الگوریتم باید برای هر حالت در حالت باور راه‌حلی بیابد؛ تفاوت در آن است که جستجوی AND–OR می‌تواند برای هر شاخه راه‌حل متفاوتی بیابد، حال آنکه جستجوی حالت باور افزایشی باید یک راه‌حل بیابد که برای همهٔ حالت‌ها کار کند.

مزیت اصلی رویکرد افزایشی آن است که معمولاً می‌تواند شکست را سریع تشخیص دهد — وقتی حالت باوری حل‌نشدنی است، معمولاً چنین است که زیرمجموعهٔ کوچکی از حالت باور، شامل چند حالت نخستِ بررسی‌شده، نیز حل‌نشدنی است. در برخی موارد، این به شتابی متناسب با اندازهٔ حالت‌های باور می‌انجامد که خودشان ممکن است به اندازهٔ فضای حالت فیزیکی بزرگ باشند.

۴.۴.۲ جستجو در محیط‌های تا حدی قابل مشاهده

بسیاری از مسائل بدون حس‌کردن قابل حل نیستند. برای نمونه، معمای ۸ بی‌حسگر ناممکن است. از سوی دیگر، کمی حس‌کردن می‌تواند راهِ دوری برود: اگر فقط گوشهٔ بالا-چپ را ببینیم می‌توانیم معمای ۸ را حل کنیم. راه‌حل شامل حرکت‌دادن تک‌تک هر کاشی به خانهٔ قابل مشاهده و پیگیری مکانش از آن پس است.

برای مسئلهٔ تا حدی قابل مشاهده، مشخصات مسئله تابع PERCEPT(s) را مشخص می‌کند که ادراک دریافتی عامل در حالتِ داده‌شده را بازمی‌گرداند. اگر حس‌کردن ناجبرگرایانه باشد، می‌توانیم از تابع PERCEPTS استفاده کنیم که مجموعه‌ای از ادراک‌های ممکن بازمی‌گرداند. برای مسائل کاملاً قابل مشاهده، PERCEPT(s)=s برای هر حالت s، و برای مسائل بی‌حسگر PERCEPT(s)=null.

جهان جاروبرقی با حس‌کردن محلی را در نظر بگیرید که در آن عامل حسگر موقعیتی دارد که در خانهٔ چپ ادراک L و در خانهٔ راست ادراک R می‌دهد، و حسگر کثیفی که وقتی خانهٔ کنونی کثیف است Dirty و وقتی تمیز است Clean می‌دهد. پس ادراک در حالت ۱ برابر [L,Dirty] است. با مشاهده‌ناپذیری تا حدی، معمولاً چند حالت همان ادراک را تولید می‌کنند؛ حالت ۳ نیز [L,Dirty] تولید می‌کند. از این رو، با این ادراک آغازین، حالت باور آغازین {1,3} خواهد بود.

می‌توانیم به مدل گذار میان حالت‌های باور برای مسائل تا حدی قابل مشاهده در سه مرحله بیندیشیم، چنان‌که در شکل ۴.۱۵ نشان داده شده:

- **مرحلهٔ پیش‌بینی (Prediction):** حالت باورِ حاصل از کنش، RESULT(b, a)، را دقیقاً چنان‌که با مسائل بی‌حسگر انجام دادیم محاسبه می‌کند. برای تأکید بر اینکه این پیش‌بینی است، از نماد $\hat{b}$ = RESULT(b, a) استفاده می‌کنیم که «کلاه» روی b به معنای «تخمینی» است، و نیز از PREDICT(b,a) به‌عنوان مترادف RESULT(b, a) استفاده می‌کنیم.
- **مرحلهٔ ادراک‌های ممکن:** مجموعهٔ ادراک‌هایی را محاسبه می‌کند که می‌توان در حالت باور پیش‌بینی‌شده مشاهده کرد (با حرف o برای مشاهده):

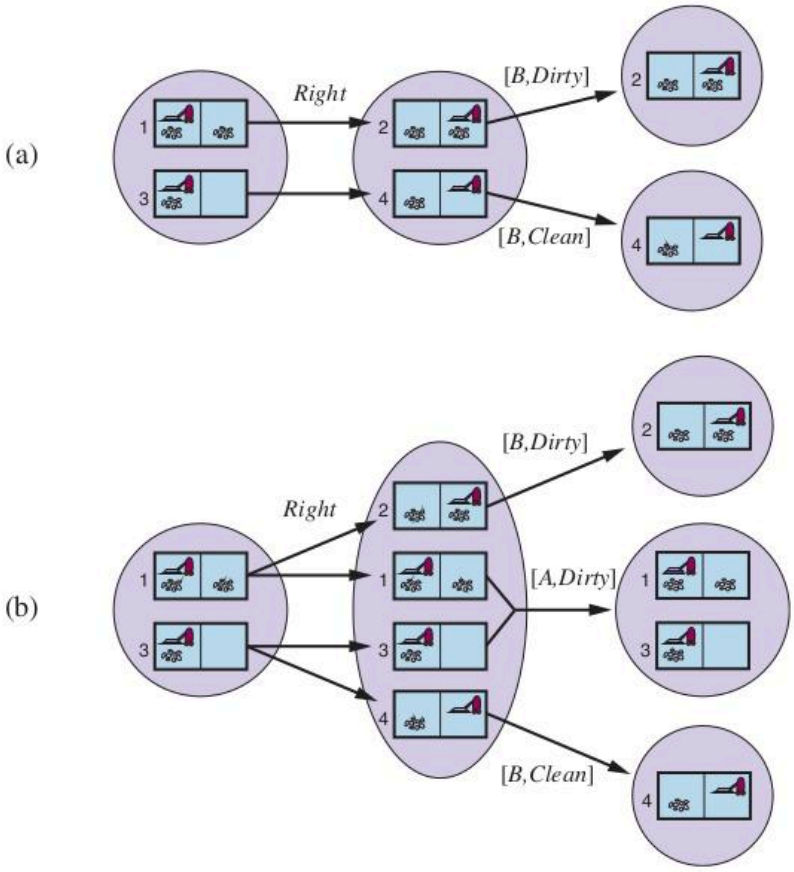
$$\{o : o = \text{PERCEPT}(s) \text{ و } s \in \hat{b}\} = \text{POSSIBLE-PERCEPTS}(\hat{b})$$

- **مرحلهٔ به‌روزرسانی (Update):** برای هر ادراک ممکن، حالت باوری را محاسبه می‌کند که از آن ادراک حاصل می‌شود. حالت باور به‌روزشدهٔ bo مجموعهٔ حالت‌های $\hat{b}$ است که می‌توانستند آن ادراک را تولید کنند:

$$bo = \text{UPDATE}(\hat{b}, o) = \{s : o = \text{PERCEPT}(s) \text{ و } s \in \hat{b}\}$$

عامل باید هنگام برنامه‌ریزی با ادراک‌های ممکن سروکار داشته باشد، زیرا تا اجرای برنامه ادراک‌های واقعی را نخواهد دانست. توجه کنید ناجبرگرایی در محیط فیزیکی می‌تواند حالت باور را در مرحلهٔ پیش‌بینی بزرگ کند، اما هر حالت باور به‌روزشدهٔ bo نمی‌تواند بزرگ‌تر از حالت باور پیش‌بینی‌شدهٔ $\hat{b}$ باشد؛ مشاهده‌ها فقط می‌توانند به کاهش عدم قطعیت کمک کنند. وانگهی، برای حس‌کردن جبرگرایانه، حالت‌های باورِ ادراک‌های ممکنِ متفاوت ناهم‌پوش خواهند بود و افرازی از حالت باور پیش‌بینی‌شدهٔ اصلی می‌سازند. با کنار هم گذاشتن این سه مرحله، حالت‌های باورِ ممکنِ حاصل از کنشِ داده‌شده و ادراک‌های ممکنِ پسین را به دست می‌آوریم:

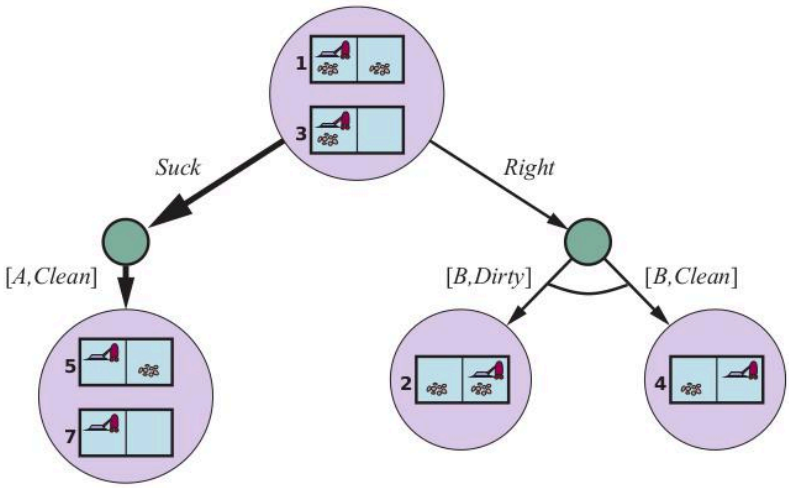
$$o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a), o)\} = \text{RESULTS}(b, a) \quad \text{(4.5)} \quad \{bo : bo = \text{UPDATE}(\text{PREDICT}(b, a) \text{ و } a))$$



شکل ۴.۱۵ – دو نمونه از گذارها در جهان‌های جاروبرقی با حس‌کردن محلی. (الف) در جهان جبرگرایانه، Right در حالت باور آغازین اعمال می‌شود و به حالت باور پیش‌بینی‌شدهٔ تازه‌ای با دو حالت فیزیکی ممکن می‌انجامد؛ برای آن حالت‌ها، ادراک‌های ممکن [R,Dirty] و [R,Clean]اند که به دو حالت باور می‌انجامند که هر یک تکی‌اند. (ب) در جهان لغزنده، Right در حالت باور آغازین اعمال می‌شود و حالت باور تازه‌ای با چهار حالت فیزیکی می‌دهد؛ برای آن حالت‌ها، ادراک‌های ممکن [L,Dirty]، [R,Dirty] و [R,Clean]اند که به سه حالت باور چنان‌که نشان داده شده می‌انجامند.

۴.۴.۳ حل مسائل تا حدی قابل مشاهده

بخش پیشین نشان داد چگونه تابع RESULTS را برای مسئلهٔ حالت باورِ ناجبرگرایانه از مسئلهٔ فیزیکی زیرین با توجه به تابع PERCEPT مشتق کنیم. با این صورت‌بندی، الگوریتم جستجوی AND–OR شکل ۴.۱۱ را می‌توان مستقیماً برای استخراج راه‌حل اعمال کرد. شکل ۴.۱۶ بخشی از درخت جستجو را برای جهان جاروبرقی با حس‌کردن محلی نشان می‌دهد، با فرض ادراک آغازین [A,Dirty]. راه‌حل برنامهٔ شرطی `Suck, Right, if Bstate={6} then Suck]` است. `[[] else`



شکل ۴.۱۶ – سطح نخست درخت جستجوی AND–OR برای مسئله‌ای در جهان جاروبرقی با حس‌کردن محلی؛ Suck نخستین کنش در راه‌حل است.

توجه کنید چون مسئلهٔ حالت باور را به الگوریتم جستجوی AND–OR دادیم، برنامهٔ شرطی‌ای بازگرداند که حالت باور را آزمایش می‌کند نه حالت واقعی را. این چنان‌که باید است: در محیط تا حدی قابل مشاهده، عامل حالت واقعی را نخواهد دانست.

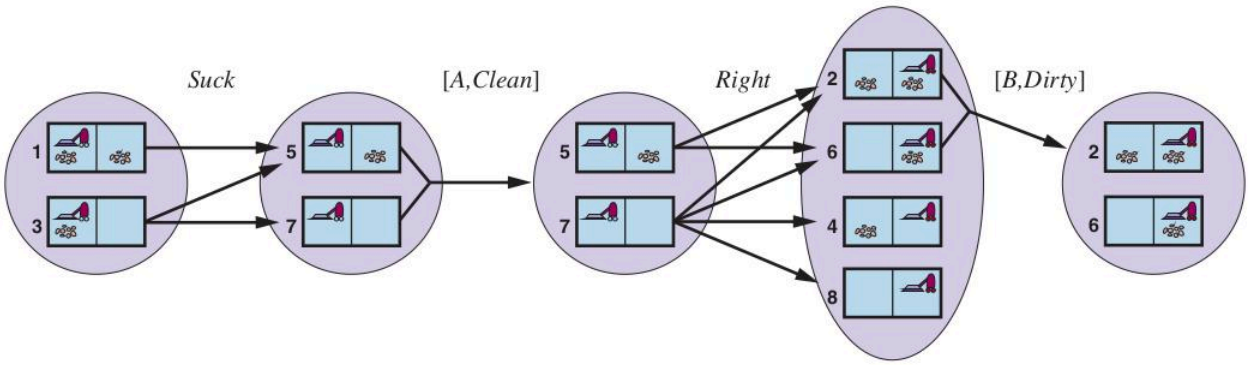
مانند مورد الگوریتم‌های جستجوی استانداردِ اعمال‌شده بر مسائل بی‌حسگر، الگوریتم جستجوی AND–OR حالت‌های باور را مانند هر حالت دیگر جعبهٔ سیاه می‌داند. می‌توان با بررسی حالت‌های باورِ قبلاً تولیدشده که زیرمجموعه یا فوق‌مجموعهٔ حالت کنونی‌اند، درست مانند مسائل بی‌حسگر، این را بهبود داد. همچنین می‌توان الگوریتم‌های جستجوی افزایشی مشابه آنچه برای مسائل بی‌حسگر توصیف شد مشتق کرد که شتاب‌های چشمگیری بر رویکرد جعبهٔ سیاه می‌دهند.

۴.۴.۴ عامل برای محیط‌های تا حدی قابل مشاهده

عاملِ محیط‌های تا حدی قابل مشاهده مسئله‌ای صورت‌بندی می‌کند، الگوریتم جستجویی (مانند AND-OR-SEARCH) برای حلش فراخوانی می‌کند و راه‌حل را اجرا می‌کند. دو تفاوت اصلی میان این عامل و عاملِ محیط‌های کاملاً قابل مشاهده و جبرگرایانه هست.

نخست، راه‌حل برنامهٔ شرطی خواهد بود نه توالی؛ برای اجرای عبارت if-then-else، عامل باید شرط را آزمایش کند و شاخهٔ مناسب شرطی را اجرا کند. دوم، عامل باید حالت باورش را هنگام انجام کنش‌ها و دریافت ادراک‌ها نگه دارد. این فرایند شبیه فرایند پیش‌بینی–مشاهده–به‌روزرسانی معادلهٔ (۴.۵) است اما در واقع ساده‌تر است، زیرا ادراک توسط محیط داده می‌شود نه محاسبه توسط عامل. با حالت باور آغازین b، کنش a و ادراک o، حالت باور تازه چنین است:

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o) \qquad (4.6)$$

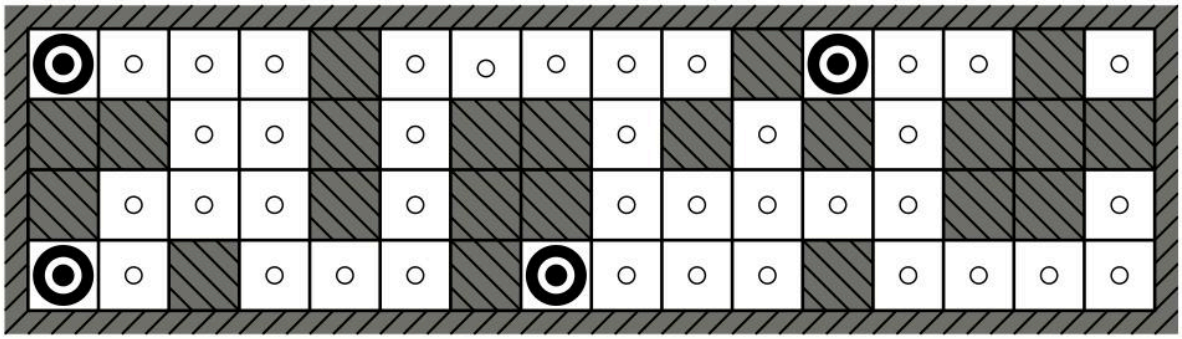


شکل ۴.۱۷ – دو چرخهٔ پیش‌بینی–به‌روزرسانیِ نگه‌داری حالت باور در جهان جاروبرقی کودکستانی با حس‌کردن محلی.

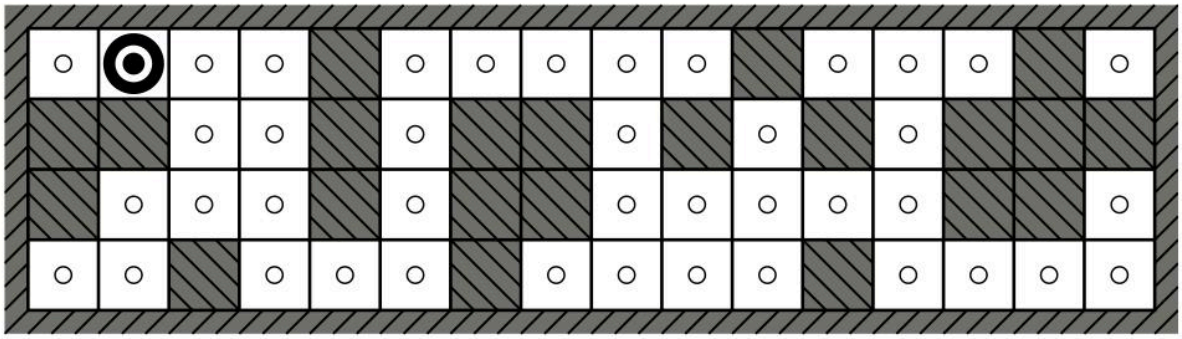
جهان جاروبرقی «کودکستانی» (Kindergarten) را در نظر بگیرید که در آن عامل‌ها فقط حالت خانهٔ کنونی‌شان را حس می‌کنند و هر خانه‌ای ممکن است در هر زمان کثیف شود، مگر آنکه عامل در همان لحظه فعالانه تمیزش کند.[7] شکل ۴.۱۷ حالت باورِ نگه‌داری‌شده در این محیط را نشان می‌دهد.

در محیط‌های تا حدی قابل مشاهده – که اکثریت قریب به اتفاق محیط‌های جهان واقعی را شامل می‌شود – نگه‌داری حالت باور کارکردی هسته‌ای هر سامانهٔ هوشمند است. این کارکرد با نام‌های گوناگونی خوانده می‌شود: پایش (Monitoring)، پالایش (Filtering) و تخمین حالت (State Estimation). معادلهٔ (۴.۶) «تخمین‌گر حالت بازگشتی» (Recursive State Estimator) نامیده می‌شود، زیرا حالت باور تازه را از حالت قبلی محاسبه می‌کند نه با بررسی کل توالی ادراک. اگر عامل قرار نیست «عقب بماند»، محاسبه باید به سرعتی که ادراک‌ها وارد می‌شوند رخ دهد. با پیچیده‌تر شدن محیط، عامل فقط زمان محاسبهٔ حالت باورِ تقریبی را خواهد داشت، شاید با تمرکز بر پیامدهای ادراک برای جنبه‌هایی از محیط که در حال حاضر مورد علاقه‌اند. بیشتر کار روی این مسئله برای محیط‌های تصادفی با حالت پیوسته و با ابزارهای نظریهٔ احتمال انجام شده، چنان‌که در فصل ۱۴ توضیح داده می‌شود. در این بخش مثالی در محیط گسسته با حسگرهای جبرگرایانه و کنش‌های ناجبرگرایانه نشان می‌دهیم.

مثال دربارهٔ رباتی با تکلیف تخمین حالتِ خاصی به نام «جایگاه‌سازی» (Localization) است: فهمیدن اینکه کجاست، با داشتن نقشهٔ جهان و توالی‌ای از ادراک‌ها و کنش‌ها. ربات ما در محیط مارپیچ‌مانند شکل ۴.۱۸ گذاشته شده. ربات به چهار حسگر صوتی (سونار) مجهز است که می‌گویند آیا مانعی – دیوار بیرونی یا خانهٔ تیره‌سایه‌خورده در شکل – در هر یک از چهار جهت اصلی هست. ادراک به شکل بردار بیتی است، برای هر یک از جهت‌های شمال، شرق، جنوب و غرب به همین ترتیب یک بیت؛ پس 1011 یعنی موانعی در شمال، جنوب و غرب هست اما نه شرق. فرض می‌کنیم حسگرها دادهٔ کاملاً درست می‌دهند و ربات نقشهٔ درستی از محیط دارد. اما متأسفانه سامانهٔ ناوبری ربات خراب است، پس وقتی کنش Right را اجرا می‌کند، تصادفی به یکی از خانه‌های مجاور حرکت می‌کند. تکلیف ربات تعیین مکان کنونی‌اش است.



(a) Possible locations of robot after $E_1 = 1011$



(b) Possible locations of robot after $E_1 = 1011, E_2 = 1010$

شکل ۴.۱۸ – موقعیت‌های ممکن ربات، ⊙، (الف) پس از یک مشاهده، E1 = 1011، و (ب) پس از جابه‌جایی یک خانه و مشاهدهٔ دوم، E2 = 1010. وقتی حسگرها بی‌نویز و مدل گذار دقیق باشد، تنها یک مکان ممکن برای ربات سازگار با این توالی دو مشاهده هست.

فرض کنیم ربات تازه روشن شده و نمی‌داند کجاست — حالت باور آغازینش b مجموعهٔ همهٔ مکان‌هاست. ربات سپس ادراک 1011 را دریافت و با معادلهٔ bo = UPDATE(1011) به‌روزرسانی انجام می‌دهد و ۴ مکانی را می‌دهد که در شکل ۴.۱۸(الف) نشان داده شده. می‌توانید مارپیچ را بررسی کنید و ببینید آن‌ها تنها چهار مکانی هستند که ادراک 1011 را تولید می‌کنند. سپس ربات کنش Right را اجرا می‌کند، اما نتیجه ناجبرگرایانه است. حالت باور تازه، ba = PREDICT(bo, Right)، شامل همهٔ مکان‌هایی است که یک قدم از مکان‌های bo فاصله دارند. وقتی ادراک دوم، 1010، می‌رسد، ربات UPDATE(ba, 1010) را انجام می‌دهد و می‌یابد حالت باور به مکان تکیِ نشان‌داده‌شده در شکل ۴.۱۸(ب) فروریخته است. این تنها مکانی است که می‌تواند نتیجهٔ UPDATE(PREDICT(UPDATE(b, 1011), Right), 1010) باشد.

با کنش‌های ناجبرگرایانه، گام PREDICT حالت باور را بزرگ می‌کند، اما گام UPDATE آن را دوباره کوچک می‌کند — تا وقتی ادراک‌ها اطلاعات شناساییِ مفیدی فراهم کنند. گاهی ادراک‌ها برای جایگاه‌سازی چندان کمکی نمی‌کنند: اگر یک یا چند راهروی بلند شرقی-غربی بود، ربات می‌توانست توالی بلندی از ادراک‌های 1010 دریافت کند، اما هرگز نداند در کجای راهرو(ها)ست. اما برای محیط‌های با تنوع جغرافیایی معقول، جایگاه‌سازی اغلب به‌سرعت به نقطهٔ تکی همگرا می‌شود، حتی وقتی کنش‌ها ناجبرگرایانه‌اند.

اگر حسگرها معیوب باشند چه؟ اگر فقط با منطق بولی بتوانیم استدلال کنیم، باید هر بیت حسگر را درست یا نادرست بدانیم، که همان نداشتن هیچ اطلاعات ادراکی است. اما خواهیم دید استدلال احتمالاتی (فصل ۱۲) به ما امکان می‌دهد از حسگر معیوب اطلاعات مفید استخراج کنیم، تا وقتی کمتر از نیمی از اوقات اشتباه کند.

۴.۵ عامل‌های جستجوی آنلاین و محیط‌های ناشناخته

تاکنون بر عامل‌هایی تمرکز کرده‌ایم که از الگوریتم‌های جستجوی آفلاین (Offline Search) استفاده می‌کنند. آن‌ها پیش از برداشتن نخستین کنش، راه‌حل کامل محاسبه می‌کنند. در برابر، عامل جستجوی آنلاین (Online Search)[8] محاسبه و کنش را در هم می‌آمیزد: نخست کنشی برمی‌دارد، سپس محیط را مشاهده و کنش بعدی را محاسبه می‌کند. جستجوی آنلاین در محیط‌های پویا یا نیمه‌پویا ایدهٔ خوبی است، جایی که برای نشستن و محاسبهٔ بیش از حد طولانی جریمه هست. جستجوی آنلاین در حوزه‌های ناجبرگرایانه نیز مفید است، زیرا به عامل امکان می‌دهد تلاش محاسباتی‌اش را بر احتیاط‌هایی متمرکز کند که واقعاً رخ می‌دهند نه آن‌هایی که ممکن است رخ دهند اما احتمالاً نخواهند. البته دادوستدی هست: هرچه عامل بیشتر از پیش برنامه‌ریزی کند، کمتر خود را بی‌دستمایه در جریان می‌یابد.

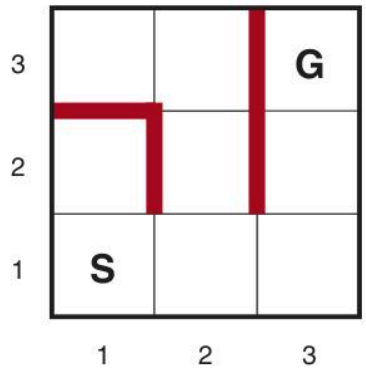
در محیط‌های ناشناخته، که عامل نمی‌داند چه حالت‌هایی وجود دارند یا کنش‌هایش چه می‌کنند، عامل باید کنش‌هایش را به‌عنوان آزمایش برای یادگیری دربارهٔ محیط به کار گیرد. نمونهٔ شاخص جستجوی آنلاین «مسئلهٔ نقشه‌سازی» (Mapping Problem) است: رباتی در ساختمانی ناشناخته گذاشته می‌شود و باید کاوش کند تا نقشه‌ای بسازد که بعداً برای رفتن از A به B به کار رود. روش‌های گریز از هزارتوها — دانش لازمِ قهرمانان مشتاق دوران باستان — نیز نمونه‌هایی از الگوریتم‌های جستجوی آنلاین‌اند. کاوش فضایی تنها شکل کاوش آنلاین نیست. نوزاد تازه‌به‌دنیاآمده را در نظر بگیرید: کنش‌های ممکن بسیاری دارد اما پیامد هیچ‌کدام را نمی‌داند و فقط چند حالتِ ممکن را تجربه کرده که می‌تواند به آن‌ها برسد.

۴.۵.۱ مسائل جستجوی آنلاین

مسئلهٔ جستجوی آنلاین با درهم‌آمیختن محاسبه، حس‌کردن و کنش حل می‌شود. با فرض محیط جبرگرایانه و کاملاً قابل مشاهده آغاز می‌کنیم (فصل ۱۷ این فرض‌ها را شل می‌کند) و مقرر می‌کنیم عامل فقط موارد زیر را می‌داند:

- ACTIONS(s)، کنش‌های قانونی در حالت s؛
- c(s,a,s')، هزینهٔ اعمال کنش a در حالت s برای رسیدن به حالت s'. توجه کنید تا وقتی عامل نداند s' پیامد است، نمی‌توان از این استفاده کرد.
- IS-GOAL(s)، آزمون هدف. به‌ویژه توجه کنید عامل نمی‌تواند RESULT(s, a) را تعیین کند مگر با واقعاً بودن در s و انجام a.

برای نمونه، در مسئلهٔ هزارتوی شکل ۴.۱۹، عامل نمی‌داند رفتن به بالا از (1,1) به (1,2) می‌رسد؛ و پس از آنکه چنین کرد، نمی‌داند رفتن به پایین او را به (1,1) بازمی‌گرداند. این درجه از نادانی در برخی کاربردها قابل کاهش است — برای نمونه، ربات کاوشگر ممکن است بداند کنش‌های حرکتی‌اش چگونه کار می‌کنند و فقط از مکان موانع بی‌اطلاع باشد. سرانجام، عامل ممکن است به تابع اکتشافی مقبول h(s) دسترسی داشته باشد که فاصلهٔ حالت کنونی تا حالت هدف را تخمین می‌زند. برای نمونه، در شکل ۴.۱۹، عامل ممکن است مکان هدف را بداند و بتواند از اکتشافی فاصلهٔ منهتن استفاده کند (صفحهٔ ۹۷).



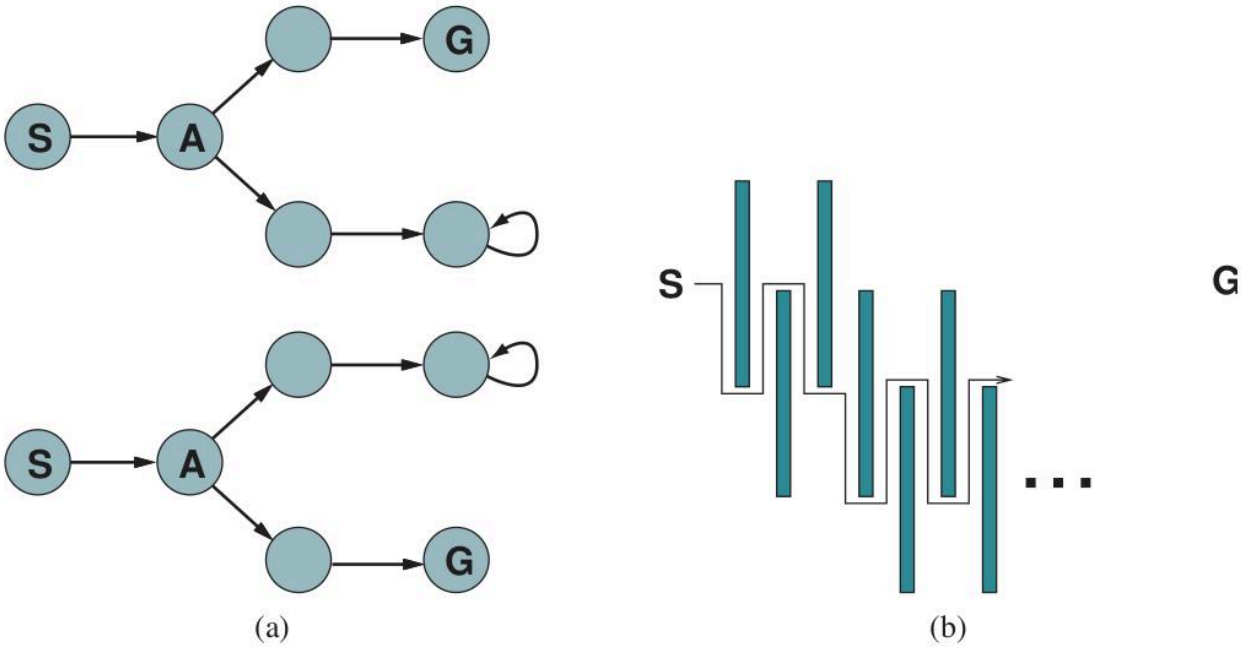
شکل ۴.۱۹ — مسئلهٔ هزارتوی ساده. عامل از S آغاز می‌کند و باید به G برسد، اما هیچ‌چیز از محیط نمی‌داند.

معمولاً هدف عامل رسیدن به حالت هدف با کمینه‌کردن هزینه است. (هدف ممکن دیگر صرفاً کاوش کل محیط است.) هزینه، هزینهٔ کل مسیری است که عامل هنگام سفر متحمل می‌شود. رایج است این هزینه را با هزینهٔ مسیری مقایسه کنند که عامل اگر فضای جستجو را از پیش می‌دانست متحمل می‌شد — یعنی مسیر بهینه در محیط شناخته‌شده. به زبان الگوریتم‌های آنلاین، این مقایسه «نسبت رقابتی» (Competitive Ratio) نامیده می‌شود؛ می‌خواهیم تا حد ممکن کوچک باشد.

کاوشگران آنلاین در برابر بن‌بست‌ها (Dead Ends) آسیب‌پذیرند: حالت‌هایی که از آن‌ها هیچ حالت هدفی قابل دسترسی نیست. اگر عامل نداند هر کنش چه می‌کند، ممکن است کنش «پرش به چاه بی‌پایان» را اجرا کند و هرگز به هدف نرسد. در حالت کلی، هیچ الگوریتمی نمی‌تواند در همهٔ فضاهای حالت از بن‌بست‌ها بپرهیزد. دو فضای حالت بن‌بست‌دار شکل ۴.۲۰(الف) را در نظر بگیرید. الگوریتم جستجوی آنلاینی که حالت‌های S و A را بازدید کرده نمی‌تواند بگوید در حالت بالایی است یا پایینی؛ بر اساس آنچه عامل دیده، دو حالت یکسان می‌نمایند. پس هیچ راهی نیست که بداند در هر دو فضای حالت چگونه کنش درست را برگزیند.

این نمونه‌ای از «استدلال حریف» (Adversary Argument) است — می‌توانیم حریفی را تصور کنیم که فضای حالت را هنگام کاوش عامل می‌سازد و هدف‌ها و بن‌بست‌ها را هر جا که بخواهد می‌گذارد، چنان‌که در شکل ۴.۲۰(ب). بن‌بست‌ها دشواری واقعی برای کاوش رباتاند — پله‌ها، سطح‌های شیب‌دار، پرتگاه‌ها، خیابان‌های یک‌طرفه و حتی زمین طبیعی همه حالت‌هایی دارند که برخی کنش‌ها در آن‌ها برگشت‌ناپذیرند (Irreversible Action) — هیچ راهی برای بازگشت به حالت قبلی نیست. الگوریتم کاوشی که ارائه می‌کنیم فقط در فضاهای حالتی تضمین‌شده کار می‌کند که «به‌شکل ایمن قابل کاوش» (Safely Explorable) باشند — یعنی از هر حالت قابل دسترسی، حالتی هدف قابل

دسترسی باشد. فضاهای حالت با فقط کنش‌های برگشت‌پذیر، مانند هزارتوها و معمای ۸، بدیهی است به‌شکل ایمن قابل کاوش‌اند (اگر اصلاً راه‌حلی داشته باشند). موضوع کاوش ایمن را در بخش ۲۲.۳.۲ با ژرفای بیشتری پوشش می‌دهیم.



شکل ۴.۲۰ – (الف) دو فضای حالت که ممکن است عامل جستجوی آنلاین را به بن‌بست بکشانند. هر عاملِ داده‌شده در دست‌کم یکی از این فضاها شکست خواهد خورد. (ب) محیط دوبعدی که می‌تواند عامل جستجوی آنلاین را به مسیری خودسرانه ناکارآمد به سوی هدف بکشاند. هر انتخابی که عامل بکند، حریف آن مسیر را با دیوار بلند و باریک دیگری مسدود می‌کند، پس مسیر طی‌شده بسیار بلندتر از بهترین مسیر ممکن است.

حتی در محیط‌های به‌شکل ایمن قابل کاوش، اگر مسیرهای با هزینهٔ بی‌کران وجود داشته باشند، هیچ نسبت رقابتی کران‌داری قابل تضمین نیست. نشان‌دادن این در محیط‌های با کنش‌های برگشت‌ناپذیر آسان است، اما در واقع برای حالت برگشت‌پذیر نیز صادق می‌ماند، چنان‌که شکل ۴.۲۰(ب) نشان می‌دهد. به همین دلیل، رایج است عملکرد الگوریتم‌های جستجوی آنلاین را بر حسب اندازهٔ کل فضای حالت توصیف کنیم نه فقط عمق کم‌عمق‌ترین هدف.

۴.۵.۲ عامل‌های جستجوی آنلاین

پس از هر کنش، عامل آنلاین در محیط قابل مشاهده، ادراکی دریافت می‌کند که به او می‌گوید به چه حالتی رسیده؛ از این اطلاعات می‌تواند نقشهٔ محیطش را بیفزاید. سپس از نقشهٔ به‌روزشده برای برنامه‌ریزی اینکه بعداً کجا برود استفاده می‌شود. این درهم‌آمیختن برنامه‌ریزی و کنش بدان معناست که الگوریتم‌های جستجوی آنلاین کاملاً با الگوریتم‌های جستجوی آفلاینِ قبلاً‌دیده‌شده متفاوت‌اند: الگوریتم‌های آفلاین مدلشان از فضای حالت را کاوش می‌کنند، حال آنکه الگوریتم‌های آنلاین جهان واقعی را کاوش می‌کنند. برای نمونه، A* می‌تواند گره‌ای در بخشی از فضا گسترش دهد و سپس بلافاصله گره‌ای در بخشی دور از فضا گسترش دهد، زیرا گسترش گره شامل کنش‌های شبیه‌سازی‌شده است نه واقعی.

الگوریتم آنلاین اما فقط برای حالتی که فیزیکیاً در آن است می‌تواند جانشین‌ها را کشف کند. برای پرهیز از سفر تمام‌راه تا حالتی دور برای گسترش گرهٔ بعدی، بهتر می‌نماید گره‌ها را به ترتیبی محلی گسترش دهد. جستجوی عمق-اول دقیقاً این ویژگی را دارد، زیرا (به‌جز هنگام بازگشت الگوریتم) گرهٔ بعدی‌گسترش‌یافته فرزند گرهٔ قبلی‌گسترش‌یافته است. عامل کاوش عمق-اول آنلاین (برای کنش‌های جبرگرایانه اما ناشناخته) در شکل ۴.۲۱ نشان داده شده.

```
function ONLINE-DFS-AGENT(problem, s′) returns an action
                                                    :persistent
  s, a, the previous state and action, initially null
  result, a table mapping (s, a) to s′, initially empty
  untried, a table mapping s to a list of untried actions
  unbacktracked, a table mapping s to a list of states never backtracked to
                                  if problem.IS-GOAL(s′) then return stop
if s′ is a new state (not in untried) then untried[s′] ← problem.ACTIONS(s′)
                                                  if s is not null then
                                                    result[s, a] ← s′
                                      add s to the front of unbacktracked[s′]
                                                  if untried[s′] is empty then
                                  if unbacktracked[s′] is empty then return stop
                        else a ← an action b such that result[s′, b] = POP(unbacktracked[s′])
                                                  else a ← POP(untried[s′])
                                                                    s ← s′
                                                                 return a
```

شکل ۴.۲۱ — عامل جستجوی آنلاین که از کاوش عمق-اول استفاده می‌کند. عامل فقط در فضاهای حالتی می‌تواند به‌شکل ایمن کاوش کند که هر کنش با کنش دیگری «خنثی» شود.

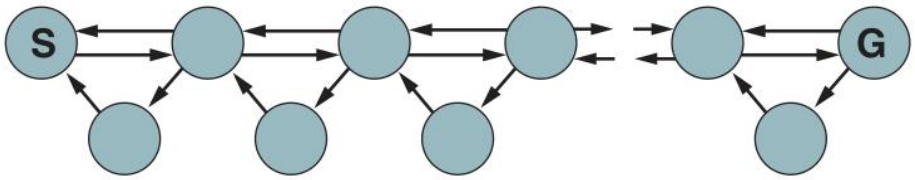
این عامل نقشه‌اش را در جدولی به نام result[s,a] ذخیره می‌کند که حالتی را ثبت می‌کند که از اجرای کنش a در حالت s حاصل می‌شود. (برای کنش‌های ناجبرگرایانه، عامل می‌تواند مجموعه‌ای از حالت‌ها را در result[s,a] ثبت کند.) هرگاه حالت کنونی کنش‌های کاوش‌نشده داشته باشد، عامل یکی از آن کنش‌ها را می‌آزماید. دشواری وقتی پیش می‌آید که عامل همهٔ کنش‌های یک حالت را آزموده باشد. در جستجوی عمق-اول آفلاین، حالت صرفاً از صف حذف می‌شود؛ در جستجوی آنلاین، عامل باید در جهان فیزیکی به عقب برگردد (Backtrack). در جستجوی عمق-اول، این به معنای بازگشت به حالتی است که عامل تازه‌ترین بار از آن وارد حالت کنونی شده. برای دستیابی به این، الگوریتم جدول دیگری نگه می‌دارد که برای هر حالت، حالت‌های پیشین را فهرست می‌کند که عامل هنوز به آن‌ها بازنگشته. اگر عامل از حالت‌هایی که می‌تواند به آن‌ها بازگردد تهی شده باشد، جستجویش کامل است. توصیه می‌کنیم خواننده پیشرفت ONLINE-DFS-AGENT را هنگام اعمال بر هزارتوی شکل ۴.۱۹ دنبال کند. به‌راحتی دیده می‌شود عامل در بدترین حالت هر پیوند فضای حالت را دقیقاً دو بار می‌پیماید. برای کاوش، این بهینه است؛ برای یافتن هدف از سوی دیگر، نسبت رقابتی عامل می‌تواند خودسرانه بد باشد اگر وقتی هدفی دقیقاً کنار حالت آغازین است، به گشت‌وگذاری بلند برود. تغییر آنلاینِ تعمیق تکراری این مسئله را حل می‌کند؛ برای محیطی که درخت یکنواخت باشد، نسبت رقابتی چنین عاملی ثابت کوچکی است.

به‌سبب روش بازگشتش، ONLINE-DFS-AGENT فقط در فضاهای حالتی کار می‌کند که کنش‌ها برگشت‌پذیرند. الگوریتم‌های کمی پیچیده‌تری هست که در فضاهای حالت عمومی کار می‌کنند، اما هیچ چنین الگوریتمی نسبت رقابتی کران‌داری ندارد.

۴.۵.۳ جستجوی محلی آنلاین

مانند جستجوی عمق-اول، جستجوی تپه‌نوردی ویژگی محلی‌بودن در گسترش گره‌هایش را دارد. در واقع، چون فقط یک حالت کنونی در حافظه نگه می‌دارد، جستجوی تپه‌نوردی از قبل الگوریتم جستجوی آنلاین است! متأسفانه، الگوریتم پایه برای کاوش چندان خوب نیست، زیرا عامل را در بیشینه‌های محلی بی‌راه‌خروج رها می‌کند. وانگهی، راه‌اندازی مجدد تصادفی قابل استفاده نیست، زیرا عامل نمی‌تواند خود را به حالت آغازین تازه دورنما کند.

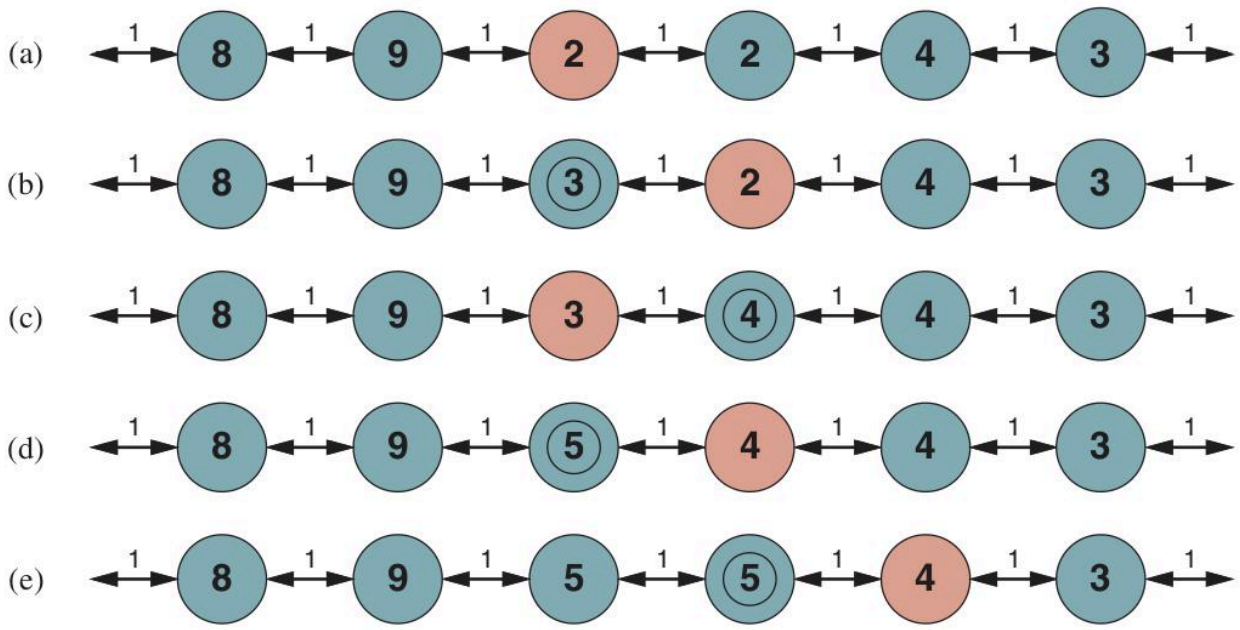
به‌جای راه‌اندازی مجدد تصادفی، می‌توان «گام تصادفی» (Random Walk) را برای کاوش محیط در نظر گرفت. گام تصادفی صرفاً به‌صورت تصادفی یکی از کنش‌های در دسترس را از حالت کنونی برمی‌گزیند؛ می‌توان به کنش‌هایی که هنوز آزموده نشده‌اند اولویت داد. به‌راحتی اثبات می‌شود گام تصادفی سرانجام هدف را خواهد یافت یا کاوشش را کامل می‌کند، به شرط آنکه فضا متناهی و به‌شکل ایمن قابل کاوش باشد.[9] از سوی دیگر، فرایند می‌تواند بسیار کند باشد. شکل ۴.۲۲ محیطی را نشان می‌دهد که در آن گام تصادفی گام‌های نمایی‌چندانی برای یافتن هدف برمی‌دارد، زیرا برای هر حالت در سطر بالا به‌جز S، پیشرفت به عقب دو برابر پیشرفت به جلو محتمل است. مثال البته ساختگی است، اما فضاهای حالت جهان واقعیِ بسیاری هست که توپولوژی‌شان این‌گونه «تله‌ها» را برای گام‌های تصادفی ایجاد می‌کند.



شکل ۴.۲۲ – محیطی که در آن گام تصادفی گام‌های نمایی‌چندانی برای یافتن هدف برمی‌دارد.

افزودن حافظه به تپه‌نوردی به‌جای تصادفی‌بودن، رویکرد مؤثرتری از آب درمی‌آید. ایدهٔ پایه ذخیرهٔ «بهترین تخمین کنونی» H(s) از هزینهٔ رسیدن به هدف از هر حالت بازدیدشده است. H(s) در آغاز فقط تخمین اکتشافی h(s) است و با کسب تجربه توسط عامل در فضای حالت به‌روز می‌شود. شکل ۴.۲۳ مثال ساده‌ای در فضای حالت یک‌بعدی نشان می‌دهد. در (الف)، عامل به نظر می‌رسد در کمینهٔ محلی تختِ حالت قرمز گیر کرده. به‌جای ماندن همان‌جا، عامل باید به‌ظاهر بهترین مسیر به هدف را با توجه به تخمین‌های هزینهٔ کنونی همسایه‌هایش دنبال کند. هزینهٔ تخمینی رسیدن به هدف از طریق همسایهٔ s'، هزینهٔ رسیدن به s' به‌علاوهٔ هزینهٔ تخمینی رسیدن به هدف از آن‌جاست – یعنی c(s,a,s') + H(s'). در مثال، دو کنش با هزینه‌های تخمینی 1+9 به چپ و 1+2 به راست هست، پس بهتر می‌نماید به راست حرکت کنیم. در (ب) روشن است تخمین هزینهٔ ۲ برای حالت قرمزِ (الف) بیش از حد خوش‌بینانه بود. چون

بهترین حرکت ۱ هزینه داشت و به حالتی رساند که دست‌کم ۲ گام با هدف فاصله دارد، حالت قرمز باید دست‌کم ۳ گام با هدف فاصله داشته باشد، پس H آن باید متناسباً به‌روز شود، چنان‌که در شکل ۴.۲۳(ب). با ادامهٔ این فرایند، عامل دو بار دیگر رفت‌وبرگشت می‌کند و هر بار H را به‌روز و کمینهٔ محلی را «صاف» می‌کند تا آنکه به راست می‌گریزد.



شکل ۴.۲۳ – پنج تکرار LRTA* در فضای حالت یک‌بعدی. هر حالت با H(s)، تخمین هزینهٔ کنونی رسیدن به هدف، برچسب‌گذاری شده و هر پیوند هزینهٔ کنش ۱ دارد. حالت قرمز مکان عامل را نشان می‌دهد و تخمین‌های هزینهٔ به‌روزشده در هر تکرار دایرهٔ دوتایی دارند.

عاملی که این طرح را پیاده می‌کند، «یادگیری A* زمان-واقعی» (Learning Real-Time A*، یا LRTA*) نامیده می‌شود و در شکل ۴.۲۴ نشان داده شده. مانند ONLINE-DFS-AGENT، نقشه‌ای از محیط در جدول result می‌سازد. تخمین هزینهٔ حالتی را که تازه ترک کرده به‌روز می‌کند و سپس حرکتِ «ظاهراً بهترین» را بر اساس تخمین‌های هزینهٔ کنونی‌اش برمی‌گزیند. یک جزئیات مهم آن است که کنش‌هایی که هنوز در حالت s آزموده نشده‌اند، همیشه فرض می‌شود بلافاصله با کمترین هزینهٔ ممکن، یعنی h(s)، به هدف می‌انجامند. این «خوش‌بینی در شرایط عدم قطعیت» (Optimism Under Uncertainty) عامل را به کاوش مسیرهای تازه و بالقوه امیدوارکننده تشویق می‌کند.

```
function LRTA*-AGENT(problem, s′, h) returns an action
                                                    :persistent
    s, a, the previous state and action, initially null
  result, a table mapping (s, a) to s′, initially empty
H, a table mapping s to a cost estimate, initially empty
                                  if IS-GOAL(s′) then return stop
    if s′ is a new state (not in H) then H[s′] ← h(s′)
                                        if s is not null then
                                          ′result[s, a] ← s
                H[s] ← min_{b∈ACTIONS(s)} LRTA*-COST(s, b, result[s, b], H)
a ← argmin_{b∈ACTIONS(s′)} LRTA*-COST(problem, s′, b, result[s′, b], H)
                                                        ′s ← s
                                                    return a
```

شکل ۴.۲۴ – عامل LRTA* کنش را بر اساس مقادیر حالت‌های همسایه برمی‌گزیند که با حرکت عامل در فضای حالت به‌روز می‌شوند.

عامل LRTA* تضمین‌شده در هر محیط متناهیِ به‌شکل ایمن قابل کاوش هدفی می‌یابد. برخلاف A* اما برای فضاهای حالت بی‌نهایت کامل نیست – مواردی هست که می‌تواند بی‌نهایت به گمراهی کشیده شود. می‌تواند محیطی با n حالت را در بدترین حالت در $O(n^2)$ گام کاوش کند، اما اغلب بسیار بهتر عمل می‌کند. عامل LRTA* فقط یکی از خانوادهٔ بزرگ عامل‌های آنلاینی است که می‌توان با مشخص کردن قاعدهٔ انتخاب کنش و قاعدهٔ به‌روزرسانی به شیوه‌های گوناگون تعریف کرد. این خانواده را که در اصل برای محیط‌های تصادفی توسعه یافته، در فصل ۲۲ بحث می‌کنیم.

۴.۵.۴ یادگیری در جستجوی آنلاین

نادانی آغازین عامل‌های جستجوی آنلاین فرصت‌های فراوانی برای یادگیری فراهم می‌کند. نخست، عامل‌ها «نقشهٔ» محیط را می‌آموزند – دقیق‌تر، پیامد هر کنش در هر حالت را – صرفاً با ثبت هر یک از تجربه‌هایشان. دوم، عامل‌های جستجوی محلی تخمین‌های دقیق‌تری از هزینهٔ هر حالت با استفاده از قواعد به‌روزرسانی محلی، مانند LRTA*، به دست می‌آورند. در فصل ۲۲ نشان می‌دهیم این به‌روزرسانی‌ها سرانجام برای هر حالت به مقادیر دقیق همگرا می‌شوند، به شرط آنکه عامل فضای حالت را به‌شیوهٔ درست کاوش کند. وقتی مقادیر دقیق شناخته شدند، تصمیم‌های بهینه را می‌توان صرفاً با حرکت به کم‌هزینه‌ترین جانشین گرفت – یعنی تپه‌نوردی ناب آنگاه راهبردی بهینه است.

اگر توصیه‌مان را برای دنبال‌کردن رفتار ONLINE-DFS-AGENT در محیط شکل ۴.۱۹ اجرا کرده باشید، متوجه شده‌اید عامل چندان باهوش نیست. برای نمونه، پس از آنکه دیده کنش Up از (1,1) به (1,2) می‌رود، عامل هنوز هیچ ایده‌ای ندارد که کنش Down به (1,1) بازمی‌گردد یا کنش Up نیز از (2,1) به (2,2)، از (2,2) به (2,3) و الی آخر می‌رود. در حالت کلی، می‌خواهیم عامل بیاموزد که Up مختصهٔ y را زیاد می‌کند مگر آنکه دیواری سر راه باشد، Down آن را کم می‌کند، و الی آخر. برای این‌که این رخ دهد، به دو چیز نیاز داریم. نخست، به بازنمایی صوری و به‌صراحت دست‌کاری‌شدنی برای این‌گونه قواعد عمومی نیاز داریم؛ تاکنون اطلاعات را درون جعبهٔ سیاهی به نام تابع RESULT پنهان کرده‌ایم. فصل‌های ۸ تا ۱۱ به این مسئله اختصاص دارند. دوم، به الگوریتم‌هایی نیاز داریم که بتوانند از مشاهده‌های خاصِ عامل، قواعد عمومی مناسبی بسازند. این‌ها در فصل ۱۹ پوشش داده می‌شوند.

راه‌های متعددی برای این کار هست که همگی زیر عنوان جستجوی افزایشی (Incremental Search) می‌گنجند. می‌توانیم درخت جستجو را در حافظه نگه داریم و بخش‌هایی از آن را که در مسئلهٔ تازه تغییر نکرده‌اند دوباره استفاده کنیم. می‌توانیم مقادیر اکتشافی h را نگه داریم و با کسب اطلاعات تازه – چه به‌سبب تغییر جهان یا محاسبهٔ تخمین بهتر – آن‌ها را به‌روز کنیم. یا می‌توانیم مقادیر بهترین‌مسیر g را نگه داریم و از آن‌ها برای سرهم‌کردن راه‌حل تازه استفاده کنیم و هنگام تغییر جهان آن‌ها را به‌روز کنیم.

خلاصهٔ فصل

این فصل الگوریتم‌های جستجو را برای مسائلِ محیط‌های تا حدی قابل مشاهده، ناجبرگرایانه، ناشناخته و پیوسته بررسی کرد.

- روش‌های جستجوی محلی مانند تپه‌نوردی فقط شمار کمی حالت در حافظه نگه می‌دارند. آن‌ها بر مسائل بهینه‌سازی اعمال شده‌اند، جایی که ایده یافتن حالتی با امتیاز بالاست، بدون نگرانی از مسیر رسیدن به حالت. چندین الگوریتم جستجوی محلی تصادفی توسعه یافته، از جمله بازپخت شبیه‌سازی‌شده که با برنامهٔ سردکردن مناسب، راه‌حل‌های بهینه بازمی‌گرداند.
- بسیاری از روش‌های جستجوی محلی بر مسائل فضاهای پیوسته نیز کاربردپذیرند. برنامه‌ریزی خطی و مسائل بهینه‌سازی محدب از قیدهای خاصی بر شکل فضای حالت و سرشت تابع هدف پیروی می‌کنند و الگوریتم‌های چندجمله‌ای می‌پذیرند که اغلب در عمل فوق‌العاده کارآمدند. برای برخی مسائلِ خوب‌صورت‌بندی‌شدهٔ ریاضی، می‌توانیم بیشینه را با حساب دیفرانسیل و یافتن نقطهٔ گرادیان صفر بیابیم؛ برای مسائل دیگر باید به گرادیان تجربی بسنده کنیم که تفاوت برازش میان دو نقطهٔ نزدیک را می‌سنجد.
- الگوریتم تکاملی، جستجوی تپه‌نوردی تصادفی است که در آن جمعیتی از حالت‌ها نگه‌داری می‌شود. حالت‌های تازه با جهش و تلاقی — که جفت‌هایی از حالت‌ها را ترکیب می‌کند — تولید می‌شوند.
- در محیط‌های ناجبرگرایانه، عامل‌ها می‌توانند جستجوی AND–OR را برای تولید برنامه‌های شرطی اعمال کنند که صرف‌نظر از وقوع هر پیامد در طول اجرا به هدف می‌رسند.
- وقتی محیط تا حدی قابل مشاهده باشد، حالت باور مجموعهٔ حالت‌های ممکنی را بازنمایی می‌کند که عامل ممکن است در آن‌ها باشد.
- الگوریتم‌های جستجوی استاندارد را می‌توان مستقیماً بر فضای حالت باور برای حل مسائل بی‌حسگر اعمال کرد و جستجوی AND–OR حالت باور می‌تواند مسائل تا حدی قابل مشاهدهٔ عمومی را حل کند. الگوریتم‌های افزایشی که راه‌حل‌ها را حالت‌به‌حالت درون حالت باور می‌سازند اغلب کارآمدترند.
- مسائل کاوش وقتی پدید می‌آیند که عامل هیچ ایده‌ای از حالت‌ها و کنش‌های محیطش ندارد. برای محیط‌های به‌شکل ایمن قابل کاوش، عامل‌های جستجوی آنلاین می‌توانند نقشه بسازند و اگر هدفی باشد آن را بیابند. به‌روزرسانی تخمین‌های اکتشافی از تجربه، روش مؤثری برای گریز از کمینه‌های محلی فراهم می‌کند.

یادداشت‌های کتاب‌شناختی و تاریخی

فنون جستجوی محلی تاریخچهٔ بلندی در ریاضیات و علوم رایانه دارند. در واقع، روش نیوتن–رافسون (نیوتن، ۱۶۷۱؛ رافسون، ۱۶۹۰) را می‌توان روش جستجوی محلی بسیار کارآمدی برای فضاهای پیوسته‌ای دانست که اطلاعات گرادیان در آن‌ها در دسترس است. برنت (۱۹۷۳) مرجع کلاسیکی برای روش‌های بهینه‌سازی یک‌بعدی است. کالج (۲۰۰۸) و نیلدر و مید (۱۹۶۵) بر الگوریتم‌های بهینه‌سازی چندبعدی بدون مشتق تمرکز دارند؛ این روش‌ها در علوم تجربی (مانند فیزیک ذرات) و نیز در یادگیری ماشین (مانند تنظیم فراپارامترها) کاربرد دارند. نِدرر (۲۰۰۹) و رایت (۲۰۰۶) مرورهایی دربارهٔ مسائل بهینه‌سازی مقید و الگوریتم‌های مربوطه ارائه می‌دهند. برای توابعی که مشتق‌پذیرند، الگوریتم‌های گرادیان در متون متعددی پوشش داده شده‌اند (لونبرگر، ۲۰۰۳؛ بوید و وندنبرگ، ۲۰۰۴؛ نوسدال و رایت، ۲۰۰۶)، و پیشرفت‌های اخیر در یادگیری عمیق (فصل ۲۱) روش‌های نزول گرادیان تصادفی را بسیار بهبود بخشیده است (بوتبو و همکاران، ۲۰۱۸). فنون مربوطه عبارت‌اند از روش‌های نیوتن، شبه‌نیوتن و روش‌های گرادیان مزدوج که در متون استاندارد پوشش داده شده‌اند، و حساب تغییرات.

فنون پایه توسط بیشاپ (۱۹۹۵) خوب توضیح داده شده؛ پرس و همکاران (۲۰۰۷) گسترهٔ وسیعی از الگوریتم‌ها را پوشش می‌دهند و نرم‌افزار کاری فراهم می‌کنند.

پژوهشگران برای الگوریتم‌های جستجو و بهینه‌سازی از گسترهٔ وسیعی از حوزه‌های مطالعاتی الهام گرفته‌اند: متالورژی (بازپخت شبیه‌سازی‌شده)؛ زیست‌شناسی (الگوریتم‌های ژنتیک)؛ علوم اعصاب (شبکه‌های عصبی)؛ کوهنوردی (تپه‌نوردی)؛ اقتصاد (الگوریتم‌های بازارمحور (دیاس و همکاران، ۲۰۰۶))؛ فیزیک (دسته‌های ذره (لی و یائو، ۲۰۱۲) و شیشه‌های اسپینی (مزارد و همکاران، ۱۹۸۷))؛ رفتار جانوری (یادگیری تقویتی، بهینه‌سازهای گرگ خاکستری (میرجلیلی و لوئیس، ۲۰۱۴))؛ پرنده‌شناسی (جستجوی فاخته (یانگ و دب، ۲۰۱۴))؛ حشره‌شناسی (بهینه‌سازی کلونی مورچه (دوریگو و همکاران، ۲۰۰۸)، کلونی زنبور (کارابوگا و باستورک، ۲۰۰۷)، کرم شب‌تاب (یانگ، ۲۰۰۹) و کرم شب‌تاب‌تاب (کریشنانند و گوس، ۲۰۰۹))؛ و دیگران.

برنامه‌ریزی خطی (LP) نخستین بار به‌صورت نظام‌مند توسط ریاضی‌دان لئونید کانتوروویچ (۱۹۳۹) مطالعه شد. این یکی از نخستین کاربردهای رایانه بود؛ الگوریتم سیمپلکس (دانتزیگ، ۱۹۴۹) با وجود پیچیدگی نماییِ بدترین حالت همچنان استفاده می‌شود. کارمارکار (۱۹۸۴) خانوادهٔ بسیار کارآمدترِ روش‌های نقطهٔ داخلی را توسعه داد که نستروف و نمویروسکی (۱۹۹۴) نشان دادند برای ردهٔ عمومی‌تر مسائل بهینه‌سازی محدب پیچیدگی چندجمله‌ای دارد. مقدمه‌های عالی برای بهینه‌سازی محدب توسط بن‌تال و نمویروسکی (۲۰۰۱) و بوید و وندنبرگ (۲۰۰۴) ارائه شده.

کار سول رایت (۱۹۳۱) بر مفهوم چشم‌انداز برازش پیش‌آهنگ مهمی برای توسعهٔ الگوریتم‌های ژنتیک بود. در دههٔ ۱۹۵۰، چند آماردان، از جمله باکس (۱۹۵۷) و فریدمن (۱۹۵۹)، از فنون تکاملی برای مسائل بهینه‌سازی استفاده کردند، اما تا آنکه رشنبرگ (۱۹۶۵) راهبردهای تکاملی را برای حل مسائل بهینه‌سازی ایرفویل‌ها معرفی کرد، رویکرد محبوبیت نیافت. در دهه‌های ۱۹۶۰ و ۱۹۷۰، جان هالند (۱۹۷۵) الگوریتم‌های ژنتیک را تبلیغ کرد، هم به‌عنوان ابزار بهینه‌سازی مفید و هم به‌عنوان روشی برای گسترش فهم ما از سازگاری (هالند، ۱۹۹۵).

جنبش حیات مصنوعی (لنگتون، ۱۹۹۵) این ایده را یک گام فراتر برد و محصولات الگوریتم‌های ژنتیک را جانداران دید نه راه‌حل‌های مسئله. اثر بالدوینِ بحث‌شده در فصل تقریباً هم‌زمان توسط کانوی لوید مورگان (۱۸۹۶) و جیمز بالدوین (۱۸۹۶) پیشنهاد شد. شبیه‌سازی‌های رایانه‌ای به روشن‌کردن پیامدهایش کمک کرده‌اند (هینتون و نولان، ۱۹۸۷؛ اکلی و لیتمن، ۱۹۹۱؛ مورگان و گریفیتس، ۲۰۱۵). اسمیت و ساتماری (۱۹۹۹)، ریدلی (۲۰۰۴) و کارول (۲۰۰۷) زمینهٔ عمومی تکامل را ارائه می‌دهند.

بیشتر مقایسه‌های الگوریتم‌های ژنتیک با رویکردهای دیگر (به‌ویژه تپه‌نوردی تصادفی) نشان داده‌اند الگوریتم‌های ژنتیک کندتر همگرا می‌شوند (اوریلی و اوپاخر، ۱۹۹۴؛ میچل و همکاران، ۱۹۹۶؛ جوئلز و واتنبرگ، ۱۹۹۶؛ بالوپا، ۱۹۹۷). چنین یافته‌هایی در جامعهٔ GA جهان‌شمول محبوب نیستند، اما تلاش‌های اخیر درون آن جامعه برای فهم جستجوی جمعیت‌محور به‌عنوان شکل تقریبی یادگیری بیزی (فصل ۲۰ را ببینید) ممکن است به نزدیک‌کردن این حوزه به منتقدانش کمک کند (پلیکان و همکاران، ۱۹۹۹). نظریهٔ سامانه‌های دینامیکی درجه‌دوم نیز ممکن است عملکرد GAها را توضیح دهد (ربانی و همکاران، ۱۹۹۸). کاربردهای عملی چشمگیری برای GAها هست، در حوزه‌هایی به‌اندازهٔ طراحی آنتن (لون و همکاران، ۲۰۰۱)، طراحی به‌کمک رایانه (رنر و اکارت، ۲۰۰۳)، مدل‌های اقلیمی (استانیسواسکا و همکاران، ۲۰۱۵)، پزشکی (غاهری و همکاران، ۲۰۱۵) و طراحی شبکه‌های عصبی عمیق (میکولاینن و همکاران، ۲۰۱۹).

حوزهٔ برنامه‌نویسی ژنتیک زیرحوزه‌ای از الگوریتم‌های ژنتیک است که در آن بازنمایی‌ها برنامه‌اند نه رشته‌های بیتی. برنامه‌ها به‌صورت درخت‌های نحوی بازنمایی می‌شوند، یا در زبان برنامه‌نویسی استاندارد یا در قالب‌های ویژه‌طراحی‌شده برای بازنمایی مدارهای الکترونیکی، کنترل‌گرهای ربات و مانند آن. تلاقی شامل به‌هم‌دوختن زیردرخت‌ها چنان است که تضمین شود فرزندان عبارت‌های خوش‌ساخت‌اند. علاقه به برنامه‌نویسی ژنتیک توسط کار جان کوزا (۱۹۹۲، ۱۹۹۴) برانگیخته شد، اما دست‌کم به آزمایش‌های آغازین با کد ماشین توسط فریدبرگ (۱۹۵۸) و با اتوماتاهای متناهی توسط فوگل و همکاران (۱۹۶۶) بازمی‌گردد. مانند الگوریتم‌های ژنتیک، دربارهٔ اثربخشی این فن بحث هست. کوزا و همکاران (۱۹۹۹) آزمایش‌هایی در استفاده از برنامه‌نویسی ژنتیک برای طراحی دستگاه‌های مداری توصیف می‌کنند.

ژورنال‌های Evolutionary Computation و IEEE Transactions on Evolutionary Computation الگوریتم‌های تکاملی را پوشش می‌دهند؛ مقاله‌ها همچنین در Adaptive Behavior، Complex Systems و Artificial Life یافت می‌شوند. کنفرانس اصلی، کنفرانس محاسبات ژنتیک و تکاملی (GECCO) است. متون مرور خوب الگوریتم‌های ژنتیک شامل میچل (۱۹۹۶)، فوگل (۲۰۰۰)، لنگدون و پولی (۲۰۰۲) و پولی و همکاران (۲۰۰۸) است.

پیش‌بینی‌ناپذیری و مشاهده‌ناپذیری تا حدیِ محیط‌های واقعی در پروژه‌های رباتیک اولیه‌ای که از فنون برنامه‌ریزی استفاده می‌کردند، از جمله شیکی (فایکز و همکاران، ۱۹۷۲) و فردی (مایکی، ۱۹۷۲)، زود شناخته شد. مسائل پس از انتشار مقالهٔ تأثیرگذار مک‌درموت (۱۹۷۸) با عنوان «برنامه‌ریزی و کنش» توجه بیشتری گرفت.

نخستین کاری که به‌صراحت از درخت‌های AND–OR استفاده کرد به نظر می‌رسد برنامهٔ SAINT اسلگل برای انتگرال‌گیری نمادین بوده باشد که در فصل ۱ یاد شد. آمارل (۱۹۶۷) این ایده را بر اثبات قضیهٔ گزاره‌ای اعمال کرد، موضوعی که در فصل ۷ بحث می‌شود، و الگوریتم جستجویی شبیه AND-OR-GRAPH-SEARCH معرفی کرد. الگوریتم توسط نیلسون (۱۹۷۱) بیشتر توسعه یافت، که همچنین AO* را توصیف کرد — که چنان‌که نامش می‌گوید، راه‌حل‌های بهینه می‌یابد. AO* توسط مارتلی و مونتاناری (۱۹۷۳) بیشتر بهبود یافت.

AO* الگوریتمی بالا-به-پایین است؛ تعمیم پایین-به-بالای A*LD ،A* برای A* مشتقِ سبک‌ترین (فلزنسوالب و مک‌آلیستر، ۲۰۰۷) است. علاقه به جستجوی AND–OR در اوایل دههٔ ۲۰۰۰ احیا شد، با الگوریتم‌های تازه برای یافتن راه‌حل‌های چرخه‌ای (خیمنز و توراس، ۲۰۰۰؛ هانسن و زیلبرشتاین، ۲۰۰۱) و فنون تازه‌ای که از برنامه‌نویسی پویا الهام گرفته‌اند (بونت و گفنر، ۲۰۰۵).

ایدهٔ تبدیل مسائل تا حدی قابل مشاهده به مسائل حالت باور از آستروم (۱۹۶۵) برای حالت بسیار پیچیده‌تر عدم قطعیت احتمالاتی (فصل ۱۷ را ببینید) سرچشمه گرفت. اردمان و میسون (۱۹۸۸) مسئلهٔ دست‌کاری رباتیک بدون حسگر را با شکل پیوسته‌ای از جستجوی حالت باور مطالعه کردند. آن‌ها نشان دادند ممکن است قطعه‌ای روی میز را از موقعیت آغازین دلخواه با توالی خوب‌طراحی‌شده‌ای از کنش‌های کج‌کردن جهت دهیم. روش‌های عملی‌تری که بر سلسله‌ای از موانع موربِ دقیقاً جهت‌دار در عرض نوار نقاله مبتنی‌اند، از همان بینش‌های الگوریتمی استفاده می‌کنند (ویگلی و همکاران، ۱۹۹۶).

رویکرد حالت باور در زمینهٔ مسائل جستجوی بی‌حسگر و تا حدی قابل مشاهده توسط جنزِرت و نوربخش (۱۹۹۳) بازآفرینی شد. کار بیشتری بر مسائل بی‌حسگر در جامعهٔ برنامه‌ریزی مبتنی بر منطق انجام شد (گلدمن و بادی، ۱۹۹۶؛ اسمیت و ولد، ۱۹۹۸). این کار بر بازنمایی‌های موجز برای حالت‌های باور تأکید داشته، چنان‌که در فصل ۱۱ توضیح داده شده. بونت و گفنر (۲۰۰۰) نخستین اکتشافی‌های مؤثر برای جستجوی حالت باور را معرفی کردند؛ این‌ها توسط برایس و همکاران (۲۰۰۶) پالایش شدند. رویکرد افزایشی به جستجوی حالت باور، که در آن راه‌حل‌ها به‌صورت افزایشی برای زیرمجموعه‌هایی از حالت‌ها درون هر حالت باور ساخته می‌شوند، در ادبیات برنامه‌ریزی توسط کورین و همکاران (۲۰۰۲) مطالعه شد؛ چند الگوریتم افزایشی تازه برای مسائل ناجبرگرایانه و تا حدی قابل مشاهده توسط راسل و ولف (۲۰۰۵) معرفی شد. ارجاع‌های اضافی برای برنامه‌ریزی در محیط‌های تصادفی تا حدی قابل مشاهده در فصل ۱۷ می‌آید.

الگوریتم‌های کاوش فضاهای حالت ناشناخته برای قرن‌ها مورد علاقه بوده‌اند. جستجوی عمق-اول در هزارتوی برگشت‌پذیر را می‌توان با نگه‌داشتن دست چپ روی دیوار پیاده کرد؛ حلقه‌ها را می‌توان با علامت‌گذاری هر تقاطع اجتناب کرد. مسئلهٔ عمومی‌تر کاوش گراف‌های اویلری (یعنی گراف‌هایی که در آن‌ها هر گره شمار برابری یال ورودی و خروجی دارد) توسط الگوریتمی از هیرهولزر (۱۸۷۳) حل شد.

نخستین مطالعهٔ الگوریتمی کامل مسئلهٔ کاوش برای گراف‌های دلخواه توسط دنگ و پاپادیمیتریو (۱۹۹۰) انجام شد که الگوریتمی کاملاً عمومی توسعه دادند اما نشان دادند هیچ نسبت رقابتی کران‌داری برای کاوش گراف عمومی ممکن نیست. پاپادیمیتریو و یاناکاکیس (۱۹۹۱) پرسش یافتن مسیرهای هدف را در محیط‌های برنامه‌ریزی مسیر هندسی (که همهٔ کنش‌ها برگشت‌پذیرند) بررسی کردند. آن‌ها نشان دادند با موانع مربعی نسبت رقابتی کوچکی دست‌یافتنی است، اما با موانع مستطیلی عمومی هیچ نسبت کران‌داری ممکن نیست. (شکل ۴.۲۰ را ببینید.) در محیط پویا، حالت جهان می‌تواند بدون هیچ کنشی از عامل خودبه‌خود تغییر کند. برای نمونه، عامل می‌تواند مسیر رانندگی بهینه‌ای از A به B برنامه‌ریزی کند، اما تصادفی یا ترافیک ساعت شلوغیِ غیرعادی بد می‌تواند برنامه را خراب کند. الگوریتم‌های جستجوی افزایشی مانند A* مادام‌العمر (کونیگ و همکاران، ۲۰۰۴) و D*Lite (کونیگ و لیخاچف، ۲۰۰۲) با این وضعیت سروکار دارند.

الگوریتم LRTA* توسط کورف (۱۹۹۰) به‌عنوان بخشی از پژوهشی در جستجوی زمان-واقعی برای محیط‌هایی توسعه یافت که عامل باید پس از جستجو فقط برای مدت ثابتی کنش کند (وضعیتی رایج در بازی‌های دو-نفره). LRTA* در واقع حالت خاصی از الگوریتم‌های یادگیری تقویتی برای محیط‌های تصادفی است (بارتو و همکاران، ۱۹۹۵). سیاست خوش‌بینی در شرایط عدم قطعیتش — همیشه به سوی نزدیک‌ترین حالت بازدیدنشده رفتن — می‌تواند به الگوی کاوشی بینجامد که در حالت بی‌اطلاع از جستجوی عمق-اول ساده ناکارآمدتر است (کونیگ، ۲۰۰۰). داسگوپتا و همکاران (۱۹۹۴) نشان می‌دهند جستجوی تعمیق تکراری آنلاین برای یافتن هدف در درخت یکنواخت بدون اطلاعات اکتشافی بهینه‌کاراست.

پانوشت‌ها:

[1] لوبی و همکاران (۱۹۹۳) پیشنهاد می‌کنند پس از شمار ثابتی گام، راه‌اندازی مجدد انجام دهیم و نشان می‌دهند این می‌تواند بسیار کارآمدتر از رهاکردن هر جستجو برای ادامهٔ بی‌نهایت باشد.

[2] دانش بردارها، ماتریس‌ها و مشتق‌ها برای این بخش مفید است (پیوست الف را ببینید).

[3] در حالت کلی، به‌روزرسانی نیوتن–رافسون را می‌توان چنان دید که سطحی درجه‌دوم را بر f در x برازش می‌دهد و سپس مستقیم به کمینهٔ آن سطح می‌رود — که اگر f درجه‌دوم باشد، همان کمینهٔ f است.

[4] مجموعهٔ S از نقطه‌ها محدب است اگر خطِ واصل هر دو نقطه در S نیز در S باشد. تابع محدب تابعی است که فضای «بالای» آن مجموعهٔ محدبی بسازد؛ طبق تعریف، توابع محدب هیچ کمینهٔ محلی (در برابر سراسری) ندارند.

[5] فرض می‌کنیم بیشتر خوانندگان با مسائل مشابه روبه‌رویند و می‌توانند با عامل ما همدردی کنند. از دارندگان لوازم تمیزکنندهٔ مدرن و کارآمد که نمی‌توانند از این ابزار آموزشی بهره ببرند پوزش می‌خواهیم.

[6] در محیط کاملاً قابل مشاهده، هر حالت باور شامل یک حالت فیزیکی است. پس می‌توانیم الگوریتم‌های فصل ۳ را جستجو در فضای حالت باوری از حالت‌های باورِ تکی ببینیم.

[7] پوزش‌های معمول به کسانی که با اثر کودکان کوچک بر محیط آشنا نیستند.

[8] اصطلاح «آنلاین» در این‌جا به الگوریتم‌هایی اشاره دارد که باید ورودی را هنگام دریافت پردازش کنند نه آنکه منتظر در دسترس‌شدن کل مجموعه‌دادهٔ ورودی بمانند. این کاربرد «آنلاین» با مفهوم «داشتن اتصال اینترنت» بی‌ربط است.

[9] گام‌های تصادفی بر شبکه‌های بی‌نهایت یک‌بعدی و دوبعدی کامل‌اند. بر شبکهٔ سه‌بعدی، احتمال بازگشتِ هرگزِ گام به نقطهٔ آغازین فقط حدود ۰٫۳۴۰۵ است (هیوز، ۱۹۹۵).

فصل ۵: جستجوی رقابتی و بازی‌ها (Adversarial Search and Games)

در این فصل محیط‌هایی را می‌کاویم که در آن‌ها عامل‌های دیگر علیه ما توطئه می‌کنند.

در این فصل محیط‌های رقابتی را پوشش می‌دهیم که در آن‌ها دو یا چند عامل هدف‌های متعارض دارند و این به مسائل جستجوی رقابتی (Adversarial Search) می‌انجامد. به‌جای سروکار داشتن با هرج‌ومرج درگیری‌های جهان واقعی، بر بازی‌ها تمرکز می‌کنیم: شطرنج، گو، پوکر و مانند آن. برای پژوهشگران هوش مصنوعی، سرشت ساده‌شدهٔ این بازی‌ها یک امتیاز است: حالت بازی به‌راحتی بازنمایی می‌شود و عامل‌ها معمولاً به شمار کمی کنش محدودند که اثراتشان با قواعد دقیق تعریف می‌شود. بازی‌های فیزیکی مانند کروکت و هاکی روی یخ توصیف‌های پیچیده‌تری دارند، گسترهٔ بزرگ‌تری از کنش‌های ممکن، و قواعد نسبتاً نادقیقی برای تعریف قانونی‌بودن کنش‌ها. به‌جز فوتبال رباتیک، این بازی‌های فیزیکی چندان توجه جامعهٔ هوش مصنوعی را جلب نکرده‌اند.

۵.۱ نظریهٔ بازی

دست‌کم سه نگرش می‌توانیم به محیط‌های چندعاملی داشته باشیم. نگرش نخست، مناسب وقتی که شمار بسیار زیادی عامل هست، آن است که آن‌ها را به‌صورت مجموعی به‌مثابه یک اقتصاد (Economy) در نظر بگیریم؛ این به ما امکان می‌دهد چیزهایی مانند افزایش تقاضا → افزایش قیمت‌ها را پیش‌بینی کنیم، بدون آنکه کنش هیچ عامل فردی را پیش‌بینی کنیم.

دوم، می‌توانیم عامل‌های رقیب را فقط بخشی از محیط بدانیم — بخشی که محیط را ناجبرگرایانه می‌کند. اما اگر رقبا را همان‌گونه مدل کنیم که مثلاً باران را (که گاهی می‌بارد و گاهی نمی‌بارد)، این ایده را از دست می‌دهیم که رقبای ما فعالانه می‌کوشند ما را شکست دهند، حال آنکه باران ظاهراً چنین نیتی ندارد.

نگرش سوم آن است که عامل‌های رقیب را به‌صراحت با فنون جستجوی درخت بازی رقابتی مدل کنیم. این همان چیزی است که این فصل پوشش می‌دهد. با ردهٔ محدودی از بازی‌ها آغاز می‌کنیم و حرکت بهینه و الگوریتمی برای یافتنش تعریف می‌کنیم: جستجوی مینیمکس (Minimax)، تعمیمی از جستجوی AND–OR (از شکل ۴.۱۱). نشان می‌دهیم هرس (Pruning) با نادیده‌گرفتن بخش‌هایی از درخت جستجو که هیچ تفاوتی در حرکت بهینه ایجاد نمی‌کنند، جستجو را کارآمدتر می‌کند. برای بازی‌های غیربدیهی، معمولاً زمان کافی برای اطمینان از یافتن حرکت بهینه نخواهیم داشت (حتی با هرس)؛ باید در نقطه‌ای جستجو را قطع کنیم.

برای هر حالتی که انتخاب می‌کنیم جستجو را در آن متوقف کنیم، می‌پرسیم چه کسی برنده است. برای پاسخ به این پرسش انتخابی داریم: می‌توانیم تابع ارزیابی اکتشافی را اعمال کنیم تا بر اساس ویژگی‌های حالت تخمین بزنیم چه کسی برنده است (بخش ۵.۳)، یا می‌توانیم میانگین پیامدهای بسیاری شبیه‌سازی سریع بازی از آن حالت تا پایان را بگیریم (بخش ۵.۴).

بخش ۵.۵ بازی‌های شامل عنصر شانس را بحث می‌کند (از طریق ریختن تاس یا به‌هم‌زدن ورق) و بخش ۵.۶ بازی‌های اطلاعات ناقص (مانند پوکر و بریج که همهٔ ورق‌ها برای همهٔ بازیکنان قابل مشاهده نیست) را پوشش می‌دهد.

۵.۱.۱ بازی‌های دونفرهٔ جمع-صفر

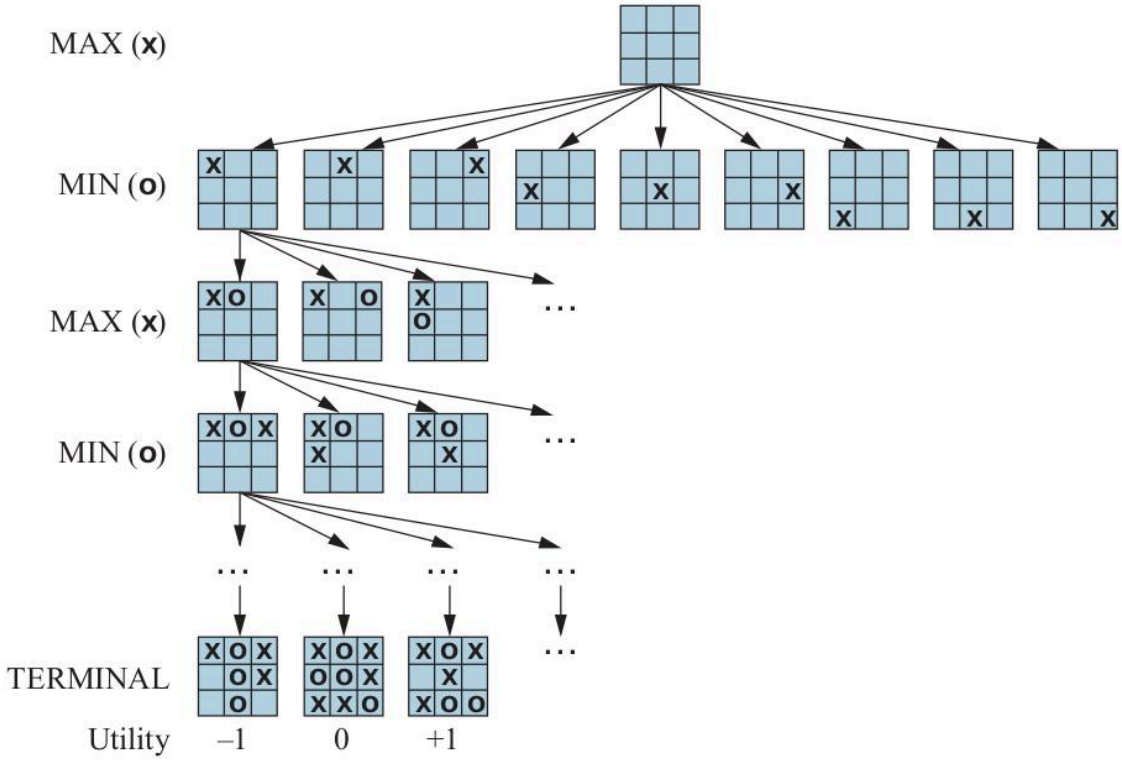
بازی‌هایی که بیشتر در هوش مصنوعی مطالعه می‌شوند (مانند شطرنج و گو) همان‌هایی هستند که نظریه‌پردازان بازی «جبرگرایانه، دونفره، نوبتی، اطلاعات کامل، جمع-صفر» می‌نامند. «اطلاعات کامل» (Perfect Information) مترادف «کاملاً قابل مشاهده» است[1] و «جمع-صفر» (Zero-Sum) بدان معناست که آنچه برای یک بازیکن خوب است دقیقاً به همان اندازه برای دیگری بد است: هیچ پیامد «برد-برد»ی نیست. برای بازی‌ها اغلب از اصطلاح «حرکت» (Move) به‌عنوان مترادف «کنش» و «موضع» (Position) به‌عنوان مترادف «حالت» استفاده می‌کنیم.

دو بازیکن خود را MAX و MIN می‌نامیم، به دلایلی که به‌زودی آشکار خواهد شد. MAX نخست حرکت می‌کند و سپس بازیکنان نوبتی حرکت می‌کنند تا بازی تمام شود. در پایان بازی، به بازیکن برنده امتیاز داده می‌شود و به بازنده جریمه. بازی را می‌توان به‌صورت صوری با عناصر زیر تعریف کرد:

- **S0:** حالت آغازین، که مشخص می‌کند بازی در آغاز چگونه چیده شده.
- **TO-MOVE(s):** بازیکنی که در حالت s نوبت حرکت اوست.
- **ACTIONS(s):** مجموعهٔ حرکت‌های قانونی در حالت s.
- **RESULT(s, a):** مدل گذار (Transition Model) که حالتی را تعریف می‌کند که از برداشتن کنش a در حالت s حاصل می‌شود.
- **IS-TERMINAL(s):** آزمون پایانی (Terminal Test) که وقتی بازی تمام شده true و وگرنه false است. حالت‌هایی که بازی در آن‌ها تمام شده، حالت‌های پایانی (Terminal States) نامیده می‌شوند.
- **UTILITY(s, p):** تابع مطلوبیت (که تابع هدف یا تابع بازده نیز نامیده می‌شود) که مقدار عددی نهایی را برای بازیکن p وقتی بازی در حالت پایانی s تمام می‌شود تعریف می‌کند.

در شطرنج، پیامد برد، باخت یا تساوی است با مقادیر ۱، ۰ یا ۱/۲.[2] برخی بازی‌ها گسترهٔ وسیع‌تری از پیامدهای ممکن دارند — برای نمونه، بازده‌های تخته‌نرد از ۰ تا ۱۹۲ می‌روند. بسیار شبیه فصل ۳، حالت آغازین، تابع ACTIONS و تابع RESULT «گراف فضای حالت» را تعریف می‌کنند — گرافی که رأس‌هایش حالت‌ها، یال‌هایش حرکت‌ها و هر حالتی ممکن است با چند مسیر رسیده شود. مانند فصل ۳، می‌توانیم «درخت جستجو» را بر بخشی از آن گراف سوار کنیم تا تعیین کنیم چه حرکتی بکنیم. «درخت بازی کامل» (Game Tree) را به‌عنوان درخت جستجویی تعریف می‌کنیم که هر توالی حرکت را تا حالت پایانی دنبال می‌کند. درخت بازی ممکن است بی‌نهایت باشد اگر خودِ فضای حالت بی‌کران باشد یا قواعد بازی امکان تکرار بی‌نهایت موضع‌ها را بدهند.

شکل ۵.۱ بخشی از درخت بازی برای دوز (tic-tac-toe) را نشان می‌دهد. از حالت آغازین، MAX نه حرکت ممکن دارد. بازی میان گذاشتن X توسط MAX و O توسط MIN در نوسان است تا آنکه به گره‌های برگِ متناظر با حالت‌های پایانی برسیم، چنان‌که یک بازیکن سه خانه در یک ردیف داشته باشد یا همهٔ خانه‌ها پر شوند. عدد روی هر گرهٔ برگ، مقدار مطلوبیت حالت پایانی را از دیدگاه MAX نشان می‌دهد؛ مقادیر بالا برای MAX خوب و برای MIN بد است (و بدین ترتیب بازیکنان نام‌هایشان را می‌گیرند).



شکل ۵.۱ – درخت بازی (بخشی از) برای بازی دوز. گرهٔ بالایی حالت آغازین است و MAX نخست حرکت می‌کند و X را در خانهٔ خالی می‌گذارد. بخشی از درخت را نشان می‌دهیم با حرکت‌های متناوب MIN (O) و MAX (X)، تا آنکه سرانجام به حالت‌های پایانی می‌رسیم که می‌توان بر اساس قواعد بازی به آن‌ها مطلوبیت نسبت داد.

برای دوز، درخت بازی نسبتاً کوچک است – کمتر از 9! = ۳۶۲,۸۸۰ گرهٔ پایانی (با فقط ۵,۴۷۸ حالت متمایز). اما برای شطرنج بیش از 10^{40} گره هست، پس درخت بازی را بهتر است سازه‌ای نظری دانست که نمی‌توانیم در جهان فیزیکی محقق کنیم.

۵.۲ تصمیم‌های بهینه در بازی‌ها

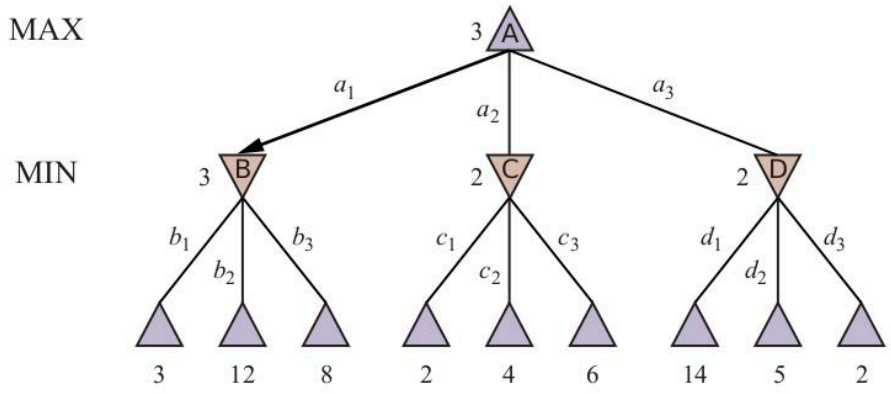
MAX می‌خواهد توالی کنشی بیابد که به برد می‌انجامد، اما MIN حرفی برای گفتن دارد. این بدان معناست که راهبرد MAX باید برنامهٔ شرطی باشد – راهبردی احتیاطی که پاسخی برای هر یک از حرکت‌های ممکن MIN مشخص می‌کند. در بازی‌هایی که پیامد دودویی دارند (برد یا باخت)، می‌توانیم از جستجوی AND–OR (صفحهٔ ۱۲۵) برای تولید برنامهٔ

شرطی استفاده کنیم. در واقع، برای چنین بازی‌هایی، تعریف راهبرد برنده برای بازی با تعریف راه‌حل برای مسئلهٔ برنامه‌ریزی ناجبرگرایانه یکسان است: در هر دو مورد، پیامد مطلوب باید تضمین شود، صرف‌نظر از آنچه «طرف دیگر» می‌کند. برای بازی‌های با چند امتیاز پیامد، به الگوریتم کمی کلی‌تر به نام جستجوی مینیمکس نیاز داریم.

جستجوی مینیمکس (Minimax Search): بازی پیش‌پاافتادهٔ شکل ۵.۲ را در نظر بگیرید. حرکت‌های ممکن برای MAX در گرهٔ ریشه a1، a2 و a3 برچسب‌گذاری شده‌اند. پاسخ‌های ممکن به a1 برای MIN عبارت‌اند از b1، b2، b3 و الی آخر. این بازی خاص پس از یک حرکت از هر یک از MAX و MIN پایان می‌یابد. (توجه: در برخی بازی‌ها، واژهٔ «حرکت» یعنی هر دو بازیکن کنشی برداشته‌اند؛ پس از واژهٔ «نیم‌حرکت» (Ply) برای معنای بدون‌ابهامِ یک حرکت توسط یک بازیکن استفاده می‌شود که ما را یک سطح در درخت بازی عمیق‌تر می‌کند.) مطلوبیت‌های حالت‌های پایانی این بازی از ۲ تا ۱۴ می‌روند.

با درخت بازیِ داده‌شده، راهبرد بهینه را می‌توان با محاسبهٔ «مقدار مینیمکس» هر حالت در درخت تعیین کرد که آن را MINIMAX(s) می‌نویسیم. مقدار مینیمکس، مطلوبیت (برای MAX) بودن در آن حالت است، با این فرض که هر دو بازیکن از آن‌جا تا پایان بازی بهینه بازی می‌کنند. مقدار مینیمکس حالت پایانی فقط مطلوبیتش است. در حالت غیرپایانی، MAX ترجیح می‌دهد وقتی نوبت اوست به حالتی با بیشینهٔ مقدار حرکت کند و MIN حالتی با کمینهٔ مقدار را ترجیح می‌دهد (یعنی کمینهٔ مقدار برای MAX و بنابراین بیشینهٔ مقدار برای MIN). پس داریم:

```
MINIMAX(s) =
    UTILITY(s, MAX)                                  اگر IS-TERMINAL(s)
    max_{a ∈ Actions(s)} MINIMAX(RESULT(s, a))       اگر TO-MOVE(s) = MAX
    min_{a ∈ Actions(s)} MINIMAX(RESULT(s, a))       اگر TO-MOVE(s) = MIN
```



شکل ۵.۲ – درخت بازی دونیم‌حرکته. گره‌های △ «گره‌های MAX» هستند که نوبت حرکت MAX است و گره‌های ▽ «گره‌های MIN». گره‌های پایانی مقادیر مطلوبیت را برای MAX نشان می‌دهند؛ بقیهٔ گره‌ها با مقادیر مینیمکس‌شان برچسب‌گذاری شده‌اند. بهترین حرکت MAX در ریشه a1 است، زیرا به حالتی با بیشترین مقدار مینیمکس می‌انجامد، و بهترین پاسخ MIN b1 است، زیرا به حالتی با کمترین مقدار مینیمکس می‌انجامد.

این تعریف‌ها را بر درخت بازی شکل ۵.۲ اعمال کنیم. گره‌های پایانی سطح پایین مقادیر مطلوبیت‌شان را از تابع UTILITY بازی می‌گیرند. نخستین گرهٔ MIN، با برچسب B، سه حالت جانشین با مقادیر ۳، ۱۲ و ۸ دارد، پس مقدار مینیمکسش ۳ است. به همین ترتیب، دو گرهٔ MIN دیگر مقدار مینیمکس ۲ دارند. گرهٔ ریشه گرهٔ MAX است؛

حالت‌های جانشینش مقادیر مینیمکس ۳، ۲ و ۲ دارند؛ پس مقدار مینیمکس ۳ دارد. همچنین می‌توانیم «تصمیم مینیمکس» (Minimax Decision) را در ریشه شناسایی کنیم: کنش a1 انتخاب بهینه برای MAX است، زیرا به حالتی با بیشترین مقدار مینیمکس می‌انجامد.

این تعریف از بازی بهینه برای MAX فرض می‌کند MIN نیز بهینه بازی می‌کند. اگر MIN بهینه بازی نکند چه؟ آنگاه MAX دست‌کم به خوبیِ بازی‌کردن در برابر بازیکن بهینه عمل خواهد کرد، احتمالاً بهتر. اما این بدان معنا نیست که هنگام رویارویی با حریف فروبهینه، همیشه بهترین کار بازی‌کردنِ حرکت بهینهٔ مینیمکس است. موقعیتی را در نظر بگیرید که بازی بهینهٔ هر دو طرف به تساوی می‌انجامد، اما برای MAX یک حرکت پرریسک هست که به حالتی می‌انجامد که در آن ۱۰ پاسخ ممکن از MIN هست که همگی معقول می‌نمایند، اما ۹ تای آن‌ها باخت برای MIN است و یکی باخت برای MAX. اگر MAX باور کند MIN توان محاسباتی کافی برای کشف حرکت بهینه را ندارد، شاید بخواهد حرکت پرریسک را امتحان کند، به این دلیل که شانس 9/10 برد از تساوی قطعی بهتر است.

۵.۲.۱ الگوریتم جستجوی مینیمکس

حال که می‌توانیم MINIMAX(s) را محاسبه کنیم، می‌توانیم آن را به الگوریتم جستجویی تبدیل کنیم که بهترین حرکت را برای MAX با آزمودن همهٔ کنش‌ها و برگزیدن کنشی که حالت حاصلش بیشترین مقدار MINIMAX را دارد می‌یابد. شکل ۵.۳ الگوریتم را نشان می‌دهد. الگوریتمی بازگشتی است که تمام‌راه تا برگ‌های درخت پیش می‌رود و سپس هنگام بازشدن بازگشت، مقادیر مینیمکس را در درخت به عقب پشتیبان می‌گیرد. برای نمونه، در شکل ۵.۲، الگوریتم نخست به سه گرهٔ پایین-چپ بازگشت می‌کند و از تابع UTILITY روی آن‌ها استفاده می‌کند تا کشف کند مقادیرشان به‌ترتیب ۳، ۱۲ و ۸ است. سپس کمینهٔ این مقادیر، یعنی ۳، را می‌گیرد و آن را به‌عنوان مقدار پشتیبان‌گرفته‌شدهٔ گرهٔ B بازمی‌گرداند. فرایند مشابهی مقادیر پشتیبان‌گرفته‌شدهٔ ۲ برای C و ۲ برای D می‌دهد. سرانجام، بیشینهٔ ۳، ۲ و ۲ را می‌گیریم تا مقدار پشتیبان‌گرفته‌شدهٔ ۳ برای گرهٔ ریشه به دست آید.

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
      v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
      if v2 > v then v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
      v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
      if v2 < v then v, move ← v2, a
    return v, move
```

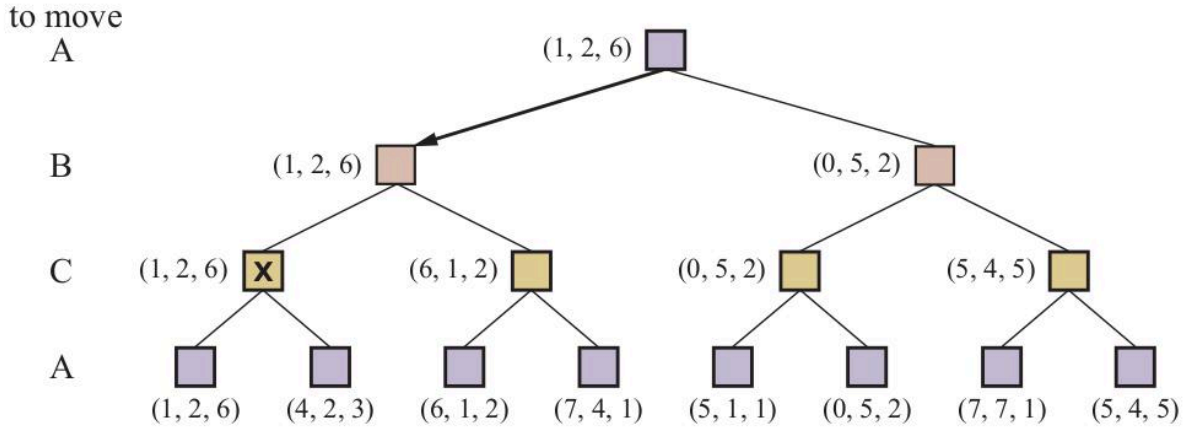
شکل ۵.۳ – الگوریتم محاسبهٔ حرکت بهینه با مینیمکس – حرکتی که به حالت پایانی با بیشینهٔ مطلوبیت می‌انجامد، با این فرض که حریف برای کمینه‌کردن مطلوبیت بازی می‌کند. توابع MAX-VALUE و MIN-VALUE تمام درخت بازی را تا برگ‌ها می‌پیمایند تا مقدار پشتیبان‌گرفته‌شدهٔ حالت و حرکتی که به آن‌جا می‌رسد را تعیین کنند.

الگوریتم مینیمکس کاوش عمق-اول کاملی از درخت بازی انجام می‌دهد. اگر بیشینهٔ عمق درخت m باشد و در هر نقطه b حرکت قانونی باشد، پیچیدگی زمانی الگوریتم مینیمکس $O(b^m)$ است. پیچیدگی فضایی برای الگوریتمی که همهٔ کنش‌ها را یک‌جا تولید می‌کند $O(bm)$ است، یا $O(m)$ برای الگوریتمی که کنش‌ها را یکی‌یکی تولید می‌کند (صفحهٔ ۸۰ را ببینید). پیچیدگی نمایی MINIMAX را برای بازی‌های پیچیده غیرعملی می‌کند؛ برای نمونه، شطرنج ضریب انشعابی حدود ۳۵ دارد و میانگین بازی عمقی حدود ۸۰ نیم‌حرکت دارد، و جستجوی 35^{80} ≈ 10^{123} حالت شدنی نیست. MINIMAX اما به‌عنوان مبنایی برای تحلیل ریاضی بازی‌ها عمل می‌کند. با تقریب زدن تحلیل مینیمکس به شیوه‌های گوناگون، می‌توانیم الگوریتم‌های عملی‌تری مشتق کنیم.

۵.۲.۲ تصمیم‌های بهینه در بازی‌های چندبازیکنه

بسیاری از بازی‌های محبوب بیش از دو بازیکن دارند. بگذارید بررسی کنیم چگونه ایدهٔ مینیمکس را به بازی‌های چندبازیکنه بسط دهیم. این از دیدگاه فنی سرراست است، اما مسائل مفهومی تازهٔ جالبی برمی‌انگیزد.

نخست، باید مقدار تکی هر گره را با برداری از مقادیر جایگزین کنیم. برای نمونه، در بازی سه‌بازیکنه با بازیکنان A، B و C، بردار ⟨vA,vB,vC⟩ با هر گره همراه می‌شود. برای حالت‌های پایانی، این بردار مطلوبیت حالت را از دیدگاه هر بازیکن می‌دهد. (در بازی‌های دوبازیکنهٔ جمع-صفر، بردار دوعنصری را می‌توان به مقدار تکی کاهش داد، زیرا مقادیر همیشه متضادند.) ساده‌ترین راه پیاده‌سازی این است که تابع UTILITY بردار مطلوبیت‌ها را بازگرداند.



شکل ۵.۴ – سه نیم‌حرکت نخست درخت بازی با سه بازیکن (A، B، C). هر گره با مقادیری از دیدگاه هر بازیکن برچسب‌گذاری شده. بهترین حرکت در ریشه علامت‌گذاری شده.

حال باید حالت‌های غیرپایانی را در نظر بگیریم. گرهٔ علامت‌گذاری‌شده با X در درخت بازی شکل ۵.۴ را در نظر بگیرید. در آن حالت، بازیکن C انتخاب می‌کند چه کند. دو انتخاب به حالت‌های پایانی با بردارهای مطلوبیت ⟨vA=1,vB=2,vC=6⟩ و ⟨vA=4,vB=2,vC=3⟩ می‌انجامند. چون ۶ از ۳ بزرگ‌تر است، C باید نخستین حرکت را برگزیند. این بدان معناست اگر

حالت X رسیده شود، بازی بعدی به حالت پایانی با مطلوبیت‌های ⟨vA=1,vB=2,vC=6⟩ می‌انجامد. از این رو، مقدار پشتیبان‌گرفته‌شدهٔ X این بردار است. در حالت کلی، مقدار پشتیبان‌گرفته‌شدهٔ گرهٔ n، بردار مطلوبیتِ حالت جانشینی است که بیشترین مقدار را برای بازیکنی که در n انتخاب می‌کند دارد.

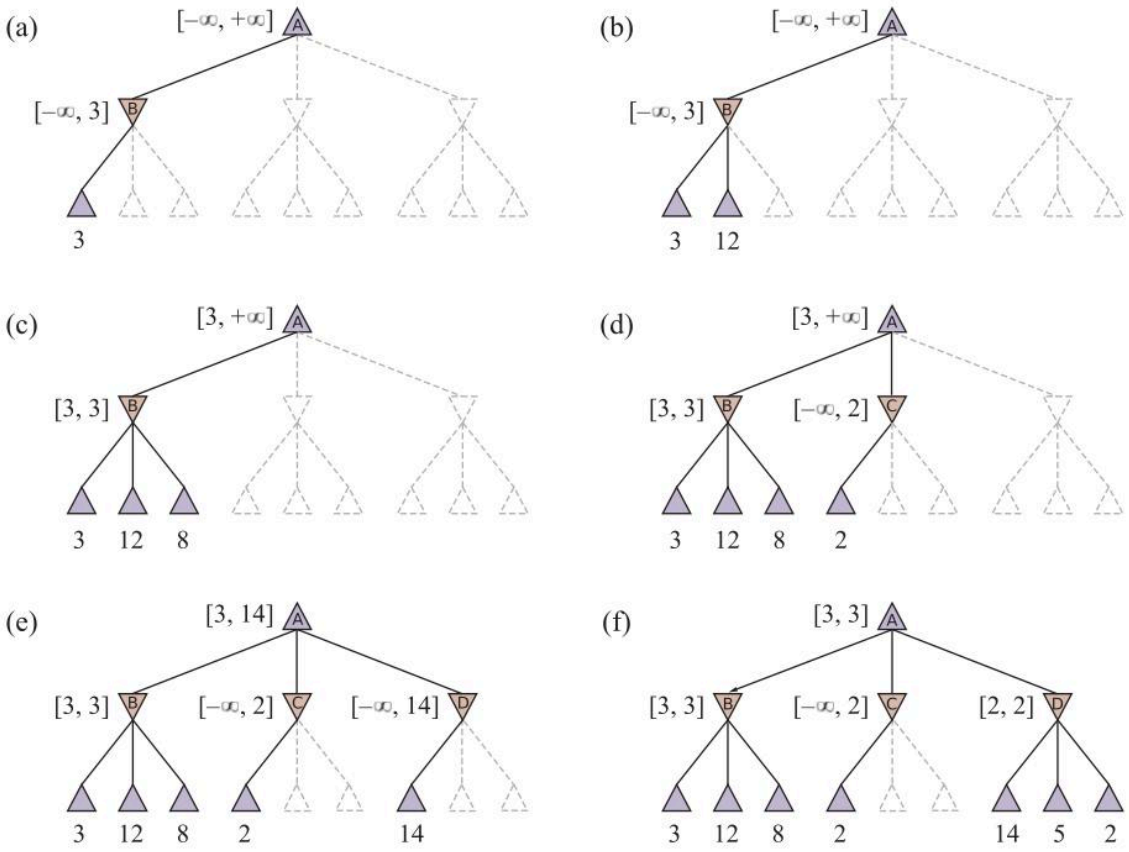
هر کسی که بازی‌های چندبازیکنه مانند دیپلماسی یا «مهاجران کاتان» بازی کند، به‌سرعت متوجه می‌شود خیلی چیزهای بیشتری از بازی‌های دوبازیکنه در جریان است. بازی‌های چندبازیکنه معمولاً شامل اتحادها (Alliances) — رسمی یا غیررسمی — میان بازیکنان‌اند. اتحادها با پیش‌رفتن بازی بسته و شکسته می‌شوند. چگونه باید چنین رفتاری را فهمید؟ آیا اتحادها پیامد طبیعی راهبردهای بهینه برای هر بازیکن در بازی چندبازیکنه‌اند؟ معلوم می‌شود می‌توانند باشند. برای نمونه، فرض کنیم A و B در مواضع ضعیف‌اند و C در موضع قوی‌تر. آنگاه اغلب برای هر دو A و B بهینه است که به C حمله کنند نه به همدیگر، مبادا C هر یک را جداگانه نابود کند. بدین ترتیب، همکاری از رفتار کاملاً خودخواهانه پدیدار می‌شود. البته به‌محض آنکه C زیر هجوم مشترک ضعیف شود، اتحاد ارزشش را از دست می‌دهد و A یا B می‌توانند توافق را نقض کنند. در برخی موارد، اتحادهای صریح فقط آنچه به هر حال رخ می‌داد را عینیت می‌بخشند. در موارد دیگر، انگ اجتماعی به شکستن اتحاد چسبیده، پس بازیکنان باید مزیت فوری شکستن اتحاد را در برابر زیان بلندمدتِ نادیده‌دیده‌شدن به‌عنوان فردی غیرقابل‌اعتماد بسنجند. برای این پیچیدگی‌ها بخش ۱۸.۲ را ببینید.

اگر بازی جمع-صفر نباشد، همکاری می‌تواند حتی با فقط دو بازیکن رخ دهد. فرض کنیم، برای نمونه، حالت پایانی با مطلوبیت‌های ⟨vA=1000,vB=1000⟩ هست و ۱۰۰۰ بیشترین مطلوبیت ممکن برای هر بازیکن است. آنگاه راهبرد بهینه برای هر دو بازیکن آن است که هر کاری ممکن است برای رسیدن به این حالت انجام دهند — یعنی بازیکنان خودکار برای دستیابی به هدفی دوجانبهٔ مطلوب همکاری خواهند کرد.

۵.۲.۳ هرس آلفا–بتا

شمار حالت‌های بازی در عمق درخت نمایی است. هیچ الگوریتمی نمی‌تواند نما را کامل حذف کند، اما گاهی می‌توانیم آن را نصف کنیم: محاسبهٔ تصمیم درست مینیمکس بدون بررسی هر حالت، با هرس (صفحهٔ ۹۰ را ببینید) بخش‌های بزرگی از درخت که هیچ تفاوتی در پیامد ایجاد نمی‌کنند. فن خاصی که بررسی می‌کنیم «هرس آلفا–بتا» (Alpha–Beta Pruning) نامیده می‌شود.

درخت بازی دونیم‌حرکتهٔ شکل ۵.۲ را دوباره در نظر بگیرید. بار دیگر محاسبهٔ تصمیم بهینه را مرور کنیم، این بار با توجه دقیق به آنچه در هر نقطه از فرایند می‌دانیم. گام‌ها در شکل ۵.۵ توضیح داده شده‌اند. پیامد آن است که می‌توانیم تصمیم مینیمکس را بدون ارزیابی هرگزِ دو تا از گره‌های برگ شناسایی کنیم.



شکل ۵.۵ – مراحل محاسبهٔ تصمیم بهینه برای درخت بازی شکل ۵.۲. در هر نقطه، گسترهٔ مقادیر ممکن برای هر گره را نشان می‌دهیم. (الف) نخستین برگ زیر B مقدار ۳ دارد. از این رو B، که گرهٔ MIN است، مقداری حداکثر ۳ دارد. (ب) دومین برگ زیر B مقدار ۱۲ دارد؛ MIN از این حرکت می‌پرهیزد، پس مقدار B همچنان حداکثر ۳ است. (ج) سومین برگ زیر B مقدار ۸ دارد؛ همهٔ حالت‌های جانشین B را دیده‌ایم، پس مقدار B دقیقاً ۳ است. حال می‌توانیم استنتاج کنیم مقدار ریشه دست‌کم ۳ است، زیرا MAX در ریشه انتخابی به‌ارزش ۳ دارد. (د) نخستین برگ زیر C مقدار ۲ دارد. از این رو C، که گرهٔ MIN است، مقداری حداکثر ۲ دارد. اما می‌دانیم B به‌ارزش ۳ است، پس MAX هرگز C را برنمی‌گزیند. بنابراین دلیلی برای نگاه‌کردن به سایر حالت‌های جانشین C نیست. این نمونه‌ای از هرس آلفا–بتاست. (هـ) نخستین برگ زیر D مقدار ۱۴ دارد، پس D حداکثر ۱۴ می‌ارزد. این همچنان از بهترین بدیل MAX (یعنی ۳) بالاتر است، پس باید کاوش حالت‌های جانشین D را ادامه دهیم. همچنین توجه کنید اکنون برای همهٔ جانشین‌های ریشه کران داریم، پس مقدار ریشه نیز حداکثر ۱۴ است. (و) دومین جانشین D به‌ارزش ۵ است، پس باز باید کاوش را ادامه دهیم. سومین جانشین به‌ارزش ۲ است، پس اکنون D دقیقاً ۲ می‌ارزد. تصمیم MAX در ریشه حرکت به B است، با مقدار ۳.

راه دیگر نگاه‌کردن به این، ساده‌سازی فرمول MINIMAX است. دو جانشین ارزیابی‌نشدهٔ گرهٔ C در شکل ۵.۵ را x و y بگذارید. آنگاه مقدار گرهٔ ریشه چنین است:

```
MINIMAX(root) = max(min(3, 12, 8), min(2, x, y), min(14, 5, 2))
              = max(3, min(2, x, y), 2)
              = max(3, z, 2)   که z = min(2, x, y) ≤ 2
              = 3
```

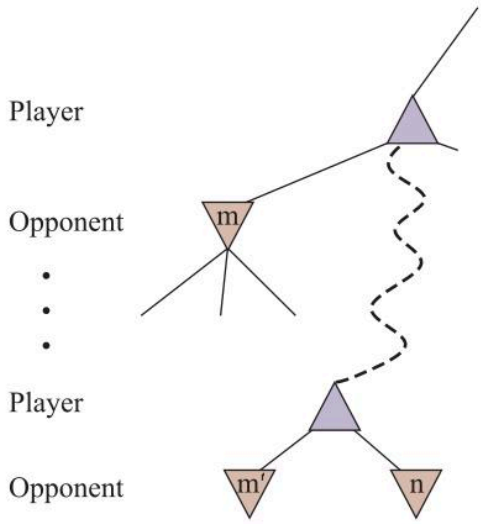
به بیان دیگر، مقدار ریشه و در نتیجه تصمیم مینیمکس مستقل از مقادیر برگ‌های x و y است و بنابراین می‌توان آن‌ها را هرس کرد.

هرس آلفا–بتا را می‌توان بر درخت‌های هر عمقی اعمال کرد و اغلب می‌توان کل زیردرخت‌ها را هرس کرد نه فقط برگ‌ها را. اصل کلی چنین است: گرهٔ n را در جایی از درخت در نظر بگیرید (شکل ۵.۶ را ببینید)، چنان‌که بازیکن انتخابی برای حرکت به n دارد. اگر بازیکن انتخاب بهتری یا در همان سطح (مثلاً m' در شکل ۵.۶) یا در هر نقطه‌ای بالاتر در درخت (مثلاً m در شکل ۵.۶) داشته باشد، بازیکن هرگز به n حرکت نخواهد کرد. پس به‌محض آنکه به اندازهٔ کافی دربارهٔ n فهمیدیم (با بررسی برخی از نوادگانش) تا به این نتیجه برسیم، می‌توانیم آن را هرس کنیم.

به خاطر بسپارید جستجوی مینیمکس عمق-اول است، پس در هر زمان فقط باید گره‌های طول یک مسیر واحد در درخت را در نظر بگیریم. هرس آلفا–بتا نامش را از دو پارامتر اضافی در MAX-VALUE(state, α, β) (شکل ۵.۷) می‌گیرد که کران‌هایی بر مقادیر پشتیبان‌گرفته‌شده‌ای توصیف می‌کنند که در هر جای مسیر ظاهر می‌شوند:

- **α** = مقدار بهترین (یعنی بیشترین‌مقدار) انتخابی که تاکنون در هر نقطهٔ انتخاب در طول مسیر برای MAX یافته‌ایم. بیندیشید: α = «دست‌کم».
- **β** = مقدار بهترین (یعنی کمترین‌مقدار) انتخابی که تاکنون در هر نقطهٔ انتخاب در طول مسیر برای MIN یافته‌ایم. بیندیشید: β = «حداکثر».

جستجوی آلفا–بتا مقادیر α و β را هنگام پیش‌رفتن به‌روز می‌کند و به‌محض اینکه دانسته شود مقدار گرهٔ کنونی از مقدار α یا β کنونی برای MAX یا MIN بدتر است، شاخه‌های باقی‌مانده را در یک گره هرس می‌کند (یعنی فراخوانی بازگشتی را خاتمه می‌دهد). الگوریتم کامل در شکل ۵.۷ داده شده. شکل ۵.۵ پیشرفت الگوریتم را روی درخت بازی دنبال می‌کند.



شکل ۵.۶ – حالت عمومی هرس آلفا–بتا. اگر m یا m' برای بازیکن از n بهتر باشد، هرگز در بازی به n نخواهیم رسید.

```
function ALPHA-BETA-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state, −∞, +∞)
    return move

function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
      v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
      if v2 > v then v, move ← v2, a
        α ← MAX(α, v)
      if v ≥ β then return v, move
    return v, move

function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
      v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
      if v2 < v then v, move ← v2, a
        β ← MIN(β, v)
      if v ≤ α then return v, move
    return v, move
```

شکل ۵.۷ — الگوریتم جستجوی آلفا–بتا. توجه کنید این توابع همان توابع MINIMAX-SEARCH شکل ۵.۳ هستند، به‌جز آنکه کران‌ها را در متغیرهای α و β نگه می‌داریم و از آن‌ها برای قطع جستجو وقتی مقداری بیرون از کران‌هاست استفاده می‌کنیم.

۵.۲.۴ ترتیب حرکت

اثربخشی هرس آلفا–بتا به‌شدت به ترتیبی که حالت‌ها بررسی می‌شوند بستگی دارد. برای نمونه، در شکل ۵.۵(هـ) و (و)، نتوانستیم هیچ جانشینی از D را هرس کنیم، زیرا بدترین جانشین‌ها (از دیدگاه MIN) نخست تولید شدند. اگر سومین جانشین D نخست تولید شده بود، با مقدار ۲، می‌توانستیم دو جانشین دیگر را هرس کنیم. این پیشنهاد می‌کند ارزشش را دارد نخست جانشین‌هایی را بررسی کنیم که به احتمال زیاد بهترین‌اند.

اگر این کار به‌صورت کامل انجام می‌شد، آلفا–بتا برای انتخاب بهترین حرکت فقط به بررسی $O(b^{m/2})$ گره نیاز داشت، به‌جای $O(b^m)$ برای مینیمکس. این بدان معناست ضریب انشعاب مؤثر به‌جای $\sqrt{b}$، b می‌شود — برای شطرنج، حدود ۶ به‌جای ۳۵. به بیان دیگر، آلفا–بتا با ترتیب حرکت کامل می‌تواند درختِ تقریباً دو برابر عمیق‌تر از مینیمکس را در همان زمان حل کند. با ترتیب حرکت تصادفی، شمار کل گره‌های بررسی‌شده برای b متوسط تقریباً $O(b^{3m/4})$ خواهد بود. حال، بدیهی است نمی‌توانیم به ترتیب حرکت کامل برسیم — در آن صورت تابع ترتیب‌دهی را می‌شد برای بازی کامل به کار برد! اما اغلب می‌توانیم نسبتاً نزدیک شویم. برای شطرنج، تابع ترتیب‌دهی نسبتاً ساده (مانند نخست شکارها، سپس تهدیدها، سپس حرکت‌های رو به جلو، و سپس حرکت‌های عقب) شما را تا حدود ضریب ۲ از نتیجهٔ بهترین حالت $O(b^{m/2})$ می‌رساند.

افزودن طرح‌های ترتیب حرکت پویا، مانند آزمودن نخست حرکت‌هایی که در گذشته بهترین یافته شده‌اند، ما را کاملاً نزدیک حد نظری می‌کند. «گذشته» می‌تواند حرکت قبلی باشد — اغلب همان تهدیدها باقی می‌مانند — یا می‌تواند از کاوش قبلیِ حرکت کنونی از طریق فرایند تعمیق تکراری (صفحهٔ ۸۰ را ببینید) بیاید. نخست، یک نیم‌حرکت جستجو کنید و رتبه‌بندی حرکت‌ها را بر اساس ارزیابی‌هایشان ثبت کنید. سپس یک نیم‌حرکت عمیق‌تر جستجو کنید و از رتبه‌بندی قبلی برای آگاه‌کردن ترتیب حرکت استفاده کنید؛ و الی آخر. زمان جستجوی افزایش‌یافته از تعمیق تکراری را می‌توان بیش از آن با ترتیب حرکت بهتر جبران کرد. بهترین حرکت‌ها به‌عنوان «حرکت‌های قاتل» (Killer Moves) شناخته می‌شوند و آزمودن نخست آن‌ها «اکتشافی حرکت قاتل» (Killer Move Heuristic) نامیده می‌شود.

در بخش ۳.۳.۳ یادآوری کردیم مسیرهای زائد به حالت‌های تکراری می‌توانند افزایش نمایی در هزینهٔ جستجو ایجاد کنند و نگه‌داشتن جدولی از حالت‌های قبلاً رسیده می‌تواند این مسئله را حل کند. در جستجوی درخت بازی، حالت‌های تکراری می‌توانند به‌سبب «جابه‌جایی‌ها» (Transpositions) رخ دهند — جایگشت‌های متفاوتی از توالی حرکت که به همان موضع می‌انجامند — و مسئله را می‌توان با «جدول جابه‌جایی» (Transposition Table) که مقدار اکتشافی حالت‌ها را کش می‌کند حل کرد. برای نمونه، فرض کنیم سفید حرکتی w1 دارد که سیاه می‌تواند با b1 پاسخش دهد و حرکتی نامرتبط w2 در آن‌سوی تخته که می‌توان با b2 پاسخش داد، و توالی حرکت [w1,b1,w2,b2] را جستجو کنیم؛ حالت حاصل را s بنامیم. پس از کاوش زیردرخت بزرگ زیر s، مقدار پشتیبان‌گرفته‌شده‌اش را می‌یابیم که در جدول جابه‌جایی ذخیره می‌کنیم. وقتی بعداً توالی [w2,b2,w1,b1] را جستجو می‌کنیم، دوباره به s می‌رسیم و می‌توانیم مقدار را در جدول نگاه کنیم به‌جای تکرار جستجو. در شطرنج، استفاده از جدول‌های جابه‌جایی بسیار مؤثر است و به ما امکان می‌دهد عمق جستجوی قابل دسترسی را در همان زمان دو برابر کنیم.

حتی با هرس آلفا–بتا و ترتیب حرکت هوشمندانه، مینیمکس برای بازی‌هایی مانند شطرنج و گو کار نخواهد کرد، زیرا هنوز حالت‌های بسیار زیادی برای کاوش در زمان در دسترس هست. کلود شانون در نخستین مقالهٔ بازی رایانه‌ای، «برنامه‌نویسی یک رایانه برای بازی شطرنج» (شانون، ۱۹۵۰)، این مسئله را شناخت و دو راهبرد پیشنهاد کرد: راهبرد نوع A همهٔ حرکت‌های ممکن را تا عمقی معین در درخت جستجو در نظر می‌گیرد و سپس از تابع ارزیابی اکتشافی برای تخمین مطلوبیت حالت‌های آن عمق استفاده می‌کند. بخشی پهن اما کم‌عمق از درخت را کاوش می‌کند. راهبرد نوع B حرکت‌هایی را که بد به نظر می‌رسند نادیده می‌گیرد و خطوط امیدوارکننده را «تا آنجا که ممکن است» دنبال می‌کند. (این در بخش ۵.۴ پوشش داده می‌شود، زیرا ضریب انشعاب در گو بسیار بالاتر است.) اخیراً، برنامه‌های نوع B سطح قهرمانی جهان را در گستره‌ای از بازی‌ها نشان داده‌اند، از جمله شطرنج (سیلور و همکاران، ۲۰۱۸).

۵.۳ جستجوی درختی آلفا–بتا اکتشافی

برای بهره‌بردن از زمان محاسباتی محدودمان، می‌توانیم جستجو را زود قطع کنیم و تابع ارزیابی اکتشافی را بر حالت‌ها اعمال کنیم و عملاً گره‌های غیرپایانی را چنان رفتار کنیم که گویی پایانی‌اند. به بیان دیگر، تابع UTILITY را با EVAL جایگزین می‌کنیم که مطلوبیت حالت را تخمین می‌زند. همچنین آزمون پایانی را با «آزمون قطع» (Cutoff Test)

جایگزین می‌کنیم که باید برای حالت‌های پایانی true بازگرداند، اما در بقیهٔ موارد آزاد است تصمیم بگیرد چه‌وقت جستجو را قطع کند، بر اساس عمق جستجو و هر ویژگی از حالت که انتخاب می‌کند در نظر بگیرد. این فرمول H-MINIMAX(s, d) را برای مقدار مینیمکس اکتشافی حالت s در عمق جستجوی d به ما می‌دهد:

```
                                                                = H-MINIMAX(s, d)
                                                                  EVAL(s, MAX)                                  اگر IS-CUTOFF(s, d)
max_{a ∈ Actions(s)} H-MINIMAX(RESULT(s, a), d+1)  اگر TO-MOVE(s) = MAX
min_{a ∈ Actions(s)} H-MINIMAX(RESULT(s, a), d+1)  اگر TO-MOVE(s) = MIN
```

۵.۳.۱ توابع ارزیابی

تابع ارزیابی اکتشافی EVAL(s, p) تخمینی از مطلوبیت مورد انتظار حالت s برای بازیکن p بازمی‌گرداند، درست همان‌گونه که توابع اکتشافی فصل ۳ تخمینی از فاصله تا هدف بازمی‌گردانند. برای حالت‌های پایانی باید EVAL(s, p) = UTILITY(s, p) باشد و برای حالت‌های غیرپایانی، ارزیابی باید جایی میان باخت و برد باشد: UTILITY(loss, p) ≤ EVAL(s, p) ≤ UTILITY(win, p).

فراتر از این الزام‌ها، چه چیزی تابع ارزیابی خوب می‌سازد؟ نخست، محاسبه نباید زیاد طول بکشد! (تمام نکته جستجوی سریع‌تر است.) دوم، تابع ارزیابی باید با شانس‌های واقعی برد همبستگی قوی داشته باشد. ممکن است دربارهٔ عبارت «شانس‌های برد» سؤال کنید. بالاخره شطرنج بازی شانسی نیست: حالت کنونی را با قطعیت می‌دانیم و هیچ تاسی در کار نیست؛ اگر هیچ بازیکنی اشتباه نکند، پیامد از پیش تعیین‌شده است. اما اگر جستجو باید در حالت‌های غیرپایانی قطع شود، الگوریتم ناگزیر دربارهٔ پیامدهای نهایی آن حالت‌ها نامطمئن خواهد بود (حتی اگر آن عدم قطعیت با منابع محاسباتی بی‌نهایت قابل حل بود).

این ایده را عینی‌تر کنیم. بیشتر توابع ارزیابی با محاسبهٔ ویژگی‌های (Features) گوناگون حالت کار می‌کنند — برای نمونه، در شطرنج، ویژگی‌هایی برای شمار سربازهای سفید، سربازهای سیاه، وزیرهای سفید، وزیرهای سیاه و الی آخر داریم. ویژگی‌ها با هم، رده‌ها یا رده‌های هم‌ارزی گوناگونی از حالت‌ها تعریف می‌کنند: حالت‌های هر رده مقادیر یکسانی برای همهٔ ویژگی‌ها دارند. برای نمونه، رده‌ای ممکن است شامل همهٔ پایان‌بازی‌های دوسرباز در برابر یک‌سرباز باشد. هر ردهٔ داده‌شده شامل برخی حالت‌هاست که (با بازی کامل) به برد، برخی که به تساوی و برخی که به باخت می‌انجامند.

تابع ارزیابی نمی‌داند کدام حالت‌ها کدام‌اند، اما می‌تواند مقدار تکی‌ای بازگرداند که نسبت حالت‌های هر پیامد را تخمین می‌زند. برای نمونه، فرض کنیم تجربه‌مان پیشنهاد می‌کند ۸۲٪ از حالت‌های روی‌آمده در ردهٔ دوسرباز در برابر یک‌سرباز به برد می‌انجامند (مطلوبیت +1)؛ ۲٪ به باخت (۰)؛ و ۱۶٪ به تساوی (1/2). آنگاه ارزیابی معقول برای حالت‌های آن رده «مقدار مورد انتظار» (Expected Value) است: (0.82 × +1) + (0 × 0.02) + (1/2 × 0.16) = 0.90. در اصل، مقدار مورد انتظار را می‌توان برای هر رده از حالت‌ها تعیین کرد و به تابع ارزیابی‌ای رسید که برای هر حالتی کار می‌کند.

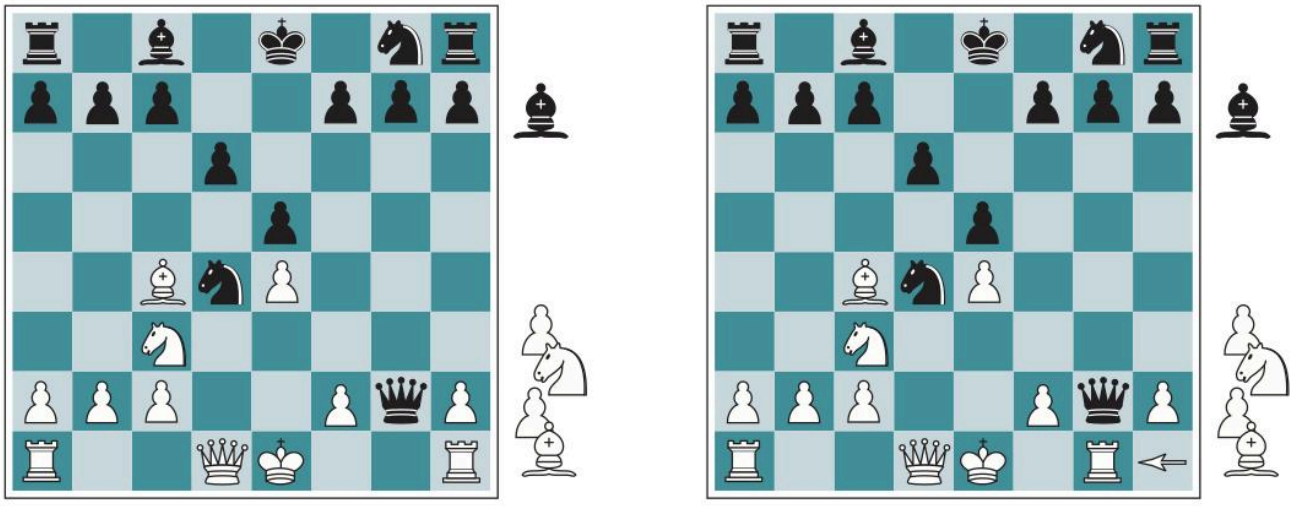
در عمل، این نوع تحلیل به رده‌های بسیار زیادی نیاز دارد و از این رو به تجربهٔ بسیار زیادی برای تخمین همهٔ احتمال‌ها. در عوض، بیشتر توابع ارزیابی سهم‌های عددی جداگانه‌ای از هر ویژگی محاسبه و سپس برای یافتن مقدار کل ترکیب می‌کنند. قرن‌هاست شطرنج‌بازان با همین ایده روش‌هایی برای قضاوت دربارهٔ ارزش موضع توسعه داده‌اند. برای نمونه،

کتاب‌های مقدماتی شطرنج «ارزش مادی» (Material Value) تقریبی برای هر مهره می‌دهند: هر سرباز ۱، اسب یا فیل ۳، رخ ۵ و وزیر ۹ می‌ارزد. دیگر ویژگی‌ها مانند «ساختار خوب سربازها» و «ایمنی شاه» شاید نیم‌سرباز بی‌ارزند. سپس این مقادیر ویژگی به‌سادگی جمع می‌شوند تا ارزیابی موضع به دست آید.

از نظر ریاضی، این نوع تابع ارزیابی «تابع خطی وزن‌دار» (Weighted Linear Function) نامیده می‌شود، زیرا می‌توان آن را چنین بیان کرد:

$$EVAL(s) = w_1f_1(s) + w_2f_2(s) + \ldots + w_nf_n(s) = \sum_i w_if_i(s)$$

که هر f_i ویژگی موضع (مانند «شمار فیل‌های سفید») و هر w_i وزن (که می‌گوید آن ویژگی چقدر مهم است) است. وزن‌ها باید نرمال شوند تا مجموع همیشه در گسترهٔ باخت (۰) تا برد (+1) باشد. برتری امنی معادل یک سرباز، احتمال بردِ چشمگیری می‌دهد و برتری امنی معادل سه سرباز باید پیروزی تقریباً قطعی بدهد، چنان‌که شکل ۵.۸(الف) نشان می‌دهد. گفتیم تابع ارزیابی باید با شانس‌های واقعی برد همبستگی قوی داشته باشد، اما لازم نیست همبستگی خطی باشد: اگر حالت s دو برابر حالت 's احتمال برد داشته باشد، الزامی نیست EVAL(s) دو برابر EVAL('s) باشد؛ فقط EVAL(s) > EVAL('s) لازم است.



شکل ۵.۸ – دو موضع شطرنج که فقط در مکان رخِ پایین-راست تفاوت دارند. در (الف)، سیاه برتری اسب و دو سرباز دارد که باید برای بردن بازی کافی باشد. در (ب)، سفید وزیر را خواهد گرفت و برتری‌ای خواهد داشت که باید برای برد به اندازهٔ کافی قوی باشد.

جمع‌کردن مقادیر ویژگی‌ها کار معقولی به نظر می‌رسد، اما در واقع فرض قوی‌ای در بر دارد: اینکه سهم هر ویژگی مستقل از مقادیر دیگر ویژگی‌هاست. به همین دلیل، برنامه‌های کنونی شطرنج و دیگر بازی‌ها از ترکیب‌های غیرخطی ویژگی‌ها نیز استفاده می‌کنند. برای نمونه، یک جفت فیل ممکن است بیش از دو برابر ارزش فیل تکی بی‌ارزد و فیل در پایان‌بازی بیشتر از اوایل بازی می‌ارزد – وقتی ویژگی شمار حرکت بالا یا ویژگی شمار مهره‌های باقی‌مانده پایین است.

ویژگی‌ها و وزن‌ها از کجا می‌آیند؟ آن‌ها بخشی از قواعد شطرنج نیستند، اما بخشی از فرهنگ تجربهٔ شطرنج‌بازی انسانی‌اند. در بازی‌هایی که این نوع تجربه در دسترس نیست، وزن‌های تابع ارزیابی را می‌توان با فنون یادگیری ماشین فصل ۲۲ تخمین زد. اعمال این فنون بر شطرنج تأیید کرده فیل واقعاً حدود سه سرباز می‌ارزد و به نظر می‌رسد قرن‌ها تجربهٔ انسانی را می‌توان در فقط چند ساعت یادگیری ماشین بازتولید کرد.

۵.۳.۲ قطع جستجو

گام بعدی اصلاح ALPHA-BETA-SEARCH است تا وقتی مناسب است جستجو را قطع کند و تابع اکتشافی EVAL را فراخوانی کند. دو خطِ شکل ۵.۷ که به IS-TERMINAL اشاره می‌کنند را با خط زیر جایگزین می‌کنیم:

```
if game.IS-CUTOFF(state, depth) then return game.EVAL(state, player), null
```

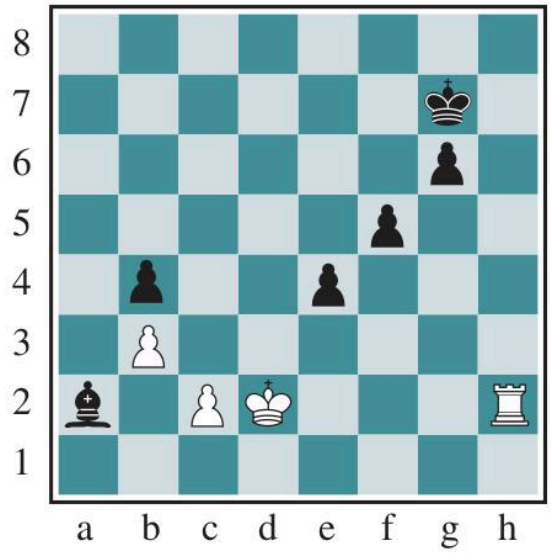
همچنین باید کتابداری‌ای ترتیب دهیم تا عمق کنونی در هر فراخوانی بازگشتی افزایش یابد. سرراست‌ترین رویکرد برای کنترل مقدار جستجو، تعیین حد عمق ثابت است، چنان‌که IS-CUTOFF(state, depth) برای همهٔ عمق‌های بزرگ‌تر از عمق ثابت d (و نیز برای همهٔ حالت‌های پایانی) true بازگرداند. عمق d چنان برگزیده می‌شود که حرکتی در زمان تخصیص‌یافته انتخاب شود. رویکرد استوارتر، اعمال تعمیق تکراری است (فصل ۳ را ببینید). وقتی زمان تمام می‌شود، برنامه حرکتی را بازمی‌گرداند که عمیق‌ترین جستجوی کامل‌شده انتخاب کرده. به‌عنوان امتیاز، اگر در هر دور تعمیق تکراری مدخل‌هایی در جدول جابه‌جایی نگه داریم، دورهای بعدی سریع‌تر خواهند بود و می‌توانیم از ارزیابی‌ها برای بهبود ترتیب حرکت استفاده کنیم.

این رویکردهای ساده می‌توانند به‌سبب سرشت تقریبی تابع ارزیابی به خطاهایی بینجامند. تابع ارزیابی سادهٔ شطرنج بر اساس برتری مادی را دوباره در نظر بگیرید. فرض کنیم برنامه تا حد عمق جستجو می‌کند و به موضع شکل ۵.۸(ب) می‌رسد، جایی که سیاه با یک اسب و دو سرباز جلوست. آن را به‌عنوان مقدار اکتشافی حالت گزارش می‌کند و بدین ترتیب اعلام می‌کند حالت، بردِ احتمالی سیاه است. اما حرکت بعدی سفید وزیر سیاه را بدون جبران می‌گیرد. از این رو، موضع در واقع به نفع سفید است، اما این فقط با نگاه به جلو دیده می‌شود.

تابع ارزیابی باید فقط بر موضع‌هایی اعمال شود که «ساکن» (Quiescent) باشند — یعنی موضع‌هایی که در آن‌ها هیچ حرکتِ در انتظاری (مانند شکار وزیر) نیست که ارزیابی را به‌شکل وحشیانه‌ای بچرخاند. برای موضع‌های ناساکن، IS-CUTOFF false بازمی‌گرداند و جستجو ادامه می‌یابد تا به موضع‌های ساکن برسد. این «جستجوی سکون» (Quiescence Search) اضافی گاهی محدود به در نظر گرفتن فقط انواع خاصی از حرکت‌ها، مانند حرکت‌های شکار، می‌شود که عدم قطعیت‌های موضع را به‌سرعت حل می‌کنند.

«اثر افق» (Horizon Effect) دشوارتر است و حذفش سخت‌تر. وقتی پدیدار می‌شود که برنامه با حرکت حریفی روبه‌روست که آسیب جدی وارد می‌کند و در نهایت گریزناپذیر است، اما می‌توان با تاکتیک‌های تأخیری موقتاً از آن گریخت. موضع شطرنج شکل ۵.۹ را در نظر بگیرید. روشن است هیچ راهی برای گریز فیل سیاه نیست. برای نمونه، رخ سفید می‌تواند با حرکت به h1، سپس a1 و سپس a2 آن را شکار کند؛ شکار در عمق ۶ نیم‌حرکت.

اما سیاه توالی حرکتی دارد که شکار فیل را «آنسوی افق» می‌برد. فرض کنیم سیاه تا عمق ۸ نیم‌حرکت جستجو کند. بیشتر حرکت‌های سیاه به شکار نهایی فیل می‌انجامند و بنابراین به‌عنوان حرکت‌های «بد» علامت می‌خورند. اما سیاه همچنین توالی حرکتی را در نظر می‌گیرد که با کیش‌کردن شاه با سرباز آغاز می‌شود و شاه را وسوسه می‌کند سرباز را بگیرد. سیاه می‌تواند همین کار را با سرباز دوم بکند. این به اندازهٔ کافی حرکت مصرف می‌کند که شکار فیل در بقیهٔ جستجوی سیاه کشف نشود. سیاه می‌اندیشد این خط بازی فیل را به بهای دو سرباز نجات داده، در حالی که در واقع فقط سربازها را تلف کرده و شکار ناگزیر فیل را آنسوی افقی که سیاه می‌بیند برده است.



شکل ۵.۹ – اثر افق. با نوبت سیاه، فیل سیاه قطعاً محکوم است. اما سیاه می‌تواند آن رویداد را با کیش‌کردن شاه سفید با سربازهایش به تأخیر بیندازد و شاه را تشویق کند سربازها را بگیرد. این باخت ناگزیر فیل را آنسوی افق می‌برد و بدین ترتیب قربانی‌کردن سربازها از دید الگوریتم جستجو حرکت‌های خوبی دیده می‌شوند نه بد.

یکی از راهبردهای کاهش اثر افق، اجازهٔ «بسط تکین» (Singular Extensions) است: حرکت‌هایی که در موضعِ داده‌شده «به‌وضوح بهتر» از همهٔ حرکت‌های دیگرند، حتی وقتی جستجو معمولاً باید در آن نقطه قطع شود. در مثال ما، جستجو آشکار خواهد کرد سه حرکت رخ سفید – h2 به h1، سپس h1 به a1، و سپس a1 شکار فیل در a2 – هر یک به‌نوبت حرکت‌های به‌وضوح بهترند، پس حتی اگر توالی حرکت سربازها ما را به افق برساند، این حرکت‌های به‌وضوح بهتر شانس بسط جستجو را خواهند گرفت. این درخت را عمیق‌تر می‌کند، اما چون معمولاً بسط‌های تکین اندک‌اند، راهبرد گره‌های چندانی به درخت نمی‌افزاید و در عمل مؤثر ثابت شده.

۵.۳.۳ هرس رو به جلو

هرس آلفا–بتا شاخه‌هایی از درخت را هرس می‌کند که هیچ اثری بر ارزیابی نهایی نمی‌توانند داشته باشند، اما «هرس رو به جلو» (Forward Pruning) حرکت‌هایی را هرس می‌کند که به نظر می‌رسد حرکت‌های ضعیفی‌اند، اما شاید خوب باشند. پس راهبرد به بهای ریسک خطا، زمان محاسبه صرفه‌جویی می‌کند. به زبان شانون، این راهبرد نوع B است. بدیهی است بیشتر شطرنج‌بازان انسانی این کار را می‌کنند و از هر موضع فقط چند حرکت را (دست‌کم آگاهانه) در نظر می‌گیرند.

یک رویکرد برای هرس رو به جلو، جستجوی پرتوی (Beam Search) است (صفحهٔ ۱۱۵ را ببینید): در هر نیم‌حرکت، فقط «پرتویی» از n حرکت برتر (بر اساس تابع ارزیابی) را در نظر بگیرید به‌جای همهٔ حرکت‌های ممکن. متأسفانه این رویکرد نسبتاً خطرناک است، زیرا هیچ تضمینی نیست که بهترین حرکت هرس نشود.

الگوریتم PROBCUT یا «قطع احتمالاتی» (بورو، ۱۹۹۵) نسخهٔ هرس-رو-به-جلوِ جستجوی آلفا–بتاست که از آمار به‌دست‌آمده از تجربهٔ قبلی برای کم‌کردن احتمال هرس‌شدن بهترین حرکت استفاده می‌کند. جستجوی آلفا–بتا هر گره‌ای را که به‌شکل اثبات‌پذیر بیرون از پنجرهٔ (α,β) کنونی است هرس می‌کند. PROBCUT همچنین گره‌هایی را هرس می‌کند که «احتمالاً» بیرون از پنجره‌اند. این احتمال را با جستجوی کم‌عمق برای محاسبهٔ مقدار پشتیبان‌گرفته‌شدهٔ v یک گره و سپس استفاده از تجربهٔ گذشته برای تخمین این محاسبه می‌کند که چقدر محتمل است امتیاز v در عمق d از درخت بیرون از (α,β) باشد. بورو این فن را بر برنامهٔ اوتلویش، LOGISTELLO، اعمال کرد و یافت نسخه‌ای از برنامه‌اش با PROBCUT در ۶۴٪ مواقع نسخهٔ عادی را می‌برد، حتی وقتی به نسخهٔ عادی دو برابر زمان داده می‌شد.

فن دیگر، «کاهش حرکت دیرهنگام» (Late Move Reduction)، با این فرض کار می‌کند که ترتیب حرکت خوب انجام شده و بنابراین حرکت‌هایی که دیرتر در فهرست حرکت‌های ممکن ظاهر می‌شوند کمتر احتمال دارد حرکت‌های خوبی باشند. اما به‌جای هرس کامل آن‌ها، فقط عمق جستجوی این حرکت‌ها را کاهش می‌دهیم و بدین ترتیب زمان صرفه‌جویی می‌کنیم. اگر جستجوی کاهش‌یافته با مقداری بالاتر از مقدار α کنونی بازگردد، می‌توانیم جستجو را با عمق کامل دوباره اجرا کنیم.

ترکیب همهٔ فنون توصیف‌شده در این‌جا به برنامه‌ای می‌انجامد که می‌تواند شطرنج (یا بازی‌های دیگر) قابل قبولی بازی کند. فرض کنیم تابع ارزیابی شطرنج، آزمون قطع معقول با جستجوی سکون پیاده کرده‌ایم. همچنین فرض کنیم پس از ماه‌ها بیت‌بازی خسته‌کننده، می‌توانیم روی تازه‌ترین رایانهٔ شخصی حدود یک میلیون گره در ثانیه تولید و ارزیابی کنیم. ضریب انشعاب شطرنج به‌طور میانگین حدود ۳۵ است و 35^5 حدود ۵۰ میلیون است، پس اگر از جستجوی مینیمکس استفاده می‌کردیم، فقط می‌توانستیم در حدود یک دقیقه محاسبه، پنج نیم‌حرکت به جلو بنگریم؛ قواعد رقابت به ما زمان کافی برای جستجوی شش نیم‌حرکت نمی‌داد.

هرچند نالایق نیست، چنین برنامه‌ای را یک شطرنج‌باز متوسط انسانی می‌تواند شکست دهد، که گاهی می‌تواند شش یا هشت نیم‌حرکت جلوتر برنامه‌ریزی کند. با جستجوی آلفا–بتا و جدول جابه‌جایی بزرگ به حدود ۱۴ نیم‌حرکت می‌رسیم که به سطح بازی استادی می‌انجامد. می‌توانستیم رایانهٔ شخصی‌مان را با ایستگاه کاری ۸ GPU معاوضه کنیم و به بیش از یک میلیارد گره در ثانیه برسیم، اما برای رسیدن به مقام استادبزرگ همچنان به تابع ارزیابیِ به‌طور گسترده تنظیم‌شده و پایگاه‌دادهٔ بزرگی از حرکت‌های پایان‌بازی نیاز داشتیم. برنامه‌های برتر شطرنج مانند استاک‌فیش (STOCKFISH) همهٔ این‌ها را دارند و اغلب در درخت جستجو به عمق ۳۰ یا بیشتر می‌رسند و بسیار فراتر از توانایی هر بازیکن انسانی‌اند.

۵.۳.۴ جستجو در برابر جدول‌جست‌وجو

به نوعی اسراف می‌نماید برنامهٔ شطرنج بازی را با در نظر گرفتن درختی از میلیارد حالت بازی آغاز کند، فقط برای اینکه نتیجه بگیرد سرباز به e4 بازی می‌کند (محبوب‌ترین حرکت نخست). کتاب‌هایی که بازی خوب در گشایش و پایان‌بازی شطرنج را توصیف می‌کنند بیش از یک قرن در دسترس بوده‌اند (تاترزال، ۱۹۱۱). پس تعجب‌آور نیست بسیاری از برنامه‌های بازی‌کننده به‌جای جستجو از جدول‌جست‌وجو برای گشایش و پایان بازی‌ها استفاده می‌کنند.

برای گشایش‌ها، رایانه بیشتر به تخصص انسان‌ها تکیه می‌کند. بهترین توصیهٔ کارشناسان انسانی دربارهٔ چگونگی بازی هر گشایش را می‌توان از کتاب‌ها کپی و برای استفادهٔ رایانه در جدول‌ها وارد کرد. افزون بر این، رایانه‌ها می‌توانند از پایگاه‌دادهٔ بازی‌های قبلی‌بازی‌شده آمار جمع کنند تا ببینند کدام توالی‌های گشایش بیشتر به برد می‌انجامند. برای چند حرکت نخست امکان‌های اندکی هست و بیشتر موضع‌ها در جدول خواهند بود. معمولاً پس از حدود ۱۰ یا ۱۵ حرکت به موضعی به‌ندرت‌دیده‌شده می‌رسیم و برنامه باید از جدول‌جست‌وجو به جستجو سوئیچ کند. نزدیک پایان بازی نیز دوباره موضع‌های ممکن کمتری هست و بنابراین جدول‌جست‌وجو آسان‌تر است. اما این‌جا این رایانه است که تخصص دارد: تحلیل رایانه‌ای پایان‌بازی‌ها بسیار فراتر از توانایی‌های انسانی می‌رود. انسان‌های تازه‌کار می‌توانند پایان‌بازی شاه-و-رخ در برابر شاه (KRK) را با دنبال‌کردن چند قاعدهٔ ساده ببرند. پایان‌های دیگر، مانند شاه، فیل و اسب در برابر شاه (KBNK)، برای استادی دشوارند و توصیف راهبردی موجزی ندارند.

رایانه اما می‌تواند پایان‌بازی را به‌طور کامل با تولید «سیاست» (Policy) حل کند؛ سیاست، نگاشتی از هر حالت ممکن به بهترین حرکت در آن حالت است. آنگاه رایانه می‌تواند با نگاه‌کردن به حرکت درست در این جدول کامل بازی کند. جدول با «جستجوی مینیمکس قهقرایی» (Retrograde Minimax Search) ساخته می‌شود: با در نظر گرفتن همهٔ راه‌های چیدن مهره‌های KBNK روی تخته آغاز کنید. برخی از موضع‌ها برد سفیدند؛ آن‌ها را چنین علامت بزنید. سپس قواعد شطرنج را معکوس کنید تا به‌جای حرکت‌ها، حرکت‌های معکوس انجام دهید. هر حرکتی از سفید که، صرف‌نظر از حرکتی که سیاه پاسخ می‌دهد، به موضعی علامت‌خورده به‌عنوان برد ختم شود، باید برد نیز باشد. این جستجو را ادامه دهید تا همهٔ موضع‌های ممکن به‌صورت برد، باخت یا تساوی حل شوند و جدول‌جست‌وجوی خطاناپذیری برای همهٔ پایان‌بازی‌ها با آن مهره‌ها داشته باشید. این نه فقط برای پایان‌های KBNK، بلکه برای همهٔ پایان‌ها با هفت مهره یا کمتر انجام شده. جدول‌ها شامل ۴۰۰ تریلیون موضع‌اند. جدول هشت‌مهره‌ای به ۴۰ کوادریلیون موضع نیاز خواهد داشت.

۵.۴ جستجوی درختی مونت‌کارلو

بازی گو دو ضعف عمدهٔ جستجوی درختی آلفا–بتا اکتشافی را نشان می‌دهد: نخست، گو ضریب انشعابی دارد که از ۳۶۱ آغاز می‌شود، یعنی جستجوی آلفا–بتا فقط به ۴ یا ۵ نیم‌حرکت محدود می‌شد. دوم، تعریف تابع ارزیابی خوب برای گو دشوار است، زیرا ارزش مادی شاخص قوی‌ای نیست و بیشتر موضع‌ها تا پایان‌بازی در تلاطم‌اند. در پاسخ به این دو چالش، برنامه‌های گوی مدرن جستجوی آلفا–بتا را رها کرده و به‌جایش از راهبردی به نام جستجوی درختی مونت‌کارلو (Monte Carlo Tree Search، یا MCTS)[3] استفاده می‌کنند.

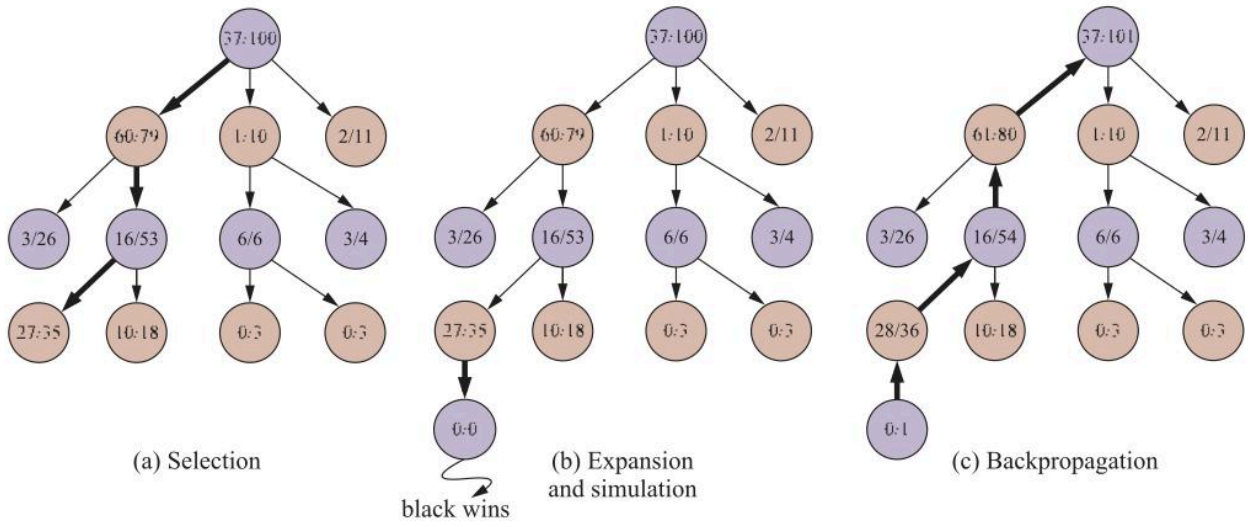
راهبرد پایهٔ MCTS از تابع ارزیابی اکتشافی استفاده نمی‌کند. در عوض، مقدار یک حالت به‌عنوان میانگین مطلوبیت بر روی شماری «شبیه‌سازی» (Simulation) از بازی‌های کاملِ شروع‌شده از آن حالت تخمین زده می‌شود. شبیه‌سازی (که «بازی‌آزمایی» (Playout) یا «رول‌اوت» (Rollout) نیز نامیده می‌شود) نخست برای یک بازیکن حرکت‌ها را برمی‌گزیند، سپس برای دیگری، و تکرار می‌کند تا به موضع پایانی برسد. در آن نقطه، قواعد بازی (نه اکتشافی‌های خطاپذیر) تعیین می‌کنند چه کسی برده یا باخته و با چه امتیازی.

برای بازی‌هایی که تنها پیامدها برد یا باخت‌اند، «میانگین مطلوبیت» همان «درصد برد» است. چگونه انتخاب می‌کنیم چه حرکت‌هایی در طول بازی‌آزمایی انجام دهیم؟ اگر فقط تصادفی انتخاب کنیم، پس از چند شبیه‌سازی پاسخی برای این پرسش می‌گیریم: «بهترین حرکت چیست اگر هر دو بازیکن تصادفی بازی کنند؟» برای برخی بازی‌های ساده، این اتفاقاً همان پاسخ «بهترین حرکت چیست اگر هر دو بازیکن خوب بازی کنند؟» است، اما برای بیشتر بازی‌ها نیست. برای گرفتن اطلاعات مفید از بازی‌آزمایی، به «سیاست بازی‌آزمایی» (Playout Policy) نیاز داریم که حرکت‌ها را به سوی حرکت‌های خوب سوگیری دهد. برای گو و بازی‌های دیگر، سیاست‌های بازی‌آزمایی با موفقیت از بازی خودآموز با استفاده از شبکه‌های عصبی آموخته شده‌اند. گاهی اکتشافی‌های ویژهٔ بازی به کار می‌روند، مانند «حرکت‌های شکار را در نظر بگیر» در شطرنج یا «خانهٔ گوشه را بگیر» در اوتلو.

با داشتن سیاست بازی‌آزمایی، باید دو چیز را تصمیم بگیریم: از چه موضع‌هایی بازی‌آزمایی‌ها را آغاز کنیم و چند بازی‌آزمایی به هر موضع تخصیص دهیم؟ ساده‌ترین پاسخ، که «جستجوی مونت‌کارلو ناب» (Pure Monte Carlo Search) نامیده می‌شود، انجام N شبیه‌سازی از حالت کنونی بازی و پیگیری اینکه کدام یک از حرکت‌های ممکن از موضع کنونی بیشترین درصد برد را دارد. برای برخی بازی‌های تصادفی، این با افزایش N به بازی بهینه همگرا می‌شود، اما برای بیشتر بازی‌ها کافی نیست — به «سیاست انتخاب» (Selection Policy) نیاز داریم که منابع محاسباتی را به‌صورت انتخابی بر بخش‌های مهم درخت بازی متمرکز کند. این سیاست دو عامل را موازنه می‌کند: «اکتشاف» (Exploration) حالت‌هایی که بازی‌آزمایی‌های کمی داشته‌اند، و «بهره‌برداری» (Exploitation) از حالت‌هایی که در بازی‌آزمایی‌های گذشته خوب عمل کرده‌اند تا تخمین دقیق‌تری از مقدارشان بگیرد.

فرایند MCTS در هر تکرار شامل چهار گام اصلی است:

1. **انتخاب (Selection):** با شروع از ریشهٔ درخت جستجو، بر اساس یک سیاست انتخاب (مانند UCT) در درخت پایین می‌رویم تا به یک گرهٔ برگ برسیم.
2. **بسط (Expansion):** یک یا چند گرهٔ فرزند جدید (متناظر با حرکت‌های قانونی) تولید و به درخت افزوده می‌شوند.
3. **شبیه‌سازی (Simulation):** از گرهٔ جدید، با استفاده از سیاست بازی‌آزمایی (معمولاً با حرکت‌های تصادفی یا اکتشافی سریع) یک بازی کامل تا پایان شبیه‌سازی می‌شود تا نتیجه مشخص گردد.
4. **بازانتشار (Back-propagation):** نتیجهٔ بازی‌آزمایی در طول مسیر به سمت بالا تا ریشه منتشر می‌شود و آمار شمار بازدیدها و بردهای گره‌های مسیر به‌روزرسانی می‌گردد.



شکل ۵.۱۰ — یک تکرار از فرایند انتخاب حرکت با جستجوی درختی مونت‌کارلو (MCTS) با معیار انتخاب UCT، پس از ۱۰۰ تکرارِ ازپیش‌انجام‌شده. در (الف) حرکت‌ها را تا پایین درخت انتخاب می‌کنیم و به گرهٔ برگِ علامت‌خوردهٔ 27/35 می‌رسیم (برای ۲۷ برد سیاه از ۳۵ بازی‌آزمایی). در (ب) گرهٔ انتخاب‌شده را بسط می‌دهیم و شبیه‌سازی (بازی‌آزمایی) انجام می‌دهیم که به برد سیاه می‌انجامد. در (ج) نتایج شبیه‌سازی به بالای درخت بازانتشار می‌یابد.

این چهار گام را یا برای شمار ثابتی تکرار یا تا انقضای زمان تخصیص‌یافته تکرار می‌کنیم و سپس حرکتی را با بیشترین شمار بازی‌آزمایی بازمی‌گردانیم.

```
function MONTE-CARLO-TREE-SEARCH(state) returns an action
                                        tree ← NODE(state)
                                while IS-TIME-REMAINING() do
                                       leaf ← SELECT(tree)
                                      child ← EXPAND(leaf)
                                     result ← SIMULATE(child)
                                BACK-PROPAGATE(result, child)
return the action in ACTIONS(state) whose node has the most playouts
```

شکل ۵.۱۱ — الگوریتم جستجوی درختی مونت‌کارلو. درخت بازی، tree، مقداردهی می‌شود و سپس چرخهٔ SELECT / EXPAND / SIMULATE / BACK-PROPAGATE را تا تمام‌شدن زمان تکرار می‌کنیم و حرکتی را بازمی‌گردانیم که به گره‌ای با بیشترین شمار بازی‌آزمایی انجامید.

یکی از سیاست‌های انتخاب بسیار مؤثر «کران اطمینان بالایی اعمال‌شده بر درخت‌ها» یا UCT نامیده می‌شود. سیاست هر حرکت ممکن را بر اساس فرمول کران اطمینان بالایی به نام UCB1 رتبه‌بندی می‌کند. (برای جزئیات بیشتر بخش ۱۷.۳.۳ را ببینید.) برای گرهٔ n، فرمول چنین است:

$$\mathrm{UCB1}(n) = U(n)/N(n) + C \times \sqrt{\ln N(\mathrm{PARENT}(n)) / N(n)}$$

که U(n) مطلوبیت کل همهٔ بازی‌آزمایی‌هایی است که از گرهٔ n گذشتند، N(n) شمار بازی‌آزمایی‌های گذرنده از n، و PARENT(n) گرهٔ والد n در درخت است. پس U(n)/N(n) جملهٔ بهره‌برداری است: میانگین مطلوبیت n. جملهٔ با ریشهٔ دوم جملهٔ اکتشاف است: شمار N(n) در مخرج دارد، یعنی جمله برای گره‌هایی که فقط چند بار کاوش شده‌اند بالا

خواهد بود. در صورت، لگاریتم تعداد دفعاتی را دارد که والد n را کاوش کرده‌ایم. این بدان معناست اگر n را درصدی غیرصفر از زمان انتخاب کنیم، جملهٔ اکتشاف با افزایش شمارها به صفر می‌گراید و سرانجام بازی‌آزمایی‌ها به گره‌ای با بیشترین میانگین مطلوبیت داده می‌شوند.

C ثابتی است که بهره‌برداری و اکتشاف را موازنه می‌کند. استدلال نظری‌ای هست که C باید $\sqrt{2}$ باشد، اما در عمل، برنامه‌نویسان بازی مقادیر متعددی برای C آزمایش و بهترین را برمی‌گزینند. (برخی برنامه‌ها فرمول‌های کمی متفاوت استفاده می‌کنند؛ برای نمونه، آلفازیرو جمله‌ای برای احتمال حرکت می‌افزاید که توسط شبکهٔ عصبیِ آموزش‌دیده از بازی خودآموزِ گذشته محاسبه می‌شود.) با C=1.4، گرهٔ 60/79 در شکل ۵.۱۰ بالاترین امتیاز UCB1 را دارد، اما با C=1.5، گرهٔ 2/11 خواهد بود.

شکل ۵.۱۱ الگوریتم کامل MCTS با UCT را نشان می‌دهد. وقتی تکرارها پایان می‌یابند، حرکتی با بیشترین شمار بازی‌آزمایی بازگردانده می‌شود. شاید فکر کنید بهتر است گره‌ای با بیشترین میانگین مطلوبیت بازگردانده شود، اما ایده آن است که گره با 65/100 برد از گره با 2/3 برد بهتر است، زیرا دومی عدم قطعیت زیادی دارد. در هر حال، فرمول UCB1 تضمین می‌کند گره‌ای با بیشترین بازی‌آزمایی تقریباً همیشه گره‌ای با بیشترین درصد برد است، زیرا فرایند انتخاب با افزایش شمار بازی‌آزمایی‌ها بیشتر و بیشتر درصد برد را ترجیح می‌دهد.

زمان محاسبهٔ یک بازی‌آزمایی در عمق درخت بازی خطی است نه نمایی، زیرا در هر نقطهٔ انتخاب فقط یک حرکت برداشته می‌شود. این زمان فراوانی برای چندین بازی‌آزمایی به ما می‌دهد. برای نمونه: بازی‌ای با ضریب انشعاب ۳۲ را در نظر بگیرید که میانگین بازی ۱۰۰ نیم‌حرکت طول می‌کشد. اگر توان محاسباتی کافی برای در نظر گرفتن یک میلیارد حالت بازی پیش از برداشتن حرکت داشته باشیم، مینیمکس می‌تواند ۶ نیم‌حرکت عمق جستجو کند، آلفا–بتا با ترتیب حرکت کامل ۱۲ نیم‌حرکت، و جستجوی مونت‌کارلو ۱۰ میلیون بازی‌آزمایی. کدام رویکرد بهتر خواهد بود؟ به دقت تابع اکتشافی در برابر سیاست‌های انتخاب و بازی‌آزمایی بستگی دارد.

حکمت مرسوم چنین بوده که جستجوی مونت‌کارلو بر آلفا–بتا برای بازی‌هایی مانند گو برتری دارد که ضریب انشعاب بسیار بالاست (و بنابراین آلفا–بتا نمی‌تواند به اندازهٔ کافی عمیق جستجو کند)، یا وقتی تعریف تابع ارزیابی خوب دشوار است. کاری که آلفا–بتا می‌کند، انتخاب مسیر به گره‌ای با بیشترین امتیاز تابع ارزیابیِ دست‌یافتنی است، با توجه به اینکه حریف خواهد کوشید امتیاز را کمینه کند. پس اگر تابع ارزیابی نادقیق باشد، آلفا–بتا نادقیق خواهد بود. محاسبهٔ اشتباه روی یک گره تکی می‌تواند آلفا–بتا را به‌اشتباه به انتخاب (یا اجتناب از) مسیری به آن گره بکشاند. اما جستجوی مونت‌کارلو بر مجموع بسیاری بازی‌آزمایی تکیه می‌کند و بنابراین به‌اندازهٔ یک خطای تکی آسیب‌پذیر نیست. ترکیب MCTS و توابع ارزیابی ممکن است: برای تعداد معینی حرکت بازی‌آزمایی انجام دهید، اما سپس بازی‌آزمایی را کوتاه کنید و تابع ارزیابی را اعمال کنید. همچنین ترکیب جنبه‌های آلفا–بتا و جستجوی مونت‌کارلو ممکن است. برای نمونه، در بازی‌هایی که می‌توانند حرکت‌های بسیاری ادامه یابند، ممکن است بخواهیم از «خاتمهٔ زودهنگام بازی‌آزمایی» (Early Playout Termination) استفاده کنیم که در آن بازی‌آزمایی‌ای را که حرکت‌های زیادی می‌برد متوقف می‌کنیم و یا با تابع ارزیابی اکتشافی ارزیابی‌اش می‌کنیم یا صرفاً تساوی اعلامش می‌کنیم.

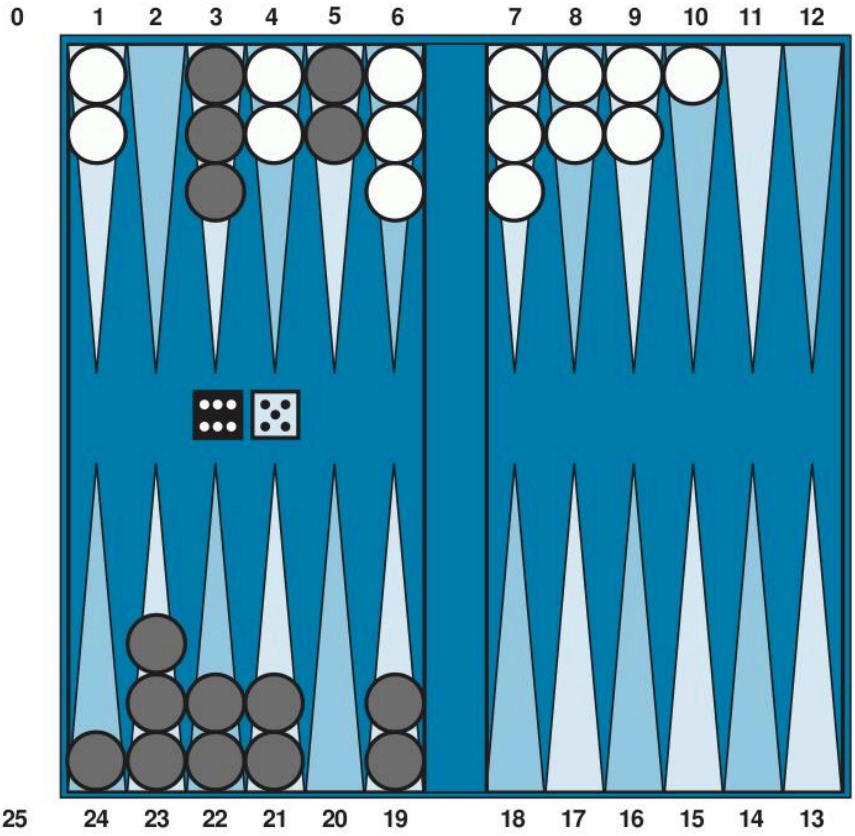
جستجوی مونت‌کارلو را می‌توان بر بازی‌های کاملاً تازه اعمال کرد که هیچ مجموعه‌تجربه‌ای برای تعریف تابع ارزیابی وجود ندارد. تا وقتی قواعد بازی را بدانیم، جستجوی مونت‌کارلو به هیچ اطلاعات اضافی نیاز ندارد. سیاست‌های انتخاب و بازی‌آزمایی می‌توانند از دانش خبرهٔ دست‌ساخته وقتی در دسترس است استفادهٔ خوبی کنند، اما سیاست‌های خوب را می‌توان با شبکه‌های عصبیِ آموزش‌دیده با بازی خودآموزِ تنها یاد گرفت.

جستجوی مونت‌کارلو زمانی نقطه‌ضعف دارد که محتمل باشد یک حرکت تکی بتواند مسیر بازی را تغییر دهد، زیرا سرشت تصادفی جستجوی مونت‌کارلو بدان معناست ممکن است آن حرکت را در نظر نگیرد. به بیان دیگر، هرس نوع B در جستجوی مونت‌کارلو بدان معناست که خط حیاتی بازی ممکن است اصلاً کاوش نشود. جستجوی مونت‌کارلو همچنین وقتی نقطه‌ضعف دارد که حالت‌های بازی «بدیهیاً» برد یک طرف یا طرف دیگرند (بر اساس دانش انسانی و تابع ارزیابی)، اما با این حال در بازی‌آزمایی حرکت‌های بسیاری لازم است تا برنده تأیید شود. مدتی طولانی بر این باور بودند آلفا–بتا برای بازی‌هایی مانند شطرنج با ضریب انشعاب کم و توابع ارزیابی خوب مناسب‌تر است، اما اخیراً رویکردهای مونت‌کارلو در شطرنج و بازی‌های دیگر موفقیت نشان داده‌اند.

ایدهٔ کلیِ شبیه‌سازی حرکت‌ها در آینده، مشاهدهٔ پیامد و استفاده از پیامد برای تعیین کدام حرکت‌ها خوب‌اند، نوعی یادگیری تقویتی است که در فصل ۲۲ پوشش داده می‌شود.

۵.۵ بازی‌های تصادفی

بازی‌های تصادفی (Stochastic Games) با گنجاندن عنصر تصادفی، مانند ریختن تاس، ما را کمی به پیش‌بینی‌ناپذیری زندگی واقعی نزدیک‌تر می‌کنند. تخته‌نرد بازی تصادفیِ نمونه‌ای است که شانس و مهارت را ترکیب می‌کند. در موضع تخته‌نرد شکل ۵.۱۲، سفید تاس 6–5 ریخته و چهار حرکت ممکن دارد (هر یک یک مهره را ۵ خانه به جلو (در جهت عقربه‌های ساعت) و یک مهره را ۶ خانه به جلو می‌برد).



شکل ۵.۱۲ — موضع نمونهٔ تخته‌نرد. هدف بازی بردن همهٔ مهره‌های خود از تخته است. سیاه در جهت عقربه‌های ساعت به سوی ۲۵ حرکت می‌کند و سفید در خلاف جهت عقربه‌های ساعت به سوی ۰. مهره می‌تواند به هر موضعی حرکت کند مگر آنکه چند مهرهٔ حریف آن‌جا باشد؛ اگر یک مهرهٔ حریف باشد، شکار می‌شود و باید از نو آغاز کند. در موضع نشان‌داده‌شده، سیاه تاس 6–5 ریخته و باید از میان چهار حرکت قانونی انتخاب کند: (5–11,5–10)، (5–11,19–24)، (5–10,10–16) و (5–11,11–16)، که نشانهٔ (5–11,11–16) یعنی یک مهره را از موضع ۵ به ۱۱ ببر و سپس مهره‌ای را از ۱۱ به ۱۶ ببر.

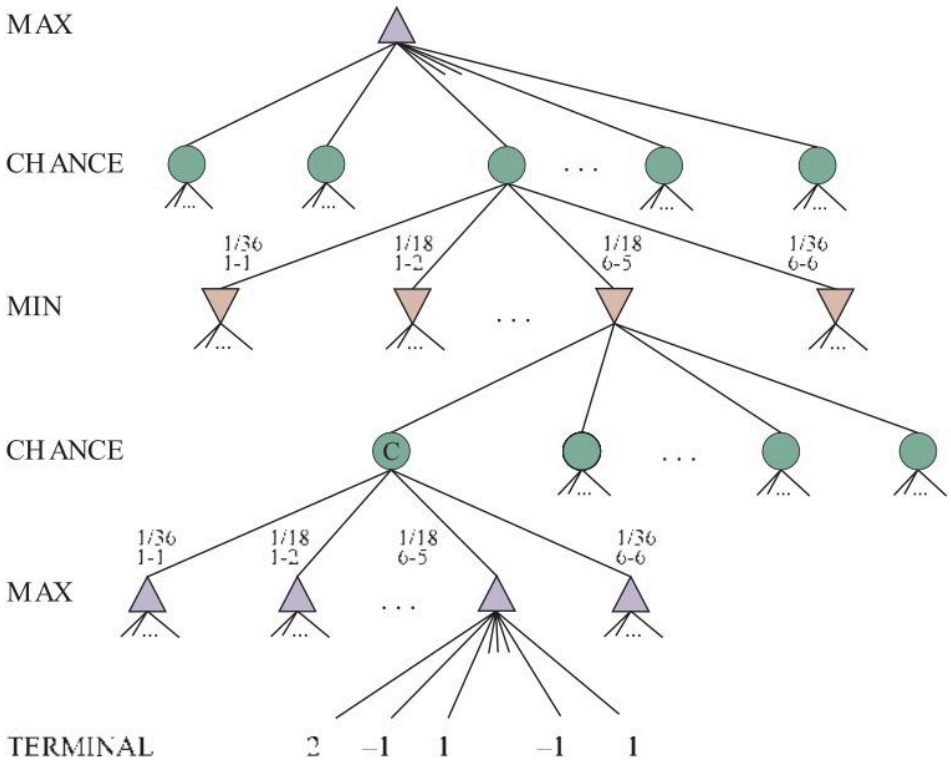
در این نقطه سیاه می‌داند چه حرکت‌هایی می‌تواند بکند، اما نمی‌داند سفید چه تاسی خواهد ریخت و بنابراین نمی‌داند حرکت‌های قانونی سفید چه خواهند بود. این بدان معناست سیاه نمی‌تواند درخت بازی استانداردی از نوعی که در شطرنج و دوز دیدیم بسازد. درخت بازی در تخته‌نرد باید افزون بر گره‌های MAX و MIN، «گره‌های شانس» (Chance Nodes) نیز شامل شود. گره‌های شانس در شکل ۵.۱۳ به‌صورت دایره نشان داده شده‌اند. شاخه‌های خروجی از هر گرهٔ شانس، تاس‌های ممکن را نشان می‌دهند؛ هر شاخه با تاس و احتمال آن برچسب‌گذاری شده. ۳۶ راه برای ریختن دو تاس هست که هر یک به یک اندازه محتمل است؛ اما چون 6–5 همان 5–6 است، فقط ۲۱ تاس متمایز هست. شش تاس دوبل (1–1 تا 6–6) هر یک احتمال 1/36 دارند، پس می‌گوییم P(1–1) = 1/36. پانزده تاس متمایز دیگر هر یک احتمال 1/18 دارند.

گام بعدی فهم چگونگی گرفتن تصمیم‌های درست است. بدیهی است همچنان می‌خواهیم حرکتی را برگزینیم که به بهترین موضع می‌انجامد. اما موضع‌ها مقدار مینیمکس قطعی ندارند. در عوض، فقط می‌توانیم «مقدار مورد انتظار» (Expected Value) موضع را محاسبه کنیم: میانگین روی همهٔ پیامدهای ممکن گره‌های شانس.

این ما را به «مقدار اکسپکتیمینیمکس» (Expectiminimax Value) برای بازی‌های با گره‌های شانس می‌رساند؛ تعمیمی از مقدار مینیمکس برای بازی‌های جبرگرایانه. گره‌های پایانی و گره‌های MAX و MIN دقیقاً مثل قبل کار می‌کنند (با این ملاحظه که حرکت‌های قانونی MAX و MIN به پیامد تاسِ گرهٔ شانس قبلی بستگی خواهند داشت). برای گره‌های شانس، مقدار مورد انتظار را محاسبه می‌کنیم که مجموع مقدار روی همهٔ پیامدها، وزن‌دهی‌شده با احتمال هر کنش شانسی است:

```
EXPECTIMINIMAX(s) =
  UTILITY(s, MAX)                                  اگر IS-TERMINAL(s)
  max_{a ∈ Actions(s)} EXPECTIMINIMAX(RESULT(s, a))  اگر TO-MOVE(s) = MAX
  min_{a ∈ Actions(s)} EXPECTIMINIMAX(RESULT(s, a))  اگر TO-MOVE(s) = MIN
  Σ_r P(r) EXPECTIMINIMAX(RESULT(s, r))            اگر TO-MOVE(s) = CHANCE
```

که r یک تاس ممکن (یا رویداد شانسی دیگر) و RESULT(s, r) همان حالت s با این واقعیت اضافی است که نتیجهٔ تاس r است.

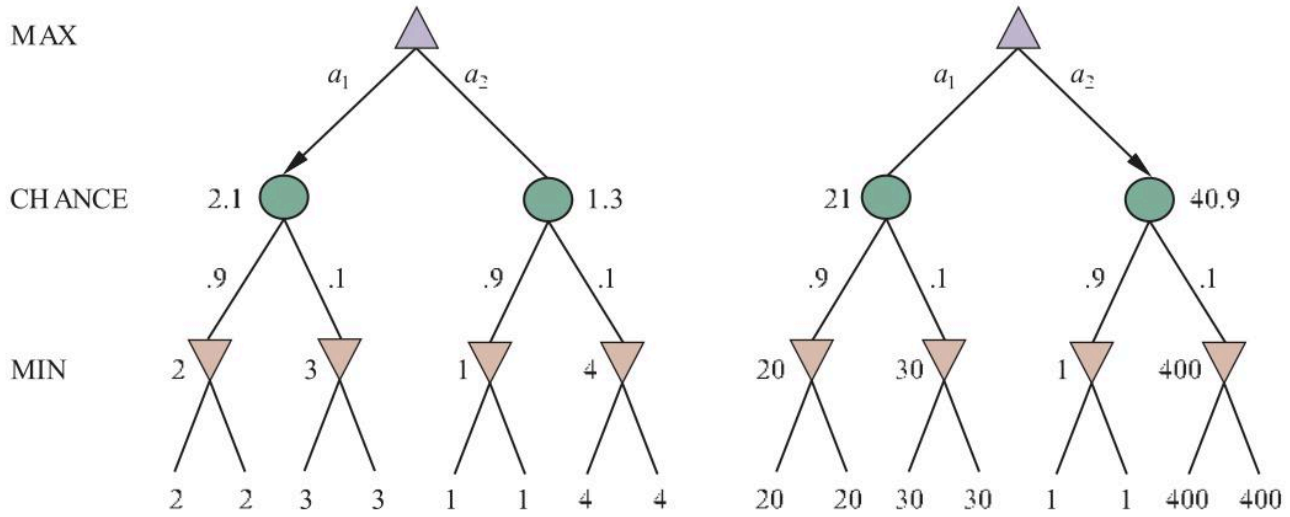


شکل ۵.۱۳ – درخت بازی شماتیک برای موضعی در تخته‌نرد.

۵.۵.۱ توابع ارزیابی برای بازی‌های شانسی

مانند مینیمکس، تقریب بدیهی برای اکسپکتیمینیمکس آن است که جستجو را در نقطه‌ای قطع کنیم و تابع ارزیابی را بر هر برگ اعمال کنیم. شاید فکر کنید توابع ارزیابی برای بازی‌هایی مانند تخته‌نرد باید دقیقاً مانند توابع ارزیابی شطرنج باشند – فقط باید به موضع‌های بهتر مقادیر بالاتر بدهند. اما در واقع، حضور گره‌های شانس بدان معناست باید دربارهٔ معنای مقادیر دقت بیشتری به خرج داد.

شکل ۵.۱۴ نشان می‌دهد چه اتفاقی می‌افتد: با تابع ارزیابی که مقادیر [1, 2, 3, 4] را به برگ‌ها نسبت می‌دهد، حرکت a1 بهترین است؛ با مقادیر [1, 20, 30, 400]، حرکت a2 بهترین است. از این رو، برنامه اگر تغییری در برخی از مقادیر ارزیابی بدهیم رفتاری کاملاً متفاوت خواهد داشت، حتی اگر ترتیب ترجیح یکسان بماند.



شکل ۵.۱۴ – تبدیل حفظ‌کنندهٔ ترتیب روی مقادیر برگ، بهترین حرکت را تغییر می‌دهد.

معلوم می‌شود برای پرهیز از این مسئله، تابع ارزیابی باید مقادیری بازگرداند که «تبدیل خطی مثبتِ» احتمال برد باشند (یا مطلوبیت مورد انتظار، برای بازی‌هایی که پیامدهایی غیر از برد/باخت دارند). این رابطه با احتمال، ویژگی مهم و عمومیِ موقعیت‌های درگیر با عدم قطعیت است و در فصل ۱۶ بیشتر بحثش می‌کنیم. اگر برنامه از پیش همهٔ تاس‌هایی را می‌دانست که برای بقیهٔ بازی رخ خواهند داد، حل بازی با تاس دقیقاً مانند حل بازی بدون تاس بود که مینیمکس در $O(b^m)$ انجامش می‌دهد، که b ضریب انشعاب و m بیشینهٔ عمق درخت بازی است. چون اکسپکتیمینیمکس همچنین همهٔ توالی‌های تاسِ ممکن را در نظر می‌گیرد، $O(b^m n^m)$ طول می‌کشد، که n شمار تاس‌های متمایز است.

حتی اگر جستجو به عمق کوچک d محدود شود، هزینهٔ اضافی در مقایسه با مینیمکس، در نظر گرفتنِ خیلی دور به جلو را در بیشتر بازی‌های شانسی غیرواقعی می‌کند. در تخته‌نرد n برابر ۲۱ و b معمولاً حدود ۲۰ است، اما در برخی موقعیت‌ها برای تاس‌های دوبل می‌تواند تا ۴۰۰۰ هم برود. ما احتمالاً فقط می‌توانستیم سه نیم‌حرکت جستجو را مدیریت کنیم.

راه دیگر اندیشیدن به مسئله چنین است: مزیت آلفا–بتا آن است که تحولات آینده‌ای را نادیده می‌گیرد که با بازی خوب قرار نیست رخ دهند. پس بر رخدادهای محتمل تمرکز می‌کند. اما در بازی‌ای که پرتاب دو تاس پیش از هر حرکت است، توالی‌های حرکتِ محتملی وجود ندارد؛ حتی محتمل‌ترین حرکت فقط در 2/36 مواقع رخ می‌دهد، زیرا برای وقوع حرکت، تاس‌ها باید نخست به‌گونه‌ای درست درآیند که آن را قانونی کنند. این مسئلهٔ عمومی هرگاه عدم قطعیت وارد تصویر می‌شود هست: امکان‌ها به‌شکل عظیمی چند برابر می‌شوند و ساختن برنامه‌های کنشِ مفصل بی‌معنا می‌شود، زیرا جهان احتمالاً همکاری نخواهد کرد.

شاید به ذهنتان رسیده باشد چیزی مانند هرس آلفا–بتا را می‌توان بر درخت‌های بازی با گره‌های شانس اعمال کرد. معلوم می‌شود می‌شود. تحلیل برای گره‌های MIN و MAX تغییر نمی‌کند، اما می‌توانیم با کمی نبوغ گره‌های شانس را نیز هرس کنیم. گرهٔ شانس C در شکل ۵.۱۳ و آنچه با مقدارش هنگام ارزیابی فرزندانش رخ می‌دهد را در نظر بگیرید. آیا ممکن است کران بالایی بر مقدار C پیش از نگاه‌کردن به همهٔ فرزندانش بیابیم؟ (به یاد آورید این همان چیزی است که آلفا–بتا برای هرس گره و زیردرختش نیاز دارد.) در نگاه نخست، ممکن است ناممکن بنماید، زیرا مقدار C میانگین مقادیر فرزندانش است و برای محاسبهٔ میانگین مجموعه‌ای از اعداد، باید به همهٔ اعداد نگاه کنیم. اما اگر کران‌هایی بر مقادیر ممکن تابع مطلوبیت بگذاریم، می‌توانیم بدون نگاه‌کردن به هر عدد به کران‌هایی برای میانگین برسیم. برای نمونه، بگوییم همهٔ مقادیر مطلوبیت بین -2 و +2 هستند؛ آنگاه مقدار گره‌های برگ کران‌دار است و به‌نوبهٔ خود می‌توانیم کران بالایی بر مقدار گرهٔ شانس بدون نگاه‌کردن به همهٔ فرزندانش بگذاریم.

در بازی‌هایی که ضریب انشعاب گره‌های شانس بالاست — بازی‌ای مانند یاتزی را در نظر بگیرید که در هر نوبت ۵ تاس می‌ریزید — شاید بخواهید هرس رو به جلویی را در نظر بگیرید که شمار کوچک‌تری از شاخه‌های شانسِ ممکن را نمونه‌گیری می‌کند. یا شاید بخواهید به‌کلی از تابع ارزیابی اجتناب کنید و به‌جایش جستجوی درختی مونت‌کارلو را برگزینید، که در آن هر بازی‌آزمایی شامل تاس‌های تصادفی است.

۵.۶ بازی‌های تا حدی قابل مشاهده

بابی فیشر اعلام کرد «شطرنج جنگ است»، اما شطرنج دست‌کم یک ویژگی عمدهٔ جنگ‌های واقعی را ندارد، یعنی مشاهده‌ناپذیری تا حدی. در «مه جنگ»، مکان یگان‌های دشمن اغلب تا آشکارشدن با تماس مستقیم ناشناخته است. در نتیجه، جنگ شامل استفاده از پیش‌آهنگ‌ها و جاسوس‌ها برای جمع‌آوری اطلاعات و استفاده از پنهان‌کاری و بلوف برای گیج‌کردن دشمن است. بازی‌های تا حدی قابل مشاهده این ویژگی‌ها را مشترک‌اند و بنابراین از نظر کیفی با بازی‌های بخش‌های پیشین متفاوت‌اند. بازی‌های ویدئویی مانند استارکرافت به‌ویژه چالش‌برانگیزند، زیرا تا حدی قابل مشاهده، چندعاملی، ناجبرگرایانه، پویا و ناشناخته‌اند.

در بازی‌های جبرگرایانهٔ تا حدی قابل مشاهده، عدم قطعیت دربارهٔ حالت تخته کاملاً از نبودِ دسترسی به انتخاب‌های حریف ناشی می‌شود. این رده شامل بازی‌های کودکان مانند نبرد کشتی‌ها (جایی که کشتی‌های هر بازیکن در مکان‌های پنهان از حریف گذاشته می‌شوند) و استراتگو (جایی که مکان مهره‌ها معلوم اما نوع مهره‌ها پنهان است) می‌شود. بازی «کریگ‌اشپیل» (Kriegspiel) را بررسی می‌کنیم؛ تغییر تا حدی قابل مشاهدهٔ شطرنج که در آن مهره‌ها کاملاً از حریف نامرئی‌اند. بازی‌های دیگر نیز نسخه‌های تا حدی قابل مشاهده دارند: گوی شبح، دوز شبح و شوگی پرده‌ای.

۵.۶.۱ کریگ‌اشپیل: شطرنج تا حدی قابل مشاهده

قواعد کریگ‌اشپیل چنین است: سفید و سیاه هر یک تخته‌ای می‌بینند که فقط مهره‌های خودشان را دارد. داور (Referee) که همهٔ مهره‌ها را می‌بیند، بازی را داوری می‌کند و دوره‌ای اعلام‌هایی می‌کند که هر دو بازیکن می‌شنوند. نخست، سفید حرکتی را به داور پیشنهاد می‌کند که اگر مهره‌های سیاه نبودند قانونی می‌بود. اگر مهره‌های سیاه مانع

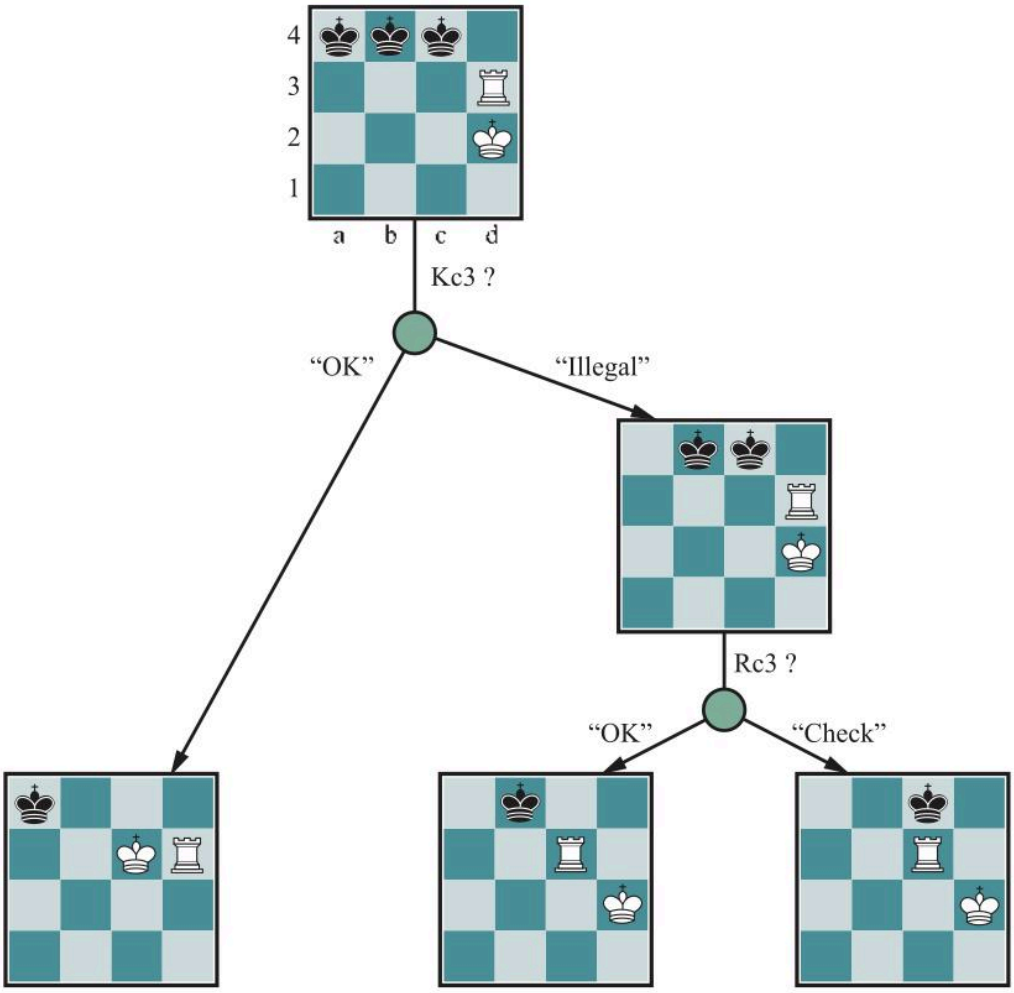
حرکت شوند، داور اعلام می‌کند «غیرقانونی» و سفید به پیشنهاد حرکت‌ها ادامه می‌دهد تا حرکت قانونی‌ای یافت شود — و در این فرایند دربارهٔ مکان مهره‌های سیاه بیشتر می‌آموزد.

به‌محض پیشنهاد حرکت قانونی، داور یک یا چند مورد از این‌ها را اعلام می‌کند: «شکار در خانهٔ X» اگر شکاری هست، و «کیش از D» اگر شاه سیاه در کیش است، که D جهت کیش است و می‌تواند یکی از «اسب»، «ردیف»، «ستون»، «قطر بلند» یا «قطر کوتاه» باشد. اگر سیاه کیش‌ومات یا پات باشد، داور می‌گوید؛ وگرنه نوبت حرکت سیاه است.

کریگ‌اشپیل ممکن است به‌طرز وحشتناکی ناممکن بنماید، اما انسان‌ها آن را خوب مدیریت می‌کنند و برنامه‌های رایانه‌ای در حال رسیدن‌اند. یادآوری مفهوم حالت باورِ تعریف‌شده در بخش ۴.۴ و نشان‌داده‌شده در شکل ۴.۱۴ کمک می‌کند — مجموعهٔ همهٔ حالت‌های تختهٔ منطقاً ممکن با توجه به تاریخچهٔ کامل ادراک‌های تاکنون. در آغاز، حالت باور سفید تکی است، زیرا مهره‌های سیاه هنوز حرکت نکرده‌اند. پس از آنکه سفید حرکتی می‌کند و سیاه پاسخ می‌دهد، حالت باور سفید شامل ۲۰ موضع است، زیرا سیاه ۲۰ پاسخ به هر حرکت گشایشی دارد. پیگیری حالت باور هنگام پیش‌رفتن بازی دقیقاً مسئلهٔ تخمین حالت است که گام به‌روزرسانی‌اش در معادلهٔ (۴.۶) در صفحهٔ ۱۳۲ داده شده.[4] می‌توانیم تخمین حالت کریگ‌اشپیل را مستقیماً بر چارچوب تا حدی قابل مشاهده و ناجبرگرایانهٔ بخش ۴.۴ نگاشت کنیم اگر حریف را منشأ ناجبرگرایی بدانیم؛ یعنی RESULTS حرکت سفید از پیامد (قابل پیش‌بینی) حرکتِ خودِ سفید و پیامدِ پیش‌بینی‌ناپذیرِ داده‌شده توسط پاسخ سیاه ترکیب می‌شود.

با حالت باور کنونی، سفید ممکن است بپرسد: «آیا می‌توانم بازی را ببرم؟» برای بازی تا حدی قابل مشاهده، مفهوم راهبرد تغییر می‌کند؛ به‌جای مشخص‌کردن حرکتی برای هر حرکت ممکن حریف، برای هر توالی ادراکِ ممکنِ دریافتی به حرکتی نیاز داریم.

برای کریگ‌اشپیل، راهبرد برنده، یا «کیش‌ومات تضمینی» (Guaranteed Checkmate)، راهبردی است که برای هر توالی ادراک ممکن، به کیش‌ومات واقعی برای هر حالت تختهٔ ممکن در حالت باور کنونی می‌انجامد، صرف‌نظر از نحوهٔ حرکت حریف. با این تعریف، حالت باور حریف بی‌ربط است — راهبرد باید حتی اگر حریف همهٔ مهره‌ها را ببیند کار کند. این محاسبه را بسیار ساده می‌کند. شکل ۵.۱۵ بخشی از کیش‌ومات تضمینی را در پایان‌بازی KRK (شاه و رخ در برابر شاه) نشان می‌دهد. در این مورد، سیاه فقط یک مهره (شاه) دارد، پس حالت باور سفید را می‌توان در یک تخته با علامت‌زدن هر موضع ممکن شاه سیاه نشان داد. الگوریتم عمومی جستجوی AND–OR را می‌توان بر فضای حالت باور برای یافتن کیش‌ومات‌های تضمینی اعمال کرد، درست مانند بخش ۴.۴. الگوریتم حالت باورِ افزایشیِ یادشده در بخش ۴.۴.۲ اغلب کیش‌ومات‌های میانه‌بازی تا عمق ۹ می‌یابد — بسیار فراتر از توانایی‌های بیشتر بازیکنان انسانی.



شکل ۵.۱۵ – بخشی از کیش‌ومات تضمینی در پایان‌بازی KRK، نشان‌داده‌شده روی تختهٔ کوچک‌شده. در حالت باور آغازین، شاه سیاه در یکی از سه مکان ممکن است. با ترکیبی از حرکت‌های کاوشگر، راهبرد این را به یک مکان محدود می‌کند. تکمیل کیش‌ومات به‌عنوان تمرین واگذار می‌شود.

افزون بر کیش‌ومات‌های تضمینی، کریگ‌اشپیل مفهوم کاملاً تازه‌ای را می‌پذیرد که در بازی‌های کاملاً قابل مشاهده معنایی ندارد: «کیش‌ومات احتمالاتی» (Probabilistic Checkmate). چنین کیش‌ومات‌هایی همچنان باید در هر حالت تختهٔ حالت باور کار کنند؛ آن‌ها از نظر تصادفی‌سازیِ حرکت‌های بازیکن برنده احتمالاتی‌اند. برای درک ایدهٔ پایه، مسئلهٔ یافتن شاه سیاهِ تنها را با فقط شاه سفید در نظر بگیرید. صرفاً با حرکت تصادفی، شاه سفید سرانجام به شاه سیاه خواهد خورد حتی اگر دومی بکوشد از این سرنوشت بگریزد، زیرا سیاه نمی‌تواند بی‌نهایت حرکت‌های فرارِی درست را حدس بزند. به زبان نظریهٔ احتمال، شناسایی با احتمال ۱ رخ می‌دهد.

پایان‌بازی KBNK – شاه، فیل و اسب در برابر شاه – به این معنا برده می‌شود؛ سفید توالی تصادفی بی‌نهایتی از انتخاب‌ها به سیاه ارائه می‌دهد که برای یکی از آن‌ها سیاه اشتباه حدس می‌زند و موضعش را آشکار می‌کند و به کیش‌ومات می‌انجامد. از سوی دیگر، پایان‌بازی KBBK با احتمال 1 - ε برده می‌شود. سفید تنها با پذیرفتن یک ریسک حساب‌شده می‌تواند برد را تحمیل کند: رها کردن یکی از فیل‌ها بدون محافظ برای یک حرکت. اگر سیاه اتفاقاً در جای

درست باشد و فیل را شکار کند (حرکتی که اگر فیل‌ها محافظت می‌شدند غیرقانونی می‌بود)، بازی مساوی می‌شود. سفید می‌تواند انتخاب کند حرکت پرریسک را در نقطه‌ای تصادفی‌برگزیده در میانهٔ توالی بسیار بلندی انجام دهد و بدین ترتیب ε را به ثابتی دلخواه کوچک کاهش دهد، اما نمی‌تواند ε را به صفر برساند.

گاهی راهبرد کیش‌ومات برای برخی از حالت‌های تخته در حالت باور کنونی کار می‌کند اما نه بقیه. آزمودن چنین راهبردی ممکن است موفق شود و به «کیش‌ومات اتفاقی» (Accidental Checkmate) بینجامد — اتفاقی به این معنا که سفید نمی‌توانست بداند کیش‌ومات خواهد بود — اگر مهره‌های سیاه اتفاقاً در مکان‌های درست باشند. (بیشتر کیش‌ومات‌ها در بازی‌های میان انسان‌ها از این نوع اتفاقی‌اند.) این ایده به‌طور طبیعی به این پرسش می‌انجامد که احتمال بردِ راهبردِ داده‌شده چقدر است، که به‌نوبهٔ خود به این پرسش می‌انجامد که احتمال اینکه هر حالت تخته در حالت باور کنونی، حالت تختهٔ واقعی باشد چقدر است.

نخستین گرایش ممکن است پیشنهاد این باشد که همهٔ حالت‌های تخته در حالت باور کنونی به یک اندازه محتمل‌اند — اما این نمی‌تواند درست باشد. برای نمونه، حالت باور سفید را پس از نخستین حرکت سیاه در بازی در نظر بگیرید. طبق تعریف (با فرض بازی بهینهٔ سیاه)، سیاه باید حرکت بهینه‌ای بازی کرده باشد، پس به همهٔ حالت‌های تختهٔ برآمده از حرکت‌های فروبهینه باید احتمال صفر نسبت داده شود.

این استدلال نیز کاملاً درست نیست، زیرا هدف هر بازیکن نه فقط حرکت‌دادن مهره‌ها به خانه‌های درست، بلکه کمینه‌کردن اطلاعاتی است که حریف دربارهٔ مکان آن‌ها دارد. بازی‌کردن هر راهبردِ قابل‌پیش‌بینی «بهینه»، اطلاعات به حریف می‌دهد. از این رو، بازی بهینه در بازی‌های تا حدی قابل مشاهده مستلزم تمایل به بازی تا حدی تصادفی است. (به همین دلیل بازرسان بهداشت رستوران بازرسی‌های بازدید تصادفی انجام می‌دهند.) این بدان معناست گاهی حرکت‌هایی انتخاب شوند که «ذاتاً» ضعیف می‌نمایند — اما از همان پیش‌بینی‌ناپذیری‌شان قدرت می‌گیرند، زیرا حریف به احتمال زیاد دفاعی برایشان آماده نکرده است.

از این ملاحظات، به نظر می‌رسد احتمال‌های مرتبط با حالت‌های تخته در حالت باور کنونی را فقط با راهبرد تصادفیِ بهینه‌ای می‌توان محاسبه کرد؛ به‌نوبهٔ خود، محاسبهٔ آن راهبرد به‌نظر می‌رسد مستلزم دانستن احتمال‌های حالت‌های گوناگونی است که تخته ممکن است در آن‌ها باشد. این معمّا را می‌توان با پذیرش مفهوم نظریه‌بازیِ «راه‌حل تعادلی» (Equilibrium Solution) حل کرد که در فصل ۱۷ بیشتر دنبالش می‌کنیم. تعادل، راهبرد تصادفی بهینه‌ای برای هر بازیکن مشخص می‌کند. محاسبهٔ تعادل‌ها برای کریگ‌اشپیل بسیار پرهزینه است. در حال حاضر، طراحی الگوریتم‌های مؤثر برای بازی عمومی کریگ‌اشپیل موضوع پژوهشی باز است. بیشتر سامانه‌ها نگاه به جلوی عمق‌کران‌دار در فضای حالت باور خودشان انجام می‌دهند و حالت باور حریف را نادیده می‌گیرند. توابع ارزیابی شبیه توابع بازی قابل مشاهده‌اند اما مؤلفه‌ای برای اندازهٔ حالت باور شامل می‌شوند — کوچک‌تر بهتر است!

در بخش ۱۸.۲ زیر عنوان نظریهٔ بازی به بازی‌های تا حدی قابل مشاهده بازخواهیم گشت.

۵.۶.۲ بازی‌های ورقی

بازی‌های ورقی مانند بریج، ویست، دل‌ها و پوکر دارای مشاهده‌ناپذیری تصادفیِ تا حدی‌اند، که اطلاعات گم‌شده با پخش تصادفی ورق‌ها تولید می‌شود.

در نگاه نخست، ممکن است به نظر برسد این بازی‌های ورقی دقیقاً مانند بازی‌های تاس‌اند: ورق‌ها تصادفی پخش می‌شوند و حرکت‌های در دسترس هر بازیکن را تعیین می‌کنند، اما همهٔ «تاس‌ها» در آغاز ریخته می‌شوند! حتی با آنکه این قیاس نادرست از آب درمی‌آید، الگوریتمی پیشنهاد می‌کند: آغاز بازی را به‌عنوان گرهٔ شانسی با هر پخشِ ممکن به‌عنوان پیامد در نظر بگیرید و سپس از فرمول EXPECTIMINIMAX برای برگزیدن بهترین حرکت استفاده کنید. توجه کنید در این رویکرد تنها گرهٔ شانس، گرهٔ ریشه است؛ پس از آن بازی کاملاً قابل مشاهده می‌شود. این رویکرد گاهی «میانگین‌گیری روی آینده‌بینی» (Averaging over Clairvoyance) نامیده می‌شود، زیرا فرض می‌کند به‌محض وقوع پخشِ واقعی، بازی برای هر دو بازیکن کاملاً قابل مشاهده می‌شود. با وجود جذابیت شهودی‌اش، این راهبرد می‌تواند انسان را به گمراهی بکشاند. داستان زیر را در نظر بگیرید:

- روز ۱: جادهٔ A به دیگ طلا می‌رسد؛ جادهٔ B به دوشاخه می‌رسد. می‌بینید شاخهٔ چپ به دو دیگ طلا می‌رسد و شاخهٔ راست به زیرگرفتن شما توسط اتوبوس.
- روز ۲: جادهٔ A به دیگ طلا می‌رسد؛ جادهٔ B به دوشاخه می‌رسد. می‌بینید شاخهٔ راست به دو دیگ طلا می‌رسد و شاخهٔ چپ به زیرگرفتن شما توسط اتوبوس.
- روز ۳: جادهٔ A به دیگ طلا می‌رسد؛ جادهٔ B به دوشاخه می‌رسد. به شما گفته می‌شود یکی از شاخه‌ها به دو دیگ طلا می‌رسد و یکی به زیرگرفتن شما توسط اتوبوس. متأسفانه نمی‌دانید کدام شاخه کدام است.

میانگین‌گیری روی آینده‌بینی به استدلال زیر می‌انجامد: در روز ۱، B انتخاب درست است؛ در روز ۲، B انتخاب درست است؛ در روز ۳، وضعیت همان روز ۱ یا روز ۲ است، پس B باید همچنان انتخاب درست باشد.

حال می‌بینیم میانگین‌گیری روی آینده‌بینی چگونه شکست می‌خورد: حالت باوری را که عامل پس از کنش در آن خواهد بود در نظر نمی‌گیرد. حالت باورِ نادانی کامل مطلوب نیست، به‌ویژه وقتی یکی از امکان‌ها مرگ قطعی است. چون فرض می‌کند هر حالت آینده به‌طور خودکار از نوع دانش کامل خواهد بود، رویکرد آینده‌بینی هرگز کنش‌هایی را که اطلاعات جمع می‌کنند (مانند نخستین حرکت در شکل ۵.۱۵) انتخاب نمی‌کند؛ همچنین کنش‌هایی را که اطلاعات را از حریف پنهان می‌کنند یا اطلاعات به هم‌دسته می‌دهند انتخاب نمی‌کند، زیرا فرض می‌کند آن‌ها از قبل اطلاعات را می‌دانند؛ و هرگز در پوکر «بلوف» (Bluff) نخواهد زد،[5] زیرا فرض می‌کند حریف ورق‌هایش را می‌بیند. در فصل ۱۷ نشان می‌دهیم چگونه الگوریتم‌هایی بسازیم که همهٔ این کارها را با حلِ مسئلهٔ تصمیمِ واقعاً تا حدی قابل مشاهده انجام می‌دهند و به راهبرد تعادلی بهینه می‌انجامند (بخش ۱۸.۲ را ببینید).

با وجود کاستی‌ها، میانگین‌گیری روی آینده‌بینی می‌تواند راهبردی مؤثر باشد، با چند حربه برای بهتر کارکردنش. در بیشتر بازی‌های ورقی، شمار پخش‌های ممکن نسبتاً بزرگ است. برای نمونه، در بازی بریج، هر بازیکن فقط دو دست از چهار دست را می‌بیند؛ دو دستِ نادیده با ۱۳ ورق هر یک هست، پس شمار پخش‌ها ۱۰,۴۰۰,۶۰۰ = C(26,13) است. حل حتی

یک پخش بسیار دشوار است، پس حل ده میلیون خارج از بحث است. یکی از راه‌های سروکار داشتن با این شمار عظیم، «انتزاع» است: یعنی درمان دست‌های مشابه به‌عنوان یکسان. برای نمونه، بسیار مهم است کدام آس‌ها و شاه‌ها در دست هستند، اما اینکه دست ۴ دارد یا ۵ چندان مهم نیست و می‌توان انتزاعش کرد.

راه دیگر سروکار داشتن با شمار بزرگ، هرس رو به جلوست: فقط نمونهٔ تصادفی کوچکی از N پخش را در نظر بگیرید و دوباره امتیاز EXPECTIMINIMAX را محاسبه کنید. حتی برای N نسبتاً کوچک — مثلاً ۱۰۰ تا ۱۰۰۰ — این روش تقریب خوبی می‌دهد. همچنین می‌توان آن را بر بازی‌های جبرگرایانه مانند کریگ‌اشپیل اعمال کرد، که در آن‌ها به‌جای نمونه‌گیری روی پخش‌های ممکن، روی حالت‌های ممکن بازی نمونه می‌گیریم، تا وقتی راهی برای تخمین محتمل‌بودن هر حالت داشته باشیم. همچنین می‌تواند مفید باشد که به‌جای جستجوی کل درخت بازی، جستجوی اکتشافی با قطع عمق انجام دهیم. تاکنون فرض کرده‌ایم هر پخش به یک اندازه محتمل است. این برای بازی‌هایی مانند ویست و دل‌ها منطقی است. اما برای بریج، پیش از بازی «فاز مناقصه» (Bidding Phase) هست که در آن هر تیم نشان می‌دهد انتظار دارد چند دست ببرد. چون بازیکنان بر اساس ورق‌هایی که دارند مناقصه می‌کنند، دیگر بازیکنان چیزی دربارهٔ احتمال P(s) هر پخش می‌آموزند. در نظر گرفتن این هنگام تصمیم‌گیری دربارهٔ نحوهٔ بازی دست، به دلایل یادشده در توصیف کریگ‌اشپیل، ظریف است: بازیکنان ممکن است چنان مناقصه کنند که اطلاعات منتقل‌شده به حریفان را کمینه کنند.

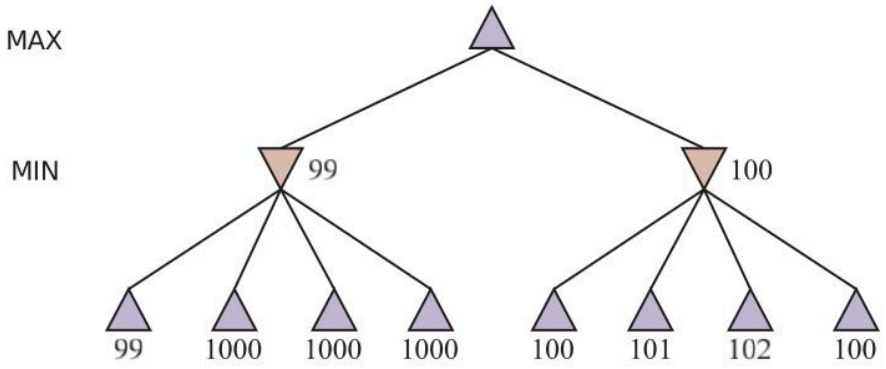
رایانه‌ها در پوکر به سطح عملکردی ابرانسانی رسیده‌اند. برنامهٔ پوکر لیبراتوس (Libratus) در دیداری ۲۰ روزه از پوکر تگزاس هولدمِ بدون-محدودیت با چهار نفر از برترین بازیکنان پوکر جهان روبه‌رو شد و قاطعانه همه را شکست داد. چون حالت‌های ممکن در پوکر بسیار زیاد است، لیبراتوس از انتزاع برای کاهش فضای حالت استفاده می‌کند: ممکن است دو دست AAA72 و AAA64 را معادل بداند (هر دو «سه آس و چند ورق پایین»اند) و ممکن است شرط ۲۰۰ دلار را با ۲۰۱ دلار یکسان بداند. اما لیبراتوس همچنین بازیکنان دیگر را زیر نظر دارد و اگر تشخیص دهد از انتزاعی بهره‌برداری می‌کنند، یک‌شبانه‌روز محاسبهٔ اضافی انجام می‌دهد تا آن سوراخ را ببندد. در مجموع از ۲۵ میلیون ساعت CPU روی ابررایانه برای به‌دست‌آوردن برد استفاده کرد.

هزینه‌های محاسباتیِ متحمل‌شده توسط لیبراتوس (و هزینه‌های مشابه آلفازیرو و سامانه‌های دیگر) پیشنهاد می‌کند بازی قهرمانی جهان ممکن است برای پژوهشگران با بودجهٔ محدود دست‌یافتنی نباشد. تا حدی درست است: همان‌گونه که نباید انتظار داشته باشید بتوانید ماشین مسابقهٔ فرمول یک قهرمان را از قطعات یدکی در گاراژتان سرهم کنید، دسترسی به ابررایانه‌ها یا سخت‌افزارهای تخصصی مانند واحدهای پردازش تنسور برتری‌ای دارد. این به‌ویژه هنگام آموزش سامانه صادق است، اما آموزش را می‌توان از راه جمع‌سپاری (Crowdsourcing) نیز انجام داد. برای نمونه، سامانهٔ متن‌باز لیلازیرو (LEELAZERO) بازپیاده‌سازی آلفازیرو است که با بازی خودآموز روی رایانه‌های داوطلبان آموزش می‌بیند. پس از آموزش، نیازهای محاسباتی برای بازی مسابقه‌ایِ واقعی اندک‌اند. آلفاستار (ALPHASTAR) بازی‌های استارکرافت ۲ را روی رایانهٔ رومیزی معمولی با یک GPU برد و آلفازیرو را می‌شد در آن حالت اجرا کرد.

۵.۷ محدودیت‌های الگوریتم‌های جستجوی بازی

چون محاسبهٔ تصمیم‌های بهینه در بازی‌های پیچیده ناشدنی است، همهٔ الگوریتم‌ها باید فرض‌ها و تقریب‌هایی کنند. جستجوی آلفا–بتا از تابع ارزیابی اکتشافی به‌عنوان تقریب استفاده می‌کند و جستجوی مونت‌کارلو میانگین تقریبی را بر روی انتخاب تصادفی بازی‌آزمایی‌ها محاسبه می‌کند. انتخاب الگوریتمِ استفاده‌شده تا حدی به ویژگی‌های هر بازی بستگی دارد: وقتی ضریب انشعاب بالاست یا تعریف تابع ارزیابی دشوار است، جستجوی مونت‌کارلو ترجیح داده می‌شود. اما هر دو الگوریتم از محدودیت‌های بنیادین رنج می‌برند.

یکی از محدودیت‌های جستجوی آلفا–بتا، آسیب‌پذیری‌اش در برابر خطاهای تابع اکتشافی است. شکل ۵.۱۶ درخت بازی دونیم‌حرکته‌ای را نشان می‌دهد که مینیمکس برایش برداشتن شاخهٔ راست را پیشنهاد می‌کند، زیرا 100 > 99. اگر همهٔ ارزیابی‌ها دقیقاً دقیق باشند، این حرکت درست است. اما فرض کنیم ارزیابی هر گره خطایی دارد که مستقل از گره‌های دیگر است و با انحراف معیار σ به‌صورت تصادفی توزیع شده. آنگاه شاخهٔ چپ در واقع ۷۱٪ مواقع بهتر است وقتی σ = 5، و ۵۸٪ مواقع وقتی σ = 2 (زیرا یکی از چهار برگ سمت راست در این موارد احتمالاً به پایین‌تر از ۹۹ لغزد). اگر خطاهای تابع ارزیابی مستقل نباشند، احتمال اشتباه بالا می‌رود. جبران این دشوار است، زیرا مدل خوبی از وابستگی‌های میان مقادیر گره‌های خواهر نداریم.



شکل ۵.۱۶ – درخت بازی دونیم‌حرکته‌ای که مینیمکس اکتشافی ممکن است در آن خطا کند.

محدودیت دوم هر دو آلفا–بتا و مونت‌کارلو آن است که برای محاسبهٔ (کران‌هایی بر) مقادیر حرکت‌های قانونی طراحی شده‌اند. اما گاهی حرکتی هست که بدیهیاً بهترین است (برای نمونه وقتی فقط یک حرکت قانونی هست) و در آن صورت دلیلی ندارد زمان محاسبه را برای فهمیدن مقدار حرکت تلف کنیم – بهتر است فقط حرکت را انجام دهیم. الگوریتم جستجوی بهتر از ایدهٔ «مطلوبیت گسترش گره» استفاده می‌کرد و گسترش‌های گرهِ با مطلوبیت بالا را انتخاب می‌کرد – یعنی آن‌هایی که به احتمال زیاد به کشف حرکتِ به‌شکل چشمگیری بهتر می‌انجامند. اگر هیچ گسترش گرهی با مطلوبیت بالاتر از هزینه‌اش (بر حسب زمان) نباشد، الگوریتم باید جستجو را متوقف و حرکتی انجام دهد. این نه فقط برای موقعیت‌های با نامزدِ واضح، بلکه برای مورد حرکت‌های متقارن نیز کار می‌کند، که هیچ مقدار جستجویی نشان نمی‌دهد یکی بهتر از دیگری است.

این نوع استدلال دربارهٔ اینکه چه محاسباتی انجام شود «فرااستدلال» (Metareasoning) نامیده می‌شود (استدلال دربارهٔ استدلال). نه فقط بر بازی‌کردن، بلکه بر هر نوع استدلالی اعمال می‌شود. همهٔ محاسبات در خدمت تلاش برای رسیدن به تصمیم‌های بهتر انجام می‌شوند، همه هزینه دارند و همه احتمالِ رسیدن به بهبودی معین در کیفیت تصمیم را دارند. جستجوی مونت‌کارلو تلاش می‌کند فرااستدلال را برای تخصیص منابع به مهم‌ترین بخش‌های درخت انجام دهد، اما آن را به‌شیوهٔ بهینه انجام نمی‌دهد.

محدودیت سوم آن است که هر دو آلفا–بتا و مونت‌کارلو همهٔ استدلال‌شان را در سطح حرکت‌های فردی انجام می‌دهند. بدیهی است انسان‌ها متفاوت بازی می‌کنند: آن‌ها می‌توانند در سطح انتزاعی‌تری استدلال کنند، هدف سطح‌بالاتری را در نظر بگیرند — برای نمونه، به دام انداختن وزیر حریف — و از هدف برای تولید انتخابیِ برنامه‌های محتمل استفاده کنند. در فصل ۱۱ این نوع برنامه‌ریزی را مطالعه خواهیم کرد و در بخش ۱۱.۴ نشان خواهیم داد چگونه با سلسله‌مراتبی از بازنمایی‌های انتزاعی تا عینی برنامه‌ریزی کنیم.

مسئلهٔ چهارم، توانایی گنجاندن یادگیری ماشین در فرایند جستجوی بازی است. برنامه‌های اولیهٔ بازی به تخصص انسانی برای دست‌ساختن توابع ارزیابی، کتاب‌های گشایش، راهبردهای جستجو و حربه‌های کارایی تکیه می‌کردند. ما تازه داریم برنامه‌هایی مانند آلفازیرو (سیلور و همکاران، ۲۰۱۸) را می‌بینیم که به‌جای تخصص انسانیِ تولیدشدهٔ ویژهٔ بازی، بر یادگیری ماشین از بازی خودآموز تکیه کردند. یادگیری ماشین را با ژرفا از فصل ۱۹ آغاز می‌کنیم.

خلاصهٔ فصل

به گستره‌ای از بازی‌ها نگاه کردیم تا بفهمیم بازی بهینه به چه معناست، بفهمیم چگونه در عمل خوب بازی کنیم، و حسی به دست آوریم که عامل چگونه باید در هر نوع محیط رقابتی کنش کند. مهم‌ترین ایده‌ها چنین‌اند:

- بازی را می‌توان با حالت آغازین (تخته چگونه چیده می‌شود)، کنش‌های قانونی در هر حالت، نتیجهٔ هر کنش، آزمون پایانی (که می‌گوید بازی کی تمام می‌شود) و تابع مطلوبیتی که بر حالت‌های پایانی اعمال می‌شود تا بگوید چه کسی برد و امتیاز نهایی چیست، تعریف کرد.
- در بازی‌های دوبازیکنه، گسسته، جبرگرایانه، نوبتیِ جمع-صفر با اطلاعات کامل، الگوریتم مینیمکس می‌تواند حرکت‌های بهینه را با شمارش عمق-اول درخت بازی انتخاب کند.
- الگوریتم جستجوی آلفا–بتا همان حرکت بهینهٔ مینیمکس را محاسبه می‌کند، اما با حذف زیردرخت‌هایی که به‌شکل اثبات‌پذیر بی‌ربط‌اند کارایی بسیار بیشتری به دست می‌آورد.
- معمولاً در نظر گرفتن کل درخت بازی (حتی با آلفا–بتا) شدنی نیست، پس باید جستجو را در نقطه‌ای قطع کنیم و تابع ارزیابی اکتشافی‌ای اعمال کنیم که مطلوبیت حالت را تخمین می‌زند.
- بدیلی به نام جستجوی درختی مونت‌کارلو (MCTS) حالت‌ها را نه با اعمال تابع اکتشافی، بلکه با بازی‌کردن بازی تا پایان و استفاده از قواعد بازی برای دیدن اینکه چه کسی برد ارزیابی می‌کند. چون حرکت‌های انتخاب‌شده در طول بازی‌آزمایی ممکن است حرکت‌های بهینه نبوده باشند، فرایند چند بار تکرار می‌شود و ارزیابی میانگین نتایج است.
- بسیاری از برنامه‌های بازی جدول‌هایی از بهترین حرکت‌ها در گشایش و پایان‌بازی از پیش محاسبه می‌کنند تا بتوانند حرکت را در جدول نگاه کنند به‌جای جستجو.
- بازی‌های شانسی را می‌توان با اکسپکتیمینیمکس مدیریت کرد؛ بسطی بر الگوریتم مینیمکس که گرهٔ شانس را با گرفتن میانگین مطلوبیت همهٔ فرزندانش، وزن‌دهی‌شده با احتمال هر فرزند، ارزیابی می‌کند.
- در بازی‌های اطلاعات ناقص، مانند کریگ‌اشپیل و پوکر، بازی بهینه مستلزم استدلال دربارهٔ حالت‌های باور کنونی و آیندهٔ هر بازیکن است. تقریب ساده‌ای را می‌توان با میانگین‌گیری مقدار کنش روی هر پیکربندی ممکنِ اطلاعاتِ گم‌شده به دست آورد.
- برنامه‌ها قهرمانان شطرنج، چکرز، اوتلو، گو، پوکر و بسیاری بازی‌های دیگر را قاطعانه شکست داده‌اند. انسان‌ها در چند بازی اطلاعات ناقص، مانند بریج و کریگ‌اشپیل، برتری را حفظ کرده‌اند. در بازی‌های ویدئویی مانند استارکرافت و دوتا ۲، برنامه‌ها با کارشناسان انسانی رقابتی‌اند، اما بخشی از موفقیتشان ممکن است به‌سبب توانایی‌شان در انجام بسیار سریع کنش‌های فراوان باشد.

یادداشت‌های کتاب‌شناختی و تاریخی

در ۱۸۴۶، چارلز بابیج دربارهٔ امکان‌پذیری شطرنج و چکرز رایانه‌ای بحث کرد (موریسون و موریسون، ۱۹۶۱). او پیچیدگی نمایی درخت‌های جستجو را نفهمید و ادعا کرد «ترکیب‌های درگیر در موتور تحلیلی به‌شکل عظیمی فراتر از هر چیزی است که حتی بازی شطرنج لازم دارد.» بابیج همچنین ماشین ویژه‌منظوره‌ای برای بازی دوز طراحی کرد اما نساخت. نخستین ماشین بازی‌کننده حدود ۱۸۹۰ توسط مهندس اسپانیایی لئوناردو تورس ای کوودو ساخته شد. در پایان‌بازی شطرنج «KRK» (شاه و رخ در برابر شاه) تخصص داشت و برد را وقتی طرفِ دارای رخ نوبت دارد تضمین می‌کرد. الگوریتم مینیمکس به مقاله‌ای از ۱۹۱۲ از ارنست تسرملو، توسعه‌دهندهٔ نظریهٔ مجموعه‌های مدرن، بازمی‌گردد. بازی‌کردن یکی از نخستین تکلیف‌های انجام‌شده در هوش مصنوعی بود، با تلاش‌های آغازینِ پیشگامانی مانند کنراد زوزه (۱۹۴۵)، نوربرت وینر در کتابش «سایبرنتیک» (۱۹۴۸)، و آلن تورینگ (۱۹۵۳). اما مقالهٔ کلود شانون، «برنامه‌نویسی یک رایانه برای بازی شطرنج» (۱۹۵۰)، بود که همهٔ ایده‌های اصلی را طرح کرد: بازنمایی برای موضع‌های تخته، تابع ارزیابی، جستجوی سکون و ایده‌هایی برای جستجوی انتخابی درخت بازی. اسلیتر (۱۹۵۰) ایدهٔ تابع ارزیابی به‌عنوان ترکیب خطی ویژگی‌ها را داشت و بر ویژگی تحرک در شطرنج تأکید کرد. جان مک‌کارتی ایدهٔ جستجوی آلفا–بتا را در ۱۹۵۶ در سر پروراند، هرچند این ایده تا بعداً چاپ نشد (هارت و ادواردز، ۱۹۶۱). کنوت و مور (۱۹۷۵) درستی آلفا–بتا را اثبات و پیچیدگی زمانی‌اش را تحلیل کردند، در حالی که پیرل (۱۹۸۲) نشان داد آلفا–بتا در میان همهٔ الگوریتم‌های جستجوی درخت بازی با عمق ثابت مجانباً بهینه است.

برلینر (۱۹۷۹) B* را معرفی کرد؛ الگوریتم جستجوی اکتشافی که به‌جای تخمین نقطه‌ای تکی، کران‌های بازه‌ای بر مقدار ممکن گره در درخت بازی نگه می‌دارد. «جستجوی شمار توطئه» دیوید مک‌آلیستر (۱۹۸۸) گره‌های برگ را گسترش می‌دهد که با تغییر مقادیرشان، می‌توانند باعث ترجیح حرکت تازه‌ای در ریشهٔ درخت توسط برنامه شوند. MGSS* (راسل و وفالد، ۱۹۸۹) از فنون نظریه‌تصمیم فصل ۱۶ برای تخمین ارزش گسترش هر برگ بر حسب بهبود مورد انتظار در کیفیت تصمیم در ریشه استفاده می‌کند. الگوریتم SSS* (استاکمن، ۱۹۷۹) را می‌توان A* دوبازیکنه‌ای دید که هرگز گره‌های بیشتری از آلفا–بتا گسترش نمی‌دهد. نیازهای حافظه آن را غیرعملی می‌کند، اما نسخهٔ فضای-خطی از الگوریتم RBFS توسعه یافته (کورف و چیکرینگ، ۱۹۹۶). بام و اسمیت (۱۹۹۷) جایگزینی مبتنی بر احتمال برای مینیمکس پیشنهاد می‌کنند و نشان می‌دهند در برخی بازی‌ها به انتخاب‌های بهتر می‌انجامد. الگوریتم اکسپکتیمینیمکس توسط دونالد مایکی (۱۹۶۶) پیشنهاد شد. بروس بالارد (۱۹۸۳) هرس آلفا–بتا را برای پوشش درخت‌های با گره‌های شانس بسط داد.

کتاب «اکتشافی‌ها» پیرل (۱۹۸۴) بسیاری از الگوریتم‌های بازی‌کردن را به‌طور کامل تحلیل می‌کند. شبیه‌سازی مونت‌کارلو توسط متروپلیس و اولام (۱۹۴۹) برای محاسبات مرتبط با توسعهٔ بمب اتمی پیش‌گام شد. جستجوی درختی مونت‌کارلو (MCTS) توسط آبرامسون (۱۹۸۷) معرفی شد. تسائورو و گالپرین (۱۹۹۷) نشان دادند چگونه جستجوی مونت‌کارلو را با تابع ارزیابی برای بازی تخته‌نرد ترکیب کنند. خاتمهٔ زودهنگام بازی‌آزمایی توسط لورنتس (۲۰۱۵) مطالعه شده. آلفاگو بازی‌آزمایی‌ها را خاتمه می‌داد و تابع ارزیابی اعمال می‌کرد (سیلور و همکاران، ۲۰۱۶). کوسیچ و سپشواری (۲۰۰۶) رویکرد را با فرمول UCB1 پالایش کردند (Auers et al., 2002؛ کوسیچ و سپشواری، ۲۰۰۶). براون و همکاران (۲۰۱۲) نشان دادند UCT در برخی مسائل به‌صورت مجانبی به همگرایی می‌رسد اما در برخی دیگر نه.

جدول‌های پایان‌بازی شطرنج با جستجوی قهقرایی توسط بل (۱۹۶۵) مطالعه شد و با جدول‌های پنج‌مهره‌ای استیلر (۱۹۹۶) و تامپسون (۱۹۹۶) پیش رفت. استیلر موردی کشف کرد که در آن مات اجباری وجود داشت اما ۲۶۲ حرکت لازم بود؛ این باعث شگفتی شد، زیرا قواعد شطرنج ایجاب می‌کنند شکار یا حرکت سرباز در عرض ۵۰ حرکت رخ دهد وگرنه تساوی اعلام می‌شود. در ۲۰۱۲ ولادیمیر مخنیچف و ویکتور زاخاروف «جدول پایگاه پایان‌بازی لومونوسوف» را گردآوری کردند که همهٔ موضع‌های پایان‌بازی با حداکثر هفت مهره را حل کرد — برخی به بیش از ۵۰۰ حرکت بدون شکار نیاز دارند. جدول ۷-مهره ۱۴۰ ترابایت مصرف می‌کند؛ جدول ۸-مهره ۱۰۰ برابر بزرگ‌تر خواهد بود.

در ۲۰۱۷، آلفازیرو (سیلور و همکاران، ۲۰۱۸) استاک‌فیش (قهرمان شطرنج رایانه‌ای TCEC 2017) را در آزمونی ۱۰۰۰ بازی‌ای با ۱۵۵ برد و ۶ باخت شکست داد. دیدارهای اضافی نیز به بردهای قاطع آلفازیرو انجامید، حتی وقتی فقط 1/10 زمانی که به استاک‌فیش داده می‌شد به او داده می‌شد. استادبزرگ لری کافمن از موفقیت این برنامهٔ مونت‌کارلو شگفت‌زده شد و یادآور شد: «شاید سلطهٔ کنونی موتورهای مینیمکس شطرنج به پایان رسیده باشد، اما برای گفتنش زود است.» گری کاسپاروف نظر داد: «دستاوردی چشمگیر است، حتی اگر پس از آلفاگو باید انتظارش را می‌داشتیم. به رویکرد نوع ِB انسان‌مانندِ شطرنج ماشینی نزدیک می‌شود که کلود شانون و آلن تورینگ آرزویش را داشتند، به‌جای نیروی بی‌رحم.» او سپس پیش‌بینی کرد «شطرنج تا ریشه‌هایش توسط آلفازیرو لرزانده شده، اما این فقط نمونهٔ کوچکی از آنچه خواهد آمد است. رشته‌های پایبند به سنت مانند آموزش و پزشکی نیز لرزانده خواهند شد» (سادلر و ریگان، ۲۰۱۹).

چکرز نخستین بازی کلاسیک بود که رایانه انجامش داد (استراچی، ۱۹۵۲). آرتور ساموئل (۱۹۵۹، ۱۹۶۷) برنامهٔ چکرزی توسعه داد که تابع ارزیابی خودش را با بازی خودآموز با استفاده از نوعی یادگیری تقویتی می‌آموخت. دستاورد قابل توجهی است که ساموئل توانست برنامه‌ای بسازد که بهتر از خودش بازی می‌کرد، روی آی‌بی‌ام ۷۰۴ با فقط ۱۰,۰۰۰ واژه حافظه و پردازندهٔ 0.000001 گیگاهرتزی. MENACE — موتور ماشینی آموزش‌پذیر دوز (مایکی، ۱۹۶۳) — نیز از یادگیری تقویتی برای رسیدن به شایستگی در دوز استفاده کرد. پردازنده‌اش حتی کندتر بود: مجموعه‌ای از ۳۰۴ جعبه کبریت حاوی مهره‌های رنگی برای بازنمایی بهترین حرکت آموخته‌شده در هر موضع.

در ۱۹۹۲، برنامهٔ چکرز چینوک (CHINOOK) جاناتان شفر، ماریون تینزلی افسانه‌ای را به چالش کشید که بیش از ۲۰ سال قهرمان جهان بود. تینزلی دیدار را برد، اما دو بازی باخت — چهارمین و پنجمین باخت کل دوران حرفه‌ای‌اش. پس از بازنشستگی تینزلی به دلایل سلامتی، چینوک تاج را گرفت. این حماسه توسط شفر (۲۰۰۸) ثبت شد. در ۲۰۰۷ شفر و تیمش چکرز را «حل» کردند (شفر و همکاران، ۲۰۰۷): بازی با بازی کامل تساوی است. ریچارد بلمن (۱۹۶۵) این را پیش‌بینی کرده بود: «در چکرز، شمار حرکت‌های ممکن در هر موقعیتِ داده‌شده چنان کوچک است که می‌توانیم با اطمینان انتظار راه‌حل کامل دیجیتالی رایانه‌ای برای مسئلهٔ بازی بهینه در این بازی را داشته باشیم.» بلمن مقیاس تلاش را پیش‌بینی نکرده بود: جدول پایان‌بازی برای ۱۰ مهره ۳۹ تریلیون مدخل دارد. با این جدول، ۱۸ سال CPU جستجوی آلفا–بتا برای حل بازی لازم بود.

آی. جی. گود، که بازی گو را آلن تورینگ به او آموخته بود، نوشت (۱۹۶۵): «فکر می‌کنم برنامه‌نویسی رایانه‌ای برای بازی معقول گو حتی از شطرنج دشوارتر خواهد بود.» حق با او بود: تا ۲۰۱۵، برنامه‌های گو فقط در سطح آماتوری بازی می‌کردند. ادبیات اولیه توسط بوزی و کازناو (۲۰۰۱) و مولر (۲۰۰۲) خلاصه شده. تشخیص الگوی بصری به‌عنوان فن امیدوارکننده برای گو توسط زوبریست (۱۹۷۰) پیشنهاد شد، در حالی که شرودولف و همکاران (۱۹۹۴) استفاده از

یادگیری تقویتی را تحلیل کردند، لوببرتس و میکولاینن (۲۰۰۱) شبکه‌های عصبی را توصیه کردند، و بروگمن (۱۹۹۳) جستجوی درختی مونت‌کارلو را به گو معرفی کرد. آلفاگو (سیلور و همکاران، ۲۰۱۶) آن چهار ایده را کنار هم گذاشت تا حرفه‌ای‌های رده‌بالا لی سدول (با نتیجهٔ 4–1 در ۲۰۱۵) و که جی (با 3–0 در ۲۰۱۶) را شکست دهد. که جی گفت: «پس از آنکه بشریت هزاران سال تاکتیک‌هایش را بهبود داد، رایانه‌ها به ما می‌گویند انسان‌ها کاملاً اشتباه می‌کنند. تا آنجا پیش می‌روم که بگویم هیچ انسان تکی به لبهٔ حقیقت گو دست نزده است.» لی سدول از گو بازنشسته شد و lament کرد: «حتی اگر شمارهٔ یک می‌شدم، موجودیتی هست که نمی‌توان شکستش داد.» در ۲۰۱۸، آلفازیرو در گو از آلفاگو فراتر رفت و نیز برنامه‌های برتر شطرنج و شوگی را شکست داد، با یادگیری از بازی خودآموز بدون هیچ دانش تخصصی انسانی و بدون دسترسی به بازی‌های گذشته. (البته به انسان‌ها برای تعریف معماری پایه به‌عنوان جستجوی درختی مونت‌کارلو با شبکه‌های عصبی عمیق و یادگیری تقویتی و رمزگذاری قواعد بازی تکیه می‌کند.) موفقیت آلفازیرو به افزایش علاقه به یادگیری تقویتی به‌عنوان مؤلفهٔ کلیدی هوش مصنوعی عمومی انجامیده (فصل ۲۲ را ببینید). یک گام فراتر، سامانهٔ موزیرو (MUZERO) حتی بدون گفته‌شدنِ قواعد بازی‌ای که بازی می‌کند عمل می‌کند — باید قواعد را با انجام بازی‌ها دریابد. موزیرو در پک‌من، شطرنج، گو و ۷۵ بازی آتاری نتایج پیشرفته به دست آورد (شریت‌ویزر و همکاران، ۲۰۱۹). یاد می‌گیرد تعمیم دهد؛ برای نمونه، یاد می‌گیرد در پک‌من کنش «بالا» بازیکن را یک خانه بالا می‌برد (مگر آنکه دیواری آن‌جا باشد)، حتی با آنکه نتیجهٔ کنش «بالا» را فقط در درصد کوچکی از مکان‌های تخته مشاهده کرده.

اوتلو، که ریورسی نیز نامیده می‌شود، فضای جستجوی کوچک‌تری از شطرنج دارد، اما تعریف تابع ارزیابی دشوار است، زیرا برتری مادی به اندازهٔ تحرک مهم نیست. برنامه‌ها از ۱۹۹۷ در سطح ابرانسانی بوده‌اند (بورو، ۲۰۰۲).

تخته‌نرد، بازی شانسی، به‌صورت ریاضی توسط جرولامو کاردانو (۱۶۶۳) تحلیل شد و با برنامهٔ BKG برای بازی رایانه‌ای گرفته شد (برلینر، ۱۹۸۰) که از تابع ارزیابی دست‌ساخته استفاده می‌کرد و فقط تا عمق ۱ جستجو می‌کرد. نخستین برنامه‌ای بود که قهرمان جهانِ انسانی را در بازی مهمی شکست داد (برلینر، ۱۹۸۰)، هرچند برلینر صادقانه اذعان کرد BKG با تاس‌ها بسیار خوش‌شانس بود. تیدی-گمن (TD-GAMMON) جری تسائورو (۱۹۹۵) تابع ارزیابی‌اش را با شبکه‌های عصبیِ آموزش‌دیده با بازی خودآموز آموخت. پیوسته در سطح قهرمان جهان بازی می‌کرد و تحلیلگران انسانی را واداشت نظرشان را دربارهٔ بهترین حرکت گشایشی برای چند تاس تغییر دهند.

پوکر، مانند گو، پیشرفت‌های شگفت‌آوری در سال‌های اخیر دیده است. بولینگ و همکاران (۲۰۱۵) از نظریهٔ بازی (بخش ۱۸.۲ را ببینید) برای تعیین راهبرد دقیق بهینه برای نسخه‌ای از پوکر با فقط دو بازیکن و شمار ثابتی افزایش شرط با اندازه‌های شرط ثابت استفاده کردند. در ۲۰۱۷، برای نخستین بار، قهرمانان پوکر در پوکر تگزاس هولدم بدون-محدودیتِ رو-در-رو (دونفره) در دو دیدار جداگانه در برابر برنامه‌های لیبراتوس (براون و سندهولم، ۲۰۱۷) و دیپ‌استک (موراوچیک و همکاران، ۲۰۱۷) شکست خوردند. در ۲۰۱۹، پلوریبوس (براون و سندهولم، ۲۰۱۹) بازیکنان حرفه‌ای انسانی رده‌بالا را در بازی‌های تگزاس هولدم با شش بازیکن شکست داد. بازی‌های چندبازیکنه نگرانی‌های راهبردی‌ای معرفی می‌کنند که در فصل ۱۸ پوشش می‌دهیم. پتوسا و بالچ (۲۰۱۹) نسخهٔ چندبازیکنهٔ آلفازیرو پیاده می‌کنند.

بریج: اسمیت و همکاران (۱۹۹۸) گزارش می‌دهند چگونه بریج بارون قهرمانی بریج رایانه‌ای ۱۹۹۸ را برد، با استفاده از برنامه‌های سلسله‌مراتبی (فصل ۱۱ را ببینید) و کنش‌های سطح‌بالا مانند فینس و فشردن که برای بازیکنان بریج آشناست. گینزبرگ (۲۰۰۱) توصیف می‌کند چگونه برنامهٔ GIB او، مبتنی بر شبیه‌سازی مونت‌کارلو (که نخستین بار توسط لوی (۱۹۸۹) برای بریج پیشنهاد شد)، قهرمانی رایانه‌ای بعدی را برد و در برابر بازیکنان متخصص انسانی به‌شکل

شگفت‌آوری خوب عمل کرد. در قرن بیست‌ویکم، قهرمانی بریج رایانه‌ای توسط دو برنامهٔ تجاری، JACK و WBRIDGE5، مسلط شده. هیچ‌کدام در مقاله‌های منتشرشده توصیف نشده‌اند، اما باور می‌رود هر دو از فنون مونت‌کارلو استفاده می‌کنند. در حالت کلی، برنامه‌های بریج هنگام بازیِ واقعیِ دست‌ها در سطح قهرمان انسانی‌اند، اما در فاز مناقصه عقب‌اند، زیرا قراردادهای مورد استفاده انسان‌ها برای ارتباط با هم‌دسته‌هایشان را کامل نمی‌فهمند. برنامه‌نویسان بریج بیشتر بر تولید برنامه‌های مفید و آموزشی تمرکز کرده‌اند که مردم را به روی‌آوردن به بازی تشویق می‌کنند، نه بر شکست‌دادن قهرمانان انسانی.

اسکربل بازی‌ای است که بازیکنان آماتور انسانی در یافتن واژه‌های پرامتیاز دشواری دارند، اما برای رایانه، یافتن بیشترین امتیاز ممکن برای دستِ داده‌شده آسان است (گوردون، ۱۹۹۴)؛ بخش دشوار، برنامه‌ریزی به جلو در بازیِ تصادفیِ تا حدی قابل مشاهده است. با این حال، در ۲۰۰۶، برنامهٔ کوکل (QUACKLE) قهرمان سابق جهان، دیوید بویز، را 3–2 شکست داد. بویز خوب پذیرفت و گفت: «هنوز انسان بودن بهتر از رایانه بودن است.» توصیف خوبی از برنامهٔ برتر، MAVEN، توسط شپارد (۲۰۰۲) داده شده.

بازی‌های ویدئویی مانند استارکرافت ۲ شامل صدها واحد تا حدی قابل مشاهده‌اند که در زمان واقعی با فضاهای مشاهده و کنشِ پرابعادِ تقریباً پیوسته[6] و قواعد پیچیده حرکت می‌کنند. اوریول وینیالز، که در ۱۵ سالگی قهرمان استارکرافت اسپانیا بود، توصیف کرد بازی چگونه می‌تواند به‌عنوان بستر آزمون و چالش بزرگ برای یادگیری تقویتی خدمت کند (وینیالز و همکاران، ۲۰۱۷). در ۲۰۱۹، وینیالز و تیم دیپ‌مایند برنامهٔ آلفاستار را رونمایی کردند، مبتنی بر یادگیری عمیق و یادگیری تقویتی، که بازیکنان متخصص انسانی را ۱۰ بر ۱ شکست داد و در میان ۰٫۰۲٪ برتر بازیکنان انسانیِ رسماً رتبه‌بندی‌شده قرار گرفت (وینیالز و همکاران، ۲۰۱۹). آلفاستار گام‌هایی برای محدودکردن شمار کنش‌های در دقیقه‌ای که می‌توانست در جهش‌های بحرانی انجام دهد برداشت، در پاسخ به منتقدانی که احساس می‌کردند برتری ناعادلانه‌ای دارد.

رایانه‌ها انسان‌های برتر را در دیگر بازی‌های ویدئویی محبوب مانند سوپر اسمش برادرز (فیرویو و همکاران، ۲۰۱۷)، کوئیک ۳ (جادربرگ و همکاران، ۲۰۱۹) و دوتا ۲ (فرناندز و مالمان، ۲۰۱۸) شکست داده‌اند، که همگی از فنون یادگیری عمیق استفاده می‌کنند.

بازی‌های فیزیکی مانند فوتبال رباتیک (ویسسر و همکاران، ۲۰۰۸؛ بارت و استون، ۲۰۱۵)، بیلیارد (لام و گرین‌سپن، ۲۰۰۸؛ آرچیبالد و همکاران، ۲۰۰۹) و پینگ‌پنگ (سیلوا و همکاران، ۲۰۱۵) توجهی در هوش مصنوعی جلب کرده‌اند. آن‌ها همهٔ پیچیدگی‌های بازی‌های ویدئویی را با درهم‌ریختگی جهان واقعی ترکیب می‌کنند.

پانوشت‌ها:

[1] برخی نویسندگان تمایز قائل می‌شوند: «بازی اطلاعات ناقص» را برای بازی‌ای مانند پوکر به کار می‌برند که بازیکنان اطلاعات خصوصی دربارهٔ دست‌های خودشان دارند که دیگر بازیکنان ندارند، و «بازی تا حدی قابل مشاهده» را برای بازی‌ای مانند استارکرافت ۲ به معنای اینکه هر بازیکن محیط نزدیک را می‌بیند اما محیط دور را نه.

[2] شطرنج بازی «جمع-صفر» دانسته می‌شود، حتی با آنکه مجموع پیامدهای دو بازیکن برای هر بازی 1+ است نه صفر. «جمع-ثابت» اصطلاح دقیق‌تری می‌بود، اما جمع-صفر سنتی است و اگر تصور کنید از هر بازیکن ورودی‌ای 1/2 گرفته می‌شود معنا می‌دهد.

[3] «مونت‌کارلو» در این‌جا به الگوریتم‌هایی اشاره دارد که از اعداد تصادفی استفاده می‌کنند.

[4] گاهی حالت باور آن‌قدر بزرگ می‌شود که نتوان آن را فقط به‌صورت فهرستی از حالت‌های تخته بازنمایی کرد، اما فعلاً این مسئله را نادیده می‌گیریم؛ فصل‌های ۷ و ۸ روش‌هایی برای بازنمایی فشردهٔ حالت‌های باور بسیار بزرگ پیشنهاد می‌کنند.

[5] بلوف‌زدن — شرط‌بستن چنان‌که گویی دستت خوب است، حتی وقتی نیست — بخش هسته‌ای راهبرد پوکر است.

[6] فضای مشاهدهٔ تقریباً پیوسته، فضایی است که در آن برخی از ابعاد پیوسته‌اند.

فصل ۶: مسئله‌های ارضای قیود (Constraint Satisfaction Problems)

در این فصل می‌بینیم چگونه در نظر گرفتن حالت‌ها به‌عنوان چیزی بیش از جعبه‌های سیاه کوچک، به روش‌های جستجوی تازه و فهم عمیق‌تر از ساختار مسئله می‌انجامد.

فصل‌های ۳ و ۴ این ایده را کاویدند که مسائل را می‌توان با جستجو در فضای حالت حل کرد: گرافی که گره‌هایش حالت‌ها و یال‌های میانشان کنش‌هایند. دیدیم اکتشافی‌های حوزه‌ویژه می‌توانند هزینهٔ رسیدن به هدف از حالتِ داده‌شده را تخمین بزنند، اما از دیدگاه الگوریتم جستجو، هر حالت اتمی یا تقسیم‌ناپذیر است — جعبهٔ سیاهی بدون ساختار درونی. برای هر مسئله به کدِ حوزه‌ویژه برای توصیف گذارهای میان حالت‌ها نیاز داریم.

در این فصل جعبهٔ سیاه را می‌شکنیم و برای هر حالت از بازنمایی عاملی (Factored Representation) استفاده می‌کنیم: مجموعه‌ای از متغیرها که هر یک مقداری دارد. مسئله وقتی حل می‌شود که هر متغیر مقداری داشته باشد که همهٔ قیدهای (Constraints) آن متغیر را برآورده کند. مسئله‌ای که چنین توصیف شود، «مسئلهٔ ارضای قیود» یا CSP نامیده می‌شود.

الگوریتم‌های جستجوی CSP از ساختار حالت‌ها بهره می‌گیرند و برای امکان‌پذیر کردن حل مسائل پیچیده، به‌جای اکتشافی‌های حوزه‌ویژه از اکتشافی‌های عمومی استفاده می‌کنند. ایدهٔ اصلی، حذف یک‌بارهٔ بخش‌های بزرگ فضای جستجو با شناسایی ترکیب‌های متغیر/مقداری است که قیدها را نقض می‌کنند. CSPها مزیت اضافی نیز دارند: کنش‌ها و مدل گذار را می‌توان از توصیف مسئله استنتاج کرد.

۶.۱ تعریف مسئله‌های ارضای قیود

مسئلهٔ ارضای قیود از سه مؤلفه تشکیل شده: X، D و C:

- **X** مجموعه‌ای از متغیرهاست: {X1, ..., Xn}.
- **D** مجموعه‌ای از دامنه‌هاست: {D1, ..., Dn}، برای هر متغیر یکی.
- **C** مجموعه‌ای از قیدهاست که ترکیب‌های مجاز مقادیر را مشخص می‌کنند.

دامنهٔ Di از مجموعه‌ای از مقادیر مجاز، {v1, ..., vk}، برای متغیر Xi تشکیل شده. برای نمونه، متغیر بولی دامنهٔ {true, false} دارد. متغیرهای متفاوت می‌توانند دامنه‌های متفاوت با اندازه‌های متفاوت داشته باشند. هر قید Cj از جفت ⟨scope, rel⟩ تشکیل شده که scope چندتاییِ (Tuple) متغیرهای شرکت‌کننده در قید و rel «رابطه»ای (Relation) است که مقادیری را تعریف می‌کند آن متغیرها می‌توانند بگیرند. رابطه را می‌توان به‌صورت مجموعهٔ صریحی از همهٔ

چندتایی‌های مقادیری که قید را برآورده می‌کنند بازنمایی کرد، یا به‌صورت تابعی که می‌تواند محاسبه کند آیا چندتایی‌ای عضو رابطه است یا نه. برای نمونه، اگر X1 و X2 هر دو دامنهٔ {1,2,3} داشته باشند، قیدی که می‌گوید X1 باید از X2 بزرگ‌تر باشد را می‌توان چنین نوشت: ⟨(X1,X2), {(3,1),(3,2),(2,1)}⟩ یا ⟨(X1,X2), X1 > X2⟩.

CSPها با «تخصیص»هایی (Assignments) سروکار دارند: {Xi = vi, Xj = vj, ...}. تخصیصی که هیچ قیدی را نقض نکند، تخصیص سازگار (Consistent) یا قانونی نامیده می‌شود. تخصیص کامل (Complete Assignment) تخصیصی است که در آن به هر متغیر مقداری تخصیص داده شده، و «راه‌حل» (Solution) یک CSP، تخصیص سازگار و کاملی است. تخصیص جزئی (Partial Assignment) تخصیصی است که برخی متغیرها را بی‌تخصیص می‌گذارد، و راه‌حل جزئی (Partial Solution) تخصیص جزئی سازگاری است. حل یک CSP در حالت کلی مسئله‌ای NP-کامل است، هرچند زیررده‌های مهمی از CSPها هست که می‌توان آن‌ها را بسیار کارآمد حل کرد.

۶.۱.۱ مسئلهٔ نمونه: رنگ‌آمیزی نقشه

فرض کنیم از رومانی خسته شده‌ایم و به نقشهٔ استرالیا نگاه می‌کنیم که هر یک از ایالت‌ها و قلمروهایش را نشان می‌دهد (شکل ۶.۱(الف)). تکلیف ما رنگ‌کردن هر ناحیه با قرمز، سبز یا آبی است، چنان‌که هیچ دو ناحیهٔ همسایه رنگ یکسانی نداشته باشند. برای صورت‌بندی این به‌صورت CSP، متغیرها را ناحیه‌ها تعریف می‌کنیم:

```
X = {WA, NT, Q, NSW, V, SA, T}
```

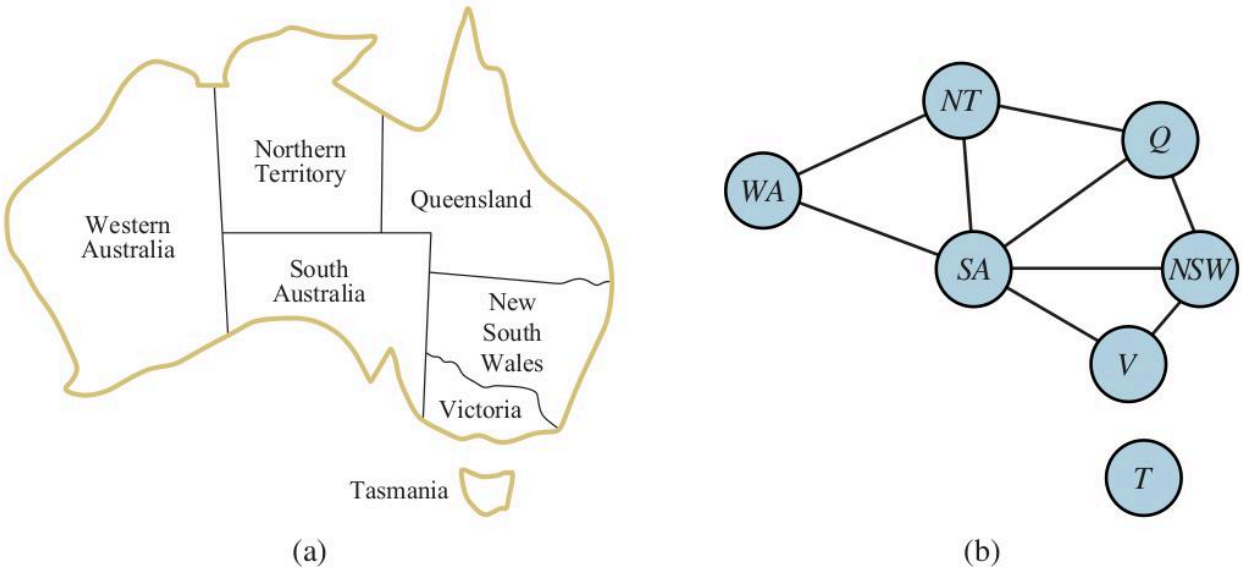
دامنهٔ هر متغیر مجموعهٔ Di = {red, green, blue} است. قیدها ایجاب می‌کنند ناحیه‌های همسایه رنگ‌های متمایز داشته باشند. چون نه محل مرز میان ناحیه‌ها هست، نه قید وجود دارد:

```
C = {SA≠WA, SA≠NT, SA≠Q, SA≠NSW, SA≠V, WA≠NT, NT≠Q, Q≠NSW, NSW≠V}
```

در این‌جا از اختصارها استفاده می‌کنیم؛ SA≠WA میان‌بری است برای ⟨(SA,WA), SA≠WA⟩، که به‌نوبهٔ خود SA≠WA را می‌توان کامل چنین برشمرد: {(red,green), (red,blue), (green,red), (green,blue), (blue,red), (blue,green)}.

راه‌حل‌های بسیاری برای این مسئله هست، مانند {WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=red}.

تجسم یک CSP به‌صورت «گراف قید» (Constraint Graph) کمک‌کننده است، چنان‌که در شکل ۶.۱(ب). گره‌های گراف با متغیرهای مسئله متناظرند و یالی هر دو متغیری را که در قیدی شرکت می‌کنند به هم متصل می‌کند.



شکل ۶.۱ – (الف) ایالت‌ها و قلمروهای اصلی استرالیا. رنگ‌آمیزی این نقشه را می‌توان مسئلهٔ ارضای قیود (CSP) دید. هدف تخصیص رنگ به هر ناحیه است چنان‌که هیچ دو ناحیهٔ همسایه رنگ یکسانی نداشته باشند. (ب) مسئلهٔ رنگ‌آمیزی نقشه به‌صورت گراف قید بازنمایی شده.

چرا مسئله‌ای را به‌صورت CSP صورت‌بندی کنیم؟ یک دلیل آن است که CSPها بازنمایی طبیعی برای گسترهٔ وسیعی از مسائل به دست می‌دهند؛ اغلب صورت‌بندی مسئله به‌صورت CSP آسان است. دلیل دیگر آن است که سال‌ها کار توسعه صرف سریع و کارآمد کردن حل‌کننده‌های CSP شده است. دلیل سوم آن است که حل‌کنندهٔ CSP می‌تواند به‌سرعت بخش‌های بزرگِ فضای جستجو را هرس کند که جستجوگر فضای حالت اتمی نمی‌تواند. برای نمونه، پس از آنکه در مسئلهٔ استرالیا {SA=blue} را برگزیدیم، می‌توانیم نتیجه بگیریم هیچ‌یک از پنج متغیر همسایه نمی‌تواند مقدار blue بگیرد. رویهٔ جستجویی که از قیدها استفاده نمی‌کند باید $3^5 = ۲۴۳$ تخصیص را برای پنج متغیر همسایه در نظر بگیرد؛ با قیدها فقط $2^5 = ۳۲$ تخصیص برای در نظر گرفتن داریم، کاهشی ۸۷٪.

در جستجوی فضای حالت اتمی فقط می‌توانیم بپرسیم: آیا این حالت خاص هدف است؟ نه؟ این یکی چطور؟ با CSPها، به‌محض آنکه دریابیم تخصیص جزئی‌ای قیدی را نقض می‌کند، می‌توانیم بلافاصله پالایش‌های بیشترِ آن تخصیص جزئی را کنار بگذاریم. وانگهی، می‌توانیم ببینیم چرا تخصیص راه‌حل نیست — می‌بینیم کدام متغیرها قیدی را نقض می‌کنند — پس می‌توانیم توجه را بر متغیرهای مهم متمرکز کنیم. در نتیجه، بسیاری از مسائلی که برای جستجوی فضای حالت اتمی ناشدنی‌اند، وقتی به‌صورت CSP صورت‌بندی شوند به‌سرعت حل می‌شوند.

۶.۱.۲ مسئلهٔ نمونه: زمان‌بندی کارگاه

کارخانه‌ها مسئلهٔ زمان‌بندی یک روز کاریِ کارها را دارند، با توجه به قیدهای گوناگون. در عمل، بسیاری از این مسائل با فنون CSP حل می‌شوند. مسئلهٔ زمان‌بندی مونتاژ خودرو را در نظر بگیرید. کل کار از تکلیف‌هایی (Tasks) تشکیل شده و می‌توانیم هر تکلیف را به‌صورت متغیری مدل کنیم که مقدار هر متغیر، زمان شروع آن تکلیف است و به‌صورت عدد صحیحی از دقیقه بیان می‌شود. قیدها می‌توانند اعلام کنند تکلیفی باید پیش از تکلیف دیگر رخ دهد — برای نمونه، چرخ باید پیش از گذاشتن قالپاق نصب شود — و اینکه فقط تعداد معینی تکلیف می‌تواند هم‌زمان ادامه یابد. قیدها

همچنین می‌توانند مشخص کنند تکلیفی زمان معینی برای تکمیل می‌برد. بخش کوچکی از مونتاژ خودرو را در نظر می‌گیریم که از ۱۵ تکلیف تشکیل شده: نصب اکسل‌ها (جلو و عقب)، نصب هر چهار چرخ (راست و چپ، جلو و عقب)، سفت‌کردن مهره‌های هر چرخ، نصب قالپاق‌ها و بازرسی مونتاژ نهایی. می‌توانیم تکلیف‌ها را با ۱۵ متغیر بازنمایی کنیم:

```
X = {AxleF, AxleB, WheelRF, WheelLF, WheelRB, WheelLB, NutsRF, NutsLF, NutsRB, NutsLB,
                                                CapRF, CapLF, CapRB, CapLB, Inspect}
```

سپس «قیدهای تقدم» (Precedence Constraints) میان تکلیف‌های فردی را بازنمایی می‌کنیم. هرگاه تکلیفی T1 باید پیش از تکلیف T2 رخ دهد و تکلیف T1 مدت d1 برای تکمیل می‌برد، قیدی حسابی به شکل T1 + d1 ≤ T2 می‌افزاییم. در مثال ما، اکسل‌ها باید پیش از نصب چرخ‌ها سر جایشان باشند و نصب اکسل ۱۰ دقیقه می‌برد، پس می‌نویسیم:

```
AxleF + 10 ≤ WheelRF; AxleF + 10 ≤ WheelLF; AxleB + 10 ≤ WheelRB; AxleB + 10 ≤ WheelLB
```

سپس می‌گوییم برای هر چرخ، باید چرخ را نصب کنیم (۱ دقیقه می‌برد)، سپس مهره‌ها را سفت کنیم (۲ دقیقه) و سرانجام قالپاق را بچسبانیم (۱ دقیقه، اما هنوز بازنمایی نشده):

```
WheelRF + 1 ≤ NutsRF; NutsRF + 2 ≤ CapRF
 WheelLF + 1 ≤ NutsLF; NutsLF + 2 ≤ CapLF
 WheelRB + 1 ≤ NutsRB; NutsRB + 2 ≤ CapRB
 WheelLB + 1 ≤ NutsLB; NutsLB + 2 ≤ CapLB
```

فرض کنیم چهار کارگر برای نصب چرخ‌ها داریم، اما باید یک ابزار را شریک شوند که به نصب اکسل کمک می‌کند. به قیدی فصلی (Disjunctive Constraint) نیاز داریم که بگوید AxleF و AxleB نباید در زمان هم‌پوشانی کنند؛ یا یکی اول می‌آید یا دیگری:

```
or (AxleB + 10 ≤ AxleF) (AxleF + 10 ≤ AxleB)
```

این قید پیچیده‌تر به نظر می‌رسد، ترکیبی از حساب و منطق. اما همچنان به مجموعه‌ای از جفت‌مقادیری کاهش می‌یابد که AxleF و AxleB می‌توانند بگیرند.

همچنین باید اعلام کنیم بازرسی آخر می‌آید و ۳ دقیقه می‌برد. برای هر متغیر به‌جز Inspect قیدی به شکل X + dX ≤ Inspect می‌افزاییم. سرانجام، فرض کنیم الزامی هست کل مونتاژ در ۳۰ دقیقه تمام شود. می‌توانیم با محدودکردن دامنهٔ همهٔ متغیرها به آن برسیم:

```
Di = {0, 1, 2, 3, ..., 30}
```

این مسئلهٔ خاص حل‌شدنش بدیهی است، اما CSPها با موفقیت بر مسائل زمان‌بندی کارگاه مانند این با هزاران متغیر اعمال شده‌اند.

۶.۱.۳ تغییرهای صورت‌بندی CSP

ساده‌ترین نوع CSP شامل متغیرهایی با دامنه‌های گسستهٔ متناهی (Discrete/Finite Domains) است. مسائل رنگ‌آمیزی نقشه و زمان‌بندی با محدودیت زمانی هر دو از این نوع‌اند. مسئلهٔ هشت‌وزیر (شکل 4.3) را نیز می‌توان CSP با دامنهٔ متناهی دید، که در آن متغیرهای Q1,...,Q8 با وزیرهای ستون‌های ۱ تا ۸ متناظرند و دامنهٔ هر متغیر شمار سطرهای ممکن برای وزیر آن ستون را مشخص می‌کند: Di = {1,2,3,4,5,6,7,8}. قیدها می‌گویند هیچ دو وزیری نمی‌توانند در یک سطر یا قطر باشند.

دامنهٔ گسسته می‌تواند نامتناهی (Infinite) باشد، مانند مجموعهٔ اعداد صحیح یا رشته‌ها. (اگر برای مسئلهٔ زمان‌بندی کار مهلتی نمی‌گذاشتیم، برای هر متغیر تعداد بی‌نهایتی زمان شروع وجود داشت.) با دامنه‌های نامتناهی، باید از قیدهای ضمنی مانند T1 + d1 ≤ T2 استفاده کنیم نه چندتایی‌های صریح مقادیر. الگوریتم‌های حل خاصی (که این‌جا بحث نمی‌کنیم) برای «قیدهای خطی» (Linear Constraints) روی متغیرهای صحیح وجود دارد — یعنی قیدهایی مانند آنچه داده شد که در آن هر متغیر فقط به‌صورت خطی ظاهر می‌شود. می‌توان نشان داد هیچ الگوریتمی برای حل قیدهای غیرخطی (Nonlinear Constraints) عمومی روی متغیرهای صحیح وجود ندارد — مسئله تصمیم‌ناپذیر است.

مسئله‌های ارضای قیود با دامنه‌های پیوسته (Continuous Domains) در جهان واقعی رایج‌اند و در رشتهٔ تحقیق در عملیات به‌طور گسترده مطالعه می‌شوند. برای نمونه، زمان‌بندی آزمایش‌ها روی تلسکوپ فضایی هابل به زمان‌بندی بسیار دقیق مشاهدات نیاز دارد؛ شروع و پایان هر مشاهده و مانور متغیرهایی با مقدار پیوسته‌اند که باید از گستره‌ای از قیدهای نجومی، تقدم و توان پیروی کنند. شناخته‌شده‌ترین ردهٔ CSPهای با دامنهٔ پیوسته، مسائل برنامه‌ریزی خطی است که در آن قیدها باید برابری‌ها یا نابرابری‌های خطی باشند. مسائل برنامه‌ریزی خطی را می‌توان در زمانی چندجمله‌ای نسبت به شمار متغیرها حل کرد. مسائل با انواع مختلف قید و تابع هدف نیز مطالعه شده‌اند — برنامه‌ریزی درجه‌دوم، برنامه‌ریزی مخروطی مرتبهٔ دوم و الی آخر. این مسائل حوزهٔ مهمی از ریاضیات کاربردی را تشکیل می‌دهند.

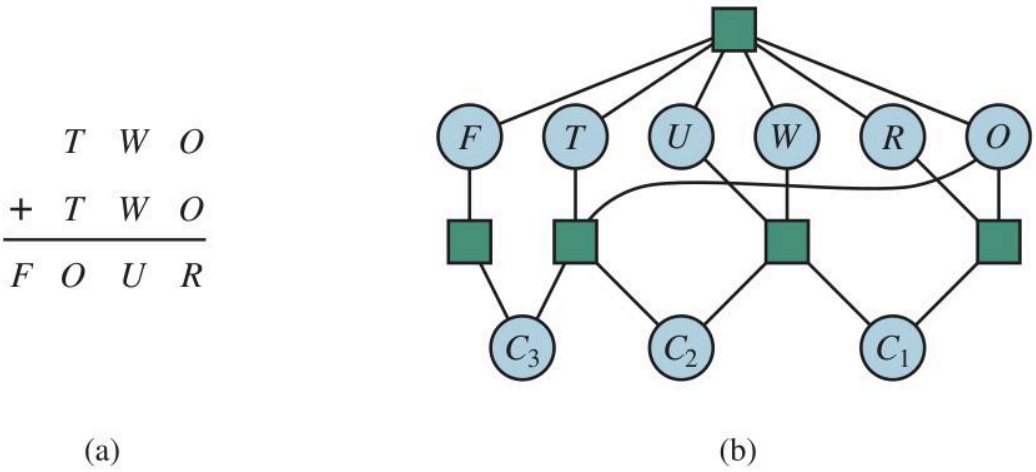
افزون بر بررسی انواع متغیرهایی که می‌توانند در CSPها ظاهر شوند، بررسی انواع قیدها نیز مفید است. ساده‌ترین نوع، «قید یک‌تایی» (Unary Constraint) است که مقدار یک متغیر تکی را محدود می‌کند. برای نمونه، در مسئلهٔ رنگ‌آمیزی نقشه ممکن است چنین باشد که استرالیایی‌های جنوبی رنگ سبز را تحمل نمی‌کنند؛ می‌توانیم آن را با قید یک‌تایی ⟨(SA), SA≠green⟩ بیان کنیم. (مشخصات آغازین دامنهٔ یک متغیر را نیز می‌توان قید یک‌تایی دید.) «قید دوتایی» (Binary Constraint) دو متغیر را به هم مربوط می‌کند. برای نمونه، SA≠NSW قید دوتایی است. «CSP دوتایی» (Binary CSP) CSPی است با فقط قیدهای یک‌تایی و دوتایی؛ می‌توان آن را به‌صورت گراف قید، مانند شکل ۶.۱(ب)، بازنمایی کرد.

همچنین می‌توانیم «قیدهای مرتبهٔ بالاتر» (Higher-Order Constraints) تعریف کنیم. قید سه‌تایی Between(X,Y,Z)، برای نمونه، چنین تعریف می‌شود: ⟨(X,Y,Z), X < Y < Z or X > Y > Z⟩.

قیدی که شامل تعداد دلخواهی متغیر است «قید سراسری» (Global Constraint) نامیده می‌شود. (این نام سنتی اما گیج‌کننده است، زیرا قید سراسری لازم نیست همهٔ متغیرهای مسئله را در بر بگیرد.) یکی از رایج‌ترین قیدهای سراسری Alldiff است که می‌گوید همهٔ متغیرهای درگیر در قید باید مقادیر متفاوتی داشته باشند. در مسائل سودوکو (بخش ۶.۲.۶ را ببینید)، همهٔ متغیرهای یک سطر، ستون یا جعبهٔ 3×3 باید قید Alldiff را برآورده کنند. مثال دیگر، معماهای محاسبات‌نویسی (Cryptarithmetic) است (شکل ۶.۲(الف)). هر حرف در معمای محاسبات‌نویسی رقم متفاوتی را بازنمایی می‌کند. برای مورد شکل ۶.۲(الف)، این به‌صورت قید سراسری Alldiff(F,T,U,W,R,O) بازنمایی می‌شود. قیدهای جمع روی چهار ستون معما را می‌توان به‌صورت قیدهای n-تایی زیر نوشت:

```
O + O = R + 10·C1
C1 + W + W = U + 10·C2
C2 + T + T = O + 10·C3
C3 = F
```

که C1، C2 و C3 متغیرهای کمکی‌اند که رقم نقلی به ستون دهگان، صدگان یا هزارگان را بازنمایی می‌کنند. این قیدها را می‌توان در «ابرگراف قید» (Constraint Hypergraph) بازنمایی کرد، مانند آنچه در شکل ۶.۲(ب) نشان داده شده. ابرگراف از گره‌های معمولی (دایره‌ها در شکل) و «ابرگره»ها (مربع‌ها) تشکیل شده که قیدهای n-تایی – قیدهای درگیر با n متغیر – را بازنمایی می‌کنند.



شکل ۶.۲ – (الف) مسئلهٔ محاسبات‌نویسی. هر حرف رقم متمایزی را نشان می‌دهد؛ هدف یافتن جانشینی از رقم‌ها برای حروف است چنان‌که جمع حاصل از نظر حسابی درست باشد، با این قید اضافی که صفرهای ابتدایی مجاز نیستند. (ب) ابرگراف قید برای مسئلهٔ محاسبات‌نویسی، که قید Alldiff (جعبهٔ مربع بالایی) و نیز قیدهای جمع ستون (چهار جعبهٔ مربع میانی) را نشان می‌دهد. متغیرهای C1، C2 و C3 رقم‌های نقلی سه ستون را از راست به چپ بازنمایی می‌کنند.

از سوی دیگر، چنان‌که تمرین NARY.6 از شما می‌خواهد اثبات کنید، هر قید دامنهٔ متناهی را می‌توان با معرفی متغیرهای کمکیِ کافی به مجموعه‌ای از قیدهای دوتایی کاهش داد. این بدان معناست می‌توانستیم هر CSP را به CSPی با فقط قیدهای دوتایی تبدیل کنیم – که قطعاً زندگی طراح الگوریتم را ساده‌تر می‌کند. راه دیگر تبدیل CSP با قیدهای n-تایی به CSP دوتایی، «تبدیل گراف دوتایی» (Dual Graph Transformation) است: گراف تازه‌ای ایجاد کنید که در آن برای هر قید گراف اصلی یک متغیر و برای هر جفت قیدِ گراف اصلی که متغیر مشترکی دارند، یک قید

دوتایی باشد. برای نمونه، CSPی را با متغیرهای {X,Y,Z} = X در نظر بگیرید، هر یک با دامنهٔ {1,2,3,4,5}، و با دو قید ⟨(X,Y,Z), X+Y=Z⟩ :C1 و ⟨(X,Y), X+1=Y⟩ :C2. آنگاه گراف دوتایی متغیرهای {C1,C2} = X را خواهد داشت، که دامنهٔ متغیر C1 در گراف دوتایی، مجموعهٔ چندتاییهای {(xi,yj,zk)} از قید C1 در مسئلهٔ اصلی است و به همین ترتیب دامنهٔ C2 مجموعهٔ چندتاییهای {(xi,yj)} است. گراف دوتایی قید دوتایی ⟨(C1,C2), R1⟩ را دارد که R1 رابطهٔ تازهای است که قید میان C1 و C2 را تعریف میکند؛ در این مورد {((2,3),(2,3,5)) ,((1,2),(1,2,3))} = R1 خواهد بود.

با این حال دو دلیل هست که ممکن است قید سراسری مانند Alldiff را بر مجموعهای از قیدهای دوتایی ترجیح دهیم. نخست، نوشتن توصیف مسئله با Alldiff آسانتر و کمخطاتر است. دوم، میتوان الگوریتمهای استنتاج ویژهمنظوره برای قیدهای سراسری طراحی کرد که از کارکردن با قیدهای اولیه کارآمدترند. این الگوریتمهای استنتاج را در بخش ۶.۲.۵ توصیف میکنیم.

قیدهایی که تاکنون توصیف کردهایم همگی «قیدهای مطلق» (Absolute Constraints) بودهاند که نقضشان راهحل بالقوه را ناممکن میکند. بسیاری از CSPهای جهان واقعی شامل «قیدهای ترجیحی» (Preference Constraints) هستند که نشان میدهند کدام راهحلها ترجیح داده میشوند. برای نمونه، در مسئلهٔ زمانبندی کلاسهای دانشگاه قیدهای مطلقی هست که هیچ استادی نمیتواند همزمان دو کلاس تدریس کند. اما ممکن است قیدهای ترجیحی هم مجاز کنیم: استاد ر. ترجیح میدهد صبحها تدریس کند، در حالی که استاد ن. بعدازظهرها را ترجیح میدهد. برنامهای که استاد ر. ساعت ۲ بعدازظهر تدریس میکند همچنان راهحل مجازی خواهد بود (مگر آنکه استاد ر. اتفاقاً رئیس بخش باشد) اما راهحل بهینهای نخواهد بود.

قیدهای ترجیحی را اغلب میتوان بهصورت هزینههایی بر تخصیصهای متغیرِ فردی رمزگذاری کرد — برای نمونه، تخصیص جایگاه بعدازظهر به استاد ر. ۲ امتیاز در برابر تابع هدف کلی هزینه دارد، در حالی که جایگاه صبح ۱ هزینه دارد. با این صورتبندی، CSPهای با ترجیح را میتوان با روشهای جستجوی بهینهسازی حل کرد، چه مسیرمحور چه محلی. چنین مسئلهای را «مسئلهٔ بهینهسازی مقید» (Constrained Optimization Problem) یا COP مینامیم. برنامههای خطی یکی از ردههای COP هستند.

۶.۲ انتشار قید: استنتاج در CSPها

الگوریتم جستجوی فضای حالت اتمی فقط از یک راه پیشرفت میکند: با گسترش گره برای بازدید از جانشینها. الگوریتم CSP انتخابها دارد. میتواند با انتخاب تخصیص متغیرِ تازهای جانشین تولید کند، یا میتواند نوع خاصی از استنتاج به نام «انتشار قید» (Constraint Propagation) انجام دهد: استفاده از قیدها برای کاهش شمار مقادیر قانونی یک متغیر، که به نوبهٔ خود میتواند مقادیر قانونی متغیر دیگر را کاهش دهد، و الی آخر. ایده آن است که این کار انتخابهای کمتری برای در نظر گرفتن هنگام برداشتن انتخاب بعدی تخصیص متغیر باقی میگذارد.

انتشار قید ممکن است با جستجو در هم بافته شود، یا ممکن است بهصورت گام پیشپردازشی، پیش از آغاز جستجو انجام شود. گاهی این پیشپردازش میتواند کل مسئله را حل کند، پس اصلاً جستجویی لازم نیست. ایدهٔ کلیدی «سازگاری محلی» (Local Consistency) است. اگر هر متغیر را گرهای در گراف (شکل ۶.۱(ب) را ببینید) و هر قید

دوتایی را یالی بدانیم، آنگاه فرایند اعمال سازگاری محلی در هر بخش از گراف باعث حذف مقادیر ناسازگار در سراسر گراف می‌شود. انواع مختلفی از سازگاری محلی هست که اکنون به‌نوبت پوشش می‌دهیم.

۶.۲.۱ سازگاری گره‌ای

یک متغیر تکی (متناظر با گره‌ای در گراف CSP) «سازگار-گره‌ای» (Node-Consistent) است اگر همهٔ مقادیر دامنهٔ متغیر، قیدهای یک‌تایی متغیر را برآورده کنند. برای نمونه، در تغییرِ مسئلهٔ رنگ‌آمیزی نقشهٔ استرالیا (شکل ۶.۱) که در آن استرالیایی‌های جنوبی سبز را دوست ندارند، متغیر SA با دامنهٔ {red, green, blue} آغاز می‌کند و می‌توانیم با حذف green آن را سازگار-گره‌ای کنیم و SA را با دامنهٔ کاهش‌یافتهٔ {red, blue} باقی بگذاریم. می‌گوییم گرافی سازگار-گره‌ای است اگر هر متغیر در گراف سازگار-گره‌ای باشد.

حذف همهٔ قیدهای یک‌تایی در CSP با کاهش دامنهٔ متغیرهای دارای قید یک‌تایی در آغاز فرایند حل آسان است. چنان‌که پیش‌تر یاد شد، تبدیل همهٔ قیدهای n-تایی به دوتایی نیز ممکن است. به همین سبب، برخی حل‌کننده‌های CSP فقط با قیدهای دوتایی کار می‌کنند و انتظار دارند کاربر بقیهٔ قیدها را از پیش حذف کند. برای بقیهٔ این فصل این فرض را می‌کنیم، مگر جاهایی که خلاف آن یاد شود.

۶.۲.۲ سازگاری کمانی

متغیری در CSP «سازگار-کمانی» (Arc-Consistent)[1] است اگر هر مقدار در دامنه‌اش قیدهای دوتایی متغیر را برآورده کند. صوری‌تر، Xi نسبت به متغیر دیگر Xj سازگار-کمانی است اگر برای هر مقدار در دامنهٔ کنونی Di مقداری در دامنهٔ Dj وجود داشته باشد که قید دوتایی روی کمان (Xi,Xj) را برآورده کند. گرافی سازگار-کمانی است اگر هر متغیر نسبت به هر متغیر دیگر سازگار-کمانی باشد. برای نمونه، قید $Y = X^2$ را در نظر بگیرید که دامنهٔ هر دو X و Y مجموعهٔ رقم‌های ده‌دهی است. می‌توانیم این قید را صریحاً چنین بنویسیم: ⟨(X,Y), {(0,0),(1,1),(2,4),(3,9)}⟩.

برای سازگار-کمانی‌کردن X نسبت به Y، دامنهٔ X را به {0,1,2,3} کاهش می‌دهیم. اگر Y را نیز نسبت به X سازگار-کمانی کنیم، دامنهٔ Y به {0,1,4,9} تبدیل می‌شود و کل CSP سازگار-کمانی است. از سوی دیگر، سازگاری کمانی هیچ کاری برای مسئلهٔ رنگ‌آمیزی نقشهٔ استرالیا نمی‌تواند بکند. قید نابرابری زیر روی (SA,WA) را در نظر بگیرید:

```
{(blue,green) ,(blue,red) ,(green,blue) ,(green,red) ,(red,blue) ,(red,green)}
```

هر مقداری که برای SA (یا WA) برگزینید، مقدار معتبری برای متغیر دیگر هست. پس اعمال سازگاری کمانی هیچ اثری بر دامنه‌های هیچ‌یک از دو متغیر ندارد.

محبوب‌ترین الگوریتم برای اعمال سازگاری کمانی AC-3 نامیده می‌شود (شکل ۶.۳ را ببینید). برای سازگار-کمانی‌کردن هر متغیر، الگوریتم AC-3 صفی از کمان‌ها برای بررسی نگه می‌دارد. در آغاز، صف شامل همهٔ کمان‌های CSP است. (هر قید دوتایی دو کمان می‌شود، یکی در هر جهت.) AC-3 سپس کمان دلخواهی (Xi,Xj) را از صف بیرون می‌آورد و Xi را نسبت به Xj سازگار-کمانی می‌کند. اگر این Di را بدون تغییر بگذارد، الگوریتم فقط به کمان بعدی می‌رود. اما اگر

Di را بازبینی کند (دامنه را کوچک‌تر کند)، همهٔ کمان‌های (Xk,Xi) را به صف می‌افزاییم که در آن Xk همسایهٔ Xi است. باید این کار را بکنیم، زیرا تغییر Di ممکن است کاهش‌های بیشتری در Dk را ممکن کند، حتی اگر Xk را قبلاً در نظر گرفته باشیم.

اگر Di به هیچ بازبینی شود، می‌دانیم کل CSP هیچ راه‌حل سازگاری ندارد و AC-3 می‌تواند بلافاصله شکست بازگرداند. وگرنه، به بررسی ادامه می‌دهیم و می‌کوشیم مقادیری را از دامنه‌های متغیرها حذف کنیم تا آنکه دیگر کمانی در صف نماند. در آن نقطه، با CSPی روبه‌روییم که با CSP اصلی هم‌ارز است — هر دو راه‌حل‌های یکسانی دارند — اما CSP سازگار-کمانی سریع‌تر جستجو خواهد شد، زیرا متغیرهایش دامنه‌های کوچک‌تری دارند. در برخی موارد، مسئله را کامل حل می‌کند (با کاهش هر دامنه به اندازهٔ ۱) و در موارد دیگر اثبات می‌کند هیچ راه‌حلی نیست (با کاهش دامنه‌ای به اندازهٔ ۰).

```
function AC-3(csp) returns false if an inconsistency is found and true otherwise
                        queue ← a queue of arcs, initially all the arcs in csp
                                                  while queue is not empty do
                                                      POP(queue) ← (Xi, Xj)
                                                if REVISE(csp, Xi, Xj) then
                                          if size of Di = 0 then return false
                                       for each Xk in Xi.NEIGHBORS - {Xj} do
                                                    add (Xk, Xi) to queue
                                                                 return true

                function REVISE(csp, Xi, Xj) returns true iff we revise the domain of Xi
                                                                revised ← false
                                                              for each x in Di do
    if no value y in Dj allows (x,y) to satisfy the constraint between Xi and Xj then
                                                            delete x from Di
                                                              revised ← true
                                                                  return revised
```

شکل ۶.۳ — الگوریتم سازگاری کمانی AC-3. پس از اعمال AC-3، یا هر کمان سازگار-کمانی است، یا متغیری دامنهٔ خالی دارد که نشان می‌دهد CSP قابل حل نیست. نام «AC-3» توسط مخترع الگوریتم (مک‌ورث، ۱۹۷۷) به کار رفت، زیرا سومین نسخه‌ای بود که در آن مقاله توسعه یافت.

پیچیدگی AC-3 را می‌توان چنین تحلیل کرد. CSPی را با n متغیر فرض کنید، هر یک با اندازهٔ دامنهٔ حداکثر d، و با c قید دوتایی (کمان). هر کمان (Xk,Xi) فقط d بار می‌تواند در صف درج شود، زیرا Xi حداکثر d مقدار برای حذف دارد. بررسی سازگاری یک کمان را می‌توان در زمان $O(d^2)$ انجام داد، پس در مجموع بدترین حالت $O(cd^3)$ زمان می‌گیریم.

۶.۲.۳ سازگاری مسیر

سازگاری کمانی دامنه‌ها (قیدهای یک‌تایی) را با استفاده از کمان‌ها (قیدهای دوتایی) سفت می‌کند. برای پیشرفت در مسائلی مانند رنگ‌آمیزی نقشه، به مفهوم قوی‌تری از سازگاری نیاز داریم. «سازگاری مسیر» (Path Consistency) قیدهای دوتایی را با استفاده از قیدهای ضمنی‌ای که با نگاه‌کردن به سه‌تایی متغیرها استنتاج می‌شوند سفت می‌کند.

مجموعهٔ دو متغیرهٔ {Xi,Xj} نسبت به متغیر سوم Xm «سازگار-مسیر» است اگر برای هر تخصیص {Xi = a, Xj = b} سازگار با قیدهای (در صورت وجود) روی {Xi,Xj}، تخصیصی به Xm وجود داشته باشد که قیدهای روی {Xi,Xm} و {Xm,Xj} را برآورده کند. این نام به سازگاری کلی مسیر از Xi تا Xj با Xm در میانه اشاره دارد.

ببینیم سازگاری مسیر در رنگ‌آمیزی نقشهٔ استرالیا با دو رنگ چگونه عمل می‌کند. مجموعهٔ {WA,SA} را نسبت به NT سازگار-مسیر می‌کنیم. با برشمردن تخصیص‌های سازگار به مجموعه آغاز می‌کنیم. در این مورد فقط دو تای آن هست: {WA = red, SA = blue} و {WA = blue, SA = red}. می‌بینیم با هر دوی این تخصیص‌ها، NT نمی‌تواند نه red باشد نه blue (زیرا یا با WA یا با SA در تعارض می‌افتد). چون انتخاب معتبری برای NT نیست، هر دو تخصیص را حذف می‌کنیم و در پایان هیچ تخصیص معتبری برای {WA,SA} نداریم. پس می‌دانیم هیچ راه‌حلی برای این مسئله نمی‌تواند وجود داشته باشد.

۶.۲.۴ سازگاری k-تایی

شکل‌های قوی‌تر انتشار را می‌توان با مفهوم «سازگاری k-تایی» (K-Consistency) تعریف کرد. CSP «سازگار-k» است اگر برای هر مجموعه از k-1 متغیر و برای هر تخصیص سازگار به آن متغیرها، مقدار سازگاری برای هر متغیر kام وجود داشته باشد. برای نمونه، ۱-سازگاری یعنی هر متغیر تکی مقداری دارد که همهٔ قیدهای یک‌تایی را برآورده می‌کند؛ این همان سازگاری گره‌ای است. ۲-سازگاری همان سازگاری کمانی است. ۳-سازگاری یعنی هر جفت متغیرِ سازگار را می‌توان به متغیر سومی بسط داد که سازگار باشد؛ و به همین ترتیب برای سازگاری ۴-تایی، ۵-تایی و الی آخر.

می‌توان اثبات کرد CSPی که سازگار-n است (n تعداد متغیرها) بدون جستجو قابل حل است، به این معنا که می‌توان یک به یک متغیرها را مقداردهی کرد، و در هر گام مقدارِ بعدی را می‌توان با بررسی قیدها انتخاب کرد. برای هر متغیر Xi فقط باید d مقدار موجود در دامنه را جستجو کنیم تا مقداری سازگار با X1,...,Xi-1 بیابیم. زمان کل اجرا فقط $O(n^2 d)$ است.

البته ناهار مجانی وجود ندارد: ارضای قید در حالت کلی NP-کامل است و هر الگوریتمی برای برقراری سازگاری n باید در بدترین حالت زمانی نمایی نسبت به n بگیرد. بدتر از آن، سازگاری-n به فضایی نمایی نسبت به n نیز نیاز دارد. در عمل، تعیین سطح مناسب بررسی سازگاری بیشتر علمِ تجربی است. محاسبهٔ ۲-سازگاری رایج است و ۳-سازگاری کمتر رایج.

۶.۲.۵ قیدهای سراسری

به یاد آورید قید سراسری قیدی است شامل تعداد دلخواهی متغیر (اما نه لزوماً همهٔ متغیرها). قیدهای سراسری در مسائل واقعی مکرراً رخ می‌دهند و می‌توان آن‌ها را با الگوریتم‌های ویژه‌منظوره‌ای مدیریت کرد که از روش‌های عمومیِ تاکنون‌توصیف‌شده کارآمدترند. برای نمونه، قید Alldiff می‌گوید همهٔ متغیرهای درگیر در قید باید مقادیر متمایز داشته باشند (مانند مسئلهٔ محاسبات‌نویسیِ بالا و معماهای سودوکو در ادامه). یکی از شکل‌های سادهٔ تشخیص ناسازگاری برای قیدهای Alldiff چنین کار می‌کند: اگر m متغیر در قید درگیر باشند و در مجموع n مقدار متمایز ممکن داشته باشند، و m > n، آنگاه قید قابل ارضای نیست.

این به الگوریتم سادهٔ زیر می‌انجامد: نخست، هر متغیری در قید که دامنهٔ تک‌تایی دارد را حذف کنید و مقدار آن متغیر را از دامنه‌های متغیرهای باقی‌مانده حذف کنید. تا وقتی متغیر تک‌تایی هست تکرار کنید. اگر در هر نقطه دامنهٔ خالی تولید شود یا متغیرهای بیشتری از مقادیر دامنهٔ باقی‌مانده باشند، ناسازگاری تشخیص داده شده است.

این روش می‌تواند ناسازگاری تخصیص {WA=red, NSW=red} را برای شکل ۶.۱ تشخیص دهد. توجه کنید متغیرهای SA، NT و Q عملاً با قید Alldiff به هم متصل‌اند، زیرا هر جفت باید دو رنگ متفاوت داشته باشند. پس از اعمال AC-3 با تخصیص جزئی، دامنه‌های SA، NT و Q همگی به {green, blue} کاهش می‌یابند. یعنی سه متغیر و فقط دو رنگ داریم، پس قید Alldiff نقض شده. بنابراین، رویهٔ سازگاری ساده‌ای برای قید مرتبه‌بالاتر گاهی مؤثرتر از اعمال سازگاری کمانی بر مجموعه‌ای هم‌ارز از قیدهای دوتایی است.

قید مرتبه‌بالاتر مهم دیگر «قید منبع» (Resource Constraint) است که گاهی قید Atmost نامیده می‌شود. برای نمونه، در مسئلهٔ زمان‌بندی، P1,...,P4 شمار پرسنل تخصیص‌یافته به هر یک از چهار تکلیف باشد. قید اینکه در مجموع بیش از ۱۰ پرسنل تخصیص نیافته باشد چنین نوشته می‌شود: Atmost(10, P1, P2, P3, P4). می‌توانیم ناسازگاری را صرفاً با بررسی مجموع کمینه‌های دامنه‌های کنونی تشخیص دهیم؛ برای نمونه، اگر هر متغیر دامنهٔ {3,4,5,6} داشته باشد، قید Atmost قابل ارضای نیست. همچنین می‌توانیم با حذف بیشینهٔ هر دامنه اگر با کمینه‌های دامنه‌های دیگر سازگار نباشد، سازگاری را اعمال کنیم. پس اگر در مثال ما هر متغیر دامنهٔ {2,3,4,5,6} داشته باشد، مقادیر ۵ و ۶ را می‌توان از هر دامنه حذف کرد.

برای مسائل بزرگِ محدود-به-منبع با مقادیر صحیح — مانند مسائل لجستیکیِ جابه‌جایی هزاران نفر در صدها وسیلهٔ نقلیه — معمولاً نمی‌توان دامنهٔ هر متغیر را به‌صورت مجموعهٔ بزرگی از اعداد صحیح بازنمایی کرد و آن مجموعه را به‌تدریج با روش‌های بررسی سازگاری کاهش داد. در عوض، دامنه‌ها با کران بالا و پایین بازنمایی و با «انتشار کران‌ها» (Bounds Propagation) مدیریت می‌شوند. برای نمونه، در مسئلهٔ زمان‌بندی خطوط هوایی، فرض کنیم دو پرواز F1 و F2 هست که هواپیماهایشان به‌ترتیب ظرفیت‌های ۱۶۵ و ۳۸۵ دارند. دامنه‌های آغازین شمار مسافران پروازهای F1 و F2 پس چنین‌اند:

```
D1 = [0, 165]
D2 = [0, 385]
```

حال فرض کنیم قید اضافی هست که دو پرواز با هم باید ۴۲۰ نفر را حمل کنند: F1 + F2 = 420. با انتشار قیدهای کران، دامنه‌ها را به این‌ها کاهش می‌دهیم:

```
D1 = [35, 165]
D2 = [255, 385]
```

می‌گوییم CSP «سازگار-کران» (Bounds-Consistent) است اگر برای هر متغیر X، و برای هر دو مقدار کران پایین و کران بالای X، مقداری از Y وجود داشته باشد که قید میان X و Y را برای هر متغیر Y برآورده کند. این نوع انتشار کران به‌طور گسترده در مسائل قید عملی استفاده می‌شود.

۶.۲.۶ سودوکو

معمای پرطرفدار سودوکو میلیون‌ها نفر را با مسئله‌های ارضای قیود آشنا کرده، هرچند شاید خودشان نفهمند. تختهٔ سودوکو از ۸۱ خانه تشکیل شده که برخی از آن‌ها در آغاز با رقم‌هایی از ۱ تا ۹ پر شده‌اند. معما پرکردن همهٔ خانه‌های باقی‌مانده است چنان‌که هیچ رقمی در هیچ سطر، ستون یا جعبهٔ 3×3 دوبار تکرار نشود (شکل ۶.۴ را ببینید). سطر، ستون یا جعبه «واحد» (Unit) نامیده می‌شود.

	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

(a)

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

(b)

شکل ۶.۴ – (الف) معمای سودوکو و (ب) راه‌حل آن. (تخته با سطرهای A تا I و ستون‌های ۱ تا ۹)

معماهای سودوکویی که در روزنامه‌ها و کتاب‌های معما می‌آیند این ویژگی را دارند که دقیقاً یک راه‌حل دارند. هرچند برخی را با دست می‌توان حل‌کردنیِ دشوار دانست و ده‌ها دقیقه می‌برند، یک حل‌کنندهٔ CSP می‌تواند هزاران معما را در ثانیه مدیریت کند.

معمای سودوکو را می‌توان CSP با ۸۱ متغیر دانست، برای هر خانه یکی. از نام‌های متغیر A1 تا A9 برای سطر بالا (چپ به راست) استفاده می‌کنیم و تا I1 تا I9 برای سطر پایین. خانه‌های خالی دامنهٔ {1,2,3,4,5,6,7,8,9} دارند و خانه‌های ازپیش‌پر، دامنه‌ای شامل یک مقدار تکی. افزون بر این، ۲۷ قید Alldiff متفاوت هست، یکی برای هر واحد (سطر، ستون و جعبهٔ ۹خانه‌ای):

```
Alldiff(A1, A2, A3, A4, A5, A6, A7, A8, A9)
Alldiff(B1, B2, B3, B4, B5, B6, B7, B8, B9)
                                        ...
Alldiff(A1, B1, C1, D1, E1, F1, G1, H1, I1)
Alldiff(A2, B2, C2, D2, E2, F2, G2, H2, I2)
                                        ...
Alldiff(A1, A2, A3, B1, B2, B3, C1, C2, C3)
Alldiff(A4, A5, A6, B4, B5, B6, C4, C5, C6)
                                        ...
```

ببینیم سازگاری کمانی ما را تا کجا می‌برد. فرض کنیم قیدهای Alldiff به قیدهای دوتایی بسط یافته‌اند (مانند A1≠A2) تا بتوانیم الگوریتم AC-3 را مستقیماً اعمال کنیم. متغیر E6 از شکل ۶.۴(الف) را در نظر بگیرید — خانهٔ خالی میان ۲ و ۸ در جعبهٔ میانی. از قیدهای جعبه، می‌توانیم ۱، ۲، ۷ و ۸ را از دامنهٔ E6 حذف کنیم. از قیدهای ستونش، می‌توانیم ۵، ۶، ۲، ۸، ۹ و ۳ را حذف کنیم (هرچند ۲ و ۸ از قبل حذف شده بودند). این E6 را با دامنهٔ {4} باقی می‌گذارد؛ به بیان دیگر، پاسخ E6 را می‌دانیم. حال متغیر I6 را در نظر بگیرید — خانهٔ جعبهٔ میانی پایین که با ۱، ۳ و ۳ احاطه شده. با اعمال سازگاری کمانی در ستونش، ۵، ۶، ۲، ۴ (چون اکنون می‌دانیم E6 باید ۴ باشد)، ۸، ۹ و ۳ را حذف می‌کنیم. با سازگاری کمانی با I5 مقدار ۱ را حذف می‌کنیم و فقط مقدار ۷ در دامنهٔ I6 می‌ماند. حال در ستون ۶ هشت مقدار شناخته‌شده هست، پس سازگاری کمانی می‌تواند استنتاج کند A6 باید ۱ باشد. استنتاج در همین مسیر ادامه می‌یابد و سرانجام AC-3 می‌تواند کل معما را حل کند — همهٔ متغیرها دامنه‌هایشان به یک مقدار تکی کاهش می‌یابد، چنان‌که در شکل ۶.۴(ب) نشان داده شده.

البته سودوکو به‌زودی جذابیتش را از دست می‌داد اگر هر معما با اعمال مکانیکی AC-3 حل می‌شد، و واقعاً AC-3 فقط برای آسان‌ترین معماهای سودوکو کار می‌کند. معماهای کمی سخت‌تر را PC-2 می‌تواند حل کند، اما به هزینهٔ محاسباتی بیشتر: در معمای سودوکو ۲۵۵,۹۶۰ قید مسیر متفاوت برای در نظر گرفتن هست. برای حل سخت‌ترین معماها و پیشرفت کارآمد، باید هوشمندانه‌تر باشیم.

در واقع، جذابیت معماهای سودوکو برای حل‌کنندهٔ انسانی نیاز به خلاقیت در اعمال راهبردهای استنتاج پیچیده‌تر است. هواداران نام‌های رنگارنگی برایشان دارند، مانند «سه‌تایی‌های لخت». آن راهبرد چنین کار می‌کند: در هر واحد (سطر، ستون یا جعبه)، سه خانه بیابید که هر یک دامنه‌ای شامل همان سه عدد یا زیرمجموعه‌ای از آن‌ها دارد. برای نمونه، سه دامنه ممکن است {1,8}، {3,8} و {1,3,8} باشند. از این نمی‌دانیم کدام خانه شامل ۱، ۳ یا ۸ است، اما می‌دانیم سه عدد باید میان سه خانه توزیع شوند. پس می‌توانیم ۱، ۳ و ۸ را از دامنه‌های هر خانهٔ دیگر در واحد حذف کنیم.

جالب است توجه کنیم چقدر می‌توانیم پیش برویم بدون آنکه چیز خاصی دربارهٔ سودوکو بگوییم. البته باید بگوییم ۸۱ متغیر هست، دامنه‌هایشان رقم‌های ۱ تا ۹ است و ۲۷ قید Alldiff هست. اما فراتر از آن، همهٔ راهبردها — سازگاری کمانی، سازگاری مسیر و الی آخر — به‌طور کلی بر همهٔ CSPها اعمال می‌شوند، نه فقط مسائل سودوکو. حتی سه‌تایی‌های لخت واقعاً راهبردی برای اعمال سازگاری قیدهای Alldiff است و به خودی خود ویژهٔ سودوکو نیست. این قدرت صورت‌بندی CSP است: برای هر حوزهٔ مسئلهٔ تازه، فقط باید مسئله را بر حسب قیدها تعریف کنیم؛ سپس سازوکارهای عمومی حل قید می‌توانند کار را به دست بگیرند.

۶.۳ جستجوی عقبگرد برای CSPها

گاهی می‌توانیم فرایند انتشار قید را کامل کنیم و همچنان متغیرهایی با چند مقدار ممکن داشته باشیم. در آن صورت باید برای راه‌حل جستجو کنیم. در این بخش الگوریتم‌های جستجوی عقبگرد (Backtracking Search) را پوشش می‌دهیم که روی تخصیص‌های جزئی کار می‌کنند؛ در بخش بعد الگوریتم‌های جستجوی محلی روی تخصیص‌های کامل را می‌بینیم.

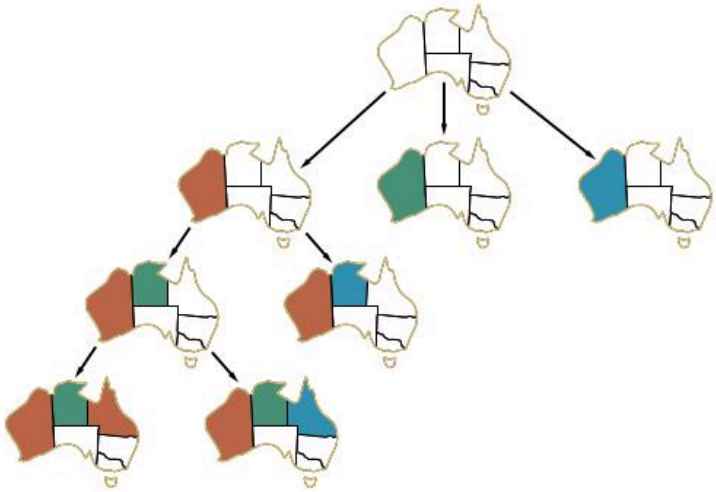
در نظر بگیرید جستجوی استاندارد محدود-به-عمق (از فصل ۳) چگونه می‌توانست CSPها را حل کند. حالت، تخصیص جزئی می‌بود و کنش، بسط تخصیص با افزودن مثلاً NSW = red یا SA = blue برای مسئلهٔ رنگ‌آمیزی نقشهٔ استرالیا. برای CSP با n متغیرِ دامنهٔ اندازهٔ d، به درخت جستجویی می‌رسیدیم که در آن همهٔ تخصیص‌های کامل (و در نتیجه همهٔ راه‌حل‌ها) گره‌های برگ در عمق n هستند. اما توجه کنید ضریب انشعاب در سطح بالا nd می‌بود، زیرا هر یک از d مقدار را می‌توان به هر یک از n متغیر تخصیص داد. در سطح بعد، ضریب انشعاب (n-1)d است و الی آخر برای n سطح. پس درخت $n! \cdot d^n$ برگ دارد، حتی با آنکه فقط d^n تخصیص کامل ممکن هست!

می‌توانیم آن ضریب n! را با شناخت ویژگی حیاتی CSPها بازپس بگیریم: «جابه‌جایی‌پذیری» (Commutativity). مسئله جابه‌جایی‌پذیر است اگر ترتیب اعمال هر مجموعهٔ داده‌شده از کنش‌ها مهم نباشد. در CSPها، فرقی نمی‌کند نخست NSW = red را تخصیص دهیم و سپس SA = blue، یا برعکس. پس در هر گره از درخت جستجو فقط باید یک متغیر را در نظر بگیریم. در ریشه ممکن است انتخابی میان SA=green، SA=red و SA=blue بکنیم، اما هرگز بین NSW=red و SA=blue انتخاب نمی‌کنیم. با این محدودیت، شمار برگ‌ها d^n است، چنان‌که امیدوار بودیم. در هر سطح درخت باید انتخاب کنیم با کدام متغیر سروکار داریم، اما هرگز لازم نیست بر آن انتخاب عقبگرد کنیم.

شکل ۶.۵ رویهٔ جستجوی عقبگرد را برای CSPها نشان می‌دهد. به‌طور مکرر متغیر تخصیص‌نیافته‌ای را برمی‌گزیند و سپس همهٔ مقادیر دامنهٔ آن متغیر را به‌نوبت می‌آزماید و می‌کوشد هر یک را با فراخوانی بازگشتی به راه‌حل بسط دهد. اگر فراخوانی موفق شود، راه‌حل بازگردانده می‌شود و اگر شکست بخورد، تخصیص به حالت قبلی بازگردانده می‌شود و مقدار بعدی را می‌آزماییم. اگر هیچ مقداری کار نکند، شکست بازمی‌گردانیم. بخشی از درخت جستجو برای مسئلهٔ استرالیا در شکل ۶.۶ نشان داده شده، که در آن متغیرها را به ترتیب WA, NT, Q, ... تخصیص داده‌ایم. توجه کنید BACKTRACKING-SEARCH فقط یک بازنمایی از حالت (تخصیص) نگه می‌دارد و آن بازنمایی را تغییر می‌دهد، نه آنکه بازنمایی‌های تازه ایجاد کند (صفحهٔ ۸۰ را ببینید).

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
    return BACKTRACK(csp, {})

function BACKTRACK(csp, assignment) returns a solution or failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
        if value is consistent with assignment then
            add {var = value} to assignment
            inferences ← INFERENCE(csp, var, assignment)
            if inferences ≠ failure then
                add inferences to assignment
                result ← BACKTRACK(csp, assignment)
                if result ≠ failure then return result
        remove {var = value} and inferences from assignment
    return failure
```

شکل ۶.۵ – الگوریتم عقبگرد ساده برای مسئله‌های ارضای قیود. الگوریتم بر اساس جستجوی بازگشتی عمق-اول فصل ۳ مدل شده. توابع SELECT-UNASSIGNED-VARIABLE و ORDER-DOMAIN-VALUES اکتشافی‌های عمومیِ بحث‌شده در بخش ۶.۳.۱ را پیاده می‌کنند. تابع INFERENCE بنا به دلخواه می‌تواند سازگاری کمانی، مسیری یا k-تایی اعمال کند. اگر انتخاب مقداری به شکست بینجامد (چه توسط INFERENCE چه توسط BACKTRACK تشخیص داده شود)، تخصیص‌های مقدار (شامل آن‌هایی که INFERENCE انجام داده) پس گرفته می‌شوند و مقدار تازه‌ای آزموده می‌شود.



شکل ۶.۶ – بخشی از درخت جستجو برای مسئلهٔ رنگ‌آمیزی نقشه در شکل ۶.۱.

در حالی که الگوریتم‌های جستجوی نامطلع فصل ۳ فقط با عرضهٔ اکتشافی‌های حوزه‌ویژه بهبود می‌یافتند، معلوم می‌شود جستجوی عقبگرد را می‌توان با اکتشافی‌های مستقل از حوزه بهبود داد که از بازنمایی عاملی CSPها بهره می‌گیرند. در چهار بخش زیر نشان می‌دهیم چگونه:

- (۶.۳.۱) کدام متغیر باید بعدی تخصیص یابد (SELECT-UNASSIGNED-VARIABLE)، و مقادیرش به چه ترتیبی آزموده شوند (ORDER-DOMAIN-VALUES)؟
- (۶.۳.۲) در هر گام از جستجو چه استنتاج‌هایی باید انجام شود (INFERENCE)؟
- (۶.۳.۳) آیا می‌توانیم هنگام مقتضی بیش از یک گام عقبگرد کنیم؟
- (۶.۳.۴) آیا می‌توانیم نتایج جزئی جستجو را ذخیره و دوباره استفاده کنیم؟

۶.۳.۱ ترتیب متغیر و مقدار

الگوریتم عقبگرد شامل این خط است:

```
var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
```

ساده‌ترین راهبرد برای SELECT-UNASSIGNED-VARIABLE «ترتیب ایستا» است: متغیرها را به‌ترتیب {X1, X2, ...} برگزین. ساده‌ترین بعدی، انتخاب تصادفی است. هیچ‌کدام از این راهبردها بهینه نیست. برای نمونه، پس از تخصیص‌های WA=red و NT=green در شکل ۶.۶، فقط یک مقدار ممکن برای SA هست، پس منطقی است

SA=blue را بعدی تخصیص دهیم نه Q را. در واقع، پس از تخصیص SA، انتخاب‌های NSW ،Q و V همگی اجباری می‌شوند.

این ایدهٔ شهودی — برگزیدن متغیری با کمترین مقدار «قانونی» — «اکتشافی کمترین-مقادیر-باقی‌مانده» (Minimum-Remaining-Values، یا MRV) نامیده می‌شود. همچنین «متغیرِ بیشترین-قید» یا اکتشافی «اول-شکست» نامیده شده؛ دومی از آن رو که متغیری را برمی‌گزیند که به احتمال زیاد به‌زودی شکست می‌خورد و بدین ترتیب درخت جستجو را هرس می‌کند. اگر متغیری X هیچ مقدار قانونی باقی نگذاشته باشد، اکتشافی MRV آن را برخواهد گزید و شکست بلافاصله تشخیص داده می‌شود — و از جستجوهای بیهوده در میان متغیرهای دیگر پرهیز می‌شود. اکتشافی MRV معمولاً بهتر از ترتیب تصادفی یا ایستا عمل می‌کند، گاهی چند مرتبهٔ بزرگی بهتر، هرچند نتایج به مسئله بستگی دارد.

اکتشافی MRV در انتخاب نخستین ناحیه برای رنگ‌کردن در استرالیا اصلاً کمک نمی‌کند، زیرا در آغاز هر ناحیه سه رنگ قانونی دارد. در این مورد، «اکتشافی درجه» (Degree Heuristic) به کار می‌آید. این اکتشافی می‌کوشد ضریب انشعاب انتخاب‌های آینده را با برگزیدن متغیری که در بیشترین شمار قیدها با دیگر متغیرهای تخصیص‌نیافته شرکت دارد کاهش دهد. در شکل ۶.۱، SA متغیری با بیشترین درجه، یعنی ۵، است؛ دیگر متغیرها درجهٔ ۲ یا ۳ دارند، به‌جز T که درجهٔ ۰ دارد. در واقع، پس از انتخاب SA، اعمال اکتشافی درجه مسئله را بدون هیچ گام اشتباهی حل می‌کند — در هر نقطهٔ انتخاب می‌توانید هر رنگ سازگاری برگزینید و همچنان بدون هیچ عقبگردی به راه‌حل برسید. اکتشافی کمترین-مقادیر-باقی‌مانده معمولاً راهنمای قدرتمندتری است، اما اکتشافی درجه به‌عنوان شکست‌کنندهٔ تساوی مفید است.

پس از انتخاب متغیر، الگوریتم باید دربارهٔ ترتیب بررسی مقادیرش تصمیم بگیرد. «اکتشافی کمترین-قید-محدودکننده» (Least-Constraining-Value Heuristic) برای این کار مؤثر است. مقداری را ترجیح می‌دهد که کمترین انتخاب‌ها را برای متغیرهای همسایه در گراف قید حذف کند. برای نمونه، فرض کنیم در شکل ۶.۱ تخصیص جزئی WA=red و NT=green را تولید کرده‌ایم و انتخاب بعدی‌مان برای Q است. Blue انتخاب بدی خواهد بود، زیرا آخرین مقدار قانونیِ باقی‌مانده برای همسایهٔ Q یعنی SA را حذف می‌کند. اکتشافی کمترین-قید-محدودکننده پس red را به blue ترجیح می‌دهد. در حالت کلی، اکتشافی می‌کوشد بیشترین انعطاف را برای تخصیص‌های متغیر بعدی باقی بگذارد.

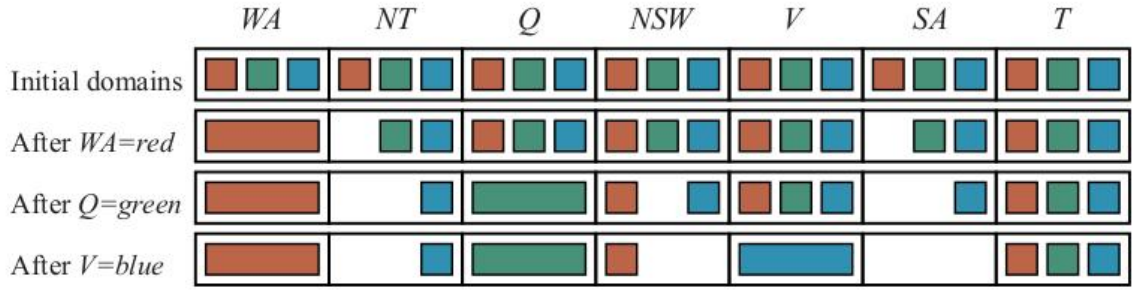
چرا انتخاب متغیر باید اول-شکست باشد، اما انتخاب مقدار باید آخر-شکست؟ هر متغیر بالاخره باید تخصیص یابد، پس با برگزیدن متغیرهایی که به احتمال زیاد اول شکست می‌خورند، به‌طور میانگین تخصیص‌های موفقِ کمتری برای عقبگرد خواهیم داشت. برای ترتیب مقدار، حربه آن است که فقط به یک راه‌حل نیاز داریم؛ پس منطقی است نخست به‌دنبال محتمل‌ترین مقادیر بگردیم. اگر می‌خواستیم همهٔ راه‌حل‌ها را برشماریم نه اینکه فقط یکی بیابیم، ترتیب مقدار بی‌ربط می‌بود.

۶.۳.۲ درهم‌آمیختن جستجو و استنتاج

دیدیم AC-3 چگونه می‌تواند دامنه‌های متغیرها را پیش از آغاز جستجو کاهش دهد. اما استنتاج در طول جستجو می‌تواند حتی قدرتمندتر باشد: هر بار که انتخابی برای مقدار یک متغیر می‌کنیم، فرصتی کاملاً تازه برای استنتاج کاهش‌های دامنهٔ تازه روی متغیرهای همسایه داریم.

یکی از ساده‌ترین شکل‌های استنتاج «بررسی رو به جلو» (Forward Checking) نامیده می‌شود. هرگاه متغیری X تخصیص یابد، فرایند بررسی رو به جلو برای آن سازگاری کمانی برقرار می‌کند: برای هر متغیر تخصیص‌نیافتهٔ Y که با قیدی به X متصل است، هر مقداری را که با مقدارِ برگزیده‌شده برای X ناسازگار است از دامنهٔ Y حذف کن.

شکل ۶.۷ پیشرفت جستجوی عقبگرد را روی CSP استرالیا با بررسی رو به جلو نشان می‌دهد. دو نکتهٔ مهم دربارهٔ این مثال هست. نخست، توجه کنید پس از تخصیص WA=red و Q=green، دامنه‌های NT و SA به یک مقدار تکی کاهش می‌یابند؛ با انتشار اطلاعات از WA و Q انشعاب روی این متغیرها را کلاً حذف کرده‌ایم. نکتهٔ دوم آن است که پس از V=blue، دامنهٔ SA خالی می‌شود. از این رو، بررسی رو به جلو تشخیص داده تخصیص جزئی {WA=red, Q=green, V=blue} با قیدهای مسئله ناسازگار است و الگوریتم بلافاصله عقبگرد می‌کند.



شکل ۶.۷ – پیشرفت جستجوی رنگ‌آمیزی نقشه با بررسی رو به جلو. نخست WA=red تخصیص می‌یابد؛ سپس بررسی رو به جلو red را از دامنه‌های متغیرهای همسایه NT و SA حذف می‌کند. پس از تخصیص Q=green، green از دامنه‌های NT، SA و NSW حذف می‌شود. پس از تخصیص V=blue، blue از دامنه‌های NSW و SA حذف می‌شود و SA بدون هیچ مقدار قانونی می‌ماند.

برای بسیاری از مسائل، جستجو مؤثرتر خواهد بود اگر اکتشافی MRV را با بررسی رو به جلو ترکیب کنیم. شکل ۶.۷ را پس از تخصیص {WA=red} در نظر بگیرید. به‌طور شهودی به نظر می‌رسد آن تخصیص همسایه‌هایش NT و SA را مقید می‌کند، پس باید بعدی با آن‌ها سروکار داشته باشیم و آنگاه بقیهٔ متغیرها سر جایشان می‌افتند. این دقیقاً همان چیزی است که با MRV رخ می‌دهد: NT و SA هر یک دو مقدار دارند، پس یکی از آن‌ها نخست انتخاب می‌شود، سپس دیگری، سپس NSW، Q و V به‌ترتیب. سرانجام T هنوز سه مقدار دارد و هر یک از آن‌ها کار می‌کند. می‌توانیم بررسی رو به جلو را راهی کارآمد برای محاسبهٔ افزایشیِ اطلاعاتی بدانیم که اکتشافی MRV برای انجام کارش نیاز دارد.

هرچند بررسی رو به جلو بسیاری از ناسازگاری‌ها را تشخیص می‌دهد، همهٔ آن‌ها را تشخیص نمی‌دهد. مسئله آن است که به اندازهٔ کافی به جلو نگاه نمی‌کند. برای نمونه، ردیف Q=green شکل ۶.۷ را در نظر بگیرید. WA و Q را سازگار-کمانی کرده‌ایم، اما NT و SA را هر دو با blue به‌عنوان تنها مقدار ممکن‌شان باقی گذاشته‌ایم، که ناسازگاری است، زیرا آن‌ها همسایه‌اند.

الگوریتمی به نام MAC (مخفف Maintaining Arc Consistency – حفظ سازگاری کمانی) ناسازگاری‌هایی مانند این را تشخیص می‌دهد. پس از آنکه متغیری Xi مقداری تخصیص یافت، رویهٔ INFERENCE با AC-3 تماس می‌گیرد، اما به‌جای صفی از همهٔ کمان‌های CSP، فقط با کمان‌های (Xj, Xi) برای همهٔ Xjهایی که متغیرهای تخصیص‌نیافته و همسایهٔ Xi هستند آغاز می‌کنیم. از آن‌جا، AC-3 انتشار قید را به‌روش معمول انجام می‌دهد و اگر متغیری دامنه‌اش به مجموعهٔ خالی کاهش یابد، تماس با AC-3 شکست می‌خورد و می‌دانیم باید بلافاصله عقبگرد کنیم. می‌بینیم MAC

به‌طور دقیق قدرتمندتر از بررسی رو به جلوست، زیرا بررسی رو به جلو همان کار MAC را روی کمان‌های آغازین صف MAC انجام می‌دهد؛ اما برخلاف MAC، بررسی رو به جلو هنگام ایجاد تغییر در دامنه‌های متغیرها قیدها را به‌صورت بازگشتی انتشار نمی‌دهد.

۶.۳.۳ عقبگرد هوشمند: نگاه به عقب

الگوریتم BACKTRACKING-SEARCH در شکل ۶.۵ سیاست بسیار ساده‌ای برای آنچه هنگام شکست شاخه‌ای از جستجو باید کرد دارد: به متغیر قبلی برگرد و مقدار متفاوتی برایش امتحان کن. این «عقبگرد زمانی» (Chronological Backtracking) نامیده می‌شود، زیرا به تازه‌ترین نقطهٔ تصمیم بازمی‌گردد. در این زیربخش امکان‌های بهتری را در نظر می‌گیریم.

در نظر بگیرید هنگام اعمال عقبگرد ساده در شکل ۶.۱ با ترتیب متغیر ثابت Q, NSW, V, T, SA, WA, NT چه رخ می‌دهد. فرض کنیم تخصیص جزئی {Q=red, NSW=green, V=blue, T=red} را تولید کرده‌ایم. وقتی متغیر بعدی، SA، را می‌آزماییم، می‌بینیم هر مقدار قیدی را نقض می‌کند. به T برمی‌گردیم و رنگ تازه‌ای برای تاسمانی امتحان می‌کنیم! بدیهی است این احمقانه است — رنگ‌کردن دوبارهٔ تاسمانی به‌هیچ‌وجه نمی‌تواند در حل مسئله با استرالیای جنوبی کمک کند.

رویکرد هوشمندانه‌تر، عقبگرد به متغیری است که ممکن است مسئله را اصلاح کند — متغیری که مسئول ناممکن‌کردن یکی از مقادیر ممکن SA بوده. برای این کار، مجموعه‌ای از تخصیص‌ها را پیگیری می‌کنیم که با برخی مقادیر SA در تعارض‌اند. آن مجموعه (در این مورد {Q=red, NSW=green, V=blue}) «مجموعهٔ تعارض» (Conflict Set) برای SA نامیده می‌شود. روش «پرش به عقب» (Backjumping) به تازه‌ترین تخصیص در مجموعهٔ تعارض بازمی‌گردد؛ در این مورد، پرش به عقب از تاسمانی می‌پرد و مقدار تازه‌ای برای V می‌آزماید. این روش به‌راحتی با اصلاحی در BACKTRACK پیاده می‌شود، چنان‌که هنگام بررسی مقدار قانونی برای تخصیص، مجموعهٔ تعارض را انباشته کند. اگر مقدار قانونی‌ای یافت نشود، الگوریتم باید تازه‌ترین عنصر مجموعهٔ تعارض را همراه با نشانهٔ شکست بازگرداند.

خوانندهٔ تیزبین ممکن است متوجه شده باشد بررسی رو به جلو می‌تواند مجموعهٔ تعارض را بدون هیچ کار اضافه‌ای فراهم کند: هرگاه بررسی رو به جلو بر اساس تخصیص X=x مقداری را از دامنهٔ Y حذف کند، باید X=x را به مجموعهٔ تعارض Y بیفزاید. اگر آخرین مقدار از دامنهٔ Y حذف شود، تخصیص‌های مجموعهٔ تعارض Y به مجموعهٔ تعارض X افزوده می‌شوند. یعنی اکنون می‌دانیم X=x به تناقض (در Y) می‌انجامد و بنابراین باید تخصیص متفاوتی برای X آزموده شود.

خوانندهٔ عقاب‌چشم ممکن است چیز عجیبی متوجه شده باشد: پرش به عقب وقتی رخ می‌دهد که هر مقدار در یک دامنه با تخصیص کنونی در تعارض باشد؛ اما بررسی رو به جلو این رویداد را تشخیص می‌دهد و جلوی رسیدن جستجو به چنین گره‌ای را می‌گیرد! در واقع، می‌توان نشان داد هر شاخه‌ای که پرش به عقب هرس می‌کند، بررسی رو به جلو نیز هرس می‌کند. از این رو، پرش به عقب ساده در جستجوی بررسی-رو-به-جلو، یا در جستجویی که بررسی سازگاری قوی‌تری مانند MAC استفاده می‌کند، زائد است — فقط باید یکی از آن‌ها را انجام دهید.

با وجود مشاهده‌های بند پیشین، ایدهٔ پشت پرش به عقب همچنان خوب است: عقبگرد بر اساس دلایل شکست. پرش به عقب وقتی شکست را متوجه می‌شود که دامنهٔ متغیری خالی شود، اما در بسیاری از موارد شاخه مدت‌ها پیش از این محکوم است. تخصیص جزئی {WA=red, NSW=red} را دوباره در نظر بگیرید (که از بحث قبلی‌مان ناسازگار است). فرض کنیم سپس T=red را امتحان کنیم و بعد NT، Q، V، SA را تخصیص دهیم. می‌دانیم هیچ تخصیصی برای این چهار متغیر آخر کار نمی‌کند، پس سرانجام مقادیر قابل امتحان در NT تمام می‌شود. حال، پرسش این است: به کجا عقبگرد کنیم؟ پرش به عقب کار نمی‌کند، زیرا NT مقادیری سازگار با متغیرهای تخصیص‌یافتهٔ قبلی دارد — NT مجموعهٔ تعارض کاملی از متغیرهای قبلی که باعث شکستش شدند ندارد. اما می‌دانیم چهار متغیر NT، Q، V و SA با هم به‌سبب مجموعه‌ای از متغیرهای قبلی شکست خوردند که باید همان متغیرهایی باشند که مستقیماً با آن چهار در تعارض‌اند.

این به مفهوم متفاوت — و عمیق‌تر — از مجموعهٔ تعارض برای متغیری مانند NT می‌انجامد: مجموعهٔ متغیرهای قبلی که باعث شدند NT، همراه با هر متغیر بعدی، راه‌حل سازگاری نداشته باشد. در این مورد، مجموعه {WA, NSW} است، پس الگوریتم باید به NSW عقبگرد کند و از تاسمانی بپرد. الگوریتم پرش به عقبی که از مجموعه‌های تعارضِ چنین‌تعریف‌شده استفاده می‌کند، «پرش به عقب معطوف-به-تعارض» (Conflict-Directed Backjumping) نامیده می‌شود.

حال باید توضیح دهیم این مجموعه‌های تعارض تازه چگونه محاسبه می‌شوند. روش در واقع بسیار ساده است. شکست «پایانی» شاخه‌ای از جستجو همیشه به‌سبب خالی‌شدن دامنهٔ متغیری رخ می‌دهد؛ آن متغیر مجموعهٔ تعارض استانداردی دارد. در مثال ما، SA شکست می‌خورد و مجموعهٔ تعارضش (مثلاً) {WA, NT, Q} است. به Q می‌پریم و Q مجموعهٔ تعارض SA (منهای خودِ Q، البته) را در مجموعهٔ تعارض مستقیم خودش که {NT, NSW} است جذب می‌کند؛ مجموعهٔ تعارض تازه {WA, NT, NSW} است. یعنی با تخصیص قبلی به {WA, NT, NSW}، از Q به بعد راه‌حلی نیست. پس به NT عقبگرد می‌کنیم، تازه‌ترینِ این‌ها. NT مجموعهٔ {WA, NT, NSW} - {NT} را در مجموعهٔ تعارض مستقیم خودش {WA} جذب می‌کند و {WA, NSW} می‌دهد (چنان‌که در بند پیشین گفته شد). حال الگوریتم به NSW می‌پرد، چنان‌که امیدوار بودیم. برای جمع‌بندی: فرض کنید Xj متغیر کنونی و conf(Xj) مجموعهٔ تعارضش باشد. اگر هر مقدار ممکن برای Xj شکست بخورد، به تازه‌ترین متغیر Xi در conf(Xj) بپر و مجموعهٔ تعارض Xi را چنین بازمحاسبه کن:

```
conf(Xi) ← conf(Xi) ∪ conf(Xj) − {Xi}
```

۶.۳.۴ یادگیری قید

وقتی به تناقض می‌رسیم، پرش به عقب می‌تواند بگوید چقدر عقب برویم، تا زمان را برای تغییر متغیرهایی که مسئله را اصلاح نمی‌کنند تلف نکنیم. اما همچنین می‌خواهیم از برخورد با همان مسئله دوباره بپرهیزیم. وقتی جستجو به تناقض می‌رسد، می‌دانیم زیرمجموعه‌ای از مجموعهٔ تعارض مسئول مشکل است. «یادگیری قید» (Constraint Learning) ایدهٔ یافتن مجموعهٔ کمینه‌ای از متغیرها از مجموعهٔ تعارض است که مشکل را ایجاد می‌کند. این مجموعهٔ متغیرها، همراه با مقادیر متناظرشان، «ناخوب» (No-Good) نامیده می‌شود. سپس ناخوب را ثبت می‌کنیم، یا با افزودن قیدی تازه به CSP برای منع این ترکیب از تخصیص‌ها، یا با نگه‌داشتن کش جداگانه‌ای از ناخوب‌ها.

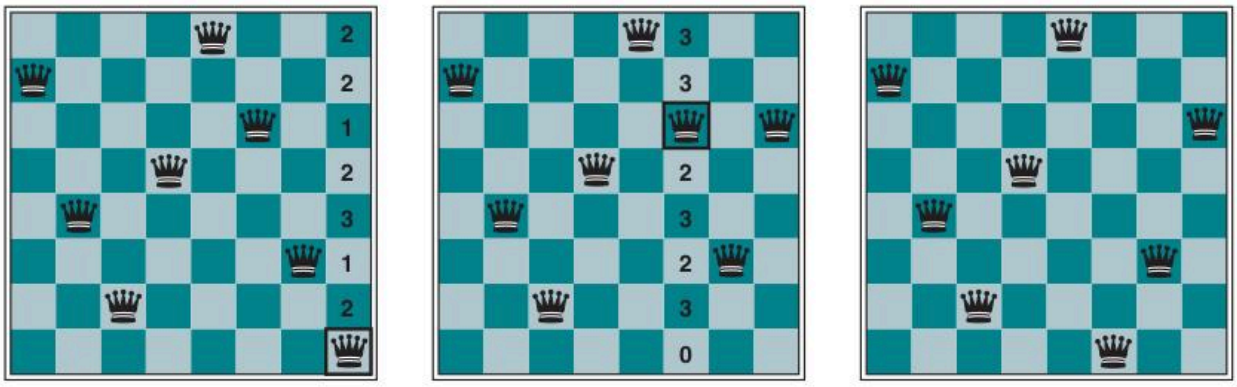
برای نمونه، حالت {WA = red, NT = green, Q = blue} را در ردیف پایین شکل ۶.۶ در نظر بگیرید. بررسی رو به جلو می‌تواند بگوید این حالت ناخوب است، زیرا هیچ تخصیص معتبری برای SA نیست. در این مورد خاص، ثبت ناخوب کمکی نمی‌کند، زیرا پس از هرس این شاخه از درخت جستجو، هرگز دوباره با این ترکیب مواجه نخواهیم شد. اما فرض کنیم درخت جستجوی شکل ۶.۶ در واقع بخشی از درخت جستجوی بزرگ‌تری بود که با تخصیص نخست مقادیر V و T آغاز می‌شد. آنگاه ثبت {WA = red, NT = green, Q = blue} به‌عنوان ناخوب ارزشمند می‌بود، زیرا برای هر مجموعه‌ی تخصیص به V و T دوباره با همان مسئله مواجه خواهیم شد.

ناخوب‌ها را می‌توان به‌طور مؤثر توسط بررسی رو به جلو یا پرش به عقب استفاده کرد. یادگیری قید یکی از مهم‌ترین فنونی است که حل‌کننده‌های CSP مدرن برای دستیابی به کارایی در مسائل پیچیده استفاده می‌کنند.

۶.۴ جستجوی محلی برای CSPها

الگوریتم‌های جستجوی محلی (بخش 4.1 را ببینید) در حل بسیاری از CSPها بسیار مؤثر از آب درمی‌آیند. آن‌ها از صورت‌بندی حالت-کامل (چنان‌که در بخش 4.1.1 معرفی شد) استفاده می‌کنند که در آن هر حالت به هر متغیر مقداری تخصیص می‌دهد و جستجو مقدار یک متغیر را در هر بار تغییر می‌دهد. به‌عنوان مثال، از مسئلهٔ هشت‌وزیر استفاده می‌کنیم، چنان‌که در صفحهٔ ۱۸۳ به‌صورت CSP تعریف شد. در شکل ۶.۸ از چپ با تخصیص کاملی به ۸ متغیر آغاز می‌کنیم؛ معمولاً این چند قید را نقض می‌کند. سپس به‌صورت تصادفی متغیر درتعارضی را برمی‌گزینیم که معلوم می‌شود Q8 است، راست‌ترین ستون. می‌خواهیم مقدار را به چیزی تغییر دهیم که ما را به راه‌حل نزدیک‌تر کند؛ بدیهی‌ترین رویکرد، انتخاب مقداری است که به کمترین شمار تعارض با متغیرهای دیگر می‌انجامد — اکتشافی «کمترین-تعارض» (Min-Conflicts).

در شکل می‌بینیم دو سطر هست که فقط یک قید را نقض می‌کنند؛ Q8 = 3 را برمی‌گزینیم (یعنی وزیر را به ستون ۸، سطر ۳ می‌بریم). در تکرار بعدی، در تختهٔ میانی شکل، Q6 را به‌عنوان متغیر تغییر انتخاب می‌کنیم و توجه می‌کنیم بردن وزیر به سطر ۸ به هیچ تعارضی نمی‌انجامد. در این نقطه دیگر متغیر درتعارضی نیست، پس راه‌حل داریم. الگوریتم در شکل ۶.۹ نشان داده شده.



شکل ۶.۸ — راه‌حل دومرحله‌ای با کمترین-تعارض برای مسئلهٔ هشت‌وزیر. در هر مرحله، وزیری برای تخصیص دوباره در ستونش برگزیده می‌شود. شمار تعارض‌ها (در این مورد، شمار وزیرهای حمله‌کننده) در هر خانه نشان داده شده. الگوریتم وزیر را به خانهٔ کمترین-تعارض می‌برد و تساوی‌ها را تصادفی می‌شکند.

```
function MIN-CONFLICTS(csp, max steps) returns a solution or failure
    inputs: csp, a constraint satisfaction problem
        max steps, the maximum number of steps before giving up
    current ← an initial complete assignment for csp
    for i = 1 to max steps do
        if current is a solution for csp then return current
        var ← a randomly selected conflicted variable from csp.VARIABLES
        value ← a value v for var that minimizes CONFLICTS(csp, var, v, current)
        set var = value in current
    return failure
```

شکل ۶.۹ — الگوریتم جستجوی محلی MIN-CONFLICTS برای CSPها. حالت آغازین را می‌توان تصادفی یا با فرایند تخصیص حریصانه‌ای که برای هر متغیر به‌نوبت مقدار کمترین-تعارض را برمی‌گزیند انتخاب کرد. تابع CONFLICTS شمار قیدهای نقض‌شده توسط مقدارِ خاص را با توجه به بقیهٔ تخصیص کنونی می‌شمارد.

کمترین-تعارض برای بسیاری از CSPها به‌شکل شگفت‌آوری مؤثر است. به‌طرز حیرت‌آوری، در مسئلهٔ n-وزیر، اگر جای‌گذاری آغازین وزیرها را نشمارید، زمان اجرای کمترین-تعارض تقریباً مستقل از اندازهٔ مسئله است. حتی مسئلهٔ میلیون-وزیر را به‌طور میانگین در ۵۰ گام حل می‌کند (پس از تخصیص آغازین). این مشاهدهٔ چشمگیر محرک پژوهش‌های فراوانی در دههٔ ۱۹۹۰ دربارهٔ جستجوی محلی و تمایز میان مسائل آسان و سخت شد، که در بخش 7.6.3 به آن می‌پردازیم. به بیان تقریبی، n-وزیر برای جستجوی محلی آسان است، زیرا راه‌حل‌ها به‌صورت چگال در سراسر فضای حالت توزیع شده‌اند. کمترین-تعارض همچنین برای مسائل سخت خوب کار می‌کند. برای نمونه، برای زمان‌بندی مشاهده‌های تلسکوپ فضایی هابل استفاده شده و زمان زمان‌بندی یک هفته مشاهده را از سه هفته (!) به حدود ۱۰ دقیقه کاهش داده است.

همهٔ فنون جستجوی محلی از بخش 4.1 نامزد کاربرد بر CSPها هستند و برخی از آن‌ها به‌ویژه مؤثر ثابت شده‌اند. چشم‌انداز یک CSP تحت اکتشافی کمترین-تعارض معمولاً سلسله‌ای از فلات‌ها دارد. ممکن است میلیون‌ها تخصیص متغیر وجود داشته باشد که فقط یک تعارض با راه‌حل فاصله دارند. «جستجوی فلات» — اجازهٔ حرکت‌های کناری به حالت دیگری با همان امتیاز — می‌تواند به جستجوی محلی کمک کند راهش را از فلات پیدا کند. این سرگردانی در فلات

را می‌توان با فنونی به نام «جستجوی تابو» (Tabu Search) هدایت کرد: نگه‌داشتن فهرست کوچکی از حالت‌های اخیراً بازدیدشده و منع بازگشت الگوریتم به آن حالت‌ها. بازپخت شبیه‌سازی‌شده نیز می‌تواند برای گریز از فلات‌ها استفاده شود.

فن دیگر به نام «وزن‌دهی قید» (Constraint Weighting) می‌کوشد جستجو را بر قیدهای مهم متمرکز کند. به هر قید وزنی عددی داده می‌شود، در آغاز همه ۱. در هر گام جستجو، الگوریتم جفت متغیر/مقداری را برای تغییر برمی‌گزیند که به کمترین وزن کل همهٔ قیدهای نقض‌شده می‌انجامد. سپس وزن‌ها با افزایش وزن هر قیدی که توسط تخصیص کنونی نقض شده تنظیم می‌شوند. این دو مزیت دارد: به فلات‌ها توپوگرافی می‌افزاید و مطمئن می‌شود بهبود از حالت کنونی ممکن است، و نیز یادگیری می‌افزاید: با گذشت زمان به قیدهای دشوار وزن‌های بالاتری تخصیص داده می‌شود.

مزیت دیگر جستجوی محلی آن است که می‌توان آن را در محیط آنلاین (بخش 4.5 را ببینید) وقتی مسئله تغییر می‌کند استفاده کرد. مسئلهٔ زمان‌بندی پروازهای هفتگی خط هوایی را در نظر بگیرید. زمان‌بندی ممکن است شامل هزاران پرواز و ده‌ها هزار تخصیص پرسنل باشد، اما آب‌وهوای بد در یک فرودگاه می‌تواند زمان‌بندی را ناشدنی کند. می‌خواهیم زمان‌بندی را با کمترین شمار تغییرات تعمیر کنیم. این را می‌توان به‌راحتی با الگوریتم جستجوی محلیِ شروع‌شده از زمان‌بندی کنونی انجام داد. جستجوی عقبگرد با مجموعه‌قیدهای تازه معمولاً زمان بسیار بیشتری نیاز دارد و ممکن است راه‌حلی با تغییرات فراوان از زمان‌بندی کنونی بیابد.

۶.۵ ساختار مسائل

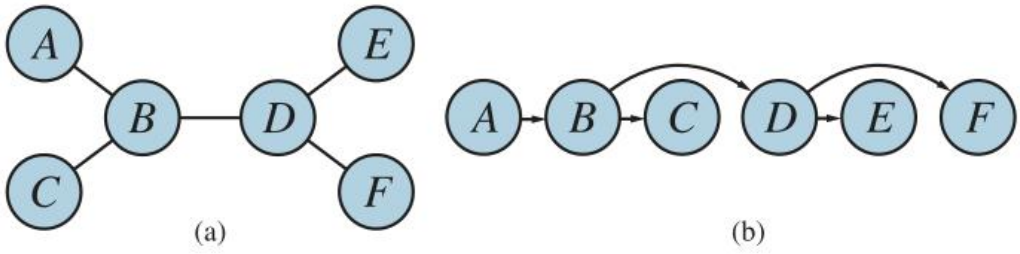
در این بخش، راه‌هایی را بررسی می‌کنیم که ساختار مسئله، چنان‌که توسط گراف قید بازنمایی شده، می‌تواند برای یافتن سریع راه‌حل‌ها استفاده شود. بیشتر رویکردهای این‌جا بر مسائل دیگر نیز اعمال می‌شوند، نه فقط CSPها، مانند استدلال احتمالاتی.

تنها راهی که می‌توانیم امیدوار باشیم با جهان واقعیِ وسیع سروکار داشته باشیم، تجزیهٔ آن به زیرمسئله‌هاست. بار دیگر به گراف قید استرالیا نگاه کنیم (شکل ۶.۱(ب))، که در شکل ۶.۱۲(الف) تکرار شده)؛ یک واقعیت برجسته است: تاسمانی به سرزمین اصلی متصل نیست.[3] به‌طور شهودی، بدیهی است رنگ‌آمیزی تاسمانی و رنگ‌آمیزی سرزمین اصلی زیرمسئله‌های مستقلی هستند — هر راه‌حلی برای سرزمین اصلی که با هر راه‌حلی برای تاسمانی ترکیب شود، راه‌حلی برای کل نقشه می‌دهد.

استقلال را می‌توان صرفاً با یافتن «مؤلفه‌های همبندی» (Connected Components) گراف قید ascertain کرد. هر مؤلفه با زیرمسئلهٔ CSPi متناظر است. اگر تخصیص Si راه‌حلی برای CSPi باشد، آنگاه اجتماع همهٔ Si راه‌حلی برای اجتماع همهٔ CSPi است. چرا این مهم است؟ فرض کنیم هر CSPi دارای c متغیر از مجموع n متغیر باشد، که c ثابت است. آنگاه n/c زیرمسئله هست که هر یک حداکثر d^c کار برای حل می‌برد، که d اندازهٔ دامنه است. از این رو، کل کار $O(d^c n/c)$ است که در n خطی است؛ بدون تجزیه، کل کار $O(d^n)$ است که در n نمایی است. این را عینی‌تر کنیم: تقسیم CSP بولی با ۱۰۰ متغیر به چهار زیرمسئله، بدترین زمان حل را از عمر جهان به کمتر از یک ثانیه کاهش می‌دهد.

زیرمسئله‌های کاملاً مستقل پس خوشمزه‌اند، اما نادر. خوشبختانه، برخی ساختارهای گرافی دیگر نیز آسان‌اند. برای نمونه، گراف قید «درخت» (Tree) است وقتی هر دو متغیر فقط با یک مسیر به هم متصل‌اند. نشان خواهیم داد هر CSP درخت‌ساختار ◭ را می‌توان در زمانی خطی نسبت به شمار متغیرها حل کرد.[4] کلید، مفهوم تازه‌ای از سازگاری به نام «سازگاری کمانی جهتی» (Directional Arc Consistency، یا DAC) است. CSP تحت ترتیبی از متغیرهای X1, X2, ..., Xn «سازگار-کمانی جهتی» تعریف می‌شود اگر و فقط اگر هر Xi نسبت به هر Xj برای i < j سازگار-کمانی باشد.

برای حل CSP درخت‌ساختار، نخست هر متغیری را به‌عنوان ریشهٔ درخت برگزین و ترتیبی از متغیرها انتخاب کن چنان‌که هر متغیر پس از والدش در درخت بیاید. چنین ترتیبی «مرتب‌سازی توپولوژیک» (Topological Sort) نامیده می‌شود. شکل ۶.۱۰(الف) درخت نمونه‌ای و (ب) یکی از ترتیب‌های ممکن را نشان می‌دهد. هر درخت با n گره n-1 یال دارد، پس می‌توانیم این گراف را در O(n) گام سازگار-کمانی جهتی کنیم که هر یک باید تا d مقدار دامنهٔ ممکن را برای دو متغیر مقایسه کند، برای کل زمان $O(nd^2)$. پس از آنکه گراف سازگار-کمانی جهتی داشتیم، می‌توانیم به‌سادگی در فهرست متغیرها پایین برویم و هر مقدار باقی‌مانده را برگزینیم. چون هر یال از والد به فرزندش سازگار-کمانی است، می‌دانیم برای هر مقداری که برای والد برمی‌گزینیم، مقدار معتبری برای انتخابِ فرزند باقی خواهد ماند. این بدان معناست عقبگرد نخواهیم کرد؛ می‌توانیم به‌صورت خطی در متغیرها حرکت کنیم. الگوریتم کامل در شکل ۶.۱۱ نشان داده شده.



شکل ۶.۱۰ — (الف) گراف قید یک CSP درخت‌ساختار. (ب) ترتیب خطی متغیرها سازگار با درخت با A به‌عنوان ریشه. این به‌عنوان مرتب‌سازی توپولوژیک متغیرها شناخته می‌شود.

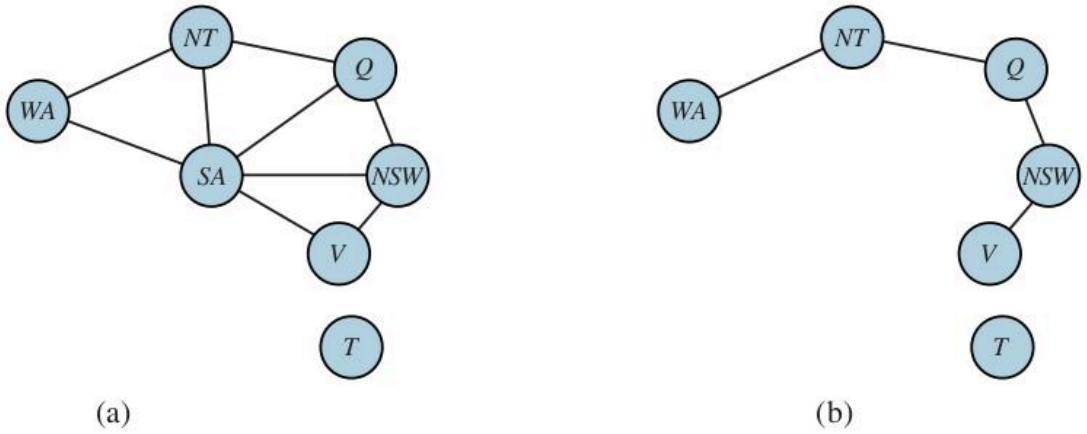
```
function TREE-CSP-SOLVER(csp) returns a solution, or failure
    inputs: csp, a CSP with components X, D, C
    n ← number of variables in X
    assignment ← an empty assignment
    root ← any variable in X
    X ← TOPOLOGICAL-SORT(X, root)
    for j = n down to 2 do
        MAKE-ARC-CONSISTENT(PARENT(Xj), Xj)
        if it cannot be made consistent then return failure
    for i = 1 to n do
        assignment[Xi] ← any consistent value from Di
        if there is no consistent value then return failure
    return assignment
```

شکل ۶.۱۱ — الگوریتم TREE-CSP-SOLVER برای حل CSPهای درخت‌ساختار. اگر CSP راه‌حلی داشته باشد، آن را در زمان خطی خواهیم یافت؛ اگر نه، تناقضی تشخیص خواهیم داد.

حال که الگوریتم کارآمدی برای درخت‌ها داریم، می‌توانیم در نظر بگیریم آیا گراف‌های قید عمومی‌تر را می‌توان به‌نحوی به درخت تقلیل داد. دو راه برای این کار هست: با حذف گره‌ها (بخش ۶.۵.۱) یا با فروریختن گره‌ها در هم (بخش ۶.۵.۲).

۶.۵.۱ شرطی‌سازی مجموعه‌برش

نخستین راه تقلیل گراف قید به درخت، تخصیص مقادیر به برخی متغیرهاست تا متغیرهای باقی‌مانده درختی بسازند. گراف قید استرالیا را در نظر بگیرید، که دوباره در شکل ۶.۱۲(الف) نشان داده شده. بدون استرالیای جنوبی، گراف درختی می‌شد، مانند (ب). خوشبختانه، می‌توانیم استرالیای جنوبی را (در گراف، نه کشور) با تثبیت مقداری برای SA و حذف هر مقداری که با مقدارِ برگزیده‌شده برای SA ناسازگار است از دامنه‌های متغیرهای دیگر حذف کنیم.



شکل ۶.۱۲ – (الف) گراف قید اصلی از شکل ۶.۱. (ب) پس از حذف SA، گراف قید جنگلی از دو درخت می‌شود.

حال، هر راه‌حلی برای CSP پس از حذف SA و قیدهایش با مقدارِ برگزیده‌شده برای SA سازگار خواهد بود. (این برای CSPهای دوتایی کار می‌کند؛ وضعیت با قیدهای مرتبه‌بالاتر پیچیده‌تر است.) پس می‌توانیم درخت باقی‌مانده را با الگوریتم داده‌شده در بالا حل کنیم و بدین ترتیب کل مسئله را حل کنیم. البته، در حالت کلی (در برابر رنگ‌آمیزی نقشه)، مقدارِ برگزیده‌شده برای SA می‌تواند اشتباه باشد، پس باید هر مقدار ممکن را امتحان کنیم. الگوریتم عمومی چنین است:

1. زیرمجموعهٔ S از متغیرهای CSP برگزین چنان‌که گراف قید پس از حذف S درختی شود. S «مجموعهٔ برش چرخه» (Cycle Cutset) نامیده می‌شود.
2. برای هر تخصیص ممکن به متغیرهای S که همهٔ قیدهای روی S را برآورده می‌کند:

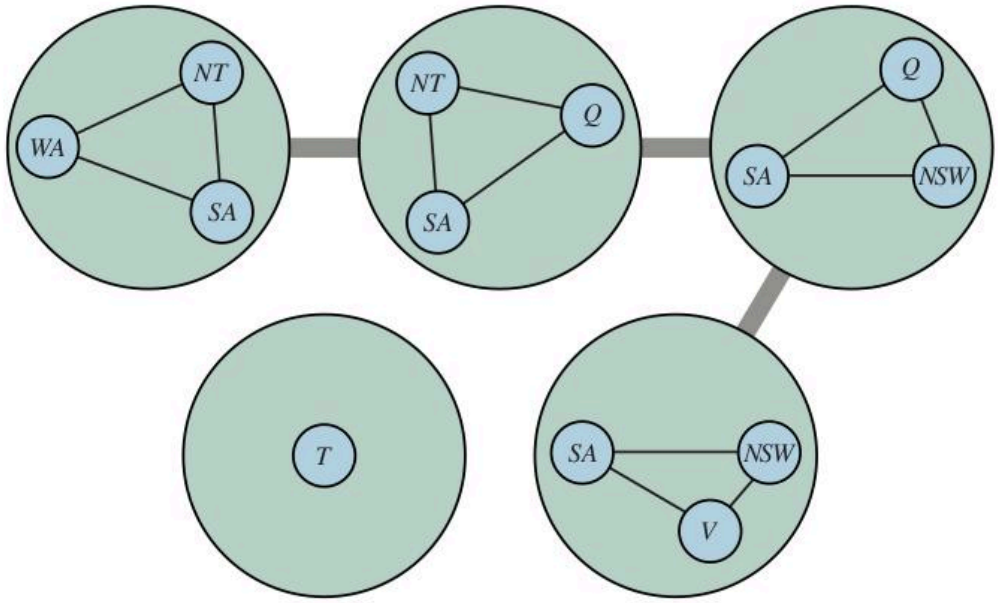
(الف) هر مقداری را که با تخصیص برای S ناسازگار است از دامنه‌های متغیرهای باقی‌مانده حذف کن، و (ب) اگر CSP باقی‌مانده راه‌حلی دارد، آن را همراه با تخصیص برای S بازگردان.

اگر مجموعهٔ برش چرخه اندازهٔ c داشته باشد، زمان کل اجرا $O(d^{c}\cdot(n-c)d^{2})$ است: باید هر یک از d^{c} ترکیب مقادیر را برای متغیرهای S امتحان کنیم و برای هر ترکیب باید مسئلهٔ درختی به اندازهٔ n-c حل کنیم. اگر گراف «تقریباً درخت» باشد، c کوچک خواهد بود و صرفه‌جویی نسبت به عقبگرد ساده عظیم خواهد بود — برای مثال ۱۰۰ متغیر بولی‌مان، اگر می‌توانستیم مجموعه‌برشی به اندازهٔ c=20 بیابیم، زمان از عمر جهان به چند دقیقه کاهش می‌یافت. در بدترین حالت اما c می‌تواند تا (n-2) بزرگ شود. یافتن کوچک‌ترین مجموعهٔ برش چرخه NP-سخت است، اما الگوریتم‌های تقریب کارآمدی شناخته شده‌اند. رویکرد الگوریتمی کلی «شرطی‌سازی مجموعه‌برش» (Cutset Conditioning) نامیده می‌شود؛ در فصل ۱۳ دوباره می‌آید، جایی که برای استدلال دربارهٔ احتمال‌ها استفاده می‌شود.

۶.۵.۲ تجزیهٔ درختی

دومین راه تقلیل گراف قید به درخت بر ساختن «تجزیهٔ درختی» (Tree Decomposition) گراف قید مبتنی است: تبدیلی از گراف اصلی به درختی که هر گرهٔ درخت از مجموعه‌ای از متغیرها تشکیل شده، چنان‌که در شکل ۶.۱۳. تجزیهٔ درختی باید این سه الزام را برآورده کند:

- هر متغیر در مسئلهٔ اصلی دست‌کم در یکی از گره‌های درخت ظاهر شود.
- اگر دو متغیر در مسئلهٔ اصلی با قیدی به هم متصل‌اند، باید با هم (همراه با قید) دست‌کم در یکی از گره‌های درخت ظاهر شوند.
- اگر متغیری در دو گرهٔ درخت ظاهر شود، باید در هر گره‌ای در طول مسیر متصل‌کنندهٔ آن گره‌ها نیز ظاهر شود.



شکل ۶.۱۳ — تجزیهٔ درختی گراف قید در شکل ۶.۱۲(الف).

دو شرط نخست تضمین می‌کنند همهٔ متغیرها و قیدها در تجزیهٔ درختی بازنمایی شده‌اند. شرط سوم نسبتاً فنی به نظر می‌رسد، اما به ما امکان می‌دهد بگوییم هر متغیری از مسئلهٔ اصلی باید هر جا ظاهر می‌شود همان مقدار را داشته باشد: قیدهای درخت می‌گویند متغیری در یک گرهٔ درخت باید همان مقدار متغیر متناظر در گرهٔ مجاور درخت را

داشته باشد. برای نمونه، SA در هر چهار گرهٔ متصل در شکل ۶.۱۳ ظاهر می‌شود، پس هر یال در تجزیهٔ درختی شامل این قید است که مقدار SA در یک گره باید همان مقدار SA در گرهٔ بعدی باشد. می‌توانید از شکل ۶.۱۲ راستی‌آزمایی کنید که این تجزیه معنا دارد.

پس از داشتن گراف درخت‌ساختار، می‌توانیم TREE-CSP-SOLVER را برای گرفتن راه‌حل در زمان $O(nd^2)$ اعمال کنیم، که n شمار گره‌های درخت و d اندازهٔ بزرگ‌ترین دامنه است. اما توجه کنید در درخت، دامنه مجموعه‌ای از چندتایی‌های مقادیر است، نه فقط مقادیر فردی. برای نمونه، گرهٔ بالا-چپ شکل ۶.۱۳ در سطح مسئلهٔ اصلی، زیرمسئله‌ای با متغیرهای {WA, NT, SA}، دامنهٔ {red, green, blue} و قیدهای WA≠NT, SA≠NT, WA≠SA را بازنمایی می‌کند. در سطح درخت، گره یک متغیر تکی را بازنمایی می‌کند که می‌توانیم SANTWA بنامیمش و مقدارش باید سه‌تایی از رنگ‌ها باشد، مانند (red, green, blue)، اما نه (red, red, blue)، زیرا آن قید SA≠NT از مسئلهٔ اصلی را نقض می‌کند. سپس می‌توانیم از آن گره به گرهٔ مجاور برویم، با متغیری که می‌توانیم SANTQ بنامیمش، و بیابیم فقط یک سه‌تایی، (red, green, blue)، هست که با انتخاب برای SANTWA سازگار است. دقیقاً همان فرایند برای دو گرهٔ بعدی تکرار می‌شود و به‌طور مستقل می‌توانیم هر انتخابی برای T بکنیم.

هر مسئلهٔ تجزیهٔ درختی را می‌توان در زمان $O(nd^2)$ با TREE-CSP-SOLVER حل کرد، که تا وقتی d کوچک بماند کارآمد خواهد بود. به مثال ۱۰۰ متغیر بولی‌مان برگردیم، اگر هر گره ۱۰ متغیر داشته باشد، $d = 2^{10}$ است و باید بتوانیم مسئله را در چند ثانیه حل کنیم. اما اگر گره‌ای با ۳۰ متغیر باشد، قرن‌ها طول می‌کشد.

گرافِ داده‌شده تجزیه‌های درختی بسیاری می‌پذیرد؛ در انتخاب تجزیه، هدف کوچک‌کردن هرچه بیشتر زیرمسئله‌هاست. (گذاشتن همهٔ متغیرها در یک گره از نظر فنی درخت است، اما کمکی نمی‌کند.) «عرض درختی» (Tree Width) تجزیهٔ درختی یک گراف، یک کمتر از اندازهٔ بزرگ‌ترین گره است؛ عرض درختی خودِ گراف به‌صورت کمینهٔ عرض میان همهٔ تجزیه‌های درختی‌اش تعریف می‌شود. اگر گرافی عرض درختی w داشته باشد، مسئله را می‌توان در زمان $O(nd^{(w+1)})$ با داشتن تجزیهٔ درختی متناظر حل کرد. از این رو، CSPهای با گراف‌های قیدِ عرض درختی کران‌دار در زمان چندجمله‌ای حل‌شدنی‌اند. ⚠

متأسفانه، یافتن تجزیه با کمینهٔ عرض درختی NP-سخت است، اما روش‌های اکتشافی هست که در عمل خوب کار می‌کنند. کدام بهتر است: تجزیهٔ مجموعه‌برش با زمان $O(d^{c \cdot (n-c)} d^2)$، یا تجزیهٔ درختی با زمان $O(nd^{(w+1)})$؟ هرگاه مجموعه‌برش چرخه‌ای به اندازهٔ c داشته باشید، عرض درختی به اندازهٔ $w < c + 1$ نیز هست و در برخی موارد ممکن است بسیار کوچک‌تر باشد. پس ملاحظهٔ زمان به نفع تجزیهٔ درختی است، اما مزیت رویکرد مجموعهٔ برش چرخه آن است که می‌توان آن را با حافظهٔ خطی اجرا کرد، در حالی که تجزیهٔ درختی به حافظه‌ای نمایی نسبت به w نیاز دارد.

۶.۵.۳ تقارن مقدار

تاکنون به ساختار گراف قید نگاه کرده‌ایم. همچنین می‌تواند ساختار مهمی در مقادیر متغیرها، یا در ساختار خودِ روابط قید باشد. مسئلهٔ رنگ‌آمیزی نقشه با d رنگ را در نظر بگیرید. برای هر راه‌حل سازگار، در واقع مجموعه‌ای از d! راه‌حل هست که با جای‌گذاری نام رنگ‌ها تشکیل می‌شود. برای نمونه، در نقشهٔ استرالیا می‌دانیم WA، NT و SA همگی باید رنگ‌های متفاوتی داشته باشند، اما 3! = ۶ راه برای تخصیص سه رنگ به سه ناحیه هست. این «تقارن مقدار» (Value

Symmetry) نامیده می‌شود. می‌خواهیم فضای جستجو را با ضریب d! با شکستن تقارن در تخصیص‌ها کاهش دهیم. این کار را با معرفی «قید شکستن تقارن» (Symmetry-Breaking Constraint) انجام می‌دهیم. برای مثال ما، ممکن است قید ترتیب‌دهی دلخواهی تحمیل کنیم، NT < SA < WA، که ایجاب می‌کند سه مقدار به‌ترتیب الفبایی باشند. این قید تضمین می‌کند فقط یکی از d! راه‌حل‌ها ممکن است: {NT = blue, SA = green, WA = red}.

برای رنگ‌آمیزی نقشه، یافتن قیدی که تقارن را حذف کند آسان بود. در حالت کلی حذف همهٔ تقارن NP-سخت است، اما شکستن تقارن مقدار در گسترهٔ وسیعی از مسائل مهم و مؤثر ثابت شده است.

خلاصهٔ فصل

- مسئله‌های ارضای قیود (CSPها) حالتی را با مجموعه‌ای از جفت‌های متغیر/مقدار بازنمایی می‌کنند و شرایط راه‌حل را با مجموعه‌ای از قیدها روی متغیرها بازنمایی می‌کنند. بسیاری از مسائل مهم جهان واقعی را می‌توان به‌صورت CSP توصیف کرد.
- شمار زیادی فنون استنتاج از قیدها برای منتفی‌کردن برخی تخصیص‌های متغیر استفاده می‌کنند. این‌ها شامل سازگاری گره‌ای، کمانی، مسیری و k-تایی‌اند.
- جستجوی عقبگرد، شکلی از جستجوی عمق-اول، معمولاً برای حل CSPها استفاده می‌شود. استنتاج را می‌توان با جستجو در هم بافت.
- اکتشافی‌های کمترین-مقادیر-باقی‌مانده و درجه، روش‌های مستقل از حوزه برای تصمیم‌گیری دربارهٔ اینکه کدام متغیر را بعدی در جستجوی عقبگرد برگزینیم هستند. اکتشافی کمترین-قید-محدودکننده در تصمیم‌گیری دربارهٔ اینکه کدام مقدار را نخست برای متغیرِ داده‌شده امتحان کنیم کمک می‌کند. عقبگرد وقتی رخ می‌دهد که برای متغیری هیچ تخصیص قانونی یافت نشود. پرش به عقب معطوف-به-تعارض مستقیماً به منشأ مسئله عقبگرد می‌کند. یادگیری قید، تعارض‌ها را هنگام مواجهه در طول جستجو ثبت می‌کند تا از همان تعارض بعداً در جستجو پرهیز شود.
- جستجوی محلی با اکتشافی کمترین-تعارض نیز با موفقیت بزرگ بر مسئله‌های ارضای قیود اعمال شده.
- پیچیدگی حل یک CSP به‌شدت با ساختار گراف قیدش مرتبط است. مسائل درخت‌ساختار را می‌توان در زمان خطی حل کرد. شرطی‌سازی مجموعه‌برش می‌تواند CSP عمومی را به CSP درخت‌ساختار کاهش دهد و اگر مجموعه‌برش کوچکی یافت شود کاملاً کارآمد است (فقط به حافظهٔ خطی نیاز دارد). فنون تجزیهٔ درختی CSP را به درختی از زیرمسئله‌ها تبدیل می‌کنند و اگر عرض درختی گراف قید کوچک باشد کارآمدند؛ اما به حافظه‌ای نمایی نسبت به عرض درختی گراف قید نیاز دارند. ترکیب شرطی‌سازی مجموعه‌برش با تجزیهٔ درختی می‌تواند دادوستد بهتری میان حافظه و زمان ممکن کند.

یادداشت‌های کتاب‌شناختی و تاریخی

ریاضی‌دان یونانی دیوفانتوس (حدود ۲۰۰–۲۸۴) مسائل درگیر با قیدهای جبری روی معادلات را ارائه و حل کرد، هرچند روش‌شناسی تعمیم‌یافته‌ای توسعه نداد. اکنون معادلات روی دامنه‌های صحیح را «معادلات دیوفانتینی» (Diophantine Equations) می‌نامیم. ریاضی‌دان هندی براهماگوپتا (حدود ۶۵۰) نخستین کسی بود که راه‌حل عمومی معادلهٔ $ax+by = c$ را بر دامنهٔ اعداد صحیح نشان داد. روش‌های نظام‌مند حل معادلات خطی با حذف متغیر توسط گاوس (۱۸۲۹) مطالعه شد؛ حل قیدهای نابرابری خطی به فوریه (۱۸۲۷) بازمی‌گردد.

مسئله‌های ارضای قیود دامنهٔ متناهی نیز تاریخچهٔ بلندی دارند. برای نمونه، رنگ‌آمیزی گراف (که رنگ‌آمیزی نقشه حالت خاصی از آن است) مسئلهٔ کهنی در ریاضیات است. حدس چهاررنگ (که هر گراف مسطح را می‌توان با چهار رنگ یا کمتر رنگ کرد) نخستین بار توسط فرانسیس گاتری، دانشجوی د مورگان، در ۱۸۵۲ مطرح شد. با وجود چند ادعای منتشرشدهٔ خلاف، تا آنکه اپل و هکن (۱۹۷۷) برهانی تدوین کردند در برابر حل مقاومت کرد (کتاب «چهار رنگ کافی است» (ویلسون، ۲۰۰۴) را ببینید). خالص‌گرایان ناامید شدند که بخشی از برهان به رایانه تکیه دارد، پس ژرژ گونتیه (۲۰۰۸) با استفاده از اثبات‌کنندهٔ قضیهٔ COQ برهان صوری‌ای مشتق کرد که برنامهٔ برهان اپل و هکن درست است.

رده‌های خاصی از مسئله‌های ارضای قیود در سراسر تاریخ علوم رایانه رخ می‌دهند. یکی از تأثیرگذارترین مثال‌های آغازین SKETCHPAD (سادرلند، ۱۹۶۳) بود که قیدهای هندسی در نمودارها را حل می‌کرد و پیش‌آهنگ برنامه‌های طراحی مدرن و ابزارهای CAD بود. شناسایی CSPها به‌عنوان رده‌ای عمومی به اوگو مونتاناری (۱۹۷۴) منسوب است. تقلیل CSPهای مرتبه‌بالاتر به CSPهای صرفاً دوتایی با متغیرهای کمکی (تمرین NARY.6 را ببینید) در اصل به منطق‌دان سدهٔ نوزدهم، چارلز سندرز پیرس، بازمی‌گردد. توسط دچتر (۱۹۹۰) به ادبیات CSP معرفی و توسط باکوس و ون بیک (۱۹۹۸) بسط داده شد. CSPهای با ترجیح میان راه‌حل‌ها به‌طور گسترده در ادبیات بهینه‌سازی مطالعه می‌شوند؛ بیستارلی و همکاران (۱۹۹۷) را برای تعمیمی از چارچوب CSP که امکان ترجیح‌ها را می‌دهد ببینید.

روش‌های انتشار قید با موفقیت والتز (۱۹۷۵) در مسائل برچسب‌گذاری خطوط چندوجهی برای بینایی رایانه رایج شد. والتز نشان داد در بسیاری از مسائل، انتشار کاملاً نیاز به عقبگرد را حذف می‌کند. مونتاناری (۱۹۷۴) مفهوم گراف‌های قید و انتشار با سازگاری مسیر را معرفی کرد. آلن مک‌ورث (۱۹۷۷) الگوریتم AC-3 را برای اعمال سازگاری کمانی و نیز ایدهٔ کلی ترکیب عقبگرد با درجه‌ای از اعمال سازگاری پیشنهاد کرد. AC-4، الگوریتم سازگاری کمانی کارآمدتری که توسط مور و هندرسون (۱۹۸۶) توسعه یافت، در بدترین حالت $O(cd^2)$ اجرا می‌شود اما در موارد میانگین می‌تواند از AC-3 کندتر باشد. الگوریتم PC-2 (مک‌ورث، ۱۹۷۷) سازگاری مسیر را دقیقاً همان‌گونه به دست می‌آورد که AC-3 سازگاری کمانی را.

اندکی پس از انتشار مقالهٔ مک‌ورث، پژوهشگران آزمایش با دادوستد میان هزینهٔ اعمال سازگاری و منافعش بر حسب کاهش جستجو را آغاز کردند. هارالیک و الیوت (۱۹۸۰) الگوریتم کمینهٔ بررسی رو به جلو را که مک‌گرگور (۱۹۷۹) توصیف کرده ترجیح دادند، در حالی که گاشنیگ (۱۹۷۹) بررسی کامل سازگاری کمانی پس از هر تخصیص متغیر را پیشنهاد کرد — الگوریتمی که بعداً توسط سابین و فرودر (۱۹۹۴) MAC نامیده شد. مقالهٔ اخیر شواهد نسبتاً قانع‌کننده‌ای ارائه می‌دهد که در CSPهای سخت‌تر، بررسی کامل سازگاری کمانی به صرفه است. فرودر (۱۹۷۸، ۱۹۸۲) مفهوم سازگاری k-

تایی و رابطه‌اش با پیچیدگی حل CSPها را بررسی کرد. دچتر و دچتر (۱۹۸۷) سازگاری کمانی جهتی را معرفی کردند. آپت (۱۹۹۹) چارچوب الگوریتمی عمومی‌ای توصیف می‌کند که در آن الگوریتم‌های انتشار سازگاری را می‌توان تحلیل کرد و مرورهایی توسط بسییر (۲۰۰۶) و بارتاک و همکاران (۲۰۱۰) ارائه شده.

روش‌های ویژه برای مدیریت قیدهای مرتبه‌بالاتر یا سراسری نخست در بافت برنامه‌نویسی منطق قیدی توسعه یافتند. ماریوت و استاکی (۱۹۹۸) پوشش عالی پژوهش این حوزه را فراهم می‌کنند. قید Alldiff توسط رژین (۱۹۹۴)، استرگیو و والش (۱۹۹۹) و فان هوو (۲۰۰۱) مطالعه شد. الگوریتم‌های استنتاج پیچیده‌تری برای Alldiff هست (فان هوو و کاتریل، ۲۰۰۶ را ببینید) که قیدهای بیشتری منتشر می‌کنند اما اجرایشان از نظر محاسباتی پرهزینه‌تر است. قیدهای کران توسط فان هنتریک و همکاران (۱۹۹۸) در برنامه‌نویسی منطق قیدی گنجانده شد. مروری بر قیدهای سراسری توسط فان هوو و کاتریل (۲۰۰۶) ارائه شده.

سودوکو به شناخته‌شده‌ترین CSP بدل شده و سیمونیس (۲۰۰۵) آن را چنین توصیف کرد. آگربک و هانسن (۲۰۰۸) برخی از راهبردها را توصیف می‌کنند و نشان می‌دهند سودوکو روی تختهٔ $n^2 \times n^2$ در ردهٔ مسائل NP-سخت است.

در ۱۸۵۰، سی. اف. گاوس الگوریتم عقبگرد بازگشتی برای حل مسئلهٔ هشت‌وزیر توصیف کرد که در ۱۸۴۸ در مجلهٔ شطرنج آلمانی شاخ‌تسایتونگ منتشر شده بود. گاوس روشش را «تاتونیرن» نامید، برگرفته از واژهٔ فرانسوی tâtonner — یعنی دست‌وپازدن، مانند در تاریکی.

بنابر دونالد کنوت (ارتباط شخصی)، آر. جی. واکر اصطلاح «عقبگرد» را در دههٔ ۱۹۵۰ معرفی کرد. واکر (۱۹۶۰) الگوریتم عقبگرد پایه را توصیف و از آن برای یافتن همهٔ راه‌حل‌های مسئلهٔ ۱۳-وزیر استفاده کرد. گولومب و باومرت (۱۹۶۵) با مثال‌هایی ردهٔ عمومی مسائل ترکیبیاتی را که عقبگرد بر آن‌ها قابل اعمال است صورت‌بندی کردند و آنچه را ما اکتشافی MRV می‌نامیم معرفی کردند. بیتنر و رینگولد (۱۹۷۵) مرور تأثیرگذاری بر فنون عقبگرد ارائه کردند. برلاز (۱۹۷۹) از اکتشافی درجه به‌عنوان شکست‌کنندهٔ تساوی پس از اعمال اکتشافی MRV استفاده کرد. الگوریتم حاصل، با وجود سادگی‌اش، همچنان بهترین روش برای k-رنگ‌کردن گراف‌های دلخواه است. هارالیک و الیوت (۱۹۸۰) اکتشافی کمترین-قید-محدودکننده را پیشنهاد کردند.

روش پایهٔ پرش به عقب از جان گاشنیگ (۱۹۷۷، ۱۹۷۹) است. کوندراک و ون بیک (۱۹۹۷) نشان دادند این الگوریتم اساساً توسط بررسی رو به جلو در بر گرفته می‌شود. پرش به عقب معطوف-به-تعارض توسط پروسر (۱۹۹۳) ابداع شد. دچتر (۱۹۹۰) پرش به عقب مبتنی بر گراف را معرفی کرد که پیچیدگی الگوریتم‌های مبتنی بر پرش به عقب را به‌عنوان تابعی از گراف قید کران‌دار می‌کند (دچتر و فراست، ۲۰۰۲).

شکل بسیار عمومی‌ای از عقبگرد هوشمند از همان آغاز توسط استالمن و ساسمن (۱۹۷۷) توسعه یافت. فن آن‌ها، عقبگرد معطوف-به-وابستگی (Dependency-Directed Backtracking)، پرش به عقب را با یادگیری ناخوب (مک‌آلیستر، ۱۹۹۰) ترکیب می‌کند و به توسعهٔ سامانه‌های نگه‌داری حقیقت (دویل، ۱۹۷۹) انجامید که در بخش 10.6.2 بحث می‌کنیم. پیوند میان دو حوزه توسط د کلیر (۱۹۸۹) تحلیل شده.

کار استالمن و ساسمن همچنین ایدهٔ یادگیری قید را معرفی کرد، که در آن نتایج جزئیِ به‌دست‌آمده از جستجو را می‌توان ذخیره و بعداً در جستجو دوباره استفاده کرد. ایده توسط دچتر (۱۹۹۰) صوری شد. عقب‌نشانه‌گذاری (Backmarking) (گاشنیگ، ۱۹۷۹) روش به‌ویژه ساده‌ای است که در آن تخصیص‌های جفتیِ سازگار و ناسازگار ذخیره و برای پرهیز از بررسی دوبارهٔ قیدها استفاده می‌شوند. عقب‌نشانه‌گذاری را می‌توان با پرش به عقب معطوف-به-تعارض ترکیب کرد؛ کوندراک و ون بیک (۱۹۹۷) الگوریتم هیبریدی ارائه می‌کنند که به‌شکل اثبات‌پذیر هر یک از روش‌ها را به‌طور جداگانه در بر می‌گیرد.

روش عقبگرد پویا (گینزبرگ، ۱۹۹۳) تخصیص‌های جزئی موفق از زیرمجموعه‌های بعدی متغیرها را هنگام عقبگرد بر انتخابی قبلی که موفقیت بعدی را باطل نمی‌کند حفظ می‌کند. موسکویتز و همکاران (۲۰۰۱) نشان می‌دهند چگونه این فنون و دیگران برای ایجاد حل‌کنندهٔ SAT کارآمد استفاده می‌شوند. مطالعات تجربی چند روش عقبگرد تصادفی توسط گومز و همکاران (۲۰۰۰) و گومز و سلمن (۲۰۰۱) انجام شد. ون بیک (۲۰۰۶) عقبگرد را مرور می‌کند.

جستجوی محلی در مسئله‌های ارضای قیود توسط کار کیرک‌پاتریک و همکاران (۱۹۸۳) بر بازپخت شبیه‌سازی‌شده (فصل ۴ را ببینید) رایج شد که به‌طور گسترده برای چیدمان VLSI و مسائل زمان‌بندی استفاده می‌شود. بک و همکاران (۲۰۱۱) مروری بر کارهای اخیر زمان‌بندی کارگاه می‌دهند. اکتشافی کمترین-تعارض نخستین بار توسط گو (۱۹۸۹) پیشنهاد و به‌طور مستقل توسط مینتون و همکاران (۱۹۹۲) توسعه یافت. سوسیچ و گو (۱۹۹۴) نشان دادند چگونه می‌توان آن را برای حل مسئلهٔ ۳,۰۰۰,۰۰۰ وزیر در کمتر از یک دقیقه اعمال کرد. موفقیت خیره‌کنندهٔ جستجوی محلی با کمترین-تعارض در مسئلهٔ n-وزیر به بازنگری در سرشت و شیوع مسائل «آسان» و «سخت» انجامید. پیتر چیزمن و همکاران (۱۹۹۱) دشواری CSPهای تصادفی‌تولیدشده را کاویدند و کشف کردند تقریباً همهٔ چنین مسائلی یا به‌طرز پیش‌پاافتاده‌ای آسان‌اند یا هیچ راه‌حلی ندارند. فقط اگر پارامترهای مولد مسئله در بازهٔ باریک معینی تنظیم شوند، که در آن تقریباً نیمی از مسائل حل‌شدنی‌اند، نمونه‌های مسئلهٔ «سخت» می‌یابیم. این پدیده را در فصل ۷ بیشتر بحث می‌کنیم.

کونولیگه (۱۹۹۴) نشان داد جستجوی محلی در مسائل با درجه‌ای از ساختار محلی از جستجوی عقبگرد پایین‌تر است؛ این به کارهایی انجامید که جستجوی محلی و استنتاج را ترکیب می‌کردند، مانند کار پینکاس و دچتر (۱۹۹۵). هوس و تسانگ (۲۰۰۶) مروری بر فنون جستجوی محلی ارائه می‌دهند و کتاب‌های درسی توسط هوس و استوتزله (۲۰۰۴) و آرتس و لنسترا (۲۰۰۳) عرضه شده.

کار مربوط به ساختار و پیچیدگی CSPها از فرودر (۱۹۸۵) و مک‌ورث و فرودر (۱۹۸۵) سرچشمه می‌گیرد که نشان دادند جستجو روی درخت‌های سازگار-کمانی بدون هیچ عقبگردی کار می‌کند. نتیجهٔ مشابهی، با بسط‌هایی به ابرگراف‌های غیرچرخه‌ای، در جامعهٔ پایگاه‌داده توسعه یافت (بیری و همکاران، ۱۹۸۳). بایاردو و میرانکر (۱۹۹۴) الگوریتمی برای CSPهای درخت‌ساختار ارائه می‌کنند که بدون هیچ پیش‌پردازشی در زمان خطی اجرا می‌شود. دچتر (۱۹۹۰) رویکرد مجموعهٔ برش چرخه را توصیف می‌کند.

از زمان انتشار آن مقاله‌ها، پیشرفت فراوانی در توسعهٔ نتایج عمومی‌تری رخ داده که پیچیدگی حل یک CSP را با ساختار گراف قیدش مرتبط می‌کنند. مفهوم عرض درختی توسط نظریه‌پردازان گراف رابرتسون و سیمور (۱۹۸۶) معرفی شد. دچتر و پیرل (۱۹۸۷، ۱۹۸۹)، با بنا بر کار فرودر، مفهوم مرتبطی (که آن را عرض القایی نامیدند اما با عرض درختی

یکسان است) را بر مسئله‌های ارضای قیود اعمال کردند و رویکرد تجزیهٔ درختیِ ترسیم‌شده در بخش ۶.۵ را توسعه دادند. با بهره از این کار و نتایج نظریهٔ پایگاه‌داده، گوتلوب و همکاران (۱۹۹۹) مفهومی به نام عرض ابردرختی توسعه دادند که بر توصیف CSP به‌صورت ابرگراف مبتنی است. افزون بر نشان‌دادن اینکه هر CSP با عرض ابردرختی w را می‌توان در زمان O(n^(w+1) log n) حل کرد، آن‌ها همچنین نشان دادند عرض ابردرختی همهٔ معیارهای قبلی «عرض» را در بر می‌گیرد، به این معنا که مواردی هست که عرض ابردرختی کران‌دار و دیگر معیارها بی‌کران‌اند.

الگوریتم RELSAT بایاردو و شراگ (۱۹۹۷) یادگیری قید و پرش به عقب را ترکیب کرد و نشان داده شد از بسیاری الگوریتم‌های دیگر آن زمان بهتر عمل می‌کند. این به الگوریتم‌های جستجوی AND–OR انجامید که هم بر CSPها و هم بر استدلال احتمالاتی کاربردپذیرند (دچتر و ماتسکو، ۲۰۰۷). براون و همکاران (۱۹۸۸) ایدهٔ شکستن تقارن در CSPها را معرفی کردند و جنت و همکاران (۲۰۰۶) مروری ارائه می‌دهند.

حوزهٔ ارضای قید توزیع‌شده به حل CSPها می‌نگرد وقتی مجموعه‌ای از عامل‌ها هست که هر یک زیرمجموعه‌ای از متغیرهای قید را کنترل می‌کند. از ۲۰۰۰ کارگاه‌های سالانه‌ای بر این مسئله برگزار شده و پوشش خوبی در جای دیگر هست (کولین و همکاران، ۱۹۹۹؛ پیرس و همکاران، ۲۰۰۸).

مقایسهٔ الگوریتم‌های CSP بیشتر علم تجربی است: نتایج نظری اندکی نشان می‌دهند الگوریتمی بر الگوریتم دیگر در همهٔ مسائل برتری دارد؛ در عوض، باید آزمایش اجرا کنیم تا ببینیم کدام الگوریتم‌ها در نمونه‌های معمولی مسائل بهتر عمل می‌کنند. چنان‌که هوکر (۱۹۹۵) اشاره می‌کند، باید مراقب باشیم میان آزمون رقابتی — چنان‌که در رقابت‌های میان الگوریتم‌ها بر اساس زمان اجرا رخ می‌دهد — و آزمون علمی، که هدفش شناسایی ویژگی‌های الگوریتمی است که اثربخشی‌اش را بر رده‌ای از مسائل تعیین می‌کند، تمایز قائل شویم.

کتاب‌های درسی آپت (۲۰۰۳)، دچتر (۲۰۰۳)، تسانگ (۱۹۹۳) و لکوتر (۲۰۰۹)، و مجموعهٔ روسی و همکاران (۲۰۰۶) منابع عالی دربارهٔ پردازش قید هستند. چند مقالهٔ مرور خوب هست، از جمله دچتر و فراست (۲۰۰۲) و بارتاک و همکاران (۲۰۱۰). کاربونل و کوپر (۲۰۱۶) رده‌های حل‌شدنی CSPها را مرور می‌کنند. کوندراک و ون بیک (۱۹۹۷) مرور تحلیلی الگوریتم‌های جستجوی عقبگرد می‌دهند و باکوس و ون ران (۱۹۹۵) مرور تجربی‌تری. برنامه‌نویسی قید در کتاب‌های آپت (۲۰۰۳) و فرویرت و عبدالنادر (۲۰۰۳) پوشش داده شده. مقاله‌های ارضای قید به‌طور منظم در «هوش مصنوعی» و در ژورنال تخصصی «قیدها» ظاهر می‌شوند؛ تازه‌ترین حل‌کننده‌های SAT در رقابت سالانهٔ بین‌المللی SAT توصیف می‌شوند. محل کنفرانس اصلی، کنفرانس بین‌المللی اصول و عمل برنامه‌نویسی قید است که اغلب CP نامیده می‌شود.

پانوشت‌ها:

[1] ما از اصطلاح «یال» به‌جای «کمان» استفاده کرده‌ایم، پس منطقی‌تر بود این را «سازگار-یالی» بنامیم، اما نام «سازگار-کمانی» تاریخی است.

فصل ۷: عامل‌های منطقی (Logical Agents)

در این فصل عامل‌هایی طراحی می‌کنیم که می‌توانند بازنمایی‌هایی از یک جهان پیچیده بسازند، از فرایند استنتاج برای استخراج بازنمایی‌های تازه دربارهٔ جهان استفاده کنند و از این بازنمایی‌های تازه برای استنتاج اینکه چه باید بکنند بهره بگیرند.

به نظر می‌رسد انسان‌ها چیزهایی می‌دانند؛ و آنچه می‌دانند به آن‌ها در انجام کارها کمک می‌کند. در هوش مصنوعی، عامل‌های مبتنی بر دانش (Knowledge-Based Agents) از فرایند استدلال (Reasoning) روی بازنمایی درونی دانش برای تصمیم‌گیری دربارهٔ کنش‌هایی که باید انجام دهند استفاده می‌کنند.

عامل‌های حل‌کنندهٔ مسئلهٔ فصل‌های ۳ و ۴ چیزهایی می‌دانند، اما فقط به معنایی بسیار محدود و انعطاف‌ناپذیر. آن‌ها می‌دانند چه کنش‌هایی در دسترس است و نتیجهٔ انجام کنش خاصی از حالت خاصی چه خواهد بود، اما واقعیت‌های عمومی را نمی‌دانند. عامل مسیریاب نمی‌داند ممکن نیست جاده‌ای طولش تعداد منفی کیلومتر باشد. عامل معمای ۸ نمی‌داند دو کاشی نمی‌توانند یک فضا را اشغال کنند. دانشی که دارند برای یافتن مسیری از آغاز تا هدف بسیار مفید است، اما برای هیچ چیز دیگر.

بازنمایی‌های اتمیِ به‌کاررفته توسط عامل‌های حل‌کنندهٔ مسئله نیز بسیار محدودکننده‌اند. در محیط تا حدی قابل مشاهده، برای نمونه، تنها انتخاب عامل حل‌کنندهٔ مسئله برای بازنمایی آنچه دربارهٔ حالت کنونی می‌داند، فهرست‌کردن همهٔ حالت‌های عینی ممکن است. می‌توانم به انسانی هدفِ رانندگی به شهری در آمریکا با جمعیت کمتر از ۱۰,۰۰۰ بدهم، اما برای گفتن آن به عامل حل‌کنندهٔ مسئله، فقط می‌توانم هدف را به‌صورت صوری به‌عنوان مجموعهٔ صریحی از حدود ۱۶,۰۰۰ شهری که توصیف را برآورده می‌کنند توصیف کنم.

فصل ۶ نخستین بازنمایی عاملی ما را معرفی کرد، که در آن حالت‌ها به‌صورت تخصیص مقادیر به متغیرها بازنمایی می‌شوند؛ این گامی در جهت درست است، بخش‌هایی از عامل را قادر می‌کند به‌صورت مستقل از حوزه کار کنند و الگوریتم‌های کارآمدتری را ممکن می‌کند. در این فصل، این گام را به نتیجهٔ منطقی‌اش می‌بریم — منطق را به‌عنوان رده‌ای عمومی از بازنمایی‌ها برای پشتیبانی از عامل‌های مبتنی بر دانش توسعه می‌دهیم. این عامل‌ها می‌توانند اطلاعات را برای اهداف بی‌شمار ترکیب و بازترکیب کنند. این می‌تواند بسیار دور از نیازهای لحظه باشد — مانند زمانی که ریاضی‌دانی قضیه‌ای را اثبات می‌کند یا اخترشناسی امید به حیات زمین را محاسبه می‌کند. عامل‌های مبتنی بر دانش می‌توانند تکلیف‌های تازه را در قالب هدف‌های صریحاً توصیف‌شده بپذیرند؛ می‌توانند با گفته‌شدن یا یادگیری دانش تازه دربارهٔ محیط به‌سرعت به شایستگی برسند؛ و می‌توانند با به‌روزرسانی دانش مرتبط با تغییرات محیط سازگار شوند.

در بخش ۷.۱ با طراحی کلی عامل آغاز می‌کنیم. بخش ۷.۲ محیط سادهٔ تازه‌ای به نام «جهان وومپوس» معرفی می‌کند و عملکرد عامل مبتنی بر دانش را بدون ورود به جزئیات فنی نشان می‌دهد. سپس اصول عمومی منطق را در بخش ۷.۳ و مشخصات منطق گزاره‌ای را در بخش ۷.۴ توضیح می‌دهیم. منطق گزاره‌ای بازنمایی عاملی است؛ هرچند از منطق مرتبهٔ اول (فصل ۸) که بازنمایی ساختاریافتهٔ شاخص است کم‌بیان‌تر است، منطق گزاره‌ای همهٔ مفاهیم پایهٔ منطق را

نشان می‌دهد. همچنین با فناوری‌های استنتاجی خوب‌توسعه‌یافته‌ای همراه است که در بخش‌های ۷.۵ و ۷.۶ توصیف می‌کنیم. سرانجام، بخش ۷.۷ مفهوم عامل‌های مبتنی بر دانش را با فناوری منطق گزاره‌ای ترکیب می‌کند تا عامل‌های ساده‌ای برای جهان وومپوس بسازد.

۷.۱ عامل‌های مبتنی بر دانش

مؤلفهٔ مرکزی عامل مبتنی بر دانش، «پایگاه دانش» (Knowledge Base) یا KB آن است. پایگاه دانش مجموعه‌ای از «جمله»ها (Sentences) است. (در این‌جا «جمله» اصطلاحی فنی است. با جمله‌های انگلیسی و دیگر زبان‌های طبیعی مرتبط است اما یکسان نیست.) هر جمله به زبانی به نام «زبان بازنمایی دانش» (Knowledge Representation Language) بیان می‌شود و ادعایی (Assertion) دربارهٔ جهان را بازنمایی می‌کند. وقتی جمله را داده‌شده بدانیم بدون آنکه از جمله‌های دیگر مشتق شده باشد، آن را «اصل‌موضوعه» (Axiom) می‌نامیم.

باید راهی برای افزودن جمله‌های تازه به پایگاه دانش و راهی برای پرس‌وجو از آنچه شناخته شده باشد. نام‌های استاندارد این عملیات به‌ترتیب TELL و ASK است. هر دو عملیات ممکن است شامل استنتاج (Inference) باشند — یعنی مشتق‌کردن جمله‌های تازه از جمله‌های قدیمی. استنتاج باید این الزام را رعایت کند که وقتی از پایگاه دانش پرسشی ASK می‌شود، پاسخ باید از آنچه قبلاً به پایگاه دانش TELL شده «برآید». بعداً در این فصل دربارهٔ واژهٔ حیاتی «برآمدن» دقیق‌تر خواهیم بود. فعلاً آن را به این معنا بگیرید که فرایند استنتاج نباید هنگام پیش‌رفتن چیزهایی از خودش بسازد.

شکل ۷.۱ نمای کلی برنامهٔ عامل مبتنی بر دانش را نشان می‌دهد. مانند همهٔ عامل‌های ما، ادراک حسی را به‌عنوان ورودی می‌گیرد و کنشی بازمی‌گرداند. عامل پایگاه دانشی به نام KB نگه می‌دارد که در آغاز ممکن است مقداری «دانش زمینه‌ای» (Background Knowledge) داشته باشد.

```
function KB-AGENT(percept) returns an action
    persistent: KB, a knowledge base
    t, a counter, initially 0, indicating the time
    TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
    action ← ASK(KB, MAKE-ACTION-QUERY(t))
    TELL(KB, MAKE-ACTION-SENTENCE(action, t))
    t ← t + 1
    return action
```

شکل ۷.۱ — عامل مبتنی بر دانش عمومی. با دریافت ادراک حسی، عامل ادراک را به پایگاه دانشش می‌افزاید، از پایگاه دانش بهترین کنش را می‌پرسد و به پایگاه دانش می‌گوید که در واقع آن کنش را انجام داده.

هر بار که برنامهٔ عامل فراخوانی می‌شود، سه کار می‌کند. نخست، آنچه را ادراک می‌کند به پایگاه دانش TELL می‌کند. دوم، از پایگاه دانش ASK می‌کند چه کنشی باید انجام دهد. در فرایند پاسخ به این پرس‌وجو، ممکن است استدلال گسترده‌ای دربارهٔ حالت کنونی جهان، دربارهٔ پیامدهای توالی‌های کنشِ ممکن، و مانند آن انجام شود. سوم، برنامهٔ عامل به پایگاه دانش می‌گوید کدام کنش برگزیده شده و کنش را بازمی‌گرداند تا اجرا شود.

جزئیات زبان بازنمایی درون سه تابع پنهان است که واسط میان حسگرها و عملگرها از یک سو و سامانهٔ مرکزی بازنمایی و استدلال از سوی دیگر را پیاده می‌کنند. MAKE-PERCEPT-SENTENCE جمله‌ای می‌سازد که ادعا می‌کند عامل ادراکِ داده‌شده را در زمانِ داده‌شده ادراک کرده. MAKE-ACTION-QUERY جمله‌ای می‌سازد که می‌پرسد در زمان کنونی چه کنشی باید انجام شود. سرانجام، MAKE-ACTION-SENTENCE جمله‌ای می‌سازد که ادعا می‌کند کنشِ برگزیده‌شده اجرا شده. جزئیات سازوکارهای استنتاج درون TELL و ASK پنهان است. بخش‌های بعدی این جزئیات را آشکار خواهند کرد.

عامل شکل ۷.۱ به عامل‌های با حالت درونیِ توصیف‌شده در فصل ۲ کاملاً شبیه می‌نماید. اما به‌سبب تعریف‌های TELL و ASK، عامل مبتنی بر دانش برنامهٔ دلخواهی برای محاسبهٔ کنش‌ها نیست. این عامل با توصیف در «سطح دانش» (Knowledge Level) سازگار است، جایی که برای تعیین رفتارش فقط باید مشخص کنیم عامل چه می‌داند و هدف‌هایش چیست.

برای نمونه، تاکسی خودکار ممکن است هدف بردن مسافری از سانفرانسیسکو به شهرستان مارین را داشته باشد و بداند پل گلدن گیت تنها پیوند میان دو مکان است. آنگاه می‌توانیم انتظار داشته باشیم از پل گلدن گیت بگذرد، زیرا می‌داند آن کار هدفش را محقق می‌کند. توجه کنید این تحلیل مستقل از نحوهٔ کار تاکسی در «سطح پیاده‌سازی» (Implementation Level) است. فرقی نمی‌کند دانش جغرافیایی‌اش با فهرست‌های پیوندی پیاده شود یا نقشه‌های پیکسلی، یا با دست‌کاری رشته‌های نمادِ ذخیره‌شده در ثبات‌ها استدلال کند یا با انتشار سیگنال‌های پرسروصدا در شبکه‌ای از نورون‌ها.

عامل مبتنی بر دانش را می‌توان به‌سادگی با TELL کردن آنچه باید بداند ساخت. با شروع از پایگاه دانش خالی، طراح عامل می‌تواند جمله‌ها را یکی‌یکی TELL کند تا عامل بداند چگونه در محیطش عمل کند. این «رویکرد اعلانی» (Declarative Approach) به ساخت سامانه نامیده می‌شود. در برابر، «رویکرد رویه‌ای» (Procedural Approach) رفتارهای مطلوب را مستقیماً به‌صورت کد برنامه رمزگذاری می‌کند. در دهه‌های ۱۹۷۰ و ۱۹۸۰، طرفداران دو رویکرد بحث‌های داغی کردند. اکنون می‌دانیم عامل موفق اغلب عناصر اعلانی و رویه‌ای را در طراحی‌اش ترکیب می‌کند و دانش اعلانی را اغلب می‌توان به کد رویه‌ای کارآمدتری ترجمه (Compile) کرد.

همچنین می‌توانیم به عامل مبتنی بر دانش سازوکارهایی بدهیم که به آن امکان می‌دهد خودش یاد بگیرد. این سازوکارها، که در فصل ۱۹ بحث می‌شوند، از سلسله‌ای ادراک حسی، دانش عمومی دربارهٔ محیط ایجاد می‌کنند. عامل یادگیرنده می‌تواند کاملاً خودمختار باشد.

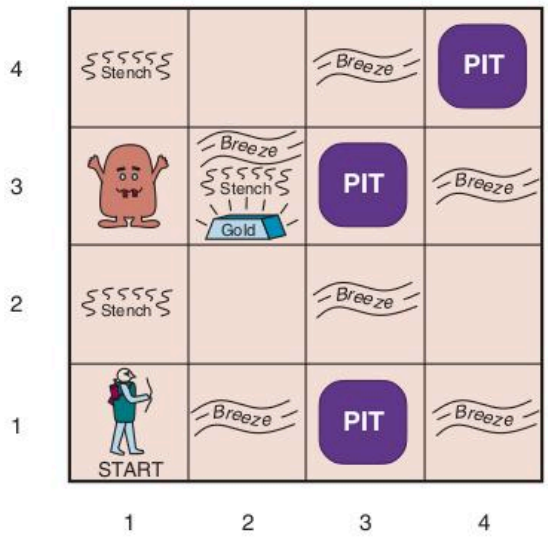
۷.۲ جهان وومپوس

در این بخش محیطی را توصیف می‌کنیم که عامل‌های مبتنی بر دانش می‌توانند ارزششان را نشان دهند. «جهان وومپوس» (Wumpus World) غاری است شامل اتاق‌هایی که با راهروها به هم متصل‌اند. جایی در غار وومپوسِ وحشتناک کمین کرده؛ جانوری که هر کسی را که وارد اتاقش شود می‌خورد. وومپوس را می‌توان با تیر زد، اما عامل فقط یک تیر دارد. برخی اتاق‌ها گودال‌های بی‌پایان دارند که هر کسی را که به آن اتاق‌ها سرگردان شود به دام می‌اندازند

(به‌جز وومپوس که برای افتادن زیادی بزرگ است). تنها ویژگی جبران‌کنندهٔ این محیط تاریک، امکان یافتن توده‌ای طلاست. هرچند جهان وومپوس با معیارهای بازی‌های رایانه‌ای مدرن نسبتاً بی‌آزار است، نکته‌های مهمی دربارهٔ هوش نشان می‌دهد.

نمونه‌ای از جهان وومپوس در شکل ۷.۲ نشان داده شده. تعریف دقیق محیط تکلیف، چنان‌که در بخش 2.3 پیشنهاد شد، با توصیف PEAS داده می‌شود:

- **معیار عملکرد:** ۱۰۰۰+ برای بیرون‌آمدن از غار با طلا، ۱۰۰۰- برای افتادن در گودال یا خورده‌شدن توسط وومپوس، ۱- برای هر کنش انجام‌شده، و ۱۰- برای مصرف تیر. بازی یا وقتی عامل می‌میرد پایان می‌یابد یا وقتی عامل از غار بیرون می‌آید.



شکل ۷.۲ — جهان وومپوس نمونه. عامل در گوشهٔ پایین-چپ، رو به شرق (راست) است. (نمادها: PIT = گودال، Stench = بوی گند، Breeze = نسیم، Gold = طلا، START = آغاز)

- **محیط:** شبکهٔ 4×4 از اتاق‌ها، با دیوارهایی دور شبکه. عامل همیشه در خانهٔ [1,1] آغاز می‌کند، رو به شرق. مکان‌های طلا و وومپوس به‌صورت تصادفی، با توزیع یکنواخت، از میان خانه‌های غیر از خانهٔ آغاز برگزیده می‌شوند. افزون بر این، هر خانهٔ غیر از آغاز می‌تواند گودال باشد، با احتمال ۰٫۲.
- **عملگرها:** عامل می‌تواند Forward (به جلو)، TurnLeft با ۹۰ درجه یا TurnRight با ۹۰ درجه حرکت کند. اگر وارد خانه‌ای شود که گودال یا وومپوس زنده دارد، مرگی بد می‌میرد. (واردشدن به خانه‌ای با وومپوس مرده ایمن است، هرچند بدبو.) اگر عامل بخواهد به جلو حرکت کند و به دیوار بخورد، حرکت نمی‌کند. کنش Grab (برداشتن) برای برداشتن طلا اگر در همان خانهٔ عامل باشد استفاده می‌شود. کنش Shoot (تیراندازی) برای شلیک تیری در خط مستقیم در جهتی که عامل رو به آن است استفاده می‌شود. تیر ادامه می‌یابد تا آنکه یا به وومپوس بخورد (و او را بکشد) یا به دیوار بخورد. عامل فقط یک تیر دارد، پس فقط نخستین کنش Shoot اثری دارد. سرانجام، کنش Climb (بالارفتن) برای بیرون‌آمدن از غار استفاده می‌شود، اما فقط از خانهٔ [1,1].
- **حسگرها:** عامل پنج حسگر دارد که هر یک یک بیت اطلاعات می‌دهد:
  - در خانه‌های مستقیماً (نه مورب) مجاور وومپوس، عامل «بوی گند» (Stench) ادراک می‌کند.[1]

- در خانه‌های مستقیماً مجاور گودال، عامل «نسیم» (Breeze) ادراک می‌کند.
- در خانه‌ای که طلاست، عامل «درخشش» (Glitter) ادراک می‌کند.
- وقتی عامل به دیوار می‌خورد، «برخورد» (Bump) ادراک می‌کند.
- وقتی وومپوس کشته می‌شود، «فریاد» (Scream) اندوهگینی برمی‌آورد که در هر جای غار قابل ادراک است.

ادراک‌ها به برنامهٔ عامل در قالب فهرستی از پنج نماد داده می‌شوند؛ برای نمونه، اگر بوی گند و نسیم هست اما درخشش، برخورد یا فریاد نیست، برنامهٔ عامل [Stench, Breeze, None, None, None] دریافت می‌کند.

می‌توانیم محیط وومپوس را در امتداد ابعاد گوناگون فصل ۲ مشخص کنیم. بدیهی است جبرگرایانه، گسسته، ایستا و تک‌عاملی است. (وومپوس حرکت نمی‌کند، خوشبختانه.) توالی‌دار است، زیرا پاداش‌ها ممکن است فقط پس از کنش‌های فراوان بیایند. تا حدی قابل مشاهده است، زیرا برخی جنبه‌های حالت مستقیماً قابل ادراک نیستند: مکان عامل، وضعیت سلامتی وومپوس و در دسترس‌بودن تیر. دربارهٔ مکان‌های گودال‌ها و وومپوس: می‌توانیم آن‌ها را بخش‌های مشاهده‌نشدهٔ حالت بدانیم — در آن صورت، مدل گذار محیط کاملاً شناخته‌شده است و یافتن مکان‌های گودال‌ها دانش عامل از حالت را کامل می‌کند. از سوی دیگر، می‌توانیم بگوییم خودِ مدل گذار ناشناخته است، زیرا عامل نمی‌داند کدام کنش‌های Forward کشنده‌اند — در آن صورت، کشف مکان‌های گودال‌ها و وومپوس دانش عامل از مدل گذار را کامل می‌کند.

برای عامل در محیط، چالش اصلی نادانی آغازین‌اش از پیکربندی محیط است؛ غلبه بر این نادانی به نظر می‌رسد به استدلال منطقی نیاز دارد. در بیشتر نمونه‌های جهان وومپوس، ممکن است عامل با خیال راحت طلا را به دست آورد. گاهی، عامل باید میان رفتن به خانه دست‌خالی و به خطر انداختن جان برای یافتن طلا یکی را برگزیند. حدود ۲۱٪ از محیط‌ها کاملاً ناعادلانه‌اند، زیرا طلا در گودالی است یا با گودال‌ها احاطه شده.

بگذارید عامل وومپوس مبتنی بر دانش را هنگام کاوش محیطِ نشان‌داده‌شده در شکل ۷.۲ تماشا کنیم. از زبان بازنمایی دانش غیررسمی استفاده می‌کنیم که شامل نوشتن نمادها در شبکه است (مانند شکل‌های ۷.۳ و ۷.۴).

پایگاه دانش آغازین عامل شامل قواعد محیط است، چنان‌که پیش‌تر توصیف شد؛ به‌ویژه، می‌داند در [1,1] است و [1,1] خانهٔ امنی است؛ آن را به‌ترتیب با «A» و «OK» در خانهٔ [1,1] نشان می‌دهیم.

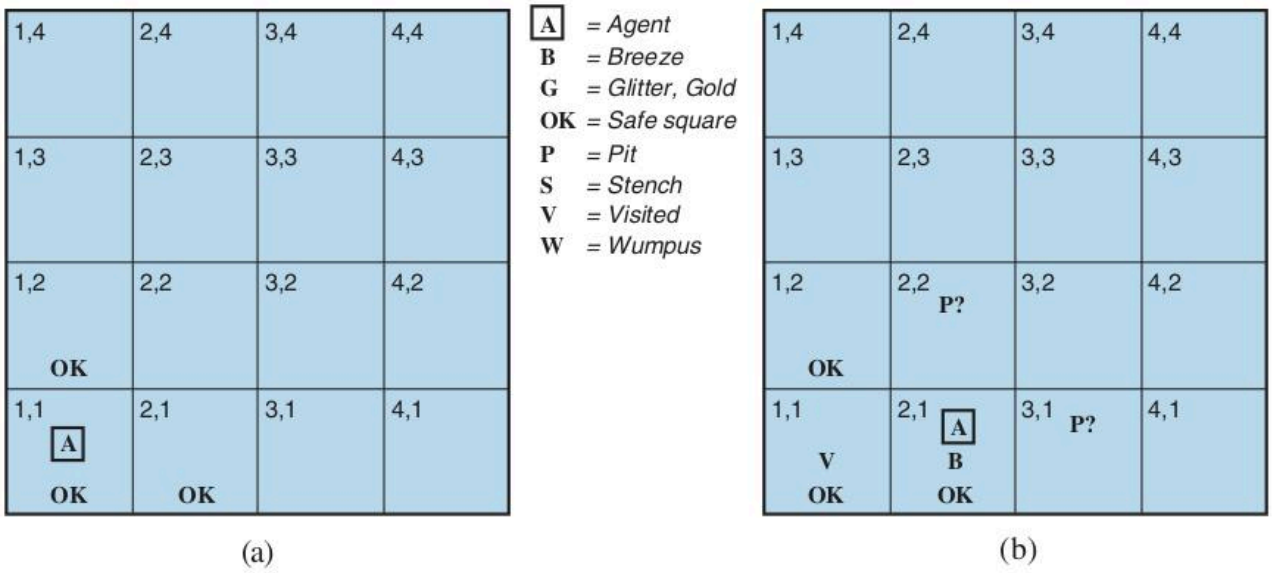
نخستین ادراک [None,None,None,None,None] است که از آن عامل می‌تواند نتیجه بگیرد خانه‌های همسایه‌اش، [1,2] و [2,1]، عاری از خطرند — آن‌ها OK هستند. شکل ۷.۳(الف) وضعیت دانش عامل را در این نقطه نشان می‌دهد.

عامل محتاط فقط به خانه‌ای حرکت می‌کند که می‌داند OK است. فرض کنیم عامل تصمیم می‌گیرد به جلو به [2,1] حرکت کند. عامل در [2,1] نسیمی ادراک می‌کند (با «B» نشان داده شده)، پس باید گودالی در خانهٔ همسایه باشد. گودال نمی‌تواند در [1,1] باشد، طبق قواعد بازی، پس باید گودالی در [2,2] یا [3,1] یا هر دو باشد. نماد «P?» در شکل ۷.۳(ب) گودالِ ممکن را در آن خانه‌ها نشان می‌دهد. در این نقطه، فقط یک خانهٔ شناخته‌شده هست که OK است و هنوز بازدید نشده. پس عامل عاقل برمی‌گردد، به [1,1] بازمی‌گردد و سپس به [1,2] می‌رود.

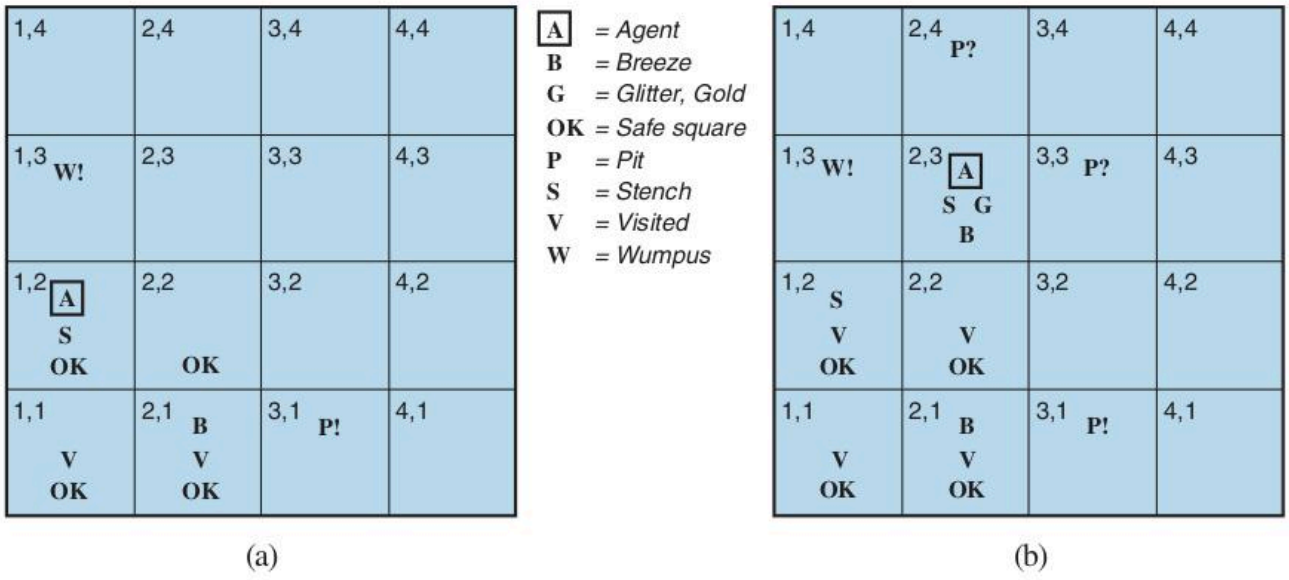
عامل در [1,2] بوی گند ادراک می‌کند و به وضعیت دانشِ نشان‌داده‌شده در شکل ۷.۴(الف) می‌رسد. بوی گند در [1,2] بدان معناست که وومپوسی باید نزدیک باشد. اما وومپوس نمی‌تواند در [1,1] باشد، طبق قواعد بازی، و نمی‌تواند در [2,2] باشد (یا عامل وقتی در [2,1] بود بوی گند تشخیص می‌داد). پس عامل می‌تواند استنتاج کند وومپوس در [1,3] است. نماد W! این استنتاج را نشان می‌دهد. وانگهی، نبودِ نسیم در [1,2] دلالت دارد هیچ گودالی در [2,2] نیست. اما عامل قبلاً استنتاج کرده باید گودالی در [2,2] یا [3,1] باشد، پس این بدان معناست باید در [3,1] باشد. این استنتاج نسبتاً دشواری است، زیرا دانشِ به‌دست‌آمده در زمان‌های مختلف و مکان‌های مختلف را ترکیب می‌کند و برای یک گام حیاتی به نبودِ ادراک تکیه می‌کند.

عامل اکنون به خودش اثبات کرده در [2,2] نه گودالی هست نه وومپوسی، پس حرکت به آن‌جا OK است. وضعیت دانش عامل را در [2,2] نشان نمی‌دهیم؛ فقط فرض می‌کنیم عامل می‌چرخد و به [2,3] حرکت می‌کند و شکل ۷.۴(ب) را به ما می‌دهد. در [2,3]، عامل درخششی تشخیص می‌دهد، پس باید طلا را بردارد و سپس به خانه برگردد.

توجه کنید در هر موردی که عامل از اطلاعات در دسترس نتیجه‌ای می‌گیرد، اگر اطلاعات در دسترس درست باشند، آن نتیجه تضمین‌شده درست است. این ویژگی بنیادین استدلال منطقی است. در بقیهٔ این فصل توصیف می‌کنیم چگونه عامل‌های منطقی بسازیم که بتوانند اطلاعات را بازنمایی کنند و نتیجه‌هایی مانند آنچه در بندهای پیشین توصیف شد بگیرند.



شکل ۷.۳ – نخستین گام برداشته‌شده توسط عامل در جهان وومپوس. (الف) وضعیت آغازین، پس از ادراک [None,None,None,None,None]. (ب) پس از حرکت به [2,1] و ادراک [None,Breeze,None,None,None]. (نمادها: OK = خانهٔ امن، V = بازدیدشده، B = نسیم، P? = گودال احتمالی، A = عامل)



شکل ۷.۴ – دو مرحلهٔ بعدی در پیشرفت عامل. (الف) پس از حرکت به [1,1] و سپس [1,2]، و ادراک [Stench,None,None,None,None]. (ب) پس از حرکت به [2,2] و سپس [2,3]، و ادراک [Stench,Breeze,Glitter,None,None]. (نمادها: S = بوی گند، W! = وومپوس استنتاج‌شده، G = درخشش/طلا، P! = گودال استنتاج‌شده)

۷.۳ منطق

این بخش مفاهیم بنیادین بازنمایی و استدلال منطقی را خلاصه می‌کند. این ایده‌های زیبا مستقل از شکل‌های خاص منطق‌اند. پس جزئیات فنی آن شکل‌ها را به بخش بعد موکول می‌کنیم و در عوض از مثال آشنای حساب معمولی استفاده می‌کنیم.

در بخش ۷.۱ گفتیم پایگاه‌های دانش از جمله‌ها تشکیل شده‌اند. این جمله‌ها بر اساس «نحو» (Syntax) زبان بازنمایی بیان می‌شوند که همهٔ جمله‌های خوش‌ساخت را مشخص می‌کند. مفهوم نحو در حساب معمولی به اندازهٔ کافی روشن است: «x+y = 4» جملهٔ خوش‌ساختی است، حال آنکه «x4y+ =» نیست.

منطق همچنین باید «معناشناسی» (Semantics) یا معنای جمله‌ها را تعریف کند. معناشناسی «صدق» (Truth) هر جمله را نسبت به هر «جهان ممکن» (Possible World) تعریف می‌کند. برای نمونه، معناشناسی حساب مشخص می‌کند جملهٔ «x+y=4» در جهانی که x برابر ۲ و y برابر ۲ است صادق است، اما در جهانی که x برابر ۱ و y برابر ۱ است کاذب. در منطق‌های استاندارد، هر جمله باید در هر جهان ممکن یا صادق باشد یا کاذب – هیچ «وسطی» نیست.[۲]

وقتی دقیق باشیم، به‌جای «جهان ممکن» از اصطلاح «مدل» (Model) استفاده می‌کنیم. در حالی که جهان‌های ممکن را می‌توان محیط‌های واقعی (بالقوه) دانست که عامل ممکن است در آن‌ها باشد یا نباشد، مدل‌ها انتزاع‌های ریاضی‌اند که هر یک برای هر جملهٔ مرتبط مقدار صدق ثابتی (صادق یا کاذب) دارند. به‌صورت غیررسمی، می‌توانیم جهان ممکن را مثلاً دارای x مرد و y زنِ نشسته‌برسر میزی که بریج بازی می‌کنند بدانیم، و جملهٔ x + y = 4 وقتی چهار نفر در مجموع هستند صادق است. به‌صورت صوری، مدل‌های ممکن فقط همهٔ تخصیص‌های ممکن اعداد صحیح نامنفی به

متغیرهای x و y هستند. هر چنین تخصیصی صدق هر جملهٔ حسابی را که متغیرهایش x و y هستند تعیین می‌کند. اگر جملهٔ α در مدل m صادق باشد، می‌گوییم m» α» را ارضا می‌کند» (Satisfaction) یا گاهی m مدلی از α است. از نماد M(α) برای مجموعهٔ همهٔ مدل‌های α استفاده می‌کنیم.

حال که مفهوم صدق را داریم، آماده‌ایم دربارهٔ استدلال منطقی صحبت کنیم. این شامل رابطهٔ «لازمهٔ منطقی» (Entailment) میان جمله‌هاست — این ایده که جمله‌ای به‌صورت منطقی از جمله‌ای دیگر برمی‌آید. در نماد ریاضی می‌نویسیم:

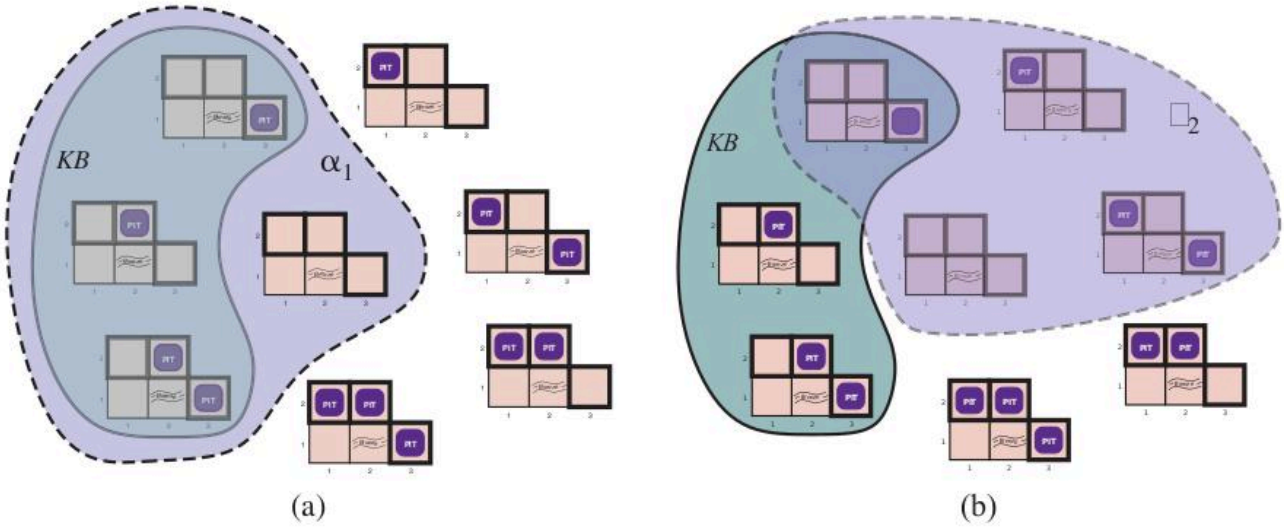
```
α ⊨ β
```

به این معنا که جملهٔ α مستلزم جملهٔ β است. تعریف صوری لازمه چنین است: α ⊨ β اگر و فقط اگر در هر مدلی که α در آن صادق است، β نیز صادق باشد. با نمادی که تازه معرفی کردیم می‌توانیم بنویسیم:

```
α ⊨ β  اگر و فقط اگر  M(α) ⊆ M(β)
```

(به جهت ⊇ توجه کنید: اگر α ⊨ β، آنگاه α ادعایی قوی‌تر از β است: جهان‌های ممکن بیشتری را کنار می‌گذارد.) رابطهٔ لازمه از حساب آشناست؛ با این ایده راحتیم که جملهٔ x = 0 مستلزم جملهٔ xy = 0 است. بدیهی است در هر مدلی که x صفر است، xy صفر است (صرف‌نظر از مقدار y).

می‌توانیم همان نوع تحلیل را بر مثال استدلال جهان وومپوسِ بخش پیشین اعمال کنیم. وضعیت شکل ۷.۳(ب) را در نظر بگیرید: عامل در [1,1] هیچ‌چیز و در [2,1] نسیم تشخیص داده. این ادراک‌ها، همراه با دانش عامل از قواعد جهان وومپوس، KB را تشکیل می‌دهند. عامل علاقه‌مند است آیا خانه‌های مجاور [1,2]، [2,2] و [3,1] گودال دارند یا نه. هر یک از سه خانه ممکن است گودال داشته باشد یا نه، پس (فعلاً با نادیده‌گرفتن دیگر جنبه‌های جهان) $2^3 = 8$ مدل ممکن هست. این هشت مدل در شکل ۷.۵ نشان داده شده.[3]



شکل ۷.۵ – مدل‌های ممکن برای حضور گودال‌ها در خانه‌های [1,2]، [2,2] و [3,1]. KB متناظر با مشاهده‌های «هیچ‌چیز در [1,1]» و «نسیم در [2,1]» با خط توپر نشان داده شده. (الف) خط نقطه‌چین مدل‌های α1 (هیچ گودالی در [1,2]) را نشان می‌دهد. (ب) خط نقطه‌چین مدل‌های α2 (هیچ گودالی در [2,2]) را نشان می‌دهد.

KB را می‌توان مجموعه‌ای از جمله‌ها یا جملهٔ تکی‌ای دانست که همهٔ جمله‌های فردی را ادعا می‌کند. KB در مدل‌هایی که با آنچه عامل می‌داند تناقض دارند کاذب است – برای نمونه، KB در هر مدلی که [1,2] گودال دارد کاذب است، زیرا در [1,1] نسیمی نیست. در واقع فقط سه مدل هست که KB در آن‌ها صادق است و این‌ها با خط توپر در شکل ۷.۵ محاصره شده‌اند. حال دو نتیجهٔ ممکن را در نظر بگیریم:

α1 = «در [1,2] گودالی نیست.» α2 = «در [2,2] گودالی نیست.»

مدل‌های α1 و α2 را به‌ترتیب در شکل‌های ۷.۵(الف) و ۷.۵(ب) با خط نقطه‌چین محاصره کرده‌ایم. با بازرسی، موارد زیر را می‌بینیم:

در هر مدلی که KB در آن صادق است، α1 نیز صادق است.

از این رو، KB ⊨ α1: در [1,2] گودالی نیست. همچنین می‌بینیم:

در برخی مدل‌هایی که KB در آن‌ها صادق است، α2 کاذب است.

از این رو، KB مستلزم α2 نیست: عامل نمی‌تواند نتیجه بگیرد در [2,2] گودالی نیست. (و نه می‌تواند نتیجه بگیرد در [2,2] گودالی هست.)[4]

مثال پیشین نه فقط لازمه را نشان می‌دهد، بلکه نشان می‌دهد چگونه تعریف لازمه را می‌توان برای استخراج نتیجه‌ها – یعنی انجام «استنتاج منطقی» (Logical Inference) – اعمال کرد. الگوریتم استنتاجِ نشان‌داده‌شده در شکل ۷.۵ «بررسی مدل» (Model Checking) نامیده می‌شود، زیرا همهٔ مدل‌های ممکن را برمی‌شمارد تا بررسی کند α در همهٔ مدل‌هایی که KB در آن‌ها صادق است صادق است، یعنی M(KB) ⊆ M(α).

در فهم لازمه و استنتاج، کمک می‌کند مجموعهٔ همهٔ پیامدهای KB را کوه یونجه و α را سوزنی بدانیم. لازمه مانند بودنِ سوزن در کوه یونجه است؛ استنتاج مانند یافتن آن. این تمایز در نماد صوری تجسم یافته: اگر الگوریتم استنتاج i بتواند α را از KB مشتق کند، می‌نویسیم:

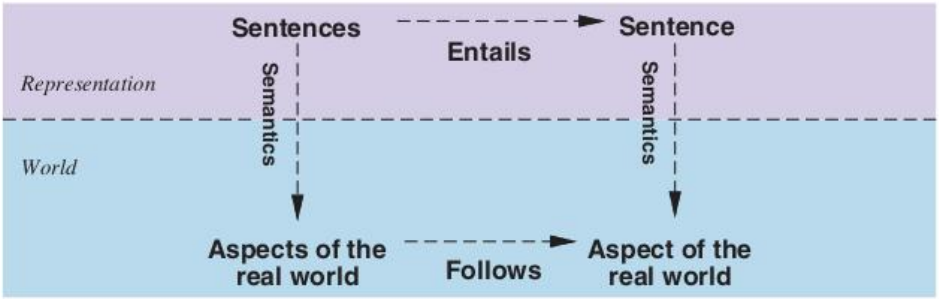
$$KB \vdash_i \alpha$$

که تلفظ می‌شود «α از KB توسط i مشتق شده» یا «α i را از KB مشتق می‌کند».

الگوریتم استنتاجی که فقط جمله‌های مستلزم را مشتق می‌کند «درست» (Sound) یا «حافظ صدق» (Truth-Preserving) نامیده می‌شود. درستی ویژگی بسیار مطلوبی است. رویهٔ استنتاج نادرست اساساً هنگام پیش‌رفتن چیزهایی از خودش می‌سازد — یافتن سوزن‌های ناموجود را اعلام می‌کند. به‌راحتی دیده می‌شود بررسی مدل، وقتی کاربردپذیر باشد،[5] رویهٔ درستی است.

ویژگی «کامل بودن» (Completeness) نیز مطلوب است: الگوریتم استنتاج کامل است اگر بتواند هر جمله‌ای را که مستلزم است مشتق کند. برای کوه‌های یونجه واقعی، که در گستره متناهی‌اند، بدیهی می‌نماید بررسی نظام‌مند همیشه می‌تواند تصمیم بگیرد آیا سوزن در کوه یونجه هست یا نه. اما برای بسیاری از پایگاه‌های دانش، کوه یونجهٔ پیامدها نامتناهی است و کامل بودن مسئلهٔ مهمی می‌شود.[6] خوشبختانه، رویه‌های استنتاج کاملی برای منطق‌هایی هست که به اندازهٔ کافی بیانی‌اند که بسیاری از پایگاه‌های دانش را مدیریت کنند.

فرایند استدلالی را توصیف کرده‌ایم که نتیجه‌هایش در هر جهانی که مقدمه‌ها در آن صادق‌اند تضمین‌شده صادق‌اند؛ به‌ویژه، اگر KB در جهان واقعی صادق باشد، هر جملهٔ α که از KB توسط رویهٔ استنتاج درست مشتق شود نیز در جهان واقعی صادق است. پس، در حالی که فرایند استنتاج روی «نحو» — پیکربندی‌های فیزیکی درونی مانند بیت‌های ثبات‌ها یا الگوهای برق‌های مغز — عمل می‌کند، این فرایند با رابطهٔ جهان واقعی متناظر است که بر اساس آن برخی جنبه‌های جهان واقعی به‌واسطهٔ صادق‌بودن جنبه‌های دیگر جهان واقعی صادق‌اند.[7] این مطابقت میان جهان و بازنمایی در شکل ۷.۶ نشان داده شده.



شکل ۷.۶ — جمله‌ها پیکربندی‌های فیزیکی عامل‌اند و استدلال فرایند ساختن پیکربندی‌های فیزیکی تازه از پیکربندی‌های قدیمی است. استدلال منطقی باید اطمینان دهد پیکربندی‌های تازه جنبه‌هایی از جهان را بازنمایی می‌کنند که واقعاً از جنبه‌هایی که پیکربندی‌های قدیمی بازنمایی می‌کنند برمی‌آیند.

آخرین مسئله برای بررسی «زمینه‌سازی» (Grounding) است — پیوند میان فرایندهای استدلال منطقی و محیط واقعی که عامل در آن وجود دارد. به‌ویژه، چگونه می‌دانیم KB در جهان واقعی صادق است؟ (بالاخره KB فقط «نحو» درون سر عامل است.) این پرسش فلسفی‌ای است که کتاب‌های بسیار زیادی درباره‌اش نوشته شده. (فصل ۲۷ را ببینید.) پاسخ ساده آن است که حسگرهای عامل پیوند را ایجاد می‌کنند. برای نمونه، عامل جهان وومپوس ما حسگر بو دارد. برنامهٔ عامل هرگاه بویی هست جملهٔ مناسبی می‌سازد. سپس، هرگاه آن جمله در پایگاه دانش است، در جهان واقعی صادق است. پس معنا و صدق جمله‌های ادراکی با فرایندهای حس‌کردن و ساخت جمله‌ای که آن‌ها را تولید می‌کنند تعریف می‌شود. دربارهٔ بقیهٔ دانش عامل، مانند باورش به اینکه وومپوس‌ها بوی گند را در خانه‌های مجاور ایجاد می‌کنند، چه؟ این بازنمایی مستقیم یک ادراک تکی نیست، بلکه قاعده‌ای عمومی است — شاید از تجربهٔ ادراکی مشتق شده، اما با گزاره‌ای از آن تجربه یکسان نیست. قواعد عمومی مانند این توسط فرایند ساخت جمله‌ای به نام یادگیری تولید می‌شوند که موضوع بخش پنجم است. یادگیری خطاپذیر است. ممکن است چنین باشد که وومپوس‌ها بوی گند ایجاد می‌کنند به‌جز ۲۹ فوریه در سال‌های کبیسه، که روز حمام‌گرفتن آن‌هاست. پس KB ممکن است در جهان واقعی صادق نباشد، اما با رویه‌های یادگیری خوب، دلیلی برای خوش‌بینی هست.

۷.۴ منطق گزاره‌ای: منطقی بسیار ساده

اکنون منطق گزاره‌ای (Propositional Logic) را ارائه می‌کنیم. نحو آن (ساختار جمله‌ها) و معناشناسی‌اش (شیوهٔ تعیین صدق جمله‌ها) را توصیف می‌کنیم. از این‌ها، الگوریتم نحوی ساده‌ای برای استنتاج منطقی مشتق می‌کنیم که مفهوم معنایی لازمه را پیاده می‌کند. همه‌چیز، البته، در جهان وومپوس رخ می‌دهد.

۷.۴.۱ نحو

نحو منطق گزاره‌ای جمله‌های مجاز را تعریف می‌کند. «جمله‌های اتمی» (Atomic Sentences) از یک «نماد گزاره» (Proposition Symbol) تکی تشکیل شده‌اند. هر چنین نمادی نشان‌دهندهٔ گزاره‌ای است که می‌تواند صادق یا کاذب باشد. از نمادهایی استفاده می‌کنیم که با حرف بزرگ آغاز می‌شوند و ممکن است حروف یا زیرنویس‌های دیگر داشته باشند؛ برای نمونه: $W_{1,3}$، R، Q، P و FacingEast. نام‌ها دلبخواهی‌اند اما اغلب طوری برگزیده می‌شوند که ارزش یادآورانه داشته باشند — از $W_{1,3}$ برای گزارهٔ «وومپوس در [1,3] است» استفاده می‌کنیم. (به یاد آورید نمادهایی مانند $W_{1,3}$ اتمی‌اند، یعنی W، 1 و 3 بخش‌های معنادار نماد نیستند.) دو نماد گزاره با معانی ثابت هست: True گزارهٔ همیشه‌صادق و False گزارهٔ همیشه‌کاذب است.

«جمله‌های پیچیده» (Complex Sentences) از جمله‌های ساده‌تر، با پرانتزها و عملگرهایی به نام «عوامل ربط منطقی» (Logical Connectives) ساخته می‌شوند. پنج عامل ربط رایج‌اند:

- **¬ (نقیض).** جمله‌ای مانند $\neg W_{1,3}$ «نقیض» $W_{1,3}$ (Negation) نامیده می‌شود. «لیترال» (Literal) یا جملهٔ اتمی است (لیترال مثبت) یا جملهٔ اتمیِ نقیض‌شده (لیترال منفی).
- **∧ (و).** جمله‌ای که عامل ربط اصلی‌اش ∧ است، مانند $P_{3,1} \wedge W_{1,3}$، «عطف» (Conjunction) نامیده می‌شود؛ بخش‌هایش «عطف‌شونده»ها (Conjuncts) هستند. (∧ شبیه «A» برای «And» است.)

- **∨ (یا).** جمله‌ای که عامل ربط اصلی‌اش ∨ است، مانند (W1,3 ∧ P3,1) ∨ W2,2، «فصل» (Disjunction) است؛ بخش‌هایش «فصل‌شونده»ها (Disjuncts) هستند — در این مثال، (W1,3 ∧ P3,1) و W2,2.
- **⇒ (دلالت).** جمله‌ای مانند (W1,3 ∧ P3,1) ⇒ ¬W2,2 «دلالت» (Implication) (یا شرطی) نامیده می‌شود. «مقدم» (Premise) یا «سابق» (Antecedent) آن (W1,3 ∧ P3,1) است و «تالی» (Conclusion) یا «لاحق» (Consequent) آن ¬W2,2. دلالت‌ها همچنین به‌عنوان «قواعد» (Rules) یا گزاره‌های اگر-آنگاه شناخته می‌شوند. نماد دلالت گاهی در کتاب‌های دیگر ⊃ یا → نوشته می‌شود.
- **⇔ (اگر و فقط اگر).** جملهٔ ¬W2,2 ⇔ W1,3 «دوسویه» (Biconditional) است.

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\
&\mid \neg \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

شکل ۷.۷ — دستور زبان BNF (بکوس–نائور) جمله‌های منطق گزاره‌ای، همراه با تقدم عملگرها، از بالاترین به پایین‌ترین. (نحو: جمله ← جملهٔ اتمی | جملهٔ پیچیده؛ جملهٔ اتمی ← True | False | P | Q | R | ...؛ جملهٔ پیچیده ← (جمله) | ¬ جمله | جمله ∧ جمله | جمله ∨ جمله | جمله ⇒ جمله | جمله ⇔ جمله؛ تقدم عملگرها: ¬, ∧, ∨, ⇒, ⇔)

شکل ۷.۷ دستور زبان صوری منطق گزاره‌ای را می‌دهد. (نماد BNF در صفحهٔ ۱۰۳۰ توضیح داده شده.) دستور زبان BNF با فهرست تقدم عملگرها تقویت شده تا ابهام هنگام استفاده از چند عملگر حذف شود. عملگر «نقیض» (¬) بالاترین تقدم را دارد، یعنی در جملهٔ ¬A ∧ B، ¬ محکم‌ترین چسبندگی را دارد و هم‌ارز (¬A) ∧ B را به ما می‌دهد نه ¬(A ∧ B). (نماد حساب معمولی هم همین است: -2+4 برابر ۲ است، نه ۶-.) هنگام مقتضی، از پرانتزها و کروشه‌ها نیز برای روشن‌کردن ساختار جملهٔ مورد نظر و بهبود خوانایی استفاده می‌کنیم.

۷.۴.۲ معناشناسی

پس از مشخص‌کردن نحو منطق گزاره‌ای، اکنون معناشناسی‌اش را مشخص می‌کنیم. معناشناسی قواعد تعیین صدق جمله را نسبت به مدل خاصی تعریف می‌کند. در منطق گزاره‌ای، مدل صرفاً مقدار صدق — صادق یا کاذب — را برای هر نماد گزاره تنظیم می‌کند. برای نمونه، اگر جمله‌های پایگاه دانش از نمادهای گزاره‌ای P2,2، P1,2 و P3,1 استفاده کنند، یکی از مدل‌های ممکن است:

```
m1 = {P1,2 = false, P2,2 = false, P3,1 = true}
```

با سه نماد گزاره، $2^3 = 8$ مدل ممکن هست — دقیقاً همان‌هایی که در شکل ۷.۵ تصویر شده‌اند. توجه کنید مدل‌ها اشیای صرفاً ریاضی‌اند و پیوند ضروری با جهان‌های وومپوس ندارند. P1,2 فقط نمادی است؛ ممکن است «در [1,2] گودالی هست» معنا دهد یا «من امروز و فردا در پاریس‌ام».

معناشناسی منطق گزاره‌ای باید مشخص کند چگونه مقدار صدق هر جمله را با توجه به مدلی محاسبه کنیم. این به‌صورت بازگشتی انجام می‌شود. همهٔ جمله‌ها از جمله‌های اتمی و پنج عامل ربط ساخته می‌شوند؛ پس باید مشخص کنیم چگونه صدق جمله‌های اتمی و چگونه صدق جمله‌های ساخته‌شده با هر یک از پنج عامل ربط را محاسبه کنیم. جمله‌های اتمی آسان‌اند:

- True در هر مدل صادق و False در هر مدل کاذب است.
- مقدار صدق هر نماد گزارهٔ دیگر باید مستقیماً در مدل مشخص شود. برای نمونه، در مدل m1 که پیش‌تر داده شد، P1,2 کاذب است.

برای جمله‌های پیچیده، پنج قاعده داریم که برای هر زیرجملهٔ P و Q (اتمی یا پیچیده) در هر مدل m برقرارند (در این‌جا «اگر و فقط اگر» به عنوان شرط دوطرفه به کار رفته است):

- $\neg P$ صادق است اگر و فقط اگر P در m کاذب باشد.
- $P \wedge Q$ صادق است اگر و فقط اگر هر دو P و Q در m صادق باشند.
- $P \vee Q$ صادق است اگر و فقط اگر دست‌کم یکی از P یا Q در m صادق باشد.
- $P \Rightarrow Q$ صادق است مگر آنکه P در m صادق و Q کاذب باشد.
- $P \Leftrightarrow Q$ صادق است اگر و فقط اگر P و Q هر دو در m صادق یا هر دو کاذب باشند.

قواعد را همچنین می‌توان با «جدول‌های صدق» (Truth Tables) بیان کرد که مقدار صدق جملهٔ پیچیده را برای هر تخصیص ممکن مقادیر صدق به مؤلفه‌هایش مشخص می‌کنند. جدول‌های صدق پنج عامل ربط در شکل ۷.۸ داده شده. از این جدول‌ها، مقدار صدق هر جملهٔ s را می‌توان نسبت به هر مدل m با ارزیابی بازگشتی ساده محاسبه کرد. برای نمونه، جملهٔ $\neg(P2,2 \vee P3,1) \wedge P1,2$، ارزیابی‌شده در m1، $\text{true} \wedge (\text{false} \vee \text{true}) = \text{true} \wedge \text{true} = \text{true}$ می‌دهد. تمرین TRUV.7 از شما می‌خواهد الگوریتم PL-TRUE?(s, m) را بنویسید که مقدار صدق جملهٔ منطق گزاره‌ای s را در مدل m محاسبه می‌کند.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

شکل ۷.۸ – جدول‌های صدق برای پنج عامل ربط منطقی. برای استفاده از جدول برای محاسبهٔ مثلاً مقدار P ∨ Q وقتی P صادق و Q کاذب است، نخست در چپ به دنبال ردیفی بگردید که P صادق و Q کاذب است (ردیف سوم). سپس در آن ردیف زیر ستون P ∨ Q به نتیجه نگاه کنید: true.

جدول‌های صدق برای «و»، «یا» و «نقیض» با شهودهای ما دربارهٔ واژه‌های انگلیسی هم‌خوانی نزدیک دارند. نکتهٔ اصلی سردرگمی احتمالی آن است که P ∨ Q وقتی صادق است که P صادق باشد یا Q صادق یا هر دو. عامل ربط متفاوتی به نام «یا-انحصاری» («xor») به اختصار) هست که وقتی هر دو فصل‌شونده صادق‌اند کاذب می‌دهد.[8] اجماعی بر نماد یا-انحصاری نیست؛ برخی انتخاب‌ها ∨ یا ≠ یا ⊕ است.

جدول صدق ⇒ ممکن است کاملاً با فهم شهودی «P دلالت بر Q دارد» یا «اگر P پس Q» هم‌خوان نباشد. برای یکی، منطق گزاره‌ای هیچ رابطهٔ علیت یا ارتباطی میان P و Q لازم نمی‌داند. جملهٔ «۵ فرد است دلالت بر این دارد که توکیو پایتخت ژاپن است» جملهٔ صادقی از منطق گزاره‌ای است (تحت تفسیر عادی)، حتی با آنکه جملهٔ انگلیسیِ قطعاً عجیبی است. نکتهٔ سردرگمی دیگر آن است که هر دلالتی وقتی سابقش کاذب است صادق است. برای نمونه، «۵ زوج است دلالت بر این دارد که سم باهوش است» صادق است، صرف‌نظر از اینکه سم باهوش باشد یا نه. این عجیب می‌نماید، اما اگر «P ⇒ Q» را چنین بیندیشید معنا می‌دهد: «اگر P صادق است، پس ادعا می‌کنم Q صادق است؛ وگرنه هیچ ادعایی نمی‌کنم.» تنها راه کاذب‌بودن این جمله آن است که P صادق اما Q کاذب باشد.

دوسویه، P ⇔ Q، هرگاه هر دو P ⇒ Q و Q ⇒ P صادق‌اند صادق است. در انگلیسی، این اغلب «P اگر و فقط اگر Q» نوشته می‌شود. بسیاری از قواعد جهان وومپوس بهتر است با ⇔ نوشته شوند. برای نمونه، خانه‌ای نسیم‌دار است اگر خانهٔ همسایه‌ای گودال داشته باشد، و خانه‌ای نسیم‌دار است فقط اگر خانهٔ همسایه‌ای گودال داشته باشد. پس به دوسویه نیاز داریم:

```
B1,1 ⇔ (P1,2 ∨ P2,1)
```

که B1,1 یعنی در [1,1] نسیمی هست.

۷.۴.۳ پایگاه دانش ساده

حال که معناشناسی منطق گزاره‌ای را تعریف کرده‌ایم، می‌توانیم پایگاه دانشی برای جهان وومپوس بسازیم. نخست بر جنبه‌های تغییرناپذیر جهان وومپوس تمرکز می‌کنیم و جنبه‌های متغیر را به بخش بعد می‌گذاریم. فعلاً به نمادهای زیر برای هر مکان [x,y] نیاز داریم:

- Px,y صادق است اگر در [x,y] گودالی هست.
- Wx,y صادق است اگر در [x,y] وومپوسی هست، مرده یا زنده.
- Bx,y صادق است اگر در [x,y] نسیمی هست.
- Sx,y صادق است اگر در [x,y] بوی گندی هست.
- Lx,y صادق است اگر عامل در مکان [x,y] است.

جمله‌هایی که می‌نویسیم برای مشتق‌کردن ¬P1,2 (در [1,2] گودالی نیست)، چنان‌که در بخش ۷.۳ به‌صورت غیررسمی انجام شد، کافی خواهند بود. هر جمله را Ri برچسب می‌زنیم تا بتوانیم به آن‌ها ارجاع دهیم:

- در [1,1] گودالی نیست:

```
R1: ¬P1,1
```

- خانه‌ای نسیم‌دار است اگر و فقط اگر در خانهٔ همسایه‌ای گودالی باشد. این باید برای هر خانه گفته شود؛ فعلاً فقط خانه‌های مرتبط را شامل می‌شویم:

```
R2: B1,1 ⇔ (P1,2 ∨ P2,1)
R3: B2,1 ⇔ (P1,1 ∨ P2,2 ∨ P3,1)
```

- جمله‌های پیشین در همهٔ جهان‌های وومپوس صادق‌اند. حال ادراک‌های نسیم را برای دو خانهٔ نخستِ بازدیدشده در جهانِ خاصی که عامل در آن است شامل می‌شویم، و به وضعیت شکل ۷.۳(ب) می‌رسیم.

```
R4: ¬B1,1
R5: B2,1
```

۷.۴.۴ رویهٔ استنتاج ساده

هدف ما اکنون تصمیم‌گیری دربارهٔ اینکه آیا KB ⊨ α برای جملهٔ α است. برای نمونه، آیا ¬P1,2 از KB ما مستلزم است؟ نخستین الگوریتم استنتاج ما رویکرد بررسی-مدلی است که پیاده‌سازی مستقیم تعریف لازمه است: مدل‌ها را برشمار و بررسی کن α در هر مدلی که KB در آن صادق است صادق است. مدل‌ها تخصیص‌های صادق یا کاذب به هر نماد گزاره هستند. به مثال جهان وومپوس‌مان بازگردیم، نمادهای گزاره‌ای مرتبط عبارت‌اند از P2,1، P1,2، P1,1، B2,1، B1,1 P2,2 و P3,1. با هفت نماد، 2^7 = ۱۲۸ مدل ممکن هست؛ در سه تای آن‌ها KB صادق است (شکل ۷.۹). در آن سه مدل، ¬P1,2 صادق است، پس در [1,2] گودالی نیست. از سوی دیگر، P2,2 در دو تا از سه مدل صادق و در یکی کاذب است، پس هنوز نمی‌توانیم بگوییم آیا در [2,2] گودالی هست.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

شکل ۷.۹ – جدول صدقی که برای پایگاه دانشِ داده‌شده در متن ساخته شده. KB صادق است اگر R1 تا R5 صادق باشند، که فقط در ۳ تا از ۱۲۸ ردیف رخ می‌دهد (آن‌هایی که در ستون راست زیرخط دارند). در هر ۳ ردیف، P1,2 کاذب است، پس در [1,2] گودالی نیست. از سوی دیگر، ممکن است در [2,2] گودالی باشد (یا نباشد).

شکل ۷.۹ به شکل دقیق‌تری استدلالِ نشان‌داده‌شده در شکل ۷.۵ را بازتولید می‌کند. الگوریتم عمومی برای تصمیم‌گیری دربارهٔ لازمه در منطق گزاره‌ای در شکل ۷.۱۰ نشان داده شده. مانند الگوریتم BACKTRACKING-SEARCH در صفحهٔ ۱۹۲، TT-ENTAILS? شمارش بازگشتی فضای متناهی تخصیص‌ها به نمادها را انجام می‌دهد. الگوریتم درست است، زیرا مستقیماً تعریف لازمه را پیاده می‌کند، و کامل است، زیرا برای هر KB و α کار می‌کند و همیشه پایان می‌یابد – فقط مدل‌های متناهی برای بررسی هست.

البته «متناهی» همیشه همان «اندک» نیست. اگر KB و α در مجموع n نماد داشته باشند، 2^n مدل هست. پس پیچیدگی زمانی الگوریتم $O(2^n)$ است. (پیچیدگی فضایی فقط $O(n)$ است، زیرا شمارش عمق-اول است.) بعداً در این فصل الگوریتم‌هایی نشان می‌دهیم که در بسیاری از موارد بسیار کارآمدترند. متأسفانه، لازمهٔ گزاره‌ای هم‌NP-کامل است (یعنی احتمالاً آسان‌تر از NP-کامل نیست – پیوست الف را ببینید)، پس هر الگوریتم استنتاج شناخته‌شده برای منطق گزاره‌ای پیچیدگی بدترین حالتی ◭ دارد که در اندازهٔ ورودی نمایی است.

```
                                function TT-ENTAILS?(KB, α) returns true or false
inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic

          symbols ← a list of the proposition symbols in KB and α
                        return TT-CHECK-ALL(KB, α, symbols, {})

function TT-CHECK-ALL(KB, α, symbols, model) returns true or false
                                            if EMPTY?(symbols) then
          if PL-TRUE?(KB, model) then return PL-TRUE?(α, model)
      else return true // when KB is false, always return true
                                                              else
                                            P ← FIRST(symbols)
                                         rest ← REST(symbols)
          return (TT-CHECK-ALL(KB, α, rest, model ∪ {P = true})
                                                        and
        (TT-CHECK-ALL(KB, α, rest, model ∪ {P = false})
```

شکل ۷.۱۰ — الگوریتم شمارش جدول-صدق برای تصمیم‌گیری دربارهٔ لازمهٔ گزاره‌ای. (TT مخفف جدول صدق است.) مانند BACKTRACKING-SEARCH در شکل ۶.۵، TT-CHECK-ALL در فضای تخصیص‌های جزئی به نمادها عقبگرد انجام می‌دهد (که در متغیر model ارزیابی می‌شود). تابع PL-TRUE? مقدار صدق جمله را در مدلی که مقادیری برای همهٔ نمادهای جمله دارد بازمی‌گرداند.

۷.۵ اثبات قضیهٔ گزاره‌ای

تاکنون نشان داده‌ایم چگونه لازمه را با بررسی مدل تعیین کنیم: برشمردن مدل‌ها و نشان‌دادن اینکه جمله باید در همهٔ مدل‌ها برقرار باشد. در این بخش نشان می‌دهیم چگونه لازمه را با «اثبات قضیه» (Theorem Proving) انجام دهیم — اعمال قواعد استنتاج مستقیماً بر جمله‌های پایگاه دانش برای ساختن برهانی از جملهٔ مورد نظر بدون مراجعه به مدل‌ها. اگر شمار مدل‌ها زیاد اما طول برهان کوتاه باشد، اثبات قضیه می‌تواند از بررسی مدل کارآمدتر باشد.

پیش از شیرجه در جزئیات الگوریتم‌های اثبات قضیه، به چند مفهوم اضافی مرتبط با لازمه نیاز خواهیم داشت. نخستین مفهوم «هم‌ارزی منطقی» (Logical Equivalence) است: دو جملهٔ α و β هم‌ارز منطقی‌اند اگر در همان مجموعه مدل‌ها صادق باشند. این را α ≡ β می‌نویسیم. (توجه کنید ≡ برای ادعا دربارهٔ جمله‌ها استفاده می‌شود، در حالی که ⇔ به‌عنوان بخشی از جمله استفاده می‌شود.) برای نمونه، به‌راحتی می‌توانیم (با جدول‌های صدق) نشان دهیم P ∧ Q و Q ∧ P هم‌ارز منطقی‌اند؛ هم‌ارزی‌های دیگر در شکل ۷.۱۱ نشان داده شده. این هم‌ارزی‌ها در منطق همان نقشی را بازی می‌کنند که اتحادهای حسابی در ریاضیات معمولی. تعریف جایگزین هم‌ارزی چنین است: هر دو جملهٔ α و β هم‌ارزند اگر و فقط اگر هر یک مستلزم دیگری باشد:

$$\alpha \equiv \beta \text{ اگر و فقط اگر } \alpha \models \beta \text{ و } \beta \models \alpha$$

دومین مفهومی که نیاز داریم «اعتبار» (Validity) است. جمله‌ای معتبر است اگر در همهٔ مدل‌ها صادق باشد. برای نمونه، جملهٔ P ∨ ¬P معتبر است. جمله‌های معتبر همچنین به‌عنوان «تاتولوژی» (Tautologies) شناخته می‌شوند — آن‌ها لزوماً صادق‌اند. چون جملهٔ True در همهٔ مدل‌ها صادق است، هر جملهٔ معتبری با True هم‌ارز منطقی است.

جمله‌های معتبر به چه درد می‌خورند؟ از تعریف لازمه، می‌توانیم «قضیهٔ استنتاج» (Deduction Theorem) را مشتق کنیم که یونانیان باستان می‌شناختند: ⚠ برای هر جملهٔ α و β، α ⊨ β اگر و فقط اگر جملهٔ (α ⇒ β) معتبر باشد. (تمرین DEDU.7 برهان می‌خواهد.) از این رو، می‌توانیم تصمیم بگیریم α ⊨ β با بررسی اینکه (α ⇒ β) در هر مدل صادق است — که اساساً همان کاری است که الگوریتم استنتاج شکل ۷.۱۰ می‌کند — یا با اثبات اینکه (α ⇒ β) با True هم‌ارز است. برعکس، قضیهٔ استنتاج بیان می‌کند هر جملهٔ دلالتیِ معتبر، استنتاجی مشروع را توصیف می‌کند.

آخرین مفهومی که نیاز داریم «ارضاپذیری» (Satisfiability) است. جمله‌ای ارضاپذیر است اگر در مدلی صادق باشد، یا توسط مدلی ارضا شود. برای نمونه، پایگاه دانشِ پیش‌ترداده‌شده، (R1 ∧ R2 ∧ R3 ∧ R4 ∧ R5)، ارضاپذیر است، زیرا سه مدل هست که در آن‌ها صادق است، چنان‌که در شکل ۷.۹. ارضاپذیری را می‌توان با برشمردن مدل‌های ممکن تا یافتن مدلی که جمله را ارضا می‌کند بررسی کرد. مسئلهٔ تعیین ارضاپذیری جمله‌های منطق گزاره‌ای — مسئلهٔ SAT — نخستین مسئله‌ای بود که NP-کامل اثبات شد. بسیاری از مسائل علوم رایانه واقعاً مسائل ارضاپذیری‌اند. برای نمونه، همهٔ مسئله‌های ارضای قیود فصل ۶ می‌پرسند آیا قیدها توسط تخصیصی ارضا می‌شوند.

اعتبار و ارضاپذیری البته به هم متصل‌اند: α معتبر است اگر و فقط اگر ¬α ارضاپذیر نباشد؛ برعکس، α ارضاپذیر است اگر و فقط اگر ¬α معتبر نباشد. همچنین نتیجهٔ مفید زیر را داریم: ⚠

```
α ⊨ β اگر و فقط اگر جملهٔ (α ∧ ¬β) ارضاپذیر نباشد.
```

اثبات β از α با بررسی ارضاپذیرنبودن (α ∧ ¬β) دقیقاً با فن استاندارد اثبات ریاضیِ «برهان خلف» (Reductio ad Absurdum، لفظاً «تقلیل به امر محال») مطابقت دارد. همچنین «اثبات به‌وسیلهٔ ابطال» (Refutation) یا «اثبات به‌وسیلهٔ تناقض» (Contradiction) نامیده می‌شود. فرض می‌کنیم جملهٔ β کاذب است و نشان می‌دهیم این به تناقض با اصل‌های شناخته‌شدهٔ α می‌انجامد. این تناقض دقیقاً همان چیزی است که با گفتن اینکه جملهٔ (α ∧ ¬β) ارضاپذیر نیست مراد است.

۷.۵.۱ استنتاج و برهان‌ها

این بخش قواعد استنتاجی را پوشش می‌دهد که می‌توان برای مشتق‌کردن «برهان» (Proof) اعمال کرد — زنجیره‌ای از نتیجه‌ها که به هدف مطلوب می‌انجامد. شناخته‌شده‌ترین قاعده «مُدوس پوننس» (Modus Ponens، لاتین برای «حالت تأییدکننده») نامیده می‌شود و چنین نوشته می‌شود:

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

نماد بدان معناست که هرگاه جمله‌هایی به شکل α ⇒ β و α داده شوند، آنگاه جملهٔ β قابل استنتاج است. برای نمونه، اگر (WumpusAhead ∧ WumpusAlive) ⇒ Shoot و (WumpusAhead ∧ WumpusAlive) داده شوند، آنگاه Shoot قابل استنتاج است.

قاعدهٔ استنتاج مفید دیگر «حذف-عطف» (And-Elimination) است که می‌گوید از یک عطف، هر یک از عطف‌شونده‌ها قابل استنتاج است:

$$\frac{\alpha \wedge \beta}{\alpha}$$

برای نمونه، از (WumpusAhead ∧ WumpusAlive)، WumpusAlive قابل استنتاج است.

با در نظر گرفتن مقادیر صدق ممکن α و β، به‌راحتی می‌توان یک‌بار برای همیشه نشان داد مدوس پوننس و حذف-عطف درست‌اند. این قواعد را سپس می‌توان در هر نمونهٔ خاصی که کاربردپذیرند استفاده کرد و استنتاج‌های درستی تولید کرد بدون نیاز به برشمردن مدل‌ها.

همهٔ هم‌ارزی‌های منطقی شکل ۷.۱۱ را می‌توان به‌عنوان قواعد استنتاج استفاده کرد. برای نمونه، هم‌ارزی برای حذف دوسویه دو قاعدهٔ استنتاج زیر را می‌دهد:

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{(\beta \Rightarrow \alpha) \wedge (\alpha \Rightarrow \beta)} \qquad \frac{\alpha \Leftrightarrow \beta}{\alpha \Leftrightarrow \beta}$$

همهٔ قواعد استنتاج مثل این در هر دو جهت کار نمی‌کنند. برای نمونه، نمی‌توانیم مدوس پوننس را در جهت مخالف اجرا کنیم تا از α ⇒ β، β و α به دست آوریم.

بگذارید ببینیم این قواعد و هم‌ارزی‌های استنتاج چگونه در جهان وومپوس استفاده می‌شوند. با پایگاه دانشِ شامل R1 تا R5 آغاز می‌کنیم و نشان می‌دهیم چگونه ¬P1,2 را اثبات کنیم، یعنی در [1,2] گودالی نیست:

1. حذف دوسویه را بر R2 اعمال کن تا به دست آید:

```
R6: (B1,1 ⇒ (P1,2 ∨ P2,1)) ∧ ((P1,2 ∨ P2,1) ⇒ B1,1)
```

1. حذف-عطف را بر R6 اعمال کن تا به دست آید:

```
R7: ((P1,2 ∨ P2,1) ⇒ B1,1)
```

1. هم‌ارزی منطقی برای عکس‌نقیض می‌دهد:

```
R8: (¬B1,1 ⇒ ¬(P1,2 ∨ P2,1))
```

1. مدوس پوننس را با R8 و ادراک R4 (یعنی ¬B1,1) اعمال کن تا به دست آید:

```
R9: ¬(P1,2 ∨ P2,1)
```

1. قاعدهٔ دمورگان را اعمال کن و نتیجه را بده:

```
R10: ¬P1,2 ∧ ¬P2,1
```

یعنی نه [1,2] و نه [2,1] گودال دارد.

هر یک از الگوریتم‌های جستجوی فصل ۳ را می‌توان برای یافتن توالی گامی که برهانی مانند این می‌سازد استفاده کرد. فقط باید مسئلهٔ برهان را چنین تعریف کنیم:

- حالت آغازین: پایگاه دانش آغازین.
- کنش‌ها: مجموعهٔ کنش‌ها شامل همهٔ قواعد استنتاجِ اعمال‌شده بر همهٔ جمله‌هایی است که با نیمهٔ بالایی قاعدهٔ استنتاج همخوانی دارند.
- نتیجه: نتیجهٔ کنش، افزودن جملهٔ نیمهٔ پایینی قاعدهٔ استنتاج است.
- هدف: هدف حالتی است که شامل جمله‌ای است که می‌کوشیم اثبات کنیم.

پس جستجو برای برهان‌ها جایگزینی برای برشمردن مدل‌هاست. در بسیاری از موارد عملی، یافتن برهان می‌تواند کارآمدتر باشد، زیرا برهان می‌تواند گزاره‌های بی‌ربط را نادیده بگیرد، ⚠ مهم نیست چقدر از آن‌ها باشد. برای نمونه، برهانِ تازه‌داده‌شده که به ¬P2,1 ∧ P1,2 می‌انجامد به گزاره‌های P2,2، P1,1، B2,1 یا P3,1 اشاره نمی‌کند. آن‌ها را می‌توان نادیده گرفت، زیرا گزارهٔ هدف، P1,2، فقط در جملهٔ R2 ظاهر می‌شود؛ دیگر گزاره‌های R2 فقط در R4 و R2 ظاهر می‌شوند؛ پس R1، R3 و R5 هیچ ارتباطی با برهان ندارند. همین حتی اگر یک میلیون جملهٔ دیگر به پایگاه دانش اضافه می‌کردیم برقرار می‌بود؛ الگوریتم سادهٔ جدول-صدق اما توسط انفجار نمایی مدل‌ها مغلوب می‌شد.

هم‌ارزی	نام
(α∧β) ≡ (β∧α)	جابه‌جایی ∧
(α∨β) ≡ (β∨α)	جابه‌جایی ∨
((α∧β)∧γ) ≡ (α∧(β∧γ))	شرکت‌پذیری ∧
((α∨β)∨γ) ≡ (α∨(β∨γ))	شرکت‌پذیری ∨
α ≡ ¬(¬α)	حذف نقیض دوگانه
(α ⇒ β) ≡ (¬β ⇒ ¬α)	عکس‌نقیض
(α ⇒ β) ≡ (¬α ∨ β)	حذف دلالت

هم‌ارزی	نام
$((\beta \Rightarrow \alpha) \wedge (\alpha \Rightarrow \beta)) \equiv (\alpha \Leftrightarrow \beta)$	حذف دوسویه
$(\alpha \vee \neg\beta\neg) \equiv (\alpha \wedge \beta)\neg$	دمورگان
$(\alpha \wedge \neg\beta\neg) \equiv (\alpha \vee \beta)\neg$	دمورگان
$((\alpha \wedge \gamma) \vee (\alpha \wedge \beta)) \equiv (\alpha \wedge (\beta \vee \gamma))$	توزیع‌پذیری ∧ بر ∨
$((\alpha \vee \gamma) \wedge (\alpha \vee \beta)) \equiv (\alpha \vee (\beta \wedge \gamma))$	توزیع‌پذیری ∨ بر ∧

شکل ۷.۱۱ — هم‌ارزی‌های منطقی استاندارد. نمادهای α، β و γ جمله‌های دلخواه منطق گزاره‌ای‌اند.

یکی از ویژگی‌های پایانی سامانه‌های منطقی «یکنواختی» (Monotonicity) است که می‌گوید مجموعهٔ جمله‌های مستلزم با افزودن اطلاعات به پایگاه دانش فقط می‌تواند افزایش یابد.[9] برای هر جملهٔ α و β:

```
اگر KB ⊨ α آنگاه KB ∧ β ⊨ α
```

برای نمونه، فرض کنیم پایگاه دانش ادعای اضافی β را در بر دارد که می‌گوید دقیقاً هشت گودال در جهان هست. این دانش ممکن است به عامل در نتیجه‌گیری‌های اضافی کمک کند، اما نمی‌تواند نتیجهٔ α ازپیش‌استنتاج‌شده را باطل کند — مانند نتیجهٔ اینکه در [1,2] گودالی نیست. یکنواختی بدان معناست قواعد استنتاج را می‌توان هرگاه مقدمه‌های مناسبی در پایگاه دانش یافت شد اعمال کرد — نتیجهٔ قاعده باید صرف‌نظر از آنچه دیگر در پایگاه دانش هست برآید.

۷.۵.۲ برهان با حل

استدلال کرده‌ایم قواعد استنتاجِ تاکنون‌پوشش‌داده‌شده درست‌اند، اما دربارهٔ مسئلهٔ کامل بودن برای الگوریتم‌های استنتاجی که از آن‌ها استفاده می‌کنند بحث نکرده‌ایم. الگوریتم‌های جستجو مانند جستجوی تعمیق تکراری (صفحهٔ ۸۱) به این معنا کامل‌اند که هر هدف قابل دسترسی را خواهند یافت، اما اگر قواعد استنتاجِ در دسترس ناکافی باشند، هدف قابل دسترسی نیست — هیچ برهانی وجود ندارد که فقط از آن قواعد استنتاج استفاده کند. برای نمونه، اگر قاعدهٔ حذف دوسویه را حذف می‌کردیم، برهان بخش پیشین به نتیجه نمی‌رسید. بخش کنونی قاعدهٔ استنتاج تکی‌ای معرفی می‌کند، «حل» (Resolution)، که با هر الگوریتم جستجوی کاملی، الگوریتم استنتاج کاملی به دست می‌دهد.

با نسخهٔ ساده‌ای از قاعدهٔ حل در جهان وومپوس آغاز می‌کنیم. گام‌های منتهی به شکل ۷.۴(الف) را در نظر بگیرید: عامل از [2,1] به [1,1] بازمی‌گردد و سپس به [1,2] می‌رود، جایی که بوی گند ادراک می‌کند، اما نسیمی نه. واقعیت‌های زیر را به پایگاه دانش می‌افزاییم:

```
R11: ¬B1,2
R12: B1,2 ⇔ (P1,1 ∨ P2,2 ∨ P1,3)
```

با همان فرایندی که پیش‌تر به R10 انجامید، اکنون می‌توانیم نبود گودال‌ها در [2,2] و [1,3] را مشتق کنیم (به یاد آورید [1,1] از قبل بی‌گودال شناخته شده):

```
R13: ¬P2,2
R14: ¬P1,3
```

همچنین می‌توانیم حذف دوسویه را بر R3 اعمال کنیم، سپس مدوس پوننس با R5، تا این واقعیت را به دست آوریم که در [1,1]، [2,2] یا [3,1] گودالی هست:

```
R15: P1,1 ∨ P2,2 ∨ P3,1
```

حال نخستین کاربرد قاعدهٔ حل فرا می‌رسد: لیتِرال ¬P2,2 در R13 با لیتِرال P2,2 در R15 حل می‌شود و «حاصل‌حل» (Resolvent) را می‌دهد:

```
R16: P1,1 ∨ P3,1
```

به انگلیسی: اگر در یکی از [1,1]، [2,2] و [3,1] گودالی هست و در [2,2] نیست، پس در [1,1] یا [3,1] است. به همین ترتیب، لیتِرال ¬P1,1 در R1 با لیتِرال P1,1 در R16 حل می‌شود و می‌دهد:

```
R17: P3,1
```

به انگلیسی: اگر در [1,1] یا [3,1] گودالی هست و در [1,1] نیست، پس در [3,1] است. این دو گام استنتاج آخر نمونه‌هایی از قاعدهٔ استنتاج «حل واحد» (Unit Resolution) هستند:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

که هر ℓ لیتِرال است و ℓ_i و m «لیتِرال‌های مکمل» (Complementary Literals)اند (یعنی یکی نقیض دیگری است). پس قاعدهٔ حل واحد یک «بند» (Clause) — فصلی از لیتِرال‌ها — و یک لیتِرال می‌گیرد و بند تازه‌ای تولید می‌کند. توجه کنید لیتِرال تکی را می‌توان فصلی از یک لیتِرال دید، که «بند واحد» (Unit Clause) نیز نامیده می‌شود.

قاعدهٔ حل واحد را می‌توان به قاعدهٔ کامل «حل» (Resolution) تعمیم داد:

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

که ℓ_i و m_j لیترال‌های مکمل‌اند. این می‌گوید حل دو بند می‌گیرد و بند تازه‌ای تولید می‌کند شامل همهٔ لیترال‌های دو بند اصلی به‌جز دو لیترال مکمل. برای نمونه:

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

فقط می‌توانید یک جفت لیترال مکمل را در هر بار حل کنید. برای نمونه، می‌توانیم P و ¬P را حل کنیم تا استنتاج کنیم:

$$\frac{P \vee \neg Q \vee R, \quad \neg P \vee Q}{Q \vee Q \vee R\neg}$$

اما نمی‌توانید هم‌زمان روی P و Q حل کنید تا R استنتاج شود. یک جنبهٔ فنی دیگر قاعدهٔ حل هست: بند حاصل باید فقط یک نسخه از هر لیترال داشته باشد.[10] حذف نسخه‌های چندگانهٔ لیترال‌ها «عامل‌گیری» (Factoring) نامیده می‌شود. برای نمونه، اگر $(A \vee B)$ را با $(A \vee \neg B)$ حل کنیم، $(A \vee A)$ به دست می‌آید که با عامل‌گیری فقط به A کاهش می‌یابد.

درستی قاعدهٔ حل را می‌توان به‌راحتی با در نظر گرفتن لیترال ℓ_i که مکمل لیترال m_j در بند دیگر است دید. اگر ℓ_i صادق باشد، m_j کاذب است و از این رو $m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n$ باید صادق باشد، زیرا $m_1 \vee \ldots \vee m_n$ داده شده. اگر ℓ_i کاذب باشد، آنگاه $\ell_1 \vee \ldots \vee \ell_{i-1} \vee \ell_{i+1} \vee \ldots \vee \ell_k$ باید صادق باشد، زیرا $\ell_1 \vee \ldots \vee \ell_k$ داده شده. حال ℓ_i یا صادق است یا کاذب، پس یکی یا دیگری از این نتیجه‌ها برقرار است — دقیقاً چنان‌که قاعدهٔ حل بیان می‌کند.

چیزی که دربارهٔ قاعدهٔ حل شگفت‌آورتر است آن است که پایهٔ خانواده‌ای از رویه‌های استنتاج کامل را تشکیل می‌دهد. اثبات‌کنندهٔ قضیهٔ مبتنی بر حل می‌تواند برای هر جملهٔ α و β در منطق گزاره‌ای تصمیم بگیرد آیا $\alpha \models \beta$. دو زیربخش بعد توضیح می‌دهند حل چگونه این کار را انجام می‌دهد.

شکل صوری عادی عطفی (CNF). قاعدهٔ حل فقط بر بندها (یعنی فصل‌های لیترال‌ها) اعمال می‌شود، پس به نظر می‌رسد فقط به پایگاه‌های دانش و پرس‌وجوهای متشکل از بندها مربوط باشد. پس چگونه می‌تواند به رویهٔ استنتاج کاملی برای کل منطق گزاره‌ای بینجامد؟ پاسخ آن است که هر $\blacktriangle$ جملهٔ منطق گزاره‌ای با عطفی از بندها هم‌ارز منطقی است.

جمله‌ای که به‌صورت عطفی از بندها بیان شده در «شکل صوری عادی عطفی» (Conjunctive Normal Form) یا CNF است (شکل ۷.۱۲ را ببینید). اکنون رویه‌ای برای تبدیل به CNF توصیف می‌کنیم. رویه را با تبدیل جملهٔ $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ به CNF نشان می‌دهیم. گام‌ها چنین‌اند:

1. $\Leftrightarrow$ را حذف کن، $\alpha \Leftrightarrow \beta$ را با $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$ جایگزین کن:

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

1. ⇒ را حذف کن، α ⇒ β را با ¬α ∨ β جایگزین کن:

```
(B1,1 ∨ (P1,2 ∨ P2,1)¬) ∧ (B1,1 ∨ P1,2 ∨ P2,1¬)
```

1. CNF ایجاب می‌کند ¬ فقط در لیترال‌ها ظاهر شود، پس با اعمال مکرر هم‌ارزی‌های زیر از شکل ۷.۱۱ «¬ را به درون می‌بریم»:

```
¬(¬α) ≡ α (حذف نقیض دوگانه)
¬(α ∧ β) ≡ (¬α ∨ ¬β) (دمورگان)
¬(α ∨ β) ≡ (¬α ∧ ¬β) (دمورگان)
```

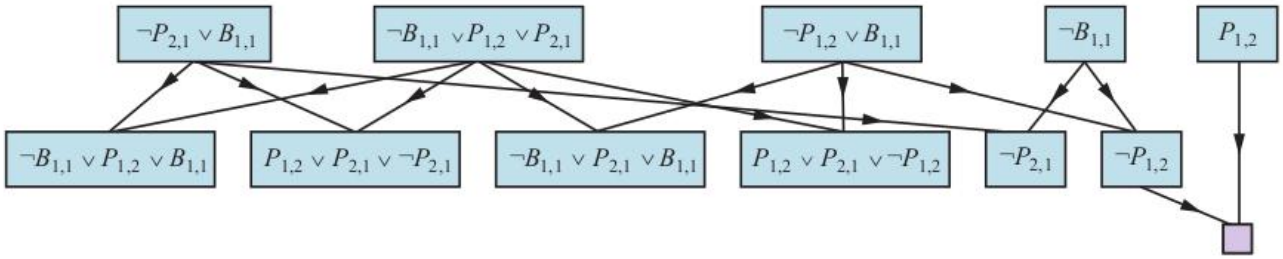
در مثال، فقط به یک بار اعمال قاعدهٔ آخر نیاز داریم:

```
(B1,1 ∨ (P1,2 ∧ ¬P2,1¬)) ∧ (B1,1 ∨ P1,2 ∨ P2,1¬)
```

1. حال جمله‌ای داریم شامل عملگرهای ۸ و ۷ تودرتو اعمال‌شده بر لیترال‌ها. قانون توزیع‌پذیری شکل ۷.۱۱ را اعمال می‌کنیم و هرجا ممکن است ۷ را بر ۸ توزیع می‌کنیم:

```
(P2,1 ∨ B1,1¬) ∧ (P1,2 ∨ B1,1¬) ∧ (B1,1 ∨ P1,2 ∨ P2,1¬)
```

جملهٔ اصلی اکنون در CNF است، به‌صورت عطفی از سه بند. خواندنش بسیار دشوارتر است، اما می‌تواند به‌عنوان ورودی رویهٔ حل استفاده شود.



شکل ۷.۱۲ — دستور زبان برای شکل عادی عطفی، بندهای هورن و بندهای معین.

الگوریتم حل. رویه‌های استنتاج مبتنی بر حل با استفاده از اصل اثبات به‌وسیلهٔ تناقض که در صفحهٔ ۲۲۳ معرفی شد کار می‌کنند. یعنی برای نشان‌دادن KB ⊨ α، نشان می‌دهیم (KB ∧ ¬α) ارضاپذیر نیست. این کار را با اثبات تناقض انجام می‌دهیم.

الگوریتم حل در شکل ۷.۱۳ نشان داده شده. نخست، (KB ∧ ¬α) به CNF تبدیل می‌شود. سپس، قاعدهٔ حل بر بندهای حاصل اعمال می‌شود. هر جفتی که لیترال‌های مکمل دارد حل می‌شود تا بند تازه‌ای تولید شود که اگر از قبل حاضر نباشد به مجموعه افزوده می‌شود. فرایند ادامه می‌یابد تا یکی از دو اتفاق رخ دهد:

- بند تازه‌ای برای افزودن نیست، که در آن صورت KB مستلزم α نیست؛ یا،
- دو بند حل می‌شوند و بند خالی تولید می‌کنند، که در آن صورت KB مستلزم α است.

```
function PL-RESOLUTION(KB, α) returns true or false
inputs: KB, the knowledge base, a sentence in propositional logic
        α, the query, a sentence in propositional logic

clauses ← the set of clauses in the CNF representation of KB ∧ ¬α
                                                        {} ← new
                                                         loop do
                    for each pair of clauses Ci, Cj in clauses do
                                    resolvents ← PL-RESOLVE(Ci, Cj)
          if resolvents contains the empty clause then return true
                                            new ← new ∪ resolvents
                                    if new ⊆ clauses then return false
                                              clauses ← clauses ∪ new
```

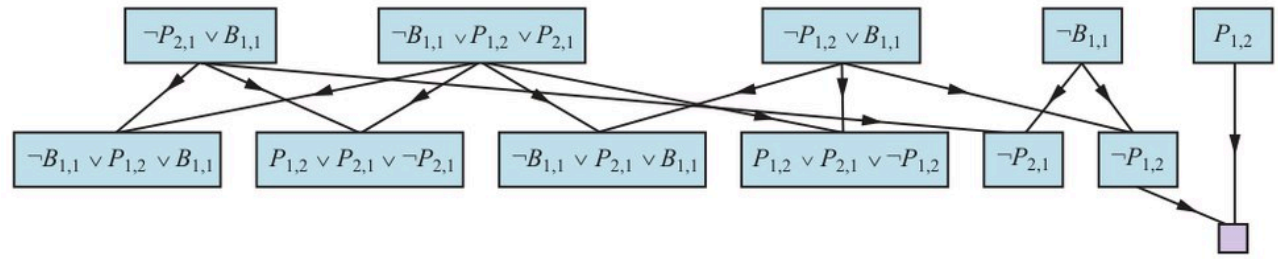
شکل ۷.۱۳ — الگوریتم حل ساده برای منطق گزاره‌ای. PL-RESOLVE مجموعهٔ همهٔ بندهای ممکنِ حاصل از حل دو بند ورودی‌اش را بازمی‌گرداند.

بند خالی — فصلی از هیچ فصل‌شونده‌ای — با False هم‌ارز است، زیرا فصل فقط اگر دست‌کم یکی از فصل‌شونده‌هایش صادق باشد صادق است. وانگهی، بند خالی فقط از حل دو بند واحد متناقض مانند P و ¬P پدید می‌آید.

می‌توانیم رویهٔ حل را بر استنتاج بسیار ساده‌ای در جهان وومپوس اعمال کنیم. وقتی عامل در [1,1] است، نسیمی نیست، پس نمی‌تواند گودال‌هایی در خانه‌های همسایه باشد. پایگاه دانش مرتبط چنین است:

```
KB = R2 ∧ R4 = (B1,1 ⇔ (P1,2 ∨ P2,1)) ∧ ¬B1,1
```

و می‌خواهیم α را اثبات کنیم که مثلاً ¬P1,2 است. وقتی (KB ∧ ¬α) را به CNF تبدیل می‌کنیم، بندهای نشان‌داده‌شده در بالای شکل ۷.۱۴ را به دست می‌آوریم. ردیف دوم شکل بندهایی را نشان می‌دهد که با حل جفت‌های ردیف اول به دست آمده‌اند. سپس، وقتی P1,2 با ¬P1,2 حل می‌شود، بند خالی را به دست می‌آوریم که به‌صورت مربع کوچکی نشان داده شده. بازرسی شکل ۷.۱۴ آشکار می‌کند بسیاری از گام‌های حل بی‌فایده‌اند. برای نمونه، بند B1,1 ∨ ¬B1,1 با P1,2 ∨ True هم‌ارز است که با True هم‌ارز است. استنتاج اینکه True صادق است چندان کمکی نمی‌کند. پس هر بندی که دو لیترال مکمل در آن ظاهر شوند را می‌توان دور ریخت.



شکل ۷.۱۴ – کاربرد جزئی PL-RESOLUTION بر استنتاج ساده‌ای در جهان وومپوس برای اثبات پرس‌وجوی ¬P1,2. هر یک از چهار بند چپ‌ترین ردیف بالا با هر یک از سه بند دیگر جفت می‌شود و قاعدهٔ حل برای تولید بندهای ردیف پایین اعمال می‌شود. می‌بینیم بندهای سوم و چهارم ردیف بالا ترکیب می‌شوند و بند ¬P1,2 را تولید می‌کنند که سپس با P1,2 حل می‌شود و بند خالی را تولید می‌کند، یعنی پرس‌وجو اثبات شده.

کامل بودن حل. برای پایان‌دادن بحث‌مان دربارهٔ حل، اکنون نشان می‌دهیم چرا PL-RESOLUTION کامل است. برای این کار، «بستار حل» (Resolution Closure) RC(S) مجموعه‌بندهای S را معرفی می‌کنیم که مجموعهٔ همهٔ بندهای قابل اشتقاق با اعمال مکرر قاعدهٔ حل بر بندهای S یا مشتق‌هایشان است. بستار حل همان چیزی است که PL-RESOLUTION به‌عنوان مقدار نهایی متغیر clauses محاسبه می‌کند. به‌راحتی دیده می‌شود RC(S) باید متناهی باشد: به لطف گام عامل‌گیری، فقط بندهای متمایزِ متناهی‌ای هست که می‌توان از نمادهای P1,...,Pk حاضر در S ساخت. از این رو، PL-RESOLUTION همیشه پایان می‌یابد.

قضیهٔ کامل بودن حل در منطق گزاره‌ای «قضیهٔ حل زمینی» (Ground Resolution Theorem) نامیده می‌شود:

اگر مجموعه‌بندی ارضاپذیر نباشد، آنگاه بستار حل آن بندها شامل بند خالی است.

این قضیه با نشان‌دادن عکس نقیضش اثبات می‌شود: اگر بستار RC(S) شامل بند خالی نباشد، آنگاه S ارضاپذیر است. در واقع، می‌توانیم مدلی برای S با مقادیر صدق مناسب برای P1,...,Pk بسازیم. رویهٔ ساخت چنین است:

برای i از ۱ تا k:

- اگر بندی در RC(S) شامل لیترال ¬Pi باشد و همهٔ لیترال‌های دیگرش تحت تخصیصِ برگزیده‌شده برای P1,...,Pi-1 کاذب باشند، آنگاه false را به Pi تخصیص بده.
- وگرنه، true را به Pi تخصیص بده.

این تخصیص به P1,...,Pk مدلی از S است. برای دیدن این، خلافش را فرض کنید – که در مرحله‌ای i از توالی، تخصیص نماد Pi باعث کاذب‌شدن بندی C می‌شود. برای وقوع این، باید چنین باشد که همهٔ لیترال‌های دیگر C از قبل توسط تخصیص‌های به P1,...,Pi-1 کاذب شده‌اند. پس C اکنون باید شبیه (false ∨ false ∨ ... ∨ false ∨ Pi) یا (false ∨ false ∨ ... ∨ false ∨ ¬Pi) باشد. اگر فقط یکی از این دو در RC(S) باشد، الگوریتم مقدار صدق مناسبی به Pi تخصیص می‌دهد تا C صادق شود، پس C فقط اگر هر دو بند در RC(S) باشند می‌تواند کاذب شود.

حال، چون RC(S) تحت حل بسته است، شامل حاصل‌حل این دو بند خواهد بود و آن حاصل‌حل همهٔ لیترال‌هایش را از قبل توسط تخصیص‌های به P1,...,Pi-1 کاذب خواهد داشت. این با فرض ما که نخستین بند کاذب‌شده در مرحلهٔ i ظاهر می‌شود تناقض دارد. از این رو، اثبات کرده‌ایم ساخت هرگز بندی در RC(S) را کاذب نمی‌کند؛ یعنی مدلی از RC(S) تولید می‌کند. سرانجام، چون S در RC(S) موجود است، هر مدل RC(S) مدلی از خودِ S است.

۷.۵.۳ بندهای هورن و بندهای معین

کامل بودن حل آن را به روش استنتاج بسیار مهمی تبدیل می‌کند. در بسیاری از موقعیت‌های عملی اما، قدرت کامل حل لازم نیست. برخی پایگاه‌های دانش جهان واقعی قیدهای خاصی بر شکل جمله‌هایی که شامل می‌شوند برآورده می‌کنند که آن‌ها را قادر می‌سازد از الگوریتم استنتاج محدودتر و کارآمدتری استفاده کنند.

یکی از این شکل‌های محدود «بند معین» (Definite Clause) است که فصلی از لیترال‌هاست که دقیقاً یکی از آن‌ها مثبت است. برای نمونه، بند (¬B1,1 ∨ ¬Breeze ∨ L1,1) بند معینی است، حال آنکه (¬P2,1 ∨ P1,2 ∨ B1,1) نیست، زیرا دو بند مثبت دارد.

کمی کلی‌تر «بند هورن» (Horn Clause) است که فصلی از لیترال‌هاست که حداکثر یکی از آن‌ها مثبت است. پس همهٔ بندهای معین بندهای هورن‌اند، و نیز بندهای بدون لیترال مثبت؛ این‌ها «بندهای هدف» (Goal Clauses) نامیده می‌شوند. بندهای هورن تحت حل بسته‌اند: اگر دو بند هورن را حل کنید، بند هورنی بازمی‌گیرید. ردهٔ دیگری «جملهٔ k-CNF» است که جملهٔ CNF است که هر بندش حداکثر k لیترال دارد.

پایگاه‌های دانشی که فقط بندهای معین دارند به سه دلیل جالب‌اند:

1. هر بند معین را می‌توان به‌صورت دلالتی نوشت که مقدمش عطفی از لیترال‌های مثبت و تالی‌اش لیترال مثبت تکی است. (تمرین DISJ.7 را ببینید.) برای نمونه، بند معین (¬B1,1 ∨ ¬Breeze ∨ L1,1) را می‌توان به‌صورت دلالت (Breeze ∧ L1,1) ⇒ B1,1 نوشت. در شکل دلالت، جمله آسان‌تر فهمیده می‌شود: می‌گوید اگر عامل در [1,1] است و ادراک نسیم هست، پس [1,1] نسیم‌دار است. در شکل هورن، مقدم «بدنه» (Body) و تالی «سر» (Head) نامیده می‌شود. جمله‌ای شامل لیترال مثبت تکی، مانند L1,1، «واقعیت» (Fact) نامیده می‌شود. آن را نیز می‌توان در شکل دلالت به‌صورت True ⇒ L1,1 نوشت، اما ساده‌تر است فقط L1,1 نوشته شود.
2. استنتاج با بندهای هورن را می‌توان با الگوریتم‌های «زنجیره‌سازی پیشرو» (Forward Chaining) و «زنجیره‌سازی پسرو» (Backward Chaining) انجام داد که در ادامه توضیح می‌دهیم. هر دو الگوریتم طبیعی‌اند، به این معنا که گام‌های استنتاج واضح و آسان برای دنبال‌کردن انسان‌هاست. این نوع استنتاج پایهٔ برنامه‌نویسی منطقی است که در فصل ۹ بحث می‌شود.
3. تصمیم‌گیری دربارهٔ لازمه با بندهای هورن را می‌توان در زمانی انجام داد که در اندازهٔ پایگاه دانش خطی است — شگفتی خوشایندی.

۷.۵.۴ زنجیره‌سازی پیشرو و پسرو

الگوریتم زنجیره‌سازی پیشرو PL-FC-ENTAILS?(KB, q) تعیین می‌کند آیا نماد گزاره‌ای تکی q — پرس‌وجو — توسط پایگاه دانشی از بندهای معین مستلزم است. از واقعیت‌های شناخته‌شده (لیترال‌های مثبت) در پایگاه دانش آغاز می‌کند. اگر همهٔ مقدمه‌های دلالتی شناخته شوند، نتیجه‌اش به مجموعهٔ واقعیت‌های شناخته‌شده افزوده می‌شود. برای نمونه، اگر L1,1 و Breeze شناخته شوند و (L1,1 ∧ Breeze) ⇒ B1,1 در پایگاه دانش باشد، آنگاه B1,1 را می‌توان افزود. این فرایند ادامه می‌یابد تا پرس‌وجوی q افزوده شود یا تا آنکه دیگر استنتاجی ممکن نباشد. الگوریتم در شکل ۷.۱۵ نشان داده شده؛ نکتهٔ اصلی برای به خاطر سپردن آن است که در زمان خطی اجرا می‌شود.

بهترین راه فهم الگوریتم از طریق مثال و تصویر است. شکل ۷.۱۶(الف) پایگاه دانش ساده‌ای از بندهای هورن با A و B به‌عنوان واقعیت‌های شناخته‌شده نشان می‌دهد. شکل ۷.۱۶(ب) همان پایگاه دانش را به‌صورت گراف AND–OR (فصل 4 را ببینید) ترسیم‌شده نشان می‌دهد. در گراف‌های AND–OR، یال‌های چندگانهٔ پیوسته با کمان، عطف را نشان می‌دهند — هر یال باید اثبات شود — در حالی که یال‌های چندگانه بدون کمان، فصل را نشان می‌دهند — هر یالی قابل اثبات است. به‌راحتی دیده می‌شود زنجیره‌سازی پیشرو در گراف چگونه کار می‌کند. برگ‌های شناخته‌شده (اینجا، A و B) تنظیم می‌شوند و استنتاج تا حد ممکن در گراف به بالا منتشر می‌شود. هرجا عطفی ظاهر شود، انتشار صبر می‌کند تا همهٔ عطف‌شونده‌ها شناخته شوند قبل از ادامه. خواننده تشویق می‌شود مثال را با جزئیات کار کند.

به‌راحتی دیده می‌شود زنجیره‌سازی پیشرو درست است: هر استنتاج اساساً کاربردی از مدوس پوننس است. زنجیره‌سازی پیشرو همچنین کامل است: هر جملهٔ اتمیِ مستلزم مشتق خواهد شد. ساده‌ترین راه دیدن این، در نظر گرفتن حالت نهایی جدول inferred (پس از آنکه الگوریتم به نقطهٔ ثابتی رسید که در آن استنتاج تازه‌ای ممکن نیست) است. جدول برای هر نمادِ استنتاج‌شده در طول فرایند true و برای همهٔ نمادهای دیگر false دارد. می‌توانیم جدول را مدلی منطقی ببینیم؛ وانگهی، هر بند معین در KB اصلی در این مدل صادق است. ◭

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count ← a table, where count[c] is the number of symbols in c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  agenda ← a queue of symbols, initially symbols known to be true in KB

  while agenda is not empty do
    p ← POP(agenda)
    if p = q then return true
    if inferred[p] = false then
      inferred[p] ← true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to agenda
  return false
```

شکل ۷.۱۵ — الگوریتم زنجیره‌سازی پیشرو برای منطق گزاره‌ای. دستور کار (agenda) نمادهای شناخته‌شده‌صادق را نگه می‌دارد. جدول count تعداد مقدمه‌های ناشناخته برای هر دلالت را ثبت می‌کند. inferred نمادهایی را که از پیش استنتاج شده‌اند دنبال می‌کند تا کار زائد انجام نشود.

برای دیدن این، خلافش را فرض کنید، یعنی بندی مانند a1 ∧ ... ∧ ak ⇒ b در مدل کاذب است. آنگاه a1 ∧ ... ∧ ak باید در مدل صادق و b در مدل کاذب باشد. اما این با فرض‌مان که الگوریتم به نقطهٔ ثابت رسیده تناقض دارد، زیرا اکنون مجاز می‌شدیم b را به KB بیفزاییم. پس می‌توانیم نتیجه بگیریم مجموعهٔ جمله‌های اتمیِ استنتاج‌شده در نقطهٔ ثابت، مدلی از KB اصلی تعریف می‌کند. وانگهی، هر جملهٔ اتمی q که توسط KB مستلزم است باید در همهٔ مدل‌هایش و به‌ویژه در این مدل صادق باشد. از این رو، هر جملهٔ اتمیِ مستلزم q باید توسط الگوریتم استنتاج شود.

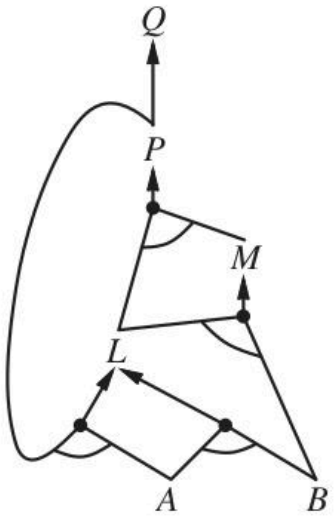
زنجیره‌سازی پیشرو نمونه‌ای از مفهوم عمومی «استدلال داده‌محور» (Data-Driven Reasoning) است — یعنی استدلالی که کانون توجه در آن از داده‌های شناخته‌شده آغاز می‌شود. می‌توان آن را درون عامل برای مشتق‌کردن نتیجه‌ها از ادراک‌های ورودی استفاده کرد، اغلب بدون پرس‌وجوی خاصی در ذهن. برای نمونه، عامل وومپوس ممکن است ادراک‌هایش را با الگوریتم زنجیره‌سازی پیشروِ افزایشی به پایگاه دانش TELL کند که در آن واقعیت‌های تازه می‌توانند به دستور کار افزوده شوند تا استنتاج‌های تازه آغاز شوند. در انسان‌ها، مقدار معینی استدلال داده‌محور هنگام رسیدن اطلاعات تازه رخ می‌دهد. برای نمونه، اگر داخل خانه باشم و بشنوم باران شروع به باریدن می‌کند، ممکن است به ذهنم برسد پیک‌نیک لغو خواهد شد. اما احتمالاً به ذهنم نخواهد رسید هفدهمین گلبرگ بزرگ‌ترین رز باغ همسایه خیس خواهد شد؛ انسان‌ها زنجیره‌سازی پیشرو را تحت کنترل دقیق نگه می‌دارند، مبادا با پیامدهای بی‌ربط غرق شوند.

الگوریتم زنجیره‌سازی پسرو (Backward Chaining)، چنان‌که نامش می‌گوید، از پرس‌وجو به عقب کار می‌کند. اگر پرس‌وجوی q شناخته‌شده‌صادق باشد، هیچ کاری لازم نیست. وگرنه، الگوریتم آن دلالت‌هایی را در پایگاه دانش می‌یابد که نتیجه‌شان q است. اگر همهٔ مقدمه‌های یکی از آن دلالت‌ها (با زنجیره‌سازی پسرو) قابل اثبات باشد، آنگاه q صادق است. وقتی بر پرس‌وجوی Q در شکل ۷.۱۶ اعمال شود، در گراف به پایین کار می‌کند تا به مجموعه‌ای از واقعیت‌های شناخته‌شده، A و B، برسد که پایهٔ برهان را تشکیل می‌دهند. الگوریتم اساساً با الگوریتم AND-OR-GRAPH-SEARCH در شکل 4.11 یکسان است. مانند زنجیره‌سازی پیشرو، پیاده‌سازی کارآمد در زمان خطی اجرا می‌شود.

زنجیره‌سازی پسرو شکلی از «استدلال هدف‌محور» (Goal-Directed Reasoning) است. برای پاسخ به پرسش‌های خاص مانند «حالا چه کنم؟» و «کلیدهایم کجاست؟» مفید است. اغلب، هزینهٔ زنجیره‌سازی پسرو بسیار کمتر از خطی نسبت به اندازهٔ پایگاه دانش است، زیرا فرایند فقط واقعیت‌های مرتبط را لمس می‌کند.

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$

(a)



(b)

شکل ۷.۱۶ – (الف) مجموعه‌ای از بندهای هورن. (ب) گراف AND–OR متناظر.

۷.۶ بررسی مدل گزاره‌ای مؤثر

در این بخش دو خانواده از الگوریتم‌های کارآمد برای استنتاج گزاره‌ای عمومی بر اساس بررسی مدل توصیف می‌کنیم: رویکردی مبتنی بر جستجوی عقبگرد و رویکردی مبتنی بر جستجوی محلی تپه‌نوردی. این الگوریتم‌ها بخشی از «فناوری» منطق گزاره‌ای‌اند. این بخش را می‌توان در خوانش نخست فصل مرور کرد.

الگوریتم‌هایی که توصیف می‌کنیم برای بررسی ارضاپذیری‌اند: مسئلهٔ SAT. (چنان‌که در بخش ۷.۵ یاد شد، آزمودن لازمه، $\alpha \models \beta$، با آزمودن ارضاپذیرنبودن $\alpha \wedge \neg\beta$ انجام می‌شود.) در صفحهٔ ۲۲۳ به پیوند میان یافتن مدل ارضاکننده برای جملهٔ منطقی و یافتن راه‌حل برای مسئلهٔ ارضای قیود اشاره کردیم، پس شاید تعجب‌آور نباشد که دو خانواده از الگوریتم‌های ارضاپذیری گزاره‌ای شباهت نزدیکی به الگوریتم‌های عقبگرد بخش 6.3 و الگوریتم‌های جستجوی محلی بخش 6.4 دارند. آن‌ها اما به خودی خود بسیار مهم‌اند، زیرا بسیاری از مسائل ترکیبیاتی علوم رایانه را می‌توان به بررسی ارضاپذیری جملهٔ گزاره‌ای تقلیل داد. هر بهبودی در الگوریتم‌های ارضاپذیری پیامدهای عظیمی برای توانایی ما در مدیریت پیچیدگی در حالت کلی دارد.

۷.۶.۱ الگوریتم عقبگرد کامل

نخستین الگوریتمی که در نظر می‌گیریم اغلب «الگوریتم دیویس–پاتنم» نامیده می‌شود، به نام مقالهٔ بنیادین مارتین دیویس و هیلاری پاتنم (۱۹۶۰). الگوریتم در واقع نسخه‌ای است که دیویس، لوگمن و لولند (۱۹۶۲) توصیف کردند، پس آن را DPLL به حروف اول هر چهار نویسنده می‌نامیم. DPLL جمله‌ای در شکل عادی عطفی – مجموعه‌ای از بندها – را به‌عنوان ورودی می‌گیرد. مانند BACKTRACKING-SEARCH و TT-ENTAILS?، اساساً شمارش بازگشتی و عمق-اول مدل‌های ممکن است. سه بهبود بر طرح سادهٔ TT-ENTAILS? تجسم می‌بخشد:

- **خاتمهٔ زودهنگام (Early Termination):** الگوریتم تشخیص می‌دهد آیا جمله باید صادق یا کاذب باشد، حتی با مدل نیمه‌کامل. بندی صادق است اگر هر لیترالی صادق باشد، حتی اگر دیگر لیترال‌ها هنوز مقدار صدق نداشته باشند؛ از این رو، کل جمله را می‌توان حتی پیش از کامل‌شدن مدل صادق داوری کرد. برای نمونه، جملهٔ (A ∨ B) ∧ (A ∨ C) اگر A صادق باشد صادق است، صرف‌نظر از مقادیر B و C. به همین ترتیب، جمله‌ای کاذب است اگر بندی کاذب باشد، که وقتی رخ می‌دهد که هر یک از لیترال‌هایش کاذب باشد. باز هم، این می‌تواند مدت‌ها پیش از کامل‌شدن مدل رخ دهد. خاتمهٔ زودهنگام از بررسی کل زیردرخت‌ها در فضای جستجو اجتناب می‌کند.
- **اکتشافی نماد ناب (Pure Symbol Heuristic):** «نماد ناب» (Pure Symbol) نمادی است که همیشه با همان «علامت» در همهٔ بندها ظاهر می‌شود. برای نمونه، در سه بند (A ∨ ¬B)، (B ∨ ¬C) و (C ∨ A)، نماد A ناب است، زیرا فقط لیترال مثبت ظاهر می‌شود، B ناب است، زیرا فقط لیترال منفی ظاهر می‌شود، و C ناب نیست. به‌راحتی دیده می‌شود اگر جمله‌ای مدلی داشته باشد، مدلی با نمادهای نابِ تخصیص‌یافته چنان دارد که لیترال‌هایشان صادق شوند، زیرا انجام این هرگز نمی‌تواند بندی را کاذب کند. توجه کنید در تعیین نابی نماد، الگوریتم می‌تواند بندهایی را که از قبل در مدلِ ساخته‌شده تاکنون صادق‌اند نادیده بگیرد. برای نمونه، اگر مدل شامل B=false باشد، بند (B ∨ ¬C) از قبل صادق است و در بندهای باقی‌مانده C فقط به‌صورت لیترال مثبت ظاهر می‌شود؛ پس C ناب می‌شود.
- **اکتشافی بند واحد (Unit Clause Heuristic):** بند واحد پیش‌تر به‌عنوان بندی با فقط یک لیترال تعریف شد. در بافت DPLL، همچنین به بندهایی اشاره دارد که همهٔ لیترال‌هایشان به‌جز یکی از قبل توسط مدل کاذب تخصیص داده شده‌اند. برای نمونه، اگر مدل شامل B=true باشد، (B ∨ ¬C) به ¬C ساده می‌شود که بند واحد است. بدیهی است برای صادق‌بودن این بند، C باید false تخصیص یابد. اکتشافی بند واحد همهٔ چنین نمادهایی را پیش از انشعاب بر بقیه تخصیص می‌دهد. یکی از پیامدهای مهم اکتشافی آن است که هر تلاشی برای اثبات (با ابطال) لیترالی که از قبل در پایگاه دانش است بلافاصله موفق خواهد شد (تمرین KNOW.7). همچنین توجه کنید تخصیص یک بند واحد می‌تواند بند واحد دیگری ایجاد کند — برای نمونه، وقتی C برابر false تخصیص یابد، (C ∨ A) بند واحد می‌شود و باعث می‌شود true به A تخصیص یابد. این «آبشار» از تخصیص‌های اجباری «انتشار واحد» (Unit Propagation) نامیده می‌شود. شبیه فرایند زنجیره‌سازی پیشرو با بندهای معین است و در واقع، اگر عبارت CNF فقط بندهای معین داشته باشد، DPLL اساساً زنجیره‌سازی پیشرو را بازتولید می‌کند. (تمرین DPLL.7 را ببینید.)

الگوریتم DPLL در شکل ۷.۱۷ نشان داده شده، که اسکلت ضروری فرایند جستجو را بدون جزئیات پیاده‌سازی می‌دهد.

```
function DPLL-SATISFIABLE?(s) returns true or false
     inputs: s, a sentence in propositional logic

clauses ← the set of clauses in the CNF representation of s
          symbols ← a list of the proposition symbols in s
                    return DPLL(clauses, symbols, {})

function DPLL(clauses, symbols, model) returns true or false
 if every clause in clauses is true in model then return true
if some clause in clauses is false in model then return false
        P, value ← FIND-PURE-SYMBOL(symbols, clauses, model)
if P is non-null then return DPLL(clauses, symbols – {P}, model ∪ {P = value})
                      P, value ← FIND-UNIT-CLAUSE(clauses, model)
if P is non-null then return DPLL(clauses, symbols – {P}, model ∪ {P = value})
                          P ← FIRST(symbols); rest ← REST(symbols)
                 return (DPLL(clauses, rest, model ∪ {P = true}) or
                  (DPLL(clauses, rest, model ∪ {P = false})
```

شکل ۷.۱۷ — الگوریتم DPLL برای بررسی ارضاپذیری جمله‌ای در منطق گزاره‌ای. ایده‌های پشت FIND-PURE-SYMBOL و FIND-UNIT-CLAUSE در متن توضیح داده شده‌اند؛ پیاده‌سازی‌های دقیق در تمرین‌ها پوشش داده می‌شوند.

آنچه شکل ۷.۱۷ نشان نمی‌دهد حربه‌هایی است که حل‌کننده‌های SAT را قادر می‌سازند به مسائل بزرگ مقیاس برسند. جالب است که بیشتر این حربه‌ها در واقع نسبتاً عمومی‌اند و قبلاً در لباس‌های دیگر دیده‌ایم:

1. **تحلیل مؤلفه** (چنان‌که با تاسمانی در CSPها دیدیم): وقتی DPLL مقادیر صدق را به متغیرها تخصیص می‌دهد، مجموعهٔ بندها ممکن است به زیرمجموعه‌های ناهم‌پوشی جدا شود، که «مؤلفه» نامیده می‌شوند و متغیر تخصیص‌نیافته‌ای مشترک ندارند. با راهی کارآمد برای تشخیص وقوع این، حل‌کننده می‌تواند با کار روی هر مؤلفه به‌طور جداگانه سرعت قابل توجهی به دست آورد.
2. **ترتیب متغیر و مقدار** (چنان‌که در بخش 6.3.1 برای CSPها دیدیم): پیاده‌سازی سادهٔ ما از DPLL از ترتیب متغیر دلخواه استفاده می‌کند و همیشه مقدار true را پیش از false می‌آزماید. اکتشافی درجه (صفحهٔ ۱۹۳ را ببینید) انتخاب متغیری را پیشنهاد می‌کند که بیشتر از همه در همهٔ بندهای باقی‌مانده ظاهر می‌شود.
3. **عقبگرد هوشمند** (چنان‌که در بخش 6.3.3 برای CSPها دیدیم): بسیاری از مسائل که با عقبگرد زمانی در ساعت‌ها زمان اجرا قابل حل نیستند، با عقبگرد هوشمندی که تمام‌راه تا نقطهٔ تعارضِ مرتبط عقب می‌رود در چند ثانیه حل می‌شوند. همهٔ حل‌کننده‌های SAT که عقبگرد هوشمند انجام می‌دهند از نوعی یادگیری بند تعارض برای ثبت تعارض‌ها استفاده می‌کنند تا بعداً در جستجو تکرار نشوند. معمولاً مجموعه‌ای با اندازهٔ محدود از تعارض‌ها نگه داشته می‌شود و موارد کم‌استفاده کنار گذاشته می‌شوند.
4. **راه‌اندازی مجدد تصادفی** (چنان‌که در صفحهٔ ۱۱۳ برای تپه‌نوردی دیدیم): گاهی به نظر می‌رسد اجرایی پیشرفتی نمی‌کند. در این صورت، می‌توانیم از بالای درخت جستجو از نو شروع کنیم، به‌جای تلاش برای ادامه. پس از راه‌اندازی مجدد، انتخاب‌های تصادفی متفاوتی (در انتخاب متغیر و مقدار) انجام می‌شود. بندهای آموخته‌شده در اجرای نخست پس از راه‌اندازی مجدد حفظ می‌شوند و می‌توانند به هرس فضای جستجو کمک کنند. راه‌اندازی مجدد تضمین نمی‌کند راه‌حل سریع‌تر یافت شود، اما واریانس زمان تا راه‌حل را کاهش می‌دهد.

5. **فهرست‌بندی هوشمند** (چنان‌که در بسیاری الگوریتم‌ها دیدیم): روش‌های سرعتبخشی که در خودِ DPLL استفاده می‌شوند، و نیز حربه‌های به‌کاررفته در حل‌کننده‌های مدرن، به فهرست‌بندی سریع چیزهایی مانند «مجموعهٔ بندهایی که متغیر Xi در آن‌ها به‌صورت لیترال مثبت ظاهر می‌شود» نیاز دارند. این تکلیف با این واقعیت پیچیده می‌شود که الگوریتم‌ها فقط به بندهایی علاقه‌مندند که هنوز توسط تخصیص‌های قبلی به متغیرها ارضا نشده‌اند، پس ساختارهای فهرست‌بندی باید هنگام پیش‌رفتن محاسبه به‌صورت پویا به‌روز شوند.

با این بهبودها، حل‌کننده‌های مدرن می‌توانند مسائل با ده‌ها میلیون متغیر را مدیریت کنند. آن‌ها حوزه‌هایی مانند تأیید سخت‌افزار و تأیید پروتکل‌های امنیتی را متحول کرده‌اند که قبلاً به برهان‌های پرزحمت و دستی هدایت‌شده نیاز داشتند.

۷.۶.۲ الگوریتم‌های جستجوی محلی

تاکنون چند الگوریتم جستجوی محلی در این کتاب دیده‌ایم، از جمله HILL-CLIMBING (صفحهٔ ۱۱۱) و SIMULATED-ANNEALING (صفحهٔ ۱۱۵). این الگوریتم‌ها را می‌توان مستقیماً بر مسائل ارضاپذیری اعمال کرد، به شرط آنکه تابع ارزیابی درستی برگزینیم. چون هدف یافتن تخصیصی است که هر بند را ارضا می‌کند، تابع ارزیابی که شمار بندهای ارضانشده را می‌شمارد کار را انجام می‌دهد. در واقع، این دقیقاً معیاری است که الگوریتم MIN-CONFLICTS برای CSPها استفاده می‌کند (صفحهٔ ۱۹۸). همهٔ این الگوریتم‌ها در فضای تخصیص‌های کامل گام برمی‌دارند و مقدار صدق یک نماد را در هر بار وارونه می‌کنند. فضا معمولاً شامل کمینه‌های محلی بسیاری است که برای گریز از آن‌ها اشکال گوناگون تصادفی بودن لازم است. در سال‌های اخیر، آزمایش‌های فراوانی برای یافتن تعادل خوب میان حرص و تصادفی بودن انجام شده.

یکی از ساده‌ترین و مؤثرترین الگوریتم‌های برآمده از همهٔ این کارها WALKSAT نامیده می‌شود (شکل ۷.۱۸). در هر تکرار، الگوریتم بند ارضانشده‌ای برمی‌گزیند و نمادی در بند برای وارون‌کردن برمی‌گزیند. بین دو راه انتخاب نماد برای وارون‌کردن تصادفی یکی را برمی‌گزیند: (۱) گام «کمترین-تعارض» که شمار بندهای ارضانشده در حالت تازه را کمینه می‌کند و (۲) گام «گام تصادفی» که نماد را تصادفی برمی‌گزیند.

```
function WALKSAT(clauses, p, max flips) returns a satisfying model or failure
                                inputs: clauses, a set of clauses in propositional logic
    p, the probability of choosing to do a "random walk" step, typically around 0.5
                                max flips, number of flips allowed before giving up

                        model ← a random assignment of true/false to the symbols in clauses
                                                                for i = 1 to max flips do
                                            if model satisfies clauses then return model
                    clause ← a randomly selected clause from clauses that is false in model
     with probability p flip the value in model of a randomly selected symbol from clause
           else flip whichever symbol in clause maximizes the number of satisfied clauses
                                                                        return failure
```

شکل ۷.۱۸ — الگوریتم WALKSAT برای بررسی ارضاپذیری با وارون‌کردن تصادفی مقادیر متغیرها. نسخه‌های بازگشتی به یک کمینهٔ محلی گیر می‌افتند؛ راه‌حل‌ها بهینه‌سازی تصادفی تپه‌نوردی با راه‌اندازی‌های مجدد تصادفی و گشت‌وگذار تصادفی را می‌آمیزند.

وقتی WALKSAT مدلی بازمی‌گرداند، جملهٔ ورودی واقعاً ارضاپذیر است، اما وقتی شکست بازمی‌گرداند، دو علت ممکن هست: یا جمله ارضاپذیر نیست یا باید به الگوریتم زمان بیشتری بدهیم. اگر max flips = ∞ و p > 0 تنظیم کنیم، WALKSAT سرانجام مدلی بازخواهد گرداند (اگر وجود داشته باشد)، زیرا گام‌های تصادفی سرانجام به راه‌حل خواهند خورد. افسوس، اگر max flips بی‌نهایت باشد و جمله ارضاپذیر نباشد، الگوریتم هرگز پایان نمی‌یابد!

به همین دلیل، WALKSAT وقتی مفیدتر است که انتظار داریم راه‌حلی وجود داشته باشد — برای نمونه، مسائل بحث‌شده در فصل‌های ۳ و ۶ معمولاً راه‌حل دارند. از سوی دیگر، WALKSAT نمی‌تواند همیشه ارضاپذیرنبودن را تشخیص دهد، که برای تصمیم‌گیری دربارهٔ لازمه لازم است. برای نمونه، عامل نمی‌تواند به‌طور قابل اعتمادی از WALKSAT برای اثبات امن‌بودن خانه‌ای در جهان وومپوس استفاده کند. در عوض، می‌تواند بگوید: «یک ساعت فکر کردم و نتوانستم جهان ممکنی درآورم که در آن خانه امن نباشد.» این ممکن است نشانگر تجربی خوبی باشد که خانه امن است، اما قطعاً برهان نیست.

۷.۶.۳ چشم‌انداز مسائل SAT تصادفی

برخی مسائل SAT از برخی دیگر سخت‌ترند. مسائل آسان را هر الگوریتمی می‌تواند حل کند، اما چون می‌دانیم SAT NP-کامل است، دست‌کم برخی نمونه‌های مسئله باید زمان اجرای نمایی لازم داشته باشند. در فصل ۶ کشفیات شگفت‌آوری دربارهٔ انواع خاصی از مسائل دیدیم. برای نمونه، مسئلهٔ n-وزیر — که برای الگوریتم‌های جستجوی عقبگرد بسیار دشوار پنداشته می‌شد — برای روش‌های جستجوی محلی مانند کمترین-تعارض به‌طرز پیش‌پاافتاده‌ای آسان از آب درآمد. این به‌سبب آن است که راه‌حل‌ها بسیار چگال در فضای تخصیص‌ها توزیع شده‌اند و هر تخصیص آغازینی تضمین‌شده راه‌حلی نزدیک دارد. پس n-وزیر آسان است، زیرا «کم‌قید» (Underconstrained) است.

وقتی به مسائل ارضاپذیری در شکل عادی عطفی نگاه می‌کنیم، مسئلهٔ کم‌قید مسئله‌ای است با بندهای نسبتاً اندکی که متغیرها را قید می‌کنند. برای نمونه، اینجا جملهٔ CNF-3 تصادفی‌تولیدشده با پنج نماد و پنج بند است:

```
(B ∨ E ∨ ¬C) ∧ (E ∨ ¬D ∨ B) ∧ (C ∨ ¬B ∨ E¬) ∧ (B ∨ ¬A ∨ ¬C) ∧ (D ∨ ¬B ∨ C¬)
```

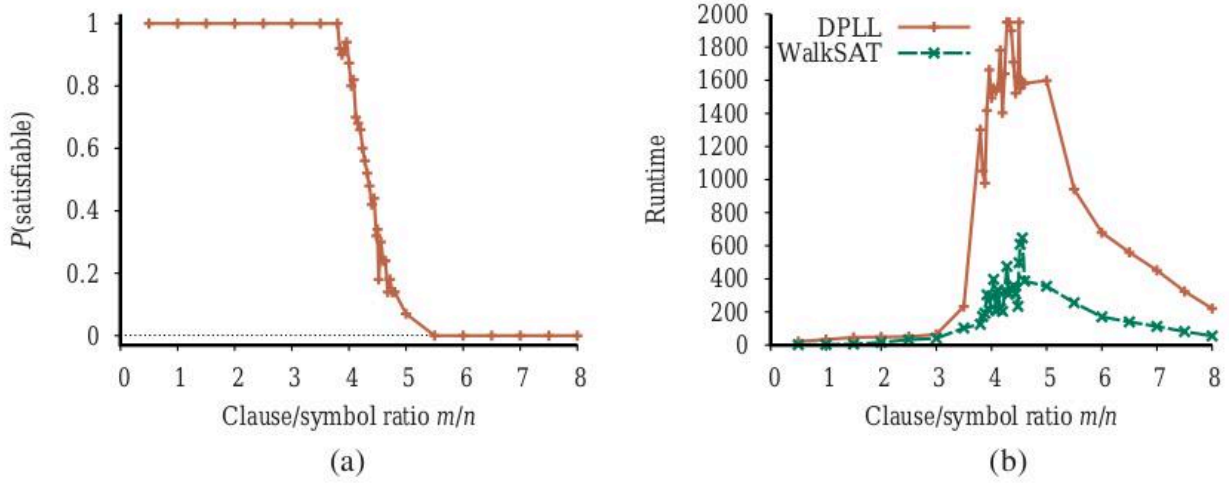
شانزده تا از ۳۲ تخصیص ممکن مدل‌های این جمله‌اند، پس به‌طور میانگین فقط دو حدس تصادفی برای یافتن مدلی لازم است. این مسئلهٔ ارضاپذیری آسانی است، مانند بیشتر چنین مسائل کم‌قیدی. از سوی دیگر، مسئلهٔ «پرقید» (Overconstrained) بندهای زیادی نسبت به شمار متغیرها دارد و به احتمال زیاد هیچ راه‌حلی ندارد. مسائل پرقید اغلب حل‌شان آسان است، زیرا قیدها به‌سرعت یا به راه‌حل می‌انجامند یا به بن‌بستی که گریزی از آن نیست.

برای فراتر رفتن از این شهودهای پایه، باید دقیقاً تعریف کنیم جمله‌های تصادفی چگونه تولید می‌شوند. نماد CNFk(m,n) جملهٔ k-CNF با m بند و n نماد را نشان می‌دهد، که در آن بندها به‌صورت یکنواخت، مستقل و بدون جای‌گذاری از میان همهٔ بندهای با k لیترال متفاوت، که تصادفی مثبت یا منفی‌اند، برگزیده می‌شوند. (نمادی نمی‌تواند دو بار در بندی ظاهر شود، و بندی نمی‌تواند دو بار در جمله‌ای ظاهر شود.)

با منبعی از جمله‌های تصادفی، می‌توانیم احتمال ارضاپذیری را اندازه بگیریم. شکل ۷.۱۹(الف) احتمال را برای CNF3(m,50) — یعنی جمله‌های با ۵۰ متغیر و ۳ لیتِرال در هر بند — به‌عنوان تابعی از نسبت بند/نماد، m/n، رسم می‌کند. چنان‌که انتظار داریم، برای m/n کوچک احتمال ارضاپذیری نزدیک ۱ است و در m/n بزرگ احتمال نزدیک ۰ است. احتمال در حدود m/n = 4.3 نسبتاً تند افت می‌کند. به‌صورت تجربی می‌یابیم «پرتگاه» تقریباً در همان جا می‌ماند (برای k=3) و با افزایش n تیزتر و تیزتر می‌شود.

به‌صورت نظری، «حدس آستانهٔ ارضاپذیری» (Satisfiability Threshold Conjecture) می‌گوید برای هر $k \geq 3$، نسبت آستانه‌ای r_k هست چنان‌که با رفتن n به بی‌نهایت، احتمال ارضاپذیربودن CNFk(rn,n) برای همهٔ مقادیر r زیر آستانه ۱ و برای همهٔ مقادیر بالای آن ۰ می‌شود. این حدس حتی برای حالت‌های خاص مانند $k = 3$ اثبات‌نشده باقی مانده. قضیه باشد یا نه، این نوع اثر آستانه‌ای قطعاً رایج است، هم برای مسائل ارضاپذیری و هم برای انواع دیگر مسائل NP-سخت.

حال که ایدهٔ خوبی داریم مسائل ارضاپذیر و ارضاپذیرنبودن کجا هستند، پرسش بعدی این است: مسائل سخت کجا هستند؟ معلوم می‌شود آن‌ها نیز اغلب در مقدار آستانه‌اند. شکل ۷.۱۹(ب) نشان می‌دهد مسائل ۵۰-نمادی در مقدار آستانهٔ 4.3 حدود ۲۰ برابر دشوارتر از مسائل با نسبت 3.3 هستند. مسائل کم‌قید آسان‌ترین‌اند (زیرا حدس زدن راه‌حل بسیار آسان است)؛ مسائل پرقید به اندازهٔ کم‌قید آسان نیستند، اما همچنان بسیار آسان‌تر از مسائل دقیقاً در آستانه‌اند.



شکل ۷.۱۹ — (الف) نمودار نشان‌دهندهٔ احتمال ارضاپذیربودن جملهٔ CNF-3 تصادفی با n=50 نماد، به‌عنوان تابعی از نسبت بند/نماد m/n. (ب) نمودار میانهٔ زمان اجرا (اندازه‌گیری‌شده بر حسب شمار تکرارها) برای هر دو DPLL و WALKSAT روی جمله‌های CNF-3 تصادفی. دشوارترین مسائل نسبت بند/نماد حدود 4.3 دارند.

۷.۷ عامل‌های مبتنی بر منطق گزاره‌ای

در این بخش، آنچه تاکنون آموخته‌ایم را گرد هم می‌آوریم تا عامل‌های جهان وومپوس بسازیم که از منطق گزاره‌ای استفاده می‌کنند. نخستین گام، توانمندکردن عامل برای استنتاج، تا حد ممکن، حالت جهان با توجه به تاریخچهٔ ادراک‌هایش است. این مستلزم نوشتن مدل منطقی کاملی از اثرات کنش‌هاست. سپس نشان می‌دهیم چگونه استنتاج منطقی را می‌توان توسط عامل در جهان وومپوس استفاده کرد. همچنین نشان می‌دهیم عامل چگونه می‌تواند جهان را کارآمد پیگیری کند بدون آنکه برای هر استنتاج به تاریخچهٔ ادراک برگردد. سرانجام، نشان می‌دهیم عامل چگونه می‌تواند از استنتاج منطقی برای ساختن برنامه‌هایی استفاده کند که تضمین‌شده به هدف‌هایش می‌رسند، به شرط آنکه پایگاه دانشش در جهان واقعی صادق باشد.

۷.۷.۱ حالت کنونی جهان

چنان‌که در آغاز فصل گفته شد، عامل منطقی با استنتاج اینکه چه کند از پایگاه دانشی از جمله‌ها دربارهٔ جهان عمل می‌کند. پایگاه دانش از اصل‌موضوعه‌ها — دانش عمومی دربارهٔ چگونگی کارکرد جهان — و جمله‌های ادراکیِ به‌دست‌آمده از تجربهٔ عامل در جهان خاصی تشکیل شده. در این بخش، بر مسئلهٔ استنتاج حالت کنونی جهان وومپوس تمرکز می‌کنیم — من کجا هستم، آن خانه امن است، و مانند آن.

جمع‌آوری اصل‌موضوعه‌ها را در بخش ۷.۴.۳ آغاز کردیم. عامل می‌داند خانهٔ آغازین گودالی ندارد (¬P1,1) و وومپوسی ندارد (¬W1,1). وانگهی، برای هر خانه می‌داند خانه نسیم‌دار است اگر و فقط اگر خانهٔ همسایه‌ای گودال داشته باشد؛ و خانه بدبو است اگر و فقط اگر خانهٔ همسایه‌ای وومپوس داشته باشد. پس مجموعهٔ بزرگی از جمله‌های شکل زیر را شامل می‌شویم:

```
B1,1 ⇔ (P1,2 ∨ P2,1)
S1,1 ⇔ (W1,2 ∨ W2,1)
...
```

عامل همچنین می‌داند دقیقاً یک وومپوس هست. این در دو بخش بیان می‌شود. نخست، باید بگوییم دست‌کم یک وومپوس هست:

```
W1,1 ∨ W1,2 ∨ ... ∨ W4,3 ∨ W4,4
```

سپس باید بگوییم حداکثر یک وومپوس هست. برای هر جفت مکان، جمله‌ای می‌افزاییم که می‌گوید دست‌کم یکی از آن‌ها باید بی‌وومپوس باشد:

```
¬W1,1 ∨ ¬W1,2¬
¬W1,1 ∨ ¬W1,3¬
...
¬W4,3 ∨ ¬W4,4¬
```

تا این‌جا خوب است. حال ادراک‌های عامل را در نظر بگیریم. از $S_{1,1}$ برای معنای «در [1,1] بوی گند هست» استفاده می‌کنیم؛ آیا می‌توانیم از گزارهٔ تکی Stench برای معنای «عامل بوی گند ادراک می‌کند» استفاده کنیم؟ متأسفانه نمی‌توانیم: اگر در گام زمانی قبلی بوی گندی نبود، ¬Stench از قبل ادعا شده بود و ادعای تازه صرفاً به تناقض می‌انجامید. مسئله وقتی حل می‌شود که دریابیم ادراک فقط چیزی را دربارهٔ زمان کنونی ادعا می‌کند. پس اگر گام زمانی (چنان‌که به MAKE-PERCEPT-SENTENCE در شکل ۷.۱ داده می‌شود) ۴ باشد، به‌جای Stench4، Stench را به پایگاه دانش می‌افزاییم — و به‌طور مرتب از هر تناقضی با ¬Stench3 اجتناب می‌کنیم. همین برای ادراک‌های نسیم، برخورد، درخشش و فریاد صادق است.

ایدهٔ مرتبط‌کردن گزاره‌ها با گام‌های زمانی به هر جنبه‌ای از جهان که در طول زمان تغییر می‌کند بسط می‌یابد. برای نمونه، پایگاه دانش آغازین شامل $L^0_{1,1}$ — عامل در زمان ۰ در خانهٔ [1,1] است — و نیز $HaveArrow^0$، $FacingEast^0$ و $WumpusAlive^0$ است. از اسم «روان» (Fluent) (از لاتین fluens، جاری) برای اشاره به جنبه‌ای از جهان که تغییر می‌کند استفاده می‌کنیم. «روان» مترادف «متغیر حالت» است، به معنای توصیف‌شده در بحث بازنمایی‌های عاملی در بخش 2.4.7 در صفحهٔ ۵۸. نمادهای مرتبط با جنبه‌های دائمی جهان به زیرنویس زمانی نیاز ندارند و گاهی «متغیرهای بی‌زمان» (Atemporal Variables) نامیده می‌شوند.

می‌توانیم ادراک‌های بوی گند و نسیم را مستقیماً به ویژگی‌های خانه‌هایی که در آن‌ها تجربه می‌شوند چنین پیوند دهیم.[11] برای هر گام زمانی t و هر خانهٔ [x,y]، ادعا می‌کنیم:

$$L^t_{x,y} \Rightarrow (Breeze^t \Leftrightarrow B_{x,y})$$
$$L^t_{x,y} \Rightarrow (Stench^t \Leftrightarrow S_{x,y})$$

حال، البته، به اصل‌موضوعه‌هایی نیاز داریم که به عامل امکان دهند روان‌هایی مانند $L^t_{x,y}$ را پیگیری کند. این روان‌ها در نتیجهٔ کنش‌های عامل تغییر می‌کنند، پس به زبان فصل ۳، باید مدل گذار جهان وومپوس را به‌صورت مجموعه‌ای از جمله‌های منطقی بنویسیم.

نخست به نمادهای گزاره‌ای برای رخداد کنش‌ها نیاز داریم. مانند ادراک‌ها، این نمادها با زمان فهرست‌بندی می‌شوند؛ پس $Forward^0$ یعنی عامل کنش Forward را در زمان ۰ اجرا می‌کند. طبق قرارداد، ادراک برای گام زمانیِ داده‌شده نخست رخ می‌دهد، سپس کنش آن گام زمانی، سپس گذار به گام زمانی بعد.

برای توصیف چگونگی تغییر جهان، می‌توانیم «اصل‌موضوعه‌های اثر» (Effect Axioms) بنویسیم که پیامد کنش را در گام زمانی بعد مشخص می‌کنند. برای نمونه، اگر عامل در زمان ۰ در مکان [1,1] رو به شرق باشد و Forward برود، نتیجه آن است که عامل در خانهٔ [2,1] است و دیگر در [1,1] نیست:

$$L^0_{1,1} \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1}) \quad (۷.۱)$$

برای هر گام زمانی ممکن، برای هر یک از ۱۶ خانه و هر یک از چهار جهت، به یکی از این جمله‌ها نیاز داریم. همچنین برای کنش‌های دیگر به جمله‌های مشابهی نیاز داریم: TurnLeft، Climb، Shoot، Grab و TurnRight.

فرض کنیم عامل تصمیم می‌گیرد در زمان ۰ Forward حرکت کند و این واقعیت را به پایگاه دانشش ادعا می‌کند. با اصل‌موضوعهٔ اثر معادلهٔ (۷.۱)، همراه با ادعاهای آغازین دربارهٔ حالت در زمان ۰، عامل اکنون می‌تواند استنتاج کند در [2,1] است. یعنی ASK(KB, $L^1_{2,1}$) = true. تا این‌جا خوب است. متأسفانه، اگر ASK(KB, $HaveArrow^1$) بپرسیم، پاسخ false است، یعنی عامل نمی‌تواند اثبات کند هنوز تیر را دارد؛ و نه می‌تواند اثبات کند ندارد! اطلاعات از دست رفته، زیرا اصل‌موضوعهٔ اثر از بیان آنچه در نتیجهٔ کنش تغییرنکرده باقی می‌ماند ناتوان است. نیاز به انجام این کار «مسئلهٔ قاب» (Frame Problem)[12] را پدید می‌آورد.

یکی از راه‌حل‌های ممکن برای مسئلهٔ قاب، افزودن «اصل‌موضوعه‌های قاب» (Frame Axioms) است که صریحاً همهٔ گزاره‌هایی را ادعا می‌کنند که یکسان می‌مانند. برای نمونه، برای هر زمان t خواهیم داشت:

$$Forward^t \Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1})$$
$$Forward^t \Rightarrow (WumpusAlive^t \Leftrightarrow WumpusAlive^{t+1})$$
$$...$$

که در آن هر گزاره‌ای را که از زمان t تا t+1 تحت کنش Forward بدون تغییر می‌ماند صریحاً ذکر می‌کنیم. هرچند عامل اکنون می‌داند پس از حرکت به جلو همچنان تیر را دارد و وومپوس نمرده یا زنده نشده، تکثیر اصل‌موضوعه‌های قاب به‌طرز چشمگیری ناکارآمد می‌نماید. در جهانی با m کنش متفاوت و n روان، مجموعهٔ اصل‌موضوعه‌های قاب به اندازهٔ O(mn) خواهد بود. این تجلی خاص مسئلهٔ قاب گاهی «مسئلهٔ قاب بازنمایشی» (Representational Frame Problem) نامیده می‌شود. این مسئله نقش مهمی در تاریخ هوش مصنوعی بازی کرد؛ آن را بیشتر در یادداشت‌های پایان فصل کاوش می‌کنیم.

مسئلهٔ قاب بازنمایشی مهم است، زیرا جهان واقعی روان‌های بسیار زیادی دارد، به‌بیان ملایم. خوشبختانه برای ما انسان‌ها، هر کنش معمولاً حداکثر شمار کوچکی k از آن روان‌ها را تغییر می‌دهد — جهان «محلی‌بودن» (Locality) نشان می‌دهد. حل مسئلهٔ قاب بازنمایشی مستلزم تعریف مدل گذار با مجموعه‌ای از اصل‌موضوعه‌ها به اندازهٔ O(mk) است نه O(mn). همچنین «مسئلهٔ قاب استنتاجی» (Inferential Frame Problem) هست: مسئلهٔ پیش‌افکنی نتایج برنامهٔ کنش t-گامی در زمان O(kt) نه O(nt).

راه‌حل مسئله شامل تغییر کانون توجه از نوشتن اصل‌موضوعه دربارهٔ کنش‌ها به نوشتن اصل‌موضوعه دربارهٔ روان‌هاست. پس برای هر روان F، اصل‌موضوعه‌ای خواهیم داشت که مقدار صدق F^{t+1} را بر حسب روان‌ها (شامل خودِ F) در زمان t و کنش‌هایی که ممکن است در زمان t رخ داده باشند تعریف می‌کند. حال، مقدار صدق F^{t+1} را می‌توان از دو راه تنظیم کرد: یا کنش در زمان t باعث صادق‌شدن F در t+1 می‌شود، یا F از قبل در زمان t صادق بود و کنش در زمان t باعث کاذب‌شدنش نمی‌شود. اصل‌موضوعه‌ای از این شکل «اصل‌موضوعهٔ حالت-جانشین» (Successor-State Axiom) نامیده می‌شود و این شکل را دارد:

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$$

یکی از ساده‌ترین اصل‌موضوعه‌های حالت-جانشین، مربوط به HaveArrow است. چون کنش بارگذاری دوباره‌ای نیست، بخش ActionCausesF^t حذف می‌شود و باقی می‌ماند:

$$\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \wedge \neg \text{Shoot}^t) \quad (۷.۲)$$

برای مکان عامل، اصل‌موضوعه‌های حالت-جانشین مفصل‌ترند. برای نمونه، $L^{t+1}_{1,1}$ صادق است اگر یا (الف) عامل از [1,2] هنگام رو به جنوب، یا از [2,1] هنگام رو به غرب، Forward حرکت کرده باشد؛ یا (ب) $L^t_{1,1}$ از قبل صادق بوده و کنش باعث حرکت نشده باشد (یا به‌سبب آنکه کنش Forward نبوده یا به‌سبب آنکه کنش به دیوار خورده). به‌صورت نوشته‌شده در منطق گزاره‌ای، این می‌شود:

$$L^{t+1}_{1,1} \Leftrightarrow (L^t_{1,1} \wedge (\neg \text{Forward}^t \vee \text{Bump}^{t+1})) \vee (L^t_{1,2} \wedge (\text{FacingSouth}^t \wedge \text{Forward}^t)) \vee (L^t_{2,1} \wedge (\text{FacingWest}^t \wedge \text{Forward}^t)) \quad (۷.۳)$$

تمرین SSAX.7 از شما می‌خواهد اصل‌موضوعه‌ها را برای روان‌های باقی‌ماندهٔ جهان وومپوس بنویسید.

با مجموعهٔ کاملی از اصل‌موضوعه‌های حالت-جانشین و دیگر اصل‌موضوعه‌های فهرست‌شده در آغاز این بخش، عامل قادر خواهد بود ASK کند و به هر پرسش قابل‌پاسخی دربارهٔ حالت کنونی جهان پاسخ دهد. برای نمونه، در بخش ۷.۲ توالی آغازین ادراک‌ها و کنش‌ها چنین است:

$$\begin{aligned}
&\text{Stench}^0 \wedge \neg\text{Breeze}^0 \wedge \neg\text{Glitter}^0 \wedge \neg\text{Bump}^0 \wedge \neg\text{Scream}^0; \text{Forward}^0 \neg \\
&\text{Stench}^1 \wedge \text{Breeze}^1 \wedge \neg\text{Glitter}^1 \wedge \neg\text{Bump}^1 \wedge \neg\text{Scream}^1; \text{TurnRight}^1 \neg \\
&\text{Stench}^2 \wedge \text{Breeze}^2 \wedge \neg\text{Glitter}^2 \wedge \neg\text{Bump}^2 \wedge \neg\text{Scream}^2; \text{TurnRight}^2 \neg \\
&\text{Stench}^3 \wedge \text{Breeze}^3 \wedge \neg\text{Glitter}^3 \wedge \neg\text{Bump}^3 \wedge \neg\text{Scream}^3; \text{Forward}^3 \neg \\
&\text{Stench}^4 \wedge \neg\text{Breeze}^4 \wedge \neg\text{Glitter}^4 \wedge \neg\text{Bump}^4 \wedge \neg\text{Scream}^4; \text{TurnRight}^4 \neg \\
&\text{Stench}^5 \wedge \neg\text{Breeze}^5 \wedge \neg\text{Glitter}^5 \wedge \neg\text{Bump}^5 \wedge \neg\text{Scream}^5; \text{Forward}^5 \neg \\
&\text{Stench}^6 \wedge \neg\text{Breeze}^6 \wedge \neg\text{Glitter}^6 \wedge \neg\text{Bump}^6 \wedge \neg\text{Scream}^6
\end{aligned}$$

در این نقطه، $\text{ASK}(KB, L^6_{1,2}) = \text{true}$ داریم، پس عامل می‌داند کجاست. وانگهی، $\text{ASK}(KB, W_{1,3}) = \text{true}$ و $\text{ASK}(KB, P_{3,1}) = \text{true}$، پس عامل وومپوس و یکی از گودال‌ها را یافته. مهم‌ترین پرسش برای عامل آن است که آیا خانه‌ای OK برای حرکت به آن است — یعنی آیا خانه از گودال یا وومپوس زنده آزاد است. افزودن اصل‌موضوعه‌هایی برای این راحت است و شکل زیر را دارند:

$$\text{OK}^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge \text{WumpusAlive}^t)$$

سرانجام، $\text{ASK}(KB, \text{OK}^6_{2,2}) = \text{true}$، پس خانهٔ [2,2] OK برای حرکت به آن است. در واقع، با الگوریتم استنتاج درست و کاملی مانند DPLL، عامل می‌تواند به هر پرسش قابل‌پاسخی دربارهٔ اینکه کدام خانه‌ها OK هستند پاسخ دهد — و این کار را فقط در چند میلی‌ثانیه برای جهان‌های وومپوس کوچک‌تا-متوسط انجام دهد.

حل مسئله‌های قاب بازنمایشی و استنتاجی گام بزرگی به جلوست، اما مسئلهٔ موذیانه‌ای باقی می‌ماند: باید تأیید کنیم همهٔ پیش‌شرط‌های لازم یک کنش برای اثر موردنظرش برقرارند. گفتیم کنش Forward عامل را به جلو می‌برد مگر آنکه دیواری سر راه باشد، اما استثناهای غیرعادی فراوان دیگری هست که می‌تواند باعث شکست کنش شود: عامل ممکن است زمین بخورد، سکتهٔ قلبی کند، توسط خفاش‌های غول‌پیکر برده شود و... مشخص‌کردن همهٔ این استثناها «مسئلهٔ شرایط‌مندی» (Qualification Problem) نامیده می‌شود. هیچ راه‌حل کاملی درون منطق نیست؛ طراحان سامانه باید از قضاوت خوب در تصمیم‌گیری دربارهٔ اینکه چقدر جزئی می‌خواهند در مشخص‌کردن مدل‌شان باشند و چه جزئیاتی را می‌خواهند بیرون بگذارند استفاده کنند. در فصل ۱۲ خواهیم دید نظریهٔ احتمال به ما امکان می‌دهد همهٔ استثناها را بدون نام‌بردن صریح از آن‌ها خلاصه کنیم.

۷.۷.۲ عامل ترکیبی

توانایی استنتاج جنبه‌های گوناگون حالت جهان را می‌توان نسبتاً سرراست با قواعد شرط–کنش (بخش 2.4.2 را ببینید) و با الگوریتم‌های حل مسئلهٔ فصل‌های ۳ و ۴ ترکیب کرد تا «عامل ترکیبی» (Hybrid Agent) برای جهان وومپوس تولید شود. شکل ۷.۲۰ یکی از راه‌های ممکن این کار را نشان می‌دهد. برنامهٔ عامل پایگاه دانش و نیز برنامهٔ کنونی را نگه می‌دارد و به‌روز می‌کند. پایگاه دانش آغازین شامل اصل‌موضوعه‌های بی‌زمان است — آن‌هایی که به t وابسته نیستند، مانند اصل‌موضوعهٔ مربوط‌کنندهٔ نسیم‌داری خانه‌ها به حضور گودال‌ها. در هر گام زمانی، جملهٔ ادراک تازه همراه با همهٔ اصل‌موضوعه‌های وابسته به t، مانند اصل‌موضوعه‌های حالت-جانشین، افزوده می‌شود. (بخش بعد توضیح می‌دهد چرا عامل به اصل‌موضوعه‌هایی برای گام‌های زمانی آینده نیاز ندارد.) سپس، عامل با استنتاج منطقی، با ASK کردن پرسش‌ها از پایگاه دانش، تعیین می‌کند کدام خانه‌ها امن‌اند و کدام‌ها هنوز بازدید نشده‌اند.

بدنهٔ اصلی برنامهٔ عامل برنامه‌ای بر اساس اولویت نزولی هدف‌ها می‌سازد. نخست، اگر درخششی هست، برنامه برنامه‌ای برای برداشتن طلا، دنبال‌کردن مسیری بازگشت به مکان آغازین و بیرون‌آمدن از غار می‌سازد. وگرنه، اگر برنامهٔ کنونی‌ای نیست، برنامه مسیری به نزدیک‌ترین خانهٔ امنی برنامه‌ریزی می‌کند که هنوز بازدید نکرده، و مطمئن می‌شود مسیر فقط از خانه‌های امن می‌گذرد.

برنامه‌ریزی مسیر با جستجوی A* انجام می‌شود، نه با ASK. اگر خانه‌های امنی برای کاوش نمانده باشد، گام بعدی — اگر عامل هنوز تیر دارد — تلاش برای امن‌کردن خانه‌ای با تیراندازی به یکی از مکان‌های احتمالی وومپوس است. این‌ها با پرسیدن اینکه کجا $ASK(KB, \neg W_{x,y})$ کاذب است — یعنی کجا معلوم نیست وومپوسی نیست — تعیین می‌شوند. تابع PLAN-SHOT (نشان داده نشده) از PLAN-ROUTE برای برنامه‌ریزی توالی کنشی استفاده می‌کند که این شلیک را هم‌راستا کند. اگر این شکست بخورد، برنامه به‌دنبال خانه‌ای برای کاوش می‌گردد که به‌شکل اثبات‌پذیر ناامن نباشد — یعنی خانه‌ای که $ASK(KB, \neg OK^t_{x,y})$ برایش false بازمی‌گرداند. اگر چنین خانه‌ای نباشد، مأموریت ناممکن است و عامل به [1,1] عقب‌نشینی می‌کند و از غار بیرون می‌آید.

```
function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench, breeze, glitter, bump, scream]
  persistent: KB, a knowledge base, initially the atemporal axioms
              t, a counter, initially 0, indicating time
              plan, an action sequence, initially empty

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  TELL(KB, the temporal physics sentences for time t)
  safe ← {[x, y] : ASK(KB, OK t x,y) = true}
  if ASK(KB, Glitter t) = true then
    plan ← [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  if plan is empty then
    unvisited ← {[x, y] : ASK(KB, L t′ x,y) = false for all t′ ≤ t}
    plan ← PLAN-ROUTE(current, unvisited ∩ safe, safe)
  if plan is empty and ASK(KB, HaveArrow t) = true then
    possible wumpus ← {[x, y] : ASK(KB, ¬W x,y) = false}
    plan ← PLAN-SHOT(current, possible wumpus, safe)
  if plan is empty then // no choice but to take a risk
    not safe ← {[x, y] : ASK(KB, ¬OK t x,y) = false}
    plan ← PLAN-ROUTE(current, unvisited ∩ not safe, safe)
  if plan is empty then
    plan ← PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  action ← FIRST(plan)
  plan ← REST(plan)
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
  inputs: current, the agent's current location
  goals, a set of squares; attempt to plan a route to one of them
  allowed, a set of squares that may be part of the route
  problem ← ROUTE-PROBLEM(current, goals, allowed)
  return SEARCH(problem)   // any search algorithm from Chapter 3
```

شکل ۷.۲۰ — برنامهٔ عامل ترکیبی برای جهان وومپوس. از پایگاه دانش گزاره‌ای برای استنتاج حالت جهان استفاده می‌کند و الگوریتم‌های جستجو را برای یافتن برنامه‌ها فرامی‌خواند.

۷.۷.۳ تخمین حالت منطقی

برنامهٔ عامل شکل ۷.۲۰ کاملاً خوب کار می‌کند، اما یک ضعف عمده دارد: با گذشت زمان، هزینهٔ محاسباتی درگیر در فراخوانی‌های ASK بیشتر و بیشتر می‌شود. این عمدتاً به‌سبب آن است که استنتاج‌های لازم باید در زمان بیشتر و بیشتر به عقب بروند و نمادهای گزارهٔ بیشتری را درگیر کنند. بدیهی است این پایدار نیست — نمی‌توانیم عاملی داشته باشیم که زمان پردازش هر ادراکش متناسب با طول عمرش رشد کند! آنچه واقعاً نیاز داریم زمان به‌روزرسانی ثابت است — یعنی مستقل از t. پاسخ بدیهی آن است که نتایج استنتاج را ذخیره یا «کش» (Cache) کنیم، تا فرایند استنتاج در گام زمانی بعد بتواند بر نتایج گام‌های قبلی بنا کند به‌جای آنکه مجبور باشد از صفر از نو آغاز کند.

چنان‌که در بخش 4.4 دیدیم، تاریخچهٔ ادراک‌ها و همهٔ پیامدهایش را می‌توان با حالت باور جایگزین کرد — یعنی بازنمایی‌ای از مجموعهٔ همهٔ حالت‌های کنونی ممکن جهان.[13] فرایند به‌روزرسانی حالت باور هنگام رسیدن ادراک‌های تازه «تخمین حالت» (State Estimation) نامیده می‌شود (صفحهٔ ۱۳۲ را ببینید). در حالی که در بخش 4.4 حالت باور فهرست صریحی از حالت‌ها بود، این‌جا می‌توانیم از جملهٔ منطقی شامل نمادهای گزاره‌ای مرتبط با گام زمانی کنونی و نیز نمادهای بی‌زمان استفاده کنیم. برای نمونه، جملهٔ منطقی

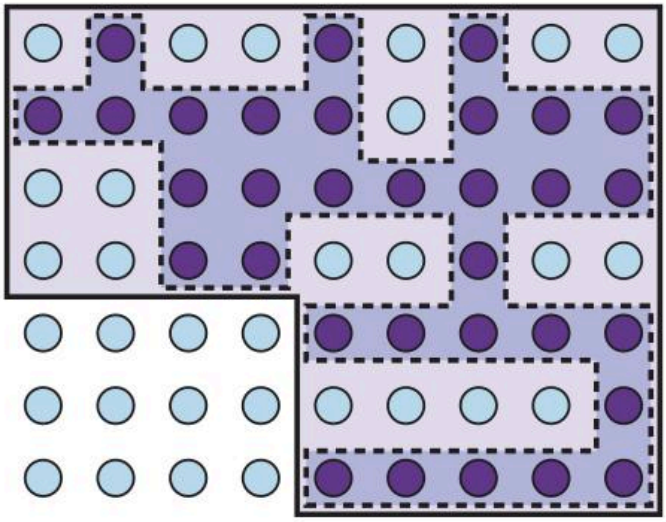
$$\text{WumpusAlive}^1 \wedge L^1_{2,1} \wedge B_{2,1} \wedge (P_{3,1} \vee P_{2,2}) \quad (۷.۴)$$

مجموعهٔ همهٔ حالت‌های زمان ۱ را بازنمایی می‌کند که در آن‌ها وومپوس زنده است، عامل در [2,1] است، آن خانه نسیم‌دار است و در [3,1] یا [2,2] یا هر دو گودالی هست.

نگه‌داشتن حالت باور دقیق به‌صورت فرمول منطقی آسان از آب درنمی‌آید. اگر n نماد روان برای زمان t باشد، 2^n حالت ممکن هست — یعنی تخصیص مقادیر صدق به آن نمادها. حال، مجموعهٔ حالت‌های باور، مجموعهٔ توانی (مجموعهٔ همهٔ زیرمجموعه‌ها) مجموعهٔ حالت‌های فیزیکی است. 2^n حالت فیزیکی هست، از این رو $2^{(2^n)}$ حالت باور. حتی اگر از فشرده‌ترین رمزگذاری ممکن فرمول‌های منطقی استفاده می‌کردیم، با نمایش هر حالت باور با عدد دودویی یکتا، به اعدادی با $2^n = \log2(2^{(2^n)})$ بیت برای برچسب‌زدن حالت باور کنونی نیاز داشتیم. یعنی تخمین حالت دقیق ممکن است به فرمول‌های منطقی نیاز داشته باشد که اندازه‌شان در شمار نمادها نمایی است.

یکی از طرح‌های بسیار رایج و طبیعی برای تخمین حالت تقریبی، بازنمایی حالت‌های باور به‌صورت عطف‌هایی از لیترال‌هاست، یعنی فرمول‌های CNF-1. برای این کار، برنامهٔ عامل صرفاً می‌کوشد X^t و $\neg X^t$ را برای هر نماد X^t (و نیز هر نماد بی‌زمانی که مقدار صدقش هنوز معلوم نیست) اثبات کند، با توجه به حالت باور در t-1. عطف لیترال‌های قابل اثبات حالت باور تازه می‌شود و حالت باور قبلی دور ریخته می‌شود.

مهم است بفهمیم این طرح ممکن است با گذشت زمان اطلاعاتی را از دست بدهد. برای نمونه، اگر جملهٔ معادلهٔ (۷.۴) حالت باور واقعی بود، نه $P_{3,1}$ و نه $P_{2,2}$ به‌صورت فردی قابل اثبات نبودند و هیچ‌کدام در حالت باور CNF-1 ظاهر نمی‌شدند. (تمرین HYBR.7 یکی از راه‌حل‌های ممکن این مسئله را کاوش می‌کند.) از سوی دیگر، چون هر لیترال در حالت باور CNF-1 از حالت باور قبلی اثبات می‌شود و حالت باور آغازین ادعای صادقی است، می‌دانیم کل حالت باور 1-CNF باید صادق باشد. پس مجموعهٔ حالت‌های ممکنِ بازنمایی‌شده توسط حالت باور CNF-1 همهٔ حالت‌هایی را شامل می‌شود که در واقع با توجه به تاریخچهٔ کامل ادراک ممکن‌اند. ⚠ چنان‌که در شکل ۷.۲۱ نشان داده شده، حالت باور CNF-1 به‌عنوان پوشش بیرونی ساده، یا «تقریب محافظه‌کارانه» (Conservative Approximation)، دور حالت باور دقیق عمل می‌کند. این ایدهٔ تقریب‌های محافظه‌کارانه برای مجموعه‌های پیچیده را مضمونی تکرارشونده در بسیاری از حوزه‌های هوش مصنوعی می‌بینیم.



شکل ۷.۲۱ — تصویر حالت باور CNF-1 (خط ضخیم) به‌عنوان تقریب محافظه‌کارانهٔ ساده‌بازنمایی از حالت باور دقیق (ناحیهٔ سایه‌خورده با خط‌چین). هر جهان ممکن به‌صورت دایره نشان داده شده؛ دایره‌های سایه‌خورده با همهٔ ادراک‌ها سازگارند.

۷.۷.۴ ساختن برنامه‌ها با استنتاج گزاره‌ای

عامل شکل ۷.۲۰ از استنتاج منطقی برای تعیین کدام خانه‌ها امن استفاده می‌کند، اما برای ساختن برنامه‌ها از جستجوی A* استفاده می‌کند. در این بخش نشان می‌دهیم چگونه با استنتاج منطقی برنامه بسازیم. ایدهٔ پایه بسیار ساده است:

1. جمله‌ای بسازید که شامل:

(الف) $Init^0$، مجموعه‌ای از ادعاها دربارهٔ حالت آغازین؛ (ب) $Transition^1,...,Transition^t$، اصل‌موضوعه‌های حالت-جانشین برای همهٔ کنش‌های ممکن در هر زمان تا بیشینهٔ زمان t؛ (ج) ادعای اینکه هدف در زمان t محقق شده: $HaveGold^t \wedge ClimbedOut^t$.

1. کل جمله را به حل‌کنندهٔ SAT ارائه دهید. اگر حل‌کننده مدل ارضاکننده‌ای بیابد، هدف دست‌یافتنی است؛ اگر جمله ارضاپذیر نباشد، مسئله حل‌نشدنی است.
2. با فرض یافتن مدل، از مدل آن متغیرهایی را استخراج کنید که کنش‌ها را بازنمایی می‌کنند و true تخصیص یافته‌اند. آن‌ها با هم برنامه‌ای برای رسیدن به هدف‌ها بازنمایی می‌کنند.

رویهٔ برنامه‌ریزی گزاره‌ای، SATPLAN، در شکل ۷.۲۲ نشان داده شده. ایدهٔ پایهٔ تازه‌داده‌شده را با یک پیچش پیاده می‌کند. چون عامل نمی‌داند چند گام برای رسیدن به هدف لازم است، الگوریتم هر شمار ممکن گام‌ها t را تا بیشینهٔ طول برنامهٔ قابل تصور Tmax امتحان می‌کند. بدین ترتیب، تضمین‌شده کوتاه‌ترین برنامه را اگر وجود داشته باشد می‌یابد. به‌سبب شیوهٔ جستجوی SATPLAN برای راه‌حل، این رویکرد را نمی‌توان در محیط تا حدی قابل مشاهده استفاده کرد؛ SATPLAN متغیرهای مشاهده‌ناپذیر را به مقادیری که برای ایجاد راه‌حل نیاز دارد تنظیم می‌کرد.

```
function SATPLAN(init, transition, goal, T max) returns solution or failure
                    inputs: init, transition, goal, propositional sentences
          T max, an integer indicating the maximum allowed plan length

                                                          for t = 0 to T max do
                                  cnf ← TRANSLATE-TO-SAT(init ∧ transition ∧ goal, t)
                                                        model ← SAT-SOLVER(cnf)
                                                    if model is not failure then
                                                return EXTRACT-SOLUTION(model)
                                                                  return failure
```

شکل ۷.۲۲ — الگوریتم SATPLAN. مسئلهٔ برنامه‌ریزی به جملهٔ CNF ترجمه می‌شود که در آن ادعا می‌شود هدف در زمان T برآورده می‌شود. سپس از حل‌کنندهٔ SAT برای یافتن مدلی استفاده می‌شود که جمله را صادق کند. اگر مدلی یافت شود، استخراج برنامه از آن سرراست است؛ وگرنه، افق T افزایش می‌یابد.

گام کلیدی در استفاده از SATPLAN، ساخت پایگاه دانش است. ممکن است در بازرسی گذرا به نظر برسد اصل‌موضوعه‌های جهان وومپوس در بخش ۷.۷.۱ برای گام‌های 1(الف) و 1(ب) بالا کافی‌اند. اما تفاوت معناداری میان الزام‌های لازمه (چنان‌که با ASK آزموده می‌شود) و ارضاپذیری هست.

برای نمونه، مکان عامل را در نظر بگیرید، در آغاز [1,1]، و فرض کنیم هدف بی‌جاه‌طلبیِ بودن در [2,1] در زمان ۱ است. پایگاه دانش آغازین شامل $L^0_{1,1}$ است و هدف $L^1_{2,1}$. با ASK، می‌توانیم $L^1_{2,1}$ را اثبات کنیم اگر $Forward^0$ ادعا شود، و اطمینان‌بخش، نمی‌توانیم $L^1_{2,1}$ را اثبات کنیم اگر مثلاً $Shoot^0$ به‌جایش ادعا شود. حال، SATPLAN برنامهٔ $[Forward^0]$ را خواهد یافت؛ تا این‌جا خوب است.

متأسفانه، SATPLAN همچنین برنامهٔ $[Shoot^0]$ را می‌یابد. این چگونه ممکن است؟ برای فهمیدن، مدلی را که SATPLAN می‌سازد بازرسی می‌کنیم: شامل تخصیص $L^0_{2,1}$ است، یعنی عامل می‌تواند در زمان ۱ در [2,1] باشد با آنکه در زمان ۰ آن‌جاست و شلیک می‌کند. ممکن است بپرسید: «مگر نگفتیم عامل در زمان ۰ در [1,1] است؟» بله، گفتیم، اما به عامل نگفتیم نمی‌تواند هم‌زمان در دو مکان باشد! برای لازمه، $L^0_{2,1}$ ناشناخته است و بنابراین نمی‌توان در برهان استفاده کرد؛ برای ارضاپذیری، از سوی دیگر، $L^0_{2,1}$ ناشناخته است و بنابراین می‌توان آن را به هر مقداری که به صادق‌کردن هدف کمک می‌کند تنظیم کرد.

SATPLAN ابزار اشکال‌زدایی خوبی برای پایگاه‌های دانش است، زیرا جاهایی را آشکار می‌کند که دانش گم شده. در این مورد خاص، می‌توانیم پایگاه دانش را با ادعای اینکه در هر گام زمانی، عامل دقیقاً در یک مکان است، با مجموعه‌ای از جمله‌های مشابه آن‌هایی که برای ادعای وجود دقیقاً یک وومپوس استفاده شد، اصلاح کنیم. از سوی دیگر، می‌توانیم $\neg L^0_{x,y}$ را برای همهٔ مکان‌های غیر از [1,1] ادعا کنیم؛ اصل‌موضوعهٔ حالت-جانشین برای مکان از گام‌های زمانی بعدی مراقبت می‌کند. همان اصلاحات برای اطمینان از اینکه عامل در هر زمان یک و تنها یک جهت دارد نیز کار می‌کند.

SATPLAN شگفتی‌های بیشتری در آستین دارد. نخستین آنکه مدل‌هایی با کنش‌های ناممکن می‌یابد، مانند شلیک بدون تیر. برای فهم چرایی، باید دقیق‌تر نگاه کنیم اصل‌موضوعه‌های حالت-جانشین (مانند معادلهٔ (۷.۳)) دربارهٔ کنش‌هایی که پیش‌شرط‌هایشان برآورده نیست چه می‌گویند. اصل‌موضوعه‌ها درست پیش‌بینی می‌کنند وقتی چنین

کنشی اجرا شود هیچ اتفاقی نخواهد افتاد (تمرین SATP.7 را ببینید)، اما نمی‌گویند کنش قابل اجرا نیست! برای پرهیز از تولید برنامه‌های با کنش‌های غیرقانونی، باید «اصل‌موضوعه‌های پیش‌شرط» (Precondition Axioms) اضافه کنیم که می‌گویند رخداد کنش مستلزم برآورده‌شدن پیش‌شرط‌هاست.[14] برای نمونه، باید برای هر زمان t بگوییم:

$$\text{Shoot}^t \Rightarrow \text{HaveArrow}^t$$

این تضمین می‌کند اگر برنامه‌ای کنش Shoot را در هر زمانی انتخاب کند، باید عامل در آن زمان تیر داشته باشد.

شگفتی دوم SATPLAN، ایجاد برنامه‌های با چند کنش هم‌زمان است. برای نمونه، ممکن است مدلی درآورد که در آن هم Forward^0 و هم Shoot^0 صادق‌اند، که مجاز نیست. برای حذف این مسئله، «اصل‌موضوعه‌های طرد کنش» (Action Exclusion Axioms) را معرفی می‌کنیم: برای هر جفت کنش $A^t i$ و $A^t j$ اصل‌موضوعه را می‌افزاییم:

$$A^t i \vee \neg A^t j \neg$$

ممکن است اشاره شود راه‌رفتن به جلو و شلیک‌کردن هم‌زمان چندان سخت نیست، در حالی که مثلاً شلیک و برداشتن هم‌زمان نسبتاً غیرعملی است. با اعمال اصل‌موضوعه‌های طرد کنش فقط بر جفت‌کنش‌هایی که واقعاً با هم تداخل می‌کنند، می‌توانیم برنامه‌هایی شامل چند کنش هم‌زمان را مجاز کنیم — و چون SATPLAN کوتاه‌ترین برنامهٔ قانونی را می‌یابد، می‌توانیم مطمئن باشیم از این قابلیت بهره خواهد برد.

برای جمع‌بندی، SATPLAN مدل‌هایی برای جمله‌ای شامل حالت آغازین، هدف، اصل‌موضوعه‌های حالت-جانشین، اصل‌موضوعه‌های پیش‌شرط و اصل‌موضوعه‌های طرد کنش می‌یابد. می‌توان نشان داد این مجموعه‌اصل‌موضوعه کافی است، به این معنا که دیگر «راه‌حل»های جعلی نیست. هر مدلی که جملهٔ گزاره‌ای را ارضا کند، برنامهٔ معتبری برای مسئلهٔ اصلی خواهد بود. فناوری مدرن حل SAT رویکرد را کاملاً عملی می‌کند. برای نمونه، حل‌کنندهٔ سبک DPLL در تولید راه‌حل برای نمونهٔ جهان وومپوسِ نشان‌داده‌شده در شکل ۷.۲ هیچ دشواری‌ای ندارد.

این بخش رویکردی اعلانی به ساخت عامل توصیف کرده: عامل با ترکیبی از ادعای جمله‌ها در پایگاه دانش و انجام استنتاج منطقی کار می‌کند. این رویکرد ضعف‌هایی دارد که در عبارت‌هایی مانند «برای هر زمان t» و «برای هر خانهٔ [x,y]» پنهان‌اند. برای هر عامل عملی، این عبارت‌ها باید با کدی پیاده شوند که نمونه‌های طرح جملهٔ عمومی را به‌صورت خودکار برای درج در پایگاه دانش تولید می‌کند. برای جهان وومپوس با اندازهٔ معقول — اندازه‌ای قابل مقایسه با بازی رایانه‌ای نسبتاً کوچک — ممکن است به تختهٔ ۱۰۰×۱۰۰ و ۱۰۰۰ گام زمانی نیاز داشته باشیم که به پایگاه‌های دانش با ده‌ها یا صدها میلیون جمله می‌انجامد.

نه فقط این نسبتاً غیرعملی می‌شود، بلکه مسئلهٔ عمیق‌تری را نیز نشان می‌دهد: ما چیزی دربارهٔ جهان وومپوس می‌دانیم — یعنی اینکه «فیزیک» در همهٔ خانه‌ها و همهٔ گام‌های زمانی یکسان کار می‌کند — که نمی‌توانیم مستقیماً در زبان منطق گزاره‌ای بیان کنیم. برای حل این مسئله، به زبان بیانی‌تری نیاز داریم، زبانی که در آن عبارت‌هایی مانند «برای

هر زمان t» و «برای هر خانهٔ [x,y]» به‌صورت طبیعی نوشته شوند. منطق مرتبهٔ اول، توصیف‌شده در فصل ۸، چنین زبانی است؛ در منطق مرتبهٔ اول، جهان وومپوس با هر اندازه و مدتی را می‌توان در حدود ده جملهٔ منطقی توصیف کرد نه ده میلیون یا ده تریلیون.

خلاصهٔ فصل

عامل‌های مبتنی بر دانش را معرفی کرده‌ایم و نشان داده‌ایم چگونه منطقی تعریف کنیم که با آن چنین عامل‌هایی بتوانند دربارهٔ جهان استدلال کنند. نکته‌های اصلی چنین‌اند:

- عامل‌های هوشمند برای رسیدن به تصمیم‌های خوب به دانش دربارهٔ جهان نیاز دارند.
- دانش در عامل‌ها در قالب جمله‌هایی در زبان بازنمایی دانش که در پایگاه دانش ذخیره می‌شوند موجود است.
- عامل مبتنی بر دانش از پایگاه دانش و سازوکار استنتاجی تشکیل شده. با ذخیرهٔ جمله‌هایی دربارهٔ جهان در پایگاه دانشش، استفاده از سازوکار استنتاج برای استنتاج جمله‌های تازه و استفاده از این جمله‌ها برای تصمیم‌گیری دربارهٔ کنش عمل می‌کند.
- زبان بازنمایی با نحواش تعریف می‌شود که ساختار جمله‌ها را مشخص می‌کند، و معناشناسی‌اش که صدق هر جمله را در هر جهان ممکن یا مدل تعریف می‌کند.
- رابطهٔ لازمه میان جمله‌ها برای فهم ما از استدلال حیاتی است. جملهٔ α مستلزم جملهٔ β است اگر β در همهٔ جهان‌هایی که α در آن‌ها صادق است صادق باشد. تعریف‌های هم‌ارز شامل اعتبار جملهٔ $\alpha \Rightarrow \beta$ و ارضاپذیرنبودن جملهٔ $\alpha \wedge \neg\beta$ است.
- استنتاج فرایند مشتق‌کردن جمله‌های تازه از جمله‌های قدیمی است. الگوریتم‌های استنتاج درست فقط جمله‌های مستلزم را مشتق می‌کنند؛ الگوریتم‌های کامل همهٔ جمله‌های مستلزم را مشتق می‌کنند.
- منطق گزاره‌ای زبان ساده‌ای است شامل نمادهای گزاره و عوامل ربط منطقی. می‌تواند گزاره‌هایی را مدیریت کند که شناخته‌شده‌صادق، شناخته‌شده‌کاذب یا کاملاً ناشناخته‌اند.
- مجموعهٔ مدل‌های ممکن، با واژگان گزاره‌ای ثابت، متناهی است، پس لازمه را می‌توان با برشمردن مدل‌ها بررسی کرد. الگوریتم‌های استنتاج بررسی-مدل کارآمد برای منطق گزاره‌ای شامل روش‌های عقبگرد و جستجوی محلی‌اند و اغلب می‌توانند مسائل بزرگ را سریع حل کنند.
- قواعد استنتاج الگوهایی از استنتاج درست‌اند که می‌توان برای یافتن برهان‌ها استفاده کرد. قاعدهٔ حل الگوریتم استنتاج کاملی برای پایگاه‌های دانشِ بیان‌شده در شکل عادی عطفی به دست می‌دهد. زنجیره‌سازی پیشرو و پسرو الگوریتم‌های استدلال بسیار طبیعی برای پایگاه‌های دانش در شکل هورن‌اند.
- روش‌های جستجوی محلی مانند WALKSAT را می‌توان برای یافتن راه‌حل‌ها استفاده کرد. چنین الگوریتم‌هایی درست‌اند اما کامل نیستند.
- تخمین حالت منطقی شامل نگه‌داشتن جملهٔ منطقی است که مجموعهٔ حالت‌های ممکنِ سازگار با تاریخچهٔ مشاهده را توصیف می‌کند. هر گام به‌روزرسانی مستلزم استنتاج با استفاده از مدل گذار محیط است که از اصل‌موضوعه‌های حالت-جانشینِ مشخص‌کنندهٔ چگونگی تغییر هر روان ساخته می‌شود.
- تصمیم‌ها درون عامل منطقی را می‌توان با حل SAT انجام داد: یافتن مدل‌های ممکنِ مشخص‌کنندهٔ توالی‌های کنش آینده که به هدف می‌رسند. این رویکرد فقط برای محیط‌های کاملاً قابل مشاهده یا بی‌حسگر کار می‌کند.

- منطق گزاره‌ای به محیط‌های با اندازهٔ بی‌کران مقیاس نمی‌شود، زیرا قدرت بیانی کافی برای سروکار داشتن موجز با زمان، مکان و الگوهای همگانی روابط میان اشیا را ندارد.

یادداشت‌های کتاب‌شناختی و تاریخی

مقالهٔ جان مک‌کارتی «برنامه‌هایی با فهم عقل سلیم» (مک‌کارتی، ۱۹۵۸، ۱۹۶۸) مفهوم عامل‌هایی را ترویج کرد که از استدلال منطقی برای میانجی‌گری میان ادراک‌ها و کنش‌ها استفاده می‌کنند. همچنین پرچم اعلانی‌گرایی را برافراشت و اشاره کرد گفتنِ آنچه عامل باید بداند راهی زیبا برای ساختن نرم‌افزار است. مقالهٔ آلن نیوول (۱۹۸۲) با عنوان «سطح دانش» استدلال می‌کند عامل‌های عقلایی را می‌توان در سطح انتزاعی‌ای توصیف و تحلیل کرد که با دانشی که دارند تعریف می‌شود نه برنامه‌هایی که اجرا می‌کنند.

خودِ منطق ریشه‌هایش در فلسفه و ریاضیات یونان باستان دارد. افلاطون دربارهٔ ساختار نحوی جمله‌ها، صدق و کذبشان، معنایشان و اعتبار استدلال‌های منطقی بحث کرد. نخستین مطالعهٔ نظام‌مند شناخته‌شدهٔ منطق، «ارغنون» ارسطو بود. قیاس‌های (Syllogism) او همان‌هایی بودند که ما اکنون قواعد استنتاج می‌نامیم، هرچند ترکیب‌پذیری قواعد کنونی ما را نداشتند.

مکتب‌های مگارایی و رواقی مطالعهٔ نظام‌مند عوامل ربط منطقی پایه را در سدهٔ پنجم پیش از میلاد آغاز کردند. جدول‌های صدق از فیلون مگارایی است. رواقیان پنج قاعدهٔ استنتاج پایه را بدون برهان معتبر دانستند، از جمله قاعده‌ای که اکنون مدوس پونس می‌نامیم. آن‌ها شمار دیگری از قواعد را از این پنج قاعده مشتق کردند، با استفاده از، میان دیگر اصول، قضیهٔ استنتاج (صفحهٔ ۲۲۲)، و دربارهٔ برهان از ارسطو روشن‌تر بودند (میتس، ۱۹۵۳).

ایدهٔ تقلیل استنتاج منطقی به فرایندی صرفاً مکانیکی از ویلهلم لایب‌نیتس (۱۶۴۶–۱۷۱۶) است. جورج بول (۱۸۴۷) نخستین سامانهٔ جامع و کاراکنندهٔ منطق صوری را در کتابش «تحلیل ریاضی منطق» معرفی کرد. منطق بول از نزدیک بر جبر معمولی اعداد حقیقی مدل شده بود و از جانشینی عبارت‌های هم‌ارز منطقی به‌عنوان روش استنتاج اولیه‌اش استفاده می‌کرد. هرچند کل منطق گزاره‌ای را مدیریت نمی‌کرد، دیگر ریاضی‌دانان به‌زودی قطعات گم‌شده را پر کردند. شرودر (۱۸۷۷) شکل عادی عطفی را توصیف کرد، در حالی که شکل هورن بسیار بعدتر توسط آلفرد هورن (۱۹۵۱) معرفی شد. نخستین شرح جامع منطق گزاره‌ای مدرن (و منطق مرتبهٔ اول) در «نگارش مفهومی» (Begriffsschrift) گوتلوب فرگه (۱۸۷۹) یافت می‌شود.

نخستین دستگاه مکانیکی‌ای که استنتاج‌های منطقی انجام داد «نمایش‌گر استانهوپ» بود که توسط سومین ارل استانهوپ (۱۷۵۳–۱۸۱۶) ساخته شد. ویلیام استنلی جونز، یکی از ریاضی‌دانانی که کار بول را بسط دادند، «پیانوی منطقی» خود را در ۱۸۶۹ برای انجام استنتاج‌ها در منطق بولی ساخت. تاریخچهٔ سرگرم‌کننده‌ای از این دستگاه‌های استنتاج مکانیکی آغازین توسط مارتین گاردنر (۱۹۶۸) داده شده. نخستین برنامه‌های رایانه‌ای برای استنتاج منطقی، برنامهٔ ۱۹۵۴ مارتین دیویس برای برهان‌ها در حساب پرسبرگر (دیویس، ۱۹۵۷) و نظریه‌پرداز منطقی نیوول، شاو و سایمون (۱۹۵۷) بودند.

امیل پست (۱۹۲۱) و لودویگ ویتگنشتاین (۱۹۲۲) به‌طور مستقل از جدول‌های صدق به‌عنوان روش آزمودن اعتبار جمله‌های منطق گزاره‌ای استفاده کردند. الگوریتم دیویس–پاتنم (دیویس و پاتنم، ۱۹۶۰) نخستین الگوریتم حل گزاره‌ای بود و الگوریتم عقبگرد بهبودیافتهٔ DPLL (دیویس و همکاران، ۱۹۶۲) کارآمدتر از آب درآمد. قاعدهٔ حل و برهان کامل بودنش با کلیت کامل برای منطق مرتبهٔ اول توسط جی. ای. رابینسون (۱۹۶۵) توسعه یافت.

استفن کوک (۱۹۷۱) نشان داد تصمیم‌گیری دربارهٔ ارضاپذیری جمله‌ای در منطق گزاره‌ای (مسئلهٔ SAT) NP-کامل است. زیرمجموعه‌های بسیاری از منطق گزاره‌ای شناخته شده‌اند که مسئلهٔ ارضاپذیری برای آن‌ها به‌صورت چندجمله‌ای حل‌شدنی است؛ بندهای هورن یکی از این زیرمجموعه‌هاست. بررسی‌های آغازین نشان داد DPLL پیچیدگی میانگین چندجمله‌ای برای توزیع‌های طبیعی خاصی از مسائل دارد. حتی بهتر، فرانکو و پاول (۱۹۸۳) نشان دادند همان مسائل را می‌توان در زمان ثابت با صرفاً حدس‌زدن تخصیص‌های تصادفی حل کرد. با انگیزهٔ موفقیت تجربی جستجوی محلی، کوتسوپیاس و پاپادیمیتریو (۱۹۹۲) نشان دادند الگوریتم سادهٔ تپه‌نوردی می‌تواند تقریباً همهٔ نمونه‌های مسئلهٔ ارضاپذیری را خیلی سریع حل کند و پیشنهاد داد مسائل سخت نادرند. شونینگ (۱۹۹۹) الگوریتم تپه‌نوردی تصادفی‌ای نشان داد که زمان اجرای مورد انتظار بدترین حالتش بر مسائل 3-SAT $O(1.333^n)$ است — همچنان نمایی، اما به‌شکل چشمگیری از کران‌های بدترین حالت قبلی سریع‌تر. رکورد کنونی $O(1.32216^n)$ است (رولف، ۲۰۰۶).

دستاوردهای کارایی در حل‌کننده‌های گزاره‌ای سریع بوده. با ده دقیقه زمان محاسبه، الگوریتم اصلی DPLL روی سخت‌افزار ۱۹۶۲ فقط می‌توانست مسائل با ۱۰ یا ۱۵ متغیر را حل کند (روی لپ‌تاپ ۲۰۱۹ حدود ۳۰ متغیر). تا ۱۹۹۵ حل‌کنندهٔ SATZ (لی و آنبولگان، ۱۹۹۷) می‌توانست ۱۰۰۰ متغیر را مدیریت کند، به لطف ساختارهای دادهٔ بهینه‌شده برای فهرست‌بندی متغیرها. دو مشارکت حیاتی، فن فهرست‌بندی لیترال تماشاشده (Watched Literal) ژانگ و استیکل (۱۹۹۶) بود که انتشار واحد را بسیار کارآمد می‌کند، و معرفی فنون یادگیری بند (یعنی قید) از جامعهٔ CSP توسط بایاردو و شراگ (۱۹۹۷). با استفاده از این ایده‌ها، و با انگیزهٔ چشم‌انداز حل مسائل تأیید مدار مقیاس صنعتی، موسکویتز و همکاران (۲۰۰۱) حل‌کنندهٔ CHAFF را توسعه دادند که می‌توانست مسائل با میلیون‌ها متغیر را مدیریت کند. از ۲۰۰۲، رقابت‌های سالانهٔ SAT برگزار شده؛ بیشتر مدخل‌های برنده تغییرهایی از CHAFF بوده‌اند. چشم‌انداز حل‌کننده‌ها توسط گومز و همکاران (۲۰۰۸) مرور شده.

الگوریتم‌های جستجوی محلی برای ارضاپذیری توسط نویسندگان گوناگونی در سراسر دههٔ ۱۹۸۰ امتحان شد، بر اساس ایدهٔ کمینه‌کردن شمار بندهای ارضانشده (هانسن و ژومار، ۱۹۹۰). الگوریتم به‌ویژه مؤثری توسط گو (۱۹۸۹) و مستقل از او توسط سلمن و همکاران (۱۹۹۲) توسعه یافت که آن را GSAT نامیدند و نشان دادند قادر است گسترهٔ وسیعی از مسائل بسیار سخت را خیلی سریع حل کند. الگوریتم WALKSAT توصیف‌شده در این فصل از سلمن و همکاران (۱۹۹۶) است.

«گذار فاز» در ارضاپذیری مسائل k-SAT تصادفی نخستین بار توسط سیمون و دوبوا (۱۹۸۹) مشاهده شد و به پژوهش نظری و تجربی فراوانی انجامیده — تا حدی به‌سبب پیوند با پدیده‌های گذار فاز در فیزیک آماری. کرافورد و اوتون (۱۹۹۳) گذار 3-SAT را در نسبت بند/متغیر حدود 4.26 جای دادند و یادآور شدند این با قلهٔ تیزی در زمان اجرای حل‌کنندهٔ SAT آن‌ها هم‌زمان است. کوک و میچل (۱۹۹۷) خلاصهٔ عالی از ادبیات آغازین مسئله ارائه می‌دهند.

الگوریتم‌هایی مانند انتشار پیمایش (Survey Propagation) (پاریسی و زکینا، ۲۰۰۲؛ مانوا و همکاران، ۲۰۰۷) از ویژگی‌های خاص نمونه‌های SAT تصادفیِ نزدیک آستانهٔ ارضاپذیری بهره می‌گیرند و بر چنین نمونه‌هایی بسیار از حل‌کننده‌های SAT عمومی بهتر عمل می‌کنند. وضعیت کنونی فهم نظری توسط آخلپتاس (۲۰۰۹) خلاصه شده.

منابع خوب برای اطلاعات دربارهٔ ارضاپذیری، هم نظری هم عملی، شامل «راهنمای ارضاپذیری» (بیر و همکاران، ۲۰۰۹)، جزوهٔ دونالد کنوت (۲۰۱۵) دربارهٔ ارضاپذیری، و کنفرانس‌های بین‌المللی منظم نظریه و کاربردهای آزمون ارضاپذیری، که SAT نامیده می‌شوند، است.

ایدهٔ ساختن عامل‌ها با منطق گزاره‌ای به مقالهٔ بنیادین مک‌کالوک و پیتس (۱۹۴۳) بازمی‌گردد، که به‌خاطر آغاز حوزهٔ شبکه‌های عصبی شناخته شده، اما در واقع به پیاده‌سازی طراحی عامل مبتنی بر مدار بولی در مغز مربوط بود. استن روزن‌شاین (روزن‌شاین، ۱۹۸۵؛ کالب‌لینگ و روزن‌شاین، ۱۹۹۰) راه‌هایی برای ترجمهٔ عامل‌های مبتنی بر مدار از توصیف‌های اعلانی محیط تکلیف توسعه داد. راد بروکس (۱۹۸۶، ۱۹۸۹) اثربخشی طراحی‌های مبتنی بر مدار را برای کنترل ربات‌ها نشان می‌دهد (فصل ۲۶ را ببینید). بروکس (۱۹۹۱) استدلال می‌کند طراحی‌های مبتنی بر مدار تنها چیزی است که هوش مصنوعی لازم دارد — اینکه بازنمایی و استدلال دست‌وپاگیر، پرهزینه و غیرضروری‌اند. به نظر ما، هم استدلال و هم مدارها لازم‌اند. ویلیامز و همکاران (۲۰۰۳) عامل ترکیبی — نه چندان متفاوت از عامل وومپوس ما — توصیف می‌کنند که فضاپیماهای ناسا را کنترل می‌کند، توالی‌های کنش را برنامه‌ریزی و خطاها را تشخیص و بازیابی می‌کند.

مسئلهٔ عمومی پیگیری محیط تا حدی قابل مشاهده برای بازنمایی‌های حالت-محور در فصل ۴ معرفی شد. نمونه‌سازی‌اش برای بازنمایی‌های گزاره‌ای توسط امیر و راسل (۲۰۰۳) مطالعه شد که چند رده از محیط‌ها را شناسایی کردند که الگوریتم‌های تخمین حالت کارآمد می‌پذیرند و نشان دادند برای چند ردهٔ دیگر مسئله ناشدنی است. مسئلهٔ پیش‌افکنی زمانی (Temporal Projection) که شامل تعیین اینکه چه گزاره‌هایی پس از اجرای توالی کنشی صادق‌اند، را می‌توان حالت خاصی از تخمین حالت با ادراک‌های خالی دید. بسیاری از نویسندگان این مسئله را به‌سبب اهمیتش در برنامه‌ریزی مطالعه کرده‌اند؛ برخی نتایج سختی مهم توسط لیبراتوره (۱۹۹۷) برقرار شد. ایدهٔ بازنمایی حالت باور با گزاره‌ها به ویتگنشتاین (۱۹۲۲) بازمی‌گردد.

رویکرد تخمین حالت منطقی با استفاده از فهرست‌های زمانی روی متغیرهای گزاره‌ای توسط کاوتز و سلمن (۱۹۹۲) پیشنهاد شد. نسل‌های بعدی SATPLAN توانستند از پیشرفت‌های حل‌کننده‌های SAT بهره بگیرند و از مؤثرترین راه‌های حل مسائل برنامه‌ریزی دشوار باقی بمانند (کاوتز، ۲۰۰۶).

مسئلهٔ قاب نخستین بار توسط مک‌کارتی و هیس (۱۹۶۹) شناخته شد. بسیاری از پژوهشگران مسئله را درون منطق مرتبهٔ اول حل‌نشدنی دانستند و پژوهش فراوانی را در منطق‌های غیریکنواخت برانگیخت. فیلسوفانی از دریفوس (۱۹۷۲) تا کراکت...

1. مسئلهٔ قاب را یکی از نشانه‌های شکست اجتناب‌ناپذیر کل پروژهٔ هوش مصنوعی دانسته‌اند. راه‌حل مسئلهٔ قاب با اصل‌موضوعه‌های حالت-جانشین از رِی رایتر (۱۹۹۱) است. تیلشر (۱۹۹۹) مسئلهٔ قاب استنتاجی را ایدهٔ جدایی شناسایی و راه‌حلی برایش ارائه می‌کند. با نگاه به گذشته، می‌توان دید عامل‌های روزن‌شاین (۱۹۸۵) از مدارهایی

استفاده می‌کردند که اصل‌موضوعه‌های حالت-جانشین را پیاده می‌کردند، اما روزن‌شاین متوجه نشد مسئلهٔ قاب بدین ترتیب تا حد زیادی حل شده.

حل‌کننده‌های گزاره‌ای مدرن بر گستره‌ای از کاربردهای صنعتی اعمال شده‌اند، مانند سنتز سخت‌افزار رایانه (نوویک و همکاران، ۱۹۹۳). بررسی‌کنندهٔ ارضاپذیری SATMC برای شناسایی آسیب‌پذیریِ پیش‌ترناشناخته‌ای در پروتکل ورود به وب‌مرورگر استفاده شد (آرماندو و همکاران، ۲۰۰۸).

جهان وومپوس به‌عنوان بازی توسط گرگوری یوب (۱۹۷۵) اختراع شد. به‌طرز طنزآمیزی، یوب آن را توسعه داد، زیرا از بازی‌های انجام‌شده روی شبکهٔ مستطیلی خسته شده بود: وومپوسش را روی دوازده‌وجهی گذاشت و ما آن را به شبکهٔ قدیمیِ خسته‌کننده برگرداندیم. مایکل جنزِرت پیشنهاد کرد جهان وومپوس به‌عنوان بستر آزمون عامل‌ها استفاده شود.

فصل ۸: منطق مرتبهٔ اول (First-Order Logic)

در این فصل متوجه می‌شویم جهان از اشیای فراوانی بهره‌مند است، برخی از آن‌ها با اشیای دیگر رابطه دارند، و در این فصل می‌کوشیم دربارهٔ آن‌ها استدلال کنیم.

منطق گزاره‌ای برای نشان دادن مفاهیم پایهٔ منطق، استنتاج و عامل‌های مبتنی بر دانش کافی بود. متأسفانه، منطق گزاره‌ای در آنچه می‌تواند بگوید محدود است. در این فصل، «منطق مرتبهٔ اول»[1] را بررسی می‌کنیم که می‌تواند به‌صورت موجز بسیار بیشتر را بازنمایی کند. در بخش ۸.۱ با بحث دربارهٔ زبان‌های بازنمایی به‌طور کلی آغاز می‌کنیم؛ بخش ۸.۲ نحو و معناشناسی منطق مرتبهٔ اول را پوشش می‌دهد؛ بخش‌های 8.3 و 8.4 کاربرد منطق مرتبهٔ اول را برای بازنمایی‌های ساده نشان می‌دهند.

۸.۱ بازنگری بازنمایی

در این بخش دربارهٔ سرشت زبان‌های بازنمایی بحث می‌کنیم. زبان‌های برنامه‌نویسی (مانند C++ یا جاوا یا پایتون) بزرگ‌ترین ردهٔ زبان‌های صوریِ در حال استفادهٔ معمول‌اند. ساختارهای دادهٔ درون برنامه‌ها را می‌توان برای بازنمایی واقعیت‌ها استفاده کرد؛ برای نمونه، برنامه‌ای می‌تواند از آرایهٔ 4×4 برای بازنمایی محتوای جهان وومپوس استفاده کند. پس گزارهٔ زبان برنامه‌نویسی Pit ← World[2,2] راه نسبتاً طبیعی‌ای برای ادعای اینکه در خانهٔ [2,2] گودالی هست است. به هم چسباندن رشته‌ای از چنین گزاره‌هایی برای اجرای شبیه‌سازی جهان وومپوس کافی است.

آنچه زبان‌های برنامه‌نویسی فاقد آن‌اند، سازوکاری عمومی برای مشتق‌کردن واقعیت‌ها از واقعیت‌های دیگر است؛ هر به‌روزرسانی ساختار داده توسط رویهٔ حوزه‌ویژه‌ای انجام می‌شود که جزئیاتش توسط برنامه‌نویس از دانش خودش از حوزه استخراج می‌شود. این رویکرد رویه‌ای را می‌توان با سرشت اعلانی منطق گزاره‌ای مقایسه کرد، که در آن دانش و استنتاج جدا از هم‌اند و استنتاج کاملاً مستقل از حوزه است. پایگاه‌های دادهٔ SQL ترکیبی از دانش اعلانی و رویه‌ای می‌گیرند.

دومین نقطه‌ضعف ساختارهای داده در برنامه‌ها (و پایگاه‌های داده)، نبود راهی آسان برای گفتن مثلاً «در [2,2] یا [3,1] گودالی هست» یا «اگر وومپوس در [1,1] است پس در [2,2] نیست» است. برنامه‌ها می‌توانند مقدار تکی‌ای برای هر متغیر ذخیره کنند و برخی سامانه‌ها اجازه می‌دهند مقدار «ناشناخته» باشد، اما فاقد قدرت بیانی لازم برای سروکار داشتن مستقیم با اطلاعات جزئی‌اند.

منطق گزاره‌ای زبان اعلانی است، زیرا معناشناسی‌اش بر رابطهٔ صدق میان جمله‌ها و جهان‌های ممکن مبتنی است. همچنین قدرت بیانی کافی برای سروکار داشتن با اطلاعات جزئی، با استفاده از فصل و نقیض، دارد. منطق گزاره‌ای ویژگی سومی دارد که در زبان‌های بازنمایی مطلوب است، یعنی «ترکیب‌پذیری» (Compositionality). در زبان ترکیب‌پذیر، معنای جمله تابعی از معنای اجزایش است. برای نمونه، معنای «S1,2 ∧ S1,4» به معانی «S1,4» و «S1,2»

مربوط است. بسیار عجیب می‌بود اگر «S1,4» معنا می‌داد در خانهٔ [1,4] بوی گندی هست و «S1,2» معنا می‌داد در خانهٔ [1,2] بوی گندی هست، اما «S1,2 ∧ S1,4» معنا می‌داد فرانسه و لهستان در مسابقهٔ مقدماتی هاکی روی یخ هفتهٔ پیش 1–1 مساوی کردند.

اما منطق گزاره‌ای، به‌عنوان بازنمایی عاملی، فاقد قدرت بیانی برای توصیف موجز محیطی با اشیای فراوان است. برای نمونه، مجبور شدیم برای هر خانه قاعدهٔ جداگانه‌ای دربارهٔ نسیم‌ها و گودال‌ها بنویسیم، مانند:

```
B1,1 ⇔ (P1,2 ∨ P2,1)
```

به انگلیسی، از سوی دیگر، به نظر می‌رسد به‌سادگی می‌توان یک‌بار برای همیشه گفت: «خانه‌های مجاور گودال‌ها نسیم‌دارند.» نحو و معناشناسی انگلیسی توصیف موجز محیط را ممکن می‌کند: انگلیسی، مانند منطق مرتبهٔ اول، بازنمایی ساختاریافته است.

8.1.1 زبان اندیشه

زبان‌های طبیعی (مانند انگلیسی یا اسپانیایی) واقعاً بسیار بیانی‌اند. ما توانستیم تقریباً تمام این کتاب را به زبان طبیعی بنویسیم، با لغزش‌های گاه‌به‌گاه به زبان‌های دیگر (عمدتاً ریاضیات و نمودارها). سنت بلندی در زبان‌شناسی و فلسفهٔ زبان هست که زبان طبیعی را زبان بازنمایی دانش اعلانی می‌بیند. اگر می‌توانستیم قواعد زبان طبیعی را کشف کنیم، می‌توانستیم از آن‌ها در سامانه‌های بازنمایی و استدلال استفاده کنیم و از سود میلیاردها صفحه‌ای که به زبان طبیعی نوشته شده بهره ببریم.

دیدگاه مدرن زبان طبیعی آن است که به‌عنوان رسانه‌ای برای ارتباط خدمت می‌کند نه بازنمایی ناب. وقتی گوینده‌ای اشاره می‌کند و می‌گوید «نگاه کن!»، شنونده می‌فهمد که مثلاً سوپرمن سرانجام بر فراز بام‌ها ظاهر شده. اما نمی‌خواهیم بگوییم جملهٔ «نگاه کن!» آن واقعیت را بازنمایی می‌کند. بلکه، معنای جمله هم به خودِ جمله و هم به بافتی که جمله در آن گفته شده بستگی دارد. بدیهی است نمی‌شد جمله‌ای مانند «نگاه کن!» را در پایگاه دانش ذخیره کرد و انتظار بازیابی معنایش را داشت بدون آنکه بازنمایی بافت را نیز ذخیره کرد — که این پرسش را پیش می‌آورد که خودِ بافت چگونه بازنمایی می‌شود.

زبان‌های طبیعی همچنین از ابهام رنج می‌برند، مسئله‌ای برای زبان بازنمایی. چنان‌که پینکر (۱۹۹۵) می‌گوید: «وقتی مردم به بهار می‌اندیشند، قطعاً گیج نمی‌شوند که به فصل فکر می‌کنند یا به چیزی که صدای بونگ می‌دهد — و اگر یک واژه بتواند با دو اندیشه متناظر باشد، اندیشه‌ها نمی‌توانند واژه باشند.»

فرضیهٔ نامدار ساپیر–ورف (ورف، ۱۹۵۶) ادعا می‌کند فهم ما از جهان به‌شدت تحت تأثیر زبانی است که به آن صحبت می‌کنیم. قطعاً درست است که جوامع گفتاری متفاوت جهان را متفاوت تقسیم می‌کنند. فرانسوی‌ها دو واژهٔ «chaise» و «fauteuil» دارند، برای مفهومی که انگلیسی‌زبان‌ها با یک واژه می‌پوشانند: «chair». اما انگلیسی‌زبان‌ها به‌راحتی

می‌توانند ردهٔ fauteuil را تشخیص دهند و نامی برایش بگذارند — تقریباً «صندلی دسته‌باز» — پس آیا زبان واقعاً تفاوت ایجاد می‌کند؟ ورف عمدتاً به شهود و حدس تکیه می‌کرد و ایده‌هایش تا حد زیادی کنار گذاشته شده، اما در سال‌های میانی در واقع داده‌های واقعی از مطالعات مردم‌شناختی، روان‌شناختی و عصب‌شناختی داریم.

برای نمونه، آیا می‌توانید به یاد آورید کدام یک از دو عبارت زیر آغازگر بخش 8.1 بود؟

«در این بخش، دربارهٔ سرشت زبان‌های بازنمایی بحث می‌کنیم ...» «این بخش موضوع زبان‌های بازنمایی دانش را پوشش می‌دهد ...»

وانر (۱۹۷۴) آزمایش مشابهی انجام داد و یافت آزمودنی‌ها در سطح شانس — حدود ۵۰٪ مواقع — انتخاب درست را می‌کردند، اما محتوای آنچه خواندند را با دقت بیش از ۹۰٪ به خاطر می‌آوردند. این پیشنهاد می‌کند مردم واژه‌هایی را که می‌خوانند تفسیر می‌کنند و بازنمایی درونیِ غیرکلامی می‌سازند و واژه‌های دقیق چندان اهمیت ندارند.

جالب‌تر موردی است که مفهومی کاملاً در زبانی غایب است. گویندگان زبان بومی استرالیایی گوگو یِیمیتیر هیچ واژه‌ای برای جهت‌های نسبی (یا خودمحور) مانند جلو، عقب، راست یا چپ ندارند. در عوض از جهت‌های مطلق استفاده می‌کنند و مثلاً معادل «دستم شمالی درد می‌کند» را می‌گویند. این تفاوت در زبان تفاوتی در رفتار ایجاد می‌کند: گویندگان گوگو یِیمیتیر در ناوبری در زمین باز بهترند، در حالی که انگلیسی‌زبان‌ها در گذاشتن چنگال سمت راست بشقاب بهترند.

زبان همچنین به نظر می‌رسد از طریق ویژگی‌های دستوریِ ظاهراً دلبخواهی مانند جنسیت اسم‌ها بر اندیشه تأثیر می‌گذارد. برای نمونه، «bridge» (پل) در اسپانیایی مذکر و در آلمانی مؤنث است. بورودیتسکی (۲۰۰۳) از آزمودنی‌ها خواست صفت‌های انگلیسی برای توصیف عکسی از پلی خاص برگزینند. اسپانیایی‌زبان‌ها strong، dangerous، big و towering را برگزیدند، در حالی که آلمانی‌زبان‌ها fragile، elegant، beautiful و slender را.

واژه‌ها می‌توانند به‌عنوان نقطه‌های لنگر خدمت کنند که بر شیوهٔ ادراک ما از جهان اثر می‌گذارند. لافتوس و پالمر (۱۹۷۴) به آزمودنی‌های تجربی فیلمی از تصادف خودرو نشان دادند. آزمودنی‌هایی که از آن‌ها پرسیده شد «خودروها وقتی به هم *برخورد کردند* چقدر سریع می‌رفتند؟» میانگین ۳۲ مایل بر ساعت گزارش کردند، در حالی که آزمودنی‌هایی که پرسش با واژهٔ «smashed» (کوبیده شدند) به‌جای «contacted» پرسیده شد، برای همان خودروها در همان فیلم ۴۱ مایل بر ساعت گزارش کردند. در مجموع، تفاوت‌های قابل اندازه‌گیری اما کوچکی در پردازش شناختی گویندگان زبان‌های مختلف هست، اما هیچ شواهد قانع‌کننده‌ای نیست که این به تفاوت عمده‌ای در جهان‌بینی می‌انجامد.

در سامانهٔ استدلال منطقی که از شکل عادی عطفی (CNF) استفاده می‌کند، می‌بینیم شکل‌های زبانی «¬(A ∨ B)» و «¬A ∧ ¬B» یکسان‌اند، زیرا می‌توانیم به درون سامانه نگاه کنیم و ببینیم دو جمله به‌صورت همان شکل CNF متعارف ذخیره شده‌اند. شروع به ممکن‌شدن کرده بتوانیم با مغز انسان کار مشابهی بکنیم. میچل و همکاران (۲۰۰۸) آزمودنی‌ها را در دستگاه تصویربرداری تشدید مغناطیسی کارکردی (fMRI) گذاشتند، واژه‌هایی مانند «celery» (کرفس) نشانشان دادند و مغزشان را تصویربرداری کردند. برنامهٔ یادگیری ماشینِ آموزش‌دیده با جفت‌های (واژه، تصویر) توانست در تکلیف‌های انتخاب دودویی (مثلاً «کرفس» یا «هواپیما») در ۷۷٪ مواقع درست پیش‌بینی کند. سامانه حتی می‌تواند برای واژه‌هایی که هرگز تصویر fMRI از آن‌ها ندیده (با در نظر گرفتن تصاویر واژه‌های مرتبط) و برای افرادی که

هرگز ندیده (که اثبات می‌کند fMRI سطحی از بازنمایی مشترک میان مردم را آشکار می‌کند) بالاتر از سطح شانس پیش‌بینی کند. این نوع کار هنوز در مراحل آغازین است، اما fMRI (و دیگر فناوری‌های تصویربرداری مانند الکتروفیزیولوژی درون‌جمجمه‌ای (ساهین و همکاران، ۲۰۰۹)) وعده می‌دهد ایده‌های بسیار عینی‌تری از اینکه بازنمایی‌های دانش انسانی چگونه‌اند به ما بدهد.

از دیدگاه منطق صوری، بازنمایی همان دانش به دو روش متفاوت هیچ تفاوتی ایجاد نمی‌کند؛ همان واقعیت‌ها از هر دو بازنمایی قابل اشتقاق خواهند بود. در عمل اما، یک بازنمایی ممکن است به گام‌های کمتری برای استنتاج نتیجه‌ای نیاز داشته باشد، یعنی استدلال‌گری با منابع محدود می‌تواند با یک بازنمایی به نتیجه برسد اما با دیگری نه. برای تکلیف‌های غیرقیاسی مانند یادگیری از تجربه، پیامدها لزوماً به شکل بازنمایی‌های استفاده‌شده وابسته‌اند. در فصل ۱۹ نشان می‌دهیم وقتی برنامهٔ یادگیری دو نظریهٔ ممکن از جهان را در نظر می‌گیرد که هر دو با همهٔ داده‌ها سازگارند، رایج‌ترین راه شکستن تساوی انتخاب موجزترین نظریه است — و آن به زبانِ استفاده‌شده برای بازنمایی نظریه‌ها بستگی دارد. پس تأثیر زبان بر اندیشه برای هر عاملی که یادگیری انجام می‌دهد اجتناب‌ناپذیر است.

8.1.2 ترکیب بهترین زبان‌های صوری و طبیعی

می‌توانیم بنیان منطق گزاره‌ای — معناشناسی اعلانی، ترکیب‌پذیر، مستقل از بافت و بدون‌ابهام — را بپذیریم و منطق بیانی‌تری بر آن بنیان بسازیم، با قرض‌گرفتن ایده‌های بازنمایشی از زبان طبیعی در حالی که از نقطه‌ضعف‌هایش می‌پرهیزیم. وقتی به نحو زبان طبیعی نگاه می‌کنیم، بدیهی‌ترین عناصر، اسم‌ها و گروه‌های اسمی‌اند که به اشیا (Object) اشاره می‌کنند (خانه‌ها، گودال‌ها، وومپوس‌ها) و فعل‌ها و گروه‌های فعلی همراه با صفت‌ها و قیدها که به روابط (Relations) میان اشیا اشاره می‌کنند (نسیم‌دار است، مجاور است، شلیک می‌کند). برخی از این روابط توابع‌اند — روابطی که در آن‌ها فقط یک «مقدار» برای «ورودیِ» داده‌شده هست. به‌راحتی می‌توان فهرست‌کردن مثال‌هایی از اشیا، روابط و توابع را آغاز کرد:

- **اشیاء:** مردم، خانه‌ها، اعداد، نظریه‌ها، رونالد مک‌دونالد، رنگ‌ها، بازی‌های بیسبال، جنگ‌ها، قرن‌ها ...
- **روابط:** این‌ها می‌توانند روابط یک‌تایی یا «ویژگی»ها (Properties) مانند قرمز، گرد، جعلی، اول، چندطبقه باشند ...، یا روابط n-تایی عمومی‌تر مانند برادرِ، بزرگ‌تر از، درونِ، بخشی از، دارای رنگ، پس از رخ داد، مالکِ، میانِ می‌آید، ...
- **توابع:** پدرِ، بهترین دوستِ، اینینگ سومِ، یک بیش ازِ، آغازِ ...

در واقع، تقریباً هر ادعایی را می‌توان اشاره به اشیا و ویژگی‌ها یا روابط دانست. چند مثال در پی می‌آید:

- «یک به‌علاوهٔ دو برابر سه است.»

اشیا: یک، دو، سه، یک به‌علاوهٔ دو؛ رابطه: برابر است؛ تابع: به‌علاوه. («یک به‌علاوهٔ دو» نامی برای شیئی است که با اعمال تابع «به‌علاوه» بر اشیای «یک» و «دو» به دست می‌آید. «سه» نام دیگری برای این شیء است.)

- «خانه‌های همسایهٔ وومپوس بدبو هستند.»

اشیا: وومپوس، خانه‌ها؛ ویژگی: بدبو؛ رابطه: همسایه.

- «پادشاه شرور جان در ۱۲۰۰ بر انگلستان حکومت کرد.»

اشیا: جان، انگلستان، ۱۲۰۰؛ رابطه: در طول حکومت کرد؛ ویژگی‌ها: شرور، پادشاه.

زبان منطق مرتبهٔ اول، که نحو و معناشناسی‌اش را در بخش بعد تعریف می‌کنیم، حول اشیا و روابط ساخته شده. این زبان برای ریاضیات، فلسفه و هوش مصنوعی دقیقاً به‌سبب آن مهم بوده که آن حوزه‌ها — و در واقع، بخش بزرگی از وجود روزمرهٔ انسان — را می‌توان سودمندانه سروکار داشتن با اشیا و روابط میان آن‌ها دانست. منطق مرتبهٔ اول همچنین می‌تواند واقعیت‌هایی دربارهٔ برخی یا همهٔ اشیای جهان بیان کند. این امکان بازنمایی قوانین یا قواعد عمومی را می‌دهد، مانند گزارهٔ «خانه‌های همسایهٔ وومپوس بدبو هستند.»

تفاوت اصلی میان منطق گزاره‌ای و مرتبهٔ اول در «تعهد هستی‌شناختی» (Ontological Commitment) هر زبان نهفته است — یعنی آنچه دربارهٔ سرشت واقعیت فرض می‌کند. به‌صورت ریاضی، این تعهد از طریق سرشت مدل‌های صوری‌ای بیان می‌شود که صدق جمله‌ها نسبت به آن‌ها تعریف می‌شود. برای نمونه، منطق گزاره‌ای فرض می‌کند واقعیت‌هایی هست که یا در جهان برقرارند یا برقرار نیستند. هر واقعیت می‌تواند در یکی از دو حالت باشد — صادق یا کاذب — و هر مدل به هر نماد گزاره صادق یا کاذب تخصیص می‌دهد (بخش 7.4.2 را ببینید). منطق مرتبهٔ اول بیشتر فرض می‌کند؛ namely اینکه جهان از اشیایی با روابط معینی میان آن‌ها تشکیل شده که برقرارند یا برقرار نیستند. (شکل ۸.۱ را ببینید.) مدل‌های صوری متناظراً پیچیده‌تر از مدل‌های منطق گزاره‌ای‌اند.

زبان	تعهد هستی‌شناختی (چه چیزی در جهان وجود دارد)	تعهد شناختی (عامل دربارهٔ واقعیت‌ها چه باوری دارد)
منطق گزاره‌ای	واقعیت‌ها	صادق/کاذب/ناشناخته
منطق مرتبهٔ اول	واقعیت‌ها، اشیا، روابط	صادق/کاذب/ناشناخته
منطق زمانی	واقعیت‌ها، اشیا، روابط، زمان‌ها	صادق/کاذب/ناشناخته
نظریهٔ احتمال	واقعیت‌ها	درجهٔ باور $\in [0,1]$
منطق فازی	واقعیت‌ها با درجهٔ صدق $\in [0,1]$	مقدار بازه‌ای شناخته‌شده

شکل ۸.۱ — زبان‌های صوری و تعهدهای هستی‌شناختی و شناختی‌شان.

این تعهد هستی‌شناختی نقطهٔ قوت بزرگی برای منطق است (هم گزاره‌ای و هم مرتبهٔ اول)، زیرا به ما امکان می‌دهد با گزاره‌های صادق آغاز و گزاره‌های صادق دیگر استنتاج کنیم. این به‌ویژه در حوزه‌هایی قدرتمند است که هر گزاره مرزهای روشنی دارد، مانند ریاضیات یا جهان وومپوس، جایی که خانه‌ای یا گودال دارد یا ندارد؛ هیچ امکان خانه‌ای با فرورفتگیِ

مبهمِ گودال‌مانند نیست. اما در جهان واقعی، بسیاری از گزاره‌ها مرزهای مبهم دارند: آیا وین شهر بزرگی است؟ آیا این رستوران غذای خوشمزه سرو می‌کند؟ آیا آن شخص قدبلند است؟ به این بستگی دارد از چه کسی بپرسید، و پاسخشان ممکن است «یک‌جورهایی» باشد.

یک پاسخ، پالایش بازنمایی است: اگر خط خامی که شهرها را به «بزرگ» و «نه بزرگ» تقسیم می‌کند اطلاعات زیادی را برای کاربرد موردنظر از قلم بیندازد، می‌توان شمار رده‌های اندازه را افزایش داد یا از نماد تابع Population استفاده کرد. راه‌حل پیشنهادی دیگر از «منطق فازی» (Fuzzy Logic) می‌آید که تعهد هستی‌شناختی می‌کند گزاره‌ها «درجهٔ صدق» (Degree of Truth) بین ۰ و ۱ دارند. برای نمونه، جملهٔ «وین شهر بزرگی است» ممکن است در منطق فازی به درجهٔ ۰٫۸ صادق باشد، در حالی که «پاریس شهر بزرگی است» ممکن است به درجهٔ ۰٫۹ صادق باشد. این با برداشت شهودی ما از جهان هم‌خوانی بهتری دارد، اما استنتاج را دشوارتر می‌کند: به‌جای یک قاعده برای تعیین صدق A ∧ B، منطق فازی به قواعد متفاوتی بسته به حوزه نیاز دارد. امکان دیگر، پوشش‌داده‌شده در بخش 24.1، آن است که هر مفهوم را به نقطه‌ای در فضای چندبعدی تخصیص دهیم و سپس فاصلهٔ میان مفهوم «شهر بزرگ» و مفهوم «وین» یا «پاریس» را اندازه بگیریم.

منطق‌های ویژه‌منظورهٔ گوناگون تعهدهای هستی‌شناختی بیشتری می‌کنند؛ برای نمونه، «منطق زمانی» (Temporal Logic) فرض می‌کند واقعیت‌ها در زمان‌های خاصی برقرارند و آن زمان‌ها (که ممکن است نقطه یا بازه باشند) مرتب‌اند. پس منطق‌های ویژه‌منظوره به انواع خاصی از اشیا (و اصل‌موضوعه‌هایشان) جایگاه «درجه‌یک» درون منطق می‌دهند، به‌جای آنکه صرفاً آن‌ها را درون پایگاه دانش تعریف کنند. «منطق مرتبهٔ بالاتر» (Higher-Order Logic) روابط و توابعِ ارجاع‌شده توسط منطق مرتبهٔ اول را خودِ اشیا می‌بیند. این به فرد امکان می‌دهد دربارهٔ همهٔ روابط ادعاهایی بکند — برای نمونه، می‌توان خواست تعریف کرد رابطه‌ای متعدی به چه معناست. برخلاف بیشتر منطق‌های ویژه‌منظوره، منطق مرتبهٔ بالاتر به‌طور دقیق از منطق مرتبهٔ اول بیانی‌تر است، به این معنا که برخی جمله‌های منطق مرتبهٔ بالاتر را نمی‌توان با هیچ شمار متناهی‌ای از جمله‌های منطق مرتبهٔ اول بیان کرد.

منطق را همچنین می‌توان با «تعهدهای شناختی» (Epistemological Commitments)اش مشخص کرد — حالت‌های دانش ممکنی که نسبت به هر واقعیت اجازه می‌دهد. در هر دو منطق گزاره‌ای و مرتبهٔ اول، جمله‌ای واقعیتی را بازنمایی می‌کند و عامل یا باور دارد جمله صادق است، یا باور دارد کاذب است، یا نظری ندارد. پس این منطق‌ها دربارهٔ هر جمله سه حالت ممکن دانش دارند.

سامانه‌های استفاده‌کننده از نظریهٔ احتمال، از سوی دیگر، می‌توانند هر درجهٔ باوری، یا احتمال ذهنی، از ۰ (بی‌باوری کامل) تا ۱ (باور کامل) داشته باشند. مهم است درجهٔ باور در نظریهٔ احتمال را با درجهٔ صدق در منطق فازی اشتباه نگیریم. در واقع، برخی سامانه‌های فازی عدم قطعیت (درجهٔ باور) دربارهٔ درجه‌های صدق را مجاز می‌دانند. برای نمونه، عامل احتمالاتی جهان وومپوس ممکن است باور کند وومپوس با احتمال ۰٫۷۵ در [1,3] و با احتمال ۰٫۲۵ در [2,3] است (هرچند وومپوس قطعاً در یک خانهٔ خاص است).

۸.۲ نحو و معناشناسی منطق مرتبهٔ اول

این بخش را با مشخص‌کردن دقیق‌تر شیوه‌ای آغاز می‌کنیم که جهان‌های ممکن منطق مرتبهٔ اول تعهد هستی‌شناختی به اشیا و روابط را بازتاب می‌دهند. سپس عناصر گوناگون زبان را معرفی می‌کنیم و معناشناسی‌شان را هنگام پیش‌رفتن توضیح می‌دهیم. نکته‌های اصلی چگونگی تسهیل بازنمایی‌های موجز توسط زبان و چگونگی انجام معناشناسی‌اش به رویه‌های استدلال درست است.

۸.۲.۱ مدل‌های منطق مرتبهٔ اول

فصل ۷ گفت مدل‌های زبان منطقی ساختارهای صوری‌ای هستند که جهان‌های ممکنِ تحت‌بررسی را تشکیل می‌دهند. هر مدل واژگان جمله‌های منطقی را به عناصر جهان ممکن پیوند می‌زند، تا صدق هر جمله تعیین شود. پس مدل‌های منطق گزاره‌ای نمادهای گزاره را به مقادیر صدق ازپیش‌تعریف‌شده پیوند می‌زنند.

مدل‌های منطق مرتبهٔ اول بسیار جالب‌ترند. نخست، آن‌ها اشیا در خود دارند! «دامنه» (Domain) مدل مجموعهٔ اشیا یا «عناصر دامنه» (Domain Elements) آن است. دامنه لازم است غیرخالی باشد — هر جهان ممکن باید دست‌کم یک شیء داشته باشد. (تمرین EMPT.۸ را برای بحث دربارهٔ جهان‌های خالی ببینید.) از نظر ریاضی، مهم نیست این اشیا چه‌اند — فقط مهم است در هر مدل خاص چندتا هستند — اما برای اهداف آموزشی از مثال عینی استفاده می‌کنیم. شکل ۸.۲ مدلی با پنج شیء نشان می‌دهد: ریچارد شیردل، پادشاه انگلستان از ۱۱۸۹ تا ۱۱۹۹؛ برادر کوچک‌ترش، پادشاه شرور جان، که از ۱۱۹۹ تا ۱۲۱۵ حکومت کرد؛ پای چپ ریچارد و جان؛ و تاجی.

اشیای مدل ممکن است به شیوه‌های گوناگون مرتبط باشند. در شکل، ریچارد و جان برادرند. به‌صورت صوری، رابطه فقط مجموعه‌ای از چندتایی‌های (Tuples) اشیای مرتبط است. (چندتایی مجموعه‌ای از اشیاست که در ترتیب ثابتی چیده شده و با کمانک‌های دور اشیا نوشته می‌شود.) پس رابطهٔ برادری در این مدل مجموعهٔ زیر است:

```
{(ریچارد شیردل، پادشاه جان), (پادشاه جان، ریچارد شیردل)}  (8.1)
```

(در این‌جا اشیا را به انگلیسی نام‌گذاری کرده‌ایم، اما می‌توانید اگر خواستید ذهناً تصویرها را به‌جای نام‌ها بگذارید.) تاج بر سر پادشاه جان است، پس رابطهٔ «بر سر» فقط یک چندتایی دارد، (تاج، پادشاه جان). روابط «برادر» و «بر سر» روابط دوتایی‌اند — یعنی جفت‌های اشیا را مرتبط می‌کنند. مدل همچنین روابط یک‌تایی، یا ویژگی‌ها، دارد: ویژگی «شخص» برای هر دو ریچارد و جان صادق است؛ ویژگی «پادشاه» فقط برای جان صادق است (احتمالاً به‌سبب آنکه ریچارد در این نقطه مرده)؛ و ویژگی «تاج» فقط برای تاج صادق است.

انواع خاصی از روابط بهتر است توابع در نظر گرفته شوند، از آن رو که شیءِ داده‌شده باید دقیقاً با یک شیء از این راه مرتبط باشد. برای نمونه، هر شخص یک پای چپ دارد، پس مدل تابع یک‌تایی «پای چپ» دارد — نگاشتی از چندتاییِ تک‌عنصری به شیء — که شامل نگاشت‌های زیر است:

```
(ریچارد شیردل) → پای چپ ریچارد
(پادشاه جان) → پای چپ جان  (8.2)
```

به بیان دقیق، مدل‌های منطق مرتبهٔ اول به توابع کامل (Total Functions) نیاز دارند، یعنی باید برای هر چندتایی ورودی مقداری باشد. پس تاج باید پای چپ داشته باشد و هر یک از پای‌های چپ نیز. راه‌حل فنی برای این مسئلهٔ ناخوشایند شامل شیء «نامرئی» اضافی‌ای است که پای چپ هر چیزی است که پای چپ ندارد، از جمله خودش. خوشبختانه، تا وقتی هیچ ادعایی دربارهٔ پای‌های چپِ چیزهای بدون پای چپ نمی‌کنیم، این ظرافت‌های فنی بی‌اهمیت‌اند.

تاکنون عناصری را توصیف کرده‌ایم که مدل‌های منطق مرتبهٔ اول را پر می‌کنند. بخش ضروری دیگر مدل، پیوند میان آن عناصر و واژگان جمله‌های منطقی است که در ادامه توضیح می‌دهیم.

۸.۲.۲ نمادها و تفسیرها

اکنون به نحو منطق مرتبهٔ اول می‌پردازیم. خوانندهٔ بی‌حوصله می‌تواند توصیف کاملی از دستور زبان صوری شکل ۸.۳ بگیرد.

عناصر نحوی پایهٔ منطق مرتبهٔ اول نمادهایی هستند که برای اشیا، روابط و توابع می‌ایستند. نمادها پس سه نوع‌اند: «نمادهای ثابت» (Constant Symbols) که برای اشیا می‌ایستند؛ «نمادهای گزاره‌ای» (Predicate Symbols) که برای روابط می‌ایستند؛ و «نمادهای تابع» (Function Symbols) که برای توابع می‌ایستند. این قرارداد را می‌پذیریم که این نمادها با حروف بزرگ آغاز شوند. برای نمونه، ممکن است از نمادهای ثابت Richard و John استفاده کنیم؛ نمادهای گزاره‌ای King، Person، OnHead، Brother و Crown؛ و نماد تابع LeftLeg. مانند نمادهای گزاره، انتخاب نام‌ها کاملاً به کاربر بستگی دارد. هر نماد گزاره و تابع با «تعداد آرگومان» (Arity) می‌آید که شمار آرگومان‌ها را ثابت می‌کند.

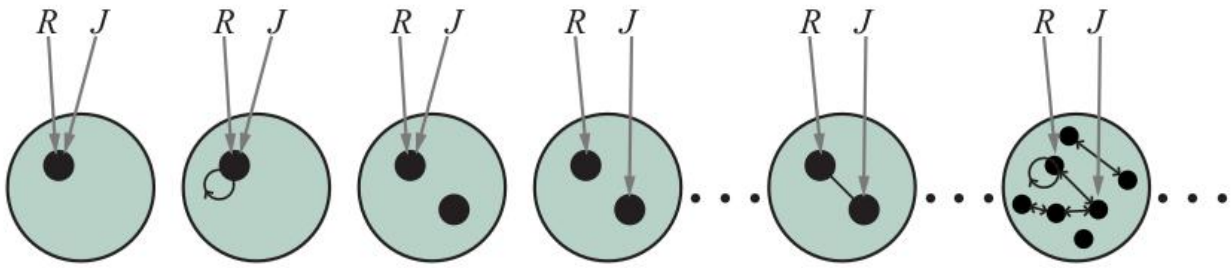
$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{Predicate} \mid \textit{Predicate}(\textit{Term}, \ldots) \mid \textit{Term} = \textit{Term} \\
\textit{ComplexSentence} &\rightarrow (\textit{Sentence}) \\
&\mid \neg \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
&\mid \textit{Quantifier Variable}, \ldots \textit{Sentence} \\
\textit{Term} &\rightarrow \textit{Function}(\textit{Term}, \ldots) \\
&\mid \textit{Constant} \\
&\mid \textit{Variable} \\
\textit{Quantifier} &\rightarrow \forall \mid \exists \\
\textit{Constant} &\rightarrow A \mid X_1 \mid \textit{John} \mid \cdots \\
\textit{Variable} &\rightarrow a \mid x \mid s \mid \cdots \\
\textit{Predicate} &\rightarrow \textit{True} \mid \textit{False} \mid \textit{After} \mid \textit{Loves} \mid \textit{Raining} \mid \cdots \\
\textit{Function} &\rightarrow \textit{Mother} \mid \textit{LeftLeg} \mid \cdots \\
\text{OPERATOR PRECEDENCE} &: \neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

شکل 8.3 – نحو منطق مرتبهٔ اول با برابری، مشخص‌شده در شکل بکوس–نائور (اگر با این نماد آشنا نیستید صفحهٔ ۱۰۳۰ را ببینید). تقدم عملگرها مشخص شده، از بالاترین به پایین‌ترین. تقدم سورها چنان است که سور بر همهٔ چیز سمت راستش حاکم است.

هر مدل باید اطلاعات لازم برای تعیین صادق یا کاذب‌بودن هر جملهٔ داده‌شده را فراهم کند. پس، علاوه بر اشیا، روابط و توابعش، هر مدل شامل «تفسیری» (Interpretation) است که دقیقاً مشخص می‌کند نمادهای ثابت، گزاره و تابع به کدام اشیا، روابط و توابع اشاره می‌کنند. یکی از تفسیرهای ممکن برای مثال ما – که منطق‌دان آن را «تفسیر موردنظر» (Intended Interpretation) می‌نامد – چنین است:

- Richard به ریچارد شیردل و John به پادشاه شرور جان اشاره می‌کند.
- Brother به رابطهٔ برادری اشاره می‌کند – یعنی مجموعهٔ چندتایی‌های اشیای داده‌شده در معادلهٔ (۸.۱)؛ OnHead رابطه‌ای است که میان تاج و پادشاه جان برقرار است؛ Person، King و Crown روابط یک‌تایی‌اند که اشخاص، پادشاهان و تاج‌ها را شناسایی می‌کنند.
- LeftLeg به تابع «پای چپ» چنان‌که در معادلهٔ (۸.۲) تعریف شد اشاره می‌کند.

تفسیرهای ممکن دیگری نیز هست، البته. برای نمونه، تفسیری Richard را به تاج و John را به پای چپ پادشاه جان نگاشت می‌کند. پنج شیء در مدل هست، پس فقط برای نمادهای ثابت Richard و John ۲۵ تفسیر ممکن هست.



شکل 8.4 – برخی اعضای مجموعهٔ همهٔ مدل‌ها برای زبانی با دو نماد ثابت، R و J، و یک نماد رابطهٔ دوتایی. تفسیر هر نماد ثابت با پیکان خاکستری نشان داده شده. درون هر مدل، اشیای مرتبط با پیکان‌ها به هم وصل‌اند.

توجه کنید همهٔ اشیا لازم نیست نام داشته باشند – برای نمونه، تفسیر موردنظر تاج یا پاها را نام‌گذاری نمی‌کند. همچنین ممکن است شیئی چند نام داشته باشد؛ تفسیری هست که تحت آن هم Richard و هم John به تاج اشاره می‌کنند.[۲] اگر این امکان را گیج‌کننده می‌یابید، به یاد آورید در منطق گزاره‌ای، کاملاً ممکن است مدلی داشته باشیم که در آن Cloudy و Sunny هر دو صادق‌اند؛ کار پایگاه دانش است که مدل‌های ناسازگار با دانش ما را کنار بگذارد.

در خلاصه، مدل در منطق مرتبهٔ اول از مجموعه‌ای اشیا و تفسیری تشکیل شده که نمادهای ثابت را به اشیا، نمادهای تابع را به توابع روی آن اشیا، و نمادهای گزاره را به روابط نگاشت می‌کند. دقیقاً مانند منطق گزاره‌ای، لازمه، اعتبار و مانند آن بر حسب همهٔ مدل‌های ممکن تعریف می‌شوند. برای ایده‌ای از اینکه مجموعهٔ همهٔ مدل‌های ممکن چه شکلی است، شکل 8.4 را ببینید. نشان می‌دهد مدل‌ها در تعداد اشیایی که شامل می‌شوند – از یک تا بی‌نهایت – و در شیوهٔ نگاشت نمادهای ثابت به اشیا متفاوت‌اند.

چون شمار مدل‌های مرتبهٔ اول بی‌کران است، نمی‌توانیم لازمه را با برشمردن همهٔ آن‌ها بررسی کنیم (چنان‌که برای منطق گزاره‌ای کردیم). حتی اگر شمار اشیا محدود شود، شمار ترکیب‌ها می‌تواند بسیار بزرگ باشد. (تمرین MCNT.8 را ببینید.) برای مثال شکل 8.4، ۱۳۷,۵۰۶,۱۹۴,۴۶۶ مدل با شش شیء یا کمتر هست.

۸.۲.۳ عبارت‌ها (Terms)

«عبارت» (Term) عبارتِ منطقیِ اشاره‌کننده به شیء است. نمادهای ثابت عبارت‌اند، اما همیشه راحت نیست برای نام‌گذاری هر شیء نماد متمایزی داشته باشیم. در انگلیسی ممکن است از عبارت «پای چپ پادشاه جان» استفاده کنیم به‌جای نام‌دادن به پایش. نمادهای تابع برای این‌اند: به‌جای استفاده از نماد ثابت، از LeftLeg(John) استفاده می‌کنیم.[۳]

در حالت کلی، عبارت پیچیده از نماد تابعی تشکیل شده که فهرست پرانتزی از عبارت‌ها به‌عنوان آرگومان‌های نماد تابع دنبالش می‌آید. مهم است به یاد آوریم عبارت پیچیده فقط نوع پیچیده‌ای از نام است. «فراخوانی زیرروال» نیست که «مقداری بازگرداند». زیرروال LeftLeg وجود ندارد که شخصی را ورودی بگیرد و پایی بازگرداند. می‌توانیم دربارهٔ پای‌های چپ استدلال کنیم (مثلاً قاعدهٔ کلی اینکه هرکس یکی دارد را بیان کنیم و سپس استنتاج کنیم جان باید یکی داشته باشد) بدون آنکه هرگز تعریفی از LeftLeg ارائه دهیم. این چیزی است که با زیرروال‌ها در زبان‌های برنامه‌نویسی نمی‌شود انجام داد.

معناشناسی صوری عبارت‌ها سرراست است. عبارت f(t1,...,tn) را در نظر بگیرید. نماد تابع f به تابعی در مدل اشاره می‌کند (آن را F بنامید)؛ عبارت‌های آرگومان به اشیایی در دامنه اشاره می‌کنند (آن‌ها را d1,...,dn بنامید)؛ و کل عبارت به شیئی اشاره می‌کند که مقدار تابع F اعمال‌شده بر d1,...,dn است. برای نمونه، فرض کنید نماد تابع LeftLeg به تابع نشان‌داده‌شده در معادلهٔ (8.2) اشاره کند و John به پادشاه جان؛ آنگاه LeftLeg(John) به پای چپ پادشاه جان اشاره می‌کند. بدین ترتیب، تفسیر مرجع هر عبارت را ثابت می‌کند.

۸.۲.۴ جمله‌های اتمی

حال که عبارت‌هایی برای اشاره به اشیا و نمادهای گزاره‌ای برای اشاره به روابط داریم، می‌توانیم آن‌ها را ترکیب کنیم تا «جمله‌های اتمی» (Atomic Sentences) بسازیم که واقعیت‌ها را بیان می‌کنند. جملهٔ اتمی (یا به اختصار اتم) از نماد گزاره‌ای تشکیل شده که اختیاراً فهرست پرانتزی از عبارت‌ها دنبالش می‌آید، مانند:

```
Brother(Richard, John)
```

این، تحت تفسیر موردنظرِ پیش‌ترداده‌شده، بیان می‌کند ریچارد شیردل برادر پادشاه جان است.[4] جمله‌های اتمی می‌توانند عبارت‌های پیچیده به‌عنوان آرگومان داشته باشند. پس،

```
Married(Father(Richard), Mother(John))
```

بیان می‌کند پدر ریچارد شیردل با مادر پادشاه جان ازدواج کرده (باز هم، تحت تفسیر مناسبی).[5]

جملهٔ اتمی در مدلِ داده‌شده صادق است اگر رابطه‌ای که نماد گزاره به آن اشاره می‌کند ◭ میان اشیایی که آرگومان‌ها به آن‌ها اشاره می‌کنند برقرار باشد.

۸.۲.۵ جمله‌های پیچیده

می‌توانیم از عوامل ربط منطقی برای ساختن جمله‌های پیچیده‌تر استفاده کنیم، با همان نحو و معناشناسی حساب گزاره‌ای. این‌جا چهار جمله هست که در مدل شکل 8.2 تحت تفسیر موردنظرمان صادق‌اند:

```
Brother(LeftLeg(Richard), John)¬
Brother(Richard, John) ∧ Brother(John, Richard)
King(Richard) ∨ King(John)
King(Richard) ⇒ King(John)¬
```

۸.۲.۶ سورها (Quantifiers)

وقتی منطقی داریم که اشیا را اجازه می‌دهد، طبیعی است بخواهیم ویژگی‌های کل مجموعه‌های اشیا را بیان کنیم، به‌جای برشمردن اشیا با نام. سورها (Quantifiers) این را ممکن می‌کنند. منطق مرتبهٔ اول دو سور استاندارد دارد، به نام‌های همگانی و وجودی.

سور همگانی (∀)

دشواری‌ای را به یاد آورید که در فصل ۷ با بیان قواعد عمومی در منطق گزاره‌ای داشتیم. قواعدی مانند «خانه‌های همسایهٔ وومپوس بدبو هستند» و «همهٔ پادشاهان اشخاص‌اند» نان‌وکرهٔ منطق مرتبهٔ اول‌اند. با نخستین مورد در بخش ۸.۳ سروکار داریم. قاعدهٔ دوم، «همهٔ پادشاهان اشخاص‌اند»، در منطق مرتبهٔ اول چنین نوشته می‌شود:

```
∀x King(x) ⇒ Person(x)
```

سور همگانی ∀ معمولاً «برای همهٔ ...» تلفظ می‌شود. (به یاد آورید A وارونه برای «all» می‌ایستد.) پس جمله می‌گوید: «برای همهٔ x، اگر x پادشاه است، پس x شخص است.» نماد x «متغیر» (Variable) نامیده می‌شود. طبق قرارداد، متغیرها حروف کوچک‌اند. متغیر به خودی خود عبارت است و به‌عنوان چنین می‌تواند آرگومان تابع نیز باشد — برای نمونه، LeftLeg(x). عبارت بدون متغیر «عبارت زمینی» (Ground Term) نامیده می‌شود.

به‌صورت شهودی، جملهٔ ∀x P، که P هر جملهٔ منطقی است، می‌گوید P برای هر شیء x صادق است. دقیق‌تر، ∀x P در مدلِ داده‌شده صادق است اگر P در همهٔ «تفسیرهای بسط‌یافته» (Extended Interpretations) ممکنِ ساخته‌شده از تفسیرِ داده‌شده در مدل صادق باشد، که هر تفسیر بسط‌یافته عنصر دامنه‌ای را مشخص می‌کند که x به آن اشاره می‌کند.

این پیچیده به نظر می‌رسد، اما واقعاً فقط بیان دقیقی از معنای شهودی سور همگانی است. مدل نشان‌داده‌شده در شکل 8.2 و تفسیر موردنظرِ همراه آن را در نظر بگیرید. می‌توانیم تفسیر را به پنج طریق بسط دهیم:

x → ریچارد شیردل، x → پادشاه جان، x → پای چپ ریچارد، x → پای چپ جان، x → تاج.

جملهٔ سوردار ∀x King(x) ⇒ Person(x) در مدل اصلی صادق است اگر جملهٔ King(x) ⇒ Person(x) تحت هر یک از پنج تفسیر بسط‌یافته صادق باشد. یعنی جملهٔ سوردار هم‌ارز ادعای پنج جملهٔ زیر است:

```
ریچارد شیردل پادشاه است ⇒ ریچارد شیردل شخص است.
پادشاه جان پادشاه است ⇒ پادشاه جان شخص است.
پای چپ ریچارد پادشاه است ⇒ پای چپ ریچارد شخص است.
پای چپ جان پادشاه است ⇒ پای چپ جان شخص است.
تاج پادشاه است ⇒ تاج شخص است.
```

به این مجموعه ادعاها دقیق نگاه کنیم. چون در مدل ما پادشاه جان تنها پادشاه است، جملهٔ دوم ادعا می‌کند او شخص است، چنان‌که امیدوار بودیم. اما چهار جملهٔ دیگر که ظاهراً دربارهٔ پاها و تاج ادعا می‌کنند چه؟ آیا آن بخشی از معنای «همهٔ پادشاهان اشخاص‌اند» است؟ در واقع، چهار ادعای دیگر در مدل صادق‌اند، اما هیچ ادعایی دربارهٔ صلاحیت شخص‌بودن پاها، تاج‌ها یا در واقع ریچارد نمی‌کنند. این به‌سبب آن است که هیچ‌یک از این اشیا پادشاه نیستند. با نگاه به جدول صدق ⇒ (شکل 7.8 در صفحهٔ ۲۱۹)، می‌بینیم دلالت هرگاه مقدمش کاذب باشد صادق است — صرف‌نظر از صدق تالی. پس با ادعای جملهٔ سوردار، که هم‌ارز ادعای فهرست کاملی از دلالت‌های فردی است، در نهایت تالی قاعده را فقط برای آن اشیایی ادعا می‌کنیم که مقدم برایشان صادق است و دربارهٔ اشیایی که مقدم برایشان کاذب است هیچ‌چیز نمی‌گوییم. پس تعریف جدول-صدقی ⇒ برای نوشتن قواعد عمومی با سورهای همگانی عالی از آب درمی‌آید.

اشتباه رایجی که حتی خوانندگان کوشایی که این بند را چند بار خوانده‌اند مکرراً مرتکب می‌شوند، استفاده از عطف به‌جای دلالت است. جملهٔ

```
∀x King(x) ∧ Person(x)
```

هم‌ارز ادعای این‌ها خواهد بود:

```
ریچارد شیردل پادشاه است ∧ ریچارد شیردل شخص است،
پادشاه جان پادشاه است ∧ پادشاه جان شخص است،
پای چپ ریچارد پادشاه است ∧ پای چپ ریچارد شخص است،
و الی آخر.
```

بدیهی است این آن چیزی را که می‌خواهیم در بر نمی‌گیرد.

سور وجودی (∃)

سور همگانی دربارهٔ هر شیء گزاره می‌سازد. به همین ترتیب، می‌توانیم دربارهٔ شیئی بدون نام‌بردن از آن، با استفاده از سور وجودی، گزاره بسازیم. برای گفتن مثلاً اینکه پادشاه جان تاجی بر سر دارد، می‌نویسیم:

```
∃x Crown(x) ∧ OnHead(x, John)
```

∃x تلفظ می‌شود «چنان x ای وجود دارد که ...» یا «برای برخی x ...».

به‌صورت شهودی، جملهٔ ∃x P می‌گوید P برای دست‌کم یک شیء x صادق است. دقیق‌تر، ∃x P در مدلِ داده‌شده صادق است اگر P در دست‌کم یک تفسیر بسط‌یافته که x را به عنصر دامنه‌ای تخصیص می‌دهد صادق باشد. یعنی دست‌کم یکی از موارد زیر صادق است:

```
ریچارد شیردل تاج است ∧ ریچارد شیردل بر سر جان است؛
پادشاه جان تاج است ∧ پادشاه جان بر سر جان است؛
پای چپ ریچارد تاج است ∧ پای چپ ریچارد بر سر جان است؛
پای چپ جان تاج است ∧ پای چپ جان بر سر جان است؛
تاج تاج است ∧ تاج بر سر جان است.
```

ادعای پنجم در مدل صادق است، پس جملهٔ وجودیِ اصلی در مدل صادق است. توجه کنید، طبق تعریف‌مان، جمله در مدلی نیز صادق می‌بود که پادشاه جان دو تاج بر سر داشت. این کاملاً با جملهٔ اصلی «پادشاه جان تاجی بر سر دارد» سازگار است.[6]

درست همان‌گونه که ⇒ عامل ربط طبیعی برای استفاده با ∀ به نظر می‌رسد، ∧ عامل ربط طبیعی برای استفاده با ∃ است. استفاده از ∧ به‌عنوان عامل ربط اصلی با ∀ به گزاره‌ای بیش از حد قوی در مثال بخش قبل انجامید؛ استفاده از ⇒ با ∃ معمولاً به گزاره‌ای بسیار ضعیف می‌انجامد، واقعاً. جملهٔ زیر را در نظر بگیرید:

```
x Crown(x) ⇒ OnHead(x, John)∃
```

در ظاهر، این ممکن است بازگویی معقولی از جملهٔ ما به نظر برسد. با اعمال معناشناسی، می‌بینیم جمله می‌گوید دست‌کم یکی از ادعاهای زیر صادق است:

```
ریچارد شیردل تاج است ⇒ ریچارد شیردل بر سر جان است؛
پادشاه جان تاج است ⇒ پادشاه جان بر سر جان است؛
پای چپ ریچارد تاج است ⇒ پای چپ ریچارد بر سر جان است؛
و الی آخر.
```

دلالت اگر هر دو مقدم و تالی صادق باشند، یا اگر مقدم کاذب باشد، صادق است؛ پس اگر ریچارد شیردل تاج نباشد، ادعای اول صادق و سور وجودی ارضا می‌شود. پس جملهٔ دلالتیِ سوردار هرگاه هر شیئی در برآوردن مقدم ناکام باشد صادق است؛ از این رو چنین جمله‌هایی واقعاً چیز زیادی نمی‌گویند.

سورهای تودرتو

اغلب می‌خواهیم جمله‌های پیچیده‌تری با چند سور بیان کنیم. ساده‌ترین حالت آن است که سورها از یک نوع باشند. برای نمونه، «برادرها خواهروبرادرند» را می‌توان چنین نوشت:

```
x ∀y Brother(x, y) ⇒ Sibling(x, y)∀
```

سورهای پیاپیِ هم‌نوع را می‌توان به‌صورت یک سور با چند متغیر نوشت. برای نمونه، برای گفتن اینکه رابطهٔ خواهروبرادری متقارن است، می‌توانیم بنویسیم:

```
x,y Sibling(x, y) ⇔ Sibling(y, x)∀
```

در موارد دیگر ترکیب‌هایی خواهیم داشت. «همه‌کس کسی را دوست دارد» یعنی برای هر شخص، کسی هست که آن شخص او را دوست دارد:

```
x ∃y Loves(x, y)∀
```

از سوی دیگر، برای گفتن «کسی هست که همه او را دوست دارند»، می‌نویسیم:

```
y ∀x Loves(x, y)∃
```

ترتیب سورها پس بسیار مهم است. با درج پرانتزها روشن‌تر می‌شود. ∀x (∃y Loves(x, y)) می‌گوید هرکس ویژگی خاصی دارد، namely ویژگیِ دوست‌داشتن کسی. از سوی دیگر، ∃y (∀x Loves(x, y)) می‌گوید کسی در جهان ویژگی خاصی دارد، namely ویژگیِ دوست‌داشته‌شدن توسط همه.

سردرگمی می‌تواند پیش آید وقتی دو سور با همان نام متغیر استفاده شوند. جملهٔ زیر را در نظر بگیرید:

```
x (Crown(x) ∨ (∃x Brother(Richard, x)))∀
```

در این‌جا x در Brother(Richard, x) وجودی سوردار شده. قاعده آن است که متغیر به درونی‌ترین سوری تعلق دارد که به آن اشاره می‌کند؛ پس تابع هیچ سور دیگری نخواهد بود. راه دیگر اندیشیدن به آن این است: ∃x Brother(Richard, x) جمله‌ای دربارهٔ ریچارد است (که برادری دارد)، نه دربارهٔ x؛ پس گذاشتن ∀x بیرونش اثری ندارد. به همان خوبی می‌شد ∃z Brother(Richard, z) نوشت. چون این می‌تواند منبع سردرگمی باشد، همیشه با سورهای تودرتو از نام‌های متغیر متفاوت استفاده خواهیم کرد.

پیوندهای میان ∀ و ∃

دو سور در واقع از طریق نقیض به‌شکل صمیمانه‌ای به هم متصل‌اند. ادعای اینکه همه از جعفری بدشان می‌آید همان ادعای عدم وجود کسی است که آن را دوست دارد، و برعکس:

```
x ¬Likes(x, Parsnips)∀
```

هم‌ارز است با:

```
x Likes(x, Parsnips)∃¬
```

می‌توانیم یک گام فراتر برویم: «همه بستنی دوست دارند» یعنی هیچ‌کس نیست که بستنی دوست نداشته باشد:

```
x Likes(x, IceCream)∀
```

هم‌ارز است با:

```
x ¬Likes(x, IceCream)∃¬
```

چون ∀ واقعاً عطفی بر روی جهان اشیاست و ∃ فصلی، تعجب‌آور نیست که از قواعد دمورگان پیروی می‌کنند. قواعد دمورگان برای جمله‌های سوردار و بی‌سور چنین‌اند:

x P ≡ ∀x ¬P∃¬	P ∧ ¬Q¬ ≡ (P ∨ Q)¬
x P ≡ ∃x ¬P∀¬	P ∨ ¬Q¬ ≡ (P ∧ Q)¬
x P ≡ ¬∃x ¬P∀	P ∧ Q ≡ ¬(¬P ∨ ¬Q)
x P ≡ ¬∀x ¬P∃	P ∨ Q ≡ ¬(¬P ∧ ¬Q)

پس واقعاً به هر دو ∀ و ∃ نیاز نداریم، درست همان‌گونه که واقعاً به هر دو ∧ و ∨ نیاز نداریم. باز هم، خوانایی از سادگی مهم‌تر است، پس هر دو سور را نگه می‌داریم.

۸.۲.۷ برابری

منطق مرتبهٔ اول یک راه دیگر برای ساختن جمله‌های اتمی دارد، غیر از استفاده از گزاره و عبارت‌ها چنان‌که پیش‌تر توصیف شد. می‌توانیم از «نماد برابری» (Equality Symbol) برای دلالت بر اینکه دو عبارت به همان شیء اشاره می‌کنند استفاده کنیم. برای نمونه،

```
Father(John) = Henry
```

می‌گوید شیئی که Father(John) به آن اشاره می‌کند و شیئی که Henry به آن اشاره می‌کند یکی‌اند. چون تفسیر مرجع هر عبارت را ثابت می‌کند، تعیین صدق جملهٔ برابری صرفاً دیدن یکسان‌بودن مرجع‌های دو عبارت است.

نماد برابری را می‌توان برای بیان واقعیت‌هایی دربارهٔ تابعِ داده‌شده استفاده کرد، چنان‌که برای نماد Father کردیم. همچنین می‌توان با نقیض از آن استفاده کرد تا اصرار کنیم دو عبارت به شیء یکسانی اشاره نمی‌کنند. برای گفتن اینکه ریچارد دست‌کم دو برادر دارد، می‌نویسیم:

```
x,y Brother(x, Richard) ∧ Brother(y, Richard) ∧ ¬(x = y)∃
```

جملهٔ

```
x,y Brother(x, Richard) ∧ Brother(y, Richard)∃
```

معنای موردنظر را ندارد. به‌ویژه، در مدل شکل 8.2 صادق است، جایی که ریچارد فقط یک برادر دارد. برای دیدن این، تفسیر بسط‌یافته‌ای را در نظر بگیرید که در آن هر دو x و y به پادشاه جان تخصیص یافته‌اند. افزودن ¬(x = y) چنین مدل‌هایی را کنار می‌گذارد. نماد x ≠ y گاهی به‌عنوان اختصار ¬(x = y) استفاده می‌شود.

۸.۲.۸ معناشناسی پایگاه‌داده

با ادامهٔ مثال بخش قبل، فرض کنیم باور داریم ریچارد دو برادر دارد، جان و جفری.[7] می‌توانیم بنویسیم:

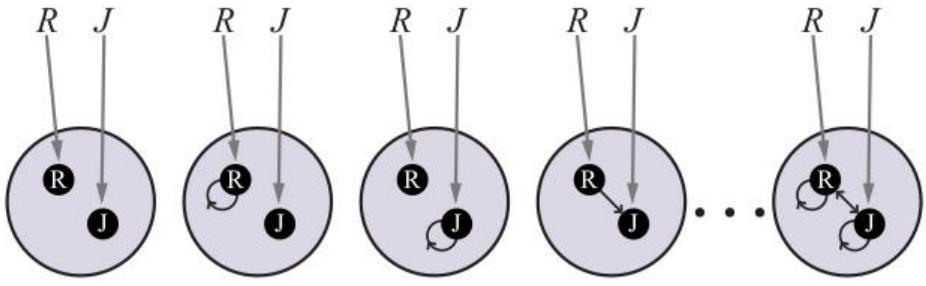
$$\text{Brother}(\text{John}, \text{Richard}) \wedge \text{Brother}(\text{Geoffrey}, \text{Richard}) \quad (8.3)$$

اما این وضعیت امور را کامل در نمی‌گیرد. نخست، این ادعا در مدلی صادق است که ریچارد فقط یک برادر دارد — باید John ≠ Geoffrey را اضافه کنیم. دوم، جمله مدل‌هایی را که ریچارد برادرهای بسیار بیشتری علاوه بر جان و جفری دارد کنار نمی‌گذارد. پس ترجمهٔ درست «برادرهای ریچارد جان و جفری‌اند» چنین است:

$$\text{Brother}(\text{John}, \text{Richard}) \wedge \text{Brother}(\text{Geoffrey}, \text{Richard}) \wedge \text{John} \neq \text{Geoffrey} \wedge \forall x\ \text{Brother}(x, \text{Richard}) \Rightarrow (x = \text{John} \vee x = \text{Geoffrey})$$

این جملهٔ منطقی بسیار دست‌وپاگیرتر از جملهٔ انگلیسی متناظرش به نظر می‌رسد. اما اگر انگلیسی را درست ترجمه نکنیم، سامانهٔ استدلال منطقی‌مان اشتباه خواهد کرد. آیا می‌توانیم معناشناسی‌ای ابداع کنیم که جملهٔ منطقی سرراست‌تری را اجازه دهد؟

پیشنهادی که در سامانه‌های پایگاه‌داده بسیار محبوب است چنین کار می‌کند. نخست، اصرار می‌کنیم هر نماد ثابت به شیء متمایزی اشاره کند — «فرض نام‌های یکتا» (Unique-Names Assumption). دوم، فرض می‌کنیم جمله‌های اتمیِ شناخته‌نشده‌صادق در واقع کاذب‌اند — «فرض جهان بسته» (Closed-World Assumption). سرانجام، «بستار دامنه» (Domain Closure) را فرا می‌خوانیم، یعنی هر مدل شامل عناصر دامنه‌ای بیش از آن‌هایی که توسط نمادهای ثابت نام‌گذاری شده‌اند نیست.



شکل 8.5 — برخی اعضای مجموعهٔ همهٔ مدل‌ها برای زبانی با دو نماد ثابت، R و J، و یک نماد رابطهٔ دوتایی، تحت معناشناسی پایگاه‌داده. تفسیر نمادهای ثابت ثابت است و برای هر نماد ثابت شیء متمایزی هست.

تحت معناشناسی حاصل، معادلهٔ (۸.۳) در واقع بیان می‌کند ریچارد دقیقاً دو برادر دارد، جان و جفری. این را «معناشناسی پایگاه‌داده» (Database Semantics) می‌نامیم تا آن را از معناشناسی استاندارد منطق مرتبهٔ اول متمایز کنیم. معناشناسی پایگاه‌داده همچنین در سامانه‌های برنامه‌نویسی منطقی استفاده می‌شود، چنان‌که در بخش 9.4.4 توضیح داده شده.

آموزنده است مجموعهٔ همهٔ مدل‌های ممکن تحت معناشناسی پایگاه‌داده را برای همان مورد نشان‌داده‌شده در شکل 8.4 (صفحهٔ ۲۵۹) در نظر بگیریم. شکل 8.5 برخی از مدل‌ها را نشان می‌دهد، از مدل بدون هیچ چندتاییِ ارضاکنندهٔ رابطه تا مدل با همهٔ چندتایی‌های ارضاکنندهٔ رابطه. با دو شیء، چهار چندتایی دوعنصری ممکن هست، پس $2^4 = ۱۶$ زیرمجموعهٔ متفاوت از چندتایی‌ها می‌توانند رابطه را ارضا کنند. پس در مجموع ۱۶ مدل ممکن هست — بسیار کمتر از مدل‌های بی‌شمار معناشناسی استاندارد مرتبهٔ اول. از سوی دیگر، معناشناسی پایگاه‌داده به دانش قطعی از آنچه جهان شامل می‌شود نیاز دارد.

این مثال نکتهٔ مهمی را پیش می‌آورد: هیچ معناشناسی «درستی» واحدی برای منطق نیست. سودمندی هر معناشناسیِ پیشنهادی به این بستگی دارد که بیان انواع دانشی که می‌خواهیم بنویسیم را چقدر موجز و شهودی می‌کند، و به اینکه توسعهٔ قواعد استنتاج متناظر چقدر آسان و طبیعی است. معناشناسی پایگاه‌داده وقتی مفیدتر است که از هویت همهٔ اشیای توصیف‌شده در پایگاه دانش مطمئن باشیم و همهٔ واقعیت‌ها را در دست داشته باشیم؛ در موارد دیگر، نسبتاً دست‌وپاگیر است. برای بقیهٔ این فصل، معناشناسی استاندارد را فرض می‌کنیم و مواردی را که این انتخاب به عبارت‌های دست‌وپاگیر می‌انجامد یادآور می‌شویم.

۸.۳ استفاده از منطق مرتبهٔ اول

حال که زبان منطقی بیانی‌ای تعریف کرده‌ایم، بیاموزیم چگونه از آن استفاده کنیم. در این بخش، جمله‌های نمونه‌ای در چند حوزهٔ ساده ارائه می‌کنیم. در بازنمایی دانش، «حوزه» (Domain) فقط بخشی از جهان است که می‌خواهیم درباره‌اش دانشی بیان کنیم.

با توصیف کوتاهی از واسط TELL/ASK برای پایگاه‌های دانش مرتبهٔ اول آغاز می‌کنیم. سپس به حوزه‌های روابط خانوادگی، اعداد، مجموعه‌ها و فهرست‌ها، و جهان وومپوس می‌نگریم. بخش ۸.۴.۲ مثال پرمایه‌تری (مدارهای الکترونیکی) شامل می‌شود و فصل ۱۰ همه‌چیز در جهان را پوشش می‌دهد.

۸.۳.۱ ادعاها و پرس‌وجوها در منطق مرتبهٔ اول

جمله‌ها با TELL به پایگاه دانش افزوده می‌شوند، دقیقاً مانند منطق گزاره‌ای. چنین جمله‌هایی «ادعا»ها (Assertions) نامیده می‌شوند. برای نمونه، می‌توانیم ادعا کنیم جان پادشاه است، ریچارد شخص است و همهٔ پادشاهان اشخاص‌اند:

```
TELL(KB, King(John))
TELL(KB, Person(Richard))
TELL(KB, ∀x King(x) ⇒ Person(x))
```

می‌توانیم با ASK از پایگاه دانش پرسش کنیم. برای نمونه،

```
ASK(KB, King(John))
```

true بازمی‌گرداند. پرسش‌های پرسیده‌شده با ASK «پرس‌وجو»ها یا «هدف»ها نامیده می‌شوند. به‌طور کلی، هر پرس‌وجویی که به‌صورت منطقی توسط پایگاه دانش مستلزم است باید به‌صورت مثبت پاسخ داده شود. برای نمونه، با سه ادعای بالا، پرس‌وجوی

```
ASK(KB, Person(John))
```

نیز باید true بازگرداند. می‌توانیم پرس‌وجوهای سوردار بپرسیم، مانند

```
ASK(KB, ∃x Person(x))
```

پاسخ true است، اما این شاید به‌اندازه‌ای که می‌خواهیم کمکی نباشد. بیشتر شبیه پاسخ «بله» به «می‌توانید ساعت را به من بگویید؟» است. اگر بخواهیم بدانیم کدام مقدار x جمله را صادق می‌کند، به تابع متفاوتی نیاز داریم که ASKVARS می‌نامیم:

```
ASKVARS(KB, Person(x))
```

و جریانی از پاسخ‌ها می‌دهد. در این مورد دو پاسخ خواهد بود: {x/John} و {x/Richard}. چنین پاسخی «جانشینی» (Substitution) یا «فهرست اتصال» (Binding List) نامیده می‌شود. ASKVARS معمولاً برای پایگاه‌های دانشِ صرفاً متشکل از بندهای هورن ذخیره می‌شود، زیرا در چنین پایگاه‌های دانشی هر راهِ صادق‌کردن پرس‌وجو متغیرها را به مقادیر خاصی اتصال می‌دهد. این در منطق مرتبهٔ اول چنین نیست؛ در KBی که فقط به آن گفته شده King(John) ∨ King(Richard)، هیچ اتصال تکی‌ای به x نیست که پرس‌وجوی ∃x King(x) را صادق کند، حتی با آنکه پرس‌وجو در واقع صادق است.

۸.۳.۲ حوزهٔ خویشاوندی

نخستین مثالی که در نظر می‌گیریم حوزهٔ روابط خانوادگی، یا خویشاوندی است. این حوزه شامل واقعیت‌هایی مانند «الیزابت مادر چارلز است» و «چارلز پدر ویلیام است» و قواعدی مانند «مادربزرگِ هرکس، مادرِ والدِ اوست» است.

بدیهی است اشیای حوزهٔ ما مردم‌اند. گزاره‌های یک‌تایی شامل Male و Female، میان دیگران‌اند. روابط خویشاوندی — والدبودن، برادری، ازدواج و مانند آن — با گزاره‌های دوتایی بازنمایی می‌شوند: Parent، Sibling، Brother، Sister، Aunt، Cousin، Grandchild، Grandparent، Husband، Wife، Spouse، Son، Daughter، Child و Uncle. برای Mother و Father از توابع استفاده می‌کنیم، زیرا هر شخص دقیقاً یکی از هر یک از این‌ها دارد، از نظر زیستی (هرچند می‌توانستیم توابع اضافی برای مادرخوانده، مادر جایگزین و مانند آن معرفی کنیم).

می‌توانیم از هر تابع و گزاره بگذریم و آنچه را می‌دانیم بر حسب نمادهای دیگر بنویسیم. برای نمونه، مادرِ هرکس، والدِ مؤنث اوست:

```
∀m,c Mother(c) = m ⇔ Female(m) ∧ Parent(m, c)
```

شوهرِ هرکس، همسرِ مذکر اوست:

```
∀w,h Husband(h, w) ⇔ Male(h) ∧ Spouse(h, w)
```

والد و فرزند روابط معکوس‌اند:

```
∀p,c Parent(p, c) ⇔ Child(c, p)
```

پدربزرگ‌ومادربزرگ، والدِ والدِ هرکس است:

```
∀g,c Grandparent(g, c) ⇔ ∃p Parent(g, p) ∧ Parent(p, c)
```

خواهروبرادر، فرزند دیگری از والدِ هرکس است:

```
∀x,y Sibling(x, y) ⇔ x ≠ y ∧ ∃p Parent(p, x) ∧ Parent(p, y)
```

می‌توانستیم چند صفحهٔ دیگر از این‌ها ادامه دهیم و تمرین KINS.8 از شما همین را می‌خواهد. هر یک از این جمله‌ها را می‌توان اصل‌موضوعه‌ای از حوزهٔ خویشاوندی دید، چنان‌که در بخش 7.1 توضیح داده شد. اصل‌موضوعه‌ها معمولاً با حوزه‌های صرفاً ریاضی همراه‌اند — به‌زودی چند اصل‌موضوعه برای اعداد خواهیم دید — اما در همهٔ حوزه‌ها لازم‌اند. آن‌ها اطلاعات واقعی پایه‌ای را فراهم می‌کنند که نتیجه‌های مفید از آن قابل اشتقاق است. اصل‌موضوعه‌های خویشاوندی ما همچنین «تعریف»ها (Definitions) هستند؛ آن‌ها شکل ∀x,y P(x,y) ⇔ ... دارند. اصل‌موضوعه‌ها تابع Mother و گزاره‌های Grandparent، Parent، Male، Husband و Sibling را بر حسب گزاره‌های دیگر تعریف می‌کنند. تعریف‌های ما در مجموعه‌ای پایه از گزاره‌ها (Child، Female و...) «ته می‌گیرند» که بقیه در نهایت بر حسب آن‌ها تعریف می‌شوند.

این راه طبیعی‌ای برای ساختن بازنمایی یک حوزه است و مشابه شیوه‌ای است که بسته‌های نرم‌افزاری با تعریف‌های پیاپی زیرروال‌ها از توابع کتابخانهٔ اولیه ساخته می‌شوند. توجه کنید لزوماً مجموعهٔ یکتایی از گزاره‌های اولیه نیست؛ به همان خوبی می‌توانستیم به‌جای Child از Parent استفاده کنیم. در برخی حوزه‌ها، چنان‌که نشان می‌دهیم، مجموعهٔ پایهٔ به‌وضوح قابل شناسایی‌ای نیست.

همهٔ جمله‌های منطقی دربارهٔ یک حوزه اصل‌موضوعه نیستند. برخی «قضیه»ها (Theorems) هستند — یعنی توسط اصل‌موضوعه‌ها مستلزم‌اند. برای نمونه، ادعای اینکه خواهروبرادری متقارن است را در نظر بگیرید:

```
∀x,y Sibling(x, y) ⇔ Sibling(y, x)
```

آیا این اصل‌موضوعه است یا قضیه؟ در واقع، قضیه‌ای است که به‌صورت منطقی از اصل‌موضوعه‌ای که خواهروبرادری را تعریف می‌کند برمی‌آید. اگر این جمله را از پایگاه دانش ASK کنیم، باید true بازگرداند.

از دیدگاه صرفاً منطقی، پایگاه دانش فقط باید اصل‌موضوعه‌ها را شامل شود و نه قضیه‌ها را، زیرا قضیه‌ها مجموعهٔ نتیجه‌هایی که از پایگاه دانش برمی‌آیند را افزایش نمی‌دهند. از دیدگاه عملی، قضیه‌ها برای کاهش هزینهٔ محاسباتی مشتق‌کردن جمله‌های تازه ضروری‌اند. بدون آن‌ها، سامانهٔ استدلال باید هر بار از اصول اولیه شروع کند، مانند فیزیک‌دانی که مجبور است قواعد حساب دیفرانسیل را برای هر مسئلهٔ تازه دوباره مشتق کند.

همهٔ اصل‌موضوعه‌ها تعریف نیستند. برخی اطلاعات عمومی‌تری دربارهٔ گزاره‌های خاص فراهم می‌کنند بدون آنکه تعریفی تشکیل دهند. در واقع، برخی گزاره‌ها تعریف کاملی ندارند، زیرا به اندازهٔ کافی دربارهٔ آن‌ها نمی‌دانیم تا کاملاً مشخصشان کنیم. برای نمونه، هیچ راه بدیهی قطعی‌ای برای کامل‌کردن جملهٔ

```
∀x ... ⇔ Person(x)
```

نیست. خوشبختانه، منطق مرتبهٔ اول به ما امکان می‌دهد از گزارهٔ Person بدون تعریف کاملش استفاده کنیم. در عوض، می‌توانیم مشخصات جزئی از ویژگی‌هایی که هر شخص دارد و ویژگی‌هایی که چیزی را شخص می‌کند بنویسیم:

```
∀x Person(x) ⇒ ...
∀x Person(x) ⇒ ... x
```

اصل‌موضوعه‌ها همچنین می‌توانند «فقط واقعیت‌های ساده» باشند، مانند Male(Jim) و Spouse(Jim, Laura). چنین واقعیت‌هایی توصیف‌های نمونه‌های مسئلهٔ خاص را تشکیل می‌دهند و پاسخ به پرسش‌های خاص را ممکن می‌کنند. اگر همه‌چیز خوب پیش برود، پاسخ‌های این پرسش‌ها قضیه‌هایی خواهند بود که از اصل‌موضوعه‌ها برمی‌آیند.

اغلب، می‌یابیم پاسخ‌های مورد انتظار حاصل نمی‌شوند — برای نمونه، از Spouse(Jim, Laura) انتظار می‌رود (تحت قوانین بسیاری کشورها) بتوان استنتاج کرد ¬Spouse(George, Laura)؛ اما این از اصل‌موضوعه‌های پیش‌ترداده‌شده برنمی‌آید — حتی پس از افزودن Jim ≠ George چنان‌که در بخش 8.2.8 پیشنهاد شد. این نشانهٔ آن است که

اصل‌موضوعه‌ای گم شده. تمرین HILL.8 از خواننده می‌خواهد آن را فراهم کند.

۸.۳.۳ اعداد، مجموعه‌ها و فهرست‌ها

اعداد شاید زنده‌ترین مثال از اینکه چگونه نظریهٔ بزرگی را می‌توان از هستهٔ کوچکی از اصل‌موضوعه‌ها ساخت. در این‌جا نظریهٔ «اعداد طبیعی» (Natural Numbers) یا اعداد صحیح نامنفی را توصیف می‌کنیم. به گزاره‌ای NatNum نیاز داریم که برای اعداد طبیعی صادق باشد؛ به یک نماد ثابت، 0، نیاز داریم؛ و به یک نماد تابع، S (جانشین) نیاز داریم. «اصل‌موضوعه‌های پئانو» (Peano Axioms) اعداد طبیعی و جمع را تعریف می‌کنند.[8] اعداد طبیعی به‌صورت بازگشتی تعریف می‌شوند:

```
                                                NatNum(0)
                                  n NatNum(n) ⇒ NatNum(S(n))∀
```

یعنی ۰ عدد طبیعی است و برای هر شیء n، اگر n عدد طبیعی باشد، S(n) عدد طبیعی است. پس اعداد طبیعی 0، S(0)، S(S(0)) و الی آخرند. همچنین به اصل‌موضوعه‌هایی برای قیدکردن تابع جانشین نیاز داریم:

```
                                                n 0 ≠ S(n)∀
                                  m,n m ≠ n ⇒ S(m) ≠ S(n)∀
```

حال می‌توانیم جمع را بر حسب تابع جانشین تعریف کنیم:

```
                                          m NatNum(m) ⇒ +(0, m) = m∀
                  m,n NatNum(m) ∧ NatNum(n) ⇒ +(S(m), n) = S(+(m, n))∀
```

نخستینِ این اصل‌موضوعه‌ها می‌گوید افزودن ۰ به هر عدد طبیعی m خودِ m را می‌دهد. به استفاده از نماد تابع دوتایی «+» در عبارت +(m,0) توجه کنید؛ در ریاضیات معمول، عبارت با نماد میانوندی (Infix) m+0 نوشته می‌شد. (نمادی که برای منطق مرتبهٔ اول استفاده کرده‌ایم پیشوندی (Prefix) نامیده می‌شود.) برای آسان‌ترکردن خواندن جمله‌های عددی‌مان، استفاده از نماد میانوندی را مجاز می‌کنیم. همچنین می‌توانیم S(n) را n+1 بنویسیم، پس اصل‌موضوعهٔ دوم می‌شود:

```
                  m,n NatNum(m) ∧ NatNum(n) ⇒ (m+1)+n = (m+n)+1∀
```

این اصل‌موضوعه جمع را به اعمال مکرر تابع جانشین تقلیل می‌دهد.

استفاده از نماد میانوندی مثالی از «شکر نحوی» (Syntactic Sugar) است، یعنی بسط یا اختصاری از نحو استاندارد که معناشناسی را تغییر نمی‌دهد. هر جمله‌ای که از شکر استفاده کند را می‌توان «بی‌شکر» کرد تا جملهٔ هم‌ارزی در منطق مرتبهٔ اول معمولی تولید شود. مثال دیگر، استفاده از کروشه‌ها به‌جای پرانتزها برای آسان‌کردن دیدن اینکه کدام کروشهٔ چپ با کدام کروشهٔ راست جفت می‌شود. مثال دیگر، فروریختن سورهاست: جایگزینی ∀x ∀y P(x,y) با ∀x,y P(x,y).

وقتی جمع را داشته باشیم، تعریف ضرب به‌عنوان جمع مکرر، توان به‌عنوان ضرب مکرر، تقسیم صحیح و باقیمانده، اعداد اول و مانند آن سرراست است. پس کل نظریهٔ اعداد (شامل رمزنگاری) را می‌توان از یک ثابت، یک تابع، یک گزاره و چهار اصل‌موضوعه ساخت.

حوزهٔ مجموعه‌ها (Sets) نیز برای ریاضیات و نیز برای استدلال عقل سلیم بنیادین است. (در واقع، می‌توان نظریهٔ اعداد را بر حسب نظریهٔ مجموعه‌ها تعریف کرد.) می‌خواهیم بتوانیم مجموعه‌های فردی، شامل مجموعهٔ خالی، را بازنمایی کنیم. به راهی برای ساختن مجموعه‌ها از عناصر یا از عملیات روی مجموعه‌های دیگر نیاز داریم. می‌خواهیم بدانیم آیا عنصری عضو مجموعه‌ای هست و می‌خواهیم مجموعه‌ها را از اشیای غیرمجموعه متمایز کنیم.

از واژگان معمول نظریهٔ مجموعه‌ها به‌عنوان شکر نحوی استفاده می‌کنیم. مجموعهٔ خالی ثابتی است که {} نوشته می‌شود. یک گزارهٔ یک‌تایی، Set، هست که برای مجموعه‌ها صادق است. گزاره‌های دوتایی عبارت‌اند از x∈s (x عضو مجموعهٔ s است) و s1 ⊆ s2 (مجموعهٔ s1 زیرمجموعهٔ s2 است، احتمالاً برابر s2). توابع دوتایی عبارت‌اند از s1 ∩ s2 (اشتراک)، s1 ∪ s2 (اجتماع) و Add(x,s) (مجموعهٔ حاصل از افزودن عنصر x به مجموعهٔ s). یکی از مجموعه‌های ممکن اصل‌موضوعه‌ها چنین است:

1. تنها مجموعه‌ها مجموعهٔ خالی و آن‌هایی هستند که با افزودن چیزی به مجموعه‌ای ساخته می‌شوند:

```
∀s Set(s) ⇔ (s = {}) ∨ (∃x, s2 Set(s2) ∧ s = Add(x, s2))
```

2. مجموعهٔ خالی هیچ عنصری به آن افزوده نشده. به بیان دیگر، هیچ راهی برای تجزیهٔ {} به مجموعهٔ کوچک‌تر و عنصری نیست:

```
¬∃x, s Add(x, s) = {}
```

3. افزودن عنصری که از قبل در مجموعه است اثری ندارد:

```
∀x, s x ∈ s ⇔ s = Add(x, s)
```

4. تنها اعضای یک مجموعه عناصری هستند که به آن افزوده شده‌اند. این را به‌صورت بازگشتی بیان می‌کنیم و می‌گوییم x عضو s است اگر و فقط اگر s برابر عنصری y افزوده‌شده به مجموعه‌ای s2 باشد، که در آن y همان x است یا x عضو s2 است:

```
∀x, s x ∈ s ⇔ ∃y, s2 (s = Add(y, s2) ∧ (x = y ∨ x ∈ s2))
```

5. مجموعه‌ای زیرمجموعهٔ مجموعهٔ دیگر است اگر و فقط اگر همهٔ اعضای مجموعهٔ نخست اعضای مجموعهٔ دوم باشند:

```
∀s1, s2 s1 ⊆ s2 ⇔ (∀x x ∈ s1 ⇒ x ∈ s2)
```

6. دو مجموعه برابرند اگر و فقط اگر هر یک زیرمجموعهٔ دیگری باشد:

```
∀s1, s2 (s1 = s2) ⇔ (s1 ⊆ s2 ∧ s2 ⊆ s1)
```

7. شیئی در اشتراک دو مجموعه است اگر و فقط اگر عضو هر دو مجموعه باشد:

```
∀x, s1, s2 x ∈ (s1 ∩ s2) ⇔ (x ∈ s1 ∧ x ∈ s2)
```

8. شیئی در اجتماع دو مجموعه است اگر و فقط اگر عضو یکی از آن‌ها باشد:

```
∀x, s1, s2 x ∈ (s1 ∪ s2) ⇔ (x ∈ s1 ∨ x ∈ s2)
```

فهرست‌ها (Lists) شبیه مجموعه‌ها هستند. تفاوت‌ها آن است که فهرست‌ها مرتب‌اند و همان عنصر می‌تواند بیش از یک بار در فهرست ظاهر شود. می‌توانیم از واژگان لیسپ برای فهرست‌ها استفاده کنیم: Nil فهرست ثابت بدون عنصر است؛ First، Append، Cons و Rest توابع‌اند؛ و Find گزاره‌ای است که برای فهرست‌ها همان کاری را می‌کند که Member برای مجموعه‌ها. List گزاره‌ای است که فقط برای فهرست‌ها صادق است. مانند مجموعه‌ها، استفاده از شکر نحوی در جمله‌های منطقی شامل فهرست‌ها رایج است. فهرست خالی [] است. عبارت Cons(x, Nil) (یعنی فهرست شامل عنصر x و پس از آن هیچ) به‌صورت [x] نوشته می‌شود. فهرست چند عنصری، مانند [A,B,C]، با عبارت تودرتوی Cons(A, Cons(B, Cons(C, Nil))) متناظر است. تمرین LIST.8 از شما می‌خواهد اصل‌موضوعه‌های فهرست‌ها را بنویسید.

۸.۳.۴ جهان وومپوس

برخی اصل‌موضوعه‌های منطق گزاره‌ای برای جهان وومپوس در فصل ۷ داده شد. اصل‌موضوعه‌های مرتبهٔ اول این بخش بسیار موجزترند و به‌صورت طبیعی دقیقاً آنچه می‌خواهیم بگوییم را در می‌گیرند.

به یاد آورید عامل وومپوس بردار ادراکی با پنج عنصر دریافت می‌کند. جملهٔ مرتبهٔ اولِ متناظرِ ذخیره‌شده در پایگاه دانش باید هم ادراک و هم زمان وقوعش را شامل شود؛ وگرنه عامل دربارهٔ اینکه چه‌وقت چه چیزی دیده گیج می‌شود. برای گام‌های زمانی از اعداد صحیح استفاده می‌کنیم. جملهٔ ادراک نمونه‌ای چنین است:

```
Percept([Stench, Breeze, Glitter, None, None], 5)
```

در این‌جا، Percept گزارهٔ دوتایی است و Stench و مانند آن ثابت‌هایی هستند که در فهرستی گذاشته شده‌اند. کنش‌های جهان وومپوس را می‌توان با عبارت‌های منطقی بازنمایی کرد:

```
Turn(Right), Turn(Left), Forward, Shoot, Grab, Climb
```

برای تعیین کدام بهترین است، برنامهٔ عامل پرس‌وجوی زیر را اجرا می‌کند:

```
ASKVARS(KB, BestAction(a, 5))
```

که فهرست اتصالی مانند {a/Grab} بازمی‌گرداند. برنامهٔ عامل سپس می‌تواند Grab را به‌عنوان کنش برای انجام بازگرداند. دادهٔ ادراک خام دلالت بر واقعیت‌های خاصی دربارهٔ حالت کنونی دارد. برای نمونه:

```
∀t,s,g,w,c Percept([s, Breeze, g, w, c], t) ⇒ Breeze(t)
∀t,s,g,w,c Percept([s, None, g, w, c], t) ⇒ ¬Breeze(t)
∀t,s,b,w,c Percept([s, b, Glitter, w, c], t) ⇒ Glitter(t)
∀t,s,b,w,c Percept([s, b, None, w, c], t) ⇒ ¬Glitter(t)
```

و الی آخر. این قواعد شکل پیش‌پاافتاده‌ای از فرایند استدلال به نام «ادراک» (Perception) را نشان می‌دهند که در فصل ۲۵ با ژرفا مطالعه می‌کنیم. به سورزدن روی زمان t توجه کنید. در منطق گزاره‌ای، برای هر گام زمانی به نسخه‌هایی از هر جمله نیاز داشتیم.

رفتار «بازتابی» ساده را نیز می‌توان با جمله‌های دلالتی سوردار پیاده کرد. برای نمونه:

```
∀t Glitter(t) ⇒ BestAction(Grab, t)
```

با ادراک و قواعد بندهای پیشین، این نتیجهٔ مطلوب BestAction(Grab, 5) را می‌دهد — یعنی Grab کار درست برای انجام است.

ورودی‌ها و خروجی‌های عامل را بازنمایی کرده‌ایم؛ حال وقت بازنمایی خودِ محیط است. با اشیا آغاز کنیم. نامزدهای بدیهی خانه‌ها، گودال‌ها و وومپوس‌اند. می‌توانستیم هر خانه را نام‌گذاری کنیم — Square1,2 و الی آخر — اما آنگاه واقعیتِ مجاوربودن Square1,2 و Square1,3 باید واقعیت «اضافی» می‌بود و برای هر جفت خانه به یکی از این واقعیت‌ها نیاز داشتیم. بهتر است از عبارت پیچیده‌ای استفاده کنیم که سطر و ستون در آن به‌صورت اعداد صحیح ظاهر می‌شوند؛ برای نمونه، به‌سادگی می‌توانیم از عبارت فهرستی [1,2] استفاده کنیم. مجاورت هر دو خانه را می‌توان چنین تعریف کرد:

```
∀x,y,a,b Adjacent([x,y],[a,b]) ⇔ (x = a ∧ (y = b−1 ∨ y = b+1)) ∨ (y = b ∧ (x = a−1 ∨ x = a+1))
```

می‌توانستیم هر گودال را نام‌گذاری کنیم، اما این به دلیل دیگری نامناسب می‌بود: دلیلی برای تمایز میان گودال‌ها نیست.[9] ساده‌تر است از گزارهٔ یک‌تایی Pit استفاده کنیم که برای خانه‌های دارای گودال صادق است. سرانجام، چون دقیقاً یک وومپوس هست، ثابت Wumpus به همان خوبی گزارهٔ یک‌تایی است (و شاید از دیدگاه وومپوس باوقارتر).

مکان عامل در طول زمان تغییر می‌کند، پس At(Agent, s, t) می‌نویسیم تا یعنی عامل در زمان t در خانهٔ s است. می‌توانیم وومپوس را برای همیشه به مکان خاصی تثبیت کنیم با:

```
∀t At(Wumpus, [1,3], t)
```

سپس می‌توانیم بگوییم اشیا فقط می‌توانند در هر زمان در یک مکان باشند:

```
∀x, s1, s2, t At(x, s1, t) ∧ At(x, s2, t) ⇒ s1 = s2
```

با مکان کنونی‌اش، عامل می‌تواند ویژگی‌های خانه را از ویژگی‌های ادراک کنونی‌اش استنتاج کند. برای نمونه، اگر عامل در خانه‌ای باشد و نسیمی ادراک کند، آن خانه نسیم‌دار است:

```
∀s,t At(Agent, s, t) ∧ Breeze(t) ⇒ Breezy(s)
```

دانستن اینکه خانه‌ای نسیم‌دار است مفید است، زیرا می‌دانیم گودال‌ها نمی‌توانند حرکت کنند. توجه کنید Breezy آرگومان زمانی ندارد.

عامل پس از کشف اینکه کدام مکان‌ها نسیم‌دار (یا بدبو) و بسیار مهم‌تر، کدام‌ها نسیم‌دار (یا بدبو) نیستند، می‌تواند استنتاج کند گودال‌ها کجا هستند (و وومپوس کجاست). در حالی که منطق گزاره‌ای برای هر خانه به اصل‌موضوعهٔ جداگانه‌ای نیاز دارد (R2 و R3 در صفحهٔ ۲۲۰ را ببینید) و برای هر چیدمان جغرافیایی جهان به مجموعه‌ای متفاوت از اصل‌موضوعه‌ها نیاز داشت، منطق مرتبهٔ اول فقط به یک اصل‌موضوعه نیاز دارد:

```
∀s Breezy(s) ⇔ ∃r Adjacent(r, s) ∧ Pit(r)  (8.4)
```

به همین ترتیب، در منطق مرتبهٔ اول می‌توانیم روی زمان سور بزنیم، پس فقط به یک اصل‌موضوعهٔ حالت-جانشین برای هر گزاره نیاز داریم، نه نسخه‌ای متفاوت برای هر گام زمانی. برای نمونه، اصل‌موضوعهٔ تیر (معادلهٔ (7.2) در صفحهٔ ۲۴۰) می‌شود:

```
∀t HaveArrow(t+1) ⇔ (HaveArrow(t) ∧ ¬Action(Shoot, t))
```

از این دو جملهٔ نمونه، می‌بینیم صورت‌بندی منطق مرتبهٔ اول کم‌موجزتر از توصیف انگلیسی اصلیِ داده‌شده در فصل ۷ نیست. خواننده دعوت می‌شود اصل‌موضوعه‌های مشابهی برای مکان و جهت‌گیری عامل بسازد؛ در این موارد، اصل‌موضوعه‌ها روی هر دو فضا و زمان سور می‌زنند. مانند مورد تخمین حالت گزاره‌ای، عامل می‌تواند از استنتاج منطقی با اصل‌موضوعه‌هایی از این نوع برای پیگیری جنبه‌هایی از جهان که مستقیماً مشاهده نمی‌شوند استفاده کند. فصل ۱۱ دربارهٔ موضوع اصل‌موضوعه‌های حالت-جانشین مرتبهٔ اول و کاربردهایشان برای ساختن برنامه‌ها با ژرفای بیشتری بحث می‌کند.

۸.۴ مهندسی دانش در منطق مرتبهٔ اول

بخش پیشین استفاده از منطق مرتبهٔ اول را برای بازنمایی دانش در سه حوزهٔ ساده نشان داد. این بخش فرایند عمومی ساخت پایگاه دانش — فرایندی به نام «مهندسی دانش» (Knowledge Engineering) — را توصیف می‌کند. «مهندس دانش» (Knowledge Engineer) کسی است که حوزهٔ خاصی را بررسی می‌کند، می‌آموزد چه مفاهیمی در آن حوزه مهم‌اند، و بازنمایی صوری از اشیا و روابط حوزه می‌سازد. فرایند مهندسی دانش را در حوزهٔ مدار الکترونیکی نشان می‌دهیم. رویکردی که می‌گیریم برای توسعهٔ پایگاه‌های دانش ویژه‌منظوره‌ای مناسب است که حوزه‌شان با دقت محدود شده و گسترهٔ پرس‌وجوهایشان از پیش معلوم است. پایگاه‌های دانش عمومی‌منظوره، که گسترهٔ وسیعی از دانش انسانی را پوشش می‌دهند و برای پشتیبانی از تکلیف‌هایی مانند فهم زبان طبیعی در نظر گرفته شده‌اند، در فصل ۱۰ بحث می‌شوند.

۸.۴.۱ فرایند مهندسی دانش

پروژه‌های مهندسی دانش در محتوا، گستره و دشواری بسیار متفاوت‌اند، اما همهٔ این پروژه‌ها گام‌های زیر را شامل می‌شوند:

1. **پرسش‌ها را شناسایی کن.** مهندس دانش باید گسترهٔ پرسش‌هایی را که پایگاه دانش پشتیبانی خواهد کرد و انواع واقعیت‌هایی را که برای هر نمونهٔ مسئلهٔ خاص در دسترس خواهند بود مشخص کند. برای نمونه، آیا پایگاه دانش وومپوس باید بتواند کنش‌ها را انتخاب کند، یا فقط لازم است به پرسش‌هایی دربارهٔ محتوای محیط پاسخ دهد؟ آیا واقعیت‌های حسگر شامل مکان کنونی خواهند بود؟ تکلیف تعیین خواهد کرد چه دانشی باید بازنمایی شود تا نمونه‌های مسئله را به پاسخ‌ها متصل کند. این گام مشابه فرایند PEAS برای طراحی عامل‌ها در فصل ۲ است.
2. **دانش مرتبط را گردآوری کن.** مهندس دانش ممکن است از قبل در حوزه متخصص باشد، یا ممکن است لازم باشد با متخصصان واقعی کار کند تا آنچه را می‌دانند استخراج کند — فرایندی به نام «اکتساب دانش» (Knowledge Acquisition). در این مرحله، دانش به‌صورت صوری بازنمایی نشده. ایده آن است که گسترهٔ پایگاه دانش، چنان‌که توسط تکلیف تعیین می‌شود، را بفهمیم و بفهمیم حوزه واقعاً چگونه کار می‌کند.

برای جهان وومپوس، که با مجموعه‌ای مصنوعی از قواعد تعریف می‌شود، دانش مرتبط به‌راحتی قابل شناسایی است. (توجه کنید با این حال، تعریف مجاورت به‌صراحت در قواعد جهان وومپوس عرضه نشده بود.) برای حوزه‌های واقعی، مسئلهٔ ارتباط می‌تواند بسیار دشوار باشد — برای نمونه، سامانه‌ای برای شبیه‌سازی طراحی‌های VLSI ممکن است لازم باشد ظرفیت‌های سرگردان و اثر پوستی را در نظر بگیرد یا نگیرد.

1. **دربارهٔ واژگانی از گزاره‌ها، توابع و ثابت‌ها تصمیم بگیر.** یعنی، مفاهیم مهم سطح-حوزه را به نام‌های سطح-منطق ترجمه کن. این شامل پرسش‌های فراوانی دربارهٔ سبک مهندسی دانش است. مانند سبک برنامه‌نویسی، این می‌تواند تأثیر معناداری بر موفقیت نهایی پروژه داشته باشد. برای نمونه، آیا گودال‌ها باید با اشیا بازنمایی شوند یا با گزارهٔ یک‌تایی روی خانه‌ها؟ آیا جهت‌گیری عامل باید تابع باشد یا گزاره؟ آیا مکان وومپوس باید به

زمان وابسته باشد؟ پس از انجام انتخاب‌ها، نتیجه واژگانی است که به‌عنوان «هستی‌شناسی» (Ontology) حوزه شناخته می‌شود. واژهٔ هستی‌شناسی به معنای نظریهٔ خاصی از سرشت بودن یا وجود است. هستی‌شناسی تعیین می‌کند چه نوع چیزهایی وجود دارند، اما ویژگی‌ها و روابط متقابل خاص آن‌ها را تعیین نمی‌کند.

2. **دانش عمومی دربارهٔ حوزه را رمزگذاری کن.** مهندس دانش اصل‌موضوعه‌ها را برای همهٔ اصطلاحات واژگان می‌نویسد. این معنای اصطلاح‌ها را (تا حد ممکن) تثبیت می‌کند و متخصص را قادر می‌سازد محتوا را بررسی کند. اغلب، این گام باورهای نادرست یا شکاف‌هایی در واژگان آشکار می‌کند که باید با بازگشت به گام ۳ و تکرار فرایند اصلاح شوند.
3. **توصیفی از نمونهٔ مسئله رمزگذاری کن.** اگر هستی‌شناسی خوب اندیشیده شده باشد، این گام آسان است. شامل نوشتن جمله‌های اتمی ساده دربارهٔ مصداق‌هایی از مفهوم‌هایی است که از قبل بخشی از هستی‌شناسی‌اند. برای عامل منطقی، نمونه‌های مسئله توسط حسگرها عرضه می‌شوند، در حالی که پایگاه دانش «بی‌جسم» جمله‌هایی را همان‌گونه دریافت می‌کند که برنامه‌های سنتی دادهٔ ورودی دریافت می‌کنند.
4. **پرس‌وجوها را از رویهٔ استنتاج بپرس و پاسخ‌ها را بگیر.** این جایی است که پاداش هست: می‌توانیم اجازه دهیم رویهٔ استنتاج بر اصل‌موضوعه‌ها و واقعیت‌های خاص مسئله عمل کند تا واقعیت‌هایی را که به دانستن‌شان علاقه‌مندیم مشتق کند. پس از نیاز به نوشتن الگوریتم راه‌حل ویژهٔ کاربرد اجتناب می‌کنیم.
5. **پایگاه دانش را اشکال‌زدایی و ارزیابی کن.** افسوس، پاسخ‌های پرس‌وجوها به‌ندرت در اولین تلاش درست‌اند. دقیق‌تر، پاسخ‌ها برای پایگاه دانشِ چنان‌که نوشته شده درست خواهند بود، با فرض درست‌بودن رویهٔ استنتاج، اما پاسخ‌هایی که کاربر انتظار دارد نخواهند بود. برای نمونه، اگر اصل‌موضوعه‌ای گم شده باشد، برخی پرس‌وجوها از پایگاه دانش قابل پاسخ نخواهند بود. فرایند اشکال‌زدایی قابل توجهی می‌تواند در پی آید. اصل‌موضوعه‌های گم‌شده یا اصل‌موضوعه‌هایی که بیش از حد ضعیف‌اند را می‌توان به‌راحتی با توجه به جاهایی که زنجیرهٔ استدلال به‌شکل غیرمنتظره‌ای می‌ایستد شناسایی کرد. برای نمونه، اگر پایگاه دانش به‌جای دوسویه، قاعدهٔ تشخیصی زیر را برای یافتن وومپوس شامل شود (تمرین WUMD.8 را ببینید):

```
∀s Smelly(s) ⇒ Adjacent(Home(Wumpus), s)
```

آنگاه عامل هرگز نخواهد توانست نبودِ وومپوس‌ها را اثبات کند. اصل‌موضوعه‌های نادرست را می‌توان شناسایی کرد، زیرا گزاره‌های کاذبی دربارهٔ جهان‌اند. برای نمونه، جملهٔ

```
∀x NumOfLegs(x, 4) ⇒ Mammal(x)
```

برای خزندگان، دوزیستان و میزها کاذب است. کاذب‌بودن این جمله را می‌توان ⚠ مستقل از بقیهٔ پایگاه دانش تعیین کرد. در برابر، خطای نمونه در برنامه‌ای شبیه این است:

```
offset = position + 1
```

نمی‌توان گفت offset باید position باشد یا position + 1 بدون فهم بافت اطراف.

وقتی به نقطه‌ای رسیدید که خطاهای بدیهی در پایگاه دانش‌تان نیست، وسوسه می‌شوید موفقیت اعلام کنید. اما مگر آنکه بدیهیاً خطایی نباشد، بهتر است سامانه‌تان را به‌صورت صوری با اجرای آن روی مجموعه‌آزمونی از پرس‌وجوها و اندازه‌گیری اینکه چندتا را درست جواب می‌دهید ارزیابی کنید. بدون اندازه‌گیری عینی، خیلی آسان است خودتان را قانع کنید کار تمام شده.

برای فهم بهتر این فرایند هفت‌گامه، اکنون آن را بر مثال گسترده‌ای اعمال می‌کنیم — حوزهٔ مدارهای الکترونیکی.

۸.۴.۲ حوزهٔ مدارهای الکترونیکی

هستی‌شناسی و پایگاه دانشی توسعه می‌دهیم که به ما امکان می‌دهد دربارهٔ مدارهای دیجیتال از نوع نشان‌داده‌شده در شکل ۸.۶ استدلال کنیم. فرایند هفت‌گامهٔ مهندسی دانش را دنبال می‌کنیم.

پرسش‌ها را شناسایی کن. تکلیف‌های استدلالی بسیاری با مدارهای دیجیتال مرتبط‌اند. در بالاترین سطح، کارکرد مدار را تحلیل می‌کنید. برای نمونه، آیا مدار شکل 8.6 واقعاً درست جمع می‌کند؟ اگر همهٔ ورودی‌ها بالا باشند، خروجی گیت A2 چیست؟ پرسش‌هایی دربارهٔ ساختار مدار نیز جالب‌اند. برای نمونه، همهٔ گیت‌های متصل به نخستین پایانهٔ ورودی کدام‌اند؟ آیا مدار حلقه‌های بازخورد دارد؟ این‌ها تکلیف‌های ما در این بخش خواهند بود. سطوح جزئی‌تری از تحلیل هست، شامل موارد مرتبط با تأخیرهای زمانی، مساحت مدار، مصرف توان، هزینهٔ تولید و الی آخر. هر یک از این سطوح به دانش اضافی نیاز خواهد داشت.

دانش مرتبط را گردآوری کن. دربارهٔ مدارهای دیجیتال چه می‌دانیم؟ برای مقاصد ما، آن‌ها از سیم‌ها و گیت‌ها تشکیل شده‌اند. سیگنال‌ها در طول سیم‌ها به پایانه‌های ورودی گیت‌ها جریان می‌یابند و هر گیت سیگنالی روی پایانهٔ خروجی تولید می‌کند که در طول سیم دیگری جریان می‌یابد. برای تعیین این سیگنال‌ها چه خواهند بود، باید بدانیم گیت‌ها چگونه سیگنال‌های ورودی‌شان را تبدیل می‌کنند. چهار نوع گیت هست: گیت‌های OR، AND و XOR دو پایانهٔ ورودی دارند و گیت‌های NOT یک پایانه. همهٔ گیت‌ها یک پایانهٔ خروجی دارند. مدارها، مانند گیت‌ها، پایانه‌های ورودی و خروجی دارند.

برای استدلال دربارهٔ کارکرد و اتصال، لازم نیست دربارهٔ خودِ سیم‌ها، مسیرهایی که می‌گیرند، یا اتصال‌هایی که در آن‌ها به هم می‌رسند صحبت کنیم. تنها چیزی که اهمیت دارد اتصال‌های میان پایانه‌هاست — می‌توانیم بگوییم پایانهٔ خروجی‌ای به پایانهٔ ورودی دیگری متصل است بدون آنکه بگوییم چه چیزی واقعاً آن‌ها را متصل می‌کند. عوامل دیگر مانند اندازه، شکل، رنگ یا هزینهٔ مؤلفه‌های گوناگون برای تحلیل ما بی‌ربط‌اند.

اگر هدفمان چیزی غیر از تأیید طراحی‌ها در سطح گیت بود، هستی‌شناسی متفاوت می‌بود. برای نمونه، اگر به اشکال‌زدایی مدارهای معیوب علاقه‌مند بودیم، احتمالاً ایدهٔ خوبی بود سیم‌ها را در هستی‌شناسی بگنجانیم، زیرا سیم معیوب می‌تواند سیگنالِ جاری در طولش را فاسد کند. برای حل خطاهای زمانی، باید تأخیرهای گیت را شامل می‌کردیم. اگر به طراحی محصولی علاقه‌مند بودیم که سودآور باشد، هزینهٔ مدار و سرعتش نسبت به دیگر محصولات بازار مهم می‌بود.

دربارهٔ واژگان تصمیم بگیر. اکنون می‌دانیم می‌خواهیم دربارهٔ مدارها، پایانه‌ها، سیگنال‌ها و گیت‌ها صحبت کنیم. گام بعدی انتخاب توابع، گزاره‌ها و ثابت‌ها برای بازنمایی آن‌هاست. نخست، باید بتوانیم گیت‌ها را از یکدیگر و از اشیای دیگر متمایز کنیم. هر گیت به‌صورت شیئی که با ثابتی نام‌گذاری شده بازنمایی می‌شود، که درباره‌اش ادعا می‌کنیم گیت است، مثلاً Gate(X1). رفتار هر گیت توسط نوعش تعیین می‌شود: یکی از ثابت‌های XOR، OR، AND یا NOT. چون گیت دقیقاً یک نوع دارد، تابع مناسب است: Type(X1) = XOR.

مدارها، مانند گیت‌ها، با گزاره‌ای شناسایی می‌شوند: Circuit(C1).

سپس پایانه‌ها را در نظر می‌گیریم که با گزارهٔ Terminal(x) شناسایی می‌شوند. مدار می‌تواند یک یا چند پایانهٔ ورودی و یک یا چند پایانهٔ خروجی داشته باشد. از تابع In(1, X1) برای نشان دادن نخستین پایانهٔ ورودی مدار X1 استفاده می‌کنیم. تابع مشابهی Out(n,c) برای پایانه‌های خروجی استفاده می‌شود. تابع Arity(c,i,j) می‌گوید مدار c دارای i پایانهٔ ورودی و j پایانهٔ خروجی است. اتصال میان گیت‌ها را می‌توان با گزاره‌ای به نام Connected بازنمایی کرد که دو پایانه را آرگومان می‌گیرد، مانند Connected(Out(1,X1), In(1,X2)).

سرانجام، باید بدانیم آیا سیگنالی روشن است یا خاموش. یک امکان استفاده از گزارهٔ یک‌تایی On(t) است که وقتی سیگنال پایانه‌ای روشن است صادق است. این اما پرسیدن پرسش‌هایی مانند «همهٔ مقادیر ممکن سیگنال‌ها در پایانه‌های خروجی مدار C1 چیست؟» را کمی دشوار می‌کند. پس دو مقدار سیگنال را به‌عنوان اشیا معرفی می‌کنیم، 1 و 0، که به‌ترتیب «روشن» و «خاموش» را بازنمایی می‌کنند، و تابعی Signal(t) که مقدار سیگنال را برای پایانهٔ t نشان می‌دهد.

دانش عمومی حوزه را رمزگذاری کن. یکی از نشانه‌های اینکه هستی‌شناسی خوبی داریم آن است که فقط به چند قاعدهٔ عمومی نیاز داریم که بتوان به‌وضوح و موجز بیانشان کرد. این‌ها همهٔ اصل‌موضوعه‌هایی هستند که نیاز خواهیم داشت:

1. اگر دو پایانه متصل باشند، سیگنال یکسانی دارند:

```
∀t1, t2 Terminal(t1) ∧ Terminal(t2) ∧ Connected(t1, t2) ⇒ Signal(t1) = Signal(t2)
```

2. سیگنال هر پایانه یا ۱ است یا ۰:

```
∀t Terminal(t) ⇒ Signal(t) = 1 ∨ Signal(t) = 0
```

3. Connected جابه‌جایی‌پذیر است:

```
∀t1, t2 Connected(t1, t2) ⇔ Connected(t2, t1)
```

4. چهار نوع گیت هست:

```
∀g Gate(g) ∧ k = Type(g) ⇒ k = AND ∨ k = OR ∨ k = XOR ∨ k = NOT
```

5. خروجی گیت AND اگر و فقط اگر هر یک از ورودی‌هایش ۰ باشد ۰ است:

```
∀g Gate(g) ∧ Type(g) = AND ⇒ Signal(Out(1,g)) = 0 ⇔ ∃n Signal(In(n,g)) = 0
```

6. خروجی گیت OR اگر و فقط اگر هر یک از ورودی‌هایش ۱ باشد ۱ است:

```
∀g Gate(g) ∧ Type(g) = OR ⇒ Signal(Out(1,g)) = 1 ⇔ ∃n Signal(In(n,g)) = 1
```

7. خروجی گیت XOR اگر و فقط اگر ورودی‌هایش متفاوت باشند ۱ است:

```
∀g Gate(g) ∧ Type(g) = XOR ⇒ Signal(Out(1,g)) = 1 ⇔ Signal(In(1,g)) ≠
Signal(In(2,g))
```

8. خروجی گیت NOT با ورودی‌اش متفاوت است:

```
∀g Gate(g) ∧ Type(g) = NOT ⇒ Signal(Out(1,g)) ≠ Signal(In(1,g))
```

9. گیت‌ها (به‌جز NOT) دو ورودی و یک خروجی دارند:

```
∀g Gate(g) ∧ Type(g) = NOT ⇒ Arity(g, 1, 1)
∀g Gate(g) ∧ k = Type(g) ∧ (k = AND ∨ k = OR ∨ k = XOR) ⇒ Arity(g, 2, 1)
```

10. مدار تا ظرفیت ورودی و خروجی‌اش پایانه دارد، و فراتر از ظرفیتش هیچ:

```
∀c,i,j Circuit(c) ∧ Arity(c, i, j) ⇒ ∀n (n ≤ i ⇒ Terminal(In(n,c))) ∧ (n > i ⇒
  In(n,c) = Nothing) ∧ ∀n (n ≤ j ⇒ Terminal(Out(n,c))) ∧ (n > j ⇒ Out(n,c) = Nothing)
```

11. گیت‌ها، پایانه‌ها و سیگنال‌ها همگی متمایزند:

```
∀g,t,s Gate(g) ∧ Terminal(t) ∧ Signal(s) ⇒ g ≠ t ∧ g ≠ s ∧ t ≠ s
```

12. گیت‌ها مدارند:

```
∀g Gate(g) ⇒ Circuit(g)
```

نمونهٔ مسئلهٔ خاص را رمزگذاری کن. مدار نشان‌داده‌شده در شکل 8.6 به‌صورت مدار C1 با توصیف زیر رمزگذاری می‌شود. نخست مدار و گیت‌های مؤلفه‌اش را رده‌بندی می‌کنیم:

```
Circuit(C1) ∧ Arity(C1, 3, 2)
     Gate(X1) ∧ Type(X1) = XOR
     Gate(X2) ∧ Type(X2) = XOR
     Gate(A1) ∧ Type(A1) = AND
     Gate(A2) ∧ Type(A2) = AND
      Gate(O1) ∧ Type(O1) = OR
```

سپس اتصال‌های میان آن‌ها را نشان می‌دهیم:

```
Connected(Out(1,X1), In(1,X2))
  Connected(In(1,C1), In(1,X1))
 Connected(Out(1,X1), In(2,A2))
  Connected(In(1,C1), In(1,A1))
 Connected(Out(1,A2), In(1,O1))
  Connected(In(2,C1), In(2,X1))
 Connected(Out(1,A1), In(2,O1))
  Connected(In(2,C1), In(2,A1))
Connected(Out(1,X2), Out(1,C1))
  Connected(In(3,C1), In(2,X2))
Connected(Out(1,O1), Out(2,C1))
  Connected(In(3,C1), In(1,A2))
```

پرس‌وجوها را از رویهٔ استنتاج بپرس. چه ترکیب‌هایی از ورودی‌ها باعث می‌شود نخستین خروجی C1 (بیت جمع) ۰ و دومین خروجی C1 (بیت نقلی) ۱ باشد؟

```
i1, i2, i3 Signal(In(1,C1)) = i1 ∧ Signal(In(2,C1)) = i2 ∧ Signal(In(3,C1)) = i3 ∧∃
                                Signal(Out(1,C1)) = 0 ∧ Signal(Out(2,C1)) = 1
```

پاسخ‌ها جانشینی‌هایی برای متغیرهای i1، i2 و i3 هستند چنان‌که جملهٔ حاصل توسط پایگاه دانش مستلزم است. ASKVARS سه جانشینی از این دست به ما می‌دهد:

```
{i1/1, i2/1, i3/0}
 {i1/1, i2/0, i3/1}
 {i1/0, i2/1, i3/1}
```

مجموعه‌های ممکن مقادیر همهٔ پایانه‌ها برای مدار جمع‌کننده چه‌اند؟

```
i1, i2, i3, o1, o2 Signal(In(1,C1)) = i1 ∧ Signal(In(2,C1)) = i2 ∧ Signal(In(3,C1)) = i3∃
                   ∧ Signal(Out(1,C1)) = o1 ∧ Signal(Out(2,C1)) = o2
```

این پرس‌وجوی نهایی جدول ورودی–خروجی کاملی برای دستگاه بازمی‌گرداند که می‌توان از آن برای بررسی اینکه آیا واقعاً ورودی‌هایش را درست جمع می‌کند استفاده کرد. این مثال ساده‌ای از «تأیید مدار» (Circuit Verification) است. همچنین می‌توانیم از تعریف مدار برای ساختن سامانه‌های دیجیتال بزرگ‌تری استفاده کنیم که همان نوع رویهٔ تأیید برایشان قابل انجام است. (تمرین ADDR.8 را ببینید.) بسیاری از حوزه‌ها پذیرای همان نوع توسعهٔ پایگاه دانشِ ساختاریافته‌اند، که در آن مفاهیم پیچیده‌تر بر روی مفاهیم ساده‌تر تعریف می‌شوند.

پایگاه دانش را اشکال‌زدایی کن. می‌توانیم پایگاه دانش را به شیوه‌های گوناگون برهم بزنیم تا ببینیم چه نوع رفتارهای نادرستی پدیدار می‌شوند. برای نمونه، فرض کنیم بخش 8.2.8 را نخوانده‌ایم و از این رو فراموش کرده‌ایم ادعا کنیم 1 ≠ 0. ناگهان، سامانه قادر به اثبات هیچ خروجی‌ای برای مدار نخواهد بود، به‌جز موارد ورودی 000 و 110. می‌توانیم مسئله را با پرسیدن خروجی‌های هر گیت دقیقاً مشخص کنیم. برای نمونه، می‌توانیم بپرسیم:

```
∃i1, i2, o Signal(In(1,C1)) = i1 ∧ Signal(In(2,C1)) = i2 ∧ Signal(Out(1,X1)) = o
```

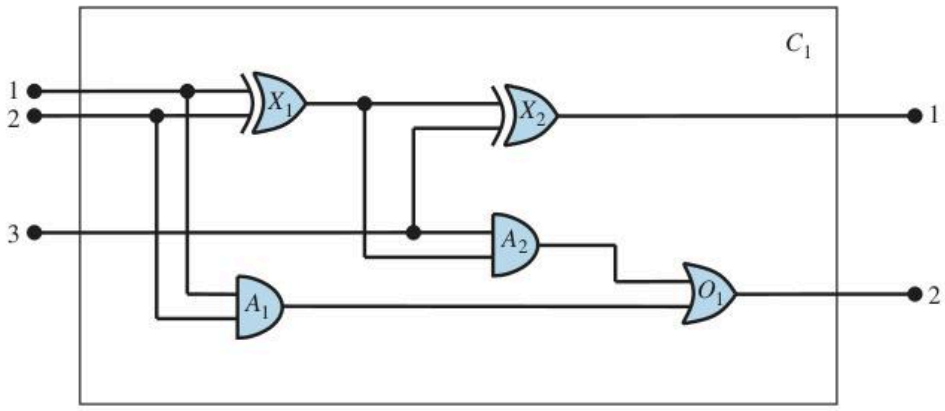
که آشکار می‌کند برای موارد ورودی 10 و 01 هیچ خروجی‌ای در X1 شناخته نشده. سپس، به اصل‌موضوعهٔ گیت‌های XOR نگاه می‌کنیم، چنان‌که بر X1 اعمال می‌شود:

```
Signal(Out(1,X1)) = 1 ⇔ Signal(In(1,X1)) ≠ Signal(In(2,X1))
```

اگر ورودی‌ها معلوم باشند، مثلاً 1 و 0، این به این تقلیل می‌یابد:

```
Signal(Out(1,X1)) = 1 ⇔ 1 ≠ 0
```

حال مسئله آشکار است: سامانه قادر به استنتاج Signal(Out(1,X1)) = 1 نیست، پس باید به آن بگوییم 1 ≠ 0.



شکل 8.6 – مدار دیجیتال C1، که ادعا می‌شود جمع‌کنندهٔ کامل یک‌بیتی است. دو ورودی نخست دو بیتی هستند که باید جمع شوند و ورودی سوم بیت نقلی است. خروجی نخست جمع و خروجی دوم بیت نقلی برای جمع‌کنندهٔ بعدی است. مدار شامل دو گیت XOR، دو گیت AND و یک گیت OR است.

خلاصهٔ فصل

این فصل منطق مرتبهٔ اول را معرفی کرده، زبان بازنمایی‌ای که بسیار قدرتمندتر از منطق گزاره‌ای است. نکته‌های مهم چنین‌اند:

- زبان‌های بازنمایی دانش باید اعلانی، ترکیب‌پذیر، بیانی، مستقل از بافت و بدون‌ابهام باشند.
- منطق‌ها در تعهدهای هستی‌شناختی و تعهدهای شناختی‌شان متفاوت‌اند. در حالی که منطق گزاره‌ای فقط به وجود واقعیت‌ها متعهد می‌شود، منطق مرتبهٔ اول به وجود اشیا و روابط متعهد می‌شود و بدین ترتیب قدرت بیانی به دست می‌آورد، مناسب برای حوزه‌هایی مانند جهان وومپوس و مدارهای الکترونیکی.
- هر دو منطق گزاره‌ای و مرتبهٔ اول در بازنمایی گزاره‌های مبهم دشواری مشترکی دارند. این دشواری کاربردپذیری آن‌ها را در حوزه‌هایی که به قضاوت شخصی نیاز دارند، مانند سیاست یا آشپزی، محدود می‌کند.
- نحو منطق مرتبهٔ اول بر نحو منطق گزاره‌ای بنا می‌شود. عبارت‌هایی برای بازنمایی اشیا می‌افزاید و سورهای همگانی و وجودی برای ساختن ادعاهایی دربارهٔ همه یا برخی از مقادیر ممکن متغیرهای سوردار دارد.
- جهان ممکن، یا مدل، برای منطق مرتبهٔ اول شامل مجموعه‌ای از اشیا و تفسیری است که نمادهای ثابت را به اشیا، نمادهای گزاره را به روابط میان اشیا، و نمادهای تابع را به توابع روی اشیا نگاشت می‌کند.
- جملهٔ اتمی فقط وقتی صادق است که رابطه‌ای که گزاره نام می‌برد میان اشیایی که عبارت‌ها نام می‌برند برقرار باشد. تفسیرهای بسط‌یافته، که متغیرهای سور را به اشیای مدل نگاشت می‌کنند، صدق جمله‌های سوردار را تعریف می‌کنند.
- توسعهٔ پایگاه دانش در منطق مرتبهٔ اول به فرایند دقیقی از تحلیل حوزه، انتخاب واژگان و رمزگذاری اصل‌موضوعه‌های لازم برای پشتیبانی از استنتاج‌های مطلوب نیاز دارد.

یادداشت‌های کتاب‌شناختی و تاریخی

هرچند منطق ارسطو با تعمیم‌هایی روی اشیا سروکار داشت، بسیار پایین‌تر از قدرت بیانی منطق مرتبهٔ اول بود. سد بزرگ بر سر راه توسعهٔ بیشترش، تمرکزش بر گزاره‌های یک‌جایی به استثنای گزاره‌های رابطه‌ای چندجایی بود. نخستین برخورد نظام‌مند با روابط توسط آگوستوس دمورگان (۱۸۶۴) انجام شد، که مثال زیر را برای نشان دادن انواع استنتاج‌هایی که منطق ارسطو نمی‌توانست مدیریت کند ذکر کرد: «همهٔ اسب‌ها جانورند؛ پس سرِ اسب، سرِ جانور است.» این استنتاج برای ارسطو غیرقابل دسترسی است، زیرا هر قاعدهٔ معتبری که بتواند این استنتاج را پشتیبانی کند باید نخست جمله را با گزارهٔ دوجایی «x سرِ y است» تحلیل کند. منطق روابط با ژرفا توسط چارلز سندرز پیرس (پیرس، ۱۸۷۰؛ میساک، ۲۰۰۴) مطالعه شد.

منطق مرتبهٔ اول واقعی از معرفی سورها در «نگارش مفهومی» گوتلوب فرگه (۱۸۷۹) تاریخ می‌گیرد. پیرس (۱۸۸۳) نیز مستقل از فرگه، هرچند کمی بعدتر، منطق مرتبهٔ اول را توسعه داد. توانایی فرگه در تودرتوسازی سورها گام بزرگی به جلو بود، اما او از نماد دست‌وپاگیری استفاده کرد. نماد کنونی منطق مرتبهٔ اول تا حد زیادی از جوزپه پئانو (۱۸۸۹) است، اما معناشناسی عملاً با معناشناسی فرگه یکسان است. به‌طرز عجیبی، اصل‌موضوعه‌های پئانو تا حد زیادی از گراسمن (۱۸۶۱) و ددکیند (۱۸۸۸) بود.

لئوپولد لوونهایم (۱۹۱۵) برخورد نظام‌مندی با نظریهٔ مدل برای منطق مرتبهٔ اول ارائه داد، شامل نخستین برخورد درست با نماد برابری. نتایج لوونهایم بیشتر توسط تورالف اسکولم (۱۹۲۰) بسط یافت. آلفرد تارسکی (۱۹۳۵، ۱۹۵۶) تعریف صریحی از صدق و ارضای نظریه‌مدل در منطق مرتبهٔ اول، با استفاده از نظریهٔ مجموعه‌ها، ارائه داد.

جان مک‌کارتی (۱۹۵۸) عمدتاً مسئول معرفی منطق مرتبهٔ اول به‌عنوان ابزاری برای ساختن سامانه‌های هوش مصنوعی بود. چشم‌اندازهای هوش مصنوعی مبتنی بر منطق به‌شکل معناداری توسط توسعهٔ حل (Resolution) توسط رابینسون (۱۹۶۵)، رویه‌ای کامل برای استنتاج مرتبهٔ اول، پیش برده شد. رویکرد منطق‌گرایانه در دانشگاه استنفورد ریشه دواند. کوردل گرین (۱۹۶۹) سامانهٔ استدلال مرتبهٔ اولی به نام QA3 توسعه داد که به نخستین تلاش‌ها برای ساختن ربات منطقی در SRI انجامید (فایکز و نیلسون، ۱۹۷۱). منطق مرتبهٔ اول توسط زوهار مانا و ریچارد والدینگر (۱۹۷۱) برای استدلال دربارهٔ برنامه‌ها و بعداً توسط مایکل جنزِرت (۱۹۸۴) برای استدلال دربارهٔ مدارها اعمال شد. در اروپا، برنامه‌نویسی منطقی (شکل محدودی از استدلال مرتبهٔ اول) برای تحلیل زبانی (کولمرائر و همکاران، ۱۹۷۳) و برای سامانه‌های اعلانی عمومی (کوالسکی، ۱۹۷۴) توسعه یافت. منطق محاسباتی همچنین در ادینبورگ از طریق پروژهٔ LCF (منطق برای توابع قابل محاسبه) (گوردون و همکاران، ۱۹۷۹) به خوبی جا افتاد. این تحولات بیشتر در فصل‌های ۹ و ۱۰ ثبت شده‌اند.

کاربردهای عملی ساخته‌شده با منطق مرتبهٔ اول شامل سامانه‌ای برای ارزیابی الزام‌های تولیدی محصولات الکترونیکی (منیون، ۲۰۰۲)، سامانه‌ای برای استدلال دربارهٔ سیاست‌های دسترسی به پرونده و مدیریت حقوق دیجیتال (هالپرن و وایسمن، ۲۰۰۸)، و سامانه‌ای برای ترکیب خودکار خدمات وب (مک‌ایلریث و زنگ، ۲۰۰۱) است.

واکنش‌ها به فرضیهٔ ورف (ورف، ۱۹۵۶) و مسئلهٔ زبان و اندیشه به‌طور کلی، در کتاب‌های متعددی ظاهر می‌شود (پولوم، ۱۹۹۱؛ پینکر، ۲۰۰۳) از جمله عنوان‌های ظاهراً متضاد «چرا جهان در زبان‌های دیگر متفاوت به نظر می‌رسد» (دویچر، ۲۰۱۰) و «چرا جهان در هر زبانی یکسان به نظر می‌رسد» (مک‌ورتر، ۲۰۱۴) (هرچند هر دو نویسنده توافق دارند تفاوت‌هایی هست و تفاوت‌ها کوچک‌اند). «نظریه»ِ نظریه (گاپنیک و گلیمور، ۲۰۰۲؛ تننبام و همکاران، ۲۰۰۷) یادگیری کودکان دربارهٔ جهان را مشابه ساخت نظریه‌های علمی می‌بیند. همان‌گونه که پیش‌بینی‌های الگوریتم یادگیری ماشین به‌شدت به واژگان عرضه‌شده به آن بستگی دارد، صورت‌بندی نظریه‌ها توسط کودک نیز به محیط زبانی‌ای که یادگیری در آن رخ می‌دهد بستگی خواهد داشت.

مقدمه‌های خوب متعددی بر منطق مرتبهٔ اول هست، از جمله برخی از چهره‌های برجستهٔ تاریخ منطق: آلفرد تارسکی (۱۹۴۱)، آلونزو چرچ (۱۹۵۶) و دبلیو. وی. کواین (۱۹۸۲) (که یکی از خواندنی‌ترین‌هاست). اندرتون (۱۹۷۲) دیدگاه ریاضی‌محورتری می‌دهد. برخورد بسیار صوری با منطق مرتبهٔ اول، همراه با بسیاری موضوعات پیشرفته‌تر در منطق، توسط بل و ماشوور (۱۹۷۷) ارائه شده. مانا و والدینگر (۱۹۸۵) مقدمهٔ خواندنی از منطق از دیدگاه علوم رایانه می‌دهند،

و هاث و رایان (۲۰۰۴) نیز که بر تأیید برنامه تمرکز دارند. باروایز و اچمندای (۲۰۰۲) رویکردی شبیه رویکرد استفاده‌شده در این‌جا می‌گیرند. اسمولیان (۱۹۹۵) نتایج را موجز، با استفاده از قالب جدول، ارائه می‌دهد. گالیه (۱۹۸۶) شرح ریاضی فوق‌العاده دقیقی از منطق مرتبهٔ اول، همراه با مطالب فراوانی دربارهٔ استفاده‌اش در استدلال خودکار، ارائه می‌دهد. «بنیان‌های منطقی هوش مصنوعی» (جنزِرت و نیلسون، ۱۹۸۷) هم مقدمه‌ای استوار بر منطق است و هم نخستین برخورد نظام‌مند با عامل‌های منطقیِ دارای ادراک و کنش، و دو کتاب راهنمای خوب هست: فان بنتام و تر مولن (۱۹۹۷) و رابینسون و ورونکوف (۲۰۰۱). ژورنال ثبت حوزهٔ منطق ریاضی ناب «ژورنال منطق نمادین» است، در حالی که «ژورنال منطق کاربردی» با دغدغه‌های نزدیک‌تر به هوش مصنوعی سروکار دارد.

پانوشت‌ها:

[1] منطق مرتبهٔ اول همچنین حساب گزاره‌ای مرتبهٔ اول نامیده می‌شود؛ ممکن است به اختصار FOL یا FOPC نوشته شود.

[2] بعداً، در بخش 8.2.8، معناشناسی‌ای بررسی می‌کنیم که در آن هر شیء باید دقیقاً یک نام داشته باشد.

[3] عبارت‌های λ (عبارت‌های لامبدا) نماد مفیدی فراهم می‌کنند که در آن نمادهای تابع تازه «در حال اجرا» ساخته می‌شوند. برای نمونه، تابعی که آرگومانش را مربع می‌کند را می‌توان (λx : x×x) نوشت و مانند هر نماد تابع دیگری بر آرگومان‌ها اعمال کرد. عبارت λ را می‌توان تعریف و به‌عنوان نماد گزاره نیز استفاده کرد. عملگر لامبدا در لیسپ و پایتون دقیقاً همان نقش را بازی می‌کند. توجه کنید استفاده از λ به این شکل قدرت بیانی صوری منطق مرتبهٔ اول را افزایش نمی‌دهد، زیرا هر جمله‌ای شامل عبارت λ را می‌توان با «جای‌گذاری» آرگومان‌هایش بازنویسی کرد تا جملهٔ هم‌ارزی به دست آید.

[4] معمولاً قرارداد ترتیب آرگومان را دنبال می‌کنیم که P(x,y) به‌صورت «x یک P از y است» خوانده شود.

[5] این هستی‌شناسی فقط یک پدر و یک مادر برای هر شخص می‌شناسد. هستی‌شناسی پیچیده‌تری می‌تواند مادر زیستی، مادر تولد، مادرخوانده و... را بشناسد.

[6] تغییری از سور وجودی هست، معمولاً ∃1 یا ∃! نوشته می‌شود، که یعنی «دقیقاً یکی وجود دارد». همان معنا را می‌توان با گزاره‌های برابری بیان کرد.

[7] در واقع او چهار برادر داشت، بقیه ویلیام و هنری بودند.

[8] اصل‌موضوعه‌های پئانو همچنین اصل استقرا را شامل می‌شوند که جمله‌ای از منطق مرتبهٔ دوم است نه منطق مرتبهٔ اول. اهمیت این تمایز در فصل ۹ توضیح داده شده.

[9] به همین ترتیب، بیشتر ما هر پرنده‌ای را که در زمستان به مناطق گرم‌تر مهاجرت می‌کند نام‌گذاری نمی‌کنیم. پرنده‌شناسی که می‌خواهد الگوهای مهاجرت، نرخ بقا و مانند آن را مطالعه کند، هر پرنده را با حلقه‌ای روی پایش نام‌گذاری می‌کند، زیرا پرنده‌های فردی باید ردیابی شوند.

فصل ۹: استنتاج در منطق مرتبهٔ اول (Inference in First-Order Logic)

در این فصل رویه‌های مؤثری تعریف می‌کنیم برای پاسخ به پرسش‌هایی که در منطق مرتبهٔ اول مطرح می‌شوند.

در این فصل، الگوریتم‌هایی را توصیف می‌کنیم که می‌توانند به هر پرسشِ پاسخ‌پذیرِ منطق مرتبهٔ اول پاسخ دهند. بخش ۹.۱ قواعد استنتاج برای سورها را معرفی می‌کند و نشان می‌دهد چگونه استنتاج مرتبهٔ اول را به استنتاج گزاره‌ای تقلیل دهیم، هرچند احتمالاً به هزینه‌ای بالقوهٔ بزرگ. بخش ۹.۲ توصیف می‌کند چگونه یکسان‌سازی (Unification) را می‌توان برای ساختن قواعد استنتاجی که مستقیماً با جمله‌های مرتبهٔ اول کار می‌کنند استفاده کرد. سپس سه خانوادهٔ بزرگ الگوریتم‌های استنتاج مرتبهٔ اول را بحث می‌کنیم: زنجیره‌سازی پیشرو (بخش ۹.۳)، زنجیره‌سازی پسرو (بخش ۹.۴) و اثبات قضیهٔ مبتنی بر حل (بخش ۹.۵).

۹.۱ استنتاج گزاره‌ای در برابر مرتبهٔ اول

یکی از راه‌های انجام استنتاج مرتبهٔ اول، تبدیل پایگاه دانش مرتبهٔ اول به منطق گزاره‌ای و استفاده از استنتاج گزاره‌ای است، که از قبل می‌دانیم چگونه انجامش دهیم. گام نخست حذف سورهای همگانی است. برای نمونه، فرض کنیم پایگاه دانش ما این فرض سنتی را دارد که همهٔ پادشاهان حریص شرورند:

$$\forall x \; King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$

از آن می‌توانیم هر یک از جمله‌های زیر را استنتاج کنیم:

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$
$$King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$$
$$King(Father(John)) \wedge Greedy(Father(John)) \Rightarrow Evil(Father(John))$$
$$\ldots$$

در حالت کلی، قاعدهٔ «مصداق‌سازی همگانی» (Universal Instantiation، به اختصار UI) می‌گوید می‌توانیم هر جمله‌ای را که با جانشینی یک عبارت زمینی (عبارتی بدون متغیر) برای متغیرِ همگانی‌سوردار به دست می‌آید استنتاج کنیم.[1]

برای نوشتن صوری قاعدهٔ استنتاج، از مفهوم جانشینی‌های معرفی‌شده در بخش ۸.۳ استفاده می‌کنیم. $SUBST(\theta, \alpha)$ نتیجهٔ اعمال جانشینی θ بر جملهٔ α باشد. آنگاه قاعده چنین نوشته می‌شود:

```
        v αᐁ
  ──────────
SUBST({v/g}, α)
```

برای هر متغیر v و عبارت زمینی g. برای نمونه، سه جملهٔ پیش‌ترداده‌شده با جانشینی‌های {x/John}، {x/Richard} و {x/Father(John)} به دست می‌آیند.

به همین ترتیب، قاعدهٔ «مصداق‌سازی وجودی» (Existential Instantiation) متغیرِ وجودی‌سوردار را با یک نماد ثابت تازه جایگزین می‌کند. بیان صوری چنین است: برای هر جملهٔ α، متغیر v و نماد ثابت k که در جای دیگر پایگاه دانش ظاهر نمی‌شود:

```
        v αᐃ
  ──────────
SUBST({v/k}, α)
```

برای نمونه، از جملهٔ

```
x Crown(x) ∧ OnHead(x, John)ᐃ
```

می‌توانیم جملهٔ

```
Crown(C1) ∧ OnHead(C1, John)
```

را استنتاج کنیم، تا وقتی C1 در جای دیگر پایگاه دانش ظاهر نشود. اساساً، جملهٔ وجودی می‌گوید شیئی هست که شرطی را برآورده می‌کند و اعمال قاعدهٔ مصداق‌سازی وجودی فقط به آن شیء نامی می‌دهد. البته، آن نام نباید از قبل متعلق به شیء دیگری باشد. ریاضیات مثال خوبی می‌دهد: فرض کنیم کشف کنیم عددی هست که کمی بزرگ‌تر از 2.71828 است و معادلهٔ d(x^y)/dy = x^y را برای x برآورده می‌کند. می‌توانیم به این عدد نام e بدهیم، اما اشتباه است نام شیء موجودی مانند π را به آن بدهیم. در منطق، نام تازه «ثابت اسکولم» (Skolem Constant) نامیده می‌شود.

در حالی که مصداق‌سازی همگانی را می‌توان بارها بر همان اصل‌موضوعه اعمال کرد تا پیامدهای متفاوت فراوانی تولید کند، مصداق‌سازی وجودی فقط یک بار لازم است اعمال شود و سپس جملهٔ وجودی‌سوردار را می‌توان دور ریخت. برای نمونه، پس از افزودن جملهٔ Kill(Murderer, Victim) دیگر به x Kill(x, Victim)ᐃ نیاز نداریم.

۹.۱.۱ تقلیل به استنتاج گزاره‌ای

اکنون نشان می‌دهیم چگونه هر پایگاه دانش مرتبهٔ اولی را به پایگاه دانش گزاره‌ای تبدیل کنیم. ایدهٔ نخست آن است که، درست همان‌گونه که جملهٔ وجودی‌سوردار را می‌توان با یک مصداق‌سازی جایگزین کرد، جملهٔ همگانی‌سوردار را می‌توان با مجموعهٔ همهٔ مصداق‌سازی‌های ممکن جایگزین کرد. برای نمونه، فرض کنیم پایگاه دانش ما فقط جمله‌های زیر را دارد:

$$\begin{aligned} &\forall x \ King(x) \wedge Greedy(x) \Rightarrow Evil(x) \\ &King(John) \\ &Greedy(John) \\ &Brother(Richard, John) \end{aligned} \quad (۹.۱)$$

و تنها اشیا John و Richard هستند. UI را با همهٔ جانشینی‌های ممکن، {x/John} و {x/Richard}، بر جملهٔ نخست اعمال می‌کنیم. به دست می‌آوریم:

$$\begin{aligned} &King(John) \wedge Greedy(John) \Rightarrow Evil(John) \\ &King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard) \end{aligned}$$

سپس جمله‌های اتمی زمینی، مانند King(John)، را با نمادهای گزاره‌ای، مانند JohnIsKing، جایگزین می‌کنیم. سرانجام، هر یک از الگوریتم‌های گزاره‌ای کامل فصل ۷ را اعمال می‌کنیم تا نتیجه‌هایی مانند JohnIsEvil به دست آوریم که با Evil(John) هم‌ارز است.

این فن «گزاره‌سازی» (Propositionalization) را می‌توان کاملاً عمومی کرد، چنان‌که در بخش ۹.۵ نشان می‌دهیم. اما مسئله‌ای هست وقتی پایگاه دانش نماد تابعی داشته باشد: مجموعهٔ جانشینی‌های عبارت زمینی ممکن نامتناهی است! برای نمونه، اگر پایگاه دانش نماد Father را ذکر کند، عبارت‌های تودرتوی بی‌شماری مانند Father(Father(Father(John))) قابل ساخت‌اند.

خوشبختانه، قضیهٔ نامداری از ژاک اربن (۱۹۳۰) هست که می‌گوید اگر جمله‌ای توسط پایگاه دانش مرتبهٔ اول اصلی مستلزم باشد، برهانی شامل فقط زیرمجموعهٔ متناهی‌ای از پایگاه دانش گزاره‌شده وجود دارد. چون هر چنین زیرمجموعه‌ای بیشینهٔ عمق تودرتویی در میان عبارت‌های زمینی‌اش دارد، می‌توانیم زیرمجموعه را بیابیم با نخست تولید همهٔ مصداق‌سازی‌ها با نمادهای ثابت (Richard و John)، سپس همهٔ عبارت‌های عمق ۱ (Father(Richard) و Father(John))، سپس همهٔ عبارت‌های عمق ۲، و الی آخر، تا آنکه بتوانیم برهان گزاره‌ای جملهٔ مستلزم را بسازیم.

رویکردی به استنتاج مرتبهٔ اول از طریق گزاره‌سازی ترسیم کرده‌ایم که کامل است — یعنی هر جملهٔ مستلزمی قابل اثبات است. این دستاورد بزرگی است، با توجه به اینکه فضای مدل‌های ممکن نامتناهی است. از سوی دیگر، تا پایان برهان نمی‌دانیم جمله مستلزم است! وقتی جمله مستلزم نباشد چه رخ می‌دهد؟ آیا می‌توانیم بگوییم؟ خوب، برای منطق مرتبهٔ اول، معلوم می‌شود نمی‌توانیم. رویهٔ برهان ما می‌تواند ادامه دهد و ادامه دهد و عبارت‌های عمیق‌تر و عمیق‌تری تولید کند، اما نمی‌دانیم آیا در حلقه‌ای ناامیدکننده گیر کرده یا برهان در شرف به‌دست‌آمدن است. این بسیار

شبیه مسئلهٔ توقف برای ماشین‌های تورینگ است. آلن تورینگ (۱۹۳۶) و آلونزو چرچ (۱۹۳۶) هر دو، به روش‌های نسبتاً متفاوت، گریزناپذیری این وضعیت امور را اثبات کردند. پرسش Δ لازمه برای منطق مرتبهٔ اول «نیمه‌تصمیم‌پذیر» (Semidecidable) است — یعنی الگوریتم‌هایی وجود دارند که به هر جملهٔ مستلزم بله می‌گویند، اما هیچ الگوریتمی وجود ندارد که به هر جملهٔ غیرمستلزم نیز نه بگوید.

۹.۲ یکسان‌سازی و استنتاج مرتبهٔ اول

خوانندهٔ تیزبین متوجه شده رویکرد گزاره‌سازی مصداق‌سازی‌های غیرضروری فراوانی از جمله‌های همگانی‌سوردار تولید می‌کند. ترجیح می‌دهیم رویکردی داشته باشیم که فقط از یک قاعده استفاده کند، با این استدلال که {x/John} پرس‌وجوی Evil(x) را چنین حل می‌کند: با توجه به قاعدهٔ «پادشاهان حریص شرورند»، برخی x را بیاب چنان‌که x پادشاه و x حریص باشد، و سپس استنتاج کن این x شرور است. عمومی‌تر، اگر جانشینی θ وجود داشته باشد که هر یک از عطف‌شونده‌های مقدم دلالت را با جمله‌هایی که از قبل در پایگاه دانش‌اند یکسان کند، آنگاه می‌توانیم تالی دلالت را، پس از اعمال θ، ادعا کنیم. در این مورد، جانشینی θ={x/John} به آن هدف می‌رسد. حال فرض کنیم به‌جای دانستن Greedy(John)، بدانیم همه حریص‌اند:

$$\forall y \ \text{Greedy}(y) \qquad (۹.۲)$$

آنگاه باز هم می‌خواهیم بتوانیم نتیجه بگیریم Evil(John)، زیرا می‌دانیم جان پادشاه است (داده‌شده) و جان حریص است (زیرا همه حریص‌اند). آنچه برای این کار نیاز داریم یافتن جانشینی برای هم متغیرهای جملهٔ دلالت و هم متغیرهای جمله‌های موجود در پایگاه دانش است. در این مورد، اعمال جانشینی {x/John, y/John} بر مقدمه‌های دلالت King(x) و Greedy(x) و جمله‌های پایگاه دانش King(John) و Greedy(y) آن‌ها را یکسان می‌کند. پس می‌توانیم تالی دلالت را استنتاج کنیم.

این فرایند استنتاج را می‌توان در قاعدهٔ استنتاج تکی‌ای به نام «مدوس پوننس تعمیم‌یافته» (Generalized Modus Ponens)[۲] در آورد: برای جمله‌های اتمی p'_i، p_i و q، که در آن جانشینی θ وجود دارد چنان‌که:

$$\text{SUBST}(\theta, p'_i) = \text{SUBST}(\theta, p_i)، \quad \text{برای همهٔ } i$$

$$\frac{p'_1, p'_2, \ldots, p'_n, (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

این قاعده n+1 مقدمه دارد: n جملهٔ اتمی p'_i و یک دلالت. نتیجه، حاصل اعمال جانشینی θ بر تالی q است. برای مثال ما:

```
                p′1 = King(John)
                   p1  = King(x)
                 p′2 = Greedy(y)
                  p2  = Greedy(x)
θ   = {x/John, y/John}
                   q   = Evil(x)
SUBST(θ, q) = Evil(John)
```

نشان‌دادن درست‌بودن مدوس پوننس تعمیم‌یافته آسان است. نخست، مشاهده می‌کنیم برای هر جملهٔ p (که متغیرهایش همگانی‌سوردار فرض می‌شوند) و هر جانشینی θ،

```
p ⊨ SUBST(θ, p)
```

طبق مصداق‌سازی همگانی صادق است. به‌ویژه برای θای صادق است که شرایط قاعدهٔ مدوس پوننس تعمیم‌یافته را برآورده می‌کند. پس از p'1,...,p'n می‌توانیم استنتاج کنیم:

```
SUBST(θ, p′1) ∧ ... ∧ SUBST(θ, p′n)
```

و از دلالت p1 ∧ ... ∧ pn ⇒ q می‌توانیم استنتاج کنیم:

```
SUBST(θ, p1) ∧ ... ∧ SUBST(θ, pn) ⇒ SUBST(θ, q)
```

حال، θ در مدوس پوننس تعمیم‌یافته چنان تعریف شده که SUBST(θ, p'i) = SUBST(θ, pi) برای همهٔ i؛ پس نخستینِ این دو جمله دقیقاً با مقدم جملهٔ دوم مطابقت می‌کند. از این رو، SUBST(θ, q) با مدوس پوننس برمی‌آید.

مدوس پوننس تعمیم‌یافته نسخهٔ «بالابرده‌شده» (Lifted) مدوس پوننس است — مدوس پوننس را از منطق گزاره‌ای زمینی (بدون متغیر) به منطق مرتبهٔ اول بالا می‌برد. در بقیهٔ این فصل خواهیم دید که می‌توانیم نسخه‌های بالابرده‌شدهٔ زنجیره‌سازی پیشرو، زنجیره‌سازی پسرو و الگوریتم‌های حلِ معرفی‌شده در فصل ۷ را توسعه دهیم. مزیت کلیدی قواعد استنتاج بالابرده نسبت به گزاره‌سازی آن است که فقط آن جانشینی‌هایی را انجام می‌دهند که برای اجازه‌دادن به پیش‌رفتن استنتاج‌های خاص لازم‌اند.

۹.۲.۱ یکسان‌سازی

قواعد استنتاج بالابرده مستلزم یافتن جانشینی‌هایی هستند که عبارت‌های منطقی متفاوت را یکسان کنند. این فرایند «یکسان‌سازی» (Unification) نامیده می‌شود و مؤلفهٔ کلیدی همهٔ الگوریتم‌های استنتاج مرتبهٔ اول است. الگوریتم UNIFY دو جمله می‌گیرد و «یکسان‌ساز» (Unifier) آن‌ها (یک جانشینی) را بازمی‌گرداند، اگر وجود داشته باشد:

```
UNIFY(p, q) = θ  که در آن  SUBST(θ, p) = SUBST(θ, q)
```

به چند مثال از چگونگی رفتار UNIFY نگاه کنیم. فرض کنیم پرس‌وجوی AskVars(Knows(John,x)) داریم: جان چه کسی را می‌شناسد؟ پاسخ‌های این پرس‌وجو را می‌توان با یافتن همهٔ جمله‌های پایگاه دانش که با Knows(John,x) یکسان می‌شوند پیدا کرد. این‌ها نتایج یکسان‌سازی با چهار جملهٔ متفاوت است که ممکن است در پایگاه دانش باشد:

```
UNIFY(Knows(John,x), Knows(John,Jane)) = {x/Jane}
UNIFY(Knows(John,x), Knows(y,Bill)) = {x/Bill, y/John}
UNIFY(Knows(John,x), Knows(y,Mother(y))) = {y/John, x/Mother(John)}
UNIFY(Knows(John,x), Knows(x,Elizabeth)) = failure
```

آخرین یکسان‌سازی شکست می‌خورد، زیرا x نمی‌تواند هم‌زمان مقادیر John و Elizabeth را بگیرد. حال، به یاد آورید Knows(x,Elizabeth) یعنی «همه الیزابت را می‌شناسند»، پس باید بتوانیم استنتاج کنیم جان الیزابت را می‌شناسد. مسئله فقط به‌سبب آن پیش می‌آید که دو جمله اتفاقاً از همان نام متغیر x استفاده می‌کنند. مسئله را می‌توان با «استانداردسازی جداسازی» (Standardizing Apart) یکی از دو جملهٔ در حال یکسان‌سازی اجتناب کرد، که یعنی تغییر نام متغیرهایش برای پرهیز از برخورد نام‌ها. برای نمونه، می‌توانیم x در Knows(x,Elizabeth) را به x17 (نام متغیر تازه‌ای) تغییر نام دهیم بدون تغییر معنا. حال یکسان‌سازی کار خواهد کرد:

```
UNIFY(Knows(John,x), Knows(x17,Elizabeth)) = {x/Elizabeth, x17/John}
```

تمرین STAN.9 عمیق‌تر به نیاز به استانداردسازی جداسازی می‌پردازد.

یک پیچیدگی دیگر هست: گفتیم UNIFY باید جانشینی‌ای بازگرداند که دو آرگومان را یکسان کند. اما می‌تواند بیش از یک چنین یکسان‌سازی وجود داشته باشد. برای نمونه، UNIFY(Knows(John,x), Knows(y,z)) می‌تواند {y/John, x/z} بازگرداند یا می‌تواند {y/John, x/John, z/John} بازگرداند. یکسان‌ساز نخست Knows(John,z) را به‌عنوان نتیجهٔ یکسان‌سازی می‌دهد، در حالی که دومین Knows(John,John) می‌دهد. نتیجهٔ دوم را می‌توان از نخستین با جانشینی اضافی {z/John} به دست آورد؛ می‌گوییم یکسان‌ساز نخست «عمومی‌تر» از دومین است، زیرا قیدهای کمتری بر مقادیر متغیرها می‌گذارد.

هر جفت عبارتِ یکسان‌شدنی یک «عمومی‌ترین یکسان‌ساز» (Most General Unifier، یا MGU) تکی دارد که تا حد تغییر نام و جانشینی متغیر یکتاست. برای نمونه، {x/John} و {y/John} هم‌ارز دانسته می‌شوند، چنان‌که {x/John, y/John} و {y/John, x/John}.

الگوریتمی برای محاسبهٔ عمومی‌ترین یکسان‌سازها در شکل 9.1 نشان داده شده. فرایند ساده است: دو عبارت را به‌صورت بازگشتی هم‌زمان «کنار هم» کاوش کن و در طول راه یکسان‌سازی را بساز، اما اگر دو نقطهٔ متناظر در ساختارها مطابقت نکردند شکست بخور. یک گام پرهزینه هست: هنگام مطابقت متغیر با عبارت پیچیده، باید بررسی شود آیا خودِ متغیر درون عبارت رخ می‌دهد؛ اگر رخ دهد، مطابقت شکست می‌خورد، زیرا هیچ یکسان‌ساز سازگاری نمی‌توان ساخت. برای نمونه، S(x) نمی‌تواند با S(S(x)) یکسان شود. این به اصطلاح «بررسی رخداد» (Occur Check) پیچیدگی کل الگوریتم را در اندازهٔ عبارت‌های در حال یکسان‌سازی درجهٔ دوم می‌کند. برخی سامانه‌ها، از جمله بسیاری از

سامانه‌های برنامه‌نویسی منطقی، صرفاً بررسی رخداد را حذف می‌کنند و بار اجتناب از استنتاج‌های نادرستِ ناشی از آن را بر دوش کاربر می‌گذارند. سامانه‌های دیگر از الگوریتم‌های یکسان‌سازی پیچیده‌تری با پیچیدگی زمانی خطی استفاده می‌کنند.

```
function UNIFY(x, y, θ = empty) returns a substitution to make x and y identical, or
                                                                        failure
                                                  if θ = failure then return failure
                                                        else if x = y then return θ
                                        else if VARIABLE?(x) then return UNIFY-VAR(x, y, θ)
                                        else if VARIABLE?(y) then return UNIFY-VAR(y, x, θ)
                                                else if COMPOUND?(x) and COMPOUND?(y) then
                                    return UNIFY(ARGS(x), ARGS(y), UNIFY(OP(x), OP(y), θ))
                                                  else if LIST?(x) and LIST?(y) then
                                return UNIFY(REST(x), REST(y), UNIFY(FIRST(x), FIRST(y), θ))
                                                                  else return failure

                                        function UNIFY-VAR(var, x, θ) returns a substitution
                                    if {var/val} ∈ θ for some val then return UNIFY(val, x, θ)
                                else if {x/val} ∈ θ for some val then return UNIFY(var, val, θ)
                                        else if OCCUR-CHECK?(var, x) then return failure
                                                        else return add {var/x} to θ
```

شکل ۹.۱ — الگوریتم یکسان‌سازی. آرگومان‌های x و y می‌توانند هر عبارتی باشند: ثابت یا متغیر، یا عبارت مرکبی مانند جملهٔ پیچیده یا عبارت، یا فهرستی از عبارت‌ها. آرگومان θ جانشینی است، در آغاز جانشینی خالی، اما با جفت‌های {var/val} که هنگام بازگشت در ورودی‌ها، با مقایسهٔ عبارت‌ها عنصربه‌عنصر، به آن افزوده می‌شود. در عبارت مرکبی مانند F(A,B)، فیلد OP(x) نماد تابع F و فیلد ARGS(x) فهرست آرگومان (A,B) را برمی‌گزینند.

۹.۲.۲ ذخیره‌سازی و بازیابی

زیربنای توابع ASK، TELL و ASKVARS که برای آگاه‌کردن و بازجویی از پایگاه دانش استفاده می‌شوند، توابع ابتدایی‌تر STORE و FETCH هستند. STORE(s) جملهٔ s را در پایگاه دانش ذخیره می‌کند و FETCH(q) همهٔ یکسان‌سازهایی را بازمی‌گرداند چنان‌که پرس‌وجوی q با جمله‌ای در پایگاه دانش یکسان شود. مسئله‌ای که برای نشان دادن یکسان‌سازی استفاده کردیم — یافتن همهٔ واقعیت‌هایی که با Knows(John,x) یکسان می‌شوند — مصداقی از FETCH است.

ساده‌ترین راه پیاده‌سازی STORE و FETCH آن است که همهٔ واقعیت‌ها را در یک فهرست بلند نگه داریم و هر پرس‌وجو را با هر عنصر فهرست یکسان کنیم. چنین فرایندی ناکارآمد است، اما کار می‌کند. بقیهٔ این بخش راه‌هایی برای کارآمدترکردن بازیابی ترسیم می‌کند.

می‌توانیم FETCH را کارآمدتر کنیم با اطمینان از اینکه یکسان‌سازی‌ها فقط با جمله‌هایی که شانسی برای یکسان‌شدن دارند انجام می‌شوند. برای نمونه، هیچ فایده‌ای در تلاش برای یکسان‌کردن Knows(John,x) با Brother(Richard,John) نیست. می‌توانیم با «فهرست‌بندی» (Indexing) واقعیت‌های پایگاه دانش از چنین یکسان‌سازی‌هایی اجتناب کنیم. طرح ساده‌ای به نام «فهرست‌بندی گزاره‌ای» (Predicate Indexing) همهٔ واقعیت‌های Knows را در یک سطل و همهٔ واقعیت‌های Brother را در سطلی دیگر می‌گذارد. سطل‌ها را می‌توان در جدول درهم‌سازی برای دسترسی کارآمد ذخیره کرد.

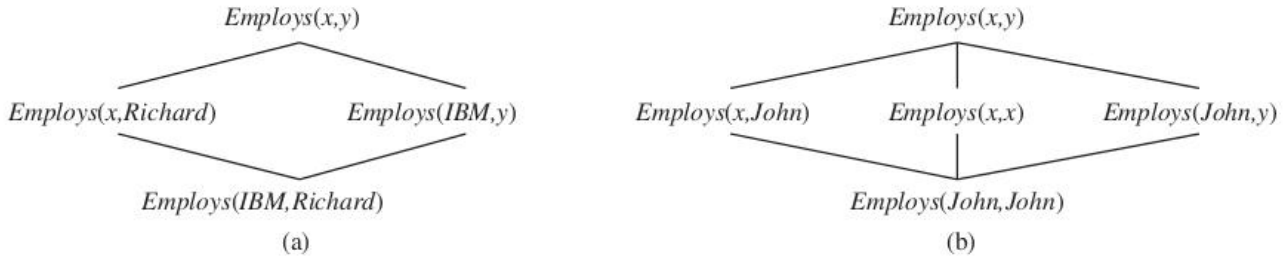
فهرست‌بندی گزاره‌ای وقتی مفید است که نمادهای گزاره‌ای فراوان اما برای هر نماد بندهای اندکی باشد. گاهی اما، گزاره‌ای بندهای فراوان دارد. برای نمونه، فرض کنیم مقام‌های مالیاتی می‌خواهند پیگیری کنند چه کسی چه کسی را استخدام می‌کند، با گزارهٔ Employs(x,y). این سطل بسیار بزرگی خواهد بود با شاید میلیون‌ها کارفرما و ده‌ها میلیون کارمند. پاسخ به پرس‌وجویی مانند Employs(x,Richard) با فهرست‌بندی گزاره‌ای مستلزم پویش کل سطل خواهد بود.

برای این پرس‌وجوی خاص، کمک می‌کرد اگر واقعیت‌ها هم با گزاره و هم با آرگومان دوم فهرست‌بندی می‌شدند، شاید با کلید جدول درهم‌سازی ترکیبی. آنگاه می‌توانستیم به‌سادگی کلید را از پرس‌وجو بسازیم و دقیقاً آن واقعیت‌هایی را بازیابی کنیم که با پرس‌وجو یکسان می‌شوند. برای پرس‌وجوهای دیگر، مانند Employs(IBM,y)، لازم بود واقعیت‌ها را با ترکیب گزاره با آرگومان نخست فهرست‌بندی کنیم. پس واقعیت‌ها را می‌توان زیر چندین کلید فهرست ذخیره کرد و آن‌ها را برای پرس‌وجوهای گوناگونی که ممکن است با آن‌ها یکسان شوند فوراً قابل دسترسی کرد.

با جمله‌ای برای ذخیره، می‌توان فهرست‌هایی برای همهٔ پرس‌وجوهای ممکنی که با آن یکسان می‌شوند ساخت. برای واقعیت Employs(IBM,Richard)، پرس‌وجوها عبارت‌اند از:

پرس‌وجو	معنی
Employs(IBM,Richard)	آیا IBM ریچارد را استخدام می‌کند؟
Employs(x,Richard)	چه کسی ریچارد را استخدام می‌کند؟
Employs(IBM,y)	IBM چه کسی را استخدام می‌کند؟
Employs(x,y)	چه کسی چه کسی را استخدام می‌کند؟

این پرس‌وجوها «شبکهٔ زیرشمولی» (Subsumption Lattice) تشکیل می‌دهند، چنان‌که در شکل ۹.۲(الف). شبکه خواص جالبی دارد. فرزند هر گره در شبکه از والدش با یک جانشینی تکی به دست می‌آید؛ و «بالاترین» نوادهٔ مشترک هر دو گره، نتیجهٔ اعمال عمومی‌ترین یکسان‌ساز آن‌هاست. جمله‌ای با ثابت‌های تکراری شبکه‌ای کمی متفاوت دارد، چنان‌که در شکل ۹.۲(ب). هرچند نمادهای تابع در شکل نشان داده نشده‌اند، آن‌ها را نیز می‌توان در ساختار شبکه گنجاند.



شکل ۹.۲ — (الف) شبکهٔ زیرشمولی که پایین‌ترین گره‌اش Employs(IBM,Richard) است. (ب) شبکهٔ زیرشمولی برای جملهٔ Employs(John,John).

برای گزاره‌های با شمار کم آرگومان، ایجاد فهرستی برای هر نقطه از شبکهٔ زیرشمولی دادوستد خوبی است. این کار کمی بیشتر در زمان ذخیره‌سازی لازم دارد، اما زمان بازیابی را سرعت می‌بخشد. اما برای گزاره‌ای با n آرگومان، شبکه $O(2^n)$ گره دارد. اگر نمادهای تابع مجاز باشند، شمار گره‌ها نیز در اندازهٔ عبارت‌های جملهٔ در حال ذخیره نمایی است. این می‌تواند به شمار عظیمی فهرست بینجامد.

باید به‌نحوی فهرست‌ها را به آن‌هایی محدود کنیم که احتمالاً در پرس‌وجوها مکرراً استفاده می‌شوند؛ وگرنه زمان بیشتری در ایجاد فهرست‌ها تلف می‌کنیم تا آنچه با داشتنشان صرفه‌جویی می‌کنیم. می‌توانیم سیاست ثابتی بپذیریم، مانند نگه‌داشتن فهرست‌ها فقط روی کلیدهای متشکل از گزاره به‌علاوهٔ یک آرگومان تکی. یا می‌توانیم سیاست سازگارشونده‌ای یاد بگیریم که فهرست‌هایی برای برآوردن تقاضاهای انواع پرس‌وجوهای پرسیده‌شده ایجاد کند. برای پایگاه‌های دادهٔ تجاری که واقعیت‌ها در آن‌ها به میلیاردها می‌رسد، این مسئله موضوع مطالعهٔ فشرده، توسعهٔ فناوری و بهینه‌سازی مداوم بوده است.

۹.۳ زنجیره‌سازی پیشرو

در بخش 7.5 الگوریتم زنجیره‌سازی پیشرو برای پایگاه‌های دانش بندهای معین گزاره‌ای نشان دادیم. در این‌جا آن ایده را بسط می‌دهیم تا بندهای معین مرتبهٔ اول را پوشش دهد.

البته برخی جمله‌های منطقی هست که نمی‌توان آن‌ها را به‌صورت بند معین بیان کرد و بنابراین این رویکرد نمی‌تواند آن‌ها را مدیریت کند. اما قواعدی به شکل مقدم ⇒ تالی برای پوشش گسترهٔ وسیعی از سامانه‌های جهان واقعیِ جالب کافی‌اند.

۹.۳.۱ بندهای معین مرتبهٔ اول

بندهای معین مرتبهٔ اول فصل‌هایی از لیترال‌ها هستند که دقیقاً یکی از آن‌ها مثبت است. این بدان معناست بند معین یا اتمی است، یا دلالتی است که مقدمش عطفی از لیترال‌های مثبت و تالی‌اش لیترال مثبت تکی است. سورهای وجودی مجاز نیستند و سورهای همگانی ضمنی‌اند: اگر x را در بند معینی می‌بینید، یعنی سور ضمنی $\forall x$ هست. بند معین مرتبهٔ اول نمونه‌ای شبیه این است:

```
King(x) ∧ Greedy(x) ⇒ Evil(x)
```

اما لیترال‌های King(John) و Greedy(y) نیز بند معین شمرده می‌شوند. لیترال‌های مرتبهٔ اول می‌توانند متغیر داشته باشند، پس Greedy(y) به‌صورت «همه حریص‌اند» تفسیر می‌شود (سور همگانی ضمنی است).

بگذارید بندهای معین را در بازنمایی مسئلهٔ زیر به کار ببریم:

قانون می‌گوید جنایت است برای یک آمریکایی که به ملت‌های خصمانه سلاح بفروشد. کشور نونو، دشمن آمریکا، موشک‌هایی دارد و همهٔ موشک‌هایش توسط سرهنگ وست، که آمریکایی است، به آن فروخته شده.

نخست، این واقعیت‌ها را به‌صورت بندهای معین مرتبهٔ اول بازنمایی می‌کنیم:

«... جنایت است برای یک آمریکایی که به ملت‌های خصمانه سلاح بفروشد»:

$$American(x) \land Weapon(y) \land Sells(x,y,z) \land Hostile(z) \Rightarrow Criminal(x) \quad (۹.۳)$$

«نونو ... موشک‌هایی دارد». جملهٔ $\exists x\ Owns(Nono,x) \land Missile(x)$ با مصداق‌سازی وجودی به دو بند معین تبدیل می‌شود و ثابت تازهٔ M1 معرفی می‌شود:

$$Owns(Nono, M1) \quad (۹.۴)$$
$$Missile(M1) \quad (۹.۵)$$

«همهٔ موشک‌هایش توسط سرهنگ وست به آن فروخته شده»:

$$Missile(x) \land Owns(Nono,x) \Rightarrow Sells(West,x,Nono) \quad (۹.۶)$$

همچنین باید بدانیم موشک‌ها سلاح‌اند:

$$Missile(x) \Rightarrow Weapon(x) \quad (۹.۷)$$

و باید بدانیم دشمن آمریکا «خصمانه» شمرده می‌شود:

$$Enemy(x, America) \Rightarrow Hostile(x) \quad (۹.۸)$$

«وست، که آمریکایی است ...»:

$$American(West) \quad (۹.۹)$$

«کشور نونو، دشمن آمریکا ...»:

$$Enemy(Nono, America) \quad (۹.۱۰)$$

این پایگاه دانش اتفاقاً «پایگاه دانش دیتالاگ» (Datalog) است: دیتالاگ زبانی است شامل بندهای معین مرتبهٔ اول بدون نمادهای تابع. دیتالاگ نامش را از آن می‌گیرد که می‌تواند نوع گزاره‌هایی را که معمولاً در پایگاه‌های دادهٔ رابطه‌ای بیان می‌شوند بازنمایی کند. نبود نمادهای تابع استنتاج را بسیار آسان‌تر می‌کند.

۹.۳.۲ الگوریتم سادهٔ زنجیره‌سازی پیشرو

شکل 9.3 الگوریتم استنتاج زنجیره‌سازی پیشروی ساده نشان می‌دهد. با شروع از واقعیت‌های شناخته‌شده، همهٔ قواعدی را که مقدمه‌هایشان برآورده شده فعال می‌کند و نتیجه‌هایشان را به واقعیت‌های شناخته‌شده می‌افزاید. فرایند تکرار می‌شود تا پرس‌وجو پاسخ داده شود (با فرض اینکه فقط یک پاسخ لازم است) یا واقعیت تازه‌ای افزوده نشود. توجه کنید واقعیت «تازه» نیست اگر فقط «تغییر نام» (Renaming) واقعیت شناخته‌شده‌ای باشد — جمله‌ای تغییر نام دیگری است اگر جز نام متغیرها یکسان باشند. برای نمونه، Likes(x, IceCream) و Likes(y, IceCream) تغییر نام یکدیگرند. هر دو همان معنا را می‌دهند: «همه بستنی دوست دارند.»

```
                    function FOL-FC-ASK(KB, α) returns a substitution or false
inputs: KB, a knowledge base, a set of definite clauses in first-order logic
                                        α, a query, an atomic sentence
                                                          while true do
          new ← {}    // set of newly inferred sentences in each iteration
                                                    for each rule in KB do
                        STANDARDIZE-VARIABLES(rule) ← (p1 ∧ ... ∧ pn ⇒ q)
for each θ such that SUBST(θ, p1 ∧ ... ∧ pn) = SUBST(θ, p′1 ∧ ... ∧ p′n)
                                  for some p′1, ..., p′n in KB do
                                                    q′ ← SUBST(θ, q)
                                    if q′ is not already in KB or new then
                                                      add q′ to new
                                                  φ ← UNIFY(q′, α)
                                     if φ is not failure then return φ
                                              if new = {} then return false
                                                          add new to KB
```

شکل ۹.۳ — الگوریتم زنجیره‌سازی پیشروِ از نظر مفهومی سرراست اما ناکارآمد. در هر تکرار، همهٔ جمله‌های اتمی را که می‌توان در یک گام از جمله‌های دلالتی و جمله‌های اتمیِ از قبل در KB استنتاج کرد به KB می‌افزاید. تابع STANDARDIZE-VARIABLES همهٔ متغیرهای آرگومان‌هایش را با متغیرهای تازه‌ای که قبلاً استفاده نشده‌اند جایگزین می‌کند.

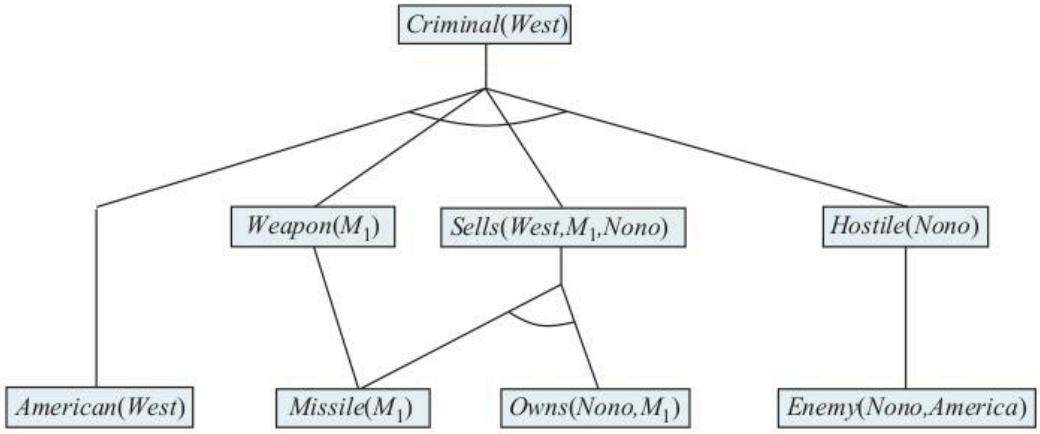
از مثال جنایت برای نشان دادن نحوهٔ کارکرد FOL-FC-ASK استفاده می‌کنیم. جمله‌های دلالتیِ در دسترس برای زنجیره‌سازی عبارت‌اند از (9.3)، (9.6)، (9.7) و (9.8). دو تکرار لازم است:

- در تکرار نخست، قاعدهٔ (9.3) مقدمه‌های برآورده‌نشده دارد.

قاعدهٔ (9.6) با {x/M1} برآورده می‌شود و Sells(West,M1,Nono) افزوده می‌شود. قاعدهٔ (9.7) با {x/M1} برآورده می‌شود و Weapon(M1) افزوده می‌شود. قاعدهٔ (9.8) با {x/Nono} برآورده می‌شود و Hostile(Nono) افزوده می‌شود.

- در تکرار دوم، قاعدهٔ (9.3) با {x/West, y/M1, z/Nono} برآورده می‌شود و استنتاج Criminal(West) افزوده می‌شود.

شکل ۹.۴ درخت برهانی را نشان می‌دهد که تولید می‌شود. توجه کنید در این نقطه استنتاج تازه‌ای ممکن نیست، زیرا هر جمله‌ای که می‌شد با زنجیره‌سازی پیشرو نتیجه گرفت از قبل صریحاً در KB موجود است. چنین پایگاه دانشی «نقطهٔ ثابت» (Fixed Point) فرایند استنتاج نامیده می‌شود. نقاط ثابتِ رسیده‌شده با زنجیره‌سازی پیشرو با بندهای معین مرتبهٔ اول شبیه آن‌ها برای زنجیره‌سازی پیشرو گزاره‌ای (صفحهٔ ۲۳۱) هستند؛ تفاوت اصلی آن است که نقطهٔ ثابت مرتبهٔ اول می‌تواند جمله‌های اتمی همگانی‌سوردار را شامل شود.



شکل ۹.۴ — درخت برهان تولیدشده توسط زنجیره‌سازی پیشرو در مثال جنایت. واقعیت‌های آغازین در سطح پایین، واقعیت‌های استنتاج‌شده در تکرار نخست در سطح میانی، و واقعیت‌های استنتاج‌شده در تکرار دوم در سطح بالا ظاهر می‌شوند. (واقعیت‌ها: Enemy(Nono,America)، (Owns(Nono,M1)، Missile(M1)، American(West) → Hostile(Nono)، Weapon(M1)، Sells(West,M1,Nono) → Criminal(West)

تحلیل FOL-FC-ASK آسان است. نخست، درست است، زیرا هر استنتاج فقط کاربردی از مدوس پوننس تعمیم‌یافته است که درست است. دوم، برای پایگاه‌های دانش بند معین کامل است؛ یعنی به هر پرس‌وجویی که پاسخ‌هایش توسط هر پایگاه دانشی از بندهای معین مستلزم‌اند پاسخ می‌دهد.

برای پایگاه‌های دانش دیتالاگ، که نماد تابع ندارند، برهان کامل بودن نسبتاً آسان است. با شمارش تعداد واقعیت‌هایی که می‌توان افزود آغاز می‌کنیم، که بیشینهٔ تعداد تکرارها را تعیین می‌کند. فرض کنیم k بیشینهٔ تعداد آرگومان (تعداد آرگومان‌ها) هر گزاره، p تعداد گزاره‌ها، و n تعداد نمادهای ثابت باشد. بدیهی است نمی‌توان بیش از $p \cdot n^k$ واقعیت زمینی متمایز باشد، پس پس از این تعداد تکرار الگوریتم باید به نقطهٔ ثابت رسیده باشد. سپس می‌توانیم استدلالی بسیار شبیه برهان کامل بودن زنجیره‌سازی پیشرو گزاره‌ای کنیم. (صفحهٔ ۲۳۱ را ببینید.) جزئیات چگونگی گذار از کامل بودن گزاره‌ای به مرتبهٔ اول برای الگوریتم حل در بخش ۹.۵ داده شده.

برای بندهای معین عمومی با نمادهای تابع، FOL-FC-ASK می‌تواند واقعیت‌های تازهٔ بی‌شماری تولید کند، پس باید دقیق‌تر باشیم. برای موردی که پاسخ پرس‌وجوی q مستلزم است، باید به قضیهٔ اربن (صفحهٔ ۲۸۲) توسل بجوییم تا ثابت کنیم الگوریتم برهانی خواهد یافت. (برای مورد حل، بخش ۹.۵ را ببینید.) اگر پرس‌وجو پاسخی نداشته باشد، الگوریتم در برخی موارد ممکن است پایان نیابد. برای نمونه، اگر پایگاه دانش شامل اصل‌موضوعه‌های پئانو باشد:

```
NatNum(0)
∀n NatNum(n) ⇒ NatNum(S(n))
```

آنگاه زنجیره‌سازی پیشرو (NatNum(S(0))، NatNum(S(S(0)))، NatNum(S(S(S(0)))) و الی آخر را می‌افزاید. این مسئله در حالت کلی اجتناب‌ناپذیر است. مانند منطق مرتبهٔ اول عمومی، لازمه با بندهای معین نیمه‌تصمیم‌پذیر است.

۹.۳.۳ زنجیره‌سازی پیشرو کارآمد

الگوریتم زنجیره‌سازی پیشرو در شکل 9.3 برای سهولت فهم طراحی شده، نه کارایی. سه منبع ناکارآمدی هست. نخست، حلقهٔ درونی الگوریتم می‌کوشد هر قاعده را با هر واقعیت پایگاه دانش مطابقت دهد. دوم، الگوریتم هر قاعده را در هر تکرار دوباره بررسی می‌کند، حتی اگر تعداد بسیار کمی افزوده به پایگاه دانش شده باشد. سوم، الگوریتم می‌تواند واقعیت‌های فراوانی تولید کند که به هدف بی‌ربط‌اند. به هر یک از این مسائل به‌نوبت می‌پردازیم.

مطابقت قواعد با واقعیت‌های شناخته‌شده

مسئلهٔ مطابقت مقدم قاعده با واقعیت‌های پایگاه دانش ممکن است به اندازهٔ کافی ساده به نظر برسد. برای نمونه، فرض کنیم می‌خواهیم قاعدهٔ

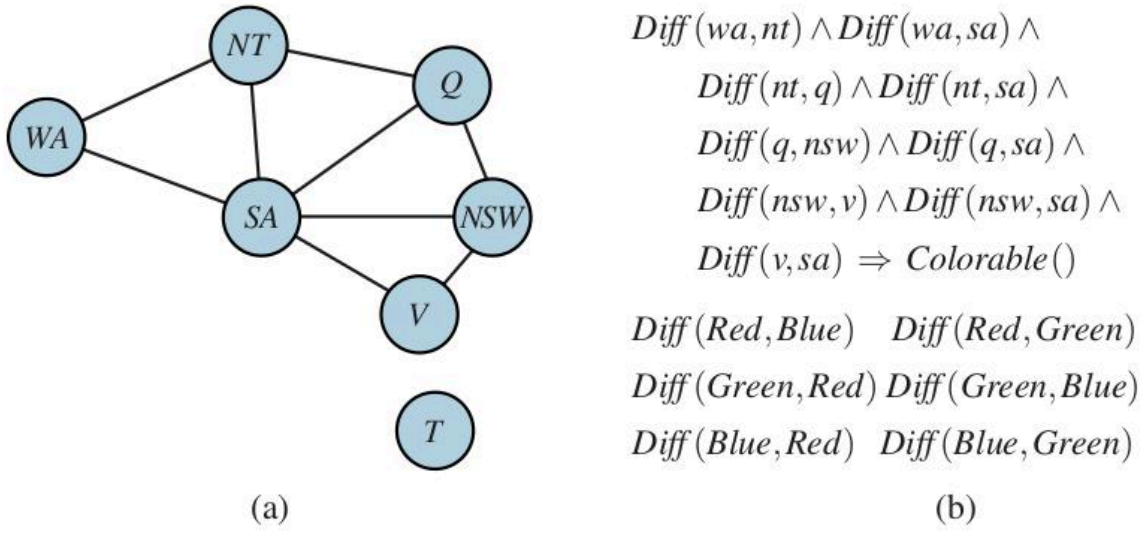
```
Missile(x) ⇒ Weapon(x)
```

را اعمال کنیم. پس باید همهٔ واقعیت‌هایی را بیابیم که با Missile(x) یکسان می‌شوند؛ در پایگاه دانشِ به‌درستی فهرست‌بندی‌شده، این می‌تواند در زمان ثابت برای هر واقعیت انجام شود. حال قاعده‌ای مانند

```
Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)
```

را در نظر بگیرید. باز هم، می‌توانیم همهٔ اشیای متعلق به نونو را در زمان ثابت برای هر شیء بیابیم؛ سپس، برای هر شیء، می‌توانیم بررسی کنیم آیا موشک است. اما اگر پایگاه دانش شامل اشیای فراوانِ متعلق به نونو و موشک‌های بسیار اندک باشد، بهتر بود نخست همهٔ موشک‌ها را بیابیم و سپس بررسی کنیم آیا متعلق به نونو هستند. این «مسئلهٔ ترتیب عطف‌شونده‌ها» (Conjunct Ordering) است: ترتیبی برای حل عطف‌شونده‌های مقدم قاعده بیاب چنان‌که هزینهٔ کل کمینه شود. معلوم می‌شود یافتن ترتیب بهینه NP-سخت است، اما اکتشافی‌های خوبی در دسترس‌اند. برای نمونه، اکتشافی کمترین-مقادیر-باقی‌مانده (MRV) که برای CSPها در فصل ۶ استفاده شد، ترتیب عطف‌شونده‌ها را پیشنهاد می‌کرد اگر موشک‌ها از اشیای متعلق به نونو کمترند، نخست به دنبال موشک‌ها بگرد.

پیوند میان این مطابقت الگو و ارضای قید در واقع بسیار نزدیک است. می‌توانیم هر عطف‌شونده را قیدی بر متغیرهایی که شاملش می‌کند ببینیم — برای نمونه، Missile(x) قید یک‌تایی بر x است. با بسط این ایده، می‌توانیم هر مسئلهٔ ارضای قید (CSP) با دامنهٔ متناهی را به‌صورت یک بند معین تکی همراه با برخی واقعیت‌های زمینی مرتبط بیان کنیم. مسئلهٔ رنگ‌آمیزی نقشه از شکل 6.1 را در نظر بگیرید، که دوباره در شکل ۹.۵(الف) نشان داده شده. صورت‌بندی هم‌ارز به‌صورت بند معین تکی در شکل ۹.۵(ب) داده شده. بدیهی است نتیجهٔ Colorable() فقط اگر CSP راه‌حلی داشته باشد قابل استنتاج است. چون CSPها در حالت کلی مسائل SAT-3 را به‌عنوان حالت خاص شامل می‌شوند، می‌توانیم نتیجه بگیریم مطابقت یک بند معین با مجموعه‌ای از واقعیت‌ها در حالت کلی NP-سخت است.



شکل ۹.۵ – (الف) گراف قید برای رنگ‌آمیزی نقشهٔ استرالیا. (ب) مسئلهٔ رنگ‌آمیزی نقشه به‌صورت بند معین تکی بیان شده. هر ناحیهٔ نقشه به‌صورت متغیری بازنمایی می‌شود که مقدارش می‌تواند یکی از ثابت‌های Red، Green یا Blue باشد (که Diff اعلام شده‌اند).

ممکن است افسرده‌کننده به نظر برسد که زنجیره‌سازی پیشرو مسئلهٔ مطابقت NP-سختی در حلقهٔ درونی‌اش دارد. سه راه برای دلداری خودمان هست:

- می‌توانیم به خود یادآوری کنیم بیشتر قواعد در پایگاه‌های دانش جهان واقعی کوچک و ساده‌اند (مانند قواعد مثال جنایت ما) نه بزرگ و پیچیده (مانند صورت‌بندی CSP در شکل ۹.۵). در دنیای پایگاه‌داده رایج است فرض کنیم هم اندازهٔ قواعد و هم تعداد آرگومان‌های گزاره‌ها با ثابتی کران‌دارند و فقط نگران «پیچیدگی داده» (Data Complexity) باشیم – یعنی پیچیدگی استنتاج به‌عنوان تابعی از تعداد واقعیت‌های زمینی در پایگاه دانش. نشان‌دادن اینکه پیچیدگی دادهٔ زنجیره‌سازی پیشرو چندجمله‌ای است، نه نمایی، آسان است.
- می‌توانیم زیررده‌هایی از قواعد را در نظر بگیریم که مطابقت برایشان کارآمد است. اساساً هر بند دیتالاگی را می‌توان تعریف‌کنندهٔ CSPای دید، پس مطابقت دقیقاً وقتی حل‌پذیر خواهد بود که CSP متناظر حل‌پذیر باشد. فصل ۶ چند خانوادهٔ حل‌پذیر از CSPها را توصیف می‌کند. برای نمونه، اگر گراف قید (گرافی که گره‌هایش متغیرها و پیوندهایش قیدها هستند) درختی را تشکیل دهد، CSP را می‌توان در زمان خطی حل کرد. دقیقاً همان نتیجه برای مطابقت قاعده برقرار است. برای نمونه، اگر استرالیای جنوبی را از نقشهٔ شکل ۹.۵ حذف کنیم، بند حاصل

```
()Diff(wa,nt) ∧ Diff(nt,q) ∧ Diff(q,nsw) ∧ Diff(nsw,v) ⇒ Colorable
```

با CSP کاهش‌یافتهٔ نشان‌داده‌شده در شکل 6.12 در صفحهٔ ۲۰۱ متناظر است. الگوریتم‌های حل CSPهای درخت‌ساختار را می‌توان مستقیماً بر مسئلهٔ مطابقت قاعده اعمال کرد.

- می‌توانیم بکوشیم تلاش‌های مطابقت قاعدهٔ زائد را در الگوریتم زنجیره‌سازی پیشرو حذف کنیم، چنان‌که در ادامه توصیف می‌شود.

زنجیره‌سازی پیشرو افزایشی

وقتی نشان دادیم زنجیره‌سازی پیشرو چگونه روی مثال جنایت کار می‌کند، تقلب کردیم. به‌ویژه، برخی از مطابقت‌های قاعده‌ای را که الگوریتم نشان‌داده‌شده در شکل 9.3 انجام می‌دهد حذف کردیم. برای نمونه، در تکرار دوم، قاعدهٔ

```
Missile(x) ⇒ Weapon(x)
```

با Missile(M1) مطابقت می‌کند (دوباره)، و البته نتیجهٔ Weapon(M1) از قبل معلوم است، پس چیزی رخ نمی‌دهد. چنین مطابقت قاعده‌ایِ زائدی را می‌توان اجتناب کرد اگر مشاهدهٔ زیر را بکنیم: هر واقعیت تازه‌ای که در تکرار t استنتاج می‌شود باید از دست‌کم یک ⚠ واقعیت تازهٔ استنتاج‌شده در تکرار t-1 مشتق شده باشد. این درست است، زیرا هر استنتاجی که به واقعیت تازه‌ای از تکرار t-1 نیاز نداشته باشد، می‌شد از قبل در تکرار t-1 انجام شود.

این مشاهده به‌طور طبیعی به الگوریتم زنجیره‌سازی پیشروی افزایشی می‌انجامد که در آن، در تکرار t، قاعده‌ای را فقط اگر مقدمش شامل عطف‌شوندهٔ pi باشد که با واقعیتِ تازه‌استنتاج‌شده‌ای p'i در تکرار t-1 یکسان شود بررسی می‌کنیم. گام مطابقت قاعده سپس pi را برای مطابقت با p'i ثابت می‌کند، اما به دیگر عطف‌شونده‌های قاعده اجازه می‌دهد با واقعیت‌های هر تکرار قبلی مطابقت کنند. این الگوریتم در هر تکرار دقیقاً همان واقعیت‌های الگوریتم شکل 9.3 را تولید می‌کند، اما بسیار کارآمدتر است.

با فهرست‌بندی مناسب، شناسایی همهٔ قواعدی که توسط واقعیتِ داده‌شده‌ای فعال می‌شوند آسان است و بسیاری از سامانه‌های واقعی در حالت «به‌روزرسانی» عمل می‌کنند که در آن زنجیره‌سازی پیشرو در پاسخ به هر TELL رخ می‌دهد. استنتاج‌ها در میان مجموعهٔ قواعد آبشاری می‌شوند تا نقطهٔ ثابت رسیده شود و سپس فرایند برای واقعیت تازهٔ بعدی دوباره آغاز می‌شود.

معمولاً فقط کسر کوچکی از قواعد پایگاه دانش واقعاً توسط افزودن واقعیتِ داده‌شده‌ای فعال می‌شوند. این بدان معناست کار زائد فراوانی در ساختن مکرر مطابقت‌های جزئی که برخی مقدمه‌های برآورده‌نشده دارند انجام می‌شود. مثال جنایت ما برای نشان دادن مؤثر این خیلی کوچک است، اما توجه کنید مطابقت جزئی در تکرار نخست میان قاعدهٔ

```
American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)
```

و واقعیت American(West) ساخته می‌شود. سپس این مطابقت جزئی دور ریخته و در تکرار دوم دوباره ساخته می‌شود (وقتی قاعده موفق می‌شود). بهتر بود مطابقت‌های جزئی را با رسیدن واقعیت‌های تازه نگه داشت و به‌تدریج کامل کرد، به‌جای دور ریختنشان.

الگوریتم ریت (Rete)[3] نخستین الگوریتمی بود که این مسئله را حل کرد. الگوریتم مجموعهٔ قواعد پایگاه دانش را پیش‌پردازش می‌کند تا شبکهٔ جریان داده‌ای بسازد که در آن هر گره لیتِرالی از مقدم قاعده است. اتصال‌های متغیر در شبکه جریان می‌یابند و وقتی در مطابقت با لیتِرالی شکست می‌خورند فیلتر می‌شوند. اگر دو لیتِرال در قاعده متغیری مشترک داشته باشند — برای نمونه، Sells(x,y,z) ∧ Hostile(z) در مثال جنایت — آنگاه اتصال‌های هر لیتِرال از گرهٔ

برابری فیلتر می‌شوند. اتصال متغیری که به گره‌ای برای لیترِال n-تایی مانند Sells(x,y,z) می‌رسد ممکن است مجبور باشد برای برقرارشدن اتصال‌های متغیرهای دیگر صبر کند تا فرایند بتواند ادامه یابد. در هر نقطهٔ معین، حالت شبکهٔ ریت همهٔ مطابقت‌های جزئی قواعد را در می‌گیرد و از بازمحاسبهٔ فراوان جلوگیری می‌کند.

شبکه‌های ریت، و بهبودهای گوناگونشان، مؤلفهٔ کلیدیِ به اصطلاح «سامانه‌های تولیدی» (Production Systems) بوده‌اند که از نخستین سامانه‌های زنجیره‌سازی پیشرو در استفادهٔ گسترده بودند.[۴] سامانهٔ XCON (در اصل R1 نامیده می‌شد؛ مک‌درموت، ۱۹۸۲) با معماری سامانهٔ تولیدی ساخته شد. XCON شامل چند هزار قاعده برای طراحی پیکربندی‌های مؤلفه‌های رایانه‌ای برای مشتریان شرکت تجهیزات دیجیتال بود. این یکی از نخستین موفقیت‌های تجاری روشن در حوزهٔ نوظهور سامانه‌های خبره بود. بسیاری سامانه‌های مشابه دیگر با همان فناوری زیربنایی ساخته شده‌اند که در زبان عمومی‌منظورهٔ OPS-5 پیاده شده است.

سامانه‌های تولیدی همچنین در «معماری‌های شناختی» (Cognitive Architectures) — یعنی مدل‌های استدلال انسانی — مانند ACT (اندرسون، ۱۹۸۳) و SOAR (لیرد و همکاران، ۱۹۸۷) محبوب‌اند. در چنین سامانه‌هایی، «حافظهٔ کاری» سامانه، حافظهٔ کوتاه‌مدت انسانی را مدل می‌کند و تولیدها بخشی از حافظهٔ بلندمدت‌اند. در هر چرخهٔ عملیات، تولیدها با حافظهٔ کاری واقعیت‌ها مطابقت داده می‌شوند. تولیدی که شرایطش برآورده شده می‌تواند واقعیت‌هایی در حافظهٔ کاری بیفزاید یا حذف کند. در برابر وضعیت معمول در پایگاه‌های داده، سامانه‌های تولیدی اغلب قواعد فراوان و واقعیت‌های نسبتاً اندک دارند. با فناوری مطابقتِ به‌درستی بهینه‌شده، سامانه‌ها می‌توانند با ده‌ها میلیون قاعده در زمان واقعی عمل کنند.

واقعیت‌های بی‌ربط

منبع دیگر ناکارآمدی آن است که زنجیره‌سازی پیشرو همهٔ استنتاج‌های مجاز را بر اساس واقعیت‌های شناخته‌شده انجام می‌دهد، حتی اگر به هدف بی‌ربط باشند. در مثال جنایت ما، قاعده‌ای قادر به نتیجه‌گیری‌های بی‌ربط نبود. اما اگر قواعد فراوانی وجود داشت که عادت‌های غذایی آمریکایی‌ها یا مؤلفه‌ها و قیمت‌های موشک‌ها را توصیف می‌کرد، FOL-FC-ASK نتیجه‌گیری‌های بی‌ربط تولید می‌کرد.

یکی از راه‌های پرهیز از نتیجه‌گیری‌های بی‌ربط، استفاده از زنجیره‌سازی پسرو است، چنان‌که در بخش ۹.۴ توصیف شده. راه دیگر محدودکردن زنجیره‌سازی پیشرو به زیرمجموعهٔ انتخابی از قواعد است، مانند PL-FC-ENTAILS? (صفحهٔ ۲۳۱). رویکرد سوم در حوزهٔ «پایگاه‌های دادهٔ قیاسی» (Deductive Databases) پدیدار شده، که پایگاه‌های دادهٔ بزرگ‌مقیاسی مانند پایگاه‌های دادهٔ رابطه‌ای‌اند، اما از زنجیره‌سازی پیشرو به‌عنوان ابزار استنتاج استاندارد استفاده می‌کنند نه پرس‌وجوهای SQL. ایده آن است که مجموعه‌قواعد را، با استفاده از اطلاعات هدف، بازنویسی کنیم تا فقط اتصال‌های متغیرِ مرتبط — آن‌هایی که متعلق به اصطلاحاً «مجموعهٔ جادویی» (Magic Set) هستند — در استنتاج پیشرو در نظر گرفته شوند. برای نمونه، اگر هدف Criminal(West) باشد، قاعده‌ای که Criminal(x) را نتیجه می‌گیرد بازنویسی می‌شود تا عطف‌شوندهٔ اضافی‌ای شامل شود که مقدار x را قید می‌کند:

```
Magic(x) ∧ American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)
```

واقعیت Magic(West) نیز به KB افزوده می‌شود. بدین ترتیب، حتی اگر پایگاه دانش شامل واقعیت‌هایی دربارهٔ میلیون‌ها آمریکایی باشد، فقط سرهنگ وست در فرایند استنتاج پیشرو در نظر گرفته خواهد شد. فرایند کامل تعریف مجموعه‌های جادویی و بازنویسی پایگاه دانش برای توضیح دادن در این‌جا بیش از حد پیچیده است، اما ایدهٔ پایه انجام نوعی «استنتاج پسروی عمومی» از هدف است تا مشخص شود کدام اتصال‌های متغیر باید قید شوند. رویکرد مجموعه‌های جادویی را پس می‌توان نوعی دورگه میان استنتاج پیشرو و پیش‌پردازش پسرو دانست.

۹.۴ زنجیره‌سازی پسرو

دومین خانوادهٔ بزرگ الگوریتم‌های استنتاج منطقی از زنجیره‌سازی پسرو روی بندهای معین استفاده می‌کند. این الگوریتم‌ها از هدف به عقب کار می‌کنند و از میان قواعد زنجیره می‌کنند تا واقعیت‌های شناخته‌شده‌ای بیابند که برهان را پشتیبانی کنند.

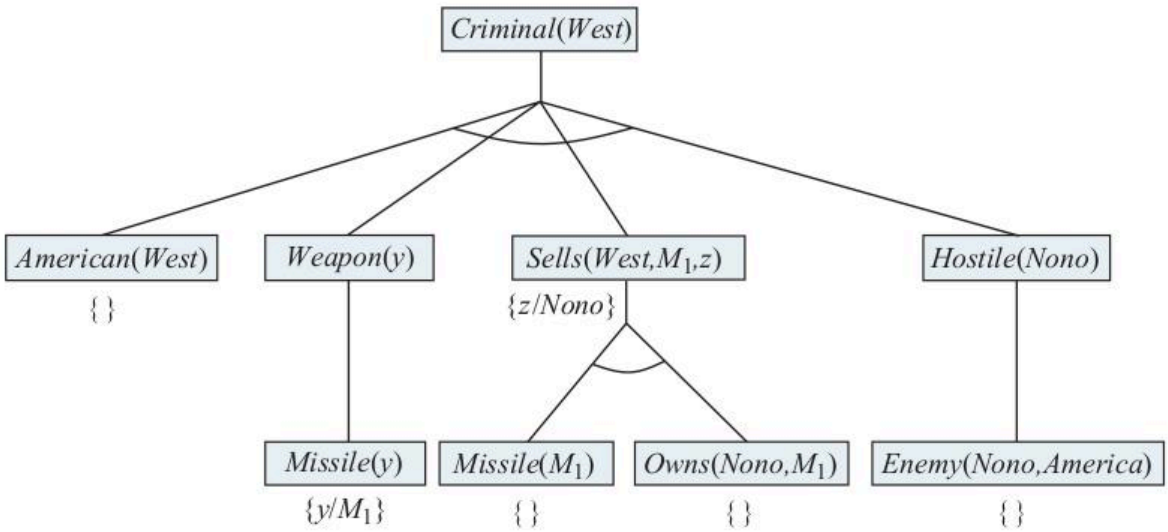
۹.۴.۱ الگوریتم زنجیره‌سازی پسرو

شکل 9.6 الگوریتم زنجیره‌سازی پسرو برای بندهای معین نشان می‌دهد. FOL-BC-ASK(KB, goal) اثبات خواهد شد اگر پایگاه دانش قاعده‌ای به شکل lhs ⇒ goal شامل شود، که lhs (سمت چپ) فهرستی از عطف‌شونده‌هاست. واقعیت اتمی مانند American(West) بندی با lhs خالی در نظر گرفته می‌شود. حال پرس‌وجویی که متغیر دارد ممکن است به چند طریق اثبات شود. برای نمونه، پرس‌وجوی Person(x) می‌تواند با جانشینی {x/John} و نیز با {x/Richard} اثبات شود. پس FOL-BC-ASK را به‌صورت مولد (Generator) پیاده می‌کنیم — تابعی که چند بار بازمی‌گردد و هر بار یک نتیجهٔ ممکن می‌دهد (پیوست ب را ببینید).

زنجیره‌سازی پسرو نوعی جستجوی AND/OR است — بخش OR از آن رو که پرس‌وجوی هدف را می‌توان با هر قاعده‌ای در پایگاه دانش اثبات کرد، و بخش AND از آن رو که همهٔ عطف‌شونده‌های lhs بند باید اثبات شوند. FOL-BC-OR با واکشی همهٔ بندهایی کار می‌کند که ممکن است با هدف یکسان شوند، متغیرهای بند را به متغیرهای کاملاً تازه استانداردسازی می‌کند، و سپس، اگر rhs بند واقعاً با هدف یکسان شود، هر عطف‌شوندهٔ lhs را با FOL-BC-AND اثبات می‌کند. آن تابع با اثبات هر یک از عطف‌شونده‌ها به‌نوبت کار می‌کند و جانشینی انباشته‌شده را هنگام پیش‌رفتن پیگیری می‌کند. شکل ۹.۷ درخت برهان برای مشتق‌کردن Criminal(West) از جمله‌های (9.3) تا (9.10) است.

```
function FOL-BC-ASK(KB, query) returns a generator of substitutions
                                        return FOL-BC-OR(KB, query, {})

                    function FOL-BC-OR(KB, goal, θ) returns a substitution
                      for each rule in FETCH-RULES-FOR-GOAL(KB, goal) do
                            STANDARDIZE-VARIABLES(rule) ← (lhs ⇒ rhs)
          for each θ′ in FOL-BC-AND(KB, lhs, UNIFY(rhs, goal, θ)) do
                                                        ′yield θ

                function FOL-BC-AND(KB, goals, θ) returns a substitution
                                          if θ = failure then return
                                  else if LENGTH(goals) = 0 then yield θ
                                                                  else
                                  first, rest ← FIRST(goals), REST(goals)
                 for each θ′ in FOL-BC-OR(KB, SUBST(θ, first), θ) do
                       for each θ′′ in FOL-BC-AND(KB, rest, θ′) do
                                                       ′′yield θ
```

شکل ۹.۶ – الگوریتم زنجیره‌سازی پسروی ساده برای پایگاه‌های دانش مرتبهٔ اول.



شکل ۹.۷ – درخت برهان ساخته‌شده توسط زنجیره‌سازی پسرو برای اثبات اینکه وست جنایتکار است. درخت باید عمق-اول، چپ به راست خوانده شود. برای اثبات Criminal(West)، باید چهار عطف‌شوندهٔ زیر آن را اثبات کنیم. برخی از آن‌ها در پایگاه دانش‌اند و برخی دیگر به زنجیره‌سازی پسروی بیشتر نیاز دارند. اتصال‌های هر یکسان‌سازی موفق کنار زیرهدف متناظر نشان داده شده. توجه کنید پس از موفقیت یک زیرهدف در عطف، جانشینی‌اش بر زیرهدف‌های بعدی اعمال می‌شود. پس تا وقتی FOL-BC-ASK به آخرین عطف‌شونده می‌رسد، که در اصل Hostile(z) است، z از قبل به Nono اتصال یافته.

زنجیره‌سازی پسرو، چنان‌که نوشته‌ایم، بدیهی است الگوریتم جستجوی عمق-اول است. این بدان معناست نیازهای فضایش در اندازهٔ برهان خطی است. همچنین بدان معناست زنجیره‌سازی پسرو (برخلاف زنجیره‌سازی پیشرو) از مسائل حالت‌های تکراری و ناکامل بودن رنج می‌برد. با وجود این محدودیت‌ها، زنجیره‌سازی پسرو در زبان‌های برنامه‌نویسی منطقی محبوب و مؤثر ثابت شده.

۹.۴.۲ برنامه‌نویسی منطقی

برنامه‌نویسی منطقی فناوری‌ای است که به آرمان اعلانیِ توصیف‌شده در فصل ۷ نزدیک می‌شود: اینکه سامانه‌ها باید با بیان دانش در زبان صوری ساخته شوند و مسائل باید با اجرای فرایندهای استنتاج روی آن دانش حل شوند. این آرمان در معادلهٔ رابرت کوالسکی خلاصه می‌شود:

> *الگوریتم = منطق + کنترل*

پرولوگ (Prolog) پرکاربردترین زبان برنامه‌نویسی منطقی است. عمدتاً به‌عنوان زبان نمونه‌سازی سریع و برای تکلیف‌های دست‌کاری نماد مانند نوشتن کامپایلر (ون روی، ۱۹۹۰) و تجزیهٔ زبان طبیعی (پریرا و وارن، ۱۹۸۰) استفاده می‌شود. سامانه‌های خبرهٔ بسیاری در پرولوگ برای حوزه‌های حقوقی، پزشکی، مالی و دیگر حوزه‌ها نوشته شده.

برنامه‌های پرولوگ مجموعه‌هایی از بندهای معین‌اند که با نمادی تا حدی متفاوت از منطق مرتبهٔ اول استاندارد نوشته می‌شوند. پرولوگ برای متغیرها حروف بزرگ و برای ثابت‌ها حروف کوچک استفاده می‌کند — برعکس قرارداد ما برای منطق. ویرگول‌ها عطف‌شونده‌ها را در بند جدا می‌کنند و بند «برعکس» آنچه به آن عادت داریم نوشته می‌شود؛ به‌جای $A \wedge B \Rightarrow C$ در پرولوگ داریم C :- A, B. این‌جا مثال نمونه‌ای است:

```
criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).
```

در پرولوگ نماد [E|L] فهرستی را نشان می‌دهد که نخستین عنصرش E و بقیه‌اش L است. این‌جا برنامهٔ پرولوگی برای append(X,Y,Z) است که اگر فهرست Z نتیجهٔ به‌هم‌چسباندن فهرست‌های X و Y باشد موفق می‌شود:

```
append([],Y,Y).
append([A|X],Y,[A|Z]) :- append(X,Y,Z).
```

به انگلیسی، می‌توانیم این بندها را چنین بخوانیم: (۱) به‌هم‌چسباندن فهرست خالی و فهرست Y همان فهرست Y را تولید می‌کند؛ و (۲) [A|Z] نتیجهٔ به‌هم‌چسباندن [A|X] و Y است، به شرط آنکه Z نتیجهٔ به‌هم‌چسباندن X و Y باشد. در بیشتر زبان‌های سطح‌بالا می‌توانیم تابع بازگشتی مشابهی بنویسیم که توصیف می‌کند چگونه دو فهرست را به هم بچسبانیم. تعریف پرولوگ اما در واقع قدرتمندتر است، زیرا رابطه‌ای را توصیف می‌کند که میان سه آرگومان برقرار است، نه تابعی که از دو آرگومان محاسبه می‌شود. برای نمونه، می‌توانیم پرس‌وجوی append(X,Y,[1,2,3]) را بپرسیم: کدام دو فهرست را می‌توان به هم چسباند تا [1,2,3] داده شود؟ پرولوگ راه‌حل‌ها را به ما برمی‌گرداند:

```
X=[] Y=[1,2,3];
X=[1] Y=[2,3];
X=[1,2] Y=[3];
X=[1,2,3] Y=[];
```

اجرای برنامه‌های پرولوگ از طریق زنجیره‌سازی پسروی عمق-اول انجام می‌شود، که در آن بندها به ترتیبی که در پایگاه دانش نوشته شده‌اند امتحان می‌شوند. طراحی پرولوگ مصالحه‌ای میان اعلانی‌بودن و کارایی اجراست. برخی جنبه‌های پرولوگ خارج از استنتاج منطقی استاندارد می‌افتند:

- پرولوگ از معناشناسی پایگاه‌دادهٔ بخش 8.2.8 استفاده می‌کند نه معناشناسی مرتبهٔ اول، و این در برخوردش با برابری و نقیض آشکار است (بخش ۹.۴.۴ را ببینید).
- مجموعه‌ای از توابع ساخته‌شده برای حساب وجود دارد. لیترال‌هایی که از این نمادهای تابع استفاده می‌کنند با اجرای کد «اثبات» می‌شوند نه با انجام استنتاج بیشتر. برای نمونه، هدف «X is 4+3» با X اتصال‌یافته به ۷ موفق می‌شود. از سوی دیگر، هدف «is X+Y 5» شکست می‌خورد، زیرا توابع ساخته‌شده حل معادلهٔ دلبخواهی انجام نمی‌دهند.
- گزاره‌های ساخته‌شده‌ای هست که هنگام اجرا عوارض جانبی دارند. این‌ها شامل گزاره‌های ورودی–خروجی و گزاره‌های assert/retract برای اصلاح پایگاه دانش‌اند. چنین گزاره‌هایی در منطق همتایی ندارند و می‌توانند نتایج گیج‌کننده‌ای تولید کنند — برای نمونه، اگر واقعیت‌هایی در شاخه‌ای از درخت برهان ادعا شوند که سرانجام شکست می‌خورد.
- بررسی رخداد از الگوریتم یکسان‌سازی پرولوگ حذف شده. این بدان معناست برخی استنتاج‌های نادرست می‌تواند انجام شود؛ این‌ها تقریباً هرگز در عمل مسئله نیستند.
- پرولوگ از جستجوی زنجیره‌سازی پسروی عمق-اول بدون بررسی‌های بازگشت بی‌نهایت استفاده می‌کند. این زبان برنامه‌نویسی کاربردی‌ای می‌سازد که وقتی درست استفاده شود بسیار سریع است، اما بدان معناست برخی برنامه‌هایی که منطق معتبر به نظر می‌رسند پایان نخواهند یافت.

۹.۴.۳ استنتاج زائد و حلقه‌های بی‌نهایت

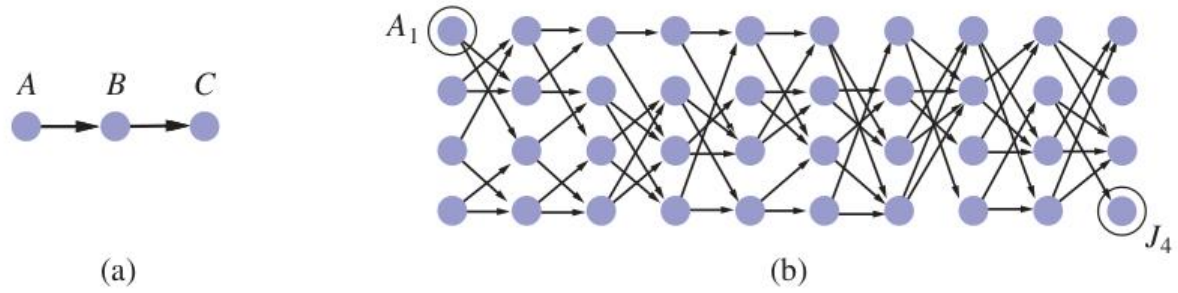
اکنون به پاشنهٔ آشیل پرولوگ می‌پردازیم: ناهماهنگی میان جستجوی عمق-اول و درخت‌های جستجویی که حالت‌های تکراری و مسیرهای بی‌نهایت شامل می‌شوند. برنامهٔ منطقی زیر را در نظر بگیرید که تصمیم می‌گیرد آیا مسیری بین دو نقطه روی گراف جهت‌دار وجود دارد:

```
.path(X,Z) :- link(X,Z)
.path(X,Z) :- path(X,Y), link(Y,Z)
```

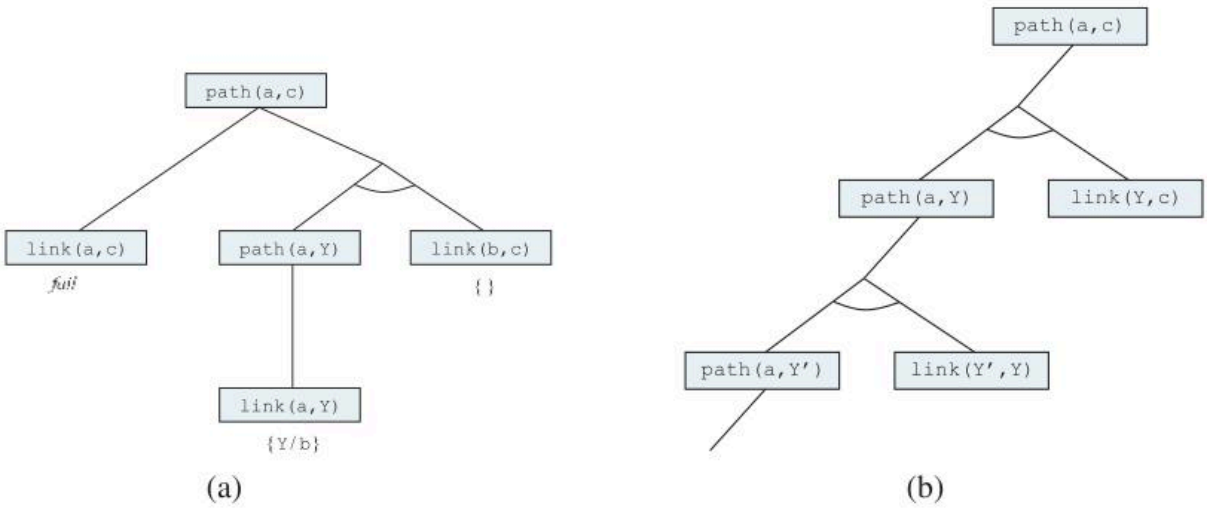
گراف سادهٔ سه‌گره‌ای، که با واقعیت‌های link(a,b) و link(b,c) توصیف شده، در شکل ۹.۸(الف) نشان داده شده. با این برنامه، پرس‌وجوی path(a,c) درخت برهانِ نشان‌داده‌شده در شکل ۹.۹(الف) را تولید می‌کند. از سوی دیگر، اگر دو بند را به این ترتیب بگذاریم:

```
.path(X,Z) :- path(X,Y), link(Y,Z)
.path(X,Z) :- link(X,Z)
```

آنگاه پرولوگ مسیر بی‌نهایتِ نشان‌داده‌شده در شکل ۹.۹(ب) را دنبال می‌کند. پرولوگ پس به‌عنوان اثبات‌کنندهٔ قضیه برای بندهای معین ناقص است — حتی برای برنامه‌های دیتالاگ، چنان‌که این مثال نشان می‌دهد — زیرا برای برخی پایگاه‌های دانش، در اثبات جمله‌های مستلزم شکست می‌خورد. توجه کنید زنجیره‌سازی پیشرو از این مسئله رنج نمی‌برد: پس از استنتاج path(b,c)، path(a,b) و path(a,c)، زنجیره‌سازی پیشرو می‌ایستد.



شکل ۹.۸ — (الف) یافتن مسیری از A به C می‌تواند پرولوگ را به حلقهٔ بی‌نهایت بکشاند. (ب) گرافی که در آن هر گره به دو جانشین تصادفی در لایهٔ بعد متصل است. یافتن مسیری از A1 به J4 به ۸۷۷ استنتاج نیاز دارد.



شکل ۹.۹ — (الف) اثبات وجود مسیر از A به C. (ب) درخت برهان بی‌نهایت تولیدشده وقتی بندها در ترتیب «اشتباه» باشند.

زنجیره‌سازی پسروی عمق-اول همچنین با محاسبات زائد مسئله دارد. برای نمونه، هنگام یافتن مسیری از A1 به J4 در شکل ۹.۸(ب)، پرولوگ ۸۷۷ استنتاج انجام می‌دهد که بیشتر آن‌ها شامل یافتن همهٔ مسیرهای ممکن به گره‌هایی است که هدف از آن‌ها دست‌نیافتنی است. این شبیه مسئلهٔ حالت‌های تکرارِی بحث‌شده در فصل ۳ است. مقدار کل استنتاج می‌تواند در تعداد واقعیت‌های زمینیِ تولیدشده نمایی باشد. اگر به‌جایش زنجیره‌سازی پیشرو اعمال کنیم، حداکثر n^2 واقعیت path(X,Y) می‌تواند تولید شود که n گره را متصل کند. برای مسئلهٔ شکل ۹.۸(ب)، فقط ۶۲ استنتاج لازم است.

زنجیره‌سازی پیشرو روی مسائل جستجوی گراف مثالی از «برنامه‌نویسی پویا» (Dynamic Programming) است، که در آن راه‌حل‌های زیرمسئله‌ها به‌صورت افزایشی از راه‌حل‌های زیرمسئله‌های کوچک‌تر ساخته و برای پرهیز از بازمحاسبه کش می‌شوند. می‌توانیم اثر مشابهی در سامانهٔ زنجیره‌سازی پسرو به دست آوریم، به‌جز آنکه در این‌جا هدف‌های بزرگ

را به هدف‌های کوچک‌تر می‌شکنیم، نه آنکه آن‌ها را بسازیم.

در هر دو صورت، ذخیرهٔ نتایج میانی برای پرهیز از تکرار کلیدی است. این رویکردی است که «سامانه‌های برنامه‌نویسی منطقی جدول‌بندی‌شده» (Tabled Logic Programming) می‌گیرند، که از سازوکارهای کارآمد ذخیره‌سازی و بازیابی استفاده می‌کنند. برنامه‌نویسی منطقی جدول‌بندی‌شده هدف‌محوری زنجیره‌سازی پسرو را با کارایی برنامه‌نویسی پویای زنجیره‌سازی پیشرو ترکیب می‌کند. همچنین برای پایگاه‌های دانش دیتالاگ کامل است، یعنی برنامه‌نویس کمتر باید نگران حلقه‌های بی‌نهایت باشد. (هنوز امکان گرفتن حلقهٔ بی‌نهایت با گزاره‌هایی مانند father(X,Y) که به تعداد بالقوه بی‌کران اشیا ارجاع می‌کنند هست.)

۹.۴.۴ معناشناسی پایگاه‌دادهٔ پرولوگ

پرولوگ از معناشناسی پایگاه‌داده استفاده می‌کند، چنان‌که در بخش 8.2.8 بحث شد. فرض نام‌های یکتا می‌گوید هر ثابت پرولوگ و هر عبارت زمینی به شیء متمایزی اشاره می‌کند، و فرض جهان بسته می‌گوید تنها جمله‌های صادق آن‌هایی هستند که توسط پایگاه دانش مستلزم‌اند. هیچ راهی برای ادعای کاذب‌بودن جمله‌ای در پرولوگ نیست. این پرولوگ را از منطق مرتبهٔ اول کم‌بیان‌تر می‌کند، اما بخشی از آن چیزی است که پرولوگ را کارآمدتر و موجزتر می‌کند. ادعاهای زیر را دربارهٔ برخی درس‌های ارائه‌شده در نظر بگیرید:

$$\text{Course(CS,101), Course(CS,102), Course(CS,106), Course(EE,101)} \quad (۹.۱۱)$$

تحت فرض نام‌های یکتا، CS و EE متفاوت‌اند (چنان‌که 101، 102 و 106)، پس این بدان معناست چهار درس متمایز هست. تحت فرض جهان بسته درس دیگری نیست، پس دقیقاً چهار درس هست. اما اگر این‌ها ادعاهایی در FOL بودند نه در معناشناسی پایگاه‌داده، تنها چیزی که می‌توانستیم بگوییم آن بود که بین یک و بی‌نهایت درس هست. این به‌سبب آن است که ادعاها (در FOL) امکان ارائه‌شدن درس‌های نام‌برده‌نشدهٔ دیگر را نفی نمی‌کنند، و نمی‌گویند درس‌های نام‌برده از یکدیگر متمایزند. اگر می‌خواستیم معادلهٔ (9.11) را به FOL ترجمه کنیم، جملهٔ زیر را می‌گرفتیم:

$$\text{Course}(d,n) \Leftrightarrow (d = \text{CS} \wedge n = 101) \vee (d = \text{CS} \wedge n = 102) \vee (d = \text{CS} \wedge n = 106) \vee (d = \text{EE} \wedge n = 101) \quad (۹.۱۲)$$

این «تکمیل» (Completion) معادلهٔ (9.11) نامیده می‌شود. در FOL ایدهٔ «حداکثر چهار درس» را بیان می‌کند. برای بیان در FOL ایدهٔ «حداقل چهار درس»، باید تکمیل گزارهٔ برابری را بنویسیم:

$$x = y \Leftrightarrow (x = \text{CS} \wedge y = \text{CS}) \vee (x = \text{EE} \wedge y = \text{EE}) \vee (x = 101 \wedge y = 101) \vee (x = 102 \wedge y = 102) \vee (x = 106 \wedge y = 106)$$

تکمیل برای فهم معناشناسی پایگاه‌داده مفید است، اما برای مقاصد عملی، اگر مسئله‌تان را بتوان با معناشناسی پایگاه‌داده توصیف کرد، کارآمدتر است با پرولوگ یا سامانهٔ معناشناسی پایگاه‌دادهٔ دیگر استدلال کنید، تا ترجمه به FOL و استدلال با اثبات‌کنندهٔ قضیهٔ کامل FOL.

۹.۴.۵ برنامه‌نویسی منطق قیدی

در بحث‌مان از زنجیره‌سازی پیشرو (بخش ۹.۳)، نشان دادیم چگونه مسئله‌های ارضای قیود (CSPها) را می‌توان به‌صورت بندهای معین رمزگذاری کرد. پرولوگ استاندارد چنین مسائلی را دقیقاً به همان شیوهٔ الگوریتم عقبگردِ داده‌شده در شکل 6.5 حل می‌کند.

چون عقبگرد دامنه‌های متغیرها را برمی‌شمارد، فقط برای CSPهای دامنه‌متناهی کار می‌کند. به زبان پرولوگ، برای هر هدف با متغیرهای بی‌اتصال باید تعداد متناهی راه‌حل باشد. (برای نمونه، مسئلهٔ رنگ‌آمیزی نقشه‌ای که در آن هر متغیر می‌تواند یکی از چهار رنگ مختلف بگیرد.) CSPهای دامنه‌نامتناهی — برای نمونه، با متغیرهای مقدار صحیح یا حقیقی — به الگوریتم‌های کاملاً متفاوتی نیاز دارند، مانند انتشار کران یا برنامه‌ریزی خطی.

مثال زیر را در نظر بگیرید. triangle(X,Y,Z) را به‌صورت گزاره‌ای تعریف می‌کنیم که اگر سه آرگومان اعدادی باشند که نابرابری مثلث را برآورده می‌کنند برقرار است:

```
.triangle(X,Y,Z) :- X>0, Y>0, Z>0, X+Y>Z, Y+Z>X, X+Z>Y
```

اگر از پرولوگ پرس‌وجوی triangle(3,4,5) را بپرسیم، موفق می‌شود. از سوی دیگر، اگر triangle(3,4,Z) را بپرسیم، هیچ راه‌حلی یافت نمی‌شود، زیرا زیرهدف Z>0 توسط پرولوگ قابل مدیریت نیست؛ نمی‌توانیم مقدار بی‌اتصال را با ۰ مقایسه کنیم.

«برنامه‌نویسی منطق قیدی» (Constraint Logic Programming، یا CLP) به متغیرها اجازه می‌دهد «قید» شوند نه اتصال. راه‌حل CLP مشخص‌ترین مجموعه‌قیدهای روی متغیرهای پرس‌وجوست که از پایگاه دانش قابل اشتقاق است. برای نمونه، راه‌حل پرس‌وجوی triangle(3,4,Z) قید 7 > Z > 1 است. برنامه‌های منطقی استاندارد فقط حالت خاصی از CLP هستند که در آن قیدهای راه‌حل باید قیدهای برابری باشند — یعنی اتصال‌ها.

سامانه‌های CLP الگوریتم‌های حل قید گوناگونی را برای قیدهای مجاز در زبان در خود جای می‌دهند. برای نمونه، سامانه‌ای که نابرابری‌های خطی روی متغیرهای مقدار حقیقی را اجازه می‌دهد ممکن است الگوریتم برنامه‌ریزی خطی برای حل آن قیدها شامل شود. سامانه‌های CLP همچنین رویکرد بسیار انعطاف‌پذیرتری به حل پرس‌وجوهای استاندارد برنامه‌نویسی منطقی اتخاذ می‌کنند. برای نمونه، به‌جای عقبگرد عمق-اول چپ-به-راست، ممکن است از هر یک از الگوریتم‌های کارآمدتر بحث‌شده در فصل ۶ استفاده کنند، از جمله ترتیب عطف‌شوندهٔ اکتشافی، پرش به عقب، شرطی‌سازی مجموعه‌برش و الی آخر. سامانه‌های CLP پس عناصر الگوریتم‌های ارضای قید، برنامه‌نویسی منطقی و پایگاه‌های دادهٔ قیاسی را ترکیب می‌کنند.

چند سامانه که به برنامه‌نویس کنترل بیشتری بر ترتیب جستجوی استنتاج می‌دهند تعریف شده‌اند. زبان MRS (جنزِرت و اسمیت، ۱۹۸۱؛ راسل، ۱۹۸۵) به برنامه‌نویس اجازه می‌دهد «فراقاعده»ها (Metarules) بنویسد تا تعیین کند کدام عطف‌شونده‌ها نخست امتحان شوند. کاربر می‌توانست قاعده‌ای بنویسد که می‌گوید هدف با کمترین متغیر باید نخست امتحان شود یا می‌توانست قواعد حوزه‌ویژه برای گزاره‌های خاص بنویسد.

۹.۵ حل

آخرینِ سه خانوادهٔ سامانه‌های منطقی ما، و تنها یکی که برای هر پایگاه دانشی کار می‌کند، نه فقط بندهای معین، «حل» (Resolution) است. در صفحهٔ ۲۲۳ دیدیم حل گزاره‌ای رویهٔ استنتاج کاملی برای منطق گزاره‌ای است؛ در این بخش، آن را به منطق مرتبهٔ اول بسط می‌دهیم.

۹.۵.۱ شکل عادی عطفی برای منطق مرتبهٔ اول

گام نخست تبدیل جمله‌ها به شکل عادی عطفی (CNF) است — یعنی عطفی از بندها، که هر بند فصلی از لیترال‌هاست.[5] در CNF، لیترال‌ها می‌توانند متغیر داشته باشند، که همگانی‌سوردار فرض می‌شوند. برای نمونه، جملهٔ

```
∀x,y,z American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)
```

در CNF می‌شود:

```
American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x)¬
```

نکتهٔ کلیدی آن است که هر جملهٔ منطق مرتبهٔ اول را می‌توان به جملهٔ CNF هم‌ارز استنتاجی تبدیل کرد.

رویهٔ تبدیل به CNF شبیه حالت گزاره‌ای است که در صفحهٔ ۲۲۶ دیدیم. تفاوت اصلی از نیاز به حذف سورهای وجودی ناشی می‌شود. رویه را با ترجمهٔ جملهٔ «هرکس که همهٔ جانوران را دوست دارد، توسط کسی دوست داشته می‌شود»، یا

```
∀x [∀y Animal(y) ⇒ Loves(x,y)] ⇒ [∃y Loves(y,x)]
```

نشان می‌دهیم. گام‌ها چنین‌اند:

- **دلالت‌ها را حذف کن:** P ⇒ Q را با ¬P ∨ Q جایگزین کن. برای جملهٔ نمونهٔ ما، این باید دو بار انجام شود:

```
∀x ¬[∀y Animal(y) ⇒ Loves(x,y)] ∨ [∃y Loves(y,x)]
∀x ¬[∀y ¬Animal(y) ∨ Loves(x,y)] ∨ [∃y Loves(y,x)]
```

- **¬ را به درون ببر:** افزون بر قواعد معمول برای عوامل ربط نقیض‌شده، به قواعدی برای سورهای نقیض‌شده نیاز داریم:

```
¬∀x p  تبدیل می‌شود به  ∃x ¬p
¬∃x p  تبدیل می‌شود به  ∀x ¬p
```

جملهٔ ما از تبدیل‌های زیر می‌گذرد:

```
∀x [∃y ¬(¬Animal(y) ∨ Loves(x,y))] ∨ [∃y Loves(y,x)]
∀x [∃y ¬¬Animal(y) ∧ ¬Loves(x,y)] ∨ [∃y Loves(y,x)]
∀x [∃y Animal(y) ∧ ¬Loves(x,y)] ∨ [∃y Loves(y,x)]
```

توجه کنید چگونه سور همگانی (∀y) در مقدم دلالت وجودی شده. جمله اکنون چنین خوانده می‌شود: «یا جانوری هست که x دوستش ندارد، یا (اگر چنین نیست) کسی هست که x را دوست دارد.» بدیهی است معنای جملهٔ اصلی حفظ شده.

- **متغیرها را استانداردسازی کن:** برای جمله‌هایی مانند (∃x P(x)) ∨ (∃x Q(x)) که از همان نام متغیر دو بار استفاده می‌کنند، نام یکی از متغیرها را تغییر بده. این از سردرگمی بعداً هنگام انداختن سورها جلوگیری می‌کند:

```
∀x [∃y Animal(y) ∧ ¬Loves(x,y)] ∨ [∃z Loves(z,x)]
```

- **اسکولم‌سازی (Skolemization):** اسکولم‌سازی فرایند حذف سورهای وجودی است. در حالت ساده، دقیقاً مانند قاعدهٔ مصداق‌سازی وجودی بخش ۹.۱ است: ∃x P(x) را به P(A) ترجمه کن، که A نماد ثابت تازه‌ای است. اما نمی‌توانیم مصداق‌سازی وجودی را بر جملهٔ بالای خود اعمال کنیم، زیرا با الگوی ∃α ∨ مطابقت ندارد؛ فقط بخش‌هایی از جمله با الگو مطابقت دارند. اگر کورکورانه قاعده را بر دو بخش مطابق اعمال کنیم می‌گیریم:

```
∀x [Animal(A) ∧ ¬Loves(x,A)] ∨ Loves(B,x)
```

که معنایی کاملاً اشتباه دارد: می‌گوید هرکس یا در دوست‌داشتن جانور خاصی A ناکام است یا توسط موجود خاصی B دوست داشته می‌شود. در واقع، جملهٔ اصلی به هر شخص اجازه می‌دهد در دوست‌داشتن جانوری متفاوت ناکام باشد یا توسط شخص متفاوتی دوست داشته شود. پس می‌خواهیم موجودیت‌های اسکولم به x وابسته باشند:

```
∀x [Animal(F(x)) ∧ ¬Loves(x,F(x))] ∨ Loves(G(x),x)
```

در این‌جا F و G «توابع اسکولم» (Skolem Functions) هستند. قاعدهٔ کلی آن است که آرگومان‌های تابع اسکولم همهٔ متغیرهای همگانی‌سورداری هستند که سور وجودی در حوزهٔ آن‌ها ظاهر می‌شود. مانند مصداق‌سازی وجودی، جملهٔ اسکولم‌شده دقیقاً وقتی ارضاپذیر است که جملهٔ اصلی ارضاپذیر باشد.

- **سورهای همگانی را بینداز:** در این نقطه، همهٔ متغیرهای باقی‌مانده باید همگانی‌سوردار باشند. پس اگر سور را بیندازیم اطلاعاتی از دست نمی‌دهیم:

```
Loves(G(x),x) ∨ [Animal(F(x)) ∧ ¬Loves(x,F(x))]
```

- **∨ را بر ∧ توزیع کن:**

```
[Loves(x,F(x)) ∨ Loves(G(x),x)¬] ∧ [Animal(F(x)) ∨ Loves(G(x),x)]
```

این گام ممکن است همچنین به صاف‌کردن عطف‌ها و فصل‌های تودرتو نیاز داشته باشد.

جمله اکنون در CNF است و از دو بند تشکیل شده. خواندنش بسیار دشوارتر از جملهٔ اصلی با دلالت‌هاست. (ممکن است کمک کند توضیح دهیم تابع اسکولم F(x) به جانوری اشاره دارد که احتمالاً توسط x دوست داشته نمی‌شود، در حالی که G(x) به کسی اشاره دارد که ممکن است x را دوست داشته باشد.) خوشبختانه، انسان‌ها به‌ندرت لازم است به جمله‌های CNF نگاه کنند — فرایند ترجمه به‌راحتی خودکار می‌شود.

۹.۵.۲ قاعدهٔ استنتاج حل

قاعدهٔ حل برای بندهای مرتبهٔ اول صرفاً نسخهٔ بالابرده‌شدهٔ قاعدهٔ حل گزاره‌ایِ داده‌شده در صفحهٔ ۲۲۶ است. دو بند، که فرض می‌شود استانداردسازی جداسازی شده‌اند تا متغیر مشترکی نداشته باشند، اگر لیتِرال‌های مکمل داشته باشند قابل حل‌اند. لیتِرال‌های گزاره‌ای مکمل‌اند اگر یکی نقیض دیگری باشد؛ لیتِرال‌های مرتبهٔ اول مکمل‌اند اگر یکی با نقیض دیگری یکسان شود. پس داریم:

$$\frac{\ell_1 \vee \ldots \vee \ell_k, \quad m_1 \vee \ldots \vee m_n}{\text{SUBST}(\theta, \ell_1 \vee \ldots \vee \ell_{i-1} \vee \ell_{i+1} \vee \ldots \vee \ell_k \vee m_1 \vee \ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots \vee m_n)}$$

که $\theta = \text{UNIFY}(\ell_i, \neg m_j)$. برای نمونه، می‌توانیم دو بند

```
[Animal(F(x)) ∨ Loves(G(x),x)]
```

و

```
[Loves(u,v) ∨ ¬Kills(u,v)¬]
```

را با حذف لیتِرال‌های مکمل Loves(G(x),x) و ¬Loves(u,v)، با یکسان‌ساز {θ={u/G(x), v/x}، حل کنیم تا بند حاصل‌حل تولید شود:

```
[Animal(F(x)) ∨ ¬Kills(G(x),x)]
```

این قاعده «قاعدهٔ حل دودویی» (Binary Resolution) نامیده می‌شود، زیرا دقیقاً دو لیتِرال را حل می‌کند. قاعدهٔ حل دودویی به تنهایی رویهٔ استنتاج کاملی به دست نمی‌دهد. قاعدهٔ کامل حل، زیرمجموعه‌هایی از لیتِرال‌ها در هر بند را که یکسان‌شدنی‌اند حل می‌کند. رویکرد جایگزین بسط «عامل‌گیری» — حذف لیتِرال‌های زائد — به حالت مرتبهٔ اول

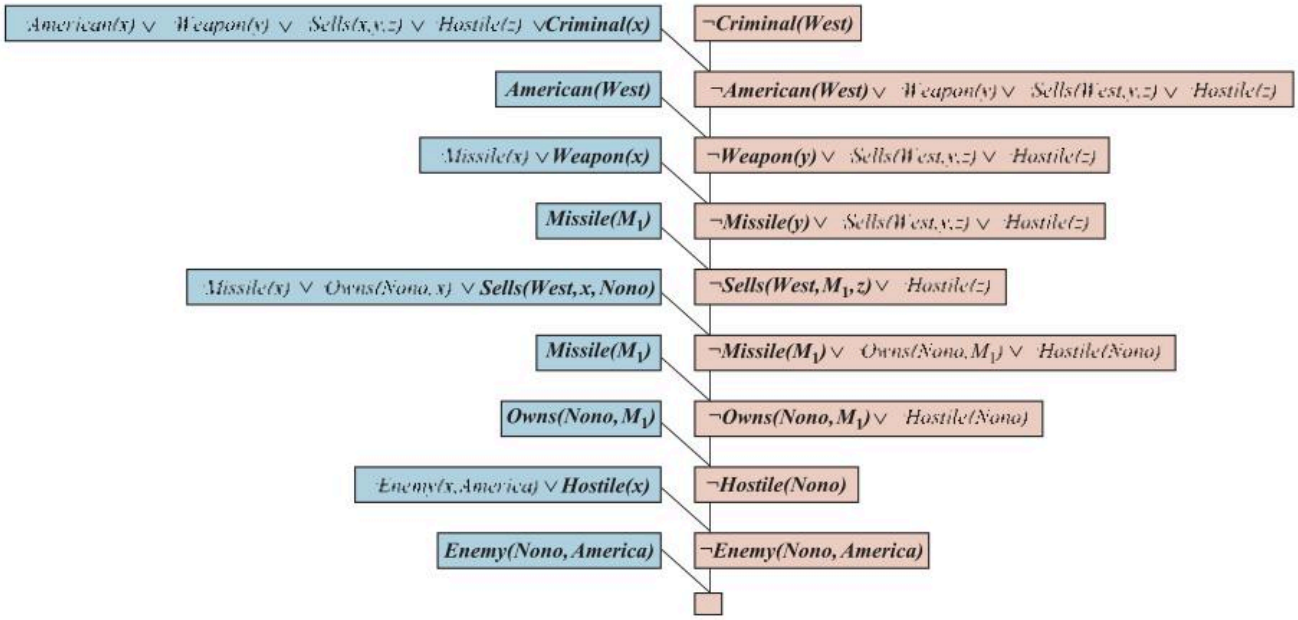
است. عامل‌گیری گزاره‌ای دو لیترال را اگر یکسان باشند به یکی کاهش می‌دهد؛ عامل‌گیری مرتبهٔ اول دو لیترال را اگر یکسان‌شدنی باشند به یکی کاهش می‌دهد. یکسان‌ساز باید بر کل بند اعمال شود. ترکیب حل دودویی و عامل‌گیری کامل است.

۹.۵.۳ برهان‌های نمونه

حل ثابت می‌کند $KB \models \alpha$ با اثبات ارضاپذیرنبودن $KB \wedge \neg\alpha$ — یعنی با مشتق‌کردن بند خالی. رویکرد الگوریتمی با حالت گزاره‌ایِ توصیف‌شده در شکل 7.13 یکسان است، پس لازم نیست این‌جا تکرارش کنیم. در عوض، دو برهان نمونه می‌دهیم. نخستین، مثال جنایت از بخش ۹.۳ است. جمله‌های در CNF عبارت‌اند از:

```
American(x) ∨ ¬Weapon(y) ∨ ¬Sells(x,y,z) ∨ ¬Hostile(z) ∨ Criminal(x)¬
Missile(x) ∨ ¬Owns(Nono,x) ∨ Sells(West,x,Nono)¬
Enemy(x,America) ∨ Hostile(x)¬
Missile(x) ∨ Weapon(x)¬
Owns(Nono,M1)
Missile(M1)
American(West)
Enemy(Nono,America)
```

همچنین هدف نقیض‌شدهٔ ¬Criminal(West) را شامل می‌شویم. برهان حل در شکل ۹.۱۰ نشان داده شده. به ساختار توجه کنید: «ستون فقرات» تکی که با بند هدف آغاز می‌شود و با بندهایی از پایگاه دانش حل می‌شود تا بند خالی تولید شود. این ویژگی حل روی پایگاه‌های دانش بند هورن است. در واقع، بندهای طول ستون فقرات اصلی دقیقاً با مقادیر پیاپی متغیر goals در الگوریتم زنجیره‌سازی پسروی شکل 9.6 متناظرند. این به‌سبب آن است که همیشه انتخاب می‌کنیم با بندی حل کنیم که لیترال مثبتش با چپ‌ترین لیترال بند «کنونی» روی ستون فقرات یکسان می‌شود؛ این دقیقاً همان چیزی است که در زنجیره‌سازی پسرو رخ می‌دهد. پس زنجیره‌سازی پسرو فقط حالت خاصی از حل با راهبرد کنترل خاصی برای تصمیم‌گیری اینکه کدام حل بعدی انجام شود است.



شکل ۹.۱۰ – برهان حل که وست جنایتکار است. در هر گام حل، لیترال‌های یکسان‌شده ضخیم‌اند و بند با لیترال مثبت آبی سایه‌خورده.

مثال دوم ما از اسکولم‌سازی استفاده می‌کند و بندهایی را شامل می‌شود که بند معین نیستند. این به ساختار برهان تا حدی پیچیده‌تری می‌انجامد. به انگلیسی:

هرکس که همهٔ جانوران را دوست دارد، توسط کسی دوست داشته می‌شود. هرکس که جانوری را بکشد، توسط هیچ‌کس دوست داشته نمی‌شود. جک همهٔ جانوران را دوست دارد. یا جک یا کوریوسیتی گربه را کشت، که تونا نام دارد. آیا کوریوسیتی گربه را کشت؟

نخست، جمله‌های اصلی، برخی دانش زمینه‌ای و هدف نقیض‌شدهٔ G را در منطق مرتبهٔ اول بیان می‌کنیم:

```
A. ∀x [∀y Animal(y) ⇒ Loves(x,y)] ⇒ [∃y Loves(y,x)]
B. ∀x [∃z Animal(z) ∧ Kills(x,z)] ⇒ [∀y ¬Loves(y,x)]
C. ∀x Animal(x) ⇒ Loves(Jack,x)
D. Kills(Jack,Tuna) ∨ Kills(Curiosity,Tuna)
E. Cat(Tuna)
F. ∀x Cat(x) ⇒ Animal(x)
G. ¬Kills(Curiosity,Tuna)¬
```

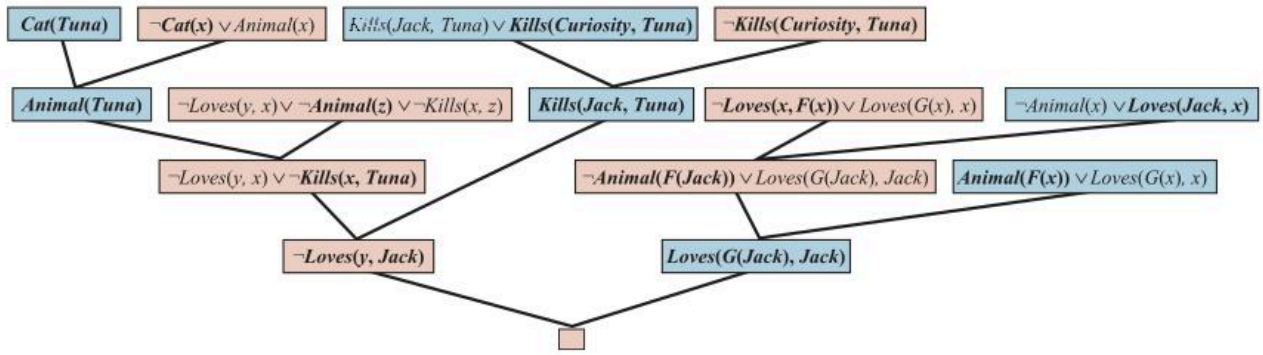
حال رویهٔ تبدیل را برای تبدیل هر جمله به CNF اعمال می‌کنیم:

```
A1. Animal(F(x)) ∨ Loves(G(x),x)
A2. ¬Loves(x,F(x)) ∨ Loves(G(x),x)
B.  ¬Loves(y,x) ∨ ¬Animal(z) ∨ ¬Kills(x,z)
C.  ¬Animal(x) ∨ Loves(Jack,x)
D.  Kills(Jack,Tuna) ∨ Kills(Curiosity,Tuna)
E.  Cat(Tuna)
F.  ¬Cat(x) ∨ Animal(x)
G. ¬Kills(Curiosity,Tuna)¬
```

برهان حل که کوریوسیتی گربه را کشت در شکل ۹.۱۱ داده شده. به انگلیسی، برهان را می‌توان چنین بازگویی کرد:

فرض کن کوریوسیتی تونا را نکشت. می‌دانیم یا جک یا کوریوسیتی کشت؛ پس جک باید کشته باشد. حال، تونا گربه است و گربه‌ها جانورند، پس تونا جانور است. چون هرکس که جانوری را بکشد توسط هیچ‌کس دوست داشته نمی‌شود، می‌دانیم هیچ‌کس جک را دوست ندارد. از سوی دیگر، جک همهٔ جانوران را دوست دارد، پس کسی او را دوست دارد؛ پس تناقض داریم. بنابراین، کوریوسیتی گربه را کشت.

برهان به پرسش «آیا کوریوسیتی گربه را کشت؟» پاسخ می‌دهد، اما اغلب می‌خواهیم پرسش‌های عمومی‌تری بپرسیم، مانند «چه کسی گربه را کشت؟» حل می‌تواند این کار را بکند، اما برای گرفتن پاسخ کمی کار بیشتر لازم است. هدف ∃w Kills(w,Tuna) است که وقتی نقیض شود، در CNF ¬Kills(w,Tuna) می‌شود. با تکرار برهان شکل ۹.۱۱ با هدف نقیض‌شدهٔ تازه، درخت برهان مشابهی به دست می‌آوریم، اما با جانشینی {w/Curiosity} در یکی از گام‌ها. پس در این مورد، فهمیدن اینکه چه کسی گربه را کشت فقط مسئلهٔ پیگیری اتصال‌های متغیرهای پرس‌وجو در برهان است. متأسفانه، حل گاهی می‌تواند برای هدف‌های وجودی «برهان‌های غیرسازنده» (Nonconstructive Proofs) تولید کند، که در آن می‌دانیم پرس‌وجو صادق است، اما اتصال یکتایی برای متغیر نیست.



شکل ۹.۱۱ – برهان حل که کوریوسیتی گربه را کشت. به استفاده از عامل‌گیری در اشتقاق بند Loves(G(Jack),Jack) توجه کنید. همچنین در بالا-راست توجه کنید، یکسان‌سازی Loves(x,F(x)) و Loves(Jack,x) فقط پس از استانداردسازی جداسازی متغیرها می‌تواند موفق شود.

۹.۵.۴ کامل بودن حل

این بخش برهان کاملی از حل می‌دهد. کسانی که مایل‌اند آن را بدیهی بگیرند می‌توانند با خیال راحت از آن بگذرند.

نشان می‌دهیم حل «ابطال‌کامل» (Refutation Complete) است، یعنی اگر مجموعه‌ای از جمله‌ها ارضاپذیر نباشد، حل همیشه قادر به مشتق‌کردن تناقض خواهد بود. حل را نمی‌توان برای تولید همهٔ پیامدهای منطقی مجموعه‌ای از جمله‌ها استفاده کرد، اما می‌توان از آن برای اثبات اینکه جملهٔ داده‌شده‌ای توسط مجموعه‌جمله مستلزم است استفاده کرد. از این رو، می‌توان از آن برای یافتن همهٔ پاسخ‌های پرسشِ داده‌شدهٔ Q(x) با اثبات ارضاپذیرنبودن KB ∧ ¬Q(x) استفاده کرد.

بدیهی می‌گیریم هر جمله‌ای در منطق مرتبهٔ اول (بدون برابری) را می‌توان به‌صورت مجموعه‌ای از بندها در CNF بازنویسی کرد. این را می‌توان با استقرا بر شکل جمله، با استفاده از جمله‌های اتمی به‌عنوان حالت پایه، اثبات کرد (دیویس و پاتنم، ۱۹۶۰). هدف ما پس اثبات زیر است: اگر S مجموعه‌بندی ارضاپذیر نباشد، آنگاه اعمال تعداد متناهی گام حل بر S تناقضی به بار خواهد آورد.

طرح برهان‌مان از برهان اصلی رابینسون با برخی ساده‌سازی‌ها از جنزِرت و نیلسون (۱۹۸۷) پیروی می‌کند. ساختار پایهٔ برهان (شکل ۹.۱۲) چنین است:

1. نخست، مشاهده می‌کنیم اگر S ارضاپذیر نباشد، مجموعهٔ خاصی از مصداق‌های زمینی بندهای S وجود دارد چنان‌که این مجموعه نیز ارضاپذیر نباشد (قضیهٔ اربن).
2. سپس به قضیهٔ حل زمینیِ داده‌شده در فصل ۷ توسل می‌جوییم که می‌گوید حل گزاره‌ای برای جمله‌های زمینی کامل است.
3. سپس از «لم بالابردن» (Lifting Lemma) استفاده می‌کنیم تا نشان دهیم برای هر برهان حل گزاره‌ای با استفاده از مجموعه‌جمله‌های زمینی، برهان حل مرتبهٔ اول متناظری با استفاده از جمله‌های مرتبهٔ اولی که جمله‌های زمینی از آن‌ها به دست آمده‌اند وجود دارد.

برای انجام گام نخست، به سه مفهوم تازه نیاز داریم:

- **جهان اربن (Herbrand Universe):** اگر S مجموعه‌بندی باشد، HS، جهان اربن S، مجموعهٔ همهٔ عبارت‌های زمینی قابل ساخت از موارد زیر است:

(الف) نمادهای تابع در S، اگر باشند. (ب) نمادهای ثابت در S، اگر باشند؛ اگر هیچ نبود، نماد ثابت پیش‌فرض، A. برای نمونه، اگر S فقط بند ¬P(x,F(x,A)) ∨ ¬Q(x,A) ∨ R(x,B) را شامل شود، HS مجموعهٔ نامتناهی زیر از عبارت‌های زمینی است:

```
{... ,A, B, F(A,A), F(A,B), F(B,A), F(B,B), F(A,F(A,A))}
```

- **اشباع (Saturation):** اگر S مجموعه‌بندی و P مجموعه‌ای از عبارت‌های زمینی باشد، P(S)، اشباع S نسبت به P، مجموعهٔ همهٔ بندهای زمینیِ به‌دست‌آمده با اعمال همهٔ جانشینی‌های سازگار ممکن عبارت‌های زمینی در P برای متغیرهای S است.
- **پایهٔ اربن (Herbrand Base):** اشباع مجموعه‌بندی S نسبت به جهان اربنش «پایهٔ اربن» S نامیده می‌شود و HS(S) نوشته می‌شود. برای نمونه، اگر S تنها بند داده‌شده در بالا را شامل شود، HS(S) مجموعهٔ نامتناهی بندهای زیر است:

```
,P(A,F(A,A)) ∨ ¬Q(A,A) ∨ R(A,B)¬}
,P(B,F(B,A)) ∨ ¬Q(B,A) ∨ R(B,B)¬
,P(F(A,A),F(F(A,A),A)) ∨ ¬Q(F(A,A),A) ∨ R(F(A,A),B)¬
,P(F(A,B),F(F(A,B),A)) ∨ ¬Q(F(A,B),A) ∨ R(F(A,B),B)¬
{...
```

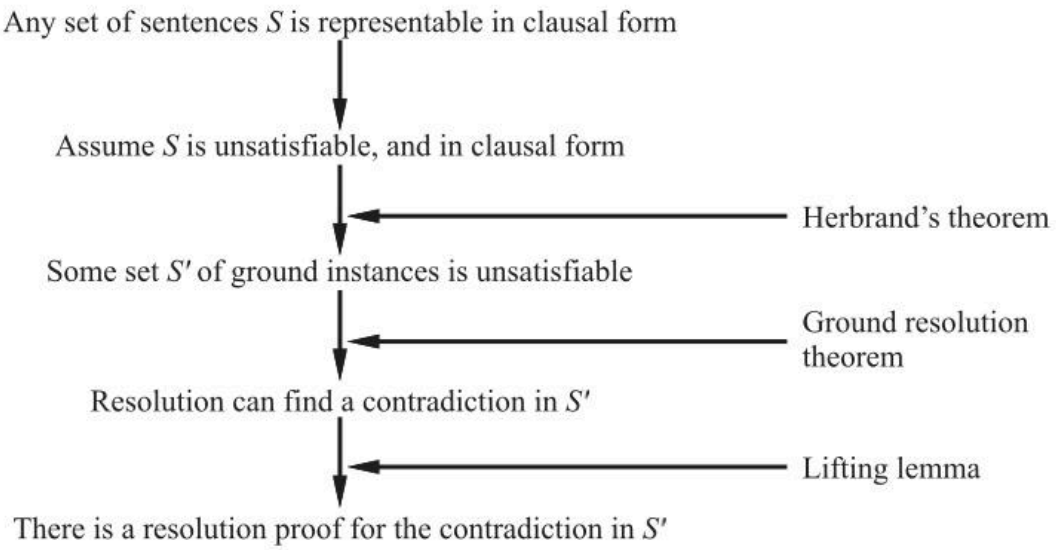
این تعریف‌ها به ما امکان می‌دهند شکلی از قضیهٔ اربن را بیان کنیم (اربن، ۱۹۳۰):

> *اگر مجموعه‌بندی S ارضاپذیر نباشد، زیرمجموعهٔ متناهی‌ای از HS(S) وجود دارد که نیز ارضاپذیر نیست.*

'S این زیرمجموعهٔ متناهی از جمله‌های زمینی باشد. حال، می‌توانیم به قضیهٔ حل زمینی (صفحهٔ ۲۲۸) توسل بجوییم تا نشان دهیم بستار حل RC('S) شامل بند خالی است. یعنی اجرای حل گزاره‌ای تا تکمیل بر 'S تناقضی مشتق خواهد کرد.

حال که ثابت کرده‌ایم همیشه برهان حلی شامل زیرمجموعهٔ متناهی‌ای از پایهٔ اربن S هست، گام بعدی نشان‌دادن این است که برهان حلی با استفاده از خودِ بندهای S وجود دارد، که لزوماً بندهای زمینی نیستند. با در نظر گرفتن یک کاربرد تکی از قاعدهٔ حل آغاز می‌کنیم. رابینسون این لم را بیان کرد:

> *فرض کنید C1 و C2 دو بند بدون متغیر مشترک باشند، و C'1 و C'2 مصداق‌های زمینی C1 و C2 باشند. اگر C' حاصل‌حلی از C'1 و C'2 باشد، آنگاه بندی C وجود دارد چنان‌که (۱) C حاصل‌حلی از C1 و C2 است و (۲) C' مصداق زمینی C است.*



شکل ۹.۱۲ – ساختار برهان کاملی برای حل.

این «لم بالابردن» نامیده می‌شود، زیرا گام برهان را از بندهای زمینی به بندهای مرتبهٔ اول عمومی بالا می‌برد. برای اثبات لم بالابردن پایه‌اش، رابینسون مجبور شد یکسان‌سازی را اختراع کند و همهٔ ویژگی‌های عمومی‌ترین یکسان‌سازها را مشتق کند. به‌جای تکرار برهان در این‌جا، صرفاً لم را نشان می‌دهیم:

```
                                C1 = ¬P(x,F(x,A)) ∨ ¬Q(x,A) ∨ R(x,B)
                                     C2 = ¬N(G(y),z) ∨ P(H(y),z)
                  C′1 = ¬P(H(B),F(H(B),A)) ∨ ¬Q(H(B),A) ∨ R(H(B),B)
                       C′2 = ¬N(G(B),F(H(B),A)) ∨ P(H(B),F(H(B),A))
                   C′ = ¬N(G(B),F(H(B),A)) ∨ ¬Q(H(B),A) ∨ R(H(B),B)
                    C  = ¬N(G(y),F(H(y),A)) ∨ ¬Q(H(y),A) ∨ R(H(y),B)
```

می‌بینیم در واقع 'C مصداق زمینی C است. در حالت کلی، برای اینکه C'1 و C'2 حاصل‌حلی داشته باشند، باید با اعمال عمومی‌ترین یکسان‌ساز یک جفت لیترال مکمل در C1 و C2 بر C1 و C2 ساخته شده باشند. از لم بالابردن، مشتق‌کردن گزارهٔ مشابهی دربارهٔ هر توالی از کاربردهای قاعدهٔ حل آسان است:

> *برای هر بند 'C در بستار حل 'S بندی C در بستار حل S وجود دارد چنان‌که 'C مصداق زمینی C است و اشتقاق C همان طول اشتقاق 'C را دارد.*

از این واقعیت برمی‌آید که اگر بند خالی در بستار حل 'S ظاهر شود، باید در بستار حل S نیز ظاهر شود. این به‌سبب آن است که بند خالی نمی‌تواند مصداق زمینی بند دیگری باشد. برای جمع‌بندی: نشان داده‌ایم اگر S ارضاپذیر نباشد، اشتقاق متناهی‌ای از بند خالی با استفاده از قاعدهٔ حل وجود دارد.

بالابردن اثبات قضیه از بندهای زمینی به بندهای مرتبهٔ اول افزایش عظیمی در قدرت می‌دهد. این افزایش از این واقعیت می‌آید که برهان مرتبهٔ اول فقط تا حد لازم برای برهان متغیرها را مصداق‌سازی می‌کند، در حالی که روش‌های بند زمینی مجبور بودند شمار عظیمی از مصداق‌سازی‌های دلخواه را بررسی کنند.

۹.۵.۵ برابری

هیچ‌یک از روش‌های استنتاجِ تاکنون‌توصیف‌شده در این فصل نمی‌تواند ادعایی به شکل x = y را بدون کار اضافی مدیریت کند. سه رویکرد متمایز می‌توان گرفت. نخستین، اصل‌موضوعه‌سازی برابری است — نوشتن جمله‌هایی دربارهٔ رابطهٔ برابری در پایگاه دانش. باید بگوییم برابری بازتابی، متقارن و متعدی است، و نیز باید بگوییم می‌توانیم برابر را در هر گزاره یا تابعی با برابر جانشین کنیم. پس به سه اصل‌موضوعهٔ پایه نیاز داریم، و سپس یکی برای هر گزاره و تابع:

```
                                                    ∀x x = x
                                                ∀x,y x = y ⇒ y = x
                                            ∀x,y,z x = y ∧ y = z ⇒ x = z
                                          ∀x,y x = y ⇒ (P1(x) ⇔ P1(y))
                                        ∀x,y x = y ⇒ (P2(x) ⇔ P2(y)) ...
                                  ∀w,x,y,z w = y ∧ x = z ⇒ (F1(w,x) = F1(y,z))
                                ∀w,x,y,z w = y ∧ x = z ⇒ (F2(w,x) = F2(y,z)) ...
```

با این جمله‌ها، رویهٔ استنتاج استانداردی مانند حل می‌تواند تکلیف‌های نیازمند استدلال برابری، مانند حل معادله‌های ریاضی، را انجام دهد. اما این اصل‌موضوعه‌ها نتیجه‌های فراوانی تولید خواهند کرد، بیشترشان به برهان کمکی نمی‌کنند. پس رویکرد دوم افزودن قواعد استنتاج به‌جای اصل‌موضوعه‌هاست. ساده‌ترین قاعده، «دمادولاسیون»

(Demodulation)، بند واحد x=y و بندی α که شامل عبارت x است می‌گیرد و بند تازه‌ای می‌دهد که با جانشینی y به‌جای x درون α تشکیل شده. اگر عبارت درون α با x یکسان شود کار می‌کند؛ لازم نیست دقیقاً برابر x باشد. توجه کنید دمادولاسیون جهتی است؛ با x = y داده‌شده، x همیشه با y جایگزین می‌شود، هرگز برعکس. این بدان معناست دمادولاسیون را می‌توان برای ساده‌سازی عبارت‌ها با دمادولاتورهایی مانند z+0=z یا z·1=z استفاده کرد. به‌عنوان مثال دیگر، با داده‌شده:

```
Father(Father(x)) = PaternalGrandfather(x)
Birthdate(Father(Father(Bella)), 1926)
```

می‌توانیم با دمادولاسیون نتیجه بگیریم:

```
Birthdate(PaternalGrandfather(Bella), 1926)
```

صورتی‌تر، داریم:

- **دمادولاسیون:** برای هر عبارت y ،x و z، که z در جایی از لیتِرال mi ظاهر می‌شود و UNIFY(x,z) = θ:

$$\frac{x = y, \quad m1 \vee \ldots \vee mn}{SUB(SUBST(\theta,x), SUBST(\theta,y), m1 \vee \ldots \vee mn)}$$

که SUBST جانشینی معمول فهرست اتصال است و SUB(x,y,m) یعنی x را با y در جایی درون m جایگزین کن.

قاعده را می‌توان برای مدیریت بندهای غیرواحد که در آن‌ها علامت تساوی ظاهر می‌شود نیز بسط داد:

- **پارامدولاسیون (Paramodulation):** برای هر عبارت y ،x و z، که z در جایی از لیتِرال mi ظاهر می‌شود و UNIFY(x,z) = θ:

$$\frac{\ell1 \vee \ldots \vee \ell k \vee x = y, \quad m1 \vee \ldots \vee mn}{SUB(SUBST(\theta,x), SUBST(\theta,y), SUBST(\theta, \ell1 \vee \ldots \vee \ell k \vee m1 \vee \ldots \vee mn))}$$

برای نمونه، از

```
P(F(x,B),x) ∨ Q(x)
```

۹

```
F(A,y) = y ∨ R(y)
```

داریم θ = UNIFY(F(A,y), F(x,B)) = {x/A, y/B}، و می‌توانیم با پارامدولاسیون جملهٔ

```
P(B,A) ∨ Q(A) ∨ R(B)
```

را نتیجه بگیریم. پارامدولاسیون رویهٔ استنتاج کاملی برای منطق مرتبهٔ اول با برابری می‌دهد.

رویکرد سوم استدلال برابری را کاملاً درون الگوریتم یکسان‌سازی بسط‌یافته مدیریت می‌کند. یعنی عبارت‌ها یکسان‌شدنی‌اند اگر تحت برخی جانشینی‌ها به‌شکل اثبات‌پذیر برابر باشند، که «اثبات‌پذیر» اجازهٔ استدلال برابری را می‌دهد. برای نمونه، عبارت‌های 1+2 و 2+1 معمولاً یکسان‌شدنی نیستند، اما الگوریتم یکسان‌سازی که می‌داند x+y = y+x می‌تواند آن‌ها را با جانشینی خالی یکسان کند. «یکسان‌سازی معادله‌ای» (Equational Unification) از این نوع را می‌توان با الگوریتم‌های کارآمدی که برای اصل‌موضوعه‌های خاص طراحی شده‌اند (جابه‌جایی، شرکت‌پذیری و الی آخر) انجام داد، نه با استنتاج صریح با آن اصل‌موضوعه‌ها. اثبات‌کننده‌های قضیه‌ای که از این فن استفاده می‌کنند با سامانه‌های CLP توصیف‌شده در بخش ۹.۴ ارتباط نزدیک دارند.

۹.۵.۶ راهبردهای حل

می‌دانیم کاربردهای مکرر قاعدهٔ استنتاج حل سرانجام برهانی را اگر وجود داشته باشد خواهد یافت. در این زیربخش، راهبردهایی را بررسی می‌کنیم که به یافتن کارآمد برهان‌ها کمک می‌کنند.

ترجیح واحد (Unit Preference): این راهبرد ترجیح می‌دهد حل‌هایی انجام دهد که یکی از جمله‌هایش لیترال تکی (که بند واحد نیز نامیده می‌شود) باشد. ایدهٔ پشت راهبرد آن است که می‌کوشیم بند خالی تولید کنیم، پس ممکن است ایدهٔ خوبی باشد استنتاج‌هایی را ترجیح دهیم که بندهای کوتاه‌تر تولید می‌کنند. حل جملهٔ واحد (مانند P) با هر جملهٔ دیگر (مانند P ∨ ¬Q ∨ R) همیشه بندی (در این مورد، Q ∨ R¬) می‌دهد که از بند دیگر کوتاه‌تر است. وقتی راهبرد ترجیح واحد نخستین بار برای استنتاج گزاره‌ای در ۱۹۶۴ امتحان شد، به شتاب چشمگیری انجامید و اثبات قضیه‌هایی را ممکن کرد که بدون ترجیح قابل مدیریت نبودند. حل واحد شکل محدودی از حل است که در آن هر گام حل باید شامل بند واحد باشد. حل واحد در حالت کلی ناقص است، اما برای بندهای هورن کامل است. برهان‌های حل واحد روی بندهای هورن شبیه زنجیره‌سازی پیشرو هستند.

اثبات‌کنندهٔ قضیهٔ OTTER (مک‌کیون، ۱۹۹۰) از شکلی از جستجوی بهترین-اول استفاده می‌کند. تابع اکتشافی‌اش «وزن» هر بند را اندازه می‌گیرد، که بندهای سبک‌تر ترجیح داده می‌شوند. انتخاب دقیق اکتشافی به کاربر بستگی دارد، اما در حالت کلی، وزن بند باید با اندازه یا دشواری‌اش همبسته باشد. بندهای واحد سبک تلقی می‌شوند؛ جستجو را پس می‌توان تعمیمی از راهبرد ترجیح واحد دید.

مجموعهٔ پشتیبان (Set of Support): ترجیح‌هایی که برخی حل‌ها را نخست امتحان می‌کنند کمک‌کننده‌اند، اما در حالت کلی مؤثرتر است بکوشیم برخی حل‌های بالقوه را یکجا حذف کنیم. برای نمونه، می‌توانیم اصرار کنیم هر گام حل شامل دست‌کم یک عنصر از مجموعهٔ خاصی از بندها — مجموعهٔ پشتیبان — باشد. حاصل‌حل سپس به مجموعهٔ پشتیبان افزوده می‌شود. اگر مجموعهٔ پشتیبان نسبت به کل پایگاه دانش کوچک باشد، فضای جستجو به‌شکل

چشمگیری کاهش می‌یابد. برای تضمین کامل بودن این راهبرد، می‌توانیم مجموعهٔ پشتیبان S را چنان برگزینیم که بقیهٔ جمله‌ها به‌طور مشترک ارضاپذیر باشند. برای نمونه، می‌توان از پرس‌وجوی نقیض‌شده به‌عنوان مجموعهٔ پشتیبان استفاده کرد، با این فرض که پایگاه دانش اصلی سازگار است. (بالاخره، اگر سازگار نباشد، اینکه پرس‌وجو از آن برمی‌آید توخالی است.) راهبرد مجموعهٔ پشتیبان مزیت اضافی تولید درخت‌های برهان هدف‌محور را دارد که اغلب برای انسان‌ها آسان‌فهم‌اند.

حل ورودی (Input Resolution): در این راهبرد، هر حل یکی از جمله‌های ورودی (از KB یا پرس‌وجو) را با جملهٔ دیگری ترکیب می‌کند. برهان شکل ۹.۱۰ در صفحهٔ ۳۰۱ فقط از حل‌های ورودی استفاده می‌کند و شکل مشخص «ستون فقرات» تکی با جمله‌های تکی که به ستون فقرات ترکیب می‌شوند را دارد. بدیهی است فضای درخت‌های برهان این شکل از فضای همهٔ گراف‌های برهان کوچک‌تر است. در پایگاه‌های دانش هورنی، مدوس پوننس نوعی راهبرد حل ورودی است، زیرا دلالتی از KB اصلی را با جمله‌های دیگر ترکیب می‌کند. پس تعجب‌آور نیست حل ورودی برای پایگاه‌های دانشی که در شکل هورن‌اند کامل است، اما در حالت کلی ناقص. راهبرد «حل خطی» (Linear Resolution) تعمیم جزئی‌ای است که به P و Q اجازه می‌دهد با هم حل شوند یا اگر P در KB اصلی است یا اگر P نیای Q در درخت برهان است. حل خطی کامل است.

زیرشمولی (Subsumption): روش زیرشمولی همهٔ جمله‌هایی را حذف می‌کند که توسط جملهٔ موجود در KB «زیرشمول» می‌شوند (یعنی خاص‌تر از آن‌اند). برای نمونه، اگر P(x) در KB باشد، دلیلی برای افزودن P(A) نیست و حتی دلیل کمتری برای افزودن P(A) ∨ Q(B). زیرشمولی به کوچک‌ماندن KB و در نتیجه کوچک‌ماندن فضای جستجو کمک می‌کند.

یادگیری (Learning): می‌توانیم اثبات‌کنندهٔ قضیه را با یادگیری از تجربه بهبود دهیم. با مجموعه‌ای از قضیه‌های قبلاً اثبات‌شده، سامانهٔ یادگیری ماشین را آموزش بده تا به این پرسش پاسخ دهد: با مجموعه‌ای از مقدمه‌ها و هدفی برای اثبات، چه گام‌های برهانی شبیه گام‌هایی هستند که در گذشته موفق بوده‌اند؟ سامانهٔ DEEPHOL (بانسال و همکاران، ۲۰۱۹) دقیقاً همین کار را می‌کند، با استفاده از شبکه‌های عصبی عمیق (فصل ۲۱ را ببینید) برای ساختن مدل‌هایی (به نام «جاسازی»ها) از هدف‌ها و مقدمه‌ها و استفاده از آن‌ها برای انتخاب‌ها. آموزش می‌تواند از برهان‌های تولیدشده توسط انسان و رایانه به‌عنوان مثال استفاده کند، با شروع از مجموعه‌ای از ۱۰,۰۰۰ برهان.

کاربردهای عملی اثبات‌کننده‌های قضیهٔ مبتنی بر حل

نشان داده‌ایم منطق مرتبهٔ اول چگونه می‌تواند سناریوی سادهٔ جهان واقعی شامل مفاهیمی مانند فروش، سلاح و شهروندی را بازنمایی کند. اما سناریوهای پیچیدهٔ جهان واقعی عدم قطعیت و ناشناخته‌های زیادی دارند. منطق برای سناریوهای شامل مفاهیم صوری و دقیقاً تعریف‌شده موفق‌تر ثابت شده، مانند «سنتز» (Synthesis) و «تأیید» (Verification) سخت‌افزار و نرم‌افزار. پژوهش اثبات قضیه در حوزه‌های طراحی سخت‌افزار، زبان‌های برنامه‌نویسی و مهندسی نرم‌افزار انجام می‌شود — نه فقط در هوش مصنوعی.

در مورد سخت‌افزار، اصل‌موضوعه‌ها تعامل‌های میان سیگنال‌ها و عناصر مدار را توصیف می‌کنند. (بخش 8.4.2 در صفحهٔ ۲۷۳ را برای مثالی ببینید.) استدلال‌گرهای منطقیِ ویژه‌طراحی‌شده برای تأیید توانسته‌اند کل CPUها، از جمله ویژگی‌های زمانی‌شان، را تأیید کنند (سریواس و بیکفورد، ۱۹۹۰). اثبات‌کنندهٔ قضیهٔ AURA برای طراحی مدارهایی که فشرده‌تر از هر طراحی قبلی‌اند اعمال شده (وویچخوفسکی و وویچیک، ۱۹۸۳).

در مورد نرم‌افزار، استدلال دربارهٔ برنامه‌ها بسیار شبیه استدلال دربارهٔ کنش‌هاست، مانند فصل ۷: اصل‌موضوعه‌ها پیش‌شرط‌ها و اثرات هر گزاره را توصیف می‌کنند. سنتز صوری الگوریتم‌ها یکی از نخستین کاربردهای اثبات‌کننده‌های قضیه بود، چنان‌که کوردل گرین (۱۹۶۹) ترسیم کرد، که بر ایده‌های پیشین هربرت سایمون (۱۹۶۳) بنا کرد. ایده آن است که به‌صورت سازنده قضیه‌ای به این مضمون اثبات کنیم که «برنامهٔ p وجود دارد که مشخصات معینی را برآورده می‌کند.» هرچند سنتز قیاسیِ کاملاً خودکار، چنان‌که نامیده می‌شود، هنوز برای برنامه‌نویسی عمومی‌منظوره شدنی نشده، سنتز قیاسی دست‌هدایت‌شده در طراحی چند الگوریتم نوآورانه و پیچیده موفق بوده است. سنتز برنامه‌های ویژه‌منظوره، مانند کد محاسبات علمی، نیز حوزهٔ پژوهشی فعالی است.

فنون مشابهی اکنون توسط سامانه‌هایی مانند بررسی‌کنندهٔ مدل SPIN (هولتزمن، ۱۹۹۷) بر تأیید نرم‌افزار اعمال می‌شوند. برای نمونه، برنامهٔ کنترل فضاپیمای «عامل از راه دور» پیش و پس از پرواز تأیید شد (هاولوند و همکاران، ۲۰۰۰). الگوریتم رمزنگاری کلید عمومی RSA و الگوریتم تطبیق رشتهٔ بویر–مور بدین ترتیب تأیید شده‌اند (بویر و مور، ۱۹۸۴).

خلاصهٔ فصل

تحلیلی از استنتاج منطقی در منطق مرتبهٔ اول و تعدادی الگوریتم برای انجامش ارائه کرده‌ایم.

- رویکرد نخست از قواعد استنتاج (مصداق‌سازی همگانی و مصداق‌سازی وجودی) برای گزاره‌سازی مسئلهٔ استنتاج استفاده می‌کند. معمولاً این رویکرد کند است، مگر آنکه حوزه کوچک باشد.
- استفاده از یکسان‌سازی برای شناسایی جانشینی‌های مناسب برای متغیرها، گام مصداق‌سازی را در برهان‌های مرتبهٔ اول حذف می‌کند و فرایند را در بسیاری موارد کارآمدتر می‌کند.
- نسخهٔ بالابردهٔ مدوس پوننس از یکسان‌سازی برای ارائهٔ قاعدهٔ استنتاج طبیعی و قدرتمندی، مدوس پوننس تعمیم‌یافته، استفاده می‌کند. الگوریتم‌های زنجیره‌سازی پیشرو و پسرو این قاعده را بر مجموعه‌های بند معین اعمال می‌کنند.
- مدوس پوننس تعمیم‌یافته برای بندهای معین کامل است، هرچند مسئلهٔ لازمه نیمه‌تصمیم‌پذیر است. برای پایگاه‌های دانش دیتالاگِ متشکل از بندهای معین بدون تابع، لازمه تصمیم‌پذیر است.
- زنجیره‌سازی پیشرو در پایگاه‌های دادهٔ قیاسی استفاده می‌شود، جایی که می‌توان آن را با عملیات پایگاه دادهٔ رابطه‌ای ترکیب کرد. همچنین در سامانه‌های تولیدی استفاده می‌شود که با مجموعه‌قواعد بسیار بزرگ به‌روزرسانی‌های کارآمد انجام می‌دهند. زنجیره‌سازی پیشرو برای دیتالاگ کامل است و در زمان چندجمله‌ای اجرا می‌شود.
- زنجیره‌سازی پسرو در سامانه‌های برنامه‌نویسی منطقی استفاده می‌شود که از فناوری کامپایلر پیچیده برای استنتاج بسیار سریع استفاده می‌کنند. زنجیره‌سازی پسرو از استنتاج‌های زائد و حلقه‌های بی‌نهایت رنج می‌برد؛ این‌ها را می‌توان با یادسپاری (Memoization) تخفیف داد.
- پرولوگ، برخلاف منطق مرتبهٔ اول، از جهان بسته با فرض نام‌های یکتا و نقیض-به‌مثابه-شکست استفاده می‌کند. این‌ها پرولوگ را زبان برنامه‌نویسی عملی‌تری می‌کنند، اما آن را از منطق ناب دورتر می‌کنند.
- قاعدهٔ استنتاج حل تعمیم‌یافته سامانهٔ برهان کاملی برای منطق مرتبهٔ اول فراهم می‌کند، با استفاده از پایگاه‌های دانش در شکل عادی عطفی.
- راهبردهای متعددی برای کاهش فضای جستجوی سامانهٔ حل بدون به خطر انداختن کامل بودن وجود دارد. یکی از مهم‌ترین مسائل سروکار داشتن با برابری است؛ نشان دادیم چگونه دمادولاسیون و پارامدولاسیون را می‌توان استفاده کرد.
- اثبات‌کننده‌های قضیهٔ کارآمد مبتنی بر حل برای اثبات قضیه‌های ریاضی جالب و تأیید و سنتز نرم‌افزار و سخت‌افزار استفاده شده‌اند.

یادداشت‌های کتاب‌شناختی و تاریخی

گوتلوب فرگه، که منطق مرتبهٔ اول کامل را در ۱۸۷۹ توسعه داد، سامانهٔ استنتاجش را بر مجموعه‌ای از طرح‌های معتبر به‌علاوهٔ یک قاعدهٔ استنتاج تکی، مدوس پونس، بنا کرد. وایتهد و راسل (۱۹۱۰) به اصطلاح «قواعد گذر» شرح دادند (خود اصطلاح از اربن (۱۹۳۰) است) که برای حرکت سورها به جلوی فرمول‌ها استفاده می‌شود. ثابت‌های اسکولم و توابع اسکولم، به‌طور مناسب، توسط تورالف اسکولم (۱۹۲۰) معرفی شدند. به‌طرز عجیبی، اسکولم بود که جهان اربن را معرفی کرد (اسکولم، ۱۹۲۸).

قضیهٔ اربن (اربن، ۱۹۳۰) نقشی حیاتی در توسعهٔ استدلال خودکار بازی کرده. اربن همچنین مخترع یکسان‌سازی است. گودل (۱۹۳۰) بر ایده‌های اسکولم و اربن بنا کرد تا نشان دهد منطق مرتبهٔ اول رویهٔ برهان کاملی دارد. آلن تورینگ (۱۹۳۶) و آلونزو چرچ (۱۹۳۶) هم‌زمان، با برهان‌های بسیار متفاوت، نشان دادند اعتبار در منطق مرتبهٔ اول تصمیم‌پذیر نیست. متن عالی اندرتون (۱۹۷۲) همهٔ این نتایج را به‌شیوه‌ای دقیق و در عین حال قابل فهم توضیح می‌دهد.

آبراهام رابینسون پیشنهاد کرد می‌توان استدلال‌گر خودکاری با گزاره‌سازی و قضیهٔ اربن ساخت و پل گیلمور (۱۹۶۰) نخستین برنامه را نوشت. دیویس و پاتنم (۱۹۶۰) روش گزاره‌سازی بخش ۹.۱ را معرفی کردند. پراویتز (۱۹۶۰) ایدهٔ کلیدیِ اجازه‌دادن به جست‌وجوی ناسازگاری گزاره‌ای برای هدایت جستجو و تولید عبارت‌ها از جهان اربن فقط وقتی برای برقراری ناسازگاری گزاره‌ای لازم‌اند را توسعه داد. این ایده جان آلن رابینسون (بدون نسبت) را به توسعهٔ حل (رابینسون، ۱۹۶۵) کشاند.

حل برای سامانه‌های پرسش‌وپاسخ توسط کوردل گرین و برترام رافائل (۱۹۶۸) پذیرفته شد. پیاده‌سازی‌های اولیهٔ هوش مصنوعی تلاش زیادی بر ساختارهای داده‌ای صرف کردند که بازیابی کارآمد واقعیت‌ها را اجازه می‌داد؛ این کار در متون برنامه‌نویسی هوش مصنوعی پوشش داده شده (چارنیاک و همکاران، ۱۹۸۷؛ نورویگ، ۱۹۹۲؛ فورباس و د کلیر، ۱۹۹۳). تا اوایل دههٔ ۱۹۷۰، زنجیره‌سازی پیشرو در هوش مصنوعی به‌عنوان جایگزینی آسان‌فهم برای حل جا افتاده بود. کاربردهای هوش مصنوعی معمولاً شامل شمار زیادی قاعده بود، پس توسعهٔ فناوری مطابقت قاعدهٔ کارآمد، به‌ویژه برای به‌روزرسانی‌های افزایشی، مهم بود.

فناوری سامانه‌های تولیدی برای پشتیبانی از چنین کاربردهایی توسعه یافت. زبان سامانهٔ تولیدی OPS-5 (فورگی، ۱۹۸۱؛ براونستون و همکاران، ۱۹۸۵)، شامل فرایند مطابقت کارآمد ریت (فورگی، ۱۹۸۲)، برای کاربردهایی مانند سامانهٔ خبرهٔ R1 برای پیکربندی ریزکامپیوتر استفاده شد (مک‌درموت، ۱۹۸۲). کراسکا و همکاران (۲۰۱۷) توصیف می‌کنند چگونه شبکه‌های عصبی می‌توانند طرح فهرست‌بندی کارآمدی برای مجموعه‌داده‌های خاص یاد بگیرند.

معماری شناختی SOAR (لیرد و همکاران، ۱۹۸۷؛ لیرد، ۲۰۰۸) برای مدیریت مجموعه‌قواعد بسیار بزرگ طراحی شد — تا یک میلیون قاعده (دورنبوس، ۱۹۹۴). مثال‌های کاربرد SOAR شامل کنترل هواپیمای جنگندهٔ شبیه‌سازی‌شده (جونز و همکاران، ۱۹۹۸)، مدیریت فضای هوایی (تیلور و همکاران، ۲۰۰۷)، شخصیت‌های هوش مصنوعی برای بازی‌های رایانه‌ای (وینترمیوت و همکاران، ۲۰۰۷) و ابزارهای آموزش سربازان (ری و جونز، ۲۰۰۵) است.

حوزهٔ پایگاه‌های دادهٔ قیاسی با کارگاهی در تولوز در ۱۹۷۷ آغاز شد که کارشناسان استنتاج منطقی و پایگاه‌های داده در آن شرکت کردند (گالر و مینکر، ۱۹۷۸). کار تأثیرگذار چاندرا و هارل (۱۹۸۰) و اولمن (۱۹۸۵) به پذیرش دیتالاگ به‌عنوان زبان استاندارد پایگاه‌های دادهٔ قیاسی انجامید. توسعهٔ فن مجموعه‌های جادویی برای بازنویسی قاعده توسط بَنسیلون و همکاران (۱۹۸۶) به زنجیره‌سازی پیشرو اجازه داد مزیت هدف‌محوری را از زنجیره‌سازی پسرو قرض بگیرد.

ظهور اینترنت به افزایش در دسترس‌بودن پایگاه‌های دادهٔ آنلاین عظیم انجامید. این علاقه به یکپارچه‌سازی پایگاه‌های دادهٔ متعدد در «فضای داده»ی سازگار را افزایش داد (هالیوی، ۲۰۰۷). کراسکا و همکاران (۲۰۱۷) شتاب‌های تا ۷۰٪ با استفاده از یادگیری ماشین برای ایجاد ساختارهای فهرست آموخته‌شده برای جست‌وجوی کارآمد داده نشان دادند.

زنجیره‌سازی پسرو برای استنتاج منطقی در زبان PLANNER (هویت، ۱۹۶۹) سرچشمه گرفت. در همین حال، در ۱۹۷۲، آلن کولمرائر پرولوگ را برای هدف تجزیهٔ زبان طبیعی توسعه و پیاده کرده بود — بندهای پرولوگ در آغاز به‌عنوان قواعد دستور زبان مستقل از بافت در نظر گرفته شده بودند (روسل، ۱۹۷۵؛ کولمرائر و همکاران، ۱۹۷۳).

بخش بزرگی از زمینهٔ نظری برنامه‌نویسی منطقی توسط رابرت کوالسکی در کالج سلطنتی لندن، همراه با کولمرائر، توسعه یافت؛ برای مرور تاریخی کوالسکی (۱۹۸۸) و کولمرائر و روسل (۱۹۹۳) را ببینید. کامپایلرهای کارآمد پرولوگ عموماً بر مدل محاسباتی ماشین انتزاعی وارن (WAM) که توسط دیوید اچ. دی. وارن (۱۹۸۳) توسعه یافت مبتنی‌اند. ون روی (۱۹۹۰) نشان داد برنامه‌های پرولوگ می‌توانند از نظر سرعت با برنامه‌های C رقابت کنند.

روش‌های پرهیز از حلقه‌زنی غیرضروری در برنامه‌های منطقی بازگشتی به‌طور مستقل توسط اسمیت و همکاران (۱۹۸۶) و تاماکی و ساتو (۱۹۸۶) توسعه یافت. مقالهٔ اخیر همچنین یادسپاری برای برنامه‌های منطقی را شامل شد، روشی که به‌عنوان برنامه‌نویسی منطقی جدول‌بندی‌شده توسط دیوید اس. وارن به‌طور گسترده توسعه یافت. سویفت و وارن (۱۹۹۴) نشان می‌دهند چگونه WAM را برای مدیریت جدول‌بندی بسط دهند و برنامه‌های دیتالاگ را قادر می‌سازند مرتبه‌ای بزرگی سریع‌تر از سامانه‌های پایگاه دادهٔ قیاسی زنجیره‌سازی پیشرو اجرا شوند.

کار اولیه بر برنامه‌نویسی منطق قیدی توسط جفار و لاسز (۱۹۸۷) انجام شد. جفار و همکاران (۱۹۹۲) سامانهٔ CLP(R) را برای مدیریت قیدهای مقدار حقیقی توسعه دادند. اکنون محصولات تجاری برای حل مسائل پیکربندی و بهینه‌سازی بزرگ‌مقیاس با برنامه‌نویسی قید هست؛ یکی از شناخته‌شده‌ترین‌ها ILOG است (جانکر، ۲۰۰۳). برنامه‌نویسی مجموعه‌پاسخ (گلفوند، ۲۰۰۸) پرولوگ را بسط می‌دهد و فصل و نقیض را اجازه می‌دهد.

متون برنامه‌نویسی منطقی و پرولوگ شامل شوهام (۱۹۹۴)، براتکو (۲۰۰۹)، کلاکسین (۲۰۰۳) و کلاکسین و ملیش (۲۰۰۳) است. پیش از ۲۰۰۰، «ژورنال برنامه‌نویسی منطقی» ژورنال ثبت حوزه بود؛ «نظریه و عمل برنامه‌نویسی منطقی» جایگزینش شده. کنفرانس‌های برنامه‌نویسی منطقی شامل کنفرانس بین‌المللی برنامه‌نویسی منطقی (ICLP) و همایش بین‌المللی برنامه‌نویسی منطقی (ILPS) است.

پژوهش در اثبات قضیهٔ ریاضی حتی پیش از توسعهٔ نخستین سامانه‌های کامل مرتبهٔ اول آغاز شد. اثبات‌کنندهٔ قضیهٔ هندسهٔ هربرت گلرنتر (گلرنتر، ۱۹۵۹) از روش‌های جستجوی اکتشافی همراه با نمودارها برای هرس زیرهدف‌های کاذب استفاده کرد و توانست برخی نتایج بسیار پیچیده در هندسهٔ اقلیدسی اثبات کند. قواعد دمادولاسیون و پارامدولاسیون برای استدلال برابری به‌ترتیب توسط ووس و همکاران (۱۹۶۷) و ووس و رابینسون (۱۹۶۸) معرفی شدند. این قواعد

همچنین مستقل در بافت سامانه‌های بازنویسی عبارت (کنوت و بندیکس، ۱۹۷۰) توسعه یافتند. گنجاندن استدلال برابری در الگوریتم یکسان‌سازی از گوردون پلاتکین (۱۹۷۲) است. ژوانو و کرشنر (۱۹۹۱) یکسان‌سازی معادله‌ای را از دیدگاه بازنویسی عبارت مرور می‌کنند. مروری بر یکسان‌سازی توسط بادر و اسنایدر (۲۰۰۱) داده شده.

تعدادی راهبرد کنترل برای حل پیشنهاد شده، با راهبرد ترجیح واحد آغاز می‌شود (ووس و همکاران، ۱۹۶۴). راهبرد مجموعهٔ پشتیبان توسط ووس و همکاران (۱۹۶۵) برای فراهم‌کردن درجه‌ای از هدف‌محوری در حل پیشنهاد شد. حل خطی نخستین بار در لولند (۱۹۷۰) ظاهر شد. جنزِرت و نیلسون (۱۹۸۷، فصل ۵) تحلیلی از گسترهٔ وسیعی از راهبردهای کنترل ارائه می‌دهند. آلمی و همکاران (۲۰۱۷) نشان می‌دهند چگونه سامانهٔ DEEPMATH از شبکه‌های عصبی عمیق برای انتخاب اصل‌موضوعه‌هایی استفاده می‌کند که به احتمال زیاد به برهان می‌انجامند وقتی به اثبات‌کنندهٔ قضیهٔ سنتی داده می‌شوند. از جهتی، شبکهٔ عصبی نقش شهود ریاضی‌دان را بازی می‌کند و اثبات‌کنندهٔ قضیه نقش تخصص فنی ریاضی‌دان را. (لوس و همکاران، ۲۰۱۷) نشان می‌دهند این رویکرد را می‌توان برای کمک به هدایت جستجو بسط داد و به اثبات قضیه‌های بیشتری اجازه داد.

«یک منطق محاسباتی» (بویر و مور، ۱۹۷۹) مرجع پایهٔ اثبات‌کنندهٔ قضیهٔ بویر–مور است. استیکل (۱۹۹۲) اثبات‌کنندهٔ قضیهٔ فناوری پرولوگ (PTTP) را توصیف می‌کند که کامپایل پرولوگ و حذف مدل را ترکیب می‌کند. SETHEO (لتز و همکاران، ۱۹۹۲) اثبات‌کنندهٔ قضیهٔ پرکاربرد دیگری بر این رویکرد مبتنی است. LEANTAP (بکرت و پوزگا، ۱۹۹۵) اثبات‌کنندهٔ قضیهٔ کارآمدی است که فقط در ۲۵ خط پرولوگ پیاده شده. وایدنباخ (۲۰۰۱) SPASS را توصیف می‌کند، یکی از قوی‌ترین اثبات‌کننده‌های قضیهٔ کنونی. موفق‌ترین اثبات‌کنندهٔ قضیه در رقابت‌های سالانهٔ اخیر VAMPIRE بوده است (ریازانوف و ورونکوف، ۲۰۰۲). سامانهٔ COQ (برتوت و همکاران، ۲۰۰۴) و حل‌کنندهٔ معادله‌ای E (شولتز، ۲۰۰۴) نیز ابزارهای ارزشمندی برای اثبات درستی ثابت کرده‌اند.

اثبات‌کننده‌های قضیه برای سنتز و تأیید خودکار نرم‌افزار استفاده شده‌اند. مثال‌ها شامل نرم‌افزار کنترل کپسول اوریون ناسا (لوری، ۲۰۰۸) و دیگر فضاپیماها (دنی و همکاران، ۲۰۰۶) است. طراحی ریزپردازندهٔ ۳۲-بیتی FM9001 توسط سامانهٔ اثبات قضیهٔ NQTHM درست اثبات شد (هانت و براک، ۱۹۹۲).

کنفرانس استنتاج خودکار (CADE) رقابت سالانه‌ای برای اثبات‌کننده‌های قضیهٔ خودکار برگزار می‌کند. ساتکلیف (۲۰۱۶) رقابت ۲۰۱۶ را توصیف می‌کند؛ سامانه‌های برتر شامل VAMPIRE (ریازانوف و ورونکوف، ۲۰۰۲)، PROVER9 (صبری، ۲۰۱۵) و نسخهٔ به‌روزشدهٔ E (شولتز، ۲۰۱۳) است. ویدیک (۲۰۰۳) قدرت ۱۵ اثبات‌کنندهٔ ریاضی را مقایسه می‌کند. TPTP (هزاران مسئله برای اثبات‌کننده‌های قضیه) کتابخانه‌ای از مسائل اثبات قضیه است که برای مقایسهٔ عملکرد سامانه‌ها مفید است (ساتکلیف و ساتنر، ۱۹۹۸؛ ساتکلیف و همکاران، ۲۰۰۶).

اثبات‌کننده‌های قضیه نتایج ریاضی تازه‌ای به دست آورده‌اند که برای دهه‌ها از ریاضی‌دانان انسانی گریخته بود، چنان‌که در کتاب «استدلال خودکار و کشف برهان‌های زیبای گم‌شده» (ووس و پیپر، ۲۰۰۳) شرح داده شده. برنامهٔ SAM (ریاضیات نیمه‌خودکار) نخستین بود و لمی در نظریهٔ شبکه‌ها اثبات کرد (گارد و همکاران، ۱۹۶۹). برنامهٔ AURA همچنین به پرسش‌های باز در چند حوزهٔ ریاضیات پاسخ داده (ووس و وینکر، ۱۹۸۳). اثبات‌کنندهٔ قضیهٔ بویر–مور (بویر و مور، ۱۹۷۹) توسط ناتاراجان شانکار برای ساختن برهان صوری قضیهٔ ناتمامیت گودل استفاده شد (شانکار، ۱۹۸۶). سامانهٔ NUPRL پارادوکس ژیرار (هو، ۱۹۸۷) و لم هیگمن (مورتی و راسل، ۱۹۹۰) را اثبات کرد.

در ۱۹۳۳، هربرت رابینز مجموعه‌ای ساده از اصل‌موضوعه‌ها — جبر رابینز — پیشنهاد کرد که به نظر می‌رسید جبر بولی را تعریف می‌کند، اما هیچ برهانی یافت نمی‌شد (با وجود کار جدی آلفرد تارسکی و دیگران) تا آنکه EQP (نسخه‌ای از OTTER) برهانی محاسبه کرد (مک‌کیون، ۱۹۹۷). بنزمولر و پالئو (۲۰۱۳) از اثبات‌کنندهٔ قضیهٔ مرتبهٔ بالاتر برای تأیید برهان صوریِ گودل از وجود خدا استفاده کردند (گودل، ۱۹۹۵).

پانوشت‌ها:

[1] این جانشینی‌ها را با تفسیرهای بسط‌یافته‌ای که برای تعریف معناشناسی سورها در بخش 8.2.6 استفاده شد اشتباه نگیرید. جانشینی، متغیر را با عبارت (تکه‌ای از نحو) جایگزین می‌کند تا جملهٔ تازه‌ای تولید کند، در حالی که تفسیر، متغیر را به شیئی در دامنه نگاشت می‌کند.

[2] مدوس پوننس تعمیم‌یافته از مدوس پوننس (صفحهٔ ۲۲۲) عمومی‌تر است به این معنا که واقعیت‌های شناخته‌شده و مقدم دلالت فقط باید تا حد جانشینی مطابقت کنند، نه دقیقاً. از سوی دیگر، مدوس پوننس هر جملهٔ α را به‌عنوان مقدم اجازه می‌دهد، نه فقط عطفی از جمله‌های اتمی.

[3] ریت (Rete) لاتین برای «تور» است. با «ریتی» هم‌قافیه است.

[4] واژهٔ «تولید» (Production) در سامانه‌های تولیدی قاعدهٔ شرط–کنش را نشان می‌دهد.

[5] بند را همچنین می‌توان به‌صورت دلالتی با عطفی از اتم‌ها در مقدم و فصلی از اتم‌ها در تالی بازنمایی کرد (تمرین DISJ.9). این «شکل عادی دلالتی» یا شکل کوالسکی نامیده می‌شود (به‌ویژه وقتی با نماد دلالت راست-به-چپ نوشته شود (کوالسکی، ۱۹۷۹)) و خواندنش عموماً از فصلی با لیترال‌های نقیض‌شدهٔ فراوان بسیار آسان‌تر است.

فصل ۱۰: بازنمایی دانش (Knowledge Representation)

در این فصل نشان می‌دهیم چگونه واقعیت‌های گوناگون دربارهٔ جهان واقعی را در قالبی بازنمایی کنیم که بتوان از آن برای استدلال و حل مسئله استفاده کرد.

فصل‌های پیشین نشان دادند عامل دارای پایگاه دانش چگونه می‌تواند استنتاج‌هایی کند که آن را قادر می‌سازد به‌طور مناسب عمل کند. در این فصل به پرسش محتوایی می‌پردازیم که چه چیزی در پایگاه دانش چنین عاملی گذاشته شود – چگونه واقعیت‌ها دربارهٔ جهان بازنمایی شوند. از منطق مرتبهٔ اول به‌عنوان زبان بازنمایی استفاده خواهیم کرد، اما فصل‌های بعدی صورت‌های بازنمایی متفاوتی معرفی خواهند کرد، مانند شبکه‌های تکلیف سلسله‌مراتبی برای استدلال دربارهٔ برنامه‌ها (فصل ۱۱)، شبکه‌های بیزی برای استدلال با عدم قطعیت (فصل ۱۳)، مدل‌های مارکوف برای استدلال در طول زمان (فصل ۱۷)، و شبکه‌های عصبی عمیق برای استدلال دربارهٔ تصاویر، صداها و دیگر داده‌ها (فصل ۲۱). اما مهم نیست از چه بازنمایی استفاده کنید، واقعیت‌های جهان همچنان باید مدیریت شوند و این فصل به شما حسی از مسائل می‌دهد.

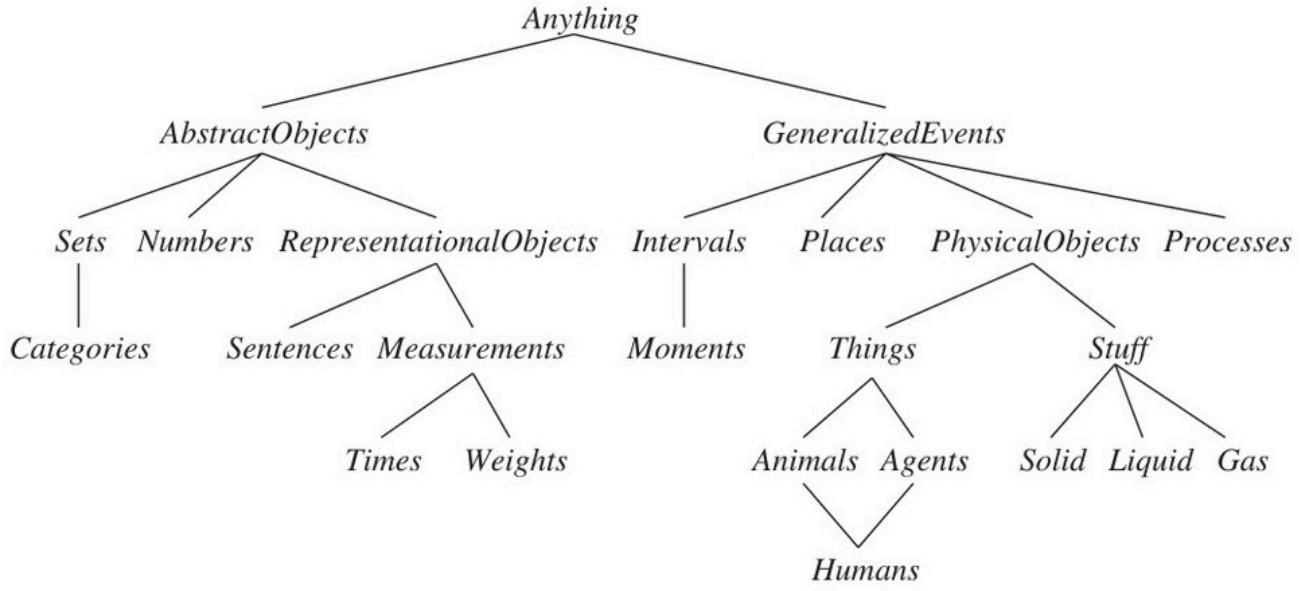
بخش ۱۰.۱ ایدهٔ هستی‌شناسی عمومی را معرفی می‌کند که همه‌چیز در جهان را در سلسله‌مراتبی از رده‌ها سازمان می‌دهد. بخش ۱۰.۲ رده‌های پایهٔ اشیا، مواد و اندازه‌ها را پوشش می‌دهد؛ بخش ۱۰.۳ رویدادها را؛ و بخش ۱۰.۴ دانش دربارهٔ باورها را بحث می‌کند. سپس به در نظر گرفتن فناوری استدلال با این محتوا بازمی‌گردیم: بخش ۱۰.۵ سامانه‌های استدلالی طراحی‌شده برای استنتاج کارآمد با رده‌ها را بحث می‌کند و بخش ۱۰.۶ استدلال با اطلاعات پیش‌فرض را.

۱۰.۱ مهندسی هستی‌شناسی

در حوزه‌های «اسباب‌بازی»، انتخاب بازنمایی چندان مهم نیست؛ بسیاری از انتخاب‌ها کار خواهند کرد. حوزه‌های پیچیده مانند خرید در اینترنت یا رانندگی با خودرو در ترافیک به بازنمایی‌های عمومی‌تر و انعطاف‌پذیرتری نیاز دارند. این فصل نشان می‌دهد چگونه این بازنمایی‌ها را بسازیم، با تمرکز بر مفاهیم عمومی – مانند رویدادها، زمان، اشیای فیزیکی و باورها – که در بسیاری از حوزه‌های متفاوت رخ می‌دهند. بازنمایی این مفاهیم انتزاعی گاهی «مهندسی هستی‌شناسی» (Ontological Engineering) نامیده می‌شود.

نمی‌توانیم امیدوار باشیم همه‌چیز در جهان را بازنمایی کنیم، حتی در کتاب درسی ۱۰۰۰ صفحه‌ای، اما جاگذاری‌هایی می‌گذاریم که دانش تازه برای هر حوزه‌ای بتواند در آن‌ها جا بگیرد. برای نمونه، تعریف می‌کنیم شیء فیزیکی بودن به چه معناست و جزئیات انواع متفاوت اشیا – ربات‌ها، تلویزیون‌ها، کتاب‌ها یا هر چیز دیگر – را می‌توان بعداً پر کرد. این مشابه شیوه‌ای است که طراحان چارچوب برنامه‌نویسی شئ‌گرا (مانند چارچوب گرافیکی جاوا سوییینگ) مفاهیم عمومی

مانند Window را تعریف می‌کنند و انتظار دارند کاربران از آن‌ها برای تعریف مفاهیم خاص‌تر مانند SpreadsheetWindow استفاده کنند. چارچوب عمومی مفاهیم «هستی‌شناسی بالایی» (Upper Ontology) نامیده می‌شود، به‌سبب قرارداد رسم گراف‌ها با مفاهیم عمومی در بالا و مفاهیم خاص‌تر در پایین، چنان‌که در شکل ۱۰.۱.



شکل ۱۰.۱ – هستی‌شناسی بالایی جهان، که موضوعات پوشش‌داده‌شده در ادامهٔ فصل را نشان می‌دهد. هر پیوند نشان می‌دهد مفهومِ پایینی تخصیص (ویژه‌سازیِ) مفهومِ بالایی است. تخصیص‌ها لزوماً ناهم‌پوش نیستند — انسان هم جانور است و هم عامل. در بخش ۱۰.۳.۲ خواهیم دید چرا اشیای فیزیکی زیر رویدادهای تعمیم‌یافته می‌آیند. (ساختار: هرچیز ← اشیای انتزاعی [مجموعه‌ها، اعداد، اشیای بازنمایشی، مقوله‌ها، جمله‌ها، اندازه‌گیری‌ها، وزن‌ها، زمان‌ها] و اشیای فیزیکی [مکان‌ها، فرایندها، جانوران، عامل‌ها، انسان‌ها، مواد [جامد، مایع، گاز]] و رویدادهای تعمیم‌یافته [بازه‌ها، لحظه‌ها])

پیش از بررسی بیشتر هستی‌شناسی، باید یک هشدار مهم بدهیم. ما انتخاب کرده‌ایم از منطق مرتبهٔ اول برای بحث دربارهٔ محتوا و سازمان دانش استفاده کنیم، هرچند جنبه‌های خاصی از جهان واقعی در FOL به‌سختی قابل درک‌اند. دشواری اصلی آن است که بیشتر تعمیم‌ها استثنا دارند یا فقط تا درجه‌ای برقرارند. برای نمونه، هرچند «گوجه‌فرنگی‌ها قرمزند» قاعدهٔ مفیدی است، برخی گوجه‌فرنگی‌ها سبز، زرد یا نارنجی‌اند. استثناهای مشابهی را می‌توان در تقریباً همهٔ قواعد این فصل یافت. توانایی مدیریت استثناها و عدم قطعیت بسیار مهم است، اما عمود بر تکلیف فهم هستی‌شناسی عمومی است. به همین دلیل، بحث استثناها را به بخش ۱۰.۵ این فصل و موضوع عمومی‌تر استدلال با عدم قطعیت را به فصل ۱۲ موکول می‌کنیم.

هستی‌شناسی بالایی به چه درد می‌خورد؟ هستی‌شناسی مدارها در بخش ۸.۴.۲ را در نظر بگیرید. فرض‌های ساده‌کنندهٔ فراوانی می‌کند: زمان کاملاً حذف شده؛ سیگنال‌ها ثابت‌اند و منتشر نمی‌شوند؛ ساختار مدار ثابت می‌ماند. هستی‌شناسی عمومی‌تر سیگنال‌ها را در زمان‌های خاص در نظر می‌گرفت و طول سیم‌ها و تأخیرهای انتشار را شامل می‌شد. این به ما امکان می‌داد ویژگی‌های زمانی مدار را شبیه‌سازی کنیم، و در واقع چنین شبیه‌سازی‌هایی اغلب توسط طراحان مدار انجام می‌شوند.

همچنین می‌توانستیم رده‌های جالب‌تری از گیت‌ها معرفی کنیم، برای نمونه، با توصیف فناوری (CMOS، TTL و مانند آن) به‌همراه مشخصات ورودی–خروجی. اگر می‌خواستیم دربارهٔ قابلیت اطمینان یا تشخیص بحث کنیم، امکان تغییر خودبه‌خودی ساختار مدار یا ویژگی‌های گیت‌ها را شامل می‌کردیم. برای حساب‌کردن ظرفیت‌های سرگردان، باید بازنمایی کنیم سیم‌ها روی برد کجا هستند.

اگر به جهان وومپوس نگاه کنیم، ملاحظات مشابهی اعمال می‌شود. هرچند زمان را بازنمایی می‌کنیم، ساختار ساده‌ای دارد: هیچ‌چیز اتفاق نمی‌افتد مگر وقتی عامل کنش کند، و همهٔ تغییرها آنی‌اند. هستی‌شناسی عمومی‌تر، مناسب‌تر برای جهان واقعی، امکان تغییرهای هم‌زمانِ گسترده‌درزمان را می‌داد. همچنین از گزارهٔ Pit برای گفتن اینکه کدام خانه‌ها گودال دارند استفاده کردیم. می‌توانستیم انواع متفاوت گودال‌ها را با داشتن چند فرد متعلق به ردهٔ گودال‌ها، هر یک با ویژگی‌های متفاوت، اجازه دهیم. به همین ترتیب، شاید بخواهیم جانوران دیگری غیر از وومپوس‌ها را اجازه دهیم. ممکن است نتوان از ادراک‌های در دسترس گونهٔ دقیق را مشخص کرد، پس باید رده‌بندی زیستی بسازیم تا به عامل در پیش‌بینی رفتار ساکنان غار از سرنخ‌های اندک کمک کند.

برای هر هستی‌شناسی ویژه‌منظوره، امکان ایجاد چنین تغییرهایی برای حرکت به سوی عمومیت بیشتر هست. پس پرسش بدیهی‌ای پدیدار می‌شود: آیا همهٔ این هستی‌شناسی‌ها به هستی‌شناسی عمومی‌منظوره‌ای همگرا می‌شوند؟ پس از قرن‌ها بررسی فلسفی و محاسباتی، پاسخ «شاید» است. در این بخش، یک هستی‌شناسی عمومی‌منظوره ارائه می‌کنیم که ایده‌های آن قرن‌ها را ترکیب می‌کند. دو ویژگی عمدهٔ هستی‌شناسی‌های عمومی‌منظوره آن‌ها را از مجموعه‌های هستی‌شناسی‌های ویژه‌منظوره متمایز می‌کند:

- هستی‌شناسی عمومی‌منظوره باید تقریباً در هر حوزهٔ ویژه‌منظوره‌ای کاربردپذیر باشد (با افزودن اصل‌موضوعه‌های حوزه‌ویژه). این بدان معناست هیچ مسئلهٔ بازنمایشی را نمی‌توان نرم کرد یا زیر فرش جارو کرد.
- در هر حوزهٔ به‌اندازهٔ کافی دشوار، حوزه‌های دانشِ متفاوت باید یکپارچه شوند، زیرا استدلال و حل مسئله می‌تواند هم‌زمان چند حوزه را درگیر کند. برای نمونه، سامانهٔ رباتیک تعمیر مدار باید دربارهٔ مدارها بر حسب اتصال الکتریکی و چیدمان فیزیکی استدلال کند، و دربارهٔ زمان، هم برای تحلیل زمانی مدار و هم برای تخمین هزینه‌های کار. جمله‌های توصیف‌کنندهٔ زمان پس باید قابل ترکیب‌شدن با جمله‌های توصیف‌کنندهٔ چیدمان فضایی باشند و باید به یک اندازه برای نانوثانیه‌ها و دقیقه‌ها و برای آنگستروم‌ها و مترها خوب کار کنند.

باید از همان ابتدا بگوییم پروژهٔ مهندسی هستی‌شناسی عمومی تاکنون فقط موفقیت محدودی داشته. هیچ‌یک از کاربردهای برتر هوش مصنوعی (چنان‌که در فصل ۱ فهرست شده) از هستی‌شناسی عمومی استفاده نمی‌کنند — همه از مهندسی دانش ویژه‌منظوره و یادگیری ماشین استفاده می‌کنند. ملاحظات اجتماعی/سیاسی می‌تواند توافق طرف‌های رقیب بر هستی‌شناسی را دشوار کند. چنان‌که تام گروبر (۲۰۰۴) می‌گوید: «هر هستی‌شناسی یک معاهده است — توافقی اجتماعی — میان مردمی با انگیزهٔ مشترکی در اشتراک‌گذاری.» وقتی دغدغه‌های رقیب بر انگیزهٔ اشتراک‌گذاری می‌چربد، هیچ هستی‌شناسی مشترکی نمی‌تواند وجود داشته باشد. هرچه شمار ذی‌نفعان کوچک‌تر باشد، ایجاد هستی‌شناسی آسان‌تر است، و بنابراین ایجاد هستی‌شناسی عمومی‌منظوره از هستی‌شناسی با هدف محدود، مانند هستی‌شناسی زیست‌پزشکی باز (اسمیت و همکاران، ۲۰۰۷)، دشوارتر است. آن هستی‌شناسی‌هایی که وجود دارند از چهار مسیر ایجاد شده‌اند:

1. توسط تیمی از هستی‌شناسان یا منطق‌دانان آموزش‌دیده، که هستی‌شناسی را معماری و اصل‌موضوعه‌ها را می‌نویسند. سامانهٔ CYC بیشتر بدین روش ساخته شد (لنات و گوها، ۱۹۹۰).
2. با واردکردن رده‌ها، ویژگی‌ها و مقادیر از پایگاه داده یا پایگاه‌های دادهٔ موجود. DBPEDIA با واردکردن واقعیت‌های ساخت‌یافته از ویکی‌پدیا ساخته شد (بیزر و همکاران، ۲۰۰۷).
3. با تجزیهٔ اسناد متنی و استخراج اطلاعات از آن‌ها. TEXTRUNNER با خواندن پیکرهٔ بزرگی از صفحات وب ساخته شد (بانکو و اتزیونی، ۲۰۰۸).
4. با وسوسه‌کردن آماتورهای ماهرنشده به واردکردن دانش عقل سلیم. سامانهٔ OPENMIND توسط داوطلبانی ساخته شد که واقعیت‌هایی به انگلیسی پیشنهاد می‌کردند (سینگ و همکاران، ۲۰۰۲؛ چکلوسکی و گیل، ۲۰۰۵).

به‌عنوان مثال، گراف دانش گوگل از محتوای نیمه‌ساخت‌یافتهٔ ویکی‌پدیا استفاده می‌کند و آن را با دیگر محتوای گردآوری‌شده از سراسر وب تحت نظارت انسانی ترکیب می‌کند. بیش از ۷۰ میلیارد واقعیت دارد و برای حدود یک‌سوم جستجوهای گوگل پاسخ فراهم می‌کند (دونگ و همکاران، ۲۰۱۴).

۱۰.۲ رده‌ها و اشیا

سازمان‌دهی اشیا در رده‌ها (Categories) بخش حیاتی بازنمایی دانش است. هرچند تعامل با جهان در سطح اشیای فردی رخ می‌دهد، بخش بزرگی از استدلال در سطح رده‌ها رخ می‌دهد. برای نمونه، خریدار معمولاً هدف خرید یک توپ بسکتبال را دارد، نه توپ بسکتبالِ خاصی مانند BB9. رده‌ها همچنین برای پیش‌بینی دربارهٔ اشیا پس از طبقه‌بندی‌شان خدمت می‌کنند. از ورودی ادراکی وجود اشیای خاصی را استنتاج می‌کنیم، عضویت رده را از ویژگی‌های ادراک‌شدهٔ اشیا استنتاج می‌کنیم، و سپس از اطلاعات رده برای پیش‌بینی دربارهٔ اشیا استفاده می‌کنیم. برای نمونه، از پوست خال‌دار سبزوزرد، قطر یک‌فوت، شکل تخم‌مرغی، گوشت قرمز، دانه‌های سیاه و حضور در راهروی میوه، می‌توان استنتاج کرد شیئی هندوانه است؛ از این، استنتاج می‌شود برای سالاد میوه مفید خواهد بود.

دو انتخاب برای بازنمایی رده‌ها در منطق مرتبهٔ اول هست: گزاره‌ها و اشیا. یعنی می‌توانیم از گزارهٔ Basketball(b) استفاده کنیم، یا می‌توانیم رده را «شیءسازی» (Reification)[1] کنیم، Basketballs. سپس می‌توانیم Member(b, Basketballs) بگوییم، که به اختصار $b \in Basketballs$ می‌نویسیم، تا بگوییم b عضوی از ردهٔ توپ‌های بسکتبال است. Subset(Basketballs, Balls) می‌گوییم، که به اختصار $Basketballs \subset Balls$، تا بگوییم Basketballs زیرردهٔ Balls است. از «زیررده»، «زیرکلاس» و «زیرمجموعه» به‌جای یکدیگر استفاده خواهیم کرد.

رده‌ها دانش را از طریق «وراثت» (Inheritance) سازمان می‌دهند. اگر بگوییم همهٔ مصداق‌های ردهٔ Food خوراکی‌اند، و ادعا کنیم Fruit زیرکلاس Food و Apples زیرکلاس Fruit است، آنگاه می‌توانیم استنتاج کنیم هر سیبی خوراکی است. می‌گوییم سیب‌های فردی ویژگی خوراکی‌بودن را به ارث می‌برند، در این مورد از عضویتشان در ردهٔ Food.

روابط زیرکلاس رده‌ها را در «سلسله‌مراتب رده‌بندی» یا رده‌بندی (Taxonomy) سازمان می‌دهند. رده‌بندی‌ها برای قرن‌ها به‌صراحت در حوزه‌های فنی استفاده شده‌اند. بزرگ‌ترین چنین رده‌بندی‌ای حدود ۱۰ میلیون گونهٔ زنده و منقرض‌شده، بسیاری از آن‌ها سوسک،[2] را در سلسله‌مراتب تکی سازمان می‌دهد؛ علم کتابداری رده‌بندی همهٔ حوزه‌های دانش را

توسعه داده که به‌صورت سامانهٔ دهدهی دیویی رمزگذاری شده؛ و مقام‌های مالیاتی و دیگر ادارات دولتی رده‌بندی‌های گسترده‌ای از مشاغل و محصولات تجاری توسعه داده‌اند.

منطق مرتبهٔ اول بیان واقعیت‌ها دربارهٔ رده‌ها را آسان می‌کند، یا با مرتبط‌کردن اشیا به رده‌ها یا با سورزدن روی اعضایشان. این‌جا چند واقعیت نمونه است:

- شیئی عضو رده‌ای است:

```
BB9 ∈ Basketballs
```

- رده‌ای زیرکلاس ردهٔ دیگر است:

```
Basketballs ⊂ Balls
```

- همهٔ اعضای رده‌ای برخی ویژگی‌ها را دارند:

```
Spherical(x) ⇒ (x ∈ Basketballs)
```

- اعضای رده‌ای را می‌توان با برخی ویژگی‌ها شناخت:

```
Orange(x) ∧ Round(x) ∧ Diameter(x) = 9.5″ ∧ x ∈ Balls ⇒ x ∈ Basketballs
```

- رده‌ای به‌عنوان کل برخی ویژگی‌ها را دارد:

```
Dogs ∈ DomesticatedSpecies
```

توجه کنید چون Dogs رده است و عضو DomesticatedSpecies است، دومی باید ردهٔ رده‌ها باشد. البته برای بسیاری از قواعد بالا استثناهایی هست (توپ‌های بسکتبال سوراخ‌شده کروی نیستند)؛ بعداً با این استثناها سروکار می‌دهیم.

هرچند روابط زیرکلاس و عضویت مهم‌ترین روابط برای رده‌ها هستند، همچنین می‌خواهیم بتوانیم روابط میان رده‌هایی که زیرکلاس یکدیگر نیستند بیان کنیم. برای نمونه، اگر فقط بگوییم Undergraduates و GraduateStudents زیرکلاس‌های Students هستند، نگفته‌ایم دانشجوی کارشناسی نمی‌تواند هم‌زمان دانشجوی تحصیلات تکمیلی باشد. می‌گوییم دو یا چند رده «ناهم‌پوش» (Disjoint) هستند اگر هیچ عضوی مشترک نداشته باشند. همچنین ممکن است بخواهیم بگوییم رده‌های «کارشناسی» و «تحصیلات تکمیلی» «تجزیهٔ جامع» (Exhaustive Decomposition) دانشجویان دانشگاه را تشکیل می‌دهند. تجزیهٔ جامعِ مجموعه‌های ناهم‌پوش «افراز» (Partition) نامیده می‌شود. چند مثال دیگر از این سه مفهوم:

```
Disjoint({Animals, Vegetables})
ExhaustiveDecomposition({Americans, Canadians, Mexicans}, NorthAmericans)
Partition({Animals, Plants, Fungi, Protista, Monera}, LivingThings)
```

(توجه کنید ExhaustiveDecomposition آمریکای شمالی‌ها افراز نیست، زیرا برخی مردم تابعیت دوگانه دارند.) سه گزاره چنین تعریف می‌شوند:

```
Disjoint(s) ⇔ (∀c1,c2 c1 ∈ s ∧ c2 ∈ s ∧ c1 ≠ c2 ⇒ Intersection(c1,c2) = {})
ExhaustiveDecomposition(s,c) ⇔ (∀i i ∈ c ⇔ ∃c2 c2 ∈ s ∧ i ∈ c2)
Partition(s,c) ⇔ Disjoint(s) ∧ ExhaustiveDecomposition(s,c)
```

رده‌ها را همچنین می‌توان با ارائهٔ شرایط لازم و کافی برای عضویت تعریف کرد. برای نمونه، مجرد، مرد بالغ ازدواج‌نکرده است:

```
x ∈ Bachelors ⇔ Unmarried(x) ∧ x ∈ Adults ∧ x ∈ Males
```

چنان‌که در حاشیهٔ «انواع طبیعی» در صفحهٔ ۳۲۰ بحث می‌کنیم، تعریف‌های منطقی صلب و دقیق برای رده‌ها معمولاً فقط برای مفاهیم صوری مصنوعی ممکن است، نه برای اشیای معمولی. اما تعریف‌ها همیشه لازم نیستند.

۱۰.۲.۱ ترکیب فیزیکی

ایدهٔ اینکه شیئی می‌تواند بخشی از شیء دیگر باشد آشناست. بینیِ انسان بخشی از سر اوست، رومانی بخشی از اروپاست، و این فصل بخشی از این کتاب است. از رابطهٔ عمومی PartOf برای گفتن اینکه چیزی بخشی از چیز دیگر است استفاده می‌کنیم. اشیا را می‌توان در سلسله‌مراتب‌های PartOf گروه‌بندی کرد، یادآور سلسله‌مراتب Subset:

```
PartOf(Bucharest, Romania)
PartOf(Romania, EasternEurope)
PartOf(EasternEurope, Europe)
PartOf(Europe, Earth)
```

رابطهٔ PartOf متعدی و بازتابی است؛ یعنی:

```
PartOf(x,y) ∧ PartOf(y,z) ⇒ PartOf(x,z)
PartOf(x,x)
```

بنابراین، می‌توانیم نتیجه بگیریم PartOf(Bucharest, Earth). رده‌های اشیای مرکب (Composite Objects) اغلب با روابط ساختاری میان اجزا مشخص می‌شوند. برای نمونه، دودَپا شیئی است با دقیقاً دو پا متصل به بدنی:

```
Biped(a) ⇒ ∃l1,l2,b Leg(l1) ∧ Leg(l2) ∧ Body(b) ∧ PartOf(l1,a) ∧ PartOf(l2,a) ∧
PartOf(b,a) ∧ Attached(l1,b) ∧ Attached(l2,b) ∧ l1 ≠ l2 ∧ [∀l3 Leg(l3) ∧ PartOf(l3,a) ⇒
(l3 = l1 ∨ l3 = l2)]
```

نماد «دقیقاً دو» کمی ناشیانه است؛ مجبوریم بگوییم دو پا هست، یکسان نیستند، و اگر کسی پای سومی پیشنهاد کند، باید با یکی از دو پای دیگر یکسان باشد. در بخش ۱۰.۵.۲، صورت‌بندی به نام «منطق توصیف» (Description Logic) توصیف می‌کنیم که بازنمایی قیدهایی مانند «دقیقاً دو» را آسان‌تر می‌کند.

می‌توانیم رابطهٔ PartPartition را مشابه رابطهٔ Partition برای رده‌ها تعریف کنیم. (تمرین DECM.۱۰ را ببینید.) شیئی از اجزای PartPartition خودش ترکیب شده و می‌توان آن را برگرفتن برخی ویژگی‌ها از آن اجزا دید. برای نمونه، جرم شیء مرکب، مجموع جرم‌های اجزاست. توجه کنید این برای رده‌ها صادق نیست، که جرم ندارند، حتی اگر عناصرشان داشته باشند.

همچنین مفید است اشیای مرکب با اجزای معین اما بدون ساختار خاص تعریف کنیم. برای نمونه، ممکن است بخواهیم بگوییم «سیب‌های این کیسه دو پوند وزن دارند.» وسوسه این است که این وزن را به مجموعهٔ سیب‌های کیسه نسبت دهیم، اما این اشتباه است، زیرا مجموعه مفهوم انتزاعی ریاضی‌ای است که عناصر دارد اما وزن ندارد. در عوض، به مفهوم تازه‌ای نیاز داریم که «دسته» (Bunch) می‌نامیم. برای نمونه، اگر سیب‌ها Apple1، Apple2 و Apple3 باشند، آنگاه:

```
BunchOf({Apple1, Apple2, Apple3})
```

شیء مرکبی را نشان می‌دهد که سه سیب اجزای آن‌اند (نه عناصر). سپس می‌توانیم از دسته به‌عنوان شیء عادی، هرچند بدون‌ساختار، استفاده کنیم. توجه کنید x = BunchOf({x}). وانگهی، BunchOf(Apples) شیء مرکبی است شامل همهٔ سیب‌ها — که نباید با Apples، رده یا مجموعهٔ همهٔ سیب‌ها، اشتباه گرفته شود.

می‌توانیم BunchOf را بر حسب رابطهٔ PartOf تعریف کنیم. بدیهی است هر عنصر s بخشی از BunchOf(s) است:

```
∀x x ∈ s ⇒ PartOf(x, BunchOf(s))
```

وانگهی، BunchOf(s) کوچک‌ترین شیءِ ارضاکنندهٔ این شرط است. به بیان دیگر، BunchOf(s) باید بخشی از هر شیئی باشد که همهٔ عناصر s را به‌عنوان جزء دارد:

```
∀y [∀x x ∈ s ⇒ PartOf(x,y)] ⇒ PartOf(BunchOf(s), y)
```

این اصل‌موضوعه‌ها مثالی از فن عمومی به نام «کمینه‌سازی منطقی» (Logical Minimization) هستند، که یعنی تعریف شیء به‌عنوان کوچک‌ترین شیءِ ارضاکنندهٔ شرایط معین.

۱۰.۲.۲ اندازه‌گیری‌ها

در هر دو نظریهٔ علمی و عقل‌سلیمی جهان، اشیا ارتفاع، جرم، هزینه و مانند آن دارند. مقادیری که برای این ویژگی‌ها تخصیص می‌دهیم «اندازه»ها (Measures) نامیده می‌شوند. اندازه‌های کمی معمولی را به‌راحتی می‌توان بازنمایی کرد. تصور می‌کنیم جهان شامل «اشیای اندازه» انتزاعی است، مانند طولی که طول این پاره‌خط است. می‌توانیم این طول را ۱٫۵ اینچ یا ۳٫۸۱ سانتی‌متر بنامیم. پس، همان طول در زبان ما نام‌های متفاوتی دارد. طول را با «تابع واحد» (Units Function) بازنمایی می‌کنیم که عددی را آرگومان می‌گیرد. (بدیلی در تمرین ALTM.۱۰ کاوش شده.)

حاشیه: انواع طبیعی

برخی رده‌ها تعریف‌های صلب و دقیق دارند: شیئی مثلث است اگر و فقط اگر چندضلعی سه‌ضلعی باشد. از سوی دیگر، بیشتر رده‌های جهان واقعی تعریف روشنی ندارند؛ این‌ها «رده‌های نوع طبیعی» (Natural Kind Categories) نامیده می‌شوند. برای نمونه، گوجه‌فرنگی‌ها تمایل دارند قرمز تیره‌ی محو باشند؛ تقریباً کروی؛ با فرورفتگی در بالا جایی که ساقه بود؛ حدود دو تا چهار اینچ قطر؛ با پوستی نازک اما محکم؛ و با گوشت، دانه و آبمیوه درون. اما تغییر هست: برخی گوجه‌فرنگی‌ها زرد یا نارنجی‌اند، گوجه‌فرنگی‌های نارس سبزند، برخی کوچک‌تر یا بزرگ‌تر از میانگین‌اند و گوجه‌فرنگی‌های گیلاسی یکنواخت کوچک‌اند. به‌جای تعریف کامل گوجه‌فرنگی‌ها، مجموعه‌ای ویژگی داریم که برای شناسایی اشیایی که به‌وضوح گوجه‌فرنگی‌های نمونه‌اند خدمت می‌کند، اما ممکن است اشیای دیگر را قطعاً شناسایی نکند. (آیا می‌تواند گوجه‌فرنگی‌ای وجود داشته باشد که مانند هلو کرکی باشد؟)

این برای عامل منطقی مسئله ایجاد می‌کند. عامل نمی‌تواند مطمئن باشد شیئی که ادراک کرده گوجه‌فرنگی است، و حتی اگر مطمئن بود، نمی‌توانست مطمئن باشد این یکی کدام‌یک از ویژگی‌های گوجه‌فرنگی‌های نمونه را دارد. این مسئله پیامد اجتناب‌ناپذیر عمل در محیط‌های تا حدی قابل مشاهده است.

یک رویکرد مفید، جداکردن آنچه برای همهٔ مصداق‌های رده صادق است از آنچه فقط برای مصداق‌های نمونه صادق است. پس علاوه بر ردهٔ Tomatoes، ردهٔ Typical(Tomatoes) را نیز خواهیم داشت. در این‌جا، تابع Typical رده‌ای را به زیرکلاسی که فقط مصداق‌های نمونه را شامل می‌شود نگاشت می‌کند:

```
Typical(c) ⊆ c
```

بیشتر دانش دربارهٔ انواع طبیعی در واقع دربارهٔ مصداق‌های نمونه‌شان خواهد بود:

```
x ∈ Typical(Tomatoes) ⇒ Red(x) ∧ Round(x)
```

پس می‌توانیم واقعیت‌های مفیدی دربارهٔ رده‌ها بدون تعریف‌های دقیق بنویسیم. دشواری ارائهٔ تعریف‌های دقیق برای بیشتر رده‌های طبیعی با ژرفا توسط ویتگنشتاین (۱۹۵۳) توضیح داده شد. او از مثال بازی‌ها استفاده کرد تا نشان دهد اعضای یک رده «شباهت‌های خانوادگی» مشترک دارند نه ویژگی‌های لازم و کافی: چه تعریف صلبی شطرنج، قایم‌موشک، سولیتیر و توپ‌گیری را در بر می‌گیرد؟

سودمندی مفهوم تعریف صلب و دقیق همچنین توسط کواین (۱۹۵۳) به چالش کشیده شد. او اشاره کرد حتی تعریف «مجرد» به‌عنوان مرد بالغ ازدواج‌نکرده نیز مشکوک است؛ ممکن است کسی گزاره‌ای مانند «پاپ مجرد است» را زیر سؤال ببرد. هرچند به‌طور دقیق کاذب نیست، این کاربرد قطعاً ناخوشایند است، زیرا استنتاج‌های ناخواسته‌ای در شنونده برمی‌انگیزد. تنش را شاید بتوان با تمایز میان تعریف‌های منطقیِ مناسب برای بازنمایی دانش درونی و معیارهای دقیق‌ترِ کاربرد زبانیِ خوشایند حل کرد. دومی را شاید بتوان با «پالایش» ادعاهای مشتق‌شده از اولی به دست آورد. همچنین ممکن است شکست‌های کاربرد زبانی به‌عنوان بازخوردی برای اصلاح تعریف‌های درونی خدمت کنند، چنان‌که پالایش غیرضروری شود.

اگر پاره‌خط L1 نامیده شود، می‌توانیم بنویسیم:

```
Length(L1) = Inches(1.5) = Centimeters(3.81)
```

تبدیل میان واحدها با معادل‌کردن مضارب یک واحد با واحد دیگر انجام می‌شود:

```
Centimeters(2.54 × d) = Inches(d)
```

اصل‌موضوعه‌های مشابهی را می‌توان برای پوند و کیلوگرم، ثانیه و روز، و دلار و سنت نوشت. اندازه‌ها را می‌توان برای توصیف اشیا چنین استفاده کرد:

```
Diameter(Basketball12) = Inches(9.5)
ListPrice(Basketball12) = $(19)
Weight(BunchOf({Apple1, Apple2, Apple3})) = Pounds(2)
d ∈ Days ⇒ Duration(d) = Hours(24)
```

توجه کنید $(1) اسکناس دلار نیست — قیمت است. می‌توان دو اسکناس دلار داشت، اما فقط یک شیء به نام $(1) هست. همچنین توجه کنید، در حالی که Inches(0) و Centimeters(0) به همان طول صفر اشاره می‌کنند، با دیگر اندازه‌های صفر، مانند Seconds(0)، یکسان نیستند.

اندازه‌های کمی ساده به‌راحتی قابل بازنمایی‌اند. اندازه‌های دیگر مسئلهٔ بیشتری ایجاد می‌کنند، زیرا مقیاس توافقی مقادیر ندارند. تمرین‌ها دشواری دارند، دسرها خوشمزگی دارند و شعرها زیبایی، اما نمی‌توان اعداد را به این کیفیات تخصیص داد. ممکن است کسی، در لحظه‌ای حساب‌داری ناب، چنین ویژگی‌هایی را برای هدف استدلال منطقی بی‌کاربرد کنار بگذارد؛ یا بدتر، بکوشد مقیاس عددی بر زیبایی تحمیل کند. این اشتباه فاحشی می‌بود، زیرا غیرضروری است. مهم‌ترین جنبهٔ اندازه‌ها، مقادیر عددی خاص نیست، بلکه واقعیتِ قابل‌ترتیب‌بودنِ اندازه‌هاست.

هرچند اندازه‌ها عدد نیستند، باز هم می‌توانیم آن‌ها را مقایسه کنیم، با نماد ترتیبی مانند >. برای نمونه، شاید باور داشته باشیم تمرین‌های نورویگ از تمرین‌های راسل سخت‌ترند، و در تمرین‌های سخت‌تر امتیاز کمتری می‌گیریم:

```
e1 ∈ Exercises ∧ e2 ∈ Exercises ∧ Wrote(Norvig, e1) ∧ Wrote(Russell, e2) ⇒ Difficulty(e1)
                                                                          > Difficulty(e2)
   e1 ∈ Exercises ∧ e2 ∈ Exercises ∧ Difficulty(e1) > Difficulty(e2) ⇒ ExpectedScore(e1) <
                                                                        ExpectedScore(e2)
```

این برای اجازه دادن به فرد در تصمیم‌گیری اینکه کدام تمرین‌ها را انجام دهد کافی است، حتی با آنکه هرگز از مقادیر عددی برای دشواری استفاده نشد. (البته باید کشف کرد چه کسی کدام تمرین‌ها را نوشته.) این نوع روابط یکنواخت میان اندازه‌ها مبنای حوزهٔ «فیزیک کیفی» را تشکیل می‌دهد، زیرحوزه‌ای از هوش مصنوعی که بررسی می‌کند چگونه دربارهٔ سامانه‌های فیزیکی بدون شیرجه در معادلات مفصل و شبیه‌سازی‌های عددی استدلال کنیم. فیزیک کیفی در یادداشت‌های تاریخی بحث شده.

۱۰.۲.۳ اشیا: چیزها و مواد

جهان واقعی را می‌توان شامل اشیای اولیه (مثلاً ذرات اتمی) و اشیای مرکبِ ساخته‌شده از آن‌ها دید. با استدلال در سطح اشیای بزرگ مانند سیب‌ها و خودروها، می‌توانیم بر پیچیدگیِ سروکار داشتن با شمار عظیم اشیای اولیه به‌صورت فردی غلبه کنیم. اما بخش قابل توجهی از واقعیت هست که به نظر می‌رسد هر فردسازیِ بدیهی — تقسیم به اشیای متمایز — را به چالش می‌کشد. به این بخش نام عمومی «ماده» (Stuff) می‌دهیم. برای نمونه، فرض کنیم جلوی من مقداری کره و یک مورچه‌خوار هست. می‌توانم بگویم یک مورچه‌خوار هست، اما هیچ شمار بدیهی‌ای از «شیء-کره»ها نیست، زیرا هر جزء از شیء-کره نیز شیء-کره است، دست‌کم تا وقتی به اجزای واقعاً بسیار کوچک برسیم. این تمایز عمده میان ماده و چیز است. اگر مورچه‌خوار را از وسط ببریم، دو مورچه‌خوار نمی‌گیریم (متأسفانه).

زبان انگلیسی به‌وضوح میان ماده و چیز تمایز می‌گذارد. می‌گوییم «a aardvark» (یک مورچه‌خوار)، اما، به‌جز در رستوران‌های متظاهر کالیفرنیا، نمی‌توان گفت «a butter» (یک کره). زبان‌شناسان میان «اسم‌های شمارشی» (Count Nouns)، مانند مورچه‌خوارها، حفره‌ها و قضیه‌ها، و «اسم‌های جرمی» (Mass Nouns)، مانند کره، آب و انرژی، تمایز می‌گذارند. چند هستی‌شناسی رقیب ادعای مدیریت این تمایز را دارند. در این‌جا فقط یکی را توصیف می‌کنیم؛ بقیه در یادداشت‌های تاریخی پوشش داده شده‌اند.

برای بازنمایی درست ماده، با امر بدیهی آغاز می‌کنیم. باید دست‌کم «توده»های درشت ماده‌ای که با آن‌ها تعامل می‌کنیم را به‌عنوان اشیا در هستی‌شناسی‌مان داشته باشیم. برای نمونه، ممکن است تودهٔ کره‌ای را بشناسیم که شب قبل روی میز مانده؛ ممکن است آن را برداریم، وزن کنیم، بفروشیم یا هر چه. در این معانی، شیئی است دقیقاً مانند مورچه‌خوار. آن را Butter3 بنامیم. همچنین ردهٔ Butter را تعریف می‌کنیم. به‌صورت غیررسمی، عناصر آن همهٔ چیزهایی خواهند بود که می‌توان درباره‌شان گفت «کره است»، شامل Butter3. با چند ملاحظه دربارهٔ اجزای بسیار کوچک که فعلاً حذف می‌کنیم، هر جزء از شیء-کره نیز شیء-کره است:

```
b ∈ Butter ∧ PartOf(p,b) ⇒ p ∈ Butter
```

حال می‌توانیم بگوییم کره حدود ۳۰ درجه سانتی‌گراد ذوب می‌شود:

```
b ∈ Butter ⇒ MeltingPoint(b, Centigrade(30))
```

می‌توانستیم ادامه دهیم و بگوییم کره زرد است، از آب کم‌چگالی‌تر است، در دمای اتاق نرم است، چربی بالایی دارد و الی آخر. از سوی دیگر، کره اندازه، شکل یا وزن خاصی ندارد. می‌توانیم رده‌های تخصصی‌تر کره مانند UnsaltedButter تعریف کنیم که آن نیز نوعی ماده است. توجه کنید ردهٔ PoundOfButter، که همهٔ شیء-کره‌های یک‌پوندی را به‌عنوان عضو شامل می‌شود، نوعی ماده نیست. اگر یک پوند کره را از وسط ببریم، افسوس، دو پوند کره نمی‌گیریم.

آنچه واقعاً رخ می‌دهد این است: برخی ویژگی‌ها «ذاتی» (Intrinsic) هستند: آن‌ها به خودِ جوهر شیء تعلق دارند، نه به شیء به‌عنوان کل. وقتی مصداقی از ماده را از وسط می‌برید، دو تکه ویژگی‌های ذاتی — چیزهایی مانند چگالی، نقطهٔ جوش، طعم، رنگ، مالکیت و مانند آن — را حفظ می‌کنند. از سوی دیگر، ویژگی‌های «عَرَضی» (Extrinsic) آن‌ها — وزن، طول، شکل و مانند آن — زیر تقسیم حفظ نمی‌شوند. رده‌ای از اشیا که در تعریفش فقط ویژگی‌های ذاتی را شامل شود، پس «ماده» یا اسم جرمی است؛ رده‌ای که هر ویژگی عَرَضی را در تعریفش شامل شود، اسم شمارشی است. Stuff و Thing به‌ترتیب عمومی‌ترین رده‌های ماده و شیء هستند.

۱۰.۳ رویدادها

در بخش ۷.۷.۱ دربارهٔ کنش‌ها بحث کردیم: چیزهایی که رخ می‌دهند، مانند Shoott؛ و روان‌ها: جنبه‌هایی از جهان که تغییر می‌کنند، مانند HaveArrowt. هر دو به‌صورت گزاره بازنمایی شدند و از اصل‌موضوعه‌های حالت-جانشین استفاده کردیم تا بگوییم روانی در زمان t+1 صادق خواهد بود اگر کنش در زمان t باعث صادق‌شدنش شده باشد، یا اگر از قبل در زمان t صادق بوده و کنش باعث کاذب‌شدنش نشده باشد. آن برای جهانی بود که در آن کنش‌ها گسسته، آنی، یکی‌دریک‌زمان و بدون تغییر در نحوهٔ اجراشان هستند (یعنی فقط یک نوع کنش Shoot هست، تمایزی میان سریع، آهسته، عصبی و... شلیک‌کردن نیست).

اما با حرکت از حوزه‌های ساده‌لوحانه به جهان واقعی، گسترهٔ بسیار غنی‌تری از کنش‌ها یا رویدادها[3] برای سروکار داشتن هست. کنش پیوسته‌ای مانند پرکردن وان حمام را در نظر بگیرید. اصل‌موضوعهٔ حالت-جانشین می‌تواند بگوید وان پیش از کنش خالی و هنگام تمام‌شدن کنش پر است، اما نمی‌تواند دربارهٔ آنچه در طول کنش رخ می‌دهد صحبت کند. همچنین نمی‌تواند به‌راحتی دو کنشِ هم‌زمان را توصیف کند — مانند مسواک‌زدن در حالی که منتظر پرشدن وان هستید. برای مدیریت چنین مواردی، رویکردی به نام «حساب رویدادها» (Event Calculus) معرفی می‌کنیم.

اشیای حساب رویدادها رویدادها، روان‌ها و نقطه‌های زمانی‌اند. At(Shankar, Berkeley) روانی است: شیئی که به واقعیتِ بودنِ شانکار در برکلی اشاره می‌کند. رویداد E1 پرواز شانکار از سانفرانسیسکو به واشینگتن دی.سی. چنین توصیف می‌شود:

```
E1 ∈ Flyings ∧ Flyer(E1, Shankar) ∧ Origin(E1, SF) ∧ Destination(E1, DC)
```

که Flyings ردهٔ همهٔ رویدادهای پرواز است. با شیءسازی رویدادها، افزودن هر مقدار اطلاعات دلخواه دربارهٔ آن‌ها ممکن می‌شود. برای نمونه، می‌توانیم بگوییم پرواز شانکار پرتلاطم بود با Bumpy(E1). در هستی‌شناسی‌ای که رویدادها گزاره‌های n-تایی‌اند، هیچ راهی برای افزودن اطلاعات اضافی مانند این نیست؛ رفتن به گزارهٔ n+1-تایی راه‌حل مقیاس‌پذیری نیست.

برای ادعای اینکه روانی واقعاً از نقطهٔ زمانی t1 شروع و تا زمان t2 ادامه می‌یابد، از گزارهٔ T استفاده می‌کنیم، مانند T(At(Shankar,Berkeley), t1, t2). به همین ترتیب، از Happens(E1, t1, t2) استفاده می‌کنیم تا بگوییم رویداد E1 واقعاً رخ داد، از زمان t1 شروع و در زمان t2 پایان یافت. مجموعهٔ کامل گزاره‌ها برای یک نسخه از حساب رویدادها[4] چنین است:

گزاره	معنی
T(f, t1, t2)	روان f برای همهٔ زمان‌های بین t1 و t2 صادق است
Happens(e, t1, t2)	رویداد e در t1 شروع و در t2 پایان می‌یابد
Initiates(e, f, t)	رویداد e باعث صادق‌شدن روان f در زمان t می‌شود
Terminates(e, f, t)	رویداد e باعث ازبین‌رفتن صدق روان f در زمان t می‌شود
Initiated(f, t1, t2)	روان f در نقطه‌ای بین t1 و t2 صادق می‌شود
Terminated(f, t1, t2)	روان f در نقطه‌ای بین t1 و t2 از صدق می‌افتد
t1 < t2	نقطهٔ زمانی t1 پیش از نقطهٔ زمانی t2 رخ می‌دهد

می‌توانیم اثرات رویداد پرواز را توصیف کنیم:

```
E = Flyings(a, here, there) ∧ Happens(E, t1, t2) ⇒ Terminates(E, At(a, here), t1) ∧
                                                   Initiates(E, At(a, there), t2)
```

رویداد متمایزی به نام Start فرض می‌کنیم که حالت آغازین را با گفتن کدام روان‌ها صادق‌اند (با Initiates) یا کاذب‌اند (با Terminated) در زمان آغاز توصیف می‌کند. سپس می‌توانیم توصیف کنیم چه روان‌هایی در چه نقطه‌های زمانی صادق‌اند با جفت اصل‌موضوعه برای T و ¬T که همان قالب عمومی اصل‌موضوعه‌های حالت-جانشین را دنبال می‌کنند: فرض کن رویدادی بین زمان t1 و t3 رخ می‌دهد، و در t2 جایی در آن بازهٔ زمانی رویداد مقدار روان f را تغییر می‌دهد، یا آن را آغاز می‌کند (صادق می‌کند) یا خاتمه می‌دهد (کاذب می‌کند). آنگاه در زمان t4 در آینده، اگر هیچ رویداد مداخله‌گر دیگری روان را تغییر نداده باشد (به‌ترتیب خاتمه یا آغازش نکرده باشد)، روان مقدارش را حفظ کرده است. صوری، اصل‌موضوعه‌ها چنین‌اند:

```
Happens(e, t1, t3) ∧ Initiates(e, f, t2) ∧ ¬Terminated(f, t2, t4) ∧ t1 ≤ t2 ≤ t3 ≤ t4 ⇒
                                                                    T(f, t2, t4)
 Happens(e, t1, t3) ∧ Terminates(e, f, t2) ∧ ¬Initiated(f, t2, t4) ∧ t1 ≤ t2 ≤ t3 ≤ t4 ⇒
                                                                   ¬T(f, t2, t4)
```

که Terminated و Initiated چنین تعریف می‌شوند:

```
Terminated(f, t1, t5) ⇔ ∃e,t2,t3,t4 Happens(e, t2, t4) ∧ Terminates(e, f, t3) ∧ t1 ≤ t2 ≤
                                                                              t3 ≤ t4 ≤ t5
Initiated(f, t1, t5) ⇔ ∃e,t2,t3,t4 Happens(e, t2, t4) ∧ Initiates(e, f, t3) ∧ t1 ≤ t2 ≤ t3
                                                                                 ≤ t4 ≤ t5
```

می‌توانیم حساب رویدادها را برای بازنمایی رویدادهای هم‌زمان (مانند نیاز دو نفر به سوارشدن بر الاکلنگ)، رویدادهای برون‌زا (مانند حرکت‌دادن شیء توسط باد)، رویدادهای پیوسته (مانند بالا آمدن جزر)، رویدادهای ناجبرگرایانه (مانند انداختن سکه و روآمدن شیر یا خط)، و دیگر پیچیدگی‌ها بسط دهیم.

۱۰.۳.۱ زمان

حساب رویدادها ما را به امکان صحبت دربارهٔ نقطه‌های زمانی و بازه‌های زمانی باز می‌کند. دو نوع بازهٔ زمانی را در نظر خواهیم گرفت: لحظه‌ها و بازه‌های بسط‌یافته. تمایز آن است که فقط لحظه‌ها مدت صفر دارند:

```
                                    Partition({Moments, ExtendedIntervals}, Intervals)
                                                  i ∈ Moments ⇔ Duration(i) = Seconds(0)
```

سپس مقیاس زمانی می‌سازیم و نقطه‌هایی روی آن مقیاس را با لحظه‌ها مرتبط می‌کنیم و زمان‌های مطلق به ما می‌دهد. مقیاس زمانی دلبخواهی است؛ آن را بر حسب ثانیه اندازه می‌گیریم و می‌گوییم لحظهٔ نیمه‌شب (GMT) اول ژانویهٔ ۱۹۰۰ زمان ۰ دارد. توابع Begin و End اولین و آخرین لحظه‌های یک بازه را برمی‌گزینند و تابع Time نقطهٔ روی مقیاس زمانی را برای یک لحظه تحویل می‌دهد. تابع Duration تفاوت میان زمان پایان و زمان آغاز را می‌دهد.

```
                                  Interval(i) ⇒ Duration(i) = (Time(End(i)) − Time(Begin(i)))
                                                          Time(Begin(AD1900)) = Seconds(0)
                                                     Time(Begin(AD2001)) = Seconds(3187324800)
                                                       Time(End(AD2001)) = Seconds(3218860800)
                                                          Duration(AD2001) = Seconds(31536000)
```

برای خواندنی‌ترشدن این اعداد، تابع Date را نیز معرفی می‌کنیم که شش آرگومان می‌گیرد (ساعت، دقیقه، ثانیه، روز، ماه و سال) و نقطهٔ زمانی بازمی‌گرداند:

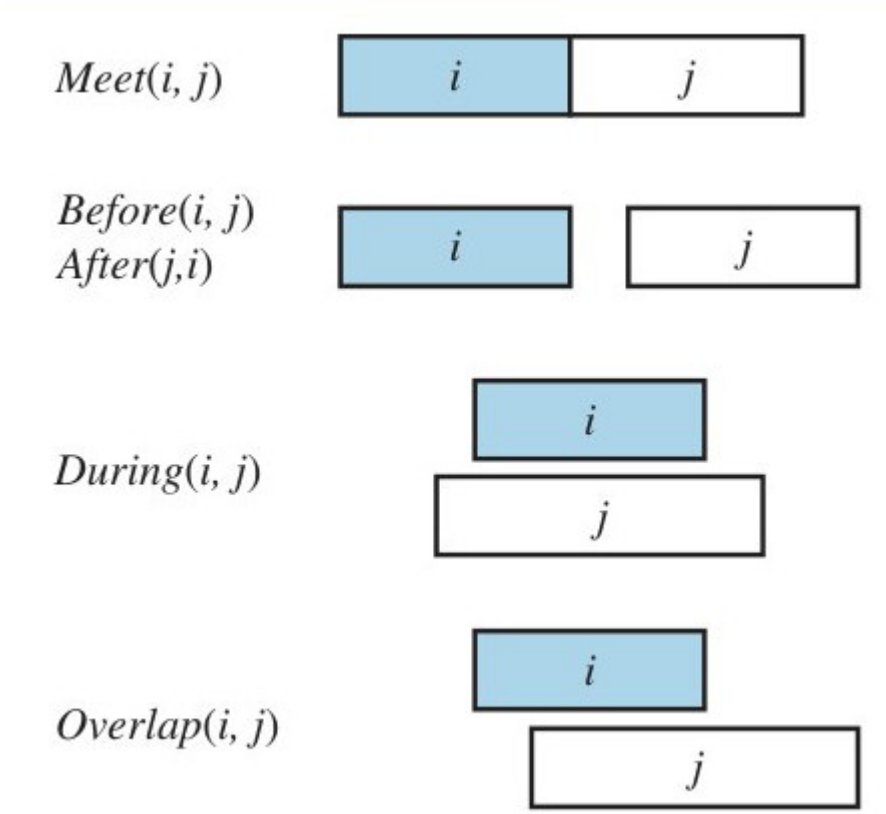
```
                                                 Time(Begin(AD2001)) = Date(0,0,0,1,Jan,2001)
                                                Date(0,20,21,24,1,1995) = Seconds(3000000000)
```

دو بازه «به هم می‌رسند» (Meet) اگر زمان پایان اولی برابر زمان آغاز دومی باشد. مجموعهٔ کامل روابط بازه‌ای (آلن، ۱۹۸۳) در زیر و در شکل ۱۰.۲ نشان داده شده:

رابطه	تعریف
Meet(i, j)	End(i) = Begin(j) ⇔
Before(i, j)	End(i) < Begin(j) ⇔
After(j, i)	Before(i, j) ⇔
During(i, j)	Begin(j) < Begin(i) < End(i) < End(j) ⇔
Overlap(i, j)	Begin(i) < Begin(j) < End(i) < End(j) ⇔
Starts(i, j)	Begin(i) = Begin(j) ⇔
Finishes(i, j)	End(i) = End(j) ⇔
Equals(i, j)	Begin(i) = Begin(j) ∧ End(i) = End(j) ⇔

همهٔ این‌ها معنای شهودی خود را دارند، به‌جز Overlap: ما تمایل داریم هم‌پوشانی را متقارن بدانیم (اگر i با j هم‌پوشانی دارد پس j با i)، اما در این تعریف، Overlap(i, j) فقط اگر i پیش از j آغاز شود صادق است. تجربه نشان داده این تعریف برای نوشتن اصل‌موضوعه‌ها مفیدتر است. برای گفتن اینکه پادشاهی الیزابت دوم بلافاصله پس از پادشاهی جورج ششم آمد و پادشاهی الویس با دههٔ ۱۹۵۰ هم‌پوشانی داشت، می‌توانیم بنویسیم:

```
Meets(ReignOf(GeorgeVI), ReignOf(ElizabethII))
Overlap(Fifties, ReignOf(Elvis))
Begin(Fifties) = Begin(AD1950)
End(Fifties) = End(AD1959)
```



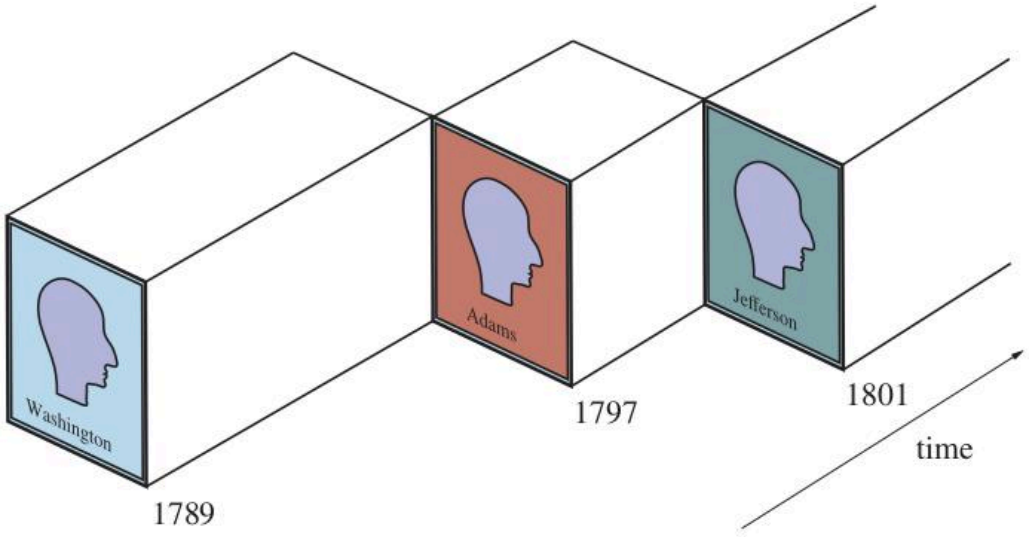
شکل ۱۰.۲ – گزاره‌های روی بازه‌های زمانی.

۱۰.۳.۲ روان‌ها و اشیا

اشیای فیزیکی را می‌توان رویدادهای تعمیم‌یافته دید، به این معنا که شیء فیزیکی تکه‌ای از فضا–زمان است. برای نمونه، USA را می‌توان رویدادی دانست که در ۱۷۷۶ به‌عنوان اتحاد ۱۳ ایالت آغاز شد و امروز به‌عنوان اتحاد ۵۰ ایالت هنوز در جریان است. می‌توانیم ویژگی‌های در حال تغییر USA را با روان‌های حالت، مانند Population(USA)، توصیف کنیم. ویژگی USA که هر چهار یا هشت سال، به‌جز مصیبت‌ها، تغییر می‌کند، رئیس‌جمهورش است. ممکن است پیشنهاد شود President(USA) عبارت منطقی‌ای است که در زمان‌های مختلف شیء متفاوتی را نشان می‌دهد. متأسفانه، این ممکن نیست، زیرا عبارت در ساختار مدلِ داده‌شده دقیقاً یک شیء را نشان می‌دهد. (عبارت President(USA,t) می‌تواند اشیای متفاوتی را نشان دهد، بسته به مقدار t، اما هستی‌شناسی ما شاخص‌های زمانی را از روان‌ها جدا نگه می‌دارد.) تنها امکان آن است که President(USA) شیء تکی‌ای را نشان دهد که در زمان‌های مختلف از افراد متفاوت تشکیل شده. این شیئی است که از ۱۷۸۹ تا ۱۷۹۷ جورج واشینگتن است، از ۱۷۹۷ تا ۱۸۰۱ جان آدامز، و الی آخر، چنان‌که در شکل ۱۰.۳. برای گفتن اینکه جورج واشینگتن در تمام ۱۷۹۰ رئیس‌جمهور بود، می‌توانیم بنویسیم:

```
T(Equals(President(USA), GeorgeWashington), Begin(AD1790), End(AD1790))
```

از نماد تابع Equals به‌جای گزارهٔ منطقی استاندارد = استفاده می‌کنیم، زیرا نمی‌توانیم گزاره‌ای به‌عنوان آرگومان T داشته باشیم، و به‌سبب آنکه تفسیر این نیست که GeorgeWashington و President(USA) در ۱۷۹۰ به‌صورت منطقی یکسان‌اند؛ هویت منطقی چیزی نیست که در طول زمان تغییر کند. هویت میان زیررویدادهای اشیای President(USA) و GeorgeWashington است که توسط دورهٔ ۱۷۹۰ تعریف می‌شوند.



شکل ۱۰.۳ — نمای شماتیک شیء President(USA) برای سال‌های آغازین.

۱۰.۴ اشیای ذهنی و منطق موجهات

عامل‌هایی که تاکنون ساخته‌ایم باورهایی دارند و می‌توانند باورهای تازه استنتاج کنند. اما هیچ‌کدام دانشی دربارهٔ باورها یا دربارهٔ استنتاج ندارد. دانش دربارهٔ دانش و فرایندهای استدلال خود برای کنترل استنتاج مفید است. برای نمونه، فرض کنیم آلیس می‌پرسد «جذر ۱۷۶۴ چیست» و باب پاسخ می‌دهد «نمی‌دانم.» اگر آلیس اصرار کند «بیشتر فکر کن»، باب باید دریابد با کمی فکر بیشتر، این پرسش در واقع قابل پاسخ است. از سوی دیگر، اگر پرسش «آیا رئیس‌جمهور الان نشسته است؟» بود، باب باید دریابد فکر بیشتر احتمالاً کمکی نمی‌کند. دانش دربارهٔ دانش عامل‌های دیگر نیز مهم است؛ باب باید دریابد رئیس‌جمهور می‌داند.

آنچه نیاز داریم مدلی از اشیای ذهنیِ درون سر کسی (یا پایگاه دانش چیزی) و فرایندهای ذهنیِ دست‌کاری‌کنندهٔ آن اشیای ذهنی است. مدل لازم نیست مفصل باشد. لازم نیست بتوانیم پیش‌بینی کنیم چند میلی‌ثانیه طول می‌کشد تا عاملِ خاصی استنتاجی انجام دهد. خوشحال خواهیم بود فقط بتوانیم نتیجه بگیریم مادر می‌داند نشسته است یا نه.

با «نگرش‌های گزاره‌ای» (Propositional Attitudes) آغاز می‌کنیم که عامل می‌تواند نسبت به اشیای ذهنی داشته باشد: نگرش‌هایی مانند Wants، Knows، Believes و Informs. دشواری آن است که این نگرش‌ها مانند گزاره‌های «عادی» رفتار نمی‌کنند. برای نمونه، فرض کنیم بکوشیم ادعا کنیم لوییز می‌داند سوپرمن می‌تواند پرواز کند:

```
Knows(Lois, CanFly(Superman))
```

یک مسئلهٔ جزئی این است که ما معمولاً CanFly(Superman) را جمله می‌دانیم، اما این‌جا به‌صورت عبارت ظاهر می‌شود. آن مسئله را می‌توان با شیءسازی CanFly(Superman) وصله کرد؛ آن را روان کردن. مسئلهٔ جدی‌تر این است که اگر صادق باشد سوپرمن همان کلارک کنت است، پس باید نتیجه بگیریم لوییز می‌داند کلارک می‌تواند پرواز کند، که نادرست است، زیرا (در بیشتر نسخه‌های داستان) لوییز نمی‌داند کلارک سوپرمن است.

```
Knows(Lois, CanFly(Superman)) ⊨ Knows(Lois, CanFly(Clark)) ∧ (Superman = Clark)
```

این پیامد این واقعیت است که استدلال برابری در منطق ساخته شده. معمولاً این چیز خوبی است؛ اگر عامل ما بداند 2+2 = 4 و 4 < 5، می‌خواهیم عامل بداند 2+2 < 5. این ویژگی «شفافیت ارجاعی» (Referential Transparency) نامیده می‌شود — مهم نیست منطق برای اشاره به شیء از چه عبارتی استفاده می‌کند، مهم شیئی است که عبارت نام می‌برد. اما برای نگرش‌های گزاره‌ای مانند باور و دانستن، می‌خواهیم «تیرگی ارجاعی» (Referential Opacity) داشته باشیم — عبارت‌های استفاده‌شده مهم‌اند، زیرا همهٔ عامل‌ها نمی‌دانند کدام عبارت‌ها هم‌ارجاع‌اند.

می‌توانیم این را با شیءسازی بیشتر وصله کنیم: می‌توانستیم شیئی برای بازنمایی کلارک/سوپرمن داشته باشیم، شیء دیگری برای بازنمایی شخصی که لوییز به‌عنوان کلارک می‌شناسد، و سومی برای شخصی که لوییز به‌عنوان سوپرمن می‌شناسد. اما این تکثیر اشیا بدان معناست جمله‌هایی که می‌خواهیم بنویسیم به‌سرعت پرگویی و دست‌وپاگیر می‌شوند.

«منطق موجهات» (Modal Logic) برای پرداختن به این مسئله طراحی شده. منطق معمولی با «وجه» تکی، وجه صدق، سروکار دارد و به ما امکان می‌دهد «P صادق است» یا «P کاذب است» را بیان کنیم. منطق موجهات «عملگرهای موجهاتی» (Modal Operators) خاصی را شامل می‌شود که جمله‌ها (نه عبارت‌ها) را آرگومان می‌گیرند. برای نمونه، «A می‌داند P» با نماد K_A P بازنمایی می‌شود، که K عملگر موجهاتی دانستن است. دو آرگومان می‌گیرد، یک عامل (که زیرنویس نوشته می‌شود) و یک جمله. نحو منطق موجهات همان منطق مرتبهٔ اول است، به‌جز آنکه جمله‌ها را می‌توان با عملگرهای موجهاتی نیز ساخت.

معناشناسی منطق موجهات پیچیده‌تر است. در منطق مرتبهٔ اول مدل شامل مجموعه‌ای از اشیا و تفسیری است که هر نام را به شیء، رابطه یا تابع مناسب نگاشت می‌کند. در منطق موجهات می‌خواهیم هم امکان اینکه هویت پنهان سوپرمن کلارک است و هم امکان اینکه نیست را در نظر بگیریم.

پس به مدل پیچیده‌تری نیاز خواهیم داشت، مدلی که از مجموعه‌ای از «جهان‌های ممکن» (Possible Worlds) تشکیل شده نه فقط یک جهان صادق. جهان‌ها در گرافی با «روابط دسترسی‌پذیری» (Accessibility Relations) به هم متصل‌اند، برای هر عملگر موجهاتی یک رابطه. می‌گوییم جهان w1 از جهان w0 نسبت به عملگر موجهاتی K_A دسترسی‌پذیر است اگر همه‌چیز در w1 با آنچه A در w0 می‌داند سازگار باشد. به‌عنوان مثال، در جهان واقعی، بخارست پایتخت رومانی است، اما برای عاملی که آن را نمی‌دانست، جهانی که پایتخت رومانی در آن مثلاً صوفیه است دسترسی‌پذیر است. امیدواریم جهانی که 2+2 = 5 در آن صادق است برای هیچ عاملی دسترسی‌پذیر نباشد.

در حالت کلی، اتم دانش $K_A\ P$ در جهان w صادق است اگر و فقط اگر P در هر جهانِ دسترسی‌پذیر از w صادق باشد. صدق جمله‌های پیچیده‌تر با اعمال بازگشتی این قاعده و قواعد معمول منطق مرتبهٔ اول مشتق می‌شود. این بدان معناست منطق موجهات را می‌توان برای استدلال دربارهٔ جمله‌های دانشِ تودرتو استفاده کرد: آنچه یک عامل دربارهٔ دانش عامل دیگر می‌داند. برای نمونه، می‌توانیم بگوییم هرچند لوییز نمی‌داند آیا هویت پنهان سوپرمن کلارک کنت است، می‌داند کلارک می‌داند:

```
K_Lois [K_Clark Identity(Superman, Clark) ∨ K_Clark ¬Identity(Superman, Clark)]
```

منطق موجهات برخی مسائل ظریفِ تعامل سورها و دانش را حل می‌کند. جملهٔ انگلیسی «باند می‌داند کسی جاسوس است» مبهم است. خوانش نخست آن است که شخص خاصی هست که باند می‌داند جاسوس است؛ می‌توانیم این را چنین بنویسیم:

```
∃x K_Bond Spy(x)
```

که در منطق موجهات بدان معناست xای هست که، در همهٔ جهان‌های دسترسی‌پذیر، باند می‌داند جاسوس است. خوانش دوم آن است که باند فقط می‌داند دست‌کم یک جاسوس هست:

```
K_Bond ∃x Spy(x)
```

تفسیر منطق موجهات آن است که در هر جهان دسترسی‌پذیر xای هست که جاسوس است، اما لازم نیست در هر جهان همان x باشد.

حال که عملگر موجهاتی دانش داریم، می‌توانیم اصل‌موضوعه‌هایی برایش بنویسیم. نخست، می‌توانیم بگوییم عامل‌ها قادر به نتیجه‌گیری‌اند؛ اگر عاملی P را بداند و بداند P دلالت بر Q دارد، آنگاه عامل Q را می‌داند:

```
K_a Q ⇒ (K_a P ∧ K_a(P ⇒ Q))
```

از این (و چند قاعدهٔ دیگر دربارهٔ هویت‌های منطقی) می‌توانیم ثابت کنیم $K_a(P \vee \neg P)$ تاتولوژی است؛ هر عامل می‌داند هر گزارهٔ P یا صادق است یا کاذب. از سوی دیگر، $(K_a\ P) \vee (K_a\ \neg P)$ تاتولوژی نیست؛ در حالت کلی، گزاره‌های زیادی هست که عامل نمی‌داند صادق‌اند و نمی‌داند کاذب‌اند.

گفته می‌شود (به افلاطون بازمی‌گردد) دانش، باور صادق موجّه است. یعنی اگر صادق باشد، باورش داشته باشید، و دلیل غیرقابل‌خدشه‌ای داشته باشید، آنگاه آن را می‌دانید. این بدان معناست اگر چیزی را بدانید، باید صادق باشد، و اصل‌موضوعه را داریم:

```
K_a P ⇒ P
```

وانگهی، عامل‌های منطقی (اما نه همهٔ مردم) قادر به درون‌نگری دانش خودشان‌اند. اگر چیزی را بدانند، می‌دانند که می‌دانند:

```
K_a P ⇒ K_a(K_a P)
```

می‌توانیم اصل‌موضوعه‌های مشابهی برای باور (اغلب با B نشان داده می‌شود) و دیگر وجه‌ها تعریف کنیم. اما یکی از مشکلات رویکرد منطق موجهات آن است که «دانایی منطقی» (Logical Omniscience) را از جانب عامل‌ها فرض می‌کند. یعنی اگر عاملی مجموعه‌ای از اصل‌موضوعه‌ها را بداند، همهٔ پیامدهای آن اصل‌موضوعه‌ها را نیز می‌داند. این حتی برای مفهوم نسبتاً انتزاعی دانش بر پایه‌ای لرزان است، اما برای باور بدتر به نظر می‌رسد، زیرا باور مفهوم بیشتری از اشاره به چیزهایی دارد که به‌صورت فیزیکی در عامل بازنمایی شده‌اند، نه فقط بالقوه قابل اشتقاق.

تلاش‌هایی برای تعریف شکلی از عقلانیت محدود برای عامل‌ها شده — برای گفتن اینکه عامل‌ها فقط آن ادعاهایی را باور دارند که با اعمال حداکثر k گام استدلال، یا حداکثر s ثانیه محاسبه، قابل اشتقاق‌اند. این تلاش‌ها عموماً ناراضی‌کننده بوده‌اند.

۱۰.۴.۱ دیگر منطق‌های موجهاتی

منطق‌های موجهاتی فراوانی برای وجه‌های متفاوت غیر از دانش پیشنهاد شده. پیشنهادی افزودن عملگرهای موجهاتی برای امکان و ضرورت است: «ممکن است صادق باشد که یکی از نویسندگان این کتاب الان نشسته است» و «لزوماً صادق است که 2+2 = 4». چنان‌که در بخش ۸.۱.۲ یاد شد، برخی منطق‌دانان وجه‌های مرتبط با زمان را ترجیح می‌دهند. در «منطق زمانی خطی» (Linear Temporal Logic)، عملگرهای موجهاتی زیر را می‌افزاییم:

- «P **X** P: در گام زمانی بعد صادق خواهد بود» (Next)
- «P **F** P: سرانجام (Finally) در برخی گام‌های زمانی آینده صادق خواهد بود»
- «P **G** P: همیشه (Globally) صادق است»
- «P Q **U** P: تا رخدادن Q صادق می‌ماند» (Until)

گاهی عملگرهای اضافی هست که از این‌ها قابل اشتقاق‌اند. افزودن این عملگرهای موجهاتی خودِ منطق را پیچیده‌تر می‌کند (و بنابراین یافتن برهان را برای الگوریتم استنتاج منطقی دشوارتر می‌کند). اما عملگرها همچنین به ما اجازه می‌دهند برخی واقعیت‌ها را به‌صورت موجزتری بیان کنیم (که استنتاج منطقی را سریع‌تر می‌کند). انتخاب منطقِ استفاده‌شده شبیه انتخاب زبان برنامه‌نویسی است: زبانی برگزینید که برای تکلیفتان مناسب است، با شما و دیگرانی که کارتان را به اشتراک می‌گذارند آشناست، و برای مقاصدتان به اندازهٔ کافی کارآمد است.

۱۰.۵ سامانه‌های استدلال برای رده‌ها

رده‌ها بلوک‌های ساختمانی اولیهٔ طرح‌های بازنمایی دانش بزرگ‌مقیاس‌اند. این بخش سامانه‌هایی را توصیف می‌کند که به‌طور خاص برای سازمان‌دهی و استدلال با رده‌ها طراحی شده‌اند. دو خانوادهٔ نزدیک‌به‌هم از سامانه‌ها هست: «شبکه‌های معنایی» (Semantic Networks) کمک‌های گرافیکی برای تجسم پایگاه دانش و الگوریتم‌های کارآمد برای استنتاج ویژگی‌های شیء بر اساس عضویت رده‌ای‌اش فراهم می‌کنند؛ و «منطق‌های توصیف» (Description Logics) زبان صوری برای ساخت و ترکیب تعریف‌های رده و الگوریتم‌های کارآمد برای تصمیم‌گیری دربارهٔ روابط زیرمجموعه و فوق‌مجموعه میان رده‌ها فراهم می‌کنند.

۱۰.۵.۱ شبکه‌های معنایی

در ۱۹۰۹، چارلز اس. پیرس نماد گرافیکی از گره‌ها و یال‌ها به نام «گراف‌های وجودی» (Existential Graphs) پیشنهاد کرد که آن را «منطق آینده» نامید. بدین ترتیب، مناظرهٔ دیرپایی میان طرفداران «منطق» و طرفداران «شبکه‌های معنایی» آغاز شد. متأسفانه، مناظره این واقعیت را پنهان کرد که شبکه‌های معنایی شکلی از منطق‌اند. نمادی که شبکه‌های معنایی برای انواع خاصی از جمله‌ها فراهم می‌کنند اغلب راحت‌تر است، اما اگر مسائل «واسط انسانی» را کنار بگذاریم، مفاهیم زیربنایی — اشیا، روابط، سورها و مانند آن — یکسان‌اند.

تغییرهای بسیاری از شبکه‌های معنایی هست، اما همه قادر به بازنمایی اشیای فردی، رده‌های اشیا و روابط میان اشیا هستند. نماد گرافیکی نمونه‌ای نام‌های اشیا یا رده‌ها را در بیضی‌ها یا جعبه‌ها نمایش می‌دهد و آن‌ها را با پیوندهای برچسب‌دار متصل می‌کند. برای نمونه، شکل ۱۰.۴ پیوند MemberOf میان Mary و FemalePersons دارد، متناظر با ادعای منطقی Mary ∈ FemalePersons؛ به همین ترتیب، پیوند SisterOf میان Mary و John با ادعای SisterOf(Mary, John) متناظر است. می‌توانیم رده‌ها را با پیوندهای SubsetOf متصل کنیم و الی آخر. کشیدن حباب و پیکان چنان سرگرم‌کننده است که می‌توان از آن غافل شد. برای نمونه، می‌دانیم اشخاص مادرانی زن دارند، پس آیا می‌توانیم پیوند HasMother از Persons به FemalePersons بکشیم؟ پاسخ نه است، زیرا HasMother رابطه‌ای میان شخص و مادرش است و رده‌ها مادر ندارند.[5]

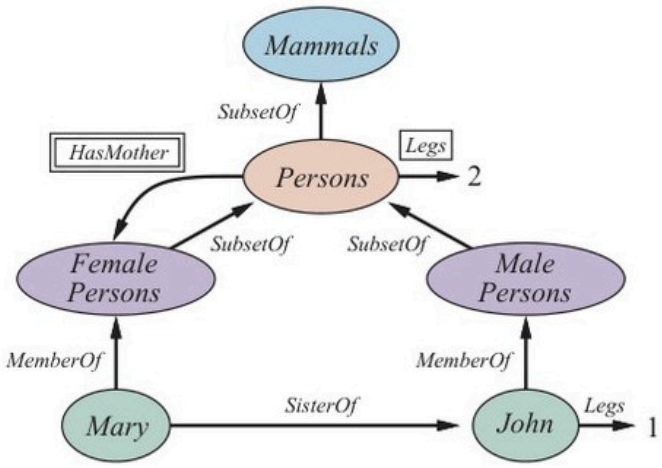
به همین دلیل، در شکل ۱۰.۴ از نماد خاصی — پیوند دوجداره — استفاده کرده‌ایم. این پیوند ادعا می‌کند:

```
∀x x ∈ Persons ⇒ [∀y HasMother(x,y) ⇒ y ∈ FemalePersons]
```

همچنین ممکن است بخواهیم ادعا کنیم اشخاص دو پا دارند — یعنی:

```
∀x x ∈ Persons ⇒ Legs(x, 2)
```

مانند قبل، باید مراقب باشیم ادعا نکنیم رده‌ای پا دارد؛ پیوند تک‌جداره در شکل ۱۰.۴ برای ادعای ویژگی‌های هر عضو رده استفاده می‌شود.

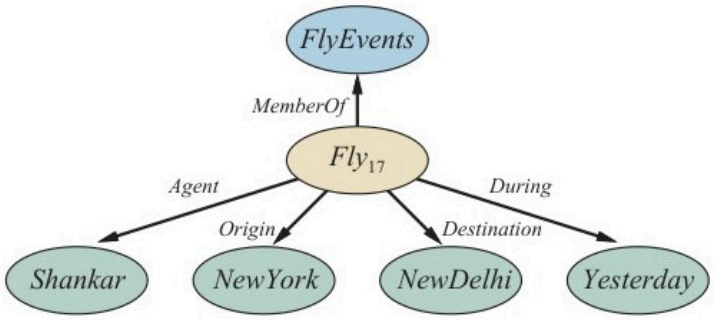


شکل ۱۰.۴ – شبکهٔ معنایی با چهار شیء (Mary، John و 1 و 2) و چهار رده. روابط با پیوندهای برچسب‌دار نشان داده شده‌اند. (رده‌ها: Mammals، Persons، MalePersons، FemalePersons؛ پیوندها: SubsetOf، MemberOf، SisterOf، Legs، HasMother)

نماد شبکهٔ معنایی انجام «استدلال وراثت» از نوع معرفی‌شده در بخش ۱۰.۲ را راحت می‌کند. برای نمونه، به‌واسطهٔ شخص‌بودن، مری ویژگی دوپاداشتن را به ارث می‌برد. پس برای فهمیدن اینکه مری چند پا دارد، الگوریتم وراثت پیوند MemberOf را از مری به رده‌ای که به آن تعلق دارد دنبال می‌کند و سپس پیوندهای SubsetOf را در سلسله‌مراتب بالا دنبال می‌کند تا رده‌ای را بیابد که برایش پیوند جعبه‌دار Legs هست – در این مورد، ردهٔ Persons. سادگی و کارایی این سازوکار استنتاج، در مقایسه با اثبات قضیهٔ منطقی نیمه‌تصمیم‌پذیر، یکی از جاذبه‌های اصلی شبکه‌های معنایی بوده.

وراثت وقتی پیچیده می‌شود که شیئی بتواند به بیش از یک رده تعلق داشته باشد یا رده‌ای بتواند زیرمجموعهٔ بیش از یک ردهٔ دیگر باشد؛ این «وراثت چندگانه» (Multiple Inheritance) نامیده می‌شود. در چنین مواردی، الگوریتم وراثت ممکن است دو یا چند مقدار متعارض برای پاسخ به پرس‌وجو بیابد. به همین دلیل، وراثت چندگانه در برخی زبان‌های برنامه‌نویسی شئ‌گرا (OOP)، مانند جاوا، که از وراثت در سلسله‌مراتب کلاس استفاده می‌کنند، ممنوع است. در شبکه‌های معنایی معمولاً مجاز است، اما بحث آن را به بخش ۱۰.۶ موکول می‌کنیم.

خواننده احتمالاً متوجه نقطه‌ضعف بدیهی نماد شبکهٔ معنایی، در مقایسه با منطق مرتبهٔ اول، شده: واقعیت که پیوندهای میان حباب‌ها فقط روابط دوتایی را بازنمایی می‌کنند. برای نمونه، جملهٔ Fly(Shankar, NewYork, NewDelhi, Yesterday) را نمی‌توان مستقیماً در شبکهٔ معنایی ادعا کرد. با این حال، می‌توانیم اثر ادعاهای n-تایی را با شیءسازی خودِ گزاره به‌عنوان رویدادی متعلق به ردهٔ رویداد مناسب به دست آوریم. شکل ۱۰.۵ ساختار شبکهٔ معنایی را برای این رویداد خاص نشان می‌دهد. توجه کنید محدودیت به روابط دوتایی، ایجاد هستی‌شناسی غنی از مفاهیم شیءسازی‌شده را ایجاب می‌کند.



شکل ۱۰.۵ – پاره‌ای از شبکهٔ معنایی نشان‌دهندهٔ بازنمایی ادعای منطقی Fly(Shankar, NewYork, NewDelhi, Yesterday). (گره‌ها: FlyEvents، Fly17، Shankar، NewYork، NewDelhi، Yesterday؛ پیوندها: MemberOf، Agent، Origin، Destination، During)

شیءسازی گزاره‌ها بازنمایی هر جملهٔ اتمیِ بدون‌تابع و زمینی منطق مرتبهٔ اول را در نماد شبکهٔ معنایی ممکن می‌کند. انواع خاصی از جمله‌های همگانی‌سوردار را می‌توان با پیوندهای معکوس و پیکان‌های تک‌جداره و دوجدارهٔ اعمال‌شده بر رده‌ها ادعا کرد، اما آن همچنان ما را از منطق مرتبهٔ اول کامل فاصله می‌دهد. نقیض، فصل، نمادهای تابع تودرتو و سور وجودی همگی غایب‌اند. اکنون امکان بسط نماد برای هم‌ارزکردنش با منطق مرتبهٔ اول هست — مانند گراف‌های وجودی پیرس — اما انجام این یکی از مزیت‌های اصلی شبکه‌های معنایی، یعنی سادگی و شفافیت فرایندهای استنتاج، را نفی می‌کند. طراحان می‌توانند شبکهٔ بزرگی بسازند و همچنان ایدهٔ خوبی داشته باشند کدام پرس‌وجوها کارآمد خواهند بود، زیرا (الف) تجسم گام‌هایی که رویهٔ استنتاج طی خواهد کرد آسان است و (ب) در برخی موارد زبان پرس‌وجو چنان ساده است که پرس‌وجوهای دشوار قابل طرح نیستند.

در مواردی که قدرت بیانی بیش از حد محدودکننده ثابت شود، بسیاری از سامانه‌های شبکهٔ معنایی برای پرکردن شکاف‌ها «پیوست رویه‌ای» (Procedural Attachment) فراهم می‌کنند. پیوست رویه‌ای فنّی است که در آن پرس‌وجو دربارهٔ (و گاهی ادعای) رابطهٔ خاصی به فراخوانی رویهٔ خاصِ طراحی‌شده برای آن رابطه می‌انجامد، به‌جای الگوریتم استنتاج عمومی.

یکی از مهم‌ترین جنبه‌های شبکه‌های معنایی توانایی‌شان در بازنمایی «مقادیر پیش‌فرض» (Default Values) برای رده‌هاست. با بررسی دقیق شکل ۱۰.۴، متوجه می‌شویم جان یک پا دارد، با وجود اینکه شخص است و همهٔ اشخاص دو پا دارند. در KB صرفاً منطقی، این تناقض می‌بود، اما در شبکهٔ معنایی، ادعای اینکه همهٔ اشخاص دو پا دارند فقط وضعیت پیش‌فرض دارد؛ یعنی فرض می‌شود شخصی دو پا دارد مگر آنکه با اطلاعات خاص‌تر تناقض داده شود. معناشناسی پیش‌فرض به‌طور طبیعی توسط الگوریتم وراثت اعمال می‌شود، زیرا پیوندها را از خودِ شیء (در این مورد جان) به بالا دنبال می‌کند و به‌محض یافتن مقدار می‌ایستد. می‌گوییم پیش‌فرض با مقدار خاص‌تر «لغو» (Overriding) می‌شود. توجه کنید می‌توانستیم پیش‌فرض تعداد پاها را با ایجاد رده‌ای از OneLeggedPersons، زیرمجموعه‌ای از Persons که جان عضو آن است، نیز لغو کنیم.

می‌توانیم معناشناسی صرفاً منطقی را برای شبکه حفظ کنیم اگر بگوییم ادعای Legs برای Persons استثنایی برای جان شامل می‌شود:

```
∀x x ∈ Persons ∧ x ≠ John ⇒ Legs(x, 2)
```

برای شبکهٔ ثابت، این از نظر معنایی کافی است، اما اگر استثناهای فراوان باشند بسیار کم‌موجزتر از خودِ نماد شبکه خواهد بود. برای شبکه‌ای که با ادعاهای بیشتری به‌روز می‌شود، چنین رویکردی شکست می‌خورد — واقعاً می‌خواهیم بگوییم هر شخصِ تاکنون‌ناشناخته‌ای با یک پا نیز استثناست. بخش ۱۰.۶ با ژرفای بیشتری به این مسئله و به استدلال پیش‌فرض به‌طور کلی می‌پردازد.

۱۰.۵.۲ منطق‌های توصیف

نحو منطق مرتبهٔ اول برای آسان‌کردن گفتن چیزها دربارهٔ اشیا طراحی شده. منطق‌های توصیف نمادهایی هستند که برای آسان‌کردن توصیف تعریف‌ها و ویژگی‌های رده‌ها طراحی شده‌اند. سامانه‌های منطق توصیف از شبکه‌های معنایی در پاسخ به فشار برای صوری‌کردن معنای شبکه‌ها تکامل یافتند، در حالی که تأکید بر ساختار رده‌بندی به‌عنوان اصل سازمان‌دهنده حفظ شد.

تکلیف‌های استنتاج اصلی برای منطق‌های توصیف «زیرشمولی» (Subsumption) (بررسی اینکه آیا رده‌ای زیرمجموعهٔ ردهٔ دیگر است با مقایسهٔ تعریف‌هایشان) و «طبقه‌بندی» (Classification) (بررسی اینکه آیا شیئی به رده‌ای تعلق دارد) هستند. برخی سامانه‌ها همچنین «سازگاری» (Consistency) تعریف رده — اینکه آیا معیارهای عضویت به‌صورت منطقی ارضاپذیرند — را شامل می‌شوند.

$$
\begin{aligned}
\mathit{Concept} &\rightarrow \mathbf{Thing} \mid \mathit{ConceptName} \\
&\mid \mathbf{And}(\mathit{Concept}, \ldots) \\
&\mid \mathbf{All}(\mathit{RoleName}, \mathit{Concept}) \\
&\mid \mathbf{AtLeast}(\mathit{Integer}, \mathit{RoleName}) \\
&\mid \mathbf{AtMost}(\mathit{Integer}, \mathit{RoleName}) \\
&\mid \mathbf{Fills}(\mathit{RoleName}, \mathit{IndividualName}, \ldots) \\
&\mid \mathbf{SameAs}(\mathit{Path}, \mathit{Path}) \\
&\mid \mathbf{OneOf}(\mathit{IndividualName}, \ldots) \\
\mathit{Path} &\rightarrow [\mathit{RoleName}, \ldots] \\
\mathit{ConceptName} &\rightarrow \mathit{Adult} \mid \mathit{Female} \mid \mathit{Male} \mid \ldots \\
\mathit{RoleName} &\rightarrow \mathit{Spouse} \mid \mathit{Daughter} \mid \mathit{Son} \mid \ldots
\end{aligned}
$$

شکل ۱۰.۶ — نحو توصیف‌ها در زیرمجموعه‌ای از زبان CLASSIC.

زبان CLASSIC (بورگیدا و همکاران، ۱۹۸۹) منطق توصیف نمونه‌ای است. نحو توصیف‌های CLASSIC در شکل ۱۰.۶ نشان داده شده.[6] برای نمونه، برای گفتن اینکه مجردها مردان بالغ ازدواج‌نکرده‌اند می‌نویسیم:

```
Bachelor = And(Unmarried, Adult, Male)
```

معادل در منطق مرتبهٔ اول:

```
Bachelor(x) ⇔ Unmarried(x) ∧ Adult(x) ∧ Male(x)
```

توجه کنید منطق توصیف جبری از عملیات روی گزاره‌ها دارد، که البته در منطق مرتبهٔ اول نمی‌توانیم انجام دهیم. هر توصیفی در CLASSIC را می‌توان به جملهٔ مرتبهٔ اولِ هم‌ارزی ترجمه کرد، اما برخی توصیف‌ها در CLASSIC سرراست‌ترند. برای نمونه، برای توصیف مجموعهٔ مردانی با دست‌کم سه پسر که همگی بیکارند و با پزشکان ازدواج کرده‌اند، و حداکثر دو دختر که همگی استاد دپارتمان‌های فیزیک یا ریاضی‌اند، از این استفاده می‌کنیم:

```
And(Man, AtLeast(3,Son), AtMost(2,Daughter), All(Son, And(Unemployed, Married, All(Spouse,
                Doctor))), All(Daughter, And(Professor, Fills(Department, Physics, Math))))
```

ترجمهٔ این را به منطق مرتبهٔ اول به‌عنوان تمرین واگذار می‌کنیم.

شاید مهم‌ترین جنبهٔ منطق‌های توصیف تأکیدشان بر حل‌پذیری استنتاج باشد. نمونهٔ مسئله با توصیفش حل می‌شود و سپس پرسیده می‌شود آیا توسط یکی از چند ردهٔ راه‌حل ممکن زیرشمول می‌شود. در سامانه‌های منطق مرتبهٔ اول استاندارد، پیش‌بینی زمان راه‌حل اغلب ناممکن است. اغلب به کاربر واگذار می‌شود بازنمایی را چنان مهندسی کند که دور مجموعه‌جمله‌هایی که به نظر می‌رسد سامانه را هفته‌ها مشغول می‌کنند دور بزند. رویکرد و تمرکز اصلی در منطق‌های توصیف، از سوی دیگر، اطمینان از آن است که آزمون زیرشمولی در زمان چندجمله‌ای نسبت به اندازهٔ توصیف‌ها قابل حل باشد.[7]

این در اصل عالی به نظر می‌رسد، تا آنکه کسی دریابد فقط می‌تواند یکی از دو پیامد داشته باشد: یا مسائل سخت اصلاً قابل بیان نیستند، یا به توصیف‌های نمایی‌بزرگ نیاز دارند! اما نتایج حل‌پذیری بر این روشنایی می‌افکنند که چه نوع سازه‌هایی مشکل‌سازند و بدین ترتیب به کاربر کمک می‌کنند بفهمد بازنمایی‌های مختلف چگونه رفتار می‌کنند. برای نمونه، منطق‌های توصیف معمولاً نقیض و فصل ندارند. هر یک سامانه‌های منطق مرتبهٔ اول را مجبور می‌کند تحلیل حالتِ بالقوه نمایی‌ای را طی کنند تا کامل بودن تضمین شود. CLASSIC فقط شکل محدودی از فصل را در سازه‌های Fills و OneOf اجازه می‌دهد، که فصل را روی افرادِ صریحاً برشمرده‌شده اجازه می‌دهند اما نه روی توصیف‌ها. با توصیف‌های فصلی، تعریف‌های تودرتو به‌راحتی می‌توانند به شمار نمایی مسیرهای جایگزینی بیانجامند که رده‌ای به‌واسطهٔ آن‌ها ردهٔ دیگر را زیرشمول می‌کند.

۱۰.۶ استدلال با اطلاعات پیش‌فرض

در بخش پیشین، مثال ساده‌ای از ادعایی با وضعیت پیش‌فرض دیدیم: مردم دو پا دارند. این پیش‌فرض را می‌توان با اطلاعات خاص‌تر، مانند اینکه جان سیلورِ لنگ یک پا دارد، لغو کرد. دیدیم سازوکار وراثت در شبکه‌های معنایی لغو پیش‌فرض‌ها را به‌شیوه‌ای ساده و طبیعی پیاده می‌کند. در این بخش، پیش‌فرض‌ها را عمومی‌تر مطالعه می‌کنیم، با دید فهم معناشناسی پیش‌فرض‌ها نه صرفاً فراهم‌کردن سازوکار رویه‌ای.

۱۰.۶.۱ تحدید (Circumscription) و منطق پیش‌فرض

دو مثال از فرایندهای استدلالی دیده‌ایم که ویژگی «یکنواختی» (Monotonicity) منطقِ اثبات‌شده در فصل ۷ را نقض می‌کنند.[8] در این فصل دیدیم ویژگیِ به‌ارث‌رفته توسط همهٔ اعضای رده‌ای در شبکهٔ معنایی را می‌توان با اطلاعات خاص‌تر برای زیررده‌ای لغو کرد. در بخش ۹.۴.۴ دیدیم که تحت فرض جهان بسته، اگر گزارهٔ α در KB ذکر نشده باشد، $KB \models \neg\alpha$، اما $KB \wedge \alpha \models \alpha$.

درون‌نگری ساده پیشنهاد می‌کند این شکست‌های یکنواختی در استدلال عقل‌سلیمی فراگیرند. به نظر می‌رسد انسان‌ها اغلب «به نتیجه می‌پرند». برای نمونه، وقتی کسی خودرویی را پارک‌شده در خیابان می‌بیند، معمولاً مایل است باور کند چهار چرخ دارد، حتی با آنکه فقط سه تا نمایان است. حال، نظریهٔ احتمال قطعاً می‌تواند نتیجه‌ای فراهم کند که چرخ چهارم با احتمال بالا وجود دارد؛ اما برای بیشتر مردم، امکانِ چهارچرخ‌نداشتنِ خودرو مطرح نخواهد شد مگر آنکه شاهد تازه‌ای پدیدار شود. پس به نظر می‌رسد نتیجهٔ چهارچرخ به‌طور پیش‌فرض، در غیاب دلیلی برای تردید، گرفته می‌شود. اگر شاهد تازه‌ای برسد — برای نمونه، اگر کسی مالک را ببیند که چرخی حمل می‌کند و متوجه شود خودرو جک زده شده — آنگاه نتیجه را می‌توان پس گرفت. به این نوع استدلال گفته می‌شود «غیریکنواخت» (Nonmonotonic) است، زیرا مجموعهٔ باورها با رسیدن شواهد تازه به‌صورت یکنواخت رشد نمی‌کند. «منطق‌های غیریکنواخت» (Nonmonotonic Logics) با مفاهیم اصلاح‌شدهٔ صدق و لازمه برای درک چنین رفتاری ابداع شده‌اند. به دو چنین منطقی که به‌طور گسترده مطالعه شده‌اند نگاه خواهیم کرد: تحدید و منطق پیش‌فرض.

«تحدید» (Circumscription) را می‌توان نسخهٔ قدرتمندتر و دقیق‌تری از فرض جهان بسته دید. ایده آن است که گزاره‌های خاصی را مشخص کنیم که فرض می‌شود «تا حد ممکن کاذب» هستند — یعنی برای هر شیء به‌جز آن‌هایی که برایشان معلوم است صادق‌اند کاذب‌اند. برای نمونه، فرض کنیم می‌خواهیم قاعدهٔ پیش‌فرض «پرندگان پرواز می‌کنند» را ادعا کنیم. گزاره‌ای، مثلاً Abnormal1(x)، معرفی می‌کنیم و می‌نویسیم:

```
Bird(x) ∧ ¬Abnormal1(x) ⇒ Flies(x)
```

اگر بگوییم Abnormal1 قرار است تحدید شود، استدلال‌گر تحدیدی مجاز است ¬Abnormal1(x) را فرض کند مگر آنکه Abnormal1(x) معلوم‌الصدق باشد. این اجازه می‌دهد نتیجهٔ Flies(Tweety) از مقدمهٔ Bird(Tweety) گرفته شود، اما نتیجه دیگر برقرار نیست اگر Abnormal1(Tweety) ادعا شود.

تحدید را می‌توان مثالی از «منطق ترجیح مدل» (Model Preference Logic) دید. در چنین منطق‌هایی، جمله‌ای (با وضعیت پیش‌فرض) مستلزم است اگر در همهٔ مدل‌های ترجیحی KB صادق باشد، در برابر الزام صدق در همهٔ مدل‌ها در منطق کلاسیک. برای تحدید، مدلی بر مدل دیگر ترجیح دارد اگر اشیای نابهنجار کمتری داشته باشد.[9] بگذارید ببینیم این ایده در بافت وراثت چندگانه در شبکه‌های معنایی چگونه کار می‌کند. مثال استانداردی که وراثت چندگانه برایش مشکل‌ساز است «الماس نیکسون» نامیده می‌شود. از این مشاهده ناشی می‌شود که ریچارد نیکسون هم کوییکر بود (و از این رو طبق پیش‌فرض صلح‌طلب) و هم جمهوری‌خواه (و از این رو طبق پیش‌فرض صلح‌طلب نبود). می‌توانیم این را چنین بنویسیم:

```
Republican(Nixon) ∧ Quaker(Nixon)
Republican(x) ∧ ¬Abnormal2(x) ⇒ ¬Pacifist(x)
Quaker(x) ∧ ¬Abnormal3(x) ⇒ Pacifist(x)
```

اگر Abnormal2 و Abnormal3 را تحدید کنیم، دو مدل ترجیحی هست: یکی که در آن Abnormal2(Nixon) و Pacifist(Nixon) صادق‌اند و یکی که در آن Abnormal3(Nixon) و ¬Pacifist(Nixon) صادق‌اند. پس استدلال‌گر تحدیدی به‌درستی دربارهٔ صلح‌طلب‌بودن یا نبودن نیکسون بی‌نظر می‌ماند. اگر بخواهیم، علاوه بر آن، ادعا کنیم باورهای مذهبی بر باورهای سیاسی اولویت دارند، می‌توانیم از صورت‌بندی به نام «تحدید اولویت‌دار» (Prioritized Circumscription) برای اولویت‌دادن به مدل‌هایی که Abnormal3 در آن‌ها کمینه شده استفاده کنیم.

«منطق پیش‌فرض» (Default Logic) صورت‌بندی‌ای است که در آن «قواعد پیش‌فرض» (Default Rules) را می‌توان برای تولید نتیجه‌های اقتضایی و غیریکنواخت نوشت. قاعدهٔ پیش‌فرض شبیه این است:

```
Bird(x) : Flies(x) / Flies(x)
```

این قاعده بدان معناست اگر Bird(x) صادق است، و اگر Flies(x) با پایگاه دانش سازگار است، آنگاه Flies(x) را می‌توان طبق پیش‌فرض نتیجه گرفت. در حالت کلی، قاعدهٔ پیش‌فرض شکل

```
P : J1, ..., Jn / C
```

را دارد که P «پیش‌نیاز»، C «نتیجه»، و Ji «توجیه»ها نامیده می‌شوند — اگر هر یک از آن‌ها به‌شکل اثبات‌پذیر کاذب باشد، نتیجه را نمی‌توان گرفت. هر متغیری که در Ji یا C ظاهر می‌شود باید در P نیز ظاهر شود. مثال الماس نیکسون را می‌توان در منطق پیش‌فرض با یک واقعیت و دو قاعدهٔ پیش‌فرض بازنمایی کرد:

```
Republican(Nixon) ∧ Quaker(Nixon)
Republican(x) : ¬Pacifist(x) / ¬Pacifist(x)
Quaker(x) : Pacifist(x) / Pacifist(x)
```

برای تفسیر معنای قواعد پیش‌فرض، مفهوم «بسط» (Extension) نظریهٔ پیش‌فرض را به‌عنوان مجموعهٔ بیشینهٔ پیامدهای نظریه تعریف می‌کنیم. یعنی بسط S شامل واقعیت‌های شناخته‌شدهٔ اصلی و مجموعه‌ای از نتیجه‌های قواعد پیش‌فرض است، چنان‌که هیچ نتیجهٔ اضافی‌ای از S قابل گرفتن نیست و توجیه‌های هر نتیجهٔ پیش‌فرضی در S با S سازگارند. مانند مورد مدل‌های ترجیحی در تحدید، دو بسط ممکن برای الماس نیکسون داریم: یکی که در آن صلح‌طلب است و یکی که در آن نیست. طرح‌های اولویت‌دار وجود دارند که در آن‌ها می‌توان به برخی قواعد پیش‌فرض بر دیگران اولویت داد و به برخی ابهام‌ها اجازه حل‌شدن می‌دهد.

از ۱۹۸۰، وقتی منطق‌های غیریکنواخت نخستین بار پیشنهاد شدند، پیشرفت فراوانی در فهم ویژگی‌های ریاضی‌شان حاصل شده. هنوز پرسش‌های حل‌نشده هست. برای نمونه، اگر «خودروها چهار چرخ دارند» کاذب است، داشتن آن در پایگاه دانش به چه معناست؟ مجموعهٔ خوبی از قواعد پیش‌فرض برای داشتن چیست؟ اگر نتوانیم برای هر قاعده به‌طور جداگانه تصمیم بگیریم آیا به پایگاه دانش‌مان تعلق دارد، مشکل جدی ناپیمانه‌ای (Nonmodularity) داریم. سرانجام، باورهای دارای وضعیت پیش‌فرض چگونه برای تصمیم‌گیری استفاده می‌شوند؟ این احتمالاً سخت‌ترین مسئله برای استدلال پیش‌فرض است.

تصمیم‌ها اغلب شامل دادوستد هستند و بنابراین باید قدرت باور به پیامدهای کنش‌های متفاوت و هزینه‌های تصمیم اشتباه را مقایسه کرد. در مواردی که همان نوع تصمیم‌ها مکرراً گرفته می‌شود، می‌توان قواعد پیش‌فرض را به‌صورت گزاره‌های «احتمال آستانه‌ای» تفسیر کرد. برای نمونه، قاعدهٔ پیش‌فرض «ترمزهای من همیشه خوب‌اند» واقعاً بدان معناست «احتمال خوب‌بودن ترمزهای من، با نبود اطلاعات دیگر، به‌اندازهٔ کافی بالاست که تصمیم بهینه برای من رانندگی بدون بررسی آن‌هاست.» وقتی بافت تصمیم تغییر می‌کند — برای نمونه، وقتی با کامیون سنگینی از جادهٔ کوهستانی شیب‌دار پایین می‌روید — قاعدهٔ پیش‌فرض ناگهان نامناسب می‌شود، حتی با آنکه شاهد تازه‌ای از ترمزهای معیوب نیست. این ملاحظات پژوهشگران را به این فکر انداخته که چگونه استدلال پیش‌فرض را درون نظریهٔ احتمال یا نظریهٔ مطلوبیت جاسازی کنند.

۱۰.۶.۲ سامانه‌های نگه‌داری صدق

دیده‌ایم بسیاری از استنتاج‌های گرفته‌شده توسط سامانهٔ بازنمایی دانش فقط وضعیت پیش‌فرض خواهند داشت، نه قطعیت مطلق. ناگزیر، برخی از این واقعیت‌های استنتاج‌شده نادرست از آب درخواهند آمد و باید در برابر اطلاعات تازه پس گرفته شوند. این فرایند «بازبینی باور» (Belief Revision) نامیده می‌شود.[10] فرض کنیم پایگاه دانش KB شامل جملهٔ P است — شاید نتیجهٔ پیش‌فرضی که توسط الگوریتم زنجیره‌سازی پیشرو ثبت شده، یا شاید فقط ادعای نادرستی — و می‌خواهیم TELL(KB, ¬P) را اجرا کنیم. برای پرهیز از ایجاد تناقض، باید نخست RETRACT(KB, P) را اجرا کنیم. این به اندازهٔ کافی آسان به نظر می‌رسد. اما اگر جمله‌های اضافی از P استنتاج و در KB ادعا شده باشند، مسائل پیش می‌آیند. برای نمونه، ممکن است از دلالت P ⇒ Q برای افزودن Q استفاده شده باشد. «راه‌حل» بدیهی — پس‌گرفتن همهٔ جمله‌های استنتاج‌شده از P — شکست می‌خورد، زیرا چنین جمله‌هایی ممکن است توجیه‌های دیگری غیر از P داشته باشند. برای نمونه، اگر R و R ⇒ Q نیز در KB باشند، پس Q پس از همه لازم نیست حذف شود. «سامانه‌های نگه‌داری صدق» (Truth Maintenance Systems) یا TMSها دقیقاً برای مدیریت این نوع پیچیدگی‌ها طراحی شده‌اند.

یک رویکرد ساده به نگه‌داری صدق، پیگیری ترتیبی است که جمله‌ها به پایگاه دانش گفته می‌شوند، با شماره‌گذاری آن‌ها از P1 تا Pn. وقتی فراخوانی RETRACT(KB, Pi) انجام می‌شود، سامانه به حالت درست پیش از افزودن Pi بازمی‌گردد و بدین ترتیب هم Pi و هم هر استنتاجِ مشتق‌شده از Pi حذف می‌شوند. سپس جمله‌های Pi+1 تا Pn را می‌توان دوباره افزود. این ساده است و تضمین می‌کند پایگاه دانش سازگار خواهد بود، اما پس‌گرفتن Pi مستلزم پس‌گرفتن و دوباره‌ادعاکردن n-i جمله و نیز خنثی‌کردن و دوباره‌انجام‌دادن همهٔ استنتاج‌های گرفته‌شده از آن جمله‌هاست. برای سامانه‌هایی که واقعیت‌های فراوانی به آن‌ها افزوده می‌شود — مانند پایگاه‌های دادهٔ تجاری بزرگ — این غیرعملی است.

رویکرد کارآمدتر «سامانهٔ نگه‌داری صدق مبتنی بر توجیه» یا JTMS است. در JTMS، هر جمله در پایگاه دانش با «توجیه»ی (Justification) حاشیه‌نویسی می‌شود که شامل مجموعه‌جمله‌هایی است که از آن‌ها استنتاج شده. برای نمونه، اگر پایگاه دانش از قبل P ⇒ Q را شامل شود، TELL(P) باعث می‌شود Q با توجیه {P, P ⇒ Q} افزوده شود. در حالت کلی، جمله می‌تواند هر تعداد توجیه داشته باشد. توجیه‌ها پس‌گرفتن را کارآمد می‌کنند. با فراخوانی RETRACT(P)، JTMS دقیقاً آن جمله‌هایی را حذف می‌کند که P عضو هر توجیه‌شان است. پس اگر جملهٔ Q توجیه تکی {P, P ⇒ Q} داشت، حذف می‌شد؛ اگر توجیه اضافی {P, P ∨ R ⇒ Q} داشت، باز حذف می‌شد؛ اما اگر توجیه {R, P ∨ R ⇒ Q} را نیز داشت، نجات می‌یافت. بدین ترتیب، زمان لازم برای پس‌گرفتن P فقط به شمار جمله‌های مشتق‌شده از P بستگی دارد نه به شمار جمله‌های افزوده‌شده پس از P.

JTMS فرض می‌کند جمله‌هایی که یک بار در نظر گرفته شده‌اند احتمالاً دوباره در نظر گرفته خواهند شد، پس به‌جای حذف کامل جمله از پایگاه دانش وقتی همهٔ توجیه‌هایش را از دست می‌دهد، صرفاً جمله را به‌عنوان «بیرون» از پایگاه دانش علامت می‌زنیم. اگر ادعای بعدی یکی از توجیه‌ها را بازگرداند، جمله را به‌عنوان «داخل» علامت می‌زنیم. بدین ترتیب، JTMS همهٔ زنجیره‌های استنتاجی را که استفاده می‌کند حفظ می‌کند و وقتی توجیه‌ای دوباره معتبر می‌شود لازم نیست جمله‌ها را دوباره مشتق کند.

علاوه بر مدیریت پس‌گرفتن اطلاعات نادرست، TMSها را می‌توان برای سرعت‌بخشیدن به تحلیل چند وضعیت فرضی استفاده کرد. فرض کنیم، برای نمونه، کمیتهٔ المپیک رومانی در حال انتخاب مکان‌های رویدادهای شنا، دوومیدانی و سوارکاری برای بازی‌های ۲۰۴۸ است که قرار است در رومانی برگزار شود. برای نمونه، فرضیهٔ نخست Site(Swimming, Pitesti)، Site(Athletics, Bucharest) و Site(Equestrian, Arad) باشد. سپس باید استدلال فراوانی برای محاسبهٔ پیامدهای لجستیکی و در نتیجه مطلوبیت این انتخاب انجام شود. اگر بخواهیم به‌جایش Site(Athletics, Sibiu) را در نظر بگیریم، TMS از نیاز به شروع دوباره از صفر اجتناب می‌کند. در عوض، به‌سادگی Site(Athletics, Bucharest) را پس می‌گیریم و Site(Athletics, Sibiu) را ادعا می‌کنیم و TMS از بازبینی‌های لازم مراقبت می‌کند. زنجیره‌های استنتاجِ تولیدشده از انتخاب بخارست را می‌توان با سیبیو دوباره استفاده کرد، به شرط آنکه نتیجه‌ها یکسان باشند.

«سامانهٔ نگه‌داری صدق مبتنی بر فرض» یا ATMS این نوع سوئیچ بافت میان جهان‌های فرضی را به‌ویژه کارآمد می‌کند. در JTMS، نگه‌داری توجیه‌ها به شما امکان می‌دهد با چند پس‌گرفتن و ادعا به‌سرعت از حالتی به حالت دیگر بروید، اما در هر زمان فقط یک حالت بازنمایی می‌شود. ATMS همهٔ حالت‌هایی را که تاکنون در نظر گرفته شده‌اند هم‌زمان بازنمایی می‌کند. در حالی که JTMS صرفاً هر جمله را به‌عنوان داخل یا بیرون علامت می‌زند، ATMS برای هر جمله

پیگیری می‌کند کدام فرض‌ها باعث صادق‌بودن جمله می‌شوند. به بیان دیگر، هر جمله برچسبی دارد که از مجموعه‌ای از مجموعه‌فرض‌ها تشکیل شده. جمله دقیقاً در آن مواردی صادق است که همهٔ فرض‌های یکی از مجموعه‌فرض‌ها صادق باشند.

سامانه‌های نگه‌داری صدق همچنین سازوکاری برای تولید «تبیین» (Explanation) فراهم می‌کنند. از نظر فنی، تبیین جملهٔ P مجموعه‌جمله‌ای E است چنان‌که E مستلزم P است. اگر جمله‌های E از قبل معلوم‌الصدق باشند، E صرفاً مبنای کافی برای اثبات اینکه P باید صادق باشد فراهم می‌کند. اما تبیین‌ها می‌توانند «فرض»ها (Assumptions) را نیز شامل شوند — جمله‌هایی که معلوم‌الصدق نیستند، اما اگر صادق بودند برای اثبات P کافی می‌بودند. برای نمونه، اگر خودروتان روشن نمی‌شود، احتمالاً اطلاعات کافی برای اثبات قطعی علت مشکل ندارید. اما تبیین معقولی ممکن است شامل فرضِ خراب‌بودن باتری باشد. این، همراه با دانش چگونگی کارکرد خودروها، رفتار ناکارِ مشاهده‌شده را تبیین می‌کند. در بیشتر موارد، تبیین E را ترجیح می‌دهیم که «کمینه» باشد، یعنی هیچ زیرمجموعهٔ سره‌ای از E تبیین نباشد. ATMS می‌تواند تبیین‌هایی برای مسئلهٔ «خودرو روشن نمی‌شود» با فرض‌کردن (مانند «بنزین در خودرو نیست» یا «باتری خراب است») به هر ترتیبی که بخواهیم تولید کند، حتی اگر برخی فرض‌ها متناقض باشند. سپس به برچسب جملهٔ «خودرو روشن نمی‌شود» نگاه می‌کنیم تا مجموعه‌فرض‌هایی را که جمله را توجیه می‌کنند بخوانیم.

الگوریتم‌های دقیق استفاده‌شده برای پیاده‌سازی سامانه‌های نگه‌داری صدق کمی پیچیده‌اند و این‌جا پوشش‌شان نمی‌دهیم. پیچیدگی محاسباتی مسئلهٔ نگه‌داری صدق دست‌کم به اندازهٔ پیچیدگی استنتاج گزاره‌ای است — یعنی NP-سخت. پس نباید انتظار داشته باشید نگه‌داری صدق نوشدارو باشد. اما وقتی با دقت استفاده شود، TMS می‌تواند افزایش چشمگیری در توانایی سامانهٔ منطقی برای مدیریت محیط‌ها و فرضیه‌های پیچیده فراهم کند.

خلاصهٔ فصل

با شیرجه در جزئیات چگونگی بازنمایی انواع دانش، امیدواریم به خواننده حسی از چگونگی ساخت پایگاه‌های دانش واقعی و حسی از مسائل فلسفی جالبِ پیش‌آمده داده باشیم. نکته‌های اصلی چنین‌اند:

- بازنمایی دانش بزرگ‌مقیاس به هستی‌شناسی عمومی‌منظوره‌ای نیاز دارد تا حوزه‌های خاص گوناگون دانش را سازمان دهد و به هم پیوند دهد.
- هستی‌شناسی عمومی‌منظوره باید گسترهٔ وسیعی از دانش را پوشش دهد و باید در اصل قادر به مدیریت هر حوزه‌ای باشد.
- ساختن هستی‌شناسی بزرگ و عمومی‌منظوره چالش مهمی است که هنوز کاملاً محقق نشده، هرچند چارچوب‌های کنونی نسبتاً استوار به نظر می‌رسند.
- هستی‌شناسی بالایی بر اساس رده‌ها و حساب رویدادها ارائه کردیم. رده‌ها، زیررده‌ها، اجزا، اشیای ساخت‌یافته، اندازه‌گیری‌ها، مواد، رویدادها، زمان و مکان، تغییر و باورها را پوشش دادیم.
- انواع طبیعی را نمی‌توان به‌طور کامل در منطق تعریف کرد، اما ویژگی‌های انواع طبیعی قابل بازنمایی‌اند.
- کنش‌ها، رویدادها و زمان را می‌توان با حساب رویدادها بازنمایی کرد. چنین بازنمایی‌هایی عامل را قادر می‌سازند توالی‌های کنش بسازد و استنتاج‌های منطقی دربارهٔ آنچه هنگام رخ‌دادن این کنش‌ها صادق خواهد بود انجام دهد.
- سامانه‌های بازنمایی ویژه‌منظوره، مانند شبکه‌های معنایی و منطق‌های توصیف، برای کمک به سازمان‌دهی سلسله‌مراتب رده‌ها ابداع شده‌اند. وراثت شکل مهمی از استنتاج است که به ویژگی‌های اشیا اجازه می‌دهد از عضویت‌شان در رده‌ها استنتاج شود.
- فرض جهان بسته، چنان‌که در برنامه‌های منطقی پیاده شده، راه ساده‌ای برای پرهیز از مشخص‌کردن حجم زیادی اطلاعات منفی فراهم می‌کند. بهتر است به‌عنوان پیش‌فرضی تفسیر شود که می‌توان با اطلاعات اضافی لغوش کرد.
- منطق‌های غیریکنواخت، مانند تحدید و منطق پیش‌فرض، برای درک استدلال پیش‌فرض در حالت کلی در نظر گرفته شده‌اند.
- سامانه‌های نگه‌داری صدق به‌روزرسانی و بازبینی دانش را کارآمد مدیریت می‌کنند.
- ساخت هستی‌شناسی‌های بزرگ با دست دشوار است؛ استخراج دانش از متن کار را آسان‌تر می‌کند.

یادداشت‌های کتاب‌شناختی و تاریخی

بریگز (۱۹۸۵) ادعا می‌کند پژوهش بازنمایی دانش با نظریه‌پردازی هندی‌های هزارهٔ اول پیش از میلاد دربارهٔ دستور زبان سانسکریت شاستریک آغاز شد. فیلسوفان غربی کارشان در این موضوع را به حدود ۳۰۰ پیش از میلاد در «متافیزیک» ارسطو بازمی‌گردانند (لفظاً، آنچه پس از کتاب فیزیک می‌آید). توسعهٔ اصطلاحات فنی در هر حوزه‌ای را می‌توان شکلی از

بازنمایی دانش دانست.

بحث‌های آغازین بازنمایی در هوش مصنوعی تمایل داشتند بر «بازنمایی مسئله» تمرکز کنند نه «بازنمایی دانش». (برای نمونه، بحث آمارل (۱۹۶۸) دربارهٔ مسئلهٔ «مبلغان و آدم‌خواران» را ببینید.) در دههٔ ۱۹۷۰، هوش مصنوعی بر توسعهٔ «سامانه‌های خبره» (که «سامانه‌های مبتنی بر دانش» نیز نامیده می‌شوند) تأکید کرد که اگر دانش حوزهٔ مناسب داده شود، می‌توانستند با عملکرد متخصصان انسانی در تکلیف‌های محدود برابری کنند یا از آن فراتر روند. برای نمونه، نخستین سامانهٔ خبره، DENDRAL (فایگنباوم و همکاران، ۱۹۷۱؛ لیندزی و همکاران، ۱۹۸۰)، خروجی طیف‌سنج جرمی (نوعی ابزار مورد استفاده برای تحلیل ساختار ترکیبات شیمیایی آلی) را به همان دقت شیمی‌دانان خبره تفسیر می‌کرد. هرچند موفقیت DENDRAL در قانع‌کردن جامعهٔ پژوهش هوش مصنوعی به اهمیت بازنمایی دانش نقش‌آفرین بود، صورت‌های بازنمایی مورد استفاده در DENDRAL به‌شدت خاص حوزهٔ شیمی‌اند.

با گذشت زمان، پژوهشگران به صورت‌های بازنمایی دانش استانداردشده و هستی‌شناسی‌هایی علاقه‌مند شدند که می‌توانستند در ایجاد سامانه‌های خبرهٔ تازه کمک کنند. این آن‌ها را به قلمرویی کشاند که پیش‌تر توسط فیلسوفان علم و زبان کاوش شده بود. انضباط تحمیل‌شده در هوش مصنوعی توسط نیاز به «کارکردن» نظریه‌ها به پیشرفت سریع‌تر و عمیق‌تری انجامیده تا زمانی که این مسائل قلمرو انحصاری فلسفه بودند (هرچند در مواردی به اختراع مکرر چرخ نیز انجامیده).

اما تا چه حد می‌توان به دانش خبره اعتماد کرد؟ در ۱۹۵۵، پل میل (نیز گروو و میل، ۱۹۹۶ را ببینید) فرایندهای تصمیم‌گیری متخصصان آموزش‌دیده را در تکلیف‌های ذهنی مانند پیش‌بینی موفقیت دانشجو در برنامهٔ آموزشی یا بازگشت به جرم جنایتکار مطالعه کرد. در ۱۹ از ۲۰ مطالعه‌ای که بررسی کرد، میل یافت الگوریتم‌های یادگیری آماری ساده (مانند رگرسیون خطی یا بیز ساده‌لوح) بهتر از متخصصان پیش‌بینی می‌کنند. تتلاک (۲۰۱۷) نیز دانش خبره را مطالعه می‌کند و در موارد دشوار آن را ناکافی می‌یابد. سرویس آزمون آموزشی از ۱۹۹۹ برای نمره‌دهی به میلیون‌ها سؤال انشایی در آزمون GMAT از برنامهٔ خودکار استفاده کرده. برنامه در ۹۷٪ موارد با نمره‌دهندگان انسانی موافق است، تقریباً همان سطحی که دو نمره‌دهندهٔ انسانی با هم توافق دارند (بورشتاین و همکاران، ۲۰۰۱). (این بدان معنا نیست برنامه مقاله‌ها را می‌فهمد، فقط اینکه می‌تواند خوب‌ها را از بدها تقریباً به خوبی نمره‌دهندگان انسانی تشخیص دهد.)

ایجاد رده‌بندی‌ها یا طبقه‌بندی‌های جامع به دوران باستان بازمی‌گردد. ارسطو (۳۸۴–۳۲۲ پیش از میلاد) به‌شدت بر طرح‌های طبقه‌بندی و رده‌بندی تأکید کرد. «ارغنون» او، مجموعه‌آثاری دربارهٔ منطق که پس از مرگش توسط شاگردانش گردآوری شد، رساله‌ای به نام «مقوله‌ها» شامل می‌شد که در آن می‌کوشید آنچه را اکنون هستی‌شناسی بالایی می‌نامیم بسازد. او همچنین مفاهیم جنس و گونه را برای طبقه‌بندی سطح‌پایین‌تر معرفی کرد. سامانهٔ کنونی طبقه‌بندی زیستی ما، شامل استفاده از «نام‌گذاری دوجمله‌ای» (طبقه‌بندی از طریق جنس و گونه به معنای فنی)، توسط زیست‌شناس سوئدی کارلوس لینه، یا کارل فون لینه (۱۷۰۷–۱۷۷۸)، اختراع شد. مسائل مرتبط با انواع طبیعی و مرزهای رده‌ای نادقیق توسط ویتگنشتاین (۱۹۵۳)، کواین (۱۹۵۳)، لیکاف (۱۹۸۷) و شوارتز (۱۹۷۷)، میان دیگران، پرداخته شده.

برای بحث دربارهٔ بازنمایی‌های شبکه عصبی عمیق واژه‌ها و مفاهیم که از برخی مشکلات هستی‌شناسی صلب و انعطاف‌ناپذیر می‌گریزند اما دقت را نیز قربانی می‌کنند، فصل ۲۴ را ببینید. هنوز نمی‌دانیم بهترین راه ترکیب مزایای شبکه‌های عصبی و معناشناسی منطقی برای بازنمایی چیست.

علاقه به هستی‌شناسی‌های بزرگ‌مقیاس رو به افزایش است، چنان‌که «راهنمای هستی‌شناسی‌ها» (استاب، ۲۰۰۴) ثبت می‌کند. پروژهٔ OPENCYC (لنات و گوها، ۱۹۹۰؛ ماتوزک و همکاران، ۲۰۰۶) هستی‌شناسی ۱۵۰,۰۰۰-مفهومی منتشر کرده، با هستی‌شناسی بالایی مشابه شکل ۱۰.۱ و نیز مفاهیم خاصی مانند «نمایشگر OLED» و «آی‌فون»، که نوعی «تلفن همراه» است، که به نوبهٔ خود نوعی «الکترونیک مصرفی»، «تلفن»، «دستگاه ارتباط بی‌سیم» و مفاهیم دیگر است. پروژهٔ NEXTKB منابع CYC و دیگر منابع شامل FrameNet و WordNet را در پایگاه دانشی با نزدیک به ۳ میلیون واقعیت بسط می‌دهد و موتور استدلالی به نام FIRE همراه آن فراهم می‌کند (فورباس و همکاران، ۲۰۱۰).

پروژهٔ DBPEDIA دادهٔ ساخت‌یافته را از ویکی‌پدیا، به‌ویژه از Infoboxها: جفت‌های ویژگی/مقدار که همراه بسیاری از مقاله‌های ویکی‌پدیا می‌آیند، استخراج می‌کند (وو و ولد، ۲۰۰۸؛ بیزر و همکاران، ۲۰۰۷). از ۲۰۱۵، DBPEDIA در نسخهٔ انگلیسی به تنهایی ۴۰۰ میلیون واقعیت دربارهٔ ۴ میلیون شیء شامل می‌شد؛ شمردن همهٔ ۱۱۰ زبان ۱/۵ میلیارد واقعیت می‌دهد (من و همکاران، ۲۰۱۵).

گروه کاری IEEE با شمارهٔ P1600.1 SUMO، «هستی‌شناسی بالایی ادغام‌شدهٔ پیشنهادی» (نایلز و پیرس، ۲۰۰۱؛ پیرس و نایلز، ۲۰۰۲)، را با حدود ۱۰۰۰ اصطلاح در هستی‌شناسی بالایی و پیوندهایی به بیش از ۲۰,۰۰۰ اصطلاح حوزه‌ویژه ایجاد کرد. استوفل و همکاران (۱۹۹۷) الگوریتم‌هایی برای مدیریت کارآمد هستی‌شناسی بسیار بزرگ توصیف می‌کنند. مروری بر فنون استخراج دانش از صفحات وب توسط اتزیونی و همکاران (۲۰۰۸) داده شده.

در وب، زبان‌های بازنمایی در حال پدیدارشدن‌اند. RDF (بریکلی و گوها، ۲۰۰۴) امکان ادعا در قالب سه‌گانه‌های رابطه‌ای را می‌دهد و ابزارهایی برای تکامل معنای نام‌ها در طول زمان فراهم می‌کند. OWL (اسمیت و همکاران، ۲۰۰۴) منطق توصیفی است که از استنتاج‌های روی این سه‌گانه‌ها پشتیبانی می‌کند. تاکنون، به نظر می‌رسد استفاده با پیچیدگی بازنمایشی نسبت معکوس دارد: قالب‌های سنتی HTML و CSS بیش از ۹۹٪ محتوای وب را تشکیل می‌دهند، پس از آن ساده‌ترین طرح‌های بازنمایی، مانند RDFa (آدیدا و بیربک، ۲۰۰۸) و ریزقالب‌ها (خاره، ۲۰۰۶؛ پاتل-شنایدر، ۲۰۱۴) که از نشانه‌گذاری HTML و XHTML برای افزودن ویژگی به متن صفحات وب استفاده می‌کنند. استفاده از هستی‌شناسی‌های پیچیدهٔ RDF و OWL هنوز گسترده نیست و چشم‌انداز کامل وب معنایی (برنرز-لی و همکاران، ۲۰۰۱) محقق نشده. کنفرانس‌های هستی‌شناسی صوری در سامانه‌های اطلاعاتی (FOIS) هم هستی‌شناسی‌های عمومی و هم حوزه‌ویژه را پوشش می‌دهند.

رده‌بندی استفاده‌شده در این فصل توسط نویسندگان توسعه یافته و تا حدی بر تجربهٔ آن‌ها در پروژهٔ CYC و تا حدی بر کار هوانگ و شوبرت (۱۹۹۳) و دیویس (۱۹۹۰، ۲۰۰۵) مبتنی است. بحث الهام‌بخش پروژهٔ عمومی بازنمایی دانش عقل سلیمی در «بیانیهٔ فیزیک ساده‌لوحانه» هیس (۱۹۷۸، ۱۹۸۵) آمده.

هستی‌شناسی‌های عمیق موفق در حوزهٔ خاص شامل پروژهٔ هستی‌شناسی ژن (کنسرسیوم هستی‌شناسی ژن، ۲۰۰۸) و زبان نشانه‌گذاری شیمیایی (موری-راست و همکاران، ۲۰۰۳) است. تردیدها دربارهٔ امکان‌پذیری هستی‌شناسی تکی برای همهٔ دانش توسط داکترو (۲۰۰۱)، گروبر (۲۰۰۴)، هالیوی و همکاران (۲۰۰۹) و اسمیت (۲۰۰۴) بیان شده.

حساب رویدادها توسط کوالسکی و سرگوت (۱۹۸۶) برای مدیریت زمان پیوسته معرفی شد و تغییرهای متعددی داشته (سدری و کوالسکی، ۱۹۹۵؛ شاناهان، ۱۹۹۷) و مرورهایی (شاناهان، ۱۹۹۹؛ مولر، ۲۰۰۶). جیمز آلن بازه‌های زمانی را به همان دلیل معرفی کرد (آلن، ۱۹۸۴) و استدلال کرد بازه‌ها برای استدلال دربارهٔ رویدادهای بسط‌یافته و هم‌زمان بسیار طبیعی‌تر از وضعیت‌ها هستند. در فان لامبالگن و هام (۲۰۰۵) می‌بینیم منطق رویدادها چگونه بر زبانی که برای صحبت دربارهٔ رویدادها استفاده می‌کنیم نگاشت می‌شود. بدیلی برای حساب‌های رویداد و وضعیت، «حساب روان» (Thielscher، 1999) است که واقعیت‌هایی را که حالت‌ها از آن‌ها ترکیب می‌شوند شیءسازی می‌کند.

پیتر لادکین (۱۹۸۶) بازه‌های زمانی «مقعر» (بازه‌های با شکاف — اساساً اتحاد بازه‌های زمانی «محدب» معمولی) را معرفی کرد و فنون جبر مجرد ریاضی را بر بازنمایی زمان اعمال کرد. آلن (۱۹۹۱) به‌صورت نظام‌مند گسترهٔ وسیعی از فنون موجود برای بازنمایی زمان را بررسی می‌کند؛ فان بیک و مانچاک (۱۹۹۶) الگوریتم‌های استدلال زمانی را تحلیل می‌کنند. هم‌گونی‌های معناداری میان هستی‌شناسی رویدادمحورِ داده‌شده در این فصل و تحلیلی از رویدادها از فیلسوف دونالد دیویدسون (۱۹۸۰) هست. تاریخ‌های موجود در هستی‌شناسی مایعات پت هیس (۱۹۸۵) و وقایع‌نگاری‌های نظریهٔ برنامه‌های مک‌درموت (۱۹۸۵) نیز تأثیرات مهمی بر حوزه و بر این فصل بودند.

پرسش دربارهٔ وضعیت هستی‌شناختی مواد تاریخچهٔ بلندی دارد. افلاطون پیشنهاد کرد مواد موجودیت‌های انتزاعیِ کاملاً متمایز از اشیای فیزیکی‌اند؛ او می‌گفت MadeOf(Butter3, Butter) به‌جای Butter3 ∈ Butter. این به سلسله‌مراتب ماده‌ای می‌انجامد که در آن، برای نمونه، UnsaltedButter ماده‌ای خاص‌تر از Butter است. موضع اتخاذشده در این فصل، که در آن مواد رده‌هایی از اشیا هستند، توسط ریچارد مونتاگو (۱۹۷۳) دفاع شد. در پروژهٔ CYC نیز اتخاذ شده. کوپلند (۱۹۹۳) حملهٔ جدی، اما نه شکست‌ناپذیر، برپا می‌کند.

رویکرد جایگزینِ ذکرشده در فصل، که در آن کره شیئی واحد شامل همهٔ اشیای کرانه‌ای جهان است، در اصل توسط منطق‌دان لهستانی لِشنفسکی (۱۹۱۶) پیشنهاد شد. «مره‌شناسی» او (این نام از واژهٔ یونانی «جزء» گرفته شده) از رابطهٔ جزء–کل به‌عنوان جانشین نظریهٔ مجموعه‌های ریاضی استفاده می‌کرد، با هدف حذف موجودیت‌های انتزاعی مانند مجموعه‌ها. شرح خواندنی‌تری از این ایده‌ها توسط لئونارد و گودمن (۱۹۴۰) داده شده و «ساختار ظاهر» گودمن (۱۹۷۷) ایده‌ها را بر مسائل گوناگون بازنمایی دانش اعمال می‌کند.

هرچند برخی جنبه‌های رویکرد مره‌شناختی ناشیانه‌اند — برای نمونه، نیاز به سازوکار وراثت جداگانه بر اساس روابط جزء–کل — این رویکرد حمایت کواین (۱۹۶۰) را جلب کرد. هری بانت (۱۹۸۵) تحلیل گسترده‌ای از استفاده‌اش در بازنمایی دانش ارائه داده. کاساتی و وارزی (۱۹۹۹) اجزا، کل‌ها و نظریهٔ عمومی مکان‌های فضایی را پوشش می‌دهند.

سه رویکرد اصلی برای مطالعهٔ اشیای ذهنی هست. رویکرد اتخاذشده در این فصل، مبتنی بر منطق موجهات و جهان‌های ممکن، رویکرد کلاسیک از فلسفه است (هینتیکا، ۱۹۶۲؛ کریپکه، ۱۹۶۳؛ هیوز و کرسول، ۱۹۹۶). کتاب «استدلال دربارهٔ دانش» (فاگین و همکاران، ۱۹۹۵) مقدمهٔ کاملی فراهم می‌کند و گوردون و هابز (۲۰۱۷) «نظریهٔ صوری روان‌شناسی عقل سلیمی» ارائه می‌دهند.

رویکرد دوم نظریهٔ مرتبهٔ اولی است که در آن اشیای ذهنی روان‌اند. دیویس (۲۰۰۵) و دیویس و مورگنسترن (۲۰۰۵) این رویکرد را توصیف می‌کنند. بر صورت‌بندی جهان‌های ممکن تکیه دارد و بر کار رابرت مور (۱۹۸۰، ۱۹۸۵) بنا می‌شود.

رویکرد سوم نظریهٔ نحوی است، که در آن اشیای ذهنی با رشته‌های نویسه‌ای بازنمایی می‌شوند. رشته فقط عبارت پیچیده‌ای است که فهرستی از نمادها را نشان می‌دهد، پس CanFly(Clark) را می‌توان با فهرست نمادهای [C,a,n,F,l,y,(,C,l,a,r,k,)] بازنمایی کرد. نظریهٔ نحوی اشیای ذهنی نخستین بار با ژرفا توسط کاپلان و مونتاگو (۱۹۶۰) مطالعه شد، که نشان دادند اگر با دقت مدیریت نشود به پارادوکس‌ها می‌انجامد. ارنی دیویس (۱۹۹۰) مقایسهٔ عالی نظریه‌های نحوی و موجهاتی دانش ارائه می‌دهد. پنوئلی (۱۹۷۷) منطق زمانی‌ای توصیف می‌کند که برای استدلال دربارهٔ برنامه‌ها استفاده می‌شود؛ کاری که جایزهٔ تورینگ برایش گرفت و وردی (۱۹۹۶) آن را بسط داد. لیتمن و همکاران (۲۰۱۷) نشان می‌دهند منطق زمانی می‌تواند زبان خوبی برای مشخص‌کردن هدف‌ها برای ربات یادگیری تقویتی باشد، به‌شیوه‌ای که برای انسان آسان‌قابل‌مشخص‌کردن است و به محیط‌های متفاوت خوب تعمیم می‌یابد.

فیلسوف یونانی پورفیری (حدود ۲۳۴–۳۰۵ میلادی)، در تفسیر «مقوله‌ها»ی ارسطو، آنچه را شاید نخستین شبکهٔ معنایی باشد رسم کرد. چارلز اس. پیرس (۱۹۰۹) گراف‌های وجودی را به‌عنوان نخستین صورت‌بندی شبکهٔ معنایی با استفاده از منطق مدرن توسعه داد. راس کویلیان (۱۹۶۱)، با انگیزهٔ علاقه به حافظهٔ انسانی و پردازش زبان، کار شبکه‌های معنایی را در هوش مصنوعی آغاز کرد. مقالهٔ تأثیرگذار ماروین مینسکی (۱۹۷۵) نسخه‌ای از شبکه‌های معنایی به نام «قاب»ها (Frames) ارائه کرد؛ قاب بازنمایی شیء یا رده‌ای بود، با ویژگی‌ها و روابط با اشیا یا رده‌های دیگر.

پرسش معناشناسی به‌طور حاد دربارهٔ شبکه‌های معنایی کویلیان (و دیگرانی که رویکردش را دنبال کردند) با «پیوندهای IS-A» فراگیر و بسیار مبهم‌شان پیش آمد. مقالهٔ نامدار بیل وودز (۱۹۷۵) با عنوان «در یک پیوند چیست؟» توجه پژوهشگران هوش مصنوعی را به نیاز به معناشناسی دقیق در صورت‌های بازنمایی دانش جلب کرد. ران براچمن (۱۹۷۹) این نکته را بسط داد و راه‌حل‌هایی پیشنهاد کرد. «منطق قاب‌ها» پاتریک هیس (۱۹۷۹) حتی عمیق‌تر برید و ادعا کرد «بیشترِ "قاب‌ها" فقط نحو تازه‌ای برای بخش‌هایی از منطق مرتبهٔ اول است.» «معناشناسی تارسکی، یا، بدون دلالت هیچ نمادی نیست!» درو مک‌درموت (۱۹۷۸) استدلال کرد رویکرد نظریه‌مدل به معناشناسیِ استفاده‌شده در منطق مرتبهٔ اول باید بر همهٔ صورت‌های بازنمایی دانش اعمال شود. این همچنان ایده‌ای جنجالی است؛ قابل توجه است که خودِ مک‌درموت در «نقد عقل ناب» (مک‌درموت، ۱۹۸۷) موضعش را معکوس کرده. سلمن و لوسک (۱۹۹۳) پیچیدگی وراثت با استثناها را بحث می‌کنند و نشان می‌دهند در بیشتر صورت‌بندی‌ها NP-کامل است.

منطق‌های توصیف به‌عنوان زیرمجموعهٔ مفیدی از منطق مرتبهٔ اول توسعه یافتند که استنتاج برایشان از نظر محاسباتی حل‌پذیر است. هکتور لوسک و ران براچمن (۱۹۸۷) نشان دادند برخی کاربردهای فصل و نقیض عمدتاً مسئول ناحل‌پذیری استنتاج منطقی‌اند. این به فهم بهتر تعامل میان پیچیدگی و بیانگری در سامانه‌های استدلالی انجامید. کالوانزه و همکاران (۱۹۹۹) وضعیت هنر را خلاصه می‌کنند و بادر و همکاران (۲۰۰۷) راهنمای جامع منطق توصیف ارائه می‌دهند.

سه صورت‌بندی اصلی برای مدیریت استنتاج غیریکنواخت — تحدید (مک‌کارتی، ۱۹۸۰)، منطق پیش‌فرض (رایتر، ۱۹۸۰) و منطق غیریکنواخت موجهاتی (مک‌درموت و دویل، ۱۹۸۰) — همگی در یک شمارهٔ ویژه از ژورنال هوش مصنوعی معرفی شدند. دلگراند و شاوب (۲۰۰۳) شایستگی تغییرها را با ۲۵ سال نگاه به گذشته بحث می‌کنند. برنامه‌نویسی مجموعه‌پاسخ را می‌توان بسط نقیض-به‌مثابه-شکست یا پالایشی از تحدید دید؛ نظریهٔ زیربنایی معناشناسی مدل پایدار توسط گلفوند و لیفشیتز (۱۹۸۸) معرفی شد و سامانه‌های پیشرو برنامه‌نویسی مجموعه‌پاسخ DLV (ایتر و

همکاران، ۱۹۹۸) و SMODELS (نیملا و همکاران، ۲۰۰۰) هستند. لیفشیتز (۲۰۰۱) استفاده از برنامه‌نویسی مجموعه‌پاسخ برای برنامه‌ریزی را بحث می‌کند. بروکا و همکاران (۱۹۹۷) مرور خوبی از رویکردهای گوناگون منطق غیریکنواخت می‌دهند. کلارک (۱۹۷۸) رویکرد نقیض-به‌مثابه-شکست به برنامه‌نویسی منطقی و تکمیل کلارک را پوشش می‌دهد. انواع سامانه‌های استدلال غیریکنواخت مبتنی بر برنامه‌نویسی منطقی در مجموعه‌مقاله‌های کنفرانس‌های برنامه‌نویسی منطقی و استدلال غیریکنواخت (LPNMR) ثبت شده‌اند.

مطالعهٔ سامانه‌های نگه‌داری صدق با سامانه‌های TMS (دویل، ۱۹۷۹) و RUP (مک‌آلیستر، ۱۹۸۰) آغاز شد که هر دو اساساً JTMS بودند. فورباس و د کلیر (۱۹۹۳) با ژرفا توضیح می‌دهند چگونه TMSها را می‌توان در کاربردهای هوش مصنوعی استفاده کرد. نایاک و ویلیامز (۱۹۹۷) نشان می‌دهند چگونه TMS افزایشی کارآمدی به نام ITMS برنامه‌ریزی عملیات فضاپیمای ناسا را در زمان واقعی ممکن می‌کند.

این فصل نمی‌توانست هر حوزهٔ بازنمایی دانش را با ژرفا پوشش دهد. سه موضوع اصلی حذف‌شده چنین‌اند:

فیزیک کیفی (Qualitative Physics): فیزیک کیفی زیرحوزه‌ای از بازنمایی دانش است که به‌طور خاص به ساختن نظریهٔ منطقی و غیرعددی اشیای فیزیکی و فرایندها مربوط است. این اصطلاح توسط یوهان د کلیر (۱۹۷۵) ابداع شد، هرچند می‌توان گفت این پروژه در BUILD فاهلمن (۱۹۷۴)، برنامه‌ریز پیچیده‌ای برای ساخت برج‌های پیچیده از بلوک‌ها، آغاز شد. فاهلمن در فرایند طراحی‌اش کشف کرد بیشتر تلاش (۸۰٪، طبق تخمین او) صرف مدل‌کردن فیزیک جهان بلوک‌ها برای محاسبهٔ پایداری زیرمجموعه‌های گوناگون بلوک‌ها می‌شد، نه برنامه‌ریزی به خودی خود. او فرایند فرضیِ فیزیک-ساده‌لوحانه‌ای ترسیم می‌کند تا توضیح دهد چرا کودکان خردسال می‌توانند مسائل شبیه BUILD را بدون دسترسی به حساب ممیز شناور پرسرعتِ استفاده‌شده در مدل‌سازی فیزیکی BUILD حل کنند. هیس (۱۹۸۵) از «تاریخ»ها — برش‌های چهاربعدی فضا-زمان مشابه رویدادهای دیویدسون — برای ساختن فیزیک ساده‌لوحانهٔ نسبتاً پیچیده‌ای از مایعات استفاده می‌کند. دیویس (۲۰۰۸) به‌روزرسانی هستی‌شناسی مایعات می‌دهد که ریختن مایعات در ظرف‌ها را توصیف می‌کند.

د کلیر و براون (۱۹۸۵)، کن فورباس (۱۹۸۵) و بنجامین کوپیرس (۱۹۸۵) به‌طور مستقل و تقریباً هم‌زمان سامانه‌هایی توسعه دادند که می‌توانند دربارهٔ سامانهٔ فیزیکی بر اساس انتزاع‌های کیفی معادلات زیربنایی استدلال کنند. فیزیک کیفی به‌زودی تا جایی توسعه یافت که تحلیل مجموعه‌ای چشمگیر از سامانه‌های فیزیکی پیچیده ممکن شد (ییپ، ۱۹۹۱). فنون کیفی برای ساخت طرح‌های نوآورانه برای ساعت‌ها، برف‌پاک‌کن‌ها و راه‌روهای شش‌پا استفاده شده (سوبرامانیان و وانگ، ۱۹۹۴). مجموعهٔ «خواندنی‌هایی در استدلال کیفی دربارهٔ سامانه‌های فیزیکی» (ولد و د کلیر، ۱۹۹۰)، مقالهٔ دایرةالمعارفی کوپیرس (۲۰۰۱) و مقالهٔ راهنمای دیویس (۲۰۰۷) مقدمه‌های خوبی برای حوزه فراهم می‌کنند.

استدلال فضایی (Spatial Reasoning): استدلال لازم برای ناوبری در جهان وومپوس در مقایسه با ساختار فضایی غنی جهان واقعی پیش‌پاافتاده است. نخستین تلاش جدی برای درک استدلال عقل‌سلیمی دربارهٔ فضا در کار ارنست دیویس (۱۹۸۶، ۱۹۹۰) ظاهر می‌شود. حساب اتصال ناحیهٔ کوهن و همکاران (۱۹۹۷) شکلی از استدلال فضایی کیفی را پشتیبانی می‌کند و به انواع تازه‌ای از سامانه‌های اطلاعات جغرافیایی انجامیده؛ نیز (دیویس، ۲۰۰۶) را ببینید. مانند فیزیک کیفی، عامل می‌تواند، به اصطلاح، راه درازی برود بدون توسل به بازنمایی متریک کامل.

استدلال روان‌شناختی (Psychological Reasoning): استدلال روان‌شناختی شامل توسعهٔ روان‌شناسی کاری برای عامل‌های مصنوعی است که در استدلال دربارهٔ خودشان و عامل‌های دیگر استفاده می‌شود. این اغلب بر اصطلاحاً «روان‌شناسی عامیانه» مبتنی است، نظریه‌ای که باور می‌رود انسان‌ها به‌طور کلی در استدلال دربارهٔ خودشان و دیگر انسان‌ها استفاده می‌کنند. وقتی پژوهشگران هوش مصنوعی عامل‌های مصنوعی‌شان را با نظریه‌های روان‌شناختی برای استدلال دربارهٔ عامل‌های دیگر فراهم می‌کنند، نظریه‌ها اغلب بر توصیف پژوهشگران از طراحی خودِ عامل‌های منطقی مبتنی‌اند. استدلال روان‌شناختی در حال حاضر در بافت فهم زبان طبیعی مفیدترین است، جایی که دریافتن نیات گوینده اهمیتی فوق‌العاده دارد.

مینکر (۲۰۰۱) مقاله‌های پژوهشگران برجسته بازنمایی دانش را گردآوری می‌کند و ۴۰ سال کار در حوزه را خلاصه می‌کند. مجموعه‌مقاله‌های کنفرانس‌های بین‌المللی اصول بازنمایی دانش و استدلال (مشهور به KR)، همایش برتر در این حوزه است.

پانوشت‌ها:

[1] تبدیل گزاره به شیء «شیءسازی» (Reification) نامیده می‌شود، از واژهٔ لاتین res، یا چیز. جان مک‌کارتی اصطلاح «thingification» را پیشنهاد کرد، اما هرگز جا نیفتاد.

[2] وقتی از زیست‌شناس جی. بی. اس. هالدین پرسیده شد از مطالعهٔ طبیعت دربارهٔ خالق چه می‌توان استنتاج کرد، گفت «علاقهٔ بی‌اندازه به سوسک‌ها».

[3] اصطلاح‌های «رویداد» و «کنش» را می‌توان به‌جای یکدیگر استفاده کرد — هر دو یعنی «چیزی که می‌تواند رخ دهد».

[4] نسخهٔ ما بر شاناهان (۱۹۹۹) مبتنی است، اما با برخی تغییرها.

[5] چند سامانهٔ اولیه در تمایز میان ویژگی‌های اعضای رده و ویژگی‌های خودِ رده شکست خوردند. این می‌تواند مستقیماً به ناسازگاری‌ها انجامد، چنان‌که درو مک‌درموت (۱۹۷۶) در مقاله‌اش «هوش مصنوعی با حماقت طبیعی ملاقات می‌کند» اشاره کرد. مشکل رایج دیگر استفاده از پیوندهای IsA هم برای روابط زیرمجموعه و هم عضویت، در مطابقت با کاربرد انگلیسی بود: «گربه پستاندار است» و «فیفی گربه است». برای این مسائل تمرین NATS.۱۰ را ببینید.

[6] توجه کنید زبان اجازه نمی‌دهد صرفاً بیان کنیم مفهومی یا رده‌ای زیرمجموعهٔ مفهوم دیگری است. این سیاست عمدی است: زیرشمولی میان رده‌ها باید از برخی جنبه‌های توصیف‌های رده‌ها قابل اشتقاق باشد. اگر نه، چیزی از توصیف‌ها گم شده.

[7] CLASSIC آزمون زیرشمولی کارآمدی در عمل فراهم می‌کند، اما زمان اجرای بدترین حالت نمایی است.

[8] به یاد آورید یکنواختی ایجاب می‌کند همهٔ جمله‌های مستلزم پس از افزودن جمله‌های تازه به KB مستلزم بمانند. یعنی اگر $KB \models \alpha$ آنگاه $KB \wedge \beta \models \alpha$.

[9] برای فرض جهان بسته، مدلی بر مدل دیگر ترجیح دارد اگر اتم‌های صادق کمتری داشته باشد — یعنی مدل‌های ترجیحی مدل‌های کمینه‌اند. پیوند طبیعی میان فرض جهان بسته و KBهای بند معین هست، زیرا نقطهٔ ثابتِ رسیده‌شده توسط زنجیره‌سازی پیشرو روی KBهای بند معین مدل کمینهٔ یکتاست. برای این نکته صفحهٔ ۲۳۱ را ببینید.

[10] بازبینی باور اغلب با به‌روزرسانی باور مقایسه می‌شود، که وقتی رخ می‌دهد پایگاه دانش برای بازتاب تغییری در جهان بازبینی می‌شود نه اطلاعات تازه‌ای دربارهٔ جهان ثابت. به‌روزرسانی باور بازبینی باور را با استدلال دربارهٔ زمان و تغییر ترکیب می‌کند؛ همچنین با فرایند پالایشِ توصیف‌شده در فصل ۱۴ مرتبط است.

فصل ۱۱: برنامه‌ریزی خودکار (Automated Planning)

در این فصل می‌بینیم عامل چگونه می‌تواند از ساختار مسئله بهره بگیرد تا برنامه‌های کنشِ پیچیده را کارآمد بسازد.

برنامه‌ریزی مسیر کنش، نیاز کلیدی عامل هوشمند است. بازنمایی درست کنش‌ها و حالت‌ها و الگوریتم‌های درست می‌توانند این کار را آسان‌تر کنند. در بخش ۱۱.۱ زبان بازنمایی عاملِ عمومی برای مسائل برنامه‌ریزی معرفی می‌کنیم که می‌تواند به‌طور طبیعی و موجز گسترهٔ وسیعی از حوزه‌ها را بازنمایی کند، کارآمد به مسائل بزرگ مقیاس شود، و برای حوزهٔ تازه به اکتشافی‌های موردی (Ad Hoc) نیاز نداشته باشد. بخش ۱۱.۴ زبان بازنمایی را برای اجازه دادن به کنش‌های سلسله‌مراتبی بسط می‌دهد و به ما امکان می‌دهد مسائل پیچیده‌تری را هدف بگیریم. الگوریتم‌های کارآمد برنامه‌ریزی را در بخش ۱۱.۲ و اکتشافی‌های آن‌ها را در بخش ۱۱.۳ پوشش می‌دهیم. در بخش ۱۱.۵ حوزه‌های تا حدی قابل مشاهده و ناجبرگرایانه را در نظر می‌گیریم و در بخش ۱۱.۶ بار دیگر زبان را برای پوشش مسائل زمان‌بندی با قیدهای منبع بسط می‌دهیم. این ما را به برنامه‌ریزهایی که در جهان واقعی برای برنامه‌ریزی و زمان‌بندی عملیات فضاپیماها، کارخانه‌ها و لشکرکشی‌های نظامی استفاده می‌شوند نزدیک‌تر می‌کند.

۱۱.۱ تعریف برنامه‌ریزی کلاسیک

برنامه‌ریزی کلاسیک (Classical Planning) به‌عنوان تکلیف یافتن توالی کنشی برای انجام هدفی در محیط گسسته، جبرگرایانه، ایستا و کاملاً قابل مشاهده تعریف می‌شود. دو رویکرد به این تکلیف دیده‌ایم: عامل حل‌کنندهٔ مسئلهٔ فصل ۳ و عامل منطقی گزاره‌ای ترکیبی فصل ۷. هر دو دو محدودیت مشترک دارند. نخست، هر دو برای هر حوزهٔ تازه به اکتشافی‌های موردی نیاز دارند: تابع ارزیابی اکتشافی برای جستجو، و کد دست‌نوشته برای عامل وومپوس ترکیبی. دوم، هر دو نیاز به بازنمایی صریح فضای حالتی به‌شکل نمایی بزرگ دارند. برای نمونه، در مدل منطق گزاره‌ای جهان وومپوس، اصل‌موضوعهٔ حرکت یک گام به جلو باید برای هر چهار جهت‌گیری عامل، T گام زمانی و n^2 مکان کنونی تکرار می‌شد.

در پاسخ به این محدودیت‌ها، پژوهشگران برنامه‌ریزی روی بازنمایی عاملی با خانواده‌ای از زبان‌ها به نام PDDL، «زبان تعریف حوزهٔ برنامه‌ریزی» (Planning Domain Definition Language) (غَلّاب و همکاران، ۱۹۹۸)، سرمایه‌گذاری کرده‌اند که به ما اجازه می‌دهد همهٔ $4Tn^2$ کنش را با یک «طرح کنش» (Action Schema) تکی بیان کنیم و به دانش حوزه‌ویژه نیاز ندارد. PDDL پایه می‌تواند حوزه‌های برنامه‌ریزی کلاسیک را مدیریت کند و بسط‌ها می‌توانند حوزه‌های غیرکلاسیکِ پیوسته، تا حدی قابل مشاهده، هم‌زمان و چندعاملی را مدیریت کنند. نحو PDDL بر لیسپ مبتنی است، اما آن را به شکلی ترجمه می‌کنیم که با نماد استفاده‌شده در این کتاب همخوان باشد.

در PDDL، «حالت» (State) به‌صورت عطفی از روان‌های اتمی زمینی بازنمایی می‌شود. به یاد آورید «زمینی» یعنی بدون متغیر، «روان» یعنی جنبه‌ای از جهان که در طول زمان تغییر می‌کند، و «اتمی زمینی» یعنی گزارهٔ تکی‌ای هست و اگر آرگومانی دارد، باید ثابت باشد. برای نمونه، Poor ∧ Unknown می‌تواند حالت عامل بدبختی را بازنمایی کند و At(Truck1, Melbourne) ∧ At(Truck2, Sydney) می‌تواند حالتی در مسئلهٔ تحویل بسته باشد. PDDL از معناشناسی پایگاه‌داده استفاده می‌کند: فرض جهان بسته بدان معناست هر روانی که ذکر نشده کاذب است، و فرض نام‌های یکتا بدان معناست Truck1 و Truck2 متمایزند.

روان‌های زیر در حالت مجاز نیستند: At(x,y) (زیرا متغیر دارد)، ¬Poor (زیرا نقیض است)، و At(Spouse(Ali), Sydney) (زیرا از نماد تابع Spouse استفاده می‌کند). وقتی راحت باشد، می‌توانیم عطف روان‌ها را مجموعه‌ای از روان‌ها بدانیم.

«طرح کنش» (Action Schema) خانواده‌ای از کنش‌های زمینی را بازنمایی می‌کند. برای نمونه، این‌جا طرح کنشی برای پرواز هواپیما از مکانی به مکان دیگر است:

```
Action(Fly(p, from, to),
PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
(EFFECT: ¬At(p, from) ∧ At(p, to)
```

طرح از نام کنش، فهرستی از همهٔ متغیرهای استفاده‌شده در طرح، «پیش‌شرط» (Precondition) و «اثر» (Effect) تشکیل شده. پیش‌شرط و اثر هر یک عطف‌هایی از لیترال‌ها (جمله‌های اتمی مثبت یا نقیض‌شده) هستند. می‌توانیم ثابت‌هایی برای مصداق‌کردن متغیرها برگزینیم و کنش زمینی (بدون متغیر) به دست آوریم:

```
Action(Fly(P1, SFO, JFK),
PRECOND: At(P1, SFO) ∧ Plane(P1) ∧ Airport(SFO) ∧ Airport(JFK)
(EFFECT: ¬At(P1, SFO) ∧ At(P1, JFK)
```

کنش زمینی a در حالت s «کاربردپذیر» (Applicable) است اگر s پیش‌شرط a را مستلزم باشد؛ یعنی اگر هر لیترال مثبت در پیش‌شرط در s باشد و هر لیترال نقیض‌شده نباشد.

نتیجهٔ اجرای کنش کاربردپذیر a در حالت s به‌صورت حالت s' تعریف می‌شود که با مجموعهٔ روان‌های تشکیل‌شده بازنمایی می‌شود: با s شروع کن، روان‌هایی را که در اثرات کنش به‌صورت لیترال‌های منفی ظاهر می‌شوند حذف کن (آنچه «فهرست حذف» یا DEL(a) می‌نامیم)، و روان‌هایی را که لیترال‌های مثبت در اثرات کنش‌اند بیفزا (آنچه «فهرست افزودن» یا ADD(a) می‌نامیم):

$$\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a) \qquad (11.1)$$

برای نمونه، با کنش Fly(P1, SFO, JFK)، روان At(P1, SFO) را حذف و روان At(P1, JFK) را می‌افزاییم.

مجموعه‌ای از طرح‌های کنش به‌عنوان تعریف «حوزهٔ برنامه‌ریزی» (Planning Domain) خدمت می‌کند. مسئلهٔ خاص درون حوزه با افزودن حالت آغازین و هدف تعریف می‌شود. حالت آغازین عطفی از روان‌های زمینی است (با کلیدواژهٔ Init در شکل ۱۱.۱ معرفی می‌شود). مانند همهٔ حالت‌ها، فرض جهان بسته استفاده می‌شود، یعنی هر اتمی که ذکر نشده کاذب است. هدف (با Goal معرفی می‌شود) دقیقاً مانند پیش‌شرط است: عطفی از لیترال‌ها (مثبت یا منفی) که ممکن است متغیر داشته باشند. برای نمونه، هدف At(p, SFO) ∧ ¬At(C2, SFO) ∧ At(C1, SFO) به هر حالتی اشاره دارد که در آن محمولهٔ C1 در SFO است اما C2 نیست، و هواپیمایی در SFO هست.

۱۱.۱.۱ حوزهٔ نمونه: حمل‌ونقل بار هوایی

شکل ۱۱.۱ مسئلهٔ حمل‌ونقل بار هوایی شامل بارگیری و بارخالی‌کردن محموله و پروازدادنش از مکانی به مکان دیگر را نشان می‌دهد. مسئله را می‌توان با سه کنش تعریف کرد: Load (بارگیری)، Unload (بارخالی) و Fly (پرواز). کنش‌ها روی دو گزاره اثر می‌گذارند: In(c, p) یعنی محمولهٔ c درون هواپیمای p است، و At(x, a) یعنی شیء x (یا هواپیما یا محموله) در فرودگاه a است. توجه کنید باید مراقبت شود گزاره‌های At به‌درستی نگه‌داری شوند. وقتی هواپیما از فرودگاهی به فرودگاه دیگر پرواز می‌کند، همهٔ محموله‌های درون هواپیما با آن می‌روند. در منطق مرتبهٔ اول سورزدن روی همهٔ اشیای درون هواپیما آسان بود. اما PDDL سور همگانی ندارد، پس به راه‌حل متفاوتی نیاز داریم. رویکردی که استفاده می‌کنیم آن است که بگوییم محموله وقتی در هواپیماست، دیگر هیچ‌جا At نیست؛ محموله فقط وقتی بارخالی شود در فرودگاه تازه At می‌شود. پس At واقعاً یعنی «برای استفاده در مکانِ داده‌شده در دسترس».

```
Init(At(C1, SFO) ∧ At(C2, JFK) ∧ At(P1, SFO) ∧ At(P2, JFK)
      Cargo(C1) ∧ Cargo(C2) ∧ Plane(P1) ∧ Plane(P2) ∧
                  (Airport(JFK) ∧ Airport(SFO) ∧
                      Goal(At(C1, JFK) ∧ At(C2, SFO))
                                 ,Action(Load(c, p, a)
PRECOND: At(c, a) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
                       (EFFECT: ¬At(c, a) ∧ In(c, p)
                                ,Action(Unload(c, p, a)
PRECOND: In(c, p) ∧ At(p, a) ∧ Cargo(c) ∧ Plane(p) ∧ Airport(a)
                       (EFFECT: At(c, a) ∧ ¬In(c, p)
                               ,Action(Fly(p, from, to)
  PRECOND: At(p, from) ∧ Plane(p) ∧ Airport(from) ∧ Airport(to)
                       (EFFECT: ¬At(p, from) ∧ At(p, to)
```

شکل ۱۱.۱ — توصیف PDDL مسئلهٔ برنامه‌ریزی حمل‌ونقل بار هوایی.

برنامهٔ زیر راه‌حل مسئله است:

```
Load(C1, P1, SFO), Fly(P1, SFO, JFK), Unload(C1, P1, JFK), Load(C2, P2, JFK), Fly(P2,]
                                                          [JFK, SFO), Unload(C2, P2, SFO)
```

۱۱.۱.۲ حوزهٔ نمونه: مسئلهٔ لاستیک زاپاس

مسئلهٔ عوض‌کردن لاستیک پنچر را در نظر بگیرید (شکل ۱۱.۲). هدف آن است که لاستیک زاپاسِ خوب به‌درستی روی اکسل خودرو سوار شود، که حالت آغازین لاستیک پنچر روی اکسل و لاستیک زاپاس خوب در صندوق دارد. برای ساده‌نگه‌داشتن، نسخهٔ ما از مسئله انتزاعی است، بدون مهرهٔ چرخ چسبناک یا پیچیدگی‌های دیگر. فقط چهار کنش هست: برداشتن زاپاس از صندوق، برداشتن لاستیک پنچر از اکسل، گذاشتن زاپاس روی اکسل، و رهاکردن خودرو بدون مراقبت یک‌شب. فرض می‌کنیم خودرو در محلهٔ به‌ویژه بدی پارک شده، پس اثر رهاکردن یک‌شبه آن است که لاستیک‌ها ناپدید می‌شوند.

```
Init(Tire(Flat) ∧ Tire(Spare) ∧ At(Flat, Axle) ∧ At(Spare, Trunk))
                                                  Goal(At(Spare, Axle))
                                                 ,Action(Remove(obj, loc)
                                                  PRECOND: At(obj, loc)
                                        (EFFECT: ¬At(obj, loc) ∧ At(obj, Ground)
                                                 ,Action(PutOn(t, Axle)
PRECOND: Tire(t) ∧ At(t, Ground) ∧ ¬At(Flat, Axle) ∧ ¬At(Spare, Axle)
                                     (EFFECT: ¬At(t, Ground) ∧ At(t, Axle)
                                                 ,Action(LeaveOvernight
                                                                :PRECOND
    EFFECT: ¬At(Spare, Ground) ∧ ¬At(Spare, Axle) ∧ ¬At(Spare, Trunk)
    (At(Flat, Ground) ∧ ¬At(Flat, Axle) ∧ ¬At(Flat, Trunk)¬ ∧
```

شکل ۱۱.۲ — مسئلهٔ سادهٔ لاستیک زاپاس.

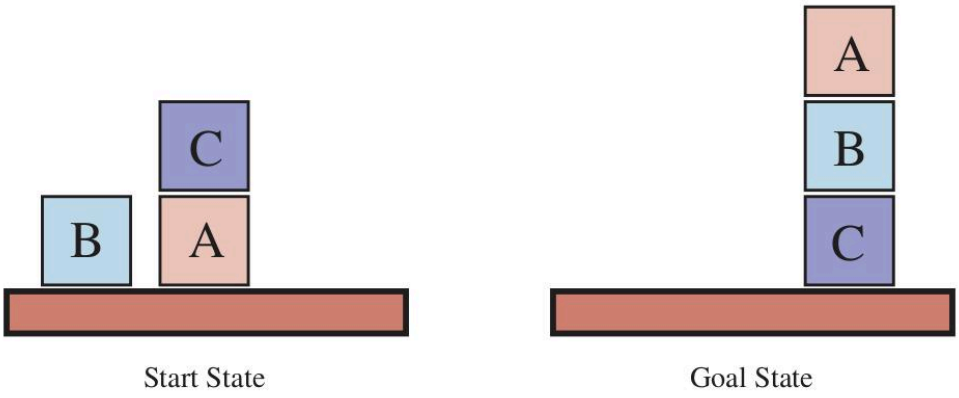
برنامهٔ زیر راه‌حل مسئله است:

```
[Remove(Flat, Axle), Remove(Spare, Trunk), PutOn(Spare, Axle)]
```

۱۱.۱.۳ حوزهٔ نمونه: جهان بلوک‌ها

یکی از نامدارترین حوزه‌های برنامه‌ریزی «جهان بلوک‌ها» (Blocks World) است. این حوزه از مجموعه‌ای بلوک مکعب‌شکل روی میزی به‌اندازهٔ دلخواه بزرگ تشکیل شده.[1] بلوک‌ها را می‌توان روی هم چید، اما فقط یک بلوک می‌تواند مستقیم روی بلوک دیگر قرار گیرد. بازوی ربات می‌تواند بلوکی را بردارد و به موقعیت دیگری ببرد، یا روی میز یا روی بلوک دیگر. بازو فقط می‌تواند هر بار یک بلوک بردارد، پس نمی‌تواند بلوکی را بردارد که بلوک دیگری روی آن است. هدف نمونهٔ معمول، گذاشتن بلوک A روی B و بلوک B روی C است (شکل ۱۱.۳ را ببینید).

از On(b,x) استفاده می‌کنیم تا نشان دهیم بلوک b روی x است، که x یا بلوک دیگر است یا میز. کنش برای حرکت بلوک b از بالای x به بالای y، Move(b,x,y) خواهد بود. حال، یکی از پیش‌شرط‌های حرکت b آن است که هیچ بلوک دیگری روی آن نباشد. در منطق مرتبهٔ اول، این ¬∃x On(x,b) یا به‌طور معادل ∀x ¬On(x,b) می‌شد. PDDL پایه سور اجازه نمی‌دهد، پس در عوض گزارهٔ Clear(x) معرفی می‌کنیم که وقتی چیزی روی x نیست صادق است. (توصیف کامل مسئله در شکل ۱۱.۴ است.)



شکل ۱۱.۳ – نمودار مسئلهٔ جهان بلوک‌ها در شکل ۱۱.۴. (حالت آغاز: B روی میز، A روی میز، C روی A؛ حالت هدف: A روی B، B روی C)

```
Init(On(A,Table) ∧ On(B,Table) ∧ On(C,A)
(Block(A) ∧ Block(B) ∧ Block(C) ∧ Clear(B) ∧ Clear(C) ∧ Clear(Table) ∧
                                                Goal(On(A,B) ∧ On(B,C))
                                                    ,Action(Move(b,x,y)
        ∧ PRECOND: On(b,x) ∧ Clear(b) ∧ Clear(y) ∧ Block(b) ∧ Block(y)
                                            ,(x≠y) ∧ (b≠y) ∧ (b≠x)
                                    (EFFECT: On(b,y) ∧ Clear(x) ∧ ¬On(b,x) ∧ ¬Clear(y)
                                                    ,Action(MoveToTable(b,x)
                                    ,PRECOND: On(b,x) ∧ Clear(b) ∧ Block(b) ∧ Block(x)
                                            (EFFECT: On(b,Table) ∧ Clear(x) ∧ ¬On(b,x)
```

شکل ۱۱.۴ – مسئلهٔ برنامه‌ریزی در جهان بلوک‌ها: ساختن برج سه‌بلوکی. یک راه‌حل توالی [Move(B,Table,C), MoveToTable(C,A), Move(A,Table,B)] است.

کنش Move بلوک b را از x به y می‌برد اگر هر دو b و y Clear باشند. پس از انجام حرکت، b همچنان Clear است اما y نیست. تلاش نخست برای طرح Move چنین است:

```
                                                    ,Action(Move(b,x,y)
                                            ,PRECOND: On(b,x) ∧ Clear(b) ∧ Clear(y)
                                    (EFFECT: On(b,y) ∧ Clear(x) ∧ ¬On(b,x) ∧ ¬Clear(y)
```

متأسفانه، این Clear را وقتی x یا y میز است درست نگه نمی‌دارد. وقتی x میز است، این کنش اثر Clear(Table) دارد، اما میز نباید Clear شود؛ و وقتی y = Table است، پیش‌شرط Clear(Table) دارد، اما میز لازم نیست برای حرکت بلوکی روی آن Clear باشد. برای اصلاح این، دو کار می‌کنیم. نخست، کنش دیگری برای حرکت بلوک b از x به میز معرفی می‌کنیم:

```
                                                    ,Action(MoveToTable(b,x)
                                            ,PRECOND: On(b,x) ∧ Clear(b)
                                    (EFFECT: On(b,Table) ∧ Clear(x) ∧ ¬On(b,x)
```

دوم، تفسیر Clear(x) را «جای خالی روی x برای نگه‌داشتن بلوکی» می‌گیریم. تحت این تفسیر، Clear(Table) همیشه صادق خواهد بود. تنها مشکل آن است که هیچ‌چیز برنامه‌ریز را از استفاده از Move(b,x,Table) به‌جای MoveToTable(b,x) باز نمی‌دارد. می‌توانیم با این مشکل زندگی کنیم — به فضای جستجوی بزرگ‌تر از لازم می‌انجامد، اما به پاسخ‌های نادرست نمی‌انجامد — یا می‌توانیم گزارهٔ Block را معرفی و Block(y) ۸ Block(b) را به پیش‌شرط Move بیفزاییم، چنان‌که در شکل ۱۱.۴.

۱۱.۲ الگوریتم‌های برنامه‌ریزی کلاسیک

توصیف مسئلهٔ برنامه‌ریزی راهی بدیهی برای جستجو از حالت آغازین در فضای حالت‌ها، در جست‌وجوی هدف، فراهم می‌کند. مزیت خوب بازنمایی اعلانی طرح‌های کنش آن است که می‌توانیم از هدف نیز به عقب جستجو کنیم، در جست‌وجوی حالت آغازین (شکل ۱۱.۵ جستجوی پیشرو و پسرو را مقایسه می‌کند). امکان سوم ترجمهٔ توصیف مسئله به مجموعه‌ای از جمله‌های منطقی است که می‌توان الگوریتم استنتاج منطقی را برای یافتن راه‌حل بر آن‌ها اعمال کرد.

۱۱.۲.۱ جستجوی فضای حالت پیشرو برای برنامه‌ریزی

می‌توانیم مسائل برنامه‌ریزی را با اعمال هر یک از الگوریتم‌های جستجوی اکتشافی فصل ۳ یا فصل ۴ حل کنیم. حالت‌های این فضای جستجو حالت‌های زمینی‌اند، که در آن‌ها هر روان یا صادق است یا نه. هدف حالتی است که همهٔ روان‌های مثبت هدف مسئله و هیچ‌یک از روان‌های منفی را نداشته باشد. کنش‌های کاربردپذیر در حالت، Actions(s)، مصداق‌های زمینی طرح‌های کنش‌اند — یعنی کنش‌هایی که متغیرهایشان همه با مقادیر ثابت جایگزین شده‌اند.

برای تعیین کنش‌های کاربردپذیر، حالت کنونی را با پیش‌شرط‌های هر طرح کنش یکسان می‌کنیم. برای هر یکسان‌سازی که به‌طور موفق به جانشینی‌ای می‌انجامد، جانشینی را بر طرح کنش اعمال می‌کنیم تا کنش زمینی بدون متغیر به دست آید. (الزام طرح‌های کنش آن است که هر متغیری در اثر باید در پیش‌شرط نیز ظاهر شود؛ بدین ترتیب، تضمین می‌شود پس از جانشینی متغیری باقی نمی‌ماند.) هر طرح ممکن است به چند طریق یکسان شود. در مثال لاستیک زاپاس (صفحهٔ ۳۴۶)، کنش Remove پیش‌شرط At(obj,loc) دارد که با حالت آغازین به دو طریق مطابقت می‌کند و به دو جانشینی {obj/Flat, loc/Axle} و {obj/Spare, loc/Trunk} می‌انجامد؛ اعمال این جانشینی‌ها دو کنش زمینی می‌دهد. اگر کنشی چند لیترال در پیش‌شرط داشته باشد، هر یک از آن‌ها بالقوه به چند طریق با حالت کنونی قابل مطابقت است.

در آغاز، به نظر می‌رسد فضای حالت ممکن است برای بسیاری از مسائل بیش از حد بزرگ باشد. مسئلهٔ بار هوایی با ۱۰ فرودگاه را در نظر بگیرید، که هر فرودگاه در آغاز ۵ هواپیما و ۲۰ قطعه محموله دارد. هدف انتقال همهٔ محمولهٔ فرودگاه A به فرودگاه B است. راه‌حل ۴۱-گامی برای مسئله هست: ۲۰ قطعه محموله را در یکی از هواپیماهای A بارگیری کن، هواپیما را به B پرواز بده و ۲۰ قطعه را بارخالی کن.

یافتن این راه‌حل ظاهراً سرراست می‌تواند دشوار باشد، زیرا ضریب انشعاب میانگین عظیم است: هر یک از ۵۰ هواپیما می‌تواند به ۹ فرودگاه دیگر پرواز کند و هر یک از ۲۰۰ بسته یا بارخالی می‌شود (اگر بارگیری شده) یا در هر هواپیمای فرودگاهش بارگیری می‌شود (اگر بارخالی شده). پس در هر حالت حداقل ۴۵۰ کنش هست (وقتی همهٔ بسته‌ها در فرودگاه‌های بدون هواپیما هستند) و حداکثر ۱۰,۴۵۰ (وقتی همهٔ بسته‌ها و هواپیماها در همان فرودگاه‌اند). به‌طور میانگین، بگوییم حدود ۲۰۰۰ کنش ممکن در هر حالت است، پس گراف جستجو تا عمق راه‌حل ۴۱-گامی حدود 2000^{41} گره دارد.

بدیهی است حتی این نمونهٔ مسئلهٔ نسبتاً کوچک بدون اکتشافی دقیق ناامیدکننده است. هرچند بسیاری از کاربردهای جهان واقعی برنامه‌ریزی به اکتشافی‌های حوزه‌ویژه تکیه کرده‌اند، معلوم می‌شود (چنان‌که در بخش ۱۱.۳ می‌بینیم) اکتشافی‌های مستقل از حوزهٔ قوی را می‌توان به‌صورت خودکار مشتق کرد؛ همین جستجوی پیشرو را شدنی می‌کند.

۱۱.۲.۲ جستجوی پسرو برای برنامه‌ریزی

در جستجوی پسرو (که جستجوی بازگشتی (Regression Search) نیز نامیده می‌شود) از هدف آغاز می‌کنیم و کنش‌ها را به عقب اعمال می‌کنیم تا توالی گامی بیابیم که به حالت آغازین می‌رسد. در هر گام «کنش‌های مرتبط» (Relevant Actions) را در نظر می‌گیریم (در برابر جستجوی پیشرو که کنش‌های کاربردپذیر را در نظر می‌گیرد). این ضریب انشعاب را به‌شکل چشمگیری کاهش می‌دهد، به‌ویژه در حوزه‌های با کنش‌های ممکن فراوان.

کنش مرتبط کنشی است که اثری دارد با یکی از لیترال‌های هدف یکسان می‌شود، اما اثری ندارد که بخشی از هدف را نقیض کند. برای نمونه، با هدف ¬Famous ∧ Poor، کنشی با اثر تکی Famous مرتبط خواهد بود، اما کنشی با اثر Famous ∧ Poor مرتبط دانسته نمی‌شود: حتی اگر آن کنش ممکن است در نقطه‌ای از برنامه استفاده شود (برای برقراری Famous)، نمی‌تواند در این نقطه از برنامه ظاهر شود، زیرا آنگاه Poor در حالت نهایی ظاهر می‌شد.

اعمال کنش در جهت عقب به چه معناست؟ با هدف g و کنش a، «بازگشت» (Regression) از g روی a توصیف حالتی g′ به ما می‌دهد که لیترال‌های مثبت و منفی‌اش چنین‌اند:

```
POS(g′) = (POS(g) − ADD(a)) ∪ POS(Precond(a))
NEG(g′) = (NEG(g) − DEL(a)) ∪ NEG(Precond(a))
```

یعنی، پیش‌شرط‌ها باید قبلاً برقرار بوده‌اند، یا کنش نمی‌توانست اجرا شده باشد، اما لیترال‌های مثبت/منفیِ افزوده/حذف‌شده توسط کنش لازم نبوده قبلاً صادق باشند.

این معادلات برای لیترال‌های زمینی سرراست‌اند، اما وقتی در g و a متغیرهایی هست کمی مراقبت لازم است. برای نمونه، فرض کنیم هدف تحویل قطعه محمولهٔ خاصی به SFO است: At(C2, SFO). طرح کنش Unload اثر At(c,a) دارد. وقتی آن را با هدف یکسان می‌کنیم، جانشینی {c/C2, a/SFO} را می‌گیریم؛ اعمال آن جانشینی بر طرح، طرح تازه‌ای به ما می‌دهد که ایدهٔ استفاده از هر هواپیمایی که در SFO است را در می‌گیرد:

```
,Action(Unload(C2, p′, SFO)
PRECOND: In(C2, p′) ∧ At(p′, SFO) ∧ Cargo(C2) ∧ Plane(p′) ∧ Airport(SFO)
(EFFECT: At(C2, SFO) ∧ ¬In(C2, p′)
```

در این‌جا p را با متغیر تازه‌ای به نام p' جایگزین کردیم. این مصداقی از استانداردسازی جداسازی نام‌های متغیر است تا تعارضی میان متغیرهای متفاوتی که اتفاقاً همان نام را دارند نباشد (صفحهٔ ۲۸۴ را ببینید). توصیف حالت بازگشت‌خورده هدف تازه‌ای به ما می‌دهد:

```
g′ = In(C2, p′) ∧ At(p′, SFO) ∧ Cargo(C2) ∧ Plane(p′) ∧ Airport(SFO)
```

به‌عنوان مثال دیگر، هدف داشتن کتابی با شماره شابک خاص را در نظر بگیرید: Own(9780134610993). با یک تریلیون شابک ۱۳رقمی و طرح کنش تکی

```
A = Action(Buy(i), PRECOND: ISBN(i), EFFECT: Own(i))
```

جستجوی پیشرو بدون اکتشافی باید شروع به برشمردن ۱۰ میلیارد کنش زمینی Buy کند. اما با جستجوی پسرو، هدف Own(9780134610993) را با اثر Own(i') یکسان می‌کنیم و جانشینی {i'/9780134610993} = θ را می‌گیریم. سپس روی کنش Subst(θ,A) بازگشت می‌کنیم تا توصیف حالت پیشین ISBN(9780134610993) به دست آید. این بخشی از حالت آغازین است، پس راه‌حل داریم و تمام، با در نظر گرفتن فقط یک کنش، نه یک تریلیون.

صورتی‌تر، توصیف هدف g را فرض کن که شامل لیترِال هدف gi و طرح کنش A است. اگر A لیترال اثری e'j داشته باشد که Unify(gi, e'j) = θ و A' = SUBST(θ, A) تعریف کنیم و هیچ اثری در A' نباشد که نقیض لیترِالی در g باشد، آنگاه A' کنش مرتبطی به سوی g است.

برای بیشتر حوزه‌های مسئله، جستجوی پسرو ضریب انشعاب را کمتر از جستجوی پیشرو نگه می‌دارد. اما این واقعیت که جستجوی پسرو از حالت‌های با متغیر استفاده می‌کند نه حالت‌های زمینی، ارائهٔ اکتشافی‌های خوب را دشوارتر می‌کند. این دلیل اصلی ترجیح جستجوی پیشرو توسط اکثریت سامانه‌های کنونی است.

۱۱.۲.۳ برنامه‌ریزی به‌عنوان ارضاپذیری بولی

در بخش 7.7.4 نشان دادیم چگونه برخی بازنویسی هوشمندانهٔ اصل‌موضوعه می‌توانست مسئلهٔ جهان وومپوس را به مسئلهٔ ارضاپذیری منطق گزاره‌ای تبدیل کند که می‌شد به حل‌کنندهٔ ارضاپذیری کارآمد سپرد. برنامه‌ریزهای مبتنی بر SAT مانند SATPLAN با ترجمهٔ توصیف مسئلهٔ PDDL به شکل گزاره‌ای عمل می‌کنند. ترجمه شامل سلسله‌ای از گام‌هاست:

- **کنش‌ها را گزاره‌سازی کن:** برای هر طرح کنش، با جانشینی ثابت‌ها برای هر یک از متغیرها، گزاره‌های زمینی تشکیل بده. پس به‌جای طرح تکی Unload(c,p,a)، برای هر ترکیب محموله، هواپیما و فرودگاه (که این‌جا با زیرنویس نوشته می‌شود) و برای هر گام زمانی (که این‌جا با بالانویس نوشته می‌شود) گزاره‌های کنش جداگانه

خواهیم داشت.

- **اصل‌موضوعه‌های طرد کنش را بیفزا** که می‌گویند هیچ دو کنشی نمی‌توانند هم‌زمان رخ دهند، مثلاً $\neg(\text{FlyP1SFOJFK}^1 \wedge \text{FlyP1SFOBUH}^1)$.
- **اصل‌موضوعه‌های پیش‌شرط را بیفزا:** برای هر کنش زمینی A^t، اصل‌موضوعهٔ $A^t \Rightarrow \text{PRE}(A)^t$ را بیفزا، یعنی اگر کنشی در زمان t گرفته شود، پیش‌شرط‌ها باید صادق بوده باشند. برای نمونه:

$$\text{FlyP1SFOJFK}^1 \Rightarrow \text{At}(\text{P1}, \text{SFO}) \wedge \text{Plane}(\text{P1}) \wedge \text{Airport}(\text{SFO}) \wedge \text{Airport}(\text{JFK})$$

- **حالت آغازین را تعریف کن:** برای هر روان F در حالت آغازین مسئله F^0 را ادعا کن، و برای هر روانی که در حالت آغازین ذکر نشده $\neg F^0$ را.
- **هدف را گزاره‌سازی کن:** هدف فصل‌ای بر روی همهٔ مصداق‌های زمینی‌اش می‌شود، که در آن متغیرها با ثابت‌ها جایگزین می‌شوند. برای نمونه، هدفِ داشتن بلوک A روی بلوکی دیگر، $\text{On}(A,x) \wedge \text{Block}(x)$، در جهانی با اشیای A، B و C، با هدف زیر جایگزین می‌شود:

$$(\text{On}(A, C) \wedge \text{Block}(C)) \vee (\text{On}(A, B) \wedge \text{Block}(B)) \vee (\text{On}(A, A) \wedge \text{Block}(A))$$

- **اصل‌موضوعه‌های حالت-جانشین را بیفزا:** برای هر روان F، اصل‌موضوعه‌ای به شکل:

$$F^{t+1} \Leftrightarrow \text{ActionCausesF}^t \vee (F^t \wedge \neg\text{ActionCausesNotF}^t)$$

بیفزا، که ActionCausesF فصل‌ای از همهٔ کنش‌های زمینی‌ای است که F را می‌افزایند، و ActionCausesNotF فصل‌ای از همهٔ کنش‌های زمینی‌ای است که F را حذف می‌کنند.

ترجمهٔ حاصل معمولاً بسیار بزرگ‌تر از PDDL اصلی است، اما کارایی حل‌کننده‌های SAT مدرن اغلب بیش از آن جبران می‌کند.

۱۱.۲.۴ دیگر رویکردهای برنامه‌ریزی کلاسیک

سه رویکردی که پوشش دادیم تنها رویکردهای آزموده‌شده در تاریخ ۵۰ سالهٔ برنامه‌ریزی خودکار نیستند. برخی دیگر را این‌جا مختصراً توصیف می‌کنیم.

رویکردی به نام Graphplan از ساختار دادهٔ تخصصی، «گراف برنامه‌ریزی» (Planning Graph)، برای رمزگذاری قیدهایی دربارهٔ چگونگی ارتباط کنش‌ها با پیش‌شرط‌ها و اثراتشان و دربارهٔ چیزهایی که دوجانبه منحصرند استفاده می‌کند.

«حساب وضعیت» (Situation Calculus) روشی برای توصیف مسائل برنامه‌ریزی در منطق مرتبهٔ اول است. دقیقاً مانند SATPLAN از اصل‌موضوعه‌های حالت-جانشین استفاده می‌کند، اما منطق مرتبهٔ اول انعطاف و اصل‌موضوعه‌های موجزتری اجازه می‌دهد. در مجموع، این رویکرد به فهم نظری ما از برنامه‌ریزی کمک کرده، اما در

کاربردهای عملی تأثیر بزرگی نداشته، شاید به‌سبب آنکه اثبات‌کننده‌های مرتبهٔ اول به خوبی برنامه‌های ارضاپذیری گزاره‌ای توسعه نیافته‌اند.

می‌توان مسئلهٔ برنامه‌ریزی کران‌دار (یعنی مسئلهٔ یافتن برنامه‌ای به طول k) را به‌صورت مسئلهٔ ارضای قید (CSP) رمزگذاری کرد. رمزگذاری شبیه رمزگذاری به مسئلهٔ SAT است (بخش ۱۱.۲.۳)، با یک ساده‌سازی مهم: در هر گام زمانی فقط به یک متغیر تکی، $Action^t$، نیاز داریم که دامنه‌اش مجموعهٔ کنش‌های ممکن است. دیگر برای هر کنش به یک متغیر نیاز نداریم و به اصل‌موضوعه‌های طرد کنش هم نیاز نداریم.

همهٔ رویکردهایی که تاکنون دیده‌ایم برنامه‌های کاملاً ترتیب‌یافتهٔ متشکل از توالی‌های کاملاً خطی کنش می‌سازند. اما اگر مسئلهٔ بار هوایی ۳۰ بسته در حال بارگیری در یک هواپیما و ۵۰ بسته در حال بارگیری در هواپیمای دیگر داشته باشد، بی‌فایده می‌نماید ترتیب خطی خاصی برای ۸۰ کنش بارگیری حکم کنیم.

بدیلی به نام «برنامه‌ریزی ترتیب جزئی» (Partial-Order Planning) برنامه را به‌صورت گراف بازنمایی می‌کند نه توالی خطی: هر کنش گره‌ای در گراف است و برای هر پیش‌شرط کنش، یالی از کنش دیگر (یا از حالت آغازین) هست که نشان می‌دهد کنش پیشین پیش‌شرط را برقرار می‌کند. پس می‌توانستیم برنامهٔ ترتیب‌جزئی داشته باشیم که می‌گوید کنش‌های Remove(Spare,Trunk) و Remove(Flat,Axle) باید پیش از PutOn(Spare,Axle) بیایند، اما بدون گفتن اینکه کدام یک از دو کنش Remove اول باید بیاید. در فضای برنامه‌ها جستجو می‌کنیم نه حالت‌های جهان، و کنش‌هایی درج می‌کنیم تا شرایط را برآورده کنیم.

در دهه‌های ۱۹۸۰ و ۱۹۹۰، برنامه‌ریزی ترتیب‌جزئی بهترین راه مدیریت مسائل برنامه‌ریزی با زیرمسئله‌های مستقل دیده می‌شد. تا ۲۰۰۰، برنامه‌ریزهای جستجوی پیشرو اکتشافی‌های عالی توسعه داده بودند که به آن‌ها امکان می‌داد زیرمسئله‌های مستقلِ موردنظر برنامه‌ریزی ترتیب‌جزئی را کارآمد کشف کنند. وانگهی، SATPLAN توانست از قانون مور بهره بگیرد: گزاره‌سازی‌ای که در ۱۹۸۰ به‌طرز ناامیدکننده‌ای بزرگ بود اکنون کوچک به نظر می‌رسد، زیرا رایانه‌ها امروز ۱۰,۰۰۰ برابر حافظهٔ بیشتری دارند. در نتیجه، برنامه‌ریزهای ترتیب‌جزئی در مسائل برنامه‌ریزی کلاسیک کاملاً خودکار رقابتی نیستند.

با این حال، برنامه‌ریزی ترتیب‌جزئی بخش مهمی از حوزه باقی مانده. برای برخی تکلیف‌های خاص، مانند زمان‌بندی عملیات، برنامه‌ریزی ترتیب‌جزئی با اکتشافی‌های حوزه‌ویژه فناوری برگزیده است. بسیاری از این سامانه‌ها از کتابخانه‌های برنامه‌های سطح‌بالا استفاده می‌کنند، چنان‌که در بخش ۱۱.۴ توصیف شده.

برنامه‌ریزی ترتیب‌جزئی همچنین اغلب در حوزه‌هایی استفاده می‌شود که فهم برنامه‌ها توسط انسان‌ها مهم است. برای نمونه، برنامه‌های عملیاتی فضاپیماها و مریخ‌نوردها توسط برنامه‌ریزهای ترتیب‌جزئی تولید و سپس توسط اپراتورهای انسانی پیش از بارگذاری روی وسایل نقلیه برای اجرا بررسی می‌شوند. رویکرد پالایش برنامه، فهم آنچه الگوریتم‌های برنامه‌ریزی انجام می‌دهند و تأیید درستی برنامه‌ها پیش از اجرا را برای انسان‌ها آسان‌تر می‌کند.

۱۱.۳ اکتشافی‌های برنامه‌ریزی

نه جستجوی پیشرو و نه پسرو بدون تابع اکتشافی خوب کارآمد نیست. از فصل ۳ به یاد آورید تابع اکتشافی h(s) فاصلهٔ حالت s تا هدف را تخمین می‌زند، و اگر بتوانیم اکتشافی مقبولی برای این فاصله — اکتشافی که بیش‌تخمین نزند — مشتق کنیم، می‌توانیم از جستجوی A* برای یافتن راه‌حل‌های بهینه استفاده کنیم.

طبق تعریف، هیچ راهی برای تحلیل حالت اتمی نیست و بنابراین برای مسائل جستجو با حالت‌های اتمی، تعریف اکتشافی‌های خوب حوزه‌ویژه به نبوغ تحلیلگری (معمولاً انسانی) نیاز دارد. اما برنامه‌ریزی از بازنمایی عاملی برای حالت‌ها و کنش‌ها استفاده می‌کند که تعریف اکتشافی‌های خوب مستقل از حوزه را ممکن می‌کند.

به یاد آورید اکتشافی مقبول را می‌توان با تعریف مسئلهٔ ساده‌شده‌ای که حلش آسان‌تر است مشتق کرد. هزینهٔ دقیق راه‌حل این مسئلهٔ آسان‌تر سپس اکتشافی مسئلهٔ اصلی می‌شود. مسئلهٔ جستجو گرافی است که گره‌هایش حالت‌ها و یال‌هایش کنش‌ها هستند. مسئله یافتن مسیری است که حالت آغازین را به حالت هدف متصل کند. دو راه اصلی برای ساده‌کردن این مسئله هست: افزودن یال‌های بیشتر به گراف، که یافتن مسیر را به‌طور دقیق آسان‌تر می‌کند، یا گروه‌بندی چند گره در هم، که «انتزاع حالت»ی (State Abstraction) با حالت‌های کمتر می‌سازد و بنابراین جستجویش آسان‌تر است.

نخست به اکتشافی‌هایی که یال به گراف می‌افزایند نگاه می‌کنیم. شاید ساده‌ترین، «اکتشافی نادیده‌گرفتن-پیش‌شرط‌ها» (Ignore-Preconditions Heuristic) باشد که همهٔ پیش‌شرط‌ها را از کنش‌ها حذف می‌کند. هر کنش در هر حالتی کاربردپذیر می‌شود و هر روان هدف تکی در یک گام قابل دستیابی است (اگر کنش کاربردپذیری باشد — اگر نباشد، مسئله ناممکن است). این تقریباً دلالت دارد شمار گام‌های لازم برای حل مسئلهٔ ساده‌شده، شمار هدف‌های ارضانشده است — تقریباً اما نه دقیقاً، زیرا (۱) کنشی ممکن است چند هدف را محقق کند و (۲) برخی کنش‌ها ممکن است اثرات کنش‌های دیگر را خنثی کنند.

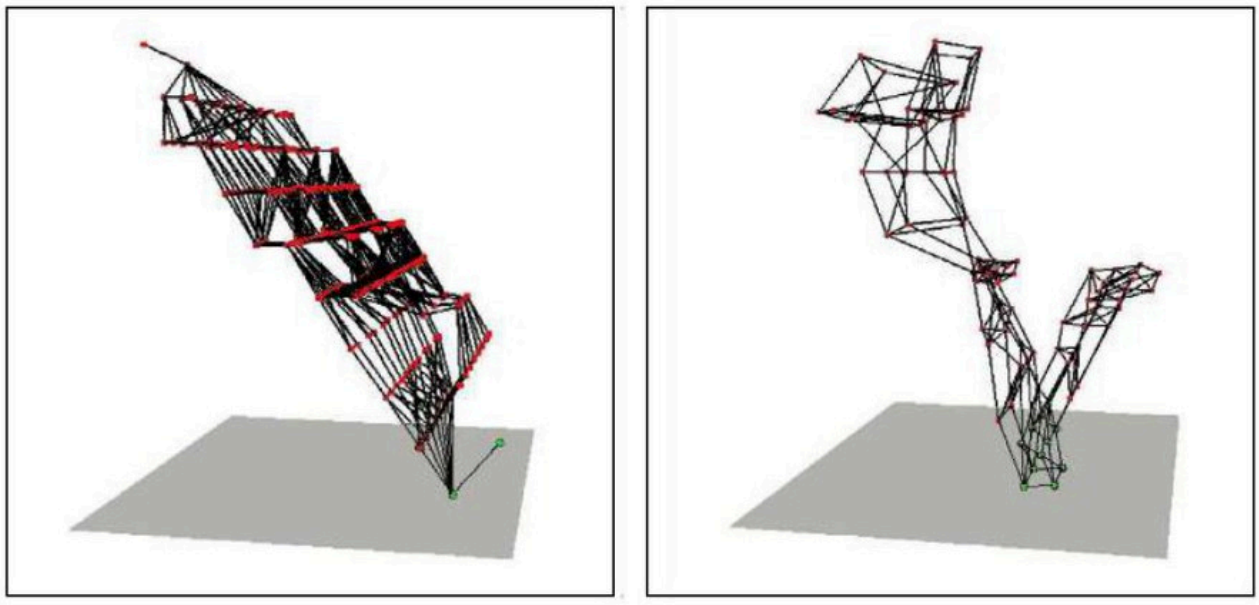
برای بسیاری از مسائل، اکتشافی دقیقی با در نظر گرفتن (۱) و نادیده‌گرفتن (۲) به دست می‌آید. نخست، کنش‌ها را با حذف همهٔ پیش‌شرط‌ها و همهٔ اثرات به‌جز آن‌هایی که لیترال در هدف‌اند ساده می‌کنیم. سپس، کمینهٔ شمار کنش‌های لازم را می‌شماریم چنان‌که اتحاد اثرات آن کنش‌ها هدف را ارضا کند. این مصداقی از «مسئلهٔ پوشش مجموعه» (Set-Cover Problem) است. یک آزردگی جزئی هست: مسئلهٔ پوشش مجموعه NP-سخت است. خوشبختانه الگوریتم حریصانهٔ ساده‌ای تضمین می‌شود پوشش مجموعه‌ای بازگرداند که اندازه‌اش در ضریب log n از پوشش کمینهٔ واقعی فاصله دارد، که n شمار لیترال‌های هدف است. متأسفانه، الگوریتم حریصانه تضمین مقبولیت را از دست می‌دهد.

همچنین امکان نادیده‌گرفتن فقط پیش‌شرط‌های برگزیدهٔ کنش‌ها هست. معمای کاشی‌لغزان (معمای ۸ یا ۱۵) از بخش 3.2 را در نظر بگیرید. می‌توانستیم این را به‌صورت مسئلهٔ برنامه‌ریزی شامل کاشی‌ها با طرح تکی Slide رمزگذاری کنیم:

```
,Action(Slide(t, s1, s2)
PRECOND: On(t, s1) ∧ Tile(t) ∧ Blank(s2) ∧ Adjacent(s1, s2)
(EFFECT: On(t, s2) ∧ Blank(s1) ∧ ¬On(t, s1) ∧ ¬Blank(s2)
```

چنان‌که در بخش 3.6 دیدیم، اگر پیش‌شرط‌های Adjacent(s1,s2) ∧ Blank(s2) را حذف کنیم، هر کاشی می‌تواند در یک کنش به هر فضایی حرکت کند و اکتشافی شمار-کاشی‌های-نابجا را می‌گیریم. اگر فقط پیش‌شرط Blank(s2) را حذف کنیم، اکتشافی فاصلهٔ منهتن را می‌گیریم. به‌راحتی دیده می‌شود این اکتشافی‌ها چگونه می‌توانستند به‌صورت خودکار از توصیف طرح کنش مشتق شوند. سهولت دست‌کاری طرح‌های کنش مزیت بزرگ بازنمایی عاملی مسائل برنامه‌ریزی است، در مقایسه با بازنمایی اتمی مسائل جستجو.

امکان دیگر «اکتشافی نادیده‌گرفتن-فهرست‌های-حذف» (Ignore-Delete-Lists Heuristic) است. لحظه‌ای فرض کنیم همهٔ هدف‌ها و پیش‌شرط‌ها فقط لیترال‌های مثبت دارند.[۲] می‌خواهیم نسخهٔ ساده‌شده‌ای از مسئلهٔ اصلی بسازیم که حلش آسان‌تر باشد و طول راه‌حلش به‌عنوان اکتشافی خوب خدمت کند. می‌توانیم این کار را با حذف فهرست‌های حذف از همهٔ کنش‌ها (یعنی حذف همهٔ لیترال‌های منفی از اثرات) انجام دهیم. این پیشرفت یکنواخت به سوی هدف را ممکن می‌کند — هیچ کنشی هرگز پیشرفت کنش دیگر را خنثی نمی‌کند. معلوم می‌شود یافتن راه‌حل بهینه برای این مسئلهٔ ساده‌شده همچنان NP-سخت است، اما راه‌حل تقریبی را می‌توان در زمان چندجمله‌ای با تپه‌نوردی یافت.



شکل ۱۱.۶ — دو فضای حالت از مسائل برنامه‌ریزی با اکتشافی نادیده‌گرفتن-فهرست‌های-حذف. ارتفاع بالای صفحهٔ پایین، امتیاز اکتشافی حالت است؛ حالت‌های روی صفحهٔ پایین هدف‌اند. هیچ کمینهٔ محلی نیست، پس جستجوی هدف سرراست است. از هافمن (۲۰۰۵).

شکل ۱۱.۶ بخشی از فضای حالت را برای دو مسئلهٔ برنامه‌ریزی با اکتشافی نادیده‌گرفتن-فهرست‌های-حذف نمودار می‌کند. نقطه‌ها حالت‌ها و یال‌ها کنش‌ها را نشان می‌دهند و ارتفاع هر نقطه بالای صفحهٔ پایین مقدار اکتشافی را بازنمایی می‌کند. حالت‌های روی صفحهٔ پایین راه‌حل‌اند. در هر دو مسئله، مسیر پهن به سوی هدف هست. هیچ

بن‌بستی نیست، پس نیازی به عقبگرد نیست؛ جستجوی سادهٔ تپه‌نوردی به‌راحتی راه‌حلی برای این مسائل می‌یابد (هرچند ممکن است راه‌حل بهینه نباشد).

۱۱.۳.۱ هرس مستقل از حوزه

بازنمایی‌های عاملی بدیهی می‌کنند بسیاری از حالت‌ها فقط تغییرهایی از حالت‌های دیگرند. برای نمونه، فرض کنیم یک دوجین بلوک روی میز داریم و هدف آن است که بلوک A بالای برج سه‌بلوکی باشد. نخستین گام راه‌حل، گذاشتن بلوکی x بالای بلوکی y است (که x، y و A همگی متفاوت‌اند). پس از آن، A را بالای x بگذار و تمام. ۱۱ انتخاب برای x و با داشتن ۱۰ ،x انتخاب برای y هست و بدین ترتیب ۱۱۰ حالت برای در نظر گرفتن. اما همهٔ این حالت‌ها متقارن‌اند: انتخاب یکی بر دیگری تفاوتی ایجاد نمی‌کند و بنابراین برنامه‌ریز باید فقط یکی از آن‌ها را در نظر بگیرد. این فرایند «کاهش تقارن» (Symmetry Reduction) است: همهٔ شاخه‌های متقارن درخت جستجو را به‌جز یکی از ملاحظه هرس می‌کنیم. برای بسیاری از حوزه‌ها، این تفاوت میان ناشدنی و حل‌شدنی کارآمد است.

امکان دیگر انجام «هرس رو به جلو» است، با پذیرش ریسک هرس‌کردن راه‌حل بهینه، برای متمرکزکردن جستجو بر شاخه‌های امیدوارکننده. می‌توانیم «کنش ترجیحی» (Preferred Action) را چنین تعریف کنیم: نخست، نسخهٔ ساده‌شدهٔ مسئله را تعریف و حل کن تا برنامهٔ ساده‌شده به دست آید. سپس کنش ترجیحی یا گامی از برنامهٔ ساده‌شده است، یا برخی پیش‌شرط‌های برنامهٔ ساده‌شده را محقق می‌کند.

گاهی ممکن است مسئله‌ای را با تشخیص اینکه تعامل‌های منفی را می‌توان منتفی دانست کارآمد حل کرد. می‌گوییم مسئله‌ای «زیرهدف‌های سریالی‌پذیر» (Serializable Subgoals) دارد اگر ترتیبی از زیرهدف‌ها وجود داشته باشد چنان‌که برنامه‌ریز بتواند آن‌ها را در آن ترتیب محقق کند بدون آنکه مجبور شود هیچ‌یک از زیرهدف‌های قبلاً محقق‌شده را خنثی کند. برای نمونه، در جهان بلوک‌ها، اگر هدف ساختن برجی باشد (مثلاً A روی B، که به نوبهٔ خود روی C، که به نوبهٔ خود روی میز، مانند شکل ۱۱.۳ در صفحهٔ ۳۴۷)، آنگاه زیرهدف‌ها از پایین به بالا سریالی‌پذیرند: اگر نخست C را روی میز محقق کنیم، هنگام محقق‌کردن دیگر زیرهدف‌ها هرگز لازم نیست آن را خنثی کنیم. برنامه‌ریزی که از حربهٔ پایین-به-بالا استفاده کند می‌تواند هر مسئله‌ای در جهان بلوک‌ها را بدون عقبگرد حل کند (هرچند ممکن است همیشه کوتاه‌ترین برنامه را نیابد). به‌عنوان مثال دیگر، اگر اتاقی با n کلید چراغ باشد که هر یک چراغ جداگانه‌ای را کنترل می‌کند و هدف روشن‌کردن همهٔ آن‌هاست، لازم نیست جایگشت‌های ترتیب را در نظر بگیریم؛ می‌توانستیم خود را به‌طور دلخواه به برنامه‌هایی محدود کنیم که کلیدها را مثلاً به ترتیب صعودی می‌زنند.

برای برنامه‌ریز «عامل از راه دور» (Remote Agent) که فضاپیمای دیپ‌اسپیس وان ناسا را فرمان می‌داد، تعیین شد گزاره‌های درگیر در فرمان‌دهی فضاپیما سریالی‌پذیرند. این شاید چندان شگفت‌آور نباشد، زیرا فضاپیما توسط مهندسانش طوری طراحی می‌شود که تا حد ممکن کنترلش آسان باشد (مشروط به قیدهای دیگر). با بهره‌گیری از ترتیب سریالی هدف‌ها، برنامه‌ریز عامل از راه دور توانست بیشتر جستجو را حذف کند. این بدان معنا بود به اندازهٔ کافی سریع بود که فضاپیما را در زمان واقعی کنترل کند، چیزی که پیش‌تر ناممکن دانسته می‌شد.

۱۱.۳.۲ انتزاع حالت در برنامه‌ریزی

مسئلهٔ ساده‌شده ما را با مسئلهٔ برنامه‌ریزی ساده‌شده‌ای رها می‌کند فقط برای محاسبهٔ مقدار تابع اکتشافی. بسیاری از مسائل برنامه‌ریزی 10^{100} حالت یا بیشتر دارند و ساده‌کردن کنش‌ها هیچ‌کاری برای کاهش شمار حالت‌ها نمی‌کند، یعنی محاسبهٔ اکتشافی ممکن است همچنان پرهزینه باشد. پس اکنون به ساده‌سازی‌هایی نگاه می‌کنیم که شمار حالت‌ها را با تشکیل «انتزاع حالت» (State Abstraction) کاهش می‌دهند — نگاشت چند-به-یک از حالت‌های بازنمایی زمینی مسئله به بازنمایی انتزاعی.

آسان‌ترین شکل انتزاع حالت، نادیده‌گرفتن برخی روان‌هاست. برای نمونه، مسئلهٔ بار هوایی با ۱۰ فرودگاه، ۵۰ هواپیما و ۲۰۰ قطعه محموله را در نظر بگیرید. هر هواپیما می‌تواند در یکی از ۱۰ فرودگاه باشد و هر بسته یا در یکی از هواپیماهاست یا در یکی از فرودگاه‌ها بارخالی شده. پس $10^{50} \times (10+50)^{200} \approx 10^{405}$ حالت هست. حال مسئلهٔ خاصی در آن حوزه را در نظر بگیر که در آن اتفاقاً همهٔ بسته‌ها فقط در ۵ تا از فرودگاه‌ها هستند و همهٔ بسته‌های یک فرودگاهِ داده‌شده همان مقصد را دارند. آنگاه انتزاع مفید مسئله، انداختن همهٔ روان‌های At به‌جز آن‌هایی است که شامل یک هواپیما و یک بسته در هر یک از ۵ فرودگاه‌اند. حال فقط $10^5 \times (5+5)^5 \approx 10^{11}$ حالت هست. راه‌حلی در این فضای حالت انتزاعی کوتاه‌تر از راه‌حل فضای اصلی خواهد بود (و بنابراین اکتشافی مقبول خواهد بود) و بسط راه‌حل انتزاعی به راه‌حل مسئلهٔ اصلی آسان است (با افزودن کنش‌های Load و Unload اضافی).

ایدهٔ کلیدی در تعریف اکتشافی‌ها «تجزیه» (Decomposition) است: تقسیم مسئله به اجزا، حل مستقل هر جزء، و سپس ترکیب اجزا. «فرض استقلال زیرهدف» (Subgoal Independence Assumption) آن است که هزینهٔ حل عطفی از زیرهدف‌ها با مجموع هزینه‌های حل مستقل هر زیرهدف تقریب زده می‌شود. فرض استقلال زیرهدف می‌تواند خوش‌بینانه یا بدبینانه باشد. وقتی تعامل‌های منفی میان زیربرنامه‌های هر زیرهدف هست خوش‌بینانه است — برای نمونه، وقتی کنشی در یک زیربرنامه هدفِ محقق‌شده توسط زیربرنامهٔ دیگر را حذف می‌کند. وقتی زیربرنامه‌ها کنش‌های زائد دارند بدبینانه و بنابراین نامقبول است — برای نمونه، دو کنش که در برنامهٔ ادغام‌شده می‌شد با کنش تکی جایگزین شوند.

فرض کنیم هدف مجموعه‌ای از روان‌ها G باشد، که به زیرمجموعه‌های ناهم‌پوش G1,...,Gn تقسیمش می‌کنیم. سپس برنامه‌های بهینهٔ P1,...,Pn را می‌یابیم که زیرهدف‌های مربوطه را حل می‌کنند. تخمین هزینهٔ برنامه برای محقق‌کردن همهٔ G چیست؟ می‌توانیم هر COST(Pi) را تخمین اکتشافی بدانیم و می‌دانیم اگر تخمین‌ها را با گرفتن بیشینهٔ آن‌ها ترکیب کنیم، همیشه اکتشافی مقبول می‌گیریم. پس maxi COST(Pi) مقبول است و گاهی دقیقاً درست است: ممکن است P1 اتفاقاً همهٔ Gi را محقق کند. اما معمولاً تخمین بیش از حد پایین است. آیا می‌توانیم هزینه‌ها را جمع کنیم؟ برای بسیاری از مسائل این تخمین معقولی است، اما مقبول نیست. بهترین حالت وقتی است که Gi و Gj مستقل باشند، به این معنا که برنامه‌های یکی نمی‌توانند هزینهٔ برنامه‌های دیگری را کاهش دهند. در آن صورت، تخمین COST(Pi) + COST(Pj) مقبول و از تخمین بیشینه دقیق‌تر است.

روشن است پتانسیل بزرگی برای کاهش فضای جستجو با تشکیل انتزاع‌ها هست. حربه انتخاب انتزاع‌های درست و استفاده از آن‌ها به‌شیوه‌ای است که هزینهٔ کل — تعریف انتزاع، انجام جستجوی انتزاعی و نگاشت انتزاع به مسئلهٔ اصلی — کمتر از هزینهٔ حل مسئلهٔ اصلی شود. فنون پایگاه‌داده‌های الگو از بخش 3.6.3 می‌توانند مفید باشند، زیرا

هزینهٔ ایجاد پایگاه‌دادهٔ الگو را می‌توان روی چند نمونهٔ مسئله سرشکن کرد.

سامانه‌ای که از اکتشافی‌های مؤثر استفاده می‌کند FF یا FASTFORWARD است (هافمن، ۲۰۰۵)، جستجوگر فضای حالت پیشرویی که از اکتشافی نادیده‌گرفتن-فهرست‌های-حذف استفاده می‌کند و اکتشافی را با کمک گراف برنامه‌ریزی تخمین می‌زند. FF سپس از جستجوی تپه‌نوردی (اصلاح‌شده برای پیگیری برنامه) با اکتشافی برای یافتن راه‌حل استفاده می‌کند. الگوریتم تپه‌نوردی FF غیراستاندارد است: با اجرای جستجوی سطح-اول از حالت کنونی تا یافتن حالت بهتر، از بیشینه‌های محلی اجتناب می‌کند. اگر این شکست بخورد، FF به جستجوی بهترین-اول حریصانه سوئیچ می‌کند.

۱۱.۴ برنامه‌ریزی سلسله‌مراتبی

روش‌های حل مسئله و برنامه‌ریزی فصل‌های پیشین همگی با مجموعه‌ای ثابت از کنش‌های اتمی عمل می‌کنند. کنش‌ها را می‌توان به هم رشته کرد و الگوریتم‌های پیشرفته می‌توانند راه‌حل‌هایی شامل هزاران کنش تولید کنند. این برای برنامه‌ریزی تعطیلات خوب است اگر کنش‌ها در سطح «پرواز از سانفرانسیسکو به هونولولو» باشند، اما در سطح کنترل موتور یعنی «زانوی چپ را ۵ درجه خم کن» باید میلیون‌ها یا میلیاردها کنش به هم رشته کنیم، نه هزاران.

پل‌زدن این شکاف مستلزم برنامه‌ریزی در سطح‌های انتزاعی بالاتر است. برنامهٔ سطح‌بالای تعطیلات هاوایی ممکن است «به فرودگاه سانفرانسیسکو برو؛ پرواز HA 11 به هونولولو بگیر؛ دو هفته کارهای تعطیلاتی بکن؛ HA 12 را به سانفرانسیسکو برگردان؛ به خانه برو» باشد. با چنین برنامه‌ای، کنش «به فرودگاه سانفرانسیسکو برو» را می‌توان خودش تکلیف برنامه‌ریزی‌ای دید، با راه‌حلی مانند «سرویس مسافرتی آنلاین برگزین؛ خودرو سفارش بده؛ به فرودگاه برو.» هر یک از این کنش‌ها، به نوبهٔ خود، می‌تواند بیشتر تجزیه شود، تا آنکه به کنش‌های کنترل موتور سطح‌پایین مانند فشردن دکمه برسیم.

در این مثال، برنامه‌ریزی و کنش در هم آمیخته‌اند؛ برای نمونه، برنامه‌ریزیِ پیاده‌روی از حاشیه تا دروازه را تا پس از پیاده‌شدن به تأخیر می‌اندازیم. پس آن کنش خاص پیش از فاز اجرا در سطح انتزاعی باقی می‌ماند. بحث این موضوع را به بخش ۱۱.۵ موکول می‌کنیم. در این‌جا بر ایدهٔ «تجزیهٔ سلسله‌مراتبی» (Hierarchical Decomposition) تمرکز می‌کنیم، ایده‌ای که تقریباً همهٔ تلاش‌ها برای مدیریت پیچیدگی را در بر می‌گیرد. برای نمونه، نرم‌افزار پیچیده از سلسله‌مراتبی از زیرروال‌ها و کلاس‌ها ساخته می‌شود؛ ارتش‌ها، دولت‌ها و شرکت‌ها سلسله‌مراتب سازمانی دارند. منفعت کلیدی ساختار سلسله‌مراتبی آن است که در هر سطح سلسله‌مراتب، تکلیف محاسباتی، مأموریت نظامی یا کارکرد اداری به شمار کوچکی فعالیت در سطح پایین‌تر بعدی کاهش می‌یابد، پس هزینهٔ محاسباتی یافتن راه درستِ چیدن آن فعالیت‌ها برای مسئلهٔ کنونی کوچک است.

۱۱.۴.۱ کنش‌های سطح‌بالا

صورت‌بندی پایه‌ای که برای فهم تجزیهٔ سلسله‌مراتبی اتخاذ می‌کنیم از حوزهٔ «شبکه‌های تکلیف سلسله‌مراتبی» (Hierarchical Task Networks) یا برنامه‌ریزی HTN می‌آید. فعلاً مشاهده‌پذیری کامل و جبرگرایی و مجموعه‌ای از کنش‌ها، که اکنون «کنش‌های اولیه» (Primitive Actions) نامیده می‌شوند، با طرح‌های پیش‌شرط–اثر استاندارد

فرض می‌کنیم. مفهوم اضافی کلیدی «کنش سطح‌بالا» یا HLA است — برای نمونه، کنش «به فرودگاه سانفرانسیسکو برو». هر HLA یک یا چند «پالایش» (Refinement) ممکن به توالی‌ای از کنش‌ها دارد، که هر یک ممکن است HLA یا کنش اولیه باشد. برای نمونه، کنش «به فرودگاه سانفرانسیسکو برو»، که صوری Go(Home, SFO) بازنمایی می‌شود، ممکن است دو پالایش ممکن داشته باشد، چنان‌که در شکل ۱۱.۷. همان شکل پالایش بازگشتی برای ناوبری در جهان جاروبرقی را نشان می‌دهد: برای رسیدن به مقصد، یک گام بردار و سپس به مقصد برو.

```
                                                    ,Refinement(Go(Home, SFO)
                                        ,STEPS: [Drive(Home, SFOLongTermParking)
                                    ([Shuttle(SFOLongTermParking, SFO)
                                                     ,Refinement(Go(Home, SFO)
                                                (STEPS: [Taxi(Home, SFO)]
                                            ,Refinement(Navigate([a,b],[x,y])
                                                        PRECOND: a=x ∧ b=y
                                                                ([] :STEPS
                                            ,Refinement(Navigate([a,b],[x,y])
                                                PRECOND: Connected([a,b],[a−1,b])
                                        (STEPS: [Left, Navigate([a−1,b],[x,y])]
                                            ,Refinement(Navigate([a,b],[x,y])
                                                PRECOND: Connected([a,b],[a+1,b])
                                        (STEPS: [Right, Navigate([a+1,b],[x,y])]
                                                                            ...
```

شکل ۱۱.۷ — تعریف‌های پالایش‌های ممکن برای دو کنش سطح‌بالا: رفتن به فرودگاه سانفرانسیسکو و ناوبری در جهان جاروبرقی. در مورد اخیر، به سرشت بازگشتی پالایش‌ها و استفاده از پیش‌شرط‌ها توجه کنید.

این مثال‌ها نشان می‌دهند کنش‌های سطح‌بالا و پالایش‌هایشان دانشی دربارهٔ چگونگی انجام کارها تجسم می‌بخشند. برای نمونه، پالایش‌های Go(Home,SFO) می‌گویند برای رسیدن به فرودگاه می‌توانی رانندگی کنی یا سرویس مسافرتی آنلاین بگیری؛ خریدن شیر، نشستن و حرکت‌دادن اسب به e4 نباید در نظر گرفته شوند.

پالایش HLA که فقط کنش‌های اولیه دارد «پیاده‌سازی» (Implementation) HLA نامیده می‌شود. در جهان شبکه‌ای، توالی‌های [Right,Right,Down] و [Down,Right,Right] هر دو HLA ناوبری Navigate([1,3],[3,2]) را پیاده می‌کنند. پیاده‌سازی برنامهٔ سطح‌بالا (توالی‌ای از HLAها) به هم چسباندن پیاده‌سازی‌های هر HLA در توالی است. با تعریف‌های پیش‌شرط–اثر هر کنش اولیه، تعیین اینکه آیا هر پیاده‌سازیِ داده‌شده‌ای از برنامهٔ سطح‌بالا هدف را محقق می‌کند سرراست است.

پس می‌توانیم بگوییم برنامهٔ سطح‌بالا از حالتِ داده‌شده هدف را محقق می‌کند اگر دست‌کم یکی از پیاده‌سازی‌هایش هدف را از آن حالت محقق کند. «دست‌کم یکی» در این تعریف حیاتی است — لازم نیست همهٔ پیاده‌سازی‌ها هدف را محقق کنند، زیرا خودِ عامل تصمیم می‌گیرد کدام پیاده‌سازی را اجرا کند. پس مجموعهٔ پیاده‌سازی‌های ممکن در برنامه‌ریزی HTN — که هر یک ممکن است پیامد متفاوتی داشته باشد — با مجموعهٔ پیامدهای ممکن در برنامه‌ریزی ناجبرگرایانه یکسان نیست. در آن‌جا، ایجاب می‌کردیم برنامه برای همهٔ پیامدها کار کند، زیرا عامل انتخاب پیامد را ندارد؛ طبیعت دارد.

ساده‌ترین حالت HLAای است که دقیقاً یک پیاده‌سازی دارد. در آن صورت، می‌توانیم پیش‌شرط‌ها و اثرات HLA را از آن‌های پیاده‌سازی محاسبه کنیم (تمرین HLAU.11 را ببینید) و سپس HLA را دقیقاً چنان رفتار کنیم که گویی خودش کنش اولیه است. می‌توان نشان داد مجموعهٔ درستی از HLAها می‌تواند به کاهش پیچیدگی زمانی جستجوی کور از نمایی در عمق راه‌حل به خطی در عمق راه‌حل بینجامد، هرچند ابداع چنین مجموعه‌ای از HLAها ممکن است خودش تکلیفی غیربدیهی باشد. وقتی HLAها چند پیاده‌سازی ممکن دارند، دو گزینه هست: یکی جستجو میان پیاده‌سازی‌ها برای یافتن پیاده‌سازیِ کارکننده، مانند بخش ۱۱.۴.۲؛ دیگری استدلال مستقیم دربارهٔ HLAها — با وجود تکثر پیاده‌سازی‌ها — چنان‌که در بخش ۱۱.۴.۳ توضیح داده شده. روش اخیر اشتقاق برنامه‌های انتزاعیِ اثباتاً درست را بدون نیاز به در نظر گرفتن پیاده‌سازی‌هایشان ممکن می‌کند.

۱۱.۴.۲ جستجو برای راه‌حل‌های اولیه

برنامه‌ریزی HTN اغلب با کنش «سطح بالای» تکی به نام Act صورت‌بندی می‌شود، که هدف یافتن پیاده‌سازی Act است که هدف را محقق کند. این رویکرد کاملاً عمومی است. برای نمونه، مسائل برنامه‌ریزی کلاسیک را می‌توان چنین تعریف کرد: برای هر کنش اولیهٔ ai، یک پالایش از Act با گام‌های [ai, Act] فراهم کن. این تعریف بازگشتی از Act می‌سازد که به ما اجازه می‌دهد کنش بیفزاییم. اما به راهی برای توقف بازگشت نیاز داریم؛ با فراهم‌کردن پالایش دیگری برای Act، با فهرست گام خالی و پیش‌شرطی برابر با هدف مسئله، این کار را می‌کنیم. این می‌گوید اگر هدف از قبل محقق شده، پیاده‌سازی درست هیچ‌کاری‌نکردن است.

رویکرد به الگوریتم ساده‌ای می‌انجامد: به‌طور مکرر HLAیی را در برنامهٔ کنونی برگزین و آن را با یکی از پالایش‌هایش جایگزین کن، تا آنکه برنامه هدف را محقق کند. یکی از پیاده‌سازی‌های ممکن مبتنی بر جستجوی درختی سطح-اول در شکل ۱۱.۸ نشان داده شده. برنامه‌ها به ترتیب عمق تودرتویی پالایش‌ها در نظر گرفته می‌شوند، نه شمار گام‌های اولیه. طراحی نسخهٔ جستجوی گرافی الگوریتم و نیز نسخه‌های عمق-اول و تعمیق تکراری سرراست است.

```
function HIERARCHICAL-SEARCH(problem, hierarchy) returns a solution or failure
                        frontier ← a FIFO queue with [Act] as the only element
                                                                  while true do
                                    if IS-EMPTY(frontier) then return failure
    plan ← POP(frontier)          // selects the shallowest plan in frontier
                                hla ← the first HLA in plan, or null if none
       prefix, suffix ← subsequences of actions before and after hla in plan
                                    outcome ← RESULT(problem.INITIAL, prefix)
if hla is null then         // so plan is primitive and outcome is its result
                                  if problem.IS-GOAL(outcome) then return plan
            else for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
                                 add APPEND(prefix, sequence, suffix) to frontier
```

شکل ۱۱.۸ — پیاده‌سازی سطح-اول جستجوی برنامه‌ریزی سلسله‌مراتبی پیشرو. برنامهٔ آغازین عرضه‌شده به الگوریتم [Act] است. تابع REFINEMENTS مجموعه‌ای از توالی‌های کنش بازمی‌گرداند، برای هر پالایش HLA که پیش‌شرط‌هایش توسط حالتِ مشخص‌شده، outcome، برآورده می‌شوند.

در اصل، این شکل از جستجوی سلسله‌مراتبی فضای توالی‌هایی را کاوش می‌کند که با دانشِ موجود در کتابخانهٔ HLA دربارهٔ چگونگی انجام کارها سازگارند. دانش فراوانی را می‌توان رمزگذاری کرد، نه فقط در توالی‌های کنشِ مشخص‌شده در هر پالایش، بلکه در پیش‌شرط‌های پالایش‌ها. برای برخی حوزه‌ها، برنامه‌ریزهای HTN توانسته‌اند برنامه‌های عظیمی با جستجوی بسیار اندک تولید کنند. برای نمونه، O-PLAN (بل و تیت، ۱۹۸۵)، که برنامه‌ریزی HTN را با زمان‌بندی ترکیب می‌کند، برای توسعهٔ برنامه‌های تولیدی هیتاچی استفاده شده. مسئلهٔ نمونه شامل خط تولیدی با ۳۵۰ محصول متفاوت، ۳۵ ماشین مونتاژ و بیش از ۲۰۰۰ عملیات متفاوت است. برنامه‌ریز زمان‌بندی ۳۰روزه با سه شیفت ۸ساعته در روز تولید می‌کند، شامل ده‌ها میلیون گام. جنبهٔ مهم دیگر برنامه‌های HTN آن است که، طبق تعریف، ساختار سلسله‌مراتبی دارند؛ معمولاً این فهم آن‌ها را برای انسان‌ها آسان می‌کند.

منافع محاسباتی جستجوی سلسله‌مراتبی را می‌توان با بررسی حالت ایده‌آل‌شده‌ای دید. فرض کنیم مسئلهٔ برنامه‌ریزی راه‌حلی با d کنش اولیه دارد. برای برنامه‌ریز فضای حالت پیشروی غیرسلسله‌مراتبی با b کنش مجاز در هر حالت، هزینه $O(b^d)$ است، چنان‌که در فصل ۳ توضیح داده شد. برای برنامه‌ریز HTN، ساختار پالایشِ بسیار منظمی فرض کنیم: هر کنش غیراولیه r پالایش ممکن دارد، هر یک به k کنش در سطح پایین‌تر بعدی. می‌خواهیم بدانیم با این ساختار چند درخت پالایش متفاوت هست. حال، اگر d کنش در سطح اولیه باشد، شمار سطح‌های زیر ریشه logk d است، پس شمار گره‌های پالایش داخلی k+k²+...+k^(logk d -1) = (d-1)/(k-1)+1 است. هر گره داخلی r پالایش ممکن دارد، پس r^((d-1)/(k-1)) درخت تجزیهٔ ممکن می‌توان ساخت.

با بررسی این فرمول، می‌بینیم کوچک‌نگه‌داشتن r و بزرگ‌نگه‌داشتن k می‌تواند صرفه‌جویی عظیمی به بار آورد: ریشهٔ kام هزینهٔ غیرسلسله‌مراتبی را می‌گیریم، اگر b و r قابل مقایسه باشند. r کوچک و k بزرگ یعنی کتابخانهٔ HLA با شمار کمی پالایش که هر یک توالی کنش بلندی می‌دهد. این همیشه ممکن نیست: توالی‌های کنش بلندی که در گسترهٔ وسیعی از مسائل قابل استفاده‌اند بسیار نادرند.

کلید برنامه‌ریزی HTN کتابخانهٔ برنامه‌ای است شامل روش‌های شناخته‌شده برای پیاده‌سازی کنش‌های پیچیده و سطح‌بالا. یکی از راه‌های ساخت کتابخانه، یادگیری روش‌ها از تجربهٔ حل مسئله است. پس از تجربهٔ رنج‌آور ساختن برنامه‌ای از صفر، عامل می‌تواند برنامه را در کتابخانه به‌عنوان روشی برای پیاده‌سازی کنش سطح‌بالای تعریف‌شده توسط تکلیف ذخیره کند. بدین ترتیب، عامل می‌تواند با گذشت زمان شایسته‌تر و شایسته‌تر شود، زیرا روش‌های تازه بر روی روش‌های قدیمی ساخته می‌شوند. یکی از جنبه‌های مهم این فرایند یادگیری، توانایی تعمیم روش‌های ساخته‌شده است، با حذف جزئیات خاصِ نمونهٔ مسئله (مثلاً نام سازنده یا آدرس زمین) و نگه‌داشتن فقط عناصر کلیدی برنامه. به نظر ما غیرقابل‌تصور است انسان‌ها بدون چنین سازوکاری بتوانند به همین اندازه شایسته باشند.

۱۱.۴.۳ جستجو برای راه‌حل‌های انتزاعی

الگوریتم جستجوی سلسله‌مراتبی بخش پیشین HLAها را تمام‌راه تا توالی‌های کنش اولیه پالایش می‌کند تا تعیین کند برنامه‌ای عملی است یا نه. این با عقل سلیم تناقض دارد: باید بتوان تعیین کرد برنامهٔ سطح‌بالای دو-HLAایِ

```
[Drive(Home, SFOLongTermParking), Shuttle(SFOLongTermParking, SFO)]
```

فرد را به فرودگاه می‌رساند، بدون آنکه مجبور باشیم مسیر دقیق، انتخاب جای پارک و مانند آن را تعیین کنیم. راه‌حل نوشتن توصیف‌های پیش‌شرط–اثر HLAهاست، دقیقاً همان‌گونه که برای کنش‌های اولیه می‌کنیم. از توصیف‌ها، اثبات اینکه برنامهٔ سطح‌بالا هدف را محقق می‌کند باید آسان باشد. این به اصطلاح جام مقدس برنامه‌ریزی سلسله‌مراتبی است، زیرا اگر برنامهٔ سطح‌بالایی مشتق کنیم که به‌شکل اثبات‌پذیر هدف را محقق می‌کند، با کار در فضای جستجوی کوچک کنش‌های سطح‌بالا، می‌توانیم به آن برنامه متعهد شویم و روی مسئلهٔ پالایش هر گام برنامه کار کنیم. این کاهش نمایی موردنظر ما را می‌دهد.

برای این‌که این کار کند، باید چنین باشد که هر برنامهٔ سطح‌بالایی که (به‌واسطهٔ توصیف‌های گام‌هایش) «ادعای» محقق‌کردن هدف را دارد، در واقع هدف را به معنای پیش‌ترتعریف‌شده محقق کند: باید دست‌کم یک پیاده‌سازی داشته باشد که هدف را محقق کند. این ویژگی «ویژگی پالایش رو به پایین» (Downward Refinement Property) برای توصیف‌های HLA نامیده شده.

نوشتن توصیف‌های HLA که ویژگی پالایش رو به پایین را برآورده کنند در اصل آسان است: تا وقتی توصیف‌ها صادق‌اند، هر برنامهٔ سطح‌بالایی که ادعای محقق‌کردن هدف را دارد باید در واقع چنین کند — وگرنه توصیف‌ها ادعای کاذبی دربارهٔ آنچه HLAها انجام می‌دهند می‌کنند. پیش‌تر دیده‌ایم چگونه توصیف‌های صادق برای HLAهایی که دقیقاً یک پیاده‌سازی دارند بنویسیم (تمرین HLAU.11)؛ وقتی HLA چند پیاده‌سازی دارد مسئله پیش می‌آید. چگونه می‌توانیم اثرات کنشی را توصیف کنیم که به روش‌های بسیار متفاوت قابل پیاده‌سازی است؟

یک پاسخ ایمن (دست‌کم برای مسائلی که همهٔ پیش‌شرط‌ها و هدف‌ها مثبت‌اند) آن است که فقط اثرات مثبتی را شامل شویم که توسط هر پیاده‌سازی HLA محقق می‌شوند و اثرات منفی هر پیاده‌سازی را. آنگاه ویژگی پالایش رو به پایین برآورده می‌شد. متأسفانه، این معناشناسی برای HLAها بیش از حد محافظه‌کارانه است.

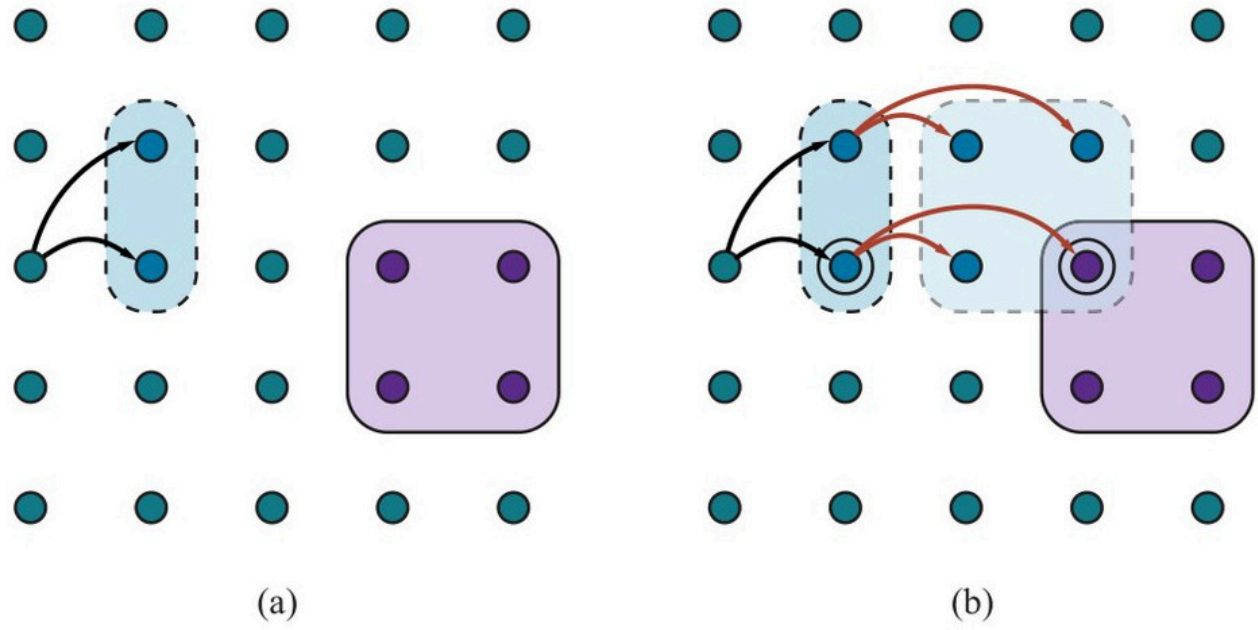
HLA Go(Home,SFO) را دوباره در نظر بگیر، که دو پالایش دارد، و فرض کنیم، برای sake استدلال، جهان ساده‌ای که در آن همیشه می‌توان رانندگی کرد و پارک کرد، اما گرفتن تاکسی به پیش‌شرط Cash نیاز دارد. در آن صورت، Go(Home,SFO) همیشه شما را به فرودگاه نمی‌رساند. به‌ویژه، اگر Cash کاذب باشد شکست می‌خورد، و بنابراین نمی‌توانیم At(Agent,SFO) را به‌عنوان اثر HLA ادعا کنیم. این اما بی‌معناست؛ اگر عامل Cash نداشت، خودش رانندگی می‌کرد. ایجاب اینکه اثری برای هر پیاده‌سازی برقرار باشد معادل فرض این است که شخص دیگری — حریفی — پیاده‌سازی را انتخاب خواهد کرد. پیامدهای چندگانهٔ HLA را دقیقاً چنان رفتار می‌کند که گویی HLA کنش ناجبرگرایانه‌ای است، مانند بخش 4.3. در مورد ما، خودِ عامل پیاده‌سازی را انتخاب خواهد کرد.

جامعهٔ زبان‌های برنامه‌نویسی اصطلاح «ناجبرگرایی اهریمنی» (Demonic Nondeterminism) را برای موردی که حریف انتخاب‌ها را می‌کند ابداع کرده، در برابر «ناجبرگرایی فرشته‌ای» (Angelic Nondeterminism) که خودِ عامل انتخاب‌ها را می‌کند. این اصطلاح را قرض می‌گیریم تا «معناشناسی فرشته‌ای» (Angelic Semantics) برای توصیف‌های HLA تعریف کنیم. مفهوم پایهٔ لازم برای فهم معناشناسی فرشته‌ای «مجموعهٔ دست‌یافتنی» (Reachable Set) یک HLA است: با حالت s، مجموعهٔ دست‌یافتنی برای HLA h، که REACH(s, h) نوشته می‌شود، مجموعهٔ حالت‌های قابل دستیابی توسط هر یک از پیاده‌سازی‌های HLA است. ایدهٔ کلیدی آن است که عامل می‌تواند انتخاب کند هنگام اجرای HLA به کدام عنصر از مجموعهٔ دست‌یافتنی برسد؛ پس HLA با چند پالایش از همان HLA با پالایش‌های کمتر

«قدرتمندتر» است. همچنین می‌توانیم مجموعهٔ دست‌یافتنی توالی‌ای از HLAها را تعریف کنیم. برای نمونه، مجموعهٔ دست‌یافتنی توالی [h1,h2] اتحاد همهٔ مجموعه‌های دست‌یافتنیِ به‌دست‌آمده با اعمال h2 در هر حالت در مجموعهٔ دست‌یافتنی h1 است:

```
REACH(s, [h1, h2]) = ⋃_{s′ ∈ REACH(s, h1)} REACH(s′, h2)
```

با این تعریف‌ها، برنامهٔ سطح‌بالا — توالی‌ای از HLAها — هدف را محقق می‌کند اگر مجموعهٔ دست‌یافتنی‌اش با مجموعهٔ حالت‌های هدف اشتراک داشته باشد. (این را با شرط بسیار قوی‌تر معناشناسی اهریمنی مقایسه کنید، که در آن هر عضو مجموعهٔ دست‌یافتنی باید حالت هدف باشد.) برعکس، اگر مجموعهٔ دست‌یافتنی با هدف اشتراک نداشته باشد، برنامه قطعاً کار نمی‌کند. شکل ۱۱.۹ این ایده‌ها را نشان می‌دهد.



شکل ۱۱.۹ — مثال‌های شماتیک از مجموعه‌های دست‌یافتنی. مجموعهٔ حالت‌های هدف بنفش سایه‌خورده. پیکان‌های سیاه و خاکستری پیاده‌سازی‌های ممکن h1 و h2 را نشان می‌دهند. (الف) مجموعهٔ دست‌یافتنی HLA هـ h1 در حالت s. (ب) مجموعهٔ دست‌یافتنی توالی [h1,h2]. چون با مجموعهٔ هدف اشتراک دارد، توالی هدف را محقق می‌کند.

مفهوم مجموعه‌های دست‌یافتنی الگوریتم سرراستی می‌دهد: میان برنامه‌های سطح‌بالا جستجو کن، در جست‌وجوی برنامه‌ای که مجموعهٔ دست‌یافتنی‌اش با هدف اشتراک داشته باشد؛ به‌محض وقوع این، الگوریتم می‌تواند به آن برنامهٔ انتزاعی متعهد شود، با دانستن اینکه کار می‌کند، و بر پالایش بیشتر برنامه تمرکز کند. بعداً به مسائل الگوریتمی بازخواهیم گشت؛ فعلاً در نظر بگیرید اثرات HLA — مجموعهٔ دست‌یافتنی برای هر حالت آغازین ممکن — چگونه بازنمایی می‌شوند. کنش اولیه می‌تواند روانی را صادق یا کاذب کند یا بدون تغییر بگذارد. (با اثرات شرطی (بخش ۱۱.۵.۱ را ببینید) امکان چهارمی هست: وارون‌کردن متغیر به ضدش.)

HLA تحت معناشناسی فرشته‌ای می‌تواند بیشتر کند: می‌تواند مقدار روانی را کنترل کند، بسته به اینکه کدام پیاده‌سازی برگزیده شود آن را صادق یا کاذب کند. این بدان معناست HLA می‌تواند نه اثر متفاوت روی روان داشته باشد: اگر متغیر صادق آغاز شود، می‌تواند همیشه آن را صادق نگه دارد، همیشه کاذبش کند، یا انتخاب داشته باشد؛ اگر روان کاذب آغاز شود، می‌تواند همیشه کاذبش نگه دارد، همیشه صادقش کند، یا انتخاب داشته باشد؛ و سه انتخاب برای هر دو مورد را می‌توان خودسرانه ترکیب کرد و نه ساخت.

نمادی، این کمی چالش‌برانگیز است. از زبان فهرست‌های افزودن و حذف (به‌جای روان‌های صادق/کاذب) همراه با نماد ε به معنای «ممکن است، اگر عامل چنین انتخاب کند» استفاده می‌کنیم. پس اثر ε+A یعنی «احتمالاً A را می‌افزاید»، یعنی یا A را بدون تغییر می‌گذارد یا صادقش می‌کند. به همین ترتیب، ε-A یعنی «احتمالاً A را حذف می‌کند» و ε±A یعنی «احتمالاً A را می‌افزاید یا حذف می‌کند». برای نمونه، HLA Go(Home,SFO)، با دو پالایش نشان‌داده‌شده در شکل ۱۱.۷، احتمالاً Cash را حذف می‌کند (اگر عامل تصمیم بگیرد تاکسی بگیرد)، پس باید اثر ε-Cash داشته باشد. پس می‌بینیم توصیف‌های HLAها از توصیف‌های پالایش‌هایشان قابل اشتقاق‌اند. حال، فرض کنیم طرح‌های زیر را برای HLAهای h1 و h2 داشته باشیم:

```
Action(h1, PRECOND: ¬A, EFFECT: A ∧ ε−B)
Action(h2, PRECOND: ¬B, EFFECT: ε+A ∧ ε±C)
```

یعنی h1 به A می‌افزاید و احتمالاً B را حذف می‌کند، در حالی که h2 احتمالاً A را می‌افزاید و کنترل کامل C را دارد. حال، اگر فقط B در حالت آغازین صادق باشد و هدف A ∧ C باشد، آنگاه توالی [h1,h2] هدف را محقق می‌کند: پیاده‌سازی‌ای از h1 برمی‌گزینیم که B را کاذب کند، سپس پیاده‌سازی‌ای از h2 برمی‌گزینیم که A را صادق و C را صادق نگه دارد.

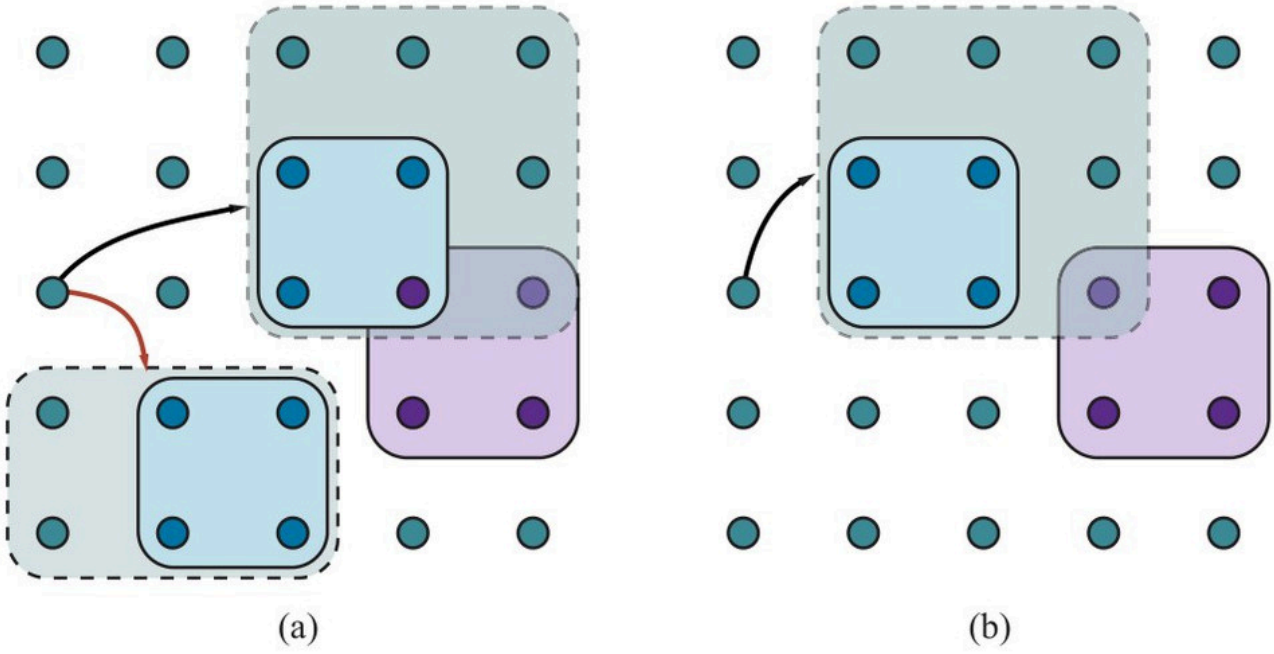
بحث پیشین فرض می‌کند اثرات HLA — مجموعهٔ دست‌یافتنی برای هر حالت آغازینِ داده‌شده — را می‌توان با توصیف اثر روی هر روان دقیقاً توصیف کرد. خوب می‌بود اگر همیشه این‌گونه بود، اما در بسیاری از موارد فقط می‌توانیم اثرات را تقریب بزنیم، زیرا HLA ممکن است پیاده‌سازی‌های بی‌شمار داشته باشد و مجموعه‌های دست‌یافتنیِ خودسرانه موج‌داری تولید کند — بسیار شبیه مسئلهٔ حالت باورِ موج‌دارِ نشان‌داده‌شده در شکل 7.21 در صفحهٔ ۲۴۳. برای نمونه، گفتیم Go(Home,SFO) احتمالاً Cash را حذف می‌کند؛ همچنین احتمالاً At(Car,SFOLongTermParking) را می‌افزاید؛ اما نمی‌تواند هر دو را بکند — در واقع، باید دقیقاً یکی را بکند. مانند حالت‌های باور، ممکن است لازم باشد توصیف‌های تقریبی بنویسیم. از دو نوع تقریب استفاده خواهیم کرد: «توصیف خوش‌بینانه» (Optimistic Description) REACH+(s, h) از HLA هـ h ممکن است مجموعهٔ دست‌یافتنی را بیش‌بیان کند، در حالی که «توصیف بدبینانه» (Pessimistic Description) REACH-(s, h) ممکن است مجموعهٔ دست‌یافتنی را کم‌بیان کند. پس داریم:

```
REACH−(s, h) ⊆ REACH(s, h) ⊆ REACH+(s, h)
```

برای نمونه، توصیف خوش‌بینانهٔ Go(Home,SFO) می‌گوید احتمالاً Cash را حذف می‌کند و احتمالاً At(Car,SFOLongTermParking) را می‌افزاید. مثال خوب دیگر در معمای ۸ رخ می‌دهد، که نیمی از حالت‌هایش از هر حالتِ داده‌شده‌ای دست‌یافتنی نیستند (تمرین PART.11 را ببینید): توصیف خوش‌بینانهٔ Act ممکن است به‌خوبی کل

فضای حالت را شامل شود، زیرا مجموعهٔ دست‌یافتنی دقیق کاملاً موجدار است.

با توصیف‌های تقریبی، آزمون اینکه آیا برنامه‌ای هدف را محقق می‌کند باید کمی اصلاح شود. اگر مجموعهٔ دست‌یافتنی خوش‌بینانه برای برنامه با هدف اشتراک نداشته باشد، برنامه کار نمی‌کند؛ اگر مجموعهٔ دست‌یافتنی بدبینانه با هدف اشتراک داشته باشد، برنامه کار می‌کند (شکل ۱۱.۱۰(الف)). با توصیف‌های دقیق، برنامه یا کار می‌کند یا نه، اما با توصیف‌های تقریبی، میانه‌ای هست: اگر مجموعهٔ خوش‌بینانه با هدف اشتراک داشته باشد اما مجموعهٔ بدبینانه نه، نمی‌توانیم بگوییم برنامه کار می‌کند (شکل ۱۱.۱۰(ب)). وقتی این وضعیت پیش می‌آید، می‌توان عدم قطعیت را با پالایش برنامه حل کرد. این وضعیت بسیار رایج در استدلال انسانی است. برای نمونه، در برنامه‌ریزی تعطیلات دو هفته‌ای هاوایِ یادشده، ممکن است پیشنهاد شود دو روز در هر یک از هفت جزیره بگذرانیم. احتیاط حکم می‌کند این برنامهٔ جاه‌طلبانه با افزودن جزئیات حمل‌ونقل میان‌جزیره‌ای پالایش شود.



شکل ۱۱.۱۰ – دستیابی به هدف برای برنامه‌های سطح‌بالا با توصیف‌های تقریبی. مجموعهٔ حالت‌های هدف بنفش سایه‌خورده. برای هر برنامه، مجموعه‌های دست‌یافتنی بدبینانه (خط‌های توپر، آبی روشن) و خوش‌بینانه (خط‌های چین، سبز روشن) نشان داده شده. (الف) برنامهٔ نشان‌داده‌شده با پیکان سیاه قطعاً هدف را محقق می‌کند، در حالی که برنامهٔ نشان‌داده‌شده با پیکان خاکستری قطعاً محقق نمی‌کند. (ب) برنامه‌ای که احتمالاً هدف را محقق می‌کند (مجموعهٔ خوش‌بینانه با هدف اشتراک دارد) اما لزوماً نه (مجموعهٔ بدبینانه با هدف اشتراک ندارد). برنامه باید بیشتر پالایش شود تا تعیین شود واقعاً هدف را محقق می‌کند یا نه.

الگوریتم برنامه‌ریزی سلسله‌مراتبی با توصیف‌های فرشته‌ای تقریبی در شکل ۱۱.۱۱ نشان داده شده. برای سادگی، همان طرح کلی استفاده‌شده در شکل ۱۱.۸ را حفظ کرده‌ایم، یعنی جستجوی سطح-اول در فضای پالایش‌ها. چنان‌که تازه توضیح دادیم، الگوریتم می‌تواند برنامه‌هایی را که کار خواهند کرد و نمی‌کنند با بررسی اشتراک‌های مجموعه‌های دست‌یافتنی خوش‌بینانه و بدبینانه با هدف شناسایی کند. (جزئیات محاسبهٔ مجموعه‌های دست‌یافتنی برنامه با توصیف‌های تقریبی هر گام در تمرین HLAP.11 پوشش داده شده.)

```
function ANGELIC-SEARCH(problem, hierarchy, initialPlan) returns solution or fail
                    frontier ← a FIFO queue with initialPlan as the only element
                                                                  while true do
                                                  if EMPTY?(frontier) then return fail
      plan ← POP(frontier)          // selects the shallowest node in frontier
                 if REACH+(problem.INITIAL, plan) intersects problem.GOAL then
                                                  if plan is initial then return plan
                    guaranteed ← REACH−(problem.INITIAL, plan) ∩ problem.GOAL
              if guaranteed ≠ {} and MAKING-PROGRESS(plan, initialPlan) then
                                                  finalState ← an element of guaranteed
          return DECOMPOSE(hierarchy, problem.INITIAL, plan, finalState)
                                                                hla ← some HLA in plan
    prefix, suffix ← subsequences of actions before and after hla in plan
                                                  outcome ← RESULT(problem.INITIAL, prefix)
                  for each sequence in REFINEMENTS(hla, outcome, hierarchy) do
                frontier ← Insert(APPEND(prefix, sequence, suffix), frontier)

                    function DECOMPOSE(hierarchy, s0, plan, sf) returns a solution
                                                                  solution ← an empty plan
                                                               while plan is not empty do
                                                             action ← REMOVE-LAST(plan)
          si ← a state in REACH−(s0, plan) such that sf ∈ REACH−(si, action)
                                  problem ← a problem with INITIAL = si and GOAL = sf
      solution ← APPEND(ANGELIC-SEARCH(problem, hierarchy, action), solution)
                                                                              sf ← si
                                                                    return solution
```

شکل ۱۱.۱۱ – الگوریتم برنامه‌ریزی سلسله‌مراتبی که از معناشناسی فرشته‌ای برای شناسایی و تعهد به برنامه‌های سطح‌بالای کارکننده استفاده می‌کند، در حالی که از برنامه‌های سطح‌بالای غیرکارکننده اجتناب می‌کند. گزارهٔ MAKING-PROGRESS بررسی می‌کند مطمئن شویم در بازگشت بی‌نهایت پالایش‌ها گیر نکرده‌ایم. در سطح بالا، ANGELIC-SEARCH را با [Act] به‌عنوان initialPlan فراخوانی کن.

وقتی برنامهٔ انتزاعی کارکننده‌ای یافت شد، الگوریتم مسئلهٔ اصلی را به زیرمسئله‌هایی تجزیه می‌کند، برای هر گام برنامه یکی. حالت آغازین و هدف هر زیرمسئله با بازگشت حالت هدفِ تضمیناً-دست‌یافتنی از طریق طرح‌های کنش برای هر گام برنامه به دست می‌آید. (برای بحث چگونگی کار بازگشت، بخش ۱۱.۲.۲ را ببینید.) شکل ۱۱.۹(ب) ایدهٔ پایه را نشان می‌دهد: حالت دایره‌شدهٔ سمت راست، حالت هدفِ تضمیناً-دست‌یافتنی است و حالت دایره‌شدهٔ سمت چپ، هدف میانیِ به‌دست‌آمده با بازگشت هدف از طریق کنش نهایی است.

توانایی تعهد یا رد برنامه‌های سطح‌بالا می‌تواند به ANGELIC-SEARCH برتری محاسباتی معناداری بر HIERARCHICAL-SEARCH بدهد، که به نوبهٔ خود ممکن است برتری بزرگ بر BREADTH-FIRST-SEARCH قدیمی داشته باشد. برای نمونه، تمیزکردن جهان جاروبرقی بزرگی متشکل از آرایشی از اتاق‌های متصل با راهروهای باریک را در نظر بگیرید، که هر اتاق مستطیل $h \times w$ از خانه‌هاست. منطقی است HLA برای Navigate (چنان‌که در شکل ۱۱.۷) و یکی برای CleanWholeRoom داشته باشیم. (تمیزکردن اتاق را می‌شد با اعمال مکرر HLA دیگری برای تمیزکردن هر ردیف پیاده کرد.) چون پنج کنش اولیه هست، هزینهٔ BREADTH-FIRST-SEARCH به‌صورت 5^d رشد می‌کند، که d طول کوتاه‌ترین راه‌حل است (تقریباً دو برابر کل شمار خانه‌ها)؛ الگوریتم حتی دو اتاق 3×3 را هم نمی‌تواند مدیریت کند.

HIERARCHICAL-SEARCH کارآمدتر است، اما همچنان از رشد نمایی رنج می‌برد، زیرا همهٔ راه‌های تمیزکردنِ سازگار با سلسله‌مراتب را می‌آزماید. ANGELIC-SEARCH تقریباً به‌صورت خطی در شمار خانه‌ها مقیاس می‌شود — به توالی سطح‌بالای خوبی از گام‌های تمیزکردن-اتاق و ناوبری متعهد می‌شود و دیگر گزینه‌ها را هرس می‌کند.

تمیزکردن مجموعه‌اتاق‌ها با تمیزکردن هر اتاق به‌نوبت کار شاقی نیست: این کار برای انسان‌ها به‌سببِ ساختارِ سلسله‌مراتبیِ مسئله آسان است. وقتی در نظر بگیریم حل معماهای کوچک مانند معمای ۸ برای انسان‌ها چقدر دشوار است، محتمل می‌نماید توانایی انسان در حل مسائل پیچیده نه از در نظر گرفتن ترکیبیات، بلکه از مهارت در انتزاع و تجزیهٔ مسائل برای حذف ترکیبیات ناشی می‌شود.

رویکرد فرشته‌ای را می‌توان با تعمیم مفهوم مجموعهٔ دست‌یافتنی برای یافتن راه‌حل‌های کم‌هزینه بسط داد. به‌جای دست‌یافتنی‌بودن یا نبودن حالت، هر حالت هزینه‌ای برای کارآمدترین راه رسیدن به آن‌جا خواهد داشت. (هزینه برای حالت‌های دست‌یافتنی‌ناشدنی بی‌نهایت است.) توصیف‌های خوش‌بینانه و بدبینانه این هزینه‌ها را کران‌دار می‌کنند. بدین ترتیب، جستجوی فرشته‌ای می‌تواند برنامه‌های انتزاعیِ اثباتاً بهینه را بدون در نظر گرفتن پیاده‌سازی‌هایشان بیابد. همان رویکرد را می‌توان برای به‌دست‌آوردن الگوریتم‌های «نگاه به جلو» (Look-Ahead) سلسله‌مراتبی مؤثر برای جستجوی آنلاین، به سبک LRTA* (صفحهٔ ۱۴۰)، استفاده کرد.

از برخی جهات، چنین الگوریتم‌هایی جنبه‌های تأمل انسانی را در تکلیف‌هایی مانند برنامه‌ریزی تعطیلات هاوایی بازتاب می‌دهند — در نظر گرفتن بدیل‌ها در آغاز در سطح انتزاعی روی مقیاس‌های زمانی بلند انجام می‌شود؛ برخی بخش‌های برنامه تا زمان اجرا کاملاً انتزاعی رها می‌شوند، مانند چگونگی گذراندن دو روز بی‌خیال در مولوکای، در حالی که بخش‌های دیگر با جزئیات برنامه‌ریزی می‌شوند، مانند پروازهای گرفته‌شده و اقامتگاهِ رزروشده — بدون این پالایش‌های اخیر، هیچ تضمینی نیست برنامه شدنی باشد.

۱۱.۵ برنامه‌ریزی و کنش در حوزه‌های ناجبرگرایانه

در این بخش برنامه‌ریزی را برای مدیریت محیط‌های تا حدی قابل مشاهده، ناجبرگرایانه و ناشناخته بسط می‌دهیم. مفاهیم پایه مفاهیم فصل ۴ را بازتاب می‌دهند، اما تفاوت‌هایی از استفادهٔ بازنمایی‌های عاملی به‌جای بازنمایی‌های اتمی ناشی می‌شود. این بر شیوهٔ بازنمایی توانایی عامل برای کنش و مشاهده و بر شیوهٔ بازنمایی حالت‌های باور — مجموعه‌های حالت‌های فیزیکی ممکنی که عامل ممکن است در آن‌ها باشد — برای محیط‌های تا حدی قابل مشاهده اثر می‌گذارد.

همچنین می‌توانیم از بسیاری از روش‌های مستقل از حوزهٔ بخش ۱۱.۳ برای محاسبهٔ اکتشافی‌های جستجو بهره بگیریم.

برنامه‌ریزی بی‌حسگر (که برنامه‌ریزی سازگار (Conformant Planning) نیز نامیده می‌شود) برای محیط‌های بدون مشاهده؛ برنامه‌ریزی احتیاطی (Contingency Planning) برای محیط‌های تا حدی قابل مشاهده و ناجبرگرایانه؛ و برنامه‌ریزی آنلاین و برنامه‌ریزی مجدد برای محیط‌های ناشناخته را پوشش خواهیم داد. این به ما امکان می‌دهد مسائل جهان واقعیِ قابل توجهی را هدف بگیریم.

این مسئله را در نظر بگیرید: با یک صندلی و یک میز، هدف هماهنگ‌بودن آن‌هاست — همان رنگ را داشته باشند. در حالت آغازین دو قوطی رنگ داریم، اما رنگ‌های رنگ و مبلمان ناشناخته‌اند. فقط میز در آغاز در میدان دید عامل است:

```
Init(Object(Table) ∧ Object(Chair) ∧ Can(C1) ∧ Can(C2) ∧ InView(Table))
Goal(Color(Chair,c) ∧ Color(Table,c))
```

دو کنش هست: برداشتن درِ قوطی رنگ و رنگ‌کردن شیء با رنگ قوطیِ باز.

```
,Action(RemoveLid(can)
   PRECOND: Can(can)
  (EFFECT: Open(can)
,Action(Paint(x, can)
PRECOND: Object(x) ∧ Can(can) ∧ Color(can,c) ∧ Open(can)
  (EFFECT: Color(x,c)
```

طرح‌های کنش سرراست‌اند، با یک استثنا: پیش‌شرط‌ها و اثرات اکنون ممکن است متغیرهایی داشته باشند که بخشی از فهرست متغیرهای کنش نیستند. یعنی Paint(x,can) به متغیر c، نمایانگر رنگ رنگِ درون قوطی، اشاره نمی‌کند. در حالت کاملاً قابل مشاهده، این مجاز نیست — باید کنش را Paint(x,can,c) نام‌گذاری می‌کردیم. اما در حالت تا حدی قابل مشاهده، ممکن است بدانیم یا ندانیم چه رنگی در قوطی است.

برای حل مسئلهٔ تا حدی قابل مشاهده، عامل باید دربارهٔ ادراک‌هایی که هنگام اجرای برنامه به دست خواهد آورد استدلال کند. ادراک هنگام کنش واقعی توسط حسگرهای عامل عرضه خواهد شد، اما هنگام برنامه‌ریزی به مدلی از حسگرهایش نیاز خواهد داشت. در فصل ۴، این مدل با تابع PERCEPT(s) داده می‌شد. برای برنامه‌ریزی، PDDL را با نوع تازه‌ای از طرح، «طرح ادراک» (Percept Schema)، تقویت می‌کنیم:

```
,Percept(Color(x,c)
(PRECOND: Object(x) ∧ InView(x)
,Percept(Color(can,c)
(PRECOND: Can(can) ∧ InView(can) ∧ Open(can)
```

طرح نخست می‌گوید هرگاه شیئی در دید است، عامل رنگ شیء را ادراک خواهد کرد (یعنی برای شیء x، عامل مقدار صدق Color(x,c) را برای همهٔ c خواهد آموخت). طرح دوم می‌گوید اگر قوطی بازِ در دید باشد، عامل رنگ رنگِ درون قوطی را ادراک می‌کند. چون در این جهان رویدادهای برون‌زا نیست، رنگ شیء همان باقی می‌ماند، حتی اگر ادراک نشود، تا آنکه عامل کنشی برای تغییر رنگ شیء انجام دهد. البته، عامل به کنشی نیاز خواهد داشت که باعث شود اشیا (یکی‌یکی) به دید آیند:

```
,Action(LookAt(x)
PRECOND: InView(y) ∧ (x ≠ y)
(EFFECT: InView(x) ∧ ¬InView(y)
```

برای محیط کاملاً قابل مشاهده، برای هر روان طرح ادراکی بدون پیش‌شرط می‌داشتیم. عامل بی‌حسگر، از سوی دیگر، اصلاً طرح ادراک ندارد. توجه کنید حتی عامل بی‌حسگر هم می‌تواند مسئلهٔ رنگ‌کردن را حل کند. یکی از راه‌حل‌ها بازکردن هر قوطی رنگی و اعمال آن بر هر دو صندلی و میز است، و بدین ترتیب آن‌ها را به همان رنگ وادار می‌کند (حتی با آنکه عامل نمی‌داند رنگ چیست).

عامل برنامه‌ریزی احتیاطی با حسگرها می‌تواند برنامهٔ بهتری تولید کند. نخست، به میز و صندلی نگاه کن تا رنگ‌هایشان به دست آید؛ اگر از قبل یکسان‌اند، برنامه تمام است. اگر نه، به قوطی‌های رنگ نگاه کن؛ اگر رنگ قوطی‌ای همان رنگ یکی از مبلمان است، آن رنگ را به قطعهٔ دیگر اعمال کن. وگرنه، هر دو قطعه را با هر رنگی رنگ کن.

سرانجام، عامل برنامه‌ریزی آنلاین ممکن است ابتدا برنامهٔ احتیاطی با شاخه‌های کمتر تولید کند — شاید امکانِ همخوان‌نداشتن هیچ قوطی‌ای با هیچ‌یک از مبلمان را نادیده بگیرد — و هنگام بروز مسائل با برنامه‌ریزی مجدد با آن‌ها کنار بیاید. همچنین می‌تواند با نادرستی طرح‌های کنشش کنار بیاید. در حالی که برنامه‌ریز احتیاطی صرفاً فرض می‌کند اثرات کنش همیشه موفق‌اند — که رنگ‌کردن صندلی کار را تمام می‌کند — عامل برنامه‌ریزی مجدد نتیجه را بررسی می‌کرد و برنامهٔ اضافی برای اصلاح هر شکست غیرمنتظره می‌ساخت، مانند ناحیهٔ رنگ‌نشده یا نمایان‌بودن رنگ اصلی.

در جهان واقعی، عامل‌ها از ترکیبی از رویکردها استفاده می‌کنند. خودروسازان لاستیک زاپاس و کیسهٔ هوا می‌فروشند، که تجسم‌های فیزیکی شاخه‌های برنامهٔ احتیاطی طراحی‌شده برای مدیریت پنچری یا تصادف‌اند. از سوی دیگر، بیشتر رانندگان هرگز این امکان‌ها را در نظر نمی‌گیرند؛ وقتی مسئله‌ای پیش می‌آید، به‌عنوان عامل‌های برنامه‌ریزی مجدد پاسخ می‌دهند. در حالت کلی، عامل‌ها فقط برای احتیاط‌هایی برنامه‌ریزی می‌کنند که پیامدهای مهم و شانس وقوع ناچیزنداشتنی دارند. پس راننده‌ای که سفر با خودرو در صحرای صحرا را در نظر دارد باید برنامه‌های احتیاطی صریحی برای خرابی‌ها بسازد، در حالی که سفر به سوپرمارکت به برنامه‌ریزی قبلی کمتری نیاز دارد. در ادامه به هر یک از سه رویکرد با جزئیات بیشتری نگاه می‌کنیم.

۱۱.۵.۱ برنامه‌ریزی بی‌حسگر

بخش 4.4.1 (صفحهٔ ۱۲۶) ایدهٔ پایهٔ جستجو در فضای حالت باور برای یافتن راه‌حل مسائل بی‌حسگر را معرفی کرد. تبدیل مسئلهٔ برنامه‌ریزی بی‌حسگر به مسئلهٔ برنامه‌ریزی حالت باور تقریباً همان‌گونه که در بخش 4.4.1 انجام شد کار می‌کند؛ تفاوت‌های اصلی آن است که مدل گذار فیزیکی زیرین با مجموعه‌ای از طرح‌های کنش بازنمایی می‌شود و حالت باور را می‌توان با فرمول منطقی بازنمایی کرد نه با مجموعه‌ای صریحاً برشمرده‌شده از حالت‌ها. فرض می‌کنیم مسئلهٔ برنامه‌ریزی زیرین جبرگرایانه است.

حالت باور آغازین برای مسئلهٔ رنگ‌کردن بی‌حسگر می‌تواند روان‌های InView را نادیده بگیرد، زیرا عامل حسگری ندارد. وانگهی، واقعیت‌های تغییرناپذیر Can(C2) ∧ Can(C1) ∧ Object(Chair) ∧ Object(Table) را داده‌شده می‌گیریم، زیرا در هر حالت باوری برقرارند. عامل رنگ‌های قوطی‌ها یا اشیا را نمی‌داند، یا باز یا بسته‌بودن قوطی‌ها را، اما می‌داند اشیا و قوطی‌ها رنگی دارند: ∀x ∃c Color(x,c). پس از اسکولم‌سازی (بخش 9.5.1 را ببینید)، حالت باور آغازین را به دست می‌آوریم:

```
b0 = Color(x, C(x))
```

در برنامه‌ریزی کلاسیک، که در آن فرض جهان بسته می‌شود، فرض می‌کردیم هر روانی که در حالت ذکر نشده کاذب است، اما در برنامه‌ریزی بی‌حسگر (و تا حدی قابل مشاهده) باید به فرض جهان باز سوئیچ کنیم که در آن حالت‌ها شامل روان‌های مثبت و منفی‌اند و اگر روانی ظاهر نشود، مقدارش ناشناخته است. پس حالت باور دقیقاً با مجموعهٔ جهان‌های ممکنِ ارضاکنندهٔ فرمول متناظر است. با این حالت باور آغازین، توالی کنش زیر راه‌حل است:

```
[RemoveLid(Can1), Paint(Chair, Can1), Paint(Table, Can1)]
```

اکنون نشان می‌دهیم چگونه حالت باور را از میان توالی کنش پیش ببریم تا نشان دهیم حالت باور نهایی هدف را ارضا می‌کند.

نخست، توجه کنید در حالت باورِ داده‌شدهٔ b، عامل می‌تواند هر کنشی را در نظر بگیرد که پیش‌شرط‌هایش توسط b برآورده شوند. (کنش‌های دیگر را نمی‌توان استفاده کرد، زیرا مدل گذار اثرات کنش‌هایی را که پیش‌شرط‌هایشان ممکن است برآورده نباشند تعریف نمی‌کند.) طبق معادلهٔ (4.4) (صفحهٔ ۱۲۷)، فرمول عمومی به‌روزرسانی حالت باور b با کنش کاربردپذیر a در جهان جبرگرایانه چنین است:

```
{s ∈ b و s′ : s′ = RESULTP(s, a)} = b′ = RESULT(b, a)
```

که RESULTP مدل گذار فیزیکی را تعریف می‌کند. فعلاً فرض می‌کنیم حالت باور آغازین همیشه عطفی از لیترال‌هاست، یعنی فرمول CNF-1. برای ساختن حالت باور تازهٔ b'، باید در نظر بگیریم چه اتفاقی برای هر لیترال ℓ در هر حالت فیزیکی s در b هنگام اعمال کنش a می‌افتد. برای لیترال‌هایی که مقدار صدقشان از قبل در b معلوم است، مقدار صدق در b' از مقدار کنونی و فهرست افزودن و فهرست حذف کنش محاسبه می‌شود. (برای نمونه، اگر ℓ در فهرست حذف کنش باشد، ¬ℓ به b' افزوده می‌شود.) دربارهٔ لیترالی که مقدار صدقش در b ناشناخته است چه؟ سه حالت هست:

1. اگر کنش ℓ را بیفزاید، ℓ در b' صادق خواهد بود صرف‌نظر از مقدار آغازینش.
2. اگر کنش ℓ را حذف کند، ℓ در b' کاذب خواهد بود صرف‌نظر از مقدار آغازینش.
3. اگر کنش روی ℓ اثری نگذارد، ℓ مقدار آغازینش را (که ناشناخته است) حفظ می‌کند و در b' ظاهر نخواهد شد.

از این رو، می‌بینیم محاسبهٔ b' تقریباً با حالت قابل مشاهده یکسان است، که با معادلهٔ (۱۱.۱) در صفحهٔ ۳۴۵ مشخص شد:

```
b′ = RESULT(b, a) = (b − DEL(a)) ∪ ADD(a)
```

نمی‌توانیم دقیقاً از معناشناسی مجموعه استفاده کنیم، زیرا (۱) باید مطمئن شویم 'b شامل هر دو ℓ و ¬ℓ نیست، و (۲) اتم‌ها ممکن است متغیرهای بی‌اتصال داشته باشند. اما همچنان چنین است که RESULT(b, a) با شروع از b، کاذب‌کردن هر اتمی که در DEL(a) ظاهر می‌شود و صادق‌کردن هر اتمی که در ADD(a) ظاهر می‌شود محاسبه می‌شود. برای نمونه، اگر RemoveLid(Can1) را بر حالت باور آغازین b0 اعمال کنیم، می‌گیریم:

```
b1 = Color(x, C(x)) ∧ Open(Can1)
```

وقتی کنش Paint(Chair,Can1) را اعمال می‌کنیم، پیش‌شرط Color(Can1,c) توسط لیترال Color(x,C(x)) با اتصال {x/Can1, c/C(Can1)} برآورده می‌شود و حالت باور تازه است:

```
b2 = Color(x, C(x)) ∧ Open(Can1) ∧ Color(Chair, C(Can1))
```

سرانجام، کنش Paint(Table,Can1) را اعمال می‌کنیم تا به دست آوریم:

```
b3 = Color(x, C(x)) ∧ Open(Can1) ∧ Color(Chair, C(Can1)) ∧ Color(Table, C(Can1))
```

حالت باور نهایی هدف، Color(Chair,c) ∧ Color(Table,c)، را با متغیر c اتصال‌یافته به C(Can1) ارضا می‌کند.

تحلیل پیشین قاعدهٔ به‌روزرسانی واقعیت بسیار مهمی را نشان داده: خانوادهٔ حالت‌های باورِ تعریف‌شده به‌صورت عطف‌هایی از لیترال‌ها تحت به‌روزرسانی‌های تعریف‌شده توسط طرح‌های کنش PDDL بسته است. یعنی اگر حالت باور به‌صورت عطفی از لیترال‌ها آغاز شود، هر به‌روزرسانی عطفی از لیترال‌ها خواهد داد. این بدان معناست در جهانی با n روان، هر حالت باور را می‌توان با عطفی به اندازهٔ O(n) بازنمایی کرد. این نتیجهٔ بسیار آرامش‌بخشی است، با توجه به اینکه 2^n حالت در جهان هست. می‌گوید می‌توانیم همهٔ زیرمجموعه‌های آن 2^n حالت را که هرگز لازم خواهیم داشت فشرده بازنمایی کنیم. وانگهی، فرایند بررسی حالت‌های باوری که زیرمجموعه یا فوق‌مجموعهٔ حالت‌های باورِ قبلاً بازدیدشده‌اند نیز آسان است، دست‌کم در حالت گزاره‌ای.

اشکالِ کار در این تصویرِ دلپذیر آن است که فقط برای طرح‌های کنشی کار می‌کند که در همهٔ حالت‌هایی که پیش‌شرط‌هایشان برآورده می‌شود اثرات یکسانی دارند. این ویژگی است که حفظ بازنمایی حالت باور CNF-1 را ممکن می‌کند. به‌محض اینکه اثر بتواند به حالت بستگی داشته باشد، وابستگی‌هایی میان روان‌ها معرفی می‌شوند و ویژگی CNF-1 از دست می‌رود.

برای نمونه، جهان جاروبرقی سادهٔ تعریف‌شده در بخش 3.2.1 را در نظر بگیرید. روان‌ها AtL و AtR برای مکان ربات و CleanL و CleanR برای حالت خانه‌ها باشند. طبق تعریف مسئله، کنش Suck پیش‌شرط ندارد — همیشه می‌توان انجامش داد. دشواری آن است که اثرش به مکان ربات بستگی دارد: وقتی ربات AtL است، نتیجه CleanL است، اما وقتی AtR است، نتیجه CleanR است. برای چنین کنش‌هایی، طرح‌های کنش ما به چیز تازه‌ای نیاز خواهند داشت:

«اثر شرطی» (Conditional Effect). این‌ها نحو «when condition: effect» دارند، که condition فرمول منطقی‌ای است که با حالت کنونی مقایسه می‌شود و effect فرمولی است که حالت حاصل را توصیف می‌کند. برای جهان جاروبرقی:

```
,Action(Suck
(EFFECT: when AtL: CleanL ∧ when AtR: CleanR
```

وقتی بر حالت باور آغازین True اعمال شود، حالت باور حاصل (AtL ∧ CleanL) ∨ (AtR ∧ CleanR) است که دیگر در CNF-1 نیست. (این گذار را می‌توان در شکل 4.14 در صفحهٔ ۱۲۹ دید.) در حالت کلی، اثرات شرطی می‌توانند وابستگی‌های دلخواهی میان روان‌های حالت باور القا کنند و در بدترین حالت به حالت‌های باور با اندازهٔ نمایی بینجامند.

مهم است تفاوت میان پیش‌شرط‌ها و اثرات شرطی را بفهمیم. همهٔ اثرات شرطی که شرایطشان برآورده می‌شود برای تولید حالت باور حاصل اعمال می‌شوند؛ اگر هیچ‌کدام برآورده نشود، حالت حاصل بدون تغییر است. از سوی دیگر، اگر پیش‌شرطی برآورده نشود، کنش ناکاربردپذیر است و حالت حاصل تعریف‌نشده. از دیدگاه برنامه‌ریزی بی‌حسگر، داشتن اثرات شرطی بهتر از کنش ناکاربردپذیر است. برای نمونه، می‌توانستیم Suck را به دو کنش با اثرات بدون‌شرط تقسیم کنیم:

```
Action(SuckL, PRECOND: AtL; EFFECT: CleanL)
 Action(SuckR, PRECOND: AtR; EFFECT: CleanR)
```

حال فقط طرح‌های بدون‌شرط داریم، پس حالت‌های باور همگی در CNF-1 می‌مانند؛ متأسفانه، نمی‌توانیم کاربردپذیری SuckL و SuckR را در حالت باور آغازین تعیین کنیم.

پس اجتناب‌ناپذیر می‌نماید مسائل غیربدیهی شامل حالت‌های باورِ موج‌دار باشند، دقیقاً مانند آن‌هایی که هنگام در نظر گرفتن مسئلهٔ تخمین حالت برای جهان وومپوس دیدیم (شکل 7.21 در صفحهٔ ۲۴۳ را ببینید). راه‌حل پیشنهادیِ آن‌جا استفاده از تقریب محافظه‌کارانه از حالت باور دقیق بود؛ برای نمونه، حالت باور می‌تواند در CNF-1 بماند اگر شامل همهٔ لیترال‌هایی باشد که مقادیر صدقشان قابل تعیین است و همهٔ لیترال‌های دیگر را ناشناخته تلقی کند. هرچند این رویکرد درست است، از آن جهت که هرگز برنامهٔ نادرستی تولید نمی‌کند، ناقص است، زیرا ممکن است نتواند راه‌حل‌هایی برای مسائلی بیابد که لزوماً شامل تعامل‌های میان لیترال‌ها هستند. برای دادن مثال پیش‌پاافتاده، اگر هدف این باشد که ربات روی خانهٔ تمیزی باشد، [Suck] راه‌حل است، اما عامل بی‌حسگری که بر حالت‌های باور 1-CNF اصرار دارد آن را نخواهد یافت.

شاید راه‌حل بهتر، جست‌وجوی توالی‌های کنشی باشد که حالت باور را تا حد ممکن ساده نگه می‌دارند. در جهان جاروبرقی بی‌حسگر، توالی کنش [Right,Suck,Left,Suck] توالی حالت‌های باور زیر را تولید می‌کند:

```
b0 = True
b1 = AtR
b2 = AtR ∧ CleanR
b3 = AtL ∧ CleanR
b4 = AtL ∧ CleanR ∧ CleanL
```

یعنی عامل می‌تواند مسئله را در حالی که حالت باور CNF-1 حفظ می‌شود حل کند، حتی با آنکه برخی توالی‌ها (مثلاً آن‌هایی که با Suck آغاز می‌شوند) خارج از CNF-1 می‌روند. درس عمومی بر انسان‌ها گم نمی‌شود: ما همیشه کنش‌های کوچکی انجام می‌دهیم (نگاه‌کردن به ساعت، دست‌زدن به جیب‌ها برای اطمینان از داشتن کلید خودرو، خواندن تابلوهای خیابان هنگام ناوبری در شهر) تا عدم قطعیت را حذف کنیم و حالت باورمان را قابل‌مدیریت نگه داریم.

رویکرد دیگری، کاملاً متفاوت، به مسئلهٔ حالت‌های باورِ به‌طرز غیرقابل‌مدیریتی موجودار هست: اصلاً زحمت محاسبهٔ آن‌ها را به خود ندهید. فرض کنیم حالت باور آغازین b0 است و می‌خواهیم حالت باورِ حاصل از توالی کنش [a1,...,am] را بدانیم. به‌جای محاسبهٔ صریح آن، فقط آن را «b0 سپس [a1,...,am]» بازنمایی کنید. این بازنمایی تنبل اما بدون‌ابهام از حالت باور است و کاملاً موجز است — O(n+m)، که n اندازهٔ حالت باور آغازین (فرض‌شده در CNF-1) و m بیشینهٔ طول توالی کنش است. به‌عنوان بازنمایی حالت باور، یک نقطه‌ضعف دارد: تعیین اینکه آیا هدف ارضا می‌شود یا کنشی کاربردپذیر است ممکن است محاسبهٔ زیادی لازم داشته باشد.

محاسبه را می‌توان به‌صورت آزمون لازمه پیاده کرد: اگر Am مجموعهٔ اصل‌موضوعه‌های حالت-جانشین لازم برای تعریف رخدادهای کنش‌های a1,...,am باشد — چنان‌که برای SATPLAN در بخش ۱۱.۲.۳ توضیح داده شد — و Gm ادعا کند هدف پس از m گام صادق است، آنگاه برنامه هدف را محقق می‌کند اگر b0 ∧ Am ⊨ Gm — یعنی اگر b0 ∧ Am ∧ ¬Gm ارضاپذیر نباشد. با حل‌کنندهٔ SAT مدرن، ممکن است این کار بسیار سریع‌تر از محاسبهٔ حالت باور کامل انجام شود. برای نمونه، اگر هیچ‌یک از کنش‌های توالی، روان هدف خاصی را در فهرست افزودنش نداشته باشد، حل‌کننده بلافاصله این را تشخیص می‌دهد. همچنین کمک می‌کند اگر نتایج جزئی دربارهٔ حالت باور — برای نمونه، روان‌های معلوم‌الصدق یا معلوم‌الکذب — کش شوند تا محاسبات بعدی ساده شوند.

قطعهٔ پایانی معمای برنامه‌ریزی بی‌حسگر تابع اکتشافی برای هدایت جست‌وجوست. معنای تابع اکتشافی همان برنامه‌ریزی کلاسیک است: تخمینی (شاید مقبول) از هزینهٔ رسیدن به هدف از حالت باورِ داده‌شده. با حالت‌های باور، یک واقعیت اضافی داریم: حل هر زیرمجموعه‌ای از حالت باور لزوماً آسان‌تر از حل خودِ حالت باور است:

```
اگر b1 ⊆ b2 آنگاه h*(b1) ≤ h*(b2)
```

از این رو، هر اکتشافی مقبولی که برای زیرمجموعه‌ای محاسبه شود برای خودِ حالت باور مقبول است. بدیهی‌ترین نامزدها زیرمجموعه‌های تکی‌اند، یعنی حالت‌های فیزیکی فردی. می‌توانیم هر مجموعهٔ تصادفی از حالت‌های s1,...,sN را که در حالت باور b هستند بگیریم، هر اکتشافی مقبول h را اعمال کنیم و

```
H(b) = max{h(s1), ..., h(sN)}
```

را به‌عنوان تخمین اکتشافی برای حل b بازگردانیم. همچنین می‌توانیم از اکتشافی‌های نامقبول مانند اکتشافی نادیده‌گرفتن-فهرست‌های-حذف (صفحهٔ ۳۵۴) استفاده کنیم که در عمل خوب کار می‌کند.

۱۱.۵.۲ برنامه‌ریزی احتیاطی

در فصل ۴ دیدیم برنامه‌ریزی احتیاطی — تولید برنامه‌های با انشعاب شرطی بر اساس ادراک‌ها — برای محیط‌های با مشاهده‌ناپذیری تا حدی، ناجبرگرایی، یا هر دو مناسب است. برای مسئلهٔ رنگ‌کردنِ تا حدی قابل مشاهده با طرح‌های ادراکِ پیش‌ترداده‌شده، یکی از راه‌حل‌های شرطی ممکن چنین است:

```
                                                ,LookAt(Table), LookAt(Chair)]
                                    if Color(Table, c) ∧ Color(Chair, c) then NoOp
                ,else [RemoveLid(Can1), LookAt(Can1), RemoveLid(Can2), LookAt(Can2)
                    if Color(Table, c) ∧ Color(can, c) then Paint(Chair, can)
            else if Color(Chair, c) ∧ Color(can, c) then Paint(Table, can)
                            [[else [Paint(Chair, Can1), Paint(Table, Can1)]
```

متغیرهای این برنامه را باید وجودی‌سوردار در نظر گرفت؛ خط دوم می‌گوید اگر رنگی c وجود دارد که رنگ میز و صندلی است، عامل برای رسیدن به هدف لازم نیست کاری کند. هنگام اجرای این برنامه، عامل برنامه‌ریزی احتیاطی می‌تواند حالت باورش را به‌صورت فرمول منطقی نگه دارد و هر شرط شاخه را با تعیین اینکه آیا حالت باور فرمول شرط یا نقیضش را مستلزم است ارزیابی کند. (وظیفهٔ الگوریتم برنامه‌ریزی احتیاطی است که مطمئن شود عامل هرگز به حالت باوری نمی‌رسد که در آن مقدار صدق فرمول شرط ناشناخته است.) توجه کنید با شرایط مرتبهٔ اول، فرمول ممکن است به بیش از یک راه ارضا شود؛ برای نمونه، شرط Color(can,c) ∧ Color(Table,c) ممکن است با {can/Can1} و با {can/Can2} ارضا شود اگر هر دو قوطی همان رنگ میز باشند. در آن صورت، عامل می‌تواند هر جانشینی ارضاکننده‌ای را برای اعمال به بقیهٔ برنامه برگزیند.

چنان‌که در بخش 4.4.2 نشان داده شد، محاسبهٔ حالت باور تازهٔ $\hat{b}$ پس از کنش a و ادراک پسین در دو مرحله انجام می‌شود. مرحلهٔ نخست حالت باور پس از کنش را محاسبه می‌کند، دقیقاً مانند عامل بی‌حسگر:

$$\hat{b} = (b - \text{DEL}(a)) \cup \text{ADD}(a)$$

که در آن، مانند قبل، حالت باوری را فرض کرده‌ایم که به‌صورت عطفی از لیترال‌ها بازنمایی می‌شود. مرحلهٔ دوم کمی مشکل‌تر است. فرض کنیم لیترال‌های ادراک p1,...,pk دریافت شده‌اند. ممکن است فکر کنید به‌سادگی باید این‌ها را به حالت باور بیفزاییم؛ در واقع، همچنین می‌توانیم استنتاج کنیم پیش‌شرط‌های حس‌کردن برآورده‌اند. حال، اگر ادراک p دقیقاً یک طرح ادراک داشته باشد، Percept(p, PRECOND: c)، که c عطفی از لیترال‌هاست، آنگاه آن لیترال‌ها را

می‌توان همراه با p به حالت باور انداخت. از سوی دیگر، اگر p بیش از یک طرح ادراک داشته باشد که پیش‌شرط‌هایش ممکن است طبق حالت باور پیش‌بینی‌شدهٔ $\hat{b}$ برقرار باشند، باید فصل پیش‌شرط‌ها را بیفزاییم. بدیهی است این حالت باور را خارج از CNF-1 می‌برد و همان پیچیدگی‌های اثرات شرطی را پیش می‌آورد، با همان رده‌های راه‌حل.

با سازوکاری برای محاسبهٔ حالت‌های باور دقیق یا تقریبی، می‌توانیم برنامه‌های احتیاطی را با بسط جستجوی پیشروی AND–OR روی حالت‌های باورِ استفاده‌شده در بخش 4.4 تولید کنیم. کنش‌های با اثرات ناجبرگرایانه — که به‌سادگی با استفاده از فصلی در EFFECT طرح کنش تعریف می‌شوند — را می‌توان با تغییرهای جزئی در محاسبهٔ به‌روزرسانی حالت باور و بدون تغییر در الگوریتم جستجو جای داد.[3] برای تابع اکتشافی، بسیاری از روش‌های پیشنهادی برای برنامه‌ریزی بی‌حسگر در حالت تا حدی قابل مشاهده و ناجبرگرایانه نیز کاربردپذیرند.

۱۱.۵.۳ برنامه‌ریزی آنلاین

تماشای ربات جوش‌کار نقطه‌ای در کارخانهٔ خودروسازی را تصور کنید. حرکت‌های سریع و دقیق ربات بارها و بارها تکرار می‌شود، هر بار که خودرویی از خط عبور می‌کند. هرچند از نظر فنی چشمگیر است، ربات احتمالاً اصلاً هوشمند به نظر نمی‌رسد، زیرا حرکت توالی ثابت و ازپیش‌برنامه‌ریزی‌شده‌ای است؛ ربات بدیهی است به هیچ معنای معناداری «نمی‌داند چه می‌کند». حال فرض کنیم دری که بد وصل شده دقیقاً وقتی ربات در آستانهٔ اعمال جوش نقطه‌ای است از خودرو می‌افتد. ربات به‌سرعت عملگر جوشش را با گیره‌ای جایگزین می‌کند، در را برمی‌دارد، از نظر خش بررسی می‌کند، دوباره به خودرو وصلش می‌کند، ایمیلی به سرپرست کارگاه می‌فرستد، به عملگر جوشش برمی‌گردد و کارش را از سر می‌گیرد. ناگهان، رفتار ربات هدفمند می‌نماید نه کلیشه‌ای؛ فرض می‌کنیم نه از برنامهٔ احتیاطی عظیم ازپیش‌محاسبه‌شده، بلکه از فرایند برنامه‌ریزی مجدد آنلاین ناشی می‌شود — یعنی ربات باید بداند می‌خواهد چه کند.

برنامه‌ریزی مجدد پیشاپیش شکلی از «پایش اجرا» (Execution Monitoring) را برای تعیین نیاز به برنامهٔ تازه فرض می‌کند. یکی از این نیازها وقتی پیش می‌آید که عامل برنامه‌ریزی احتیاطی از برنامه‌ریزی برای هر احتیاط کوچک، مانند احتمال افتادن آسمان روی سرش،[4] خسته می‌شود. این بدان معناست برنامهٔ احتیاطی به‌شکل ناقص رها می‌شود. برای نمونه، برخی شاخه‌های برنامهٔ احتیاطی نیمه‌ساخته می‌توانند به‌سادگی بگویند Replan؛ اگر در طول اجرا به چنین شاخه‌ای رسیده شود، عامل به حالت برنامه‌ریزی بازمی‌گردد. چنان‌که پیش‌تر یاد کردیم، تصمیم دربارهٔ اینکه چقدر از مسئله را از پیش حل کنیم و چقدر را به برنامه‌ریزی مجدد واگذاریم، دادوستدی میان رویدادهای ممکن با هزینه‌ها و احتمال‌های وقوع متفاوت است. هیچ‌کس نمی‌خواهد خودرویی وسط صحرای صحرا خراب شود و تازه به فکر آب کافی بیفتد.

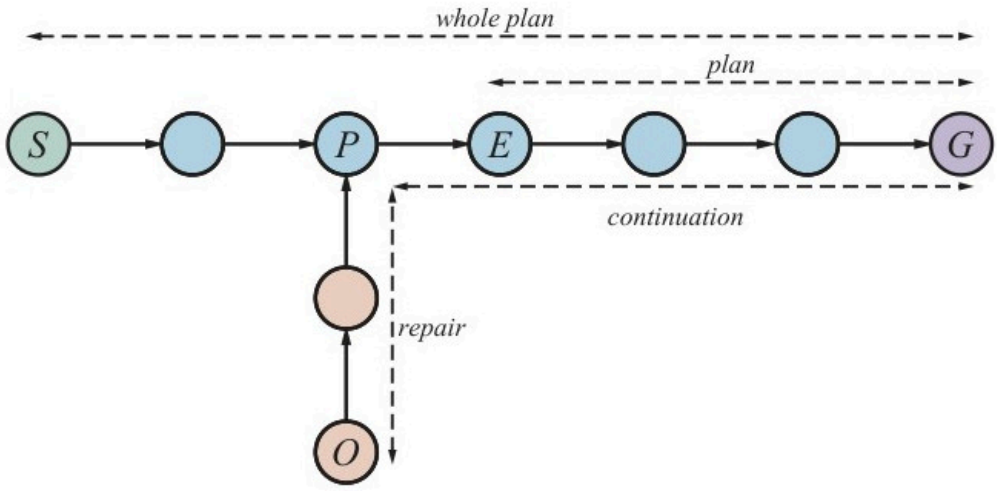
برنامه‌ریزی مجدد ممکن است اگر مدل عامل از جهان نادرست باشد لازم شود. مدل کنش ممکن است «پیش‌شرط گم‌شده» (Missing Precondition) داشته باشد — برای نمونه، عامل ممکن است نداند برداشتن درِ قوطی رنگ اغلب به پیچ‌گوشتی نیاز دارد. مدل ممکن است «اثر گم‌شده» (Missing Effect) داشته باشد — رنگ‌کردن شیء ممکن است رنگ را روی زمین هم بریزد. یا مدل ممکن است «روان گم‌شده» (Missing Fluent) داشته باشد که کلاً از بازنمایی غایب است — برای نمونه، مدل پیش‌ترداده‌شده هیچ تصوری از مقدار رنگ درون قوطی، از چگونگی اثر کنش‌ها بر این مقدار، یا از نیاز به غیرصفربودن مقدار ندارد. مدل همچنین ممکن است تأمینی برای «رویدادهای برون‌زا» (Exogenous

Events) مانند برانداختن قوطی رنگ توسط کسی نداشته باشد. رویدادهای برون‌زا می‌توانند شامل تغییرهای در هدف نیز باشند، مانند افزودن الزامِ سیاهرنگ‌نبودن میز و صندلی. بدون توانایی پایش و برنامه‌ریزی مجدد، رفتار عامل اگر به درستی مطلق مدلش تکیه کند احتمالاً شکننده خواهد بود.

عامل آنلاین در پایش محیط در طول اجرای برنامه دست‌کم سه رویکرد متفاوت برای انتخاب دارد:

- **پایش کنش (Action Monitoring):** پیش از اجرای کنش، عامل تأیید می‌کند همهٔ پیش‌شرط‌ها همچنان برقرارند.
- **پایش برنامه (Plan Monitoring):** پیش از اجرای کنش، عامل تأیید می‌کند برنامهٔ باقی‌مانده همچنان موفق خواهد شد.
- **پایش هدف (Goal Monitoring):** پیش از اجرای کنش، عامل بررسی می‌کند آیا مجموعه‌هدف بهتری هست که می‌توانست برای رسیدن به آن بکوشد.

در شکل ۱۱.۱۲ نمای شماتیک پایش کنش را می‌بینیم. عامل هم برنامهٔ اصلی‌اش، whole plan، و هم بخش اجرانشدهٔ برنامه را، که plan نشان داده می‌شود، پیگیری می‌کند. پس از اجرای چند گام نخست برنامه، عامل انتظار دارد در حالت E باشد. اما عامل مشاهده می‌کند در واقع در حالت O است. سپس باید برنامه را با یافتن نقطه‌ای P روی برنامهٔ اصلی که می‌تواند به آن بازگردد تعمیر کند. (ممکن است P حالت هدف، G، باشد.) عامل می‌کوشد هزینهٔ کل برنامه را کمینه کند: بخش تعمیر (از O تا P) به‌علاوهٔ ادامه (از P تا G).



شکل ۱۱.۱۲ — در آغاز، توالی «whole plan» انتظار می‌رود عامل را از S به G برساند. عامل گام‌های برنامه را اجرا می‌کند تا انتظار دارد در حالت E باشد، اما مشاهده می‌کند در واقع در O است. سپس عامل برای تعمیر کمینه به‌علاوهٔ ادامه برای رسیدن به G برنامه‌ریزی مجدد می‌کند.

حال به مثال مسئلهٔ رسیدن به صندلی و میزی با رنگ هماهنگ برگردیم. فرض کنیم عامل این برنامه را ارائه می‌کند:

```
                                                    ,LookAt(Table), LookAt(Chair)]
                                  if Color(Table, c) ∧ Color(Chair, c) then NoOp
                                                    ,else [RemoveLid(Can1), LookAt(Can1)
                        if Color(Table, c) ∧ Color(Can1, c) then Paint(Chair, Can1)
                                                                    [[else REPLAN
```

حال عامل آمادهٔ اجرای برنامه است. عامل مشاهده می‌کند میز و قوطی رنگ سفیدند و صندلی سیاه است. سپس Paint(Chair,Can1) را اجرا می‌کند. در این نقطه برنامه‌ریز کلاسیک پیروزی اعلام می‌کرد؛ برنامه اجرا شده. اما عامل پایش اجرای آنلاین باید بررسی کند کنش موفق شده.

فرض کنیم عامل ادراک می‌کند صندلی خاکستریِ خال‌خالی است، زیرا رنگ سیاه از زیر نمایان است. عامل سپس باید موقعیت بازیابی در برنامه را برای هدف‌گرفتن و توالی کنشِ تعمیر برای رسیدن به آن‌جا بیابد. عامل متوجه می‌شود حالت کنونی با پیش‌شرط پیش از کنش Paint(Chair,Can1) یکسان است، پس توالی خالی را برای تعمیر برمی‌گزیند و برنامه‌اش را همان توالی [Paint] که تازه امتحان کرده می‌کند. با این برنامهٔ تازه، پایش اجرا از سر گرفته می‌شود و کنش Paint دوباره امتحان می‌شود. این رفتار حلقه می‌زند تا آنکه صندلی کاملاً رنگ‌شده ادراک شود. اما توجه کنید حلقه توسط فرایند برنامه‌ریزی–اجرا–برنامه‌ریزی مجدد ایجاد می‌شود، نه توسط حلقهٔ صریح در برنامه. همچنین توجه کنید برنامهٔ اصلی لازم نیست هر احتیاطی را پوشش دهد. اگر عامل به گامِ علامت‌خورده با REPLAN برسد، می‌تواند برنامهٔ تازه‌ای (شاید شامل Can2) تولید کند.

پایش کنش روش ساده‌ای از پایش اجراست، اما گاهی می‌تواند به رفتار کمتر از هوشمندانه بینجامد. برای نمونه، فرض کنیم رنگ سیاه یا سفیدی نیست و عامل برنامه‌ای برای حل مسئلهٔ رنگ‌کردن با رنگ‌کردن هر دو صندلی و میز قرمز می‌سازد. فرض کنیم فقط به اندازهٔ کافی رنگ قرمز برای صندلی هست. با پایش کنش، عامل می‌رفت و صندلی را قرمز رنگ می‌کرد، سپس متوجه می‌شد رنگ تمام شده و نمی‌تواند میز را رنگ کند، و در آن نقطه برای تعمیر برنامه‌ریزی مجدد می‌کرد — شاید هر دو صندلی و میز را سبز رنگ کند. عامل پایش برنامه می‌تواند شکست را هرگاه حالت کنونی چنان باشد که برنامهٔ باقی‌مانده دیگر کار نکند تشخیص دهد. پس زمان را با رنگ‌کردن صندلی قرمز تلف نمی‌کرد.

پایش برنامه این را با بررسی پیش‌شرط‌های موفقیت کل برنامهٔ باقی‌مانده به دست می‌آورد — یعنی پیش‌شرط‌های هر گام برنامه، به‌جز آن پیش‌شرط‌هایی که توسط گام دیگری در برنامهٔ باقی‌مانده محقق می‌شوند. پایش برنامه اجرای برنامهٔ محکوم را هرچه زودتر قطع می‌کند، به‌جای ادامه تا وقوع واقعی شکست.[5] پایش برنامه همچنین اجازهٔ خوش‌شانسی — موفقیت تصادفی — را می‌دهد. اگر کسی بیاید و هم‌زمان با رنگ‌کردن صندلی توسط عامل، میز را قرمز رنگ کند، آنگاه پیش‌شرط‌های برنامهٔ نهایی برآورده شده‌اند (هدف محقق شده) و عامل می‌تواند زودتر به خانه برود.

اصلاح الگوریتم برنامه‌ریزی به‌گونه‌ای که هر کنش در برنامه با پیش‌شرط‌های کنش حاشیه‌نویسی شود، برای فعال‌کردن پایش کنش سرراست است. فعال‌کردن پایش برنامه کمی پیچیده‌تر است. برنامه‌ریزهای ترتیب‌جزئی این مزیت را دارند که از قبل ساختارهایی شامل روابط لازم برای پایش برنامه ساخته‌اند. تقویت برنامه‌ریزهای فضای حالت با حاشیه‌نویسی‌های لازم را می‌توان با کتابداری دقیق هنگام بازگشت روان‌های هدف از میان برنامه انجام داد.

حال که روشی برای پایش و برنامه‌ریزی مجدد توصیف کرده‌ایم، باید بپرسیم «آیا کار می‌کند؟» این پرسش به‌شکل شگفت‌آوری ظریف است. اگر منظورمان «آیا می‌توانیم تضمین کنیم عامل همیشه به هدف می‌رسد؟» باشد، پاسخ نه است، زیرا عامل می‌تواند سهواً به بن‌بستی برسد که هیچ تعمیری از آن نیست. برای نمونه، عامل جاروبرقی ممکن است مدل ناقصی از خودش داشته باشد و نداند باتری‌هایش می‌توانند تمام شوند. وقتی تمام شوند، نمی‌تواند هیچ برنامه‌ای را تعمیر کند. اگر بن‌بست‌ها را منتفی کنیم — فرض کنیم برنامه‌ای برای رسیدن به هدف از هر حالت در محیط وجود دارد — و فرض کنیم محیط واقعاً ناجبرگرایانه است، به این معنا که چنین برنامه‌ای در هر تلاش اجراییِ داده‌شده همیشه شانسی برای موفقیت دارد، آنگاه عامل سرانجام به هدف می‌رسد.

مشکل وقتی رخ می‌دهد که کنشِ ظاهراً ناجبرگرایانه واقعاً تصادفی نیست، بلکه به پیش‌شرطی بستگی دارد که عامل از آن بی‌خبر است. برای نمونه، گاهی قوطی رنگ ممکن است خالی باشد، پس رنگ‌کردن از آن قوطی اثری ندارد. هیچ مقدار تلاش مجدد این را تغییر نمی‌دهد.[6] یکی از راه‌حل‌ها انتخاب تصادفی از میان مجموعهٔ برنامه‌های تعمیر ممکن است، به‌جای امتحان‌کردن همان یکی هر بار. در این صورت، برنامهٔ تعمیرِ بازکردن قوطی دیگر ممکن است کار کند. رویکرد بهتر یادگیری مدل بهتر است. هر شکست پیش‌بینی فرصتی برای یادگیری است؛ عامل باید بتواند مدلش از جهان را برای هماهنگی با ادراک‌هایش اصلاح کند. از آن پس، برنامه‌ریز مجدد قادر خواهد بود تعمیری ارائه دهد که به ریشهٔ مشکل می‌رسد، به‌جای تکیه بر شانس برای انتخاب تعمیر خوب.

۱۱.۶ زمان، زمان‌بندی‌ها و منابع

برنامه‌ریزی کلاسیک دربارهٔ چه‌کاری و به چه ترتیبی صحبت می‌کند، اما دربارهٔ زمان صحبت نمی‌کند: کنش چقدر طول می‌کشد و کی رخ می‌دهد. برای نمونه، در حوزهٔ فرودگاه می‌توانستیم برنامه‌ای تولید کنیم که می‌گوید کدام هواپیماها به کجا می‌روند، چه چیزی حمل می‌کنند، اما نمی‌توانستیم زمان‌های حرکت و ورود را مشخص کنیم. این موضوع «زمان‌بندی» (Scheduling) است.

جهان واقعی همچنین «قیدهای منبع» (Resource Constraints) تحمیل می‌کند: خط هوایی تعداد محدودی کارکنان دارد و کارکنانی که در یک پروازند نمی‌توانند هم‌زمان در پرواز دیگر باشند. این بخش فنونی برای مسائل برنامه‌ریزی و زمان‌بندی با قیدهای منبع معرفی می‌کند.

رویکردی که می‌گیریم «نخست برنامه‌ریزی، سپس زمان‌بندی» است: تقسیم مسئلهٔ کلی به فاز برنامه‌ریزی که در آن کنش‌ها، با برخی قیدهای ترتیبی، برای برآوردن هدف‌های مسئله انتخاب می‌شوند، و فاز زمان‌بندی بعدی که در آن اطلاعات زمانی به برنامه افزوده می‌شود تا مطمئن شویم قیدهای منبع و مهلت‌ها را برآورده می‌کند. این رویکرد در محیط‌های تولیدی و لجستیکی جهان واقعی رایج است، که در آن فاز برنامه‌ریزی گاهی خودکار و گاهی توسط کارشناسان انسانی انجام می‌شود.

۱۱.۶.۱ بازنمایی قیدهای زمانی و منبع

مسئلهٔ نمونهٔ زمان‌بندی کارگاه (Job-Shop Scheduling Problem؛ بخش 6.1.2 را ببینید) از مجموعه‌ای «کار»ها (Jobs) تشکیل شده، که هر یک مجموعه‌ای کنش با قیدهای ترتیبی میان آن‌ها دارد. هر کنش «مدت» (Duration) و مجموعه‌ای «قیدهای منبع» (Resource Constraints) لازم توسط کنش دارد. قید نوع منبع (مثلاً پیچ، آچار یا خلبان)، شمار آن منبع لازم، و اینکه آیا منبع «مصرف‌شدنی» (Consumable) است (مثلاً پیچ‌ها دیگر برای استفاده در دسترس نیستند) یا «قابل‌استفاده‌مجدد» (Reusable) (مثلاً خلبان در طول پرواز مشغول است اما با پایان پرواز دوباره در دسترس است) را مشخص می‌کند. کنش‌ها همچنین می‌توانند منابع تولید کنند (مثلاً کنش‌های تولید و بازتأمین).

راه‌حل مسئلهٔ زمان‌بندی کارگاه، زمان‌های شروع هر کنش را مشخص می‌کند و باید همهٔ قیدهای ترتیبی زمانی و قیدهای منبع را برآورده کند. مانند مسائل جستجو و برنامه‌ریزی، راه‌حل‌ها را می‌توان بر اساس تابع هزینه ارزیابی کرد؛ این می‌تواند کاملاً پیچیده باشد، با هزینه‌های منبع غیرخطی، هزینه‌های تأخیر وابسته به زمان و الی آخر. برای سادگی، فرض می‌کنیم تابع هزینه فقط کل مدت برنامه است، که «طول برنامه» (Makespan) نامیده می‌شود.

```
                                                ,Jobs({AddEngine1<AddWheels1<Inspect1}
                                                 ({AddEngine2<AddWheels2<Inspect2}
          Resources(EngineHoists(1), WheelStations(1), Inspectors(2), LugNuts(500))
                                Action(AddEngine1, DURATION:30, USE:EngineHoists(1))
                                Action(AddEngine2, DURATION:60, USE:EngineHoists(1))
        Action(AddWheels1, DURATION:30, CONSUME:LugNuts(20), USE:WheelStations(1))
        Action(AddWheels2, DURATION:15, CONSUME:LugNuts(20), USE:WheelStations(1))
                                  Action(Inspecti, DURATION:10, USE:Inspectors(1))
```

شکل ۱۱.۱۳ — مسئلهٔ زمان‌بندی کارگاه برای مونتاژ دو خودرو، با قیدهای منبع. نماد A<B یعنی کنش A باید پیش از کنش B بیاید.

شکل ۱۱.۱۳ مثال ساده‌ای را نشان می‌دهد: مسئله‌ای شامل مونتاژ دو خودرو. مسئله از دو کار تشکیل شده، هر یک به شکل [AddEngine, AddWheels, Inspect]. سپس گزارهٔ Resources اعلام می‌کند چهار نوع منبع هست و شمار هر نوع در آغاز: ۱ جرثقیل موتور، ۱ ایستگاه چرخ، ۲ بازرس و ۵۰۰ مهرهٔ چرخ. طرح‌های کنش مدت و نیازهای منبع هر کنش را می‌دهند. مهره‌های چرخ با افزودن چرخ‌ها به خودرو مصرف می‌شوند، در حالی که دیگر منابع در آغاز کنش «قرض» گرفته می‌شوند و در پایان کنش آزاد می‌شوند.

بازنمایی منابع به‌صورت کمیت‌های عددی، مانند Inspectors(2)، به‌جای موجودیت‌های نام‌دار، مانند Inspector(I1) و Inspector(I2)، مثالی از فنّی به نام «تجمیع» (Aggregation) است: گروه‌بندی اشیای فردی به کمیت‌ها وقتی اشیا همه نامتمایزند. در مسئلهٔ مونتاژ ما، فرقی نمی‌کند کدام بازرس خودرو را بازرسی کند، پس نیازی به ایجاد تمایز نیست. تجمیع برای کاهش پیچیدگی ضروری است. در نظر بگیرید وقتی برنامهٔ زمانی پیشنهادی ۱۰ کنش بازرسی هم‌زمان داشته باشد اما فقط ۹ بازرس در دسترس باشند چه اتفاقی می‌افتد. با بازنمایی بازرسان به‌صورت کمیت‌ها، شکست بلافاصله تشخیص داده می‌شود و الگوریتم برای امتحان برنامهٔ زمانی دیگر عقبگرد می‌کند. با بازنمایی بازرسان به‌صورت افراد، الگوریتم همهٔ 9! راه تخصیص بازرسان به کنش‌ها را امتحان می‌کرد پیش از آنکه متوجه شود هیچ‌کدام کار نمی‌کنند.

۱۱.۶.۲ حل مسائل زمان‌بندی

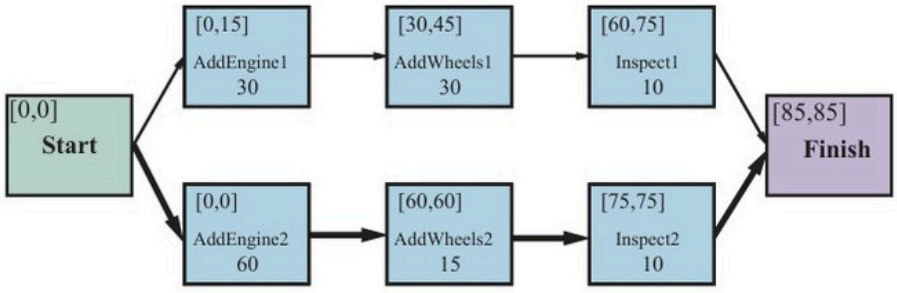
با در نظر گرفتن فقط مسئلهٔ زمان‌بندی زمانی آغاز می‌کنیم، با نادیده‌گرفتن قیدهای منبع. برای کمینه‌کردن طول برنامه (مدت برنامه)، باید نخستین زمان‌های شروع ممکن را برای همهٔ کنش‌ها، سازگار با قیدهای ترتیبیِ عرضه‌شده با مسئله، بیابیم. مفید است این قیدهای ترتیبی را گراف جهت‌داری ببینیم که کنش‌ها را مرتبط می‌کند، چنان‌که در شکل ۱۱.۱۴. می‌توانیم «روش مسیر بحرانی» (Critical Path Method، یا CPM) را بر این گراف اعمال کنیم تا زمان‌های شروع و پایان ممکن هر کنش را تعیین کنیم. مسیر در گرافِ بازنمایندهٔ برنامهٔ ترتیب‌جزئی، توالی مرتب خطی از کنش‌هاست که با Start آغاز و با Finish پایان می‌یابد. (برای نمونه، در برنامهٔ ترتیب‌جزئی شکل ۱۱.۱۴ دو مسیر هست.)

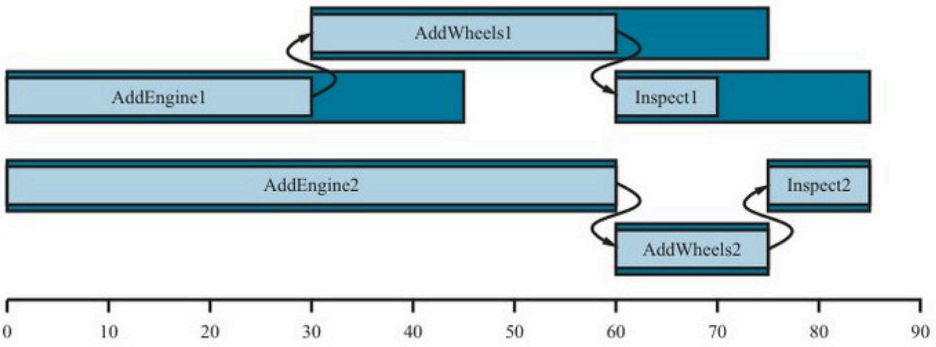
«مسیر بحرانی» (Critical Path) مسیری است که کل مدت‌اش بلندترین است؛ مسیر «بحرانی» است، زیرا مدت کل برنامه را تعیین می‌کند — کوتاه‌کردن مسیرهای دیگر برنامه را به‌عنوان کل کوتاه نمی‌کند، اما تأخیر در شروع هر کنشی روی مسیر بحرانی کل برنامه را کند می‌کند. کنش‌های خارج از مسیر بحرانی «پنجره»ی زمانی دارند که در آن می‌توانند اجرا شوند. پنجره بر حسب نخستین زمان شروع ممکن، ES، و آخرین زمان شروع ممکن، LS، مشخص می‌شود. کمیت LS – ES «اسلک» (Slack) کنش شناخته می‌شود. در شکل ۱۱.۱۴ می‌بینیم کل برنامه ۸۵ دقیقه طول خواهد کشید، هر کنش در کار بالایی ۱۵ دقیقه اسلک دارد و هر کنش روی مسیر بحرانی (طبق تعریف) اسلک ندارد. زمان‌های ES و LS برای همهٔ کنش‌ها با هم «برنامهٔ زمانی» (Schedule) مسئله را تشکیل می‌دهند.

فرمول‌های زیر ES و LS را تعریف و الگوریتم برنامه‌نویسی پویا برای محاسبهٔ آن‌ها تشکیل می‌دهند. A و B کنش‌اند و A<B یعنی A پیش از B می‌آید:

```
                                ES(Start) = 0
ES(B) = max_{A<B} ES(A) + Duration(A)
                             LS(Finish) = ES(Finish)
LS(A) = min_{B>A} LS(B) – Duration(A)
```

ایده آن است که با تخصیص ES(Start) = 0 آغاز می‌کنیم. سپس، به‌محض گرفتن کنش B که همهٔ کنش‌های بلافاصله پیش از B مقدارهای ES تخصیص‌یافته دارند، ES(B) را بیشینهٔ نخستین زمان‌های پایان آن کنش‌های بلافاصله پیشین تنظیم می‌کنیم، که نخستین زمان پایان کنش به‌صورت نخستین زمان شروع به‌علاوهٔ مدت تعریف می‌شود. این فرایند تکرار می‌شود تا به هر کنش مقدار ES تخصیص یابد. مقدارهای LS به‌روش مشابه، با کار به عقب از کنش Finish، محاسبه می‌شوند.





شکل ۱۱.۱۴ – بالا: بازنمایی قیدهای زمانی برای مسئلهٔ زمان‌بندی کارگاه شکل ۱۱.۱۳. مدت هر کنش پایین هر مستطیل داده شده. در حل مسئله، نخستین و آخرین زمان‌های شروع را به‌صورت جفت [ES,LS] محاسبه می‌کنیم که بالا-چپ نمایش داده شده. تفاضل این دو عدد اسلک کنش است؛ کنش‌های با اسلک صفر روی مسیر بحرانی‌اند، که با پیکان‌های ضخیم نشان داده شده. پایین: همان راه‌حل به‌صورت خط زمانی. مستطیل‌های خاکستری بازه‌های زمانی‌ای را نشان می‌دهند که کنش می‌تواند در آن‌ها اجرا شود، به شرط رعایت قیدهای ترتیبی. بخش اشغال‌نشدهٔ مستطیل خاکستری اسلک را نشان می‌دهد.

پیچیدگی الگوریتم مسیر بحرانی فقط O(Nb) است، که N شمار کنش‌ها و b بیشینهٔ ضریب انشعاب به درون یا بیرون کنش است. (برای دیدن این، توجه کنید محاسبات LS و ES یک بار برای هر کنش انجام می‌شود و هر محاسبه روی حداکثر b کنش دیگر تکرار می‌شود.) پس یافتن برنامهٔ زمانی کمینه‌مدت، با ترتیب جزئی داده‌شده روی کنش‌ها و بدون قیدهای منبع، کاملاً آسان است.

از نظر ریاضی، مسائل مسیر بحرانی آسان‌اند، زیرا به‌صورت عطفی از نابرابری‌های خطی روی زمان‌های شروع و پایان تعریف می‌شوند. وقتی قیدهای منبع را معرفی می‌کنیم، قیدهای حاصل روی زمان‌های شروع و پایان پیچیده‌تر می‌شوند. برای نمونه، کنش‌های AddEngine، که در شکل ۱۱.۱۴ در همان زمان آغاز می‌شوند، به همان EngineHoist نیاز دارند و بنابراین نمی‌توانند هم‌پوشانی کنند. قید «نمی‌توانند هم‌پوشانی کنند» فصل‌ای از دو نابرابری خطی است، یکی برای هر ترتیب ممکن. معرفی فصل‌ها معلوم می‌شود زمان‌بندی با قیدهای منبع را NP-سخت می‌کند.

شکل ۱۱.۱۵ راه‌حل با سریع‌ترین زمان تکمیل، ۱۱۵ دقیقه، را نشان می‌دهد. این ۳۰ دقیقه بیشتر از ۸۵ دقیقهٔ لازم برای برنامهٔ زمانی بدون قیدهای منبع است. توجه کنید هیچ زمانی نیست که هر دو بازرس لازم باشند، پس می‌توانیم بلافاصله یکی از دو بازرس‌مان را به موقعیت پربارتری منتقل کنیم.

تاریخچهٔ بلندی از کار روی زمان‌بندی بهینه هست. مسئلهٔ چالشیِ طرح‌شده در ۱۹۶۳ — یافتن برنامهٔ زمانی بهینه برای مسئله‌ای شامل فقط ۱۰ ماشین و ۱۰ کارِ هر یک با ۱۰۰ کنش — به مدت ۲۳ سال حل‌نشده ماند (لاولر و همکاران، ۱۹۹۳). رویکردهای بسیاری امتحان شده، از جمله شاخه‌وکران، بازپخت شبیه‌سازی‌شده، جستجوی تابو و ارضای قید. یکی از رویکردهای پرطرفدار «اکتشافی کمترین-اسلک» (Minimum Slack Heuristic) است: در هر تکرار، برای نخستین شروع ممکن، هر کنش زمان‌بندی‌نشده‌ای را زمان‌بندی کن که همهٔ پیشینیانش زمان‌بندی شده‌اند و کمترین اسلک را دارد؛ سپس زمان‌های ES و LS را برای هر کنش متأثر به‌روز کن و تکرار کن. این اکتشافی حریصانه شبیه اکتشافی کمترین-مقادیر-باقی‌مانده (MRV) در ارضای قید است. اغلب در عمل خوب کار می‌کند، اما برای مسئلهٔ مونتاژ ما راه‌حل ۱۳۰-دقیقه‌ای می‌دهد، نه راه‌حل ۱۱۵-دقیقه‌ای شکل ۱۱.۱۵.

تا این نقطه، فرض کرده‌ایم مجموعهٔ کنش‌ها و قیدهای ترتیبی ثابت است. تحت این فرض‌ها، هر مسئلهٔ زمان‌بندی را می‌توان با توالی ناهم‌پوشِ اجتناب‌کننده از همهٔ تعارض‌های منبع حل کرد، به شرط آنکه هر کنش به خودی خود شدنی باشد. اما اگر مسئلهٔ زمان‌بندی بسیار دشوار ثابت شود، ممکن است ایدهٔ خوبی نباشد این‌گونه حلش کنیم — ممکن است بهتر باشد کنش‌ها و قیدها را دوباره در نظر بگیریم، در صورتی که به مسئلهٔ زمان‌بندی بسیار آسان‌تری بینجامد. پس ادغام برنامه‌ریزی و زمان‌بندی با در نظر گرفتن مدت‌ها و هم‌پوشانی‌ها در طول ساخت برنامه معنا می‌کند. چند الگوریتم برنامه‌ریزی در بخش ۱۱.۲ را می‌توان برای مدیریت این اطلاعات تقویت کرد.

۱۱.۷ تحلیل رویکردهای برنامه‌ریزی

برنامه‌ریزی دو حوزهٔ بزرگ هوش مصنوعی را که تاکنون پوشش داده‌ایم ترکیب می‌کند: جستجو و منطق. برنامه‌ریز را می‌توان یا برنامه‌ای دید که برای راه‌حل جستجو می‌کند یا برنامه‌ای که (به‌صورت سازنده) وجود راه‌حل را اثبات می‌کند. بارورسازی متقابل ایده‌ها از دو حوزه به برنامه‌ریزها اجازه داده از مسائل اسباب‌بازی که شمار کنش‌ها و حالت‌ها به حدود یک دوجین محدود بود، به کاربردهای صنعتی جهان واقعی با میلیون‌ها حالت و هزاران کنش مقیاس شوند.

برنامه‌ریزی در درجهٔ نخست تمرینی در کنترل انفجار ترکیبیاتی است. اگر در حوزه‌ای n گزاره باشد، 2^n حالت هست. در برابر چنین بدبینی‌ای، شناسایی زیرمسئله‌های مستقل می‌تواند سلاح قدرتمندی باشد. در بهترین حالت — تجزیه‌پذیری کامل مسئله — شتاب نمایی می‌گیریم. تجزیه‌پذیری اما توسط تعامل‌های منفی میان کنش‌ها نابود می‌شود. SATPLAN می‌تواند روابط منطقی میان زیرمسئله‌ها را رمزگذاری کند. جستجوی پیشرو به‌صورت اکتشافی با تلاش برای یافتن الگوهایی (زیرمجموعه‌هایی از گزاره‌ها) که زیرمسئله‌های مستقل را می‌پوشانند به مسئله می‌پردازد. چون این رویکرد اکتشافی است، حتی وقتی زیرمسئله‌ها کاملاً مستقل نیستند هم می‌تواند کار کند.

متأسفانه، هنوز فهم روشنی نداریم کدام فنون روی کدام نوع مسائل بهترین کار را می‌کنند. کاملاً محتمل است فنون تازه‌ای پدیدار شوند، شاید ترکیبی از بازنمایی‌های بسیار بیانیِ مرتبهٔ اول و سلسله‌مراتبی با بازنمایی‌های عاملی و گزاره‌ایِ بسیار کارآمدِ حاکم بر امروز فراهم کنند. نمونه‌هایی از «سامانه‌های برنامه‌ریزی سبدی» (Portfolio Planning Systems) می‌بینیم، که در آن‌ها مجموعه‌ای از الگوریتم‌ها برای اعمال بر هر مسئلهٔ داده‌شده در دسترس است. این را می‌توان انتخابی (سامانه هر مسئلهٔ تازه را برای انتخاب بهترین الگوریتم طبقه‌بندی می‌کند)، موازی (همهٔ الگوریتم‌ها هم‌زمان، هر یک روی CPU متفاوت، اجرا می‌شوند)، یا با در هم آمیختن الگوریتم‌ها طبق برنامهٔ زمانی انجام داد.

خلاصهٔ فصل

در این فصل، بازنمایی PDDL را برای مسائل برنامه‌ریزی کلاسیک و بسط‌یافته توصیف کردیم و چند رویکرد الگوریتمی برای یافتن راه‌حل ارائه دادیم. نکته‌های برای یادآوری:

- سامانه‌های برنامه‌ریزی الگوریتم‌های حل مسئله‌ای هستند که روی بازنمایی‌های عاملی صریح حالت‌ها و کنش‌ها عمل می‌کنند. این بازنمایی‌ها اشتقاق اکتشافی‌های مؤثر مستقل از حوزه و توسعهٔ الگوریتم‌های قدرتمند و منعطف برای حل مسائل را ممکن می‌کنند.
- PDDL، زبان تعریف حوزهٔ برنامه‌ریزی، حالت‌های آغازین و هدف را به‌صورت عطف‌هایی از لیترال‌ها و کنش‌ها را بر حسب پیش‌شرط‌ها و اثراتشان توصیف می‌کند. بسط‌ها زمان، منابع، ادراک‌ها، برنامه‌های احتیاطی و برنامه‌های سلسله‌مراتبی را بازنمایی می‌کنند.
- جستجوی فضای حالت می‌تواند در جهت پیشرو (پیشروی) یا جهت پسرو (بازگشت) عمل کند. اکتشافی‌های مؤثر را می‌توان با فرض‌های استقلال زیرهدف و با ساده‌سازی‌های گوناگون مسئلهٔ برنامه‌ریزی مشتق کرد.
- رویکردهای دیگر شامل رمزگذاری مسئلهٔ برنامه‌ریزی به‌صورت مسئلهٔ ارضاپذیری بولی یا مسئلهٔ ارضای قید؛ و جستجوی صریح در فضای برنامه‌های ترتیب‌جزئی است.
- برنامه‌ریزی شبکه‌های تکلیف سلسله‌مراتبی (HTN) به عامل اجازه می‌دهد از طراح حوزه در قالب کنش‌های سطح‌بالا (HLA) که به روش‌های گوناگون با توالی‌های کنش سطح‌پایین قابل پیاده‌سازی‌اند مشورت بگیرد. اثرات HLAها را می‌توان با معناشناسی فرشته‌ای تعریف کرد و اشتقاق برنامه‌های سطح‌بالای اثباتاً درست را بدون در نظر گرفتن پیاده‌سازی‌های سطح‌پایین ممکن کرد. روش‌های HTN می‌توانند برنامه‌های بسیار بزرگِ لازم برای بسیاری از کاربردهای جهان واقعی را ایجاد کنند.
- برنامه‌های احتیاطی به عامل اجازه می‌دهند در طول اجرا جهان را حس کند تا تصمیم بگیرد کدام شاخه از برنامه را دنبال کند. در برخی موارد، برنامه‌ریزی بی‌حسگر یا سازگار را می‌توان برای ساختن برنامه‌ای که بدون نیاز به ادراک کار می‌کند استفاده کرد. هم برنامه‌های سازگار و هم احتیاطی را می‌توان با جستجو در فضای حالت‌های باور ساخت. بازنمایی یا محاسبهٔ کارآمد حالت‌های باور مسئلهٔ کلیدی است.
- عامل برنامه‌ریزی آنلاین از پایش اجرا استفاده می‌کند و در صورت نیاز تعمیرهایی در برنامه می‌زند تا از موقعیت‌های غیرمنتظره بازیابی کند، که می‌توانند به‌سبب کنش‌های ناجبرگرایانه، رویدادهای برون‌زا یا مدل‌های نادرست از محیط باشند.
- بسیاری از کنش‌ها منابعی مصرف می‌کنند، مانند پول، بنزین یا مواد خام. راحت است این منابع را به‌صورت اندازه‌های عددی در استخری در نظر بگیریم، به‌جای تلاش برای استدلال دربارهٔ، مثلاً، هر سکه و اسکناس فردی در جهان. زمان یکی از مهم‌ترین منابع است. می‌توان آن را با الگوریتم‌های زمان‌بندی تخصصی مدیریت کرد، یا زمان‌بندی را با برنامه‌ریزی ادغام کرد.
- این فصل برنامه‌ریزی کلاسیک را برای پوشش محیط‌های ناجبرگرایانه (که در آن‌ها پیامدهای کنش‌ها نامطمئن‌اند) بسط می‌دهد، اما حرف آخر دربارهٔ برنامه‌ریزی نیست. فصل ۱۷ فنونی برای محیط‌های تصادفی (که در آن‌ها با پیامدهای کنش‌ها احتمال‌هایی مرتبط است) توصیف می‌کند: فرایندهای تصمیم

مارکوف، فرایندهای تصمیم مارکوف تا حدی قابل مشاهده و نظریهٔ بازی. در فصل ۲۲ نشان می‌دهیم یادگیری تقویتی به عامل اجازه می‌دهد از موفقیت‌ها و شکست‌های گذشته رفتارکردن را بیاموزد.

یادداشت‌های کتاب‌شناختی و تاریخی

برنامه‌ریزی هوش مصنوعی از بررسی‌های جستجوی فضای حالت، اثبات قضیه و نظریهٔ کنترل برخاست. STRIPS (فایکز و نیلسون، ۱۹۷۱، ۱۹۹۳)، نخستین سامانهٔ برنامه‌ریزی عمده، به‌عنوان برنامه‌ریز ربات شیکی در SRI طراحی شد. نسخهٔ نخست برنامه روی رایانه‌ای با فقط ۱۹۲ کیلوبایت حافظه اجرا شد. ساختار کنترل کلی‌اش بر GPS، حل‌کنندهٔ عمومی مسئله (نیوول و سایمون، ۱۹۶۱)، سامانهٔ جستجوی فضای حالتی که از تحلیل وسیله–هدف استفاده می‌کرد، مدل شده بود.

زبان بازنمایی STRIPS به زبان توصیف کنش، یا ADL (پدنو، ۱۹۸۶)، و سپس زبان توصیف حوزهٔ مسئله، یا PDDL (غَلّاب و همکاران، ۱۹۹۸)، که از ۱۹۹۸ برای رقابت بین‌المللی برنامه‌ریزی استفاده شده، تکامل یافت. تازه‌ترین نسخه PDDL 3.1 است (کواچ، ۲۰۱۱).

برنامه‌ریزهای اوایل دههٔ ۱۹۷۰ مسائل را با محاسبهٔ زیربرنامه‌ای برای هر زیرهدف و سپس رشته‌کردن زیربرنامه‌ها به ترتیبی تجزیه می‌کردند. این رویکرد که سکرودوتی (۱۹۷۵) برنامه‌ریزی خطی (Linear Planning) نامید، به‌زودی ناقص کشف شد. نمی‌تواند برخی مسائل بسیار ساده را حل کند، مانند ناهنجاری ساسمن (تمرین SUSS.11 را ببینید)، که توسط آلن براون هنگام آزمایش با سامانهٔ HACKER یافت شد (ساسمن، ۱۹۷۵). برنامه‌ریز کامل باید اجازهٔ درهم‌آمیختن کنش‌های زیربرنامه‌های متفاوت را در توالی تکی بدهد. سامانهٔ WARPLAN وارن (۱۹۷۴) به آن دست یافت و نشان داد زبان برنامه‌نویسی منطقی پرولوگ چگونه می‌تواند برنامه‌های موجز تولید کند؛ WARPLAN فقط ۱۰۰ خط کد است.

برنامه‌ریزی ترتیب‌جزئی ۲۰ سال بعدی پژوهش را مسلط بود، با کار نظری توصیف‌کنندهٔ تشخیص تعارض‌ها (تیت، ۱۹۷۵) و محافظت از شرایط محقق‌شده (ساسمن، ۱۹۷۵)، و پیاده‌سازی‌هایی شامل NOAH (سکرودوتی، ۱۹۷۷) و NONLIN (تیت، ۱۹۷۷). آن به مدل‌های صوری (چپمن، ۱۹۸۷؛ مک‌آلیستر و روزن‌بلیت، ۱۹۹۱) انجامید که تحلیل نظری الگوریتم‌ها و مسائل برنامه‌ریزی گوناگون را اجازه می‌داد، و به سامانهٔ گسترده‌توزیع‌شدهٔ UCPOP (پمبرتی و ولد، ۱۹۹۲).

درو مک‌درموت گمان کرد تأکید بر برنامه‌ریزی ترتیب‌جزئی فنون دیگری را که شاید اکنون که رایانه‌ها ۱۰۰ برابر حافظهٔ روزگار شیکی داشتند باید دوباره در نظر گرفته می‌شدند کنار می‌زند. UNPOP او (مک‌درموت، ۱۹۹۶) برنامهٔ جستجوی فضای حالتی بود که از اکتشافی نادیده‌گرفتن-فهرست-حذف استفاده می‌کرد. HSP، برنامه‌ریز جستجوی اکتشافی (بونت و گفنر، ۱۹۹۹؛ هاسلوم، ۲۰۰۶)، جستجوی فضای حالت را برای مسائل برنامه‌ریزی بزرگ عملی کرد. برنامه‌ریز FF یا Fast Forward (هافمن، ۲۰۰۱؛ هافمن و نبل، ۲۰۰۱؛ هافمن، ۲۰۰۵) و تغییر FASTDOWNWARD (هلمرت، ۲۰۰۶) در دههٔ ۲۰۰۰ رقابت‌های برنامه‌ریزی بین‌المللی را بردند.

جستجوی دوسویه (بخش 3.4.5 را ببینید) نیز از کمبود اکتشافی‌ها رنج می‌برد، اما موفقیتی با استفاده از جستجوی پسرو برای ایجاد محیطی دور هدف و سپس پالایش اکتشافی برای جستجوی پیشرو به سوی آن محیط به دست آمده (تورالبا و همکاران، ۲۰۱۶). برنامه‌ریز جستجوی دوسویهٔ SYMBA* (تورالبا و همکاران، ۲۰۱۶) رقابت ۲۰۱۶ را برد.

پژوهشگران به PDDL و الگوی برنامه‌ریزی روی آوردند تا بتوانند از اکتشافی‌های مستقل از حوزه استفاده کنند. هافمن (۲۰۰۵) فضای جستجوی اکتشافی نادیده‌گرفتن-فهرست-حذف را تحلیل می‌کند. ادلکمپ (۲۰۰۹) و هاسلوم و همکاران (۲۰۰۷) توصیف می‌کنند چگونه پایگاه‌داده‌های الگو را برای اکتشافی‌های برنامه‌ریزی بسازند. فلنر و همکاران (۲۰۰۴) نتایج دلگرم‌کننده با پایگاه‌داده‌های الگو برای معماهای کاشی‌لغزان نشان می‌دهند، که می‌توان آن را حوزهٔ برنامه‌ریزی دانست، اما هافمن و همکاران (۲۰۰۶) برخی محدودیت‌های انتزاع را برای مسائل برنامه‌ریزی کلاسیک نشان می‌دهند. (رینتانن، ۲۰۱۲) اکتشافی‌های انتخاب متغیر ویژهٔ برنامه‌ریزی برای حل SAT را بحث می‌کند. هلمرت و همکاران (۲۰۱۱) سامانهٔ Fast Downward Stone Soup (FDSS) را توصیف می‌کنند، برنامه‌ریز سبدی که، مانند افسانهٔ سوپ سنگی، ما را دعوت می‌کند تا حد ممکن الگوریتم برنامه‌ریزی در آن بریزیم. سامانه مجموعه‌ای از مسائل آموزشی نگه می‌دارد و برای هر مسئله و هر الگوریتم زمان اجرا و هزینهٔ برنامهٔ حاصل را ثبت می‌کند. سپس هنگام رویارویی با مسئلهٔ تازه، از تجربهٔ گذشته برای تصمیم‌گیری اینکه کدام الگوریتم(ها) را با چه محدودیت‌های زمانی امتحان کند استفاده می‌کند و راه‌حل با کمترین هزینه را می‌گیرد. FDSS برندهٔ رقابت بین‌المللی برنامه‌ریزی ۲۰۱۸ بود (سیپ و رگر، ۲۰۱۸). سیپ و همکاران (۲۰۱۵) رویکرد یادگیری ماشینی برای یادگیری خودکار سبد خوب با مسئلهٔ تازه توصیف می‌کنند. والاتی و همکاران (۲۰۱۵) مروری بر برنامه‌ریزی سبدی می‌دهند. ایدهٔ سبدهای الگوریتم برای مسائل جستجوی ترکیبیاتی به گومز و سلمن (۲۰۰۱) بازمی‌گردد.

سیستلا و گودفروید (۲۰۰۴) کاهش تقارن را پوشش می‌دهند و گودفروید (۱۹۹۰) اکتشافی‌های ترتیب جزئی را. ریشتر و هلمرت (۲۰۰۹) کارایی‌های هرس رو به جلو با کنش‌های ترجیحی را نشان می‌دهند.

بلوم و فورست (۱۹۹۷) حوزهٔ برنامه‌ریزی را با سامانهٔ Graphplan خود احیا کردند، که مرتبه‌ای بزرگی از برنامه‌ریزهای ترتیب‌جزئی آن زمان سریع‌تر بود. برایس و کامبامپاتی (۲۰۰۷) مروری بر گراف‌های برنامه‌ریزی می‌دهند. استفاده از حساب وضعیت برای برنامه‌ریزی توسط جان مک‌کارتی (۱۹۶۳) معرفی و توسط ری رایتر (۲۰۰۱) پالایش شد.

کاوتز و همکاران (۱۹۹۶) راه‌های گوناگون گزاره‌سازی طرح‌های کنش را بررسی کردند و یافتند فشرده‌ترین شکل‌ها لزوماً به سریع‌ترین زمان‌های راه‌حل نمی‌انجامند. تحلیل نظام‌مندی توسط ارنست و همکاران (۱۹۹۷) انجام شد، که همچنین «کامپایلر» خودکاری برای تولید بازنمایی‌های گزاره‌ای از مسائل PDDL توسعه دادند. برنامه‌ریز BLACKBOX، که ایده‌های Graphplan و SATPLAN را ترکیب می‌کند، توسط کاوتز و سلمن (۱۹۹۸) توسعه یافت. برنامه‌ریزهای مبتنی بر ارضای قید شامل CPLAN فان بیک و چن (۱۹۹۹) و GP-CSP (دو و کامبامپاتی، ۲۰۰۳) است.

همچنین علاقه به بازنمایی برنامه به‌صورت «نمودار تصمیم دودویی» (Binary Decision Diagram، یا BDD)، ساختار دادهٔ فشرده برای عبارت‌های بولی که به‌طور گسترده در جامعهٔ تأیید سخت‌افزار مطالعه شده، بوده (کلارک و گرامبرگ، ۱۹۸۷؛ مک‌میلان، ۱۹۹۳). فنونی برای اثبات ویژگی‌های نمودارهای تصمیم دودویی، از جمله ویژگیِ راه‌حل‌بودن برای مسئلهٔ برنامه‌ریزی، هست. چیماتی و همکاران (۱۹۹۸) برنامه‌ریزی مبتنی بر این رویکرد ارائه می‌کنند. بازنمایی‌های دیگر نیز استفاده شده‌اند، مانند برنامه‌ریزی عدد صحیح (ووسن و همکاران، ۲۰۰۱).

مقایسه‌های جالبی از رویکردهای گوناگون برنامه‌ریزی هست. هلمرت (۲۰۰۱) چند رده از مسائل برنامه‌ریزی را تحلیل می‌کند و نشان می‌دهد رویکردهای مبتنی بر قید مانند Graphplan و SATPLAN برای حوزه‌های NP-سخت بهترین‌اند، در حالی که رویکردهای مبتنی بر جستجو در حوزه‌هایی که راه‌حل‌های شدنی بدون عقبگرد یافت می‌شوند بهتر عمل می‌کنند. Graphplan و SATPLAN در حوزه‌های با اشیای فراوان مشکل دارند، زیرا آن بدان معناست باید کنش‌های فراوانی بسازند. در برخی موارد می‌توان مسئله را با تولید پویای کنش‌های گزاره‌شده، فقط در صورت نیاز، به‌جای مصداق‌کردن همهٔ آن‌ها پیش از آغاز جستجو، به تأخیر انداخت یا اجتناب کرد.

نخستین سازوکار برنامه‌ریزی سلسله‌مراتبی تسهیلاتی در برنامهٔ STRIPS برای یادگیری «ماکروپ»ها — «عملگرهای کلان» شامل توالی‌ای از گام‌های اولیه (فایکز و همکاران، ۱۹۷۲) — بود. سامانهٔ ABSTRIPS (سکرودوتی، ۱۹۷۴) ایدهٔ «سلسله‌مراتب انتزاع» را معرفی کرد، که بر اساس آن برنامه‌ریزی در سطح‌های بالاتر مجاز بود پیش‌شرط‌های سطح‌پایین‌تر کنش‌ها را نادیده بگیرد تا ساختار کلی برنامهٔ کارکننده مشتق شود. پایان‌نامهٔ دکترای آستین تیت (۱۹۷۵) و کار ارل سکرودوتی (۱۹۷۷) ایده‌های پایهٔ برنامه‌ریزی HTN را توسعه دادند. اِرول، هندلر و ناو (۱۹۹۴، ۱۹۹۶) برنامه‌ریز تجزیهٔ سلسله‌مراتبی کاملی و نیز گستره‌ای از نتایج پیچیدگی برای برنامه‌ریزهای HTN ناب ارائه می‌کنند. ارائهٔ ما از HLAها و معناشناسی فرشته‌ای از مارتی و همکاران (۲۰۰۷، ۲۰۰۸) است.

یکی از هدف‌های برنامه‌ریزی سلسله‌مراتبی استفادهٔ مجدد از تجربهٔ برنامه‌ریزی قبلی در قالب برنامه‌های تعمیم‌یافته بوده. فن یادگیری مبتنی بر تبیین به‌عنوان وسیله‌ای برای تعمیم برنامه‌های قبلاً محاسبه‌شده در سامانه‌هایی مانند SOAR (لیرد و همکاران، ۱۹۸۶) و PRODIGY (کاربونل و همکاران، ۱۹۸۹) استفاده شده. رویکرد جایگزین ذخیرهٔ برنامه‌های قبلاً محاسبه‌شده در شکل اصلی‌شان و سپس استفادهٔ مجدد از آن‌ها برای حل مسائل تازه و مشابه با قیاس با مسئلهٔ اصلی است. این رویکردی است که حوزهٔ «برنامه‌ریزی مبتنی بر مورد» (Case-Based Planning) می‌گیرد (کاربونل، ۱۹۸۳؛ آلترمن، ۱۹۸۸). کامبامپاتی (۱۹۹۴) استدلال می‌کند برنامه‌ریزی مبتنی بر مورد را باید شکلی از برنامه‌ریزی پالایش تحلیل کرد و بنیاد صوری برای برنامه‌ریزی ترتیب‌جزئی مبتنی بر مورد فراهم می‌کند.

برنامه‌ریزهای اولیه شرطی‌ها و حلقه‌ها نداشتند، اما برخی می‌توانستند با وادارسازی (Coercion) برنامه‌های سازگار تشکیل دهند. NOAH سکرودوتی مسئلهٔ «کلیدها و جعبه‌ها» (که در آن برنامه‌ریز دربارهٔ حالت آغازین کم می‌داند) را با وادارسازی حل کرد. میسون (۱۹۹۳) استدلال کرد حس‌کردن را اغلب می‌توان در برنامه‌ریزی رباتیک کنار گذاشت و برنامهٔ بی‌حسگری توصیف کرد که می‌تواند ابزاری را با توالی‌ای از کنش‌های کج‌کردن، صرف‌نظر از موقعیت آغازین، به موقعیت خاصی روی میز ببرد.

گلدمن و بادی (۱۹۹۶) اصطلاح برنامه‌ریزی سازگار (Conformant Planning) را معرفی کردند و یادآور شدند برنامه‌های بی‌حسگر اغلب حتی اگر عامل حسگر داشته باشد مؤثرند. نخستین برنامه‌ریز سازگار نسبتاً کارآمد، Conformant Graphplan (CGP) اسمیت و ولد (۱۹۹۸) بود. فراریس و جیونکیلیا (۲۰۰۰) و رینتانن (۱۹۹۹) مستقل برنامه‌ریزهای سازگار مبتنی بر SATPLAN توسعه دادند. بونت و گفنر (۲۰۰۰) برنامه‌ریز سازگار مبتنی بر جستجوی اکتشافی در فضای حالت‌های باور توصیف می‌کنند، با بهره از ایده‌هایی که نخستین بار در دههٔ ۱۹۶۰ برای فرایندهای تصمیم مارکوف تا حدی قابل مشاهده، یا POMDPها، توسعه یافت (فصل ۱۷ را ببینید).

در حال حاضر، سه رویکرد اصلی به برنامه‌ریزی سازگار هست. دو رویکرد نخست از جستجوی اکتشافی در فضای حالت باور استفاده می‌کنند: HSCP (برتولی و همکاران، ۲۰۰۱) از نمودارهای تصمیم دودویی (BDD) برای بازنمایی حالت‌های باور استفاده می‌کند، در حالی که هافمن و برافمن (۲۰۰۶) رویکرد تنبلِ محاسبهٔ آزمون‌های پیش‌شرط و هدف بر اساس تقاضا با حل‌کنندهٔ SAT اتخاذ می‌کنند. رویکرد سوم، که عمدتاً توسط جوسی رینتانن (۲۰۰۷) حمایت می‌شود، کل مسئلهٔ برنامه‌ریزی بی‌حسگر را به‌صورت «فرمول بولی سوردار» (QBF) صورت‌بندی و با حل‌کنندهٔ QBF عمومی‌منظوره حل می‌کند. برنامه‌ریزهای سازگار کنونی پنج مرتبهٔ بزرگی از CGP سریع‌ترند. برندهٔ مسیر برنامه‌ریزی سازگار رقابت بین‌المللی برنامه‌ریزی ۲۰۰۶ T0 بود (پالاسیوس و گفنر، ۲۰۰۷)، که از جستجوی اکتشافی در فضای حالت باور استفاده می‌کند و بازنمایی حالت باور را با تعریف لیترال‌های مشتق‌شده‌ای که اثرات شرطی را پوشش می‌دهند ساده نگه می‌دارد. برایس و کامباماپاتی (۲۰۰۷) بحث می‌کنند چگونه گراف برنامه‌ریزی را می‌توان برای تولید اکتشافی‌های خوب برای برنامه‌ریزی سازگار و احتیاطی تعمیم داد.

رویکرد برنامه‌ریزی احتیاطیِ توصیف‌شده در فصل بر هافمن و برافمن (۲۰۰۵) مبتنی است و تحت تأثیر الگوریتم‌های جستجوی کارآمد برای گراف‌های AND–OR چرخه‌ای توسعه‌یافته توسط خیمنز و توراس (۲۰۰۰) و هانسن و زیلبرشتاین (۲۰۰۱) بود. مسئلهٔ برنامه‌ریزی احتیاطی پس از انتشار مقالهٔ تأثیرگذار درو مک‌درموت (۱۹۷۸)، «برنامه‌ریزی و کنش»، توجه بیشتری گرفت. برتولی و همکاران (۲۰۰۱) MBP (برنامه‌ریز مبتنی بر مدل) را توصیف می‌کنند که از نمودارهای تصمیم دودویی برای برنامه‌ریزی سازگار و احتیاطی استفاده می‌کند. برخی نویسندگان از «برنامه‌ریزی شرطی» و «برنامه‌ریزی احتیاطی» به‌عنوان مترادف استفاده می‌کنند؛ برخی دیگر تمایز قائل می‌شوند که «شرطی» به کنش‌های با اثرات ناجبرگرایانه اشاره دارد و «احتیاطی» یعنی استفاده از حس‌کردن برای غلبه بر مشاهده‌ناپذیری تا حدی.

با نگاه به گذشته، اکنون می‌توان دید الگوریتم‌های برنامه‌ریزی کلاسیک عمده چگونه به نسخه‌های بسط‌یافته برای حوزه‌های نامطمئن انجامیدند. جستجوی اکتشافی سریع-پیشرو در فضای حالت به جستجوی پیشرو در فضای باور انجامید (بونت و گفنر، ۲۰۰۰؛ هافمن و برافمن، ۲۰۰۵)؛ SATPLAN به SATPLAN تصادفی (ماجرچیک و لیتمن، ۲۰۰۳) و به برنامه‌ریزی با منطق بولی سوردار (رینتانن، ۲۰۰۷) انجامید؛ برنامه‌ریزی ترتیب‌جزئی به UWL (اتزیونی و همکاران، ۱۹۹۲) و CNLP (پئوت و اسمیت، ۱۹۹۲) انجامید؛ Graphplan به Sensory Graphplan یا SGP (ولد و همکاران، ۱۹۹۸) انجامید.

نخستین برنامه‌ریز آنلاین با پایش اجرا PLANEX بود (فایکز و همکاران، ۱۹۷۲)، که با برنامه‌ریز STRIPS برای کنترل ربات شیکی کار می‌کرد. SIPE (سامانهٔ برنامه‌ریزی تعاملی و پایش اجرا) (ویلکینز، ۱۹۸۸) نخستین برنامه‌ریزی بود که به‌صورت نظام‌مند با مسئلهٔ برنامه‌ریزی مجدد سروکار داشت. در پروژه‌های نمایشی در چند حوزه استفاده شده، از جمله برنامه‌ریزی عملیات روی عرشهٔ پرواز ناو هواپیمابر، زمان‌بندی کارگاه برای کارخانهٔ آبجوی استرالیایی، و برنامه‌ریزی ساخت ساختمان‌های چندطبقه (کارتام و لویت، ۱۹۹۰).

در اواسط دههٔ ۱۹۸۰، بدبینی دربارهٔ زمان‌های اجرای کند سامانه‌های برنامه‌ریزی به پیشنهاد عامل‌های بازتابی به نام «سامانه‌های برنامه‌ریزی واکنشی» (Reactive Planning Systems) انجامید (بروکس، ۱۹۸۶؛ آگر و چپمن، ۱۹۸۷). «برنامه‌های همگانی» (شوپرس، ۱۹۸۹) به‌عنوان روش جدول‌جست‌وجو برای برنامه‌ریزی واکنشی توسعه یافتند، اما معلوم شد بازکشف ایدهٔ سیاست‌هایی است که مدت‌ها در فرایندهای تصمیم مارکوف استفاده می‌شدند (فصل ۱۷ را ببینید). کونیگ (۲۰۰۱) فنون برنامه‌ریزی آنلاین را زیر نام «جستجوی عامل‌محور» مرور می‌کند.

برنامه‌ریزی با قیدهای زمانی نخستین بار توسط DEVISER (ور، ۱۹۸۳) مدیریت شد. بازنمایی زمان در برنامه‌ها توسط آلن (۱۹۸۴) و توسط دین و همکاران (۱۹۹۰) در سامانهٔ FORBIN پرداخته شد. NONLIN+ (تیت و وایتر، ۱۹۸۴) و SIPE (ویلکینز، ۱۹۹۰) می‌توانستند دربارهٔ تخصیص منابع محدود به گام‌های برنامهٔ گوناگون استدلال کنند. O-PLAN (بل و تیت، ۱۹۸۵) بر مسائل منابع مانند برنامه‌ریزی تهیهٔ نرم‌افزار در پرایس واترهاوس و برنامه‌ریزی مونتاژ اکسل عقب در جگوار کارز اعمال شده.

دو برنامه‌ریز SAPA (دو و کامباماپاتی، ۲۰۰۱) و T4 (هاسلوم و گفنر، ۲۰۰۱) هر دو از جستجوی فضای حالت پیشرو با اکتشافی‌های پیچیده برای مدیریت کنش‌های با مدت و منابع استفاده کردند. بدیل استفاده از زبان‌های کنش بسیار بیانی، اما هدایت‌شده توسط اکتشافی‌های حوزه‌ویژهٔ انسان‌نوشته است، چنان‌که ASPEN (فوکوناگا و همکاران، ۱۹۹۷)، HSTS (جونسون و همکاران، ۲۰۰۰) و IxTeT (غَلّاب و لاروئل، ۱۹۹۴) انجام می‌دهند.

تعدادی سامانهٔ برنامه‌ریزی-و-زمان‌بندی ترکیبی مستقر شده‌اند: ISIS (فاکس و همکاران، ۱۹۸۲؛ فاکس، ۱۹۹۰) برای زمان‌بندی کارگاه در وستینگهاوس استفاده شده، GARI (دسکوت و لاتومب، ۱۹۸۵) ماشین‌کاری و ساخت قطعات مکانیکی را برنامه‌ریزی کرد، FORBIN برای کنترل کارخانه و NONLIN+ برای برنامه‌ریزی لجستیک دریایی استفاده شد. ما انتخاب کردیم برنامه‌ریزی و زمان‌بندی را دو مسئلهٔ جدا ارائه کنیم؛ کوشینگ و همکاران (۲۰۰۷) نشان می‌دهند این می‌تواند در برخی مسائل به ناقصی بینجامد.

تاریخچهٔ بلندی از زمان‌بندی در هوافضا هست. T-SCHED (درابل، ۱۹۹۰) برای زمان‌بندی توالی‌های فرمان مأموریت برای ماهوارهٔ UOSAT-II استفاده شد. OPTIMUM-AIV (آروپ و همکاران، ۱۹۹۴) و PLAN-ERS1 (فوکس و همکاران، ۱۹۹۰)، هر دو بر O-PLAN مبتنی، به‌ترتیب برای برنامه‌ریزی مونتاژ فضاپیما و برنامه‌ریزی مشاهده در آژانس فضایی اروپا استفاده شدند. SPIKE (جانستون و ادورف، ۱۹۹۲) برای برنامه‌ریزی مشاهده در ناسا برای تلسکوپ فضایی هابل استفاده شد، در حالی که سامانهٔ زمان‌بندی پردازش زمینی شاتل فضایی (دیل و همکاران، ۱۹۹۴) زمان‌بندی کارگاه تا ۱۶,۰۰۰ شیفت کارگر انجام می‌دهد. عامل از راه دور (موسکتولا و همکاران، ۱۹۹۸) نخستین برنامه‌ریز-زمان‌بند خودمختاری شد که فضاپیما را کنترل کرد، وقتی در ۱۹۹۹ روی کاوشگر دیپ‌اسپیس وان پرواز کرد. کاربردهای فضایی توسعهٔ الگوریتم‌های تخصیص منابع را رانده‌اند؛ لابوری (۲۰۰۳) و موسکتولا (۲۰۰۲) را ببینید.

ادبیات زمان‌بندی در مقالهٔ مرور کلاسیک (لاولر و همکاران، ۱۹۹۳)، کتابی (پینهدو، ۲۰۰۸) و راهنمای ویرایش‌شده (بلاژویچ و همکاران، ۲۰۰۷) ارائه شده.

پیچیدگی محاسباتی برنامه‌ریزی توسط چند نویسنده تحلیل شده (بیلاندر، ۱۹۹۴؛ غَلّاب و همکاران، ۲۰۰۴؛ رینتانن، ۲۰۱۶). دو تکلیف اصلی هست: PlanSAT پرسش وجود هر برنامه‌ای است که مسئلهٔ برنامه‌ریزی را حل کند. Bounded PlanSAT می‌پرسد آیا راه‌حلی به طول k یا کمتر هست؛ این را می‌توان برای یافتن برنامهٔ بهینه استفاده کرد. هر دو برای برنامه‌ریزی کلاسیک تصمیم‌پذیرند (زیرا شمار حالت‌ها متناهی است). اما اگر نمادهای تابع را به زبان بیفزاییم، شمار حالت‌ها نامتناهی می‌شود و PlanSAT فقط نیمه‌تصمیم‌پذیر می‌شود. برای مسائل گزاره‌شده هر دو در ردهٔ پیچیدگی PSPACE هستند، رده‌ای بزرگ‌تر (و بنابراین دشوارتر) از NP که به مسائل قابل‌حل توسط ماشین تورینگ جبرگرا با

مقدار چندجمله‌ای فضا اشاره دارد. این نتایج نظری دلسردکننده‌اند، اما در عمل، مسائلی که می‌خواهیم حل کنیم تمایل دارند چندان بد نباشند. مزیت واقعی صورت‌بندی برنامه‌ریزی کلاسیک آن است که توسعهٔ اکتشافی‌های مستقل از حوزهٔ بسیار دقیق را تسهیل کرده؛ دیگر رویکردها چندان پربار نبوده‌اند.

«خواندنی‌هایی در برنامه‌ریزی» (آلن و همکاران، ۱۹۹۰) گلچین جامعی از کارهای اولیه در حوزه است. ولد (۱۹۹۴، ۱۹۹۹) دو مرور عالی از الگوریتم‌های برنامه‌ریزی دههٔ ۱۹۹۰ ارائه می‌دهد. دیدن تغییر در پنج سال میان دو مرور جالب است: نخستین بر برنامه‌ریزی ترتیب‌جزئی تمرکز دارد و دومین Graphplan و SATPLAN را معرفی می‌کند. «برنامه‌ریزی و کنش خودکار» (غَلّاب و همکاران، ۲۰۱۶) کتاب درسی عالی دربارهٔ همهٔ جنبه‌های حوزه است. متن لاوال، «الگوریتم‌های برنامه‌ریزی» (۲۰۰۶)، هم برنامه‌ریزی کلاسیک و هم تصادفی را پوشش می‌دهد، با پوشش گستردهٔ برنامه‌ریزی حرکت ربات.

پژوهش برنامه‌ریزی از آغاز هوش مصنوعی مرکزی بوده و مقاله‌های برنامه‌ریزی جزء ثابت ژورنال‌ها و کنفرانس‌های اصلی هوش مصنوعی‌اند. همچنین کنفرانس‌های تخصصی مانند کنفرانس بین‌المللی برنامه‌ریزی و زمان‌بندی خودکار و کارگاه بین‌المللی برنامه‌ریزی و زمان‌بندی برای فضا هست.

پانوشت‌ها:

[1] جهان بلوک‌هایی که معمولاً در پژوهش برنامه‌ریزی استفاده می‌شود بسیار ساده‌تر از نسخهٔ SHRDLU است (صفحهٔ ۲۰).

[2] بسیاری از مسائل با این قرارداد نوشته می‌شوند. برای مسائلی که نیستند، هر لیترال منفی ¬P را در هدف یا پیش‌شرط با لیترال مثبت تازهٔ 'P جایگزین کن و حالت آغازین و اثرات کنش را متناسباً اصلاح کن.

[3] اگر برای مسئلهٔ ناجبرگرایانه راه‌حل‌های چرخه‌ای لازم باشد، جستجوی AND–OR باید به نسخهٔ حلقه‌داری مانند LAO* تعمیم یابد (هانسن و زیلبرشتاین، ۲۰۰۱).

[4] در ۱۹۵۴، خانمی به نام هاجز در آلاباما توسط شهاب‌سنگی که از پشت‌بامش گذشت زده شد. در ۱۹۹۲، تکه‌ای از شهاب‌سنگ امباله به سر پسر کوچکی خورد؛ خوشبختانه، فرودش توسط برگ‌های موز کند شده بود (جنیسکنس و همکاران، ۱۹۹۴). و در ۲۰۰۹، پسر آلمانی ادعا کرد شهاب‌سنگی به اندازهٔ نخود به دستش خورده. هیچ‌کدام از این رویدادها آسیب جدی نداشتند، و پیشنهاد می‌کنند نیاز به برنامه‌ریزی قبلی در برابر چنین احتیاط‌هایی گاهی اغراق‌آمیز است.

[5] پایش برنامه یعنی بالاخره، پس از ۳۷۴ صفحه، عاملی داریم که از سوسک سرگین‌غلتان باهوش‌تر است (صفحهٔ ۴۱ را ببینید). عامل پایش برنامه متوجه می‌شد گلولهٔ سرگین از چنگش گم شده و برای گرفتن گلوله‌ای دیگر و بستن سوراخ برنامه‌ریزی مجدد می‌کرد.

[6] تکرار بیهودهٔ تعمیر برنامه دقیقاً رفتاری است که زنبور اسفکس نشان می‌دهد (صفحهٔ ۴۱).

فصل ۱۲: کمی کردن عدم قطعیت (Quantifying Uncertainty)

در این فصل می‌بینیم چگونه عدم قطعیت را با درجه‌های عددی باور رام کنیم.

۱۲.۱ عملکرد در شرایط عدم قطعیت

عامل‌ها در دنیای واقعی باید عدم قطعیت (Uncertainty) را مدیریت کنند؛ خواه ناشی از مشاهده‌پذیری جزئی (Partial Observability) باشد، خواه ناجبرگرایی (Nondeterminism) یا دشمنان. عامل ممکن است هرگز به‌طور قطع نداند اکنون در چه حالتی است یا پس از توالی از کنش‌ها به کجا خواهد رسید.

ما دیده‌ایم عامل‌های حل مسئله و منطقی چگونه با نگه‌داشتن «حالت باور» (Belief State)—بازنماییِ مجموعهٔ همهٔ حالت‌های جهانِ ممکنی که ممکن است در آن باشد—و تولید «برنامهٔ اقتضایی» (Contingency Plan) که با هر پیشامدِ ممکنی که حسگرها در طول اجرا گزارش کنند کنار بیاید، عدم قطعیت را مدیریت می‌کنند. این رویکرد روی مسائل ساده کار می‌کند، اما عیب‌هایی دارد:

- عامل باید هر تبیینِ ممکن برای مشاهده‌های حسگرش را در نظر بگیرد، هرچند نامحتمل. این به حالت باورِ بزرگی پر از امکان‌های نامحتمل می‌انجامد.
- برنامهٔ اقتضاییِ درستی که با هر پیشامدی کنار بیاید می‌تواند خودسرانه بزرگ شود و باید اقتضائاتِ خودسرانه نامحتمل را در نظر بگیرد.
- گاهی هیچ برنامه‌ای وجود ندارد که رسیدن به هدف را تضمین کند—و با این حال عامل باید عمل کند. باید راهی برای مقایسهٔ شایستگی برنامه‌های غیرتضمینی داشته باشد.

فرض کنید، برای نمونه، یک تاکسی خودکار هدفِ رساندن مسافر به‌موقع به فرودگاه را دارد. تاکسی برنامه‌ای به نام A90 می‌سازد که شامل ترک خانه ۹۰ دقیقه قبل از پرواز و رانندگی با سرعت معقول است. حتی با وجود اینکه فرودگاه فقط ۸ کیلومتر فاصله دارد، عامل منطقی نمی‌تواند با یقین مطلق نتیجه بگیرد که «برنامهٔ A90 ما را به‌موقع به فرودگاه می‌رساند.» در عوض، به نتیجهٔ ضعیف‌تری می‌رسد: «برنامهٔ A90 ما را به‌موقع به فرودگاه می‌رساند، تا زمانی که ماشین خراب نشود، تصادف نکنم، جاده بسته نباشد، شهاب‌سنگی به ماشین نخورد، و ... .» هیچ‌یک از این شرایط را نمی‌توان با قطعیت استنتاج کرد، پس نمی‌توانیم استنتاج کنیم که برنامه موفق می‌شود. این «مسئلهٔ تحدید» (Qualification Problem) منطقی است (صفحهٔ ۲۴۱) که تاکنون برایش راه‌حل واقعی ندیده‌ایم.

با این حال، از جهتی A90 در واقع کارِ درست است. مقصودمان چیست؟

همان‌طور که در فصل ۲ بحث کردیم، مقصود این است که از میان همهٔ برنامه‌هایی که می‌توان اجرا کرد، انتظار می‌رود A90 «معیار عملکرد» (Performance Measure) عامل را بیشینه کند (که انتظار نسبت به دانشِ عامل دربارهٔ محیط است). معیار عملکرد شامل رسیدن به‌موقع به فرودگاه برای پرواز، پرهیز از انتظارِ طولانی و بی‌حاصل در فرودگاه، و پرهیز از جریمهٔ سرعت در طول مسیر است. دانشِ عامل نمی‌تواند هیچ‌یک از این پیامدها را برای A90 تضمین کند، اما می‌تواند درجهٔ باوری از حصولِ آن‌ها فراهم کند.

برنامه‌های دیگر، مانند A180، ممکن است باورِ عامل به رسیدن به‌موقع به فرودگاه را افزایش دهند، اما احتمالِ انتظارِ طولانی و کسل‌کننده را نیز افزایش دهند. کارِ درست—«تصمیم عقلایی» (Rational Decision)—بنابراین هم به اهمیتِ نسبی هدف‌های گوناگون بستگی دارد و هم به احتمالِ اینکه به آن‌ها برسیم و میزانِ رسیدن به آن‌ها. بقیهٔ این بخش این ایده‌ها را صیقل می‌دهد، آماده‌سازی برای توسعهٔ نظریه‌های عمومیِ استدلالِ نامطمئن و تصمیم‌های عقلایی که در این فصل و فصل‌های بعدی ارائه می‌کنیم.

۱۲.۱.۱ خلاصه‌سازی عدم قطعیت

مثالی از استدلالِ نامطمئن را در نظر بگیریم: تشخیصِ دندان‌دردِ بیمارِ دندان‌پزشکی.

تشخیص—خواه برای پزشکی، خواه تعمیر خودرو یا هر چیز دیگر—تقریباً همیشه با عدم قطعیت همراه است. بیایید با منطق گزاره‌ای قاعده‌هایی برای تشخیصِ دندان‌پزشکی بنویسیم تا ببینیم رویکردِ منطقی چگونه از هم می‌پاشد. قاعدهٔ سادهٔ زیر را در نظر بگیرید:

```
Toothache ⇒ Cavity
```

مشکل این است که این قاعده نادرست است. همهٔ بیمارانِ با دندان‌درد پوسیدگی ندارند؛ برخی بیماریِ لثه دارند، برخی آبسه، یا یکی از چند مشکل دیگر:

```
... ∨ Toothache ⇒ Cavity ∨ GumProblem ∨ Abscess
```

متأسفانه، برای درست‌کردن قاعده باید فهرستی تقریباً نامحدود از مشکلاتِ ممکن بیفزاییم. می‌توانیم قاعده را به قاعدهٔ علّی تبدیل کنیم:

```
Cavity ⇒ Toothache
```

اما این قاعده هم درست نیست؛ همهٔ پوسیدگی‌ها درد ایجاد نمی‌کنند. تنها راهِ درست‌کردن قاعده آن است که آن را به‌لحاظ منطقی جامع کنیم: سمتِ چپ را با همهٔ شرایطِ لازم برای اینکه پوسیدگی موجب دندان‌درد شود تقویت کنیم. تلاش برای استفاده از منطق در حوضه‌ای مانند تشخیصِ پزشکی بدین ترتیب به سه دلیلِ اصلی شکست می‌خورد:

- **تنبلی (Laziness):** فهرست‌کردنِ مجموعهٔ کاملِ مقدمه‌ها یا تالی‌های لازم برای قاعدهٔ بدون‌استثنا کارِ زیادی می‌برد و استفاده از چنین قاعده‌هایی بسیار دشوار است.
- **نادانی نظری (Theoretical Ignorance):** دانشِ پزشکی نظریهٔ کاملی برای این حوزه ندارد.
- **نادانی عملی (Practical Ignorance):** حتی اگر همهٔ قاعده‌ها را بدانیم، ممکن است دربارهٔ بیمارِ خاصی نامطمئن باشیم، زیرا همهٔ آزمون‌های لازم انجام نشده یا نمی‌توانند انجام شوند.

ارتباط میان دندان‌درد و پوسیدگی در هیچ‌یک از دو جهت یک نتیجهٔ منطقیِ اکید نیست. این ویژگیِ حوزهٔ پزشکی و نیز اغلب حوزه‌های قضاوتیِ دیگر است: حقوق، تجارت، طراحی، تعمیر خودرو، باغبانی، دوستیابی و مانند آن. دانشِ عامل در بهترین حالت فقط می‌تواند «درجهٔ باور» (Degree of Belief) به جمله‌های مربوط فراهم کند. ابزارِ اصلی ما برای سروکار داشتن با درجه‌های باور، «نظریهٔ احتمال» (Probability Theory) است. در اصطلاح‌شناسی بخش ۸.۱، «تعهدهای هستی‌شناختی» (Ontological Commitments) منطق و نظریهٔ احتمال یکسان‌اند—جهان از واقعیت‌هایی ساخته شده که در هر موردِ خاص برقرارند یا برقرار نیستند—اما «تعهدهای شناخت‌شناختی» (Epistemological Commitments) متفاوت‌اند: عاملِ منطقی به هر جمله باور دارد که صادق یا کاذب است یا نظری ندارد، در حالی که عاملِ احتمالاتی ممکن است درجهٔ باورِ عددی بین ۰ (برای جمله‌های قطعاً کاذب) و ۱ (قطعاً صادق) داشته باشد.

نظریهٔ احتمال راهی برای خلاصه‌سازیِ عدم قطعیتِ ناشی از تنبلی و نادانیِ ما فراهم می‌کند و بدین‌ترتیب مسئلهٔ تحدید را حل می‌کند. شاید به‌طور قطع ندانیم چه چیزی بیمارِ خاصی را آزار می‌دهد، اما باور داریم که مثلاً ۸۰٪ احتمال—یعنی احتمالِ ۰.۸—دارد بیماری که دندان‌درد دارد پوسیدگی داشته باشد. یعنی انتظار داریم از میان همهٔ وضعیت‌هایی که تا آن‌جا که دانشِ ما می‌رسد از وضعیتِ کنونی تمایزناپذیرند، بیمار در ۸۰٪ آن‌ها پوسیدگی داشته باشد. این باور می‌تواند از داده‌های آماری—۸۰٪ بیمارانِ دندان‌دردِ دیده‌شده تاکنون پوسیدگی داشته‌اند—یا از دانشِ عمومیِ دندان‌پزشکی، یا از ترکیبی از منابعِ شواهد به دست آمده باشد.

نکتهٔ گیج‌کننده‌ای هست: در لحظهٔ تشخیص، در جهانِ واقعی عدم قطعیتی وجود ندارد؛ بیمار یا پوسیدگی دارد یا ندارد. پس معنای گفتن اینکه احتمالِ پوسیدگی ۰.۸ است چیست؟ نباید ۰ یا ۱ باشد؟ پاسخ این است که گزاره‌های احتمال نسبت به یک «حالتِ دانش» (Knowledge State) بیان می‌شوند، نه نسبت به جهانِ واقعی. می‌گوییم «احتمال اینکه بیمار پوسیدگی داشته باشد، به‌شرطی که دندان‌درد داشته باشد، ۰.۸ است.» اگر بعداً بیاموزیم بیمار سابقهٔ بیماریِ لثه دارد، می‌توانیم گزارهٔ متفاوتی بیان کنیم:

```
P(cavity | toothache ∧ gumdisease) = 0.8
```

این دو گزاره با هم تناقض ندارند؛ هر یک ادعایی جداگانه دربارهٔ حالتِ دانشِ متفاوتی است.

۱۲.۱.۲ عدم قطعیت و تصمیم‌های عقلایی

برنامهٔ A90 برای رسیدن به فرودگاه را بار دیگر در نظر بگیرید. فرض کنید به ما ۹۷٪ شانسِ رسیدن به پرواز می‌دهد. آیا این بدان معناست انتخابِ عقلایی است؟ نه لزوماً: ممکن است برنامه‌های دیگری مانند A180 با احتمال‌های بالاتر وجود داشته باشند. اگر از دست‌ندادنِ پرواز حیاتی باشد، پس ارزش دارد ریسکِ انتظارِ طولانی‌تر در فرودگاه را بپذیریم.

A1440 چطور، برنامه‌ای که شامل ترک خانه ۲۴ ساعت زودتر است؟ در بیشتر شرایط این انتخابِ خوبی نیست، زیرا اگرچه تقریباً رسیدنِ به‌موقع را تضمین می‌کند، انتظاری طاقت‌فرسا به همراه دارد—ناگفته نماند احتمالِ رژیمِ غذاییِ ناخوشایند از غذای فرودگاه.

برای چنین انتخاب‌هایی، عامل نخست باید «ترجیح‌هایی» (Preferences) بین پیامدهای مختلفِ برنامه‌های مختلف داشته باشد. هدف می‌تواند کمّی شود: در حالی که هدفِ منطقیِ سادهٔ «رسیدنِ به‌موقع» را می‌توان با یک بولی بیان کرد، ما ممکن است بخواهیم مقیاسِ درجه‌بندی‌شده‌ای از پیامدها داشته باشیم: رسیدنِ به‌موقع برای پرواز پیامدی بهتر از رسیدن ۳۰ دقیقه دیرتر است، که به‌نوبهٔ خود بهتر از رسیدن ۵ ساعت دیرتر است، که بهتر از رسیدن ۵ ساعت دیرتر در طوفانِ برف است. در این مقیاسِ درجه‌بندی‌شده، از مفهوم «مطلوبیت» (Utility) استفاده می‌کنیم، که ابتدا در بخش ۲.۴.۶ معرفی شد. مطلوبیتِ یک حالت نسبت به یک عامل است. برای نمونه، مطلوبیتِ حالتی که در آن سفید در شطرنج سیاه را کیش‌ومات کرده است، بدیهی است برای عاملی که با سفید بازی می‌کند بالاست، اما برای عاملی که با سیاه بازی می‌کند پایین. اما نمی‌توانیم دقیقاً از امتیازهای ۱، ۱/۲ و ۰ که قواعدِ شطرنجِ مسابقاتی تحمیل می‌کنند پیروی کنیم—برخی بازیکنان (از جمله نویسندگان) ممکن است از تساوی با قهرمانِ جهان سر از پا نشناسند، در حالی که بازیکنانِ دیگر (از جمله قهرمانِ سابقِ جهان) شاید نباشند. سلیقه یا ترجیح را نمی‌توان حساب‌کشی کرد: ممکن است فکر کنید عاملی که بستنیِ آدامسی-هالاپینو را به چیپسِ شکلاتی ترجیح می‌دهد عجیب است، اما نمی‌توانید بگویید آن عامل ناعقلایی است. تابعِ مطلوبیت می‌تواند هر مجموعه‌ای از ترجیح‌ها را توضیح دهد—عجیب یا معمولی، بزرگوارانه یا منحرفانه. توجه کنید که مطلوبیت‌ها می‌توانند نوعدوستی را نیز توضیح دهند، صرفاً با گنجاندنِ رفاهِ دیگران به‌عنوان یکی از عوامل.

ترجیح‌ها، که با مطلوبیت‌ها بیان می‌شوند، با احتمال‌ها در نظریهٔ عمومیِ تصمیم‌های عقلایی به نام «نظریهٔ تصمیم» (Decision Theory) ترکیب می‌شوند:

```
نظریهٔ تصمیم = نظریهٔ احتمال + نظریهٔ مطلوبیت
```

ایدهٔ بنیادیِ نظریهٔ تصمیم این است که عامل عقلایی است اگر و فقط اگر کنشی را برگزیند که بالاترین «مطلوبیتِ مورد انتظار» (Expected Utility) را به دست دهد، میانگین‌گرفته بر همهٔ پیامدهای ممکنِ کنش. این «اصلِ بیشینهٔ مطلوبیتِ مورد انتظار» (Maximum Expected Utility یا MEU) نامیده می‌شود. در این‌جا «مورد انتظار» یعنی «میانگین»، یا «میانگینِ آماری» مطلوبیت‌های پیامد، وزن‌داده‌شده با احتمالِ پیامد. این اصل را در فصل ۵ هنگام اشارهٔ گذرا به تصمیم‌های بهینه در تخته‌نرد در عمل دیدیم؛ در واقع اصلی کاملاً عمومی برای تصمیم‌گیریِ تک‌عاملی است. شکل ۱۲.۱ ساختارِ عاملی را ترسیم می‌کند که برای انتخابِ کنش‌ها از نظریهٔ تصمیم استفاده می‌کند:

```
function DT-AGENT(percept) returns an action
persistent: belief state, probabilistic beliefs about the current state of the world
                                        action, the agent's action
                    update the belief state based on the action and percept
                       ,calculate the probabilities of outcomes for actions
                  given the action descriptions and current belief state
                        choose the action with the highest expected utility
                 given the outcome probabilities and utility information
                                                                return action
```

شکل ۱۲.۱ – عامل نظریه‌تصمیمی که کنش‌های عقلایی را انتخاب می‌کند.

عامل در سطحِ انتزاعی یکسان با عامل‌های توصیف‌شده در فصل‌های ۴ و ۷ است که حالتِ باوری بازتاب‌دهندهٔ تاریخچهٔ ادراک‌ها تاکنون نگه‌داری می‌کنند. تفاوتِ اصلی این است که حالتِ باورِ عاملِ نظریه‌تصمیمی نه فقط امکان‌های حالت‌های جهان، بلکه احتمال‌های آن‌ها را نیز بازنمایی می‌کند. با حالتِ باور و دانشی از اثرهای کنش‌ها، عامل می‌تواند پیش‌بینی‌های احتمالاتیِ پیامدهای کنش انجام دهد و در نتیجه کنشی را برگزیند که بالاترین مطلوبیتِ مورد انتظار را دارد.

این فصل و فصلِ بعد بر وظیفهٔ بازنمایی و محاسبه با اطلاعاتِ احتمالاتی به‌طور کلی تمرکز می‌کنند. فصل ۱۴ به روش‌های مسائلِ خاصِ بازنمایی و به‌روزرسانیِ حالتِ باور در طولِ زمان و پیش‌بینیِ پیامدها می‌پردازد. فصل ۱۵ راه‌های ترکیبِ نظریهٔ احتمال با زبان‌های صوریِ بیانی مانند منطقِ مرتبهٔ اول و زبان‌های برنامه‌نویسیِ همه‌منظوره را بررسی می‌کند. فصل ۱۶ نظریهٔ مطلوبیت را عمیق‌تر پوشش می‌دهد، و فصل ۱۷ الگوریتم‌هایی برای برنامه‌ریزیِ توالی‌های کنش در محیط‌های تصادفی توسعه می‌دهد. فصل ۱۸ بسطِ این ایده‌ها به محیط‌های چندعاملی را پوشش می‌دهد.

۱۲.۲ نمادِ پایهٔ احتمال

برای اینکه عاملِ ما اطلاعاتِ احتمالاتی را بازنمایی و استفاده کند، به زبانِ صوری نیاز داریم.

زبانِ نظریهٔ احتمال به‌طور سنتی غیررسمی بوده، توسط ریاضی‌دانانِ انسانی برای ریاضی‌دانانِ انسانیِ دیگر نوشته می‌شده. پیوستِ الف مقدمهٔ استانداردی بر نظریهٔ احتمالِ مقدماتی دارد؛ در این‌جا، رویکردی متناسب‌تر با نیازهای هوش مصنوعی اتخاذ و آن را با مفاهیمِ منطقِ صوری پیوند می‌دهیم.

۱۲.۲.۱ احتمال دربارهٔ چیست

مانند گزاره‌های منطقی، گزاره‌های احتمالاتی دربارهٔ جهان‌های ممکن‌اند. در حالی که گزاره‌های منطقی می‌گویند کدام جهان‌های ممکن به‌طور قطعی کنار گذاشته می‌شوند (همهٔ آن‌هایی که گزاره در آن‌ها کاذب است)، گزاره‌های احتمالاتی دربارهٔ میزانِ محتمل‌بودنِ جهان‌های گوناگون سخن می‌گویند. در نظریهٔ احتمال، مجموعهٔ همهٔ جهان‌های ممکن «فضای نمونه» (Sample Space) نامیده می‌شود. جهان‌های ممکن دو به دو مانعةالجمع و استیفاگرند—دو جهانِ

ممکن نمی‌توانند هر دو برقرار باشند و یک جهانِ ممکن باید برقرار باشد. برای نمونه، اگر بخواهیم دو تاس (متمایز) بیندازیم، ۳۶ جهانِ ممکن برای در نظر گرفتن داریم: (1,1), (1,2), ..., (6,6). حرفِ یونانی Ω (اُمگای بزرگ) برای اشاره به فضای نمونه استفاده می‌شود و ω (اُمگای کوچک) به عناصرِ فضا، یعنی جهان‌های ممکنِ خاص، اشاره دارد.

«مدلِ احتمال» (Probability Model) کاملاً مشخص‌شده با هر جهانِ ممکنِ ω یک احتمالِ عددیِ P(ω) مرتبط می‌کند.[1] اصل‌های پایهٔ نظریهٔ احتمال می‌گویند هر جهانِ ممکن احتمالی بین ۰ و ۱ دارد و احتمالِ کلِ مجموعهٔ جهان‌های ممکن ۱ است:

$$0 \le P(\omega) \le 1 \quad \text{برای هر } \omega \qquad \text{و} \qquad \sum_{\omega \in \Omega} P(\omega) = 1 \tag{12.1}$$

برای نمونه، اگر فرض کنیم هر تاس منصفانه است و انداختن‌ها با هم تداخل نمی‌کنند، آنگاه هر یک از جهان‌های ممکن (1,1), (1,2), ..., (6,6) احتمالِ 1/36 دارد. اگر تاس‌ها تقلب‌شده باشند، برخی جهان‌ها احتمالِ بالاتر و برخی پایین‌تر خواهند داشت، اما همه همچنان به ۱ جمع خواهند شد. گزاره‌ها و پرس‌وجوهای احتمالاتی معمولاً دربارهٔ جهان‌های ممکنِ خاص نیستند، بلکه دربارهٔ مجموعه‌هایی از آن‌ها هستند. برای نمونه، ممکن است بپرسیم احتمالِ اینکه دو تاس مجموع‌شان ۱۱ شود چقدر است، احتمالِ اینکه دول بیاید چقدر است، و مانند آن. در نظریهٔ احتمال، این مجموعه‌ها «رخداد» (Event) نامیده می‌شوند—اصطلاحی که در فصل ۱۰ به‌وفور برای مفهومِ متفاوتی استفاده شد. در منطق، مجموعه‌ای از جهان‌ها با گزاره‌ای در زبانِ صوری متناظر است؛ به‌طور خاص، برای هر گزاره، مجموعهٔ متناظر فقط آن جهان‌های ممکنی را در بر می‌گیرد که گزاره در آن‌ها برقرار است. (از این رو، «رخداد» و «گزاره» در این زمینه تقریباً یک معنا دارند، جز اینکه گزاره در زبانِ صوری بیان می‌شود.) احتمالِ مرتبط با گزاره به‌صورت جمعِ احتمال‌های جهان‌هایی تعریف می‌شود که گزاره در آن‌ها برقرار است:

$$\text{برای هر گزارهٔ } \varphi، \quad P(\varphi) = \sum_{\omega \in \varphi} P(\omega) \tag{12.2}$$

برای نمونه، هنگام انداختنِ تاس‌های منصفانه، P(Total=11) = P((5,6)) + P((6,5)) = 1/36 + 1/36 = 1/18 داریم. توجه کنید که نظریهٔ احتمال به دانشِ کامل از احتمال‌های هر جهانِ ممکن نیاز ندارد. برای نمونه، اگر باور داشته باشیم تاس‌ها برای تولیدِ عددِ یکسان تبانی می‌کنند، ممکن است بدون دانستنِ اینکه آیا تاس‌ها دولِ ۶ را به دولِ ۲ ترجیح می‌دهند، P(doubles) = 1/4 را اظهار کنیم. همانند گزاره‌های منطقی، این اظهار، مدلِ احتمالِ زیربنایی را قید می‌زند بدون آنکه کاملاً تعیینش کند.

احتمال‌هایی مانند P(Total=11) و P(doubles) «احتمالِ نامشروط» (Unconditional Probability) یا «احتمالِ پیشینی» (Prior Probability) نامیده می‌شوند (و گاهی به‌اختصار فقط «پیشین»)؛ آن‌ها به درجه‌های باور به گزاره‌ها در غیابِ هر اطلاعاتِ دیگری اشاره دارند. اما بیشترِ اوقات اطلاعاتی داریم، معمولاً «شاهد» (Evidence)، که قبلاً آشکار شده. برای نمونه، تاسِ اول ممکن است ۵ نشان دهد و ما با نفسِ حبس‌شده منتظریم دیگری از چرخش بایستد. در آن صورت، به احتمالِ نامشروطِ آمدنِ دول علاقه‌مند نیستیم، بلکه به «احتمالِ شرطی» (Conditional Probability) یا «احتمالِ پسینی» (Posterior Probability) (یا به‌اختصار «پسین») آمدنِ دول به‌شرطِ اینکه تاسِ اول ۵ باشد. این

احتمال P(doubles | Die1=5) نوشته می‌شود، که در آن «|» «به‌شرط» تلفظ می‌شود.[۲] به‌طور مشابه، اگر برای معاینهٔ دوره‌ایِ برنامه‌ریزی‌شده نزد دندان‌پزشک بروم، احتمالِ پیشینیِ P(cavity)=0.2 ممکن است موردِ علاقه باشد؛ اما اگر به دلیلِ دندان‌درد نزد دندان‌پزشک بروم، احتمالِ شرطیِ P(cavity | toothache)=0.6 است که اهمیت دارد.

درکِ این نکته مهم است که P(cavity)=0.2 پس از مشاهدهٔ دندان‌درد همچنان معتبر است؛ فقط چندان مفید نیست. هنگام تصمیم‌گیری، عامل باید بر همهٔ شواهدی که مشاهده کرده شرطی کند. همچنین درکِ تفاوت میان شرطی‌کردن و استلزامِ منطقی مهم است. اظهار اینکه P(cavity | toothache)=0.6 به این معنا نیست که «هرگاه دندان‌درد صادق است، نتیجه بگیر پوسیدگی با احتمالِ ۰.۶ صادق است»؛ بلکه به این معناست که «هرگاه دندان‌درد صادق است و اطلاعاتِ بیشتری نداریم، نتیجه بگیر پوسیدگی با احتمالِ ۰.۶ صادق است.» شرطِ اضافی مهم است؛ برای نمونه، اگر اطلاعاتِ بیشتری داشتیم که دندان‌پزشک هیچ پوسیدگی‌ای نیافته، قطعاً نمی‌خواهیم نتیجه بگیریم پوسیدگی با احتمالِ ۰.۶ صادق است؛ در عوض باید از P(cavity | toothache ∧ ¬cavity) = 0 استفاده کنیم.

از نظرِ ریاضی، احتمال‌های شرطی بر حسبِ احتمال‌های نامشروط به‌صورتِ زیر تعریف می‌شوند: برای هر گزارهٔ a و b داریم

$$P(a \mid b) = P(a \wedge b) / P(b) \qquad (12.3)$$

که هرگاه P(b) > 0 برقرار است. برای نمونه، P(doubles | Die1=5) = P(doubles ∧ Die1=5) / P(Die1=5). تعریف وقتی معنا پیدا می‌کند که به یاد آوریم مشاهدهٔ b همهٔ جهان‌های ممکنی را که b در آن‌ها کاذب است کنار می‌گذارد و مجموعه‌ای به جا می‌گذارد که احتمالِ کلش دقیقاً P(b) است. درونِ آن مجموعه، جهان‌هایی که a در آن‌ها صادق است باید a ∧ b را ارضا کنند و کسریِ P(a∧b)/P(b) را تشکیل می‌دهند.

تعریفِ احتمالِ شرطی، معادلهٔ (12.3)، می‌تواند به شکلِ متفاوتی نوشته شود که «قاعدهٔ ضرب» (Product Rule) نامیده می‌شود:

$$P(a \wedge b) = P(a \mid b)\, P(b) \qquad (12.4)$$

قاعدهٔ ضرب شاید راحت‌تر به خاطر بماند: از این واقعیت می‌آید که برای صادق‌بودنِ a و b، نیاز داریم b صادق باشد، و همچنین نیاز داریم a به‌شرطِ b صادق باشد.

۱۲.۲.۲ زبانِ گزاره‌ها در اظهارهای احتمال

در این فصل و فصلِ بعد، گزاره‌های توصیف‌کنندهٔ مجموعه‌های جهان‌های ممکن معمولاً با نمادی نوشته می‌شوند که عناصرِ منطقِ گزاره‌ای و نمادِ ارضایِ قید را ترکیب می‌کند. در اصطلاح‌شناسیِ بخش ۲.۴.۷، «بازنماییِ عاملی» (Factored Representation) است، که در آن جهانِ ممکن با مجموعه‌ای از جفت‌های متغیر/مقدار بازنمایی می‌شود. بازنماییِ ساختاریافتهٔ بیانی‌تری نیز ممکن است، چنان‌که در فصل ۱۵ نشان داده می‌شود.

متغیرها در نظریهٔ احتمال «متغیرِ تصادفی» (Random Variable) نامیده می‌شوند و نام‌هایشان با حرفِ بزرگ شروع می‌شود. بدین‌ترتیب، در مثالِ تاس، Total و Die1 متغیرهای تصادفی‌اند. هر متغیرِ تصادفی تابعی است که از دامنهٔ جهان‌های ممکنِ Ω به «برد» (Range)—مجموعهٔ مقادیرِ ممکنی که می‌تواند بگیرد—نگاشت می‌کند. بردِ Total برای دو تاس مجموعهٔ {2,...,12} و بردِ Die1 مجموعهٔ {1,...,6} است. نامِ مقادیر همیشه کوچک است، پس ممکن است ∑x P(X=x) بنویسیم تا روی مقادیرِ X جمع بزنیم. متغیرِ تصادفیِ بولی بردِ {true, false} دارد؛ برای نمونه، گزارهٔ اینکه دول آمده می‌تواند Doubles=true نوشته شود. (بردِ جایگزین برای متغیرهای بولی مجموعهٔ {0,1} است، که در آن صورت گفته می‌شود متغیر «توزیعِ برنولی» (Bernoulli Distribution) دارد.) طبقِ قرارداد، گزاره‌های شکلِ برنولیِ A=true به‌اختصار a نوشته می‌شوند، در حالی که A=false به‌اختصار ¬a. (استفاده‌های cavity، doubles و toothache در بخشِ پیشین مخففِ‌هایی از این دست‌اند.) بردها می‌توانند مجموعه‌هایی از نشانه‌های دلخواه باشند. ممکن است بردِ Age را مجموعهٔ {juvenile, teen, adult} برگزینیم و بردِ Weather ممکن است {sun, rain, cloud, snow} باشد. وقتی ابهامی ممکن نیست، رایج است که مقداری به‌تنهایی به‌جای گزارهٔ «متغیرِ خاصی آن مقدار را دارد» بایستد؛ بدین‌ترتیب، sun می‌تواند به‌جای Weather=sun بایستد.[3] مثال‌های پیشین همگی بردهای متناهی دارند. متغیرها می‌توانند بردهای نامتناهی نیز داشته باشند—یا گسسته (مانند اعدادِ صحیح) یا پیوسته (مانند اعدادِ حقیقی). برای هر متغیر با بردِ مرتب، نامساوی‌ها نیز مجازند، مانند NumberOfAtomsInUniverse ≥ 10^{70}.

سرانجام، می‌توانیم این نوع گزاره‌های مقدماتی (شامل شکل‌های مخففِ متغیرهای بولی) را با استفاده از ربط‌های منطقِ گزاره‌ای ترکیب کنیم. برای نمونه، می‌توانیم «احتمال اینکه بیمار پوسیدگی داشته باشد، به‌شرطِ اینکه نوجوانی بدونِ دندان‌درد باشد، ۰.۱ است» را به‌صورتِ زیر بیان کنیم:

```
P(cavity | ¬toothache ∧ teen) = 0.1
```

در نمادِ احتمال، استفاده از ویرگول برای عطف نیز رایج است، پس می‌توانیم P(cavity | ¬toothache, teen) بنویسیم. گاهی می‌خواهیم دربارهٔ احتمال‌های همهٔ مقادیرِ ممکنِ یک متغیرِ تصادفی سخن بگوییم. می‌توانیم بنویسیم:

```
P(Weather=sun) = 0.6
P(Weather=rain) = 0.1
P(Weather=cloud) = 0.29
P(Weather=snow) = 0.01
```

اما به‌عنوانِ مخفف اجازه می‌دهیم P(Weather) = ⟨0.6, 0.1, 0.29, 0.01⟩، که در آن P پررنگ نشان می‌دهد نتیجه بردارِ اعداد است، و فرض می‌کنیم ترتیبِ ازپیش‌تعریف‌شدهٔ ⟨sun, rain, cloud, snow⟩ روی بردِ Weather وجود دارد. می‌گوییم گزارهٔ P یک «توزیعِ احتمال» (Probability Distribution) برای متغیرِ تصادفیِ Weather تعریف می‌کند—یعنی تخصیصِ احتمالی به هر مقدارِ ممکنِ متغیرِ تصادفی. (در این مورد، با بردِ متناهی و گسسته، توزیع «توزیعِ رست‌ای» (Categorical Distribution) نامیده می‌شود.) نمادِ P همچنین برای توزیع‌های شرطی استفاده می‌شود: P(X|Y) مقادیرِ P(X=xi | Y=yj) را برای هر جفتِ ممکنِ i، j می‌دهد. برای متغیرهای پیوسته، نوشتنِ کلِ توزیع به‌صورتِ

بردار ممکن نیست، زیرا بی‌نهایت مقدار وجود دارد. در عوض، می‌توانیم احتمالِ اینکه متغیرِ تصادفی مقدارِ x بگیرد را به‌صورتِ تابعِ پارامترشدهٔ x تعریف کنیم، که معمولاً «تابعِ چگالیِ احتمال» (Probability Density Function) نامیده می‌شود. برای نمونه، جملهٔ

```
P(NoonTemp=x) = Uniform(x; 18°C, 26°C)
```

باور به اینکه دمایِ ظهر به‌طور یکنواخت بین ۱۸ و ۲۶ درجهٔ سانتی‌گراد توزیع شده را بیان می‌کند. توابعِ چگالیِ احتمال (گاهی pdf نامیده می‌شوند) از نظرِ معنا با توزیع‌های گسسته تفاوت دارند. گفتن اینکه چگالیِ احتمال از ۱۸ تا ۲۶ درجهٔ سانتی‌گراد یکنواخت است بدان معناست که ۱۰۰٪ شانس وجود دارد دما جایی در آن ناحیهٔ ۸-درجه‌ای بیفتد و ۵۰٪ شانس در هر ناحیهٔ ۴-درجه‌ای فرعی، و مانند آن. چگالیِ احتمالِ متغیرِ تصادفیِ پیوستهٔ X در مقدارِ x را P(X=x) یا فقط P(x) می‌نویسیم؛ تعریفِ شهودی به‌صورتِ زیر است:

```
P(x) = lim(dx→0) P(x ≤ X ≤ x+dx) / dx
```

برای NoonTemp داریم:

```
P(NoonTemp=x) = Uniform(x; 18°C, 26°C) = { 1/(8°C)   اگر 18°C ≤ x ≤ 26°C
                                         { 0          در غیر این صورت
```

که در آن C به‌معنای سانتی‌گراد است (نه ثابت). در P(NoonTemp=20.18°C) = 1/(8°C)، توجه کنید که 1/(8°C) احتمال نیست؛ «چگالیِ احتمال» است. احتمالِ اینکه NoonTemp دقیقاً 20.18°C باشد صفر است، زیرا 20.18°C ناحیه‌ای با پهنای ۰ است. برخی نویسندگان برای احتمال‌های گسسته و چگالی‌های احتمال نمادهای متفاوتی استفاده می‌کنند؛ ما در هر دو مورد از P برای مقادیرِ احتمالِ خاص و P برای بردارهای مقادیر استفاده می‌کنیم، زیرا به‌ندرت اشتباه رخ می‌دهد و معادله‌ها معمولاً یکسان‌اند. توجه کنید که احتمال‌ها اعدادِ بدونِ واحدند، در حالی که توابعِ چگالی با یک واحد اندازه‌گیری می‌شوند، در این مورد درجهٔ سانتی‌گرادِ معکوس. اگر همان بازهٔ دمایی بر حسبِ درجهٔ فارنهایت بیان می‌شد، پهنایی ۱۴.۴ درجه داشت و چگالی 1/(14.4°F) می‌شد.

حالا نحوِ گزاره‌ها و اظهارهای احتمال را تعریف کرده‌ایم و بخشی از معناشناسی را داده‌ایم: معادلهٔ (12.2) احتمالِ گزاره را به‌صورتِ جمعِ احتمال‌های جهان‌هایی تعریف می‌کند که گزاره در آن‌ها برقرار است. برای کامل‌کردنِ معناشناسی، باید بگوییم جهان‌ها چیستند و چگونه تعیین کنیم آیا گزاره‌ای در جهانی برقرار است. این بخش را مستقیماً از معناشناسیِ منطقِ گزاره‌ای قرض می‌گیریم، به‌صورتِ زیر. «جهانِ ممکن» به‌صورتِ تخصیصِ مقادیر به همهٔ متغیرهای تصادفیِ موردمطالعه تعریف می‌شود.

به‌آسانی دیده می‌شود این تعریف الزامِ پایهٔ مانعةالجمع و استیفاکننده‌بودنِ جهان‌های ممکن را برآورده می‌کند (تمرین EXEX.۱۲). برای نمونه، اگر متغیرهای تصادفی Toothache، Cavity و Weather باشند، آنگاه 2×2×4=16 جهانِ ممکن وجود دارد. علاوه بر این، صدقِ هر گزارهٔ معین در چنین جهان‌هایی به‌آسانی با همان محاسبهٔ بازگشتیِ صدق که برای منطقِ گزاره‌ای استفاده کردیم تعیین می‌شود (صفحهٔ ۲۱۸ را ببینید).

توجه کنید که برخی متغیرهای تصادفی ممکن است زائد باشند، به این معنا که مقادیرشان در همهٔ موارد از مقادیرِ متغیرهای دیگر قابلِ حصول است. برای نمونه، متغیرِ Doubles در جهانِ دوتاسی دقیقاً وقتی صادق است که Die1 = Die2. گنجاندنِ Doubles به‌عنوانِ یکی از متغیرهای تصادفی، علاوه بر Die1 و Die2، ظاهراً شمارِ جهان‌های ممکن را از ۳۶ به ۷۲ افزایش می‌دهد، اما البته دقیقاً نیمی از آن ۷۲ به‌لحاظِ منطقی ناممکن خواهند بود و احتمالِ ۰ خواهند داشت.

از تعریفِ پیشینِ جهان‌های ممکن نتیجه می‌شود که مدلِ احتمال کاملاً توسط «توزیعِ مشترک» برای همهٔ متغیرهای تصادفی تعیین می‌شود—«توزیعِ احتمالِ مشترکِ کامل» (Full Joint Probability Distribution) موسوم. برای نمونه، با Cavity، Toothache و Weather، توزیعِ مشترکِ کامل P(Cavity, Toothache, Weather) است. این توزیعِ مشترک می‌تواند به‌صورتِ جدولِ 2×2×4 با ۱۶ درایه بازنمایی شود. از آن‌جا که احتمالِ هر گزاره جمعی روی جهان‌های ممکن است، توزیعِ مشترکِ کامل اصولاً برای محاسبهٔ احتمالِ هر گزاره‌ای کافی است. نمونه‌هایی از چگونگیِ انجامِ این کار را در بخش ۱۲.۳ خواهیم دید.

۱۲.۲.۳ اصل‌های احتمال و معقول‌بودنِ آن‌ها

اصل‌های پایهٔ احتمال (معادله‌های (12.1) و (12.2)) روابطِ معینی را میان درجه‌های باوری که می‌توان به گزاره‌های مرتبطِ منطقی نسبت داد، مستلزم می‌شوند. برای نمونه، می‌توانیم رابطهٔ آشنا میان احتمالِ یک گزاره و احتمالِ نقیضش را استخراج کنیم:

$$
\begin{aligned}
P(\neg a) &= \sum_{\omega \in \neg a} P(\omega) && \text{[با معادلهٔ (12.2)]} \\
&= \sum_{\omega \in a} P(\omega) + \sum_{\omega \in \neg a} P(\omega) - \sum_{\omega \in a} P(\omega) \\
&= \sum_{\omega \in \Omega} P(\omega) - \sum_{\omega \in a} P(\omega) && \text{[گروه‌بندی دو جملهٔ نخست]} \\
&= 1 - P(a) && \text{[با (12.1) و (12.2)]}
\end{aligned}
$$

همچنین می‌توانیم فرمولِ شناخته‌شدهٔ احتمالِ فصلی را استخراج کنیم، که گاهی «اصلِ شمول–اخراج» (Inclusion–Exclusion Principle) نامیده می‌شود:

$$P(a \vee b) = P(a) + P(b) - P(a \wedge b) \qquad (12.5)$$

این قاعده به‌آسانی به یاد می‌ماند با یادآوری اینکه حالت‌هایی که a در آن‌ها برقرار است، همراه با حالت‌هایی که b در آن‌ها برقرار است، قطعاً همهٔ حالت‌هایی را که a ∨ b در آن‌ها برقرار است می‌پوشانند؛ اما جمعِ دو مجموعهٔ حالت‌ها اشتراک‌شان را دو بار می‌شمارد، پس باید P(a∧b) را کم کنیم.

معادله‌های (12.1) و (12.5) اغلب «اصل‌های کولموگوروف» (Kolmogorov's Axioms) نامیده می‌شوند به افتخارِ ریاضی‌دانِ آندری کولموگوروف، که نشان داد چگونه بقیهٔ نظریهٔ احتمال را از این شالودهٔ ساده بسازیم و چگونه با دشواری‌های ناشی از متغیرهای پیوسته کنار بیاییم.[4] در حالی که معادلهٔ (12.2) طعمِ تعریفی دارد، معادلهٔ (12.5) آشکار می‌کند اصل‌ها واقعاً درجه‌های باوری را که عامل می‌تواند دربارهٔ گزاره‌های مرتبطِ منطقی داشته باشد قید می‌زنند. این مشابه این واقعیت است که عاملِ منطقی نمی‌تواند هم‌زمان به A، B و ¬(A∧B) باور داشته باشد، زیرا هیچ جهانِ ممکنی وجود ندارد که هر سه در آن صادق باشند. اما با احتمال‌ها، گزاره‌ها مستقیماً به جهان اشاره نمی‌کنند، بلکه به حالتِ دانشِ خودِ عامل اشاره می‌کنند. پس چرا عامل نمی‌تواند مجموعهٔ باورهای زیر را داشته باشد (حتی اگر اصل‌های کولموگوروف را نقض کنند)؟

$$P(a) = 0.4 \qquad P(b) = 0.3 \qquad P(a \wedge b) = 0.0 \qquad P(a \vee b) = 0.8 \tag{12.6}$$

این نوع پرسش دهه‌ها موضوعِ بحثِ شدید میان کسانی بوده که از احتمال به‌عنوانِ تنها شکلِ مشروعِ درجهٔ باور طرفداری می‌کنند و کسانی که رویکردهای جایگزین را پیشنهاد می‌کنند.

یک استدلال برای اصل‌های احتمال، که نخستین بار در ۱۹۳۱ توسط برونو دو فینتی بیان شد (ترجمهٔ انگلیسی: de Finetti, 1993)، چنین است: اگر عاملی درجهٔ باوری به گزارهٔ a دارد، آنگاه عامل باید بتواند شانس‌هایی را بیان کند که در آن‌ها نسبت به شرط‌بندیِ موافق یا مخالفِ a بی‌تفاوت است.[5] آن را بازی‌ای میان دو عامل در نظر بگیرید: عامل ۱ می‌گوید «درجهٔ باورِ من به رخدادِ a برابر ۰.۴ است.» عامل ۲ پس از آن آزاد است انتخاب کند برای یا علیهِ a با سرمایه‌ای سازگار با درجهٔ باورِ بیان‌شده شرط ببندد. یعنی عامل ۲ می‌تواند شرطِ عامل ۱ را بپذیرد که a رخ خواهد داد، با پیشنهادِ ۶ دلار در برابرِ ۴ دلارِ عامل ۱. یا عامل ۲ می‌تواند شرطِ عامل ۱ را بپذیرد که ¬a رخ خواهد داد، با پیشنهادِ ۴ دلار در برابرِ ۶ دلارِ عامل ۱. سپس پیامدِ a را مشاهده می‌کنیم و هر که درست گفته باشد پول را جمع می‌کند. اگر درجه‌های باورِ فرد به‌درستیِ جهان را بازتاب ندهند، انتظار می‌رود در درازمدت از عاملی رقیب که باورهایش حالتِ جهان را دقیق‌تر بازتاب می‌دهند پول از دست بدهد.

قضیهٔ دو فینتی به انتخابِ مقادیرِ درست برای احتمال‌های فردی مربوط نیست، بلکه به انتخابِ مقادیر برای احتمال‌های گزاره‌های مرتبطِ منطقی مربوط است: اگر عامل ۱ مجموعه‌ای از درجه‌های باور بیان کند که اصل‌های نظریهٔ احتمال را نقض کنند، آنگاه ترکیبی از شرط‌بندی‌ها توسط عامل ۲ وجود دارد که تضمین می‌کند عامل ۱ هر بار پول از دست بدهد. برای نمونه، فرض کنید عامل ۱ مجموعهٔ درجه‌های باور از معادلهٔ (12.6) را دارد. شکل ۱۲.۲ نشان می‌دهد اگر عامل ۲ انتخاب کند ۴ دلار روی ۳ ،a دلار روی b و ۲ دلار روی ¬(a∨b) شرط ببندد، آنگاه عامل ۱ همیشه پول از دست می‌دهد، صرف‌نظر از پیامدهای a و b. قضیهٔ دو فینتی دلالت دارد هیچ عاملِ عقلایی نمی‌تواند باورهایی داشته باشد که اصل‌های احتمال را نقض کنند.

گزاره	باورِ عامل ۱	شرط‌بندیِ عامل ۱	شرط‌بندیِ عامل ۲	سودِ عامل ۱ برای هر پیامد a,b	a,¬b	a,b¬	a,¬b¬
a	0.4	۴ دلار روی a	۶ دلار روی ¬a	-۶	-۶	۴	۴
b	0.3	۳ دلار روی b	۷ دلار روی ¬b	-۷	۳	-۷	۳
a ∨ b	0.8	۲ دلار روی ¬(a∨b)	۸ دلار روی a∨b	۲	۲	۲	-۸
جمع				**-۱۱**	**-۱**	**-۱**	**-۱**

شکل ۱۲.۲ – چون عامل ۱ باورهای ناسازگار دارد، عامل ۲ قادر است مجموعه‌ای از سه شرط طراحی کند که زیانِ عامل ۱ را تضمین می‌کند، مهم نیست پیامدِ a و b چه باشد.

یک ایرادِ رایج به قضیهٔ دو فینتی این است که این بازیِ شرط‌بندی نسبتاً تصنعی است. مثلاً اگر کسی از شرط‌بندی امتناع کند چه؟ آیا این بحث را پایان می‌دهد؟ پاسخ این است که بازیِ شرط‌بندی مدلی انتزاعی برای وضعیتِ تصمیم‌گیری است که هر عاملی در هر لحظه ناگزیر در آن درگیر است. هر کنش (شامل بی‌عملی) نوعی شرط است و هر پیامد را می‌توان سودِ شرط دید. امتناع از شرط‌بندی مانند امتناع از گذشتنِ زمان است. استدلال‌های فلسفیِ قویِ دیگری نیز برای استفاده از احتمال‌ها مطرح شده‌اند، که برجسته‌ترین‌شان از کاکس (1946)، کارناپ (1950) و جینز (2003) است. هر یک مجموعه‌ای از اصل‌ها برای استدلال با درجه‌های باور می‌سازند: نبودِ تناقض، مطابقت با منطقِ معمولی (برای نمونه، اگر باور به A بالا رود، باور به ¬A باید پایین بیاید)، و مانند آن. تنها اصلِ بحث‌برانگیز این است که درجه‌های باور باید اعداد باشند، یا دست‌کم مانندِ اعداد عمل کنند از این نظر که باید ترایایی (Transitive) داشته باشند (اگر باور به A از باور به B بیشتر است، که از باور به C بیشتر است، آنگاه باور به A باید از باور به C بیشتر باشد) و قابلِ مقایسه باشند (باور به A باید یکی از برابر، بیشتر یا کمتر از باور به B باشد). سپس می‌توان اثبات کرد که احتمال تنها رویکردی است که این اصل‌ها را ارضا می‌کند.

با این حال، دنیا همان‌طور که هست، گاهی نمایش‌های عملی بلندتر از اثبات‌ها سخن می‌گویند. موفقیتِ سامانه‌های استدلالِ مبتنی بر نظریهٔ احتمال در جذبِ پیرو بسیار مؤثرتر از استدلال‌های فلسفی بوده. اکنون می‌بینیم چگونه می‌توان اصل‌ها را برای استنتاج به کار گرفت.

۱۲.۳ استنتاج با توزیع‌های مشترکِ کامل

در این بخش روشِ ساده‌ای برای «استنتاجِ احتمالاتی» (Probabilistic Inference)—یعنی محاسبهٔ احتمال‌های پسینی برای گزاره‌های «پرس‌وجو» (Query) با توجه به شواهدِ مشاهده‌شده—توصیف می‌کنیم. از توزیعِ مشترکِ کامل به‌عنوانِ «پایگاهِ دانشی» استفاده می‌کنیم که پاسخ‌های همهٔ پرسش‌ها ممکن است از آن استخراج شود. در طولِ مسیر چند

فنِ مفید برای دست‌کاریِ معادله‌های مربوط به احتمال‌ها نیز معرفی می‌کنیم. با مثالِ ساده‌ای شروع می‌کنیم: حوزه‌ای شامل فقط سه متغیرِ بولیِ Toothache، Cavity و Catch (پروبِ فلزیِ تیزِ دندان‌پزشک در دندانِ من گیر می‌کند). توزیعِ مشترکِ کامل جدولِ 2×2×2 مانند شکل ۱۲.۳ است:

	toothache		toothache¬	
	catch	catch¬	catch	catch¬
cavity	0.108	0.012	0.072	0.008
cavity¬	0.016	0.064	0.144	0.576

شکل ۱۲.۳ — توزیعِ مشترکِ کامل برای جهانِ Toothache، Cavity و Catch.

توجه کنید که احتمال‌های توزیعِ مشترک به ۱ جمع می‌شوند، چنان‌که اصل‌های احتمال اقتضا می‌کنند. همچنین توجه کنید معادلهٔ (12.2) راهی مستقیم برای محاسبهٔ احتمالِ هر گزاره‌ای، ساده یا پیچیده، به ما می‌دهد: به‌سادگی آن جهان‌های ممکنی را که گزاره در آن‌ها صادق است شناسایی کن و احتمال‌هایشان را جمع کن. برای نمونه، شش جهانِ ممکن وجود دارد که cavity ∨ toothache در آن‌ها برقرار است:

```
P(cavity ∨ toothache) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28
```

یک مسئلهٔ به‌ویژه رایج، استخراجِ توزیع روی زیرمجموعه‌ای از متغیرها یا متغیری تکی است. برای نمونه، جمع‌کردنِ درایه‌های ردیفِ نخست «احتمالِ حاشیه‌ای» (Marginal Probability) پوسیدگی را می‌دهد:[6]

```
P(cavity) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2
```

این فرایند «حاشیه‌ای‌سازی» (Marginalization)، یا «جمع‌کردن و خارج‌کردن» (Summing Out) نامیده می‌شود—زیرا احتمال‌ها را برای هر مقدارِ ممکنِ متغیرهای دیگر جمع می‌کنیم و بدین‌ترتیب آن‌ها را از معادله خارج می‌کنیم.

می‌توانیم قاعدهٔ عمومیِ حاشیه‌ای‌سازیِ زیر را برای هر مجموعهٔ متغیرِ Y و Z بنویسیم:

$$\mathbf{P}(Y) = \sum_z \mathbf{P}(Y, Z=z) \qquad (12.7)$$

که در آن $\sum_z$ روی همهٔ ترکیب‌های ممکنِ مقادیرِ مجموعهٔ متغیرِ Z جمع می‌زند. مثلِ همیشه می‌توانیم $\mathbf{P}(Y, Z=z)$ در این معادله را به‌اختصارِ $\mathbf{P}(Y, z)$ بنویسیم. برای مثالِ پوسیدگی، معادلهٔ (12.7) با معادلهٔ زیر متناظر است:

```
P(Cavity) = P(Cavity, toothache, catch) + P(Cavity, toothache, ¬catch)
P(Cavity, ¬toothache, catch) + P(Cavity, ¬toothache, ¬catch) +
⟨0.576 ,0.008⟩ + ⟨0.144 ,0.072⟩ + ⟨0.064 ,0.012⟩ + ⟨0.016 ,0.108⟩ =
⟨0.8 ,0.2⟩ =
```

با استفاده از قاعدهٔ ضرب (معادلهٔ (12.4))، می‌توانیم P(Y, z) در معادلهٔ (12.7) را با P(Y|z)P(z) جایگزین کنیم و قاعده‌ای به نام «شرطی‌کردن» (Conditioning) به دست آوریم:

```
P(Y) = ∑z P(Y|z) P(z)        (12.8)
```

حاشیه‌ای‌سازی و شرطی‌کردن قاعده‌های مفیدی برای انواعِ استنتاج‌های مربوط به عبارت‌های احتمال‌اند.

در بیشترِ موارد، به محاسبهٔ احتمال‌های شرطیِ برخی متغیرها، با توجه به شواهدی دربارهٔ متغیرهای دیگر، علاقه‌مندیم. احتمال‌های شرطی را می‌توان با نخست استفاده از معادلهٔ (12.3) برای به‌دست‌آوردنِ عبارتی بر حسبِ احتمال‌های نامشروط و سپس ارزیابیِ عبارت از توزیعِ مشترکِ کامل یافت. برای نمونه، می‌توانیم احتمالِ پوسیدگی، با توجه به شاهدِ دندان‌درد، را چنین محاسبه کنیم:

```
P(cavity | toothache) = P(cavity ∧ toothache) / P(toothache)
(0.064 + 0.016 + 0.012 + 0.108) / (0.012 + 0.108) =
0.6 =
```

فقط برای بررسی، می‌توانیم احتمالِ نبودِ پوسیدگی، با توجه به دندان‌درد، را نیز محاسبه کنیم:

```
P(¬cavity | toothache) = P(¬cavity ∧ toothache) / P(toothache)
(0.064 + 0.016 + 0.012 + 0.108) / (0.064 + 0.016) =
0.4 =
```

این دو مقدار به 1.0 جمع می‌شوند، چنان‌که باید. توجه کنید جملهٔ P(toothache) در مخرجِ هر دو محاسبه است. اگر متغیرِ Cavity بیش از دو مقدار داشت، برای همهٔ آن‌ها در مخرج بود. در واقع، می‌توان آن را «ثابتِ نرمال‌سازی» (Normalization Constant) برای توزیعِ P(Cavity|toothache) دید، که تضمین می‌کند به ۱ جمع شود. در سراسرِ فصل‌های مربوط به احتمال، از α برای نشان‌دادنِ چنین ثابت‌هایی استفاده می‌کنیم. با این نماد، می‌توانیم دو معادلهٔ پیشین را در یکی بنویسیم:

```
P(Cavity|toothache) = α P(Cavity, toothache)
α [P(Cavity, toothache, catch) + P(Cavity, toothache, ¬catch)] =
α [⟨0.108, 0.016⟩ + ⟨0.012, 0.064⟩] =
α ⟨0.12, 0.08⟩ = ⟨0.6, 0.4⟩ =
```

به بیانِ دیگر، می‌توانیم P(Cavity|toothache) را حتی اگر مقدارِ P(toothache) را ندانیم محاسبه کنیم! موقتاً عاملِ 1/P(toothache) را فراموش می‌کنیم و مقادیرِ cavity و ¬cavity را جمع می‌کنیم و ۰.۱۲ و ۰.۰۸ می‌گیریم. این‌ها نسبت‌های نسبیِ درست‌اند، اما به ۱ جمع نمی‌شوند، پس با تقسیمِ هر یک بر 0.12 + 0.08 نرمال‌سازیشان می‌کنیم و احتمال‌های درستِ ۰.۶ و ۰.۴ را می‌گیریم. نرمال‌سازی میان‌بری مفید در بسیاری از محاسباتِ احتمال است، هم برای آسان‌ترکردنِ محاسبه و هم برای اجازه‌دادن به پیشروی وقتی برخی برآوردهای احتمال (مانند P(toothache)) در دسترس نیستند.

از مثال می‌توانیم رویهٔ استنتاجِ عمومی‌ای استخراج کنیم. با حالتی شروع می‌کنیم که پرس‌وجو شامل متغیرِ تکیِ X است (در مثال، Cavity). فرض کنیم E فهرستِ «متغیرهای شاهد» باشد (در مثال، فقط Toothache)، e فهرستِ مقادیرِ مشاهده‌شدهٔ آن‌ها، و Y متغیرهای مشاهده‌نشدهٔ باقی‌مانده (در مثال، فقط Catch). پرس‌وجو P(X|e) است و می‌توان آن را چنین ارزیابی کرد:

$$\mathbf{P}(X|e) = \alpha\, \mathbf{P}(X, e) = \alpha \sum_y \mathbf{P}(X, e, y) \qquad (12.9)$$

که در آن جمع روی همهٔ yهای ممکن (یعنی همهٔ ترکیب‌های ممکنِ مقادیرِ متغیرهای مشاهده‌نشدهٔ Y) است. توجه کنید که متغیرهای X، E و Y با هم مجموعهٔ کاملِ متغیرهای حوزه را تشکیل می‌دهند، پس P(X, e, y) به‌سادگی زیرمجموعه‌ای از احتمال‌ها از توزیعِ مشترکِ کامل است. با توزیعِ مشترکِ کامل برای کار، معادلهٔ (12.9) می‌تواند پرس‌وجوهای احتمالاتی را برای متغیرهای گسسته پاسخ دهد. اما به‌خوبی مقیاس‌پذیر نیست: برای حوزه‌ای توصیف‌شده با n متغیرِ بولی، جدولِ ورودی با اندازهٔ $O(2^n)$ لازم دارد و $O(2^n)$ زمان برای پردازشِ جدول می‌برد. در مسئلهٔ واقع‌گرایانه به‌آسانی می‌توانیم n=100 داشته باشیم و $O(2^n)$ را غیرعملی کنیم—جدولی با $2^{100} \approx 10^{30}$ درایه! مسئله فقط حافظه و محاسبه نیست: مسئلهٔ واقعی این است که اگر قرار باشد هر یک از 10^{30} احتمالِ جداگانه از مثال‌ها برآورد شود، شمارِ مثال‌های لازم نجومی خواهد بود.

به این دلایل، توزیعِ مشترکِ کامل در شکلِ جدولی به‌ندرت ابزاری عملی برای ساختِ سامانه‌های استدلالی است. در عوض، باید آن را شالودهٔ نظری‌ای دید که رویکردهای مؤثرتر ممکن است روی آن ساخته شوند، همان‌گونه که جدول‌های صدق شالودهٔ نظریِ الگوریتم‌های عملی‌تری مانند DPLL در فصل ۷ تشکیل دادند. بقیهٔ این فصل برخی از ایده‌های پایهٔ لازم برای توسعهٔ سامانه‌های واقع‌گرایانه در فصل ۱۳ را معرفی می‌کند.

۱۲.۴ استقلال

توزیعِ مشترکِ کاملِ شکل ۱۲.۳ را با افزودنِ متغیرِ چهارمی به نام Weather بسط دهیم. توزیعِ مشترکِ کامل سپس P(Toothache, Catch, Cavity, Weather) می‌شود که 2×2×2×4=32 درایه دارد. شامل چهار «نسخه» از جدولِ شکل ۱۲.۳ است، یکی برای هر نوعِ آب‌وهوا. این نسخه‌ها چه رابطه‌ای با یکدیگر و با جدولِ سه‌متغیرهٔ اصلی دارند؟ مقدارِ P(toothache, catch, cavity, cloud) چگونه با مقدارِ P(toothache, catch, cavity) مرتبط است؟ می‌توانیم از قاعدهٔ ضرب (معادلهٔ (12.4)) استفاده کنیم:

```
P(toothache, catch, cavity, cloud) = P(cloud | toothache, catch, cavity) × P(toothache,
                                                                          catch, cavity)
```

حالا، مگر اینکه کسی در کارِ خدایی باشد، نباید تصور کند مشکلاتِ دندان‌پزشکی‌اش بر آب‌وهوا اثر می‌گذارد. و برای دندان‌پزشکیِ سربسته، دست‌کم، گفتنِ این امن به نظر می‌رسد که آب‌وهوا بر متغیرهای دندان‌پزشکی اثر نمی‌گذارد. بنابراین، اظهارِ زیر معقول به نظر می‌رسد:

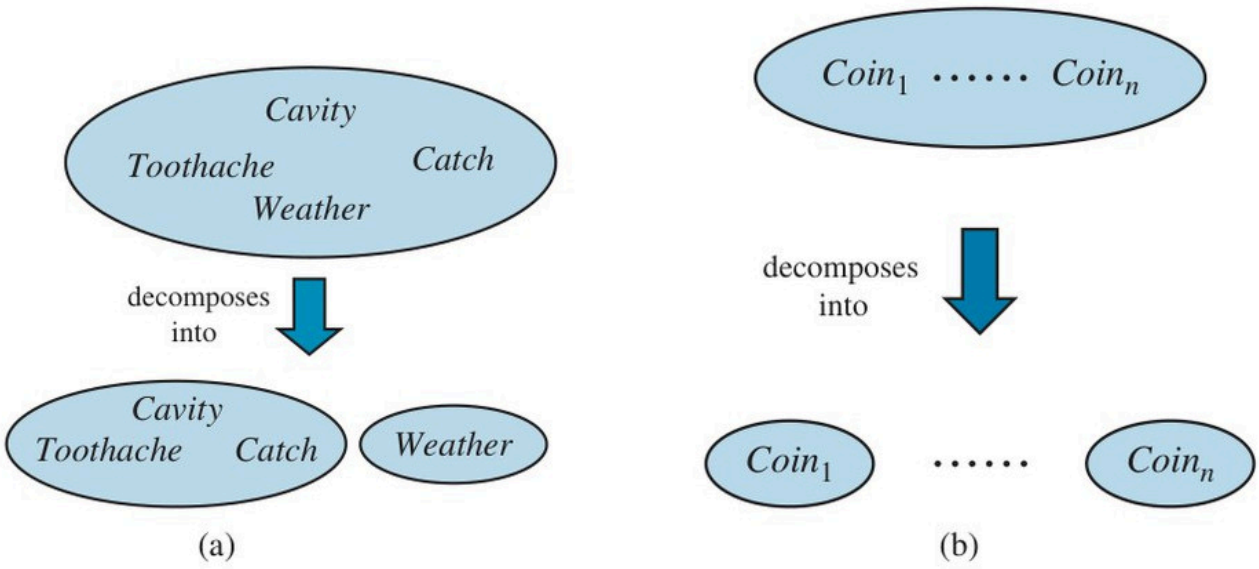
```
P(cloud | toothache, catch, cavity) = P(cloud)        (12.10)
```

از این، می‌توانیم نتیجه بگیریم:

```
P(toothache, catch, cavity, cloud) = P(cloud) × P(toothache, catch, cavity)
```

معادلهٔ مشابهی برای هر درایه از P(Toothache, Catch, Cavity, Weather) وجود دارد. در واقع، می‌توانیم معادلهٔ عمومیِ P(Weather) × P(Toothache, Catch, Cavity) = P(Toothache, Catch, Cavity, Weather) را بنویسیم.

بدین‌ترتیب، جدولِ ۳۲-عنصری برای چهار متغیر را می‌توان از یک جدولِ ۸-عنصری و یک جدولِ ۴-عنصری ساخت. این تجزیه به‌صورتِ شماتیک در شکل ۱۲.۴(الف) نشان داده شده است.



شکل ۱۲.۴ – دو نمونه از تجزیهٔ توزیعِ مشترکِ بزرگ به توزیع‌های مشترکِ کوچک‌تر با استفاده از استقلالِ مطلق. (الف) آب‌وهوا و مشکلاتِ دندان‌پزشکی مستقل‌اند. (ب) پرتاب‌های سکه مستقل‌اند.

ویژگیِ مورداستفاده در معادلهٔ (12.10) «استقلال» (Independence) نامیده می‌شود (همچنین استقلالِ حاشیه‌ای و استقلالِ مطلق). به‌طور خاص، آب‌وهوا از مشکلاتِ دندان‌پزشکیِ فرد مستقل است. استقلال میان گزاره‌های a و b را می‌توان چنین نوشت:

$$P(a \mid b) = P(a) \quad \text{یا} \quad P(b \mid a) = P(b) \quad \text{یا} \quad P(a \wedge b) = P(a)P(b) \qquad (12.11)$$

همهٔ این شکل‌ها معادل‌اند (تمرین INDI.۱۲). استقلال میان متغیرهای X و Y را می‌توان چنین نوشت (باز، همه معادل‌اند):

$$P(X \mid Y) = P(X) \quad \text{یا} \quad P(Y \mid X) = P(Y) \quad \text{یا} \quad P(X, Y) = P(X)P(Y)$$

اظهارهای استقلال معمولاً بر دانشِ حوزه مبتنی‌اند. همان‌طور که مثالِ دندان‌درد–آب‌وهوا نشان می‌دهد، می‌توانند میزانِ اطلاعاتِ لازم برای مشخص‌کردنِ توزیعِ مشترکِ کامل را به‌طور چشم‌گیری کاهش دهند. اگر مجموعهٔ کاملِ متغیرها را بتوان به زیرمجموعه‌های مستقل تقسیم کرد، آنگاه توزیعِ مشترکِ کامل را می‌توان به توزیع‌های مشترکِ جداگانه روی آن زیرمجموعه‌ها تجزیه کرد. برای نمونه، توزیعِ مشترکِ کامل روی پیامدِ n پرتابِ سکهٔ مستقل، P(C1,...,Cn)، دارای 2^n درایه است، اما می‌توان آن را به‌صورتِ حاصل‌ضربِ n توزیعِ تک‌متغیرهٔ P(Ci) بازنمایی کرد. در نگاهی عمل‌گراتر، استقلالِ دندان‌پزشکی و هواشناسی چیزِ خوبی است، زیرا در غیر این صورت ممکن بود عملِ دندان‌پزشکی دانشِ نزدیک از هواشناسی بخواهد، و برعکس.

پس وقتی در دسترس باشند، اظهارهای استقلال می‌توانند به کاهشِ اندازهٔ بازنماییِ حوزه و پیچیدگیِ مسئلهٔ استنتاج کمک کنند. متأسفانه، جداسازیِ تمیزِ مجموعه‌های کاملِ متغیرها با استقلال بسیار نادر است. هرگاه ارتباطی، هرچند غیرمستقیم، میان دو متغیر وجود داشته باشد، استقلال برقرار نخواهد بود. علاوه بر این، حتی زیرمجموعه‌های مستقل می‌توانند بسیار بزرگ باشند—برای نمونه، دندان‌پزشکی ممکن است شامل ده‌ها بیماری و صدها نشانه باشد که همه به هم مرتبط‌اند. برای مدیریتِ چنین مسائلی، به روش‌های ظریف‌تری از مفهومِ سرراستِ استقلال نیاز داریم.

۱۲.۵ قاعدهٔ بیز و کاربرد آن

در صفحهٔ ۳۹۰ قاعدهٔ ضرب (معادلهٔ (12.4)) را تعریف کردیم. در واقع می‌توان آن را به دو شکل نوشت: $P(a \wedge b) = P(a|b)P(b)$ و $P(a \wedge b) = P(b|a)P(a)$. با برابر گذاشتنِ دو سمتِ راست و تقسیم بر P(a)، داریم:

$$P(b \mid a) = P(a \mid b)P(b) / P(a) \qquad (12.12)$$

این معادله «قاعدهٔ بیز» (Bayes' Rule) شناخته می‌شود (همچنین قانونِ بیز یا قضیهٔ بیز). این معادلهٔ سادهٔ قاعدهٔ بیز زیربنای بیشترِ سامانه‌های هوش مصنوعیِ مدرن برای استنتاجِ احتمالاتی است.

حالتِ عمومی‌ترِ قاعدهٔ بیز برای متغیرهای چندمقداره را می‌توان با نمادِ P چنین نوشت:

$$P(Y \mid X) = P(X \mid Y)P(Y) / P(X)$$

همانندِ قبل، این به‌عنوانِ نمایشِ مجموعه‌ای از معادله‌ها در نظر گرفته می‌شود که هر یک با مقادیرِ خاصِ متغیرها سروکار دارد. همچنین به نسخهٔ عمومی‌تری که روی برخی شواهدِ زمینه‌ایِ e شرطی شده نیاز خواهیم داشت:

$$P(Y|X, e) = P(X|Y, e)P(Y|e) / P(X|e) \qquad (12.13)$$

۱۲.۵.۱ به‌کاربردنِ قاعدهٔ بیز: حالتِ ساده

در ظاهر، قاعدهٔ بیز چندان مفید به نظر نمی‌رسد. به ما اجازه می‌دهد جملهٔ تکیِ P(b|a) را بر حسبِ سه جمله محاسبه کنیم: P(b)، P(a|b) و P(a). این به نظر دو قدمِ عقب‌رفتن است؛ اما قاعدهٔ بیز در عمل مفید است، زیرا مواردِ بسیاری وجود دارد که برآوردهای احتمالِ خوبی برای این سه عدد داریم و لازم است عددِ چهارم را محاسبه کنیم.

اغلب، اثرِ علتِ ناشناخته‌ای را به‌عنوانِ شاهد درک می‌کنیم و می‌خواهیم آن علت را تعیین کنیم. در آن صورت، قاعدهٔ بیز می‌شود:

$$P(cause|effect) = P(effect|cause)P(cause) / P(effect)$$

احتمالِ شرطیِ P(effect|cause) رابطه را در «جهتِ علّی» (Causal Direction) کمّی می‌کند، در حالی که P(cause|effect) «جهتِ تشخیصی» (Diagnostic Direction) را توصیف می‌کند. در مسئله‌ای مانند تشخیصِ پزشکی، اغلب احتمال‌های شرطی روی روابطِ علّی داریم. پزشک P(symptoms|disease) را می‌داند و می‌خواهد تشخیصِ P(disease|symptoms) را استخراج کند.

برای نمونه، پزشکی می‌داند بیماریِ مننژیت باعث می‌شود بیمار ۷۰٪ مواقع گردنِ خشک داشته باشد. پزشک همچنین برخی واقعیت‌های نامشروط را می‌داند: احتمالِ پیشینیِ اینکه هر بیمار مننژیت داشته باشد 1/50000 است، و احتمالِ پیشینیِ اینکه هر بیمار گردنِ خشک داشته باشد 1٪ است. با s گزارهٔ «بیمار گردنِ خشک دارد» و m گزارهٔ «بیمار مننژیت دارد»:

$$P(s|m) = 0.7$$
$$P(m) = 1/50000$$
$$P(s) = 0.01$$

$$P(m|s) = P(s|m)P(m) / P(s) = (0.7 \times 1/50000) / 0.01 = 0.0014 \qquad (12.14)$$

یعنی انتظار داریم فقط ۰.۱۴٪ بیمارانِ با گردنِ خشک مننژیت داشته باشند. توجه کنید که حتی اگرچه گردنِ خشک به‌شدت توسط مننژیت نشان داده می‌شود (با احتمالِ ۰.۷)، احتمالِ مننژیت در بیمارانِ با گردنِ خشک کوچک می‌ماند. این به این دلیل است که احتمالِ پیشینیِ گردنِ خشک (از هر علتی) بسیار بالاتر از پیشینِ مننژیت است.

بخش ۱۲.۳ فرایندی را نشان داد که با آن می‌توان از ارزیابیِ احتمالِ پیشینیِ شاهد (اینجا P(s)) اجتناب کرد، با محاسبهٔ احتمالِ پسینی برای هر مقدارِ متغیرِ پرس‌وجو (اینجا m و ¬m) و سپس نرمال‌سازیِ نتایج. همان فرایند را می‌توان هنگام استفاده از قاعدهٔ بیز به کار برد. داریم:

```
P(M|s) = α ⟨P(s|m)P(m), P(s|¬m)P(¬m)⟩
```

بدین‌ترتیب، برای استفاده از این رویکرد باید P(s|¬m) را به‌جایِ P(s) برآورد کنیم. ناهارِ مجانی وجود ندارد—گاهی این آسان‌تر است، گاهی دشوارتر. شکلِ عمومیِ قاعدهٔ بیز با نرمال‌سازی چنین است:

```
P(Y|X) = α P(X|Y)P(Y)          (12.15)
```

که در آن α ثابتِ نرمال‌سازیِ لازم برای جمع‌شدنِ درایه‌های P(Y|X) به ۱ است.

یک پرسشِ بدیهی دربارهٔ قاعدهٔ بیز این است که چرا ممکن است احتمالِ شرطی را در یک جهت در دسترس داشته باشیم، اما در جهتِ دیگر نه. در حوزهٔ مننژیت، شاید پزشک بداند گردنِ خشک در ۱ از ۵۰۰۰ مورد به مننژیت دلالت می‌کند؛ یعنی پزشک اطلاعاتِ کمّی در جهتِ تشخیصی از نشانه‌ها به علت‌ها دارد. چنین پزشکی نیازی به استفاده از قاعدهٔ بیز ندارد.

متأسفانه، دانشِ تشخیصی اغلب شکننده‌تر از دانشِ علّی است. اگر ناگهان همه‌گیریِ مننژیت رخ دهد، احتمالِ نامشروطِ مننژیت، P(m)، بالا می‌رود. پزشکی که احتمالِ تشخیصیِ P(m|s) را مستقیماً از مشاهدهٔ آماریِ بیمارانِ پیش از همه‌گیری به دست آورده، هیچ ایده‌ای نخواهد داشت چگونه مقدار را به‌روزرسانی کند، اما پزشکی که P(m|s) را از سه مقدارِ دیگر محاسبه می‌کند خواهد دید P(m|s) باید متناسب با P(m) بالا برود. از همه مهم‌تر، اطلاعاتِ علّیِ P(s|m) از همه‌گیری متأثر نمی‌شود، زیرا صرفاً شیوهٔ کارِ مننژیت را بازتاب می‌دهد. استفاده از این نوع دانشِ علّیِ مستقیم یا مبتنی بر مدل، استحکامِ حیاتیِ لازم برای ممکن‌شدنِ سامانه‌های احتمالاتی در دنیای واقعی را فراهم می‌کند.

۱۲.۵.۲ استفاده از قاعدهٔ بیز: ترکیبِ شواهد

دیده‌ایم قاعدهٔ بیز می‌تواند برای پاسخ به پرس‌وجوهای احتمالاتیِ شرطی‌شده روی یک قطعه شاهد مفید باشد—برای نمونه، گردنِ خشک. به‌طور خاص، استدلال کردیم اطلاعاتِ احتمالاتی اغلب به شکلِ P(effect|cause) در دسترس است. وقتی دو یا چند قطعه شاهد داشته باشیم چه اتفاقی می‌افتد؟ برای نمونه، دندان‌پزشک چه می‌تواند نتیجه بگیرد اگر پروبِ فلزیِ تیزِ او در دندانِ دردناکِ بیمار گیر کند؟ اگر توزیعِ مشترکِ کامل (شکل ۱۲.۳) را بدانیم، می‌توانیم پاسخ را مستقیم بخوانیم:

```
P(Cavity | toothache ∧ catch) = α ⟨0.108, 0.016⟩ ≈ ⟨0.871, 0.129⟩
```

اما می‌دانیم چنین رویکردی به شمارِ بزرگ‌ترِ متغیرها مقیاس‌پذیر نیست. می‌توانیم با قاعدهٔ بیز مسئله را بازصورت‌بندی کنیم:

$$P(Cavity \mid toothache \wedge catch) = \alpha\, P(toothache \wedge catch \mid Cavity)\, P(Cavity) \qquad (12.16)$$

برای کارکردنِ این بازصورت‌بندی، باید احتمال‌های شرطیِ عطفِ toothache ∧ catch را برای هر مقدارِ Cavity بدانیم. این ممکن است برای فقط دو متغیرِ شاهد شدنی باشد، اما باز مقیاس‌پذیر نیست. اگر n متغیرِ شاهدِ ممکن (اشعهٔ ایکس، رژیمِ غذایی، بهداشتِ دهان و...) باشد، آنگاه $O(2^n)$ ترکیبِ ممکن از مقادیرِ مشاهده‌شده وجود دارد که برای‌شان باید احتمال‌های شرطی بدانیم. این بهتر از استفاده از توزیعِ مشترکِ کامل نیست. برای پیشرفت، باید برخی اظهارهای اضافی دربارهٔ حوزه بیابیم که ما را قادر کند عبارت‌ها را ساده کنیم. مفهومِ استقلال در بخش ۱۲.۴ سرنخی فراهم می‌کند، اما نیاز به پالایش دارد. خوب بود اگر Toothache و Catch مستقل بودند، اما نیستند: اگر پروب در دندان گیر کند، محتمل است دندان پوسیدگی داشته باشد و پوسیدگی دندان‌درد ایجاد کند. اما این متغیرها با توجه به حضور یا غیابِ پوسیدگی مستقل‌اند. هر یک مستقیماً توسط پوسیدگی ایجاد می‌شود، اما هیچ‌یک اثرِ مستقیمی روی دیگری ندارد: دندان‌درد به حالتِ اعصابِ دندان بستگی دارد، در حالی که دقتِ پروب عمدتاً به مهارتِ دندان‌پزشک بستگی دارد، که دندان‌درد برایش نامربوط است.[7] از نظرِ ریاضی، این ویژگی چنین نوشته می‌شود:

$$P(toothache \wedge catch \mid Cavity) = P(toothache \mid Cavity)\, P(catch \mid Cavity) \qquad (12.17)$$

این معادله «استقلالِ شرطی» (Conditional Independence) دندان‌درد و گیرکردنِ پروب با توجه به پوسیدگی را بیان می‌کند. می‌توانیم آن را در معادلهٔ (12.16) جای کنیم تا احتمالِ پوسیدگی را به دست آوریم:

$$P(Cavity \mid toothache \wedge catch) = \alpha\, P(toothache \mid Cavity)\, P(catch \mid Cavity)\, P(Cavity) \qquad (12.18)$$

حالا نیازهای اطلاعاتی همانندِ استنتاج با استفاده از هر قطعه شاهدِ جداگانه‌اند: احتمالِ پیشینیِ P(Cavity) برای متغیرِ پرس‌وجو و احتمالِ شرطیِ هر اثر، به‌شرطِ علتش.

تعریفِ عمومیِ استقلالِ شرطیِ دو متغیرِ X و Y، با توجه به متغیرِ سومِ Z، چنین است:

$$P(X, Y \mid Z) = P(X \mid Z)\, P(Y \mid Z)$$

در حوزهٔ دندان‌پزشکی، برای نمونه، اظهارِ استقلالِ شرطیِ متغیرهای Toothache و Catch با توجه به Cavity معقول به نظر می‌رسد:

$$P(Toothache, Catch \mid Cavity) = P(Toothache \mid Cavity)\, P(Catch \mid Cavity) \qquad (12.19)$$

توجه کنید این اظهار تا حدی قوی‌تر از معادلهٔ (12.17) است، که استقلال را فقط برای مقادیرِ خاصِ Toothache و Catch اظهار می‌کند. همانندِ استقلالِ مطلق در معادلهٔ (12.11)، شکل‌های معادلِ P(X|Y, Z) = P(X|Z) و P(Y|X, Z) = P(Y|Z) نیز می‌توانند استفاده شوند (تمرین PXYZ.۱۲ را ببینید). بخش ۱۲.۴ نشان داد اظهارهای استقلالِ مطلق تجزیهٔ توزیعِ مشترکِ کامل را به قطعاتِ بسیار کوچک‌تر مجاز می‌کنند. معلوم می‌شود برای اظهارهای استقلالِ شرطی نیز همین صادق است. برای نمونه، با اظهارِ معادلهٔ (12.19)، می‌توانیم تجزیه را چنین به دست آوریم:

```
P(Toothache, Catch, Cavity) = P(Toothache, Catch|Cavity) P(Cavity)   (قاعدهٔ ضرب)
= P(Toothache|Cavity) P(Catch|Cavity) P(Cavity)   (با 12.19)
```

(خواننده می‌تواند به‌آسانی بررسی کند این معادله در واقع در شکل ۱۲.۳ برقرار است.) بدین‌ترتیب، جدولِ بزرگِ اصلی به سه جدولِ کوچک‌تر تجزیه می‌شود. جدولِ اصلی ۷ عددِ مستقل دارد. (جدول $8=2^3$ درایه دارد، اما باید به ۱ جمع شوند، پس ۷ تای‌شان مستقل‌اند.) جدول‌های کوچک‌تر در مجموع 5=1+2+2 عددِ مستقل دارند. (برای توزیعِ احتمالِ شرطی مانند P(Toothache|Cavity) دو ردیفِ دونمایی هست و هر ردیف به ۱ جمع می‌شود، پس دو عددِ مستقل است؛ برای توزیعِ پیشینی مانند P(Cavity) فقط یک عددِ مستقل هست.) رفتن از ۷ به ۵ شاید پیروزیِ بزرگی به نظر نرسد، اما با شمارِ بزرگ‌ترِ نشانه‌ها می‌تواند سودها بسیار بزرگ‌تر باشد.

به‌طور کلی، برای n نشانه که همگی با توجه به Cavity به‌صورتِ شرطی مستقل‌اند، اندازهٔ بازنمایی به‌صورتِ O(n) رشد می‌کند به‌جایِ $O(2^n)$. این بدان معناست اظهارهای استقلالِ شرطی می‌توانند به مقیاس‌پذیریِ سامانه‌های احتمالاتی اجازه دهند؛ علاوه بر این، آن‌ها بسیار رایج‌تر از اظهارهای استقلالِ مطلق در دسترس‌اند. از نظرِ مفهومی، Cavity «جداسازی» (Separation) می‌کند Toothache و Catch را، زیرا علتِ مستقیمِ هر دوی آن‌هاست. تجزیهٔ حوزه‌های احتمالاتیِ بزرگ به زیرمجموعه‌های اتصال-ضعیف از طریقِ استقلالِ شرطی یکی از مهم‌ترین پیشرفت‌ها در تاریخِ اخیرِ هوش مصنوعی است.

۱۲.۶ مدل‌های بیزِ ساده

مثالِ دندان‌پزشکی الگویِ رایجی را نشان می‌دهد که در آن علتِ تکی مستقیماً بر شماری از اثرها اثر می‌گذارد که همگی با توجه به علت به‌صورتِ شرطی مستقل‌اند. توزیعِ مشترکِ کامل را می‌توان چنین نوشت:

$$\mathbf{P}(Cause, Effect_1, \ldots, Effect_n) = \mathbf{P}(Cause)\prod_i \mathbf{P}(Effect_i \mid Cause) \qquad (12.20)$$

چنین توزیعِ احتمالی «مدلِ بیزِ ساده» (Naive Bayes Model) نامیده می‌شود—«ساده» از آن رو که اغلب (به‌عنوانِ فرضِ ساده‌کننده) در مواردی استفاده می‌شود که متغیرهای «اثر» با توجه به متغیرِ علت به‌صورتِ اکیداً مستقل نیستند. (مدلِ بیزِ ساده گاهی «رده‌بندِ بیزی» (Bayesian Classifier) نیز نامیده می‌شود، کاربردی تا حدی شلخته که بیزیانِ واقعی را بر آن داشته آن را «مدلِ بیزِ احمق» (Idiot Bayes Model) بنامند.) در عمل، سامانه‌های بیزِ ساده اغلب بسیار خوب کار می‌کنند، حتی وقتی فرضِ استقلالِ شرطی به‌صورتِ دقیقاً صادق نباشد.

برای استفاده از مدلِ بیزِ ساده، می‌توانیم معادلهٔ (12.20) را برای به‌دست‌آوردنِ احتمالِ علت با توجه به برخی اثرهای مشاهده‌شده به کار ببریم. اثرهای مشاهده‌شده را E=e بنامیم، در حالی که متغیرهای اثرِ باقی‌ماندهٔ Y مشاهده‌نشده‌اند. سپس روشِ استانداردِ استنتاج از توزیعِ مشترک (معادلهٔ (12.9)) قابلِ اعمال است:

```
P(Cause|e) = α ∑y P(Cause, e, y)
```

از معادلهٔ (12.20)، سپس به دست می‌آوریم:

```
P(Cause|e) = α ∑y P(Cause) P(y|Cause) (∏j P(ej|Cause))
α P(Cause) (∏j P(ej|Cause)) ∑y P(y|Cause) =
α P(Cause) ∏j P(ej|Cause)        (12.21) =
```

که خطِ آخر به این دلیل است که جمع روی y برابر ۱ است. بازتفسیرِ این معادله با کلمات: برای هر علتِ ممکن، احتمالِ پیشینیِ علت را در حاصل‌ضربِ احتمال‌های شرطیِ اثرهای مشاهده‌شده به‌شرطِ علت ضرب کن؛ سپس نتیجه را نرمال‌سازی کن. زمانِ اجرای این محاسبه در شمارِ اثرهای مشاهده‌شده خطی است و به شمارِ اثرهای مشاهده‌نشده (که ممکن است در حوزه‌هایی مانند پزشکی بسیار بزرگ باشد) بستگی ندارد. در فصلِ بعد خواهیم دید این پدیده‌ای رایج در استنتاجِ احتمالاتی است: متغیرهای شاهد که مقادیرشان مشاهده نشده معمولاً «ناپدید می‌شوند» از محاسبه به‌طورِ کامل.

۱۲.۶.۱ رده‌بندیِ متن با بیزِ ساده

ببینیم مدلِ بیزِ ساده چگونه می‌تواند برای مسئلهٔ «رده‌بندیِ متن» (Text Classification) استفاده شود: با توجه به متنی، تعیین کن به کدام‌یک از مجموعهٔ ازپیش‌تعریف‌شدهٔ رده‌ها یا دسته‌ها تعلق دارد. در این‌جا «علت» متغیرِ Category است و متغیرهای «اثر» حضور یا غیابِ کلماتِ کلیدیِ معین، HasWordi هستند. این دو جملهٔ نمونه را در نظر بگیرید، برگرفته از مقاله‌های روزنامه:

1. سهام دوشنبه رشد کرد، با شاخص‌های اصلی ۱٪ افزایش، در حالی که خوش‌بینی بر فصلِ سودِ سه‌ماههٔ نخست ادامه داشت.
2. بارانِ سنگین به کوبیدنِ بیشترِ سواحلِ شرقی دوشنبه ادامه داد، با هشدارِ سیل در شهرِ نیویورک و مکان‌های دیگر.

هدف، رده‌بندیِ هر جمله در یک Category—بخش‌های اصلیِ روزنامه: خبر، ورزش، تجارت، آب‌وهوا یا سرگرمی—است. مدلِ بیزِ ساده از احتمال‌های پیشینیِ P(Category) و احتمال‌های شرطیِ P(HasWordi | Category) تشکیل شده.

برای هر ردهٔ c، P(Category=c) به‌صورتِ کسری از همهٔ اسنادِ دیده‌شدهٔ پیشین که از ردهٔ c هستند برآورد می‌شود. برای نمونه، اگر ۹٪ مقاله‌ها دربارهٔ آب‌وهوا باشند، P(Category=weather)=0.09 می‌گذاریم. به‌طور مشابه، P(HasWordi | Category) به‌صورتِ کسری از اسنادِ هر رده که حاوی واژهٔ i هستند برآورد می‌شود؛ شاید ۳۷٪ مقاله‌های مربوط به تجارت حاوی واژهٔ ۶، «سهام»، باشند، پس P(HasWord6=true | Category=business) برابر

0.37 گذاشته می‌شود.[8] برای رده‌بندیِ سندِ تازه، بررسی می‌کنیم کدام واژه‌های کلیدی در سند ظاهر می‌شوند و سپس معادلهٔ (12.21) را برای به‌دست‌آوردنِ توزیعِ احتمالِ پسینی روی رده‌ها اعمال می‌کنیم. اگر باید فقط یک رده را پیش‌بینی کنیم، ردهٔ با بالاترین احتمالِ پسینی را می‌گیریم.

توجه کنید برای این مسئله، هر متغیرِ اثر مشاهده شده است، زیرا همیشه می‌توانیم بگوییم آیا واژهٔ معینی در سند ظاهر می‌شود یا نه. باید مراقب باشیم احتمالِ صفر به واژه‌هایی که قبلاً در ردهٔ معینی از اسناد دیده نشده‌اند تخصیص ندهیم، زیرا صفر همهٔ شواهدِ دیگر در معادلهٔ (12.21) را از بین می‌برد. صرفاً چون واژه‌ای را ندیده‌ای به این معنا نیست که هرگز نخواهی دید. در عوض، بخشِ کوچکی از توزیعِ احتمال را برای بازنماییِ واژه‌های «پیش‌تر دیده‌نشده» ذخیره کن. برای اطلاعاتِ بیشتر دربارهٔ این مسئله به‌طورِ کلی فصل ۲۰، و برای موردِ خاصِ مدل‌های واژه بخش ۲۳.۱.۴ را ببینید.

مدلِ بیزِ ساده فرض می‌کند واژه‌ها به‌صورتِ مستقل در اسناد رخ می‌دهند، با بسامدهایی که توسط ردهٔ سند تعیین می‌شوند. این فرضِ استقلال بدیهی‌اً در عمل نقض می‌شود. برای نمونه، عبارتِ «سه‌ماههٔ نخست» در مقاله‌های تجارت (یا ورزش) بیشتر از آنچه با ضربِ احتمال‌های «نخست» و «سه‌ماهه» پیشنهاد می‌شود رخ می‌دهد. نقضِ استقلال معمولاً به این معناست که احتمال‌های پسینیِ نهایی بسیار نزدیک‌تر از آنچه باید باشند به ۱ یا ۰ خواهند بود؛ به بیانِ دیگر، مدل در پیش‌بینی‌هایش بیش‌ازحد مطمئن است. از سوی دیگر، حتی با این خطاها، رتبه‌بندیِ رده‌های ممکن اغلب کاملاً دقیق است.

مدل‌های بیزِ ساده به‌طورِ گسترده برای تعیینِ زبان، بازیابیِ سند، پالایشِ هرزنامه و دیگر مسائلِ رده‌بندی استفاده می‌شوند. برای مسائلی مانند تشخیصِ پزشکی، که مقادیرِ واقعیِ احتمال‌های پسینی واقعاً اهمیت دارند—برای نمونه، در تصمیم‌گیری دربارهٔ انجامِ آپاندکتومی—معمولاً ترجیح می‌دهیم از مدل‌های پیچیده‌ترِ توصیف‌شده در فصلِ بعد استفاده کنیم.

۱۲.۷ جهانِ وومپوس، بازبینی

می‌توانیم ایده‌های این فصل را برای حلِ مسائلِ استدلالِ احتمالاتی در جهانِ وومپوس ترکیب کنیم. (توصیفِ کاملِ جهانِ وومپوس در فصل ۷ آمده.) عدم قطعیت در جهانِ وومپوس از آن‌جا ناشی می‌شود که حسگرهای عامل فقط اطلاعاتِ جزئی دربارهٔ جهان می‌دهند. برای نمونه، شکل ۱۲.۵ وضعیتی را نشان می‌دهد که در آن هر یک از سه مربعِ بازدید-نشده ولی قابلِدسترس—[1,3]، [2,2] و [3,1]—ممکن است گودال داشته باشد. استنتاجِ منطقیِ خالص نمی‌تواند هیچ نتیجه‌ای دربارهٔ اینکه کدام مربع بیشتر محتمل است امن باشد بگیرد، پس عاملِ منطقی ممکن است مجبور شود تصادفی انتخاب کند. خواهیم دید عاملِ احتمالاتی می‌تواند بسیار بهتر از عاملِ منطقی عمل کند. هدفِ ما محاسبهٔ احتمالِ اینکه هر یک از سه مربع گودال دارد است. (برای این مثال وومپوس و طلا را نادیده می‌گیریم.) ویژگی‌های مربوطِ جهانِ وومپوس این‌اند که (۱) گودال در همهٔ مربع‌های همسایه نسیم ایجاد می‌کند، و (۲) هر مربع به‌جز [1,1] با احتمالِ 0.2 گودال دارد. گامِ نخست شناساییِ مجموعهٔ متغیرهای تصادفیِ موردنیاز است:

- همانندِ موردِ منطقِ گزاره‌ای، برای هر مربع یک متغیرِ بولیِ Pij می‌خواهیم که صادق است اگر و فقط اگر مربعِ [i,j] واقعاً گودال داشته باشد.
- همچنین متغیرهای بولیِ Bij داریم که صادق‌اند اگر و فقط اگر مربعِ [i,j] بادی (نسیم‌دار) باشد؛ این متغیرها را فقط برای مربع‌های مشاهده‌شده در نظر می‌گیریم—در این مورد، [1,1]، [1,2] و [2,1].

گامِ بعد مشخص‌کردنِ توزیعِ مشترکِ کامل، P(P1,1, ..., P4,4, B1,1, B1,2, B2,1) است. با اعمالِ قاعدهٔ ضرب:

```
P(P1,1, ..., P4,4, B1,1, B1,2, B2,1) = P(B1,1, B1,2, B2,1 | P1,1, ..., P4,4) × P(P1,1,
                                                                        ..., P4,4)
```

این تجزیه دیدنِ اینکه مقادیرِ احتمالِ مشترک چه باید باشند را آسان می‌کند. جملهٔ نخست توزیعِ احتمالِ شرطیِ پیکربندیِ نسیم، با توجه به پیکربندیِ گودال، است؛ مقادیرش اگر همهٔ مربع‌های بادی مجاورِ گودال‌ها باشند ۱ و در غیرِ این صورت ۰ است.

جملهٔ دوم احتمالِ پیشینیِ پیکربندیِ گودال است. هر مربع با احتمالِ 0.2 گودال دارد، مستقل از مربع‌های دیگر؛ از این رو:

```
                                                P(P1,1, ..., P4,4) = ∏ P(Pi,j)        (12.22)
                                                                    (i,j از 1,1 تا 4,4)
```

برای پیکربندیِ خاصی با دقیقاً n گودال، احتمال $0.2^n \times 0.8^{16-n}$ است.

در وضعیتِ شکل ۱۲.۵(الف)، شواهد شاملِ نسیمِ مشاهده‌شده (یا غیابِ آن) در هر مربعِ بازدیدشده، همراه با این واقعیت که هر چنین مربعی گودال ندارد، است. این واقعیت‌ها را به‌اختصار چنین می‌نویسیم:

```
b = ¬b1,1 ∧ b1,2 ∧ b2,1        و        known = ¬p1,1 ∧ ¬p1,2 ∧ ¬p2,1
```

علاقه‌مندیم به پرس‌وجوهایی مانند P(P1,3 | known, b) پاسخ دهیم: چقدر محتمل است [1,3] گودال داشته باشد، با توجه به مشاهده‌های تاکنون؟

برای پاسخ به این پرس‌وجو، می‌توانیم از رویکردِ استانداردِ معادلهٔ (12.9) پیروی کنیم، یعنی جمع‌زدن روی درایه‌های توزیعِ مشترکِ کامل. فرض کنیم Unknown مجموعهٔ متغیرهای Pi,j برای مربع‌های به‌جزِ مربع‌های شناخته‌شده و مربعِ پرس‌وجوِ [1,3] باشد. سپس، با معادلهٔ (12.9):

```
                                P(P1,3 | known, b) = α ∑ P(P1,3, known, b, unknown)        (12.23)
                                                     unknown
```

احتمال‌های مشترکِ کامل از پیش مشخص شده‌اند، پس کار تمام است—یعنی، مگر اینکه به محاسبه اهمیت دهیم. ۱۲ مربعِ ناشناخته هست؛ از این رو جمع شامل $2^{12} = 4096$ جمله است. به‌طورِ کلی، جمع با شمارِ مربع‌ها به‌صورتِ نمایی رشد می‌کند.

مطمئناً، ممکن است کسی بپرسد، آیا مربع‌های دیگر نامربوط نیستند؟ چگونه [4,4] می‌تواند بر اینکه آیا [1,3] گودال دارد اثر بگذارد؟ در واقع، این شهود تقریباً درست است، اما باید دقیق‌تر شود.

آنچه واقعاً منظورمان است این است که اگر مقادیرِ همهٔ متغیرهای گودالِ مجاورِ مربع‌هایی را که به آن‌ها اهمیت می‌دهیم می‌دانستیم، آنگاه گودال‌ها (یا غیاب‌شان) در مربع‌های دیگر، دورتر، نمی‌توانستند اثرِ بیشتری بر باورِ ما داشته باشند.

فرض کنیم Frontier متغیرهای گودال (به‌جزِ متغیرِ پرس‌وجو) باشند که مجاورِ مربع‌های بازدیدشده‌اند؛ در این مورد فقط [2,2] و [3,1]. همچنین فرض کنیم Other متغیرهای گودال برای مربع‌های ناشناختهٔ دیگر باشند؛ در این مورد، ۱۰ مربعِ دیگر هست، چنان‌که در شکل ۱۲.۵(ب) نشان داده شده. با این تعریف‌ها، Unknown = Frontier ∪ Other. بینشِ کلیدیِ داده‌شده در بالا اکنون می‌تواند چنین بیان شود: نسیم‌های مشاهده‌شده به‌صورتِ شرطی مستقل از متغیرهای دیگرند، با توجه به متغیرهای شناخته‌شده، جبهه و پرس‌وجو. برای استفاده از این بینش، فرمولِ پرس‌وجو را به شکلی دست‌کاری می‌کنیم که در آن نسیم‌ها بر همهٔ متغیرهای دیگر شرطی شده باشند، و سپس استقلالِ شرطی را اعمال می‌کنیم:

$$\begin{aligned}
\mathbf{P}(P_{1,3} \mid known, b) &= \alpha \sum_{unknown} \mathbf{P}(P_{1,3}, known, b, unknown) && \text{(از معادلهٔ (12.23))} \\
&= \alpha \sum_{unknown} \mathbf{P}(b \mid P_{1,3}, known, unknown)\, \mathbf{P}(P_{1,3}, known, unknown) && \text{(قاعدهٔ ضرب)} \\
&= \alpha \sum_{frontier} \sum_{other} \mathbf{P}(b \mid known, P_{1,3}, frontier, other)\, \mathbf{P}(P_{1,3}, known, frontier, other) \\
&= \alpha \sum_{frontier} \sum_{other} \mathbf{P}(b \mid known, P_{1,3}, frontier)\, \mathbf{P}(P_{1,3}, known, frontier, other)
\end{aligned}$$

که گامِ آخر از استقلالِ شرطی استفاده می‌کند: b مستقل از other با توجه به $P_{1,3}$، known و frontier است. حالا، جملهٔ نخست در این عبارت به متغیرهای Other بستگی ندارد، پس می‌توانیم جمع را به درون ببریم:

$$\mathbf{P}(P_{1,3} \mid known, b) = \alpha \sum_{frontier} \mathbf{P}(b \mid known, P_{1,3}, frontier) \sum_{other} \mathbf{P}(P_{1,3}, known, frontier, other)$$

با استقلال، همانندِ معادلهٔ (12.22)، جملهٔ سمتِ راست قابلِ تجزیه است و سپس جمله‌ها قابلِ مرتب‌سازیِ دوباره‌اند:

$$\mathbf{P}(P_{1,3} \mid known, b) = \alpha \sum_{frontier} \mathbf{P}(b \mid known, P_{1,3}, frontier) \sum_{other} \mathbf{P}(P_{1,3}) \mathbf{P}(known) \mathbf{P}(frontier) \mathbf{P}(other)$$

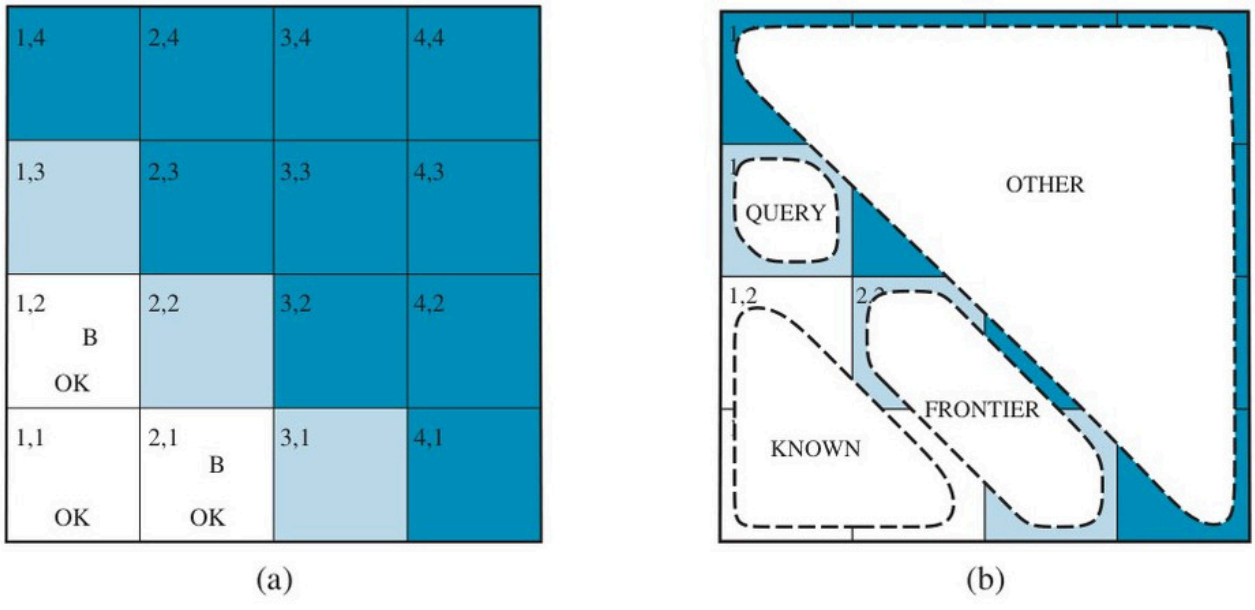
$$= \alpha \mathbf{P}(known) \mathbf{P}(P_{1,3}) \sum_{frontier} \mathbf{P}(b \mid known, P_{1,3}, frontier) \mathbf{P}(frontier) \sum_{other} \mathbf{P}(other)$$

$$= \alpha' \mathbf{P}(P_{1,3}) \sum_{frontier} \mathbf{P}(b \mid known, P_{1,3}, frontier) \mathbf{P}(frontier)$$

که گامِ آخر P(known) را در ثابتِ نرمال‌سازی ادغام می‌کند و از این واقعیت استفاده می‌کند که ∑other P(other) برابر ۱ است. حالا فقط چهار جمله در جمع روی متغیرهای جبهه، P2,2 و P3,1، هست.

استفاده از استقلال و استقلالِ شرطی مربع‌های دیگر را کاملاً از ملاحظه حذف کرده است. توجه کنید احتمال‌های موجود در P(b | known, P1,3, frontier) وقتی مشاهده‌های نسیم با متغیرهای دیگر سازگارند ۱ و در غیرِ این صورت ۰ است. بدین‌ترتیب، برای هر مقدارِ P1,3، روی مدل‌های منطقیِ متغیرهای جبهه که با واقعیت‌های شناخته‌شده سازگارند جمع می‌زنیم. (با شمارش روی مدل‌ها در شکل ۷.۵ صفحهٔ ۲۱۵ مقایسه کنید.) مدل‌ها و احتمال‌های پیشینیِ مرتبط‌شان—P(frontier)—در شکل ۱۲.۶ نشان داده شده‌اند. داریم:

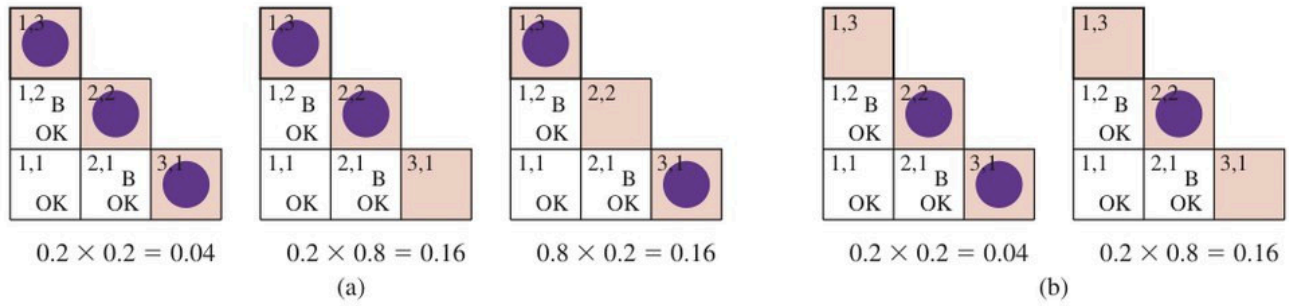
$$\mathbf{P}(P_{1,3} \mid known, b) = \alpha' \langle 0.2(0.04 + 0.16 + 0.16), 0.8(0.04 + 0.16) \rangle = \langle 0.31, 0.69 \rangle$$



شکل ۱۲.۵ — (الف) پس از یافتن نسیم در هر دو [1,2] و [2,1]، عامل گیر کرده—جای امنی برای کاوش نیست. (ب) تقسیمِ مربع‌ها به شناخته‌شده (Known)، جبهه (Frontier) و سایر (Other)، برای پرس‌وجو دربارهٔ [1,3].

یعنی [1,3] (و [3,1] به تقارن) با احتمالِ تقریباً ۳۱٪ گودال دارد. محاسبهٔ مشابهی، که خواننده ممکن است مایل باشد انجام دهد، نشان می‌دهد [2,2] با احتمالِ تقریباً ۸۶٪ گودال دارد. عاملِ وومپوس قطعاً باید از [2,2] اجتناب کند! توجه کنید عاملِ منطقیِ ما از فصل ۷ نمی‌دانست [2,2] بدتر از مربع‌های دیگر است. منطق می‌تواند به ما بگوید ناشناخته

است آیا گودالی در [2,2] هست، اما برای گفتنِ چقدر محتمل است به احتمال نیاز داریم. آنچه این بخش نشان داده این است که حتی مسائلِ ظاهراً پیچیده را می‌توان به‌طورِ دقیق در نظریهٔ احتمال صورت‌بندی و با الگوریتم‌های ساده حل کرد. برای راه‌حل‌های کارآمد، روابطِ استقلال و استقلالِ شرطی می‌توانند برای ساده‌سازیِ جمع‌های موردنیاز استفاده شوند. این روابط اغلب با درکِ طبیعیِ ما از چگونگیِ تجزیهٔ مسئله مطابقت دارند. در فصلِ بعد، بازنمایی‌های صوری برای چنین روابطی و نیز الگوریتم‌هایی که روی آن بازنمایی‌ها عمل می‌کنند تا استنتاجِ احتمالاتی را کارآمد انجام دهند، توسعه می‌دهیم.



شکل ۱۲.۶ — مدل‌های سازگار برای متغیرهای جبهه، P2,2 و P3,1، که P(frontier) را برای هر مدل نشان می‌دهد: (الف) سه مدل با P1,3=true که دو یا سه گودال نشان می‌دهند، و (ب) دو مدل با P1,3=false که یک یا دو گودال نشان می‌دهند. (احتمال‌های پیشینیِ مرتبط 0.04، 0.16، 0.16 برای حالتِ (الف) و 0.04، 0.16 برای حالتِ (ب) هستند.)

خلاصهٔ فصل

این فصل نظریهٔ احتمال را به‌عنوانِ شالودهٔ مناسب برای استدلالِ نامطمئن پیشنهاد کرده و مقدمه‌ای ملایم بر استفاده از آن ارائه داده است.

- عدم قطعیت هم از تنبلی و هم از نادانی ناشی می‌شود. در محیط‌های پیچیده، ناجبرگرایانه یا تا حدی قابلِ مشاهده گریزناپذیر است.
- احتمال‌ها ناتوانیِ عامل در رسیدن به تصمیمِ قطعی دربارهٔ صدقِ یک جمله را بیان می‌کنند. احتمال‌ها باورهای عامل را نسبت به شواهد خلاصه می‌کنند.
- نظریهٔ تصمیم باورها و خواسته‌های عامل را ترکیب می‌کند و بهترین کنش را کنشی تعریف می‌کند که مطلوبیتِ مورد انتظار را بیشینه می‌کند.
- استقلالِ مطلق میان زیرمجموعه‌های متغیرهای تصادفی به تجزیهٔ توزیعِ مشترکِ کامل به توزیع‌های مشترکِ کوچک‌تر اجازه می‌دهد و پیچیدگیِ آن را به‌شدت کاهش می‌دهد.
- قاعدهٔ بیز محاسبهٔ احتمال‌های ناشناخته را از احتمال‌های شرطیِ شناخته‌شده، معمولاً در جهتِ علّی، ممکن می‌کند. اعمالِ قاعدهٔ بیز با قطعاتِ شواهدِ زیاد به همان مشکلاتِ مقیاس‌پذیریِ توزیعِ مشترکِ کامل برمی‌خورد.
- استقلالِ شرطیِ پدیدآمده توسط روابطِ علّیِ مستقیم در حوزه به تجزیهٔ توزیعِ مشترکِ کامل به توزیع‌های شرطیِ کوچک‌تر اجازه می‌دهد.
- مدلِ بیزِ ساده استقلالِ شرطیِ همهٔ متغیرهای اثر را با توجه به متغیرِ علتِ تکی فرض می‌کند؛ اندازه‌اش با شمارِ اثرها به‌صورتِ خطی رشد می‌کند.
- عاملِ جهانِ وومپوس می‌تواند احتمال‌ها را برای جنبه‌های مشاهده‌نشدهٔ جهان محاسبه کند و بدین‌ترتیب بر تصمیم‌های عاملِ صرفاً منطقی بهبود ببخشد. استقلالِ شرطی این محاسبه‌ها را شدنی می‌کند.

یادداشت‌های کتاب‌شناختی و تاریخی

نظریهٔ احتمال به‌عنوانِ راهی برای تحلیلِ بازی‌های شانس اختراع شد. در حدودِ سال ۸۵۰ میلادی، ریاضی‌دانِ هندیِ ماهاویراچاریا توصیف کرد چگونه مجموعه‌ای از شرط‌بندی‌ها را بچینیم که نمی‌تواند ببازد (آنچه امروز «کتابِ هلندی» (Dutch Book) می‌نامیم). در اروپا، نخستین تحلیل‌های نظام‌مندِ مهم توسط جیرولامو کاردانو در حدودِ ۱۵۶۵ تولید شد، هرچند انتشار پس از مرگش بود (۱۶۶۳).

تا آن زمان، احتمال به‌واسطهٔ مجموعه‌ای از نتایج از مکاتبهٔ مشهورِ میان بلز پاسکال و پیر دو فرما در ۱۶۵۴ به‌عنوانِ رشته‌ای ریاضی تثبیت شده بود.

نخستین کتابِ درسیِ منتشرشدهٔ احتمال «دِ راتیوسینیس این لودو آله‌ائه» (De Ratiociniis in Ludo Aleae؛ دربارهٔ استدلال در بازیِ شانس) اثرِ هیگنز (۱۶۵۷) بود. دیدگاهِ «تنبلی و نادانی» دربارهٔ عدم قطعیت توسط جان آرباتنوت در مقدمهٔ ترجمه‌اش از هیگنز (آرباتنوت، ۱۶۹۲) توصیف شد: «ممکن نیست تاسی با چنین نیرو و جهتِ معینی بر چنین سمتِ معینی نیفتد، فقط من نیرو و جهتی را نمی‌دانم که باعث می‌شود بر چنین سمتِ معینی بیفتد، و بنابراین آن را شانس می‌نامم، که چیزی نیست جز فقدانِ فن.» پیوندِ میان احتمال و استدلال دست‌کم به قرنِ نوزدهم بازمی‌گردد: در ۱۸۱۹، پیر لاپلاس گفت «نظریهٔ احتمال چیزی نیست جز عقلِ سلیمِ تقلیل‌یافته به محاسبه.» در ۱۸۵۰، جیمز ماکسول گفت «منطقِ حقیقی برای این جهان حسابِ احتمالات است، که بزرگیِ احتمالی را که در ذهنِ انسانِ معقول هست یا باید باشد در نظر می‌گیرد.» بحثِ بی‌پایانی بر سرِ منشأ و جایگاهِ اعدادِ احتمال بوده است. موضعِ «فراوانی‌گرایانه» (Frequentist) این است که اعداد فقط می‌توانند از آزمایش‌ها بیایند: اگر ۱۰۰ نفر را آزمایش کنیم و دریابیم ۱۰ نفرشان پوسیدگی دارند، آنگاه می‌توانیم بگوییم احتمالِ پوسیدگی تقریباً ۰.۱ است. در این دیدگاه، اظهارِ «احتمالِ پوسیدگی ۰.۱ است» بدان معناست که ۰.۱ کسری است که در حدِّ بی‌نهایتِ نمونه مشاهده می‌شد. از هر نمونهٔ متناهی، می‌توانیم کسرِ واقعی را برآورد کنیم و نیز محاسبه کنیم برآوردمان چقدر احتمال دارد دقیق باشد. دیدگاهِ «عینیت‌گرایانه» (Objectivist) این است که احتمال‌ها جنبه‌های واقعیِ جهان—گرایش‌های اشیا به رفتار به شیوه‌های معین—هستند، نه صرفاً توصیف‌هایی از درجهٔ باورِ ناظر. برای نمونه، این واقعیت که سکهٔ منصفانه‌ای با احتمالِ ۰.۵ رو می‌آید، گرایشِ خودِ سکه است. در این دیدگاه، اندازه‌گیری‌های فراوانی‌گرایانه تلاش‌هایی برای مشاهدهٔ این گرایش‌ها هستند. بیشترِ فیزیک‌دانان موافق‌اند پدیده‌های کوانتومی به‌صورتِ عینی احتمالاتی‌اند، اما عدم قطعیت در مقیاسِ کلان—مثلاً در پرتابِ سکه—معمولاً از نادانی نسبت به شرایطِ اولیه ناشی می‌شود و با دیدگاهِ گرایشی سازگار به نظر نمی‌رسد.

دیدگاهِ «ذهن‌گرایانه» (Subjectivist) احتمال‌ها را راهی برای مشخصه‌سازیِ باورهای عامل توصیف می‌کند، نه داشتنِ اهمیتِ فیزیکیِ بیرونی. دیدگاهِ بیزیِ ذهنی به هر اسنادِ خودسازگار از احتمال‌های پیشینی به گزاره‌ها اجازه می‌دهد، اما سپس بر به‌روزرسانیِ بیزیِ درست با آمدنِ شواهد اصرار می‌ورزد.

حتی موضعِ فراوانی‌گرایانهٔ محض نیز به دلیلِ «مسئلهٔ ردهٔ مرجع» (Reference Class Problem) شاملِ ذهن‌گرایی است: در تلاش برای تعیینِ احتمالِ پیامدِ یک آزمایشِ خاص، فراوانی‌گرا باید آن را در ردهٔ مرجعی از آزمایش‌های «مشابه» با بسامدهای پیامدِ شناخته‌شده قرار دهد. اما ردهٔ درست چیست؟ آیزاک جی. گود نوشت: «هر رویدادِ زندگی منحصربه‌فرد است، و هر احتمالِ واقعیِ زندگی که در عمل برآورد می‌کنیم مربوط به رویدادی است که هرگز پیش از این رخ نداده» (گود، 1983، ص ۲۷).

برای نمونه، با توجه به بیمارِ خاصی، فراوانی‌گرایی که می‌خواهد احتمالِ پوسیدگی را برآورد کند، ردهٔ مرجعی از بیمارانِ دیگر را در نظر می‌گیرد که از جهاتِ مهم مشابه‌اند—سن، نشانه‌ها، رژیمِ غذایی—و می‌بیند چه نسبتی از آن‌ها پوسیدگی داشتند. اگر دندان‌پزشک هر چیزی را که دربارهٔ بیمار شناخته شده در نظر بگیرد—رنگِ مو، وزن تا نزدیک‌ترین گرم، نامِ خانوادگیِ مادر—آنگاه ردهٔ مرجع خالی می‌شود. این مسئلهٔ رنج‌آوری در فلسفهٔ علم بوده است.

پاسکال از احتمال به شیوه‌هایی استفاده کرد که هم به تفسیرِ عینی، به‌عنوانِ ویژگیِ جهانِ مبتنی بر تقارن یا بسامدِ نسبی، نیاز داشت و هم به تفسیرِ ذهنی، مبتنی بر درجهٔ باور—نخستین در تحلیل‌هایش از احتمال‌ها در بازی‌های شانس، دومی در استدلالِ مشهورِ «شرطِ پاسکال» دربارهٔ وجودِ احتمالیِ خدا. با این حال، پاسکال به‌وضوح تمایزِ میان این دو تفسیر را درنیافت. این تمایز نخستین بار به‌وضوح توسط جیمز برنولی (۱۷۰۵–۱۶۵۴) ترسیم شد.

لایب‌نیتس مفهومِ «کلاسیک» احتمال را به‌عنوانِ نسبتی از حالت‌های شمرده‌شده و هم‌احتمال معرفی کرد، که برنولی نیز استفاده کرد، هرچند لاپلاس (۱۸۱۶) آن را به برجستگی رساند. این مفهوم میان تفسیرِ بسامدی و تفسیرِ ذهنی مبهم است. حالت‌ها را می‌توان هم‌احتمال دانست یا به دلیلِ تقارنِ طبیعی و فیزیکیِ میان‌شان، یا صرفاً به این دلیل که دانشی نداریم که ما را به محتمل‌تردانستنِ یکی از دیگری رهنمون شود. استفاده از این ملاحظهٔ اخیر، ذهنی، برای توجیهِ تخصیصِ احتمال‌های برابر، «اصلِ بی‌تفاوتی» (Principle of Indifference) شناخته می‌شود. این اصل اغلب به لاپلاس (۱۸۱۶) نسبت داده می‌شود، اما او هرگز نامش را صریحاً به کار نبرد؛ کینز (۱۹۲۱) چنین کرد. جورج بول و جان ون هر دو به آن «اصلِ دلیلِ ناکافی» (Principle of Insufficient Reason) اشاره کردند.

بحثِ میان عینیت‌گرایان و ذهن‌گرایان در قرنِ بیستم تندتر شد. کولموگوروف (۱۹۶۳)، رونالد ای. فیشر (۱۹۲۲) و ریچارد فن میزس (۱۹۲۸) طرفدارانِ تفسیرِ بسامدِ نسبی بودند. تفسیرِ «گرایشی» کارل پوپر (۱۹۵۹، نخستین‌بار به آلمانی در ۱۹۳۴ منتشر شد) بسامدهای نسبی را به تقارنِ فیزیکیِ زیربنایی بازمی‌گرداند.

فرانک رامسی (۱۹۳۱)، برونو دو فینتی (۱۹۳۷)، ر. ت. کاکس (۱۹۴۶)، لئونارد ساویج (۱۹۵۴)، ریچارد جفری (۱۹۸۳) و ای. تی. جینز (۲۰۰۳) احتمال‌ها را به‌عنوانِ درجه‌های باورِ افرادِ خاص تفسیر کردند. تحلیل‌های آن‌ها از درجهٔ باور به‌طورِ نزدیک با مطلوبیت‌ها و رفتار—به‌طورِ خاص، با تمایل به شرط‌بندی—پیوند داشت.

رودولف کارناپ تفسیرِ متفاوتی از احتمال ارائه داد—نه به‌عنوانِ درجهٔ باوری که فرد واقعاً دارد، بلکه به‌عنوانِ درجهٔ باوری که استدلال‌گرِ ایده‌آل باید در گزارهٔ خاصِ a، با توجه به بدنهٔ شاهدِ خاصِ e، داشته باشد. کارناپ تلاش کرد این مفهومِ «درجهٔ تأیید» (Degree of Confirmation) را به‌صورتِ ریاضی دقیق کند، به‌عنوانِ رابطه‌ای منطقی میان a و e. در حالِ حاضر باور بر این است که هیچ منطقِ یگانه‌ای از این نوع وجود ندارد؛ بلکه هر چنین منطقی بر توزیعِ احتمالِ پیشینیِ ذهنی متکی است که اثرش با جمع‌آوریِ مشاهده‌های بیشتر کمرنگ می‌شود.

مطالعهٔ این رابطه قرار بود رشته‌ای ریاضی به نام «منطقِ استقرایی» (Inductive Logic) تشکیل دهد، مشابهِ منطقِ قیاسیِ معمولی (کارناپ، 1948، 1950). کارناپ نتوانست منطقِ استقرایی‌اش را خیلی فراتر از حالتِ گزاره‌ای بسط دهد، و پاتنام (۱۹۶۳) با استدلال‌های ستیزه‌جویانه نشان داد برخی دشواری‌ها ذاتی‌اند. کارِ جدیدترِ بیکوس، گروو، هالپرن و کولر (۱۹۹۲) روش‌های کارناپ را به نظریه‌های مرتبهٔ اول بسط می‌دهد.

نخستین چارچوبِ کاملاً اصل‌بندیانه برای نظریهٔ احتمال توسط کولموگوروف (۱۹۵۰، نخستین‌بار به آلمانی در ۱۹۳۳) پیشنهاد شد. رِنی (۱۹۷۰) بعداً ارائهٔ اصل‌بندیانه‌ای داد که احتمالِ شرطی را، به‌جایِ احتمالِ مطلق، به‌عنوانِ امری اولیه در نظر می‌گرفت.

علاوه بر استدلال‌های دو فینتی برای اعتبارِ اصل‌ها، کاکس (۱۹۴۶) نشان داد هر سامانهٔ استدلالِ نامطمئنی که مجموعه مفروضاتش را برآورده کند معادلِ نظریهٔ احتمال است. این اعتمادِ تازه‌ای به طرفدارانِ احتمال داد، اما دیگران قانع نشدند و به این فرض اعتراض کردند که باور باید با عددِ تکی بازنمایی شود. هالپرن (۱۹۹۹) مفروضات را توصیف و برخی شکاف‌ها در صورت‌بندیِ اصلیِ کاکس را نشان می‌دهد. هورن (۲۰۰۳) نشان می‌دهد چگونه دشواری‌ها را وصله کنیم. جینز (۲۰۰۳) استدلالِ مشابهی دارد که خواندنش آسان‌تر است.

کشیشِ توماس بیز (۱۷۶۱–۱۷۰۲) قاعدهٔ استدلال دربارهٔ احتمال‌های شرطی را معرفی کرد که پس از مرگش به نامِ او نام‌گذاری شد (بیز، ۱۷۶۳). بیز فقط حالتِ پیشین‌های یکنواخت را در نظر گرفت؛ این لاپلاس بود که به‌طورِ مستقل حالتِ عمومی را توسعه داد.

استدلالِ احتمالاتیِ بیزی از دههٔ ۱۹۶۰ در هوش مصنوعی استفاده شده، به‌ویژه در تشخیصِ پزشکی. نه فقط برای تشخیص از شواهدِ موجود، بلکه برای انتخابِ پرسش‌ها و آزمون‌های بیشتر با استفاده از نظریهٔ ارزشِ اطلاعات (بخش ۱۶.۶) وقتی شواهدِ موجود قطعی نبود استفاده می‌شد (گوری، 1968؛ گوری و همکاران، 1973). یک سامانه از متخصصانِ انسانی در تشخیصِ بیماری‌های حادِ شکمی بهتر عمل کرد (دو دامبال و همکاران، 1974). لوکاس و همکاران (۲۰۰۴) مروری ارائه می‌دهند.

این سامانه‌های بیزیِ اولیه از مشکلاتِ متعددی رنج می‌بردند. از آن‌جا که هیچ مدلِ نظری از شرایطی که تشخیص می‌دادند نداشتند، در برابرِ داده‌های غیرنمایندهٔ رخ‌داده در وضعیت‌هایی که فقط نمونهٔ کوچکی در دسترس بود آسیب‌پذیر بودند (دو دامبال و همکاران، 1981). حتی بنیادی‌تر، از آن‌جا که صورت‌بندیِ موجزی (مانند آنچه در فصل ۱۳ توصیف خواهد شد) برای بازنمایی و استفاده از اطلاعاتِ استقلالِ شرطی نداشتند، به اکتساب، ذخیره‌سازی و پردازشِ جدول‌های عظیمِ داده‌های احتمالاتی وابسته بودند. به دلیلِ این دشواری‌ها، روش‌های احتمالاتی برای مقابله با عدم قطعیت از دههٔ ۱۹۷۰ تا اواسطِ دههٔ ۱۹۸۰ در هوش مصنوعی از رواج افتادند. پیشرفت‌ها از اواخرِ دههٔ ۱۹۸۰ در فصلِ بعد توصیف می‌شوند.

مدلِ بیزِ ساده برای توزیع‌های مشترک از دههٔ ۱۹۵۰ در ادبیاتِ بازشناسیِ الگو به‌طورِ گسترده مطالعه شده (دودا و هارت، 1973). همچنین، اغلب ناخواسته، در بازیابیِ اطلاعات استفاده شده، با آغاز از کارِ مارون (۱۹۶۱). شالوده‌های احتمالاتیِ این فن، که بیشتر در تمرین BAYS.۱۲ توصیف شده، توسط رابرتسون و اسپارک جونز (۱۹۷۶) روشن شد. دومینگوس و پازانی (۱۹۹۷) تبیینی برای موفقیتِ شگفت‌انگیزِ استدلالِ بیزیِ ساده حتی در حوزه‌هایی که فرض‌های استقلال آشکارا نقض می‌شوند ارائه می‌دهند.

کتاب‌های درسیِ مقدماتیِ خوبِ بسیاری دربارهٔ نظریهٔ احتمال هست، از جمله برتسکاس و تسیتسیکلیس (2008)، راس (2015) و گرین‌ستد و اسنل (1997). دگروت و شرویش (2001) مقدمه‌ای ترکیبی بر احتمال و آمار از دیدگاهِ بیزی ارائه می‌دهند، و والپول و همکاران (2016) مقدمه‌ای برای دانشمندان و مهندسان. جینز (2003) شرحِ بسیار قانع‌کننده‌ای از رویکردِ بیزی می‌دهد. بیلینگزلی (2012) و ونکاتش (2012) برخوردهای ریاضی‌تری ارائه می‌دهند، شاملِ همهٔ پیچیدگی‌های متغیرهای پیوسته که ما از قلم انداخته‌ایم. هکینگ (1975) و هالد (1990) تاریخِ آغازینِ مفهومِ احتمال را پوشش می‌دهند و برنشتاین (1996) روایتی عامه‌پسند می‌دهد.

پانوشت‌ها:

[1] فعلاً مجموعه‌ای گسسته و شمارا از جهان‌ها را فرض می‌کنیم. برخوردِ درستِ حالتِ پیوسته پیچیدگی‌هایی به همراه می‌آورد که برای بیشترِ مقاصد در هوش مصنوعی کمتر مربوط‌اند.

[2] توجه کنید تقدمِ «|» چنین است که هر عبارتِ شکلِ P(... | ...) همیشه به‌معنای P((...) | (...)) است.

[3] این قراردادها با هم به ابهامِ بالقوه‌ای در نماد هنگام جمع‌زدن روی مقادیرِ متغیرِ بولی می‌انجامند: P(a) احتمالِ صادق‌بودنِ A است، در حالی که در عبارتِ ∑a P(a) صرفاً به احتمالِ یکی از مقادیرِ a از A اشاره دارد.

[4] دشواری‌ها شاملِ مجموعهٔ ویتالی است، زیرمجموعه‌ای خوش‌تعریف از بازهٔ [0,1] با اندازه‌ای خوش‌تعریف‌نشده.

[5] ممکن است کسی استدلال کند ترجیح‌های عامل برای ترازهای بانکیِ مختلف چنان‌اند که امکانِ ازدست‌دادنِ ۱ دلار با امکانِ برابرِ بردنِ ۱ دلار متوازن نمی‌شود. یک پاسخِ ممکن آن است که مبلغ‌های شرط را به‌اندازهٔ کافی کوچک کنیم تا از این مسئله اجتناب شود. تحلیلِ ساویج (۱۹۵۴) مسئله را به‌کلی دور می‌زند.

[6] چنین نامیده می‌شود به دلیلِ رویهٔ رایجِ حسابدارانِ بیمه (اکچوئری‌ها) در نوشتنِ جمعِ بسامدهای مشاهده‌شده در حاشیه‌های جدول‌های بیمه.

[7] فرض می‌کنیم بیمار و دندان‌پزشک افرادِ متمایزی‌اند.

[8] باید مراقب بود احتمالِ صفر به واژه‌هایی که قبلاً در ردهٔ معینی از اسناد دیده نشده‌اند تخصیص ندهیم، زیرا صفر همهٔ شواهدِ دیگر در معادلهٔ (12.21) را از بین می‌برد. صرفاً چون واژه‌ای را ندیده‌ای به این معنا نیست که هرگز نخواهی دید. در عوض، بخشِ کوچکی از توزیعِ احتمال را برای بازنماییِ واژه‌های «پیش‌تر دیده‌نشده» ذخیره کن. برای اطلاعاتِ بیشتر دربارهٔ این مسئله به‌طورِ کلی فصل ۲۰، و برای موردِ خاصِ مدل‌های واژه بخش ۲۳.۱.۴ را ببینید.

فصل ۱۳: استدلال احتمالاتی (Probabilistic Reasoning)

در این فصل توضیح می‌دهیم چگونه مدل‌های شبکه‌ایِ کارآمد برای استدلال در شرایط عدم قطعیت، مطابق با قوانین نظریهٔ احتمال، بسازیم، و چگونه میان همبستگی و علّیت تمایز بگذاریم.

فصل ۱۲ عناصرِ پایهٔ نظریهٔ احتمال را معرفی کرد و به اهمیت روابط استقلال و استقلال شرطی در ساده‌سازیِ بازنمایی‌های احتمالاتیِ جهان اشاره کرد. این فصل راهی نظام‌مند برای بازنماییِ صریح چنین روابطی در قالب «شبکه‌های بیزی» (Bayesian Networks) معرفی می‌کند. نحو و معناشناسی این شبکه‌ها را تعریف می‌کنیم و نشان می‌دهیم چگونه می‌توان از آن‌ها برای ثبتِ دانشِ نامطمئن به‌شیوه‌ای طبیعی و کارآمد استفاده کرد. سپس نشان می‌دهیم استنتاجِ احتمالاتی، اگرچه در بدترین حالت از نظرِ محاسباتی ناشدنی است، در بسیاری از موقعیت‌های عملی می‌تواند کارآمد انجام شود. همچنین انواعی از الگوریتم‌های استنتاجِ تقریبی را توصیف می‌کنیم که اغلب وقتی استنتاجِ دقیق شدنی نیست قابلِ اعمال‌اند. فصل ۱۵ ایده‌های پایهٔ شبکه‌های بیزی را به زبان‌های صوریِ بیانی‌تری برای تعریفِ مدل‌های احتمال بسط می‌دهد.

۱۳.۱ بازنمایی دانش در حوزه‌ای نامطمئن

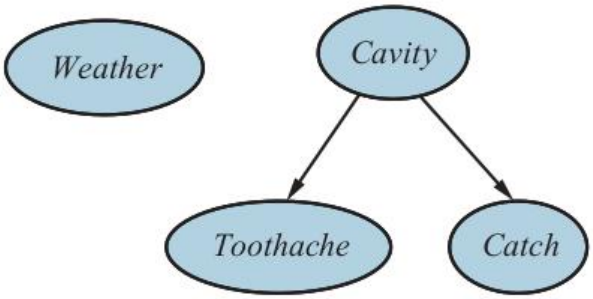
در فصل ۱۲ دیدیم توزیعِ احتمالِ مشترکِ کامل می‌تواند به هر پرسشی دربارهٔ حوزه پاسخ دهد، اما با رشدِ شمارِ متغیرها می‌تواند به‌صورتِ ناشدنی بزرگ شود. افزون بر این، مشخص‌کردنِ احتمال‌ها برای جهان‌های ممکن یک‌به‌یک غیرطبیعی و ملال‌آور است.

همچنین دیدیم روابط استقلال و استقلال شرطی میان متغیرها می‌توانند شمارِ احتمال‌های لازم برای تعریفِ توزیعِ مشترکِ کامل را بسیار کاهش دهند. این بخش ساختارِ داده‌ای به نام «شبکهٔ بیزی»[1] معرفی می‌کند تا وابستگی‌های میان متغیرها را بازنمایی کند. شبکه‌های بیزی می‌توانند اساساً هر توزیعِ احتمالِ مشترکِ کاملی را بازنمایی کنند و در بسیاری از موارد این کار را بسیار موجز انجام می‌دهند.

شبکهٔ بیزی گرافی جهت‌دار است که در آن هر گره با اطلاعاتِ احتمالاتیِ کمّی برچسب‌گذاری شده. مشخصاتِ کامل به‌صورتِ زیر است:

1. هر گره با یک متغیرِ تصادفی متناظر است، که ممکن است گسسته یا پیوسته باشد.
2. پیوندها یا پیکان‌های جهت‌دار جفت‌هایی از گره‌ها را به هم متصل می‌کنند. اگر پیکانی از گرهٔ X به گرهٔ Y باشد، می‌گوییم X «والد» (Parent) Y است. گراف هیچ دورِ جهتداری ندارد و از این رو «گرافِ جهتداری بدون دور» (Directed Acyclic Graph) یا DAG است.
3. هر گرهٔ Xi اطلاعاتِ احتمالیِ θ(Xi | Parents(Xi)) دارد که اثرِ والدها را روی گره با شمارِ متناهی پارامتر کمّی می‌کند.[1]

شبکه‌های بیزی که اغلب به‌اختصار «شبکهٔ بیز» (Bayes Net) نامیده می‌شوند، در دهه‌های ۱۹۸۰ و ۱۹۹۰ «شبکهٔ باور» (Belief Network) نامیده می‌شدند. «شبکهٔ علّی» (Causal Network) شبکهٔ بیزی با قیدهای اضافی روی معنای پیکان‌هاست (بخش ۱۳.۵ را ببینید). اصطلاح «مدلِ گرافیکی» (Graphical Model) به ردهٔ گسترده‌تری اشاره دارد که شبکه‌های بیزی را نیز شامل می‌شود.



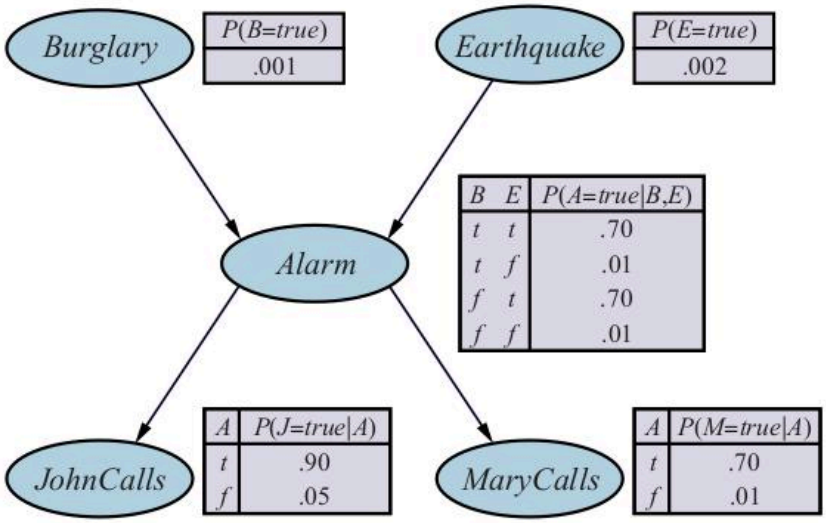
شکل ۱۳.۱ – شبکهٔ بیزیِ ساده‌ای که در آن Weather از سه متغیرِ دیگر مستقل است و Toothache و Catch با توجه به Cavity به‌صورتِ شرطی مستقل‌اند.

«توپولوژیِ» شبکه—مجموعهٔ گره‌ها و پیوندها—روابطِ استقلالِ شرطیِ برقرار در حوزه را مشخص می‌کند، به‌شیوه‌ای که به‌زودی دقیق خواهد شد. معنای شهودیِ پیکان معمولاً این است که X تأثیرِ مستقیمی روی Y دارد، که پیشنهاد می‌کند علت‌ها باید والدِ اثرها باشند. معمولاً برای کارشناسِ حوزه آسان است تصمیم بگیرد چه تأثیرهای مستقیمی در حوزه وجود دارند—در واقع بسیار آسان‌تر از مشخص‌کردنِ خودِ احتمال‌ها. پس از چیدنِ توپولوژیِ شبکهٔ بیز، فقط باید اطلاعاتِ احتمالاتیِ محلی را برای هر متغیر، در قالبِ توزیعِ شرطی به‌شرطِ والدهایش، مشخص کنیم. توزیعِ مشترکِ کاملِ همهٔ متغیرها توسط توپولوژی و اطلاعاتِ احتمالاتیِ محلی تعریف می‌شود. جهانِ سادهٔ توصیف‌شده در فصل ۱۲ را به یاد آورید، متشکل از متغیرهای Toothache، Cavity، Catch و Weather. استدلال کردیم Weather از متغیرهای دیگر مستقل است؛ علاوه بر آن، استدلال کردیم Toothache و Catch با توجه به Cavity به‌صورتِ شرطی مستقل‌اند. این روابط با ساختارِ شبکهٔ بیز نشان‌داده‌شده در شکل ۱۳.۱ بازنمایی می‌شوند.

به‌صورتِ صوری، استقلالِ شرطیِ Toothache و Catch با توجه به Cavity با نبودِ پیوند میان Toothache و Catch نشان داده می‌شود. به‌صورتِ شهودی، شبکه این واقعیت را بازنمایی می‌کند که Cavity علتِ مستقیمِ Toothache و Catch است، در حالی که هیچ رابطهٔ علّیِ مستقیمی میان Toothache و Catch وجود ندارد.

حالا مثال زیر را در نظر بگیرید که فقط کمی پیچیده‌تر است. شما دزدگیریِ تازه‌ای در خانه نصب کرده‌اید. در تشخیصِ سرقت نسبتاً قابلِ اعتماد است، اما گاهی با زلزله‌های کوچک به صدا در می‌آید. (این مثال از جودیا پرل است، ساکنِ لس‌آنجلسِ زلزله‌خیز.) همچنین دو همسایه دارید، جان و مری، که قول داده‌اند وقتی صدای آژیر را بشنوند در محلِ کار با شما تماس بگیرند. جان تقریباً همیشه وقتی صدای آژیر را می‌شنود تماس می‌گیرد، اما گاهی زنگِ تلفن را با آژیر اشتباه می‌گیرد و آن‌موقع هم تماس می‌گیرد. مری، از سوی دیگر، موسیقیِ بلندی را دوست دارد و اغلب آژیر را کلاً از دست می‌دهد. با توجه به شواهدِ اینکه چه کسی تماس گرفته یا نگرفته، می‌خواهیم احتمالِ وقوعِ سرقت را برآورد کنیم.

شبکهٔ بیزِ این حوزه در شکل ۱۳.۲ آمده. ساختارِ شبکه نشان می‌دهد سرقت و زلزله مستقیماً بر احتمالِ به‌صدا درآمدنِ آژیر اثر می‌گذارند، اما اینکه جان و مری تماس بگیرند فقط به آژیر بستگی دارد. شبکه بدین ترتیب مفروضاتِ ما را بازنمایی می‌کند: آن‌ها سرقت‌ها را مستقیماً درک نمی‌کنند، متوجهِ زلزله‌های کوچک نمی‌شوند، و پیش از تماس با یکدیگر مشورت نمی‌کنند. اطلاعاتِ احتمالاتیِ محلیِ متصل به هر گره در شکل ۱۳.۲ شکلِ «جدولِ احتمالِ شرطی» (Conditional Probability Table یا CPT) دارد. (CPTها فقط برای متغیرهای گسسته استفاده می‌شوند؛ بازنمایی‌های دیگر، از جمله بازنمایی‌های مناسبِ متغیرهای پیوسته، در بخش ۱۳.۲ توصیف شده‌اند.)



شکل ۱۳.۲ — شبکهٔ بیزیِ نمونه‌ای که هم توپولوژی و هم جدول‌های احتمالِ شرطی (CPT) را نشان می‌دهد. در CPTها، حروف B، E، A، J و M به‌ترتیب مخففِ سرقت (Burglary)، زلزله (Earthquake)، آژیر (Alarm)، تماس جان (JohnCalls) و تماس مری (MaryCalls) هستند.

هر ردیف در یک CPT شاملِ احتمالِ شرطیِ هر مقدارِ گره برای یک «حالتِ شرطی» (Conditioning Case) است. حالتِ شرطی فقط ترکیبی ممکن از مقادیرِ گره‌های والد است—در واقع جهانِ ممکنِ کوچکی، اگر دوست داشته باشید. هر ردیف باید به ۱ جمع شود، زیرا درایه‌ها مجموعه‌ای استیفاگر از حالت‌ها را برای متغیر بازنمایی می‌کنند. برای متغیرهای بولی، کافی است بدانیم احتمالِ مقدارِ صادق p است؛ احتمالِ کاذب باید 1-p باشد، پس اغلب عددِ دوم را حذف می‌کنیم، همان‌گونه که در شکل ۱۳.۲. به‌طورِ کلی، جدولِ متغیرِ بولی با k والدِ بولی شاملِ 2^k احتمالِ قابلِ‌مشخص‌کردنِ مستقل است. گرهٔ بدونِ والد فقط یک ردیف دارد، که احتمال‌های پیشینیِ هر مقدارِ ممکنِ متغیر را بازنمایی می‌کند.

توجه کنید شبکه گره‌هایی متناظر با گوش‌دادنِ فعلیِ مری به موسیقیِ بلند یا زنگ‌خوردنِ تلفن و اشتباهِ جان ندارد. این عوامل در عدم قطعیتِ مرتبط با پیوندهای Alarm به JohnCalls و MaryCalls خلاصه شده‌اند. این هر دو تنبلی و نادانی را در عمل نشان می‌دهد، چنان‌که در صفحهٔ ۳۸۶ توضیح داده شد: کارِ زیادی می‌برد تا بفهمیم چرا آن عوامل در هر موردِ خاص کم‌وبیش محتمل‌اند، و به هر حال راهِ معقولی برای به‌دست‌آوردنِ اطلاعاتِ مربوط نداریم.

احتمال‌ها در واقع مجموعه‌ای بالقوه نامتناهی از شرایط را خلاصه می‌کنند که در آن آژیر ممکن است به صدا درنیاید (رطوبتِ زیاد، قطعِ برق، باتریِ خالی، سیم‌های بریده، موشِ مرده‌ای گیرکرده در زنگ، و...) یا جان و مری ممکن است تماس نگیرند و گزارش ندهند (رفته‌اند ناهار، در مسافرت‌اند، موقتاً ناشنوا شده‌اند، هلیکوپتری در حال عبور است، و...).

بدین ترتیب، عاملِ کوچک می‌تواند با جهانِ بسیار بزرگی کنار بیاید، دست‌کم به‌صورتِ تقریبی.

۱۳.۲ معناشناسی شبکه‌های بیزی

نحوِ شبکهٔ بیز شاملِ گرافِ جهتداریِ بدونِ دور با اطلاعاتِ احتمالاتیِ محلیِ متصل به هر گره است. معناشناسی تعریف می‌کند نحو چگونه با توزیعِ مشترکی روی متغیرهای شبکه متناظر می‌شود.

فرض کنید شبکهٔ بیز شاملِ n متغیر X1,...,Xn باشد. درایه‌ای عام در توزیعِ مشترک سپس P(X1=x1 ∧ ... ∧ Xn=xn) است، یا به‌اختصار P(x1,...,xn). معناشناسیِ شبکه‌های بیز هر درایهٔ توزیعِ مشترک را چنین تعریف می‌کند:

$$P(x_1,\ldots,x_n) = \prod_{i=1..n} \theta(x_i \mid parents(X_i)) \qquad (13.1)$$

که در آن parents(Xi) مقادیرِ Parents(Xi) را نشان می‌دهد که در x1,...,xn ظاهر می‌شوند. بدین‌ترتیب، هر درایه از توزیعِ مشترک با حاصل‌ضربِ عناصرِ مناسبِ توزیع‌های شرطیِ محلی در شبکهٔ بیز بازنمایی می‌شود.

برای نشان‌دادنِ این، می‌توانیم احتمالِ اینکه آژیر به صدا درآمده، اما نه سرقتی رخ داده و نه زلزله‌ای، و هر دو جان و مری تماس بگیرند را محاسبه کنیم. به‌سادگی درایه‌های مربوط را از توزیع‌های شرطیِ محلی ضرب می‌کنیم (نامِ متغیرها را مخفف می‌کنیم):

$$P(j, m, a, \neg b, \neg e) = P(j|a)\, P(m|a)\, P(a|\neg b \wedge \neg e)\, P(\neg b)\, P(\neg e)$$
$$= 0.998 \times 0.999 \times 0.01 \times 0.70 \times 0.90 = 0.00628$$

بخش ۱۲.۳ توضیح داد توزیعِ مشترکِ کامل می‌تواند برای پاسخ به هر پرس‌وجویی دربارهٔ حوزه استفاده شود. اگر شبکهٔ بیز بازنماییِ توزیعِ مشترک باشد، آن‌گاه آن نیز می‌تواند برای پاسخ به هر پرس‌وجویی استفاده شود، با جمع‌زدنِ همهٔ مقادیرِ احتمالِ مشترکِ مربوط، که هر یک با ضربِ احتمال‌های توزیع‌های شرطیِ محلی محاسبه می‌شود. بخش ۱۳.۳ این را با جزئیاتِ بیشتر توضیح می‌دهد، اما روش‌هایی را نیز توصیف می‌کند که بسیار کارآمدترند.

تا این‌جا یک نکتهٔ مهم را از نظر انداخته‌ایم: معنای اعدادی که به توزیع‌های شرطیِ محلیِ θ(xi | parents(Xi)) می‌روند چیست؟ معلوم می‌شود از معادلهٔ (13.1) می‌توانیم اثبات کنیم پارامترهای θ(xi | parents(Xi)) دقیقاً همان احتمال‌های شرطیِ P(xi | parents(Xi)) هستند که توزیعِ مشترک دلالت می‌کند. به یاد آورید احتمال‌های شرطی را می‌توان از توزیعِ مشترک چنین محاسبه کرد:

$$P(x_i \mid parents(X_i)) \equiv P(x_i, parents(X_i)) / P(parents(X_i))$$
$$= P(x_i, parents(X_i), y) / \sum_{x'_i, y} P(x'_i, parents(X_i), y)$$

که در آن y مقادیرِ همهٔ متغیرهای به‌جزِ Xi و والدهایش را بازنمایی می‌کند. از این خطِ آخر می‌توان اثبات کرد P(xi | parents(Xi)) = θ(xi | parents(Xi)) (تمرین CPTE.۱۳). از این رو، می‌توانیم معادلهٔ (13.1) را چنین بازنویسی کنیم:

```
P(x1,...,xn) = ∏ P(xi | parents(Xi))        (13.2)
                i=1..n
```

این بدان معناست وقتی برای توزیع‌های شرطیِ محلی مقادیری برآورد می‌کنیم، آن‌ها باید احتمال‌های شرطیِ واقعیِ متغیر به‌شرطِ والدهایش باشند. پس، برای نمونه، وقتی مشخص می‌کنیم P(JohnCalls=true | Alarm=true) = 0.90، باید چنین باشد که در حدود ۹۰٪ مواقعِ به‌صدادرآمدنِ آژیر، جان تماس می‌گیرد. این واقعیت که هر پارامترِ شبکه معنایی دقیق بر حسبِ فقط مجموعه‌ای کوچک از متغیرها دارد، برای استحکام و سهولتِ مشخص‌کردنِ مدل اهمیتِ حیاتی دارد.

راهِ ساختنِ شبکهٔ بیز برای حوزه‌ای معین را می‌توان با یک روشِ کلی توصیف کرد. فرض کنید باید گره‌ها را به‌ترتیبِ X1,...,Xn بسازیم. معادلهٔ (13.2) می‌تواند چنین بازنویسی شود تا این ساختارِ خاص را بازتاب دهد:

```
P(x1,...,xn) = ∏ P(xi | parents(Xi))        (13.3)
                i=1..n
```

آنچه معادلهٔ (13.3) می‌گوید این است که شبکهٔ بیزی بازنماییِ درستی از حوزه است فقط اگر هر گره با توجه به والدهایش به‌صورتِ شرطی مستقل از دیگر پیشینیانش در ترتیبِ گره‌ها باشد. می‌توانیم این شرط را با این روش برآورده کنیم:

1. **گره‌ها:** نخست مجموعهٔ متغیرهای لازم برای مدل‌کردنِ حوزه را تعیین کن. حالا آن‌ها را مرتب کن، {X1,...,Xn}. هر ترتیبی کار می‌کند، اما شبکهٔ حاصل فشرده‌تر خواهد بود اگر متغیرها طوری مرتب شوند که علت‌ها بر اثرها مقدم باشند.
2. **پیوندها:** برای i از 1 تا n: مجموعه‌ای کمینه از والدها برای Xi از میان X1,...,Xi-1 برگزین، به‌گونه‌ای که معادلهٔ (13.3) برآورده شود. برای هر والد، پیوندی از والد به Xi درج کن.
3. **CPTها:** جدولِ احتمالِ شرطیِ P(Xi | Parents(Xi)) را بنویس.

به‌صورتِ شهودی، والدهای گرهٔ Xi باید شاملِ همهٔ آن گره‌های X1,...,Xi-1 باشند که مستقیماً بر Xi اثر می‌گذارند. برای نمونه، فرض کنید شبکهٔ شکل ۱۳.۲ را به‌جز انتخابِ والدها برای MaryCalls کامل کرده‌ایم. MaryCalls قطعاً از وجودِ سرقت یا زلزله متأثر است، اما نه به‌صورتِ مستقیم. به‌صورتِ شهودی، دانشِ ما از حوزه می‌گوید این رخدادها فقط از طریقِ اثرشان بر آژیر بر رفتارِ تماسِ مری اثر می‌گذارند. همچنین، با توجه به حالتِ آژیر، اینکه جان تماس بگیرد هیچ تأثیری بر تماسِ مری ندارد. به‌صورتِ صوری، باور داریم اظهارِ استقلالِ شرطیِ زیر برقرار است:

```
P(MaryCalls | JohnCalls, Alarm, Earthquake, Burglary) = P(MaryCalls | Alarm)
```

بدین‌ترتیب، Alarm تنها گرهٔ والدِ MaryCalls خواهد بود.

چون هر گره فقط به گره‌های پیشین متصل است، این روشِ ساخت تضمین می‌کند شبکه بدونِ دور است. ویژگیِ مهمِ دیگرِ شبکه‌های بیز این است که هیچ مقدارِ احتمالِ زائدی ندارند. اگر زائدی نباشد، جایی برای ناسازگاری نیست: ناممکن است مهندسِ دانش یا کارشناسِ حوزه شبکهٔ بیزی بسازد که اصل‌های احتمال را نقض کند.

فشردگی و ترتیبِ گره‌ها. جدا از بازنماییِ کامل و بدونِ زائدِ حوزه، شبکهٔ بیز اغلب می‌تواند بسیار فشرده‌تر از توزیعِ مشترکِ کامل باشد. این ویژگی است که مدیریتِ حوزه‌های با متغیرهای بسیار را شدنی می‌کند. فشردگیِ شبکه‌های بیزی نمونه‌ای از ویژگیِ عمومیِ سامانه‌های «ساختار-محلی» (Locally Structured، که «پراکنده» یا Sparse نیز نامیده می‌شوند) است. در سامانهٔ ساختار-محلی، هر زیرمؤلفه مستقیماً فقط با شمارِ کران‌داری از مؤلفه‌های دیگر تعامل می‌کند، صرف‌نظر از شمارِ کلِ مؤلفه‌ها. ساختارِ محلی معمولاً با رشدِ خطی به‌جای نمایی در پیچیدگی همراه است.

در موردِ شبکه‌های بیز، معقول است فرض کنیم در بیشترِ حوزه‌ها هر متغیرِ تصادفی مستقیماً حداکثر توسط k متغیرِ دیگر اثر می‌پذیرد، برای ثابتِ kای. اگر برای سادگی n متغیرِ بولی فرض کنیم، آنگاه میزانِ اطلاعاتِ لازم برای مشخص‌کردنِ هر جدولِ احتمالِ شرطی حداکثر k^2 عدد خواهد بود، و کلِ شبکه را می‌توان با n^2 × k عدد مشخص کرد. در مقابل، توزیعِ مشترک شاملِ n^2 عدد است. برای عینیت، فرض کنیم n=30 گره داریم، هر یک با پنج والد (k=5). آنگاه شبکهٔ بیزی ۹۶۰ عدد لازم دارد، اما توزیعِ مشترکِ کامل بیش از یک میلیارد لازم دارد.

مشخص‌کردنِ جدول‌های احتمالِ شرطی برای شبکهٔ کاملاً متصل، که در آن هر متغیر همهٔ پیشینیانش را والد دارد، همان میزانِ اطلاعاتِ مشخص‌کردنِ توزیعِ مشترک در شکلِ جدولی را لازم دارد. به همین دلیل، اغلب پیوندها را حذف می‌کنیم حتی اگر وابستگیِ جزیی وجود داشته باشد، زیرا افزایشِ جزییِ دقت ارزشِ پیچیدگیِ اضافیِ شبکه را ندارد. برای نمونه، ممکن است کسی به شبکهٔ سرقتِ ما اعتراض کند که اگر زلزلهٔ بزرگی رخ دهد، جان و مری حتی اگر صدای آژیر را بشنوند تماس نخواهند گرفت، زیرا فرض می‌کنند زلزله علت است. اینکه پیوندی از Earthquake به JohnCalls و MaryCalls بیفزاییم (و بدین‌ترتیب جدول‌ها را بزرگ کنیم) بستگی به اهمیتِ گرفتنِ احتمال‌های دقیق‌تر در مقایسه با هزینهٔ مشخص‌کردنِ اطلاعاتِ اضافی دارد.

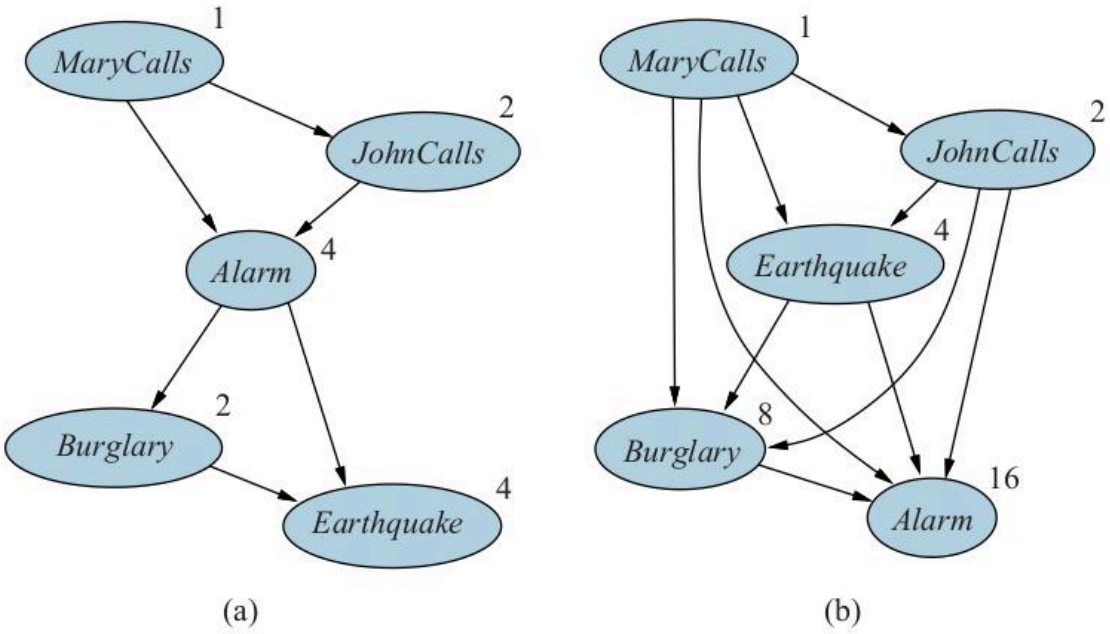
حتی در حوزهٔ ساختار-محلی، فقط اگر ترتیبِ گره‌ها را خوب انتخاب کنیم شبکهٔ بیزِ فشرده‌ای خواهیم گرفت. اگر اتفاقاً ترتیبِ اشتباهی انتخاب کنیم چه می‌شود؟ مثالِ سرقت را باز در نظر بگیرید. فرض کنیم تصمیم می‌گیریم گره‌ها را به‌ترتیبِ Earthquake، Burglary، Alarm، JohnCalls، MaryCalls بیفزاییم. سپس شبکهٔ تا حدی پیچیده‌ترِ نشان‌داده‌شده در شکل ۱۳.۳(الف) را می‌گیریم. فرایند به‌صورتِ زیر است:

- **افزودن MaryCalls:** بدونِ والد.
- **افزودن JohnCalls:** اگر مری تماس بگیرد، احتمالاً آژیر به صدا درآمده، که احتمالِ تماسِ جان را بیشتر می‌کند. بنابراین، JohnCalls به MaryCalls به‌عنوانِ والد نیاز دارد.
- **افزودن Alarm:** بدیهی است، اگر هر دو تماس بگیرند، محتمل‌تر است آژیر به صدا درآمده باشد تا وقتی فقط یکی یا هیچ‌کدام تماس بگیرند، پس به هر دو MaryCalls و JohnCalls به‌عنوانِ والد نیاز داریم.
- **افزودن Burglary:** اگر حالتِ آژیر را بدانیم، تماسِ جان یا مری ممکن است به ما اطلاعاتی دربارهٔ زنگ‌خوردنِ تلفن‌مان یا موسیقیِ مری بدهد، اما نه دربارهٔ سرقت:

```
P(Burglary | Alarm, JohnCalls, MaryCalls) = P(Burglary | Alarm)
```

از این رو، فقط به Alarm به‌عنوانِ والد نیاز داریم.

- **افزودن Earthquake:** اگر آژیر روشن باشد، محتمل‌تر است زلزله‌ای رخ داده. (آژیر نوعی آشکارسازِ زلزله است.) اما اگر بدانیم سرقتی رخ داده، آن‌گاه این آژیر را تبیین می‌کند، و احتمالِ زلزله فقط اندکی بالاتر از حالتِ عادی خواهد بود. از این رو، به هر دو Alarm و Burglary به‌عنوانِ والد نیاز داریم.



شکل ۱۳.۳ – ساختارِ شبکه و شمارِ پارامترها به ترتیبِ معرفی بستگی دارد. (الف) ساختارِ حاصل با ترتیبِ M,J,A,B,E. (ب) ساختارِ حاصل با ترتیبِ M,J,E,B,A. هر گره با شمارِ پارامترهای لازم برچسب‌گذاری شده؛ در مجموع ۱۳ برای (الف) و ۳۱ برای (ب). در شکل ۱۳.۲ فقط ۱۰ پارامتر لازم بود.

شبکهٔ حاصل دو پیوندِ بیشتر از شبکهٔ اصلی در شکل ۱۳.۲ دارد و به ۱۳ احتمالِ شرطی به‌جای ۱۰ نیاز دارد. بدتر از آن، برخی پیوندها روابطِ سستی را بازنمایی می‌کنند که به قضاوت‌های احتمالاتیِ دشوار و غیرطبیعی نیاز دارند، مانند ارزیابیِ احتمالِ زلزله به‌شرطِ سرقت و آژیر. این پدیده کاملاً عمومی است و به تمایزِ میان مدل‌های علّی و تشخیصیِ معرفی‌شده در بخش ۱۲.۵.۱ مربوط است (همچنین تمرین WUMD.۱۳ را ببینید). اگر به مدلِ علّی بچسبیم، در نهایت مجبوریم اعدادِ کمتری مشخص کنیم، و اعداد اغلب آسان‌تر به دست می‌آیند. برای نمونه، در حوزهٔ پزشکی، تورْسکی و کانمن (1982) نشان داده‌اند پزشکانِ متخصص ترجیح می‌دهند قضاوتِ احتمالاتی برای قاعده‌های علّی بدهند تا تشخیصی. بخش ۱۳.۵ ایدهٔ مدل‌های علّی را عمیق‌تر بررسی می‌کند.

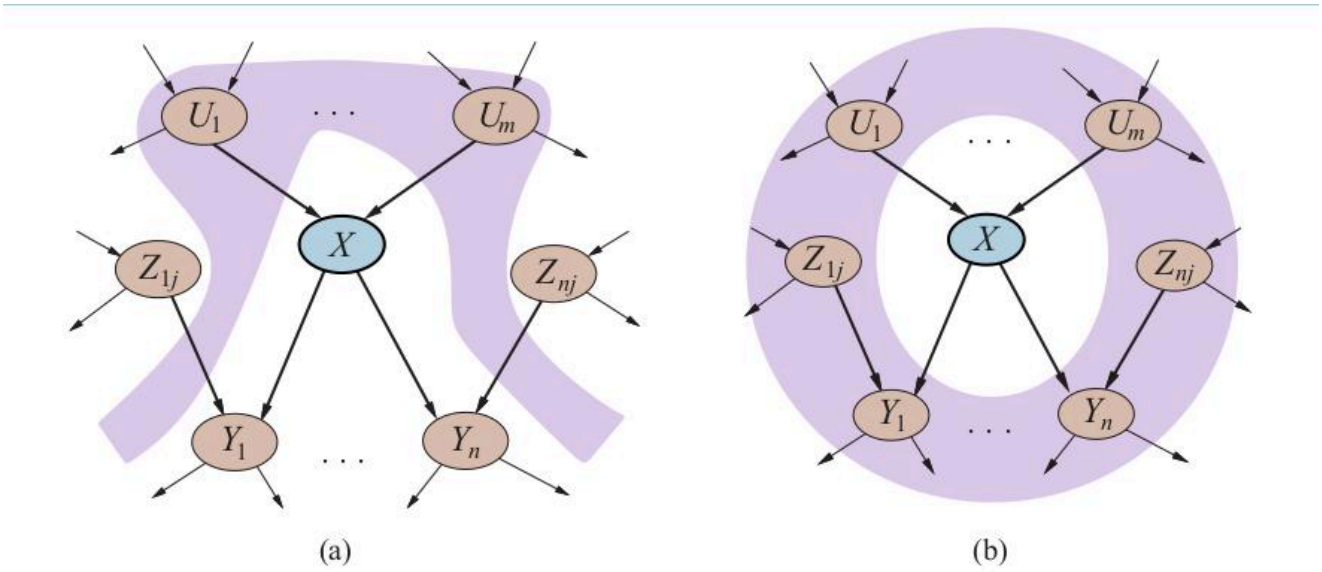
شکل ۱۳.۳(ب) ترتیبِ گره‌ایِ بسیار بدی را نشان می‌دهد: MaryCalls، JohnCalls، Earthquake، Burglary، Alarm. این شبکه به ۳۱ احتمالِ متمایز برای مشخص‌شدن نیاز دارد—دقیقاً همان شمارِ توزیعِ مشترکِ کامل. اما درکِ این نکته مهم است که هر سه شبکه می‌توانند دقیقاً همان توزیعِ مشترک را بازنمایی کنند. دو نسخهٔ شکل ۱۳.۳ صرفاً در

بازنماییِ همهٔ روابطِ استقلالِ شرطی شکست می‌خورند و از این رو به مشخص‌کردنِ شمارِ زیادی عددِ غیرضروری می‌انجامند.

۱۳.۲.۱ روابطِ استقلالِ شرطی در شبکه‌های بیزی

از معناشناسیِ شبکه‌های بیز، چنان‌که در معادلهٔ (13.2) تعریف شد، می‌توانیم شماری از ویژگی‌های استقلالِ شرطی را استخراج کنیم. قبلاً این ویژگی را دیده‌ایم که متغیر با توجه به والدهایش به‌صورتِ شرطی مستقل از دیگر پیشینیانش است. همچنین می‌توان ویژگیِ عمومی‌ترِ «غیرفرزندان» را اثبات کرد که:

هر متغیر با توجه به والدهایش به‌صورتِ شرطی مستقل از «غیرفرزندان» (Non-Descendants) خود است.



شکل ۱۳.۴ — (الف) گرهٔ X با توجه به والدهایش (Uiهای نشان‌داده‌شده در ناحیهٔ خاکستری) به‌صورتِ شرطی مستقل از غیرفرزندانش (مانند Zijها) است. (ب) گرهٔ X با توجه به «پوششِ مارکوف» (Markov Blanket) خود (ناحیهٔ خاکستری) به‌صورتِ شرطی مستقل از همهٔ گره‌های دیگرِ شبکه است.

برای نمونه، در شکل ۱۳.۲، متغیرِ JohnCalls با توجه به مقدارِ Alarm مستقل از Burglary، Earthquake و MaryCalls است. تعریف در شکل ۱۳.۴(الف) نشان داده شده.

معلوم می‌شود ویژگیِ غیرفرزندان، همراه با تفسیرِ پارامترهای شبکهٔ (θ(Xi | Parents(Xi به‌عنوانِ احتمال‌های شرطیِ (P(Xi | Parents(Xi، برای بازسازیِ توزیعِ مشترکِ کاملِ داده‌شده در معادلهٔ (13.2) کافی است. به بیانِ دیگر، می‌توان معناشناسیِ شبکه‌های بیز را به‌گونه‌ای دیگر دید: به‌جای تعریفِ توزیعِ مشترکِ کامل به‌عنوانِ حاصل‌ضربِ توزیع‌های شرطی، شبکه مجموعه‌ای از ویژگی‌های استقلالِ شرطی را تعریف می‌کند. توزیعِ مشترکِ کامل را می‌توان از آن ویژگی‌ها استخراج کرد.

ویژگیِ استقلالِ مهمِ دیگری از ویژگیِ غیرفرزندان دلالت می‌شود:

متغیر با توجه به والدها، فرزندان و والدهای فرزندانش—یعنی با توجه به «پوششِ مارکوف» (Markov Blanket) خود —به‌صورتِ شرطی مستقل از همهٔ گره‌های دیگرِ شبکه است.

(تمرین MARB.۱۳ از شما می‌خواهد این را اثبات کنید.) برای نمونه، متغیرِ Burglary با توجه به Alarm و Earthquake مستقل از JohnCalls و MaryCalls است. این ویژگی در شکل ۱۳.۴(ب) نشان داده شده. ویژگیِ پوششِ مارکوف الگوریتم‌های استنتاجی را ممکن می‌کند که از فرایندهای نمونه‌برداریِ تصادفیِ کاملاً محلی و توزیع‌شده استفاده می‌کنند، چنان‌که در بخش ۱۳.۴.۲ توضیح داده می‌شود.

عمومی‌ترین پرسشِ استقلالِ شرطی که ممکن است در شبکهٔ بیز پرسیده شود این است که آیا مجموعه‌ای از گره‌های X با توجه به مجموعهٔ سومی از Z به‌صورتِ شرطی مستقل از مجموعه‌ای دیگر از Y است؟ این را می‌توان با بررسیِ شبکهٔ بیز برای دیدنِ اینکه آیا Z بین X و Y «جداسازیِ-(D-Separation) d» انجام می‌دهد، کارآمد تعیین کرد. فرایند به‌صورتِ زیر است:

1. فقط «زیرگرافِ نیایی» (Ancestral Subgraph) متشکل از X، Y، Z و نیاکانشان را در نظر بگیر.
2. بین هر جفتِ گرهٔ بدونِ پیوند که فرزندِ مشترکی دارند پیوند بیفزا؛ حالا «گرافِ اخلاقی» (Moral Graph) موسوم را داریم.
3. همهٔ پیوندهای جهت‌دار را با پیوندهای بدونِ جهت جایگزین کن.
4. اگر Z همهٔ مسیرهای میان X و Y را در گرافِ حاصل مسدود کند، آنگاه Z بین X و Y جداسازیِ-d انجام می‌دهد. در آن صورت، X با توجه به Z به‌صورتِ شرطی مستقل از Y است. در غیرِ این صورت، شبکهٔ بیزِ اصلی استقلالِ شرطی را اقتضا نمی‌کند.

به‌طورِ خلاصه، جداسازیِ-d یعنی جداسازی در زیرگرافِ نیاییِ اخلاقی‌شدهٔ بدونِ جهت.

با اعمالِ تعریف بر شبکهٔ سرقت در شکل ۱۳.۲، می‌توانیم نتیجه بگیریم Burglary و Earthquake با توجه به مجموعهٔ خالی مستقل‌اند (یعنی به‌صورتِ مطلق مستقل‌اند)؛ با توجه به Alarm لزوماً به‌صورتِ شرطی مستقل نیستند؛ و JohnCalls و MaryCalls با توجه به Alarm به‌صورتِ شرطی مستقل‌اند. همچنین توجه کنید ویژگیِ پوششِ مارکوف مستقیماً از ویژگیِ جداسازیِ-d نتیجه می‌شود، زیرا پوششِ مارکوفِ یک متغیر آن را از همهٔ متغیرهای دیگر جداسازیِ-d می‌کند.

۱۳.۲.۲ بازنماییِ کارآمدِ توزیع‌های شرطی

حتی اگر شمارِ بیشینهٔ والدها k نسبتاً کوچک باشد، پرکردنِ CPT برای یک گره تا O(2^k) عدد و شاید تجربهٔ زیادی با همهٔ حالت‌های شرطیِ ممکن لازم دارد. در واقع، این یک سناریوی بدترین حالت است که در آن رابطهٔ میان والدها و فرزند کاملاً دلبخواه است. معمولاً چنین روابطی با «توزیعِ متعارف» (Canonical Distribution) توصیف‌شدنی‌اند که الگوی استانداردی را برازش می‌کند. در چنین مواردی، جدولِ کامل را می‌توان فقط با نامیدنِ الگو و شاید فراهم‌کردنِ چند پارامتر مشخص کرد.

ساده‌ترین مثال توسط «گره‌های قطعی» (Deterministic Nodes) فراهم می‌شود. گرهٔ قطعی مقدارش را دقیقاً با مقادیرِ والدهایش دارد، بدونِ هیچ عدم قطعیتی. رابطه می‌تواند منطقی باشد: برای نمونه، رابطهٔ میان گره‌های والدِ Canadian، US، Mexican و گرهٔ فرزندِ NorthAmerican صرفاً این است که فرزند فصلی از والدهاست. رابطه همچنین می‌تواند عددی باشد: برای نمونه، «بهترین قیمت» (BestPrice) برای خودرو کمینهٔ قیمت‌ها در هر نمایندگیِ منطقه است؛ و «آبِ ذخیره‌شده» (WaterStored) در مخزن در پایانِ سال، جمعِ مقدارِ اولیه، به‌علاوهٔ ورودی‌ها (رودها، رواناب، بارش) و منهای خروجی‌ها (رهاسازی، تبخیر، تراوش) است.

بسیاری از سامانه‌های شبکهٔ بیز به کاربر اجازه می‌دهند توابعِ قطعی را با زبانِ برنامه‌نویسیِ همه‌منظوره مشخص کند؛ این امکان را می‌دهد عناصرِ پیچیده‌ای مانند مدل‌های اقلیمِ جهانی یا شبیه‌سازهای شبکهٔ برق درونِ مدلِ احتمالاتی گنجانده شوند.

الگوی مهمِ دیگری که اغلب در عمل رخ می‌دهد «استقلالِ وابسته به بافت» (Context-Specific Independence) یا CSI است. توزیعِ شرطی اگر متغیری با توجه به مقادیرِ معینی از برخی والدها، به‌صورتِ شرطی مستقل از برخی دیگرِ والدهایش باشد، CSI نشان می‌دهد. برای نمونه، فرض کنیم «خسارتِ» (Damage) خودروی شما در دورهٔ زمانیِ معینی به «استحکامِ» (Ruggedness) خودرو و وقوع یا عدمِ وقوعِ «تصادفِ» (Accident) در آن دوره بستگی دارد. بدیهی است، اگر Accident کاذب باشد، آنگاه Damage، اگر باشد، به استحکامِ خودروی شما بستگی ندارد. (ممکن است خسارتِ خرابکاری به رنگ یا شیشه‌های خودرو وارد شود، اما فرض می‌کنیم همهٔ خودروها به‌یکسان در معرضِ چنین خسارتی‌اند.) می‌گوییم Damage به‌صورتِ وابسته به بافت مستقل از Ruggedness با توجه به Accident=false است. سامانه‌های شبکهٔ بیز اغلب CSI را با نحوِ if-then-else برای مشخص‌کردنِ توزیع‌های شرطی پیاده‌سازی می‌کنند؛ برای نمونه، ممکن است بنویسیم:

```
P(Damage | Ruggedness, Accident) = if (Accident = false) then d1 else d2(Ruggedness)
```

که در آن d1 و d2 توزیع‌های دلبخواه را بازنمایی می‌کنند. همانندِ قطعیت، وجودِ CSI در شبکه ممکن است استنتاجِ کارآمد را تسهیل کند. همهٔ الگوریتم‌های استنتاجِ دقیقِ ذکرشده در بخش ۱۳.۳ را می‌توان برای بهره‌گیری از CSI برای سرعت‌بخشیدن به محاسبه اصلاح کرد.

روابطِ نامطمئن اغلب با «روابطِ منطقیِ نویزی» (Noisy Logical Relationships) موسوم قابلِ مشخصه‌سازی‌اند. مثالِ استاندارد رابطهٔ «نویزی-OR» است، که تعمیمی از OR منطقی است. در منطقِ گزاره‌ای، ممکن است بگوییم تب (Fever) صادق است اگر و فقط اگر سرماخوردگی (Cold)، آنفولانزا (Flu) یا مالاریا (Malaria) صادق باشد. مدلِ نویزی-OR برای عدم قطعیت دربارهٔ توانایی هر والد در صادق‌شدنِ فرزند جا می‌گذارد—رابطهٔ علّی میان والد و فرزند ممکن است بازداشته شود، و بنابراین بیمار ممکن است سرماخوردگی داشته باشد اما تب نشان ندهد.

مدل دو فرض می‌کند. نخست، فرض می‌کند همهٔ علت‌های ممکن فهرست شده‌اند. (اگر برخی غایب‌اند، همیشه می‌توانیم «گرهٔ نشتیِ» (Leak Node) موسوم را بیفزاییم که «علت‌های متفرقه» را پوشش می‌دهد.) دوم، فرض می‌کند بازداریِ هر والد مستقل از بازداریِ هر والدِ دیگر است: برای نمونه، هرچه مالاریا را از ایجادِ تب بازمی‌دارد مستقل از آن

چیزی است که آنفولانزا را از ایجادِ تب بازمی‌دارد. با این فرض‌ها، Fever کاذب است اگر و فقط اگر همهٔ والدهای صادقش بازداشته شده باشند، و احتمالِ این برابرِ حاصل‌ضربِ احتمال‌های بازداریِ qj برای هر والد است. فرض کنیم این احتمال‌های بازداریِ فردی به‌صورتِ زیر باشند:

```
qcold = P(¬fever | cold, ¬flu, ¬malaria) = 0.6
  qflu = P(¬fever | ¬cold, flu, ¬malaria) = 0.2
qmalaria = P(¬fever | ¬cold, ¬flu, malaria) = 0.1
```

سپس، از این اطلاعات و فرض‌های نویزی-OR، کلِ CPT قابلِ ساخت است. قاعدهٔ عمومی این است که:

```
P(xi | parents(Xi)) = 1 − ∏ qj        (حاصل‌ضرب روی والدهای صادقِ آن ردیفِ CPT)
                      {j: Xj = true}
```

شکل ۱۳.۵ این محاسبه را نشان می‌دهد:

Cold	*Flu*	*Malaria*	$P(fever \mid \cdot)$	$P(\neg fever \mid \cdot)$
f	*f*	*f*	0.0	1.0
f	*f*	*t*	0.9	**0.1**
f	*t*	*f*	0.8	**0.2**
f	*t*	*t*	0.98	$0.02 = 0.2 \times 0.1$
t	*f*	*f*	0.4	**0.6**
t	*f*	*t*	0.94	$0.06 = 0.6 \times 0.1$
t	*t*	*f*	0.88	$0.12 = 0.6 \times 0.2$
t	*t*	*t*	0.988	$0.012 = 0.6 \times 0.2 \times 0.1$

شکل ۱۳.۵ — جدولِ احتمالِ شرطیِ کامل برای P(Fever|Cold,Flu,Malaria)، با فرضِ مدلِ نویزی-OR با سه مقدارِ q نشان‌داده‌شده به‌صورتِ پررنگ.

به‌طورِ کلی، روابطِ منطقیِ نویزی که در آن متغیری به k والد بستگی دارد، با O(k) پارامتر به‌جای O(2^k) برای جدولِ احتمالِ شرطیِ کامل توصیف می‌شوند. این ارزیابی و یادگیری را بسیار آسان‌تر می‌کند. برای نمونه، شبکهٔ CPCS (پرادهان و همکاران، 1994) از توزیع‌های نویزی-OR و نویزی-MAX برای مدل‌کردنِ روابط میان بیماری‌ها و نشانه‌ها در طب داخلی استفاده می‌کند. با ۴۴۸ گره و ۹۰۶ پیوند، فقط به ۸,۲۵۴ پارامتر نیاز دارد به‌جای ۱۳۳,۹۳۱,۴۳۰ برای شبکه‌ای با CPTهای کامل.

۱۳.۲.۳ شبکه‌های بیزی با متغیرهای پیوسته

بسیاری از مسائلِ دنیای واقعی شاملِ کمیت‌های پیوسته‌اند، مانند قد، جرم، دما و پول. طبقِ تعریف، متغیرهای پیوسته شمارِ نامتناهی مقدارِ ممکن دارند، پس مشخص‌کردنِ صریحِ احتمال‌های شرطی برای هر مقدار ناممکن است. یک راه برای مدیریتِ متغیرهای پیوسته «گسسته‌سازی» (Discretization) است—یعنی تقسیمِ مقادیرِ ممکن به

مجموعه‌ای ثابت از بازه‌ها. برای نمونه، دماها را می‌توان به سه رده تقسیم کرد: (<0°C)، (0°C تا 100°C) و (>100°C). در انتخابِ شمارِ رده‌ها، میانِ ازدست‌دادنِ دقت و CPTهای بزرگ که می‌توانند به زمان‌های اجرای کند بینجامند، معاوضه‌ای هست.

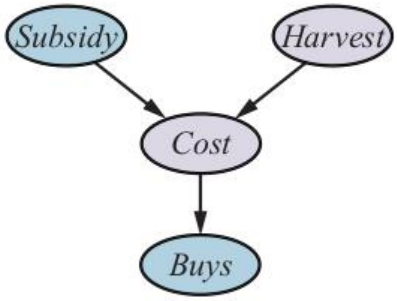
رویکردِ دیگر تعریفِ متغیرِ پیوسته با یکی از خانواده‌های استانداردِ توابعِ چگالیِ احتمال است (پیوستِ الف را ببینید). برای نمونه، توزیعِ گاوسی (یا نرمال) $N(x; \mu, \sigma^2)$ فقط با دو پارامتر، میانگینِ μ و واریانسِ σ^2، مشخص می‌شود. راهِ حلِ دیگر—که گاهی «بازنماییِ ناپارامتری» (Nonparametric Representation) نامیده می‌شود—تعریفِ ضمنیِ توزیعِ شرطی با مجموعه‌ای از نمونه‌هاست که هر یک مقادیرِ خاصِ متغیرهای والد و فرزند را دارد. این رویکرد را بیشتر در فصل ۱۹ بررسی می‌کنیم.

شبکه‌ای با هر دو متغیرِ گسسته و پیوسته «شبکهٔ بیزیِ ترکیبی» (Hybrid Bayesian Network) نامیده می‌شود. برای مشخص‌کردنِ شبکهٔ ترکیبی، باید دو نوع توزیعِ تازه مشخص کنیم: توزیعِ شرطی برای متغیرِ پیوسته با توجه به والدهای گسسته یا پیوسته؛ و توزیعِ شرطی برای متغیرِ گسسته با توجه به والدهای پیوسته. مثالِ سادهٔ شکل ۱۳.۶ را در نظر بگیرید، که در آن مشتری میوه‌ای می‌خرد بسته به هزینه‌اش، که به‌نوبهٔ خود به اندازهٔ برداشت و فعال‌بودنِ طرحِ یارانهٔ دولت بستگی دارد. متغیرِ Cost پیوسته است و والدهای پیوسته و گسسته دارد؛ متغیرِ Buys گسسته است و والدِ پیوسته دارد.

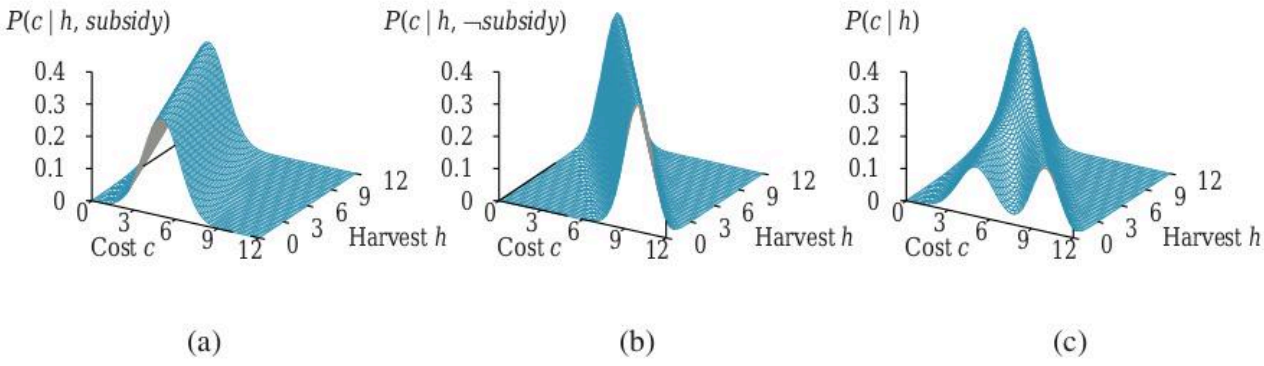
برای متغیرِ Cost، باید $P(\text{Cost} \mid \text{Harvest}, \text{Subsidy})$ را مشخص کنیم. والدِ گسسته با شمارش مدیریت می‌شود—یعنی با مشخص‌کردنِ هر دو $P(\text{Cost} \mid \text{Harvest}, \text{subsidy})$ و $P(\text{Cost} \mid \text{Harvest}, \neg\text{subsidy})$. برای مدیریتِ Harvest، مشخص می‌کنیم توزیع روی هزینهٔ c چگونه به مقدارِ پیوستهٔ h از Harvest بستگی دارد. به بیانِ دیگر، پارامترهای توزیعِ هزینه را به‌عنوانِ تابعی از h مشخص می‌کنیم. رایج‌ترین انتخاب «توزیعِ شرطیِ خطی–گاوسی» (Linear–Gaussian) است، که در آن فرزند توزیعِ گاوسی دارد که میانگینِ μ آن با مقدارِ والد به‌صورتِ خطی تغییر می‌کند و انحرافِ معیارِ σ آن ثابت است. به دو توزیع نیاز داریم، یکی برای subsidy و یکی برای ¬subsidy، با پارامترهای متفاوت:

$$P(c \mid h, \text{subsidy}) = N(c; a_t \cdot h + b_t, \sigma_t^2) = \frac{1}{\sigma_t\sqrt{2\pi}} \cdot e^{-\frac{1}{2}\left(\frac{c-(a_t \cdot h+b_t)}{\sigma_t}\right)^2}$$
$$P(c \mid h, \neg\text{subsidy}) = N(c; a_f \cdot h + b_f, \sigma_f^2) = \frac{1}{\sigma_f\sqrt{2\pi}} \cdot e^{-\frac{1}{2}\left(\frac{c-(a_f \cdot h+b_f)}{\sigma_f}\right)^2}$$

پس برای این مثال، توزیعِ شرطیِ Cost با نامیدنِ توزیعِ خطی–گاوسی و فراهم‌کردنِ پارامترهای a_t، b_t، σ_t، a_f، b_f و σ_f مشخص می‌شود. شکل‌های ۱۳.۷(الف) و (ب) این دو رابطه را نشان می‌دهند. توجه کنید در هر مورد شیبِ c بر حسبِ h منفی است، زیرا با بزرگ‌شدنِ برداشت هزینه کاهش می‌یابد. (البته، فرضِ خطی بودن دلالت دارد هزینه در نقطه‌ای منفی می‌شود؛ مدلِ خطی فقط اگر اندازهٔ برداشت به بازه‌ای باریک محدود باشد معقول است.) شکل ۱۳.۷(ج) توزیعِ $P(c|h)$ را نشان می‌دهد، با میانگین‌گیری روی دو مقدارِ ممکنِ Subsidy و فرضِ اینکه هر یک احتمالِ پیشینیِ 0.5 دارد. این نشان می‌دهد حتی با مدل‌های بسیار ساده، توزیع‌های کاملاً جالبی قابلِ بازنمایی‌اند.



شکل ۱۳.۶ — شبکهٔ ساده‌ای با متغیرهای گسسته (Subsidy و Buys) و متغیرهای پیوسته (Harvest و Cost).



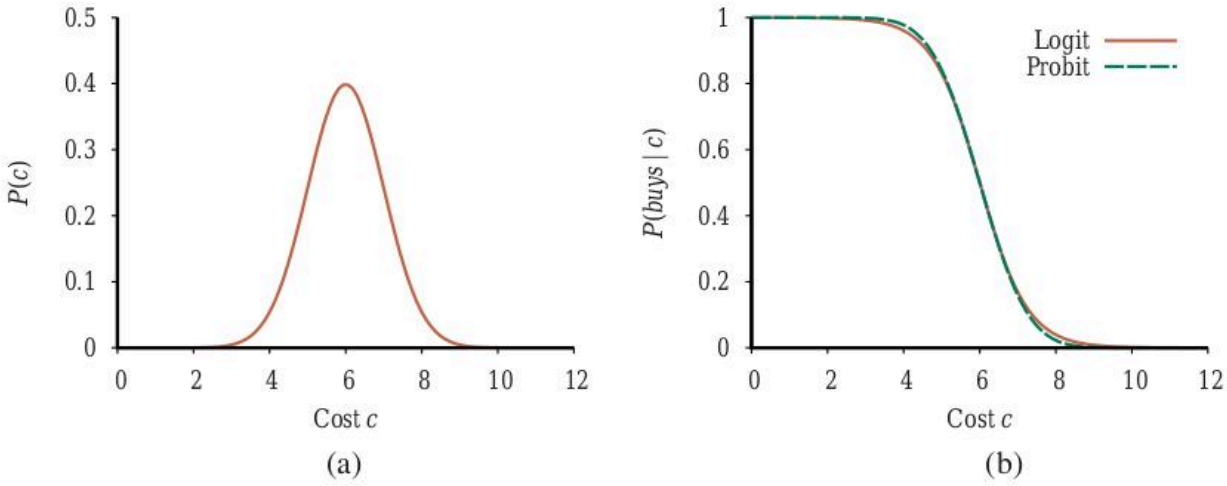
شکل ۱۳.۷ — نمودارهای (الف) و (ب) توزیع احتمال روی Cost را به‌عنوانِ تابعی از اندازهٔ Harvest نشان می‌دهند، با Subsidy به‌ترتیب صادق و کاذب. نمودار (ج) توزیعِ P(Cost|Harvest) را نشان می‌دهد، که با جمع‌زدن روی دو حالتِ یارانه به دست آمده.

توزیعِ شرطیِ خطی–گاوسی ویژگی‌های خاصی دارد. شبکه‌ای که فقط متغیرهای پیوسته با توزیع‌های خطی–گاوسی دارد، توزیعِ مشترکی دارد که «توزیعِ گاوسیِ چندمتغیره» (Multivariate Gaussian) روی همهٔ متغیرهاست (پیوستِ الف را ببینید؛ تمرین LGEX.۱۳). علاوه بر این، توزیعِ پسینی با توجه به هر شواهدی نیز این ویژگی را دارد.[۲] وقتی متغیرهای گسسته به‌عنوانِ والد (نه فرزند) متغیرهای پیوسته افزوده می‌شوند، شبکه توزیعِ «گاوسیِ شرطی» (Conditional Gaussian) یا CG تعریف می‌کند: با توجه به هر تخصیصی به متغیرهای گسسته، توزیع روی متغیرهای پیوسته گاوسیِ چندمتغیره است.

حالا به توزیع‌های متغیرهای گسسته با والدهای پیوسته می‌پردازیم. گرهٔ Buys در شکل ۱۳.۶ را در نظر بگیرید. معقول به نظر می‌رسد فرض کنیم مشتری اگر هزینه کم باشد می‌خرد و اگر زیاد باشد نمی‌خرد و احتمالِ خرید در منطقه‌ای میانی به‌صورتِ هموار تغییر می‌کند. به بیانِ دیگر، توزیعِ شرطی مانند تابعِ «آستانهٔ نرم» (Soft Threshold) است. یکی از راه‌های ساختِ آستانهٔ نرم استفاده از انتگرالِ توزیعِ نرمالِ استاندارد است:

$$\Phi(x) = \int_{-\infty}^{x} N(s; 0, 1)\, ds$$

از این نتیجه می‌شود استنتاج در شبکه‌های خطی–گاوسی در بدترین حالت فقط $O(n^3)$ زمان می‌برد، صرف‌نظر از توپولوژیِ شبکه. در بخش ۱۳.۳ می‌بینیم استنتاج برای شبکه‌های متغیرِ گسسته NP-سخت است.



شکل ۱۳.۸ — (الف) توزیعِ نرمال (گاوسی) برای آستانهٔ هزینه، متمرکز بر μ=6.0 با انحرافِ معیارِ σ=1.0. (ب) مدل‌های اکسپیت و پروبیت برای احتمالِ خرید به‌شرطِ هزینه، برای پارامترهای μ=6.0 و σ=1.0.

Φ(x) تابعی صعودی از x است، در حالی که احتمالِ خرید با هزینه کاهش می‌یابد، پس در این‌جا تابع را برمی‌گردانیم:

```
P(buys | Cost=c) = 1 − Φ((c − μ)/σ)
```

که بدان معناست آستانهٔ هزینه حدودِ μ رخ می‌دهد، پهنای ناحیهٔ آستانه متناسب با σ است، و احتمالِ خرید با افزایشِ هزینه کاهش می‌یابد. این «مدلِ پروبیت» (Probit؛ تلفظ «پرو-بیت»، مخففِ «واحدِ احتمال») در شکل ۱۳.۸(الف) نشان داده شده. شکل را می‌توان چنین توجیه کرد که فرایندِ تصمیمِ زیربنایی آستانه‌ای سخت دارد، اما مکانِ دقیقِ آستانه در معرضِ نویزِ تصادفیِ گاوسی است.

جایگزینِ مدلِ پروبیت «مدلِ اکسپیت» (Expit) یا «لاجیتِ معکوس» (Inverse Logit) است. از «تابعِ لجستیک» (Logistic Function) 1/(1+e^(-x)) برای تولیدِ آستانهٔ نرم استفاده می‌کند—هر x را به مقداری بین ۰ و ۱ نگاشت می‌کند. باز، برای مثالِ ما، آن را برمی‌گردانیم تا تابعی نزولی بسازیم؛ همچنین توان را در √/4(2π) مقیاس می‌کنیم تا با شیبِ پروبیت در میانگین هم‌خوان شود:

```
P(buys | Cost=c) = 1 − 1/(1 + exp(−(4/√(2π)) · (c − μ)/σ))
```

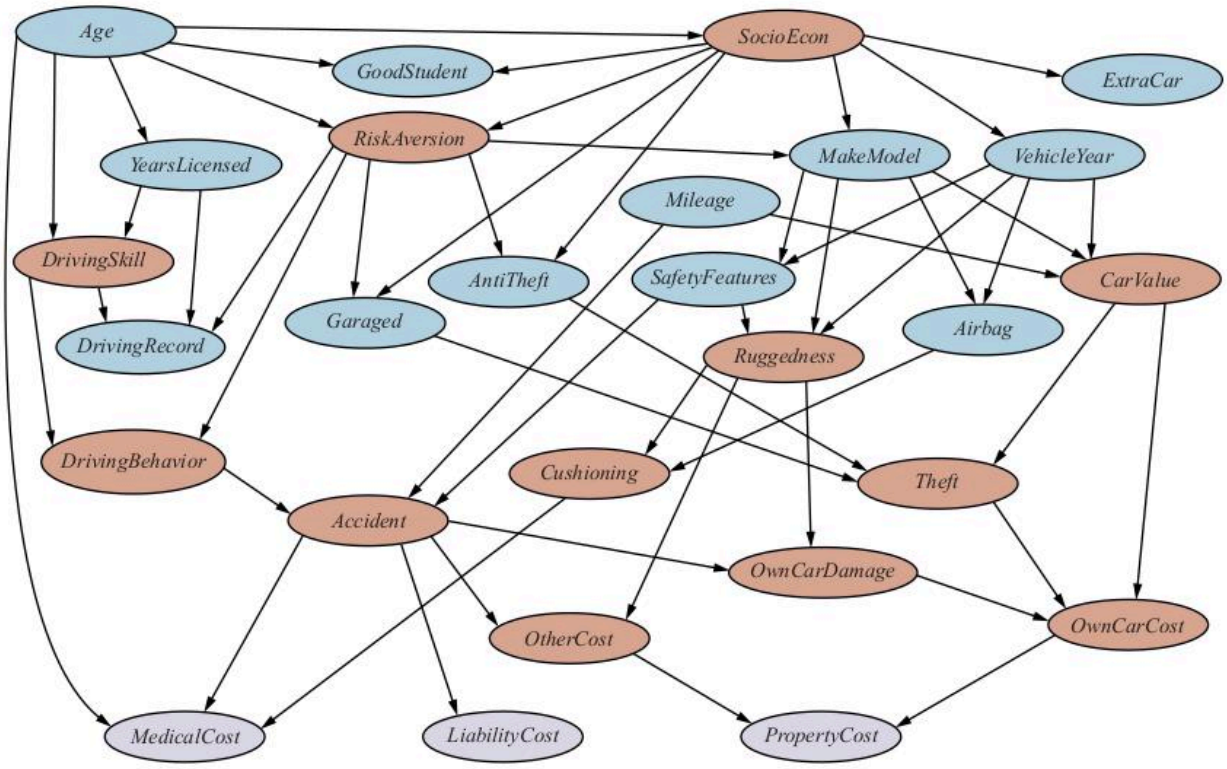
این در شکل ۱۳.۸(ب) نشان داده شده. دو توزیع مشابه به نظر می‌رسند، اما لاجیت در واقع «دنباله‌های» بسیار بلندتری دارد. پروبیت اغلب برازشِ بهتری برای موقعیت‌های واقعی است، اما تابعِ لجستیک گاهی از نظرِ ریاضی آسان‌تر است. در یادگیریِ ماشین به‌طورِ گسترده استفاده می‌شود. هر دو مدل را می‌توان با گرفتنِ ترکیبِ خطیِ مقادیرِ والدها برای مدیریتِ چند والدِ پیوسته تعمیم داد. این برای والدهای گسسته نیز کار می‌کند اگر مقادیرشان اعدادِ صحیح باشند؛ برای نمونه، با k والدِ بولی، که هر یک مقادیرِ 0 یا 1 دارد، ورودیِ توزیعِ اکسپیت یا پروبیت ترکیبِ خطیِ وزندار با k پارامتر خواهد بود، که مدلی کاملاً مشابهِ مدلِ نویزی-ORِ بحث‌شدهٔ پیشین به دست می‌دهد.

۱۳.۲.۴ مطالعهٔ موردی: بیمهٔ خودرو

شرکتِ بیمهٔ خودرو درخواستی از فردی برای بیمه‌کردنِ خودرویی خاص دریافت می‌کند و باید دربارهٔ حقِ بیمهٔ سالانهٔ مناسب تصمیم بگیرد، بر اساسِ خسارت‌های موردانتظاری که برای این متقاضی پرداخت خواهد کرد. تکلیف ساختِ شبکهٔ بیزی است که ساختارِ علّیِ حوزه را در بر بگیرد و توزیعی دقیق و به‌خوبی کالیبره‌شده روی متغیرهای خروجی با توجه به شواهدِ موجود در فرمِ درخواست ارائه دهد.[3] شبکهٔ بیزی شاملِ متغیرهای پنهان خواهد بود که نه متغیرِ ورودی‌اند و نه متغیرِ خروجی، اما برای ساختاردهی به شبکه به‌گونه‌ای که به‌طورِ معقولی تنک و با تعدادِ پارامترهای قابلِ‌مدیریت باشد ضروری‌اند. متغیرهای پنهان در شکل ۱۳.۹ با رنگِ قهوه‌ای سایه‌زده شده‌اند.

خسارت‌هایی که باید پرداخت شوند—در شکل ۱۳.۹ با رنگِ یاسی سایه‌زده شده‌اند—از سه نوع‌اند: هزینهٔ پزشکی (MedicalCost) برای هرگونه جراحتِ دیده‌شده توسط متقاضی؛ هزینهٔ مسئولیت (LiabilityCost) برای دعاویِ اقامه‌شده توسط دیگران علیه متقاضی و شرکتِ بیمه؛ و هزینهٔ اموال (PropertyCost) برای خسارتِ وارد به خودروی هر یک از طرفین و از دست رفتنِ خودرو در اثرِ سرقت. فرمِ درخواست اطلاعاتِ ورودیِ زیر را می‌پرسد (گره‌های آبیِ روشن در شکل ۱۳.۹):

- **دربارهٔ متقاضی:** سن (Age)؛ سال‌های دارایِ گواهینامه (YearsLicensed)—مدت‌زمانی که از نخستین دریافتِ گواهینامهٔ رانندگی می‌گذرد؛ سابقهٔ رانندگی (DrivingRecord)—خلاصه‌ای، احتمالاً بر پایهٔ «امتیازِ منفی»، از تصادف‌ها و تخلف‌های رانندگیِ اخیر؛ و (برای دانشجویان و دانش‌آموزان) شاخصِ دانش‌آموزِ ممتاز (GoodStudent) برای معدلِ ۳.۰ (B) در مقیاسِ ۴ نمره‌ای.
- **دربارهٔ خودرو:** مدل و سازنده (MakeModel) و سالِ ساخت (VehicleYear)؛ اینکه آیا کیسهٔ هوا (Airbag) دارد؛ و خلاصه‌ای از ویژگی‌های ایمنی (SafetyFeatures) مانند ترمزِ ضدقفل و هشدارِ برخورد.
- **دربارهٔ وضعیتِ رانندگی:** مسافتِ طی‌شدهٔ سالانه (Mileage) و اینکه خودرو با چه میزان امنیت در پارکینگ یا گاراژ پارک می‌شود (Garaged)، اگر اصلاً پارکینگی باشد؛ و خودرویِ اضافه (ExtraCar)—اینکه آیا متقاضی وسیلهٔ نقلیهٔ دیگری برای رفت‌وآمدِ روزمره دارد یا خیر.



شکل ۱۳.۹ – شبکهٔ بیزی برای ارزیابیِ درخواست‌های بیمهٔ خودرو.

اکنون باید بیاندیشیم چگونه این متغیرها را در یک ساختارِ علّی سازمان‌دهی کنیم. متغیرهای پنهانِ کلیدی این هستند که آیا سرقت (Theft) یا تصادفی (Accident) در دورهٔ زمانیِ بعد رخ خواهد داد یا خیر. بدیهی است نمی‌توان از متقاضی خواست این‌ها را پیش‌بینی کند؛ بلکه باید از اطلاعاتِ موجود و تجربهٔ پیشینِ بیمه‌گر استنباط شوند.

عواملِ علّیِ منتهی به سرقت (Theft) کدام‌اند؟ قطعاً MakeModel مهم است—برخی مدل‌ها بسیار بیشتر از بقیه سرقت می‌شوند زیرا بازارِ فروشِ مجددِ کارآمدی برای خودروها و قطعاتِ یدکی دارند؛ CarValue (ارزشِ خودرو) نیز مهم است، زیرا خودروی فرسوده، کهنه یا با کارکردِ بالا ارزشِ فروشِ مجددِ کمتری دارد. افزون بر این، خودرویی که در گاراژ پارک می‌شود (Garaged) و دستگاهِ ضدسرقت (AntiTheft) دارد، سرقتش دشوارتر است. متغیرِ پنهانِ CarValue به‌نوبهٔ خود به MakeModel، سالِ ساخت (VehicleYear) و کارکرد (Mileage) بستگی دارد. همچنین CarValue میزانِ خسارتِ وارده را هنگامِ وقوعِ سرقت دیکته می‌کند، پس یکی از عواملِ تشکیل‌دهندهٔ OwnCarCost (هزینهٔ خودرویِ خود) است (عاملِ دیگر تصادف است که اندکی بعد به آن خواهیم پرداخت).

در مدل‌هایی از این دست متداول است که متغیرِ پنهانِ دیگری به نام SocioEcon (طبقهٔ اجتماعی-اقتصادیِ متقاضی) معرفی شود. پنداشته می‌شود که این متغیر بر دامنهٔ گسترده‌ای از رفتارها و ویژگی‌ها اثر می‌گذارد. در مدلِ ما، هیچ شاهدِ مستقیمی در قالبِ متغیرهای مشاهده‌شدهٔ درآمد و شغل وجود ندارد؛[4] اما SocioEcon بر MakeModel و VehicleYear اثر می‌گذارد؛ همچنین بر ExtraCar و GoodStudent تأثیر دارد، و تا حدی به سن (Age) وابسته است.

برای هر شرکتِ بیمه‌ای، شاید مهم‌ترین متغیرِ پنهان RiskAversion (ریسک‌گریزی) باشد: افرادی که ریسک‌گریزند ریسکِ بیمه‌ایِ خوبی‌اند! Age و SocioEcon بر RiskAversion اثر می‌گذارند، و «نشانه‌های» آن شاملِ انتخابِ متقاضی است که آیا خودرو در گاراژ پارک می‌شود (Garaged) و آیا دستگاه‌های ضدسرقت (AntiTheft) و ویژگی‌های ایمنی (SafetyFeatures) دارد.

در پیش‌بینیِ تصادف‌های آینده، کلید، «رفتارِ رانندگی» (DrivingBehavior) آیندهٔ متقاضی است، که هم از RiskAversion و هم از «مهارتِ رانندگی» (DrivingSkill) متأثر است؛ دومی به‌نوبهٔ خود به Age و «سال‌هایِ گواهینامه» (YearsLicensed) بستگی دارد. رفتارِ رانندگیِ گذشتهٔ متقاضی در «سابقهٔ رانندگی» (DrivingRecord) بازتاب می‌یابد، که همچنین به RiskAversion و DrivingSkill و نیز YearsLicensed بستگی دارد (زیرا کسی که تازه رانندگی را شروع کرده ممکن است فرصتِ انباشتنِ فهرستِ بلندی از تصادف‌ها و تخلف‌ها را نداشته باشد). بدین‌ترتیب، DrivingRecord شواهدی دربارهٔ RiskAversion و DrivingSkill فراهم می‌کند، که به‌نوبهٔ خود به پیش‌بینیِ DrivingBehavior آینده کمک می‌کنند.

می‌توانیم DrivingBehavior را گرایشِ سرانهٔ هر کیلومتر به رانندگیِ مستعدِ تصادف بدانیم؛ اینکه آیا در دورهٔ زمانیِ ثابتی واقعاً «تصادفی» (Accident) رخ می‌دهد نیز به «مسافتِ سالانه» (Mileage) و «ویژگی‌های ایمنی» (SafetyFeatures) خودرو بستگی دارد. اگر تصادفی رخ دهد، سه نوع هزینه هست: «هزینهٔ پزشکی» (MedicalCost) برای متقاضی به Age و «محافظت» (Cushioning) بستگی دارد، که به‌نوبهٔ خود به «استحکام» (Ruggedness) خودرو و داشتنِ «کیسهٔ هوا» (Airbag) بستگی دارد؛ «هزینهٔ مسئولیت» (LiabilityCost) (پزشکی، درد و رنج، ازدست‌رفتنِ درآمد و...) برای رانندهٔ دیگر؛ و «هزینهٔ اموال» (PropertyCost) برای متقاضی و رانندهٔ دیگر، که هر دو (به‌شیوه‌های متفاوت) به استحکامِ خودرو و «ارزشِ خودرویِ» (CarValue) متقاضی بستگی دارند.

نوعِ استدلالی را نشان داده‌ایم که به توسعهٔ توپولوژی و متغیرهای پنهان در شبکهٔ بیز می‌رود. همچنین باید بردها و توزیع‌های شرطی را برای هر متغیر مشخص کنیم. برای بردها، تصمیمِ اصلی اغلب این است که متغیر را گسسته کنیم یا پیوسته. برای نمونه، استحکامِ خودرو می‌تواند متغیرِ پیوسته‌ای بین ۰ و ۱ باشد، یا متغیرِ گسسته‌ای با بردِ {TinCan, Normal, Tank} (قوطی‌حلبی، معمولی، تانک).

برخی شرکت‌های بیمه همچنین سابقهٔ اعتباریِ متقاضی را برای کمک به ارزیابیِ ریسک به دست می‌آورند؛ این اطلاعاتِ بسیار بیشتری دربارهٔ ردهٔ اقتصادی-اجتماعی فراهم می‌کند. هرگاه از این نوع متغیرهای پنهان استفاده می‌کنیم، باید مراقب باشیم ناخواسته به جانشینِ متغیرهایی مانند نژاد تبدیل نشوند که ممکن است در تصمیم‌های بیمه استفاده نشوند. فنونِ اجتناب از این نوع سوگیری‌ها در فصل ۱۹ توصیف شده‌اند.

متغیرهای پیوسته دقتِ بیشتری فراهم می‌کنند، اما استنتاجِ دقیق را به‌جز در چند موردِ خاص ناممکن می‌کنند. متغیرِ گسسته با مقادیرِ ممکنِ بسیار می‌تواند پرکردنِ جدول‌های احتمالِ شرطیِ متناظرِ بزرگ را ملال‌آور کند و استنتاجِ دقیق را گران‌تر کند، مگر اینکه مقدارِ متغیر همیشه مشاهده شود. برای نمونه، MakeModel در سامانهٔ واقعی هزاران مقدارِ ممکن خواهد داشت، و این باعث می‌شود فرزندش CarValue CPT عظیمی داشته باشد که باید از پایگاه‌داده‌های صنعت پر شود؛ اما، چون MakeModel همیشه مشاهده می‌شود، این به پیچیدگیِ استنتاج نمی‌افزاید: در واقع، مقادیرِ مشاهده‌شدهٔ سه والد دقیقاً یک ردیفِ مربوط از CPT_CarValue را برمی‌گزینند. توزیع‌های شرطیِ مدل در

مخزنِ کدِ کتاب داده شده‌اند؛ نسخه‌ای فقط با متغیرهای گسسته فراهم می‌کنیم که استنتاجِ دقیق برایش قابلِ انجام است. در عمل، بسیاری از متغیرها پیوسته خواهند بود و توزیع‌های شرطی از داده‌های تاریخیِ متقاضیان و خسارت‌های بیمه‌شان یاد گرفته می‌شوند. خواهیم دید چگونه مدل‌های شبکهٔ بیز را از داده‌ها در فصل ۲۰ یاد بگیریم.

پرسشِ نهایی، البته، این است که چگونه در شبکه برای پیش‌بینی استنتاج کنیم. حالا به این پرسش می‌پردازیم. برای هر روشِ استنتاجی که توصیف می‌کنیم، روش را روی شبکهٔ بیمه ارزیابی می‌کنیم تا نیازهای زمانی و فضایی روش را بسنجیم.

۱۳.۳ استنتاجِ دقیق در شبکه‌های بیزی

تکلیفِ پایهٔ هر سامانهٔ استنتاجِ احتمالاتی محاسبهٔ توزیعِ احتمالِ پسینی برای مجموعه‌ای از متغیرهای پرس‌وجو، با توجه به «رخدادِ» (Event) مشاهده‌شده‌ای—معمولاً تخصیصی از مقادیر به مجموعه‌ای از متغیرهای شاهد—است.[5] برای ساده‌کردنِ ارائه، فقط یک متغیرِ پرس‌وجو را در هر زمان در نظر می‌گیریم؛ الگوریتم‌ها را می‌توان به‌آسانی به پرس‌وجوهای چندمتغیره بسط داد. (برای نمونه، می‌توانیم پرس‌وجوی P(U,V|e) را با ضربِ P(V|e) در P(U|V,e) حل کنیم.) از نمادِ فصل ۱۲ استفاده می‌کنیم: X متغیرِ پرس‌وجو را نشان می‌دهد؛ E مجموعهٔ متغیرهای شاهدِ E1,...,Em را نشان می‌دهد و e رخدادِ مشاهده‌شدهٔ خاصی است؛ Y متغیرهای پنهان (غیرشاهد، غیرپرس‌وجو) Y1,...,Yℓ را نشان می‌دهد. بدین‌ترتیب، مجموعهٔ کاملِ متغیرها {X}∪Y∪E است. پرس‌وجوی نمونه‌ای توزیعِ احتمالِ پسینیِ P(X|e) را می‌پرسد.

در شبکهٔ سرقت، ممکن است رخدادی را مشاهده کنیم که در آن JohnCalls=true و MaryCalls=true. سپس می‌توانیم مثلاً بپرسیم احتمالِ وقوعِ سرقت چقدر است:

```
P(Burglary | JohnCalls=true, MaryCalls=true) = ⟨0.284, 0.716⟩
```

در این بخش الگوریتم‌های دقیقِ محاسبهٔ احتمال‌های پسینی و نیز پیچیدگیِ این تکلیف را بحث می‌کنیم. معلوم می‌شود حالتِ عمومی ناشدنی است، پس بخش ۱۳.۴ روش‌های استنتاجِ تقریبی را پوشش می‌دهد.

۱۳.۳.۱ استنتاج با شمارش

فصل ۱۲ توضیح داد هر احتمالِ شرطی را می‌توان با جمع‌زدنِ جمله‌ها از توزیعِ مشترکِ کامل محاسبه کرد. به‌طورِ خاص‌تر، پرس‌وجوی P(X|e) را می‌توان با معادلهٔ (12.9) پاسخ داد، که برای راحتی این‌جا تکرار می‌کنیم:

```
P(X|e) = α P(X, e) = α ∑y P(X, e, y)
```

حالا، شبکهٔ بیز بازنماییِ کاملی از توزیعِ مشترکِ کامل می‌دهد. به‌طورِ خاص‌تر، معادلهٔ (13.2) در صفحهٔ ۴۱۵ نشان می‌دهد جمله‌های P(x,e,y) در توزیعِ مشترک را می‌توان به‌صورتِ حاصل‌ضربِ احتمال‌های شرطیِ شبکه نوشت. بنابراین، پرس‌وجو را می‌توان با شبکهٔ بیز با محاسبهٔ جمعِ حاصل‌ضرب‌های احتمال‌های شرطیِ شبکه پاسخ داد. پرس‌وجوی P(Burglary | JohnCalls=true, MaryCalls=true) را در نظر بگیرید. متغیرهای پنهانِ این پرس‌وجو Earthquake و Alarm هستند. از معادلهٔ (12.9)، با استفاده از حرفِ نخستِ متغیرها برای کوتاه‌کردنِ عبارت‌ها:

```
P(B|j, m) = α P(B, j, m) = α ∑e ∑a P(B, j, m, e, a)
```

معناشناسیِ شبکه‌های بیز (معادلهٔ (13.2)) سپس عبارتی بر حسبِ درایه‌های CPT به ما می‌دهد. برای سادگی، این را فقط برای Burglary=true انجام می‌دهیم:

```
P(b|j, m) = α ∑e ∑a P(b) P(e) P(a|b, e) P(j|a) P(m|a)        (13.4)
```

برای محاسبهٔ این عبارت، باید چهار جمله بیفزاییم، که هر یک با ضربِ پنج عدد محاسبه می‌شود. در بدترین حالت، که در آن باید تقریباً همهٔ متغیرها را حذف کنیم، $O(2^n)$ جمله در جمع خواهد بود، که هر یک حاصل‌ضربِ $O(n)$ مقدارِ احتمال است. پیاده‌سازیِ اولیه بنابراین پیچیدگیِ $O(n2^n)$ خواهد داشت.

این را می‌توان با بهره‌گیری از ساختارِ تودرتوِ محاسبه به $O(2^n)$ کاهش داد. در بیانِ نمادین، این یعنی جابه‌جا کردنِ جمع‌ها به درون، تا آن‌جا که ممکن است، در عبارت‌هایی مانند معادلهٔ (13.4). می‌توانیم این کار را بکنیم، زیرا همهٔ عامل‌های حاصل‌ضربِ احتمال‌ها به همهٔ متغیرها وابسته نیستند. بدین‌ترتیب:

```
P(b|j, m) = α P(b) ∑e P(e) ∑a P(a|b, e) P(j|a) P(m|a)        (13.5)
```

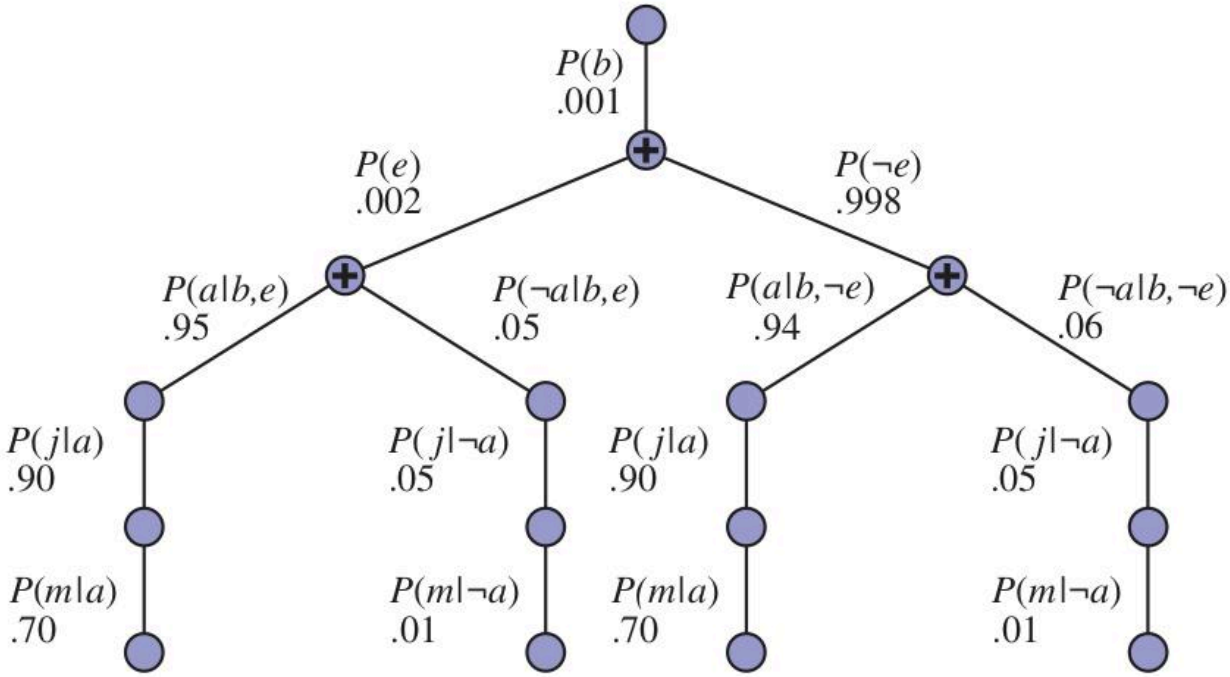
این عبارت را می‌توان با حلقه‌زدن روی متغیرها به‌ترتیب و ضربِ درایه‌های CPT در طولِ مسیر ارزیابی کرد. برای هر جمع، همچنین باید روی مقادیرِ ممکنِ متغیر حلقه بزنیم. ساختارِ این محاسبه در شکل ۱۳.۱۰ به‌صورتِ درخت نشان داده شده. با اعدادِ شکل ۱۳.۲، P(b|j,m) = α × 0.00059224 به دست می‌آوریم. محاسبهٔ متناظر برای ¬b مقدارِ α × 0.0014919 می‌دهد؛ از این رو:

```
P(B|j, m) = α ⟨0.00059224, 0.0014919⟩ ≈ ⟨0.284, 0.716⟩
```

یعنی شانسِ سرقت، با توجه به تماسِ هر دو همسایه، حدود ۲۸٪ است.

الگوریتمِ ENUMERATION-ASK در شکل ۱۳.۱۱ این درخت‌های عبارت را با بازگشتِ عمق-اولِ چپ-به-راست ارزیابی می‌کند. الگوریتم از نظرِ ساختار بسیار شبیهِ الگوریتمِ پس‌گرد برای حلِ CSPها (شکل ۶.۵) و الگوریتمِ DPLL برای ارضاپذیری (شکل ۷.۱۷) است. پیچیدگیِ فضایی‌اش فقط در شمارِ متغیرها خطی است: الگوریتم روی توزیعِ مشترکِ کامل جمع می‌زند بدون آنکه هرگز آن را صریحاً بسازد. متأسفانه، پیچیدگیِ زمانی‌اش برای شبکه‌ای با n متغیرِ بولی (بدونِ شمردنِ متغیرهای شاهد) همیشه $O(2^n)$ است—بهتر از $O(n2^n)$ رویکردِ سادهٔ توصیف‌شدهٔ پیشین، اما همچنان

نسبتاً ناخوشایند. برای شبکهٔ بیمه در شکل ۱۳.۹، که نسبتاً کوچک است، استنتاجِ دقیق با شمارش برای پرس‌وجوی نمونه‌ای روی متغیرهای هزینه حدود ۲۲۷ میلیون عملیاتِ حسابی لازم دارد. اگر به درختِ شکل ۱۳.۱۰ با دقت نگاه کنید، با این حال، خواهید دید شاملِ زیرعبارت‌های تکراری است. حاصل‌ضرب‌های P(j|a)P(m|a) و P(j|¬a)P(m|¬a) دو بار محاسبه می‌شوند، یک بار برای هر مقدارِ E. کلیدِ استنتاجِ کارآمد در شبکه‌های بیز پرهیز از چنین محاسبه‌های هدررفته‌ای است. بخشِ بعد روشی عمومی برای این کار توصیف می‌کند.



شکل ۱۳.۱۰ — ساختارِ عبارتِ نشان‌داده‌شده در معادلهٔ (13.5). ارزیابی از بالا به پایین پیش می‌رود، مقادیر را در طولِ هر مسیر ضرب می‌کند و در گره‌های «+» جمع می‌زند. تکرارِ مسیرهای j و m را توجه کنید.

```
function ENUMERATION-ASK(X, e, bn) returns a distribution over X
                                                  inputs: X, the query variable
                                      e, observed values for the variables E
                                  bn, a Bayesian network with variables vars
                                       Q(X) ← a distribution over X, initially empty
                                                                  :for each value xi of X
Q(xi) ← ENUMERATE-ALL(vars, exi)      where exi is the same as e with X = xi added
                                                                   return NORMALIZE(Q(X))

                                  function ENUMERATE-ALL(vars, e) returns a real number
                                                       if EMPTY?(vars) then return 1.0
                                                                          V ← FIRST(vars)
                                                  if V is an evidence variable with value v in e
                                      then return P(v | parents(V)) × ENUMERATE-ALL(REST(vars), e)
                                  else return ∑v P(v | parents(V)) × ENUMERATE-ALL(REST(vars), ev)
                                                  where ev is the same as e with V = v added
```

شکل ۱۳.۱۱ — الگوریتمِ شمارش برای استنتاجِ دقیق در شبکه‌های بیز.

۱۳.۳.۲ الگوریتمِ حذفِ متغیر

الگوریتمِ شمارش را می‌توان با حذفِ محاسبه‌های تکراریِ از نوعِ نشان‌داده‌شده در شکل ۱۳.۱۰ به‌طورِ قابلِ‌ملاحظه‌ای بهبود داد. ایده ساده است: محاسبه را یک بار انجام بده و نتایج را برای استفادهٔ بعدی ذخیره کن. این نوعی «برنامه‌نویسیِ پویا» (Dynamic Programming) است. چند نسخه از این رویکرد هست؛ ما «الگوریتمِ حذفِ متغیر» (Variable Elimination) را ارائه می‌کنیم که ساده‌ترین است. حذفِ متغیر با ارزیابیِ عبارت‌هایی مانند معادلهٔ (13.5) در ترتیبِ راست-به-چپ (یعنی از پایین به بالا در شکل ۱۳.۱۰) کار می‌کند. نتایجِ میانی ذخیره می‌شوند و جمع‌زدن روی هر متغیر فقط برای آن بخش‌هایی از عبارت انجام می‌شود که به متغیر وابسته‌اند. این فرایند را برای شبکهٔ سرقت نشان می‌دهیم. عبارتِ P(B|j,m) را ارزیابی می‌کنیم:

```
                                P(B|j,m) = α P(B) ∑e P(e) ∑a P(a|B,e) P(j|a) P(m|a)
                        −f1(B)−  −f2(E)−  ——f3(A,B,E)——  −f4(A)−  −f5(A)−
```

توجه کنید هر بخش از عبارت را با نامِ «عاملِ» (Factor) متناظرش برچسب‌گذاری کرده‌ایم؛ هر عامل ماتریسی است که با مقادیرِ متغیرهای آرگومانش نمایه می‌شود. برای نمونه، عامل‌های f4(A) و f5(A) متناظر با P(j|a) و P(m|a) فقط به A وابسته‌اند، زیرا J و M توسط پرس‌وجو ثابت شده‌اند. آن‌ها بنابراین بردارهای دوعنصری‌اند:

```
                                f4(A) = ⟨P(j|a), P(j|¬a)⟩ = ⟨0.90, 0.05⟩
                                f5(A) = ⟨P(m|a), P(m|¬a)⟩ = ⟨0.70, 0.01⟩
```

f3(A,B,E) ماتریسِ 2×2×2 خواهد بود، که نشان‌دادنش در صفحهٔ چاپی دشوار است. (عنصرِ «نخست» با P(a|b,e)=0.95 و «آخرین» با P(¬a|¬b,¬e)=0.999 داده می‌شود.) بر حسبِ عامل‌ها، عبارتِ پرس‌وجو چنین نوشته می‌شود:

```
                                P(B|j,m) = α f1(B) × ∑e f2(E) × ∑a f3(A,B,E) × f4(A) × f5(A)
```

در این‌جا عملگرِ «×» ضربِ معمولیِ ماتریسی نیست، بلکه عملِ «ضربِ نقطه‌ای» (Pointwise Product) است، که به‌زودی توصیف خواهد شد.

فرایندِ ارزیابی متغیرها را (راست به چپ) از حاصل‌ضرب‌های نقطه‌ایِ عامل‌ها خارج می‌کند تا عامل‌های تازه تولید شوند، و سرانجام عاملی به دست می‌آید که راه‌حل را می‌سازد—یعنی توزیعِ پسینی روی متغیرِ پرس‌وجو. گام‌ها به‌صورتِ زیرند:

- نخست، A را از حاصل‌ضربِ f4 ،f3 و f5 خارج می‌کنیم. این عاملِ تازهٔ 2×2 به نام f6(B,E) می‌دهد که نمایه‌هایش فقط روی B و E می‌روند:

```
                                f6(B,E) = ∑a f3(A,B,E) × f4(A) × f5(A)
                        (f3(¬a,B,E) × f4(¬a) × f5(¬a)) + (f3(a,B,E) × f4(a) × f5(a)) =
```

حالا با عبارتِ P(B|j,m) = α f1(B) × ∑e f2(E) × f6(B,E) رها شده‌ایم.

- بعد، E را از حاصل‌ضربِ f2 و f6 خارج می‌کنیم:

```
f7(B) = ∑e f2(E) × f6(B,E) = f2(e) × f6(B,e) + f2(¬e) × f6(B,¬e)
```

این عبارتِ P(B|j,m) = α f1(B) × f7(B) را باقی می‌گذارد، که می‌توان با گرفتنِ ضربِ نقطه‌ای و نرمال‌سازیِ نتیجه ارزیابی کرد.

X	Y	$\mathbf{f}(X,Y)$	Y	Z	$\mathbf{g}(Y,Z)$	X	Y	Z	$\mathbf{h}(X,Y,Z)$
t	t	.3	t	t	.2	t	t	t	$.3 \times .2 = .06$
t	f	.7	t	f	.8	t	t	f	$.3 \times .8 = .24$
f	t	.9	f	t	.6	t	f	t	$.7 \times .6 = .42$
f	f	.1	f	f	.4	t	f	f	$.7 \times .4 = .28$
						f	t	t	$.9 \times .2 = .18$
						f	t	f	$.9 \times .8 = .72$
						f	f	t	$.1 \times .6 = .06$
						f	f	f	$.1 \times .4 = .04$

شکل ۱۳.۱۲ — نشان‌دادنِ ضربِ نقطه‌ای: f(X,Y) × g(Y,Z) = h(X,Y,Z). (جدول: برای هر ترکیب از مقادیرِ X, Y, Z، h = f × g؛ برای نمونه h(t,t,t) = 0.3 × 0.2 = 0.06.)

با بررسیِ این توالی، می‌بینیم دو عملیاتِ محاسباتیِ پایه لازم است: ضربِ نقطه‌ایِ جفتِ عامل‌ها، و خارج‌کردنِ متغیر از حاصل‌ضربِ عامل‌ها. بخشِ بعد هر یک از این عملیات‌ها را توصیف می‌کند.

توجه کنید ماتریس‌ها ضرب نمی‌شوند تا وقتی لازم شود متغیری از حاصل‌ضربِ انباشته خارج کنیم. در آن نقطه، فقط آن ماتریس‌هایی را ضرب می‌کنیم که شاملِ متغیرِ موردِ خارج‌کردن‌اند. با توابعِ ضربِ نقطه‌ای و خارج‌کردن، خودِ الگوریتمِ حذفِ متغیر را می‌توان بسیار ساده نوشت، چنان‌که در شکل ۱۳.۱۳ نشان داده شده.

ترتیبِ متغیر و مربوط‌بودنِ متغیر. الگوریتمِ شکل ۱۳.۱۳ شاملِ تابعِ مشخص‌نشدهٔ ORDER برای انتخابِ ترتیبِ متغیرهاست. هر انتخابِ ترتیبی الگوریتمِ معتبری می‌دهد، اما ترتیب‌های متفاوت باعث تولیدِ عامل‌های میانیِ متفاوتی در طولِ محاسبه می‌شوند. برای نمونه، در محاسبهٔ نشان‌داده‌شدهٔ پیشین، A را قبل از E حذف کردیم؛ اگر به‌صورتِ معکوس انجامش دهیم، محاسبه به‌صورتِ P(B|j,m) = α f1(B) ∑a f4(A) × f5(A) ∑e f2(E) × f3(A,B,E) می‌شود، که در طولِ آن عاملِ تازهٔ f6(A,B) تولید خواهد شد.

به‌طورِ کلی، نیازهای زمانی و فضاییِ حذفِ متغیر تحتِ سیطرهٔ اندازهٔ بزرگ‌ترین عاملی است که در طولِ عملکردِ الگوریتم ساخته می‌شود. این به‌نوبهٔ خود با ترتیبِ حذفِ متغیرها و ساختارِ شبکه تعیین می‌شود. معلوم می‌شود تعیینِ ترتیبِ بهینه ناشدنی است، اما چند اکتشافیِ خوب در دسترس‌اند. یکی از روش‌های نسبتاً مؤثر روشِ حریصانه است: متغیری را حذف کن که اندازهٔ عاملِ بعدیِ ساخته‌شده را کمینه می‌کند.

یک پرس‌وجوی دیگر را در نظر بگیریم: P(JohnCalls | Burglary=true). مثلِ همیشه (معادلهٔ (13.5) را ببینید)، گامِ نخست نوشتنِ جمعِ تودرتو است:

```
P(J|b) = α P(b) ∑e P(e) ∑a P(a|b,e) P(J|a) ∑m P(m|a)
```

با ارزیابیِ این عبارت از راست به چپ، نکتهٔ جالبی می‌بینیم: ∑m P(m|a) طبقِ تعریف برابر ۱ است! از این رو، اصلاً نیازی به گنجاندنش نبود؛ متغیرِ M برای این پرس‌وجو نامربوط است. راهِ دیگرِ گفتنِ این اینکه نتیجهٔ پرس‌وجوی P(JohnCalls | Burglary=true) اگر MaryCalls را کلاً از شبکه حذف کنیم تغییر نمی‌کند. به‌طورِ کلی، می‌توانیم هر گرهٔ برگی را که متغیرِ پرس‌وجو یا شاهد نیست حذف کنیم. پس از حذفِ آن، ممکن است گره‌های برگیِ بیشتری باشند، و آن‌ها نیز ممکن است نامربوط باشند. با ادامهٔ این فرایند، سرانجام درمی‌یابیم هر متغیری که نیای متغیرِ پرس‌وجو یا شاهد نیست برای پرس‌وجو نامربوط است. الگوریتمِ حذفِ متغیر بنابراین می‌تواند همهٔ این متغیرها را قبل از ارزیابیِ پرس‌وجو حذف کند.

وقتی روی شبکهٔ بیمهٔ نشان‌داده‌شده در شکل ۱۳.۹ اعمال شود، حذفِ متغیر بهبودِ قابلِ‌ملاحظه‌ای بر الگوریتمِ شمارشِ اولیه نشان می‌دهد. با استفاده از ترتیبِ توپولوژیکِ معکوس برای متغیرها، استنتاجِ دقیق با حذف حدود ۱,۰۰۰ برابر سریع‌تر از الگوریتمِ شمارش است.

```
function ELIMINATION-ASK(X, e, bn) returns a distribution over X
                                        inputs: X, the query variable
    e, the observed event (an assignment of values to evidence variables)
                        bn, a Bayesian network with variables vars and CPTs
                                                                 [] ← factors
                                                :for each variable V in ORDER(vars)
                                    factors ← [MAKE-FACTOR(V, e) | factors]
                                                :if V is a hidden variable
                                          factors ← SUM-OUT(V, factors)
                                    return NORMALIZE(POINTWISE-PRODUCT(factors))
```

شکل ۱۳.۱۳ – الگوریتمِ حذفِ متغیر برای استنتاجِ دقیق در شبکه‌های بیز.

۱۳.۳.۳ پیچیدگیِ استنتاجِ دقیق

پیچیدگیِ استنتاجِ دقیق در شبکه‌های بیز به‌شدت به ساختارِ شبکه بستگی دارد. شبکهٔ سرقتِ شکل ۱۳.۲ به خانواده‌ای از شبکه‌ها تعلق دارد که در آن‌ها حداکثر یک مسیرِ بدونِ جهت (یعنی با نادیده‌گرفتنِ جهتِ پیکان‌ها) بین هر دو گره وجود دارد. این‌ها «شبکه‌های تک‌متصل» (Singly Connected Networks) یا «پلی‌درخت» (Polytrees) نامیده می‌شوند و ویژگیِ خاصاً خوبی دارند:

پیچیدگیِ زمانی و فضاییِ استنتاجِ دقیق در پلی‌درخت‌ها در اندازهٔ شبکه خطی است.

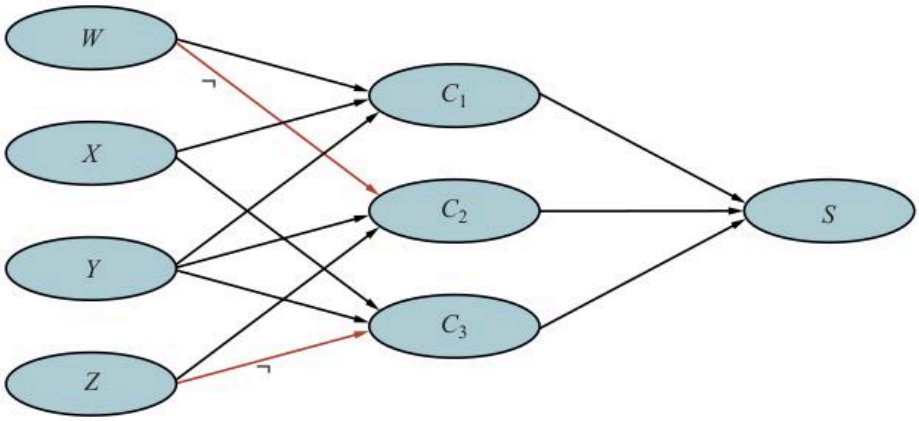
در این‌جا، اندازه به‌صورتِ شمارِ درایه‌های CPT تعریف می‌شود؛ اگر شمارِ والدهای هر گره با ثابتی کران‌دار باشد، پیچیدگی نیز در شمارِ گره‌ها خطی خواهد بود. این نتایج برای هر ترتیبی که با ترتیبِ توپولوژیکِ شبکه سازگار باشد برقرارند (تمرین VEEX.۱۳).

برای «شبکه‌های چندمتصل» (Multiply Connected Networks)، مانند شبکهٔ بیمه در شکل ۱۳.۹، حذفِ متغیر می‌تواند در بدترین حالت پیچیدگیِ زمانی و فضاییِ نمایی داشته باشد، حتی وقتی شمارِ والدهای هر گره کران‌دار است. این شگفت‌آور نیست وقتی در نظر بگیریم که چون شاملِ استنتاج در منطقِ گزاره‌ای به‌عنوانِ حالتِ خاص است، استنتاج در شبکه‌های بیز NP-سخت است. برای اثباتِ این، باید دریابیم چگونه مسئلهٔ ارضاپذیریِ گزاره‌ای را به‌صورتِ شبکهٔ بیز رمزگذاری کنیم، به‌گونه‌ای که اجرایِ استنتاج روی این شبکه به ما بگوید جمله‌های گزاره‌ایِ اصلی ارضاشدنی‌اند یا نه. (در زبانِ نظریهٔ پیچیدگی، مسائلِ ارضاپذیری را به مسائلِ استنتاجِ شبکهٔ بیز «تقلیل» می‌دهیم.) معلوم می‌شود این کاملاً سرراست است. شکل ۱۳.۱۴ نشان می‌دهد چگونه مسئلهٔ 3-SAT خاصی را رمزگذاری کنیم. متغیرهای گزاره‌ای متغیرهای ریشهٔ شبکه می‌شوند، هر یک با احتمالِ پیشینیِ 0.5.

لایهٔ بعدیِ گره‌ها متناظر با عبارت‌ها (کلاوزها)ست، با هر متغیرِ عبارتِ Cj که به متغیرهای مناسب به‌عنوانِ والد متصل است. توزیعِ شرطی برای متغیرِ عبارت، فصلیِ قطعی است، با نقیض‌گذاریِ لازم، به‌گونه‌ای که هر متغیرِ عبارت صادق است اگر و فقط اگر تخصیص به والدهایش آن عبارت را ارضا کند. سرانجام، S عطفِ متغیرهای عبارت است. برای تعیینِ ارضاشدنی‌بودنِ جملهٔ اصلی، به‌سادگی P(S=true) را ارزیابی می‌کنیم. اگر جمله ارضاشدنی باشد، آنگاه تخصیصِ ممکنی به متغیرهای منطقی هست که S را صادق می‌کند؛ در شبکهٔ بیز، این بدان معناست جهانِ ممکنی با احتمالِ غیرصفر هست که در آن متغیرهای ریشه آن تخصیص را دارند، متغیرهای عبارت مقدارِ صادق دارند و S مقدارِ صادق دارد. بنابراین، برای جملهٔ ارضاشدنی P(S=true) > 0. برعکس، P(S=true) = 0 برای جملهٔ غیرارضاشدنی: همهٔ جهان‌های با S=true احتمالِ ۰ دارند.

از این رو، می‌توانیم از استنتاجِ شبکهٔ بیز برای حلِ مسائلِ 3-SAT استفاده کنیم؛ از این، نتیجه می‌گیریم استنتاجِ شبکهٔ بیز NP-سخت است.

در واقع، می‌توانیم بیش از این‌ها انجام دهیم. توجه کنید احتمالِ هر تخصیصِ ارضاکننده 2^(-n) برای مسئله‌ای با n متغیر است. از این رو، شمارِ تخصیص‌های ارضاکننده برابر 2^(-n) / P(S=true) است. چون محاسبهٔ شمارِ تخصیص‌های ارضاکننده برای مسئلهٔ «#P-3 SAT-کامل» است («شمار-P کامل»)، این بدان معناست استنتاجِ شبکهٔ بیز «#P-سخت» است—یعنی اکیداً سخت‌تر از مسائلِ NP-کامل.



شکل ۱۳.۱۴ – رمزگذاریِ شبکهٔ بیزِ جملهٔ 3-CNF: (W ∨ X ∨ Y) ∧ (¬W ∨ Y ∨ Z) ∧ (X ∨ Y ∨ ¬Z).

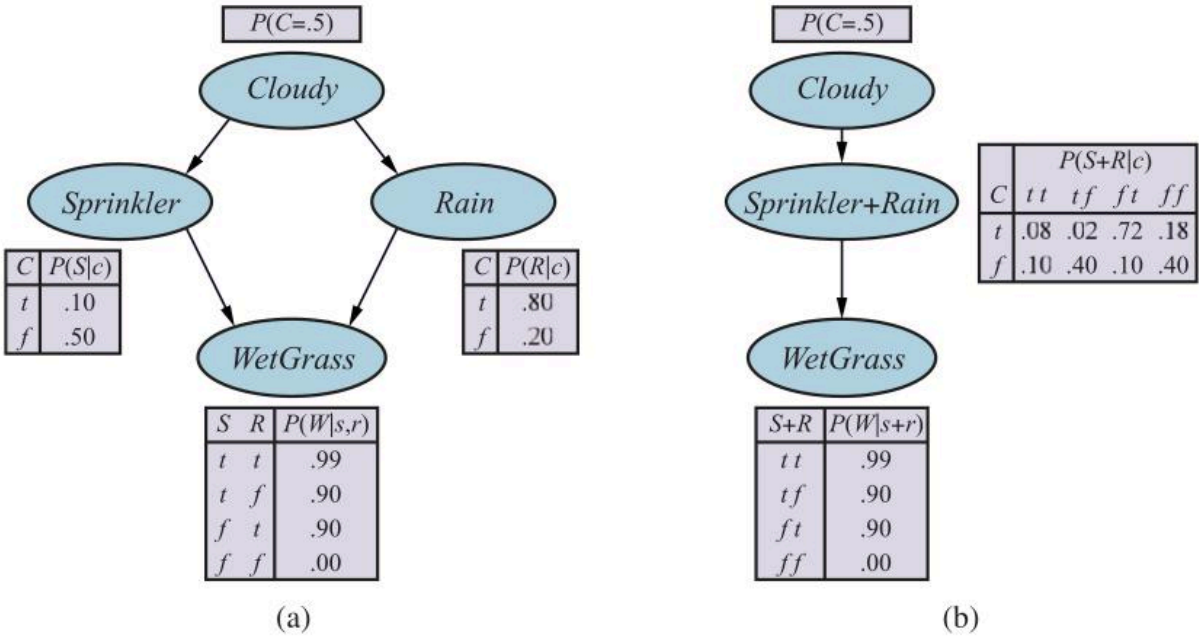
ارتباطی نزدیک میان پیچیدگیِ استنتاجِ شبکهٔ بیز و پیچیدگیِ مسائلِ ارضایِ قید (CSP) هست. چنان‌که در فصل ۶ بحث کردیم، دشواریِ حلِ CSP گسسته به اینکه گرافِ قیدش چقدر «درخت‌مانند» است مربوط می‌شود. معیارهایی مانند «پهنایِ درختی» (Tree Width)، که پیچیدگیِ حلِ CSP را کران‌دار می‌کنند، می‌توانند مستقیماً روی شبکه‌های بیز نیز اعمال شوند. علاوه بر این، الگوریتمِ حذفِ متغیر را می‌توان به حلِ CSPها نیز تعمیم داد. جدا از تقلیلِ مسائلِ ارضاپذیری به استنتاجِ شبکهٔ بیز، می‌توانیم استنتاجِ شبکهٔ بیز را به ارضاپذیری تقلیل دهیم، که به ما اجازه می‌دهد از ماشین‌آلاتِ قدرتمندِ توسعه‌یافته برای حلِ SAT بهره بگیریم (فصل ۷ را ببینید). در این مورد، تقلیل به شکلِ خاصی از حلِ SAT به نام «شمارشِ مدلِ وزندار» (Weighted Model Counting یا WMC) است. شمارشِ مدلِ معمولی شمارِ تخصیص‌های ارضاکننده را برای عبارتِ SAT می‌شمارد؛ WMC وزنِ کلِ آن تخصیص‌های ارضاکننده را جمع می‌زند—که در این کاربرد، وزن اساساً حاصل‌ضربِ احتمال‌های شرطی برای هر تخصیصِ متغیر به‌شرطِ والدهایش است. (جزئیات در تمرین WMCX.۱۳.) تا حدی به این دلیل که فناوریِ حلِ SAT برای کاربردهای بزرگ‌مقیاس به‌خوبی بهینه‌سازی شده، استنتاجِ شبکهٔ بیز از طریقِ WMC با دیگر الگوریتم‌های دقیق روی شبکه‌های با پهنایِ درختیِ بزرگ رقابتی و گاهی برتر است.

۱۳.۳.۴ الگوریتم‌های خوشه‌بندی

الگوریتمِ حذفِ متغیر برای پاسخ‌دادن به پرس‌وجوهای فردی ساده و کارآمد است. اما اگر بخواهیم احتمال‌های پسینی را برای همهٔ متغیرهای شبکه محاسبه کنیم، می‌تواند کم‌کارآمدتر باشد. برای نمونه، در شبکهٔ پلی‌درخت، باید O(n) پرس‌وجو صادر شود که هر یک O(n) هزینه دارد، در مجموع $O(n^2)$ زمان. با «الگوریتم‌های خوشه‌بندی» (Clustering Algorithms)، که «الگوریتم‌های درختِ اتصال» (Join Tree Algorithms) نیز نامیده می‌شوند)، زمان را می‌توان به O(n) کاهش داد. به همین دلیل، این الگوریتم‌ها در ابزارهای تجاریِ شبکهٔ بیز به‌طورِ گسترده استفاده می‌شوند.

ایدهٔ پایهٔ خوشه‌بندی پیوستنِ گره‌های فردیِ شبکه برای تشکیلِ «گره‌های خوشه‌ای» (Cluster Nodes) است، به‌گونه‌ای که شبکهٔ حاصل پلی‌درخت باشد. برای نمونه، شبکهٔ چندمتصلِ نشان‌داده‌شده در شکل ۱۳.۱۵(الف) را می‌توان با ترکیبِ گره‌های Sprinkler و Rain در گرهٔ خوشه‌ایِ «Sprinkler+Rain» به پلی‌درخت تبدیل کرد، چنان‌که در شکل ۱۳.۱۵(ب)

نشان داده شده. دو گرهٔ بولی با «کلان‌گره» (Meganode)ای جایگزین می‌شوند که چهار مقدارِ ممکن می‌گیرد: tt، tf، ft و ff. کلان‌گره فقط یک والد دارد، متغیرِ بولیِ Cloudy، پس دو حالتِ شرطی هست. اگرچه این مثال نشان نمی‌دهد، فرایندِ خوشه‌بندی اغلب کلان‌گره‌هایی تولید می‌کند که برخی متغیرها را شریک‌اند.



شکل ۱۳.۱۵ — (الف) شبکهٔ چندمتصلِ توصیف‌کنندهٔ روالِ روزانهٔ چمنِ مری: هر صبح، او آب‌وهوا را بررسی می‌کند؛ اگر ابری باشد، معمولاً آب‌پاش را روشن نمی‌کند؛ اگر آب‌پاش روشن باشد، یا اگر در طولِ روز باران ببارد، چمن خیس خواهد بود. بدین‌ترتیب، Cloudy از دو مسیرِ علّیِ متفاوت بر WetGrass اثر می‌گذارد. (ب) معادلِ خوشه‌بندی‌شدهٔ شبکهٔ چندمتصل.

پس از اینکه شبکه به شکلِ پلی‌درخت درآمد، الگوریتمِ استنتاجِ ویژه‌ای لازم است، زیرا روش‌های استنتاجِ معمولی نمی‌توانند کلان‌گره‌هایی را که متغیرها را با یکدیگر شریک‌اند مدیریت کنند. اساساً، الگوریتم نوعی «انتشارِ قید» (Constraint Propagation) است (فصل ۶ را ببینید) که در آن قیدها تضمین می‌کنند کلان‌گره‌های همسایه بر احتمالِ پسینیِ هر متغیری که مشترک دارند توافق کنند. با دفترداریِ دقیق، این الگوریتم قادر است احتمال‌های پسینی را برای همهٔ گره‌های غیرشاهدِ شبکه در زمانی خطی در اندازهٔ شبکهٔ خوشه‌بندی‌شده محاسبه کند. با این حال، NP-سختیِ مسئله ناپدید نشده است: اگر شبکه‌ای با حذفِ متغیر زمان و فضای نمایی لازم داشته باشد، آنگاه CPTهای شبکهٔ خوشه‌بندی‌شده لزوماً به‌صورتِ نمایی بزرگ خواهند بود.

۱۳.۴ استنتاجِ تقریبی برای شبکه‌های بیزی

با توجه به ناشدنی‌بودنِ استنتاجِ دقیق در شبکه‌های بزرگ، حالا روش‌های استنتاجِ تقریبی را در نظر می‌گیریم. این بخش الگوریتم‌های «نمونه‌برداریِ تصادفی» (Randomized Sampling) را توصیف می‌کند، که «الگوریتم‌های مونت‌کارلو» (Monte Carlo) نیز نامیده می‌شوند و پاسخ‌های تقریبی می‌دهند که دقت‌شان به شمارِ نمونه‌های

تولیدشده بستگی دارد. آن‌ها با تولیدِ رخدادهای تصادفی بر اساسِ احتمال‌های شبکهٔ بیز و شمردنِ پاسخ‌های متفاوتِ یافت‌شده در آن رخدادهای تصادفی کار می‌کنند. با نمونه‌های کافی، می‌توانیم خودسرانه به بازیابیِ توزیعِ احتمالِ واقعی نزدیک شویم—مشروط بر اینکه شبکهٔ بیز توزیع‌های شرطیِ قطعی نداشته باشد.

الگوریتم‌های مونت‌کارلو، که «بازپختِ شبیه‌سازی‌شده» (صفحهٔ ۱۱۵) نمونه‌ای از آن است، در شاخه‌های بسیاری از علوم برای برآوردِ کمیت‌هایی استفاده می‌شوند که محاسبهٔ دقیقشان دشوار است. در این بخش، به نمونه‌برداریِ اعمال‌شده بر محاسبهٔ احتمال‌های پسینی در شبکه‌های بیز علاقه‌مندیم. دو خانواده از الگوریتم‌ها را توصیف می‌کنیم: نمونه‌برداریِ مستقیم و نمونه‌برداریِ زنجیرهٔ مارکوف. چند رویکردِ دیگر برای استنتاجِ تقریبی در یادداشت‌های پایانِ فصل ذکر شده‌اند.

۱۳.۴.۱ روش‌های نمونه‌برداریِ مستقیم

عنصرِ اولیه در هر الگوریتمِ نمونه‌برداری، تولیدِ نمونه‌ها از توزیعِ احتمالِ شناخته‌شده است. برای نمونه، سکهٔ بدونِ سوگیری را می‌توان متغیرِ تصادفیِ Coin با مقادیرِ ⟨heads, tails⟩ و توزیعِ پیشینیِ P(Coin) = ⟨0.5, 0.5⟩ در نظر گرفت. نمونه‌برداری از این توزیع دقیقاً مانند انداختنِ سکه است: با احتمالِ 0.5 رو برمی‌گرداند و با احتمالِ 0.5 پشت. با منبعی از اعدادِ تصادفیِ r که به‌صورتِ یکنواخت در بازهٔ [0,1] توزیع شده‌اند، نمونه‌برداری از هر توزیعی روی متغیرِ تکی، خواه گسسته خواه پیوسته، کاری ساده است. این کار با ساختنِ «توزیعِ تجمعی» (Cumulative Distribution) برای متغیر و بازگرداندنِ نخستین مقداری که احتمالِ تجمعی‌اش از r فراتر می‌رود انجام می‌شود (تمرین PRSA.۱۳). با یک فرایندِ نمونه‌برداریِ تصادفی برای شبکهٔ بیزی شروع می‌کنیم که شاهدِ مرتبطی ندارد. ایده این است که هر متغیر را به‌نوبت، در ترتیبِ توپولوژیک، نمونه‌برداری کنیم. توزیعِ احتمالی که مقدار از آن نمونه‌برداری می‌شود بر مقادیرِ ازپیش‌تخصیص‌یافته به والدهای متغیر شرطی است. (چون در ترتیبِ توپولوژیک نمونه‌برداری می‌کنیم، والدها تضمین‌شده مقادیر دارند.) این الگوریتم در شکل ۱۳.۱۶ نشان داده شده. با اعمالِ آن بر شبکهٔ شکل ۱۳.۱۵(الف) با ترتیبِ Cloudy, Sprinkler, Rain, WetGrass، ممکن است رخدادِ تصادفیِ زیر تولید شود:

1. نمونه‌برداری از P(Cloudy) = ⟨0.5, 0.5⟩، مقدارِ true است.
2. نمونه‌برداری از P(Sprinkler | Cloudy=true) = ⟨0.1, 0.9⟩، مقدارِ false است.
3. نمونه‌برداری از P(Rain | Cloudy=true) = ⟨0.8, 0.2⟩، مقدارِ true است.
4. نمونه‌برداری از P(WetGrass | Sprinkler=false, Rain=true) = ⟨0.9, 0.1⟩، مقدارِ true است.

در این مورد، PRIOR-SAMPLE رخدادِ [true, false, true, true] را برمی‌گرداند.

```
function PRIOR-SAMPLE(bn) returns a sampled event from the prior specified by bn
  inputs: bn، a Bayesian network specifying the joint distribution P(X1,...,Xn)
                                                  x ← an event with n elements
                                              :for each variable Xi in X1,...,Xn
                                  x[i] ← a random sample from P(Xi | parents(Xi))
                                                                    return x
```

شکل ۱۳.۱۶ — الگوریتمِ نمونه‌برداری که رخدادهایی از شبکهٔ بیزی تولید می‌کند. هر متغیر بر اساسِ توزیعِ شرطی به‌شرطِ مقادیرِ ازپیش‌نمونه‌برداری‌شدهٔ والدهایش نمونه‌برداری می‌شود.

به‌آسانی دیده می‌شود PRIOR-SAMPLE نمونه‌ها را از توزیعِ مشترکِ پیشینیِ مشخص‌شده توسط شبکه تولید می‌کند. نخست، فرض کنیم SPS(x1,...,xn) احتمالِ تولیدِ رخدادِ خاصِ x1,...,xn توسط الگوریتمِ PRIOR-SAMPLE باشد. فقط با نگاه‌کردن به فرایندِ نمونه‌برداری:

$$SPS(x_1,\ldots,x_n) = \prod_{i=1..n} P(x_i \mid parents(X_i))$$

زیرا هر گامِ نمونه‌برداری فقط به مقادیرِ والد بستگی دارد. این عبارت باید آشنا به نظر برسد، زیرا همچنین احتمالِ رخداد بر اساسِ بازنماییِ شبکهٔ بیز از توزیعِ مشترک است، چنان‌که در معادلهٔ (13.2) بیان شد. یعنی SPS(x1,...,xn) = P(x1,...,xn) داریم. این واقعیتِ ساده پاسخ‌دادن به پرسش‌ها با استفاده از نمونه‌ها را آسان می‌کند.

در هر الگوریتمِ نمونه‌برداری، پاسخ‌ها با شمردنِ نمونه‌های واقعیِ تولیدشده محاسبه می‌شوند. فرض کنیم در مجموع N نمونه توسط الگوریتمِ PRIOR-SAMPLE تولید شده، و NPS(x1,...,xn) شمارِ دفعاتِ وقوعِ رخدادِ خاصِ x1,...,xn در مجموعهٔ نمونه‌ها باشد. انتظار داریم این عدد، به‌عنوانِ کسری از کل، در حدِّ نهایی به مقدارِ موردانتظارش طبقِ احتمالِ نمونه‌برداری همگرا شود:

$$\lim_{N\to\infty} NPS(x_1,\ldots,x_n) / N = SPS(x_1,\ldots,x_n) = P(x_1,\ldots,x_n) \qquad (13.6)$$

برای نمونه، رخدادِ تولیدشدهٔ پیشین را در نظر بگیرید: [true, false, true, true]. احتمالِ نمونه‌برداریِ این رخداد SPS(true, false, true, true) = 0.5 × 0.9 × 0.8 × 0.9 = 0.324 است. از این رو، در حدِّ N بزرگ، انتظار داریم ٪۳۲.۴ نمونه‌ها از این رخداد باشند.

هرگاه در ادامه از تساویِ تقریبی («≈») استفاده می‌کنیم، دقیقاً به همین معناست—که احتمالِ برآوردشده در حدِّ نمونهٔ بزرگ دقیق می‌شود. چنین برآوردی «سازگار» (Consistent) نامیده می‌شود. برای نمونه، می‌توان برآوردِ سازگاری از احتمالِ هر رخدادِ تا حدی مشخص‌شدهٔ x1,...,xm تولید کرد، که در آن m ≤ n:

$$P(x_1,\ldots,x_m) \approx NPS(x_1,\ldots,x_m) / N \qquad (13.7)$$

یعنی احتمالِ رخداد را می‌توان به‌صورتِ کسری از همهٔ رخدادهای کاملِ تولیدشده توسط فرایندِ نمونه‌برداری که با رخدادِ تا حدی مشخص‌شده همخوانی دارند برآورد کرد. از $\hat{P}$ (تلفظ «پی-هت») برای نشان‌دادنِ احتمالِ برآوردشده استفاده می‌کنیم. پس، اگر ۱,۰۰۰ نمونه از شبکهٔ آب‌پاش تولید کنیم و ۵۱۱ تایشان Rain=true داشته باشند، آنگاه احتمالِ برآوردشدهٔ باران $\hat{P}$(Rain=true) = 0.511 است.

نمونه‌برداریِ ردّی در شبکه‌های بیزی. «نمونه‌برداریِ ردّی» (Rejection Sampling) روشی عمومی برای تولیدِ نمونه‌ها از توزیعی است که نمونه‌برداری از آن دشوار است، با توجه به توزیعی که نمونه‌برداری از آن آسان است. در ساده‌ترین شکلِ آن، می‌توان برای محاسبهٔ احتمال‌های شرطی—یعنی تعیینِ P(X|e)—استفاده کرد. الگوریتمِ REJECTION-SAMPLING در شکل ۱۳.۱۷ نشان داده شده. نخست، نمونه‌ها را از توزیعِ پیشینیِ مشخص‌شده توسط شبکه تولید می‌کند. سپس، همهٔ آن‌هایی را که با شاهد همخوانی ندارند رد می‌کند. سرانجام، برآوردِ $\hat{P}(X=x|e)$ با شمردنِ اینکه X=x چند بار در نمونه‌های باقی‌مانده رخ می‌دهد به دست می‌آید.

فرض کنیم $\hat{P}(X|e)$ توزیعِ برآوردشده‌ای باشد که الگوریتم برمی‌گرداند؛ این توزیع با نرمال‌سازیِ NPS(X,e)، بردارِ شمارشِ نمونه‌ها برای هر مقدارِ X در جایی که نمونه با شاهدِ e موافق است، محاسبه می‌شود:

$$\hat{P}(X|e) = \alpha\, NPS(X, e) = NPS(X, e) / NPS(e)$$

از معادلهٔ (13.7)، این می‌شود $\hat{P}(X|e) \approx P(X,e)/P(e) = P(X|e)$. یعنی نمونه‌برداریِ ردّی به پاسخِ درست همگرا می‌شود.

```
function REJECTION-SAMPLING(X, e, bn, N) returns an estimate of P(X|e)
                                                  inputs: X, the query variable
                                        e, observed values for the variables E
                                                     bn, a Bayesian network
                                    N, the total number of samples to generate
                          local variables: C, a count vector for each value of X, initially zero
                                                                  :for j = 1 to N
                                                        x ← PRIOR-SAMPLE(bn)
if x is consistent with e then C[j] ← C[j] + 1     where xj is the value of X in x
                                                               return NORMALIZE(C)
```

شکل ۱۳.۱۷ — الگوریتمِ نمونه‌برداریِ ردّی برای پاسخ‌دادن به پرس‌وجوها با توجه به شواهد در شبکهٔ بیزی.

حالا که می‌دانیم نمونه‌برداریِ ردّی به پاسخِ درست همگرا می‌شود، پرسشِ بعدی این است که این همگرایی چقدر سریع رخ می‌دهد؟ دقیق‌تر، چند نمونه لازم است قبل از اینکه بدانیم برآوردهای حاصل با احتمالِ زیاد به پاسخ‌های درست نزدیک‌اند؟

در حالی که پیچیدگیِ الگوریتم‌های دقیق تا حدِ زیادی به توپولوژیِ شبکه بستگی دارد—درخت‌ها آسان‌اند، شبکه‌های پرپیوند سخت‌اند—پیچیدگیِ نمونه‌برداریِ ردّی عمدتاً به کسری از نمونه‌هایی بستگی دارد که پذیرفته می‌شوند. این کسر دقیقاً برابرِ احتمالِ پیشینیِ شاهد، P(e) است. متأسفانه، برای مسائلِ پیچیده با متغیرهای شاهدِ بسیار، این کسر به‌نحوِ ناچیزی کوچک است. وقتی روی نسخهٔ گسستهٔ شبکهٔ بیمهٔ خودرو در شکل ۱۳.۹ اعمال شود، کسرِ نمونه‌های سازگار با حالتِ شاهدِ نمونه‌ایِ نمونه‌برداری‌شده از خودِ شبکه معمولاً بین یک‌درهزار و یک‌در۱۰,۰۰۰ است. همگرایی بسیار کند است (شکل ۱۳.۱۹ زیر را ببینید).

انتظار داریم کسرِ نمونه‌های سازگار با شاهدِ e با رشدِ شمارِ متغیرهای شاهد به‌صورتِ نمایی کاهش یابد، پس رویه برای مسائلِ پیچیده غیرقابلِ‌استفاده است. همچنین با متغیرهای شاهدِ پیوسته‌مقدار دشواری دارد، زیرا احتمالِ تولیدِ نمونه‌ای سازگار با چنین شاهدی صفر است (اگر واقعاً پیوسته‌مقدار باشد) یا بی‌نهایت‌کوچک (اگر فقط عددِ ممیزِ شناورِ با دقتِ متناهی باشد).

توجه کنید نمونه‌برداریِ ردّی بسیار شبیهِ برآوردِ احتمال‌های شرطی در دنیای واقعی است. برای نمونه، برای برآوردِ احتمالِ شرطیِ زنده‌ماندنِ انسان‌ها پس از برخوردِ سیارکی با قطرِ ۱ کیلومتر به زمین، می‌توان به‌سادگی شمرد هر چند وقت یک‌بار انسان‌ها پس از برخوردِ سیارکی با قطرِ ۱ کیلومتر به زمین زنده می‌مانند، و همهٔ روزهایی را که چنین رویدادی رخ نمی‌دهد نادیده گرفت. (در این‌جا، خودِ جهان نقشِ الگوریتمِ تولیدِ نمونه را بازی می‌کند.) برای گرفتنِ برآوردِ مناسبی، ممکن است لازم باشد منتظر وقوعِ ۱۰۰ رویدادِ چنین ماند. بدیهی است، این ممکن است زمانِ زیادی ببرد، و این ضعفِ نمونه‌برداریِ ردّی است.

نمونه‌برداریِ اهمیت. فنِ آماریِ عمومیِ «نمونه‌برداریِ اهمیت» (Importance Sampling) قصد دارد اثرِ نمونه‌برداری از توزیعِ P را با استفاده از نمونه‌هایی از توزیعِ دیگری مانند Q شبیه‌سازی کند. تضمین می‌کنیم پاسخ‌ها در حدِّ نهایی درست‌اند با اعمالِ «عاملِ تصحیحِ» P(x)/Q(x)، که «وزن» (Weight) نیز نامیده می‌شود، به هر نمونهٔ x هنگام شمردنِ نمونه‌ها.

دلیلِ استفاده از نمونه‌برداریِ اهمیت در شبکه‌های بیز ساده است: می‌خواهیم از توزیعِ پسینیِ واقعیِ شرطی‌شده بر همهٔ شواهد نمونه‌برداری کنیم، اما معمولاً این خیلی دشوار است؛[6] پس در عوض، از توزیعی آسان نمونه‌برداری می‌کنیم و تصحیح‌های لازم را اعمال می‌کنیم.

دلیلِ کارکردنِ نمونه‌برداریِ اهمیت نیز ساده است. فرض کنیم متغیرهای غیرشاهد Z باشند. اگر می‌توانستیم مستقیماً از P(z|e) نمونه‌برداری کنیم، می‌توانستیم برآوردهایی مانند این بسازیم:

$$\hat{P}(z|e) = NP(z) / N \approx P(z|e)$$

که در آن NP(z) شمارِ نمونه‌های با Z=z هنگام نمونه‌برداری از P است. حالا فرض کنیم در عوض از Q(z) نمونه‌برداری می‌کنیم. برآورد در این مورد شاملِ عامل‌های تصحیح است:

$$\hat{P}(z|e) = (NQ(z) / N) \times (P(z|e) / Q(z)) \approx Q(z) \times P(z|e) / Q(z) = P(z|e)$$

بدین‌ترتیب، برآورد بدونِ توجه به اینکه کدام توزیعِ نمونه‌برداریِ Q استفاده شود به مقدارِ درست همگرا می‌شود. (تنها الزامِ فنی این است که Q(z) برای هیچ zای که P(z|e) در آن غیرصفر است صفر نباشد.) به‌صورتِ شهودی، عاملِ تصحیح برای بیش‌نمونه‌برداری یا کم‌نمونه‌برداری جبران می‌کند. برای نمونه، اگر برای zای Q(z) بسیار بزرگ‌تر از P(z|e) باشد، آنگاه نمونه‌های بسیار بیشتری از آن z نسبت به آنچه باید باشد وجود خواهد داشت، اما هر یک وزنِ کوچکی خواهد داشت، پس درست همان‌گونه عمل می‌کند که گویی شمارِ درست وجود دارد.

در موردِ اینکه کدام Q را استفاده کنیم، یکی می‌خواهیم که نمونه‌برداری از آن آسان و تا آن‌جا که ممکن است به پسینِ واقعیِ P(z|e) نزدیک باشد. رایج‌ترین رویکرد «وزن‌گذاریِ درستنمایی» (Likelihood Weighting) نامیده می‌شود (به دلایلی که به‌زودی خواهیم دید). چنان‌که در تابعِ WEIGHTED-SAMPLE در شکل ۱۳.۱۸ نشان داده شده، الگوریتم مقادیرِ متغیرهای شاهدِ E را ثابت می‌کند و همهٔ متغیرهای غیرشاهد را در ترتیبِ توپولوژیک نمونه‌برداری می‌کند، هر یک به‌شرطِ والدهایش. این تضمین می‌کند هر رخدادِ تولیدشده با شاهد سازگار است.

توزیعِ نمونه‌برداریِ تولیدشده توسط این الگوریتم را QWS بنامیم. اگر متغیرهای غیرشاهد Z={Z1,...,Zl} باشند، آنگاه داریم:

$$QWS(z) = \prod_{i=1..l} P(z_i \mid parents(Z_i)) \qquad (13.8)$$

زیرا هر متغیر به‌شرطِ والدهایش نمونه‌برداری می‌شود. برای کامل‌کردنِ الگوریتم، باید بدانیم چگونه وزنِ هر نمونهٔ تولیدشده از QWS را محاسبه کنیم.

طبقِ طرحِ عمومیِ نمونه‌برداریِ اهمیت، وزن باید $w(z) = P(z|e)/QWS(z) = \alpha P(z,e)/QWS(z)$ باشد، که در آن عاملِ نرمال‌سازیِ $\alpha = 1/P(e)$ برای همهٔ نمونه‌ها یکسان است. حالا z و e با هم همهٔ متغیرهای شبکهٔ بیز را پوشش می‌دهند، پس P(z,e) فقط حاصل‌ضربِ همهٔ احتمال‌های شرطی است (معادلهٔ (13.2) صفحهٔ ۴۱۵)؛ و می‌توانیم آن را به‌صورتِ حاصل‌ضربِ احتمال‌های شرطیِ متغیرهای غیرشاهد ضرب‌درِ حاصل‌ضربِ احتمال‌های شرطیِ متغیرهای شاهد بنویسیم:

$$\begin{aligned} w(z) &= \alpha\, P(z,e) / QWS(z) \\ &= \alpha \left(\prod_i P(z_i \mid parents(Z_i)) \times \prod_i P(e_i \mid parents(E_i))\right) / \left(\prod_i P(z_i \mid parents(Z_i))\right) \\ &= \alpha \prod_i P(e_i \mid parents(E_i)) \qquad (13.9) \end{aligned}$$

بدین‌ترتیب وزن، حاصل‌ضربِ احتمال‌های شرطیِ متغیرهای شاهد به‌شرطِ والدهایشان است. (احتمال‌های شاهد عموماً «درستنمایی» (Likelihood) نامیده می‌شوند، از این رو نام.) محاسبهٔ وزن در WEIGHTED-SAMPLE به‌صورتِ افزایشی پیاده‌سازی می‌شود، و هر بار با متغیرِ شاهدِ مواجه‌شده در احتمالِ شرطی ضرب می‌شود. نرمال‌سازی در پایان، قبل از بازگرداندنِ نتیجهٔ پرس‌وجو، انجام می‌شود.

```
function LIKELIHOOD-WEIGHTING(X, e, bn, N) returns an estimate of P(X|e)
                                                    inputs: X, the query variable
                                    e, observed values for the variables E
                  bn, a Bayesian network specifying the joint distribution P(X1,...,Xn)
                                    N, the total number of samples to generate
          local variables: W, a weighted count vector for each value of X, initially zero
                                                                  :for j = 1 to N
                                                 x, w ← WEIGHTED-SAMPLE(bn, e)
                                W[j] ← W[j] + w     where xj is the value of X in x
                                                              return NORMALIZE(W)

                                function WEIGHTED-SAMPLE(bn, e) returns an event and a weight
                                w ← 1; x ← an event with n elements, with values fixed by e
                                                                  :for i = 1 to n
                                      if Xi is an evidence variable with value xij in e
                                            then w ← w × P(Xi = xij | parents(Xi))
                                else x[i] ← a random sample from P(Xi | parents(Xi))
                                                                        return x, w
```

شکل ۱۳.۱۸ — الگوریتمِ وزن‌گذاریِ درستنمایی برای استنتاج در شبکه‌های بیزی. در WEIGHTED-SAMPLE، هر متغیرِ غیرشاهد بر اساسِ توزیعِ شرطی به‌شرطِ مقادیرِ ازپیش‌نمونه‌برداری‌شدهٔ والدهایش نمونه‌برداری می‌شود، در حالی که وزنی بر اساسِ درستنماییِ هر متغیرِ شاهد انباشته می‌شود.

الگوریتم را روی شبکهٔ نشان‌داده‌شده در شکل ۱۳.۱۵(الف) اعمال کنیم، با پرس‌وجوی P(Rain | Cloudy=true, WetGrass=true) و ترتیبِ Cloudy, Sprinkler, Rain, WetGrass. (هر ترتیبِ توپولوژیکی کار می‌کند.) فرایند به‌صورتِ زیر است: نخست، وزنِ w برابر 1.0 گذاشته می‌شود. سپس رخدادی تولید می‌شود:

1. Cloudy متغیرِ شاهد با مقدارِ true است. بنابراین، w ← w × P(Cloudy=true) = 0.5 می‌گذاریم.
2. Sprinkler متغیرِ شاهد نیست، پس از P(Sprinkler | Cloudy=true) = ⟨0.1, 0.9⟩ نمونه‌برداری کن؛ فرض کنیم false برمی‌گرداند.
3. Rain متغیرِ شاهد نیست، پس از P(Rain | Cloudy=true) = ⟨0.8, 0.2⟩ نمونه‌برداری کن؛ فرض کنیم true برمی‌گرداند.
4. WetGrass متغیرِ شاهد با مقدارِ true است. بنابراین، w ← w × P(WetGrass=true | Sprinkler=false, Rain=true) = 0.5 × 0.9 = 0.45 می‌گذاریم.

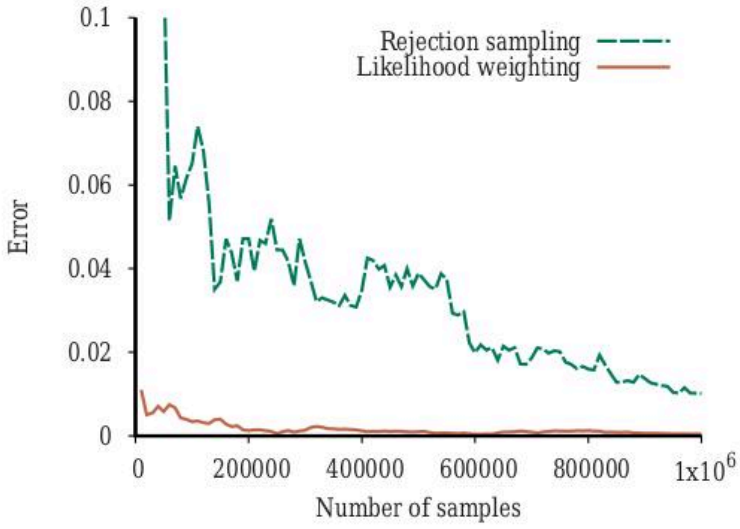
در این‌جا WEIGHTED-SAMPLE رخدادِ [true, false, true, true] را با وزنِ 0.45 برمی‌گرداند، و این زیرِ Rain=true ثبت می‌شود. توجه کنید Parents(Zi) می‌تواند هم متغیرهای غیرشاهد و هم متغیرهای شاهد را شامل شود.

برخلافِ توزیعِ پیشینیِ P(z)، توزیعِ QWS تا حدی به شاهد توجه می‌کند: مقادیرِ نمونه‌برداری‌شدهٔ هر Zi تحتِ تأثیرِ شاهد در میانِ نیاکانِ Zi خواهند بود. برای نمونه، هنگام نمونه‌برداری از Sprinkler الگوریتم به شاهدِ Cloudy=true در متغیرِ والدش توجه می‌کند. از سوی دیگر، QWS کمتر از توزیعِ پسینیِ واقعیِ P(z|e) به شاهد توجه می‌کند، زیرا مقادیرِ نمونه‌برداری‌شدهٔ هر Zi شاهدِ میانِ غیرنیاکانِ Zi را نادیده می‌گیرند. برای نمونه، هنگام نمونه‌برداری از Sprinkler و

Rain الگوریتم شاهدِ موجود در متغیرِ فرزندِ WetGrass=true را نادیده می‌گیرد؛ این بدان معناست نمونه‌های بسیاری با Sprinkler=false و Rain=false تولید خواهد کرد، با وجودِ اینکه شاهد در واقع این حالت را رد می‌کند. آن نمونه‌ها وزنِ صفر خواهند داشت.

چون وزن‌گذاریِ درستنمایی از همهٔ نمونه‌های تولیدشده استفاده می‌کند، می‌تواند بسیار کارآمدتر از نمونه‌برداریِ ردّی باشد. با این حال، با افزایشِ شمارِ متغیرهای شاهد دچار افتِ عملکرد خواهد شد. این به این دلیل است که بیشترِ نمونه‌ها وزن‌های بسیار پایینی خواهند داشت و از این رو برآوردِ وزندار توسط کسرِ ریزی از نمونه‌ها مسلط خواهد شد که بیش از درستنماییِ بی‌نهایت‌کوچک به شاهد می‌دهند. اگر متغیرهای شاهد «پایین‌دست» رخ دهند—یعنی دیر در ترتیبِ متغیرها—مسئله تشدید می‌شود، زیرا آن‌گاه متغیرهای غیرشاهد هیچ شاهدی در والدها و نیاکانشان برای هدایتِ تولیدِ نمونه‌ها نخواهند داشت. این بدان معناست نمونه‌ها صرفاً «توهماتی» خواهند بود—شبیه‌سازی‌هایی که شباهتِ اندکی به واقعیتِ پیشنهادشده توسط شاهد دارند.

وقتی روی نسخهٔ گسستهٔ شبکهٔ بیمهٔ خودرو در شکل ۱۳.۹ اعمال شود، وزن‌گذاریِ درستنمایی به‌طورِ قابلِ‌ملاحظه‌ای کارآمدتر از نمونه‌برداریِ ردّی است (شکل ۱۳.۱۹ را ببینید). شبکهٔ بیمه برای وزن‌گذاریِ درستنمایی حالتِ نسبتاً خوش‌خیم است، زیرا بسیاری از شواهد «بالادست»اند و متغیرهای پرس‌وجو گره‌های برگِ شبکه‌اند.



شکل ۱۳.۱۹ — عملکردِ نمونه‌برداریِ ردّی و وزن‌گذاریِ درستنمایی روی شبکهٔ بیمه. محورِ x شمارِ نمونه‌های تولیدشده و محورِ y بیشینهٔ خطایِ مطلق در هر یک از مقادیرِ احتمال برای پرس‌وجویی روی PropertyCost را نشان می‌دهد.

۱۳.۴.۲ استنتاج با شبیه‌سازیِ زنجیرهٔ مارکوف

الگوریتم‌های «مونت‌کارلویِ زنجیرهٔ مارکوف» (Markov Chain Monte Carlo یا MCMC) متفاوت از نمونه‌برداریِ ردّی و وزن‌گذاریِ درستنمایی کار می‌کنند. به‌جای تولیدِ هر نمونه از صفر، الگوریتم‌های MCMC نمونه‌ای را با ایجادِ تغییرِ تصادفی در نمونهٔ قبلی تولید می‌کنند. الگوریتمِ MCMC را عاملی در «حالتِ کنونیِ» خاصی در نظر بگیرید که برای هر متغیر مقداری مشخص می‌کند و با ایجادِ تغییرهای تصادفی در حالتِ کنونی حالتِ بعدی را تولید می‌کند.

اصطلاحِ «زنجیرهٔ مارکوف» (Markov Chain) به فرایندِ تصادفیِ تولیدکنندهٔ توالی از حالت‌ها اشاره دارد. (زنجیره‌های مارکوف در فصل‌های ۱۴ و ۱۷ نیز برجسته‌اند؛ الگوریتمِ بازپختِ شبیه‌سازی‌شده در فصل ۴ و الگوریتمِ WALKSAT در فصل ۷ نیز اعضای خانوادهٔ MCMC هستند.) با توصیفِ شکلِ خاصی از MCMC به نام «نمونه‌برداریِ گیبس» (Gibbs Sampling) شروع می‌کنیم، که خاصاً برای شبکه‌های بیزی مناسب است. سپس الگوریتمِ عمومی‌ترِ «متروپلیس–هستینگز» (Metropolis–Hastings) را توصیف می‌کنیم، که انعطافِ بسیار بیشتری در تولیدِ نمونه‌ها اجازه می‌دهد.

نمونه‌برداریِ گیبس در شبکه‌های بیزی. الگوریتمِ نمونه‌برداریِ گیبس برای شبکه‌های بیزی با حالتِ دلخواهی شروع می‌شود (با متغیرهای شاهدِ ثابت‌شده در مقادیرِ مشاهده‌شده‌شان) و با نمونه‌برداریِ تصادفیِ مقداری برای یکی از متغیرهای غیرشاهدِ Xi حالتِ بعدی را تولید می‌کند. از صفحهٔ ۴۱۹ به یاد آورید Xi با توجه به «پوششِ مارکوف»اش (والدها، فرزندان و دیگر والدهای فرزندان) مستقل از همهٔ متغیرهای دیگر است؛ بنابراین، نمونه‌برداریِ گیبس برای Xi یعنی نمونه‌برداریِ شرطی بر مقادیرِ کنونیِ متغیرهای در پوششِ مارکوفش. الگوریتم به‌صورتِ تصادفی در فضایِ حالت—فضایِ تخصیص‌های کاملِ ممکن—پرسه می‌زند، هر بار یک متغیر را برمی‌گرداند، اما متغیرهای شاهد را ثابت نگه می‌دارد. الگوریتمِ کامل در شکل ۱۳.۲۰ نشان داده شده.

پرس‌وجوی P(Rain | Sprinkler=true, WetGrass=true) را برای شبکهٔ شکل ۱۳.۱۵(الف) در نظر بگیرید. متغیرهای شاهدِ Sprinkler و WetGrass در مقادیرِ مشاهده‌شده‌شان ثابت‌اند (هر دو true)، و متغیرهای غیرشاهدِ Cloudy و Rain به‌صورتِ تصادفی مقداردهی اولیه می‌شوند، مثلاً به‌ترتیب true و false. بدین‌ترتیب، حالتِ اولیه [true, true, false, true] است، که در آن مقادیرِ شاهدِ ثابت را پررنگ علامت زده‌ایم. حالا متغیرهای غیرشاهدِ Zi به‌طورِ مکرر در ترتیبی تصادفی، طبقِ توزیعِ احتمالِ ρ(i) برای انتخابِ متغیرها، نمونه‌برداری می‌شوند. برای نمونه:

1. Cloudy انتخاب و سپس، با توجه به مقادیرِ کنونیِ پوششِ مارکوفش، نمونه‌برداری می‌شود: در این مورد، از P(Cloudy | Sprinkler=true, Rain=false) نمونه‌برداری می‌کنیم. فرض کنیم نتیجه Cloudy=false باشد. سپس حالتِ کنونیِ تازه [false, true, false, true] است.

```
function GIBBS-ASK(X, e, bn, N) returns an estimate of P(X|e)
local variables: C, a count vector for each value of X, initially zero
                 Z, the non-evidence variables in bn
x, the current state of the network, initialized from e
initialize x with random values for the variables in Z
for k = 1 to N:
select each variable Zi in Z according to distribution ρ(i)
set the value of Zi in x by sampling from P(Zi | mb(Zi))
C[j] ← C[j] + 1 where xj is the value of X in x
return NORMALIZE(C)
```

شکل ۱۳.۲۰ — الگوریتمِ نمونه‌برداریِ گیبس برای استنتاجِ تقریبی در شبکه‌های بیز.

خوشبختانه، محاسبهٔ توزیعِ پوششِ مارکوفِ P(Xi | mb(Xi))، که در آن mb(Xi) مقادیرِ متغیرهای در پوششِ مارکوفِ Xi یعنی MB(Xi) را نشان می‌دهد، هیچ استنتاجِ پیچیده‌ای در بر ندارد. چنان‌که در تمرین MARB.۱۳ نشان داده شده، توزیع به‌صورتِ زیر است:

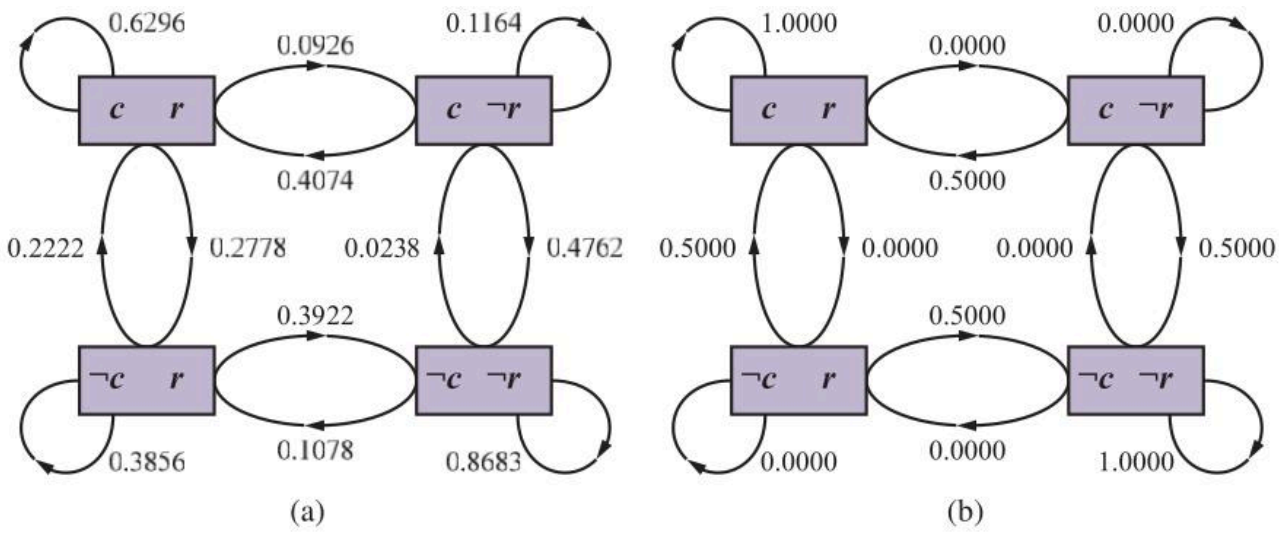
$$P(x_i \mid mb(X_i)) = \alpha\, P(x_i \mid parents(X_i)) \times \prod_{Y_j \in Children(X_i)} P(y_j \mid parents(Y_j)) \qquad (13.10)$$

به بیانِ دیگر، برای هر مقدارِ xi، احتمال با ضربِ احتمال‌ها از CPTهای Xi و فرزندانش داده می‌شود. برای نمونه، در نخستین گامِ نمونه‌برداریِ نشان‌داده‌شدهٔ بالا، از P(Cloudy | Sprinkler=true, Rain=false) نمونه‌برداری کردیم. با معادلهٔ (13.10)، و با مخفف‌کردنِ نامِ متغیرها:

$$P(c|s, \neg r) = \alpha\, P(c)\, P(s|c)\, P(\neg r|c) = \alpha\, 0.5 \cdot 0.1 \cdot 0.2$$
$$P(\neg c|s, \neg r) = \alpha\, P(\neg c)\, P(s|\neg c)\, P(\neg r|\neg c) = \alpha\, 0.5 \cdot 0.5 \cdot 0.8$$

پس توزیعِ نمونه‌برداری $\langle 0.048, 0.952 \rangle \approx \alpha \langle 0.001, 0.020 \rangle$ است.

شکل ۱۳.۲۱(الف) زنجیرهٔ مارکوفِ کامل را برای حالتی نشان می‌دهد که متغیرها یکنواخت انتخاب می‌شوند، یعنی $\rho(Cloudy) = \rho(Rain) = 0.5$. الگوریتم به‌سادگی در این گراف پرسه می‌زند، با احتمال‌های بیان‌شده از پیوندها پیروی می‌کند. هر حالتی که در این فرایند بازدید می‌شود نمونه‌ای است که به برآوردِ متغیرِ پرس‌وجویِ Rain کمک می‌کند. اگر فرایند از ۲۰ حالتِ با Rain=true و ۶۰ حالتِ با Rain=false بازدید کند، آنگاه پاسخِ پرس‌وجو NORMALIZE($\langle 20, 60 \rangle$) = $\langle 0.25, 0.75 \rangle$ است.



شکل ۱۳.۲۱ — (الف) حالت‌ها و احتمال‌های انتقالِ زنجیرهٔ مارکوف برای پرس‌وجوی P(Rain | Sprinkler=true, WetGrass=true). به حلقه‌های خودی توجه کنید: حالت یکسان می‌ماند وقتی هر متغیر انتخاب و سپس همان مقداری را که از قبل دارد دوباره نمونه‌برداری می‌کند. (ب) احتمال‌های انتقال وقتی CPT برای Rain آن را مقید می‌کند همان مقدارِ Cloudy را داشته باشد.

تحلیلِ زنجیره‌های مارکوف. گفته‌ایم نمونه‌برداریِ گیبس با پرسه‌زدنِ تصادفی در فضایِ حالت برای تولیدِ نمونه‌ها کار می‌کند. برای توضیحِ اینکه چرا نمونه‌برداریِ گیبس درست کار می‌کند—یعنی چرا برآوردهایش در حدِّ نهایی به مقادیرِ درست همگرا می‌شوند—به تحلیلِ دقیقی نیاز خواهیم داشت. (این بخش تا حدی ریاضی است و در خواندنِ نخست قابلِ رد شدن است.)

با برخی از مفاهیمِ پایه برای تحلیلِ زنجیره‌های مارکوف به‌طورِ کلی شروع می‌کنیم. هر چنین زنجیره‌ای با حالتِ اولیه‌اش و «هستهٔ انتقال» (Transition Kernel) $k(x \to x')$—احتمالِ انتقال به حالتِ x' با شروع از حالتِ x—تعریف می‌شود. حالا فرض کنیم زنجیرهٔ مارکوف را برای t گام اجرا می‌کنیم، و $\pi t(x)$ احتمالِ قرارداشتنِ سامانه در حالتِ x در زمانِ t باشد. به‌طورِ مشابه، $\pi t+1(x')$ احتمالِ بودن در حالتِ x' در زمانِ $t+1$ باشد. با توجه به $\pi t(x)$، می‌توانیم $\pi t+1(x')$ را با جمع‌زدن، برای همهٔ حالت‌های x که سامانه می‌توانست در زمانِ t در آن باشد، احتمالِ بودن در x ضرب‌درِ احتمالِ انتقال به x' محاسبه کنیم:

$$\pi t+1(x') = \sum x \; \pi t(x) \; k(x \to x')$$

می‌گوییم زنجیره به «توزیعِ مانای» (Stationary Distribution) خود رسیده اگر $\pi t+1 = \pi t$. این توزیعِ مانا را π بنامیم؛ معادلهٔ تعریف‌کننده‌اش بنابراین چنین است:

$$\pi(x') = \sum x \; \pi(x) \; k(x \to x') \qquad \text{برای همهٔ } x' \qquad (13.11)$$

مشروط بر اینکه هستهٔ انتقالِ k «ارگودیک» (Ergodic) باشد—یعنی هر حالت از هر حالتِ دیگر قابلِ‌دسترس باشد و هیچ دورِ اکیداً متناوبی نباشد—دقیقاً یک توزیعِ π هست که این معادله را برای هرِ k معین ارضا می‌کند. معادلهٔ (13.11) را می‌توان چنین خواند که «خروجیِ» موردانتظار از هر حالت (یعنی «جمعیتِ» کنونی‌اش) با «ورودیِ» موردانتظار از همهٔ حالت‌ها برابر است. یکی از راه‌های بدیهیِ ارضایِ این رابطه این است که جریانِ موردانتظار میان هر جفتِ حالت در هر دو جهت یکسان باشد؛ یعنی:

$$\pi(x) \; k(x \to x') = \pi(x') \; k(x' \to x) \qquad \text{برای همهٔ } x, x' \qquad (13.12)$$

وقتی این معادله‌ها برقرارند، می‌گوییم $k(x \to x')$ با $\pi(x)$ در «توازنِ تفصیلی» (Detailed Balance) است. حالتِ خاص، حلقهٔ خودیِ $x = x'$ است، یعنی انتقال از حالتی به خودش. در آن صورت، شرطِ توازنِ تفصیلی به $\pi(x)k(x \to x) = \pi(x)k(x \to x)$ تبدیل می‌شود که البته برای هر توزیعِ مانای π و هر هستهٔ انتقالِ k به‌طورِ پیش‌پاافتاده صادق است.

می‌توانیم نشان دهیم توازنِ تفصیلی دلالت بر مانایی دارد، صرفاً با جمع‌زدن روی x در معادلهٔ (13.12):

$$\sum x \; \pi(x) \; k(x \to x') = \sum x \; \pi(x') \; k(x' \to x) = \pi(x') \sum x \; k(x' \to x) = \pi(x')$$

که گامِ آخر به این دلیل است که انتقال از x' تضمین‌شده رخ می‌دهد.

چرا نمونه‌برداریِ گیبس کار می‌کند. حالا نشان می‌دهیم نمونه‌برداریِ گیبس برآوردهای سازگاری برای احتمال‌های پسینی برمی‌گرداند. ادعای پایه سرراست است: «توزیعِ مانای» فرایندِ نمونه‌برداریِ گیبس دقیقاً توزیعِ پسینیِ متغیرهای غیرشاهدِ شرطی‌شده بر شاهد است. این ویژگیِ قابلِ‌توجه از شیوهٔ خاصِ حرکتِ فرایندِ نمونه‌برداریِ گیبس از حالتی به حالتِ دیگر نتیجه می‌شود.

تعریفِ عمومیِ نمونه‌برداریِ گیبس این است که متغیری مانند Xi انتخاب و سپس به‌صورتِ شرطی بر مقادیرِ کنونیِ همهٔ متغیرهای دیگر نمونه‌برداری می‌شود. (وقتی به‌طورِ خاص روی شبکه‌های بیز اعمال شود، صرفاً از این واقعیتِ اضافی استفاده می‌کنیم که نمونه‌برداریِ شرطی بر همهٔ متغیرها معادلِ نمونه‌برداریِ شرطی بر پوششِ مارکوفِ متغیر است، چنان‌که در صفحهٔ ۴۱۹ نشان داده شد.) برای این متغیرهای دیگر (به‌جزِ متغیرهای شاهد) از نمادِ $\bar{X}i$ استفاده می‌کنیم؛ مقادیرشان در حالتِ کنونی $\bar{x}i$ است.

برای نوشتنِ هستهٔ انتقالِ $k(x \to x')$ برای نمونه‌برداریِ گیبس، سه حالت باید در نظر گرفته شود:

1. حالت‌های x و x' در دو یا چند متغیر تفاوت دارند. در آن صورت، $k(x \to x') = 0$، زیرا نمونه‌برداریِ گیبس فقط یک متغیر را تغییر می‌دهد.
2. حالت‌ها دقیقاً در یک متغیرِ Xi تفاوت دارند که مقدارش از xi به $x'i$ تغییر می‌کند. احتمالِ چنین وقوعی:

$$k(x \to x') = k((xi, \bar{x}i) \to (x'i, \bar{x}i)) = \rho(i)\, P(x'i \mid \bar{x}i) \qquad (13.13)$$

1. حالت‌ها یکسان‌اند: $x = x'$. در آن صورت، هر متغیری می‌تواند انتخاب شود اما سپس فرایندِ نمونه‌برداری همان مقداری را تولید می‌کند که متغیر از قبل دارد. احتمالِ چنین وقوعی:

$$k(x \to x) = \sum i\, \rho(i)\, k((xi, \bar{x}i) \to (xi, \bar{x}i)) = \sum i\, \rho(i)\, P(xi \mid \bar{x}i)$$

حالا نشان می‌دهیم این تعریفِ عمومیِ نمونه‌برداریِ گیبس معادلهٔ توازنِ تفصیلی را با توزیعِ مانایی برابرِ $P(x|e)$—توزیعِ پسینیِ واقعی روی متغیرهای غیرشاهد—ارضای می‌کند. یعنی نشان می‌دهیم $\pi(x)k(x \to x') = \pi(x')k(x' \to x)$ که در آن $\pi(x) = P(x|e)$، برای همهٔ حالت‌های x و x'.

برای حالت‌های نخست و سومِ داده‌شدهٔ بالا، توازنِ تفصیلی همیشه ارضا می‌شود: اگر دو حالت در دو یا چند متغیر تفاوت داشته باشند، احتمالِ انتقال در هر دو جهت صفر است. اگر $x \neq x'$، آنگاه از معادلهٔ (13.13):

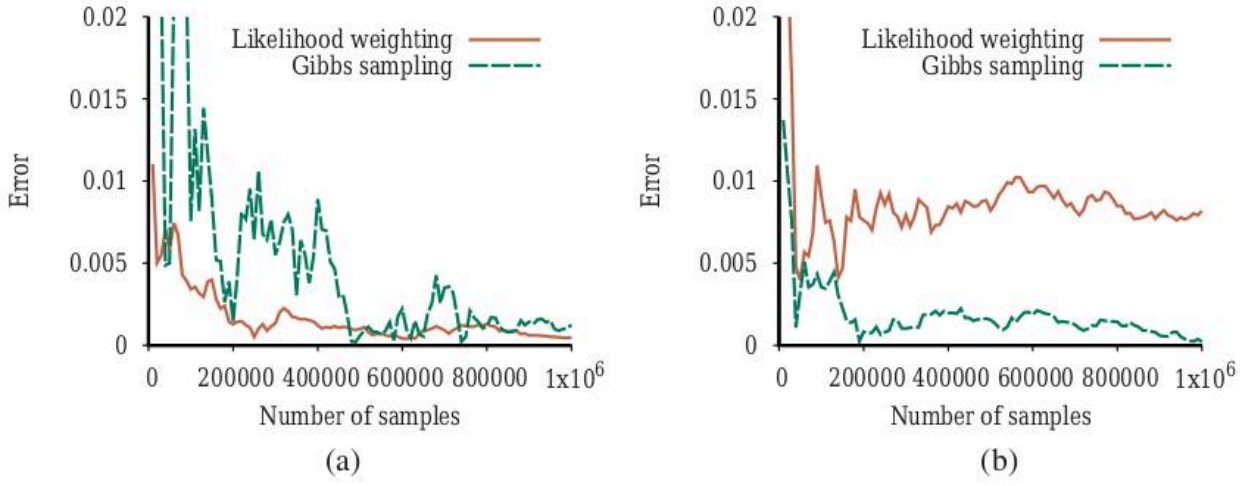
$$\begin{aligned}
\pi(x)\, k(x \to x') &= P(x|e)\, \rho(i)\, P(x'i \mid \bar{x}i, e) \\
&= \rho(i)\, P(xi, \bar{x}i \mid e)\, P(x'i \mid \bar{x}i, e) \\
&= \rho(i)\, P(\bar{x}i \mid xi, e)\, P(xi \mid e)\, P(x'i \mid \bar{x}i, e) && \text{(با قاعدهٔ زنجیره‌ای روی جملهٔ نخست)} \\
&= \rho(i)\, P(\bar{x}i \mid xi, e)\, P(x'i, \bar{x}i \mid e) && \text{(قاعدهٔ زنجیره‌ایِ معکوس روی دو جملهٔ آخر)} \\
&= \pi(x')\, k(x' \to x)
\end{aligned}$$

قطعهٔ پایانیِ پازل ارگودیک‌بودنِ زنجیره است—یعنی هر حالت باید از هر حالتِ دیگر قابلِ‌دسترس باشد و هیچ دورِ دوره‌ای‌ای نباشد. هر دو شرط برآورده می‌شوند مشروط بر اینکه CPTها احتمال‌های 0 یا 1 نداشته باشند. قابلیتِ دسترسی از این واقعیت می‌آید که می‌توانیم یک حالت را با تغییرِ یک‌به‌یکِ متغیرها به حالتِ دیگر تبدیل کنیم، و نبودِ

دورهایِ دوره‌ای از این واقعیت می‌آید که هر حالت حلقهٔ خودی با احتمالِ غیرصفر دارد. از این رو، تحتِ شرایطِ بیان‌شده، k ارگودیک است، بدان معنا که نمونه‌های تولیدشده توسط نمونه‌برداریِ گیبس سرانجام از توزیعِ پسینیِ واقعی گرفته خواهند شد.

پیچیدگیِ نمونه‌برداریِ گیبس. نخست، خبرِ خوب: هر گامِ نمونه‌برداریِ گیبس شاملِ محاسبهٔ توزیعِ پوششِ مارکوف برای متغیرِ انتخاب‌شدهٔ Xi است، که به شمارِ ضرب‌هایی متناسب با شمارِ فرزندانِ Xi و اندازهٔ بردِ Xi نیاز دارد. این مهم است، زیرا بدان معناست کارِ لازم برای تولیدِ هر نمونه مستقل از اندازهٔ شبکه است.

حالا، خبری که لزوماً بد نیست: پیچیدگیِ نمونه‌برداریِ گیبس بسیار دشوارتر از نمونه‌برداریِ ردّی و وزن‌گذاریِ درستنمایی تحلیل می‌شود. نخستین چیزی که باید توجه کرد این است که نمونه‌برداریِ گیبس، برخلافِ وزن‌گذاریِ درستنمایی، به شواهدِ پایین‌دست توجه می‌کند. اطلاعات از گره‌های شاهد در همهٔ جهت‌ها منتشر می‌شود: نخست، هر همسایهٔ گره‌های شاهد مقادیری نمونه‌برداری می‌کند که شاهدِ در آن گره‌ها را بازتاب می‌دهند؛ سپس همسایه‌هایشان، و به همین ترتیب. بدین‌ترتیب، انتظار داریم نمونه‌برداریِ گیبس وقتی شواهد عمدتاً پایین‌دست‌اند از وزن‌گذاریِ درستنمایی بهتر عمل کند؛ و در واقع، این در شکل ۱۳.۲۲ تأیید می‌شود.



شکل ۱۳.۲۲ — عملکردِ نمونه‌برداریِ گیبس در مقایسه با وزن‌گذاریِ درستنمایی روی شبکهٔ بیمهٔ خودرو: (الف) برای پرس‌وجوی استاندارد روی PropertyCost، و (ب) برای حالتی که متغیرهای خروجی مشاهده شده‌اند و Age متغیرِ پرس‌وجوست.

نرخِ همگراییِ نمونه‌برداریِ گیبس—«نرخِ آمیختن» (Mixing Rate) زنجیرهٔ مارکوفِ تعریف‌شده توسط الگوریتم—به‌شدت به ویژگی‌های کمّیِ توزیع‌های شرطیِ شبکه بستگی دارد. برای دیدنِ این، در نظر بگیرید در شکل ۱۳.۱۵(الف) چه اتفاقی می‌افتد وقتی CPT برای Rain قطعی می‌شود: باران می‌بارد اگر و فقط اگر ابری باشد. در آن صورت، توزیعِ پسینیِ واقعی برای پرس‌وجوی P(Rain | sprinkler, wetGrass) تقریباً ⟨0.18, 0.82⟩ است، اما نمونه‌برداریِ گیبس هرگز به این مقدار نخواهد رسید. مسئله این است که تنها دو حالتِ مشترکِ Cloudy و Rain با احتمالِ غیرصفر [true, true] و [false, false] هستند. با شروع از [true, true]، زنجیره هرگز نمی‌تواند به [false, false] برسد، زیرا انتقال‌ها به حالت‌های میانی احتمالِ صفر دارند (شکل ۱۳.۲۱(ب) را ببینید). پس، اگر فرایند از [true, true] شروع شود، همیشه احتمالِ پسینیِ ⟨1.0, 0.0⟩ را برای پرس‌وجو گزارش می‌کند؛ اگر از [false, false] شروع شود، همیشه ⟨0.0, 1.0⟩ گزارش می‌کند.

نمونه‌برداریِ گیبس در این مورد شکست می‌خورد، زیرا رابطهٔ قطعیِ میان Cloudy و Rain ویژگیِ ارگودیکِ لازم برای همگرایی را می‌شکند. اما اگر رابطه را تقریباً قطعی کنیم، همگرایی بازمی‌گردد، اما به‌صورتِ خودسرانه‌ای کند رخ می‌دهد. چندین اصلاح هست که به الگوریتم‌های MCMC کمک می‌کنند سریع‌تر آمیخته شوند.

یکی «نمونه‌برداریِ بلوکی» (Block Sampling) است: نمونه‌برداریِ هم‌زمانِ چند متغیر. در این مورد، می‌توانیم Cloudy و Rain را با هم، شرطی بر پوششِ مارکوفِ ترکیبی‌شان، نمونه‌برداری کنیم. دیگری تولیدِ هوشمندانه‌ترِ حالت‌های بعدی است، چنان‌که در بخشِ بعد خواهیم دید.

نمونه‌برداریِ متروپلیس–هستینگز. روشِ «متروپلیس–هستینگز» یا MH شاید پرکاربردترین الگوریتمِ MCMC باشد. مانندِ نمونه‌برداریِ گیبس، MH طراحی شده تا نمونه‌های x را (سرانجام) طبقِ احتمال‌های هدفِ π(x) تولید کند؛ در موردِ استنتاج در شبکه‌های بیزی، π(x) = P(x|e) می‌خواهیم. مانندِ بازپختِ شبیه‌سازی‌شده (صفحهٔ ۱۱۵)، MH در هر تکرارِ فرایندِ نمونه‌برداری دو مرحله دارد:

1. حالتِ تازهٔ x′ را از «توزیعِ پیشنهادی» (Proposal Distribution) q(x′|x)، با توجه به حالتِ کنونیِ x، نمونه‌برداری کن.
2. x′ را به‌صورتِ احتمالاتی بپذیر یا رد کن، طبقِ «احتمالِ پذیرشِ» (Acceptance Probability):

```
a(x′ | x) = min(1, π(x′) q(x | x′) / (π(x) q(x′ | x)))
```

اگر پیشنهاد رد شود، حالت در x باقی می‌ماند.

هستهٔ انتقالِ MH از این فرایندِ دوقسمتی تشکیل شده. توجه کنید اگر پیشنهاد رد شود، زنجیره در همان حالت می‌ماند.

توزیعِ پیشنهادی مسئولِ پیشنهادِ حالتِ بعدیِ x′ است، چنان‌که نامش می‌گوید. برای نمونه، q(x′|x) را می‌توان چنین تعریف کرد:

- با احتمالِ 0.95، یک گامِ نمونه‌برداریِ گیبس برای تولیدِ x′ انجام بده.
- در غیرِ این صورت، x′ را با اجرایِ الگوریتمِ WEIGHTED-SAMPLE از صفحهٔ ۴۴۰ تولید کن.

این توزیعِ پیشنهادی باعث می‌شود MH حدود ۲۰ گامِ گیبس انجام دهد و سپس فرایند را از حالتِ تازه‌ای (با فرضِ پذیرش) که از صفر تولید شده «راه‌اندازیِ دوباره» کند. با این حیله، از مسئلهٔ گیرکردنِ نمونه‌برداریِ گیبس در بخشی از فضایِ حالت و ناتوانی‌اش در رسیدن به بخش‌های دیگر دور می‌زند.

ممکن است بپرسید ما از کجا می‌دانیم MH با چنین پیشنهادِ عجیبی در واقع به پاسخِ درست همگرا می‌شود. نکتهٔ قابلِ‌توجه دربارهٔ MH این است که همگرایی به توزیعِ مانایِ درست برای هر توزیعِ پیشنهادی تضمین شده، مشروط بر اینکه هستهٔ انتقالِ حاصل ارگودیک باشد. این ویژگی از شیوهٔ تعریفِ احتمالِ پذیرش نتیجه می‌شود. همانندِ نمونه‌برداریِ گیبس، حلقهٔ خودیِ x′ = x به‌طورِ خودکار توازنِ تفصیلی را ارضا می‌کند، پس روی حالتِ x′ ≠ x تمرکز می‌کنیم. این فقط در صورتی رخ می‌دهد که پیشنهاد پذیرفته شود. احتمالِ وقوعِ چنین انتقالی:

```
k(x → x′) = q(x′ | x) a(x′ | x)
```

همانندِ نمونه‌برداریِ گیبس، اثباتِ توازنِ تفصیلی یعنی نشان‌دادنِ جریانِ x به x'، یعنی $\pi(x)k(x \to x')$، با جریانِ x' به x، یعنی $\pi(x')k(x' \to x)$، هم‌خوان است. پس از جای‌گذاریِ عبارتِ بالا برای $k(x \to x')$، اثبات کاملاً سرراست است:

```
                                                                π(x) q(x′|x) a(x′|x)
(تعریف a(·|·))   π(x) q(x′|x) min(1, π(x′)q(x|x′) / (π(x)q(x′|x))) =
(ضرب‌کردن به درون)                                  min(π(x)q(x′|x), π(x′)q(x|x′)) =
(تقسیم‌کردن به درون)  π(x′) q(x|x′) min(π(x)q(x′|x) / (π(x′)q(x|x′)), 1) =
                                                                π(x′) q(x|x′) a(x|x′) =
```

جدا از ویژگی‌های ریاضی، بخشِ مهمِ MH که باید رویش تمرکز کرد نسبتِ $\pi(x')/\pi(x)$ در احتمالِ پذیرش است. این می‌گوید اگر حالتِ بعدیِ پیشنهادشده محتمل‌تر از حالتِ کنونی باشد، قطعاً پذیرفته خواهد شد. (فعلاً جملهٔ $q(x|x')/q(x'|x)$ را نادیده می‌گیریم، که برای تضمینِ توازنِ تفصیلی آن‌جاست و در بسیاری از فضاهایِ حالت به دلیلِ تقارن برابر ۱ است.) اگر حالتِ پیشنهادی از حالتِ کنونی کم‌محتمل‌تر باشد، احتمالِ پذیرشش متناسباً افت می‌کند.

بدین‌ترتیب، یکی از رهنمودها برای طراحیِ توزیع‌های پیشنهادی این است که مطمئن شویم حالت‌های تازهٔ پیشنهادشده به‌صورتِ معقولی محتمل‌اند. نمونه‌برداریِ گیبس این را به‌طورِ خودکار انجام می‌دهد: از توزیعِ گیبسِ $P(X_i \mid \bar{x}_i)$ پیشنهاد می‌دهد، بدان معنا که احتمالِ تولیدِ هر مقدارِ تازهٔ خاص برای X_i مستقیماً متناسب با احتمالِ آن است. (تمرین GIBM.۱۳ از شما می‌خواهد نشان دهید گیبس حالتِ خاصِ MH با احتمالِ پذیرشِ ۱ است.) رهنمودِ دیگر این است که مطمئن شویم زنجیره خوب آمیخته می‌شود، که یعنی گاهی پیشنهادِ حرکت‌های بزرگ به بخش‌های دورِ فضایِ حالت. در مثالِ داده‌شدهٔ بالا، استفادهٔ گاه‌به‌گاه از WEIGHTED-SAMPLE برای راه‌اندازیِ دوبارهٔ زنجیره در حالتِ تازه همین هدف را برآورده می‌کند.

جدا از آزادیِ تقریباً کامل در طراحیِ توزیع‌های پیشنهادی، MH دو ویژگیِ اضافی دارد که آن را عملی می‌کند. نخست، احتمالِ پسینیِ $\pi(x) = P(x|e)$ در محاسبهٔ پذیرش فقط به شکلِ نسبتِ $\pi(x')/\pi(x)$ ظاهر می‌شود، که بسیار خوش‌شانسانه است. محاسبهٔ مستقیمِ $P(x|e)$ دقیقاً همان محاسبه‌ای است که می‌خواهیم با MH تقریب بزنیم، پس معنا نداشت آن را برای هر نمونه انجام دهیم! در عوض، از حقهٔ زیر استفاده می‌کنیم:

```
π(x′) / π(x) = P(x′|e) / P(x|e) = (P(x′, e) / P(e)) × (P(e) / P(x, e)) = P(x′, e) / P(x,
                                                                                    e)
```

جمله‌های این نسبت احتمال‌های مشترکِ کامل‌اند، یعنی حاصل‌ضرب‌های احتمال‌های شرطیِ شبکهٔ بیز. دومین ویژگیِ مفیدِ این نسبت این است که تا وقتی توزیعِ پیشنهادی فقط تغییرهای محلی در x برای تولیدِ x' انجام می‌دهد، فقط شمارِ کمی از جمله‌های حاصل‌ضربِ احتمال‌های شرطی متفاوت خواهند بود. همهٔ احتمال‌های شرطیِ شاملِ متغیرهایی که مقادیرشان تغییر نکرده در نسبت حذف می‌شوند. پس، همانندِ نمونه‌برداریِ گیبس، کارِ لازم برای تولیدِ هر نمونه مستقل از اندازهٔ شبکه است، تا زمانی که تغییرهای حالت محلی باشند.

۱۳.۴.۳ کامپایلِ استنتاجِ تقریبی

الگوریتم‌های نمونه‌برداری در شکل‌های ۱۳.۱۷، ۱۳.۱۸ و ۱۳.۲۰ ویژگیِ مشترکی دارند: آن‌ها روی شبکهٔ بیزی که به‌صورتِ ساختارِ داده بازنمایی شده عمل می‌کنند. این کاملاً طبیعی به نظر می‌رسد: بالاخره، شبکهٔ بیز گرافِ جهت‌داریِ بدونِ دور است، پس چگونه دیگر می‌شد آن را بازنمایی کرد؟ مسئلهٔ این رویکرد این است که عملیاتِ لازم برای دسترسی به ساختارِ داده—برای نمونه، یافتنِ والدهای یک گره—هزاران یا میلیون‌ها بار در طولِ اجرایِ الگوریتمِ نمونه‌برداری تکرار می‌شوند، و همهٔ این محاسبه‌ها کاملاً غیرضروری‌اند.

ساختارِ شبکه و احتمال‌های شرطی‌اش در طولِ محاسبه ثابت می‌مانند، پس فرصتی هست شبکه را به کدِ استنتاجِ مخصوصِ مدل کامپایل کنیم که فقط محاسبه‌های استنتاجیِ لازم برای آن شبکهٔ خاص را انجام دهد. (اگر این آشنا به نظر می‌رسد، همان ایده‌ای است که در کامپایلِ برنامه‌های منطقی در فصل ۹ استفاده شد.) برای نمونه، فرض کنیم می‌خواهیم متغیرِ Earthquake را در شبکهٔ سرقتِ شکل ۱۳.۲ گیبس-نمونه‌برداری کنیم. طبقِ الگوریتمِ GIBBS-ASK در شکل ۱۳.۲۰، باید محاسبهٔ زیر را انجام دهیم:

```
مقدارِ Earthquake در x را با نمونه‌برداری از P(Earthquake | mb(Earthquake)) بگذار
```

که در آن توزیعِ اخیر طبقِ معادلهٔ (13.10) محاسبه می‌شود. این محاسبه، به‌نوبهٔ خود، مستلزم جستجو برای والدها و فرزندانِ Earthquake در ساختارِ شبکهٔ بیز است؛ جستجو برای مقادیرِ کنونی‌شان؛ استفاده از آن مقادیر برای نمایه‌زدن در CPTهای متناظر (که آن‌ها نیز باید از شبکهٔ بیز پیدا شوند)؛ و ضربِ همهٔ ردیف‌های مناسب از آن CPTها برای تشکیلِ توزیعِ تازه‌ای که از آن نمونه‌برداری کنیم. سرانجام، چنان‌که در صفحهٔ ۴۳۶ ذکر شد، خودِ گامِ نمونه‌برداری باید نسخهٔ تجمعیِ توزیعِ گسسته را بسازد و سپس مقدارِ موجود در آن را که با عددِ تصادفیِ نمونه‌برداری‌شده از [0,1] متناظر است بیابد.

اگر در عوض شبکه را کامپایل کنیم، کدِ نمونه‌برداریِ مخصوصِ مدل برای متغیرِ Earthquake به‌دست می‌آوریم که به‌صورتِ زیر است:

```
r ← نمونهٔ تصادفیِ یکنواخت از [0,1]
اگر Alarm = true
  پس اگر Burglary = true
    پس بازگرداندن [r < 0.0020212]
    وگرنه بازگرداندن [r < 0.36755]
  وگرنه اگر Burglary = true
    پس بازگرداندن [r < 0.0016672]
    وگرنه بازگرداندن [r < 0.0014222]
```

در این‌جا، متغیرهای شبکهٔ بیزِ Burglary، Alarm و مانندِ آن متغیرهای برنامهٔ معمولی با مقادیری می‌شوند که حالتِ کنونیِ زنجیرهٔ مارکوف را می‌سازند. عبارت‌های عددیِ آستانه به‌صورتِ true یا false ارزیابی می‌شوند و توزیع‌های گیبسِ ازپیش‌محاسبه‌شده را برای هر ترکیب از مقادیر در پوششِ مارکوفِ Earthquake بازنمایی می‌کنند. کد چندان زیبا

نیست—معمولاً تقریباً به اندازهٔ خودِ شبکهٔ بیز بزرگ خواهد بود—اما به‌طورِ باورنکردنی کارآمد است. در مقایسه با GIBBS-ASK، کدِ کامپایل‌شده معمولاً ۲ تا ۳ مرتبهٔ بزرگی سریع‌تر خواهد بود. می‌تواند ده‌ها میلیون گامِ نمونه‌برداری در ثانیه روی لپ‌تاپِ معمولی انجام دهد، و سرعتش عمدتاً با هزینهٔ تولیدِ اعدادِ تصادفی محدود می‌شود.

۱۳.۵ شبکه‌های علّی

دربارهٔ چند مزیتِ هم‌خوان‌نگه‌داشتنِ ترتیبِ گره‌ها در شبکه‌های بیز با جهتِ علّیت بحث کرده‌ایم. به‌طورِ خاص، به سهولتِ ارزیابیِ احتمال‌های شرطی اگر چنین ترتیبی حفظ شود، و نیز فشردگیِ ساختارِ شبکهٔ حاصل، اشاره کردیم. اما اشاره کردیم که اصولاً هر ترتیبِ گره‌ای ساختِ سازگاری از شبکه را برای بازنماییِ تابعِ توزیعِ مشترک مجاز می‌کند. این در شکل ۱۳.۳ نشان داده شد، که در آن تغییرِ ترتیبِ گره‌ها شبکه‌هایی تولید کرد پرپشت‌تر و بسیار کم‌طبیعی‌تر از شبکهٔ اصلی در شکل ۱۳.۲، اما با این وجود ما را قادر می‌کرد همان توزیع را روی همهٔ متغیرها بازنمایی کنیم.

این بخش «شبکه‌های علّی» (Causal Networks) را توصیف می‌کند، ردهٔ مقیدی از شبکه‌های بیزی که همهٔ ترتیب‌های به‌جز ترتیب‌های سازگارِ علّی را منع می‌کند. بررسی می‌کنیم چگونه چنین شبکه‌هایی بسازیم، با چنین ساخت‌هایی چه به دست می‌آید، و چگونه این سود را در تکلیف‌های تصمیم‌گیری به کار بگیریم.

ساده‌ترین شبکهٔ بیزیِ قابلِ‌تصور را در نظر بگیرید، پیکانِ تکیِ Fire → Smoke. به ما می‌گوید متغیرهای Fire و Smoke ممکن است وابسته باشند، پس برای مشخص‌کردنِ توزیعِ مشترکِ P(Fire, Smoke) باید پیشینِ P(Fire) و احتمالِ شرطیِ P(Smoke | Fire) را مشخص کنیم. اما این توزیع را می‌توان به‌خوبی با پیکانِ معکوسِ Smoke ← Fire بازنمایی کرد، با استفاده از P(Smoke) و P(Fire | Smoke) مناسبِ محاسبه‌شده از قاعدهٔ بیز. این ایده که این دو شبکه معادل‌اند، از این رو اطلاعاتِ یکسانی منتقل می‌کنند، در بیشترِ مردم ناراحتی و حتی مقاومت برمی‌انگیزد. چگونه می‌توانند اطلاعاتِ یکسانی منتقل کنند وقتی می‌دانیم Fire موجب Smoke می‌شود و نه برعکس؟ به بیانِ دیگر، از تجربه و فهمِ علمی‌مان می‌دانیم پاک‌کردنِ دود آتش را متوقف نمی‌کند و خاموش‌کردنِ آتش دود را متوقف می‌کند. بنابراین انتظار داریم این عدمِ تقارن را از طریقِ جهت‌داریِ پیکان میان‌شان بازنمایی کنیم. اما اگر برگرداندنِ پیکان فقط چیزها را معادل کند، چگونه می‌توانیم امیدوار باشیم این اطلاعاتِ مهم را به‌صورتِ صوری بازنمایی کنیم.

«شبکه‌های بیزیِ علّی»، که گاهی «نمودارهای علّی» (Causal Diagrams) نامیده می‌شوند، طراحی شده‌اند تا به ما اجازه دهند عدمِ تقارن‌های علّی را بازنمایی کنیم و از عدمِ تقارن‌ها برای استدلال با اطلاعاتِ علّی بهره بگیریم. ایده این است که دربارهٔ جهت‌داریِ پیکان‌ها با ملاحظاتی تصمیم بگیریم که فراتر از وابستگیِ احتمالاتی‌اند و نوعِ کاملاً متفاوتی از قضاوت را فرا می‌خوانند. به‌جای پرسیدن از کارشناس که آیا Smoke و Fire به‌صورتِ احتمالاتی وابسته‌اند، چنان‌که در شبکه‌های بیزیِ معمولی می‌کنیم، حالا می‌پرسیم کدام به کدام پاسخ می‌دهد، Smoke به Fire یا Fire به Smoke؟

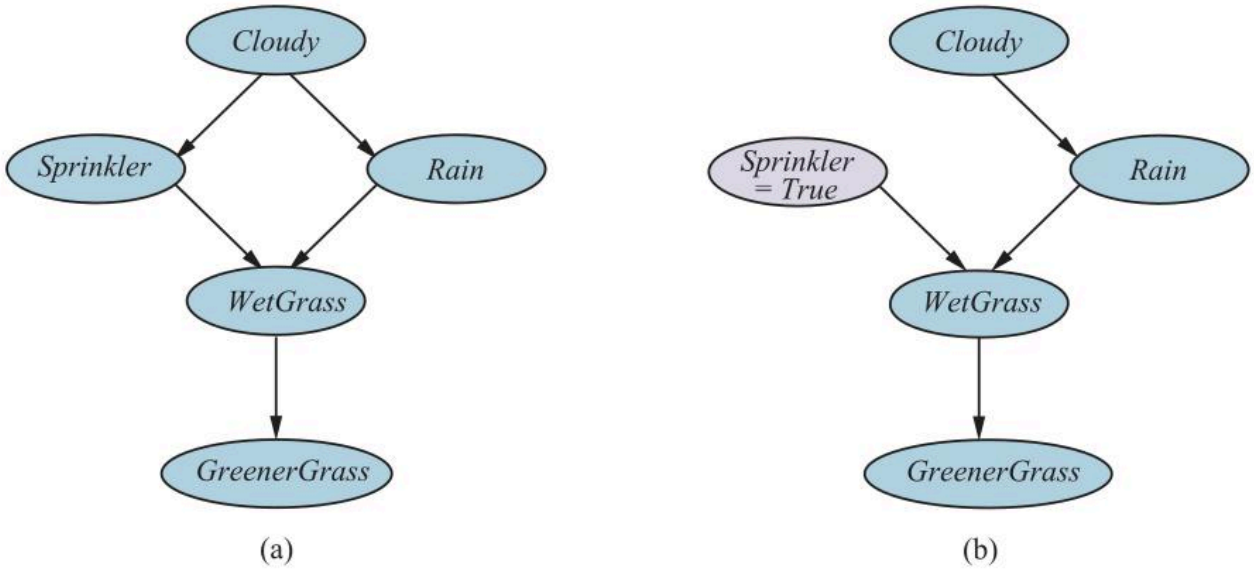
این ممکن است تا حدی عرفانی به نظر برسد، اما می‌توان از طریقِ مفهومِ «تخصیص» (Assignment) دقیقش کرد، مشابهِ عملگرِ تخصیص در زبان‌های برنامه‌نویسی. اگر طبیعت مقداری را به Smoke بر اساسِ آنچه طبیعت دربارهٔ Fire می‌آموزد تخصیص دهد، پیکانی از Smoke به Fire رسم می‌کنیم. مهم‌تر، اگر قضاوت کنیم طبیعت به Fire مقدارِ

صدقی تخصیص می‌دهد که به متغیرهای دیگر بستگی دارد، نه Smoke، از رسمِ پیکانِ Smoke ← Fire خودداری می‌کنیم. به بیانِ دیگر، مقدارِ xi هر متغیرِ Xi توسط معادله‌ای به‌صورتِ xi = fi(OtherVariables) تعیین می‌شود، و پیکانِ Xi ← Xj رسم می‌شود اگر و فقط اگر Xj یکی از آرگومان‌های fi باشد.

معادلهٔ xi = fi(·) «معادلهٔ ساختاری» (Structural Equation) نامیده می‌شود، زیرا سازوکارِ پایداری در طبیعت را توصیف می‌کند که برخلافِ احتمال‌های کمّی‌کنندهٔ شبکهٔ بیزی، نسبت به اندازه‌گیری‌ها و تغییرهای محلیِ محیط ناوردا می‌ماند.

برای درکِ این پایداری نسبت به تغییرهای محلی، شکل ۱۳.۲۳(الف) را در نظر بگیرید، که نسخهٔ کمی تغییر یافتهٔ داستانِ آب‌پاشِ چمنِ شکل ۱۳.۱۵ را به تصویر می‌کشد. برای بازنماییِ آب‌پاشِ ازکارافتاده، برای نمونه، به‌سادگی همهٔ پیوندهای واقعه بر گرهٔ Sprinkler را از شبکه حذف می‌کنیم. برای بازنماییِ چمنِ پوشیده‌شده با چادر، به‌سادگی پیکانِ Rain → WetGrass را حذف می‌کنیم.

هر پیکربندیِ محلیِ دوبارهٔ سازوکارهای محیط بدین‌ترتیب می‌تواند، با اصلاحِ جزیی، به پیکربندیِ دوبارهٔ هم‌ریختی از توپولوژیِ شبکه ترجمه شود. اگر شبکه برخلافِ ترتیبِ علّی ساخته شده بود، تبدیلِ بسیار پرکارتری لازم می‌شد. این پایداریِ محلی به‌طورِ خاص برای بازنماییِ کنش‌ها یا مداخله‌ها مهم است، موضوعِ بعدیِ بحثِ ما.



شکل ۱۳.۲۳ — (الف) شبکهٔ بیزیِ علّی که روابطِ علت–اثر را میان پنج متغیر (ابر، باران، آب‌پاش، چمنِ خیس، چمنِ سبزتر) بازنمایی می‌کند. (ب) شبکه پس از انجامِ کنشِ «آب‌پاش را روشن کن».

۱۳.۵.۱ بازنماییِ کنش‌ها: عملگرِ do

داستانِ آب‌پاشِ شکل ۱۳.۲۳(الف) را باز در نظر بگیرید. طبقِ معناشناسیِ استانداردِ شبکه‌های بیز، توزیعِ مشترکِ پنج متغیر با حاصل‌ضربِ پنج توزیعِ شرطی داده می‌شود:

$$P(c, r, s, w, g) = P(c)\,P(r|c)\,P(s|c)\,P(w|r, s)\,P(g|w) \qquad (13.14)$$

که در آن نامِ هر متغیر را با حرفِ نخستش مخفف کرده‌ایم. به‌عنوانِ سامانه‌ای از معادله‌های ساختاری، مدل به‌صورتِ زیر است:

```
C = fC(UC)
R = fR(C, UR)
S = fS(C, US)          (13.15)
W = fW(R, S, UW)
G = fG(W, UG)
```

که در آن، بدونِ ازدست‌دادنِ عمومیت، fC می‌تواند تابعِ همانی باشد. متغیرهای U در این معادله‌ها «متغیرهای مدل‌نشده» (Unmodeled Variables) را بازنمایی می‌کنند، که «جمله‌های خطا» (Error Terms) یا «اغتشاش‌ها» (Disturbances) نیز نامیده می‌شوند و رابطهٔ تابعیِ میان هر متغیر و والدهایش را بر هم می‌زنند. برای نمونه، UW ممکن است منبعِ بالقوهٔ دیگری از خیسی را بازنمایی کند، علاوه بر Sprinkler و Rain—شاید «شبنمِ صبحگاهی» (MorningDew) یا «هلیکوپترِ آتش‌نشان».

اگر همهٔ متغیرهای U متغیرهای تصادفیِ متقابلاً مستقل با پیشین‌های مناسب‌الانتخاب باشند، توزیعِ مشترکِ معادلهٔ (13.14) را می‌توان دقیقاً با معادله‌های ساختاریِ معادلهٔ (13.15) بازنمایی کرد. بدین‌ترتیب، سامانه‌ای از روابطِ تصادفی را می‌توان با سامانه‌ای از روابطِ قطعی در بر گرفت، که هر یک از اغتشاشی برون‌زا متأثر است. اما سامانهٔ معادله‌های ساختاری بیش از این به ما می‌دهد: به ما اجازه می‌دهد پیش‌بینی کنیم مداخله‌ها چگونه بر عملکردِ سامانه و از این رو بر پیامدهای قابلِ‌مشاهدهٔ آن مداخله‌ها اثر خواهند گذاشت. این فقط با توزیعِ مشترک ممکن نیست.

برای نمونه، فرض کنیم آب‌پاش را روشن می‌کنیم—یعنی اگر ما (که طبقِ تعریف، بخشی از فرایندهای علّیِ توصیف‌شده توسط مدل نیستیم) برای تحمیلِ شرطِ Sprinkler=true مداخله کنیم. در نمادِ «حسابِ (Do-Calculus) «do، که بخشِ کلیدیِ نظریهٔ شبکه‌های علّی است، این به‌صورتِ do(Sprinkler=true) نوشته می‌شود. پس از انجام، این بدان معناست متغیرِ آب‌پاش دیگر به ابری‌بودنِ هوا وابسته نیست. بنابراین معادلهٔ S = fS(C, US) را از سامانهٔ معادله‌های ساختاری حذف و آن را با S = true جایگزین می‌کنیم:

```
C = fC(UC)
R = fR(C, UR)
S = true          (13.16)
W = fW(R, S, UW)
G = fG(W, UG)
```

از این معادله‌ها، توزیعِ مشترکِ تازه را برای متغیرهای باقی‌مانده، شرطی بر do(Sprinkler=true)، به دست می‌آوریم:

```
P(c, r, w, g | do(S=true)) = P(c) P(r|c) P(w|r, s=true) P(g|w)          (13.17)
```

این با شبکهٔ «مثله‌شده» در شکل ۱۳.۲۳(ب) متناظر است. از معادلهٔ (13.17)، می‌بینیم تنها متغیرهایی که احتمال‌هایشان تغییر می‌کند WetGrass و GreenerGrass هستند، یعنی فرزندانِ متغیرِ دستکاری‌شدهٔ Sprinkler.

تفاوتِ میانِ شرطی‌کردن بر کنشِ do(Sprinkler=true) در شبکهٔ اصلی و شرطی‌کردن بر مشاهدهٔ Sprinkler=true را توجه کنید. شبکهٔ اصلی به ما می‌گوید وقتی هوا ابری است آب‌پاش کم‌محتمل‌تر روشن است، پس اگر مشاهده کنیم آب‌پاش روشن است، این احتمالِ ابری‌بودنِ هوا را کاهش می‌دهد. اما عقلِ سلیم به ما می‌گوید اگر ما (به بیانِ دیگر، از بیرونِ جهان عمل می‌کنیم) دست دراز کنیم و آب‌پاش را روشن کنیم، این بر آب‌وهوا اثر نمی‌گذارد یا اطلاعاتِ تازه‌ای دربارهٔ اینکه هوای آن روز چگونه است فراهم نمی‌کند. چنان‌که در شکل ۱۳.۲۳(ب) نشان داده شده، مداخله پیوندِ علّیِ عادی میان آب‌وهوا و آب‌پاش را می‌شکند. این از جریان‌یافتنِ هر تأثیری به‌صورتِ عقب‌گرد از Sprinkler به Cloudy جلوگیری می‌کند. بدین‌ترتیب، شرطی‌کردن بر do(Sprinkler=true) در گرافِ اصلی معادلِ شرطی‌کردن بر Sprinkler=true در گرافِ مثله‌شده است.

رویکردِ مشابهی را می‌توان برای تحلیلِ اثرِ do(Xj = xjk) در شبکهٔ علّیِ عمومی با متغیرهای X1,...,Xn به کار گرفت. شبکه با توزیعِ مشترکی متناظر است که به‌صورتِ معمول تعریف می‌شود (معادلهٔ (13.2) را ببینید):

$$P(x1,...,xn) = \prod_{i=1..n} P(xi \mid parents(Xi)) \qquad (13.18)$$

پس از اعمالِ do(Xj = xjk)، توزیعِ مشترکِ تازهٔ Pxjk صرفاً عاملِ Xj را حذف می‌کند:

$$Pxjk(x1,...,xn) = \prod_{i \neq j} P(xi \mid parents(Xi)) \quad \text{اگر } xj = xjk \quad (13.19)$$
$$= 0 \quad \text{اگر } xj \neq xjk$$

یعنی Pxjk(x1,...,xn) = P(x1,...,xn) / P(xj | parents(Xj)) برای xj = xjk. این از این واقعیت نتیجه می‌شود که گذاشتنِ Xj روی مقدارِ خاصِ xjk متناظر با حذفِ معادلهٔ Xj = fj(Parents(Xj), Uj) از سامانهٔ معادله‌های ساختاری و جایگزینیِ آن با Xj = xjk است. با کمی دستکاریِ جبریِ بیشتر، می‌توان فرمولی برای اثرِ گذاشتنِ متغیرِ Xj روی هر متغیرِ دیگرِ Xi استخراج کرد:

$$P(Xi = xi \mid do(Xj = xjk)) = Pxjk(Xi = xi) = \sum_{parents(Xj)} P(xi \mid xjk, parents(Xj)) \, P(parents(Xj)) \qquad (13.20)$$

جمله‌های احتمال در جمع با محاسبه روی شبکهٔ اصلی، با هر یک از الگوریتم‌های استنتاجِ استاندارد، به دست می‌آیند. این معادله به‌عنوانِ «فرمولِ تعدیل» (Adjustment Formula) شناخته می‌شود؛ میانگینِ احتمال-وزن‌دارِ اثرِ Xj و والدهایش بر Xi است، که در آن وزن‌ها پیشین‌های مقادیرِ والدند. اثرهای مداخله بر چند متغیر را می‌توان با تصورِ اینکه مداخله‌های فردی به‌ترتیب رخ می‌دهند محاسبه کرد، و هر یک به‌نوبت تأثیرهای علّیِ روی متغیری را حذف و مدلِ تازهٔ مثله‌شده‌ای می‌دهد.

۱۳.۵.۲ معیارِ درِ پشتی

توانایی پیش‌بینیِ اثرِ هر مداخله‌ای نتیجه‌ای قابلِ‌توجه است، اما به دانشِ دقیقِ توزیع‌های شرطیِ لازم در مدل نیاز دارد، به‌طورِ خاص P(xj | parents(Xj)). اما در بسیاری از محیط‌های دنیایِ واقعی، این توقعی بیش از حد است. برای نمونه، می‌دانیم «عواملِ ژنتیکی» در چاقی نقش دارند، اما نمی‌دانیم کدام ژن‌ها نقش دارند یا ماهیتِ دقیقِ اثرهایشان چیست. حتی در داستانِ سادهٔ تصمیم‌های آب‌پاشِ مری (شکل ۱۳.۱۵، که در شکل ۱۳.۲۳(الف) نیز صدق می‌کند)، ممکن است بدانیم او قبل از تصمیم‌گیری دربارهٔ روشن‌کردنِ آب‌پاش آب‌وهوا را بررسی می‌کند، اما ممکن است ندانیم چگونه تصمیم می‌گیرد. دلیلِ خاصِ مشکل‌بودنِ این در این مورد آن است که می‌خواهیم اثرِ روشن‌کردنِ آب‌پاش را روی متغیرِ پایین‌دستی مانند GreenerGrass پیش‌بینی کنیم، اما فرمولِ تعدیل (معادلهٔ (13.20)) باید نه فقط مسیرِ مستقیم از Sprinkler، بلکه «درِ پشتیِ» مسیرِ عبوری از Cloudy و Rain را نیز در نظر بگیرد. اگر مقدارِ Rain را می‌دانستیم، این مسیرِ درِ پشتی مسدود می‌شد—که پیشنهاد می‌کند شاید راهی برای نوشتنِ فرمولِ تعدیل هست که به‌جای Cloudy بر Rain شرطی شود. و در واقع این ممکن است:

$$P(g \mid do(S=true)) = \sum_r P(g \mid S=true, r)\, P(r) \qquad (13.21)$$

به‌طورِ کلی، اگر بخواهیم اثرِ do(Xj = xjk) را روی متغیرِ Xi بیابیم، «معیارِ درِ پشتی» (Back-Door Criterion) به ما اجازه می‌دهد فرمولِ تعدیلی بنویسیم که بر هر مجموعهٔ متغیرِ Z شرطی می‌شود که درِ پشتی را، به بیانِ دیگر، می‌بندد. در زبانِ فنی‌تر، مجموعهٔ Z را می‌خواهیم به‌گونه‌ای که Xi با توجه به Xj و Z به‌صورتِ شرطی مستقل از Parents(Xj) باشد. این کاربردِ سرراستِ جداسازیِ-d است (صفحهٔ ۴۱۹ را ببینید).

معیارِ درِ پشتی بلوکِ ساختمانیِ پایه‌ای برای نظریه‌ای از استدلالِ علّی است که در دو دههٔ گذشته پدید آمده. راهی برای استدلالِ برضدِ یک قرنِ جزمِ آماری فراهم می‌کند که ادعا می‌کرد فقط «کارآزماییِ تصادفی‌شده با کنترل» (Randomized Controlled Trial) می‌تواند اطلاعاتِ علّی فراهم کند. این نظریه ابزارهای مفهومی و الگوریتم‌هایی برای تحلیلِ علّی در گسترهٔ وسیعی از محیط‌های غیرآزمایشی و شبه‌آزمایشی فراهم کرده؛ برای محاسبهٔ احتمال‌های روی گزاره‌های «خلافِ واقع» (Counterfactual) («اگر به‌جایش این اتفاق افتاده بود، احتمال چه بود؟»)؛ برای تعیینِ اینکه یافته‌ها در جمعیتی چه زمانی می‌توانند به جمعیتِ دیگر منتقل شوند؛ و برای مدیریتِ همهٔ شکل‌های دادهٔ مفقود هنگام یادگیریِ مدل‌های احتمال.

خلاصهٔ فصل

این فصل شبکه‌های بیزی را توصیف کرده، بازنماییِ توسعه‌یافته‌ای برای دانشِ نامطمئن. شبکه‌های بیزی نقشی تقریباً مشابهِ نقشِ منطقِ گزاره‌ای برای دانشِ قطعی بازی می‌کنند.

- شبکهٔ بیزی گرافِ جهتداریِ بدونِ دور است که گره‌هایش با متغیرهای تصادفی متناظرند؛ هر گره توزیعِ شرطی برای گره، به‌شرطِ والدهایش، دارد.
- شبکه‌های بیزی راهی موجز برای بازنماییِ روابطِ استقلالِ شرطی در حوزه فراهم می‌کنند.
- شبکهٔ بیزی توزیعِ احتمالِ مشترکی روی متغیرهایش مشخص می‌کند. احتمالِ هر تخصیصِ معین به همهٔ متغیرها به‌صورتِ حاصلِ‌ضربِ درایه‌های متناظر در توزیع‌های شرطیِ محلی تعریف می‌شود. شبکهٔ بیزی اغلب به‌صورتِ نمایی کوچک‌تر از توزیعِ مشترکِ صریحاً شمرده‌شده است.
- بسیاری از توزیع‌های شرطی را می‌توان با خانواده‌های متعارفِ توزیع‌ها به‌صورتِ فشرده بازنمایی کرد. شبکه‌های بیزیِ ترکیبی، که شاملِ هر دو متغیرِ گسسته و پیوسته‌اند، از انواعِ توزیع‌های متعارف استفاده می‌کنند.
- استنتاج در شبکه‌های بیزی یعنی محاسبهٔ توزیعِ احتمالِ مجموعه‌ای از متغیرهای پرس‌وجو، با توجه به مجموعه‌ای از متغیرهای شاهد. الگوریتم‌های استنتاجِ دقیق، مانند حذفِ متغیر، جمعِ حاصل‌ضرب‌های احتمال‌های شرطی را تا آن‌جا که ممکن است کارآمد ارزیابی می‌کنند.
- در پلی‌درخت‌ها (شبکه‌های تک‌متصل)، استنتاجِ دقیق زمانی خطی در اندازهٔ شبکه می‌گیرد. در حالتِ عمومی، مسئله ناشدنی است.
- فنونِ نمونه‌برداریِ تصادفی مانند وزن‌گذاریِ درستنمایی و مونت‌کارلویِ زنجیرهٔ مارکوف می‌توانند برآوردهای معقولی از احتمال‌های پسینیِ واقعی در شبکه بدهند و با شبکه‌های بسیار بزرگ‌تری از الگوریتم‌های دقیق کنار بیایند.
- در حالی که شبکه‌های بیز تأثیرهای احتمالاتی را در بر می‌گیرند، شبکه‌های علّی روابطِ علّی را در بر می‌گیرند و پیش‌بینیِ اثرِ مداخله‌ها و نیز مشاهده‌ها را اجازه می‌دهند.

یادداشت‌های کتاب‌شناختی و تاریخی

استفاده از شبکه‌ها برای بازنماییِ اطلاعاتِ احتمالاتی در اوایلِ قرنِ بیستم آغاز شد، با کارِ سول رایت روی تحلیلِ احتمالاتیِ وراثتِ ژنتیکی و عواملِ رشدِ جانوران (رایت، 1921، 1934). ای. جی. گود (1961)، در همکاری با آلن تورینگ، بازنمایی‌های احتمالاتی و روش‌های استنتاجِ بیزی را توسعه داد که می‌توانست پیش‌روِ شبکه‌های بیزیِ مدرن دانسته شود—هرچند مقاله اغلب در این زمینه نقل نمی‌شود.[7] همان مقاله منبعِ اصلیِ مدلِ نویزی-OR است.

بازنماییِ «نمودارِ تأثیر» (Influence Diagram) برای مسائلِ تصمیم، که بازنماییِ DAG را برای متغیرهای تصادفی در بر می‌گرفت، در اواخرِ دههٔ ۱۹۷۰ در تحلیلِ تصمیم استفاده شد (فصل ۱۶ را ببینید)، اما فقط شمارش برای ارزیابی استفاده می‌شد. جودیا پرل روشِ «انتقالِ پیام» (Message Passing) را برای استنتاج در شبکه‌های درختی (پرل، 1982a) و شبکه‌های پلی‌درخت (کیم و پرل، 1983) توسعه داد و اهمیتِ مدل‌های احتمالِ علّی به‌جای تشخیصی را توضیح داد. نخستین سامانهٔ خبرهٔ استفاده‌کننده از شبکه‌های بیزی CONVINCE (کیم، 1983) بود.

چنان‌که در فصل ۱ ثبت شده، اواسطِ دههٔ ۱۹۸۰ شاهدِ رونقِ سامانه‌های خبرهٔ قاعده‌محور بود، که روش‌های موردی برای مدیریتِ عدم قطعیت در بر می‌گرفتند. احتمال هم «غیرعملی» و هم «از نظرِ شناختی نامحتمل» به‌عنوانِ مبنایی برای استدلال دانسته می‌شد. مقالهٔ ستیزه‌جویانهٔ پیتر چیزمن (1985) با عنوانِ «در دفاع از احتمال» و مقالهٔ بعدی‌اش «پژوهشی در فهمِ رایانه‌ای» (چیزمن، 1988، با تفسیرها) به چرخاندنِ میز کمک کرد.

با این حال، احیایِ احتمال عمدتاً به توسعهٔ شبکه‌های بیزی توسط پرل و توسعهٔ گستردهٔ رویکردِ احتمالاتی به هوش مصنوعی بستگی داشت، چنان‌که در کتابش «استدلالِ احتمالاتی در سامانه‌های هوشمند» (پرل، 1988) ترسیم شده. این کتاب هم مسائلِ بازنمایی، از جمله روابطِ استقلالِ شرطی و معیارِ جداسازیِ-d، و هم رویکردهای الگوریتمی را پوشش می‌داد. گایگر و همکاران (1990a) و تیان و همکاران (1998) نتایجِ محاسباتیِ کلیدی در شناساییِ کارآمدِ جداسازیِ-d را ارائه دادند.

یوجین چارنیاک با نگارشِ مقالهٔ پرطرفدارِ «شبکه‌های بیزی بدونِ اشک»[8] (1991) و کتابی در همین زمینه (1993)، به معرفیِ ایده‌های پرل به پژوهشگرانِ هوش مصنوعی کمکِ شایانی کرد. کتابِ دین و ول‌من (1991) نیز در شناساندنِ شبکه‌های بیزی به جامعهٔ هوش مصنوعی نقشِ مؤثری داشت. شاکتر (1998) روشی ساده‌شده برای تعیینِ جداسازیِ-d موسوم به الگوریتمِ «توپِ بیز» (Bayes-ball) ارائه کرد.

با توسعهٔ کاربردهای شبکه‌های بیزی، پژوهشگران دریافتند که لازم است فراتر از مدلِ پایهٔ متغیرهای گسسته با CPTها حرکت کنند. برای نمونه، سامانهٔ CPCS (پرادهان و همکاران، 1994)—یک شبکهٔ بیزی برای پزشکیِ داخلی با ۴۴۸ گره و ۹۰۶ پیوند—استفادهٔ گسترده‌ای از عملگرهای منطقیِ نویزیِ ارائه‌شده توسط گود (1961) به عمل آورد. بوتیلیه و همکاران (1996) مزایای الگوریتمیِ استقلالِ وابسته به بافت را تحلیل کردند. گنجاندنِ متغیرهای تصادفیِ پیوسته در شبکه‌های بیزی توسط پرل (1988) و شاکتر و کنلی (1989) مورد بررسی قرار گرفت؛ این مقاله‌ها شبکه‌هایی را بررسی کردند که تنها شاملِ متغیرهای پیوسته با توزیع‌های خطی-گاوسی بودند.

شبکه‌های ترکیبی (Hybrid Networks) با هر دو نوع متغیرِ گسسته و پیوسته توسط لوریتزن و ورموث (1989) بررسی شدند و در سامانهٔ cHUGIN پیاده‌سازی گشتند (اولسن، 1993). تحلیل‌های بیشتر در موردِ مدل‌های خطی-گاوسی، همراه با پیوندهایی به بسیاری از مدل‌های دیگر در آمار، در مقالهٔ روویس و قهرامانی (1999) آمده است؛ لرنر (2002) بحثی بسیار جامع از کاربردِ آن‌ها در شبکه‌های بیزیِ ترکیبی ارائه می‌دهد. توزیعِ پروبیت معمولاً به گادوم (1933) و بلیس (1934) نسبت داده می‌شود، اگرچه چندین بار در قرن نوزدهم نیز کشف شده بود. کارِ بلیس توسط فینی (1947) به‌طورِ قابلِ‌توجهی گسترش یافت. پروبیت به‌طورِ گسترده برای مدل‌سازیِ پدیده‌های انتخابِ گسسته به کار

رفته و می‌تواند برای مدیریتِ بیش از دو انتخاب تعمیم یابد (داگانزو، 1979). مدلِ اکسپیت (لوجیتِ معکوس) توسط برکسون (1944) معرفی شد؛ در ابتدا بسیار مورد تمسخر قرار گرفت، اما در نهایت از مدلِ پروبیت محبوب‌تر شد. بیشاپ (1995) توجیهِ ساده‌ای برای استفاده از آن ارائه می‌دهد.

کاربردهای اولیهٔ شبکه‌های بیزی در پزشکی شاملِ سامانهٔ MUNIN برای تشخیصِ اختلالاتِ عصبی-عضلانی (آندرسن و همکاران، 1989) و سامانهٔ PATHFINDER برای آسیب‌شناسی (هکرمن، 1991) بود. کاربردها در مهندسی شاملِ کار مؤسسهٔ تحقیقاتِ انرژی الکتریکی (EPRI) روی پایشِ ژنراتورهای برق (مورجاریا و همکاران، 1995)، کارِ ناسا روی نمایشِ اطلاعاتِ حساس به زمان در مرکزِ کنترلِ مأموریت در هیوستون (هوروتز و بَری، 1995)، و حوزهٔ کلی «توموگرافیِ شبکه» است که هدفِ آن استنباطِ ویژگی‌های محلیِ مشاهده‌نشدهٔ گره‌ها و پیوندها در اینترنت از طریقِ مشاهداتِ عملکردِ پیامِ سرتاسری است (کاسترو و همکاران، 2004). شاید پرکاربردترین سامانه‌های شبکهٔ بیزی، ماژول‌های تشخیص و تعمیر (مانند ویزارد چاپگر) در مایکروسافت ویندوز (بریز و هکرمن، 1996) و دستیارِ آفیس (Office Assistant) در مایکروسافت آفیس (هوروتز و همکاران، 1998) بوده‌اند.

یکی دیگر از حوزه‌های مهمِ کاربردی، زیست‌شناسی است: مدل‌های ریاضیِ مورد استفاده برای تحلیلِ توارثِ ژنتیکی در شجره‌نامه‌ها (به‌اصطلاح «تحلیلِ دودمان» یا Pedigree Analysis) در واقع شکلِ خاصی از شبکه‌های بیزی هستند. الگوریتم‌های استنتاجِ دقیق برای تحلیلِ دودمان، شبیه به الگوریتمِ حذفِ متغیر، در اوایل سالِ 1971 توسط الستون و استوارت توسعه یافتند؛ کانینگز و همکاران (1978) این رویکرد را به الگوریتم‌هایی شبیه به خوشه‌بندیِ درختی گسترش دادند. بسته‌های نرم‌افزاریِ تحلیلِ پیوستگیِ ژنتیکی LIPED (اوْت، 1974) و LINKAGE (لاتروپ و لالوئل، 1984) از پرکاربردترین بسته‌ها در ژنتیک بودند و برای موفقیت‌های اولیه در نقشه‌برداریِ مکانِ ژن‌ها در ژنومِ انسان نقشِ حیاتی داشتند.

جودیا پرل (1985)، در نخستین مقاله‌ای که از اصطلاح «شبکه‌های بیزی» استفاده کرد، به‌طورِ مختصر الگوریتمِ استنتاجی برای شبکه‌های عمومی را بر اساسِ ایدهٔ «شرطی‌سازیِ برش‌مجموعه» (Cutset Conditioning) که در فصل ۶ معرفی شد توصیف کرد. به‌طورِ مستقل، راس شاکتر (1986)، که در جامعهٔ نمودارهایِ تأثیر کار می‌کرد، الگوریتمِ کاملی بر اساسِ کاهشِ هدف‌گرایِ شبکه با استفاده از تبدیل‌های حفظ‌کنندهٔ پسین توسعه داد.

پرل (1986) الگوریتمِ خوشه‌بندی را برای استنتاجِ دقیق در شبکه‌های بیزیِ عمومی توسعه داد، با استفاده از تبدیل به پلی‌درختِ جهت‌داری از خوشه‌ها که در آن انتقالِ پیام برای دستیابی به سازگاری بر متغیرهای مشترکِ میان خوشه‌ها استفاده می‌شد. رویکردِ مشابهی، که توسط آماردانانِ دیوید اشپیگلهالتر و استفن لوریتزن توسعه یافت (لوریتزن و اشپیگلهالتر، 1988)، بر اساسِ تبدیل به شکلِ بدونِ جهتِ مدلِ گرافیکی به نام «شبکهٔ مارکوف» (Markov Network) است. این رویکرد در سامانهٔ HUGIN پیاده‌سازی شده، ابزارِ کارآمد و پرکاربردی برای استدلالِ نامطمئن (آندرسن و همکاران، 1989).

ایدهٔ پایهٔ حذفِ متغیر—که محاسبه‌های تکراری درونِ عبارتِ جمعِ حاصل‌ضربِ کلی را می‌توان با کش‌کردن اجتناب کرد—در الگوریتمِ «استنتاجِ احتمالاتیِ نمادین» (Symbolic Probabilistic Inference یا SPI) ظاهر شد (شاکتر و همکاران، 1990). الگوریتمِ حذفی که ما توصیف می‌کنیم نزدیک‌ترین به الگوریتمِ توسعه‌یافته توسط ژانگ و پول (1994) است. معیارهای هرسِ متغیرهای نامربوط توسط گایگر و همکاران (1990b) و توسط لوریتزن و همکاران (1990) توسعه

یافت؛ معیاری که ما می‌دهیم حالتِ خاصِ ساده‌ای از این‌هاست. دچتر (1999) نشان می‌دهد ایدهٔ حذفِ متغیر اساساً یکسان با «برنامه‌نویسیِ پویایِ غیرمتوالی» (Nonserial Dynamic Programming) (برتله و بریوشی، 1972) است. این الگوریتم‌های شبکهٔ بیزی را به روش‌های مرتبط برای حلِ CSPها متصل می‌کند و اندازهٔ مستقیمی از پیچیدگیِ استنتاجِ دقیق بر حسبِ پهنایِ درختیِ شبکه می‌دهد. جلوگیری از رشدِ نمایی در اندازهٔ عامل‌ها در حذفِ متغیر را می‌توان با حذفِ متغیرها از عامل‌های بزرگ انجام داد (دچتر و ریش، 2003)؛ همچنین می‌توان خطایِ بدین‌وسیله واردشده را کران‌دار کرد (وکسلر و میک، 2009). از طرفِ دیگر، عامل‌ها را می‌توان با بازنماییِ آن‌ها با «نمودارهای تصمیمِ جبری» (Algebraic Decision Diagrams) به‌جای جدول‌ها فشرده کرد (گوگیت و دومینگوس، 2011).

روش‌های دقیقِ مبتنی بر شمارشِ بازگشتی (شکل ۱۳.۱۱ را ببینید) همراه با کش‌کردن شاملِ الگوریتمِ «شرطی‌سازیِ بازگشتی» (Recursive Conditioning) (دارویچ، 2001)، الگوریتمِ «حذفِ مقدار» (Value Elimination) (بیکوس و همکاران، 2003) و جستجوی AND–OR (دچتر و ماتسکو، 2007) می‌شوند. روشِ «شمارشِ مدلِ وزندار» (سانگ و همکاران، 2005؛ شاویرا و دارویچ، 2008) معمولاً بر حل‌کنندهٔ SAT به سبکِ DPLL مبتنی است (شکل ۷.۱۷ صفحهٔ ۲۳۴ را ببینید). به‌این‌ترتیب، آن نیز شمارشِ بازگشتیِ تخصیص‌های متغیر را با کش‌کردن انجام می‌دهد، پس رویکرد در واقع کاملاً مشابه است. هر سه این الگوریتم‌ها می‌توانند دامنهٔ کاملی از معاوضه‌های فضا/زمان را پیاده‌سازی کنند.

چون تخصیص‌های متغیر را در نظر می‌گیرند، الگوریتم‌ها می‌توانند به‌آسانی از قطعیت و استقلالِ وابسته به بافت در مدل بهره بگیرند. آن‌ها همچنین می‌توانند برای استفاده از الگوریتمِ کارآمدِ خطی‌زمان اصلاح شوند، هرگاه تخصیصِ جزئی شبکهٔ باقی‌مانده را پلی‌درخت کند. (این نسخه‌ای از روشِ شرطی‌سازیِ برش‌مجموعه است که برای CSPها در فصل ۶ توصیف شد.) برای استنتاجِ دقیق در مدل‌های بزرگ، که نیازهای فضاییِ خوشه‌بندی و حذفِ متغیر عظیم می‌شود، این الگوریتم‌های بازگشتی اغلب عملی‌ترین رویکردند.

تکلیف‌های مهمِ استنتاجیِ دیگری در شبکه‌های بیز هست، جدا از محاسبهٔ احتمال‌های حاشیه‌ای. «محتمل‌ترین تبیین» (Most Probable Explanation یا MPE) محتمل‌ترین تخصیص به متغیرهای غیرشاهد با توجه به شواهد است. (MPE حالتِ خاصی از استنتاجِ «بیشینهٔ پسین» (Maximum A Posteriori یا MAP) است، که محتمل‌ترین تخصیص به زیرمجموعه‌ای از متغیرهای غیرشاهد را با توجه به شواهد می‌پرسد.) برای چنین مسائلی، الگوریتم‌های متفاوتِ بسیاری توسعه یافته، برخی مربوط به الگوریتم‌های کوتاه‌ترین‌مسیر یا جستجوی AND–OR؛ برای خلاصه، مارینسکو و دچتر (2009) را ببینید. نخستین نتیجه دربارهٔ پیچیدگیِ استنتاج در شبکه‌های بیز از کوپر (1990) است، که نشان داد مسئلهٔ عمومیِ محاسبهٔ حاشیه‌ها در شبکه‌های بیزی NP-سخت است؛ چنان‌که در فصل ذکر شد، این را می‌توان از طریقِ تقلیل از شمارشِ تخصیص‌های ارضاکننده به #P-سختی تقویت کرد (روت، 1996). این همچنین دلالت بر NP-سختیِ استنتاجِ تقریبی دارد (داگوم و لوبی، 1993)؛ با این حال، برای حالتی که احتمال‌ها را بتوان از 0 و 1 دور نگه داشت، شکلی از وزن‌گذاریِ درستنمایی در زمانِ (تصادفیِ) چندجمله‌ای همگرا می‌شود (داگوم و لوبی، 1997). شیمونی (1994) نشان داد یافتنِ محتمل‌ترین تبیین NP-کامل است—ناشدنی، اما تا حدی آسان‌تر از محاسبهٔ حاشیه‌ها—در حالی که پارک و دارویچ (2004) تحلیلِ پیچیدگیِ کاملی از محاسبهٔ MAP فراهم می‌کنند، نشان می‌دهند در ردهٔ مسائلِ NPPP-کامل قرار می‌گیرد—یعنی تا حدی سخت‌تر از محاسبهٔ حاشیه‌ها.

توسعهٔ الگوریتم‌های تقریبِ سریع برای استنتاجِ شبکهٔ بیزی حوزه‌ای بسیار فعال است، با مشارکت‌هایی از آمار، علومِ رایانه و فیزیک. روشِ نمونه‌برداریِ ردّی فنِ عمومی‌ای است که دست‌کم به «سوزنِ بوفون» (1777) بازمی‌گردد؛ نخستین بار توسط ماکس هنریون (1988) روی شبکه‌های بیزی اعمال شد، که آن را «نمونه‌برداریِ منطقی» (Logic Sampling) نامید. نمونه‌برداریِ اهمیت در اصل برای کاربردهایی در فیزیک اختراع شد (کان، 1950b ،1950a) و توسط فونگ و چانگ (1989) (که الگوریتم را «وزن‌گذاریِ شاهد» (Evidence Weighting) نامیدند) و توسط شاکتر و پئوت (1989) روی استنتاجِ شبکهٔ بیز اعمال شد.

در آمار، نمونه‌برداریِ تطبیقی برای انواعِ الگوریتم‌های مونت‌کارلو برای سرعت‌بخشیدن به همگرایی اعمال شده. ایدهٔ پایه تطبیقِ توزیعی است که نمونه‌ها از آن تولید می‌شوند، بر اساسِ پیامدِ نمونه‌های قبلی. گیلکس و وایلد (1992) نمونه‌برداریِ ردّیِ تطبیقی را توسعه دادند، در حالی که نمونه‌برداریِ اهمیتِ تطبیقی ظاهراً به‌طورِ مستقل در فیزیک (لپاژ، 1978)، مهندسیِ عمران (کارامچاندانی و همکاران، 1989)، آمار (اوه و برگر، 1992) و گرافیکِ رایانه‌ای (ویچ و گیباس، 1995) سرچشمه گرفته. چنگ و دروزدزل (2000) نسخهٔ تطبیقیِ نمونه‌برداریِ اهمیتِ اعمال‌شده بر استنتاجِ شبکهٔ بیز را توصیف می‌کنند. اخیرتر، له و همکاران (2017) استفاده از سامانه‌های یادگیریِ عمیق برای تولیدِ توزیع‌های پیشنهادی را نشان داده‌اند که نمونه‌برداریِ اهمیت را چند مرتبهٔ بزرگی سریع‌تر می‌کند. الگوریتم‌های مونت‌کارلویِ زنجیرهٔ مارکوف (MCMC) با الگوریتمِ متروپلیس آغاز شدند، که از متروپلیس و همکاران (1953) است و همچنین منبعِ الگوریتمِ بازپختِ شبیه‌سازی‌شدهٔ توصیف‌شده در فصل ۴ بود. هستینگز (1970) گامِ پذیرش/رد را معرفی کرد که بخشِ جدایی‌ناپذیرِ چیزی است که امروز الگوریتمِ متروپلیس–هستینگز می‌نامیم. نمونه‌بردارِ گیبس توسط گمن و گمن (1984) برای استنتاج در شبکه‌های مارکوفِ بدونِ جهت ابداع شد. کاربردِ نمونه‌برداریِ گیبس در شبکه‌های بیزی از پرل (1987) است. مقاله‌های گردآوری‌شده توسط گیلکس و همکاران (1996) هم نظریه و هم کاربردهای MCMC را پوشش می‌دهند.

از اواسطِ دههٔ ۱۹۹۰، MCMC اسبِ بارکشِ آمارِ بیزی و محاسبهٔ آماری در بسیاری از رشته‌های دیگر از جمله فیزیک و زیست‌شناسی شده. «راهنمای مونت‌کارلویِ زنجیرهٔ مارکوف» (بروکس و همکاران، 2011) بسیاری از جنبه‌های این ادبیات را پوشش می‌دهد. بستهٔ BUGS (گیلکس و همکاران، 1994) سامانهٔ اولیه و تأثیرگذاری برای مدل‌سازی و استنتاجِ شبکهٔ بیز با استفاده از نمونه‌برداریِ گیبس بود. STAN (به نامِ استانیسلاو اولام، یکی از پدیدآورندگانِ روش‌های مونت‌کارلو در فیزیک) سامانهٔ جدیدتری است که از استنتاجِ مونت‌کارلویِ همیلتونی استفاده می‌کند (کارپنتر و همکاران، 2017).

دو خانوادهٔ بسیار مهمِ روش‌های تقریبی هست که در فصل پوشش ندادیم. نخستین، خانوادهٔ «روش‌های تقریبِ تغییراتی» (Variational Approximation Methods) است، که می‌تواند برای ساده‌سازیِ محاسباتِ پیچیدهٔ همه‌گونه استفاده شود. ایدهٔ پایه پیشنهادِ نسخهٔ کاهش‌یافته‌ای از مسئلهٔ اصلی است که کار با آن ساده اما تا آن‌جا که ممکن است شبیهِ مسئلهٔ اصلی است. مسئلهٔ کاهش‌یافته با پارامترهای تغییراتیِ λ توصیف می‌شود که برای کمینه‌کردنِ تابعِ فاصلهٔ D میانِ مسئلهٔ اصلی و کاهش‌یافته تنظیم می‌شوند، اغلب با حلِ دستگاهِ معادلاتِ $\partial D/\partial \lambda = 0$. در بسیاری از موارد، کران‌های بالا و پایینِ اکید قابلِ‌دستیابی‌اند. روش‌های تغییراتی مدت‌ها در آمار استفاده شده‌اند (روستاگی، 1976). در فیزیکِ آماری، «روشِ میدانِ میانگین» (Mean-Field Method) تقریبِ تغییراتیِ خاصی است که در آن متغیرهای فردیِ سازندهٔ مدل کاملاً مستقل فرض می‌شوند. این ایده برای حلِ شبکه‌های مارکوفِ بدونِ جهتِ بزرگ

اعمال شد (پترسون و اندرسون، 1987؛ پاریسی، 1988). ساول و همکاران (1996) شالوده‌های ریاضیِ اعمالِ روش‌های تغییراتی بر شبکه‌های بیزی را توسعه دادند و تقریب‌های کرانِ پایینِ دقیقی برای شبکه‌های سیگموئیدی با استفاده از روش‌های میدانِ میانگین به دست آوردند. یاکولا و جوردن (1996) روش‌شناسی را برای به‌دست‌آوردنِ هر دو کرانِ پایین و بالا بسط دادند. از این مقاله‌های اولیه، روش‌های تغییراتی به بسیاری از خانواده‌های خاصِ مدل‌ها اعمال شده‌اند. مقالهٔ قابلِ‌توجهِ واین‌رایت و جوردن (2008) تحلیلِ نظریِ یکپارچه‌ای از ادبیاتِ روش‌های تغییراتی فراهم می‌کند.

دومین خانوادهٔ مهمِ الگوریتم‌های تقریبی بر الگوریتمِ انتقالِ پیامِ پلی‌درختِ پرل (1982a) مبتنی است. این الگوریتم را می‌توان برای شبکه‌های «حلقه‌دارِ» عمومی اعمال کرد، چنان‌که پرل (1988) پیشنهاد کرد. نتایج ممکن است نادرست باشند، یا الگوریتم ممکن است در خاتمه‌یافتن شکست بخورد، اما در بسیاری از موارد، مقادیرِ به‌دست‌آمده نزدیکِ مقادیرِ واقعی‌اند. توجهِ اندکی به این «انتشارِ باورِ حلقه‌دارِ» (Loopy Belief Propagation) موسوم معطوف شد تا اینکه مک‌الیس و همکاران (1998) مشاهده کردند این دقیقاً محاسبه‌ای است که «الگوریتمِ رمزگشاییِ توربو» (Turbo Decoding) (برو و همکاران، 1993) انجام می‌دهد، که پیشرفتِ بزرگی در طراحیِ کدهای تصحیحِ خطایِ کارآمد فراهم کرد. دلالتِ این مشاهده‌ها این است که اگرِ BP حلقه‌دار هم سریع و هم دقیق روی شبکه‌های بسیار بزرگ و بسیار پرپیوندِ استفاده‌شده برای رمزگشایی است، پس ممکن است به‌طورِ کلی‌تر مفید باشد. پشتیبانیِ نظری برای این یافته‌ها، شاملِ اثبات‌های همگرایی برای برخی حالت‌های خاص، توسط وایس (2000b)، وایس و فریمن (2001) و یدیدیا و همکاران (2005) فراهم شد، با تکیه بر ارتباط‌هایی با ایده‌های فیزیکِ آماری.

نظریه‌های استنتاجِ علّیِ فراتر از کارآزمایی‌های تصادفی‌شده با کنترل توسط روبین (1974) و رابینز (1986) پیشنهاد شد، اما این ایده‌ها هم مبهم و هم بحث‌برانگیز ماندند تا اینکه جودیا پرل نظریهٔ کاملاً بیان‌شده‌ای از علّیت را بر اساسِ شبکه‌های علّی توسعه و ارائه کرد (پرل، 2000). پیترز و همکاران (2017) نظریه را بیشتر توسعه می‌دهند، با تأکید بر یادگیری. اثری جدیدتر، «کتابِ چرا» (The Book of Why؛ پرل و مک‌کنزی، 2018)، مقدمه‌ای کم‌ریاضی‌تر اما خواناتر و گسترده‌تر فراهم می‌کند.

استدلالِ نامطمئن در هوش مصنوعی همیشه بر نظریهٔ احتمال مبتنی نبوده. چنان‌که در فصل ۱۲ ذکر شد، سامانه‌های احتمالاتیِ اولیه در اوایلِ دههٔ ۱۹۷۰ از رواج افتادند و خلأی جزئی باقی گذاشتند که با روش‌های جایگزین پر شود. این‌ها شاملِ سامانه‌های خبرهٔ قاعده‌محور، نظریهٔ دمپستر–شیفر و (تا حدی) منطقِ فازی بودند.[9] رویکردهای قاعده‌محور به عدم قطعیت امیدوار بودند بر موفقیتِ سامانه‌های قاعده‌محورِ منطقی بنا شوند، اما نوعی «عاملِ جبرانِ» (Fudge Factor)—مؤدبانه‌تر «ضریبِ قطعیت» (Certainty Factor) نامیده می‌شود—به هر قاعده بیفزایند تا عدم قطعیت را در خود جای دهند. نخستین چنین سامانه‌ای MYCIN بود (شورتلیف، 1976)، سامانهٔ خبرهٔ پزشکی برای عفونت‌های باکتریایی. مجموعهٔ «سامانه‌های خبرهٔ قاعده‌محور» (بوکانان و شورتلیف، 1984) مرورِ کاملی از MYCIN و فرزندانش فراهم می‌کند (همچنین استفیک، 1995 را ببینید).

دیوید هکرمن (1986) نشان داد نسخهٔ کمی اصلاح‌شدهٔ محاسباتِ ضریبِ قطعیت در برخی موارد نتایجِ احتمالاتیِ درستی می‌دهد، اما در مواردِ دیگر به بیش‌شماریِ جدیِ شواهد می‌انجامد. با بزرگ‌ترشدنِ مجموعه‌های قاعده، تعامل‌های نامطلوب میان قاعده‌ها رایج‌تر می‌شد، و دست‌اندرکاران دریافتند ضریب‌های قطعیتِ بسیاری از قاعده‌های

دیگر باید وقتی قاعده‌های تازه افزوده می‌شوند «تنظیم» شوند. ویژگی‌های ریاضیِ پایه‌ای که زنجیره‌های استدلال در منطق را ممکن می‌کنند به‌سادگی برای احتمال برقرار نیستند.

نظریهٔ دمپستر–شیفر از مقاله‌ای از آرتور دمپستر (1968) سرچشمه می‌گیرد که تعمیمی از احتمال به مقادیرِ بازه‌ای و قاعدهٔ ترکیبی برای استفاده از آن‌ها پیشنهاد می‌کرد. چنین رویکردی ممکن است دشواریِ مشخص‌کردنِ دقیقِ احتمال‌ها را تسکین دهد. کارِ بعدیِ گلن شیفر (1976) به این انجامید که نظریهٔ دمپستر–شیفر رویکردی رقیب برای احتمال دیده شود. پرل (1988) و روسپینی و همکاران (1992) رابطهٔ میانِ نظریهٔ دمپستر–شیفر و نظریهٔ احتمالِ استاندارد را تحلیل می‌کنند. در بسیاری از موارد، نظریهٔ احتمال الزام نمی‌کند احتمال‌ها دقیقاً مشخص شوند: می‌توانیم عدم قطعیت دربارهٔ مقادیرِ احتمال را به‌صورتِ توزیع‌های احتمالِ (مرتبهٔ دوم) بیان کنیم، چنان‌که در فصل ۲۰ توضیح داده شده.

مجموعه‌های فازی توسط لطفی زاده (1965) در پاسخ به دشواریِ درک‌شدهٔ فراهم‌کردنِ ورودی‌های دقیق برای سامانه‌های هوشمند توسعه یافت. مجموعهٔ فازی مجموعه‌ای است که در آن عضویت امری درجه‌ای است. منطقِ فازی روشی برای استدلال با عبارت‌های منطقیِ توصیف‌کنندهٔ عضویت در مجموعه‌های فازی است. کنترلِ فازی روش‌شناسی‌ای برای ساختِ سامانه‌های کنترلی است که در آن نگاشت میان پارامترهای ورودی و خروجیِ حقیقی‌مقدار با قاعده‌های فازی بازنمایی می‌شود. کنترلِ فازی در محصولاتِ تجاری مانند گیربکس‌های اتوماتیک، دوربین‌های ویدیویی و ریش‌تراش‌های برقی بسیار موفق بوده. متنِ زیمرمان (2001) مقدمهٔ کاملی بر نظریهٔ مجموعهٔ فازی فراهم می‌کند؛ مقاله‌های کاربردهای فازی در زیمرمان (1999) گردآوری شده‌اند. منطقِ فازی اغلب به‌اشتباه رقیبِ مستقیمِ نظریهٔ احتمال درک شده، در حالی که در واقع به مجموعهٔ متفاوتی از مسائل می‌پردازد: به‌جای درنظرگرفتنِ عدم قطعیت دربارهٔ صدقِ گزاره‌های خوش‌تعریف، منطقِ فازی با ابهام در نگاشت از عبارت‌های نظریهٔ نمادین به جهانِ واقعی سروکار دارد. ابهام مسئلهٔ واقعی در هر کاربردِ منطق، احتمال یا حتی مدل‌های ریاضیِ استاندارد بر واقعیت است. حتی متغیری به‌تمیزیِ جرمِ زمین، با بازرسی، معلوم می‌شود با زمان تغییر می‌کند، چنان‌که شهاب‌سنگ‌ها و مولکول‌ها می‌آیند و می‌روند. همچنین نادقیق است—آیا جو را شامل می‌شود؟ اگر بله، تا چه ارتفاعی؟ در برخی موارد، بسطِ بیشترِ مدل می‌تواند ابهام را کاهش دهد، اما منطقِ فازی ابهام را امری داده‌شده می‌گیرد و نظریه‌ای پیرامونش توسعه می‌دهد.

«نظریهٔ امکان» (Possibility Theory؛ زاده، 1978) برای مدیریتِ عدم قطعیت در سامانه‌های فازی معرفی شد و اشتراکاتِ بسیاری با احتمال دارد (دوبوا و پراد، 1994).

بسیاری از پژوهشگرانِ هوش مصنوعی در دههٔ ۱۹۷۰ احتمال را رد کردند، زیرا محاسباتِ عددی که تصور می‌شد نظریهٔ احتمال به آن‌ها نیاز دارد برای درون‌نگری آشکار نبود و سطحِ غیرواقع‌گرایانه‌ای از دقت در دانشِ نامطمئنِ ما را پیش‌فرض می‌گرفت. توسعهٔ «شبکه‌های احتمالاتیِ کیفی» (Qualitative Probabilistic Networks؛ ولمن، 1990a) انتزاعِ صرفاً کیفیِ شبکه‌های بیزی را فراهم کرد، با استفاده از مفهومِ تأثیرهای مثبت و منفی میان متغیرها. ولمن نشان می‌دهد در بسیاری از موارد چنین اطلاعاتی برای تصمیم‌گیریِ بهینه بدونِ نیاز به مشخص‌سازیِ دقیقِ مقادیرِ احتمال کافی است. گلدشمیت و پرل (1996) رویکردِ مشابهی دارند. کارِ دارویچ و گینزبرگ (1992) ویژگی‌های پایهٔ شرطی‌کردن و ترکیبِ شواهد را از نظریهٔ احتمال بیرون می‌کشد و نشان می‌دهد می‌توان آن‌ها را در استدلالِ منطقی و پیش‌فرض نیز اعمال کرد.

چندین متنِ عالی (جنسن، 2007؛ دارویچ، 2009؛ کولر و فریدمن، 2009؛ کورب و نیکلسون، 2010؛ دچتر، 2019) برخوردهای کاملی از موضوعاتِ پوشش‌داده‌شده در این فصل فراهم می‌کنند. پژوهشِ تازه دربارهٔ استدلالِ احتمالاتی هم در ژورنال‌های اصلیِ هوش مصنوعی، مانند «هوش مصنوعی» (Artificial Intelligence) و «ژورنالِ پژوهشِ هوش مصنوعی» (Journal of AI Research)، و هم در ژورنال‌های تخصصی‌تر، مانند «ژورنالِ بین‌المللیِ استدلالِ تقریبی» (International Journal of Approximate Reasoning)، ظاهر می‌شود. مقاله‌های بسیاری دربارهٔ مدل‌های گرافیکی، که شبکه‌های بیزی را شامل می‌شوند، در ژورنال‌های آماری ظاهر می‌شوند. مجموعه‌مقالاتِ کنفرانس‌های «عدم قطعیت در هوش مصنوعی» (UAI)، «سامانه‌های پردازشِ اطلاعاتِ عصبی» (NeurIPS) و «هوش مصنوعی و آمار» (AISTATS) منابعِ خوبی برای پژوهشِ جاری‌اند.

پانوشت‌ها:

[1] شبکه‌های بیزی، که اغلب به‌اختصار «شبکهٔ بیز» نامیده می‌شوند، در دهه‌های ۱۹۸۰ و ۱۹۹۰ «شبکهٔ باور» نامیده می‌شدند. شبکهٔ علّی شبکهٔ بیزی با قیدهای اضافی روی معنای پیکان‌هاست (بخش ۱۳.۵ را ببینید). اصطلاح «مدلِ گرافیکی» به ردهٔ گسترده‌تری اشاره دارد که شبکه‌های بیزی را شامل می‌شود.

[2] از این نتیجه می‌شود استنتاج در شبکه‌های خطی–گاوسی در بدترین حالت فقط $O(n^3)$ زمان می‌برد، صرف‌نظر از توپولوژیِ شبکه. در بخش ۱۳.۳ می‌بینیم استنتاج برای شبکه‌های متغیرِ گسسته NP-سخت است.

[3] شبکهٔ نشان‌داده‌شده در شکل ۱۳.۹ در واقع استفاده نمی‌شود، اما ساختارش با متخصصانِ بیمه بازبینی شده. در عمل، اطلاعاتِ خواسته‌شده در فرم‌های درخواست با شرکت و حوزهٔ قضایی متفاوت است—برای نمونه، برخی جنسیت می‌پرسند—و مدل قطعاً می‌تواند جزئی‌تر و ظریف‌تر ساخته شود.

[4] برخی شرکت‌های بیمه همچنین سابقهٔ اعتباریِ متقاضی را برای کمک به ارزیابیِ ریسک به دست می‌آورند؛ این اطلاعاتِ بسیار بیشتری دربارهٔ ردهٔ اقتصادی-اجتماعی فراهم می‌کند. هرگاه از این نوع متغیرهای پنهان استفاده می‌کنیم، باید مراقب باشیم ناخواسته به جانشینِ متغیرهایی مانند نژاد تبدیل نشوند که ممکن است در تصمیم‌های بیمه استفاده نشوند. فنونِ اجتناب از این نوع سوگیری‌ها در فصل ۱۹ توصیف شده‌اند.

[5] تکلیفِ بسیار مطالعه‌شدهٔ دیگر یافتنِ محتمل‌ترین تبیین برای برخی شواهدِ مشاهده‌شده است. این و دیگر تکلیف‌ها در یادداشت‌های پایانِ فصل بحث شده‌اند.

[6] اگر آسان بود، آنگاه می‌توانستیم احتمالِ موردنظر را با تعدادِ چندجمله‌ای از نمونه‌ها با دقتِ دلخواه تقریب بزنیم. می‌توان نشان داد چنین طرحِ تقریبِ چندجمله‌ای‌زمانی نمی‌تواند وجود داشته باشد.

[7] ای. جی. گود آماردانِ ارشدِ تیمِ شکستنِ رمزِ تورینگ در جنگِ جهانیِ دوم بود. در «۲۰۰۱: ادیسهٔ فضایی» (کلارک، 1968)، گود و مینسکی با انجامِ پیشرفتِ منجر به توسعهٔ رایانهٔ HAL 9000 اعتبار یافته‌اند.

[8] عنوانِ نسخهٔ اصلیِ مقاله «پرل برای خوک» (Pearl for swine) بود.

[9] رویکردِ چهارم، استدلالِ پیش‌فرض (Default Reasoning)، نتیجه‌ها را نه «باورشده تا درجهٔ معینی»، بلکه «باورشده تا وقتی دلیلِ بهتری برای باورِ چیزِ دیگر یافت شود» تلقی می‌کند. در فصل ۱۰ پوشش داده شده است.

فصل ۱۴: استدلال احتمالاتی در طول زمان (Probabilistic Reasoning over Time)

در این فصل می‌کوشیم حال را تفسیر کنیم، گذشته را بفهمیم و شاید آینده را پیش‌بینی کنیم، حتی وقتی چیزِ بسیار کمی روشن و واضح است.

عامل‌ها در محیط‌های تا حدی قابل مشاهده باید بتوانند حالتِ کنونی را، تا آن‌جا که حسگرهایشان اجازه می‌دهد، پیگیری کنند. در بخش ۴.۴ روشی برای این کار نشان دادیم: عامل «حالت باور» (Belief State) را نگه می‌دارد که بازنمایی می‌کند کدام حالت‌های جهان در حالِ حاضر ممکن‌اند.

عامل با حالت باور و «مدل انتقال» (Transition Model) می‌تواند پیش‌بینی کند جهان ممکن است در گامِ زمانیِ بعد چگونه تکامل یابد. با ادراک‌های مشاهده‌شده و «مدل حسگر» (Sensor Model)، عامل می‌تواند حالت باور را به‌روزرسانی کند. این ایده‌ای فراگیر است: در فصل ۴ حالت‌های باور با مجموعه‌هایی از حالت‌های صریحاً شمرده‌شده بازنمایی می‌شدند، در حالی که در فصل‌های ۷ و ۱۱ با فرمول‌های منطقی بازنمایی می‌شدند. آن رویکردها حالت‌های باور را بر حسبِ اینکه کدام حالت‌های جهان ممکن‌اند تعریف می‌کردند، اما نمی‌توانستند چیزی دربارهٔ اینکه کدام حالت‌ها محتمل یا نامحتمل‌اند بگویند. در این فصل، از نظریهٔ احتمال برای کمّی‌کردنِ درجهٔ باور به عناصرِ حالت باور استفاده می‌کنیم.

چنان‌که در بخش ۱۴.۱ نشان می‌دهیم، خودِ زمان به همان شیوهٔ فصل ۷ مدیریت می‌شود: جهانِ متغیر با متغیری برای هر جنبه از حالتِ جهان در هر نقطه از زمان مدل می‌شود. مدل‌های انتقال و حسگر ممکن است نامطمئن باشند: مدلِ انتقال توزیعِ احتمالِ متغیرها را در زمانِ t، با توجه به حالتِ جهان در زمان‌های گذشته، توصیف می‌کند، در حالی که مدلِ حسگر احتمالِ هر ادراک را در زمانِ t، با توجه به حالتِ کنونیِ جهان، توصیف می‌کند. بخش ۱۴.۲ تکلیف‌های استنتاجیِ پایه را تعریف و ساختارِ عمومیِ الگوریتم‌های استنتاج برای مدل‌های زمانی را توصیف می‌کند. سپس سه نوع خاصِ مدل را توصیف می‌کنیم: «مدل‌های مارکوفِ پنهان» (Hidden Markov Models)، «فیلترهای کالمن» (Kalman Filters) و «شبکه‌های بیزیِ پویا» (Dynamic Bayesian Networks)؛ که مدل‌های مارکوفِ پنهان و فیلترهای کالمن را به‌عنوانِ حالت‌های خاص شامل می‌شوند).

۱۴.۱ زمان و عدم قطعیت

فنونِ خود را برای استدلالِ احتمالاتی در بافتِ جهان‌های ایستا توسعه داده‌ایم، که در آن هر متغیرِ تصادفی یک مقدارِ ثابتِ تکی دارد. برای نمونه، هنگام تعمیرِ خودرو فرض می‌کنیم هرچه خراب است در طولِ فرایندِ تشخیص خراب می‌ماند؛ کارِ ما استنتاجِ حالتِ خودرو از شواهدِ مشاهده‌شده است، که آن نیز ثابت می‌ماند.

حالا مسئله‌ای کمی متفاوت را در نظر بگیرید: درمانِ بیمارِ دیابتی. همانندِ تعمیرِ خودرو، شواهدی مانند دوزهای اخیرِ انسولین، دریافتِ غذا، اندازه‌گیری‌های قند خون و دیگر نشانه‌های فیزیکی داریم. تکلیف، ارزیابیِ حالتِ کنونیِ بیمار است، شاملِ سطحِ واقعیِ قند خون و سطحِ انسولین. با این اطلاعات، می‌توانیم دربارهٔ دریافتِ غذای بیمار و دوزِ انسولین تصمیم بگیریم. برخلافِ موردِ تعمیرِ خودرو، در این‌جا جنبه‌های پویای مسئله ضروری‌اند. سطحِ قند خون و اندازه‌گیری‌های آن می‌توانند به‌سرعت در طولِ زمان تغییر کنند، بسته به دریافتِ اخیرِ غذا و دوزهای انسولین، فعالیتِ متابولیک، ساعتِ روز و مانندِ آن. برای ارزیابیِ حالتِ کنونی از تاریخچهٔ شواهد و پیش‌بینیِ پیامدهای کنش‌های درمانی، باید این تغییرها را مدل کنیم.

همین ملاحظات در بافت‌های بسیاری دیگر پیش می‌آید، مانند ردیابیِ مکانِ ربات، ردیابیِ فعالیتِ اقتصادیِ یک ملت، و فهمِ توالیِ گفتاری یا نوشتاریِ واژه‌ها. چگونه می‌توان وضعیت‌های پویایی مانند این‌ها را مدل کرد؟

۱۴.۱.۱ حالت‌ها و مشاهده‌ها

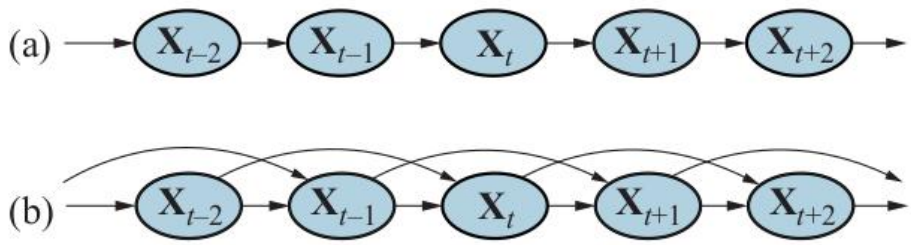
این فصل «مدل‌های زمانِ گسسته» (Discrete-Time Models) را بحث می‌کند، که در آن جهان به‌صورتِ مجموعه‌ای از «عکس‌های لحظه‌ای» یا «برش‌های زمانی» (Time Slices) دیده می‌شود.[1] فقط برش‌های زمانی را 0، 1، 2 و... شماره‌گذاری می‌کنیم، به‌جای تخصیصِ زمان‌های خاص به آن‌ها. معمولاً فرض می‌شود بازهٔ زمانیِ Δ میان برش‌ها برای هر بازه یکسان است. برای هر کاربردِ خاص، مقدارِ خاصی از Δ باید انتخاب شود. گاهی این توسط حسگر دیکته می‌شود؛ برای نمونه، دوربینِ ویدیویی ممکن است تصاویر را در بازه‌های 1/30 ثانیه تأمین کند. در مواردِ دیگر، بازه توسط نرخ‌های تغییرِ معمولِ متغیرهای مربوط دیکته می‌شود؛ برای نمونه، در موردِ پایشِ گلوکزِ خون، چیزها می‌توانند در طولِ ده دقیقه به‌طورِ قابلِ‌توجهی تغییر کنند، پس بازهٔ یک‌دقیقه‌ای ممکن است مناسب باشد. از سوی دیگر، در مدل‌کردنِ رانشِ قاره‌ها در مقیاسِ زمین‌شناختی، بازه‌ای یک‌میلیون‌ساله ممکن است خوب باشد.

هر برشِ زمانی در مدلِ احتمالِ زمانِ گسسته شاملِ مجموعه‌ای از متغیرهای تصادفی است، برخی قابلِ مشاهده و برخی نه. برای سادگی، فرض می‌کنیم همان زیرمجموعهٔ متغیرها در هر برشِ زمانی قابلِ مشاهده است (هرچند در هیچ‌کدام از آنچه در ادامه می‌آید این الزاماً ضروری نیست). از $\mathbf{X}_t$ برای نشان‌دادنِ مجموعهٔ متغیرهای حالت در زمانِ t استفاده می‌کنیم، که فرض می‌شود قابلِ مشاهده نیستند، و از $\mathbf{E}_t$ برای نشان‌دادنِ مجموعهٔ متغیرهای شاهدِ قابلِ مشاهده. مشاهده در زمانِ t برابر $\mathbf{E}_t = \mathbf{e}_t$ است، برای مجموعه‌ای از مقادیرِ $\mathbf{e}_t$.

مثالِ زیر را در نظر بگیرید: شما نگهبانِ امنیتیِ مستقر در تأسیساتِ زیرزمینیِ محرمانه‌ای هستید. می‌خواهید بدانید آیا امروز باران می‌بارد، اما تنها دسترسی‌تان به دنیای بیرون هر صبح رخ می‌دهد، وقتی می‌بینید مدیر با چتر وارد می‌شود یا بدونِ چتر. برای هر روزِ t، مجموعهٔ $\mathbf{E}_t$ بنابراین شاملِ متغیرِ شاهدِ تکیِ $\mathit{Umbrella}_t$ یا به‌اختصار U_t (آیا چتر ظاهر می‌شود) است، و مجموعهٔ $\mathbf{X}_t$ شاملِ متغیرِ حالتِ تکیِ Raint یا به‌اختصار R_t (آیا باران می‌بارد). مسائلِ دیگر می‌توانند مجموعه‌های بزرگ‌تری از متغیرها را شامل شوند. در مثالِ دیابت، متغیرهای شاهد ممکن است $\mathit{MeasuredBloodSugar}_t$ و $\mathit{PulseRate}_t$ باشند، در حالی که متغیرهای حالت ممکن است شاملِ $\mathit{BloodSugar}_t$ و

StomachContentst باشند. (توجه کنید BloodSugart و MeasuredBloodSugart متغیرِ یکسانی نیستند؛ این‌گونه با اندازه‌گیری‌های نویزیِ کمیت‌های واقعی سروکار داریم.) فرض می‌کنیم توالیِ حالت از t=0 شروع می‌شود و شواهد از t=1 شروع به رسیدن می‌کنند.

از این رو، جهانِ چترِ ما با متغیرهای حالتِ R0, R1, R2,... و متغیرهای شاهدِ U1, U2,... بازنمایی می‌شود. از نمادِ a:b برای نشان‌دادنِ توالیِ اعدادِ صحیح از a تا b به‌صورتِ شامل، و از نمادِ Xa:b برای نشان‌دادنِ مجموعهٔ متغیرها از Xa تا Xb به‌صورتِ شامل استفاده می‌کنیم. برای نمونه، U1:3 با U1, U2, U3 متناظر است. (توجه کنید این با نمادِ استفاده‌شده در زبان‌های برنامه‌نویسی مانند پایتون و گو متفاوت است، که در آن‌ها U[1:3] شاملِ U[3] نمی‌شود.) عدم قطعیت بر روی زمانِ پیوسته را می‌توان با «معادله‌های دیفرانسیلِ تصادفی» (Stochastic Differential Equations یا SDE) مدل کرد. مدل‌های مطالعه‌شده در این فصل را می‌توان تقریب‌های زمانِ گسستهٔ SDEها دانست.



شکل ۱۴.۱ – (الف) ساختارِ شبکهٔ بیزیِ متناظر با فرایندِ مارکوفِ مرتبهٔ اول با حالتی که توسط متغیرهای Xt تعریف می‌شود. (ب) فرایندِ مارکوفِ مرتبهٔ دوم.

۱۴.۱.۲ مدل‌های انتقال و حسگر

با تصمیم‌گیری دربارهٔ مجموعهٔ متغیرهای حالت و شاهد برای مسئله‌ای معین، گامِ بعد مشخص‌کردنِ چگونگیِ تکاملِ جهان (مدلِ انتقال) و چگونگیِ گرفتنِ مقادیر توسط متغیرهای شاهد (مدلِ حسگر) است.

مدلِ انتقال توزیعِ احتمال روی جدیدترین متغیرهای حالت را، با توجه به مقادیرِ قبلی، یعنی P(Xt | X0:t-1) را مشخص می‌کند. حالا با مسئله‌ای روبه‌روییم: مجموعهٔ X0:t-1 با افزایشِ t در اندازه نامحدود است. مسئله را با «فرضِ مارکوف» (Markov Assumption) حل می‌کنیم—که حالتِ کنونی فقط به شمارِ متناهی و ثابتی از حالت‌های قبلی بستگی دارد. فرایندهای ارضاکنندهٔ این فرض نخستین بار توسط آماردانِ آندری مارکوف (۱۹۲۲–۱۸۵۶) عمیقاً مطالعه شدند و «فرایندهای مارکوف» (Markov Processes) یا «زنجیره‌های مارکوف» (Markov Chains) نامیده می‌شوند. آن‌ها در طعم‌های گوناگونی می‌آیند؛ ساده‌ترین «فرایندِ مارکوفِ مرتبهٔ اول» (First-Order Markov Process) است، که در آن حالتِ کنونی فقط به حالتِ قبلی بستگی دارد و به هیچ حالتِ زودتری. به بیانِ دیگر، حالت به‌اندازه‌ای اطلاعات فراهم می‌کند که آینده را به‌صورتِ شرطی مستقل از گذشته کند، و داریم:

$$P(X_t \mid X_{0:t-1}) = P(X_t \mid X_{t-1}) \qquad (14.1)$$

از این رو، در فرایندِ مارکوفِ مرتبهٔ اول، مدلِ انتقال توزیعِ شرطیِ P(Xt | Xt-1) است. مدلِ انتقال برای فرایندِ مارکوفِ مرتبهٔ دوم توزیعِ شرطیِ P(Xt | Xt-2, Xt-1) است. شکل ۱۴.۱ ساختارهای شبکهٔ بیزیِ متناظر با فرایندهای مارکوفِ مرتبهٔ اول و دوم را نشان می‌دهد.

حتی با فرضِ مارکوف هنوز مسئله‌ای هست: بی‌نهایت مقدارِ ممکن برای t وجود دارد. آیا باید توزیعِ متفاوتی برای هر گامِ زمانی مشخص کنیم؟ از این مسئله با فرضِ اینکه تغییرهای حالتِ جهان توسط فرایندی «همگن در زمان» (Time-Homogeneous) ایجاد می‌شوند—یعنی فرایندِ تغییری که توسط قوانینی اداره می‌شود که خود در طولِ زمان تغییر نمی‌کنند—اجتناب می‌کنیم. در جهانِ چتر، پس احتمالِ شرطیِ باران، P(Rt | Rt-1)، برای همهٔ t یکسان است و فقط به مشخص‌کردنِ یک جدولِ احتمالِ شرطی نیاز داریم.

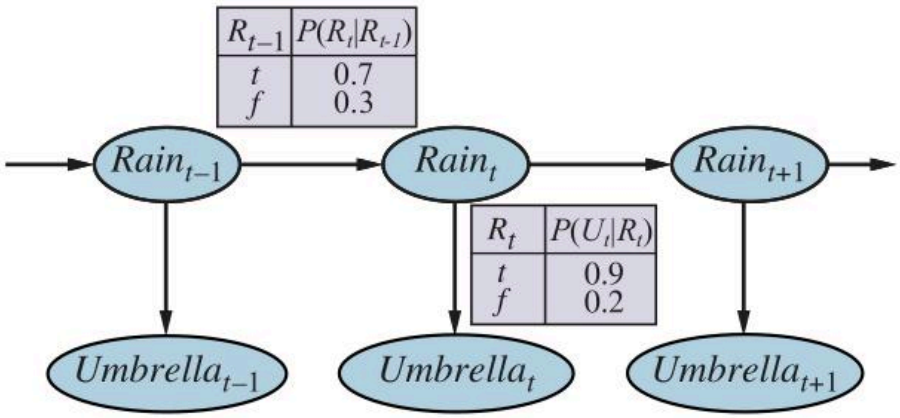
حالا نوبتِ مدلِ حسگر است. متغیرهای شاهدِ Et می‌توانند به متغیرهای قبلی و نیز متغیرهای حالتِ کنونی بستگی داشته باشند، اما هر حالتی که به‌اندازهٔ کافی خوب باشد باید برای تولیدِ مقادیرِ حسگرِ کنونی کافی باشد. بدین‌ترتیب، فرضِ مارکوفِ حسگر را به‌صورتِ زیر می‌سازیم:

$$P(Et \mid X0{:}t, E1{:}t-1) = P(Et \mid Xt) \qquad (14.2)$$

بدین‌ترتیب، P(Et | Xt) مدلِ حسگرِ ماست (که گاهی «مدلِ مشاهده» (Observation Model) نامیده می‌شود). شکل ۱۴.۲ هم مدلِ انتقال و هم مدلِ حسگر را برای مثالِ چتر نشان می‌دهد. جهتِ وابستگی میان حالت و حسگرها را توجه کنید: پیکان‌ها از حالتِ واقعیِ جهان به مقادیرِ حسگر می‌روند، زیرا حالتِ جهان باعث می‌شود حسگرها مقادیرِ خاصی بگیرند: باران باعث می‌شود چتر ظاهر شود. (فرایندِ استنتاج، البته، در جهتِ دیگر می‌رود؛ تمایز میان جهتِ وابستگی‌های مدل‌شده و جهتِ استنتاج یکی از مزیت‌های اصلیِ شبکه‌های بیزی است.) علاوه بر مشخص‌کردنِ مدل‌های انتقال و حسگر، باید بگوییم همه‌چیز چگونه شروع می‌شود—توزیعِ احتمالِ پیشینی در زمانِ 0، یعنی P(X0). با آن، مشخصاتِ کاملی از توزیعِ مشترکِ کلِ متغیرها داریم، با استفاده از معادلهٔ (13.2). برای هر گامِ زمانیِ t:

$$P(X0{:}t, E1{:}t) = P(X0) \prod_{i=1..t} P(Xi \mid Xi-1)\, P(Ei \mid Xi) \qquad (14.3)$$

سه جملهٔ سمتِ راست عبارت‌اند از مدلِ حالتِ اولیهٔ P(X0)، مدلِ انتقالِ P(Xi | Xi-1) و مدلِ حسگرِ P(Ei | Xi). این معادله معناشناسیِ خانوادهٔ مدل‌های زمانیِ بازنمایی‌شده توسط سه جمله را تعریف می‌کند. توجه کنید شبکه‌های بیزیِ استاندارد نمی‌توانند چنین مدل‌هایی را بازنمایی کنند، زیرا به مجموعهٔ متناهی از متغیرها نیاز دارند. توانایی مدیریتِ مجموعهٔ نامتناهی از متغیرها از دو چیز می‌آید: نخست، تعریفِ مجموعهٔ نامتناهی با نمایه‌های عددِ صحیح؛ و دوم، استفاده از «سورِ عمومیِ ضمنی» (بخش ۸.۲ را ببینید) برای تعریفِ مدل‌های حسگر و انتقال برای هر گامِ زمانی.



شکل ۱۴.۲ – ساختارِ شبکهٔ بیزی و توزیع‌های شرطی برای جهانِ چتر.

ساختارِ شکل ۱۴.۲ فرایندِ مارکوفِ مرتبهٔ اول است—فرض می‌شود احتمالِ باران فقط به باریدن یا نباریدنِ باران در روزِ قبل بستگی دارد. اینکه چنین فرضی معقول است یا نه به خودِ حوزه بستگی دارد. فرضِ مارکوفِ مرتبهٔ اول می‌گوید متغیرهای حالت شاملِ همهٔ اطلاعاتِ لازم برای مشخصه‌سازیِ توزیعِ احتمالِ برشِ زمانیِ بعد هستند. گاهی فرض دقیقاً صادق است—برای نمونه، اگر ذره‌ای در حالِ اجرایِ «پیاده‌رویِ تصادفی» (Random Walk) روی محورِ x باشد و در هر گامِ زمانی موقعیتش را به‌اندازهٔ ±1 تغییر دهد، آنگاه استفاده از مختصهٔ x به‌عنوانِ حالت، فرایندِ مارکوفِ مرتبهٔ اول می‌دهد. گاهی فرض فقط تقریبی است، مانندِ موردِ پیش‌بینیِ باران فقط بر اساسِ باریدن یا نباریدنِ باران در روزِ قبل. دو راه برای بهبودِ دقتِ تقریب هست:

1. افزایشِ مرتبهٔ مدلِ فرایندِ مارکوف. برای نمونه، می‌توانیم مدلِ مرتبهٔ دوم بسازیم با افزودنِ Raint-2 به‌عنوانِ والدِ Raint، که ممکن است پیش‌بینی‌های کمی دقیق‌تری بدهد. برای نمونه، در پالو آلتو، کالیفرنیا، بسیار به‌ندرت بیش از دو روز متوالی باران می‌بارد. (با این حال، افزایشِ مرتبه هزینه دارد: افزودنِ والدها پیچیدگیِ CPTها را افزایش می‌دهد و در نتیجه زمانِ استنتاج را.)
2. افزایشِ مجموعهٔ متغیرهای حالت. برای نمونه، می‌توانیم Seasont را بیفزاییم تا به ما اجازه دهد سوابقِ تاریخیِ فصل‌های بارانی را بگنجانیم، یا می‌توانیم Temperaturet (دما)، Humidityt (رطوبت) و Pressuret (فشار هوا) را (شاید در مجموعه‌ای از مکان‌ها) بیفزاییم تا به ما امکان دهد از یک مدلِ فیزیکی برای شرایطِ بارانی استفاده کنیم.

تمرین AUGM.۱۴ از شما می‌خواهد نشان دهید که راه‌حلِ نخست—افزایشِ مرتبه—همیشه می‌تواند به‌صورتِ افزایش در مجموعهٔ متغیرهای حالت با ثابت نگه داشتنِ مرتبه بازصورت‌بندی شود. توجه کنید که افزودنِ متغیرهای حالت ممکن است قدرتِ پیش‌بینیِ سامانه را بهبود بخشد، اما نیازمندی‌های پیش‌بینی را نیز افزایش می‌دهد: اکنون باید متغیرهای جدید را نیز پیش‌بینی کنیم. از این رو، ما به دنبالِ مجموعه‌ای از متغیرهای «خودبسنده» (Self-Sufficient) هستیم؛ که در واقع بدان معناست که باید «فیزیکِ» فرایندِ در حالِ مدل‌سازی را درک کنیم. نیاز به مدل‌سازیِ دقیقِ فرایند آشکارا کاهش می‌یابد اگر بتوانیم حسگرهای جدیدی بیفزاییم (مثلاً اندازه‌گیری‌های دما و فشار) که مستقیماً اطلاعاتی دربارهٔ متغیرهای حالتِ جدید فراهم می‌کنند.

برای نمونه، مسئلهٔ ردیابیِ رباتی را در نظر بگیرید که به‌صورتِ تصادفی روی صفحهٔ X–Y پرسه می‌زند. ممکن است کسی پیشنهاد کند که موقعیت و سرعت مجموعهٔ کافی از متغیرهای حالت هستند: می‌توان به‌سادگی از قوانینِ نیوتن برای محاسبهٔ موقعیتِ جدید استفاده کرد، و سرعت ممکن است به‌صورتِ غیرقابلِ‌پیش‌بینی تغییر کند. اما اگر ربات با باتری کار کند، آنگاه تخلیهٔ باتری تمایل دارد اثری نظام‌مند بر تغییرِ سرعت بگذارد. چون این امر به‌نوبهٔ خود به این بستگی دارد که چه مقدار توان در تمامِ مانورهای قبلی مصرف شده است، ویژگیِ مارکوف نقض می‌شود.

می‌توانیم ویژگیِ مارکوف را با گنجاندنِ سطحِ شارژِ Batteryt به‌عنوانِ یکی از متغیرهای حالتِ سازندهٔ Xt بازیابی کنیم. این امر به پیش‌بینیِ حرکتِ ربات کمک می‌کند، اما به‌نوبهٔ خود نیازمندِ مدلی برای پیش‌بینیِ Batteryt از روی Batteryt-1 و سرعت است. در برخی موارد، این کار را می‌توان به‌صورتِ قابلِ‌اعتماد انجام داد، اما اغلب درمی‌یابیم که خطا در طولِ زمان انباشته می‌شود. در آن حالت، دقت را می‌توان با افزودنِ حسگری جدید برای سطحِ باتری بهبود بخشید. در بخش ۱۴.۵ به مثالِ باتری بازخواهیم گشت.

۱۴.۲ استنتاج در مدل‌های زمانی

با برپاییِ ساختارِ مدلِ زمانیِ عام، می‌توانیم تکلیف‌های استنتاجیِ پایه‌ای را که باید حل شوند صورت‌بندی کنیم:

- «فیلترکردن» (Filtering)[۲] یا «برآوردِ حالت» (State Estimation) تکلیفِ محاسبهٔ حالتِ باورِ P(Xt | e1:t)—توزیعِ پسینی روی جدیدترین حالت، با توجه به همهٔ شواهدِ تاکنون—است. در مثالِ چتر، این به معنای محاسبهٔ احتمالِ باریدنِ بارانِ امروز، با توجه به همهٔ مشاهده‌های چترِ انجام‌شده تاکنون، است. فیلترکردن کاری است که عاملِ عقلایی برای پیگیریِ حالتِ کنونی انجام می‌دهد تا تصمیم‌های عقلایی گرفته شوند. معلوم می‌شود محاسبه‌ای تقریباً یکسان، درستنماییِ توالیِ شواهد، P(e1:t) را فراهم می‌کند.
- «پیش‌بینی» (Prediction): این تکلیفِ محاسبهٔ توزیعِ پسینی روی حالتِ آینده، با توجه به همهٔ شواهدِ تاکنون، است. یعنی می‌خواهیم P(Xt+k | e1:t) را برای k>0 محاسبه کنیم. در مثالِ چتر، این ممکن است به معنای محاسبهٔ احتمالِ باران سه روزِ دیگر، با توجه به همهٔ مشاهده‌های تاکنون، باشد. پیش‌بینی برای ارزیابیِ دوره‌های کنشِ ممکن بر اساسِ پیامدهای موردانتظارشان مفید است.
- «هموارسازی» (Smoothing)[۳]: این تکلیفِ محاسبهٔ توزیعِ پسینی روی حالتِ گذشته، با توجه به همهٔ شواهدِ تا حال، است. یعنی می‌خواهیم P(Xk | e1:t) را برای 0 ≤ k < t محاسبه کنیم. در مثالِ چتر، ممکن است به معنای محاسبهٔ احتمالِ اینکه چهارشنبهٔ گذشته باران باریده باشد، با توجه به همهٔ مشاهده‌های چتربَردارِ انجام‌شده تا امروز، باشد. هموارسازی برآوردِ بهتری از حالت در زمانِ k می‌دهد تا آنچه در آن زمان در دسترس بود، زیرا شواهدِ بیشتری را در بر می‌گیرد.[۳]
- «محتمل‌ترین تبیین» (Most Likely Explanation): با توجه به توالی‌ای از مشاهده‌ها، ممکن است بخواهیم توالیِ حالت‌هایی را بیابیم که به‌احتمالِ زیاد آن مشاهده‌ها را تولید کرده‌اند. یعنی می‌خواهیم argmax_{x1:t} P(x1:t | e1:t) را محاسبه کنیم. برای نمونه، اگر چتر در هر یک از سه روزِ نخست ظاهر شود و در روزِ چهارم غایب باشد، آنگاه محتمل‌ترین تبیین این است که در سه روزِ نخست باران باریده و در روزِ چهارم نباریده.

الگوریتم‌های این تکلیف در بسیاری از کاربردها مفیدند، از جملهٔ بازشناسیِ گفتار—که در آن هدف یافتنِ محتمل‌ترین توالیِ واژه‌ها، با توجه به مجموعه‌ای از اصوات، است—و بازسازیِ رشته‌های بیتیِ ارسالی از کانالِ نویزی.

علاوه بر این تکلیف‌های استنتاجی، همچنین داریم:

- «یادگیری» (Learning): مدل‌های انتقال و حسگر، اگر هنوز شناخته نشده باشند، می‌توانند از مشاهده‌ها یاد گرفته شوند. همانندِ شبکه‌های بیزیِ ایستا، یادگیریِ شبکهٔ بیزِ پویا می‌تواند به‌عنوانِ محصولِ جانبیِ استنتاج انجام شود. استنتاج برآوردی از اینکه واقعاً چه انتقال‌هایی رخ داده‌اند و چه حالت‌هایی خوانش‌های حسگر را تولید کرده‌اند فراهم می‌کند، و این برآوردها را می‌توان برای یادگیریِ مدل‌ها استفاده کرد. فرایندِ یادگیری می‌تواند از طریقِ الگوریتمِ به‌روزرسانیِ تکرارشونده‌ای به نام «بیشینه‌سازیِ انتظار» (Expectation–Maximization یا EM) عمل کند، یا می‌تواند از به‌روزرسانیِ بیزیِ پارامترهای مدل با توجه به شواهد نتیجه شود. جزئیاتِ بیشتر در فصل ۲۰.

بقیهٔ این بخش الگوریتم‌های عام را برای چهار تکلیفِ استنتاجی توصیف می‌کند، مستقل از نوعِ خاصِ مدلِ به‌کاررفته. بهبودهای خاصِ هر مدل در بخش‌های بعدی توصیف شده‌اند.

۱۴.۲.۱ فیلترکردن و پیش‌بینی

چنان‌که در بخش ۷.۷.۳ اشاره کردیم، الگوریتمِ فیلترکردنِ مفید باید برآوردِ حالتِ کنونی را نگه دارد و آن را به‌روزرسانی کند، نه اینکه برای هر به‌روزرسانی به کلِ تاریخچهٔ ادراک‌ها بازگردد. (در غیرِ این صورت، هزینهٔ هر به‌روزرسانی با گذشتِ زمان افزایش می‌یابد.) به بیانِ دیگر، با توجه به نتیجهٔ فیلترکردن تا زمانِ t، عامل باید نتیجه را برای t+1 از شاهدِ تازهٔ et+1 محاسبه کند. پس P(Xt+1 | e1:t+1) = f(et+1, P(Xt | e1:t)) برای تابعی مانند f. این فرایند «برآوردِ بازگشتی» (Recursive Estimation) نامیده می‌شود. (همچنین بخش‌های ۴.۴ و ۷.۷.۳ را ببینید.) می‌توانیم محاسبه را متشکل از دو بخش ببینیم: نخست، توزیعِ حالتِ کنونی از t به t+1 به جلو پرتاب می‌شود؛ سپس با شاهدِ تازهٔ et+1 به‌روزرسانی می‌شود. این فرایندِ دوبخشی به‌سادگی وقتی فرمول بازآرایی می‌شود ظاهر می‌گردد:

```
P(Xt+1 | e1:t+1) = P(Xt+1 | e1:t, et+1)                    (تقسیمِ شواهد)
        α P(et+1 | Xt+1, e1:t) P(Xt+1 | e1:t) =             (با قاعدهٔ بیز، به‌شرطِ e1:t)
        α P(et+1 | Xt+1) · P(Xt+1 | e1:t) =                 (با فرضِ مارکوفِ حسگر)
   به‌روزرسانی ──┘  └── پیش‌بینی ──┘                        (14.4)
```

در این‌جا و در سراسرِ این فصل، α ثابتِ نرمال‌سازی برای جمع‌شدنِ احتمال‌ها به ۱ است. حالا عبارتی برای پیش‌بینیِ تک‌گامیِ P(Xt+1 | e1:t) جای می‌کنیم، که با شرطی‌کردن بر حالتِ کنونیِ Xt به دست می‌آید. معادلهٔ حاصل برای برآوردِ حالتِ تازه، نتیجهٔ مرکزیِ این فصل است:

```
                                                                    P(Xt+1 | e1:t+1) = α P(et+1 | Xt+1) ∑ P(Xt+1 | xt, e1:t) P(xt | e1:t)
                                                                                                        xt
                                                                                                
```

به‌صورتِ شهودی، احتمالِ باران از روزِ 1 به روزِ 2 افزایش می‌یابد، زیرا باران پایدار است. تمرین CONV.۱۴(الف) از شما می‌خواهد این گرایش را بیشتر بررسی کنید.

تکلیفِ پیش‌بینی را می‌توان صرفاً فیلترکردن بدونِ افزودنِ شاهدِ تازه دید. در واقع، فرایندِ فیلترکردن قبلاً پیش‌بینیِ تک‌گامی را در بر دارد، و استخراجِ محاسبهٔ بازگشتیِ زیر برای پیش‌بینیِ حالت در t+k+1 از پیش‌بینی برای t+k آسان است:

$$P(X_{t+k+1} \mid e_{1:t}) = \sum_{x_{t+k}} P(X_{t+k+1} \mid x_{t+k}) \, P(x_{t+k} \mid e_{1:t}) \qquad (14.6)$$

(بازگشت ← مدل انتقال ← x_{t+k})

طبیعتاً، این محاسبه فقط مدلِ انتقال را شامل می‌شود، نه مدلِ حسگر را.

جالب است در نظر بگیریم هنگامِ تلاش برای پیش‌بینیِ دورتر و دورتر در آینده چه اتفاقی می‌افتد. چنان‌که تمرین CONV.۱۴(ب) نشان می‌دهد، توزیعِ پیش‌بینی‌شده برای باران به نقطهٔ ثابتی ⟨0.5, 0.5⟩ همگرا می‌شود، پس از آن برای همهٔ زمان‌ها ثابت می‌ماند.[۴] این «توزیعِ مانا» (Stationary Distribution) فرایندِ مارکوفِ تعریف‌شده توسط مدلِ انتقال است. (همچنین صفحهٔ ۴۴۴ را ببینید.) دربارهٔ ویژگی‌های چنین توزیع‌هایی و دربارهٔ «زمانِ آمیختن» (Mixing Time)—تقریباً، زمانِ لازم برای رسیدن به نقطهٔ ثابت—دانسته‌های بسیاری هست. از نظرِ عملی، این هر تلاشی برای پیش‌بینیِ حالتِ واقعی برای شماری از گام‌ها که بیش از کسری کوچک از زمانِ آمیختن است را محکوم به شکست می‌کند، مگر اینکه خودِ توزیعِ مانا در ناحیه‌ای کوچک از فضایِ حالت به‌شدت قله‌دار باشد. هرچه عدم قطعیت در مدلِ انتقال بیشتر باشد، زمانِ آمیختن کوتاه‌تر و آینده بیشتر مبهم خواهد بود.

علاوه بر فیلترکردن و پیش‌بینی، می‌توانیم از بازگشتِ پیشرو برای محاسبهٔ درستنماییِ توالیِ شواهد، $P(e_{1:t})$، استفاده کنیم. این کمیتِ مفیدی است اگر بخواهیم مدل‌های زمانیِ متفاوتی را که ممکن است همان توالیِ شواهد را تولید کرده باشند مقایسه کنیم (مثلاً دو مدلِ متفاوت برای پایداریِ باران). برای این بازگشت، از «پیامِ درستنماییِ» $\ell_{1:t}(X_t) = P(X_t, e_{1:t})$ استفاده می‌کنیم. نشان‌دادنِ اینکه محاسبهٔ پیام یکسان با فیلترکردن است آسان است (تمرین LIKL.۱۴):

$$\ell_{1:t+1} = \text{FORWARD}(\ell_{1:t}, e_{t+1})$$

پس از محاسبهٔ $\ell_{1:t}$، درستنماییِ واقعی را با خارج‌کردنِ X_t از جمع به دست می‌آوریم:

$$L_{1:t} = P(e_{1:t}) = \sum_{x_t} \ell_{1:t}(x_t) \qquad (14.7)$$

توجه کنید پیامِ درستنمایی با گذشتِ زمان احتمال‌های توالی‌های شاهدِ بلندتر و بلندتر را بازنمایی می‌کند و بنابراین از نظرِ عددی کوچک‌تر و کوچک‌تر می‌شود، که به مسائلِ «زیرریز» (Underflow) با حسابِ ممیزِ شناور می‌انجامد. این در عمل مسئلهٔ مهمی است، اما در این‌جا به راه‌حل‌ها نمی‌پردازیم.

۱۴.۲.۲ هموارسازی

چنان‌که قبلاً گفتیم، هموارسازی فرایندِ محاسبهٔ توزیع روی حالت‌های گذشته با توجه به شواهدِ تا حال است—یعنی P(Xk | e1:t) برای k < t ≥ 0. (شکل ۱۴.۳ را ببینید.) در انتظارِ رویکردِ دیگری از انتقالِ پیامِ بازگشتی، می‌توانیم محاسبه را به دو بخش تقسیم کنیم—شواهدِ تا k و شواهدِ از k+1 تا t:

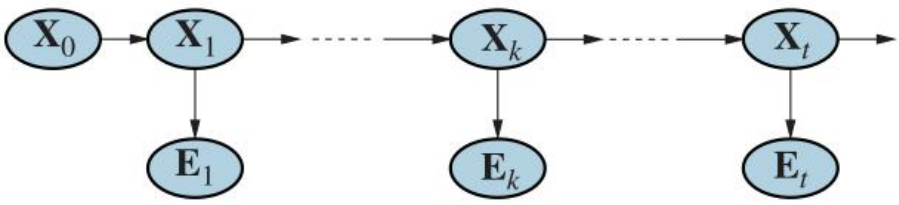
```
P(Xk | e1:t) = P(Xk | e1:k, ek+1:t)
= α P(Xk | e1:k) P(ek+1:t | Xk, e1:k)      (با قاعدهٔ بیز، به‌شرطِ e1:k)
= α P(Xk | e1:k) P(ek+1:t | Xk)            (با استقلالِ شرطی)
= α f1:k × bk+1:t                  (14.8)
```

که در آن «×» ضربِ نقطه‌ایِ بردارهاست. در این‌جا «پیامِ عقب‌رو» bk+1:t = P(ek+1:t | Xk) تعریف کرده‌ایم، مشابهِ پیامِ پیشروِ f1:k. پیامِ پیشروِ f1:k را می‌توان با فیلترکردنِ پیشرو از 1 تا k محاسبه کرد، چنان‌که معادلهٔ (14.5) می‌دهد. معلوم می‌شود پیامِ عقب‌روِ bk+1:t را می‌توان با فرایندِ بازگشتی‌ای محاسبه کرد که از t به عقب می‌رود:

```
P(ek+1:t | Xk) = ∑ P(ek+1:t | Xk, xk+1) P(xk+1 | Xk)      (شرطی‌کردن بر Xk+1)
                 xk+1
= ∑ P(ek+1:t | xk+1) P(xk+1 | Xk)                        (با استقلالِ شرطی)
  xk+1
= ∑ P(ek+1, ek+2:t | xk+1) P(xk+1 | Xk)
  xk+1
= ∑ P(ek+1 | xk+1) P(ek+2:t | xk+1) P(xk+1 | Xk)        (14.9)
  xk+1
```

که گامِ آخر با استقلالِ شرطیِ ek+1 و ek+2:t، به‌شرطِ xk+1، نتیجه می‌شود.

در این عبارت، همهٔ جمله‌ها یا از مدل می‌آیند یا از پیامِ عقب‌رویِ قبلی. از این رو، صورت‌بندیِ بازگشتیِ موردنظر را داریم. در شکلِ پیام، bk+1:t = BACKWARD(bk+2:t, ek+1) داریم، که در آن BACKWARD به‌روزرسانیِ توصیف‌شده در معادلهٔ (14.9) را پیاده‌سازی می‌کند. مانندِ بازگشتِ پیشرو، زمان و فضای لازم برای هر به‌روزرسانی ثابت و بنابراین مستقل از t است.



شکل ۱۴.۳ — هموارسازی P(Xk | e1:t) را محاسبه می‌کند، توزیعِ پسینیِ حالت در زمانِ گذشتهٔ k با توجه به توالیِ کاملِ مشاهده‌ها از 1 تا t.

حالا می‌توانیم ببینیم دو جملهٔ معادلهٔ (14.8) هر دو با بازگشت‌هایی در طولِ زمان قابلِ محاسبه‌اند، یکی پیشرو از 1 تا k با استفاده از معادلهٔ فیلترکردنِ (14.5) و دیگری عقب‌رو از t تا k+1 با استفاده از معادلهٔ (14.9).

برای مقداردهیِ اولیهٔ فازِ عقب‌رو، bt+1:t = P(et+1:t | Xt) = P(| Xt) = 1 داریم، که در آن 1 بردارِ 1هاست. دلیلش این است که et+1:t توالیِ خالی است، پس احتمالِ مشاهدهٔ آن ۱ است.

حالا این الگوریتم را روی مثالِ چتر اعمال کنیم، برآوردِ هموارشدهٔ احتمالِ باران را در زمانِ k=1، با توجه به مشاهده‌های چتر در روزهای 1 و 2، محاسبه می‌کنیم. از معادلهٔ (14.8)، این چنین داده می‌شود:

$$P(R1 \mid u1, u2) = \alpha\, P(R1 \mid u1)\, P(u2 \mid R1) \qquad (14.10)$$

جملهٔ نخست را قبلاً می‌دانیم ⟨.818, .182⟩ است، از فرایندِ فیلترکردنِ پیشرویِ توصیف‌شدهٔ پیشین. جملهٔ دوم را می‌توان با اعمالِ بازگشتِ عقب‌رو در معادلهٔ (14.9) محاسبه کرد:

$$P(u2 \mid R1) = \sum_{r2} P(u2 \mid r2)\, P(\ \mid r2)\, P(r2 \mid R1)$$
$$= (0.9 \times 1 \times \langle 0.3, 0.7\rangle) + (0.2 \times 1 \times \langle 0.7, 0.3\rangle) = \langle 0.69, 0.41\rangle$$

با جای‌گذاریِ این در معادلهٔ (14.10)، درمی‌یابیم برآوردِ هموارشدهٔ باران در روزِ 1:

$$P(R1 \mid u1, u2) = \alpha\, \langle 0.818, 0.182\rangle \times \langle 0.69, 0.41\rangle \approx \langle 0.883, 0.117\rangle$$

بدین‌ترتیب، برآوردِ هموارشدهٔ باران در روزِ 1 در این مورد از برآوردِ فیلترشده (0.818) بالاتر است. این به این دلیل است که چتر در روزِ 2 محتمل‌تر می‌کند در روزِ 2 باران باریده باشد؛ به‌نوبهٔ خود، چون باران گرایش به پایداری دارد، این محتمل‌تر می‌کند در روزِ 1 باران باریده باشد.

هر دو بازگشتِ پیشرو و عقب‌رو مقدارِ زمانیِ ثابتی در هر گام می‌گیرند؛ از این رو، پیچیدگیِ زمانیِ هموارسازی نسبت به شاهدِ e1:t برابر O(t) است. این پیچیدگیِ هموارسازی در گامِ زمانیِ خاصِ k است. اگر بخواهیم کلِ توالی را هموار کنیم، یکی از روش‌های بدیهی صرفاً اجرایِ کلِ فرایندِ هموارسازی یک بار برای هر گامِ زمانیِ موردِ هموارسازی است. این به پیچیدگیِ زمانیِ $O(t^2)$ می‌انجامد.

رویکردِ بهتر از کاربردِ ساده‌ای از برنامه‌نویسیِ پویا برای کاهشِ پیچیدگی به O(t) استفاده می‌کند. سرنخی در تحلیلِ پیشینِ مثالِ چتر ظاهر می‌شود، که در آن قادر بودیم نتایجِ فازِ فیلترکردنِ پیشرو را دوباره استفاده کنیم. کلیدِ الگوریتمِ خطی‌زمان ثبتِ نتایجِ فیلترکردنِ پیشرو روی کلِ توالی است. سپس بازگشتِ عقب‌رو را از t تا 1 اجرا می‌کنیم و برآوردِ هموارشده را در هر گامِ k از پیامِ عقب‌رویِ محاسبه‌شدهٔ bk+1:t و پیامِ پیشرویِ ذخیره‌شدهٔ f1:k محاسبه می‌کنیم. الگوریتم، که به‌درستی «الگوریتمِ پیشرو–عقب‌رو» (Forward–Backward Algorithm) نامیده می‌شود، در شکل ۱۴.۴ نشان داده شده.

```
function FORWARD-BACKWARD(ev, prior) returns a vector of probability distributions
                          inputs: ev, a vector of evidence values for steps 1,...,t
                       prior, the prior distribution over the initial state, P(X0)
                       local variables: fv, a vector of forward messages for steps 0,...,t
                  b, the backward message representation, initially all 1
                       sv, a vector of smoothed estimates for steps 1,...,t
                                                                  fv[0] ← prior
                                                                :for i = 1 to t
                                                  fv[i] ← FORWARD(fv[i–1], ev[i])
                                                               :for i = t down to 1
                                                    sv[i] ← NORMALIZE(fv[i] × b)
                                                          b ← BACKWARD(b, ev[i])
                                                                      return sv
```

شکل ۱۴.۴ — الگوریتمِ پیشرو–عقب‌رو برای هموارسازی: محاسبهٔ احتمال‌های پسینیِ توالی‌ای از حالت‌ها با توجه به توالی‌ای از مشاهده‌ها. عملگرهای FORWARD و BACKWARD به‌ترتیب توسط معادله‌های (14.5) و (14.9) تعریف می‌شوند.

خوانندهٔ هوشیار خواهد دید ساختارِ شبکهٔ بیزیِ نشان‌داده‌شده در شکل ۱۴.۳ پلیدرختِ تعریف‌شده در صفحهٔ ۴۳۳ است. این بدان معناست کاربردِ سرراستِ الگوریتمِ خوشه‌بندی نیز الگوریتمِ خطی‌زمانی می‌دهد که برآوردهای هموارشده را برای کلِ توالی محاسبه می‌کند. اکنون فهمیده شده الگوریتمِ پیشرو–عقب‌رو در واقع حالتِ خاصِ الگوریتمِ انتشارِ پلیدرختِ استفاده‌شده با روش‌های خوشه‌بندی است (هرچند این دو به‌طورِ مستقل توسعه یافتند).

الگوریتمِ پیشرو–عقب‌رو ستونِ فقراتِ محاسباتیِ بسیاری از کاربردهایی است که با توالی‌های مشاهدهٔ نویزی سروکار دارند. چنان‌که تاکنون توصیف شده، دو عیبِ عملی دارد. نخست، پیچیدگیِ فضایی‌اش می‌تواند وقتی فضایِ حالت بزرگ و توالی‌ها بلندند بیش از حد زیاد شود. از O(|f|·t) فضا استفاده می‌کند، که در آن |f| اندازهٔ بازنماییِ پیامِ پیشرو است. نیازِ فضایی را می‌توان با افزایشِ هم‌زمانِ پیچیدگیِ زمانی به‌وسیلهٔ عاملِ log t به O(|f|·log t) کاهش داد، چنان‌که در تمرین ISLE.۱۴ نشان داده شده. در برخی موارد (بخش ۱۴.۳ را ببینید)، می‌توان از الگوریتمِ فضایِ ثابت استفاده کرد.

عیبِ دومِ الگوریتمِ پایه این است که باید برای کار در محیطِ برخط اصلاح شود، جایی که برآوردهای هموارشده باید برای برش‌های زمانیِ زودتر محاسبه شوند، چون مشاهده‌های تازه به‌طورِ پیوسته به انتهایِ توالی افزوده می‌شوند. رایج‌ترین نیاز «هموارسازیِ با تأخیرِ ثابت» (Fixed-Lag Smoothing) است، که محاسبهٔ برآوردِ هموارشدهٔ P(Xt-d | e1:t) را برای d ثابت لازم دارد. یعنی هموارسازی برای برشِ زمانیِ d گامِ پشتِ زمانِ کنونیِ t انجام می‌شود؛ با افزایشِ t، هموارسازی باید هم‌گام بماند. بدیهی است، می‌توانیم الگوریتمِ پیشرو–عقب‌رو را روی «پنجرهٔ» d-گامی با افزوده‌شدنِ هر مشاهدهٔ تازه اجرا کنیم، اما این ناکارآمد به نظر می‌رسد. در بخش ۱۴.۳ خواهیم دید هموارسازیِ با تأخیرِ ثابت در برخی موارد می‌تواند در زمانِ ثابتِ هر به‌روزرسانی، مستقل از تأخیرِ d، انجام شود.

۱۴.۲.۳ یافتنِ محتمل‌ترین توالی

فرض کنیم [true, true, false, true, true] توالیِ چترِ مشاهده‌شده برای پنج روزِ نخستِ کارِ نگهبانِ امنیتی باشد. کدام توالیِ آب‌وهوا به‌احتمالِ زیاد این را تبیین می‌کند؟ آیا غیبتِ چتر در روزِ 3 بدان معناست باران نباریده، یا مدیر فراموش کرده آن را بیاورد؟ اگر در روزِ 3 باران نباریده، شاید (زیرا آب‌وهوا گرایش به پایداری دارد) در روزِ 4 نیز نباریده، اما مدیر

چتر را احتیاطاً آورده. در مجموع، $2^5 = 32$ توالیِ آب‌وهوایِ ممکن هست که می‌توانیم برگزینیم. آیا راهی برای یافتنِ محتمل‌ترین توالی هست، بدونِ شمردنِ همهٔ آن‌ها و محاسبهٔ درستنمایی‌شان؟

می‌توانیم این رویهٔ خطی‌زمان را امتحان کنیم: از هموارسازی برای یافتنِ توزیعِ پسینیِ آب‌وهوا در هر گامِ زمانی استفاده کن؛ سپس توالی را بساز، با استفاده از آب‌وهوایی که طبقِ پسین در هر گام محتمل‌ترین است. چنین رویکردی باید در سرِ خواننده زنگِ خطر به صدا درآورد، زیرا توزیع‌های پسینیِ محاسبه‌شده توسط هموارسازی توزیع‌هایی روی گام‌های زمانیِ تکی‌اند، در حالی که برای یافتنِ محتمل‌ترین توالی باید احتمال‌های مشترکِ همهٔ گام‌های زمانی را در نظر بگیریم. نتایج در واقع می‌توانند کاملاً متفاوت باشند. (تمرین VITE.۱۴ را ببینید.) الگوریتمِ خطی‌زمانی برای یافتنِ محتمل‌ترین توالی هست، اما تفکرِ بیشتری لازم دارد. به همان ویژگیِ مارکوف تکیه می‌کند که الگوریتم‌های کارآمدِ فیلترکردن و هموارسازی را به دست داد. ایده این است که هر توالی را مسیری از میانِ گرافی ببینیم که گره‌هایش حالت‌های ممکن در هر گامِ زمانی‌اند. چنین گرافی برای جهانِ چتر در شکل ۱۴.۵(الف) نشان داده شده. حالا تکلیفِ یافتنِ محتمل‌ترین مسیر را از میانِ این گراف در نظر بگیرید، که در آن درستنماییِ هر مسیر حاصلِ‌ضربِ احتمال‌های انتقالِ طولِ مسیر و احتمال‌های مشاهده‌های معین در هر حالت است.

به‌طورِ خاص روی مسیرهایی تمرکز کنیم که به حالتِ Rain5 = true می‌رسند. به دلیلِ ویژگیِ مارکوف، نتیجه می‌شود محتمل‌ترین مسیر به حالتِ Rain5 = true از محتمل‌ترین مسیر به حالتی در زمانِ 4 و سپس انتقالی به Rain5 = true تشکیل می‌شود؛ و حالتی که در زمانِ 4 بخشی از مسیرِ به Rain5 = true خواهد شد، حالتی است که درستنماییِ آن مسیر را بیشینه می‌کند. به بیانِ دیگر، رابطهٔ بازگشتی میان محتمل‌ترین مسیرها به هر حالتِ xt+1 و محتمل‌ترین مسیرها به هر حالتِ xt هست.

می‌توانیم این ویژگی را مستقیماً برای ساختِ الگوریتمِ بازگشتیِ محاسبهٔ محتمل‌ترین مسیر با توجه به شواهد به کار ببریم. از پیامِ بازگشتیِ m1:t استفاده می‌کنیم، مانندِ پیامِ پیشروِ f1:t در الگوریتمِ فیلترکردن. پیام چنین تعریف می‌شود:[5]

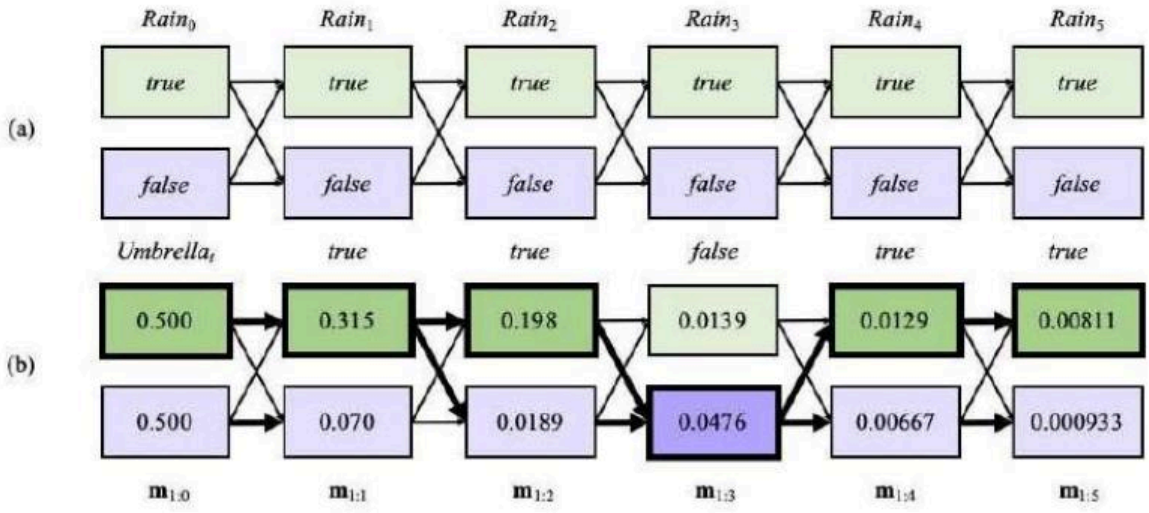
```
m1:t = max_{x1:t-1} P(x1:t-1, Xt, e1:t)
```

برای به‌دست‌آوردنِ رابطهٔ بازگشتی میان m1:t+1 و m1:t، می‌توانیم تقریباً همان گام‌هایی را که برای معادلهٔ (14.5) استفاده کردیم تکرار کنیم:

```
m1:t+1 = max_{x1:t} P(x1:t, Xt+1, e1:t+1)
max_{x1:t} P(x1:t, Xt+1, e1:t, et+1) =
max_{x1:t} P(et+1 | x1:t, Xt+1, e1:t) P(x1:t, Xt+1, e1:t) =
P(et+1 | Xt+1) max_{x1:t} P(Xt+1 | xt) P(x1:t, e1:t) =
P(et+1 | Xt+1) max_{xt} P(Xt+1 | xt) max_{x1:t-1} P(x1:t-1, xt, e1:t) =
                                                    (14.11)
```

که در آن جملهٔ نهایی max_{x1:t-1} P(x1:t-1, xt, e1:t) دقیقاً درایهٔ حالتِ خاصِ xt در بردارِ پیامِ m1:t است. معادلهٔ (14.11) اساساً یکسان با معادلهٔ فیلترکردنِ (14.5) است، به‌جز اینکه جمع روی xt در معادلهٔ (14.5) با بیشینه‌سازی روی xt در معادلهٔ (14.11) جایگزین شده و ثابتِ نرمال‌سازیِ α در معادلهٔ (14.11) وجود ندارد.

بدین‌ترتیب، الگوریتمِ محاسبهٔ محتمل‌ترین توالی شبیهِ فیلترکردن است: در زمانِ 0 با پیشینِ m1:0 = P(X0) شروع می‌شود و سپس در طولِ توالی به جلو می‌رود و پیام را در هر گامِ زمانی با معادلهٔ (14.11) محاسبه می‌کند. پیشرفتِ این محاسبه در شکل ۱۴.۵(ب) نشان داده شده. در پایانِ توالیِ مشاهده، m1:t احتمالِ محتمل‌ترین توالیِ رسیده به هر یک از حالت‌های نهایی را خواهد داشت. بنابراین می‌توان به‌آسانی حالتِ نهاییِ محتمل‌ترین توالی را در کل انتخاب کرد (حالتی که در گامِ 5 پررنگ دور کشیده شده). برای شناساییِ خودِ توالی، نه صرفاً محاسبهٔ احتمالِ آن، الگوریتم همچنین باید برای هر حالت، بهترین حالتی را که به آن می‌انجامد ثبت کند؛ این‌ها با پیکان‌های پررنگ در شکل ۱۴.۵(ب) نشان داده شده‌اند. توالیِ بهینه با دنبال‌کردنِ این پیکان‌های پررنگ به عقب از بهترین حالتِ نهایی شناسایی می‌شود. الگوریتمی که توصیف کردیم «الگوریتمِ ویتربی» (Viterbi Algorithm) نامیده می‌شود، به نامِ مخترعش اندرو ویتربی. مانندِ الگوریتمِ فیلترکردن، پیچیدگیِ زمانی‌اش در t، طولِ توالی، خطی است. برخلافِ فیلترکردن که از فضایِ ثابت استفاده می‌کند، نیازِ فضایی‌اش نیز در t خطی است. این به این دلیل است که الگوریتمِ ویتربی باید اشاره‌گرهایی را نگه دارد که بهترین توالیِ رسیده به هر حالت را شناسایی می‌کنند.



شکل ۱۴.۵ — (الف) توالی‌های حالتِ ممکن برای Raint را می‌توان مسیرهایی از میانِ گرافی از حالت‌های ممکن در هر گامِ زمانی دانست. (حالت‌ها به‌صورتِ مستطیل نشان داده شده‌اند تا با گره‌های شبکهٔ بیز اشتباه نشوند.) (ب) عملکردِ الگوریتمِ ویتربی برای توالیِ مشاهدهٔ چترِ [true, true, false, true, true]، که در آن شواهد از زمانِ 1 شروع می‌شود. برای هر t، مقادیرِ پیامِ m1:t را نشان داده‌ایم، که احتمالِ بهترین توالیِ رسیده به هر حالت در زمانِ t را می‌دهد. همچنین، برای هر حالت، پیکانِ پررنگِ واردشونده بهترین پیشینِ آن را نشان می‌دهد، چنان‌که با حاصلِ‌ضربِ احتمالِ توالیِ قبلی و احتمالِ انتقال سنجیده می‌شود. دنبال‌کردنِ پیکان‌های پررنگ به عقب از محتمل‌ترین حالت در m1:5 محتمل‌ترین توالی را می‌دهد، که با دورهای پررنگ و سایهٔ تیره‌تر نشان داده شده.

یک نکتهٔ عملیِ نهایی: «زیرریز» عددی مسئلهٔ قابلِ‌توجهی برای الگوریتمِ ویتربی است. در شکل ۱۴.۵(ب)، احتمال‌ها کوچک‌تر و کوچک‌تر می‌شوند—و این فقط یک مثالِ اسباب‌بازی است. کاربردهای واقعی در تحلیلِ DNA یا رمزگشاییِ پیام ممکن است هزاران یا میلیون‌ها گام داشته باشند. یک راه‌حلِ ممکن صرفاً نرمال‌سازیِ m در هر گام است؛ این

بازمقیاس‌گذاری درستی را تحتِ تأثیر قرار نمی‌دهد، زیرا max(cx, cy) = c·max(x, y). راه‌حلِ دوم استفاده از احتمال‌های لگاریتمی در همه‌جا و جایگزینیِ ضرب با جمع است. باز، درستی تحتِ تأثیر قرار نمی‌گیرد، زیرا تابعِ لگاریتم یکنواست، پس max(log x, log y) = log max(x, y).

۱۴.۳ مدل‌های مارکوفِ پنهان

بخشِ قبلی الگوریتم‌هایی برای استدلالِ احتمالاتیِ زمانی با چارچوبی عمومی توسعه داد که مستقل از شکلِ خاصِ مدل‌های انتقال و حسگر و مستقل از ماهیتِ متغیرهای حالت و شاهد بود. در این بخش و دو بخشِ بعد، مدل‌ها و کاربردهای عینی‌تری بحث می‌کنیم که قدرتِ الگوریتم‌های پایه را نشان می‌دهند و در برخی موارد بهبودهای بیشتری را اجازه می‌دهند.

با «مدلِ مارکوفِ پنهان» (Hidden Markov Model) یا HMM شروع می‌کنیم. HMM مدلِ زمانیِ احتمالاتی است که در آن حالتِ فرایند با متغیرِ تصادفیِ گسستهٔ تکی توصیف می‌شود. مقادیرِ ممکنِ متغیر، حالت‌های ممکنِ جهان‌اند. مثالِ چترِ توصیف‌شده در بخشِ قبلی بنابراین HMM است، زیرا فقط یک متغیرِ حالت دارد: Raint. اگر مدلی با دو یا چند متغیرِ حالت داشته باشید چه اتفاقی می‌افتد؟ باز هم می‌توانید آن را در چارچوبِ HMM جای دهید، با ترکیبِ متغیرها در «کلان‌متغیرِ» تکی که مقادیرش همهٔ چندتایی‌های ممکنِ مقادیرِ متغیرهای حالتِ فردی‌اند. خواهیم دید ساختارِ مقیدِ HMMها پیاده‌سازیِ ماتریسیِ ساده و زیبایی برای همهٔ الگوریتم‌های پایه اجازه می‌دهد.[6] اگرچه HMMها اقتضا می‌کنند حالت متغیرِ گسستهٔ تکی باشد، قیدِ متناظری برای متغیرهای شاهد وجود ندارد. این به این دلیل است که متغیرهای شاهد همیشه مشاهده می‌شوند، بدان معنا که نیازی به پیگیریِ هیچ توزیعی روی مقادیرشان نیست. (اگر متغیری مشاهده نشود، به‌سادگی می‌توان آن را برای آن گامِ زمانی از مدل حذف کرد.) متغیرهای شاهدِ بسیاری می‌توانند باشند، هم گسسته هم پیوسته.

خواننده‌ای که با عملیاتِ پایهٔ بردارها و ماتریس‌ها آشنایی ندارد شاید مایل باشد پیش از ادامهٔ این بخش به پیوستِ الف مراجعه کند.

۱۴.۳.۱ الگوریتم‌های ماتریسیِ ساده‌شده

با متغیرِ حالتِ گسستهٔ تکیِ Xt، می‌توانیم شکلِ عینی به بازنمایی‌های مدلِ انتقال، مدلِ حسگر و پیام‌های پیشرو و عقب‌رو بدهیم. فرض کنیم متغیرِ حالتِ Xt مقادیری دارد که با اعدادِ صحیحِ 1,...,S نشان داده می‌شوند، که در آن S شمارِ حالت‌های ممکن است.

مدلِ انتقالِ P(Xt | Xt-1) ماتریسِ S×S به نام T می‌شود، که در آن Tij = P(Xt = j | Xt-1 = i). یعنی Tij احتمالِ انتقال از حالتِ i به حالتِ j است. برای نمونه، اگر حالت‌های Rain=true و Rain=false را به‌ترتیب 1 و 2 شماره‌گذاری کنیم، آنگاه ماتریسِ انتقال برای جهانِ چترِ تعریف‌شده در شکل ۱۴.۲ چنین است:

```
T = P(Xt | Xt−1) = [ 0.7  0.3 ]
                   [ 0.7  0.3 ]
```

مدلِ حسگر را نیز به شکلِ ماتریسی در می‌آوریم. در این مورد، چون مقدارِ متغیرِ شاهدِ Et در زمانِ t معلوم است (آن را et بنامید)، فقط باید برای هر حالت مشخص کنیم چقدر محتمل است حالتِ et را پدید آورد: برای هر حالتِ i به P(et | Xt = i) نیاز داریم. برای راحتیِ ریاضی، این مقادیر را در «ماتریسِ مشاهدهٔ» قطریِ S×S (Observation Matrix) به نام Ot، یکی برای هر گامِ زمانی، می‌گذاریم. درایهٔ i-امِ قطرِ Ot برابر P(et | Xt = i) و بقیهٔ درایه‌ها صفرند. برای نمونه، در روزِ 1 در جهانِ چترِ شکل ۱۴.۵، U1 = true، و در روزِ 3، U3 = false، پس داریم:

```
O1 = [ 0.9  0 ]        O3 = [ 0.1  0 ]
[ 0.8   0 ]           [ 0.2   0 ]
```

حالا، اگر از بردارهای ستونی برای بازنماییِ پیام‌های پیشرو و عقب‌رو استفاده کنیم، همهٔ محاسبه‌ها عملیاتِ سادهٔ ماتریس–بردار می‌شوند. معادلهٔ پیشروِ (14.5) می‌شود:

```
f1:t+1 = α Ot+1 Tᵀ f1:t        (14.12)
```

و معادلهٔ عقب‌روِ (14.9) می‌شود:

```
bk+1:t = T Ok+1 bk+2:t        (14.13)
```

از این معادله‌ها می‌توانیم ببینیم پیچیدگیِ زمانیِ الگوریتمِ پیشرو–عقب‌رو (شکل ۱۴.۴) که روی توالی‌ای به طولِ t اعمال شود O(S²t) است، زیرا هر گام نیاز به ضربِ بردارِ S-عنصری در ماتریسِ S×S دارد. نیازِ فضایی O(St) است، زیرا گذرِ پیشرو t بردارِ اندازهٔ S را ذخیره می‌کند.

علاوه بر فراهم‌کردنِ توصیفی زیبا از الگوریتم‌های فیلترکردن و هموارسازی برای HMMها، صورت‌بندیِ ماتریسی فرصت‌هایی برای الگوریتم‌های بهبودیافته آشکار می‌کند. نخستین، تغییرِ ساده‌ای بر الگوریتمِ پیشرو–عقب‌رو است که اجازه می‌دهد هموارسازی در فضایِ ثابت، مستقل از طولِ توالی، انجام شود. ایده این است که هموارسازی برای هر برشِ زمانیِ خاصِ k به حضورِ هم‌زمانِ هر دو پیامِ پیشرو و عقب‌رو، f1:k و bk+1:t، طبقِ معادلهٔ (14.8) نیاز دارد. الگوریتمِ پیشرو–عقب‌رو این را با ذخیره‌کردنِ fهای محاسبه‌شده در گذرِ پیشرو به دست می‌آورد تا در طولِ گذرِ عقب‌رو در دسترس باشند. راهِ دیگرِ رسیدن به این، با گذرِ تکی است که هم f و هم b را در همان جهت منتشر می‌کند. برای نمونه، پیامِ «پیشروِ» f را می‌توان به عقب منتشر کرد اگر معادلهٔ (14.12) را طوری دست‌کاری کنیم که در جهتِ دیگر کار کند:

```
f1:t = α′ (Tᵀ)⁻¹ O⁻¹_{t+1} f1:t+1
```

الگوریتمِ هموارسازیِ اصلاح‌شده چنین کار می‌کند: نخست گذرِ استانداردِ پیشرو را برای محاسبهٔ ft:t اجرا می‌کند (با فراموش‌کردنِ پیام‌های ذخیره‌شده)؛ سپس گذرِ عقب‌رو را اجرا می‌کند و در هر گام پیامِ پیشروِ لازم را با «بازگشتِ پیشرو» محاسبه می‌کند. بدین‌ترتیب، هموارسازیِ فضایِ ثابت در $O(S^2t)$ زمان انجام می‌شود. (این الگوریتم در تمرین ISLE.۱۴ توضیح داده شده.) این الگوریتم دو قیدِ قابلِ‌توجه دارد: لازم است ماتریسِ انتقال وارون‌پذیر باشد و مدلِ حسگر هیچ صفری نداشته باشد—یعنی هر مشاهده‌ای در هر حالتی ممکن باشد. با این قیدها، نیازهای ذخیره‌سازی ثابت‌اند (یعنی مستقل از t، طولِ توالی).

حوزهٔ دومی که صورت‌بندیِ ماتریسی در آن بهبودی آشکار می‌کند، هموارسازیِ برخط با تأخیرِ ثابت است. این واقعیت که هموارسازی را می‌توان در فضایِ ثابت انجام داد پیشنهاد می‌کند الگوریتمِ بازگشتیِ کارآمدی برای هموارسازیِ برخط باید وجود داشته باشد—یعنی الگوریتمی که پیچیدگیِ زمانی‌اش مستقل از طولِ تأخیر باشد. فرض کنیم تأخیر d است؛ یعنی در برشِ زمانیِ t-d هموارسازی می‌کنیم، که در آن زمانِ کنونی t است. با معادلهٔ (14.8)، باید $\alpha \cdot f_{1:t-d} \times$ $bt_{-d+1:t}$ را برای برشِ t-d محاسبه کنیم. سپس، وقتی مشاهدهٔ تازه‌ای می‌رسد، باید $bt_{-d+2:t+1} \times \alpha \cdot f_{1:t-d+1}$ را برای برشِ t-d+1 محاسبه کنیم. چگونه این را به‌صورتِ افزایشی انجام داد؟ نخست، می‌توانیم $f_{1:t-d+1}$ را از $f_{1:t-d}$ با استفاده از فرایندِ استانداردِ فیلترکردن، معادلهٔ (14.5)، محاسبه کنیم.

محاسبهٔ افزایشیِ پیامِ عقب‌رو دشوارتر است، زیرا رابطهٔ ساده‌ای میان پیامِ عقب‌رویِ قدیمیِ $bt_{-d+1:t}$ و پیامِ عقب‌رویِ تازهٔ $bt_{-d+2:t+1}$ نیست. در عوض، رابطهٔ میان پیامِ عقب‌رویِ قدیمیِ $bt_{-d+1:t}$ و پیامِ عقب‌رویِ جلویِ توالی، $bt_{+1:t}$ را بررسی می‌کنیم. برای این کار، معادلهٔ (14.13) را d بار اعمال می‌کنیم:

$$bt_{-d+1:t} = \left(\prod_{i=t-d+1..t} T \cdot O_i\right) bt_{+1:t} = B_{t-d+1:t} \cdot 1 \qquad (14.14)$$

که در آن ماتریسِ $B_{t-d+1:t}$ حاصلِ‌ضربِ توالیِ ماتریس‌های T و O است و 1 بردارِ 1هاست. B را می‌توان «عملگرِ تبدیل» دانست که پیامِ عقب‌رویِ دیرتر را به زودتر تبدیل می‌کند. معادلهٔ مشابهی برای پیام‌های عقب‌رویِ تازه پس از رسیدنِ مشاهدهٔ بعدی برقرار است:

$$bt_{-d+2:t+1} = \left(\prod_{i=t-d+2..t+1} T \cdot O_i\right) bt_{+2:t+1} = B_{t-d+2:t+1} \cdot 1 \qquad (14.15)$$

با بررسیِ عبارت‌های حاصل‌ضرب در معادله‌های (14.14) و (14.15)، می‌بینیم رابطهٔ ساده‌ای دارند: برای به‌دست‌آوردنِ حاصلِ‌ضربِ دوم، حاصلِ‌ضربِ نخست را بر عنصرِ نخستِ $T \cdot O_{t-d+1}$ «تقسیم» کن و در عنصرِ آخرِ $T \cdot O_{t+1}$ ضرب کن. اگر این دو عنصر وارون‌پذیر باشند، پس رابطهٔ ساده‌ای میان ماتریس‌های B قدیمی و تازه هست:

$$B_{t-d+2:t+1} = O^{-1}_{t-d+1} \, T^{-1} \, B_{t-d+1:t} \, T \, O_{t+1} \qquad (14.16)$$

این معادله به‌روزرسانیِ افزایشی‌ای فراهم می‌کند و، همراه با معادلهٔ (14.5)، امکانِ هموارسازیِ برخط با زمانِ ثابتِ هر به‌روزرسانی، مستقل از تأخیرِ d را می‌دهد.

```
function FIXED-LAG-SMOOTHING(et, hmm, d) returns a distribution over Xt−d
                          inputs: et, the current evidence for time step t
                hmm, a hidden Markov model with S × S transition matrix T
                             d, the length of the lag for smoothing
                               persistent: t, the current time, initially 1
            f, the forward message P(Xt | e1:t), initially hmm.PRIOR
B, the d-step backward transformation matrix, initially the identity matrix
     et−d:t, double-ended list of evidence from t − d to t, initially empty
   local variables: Ot−d, Ot, diagonal matrices containing the sensor model information

                                                    add et to the end of et−d:t
                                          Ot ← diagonal matrix containing P(et | Xt)
                                                                 if t > d then
                                                      f ← FORWARD(f, et−d)
                                          remove et−d−1 from the beginning of et−d:t
                                      Ot−d ← diagonal matrix containing P(et−d | Xt−d)
                                                  B ← Ot−d⁻¹ T⁻¹ B T Ot
                                                           else B ← B T Ot
                                                                 t ← t + 1
                          if t > d + 1 then return NORMALIZE(f × B 1) else return null
```

شکل ۱۴.۶ — الگوریتمی برای هموارسازی با تأخیرِ زمانیِ ثابت به اندازهٔ d گام، پیاده‌سازی‌شده به‌صورتِ الگوریتمی برخط که برآوردِ هموارشدهٔ تازه را با توجه به مشاهده برای گامِ زمانیِ جدید خروجی می‌دهد. توجه کنید که خروجیِ نهاییِ NORMALIZE(f × B 1) طبقِ معادلهٔ (14.14) همان α f × b است.

حالا به نمونهٔ کاربردیِ HMM می‌پردازیم: «بومی‌سازی» (Localization). بخشِ ۴.۴.۱ نسخه‌ای از مسئلهٔ بومی‌سازی را توصیف کرد: رباتِ جاروبرقی‌ای که در شبکهٔ ساده‌ای از مکان‌ها حرکت می‌کند. در آن نسخه، ربات کنشِ Moveیِ ناجبرگرایانهٔ تکی داشت و حسگرهایش به‌طورِ کامل گزارش می‌دادند آیا موانع بلافاصله در شمال، جنوب، شرق و غرب‌اند؛ حالتِ باورِ ربات مجموعهٔ مکان‌های ممکنی بود که می‌توانست در آن باشد.

۱۴.۳.۲ نمونهٔ مدلِ مارکوفِ پنهان: بومی‌سازی

در این‌جا مسئله را با اجازه‌دادن به نویز در حسگرها کمی واقع‌گرایانه‌تر می‌کنیم، و این ایده را صوری می‌کنیم که ربات به‌صورتِ تصادفی حرکت می‌کند—به‌طورِ مساوی محتمل است به هر مربعِ خالیِ مجاور برود. متغیرِ حالتِ Xt مکانِ ربات روی شبکهٔ گسسته را بازنمایی می‌کند؛ دامنهٔ این متغیر مجموعهٔ مربع‌های خالی است، که با اعدادِ صحیحِ {1,...,S} برچسب‌گذاری می‌کنیم. فرض کنیم NEIGHBORS(i) مجموعهٔ مربع‌های خالیِ مجاورِ i باشد و N(i) اندازهٔ آن مجموعه. سپس مدلِ انتقال برای کنشِ Move می‌گوید ربات به‌طورِ مساوی محتمل است به هر مربعِ همسایه برسد:

```
P(Xt+1 = j | Xt = i) = Tij = { 1/N(i)   اگر j ∈ NEIGHBORS(i)
                             { 0        در غیرِ این صورت
```

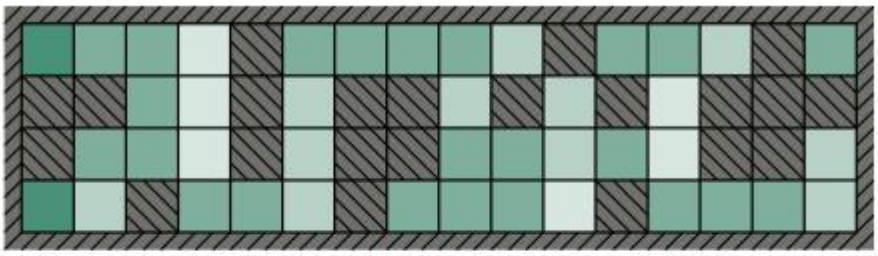
نمی‌دانیم ربات از کجا شروع می‌کند، پس توزیعِ یکنواختی روی همهٔ مربع‌ها فرض می‌کنیم؛ یعنی P(X0 = i) = 1/S. برای محیطِ خاصِ موردِ بررسی (شکل ۱۴.۷)، S=42 و ماتریسِ انتقالِ T دارای 42×42=1764 درایه است.

متغیرِ حسگرِ Et دارای 16 مقدارِ ممکن است، که هر یک توالیِ چهاربیتی است که حضور یا غیابِ مانع را در هر یک از جهت‌های قطبیِ NESW می‌دهد. برای نمونه، 1010 بدان معناست حسگرهای شمال و جنوب مانع گزارش می‌دهند و شرق و غرب نه. فرض کنیم نرخِ خطای هر حسگر ε است و خطاها برای چهار جهتِ حسگر به‌صورتِ مستقل رخ می‌دهند. در آن صورت، احتمالِ درست‌بودنِ هر چهار بیت $(1-\varepsilon)^4$ و احتمالِ اشتباه‌بودنِ همهٔ آن‌ها ε^4 است. علاوه بر این، اگر dit اختلاف—شمارِ بیت‌های متفاوت—میانِ مقادیرِ واقعیِ مربعِ i و خوانشِ واقعیِ et باشد، آنگاه احتمالِ اینکه ربات در مربعِ i خوانشِ حسگرِ et را دریافت کند چنین است:

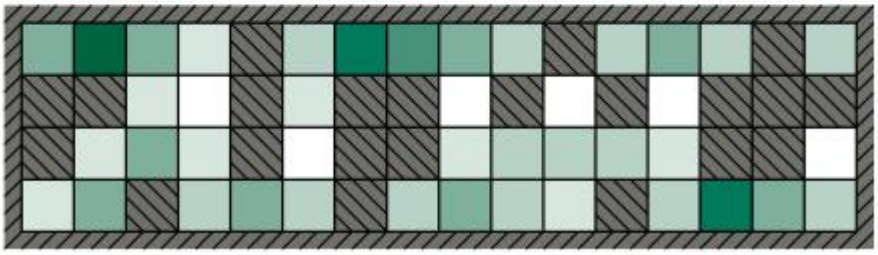
```
P(Et = et | Xt = i) = (Ot)ii = (1-ε)^(4-dit) · ε^dit
```

برای نمونه، احتمالِ اینکه مربعی با موانع در شمال و جنوب خوانشِ حسگرِ 1110 تولید کند $(1-\varepsilon)^1\cdot\varepsilon^3$ است.

با ماتریس‌های T و Ot، ربات می‌تواند از معادلهٔ (14.12) برای محاسبهٔ توزیعِ پسینی روی مکان‌ها—یعنی برای فهمیدنِ جایی که هست—استفاده کند. شکل ۱۴.۷ توزیع‌های P(X1 | E1=1011) و P(X2 | E1=1011, E2=1010) را نشان می‌دهد. این همان هزارتویی است که قبلاً در شکل ۴.۱۸ (صفحهٔ ۱۳۴) دیدیم، اما آن‌جا از فیلترکردنِ منطقی برای یافتنِ مکان‌های ممکن، با فرضِ حسِ کامل، استفاده کردیم. همان مکان‌ها هنوز با حسِ نویزی محتمل‌ترین‌اند، اما حالا هر مکان مقداری احتمالِ غیرصفر دارد، زیرا هر مکانی می‌تواند هر مقادیرِ حسگری را تولید کند.



(a) Posterior distribution over robot location after $E_1 = 1011$

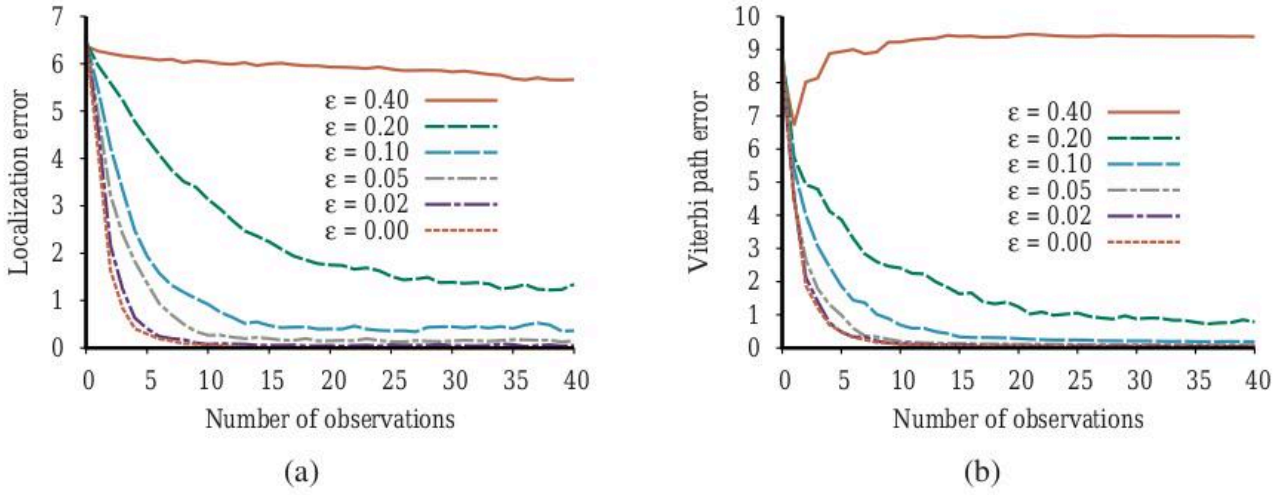


(b) Posterior distribution over robot location after $E_1 = 1011, E_2 = 1010$

شکل ۱۴.۷ – توزیعِ پسینی روی مکانِ ربات: (الف) پس از یک مشاهدهٔ E1=1011 (یعنی موانع در شمال، جنوب و غرب)؛ (ب) پس از یک حرکتِ تصادفی به مکانِ مجاور و مشاهدهٔ دومِ E2=1010 (یعنی موانع در شمال و جنوب). اندازهٔ هر دیسک با احتمالِ قرارداشتنِ ربات در آن مکان متناسب است. نرخِ خطایِ حسگر برای هر بیت ε=0.2 است.

علاوه بر فیلترکردن برای برآوردِ مکانِ کنونی، ربات می‌تواند از هموارسازی (معادلهٔ (14.13)) برای فهمیدنِ جایی که در هر زمانِ گذشته بوده—برای نمونه، از کجا در زمانِ 0 شروع کرده—استفاده کند، و می‌تواند از الگوریتمِ ویتربی برای یافتنِ محتمل‌ترین مسیری که طی کرده تا به‌جایی که حالا هست برسد، استفاده کند. شکل ۱۴.۸ خطایِ بومی‌سازی و خطایِ مسیرِ ویتربی را برای مقادیرِ مختلفِ نرخِ خطایِ حسگرِ هر بیتِ ε نشان می‌دهد. حتی وقتی ε برابر 0.20 است—که بدان

معناست خوانشِ کلیِ حسگر ۵۹٪ مواقع اشتباه است—ربات معمولاً قادر است پس از ۲۰ مشاهده مکانش را تا دو مربع تعیین کند. این به دلیلِ توانایی الگوریتم در یکپارچه‌کردنِ شواهد در طولِ زمان و در نظرگرفتنِ قیدهای احتمالاتیِ تحمیل‌شده بر توالیِ مکان توسط مدلِ انتقال است. وقتی ε برابر 0.10 یا کمتر است، ربات فقط به چند مشاهده نیاز دارد تا بفهمد کجاست و موقعیتش را دقیقاً ردیابی کند. وقتی ε برابر 0.40 است، هم خطایِ بومی‌سازی و هم خطایِ مسیرِ ویتربی بزرگ می‌مانند؛ به بیانِ دیگر، ربات گم شده. این به این دلیل است که حسگری با احتمالِ خطایِ 0.40 اطلاعاتِ بسیار کمی فراهم می‌کند تا از اطلاعاتِ ازدست‌رفته دربارهٔ موقعیتِ ربات که از حرکتِ تصادفیِ غیرقابلِ‌پیش‌بینی می‌آید بکاهد.



شکل ۱۴.۸ — عملکردِ بومی‌سازیِ HMM به‌عنوانِ تابعی از طولِ توالیِ مشاهده برای مقادیرِ مختلفِ احتمالِ خطایِ حسگرِ ε؛ داده‌ها روی ۴۰۰ اجرا میانگین‌گیری شده‌اند. (الف) خطایِ بومی‌سازی، تعریف‌شده به‌عنوانِ فاصلهٔ منهتن از مکانِ واقعی. (ب) خطایِ مسیرِ ویتربی، تعریف‌شده به‌عنوانِ میانگینِ فاصلهٔ منهتنِ حالت‌های روی مسیرِ ویتربی از حالت‌های متناظر روی مسیرِ واقعی.

متغیرِ حالت برای مثالی که در این بخش در نظر گرفتیم مکانِ فیزیکی در جهان است. مسائلِ دیگر، البته، می‌توانند جنبه‌های دیگرِ جهان را شامل شوند. تمرین ROOM.۱۴ از شما می‌خواهد نسخه‌ای از رباتِ جاروبرقی را در نظر بگیرید که سیاستِ رفتنِ مستقیم تا آن‌جا که می‌تواند را دارد؛ فقط وقتی به مانعی برخورد می‌کند، به سمتِ تازه‌ای تغییر جهت می‌دهد. برای مدل‌کردنِ این ربات، هر حالت در مدل از جفتِ (مکان، جهت) تشکیل می‌شود. برای محیطِ شکل ۱۴.۷، که ۴۲ مربعِ خالی دارد، این به ۱۶۸ حالت و ماتریسِ انتقالی با $168^2 = 28{,}224$ درایه می‌انجامد—هنوز عددِ قابلِ‌مدیریتی است. اگر امکانِ کثیفی در هر یک از ۴۲ مربع را بیفزاییم، شمارِ حالت‌ها در 2^{42} ضرب می‌شود و ماتریسِ انتقال بیش از 10^{29} درایه دارد—دیگر عددِ قابلِ‌مدیریتی نیست.

به‌طورِ کلی، اگر حالت از n متغیرِ گسسته تشکیل شود که هر یک حداکثر d مقدار دارد، ماتریسِ انتقالِ HMM متناظر اندازهٔ $O(d^{2n})$ خواهد داشت و زمانِ محاسبهٔ هر به‌روزرسانی نیز $O(d^{2n})$ خواهد بود. به این دلایل، اگرچه HMMها کاربردهای بسیاری در حوزه‌هایی از بازشناسیِ گفتار تا زیست‌شناسیِ مولکولی دارند، در توانایی بازنماییِ فرایندهای پیچیده اساساً محدودند. در اصطلاح‌شناسیِ معرفی‌شده در فصل ۲، HMMها بازنماییِ اتمی‌اند: حالت‌های جهان

ساختارِ درونی ندارند و صرفاً با اعدادِ صحیح برچسب‌گذاری می‌شوند. بخش ۱۴.۵ نشان می‌دهد چگونه از شبکه‌های بیزیِ پویا—بازنماییِ عاملی—برای مدل‌کردنِ حوزه‌های با متغیرهای حالتِ بسیار استفاده کنیم. بخشِ بعد نشان می‌دهد چگونه حوزه‌های با متغیرهای حالتِ پیوسته، که البته به فضایِ حالتِ نامتناهی می‌انجامند، را مدیریت کنیم.

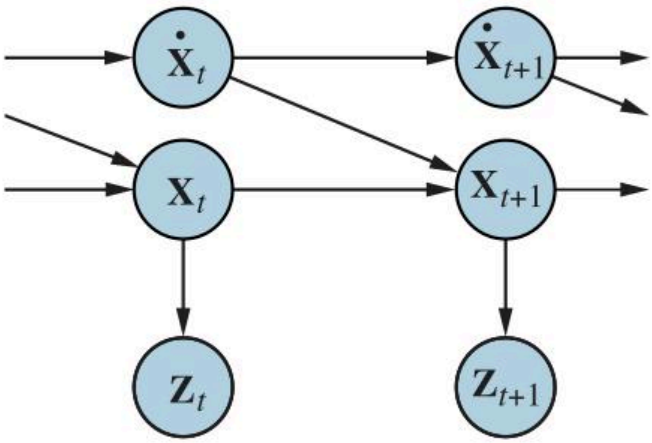
۱۴.۴ فیلترهای کالمن

تصور کنید پرندهٔ کوچکی را تماشا می‌کنید که در غروب از میانِ شاخ‌وبرگِ انبوهِ جنگل پرواز می‌کند: جرقه‌های کوتاه و ناپیوسته‌ای از حرکت می‌بینید؛ به‌سختی تلاش می‌کنید حدس بزنید پرنده کجاست و بعد کجا ظاهر خواهد شد تا گمش نکنید. یا تصور کنید اپراتورِ رادارِ جنگِ جهانیِ دومی هستید که به نقطهٔ نورانیِ محوِ سرگردانی خیره شده‌اید که هر ۱۰ ثانیه یک بار روی صفحه ظاهر می‌شود. یا، باز به عقب‌تر برویم، تصور کنید کپلر هستید که می‌کوشد حرکتِ سیاره‌ها را از مجموعه‌ای از مشاهده‌های زاویه‌ایِ بسیار نادقیق، که در بازه‌های نامنظم و با اندازه‌گیریِ نادقیق گرفته شده‌اند، بازسازی کند. در همهٔ این موارد، شما فیلترکردن انجام می‌دهید: برآوردِ متغیرهای حالت (در این‌جا، موقعیت و سرعتِ جسمِ متحرک) از مشاهده‌های نویزی در طولِ زمان. اگر متغیرها گسسته بودند، می‌توانستیم سامانه را با مدلِ مارکوفِ پنهان مدل کنیم. این بخش روش‌هایی برای مدیریتِ متغیرهای پیوسته بررسی می‌کند، با استفاده از الگوریتمی به نام «فیلترِ کالمن» (Kalman Filtering)، به نامِ یکی از مخترعانش رودولف کالمن.

پروازِ پرنده ممکن است با شش متغیرِ پیوسته در هر نقطهٔ زمانی مشخص شود؛ سه‌تا برای موقعیت (X_t, Y_t, Z_t) و سه‌تا برای سرعت $(\dot{X}_t, \dot{Y}_t, \dot{Z}_t)$. برای بازنماییِ مدل‌های انتقال و حسگر به چگالی‌های شرطیِ مناسبی نیاز خواهیم داشت؛ همانندِ فصل ۱۳، از توزیع‌های خطی–گاوسی استفاده می‌کنیم. این بدان معناست حالتِ بعدیِ X_{t+1} باید تابعی خطی از حالتِ کنونیِ X_t باشد، به‌علاوهٔ نویزِ گاوسی، شرطی که در عمل کاملاً معقول از آب درمی‌آید. برای نمونه، مختصهٔ X پرنده را در نظر بگیرید، و فعلاً سایر مختصه‌ها را نادیده بگیرید. فرض کنیم بازهٔ زمانی میان مشاهده‌ها Δ باشد و سرعت در طولِ بازه ثابت باشد؛ سپس به‌روزرسانیِ موقعیت به‌صورتِ $X_{t+\Delta} = X_t + \dot{X}\cdot\Delta$ است. با افزودنِ نویزِ گاوسی (برای حسابِ تغییراتِ باد و...)، مدلِ انتقالِ خطی–گاوسی به دست می‌آوریم:

$$P(X_{t+\Delta} = x_{t+\Delta} \mid X_t = x_t, \dot{X}_t = \dot{x}_t) = \mathcal{N}(x_{t+\Delta};\, x_t + \dot{x}_t\cdot\Delta,\, \sigma^2)$$

ساختارِ شبکهٔ بیزی برای سامانه‌ای با بردارِ موقعیتِ X_t و سرعتِ $\dot{X}_t$ در شکل ۱۴.۹ نشان داده شده. توجه کنید این شکلِ بسیار خاصی از مدلِ خطی–گاوسی است؛ شکلِ عمومی بعداً در این بخش توصیف خواهد شد و گسترهٔ عظیمی از کاربردها را فراتر از مثال‌های سادهٔ حرکتِ پاراگرافِ نخست پوشش می‌دهد. خواننده ممکن است بخواهد برای برخی از ویژگی‌های ریاضیِ توزیع‌های گاوسی به پیوستِ الف مراجعه کند؛ برای مقاصدِ فوریِ ما، مهم‌ترین ویژگی این است که توزیعِ گاوسیِ چندمتغیره برای d متغیر با میانگینِ d-عنصریِ $\boldsymbol{\mu}$ و ماتریسِ کوواریانسِ $d\times d$ به نام $\boldsymbol{\Sigma}$ مشخص می‌شود.



شکل ۱۴.۹ — ساختارِ شبکهٔ بیزی برای سامانهٔ پویای خطی با موقعیتِ Xt، سرعتِ Ẋt و اندازه‌گیریِ موقعیتِ Zt.

۱۴.۴.۱ به‌روزرسانیِ توزیع‌های گاوسی

در فصل ۱۳ صفحهٔ ۴۲۳، به ویژگیِ کلیدیِ خانوادهٔ توزیع‌های خطی–گاوسی اشاره کردیم: تحتِ به‌روزرسانیِ بیزی بسته می‌ماند. (یعنی با هر شاهدی، پسین همچنان در خانوادهٔ خطی–گاوسی است.) در این‌جا این ادعا را در بافتِ فیلترکردن در مدلِ احتمالِ زمانی دقیق می‌کنیم. ویژگی‌های لازم با محاسبهٔ دوقسمتیِ فیلترکردن در معادلهٔ (14.5) متناظرند:

1. اگر توزیعِ کنونیِ P(Xt | e1:t) گاوسی و مدلِ انتقالِ P(Xt+1 | xt) خطی–گاوسی باشد، آنگاه توزیعِ پیش‌بینی‌شدهٔ تک‌گامیِ داده‌شده توسط:

$$P(X_{t+1} \mid e_{1:t}) = \int_{x_t} P(X_{t+1} \mid x_t)\, P(x_t \mid e_{1:t})\, dx_t \qquad (14.17)$$

نیز توزیعِ گاوسی است.

1. اگر پیش‌بینیِ P(Xt+1 | e1:t) گاوسی و مدلِ حسگرِ P(et+1 | Xt+1) خطی–گاوسی باشد، آنگاه، پس از شرطی‌کردن بر شاهدِ تازه، توزیعِ به‌روزرسانی‌شدهٔ:

$$P(X_{t+1} \mid e_{1:t+1}) = \alpha\, P(e_{t+1} \mid X_{t+1})\, P(X_{t+1} \mid e_{1:t}) \qquad (14.18)$$

نیز توزیعِ گاوسی است.

بدین‌ترتیب، عملگرِ FORWARD برای فیلترِ کالمن، پیامِ پیشرویِ گاوسیِ $f_{1:t}$ را، که با میانگینِ μ_t و کوواریانسِ Σ_t مشخص می‌شود، می‌گیرد و پیامِ پیشرویِ گاوسیِ چندمتغیرهٔ تازه‌ای $f_{1:t+1}$ تولید می‌کند، که با میانگینِ μ_{t+1} و کوواریانسِ Σ_{t+1} مشخص می‌شود. پس اگر با پیشینِ گاوسیِ $P(X_0) = N(\mu_0, \Sigma_0)$ = $f_{1:0}$ شروع کنیم، فیلترکردن با مدلِ خطی–گاوسی برای همهٔ زمان‌ها توزیعِ حالتِ گاوسی تولید می‌کند.

این نتیجهٔ زیبا و ظریفی به نظر می‌رسد، اما چرا این‌قدر مهم است؟ دلیلش این است که به‌جزِ چند حالتِ خاص مانند این، فیلترکردن با شبکه‌های پیوسته یا ترکیبی (گسسته و پیوسته) توزیع‌های حالتی تولید می‌کند که بازنمایی‌شان در طولِ زمان بدونِ کران رشد می‌کند. این گزاره به‌طورِ کلی به‌آسانی اثبات‌پذیر نیست، اما تمرین KFSW.۱۴ نشان می‌دهد برای مثالِ ساده‌ای چه اتفاقی می‌افتد.

۱۴.۴.۲ مثالِ سادهٔ تک‌بعدی

گفته‌ایم عملگرِ FORWARD برای فیلترِ کالمن گاوسی را به گاوسیِ تازه نگاشت می‌کند. این به محاسبهٔ میانگین و کوواریانسِ تازه از میانگین و کوواریانسِ قبلی ترجمه می‌شود. استخراجِ قاعدهٔ به‌روزرسانی در حالتِ عمومی (چندمتغیره) به جبرِ خطیِ نسبتاً زیادی نیاز دارد، پس فعلاً به حالتِ تک‌متغیرهٔ بسیار ساده‌ای می‌چسبیم و بعداً نتایجِ حالتِ عمومی را می‌دهیم. حتی برای حالتِ تک‌متغیره، محاسبه‌ها تا حدی خسته‌کننده‌اند، اما احساس می‌کنیم ارزش دیدن دارند، زیرا سودمندیِ فیلترِ کالمن چنان با ویژگی‌های ریاضیِ توزیع‌های گاوسی پیوندِ نزدیک دارد.

مدلِ زمانی‌ای که در نظر می‌گیریم «پیاده‌رویِ تصادفیِ» متغیرِ حالتِ پیوستهٔ تکیِ Xt با مشاهدهٔ نویزیِ Zt را توصیف می‌کند. نمونه‌ای ممکن است شاخصِ «اعتمادِ مصرف‌کننده» باشد، که می‌توان مدل کرد هر ماه تغییرِ تصادفیِ گاوسیِ توزیع‌شده‌ای تجربه می‌کند و با نظرسنجیِ تصادفیِ مصرف‌کننده سنجیده می‌شود که آن نیز نویزِ نمونه‌برداریِ گاوسی وارد می‌کند.

توزیعِ پیشینی گاوسی با واریانسِ σ_0^2 فرض می‌شود:

$$P(x0) = \alpha\, e^{-(1/2)\cdot((x0 - \mu0)^2 / \sigma_0^2)}$$

(برای سادگی، در این بخش از همان نمادِ α برای همهٔ ثابت‌های نرمال‌سازی استفاده می‌کنیم.) مدلِ انتقال، اغتشاشِ گاوسیِ با واریانسِ ثابتِ σx^2 به حالتِ کنونی می‌افزاید:

$$P(xt+1 \mid xt) = \alpha\, e^{-(1/2)\cdot((xt+1 - xt)^2 / \sigma x^2)}$$

مدلِ حسگر نویزِ گاوسی با واریانسِ σz^2 را فرض می‌کند:

$$P(zt \mid xt) = \alpha\, e^{-(1/2)\cdot((zt - xt)^2 / \sigma z^2)}$$

حالا، با توجه به پیشینِ P(X0)، توزیعِ پیش‌بینی‌شدهٔ تک‌گامی از معادلهٔ (14.17) می‌آید:

$$\begin{aligned} P(x1) &= \int P(x1 \mid x0)\, P(x0)\, dx0 \\ &\alpha \int e^{-(1/2)\cdot((x1 - x0)^2 / \sigma x^2)} \cdot e^{-(1/2)\cdot((x0 - \mu0)^2 / \sigma_0^2)}\, dx0 = \\ &\alpha \int e^{-(1/2)\cdot((\sigma_0^2 (x1-x0)^2 + \sigma x^2 (x0-\mu0)^2) / (\sigma_0^2 \sigma x^2))}\, dx0 = \end{aligned}$$

این انتگرال نسبتاً پیچیده به نظر می‌رسد. کلیدِ پیشرفت توجه به این است که توان، جمعِ دو عبارتِ درجهٔ دومی در x0 است و از این رو خودِ آن درجهٔ دومی در x0 است. حقهٔ ساده‌ای به نام «تکمیلِ مربع» (Completing the Square) اجازه می‌دهد هر درجهٔ دومیِ $ax_0^2 + bx0 + c$ را به‌صورتِ جمعِ جملهٔ مربعیِ $a(x0 - (-b/2a))^2$ و جملهٔ باقی‌ماندهٔ $c - b^2/4a$ که مستقل از x0 است بازنویسی کنیم. در این مورد، داریم:

```
a = (σ₀² + σx²) / (σ₀²σx²)
b = −2(σ₀²x1 + σx²μ0) / (σ₀²σx²)
c = (σ₀²x1² + σx²μ0²) / (σ₀²σx²)
```

جملهٔ باقی‌مانده را می‌توان از انتگرال بیرون برد:

```
P(x1) = α e^(−(1/2)·(c − b²/4a)) ∫ e^(−(1/2)·a(x0 − (−b/2a))²) dx0
```

حالا انتگرال فقط انتگرالِ گاوسی روی کلِ دامنه‌اش است، که صرفاً ۱ است. بدین‌ترتیب، فقط با جملهٔ باقی‌ماندهٔ درجهٔ دومی رها می‌شویم. با جای‌گذاریِ دوبارهٔ عبارت‌های a، b و c و ساده‌سازی، به دست می‌آوریم:

```
P(x1) = α e^(−(1/2)·((x1 − μ0)² / (σ₀² + σx²)))
```

یعنی توزیعِ پیش‌بینی‌شدهٔ تک‌گامی گاوسی با همان میانگینِ μ0 و واریانسی برابرِ جمعِ واریانسِ اصلیِ σ_0^2 و واریانسِ انتقالِ σx^2 است. برای کامل‌کردنِ گامِ به‌روزرسانی، باید بر مشاهده در نخستین گامِ زمانی، یعنی z1، شرطی کنیم. از معادلهٔ (14.18)، این چنین داده می‌شود:

```
P(x1 | z1) = α P(z1 | x1) P(x1) = α e^(−(1/2)·((z1 − x1)² / σz²)) · e^(−(1/2)·((x1 − μ0)²
                                                                / (σ₀² + σx²)))
```

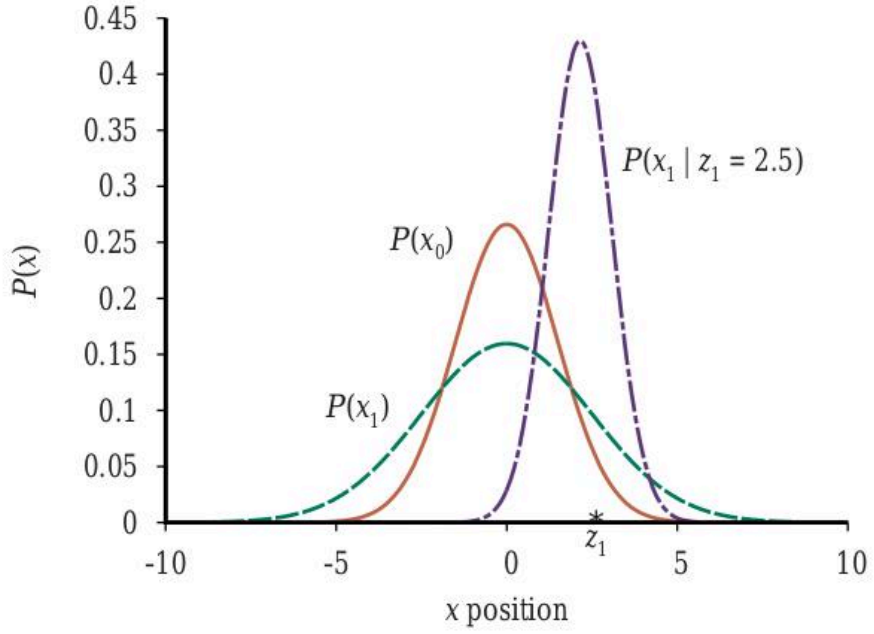
باز، توان‌ها را ترکیب و مربع را تکمیل می‌کنیم (تمرین KALM.۱۴) و عبارتِ زیر را برای پسین به دست می‌آوریم:

```
P(x1 | z1) = α e^( −(1/2) · (x1 − ((σ₀²+σx²)z1 + σz²μ0) / (σ₀²+σx²+σz²))² / ((σ₀²+σx²)σz²
                                                                / (σ₀²+σx²+σz²)) )        (14.19)
```

بدین‌ترتیب، پس از یک چرخهٔ به‌روزرسانی، توزیعِ گاوسیِ تازه‌ای برای متغیرِ حالت داریم.

از فرمولِ گاوسی در معادلهٔ (14.19)، می‌بینیم میانگین و انحرافِ معیارِ تازه را می‌توان از میانگین و انحرافِ معیارِ قبلی چنین محاسبه کرد:

```
                                                μt+1 = ((σt² + σx²)zt+1 + σz²μt) / (σt² + σx² + σz²)
σ²_{t+1} = ((σt² + σx²)σz²) / (σt² + σx² + σz²)                                        (14.20)
```



شکل ۱۴.۱۰ — مراحلِ چرخهٔ به‌روزرسانیِ فیلترِ کالمن برای پیاده‌رویِ تصادفی با پیشینی که μ0=0.0 و σ0=1.5، نویزِ انتقالِ σx=2.0، نویزِ حسگرِ σz=1.0 و نخستین مشاهدهٔ z1=2.5 (روی محورِ x علامت‌گذاری شده) داده می‌شود. توجه کنید چگونه پیش‌بینیِ P(x1) نسبت به P(x0) توسط نویزِ انتقال پهن و تخت می‌شود. همچنین توجه کنید میانگینِ توزیعِ پسینیِ P(x1|z1) اندکی چپِ مشاهدهٔ z1 است، زیرا میانگینِ میانگینِ وزنیِ پیش‌بینی و مشاهده است.

شکل ۱۴.۱۰ یک چرخهٔ به‌روزرسانیِ فیلترِ کالمن را در حالتِ یک‌بُعدی برای مقادیرِ خاصی از مدل‌های انتقال و حسگر نشان می‌دهد.

معادلهٔ (14.20) دقیقاً همان نقشی را ایفا می‌کند که معادلهٔ فیلترکردنِ عمومیِ (14.5) یا معادلهٔ فیلترکردنِ HMM در (14.12) دارد. با این حال، به دلیلِ ماهیتِ ویژهٔ توزیع‌های گاوسی، این معادله‌ها ویژگی‌های جالبِ توجهِ اضافی دارند:

نخست، می‌توانیم محاسبهٔ میانگینِ جدیدِ μt+1 را به‌عنوانِ میانگینِ وزنیِ مشاهدهٔ جدیدِ zt+1 و میانگینِ قبلیِ μt تفسیر کنیم. اگر مشاهده غیرقابلِ‌اعتماد باشد، آنگاه σz² بزرگ است و ما توجهِ بیشتری به میانگینِ قبلی می‌کنیم؛ اگر میانگینِ قبلی غیرقابلِ‌اعتماد باشد (σt² بزرگ باشد) یا فرایند بسیار غیرقابلِ‌پیش‌بینی باشد (σx² بزرگ باشد)، توجهِ بیشتری به مشاهده معطوف می‌کنیم. در حالتِ حدی، اگر مشاهده نویزِ صفر داشته باشد، آنگاه μt+1 = zt+1 خواهد بود، و اگر مشاهده نویزِ محض باشد (واریانسِ نامتناهی)، آنگاه μt+1 = μt می‌شود.

دوم، توجه کنید به‌روزرسانیِ واریانسِ σ²_{t+1} مستقل از مشاهده است. بنابراین می‌توانیم از پیش توالیِ مقادیرِ واریانسی را که رخ خواهند داد محاسبه کنیم. سوم، توالیِ مقادیرِ واریانس به‌سرعت به مقدارِ ثابتی همگرا می‌شود که فقط به σx² و σz² بستگی دارد، و بدین‌ترتیب محاسباتِ بعدی را به‌طورِ قابلِ‌توجهی ساده می‌کند. (تمرین VARI.۱۴ را ببینید.)

۱۴.۴.۳ حالتِ عمومی

اشتقاقِ پیشین ویژگیِ کلیدیِ توزیع‌های گاوسی را نشان می‌دهد که فیلترِ کالمن را ممکن می‌کند: این واقعیت که توان «شکلِ درجهٔ دومی» (Quadratic Form) است. این فقط برای حالتِ تک‌متغیره صادق نیست؛ توزیعِ گاوسیِ چندمتغیرهٔ کامل شکلِ:

```
N(x; μ, Σ) = α e^(−(1/2)·(x − μ)ᵀ Σ⁻¹ (x − μ))
```

دارد. با ضربِ جمله‌های توان، می‌بینیم توان نیز تابعِ درجهٔ دومیِ مقادیرِ xi در x است. بدین‌ترتیب، فیلترکردن ماهیتِ گاوسیِ توزیعِ حالت را حفظ می‌کند.

نخست مدلِ زمانیِ عمومیِ استفاده‌شده با فیلترِ کالمن را تعریف کنیم. هم مدلِ انتقال و هم مدلِ حسگر باید تبدیلِ خطی با نویزِ گاوسیِ افزایشی باشند. بدین‌ترتیب:

```
                    P(xt+1 | xt) = N(xt+1; F·xt, Σx)
P(zt | xt) = N(zt; H·xt, Σz)                    (14.21)
```

که در آن F و Σx ماتریس‌هایی‌اند که مدلِ انتقالِ خطی و کوواریانسِ نویزِ انتقال را توصیف می‌کنند، و H و Σz ماتریس‌های متناظر برای مدلِ حسگرند. حالا معادله‌های به‌روزرسانیِ میانگین و کوواریانس، در تمامِ جزئیاتِ ترسناک‌شان:

```
                        μt+1 = Fμt + Kt+1(zt+1 − H·Fμt)
Σt+1 = (I − Kt+1H)(FΣtFᵀ + Σx)                  (14.22)
```

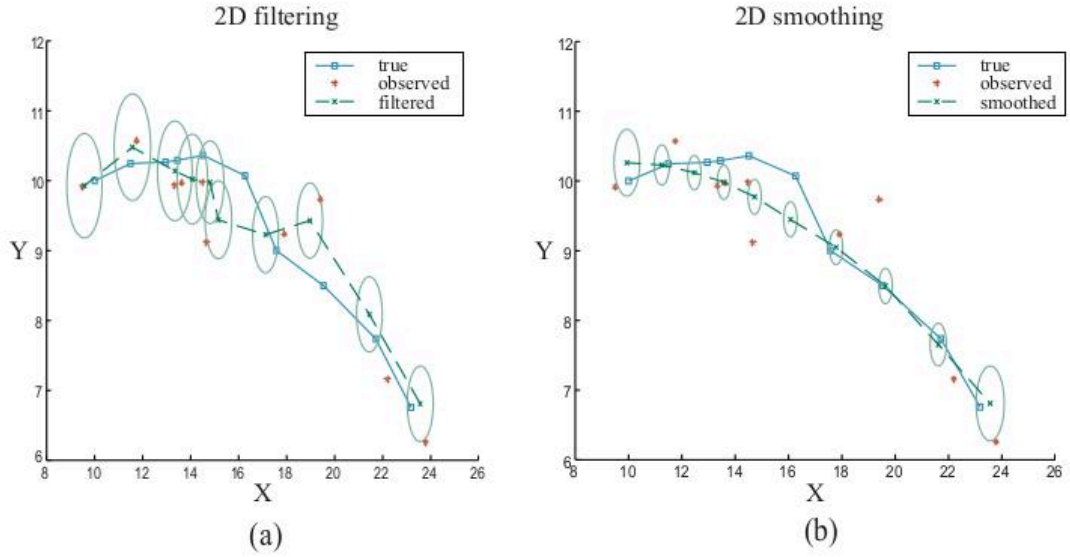
که در آن:

```
Kt+1 = (FΣtFᵀ + Σx)Hᵀ (H(FΣtFᵀ + Σx)Hᵀ + Σz)⁻¹
```

«ماتریسِ بهرهٔ کالمن» (Kalman Gain Matrix) است. باور کنید یا نه، این معادله‌ها تا حدی معنیِ شهودی دارند. برای نمونه، به‌روزرسانیِ برآوردِ میانگینِ حالتِ μ را در نظر بگیرید. جملهٔ Fμt حالتِ پیش‌بینی‌شده در t+1 است، پس H·Fμt مشاهدهٔ پیش‌بینی‌شده است. بنابراین، جملهٔ zt+1 - H·Fμt خطایِ مشاهدهٔ پیش‌بینی‌شده را بازنمایی می‌کند. این در Kt+1 ضرب می‌شود تا حالتِ پیش‌بینی‌شده تصحیح شود؛ از این رو، Kt+1 اندازه‌ای است از اینکه مشاهدهٔ تازه را چقدر جدی نسبت به پیش‌بینی بگیریم. همانندِ معادلهٔ (14.20)، همچنین این ویژگی را داریم که به‌روزرسانیِ واریانس مستقل از مشاهده‌هاست. توالیِ مقادیرِ Σt و Kt را بنابراین می‌توان آفلاین محاسبه کرد، و محاسباتِ واقعیِ لازم در طولِ ردیابیِ برخط بسیار اندک‌اند.

برای نشان‌دادنِ این معادله‌ها در کار، آن‌ها را روی مسئلهٔ ردیابیِ جسمی که در صفحهٔ X–Y حرکت می‌کند اعمال کرده‌ایم. متغیرهای حالت $X = (X, Y, \dot{X}, \dot{Y})^T$ هستند، پس F، Σx، H و Σz ماتریس‌های 4×4اند. شکل ۱۴.۱۱(الف) مسیرِ واقعی، مجموعه‌ای از مشاهده‌های نویزی و مسیرِ برآوردشده توسط فیلترِ کالمن را نشان می‌دهد، همراه با کوواریانس‌هایی که با کانتورهای یک‌انحراف‌معیاری نشان داده شده‌اند. فرایندِ فیلترکردن کارِ خوبی در ردیابیِ حرکتِ واقعی انجام می‌دهد و، چنان‌که انتظار می‌رود، واریانس به‌سرعت به نقطهٔ ثابتی می‌رسد.

همچنین می‌توانیم معادله‌هایی برای هموارسازی و نیز فیلترکردن با مدل‌های خطی–گاوسی استخراج کنیم. نتایجِ هموارسازی در شکل ۱۴.۱۱(ب) نشان داده شده. توجه کنید واریانس در برآوردِ موقعیت چگونه به‌شدت کاهش می‌یابد، به‌جز در انتهایِ مسیر (چرا؟)، و مسیرِ برآوردشده بسیار نرم‌تر است.



شکل ۱۴.۱۱ – (الف) نتایجِ فیلترِ کالمن برای جسمی که در صفحهٔ X–Y حرکت می‌کند، با نشان‌دادنِ مسیرِ واقعی (از چپ به راست)، مجموعه‌ای از مشاهده‌های نویزی و مسیرِ برآوردشده توسط فیلترِ کالمن. واریانس در برآوردِ موقعیت با بیضی‌ها نشان داده شده. (ب) نتایجِ هموارسازیِ کالمن برای همان توالیِ مشاهده.

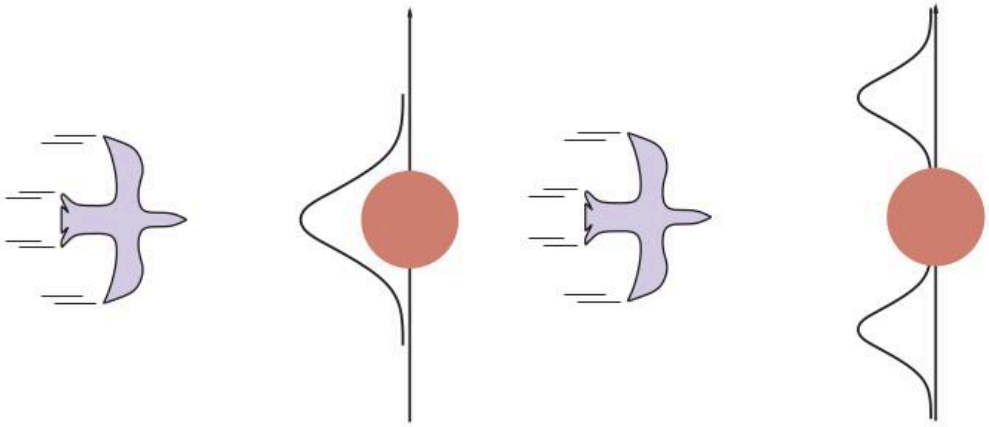
۱۴.۴.۴ کاربردپذیریِ فیلترِ کالمن

فیلترِ کالمن و بسط‌هایش در گسترهٔ عظیمی از کاربردها استفاده می‌شوند. کاربردِ «کلاسیک» ردیابیِ راداریِ هواپیماها و موشک‌هاست. کاربردهای مربوط شاملِ ردیابیِ صوتیِ زیردریایی‌ها و خودروهای زمینی و ردیابیِ دیداریِ خودروها و انسان‌هاست. در جهتی کمی عجیب‌تر، فیلترهای کالمن برای بازسازیِ مسیرِ ذرات از عکس‌های حباب‌خانه و جریان‌های اقیانوسی از اندازه‌گیری‌های سطحیِ ماهواره‌ای استفاده می‌شوند. گسترهٔ کاربرد بسیار بزرگ‌تر از صرفاً ردیابیِ حرکت است: هر سامانه‌ای که با متغیرهای حالتِ پیوسته و اندازه‌گیری‌های نویزی مشخص شود کار می‌کند. چنین سامانه‌هایی شاملِ کارخانه‌های خمیرکاغذ، کارخانه‌های شیمیایی، رآکتورهای هسته‌ای، بوم‌سازگان‌های گیاهی و اقتصادهای ملی‌اند.

این واقعیت که فیلترِ کالمن را می‌توان روی سامانه‌ای اعمال کرد به این معنا نیست که نتایج معتبر یا مفید خواهند بود. مفروضاتِ گرفته‌شده—مدل‌های انتقال و حسگرِ خطی–گاوسی—بسیار قوی‌اند. «فیلترِ کالمنِ بسط‌یافته» (Extended Kalman Filter یا EKF) می‌کوشد بر غیرخطی‌بودن‌های سامانهٔ درحالِ‌مدل‌شدن غلبه کند. سامانه غیرخطی (Nonlinear) است اگر مدلِ انتقال را نتوان به‌صورتِ ضربِ ماتریسیِ بردارِ حالت، مانندِ معادلهٔ (14.21)، توصیف کرد. EKF با مدل‌کردنِ سامانه به‌عنوانِ محلی خطی در xt در ناحیهٔ xt = μt، میانگینِ توزیعِ حالتِ کنونی، کار می‌کند. این برای سامانه‌های نرم و خوش‌رفتار خوب کار می‌کند و به ردیاب اجازه می‌دهد توزیعِ حالتِ گاوسیِ معقولی را که تقریبِ مناسبی از پسینِ واقعی است نگه دارد و به‌روزرسانی کند. مثالِ مفصلی در فصل ۲۶ داده شده.

معنای «نرم‌نبودن» یا «بدرفتاربودن» سامانه چیست؟ از نظرِ فنی، بدان معناست غیرخطی‌بودنِ قابلِ‌توجهی در پاسخِ سامانه درونِ ناحیه‌ای که طبقِ کوواریانسِ Σt به میانگینِ کنونیِ μt «نزدیک» است وجود دارد. برای فهمِ این ایده به زبانِ غیرفنی، مثالِ تلاش برای ردیابیِ پرنده‌ای را در نظر بگیرید که در جنگل پرواز می‌کند. به نظر می‌رسد پرنده با سرعتِ زیاد مستقیم به سمتِ تنهٔ درختی می‌رود. فیلترِ کالمن، خواه معمولی خواه بسط‌یافته، فقط می‌تواند پیش‌بینیِ گاوسی از مکانِ پرنده بکند، و میانگینِ این گاوسی بر تنه متمرکز خواهد بود، چنان‌که در شکل ۱۴.۱۲(الف) نشان داده شده.

از سوی دیگر، مدلِ معقولِ پرنده، کنشِ فرار به یک سمت یا سمتِ دیگر را پیش‌بینی می‌کند، چنان‌که در شکل ۱۴.۱۲(ب) نشان داده شده. چنین مدلی به‌شدت غیرخطی است، زیرا تصمیمِ پرنده بسته به مکانِ دقیقش نسبت به تنه به‌شدت تغییر می‌کند.



شکل ۱۴.۱۲ – پرنده‌ای که به سمتِ درختی پرواز می‌کند (نمای از بالا). (الف) فیلترِ کالمن مکانِ پرنده را با گاوسیِ تکی متمرکز بر مانع پیش‌بینی می‌کند. (ب) مدلِ واقع‌گرایانه‌تر به کنشِ فرارِ پرنده اجازه می‌دهد، پیش‌بینی می‌کند به یک سمت یا سمتِ دیگر پرواز کند.

برای مدیریتِ چنین مثال‌هایی، بدیهی است به زبانِ بیانیِ بیشتری برای بازنماییِ رفتارِ سامانهٔ درحالِ‌مدل‌شدن نیاز داریم. در جامعهٔ نظریهٔ کنترل، که مسائلی مانند مانورِ فرارِی هواپیماها همان نوع دشواری‌ها را برمی‌انگیزند، راه‌حلِ استاندارد «فیلترِ کالمنِ سوئیچ‌شونده» (Switching Kalman Filter) است. در این رویکرد، چند فیلترِ کالمن به‌صورتِ موازی اجرا می‌شوند، هر یک با مدلِ متفاوتی از سامانه—برای نمونه، یکی برای پروازِ مستقیم، یکی برای چرخش‌های تندِ چپ و یکی برای چرخش‌های تندِ راست. از جمعِ وزنیِ پیش‌بینی‌ها استفاده می‌شود، که در آن وزن بستگی به این دارد

که هر فیلتر چقدر با داده‌های کنونی برازش دارد. در بخشِ بعد خواهیم دید این صرفاً حالتِ خاصِ مدلِ عمومیِ شبکهٔ بیزیِ پویاست، که با افزودنِ متغیرِ حالتِ گسستهٔ «مانور» به شبکهٔ نشان‌داده‌شده در شکل ۱۴.۹ به دست می‌آید. فیلترهای کالمنِ سوئیچ‌شونده بیشتر در تمرین KFSW.۱۴ بحث شده‌اند.

۱۴.۵ شبکه‌های بیزیِ پویا

شبکه‌های بیزیِ پویا (Dynamic Bayesian Networks یا DBN) معناشناسیِ شبکه‌های بیزیِ استاندارد را برای مدیریتِ مدل‌های احتمالِ زمانیِ از نوعِ توصیف‌شده در بخش ۱۴.۱ بسط می‌دهند. قبلاً نمونه‌هایی از DBNها دیده‌ایم: شبکهٔ چتر در شکل ۱۴.۲ و شبکهٔ فیلترِ کالمن در شکل ۱۴.۹. به‌طورِ کلی، هر برش از DBN می‌تواند هر شمارِ متغیرِ حالتِ X_t و متغیرِ شاهدِ E_t داشته باشد. برای سادگی، فرض می‌کنیم متغیرها، پیوندهایشان و توزیع‌های شرطی‌شان از برشی به برشِ دیگر دقیقاً تکرار می‌شوند و DBN فرایندِ مارکوفِ مرتبهٔ اول را بازنمایی می‌کند، به‌گونه‌ای که هر متغیر فقط در برشِ خودش یا برشِ بلافاصلهٔ قبلی می‌تواند والد داشته باشد. بدین‌ترتیب، DBN با شبکهٔ بیزیِ با متغیرهای بی‌نهایت متناظر است. باید روشن باشد هر مدلِ مارکوفِ پنهانی را می‌توان به‌صورتِ DBN با یک متغیرِ حالت و یک متغیرِ شاهد بازنمایی کرد. همچنین این مورد است که هر DBN با متغیرِ گسسته را می‌توان به‌صورتِ HMM بازنمایی کرد؛ چنان‌که در بخش ۱۴.۳ توضیح داده شد، می‌توانیم همهٔ متغیرهای حالتِ DBN را در متغیرِ حالتِ تکی ترکیب کنیم که مقادیرش همهٔ چندتایی‌های ممکنِ مقادیرِ متغیرهای حالتِ فردی‌اند. حالا، اگر هر HMM یک DBN باشد و هر DBN به HMM ترجمه‌پذیر باشد، تفاوت چیست؟ تفاوت این است که با تجزیهٔ حالتِ سامانهٔ پیچیده به متغیرهای سازنده‌اش، می‌توانیم از پراکندگی (Sparseness) در مدلِ احتمالِ زمانی بهره بگیریم.

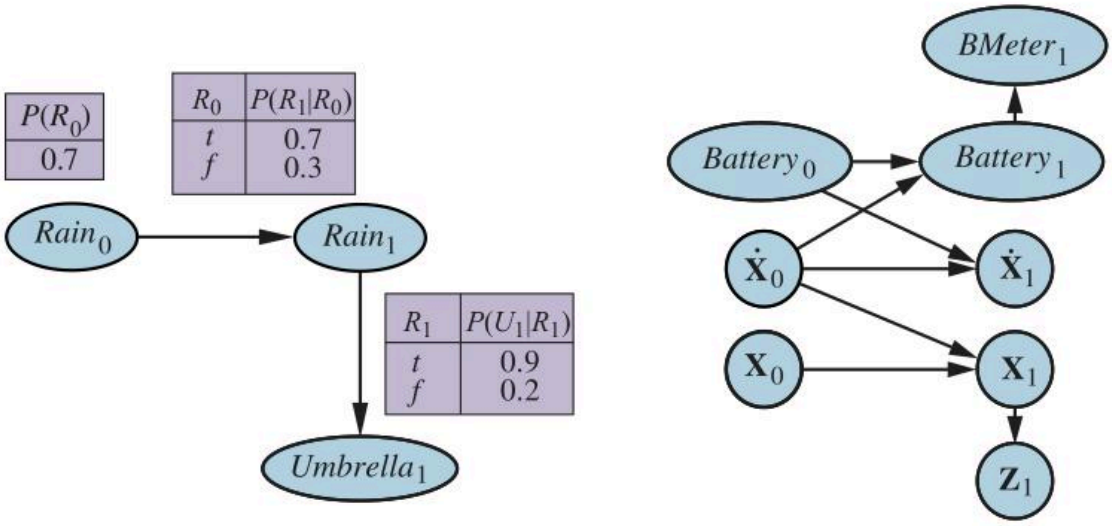
برای دیدنِ معنای این در عمل، به یاد آورید در بخش ۱۴.۳ گفتیم بازنماییِ HMM برای فرایندِ زمانی با n متغیرِ گسسته، هر یک با حداکثر d مقدار، به ماتریسِ انتقالی با اندازهٔ $O(d^{2n})$ نیاز دارد. بازنماییِ DBN، از سوی دیگر، اندازهٔ $O(n \cdot d^k)$ دارد اگر شمارِ والدهای هر متغیر با k کران‌دار باشد. به بیانِ دیگر، بازنماییِ DBN در شمارِ متغیرها خطی است نه نمایی. برای رباتِ جاروبرقی با ۴۲ مکانِ احتمالیِ کثیف، شمارِ احتمال‌های لازم از 5×10^{29} به چند هزار کاهش می‌یابد. قبلاً توضیح داده‌ایم هر مدلِ فیلترِ کالمنی را می‌توان در DBN با متغیرهای پیوسته و توزیع‌های شرطیِ خطی–گاوسی بازنمایی کرد (شکل ۱۴.۹). از بحثِ پایانِ بخشِ قبلی باید روشن باشد که هر DBN توسط مدلِ فیلترِ کالمن قابلِ بازنمایی نیست. در فیلترِ کالمن، توزیعِ حالتِ کنونی همیشه گاوسیِ چندمتغیرهٔ تکی است—یعنی «برآمدگیِ» تکی در مکانِ خاص. از سوی دیگر، DBNها می‌توانند توزیع‌های دلخواه را مدل کنند.

برای بسیاری از کاربردهای دنیایِ واقعی، این انعطاف ضروری است. برای نمونه، مکانِ کنونیِ کلیدهای من را در نظر بگیرید. ممکن است در جیبم باشند، روی میزِ کنارِ تخت، روی پیش‌خوانِ آشپزخانه، آویزان از درِ ورودی، یا قفل‌شده در ماشین. برآمدگیِ گاوسیِ تکی که همهٔ این مکان‌ها را شامل شود باید احتمالِ قابلِ‌توجهی به وسطِ هوا بالای باغچهٔ جلوییِ خانه اختصاص دهد. جنبه‌های دنیایِ واقعی مانندِ عامل‌های هدفمند، موانع و جیب‌ها «غیرخطی‌بودن‌هایی» وارد می‌کنند که برای گرفتنِ مدل‌های معقول به ترکیبِ متغیرهای گسسته و پیوسته نیاز دارند.

۱۴.۵.۱ ساختِ DBNها

برای ساختِ DBN باید سه نوع اطلاعات مشخص شود: توزیعِ پیشینی روی متغیرهای حالت، P(X0)؛ مدلِ انتقالِ P(Xt+1 | Xt)؛ و مدلِ حسگرِ P(Et | Xt).

برای مشخص‌کردنِ مدل‌های انتقال و حسگر، باید توپولوژیِ اتصال‌های میان برش‌های متوالی و میان متغیرهای حالت و شاهد را نیز مشخص کنیم. چون مدل‌های انتقال و حسگر همگن در زمان فرض می‌شوند—برای همهٔ t یکسان‌اند—راحت‌ترین کار آن است که صرفاً برای نخستین برش مشخص شوند. برای نمونه، مشخصاتِ کاملِ DBN برای جهانِ چتر با شبکهٔ سه‌گره‌ایِ نشان‌داده‌شده در شکل ۱۴.۱۳(الف) داده می‌شود. از این مشخصات، DBN کامل با شمارِ نامحدودِ برشِ زمانی را می‌توان در صورتِ نیاز با کپی‌کردنِ برشِ نخست ساخت.



شکل ۱۴.۱۳ – چپ: مشخصاتِ پیشین، مدلِ انتقال و مدلِ حسگر برایِ DBN چتر. برش‌های بعدی کپی‌هایی از برشِ 1 هستند. راست: DBN ساده برای حرکتِ ربات در صفحهٔ X–Y.

حالا مثالِ جالب‌تری را در نظر بگیریم: پایشِ رباتِ باتری‌دارِ در حالِ حرکت در صفحهٔ X–Y، چنان‌که در پایانِ بخش ۱۴.۱ معرفی شد. نخست، به متغیرهای حالت نیاز داریم، که شاملِ Xt = (Xt, Yt) برای موقعیت و Ẋt = (Ẋt, Ẏt) برای سرعت خواهد بود. فرض می‌کنیم روشی برای اندازه‌گیریِ موقعیت هست—شاید دوربینِ ثابت یا GPS (سامانهٔ موقعیت‌یابِ جهانی) رویِ‌برد—که اندازه‌گیری‌های Zt می‌دهد. موقعیت در گامِ زمانیِ بعد به موقعیت و سرعتِ کنونی بستگی دارد، مانندِ مدلِ استانداردِ فیلترِ کالمن. سرعت در گامِ بعد به سرعتِ کنونی و حالتِ باتری بستگی دارد. Batteryt را برای بازنماییِ سطحِ شارژِ واقعیِ باتری می‌افزاییم، که والدهایش سطحِ باتریِ قبلی و سرعت‌اند، و BMetert را می‌افزاییم که سطحِ شارژِ باتری را اندازه‌گیری می‌کند. این مدلِ پایهٔ نشان‌داده‌شده در شکل ۱۴.۱۳(ب) را می‌دهد.

ارزش دارد ماهیتِ مدلِ حسگرِ BMetert را عمیق‌تر بررسی کنیم. برای سادگی فرض کنیم هم Batteryt و هم BMetert می‌توانند مقادیرِ گسستهٔ 0 تا 5 بگیرند. (تمرین BATT.۱۴ از شما می‌خواهد این مدلِ گسسته را به مدلِ پیوستهٔ متناظر مرتبط کنید.) اگر کنتور همیشه دقیق باشد، آنگاه CPT P(BMetert | Batteryt) باید «روی قطر» احتمال‌های 1.0 و در جاهای دیگر 0.0 داشته باشد. در واقعیت، نویز همیشه به اندازه‌گیری‌ها می‌خزد. برای اندازه‌گیری‌های

پیوسته، ممکن است توزیعِ گاوسی با واریانسِ کوچک استفاده شود.[7] برای متغیرهای گسسته‌مان، می‌توانیم گاوسی را با توزیعی تقریب بزنیم که در آن احتمالِ خطا به‌شیوهٔ مناسب کاهش می‌یابد، به‌گونه‌ای که احتمالِ خطایِ بزرگ بسیار کوچک باشد. از اصطلاحِ «مدلِ خطایِ گاوسی» (Gaussian Error Model) برای پوششِ هر دو نسخهٔ پیوسته و گسسته استفاده می‌کنیم.

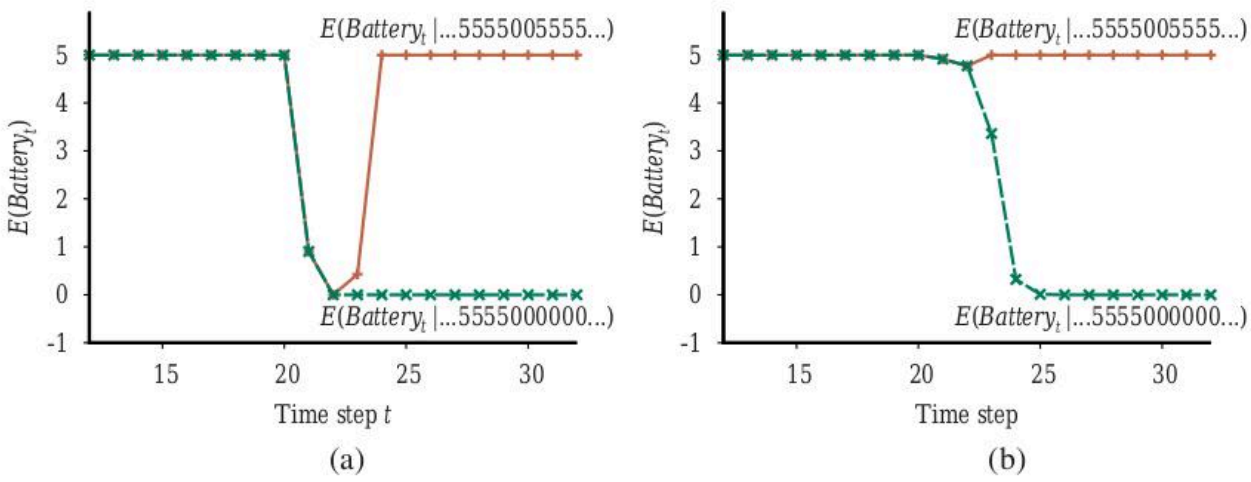
هر کسی که تجربهٔ عملی با رباتیک، کنترلِ فرایندِ رایانه‌ای یا دیگر شکل‌های حسِ خودکار داشته باشد به‌آسانی شهادت می‌دهد که مقدارهای کوچکِ نویزِ اندازه‌گیری اغلب کمترینِ مشکلاتِ آدم است. حسگرهای واقعی از کار می‌افتند. وقتی حسگری از کار می‌افتد، لزوماً سیگنالی نمی‌فرستد که بگوید «به هر حال، داده‌ای که می‌خواهم بفرستم مشتی مزخرف است.» در عوض، صرفاً مزخرف را می‌فرستد. ساده‌ترین نوعِ خرابی «خرابیِ گذرا» (Transient Failure) نامیده می‌شود، که در آن حسگر گاهی تصمیم می‌گیرد مزخرفی بفرستد. برای نمونه، حسگرِ سطحِ باتری ممکن است عادت داشته باشد وقتی کسی به ربات تنه می‌زند خوانشِ 0 بفرستد، حتی اگر باتری کاملاً شارژ باشد.

ببینیم وقتی خرابیِ گذرا با مدلِ خطایِ گاوسی که چنین خرابی‌هایی را در خود جای نمی‌دهد رخ می‌دهد چه اتفاقی می‌افتد. فرض کنیم، برای نمونه، ربات آرام نشسته و ۲۰ خوانشِ متوالیِ باتریِ 5 را مشاهده می‌کند. سپس کنتورِ باتری تشنجِ موقتی دارد و خوانشِ بعدی BMeter21 = 0 است. مدلِ خطایِ گاوسیِ ساده ما را به باورِ چه چیزی دربارهٔ Battery21 رهنمون خواهد شد؟ طبقِ قاعدهٔ بیز، پاسخ هم به مدلِ حسگرِ P(BMeter21 = 0 | Battery21) و هم به پیش‌بینیِ P(Battery21 | BMeter1:20) بستگی دارد. اگر احتمالِ خطایِ بزرگِ حسگر به‌طورِ قابلِ‌توجهی کمتر از احتمالِ پیش‌بینی‌شدهٔ کاهشِ واقعیِ ۵ واحدیِ باتری باشد، آنگاه نتیجه خواهیم گرفت باتری احتمالاً واقعاً مرده است —حتی اگر مدلِ واقعیِ خرابیِ گذرا را بدانیم و بدانیم خوانشِ 0 معنایِ کمی دارد. اگر مدلِ خرابیِ گذرا را در مدلِ حسگرِ خود بگنجانیم، نتیجه‌گیریِ درست انجام می‌شود: باتری به‌احتمالِ زیاد خوب است و خوانشِ 0 کنتور را نادیده می‌گیریم. تمرین BATT.۱۴ از شما می‌خواهد مدلِ حسگر را برای دربرگرفتنِ خرابی‌های گذرا اصلاح کنید.

ساده‌ترین نوعِ مدلِ خرابی برای حسگر احتمالِ معینی را اجازه می‌دهد که حسگر مقداری کاملاً نادرست برمی‌گرداند، صرف‌نظر از حالتِ واقعیِ جهان. برای نمونه، اگر کنتورِ باتری با برگرداندنِ 0 از کار بیفتد، ممکن است بگوییم:

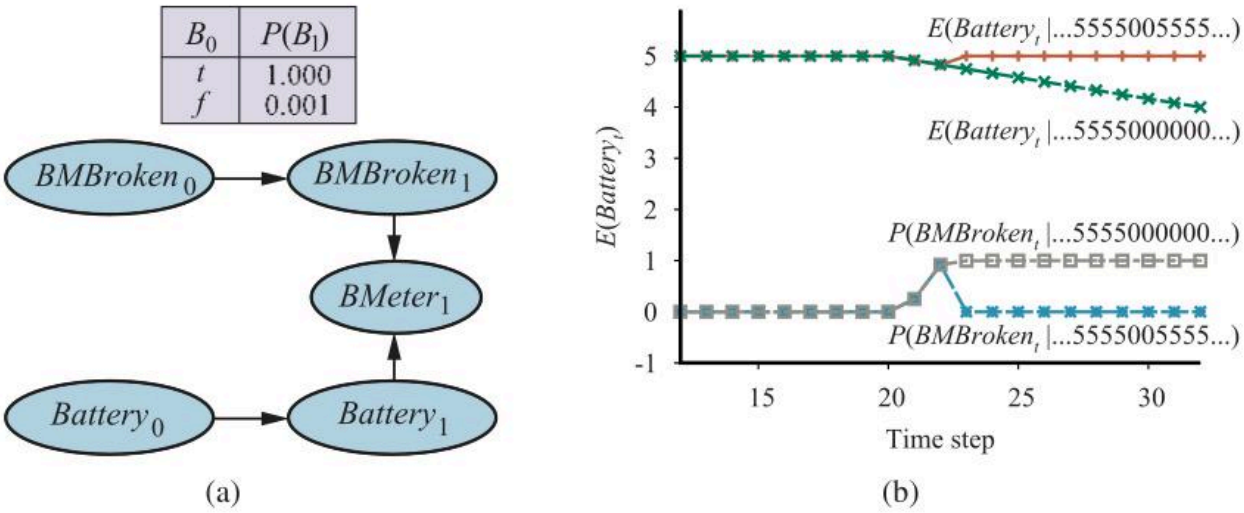
```
P(BMeter_t = 0 | Battery_t = 5) = 0.03
```

که احتمالاً بسیار بزرگ‌تر از احتمالی است که مدلِ خطایِ گاوسیِ ساده تخصیص می‌دهد. این را «مدلِ خرابیِ گذرا» بنامیم. چگونه عمل می‌کند؟ شکل ۱۴.۱۴ دو آزمایش را با مدلِ خطایِ گاوسیِ ساده و مدلِ خرابیِ گذرا مقایسه می‌کند. منحنی‌های بالایی: مسیرِ مقدارِ موردانتظارِ Battery_t برای توالیِ مشاهده‌ای متشکل از همهٔ 5ها به‌جزِ 0ها در t=21 و t=22، با استفاده از مدلِ خطایِ گاوسیِ ساده (الف) و مدلِ خرابیِ گذرا (ب). منحنی‌های پایینی: مسیر وقتی مشاهده از t=21 به بعد در 0 می‌ماند. خرابیِ گذرا به‌خوبی مدیریت می‌شود، اما خرابیِ پایدار به بدبینیِ بیش از حد دربارهٔ شارژِ باتری می‌انجامد.



شکل ۱۴.۱۴ – (الف) منحنیِ بالایی: مسیرِ مقدارِ موردانتظارِ Battery_t برای توالیِ مشاهده‌ای متشکل از همهٔ 5ها به‌جزِ 0ها در t=21 و t=22، با استفاده از مدلِ خطایِ گاوسیِ ساده. منحنیِ پایینی: مسیر وقتی مشاهده از t=21 به بعد در 0 می‌ماند. (ب) همان آزمایش با مدلِ خرابیِ گذرا اجرا شده. خرابیِ گذرا به‌خوبی مدیریت می‌شود، اما خرابیِ پایدار به بدبینیِ بیش از حد دربارهٔ شارژِ باتری می‌انجامد.

حالا موردِ خرابیِ «پایدار» (Persistent) را در نظر بگیرید: حسگر از یک نقطهٔ خاص به بعد برای همیشه از کار می‌افتد. این وضعیتِ مشترکِ بسیاری از حسگرهای واقعی است و نیازمندِ متغیرِ حالتِ دیگری برای وضعیتِ حسگر است: BMBroken_t. این متغیرِ بولی را می‌توان به‌عنوانِ والدِ BMeter_t افزود. مدلِ خرابیِ پایدار برای حسگرِ باتری در شکل ۱۴.۱۵(الف) نشان داده شده. عملکردِ مشخصه‌اش از «کمانِ پایداری» (Persistence Arc) پیونددهندهٔ BMBroken0 به BMBroken1 می‌آید. این کمانِ پایداری CPT دارد که احتمالِ کوچکی از خرابی در هر گامِ زمانیِ معین، مثلاً 0.001، می‌دهد، اما مشخص می‌کند حسگر وقتی خراب شد خراب می‌ماند. وقتی حسگر خوب است، مدلِ حسگرِ BMeter یکسان با مدلِ خرابیِ گذراست؛ وقتی حسگر خراب است، خوانشِ کنتور از توزیعِ دیگری می‌آید.



شکل ۱۴.۱۵ – (الف) قطعه‌ای از DBN که متغیرِ وضعیتِ حسگرِ لازم برای مدل‌کردنِ خرابیِ پایدارِ حسگرِ باتری را نشان می‌دهد. (ب) منحنی‌های بالایی: مسیرهای مقدارِ موردانتظارِ Battery_t برای توالی‌های مشاهدهٔ «خرابیِ گذرا» و «خرابیِ دائمی». منحنی‌های پایینی: مسیرهای احتمالِ BMBroken با توجه به دو توالیِ مشاهده.

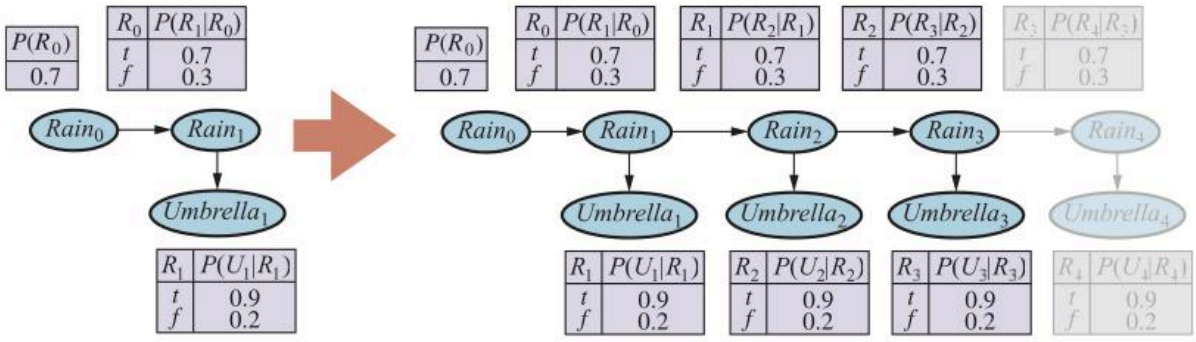
دو منحنیِ پایینی در شکل ۱۴.۱۵(ب) احتمالِ باورِ BMBroken را برای دو توالیِ مشاهده نشان می‌دهند. در حالتِ گذرا (0ها در t=21 و t=22 و سپس همه 5ها)، احتمالِ BMBroken پس از بازگشتِ خوانش‌ها به 5 به‌سرعت به صفر می‌رسد. در حالتِ پایدار (0ها از t=21 به بعد)، احتمالِ BMBroken به 1 همگرا می‌شود. این دقیقاً رفتاری است که می‌خواهیم.

مدلِ DBN حاصل در کل، چنان‌که در شکل ۱۴.۱۳(ب) نشان داده شده است:

توزیع‌های شرطی برای این DBN را می‌توان با ترکیبِ مدل‌های معرفی‌شدهٔ پیشین مشخص کرد. با چنین مدلی، می‌توانیم استنتاج را با الگوریتم‌هایی که در بخش‌های ۱۴.۲ تا ۱۴.۴ توسعه دادیم انجام دهیم. با این حال، یک نکتهٔ مهم: با افزودنِ باتری به مدل، حالتِ DBN اکنون شاملِ چهار متغیر است (X، Y، Ẋ، Ẏ، Battery)، و استنتاجِ دقیقِ حاشیه‌ای روی Battery می‌تواند گران باشد. خوشبختانه، چون Battery فقط به Ẋ، Ẏ و Battery قبلی بستگی دارد، می‌توانیم استنتاج را با استفاده از ساختارِ پراکندهٔ DBN بهینه کنیم. اما به‌طورِ کلی، استنتاجِ دقیق در DBNها می‌تواند غیرقابلِ‌مدیریت باشد؛ به همین دلیل است که الگوریتم‌های تقریبی مانندِ فیلترِ ذره‌ای اهمیت دارند.

۱۴.۵.۲ استنتاجِ دقیق در DBNها

پس از ترسیمِ ایده‌هایی برای بازنماییِ فرایندهای پیچیده به‌صورتِ DBN، حالا به پرسشِ استنتاج می‌پردازیم. از جهتی، این پرسش قبلاً پاسخ داده شده: شبکه‌های بیزیِ پویا شبکه‌های بیزی‌اند، و ما قبلاً الگوریتم‌هایی برای استنتاج در شبکه‌های بیزی داریم. با توجه به توالی‌ای از مشاهده‌ها، می‌توان بازنماییِ کاملِ شبکهٔ بیزیِ DBN را با تکثیرِ برش‌ها تا وقتی شبکه به‌اندازهٔ کافی بزرگ شود که مشاهده‌ها را در خود جای دهد ساخت، چنان‌که در شکل ۱۴.۱۶. این فن «بازکردن» (Unrolling) نامیده می‌شود. (از نظرِ فنی، DBN معادلِ شبکهٔ نیمه‌نامتناهیِ حاصل از بازکردنِ بی‌پایان است. برش‌های افزوده‌شده فراتر از آخرین مشاهده هیچ اثری بر استنتاج‌های درونِ دورهٔ مشاهده ندارند و می‌توانند حذف شوند.) پس از بازشدنِ DBN، می‌توان از هر یک از الگوریتم‌های استنتاج—حذفِ متغیر، روش‌های خوشه‌بندی و مانندِ آن—توصیف‌شده در فصل ۱۳ استفاده کرد.



شکل ۱۴.۱۶ — بازکردنِ شبکهٔ بیزیِ پویا: برش‌ها تکثیر می‌شوند تا توالیِ مشاهدهٔ Umbrella1:3 را در خود جای دهند. برش‌های بیشتر هیچ اثری بر استنتاج‌های درونِ دورهٔ مشاهده ندارند.

متأسفانه، کاربردِ ساده‌لوحانهٔ بازکردن چندان کارآمد نخواهد بود. اگر بخواهیم فیلترکردن یا هموارسازی را با توالیِ بلندی از مشاهده‌های $e_{1:t}$ انجام دهیم، شبکهٔ بازشده $O(t)$ فضا لازم دارد و بدین‌ترتیب با افزوده‌شدنِ مشاهده‌های بیشتر بدونِ کران رشد می‌کرد. علاوه بر این، اگر صرفاً الگوریتمِ استنتاج را هر بار با افزوده‌شدنِ مشاهده‌ای دوباره اجرا کنیم، زمانِ استنتاجِ هر به‌روزرسانی نیز به‌صورتِ $O(t)$ افزایش خواهد یافت.

با بازگشت به بخش ۱۴.۲.۱، می‌بینیم زمان و فضایِ ثابتِ هر به‌روزرسانیِ فیلترکردن قابلِ‌دستیابی است اگر محاسبه بتواند به‌صورتِ بازگشتی انجام شود. اساساً، به‌روزرسانیِ فیلترکردن در معادلهٔ (14.5) با خارج‌کردنِ متغیرهای حالتِ گامِ زمانیِ قبلی از جمع کار می‌کند تا توزیعِ گامِ زمانیِ تازه به دست آید. خارج‌کردنِ متغیرها از جمع دقیقاً همان کاری است که الگوریتمِ حذفِ متغیر (شکل ۱۳.۱۳) انجام می‌دهد، و معلوم می‌شود اجرایِ حذفِ متغیر با متغیرها در ترتیبِ زمانی دقیقاً عملکردِ به‌روزرسانیِ بازگشتیِ فیلترکردن در معادلهٔ (14.5) را شبیه‌سازی می‌کند. الگوریتمِ اصلاح‌شده حداکثر دو برش را در هر زمان در حافظه نگه می‌دارد: با برشِ 0 شروع می‌کنیم، برشِ 1 را می‌افزاییم، سپس برشِ 0 را خارج می‌کنیم، سپس برشِ 2 را می‌افزاییم، سپس برشِ 1 را خارج می‌کنیم، و همین‌طور ادامه می‌دهیم. بدین‌ترتیب، می‌توانیم به فضای و زمانِ ثابتِ هر به‌روزرسانیِ فیلترکردن برسیم. (همان عملکرد را می‌توان با اصلاحاتِ مناسبِ الگوریتمِ خوشه‌بندی به دست آورد.) تمرین DBNE.۱۴ از شما می‌خواهد این واقعیت را برای شبکهٔ چتر تأیید کنید.

خبرِ خوب همین‌قدر؛ حالا خبرِ بد: معلوم می‌شود «ثابتِ» پیچیدگیِ زمانی و فضاییِ هر به‌روزرسانی، در تقریباً همهٔ موارد، در شمارِ متغیرهای حالت نمایی است. اتفاقی که می‌افتد این است که با پیشرویِ حذفِ متغیر، عامل‌ها رشد می‌کنند تا همهٔ متغیرهای حالتِ قبلی را در بر گیرند. از نظرِ ریاضی، معلوم می‌شود هزینهٔ هر به‌روزرسانیِ استنتاجِ دقیق برای DBN با n متغیرِ حالت، بدترین حالتش $O(2^n)$ است، حتی اگر ساختارِ شبکه پراکنده باشد. البته، این بسیار کمتر از هزینهٔ به‌روزرسانیِ HMM است که $O(d^{2n})$ است، اما برای شمارِ زیادِ متغیرها هنوز ناشدنی است. این واقعیتِ تلخ بدان معناست حتی اگر بتوانیم از DBNها برای بازنماییِ فرایندهای زمانیِ بسیار پیچیده با متغیرهای پراکندهٔ بسیار استفاده کنیم، نمی‌توانیم دربارهٔ آن فرایندها کارآمد و دقیق استدلال کنیم. خودِ مدلِ DBN، که توزیعِ مشترکِ پیشینیِ همهٔ متغیرها را بازنمایی می‌کند، به CPTهای سازنده‌اش قابلِ تجزیه است، اما توزیعِ مشترکِ پسینیِ شرطی‌شده بر توالیِ مشاهده—یعنی پیامِ پیشرو—به‌طورِ کلی قابلِ تجزیه نیست. مسئله در حالتِ عمومی ناشدنی است، پس باید به روش‌های تقریبی پناه ببریم.

۱۴.۵.۳ استنتاجِ تقریبی در DBNها

بخش ۱۳.۴ دو الگوریتمِ تقریبی را توصیف کرد: وزن‌گذاریِ درستنمایی (شکل ۱۳.۱۸) و مونت‌کارلویِ زنجیرهٔ مارکوف (MCMC، شکل ۱۳.۲۰). از میانِ این دو، نخستین به‌آسانی‌ترین با بافتِ DBN سازگار می‌شود. (الگوریتمِ فیلترکردنِ MCMC به‌طورِ مختصر در یادداشت‌های پایانِ این فصل توصیف شده.) اما خواهیم دید پیش از پیدایشِ روشی عملی، چند بهبود بر الگوریتمِ استانداردِ وزن‌گذاریِ درستنمایی لازم است.

به یاد آورید وزن‌گذاریِ درستنمایی با نمونه‌برداری از گره‌های غیرشاهدِ شبکه در ترتیبِ توپولوژیک کار می‌کند، و هر نمونه را با درستنمایی‌ای که به متغیرهای شاهدِ مشاهده‌شده می‌دهد وزن می‌کند. مانندِ الگوریتم‌های دقیق، می‌توانستیم وزن‌گذاریِ درستنمایی را مستقیماً رویِ DBN بازشده اعمال کنیم، اما این از همان مشکلاتِ افزایشِ زمان و

فضایِ هر به‌روزرسانی با رشدِ توالیِ مشاهده رنج می‌برد. مسئله این است که الگوریتمِ استاندارد هر نمونه را به‌نوبت، تمامِ راهِ سراسرِ شبکه، اجرا می‌کند.

در عوض، می‌توانیم صرفاً همهٔ N نمونه را با هم، یک برش در هر بار، از میانِ DBN اجرا کنیم. الگوریتمِ اصلاح‌شده با الگویِ عمومیِ الگوریتم‌های فیلترکردن سازگار است، با مجموعهٔ N نمونه به‌عنوانِ پیامِ پیشرو. نخستین نوآوریِ کلیدی، پس، استفاده از خودِ نمونه‌ها به‌عنوانِ بازنماییِ تقریبیِ توزیعِ حالتِ کنونی است. این الزامِ «زمانِ ثابتِ» هر به‌روزرسانی را برآورده می‌کند، هرچند ثابت به شمارِ نمونه‌های لازم برای حفظِ تقریبِ دقیق بستگی دارد. همچنین نیازی به بازکردنِ DBN نیست، زیرا فقط باید برشِ کنونی و برشِ بعدی را در حافظه داشته باشیم. این رویکرد «نمونه‌برداریِ اهمیتِ ترتیبی» (Sequential Importance Sampling) یا SIS نامیده می‌شود.

در بحثِ وزن‌گذاریِ درستنمایی در فصل ۱۳، اشاره کردیم دقتِ الگوریتم آسیب می‌بیند اگر متغیرهای شاهد «پایین‌دستِ» متغیرهای درحالِ‌نمونه‌برداری باشند، زیرا در آن صورت نمونه‌ها بدونِ هیچ تأثیری از شواهد تولید می‌شوند و تقریباً همه وزن‌های بسیار پایینی خواهند داشت. حالا اگر به ساختارِ نمونهٔ DBN—مثلاً DBN چتر در شکل ۱۴.۱۶—نگاه کنیم، می‌بینیم در واقع متغیرهای حالتِ آغازین بدونِ بهره از شواهدِ بعدی نمونه‌برداری خواهند شد. در واقع، با نگاهِ دقیق‌تر، می‌بینیم هیچ‌کدام از متغیرهای حالت هیچ متغیرِ شاهدی در میانِ نیاکانشان ندارند! از این رو، اگرچه وزنِ هر نمونه به شواهد بستگی خواهد داشت، مجموعهٔ واقعیِ نمونه‌های تولیدشده کاملاً مستقل از شواهد خواهد بود. برای نمونه، حتی اگر رئیس هر روز چتر بیاورد، فرایندِ نمونه‌برداری می‌تواند روزهای بی‌پایانِ آفتابی را توهم کند.

معنای این در عمل این است که کسرِ نمونه‌هایی که به‌صورتِ معقولی نزدیکِ توالیِ واقعیِ رخدادها می‌مانند (و بنابراین وزن‌های غیرقابلِ‌صرف‌نظر دارند) با t، طولِ توالی، به‌صورتِ نمایی افت می‌کند. به بیانِ دیگر، برای حفظِ سطحِ معینی از دقت، باید شمارِ نمونه‌ها را با t به‌صورتِ نمایی افزایش دهیم. با توجه به اینکه الگوریتمِ فیلترکردنِ بلادرنگ فقط می‌تواند از شمارِ کران‌داری نمونه استفاده کند، آنچه در عمل رخ می‌دهد این است که خطا پس از شمارِ بسیار کمی از گام‌های به‌روزرسانی منفجر می‌شود. شکل ۱۴.۱۹ در صفحهٔ ۴۹۴ این اثر را برایِ SIS اعمال‌شده بر مسئلهٔ بومی‌سازیِ شبکه‌ای از بخش ۱۴.۳ نشان می‌دهد: حتی با ۱۰۰,۰۰۰ نمونه، تقریبِ SIS پس از حدود ۲۰ گام کاملاً شکست می‌خورد.

واضح است که به راه‌حلِ بهتری نیاز داریم. دومین نوآوریِ کلیدی، متمرکزکردنِ مجموعهٔ نمونه‌ها روی نواحیِ احتمالِ بالای فضایِ حالت است. این کار را می‌توان با دورانداختنِ نمونه‌هایی که وزنِ بسیار پایینی دارند، طبقِ مشاهده‌ها، در حالی که نمونه‌های با وزنِ بالا تکثیر می‌شوند، انجام داد. بدین‌ترتیب، جمعیتِ نمونه‌ها به‌صورتِ معقولی نزدیکِ واقعیت می‌ماند. اگر نمونه‌ها را منبعی برای مدل‌کردنِ توزیعِ پسینی بدانیم، منطقی است در نواحیِ فضایِ حالت که پسین بالاتر است از نمونه‌های بیشتری استفاده کنیم.

خانواده‌ای از الگوریتم‌ها به نام «فیلترِ ذره‌ای» (Particle Filtering) دقیقاً برای همین طراحی شده. (نامِ اولیهٔ دیگرش «نمونه‌برداریِ اهمیتِ ترتیبی با نمونه‌برداریِ مجدد» بود، اما به دلایلی جا نیفتاد.) فیلترِ ذره‌ای چنین کار می‌کند: نخست، جمعیتی از N نمونه از توزیعِ پیشینیِ P(X0) تولید می‌کنیم. سپس چرخهٔ به‌روزرسانی برای هر گامِ زمانی تکرار می‌شود:

1. هر نمونه با نمونه‌برداری از مقدارِ حالتِ بعدیِ xt+1 به‌شرطِ مقدارِ کنونیِ xt برای نمونه، بر اساسِ مدلِ انتقالِ P(Xt+1 | xt)، به جلو منتشر می‌شود.
2. هر نمونه با درستنمایی‌ای که به شاهدِ تازه، P(et+1 | xt+1)، تخصیص می‌دهد وزن‌گذاری می‌شود.
3. جمعیت برای تولیدِ جمعیتِ تازه‌ای از N نمونه دوباره نمونه‌برداری می‌شود. هر نمونهٔ تازه از جمعیتِ کنونی انتخاب می‌شود؛ احتمالِ انتخابِ نمونهٔ خاصی متناسب با وزنِ آن است. نمونه‌های تازه وزن‌نخورده‌اند.

```
function PARTICLE-FILTERING(e, N, dbn) returns a set of samples for the next time step
                                                    inputs: e, new incoming evidence
                                          N, the number of samples to retain
                        dbn, a DBN defined by P(X0), P(X1|X0), and P(E1|X1)
          persistent: S, a vector of samples of size N, initially generated from P(X0)
                                    local variables: W, a vector of weights of size N
                                                                    :for i = 1 to N
                      S[i] ← a sample from P(X1 | X0 = S[i])          // step 1
                           W[i] ← P(e | X1 = S[i])                    // step 2
                           S ← WEIGHTED-SAMPLE-WITH-REPLACEMENT(N, S, W)  // step 3
                                                                      return S
```

شکل ۱۴.۱۷ — الگوریتمِ فیلترِ ذره‌ای که به‌صورتِ عملیاتِ به‌روزرسانیِ بازگشتی با حالت (مجموعهٔ نمونه‌ها) پیاده‌سازی شده.

هر یک از عملیاتِ نمونه‌برداری شاملِ نمونه‌برداری از متغیرهای برشِ مربوط در ترتیبِ توپولوژیک است، بسیار شبیهِ PRIOR-SAMPLE. عملیاتِ WEIGHTED-SAMPLE-WITH-REPLACEMENT را می‌توان طوری پیاده‌سازی کرد که در زمانِ موردانتظارِ O(N) اجرا شود. شمارهٔ گام‌ها به توصیفِ متن اشاره دارند.

الگوریتم با جزئیات در شکل ۱۴.۱۷ نشان داده شده و عملکردش برای DBN چتر در شکل ۱۴.۱۸ نشان داده شده. می‌توانیم نشان دهیم این الگوریتم سازگار است—با گرایشِ N به بی‌نهایت احتمال‌های درست می‌دهد—با بررسیِ عملیات در یک چرخهٔ به‌روزرسانی. فرض می‌کنیم جمعیتِ نمونه با بازنماییِ درستی از پیامِ پیشرو شروع می‌شود—یعنی f1:t = P(Xt | e1:t) در زمانِ t. با نوشتنِ N(xt | e1:t) برای شمارِ نمونه‌های اشغال‌کنندهٔ حالتِ xt پس از پردازشِ مشاهده‌های e1:t، بنابراین برای N بزرگ داریم:

$$N(x_t \mid e_{1:t}) / N = P(x_t \mid e_{1:t}) \qquad (14.23)$$

حالا هر نمونه را با نمونه‌برداری از متغیرهای حالت در t+1، به‌شرطِ مقادیرِ نمونه در t، به جلو منتشر می‌کنیم. شمارِ نمونه‌های رسیده به حالتِ xt+1 از هر xt، احتمالِ انتقال ضرب‌درِ جمعیتِ xt است؛ از این رو، شمارِ کلِ نمونه‌های رسیده به xt+1 برابر:

$$N(x_{t+1} \mid e_{1:t}) = \sum_{x_t} P(x_{t+1} \mid x_t)\, N(x_t \mid e_{1:t})$$

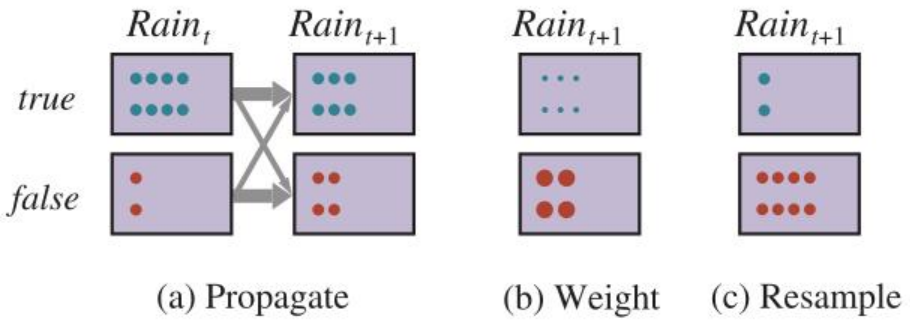
حالا هر نمونه را با درستنمایی‌اش برای شاهدِ t+1 وزن می‌کنیم. نمونه‌ای در حالتِ xt+1 وزنِ P(et+1 | xt+1) دریافت می‌کند. وزنِ کلِ نمونه‌های در xt+1 پس از دیدنِ et+1 بنابراین:

```
W(xt+1 | e1:t+1) = P(et+1 | xt+1) N(xt+1 | e1:t)
```

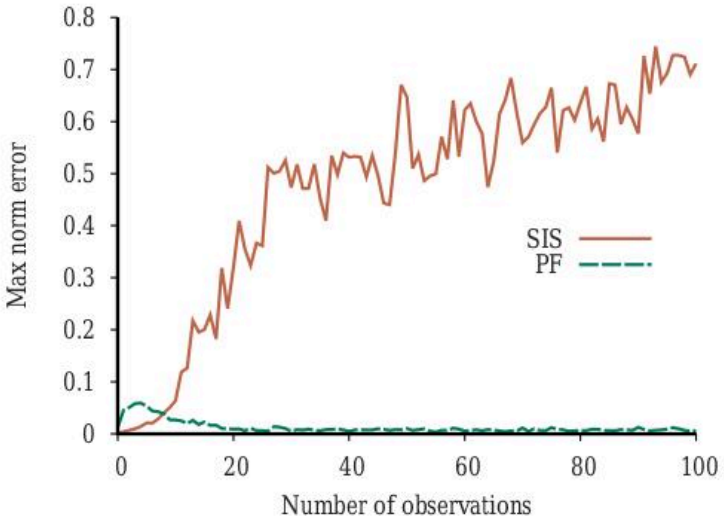
حالا نوبتِ گامِ نمونه‌برداریِ مجدد است. چون هر نمونه با احتمالی متناسب با وزنش تکثیر می‌شود، شمارِ نمونه‌های در حالتِ xt+1 پس از نمونه‌برداریِ مجدد متناسب با وزنِ کلِ در xt+1 قبل از نمونه‌برداریِ مجدد است:

```
                                        N(xt+1 | e1:t+1) / N = α W(xt+1 | e1:t+1)
                                 α P(et+1 | xt+1) N(xt+1 | e1:t) =
                          α P(et+1 | xt+1) ∑ P(xt+1 | xt) N(xt | e1:t) =
                                            xt
(14.23 با)          α N P(et+1 | xt+1) ∑ P(xt+1 | xt) P(xt | e1:t) =
                                         xt
(14.5 با)            α′ P(et+1 | xt+1) ∑ P(xt+1 | xt) P(xt | e1:t) =
                                        xt
                                            P(xt+1 | e1:t+1) =
```

بنابراین جمعیتِ نمونه پس از یک چرخهٔ به‌روزرسانی به‌درستی پیامِ پیشرو را در زمانِ t+1 بازنمایی می‌کند. فیلترِ ذره‌ای پس سازگار است، اما آیا کارآمد است؟ برای بسیاری از مواردِ عملی، به نظر می‌رسد پاسخ بله است: فیلترِ ذره‌ای به نظر می‌رسد با شمارِ ثابتی نمونه تقریبِ خوبی از پسینِ واقعی نگه می‌دارد. شکل ۱۴.۱۹ نشان می‌دهد فیلترِ ذره‌ای با فقط هزار نمونه کارِ خوبی روی مسئلهٔ بومی‌سازیِ شبکه‌ای انجام می‌دهد.



شکل ۱۴.۱۸ — چرخهٔ به‌روزرسانیِ فیلترِ ذره‌ای برایِ DBN چتر با N=10، که جمعیت‌های نمونهٔ هر حالت را نشان می‌دهد. (الف) در زمانِ t، ۸ نمونه باران و ۲ نمونه ¬باران را نشان می‌دهند. هر یک با نمونه‌برداری از حالتِ بعدی از طریقِ مدلِ انتقال به جلو منتشر می‌شود. در زمانِ t+1، ۶ نمونه باران و ۴ نمونه ¬باران را نشان می‌دهند. (ب) ¬چتر در t+1 مشاهده می‌شود. هر نمونه با درستنمایی‌اش برای مشاهده وزن‌گذاری می‌شود، چنان‌که با اندازهٔ دایره‌ها نشان داده شده. (ج) مجموعهٔ تازه‌ای از ۱۰ نمونه با انتخابِ تصادفیِ وزندار از مجموعهٔ کنونی تولید می‌شود، که به ۲ نمونهٔ باران و ۸ نمونهٔ ¬باران می‌انجامد.



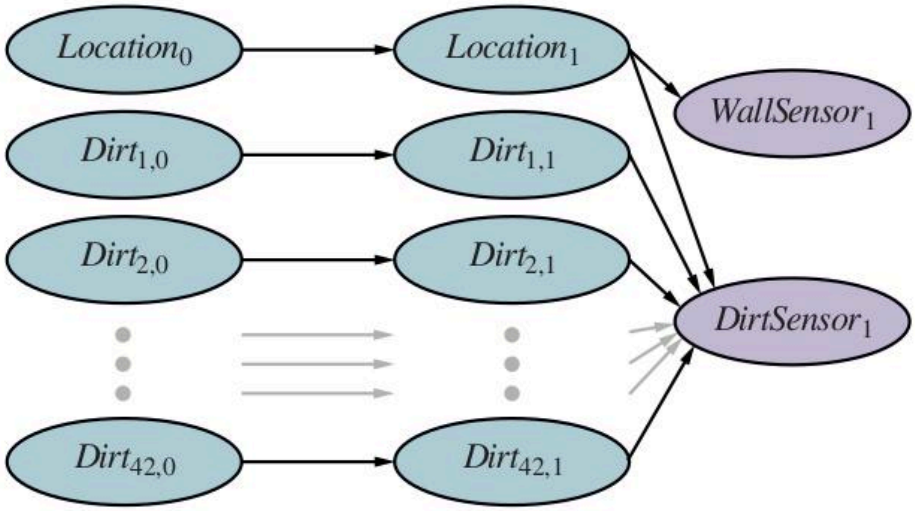
شکل ۱۴.۱۹ – خطایِ نرمِ بیشینه در برآوردِ مکانِ شبکه‌ای (در مقایسه با استنتاجِ دقیق) برای وزن‌گذاریِ درستنمایی (نمونه‌برداریِ اهمیتِ ترتیبی) با ۱۰۰,۰۰۰ نمونه و فیلترِ ذره‌ای با ۱,۰۰۰ نمونه؛ داده‌ها روی ۵۰ اجرا میانگین‌گیری شده‌اند.

همچنین روی مسائلِ دنیایِ واقعی کار می‌کند: الگوریتم از هزاران کاربرد در علم و مهندسی پشتیبانی می‌کند. (برخی مراجع در پایانِ فصل داده شده‌اند.) ترکیب‌های متغیرهای گسسته و پیوسته و نیز مدل‌های غیرخطی و غیرگاوسی برای متغیرهای پیوسته را مدیریت می‌کند. تحتِ مفروضاتِ خاصی—به‌طورِ خاص، اینکه احتمال‌های مدل‌های انتقال و حسگر از 0 و 1 دور نگه داشته شوند—همچنین می‌توان اثبات کرد تقریب خطایِ کران‌داری با احتمالِ زیاد نگه می‌دارد، چنان‌که شکل پیشنهاد می‌کند.

الگوریتمِ فیلترِ ذره‌ای با این حال ضعف‌هایی دارد. ببینیم برای جهانِ جاروبرقیِ با کثیفیِ افزوده‌شده چگونه عمل می‌کند. از بخش ۱۴.۳.۲ به یاد آورید این اندازهٔ فضایِ حالت را با عاملِ 2^{4^2} افزایش می‌دهد و استنتاجِ دقیقِ HMM را ناشدنی می‌کند. می‌خواهیم ربات پرسه بزند و نقشه‌ای از جایی که کثیفی‌ها هستند بسازد. (این مثالِ ساده‌ای از «بومی‌سازی و نقشه‌سازیِ هم‌زمان» (Simultaneous Localization and Mapping) یا SLAM است، که در فصل ۲۶ عمیق‌تر پوشش می‌دهیم.) فرض کنیم $\text{Dirt}_{i,t}$ یعنی مربعِ i در زمانِ t کثیف است و DirtSensort صادق است اگر و فقط اگر ربات در زمانِ t کثیفی را تشخیص دهد. فرض می‌کنیم در هر مربعِ معین، کثیفی با احتمالِ p باقی می‌ماند، در حالی که مربعِ تمیز با احتمالِ p-1 کثیف می‌شود (که بدان معناست هر مربع به‌طورِ میانگین نیمی از زمان کثیف است). ربات حسگرِ کثیفی برای مکانِ کنونی‌اش دارد؛ حسگر با احتمالِ 0.9 دقیق است. شکل ۱۴.۲۰ DBN را نشان می‌دهد.

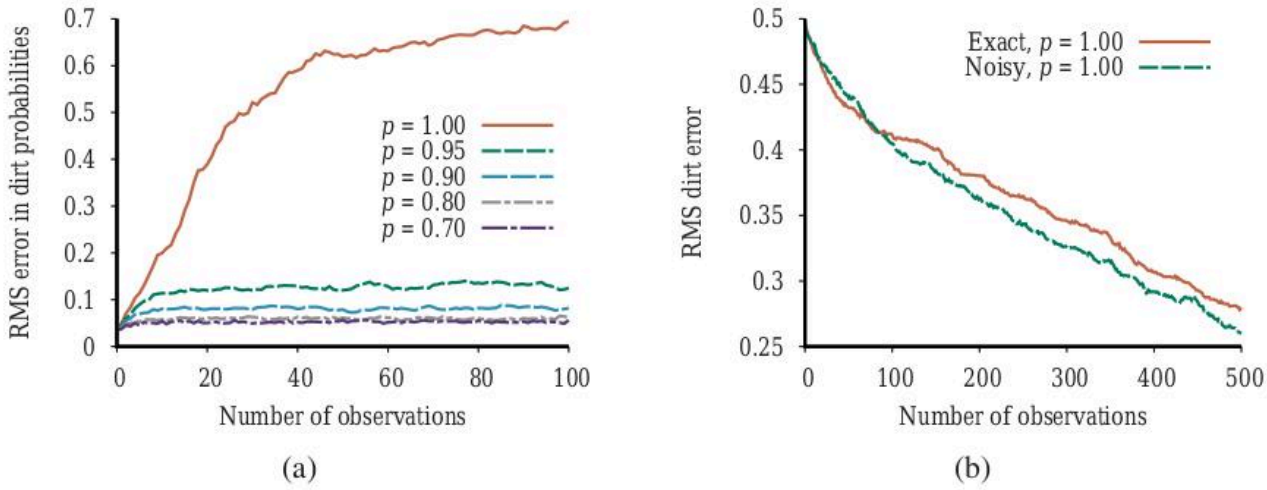
برای سادگی، با فرض اینکه ربات حسگرِ مکانِ کامل دارد، به‌جای حسگرِ نویزیِ دیوار، شروع می‌کنیم. عملکردِ الگوریتم در شکل ۱۴.۲۱(الف) نشان داده شده، که در آن برآوردهایش برای کثیفی با نتایجِ استنتاجِ دقیق مقایسه شده‌اند. (به‌زودی خواهیم دید استنتاجِ دقیق چگونه ممکن است.) برای مقادیرِ پایینِ پایداریِ کثیفیِ p، خطا کوچک می‌ماند—اما این دستاوردِ بزرگی نیست، زیرا برای هر مربع، پسینِ واقعیِ کثیفی نزدیکِ 0.5 است اگر ربات اخیراً از آن مربع بازدید نکرده باشد. برای مقادیرِ بالاترِ p، کثیفی مدتِ طولانی‌تری می‌ماند، پس بازدید از مربعی اطلاعاتِ مفیدتری می‌دهد که در دورهٔ طولانی‌تری معتبر است.

شاید در کمالِ شگفتی، فیلترِ ذره‌ای برای مقادیرِ بالاترِ p بدتر عمل می‌کند. وقتی p=1 کاملاً شکست می‌خورد، حتی اگرچه آسان‌ترین حالت به نظر می‌رسد: کثیفی در زمانِ 0 می‌آید و برای همیشه همان‌جا می‌ماند، پس پس از چند گشتِ جهان، ربات باید نقشهٔ کثیفیِ تقریباً کاملی داشته باشد. چرا فیلترِ ذره‌ای در این مورد شکست می‌خورد؟



شکل ۱۴.۲۰ — شبکهٔ بیزِ پویا برای بومی‌سازی و نقشه‌سازیِ هم‌زمان در جهانِ جاروبرقیِ با کثیفیِ تصادفی. مربع‌های کثیف با احتمالِ p پایدار می‌مانند و مربع‌های تمیز با احتمالِ p-1 کثیف می‌شوند. حسگرِ کثیفیِ محلی برای مربعی که ربات در حالِ حاضر در آن است ۹۰٪ دقیق است.

معلوم می‌شود شرطِ نظریِ الزام‌کنندهٔ «احتمال‌های مدل‌های انتقال و حسگر اکیداً بزرگ‌تر از 0 و اکیداً کوچک‌تر از 1 باشند» چیزی فراتر از باریک‌بینیِ صرفاً ریاضی است. اتفاقی که می‌افتد این است که نخست هر ذره در آغاز شاملِ ۴۲ حدس از $P(X0)$ دربارهٔ اینکه کدام مربع‌ها کثیف‌اند و کدام نه است. سپس، حالتِ هر ذره با توجه به مدلِ انتقال در طولِ زمان به جلو پرتاب می‌شود. متأسفانه، مدلِ انتقال برای کثیفیِ قطعی قطعی است: کثیفی دقیقاً همان‌جا که هست می‌ماند. بدین‌ترتیب، حدس‌های اولیه در هر ذره هرگز توسط شواهد به‌روزرسانی نمی‌شوند. شانسِ درست‌بودنِ همهٔ حدس‌های اولیه 2^{-42} یا حدود 2×10^{-13} است، پس احتمالی نزدیک به صفر دارد که هزار ذره (یا حتی یک میلیون ذره) شاملِ ذره‌ای با نقشهٔ کثیفیِ درست باشند. معمولاً، بهترین ذره از میانِ هزار حدود ۳۲ مورد را درست و ۱۰ مورد را غلط می‌گیرد، و معمولاً فقط یکی از این بهترین ذره‌ها خواهد بود، یا شاید چندتایی. یکی از آن بهترین ذره‌ها با گذشتِ زمان بر درستنماییِ کل مسلط خواهد شد و تنوعِ جمعیتِ ذرات فرومی‌پاشد. سپس، چون همهٔ ذرات بر نقشه‌ای یگانه و نادرست توافق دارند، الگوریتم متقاعد می‌شود که آن نقشه درست است و هرگز نظرش را تغییر نمی‌دهد.



شکل ۱۴.۲۱ – (الف) عملکردِ الگوریتمِ استانداردِ فیلترِ ذره‌ای با ۱٫۰۰۰ ذره، که خطایِ RMS در احتمال‌های حاشیه‌ایِ کثیفی را در مقایسه با استنتاجِ دقیق برای مقادیرِ مختلفِ پایداریِ کثیفیِ p نشان می‌دهد. (ب) عملکردِ فیلترِ ذره‌ایِ راو-بلک‌وِل‌شده (۱۰۰ ذره) در مقایسه با حقیقتِ زمینی، برای هم حسِ مکانِ دقیق و هم حسِ دیوارِ نویزی، و با کثیفیِ قطعی. داده‌ها روی ۲۰ اجرا میانگین‌گیری شده‌اند.

خوشبختانه، مسئلهٔ بومی‌سازی و نقشه‌سازیِ هم‌زمان ساختارِ ویژه‌ای دارد: به شرطِ توالیِ مکان‌های ربات، وضعیت‌های کثیفیِ مربع‌های منفرد مستقل از یکدیگرند (تمرین RBPF.۱۴). به‌طورِ مشخص‌تر:

$$P(Dirt_{1,0:t}, \ldots, Dirt_{42,0:t} \mid DirtSensor_{1:t}, WallSensor_{1:t}, Location_{1:t}) = \prod_i P(Dirt_{i,0:t} \mid DirtSensor_{1:t}, Location_{1:t}) \quad (14.24)$$

این بدان معناست که به‌کارگیریِ شگردی آماری به نام «راو-بلک‌وِل‌سازی» (Rao-Blackwellization) سودمند است، که بر این ایدهٔ ساده استوار است که استنتاجِ دقیق همواره دقیق‌تر از نمونه‌برداری است، حتی اگر صرفاً برای زیرمجموعه‌ای از متغیرها باشد (تمرین RAOB.۱۴ را ببینید). برای مسئلهٔ SLAM، فیلترِ ذره‌ای را روی مکانِ ربات اجرا می‌کنیم و سپس، برای هر ذره، استنتاجِ دقیقِ HMM را برای هر مربعِ کثیفی به‌صورتِ مستقل، به شرطِ توالیِ مکانِ موجود در آن ذره، اجرا می‌کنیم. بنابراین هر ذره شاملِ یک مکانِ نمونه‌برداری‌شده به‌علاوهٔ ۴۲ احتمالِ پسینیِ حاشیه‌ایِ دقیق برای ۴۲ مربع است—دقیق، یعنی با این فرض که مسیرِ فرضیِ مکانیِ طی‌شده توسط آن ذره درست باشد. این رویکرد، که «فیلترِ ذره‌ایِ راو-بلک‌وِل‌شده» (Rao-Blackwellized Particle Filter) نامیده می‌شود، حالتِ کثیفیِ قطعی را بدون هیچ دشواری مدیریت می‌کند و به‌تدریج نقشه‌ای دقیق از کثیفی می‌سازد، خواه با حسِ مکانِ دقیق و خواه با حسِ دیوارِ نویزی، چنان‌که در شکل ۱۴.۲۱(ب) نشان داده شده است.

در مواردی که ساختارِ استقلالِ شرطیِ نشان‌داده‌شده در معادلهٔ (14.24) را ارضا نمی‌کنند، راو-بلک‌وِل‌سازی کاربردپذیر نیست. یادداشت‌های پایانِ این فصل به شماری از الگوریتم‌هایی اشاره می‌کنند که برای مدیریتِ مسئلهٔ عمومیِ فیلترکردن با متغیرهای ایستا پیشنهاد شده‌اند. هیچ‌کدام ظرافت و کاربردپذیریِ گستردهٔ فیلترِ ذره‌ای را ندارند، اما چندین مورد در عمل روی رده‌های خاصی از مسائل مؤثرند.

خلاصهٔ فصل

این فصل به مسئلهٔ عمومیِ بازنمایی و استدلال دربارهٔ فرایندهای زمانیِ احتمالاتی پرداخته. نکاتِ اصلی به‌صورتِ زیرند:

- حالتِ متغیرِ جهان با استفاده از مجموعه‌ای از متغیرهای تصادفی برای بازنماییِ حالت در هر نقطه از زمان مدیریت می‌شود.
- بازنمایی‌ها را می‌توان طوری طراحی کرد که (تقریباً) ویژگیِ مارکوف را ارضا کنند، به‌گونه‌ای که آینده با توجه به حال مستقل از گذشته باشد. همراه با فرضِ همگن‌بودنِ فرایند در زمان، این بازنمایی را بسیار ساده می‌کند.
- مدلِ احتمالِ زمانی را می‌توان شاملِ مدلِ انتقالی دانست که تکاملِ حالت را توصیف می‌کند و مدلِ حسگری که فرایندِ مشاهده را توصیف می‌کند.
- تکلیف‌های استنتاجیِ اصلی در مدل‌های زمانی عبارت‌اند از فیلترکردن (برآوردِ حالت)، پیش‌بینی، هموارسازی و محاسبهٔ محتمل‌ترین تبیین. هر یک از این تکلیف‌ها را می‌توان با الگوریتم‌های بازگشتیِ ساده‌ای به دست آورد که زمانِ اجراشان در طولِ توالی خطی است.
- سه خانواده از مدل‌های زمانی عمیق‌تر مطالعه شدند: مدل‌های مارکوفِ پنهان، فیلترهای کالمن و شبکه‌های بیزیِ پویا (که دو تای دیگر را به‌عنوانِ حالت‌های خاص شامل می‌شوند).
- مگر اینکه مفروضاتِ خاصی گرفته شود، مانندِ فیلترهای کالمن، استنتاجِ دقیق با متغیرهای حالتِ بسیار ناشدنی است. در عمل، الگوریتمِ فیلترِ ذره‌ای و فرزندانش خانواده‌ای مؤثر از الگوریتم‌های تقریبی‌اند.

یادداشت‌های کتاب‌شناختی و تاریخی

بسیاری از ایده‌های پایه برای برآوردِ حالتِ سامانه‌های پویا از ریاضی‌دانِ کارل فریدریش گاوس (1809) آمد، که الگوریتمِ قطعیِ کمترین مربعات را برای مسئلهٔ برآوردِ مدارها از مشاهده‌های نجومی صورت‌بندی کرد. آ. آ. مارکوف (1913) آنچه بعداً فرضِ مارکوف نامیده شد را در تحلیلِ فرایندهای تصادفی‌اش توسعه داد؛ او یک زنجیرهٔ مارکوفِ مرتبهٔ اول روی حروفِ متنِ «یوگنی اونگین» برآورد کرد. نظریهٔ عمومیِ زنجیره‌های مارکوف و زمان‌های آمیختن‌شان توسط لوین و همکاران (2008) پوشش داده شده.

کارِ طبقه‌بندی‌شدهٔ قابلِ‌توجهی دربارهٔ فیلترکردن در طولِ جنگِ جهانیِ دوم توسط وینر (1942) برای فرایندهای زمانِ پیوسته و توسط کولموگوروف (1941) برای فرایندهای زمانِ گسسته انجام شد. اگرچه این کار در ۲۰ سالِ بعد به توسعه‌های فناورانهٔ مهمی انجامید، استفاده‌اش از بازنماییِ حوزهٔ بسامد بسیاری از محاسبات را بسیار دست‌وپاگیر می‌کرد. مدل‌سازیِ مستقیمِ فضایِ حالتِ فرایندِ تصادفی ساده‌تر از آب درآمد، چنان‌که پیتر سورلینگ (1959) و رودولف کالمن (1960) نشان دادند. مقالهٔ اخیر آنچه را امروز فیلترِ کالمن می‌نامیم برای استنتاجِ پیشرو در سامانه‌های خطی با

نویزِ گاوسی توصیف کرد؛ با این حال، نتایجِ کالمن قبلاً توسط ستاره‌شناسِ دانمارکیِ توروالد تیله (1880) و فیزیکدانِ روسیِ روسلان استراتونوویچ (1959) به دست آمده بود. پس از بازدید از مرکزِ پژوهشیِ ناسا در ایمز در 1960، کالمن کاربردپذیریِ روش را برای ردیابیِ مسیرِ موشک‌ها دید، و فیلتر بعداً برای مأموریت‌های آپولو پیاده‌سازی شد.

نتایجِ کلیدی دربارهٔ هموارسازی توسط راوخ و همکاران (1965) استخراج شد، و هموارسازِ با نامِ چشمگیرِ «راوخ–تونگ–استریبل» (Rauch–Tung–Striebel) هنوز امروز فنِ استانداردی است. بسیاری از نتایجِ اولیه در گلب (1974) گردآوری شده‌اند. بار-شالوم و فورتمن (1988) برخوردِ مدرن‌تری با طعمِ بیزی و نیز مراجعِ بسیاری به ادبیاتِ وسیعِ موضوع می‌دهند. چتفیلد (1989) و باکس و همکاران (2016) رویکردِ نظریهٔ کنترل به تحلیلِ سری‌های زمانی را پوشش می‌دهند.

مدلِ مارکوفِ پنهان و الگوریتم‌های مرتبطِ استنتاج و یادگیری، شاملِ الگوریتمِ پیشرو–عقب‌رو، توسط باوم و پتری (1966) توسعه یافت. الگوریتمِ ویتربی نخستین بار در (ویتربی، 1967) ظاهر شد. ایده‌های مشابهی نیز به‌طورِ مستقل در جامعهٔ فیلترِ کالمن ظاهر شد (راوخ و همکاران، 1965).

الگوریتمِ پیشرو–عقب‌رو یکی از پیش‌سازهای اصلیِ صورت‌بندیِ عمومیِ الگوریتمِ EM بود (دمپستر و همکاران، 1977)؛ همچنین فصل ۲۰ را ببینید. هموارسازیِ فضایِ ثابت در بیندر و همکاران (1997b) ظاهر می‌شود، همان‌گونه که الگوریتمِ تقسیم‌وغلبهٔ توسعه‌یافته در تمرین ISLE.۱۴. هموارسازیِ با تأخیرِ ثابتِ زمانِ ثابت برای HMMها نخستین بار در راسل و نورویگ (2003) ظاهر شد. HMMها کاربردهای بسیاری در پردازشِ زبان (چارنیاک، 1993)، بازشناسیِ گفتار (رابینر و جوانگ، 1993)، ترجمهٔ ماشینی (اوخ و نی، 2003)، زیست‌شناسیِ محاسباتی (کروگ و همکاران، 1994؛ بالدی و همکاران، 1994)، اقتصادِ مالی (بار و هاموری، 2004) و دیگر حوزه‌ها یافته‌اند. بسط‌های متعددی به مدلِ پایهٔ HMM بوده: برای نمونه، HMM سلسله‌مراتبی (فاین و همکاران، 1998) و HMM لایه‌ای (اولیور و همکاران، 2004) ساختار را به مدل بازمی‌گردانند و متغیرِ حالتِ تکیِ HMMها را جایگزین می‌کنند.

شبکه‌های بیزیِ پویا (DBN) را می‌توان رمزگذاریِ پراکندهٔ فرایندِ مارکوف دانست و نخستین بار در هوش مصنوعی توسط دین و کانازاوا (1989b)، نیکلسون و بردی (1992) و کیارولف (1992) استفاده شدند. کارِ اخیر سامانهٔ شبکهٔ بیزِ HUGIN را برای سازگاری با شبکه‌های بیزیِ پویا بسط می‌دهد. کتابِ دین و ولمن (1991) به رواجِ DBNها و رویکردِ احتمالاتی به برنامه‌ریزی و کنترل در هوش مصنوعی کمک کرد. مورفی (2002) تحلیلِ کاملی از DBNها فراهم می‌کند.

شبکه‌های بیزیِ پویا برای مدل‌کردنِ انواعِ فرایندهای حرکتِ پیچیده در بینواییِ رایانه‌ای محبوب شده‌اند (هوانگ و همکاران، 1994؛ اینتیل و بوبیک، 1999). مانندِ HMMها، در بازشناسیِ گفتار (زوایگ و راسل، 1998؛ لیوسکو و همکاران، 2003)، بومی‌سازیِ ربات (تئوکاروس و همکاران، 2004) و ژنومیکس (مورفی و میان، 1999؛ لی و همکاران، 2011) کاربرد یافته‌اند. حوزه‌های کاربردِ دیگر شاملِ تحلیلِ ژست (سوک و همکاران، 2010)، تشخیصِ خستگیِ راننده (یانگ و همکاران، 2010) و مدل‌سازیِ ترافیکِ شهری (هوفلایتنر و همکاران، 2012) می‌شوند.

پیوندِ میان HMMها و DBNها، و میانِ الگوریتمِ پیشرو–عقب‌رو و انتشارِ شبکهٔ بیزی، توسط اسمیت و همکاران (1997) تبیین شد. یکپارچه‌سازیِ بیشتری با فیلترهای کالمن (و دیگر مدل‌های آماری) در روئیز و غرامانی (1999) ظاهر می‌شود.

رویه‌هایی برای یادگیریِ پارامترها (بیندر و همکاران، 1997a؛ غرامانی، 1998) و ساختارها (فریدمن و همکاران، 1998) DBNها وجود دارد. شبکه‌های بیزیِ زمانِ پیوسته (نودلمن و همکاران، 2002) همتایِ حالتِ گسسته، زمانِ پیوستهٔ DBNها هستند، و از نیاز به انتخابِ مدتِ خاصی برای گام‌های زمانی اجتناب می‌کنند.

نخستین الگوریتم‌های نمونه‌برداری برای فیلترکردن (که «روش‌های مونت‌کارلویِ ترتیبی» (Sequential Monte Carlo) نیز نامیده می‌شوند) در جامعهٔ نظریهٔ کنترل توسط هندشین و مین (1969) توسعه یافت، و ایدهٔ نمونه‌برداریِ مجدد که هستهٔ فیلترِ ذره‌ای است در یک ژورنالِ کنترلِ روسی ظاهر شد (زاریتسکی و همکاران، 1975). بعداً در آمار به‌عنوانِ نمونه‌برداریِ اهمیتِ ترتیبی با نمونه‌برداریِ مجدد، یا SIR، دوباره اختراع شد (روبین، 1988؛ لیو و چن، 1998)، در نظریهٔ کنترل به‌عنوانِ فیلترِ ذره‌ای (گوردون و همکاران، 1993؛ گوردون، 1994)، در هوش مصنوعی به‌عنوانِ بقایِ شایسته‌ترین (کانازاوا و همکاران، 1995)، و در بینایی رایانه‌ای به‌عنوانِ چگالش (ایسارد و بلیک، 1996).

مقالهٔ کانازاوا و همکاران (1995) شاملِ بهبودی به نامِ «معکوس‌سازیِ شواهد» (Evidence Reversal) است که به موجبِ آن حالت در زمانِ t+1 هم بر حالتِ زمانِ t و هم بر شاهدِ زمانِ t+1 شرطی نمونه‌برداری می‌شود. این به شواهد اجازه می‌دهد مستقیماً بر تولیدِ نمونه اثر بگذارد و توسط دوسه (1997) و لیو و چن (1998) اثبات شد خطایِ تقریب را کاهش می‌دهد.

فیلترِ ذره‌ای در بسیاری از حوزه‌ها اعمال شده، از جملهٔ ردیابیِ الگوهای حرکتِ پیچیده در ویدیو (ایسارد و بلیک، 1996)، پیش‌بینیِ بازارِ سهام (دِ فرایتاس و همکاران، 2000) و تشخیصِ عیب روی مریخ‌نوردهای سیاره‌ای (ورما و همکاران، 2004). از زمانِ اختراعش، ده‌ها هزار مقاله دربارهٔ کاربردها و انواعِ الگوریتم منتشر شده. پیاده‌سازی‌های مقیاس‌پذیر روی سخت‌افزارِ موازی مهم شده‌اند؛ هرچند ممکن است فکر کنیم توزیعِ N ذره روی تا N رشتهٔ پردازنده سرراست است، الگوریتمِ پایه به ارتباطِ هم‌زمانِ میانِ رشته‌ها برای گامِ نمونه‌برداریِ مجدد نیاز دارد (هندبی و همکاران، 2010). الگوریتمِ «آبشارِ ذره‌ای» (Particle Cascade؛ پیج و همکاران، 2015) الزامِ هم‌زمانی را حذف می‌کند و به محاسبهٔ موازیِ بسیار سریع‌تری می‌انجامد.

فیلترِ ذره‌ایِ راو-بلک‌ولِ‌شده از دوسه و همکاران (2000) و مورفی و راسل (2001) است؛ کاربردش در مسائلِ عملیِ بومی‌سازی و نقشه‌سازی در رباتیک در فصل ۲۶ توصیف شده. الگوریتم‌های بسیار دیگری برای مدیریتِ مسائلِ فیلترکردنِ عمومی‌تر با متغیرهای ایستا یا تقریباً ایستا پیشنهاد شده‌اند، از جملهٔ الگوریتمِ «نمونه‌برداریِ مجدد–حرکت» (Resample–Move؛ گیلکس و برزوینی، 2001)، الگوریتمِ لیو–وست (لیو و وست، 2001)، فیلترِ استورویک (استورویک، 2002)، فیلترِ پارامترِ بسط‌یافته (ارول و همکاران، 2013) و فیلترِ پارامترِ مفروض (ارول و همکاران، 2017). موردِ اخیر ترکیبی از فیلترِ ذره‌ای با ایدهٔ بسیار قدیمی‌تری به نامِ «فیلترِ چگالیِ مفروض» (Assumed-Density Filter) است. فیلترِ چگالیِ مفروض فرض می‌کند توزیعِ پسینیِ حالت‌ها در زمانِ t به خانوادهٔ خاصِ متناهی‌پارامتری تعلق دارد؛ اگر گام‌های طرح‌ریزی و به‌روزرسانی آن را از این خانواده بیرون ببرند، توزیع به عقب طرح‌ریزی می‌شود تا بهترین تقریب را درونِ خانواده بدهد. برای DBNها، الگوریتمِ بوین–کولر (بوین و همکاران، 1999) و الگوریتمِ «جبههٔ عاملی» (Factored Frontier؛ مورفی و وایس، 2001) فرض می‌کنند توزیعِ پسینی را می‌توان با حاصلِ‌ضربی از عامل‌های کوچک به‌خوبی تقریب زد.

روش‌های MCMC (بخش ۱۳.۴.۲ را ببینید) را می‌توان برای مسئلهٔ فیلترکردن اعمال کرد؛ برای نمونه، نمونه‌برداریِ گیبس را می‌توان مستقیماً روی ِDBN بازشده اعمال کرد. خانوادهٔ «ذره‌ایِ MCMC» (Particle MCMC؛ آندریو و همکاران، 2010؛ لیندستن و همکاران، 2014) MCMC روی مدلِ زمانیِ بازشده را با فیلترِ ذره‌ای برای تولیدِ پیشنهادهای MCMC ترکیب می‌کند؛ هرچند به‌طورِ اثبات‌پذیر در حالتِ عمومی (یعنی با هر دو متغیرِ ایستا و پویا) به توزیعِ پسینیِ درست همگرا می‌شود، الگوریتمی آفلاین است. برای اجتناب از مسئلهٔ افزایشِ زمانِ به‌روزرسانی با رشدِ شبکهٔ بازشده، فیلترِ «ِMCMC میراشونده» (Decayed MCMC؛ مارتی و همکاران، 2002) ترجیح می‌دهد متغیرهای حالتِ جدیدتر را نمونه‌برداری کند، با احتمالی که برای متغیرهای دورتر در گذشته کاهش می‌یابد. کتابِ دوسه و همکاران (2001) مقاله‌های مهمِ بسیاری را دربارهٔ الگوریتم‌های مونت‌کارلویِ ترتیبی (SMC) گردآوری می‌کند، که فیلترِ ذره‌ای مهم‌ترین نمونه‌اش است. آموزش‌های مفیدی از آرولامپالام و همکاران (2002) و دوسه و یوهانسن (2011) هست. همچنین چند نتیجهٔ نظری دربارهٔ شرایطی هست که تحتِ آن روش‌های SMC خطایِ کران‌داری را در مقایسه با پسینِ واقعی به‌صورتِ نامحدود حفظ می‌کنند (کریسان و دوسه، 2002؛ دل مورال، 2004؛ دل مورال و همکاران، 2006).

پانوشت‌ها:

[1] عدم قطعیت روی زمانِ پیوسته را می‌توان با معادله‌های دیفرانسیلِ تصادفی (SDE) مدل کرد. مدل‌های مطالعه‌شده در این فصل را می‌توان تقریب‌های زمانِ گسستهٔ SDEها دانست.

[2] اصطلاحِ «فیلترکردن» به ریشه‌های این مسئله در کارهای اولیهٔ پردازشِ سیگنال اشاره دارد، که در آن مسئله فیلترکردنِ نویزِ یک سیگنال با برآوردِ ویژگی‌های زیربنایی‌اش است.

[3] به‌طورِ خاص، هنگام ردیابیِ جسمِ متحرکی با مشاهده‌های موقعیتِ نادقیق، هموارسازی مسیرِ برآوردشدهٔ نرم‌تری از فیلترکردن می‌دهد—از این رو نام.

[4] اگر روزی دلبخواه به‌عنوانِ t=0 برگزیده شود، منطقی است پیشینِ P(Rain0) را طوری انتخاب کنیم که با توزیعِ مانا هم‌خوان باشد، به همین دلیل ⟨0.5, 0.5⟩ را به‌عنوانِ پیشین برگزیدیم. اگر پیشینِ متفاوتی انتخاب کرده بودیم، توزیعِ مانا همچنان ⟨0.5, 0.5⟩ از آب درمی‌آمد.

[5] توجه کنید این‌ها دقیقاً احتمال‌های محتمل‌ترین مسیرهای رسیدن به حالت‌های Xt با توجه به شواهد نیستند، که می‌شد max_{x1:t-1} P(x1:t-1, Xt | e1:t)؛ اما دو بردار با عاملِ ثابتِ P(e1:t) مرتبط‌اند. تفاوت بی‌اهمیت است، زیرا عملگرِ max به عامل‌های ثابت اهمیت نمی‌دهد. با تعریفِ m1:t بدین‌گونه بازگشتِ کمی ساده‌تری می‌گیریم.

[6] خواننده‌ای که با عملیاتِ پایهٔ بردارها و ماتریس‌ها آشنایی ندارد شاید مایل باشد پیش از ادامهٔ این بخش به پیوستِ الف مراجعه کند.

[7] به‌طورِ دقیق، توزیعِ گاوسی مسئله‌دار است، زیرا احتمالِ غیرصفر به سطح‌های شارژِ منفیِ بزرگ تخصیص می‌دهد. توزیعِ بتا گاهی انتخابِ بهتری برای متغیری است که بردِ محدودی دارد.

فصل ۱۵: برنامه‌نویسی احتمالاتی (Probabilistic Programming)

در این فصل ایدهٔ زبان‌های همگانی برای بازنماییِ دانشِ احتمالاتی و استنتاج در حوزه‌های نامطمئن را توضیح می‌دهیم.

طیفِ بازنمایی‌ها—اتمی، عاملی و ساخت‌یافته—مضمونِ پایداری در هوش مصنوعی بوده. برای مدل‌های قطعی، الگوریتم‌های جستجو فقط بازنماییِ اتمی را فرض می‌کنند؛ CSPها و منطقِ گزاره‌ای بازنمایی‌های عاملی فراهم می‌کنند؛ و منطقِ مرتبهٔ اول و سامانه‌های برنامه‌ریزی از بازنمایی‌های ساخت‌یافته بهره می‌گیرند. قدرتِ بیانیِ فراهم‌شده توسط بازنمایی‌های ساخت‌یافته، مدل‌هایی بسیار موجزتر از توصیف‌های عاملی یا اتمیِ معادل می‌دهد.

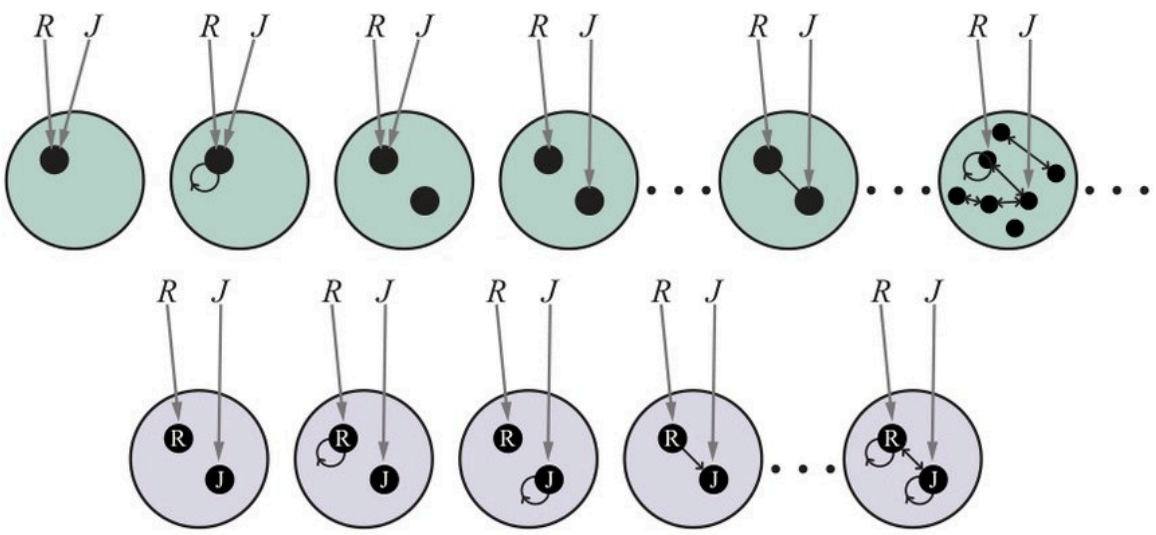
برای مدل‌های احتمالاتی، شبکه‌های بیزیِ توصیف‌شده در فصل‌های ۱۳ و ۱۴ بازنمایی‌های عاملی‌اند: مجموعهٔ متغیرهای تصادفی ثابت و متناهی است و هر یک بردِ ثابتی از مقادیرِ ممکن دارد. این واقعیت کاربردپذیریِ شبکه‌های بیزی را محدود می‌کند، زیرا بازنماییِ شبکهٔ بیزی برای حوزهٔ پیچیده صرفاً بسیار بزرگ است. این ساختِ چنین بازنمایی‌هایی را با دست ناشدنی و یادگیریِ آن‌ها را از هر مقدارِ معقولی داده ناشدنی می‌کند. مسئلهٔ ایجادِ زبانِ صوریِ بیانی برای اطلاعاتِ احتمالاتی برخی از بزرگ‌ترین ذهن‌های تاریخ را به خود مشغول کرده، از جمله گوتفرید لایب‌نیتس (هم‌مخترعِ حسابان)، ژاکوب برنولی (کاشفِ e، حسابِ تغییرات و قانونِ اعدادِ بزرگ)، آگوستوس دی مورگان، جورج بول، چارلز سندرز پیرس (یکی از منطق‌دانانِ اصلیِ قرنِ نوزدهم)، جان مینارد کینز (اقتصاددانِ برجستهٔ قرنِ بیستم) و رودولف کارناپ (یکی از بزرگ‌ترین فیلسوفانِ تحلیلیِ قرنِ بیستم).

مسئله تا دههٔ ۱۹۹۰ در برابرِ این و بسیاری تلاش‌های دیگر مقاومت کرد.

تا حدی به لطفِ توسعهٔ شبکه‌های بیزی، اکنون زبان‌های صوریِ ریاضی‌وارِ زیبا و فوق‌العاده عملی وجود دارند که ایجادِ مدل‌های احتمالاتی برای حوزه‌های بسیار پیچیده را اجازه می‌دهند. این زبان‌ها به همان معنا همگانی‌اند که ماشین‌های تورینگ همگانی‌اند: می‌توانند هر مدلِ احتمالِ محاسبه‌پذیری را بازنمایی کنند، همان‌گونه که ماشین‌های تورینگ می‌توانند هر تابعِ محاسبه‌پذیری را بازنمایی کنند. علاوه بر این، این زبان‌ها با الگوریتم‌های استنتاجِ همه‌منظوره همراه‌اند، تقریباً مشابهِ الگوریتم‌های استنتاجِ منطقیِ صدا و کامل مانندِ قطع.

دو مسیر برای واردکردنِ قدرتِ بیانی به نظریهٔ احتمال هست. نخستین از طریقِ منطق است: طراحیِ زبانی که احتمال‌ها را روی جهان‌های ممکنِ مرتبهٔ اول تعریف می‌کند، به‌جای جهان‌های ممکنِ گزاره‌ایِ شبکه‌های بیز. این مسیر در بخش‌های ۱۵.۱ و ۱۵.۲ پوشش داده می‌شود، و بخش ۱۵.۳ حالتِ خاصِ استدلالِ زمانی را پوشش می‌دهد. مسیرِ دوم از طریقِ زبان‌های برنامه‌نویسیِ سنتی است: عناصرِ تصادفی—مثلاً انتخاب‌های تصادفی—را به چنین زبان‌هایی معرفی می‌کنیم، و برنامه‌ها را به‌عنوانِ تعریف‌کنندهٔ توزیع‌های احتمال روی دنباله‌های اجرایِ خودشان می‌بینیم. این رویکرد در بخش ۱۵.۴ پوشش داده شده.

هر دو مسیر به «زبانِ برنامه‌نویسیِ احتمالاتی» (Probabilistic Programming Language یا PPL) می‌انجامند. مسیرِ نخست به PPLهای اظهاری می‌انجامد، که تقریباً همان رابطه را با PPLهای عمومی دارند که برنامه‌نویسیِ منطقی (فصل ۹) با زبان‌های برنامه‌نویسیِ عمومی دارد.



شکل ۱۵.۱ – بالا: برخی اعضای مجموعهٔ همهٔ جهان‌های ممکن برای زبانی با دو نمادِ ثابت، R و J، و یک نمادِ رابطهٔ دوتایی، تحتِ معناشناسیِ استانداردِ منطقِ مرتبهٔ اول. پایین: جهان‌های ممکن تحتِ معناشناسیِ پایگاه‌داده. تفسیرِ نمادهای ثابت ثابت است و برای هر نمادِ ثابت شیءِ متمایزی هست.

۱۵.۱ مدل‌های احتمالِ رابطه‌ای

از فصل ۱۲ به یاد آورید مدلِ احتمال مجموعهٔ Ω از جهان‌های ممکن را با احتمالِ P(ω) برای هر جهانِ ω تعریف می‌کند. برای شبکه‌های بیزی، جهان‌های ممکن تخصیص‌های مقادیر به متغیرها هستند؛ برای حالتِ بولی به‌طورِ خاص، جهان‌های ممکن یکسان با جهان‌های منطقِ گزاره‌ای‌اند.

برای مدلِ احتمالِ مرتبهٔ اول، پس، به نظر می‌رسد به جهان‌های ممکنِ منطقِ مرتبهٔ اول نیاز داریم—یعنی مجموعه‌ای از اشیا با روابطی میان‌شان و تفسیری که نمادهای ثابت را به اشیا، نمادهای گزاره‌ای را به روابط و نمادهای تابع را به توابع روی آن اشیا نگاشت می‌کند. (بخش ۸.۲ را ببینید.) مدل همچنین باید برای هر چنین جهانِ ممکنی احتمالی تعریف کند، همان‌گونه که شبکهٔ بیزی برای هر تخصیصِ مقادیر به متغیرها احتمالی تعریف می‌کند. فرض کنیم، لحظه‌ای، دریافته‌ایم چگونه این کار را بکنیم. سپس، مثلِ همیشه (صفحهٔ ۳۸۹ را ببینید)، می‌توانیم احتمالِ هر جملهٔ منطقِ مرتبهٔ اولِ φ (فی) را به‌صورتِ جمعی روی جهان‌های ممکنی که در آن‌ها صادق است به دست آوریم:

$$P(\varphi) = \sum_{\omega : \varphi \text{ در } \omega \text{ صادق است}} P(\omega) \qquad (15.1)$$

احتمال‌های شرطیِ P(φ|e) را می‌توان به‌طورِ مشابه به دست آورد، پس اصولاً می‌توانیم هر پرسشی را که می‌خواهیم از مدل‌مان بپرسیم—و پاسخ بگیریم. تا این‌جا خوب است.

با این حال، مسئله‌ای هست: مجموعهٔ مدل‌های مرتبهٔ اول نامتناهی است. این را به‌صراحت در شکل ۸.۴ صفحهٔ ۲۵۹ دیدیم، که آن را دوباره در شکل ۱۵.۱ (بالا) نشان می‌دهیم. این بدان معناست (۱) جمع در معادلهٔ (15.1) می‌تواند ناشدنی باشد، و (۲) مشخص‌کردنِ توزیعِ کامل و سازگاری روی مجموعه‌ای نامتناهی از جهان‌ها می‌تواند بسیار دشوار باشد.

در این بخش، با درنظرگرفتنِ «معناشناسیِ پایگاه‌داده» تعریف‌شده در بخش ۸.۲.۸ (صفحهٔ ۲۶۴) از این مسئله اجتناب می‌کنیم. معناشناسیِ پایگاه‌داده «فرضِ نام‌های یکتا» را می‌سازد—در این‌جا، آن را برای نمادهای ثابت اتخاذ می‌کنیم. همچنین «بستارِ دامنه» را فرض می‌کند—هیچ شیءِ دیگری فراتر از آن‌هایی که نام برده شده‌اند وجود ندارد. سپس می‌توانیم مجموعهٔ متناهی از جهان‌های ممکن را با این تضمین کنیم که مجموعهٔ اشیای هر جهان دقیقاً مجموعهٔ نمادهای ثابتِ استفاده‌شده باشد؛ چنان‌که در شکل ۱۵.۱ (پایین) نشان داده شده، هیچ عدم قطعیتی دربارهٔ نگاشتِ نمادها به اشیا یا دربارهٔ اشیای موجود نیست.

مدل‌هایی را که بدین‌گونه تعریف می‌شوند «مدل‌های احتمالِ رابطه‌ای» (Relational Probability Models) یا RPM می‌نامیم.[1] مهم‌ترین تفاوت میان معناشناسیِ RPMها و معناشناسیِ پایگاه‌دادهٔ معرفی‌شده در بخش ۸.۲.۸ این است که RPMها «فرضِ جهانِ بسته» را نمی‌سازند—در سامانهٔ استدلالِ احتمالاتی نمی‌توانیم صرفاً فرض کنیم هر واقعیتِ ناشناخته‌ای کاذب است.

۱۵.۱.۱ نحو و معناشناسی

با مثالِ ساده‌ای شروع کنیم: فرض کنیم خرده‌فروشِ آنلاینِ کتاب می‌خواهد ارزیابی‌های کلیِ محصولات را بر اساسِ توصیه‌هایی که از مشتریانش دریافت می‌کند فراهم کند. ارزیابی به شکلِ توزیعِ پسینی روی کیفیتِ کتاب، با توجه به شواهدِ موجود، خواهد بود. ساده‌ترین راه‌حل آن است که ارزیابی بر میانگینِ توصیه مبتنی باشد، شاید با واریانسی که توسط شمارِ توصیه‌ها تعیین می‌شود، اما این واقعیت را در نظر نمی‌گیرد که برخی مشتریان مهربان‌تر از دیگران‌اند و برخی کمتر از دیگران صادق‌اند. مشتریانِ مهربان تمایل دارند حتی به کتاب‌های نسبتاً متوسط هم توصیهٔ بالایی بدهند، در حالی که مشتریانِ ناصادق به دلایلی غیر از کیفیت توصیه‌های بسیار بالا یا بسیار پایین می‌دهند—ممکن است برای تبلیغِ کتاب‌های ناشری پول گرفته باشند.[2] برای مشتریِ تکیِ C1 که کتابِ تکیِ B1 را توصیه می‌کند، شبکهٔ بیز ممکن است شبیهِ شکل ۱۵.۲(الف) باشد. (درست همان‌گونه که در بخش ۹.۱، عبارت‌های با پرانتز مانند Honest(C1) نمادهای فانتزی‌اند—در این مورد، نام‌های فانتزی برای متغیرهای تصادفی.) با دو مشتری و دو کتاب، شبکهٔ بیز شبیهِ شکل ۱۵.۲(ب) می‌شود. برای شمارِ بزرگ‌ترِ کتاب‌ها و مشتریان، مشخص‌کردنِ شبکهٔ بیز با دست بسیار غیرعملی است.

خوشبختانه، شبکه ساختارِ تکراریِ زیادی دارد. هر متغیرِ Recommendation(c,b) والدهایش را متغیرهای Honest(c)، Kindness(c) و Quality(b) دارد. علاوه بر این، جدول‌های احتمالِ شرطی (CPT) برای همهٔ متغیرهای Recommendation(c,b) یکسان‌اند، همان‌گونه که برای همهٔ متغیرهای Honest(c) و مانندِ آن. وضعیت به‌نظرِ زبانِ مرتبهٔ اول دوخته شده است. می‌خواهیم چیزی مانندِ این بگوییم:

```
Recommendation(c,b) ~ RecCPT(Honest(c), Kindness(c), Quality(b))
```

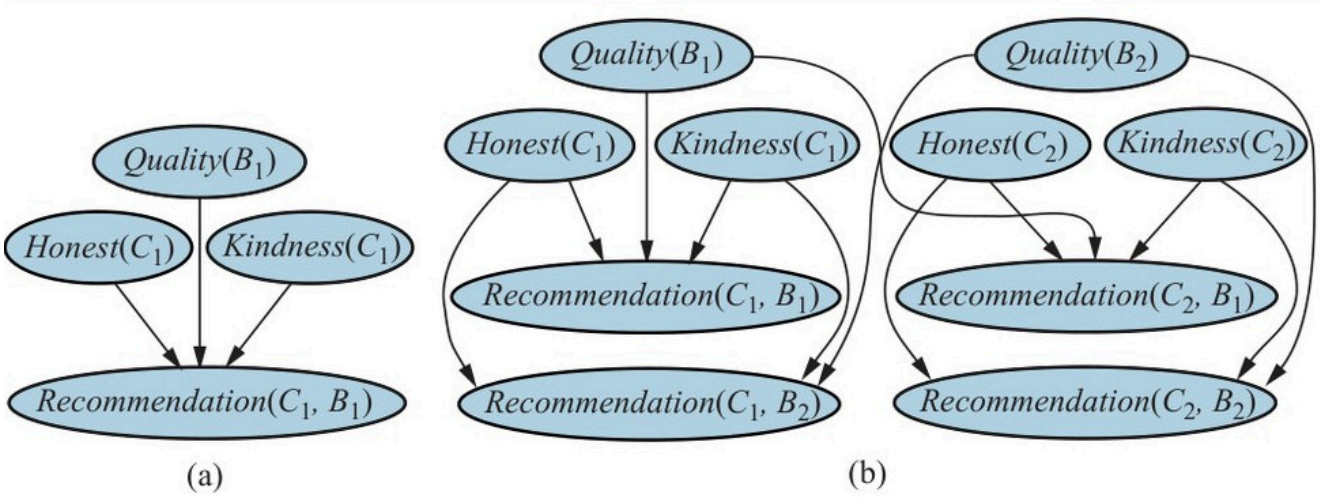
که بدان معناست توصیهٔ مشتری برای کتابی به‌صورتِ احتمالاتی به صداقت و مهربانیِ مشتری و کیفیتِ کتاب، طبقِ CPT ثابتی، بستگی دارد.

مانندِ منطقِ مرتبهٔ اول، RPMها نمادهای ثابت، تابع و گزاره‌ای دارند. همچنین برای هر تابع یک «امضای نوع» (Type Signature) فرض می‌کنیم—یعنی مشخصاتی از نوعِ هر آرگومان و مقدارِ تابع. (اگر نوعِ هر شیء معلوم باشد، بسیاری از جهان‌های ممکنِ جعلی توسط این سازوکار حذف می‌شوند؛ برای نمونه، لازم نیست نگرانِ مهربانیِ هر کتاب، یا توصیه‌کردنِ کتاب‌ها به مشتریان و مانندِ آن باشیم.) برای حوزهٔ توصیهٔ کتاب، نوع‌ها Customer و Book هستند و امضاهای نوع برای توابع و گزاره‌ها به‌صورتِ زیرند:

```
Honest : Customer → {true, false}
Kindness : Customer → {1, 2, 3, 4, 5}
Quality : Book → {1, 2, 3, 4, 5}
Recommendation : Customer × Book → {1, 2, 3, 4, 5}
```

نمادهای ثابت هر نامِ مشتری و کتابی خواهند بود که در مجموعهٔ دادهٔ خرده‌فروش ظاهر می‌شود. در مثالِ داده‌شده در شکل ۱۵.۲(ب)، این‌ها C1، C2 و B1، B2 بودند.

با توجه به ثابت‌ها و نوع‌هایشان، همراه با توابع و امضاهای نوعِشان، «متغیرهای تصادفیِ پایه» (Basic Random Variables) RPM با مصداق‌کردنِ هر تابع با هر ترکیبِ ممکنِ اشیا به دست می‌آیند. برای مدلِ توصیهٔ کتاب، متغیرهای تصادفیِ پایه شاملِ Honest(C1)، Quality(B2)، Recommendation(C1,B2) و مانندِ آن‌اند. این‌ها دقیقاً متغیرهای ظاهرشده در شکل ۱۵.۲(ب) هستند. چون هر نوع فقط شمارِ متناهی نمونه دارد (به لطفِ فرضِ بستارِ دامنه)، شمارِ متغیرهای تصادفیِ پایه نیز متناهی است. برای کامل‌کردنِ RPM، باید وابستگی‌هایی را بنویسیم که بر این متغیرهای تصادفی حکومت می‌کنند. برای هر تابع یک گزارهٔ وابستگی هست، که در آن هر آرگومانِ تابع متغیرِ منطقی است (یعنی متغیری که روی اشیا می‌رود، مانندِ منطقِ مرتبهٔ اول).



شکل ۱۵.۲ – (الف) شبکهٔ بیز برای مشتریِ تکیِ C1 که کتابِ تکیِ B1 را توصیه می‌کند. Honest(C1) بولی است، در حالی که متغیرهای دیگر مقادیرِ صحیح از 1 تا 5 دارند. (ب) شبکهٔ بیز با دو مشتری و دو کتاب.

برای نمونه، وابستگیِ زیر بیان می‌کند که برای هر مشتریِ c، احتمالِ پیشینیِ صداقت 0.99 صادق و 0.01 کاذب است:

```
Honest(c) ~ ⟨0.99, 0.01⟩
```

به‌طورِ مشابه، می‌توانیم احتمال‌های پیشینی را برای مقدارِ مهربانیِ هر مشتری و کیفیتِ هر کتاب، هر یک در مقیاسِ 1–5، بیان کنیم:

```
Kindness(c) ~ ⟨0.1, 0.1, 0.2, 0.3, 0.3⟩
Quality(b) ~ ⟨0.05, 0.2, 0.4, 0.2, 0.15⟩
```

برای وابستگیِ Recommendation، ممکن است بنویسیم:

```
Recommendation(c,b) ~ if Honest(c)
then HonestRecCPT(Kindness(c), Quality(b))
else ⟨0.4, 0.1, 0.0, 0.1, 0.4⟩
```

این نوع وابستگی ممکن است شبیهِ گزارهٔ if–then–else معمولی در زبانِ برنامه‌نویسی باشد، اما تفاوتِ کلیدی‌ای هست: موتورِ استنتاج لزوماً مقدارِ آزمونِ شرطی را نمی‌داند، زیرا Honest(c) متغیرِ تصادفی است.

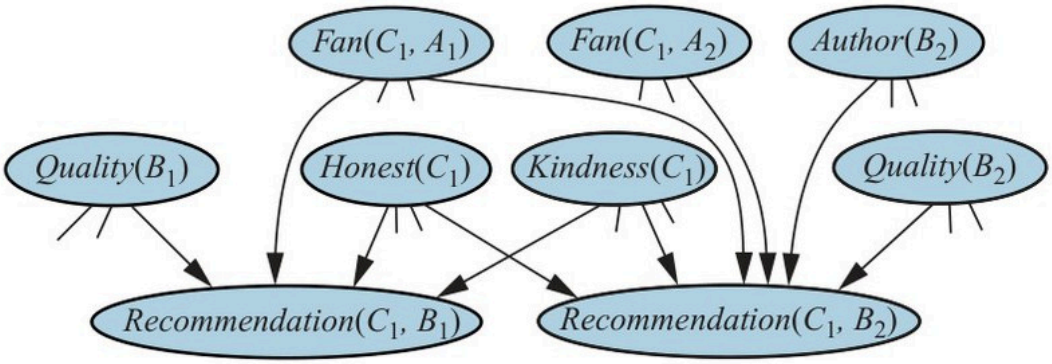
می‌توانیم این مدل را به راه‌های بی‌پایانی برای واقع‌گرایانه‌ترشدن بسط دهیم. برای نمونه، فرض کنیم مشتریِ صادقی که طرفدارِ نویسندهٔ کتاب است همیشه به کتاب 5 می‌دهد، صرف‌نظر از کیفیت:

```
Recommendation(c,b) ~ if Honest(c)
then if Fan(c, Author(b))
then Exactly(5)
else HonestRecCPT(Kindness(c), Quality(b))
else ⟨0.4, 0.1, 0.0, 0.1, 0.4⟩
```

باز، آزمونِ شرطیِ Fan(c, Author(b)) ناشناخته است، اما اگر مشتری فقط به کتاب‌های نویسندهٔ خاصی 5 بدهد و در غیرِ این صورت به‌طورِ خاص مهربان نباشد، آنگاه احتمالِ پسینیِ اینکه مشتری طرفدارِ آن نویسنده باشد بالا خواهد بود. علاوه بر این، توزیعِ پسینی تمایل خواهد داشت 5های مشتری را در ارزیابیِ کیفیتِ کتاب‌های آن نویسنده تخفیف دهد.

در این مثال، به‌صورتِ ضمنی فرض کردیم مقدارِ Author(b) برای هر b معلوم است، اما ممکن است چنین نباشد. سامانه چگونه می‌تواند دربارهٔ اینکه آیا، مثلاً، C1 طرفدارِ Author(B2) است استدلال کند وقتی Author(B2) ناشناخته است؟ پاسخ این است که سامانه ممکن است مجبور شود دربارهٔ همهٔ نویسنده‌های ممکن استدلال کند. فرض کنیم (برای سادگی) فقط دو نویسنده، A1 و A2 وجود دارند. سپس Author(B2) متغیرِ تصادفی با دو مقدارِ ممکنِ A1 و A2 است و والدِ Recommendation(C1,B2) است. متغیرهای Fan(C1,A1) و Fan(C1,A2) نیز والدند. توزیعِ شرطی برای Recommendation(C1,B2) سپس اساساً «چندگستر» (Multiplexer) است که در آن والدِ Author(B2) به‌عنوانِ

انتخاب‌گر عمل می‌کند تا برگزیند کدام‌یک از Fan(C1,A1) و Fan(C1,A2) واقعاً اجازه دارد بر توصیه اثر بگذارد. قطعه‌ای از شبکهٔ بیزِ معادل در شکل ۱۵.۳ نشان داده شده. عدم قطعیت در مقدارِ Author(B2)، که بر ساختارِ وابستگیِ شبکه اثر می‌گذارد، نمونه‌ای از «عدم قطعیتِ رابطه‌ای» (Relational Uncertainty) است.[3]



شکل ۱۵.۳ — قطعه‌ای از شبکهٔ بیزِ معادل برایِ RPM توصیهٔ کتاب وقتی Author(B2) ناشناخته است.

اگر کنجکاوید سامانه چگونه می‌تواند بفهمد نویسندهٔ B2 کیست: امکانِ زیر را در نظر بگیرید که سه مشتریِ دیگر طرفدارِ A1 هستند (و هیچ نویسندهٔ محبوبِ مشترکِ دیگری ندارند) و هر سه به B2 امتیازِ 5 داده‌اند، حتی اگرچه بیشترِ مشتریانِ دیگر آن را بسیار افتضاح می‌یابند. در آن صورت، بسیار محتمل است A1 نویسندهٔ B2 باشد. ظهورِ استدلالِ پیچیده‌ای مانندِ این از مدلِ RPM ای با فقط چند خط، نمونهٔ جذابی از گسترشِ تأثیرهای احتمالاتی از میانِ تارِ به‌هم‌پیوستگی‌های اشیای مدل است. هرچه وابستگی‌ها و اشیای بیشتری افزوده شوند، تصویری که توزیعِ پسینی منتقل می‌کند اغلب واضح‌تر و واضح‌تر می‌شود.

۱۵.۱.۲ نمونه: رتبه‌بندیِ سطحِ مهارتِ بازیکنان

بسیاری از بازی‌های رقابتی معیارِ عددیِ سطحِ مهارتِ بازیکنان دارند، که گاهی «رتبه» (Rating) نامیده می‌شود. شاید شناخته‌شده‌ترین «رتبهٔ الو» (Elo Rating) برای شطرنج‌بازان باشد، که به یک مبتدیِ نمونه حدود ۸۰۰ و به قهرمانِ جهان معمولاً جایی بالای ۲۸۰۰ رتبه می‌دهد. اگرچه رتبه‌های الو پایهٔ آماری دارند، برخی عناصرِ موردی هم دارند. می‌توانیم طرحِ رتبه‌بندیِ بیزی را چنین توسعه دهیم: هر بازیکنِ i سطحِ مهارتِ زیربناییِ Skill(i) دارد؛ در هر بازیِ g، عملکردِ واقعیِ i برابر Performance(i,g) است، که ممکن است از سطحِ مهارتِ زیربنایی منحرف شود؛ و برندهٔ g بازیکنی است که عملکردش در g بهتر است. به‌عنوانِ RPM، مدل به‌صورتِ زیر است:

```
Skill(i) ~ N(μ, σ²)
Performance(i,g) ~ N(Skill(i), β²)
Win(i,j,g) = if Game(g,i,j) then (Performance(i,g) > Performance(j,g))
```

که در آن β^2 واریانسِ عملکردِ واقعیِ بازیکن در هر بازیِ خاص نسبت به سطحِ مهارتِ زیربناییِ اوست. با مجموعه‌ای از بازیکنان و بازی‌ها، و نیز پیامدهای برخی از بازی‌ها، موتورِ استنتاجِ RPM می‌تواند توزیعِ پسینی را روی مهارتِ هر بازیکن و پیامدِ احتمالیِ هر بازیِ اضافی که ممکن است انجام شود محاسبه کند.

برای بازی‌های تیمی، به‌عنوانِ تقریبِ نخست، فرض می‌کنیم عملکردِ کلیِ تیمِ t در بازیِ g جمعِ عملکردهای فردیِ بازیکنانِ t است:

```
TeamPerformance(t,g) = ∑_{i∈t} Performance(i,g)
```

حتی اگرچه عملکردهای فردی برای موتورِ رتبه‌بندی قابلِ مشاهده نیستند، سطح‌های مهارتِ بازیکنان را همچنان می‌توان از نتایجِ چند بازی برآورد کرد، تا زمانی که ترکیب‌های تیم در بازی‌ها متفاوت باشند. موتورِ رتبه‌بندیِ «TrueSkill» مایکروسافت از این مدل، همراه با الگوریتمِ استنتاجِ تقریبیِ کارآمدی، برای خدمت‌رسانی به صدها میلیون کاربر در هر روز استفاده می‌کند. این مدل را می‌توان به راه‌های متعددی بسط داد. برای نمونه، ممکن است فرض کنیم بازیکنانِ ضعیف‌تر واریانسِ بیشتری در عملکردشان دارند؛ ممکن است نقشِ بازیکن را در تیم بگنجانیم؛ و ممکن است انواعِ خاصِ عملکرد و مهارت—مثلاً دفاع و حمله—را برای بهبودِ ترکیبِ تیم و دقتِ پیش‌بینی در نظر بگیریم.

۱۵.۱.۳ استنتاج در مدل‌های احتمالِ رابطه‌ای

سرراست‌ترین رویکرد برای استنتاج در RPMها صرفاً ساختِ شبکهٔ بیزیِ معادل، با توجه به نمادهای ثابتِ شناخته‌شدهٔ متعلق به هر نوع، است. با B کتاب و C مشتری، مدلِ پایهٔ داده‌شدهٔ پیشین را می‌توان با حلقه‌های ساده ساخت:[4]

```
برای b از 1 تا B:
  گرهٔ Quality_b را بدون والد بیفزا، با پیشینِ ⟨0.05, 0.2, 0.4, 0.2, 0.15⟩
برای c از 1 تا C:
  گرهٔ Honest_c را بدون والد بیفزا، با پیشینِ ⟨0.99, 0.01⟩
  گرهٔ Kindness_c را بدون والد بیفزا، با پیشینِ ⟨0.1, 0.1, 0.2, 0.3, 0.3⟩
برای b از 1 تا B:
  گرهٔ Recommendation_{c,b} را بیفزا، با والدهای Honest_c, Kindness_c, Quality_b
    و توزیعِ شرطیِ RecCPT(Honest_c, Kindness_c, Quality_b)
```

این فن «زمینه‌سازی» (Grounding) یا «بازکردن» (Unrolling) نامیده می‌شود؛ همتایِ دقیقِ گزاره‌سازی برای منطقِ مرتبهٔ اول است (صفحهٔ ۲۸۰). عیبِ آشکارش این است که شبکهٔ بیزیِ حاصل ممکن است بسیار بزرگ باشد. علاوه بر این، اگر اشیای نامزدِ بسیاری برای رابطه یا تابعِ ناشناخته‌ای وجود داشته باشد—برای نمونه، نویسندهٔ ناشناختهٔ B2—آنگاه برخی متغیرهای شبکه ممکن است والدهای بسیاری داشته باشند. خوشبختانه، اغلب می‌توان از تولیدِ کلِ شبکهٔ بیزِ ضمنی اجتناب کرد. چنان‌که در بحثِ الگوریتمِ حذفِ متغیر در صفحهٔ ۴۳۳ دیدیم، هر متغیری که نیای متغیرِ پرس‌وجو یا شاهد نیست برای پرس‌وجو نامربوط است. علاوه بر این، اگر پرس‌وجو با توجه به شواهد به‌صورتِ شرطی مستقل از متغیری باشد، آن متغیر نیز نامربوط است. پس، با زنجیره‌زدن در مدل با شروع از پرس‌وجو و شواهد، می‌توانیم فقط مجموعهٔ متغیرهای مربوط به پرس‌وجو را شناسایی کنیم. این‌ها تنها متغیرهایی هستند که برای ایجادِ قطعه‌ای بالقوه بسیار کوچک از شبکهٔ بیزِ ضمنی باید مصداق‌سازی شوند. استنتاج در این قطعه همان پاسخِ استنتاج در کلِ شبکهٔ بیزِ ضمنی را می‌دهد.

راهِ دیگری برای بهبودِ کاراییِ استنتاج از حضورِ زیرساختارِ تکراری در شبکهٔ بیزِ بازشده می‌آید. این بدان معناست بسیاری از عامل‌های ساخته‌شده در طولِ حذفِ متغیر (و انواعِ مشابهِ جدول‌های ساخته‌شده توسط الگوریتم‌های خوشه‌بندی) یکسان خواهند بود؛ طرح‌های کش‌کردنِ مؤثر سرعت‌هایی سه‌مرتبه‌ای برای شبکه‌های بزرگ داده‌اند. سوم، الگوریتم‌های استنتاجِ MCMC وقتی روی RPMها با عدم قطعیتِ رابطه‌ای اعمال می‌شوند ویژگی‌های جالبی دارند. MCMC با نمونه‌برداری از جهان‌های ممکنِ کامل کار می‌کند، پس در هر حالت ساختارِ رابطه‌ای کاملاً معلوم است. در مثالِ داده‌شدهٔ پیشین، هر حالتِ MCMC مقدارِ Author(B2) را مشخص می‌کرد و از این رو سایر نویسنده‌های بالقوه دیگر والدِ گره‌های توصیه برای B2 نیستند. برای MCMC، پس، عدم قطعیتِ رابطه‌ای هیچ افزایشی در پیچیدگیِ شبکه ایجاد نمی‌کند؛ در عوض، فرایندِ MCMC شاملِ انتقال‌هایی است که ساختارِ رابطه‌ای، و از این رو ساختارِ وابستگی، شبکهٔ بازشده را تغییر می‌دهند. سرانجام، در برخی موارد ممکن است بتوان کلاً از زمینه‌سازیِ مدل اجتناب کرد. اثبات‌کننده‌های قضیهٔ قطع و سامانه‌های برنامه‌نویسیِ منطقی با مصداق‌کردنِ متغیرهای منطقی فقط در حدِ لازم برای پیش‌بردنِ استنتاج از گزاره‌سازی اجتناب می‌کنند؛ یعنی فرایندِ استنتاج را بالاتر از سطحِ جمله‌های گزاره‌ایِ زمینی می‌برند و هر گامِ «برافراشته» (Lifted) کارِ بسیاری گامِ زمینی را انجام می‌دهد.

همین ایده را می‌توان در استنتاجِ احتمالاتی اعمال کرد. برای نمونه، در الگوریتمِ حذفِ متغیر، «عاملِ برافراشته» می‌تواند کلِ شبکه‌ای از عامل‌های زمینی را بازنمایی کند که احتمال‌ها را به متغیرهای تصادفیِ RPM تخصیص می‌دهند، جایی که آن متغیرهای تصادفی فقط در نمادهای ثابتِ استفاده‌شده برای ساختشان تفاوت دارند. جزئیاتِ این روش فراتر از حوصلهٔ این کتاب است، اما مراجع در پایانِ فصل داده شده‌اند.

۱۵.۲ مدل‌های احتمالِ جهانِ باز

قبلاً استدلال کردیم معناشناسیِ پایگاه‌داده برای وضعیت‌هایی مناسب است که دقیقاً مجموعهٔ اشیای مربوطِ موجود را می‌دانیم و می‌توانیم آن‌ها را بدونِ ابهام شناسایی کنیم. (به‌طورِ خاص، همهٔ مشاهده‌های مربوط به یک شیء به‌درستی با نمادِ ثابتی که نامش را می‌برد مرتبط می‌شوند.) اما در بسیاری از محیط‌های دنیایِ واقعی، این مفروضات صرفاً نامعتبرند.

برای نمونه، خرده‌فروشِ کتاب ممکن است از «شابک» (ISBN) به‌عنوانِ نمادِ ثابت برای نامیدنِ هر کتاب استفاده کند، حتی اگرچه «کتابِ» منطقیِ معینی (مثلاً «بر باد رفته») ممکن است چند شابکِ متناظر با جلدِ سخت، شومیز، حروفِ درشت، تجدیدچاپ‌ها و مانندِ آن داشته باشد. منطقی بود توصیه‌ها را روی چند شابک تجمیع کنیم، اما خرده‌فروش ممکن است مطمئن نباشد کدام شابک‌ها واقعاً همان کتاب‌اند. (توجه کنید نسخه‌های فردیِ کتاب را عینیت نمی‌بخشیم، که ممکن است برای فروشِ کتابِ دست دوم، فروشِ خودرو و مانندِ آن لازم باشد.) بدتر از آن، هر مشتری با شناسهٔ ورود شناسایی می‌شود، اما مشتریِ ناصادق ممکن است هزاران شناسه داشته باشد! در حوزهٔ امنیتِ رایانه، این شناسه‌های چندگانه «سیبیل» (Sybils) نامیده می‌شوند و استفاده از آن‌ها برای به‌هم‌ریختنِ سامانهٔ اعتبار «حملهٔ سیبیل» (Sybil Attack) نامیده می‌شود.[5] بدین‌ترتیب، حتی کاربردِ ساده‌ای در حوزهٔ نسبتاً خوش‌تعریفِ برخط، هم

«عدم قطعیتِ وجودی» (Existence Uncertainty) را در بر دارد (کتاب‌ها و مشتریانِ واقعیِ زیربناییِ داده‌های مشاهده‌شده چه‌اند) و هم «عدم قطعیتِ هویتی» (Identity Uncertainty) را (کدام عبارتِ منطقی واقعاً به همان شیء اشاره دارد).

پدیده‌های عدم قطعیتِ وجودی و هویتی بسیار فراتر از کتاب‌فروشی‌های برخط گسترش می‌یابند. در واقع فراگیرند:

- سامانهٔ بینایی نمی‌داند چه چیزی، اگر چیزی، پشتِ پیچِ بعدی وجود دارد، و ممکن است نداند شیئی که حالا می‌بیند همانی است که چند دقیقهٔ پیش دیده.
- سامانهٔ فهمِ متن از پیش نمی‌داند چه نهادهایی در متنی برجسته خواهند بود، و باید استدلال کند آیا عبارت‌هایی مانند «مریم»، «دکتر اسمیت»، «او»، «متخصصِ قلبش»، «مادرش» و مانندِ آن به همان شیء اشاره دارند.
- تحلیلگرِ اطلاعاتی که در جستجوی جاسوس‌هاست هرگز نمی‌داند واقعاً چند جاسوس هست و فقط می‌تواند حدس بزند آیا نام‌های مستعارِ گوناگون، شماره‌تلفن‌ها و مشاهده‌ها به همان فرد تعلق دارند.

در واقع، بخشِ بزرگی از شناختِ انسانی به نظر می‌رسد مستلزمِ یادگیریِ اینکه چه اشیایی وجود دارند و توانایی اتصالِ مشاهده‌ها—که تقریباً هرگز با شناسه‌های یکتای متصل نمی‌آیند—به اشیای فرضیِ جهان است.

بدین‌ترتیب، باید بتوانیم «مدلِ احتمالِ جهانِ باز» (Open Universe Probability Model یا OUPM) را بر اساسِ معناشناسیِ استانداردِ منطقِ مرتبهٔ اول تعریف کنیم، چنان‌که در بالای شکل ۱۵.۱ نشان داده شده. زبانِ OUPMها راهی برای نوشتنِ آسانِ چنین مدل‌هایی فراهم می‌کند و در عینِ حال توزیعِ احتمالِ یکتا و سازگاری را روی فضایِ نامتناهیِ جهان‌های ممکن تضمین می‌کند.

۱۵.۲.۱ نحو و معناشناسی

ایدهٔ پایه این است که بفهمیم شبکه‌های بیزیِ معمولی و RPMها چگونه مدلِ احتمالِ یکتایی را تعریف می‌کنند و آن بینش را به محیطِ مرتبهٔ اول منتقل کنیم. در اصل، شبکهٔ بیز هر جهانِ ممکن را، رخداد به رخداد، در ترتیبِ توپولوژیکِ تعریف‌شده توسط ساختارِ شبکه تولید می‌کند، که در آن هر رخداد تخصیصِ مقداری به متغیری است. RPM این را به مجموعه‌های کاملِ رخدادها بسط می‌دهد، که با مصداق‌های ممکنِ متغیرهای منطقی در گزاره یا تابعِ معینی تعریف می‌شوند. OUPMها فراتر می‌روند و گام‌های زایشی‌ای را اجازه می‌دهند که اشیا را به جهانِ ممکنِ درحالِ‌ساخت می‌افزایند، جایی که شمار و نوعِ اشیا ممکن است به اشیای ازپیش‌موجودِ آن جهان و ویژگی‌ها و روابطشان بستگی داشته باشد.

یعنی رخدادی که تولید می‌شود تخصیصِ مقداری به متغیری نیست، بلکه خودِ وجودِ اشیاست. یکی از راه‌های انجامِ این در OUPMها فراهم‌کردنِ «گزاره‌های شمارِ» (Number Statements) است که توزیع‌های شرطی روی شمارِ اشیای انواعِ گوناگون را مشخص می‌کنند. برای نمونه، در حوزهٔ توصیهٔ کتاب، ممکن است بخواهیم میان مشتریان (افرادِ واقعی) و شناسه‌های ورودشان تمایز بگذاریم. (در واقع شناسه‌های ورودند که توصیه می‌کنند، نه مشتریان!) فرض کنیم (برای سادگی) شمارِ مشتریان یکنواخت بین 1 و 3 و شمارِ کتاب‌ها یکنواخت بین 2 و 4 باشد:

```
Customer ~ UniformInt(1, 3)#
Book ~ UniformInt(2, 4)        (15.2)#
```

انتظار داریم مشتریانِ صادق فقط یک شناسه داشته باشند، در حالی که مشتریانِ ناصادق ممکن است هر جا بین 2 تا 5 شناسه داشته باشند:

```
LoginID(Owner=c) ~ if Honest(c) then Exactly(1) else UniformInt(2, 5)        (15.3)#
```

این گزارهٔ شمار، توزیعِ شمارِ شناسه‌های ورودی را که مشتریِ c مالکِ آن‌هاست مشخص می‌کند. تابعِ Owner «تابعِ خاستگاه» (Origin Function) نامیده می‌شود، زیرا می‌گوید هر شیءِ تولیدشده توسط این گزارهٔ شمار از کجا آمده.

مثالِ پاراگرافِ پیشین از توزیعِ یکنواخت روی اعدادِ صحیحِ بین 2 و 5 برای مشخص‌کردنِ شمارِ ورودهای مشتریِ ناصادق استفاده می‌کند. این توزیعِ خاص کران‌دار است، اما به‌طورِ کلی ممکن است کرانِ پیشینی برای شمارِ اشیا نباشد. پرکاربردترین توزیع روی اعدادِ صحیحِ نامنفی «توزیعِ پواسون» (Poisson Distribution) است. پواسون یک پارامتر دارد، λ، که شمارِ موردانتظارِ اشیاست. توزیعِ پواسون به‌طورِ طبیعی برای شمارش‌ها به وجود می‌آید: اگر رویدادها در زمان به‌صورتِ مستقل با نرخِ ثابتِ λ رخ دهند، شمارِ رویدادها در بازه‌ای ثابت توزیعِ پواسونی دارد. توزیعِ پواسون به‌صورتِ مکرر در مدل‌های OUPM ظاهر می‌شود، و می‌تواند گامِ زایشیِ توضیح‌داده‌شده در بالا را پیاده‌سازی کند: شمارِ اشیایی با «خاستگاهِ» معین. برای نمونه، در مدلِ توصیهٔ کتاب ممکن است بنویسیم:

```
LoginID(Owner=c) ~ Poisson(3)#
```

گزاره‌های شمار می‌توانند هر عبارتی باشند—می‌توانند به گزاره‌های شمارِ دیگر یا به ویژگی‌های اشیای موجود اشاره کنند. در یک OUPM، سپس «متغیرهای شمارِ» (Number Variables) تکی برای هر گزارهٔ شمار داریم. متغیرِ شمارِ #4 = (ω)(⟨Customer,2⟩)LoginID(Owner= بدان معناست در جهانِ ω، مشتریِ 2 مالکِ 4 شناسهٔ ورود است. همانندِ مدل‌های احتمالِ رابطه‌ای، متغیرهای تصادفیِ پایه مقادیرِ گزاره‌ها و توابع را برای همهٔ چندتایی‌های اشیا تعیین می‌کنند؛ بدین‌ترتیب، true = (ω)(⟨Customer,2⟩)Honest بدان معناست در جهانِ ω، مشتریِ 2 صادق است. جهانِ ممکن با مقادیرِ همهٔ متغیرهای شمار و متغیرهای تصادفیِ پایه تعریف می‌شود. جهانی را می‌توان از مدل با نمونه‌برداری در ترتیبِ توپولوژیک تولید کرد؛ شکل ۱۵.۴ نمونه‌ای را نشان می‌دهد. احتمالِ جهانی که بدین‌گونه ساخته شده، حاصلِ‌ضربِ احتمال‌های همهٔ مقادیرِ نمونه‌برداری‌شده است؛ در این مورد، 1.2672×10^{-11}. حالا روشن می‌شود چرا هر شیء خاستگاهِ خود را در بر دارد: این ویژگی تضمین می‌کند هر جهان را می‌توان با دقیقاً یک توالیِ زایش ساخت. اگر چنین نبود، احتمالِ جهان، جمعِ ترکیباتیِ دست‌وپاگیری روی همهٔ توالی‌های زایشِ ممکنِ سازندهٔ آن می‌بود.

مدل‌های جهانِ باز ممکن است بی‌نهایت متغیرِ تصادفی داشته باشند، پس نظریهٔ کامل ملاحظاتِ غیربدیهیِ نظریهٔ اندازه را در بر دارد. برای نمونه، گزاره‌های شمار با توزیع‌های پواسون یا مرتبه‌بزرگی شمارِ نامحدودِ اشیا را اجازه می‌دهند، که به شمارِ نامحدودِ متغیرهای تصادفی برای ویژگی‌ها و روابطِ آن اشیا می‌انجامد. علاوه بر این، OUPMها می‌توانند

وابستگی‌های بازگشتی و نوع‌های نامتناهی (اعدادِ صحیح، رشته‌ها و...) داشته باشند. سرانجام، خوش‌ساخت‌بودن وابستگی‌های چرخه‌ای و زنجیره‌های نیاییِ بی‌نهایتِ پس‌رونده را منع می‌کند؛ این شرایط در حالتِ عمومی تصمیم‌ناپذیرند، اما شرایطِ نحویِ کافیِ خاصی را می‌توان به‌آسانی بررسی کرد.

Variable	Value	Probability
$\#Customer$	2	0.3333
$\#Book$	3	0.3333
$Honest_{\langle Customer,,1\rangle}$	*true*	0.99
$Honest_{\langle Customer,,2\rangle}$	*false*	0.01
$Kindness_{\langle Customer,,1\rangle}$	4	0.3
$Kindness_{\langle Customer,,2\rangle}$	1	0.1
$Quality_{\langle Book,,1\rangle}$	1	0.05
$Quality_{\langle Book,,2\rangle}$	3	0.4
$Quality_{\langle Book,,3\rangle}$	5	0.15
$\#LoginID_{\langle Owner,\langle Customer,,1\rangle\rangle}$	1	1.0
$\#LoginID_{\langle Owner,\langle Customer,,2\rangle\rangle}$	2	0.25
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle,\langle Book,,1\rangle}$	2	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle,\langle Book,,2\rangle}$	4	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,1\rangle\rangle,1\rangle,\langle Book,,3\rangle}$	5	0.5
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle,\langle Book,,1\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle,\langle Book,,2\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,1\rangle,\langle Book,,3\rangle}$	1	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle,\langle Book,,1\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle,\langle Book,,2\rangle}$	5	0.4
$Recommendation_{\langle LoginID,\langle Owner,\langle Customer,,2\rangle\rangle,2\rangle,\langle Book,,3\rangle}$	1	0.4

شکل ۱۵.۴ — جهانی خاص برای OUPM توصیهٔ کتاب. متغیرهای شمار و متغیرهای تصادفیِ پایه در ترتیبِ توپولوژیک، همراه با مقادیرِ انتخاب‌شده و احتمال‌های آن مقادیر نشان داده شده‌اند. (نمونه: #Customer با مقدارِ 2 و احتمالِ 0.3333؛ Honest(⟨Customer,1⟩) = true با 0.99؛ #LoginID(Owner=⟨Customer,1⟩) = 1 با 1.0؛ و همین‌طور برای بقیه تا Recommendationهای گوناگون؛ احتمالِ کلِ جهان 1.2672×10^{-11}.)

۱۵.۲.۲ استنتاج در مدل‌های احتمالِ جهانِ باز

به دلیلِ اندازهٔ بالقوهٔ عظیم و گاهی نامحدودِ شبکهٔ بیزِ ضمنیِ متناظر با OUPM نمونه‌ای، بازکردنِ کاملِ آن و انجامِ استنتاجِ دقیق بسیار غیرعملی است. در عوض، باید الگوریتم‌های استنتاجِ تقریبی مانندِ MCMC را در نظر بگیریم (بخش ۱۳.۴.۲ را ببینید). به‌طورِ تقریبی، الگوریتمِ MCMC برای OUPM فضایِ جهان‌های ممکنِ تعریف‌شده توسط مجموعه‌های اشیا و روابطِ میان‌شان را کاوش می‌کند، چنان‌که در شکل ۱۵.۱ (بالا) نشان داده شده.

حرکتِ میان حالت‌های مجاور در این فضا نه فقط می‌تواند روابط و توابع را تغییر دهد، بلکه می‌تواند اشیا را بیفزاید یا کم کند و تفسیرهای نمادهای ثابت را تغییر دهد. حتی اگرچه هر جهانِ ممکن ممکن است عظیم باشد، محاسبه‌های احتمالِ لازم برای هر گام—خواه در نمونه‌برداریِ گیبس خواه متروپلیس–هستینگز—کاملاً محلی‌اند و در بیشترِ موارد

زمانِ ثابت می‌گیرند. این به این دلیل است که نسبتِ احتمال میان جهان‌های همسایه به زیرگرافی با اندازهٔ ثابت پیرامونِ متغیرهایی بستگی دارد که مقادیرشان تغییر کرده. علاوه بر این، پرس‌وجوی منطقی را می‌توان در هر جهانِ بازدیدشده به‌صورتِ افزایشی ارزیابی کرد، معمولاً در زمانِ ثابتِ هر جهان، به‌جای محاسبهٔ دوباره از صفر.

باید توجهِ ویژه‌ای به این واقعیت شود که OUPM نمونه‌ای ممکن است جهان‌های ممکنِ با اندازهٔ نامتناهی داشته باشد. به‌عنوانِ نمونه، مدلِ ردیابیِ چندهدف در شکل ۱۵.۹ را در نظر بگیرید: تابعِ X(a,t)، که حالتِ هواپیمای a در زمانِ t را نشان می‌دهد، با توالیِ نامتناهی از متغیرها برای شمارِ نامحدودِ هواپیما در هر گام متناظر است. به همین دلیل، MCMC برای OUPMها نه از جهان‌های ممکنِ کاملاً مشخص، بلکه از «جهان‌های جزئی» (Partial Worlds) نمونه‌برداری می‌کند، که هر یک با مجموعه‌ای مجزا از جهان‌های کامل متناظر است. جهانِ جزئی، مصداق‌سازیِ کمینهٔ خودپشتیبان[6] از زیرمجموعه‌ای از متغیرهای مربوط—یعنی نیاکانِ متغیرهای شاهد و پرس‌وجو—است. برای نمونه، متغیرهای X(a,t) برای مقادیرِ t بزرگ‌تر از آخرین زمانِ مشاهده (یا زمانِ پرس‌وجو، هرکدام بزرگ‌تر است) نامربوط‌اند، پس الگوریتم می‌تواند فقط پیشوندِ متناهیِ توالیِ نامتناهی را در نظر بگیرد.

۱۵.۲.۳ نمونه‌ها

«موردِ کاربردیِ» استاندارد برای OUPM سه عنصر دارد: مدل، شواهد (واقعیت‌های شناخته‌شده در سناریوی معین) و پرس‌وجو، که ممکن است هر عبارتی باشد، احتمالاً با متغیرهای منطقیِ آزاد. پاسخ، احتمالِ مشترکِ پسینی برای هر مجموعهٔ ممکنِ جانشینی برای متغیرهای آزاد، با توجه به شواهد، طبقِ مدل است.[7] هر مدل شاملِ اعلامیه‌های نوع، امضاهای نوع برای گزاره‌ها و توابع، یک یا چند گزارهٔ شمار برای هر نوع، و یک گزارهٔ وابستگی برای هر گزاره و تابع است. (در نمونه‌های زیر، اعلامیه‌ها و امضاها جایی که معنا روشن است حذف شده‌اند.) همانندِ RPMها، گزاره‌های وابستگی از نحوِ if-then-else برای مدیریتِ وابستگی‌های زمینه‌وابسته استفاده می‌کنند.

تطبیقِ استناد. میلیون‌ها مقالهٔ پژوهشیِ دانشگاهی و گزارشِ فنی به‌صورتِ برخط در قالبِ فایل‌های pdf یافت می‌شوند. چنین مقاله‌هایی معمولاً بخشی در نزدیکیِ پایان به نامِ «مراجع» یا «کتاب‌نامه» دارند، که در آن استنادها—رشته‌هایی از نویسه‌ها—برای آگاه‌کردنِ خواننده از کارهای مرتبط فراهم می‌شوند. این رشته‌ها را می‌توان با هدفِ ایجادِ بازنماییِ پایگاه‌داده‌مانندی که مقاله‌ها و پژوهشگران را با پیوندهای نویسندگی و استناد مرتبط می‌کند، مکان‌یابی و از فایل‌های pdf «خراشید» (Scrape). سامانه‌هایی مانندِ CiteSeer و Google Scholar چنین بازنمایی‌ای را به کاربرانشان ارائه می‌دهند؛ پشتِ صحنه، الگوریتم‌ها عمل می‌کنند تا مقاله‌ها را بیابند، رشته‌های استناد را استخراج کنند و مقاله‌های واقعی را که رشته‌های استناد به آن‌ها اشاره می‌کنند شناسایی کنند. این تکلیفِ دشواری است، زیرا این رشته‌ها هیچ شناسهٔ شیءای ندارند و شاملِ خطاهای نحو، املا، نقطه‌گذاری و محتوا هستند. برای نشان‌دادنِ این، دو نمونهٔ نسبتاً خوش‌خیم:

1. [Lashkari et al 94] Collaborative Interface Agents, Yezdi Lashkari, Max Metral, and Pattie Maes, Proceedings of the Twelfth National Conference on Artificial Intelligence, MIT Press, Cambridge, MA, 1994.

Metral M. Lashkari, Y. and P. Maes. Collaborative interface agents. In Conference of the .2
.American Association for Artificial Intelligence, Seattle, WA, August 1994

پرسشِ کلیدی پرسشِ هویتی است: آیا این استنادها به همان مقاله اشاره می‌کنند یا به مقاله‌های متفاوت؟ وقتی این پرسش پرسیده می‌شود، حتی متخصصان هم اختلاف دارند یا مایل به تصمیم‌گیری نیستند، که نشان می‌دهد استدلالِ تحتِ عدم قطعیت بخشِ مهمی از حلِ این مسئله خواهد بود.[8] رویکردهای موردی—مانندِ روش‌های مبتنی بر معیارِ شباهتِ متنی—اغلب به‌شکلِ فلاکت‌باری شکست می‌خورند. برای نمونه، در ۲۰۰۲، CiteSeer بیش از ۱۲۰ کتابِ متمایز نوشته‌شده توسط راسل و نورویگ گزارش کرد.

برای حلِ مسئله با رویکردِ احتمالاتی، به مدلِ زایشی برای حوزه نیاز داریم. یعنی می‌پرسیم این رشته‌های استناد چگونه در جهان پدید می‌آیند. فرایند با پژوهشگرانی شروع می‌شود که نام دارند. (لازم نیست نگرانِ چگونگیِ پیدایشِ پژوهشگران باشیم؛ فقط باید عدم قطعیتِ‌مان را دربارهٔ اینکه چندتایند بیان کنیم.) این پژوهشگران مقاله‌هایی می‌نویسند که عنوان دارند؛ مردم مقاله‌ها را استناد می‌کنند، و نامِ نویسندگان و عنوانِ مقاله را (با خطا) طبقِ گرامری در متنِ استناد ترکیب می‌کنند. عناصرِ پایهٔ این مدل در شکل ۱۵.۵ نشان داده شده، که حالتی را پوشش می‌دهد که مقاله‌ها فقط یک نویسنده دارند.[9] با توجه به فقط رشته‌های استناد به‌عنوانِ شاهد، استنتاجِ احتمالاتی روی این مدل برای برگزیدنِ محتمل‌ترین تبیینِ داده‌ها نرخِ خطایی 2 تا 3 برابرِ کمتر از CiteSeer تولید می‌کند (پاسولا و همکاران، 2003). فرایندِ استنتاج همچنین شکلی از ابهام‌زداییِ جمعی و دانش‌محور را نشان می‌دهد: هرچه استنادهای یک مقالهٔ معین بیشتر باشند، هر یک با دقتِ بیشتری تجزیه می‌شوند، زیرا تجزیه‌ها باید بر واقعیت‌های مربوط به مقاله توافق کنند.

type Researcher, Paper, Citation
random String Name(Researcher)
random String Title(Paper)
random Paper PubCited(Citation)
random String Text(Citation)
random Boolean Professor(Researcher)
origin Researcher Author(Paper)

$$\#Researcher \sim OM(3,1)$$
$$Name(r) \sim NamePrior()$$
$$Professor(r) \sim Boolean(0.2)$$
$$\#Paper(Author = r) \sim \textbf{if } Professor(r) \textbf{ then } OM(1.5,0.5) \textbf{ else } OM(1,0.5)$$
$$Title(p) \sim PaperTitlePrior()$$
$$CitedPaper(c) \sim UniformChoice(\{Paper\ p\})$$
$$Text(c) \sim HMMGrammar(Name(Author(CitedPaper(c))), Title(CitedPaper(c)))$$

شکل ۱۵.۵ — OUPM برای استخراجِ اطلاعاتِ استناد. برای سادگی، مدل فرض می‌کند هر مقاله یک نویسنده دارد و جزئیاتِ گرامر و مدل‌های خطا را حذف می‌کند.

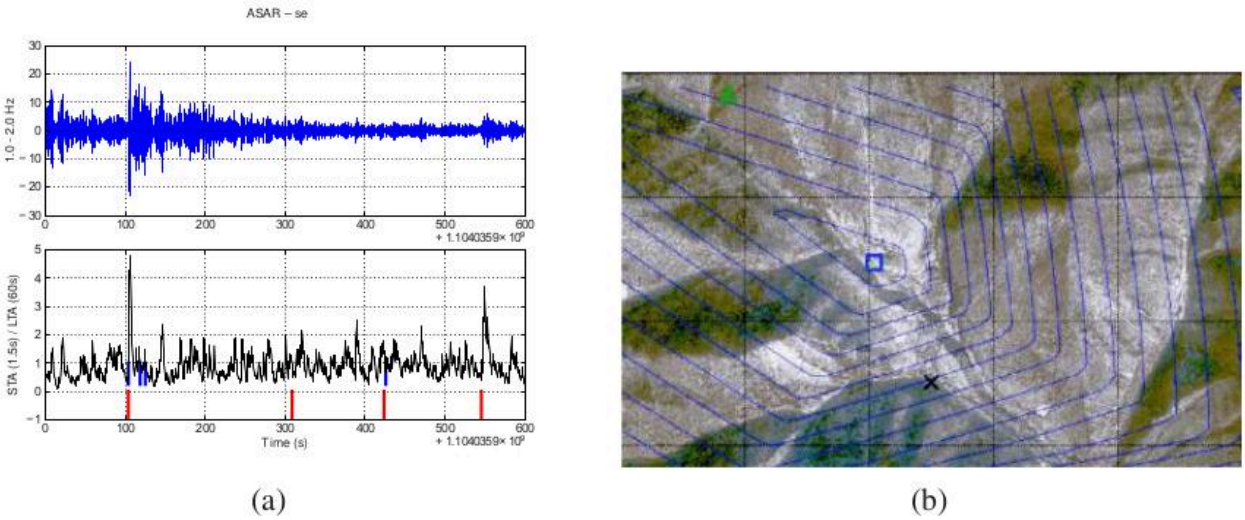
پایشِ پیمانِ هسته‌ای. راستی‌آزماییِ «پیمانِ جامعِ منعِ آزمایش‌های هسته‌ای» مستلزم یافتنِ همهٔ رخدادهای لرزه‌ایِ زمین بالای بزرگیِ کمینه است. سازمانِ CTBTO سازمانِ ملل شبکه‌ای از حسگرها را نگه می‌دارد، «سامانهٔ پایشِ بین‌المللی» (International Monitoring System یا IMS)؛ نرم‌افزارِ پردازشِ خودکارِ آن، که بر ۱۰۰ سال پژوهشِ

لرزه‌شناسی مبتنی است، نرخِ شکستِ تشخیصی حدود ۳۰٪ دارد. سامانهٔ NET-VISA (آرورا و همکاران، 2013)، مبتنی بر OUPM، شکست‌های تشخیصی را به‌طورِ قابلِ‌توجهی کاهش می‌دهد.

مدلِ NET-VISA (شکل ۱۵.۶) ژئوفیزیکِ مربوط را مستقیماً بیان می‌کند. توزیع‌هایی روی شمارِ رخدادها در بازهٔ زمانیِ معین (بیشترشان طبیعی‌اند) و نیز روی زمان، بزرگی، عمق و مکان‌شان توصیف می‌کند. مکان‌های رخدادهای طبیعی طبقِ «پیشینِ مکانی» توزیع می‌شوند که (مانندِ دیگر بخش‌های مدل) از داده‌های تاریخی آموزش دیده؛ رخدادهای انسان‌ساخت، طبقِ قواعدِ پیمان، فرض می‌شوند به‌صورتِ یکنواخت روی سطحِ زمین رخ دهند. در هر ایستگاهِ s، هر «فازِ» (نوعِ موجِ لرزه‌ای) p از رخدادِ e یا 0 تشخیص تولید می‌کند یا 1 (سیگنالِ بالای آستانه)؛ احتمالِ تشخیص به بزرگی و عمقِ رخداد و فاصله‌اش از ایستگاه بستگی دارد. تشخیص‌های «هشدارِ کاذب» نیز طبقِ پارامترِ نرخِ خاصِ ایستگاه رخ می‌دهند. زمانِ رسیدن، دامنه و دیگر ویژگی‌های اندازه‌گیری‌شدهٔ تشخیصِ d از رخدادِ واقعی به ویژگی‌های رخدادِ مبدأ و فاصله‌اش از ایستگاه بستگی دارد.

$$
\begin{aligned}
&\#SeismicEvents \sim Poisson(T * \lambda_e)\\
&Time(e) \sim UniformReal(0,T)\\
&EarthQuake(e) \sim Boolean(0.999)\\
&Location(e) \sim \textbf{if } Earthquake(e) \textbf{ then } SpatialPrior() \textbf{ else } UniformEarth()\\
&Depth(e) \sim \quad \textbf{if } Earthquake(e) \textbf{ then } UniformReal(0,700) \textbf{ else } Exactly(0)\\
&Magnitude(e) \sim Exponential(log(10))\\
&Detected(e,p,s) \sim Logistic(weights(s,p), Magnitude(e),\ Depth(e),\ Dist(e,s))\\
&\#Detections(site = s) \sim Poisson(T * \lambda_f(s))\\
&\#Detections(event{=}e,\ phase{=}p,\ station{=}s) = \textbf{if } Detected(e,p,s) \textbf{ then } 1 \textbf{ else } 0\\
&OnsetTime(a,s) \textbf{ if } (event(a) = null) \textbf{ then } \sim UniformReal(0,T)\\
&\quad \textbf{else} = Time(event(a)) + GeoTT(Dist(event(a),s), Depth(event(a)), phase(a))\\
&\qquad\qquad + Laplace(\mu_t(s), \sigma_t(s))\\
&Amplitude(a,s) \textbf{ if } (event(a) = null) \textbf{ then } \sim NoiseAmpModel(s)\\
&\quad \textbf{else} = AmpModel(Magnitude(event(a)), Dist(event(a),s), Depth(event(a)), phase(a))\\
&Azimuth(a,s) \textbf{ if } (event(a) = null) \textbf{ then } \sim UniformReal(0,\ 360)\\
&\quad \textbf{else} = GeoAzimuth(Location(event(a)), Depth(event(a)), phase(a), Site(s))\\
&\qquad\qquad + Laplace(0, \sigma_a(s))\\
&Slowness(a,s) \textbf{ if } (event(a) = null) \textbf{ then } \sim UniformReal(0,20)\\
&\quad \textbf{else} = GeoSlowness(Location(event(a)), Depth(event(a)), phase(a), Site(s))\\
&\qquad\qquad + Laplace(0, \sigma_s(s))\\
&ObservedPhase(a,s) \sim CategoricalPhaseModel(phase(a))
\end{aligned}
$$

شکل ۱۵.۶ — نسخهٔ ساده‌شدهٔ مدلِ NET-VISA.



شکل ۱۵.۷ – (الف) بالا: نمونهٔ شکلِ موجِ لرزه‌ایِ ثبت‌شده در آلیس‌اسپرینگز، استرالیا. پایین: شکلِ موج پس از پردازش برای تشخیصِ زمان‌های رسیدنِ امواجِ لرزه‌ای. خطوطِ آبی رسیدن‌های به‌طورِ خودکار تشخیص‌داده‌شده‌اند؛ خطوطِ قرمز رسیدن‌های واقعی‌اند. (ب) برآوردهای مکان برای آزمایشِ هسته‌ایِ کرهٔ شمالی در ۱۲ فوریهٔ ۲۰۱۳: بولتنِ رویدادِ دیرهنگامِ CTBTO سازمانِ ملل (مثلثِ سبز در بالا-چپ)؛ NET-VISA (مربعِ آبی در مرکز). ورودیِ تأسیساتِ آزمایشِ زیرزمینی («x» کوچک) 0.75 کیلومتر از برآوردِ NET-VISA فاصله دارد. کانتورها توزیعِ پسینیِ مکانِ NET-VISA را نشان می‌دهند. با اجازهٔ کمیسیونِ مقدماتیِ CTBTO.

پس از آموزش، مدل به‌صورتِ پیوسته اجرا می‌شود. شواهد شاملِ تشخیص‌هایی (۹۰٪ آن‌ها هشدارِ کاذب‌اند) است که از داده‌های شکلِ موجِ خامِ IMS استخراج می‌شوند، و پرس‌وجو معمولاً محتمل‌ترین تاریخچهٔ رخداد، یا «بولتن»، را با توجه به داده‌ها می‌پرسد. نتایج تاکنون دلگرم‌کننده‌اند؛ برای نمونه، در ۲۰۰۹ بولتنِ خودکارِ SEL3 سازمانِ ملل ۲۷.۴٪ از ۲۷,۲۹۴ رخداد در محدودهٔ بزرگیِ 3–4 را از دست داد، در حالی که NET-VISA ۱۱.۱٪ را از دست داد. علاوه بر این، مقایسه‌ها با شبکه‌های منطقه‌ایِ متراکم نشان می‌دهد NET-VISA تا ۵۰٪ رخدادِ واقعیِ بیشتر از بولتن‌های نهاییِ تولیدشده توسط تحلیلگرانِ لرزه‌شناسِ خبرهٔ سازمانِ ملل می‌یابد. NET-VISA همچنین تمایل دارد تشخیص‌های بیشتری را با رخدادِ معینی مرتبط کند، که به برآوردهای مکانِ دقیق‌تر می‌انجامد (شکل ۱۵.۷ را ببینید). از ۱ ژانویهٔ ۲۰۱۸، NET-VISA به‌عنوانِ بخشی از خطِ لولهٔ پایشِ CTBTO مستقر شده. با وجودِ تفاوت‌های سطحی، این دو نمونه از نظرِ ساختاری مشابه‌اند: اشیای ناشناخته‌ای (مقاله‌ها، زلزله‌ها) وجود دارند که طبقِ قوانینِ فیزیکیِ معین، ادراک‌هایی تولید می‌کنند؛ ادراک‌های مشاهده‌شده اطلاعاتِ ناقصی دربارهٔ اشیای مولدشان فراهم می‌کنند؛ و مدل باید وجودِ اشیای تشخیص‌داده‌نشده و تشخیص‌های بدونِ شیءِ مولد (هشدارهای کاذب) را در نظر بگیرد.

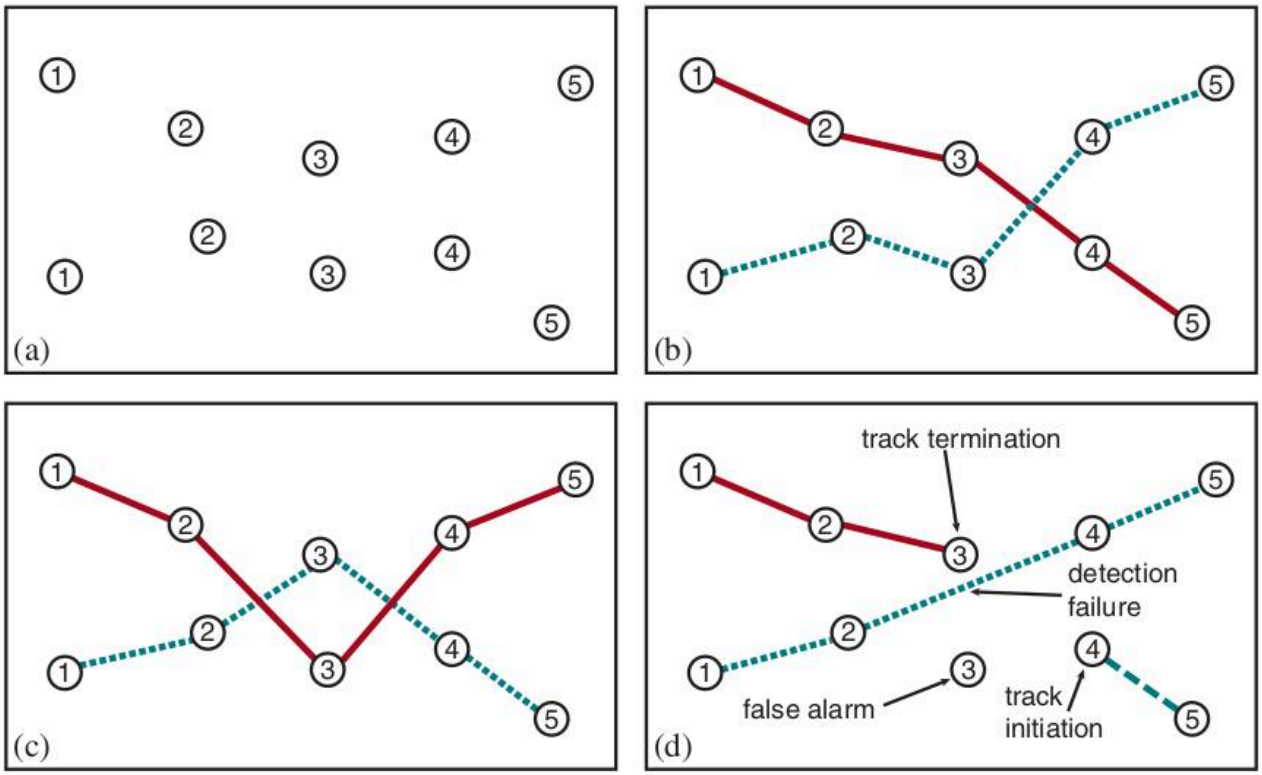
۱۵.۳ پیگیریِ جهانِ پیچیده

فصل ۱۴ مسئلهٔ پیگیریِ حالتِ جهان را در نظر گرفت، اما فقط حالتِ بازنمایی‌های اتمی (HMMها) و بازنمایی‌های عاملی (DBNها و فیلترهای کالمن) را پوشش داد. این برای جهان‌های با شیءِ تکی منطقی است—شاید بیمارِ تکی در بخشِ مراقبت‌های ویژه یا پرندهٔ تکی که در جنگل پرواز می‌کند. در این بخش، می‌بینیم وقتی دو یا چند شیء مشاهده‌ها را تولید می‌کنند چه اتفاقی می‌افتد. آنچه این حالت را از برآوردِ حالتِ سادهٔ قدیمی متفاوت می‌کند این است که اکنون امکانِ عدم قطعیت دربارهٔ اینکه کدام شیء کدام مشاهده را تولید کرده هست. این مسئلهٔ عدم قطعیتِ هویتیِ بخش ۱۵.۲ (صفحهٔ ۵۰۷) است، که حالا در بافتِ زمانی دیده می‌شود. در ادبیاتِ نظریهٔ کنترل، این

«مسئلهٔ ارتباطِ داده‌ها» (Data Association) است—یعنی مسئلهٔ مرتبط‌کردنِ داده‌های مشاهده با اشیایی که آن‌ها را تولید کرده‌اند. اگرچه می‌توانیم این را نمونهٔ دیگری از مدل‌سازیِ احتمالاتیِ جهانِ باز ببینیم، در عمل به‌اندازهٔ کافی مهم است که بخشِ خودش را سزاوار باشد.

۱۵.۳.۱ نمونه: ردیابیِ چندهدف

مسئلهٔ ارتباطِ داده‌ها در بخشِ ۴.۴.۱ معرفی شد. نسخه‌ای از مسئله که در آن‌جا بررسی شد، جهانِ «گودال-وومپوس» نبود، بلکه موردِ ساده‌تری بود: رباتی که مکانش را دنبال می‌کند. اگر دو یا چند شیءِ در حالِ حرکت را دنبال می‌کنید، چه اتفاقی می‌افتد؟ فرض کنیم n شیء داریم و در هر گامِ زمانی، حسگرِ ما n مشاهده تولید می‌کند، اما اطلاعاتِ کافی برای اینکه بگوید کدام مشاهده مربوط به کدام شیء است فراهم نمی‌کند. در حالتِ دو شیء، دو تفسیرِ ممکنِ سازگار از داده‌ها هست. اگر اشیا به هم نزدیک باشند و/یا اندازه‌گیری‌ها نویزی باشند، ممکن است برای مدتی مشخص نباشد کدام تفسیر درست است. با گذشتِ زمان، ممکن است ابهام حل شود.



شکل ۱۵.۸ – (الف) مشاهده‌های مکانِ اشیا در فضایِ دوبعدی روی پنج گامِ زمانی. هر «لکهٔ» مشاهده با گامِ زمانی برچسب‌گذاری شده اما شیءِ تولیدکننده‌اش را شناسایی نمی‌کند. (ب–ج) فرضیه‌های ممکن دربارهٔ مسیرهای زیربناییِ اشیا. (د) فرضیه‌ای برای حالتی که هشدارهای کاذب، شکستِ تشخیص، آغازِ مسیر و خاتمهٔ مسیر رخ می‌دهند.

این حالتِ ساده را با مدلِ احتمالِ رابطه‌ایِ زیر به‌صورتِ صوری در می‌آوریم. متغیرِ X(a,t) مکانِ شیءِ a را در زمانِ t نشان می‌دهد؛ متغیرِ Z(b) مکانِ مشاهده‌شدهٔ لکهٔ b را نشان می‌دهد؛ متغیرِ Source(b) شیءای را نشان می‌دهد که لکهٔ b را تولید کرده. اینکه لکه‌ای مربوط به کدام شیء است، تخصیص (Association) نامیده می‌شود. مدلِ انتقالِ حالت یکسان با مدلِ فیلترِ کالمن است (بخش ۱۴.۴ را ببینید): هواپیماها به‌صورتِ تقریبی با سرعتِ ثابت پرواز می‌کنند و نویزِ گاوسیِ کوچکی روی سرعت و مکان می‌آید. قطعهٔ پایانی مدلِ حسگر است: باز، مدلِ خطی–گاوسی فرض می‌کنیم

که در آن هواپیمایی در مکانِ x لکهٔ b تولید می‌کند که مکانِ مشاهده‌شدهٔ لکه‌اش Z(b) تابعی خطی از x با نویزِ گاوسیِ افزوده است. هر هواپیما در هر گامِ زمانی دقیقاً یک لکه تولید می‌کند، پس لکه خاستگاه‌هایش هواپیما و گامِ زمانی‌اند. پس، با حذفِ موقتِ پیشین، مدل به‌صورتِ زیر است:

```
                                  هواپیماهای تضمین‌شده: A1، A2
X(a,t) ~ if t = 0 then InitX() else N(F·X(a,t−1), Σx)
                                        Blip(Source=a, Time=t) = 1#
                                    Z(b) ~ N(H·X(Source(b), Time(b)), Σz)
```

که در آن F و Σx ماتریس‌هایی‌اند که مدلِ انتقالِ خطی و کوواریانسِ نویزِ انتقال را توصیف می‌کنند، و H و Σz ماتریس‌های متناظر برای مدلِ حسگرند. (صفحهٔ ۴۸۳ را ببینید.) تفاوتِ کلیدی میانِ این مدل و فیلترِ کالمنِ استاندارد این است که دو شیءِ تولیدکنندهٔ خوانشِ حسگر (لکه‌ها) هست. این بدان معناست در هر گامِ زمانیِ معین، عدم قطعیتی دربارهٔ اینکه کدام شیء کدام خوانشِ حسگر را تولید کرده هست. هر جهانِ ممکن در این مدل شاملِ تخصیصی—تعریف‌شده با مقادیرِ همهٔ متغیرهای Source(b) برای همهٔ گام‌های زمانی—میانِ هواپیماها و لکه‌هاست. دو فرضیهٔ تخصیصِ ممکن در شکل ۱۵.۸(ب–ج) نشان داده شده. به‌طورِ کلی، برای n شیء و T گامِ زمانی، $(n!)^T$ راهِ تخصیصِ لکه‌ها به هواپیماها هست—عددی به‌طرزِ وحشتناکی بزرگ.

سناریویِ توصیف‌شده تاکنون شاملِ n شیءِ شناخته‌شده بود که در هر گامِ زمانی n مشاهده تولید می‌کنند. کاربردهای واقعیِ ارتباطِ داده‌ها معمولاً بسیار پیچیده‌ترند. اغلب، مشاهده‌های گزارش‌شده شاملِ «هشدارهای کاذب» (False Alarms) هستند، که «آشفتگی» (Clutter) نیز نامیده می‌شوند و توسط اشیای واقعی ایجاد نشده‌اند. «شکست‌های تشخیص» (Detection Failures) می‌توانند رخ دهند، بدان معنا که برای شیءِ واقعی هیچ مشاهده‌ای گزارش نمی‌شود. سرانجام، اشیای تازه می‌رسند و اشیای قدیمی ناپدید می‌شوند. این پدیده‌ها، که جهان‌های ممکنِ بیشتری برای نگرانی ایجاد می‌کنند، در شکل ۱۵.۸(د) نشان داده شده‌اند. OUPM متناظر در شکل ۱۵.۹ داده شده.

$$
\begin{aligned}
&\#Aircraft(EntryTime = t) \sim Poisson(\lambda_a)\\
&Exists(a,t) \sim \textbf{if } InFlight(a,t) \textbf{ then } Boolean(\alpha_e)\\
&InFlight(a,t) = (t{=}EntryTime(a)) \vee (InFlight(a,t-1) \wedge \neg Exists(a,t-1))\\
&X(a,t) \sim \textbf{if } t = EntryTime(a) \textbf{ then } InitX()\\
&\quad \textbf{else if } InFlight(a,t) \textbf{ then } \mathcal{N}(\mathbf{F}X(a,t-1), \Sigma_x)\\
&\#Blip(Source{=}a, Time{=}t) \sim \textbf{if } InFlight(a,t) \textbf{ then } Bernoulli(DetectionProb(X(a,t)))\\
&\#Blip(Time{=}t) \sim Poisson(\lambda_f)\\
&Z(b) \sim \textbf{if } Source(b){=}null \textbf{ then } UniformZ(R) \textbf{ else } \mathcal{N}(\mathbf{H}X(Source(b), Time(b)), \Sigma_z)
\end{aligned}
$$

شکل ۱۵.۹ — OUPM برای ردیابیِ راداریِ چند هدف با هشدارهای کاذب، شکستِ تشخیص و ورود و خروجِ هواپیماها. نرخی که هواپیماهای تازه واردِ صحنه می‌شوند λa است، در حالی که احتمالِ هر گامِ زمانیِ خروجِ هواپیما از صحنه αe است. لکه‌های هشدارِ کاذب (یعنی آن‌هایی که توسط هواپیما تولید نشده‌اند) به‌صورتِ یکنواخت در فضا با نرخِ λf در هر گامِ زمانی ظاهر می‌شوند. احتمالِ تشخیصِ هواپیما (یعنی تولیدِ لکه) به مکانِ کنونی‌اش بستگی دارد.

به دلیلِ اهمیتِ عملی‌اش برای کاربردهای غیرنظامی و نظامی، ده‌ها هزار مقاله دربارهٔ مسئلهٔ ردیابیِ چندهدف و ارتباطِ داده‌ها نوشته شده. بسیاری از آن‌ها صرفاً سعی می‌کنند جزئیاتِ ریاضیِ پیچیدهٔ محاسباتِ احتمال را برای مدلِ شکل ۱۵.۹، یا نسخه‌های ساده‌ترش، از کار درآورند. از جهتی، این پس از بیانِ مدل در زبانِ برنامه‌نویسیِ احتمالاتی غیرضروری است، زیرا موتورِ استنتاجِ همه‌منظوره همهٔ ریاضیات را برای هر مدلی—از جمله این یکی—به‌درستی انجام می‌دهد. علاوه بر این، بسط‌های سناریو (پروازِ آرایشی، اشیایی که به مقصدهای ناشناخته می‌روند، اشیایی که بلند می‌شوند یا فرود می‌آیند و...) را می‌توان با تغییرهای کوچکِ مدل مدیریت کرد، بدونِ توسل به اشتقاق‌های ریاضیِ تازه و برنامه‌نویسیِ پیچیده. از دیدگاهِ عملی، چالش با این نوع مدل پیچیدگیِ استنتاج است. مانندِ همهٔ مدل‌های احتمال، استنتاج یعنی خارج‌کردنِ متغیرهای به‌جزِ پرس‌وجو و شواهد از جمع. برای فیلترکردن در HMMها و DBNها، توانستیم متغیرهای حالت را از 1 تا t-1 با حقهٔ سادهٔ برنامه‌نویسیِ پویا خارج کنیم؛ برای فیلترهای کالمن، از ویژگی‌های خاصِ گاوسی‌ها نیز بهره گرفتیم. برای ارتباطِ داده‌ها، کمتر خوش‌شانسیم. هیچ الگوریتمِ دقیقِ کارآمدِ (شناخته‌شده‌ای) نیست، به همان دلیلی که برای فیلترِ کالمنِ سوئیچ‌شونده (صفحهٔ ۴۸۴) نیست: توزیعِ فیلترکردن، که توزیعِ مشترکِ شمارها و مکان‌های هواپیماها را در هر گامِ زمانی توصیف می‌کند، به مخلوطی از تعدادِ نماییِ توزیع‌ها می‌انجامد، یکی برای هر راهِ برگزیدنِ توالیِ مشاهده برای تخصیص به هر هواپیما. در پاسخ به پیچیدگیِ استنتاجِ دقیق، چند روشِ تقریبی استفاده شده. ساده‌ترین رویکرد انتخابِ تخصیصِ تکیِ «بهترین» در هر گامِ زمانی، با توجه به مکان‌های پیش‌بینی‌شدهٔ اشیا در زمانِ کنونی است. این تخصیص، مشاهده‌ها را با اشیا مرتبط و مسیرِ هر شیء را به‌روزرسانی و پیش‌بینیِ گامِ بعدی را ممکن می‌کند. برای انتخابِ تخصیصِ «بهترین»، استفاده از «فیلترِ نزدیک‌ترین-همسایه» (Nearest-Neighbor Filter) رایج است، که به‌طورِ مکرر نزدیک‌ترین جفت‌شدنِ مکانِ پیش‌بینی‌شده و مشاهده را انتخاب و آن جفت‌شدن را به تخصیص می‌افزاید. فیلترِ نزدیک‌ترین-همسایه وقتی خوب کار می‌کند که اشیا در فضایِ حالت به‌خوبی جدا باشند و عدم قطعیتِ پیش‌بینی و خطایِ مشاهده کوچک باشند—به بیانِ دیگر، وقتی هیچ امکانِ اشتباهی نباشد.

وقتی عدم قطعیتِ بیشتری دربارهٔ تخصیصِ درست هست، رویکردِ بهتر انتخابِ تخصیصی است که احتمالِ مشترکِ مشاهده‌های کنونی را با توجه به مکان‌های پیش‌بینی‌شده بیشینه می‌کند. این را می‌توان با «الگوریتمِ مجارستانی» (Hungarian Algorithm) (کان، 1955) کارآمد انجام داد، حتی اگرچه با رسیدنِ هر گامِ زمانیِ تازه n! تخصیص برای انتخاب هست.

هر روشی که در هر گامِ زمانی به تخصیصِ تکیِ «بهترین» متعهد شود، در شرایطِ دشوارتر به‌شکلِ فلاکت‌باری شکست می‌خورد. به‌طورِ خاص، اگر الگوریتم به تخصیصِ نادرستی متعهد شود، پیش‌بینیِ گامِ زمانیِ بعد ممکن است به‌طورِ قابلِ‌توجهی غلط باشد، که به تخصیص‌های نادرستِ بیشتر می‌انجامد، و همین‌طور ادامه می‌یابد. رویکردهای نمونه‌برداری می‌توانند بسیار مؤثرتر باشند. الگوریتمِ فیلترِ ذره‌ای (صفحهٔ ۴۹۲ را ببینید) برای ارتباطِ داده‌ها با نگه‌داشتنِ مجموعه‌ای بزرگ از تخصیص‌های کنونیِ ممکن کار می‌کند. الگوریتمِ MCMC فضایِ تاریخچه‌های تخصیص را کاوش می‌کند—برای نمونه، شکل ۱۵.۸(ب–ج) ممکن است حالت‌هایی در فضایِ حالتِ MCMC باشند—و می‌تواند نظر خود را دربارهٔ تصمیم‌های تخصیصِ قبلی تغییر دهد.

یکی از راه‌های بدیهی برای سرعت‌بخشیدن به استنتاجِ نمونه‌برداری‌محور برای ردیابیِ چندهدف، استفاده از حقهٔ «راو-بلک‌ولِ‌سازی» از فصل ۱۴ (صفحهٔ ۴۹۶) است: با توجه به فرضیهٔ تخصیصِ خاصی برای همهٔ اشیا، محاسبهٔ فیلترکردن برای هر شیء معمولاً می‌تواند دقیق و کارآمد انجام شود، به‌جای نمونه‌برداری از بسیاری توالی‌های حالتِ ممکن برای اشیا. برای نمونه، با مدلِ شکل ۱۵.۹، محاسبهٔ فیلترکردن صرفاً به معنای اجرایِ فیلترِ کالمن برای توالیِ مشاهده‌های تخصیص‌داده‌شده به شیءِ فرضیِ معین است. علاوه بر این، هنگام تغییر از یک فرضیهٔ تخصیص به فرضیهٔ دیگر، محاسبه‌ها فقط باید برای اشیایی که مشاهده‌های مرتبط‌شان تغییر کرده دوباره انجام شوند. روش‌های کنونیِ ارتباطِ دادهٔ MCMC می‌توانند صدها شیء را در زمانِ واقعی مدیریت کنند و در عینِ حال تقریبِ خوبی از توزیع‌های پسینیِ واقعی بدهند.

۱۵.۳.۲ نمونه: پایشِ ترافیک

شکل ۱۵.۱۰ دو تصویر از دوربین‌های نظارتیِ بسیار جدا از هم در بزرگراهِ ایالتیِ کالیفرنیا را نشان می‌دهد. در این کاربرد، به دو هدف علاقه‌مندیم: برآوردِ زمانی که، تحتِ شرایطِ ترافیکیِ کنونی، برای رفتن از جایی به جاییِ دیگر در سامانهٔ بزرگراه لازم است؛ و اندازه‌گیریِ تقاضا—یعنی چند خودرو میان هر دو نقطهٔ سامانه در زمان‌های خاصِ روز و در روزهای خاصِ هفته تردد می‌کنند. هر دو هدف مستلزم حلِ مسئلهٔ ارتباطِ داده‌ها در منطقه‌ای وسیع با دوربین‌های بسیار و ده‌ها هزار خودرو در ساعت است. با نظارتِ دیداری، هشدارهای کاذب توسط سایه‌های متحرک، خودروهای مفصلی، بازتاب‌ها در گودال‌های آب و مانندِ آن ایجاد می‌شوند؛ شکست‌های تشخیص توسط انسداد، مه، تاریکی و فقدانِ کنتراستِ دیداری ایجاد می‌شوند؛ و خودروها دائماً در نقطه‌هایی که ممکن است پایش نشوند وارد سامانهٔ بزرگراه می‌شوند و از آن خارج می‌شوند. علاوه بر این، ظاهرِ هر خودرویِ معین می‌تواند بین دوربین‌ها به‌طورِ چشمگیری تغییر کند، بسته به شرایطِ نور و وضعیتِ خودرو در تصویر، و مدلِ انتقال با آمدن‌ورفتنِ ترافیکِ سنگین تغییر می‌کند. سرانجام، در ترافیکِ متراکم با دوربین‌های بسیار جدا، خطایِ پیش‌بینی در مدلِ انتقال برای خودرویی که از مکانِ یک دوربین به دوربینِ بعدی می‌رود بسیار بزرگ‌تر از جداییِ معمولِ میانِ خودروهاست. با وجودِ این مشکلات، الگوریتم‌های مدرنِ ارتباطِ داده در برآوردِ پارامترهای ترافیکی در محیط‌های دنیایِ واقعی موفق بوده‌اند.

(a) (b)

شکل ۱۵.۱۰ — تصاویری از (الف) دوربینِ نظارتیِ بالادست و (ب) پایین‌دست، تقریباً ۳ کیلومتر از هم روی بزرگراهِ 99 در ساکرامنتو، کالیفرنیا. خودرویِ قاب‌شده در هر دو دوربین شناسایی شده.

ارتباطِ داده شالودهٔ ضروریِ پیگیریِ جهانِ پیچیده است، زیرا بدونِ آن راهی برای ترکیبِ مشاهده‌های متعدد از هر شیءِ معین نیست. وقتی اشیای جهان در فعالیت‌های پیچیده با یکدیگر تعامل می‌کنند، فهمِ جهان مستلزم ترکیبِ ارتباطِ داده با مدل‌های احتمالِ رابطه‌ای و جهانِ بازِ بخش ۱۵.۲ است. این در حالِ حاضر حوزهٔ فعالِ پژوهشی است.

۱۵.۴ برنامه‌ها به‌عنوانِ مدل‌های احتمال

بسیاری از زبان‌های برنامه‌نویسیِ احتمالاتی بر این بینش ساخته شده‌اند که مدل‌های احتمال را می‌توان با کدِ اجراپذیر در هر زبانِ برنامه‌نویسی که منبعی از تصادف را در بر دارد تعریف کرد. برای چنین مدل‌هایی، جهان‌های ممکن «دنباله‌های اجرا» (Execution Traces) هستند و احتمالِ هر چنین دنباله‌ای، احتمالِ انتخاب‌های تصادفیِ لازم برای وقوعِ آن دنباله است. PPLهای ایجادشده بدین‌گونه تمامِ قدرتِ بیانیِ زبانِ زیربنایی را به ارث می‌برند، شاملِ ساختارهای دادهٔ پیچیده، بازگشت و، در برخی موارد، توابعِ مرتبهٔ بالاتر. بسیاری از PPLها در واقع از نظرِ محاسباتی همگانی‌اند: می‌توانند هر توزیعِ احتمالی را بازنمایی کنند که ماشینِ تورینگِ احتمالاتیِ متوقف‌شونده بتواند از آن نمونه‌برداری کند.

۱۵.۴.۱ نمونه: خواندنِ متن

این رویکردِ مدل‌سازی و استنتاجِ احتمالاتی را با مسئلهٔ نوشتنِ برنامه‌ای که متنِ تخریب‌شده را می‌خواند نشان می‌دهیم. این نوع مدل‌ها را می‌توان برای خواندنِ متنی ساخت که به دلیلِ آسیبِ آب لکه‌دار یا تار شده، یا به دلیلِ کهنگیِ کاغذی که رویش چاپ شده نقطه‌دار شده. آن‌ها را می‌توان برای شکستنِ برخی انواعِ CAPTCHA نیز ساخت.

شکل ۱۵.۱۱ برنامهٔ زایشی (Generative Program) شاملِ دو مؤلفه را نشان می‌دهد: (۱) راهی برای تولیدِ توالیِ حروف؛ و (۲) راهی برای تولیدِ رندرِ نویزی و تارِ این حروف با استفاده از کتابخانهٔ گرافیکیِ آماده. شکل ۱۵.۱۲ (بالا) تصاویرِ نمونه‌ای تولیدشده با فراخوانیِ نه‌بارهٔ GENERATE-IMAGE را نشان می‌دهد.

```
function GENERATE-IMAGE() returns an image with some letters
                                letters ← GENERATE-LETTERS(10)
                        return RENDER-NOISY-IMAGE(letters, 32, 128)

            function GENERATE-LETTERS(λ) returns a vector of letters
                                                    n ~ Poisson(λ)
                                                      [] ← letters
                                                   :for i = 1 to n
                                  letters[i] ~ UniformChoice({a, b, c, ...})
                                                    return letters

function RENDER-NOISY-IMAGE(letters, width, height) returns a noisy image of the letters
         clean image ← RENDER(letters, width, height, text top = 10, text left = 10)
                                                         [] ← noisy image
                                              noise variance ~ UniformReal(0.1, 1)
                                                       :for row = 1 to width
                                                  :for col = 1 to height
            noisy image[row, col] ~ N(clean image[row, col], noise variance)
                                                        return noisy image
```

شکل ۱۵.۱۱ – برنامهٔ زایشی برای مدلِ احتمالِ جهانِ بازِ بازشناسیِ نوریِ نویسه‌ها (OCR). برنامهٔ زایشی تصاویرِ تخریب‌شدهٔ شاملِ توالی‌هایی از حروف را با تولیدِ هر توالی، رندرکردنِ آن به تصویرِ دوبعدی و دربرگرفتنِ نویزِ افزایشی در هر پیکسل تولید می‌کند.

۱۵.۴.۲ نحو و معناشناسی

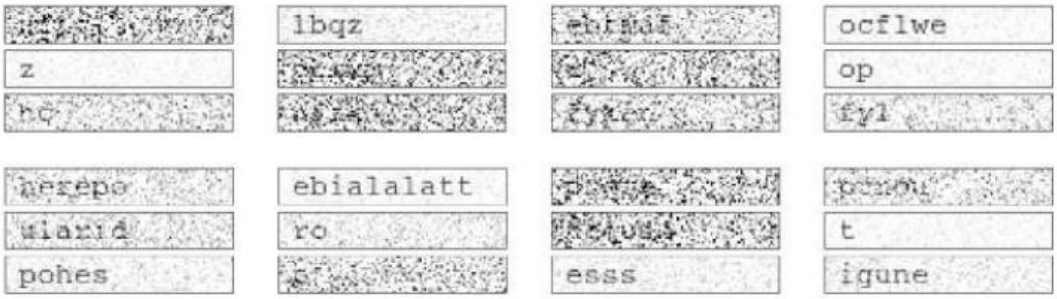
«برنامهٔ زایشی» برنامهٔ اجراپذیری است که در آن هر انتخابِ تصادفی متغیرِ تصادفیِ متناظری در مدلِ احتمالِ مرتبط تعریف می‌کند. تصور کنیم اجرایِ برنامه‌ای را که انتخاب‌های تصادفی می‌کند، گام به گام، باز کنیم. فرض کنیم Xi متغیرِ تصادفیِ متناظر با i-امین انتخابِ تصادفیِ برنامه باشد؛ مثلِ همیشه، xi مقدارِ ممکنِ Xi است. {xi} = ω را «دنبالهٔ اجرا» (Execution Trace) برنامهٔ زایشی بنامیم—یعنی توالیِ مقادیرِ ممکن برای انتخاب‌های تصادفی. اجرایِ یک‌بارِ برنامه یک چنین دنباله‌ای تولید می‌کند، از این رو اصطلاحِ «برنامهٔ زایشی». فضایِ همهٔ دنباله‌های اجرایِ ممکنِ Ω را می‌توان فضایِ نمونهٔ مدلِ احتمالِ تعریف‌شده توسط برنامهٔ زایشی دید. توزیعِ احتمال روی دنباله‌ها را می‌توان به‌صورتِ حاصلِ‌ضربِ احتمال‌های هر انتخابِ تصادفیِ فردی تعریف کرد:

$$P(\omega) = \prod_i P(x_i \mid x_1, \ldots, x_{i-1})$$

این مشابهِ توزیعِ روی جهان‌ها در OUPM است.

تبدیلِ مفهومیِ هر OUPM به برنامهٔ زایشیِ متناظر سرراست است. این برنامهٔ زایشی برای هر گزارهٔ شمار و برای مقدارِ هر متغیرِ تصادفیِ پایه‌ای که وجودش توسط گزاره‌های شمار دلالت می‌شود، انتخاب‌های تصادفی می‌کند. کارِ اضافیِ اصلی که برنامهٔ زایشی باید انجام دهد ایجادِ ساختارهای داده‌ای است که اشیا، توابع و روابطِ جهان‌های ممکنِ OUPM را بازنمایی می‌کنند. این ساختارهای داده به‌طورِ خودکار توسط موتورِ استنتاجِ OUPM ایجاد می‌شوند، زیرا OUPM فرض می‌کند هر جهانِ ممکن ساختارِ مدلِ مرتبهٔ اول است، در حالی که PPL نمونه‌ای چنین فرضی نمی‌کند.

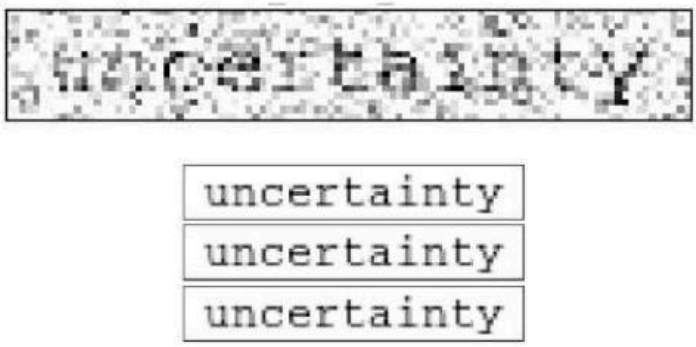
تصاویرِ شکل ۱۵.۱۲ را می‌توان برای فهمِ شهودیِ توزیعِ احتمالِ $P(\Omega)$ استفاده کرد: سطوحِ متفاوتی از نویز را می‌بینیم، و در تصاویرِ کم‌نویزتر، توالی‌هایی از حروف با طول‌های متفاوت را نیز می‌بینیم. فرض کنیم $\omega 1$ دنبالهٔ اجرایِ متناظر با تصویرِ گوشهٔ بالا-راستِ این شکل باشد، شاملِ حروفِ ocflwe. اگر این دنبالهٔ $\omega 1$ را به شبکهٔ بیزی باز کنیم، 4,104 گره خواهد داشت: ۱ گره برای متغیرِ n؛ ۶ گره برای متغیرهای letters[i]؛ ۱ گره برای واریانسِ نویز؛ و ۴,۰۹۶ گره برای پیکسل‌های noisy image. بدین‌ترتیب می‌بینیم این برنامهٔ زایشی مدلِ احتمالِ جهانِ بازی تعریف می‌کند: شمارِ انتخاب‌های تصادفی‌اش از پیش کران‌دار نیست، بلکه به مقدارِ متغیرِ تصادفیِ n بستگی دارد.



شکل ۱۵.۱۲ – پانلِ بالایی دوازده تصویرِ تخریب‌شدهٔ تولیدشده با اجرایِ برنامهٔ زایشی از شکل ۱۵.۱۱ را نشان می‌دهد. شمارِ حروف، هویت‌هایشان، میزانِ نویزِ افزایشی و نویزِ خاصِ هر پیکسل، همگی بخشی از دامنهٔ مدلِ احتمال‌اند. پانلِ پایینی دوازده تصویرِ تخریب‌شدهٔ تولیدشده با اجرایِ برنامهٔ زایشی از شکل ۱۵.۱۵ را نشان می‌دهد. مدلِ مارکوف برای حروف معمولاً توالی‌هایی از حروف می‌دهد که تلفظشان آسان‌تر است.

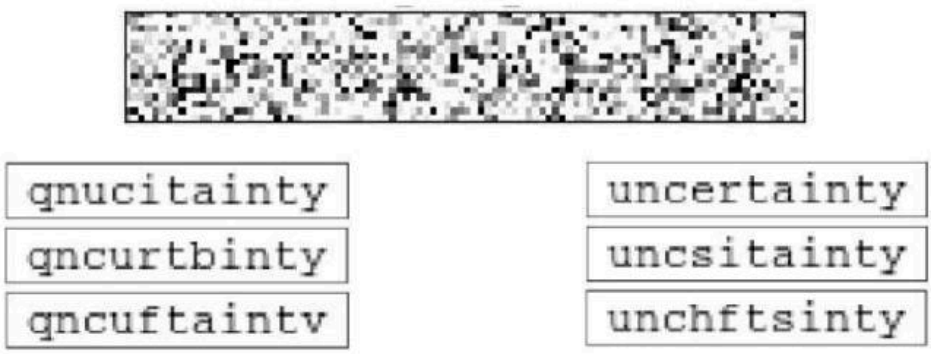
۱۵.۴.۳ نتایجِ استنتاج

این مدل را برای تفسیرِ تصاویرِ حروفِ تخریب‌شده با نویزِ افزایشی اعمال کنیم. شکل ۱۵.۱۳ تصویرِ تخریب‌شده‌ای را نشان می‌دهد، همراه با نتایجِ سه اجرایِ مستقلِ MCMC. برای هر اجرا، رندری از حروفِ موجود در دنبالهٔ اجرا پس از توقفِ زنجیرهٔ مارکوف نشان می‌دهیم. در هر سه مورد، نتیجه توالیِ حروفِ درست است، که پیشنهاد می‌کند توزیعِ پسینی به‌شدت بر تفسیرِ درست متمرکز است.



شکل ۱۵.۱۳ – تصویرِ ورودیِ نویزی (بالا) و نتایجِ استنتاج (پایین) تولیدشده توسط سه اجرا، هر یک از ۲۵ تکرارِ MCMC، با مدلِ شکل ۱۵.۱۱. توجه کنید فرایندِ استنتاج توالیِ حروف را به‌درستی شناسایی می‌کند.

حالا متن را بیشتر تخریب کنیم، آن‌قدر تار کنیم که خواندنش برای انسان‌ها دشوار شود. شکل ۱۵.۱۴ نتایجِ استنتاج را روی این ورودیِ چالش‌برانگیزتر نشان می‌دهد. این بار، اگرچه به نظر می‌رسد استنتاجِ MCMC بر شمارِ درستِ حروف (که می‌دانیم چیست) همگرا شده، نخستین حرف به‌اشتباه به‌عنوانِ q شناسایی شده و دربارهٔ پنج حرف از ده حرفِ بعدی عدم قطعیت هست. در این نقطه، راه‌های ممکنِ بسیاری برای تفسیرِ نتایج هست. می‌تواند استنتاجِ MCMC خوب آمیخته شده و نتایج بازتابِ خوبی از پسینِ واقعی با توجه به مدل و تصویر باشند؛ در آن صورت، عدم قطعیت در برخی حروف و خطایِ حرفِ نخست اجتناب‌ناپذیرند. برای نتایجِ بهتر، ممکن است لازم باشد مدلِ متن را بهبود دهیم یا سطحِ نویز را کاهش دهیم. همچنین می‌تواند استنتاجِ MCMC به‌درستی آمیخته نشده باشد: اگر ۳۰۰ زنجیره را برای ۲۵ هزار یا ۲۵ میلیون تکرار اجرا می‌کردیم، ممکن بود توزیعِ کاملاً متفاوتی از نتایج بیابیم، شاید نشان‌دهندهٔ اینکه حرفِ نخست احتمالاً u است نه q.



شکل ۱۵.۱۴ – بالا: تصویرِ ورودیِ فوق‌العاده نویزی. پایین-چپ: با سه نتیجهٔ استنتاج از ۲۵ تکرارِ MCMC با مدلِ حروفِ مستقل از شکل ۱۵.۱۱. پایین-راست: سه نتیجهٔ استنتاج با مدلِ دوتاییِ حروف (بی‌گرم) از شکل ۱۵.۱۵. هر دو مدل عدم قطعیت را در موردِ برخی حروف گزارش می‌کنند، اما مدلِ دوتایی توالیِ درست را می‌گیرد.

اجرایِ استنتاجِ بیشتر می‌تواند از نظرِ هزینه و زمانِ انتظار گران باشد. علاوه بر این، هیچ آزمونِ خطاناپذیری برای همگراییِ روش‌های استنتاجِ مونت‌کارلو نیست. می‌توانیم بهبودِ الگوریتمِ استنتاج را امتحان کنیم، شاید با طراحیِ توزیعِ پیشنهادیِ بهتر برای MCMC یا با استفاده از سرنخ‌های پایین‌به‌بالا از تصویر برای پیشنهادِ فرضیه‌های اولیهٔ بهتر. این بهبودها به تفکر، پیاده‌سازی و رفع‌اشکالِ اضافی نیاز دارند. سومین جایگزین بهبودِ مدل است. برای نمونه، می‌توانیم دانشِ واژه‌های انگلیسی، مانندِ احتمال‌های جفت‌حرف‌ها را در بر بگیریم. حالا این گزینه را در نظر می‌گیریم.

۱۵.۴.۴ بهبودِ برنامهٔ زایشی با گنجاندنِ مدلِ مارکوف

زبان‌های برنامه‌نویسیِ احتمالاتی به‌گونه‌ای پودمانی‌اند که کاوشِ بهبودهای مدلِ زیربنایی را آسان می‌کنند. شکل ۱۵.۱۵ برنامهٔ زایشی را برای مدلِ بهبودیافته نشان می‌دهد که حروف را به‌صورتِ متوالی تولید می‌کند نه مستقل. این برنامهٔ زایشی از مدلِ مارکوف استفاده می‌کند که هر حرف را به‌شرطِ حرفِ قبلی می‌کشد، با احتمال‌های انتقالِ برآوردشده از فهرستِ مرجعِ واژه‌های انگلیسی.

```
function GENERATE-MARKOV-LETTERS(λ) returns a vector of letters
                                                    n ~ Poisson(λ)
                                                      [] ← letters
                                          ()letter probs ← MARKOV-INITIAL
                                                     :for i = 1 to n
                                  letters[i] ~ Categorical(letter probs)
                                letter probs ← MARKOV-TRANSITION(letters[i])
                                                    return letters
```

شکل ۱۵.۱۵ — برنامهٔ زایشی برای مدلِ بهبودیافتهٔ بازشناسیِ نوریِ نویسه‌ها که حروف را طبقِ مدلِ دوتاییِ حروف (بی‌گرم) تولید می‌کند، که بسامدهای جفت‌حرفی‌اش از فهرستی از واژه‌های انگلیسی برآورد شده.

شکل ۱۵.۱۲ دوازده تصویرِ نمونه‌برداری‌شدهٔ تولیدشده توسط این برنامهٔ زایشی را نشان می‌دهد. توجه کنید توالی‌های حروف به‌طورِ قابلِ‌توجهی انگلیسی‌تر از آن‌هایی‌اند که از برنامهٔ شکل ۱۵.۱۱ تولید شده‌اند. پانلِ راستِ شکل ۱۵.۱۴ نتایجِ استنتاج را از این مدلِ مارکوفِ اعمال‌شده بر تصویرِ پرنویز نشان می‌دهد. تفسیرها با دنبالهٔ اجرایِ مولد سازگاریِ بیشتری دارند، هرچند هنوز تا حدی عدم قطعیت هست.

۱۵.۴.۵ استنتاج در برنامه‌های زایشی

همانندِ OUPMها، استنتاجِ دقیق در برنامه‌های زایشی معمولاً به‌صورتِ بازدارنده گران یا ناممکن است. از سوی دیگر، دیدنِ چگونگیِ انجامِ نمونه‌برداریِ ردّی آسان است: برنامه را اجرا کن، فقط دنباله‌هایی را نگه دار که با شواهد موافق‌اند، و پاسخ‌های پرس‌وجویِ متفاوتِ یافت‌شده در آن دنباله‌ها را بشمار. وزن‌گذاریِ درستنمایی نیز سرراست است: برای هر دنبالهٔ تولیدشده، وزنِ دنباله را با ضربِ همهٔ احتمال‌های مقادیرِ مشاهده‌شده در طولِ مسیر پیگیری کن. وزن‌گذاریِ درستنمایی فقط وقتی خوب کار می‌کند که داده‌ها طبقِ مدل به‌صورتِ معقولی محتمل باشند. در مواردِ دشوارتر، MCMC معمولاً روشِ انتخابی است. MCMC اعمال‌شده بر برنامه‌های احتمالاتی شاملِ نمونه‌برداری و اصلاحِ دنباله‌های اجراست. بسیاری از ملاحظاتِ برخاسته با OUPMها نیز در این‌جا اعمال می‌شوند؛ علاوه بر آن، الگوریتم باید دربارهٔ اصلاح‌های دنبالهٔ اجرا محتاط باشد، مانندِ تغییرِ پیامدِ گزارهٔ if، که ممکن است بقیهٔ دنباله را نامعتبر کند.

بهبودهای بیشتر در استنتاج از چند خطِ کار می‌آیند. برخی بهبودها می‌توانند تغییرهای بنیادین در ردهٔ مسائلِ حل‌شدنی با PPL معین ایجاد کنند، حتی اصولاً؛ «استنتاجِ برافراشته» (Lifted Inference)، که قبلاً برای RPMها توصیف شد، می‌تواند این اثر را داشته باشد. در بسیاری از موارد، MCMC عمومی بسیار کند است و برای امکان‌دادنِ آمیختنِ سریعِ فرایندِ استنتاج به پیشنهادهای خاص‌منظوره نیاز است. کانونِ مهمِ کارهای اخیر در PPLها آسان‌کردنِ تعریف و استفاده از چنین پیشنهادهایی برای کاربران بوده، تا کاراییِ استنتاجِ PPL با الگوریتم‌های استنتاجِ سفارشیِ ابداع‌شده برای مدل‌های خاص هم‌سطح شود.

بسیاری از رویکردهای امیدوارکننده با هدفِ کاهشِ سربارِ استنتاجِ احتمالاتی‌اند. ایدهٔ کامپایلِ توصیف‌شده برای شبکه‌های بیز در بخش ۱۳.۴.۳ را می‌توان برای استنتاج در OUPMها و PPLها اعمال کرد، و معمولاً سرعت‌هایی دو تا سه‌مرتبه‌ای می‌دهد. سامانه‌های مدرنِ PPL بر «راهنماییِ» استنتاج با اطلاعاتِ اضافیِ فراهم‌شده توسط کاربر، مانندِ

طرح‌های نمونه‌برداریِ سفارشی یا تحلیلِ حساسیتِ محاسباتی تکیه می‌کنند. کارهای جدیدتر بر «استنتاجِ مبتنی بر برنامه» تمرکز می‌کنند، که در آن موتورِ استنتاج، ساختارِ برنامه را برای خودکارسازیِ انتخابِ راهبردهای استنتاجِ مناسب تحلیل می‌کند.

همچنین پیشنهادهایی برای سخت‌افزارِ خاص‌منظوره برای الگوریتم‌هایی مانندِ انتقالِ پیام و MCMC بوده. برای نمونه، «سخت‌افزارِ مونت‌کارلو» از بازنمایی‌های احتمالِ کم‌دقت و موازی‌سازیِ فوق‌ریزِ عظیم بهره می‌برد تا بهبودهای 100 تا 10,000 برابری در سرعت و بازدهیِ انرژی تحویل دهد.

روش‌های مبتنی بر یادگیری نیز می‌توانند بهبودهای قابلِ‌توجهی در سرعت بدهند. برای نمونه، «توزیع‌های پیشنهادیِ تطبیقی» (Adaptive Proposal Distributions) می‌توانند به‌تدریج یاد بگیرند چگونه پیشنهادهای MCMC تولید کنند که به‌صورتِ معقولی محتمل‌اند پذیرفته شوند و در کاوشِ چشم‌اندازِ احتمالِ مدل به‌صورتِ معقولی مؤثر باشند تا آمیختنِ سریع تضمین شود. همچنین می‌توان مدل‌های یادگیریِ عمیق (فصل ۲۱ را ببینید) را برای بازنماییِ توزیع‌های پیشنهادی برای نمونه‌برداریِ اهمیت آموزش داد، با استفاده از دادهٔ مصنوعیِ تولیدشده از مدلِ زیربنایی.

به‌طورِ کلی، انتظار می‌رود هر صورت‌بندیِ ساخته‌شده روی زبان‌های برنامه‌نویسیِ عمومی به دیوارِ محاسبه‌پذیری برخورد کند، و موردِ PPLها نیز چنین است. اما اگر فرض کنیم برنامهٔ زیربنایی برای همهٔ ورودی‌ها و همهٔ انتخاب‌های تصادفی متوقف می‌شود، آیا الزامِ اضافیِ انجامِ استنتاجِ احتمالاتی همچنان مسئله را تصمیم‌ناپذیر می‌کند؟

معلوم می‌شود پاسخ بله است، اما فقط برای مدلِ محاسباتی با متغیرهای تصادفیِ پیوستهٔ با دقتِ نامتناهی. در آن صورت، نوشتنِ مدلِ احتمالِ محاسبه‌پذیری ممکن می‌شود که در آن استنتاج مسئلهٔ توقف را رمزگذاری می‌کند. از سوی دیگر، با اعدادِ با دقتِ متناهی و با توزیع‌های احتمالِ نرمی که معمولاً در کاربردهای واقعی استفاده می‌شوند، استنتاج تصمیم‌پذیر باقی می‌ماند.

خلاصهٔ فصل

این فصل بازنمایی‌های بیانیِ مدل‌های احتمال را، مبتنی بر هم منطق و هم برنامه‌ها، کاوش کرده.

- «مدل‌های احتمالِ رابطه‌ای» (RPMها) مدل‌های احتمال را روی جهان‌های مشتق‌شده از معناشناسیِ پایگاه‌داده برای زبان‌های مرتبهٔ اول تعریف می‌کنند؛ آن‌ها وقتی مناسب‌اند که همهٔ اشیا و هویت‌هایشان با قطعیت شناخته شده‌اند.
- با توجه به RPM، اشیای هر جهانِ ممکن با نمادهای ثابتِ RPM متناظرند، و متغیرهای تصادفیِ پایه همهٔ مصداق‌های ممکنِ نمادهای گزاره‌ای با جایگزینیِ اشیا به‌جای هر آرگومان‌اند. بدین‌ترتیب، مجموعهٔ جهان‌های ممکن متناهی است.
- RPMها مدل‌های بسیار موجزی برای جهان‌های با شمارِ زیادِ اشیا فراهم می‌کنند و می‌توانند عدم قطعیتِ رابطه‌ای را مدیریت کنند.
- «مدل‌های احتمالِ جهانِ باز» (OUPMها) بر معناشناسیِ کاملِ منطقِ مرتبهٔ اول بنا می‌شوند و انواعِ تازه‌ای از عدم قطعیت مانندِ عدم قطعیتِ هویتی و وجودی را اجازه می‌دهند.
- «برنامه‌های زایشی» بازنمایی‌هایی از مدل‌های احتمال—شاملِ OUPMها—به‌صورتِ برنامه‌های اجراپذیر در زبانِ برنامه‌نویسیِ احتمالاتی یا PPL هستند. برنامهٔ زایشی توزیعی را روی دنباله‌های اجرایِ برنامه بازنمایی می‌کند. PPLها معمولاً قدرتِ بیانیِ همگانی برای مدل‌های احتمال فراهم می‌کنند.

یادداشت‌های کتاب‌شناختی و تاریخی

هیلپرین (1984) و هاوسون (2003) تاریخِ بلندِ تلاش‌ها برای اتصالِ احتمال و منطق را روایت می‌کنند، که به «گفتارهای تازه» (Nouveaux Essais) لایب‌نیتس در ۱۷۰۴ بازمی‌گردد. این تلاش‌ها معمولاً شاملِ احتمال‌های متصل‌شده مستقیم به جمله‌های منطقی بودند. نخستین صورت‌بندیِ دقیق «منطقِ احتمالِ گزاره‌ای» گایفمن (Gaifman, 1964b) بود. ایده این است که اظهارِ احتمالِ $P(\varphi) \geq p$ قیدی روی توزیعِ روی جهان‌های ممکن است، همان‌گونه که جملهٔ منطقیِ معمولی قیدی روی خودِ جهان‌های ممکن است. هر توزیعِ P که قید را ارضا کند، در معنایِ منطقیِ استاندارد، مدلی از اظهارِ احتمال است، و اظهارِ احتمالی دیگری را دقیقاً وقتی مستلزم می‌شود که مدل‌های نخستین زیرمجموعه‌ای از مدل‌های دوم باشند. درونِ چنین منطقی، می‌توان، برای نمونه، اثبات کرد $P(\alpha \wedge \beta) \leq P(\alpha \Rightarrow \beta)$. ارضاپذیریِ مجموعه‌های اظهارهای احتمال را در حالتِ گزاره‌ای می‌توان با برنامه‌نویسیِ خطی تعیین کرد (هیلپرین، 1984؛ نیلسون، 1986). بدین‌ترتیب، «منطقِ احتمال» را به همان معنایی داریم که «منطقِ زمانی»—سامانهٔ منطقیِ تخصص‌یافته برای استدلالِ احتمالاتی.

برای اعمالِ منطقِ احتمال بر تکلیف‌هایی مانندِ اثباتِ قضیه‌های جالب در نظریهٔ احتمال، به زبانِ بیانی‌تری نیاز بود. گایفمن (1964a) منطقِ احتمالِ مرتبهٔ اولی پیشنهاد کرد، با جهان‌های ممکنِ ساختارهایِ مدلِ مرتبهٔ اول و احتمال‌های متصل‌شده به جمله‌های منطقِ مرتبهٔ اولِ (بدونِ تابع). اسکات و کراوس (1966) نتایجِ گایفمن را برای اجازه‌دادن به

تودرتوییِ نامتناهیِ سورها و مجموعه‌های نامتناهیِ جمله‌ها بسط دادند.

درونِ هوش مصنوعی، مستقیم‌ترین فرزندِ این ایده‌ها در «برنامه‌های منطقیِ احتمالاتی» (لوکاشویچ، 1998) ظاهر می‌شود، که در آن بازهٔ احتمالی به هر بندِ هورنِ مرتبهٔ اول متصل می‌شود و استنتاج با حلِ برنامه‌های خطی، چنان‌که هیلپرین پیشنهاد کرد، انجام می‌شود.

هالپرن (1990) و بیکوس (1990) نیز بر رویکردِ گایفمن بنا کردند و برخی از مسائلِ پایهٔ بازنماییِ دانش را از دیدگاهِ هوش مصنوعی به‌جای نظریهٔ احتمال و منطقِ ریاضی کاوش کردند. زیرحوزهٔ «پایگاه‌های دادهٔ احتمالاتی» (Probabilistic Databases) نیز جمله‌های منطقیِ برچسب‌خورده با احتمال دارد (دالوی و همکاران، 2009)—اما در این مورد احتمال‌ها مستقیماً به چندتایی‌های پایگاهِ داده متصل‌اند. (در هوش مصنوعی و آمار، احتمال به روابطِ عمومی متصل است، در حالی که مشاهده‌ها شواهدِ غیرقابلِ‌انکار دیده می‌شوند.) اگرچه پایگاه‌های دادهٔ احتمالاتی می‌توانند وابستگی‌های پیچیده را مدل کنند، در عمل اغلب می‌بینیم چنین سامانه‌هایی از مفروضاتِ استقلالِ سراسری روی چندتایی‌ها استفاده می‌کنند.

متصل‌کردنِ احتمال‌ها به جمله‌ها تعریفِ مدل‌های احتمالِ کامل و سازگار را بسیار دشوار می‌کند. هر نامساوی، مدلِ احتمالِ زیربنایی را به قرارگرفتنِ در نیم‌فضایی در فضایِ پربعدِ مدل‌های احتمال مقید می‌کند. عطفِ اظهارها با اشتراکِ قیدها متناظر است. تضمینِ اینکه اشتراک نقطه‌ای تکی به دست دهد آسان نیست. در واقع، نتیجهٔ اصلیِ گایفمن (1964a) ساختِ مدلِ احتمالِ تکی است که ۱) به احتمال برای هر جملهٔ زمینیِ ممکن نیاز دارد و ۲) قیدهای احتمال برای بی‌نهایت جملهٔ سوردارِ وجودی.

یکی از راه‌حل‌های این مسئله نوشتنِ نظریهٔ جزئی و سپس «کامل‌کردنِ» آن با برگزیدنِ مدلِ متعارفی در مجموعهٔ مجاز است. نیلسون (1986) انتخابِ «مدلِ بیشینه‌انتروپی» سازگار با قیدهای مشخص‌شده را پیشنهاد کرد. پاسکین (2002) «منطقِ احتمالاتیِ بیشینه‌انتروپی» را با قیدهای بیان‌شده به‌صورتِ وزن‌ها (احتمال‌های نسبی) متصل به بندهای مرتبهٔ اول توسعه داد. چنین مدل‌هایی اغلب «شبکه‌های منطقِ مارکوف» (Markov Logic Networks) یا MLN نامیده می‌شوند (ریچاردسون و دومینگوس، 2006) و به فنِ محبوبی برای کاربردهای شاملِ دادهٔ رابطه‌ای تبدیل شده‌اند. رویکردهای بیشینه‌انتروپی، از جملهٔ MLNها، در برخی موارد می‌توانند نتایجِ غیرشهودی تولید کنند (میلچ، 2006؛ جین و همکاران، 2007، 2010).

از اوایلِ دههٔ ۱۹۹۰، پژوهشگرانی که روی کاربردهای پیچیده کار می‌کردند محدودیت‌های بیانیِ شبکه‌های بیزی را متوجه شدند و زبان‌های گوناگونی برای نوشتنِ «قالب‌هایی» با متغیرهای منطقی توسعه دادند، که از آن‌ها می‌شد برای هر نمونهٔ مسئله شبکه‌های بزرگ به‌طورِ خودکار ساخت (بریس، 1992؛ ولمن و همکاران، 1992). مهم‌ترین چنین زبانی BUGS بود («استنتاجِ بیزی با نمونه‌برداریِ گیبس»؛ گیلکس و همکاران، 1994؛ لان و همکاران، 2013)، که شبکه‌های بیزی را با نمادِ «متغیرِ تصادفیِ نمایه‌دار» (Indexed Random Variable) رایج در آمار ترکیب کرد. (در BUGS، متغیرِ تصادفیِ نمایه‌دار شبیهِ X[i] است، که در آن i بازهٔ صحیحِ تعریف‌شده‌ای دارد.) این زبان‌های جهانِ بسته ویژگیِ کلیدیِ شبکه‌های بیزی را به ارث بردند: هر پایگاهِ دانشِ خوش‌ساخت، مدلِ احتمالِ یکتا و سازگاری را تعریف می‌کند. زبان‌های جهانِ بستهٔ دیگر بر قابلیت‌های بازنمایی و استنتاجیِ برنامه‌نویسیِ منطقی (پول، 1993؛ ساتو و کامیا، 1997؛ کرستینگ و همکاران، 2000) و شبکه‌های معنایی (کولر و ففر، 1998؛ ففر، 2000) تکیه کردند.

پژوهشِ مدل‌های احتمالِ جهانِ باز چند خاستگاه دارد. در آمار، مسئلهٔ «پیوندِ سوابق» (Record Linkage) وقتی پدید می‌آید که سوابقِ داده حاوی شناسه‌های یکتای استاندارد نیستند—برای نمونه، استنادهای گوناگونِ این کتاب ممکن است نویسندهٔ نخستش را «استوارت جی. راسل» یا «اس. راسل» یا حتی «استوارت راسل» نامیده باشند. نویسندگانِ دیگر نامِ «اس. راسل» را شریک‌اند. صدها شرکت فقط برای حلِ مسائلِ پیوندِ سوابق در داده‌های مالی، پزشکی، سرشماری و دیگر داده‌ها وجود دارند. تحلیلِ احتمالاتی به کارِ دان (1946) بازمی‌گردد؛ مدلِ فِلِگی–سونتر (1969)، که اساساً بیزِ سادهٔ اعمال‌شده بر تطبیق است، هنوز بر رویهٔ کنونی مسلط است. عدم قطعیتِ هویتی همچنین در ردیابیِ چندهدف در نظر گرفته می‌شود (سیتلر، 1964)، که تاریخچه‌اش در فصل ۱۴ ترسیم شده.

در هوش مصنوعی، فرضِ کاری تا دههٔ ۱۹۹۰ این بود که حسگرها می‌توانند جمله‌های منطقی با شناسه‌های یکتا برای اشیا تأمین کنند، چنان‌که موردِ شیکی بود. در حوزهٔ فهمِ زبانِ طبیعی، چارنیاک و گلدمن (1992) تحلیلِ احتمالاتیِ «هم‌ارجاعی» (Coreference) را پیشنهاد کردند، جایی که دو عبارتِ زبانی (مثلاً «اوباما» و «رئیس‌جمهور») ممکن است به همان نهاد اشاره کنند. هوانگ و راسل (1998) و پاسولا و همکاران (1999) تحلیلِ بیزیِ عدم قطعیتِ هویتی را برای نظارتِ ترافیکی توسعه دادند. پاسولا و همکاران (2003) مدلِ زایشیِ پیچیده‌ای برای نویسندگان، مقاله‌ها و رشته‌های استناد توسعه دادند که هم عدم قطعیتِ رابطه‌ای و هم هویتی را در بر داشت، و دقتِ بالایی برای استخراجِ اطلاعاتِ استناد نشان داد.

نخستین زبانِ صوری برای مدل‌های احتمالِ جهانِ باز BLOG بود (میلچ و همکاران، 2005؛ میلچ، 2006)، که با موتورِ استنتاجِ MCMC همه‌منظوره‌ای (بسیار کند) همراه بود. لاسکی (2008) زبانِ مدل‌سازیِ جهانِ بازِ دیگری به نامِ «شبکه‌های بیزیِ چندنهادی» (Multi-Entity Bayesian Networks) توصیف می‌کند. سامانهٔ پایشِ لرزه‌ایِ سراسریِ NET-VISA توصیف‌شده در متن از آرورا و همکاران (2013) است. سامانهٔ رتبه‌بندیِ الو در ۱۹۵۹ توسط آرپاد الو (1978) توسعه یافت، اما اساساً همان مدلِ «حالتِ V» تورستون (1927) است. مدلِ TrueSkill مایکروسافت (هربریش و همکاران، 2007؛ مینکا و همکاران، 2018) بر نسخهٔ بیزیِ مارک گلیکمن (1999) از الو مبتنی است و اکنون رویِ PPL infer.NET اجرا می‌شود.

ارتباطِ داده برای ردیابیِ چندهدف نخستین بار در محیطِ احتمالاتی توسط سیتلر (1964) توصیف شد. نخستین الگوریتمِ عملی برای مسائلِ بزرگ‌مقیاس «ردیابِ چندفرضیه‌ای» (Multiple Hypothesis Tracker) یا الگوریتمِ MHT بود (رید، 1979). مقاله‌های مهم توسط بار-شالوم و فورتمن (1988) و بار-شالوم (1992) گردآوری شده‌اند. توسعهٔ الگوریتمِ MCMC برای ارتباطِ داده از پاسولا و همکاران (1999) است، که آن را روی مسائلِ نظارتِ ترافیکی اعمال کردند. اوه و همکاران (2009) تحلیلِ صوری و مقایسه‌های تجربی با روش‌های دیگر فراهم می‌کنند. شولتز و همکاران (2003) روشِ ارتباطِ داده مبتنی بر فیلترِ ذره‌ای را توصیف می‌کنند.

اینگمار کاکس پیچیدگیِ ارتباطِ داده را تحلیل کرد (کاکس، 1993؛ کاکس و هینگورانی، 1994) و موضوع را به توجهِ جامعهٔ بینایی آورد. او همچنین کاربردپذیریِ الگوریتمِ مجارستانیِ زمانِ چندجمله‌ای را برای مسئلهٔ یافتنِ محتمل‌ترین تخصیص‌ها ذکر کرد، که مدت‌ها در جامعهٔ ردیابی مسئله‌ای ناشدنی دانسته می‌شد. خودِ الگوریتم توسط کان (1955) منتشر شد، بر اساسِ ترجمه‌هایی از مقاله‌هایی که در ۱۹۳۱ توسط دو ریاضی‌دانِ مجارستانی، دِنش کونیگ و ینو اِگرواری منتشر شده بودند. قضیهٔ پایه، با این حال، قبلاً در نسخهٔ خطیِ لاتینِ منتشرنشده‌ای توسط ریاضی‌دانِ مشهورِ کارل گوستاو یاکوبی (۱۸۵۱–۱۸۰۴) اثبات شده بود.

ایدهٔ اینکه برنامه‌های احتمالاتی نیز می‌توانند مدل‌های احتمالِ پیچیده را بازنمایی کنند از کولر و همکاران (1997) است. نخستین PPL کارآمد IBAL اثرِ آوی ففر (2001، 2007) بود، مبتنی بر زبانِ تابعیِ ساده. BLOG را می‌توان PPL اظهاری دانست. ارتباطِ میان PPLهای اظهاری و تابعی توسط مک‌آلیستر و همکاران (2008) کاوش شد.

CHURCH (گودمن و همکاران، 2008)، PPLای مبتنی بر زبانِ Scheme، ایدهٔ سوارشدن بر زبانِ برنامه‌نویسیِ موجود را پیشگام کرد. CHURCH همچنین نخستین الگوریتمِ استنتاجِ MCMC را برای مدل‌های با توابعِ تصادفیِ مرتبهٔ بالاتر معرفی کرد و در جامعهٔ علومِ شناختی به‌عنوانِ راهی برای مدل‌کردنِ شکل‌های پیچیدهٔ یادگیریِ انسانی علاقه ایجاد کرد (لیک و همکاران، 2015). PPLها همچنین به شیوه‌های جالبی به نظریهٔ محاسبه‌پذیری (آکرمن و همکاران، 2013) و پژوهشِ زبان‌های برنامه‌نویسی متصل می‌شوند.

در دههٔ ۲۰۱۰، ده‌ها PPL بر اساسِ طیفِ وسیعی از زبان‌های برنامه‌نویسیِ زیربنایی پدید آمدند. فیگارو (Figaro)، مبتنی بر زبانِ اسکالا، برای انواعِ گوناگونی از کاربردها استفاده شده (ففر، 2016). گن (Gen؛ کوزومانو-تاونر و همکاران، 2019)، مبتنی بر جولیا و TensorFlow، برای ادراکِ ماشینیِ بلادرنگ و نیز یادگیریِ ساختارِ بیزی برای تحلیلِ سری‌های زمانی استفاده شده. PPLهای ساخته‌شده روی چارچوب‌های یادگیریِ عمیق شاملِ Pyro (بینگهام و همکاران، 2019؛ ساخته‌شده روی PyTorch) و Edward (تران و همکاران، 2017؛ ساخته‌شده روی TensorFlow) هستند.

تلاش‌هایی برای دسترسی‌پذیرترکردنِ برنامه‌نویسیِ احتمالاتی برای افرادِ بیشتر، مانندِ کاربرانِ پایگاه‌داده و صفحه‌گسترده، بوده. Tabular (گوردون و همکاران، 2014) زبانِ طرح‌وارهٔ رابطه‌ایِ صفحه‌گسترده‌مانندی را روی infer.NET فراهم می‌کند. BayesDB (سعد و منسینگکا، 2017) به کاربران اجازه می‌دهد برنامه‌های احتمالاتی را با زبانی شبیهِ SQL ترکیب و پرس‌وجو کنند.

پانوشت‌ها:

[1] نامِ «مدلِ احتمالِ رابطه‌ای» توسط ففر (2000) به بازنماییِ کمی متفاوتی داده شد، اما ایده‌های زیربنایی یکسان‌اند.

[2] نظریه‌پردازِ بازی به مشتریِ ناصادق توصیه می‌کرد برای اجتناب از شناسایی، گاهی کتابِ خوبی از رقیب توصیه کند. فصل ۱۸ را ببینید.

[3] شرایطِ فنیِ خاصی برای تعریفِ توزیعِ درست توسط RPM لازم است. نخست، وابستگی‌ها باید بدونِ دور باشند؛ در غیرِ این صورت شبکهٔ بیزیِ حاصل دور خواهد داشت. دوم، وابستگی‌ها باید (معمولاً) خوش‌بنیاد باشند: هیچ زنجیرهٔ نیاییِ نامتناهی نمی‌تواند وجود داشته باشد، مانندِ آنچه ممکن است از وابستگی‌های بازگشتی برخیزد. برای استثنایی بر این قاعده، تمرین HAMD.۱۵ را ببینید.

[4] چند بستهٔ آماری این کد را به‌عنوانِ تعریف‌کنندهٔ RPM می‌بینند، نه صرفاً ساختنِ شبکهٔ بیزی برای انجامِ استنتاج در RPM. این دیدگاه، با این حال، نقشِ مهمی برای نحوِ RPM از دست می‌دهد: بدونِ نحو با معناشناسیِ روشن، هیچ راهی برای یادگیریِ ساختارِ مدل از داده نیست.

[5] نامِ «سیبیل» از موردِ مشهورِ اختلالِ شخصیتِ چندگانه می‌آید.

[6] مصداق‌سازیِ خودپشتیبانِ مجموعه‌ای از متغیرها، مصداق‌سازی‌ای است که در آن والدهای هر متغیرِ مجموعه نیز در مجموعه‌اند.

[7] همانندِ پرولوگ، ممکن است بی‌نهایت مجموعهٔ جانشینی با اندازهٔ نامحدود باشد؛ طراحیِ رابط‌های اکتشافی برای چنین پاسخ‌هایی چالشِ تجسمِ جالبی است.

[8] پاسخ بله است، آن‌ها همان مقاله‌اند. «کنفرانسِ ملیِ هوش مصنوعی» (توجه کنید «fi» گم شده، به دلیلِ خطایی در خراشیدنِ نویسهٔ لیگاتور) نامِ دیگری برای کنفرانسِ AAAI است؛ کنفرانس در سیاتل برگزار شد، در حالی که ناشرِ مجموعه‌مقالات در کمبریج است.

[9] حالتِ چندنویسنده همان ساختارِ کلی را دارد اما کمی پیچیده‌تر است. بخش‌های نشان‌داده‌نشدهٔ مدل—NamePrior، PaperTitlePrior و HMMGrammar—مدل‌های احتمالِ سنتی‌اند. برای نمونه، NamePrior مخلوطی از توزیعِ رسته‌ای روی نام‌های واقعی و مدلِ سه‌تاییِ حروف (بخش ۲۳.۱) برای پوششِ نام‌های دیده‌نشدهٔ قبلی است، که هر دو از داده‌های پایگاهِ سرشماریِ آمریکا آموزش دیده‌اند.

فصل ۱۶: تصمیم‌گیری ساده (Making Simple Decisions)

در این فصل می‌بینیم عامل چگونه باید تصمیم بگیرد تا در جهانی نامطمئن به آنچه می‌خواهد برسد—دست‌کم تا آن‌جا که ممکن است و به‌طورِ میانگین.

در این فصل، جزئیاتِ چگونگیِ ترکیبِ نظریهٔ مطلوبیت با نظریهٔ احتمال برای به‌دست‌آوردنِ «عاملِ نظریه‌تصمیم» (Decision-Theoretic Agent)—عاملی که می‌تواند بر اساسِ آنچه باور دارد و آنچه می‌خواهد تصمیم‌های عقلایی بگیرد—را پر می‌کنیم. چنین عاملی می‌تواند در بافت‌هایی تصمیم بگیرد که عدم قطعیت و هدف‌های متعارض، عاملِ منطقی را بدونِ راهی برای تصمیم‌گیری رها می‌کنند. عاملِ هدف‌محور تمایزِ دودویی میان حالت‌های خوب (هدف) و بد (غیرهدف) دارد، در حالی که عاملِ نظریه‌تصمیم به حالت‌ها دامنه‌ای پیوسته از مقادیر تخصیص می‌دهد و بنابراین می‌تواند آسان‌تر حالتِ بهتری را برگزیند حتی وقتی بهترین حالت در دسترس نیست.

بخش ۱۶.۱ اصلِ پایهٔ نظریهٔ تصمیم را معرفی می‌کند: بیشینه‌سازیِ مطلوبیتِ موردانتظار. بخش ۱۶.۲ نشان می‌دهد رفتارِ عاملِ عقلایی را می‌توان با بیشینه‌کردنِ تابعِ مطلوبیت مدل کرد. بخش ۱۶.۳ ماهیتِ توابعِ مطلوبیت را با جزئیاتِ بیشتر بحث می‌کند، و به‌طورِ خاص رابطه‌شان را با کمیت‌های فردی مانندِ پول. بخش ۱۶.۴ نشان می‌دهد چگونه توابعِ مطلوبیتی را که به چند کمیت بستگی دارند مدیریت کنیم. در بخش ۱۶.۵، پیاده‌سازیِ سامانه‌های تصمیم‌گیری را توصیف می‌کنیم. به‌طورِ خاص، صورت‌بندی‌ای به نام «شبکهٔ تصمیم» (Decision Network) معرفی می‌کنیم (که «نمودارِ تأثیر» (Influence Diagram) نیز نامیده می‌شود) که شبکه‌های بیزی را با گنجاندنِ کنش‌ها و مطلوبیت‌ها بسط می‌دهد. بخش ۱۶.۶ نشان می‌دهد عاملِ نظریه‌تصمیم چگونه می‌تواند ارزشِ کسبِ اطلاعاتِ تازه برای بهبودِ تصمیم‌هایش را محاسبه کند.

در حالی که بخش‌های ۱۶.۱ تا ۱۶.۶ فرض می‌کنند عامل با تابعِ مطلوبیتِ داده‌شده و شناخته‌شده عمل می‌کند، بخش ۱۶.۷ این فرض را سست می‌کند. پیامدهای عدم قطعیتِ ترجیح از جانبِ ماشین را بحث می‌کنیم—که مهم‌ترینش «اطاعت از انسان» (Deference to Humans) است.

۱۶.۱ ترکیبِ باورها و خواسته‌ها تحتِ عدم قطعیت

با عاملی شروع می‌کنیم که، مانندِ همهٔ عامل‌ها، باید تصمیم بگیرد. کنش‌هایی a در دسترس دارد. ممکن است عدم قطعیت دربارهٔ حالتِ کنونی باشد، پس فرض می‌کنیم عامل به هر حالتِ کنونیِ ممکنِ s احتمالی P(s) تخصیص می‌دهد. همچنین ممکن است عدم قطعیت دربارهٔ پیامدهای کنش باشد؛ مدلِ انتقال با P(s'|s,a)، احتمالِ رسیدنِ کنشِ a در حالتِ s به حالتِ s'، داده می‌شود. چون عمدتاً به پیامدِ s' علاقه‌مندیم، از نمادِ مخففِ P(RESULT(a)=s')، احتمالِ رسیدن به s' با انجامِ a در حالتِ کنونی، هرچه باشد، نیز استفاده می‌کنیم. این دو چنین به هم مربوط‌اند:

```
P(RESULT(a)=s′) = ∑s P(s) P(s′|s, a)
```

نظریهٔ تصمیم، در ساده‌ترین شکلش، با انتخاب میان کنش‌ها بر اساسِ مطلوبیتِ پیامدهای بی‌واسطه‌شان سروکار دارد؛ یعنی فرض می‌شود محیط «اپیزودیک» (Episodic) است، به معنایی که در صفحهٔ ۴۵ تعریف شد. (این فرض در فصل ۱۷ سست می‌شود.) ترجیح‌های عامل توسط «تابعِ مطلوبیت» (Utility Function) به نام U(s) در بر گرفته می‌شوند، که عددی تکی برای بیانِ مطلوبیتِ یک حالت تخصیص می‌دهد. «مطلوبیتِ موردانتظار» (Expected Utility) کنشِ a با توجه به شواهد، EU(a)، صرفاً میانگینِ مقدارِ مطلوبیتِ پیامدهاست، وزن‌داده‌شده با احتمالِ وقوعِ پیامد:

```
EU(a) = ∑s′ P(RESULT(a)=s′) U(s′)        (16.1)
```

«اصلِ بیشینهٔ مطلوبیتِ موردانتظار» (MEU) می‌گوید عاملِ عقلایی باید کنشی را برگزیند که مطلوبیتِ موردانتظارِ عامل را بیشینه می‌کند:

```
action = argmax_a EU(a)
```

از جهتی، اصلِ MEU را می‌توان نسخه‌ای برای رفتارِ هوشمند دانست. تنها کاری که عاملِ هوشمند باید بکند محاسبهٔ کمیت‌های گوناگون، بیشینه‌کردنِ مطلوبیت روی کنش‌هایش، و سپس حرکت است. اما این بدان معنا نیست مسئلهٔ هوش مصنوعی با تعریف حل شده!

اصلِ MEU این تصورِ کلی را صوری می‌کند که عاملِ هوشمند باید «کارِ درست» را انجام دهد، اما آن توصیه را عملیاتی نمی‌کند. برآوردِ توزیعِ احتمالِ P(s) روی حالت‌های ممکنِ جهان، که در P(RESULT(a)=s') جمع می‌شود، به ادراک، یادگیری، بازنماییِ دانش و استنتاج نیاز دارد. محاسبهٔ خودِ P(RESULT(a)=s') به مدلِ علّی از جهان نیاز دارد. ممکن است کنش‌های بسیاری برای درنظرگرفتن باشند، و محاسبهٔ مطلوبیت‌های پیامدِ U(s') خودش ممکن است به جستجو یا برنامه‌ریزیِ بیشتر نیاز داشته باشد، زیرا عامل ممکن است نداند حالتی چقدر خوب است تا وقتی بداند از آن حالت به کجا می‌تواند برسد. سامانهٔ هوش مصنوعیِ عمل‌کننده از جانبِ انسان ممکن است تابعِ مطلوبیتِ واقعیِ انسان را نداند، پس ممکن است عدم قطعیت دربارهٔ U باشد. به‌طورِ خلاصه، نظریهٔ تصمیم نوشدارویی نیست که مسئلهٔ هوش مصنوعی را حل کند—اما آغازِ چارچوبِ ریاضیِ پایه‌ای را فراهم می‌کند که به‌اندازهٔ کافی عمومی است تا مسئلهٔ هوش مصنوعی را تعریف کند.

اصلِ MEU رابطهٔ روشنی با ایدهٔ معیارهای عملکردِ معرفی‌شده در فصل ۲ دارد. ایدهٔ پایه ساده است. محیط‌هایی را در نظر بگیرید که می‌توانند عامل را به داشتنِ تاریخچهٔ حس‌گذارهٔ معینی برسانند، و عامل‌های متفاوتی را که می‌توانستیم طراحی کنیم در نظر بگیرید. اگر عامل چنان عمل کند که تابعِ مطلوبیتی را بیشینه کند که به‌درستی معیارِ عملکرد را بازتاب می‌دهد، آنگاه عامل بالاترین امتیازِ عملکردِ ممکن را به دست خواهد آورد (میانگین‌گرفته روی همهٔ محیط‌های ممکن). این توجیهِ مرکزیِ خودِ اصلِ MEU است. اگرچه این ادعا ممکن است بدیهی‌گونه به نظر برسد، در واقع گذارِ بسیار مهمی از معیارِ عملکردِ بیرونی به تابعِ مطلوبیتِ درونی را در بر دارد. معیارِ عملکرد برای یک تاریخچه—توالی‌ای از

حالت‌ها—امتیاز می‌دهد. بدین‌ترتیب، به‌صورتِ پس‌نگرانه پس از کامل‌کردنِ توالی‌ای از کنش‌ها توسط عامل اعمال می‌شود. تابعِ مطلوبیت دقیقاً بر حالتِ بعدی (بلافاصله بعدی) اعمال می‌شود، پس می‌توان از آن برای هدایتِ گام‌به‌گامِ کنش‌ها استفاده کرد.

۱۶.۲ مبنای نظریهٔ مطلوبیت

به‌صورتِ شهودی، اصلِ بیشینهٔ مطلوبیتِ موردانتظار (MEU) راهِ معقولی برای تصمیم‌گیری به نظر می‌رسد، اما به هیچ وجه بدیهی نیست که تنها راهِ عقلایی است. بالاخره، چرا بیشینه‌کردنِ میانگینِ مطلوبیت باید این‌قدر خاص باشد؟ چه اشکالی دارد عاملی که جمعِ وزن‌دارِ مکعب‌های مطلوبیت‌های ممکن را بیشینه می‌کند، یا سعی می‌کند بدترین زیانِ ممکن را کمینه کند؟ آیا عاملی می‌تواند صرفاً با بیانِ ترجیح‌ها میان حالت‌ها، بدونِ دادنِ مقادیرِ عددی به آن‌ها، عقلایی عمل کند؟ سرانجام، چرا تابعِ مطلوبیتی با ویژگی‌های لازم باید اصلاً وجود داشته باشد؟ خواهیم دید.

۱۶.۲.۱ قیدهای ترجیح‌های عقلایی

این پرسش‌ها را می‌توان با نوشتنِ برخی قیدها روی ترجیح‌هایی که عاملِ عقلایی باید داشته باشد و سپس نشان‌دادنِ اینکه اصلِ MEU را می‌توان از قیدها اشتقاق کرد، پاسخ داد. از نمادِ زیر برای توصیفِ ترجیح‌های عامل استفاده می‌کنیم:

```
A ≻ B      عامل A را بر B ترجیح می‌دهد.
A ~ B      عامل میان A و B بی‌تفاوت است.
A ≽ B      عامل A را بر B ترجیح می‌دهد یا میان‌شان بی‌تفاوت است.
```

حالا پرسشِ بدیهی این است که A و B چه جور چیزهایی‌اند؟ آن‌ها می‌توانند حالت‌های جهان باشند، اما بیشترِ اوقات دربارهٔ آنچه واقعاً ارائه می‌شود عدم قطعیت هست. برای نمونه، مسافرِ هواپیمایی که «ماکارونی یا مرغ» به او پیشنهاد می‌شود نمی‌داند زیرِ پوششِ فویل چه چیزی نهفته است.[1] ماکارونی می‌تواند خوشمزه یا منجمد باشد، مرغ آبدار یا فراتر از شناسایی پخته‌شده. می‌توانیم مجموعهٔ پیامدهای هر کنش را «لاتاری» (Lottery) بدانیم—هر کنش را بلیطِ لاتاری تصور کنید. لاتاریِ L با پیامدهای ممکنِ S1,...,Sn که با احتمال‌های p1,...,pn رخ می‌دهند، چنین نوشته می‌شود:

```
L = [p1, S1; p2, S2; ... pn, Sn]
```

به‌طورِ کلی، هر پیامدِ Si از لاتاری می‌تواند یا حالتِ اتمی باشد یا لاتاریِ دیگر. مسئلهٔ اصلیِ نظریهٔ مطلوبیت فهمِ این است که ترجیح‌های میانِ لاتاری‌های پیچیده چگونه به ترجیح‌های میانِ حالت‌های زیربناییِ آن لاتاری‌ها مربوط می‌شوند. برای پرداختن به این مسئله، شش قید را فهرست می‌کنیم که از هر رابطهٔ ترجیحِ معقولی می‌خواهیم پیروی کند:

- **ترتیب‌پذیری (Orderability):** با توجه به هر دو لاتاری، عاملِ عقلایی باید یا یکی را ترجیح دهد یا آن‌ها را به‌یکسان ترجیح‌پذیر ارزیابی کند. یعنی عامل نمی‌تواند از تصمیم‌گیری اجتناب کند. چنان‌که در صفحهٔ ۳۹۴ ذکر شد، امتناع از شرط‌بندی مانند امتناع از گذشتنِ زمان است. دقیقاً یکی از $(A \succ B)$، $(B \succ A)$ یا $(A \sim B)$ برقرار است.
- **ترایایی (Transitivity):** با توجه به هر سه لاتاری، اگر عامل A را بر B و B را بر C ترجیح دهد، آنگاه عامل باید A را بر C ترجیح دهد.

$$(A \succ C) \Rightarrow (B \succ C) \wedge (A \succ B)$$

- **پیوستگی (Continuity):** اگر لاتاریِ B در ترجیح میان A و C باشد، آنگاه احتمالِ pای هست که برای آن عاملِ عقلایی میانِ گرفتنِ قطعیِ B و لاتاریِ $[p,A;\ 1\text{-}p,C]$ بی‌تفاوت باشد.

$$A \succ B \succ C \Rightarrow \exists p\ [p,\ A;\ 1-p,\ C] \sim B$$

- **جایگزینی‌پذیری (Substitutability):** اگر عامل میان دو لاتاریِ A و B بی‌تفاوت باشد، آنگاه عامل میان دو لاتاریِ پیچیده‌تر که یکسان‌اند به‌جز اینکه B در یکی از آن‌ها به‌جای A جایگزین شده، بی‌تفاوت است. این صرف‌نظر از احتمال‌ها و دیگر پیامد(های) در لاتاری‌ها برقرار است.

$$A \sim B \Rightarrow [p,\ A;\ 1-p,\ C] \sim [p,\ B;\ 1-p,\ C]$$

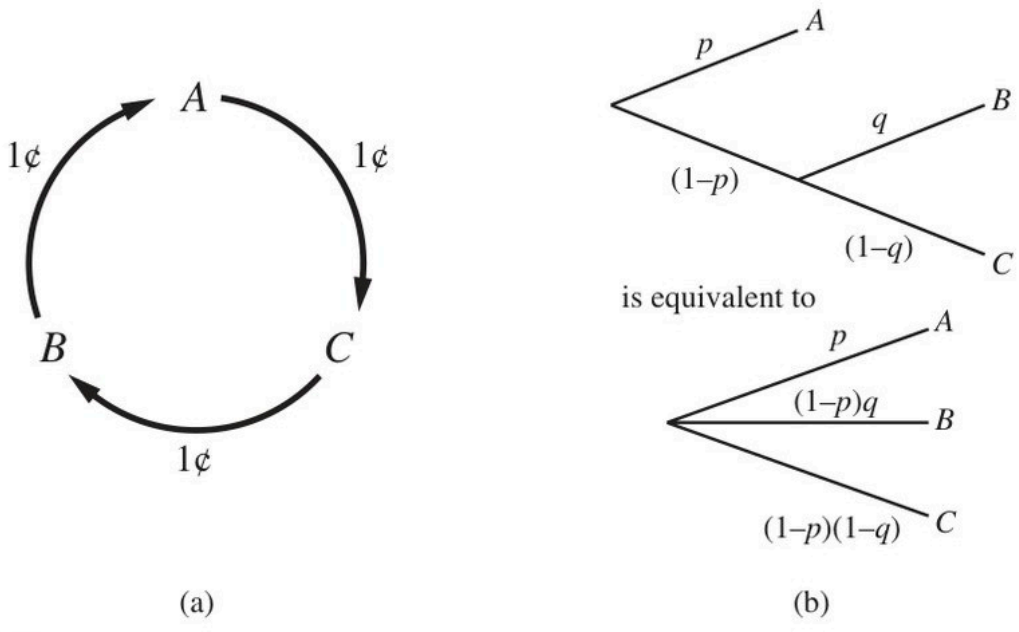
این همچنین برقرار است اگر در این اصل $\sim$ را با $\succ$ جایگزین کنیم.

- **یکنواختی (Monotonicity):** فرض کنیم دو لاتاری همان دو پیامدِ ممکنِ A و B را دارند. اگر عامل A را بر B ترجیح دهد، آنگاه عامل باید لاتاریِ با احتمالِ بالاترِ A را ترجیح دهد (و برعکس).

$$A \succ B \Rightarrow (p \succ q \Leftrightarrow [p,\ A;\ 1-p,\ B] \succ [q,\ A;\ 1-q,\ B])$$

- **تجزیه‌پذیری (Decomposability):** لاتاری‌های مرکب را می‌توان با قوانینِ احتمال به لاتاری‌های ساده‌تر کاهش داد. این «قانونِ بدونِ لذت در قمار» نامیده شده: چنان‌که شکل ۱۶.۱(ب) نشان می‌دهد، دو لاتاریِ متوالی را به لاتاریِ معادلِ تکی فشرده می‌کند.[۲]

$$[p,\ A;\ (1-p)q,\ B;\ (1-p)(1-q),\ C] \sim [p,\ A;\ 1-p,\ [q,\ B;\ 1-q,\ C]]$$



شکل ۱۶.۱ – (الف) ترجیح‌های ناترایای $A \succ B \succ C \succ A$ می‌توانند به رفتارِ غیرعقلایی بینجامند: چرخه‌ای از مبادله‌ها که هر یک یک سنت هزینه دارد. (ب) اصلِ تجزیه‌پذیری. (لاتاریِ مرکب [p,A; 1-p,[q,B; 1-q,C]] معادلِ لاتاریِ [p,A; (1-p)q,B; (1-p)(1-q),C] است.)

این قیدها به‌عنوانِ «اصل‌های نظریهٔ مطلوبیت» شناخته می‌شوند. هر اصل را می‌توان با نشان‌دادنِ اینکه عاملی که آن را نقض می‌کند در برخی موقعیت‌ها رفتارِ آشکارا غیرعقلایی از خود نشان می‌دهد، انگیزه‌مند کرد. برای نمونه، می‌توانیم ترایایی را با ساختنِ عاملی با ترجیح‌های ناترایا انگیزه‌مند کنیم که همهٔ ثروتش را به ما می‌دهد: اگر $A \succ B \succ C \succ A$ باشد، آنگاه عامل حاضر است A را با B معاوضه کند (و یک سنت بپردازد)، سپس B را با C (و یک سنت)، سپس C را با A (و یک سنت). چرخهٔ حاصل می‌تواند برای همیشه ادامه یابد و عامل را بدونِ پول و با A که ظاهراً ارزشِ کمتری نسبت به چیزی که با آن شروع کرده دارد، رها کند.

۱۶.۲.۲ ترجیح‌های عقلایی به مطلوبیت می‌انجامند

توجه کنید اصل‌های نظریهٔ مطلوبیت در واقع اصل‌هایی دربارهٔ ترجیح‌هایند—آن‌ها هیچ‌چیز دربارهٔ تابعِ مطلوبیت نمی‌گویند. اما در واقع از اصل‌های مطلوبیت می‌توانیم پیامدهای زیر را اشتقاق کنیم (برای اثبات، فون نویمان و مورگنشترن، 1944 را ببینید):

- **وجودِ تابعِ مطلوبیت:** اگر ترجیح‌های عامل از اصل‌های مطلوبیت پیروی کنند، آنگاه تابعِ U وجود دارد به‌گونه‌ای که $U(A) > U(B)$ اگر و فقط اگر A بر B ترجیح داده شود، و $U(A) = U(B)$ اگر و فقط اگر عامل میان A و B بی‌تفاوت باشد. یعنی $A \succ B \Leftrightarrow U(A) > U(B)$ و $A \sim B \Leftrightarrow U(A) = U(B)$.
- **مطلوبیتِ موردانتظارِ لاتاری:** مطلوبیتِ لاتاری، جمعِ احتمالِ هر پیامد ضرب‌درِ مطلوبیتِ آن پیامد است:

```
U([p1, S1; ...; pn, Sn]) = ∑i pi U(Si)
```

به بیانِ دیگر، وقتی احتمال‌ها و مطلوبیت‌های حالت‌های پیامدِ ممکن مشخص شدند، مطلوبیتِ لاتاریِ مرکبی که آن حالت‌ها را شامل می‌شود کاملاً تعیین می‌شود. چون پیامدِ کنشِ ناجبرگرایانه لاتاری است، نتیجه می‌شود عامل می‌تواند عقلایی عمل کند—یعنی سازگار با ترجیح‌هایش—فقط با برگزیدنِ کنشی که مطلوبیتِ موردانتظار را طبقِ معادلهٔ (16.1) بیشینه می‌کند.

قضیه‌های پیشین ثابت می‌کنند (با فرضِ قیدهای ترجیحِ عقلایی) تابعِ مطلوبیتی برای هر عاملِ عقلایی وجود دارد. قضیه‌ها ثابت نمی‌کنند تابعِ مطلوبیت یکتاست. در واقع، دیدنِ این آسان است که رفتارِ عامل تغییر نمی‌کرد اگر تابعِ مطلوبیتش U(S) طبقِ:

$$U'(S) = aU(S) + b \qquad (16.2)$$

تبدیل می‌شد، که در آن a و b ثابت‌اند و $a > 0$؛ «تبدیلِ خطی-آفینِ مثبت» (Positive Affine Transformation).[3] این واقعیت در فصل ۵ (صفحهٔ ۱۶۷) برای بازی‌های دوشانسیِ دونفره ذکر شد؛ در این‌جا می‌بینیم برای همهٔ انواعِ سناریوهای تصمیم صدق می‌کند. مانندِ بازی، در محیطِ جبرگرایانه عامل فقط به رتبه‌بندیِ ترجیحی روی حالت‌ها نیاز دارد—اعداد مهم نیستند. این «تابعِ مقدار» (Value Function) یا «تابعِ مطلوبیتِ ترتیبی» (Ordinal Utility Function) نامیده می‌شود.

مهم است به یاد داشته باشیم وجودِ تابعِ مطلوبیتی که رفتارِ ترجیحیِ عامل را توصیف می‌کند لزوماً به این معنا نیست که عامل در تأملاتِ خودش صریحاً آن تابعِ مطلوبیت را بیشینه می‌کند. چنان‌که در فصل ۲ نشان دادیم، رفتارِ عقلایی را می‌توان به هر تعدادی از راه‌ها تولید کرد. عاملِ عقلایی ممکن است با جستجوی جدولی پیاده‌سازی شود (اگر شمارِ حالت‌های ممکن به‌اندازهٔ کافی کوچک باشد). با مشاهدهٔ رفتارِ عاملِ عقلایی، ناظر می‌تواند دربارهٔ تابعِ مطلوبیتی بیاموزد که بازنمایی می‌کند عامل واقعاً در تلاش برای رسیدن به چه چیزی است (حتی اگر خودِ عامل نداند). در بخش ۱۶.۷ به این نکته بازمی‌گردیم.

۱۶.۳ توابعِ مطلوبیت

توابعِ مطلوبیت از لاتاری‌ها به اعدادِ حقیقی نگاشت می‌کنند. می‌دانیم باید از اصل‌های ترتیب‌پذیری، ترایایی، پیوستگی، جایگزینی‌پذیری، یکنواختی و تجزیه‌پذیری پیروی کنند. آیا این تنها چیزی است که می‌توانیم دربارهٔ توابعِ مطلوبیت بگوییم؟ به‌طورِ سخت‌گیرانه، بله: عامل می‌تواند هر ترجیحی که دوست دارد داشته باشد. برای نمونه، عامل ممکن است ترجیح دهد تعدادِ اولِ دلار در حسابِ بانکی‌اش داشته باشد؛ در آن صورت، اگر ۱۶ دلار داشت، ۳ دلار را می‌بخشید. این ممکن است غیرعادی باشد، اما نمی‌توانیم آن را غیرعقلایی بنامیم. عامل ممکن است یک فورد پینتوی ۱۹۷۳ تصادفی و فرورفته (ضربه خورده) را به یک مرسدسِ نو و براق ترجیح دهد. عامل ممکن است فقط وقتی صاحبِ پینتوست اعدادِ اولِ دلار را ترجیح دهد، اما وقتی مرسدس را دارد، دلارِ بیشتر را به کمتر ترجیح دهد. خوشبختانه، ترجیح‌های عامل‌های واقعی معمولاً نظام‌مندترند و بنابراین راحت‌تر قابلِ مدیریت.

۱۶.۳.۱ ارزیابیِ مطلوبیت و مقیاس‌های مطلوبیت

اگر بخواهیم سامانهٔ نظریه‌تصمیمی بسازیم که به انسان در تصمیم‌گیری کمک کند یا از جانبِ او عمل کند، نخست باید دریابیم تابعِ مطلوبیتِ انسان چیست. این فرایند، که اغلب «استخراجِ ترجیح» (Preference Elicitation) نامیده می‌شود، شاملِ ارائهٔ انتخاب‌ها به انسان و استفاده از ترجیح‌های مشاهده‌شده برای تثبیتِ تابعِ مطلوبیتِ زیربنایی است.

معادلهٔ (16.2) می‌گوید هیچ مقیاسِ مطلقی برای مطلوبیت‌ها نیست، اما با این حال مفید است مقیاسی برقرار کنیم که مطلوبیت‌ها بتوانند روی آن برای هر مسئلهٔ خاص ثبت و مقایسه شوند. مقیاس را می‌توان با تثبیتِ مطلوبیتِ هر دو پیامدِ خاص برقرار کرد، همان‌گونه که مقیاسِ دما را با تثبیتِ نقطهٔ انجماد و نقطهٔ جوشِ آب تثبیت می‌کنیم.

معمولاً، مطلوبیتِ «بهترین جایزهٔ ممکن» را $U(S) = u_\top$ و «بدترین فاجعهٔ ممکن» را $U(S) = u_\bot$ تثبیت می‌کنیم. (هر دو باید متناهی باشند.) «مطلوبیت‌های نرمال‌شده» (Normalized Utilities) از مقیاسی با $u_\bot = 0$ و $u_\top = 1$ استفاده می‌کنند. با چنین مقیاسی، هوادارِ انگلیسی ممکن است مطلوبیتِ 1 را به قهرمانیِ انگلستان در جام جهانی و مطلوبیتِ 0 را به عدمِ راه‌یابیِ انگلستان تخصیص دهد.

با توجه به مقیاسِ مطلوبیت بین $u_\top$ و $u_\bot$، می‌توانیم مطلوبیتِ هر جایزهٔ خاصِ S را با پرسیدنِ انتخابِ میان S و «لاتاریِ استانداردِ» $[p, u_\top; (1-p), u_\bot]$ از عامل ارزیابی کنیم. احتمالِ p تا وقتی تنظیم می‌شود که عامل میان S و لاتاریِ استاندارد بی‌تفاوت شود. با فرضِ مطلوبیت‌های نرمال‌شده، مطلوبیتِ S با p داده می‌شود. پس از انجامِ این برای هر جایزه، مطلوبیت‌های همهٔ لاتاری‌های شاملِ آن جایزه‌ها تعیین می‌شوند. فرض کنیم، برای نمونه، می‌خواهیم بدانیم هوادارِ انگلیسیِ ما رسیدنِ انگلستان به نیمه‌نهایی و سپس باخت را چقدر ارزش‌گذاری می‌کند. آن پیامد را با لاتاریِ استانداردی مقایسه می‌کنیم که با احتمالِ p جام را می‌برد و با احتمالِ $1-p$ به‌شکلی رسواکننده از راه‌یابی بازمی‌ماند. اگر بی‌تفاوتی در $p=0.3$ رخ دهد، آنگاه 0.3 مقدارِ رسیدن به نیمه‌نهایی و سپس باخت است.

در مسائلِ تصمیمِ پزشکی، حمل‌ونقل، زیست‌محیطی و دیگر مسائل، جانِ انسان‌ها در میان است. (بله، چیزهایی مهم‌تر از سرنوشتِ انگلستان در جام جهانی هست.) در چنین مواردی، $u_\bot$ مقداری است که به مرگِ فوری تخصیص داده می‌شود (یا در بدترین حالت‌های واقعی، مرگ‌های بسیار). اگرچه هیچ‌کس با گذاشتنِ ارزشی روی زندگیِ انسان راحت نیست، این واقعیت است که معاوضه‌ها در امورِ مرگ و زندگی همیشه انجام می‌شوند. هواپیماها در بازه‌هایی تعمیرِ اساسی می‌شوند، نه پس از هر سفر. خودروها به‌گونه‌ای ساخته می‌شوند که هزینه‌ها را با نرخ‌های بقای تصادف معاوضه می‌کنند. سطحی از آلودگیِ هوا را تحمل می‌کنیم که سالانه چهار میلیون نفر را می‌کشد. به‌صورتِ متناقض‌نما، امتناع از گذاشتنِ ارزشِ پولی بر زندگی می‌تواند بدان معنا باشد که زندگی کم‌ارزش‌گذاری می‌شود.

راس شاکتر سازمانِ دولتی‌ای را توصیف می‌کند که مطالعه‌ای دربارهٔ حذفِ پنبه‌نسوز از مدارس سفارش داد. تحلیلگرانِ تصمیمِ انجام‌دهندهٔ مطالعه، مقدارِ دلاریِ خاصی برای زندگیِ کودکِ مدرسه‌ای فرض کردند و استدلال کردند انتخابِ عقلایی تحتِ آن فرض حذفِ پنبه‌نسوز بود. سازمان، از نظرِ اخلاقی خشمگین از ایدهٔ تعیینِ ارزشِ زندگی، گزارش را یک‌جا رد کرد. سپس علیه حذفِ پنبه‌نسوز تصمیم گرفت—به‌صورتِ ضمنی ادعایِ ارزشِ کمتری برای زندگیِ کودک از آنچه تحلیلگران تخصیص داده بودند.

در حالِ حاضر چند سازمانِ دولتِ آمریکا، از جملهِ آژانسِ حفاظت از محیط زیست، سازمانِ غذا و دارو و وزارتِ حمل‌ونقل، از «ارزشِ زندگیِ آماری» (Value of a Statistical Life) برای تعیینِ هزینه‌ها و مزایای مقررات و مداخله‌ها استفاده می‌کنند. مقادیرِ نمونه‌ای در ۲۰۱۹ تقریباً ۱۰ میلیون دلارند.

برخی تلاش‌ها برای یافتنِ ارزشی که مردم روی زندگیِ خودشان می‌گذارند شده. یکی از «ارزهای» رایجِ استفاده‌شده در تحلیلِ پزشکی و ایمنی «میکرومورت» (Micromort) است، یک‌شانسِ مرگ در یک میلیون. اگر از مردم بپرسید چقدر حاضرند برای اجتناب از ریسکی بپردازند—برای نمونه، برای اجتناب از بازیِ رولتِ روسی با هفت‌تیرِ یک‌میلیون‌خشاب—با اعدادِ بسیار بزرگ پاسخ می‌دهند، شاید ده‌ها هزار دلار، اما رفتارِ واقعی‌شان ارزشِ پولیِ بسیار کمتری برای میکرومورت بازتاب می‌دهد. برای نمونه، در بریتانیا، رانندگیِ ۲۳۰ مایل با خودرو ریسکِ یک میکرومورت را در بر دارد. در طولِ عمرِ خودرو—مثلاً ۹۲,۰۰۰ مایل—آن ۴۰۰ میکرومورت است. به نظر می‌رسد مردم حاضرند حدود ۱۲,۰۰۰ دلار بیشتر برای خودروی امن‌تری بپردازند که ریسکِ مرگ را نصف می‌کند. بدین‌ترتیب، کنشِ خریدِ خودروی‌شان می‌گوید برای هر میکرومورت ارزشِ ۶۰ دلار قائل‌اند. شماری از مطالعات رقمی در این محدوده را در میانِ افراد و انواعِ ریسکِ بسیار تأیید کرده‌اند. با این حال، سازمان‌های دولتی مانندِ وزارتِ حمل‌ونقلِ آمریکا معمولاً رقمِ پایین‌تری تعیین می‌کنند؛ آن‌ها فقط حدود ۶ دلار در تعمیراتِ جاده‌ای به‌ازای هر زندگیِ موردانتظارِ نجات‌یافته هزینه می‌کنند. البته، این محاسبه‌ها فقط برای ریسک‌های کوچک برقرارند. بیشترِ مردم حاضر نمی‌شوند خودکشی کنند، حتی برای ۶۰ میلیون دلار.

معیارِ دیگر «QALY» است، یا «سالِ زندگیِ تعدیل‌شده با کیفیت» (Quality-Adjusted Life Year). بیماران حاضرند طولِ عمرِ کوتاه‌تری را برای اجتناب از ناتوانی بپذیرند. برای نمونه، بیمارانِ کلیوی به‌طورِ میانگین میانِ زندگیِ دو ساله با دیالیز و یک سالِ با سلامتِ کامل بی‌تفاوت‌اند.

۱۶.۳.۲ مطلوبیتِ پول

نظریهٔ مطلوبیت ریشه در اقتصاد دارد، و اقتصاد یک نامزدِ بدیهی برای معیارِ مطلوبیت فراهم می‌کند: پول (یا دقیق‌تر، کلِ دارایی‌های خالصِ عامل). قابلیتِ مبادلهٔ تقریباً همگانیِ پول با همهٔ انواعِ کالاها و خدمات پیشنهاد می‌کند پول نقشِ قابلِ‌توجهی در توابعِ مطلوبیتِ انسانی بازی می‌کند.

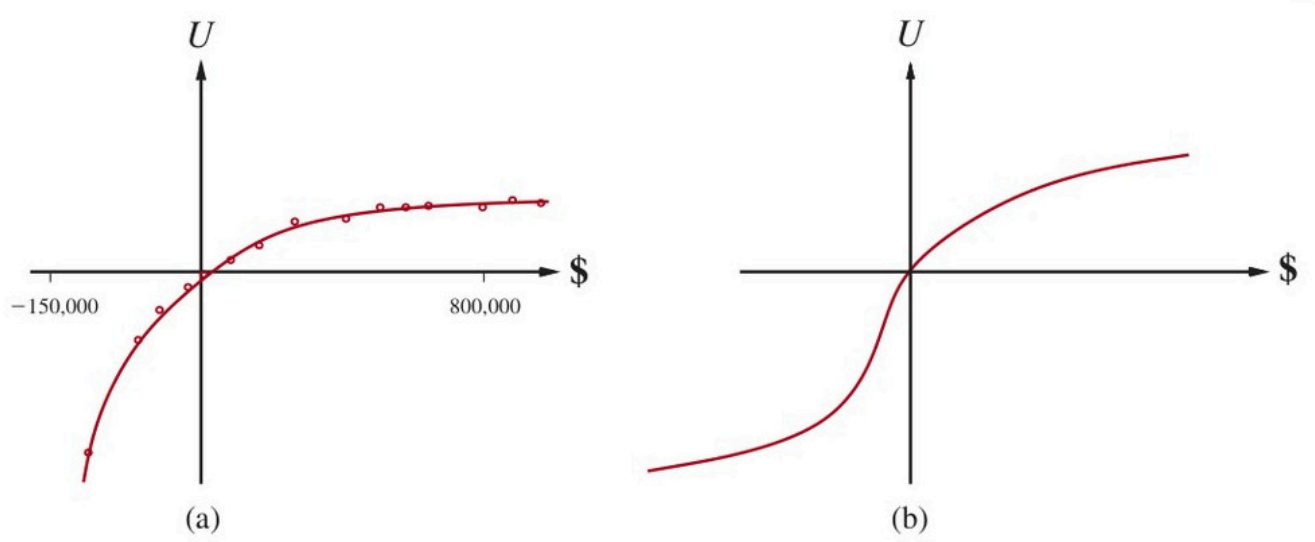
معمولاً چنین خواهد بود که عامل پولِ بیشتر را به کمتر ترجیح می‌دهد، با ثابت‌بودنِ سایرِ چیزها. می‌گوییم عامل «ترجیحِ یکنواخت» (Monotonic Preference) برای پولِ بیشتر نشان می‌دهد. این بدان معنا نیست پول مانندِ تابعِ مطلوبیت رفتار می‌کند، زیرا هیچ‌چیز دربارهٔ ترجیح‌های میانِ لاتاری‌های شاملِ پول نمی‌گوید.

فرض کنید در یک مسابقهٔ تلویزیونی بر رقبای دیگر پیروز شده‌اید. مجری حالا انتخابی به شما پیشنهاد می‌کند: یا می‌توانید جایزهٔ ۱,۰۰۰,۰۰۰ دلاری را بردارید یا آن را روی پرتابِ سکه قمار کنید. اگر سکه رو بیاید، با هیچ‌چیز تمام می‌کنید، اما اگر پشت بیاید، ۲,۵۰۰,۰۰۰ دلار می‌گیرید. اگر مثلِ بیشترِ مردم باشید، از قمار صرف‌نظر می‌کنید و میلیون را در جیب می‌گذارید. آیا غیرعقلایی رفتار می‌کنید؟

با فرضِ منصفانه‌بودنِ سکه، «ارزشِ پولیِ موردانتظار» (Expected Monetary Value یا EMV) قمار برابر (1/2)(0$) + (1/2)(2,500,000$) = 1,250,000$ است، که بیشتر از ۱,۰۰۰,۰۰۰ دلارِ اصلی است. اما این لزوماً بدان معنا نیست پذیرفتنِ قمار تصمیمِ بهتری است. فرض کنیم Sn حالتِ داشتنِ ثروتِ کلِ n دلار را نشان دهد و ثروتِ کنونی‌تان k دلار باشد. سپس مطلوبیت‌های موردانتظارِ دو کنشِ پذیرفتن و نپذیرفتنِ قمار عبارت‌اند از:

```
EU(Accept) = (1/2)U(Sk) + (1/2)U(Sk+2,500,000)
EU(Decline) = U(Sk+1,000,000)
```

برای تعیینِ اینکه چه کنیم، باید به حالت‌های پیامد مطلوبیت تخصیص دهیم. مطلوبیت با ارزشِ پولی مستقیماً متناسب نیست، زیرا مطلوبیتِ اولین میلیونِ شما بسیار بالاست (یا چنین می‌گویند)، در حالی که مطلوبیتِ میلیونِ اضافی کوچک‌تر است. فرض کنیم به وضعیتِ مالیِ کنونی‌تان (Sk) مطلوبیتِ 5، به حالتِ Sk+2,500,000 مطلوبیتِ 9 و به حالتِ Sk+1,000,000 مطلوبیتِ 8 تخصیص می‌دهید. سپس کنشِ عقلایی نپذیرفتن خواهد بود، زیرا مطلوبیتِ موردانتظارِ پذیرفتن فقط 7 است (کمتر از 8 برای نپذیرفتن). از سوی دیگر، میلیاردری به‌احتمالِ زیاد تابعِ مطلوبیتی دارد که در بازهٔ چند میلیونِ بیشتر به‌صورتِ محلی خطی است، و بنابراین قمار را می‌پذیرد.



شکل ۱۶.۲ — مطلوبیتِ پول. (الف) دادهٔ تجربی برای آقای بیرد در بازه‌ای محدود. (ب) منحنیِ نمونه‌ای برای کلِ بازه.

در مطالعه‌ای پیشگامانه از توابعِ مطلوبیتِ واقعی، گریسون (1960) دریافت مطلوبیتِ پول تقریباً دقیقاً با لگاریتمِ مقدار متناسب است. (این ایده نخستین بار توسط برنولی (1738) پیشنهاد شد؛ تمرین STPT.۱۶ را ببینید.) یکی از منحنی‌های مطلوبیتِ خاص، برای آقای بیردی معین، در شکل ۱۶.۲(الف) نشان داده شده. داده‌های به‌دست‌آمده برای ترجیح‌های آقای بیرد با تابعِ مطلوبیتِ U(Sk+n) = -263.31 + 22.09·log(n + 150,000) برای بازهٔ بین n = - 150,000$ و n = $800,000 سازگارند.

نباید فرض کنیم این تابعِ مطلوبیتِ قطعیِ ارزشِ پولی است، اما محتمل است بیشترِ مردم تابعِ مطلوبیتی داشته باشند که برای ثروتِ مثبت «مقعر» (Concave) است. به بدهی رفتن بد است، اما ترجیح‌های میانِ سطوحِ مختلفِ بدهی می‌توانند وارونگیِ تقعرِ مرتبط با ثروتِ مثبت را نشان دهند. برای نمونه، کسی که قبلاً ۱۰,۰۰۰,۰۰۰ دلار بدهکار است

ممکن است خوب قمارِ روی سکهٔ منصفانه‌ای را بپذیرد که برای رو ۱۰,۰۰۰,۰۰۰ دلار سود و برای پشت ۲۰,۰۰۰,۰۰۰ دلار زیان دارد.[۴] این منحنیِ S-شکلِ نشان‌داده‌شده در شکل ۱۶.۲(ب) را می‌دهد.

اگر توجه‌مان را به بخشِ مثبتِ منحنی‌ها محدود کنیم، جایی که شیب کاهشی است، آنگاه برای هر لاتاریِ L، مطلوبیتِ رویارویی با آن لاتاری کمتر از مطلوبیتِ دریافتِ ارزشِ پولیِ موردانتظارِ لاتاری به‌عنوانِ امری قطعی است:

```
U(L) < U(S_EMV(L))
```

یعنی عامل‌های با منحنی‌های این شکل «ریسک‌گریز» (Risk-Averse) هستند: آن‌ها امرِ قطعی با بازدهی کمتر از ارزشِ پولیِ موردانتظارِ قمار را ترجیح می‌دهند. از سوی دیگر، در ناحیهٔ «مأیوسانه» در ثروتِ منفیِ بزرگ در شکل ۱۶.۲(ب)، رفتار «ریسک‌پذیر» (Risk-Seeking) است.

مقداری که عامل در ازای لاتاری می‌پذیرد «معادلِ قطعیت» (Certainty Equivalent) لاتاری نامیده می‌شود. مطالعات نشان داده‌اند بیشترِ مردم حدود ۴۰۰ دلار در ازای قماری می‌پذیرند که نیمی از اوقات ۱۰۰۰ دلار و نیمی دیگر ۰ می‌دهد—یعنی معادلِ قطعیتِ لاتاری ۴۰۰ دلار است، در حالی که EMV آن ۵۰۰ دلار است.

توجه کنید برای تغییرهای کوچکِ ثروت نسبت به ثروتِ کنونی، تقریباً هر منحنی به‌صورتِ تقریبی خطی خواهد بود. عاملِ با منحنیِ خطی «ریسک‌خنثی» (Risk-Neutral) نامیده می‌شود. برای قمارهای با مبالغِ کوچک، پس، انتظارِ ریسک‌خنثی داریم. از جهتی، این رویهٔ ساده‌شده‌ای را توجیه می‌کند که قمارهای کوچک برای ارزیابیِ احتمال‌ها و توجیهِ اصل‌های احتمال در بخش ۱۲.۲.۳ پیشنهاد می‌کرد.

۱۶.۳.۳ مطلوبیتِ موردانتظار و ناامیدیِ پس از تصمیم

راهِ عقلاییِ انتخابِ بهترین کنش، a*، بیشینه‌کردنِ مطلوبیتِ موردانتظار است:

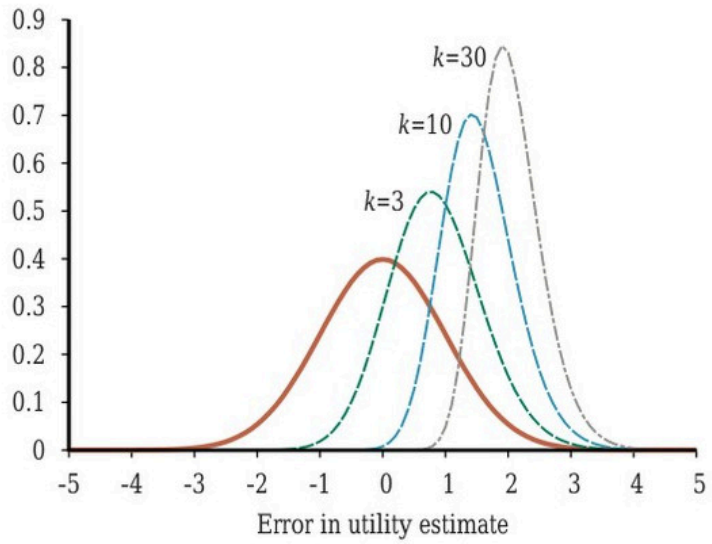
```
a* = argmax_a EU(a)
```

اگر مطلوبیتِ موردانتظار را طبقِ مدلِ احتمال‌مان به‌درستی محاسبه کرده باشیم، و اگر مدلِ احتمال به‌درستی فرایندهای تصادفیِ زیربناییِ تولیدکنندهٔ پیامدها را بازتاب دهد، آنگاه، به‌طورِ میانگین، مطلوبیتی را که انتظار داریم می‌گیریم، اگر کلِ فرایند بارها و بارها تکرار شود. در واقعیت، با این حال، مدلِ ما معمولاً وضعیتِ واقعی را ساده‌سازیِ بیش‌ازحد می‌کند، یا به این دلیل که به‌اندازهٔ کافی نمی‌دانیم (مثلاً هنگام تصمیمِ سرمایه‌گذاریِ پیچیده) یا به این دلیل که محاسبهٔ مطلوبیتِ موردانتظارِ واقعی بسیار دشوار است (مثلاً هنگام حرکت در تخته‌نرد، با نیاز به درنظرگرفتنِ همهٔ پرتاب‌های تاسِ ممکنِ آینده). در آن صورت، در واقع با برآوردهایی EÛ(a) از مطلوبیتِ موردانتظارِ واقعی کار می‌کنیم. فرض می‌کنیم، شاید از سرِ خوش‌بینی، برآوردها «بدونِ اریب» (Unbiased) هستند—یعنی مقدارِ موردانتظارِ خطا، E(EÛ(a) - EU(a))، صفر است. در آن صورت، همچنان معقول به نظر می‌رسد کنشی را با بالاترین مطلوبیتِ برآوردشده برگزینیم و انتظارِ دریافتِ آن مطلوبیت را، به‌طورِ میانگین، هنگام اجرایِ کنش داشته باشیم.

متأسفانه، پیامدِ واقعی معمولاً به‌طورِ قابلِ‌توجهی بدتر از برآوردِ ما خواهد بود، حتی اگرچه برآورد بدونِ اریب بود! برای دیدنِ چرایی، مسئلهٔ تصمیمی را در نظر بگیرید که در آن k انتخاب هست و هر یک مطلوبیتِ واقعیِ برآوردشدهٔ 0 دارد. فرض کنیم خطای هر برآوردِ مطلوبیت مستقل است و توزیعِ نرمالِ واحد دارد—یعنی گاوسی با میانگینِ صفر و انحرافِ معیارِ 1، که به‌صورتِ منحنیِ پررنگ در شکل ۱۶.۳ نشان داده شده. حالا، وقتی واقعاً شروع به تولیدِ برآوردها می‌کنیم، برخی خطاها منفی (بدبینانه) و برخی مثبت (خوش‌بینانه) خواهند بود. چون کنشی را با بالاترین برآوردِ مطلوبیت انتخاب می‌کنیم، به برآوردهای بیش‌ازحدِ خوش‌بینانه رجحان می‌دهیم، و این منشأِ اریب است.

محاسبهٔ توزیعِ بیشینهٔ k برآورد و در نتیجه کمّی‌کردنِ میزانِ ناامیدی‌مان کارِ سرراستی است. (این محاسبه حالتِ خاصی از محاسبهٔ «آمارهٔ ترتیبی» (Order Statistic) است، توزیعِ هر عنصرِ رتبه‌بندی‌شدهٔ خاصِ نمونه.) فرض کنیم هر برآوردِ Xi تابعِ چگالیِ احتمالِ f(x) و توزیعِ تجمعیِ F(x) دارد. (چنان‌که در پیوستِ الف توضیح داده شده، توزیعِ تجمعیِ F احتمالِ اینکه متغیر کمتر یا مساوی هر مقدارِ معین باشد را اندازه می‌گیرد—یعنی چگالیِ اصلیِ f را انتگرال می‌گیرد.) حالا X* بزرگ‌ترین برآورد باشد، یعنی max{X1,...,Xk}. سپس توزیعِ تجمعی برای X* چنین است:

```
P(max{X1,...,Xk} ≤ x) = P(X1 ≤ x, ..., Xk ≤ x)
P(X1 ≤ x) ... P(Xk ≤ x) =
                    F(x)^k =
```



شکل ۱۶.۳ – خوش‌بینیِ بی‌توجیهِ ناشی از انتخابِ بهترین از میانِ k گزینه: فرض می‌کنیم هر گزینه مطلوبیتِ واقعیِ 0 دارد اما برآوردِ مطلوبیتش طبقِ نرمالِ واحد توزیع شده (منحنیِ قهوه‌ای). منحنی‌های دیگر توزیع‌های بیشینهٔ k برآورد را برای k=3، 10 و 30 نشان می‌دهند.

تابعِ چگالیِ احتمال مشتقِ تابعِ توزیعِ تجمعی است، پس چگالی برای X*، بیشینهٔ k برآورد، چنین است:

```
P(x) = (d/dx)(F(x)^k) = k·f(x)·(F(x))^(k–1)
```

این چگالی‌ها برای مقادیرِ مختلفِ k در شکل ۱۶.۳ برای حالتی که f(x) نرمالِ واحد است نشان داده شده‌اند. برای k=3، چگالیِ X* میانگینی حدودِ 0.85 دارد، پس میانگینِ ناامیدی حدودِ ۸۵٪ انحرافِ معیارِ برآوردهای مطلوبیت خواهد بود. با انتخاب‌های بیشتر، برآوردهای فوق‌العاده خوش‌بینانه محتمل‌ترند: برای k=30، ناامیدی حدودِ دو برابرِ انحرافِ معیارِ برآوردها خواهد بود.

این گرایشِ بیش‌ازحدِ بالابودنِ مطلوبیتِ موردانتظارِ برآوردشدهٔ بهترین انتخاب، «نفرینِ بهینه‌ساز» (Optimizer's Curse) نامیده می‌شود (اسمیت و وینکلر، 2006). حتی باتجربه‌ترین تحلیلگرانِ تصمیم و آماردانان را گرفتار می‌کند. تظاهراتِ جدی شاملِ باور به اینکه داروی مهیجِ تازه‌ای که ۸۰٪ بیماران را در کارآزمایی درمان کرده ۸۰٪ بیماران را درمان خواهد کرد (از میانِ k = هزاران داروی نامزد انتخاب شده)، یا اینکه صندوقِ سرمایه‌گذاریِ مشترکی که به‌عنوانِ دارای بازدهِ بالاتر از میانگین تبلیغ شده همچنان آن را خواهد داشت (از میانِ k = ده‌ها صندوق در سبدِ کلیِ شرکت برای ظاهرشدن در تبلیغ انتخاب شده). حتی می‌تواند چنین باشد که آنچه بهترین انتخاب به نظر می‌رسد نباشد، اگر واریانسِ برآوردِ مطلوبیت بالا باشد: دارویی که ۹ از ۱۰ بیمار را درمان کرده و از میانِ هزاران امتحان‌شده انتخاب شده، احتمالاً بدتر از دارویی است که ۸۰۰ از ۱۰۰۰ را درمان کرده. نفرینِ بهینه‌ساز همه‌جا سر برمی‌آورد، به دلیلِ فراگیریِ فرایندهای انتخابِ بیشینه‌کنندهٔ مطلوبیت، پس گرفتنِ برآوردهای مطلوبیت به‌صورتِ ظاهری ایدهٔ بدی است. می‌توانیم با رویکردِ بیزی که از مدلِ احتمالِ صریحِ P(ÊU|EU) برای خطای برآوردهای مطلوبیت استفاده می‌کند از نفرین اجتناب کنیم. با توجه به این مدل و پیشینی دربارهٔ آنچه به‌صورتِ معقولی می‌توانیم انتظارِ مطلوبیت‌ها را داشته باشیم، برآوردِ مطلوبیت را شاهد می‌گیریم و توزیعِ پسینیِ مطلوبیتِ واقعی را با قاعدهٔ بیز محاسبه می‌کنیم.

۱۶.۳.۴ قضاوتِ انسانی و غیرعقلانیت

نظریهٔ تصمیم «نظریهٔ هنجاری» (Normative Theory) است: توصیف می‌کند عاملِ عقلایی چگونه باید عمل کند. «نظریهٔ توصیفی» (Descriptive Theory)، از سوی دیگر، توصیف می‌کند عامل‌های واقعی—برای نمونه، انسان‌ها—واقعاً چگونه عمل می‌کنند. کاربردِ نظریهٔ اقتصادی اگر این دو هم‌زمان می‌شدند بسیار تقویت می‌شد، اما به نظر می‌رسد شواهدِ تجربی برخلافِ آن است. شواهد پیشنهاد می‌کنند انسان‌ها «به‌صورتِ قابلِ‌پیش‌بینی غیرعقلایی» هستند (آریلی، 2009).

شناخته‌شده‌ترین مسئله «پارادوکسِ آله» (Allais Paradox) است (آله، 1953). به مردم میانِ لاتاری‌های A و B و سپس میانِ C و D انتخاب داده می‌شود که جایزه‌های زیر را دارند:

```
A : 80% شانسِ $4000          C : 20% شانسِ $4000
B : 100% شانسِ $3000         D : 25% شانسِ $3000
```

بیشترِ مردم به‌طورِ پیوسته B را بر A (گرفتنِ امرِ قطعی) و C را بر D (گرفتنِ EMV بالاتر) ترجیح می‌دهند. تحلیلِ هنجاری مخالف است! این را به‌آسانی می‌بینیم اگر از آزادیِ دلالت‌شده توسط معادلهٔ (16.2) برای گذاشتنِ U($0) = 0 استفاده کنیم. در آن صورت، B ≻ A دلالت دارد U($3000) > 0.8·U($4000)، در حالی که C ≻ D دقیقاً عکسِ آن دلالت دارد. به بیانِ دیگر، هیچ تابعِ مطلوبیتی نیست که با این انتخاب‌ها سازگار باشد.

یک تبیین برای ترجیح‌های ظاهراً غیرعقلایی «اثرِ قطعیت» (Certainty Effect) است (کانمن و تورسکی، 1979): مردم به‌شدت جذبِ سودهای قطعی می‌شوند. دلایلِ چند برای این می‌تواند باشد. نخست، مردم ممکن است ترجیح دهند بارِ محاسباتی‌شان را کاهش دهند؛ با انتخابِ پیامدهای قطعی، مجبور نیستند با احتمال‌ها محاسبه کنند. اما اثر حتی وقتی محاسباتِ درگیر بسیار آسان‌اند نیز باقی می‌ماند. دوم، مردم ممکن است به مشروعیتِ احتمال‌های بیان‌شده اعتماد نکنند. اعتماد دارم پرتابِ سکه تقریباً ۵۰/۵۰ است اگر بر سکه و پرتابش کنترل داشته باشم، اما ممکن است به نتیجه اعتماد نکنم اگر پرتاب توسط کسی با منافعِ شخصی در پیامد انجام شود.[5] در حضورِ بی‌اعتمادی، شاید بهتر باشد سراغِ امرِ قطعی رفت.[6] سوم، مردم ممکن است وضعیتِ عاطفی‌شان را نیز به‌حساب بیاورند، نه فقط وضعیتِ مالی‌شان را. مردم می‌دانند اگر پاداشِ قطعی (B) را برای شانسِ ۸۰٪ پاداشِ بالاتر رها می‌کردند و سپس می‌باختند، پشیمان می‌شدند. به بیانِ دیگر، اگر A انتخاب شود، ۲۰٪ شانسِ بی‌پولی و احساسِ ابلهِ کامل‌بودن هست، که بدتر از صرفاً بی‌پولی است. پس شاید افرادی که B را بر A و C را بر D انتخاب می‌کنند غیرعقلایی نیستند؛ آن‌ها حاضرند ۲۰۰ دلارِ EMV را برای اجتناب از شانسِ ۲۰٪ احساسِ ابله‌بودن ببخشند.

مسئلهٔ مرتبطی «پارادوکسِ اِلزبرگ» (Ellsberg Paradox) است.[10] در این‌جا جایزه‌ها ثابت‌اند، اما احتمال‌ها کم‌قیدند. پرداختِ شما به رنگِ گویِ انتخاب‌شده از کوزه بستگی دارد. به شما گفته می‌شود کوزه شاملِ 1/3 گویِ قرمز و 2/3 یا سیاه یا زرد است، اما نمی‌دانید چند سیاه و چند زرد. باز، از شما پرسیده می‌شود آیا لاتاریِ A یا B را ترجیح می‌دهید؛ و سپس C یا D:

```
$100 : A برای گوی قرمز          $100 : C برای گوی قرمز یا زرد
$100 : B برای گوی سیاه          $100 : D برای گوی سیاه یا زرد
```

باید روشن باشد اگر فکر کنید قرمزها از سیاه‌ها بیشترند باید A را بر B و C را بر D ترجیح دهید؛ اگر فکر کنید قرمزها از سیاه‌ها کمترند باید عکسِ آن را ترجیح دهید. اما معلوم می‌شود بیشترِ مردم A را بر B و نیز D را بر C ترجیح می‌دهند، حتی اگرچه هیچ حالتِ جهانی نیست که این برایش عقلایی باشد. به نظر می‌رسد مردم «بیزاری از ابهام» (Ambiguity Aversion) دارند: A به شما شانسِ 1/3 بردن می‌دهد، در حالی که B می‌تواند هر جای بینِ 0 و 2/3 باشد. به‌طورِ مشابه، D به شما شانسِ 2/3 می‌دهد، در حالی که C می‌تواند هر جای بینِ 1/3 و 3/3 باشد. بیشترِ مردم احتمالِ شناخته‌شده را به ناشناخته‌های ناشناخته ترجیح می‌دهند.

مسئلهٔ دیگر این است که عبارت‌بندیِ دقیقِ مسئلهٔ تصمیم می‌تواند تأثیرِ بزرگی بر انتخاب‌های عامل داشته باشد؛ این «اثرِ قاب‌بندی» (Framing Effect) نامیده می‌شود. آزمایش‌ها نشان می‌دهند مردم رویهٔ پزشکیِ توصیف‌شده با «نرخِ بقای ۹۰٪» را حدود دو برابرِ توصیف‌شده با «نرخِ مرگِ ۱۰٪» دوست دارند، حتی اگرچه این دو گزاره دقیقاً یک معنا دارند. این ناسازگاریِ قضاوتی در آزمایش‌های متعدد یافت شده و تقریباً یکسان است، خواه آزمودنی‌ها بیمارانِ درمانگاه باشند، خواه دانشجویانِ کسب‌وکارِ آماری‌ماهر، خواه پزشکانِ باتجربه.

مردم در قضاوتِ مطلوبیتِ نسبی راحت‌ترند تا مطلوبیتِ مطلق. ممکن است تصورِ اندکی داشته باشم که چقدر از شراب‌های گوناگونِ ارائه‌شده توسط رستوران لذت می‌برم. رستوران از این بهره می‌گیرد با ارائهٔ بطریِ ۲۰۰ دلاری که هیچ‌کس نمی‌خرد، اما به‌عنوانِ لنگر عمل می‌کند و برآوردِ مشتری از ارزشِ همهٔ شراب‌ها را به بالا منحرف می‌کند، و

بطریِ ۵۵ دلاری را معامله به نظر می‌رساند. این «اثرِ لنگراندازی» (Anchoring Effect) نامیده می‌شود.

اگر مطلع‌کنندگانِ انسانی بر قضاوت‌های ترجیحیِ متناقض اصرار ورزند، هیچ‌کاری نیست که عامل‌های خودکار بتوانند برای سازگاری با آن‌ها انجام دهند. خوشبختانه، قضاوت‌های ترجیحیِ انجام‌شده توسط انسان‌ها اغلب در پرتوِ ملاحظهٔ بیشتر قابلِ تجدیدنظرند. پارادوکس‌هایی مانندِ پارادوکس‌های آله و اِلزبرگ اگر انتخاب‌ها بهتر توضیح داده شوند بسیار کاهش می‌یابند (اما حذف نمی‌شوند). در کارِ مدرسهٔ کسب‌وکارِ هاروارد دربارهٔ ارزیابیِ مطلوبیتِ پول، کینی و رایفا (1976، ص ۲۱۰) دریافتند:

«آزمودنی‌ها در کوچک تمایل به ریسک‌گریزیِ بیش‌ازحد دارند و بنابراین ... توابعِ مطلوبیتِ برازش‌شده حق‌بیمه‌های ریسکِ غیرقابلِ‌قبولِ بزرگی برای لاتاری‌های با گسترهٔ زیاد نشان می‌دهند. ... با این حال، بیشترِ آزمودنی‌ها می‌توانند ناسازگاری‌هایشان را آشتی دهند و احساس می‌کنند درسِ مهمی دربارهٔ چگونگیِ رفتاری که می‌خواهند داشته باشند آموخته‌اند. در نتیجه، برخی آزمودنی‌ها بیمهٔ تصادفِ خودروی‌شان را لغو می‌کنند و بیمهٔ عمرِ مدت‌دارِ بیشتری بر زندگی‌شان می‌گیرند.»

شواهدِ غیرعقلانیتِ انسانی همچنین توسط پژوهشگرانِ حوزهٔ «روان‌شناسیِ تکاملی» (Evolutionary Psychology) زیر سؤال رفته، که به این واقعیت اشاره می‌کنند سازوکارهای تصمیم‌گیریِ مغزِ ما برای حلِ مسائلِ کلامی با احتمال‌ها و جایزه‌های بیان‌شده به‌صورتِ اعدادِ اعشاری تکامل نیافته‌اند. برای بحث، بپذیریم مغز سازوکارهای عصبیِ تعبیه‌شده‌ای برای محاسبه با احتمال‌ها و مطلوبیت‌ها دارد، یا چیزی از نظرِ عملکردی معادل. اگر چنین باشد، ورودی‌های لازم از طریقِ تجربهٔ انباشتهٔ پیامدها و پاداش‌ها به دست می‌آمدند، نه از طریقِ ارائه‌های زبانیِ مقادیرِ عددی.

به هیچ وجه بدیهی نیست که بتوانیم با ارائهٔ مسائلِ تصمیم در قالبِ زبانی/عددی مستقیماً به سازوکارهای عصبیِ تعبیه‌شدهٔ مغز دسترسی پیدا کنیم. خودِ این واقعیت که عبارت‌بندی‌های متفاوتِ همان مسئلهٔ تصمیم انتخاب‌های متفاوتی برمی‌انگیزند پیشنهاد می‌کند خودِ مسئلهٔ تصمیم به‌درستی منتقل نمی‌شود. با انگیزهٔ این مشاهده، روان‌شناسان ارائهٔ مسائل در استدلالِ نامطمئن و تصمیم‌گیری را در شکل‌های «تکاملاً مناسب» امتحان کرده‌اند؛ برای نمونه، به‌جای گفتنِ «نرخِ بقای ۹۰٪»، آزمایشگر ممکن است ۱۰۰ پویانماییِ خطی از عمل جراحی نشان دهد، که بیمار در ۱۰ تایشان می‌میرد و در ۹۰ تایشان زنده می‌ماند. با طرحِ مسائلِ تصمیم بدین‌گونه، به نظر می‌رسد رفتارِ مردم بسیار نزدیک‌تر به معیارِ عقلانیت است.

۱۶.۴ توابعِ مطلوبیتِ چندویژگی

تصمیم‌گیری در حوزهٔ سیاستِ عمومی سرمایهٔ بزرگی در میان است، هم در پول و هم در جان. برای نمونه، در تصمیم‌گیری دربارهٔ اینکه چه سطحی از انتشاراتِ مضر از نیروگاه اجازه داده شود، سیاست‌گذاران باید پیشگیریِ مرگ و ناتوانی را با منفعتِ برق و بارِ اقتصادیِ کاهشِ انتشارات بسنجند. برگزیدنِ مکانی برای فرودگاهِ تازه مستلزم درنظرگرفتنِ اختلالِ ناشی از ساخت‌وساز؛ هزینهٔ زمین؛ فاصله از مراکزِ جمعیت؛ نویزِ عملیاتِ پرواز؛ مسائلِ ایمنیِ ناشی از توپوگرافیِ

محلی و شرایطِ آب‌وهوا؛ و مانندِ آن است. مسائلی مانندِ این‌ها، که در آن‌ها پیامدها با دو یا چند ویژگی مشخص می‌شوند، توسط «نظریهٔ مطلوبیتِ چندویژگی» (Multiattribute Utility Theory) مدیریت می‌شوند. در اصل، نظریهٔ مقایسهٔ سیب و پرتقال است.

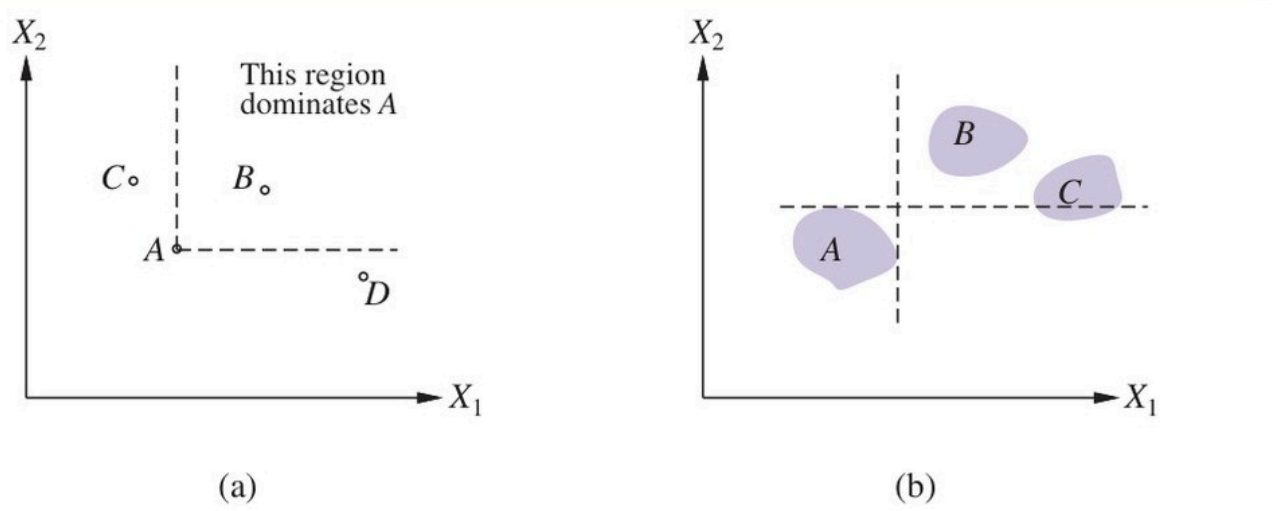
فرض کنیم ویژگی‌ها $X = X1,...,Xn$ باشند و $x = (x1,...,xn)$ بردارِ کاملی از تخصیص‌ها باشد، که در آن هر xi یا مقدارِ عددی است یا مقدارِ گسسته. با درنظرگرفتنِ یک ویژگی، ممکن است ترجیحِ یکنواختی داشته باشیم—برای نمونه، در انتخابِ محلِ فرودگاه، احتمالاً ترجیح می‌دهیم تعدادِ پرواز در روز بیشتر باشد (تا زمانی که نویز در نظر گرفته نشود). برای ویژگیِ «دما» در اتاق، ممکن است ترجیحِ یکنواخت نداشته باشیم: ۷۰ درجهٔ فارنهایت بهتر از ۹۰ است و بهتر از ۵۰، اما ۷۲ درجه به‌اندازهٔ ۷۰ درجه خوب است. برای مدل‌کردنِ این، می‌توانیم از تابعِ کمیتِ دیگری استفاده کنیم—مثلاً t-70 برای اندازه‌گیریِ اینکه اتاق چقدر گرم است و t-70 برای اندازه‌گیریِ اینکه چقدر خنک است؛ هر دو این ویژگی‌ها تا رسیدن به حداکثرِ مقدارِ مطلوبیت‌شان در 0 به‌صورتِ یکنواخت صعودی خواهند بود؛ منحنیِ مطلوبیت از آن نقطه به بعد تخت است، بدان معنا که بالای ۷۰ درجهٔ فارنهایت «گرم‌تر از حدِ کافی» به دست نمی‌آورید و زیرِ ۷۰ درجهٔ فارنهایت «خنک‌تر از حدِ کافی». ویژگی‌های مسئلهٔ فرودگاه می‌توانند شاملِ موارد زیر باشند:

- **ظرفیت (Throughput)**، اندازه‌گیری‌شده با شمارِ پروازها در روز؛
- **ایمنی (Safety)**، اندازه‌گیری‌شده با منفیِ شمارِ موردانتظارِ مرگ‌ها در سال؛
- **آرامش (Quietness)**، اندازه‌گیری‌شده با منفیِ شمارِ افرادِ ساکن زیرِ مسیرهای پرواز؛
- **صرفه‌جویی (Frugality)**، اندازه‌گیری‌شده با هزینهٔ منفیِ ساخت‌وساز.

با بررسیِ مواردی شروع می‌کنیم که در آن‌ها تصمیم‌ها را می‌توان بدونِ ترکیبِ مقادیرِ ویژگی در مقدارِ مطلوبیتِ تکی گرفت. سپس به مواردی نگاه می‌کنیم که مطلوبیت‌های ترکیب‌های ویژگی را می‌توان بسیار موجز مشخص کرد.

۱۶.۴.۱ سلطه (Dominance)

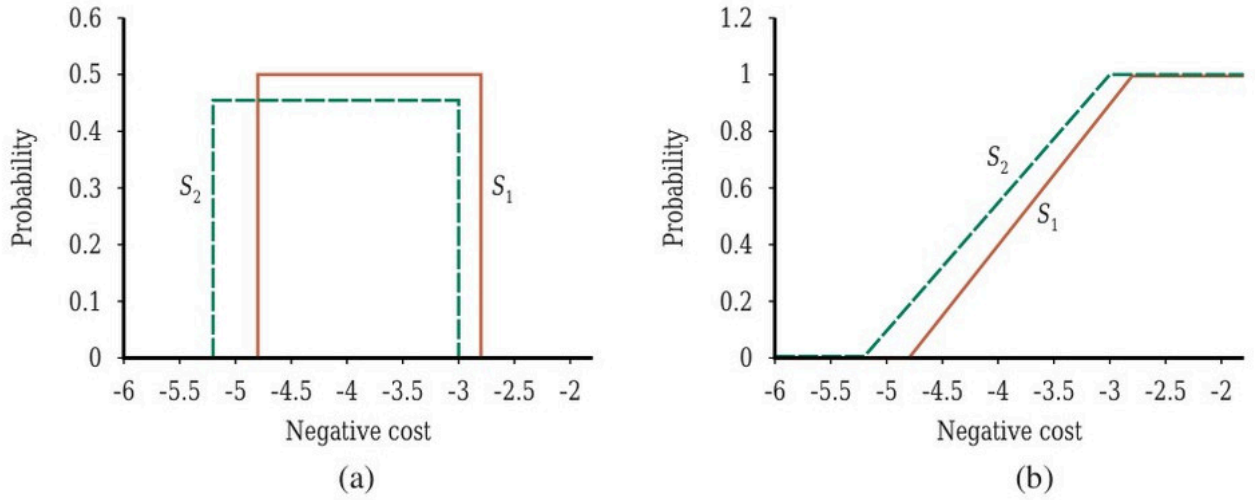
فرض کنیم مکانِ فرودگاهِ S1 هزینه‌اش کمتر است، آلودگیِ صوتیِ کمتری تولید می‌کند و از S2 ایمن‌تر است. در ردکردنِ S2 تردید نمی‌کنیم. سپس می‌گوییم «سلطهٔ اکیدِ» (Strict Dominance) S1 بر S2 هست. به‌طورِ کلی، اگر گزینه‌ای در همهٔ ویژگی‌ها از گزینهٔ دیگرِ معینی ارزشِ کمتری داشته باشد، دیگر لازم نیست در نظر گرفته شود. سلطهٔ اکید اغلب در محدودکردنِ میدانِ انتخاب‌ها به رقبای واقعی بسیار مفید است، هرچند به‌ندرت انتخابِ یکتایی می‌دهد. شکل ۱۶.۴(الف) نمودارِ شماتیکی برای حالتِ دوویژگی نشان می‌دهد.



شکل ۱۶.۴ – سلطهٔ اکید. (الف) جبرگرایانه: گزینهٔ A به‌صورتِ اکید مغلوبِ B می‌شود، اما توسط C یا D نه. (ب) نامطمئن: A به‌صورتِ اکید مغلوبِ B می‌شود، اما توسط C نه.

این برای حالتِ جبرگرایانه، که در آن مقادیرِ ویژگی با قطعیت معلوم‌اند، خوب است. حالتِ کلی که پیامدها نامطمئن‌اند چطور؟ همتایِ مستقیمی از سلطهٔ اکید می‌توان ساخت، که در آن، با وجودِ عدم قطعیت، همهٔ پیامدهای عینیِ ممکنِ S1 به‌صورتِ اکید بر همهٔ پیامدهای ممکنِ S2 غالب می‌شوند. (شکل ۱۶.۴(ب) را ببینید.) البته، این احتمالاً حتی کمتر از حالتِ جبرگرایانه رخ خواهد داد.

خوشبختانه، تعمیمِ مفیدتری به نام «سلطهٔ تصادفی» (Stochastic Dominance) هست، که در مسائلِ واقعی بسیار مکرر رخ می‌دهد. سلطهٔ تصادفی در بافتِ ویژگیِ تکی آسان‌تر فهمیده می‌شود. فرض کنیم باور داریم هزینهٔ قراردادنِ فرودگاه در S1 به‌صورتِ یکنواخت بینِ ۲.۸ میلیارد و ۴.۸ میلیارد دلار توزیع شده و هزینه در S2 به‌صورتِ یکنواخت بینِ ۳ میلیارد و ۵.۲ میلیارد دلار. ویژگیِ صرفه‌جویی را هزینهٔ منفی تعریف کنیم. شکل ۱۶.۵(الف) توزیع‌های صرفه‌جوییِ مکان‌های S1 و S2 را نشان می‌دهد. سپس، با توجه به فقط اطلاعاتِ اینکه انتخابِ صرفه‌جوتر بهتر است (با ثابت‌بودنِ سایرِ چیزها)، می‌توانیم بگوییم S1 به‌صورتِ تصادفی S2 را سلطه می‌کند (یعنی S2 را می‌توان کنار گذاشت). توجه به این نکته مهم است که این از مقایسهٔ هزینه‌های موردانتظار نتیجه نمی‌شود. برای نمونه، اگر می‌دانستیم هزینهٔ S1 دقیقاً ۳.۸ میلیارد دلار است، بدونِ اطلاعاتِ اضافی دربارهٔ مطلوبیتِ پول قادر به تصمیم‌گیری نبودیم. (ممکن است عجیب به نظر برسد که اطلاعاتِ بیشتر دربارهٔ هزینهٔ S1 عامل را کمتر قادر به تصمیم‌گیری کند. پارادوکس با یادآوریِ این حل می‌شود که در غیابِ اطلاعاتِ دقیقِ هزینه، تصمیم آسان‌تر گرفته می‌شود اما محتمل‌تر است غلط باشد.) رابطهٔ دقیقِ میانِ توزیع‌های ویژگیِ لازم برای برقراریِ سلطهٔ تصادفی بهتر است با بررسیِ توزیع‌های تجمعی، که در شکل ۱۶.۵(ب) نشان داده شده‌اند، دیده شود.



شکل ۱۶.۵ – سلطهٔ تصادفی. (الف) S1 به‌صورتِ تصادفی S2 را در صرفه‌جویی (هزینهٔ منفی) سلطه می‌کند. (ب) توزیع‌های تجمعی برای صرفه‌جوییِ S1 و S2.

اگر توزیعِ تجمعیِ S1 همیشه سمتِ راستِ توزیعِ تجمعیِ S2 باشد، آنگاه، از نظرِ تصادفی، S1 از S2 ارزان‌تر است. به‌صورتِ صوری، اگر دو کنشِ A1 و A2 به توزیع‌های احتمالِ p1(x) و p2(x) روی ویژگیِ X منتهی شوند، آنگاه A1 به‌صورتِ تصادفی A2 را در X سلطه می‌کند اگر:

$$\int_{-\infty}^{x} p1(x')\,dx' \le \int_{-\infty}^{x} p2(x')\,dx \quad \text{برای همهٔ } x$$

ارتباطِ این تعریف با انتخابِ تصمیم‌های بهینه از ویژگیِ زیر می‌آید: اگر A1 به‌صورتِ تصادفی A2 را سلطه کند، آنگاه برای هر تابعِ مطلوبیتِ یکنواختِ غیرنزولیِ U(x)، مطلوبیتِ موردانتظارِ A1 دست‌کم به‌اندازهٔ مطلوبیتِ موردانتظارِ A2 است. برای دیدنِ چراییِ درست‌بودنِ این، دو مطلوبیتِ موردانتظار، $\int p1(x)U(x)dx$ و $\int p2(x)U(x)dx$ را در نظر بگیرید. در آغاز، بدیهی نیست چرا انتگرالِ نخست از دوم بزرگ‌تر است، با توجه به اینکه شرطِ سلطهٔ تصادفی انتگرالِ p1ی دارد که از انتگرالِ p2 کوچک‌تر است. اما به‌جای فکرکردن دربارهٔ انتگرال روی x، به انتگرال روی y، احتمالِ تجمعی، فکر کنید، چنان‌که در شکل ۱۶.۵(ب) نشان داده شده. برای هر مقدارِ y، مقدارِ متناظرِ x (و از این رو U(x)) برای S1 از S2 بزرگ‌تر است؛ پس اگر کمیتِ بزرگ‌تری را روی کلِ بازهٔ y انتگرال بگیریم، قطعاً به نتیجهٔ بزرگ‌تری می‌رسیم. به‌صورتِ صوری، صرفاً جایگزینیِ y = P1(x) در انتگرالِ مقدارِ موردانتظارِ S1 و y = P2(x) در انتگرالِ S2 است. با این جایگزینی‌ها، dy = (d/dx)(P1(x))dx = p1(x)dx برای S1 و dy = p2(x)dx برای S2 داریم، از این رو:

$$\int p1(x)U(x)dx = \int U(P1^{-1}(y))dy \ge \int U(P2^{-1}(y))dy = \int p2(x)U(x)dx$$

این نامساوی به ما اجازه می‌دهد در مسئلهٔ تک‌ویژگی A1 را به A2 ترجیح دهیم. عمومی‌تر، اگر کنشی در یک مسئلهٔ چندویژگی در همهٔ ویژگی‌ها به‌صورتِ تصادفی توسط کنشِ دیگر سلطه شود، آنگاه می‌توان آن را کنار گذاشت.

شرطِ سلطهٔ تصادفی ممکن است نسبتاً فنی به نظر برسد و شاید بدونِ محاسباتِ احتمالِ گسترده به‌آسانی ارزیابی‌شدنی نباشد. در واقع، در بسیاری از موارد می‌توان آن را بسیار آسان تصمیم گرفت. برای نمونه، ترجیح می‌دهید با سر از روی بتون از ارتفاعِ ۳ میلی‌متری بیفتید یا ۳ متری؟ با فرضِ انتخابِ ۳ میلی‌متری—انتخابِ خوبی است! چرا لزوماً تصمیمِ بهتری است؟ عدم قطعیتِ قابلِ‌توجهی دربارهٔ درجهٔ آسیبِ واردشده در هر دو حالت هست؛ اما برای هر سطحِ معینِ آسیب، احتمالِ اینکه دستِ‌کم آن سطحِ آسیب را متحمل شوید هنگامِ افتادن از ۳ متر بیشتر از ۳ میلی‌متر است. به بیانِ دیگر، ۳ میلی‌متر به‌صورتِ تصادفی ۳ متر را در ویژگیِ ایمنی سلطه می‌کند.

این نوع استدلال برای انسان‌ها طبیعتِ دوم است؛ آن‌قدر بدیهی است که حتی به آن فکر هم نمی‌کنیم. سلطهٔ تصادفی در مسئلهٔ فرودگاه نیز فراوان است. فرض کنیم، برای نمونه، هزینهٔ حمل‌ونقلِ ساخت‌وساز به فاصله تا تأمین‌کننده بستگی دارد. خودِ هزینه نامطمئن است، اما هرچه فاصله بیشتر باشد، هزینه بیشتر. اگر S1 به S2 نزدیک‌تر باشد، آنگاه S1 در صرفه‌جویی بر S2 سلطه خواهد داشت. اگرچه آن‌ها را این‌جا ارائه نمی‌کنیم، الگوریتم‌هایی برای انتشارِ این نوع اطلاعاتِ کیفی میانِ متغیرهای نامطمئن در «شبکه‌های احتمالاتیِ کیفی» (Qualitative Probabilistic Networks) وجود دارند، که سامانه‌ای را بر اساسِ سلطهٔ تصادفی، بدونِ استفاده از هیچ مقدارِ عددی، قادر به تصمیم‌گیریِ عقلایی می‌کنند.

۱۶.۴.۲ ساختارِ ترجیح و مطلوبیتِ چندویژگی

فرض کنیم n ویژگی داریم که هر یک d مقدارِ متمایزِ ممکن دارد. برای مشخص‌کردنِ تابعِ مطلوبیتِ کاملِ U(x1,...,xn)، در بدترین حالت به d^n مقدار نیاز داریم. نظریهٔ مطلوبیتِ چندویژگی هدفش شناساییِ ساختارِ اضافی در ترجیح‌های انسانی است تا لازم نباشد همهٔ d^n مقدار را جداگانه مشخص کنیم. پس از شناساییِ نظم‌مندی‌ای در رفتارِ ترجیحی، «قضیه‌های بازنمایی» (Representation Theorems) را اشتقاق می‌کنیم تا نشان دهیم عاملی با نوعِ خاصی از ساختارِ ترجیح، تابعِ مطلوبیتی به‌صورتِ U(x1,...,xn) = F[f1(x1),...,fn(xn)] دارد، که در آن F (امیدواریم) تابعِ ساده‌ای مانندِ جمع است. شباهت را با استفاده از شبکه‌های بیزی برای تجزیهٔ احتمالِ مشترکِ چند متغیرِ تصادفی توجه کنید.

به‌عنوانِ نمونه، فرض کنیم هر xi مقدارِ پولی باشد که عامل در ارزِ خاصی دارد: دلار، یورو، مارک، لیره و... توابعِ fi می‌توانستند هر مقدار را به ارزِ مشترکی تبدیل کنند و F سپس صرفاً جمع می‌بود.

ترجیح‌ها بدونِ عدم قطعیت. با حالتِ جبرگرایانه شروع کنیم. در صفحهٔ ۵۳۲ اشاره کردیم برای محیط‌های جبرگرایانه، عامل تابعِ مقداری دارد، که آن را این‌جا V(x1,...,xn) می‌نویسیم؛ هدف بازنماییِ موجزِ این تابع است. نظم‌مندیِ پایه‌ای که در ساختارهای ترجیحِ جبرگرایانه پدید می‌آید «استقلالِ ترجیحی» (Preference Independence) نامیده می‌شود. دو ویژگیِ X1 و X2 از ویژگیِ سومِ X3 به‌صورتِ ترجیحی مستقل‌اند اگر ترجیح میانِ پیامدهای ⟨x1,x2,x3⟩ و ⟨x'1,x'2,x3⟩ به مقدارِ خاصِ x3 برای ویژگیِ X3 بستگی نداشته باشد.

با بازگشت به مثالِ فرودگاه، که در آن (در میانِ ویژگی‌های دیگر) آرامش، صرفه‌جویی و ایمنی را باید در نظر گرفت، ممکن است پیشنهاد شود آرامش و صرفه‌جویی از ایمنی به‌صورتِ ترجیحی مستقل‌اند. برای نمونه، اگر پیامدی با ۲۰,۰۰۰ نفرِ ساکن در مسیرِ پرواز و هزینهٔ ساختِ ۴ میلیارد دلاری را بر پیامدی با ۷۰,۰۰۰ نفرِ ساکن در مسیرِ پرواز و هزینهٔ ۳.۷ میلیارد دلاری ترجیح دهیم، وقتی سطحِ ایمنی در هر دو حالت 0.006 مرگ به‌ازای هر میلیارد مایلِ مسافر باشد، آنگاه

همان ترجیح را خواهیم داشت وقتی سطحِ ایمنی 0.012 یا 0.003 است؛ و همان استقلال برای ترجیح‌های میانِ هر جفتِ مقدارِ دیگرِ آرامش و صرفه‌جویی برقرار خواهد بود. همچنین آشکار است صرفه‌جویی و ایمنی از آرامش به‌صورتِ ترجیحی مستقل‌اند و آرامش و ایمنی از صرفه‌جویی.

می‌گوییم مجموعهٔ ویژگی‌های {آرامش، صرفه‌جویی، ایمنی} «استقلالِ ترجیحیِ دوجانبه» (Mutual Preferential Independence یا MPI) نشان می‌دهد. MPI می‌گوید، در حالی که هر ویژگی ممکن است مهم باشد، بر شیوه‌ای که در آن ویژگی‌های دیگر را با یکدیگر معاوضه می‌کنید اثر نمی‌گذارد.

استقلالِ ترجیحیِ دوجانبه نامِ پیچیده‌ای است، اما به شکلِ ساده‌ای برای تابعِ مقدارِ عامل می‌انجامد (دبرو، 1960): اگر ویژگی‌های X1,...,Xn به‌صورتِ دوجانبه ترجیحی مستقل باشند، آنگاه ترجیح‌های عامل را می‌توان با تابعِ مقداری به‌صورتِ:

```
V(x1,...,xn) = ∑i Vi(xi)
```

بازنمایی کرد، که در آن هر Vi فقط به ویژگیِ Xi اشاره دارد. برای نمونه، به‌خوبی ممکن است تصمیمِ فرودگاه را بتوان با تابعِ مقداری به‌صورتِ V(آرامش، صرفه‌جویی، ایمنی) = آرامش × 10^4 + صرفه‌جویی + ایمنی × 10^{12} گرفت.

تابعِ مقداری از این نوع «تابعِ مقدارِ افزایشی» (Additive Value Function) نامیده می‌شود. توابعِ افزایشی راهِ فوق‌العاده طبیعی برای توصیفِ ترجیح‌های عامل‌اند و در بسیاری از موقعیت‌های دنیایِ واقعی معتبرند. برای n ویژگی، ارزیابیِ تابعِ مقدارِ افزایشی به ارزیابیِ n تابعِ مقدارِ تک‌بعدیِ جداگانه نیاز دارد، نه یک تابعِ n-بعدی؛ معمولاً، این کاهشِ نمایی در شمارِ آزمایش‌های ترجیحِ لازم را بازنمایی می‌کند. حتی وقتی MPI به‌صورتِ سخت‌گیرانه برقرار نیست، چنان‌که ممکن است در مقادیرِ حدیِ ویژگی‌ها باشد، تابعِ مقدارِ افزایشی همچنان ممکن است تقریبِ خوبی از ترجیح‌های عامل فراهم کند. این به‌ویژه وقتی صادق است که نقض‌های MPI در بخش‌هایی از بازه‌های ویژگی رخ دهند که محتمل نیست در عمل رخ دهند.

برای فهمِ بهترِ MPI، کمک می‌کند به مواردی نگاه کنیم که در آن برقرار نیست. فرض کنید در بازارِ قرون‌وسطایی هستید و خریدِ سگ‌های شکاری، چند مرغ و چند قفسِ ترکه‌ای برای مرغ‌ها را در نظر دارید. سگ‌های شکاری بسیار ارزشمندند، اما اگر قفسِ کافی برای مرغ‌ها نداشته باشید، سگ‌ها مرغ‌ها را خواهند خورد؛ از این رو، معاوضهٔ میانِ سگ‌ها و مرغ‌ها به‌شدت به شمارِ قفس‌ها بستگی دارد، و MPI نقض می‌شود. وجودِ این نوع تعامل‌ها میانِ ویژگی‌های گوناگون، ارزیابیِ تابعِ مقدارِ کلی را بسیار دشوارتر می‌کند.

ترجیح‌ها با عدم قطعیت. وقتی عدم قطعیت در حوزه هست، همچنین باید ساختارِ ترجیح‌های میانِ لاتاری‌ها را در نظر بگیریم و ویژگی‌های حاصلِ توابعِ مطلوبیت را بفهمیم، نه فقط توابعِ مقدار را. ریاضیاتِ این مسئله می‌تواند کاملاً پیچیده شود، پس فقط یکی از نتایجِ اصلی را ارائه می‌کنیم تا طعمِ آنچه می‌توان انجام داد داده شود.

مفهومِ پایهٔ «استقلالِ مطلوبیت» (Utility Independence) استقلالِ ترجیحی را برای پوششِ لاتاری‌ها بسط می‌دهد: مجموعه‌ای از ویژگی‌های X از مجموعه‌ای از ویژگی‌های Y به‌صورتِ مطلوبیتی مستقل است اگر ترجیح‌های میانِ لاتاری‌های روی ویژگی‌های در X مستقل از مقادیرِ خاصِ ویژگی‌های در Y باشند. مجموعه‌ای از ویژگی‌ها «دارای استقلالِ

مطلوبیتِ متقابل» (Mutually Utility Independent یا MUI) است اگر هر یک از زیرمجموعه‌هایش از ویژگی‌های باقی‌مانده به‌صورتِ مطلوبیتی مستقل باشد. باز هم، منطقی به نظر می‌رسد که ویژگی‌های فرودگاه دارای MUI فرض شوند.

MUI دلالت دارد رفتارِ عامل را می‌توان با «تابعِ مطلوبیتِ ضربی» (Multiplicative Utility Function) توصیف کرد (کینی، 1974). شکلِ عمومیِ تابعِ مطلوبیتِ ضربی بهتر است با نگاه به حالتِ سه‌ویژگی دیده شود. برای اختصار، از Ui برای معنیِ Ui(xi) استفاده می‌کنیم:

```
U = k1U1 + k2U2 + k3U3 + k1k2U1U2 + k2k3U2U3 + k3k1U3U1 + k1k2k3U1U2U3
```

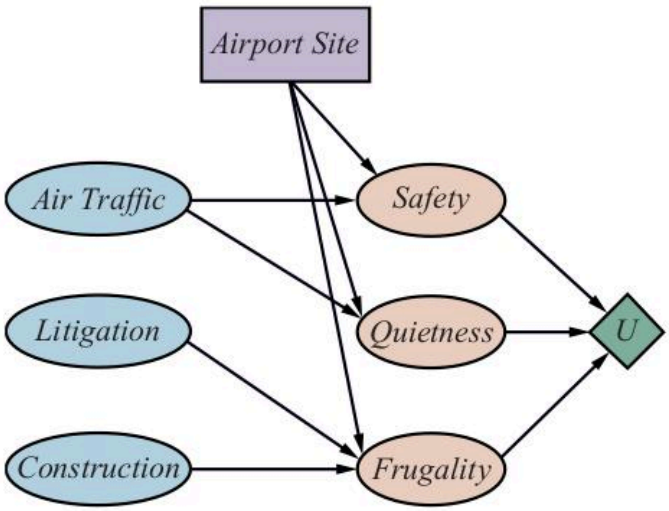
اگرچه این چندان ساده به نظر نمی‌رسد، فقط سه تابعِ مطلوبیتِ تک‌ویژگی و سه ثابت دارد. به‌طورِ کلی، مسئلهٔ n-ویژگیِ نشان‌دهندهٔ MUI را می‌توان با n مطلوبیتِ تک‌ویژگی و n ثابت مدل کرد. هر یک از توابعِ مطلوبیتِ تک‌ویژگی را می‌توان مستقل از ویژگی‌های دیگر توسعه داد، و تضمین می‌شود این ترکیب ترجیح‌های کلیِ درست را تولید کند. برای به‌دست‌آوردنِ تابعِ مطلوبیتِ صرفاً افزایشی، مفروضاتِ اضافی لازم است.

۱۶.۵ شبکه‌های تصمیم

در این بخش، به سازوکارِ عمومیِ تصمیم‌گیریِ عقلایی نگاه می‌کنیم. این نماد اغلب «نمودارِ تأثیر» (Influence Diagram) نامیده می‌شود (هاوارد و ماتیسون، 1984)، اما ما از اصطلاحِ توصیفی‌ترِ «شبکهٔ تصمیم» (Decision Network) استفاده می‌کنیم. شبکه‌های تصمیم شبکه‌های بیزی را با انواعِ گرهِ اضافی برای کنش‌ها و مطلوبیت‌ها ترکیب می‌کنند. از مسئلهٔ انتخابِ مکانِ فرودگاه به‌عنوانِ مثال استفاده می‌کنیم.

۱۶.۵.۱ بازنماییِ مسئلهٔ تصمیم با شبکهٔ تصمیم

در عمومی‌ترین شکلش، شبکهٔ تصمیم اطلاعاتِ مربوط به حالتِ کنونیِ عامل، کنش‌های ممکنِ او، حالتی که از کنشِ عامل نتیجه می‌شود و مطلوبیتِ آن حالت را بازنمایی می‌کند. بنابراین بستری برای پیاده‌سازیِ عامل‌های مبتنی بر مطلوبیت از نوعِ نخستین‌بارِ معرفی‌شده در بخش ۲.۴.۵ فراهم می‌کند. شکل ۱۶.۶ شبکهٔ تصمیم را برای مسئلهٔ مکان‌یابیِ فرودگاه نشان می‌دهد.

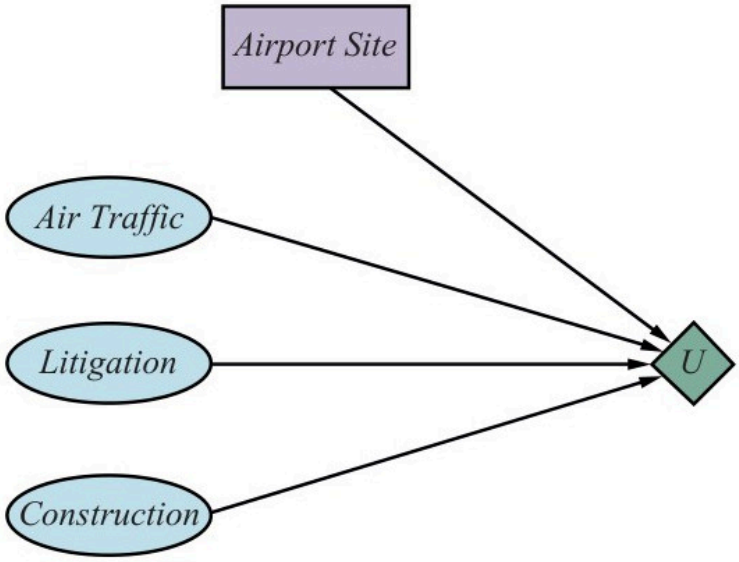


شکل ۱۶.۶ — شبکهٔ تصمیم برای مسئلهٔ مکان‌یابیِ فرودگاه. گره‌های شانس: Litigation (دادخواهی)، Construction (ساخت‌وساز)، Air Traffic (ترافیک هوایی)، Quietness (آرامش)، Frugality (صرفه‌جویی)، Safety (ایمنی)؛ گرهٔ تصمیم: Airport Site (مکان فرودگاه)؛ گرهٔ مطلوبیت: U.

سه نوع گره در شبکهٔ تصمیم هست:

- **گره‌های شانس** (Chance Nodes) متغیرهای تصادفی را بازنمایی می‌کنند. همان‌گونه که در شبکه‌های بیزی، هر گرهٔ شانس توزیعِ شرطی دارد که به والدهایش شرطی است. والدهای گرهٔ شانس می‌توانند گره‌های تصمیم یا گره‌های شانسِ دیگر باشند.
- **گره‌های تصمیم** (Decision Nodes) نقاطی را بازنمایی می‌کنند که عامل انتخابِ کنش دارد. والدهای گرهٔ تصمیم بازنماییِ اطلاعاتِ در دسترسِ عامل در زمانِ تصمیم‌گیری‌اند؛ برای سادگی، فرض می‌کنیم والدها دقیقاً متغیرهای شاهدِ ارزش‌گذاری‌شده‌اند و فقط متغیرهای شاهد می‌توانند والدِ گره‌های تصمیم باشند.
- **گره‌های مطلوبیت** (Utility Nodes)[7] مطلوبیتِ حالت‌های پیامد را بازنمایی می‌کنند.

در حالتِ ساده‌شده، گره‌های شانسِ توصیف‌کنندهٔ حالت‌های پیامد حذف می‌شوند. در عوض، گرهٔ مطلوبیت مستقیماً به گره‌های حالتِ کنونی و گرهٔ تصمیم متصل می‌شود. در این مورد، به‌جای بازنماییِ تابعِ مطلوبیت روی حالت‌های پیامد، گرهٔ مطلوبیت مطلوبیتِ موردانتظارِ مرتبط با هر کنش را بازنمایی می‌کند، چنان‌که در معادلهٔ (16.1) در صفحهٔ ۵۲۹ تعریف شد؛ یعنی گره با «تابعِ کنش-مطلوبیت» (Action-Utility Function) مرتبط است (که در یادگیریِ تقویتی «تابعِ Q» نیز نامیده می‌شود، چنان‌که در فصل ۲۲ توصیف شده). شکل ۱۶.۷ بازنماییِ کنش-مطلوبیتِ مسئلهٔ مکان‌یابیِ فرودگاه را نشان می‌دهد.



شکل ۱۶.۷ – بازنماییِ ساده‌شدهٔ مسئلهٔ مکان‌یابیِ فرودگاه. گره‌های شانسِ متناظر با حالت‌های پیامد حذف شده‌اند.

توجه کنید، چون گره‌های شانسِ آرامش، ایمنی و صرفه‌جویی در شکل ۱۶.۶ به حالت‌های آینده اشاره دارند، هرگز نمی‌توان مقادیرشان را به‌عنوانِ متغیرهای شاهد تنظیم کرد. بدین‌ترتیب، نسخهٔ ساده‌شده‌ای که این گره‌ها را حذف می‌کند هرگاه شکلِ عمومی‌تر قابلِ‌استفاده باشد قابلِ‌استفاده است. اگرچه شکلِ ساده‌شده گره‌های کمتری دارد، حذفِ توصیفِ صریحِ پیامدِ تصمیمِ مکان‌یابی بدان معناست نسبت به تغییرهای شرایط انعطافِ کمتری دارد. برای نمونه، در شکل ۱۶.۶، تغییر در سطحِ نویزِ هواپیما را می‌توان با تغییر در جدولِ احتمالِ شرطیِ مرتبط با گرهٔ آرامش بازتاب داد، در حالی که تغییر در وزنی که در تابعِ مطلوبیت به آلودگیِ صوتی داده می‌شود را می‌توان با تغییر در جدولِ مطلوبیت بازتاب داد. در نمودارِ کنش-مطلوبیت، شکل ۱۶.۷، از سوی دیگر، همهٔ چنین تغییرهایی باید با تغییر در جدولِ کنش-مطلوبیت بازتاب داده شوند. اساساً، صورت‌بندیِ کنش-مطلوبیت نسخهٔ کامپایل‌شدهٔ صورت‌بندیِ اصلی است، که با خارج‌کردنِ متغیرهای حالتِ پیامد از جمع به دست می‌آید.

۱۶.۵.۲ ارزیابیِ شبکه‌های تصمیم

کنش‌ها با ارزیابیِ شبکهٔ تصمیم برای هر تنظیمِ ممکنِ گرهٔ تصمیم انتخاب می‌شوند. پس از تنظیمِ گرهٔ تصمیم، دقیقاً مانندِ گرهٔ شانسی رفتار می‌کند که به‌عنوانِ متغیرِ شاهد تنظیم شده. الگوریتمِ ارزیابیِ شبکه‌های تصمیم به‌صورتِ زیر است:

1. متغیرهای شاهد را برای حالتِ کنونی تنظیم کن.
2. برای هر مقدارِ ممکنِ گرهٔ تصمیم:

(الف) گرهٔ تصمیم را روی آن مقدار تنظیم کن. (ب) احتمال‌های پسینی را برای گره‌های والدِ گرهٔ مطلوبیت، با یک الگوریتمِ استنتاجِ احتمالاتیِ استاندارد، محاسبه کن. (ج) مطلوبیتِ حاصل را برای کنش محاسبه کن.

1. کنشِ با بالاترین مطلوبیت را بازگردان.

این رویکردِ سرراستی است که می‌تواند از هر الگوریتمِ شبکهٔ بیزیِ موجود استفاده کند و می‌تواند مستقیماً در طراحیِ عاملِ داده‌شده در شکل ۱۲.۱ صفحهٔ ۳۸۸ گنجانده شود. در فصل ۱۷ خواهیم دید امکانِ اجرایِ چند کنش به‌صورتِ متوالی مسئله را بسیار جالب‌تر می‌کند.

۱۶.۶ ارزشِ اطلاعات

در تحلیلِ پیشین، فرض کردیم همهٔ اطلاعاتِ مربوط، یا دست‌کم همهٔ اطلاعاتِ موجود، قبل از تصمیم‌گیری به عامل داده می‌شود. در عمل، به‌ندرت چنین است. یکی از مهم‌ترین بخش‌های تصمیم‌گیری دانستنِ این است که چه پرسش‌هایی بپرسیم. برای نمونه، پزشک نمی‌تواند انتظار داشته باشد نتایجِ همهٔ آزمون‌ها و پرسش‌های تشخیصیِ ممکن در لحظهٔ ورودِ نخستینِ بیمار به اتاقِ مشاوره به او داده شود. آزمون‌ها اغلب گران‌اند و گاهی خطرناک (هم مستقیم و هم به دلیلِ تأخیرهای مرتبط). اهمیتشان به دو عامل بستگی دارد: اینکه آیا نتایجِ آزمون به طرحِ درمانیِ به‌طورِ قابلِ‌توجهی بهتر می‌انجامند، و اینکه نتایجِ گوناگونِ آزمون چقدر محتمل‌اند.

این بخش «نظریهٔ ارزشِ اطلاعات» (Information Value Theory) را توصیف می‌کند، که عاملی را قادر می‌کند انتخاب کند چه اطلاعاتی کسب کند. فرض می‌کنیم پیش از انتخابِ کنشِ «واقعیِ» بازنمایی‌شده توسط گرهٔ تصمیم، عامل می‌تواند مقدارِ هر یک از متغیرهای شانسِ بالقوه قابلِ مشاهده در مدل را کسب کند. بدین‌ترتیب، نظریهٔ ارزشِ اطلاعات شکلِ ساده‌شده‌ای از تصمیم‌گیریِ ترتیبی را در بر دارد—ساده‌شده، زیرا کنش‌های مشاهده فقط بر حالتِ باورِ عامل اثر می‌گذارند، نه بر حالتِ فیزیکیِ بیرونی. ارزشِ هر مشاهدهٔ خاص باید از پتانسیلِ اثرگذاری بر کنشِ فیزیکیِ نهاییِ عامل به دست آید؛ و این پتانسیل را می‌توان مستقیماً از خودِ مدلِ تصمیم برآورد کرد.

۱۶.۶.۱ نمونهٔ ساده

فرض کنیم شرکتِ نفتی امیدوار است یکی از n بلوکِ غیرقابلِ‌تمایزِ حقوقِ حفاریِ اقیانوسی را بخرد. همچنین فرض کنیم دقیقاً یکی از بلوک‌ها حاوی نفتی است که سودِ خالصِ C دلار تولید خواهد کرد، در حالی که بقیه بی‌ارزش‌اند. قیمتِ پیشنهادیِ هر بلوک C/n دلار است. اگر شرکت ریسک‌خنثی باشد، میانِ خریدنِ بلوکی و نخریدنِ آن بی‌تفاوت خواهد بود، زیرا سودِ موردانتظار در هر دو حالت صفر است.

حالا فرض کنیم لرزه‌شناسی نتایجِ بررسیِ بلوکِ شمارهٔ 3 را به شرکت پیشنهاد می‌دهد، که قطعاً نشان می‌دهد آیا بلوک حاوی نفت است یا نه. شرکت حاضر است چقدر برای این اطلاعات بپردازد؟ راهِ پاسخ به این پرسش بررسیِ این است که شرکت اگر اطلاعات را داشت چه می‌کرد:

- با احتمالِ n/1، بررسی نشان خواهد داد بلوکِ 3 نفت دارد. در این صورت، شرکت بلوکِ 3 را به قیمتِ C/n دلار می‌خرد و سودی برابرِ C - C/n = (n-1)C/n دلار می‌کند.
- با احتمالِ n/(n-1)، بررسی نشان خواهد داد بلوک نفت ندارد، که در آن صورت شرکت بلوکِ دیگری خواهد خرید. حالا احتمالِ یافتنِ نفت در یکی از بلوک‌های دیگر از n/1 به (n-1)/1 تغییر می‌کند، پس شرکت سودِ موردانتظاری برابرِ C/(n-1) - C/n = C/(n(n-1)) دلار می‌کند.

حالا می‌توانیم سودِ موردانتظار را، با دسترسی به اطلاعاتِ بررسی، محاسبه کنیم:

```
C/n = (C/(n(n−1))) × (n/(n−1)) + (C/n(n−1)) × (n/1)
```

بدین‌ترتیب، اطلاعات برای شرکت C/n دلار ارزش دارد و شرکت باید حاضر باشد بخشِ قابلِ‌توجهی از این مقدار را به لرزه‌شناس بپردازد.

ارزشِ اطلاعات از این واقعیت ناشی می‌شود که با اطلاعات، می‌توان دورهٔ کنش را برای تطبیق با وضعیتِ واقعی تغییر داد. می‌توان با توجه به وضعیت تمایز قائل شد، در حالی که بدونِ اطلاعات، باید آنچه در میانگینِ وضعیت‌های ممکن بهترین است انجام داد. به‌طورِ کلی، ارزشِ قطعهٔ اطلاعاتِ معین به‌صورتِ تفاوتِ ارزشِ موردانتظارِ میانِ بهترین کنش‌ها قبل و بعد از کسبِ اطلاعات تعریف می‌شود.

۱۶.۶.۲ فرمولِ عمومی برای اطلاعاتِ کامل

اشتقاقِ فرمولِ ریاضیِ عمومی برای ارزشِ اطلاعات ساده است. فرض می‌کنیم شواهدِ دقیقی دربارهٔ مقدارِ متغیرِ تصادفیِ Ej می‌توان به دست آورد (یعنی Ej = ej می‌آموزیم)، پس عبارتِ «ارزشِ اطلاعاتِ کامل» (Value of Perfect Information یا VPI) استفاده می‌شود.[8] در حالتِ اطلاعاتِ اولیهٔ عامل، ارزشِ بهترین کنشِ کنونیِ α، از معادلهٔ (16.1):

```
EU(α) = max_a ∑s′ P(RESULT(a)=s′) U(s′)
```

و ارزشِ بهترین کنشِ تازه (پس از به‌دست‌آمدنِ شاهدِ تازهٔ Ej = ej):

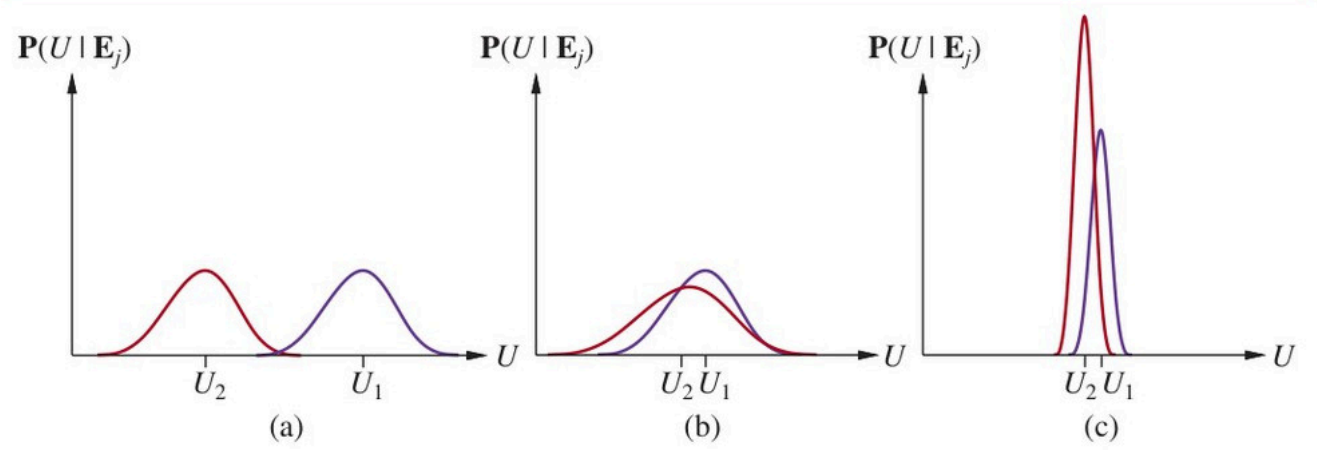
```
EU(α_ej | ej) = max_a ∑s′ P(RESULT(a)=s′ | ej) U(s′)
```

خواهد بود. اما Ej متغیرِ تصادفی است که مقدارش در حالِ حاضر ناشناخته است، پس برای تعیینِ ارزشِ کشفِ Ej باید روی همهٔ مقادیرِ ممکنِ ej که ممکن است برای Ej کشف کنیم، با استفاده از باورهای کنونی‌مان دربارهٔ مقدارش، میانگین بگیریم:

```
VPI(Ej) = (∑_ej P(Ej = ej) EU(α_ej | Ej = ej)) − EU(α)
```

برای شهودِ این فرمول، حالتِ ساده‌ای را در نظر بگیرید که فقط دو کنش، a1 و a2، برای انتخاب هست. مطلوبیت‌های موردانتظارِ کنونی‌شان U1 و U2 است. اطلاعاتِ Ej = ej مطلوبیت‌های موردانتظارِ تازه‌ای U'1 و U'2 برای کنش‌ها خواهد داد، اما قبل از به‌دست‌آوردنِ Ej، توزیع‌های احتمالیِ معینی روی مقادیرِ ممکنِ U'1 و U'2 خواهیم داشت (که مستقل فرض می‌کنیم).

فرض کنیم a1 و a2 دو مسیرِ متفاوت در رشته‌کوه در زمستان‌اند: a1 بزرگراهِ خوب و مستقیمی از میانِ تونل است و a2 جادهٔ خاکیِ پرپیچ‌وخمی روی قله. با این اطلاعات، a1 به‌وضوح ترجیح‌پذیرتر است، زیرا کاملاً ممکن است a2 با برف مسدود شده باشد، در حالی که بعید است چیزی a1 را مسدود کند. U1 پس به‌وضوح از U2 بالاتر است. ممکن است گزارش‌های ماهواره‌ایِ Ej دربارهٔ وضعیتِ واقعیِ هر جاده به دست آید که انتظارهای تازه‌ای، U'1 و U'2، برای دو گذر می‌داد. توزیع‌های این انتظارها در شکل ۱۶.۸(الف) نشان داده شده‌اند. بدیهی است، در این مورد، صرفِ هزینهٔ به‌دست‌آوردنِ گزارش‌های ماهواره‌ای ارزش ندارد، زیرا بعید است اطلاعاتِ مشتق‌شده از آن‌ها برنامه را تغییر دهد. با بدونِ تغییر، اطلاعات هیچ ارزشی ندارد.



شکل ۱۶.۸ – سه حالتِ عام برای ارزشِ اطلاعات. در (الف)، a1 تقریباً مطمئناً برتر از a2 باقی می‌ماند، پس اطلاعات لازم نیست. در (ب)، انتخاب نامشخص است و اطلاعات حیاتی است. در (ج)، انتخاب نامشخص است، اما چون تفاوتِ کمی ایجاد می‌کند، اطلاعات کم‌ارزش‌تر است. (توجه کنید این واقعیت که U2 در (ج) قلهٔ بلندی دارد بدان معناست مقدارِ موردانتظارش با قطعیتِ بالاتری از U1 شناخته شده.)

حالا فرض کنیم بین دو جادهٔ خاکیِ پرپیچ‌وخمِ متفاوت با طول‌های کمی متفاوت انتخاب می‌کنیم و مسافری به‌شدت زخمی حمل می‌کنیم. آن‌گاه، حتی وقتی U1 و U2 بسیار نزدیک‌اند، توزیع‌های U'1 و U'2 بسیار پهن‌اند. احتمالِ قابلِ‌توجهی هست مسیرِ دوم معلوم شود باز است در حالی که اولی مسدود است، و در این صورت تفاوتِ مطلوبیت‌ها بسیار زیاد خواهد بود. فرمولِ VPI نشان می‌دهد ممکن است به‌دست‌آوردنِ گزارش‌های ماهواره‌ای ارزشمند باشد. چنین وضعیتی در شکل ۱۶.۸(ب) نشان داده شده.

سرانجام، فرض کنیم در تابستان بین دو جادهٔ خاکی انتخاب می‌کنیم، وقتی مسدودشدنِ با برف بعید است. در این مورد، گزارش‌های ماهواره‌ای ممکن است نشان دهند یک مسیر به دلیلِ چمنزارهای آلپیِ گل‌دار از دیگری چشم‌نوازتر (دارای مناظر زیباتر) است، یا شاید به دلیلِ بارانِ اخیر خیس‌تر. بنابراین کاملاً محتمل است اگر اطلاعات را داشتیم برنامه‌مان را تغییر می‌دادیم. در این مورد، با این حال، تفاوتِ ارزشِ بینِ دو مسیر همچنان احتمالاً بسیار کوچک است، پس زحمتِ به‌دست‌آوردنِ گزارش‌ها را به خود نمی‌دهیم. این وضعیت در شکل ۱۶.۸(ج) نشان داده شده.

به‌طورِ خلاصه، اطلاعات تا آن‌جا ارزش دارد که محتمل است به تغییرِ برنامه بینجامد و تا آن‌جا که برنامهٔ تازه به‌طورِ قابلِ‌توجهی از برنامهٔ کهنه بهتر خواهد بود.

۱۶.۶.۳ ویژگی‌های ارزشِ اطلاعات

ممکن است کسی بپرسد آیا ممکن است اطلاعات زیان‌آور باشد: آیا می‌تواند ارزشِ موردانتظارِ منفی داشته باشد؟ به‌صورتِ شهودی، باید انتظار داشته باشیم این ناممکن باشد. بالاخره، در بدترین حالت، می‌توان صرفاً اطلاعات را نادیده گرفت و وانمود کرد هرگز دریافت نشده. این توسط قضیهٔ زیر تأیید می‌شود، که برای هر عاملِ نظریه‌تصمیمیِ استفاده‌کننده از هر شبکهٔ تصمیمی با مشاهده‌های ممکنِ Ej اعمال می‌شود:

ارزشِ موردانتظارِ اطلاعات نامنفی است: $\forall j\ \mathrm{VPI}(E_j) \geq 0$.

قضیه مستقیماً از تعریفِ VPI نتیجه می‌شود، و اثباتش را به‌عنوانِ تمرین (تمرین NNVP.۱۶) واگذار می‌کنیم. البته، قضیه‌ای دربارهٔ ارزشِ موردانتظار است، نه ارزشِ واقعی. اطلاعاتِ اضافی به‌آسانی می‌تواند به برنامه‌ای بینجامد که معلوم شود بدتر از برنامهٔ اصلی است، اگر اطلاعات اتفاقاً گمراه‌کننده باشد. برای نمونه، آزمونِ پزشکی که نتیجهٔ مثبتِ کاذب می‌دهد ممکن است به جراحیِ غیرضروری بینجامد؛ اما این بدان معنا نیست آزمون نباید انجام شود. مهم است به یاد داشته باشیم VPI به حالتِ کنونیِ اطلاعات بستگی دارد. می‌تواند با کسبِ اطلاعاتِ بیشتر تغییر کند. برای هر قطعهٔ شاهدِ معینِ Ej، ارزشِ کسبِ آن می‌تواند پایین برود (مثلاً اگر متغیرِ دیگری پسینِ Ej را به‌شدت قید کند) یا بالا برود (مثلاً اگر متغیرِ دیگری سرنخی فراهم کند که Ej روی آن بنا می‌شود و برنامهٔ تازه و بهتری را ممکن می‌کند). بدین‌ترتیب، VPI افزایشی نیست. یعنی $\mathrm{VPI}(E_j, E_k) \neq \mathrm{VPI}(E_j) + \mathrm{VPI}(E_k)$ (به‌طورِ کلی). با این حال، VPI مستقل از ترتیب است. یعنی:

$$\mathrm{VPI}(E_j, E_k) = \mathrm{VPI}(E_j) + \mathrm{VPI}(E_k \mid E_j) = \mathrm{VPI}(E_k) + \mathrm{VPI}(E_j \mid E_k) = \mathrm{VPI}(E_k, E_j)$$

که در آن نمادِ $\mathrm{VPI}(\cdot \mid \mathbf{E})$ ارزشِ VPI محاسبه‌شده طبقِ توزیعِ پسینیِ شرطی‌شده بر $\mathbf{E}$ را نشان می‌دهد.

۱۶.۶.۴ پیاده‌سازیِ عاملِ اطلاعات‌جمع‌کن

عاملِ عاقل باید پرسش‌ها را در ترتیبِ معقولی بپرسد، باید از پرسیدنِ پرسش‌های نامربوط اجتناب کند، باید اهمیتِ هر قطعهٔ اطلاعات را نسبت به هزینه‌اش در نظر بگیرد، و باید وقتی مناسب است پرسیدن را متوقف کند. همهٔ این توانایی‌ها را می‌توان با استفاده از ارزشِ اطلاعات به‌عنوانِ راهنما به دست آورد.

شکل ۱۶.۹ طراحیِ کلیِ عاملی را نشان می‌دهد که می‌تواند قبل از عمل‌کردن هوشمندانه اطلاعات جمع کند. فعلاً فرض می‌کنیم با هر متغیرِ شاهدِ قابلِ مشاهدهٔ Ej، هزینه‌ای مرتبط به نامِ $C(E_j)$ هست، که هزینهٔ کسبِ شواهد از طریقِ آزمون‌ها، مشاوران، پرسش‌ها یا هر چیزِ دیگر را بازتاب می‌دهد. عامل چیزی را درخواست می‌کند که به نظر کارآمدترین مشاهده از نظرِ سودِ مطلوبیت به‌ازای هر واحدِ هزینه باشد. فرض می‌کنیم نتیجهٔ کنشِ Request(Ej) این است که حس‌گذارهٔ بعدی مقدارِ Ej را فراهم می‌کند. اگر هیچ مشاهده‌ای ارزشِ هزینه‌اش را نداشته باشد، عامل کنشِ «واقعی» را انتخاب می‌کند.

```
function INFORMATION-GATHERING-AGENT(percept) returns an action
                                    persistent: D, a decision network
                                                merge percept into D
                                j ← the value that maximizes VPI(Ej) / C(Ej)
                                  if VPI(Ej) > C(Ej) then return Request(Ej)
                                          else return the best action from D
```

شکل ۱۶.۹ — طراحیِ عاملِ سادهٔ اطلاعات‌جمع‌کنِ کوته‌نگر. عامل با انتخابِ مکررِ مشاهده‌ای با بالاترین ارزشِ اطلاعات کار می‌کند، تا وقتی هزینهٔ مشاهدهٔ بعدی از منفعتِ موردانتظارش بیشتر شود.

الگوریتمِ عاملِ توصیف‌شده شکلی از جمع‌آوریِ اطلاعات را پیاده‌سازی می‌کند که «کوته‌نگر» (Myopic) نامیده می‌شود. این به این دلیل است که از فرمولِ VPI کوته‌نگرانه استفاده می‌کند و ارزشِ اطلاعات را چنان محاسبه می‌کند که گویی فقط یک متغیرِ شاهد کسب خواهد شد. کنترلِ کوته‌نگر بر همان ایدهٔ اکتشافیِ جستجوی حریصانه مبتنی است و اغلب در عمل خوب کار می‌کند. (برای نمونه، نشان داده شده در انتخابِ آزمون‌های تشخیصی از پزشکانِ متخصص بهتر عمل می‌کند.) اما اگر هیچ متغیرِ شاهدِ تکی‌ای نباشد که خیلی کمک کند، عاملِ کوته‌نگر ممکن است شتاب‌زده کنشی انجام دهد، در حالی که بهتر بود نخست دو یا چند متغیر را درخواست کند و سپس کنش انجام دهد. بخشِ بعد امکانِ کسبِ چند مشاهده را در نظر می‌گیرد.

۱۶.۶.۵ جمع‌آوریِ اطلاعاتِ غیرکوته‌نگر

این واقعیت که ارزشِ توالیِ مشاهده‌ها تحتِ جایگشت‌های توالی ناورداست جذاب است، اما به خودیِ خود به الگوریتم‌های کارآمد برای جمع‌آوریِ اطلاعاتِ بهینه نمی‌انجامد. حتی اگر خودمان را به انتخابِ ازپیشِ زیرمجموعهٔ ثابتی از مشاهده‌ها محدود کنیم، 2^n زیرمجموعهٔ ممکن از n مشاهدهٔ بالقوه هست. در حالتِ عمومی، با مسئلهٔ حتی پیچیده‌تری از یافتنِ برنامهٔ شرطیِ بهینه (چنان‌که در بخش ۱۱.۵.۲ توصیف شد) روبه‌روییم که مشاهده‌ای را انتخاب می‌کند و سپس، بسته به پیامد، عمل می‌کند یا مشاهده‌های بیشتری انتخاب می‌کند. چنین برنامه‌هایی درخت‌هایی تشکیل می‌دهند، و شمارِ چنین درخت‌هایی فوق‌نمایی در n است.[9] برای مشاهده‌های متغیرها در شبکهٔ تصمیم، معلوم می‌شود این مسئله ناشدنی است حتی وقتی شبکه پلیدرخت باشد. با این حال، حالت‌های خاصی هست که در آن مسئله را می‌توان کارآمد حل کرد. در این‌جا یکی از این حالت‌ها را ارائه می‌کنیم: مسئلهٔ «گنج‌یابی» (Treasure Hunt) (یا «مسئلهٔ توالیِ آزمونِ کم‌هزینه‌ترین»، برای آن‌هایی که رمانتیک‌تر نیستند). n مکانِ $1,\ldots,n$ هست؛ هر مکانِ i با احتمالِ مستقلِ P(i) حاوی گنج است؛ و بررسیِ مکانِ i هزینهٔ C(i) دارد. این با شبکهٔ تصمیمی متناظر است که در آن همهٔ متغیرهای شاهدِ بالقوهٔ Treasure_i به‌صورتِ مطلق مستقل‌اند. عامل مکان‌ها را به ترتیبی بررسی می‌کند تا گنج پیدا شود؛ پرسش این است که ترتیبِ بهینه چیست؟

برای پاسخ به این پرسش، باید هزینه‌های موردانتظار و احتمال‌های موفقیتِ توالی‌های گوناگونِ مشاهده را در نظر بگیریم، با فرضِ اینکه عامل وقتی گنج پیدا شد می‌ایستد. فرض کنیم x چنین توالی‌ای باشد؛ xy الحاقِ توالی‌های x و y؛ C(x) هزینهٔ موردانتظارِ x؛ P(x) احتمالِ اینکه توالیِ x در یافتنِ گنج موفق شود؛ و F(x) = 1-P(x) احتمالِ اینکه شکست بخورد. با این تعریف‌ها:

```
C(xy) = C(x) + F(x)C(y)          (16.3)
```

داریم، یعنی توالیِ xy قطعاً هزینهٔ x را متحمل می‌شود و، اگر x شکست بخورد، هزینهٔ y را نیز. ایدهٔ پایه در هر مسئلهٔ بهینه‌سازیِ توالی، نگاه‌کردن به تغییرِ هزینه است، که با Δ = C(wxyz) - C(wyxz) تعریف می‌شود، وقتی دو زیرتوالیِ مجاورِ x و y در توالیِ عمومیِ wxyz جابه‌جا می‌شوند. وقتی توالی بهینه است، همهٔ چنین تغییرهایی توالی را بدتر می‌کنند. گامِ نخست نشان‌دادنِ این است که علامتِ اثر (افزایش یا کاهشِ هزینه) به بافتِ فراهم‌شده توسط w و z بستگی ندارد. داریم:

```
Δ = [C(w) + F(w)C(xyz)] − [C(w) + F(w)C(yxz)]      (با معادلهٔ (16.3))
  = F(w)[C(xyz) − C(yxz)]
  = F(w)[(C(xy) + F(xy)C(z)) − (C(yx) + F(yx)C(z))]   (با معادلهٔ (16.3))
  = F(w)[C(xy) − C(yx)]                              (زیرا F(xy) = F(yx))
```

پس نشان داده‌ایم جهتِ تغییرِ هزینهٔ کلِ توالی فقط به جهتِ تغییرِ هزینهٔ جفتِ عناصرِ جابه‌جاشونده بستگی دارد؛ بافتِ جفت مهم نیست. این راهی به ما می‌دهد توالی را با مقایسه‌های دوتایی مرتب کنیم تا راه‌حلِ بهینه به دست آید. به‌طورِ خاص، اکنون:

```
Δ = F(w)[(C(x) + F(x)C(y)) − (C(y) + F(y)C(x))]   (با معادلهٔ (16.3))
  = F(w)[C(x)(1−F(y)) − C(y)(1−F(x))]
  = F(w)[C(x)P(y) − C(y)P(x)]
```

داریم. این برای هر توالیِ x و y برقرار است، پس به‌طورِ خاص وقتی x و y مشاهده‌های تکیِ مکان‌های i و j باشند برقرار است. پس می‌دانیم برای اینکه i و j در توالیِ بهینه مجاور باشند، باید C(i)P(j) ≤ C(j)P(i) داشته باشیم، یا:

```
P(i)/C(i) ≥ P(j)/C(j)
```

به بیانِ دیگر، ترتیبِ بهینه مکان‌ها را بر اساسِ احتمالِ موفقیت به‌ازای هر واحدِ هزینه رتبه‌بندی می‌کند. تمرین HUNT.۱۶ از شما می‌خواهد تعیین کنید آیا این در واقع سیاستِ دنبال‌شده توسط الگوریتمِ شکل ۱۶.۹ برای این مسئله است یا نه.

۱۶.۶.۶ تحلیلِ حساسیت و تصمیم‌های مقاوم

رویهٔ «تحلیلِ حساسیت» (Sensitivity Analysis) در رشته‌های فناورانه فراگیر است: یعنی تحلیلِ اینکه خروجیِ فرایندی با دست‌کاری‌شدنِ پارامترهای مدل چقدر تغییر می‌کند. تحلیلِ حساسیت در سامانه‌های احتمالاتی و نظریه‌تصمیمی به‌طورِ خاص مهم است، زیرا احتمال‌های استفاده‌شده معمولاً یا از داده‌ها یاد گرفته می‌شوند یا توسط کارشناسانِ انسانی برآورد می‌شوند، که بدان معناست خودشان در معرضِ عدم قطعیتِ قابلِ‌توجهی‌اند. فقط در مواردِ نادر، مانندِ پرتاب‌های تاس در تخته‌نرد، احتمال‌ها به‌صورتِ عینی شناخته شده‌اند.

برای فرایندِ تصمیم‌گیریِ مطلوبیت‌محور، می‌توانید خروجی را یا تصمیمِ واقعیِ گرفته‌شده یا مطلوبیتِ موردانتظارِ آن تصمیم بدانید. نخستِ دومی را در نظر بگیرید: چون انتظار به احتمال‌های مدل بستگی دارد، می‌توانیم مشتقِ مطلوبیتِ موردانتظارِ هر کنشِ معین را نسبت به هر یک از آن مقادیرِ احتمال محاسبه کنیم. (برای نمونه، اگر همهٔ توزیع‌های احتمالِ شرطیِ مدل به‌صورتِ صریح جدول‌بندی شده باشند، محاسبهٔ انتظار شاملِ محاسبهٔ نسبتِ دو عبارتِ جمع-حاصل‌ضرب است؛ برای اطلاعاتِ بیشتر، فصل ۲۰ را ببینید.) بدین‌ترتیب، می‌توان تعیین کرد کدام پارامترهای مدل بزرگ‌ترین اثر را بر مطلوبیتِ موردانتظارِ تصمیمِ نهایی دارند.

اگر، در عوض، نگرانِ تصمیمِ واقعیِ گرفته‌شده باشیم، نه مطلوبیتِ آن طبقِ مدل، آن‌گاه می‌توانیم به‌سادگی پارامترها را به‌صورتِ نظام‌مند تغییر دهیم (شاید با جستجوی دودویی) تا ببینیم آیا تصمیم تغییر می‌کند و، اگر می‌کند، کوچک‌ترین اغتشاشی که چنین تغییری ایجاد می‌کند چیست. ممکن است فکر کنید چندان مهم نیست کدام تصمیم گرفته می‌شود، فقط مطلوبیتِ آن مهم است. درست است، اما در عمل ممکن است تفاوتِ بسیار قابلِ‌توجهی میانِ مطلوبیتِ واقعیِ تصمیم و مطلوبیتِ طبقِ مدل باشد.

اگر همهٔ اغتشاش‌های معقولِ پارامترها تصمیمِ بهینه را تغییر ندهند، آن‌گاه معقول است فرض کنیم تصمیم تصمیمِ خوبی است، حتی اگر برآوردِ مطلوبیتِ آن تصمیم به‌طورِ قابلِ‌توجهی نادرست باشد. اگر، از سوی دیگر، تصمیمِ بهینه با تغییرِ پارامترهای مدل به‌طورِ قابلِ‌توجهی تغییر کند، آن‌گاه شانسِ خوبی هست مدل تصمیمی تولید کند که در واقعیت به‌طورِ قابلِ‌توجهی زیربهینه باشد. در آن صورت، ارزش دارد تلاشِ بیشتری برای پالایشِ مدل سرمایه‌گذاری شود.

این شهودها در چند حوزه (نظریهٔ کنترل، تحلیلِ تصمیم، مدیریتِ ریسک) صوری شده‌اند که مفهومِ تصمیمِ «مقاوم» (Robust) یا «حداکثرِ حداقل» (Minimax) را پیشنهاد می‌کنند—یعنی تصمیمی که در بدترین حالت بهترین نتیجه را می‌دهد. در این‌جا، «بدترین حالت» یعنی بدترین نسبت به همهٔ تغییرهای محتمل در مقادیرِ پارامترهای مدل. با θ برای همهٔ پارامترهای مدل، تصمیمِ مقاوم چنین تعریف می‌شود:

```
a* = argmax_a min_θ EU(a; θ)
```

در بسیاری از موارد، به‌طورِ خاص در نظریهٔ کنترل، رویکردِ مقاوم به طرح‌هایی می‌انجامد که در عمل بسیار قابلِ‌اعتماد کار می‌کنند. در مواردِ دیگر، به تصمیم‌های بیش‌ازحدِ محافظه‌کارانه می‌انجامد. برای نمونه، هنگام طراحیِ خودروی خودران، رویکردِ مقاوم بدترین حالتِ رفتارِ خودروهای دیگرِ جاده را فرض می‌کرد—یعنی همه‌شان توسط قاتلانِ دیوانه رانده می‌شوند. در آن صورت، راه‌حلِ بهینه برای خودرو ماندن در گاراژ است.

نظریهٔ تصمیمِ بیزی جایگزینی برای روش‌های مقاوم ارائه می‌دهد: اگر عدم قطعیت دربارهٔ پارامترهای مدل هست، آن‌گاه آن عدم قطعیت را با «ابرپارامترها» (Hyperparameters) مدل کن. در حالی که رویکردِ مقاوم ممکن است بگوید احتمالِ θi در مدل می‌تواند هر جایِ بینِ 0.3 و 0.7 باشد، با مقدارِ واقعیِ انتخاب‌شده توسط دشمنی برای اینکه همه‌چیز تا آن‌جا که ممکن است بد از آب درآید، رویکردِ بیزی توزیعِ احتمالِ پیشینی روی θi می‌گذارد و سپس مثلِ قبل ادامه می‌دهد. این به تلاشِ مدل‌سازیِ بیشتری نیاز دارد—برای نمونه، مدل‌سازِ بیزی باید تصمیم بگیرد آیا پارامترهای θi و θj مستقل‌اند—اما اغلب در عمل به عملکردِ بهتر می‌انجامد.

علاوه بر عدم قطعیتِ پارامتری، کاربردهای نظریهٔ تصمیم در دنیایِ واقعی از عدم قطعیتِ ساختاری نیز رنج می‌برند. برای نمونه، فرضِ استقلالِ Litigation، AirTraffic و Construction در شکل ۱۶.۶ ممکن است نادرست باشد، و ممکن است متغیرهای اضافی باشند که مدل صرفاً حذف‌شان می‌کند. در حالِ حاضر، درکِ خوبی از چگونگیِ درنظرگرفتنِ این نوع عدم قطعیت نداریم. یکی از امکان‌ها نگه‌داشتنِ مجموعه‌ای از مدل‌هاست، شاید تولیدشده توسط الگوریتم‌های یادگیریِ ماشین، به امیدِ اینکه مجموعه تغییرهای مهمِ مربوط را در بر بگیرد.

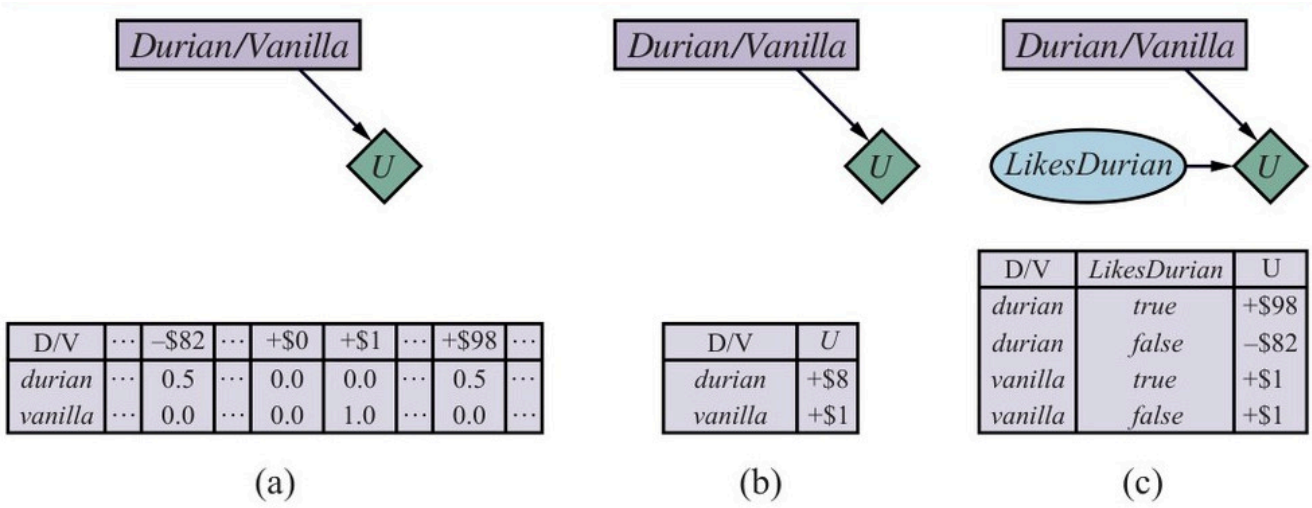
۱۶.۷ ترجیح‌های ناشناخته

در این بخش بحث می‌کنیم وقتی عدم قطعیت دربارهٔ تابعِ مطلوبیتی هست که قرار است مقدارِ موردانتظارش بهینه شود چه اتفاقی می‌افتد. دو نسخه از این مسئله هست: یکی که در آن عامل (ماشین یا انسان) دربارهٔ تابعِ مطلوبیتِ خودش نامطمئن است، و دیگری که در آن ماشین قرار است به انسان کمک کند اما دربارهٔ آنچه انسان می‌خواهد نامطمئن است.

۱۶.۷.۱ عدم قطعیت دربارهٔ ترجیح‌های خود

تصور کنید در بستنی‌فروشی‌ای در تایلند هستید و فقط دو طعم باقی مانده: وانیل و دوریان. هر دو ۲ دلار قیمت دارند. می‌دانید به وانیل علاقه‌ای متوسط دارید و در چنین روزِ گرمی حاضرید تا ۳ دلار برای بستنیِ وانیلی بپردازید، پس سودِ خالصِ ۱ دلاری برای انتخابِ وانیل هست. از سوی دیگر، هیچ ایده‌ای ندارید دوریان را دوست دارید یا نه، اما در ویکی‌پدیا خوانده‌اید دوریان پاسخ‌های متفاوتی از افرادِ مختلف برمی‌انگیزد: برخی می‌یابند «در طعم از همهٔ میوه‌های جهان فراتر است» در حالی که برخی دیگر آن را به «فاضلاب، استفراغِ مانده، اسپریِ اسکانک و تامپونِ جراحیِ استفاده‌شده» مانند می‌کنند. برای عددگذاشتن روی این، بگوییم ۵۰٪ شانس هست آن را فوق‌العاده بیابید (+۱۰۰ دلار) و ۵۰٪ شانس هست از آن متنفر شوید (-۸۰ دلار اگر طعمش تمامِ بعدازظهر بماند). در این‌جا، هیچ عدم قطعیتی دربارهٔ جایزه‌ای که می‌برید نیست—به هر حال همان بستنیِ دوریانی است—اما عدم قطعیت دربارهٔ ترجیحِ خودتان برای آن جایزه هست.

می‌توانیم صورت‌بندیِ شبکهٔ تصمیم را برای اجازه‌دادن به مطلوبیت‌های نامطمئن بسط دهیم، چنان‌که در شکل ۱۶.۱۰(الف) نشان داده شده. اما اگر اطلاعاتِ بیشتری دربارهٔ ترجیحِ دوریانی‌تان قابلِ‌کسب نباشد—برای نمونه، اگر مغازه اجازه‌ی چشیدنِ قبلی ندهد—آن‌گاه مسئلهٔ تصمیم یکسان با نشان‌داده‌شده در شکل ۱۶.۱۰(ب) است. می‌توانیم به‌سادگی مقدارِ نامطمئنِ دوریان را با سودِ خالصِ موردانتظارش، (0.5×100$) - (0.5×80$) - 2$ = 8$، جایگزین کنیم و تصمیم‌تان بدونِ تغییر می‌ماند.

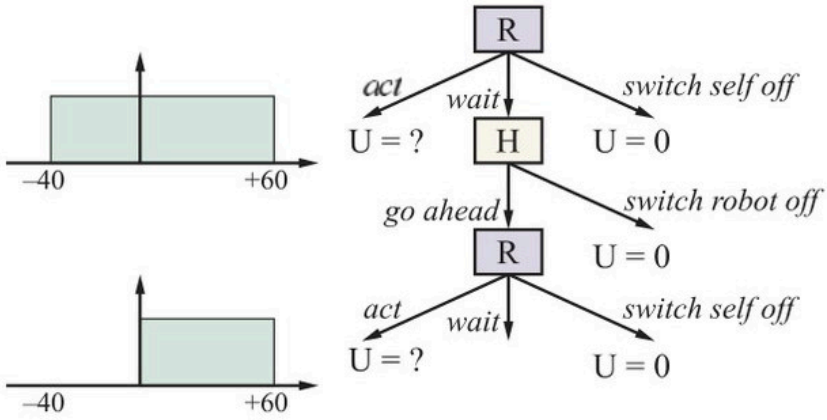


شکل ۱۶.۱۰ – (الف) شبکهٔ تصمیم برای انتخابِ بستنی با تابعِ مطلوبیتِ نامطمئن. (ب) شبکه با مطلوبیتِ موردانتظارِ هر کنش. (ج) انتقالِ عدم قطعیت از تابعِ مطلوبیت به متغیرِ تصادفیِ تازه.

اگر ممکن باشد باورهایتان دربارهٔ دوریان تغییر کنند—شاید طعمِ ریزی بچشید، یا بفهمید همهٔ خویشاوندانِ زنده‌تان دوریان را دوست دارند—آن‌گاه تبدیلِ شکل ۱۶.۱۰(ب) معتبر نیست. اما معلوم می‌شود همچنان می‌توانیم مدلِ معادلی بیابیم که در آن تابعِ مطلوبیت قطعی است. به‌جای گفتنِ اینکه عدم قطعیت دربارهٔ تابعِ مطلوبیت هست، آن عدم قطعیت را «به درونِ جهان» منتقل می‌کنیم، به بیانِ دیگر. یعنی متغیرِ تصادفیِ تازه‌ای به نام LikesDurian با احتمال‌های پیشینیِ 0.5 برای صادق و کاذب می‌سازیم، چنان‌که در شکل ۱۶.۱۰(ج) نشان داده شده. با این متغیرِ اضافی، تابعِ مطلوبیت قطعی می‌شود، اما همچنان می‌توانیم باورهای در حالِ تغییر دربارهٔ ترجیحِ دوریانی‌تان را مدیریت کنیم.

۱۶.۷.۲ اطاعت از انسان

حالا به حالتِ دومِ ذکرشده در بالا بپردازیم: ماشینی که قرار است به انسان کمک کند اما دربارهٔ آنچه انسان می‌خواهد نامطمئن است. برخوردِ کاملِ این حالت باید به فصل ۱۸ موکول شود، جایی که تصمیم‌های شاملِ بیش از یک عامل را بحث می‌کنیم. در این‌جا، یک پرسشِ ساده می‌پرسیم: تحتِ چه شرایطی چنین ماشینی از انسان اطاعت خواهد کرد؟



شکل ۱۶.۱۱ – بازیِ کلیدِ خاموش. R، ربات، می‌تواند انتخاب کند همین حالا عمل کند، با بازدهیِ بسیار نامطمئن؛ خودش را خاموش کند؛ یا به H، انسان، تفویض کند. H می‌تواند R را خاموش کند یا بگذارد ادامه دهد. R حالا دوباره همان انتخاب را دارد. عمل‌کردن همچنان بازدهیِ نامطمئن دارد، اما حالا R می‌داند بازدهی نامنفی است.

برای مطالعهٔ این پرسش، سناریوی بسیار ساده‌ای را در نظر بگیریم، چنان‌که در شکل ۱۶.۱۱ نشان داده شده. روبی رباتِ نرم‌افزاری است که برای هریت، انسانِ پرمشغله‌ای، به‌عنوانِ دستیارِ شخصی‌اش کار می‌کند. هریت برای جلسهٔ کاریِ بعدی‌اش در ژنو به اتاقِ هتل نیاز دارد. روبی می‌تواند همین حالا عمل کند—بگذارید بگوییم می‌تواند هریت را در هتلِ بسیار گرانی نزدیکِ محلِ جلسه رزرو کند. او کاملاً نامطمئن است هریت هتل و قیمتش را چقدر دوست خواهد داشت؛ بگذارید بگوییم احتمالِ یکنواختی برای ارزشِ خالصش برای هریت بینِ -40 و +60، با میانگینِ +10 دارد. او همچنین می‌تواند «خودش را خاموش کند»—کم‌دراماتیک‌تر، خودش را کلاً از فرایندِ رزروِ هتل خارج کند—که تعریف می‌کنیم (بدونِ ازدست‌دادنِ عمومیت) ارزشِ 0 برای هریت دارد. اگر این دو انتخابِ او بودند، جلو می‌رفت و هتل را رزرو می‌کرد، و ریسکِ قابلِ‌توجهیِ ناراضی‌کردنِ هریت را متحمل می‌شد. (اگر بازه -60 تا +40، با میانگینِ -10 بود، در عوض خودش را خاموش می‌کرد.) با این حال، انتخابِ سومی به روبی می‌دهیم: برنامه‌اش را توضیح بده، صبر کن و بگذار هریت او را خاموش کند. هریت یا می‌تواند او را خاموش کند یا بگذارد جلو برود و هتل را رزرو کند. چه خیرِ ممکنی از این حاصل می‌شود، ممکن است کسی بپرسد، با توجه به اینکه او می‌توانست هر دوی آن انتخاب‌ها را خودش بکند؟ نکته این است که انتخابِ هریت—خاموش‌کردنِ روبی یا گذاشتنِ او به جلو—اطلاعاتی دربارهٔ ترجیح‌های هریت به روبی فراهم می‌کند. فعلاً فرض می‌کنیم هریت عقلایی است، پس اگر هریت بگذارد روبی جلو برود، بدان معناست ارزش برای هریت مثبت است. حالا، چنان‌که در شکل ۱۶.۱۱ نشان داده شده، باورِ روبی تغییر می‌کند: یکنواخت بینِ 0 و +60، با میانگینِ +30 است. پس، اگر انتخاب‌های اولیهٔ روبی را از دیدگاهِ او ارزیابی کنیم:

1. عمل‌کردنِ همین حالا و رزروِ هتل ارزشِ موردانتظارِ +10 دارد.
2. خاموش‌کردنِ خودش ارزشِ 0 دارد.
3. صبرکردن و گذاشتنِ هریت به خاموش‌کردنش به دو پیامدِ ممکن می‌انجامد:

(الف) ۴۰٪ شانس هست، بر اساسِ عدم قطعیتِ روبی دربارهٔ ترجیح‌های هریت، که او از برنامه متنفر شود و روبی را خاموش کند، با ارزشِ 0. (ب) ۶۰٪ شانس هست هریت برنامه را دوست داشته باشد و به روبی اجازه دهد جلو برود، با ارزشِ موردانتظارِ +30. بدین‌ترتیب، صبرکردن ارزشِ موردانتظارِ (0×0.4) + (30×0.6) = +18 دارد، که بهتر از +10ای است که روبی اگر همین حالا عمل کند انتظار دارد.

نتیجهٔ نهایی این است که روبی انگیزهٔ مثبتی برای تفویض به هریت دارد—یعنی اجازه‌دادن به خاموش‌شدنِ خودش. این انگیزه مستقیماً از عدم قطعیتِ روبی دربارهٔ ترجیح‌های هریت می‌آید. روبی آگاه است شانسی هست (۴۰٪ در این مثال) که در آستانهٔ انجامِ کاری باشد که هریت را ناراحت کند، که در آن صورت خاموش‌شدن بر جلو رفتن ترجیح‌پذیر است. اگر روبی قبلاً از ترجیح‌های هریت مطمئن بود، صرفاً جلو می‌رفت و تصمیم را می‌گرفت (یا خودش را خاموش می‌کرد)؛ مطلقاً چیزی برای به‌دست‌آوردن از مشورت با هریت نبود، زیرا، طبقِ باورهای قطعیِ روبی، او می‌توانست دقیقاً پیش‌بینی کند هریت چه تصمیمی خواهد گرفت.

در واقع، می‌توان همان نتیجه را در حالتِ عمومی اثبات کرد: تا زمانی که روبی کاملاً مطمئن نیست دارد همان کاری را می‌کند که خودِ هریت می‌کرد، برایش بهتر است به هریت اجازه دهد که روبی را خاموش کند. به‌صورتِ شهودی، تصمیمِ او اطلاعاتی به روبی فراهم می‌کند، و ارزشِ موردانتظارِ اطلاعات همیشه نامنفی است. برعکس، اگر روبی از تصمیمِ هریت مطمئن باشد، تصمیمِ او اطلاعاتِ تازه‌ای فراهم نمی‌کند، و بنابراین روبی انگیزه‌ای برای اجازه‌دادن به تصمیم‌گیریِ او ندارد.

به‌صورتِ صوری، فرض کنیم $P(u)$ چگالیِ احتمالِ پیشینیِ روبی روی مطلوبیتِ هریت برای کنشِ پیشنهادیِ a باشد. سپس ارزشِ جلو رفتن با a:

$$EU(a) = \int P(u)\cdot u\, du = \int_{-\infty}^{0} P(u)\cdot u\, du + \int_{0}^{\infty} P(u)\cdot u\, du$$

(به‌زودی خواهیم دید چرا انتگرال این‌گونه تقسیم می‌شود.) از سوی دیگر، ارزشِ کنشِ d، تفویض به هریت، از دو بخش تشکیل شده: اگر $u > 0$ آن‌گاه هریت می‌گذارد روبی جلو برود، پس ارزش u است، اما اگر $u < 0$ آن‌گاه هریت روبی را خاموش می‌کند، پس ارزش 0 است:

$$EU(d) = \int_{-\infty}^{0} P(u)\cdot 0\, du + \int_{0}^{\infty} P(u)\cdot u\, du$$

با مقایسهٔ عبارت‌های $EU(a)$ و $EU(d)$، بلافاصله می‌بینیم $EU(d) \geq EU(a)$، زیرا عبارتِ $EU(d)$ ناحیهٔ مطلوبیتِ منفی را صفر کرده. دو انتخاب فقط وقتی ارزشِ برابر دارند که ناحیهٔ منفی احتمالِ صفر داشته باشد—یعنی وقتی روبی قبلاً مطمئن است هریت کنشِ پیشنهادی را دوست دارد.

بسط‌های بدیهی‌ای بر مدل هست که ارزشِ کاوشِ فوری دارند. بسطِ نخست تحمیلِ هزینه‌ای برای وقتِ هریت است. در آن صورت، روبی اگر ریسکِ نزولی کوچک باشد تمایلِ کمتری به مزاحمتِ هریت دارد. این همان‌گونه است که باید باشد. و اگر هریت واقعاً از این‌که کارش قطع شود (مزاحمش شوند) بیزار باشد، نباید زیاد تعجب کند اگر روبی گاهی کارهایی کند که دوست ندارد. بسطِ دوم اجازه‌دادن به احتمالی برای خطای انسانی است—یعنی هریت ممکن است گاهی روبی را خاموش کند حتی وقتی کنشِ پیشنهادی‌اش معقول است، و ممکن است گاهی بگذارد روبی جلو برود حتی وقتی کنشِ پیشنهادی‌اش نامطلوب است. جمع‌کردنِ این احتمالِ خطا در مدل سرراست است (تمرین OFFS.۱۶ را ببینید). چنان‌که انتظار می‌رود، راه‌حل نشان می‌دهد روبی تمایلِ کمتری به تفویض به هریتِ غیرعقلایی دارد که گاهی

برضدِ منافعِ خودش عمل می‌کند. هرچه او تصادفی‌تر رفتار کند، روبی باید دربارهٔ ترجیح‌هایش نامطمئن‌تر باشد تا به او تفویض کند. باز، این همان‌گونه است که باید باشد: برای نمونه، اگر روبی خودروی خودران باشد و هریت مسافرِ دو سالهٔ شیطانش، روبی نباید اجازه دهد هریت در وسطِ بزرگراه او را خاموش کند.

خلاصهٔ فصل

این فصل نشان می‌دهد چگونه نظریهٔ مطلوبیت را با احتمال ترکیب کنیم تا عامل بتواند کنش‌هایی را انتخاب کند که عملکردِ موردانتظارش را بیشینه می‌کنند.

- نظریهٔ احتمال توصیف می‌کند عامل بر اساسِ شواهد چه باید باور کند، نظریهٔ مطلوبیت توصیف می‌کند عامل چه می‌خواهد، و نظریهٔ تصمیم این دو را کنار هم می‌گذارد تا توصیف کند عامل چه باید بکند.
- می‌توانیم از نظریهٔ تصمیم برای ساختنِ سامانه‌ای استفاده کنیم که با درنظرگرفتنِ همهٔ کنش‌های ممکن و برگزیدنِ کنشی که به بهترین پیامدِ موردانتظار می‌انجامد تصمیم می‌گیرد. چنین سامانه‌ای «عاملِ عقلایی» شناخته می‌شود.
- نظریهٔ مطلوبیت نشان می‌دهد عاملی که ترجیح‌هایش میانِ لاتاری‌ها با مجموعه‌ای از اصل‌های ساده سازگار است را می‌توان به‌عنوانِ دارندهٔ تابعِ مطلوبیت توصیف کرد؛ علاوه بر این، عامل کنش‌ها را چنان انتخاب می‌کند که گویی مطلوبیتِ موردانتظارش را بیشینه می‌کند.
- نظریهٔ مطلوبیتِ چندویژگی با مطلوبیت‌هایی سروکار دارد که به چند ویژگیِ متمایز از حالت‌ها بستگی دارند. سلطهٔ تصادفی فنِ خاصاً مفیدی برای گرفتنِ تصمیم‌های بدونِ ابهام است، حتی بدونِ مقادیرِ مطلوبیتِ دقیق برای ویژگی‌ها.
- شبکه‌های تصمیم صورت‌بندیِ ساده‌ای برای بیان و حلِ مسائلِ تصمیم فراهم می‌کنند. آن‌ها بسطِ طبیعیِ شبکه‌های بیزی‌اند، شاملِ گره‌های تصمیم و مطلوبیت علاوه بر گره‌های شانس.
- گاهی، حلِ مسئله شاملِ یافتنِ اطلاعاتِ بیشتر قبل از گرفتنِ تصمیم است. ارزشِ اطلاعات به‌صورتِ بهبودِ موردانتظارِ مطلوبیت در مقایسه با گرفتنِ تصمیم بدونِ اطلاعات تعریف می‌شود؛ به‌طورِ خاص برای هدایتِ فرایندِ جمع‌آوریِ اطلاعات قبل از گرفتنِ تصمیمِ نهایی مفید است.
- وقتی، چنان‌که اغلب رخ می‌دهد، مشخص‌کردنِ کامل و درستِ تابعِ مطلوبیتِ انسان ناممکن است، ماشین‌ها باید تحتِ عدم قطعیت دربارهٔ هدفِ واقعی عمل کنند. این وقتی تفاوتِ قابلِ‌توجهی ایجاد می‌کند که امکانِ کسبِ اطلاعاتِ بیشتر دربارهٔ ترجیح‌های انسانی برای ماشین وجود داشته باشد. با استدلالی ساده نشان دادیم عدم قطعیت دربارهٔ ترجیح‌ها تضمین می‌کند ماشین از انسان اطاعت کند، تا آن‌جا که به خودش اجازه دهد خاموش شود.

یادداشت‌های کتاب‌شناختی و تاریخی

در رسالهٔ قرنِ هفدهمیِ «هنرِ اندیشیدن» (L'art de Penser)، یا «منطقِ پورت-رویال»، آرنو (1662) بیان می‌کند:

«برای قضاوت دربارهٔ آنچه باید برای به‌دست‌آوردنِ خیری یا اجتناب از شری انجام داد، لازم است نه فقط خودِ خیر و شر، بلکه احتمالِ وقوع یا عدمِ وقوعِ آن را نیز در نظر گرفت؛ و نسبتِ هندسیِ همهٔ این چیزها را با هم دید.»

متن‌های مدرن به‌جای خیر و شر از مطلوبیت سخن می‌گویند، اما این بیان به‌درستی یادآوری می‌کند باید مطلوبیت را در احتمال ضرب کرد («به‌صورتِ هندسی دید») تا مطلوبیتِ موردانتظار به دست آید، و آن را روی همهٔ پیامدها («همهٔ این چیزها») بیشینه کرد تا «قضاوت کرد چه باید کرد». قابلِ‌توجه است آرنو چقدر را درست گفت، بیش از ۳۵۰ سال پیش، و فقط ۸ سال پس از آنکه پاسکال و فرما نخستین بار نشان دادند چگونه احتمال را درست به کار ببریم.

دانیل برنولی (1738)، هنگام بررسیِ «پارادوکسِ سنتپترزبورگ» (تمرین STPT.۱۶ را ببینید)، نخستین کسی بود که به اهمیتِ اندازه‌گیریِ ترجیح برای لاتاری‌ها پی برد، و نوشت «ارزشِ یک قلم نباید بر قیمتش مبتنی باشد، بلکه بر مطلوبیتی که می‌دهد» (تأکید از اوست). فیلسوفِ فایده‌گرای جرمی بنتام (1823) «حسابِ لذت‌گرایانه» (Hedonic Calculus) را برای سنجیدنِ «لذت‌ها» و «دردها» پیشنهاد کرد، و استدلال کرد همهٔ تصمیم‌ها (نه فقط تصمیم‌های پولی) را می‌توان به مقایسه‌های مطلوبیت کاهش داد.

معرفیِ برنولی از مطلوبیت—کمیتِ درونی و ذهنی—برای تبیینِ رفتارِ انسانی از طریقِ نظریهٔ ریاضی، برای زمانِ خود پیشنهادِ کاملاً قابلِ‌توجهی بود. همه‌توجه‌تر از این واقعیت که برخلافِ مبالغِ پولی، مقادیرِ مطلوبیتِ شرط‌ها و جایزه‌های گوناگون مستقیماً قابلِ مشاهده نیستند؛ در عوض، مطلوبیت‌ها قرار است از ترجیح‌های نشان‌داده‌شده توسط فرد استنتاج شوند. دو قرن طول می‌کشید تا پیامدهای این ایده کاملاً کار شود و به‌طورِ گسترده توسط آماردانان و اقتصاددانان پذیرفته شود.

اشتقاقِ مطلوبیت‌های عددی از ترجیح‌ها نخستین بار توسط رامسی (1931) انجام شد؛ اصل‌های ترجیح در متنِ حاضر از نظرِ شکل به آن‌هایی نزدیک‌ترند که در «نظریهٔ بازی‌ها و رفتارِ اقتصادی» (فون نویمان و مورگنشترن، 1944) دوباره کشف شدند. رامسی احتمال‌های ذهنی (نه فقط مطلوبیت‌ها) را از ترجیح‌های عامل اشتقاق کرده بود؛ ساویج (1954) و جفری (1983) ساخت‌های جدیدتری از این نوع انجام می‌دهند. بیردون و همکاران (2002) نشان می‌دهند تابعِ مطلوبیت برای بازنماییِ ترجیح‌های ناترایا و دیگر موقعیت‌های نابهنجار کافی نیست.

در دورهٔ پس از جنگ، نظریهٔ تصمیم ابزارِ استانداردی در اقتصاد، مالی و علمِ مدیریت شد. حوزه‌ای از «تحلیلِ تصمیم» (Decision Analysis) برای عقلایی‌ترکردنِ تصمیم‌های سیاستی در حوزه‌هایی مانندِ استراتژیِ نظامی، تشخیصِ پزشکی، بهداشتِ عمومی، طراحیِ مهندسی و مدیریتِ منابع پدید آمد. فرایند شاملِ «تصمیم‌گیرنده» (Decision Maker) است که ترجیح‌های میانِ پیامدها را بیان می‌کند و «تحلیلگرِ تصمیم» (Decision Analyst) که کنش‌ها و پیامدهای ممکن را برمی‌شمارد و ترجیح‌ها را از تصمیم‌گیرنده استخراج می‌کند تا بهترین دورهٔ کنش تعیین شود. فن وینترفلت و ادواردز (1986) دیدگاهِ ظریفی دربارهٔ تحلیلِ تصمیم و رابطه‌اش با ساختارهای ترجیحِ انسانی فراهم می‌کنند. اسمیت (1988) مروری بر روش‌شناسیِ تحلیلِ تصمیم می‌دهد.

تا دههٔ ۱۹۸۰، مسائلِ تصمیمِ چندمتغیره با ساختنِ «درخت‌های تصمیم» از همهٔ مصداق‌های ممکنِ متغیرها مدیریت می‌شدند. نمودارهای تأثیر یا شبکه‌های تصمیم، که از همان ویژگی‌های استقلالِ شرطیِ شبکه‌های بیزی بهره می‌گیرند، توسط هاوارد و ماتیسون (1984) معرفی شدند، بر اساسِ کارِ قبلی در SRI (میلر و همکاران، 1976). الگوریتمِ هاوارد و ماتیسون درختِ تصمیمِ کامل (به‌صورتِ نمایی بزرگ) را از شبکهٔ تصمیم می‌ساخت. شاکتر (1986) روشی برای تصمیم‌گیریِ مستقیم بر اساسِ شبکهٔ تصمیم، بدونِ ایجادِ درختِ تصمیمِ میانی، توسعه داد. این الگوریتم همچنین یکی از نخستین‌هایی بود که استنتاجِ کامل برای شبکه‌های بیزیِ چندمتصل فراهم می‌کرد. نیلسون و لوریتزن (2000) الگوریتم‌های شبکهٔ تصمیم را به پیشرفت‌های جاری در الگوریتم‌های خوشه‌بندی برای شبکه‌های بیزی پیوند می‌دهند. مجموعهٔ اولیور و اسمیت (1990) مقاله‌های اولیهٔ مفیدِ بسیاری دربارهٔ شبکه‌های تصمیم دارد، همان‌گونه که شمارهٔ ویژهٔ 1990 ژورنالِ Networks. متنِ فنتون و نیل (2018) راهنمای عملی برای حلِ مسائلِ تصمیمِ دنیایِ واقعی با شبکه‌های تصمیم فراهم می‌کند. مقاله‌های شبکه‌های تصمیم و مدل‌سازیِ مطلوبیت نیز به‌طورِ منظم در ژورنال‌های Management Science و Decision Analysis ظاهر می‌شوند.

شگفت‌آورانه، پژوهشگرانِ معدودی از هوش مصنوعیِ اولیه پس از کاربردهای اولیهٔ تصمیم‌گیریِ پزشکیِ توصیف‌شده در فصل ۱۲ ابزارهای نظریه‌تصمیم را اتخاذ کردند. یکی از معدود استثناها جری فلدمن بود، که نظریهٔ تصمیم را بر مسائلِ بینایی (فلدمن و یاکیموفسکی، 1974) و برنامه‌ریزی (فلدمن و اسپرول، 1977) اعمال کرد. سامانه‌های خبرهٔ قاعده‌محورِ اواخرِ دههٔ ۱۹۷۰ و اوایلِ دههٔ ۱۹۸۰ بر پاسخ‌دادن به پرسش‌ها تمرکز داشتند، نه بر تصمیم‌گیری. آن سامانه‌هایی که کنش را توصیه می‌کردند عموماً با قاعده‌های شرط–کنش این کار را می‌کردند، نه با بازنمایی‌های صریحِ پیامدها و ترجیح‌ها.

شبکه‌های تصمیم رویکردِ بسیار انعطاف‌پذیرتری ارائه می‌دهند، برای نمونه با اجازه‌دادن به تغییرِ ترجیح‌ها در حالی که مدلِ انتقال ثابت می‌ماند، یا برعکس. آن‌ها همچنین محاسبهٔ اصولیِ اینکه بعداً چه اطلاعاتی جست‌وجو شود را اجازه می‌دهند. در اواخرِ دههٔ ۱۹۸۰، تا حدی به دلیلِ کارِ پرل روی شبکه‌های بیز، سامانه‌های خبرهٔ نظریه‌تصمیم پذیرشِ گسترده‌ای به دست آوردند (هورویتز و همکاران، 1988؛ کاول و همکاران، 2002). در واقع، از ۱۹۹۱ به بعد، طرحِ رویِ جلدِ ژورنالِ Artificial Intelligence یک شبکهٔ تصمیم را به تصویر کشیده، هرچند به نظر می‌رسد با جهتِ پیکان‌ها کمی مجوزِ هنری گرفته شده.

تلاش‌های عملی برای اندازه‌گیریِ مطلوبیت‌های انسانی با تحلیلِ تصمیمِ پس از جنگ آغاز شد (بالا را ببینید). معیارِ مطلوبیتِ میکرومورت توسط هاوارد (1989) بحث شده. تیلر (1992) دریافت برای شانسِ 1/1000 مرگ، پاسخ‌دهنده حاضر نبود بیش از ۲۰۰ دلار برای حذفِ ریسک بپردازد، اما ۵۰,۰۰۰ دلار برای پذیرشِ ریسک نمی‌پذیرفت.

استفاده از QALYها (سال‌های زندگیِ تعدیل‌شده با کیفیت) برای انجامِ تحلیل‌های هزینه–فایدهٔ مداخله‌های پزشکی و سیاست‌های اجتماعیِ مرتبط دست‌کم به کارِ کلارمن و همکاران (1968) بازمی‌گردد، هرچند خودِ اصطلاح نخستین بار توسط زکهاوزر و شپارد (1976) استفاده شد. مانندِ پول، QALYها فقط تحتِ مفروضاتِ نسبتاً قوی، مانندِ ریسک‌خنثی، که اغلب نقض می‌شوند، مستقیماً با مطلوبیت‌ها متناظرند (برسنیاک و همکاران، 2015)؛ با این حال، QALYها در عمل بسیار موردِ استفاده‌اند، برای نمونه در شکل‌دادن به سیاست‌های سرویسِ بهداشتِ ملی در بریتانیا.

۲۰۰۹ سالِ بزرگی برای کتاب‌های عامه‌پسند دربارهٔ غیرعقلانیتِ انسانی بود، از جملهِ «غیرعقلانیِ قابلِ‌پیش‌بینی» (آریلی، 2009)، «سوای» (برافمن و برافمن، 2009)، «تلنگر» (تیلر و سانستین، 2009)، «کلوگ» (مارکوس، 2009)، «چگونه تصمیم می‌گیریم» (لرر، 2009) و «دربارهٔ یقین‌داشتن» (برتون، 2009). آن‌ها کتابِ کلاسیکِ «قضاوت تحتِ عدم قطعیت» (کانمن و همکاران، 1982) و مقاله‌ای را که همه‌چیز را شروع کرد (کانمن و تورسکی، 1979) تکمیل می‌کنند. خودِ کانمن روایتی ژرف‌بین و خواندنی در «تفکر: سریع و آهسته» (کانمن، 2011) فراهم می‌کند.

حوزهٔ روان‌شناسیِ تکاملی (باس، 2005)، از سوی دیگر، خلافِ این ادبیات حرکت کرده و استدلال می‌کند انسان‌ها در بافت‌های تکاملاً مناسب کاملاً عقلایی‌اند. پیروانش اشاره می‌کنند غیرعقلانیت طبقِ تعریف در بافتِ تکاملی جریمه می‌شود و نشان می‌دهند در برخی موارد مصنوعِ چیدمانِ آزمایشی است (کامینز و آلن، 1998). احیایِ اخیری در علاقه به مدل‌های بیزیِ شناخت بوده، که دهه‌ها بدبینی را واژگون می‌کند (الیو، 2002؛ چاتر و اوکسفورد، 2008؛ گریفیتس و همکاران، 2008)؛ این احیا، با این حال، بدونِ منتقدانش نیست (جونز و لاو، 2011).

نظریهٔ ارزشِ اطلاعات نخست در بافتِ آزمایش‌های آماری کاوش شد، جایی که شبه‌مطلوبیتی (کاهشِ آنتروپی) استفاده می‌شد (لیندلی، 1956). نظریه‌پردازِ کنترلِ روسلان استراتونوویچ (1965) نظریهٔ عمومی‌تری را که این‌جا ارائه شده توسعه داد، که در آن اطلاعات به واسطهٔ توانایی‌اش در اثرگذاری بر تصمیم‌ها ارزش دارد. کارِ استراتونوویچ در غرب شناخته‌شده نبود، جایی که ران هاوارد (1966) همان ایده را پیشگام کرد. مقاله‌اش با این یادداشت پایان می‌یابد: «اگر نظریهٔ ارزشِ اطلاعات و ساختارهای نظریه‌تصمیمِ مرتبط در آینده بخشِ بزرگی از آموزشِ مهندسان را اشغال نکنند، آن‌گاه حرفهٔ مهندسی خواهد یافت که نقشِ سنتی‌اش در مدیریتِ منابعِ علمی و اقتصادی برای منفعتِ انسان به حرفه‌ای دیگر واگذار شده.» تا امروز، انقلابِ دلالت‌شده در روش‌های مدیریتی رخ نداده.

الگوریتمِ جمع‌آوریِ اطلاعاتِ کوته‌نگرِ توصیف‌شده در فصل در ادبیاتِ تحلیلِ تصمیم فراگیر است؛ خطوطِ کلی‌اش را می‌توان در مقالهٔ اصلیِ نمودارهای تأثیر (هاوارد و ماتیسون، 1984) دید. روش‌های محاسبهٔ کارآمد توسط دیتمِر و جنسن (1997) مطالعه شده. لاسکی (1995) و نیلسن و جنسن (2003) به‌ترتیب روش‌های تحلیلِ حساسیت را در شبکه‌های بیزی و شبکه‌های تصمیم بحث می‌کنند. متنِ کلاسیکِ «کنترلِ مقاوم و بهینه» (ژو و همکاران، 1995) پوششِ کاملی و مقایسه‌ای از رویکردهای مقاوم و نظریه‌تصمیم به تصمیم‌های تحتِ عدم قطعیت فراهم می‌کند.

مسئلهٔ گنج‌یابی به‌طورِ مستقل توسط نویسندگانِ بسیاری حل شد، دست‌کم به مقاله‌های آزمونِ ترتیبیِ گلاس (1959) و میتن (1960) بازمی‌گردد. سبکِ اثبات در این فصل بر نتیجهٔ پایه‌ای، از اسمیت (1956)، تکیه می‌کند که ارزشِ توالی را به ارزشِ همان توالی با دو عنصرِ مجاورِ جایگشته مرتبط می‌کند. این نتایج برای آزمون‌های مستقل به مسائلِ عمومی‌ترِ جستجوی درختی و گرافی (جایی که آزمون‌ها تا حدی مرتب‌اند) توسط کادان و سایمون (1977) بسط یافتند. نتایجِ پیچیدگیِ محاسباتِ غیرکوته‌نگرِ ارزشِ اطلاعات توسط کراوزه و گستریَن (2009) به دست آمد. کراوزه و همکاران (2008) مواردی را شناسایی کردند که در آن‌ها «زیرمدولاری» (Submodularity) به الگوریتمِ تقریبِ شدنی می‌انجامد، با تکیه بر کارِ بنیادینِ نموهاوزر و همکاران (1978) روی توابعِ زیرمدولار؛ کراوزه و گستریَن (2005) مواردی را شناسایی می‌کنند که الگوریتمِ دقیقِ برنامه‌نویسیِ پویا برای هر دو انتخابِ زیرمجموعهٔ شاهد و تولیدِ برنامهٔ شرطی راه‌حلِ کارآمدی می‌دهد.

هارسانی (1967) مسئلهٔ اطلاعاتِ ناقص را در نظریهٔ بازی مطالعه کرد، جایی که بازیکنان ممکن است توابعِ بازدهیِ یکدیگر را دقیقاً ندانند. او نشان داد چنین بازی‌هایی با بازی‌های با اطلاعاتِ ناقص، که در آن‌ها بازیکنان دربارهٔ حالتِ جهان نامطمئن‌اند، یکسان‌اند، از طریقِ حقهٔ افزودنِ متغیرهای حالتِ اشاره‌کننده به بازدهی‌های بازیکنان. سایرت و دِ گروت (1979) نظریه‌ای از «مطلوبیتِ تطبیقی» (Adaptive Utility) توسعه دادند که در آن عامل می‌توانست دربارهٔ تابعِ مطلوبیتِ خودش نامطمئن باشد و می‌توانست از طریقِ تجربه اطلاعاتِ بیشتری کسب کند.

کارِ «استخراجِ ترجیحِ بیزی» (چاجوسکا و همکاران، 2000؛ بوتیلیر، 2002) از فرضِ احتمالِ پیشینی روی تابعِ مطلوبیتِ عامل شروع می‌شود. فرن و همکاران (2014) مدلِ نظریه‌تصمیمیِ «کمک» (Assistance) را پیشنهاد می‌کنند که در آن ربات می‌کوشد هدفِ انسانی را که در آغاز درباره‌اش نامطمئن است دریابد و به آن کمک کند. مثالِ کلیدِ خاموش در بخش ۱۶.۷.۲ از هادفیلد-منل و همکاران (2017b) اقتباس شده. راسل (2019) چارچوبِ عمومی‌ای برای هوش مصنوعیِ سودمند پیشنهاد می‌کند که بازیِ کلیدِ خاموش نمونهٔ کلیدیِ آن است.

پانوشت‌ها:

[1] از خوانندگانی که خطوطِ هواییِ محلی‌شان دیگر در پروازهای طولانی غذا ارائه نمی‌دهد پوزش می‌خواهیم.

[2] می‌توانیم لذتِ قمار را با رمزگذاریِ رویدادهای قمار در توصیفِ حالت حساب کنیم؛ برای نمونه، «۱۰ دلار داشتن و قمارکردن» می‌تواند بر «۱۰ دلار داشتن و قمارنکردن» ترجیح داده شود.

[3] از این نظر، مطلوبیت‌ها شبیهِ دماها هستند: دمای فارنهایت 1.8 برابرِ دمای سانتی‌گراد به‌علاوهٔ 32 است، اما تبدیلِ یکی به دیگری شما را گرم‌تر یا سردتر نمی‌کند.

[4] چنین رفتاری ممکن است مأیوسانه نامیده شود، اما اگر کسی از قبل در وضعیتِ مأیوسانه باشد عقلایی است.

[5] برای نمونه، ریاضی‌دان/شعبده‌بازِ پرس د یاکونیس می‌تواند پرتابِ سکه را هر بار آن‌گونه که می‌خواهد از آب درآورد (لندهویز، 2004).

[6] حتی امرِ قطعی ممکن است قطعی نباشد. با وجودِ وعده‌های آهنین، هنوز آن ۲۷,۰۰۰,۰۰۰ دلار را از حسابِ بانکیِ نیجریه‌ایِ خویشاوندِ متوفایِ قبلاً ناشناخته دریافت نکرده‌ایم.

[7] این گره‌ها در ادبیات «گره‌های مقدار» (Value Nodes) نیز نامیده می‌شوند.

[8] الزام به داشتنِ اطلاعاتِ کامل موجبِ کاهشِ توانایی بیان (کاهشِ قدرتِ بازنمایی) نمی‌شود. فرض کنیم می‌خواهیم حالتی را مدل کنیم که در آن نسبت به متغیری کمی مطمئن‌تر می‌شویم. می‌توانیم آن را با معرفیِ متغیرِ دیگری که درباره‌اش اطلاعاتِ کامل می‌آموزیم انجام دهیم. برای نمونه، فرض کنیم در آغاز عدم قطعیتِ گسترده‌ای دربارهٔ متغیرِ Temperature داریم. سپس دانشِ کاملِ Thermometer = 37 را به دست می‌آوریم؛ این اطلاعاتِ ناقصی دربارهٔ ِTemperature واقعی می‌دهد، و عدم قطعیتِ ناشی از خطای اندازه‌گیری در مدلِ حسگرِ P(Thermometer|Temperature) رمزگذاری می‌شود. برای مثالِ دیگر، تمرین VPIX.۱۶ را ببینید.

[9] مسئلهٔ عمومیِ تولیدِ رفتارِ ترتیبی در محیطِ تا حدی قابلِ مشاهده تحتِ عنوانِ فرایندهای تصمیمِ مارکوفِ تا حدی قابلِ مشاهده قرار می‌گیرد، که در فصل ۱۷ توصیف شده‌اند.

[10] اِلزبرگ بعداً تحلیلگرِ نظامیِ RAND شد و اسنادی را که «اسنادِ پنتاگون» شناخته می‌شوند فاش کرد، و بدین‌ترتیب به پایانِ جنگِ ویتنام کمک کرد.

فصل ۱۷: تصمیم‌گیری پیچیده (Making Complex Decisions)

در این فصل روش‌هایی را بررسی می‌کنیم برای تصمیم‌گیری دربارهٔ اینکه امروز چه کنیم، با توجه به اینکه فردا ممکن است با تصمیمِ دیگری روبه‌رو شویم.

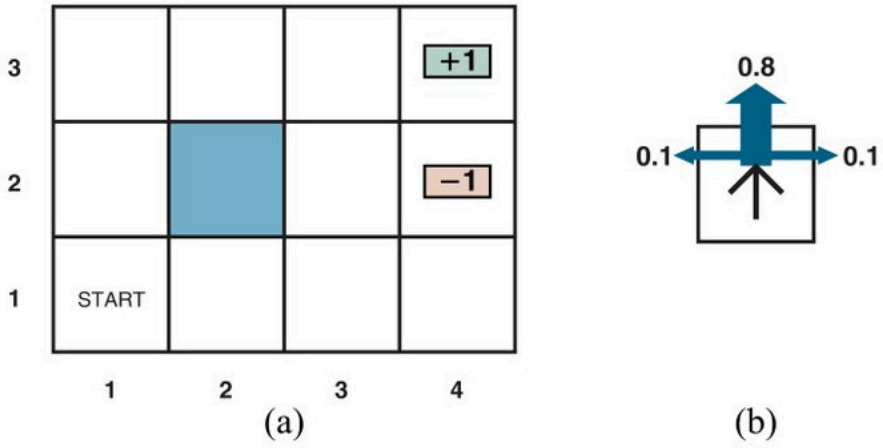
در این فصل، به مسائلِ محاسباتیِ مربوط به تصمیم‌گیری در محیطِ تصادفی می‌پردازیم. در حالی که فصل ۱۶ به مسائلِ تصمیمِ تک‌باره یا اپیزودیک می‌پرداخت، که در آن‌ها مطلوبیتِ پیامدِ هر کنش به‌خوبی شناخته‌شده بود، در این‌جا با «مسائلِ تصمیمِ ترتیبی» (Sequential Decision Problems) سروکار داریم، که در آن‌ها مطلوبیتِ عامل به توالیِ تصمیم‌ها بستگی دارد. مسائلِ تصمیمِ ترتیبی مطلوبیت‌ها، عدم قطعیت و حس‌کردن را در بر می‌گیرند، و مسائلِ جستجو و برنامه‌ریزی را به‌عنوانِ حالت‌های خاص شامل می‌شوند. بخش ۱۷.۱ توضیح می‌دهد مسائلِ تصمیمِ ترتیبی چگونه تعریف می‌شوند، و بخش ۱۷.۲ روش‌هایی برای حلِ آن‌ها برای تولیدِ رفتارهای مناسبِ محیطِ تصادفی توصیف می‌کند. بخش ۱۷.۳ مسائلِ «باندیتِ چندبازویی» (Multi-Armed Bandit) را پوشش می‌دهد، رده‌ای خاص و جذاب از مسائلِ تصمیمِ ترتیبی که در بافت‌های بسیاری پدید می‌آیند. بخش ۱۷.۴ مسائلِ تصمیم را در محیط‌های تا حدی قابل مشاهده کاوش می‌کند و بخش ۱۷.۵ توصیف می‌کند چگونه آن‌ها را حل کنیم.

۱۷.۱ مسائلِ تصمیمِ ترتیبی

فرض کنیم عامل در محیطِ 4×3 نشان‌داده‌شده در شکل ۱۷.۱(الف) قرار دارد. با شروع از حالتِ آغاز، باید در هر گامِ زمانی کنشی انتخاب کند. تعامل با محیط وقتی خاتمه می‌یابد که عامل به یکی از حالت‌های هدف، که با +1 یا -1 علامت‌گذاری شده‌اند، برسد. درست همان‌گونه که برای مسائلِ جستجو، کنش‌های موجود برای عامل در هر حالت توسط ACTIONS(s) داده می‌شوند، که گاهی به A(s) مخفف می‌شود؛ در محیطِ 4×3، کنش‌ها در هر حالت عبارت‌اند از بالا (Up)، پایین (Down)، چپ (Left) و راست (Right). فعلاً فرض می‌کنیم محیط کاملاً قابل مشاهده است، پس عامل همیشه می‌داند کجاست.

اگر محیط جبرگرایانه بود، راه‌حل آسان بود: [Up, Up, Right, Right, Right]. متأسفانه، محیط همیشه با این راه‌حل هم‌راهی نمی‌کند، زیرا کنش‌ها غیرقابل‌اعتمادند. مدلِ خاصِ حرکتِ تصادفی که اتخاذ می‌کنیم در شکل ۱۷.۱(ب) نشان داده شده. هر کنش با احتمالِ 0.8 به اثرِ موردنظر می‌رسد، اما بقیهٔ زمان، کنش عامل را در زاویهٔ قائمه نسبت به جهتِ موردنظر حرکت می‌دهد. علاوه بر این، اگر عامل به دیوار بخورد، در همان مربع می‌ماند. برای نمونه، از مربعِ آغاز (1,1)، کنشِ Up عامل را با احتمالِ 0.8 به (1,2) می‌برد، اما با احتمالِ 0.1 به راست به (2,1) می‌برد، و با احتمالِ 0.1 به چپ می‌برد، به دیوار می‌خورد و در (1,1) می‌ماند. در چنین محیطی، توالیِ [Up,Up,Right,Right,Right] از دورِ مانع بالا

می‌رود و با احتمالِ $0.8^5 = 0.32768$ به حالتِ هدف در (4,3) می‌رسد. همچنین شانسِ کوچکی برای رسیدنِ تصادفی به هدف از مسیرِ دیگر با احتمالِ $0.1^4 \times 0.8$ وجود دارد، که در مجموع 0.32776 می‌شود. (همچنین تمرین MDPX.۱۷ را ببینید.)



شکل ۱۷.۱ – (الف) محیطِ سادهٔ تصادفیِ 4×3 که مسئلهٔ تصمیمِ ترتیبی را به عامل ارائه می‌کند. (ب) نشان‌دادنِ مدلِ انتقالِ محیط: پیامدِ «موردِنظر» با احتمالِ 0.8 رخ می‌دهد، اما با احتمالِ 0.2 عامل در زاویهٔ قائمه نسبت به جهتِ موردنظر حرکت می‌کند. برخورد با دیوار به بی‌حرکتی می‌انجامد.

انتقال‌ها به دو حالتِ پایانی به‌ترتیب پاداشِ +1 و -1 دارند، و همهٔ انتقال‌های دیگر پاداشِ -0.04 دارند.

همانندِ فصل ۳، مدلِ انتقال (یا فقط «مدل»، وقتی معنا روشن است) پیامدِ هر کنش در هر حالت را توصیف می‌کند. در این‌جا، پیامد تصادفی است، پس $P(s'|s,a)$ را برای احتمالِ رسیدن به حالتِ s' اگر کنشِ a در حالتِ s انجام شود می‌نویسیم. (برخی نویسندگان $T(s,a,s')$ را برای مدلِ انتقال می‌نویسند.) فرض می‌کنیم انتقال‌ها مارکوفی‌اند: احتمالِ رسیدن به s' از s فقط به s بستگی دارد، نه به تاریخچهٔ حالت‌های زودتر.

برای کامل‌کردنِ تعریفِ محیطِ تکلیف، باید تابعِ مطلوبیت را برای عامل مشخص کنیم. چون مسئلهٔ تصمیم ترتیبی است، تابعِ مطلوبیت به توالیِ حالت‌ها و کنش‌ها—یک «تاریخچهٔ محیط» (Environment History)—بستگی خواهد داشت، نه به حالتِ تکی.

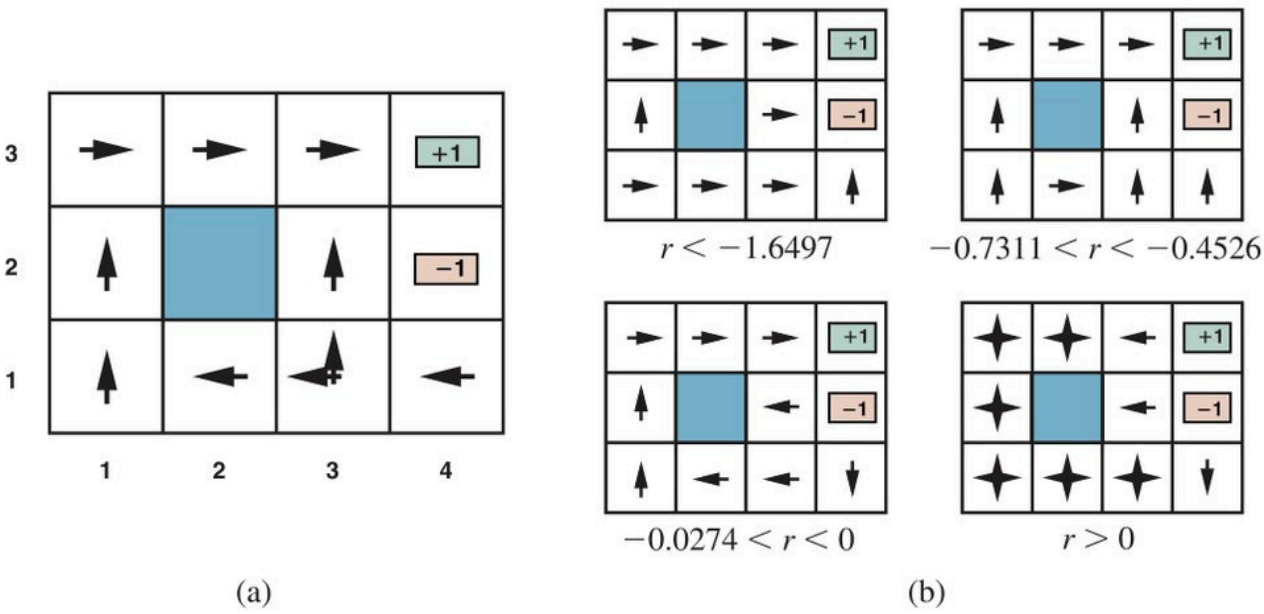
بعداً در این بخش ماهیتِ توابعِ مطلوبیت روی تاریخچه‌ها را بررسی می‌کنیم؛ فعلاً صرفاً مقرر می‌کنیم برای هر انتقال از s به s' از طریقِ کنشِ a، عامل «پاداشِ» (Reward) $R(s,a,s')$ دریافت می‌کند. پاداش‌ها می‌توانند مثبت یا منفی باشند، اما با $\pm R_{max}$ کران‌دارند.[1] برای مثالِ خاصِ ما، پاداش -0.04 برای همهٔ انتقال‌هاست به‌جزِ آن‌هایی که وارد حالت‌های پایانی می‌شوند (که پاداش‌های +1 و -1 دارند). مطلوبیتِ تاریخچهٔ محیط فعلاً (فقط فعلاً) جمعِ پاداش‌های دریافت‌شده است. برای نمونه، اگر عامل پس از ۱۰ گام به حالتِ +1 برسد، مطلوبیتِ کلش $1 + (-0.04 \times 9) = 0.64$ خواهد بود. پاداشِ منفیِ -0.04 به عامل انگیزه می‌دهد سریع به (4,3) برسد، پس محیطِ ما تعمیمِ تصادفیِ مسائلِ جستجویِ فصل ۳ است. راهِ دیگرِ بیانِ این موضوع این است که عامل از زندگی (حضور) در این محیط لذت نمی‌برد و بنابراین می‌خواهد هرچه زودتر بیرون برود.

به‌طورِ خلاصه: مسئلهٔ تصمیمِ ترتیبی برای محیطِ کاملاً قابل مشاهدهٔ تصادفی با مدلِ انتقالِ مارکوفی و پاداش‌های افزایشی، «فرایندِ تصمیمِ مارکوف» (Markov Decision Process) یا MDP نامیده می‌شود، و شاملِ مجموعه‌ای از حالت‌ها (با حالتِ اولیهٔ s0)؛ مجموعه‌ای ACTIONS(s) از کنش‌ها در هر حالت؛ مدلِ انتقالِ P(s'|s,a)؛ و تابعِ پاداشِ R(s,a,s') است. روش‌های حلِ MDPها معمولاً از «برنامه‌نویسیِ پویا» (Dynamic Programming) استفاده می‌کنند: ساده‌کردنِ مسئله با شکستنِ بازگشتیِ آن به قطعاتِ کوچک‌تر و به‌خاطرسپاریِ راه‌حل‌های بهینهٔ قطعات.

پرسشِ بعدی این است که راه‌حلِ مسئله چه شکلی است؟ هیچ توالیِ کنشِ ثابتی نمی‌تواند مسئله را حل کند، زیرا عامل ممکن است به حالتی غیر از هدف برسد. بنابراین، راه‌حل باید مشخص کند عامل برای هر حالتی که ممکن است به آن برسد چه باید بکند. راه‌حلی از این نوع «سیاست» (Policy) نامیده می‌شود. مرسوم است سیاست را با π نشان دهیم، و π(s) کنشی است که سیاستِ π برای حالتِ s توصیه می‌کند. مهم نیست پیامدِ کنش چه باشد، حالتِ حاصل در سیاست خواهد بود و عامل خواهد دانست بعداً چه کند.

هر بار که سیاستِ معینی با شروع از حالتِ اولیه اجرا شود، ماهیتِ تصادفیِ محیط ممکن است به تاریخچهٔ محیطِ متفاوتی منتهی شود. کیفیتِ سیاست بنابراین با مطلوبیتِ موردانتظارِ تاریخچه‌های محیطِ ممکنِ تولیدشده توسط آن سیاست اندازه‌گیری می‌شود. «سیاستِ بهینه» (Optimal Policy) سیاستی است که بالاترین مطلوبیتِ موردانتظار را می‌دهد. از π* برای نشان‌دادنِ سیاستِ بهینه استفاده می‌کنیم. با π*، عامل با مشورت با حس‌گذارهٔ کنونی‌اش، که حالتِ کنونیِ s را به او می‌گوید، و سپس اجرایِ کنشِ π*(s)، تصمیم می‌گیرد چه کند. سیاست، تابعِ عامل را به‌صورتِ صریح بازنمایی می‌کند و بنابراین توصیفی از عاملِ بازتابیِ ساده است، که از اطلاعاتِ استفاده‌شده برای عاملِ مبتنی بر مطلوبیت محاسبه شده.

سیاست‌های بهینه برای جهانِ شکل ۱۷.۱ در شکل ۱۷.۲(الف) نشان داده شده‌اند. دو سیاست هست، زیرا عامل دقیقاً میانِ رفتنِ به چپ و رفتنِ به بالا از (3,1) بی‌تفاوت است: رفتن به چپ امن‌تر اما طولانی‌تر است، در حالی که رفتن به بالا سریع‌تر است اما ریسکِ افتادنِ تصادفی در (4,2) را دارد. به‌طورِ کلی اغلب چند سیاستِ بهینه وجود خواهد داشت.



شکل ۱۷.۲ — (الف) سیاست‌های بهینه برای محیطِ تصادفی با r = -0.04 برای انتقال‌های میانِ حالت‌های غیرپایانی. دو سیاست هست، زیرا در حالتِ (3,1) هر دو چپ و بالا بهینه‌اند. (ب) سیاست‌های بهینه برای چهار بازهٔ مختلفِ r.

توازنِ ریسک و پاداش بسته به مقدارِ r = R(s,a,s') برای انتقال‌های میانِ حالت‌های غیرپایانی تغییر می‌کند. سیاست‌های نشان‌داده‌شده در شکل ۱۷.۲(الف) برای -0.0850 < -0.0273 > r بهینه‌اند. شکل ۱۷.۲(ب) سیاست‌های بهینه را برای چهار بازهٔ دیگرِ r نشان می‌دهد. وقتی -1.6497 > r، زندگی چنان دردناک است که عامل مستقیم به نزدیک‌ترین خروجی می‌رود، حتی اگر خروجی -1 ارزش داشته باشد. وقتی -0.7311 < -0.4526 > r، زندگی کاملاً ناخوشایند است؛ عامل کوتاه‌ترین مسیر را به حالتِ +1 از (2,1)، (3,1) و (3,2) می‌گیرد، اما از (4,1) هزینهٔ رسیدن به +1 چنان بالاست که عامل ترجیح می‌دهد مستقیم به -1 شیرجه بزند. وقتی زندگی فقط اندکی تیره است (-0.0274 < r < 0)، سیاستِ بهینه هیچ ریسکی نمی‌کند. در (4,1) و (3,2)، عامل مستقیماً از حالتِ -1 دور می‌شود تا نتواند تصادفاً در آن بیفتد، حتی اگر این به معنای کوبیدنِ چندبارهٔ سرش به دیوار باشد. سرانجام، اگر 0 < r، زندگی واقعاً لذت‌بخش است و عامل از هر دو خروجی اجتناب می‌کند. تا زمانی که کنش‌ها در (4,1)، (3,2) و (3,3) همان‌گونه‌اند که نشان داده شده، هر سیاستی بهینه است و عامل پاداشِ کلِ نامتناهی می‌گیرد، زیرا هرگز وارد حالتِ پایانی نمی‌شود. معلوم می‌شود در مجموع نه سیاستِ بهینه برای بازه‌های مختلفِ r هست؛ تمرین THRC.۱۷ از شما می‌خواهد آن‌ها را بیابید.

معرفیِ عدم قطعیت، MDPها را به دنیای واقعی نزدیک‌تر از مسائلِ جستجویِ جبرگرایانه می‌کند. به همین دلیل، MDPها در چند حوزه مطالعه شده‌اند، از جملهِ هوش مصنوعی، تحقیق در عملیات، اقتصاد و نظریهٔ کنترل. ده‌ها الگوریتمِ حل پیشنهاد شده که چندتایشان را در بخش ۱۷.۲ بحث می‌کنیم. اما نخست، تعریف‌های مطلوبیت‌ها، سیاست‌های بهینه و مدل‌های MDP را با جزئیاتِ بیشتر بیان می‌کنیم.

۱۷.۱.۱ مطلوبیت‌ها در طولِ زمان

در مثالِ MDP در شکل ۱۷.۱، عملکردِ عامل با جمعِ پاداش‌های انتقال‌های تجربه‌شده اندازه‌گیری شد. این انتخابِ معیارِ عملکرد دلبخواه نیست، اما تنها امکان برای تابعِ مطلوبیت[۲] روی تاریخچه‌های محیط نیست، که آن را Uh([s0,a0,s1,a1,...,sn]) می‌نویسیم.

نخستین پرسشی که باید پاسخ داد این است که آیا «افقِ متناهی» (Finite Horizon) هست یا «افقِ نامتناهی» (Infinite Horizon) برای تصمیم‌گیری. افقِ متناهی بدان معناست زمانِ ثابتِ N هست که پس از آن هیچ‌چیز اهمیت ندارد—بازی تمام شده، به بیانِ دیگر. بدین‌ترتیب، Uh([s0,a0,s1,a1,...,sN+k]) = Uh([s0,a0,s1,a1,...,sN]) برای همهٔ k > 0. برای نمونه، فرض کنیم عاملی در جهانِ 4×3 شکل ۱۷.۱ از (3,1) شروع می‌کند، و فرض کنیم N = 3. آن‌گاه، برای داشتنِ هر شانسی برای رسیدن به حالتِ +1، عامل باید مستقیم به سمتِ آن برود، و کنشِ بهینه رفتن به بالا است. از سوی دیگر، اگر N = 100، وقتِ فراوانی برای رفتنِ مسیرِ امن با رفتن به چپ هست. پس، با افقِ متناهی، کنشِ بهینه در حالتِ معین ممکن است به زمانِ باقی‌مانده بستگی داشته باشد. سیاستی که به زمان بستگی دارد «غیرایستا» (Nonstationary) نامیده می‌شود. از سوی دیگر، بدونِ محدودیتِ زمانیِ ثابت، دلیلی برای رفتارِ متفاوت در همان حالت در زمان‌های متفاوت نیست. از این رو، کنشِ بهینه فقط به حالتِ کنونی بستگی دارد و سیاستِ بهینه «ایستا» (Stationary) است. سیاست‌های حالتِ افقِ نامتناهی بنابراین از سیاست‌های حالتِ افقِ متناهی ساده‌ترند، و ما در این فصل عمدتاً با حالتِ افقِ نامتناهی سروکار داریم. (بعداً خواهیم دید برای محیط‌های تا حدی قابل مشاهده، حالتِ افقِ نامتناهی چندان ساده نیست.) توجه کنید «افقِ نامتناهی» لزوماً به این معنا نیست همهٔ توالی‌های حالت نامتناهی‌اند؛ فقط بدان معناست هیچ موعدِ ثابتی نیست. در MDP با افقِ نامتناهی که شاملِ حالتِ پایانی است، توالی‌های حالتِ متناهی می‌توانند باشند.

پرسشِ بعدی که باید تصمیم بگیریم این است که مطلوبیتِ توالی‌های حالت را چگونه محاسبه کنیم. در سراسرِ این فصل، «پاداش‌های افزایشیِ تنزیل‌شده» (Additive Discounted Rewards) را خواهیم داشت: مطلوبیتِ یک تاریخچه:

```
... + Uh([s0,a0,s1,a1,s2,...]) = R(s0,a0,s1) + γR(s1,a1,s2) + γ²R(s2,a2,s3)
```

که در آن «عاملِ تنزیل» γ (Discount Factor) عددی بین 0 و 1 است. عاملِ تنزیل ترجیحِ عامل را برای پاداش‌های کنونی بر پاداش‌های آینده توصیف می‌کند. وقتی γ نزدیکِ 0 است، پاداش‌های در آیندهٔ دور بی‌اهمیت دیده می‌شوند. وقتی γ نزدیکِ 1 است، عامل مایل‌تر است برای پاداش‌های بلندمدت صبر کند. وقتی γ دقیقاً 1 است، پاداش‌های تنزیل‌شده به حالتِ خاصِ پاداش‌های صرفاً افزایشی کاهش می‌یابند. توجه کنید افزایشی‌بودن به‌صورتِ ضمنی در استفاده‌مان از تابع‌های هزینهٔ مسیر در اکتشافی‌ها استفاده شده. گزینهٔ سوم این است که توالی‌های نامتناهی را بر حسبِ «پاداشِ میانگین» (Average Reward) به‌دست‌آمده در هر گامِ زمانی مقایسه کنیم. فرض کنیم انتقال‌ها به مربعِ (1,1) در جهانِ 4×3 پاداشِ 0.1 دارند، در حالی که انتقال‌ها به سایر حالت‌های غیرپایانی پاداشِ 0.01 دارند. آن‌گاه

سیاستی که بهترین تلاشش را برای ماندن در (1,1) می‌کند، پاداشِ میانگینِ بالاتری از سیاستی خواهد داشت که جایِ دیگر می‌ماند. پاداشِ میانگین معیارِ مفیدی برای برخی مسائل است، اما تحلیلِ الگوریتم‌های پاداشِ میانگین پیچیده است.

پاداش‌های افزایشیِ تنزیل‌شده کمترین دشواری‌ها را در ارزیابیِ تاریخچه‌ها ارائه می‌دهند، پس از این پس از آن‌ها استفاده می‌کنیم.

۱۷.۱.۲ سیاست‌های بهینه و مطلوبیت‌های حالت‌ها

پس از تصمیم‌گیری دربارهٔ اینکه مطلوبیتِ تاریخچهٔ معین جمعِ پاداش‌های تنزیل‌شده است، می‌توانیم سیاست‌ها را با مقایسهٔ مطلوبیت‌های موردانتظارِ به‌دست‌آمده هنگام اجرایشان مقایسه کنیم. فرض می‌کنیم عامل در حالتِ اولیهٔ s است و St (متغیرِ تصادفی) را حالتی تعریف می‌کنیم که عامل هنگام اجرایِ سیاستِ خاصِ π در زمانِ t به آن می‌رسد. (بدیهی است، S0 = s، حالتی که عامل الان در آن است.) توزیعِ احتمال روی توالی‌های حالتِ S1,S2,... توسط حالتِ اولیهٔ s، سیاستِ π و مدلِ انتقالِ محیط تعیین می‌شود.

مطلوبیتِ موردانتظارِ حاصل از اجرایِ π با شروع از s چنین داده می‌شود:

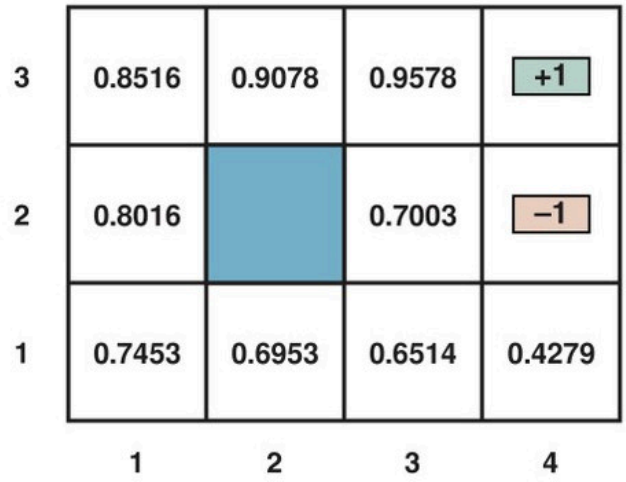
$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1})\right] \qquad (17.2)$$

که در آن انتظارِ E نسبت به توزیعِ احتمال روی توالی‌های حالتِ تعیین‌شده توسط s و π است. حالا، از میانِ همهٔ سیاست‌هایی که عامل می‌تواند با شروع از s انتخاب کند، یکی (یا بیشتر) مطلوبیتِ موردانتظارِ بالاتری از بقیه خواهد داشت. از π*s برای نشان‌دادنِ یکی از این سیاست‌ها استفاده می‌کنیم:

$$\pi^*_s = \operatorname{argmax}_{\pi} U^{\pi}(s) \qquad (17.3)$$

به یاد آورید π*s یک سیاست است، پس برای هر حالت کنشی توصیه می‌کند؛ ارتباطش با s به‌طورِ خاص این است که وقتی s حالتِ شروع است سیاستِ بهینه است. پیامدِ قابلِ‌توجهِ استفاده از مطلوبیت‌های تنزیل‌شده با افقِ نامتناهی این است که سیاستِ بهینه مستقل از حالتِ شروع است. (البته، توالیِ کنش مستقل نخواهد بود؛ به یاد آورید سیاست تابعی است که برای هر حالت کنشی مشخص می‌کند.) این واقعیت از نظرِ شهودی بدیهی به نظر می‌رسد: اگر سیاستِ π*a با شروع از a بهینه و سیاستِ π*b با شروع از b بهینه باشد، آن‌گاه وقتی به حالتِ سومی مانند c برسند، دلیلِ خوبی برای مخالفتِشان با یکدیگر، یا با π*c، دربارهٔ بعداً چه‌کردن نیست.[3] پس می‌توانیم به‌سادگی π* را برای سیاستِ بهینه بنویسیم.

با این تعریف، مطلوبیتِ واقعیِ حالت صرفاً $U^{\pi^*}(s)$ است—یعنی جمعِ موردانتظارِ پاداش‌های تنزیل‌شده اگر عامل سیاستِ بهینه را اجرا کند. این را U(s) می‌نویسیم، هم‌خوان با نمادِ استفاده‌شده در فصل ۱۶ برای مطلوبیتِ پیامد. شکل ۱۷.۳ مطلوبیت‌ها را برای جهانِ 4×3 نشان می‌دهد. توجه کنید مطلوبیت‌ها برای حالت‌های نزدیک‌تر به خروجیِ +1 بالاترند، زیرا گام‌های کمتری برای رسیدن به خروجی لازم است.



شکل ۱۷.۳ — مطلوبیت‌های حالت‌ها در جهانِ 3×4 با γ = 1 و r = -0.04 برای انتقال‌ها به حالت‌های غیرپایانی. (مقادیر: از بالا-چپ: 0.8516، 0.9078، 0.9578 در ردیفِ بالا؛ 0.8016، 0.7003، 0.7453 در ردیفِ وسط؛ و 0.6953، 0.6514، 0.4279 در ردیفِ پایین.)

تابعِ مطلوبیتِ U(s) به عامل اجازه می‌دهد کنش‌ها را با استفاده از اصلِ بیشینهٔ مطلوبیتِ موردانتظار از فصل ۱۶ انتخاب کند—یعنی کنشی را برگزیند که پاداشِ گامِ بعد به‌علاوهٔ مطلوبیتِ تنزیل‌شدهٔ موردانتظارِ حالتِ بعدی را بیشینه می‌کند:

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma U(s')] \qquad (17.4)$$

مطلوبیتِ حالت، U(s)، را به‌عنوانِ جمعِ موردانتظارِ پاداش‌های تنزیل‌شده از آن نقطه به بعد تعریف کرده‌ایم. از این، نتیجه می‌شود رابطهٔ مستقیمی میانِ مطلوبیتِ حالت و مطلوبیتِ همسایه‌هایش هست: مطلوبیتِ حالت، پاداشِ موردانتظارِ انتقالِ بعد به‌علاوهٔ مطلوبیتِ تنزیل‌شدهٔ حالتِ بعد است، با فرضِ اینکه عامل کنشِ بهینه را انتخاب کند. یعنی مطلوبیتِ حالت چنین داده می‌شود:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma U(s')] \qquad (17.5)$$

این «معادلهٔ بلمن» (Bellman Equation) نامیده می‌شود، به نامِ ریچارد بلمن (1957). مطلوبیت‌های حالت‌ها—تعریف‌شده توسط معادلهٔ (17.2) به‌عنوانِ مطلوبیتِ موردانتظارِ توالی‌های حالتِ بعدی—راه‌حل‌های مجموعهٔ معادله‌های بلمن‌اند. در واقع، آن‌ها راه‌حل‌های یکتایند، چنان‌که در بخش ۱۷.۲.۱ نشان می‌دهیم. به یکی از معادله‌های بلمن برای جهانِ 3×4 نگاه کنیم. عبارتِ U(1,1) به‌صورتِ زیر است:

$$\begin{aligned} U(1,1) = \max\{ & \\ & 0.8[0.04-+\gamma U(1,2)) + 0.1(0.04-+\gamma U(1,1)) + 0.1(0.04-+\gamma U(2,1))] \quad (\text{راست}) \\ & 0.9[0.04-+\gamma U(2,1)) + 0.1(0.04-+\gamma U(1,1))] \quad (\text{پایین}) \\ & 0.9[0.04-+\gamma U(1,1)) + 0.1(0.04-+\gamma U(1,2))] \quad (\text{چپ}) \\ & 0.8[0.04-+\gamma U(1,1)) + 0.1(0.04-+\gamma U(2,1))] \quad (\text{بالا}) \\ \} & \end{aligned}$$

که در آن چهار عبارت به‌ترتیب با حرکت‌های بالا، چپ، پایین و راست متناظرند. وقتی مقادیرِ شکل ۱۷.۳ را با $\gamma = 1$ جای‌گذاری می‌کنیم، درمی‌یابیم بالا بهترین کنش است.

کمیتِ مهمِ دیگر «تابعِ کنش-مطلوبیت» یا «تابعِ Q» است: $Q(s,a)$ مطلوبیتِ موردانتظارِ انجامِ کنشِ معین در حالتِ معین است. تابعِ Q به شیوهٔ بدیهی به مطلوبیت‌ها مربوط است:

$$U(s) = \max_a Q(s,a) \qquad (17.6)$$

علاوه بر این، سیاستِ بهینه را می‌توان از تابعِ Q چنین استخراج کرد:

$$\pi^*(s) = \operatorname{argmax}_a Q(s,a) \qquad (17.7)$$

همچنین می‌توانیم معادلهٔ بلمن را برای توابعِ Q توسعه دهیم، با یادآوریِ اینکه پاداشِ کلِ موردانتظارِ انجامِ کنش، پاداشِ بی‌واسطه‌اش به‌علاوهٔ مطلوبیتِ تنزیل‌شدهٔ حالتِ پیامد است، که به‌نوبهٔ خود بر حسبِ تابعِ Q بیان می‌شود:

$$Q(s,a) = \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma\, U(s')]$$
$$\sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma \max_{a'} Q(s',a')] \qquad (17.8)$$

حلِ معادله‌های بلمن برای U (یا برای Q) آنچه برای یافتنِ سیاستِ بهینه نیاز داریم به ما می‌دهد. تابعِ Q بارها و بارها در الگوریتم‌های حلِ MDPها ظاهر می‌شود، پس از تعریفِ زیر استفاده می‌کنیم:

```
function Q-VALUE(mdp, s, a, U) returns a utility value
        return ∑s′ P(s′|s,a) [R(s,a,s′) + γ U[s′]]
```

۱۷.۱.۳ مقیاس‌های پاداش

فصل ۱۶ اشاره کرد مقیاسِ مطلوبیت‌ها دلبخواه است: تبدیلِ خطی-آفین (Affine Transformation)، تصمیمِ بهینه را تغییر نمی‌دهد. می‌توانیم $U(s)$ را با $U'(s) = mU(s) + b$ جایگزین کنیم، که در آن m و b ثابت‌های دلخواه با $m > 0$ هستند. از تعریفِ مطلوبیت‌ها به‌عنوانِ جمع‌های تنزیل‌شدهٔ پاداش‌ها، به‌آسانی دیده می‌شود تبدیلِ مشابهی از پاداش‌ها سیاستِ بهینه را در MDP تغییر نمی‌دهد:

$$R'(s,a,s') = mR(s,a,s') + b$$

اما معلوم می‌شود تجزیهٔ افزایشیِ مطلوبیت‌ها به آزادیِ قابلِ‌توجهیِ بیشتری در تعریفِ پاداش‌ها می‌انجامد. فرض کنیم $\Phi(s)$ هر تابعی از حالتِ s باشد. آن‌گاه، طبقِ «قضیهٔ شکل‌دهی» (Shaping Theorem)، تبدیلِ زیر سیاستِ بهینه را تغییر نمی‌دهد:

```
R′(s,a,s′) = R(s,a,s′) + γΦ(s′) − Φ(s)        (17.9)
```

برای نشان‌دادنِ درست‌بودنِ این، باید اثبات کنیم دو MDP، M و M'، سیاست‌های بهینهٔ یکسانی دارند، تا زمانی که فقط در توابعِ پاداش‌شان طبقِ معادلهٔ (17.9) تفاوت داشته باشند. از معادلهٔ بلمن برای Q شروع می‌کنیم، تابعِ Q برای MDP ِM:

```
Q(s,a) = ∑s′ P(s′|s,a) [R(s,a,s′) + γ max_{a′} Q(s′,a′)]
```

حالا بگذارید Q'(s,a) = Q(s,a) - Φ(s) و آن را در این معادله جای کنیم؛ می‌گیریم:

```
Q′(s,a) + Φ(s) = ∑s′ P(s′|s,a) [R(s,a,s′) + γ max_{a′} (Q′(s′,a′) + Φ(s′))]
```

که سپس به‌صورتِ زیر ساده می‌شود:

```
Q′(s,a) = ∑s′ P(s′|s,a) [R(s,a,s′) + γΦ(s′) − Φ(s) + γ max_{a′} Q′(s′,a′)]
                    s′ P(s′|s,a) [R′(s,a,s′) + γ max_{a′} Q′(s′,a′)]∑ =
```

به بیانِ دیگر، Q'(s,a) معادلهٔ بلمن را برای MDP ِM' ارضا می‌کند. حالا می‌توانیم سیاستِ بهینه را برای M' با معادلهٔ (17.7) استخراج کنیم:

```
π*_M′(s) = argmax_a Q′(s,a) = argmax_a (Q(s,a) − Φ(s)) = argmax_a Q(s,a) = π*_M(s)
```

تابعِ Φ(s) اغلب «پتانسیل» (Potential) نامیده می‌شود، به قیاس با پتانسیلِ الکتریکی (ولتاژ) که میدان‌های الکتریکی را پدید می‌آورد. جملهٔ γΦ(s') - Φ(s) به‌عنوانِ گرادیانِ پتانسیل عمل می‌کند. بدین‌ترتیب، اگر Φ(s) در حالت‌هایی که مطلوبیتِ بالاتری دارند مقدارِ بالاتری داشته باشد، افزودنِ γΦ(s') - Φ(s) به پاداش، اثرِ هدایتِ عامل «سربالا» در مطلوبیت را دارد.

در نگاهِ نخست، ممکن است نسبتاً ضدشهود به نظر برسد که بتوانیم پاداش را بدین‌گونه بدونِ تغییرِ سیاستِ بهینه اصلاح کنیم. کمک می‌کند به یاد آوریم همهٔ سیاست‌ها با تابعِ پاداشی که همه‌جا صفر است بهینه‌اند. این بدان معناست، طبقِ قضیهٔ شکل‌دهی، همهٔ سیاست‌ها برای هر پاداشِ مبتنی بر پتانسیلِ به‌صورتِ R(s,a,s') = γΦ(s') - Φ(s) بهینه‌اند. به‌صورتِ شهودی، این به این دلیل است که با چنین پاداشی مهم نیست عامل از A به B از کدام راه برود. (این وقتی γ = 1 آسان‌تر دیده می‌شود: در طولِ هر مسیری جمعِ پاداش‌ها به Φ(B) - Φ(A) فرو می‌ریزد، پس همهٔ مسیرها به‌یکسان خوب‌اند.) پس افزودنِ پاداشِ مبتنی بر پتانسیل به هر پاداشِ دیگری نباید سیاستِ بهینه را تغییر دهد.

انعطافِ فراهم‌شده توسط قضیهٔ شکل‌دهی بدان معناست می‌توانیم در واقع با مستقیم‌ترکردنِ بازتابِ پاداشِ بی‌واسطه از آنچه عامل باید بکند، به عامل کمک کنیم. در واقع، اگر Φ(s) = U(s) بگذاریم، آن‌گاه سیاستِ حریصانهٔ πG نسبت به پاداشِ اصلاح‌شدهٔ R' نیز سیاستِ بهینه است:

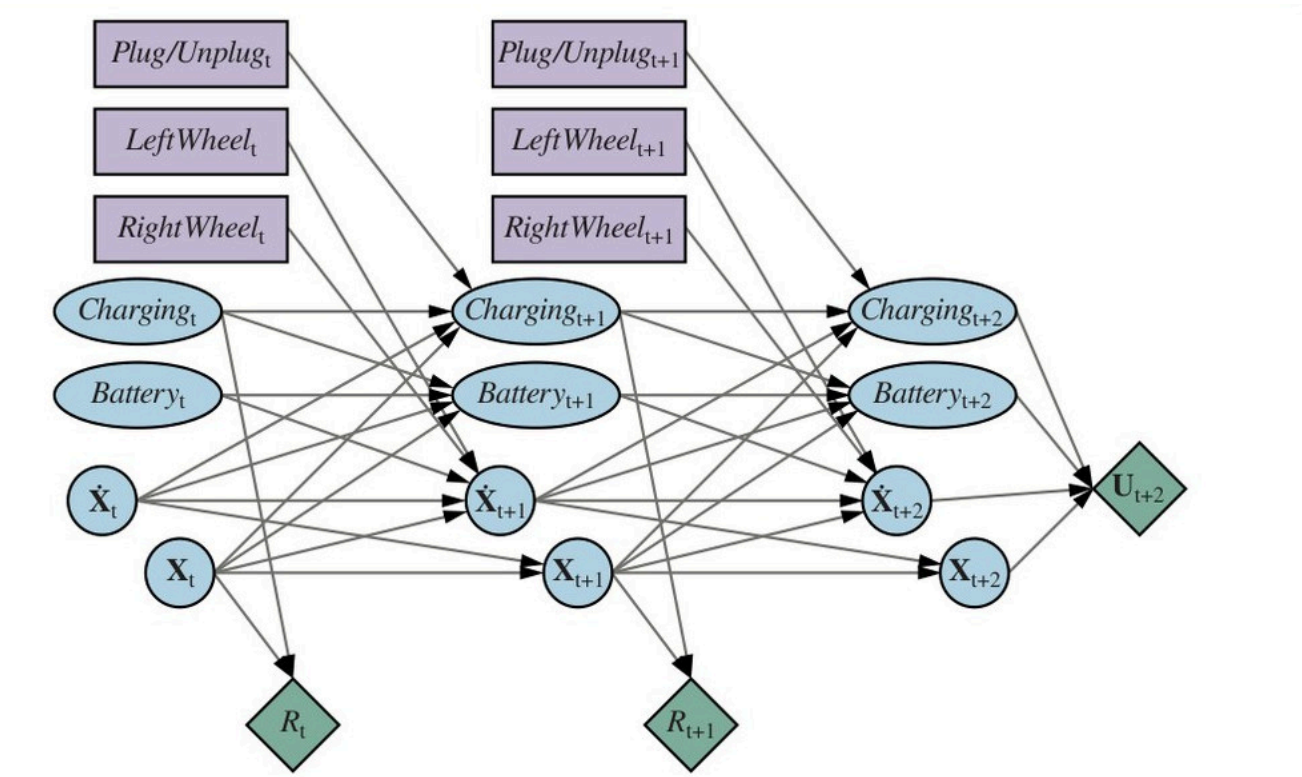
```
                                πG(s) = argmax_a ∑s′ P(s′|s,a) R′(s,a,s′)
argmax_a ∑s′ P(s′|s,a) [R(s,a,s′) + γΦ(s′) − Φ(s)] =
argmax_a ∑s′ P(s′|s,a) [R(s,a,s′) + γU(s′) − U(s)] =
       argmax_a ∑s′ P(s′|s,a) [R(s,a,s′) + γU(s′)] =
                                π*(s) =      (با معادلهٔ (17.4))
```

البته، برای گذاشتنِ Φ(s) = U(s) باید U(s) را می‌دانستیم؛ پس ناهارِ مجانی نیست، اما همچنان ارزشِ قابلِ‌توجهی در تعریفِ تابعِ پاداشی هست که تا آن‌جا که ممکن است کمک‌کننده است. این دقیقاً همان کاری است که مربیانِ حیوانات می‌کنند وقتی برای هر گام در توالیِ هدف، خوراکِ کوچکی به حیوان می‌دهند.

۱۷.۱.۴ بازنماییِ MDPها

ساده‌ترین راهِ بازنماییِ P(s'|s,a) و R(s,a,s') با جدول‌های سه‌بعدیِ بزرگِ اندازهٔ $|S|^2|A|$ است. این برای مسائلِ کوچک مانندِ جهانِ 4×3 خوب است، که برایش جدول‌ها هر یک 11² × 4 = 484 درایه دارند. در برخی موارد، جدول‌ها پراکنده‌اند—بیشترِ درایه‌ها صفرند، زیرا هر حالتِ s فقط می‌تواند به شمارِ کران‌داری از حالت‌های s' انتقال یابد—که بدان معناست جدول‌ها اندازهٔ O(|S||A|) دارند. برای مسائلِ بزرگ‌تر، حتی جدول‌های پراکنده نیز بسیار بزرگ‌اند.

درست همان‌گونه که در فصل ۱۶، جایی که شبکه‌های بیزی با گره‌های کنش و مطلوبیت بسط داده شدند تا شبکه‌های تصمیم بسازند، می‌توانیم MDPها را با بسطِ شبکه‌های بیزیِ پویا (DBNها، فصل ۱۴ را ببینید) با گره‌های تصمیم، پاداش و مطلوبیت بازنمایی کنیم تا «شبکه‌های تصمیمِ پویا» (Dynamic Decision Networks) یا DDN بسازیم. DDNها در اصطلاح‌شناسیِ فصل ۲ بازنمایی‌های عاملی‌اند؛ آن‌ها معمولاً مزیتِ پیچیدگیِ نمایی نسبت به بازنمایی‌های اتمی دارند و می‌توانند مسائلِ دنیایِ واقعیِ کاملاً قابلِ‌توجهی را مدل کنند.



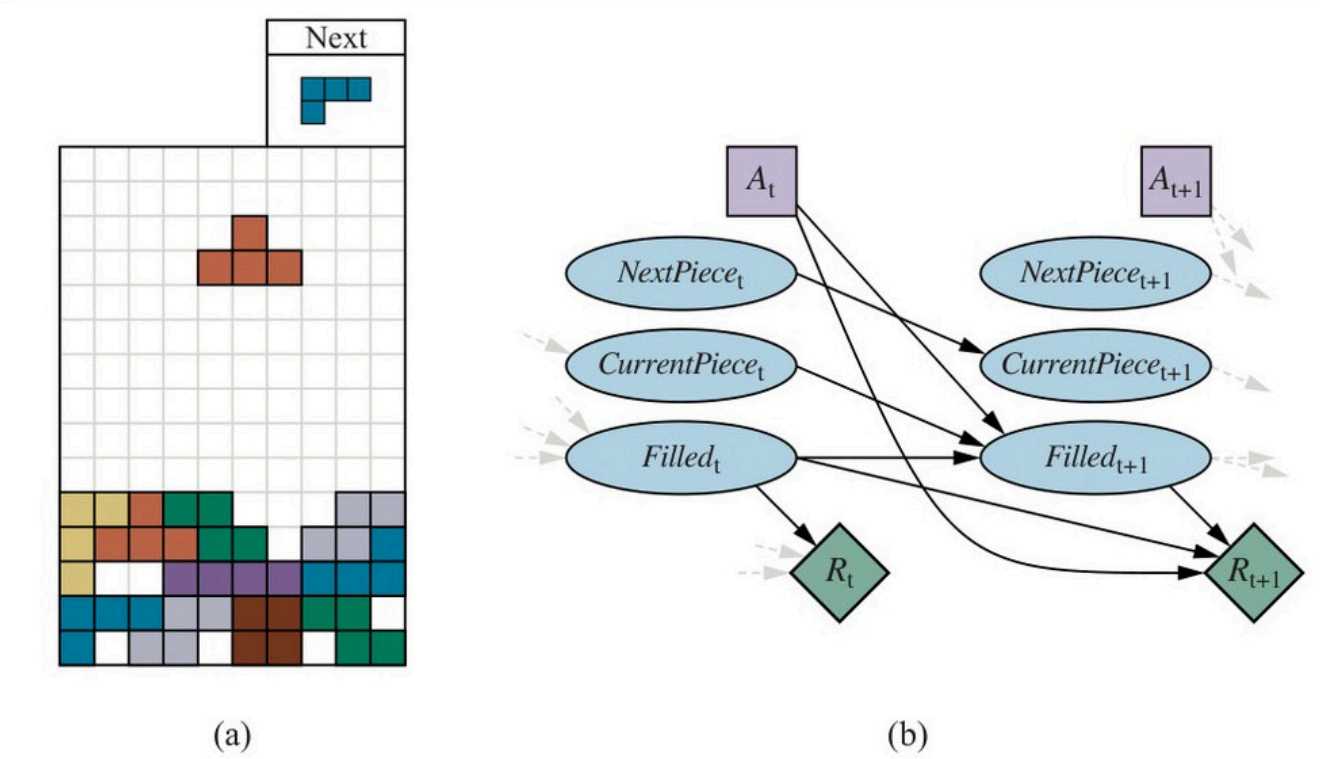
شکل ۱۷.۴ — شبکهٔ تصمیمِ پویا برای رباتِ متحرک با متغیرهای حالت برای سطحِ باتری، وضعیتِ شارژ، مکان و سرعت، و متغیرهای کنش برای موتورهای چرخِ چپ و راست و برای شارژ. (گره‌ها: Ṙ، X، R، Battery، Charging، RightWheel، LeftWheel، Plug/Unplug و U در برش‌های زمانیِ متوالی t، t+1 و t+2.)

شکل ۱۷.۴، که بر DBN شکل ۱۴.۱۳(ب) (صفحهٔ ۴۸۶) مبتنی است، برخی عناصرِ مدلِ نسبتاً واقع‌گرایانه را برای رباتِ متحرکی نشان می‌دهد که می‌تواند خودش را شارژ کند. حالتِ St به چهار متغیرِ حالت تجزیه می‌شود:

- Xt شاملِ مکانِ دوبعدی روی شبکه به‌علاوهٔ جهت‌گیری است؛
- Ẋt نرخِ تغییرِ Xt است؛
- Chargingt وقتی صادق است که ربات به منبعِ برق وصل باشد؛
- Batteryt سطحِ باتری است، که آن را عددِ صحیحی در بازهٔ 0,...,5 مدل می‌کنیم.

فضایِ حالتِ MDP حاصلِ ضربِ دکارتیِ بازه‌های این چهار متغیر است. کنش حالا مجموعه‌ای At از متغیرهای کنش است، شاملِ Plug/Unplug، که سه مقدار دارد (وصل، قطع و بی‌عمل)؛ LeftWheel برای توانِ فرستاده‌شده به چرخِ چپ؛ و RightWheel برای توانِ فرستاده‌شده به چرخِ راست. مجموعهٔ کنش‌های MDP حاصلِ ضربِ دکارتیِ بازه‌های این سه متغیر است. توجه کنید هر متغیرِ کنش فقط بر زیرمجموعه‌ای از متغیرهای حالت اثر می‌گذارد. مدلِ انتقالِ کلی، توزیعِ شرطیِ P(Xt+1|Xt,At) است، که می‌توان آن را به‌صورتِ حاصلِ‌ضربی از احتمال‌های شرطی از DDN محاسبه کرد. پاداشِ این‌جا متغیرِ تکی است که فقط به مکانِ X (برای، مثلاً، رسیدن به مقصد) و شارژ بستگی دارد، زیرا ربات باید هزینهٔ برقِ مصرفی را بپردازد؛ در این مدلِ خاص، پاداش به کنش یا حالتِ پیامد بستگی ندارد.

شبکهٔ شکل ۱۷.۴ برای سه گام در زمان گسترانده (باز) شده است. توجه کنید شبکه شاملِ گره‌هایی برای پاداش‌های t، t+1 و t+2 است، اما مطلوبیت برای t+3. این به این دلیل است که عامل باید جمعِ (تنزیل‌شدهٔ) همهٔ پاداش‌های آینده را بیشینه کند، و U(Xt+3) مطلوبیتِ حاصل از همهٔ پاداش‌ها از t+3 به بعد را بازنمایی می‌کند. اگر تقریبِ اکتشافی‌ای از U در دسترس باشد، می‌توان آن را بدین‌گونه در بازنماییِ MDP گنجاند.



شکل ۱۷.۵ — (الف) بازیِ تتریس. قطعهٔ T-شکلِ بالایِ مرکز را می‌توان در هر جهتی و در هر موقعیتِ افقی‌ای انداخت. اگر ردیفی کامل شود، آن ردیف ناپدید می‌شود و ردیف‌های بالایش پایین می‌آیند، و عامل یک امتیاز می‌گیرد. قطعهٔ بعدی (اینجا، قطعهٔ L-شکلِ بالا-راست) قطعهٔ کنونی می‌شود، و قطعهٔ تازه‌ای ظاهر می‌شود، که به‌صورتِ تصادفی از میانِ هفت نوعِ قطعه انتخاب می‌شود. بازی اگر تخته تا بالا پر شود پایان می‌یابد. (ب) DDN برایِ MDP تتریس.

۱۷.۲ الگوریتم‌های MDPها

در این بخش، چهار الگوریتمِ متفاوت برای حلِ MDPها ارائه می‌کنیم. سه تای نخست، تکرارِ مقدار (Value Iteration)، تکرارِ سیاست (Policy Iteration) و برنامه‌نویسیِ خطی، راه‌حل‌های دقیقِ آفلاین تولید می‌کنند. چهارمی خانواده‌ای از الگوریتم‌های تقریبیِ برخط است که «برنامه‌ریزیِ مونت‌کارلو» (Monte Carlo Planning) را شامل می‌شود.

۱۷.۲.۱ تکرارِ مقدار

معادلهٔ بلمن (معادلهٔ (17.5)) مبنای الگوریتمِ «تکرارِ مقدار» (Value Iteration) برای حلِ MDPهاست. اگر n حالتِ ممکن باشد، n معادلهٔ بلمن هست، یکی برای هر حالت. n معادله شاملِ n مجهول‌اند—مطلوبیت‌های حالت‌ها. پس می‌خواهیم این معادله‌های هم‌زمان را برای یافتنِ مطلوبیت‌ها حل کنیم. یک مسئله هست: معادله‌ها غیرخطی‌اند، زیرا عملگرِ «max» عملگرِ خطی نیست. در حالی که دستگاه‌های معادله‌های خطی را می‌توان با فنونِ جبرِ خطی

به‌سرعت حل کرد، دستگاه‌های معادله‌های غیرخطی مسئله‌دارترند. یکی از کارهایی که می‌توان امتحان کرد رویکردِ تکرارشونده است. با مقادیرِ اولیهٔ دلخواهی برای مطلوبیت‌های حالت‌ها شروع می‌کنیم و «به‌روزرسانیِ بلمنِ» (Bellman Update) را به‌صورتِ بازگشتی اعمال می‌کنیم:

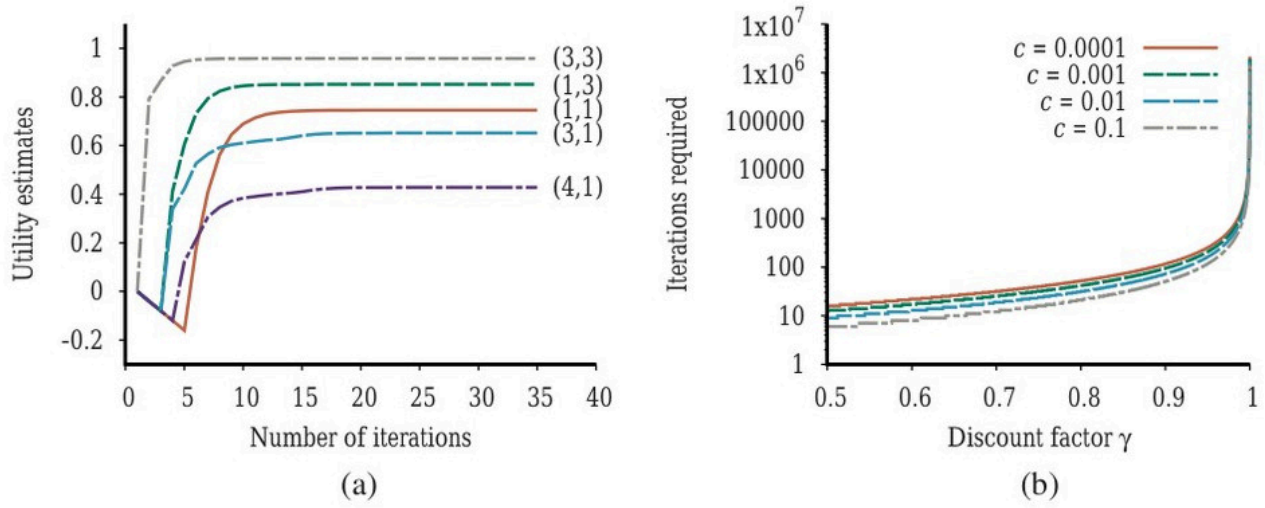
$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma U_i(s')] \qquad (17.10)$$

که در آن فرض می‌شود به‌روزرسانی در هر تکرار هم‌زمان به همهٔ حالت‌ها اعمال می‌شود. اگر به‌روزرسانیِ بلمن را بی‌نهایت بار اعمال کنیم، تضمین‌شده به تعادلی می‌رسیم («همگراییِ تکرارِ مقدار» را در زیر ببینید)، که در آن صورت مقادیرِ نهاییِ مطلوبیت باید راه‌حل‌های معادله‌های بلمن باشند. در واقع، آن‌ها همچنین راه‌حل‌های یکتایند، و سیاستِ متناظر (به‌دست‌آمده با معادلهٔ (17.4)) بهینه است. الگوریتمِ مفصل، شاملِ شرطِ خاتمه وقتی مطلوبیت‌ها «به‌اندازهٔ کافی نزدیک‌اند»، در شکل ۱۷.۶ نشان داده شده.

```
function VALUE-ITERATION(mdp, ϱ) returns a utility function
,inputs: mdp، an MDP with states S, actions A(s), transition model P(s′|s,a)
                                  rewards R(s,a,s′), discount γ
          ϱ، the maximum allowed error in the utility of any state
     local variables: U، U′، utility vectors for states in S, initially zero
            δ، the maximum change in the utility of any state
                                                          :repeat
                                                  δ ← 0 ؛U′ ← U
                                              :for each state s in S
                                  U′[s] ← max_{a∈A(s)} Q-VALUE(mdp, s, a, U)
                              |if |U′[s] − U[s]| > δ then δ ← |U′[s] − U[s]
                                                 until δ ≤ ϱ(1−γ)/γ
                                                          return U
```

شکل ۱۷.۶ — الگوریتمِ تکرارِ مقدار برای محاسبهٔ مطلوبیت‌های حالت‌ها. شرطِ خاتمه از معادلهٔ (17.12) است.

می‌توانیم تکرارِ مقدار را بر جهانِ 4×3 در شکل ۱۷.۱(الف) اعمال کنیم. با شروع از مقادیرِ اولیهٔ صفر، مطلوبیت‌ها چنان تکامل می‌یابند که در شکل ۱۷.۷(الف) نشان داده شده. توجه کنید حالت‌های با فاصله‌های مختلف از (4,3) چگونه پاداشِ منفی جمع می‌کنند تا وقتی مسیری به (4,3) یافت شود، و سپس مطلوبیت‌ها شروع به افزایش می‌کنند. می‌توانیم الگوریتمِ تکرارِ مقدار را به‌عنوانِ انتشارِ اطلاعات در فضایِ حالت از طریقِ به‌روزرسانی‌های محلی بدانیم.



شکل ۱۷.۷ — (الف) نموداری که تکاملِ مطلوبیت‌های حالت‌های برگزیده را با تکرارِ مقدار نشان می‌دهد. (ب) شمارِ تکرارهایِ مقدارِ لازم برای تضمینِ خطایی حداکثر c·Rmax = ϱ، برای مقادیرِ مختلفِ c، به‌عنوانِ تابعی از عاملِ تنزیلِ γ.

همگراییِ تکرارِ مقدار. گفتیم تکرارِ مقدار سرانجام به مجموعهٔ یکتایی از راه‌حل‌های معادله‌های بلمن همگرا می‌شود. در این بخش توضیح می‌دهیم چرا این اتفاق می‌افتد. در طولِ مسیر ایده‌های ریاضیِ مفیدی معرفی می‌کنیم، و روش‌هایی برای ارزیابیِ خطای تابعِ مطلوبیتِ بازگردانده‌شده وقتی الگوریتم زودتر خاتمه می‌یابد به دست می‌آوریم؛ این مفید است، زیرا بدان معناست لازم نیست برای همیشه اجرا کنیم. این بخش نسبتاً فنی است.

مفهومِ پایهٔ استفاده‌شده در نشان‌دادنِ همگراییِ تکرارِ مقدار، مفهومِ «انقباض» (Contraction) است. به‌طورِ تقریبی، انقباض تابعی از یک آرگومان است که، وقتی به‌نوبت روی دو ورودیِ متفاوت اعمال شود، دو مقدارِ خروجی تولید می‌کند که «به هم نزدیک‌ترند»، دستِ‌کم با ضریبِ ثابتی، از ورودی‌های اصلی. برای نمونه، تابعِ «تقسیم بر دو» انقباض است، زیرا، پس از تقسیمِ هر دو عدد بر دو، تفاضل‌شان نصف می‌شود. توجه کنید تابعِ «تقسیم بر دو» نقطهٔ ثابتی دارد، یعنی صفر، که با اعمالِ تابع تغییر نمی‌کند. از این مثال، می‌توانیم دو ویژگیِ مهمِ انقباض‌ها را دریابیم:

- انقباض فقط یک نقطهٔ ثابت دارد؛ اگر دو نقطهٔ ثابت بودند، با اعمالِ تابع به هم نزدیک‌تر نمی‌شدند، پس انقباض نبود.
- وقتی تابع روی هر آرگومانی اعمال می‌شود، مقدار باید به نقطهٔ ثابت نزدیک‌تر شود (زیرا نقطهٔ ثابت حرکت نمی‌کند)، پس اعمالِ مکررِ انقباض همیشه در حدِّ نهایی به نقطهٔ ثابت می‌رسد.

حالا، فرض کنیم به‌روزرسانیِ بلمن (معادلهٔ (17.10)) را به‌عنوانِ عملگرِ B ببینیم که هم‌زمان برای به‌روزرسانیِ مطلوبیتِ هر حالت اعمال می‌شود. سپس معادلهٔ بلمن به U = BU تبدیل می‌شود و معادلهٔ به‌روزرسانیِ بلمن را می‌توان به‌صورتِ BUi ← Ui+1 نوشت. بعد، به راهی برای اندازه‌گیریِ فاصله‌های میانِ بردارهای مطلوبیت نیاز داریم. از «نرمِ بیشینه» (Max Norm) استفاده می‌کنیم، که «طولِ» بردار را با قدرِ مطلقِ بزرگ‌ترین مؤلفه‌اش اندازه می‌گیرد:

$$\|U\| = \max_s |U(s)|$$

با این تعریف، «فاصلهٔ» میانِ دو بردار، ‖U - U'‖، بیشینهٔ تفاوت میانِ هر دو عنصرِ متناظر است. نتیجهٔ اصلیِ این بخش به‌صورتِ زیر است: فرض کنیم Ui و U'i هر دو بردارِ مطلوبیت باشند. سپس داریم:

$$\|BUi - BU'i\| \le \gamma \|Ui - U'i\| \qquad (17.11)$$

یعنی به‌روزرسانیِ بلمن انقباضی با ضریبِ γ روی فضایِ بردارهای مطلوبیت است. (تمرین VICT.۱۷ راهنمایی برای اثباتِ این ادعا فراهم می‌کند.) از این رو، از ویژگی‌های انقباض‌ها به‌طورِ کلی، نتیجه می‌شود تکرارِ مقدار همیشه به راه‌حلِ یکتای معادله‌های بلمن همگرا می‌شود، هرگاه $\gamma < 1$.

همچنین می‌توانیم از ویژگیِ انقباض برای تحلیلِ نرخِ همگرایی به راه‌حل استفاده کنیم. به‌طورِ خاص، می‌توانیم U'i را در معادلهٔ (17.11) با مطلوبیت‌های واقعیِ U جایگزین کنیم، که برایش BU = U. سپس نامساویِ ‖BUi - U‖ ≤ γ‖Ui - U‖ به دست می‌آید. اگر ‖Ui - U‖ را خطایِ برآوردِ Ui ببینیم، می‌بینیم خطا در هر تکرار دست‌کم با ضریبِ γ کاهش می‌یابد. بدین‌ترتیب، تکرارِ مقدار به‌صورتِ نمایی سریع همگرا می‌شود. می‌توانیم شمارِ تکرارهای لازم را چنین محاسبه کنیم: نخست، از معادلهٔ (17.1) به یاد آورید مطلوبیت‌های همهٔ حالت‌ها با ±Rmax/(1-γ) کران‌دارند. این بدان معناست بیشینهٔ خطایِ اولیه ‖U0 - U‖ ≤ 2Rmax/(1-γ). فرض کنیم N تکرار اجرا می‌کنیم تا به خطایی حداکثرِ ϱ برسیم. آن‌گاه، چون خطا هر بار دست‌کم با γ کاهش می‌یابد، لازم داریم:

$$\gamma^N \cdot 2Rmax/(1-\gamma) \le \varrho$$

با گرفتنِ لگاریتم، درمی‌یابیم $N = \lceil \log(2Rmax/(\varrho(1-\gamma)))/\log(1/\gamma) \rceil$ تکرار کافی است. شکل ۱۷.۷(ب) نشان می‌دهد N چگونه با γ تغییر می‌کند، برای مقادیرِ مختلفِ نسبتِ ϱ/Rmax. خبرِ خوب این است که، به دلیلِ همگراییِ نماییِ سریع، N چندان به نسبتِ ϱ/Rmax بستگی ندارد. خبرِ بد این است که N با نزدیک‌شدنِ γ به 1 به‌سرعت رشد می‌کند. اگر γ را کوچک کنیم همگراییِ سریع می‌گیریم، اما این عملاً افقِ کوتاهی به عامل می‌دهد و ممکن است اثرهای بلندمدتِ کنش‌های عامل را از دست بدهد.

کرانِ خطایِ پاراگرافِ پیشین تا حدی ایده‌ای از عواملِ مؤثر بر زمانِ اجرایِ الگوریتم می‌دهد، اما گاهی به‌عنوانِ روشی برای تصمیم‌گیری دربارهٔ زمانِ توقفِ تکرار بیش‌ازحدِ محافظه‌کارانه است. برای هدفِ اخیر، می‌توانیم از کرانی استفاده کنیم که خطا را به اندازهٔ به‌روزرسانیِ بلمن در هر تکرارِ معین مرتبط می‌کند. از ویژگیِ انقباض (معادلهٔ (17.11))، می‌توان نشان داد اگر به‌روزرسانی کوچک باشد (یعنی مطلوبیتِ هیچ حالتی چندان تغییر نکند)، آن‌گاه خطا، در مقایسه با تابعِ مطلوبیتِ واقعی، نیز کوچک است. دقیق‌تر:

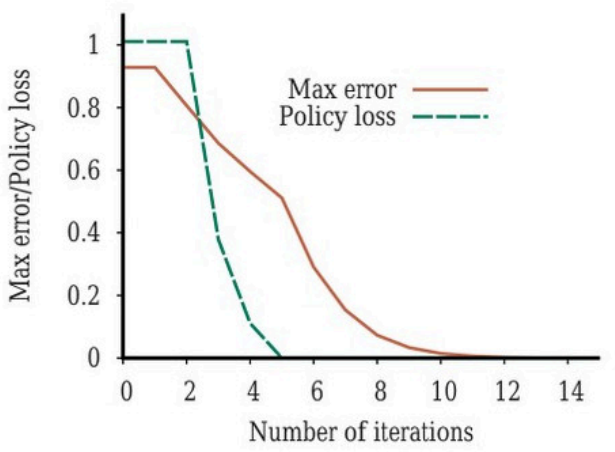
$$\text{اگر } \|Ui+1 - Ui\| < \varrho(1-\gamma)/\gamma \text{ آن‌گاه } \|Ui+1 - U\| < \varrho \qquad (17.12)$$

این شرطِ خاتمه‌ای است که در الگوریتمِ VALUE-ITERATION شکل ۱۷.۶ استفاده شده.

تاکنون خطای تابعِ مطلوبیتِ بازگردانده‌شده توسط الگوریتمِ تکرارِ مقدار را تحلیل کرده‌ایم. اما آنچه عامل واقعاً به آن اهمیت می‌دهد این است که اگر تصمیم‌هایش را بر اساسِ این تابعِ مطلوبیت بگیرد چقدر خوب عمل خواهد کرد. فرض کنیم پس از i تکرارِ تکرارِ مقدار، عامل برآوردِ Ui از مطلوبیتِ واقعیِ U دارد و سیاستِ بیشینهٔ مطلوبیتِ موردانتظار (MEU) به نام πi را بر اساسِ نگاه‌به‌جلوِ تک‌گامی با Ui به دست می‌آورد (مانندِ معادلهٔ (17.4)). آیا رفتارِ حاصل تقریباً به‌اندازهٔ رفتارِ بهینه خوب خواهد بود؟ این پرسشِ حیاتی برای هر عاملِ واقعی است، و معلوم می‌شود پاسخ بله است. Uπi(s) مطلوبیتی است که اگر πi با شروع از s اجرا شود به دست می‌آید، و «زیانِ سیاستِ» (Policy Loss) ‖Uπi - U‖ بیشترین چیزی است که عامل با اجرایِ πi به‌جای سیاستِ بهینهٔ π* می‌تواند از دست بدهد. زیانِ سیاستِ πi با خطایِ Ui با نامساویِ زیر مرتبط است:

```
اگر ‖Ui – U‖ < ε آن‌گاه ‖Uπi – U‖ < 2εγ/(1−γ)        (17.13)
```

در عمل، اغلب رخ می‌دهد πi مدت‌ها قبل از همگراییِ Ui بهینه می‌شود. شکل ۱۷.۸ نشان می‌دهد بیشینهٔ خطا در Ui و زیانِ سیاست چگونه با پیشرویِ فرایندِ تکرارِ مقدار برای محیطِ 4×3 با γ = 0.9 به صفر نزدیک می‌شوند. سیاستِ πi وقتی i = 5 بهینه است، حتی اگرچه بیشینهٔ خطا در Ui هنوز 0.51 است.



شکل ۱۷.۸ — بیشینهٔ خطای ‖Ui – U‖ برآوردهای مطلوبیت و زیانِ سیاستِ ‖Uπi – U‖، به‌عنوانِ تابعی از شمارِ تکرارهای تکرارِ مقدار بر جهانِ 4×3.

حالا هرچه برای استفادهٔ عملی از تکرارِ مقدار نیاز داریم داریم. می‌دانیم به مطلوبیت‌های درست همگرا می‌شود، می‌توانیم خطای برآوردهای مطلوبیت را اگر پس از شمارِ متناهیِ تکرار متوقف شویم کران‌دار کنیم، و می‌توانیم زیانِ سیاستِ ناشی از اجرایِ سیاستِ MEU متناظر را کران‌دار کنیم. به‌عنوانِ یادداشتِ پایانی، همهٔ نتایجِ این بخش به تنزیل با γ < 1 بستگی دارند. اگر γ = 1 و محیط شاملِ حالت‌های پایانی باشد، مجموعهٔ مشابهی از نتایجِ همگرایی و کران‌های خطا قابلِ اشتقاق است.

۱۷.۲.۲ تکرارِ سیاست

در بخشِ قبلی، مشاهده کردیم می‌توان حتی وقتی برآوردِ تابعِ مطلوبیت نادقیق است سیاستِ بهینه به دست آورد. اگر کنشی به‌وضوح از همهٔ کنش‌های دیگر بهتر باشد، آن‌گاه بزرگیِ دقیقِ مطلوبیت‌های حالت‌های درگیر لازم نیست دقیق باشد. این بینش راهِ جایگزینی برای یافتنِ سیاست‌های بهینه پیشنهاد می‌کند. الگوریتمِ «تکرارِ سیاست» (Policy Iteration) دو گامِ زیر را متناوب می‌کند، با شروع از سیاستِ اولیهٔ π0:

- **ارزیابیِ سیاست (Policy Evaluation):** با توجه به سیاستِ πi، Ui = Uπi را محاسبه کن، مطلوبیتِ هر حالت اگر πi اجرا شود.
- **بهبودِ سیاست (Policy Improvement):** سیاستِ MEU تازه‌ای πi+1 را محاسبه کن، با استفاده از نگاه‌به‌جلوِ تک‌گامی بر اساسِ Ui (مانندِ معادلهٔ (17.4)).

الگوریتم وقتی خاتمه می‌یابد که گامِ بهبودِ سیاست هیچ تغییری در سیاست ایجاد نکند. در این نقطه، می‌دانیم تابعِ مطلوبیتِ Ui نقطهٔ ثابتِ به‌روزرسانیِ بلمن است، پس راه‌حلِ معادله‌های بلمن است، و πi باید سیاستِ بهینه باشد. چون برای فضایِ حالتِ متناهی فقط شمارِ متناهی سیاست هست، و می‌توان نشان داد هر تکرار سیاستِ بهتری می‌دهد، تکرارِ سیاست باید خاتمه یابد. الگوریتم در شکل ۱۷.۹ نشان داده شده. مانندِ تکرارِ مقدار، از تابعِ Q-VALUE تعریف‌شده در صفحهٔ ۵۶۹ استفاده می‌کنیم.

```
function POLICY-ITERATION(mdp) returns a policy
inputs: mdp, an MDP with states S, actions A(s), transition model P(s′|s,a)
local variables: U, a utility vector for states in S, initially zero
π, a state-indexed policy vector, initially random
repeat:
U ← POLICY-EVALUATION(π, U, mdp)
unchanged? ← true
for each state s in S:
a* ← argmax_{a∈A(s)} Q-VALUE(mdp, s, a, U)
if Q-VALUE(mdp, s, a*, U) > Q-VALUE(mdp, s, π[s], U)
then π[s] ← a*; unchanged? ← false
until unchanged?
return π
```

شکل ۱۷.۹ — الگوریتمِ تکرارِ سیاست برای محاسبهٔ سیاستِ بهینه.

POLICY-EVALUATION را چگونه پیاده‌سازی کنیم؟ معلوم می‌شود انجامِ این ساده‌تر از حلِ معادله‌های بلمنِ استاندارد است (که تکرارِ مقدار انجام می‌دهد)، زیرا کنش در هر حالت توسط سیاست ثابت شده. در i-امین تکرار، سیاستِ πi کنشِ πi(s) را در حالتِ s مشخص می‌کند. این بدان معناست نسخهٔ ساده‌شده‌ای از معادلهٔ بلمن (17.5) داریم که مطلوبیتِ s (تحتِ πi) را به مطلوبیت‌های همسایه‌هایش مرتبط می‌کند:

$$U_i(s) = \sum_{s'} P(s'|s, \pi_i(s)) \, [R(s, \pi_i(s), s') + \gamma U_i(s')] \qquad (17.14)$$

برای نمونه، فرض کنیم πi سیاستِ نشان‌داده‌شده در شکل ۱۷.۲(الف) باشد. سپس πi(1,1) = Up، πi(1,2) = Up و مانندِ آن داریم، و معادله‌های بلمنِ ساده‌شده عبارت‌اند از:

```
Ui(1,1) = 0.8[-0.04+Ui(1,2)] + 0.1[-0.04+Ui(2,1)] + 0.1[-0.04+Ui(1,1)]
Ui(1,2) = 0.8[-0.04+Ui(1,3)] + 0.2[-0.04+Ui(1,2)]
```

و همین‌طور برای همهٔ حالت‌ها. نکتهٔ مهم این است که این معادله‌ها خطی‌اند، زیرا عملگرِ «max» حذف شده. برای n حالت، n معادلهٔ خطی با n مجهول داریم، که با روش‌های استانداردِ جبرِ خطی در زمانِ $O(n^3)$ دقیقاً حل می‌شوند. اگر مدلِ انتقال پراکنده باشد—یعنی اگر هر حالت فقط به شمارِ کمی از حالت‌های دیگر انتقال یابد—آن‌گاه فرایندِ حل می‌تواند حتی سریع‌تر باشد.

برای فضاهایِ حالتِ کوچک، ارزیابیِ سیاست با روش‌های حلِ دقیق اغلب کارآمدترین رویکرد است. برای فضاهایِ حالتِ بزرگ، زمانِ $O(n^3)$ ممکن است بازدارنده باشد. خوشبختانه، انجامِ ارزیابیِ دقیقِ سیاست ضروری نیست. در عوض، می‌توانیم شماری از گام‌های ساده‌شدهٔ تکرارِ مقدار (ساده‌شده، زیرا سیاست ثابت است) انجام دهیم تا تقریبِ نسبتاً خوبی از مطلوبیت‌ها بدهیم. به‌روزرسانیِ بلمنِ ساده‌شده برای این فرایند:

```
Ui+1(s) ← ∑s′ P(s′|s, πi(s)) [R(s, πi(s), s′) + γUi(s′)]
```

است، و این چند بار تکرار می‌شود تا برآوردِ مطلوبیتِ بعدی به‌صورتِ کارآمد تولید شود. الگوریتمِ حاصل «تکرارِ سیاستِ اصلاح‌شده» (Modified Policy Iteration) نامیده می‌شود.

الگوریتم‌های توصیف‌شده تاکنون نیاز به به‌روزرسانیِ مطلوبیت یا سیاست برای همهٔ حالت‌ها به‌طورِ هم‌زمان دارند. معلوم می‌شود این امر الزاماً ضروری نیست. در واقع، در هر تکرار، می‌توانیم هر زیرمجموعه‌ای از حالت‌ها را برگزینیم و هر نوع به‌روزرسانی (بهبودِ سیاست یا تکرارِ مقدارِ ساده‌شده) را روی آن زیرمجموعه اعمال کنیم. این الگوریتمِ بسیار عمومی «تکرارِ سیاستِ ناهمگام» (Asynchronous Policy Iteration) نامیده می‌شود. با شرایطِ معینی روی سیاستِ اولیه و تابعِ مطلوبیتِ اولیه، تکرارِ سیاستِ ناهمگام تضمین‌شده به سیاستِ بهینه همگرا می‌شود. آزادیِ انتخابِ هر حالت برای کار، بدان معناست می‌توانیم الگوریتم‌های اکتشافیِ بسیار کارآمدتری طراحی کنیم—برای نمونه، الگوریتم‌هایی که بر به‌روزرسانیِ مقادیرِ حالت‌هایی تمرکز می‌کنند که محتمل است توسط سیاستِ خوبی به آن‌ها رسیده شود. برنامه‌ریزی برای نتایجِ کنشی که هرگز انجام نخواهی داد بی‌معنی است.

۱۷.۲.۳ برنامه‌نویسیِ خطی

«برنامه‌نویسیِ خطی» (Linear Programming) یا LP، که در فصل ۴ (صفحهٔ ۱۲۱) به‌طورِ مختصر ذکر شد، رویکردِ عمومی برای صورت‌بندیِ مسائلِ بهینه‌سازیِ قیددار است، و حل‌کننده‌های LP با قدرتِ صنعتیِ بسیاری موجودند. با توجه به اینکه معادله‌های بلمن شاملِ جمع‌ها و maxهای زیادی‌اند، شاید تعجب‌آور نباشد که حلِ MDP را می‌توان به حلِ برنامهٔ خطیِ مناسب‌الصورت‌بندی کاهش داد.

ایدهٔ پایهٔ صورت‌بندی این است که مطلوبیت‌های U(s) هر حالتِ s را به‌عنوانِ متغیر در LP در نظر بگیریم، با یادآوریِ اینکه مطلوبیت‌های سیاستِ بهینه بالاترین مطلوبیت‌های قابلِ‌دستیابیِ سازگار با معادله‌های بلمن‌اند. در زبانِ LP، این بدان معناست می‌خواهیم U(s) را برای همهٔ s کمینه کنیم، مشروط به نامساوی‌های:

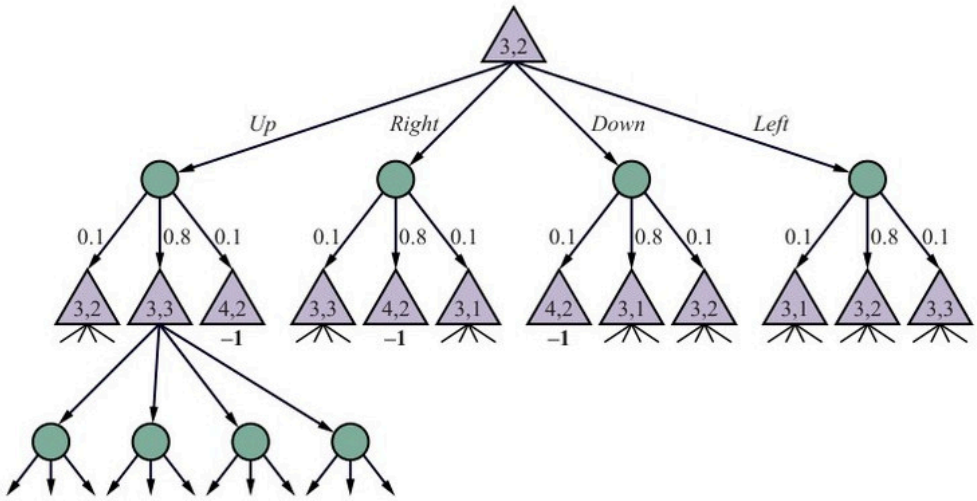
```
U(s) ≥ ∑s′ P(s′|s,a) [R(s,a,s′) + γU(s′)]        برای هر حالتِ s و هر کنشِ a
```

این ارتباطی از برنامه‌نویسیِ پویا به برنامه‌نویسیِ خطی ایجاد می‌کند، که الگوریتم‌ها و مسائلِ پیچیدگی‌اش عمیقاً مطالعه شده‌اند. برای نمونه، از این واقعیت که برنامه‌نویسیِ خطی در زمانِ چندجمله‌ای حل‌شدنی است، می‌توان نشان داد MDPها را می‌توان در زمانِ چندجمله‌ای در شمارِ حالت‌ها و کنش‌ها و شمارِ بیت‌های لازم برای مشخص‌کردنِ مدل حل کرد. در عمل، معلوم می‌شود حل‌کننده‌های LP به‌ندرت به‌اندازهٔ برنامه‌نویسیِ پویا برای حلِ MDPها کارآمدند. علاوه بر این، چندجمله‌ای‌بودن ممکن است خوب به نظر برسد، اما شمارِ حالت‌ها اغلب بسیار بزرگ است. سرانجام، ارزشِ یادآوری دارد حتی ساده‌ترین و بی‌اطلاع‌ترین الگوریتم‌های جستجویِ فصل ۳ در شمارِ حالت‌ها و کنش‌ها در زمانِ خطی اجرا می‌شوند.

۱۷.۲.۴ الگوریتم‌های برخط برای MDPها

تکرارِ مقدار و تکرارِ سیاست الگوریتم‌های آفلاین‌اند: مانندِ الگوریتمِ A* در فصل ۳، راه‌حلِ بهینه‌ای برای مسئله تولید می‌کنند، که سپس عاملِ ساده‌ای می‌تواند اجرایش کند. برای MDPهای به‌اندازهٔ کافی بزرگ، مانندِ MDP تتریس با 10^{60} حالت، حلِ دقیقِ آفلاین، حتی با الگوریتمِ زمانِ چندجمله‌ای، ممکن نیست. چند فن برای حلِ تقریبیِ آفلاینِ MDPها توسعه یافته؛ این‌ها در یادداشت‌های پایانِ فصل و در فصل ۲۲ (یادگیریِ تقویتی) پوشش داده شده‌اند.

در این‌جا الگوریتم‌های برخط را در نظر می‌گیریم، مشابهِ آن‌هایی که برای بازی‌کردن در فصل ۵ استفاده شد، جایی که عامل مقدارِ قابلِ‌توجهی محاسبه در هر نقطهٔ تصمیم انجام می‌دهد، نه اینکه عمدتاً با اطلاعاتِ ازپیش‌محاسبه‌شده عمل کند.



شکل ۱۷.۱۰ – بخشی از درختِ اکسپکتیمکس برای MDP ِ4×3 ریشه‌دار در (3,2). گره‌های مثلثی گره‌های max و گره‌های دایره‌ای گره‌های شانس‌اند. (احتمال‌های انشعاب: 0.1، 0.1، 0.8 برای هر کنش؛ مقصدها شاملِ 3,2، 3,3، 4,2 و 3,1.)

سرراست‌ترین رویکرد در واقع ساده‌سازیِ الگوریتمِ EXPECTIMINIMAX برای بازی‌هایِ دونفره است (بخش ۵.۵ را ببینید). در درختِ اکسپکتیمکس، گره‌های max به حالت‌هایی اشاره دارند که عامل باید کنش انتخاب کند، در حالی که گره‌های شانس به حالت‌هایی اشاره دارند که طبیعت نتیجهٔ کنش را با توجه به مدلِ انتقال انتخاب می‌کند. درختِ MDP ِ4×3 ریشه‌دار در (3,2) در شکل ۱۷.۱۰ نشان داده شده.

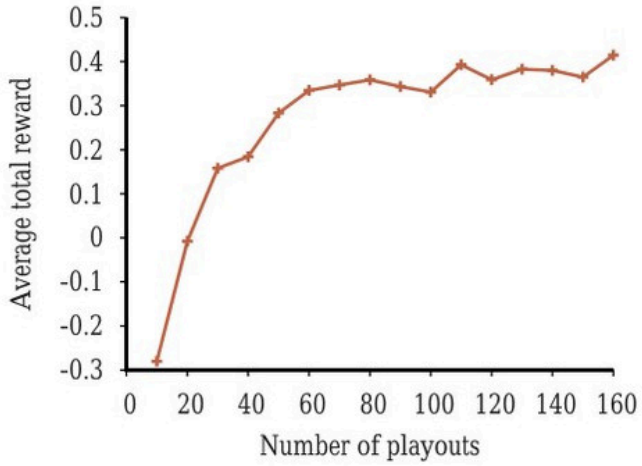
فرض کنیم درختِ اکسپکتیمکس را تا عمقِ ثابتِ H پیاده‌سازی کنیم، یعنی طبقِ انتظار به حالت‌های «جبهه» (Frontier) تا عمقِ H برسیم. همان‌گونه که در موردِ بازی‌ها، ارزیابیِ برگ‌ها توسط یک تابعِ اکتشافیِ ثابت انجام می‌شود، که می‌تواند به‌عنوانِ برآوردِ تابعِ مطلوبیتِ U تفسیر شود، یا می‌توان با نمونه‌برداریِ تصادفی از سیاستِ پیشنهادی، مقدارِ تقریبیِ حالتِ جبهه را برآورد کرد. نمونه‌برداری به‌طورِ طبیعی به تعدادِ بیشتری از پیامدهای محتملِ کنش توجه می‌کند، زیرا آن‌ها محتمل‌ترند تولید شوند.

اگر به درختِ شکل ۱۷.۱۰ با دقت نگاه کنید، چیزی را متوجه می‌شوید: واقعاً درخت نیست. برای نمونه، ریشه (3,2) نیز برگ است، پس باید این را گرافی در نظر گرفت، و باید مقدارِ برگِ (3,2) را مقید به همان مقدارِ ریشهٔ (3,2) کرد، زیرا همان حالت‌اند. در واقع، این خطِ فکری به‌سرعت ما را به معادله‌های بلمن بازمی‌گرداند که مقادیرِ حالت‌ها را به مقادیرِ حالت‌های همسایه مرتبط می‌کنند. حالت‌های کاوش‌شده در واقع زیر-MDP ای از MDP اصلی تشکیل می‌دهند، و این زیر-MDP را می‌توان با هر یک از الگوریتم‌های این فصل برای به‌دست‌آوردنِ تصمیمی برای حالتِ کنونی حل کرد. (حالت‌های جبهه معمولاً مقدارِ برآوردیِ ثابتی می‌گیرند.) این رویکردِ عمومی «برنامه‌نویسیِ پویایِ بلادرنگ» (Real-Time Dynamic Programming) یا RTDP نامیده می‌شود و کاملاً مشابهِ LRTA* در فصل ۴ است. الگوریتم‌های این نوع می‌توانند در حوزه‌های با اندازهٔ متوسط مانندِ جهان‌های شبکه‌ای کاملاً مؤثر باشند؛ در حوزه‌های بزرگ‌تر مانندِ تتریس، دو مسئله هست.

نخست، فضایِ حالت چنان است که هر مجموعهٔ قابلِ‌مدیریتی از حالت‌های کاوش‌شده شاملِ حالت‌های تکراریِ بسیار کمی است، پس به‌همان خوبی می‌توان از درختِ اکسپکتیمکسِ ساده استفاده کرد. دوم، اکتشافیِ سادهٔ گره‌های جبهه ممکن است برای هدایتِ عامل کافی نباشد، به‌طورِ خاص اگر پاداش‌ها پراکنده باشند. یکی از راه‌حل‌های ممکن اعمالِ

یادگیریِ تقویتی برای تولیدِ اکتشافیِ بسیار دقیق‌تر است (فصل ۲۲ را ببینید). رویکردِ دیگر نگاهِ دورتر به جلو در MDP با استفاده از رویکردِ مونت‌کارلویِ بخش ۵.۴ است. در واقع، الگوریتمِ UCT از شکل ۵.۱۰ در اصل برای MDPها توسعه یافته بود، نه بازی‌ها. تغییرهای لازم برای حلِ MDPها به‌جای بازی‌ها کمینه‌اند: آن‌ها عمدتاً از این واقعیت ناشی می‌شوند که حریف (طبیعت) تصادفی است و از نیاز به پیگیریِ پاداش‌ها به‌جای فقط بردها و باخت‌ها.

وقتی روی جهانِ 3×4 اعمال شود، عملکردِ UCT به‌طورِ خاص چشمگیر نیست. چنان‌که شکل ۱۷.۱۱ نشان می‌دهد، به‌طورِ میانگین ۱۶۰ پلی‌اوت (بازیِ شبیه‌سازی‌شده) لازم است تا به پاداشِ کلِ 0.4 برسیم، در حالی که سیاستِ بهینه پاداشِ کلِ موردانتظارِ 0.7453 از حالتِ اولیه دارد (شکل ۱۷.۳ را ببینید). یکی از دلایلی که UCT می‌تواند دچار مشکل شود این است که درخت می‌سازد نه گراف، و از (تقریبی از) اکسپکتیمکس استفاده می‌کند نه برنامه‌نویسیِ پویا. جهانِ 3×4 بسیار «حلقه‌دار» است: اگرچه فقط ۹ حالتِ غیرپایانی هست، پلی‌اوت‌های UCT اغلب بیش از ۵۰ کنش ادامه می‌یابند.



شکل ۱۷.۱۱ – عملکردِ UCT به‌عنوانِ تابعی از شمارِ پلی‌اوت‌هایِ انجام‌شده در جهانِ 3×4. (میانگینِ پاداشِ کلِ حاصل از حالتِ آغازین بر حسبِ شمارِ پلی‌اوت‌ها.)

UCT برای تتریس مناسب‌تر به نظر می‌رسد، جایی که پلی‌اوت‌ها به‌اندازهٔ کافی به آینده می‌روند تا به عامل حسِ اینکه آیا حرکتِ بالقوهٔ پرریسکی در پایان جواب می‌دهد یا به انباشتِ عظیمی می‌انجامد بدهند. تمرین UCTT.۱۷ کاربردِ UCT را در تتریس کاوش می‌کند. یکی از پرسش‌های خاصاً جالب این است که سیاستِ شبیه‌سازیِ ساده‌ای چقدر می‌تواند کمک کند—برای نمونه، سیاستی که از ایجادِ برآمدگی‌ها اجتناب می‌کند و قطعه‌ها را تا آن‌جا که ممکن است پایین می‌گذارد.

۱۷.۳ مسائلِ باندیت

در لاس‌وگاس، «باندیتِ یک‌بازویی» (One-Armed Bandit) ماشینِ اسلات است. قمارباز می‌تواند سکه‌ای بیندازد، اهرم را بکشد و بردها را (اگر باشد) جمع کند. باندیتِ n-بازویی n اهرم دارد. پشتِ هر اهرم توزیعِ احتمالِ ثابت اما ناشناخته‌ای از بردهاست؛ هر کشش از آن توزیعِ ناشناخته نمونه‌برداری می‌کند. قمارباز باید انتخاب کند با هر سکهٔ

متوالی کدام اهرم را بازی کند—اهرمی که بهترین بازدهی را داشته، یا شاید اهرمی که هنوز امتحان نشده؟ این نمونه‌ای از معاوضهٔ فراگیرِ میانِ «بهره‌برداری» (Exploitation) از بهترین کنشِ کنونی برای به‌دست‌آوردنِ پاداش‌ها و «کاوش» (Exploration) حالت‌ها و کنش‌های قبلاً ناشناخته برای کسبِ اطلاعات است، که در برخی موارد می‌تواند به سیاستِ بهتر و پاداش‌های بلندمدتِ بهتر تبدیل شود. در دنیای واقعی، مدام باید بینِ ادامهٔ وجودِ راحت، در برابرِ زدن به ناشناخته‌ها به امیدِ زندگیِ بهتر تصمیم گرفت.

مسئلهٔ باندیتِ n-بازویی مدلِ صوریِ مسائلِ واقعی در حوزه‌های حیاتیِ بسیاری است، مانندِ تصمیم‌گیری دربارهٔ اینکه کدام‌یک از n درمانِ ممکنِ تازه برای درمانِ بیماری امتحان شود، کدام‌یک از n سرمایه‌گذاریِ ممکن بخشی از پس‌اندازتان در آن گذاشته شود، کدام‌یک از n پروژهٔ پژوهشیِ ممکن تأمینِ مالی شود، یا کدام‌یک از n تبلیغِ ممکن هنگام بازدیدِ کاربر از صفحهٔ وبِ خاصی نشان داده شود. کارهای اولیه روی مسئله در آمریکا در طولِ جنگِ جهانیِ دوم آغاز شد؛ آن‌قدر سرسخت از آب درآمد که دانشمندانِ متفقین پیشنهاد کردند «مسئله روی آلمان رها شود، به‌عنوانِ ابزارِ نهاییِ خرابکاریِ فکری» (ویتل، 1979).

معلوم می‌شود دانشمندان، هم در طولِ جنگ و هم پس از آن، سعی می‌کردند واقعیت‌های «بدیهیاً درستِ» دربارهٔ مسائلِ باندیت را اثبات کنند که در واقع کاذب‌اند. (چنان‌که برادت و همکاران (1956) گفتند، «ویژگی‌های زیبای بسیاری هست که استراتژی‌های بهینه از آن برخوردار نیستند.») برای نمونه، به‌طورِ کلی فرض می‌شد سیاستِ بهینه در درازمدت سرانجام روی بهترین اهرم می‌ماند؛ در واقع، احتمالِ متناهی‌ای هست که سیاستِ بهینه روی اهرمِ زیربهینه بماند. اکنون درکِ نظریِ محکمی از مسائلِ باندیت و نیز الگوریتم‌های مفیدی برای حلِ آن‌ها داریم. تعریف‌های مختلفی از مسائلِ باندیت هست؛ یکی از تمیزترین و عمومی‌ترین‌ها به‌صورتِ زیر است:

- هر اهرمِ Mi یک «فرایندِ پاداشِ مارکوف» (Markov Reward Process) یا MRP است، یعنی MDP با فقط یک کنشِ ممکنِ ai. حالت‌های Si، مدلِ انتقالِ Pi(s'|s,ai) و پاداشِ Ri(s,ai,s') دارد. اهرم توزیعی را روی توالی‌های پاداشِ Ri,0, Ri,1, Ri,2,... تعریف می‌کند، که در آن هر Ri,t متغیرِ تصادفی است.
- مسئلهٔ کلیِ باندیت MDP است: فضایِ حالت با حاصلِ‌ضربِ دکارتیِ S = S1 × ... × Sn داده می‌شود؛ کنش‌ها a1,...,an هستند؛ مدلِ انتقال، حالتِ هر اهرمِ Mi که انتخاب شود را طبقِ مدلِ انتقالِ خاصش به‌روزرسانی می‌کند و سایر اهرم‌ها را بدونِ تغییر می‌گذارد؛ و عاملِ تنزیل γ است.

این تعریف بسیار عمومی است و گسترهٔ وسیعی از موارد را پوشش می‌دهد. ویژگیِ کلیدی این است که اهرم‌ها مستقل‌اند، فقط با این واقعیت که عامل می‌تواند هم‌زمان فقط روی یک اهرم کار کند جفت‌شده‌اند. تعریفِ حتی عمومی‌تری ممکن است که در آن تلاش‌های کسری می‌توانند هم‌زمان به همهٔ اهرم‌ها اعمال شوند، اما کلِ تلاشِ روی همهٔ اهرم‌ها کران‌دار است؛ نتایجِ پایهٔ توصیف‌شده در این‌جا به این حالت نیز منتقل می‌شوند.

به‌زودی خواهیم دید چگونه مسئلهٔ باندیتِ نمونه‌ای را در این چارچوب صورت‌بندی کنیم، اما بیایید با حالتِ خاصِ سادهٔ توالی‌های پاداشِ قطعی گرم کنیم. فرض کنیم γ = 0.5، و دو اهرم با برچسبِ M و M1 باشد. کشیدنِ مکررِ M توالیِ پاداشِ 0,0,2,0,7.2,0,0,... را می‌دهد، در حالی که کشیدنِ M1 به 1,1,1,... می‌انجامد (شکل ۱۷.۱۲(الف)).

اگر، در آغاز، باید به یکی از اهرم‌ها متعهد می‌شدیم و به آن می‌چسبیدیم، انتخاب با محاسبهٔ مطلوبیت (پاداشِ کلِ تنزیل‌شده) برای هر اهرم انجام می‌شد:

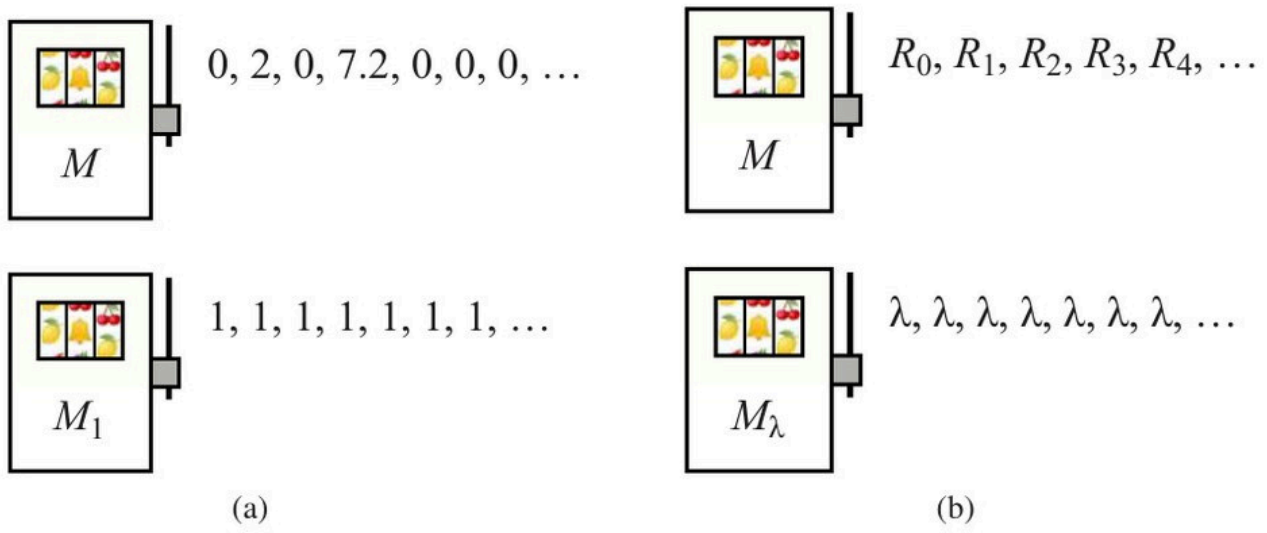
$$U(M) = (1.0\times0) + (0.5\times2) + (0.5^2\times0) + (0.5^3\times7.2) = 1.9$$
$$U(M1) = \sum_{t=0}^{\infty} 0.5^t = 2.0$$

ممکن است فکر کنیم بهترین انتخاب رفتن با M1 است، اما یک لحظه فکرِ بیشتر نشان می‌دهد شروع با M و سپس سوئیچ به M1 پس از پاداشِ چهارم، توالیِ S = 0,2,0,7.2,1,1,1,... را می‌دهد، که برایش:

$$U(S) = (1.0\times0) + (0.5\times2) + (0.5^2\times0) + (0.5^3\times7.2) + \sum_{t=4}^{\infty} 0.5^t = 2.025$$

از این رو، استراتژیِ S که در زمانِ درست از M به M1 سوئیچ می‌کند، از هر یک از اهرم‌ها به‌تنهایی بهتر است. در واقع، S برای این مسئله بهینه است: همهٔ زمان‌های سوئیچِ دیگر پاداشِ کمتری می‌دهند.

بیایید این حالت را کمی تعمیم دهیم، به‌گونه‌ای که حالا اهرمِ نخستِ M توالیِ دلخواهِ R0,R1,R2,... را بدهد (که ممکن است معلوم یا نامعلوم باشد) و اهرمِ دومِ Mλ برای ثابتِ معلومِ λ توالیِ λ,λ,λ,... را بدهد (شکل ۱۷.۱۲(ب) را ببینید). این در ادبیات «باندیتِ یک‌بازویی» (One-Armed Bandit) نامیده می‌شود، زیرا به‌صورتِ صوری معادلِ حالتی است که یک اهرمِ M پاداش‌های R0,R1,R2,... تولید می‌کند و برای هر کشش هزینهٔ λ دارد. (کشیدنِ اهرمِ M معادلِ نکشیدنِ Mλ است، پس هر بار پاداشِ λ را از دست می‌دهد.) با فقط یک اهرم، تنها انتخاب این است که دوباره بکشیم یا متوقف شویم. اگر اهرمِ نخست را T بار بکشید (یعنی در زمان‌های 0,1,...,T-1)، می‌گوییم «زمانِ توقف» (Stopping Time) برابر T است.



شکل ۱۷.۱۲ – (الف) مسئلهٔ باندیتِ قطعیِ ساده با دو اهرم. اهرم‌ها را می‌توان به هر ترتیبی کشید، و هر یک توالیِ پاداشِ نشان‌داده‌شده را می‌دهد. (ب) حالتِ عمومی‌ترِ باندیتِ (الف)، که در آن اهرمِ نخست توالیِ دلخواهی از پاداش‌ها و اهرمِ دوم پاداشِ ثابتِ λ می‌دهد.

با بازگشت به نسخه‌مان با M و Mλ، فرض کنیم پس از T کششِ اهرمِ نخست، استراتژیِ بهینه سرانجام برای نخستین بار اهرمِ دوم را بکشد. چون از این حرکت اطلاعاتی به دست نمی‌آید (از قبل می‌دانیم بازدهی λ خواهد بود)، در زمانِ T+1 در همان وضعیت خواهیم بود و بنابراین استراتژیِ بهینه باید همان انتخاب را بکند. به‌طورِ معادل، می‌توانیم بگوییم استراتژیِ بهینه این است که اهرمِ M را تا زمانِ T اجرا کند و سپس برای بقیهٔ زمان به Mλ سوئیچ کند. ممکن است T = 0 باشد اگر استراتژی بلافاصله Mλ را انتخاب کند، یا T = ∞ اگر استراتژی هرگز Mλ را انتخاب نکند، یا جایی در بین. حالا مقدارِ λ را در نظر بگیریم به‌گونه‌ای که استراتژیِ بهینه دقیقاً بی‌تفاوت باشد میانِ (الف) اجرایِ M تا بهترین زمانِ توقفِ ممکن و سپس سوئیچِ همیشگی به Mλ، و (ب) انتخابِ بلافاصلهٔ Mλ. در این نقطهٔ بی‌تفاوتی (سربه‌سر) داریم:

```
max_{T>0} E[∑_{t=0}^{T−1} γᵗRt] + ∑_{t=T}^{∞} γᵗλ = ∑_{t=0}^{∞} γᵗλ
```

که به‌صورتِ زیر ساده می‌شود:

```
λ = max_{T>0} E[∑_{t=0}^{T−1} γᵗRt] / E[∑_{t=0}^{T−1} γᵗ]        (17.15)
```

این معادله نوعی «ارزش» برای M بر حسبِ توانایی‌اش در تحویلِ جریانی از پاداش‌های به‌موقع تعریف می‌کند؛ شمارندهٔ کسر مطلوبیتی را بازنمایی می‌کند، در حالی که مخرج را می‌توان «زمانِ تنزیل‌شده» دانست، پس ارزش، حداکثرِ مطلوبیتِ قابلِ‌حصول به‌ازای هر واحدِ زمانِ تنزیل‌شده را توصیف می‌کند. (مهم است به یاد داشته باشیم T در معادله زمانِ توقف است، که با قاعده‌ای برای توقف اداره می‌شود نه عددِ صحیحِ ساده؛ فقط وقتی M توالیِ پاداشِ قطعی باشد به عددِ صحیحِ ساده کاهش می‌یابد.) ارزشِ تعریف‌شده در معادلهٔ (17.15) «شاخصِ گیتینز» (Gittins Index) M نامیده می‌شود.

نکتهٔ قابلِ‌توجهِ شاخصِ گیتینز این است که سیاستِ بهینهٔ بسیار ساده‌ای برای هر مسئلهٔ باندیتی فراهم می‌کند: اهرمی را بکش که بالاترین شاخصِ گیتینز را دارد، سپس شاخص‌های گیتینز را به‌روزرسانی کن. علاوه بر این، چون شاخصِ اهرمِ Mi فقط به ویژگی‌های آن اهرم بستگی دارد، تصمیمِ بهینه در نخستین تکرار را می‌توان در زمانِ O(n) محاسبه کرد، که در آن n شمارِ اهرم‌هاست. و چون شاخص‌های گیتینزِ اهرم‌های انتخاب‌نشده بدونِ تغییر می‌مانند، هر تصمیمِ پس از نخستین را می‌توان در زمانِ O(1) محاسبه کرد.

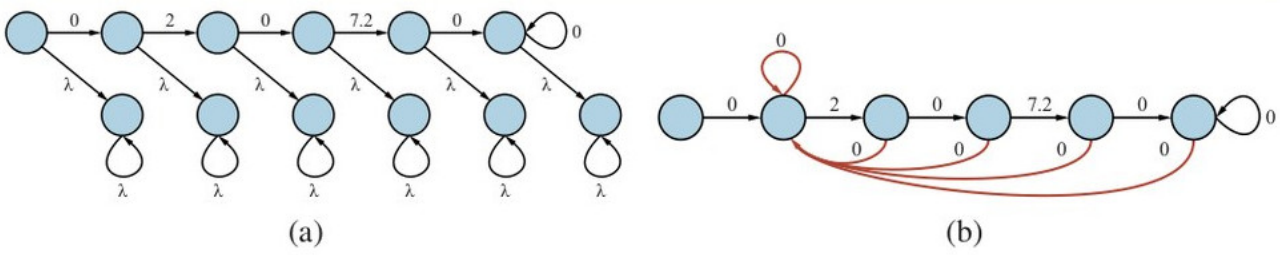
۱۷.۳.۱ محاسبهٔ شاخصِ گیتینز

برای حسِ بیشترِ شاخص، مقدارِ شمارنده، مخرج و نسبتِ معادلهٔ (17.15) را برای زمان‌های توقفِ مختلف روی توالیِ پاداشِ قطعیِ 0,0,0,7.2,0,2,0,... محاسبه کنیم:

T	1	2	3	4	5	6
Rt	0	2	0	7.2	0	0

T	1	2	3	4	5	6
$\sum \gamma^t R_t$	0.0	1.0	1.0	1.9	1.9	1.9
$\sum \gamma^t$	1.0	1.5	1.75	1.875	1.9375	1.9687
نسبت	0.0	0.6667	0.5714	1.0133	0.9806	0.9651

واضح است که نسبت از این پس کاهش خواهد یافت، زیرا شمارنده ثابت می‌ماند در حالی که مخرج همچنان افزایش می‌یابد. بدین‌ترتیب، شاخصِ گیتینز برای این اهرم 1.0133 است، بیشینهٔ مقدارِ به‌دست‌آمده توسط نسبت. در ترکیب با اهرمِ ثابتِ M_λ با $0 < \lambda \leq 1.0133$، سیاستِ بهینه چهار پاداشِ نخست را از M دریافت می‌کند و سپس به M_λ سوئیچ می‌کند. برای $\lambda > 1.0133$، سیاستِ بهینه همیشه M_λ را انتخاب می‌کند.

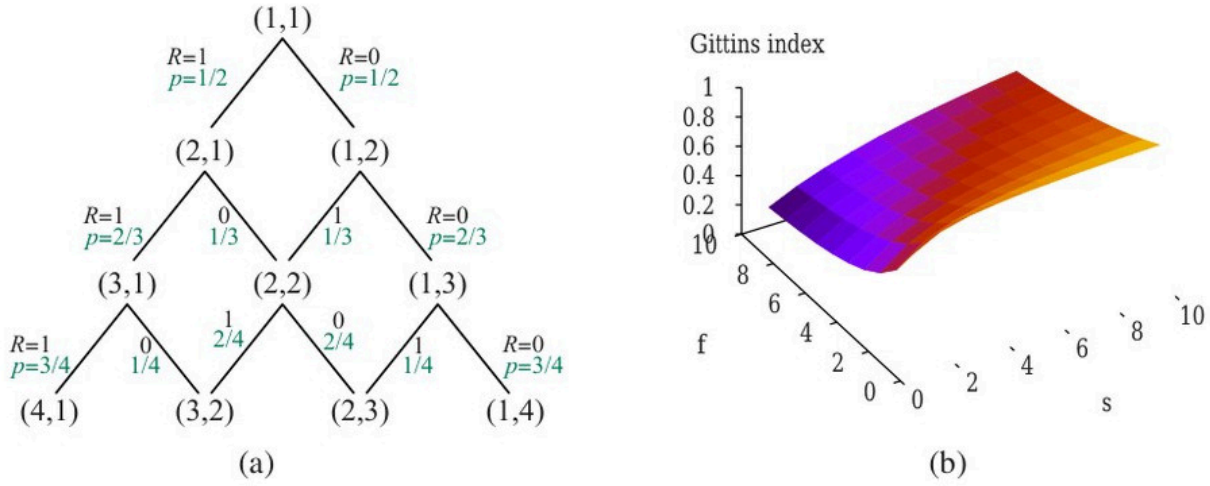


شکل ۱۷.۱۳ – (الف) توالیِ پاداشِ M = 0,2,0,7.2,0,0,0,... تقویت‌شده با انتخابی برای سوئیچِ دائمی به اهرمِ ثابتِ M_λ در هر نقطه. (ب) MDP ای که مقدارِ بهینه‌اش دقیقاً معادلِ مقدارِ بهینه برای (الف) است، در نقطه‌ای که سیاستِ بهینه میانِ M و M_λ بی‌تفاوت است.

برای محاسبهٔ شاخصِ گیتینز برای اهرمِ عمومیِ M با حالتِ کنونیِ s، صرفاً مشاهدهٔ زیر را انجام می‌دهیم: در نقطهٔ بی‌تفاوتی، جایی که سیاستِ بهینه میانِ انتخابِ اهرمِ M و انتخابِ اهرمِ ثابتِ M_λ بی‌تفاوت است، ارزشِ انتخابِ M همان ارزشِ انتخابِ توالیِ نامتناهیِ پاداش‌های λ است. فرض کنیم M را تقویت کنیم به‌گونه‌ای که در هر حالتِ M، عامل دو انتخاب داشته باشد: یا مثلِ قبل با M ادامه دهد، یا کنار بکشد و توالیِ نامتناهیِ پاداش‌های λ دریافت کند (شکل ۱۷.۱۳(الف) را ببینید). این M را به MDP تبدیل می‌کند، که سیاستِ بهینه‌اش صرفاً قاعدهٔ توقفِ بهینه برای M است. از این رو، ارزشِ سیاستِ بهینه در این MDP تازه برابرِ ارزشِ توالیِ نامتناهیِ پاداش‌های λ است، یعنی $\lambda/(1-\gamma)$. پس می‌توانیم این MDP را حل کنیم... اما، متأسفانه، مقدارِ λ را برای گذاشتن در MDP نمی‌دانیم، زیرا این دقیقاً همان چیزی است که می‌خواهیم محاسبه کنیم. اما می‌دانیم در نقطهٔ بی‌تفاوتی، سیاستِ بهینه میانِ M و M_λ بی‌تفاوت است، پس می‌توانیم انتخابِ دریافتِ توالیِ نامتناهیِ پاداش‌های λ را با انتخابِ بازگشت و راه‌اندازیِ دوبارهٔ M از حالتِ اولیه‌اش s جایگزین کنیم. (دقیق‌تر، در هر حالت کنشِ تازه‌ای می‌افزاییم که همان پاداش‌ها و پیامدهای کنشِ موجود در s را دارد؛ تمرین KATV.۱۷ را ببینید.) این MDP تازهٔ «MDP راه‌اندازیِ دوباره»، M_s، (Restart MDP) نامیده می‌شود و در شکل ۱۷.۱۳(ب) نشان داده شده. نتیجهٔ عمومی داریم که شاخصِ گیتینز برای اهرمِ M در حالتِ s برابرِ $1-\gamma$ ضرب‌درِ ارزشِ سیاستِ بهینه برای MDP راه‌اندازیِ دوبارهٔ M_s است. این MDP را می‌توان با هر یک از الگوریتم‌های بخش ۱۷.۲ حل کرد. تکرارِ مقدارِ اعمال‌شده بر M_s در شکل ۱۷.۱۳(ب) برای حالتِ شروع مقدارِ 2.0266 می‌دهد، پس $\lambda = 1.0133 = (1-\gamma)\cdot 2.0266$ داریم، همان‌گونه که قبلاً.

۱۷.۳.۲ باندیتِ برنولی

شاید ساده‌ترین و شناخته‌شده‌ترین نمونهٔ مسئلهٔ باندیت «باندیتِ برنولی» (Bernoulli Bandit) باشد، که در آن هر اهرمِ Mi با احتمالِ ثابت اما ناشناختهٔ μi پاداشِ 0 یا 1 تولید می‌کند. حالتِ اهرمِ Mi با si و fi، شمارِ موفقیت‌ها (1ها) و شکست‌ها (0ها) تاکنون برای آن اهرم، تعریف می‌شود؛ احتمالِ انتقال، پیامدِ بعدی را با احتمالِ (si)/(si + fi) برابرِ 1 و با احتمالِ (fi)/(si + fi) برابرِ 0 پیش‌بینی می‌کند. شمارش‌ها به 1 مقداردهی می‌شوند تا احتمال‌های اولیه 1/2 باشند نه 0/0.[4] فرایندِ پاداشِ مارکوف در شکل ۱۷.۱۴(الف) نشان داده شده. احتمال‌ها احتمال‌های فرایندِ به‌روزرسانیِ بیزی با پیشینِ Beta(1,1) هستند (بخش ۲۰.۲.۵ را ببینید).



شکل ۱۷.۱۴ – (الف) حالت‌ها، پاداش‌ها و احتمال‌های انتقال برای باندیتِ برنولی. (هر حالت با جفتِ (s,f) برچسب‌گذاری شده؛ پاداشِ 1 با احتمالِ s/(s+f) و پاداشِ 0 با احتمالِ f/(s+f).) (ب) شاخص‌های گیتینز برای حالت‌های فرایندِ باندیتِ برنولی.

نمی‌توانیم کاملاً تبدیلِ بخشِ قبلی را برای محاسبهٔ شاخصِ گیتینزِ اهرمِ برنولی اعمال کنیم، زیرا بی‌نهایت حالت دارد. اما می‌توانیم با حلِ MDP بُریده‌شده با حالت‌های تا si + fi = 100 و γ = 0.9 تقریبِ بسیار دقیقی به دست آوریم. نتایج در شکل ۱۷.۱۴(ب) نشان داده شده‌اند. نتایج به‌صورتِ شهودی معقول‌اند: می‌بینیم، به‌طورِ کلی، اهرم‌های با احتمال‌های بازدهیِ بالاتر ترجیح داده می‌شوند، اما همچنین «پاداشِ کاوش» (Exploration Bonus) مرتبط با اهرم‌هایی هست که فقط چند بار امتحان شده‌اند. برای نمونه، شاخصِ حالتِ (3,2) بالاتر از شاخصِ حالتِ (7,4) است (0.7057 در برابرِ 0.6922)، حتی اگرچه مقدارِ برآوردشده در (3,2) پایین‌تر است (0.6 در برابرِ 0.6364).

۱۷.۳.۳ سیاست‌های باندیتِ تقریباً بهینه

محاسبهٔ شاخص‌های گیتینز برای مسائلِ واقع‌گرایانه‌تر به‌ندرت آسان است. خوشبختانه، ویژگی‌های عمومیِ مشاهده‌شده در بخشِ قبلی—یعنی مطلوبیتِ ترکیبی از مقدارِ برآوردشده و عدم قطعیت—خود را به ایجادِ سیاست‌های ساده‌ای می‌دهند که معلوم می‌شود «تقریباً به‌اندازهٔ» سیاست‌های بهینه خوب‌اند.

نخستین رده از روش‌ها از «کرانِ بالایِ اطمینان» (Upper Confidence Bound) یا اکتشافیِ UCB استفاده می‌کند، که قبلاً برای جستجوی درختیِ مونت‌کارلو معرفی شد (شکل ۵.۱۱ صفحهٔ ۱۶۳). ایدهٔ پایه استفاده از نمونه‌های هر اهرم برای برقراریِ بازهٔ اطمینانِ مقدارِ اهرم است، یعنی بازه‌ای که مقدار با اطمینانِ بالا می‌تواند در آن تخمین زده شود؛ سپس اهرمی را انتخاب کن که بالاترین کرانِ بالای بازهٔ اطمینانش را دارد. کرانِ بالا، برآوردِ میانگینِ کنونیِ $\hat{\mu}i$ به‌علاوهٔ چند برابرِ انحرافِ معیارِ عدم قطعیتِ مقدار است. انحرافِ معیار با $\sqrt{(1/Ni)}$ متناسب است، که در آن Ni شمارِ دفعاتِ نمونه‌برداری از اهرمِ Mi است. پس مقدارِ شاخصیِ تقریبی برای اهرمِ Mi داریم:

```
UCB(Mi) = μ̂i + g(N)/√Ni
```

که در آن g(N) تابعِ مناسب‌الانتخابی از N، شمارِ کلِ نمونه‌های کشیده‌شده از همهٔ اهرم‌هاست. سیاستِ UCB صرفاً اهرمی را با بالاترین مقدارِ UCB برمی‌گزیند. توجه کنید مقدارِ UCB در معنایِ دقیق کلمه یک شاخص نیست، زیرا به N، شمارِ کلِ نمونه‌های کشیده‌شده روی همهٔ اهرم‌ها، بستگی دارد، نه فقط به خودِ اهرم.

تعریفِ دقیقِ g «پشیمانیِ» (Regret) نسبت به سیاستِ دانای‌کل (Clairvoyant) را تعیین می‌کند، که صرفاً بهترین اهرم را برمی‌گزیند و پاداشِ میانگینِ μ^* را می‌دهد. نتیجهٔ مشهوری از لای و رابینز (1985) نشان می‌دهد، برای حالتِ بدونِ تنزیل، هیچ الگوریتمِ ممکنی نمی‌تواند پشیمانی‌ای داشته باشد که کندتر از O(logN) رشد کند. چند انتخابِ مختلفِ g به سیاستِ UCBای می‌انجامند که با این رشد هم‌خوان است؛ برای نمونه، می‌توانیم از $g(N) = (2\log(1+N\cdot\log^2 N))^{1/2}$ استفاده کنیم.

روشِ دوم، «نمونه‌برداریِ تامسون» (Thompson Sampling) (تامسون، 1933)، اهرمی را به‌صورتِ تصادفی طبقِ احتمالِ اینکه اهرم در واقع بهینه است، با توجه به نمونه‌های تاکنون، انتخاب می‌کند. فرض کنیم Pi(μi) توزیعِ احتمالِ کنونیِ مقدارِ واقعیِ اهرمِ Mi باشد. سپس راهِ ساده‌ای برای پیاده‌سازیِ نمونه‌برداریِ تامسون این است که یک نمونه از هر Pi تولید کنیم، اهرمی را با بالاترین نمونه انتخاب کنیم، سپس با مشاهدهٔ پاداشِ واقعی، توزیعِ آن اهرم را به‌روزرسانی کنیم. برای اهرم‌های برنولی با توزیعِ Beta، به‌روزرسانی‌ها بسیار ساده‌اند: اگر پاداش 1 بود، پارامترِ موفقیت را افزایش بده؛ اگر 0 بود، پارامترِ شکست را. این روش‌ها به‌خوبی کار می‌کنند و به‌طورِ گسترده در صنعت استفاده می‌شوند؛ مزیتِ عمده‌شان نسبت به شاخصِ گیتینز سادگیِ‌شان است.

۱۷.۳.۴ گونه‌های غیرشاخصی

مسائلِ باندیت تا حدی با تکلیفِ آزمایشِ درمان‌های پزشکیِ تازه روی بیمارانِ به‌شدت بیمار انگیزه‌مند شدند. برای این تکلیف، هدفِ بیشینه‌کردنِ شمارِ کلِ موفقیت‌ها در طولِ زمان به‌وضوح معنا دارد: هر آزمایشِ موفق یعنی زندگیِ نجات‌یافته، هر شکست یعنی زندگیِ ازدست‌رفته. اما اگر مفروضات را کمی تغییر دهیم، مسئلهٔ متفاوتی پدید می‌آید. فرض کنیم، به‌جای تعیینِ بهترین درمانِ پزشکی برای هر بیمارِ انسانیِ تازه، در عوض داروهای متفاوتی را روی نمونه‌هایی از باکتری آزمایش می‌کنیم با هدفِ تصمیم‌گیری دربارهٔ اینکه کدام دارو بهترین است. سپس آن دارو را واردِ

تولید می‌کنیم و بقیه را رها می‌کنیم. در این سناریو هزینهٔ اضافی‌ای ندارد اگر باکتری بمیرد—هزینهٔ ثابتی برای هر آزمایش هست، اما لازم نیست شکست‌های آزمایش را کمینه کنیم؛ بلکه فقط سعی می‌کنیم هرچه سریع‌تر تصمیمِ خوبی بگیریم.

تکلیفِ انتخابِ بهترین گزینه تحتِ این شرایط «مسئلهٔ انتخاب» (Selection Problem) نامیده می‌شود. مسائلِ انتخاب در بافت‌های صنعتی و پرسنلی فراگیرند. اغلب باید تصمیم گرفت از کدام تأمین‌کننده برای فرایندی استفاده شود؛ یا کدام کارکنِ نامزد استخدام شود. مسائلِ انتخاب از نظرِ سطحی شبیهِ مسئلهٔ باندیت‌اند اما ویژگی‌های ریاضیِ متفاوتی دارند. به‌طورِ خاص، هیچ تابعِ شاخصی برای مسائلِ انتخاب وجود ندارد. اثباتِ این واقعیت مستلزم نشان‌دادنِ سناریویی است که در آن سیاستِ بهینه ترجیح‌هایش را برای دو اهرمِ M1 و M2 عوض می‌کند وقتی اهرمِ سومیِ M3 افزوده می‌شود (تمرین SELC.۱۷ را ببینید).

فصل ۵ مفهومِ مسائلِ تصمیمِ فراسطحی (Metalevel Decision Problems) را معرفی کرد، مانندِ تصمیم‌گیری دربارهٔ اینکه چه محاسباتی در طولِ جستجوی درختِ بازی قبل از حرکت انجام شود. تصمیمِ فراسطحی از این نوع نیز مسئلهٔ انتخاب است، نه مسئلهٔ باندیت. بدیهی است، بسطِ گره یا ارزیابی همان مقدارِ زمان را هزینه دارد، خواه مقدارِ خروجیِ بالا یا پایین تولید کند. پس شاید شگفت‌آور باشد که الگوریتمِ جستجوی درختیِ مونت‌کارلو (صفحهٔ ۱۶۳ را ببینید) این‌قدر موفق بوده، با توجه به اینکه می‌کوشد مسائلِ انتخاب را با اکتشافیِ UCB حل کند، که برای مسائلِ باندیت طراحی شده بود. به‌طورِ کلی، انتظار می‌رود الگوریتم‌های باندیتِ بهینه بسیار کمتر از الگوریتم‌های انتخابِ بهینه کاوش کنند، زیرا الگوریتمِ باندیت فرض می‌کند آزمایشِ ناموفق پولِ واقعی هزینه دارد.

تعمیمِ مهمِ فرایندِ باندیت «ابرفرایندِ باندیت» (Bandit Superprocess) یا BSP است، که در آن هر اهرم به خودیِ خود یک فرایندِ تصمیمِ مارکوفِ کامل است، نه اینکه فرایندِ پاداشِ مارکوف با فقط یک کنشِ ممکن باشد. همهٔ ویژگی‌های دیگر یکسان‌اند: اهرم‌ها مستقل‌اند، فقط یکی (یا شمارِ کران‌داری) می‌تواند هم‌زمان رویش کار شود، و یک عاملِ تنزیلِ تکی هست. نمونه‌های BSP شاملِ زندگیِ روزمره‌اند، جایی که می‌توان هم‌زمان به یک تکلیف پرداخت، حتی اگر چند تکلیف نیازمندِ توجه باشند؛ مدیریتِ پروژه با چند پروژه؛ تدریس با چند شاگردِ نیازمندِ راهنماییِ فردی؛ و مانندِ آن. اصطلاحِ معمولی برای این «چندوظیفگی» (Multitasking) است. آن‌قدر فراگیر است که به‌ندرت قابلِ توجه است: هنگام صورت‌بندیِ مسئلهٔ تصمیمِ دنیایِ واقعی، تحلیلگرانِ تصمیم به‌ندرت می‌پرسند آیا مشتری‌شان مسائلِ دیگر و نامرتبطی دارد.

ممکن است کسی چنین استدلال کند: «اگر n تا MDP مجزا وجود داشته باشد، بدیهی است سیاستِ بهینهٔ کلی از راه‌حل‌های بهینهٔ MDPهای فردی ساخته می‌شود. با سیاستِ بهینه‌اش πi، هر MDP به فرایندِ پاداشِ مارکوف تبدیل می‌شود که در آن فقط یک کنشِ πi(s) در هر حالتِ s هست. پس ابرفرایندِ باندیتِ n-بازویی را به فرایندِ باندیتِ n-بازویی کاهش داده‌ایم.» برای نمونه، اگر توسعه‌دهندهٔ املاک یک گروهِ ساخت‌وساز و چند مرکزِ خرید برای ساختن داشته باشد، به نظر صرفاً عقلِ سلیم می‌رسد باید برای هر مرکزِ خرید طرحِ ساختِ بهینه تدوین کرد و سپس مسئلهٔ باندیت را برای تصمیم‌گیری دربارهٔ اینکه هر روز گروه را کجا بفرستد حل کرد.

اگرچه این بسیار محتمل به نظر می‌رسد، نادرست است. در واقع، سیاستِ بهینهٔ سراسری برای BSP ممکن است کنش‌هایی شامل شود که از دیدگاهِ MDP سازنده‌ای که در آن‌ها گرفته می‌شوند به‌صورتِ محلی زیربهینه‌اند. دلیلش این است که در دسترس‌بودنِ MDPهای دیگر برای عمل‌کردن، توازنِ میانِ پاداش‌های کوتاه‌مدت و بلندمدت را درِ MDP مؤلفه‌ای تغییر می‌دهد. در واقع، تمایل به رفتارِ حریصانه‌تر در هر MDP دارد (جستجوی پاداش‌های کوتاه‌مدت)، زیرا هدف‌گرفتنِ پاداشِ بلندمدت در یک MDP، پاداش‌ها را در همهٔ MDPهای دیگر به تأخیر می‌انداخت.

برای نمونه، فرض کنیم برنامهٔ ساختِ بهینهٔ محلی برای یک مرکزِ خرید نخستین مغازه را تا هفتهٔ ۱۵ برای اجاره در دسترس دارد، در حالی که برنامهٔ زیربهینه هزینهٔ بیشتری دارد اما نخستین مغازه را تا هفتهٔ ۵ در دسترس دارد. اگر چهار مرکزِ خرید برای ساختن باشد، شاید بهتر باشد در هر یک از برنامهٔ زیربهینهٔ محلی استفاده شود تا اجاره‌ها از هفته‌های ۵، ۱۰، ۱۵ و ۲۰ جاری شوند، نه هفته‌های ۱۵، ۳۰، ۴۵ و ۶۰. به بیانِ دیگر، آنچه برایِ MDP تکی فقط ۱۰ هفته تأخیر می‌بود، برایِ MDP چهارم به ۴۰ هفته تأخیر تبدیل می‌شود. به‌طورِ کلی، سیاست‌های سراسری و محلیِ بهینه لزوماً فقط وقتی بر هم منطبق‌اند که عاملِ تنزیل 1 باشد؛ در آن صورت، هزینه‌ای برای تأخیرِ پاداش‌ها در هیچ MDPای نیست.

پرسشِ بعدی این است که BSPها را چگونه حل کنیم. بدیهی است، راه‌حلِ بهینهٔ سراسری برای BSP را می‌توان با تبدیلِ آن به MDP سراسری روی فضایِ حالتِ حاصل‌ضربِ دکارتی محاسبه کرد. شمارِ حالت‌ها در شمارِ اهرم‌های BSP نمایی می‌بود، پس این به‌طرزِ وحشتناکی غیرعملی می‌بود. در عوض، می‌توانیم از ماهیتِ سستِ تعاملِ میانِ اهرم‌ها بهره بگیریم. این تعامل فقط از توانایی محدودِ عامل برای پرداختنِ هم‌زمان به اهرم‌ها ناشی می‌شود. تا حدی، تعامل را می‌توان با مفهومِ «هزینهٔ فرصت» (Opportunity Cost) مدل کرد: چقدر مطلوبیت در هر گامِ زمانی با اختصاص‌ندادنِ آن گامِ زمانی به اهرمِ دیگر از دست می‌رود. هرچه هزینهٔ فرصت بالاتر باشد، تولیدِ پاداش‌های زودرس در اهرمِ معین ضروری‌تر است. در برخی موارد، سیاستِ بهینه در اهرمِ معین تحتِ تأثیرِ هزینهٔ فرصت قرار نمی‌گیرد. (به‌طورِ پیش‌پاافتاده، این در فرایندِ پاداشِ مارکوف صادق است، زیرا فقط یک سیاست هست.) در آن صورت، سیاستِ بهینه را می‌توان اعمال کرد و آن اهرم را به فرایندِ پاداشِ مارکوف تبدیل کرد. چنین سیاستِ بهینه‌ای، اگر وجود داشته باشد، «سیاستِ مسلط» (Dominating Policy) نامیده می‌شود. معلوم می‌شود با افزودنِ کنش‌ها به حالت‌ها، همیشه می‌توان نسخهٔ سست‌شده‌ای از MDP ساخت (بخش ۳.۶.۲ را ببینید) به‌گونه‌ای که سیاستِ مسلط داشته باشد، که بدین‌ترتیب کرانِ بالایی بر ارزشِ عمل‌کردن در اهرم می‌دهد. کرانِ پایین را می‌توان با حلِ جداگانهٔ هر اهرم (که ممکن است به‌طورِ کلی سیاستِ زیربهینه بدهد) و سپس محاسبهٔ شاخص‌های گیتینز محاسبه کرد. اگر کرانِ پایینِ عمل‌کردن در یک اهرم از کران‌های بالای همهٔ کنش‌های دیگر بالاتر باشد، مسئله حل شده؛ اگر نه، ترکیبی از جستجوی نگاه‌به‌جلو و محاسبهٔ دوبارهٔ کران‌ها تضمین‌شده سرانجام سیاستِ بهینه‌ای را برای BSP شناسایی می‌کند. با این رویکرد، BSPهای نسبتاً بزرگی (10^{40} حالت یا بیشتر) را می‌توان در چند ثانیه حل کرد.

۱۷.۴ MDPهای تا حدی قابل مشاهده

توصیفِ فرایندهای تصمیمِ مارکوف در بخش ۱۷.۱ فرض می‌کرد محیط کاملاً قابل مشاهده است. با این فرض، عامل همیشه می‌داند در کدام حالت است. این، همراه با فرضِ مارکوف برای مدلِ انتقال، بدان معناست سیاستِ بهینه فقط به حالتِ کنونی بستگی دارد.

وقتی محیط فقط تا حدی قابل مشاهده است، وضعیت، می‌توان گفت، بسیار کمتر روشن است. عامل لزوماً نمی‌داند در کدام حالت است، پس نمی‌تواند کنشِ $\pi(s)$ توصیه‌شده برای آن حالت را اجرا کند. علاوه بر این، مطلوبیتِ حالتِ s و کنشِ بهینه در s نه فقط به s، بلکه به میزانِ دانستنِ عامل هنگامِ بودن در s نیز بستگی دارد. به این دلایل، «MDPهای تا حدی قابل مشاهده» (Partially Observable MDPs) یا POMDPها—تلفظ «پام-دی-پی»—معمولاً بسیار دشوارتر از MDPهای معمولی دیده می‌شوند. با این حال، نمی‌توانیم از POMDPها اجتناب کنیم، زیرا دنیای واقعی یکی است.

۱۷.۴.۱ تعریفِ POMDPها

برای دست‌یافتن به POMDPها، نخست باید آن‌ها را درست تعریف کنیم. POMDP همان عناصر MDP را دارد—مدلِ انتقالِ $P(s'|s,a)$، کنش‌های $A(s)$ و تابعِ پاداشِ $R(s,a,s')$—اما، مانندِ مسائلِ جستجویِ تا حدی قابل مشاهدهٔ بخش ۴.۴، مدلِ حسگرِ $P(e|s)$ نیز دارد. در این‌جا، مانندِ فصل ۱۴، مدلِ حسگر احتمالِ ادراکِ شواهدِ e در حالتِ s را مشخص می‌کند.[5] برای نمونه، می‌توانیم جهانِ 4×3 شکل ۱۷.۱ را با افزودنِ حسگرِ نویزی یا جزئی به‌جای فرضِ اینکه عامل مکانش را دقیقاً می‌داند، به POMDP تبدیل کنیم. حسگرِ چهاربیتیِ نویزیِ صفحهٔ ۴۷۶ می‌تواند استفاده شود، که حضور یا غیابِ دیوار را در هر جهتِ قطبی با دقتِ $1-\varrho$ گزارش می‌کند.

مانندِ MDPها، می‌توانیم بازنمایی‌های فشرده‌ای برای POMDPهای بزرگ با استفاده از شبکه‌های تصمیمِ پویا به دست آوریم (بخش ۱۷.۱.۴ را ببینید). متغیرهای حسگرِ Et را می‌افزاییم، با فرضِ اینکه متغیرهای حالتِ Xt ممکن است مستقیماً قابل مشاهده نباشند. مدلِ حسگرِ POMDP سپس با $P(Et|Xt)$ داده می‌شود. برای نمونه، ممکن است متغیرهای حسگری به DDN شکل ۱۷.۴ بیفزاییم، مانندِ BatteryMeterت برای برآوردِ شارژِ واقعیِ Batteryt و Speedometert برای برآوردِ بزرگیِ بردارِ سرعتِ $\dot{X}t$. حسگرِ سونارِ Wallst ممکن است فواصلِ برآوردی تا نزدیک‌ترین دیوار را در هر یک از چهار جهتِ اصلیِ نسبی به جهت‌گیریِ کنونیِ ربات بدهد؛ این مقادیر به موقعیت و جهت‌گیریِ کنونیِ Xt بستگی دارند.

در فصل‌های ۴ و ۱۱، مسائلِ برنامه‌ریزیِ ناجبرگرایانه و تا حدی قابل مشاهده را مطالعه کردیم و «حالتِ باور» (Belief State)—مجموعهٔ حالت‌های واقعی که عامل ممکن است در آن باشد—را به‌عنوانِ مفهومِ کلیدی برای توصیف و محاسبهٔ راه‌حل‌ها شناسایی کردیم. در POMDPها، حالتِ باورِ b توزیعِ احتمالی روی همهٔ حالت‌های ممکن می‌شود، درست مانندِ فصل ۱۴. برای نمونه، حالتِ باورِ اولیه برای POMDP_4×3 می‌تواند توزیعِ یکنواخت روی نه حالتِ غیرپایانی همراه با 0ها برای حالت‌های پایانی باشد، یعنی ⟨1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 0, 0⟩. از نمادِ $b(s)$ برای اشاره به احتمالِ تخصیص‌داده‌شده به حالتِ واقعیِ s توسط حالتِ باورِ b استفاده می‌کنیم. عامل می‌تواند حالتِ

باورِ کنونی‌اش را به‌عنوانِ توزیعِ احتمالِ شرطی روی حالت‌های واقعی، با توجه به توالیِ حس‌گذارِه‌ها و کنش‌ها تاکنون، محاسبه کند. این اساساً همان تکلیفِ فیلترکردنِ توصیف‌شده در فصل ۱۴ است. معادلهٔ پایهٔ فیلترکردنِ بازگشتی (14.5 در صفحهٔ ۴۶۷) نشان می‌دهد چگونه حالتِ باورِ تازه را از حالتِ باورِ قبلی و شاهدِ تازه محاسبه کنیم. برای POMDPها، کنشی نیز باید در نظر بگیریم، اما نتیجه اساساً یکسان است. اگر b حالتِ باورِ قبلی بود، و عامل کنشِ a را انجام دهد و سپس شواهدِ e را ادراک کند، آن‌گاه حالتِ باورِ تازه با محاسبهٔ احتمالِ اکنون‌بودنِ در حالتِ s'، برای هر s'، با فرمولِ زیر به دست می‌آید:

$$b'(s') = \alpha P(e|s') \sum_s P(s'|s,a)\, b(s)$$

که در آن α ثابتِ نرمال‌سازی است که مجموعِ احتمال‌های حالتِ باور را برابر با ۱ می‌سازد. به قیاس با عملگرِ به‌روزرسانیِ فیلترکردن (صفحهٔ ۴۶۷)، می‌توانیم این را چنین بنویسیم:

$$b' = \alpha \text{FORWARD}(b, a, e) \qquad (17.16)$$

در 4×3 ِPOMDP، فرض کنیم عامل به چپ حرکت کند و حسگرش یک دیوارِ مجاور گزارش دهد؛ آن‌گاه کاملاً محتمل است (هرچند تضمین‌شده نیست، زیرا هم حرکت و هم حسگر نویزی‌اند) عامل حالا در (3,1) باشد. تمرین POMD.۱۷ از شما می‌خواهد مقادیرِ دقیقِ احتمال را برای حالتِ باورِ تازه محاسبه کنید.

بینشِ بنیادینِ لازم برای فهمِ POMDPها این است: **کنشِ بهینه فقط به حالتِ باورِ کنونیِ عامل بستگی دارد.** یعنی سیاستِ بهینه را می‌توان با نگاشتِ π*(b) از حالت‌های باور به کنش‌ها توصیف کرد. به حالتِ واقعی که عامل در آن است بستگی ندارد. این چیزِ خوبی است، زیرا عامل حالتِ واقعی‌اش را نمی‌داند؛ تنها چیزی که می‌داند حالتِ باور است. از این رو، چرخهٔ تصمیمِ عاملِ POMDP را می‌توان به سه گامِ زیر تقسیم کرد:

1. با توجه به حالتِ باورِ کنونیِ b، کنشِ a = π*(b) را اجرا کن.
2. حس‌گذارهٔ e را مشاهده کن.
3. حالتِ باورِ کنونی را به FORWARD(b, a, e) تنظیم کن و تکرار کن.

می‌توانیم POMDPها را نیازمندِ جستجو در فضایِ حالتِ باور بدانیم، درست مانندِ روش‌های مسائلِ بدونِ حسگر و اقتضایی در فصل ۴. تفاوتِ اصلی این است که فضایِ حالتِ باورِ POMDP پیوسته است، زیرا حالتِ باورِ POMDP توزیعِ احتمال است. برای نمونه، حالتِ باور برای جهانِ 4×3 نقطه‌ای در فضایِ پیوستهٔ ۱۱-بعدی است. کنش حالتِ باور را تغییر می‌دهد، نه فقط حالتِ فیزیکی را، زیرا بر حس‌گذارِه‌ای که دریافت می‌شود اثر می‌گذارد. از این رو، کنش دستِ‌کم تا حدی طبقِ اطلاعاتی که عامل در نتیجه کسب می‌کند ارزیابی می‌شود. POMDPها بنابراین ارزشِ اطلاعات (بخش ۱۶.۶) را به‌عنوانِ مؤلفه‌ای از مسئلهٔ تصمیم شامل می‌شوند.

بیایید پیامدِ کنش‌ها را دقیق‌تر بررسی کنیم. به‌طورِ خاص، احتمالِ اینکه عامل در حالتِ باورِ b پس از اجرایِ کنشِ a به حالتِ باورِ b' برسد را محاسبه کنیم. حالا، اگر کنش و حس‌گذارهٔ بعدی را می‌دانستیم، معادلهٔ (17.16) به‌روزرسانیِ قطعیِ حالتِ باور را فراهم می‌کرد: b' = FORWARD(b, a, e). البته، حس‌گذارهٔ بعدی هنوز معلوم نیست، پس عامل

ممکن است بسته به حس‌گذارهٔای که دریافت می‌شود به یکی از چند حالتِ باورِ ممکنِ b' برسد. احتمالِ ادراکِ e، با توجه به اینکه a با شروع از حالتِ باورِ b انجام شده، با جمع‌زدن روی همهٔ حالت‌های واقعیِ s' که عامل ممکن است به آن‌ها برسد داده می‌شود:

$$P(e|a,b) = \sum_{s'} P(e|a,s',b)\, P(s'|a,b) = \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)\, b(s)$$

احتمالِ رسیدن به b' از b، با توجه به کنشِ a، را P(b'|b,a) بنویسیم. این احتمال را می‌توان چنین محاسبه کرد:

$$P(b'|b,a) = \sum_{e} P(b'|e,a,b)\, P(e|a,b) = \sum_{e} P(b'|e,a,b) \sum_{s'} P(e|s') \sum_{s} P(s'|s,a)\, b(s) \qquad (17.17)$$

که در آن P(b'|e,a,b) برابر 1 است اگر b' = FORWARD(b,a,e) و در غیرِ این صورت 0.

معادلهٔ (17.17) را می‌توان به‌عنوانِ تعریفِ مدلِ انتقالی برای فضایِ حالتِ باور دید. همچنین می‌توانیم تابعِ پاداشی برای انتقال‌های حالتِ باور تعریف کنیم، که از پاداشِ موردانتظارِ انتقال‌های حالتِ واقعیِ احتمالاً در جریان مشتق می‌شود. در این‌جا، از شکلِ سادهٔ ρ(b,a)، پاداشِ موردانتظار اگر عامل در حالتِ باورِ b کنشِ a را انجام دهد، استفاده می‌کنیم:

$$\rho(b,a) = \sum_{s} b(s) \sum_{s'} P(s'|s,a)\, R(s,a,s')$$

با هم، P(b'|b,a) و ρ(b,a) یکِ MDP قابلِ مشاهده را روی فضایِ حالت‌های باور تعریف می‌کنند. علاوه بر این، می‌توان نشان داد سیاستِ بهینه برای این MDP، π*(b)، نیز سیاستِ بهینه برایِ POMDP اصلی است. به بیانِ دیگر، حلِ POMDP روی فضایِ حالتِ فیزیکی را می‌توان به حلِ MDP روی فضایِ حالتِ باورِ متناظر کاهش داد. این واقعیت شاید کمتر شگفت‌آور باشد اگر به یاد آوریم حالتِ باور طبقِ تعریف همیشه برای عامل قابلِ مشاهده است.

۱۷.۵ الگوریتم‌های حلِ POMDPها

نشان داده‌ایم چگونه POMDPها را به MDP کاهش دهیم، اما MDPهایی که به دست می‌آوریم فضایِ حالتِ پیوسته (و معمولاً پربعد) دارند. این بدان معناست باید الگوریتم‌های برنامه‌نویسیِ پویا را از بخش‌های ۱۷.۲.۱ و ۱۷.۲.۲ بازطراحی کنیم، که فضایِ حالتِ متناهی و شمارِ متناهی کنش را فرض می‌کردند. در این‌جا الگوریتمِ تکرارِ مقداری که به‌طورِ خاص برای POMDPها طراحی شده توصیف می‌کنیم، و پس از آن الگوریتمِ تصمیم‌گیریِ برخطی مشابهِ آن‌هایی که برای بازی‌ها در فصل ۵ توسعه یافتند.

۱۷.۵.۱ تکرارِ مقدار برای POMDPها

بخش ۱۷.۲.۱ الگوریتمِ تکرارِ مقداری را توصیف کرد که برای هر حالت یک مقدارِ مطلوبیت محاسبه می‌کرد. با بی‌نهایت حالتِ باور، باید خلاق‌تر باشیم. سیاستِ بهینهٔ π* و کاربردش را در حالتِ باورِ خاصِ b در نظر بگیرید: سیاست کنشی تولید می‌کند، سپس برای هر حس‌گذارهٔ بعدی، حالتِ باور به‌روزرسانی و کنشِ تازه‌ای تولید می‌شود، و همین‌طور ادامه می‌یابد. برای این ِb خاص، بنابراین، سیاست دقیقاً معادلِ «برنامهٔ شرطی» (Conditional Plan) است، چنان‌که در فصل ۴ برای مسائلِ ناجبرگرایانه و تا حدی قابل مشاهده تعریف شد. به‌جای فکرکردن دربارهٔ سیاست‌ها، به برنامه‌های شرطی و چگونگیِ تغییرِ مطلوبیتِ موردانتظارِ اجرایِ برنامهٔ شرطیِ ثابت با حالتِ باورِ اولیه فکر کنیم. دو مشاهده می‌کنیم:

1. مطلوبیتِ اجرایِ برنامهٔ شرطیِ ثابتِ p با شروع از حالتِ فیزیکیِ s را αp(s) بنامیم. آن‌گاه مطلوبیتِ موردانتظارِ اجرایِ p در حالتِ باورِ b صرفاً ∑ s b(s)αp(s) است، یا b·αp اگر هر دو را بردار بدانیم. از این رو، مطلوبیتِ موردانتظارِ برنامهٔ شرطیِ ثابت به‌صورتِ خطی با b تغییر می‌کند؛ یعنی با «ابرصفحه‌ای» (Hyperplane) در فضایِ باور متناظر است.

1. در هر حالتِ باورِ معینِ b، سیاستِ بهینه انتخاب می‌کند برنامهٔ شرطیِ با بالاترین مطلوبیتِ موردانتظار را اجرا کند؛ و مطلوبیتِ موردانتظارِ b تحتِ سیاستِ بهینه صرفاً مطلوبیتِ آن برنامهٔ شرطی است:

```
U(b) = Uπ*(b) = max_p b·αp
```

اگر سیاستِ بهینهٔ π* انتخاب کند p را با شروع از b اجرا کند، آن‌گاه معقول است انتظار داشته باشیم شاید انتخاب کند p را در حالت‌های باورِ بسیار نزدیک به b اجرا کند؛ در واقع، اگر عمقِ برنامه‌های شرطی را کران‌دار کنیم، فقط شمارِ متناهی چنین برنامه‌هایی هست و فضایِ پیوستهٔ حالت‌های باور به‌طورِ کلی به نواحی تقسیم می‌شود، که هر یک با برنامهٔ شرطیِ خاصی متناظر است که در آن ناحیه بهینه است.

از این دو مشاهده، می‌بینیم تابعِ مطلوبیتِ U(b) روی حالت‌های باور، که بیشینهٔ مجموعه‌ای از ابرصفحه‌هاست، «تکه‌ای خطی و محدب» (Piecewise Linear and Convex) خواهد بود.

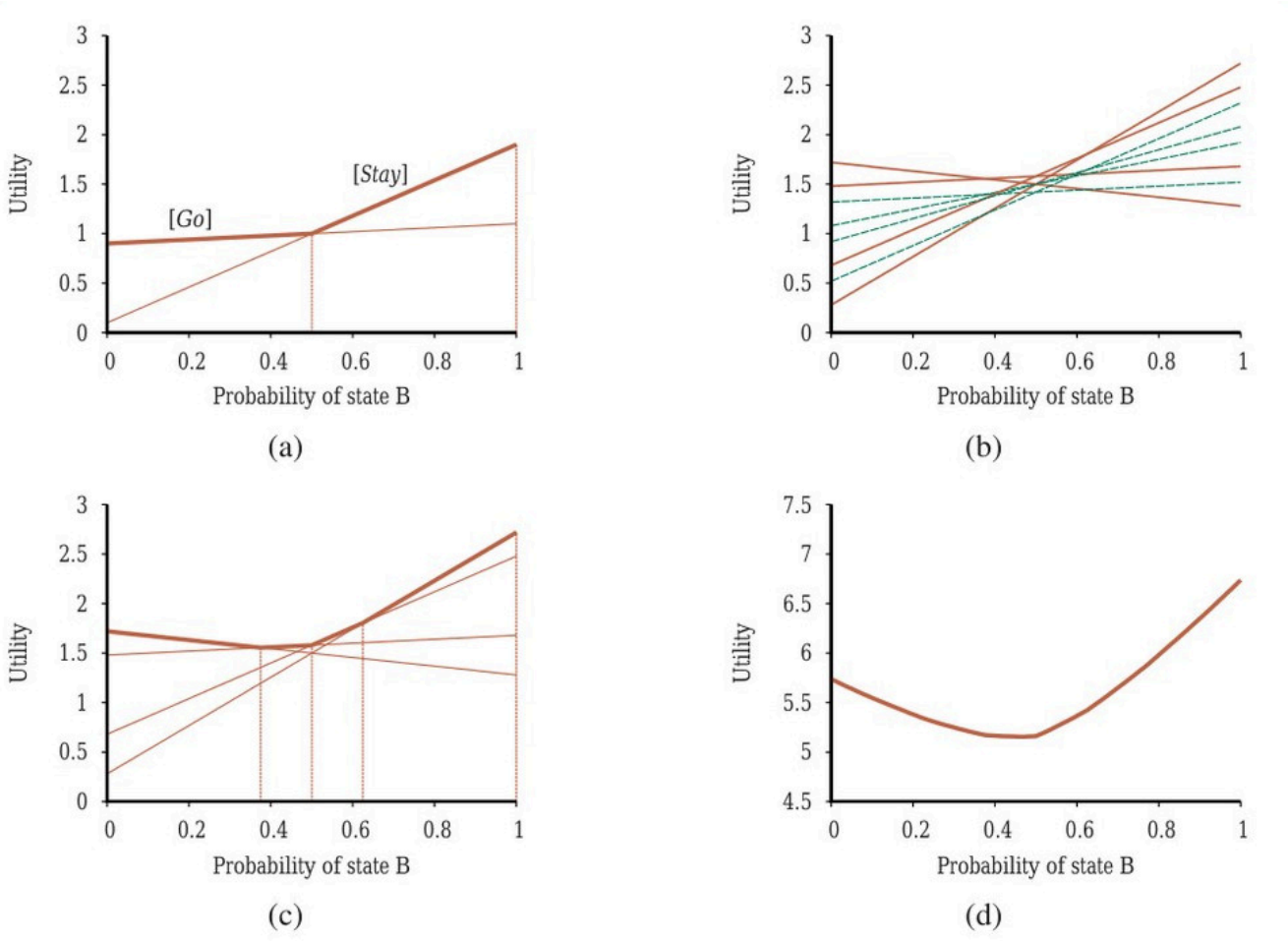
برای نشان‌دادنِ این، از جهانِ سادهٔ دوحالته استفاده می‌کنیم. حالت‌ها A و B برچسب‌گذاری شده‌اند و دو کنش هست: Stay با احتمالِ 0.9 همان‌جا می‌ماند و Go با احتمالِ 0.9 به حالتِ دیگر سوئیچ می‌کند. پاداش‌ها R(·,·,A) = 0 و R(·,·,B) = 1 هستند؛ یعنی هر انتقالی که به A ختم شود پاداشِ صفر و هر انتقالی که به B ختم شود پاداشِ 1 دارد. فعلاً عاملِ تنزیلِ γ = 1 را فرض می‌کنیم. حسگر حالتِ درست را با احتمالِ 0.6 گزارش می‌کند.

بدیهی است، عامل باید وقتی در حالتِ B است Stay و وقتی در حالتِ A است Go بکند. مسئله این است که نمی‌داند کجاست!

مزیتِ جهانِ دوحالته این است که فضایِ باور را می‌توان در یک بُعد تجسم کرد، زیرا دو احتمالِ b(A) و b(B) به 1 جمع می‌شوند. در شکل ۱۷.۱۵(الف)، محورِ x حالتِ باور را بازنمایی می‌کند، که با b(B)، احتمالِ بودن در حالتِ B، تعریف می‌شود. حالا برنامه‌های تک‌گامیِ [Stay] و [Go] را در نظر بگیریم، که هر یک پاداشِ یک انتقال را چنین دریافت می‌کند:

```
α[Stay](A) = 0.9·R(A,Stay,A) + 0.1·R(A,Stay,B) = 0.1
α[Stay](B) = 0.1·R(B,Stay,A) + 0.9·R(B,Stay,B) = 0.9
α[Go](A) = 0.1·R(A,Go,A) + 0.9·R(A,Go,B) = 0.9
α[Go](B) = 0.9·R(B,Go,A) + 0.1·R(B,Go,B) = 0.1
```

ابرصفحه‌های (خط‌های، در این مورد) b·α[Stay] و b·α[Go] در شکل ۱۷.۱۵(الف) نشان داده شده‌اند و بیشینه‌شان پررنگ است. خطِ پررنگ بنابراین تابعِ مطلوبیت را برای مسئلهٔ افقِ متناهی که فقط یک کنش اجازه می‌دهد بازنمایی می‌کند، و در هر «تکه» از تابعِ مطلوبیتِ تکه‌ای خطی، کنشِ بهینه نخستین کنشِ برنامهٔ شرطیِ متناظر است. در این مورد، سیاستِ تک‌گامیِ بهینه Stay وقتی b(B) > 0.5 و در غیرِ این صورت Go است.



شکل ۱۷.۱۵ — (الف) مطلوبیتِ دو برنامهٔ تک‌گامی به‌عنوانِ تابعی از حالتِ باورِ اولیهٔ b(B) برای جهانِ دوحالته، با تابعِ مطلوبیتِ متناظر که پررنگ نشان داده شده. (ب) مطلوبیت‌های ۸ برنامهٔ دوگامیِ متمایز. (ج) مطلوبیت‌های چهار برنامهٔ دوگامیِ مسلطنشده. (د) تابعِ مطلوبیت برای برنامه‌های هشت‌گامیِ بهینه.

وقتی مطلوبیت‌های αp(s) را برای همهٔ برنامه‌های شرطیِ p با عمقِ 1 در هر حالتِ فیزیکیِ s داریم، می‌توانیم مطلوبیت‌ها را برای برنامه‌های شرطیِ عمقِ 2 با درنظرگرفتنِ هر کنشِ اولِ ممکن، هر حس‌گذارهٔ بعدیِ ممکن، و سپس هر راهِ انتخابِ برنامهٔ عمق-1 برای اجرا برای هر حس‌گذارِه محاسبه کنیم:

```
[Stay; if Percept=A then Stay else Stay]
[Stay; if Percept=A then Stay else Go]
[Go; if Percept=A then Stay else Stay]
...
```

در مجموع هشت برنامهٔ متمایزِ عمق-2 هست، و مطلوبیت‌هایشان در شکل ۱۷.۱۵(ب) نشان داده شده. توجه کنید چهار تا از برنامه‌ها، که به‌صورتِ خط‌چین نشان داده شده‌اند، در کلِ فضایِ باور زیربهینه‌اند—می‌گوییم این برنامه‌ها «مسلط» (Dominated) هستند و دیگر لازم نیست در نظر گرفته شوند. چهار برنامهٔ مسلط‌نشده هست، که هر یک در ناحیه‌ای خاص بهینه است، چنان‌که در شکل ۱۷.۱۵(ج) نشان داده شده. نواحی، فضایِ حالتِ باور را افراز می‌کنند.

فرایند را برای عمقِ 3 تکرار می‌کنیم، و همین‌طور. به‌طورِ کلی، فرض کنیم p برنامهٔ شرطیِ عمق-d باشد که کنشِ اولش a و زیربرنامهٔ عمق-(d-1) برای حس‌گذارهٔ e برابر p.e باشد؛ آن‌گاه:

$$\alpha_p(s) = \sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma \sum_e P(e|s')\,\alpha_{p.e}(s')] \qquad (17.18)$$

این بازگشت به‌طورِ طبیعی الگوریتمِ تکرارِ مقداری به ما می‌دهد، که در شکل ۱۷.۱۶ داده شده.

```
function POMDP-VALUE-ITERATION(pomdp, ϱ) returns a utility function
,inputs: pomdp, a POMDP with states S, actions A(s), transition model P(s′|s,a)
                sensor model P(e|s), rewards R(s), discount γ
              ϱ, the maximum allowed error in the utility of any state
    local variables: U, U′, sets of plans p with associated utility vectors αp
              U′ ← a set containing only the empty plan [], with α[](s) = R(s)
                                                                  :repeat
                                                               ′U ← U
U′ ← the set of all plans consisting of an action and, for each possible next
                                                                    ,percept
a plan in U, with utility vectors computed according to Equation (17.18)
                                                 U′ ← REMOVE-DOMINATED-PLANS(U′)
                                            until MAX-DIFFERENCE(U, U′) ≤ ϱ(1−γ)/γ
                                                                 return U
```

شکل ۱۷.۱۶ — طرحِ سطح‌بالای الگوریتمِ تکرارِ مقدار برای POMDPها.

ساختارِ الگوریتم و تحلیلِ خطایش شبیهِ الگوریتمِ پایهٔ تکرارِ مقدار در شکل ۱۷.۶ صفحهٔ ۵۷۳ است؛ تفاوتِ اصلی این است که به‌جای محاسبهٔ یک عددِ مطلوبیت برای هر حالت، POMDP-VALUE-ITERATION مجموعه‌ای از برنامه‌های مسلط‌نشده را با ابرصفحه‌های مطلوبیتشان نگه می‌دارد.

پیچیدگیِ الگوریتم عمدتاً به چند برنامه تولید می‌شود بستگی دارد. با |A| کنش و |E| مشاهدهٔ ممکن، |A|^O(|E|^(d-1)) برنامهٔ متمایزِ عمق-d هست. حتی برای جهانِ فروتنِ دوحالته با d = 8، این $2^{2^{55}}$ برنامه است. حذفِ برنامه‌های مسلط برای کاهشِ این رشدِ دوتاییِ نمایی ضروری است: شمارِ برنامه‌های مسلط‌نشده با d = 8 فقط ۱۴۴ است. تابعِ مطلوبیت برای این ۱۴۴ برنامه در شکل ۱۷.۱۵(د) نشان داده شده.

توجه کنید حالت‌های باورِ میانی ارزشِ کمتری از حالتِ A و حالتِ B دارند، زیرا در حالت‌های میانی عامل فاقدِ اطلاعاتِ لازم برای انتخابِ کنشِ خوب است. به همین دلیل است که اطلاعات در معنایِ تعریف‌شده در بخش ۱۶.۶ ارزش دارد و سیاست‌های بهینه در POMDPها اغلب شاملِ کنش‌های جمع‌آوریِ اطلاعات‌اند.

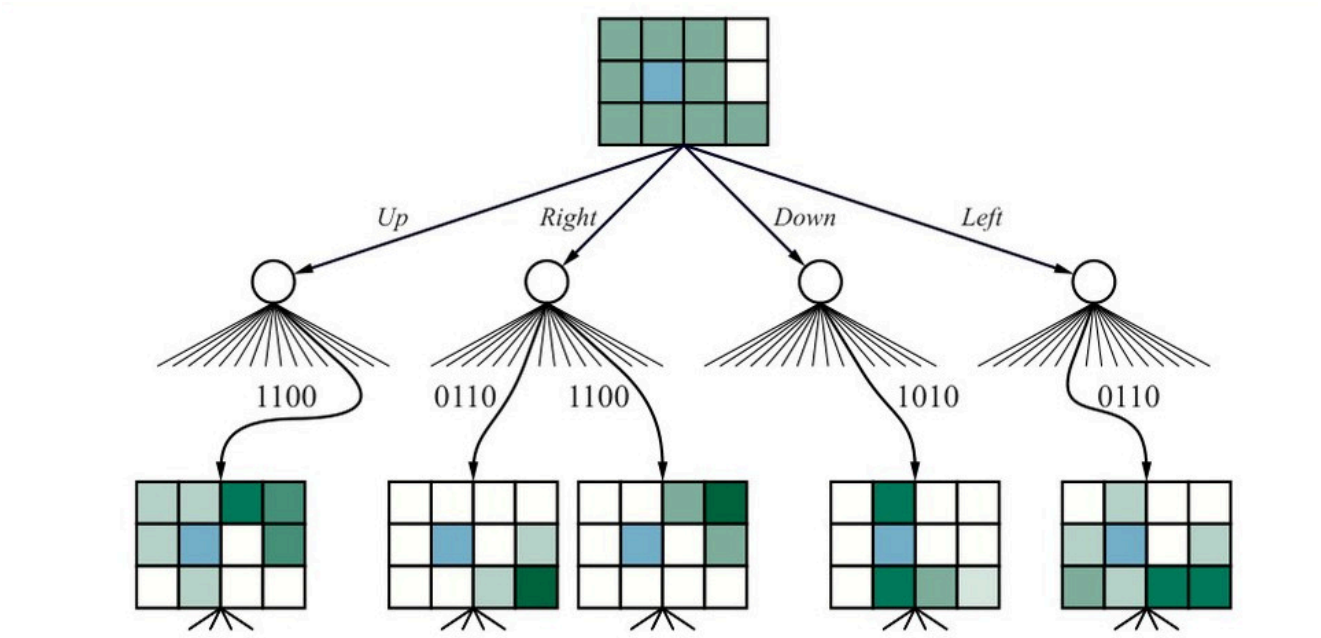
گامِ REMOVE-DOMINATED-PLANS و آزمونِ MAX-DIFFERENCE معمولاً به‌صورتِ برنامه‌های خطی پیاده‌سازی می‌شوند. با چنین تابعِ مطلوبیتی، سیاستِ اجراپذیری را می‌توان با نگاه‌کردن به اینکه کدام ابرصفحه در هر حالتِ باورِ معینِ b بهینه است و اجرایِ نخستین کنشِ برنامهٔ متناظر، استخراج کرد. در شکل ۱۷.۱۵(د)، سیاستِ بهینهٔ متناظر همچنان همان سیاستِ برنامه‌های عمق-1 است: Stay وقتی b(B) > 0.5 و در غیرِ این صورت Go.

در عمل، الگوریتمِ تکرارِ مقدار در شکل ۱۷.۱۶ برای مسائلِ بزرگ‌تر به‌طرزِ مأیوس‌کننده‌ای ناکارآمد است—حتی POMDP 4×3 بسیار دشوار است. دلیلِ اصلی این است که با n برنامهٔ شرطیِ مسلط‌نشده در سطحِ d، الگوریتم |A|·n^|E| برنامهٔ شرطی در سطحِ d+1 می‌سازد قبل از حذفِ مسلط‌ها. با حسگرِ چهاربیتی، |E| برابر 16 است و n می‌تواند در صدها باشد، پس این عملاً مأیوس‌کننده و نشدنی است. از زمانِ توسعهٔ این الگوریتم در دههٔ ۱۹۷۰، پیشرفت‌های متعددی بوده، از جملهِ شکل‌های کارآمدترِ تکرارِ مقدار و انواعِ مختلفی از الگوریتم‌های تکرارِ سیاست. برخی از این‌ها در یادداشت‌های پایانِ فصل بحث شده‌اند. اما برای POMDPهای عمومی، یافتنِ سیاست‌های بهینه بسیار دشوار است (در واقع PSPACE-سخت—یعنی واقعاً بسیار سخت). بخشِ بعد روشِ متفاوت و تقریبیِ دیگری برای حلِ POMDPها توصیف می‌کند، روشی مبتنی بر جستجوی نگاه‌به‌جلو.

۱۷.۵.۲ الگوریتم‌های برخط برای POMDPها

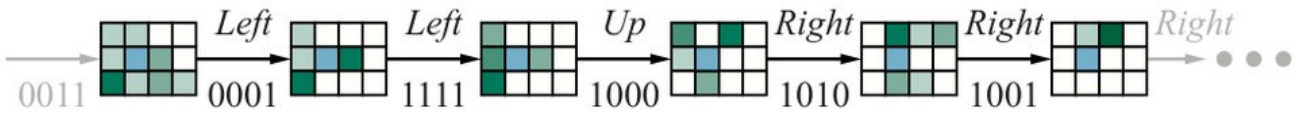
طراحیِ پایهٔ عاملِ POMDP برخط سرراست است: با حالتِ باورِ پیشینیِ معینی شروع می‌کند؛ بر اساسِ فرایندِ تأملِ متمرکز بر حالتِ باورِ کنونی‌اش کنشی انتخاب می‌کند؛ پس از عمل‌کردن، مشاهده‌ای دریافت و حالتِ باورش را با الگوریتمِ فیلترکردن به‌روزرسانی می‌کند؛ و فرایند تکرار می‌شود.

یکی از انتخاب‌های بدیهی برای فرایندِ تأمل، الگوریتمِ اکسپکتیمکس از بخش ۱۷.۲.۴ است، به‌جز با حالت‌های باور به‌جای حالت‌های فیزیکی به‌عنوانِ گره‌های تصمیم در درخت. گره‌های شانس در درختِ POMDP شاخه‌هایی دارند که با مشاهده‌های ممکن برچسب‌گذاری شده‌اند و به حالتِ باورِ بعدی می‌انجامند، با احتمال‌های انتقالِ داده‌شده توسط معادلهٔ (17.17). قطعه‌ای از درختِ اکسپکتیمکسِ حالتِ باور برای POMDP 4×3 در شکل ۱۷.۱۷ نشان داده شده.



شکل ۱۷.۱۷ – بخشی از درختِ اکسپکتیمکس برای POMDP ِ4×3 با حالتِ باورِ اولیهٔ یکنواخت. حالت‌های باور با سایه‌ای متناسب با احتمالِ بودن در هر مکان به تصویر کشیده شده‌اند. (کنش‌ها: بالا، راست، پایین، چپ.)

پیچیدگیِ زمانیِ جستجوی جامع تا عمقِ d برابر $O(|A|^d \cdot |E|^d)$ است، که در آن |A| شمارِ کنش‌های موجود و |E| شمارِ حس‌گذاره‌های ممکن است. (توجه کنید این بسیار کمتر از شمارِ برنامه‌های شرطیِ عمق-d ممکنِ تولیدشده توسط تکرارِ مقدار است.) مانندِ حالتِ قابل مشاهده، نمونه‌برداری در گره‌های شانس راهِ خوبی برای کاهشِ ضریبِ انشعاب بدونِ ازدست‌دادنِ دقتِ زیاد در تصمیمِ نهایی است. بدین‌ترتیب، پیچیدگیِ تصمیم‌گیریِ تقریبیِ برخط در POMDPها ممکن است به‌طورِ چشمگیری بدتر از MDPها نباشد. برای فضاهایِ حالتِ بسیار بزرگ، فیلترکردنِ دقیق ناشدنی است، پس عامل باید الگوریتمِ فیلترکردنِ تقریبی مانندِ فیلترِ ذره‌ای (صفحهٔ ۴۹۲ را ببینید) اجرا کند. سپس حالت‌های باور در درختِ اکسپکتیمکس مجموعه‌هایی از ذرات می‌شوند، نه توزیع‌های احتمالِ دقیق. برای مسائلِ با افق‌های بلند، همچنین ممکن است لازم باشد نوعِ پلی‌اوتِ دوربردِ استفاده‌شده در الگوریتمِ UCT (شکل ۵.۱۱) را اجرا کنیم. ترکیبِ فیلترِ ذره‌ای و UCT اعمال‌شده بر POMDPها با نامِ «برنامه‌ریزیِ مونت‌کارلویِ تا حدی قابل مشاهده» (Partially Observable Monte Carlo Planning) یا POMCP شناخته می‌شود. با بازنماییِ DDN برای مدل، الگوریتمِ POMCP، دست‌کم در اصل، برای POMDPهای بسیار بزرگ و واقع‌گرایانه قابلِ اعمال است. جزئیاتِ الگوریتم در تمرین POMC.۱۷ کاوش شده. POMCP قادر به تولیدِ رفتارِ شایسته در POMDP ِ4×3 است. نمونهٔ کوتاهی (و تا حدی خوش‌شانسانه) در شکل ۱۷.۱۸ نشان داده شده.



شکل ۱۷.۱۸ – توالیِ حس‌گذاره‌ها، حالت‌های باور و کنش‌ها در POMDP ِ4×3 با خطایِ حسِ دیوارِ ρ = 0.2. توجه کنید حرکت‌های اولیهٔ چپ چگونه امن‌اند—بسیار بعید است در (4,2) بیفتند—و مکانِ عامل را به شمارِ کمی از مکان‌های ممکن وادار می‌کنند. پس از حرکتِ بالا، عامل فکر می‌کند احتمالاً در (3,3) است، اما احتمالاً در (1,3). خوشبختانه، حرکتِ راست در هر دو مورد ایدهٔ خوبی است، پس به راست حرکت می‌کند، می‌فهمد در (1,3) بوده و حالا در (2,3) است، و سپس به حرکتِ راست ادامه می‌دهد و به هدف می‌رسد.

عامل‌های POMDP مبتنی بر شبکه‌های تصمیمِ پویا و تصمیم‌گیریِ برخط نسبت به دیگر طراحی‌های عاملِ ساده‌ترِ ارائه‌شده در فصل‌های قبلی مزایای متعددی دارند. به‌طورِ خاص، آن‌ها محیط‌های تا حدی قابل مشاهده و تصادفی را مدیریت می‌کنند و می‌توانند به‌آسانی «برنامه‌های»شان را برای مدیریتِ شواهدِ غیرمنتظره تجدیدنظر کنند. با مدل‌های حسگرِ مناسب، می‌توانند خرابیِ حسگر را مدیریت کنند و برای جمع‌آوریِ اطلاعات برنامه‌ریزی کنند. آن‌ها تحتِ فشارِ زمان و در محیط‌های پیچیده، با استفاده از فنونِ تقریبیِ گوناگون، «تخریبِ برازنده» (Graceful Degradation) نشان می‌دهند.

پس چه چیزی کم است؟ مانعِ اصلیِ استقرارِ دنیایِ واقعیِ چنین عامل‌هایی ناتوانی در تولیدِ رفتارِ موفق در مقیاس‌های زمانیِ بلند است. پلِی‌اوت‌های تصادفی یا نیمه‌تصادفی هیچ امیدی به کسبِ پاداشِ مثبت ندارند، مثلاً، در تکلیفِ چیدنِ میزِ شام، که ممکن است ده‌ها میلیون کنشِ کنترلِ حرکتی بگیرد. به نظر می‌رسد لازم است برخی از ایده‌های برنامه‌ریزیِ سلسله‌مراتبیِ توصیف‌شده در بخش ۱۱.۴ را قرض بگیریم. در زمانِ نوشتن، هنوز راه‌های رضایت‌بخش و کارآمدی برای اعمالِ این ایده‌ها در محیط‌های تصادفیِ تا حدی قابل مشاهده نیست.

خلاصهٔ فصل

این فصل نشان می‌دهد چگونه از دانش دربارهٔ جهان برای تصمیم‌گیری استفاده کنیم، حتی وقتی پیامدهای کنش نامطمئن‌اند و پاداشِ عمل‌کردن ممکن است تا پس از گذشتنِ کنش‌های بسیار به دست نیاید. نکاتِ اصلی به‌صورتِ زیرند:

- مسائلِ تصمیمِ ترتیبی در محیط‌های تصادفی، که «فرایندهای تصمیمِ مارکوف» یا MDP نیز نامیده می‌شوند، با مدلِ انتقالی مشخص می‌شوند که پیامدهای احتمالاتیِ کنش‌ها را مشخص می‌کند و تابعِ پاداشی که پاداش را در هر حالت مشخص می‌کند.
- مطلوبیتِ توالیِ حالت، جمعِ همهٔ پاداش‌های روی توالی است، احتمالاً با تنزیلِ زمانی. راه‌حلِ MDP سیاستی است که با هر حالتی که عامل ممکن است به آن برسد تصمیمی مرتبط می‌کند. سیاستِ بهینه، مطلوبیتِ توالی‌های حالتِ مواجه‌شده هنگام اجرایش را بیشینه می‌کند.
- مطلوبیتِ حالت، جمعِ موردانتظارِ پاداش‌هاست وقتی سیاستِ بهینه از آن حالت اجرا شود. الگوریتمِ تکرارِ مقدار، مجموعه‌ای از معادله‌ها را که مطلوبیتِ هر حالت را به مطلوبیت‌های همسایه‌هایش مرتبط می‌کنند به‌صورتِ تکرارشونده حل می‌کند.
- تکرارِ سیاست میانِ محاسبهٔ مطلوبیت‌های حالت‌ها تحتِ سیاستِ کنونی و بهبودِ سیاستِ کنونی نسبت به مطلوبیت‌های کنونی متناوب می‌شود.
- MDPهای تا حدی قابل مشاهده، یا POMDPها، حل‌کردنشان بسیار دشوارتر از MDPهاست. آن‌ها را می‌توان با تبدیل به MDP در فضایِ پیوستهٔ حالت‌های باور حل کرد؛ هم الگوریتم‌های تکرارِ مقدار و هم تکرارِ سیاست ابداع شده‌اند. رفتارِ بهینه در POMDPها شاملِ جمع‌آوریِ اطلاعات برای کاهشِ عدم قطعیت و بنابراین گرفتنِ تصمیم‌های بهتر در آینده است.
- عاملِ نظریه‌تصمیمی را می‌توان برای محیط‌های POMDP ساخت. عامل از شبکهٔ تصمیمِ پویا به‌عنوانِ مدلِ محیط و از الگوریتمِ فیلترکردن (مثلِ فیلترِ ذره‌ای) برای به‌روزرسانیِ حالتِ باور استفاده می‌کند. برای انتخابِ کنش، می‌توان از برنامه‌ریزیِ مونت‌کارلویِ برخط (POMCP) استفاده کرد.

یادداشت‌های کتاب‌شناختی و تاریخی

تئوریِ برنامه‌نویسیِ پویا توسط ریچارد بلمن (1957) ابداع شد. اصطلاحِ «فرایندِ تصمیمِ مارکوف» از کارِ هاوارد (1960) می‌آید. پایان‌نامهٔ دکتراِی رون هاوارد (1960) تکرارِ سیاست و ایدهٔ پاداشِ میانگین را برای حلِ مسائلِ افقِ نامتناهی معرفی کرد. چند نتیجهٔ اضافی توسط بلمن و دریفوس (1962) معرفی شد. استفاده از نگاشت‌های انقباضی در تحلیلِ الگوریتم‌های برنامه‌نویسیِ پویا از دناردو (1967) است. تکرارِ سیاستِ اصلاح‌شده از فان نونن (1976) و پوترمن و شین (1978) است. تکرارِ سیاستِ ناهمگام توسط ویلیامز و بیرد (1993) تحلیل شد، که کرانِ زیانِ سیاستِ معادلهٔ (17.13) را نیز اثبات کردند.

خانوادهٔ عمومیِ الگوریتم‌های «جاروی اولویت‌بندی‌شده» (Prioritized Sweeping) هدفشان سرعت‌بخشیدن به همگرایی به سیاست‌های بهینه با مرتب‌کردنِ اکتشافیِ محاسباتِ به‌روزرسانیِ مقدار و سیاست است (مور و اتکسن، 1993؛ آندره و همکاران، 1998؛ وینگیت و سپی، 2005).

صورت‌بندیِ حلِ MDP به‌صورتِ برنامهٔ خطی از دو گلینک (1960)، من (1960) و دپنو (1963) است. اگرچه برنامه‌نویسیِ خطی به‌طورِ سنتی به‌عنوانِ روشِ حلِ دقیقِ فروتر از برنامه‌نویسیِ پویا برای MDPها در نظر گرفته شده، د فاریاس و روی (2003) نشان می‌دهند می‌توان از برنامه‌نویسیِ خطی و بازنماییِ خطیِ تابعِ مطلوبیت برای به‌دست‌آوردنِ راه‌حل‌های تقریبیِ قابلِ‌اثباتِ خوب برای MDPهای بسیار بزرگ استفاده کرد.

پاپادیمیتریو و تسیتسیکلیس (1987) و لیتمن و همکاران (1995) نتایجِ عمومی دربارهٔ پیچیدگیِ محاسباتیِ MDPها فراهم می‌کنند. ییینیو یه (2011) رابطهٔ میانِ تکرارِ سیاست و روشِ سیمپلکس برای برنامه‌نویسیِ خطی را تحلیل می‌کند و اثبات می‌کند برای γ ثابت، زمانِ اجرایِ تکرارِ سیاست در شمارِ حالت‌ها و کنش‌ها چندجمله‌ای است.

کارِ بنیادینِ ساتون (1988) و واتکینز (1989) دربارهٔ روش‌های یادگیریِ تقویتی برای حلِ MDPها نقشِ قابلِ‌توجهی در معرفیِ MDPها به جامعهٔ هوش مصنوعی داشت. (کارِ زودترِ وربوس (1977) حاوی ایده‌های مشابهِ بسیاری بود، اما به همان میزان دنبال نشد.) پژوهشگرانِ هوش مصنوعی MDPها را در جهتِ بازنمایی‌های بیانی‌تر هل داده‌اند که می‌توانند مسائلِ بسیار بزرگ‌تری را نسبت به بازنمایی‌های اتمیِ سنتیِ مبتنی بر ماتریس‌های انتقال در خود جای دهند. ایده‌های پایه برای معماریِ عاملی با استفاده از شبکه‌های تصمیمِ پویا توسط دین و کانازاوا (1989a) پیشنهاد شد. تاتمن و شاکتر (1990) نشان دادند چگونه الگوریتم‌های برنامه‌نویسیِ پویا را روی مدل‌های DDN اعمال کنیم. چند نویسنده ارتباطِ میانِ MDPها و مسائلِ برنامه‌ریزیِ هوش مصنوعی را برقرار کردند و شکل‌های احتمالاتیِ بازنماییِ فشردهٔ STRIPS را برای مدل‌های انتقال توسعه دادند (ولمن، 1990b؛ کونیگ، 1991). کتابِ «برنامه‌ریزی و کنترل» اثرِ دین و ولمن (1991) این ارتباط را با عمقِ زیاد کاوش می‌کند.

کارهای بعدی روی «MDPهای عاملی» (Factored MDPs؛ بوتیلیر و همکاران، 2000؛ کولر و پار، 2000؛ گسترِیَن و همکاران، 2003b) از بازنمایی‌های ساخت‌یافته هم برای تابعِ مقدار و هم برای مدلِ انتقال استفاده می‌کنند، با بهبودهای قابلِ‌اثبات در پیچیدگی. «MDPهای رابطه‌ای» (Relational MDPs؛ بوتیلیر و همکاران، 2001؛ گسترِیَن و همکاران، 2003a) یک گام فراتر می‌روند و از بازنمایی‌های ساخت‌یافته برای مدیریتِ حوزه‌های با اشیای مرتبطِ بسیار استفاده می‌کنند. «MDPها و POMDPهای جهانِ باز» (سریواستاوا و همکاران، 2014b) همچنین عدم قطعیت بر سرِ وجود و هویتِ اشیا و کنش‌ها را اجازه می‌دهند.

نویسندگانِ بسیاری الگوریتم‌های تقریبیِ برخط برای تصمیم‌گیری در MDPها توسعه داده‌اند، که اغلب به‌صراحت از رویکردهای قبلیِ هوش مصنوعی به جستجوی بلادرنگ و بازی‌کردن قرض می‌گیرند (وربوس، 1992؛ دین و همکاران، 1993؛ تاش و راسل، 1994). کارِ بارتو و همکاران (1995) روی RTDP (برنامه‌نویسیِ پویایِ بلادرنگ) چارچوبِ عمومی‌ای برای فهمِ چنین الگوریتم‌هایی و ارتباطشان با یادگیریِ تقویتی و جستجوی اکتشافی فراهم کرد.

تحلیلِ اکسپکتیمکسِ عمق‌کران‌دار با نمونه‌برداری در گره‌های شانس از کرنز و همکاران (2002) است. الگوریتمِ UCT توصیف‌شده در فصل از کوچیس و سزپسواری (2006) است و از کارهای قبلی روی پلی‌اوت‌های تصادفی برای برآوردِ مقادیرِ حالت‌ها قرض می‌گیرد (آبرامسون، 1990؛ بروگمان، 1993؛ چانگ و همکاران، 2005).

مسائلِ باندیت توسط تامسون (1933) معرفی شدند اما پس از جنگِ جهانیِ دوم از طریقِ کارِ هربرت رابینز (1952) به برجستگی رسیدند. برادت و همکاران (1956) نخستین نتایج را دربارهٔ قاعده‌های توقف برای باندیت‌های یک‌بازویی اثبات کردند، که سرانجام به نتایجِ پیشگامانهٔ جان گیتینز انجامید (گیتینز و جونز، 1974؛ گیتینز، 1989). کاتِهکیس و واینوت (1987) MDP راه‌اندازیِ دوباره را به‌عنوانِ روشی برای محاسبهٔ شاخص‌های گیتینز پیشنهاد کردند. متنِ بری و فریستدت (1985) تغییراتِ بسیاری بر مسئلهٔ پایه را پوشش می‌دهد، در حالی که متنِ برخطِ روشنِ فرگوسن (2001) مسائلِ باندیت را به مسائلِ توقف مرتبط می‌کند.

لای و رابینز (1985) مطالعهٔ پشیمانیِ مجانبیِ سیاست‌های باندیتِ بهینه را آغاز کردند. اکتشافیِ UCB توسط آئر و همکاران (2002) معرفی و تحلیل شد. ابرفرایندهای باندیت (BSP) نخستین بار توسط نش (1973) مطالعه شدند اما تا حد زیادی در هوش مصنوعی ناشناخته ماندند. هادفیلد-منل و راسل (2015) الگوریتمِ کارآمدِ شاخه‌وبرگ‌ی را توصیف می‌کنند که قادر به حلِ BSPهای نسبتاً بزرگ است. مسائلِ انتخاب توسط بکهوفر (1954) معرفی شدند. هی و همکاران (2012) چارچوبِ صوریِ مسائلِ فراستدلال را توسعه دادند و نشان دادند نمونه‌های ساده به مسائلِ انتخاب نگاشت می‌شوند نه باندیت. آن‌ها همچنین نتیجهٔ رضایت‌بخشی را اثبات کردند که هزینهٔ محاسباتیِ موردانتظارِ استراتژیِ محاسباتیِ بهینه هرگز از افزایشِ موردانتظارِ کیفیتِ تصمیم بیشتر نیست—هرچند مواردی هست که سیاستِ بهینه ممکن است، با احتمالی، مدت‌ها پس از نقطه‌ای که هر سودِ ممکنی مصرف شده به محاسبه ادامه دهد.

مشاهدهٔ اینکه MDP تا حدی قابل مشاهده را می‌توان به MDP معمولی روی حالت‌های باور تبدیل کرد از آستروم (1965) و آئوکی (1965) است. نخستین الگوریتمِ کامل برای راه‌حلِ دقیقِ POMDPها—اساساً الگوریتمِ تکرارِ مقدارِ ارائه‌شده در این فصل—توسط ادوارد ساندیک (1971) در پایان‌نامهٔ دکترایش پیشنهاد شد. (مقالهٔ ژورنالیِ بعدیِ اسمالوود و ساندیک (1973) حاوی برخی خطاهاست، اما قابل‌دسترستر است.) لاوجوی (1991) بیست‌وپنج سالِ نخستِ پژوهشِ POMDP را مرور کرد و به نتیجه‌گیری‌های تا حدی بدبینانه دربارهٔ امکان‌پذیریِ حلِ مسائلِ بزرگ رسید.

نخستین مشارکتِ قابلِ‌توجه درونِ هوش مصنوعی الگوریتمِ Witness بود (کاساندرا و همکاران، 1994؛ کالب‌لینگ و همکاران، 1998)، نسخهٔ بهبودیافتهٔ تکرارِ مقدارِ POMDP. الگوریتم‌های دیگر به‌زودی دنبال شدند، از جملهِ رویکردی از هانسن (1998) که سیاست را به‌صورتِ افزایشی در قالبِ خودکارِ متناهیِ حالتی می‌سازد که حالت‌هایش حالت‌های باورِ ممکنِ عامل را تعریف می‌کنند. کارهای جدیدتر در هوش مصنوعی بر روش‌های تکرارِ مقدارِ نقطه‌ای (Point-Based) تمرکز کرده‌اند که، در هر تکرار، برنامه‌های شرطی و بردارهای α را برای مجموعهٔ متناهی از حالت‌های باور تولید می‌کنند، نه برای کلِ فضایِ باور. لاوجوی (1991) چنین الگوریتمی را برای شبکهٔ ثابتی از نقاط پیشنهاد کرد، رویکردی که بونه (2002) نیز گرفت. مقالهٔ تأثیرگذارِ پینو و همکاران (2003) تولیدِ نقاطِ قابلِ‌دسترس با شبیه‌سازیِ مسیرها به‌شیوه‌ای تا حدی حریصانه را پیشنهاد کرد؛ اسپان و ولاسیس (2005) مشاهده کردند فقط باید برای زیرمجموعهٔ کوچک و تصادفی‌انتخاب‌شده‌ای از نقاط برنامه تولید کرد تا برنامه‌های تکرارِ قبلی برای همهٔ نقاطِ مجموعه بهبود یابد. شانی و همکاران (2013) این و دیگر پیشرفت‌ها را در الگوریتم‌های نقطه‌ای مرور می‌کنند، که به راه‌حل‌های خوبی برای

مسائلِ با هزاران حالت انجامیده‌اند. چون POMDPها PSPACE-سخت‌اند (پاپادیمیتریو و تسیتسیکلیس، 1987)، پیشرفتِ بیشتر در روش‌های راه‌حلِ آفلاین ممکن است مستلزم بهره‌گیری از انواعِ ساختار در توابعِ مقدارِ ناشی از بازنماییِ عاملیِ مدل باشد.

رویکردِ برخط برای POMDPها—استفاده از جستجوی نگاه‌به‌جلو برای انتخابِ کنش برای حالتِ باورِ کنونی—نخستین بار توسط ساتیا و لاو (1973) بررسی شد. استفاده از نمونه‌برداری در گره‌های شانس به‌صورتِ تحلیلی توسط کرنز و همکاران (2000) و نگ و جوردن (2000) کاوش شد.

الگوریتمِ POMCP از سیلور و ونس (2011) است.

با توسعهٔ الگوریتم‌های تقریبِ نسبتاً مؤثر برای POMDPها، استفاده از آن‌ها به‌عنوانِ مدل برای مسائلِ دنیایِ واقعی افزایش یافته، به‌طورِ خاص در آموزش (رافرتی و همکاران، 2016)، سامانه‌های دیالوگ (یانگ و همکاران، 2013)، رباتیک (شیائو و همکاران، 2007؛ هوین و روی، 2009) و خودروهای خودران (فوربس و همکاران، 1995؛ بای و همکاران، 2015). کاربردِ مهمِ بزرگ‌مقیاس «سامانهٔ اجتناب از برخوردِ هواییِ X» (ACAS X) است، که هواپیماها و پهپادها را از برخوردِ میانهٔ هوا بازمی‌دارد. سامانه از POMDPها با شبکه‌های عصبی برای تقریبِ تابع استفاده می‌کند. ACAS X در مقایسه با سامانهٔ قدیمیِ TCAS، که در دههٔ ۱۹۷۰ با فناوریِ سامانه‌های خبره ساخته شد، ایمنی را به‌طورِ قابل‌توجهی بهبود می‌بخشد (کوچندر فر، 2015؛ جولیان و همکاران، 2018). تصمیم‌گیریِ پیچیده همچنین توسط اقتصاددانان و روان‌شناسان مطالعه شده. آن‌ها می‌یابند تصمیم‌گیرندگان همیشه عقلایی نیستند و ممکن است دقیقاً همان‌گونه که مدل‌های این فصل توصیف می‌کنند عمل نکنند. برای نمونه، وقتی انتخابی داده می‌شود، اکثریتِ مردم ۱۰۰ دلارِ امروز را بر تضمینِ ۲۰۰ دلار در دو سال ترجیح می‌دهند، اما همان مردم ۲۰۰ دلار در هشت سال را بر ۱۰۰ دلار در شش سال ترجیح می‌دهند. یکی از راه‌های تفسیرِ این نتیجه این است که مردم از پاداش‌های افزایشیِ تنزیل‌شدهٔ نمایی استفاده نمی‌کنند؛ شاید از «پاداشِ هذلولی» (Hyperbolic Reward) استفاده می‌کنند (تابعِ هذلولی در کوتاه‌مدت تندتر از تابعِ واپاشیِ نمایی فرومی‌رود). این و دیگر تفسیرهای ممکن توسط روبینشتاین (2003) بحث شده‌اند.

متن‌های برتسکاس (1987) و پوترمن (1994) مقدمه‌هایی دقیق و موشکافانه برای مسائلِ تصمیمِ ترتیبی و برنامه‌نویسیِ پویا فراهم می‌کنند. برتسکاس و تسیتسیکلیس (1996) شاملِ پوششِ یادگیریِ تقویتی‌اند. ساتون و بارتو (2018) زمینِ مشابهی را اما با سبکی در دسترس‌تر پوشش می‌دهند. سیگو و بوفه (2010)، موسام و کولوبوف (2012) و کوچندر فر (2015) تصمیم‌گیریِ ترتیبی را از دیدگاهِ هوش مصنوعی پوشش می‌دهند. کریشنامورتی (2016) پوششِ کاملی از POMDPها فراهم می‌کند.

پانوشت‌ها:

[1] همچنین می‌توان از هزینه‌های c(s,a,s') استفاده کرد، چنان‌که در تعریفِ مسائلِ جستجو در فصل ۳ انجام دادیم. با این حال، استفاده از پاداش‌ها در ادبیاتِ تصمیم‌های ترتیبیِ تحتِ عدم قطعیت استاندارد است.

[2] در این فصل از U برای تابعِ مطلوبیت استفاده می‌کنیم (برای سازگاری با بقیهٔ کتاب)، اما بسیاری از آثار دربارهٔ MDPها به‌جایش از V (برای value یعنی مقدار) استفاده می‌کنند.

[3] اگرچه این بدیهی به نظر می‌رسد، برای سیاست‌های افقِ متناهی یا برای راه‌های دیگرِ ترکیبِ پاداش‌ها در طولِ زمان، مانندِ گرفتنِ max، برقرار نیست. اثبات مستقیماً از یکتاییِ تابعِ مطلوبیت روی حالت‌ها نتیجه می‌شود، چنان‌که در بخش ۱۷.۲.۱ نشان داده شده.

[4] احتمال‌ها احتمال‌های فرایندِ به‌روزرسانیِ بیزی با پیشینِ Beta(1,1) هستند (بخش ۲۰.۲.۵ را ببینید).

[5] مدلِ حسگر همچنین می‌تواند به کنش و حالتِ پیامد بستگی داشته باشد، اما این تغییر بنیادین نیست.

فصل ۱۸: تصمیم‌گیری چندعاملی (Multiagent Decision Making)

در این فصل بررسی می‌کنیم وقتی بیش از یک عامل در محیط زندگی می‌کند چه باید کرد.

۱۸.۱ ویژگی‌های محیط‌های چندعاملی

تاکنون، تا حد زیادی فرض کرده‌ایم فقط یک عامل حس‌کردن، برنامه‌ریزی و عمل‌کردن را انجام می‌دهد. اما این فرضِ ساده‌سازیِ عظیمی را بازنمایی می‌کند که بسیاری از محیط‌های هوش مصنوعیِ دنیایِ واقعی را در بر نمی‌گیرد. در این فصل، بنابراین، مسائلِ ناشی از تصمیم‌گیریِ عامل در محیط‌هایی که شاملِ چند کنشگرند را در نظر می‌گیریم. چنین محیط‌هایی «سامانه‌های چندعاملی» (Multiagent Systems) نامیده می‌شوند، و عامل‌های چنین سامانه‌ای با «مسئلهٔ برنامه‌ریزیِ چندعاملی» (Multiagent Planning Problem) روبه‌رویند. اما، چنان‌که خواهیم دید، ماهیتِ دقیقِ مسئلهٔ برنامه‌ریزیِ چندعاملی—و فنونِ مناسبِ حلِ آن—به روابطِ میانِ عامل‌های محیط بستگی خواهد داشت.

۱۸.۱.۱ تصمیم‌گیرندهٔ تکی

نخستین امکان این است که، در حالی که محیط شاملِ چند کنشگر است، فقط یک تصمیم‌گیرنده دارد. در چنین حالتی، تصمیم‌گیرنده برای عامل‌های دیگر برنامه توسعه می‌دهد و به آن‌ها می‌گوید چه کنند. فرضِ اینکه عامل‌ها صرفاً آنچه بدان‌ها گفته می‌شود انجام می‌دهند، «فرضِ عاملِ خیرخواه» (Benevolent Agent Assumption) نامیده می‌شود. اما حتی در این محیط، برنامه‌های شاملِ چند کنشگر، کنشگرها را ملزم به هم‌زمان‌سازیِ کنش‌هایشان می‌کند. کنشگرهای A و B باید هم‌زمان برای کنش‌های مشترک (مانندِ خواندنِ دونفره) عمل کنند، در زمان‌های متفاوت برای کنش‌های مانعةالجمع (مانندِ شارژِ باتری‌ها وقتی فقط یک پریز هست)، و به‌صورتِ متوالی وقتی یکی پیش‌شرطی برای دیگری برقرار می‌کند (مانندِ شستنِ ظرف‌ها توسط A و سپس خشک‌کردنِ آن‌ها توسط B). حالتِ خاص، جایی است که تصمیم‌گیرندهٔ تکی با چند «اثرگر» (Effector) داریم که می‌توانند هم‌زمان عمل کنند—برای نمونه، انسانی که می‌تواند هم‌زمان راه برود و حرف بزند.

چنین عاملی باید «برنامه‌ریزیِ چنداثرگری» (Multieffector Planning) انجام دهد تا هر اثرگر را مدیریت کند و در عینِ حال تعامل‌های مثبت و منفیِ میانِ اثرگرها را مدیریت کند. وقتی اثرگرها از نظرِ فیزیکی به واحدهای جدا از هم گسسته شده‌اند—مانندِ ناوگانی از ربات‌های تحویل‌دهنده در کارخانه—برنامه‌ریزیِ چنداثرگری «برنامه‌ریزیِ چندبدنه» (Multibody Planning) می‌شود. مسئلهٔ چندبدنه‌ای همچنان مسئلهٔ «استاندارد» تک‌عاملی است تا زمانی که اطلاعاتِ حسگرِ مربوطِ جمع‌آوری‌شده توسط هر بدنه بتواند—یا به‌صورتِ مرکزی یا درونِ هر بدنه—برای تشکیلِ برآوردِ مشترکی از حالتِ جهان جمع شود که سپس مبنایِ اجرایِ برنامهٔ کلی قرار می‌گیرد؛ در این صورت، می‌توان چند بدنه را چنان در نظر گرفت

که گویی بدنهٔ تکی‌اند. وقتی قیدهای ارتباطی این را ناممکن می‌کنند، آنچه داریم گاهی «مسئلهٔ برنامه‌ریزیِ غیرمتمرکز» (Decentralized Planning) نامیده می‌شود؛ این شاید نامِ درستی نباشد، زیرا فازِ برنامه‌ریزی متمرکز است اما فازِ اجرا دست‌کم تا حدی از هم گسسته است. در این صورت، زیربرنامهٔ ساخته‌شده برای هر بدنه ممکن است لازم باشد شاملِ کنش‌های ارتباطیِ صریح با بدنه‌های دیگر باشد. برای نمونه، چند رباتِ شناسایی که منطقهٔ وسیعی را پوشش می‌دهند ممکن است اغلب از تماسِ رادیویی با یکدیگر خارج باشند و باید یافته‌هایشان را در زمان‌هایی که ارتباط ممکن است به اشتراک بگذارند.

۱۸.۱.۲ تصمیم‌گیرنده‌های متعدد

امکانِ دوم این است که کنشگرانِ دیگرِ محیط نیز تصمیم‌گیرنده باشند: آن‌ها هر یک ترجیح‌هایی دارند و برنامهٔ خودشان را انتخاب و اجرا می‌کنند. آن‌ها را «همتایان» (Counterparts) می‌نامیم. در این صورت، دو امکانِ دیگر را می‌توانیم تمایز دهیم.

- نخستین این است که، اگرچه تصمیم‌گیرنده‌های متعددی هستند، همه «هدفِ مشترکی» (Common Goal) را دنبال می‌کنند. این تقریباً وضعیتِ کارکنانِ شرکت است، که در آن تصمیم‌گیرنده‌های مختلف، امیدواریم، همان اهداف را از جانبِ شرکت دنبال می‌کنند. مسئلهٔ اصلیِ پیشِ رویِ تصمیم‌گیرنده‌ها در این محیط «مسئلهٔ هماهنگی» (Coordination Problem) است: آن‌ها باید مطمئن شوند همه در یک جهت تلاش می‌کنند و تصادفاً برنامه‌های یکدیگر را خراب نمی‌کنند.
- امکانِ دوم این است که تصمیم‌گیرنده‌ها هر یک ترجیح‌های شخصیِ خودشان را دارند، که هر یک تا آن‌جا که بتوانند دنبالش می‌کنند. ممکن است ترجیح‌ها به‌صورتِ قطبی مخالف باشند، چنان‌که در بازی‌های جمعِ صفر مانندِ شطرنج هست (فصل ۵ را ببینید). اما بیشترِ رویارویی‌های چندعاملی پیچیده‌تر از آن‌اند، با ترجیح‌های پیچیده‌تر. وقتی تصمیم‌گیرنده‌های متعدد، هر یک دنبالِ ترجیح‌های خودشان، هستند، عامل باید ترجیح‌های عامل‌های دیگر را در نظر بگیرد، و نیز این واقعیت را که آن عامل‌های دیگر نیز ترجیح‌های عامل‌های دیگر را در نظر می‌گیرند، و همین‌طور. این ما را به قلمروِ «نظریهٔ بازی» (Game Theory) می‌آورد: نظریهٔ تصمیم‌گیریِ استراتژیک. همین جنبهٔ استراتژیکِ استدلال—این‌که هر بازیکن در نظر می‌گیرد بازیکنانِ دیگر ممکن است چگونه عمل کنند—است که نظریهٔ بازی را از نظریهٔ تصمیم متمایز می‌کند. همان‌گونه که نظریهٔ تصمیم شالودهٔ نظریِ تصمیم‌گیری در هوش مصنوعیِ تک‌عاملی را فراهم می‌کند، نظریهٔ بازی شالودهٔ نظریِ تصمیم‌گیری در سامانه‌های چندعاملی را فراهم می‌کند.

استفاده از واژهٔ «بازی» در این‌جا نیز ایده‌آل نیست: استنتاجِ طبیعی این است که نظریهٔ بازی عمدتاً به سرگرمی‌های تفریحی یا سناریوهای مصنوعی مربوط است. هیچ‌چیز دورتر از حقیقت نیست. نظریهٔ بازی نظریهٔ «تصمیم‌گیریِ استراتژیک» (Strategic Decision Making) است. در موقعیت‌های تصمیم‌گیری استفاده می‌شود، از جملهِ حراجِ حقوقِ حفاریِ نفت و حقوقِ طیفِ بسامدِ بی‌سیم، رسیدگی‌های ورشکستگی، توسعهٔ محصول و تصمیم‌های قیمت‌گذاری، و دفاعِ ملی—موقعیت‌هایی شاملِ میلیاردها دلار و جانِ انسان‌های بی‌شمار. نظریهٔ بازی در هوش مصنوعی به دو راهِ اصلی استفاده می‌شود:

1. **طراحیِ عامل (Agent Design):** یک عامل می‌تواند از نظریهٔ بازی برای تحلیلِ تصمیم‌های ممکنش و محاسبهٔ مطلوبیتِ موردانتظارِ هر یک استفاده کند (تحتِ فرضِ اینکه عامل‌های دیگر عقلایی‌اند، طبقِ نظریهٔ بازی). بدین‌گونه، فنونِ نظریهٔ بازی می‌توانند بهترین استراتژی را در برابرِ بازیکنِ عقلایی و بازدهِ موردانتظارِ هر بازیکن تعیین کنند.
2. **طراحیِ سازوکار (Mechanism Design):** وقتی محیطی توسط عامل‌های بسیاری مسکون است، ممکن است بتوان قاعده‌های محیط (یعنی بازی‌ای که عامل‌ها باید بازی کنند) را طوری تعریف کرد که خیرِ جمعیِ همهٔ عامل‌ها بیشینه شود، وقتی هر عامل راه‌حلِ نظریه‌بازیِ بیشینه‌کنندهٔ مطلوبیتِ خودش را اتخاذ کند. برای نمونه، نظریهٔ بازی می‌تواند به طراحیِ پروتکل‌هایی برای مجموعه‌ای از مسیریاب‌های ترافیکِ اینترنت کمک کند به‌گونه‌ای که هر مسیریاب انگیزه‌ای برای عمل به شیوه‌ای داشته باشد که گذردهیِ سراسری بیشینه شود. طراحیِ سازوکار همچنین می‌تواند برای ساختِ سامانه‌های چندعاملیِ هوشمندی استفاده شود که مسائلِ پیچیده را به‌صورتِ توزیع‌شده حل می‌کنند.

نظریهٔ بازی طیفی از مدل‌های متفاوت فراهم می‌کند، که هر یک مجموعه‌ای از مفروضاتِ زیربنایی دارد؛ مهم است مدلِ درست برای هر محیط انتخاب شود. مهم‌ترین تمایز این است که آیا باید آن را «بازیِ همکارانه» (Cooperative Game) در نظر بگیریم یا نه:

- در بازیِ همکارانه، ممکن است «توافقِ الزام‌آور» (Binding Agreement) میانِ عامل‌ها وجود داشته باشد، که همکاریِ استوار را ممکن می‌کند. در جهانِ انسانی، قراردادهای حقوقی و هنجارهای اجتماعی به برقراریِ چنین توافق‌های الزام‌آوری کمک می‌کنند. در جهانِ برنامه‌های رایانه‌ای، ممکن است بتوان کدِ منبع را بازرسی کرد تا مطمئن شویم از توافقی پیروی خواهد کرد. از نظریهٔ بازیِ همکارانه برای تحلیلِ این وضعیت استفاده می‌کنیم.
- اگر توافق‌های الزام‌آور ممکن نباشند، «بازیِ غیرهمکارانه» (Non-Cooperative Game) داریم. اگرچه این اصطلاح پیشنهاد می‌کند بازی ذاتاً رقابتی است و همکاری ممکن نیست، لازم نیست چنین باشد: غیرهمکارانه صرفاً بدان معناست که هیچ توافقِ الزام‌آورِ مرکزی‌ای وجود ندارد که همهٔ عامل‌ها را متعهد کند و همکاری را تضمین نماید. اما به‌خوبی ممکن است عامل‌ها مستقلانه تصمیم به همکاری بگیرند، زیرا به نفعِ خودشان است.

از نظریهٔ بازیِ غیرهمکارانه برای تحلیلِ این وضعیت استفاده می‌کنیم.

برخی محیط‌ها چند بُعدِ متفاوت را ترکیب می‌کنند. برای نمونه، شرکتِ تحویلِ بسته ممکن است برنامه‌ریزیِ متمرکز و آفلاینی برای مسیرهای کامیون‌ها و هواپیماهایش هر روز انجام دهد، اما برخی جنبه‌ها را برای تصمیم‌های خودمختارِ رانندگان و خلبانان باز بگذارد، که می‌توانند فرداً به موقعیت‌های ترافیکی و آب‌وهوایی پاسخ دهند. همچنین، هدف‌های شرکت و کارکنانش، تا حدی، با پرداختِ «مشوق‌ها» (Incentives) (حقوق و پاداش‌ها) هم‌راستا می‌شوند—نشانهٔ قطعیِ اینکه این سامانهٔ چندعاملیِ واقعی است.

۱۸.۱.۳ برنامه‌ریزیِ چندعاملی

فعلاً، محیط‌های چنداثرگری، چندبدنه‌ای و چندعاملی را یکسان در نظر می‌گیریم و آن‌ها را به‌صورتِ عمومی محیط‌های «چندکنشگری» (Multiactor) برچسب‌گذاری می‌کنیم، و از اصطلاحِ عمومیِ «کنشگر» (Actor) برای پوششِ اثرگرها، بدنه‌ها و عامل‌ها استفاده می‌کنیم. هدفِ این بخش تبیینِ چگونگیِ تعریفِ مدل‌های انتقال، برنامه‌های درست و الگوریتم‌های برنامه‌ریزیِ کارآمد برای محیطِ چندکنشگری است.

برنامهٔ درست برنامه‌ای است که، اگر توسط کنشگرها اجرا شود، به هدف برسد. (در محیطِ واقعاً چندعاملی، البته، عامل‌ها ممکن است موافق اجرایِ هیچ برنامهٔ خاصی نباشند، اما دستِ‌کم خواهند دانست چه برنامه‌هایی اگر موافق اجرایشان بودند کار می‌کرد.) دشواریِ کلیدی در تلاش برای ارائهٔ مدلِ رضایت‌بخشی از کنشِ چندعاملی این است که باید به نحوی با مسئلهٔ دشوارِ «هم‌زمانی» (Concurrency) کنار بیاییم، که صرفاً بدان معناست برنامه‌های هر عامل ممکن است هم‌زمان اجرا شوند. اگر بخواهیم دربارهٔ اجرایِ برنامه‌های چندکنشگری استدلال کنیم، نخست به مدلی از برنامه‌های چندکنشگری نیاز خواهیم داشت که مدلِ رضایت‌بخشی از کنشِ هم‌زمان را در بر داشته باشد.

علاوه بر این، کنشِ چندکنشگری مجموعه‌ای کامل از مسائل را برمی‌انگیزد که در برنامه‌ریزیِ تک‌عاملی دغدغه نیستند. به‌طورِ خاص، عامل‌ها باید شیوه‌ای را در نظر بگیرند که کنش‌های خودشان با کنش‌های عامل‌های دیگر تعامل می‌کند. برای نمونه، عامل باید در نظر بگیرد آیا کنش‌های انجام‌شده توسط عامل‌های دیگر ممکن است پیش‌شرط‌های کنش‌های خودش را خراب کنند، آیا منابعی که هنگام اجرایِ سیاستش استفاده می‌کند قابلِ اشتراک‌اند یا ممکن است توسط عامل‌های دیگر تهی شوند؛ آیا کنش‌ها مانعةالجمع‌اند؛ و عاملِ مایل به کمک می‌تواند در نظر بگیرد چگونه کنش‌هایش ممکن است کنش‌های دیگران را تسهیل کند.

برای پاسخ به این پرسش‌ها به مدلی از کنشِ هم‌زمان نیاز داریم که بتوانیم مسائل را درونِ آن به‌درستی صورت‌بندی کنیم. مدل‌های کنشِ هم‌زمان دهه‌ها کانونِ اصلیِ پژوهش در جامعهٔ اصلیِ علومِ رایانه بوده‌اند، اما هیچ مدلِ قطعی و پذیرفته‌شدهٔ همگانی‌ای غالب نشده. با این وجود، سه رویکردِ زیر به‌طورِ گسترده تثبیت شده‌اند. رویکردِ نخست در نظرگرفتنِ «اجرایِ میان‌لایه‌ای» (Interleaved Execution) کنش‌ها در برنامه‌های مربوط است. برای نمونه، فرض کنیم دو عامل، A و B، با برنامه‌های زیر داریم:

```
A: [حرکت(A, راست), ضربه(A, توپ)]
B: [بی‌عمل(B), بی‌عمل(B)]
```

اجرایِ میان‌لایه‌ایِ این برنامه‌ها به دنباله‌های کنشِ [A1, B1, A2, B2] یا [A1, A2, B1, B2] یا هر ترتیبِ دیگری که هر برنامه حفظ شود می‌انجامد. این رویکردِ ساده این مزیت را دارد که می‌توان از آن با تکنیک‌های برنامه‌ریزیِ کلاسیک استفاده کرد، و به‌صورتِ صریح مشکلِ هم‌زمانی را با ممنوع‌کردنِ هر کنشِ هم‌زمانی حذف می‌کند. اما عیبِ آشکاری دارد: بعضی کنش‌ها در دنیای واقعی فقط در صورتی کار می‌کنند که هم‌زمان اجرا شوند. برای نمونه، «سواری‌کردن» توسط A بر «نگه‌داشتن» توسط B فقط وقتی کار می‌کند که A و B هم‌زمان عمل کنند. رویکردِ دوم، رویکردِ «هم‌زمانیِ هم‌زمان» است: در هر «گامِ زمانیِ» t، همهٔ کنشگرها کنش‌های خود را هم‌زمان انتخاب می‌کنند. این به درختِ بازیِ

هم‌زمانی منتهی می‌شود که در آن ضریبِ انشعاب، حاصلِ‌ضربِ تعدادِ کنش‌های هر کنشگر است. رویکردِ سوم مدلِ «هم‌زمانیِ جزئی» (Partial Concurrency) است: برای کنشگرها مشخص می‌کنیم کدام کنش‌ها باید هم‌زمان شروع شوند، کدام باید به‌صورتِ ترتیبی اجرا شوند، کدام‌ها می‌توانند با هم اجرا شوند اما لازم نیست، و مانندِ آن.

با کنار هم گذاشتنِ کنشگرها در سامانهٔ چندکنشگری با ضریبِ انشعابِ عظیم، کانونِ اصلیِ پژوهشِ برنامه‌ریزیِ چندکنشگری جداسازیِ کنشگرها تا آن‌جا که ممکن بوده، به‌گونه‌ای که (ایده‌آل‌تر) پیچیدگیِ مسئله به‌صورتِ خطی با n رشد کند نه نمایی با b^n. اگر کنشگرها هیچ تعاملی با یکدیگر نداشته باشند—برای نمونه، n کنشگر که هر یک بازیِ ورقِ تکی بازی می‌کنند—آن‌گاه می‌توانیم به‌سادگی n مسئلهٔ جداگانه حل کنیم. اگر کنشگرها «سست‌جفت» (Loosely Coupled) باشند، آیا می‌توانیم به چیزی نزدیکِ این بهبودِ نمایی برسیم؟ این، البته، پرسشِ مرکزی در بسیاری از حوزه‌های هوش مصنوعی است. روش‌های موفقِ حل را برای سامانه‌های سست‌جفت در بافتِ CSPها دیده‌ایم، جایی که گراف‌های قیدِ «درخت‌مانند» روش‌های حلِ کارآمد می‌دادند (صفحهٔ ۲۰۲)، و نیز در بافتِ پایگاه‌های دادهٔ الگوی مجزا (صفحهٔ ۱۰۰) و اکتشافی‌های افزایشی برای برنامه‌ریزی (صفحهٔ ۳۵۶).

رویکردِ استاندارد به مسائلِ سست‌جفت این است که وانمود کنیم مسائل کاملاً از هم جدا شده‌اند و سپس تعامل‌ها را اصلاح کنیم. برای مدلِ انتقال، این به معنای نوشتنِ طرح‌های کنش چنان است که گویی کنشگرها مستقل عمل می‌کنند.

ببینیم این برای بازیِ تنیسِ دونفره چگونه کار می‌کند. در این‌جا، دو بازیکنِ تنیسِ انسانی داریم که تیمِ دونفره‌ای را با هدفِ مشترکِ بردنِ مسابقه در برابرِ تیمِ حریف تشکیل می‌دهند. فرض کنیم در نقطه‌ای از بازی، تیم هدفِ برگرداندنِ توپی را دارد که به سویشان زده شده و تضمینِ اینکه دست‌کم یکی از آن‌ها جلوی تور را پوشش می‌دهد.

$$
\begin{aligned}
&Actors(A, B)\\
&Init(At(A, LeftBaseline) \land At(B, RightNet) \land\\
&\quad Approaching(Ball, RightBaseline) \land Partner(A, B) \land Partner(B, A))\\
&Goal(Returned(Ball) \land (At(x, RightNet) \lor At(x, LeftNet)))\\
&Action(Hit(actor, Ball),\\
&\quad \text{PRECOND:} Approaching(Ball, loc) \land At(actor, loc)\\
&\quad \text{EFFECT:} Returned(Ball))\\
&Action(Go(actor, to),\\
&\quad \text{PRECOND:} At(actor, loc) \land to \neq loc,\\
&\quad \text{EFFECT:} At(actor, to) \land \neg At(actor, loc))
\end{aligned}
$$

شکل ۱۸.۱ – مسئلهٔ تنیسِ دونفره. (شرایط اولیه، هدف و طرح‌های کنش برای دو بازیکن A و B؛ هدف: At(A,LeftNet) ∧ Returned(Ball) (At(B,LeftNet)).

به‌آسانی دیده می‌شود با «برنامهٔ مشترکِ» (Joint Plan) دواگامی که مشخص می‌کند هر بازیکن چه باید بکند می‌توانیم از شرایط اولیه به هدف برسیم: A باید به خطِ پایانیِ راست برود و توپ را بزند، در حالی که B باید سرِ تور بماند:

```
برنامه ۱: A : [Go(A, RightBaseline), Hit(A, Ball)]
          B : [NoOp(B), NoOp(B)]
```

با این حال، وقتی برنامه‌ای حکم می‌کند هر دو عامل هم‌زمان به توپ بزنند، مسائل پیش می‌آیند. در دنیای واقعی، این کار نخواهد کرد، اما طرحِ کنشِ Hit می‌گوید توپ با موفقیت برگردانده می‌شود. دشواری این است که پیش‌شرط‌ها حالتی را قید می‌کنند که در آن کنش به‌تنهایی می‌تواند با موفقیت اجرا شود، اما کنش‌های هم‌زمانِ دیگری را که ممکن است خرابش کنند قید نمی‌کنند. این مسئله را با تقویتِ طرح‌های کنش با یک ویژگیِ تازه حل می‌کنیم: «قیدِ کنشِ هم‌زمان» (Concurrent Action Constraint) که بیان می‌کند کدام کنش‌ها باید یا نباید هم‌زمان اجرا شوند. برای نمونه، کنشِ Hit را می‌توان چنین توصیف کرد:

```
,Action(Hit(actor, Ball)
CONCURRENT: ∀b (b ≠ actor ⇒ ¬Hit(b, Ball))
PRECOND: Approaching(Ball, loc) ∧ At(actor, loc)
(EFFECT: Returned(Ball)
```

به بیانِ دیگر، کنشِ Hit اثرِ بیان‌شده‌اش را فقط اگر هیچ کنشِ Hit دیگری توسط عاملِ دیگر در همان زمان رخ ندهد دارد. (در رویکردِ SATPLAN، این با اصلِ موضوعهٔ حذفِ جزئیِ کنش مدیریت می‌شد.) برای برخی کنش‌ها، اثرِ موردنظر فقط وقتی حاصل می‌شود که کنشِ دیگری هم‌زمان رخ دهد. برای نمونه، دو عامل برای حملِ کولری پر از نوشیدنی به زمینِ تنیس لازم‌اند:

```
,Action(Carry(actor, cooler, here, there)
CONCURRENT: ∃b (b ≠ actor ∧ Carry(b, cooler, here, there))
PRECOND: At(actor, here) ∧ At(cooler, here) ∧ Cooler(cooler)
(EFFECT: At(actor, there) ∧ At(cooler, there) ∧ ¬At(actor, here) ∧ ¬At(cooler, here)
```

با این نوع طرح‌های کنش، هر یک از الگوریتم‌های برنامه‌ریزیِ توصیف‌شده در فصل ۱۱ را می‌توان با اصلاح‌های کوچکی برای تولیدِ برنامه‌های چندکنشگری سازگار کرد. تا آن‌جا که جفت‌شدگیِ میانِ زیربرنامه‌ها سست است—به این معنا که قیدهای هم‌زمانی فقط به‌ندرت در طولِ جستجوی برنامه به کار می‌آیند—انتظار می‌رود اکتشافی‌های گوناگونِ اشتقاق‌شده برای برنامه‌ریزیِ تک‌عاملی در بافتِ چندکنشگری نیز مؤثر باشند.

۱۸.۱.۴ برنامه‌ریزی با عامل‌های متعدد: همکاری و هماهنگی

حالا محیطِ واقعاً چندعاملی را در نظر بگیریم که در آن هر عامل برنامهٔ خودش را می‌سازد. برای شروع، فرض کنیم هدف‌ها و پایگاهِ دانش مشترک‌اند. ممکن است فکر کنیم این به حالتِ چندبدنه‌ای کاهش می‌یابد—هر عامل به‌سادگی راه‌حلِ مشترک را محاسبه و بخشِ خودش از آن راه‌حل را اجرا می‌کند. افسوس، وجودِ حرفِ تعریف («the») در «راه‌حلِ مشترک» گمراه‌کننده است [زیرا ممکن است بیش از یک راه‌حل وجود داشته باشد]. این‌جا برنامهٔ دومی است که آن نیز به هدف می‌رسد:

```
برنامه ۲: A : [Go(A, LeftNet), NoOp(A)]
B : [Go(B, RightBaseline), Hit(B, Ball)]
```

اگر هر دو عامل بتوانند بر برنامهٔ 1 یا برنامهٔ 2 توافق کنند، هدف محقق می‌شود. اما اگر A برنامهٔ 2 و B برنامهٔ 1 را انتخاب کند، آن‌گاه هیچ‌کس توپ را برنخواهد گرداند. برعکس، اگر A برنامهٔ 1 و B برنامهٔ 2 را انتخاب کند، آن‌گاه هر دو سعی می‌کنند به توپ بزنند و آن نیز شکست خواهد خورد. عامل‌ها این را می‌دانند، اما چگونه می‌توانند هماهنگ شوند تا مطمئن شوند بر برنامه توافق می‌کنند؟

یکی از گزینه‌ها اتخاذِ «قرارداد» (Convention) قبل از درگیرشدن در فعالیتِ مشترک است. قرارداد هر قیدی بر انتخابِ برنامه‌های مشترک است. برای نمونه، قراردادِ «به سمتِ خودت بچسب» برنامهٔ 1 را رد می‌کرد و باعث می‌شد هر دو شریک برنامهٔ 2 را انتخاب کنند. رانندگان در جاده با مسئلهٔ برخورد نکردن با یکدیگر روبه‌رویند؛ این (تا حدی) با اتخاذِ قراردادِ «در سمتِ راستِ جاده بمان» در بیشترِ کشورها حل می‌شود؛ جایگزین، «در سمتِ چپ بمان»، به‌همان خوبی کار می‌کند تا زمانی که همهٔ عامل‌های محیط موافق‌اند.

ملاحظاتِ مشابهی برای توسعهٔ زبانِ انسانی صدق می‌کند، جایی که نکتهٔ مهم این نیست هر فرد چه زبانی باید صحبت کند، بلکه این واقعیت است که جامعه‌ای همه یک زبان صحبت می‌کنند. وقتی قراردادها فراگیر می‌شوند، «قوانین اجتماعی» (Social Laws) نامیده می‌شوند.

در غیابِ قرارداد، عامل‌ها می‌توانند از ارتباط برای دستیابی به دانشِ مشترکِ یک برنامهٔ مشترکِ شدنی استفاده کنند. برای نمونه، بازیکنِ تنیس می‌تواند فریاد بزند «مالِ من!» یا «مالِ تو!» تا برنامهٔ مشترکِ ترجیحی را نشان دهد. ارتباط لزوماً شاملِ تبادلِ کلامی نیست. برای نمونه، یک بازیکن می‌تواند برنامهٔ مشترکِ ترجیحی را به دیگری صرفاً با اجرایِ بخشِ نخستِ آن ارتباط دهد. اگر عاملِ A به سمتِ تور برود، آن‌گاه عاملِ B موظف است به خطِ پایانی برگردد تا توپ را بزند، زیرا برنامهٔ 2 تنها برنامهٔ مشترکی است که با رفتنِ A به سمتِ تور آغاز می‌شود. این رویکردِ هماهنگی، که گاهی «بازشناسیِ برنامه» (Plan Recognition) نامیده می‌شود، وقتی کار می‌کند که کنشِ تکی (یا توالیِ کوتاهی از کنش‌ها) توسط یک عامل برای تعیینِ بدونِ ابهامِ برنامهٔ مشترک توسط دیگری کافی باشد.

۱۸.۲ نظریهٔ بازیِ غیرهمکارانه

حالا مفاهیمِ کلیدی و فنونِ تحلیلیِ نظریهٔ بازی را معرفی می‌کنیم—نظریه‌ای که زیربنایِ تصمیم‌گیری در محیط‌های چندعاملی است. گشتِ ما با نظریهٔ بازیِ غیرهمکارانه شروع می‌شود.

۱۸.۲.۱ بازی‌های با حرکتِ تکی: بازی‌های شکلِ نرمال

نخستین مدلِ بازی که به آن نگاه می‌کنیم، بازی‌ای است که در آن همهٔ بازیکنان هم‌زمان کنش می‌گیرند و نتیجهٔ بازی بر اساسِ «نمایهٔ کنش‌هایی» (Profile of Actions) است که بدین‌گونه انتخاب می‌شوند. (در واقع، هم‌زمان‌بودنِ کنش‌ها حیاتی نیست؛ مهم این است هیچ بازیکنی از انتخاب‌های بازیکنانِ دیگر آگاهی ندارد.) این بازی‌ها «بازی‌های شکلِ نرمال» (Normal Form Games) نامیده می‌شوند. بازیِ شکلِ نرمال با سه مؤلفه تعریف می‌شود:

- «بازیکنان» (Players) یا عامل‌هایی که تصمیم خواهند گرفت. بازی‌های دونفره بیشترین توجه را دریافت کرده‌اند، هرچند بازی‌های n-نفره برای n > 2 نیز رایج‌اند. به بازیکنان نام‌های با حرفِ بزرگ می‌دهیم، مانندِ علی و بو یا O و E.
- «کنش‌هایی» (Actions) که بازیکنان می‌توانند انتخاب کنند. به کنش‌ها نام‌های با حرفِ کوچک می‌دهیم، مانندِ one یا testify. بازیکنان ممکن است مجموعهٔ کنش‌های یکسانی داشته باشند یا نه.
- «تابعِ بازدهی» (Payoff Function) که مطلوبیت را به هر بازیکن برای هر ترکیبِ کنش توسط همهٔ بازیکنان می‌دهد. برای بازی‌های دونفره، تابعِ بازدهی برای بازیکنی را می‌توان با ماتریسی بازنمایی کرد که در آن برای هر کنشِ ممکنِ یک بازیکن ردیفی و برای هر انتخابِ ممکنِ بازیکنِ دیگر ستونی هست: ردیفِ انتخاب‌شده و ستونِ انتخاب‌شده سلولی از ماتریس را تعریف می‌کنند، که با بازدهیِ بازیکنِ مربوط برچسب‌گذاری می‌شود. در حالتِ دونفره، مرسوم است دو ماتریس در «ماتریسِ بازدهی» (Payoff Matrix) تکی ترکیب شوند، که در آن هر سلول با بازدهی‌های هر دو بازیکن برچسب‌گذاری می‌شود.

برای نشان‌دادنِ این ایده‌ها، به بازیِ نمونه‌ای به نام «مورای دوانگشتی» (Two-Finger Morra) نگاه کنیم. در این بازی، دو بازیکن، O و E، هم‌زمان یک یا دو انگشت نشان می‌دهند. فرض کنیم شمارِ کلِ انگشتانِ نشان‌داده‌شده f باشد. اگر f فرد باشد، O از E به اندازهٔ f دلار دریافت می‌کند؛ و اگر f زوج باشد، E از O به اندازهٔ f دلار دریافت می‌کند.[1] ماتریسِ بازدهی برای مورای دوانگشتی به‌صورتِ زیر است:

	O: یک	O: دو
E: یک	E = +2، O = -2	E = -3، O = +3
E: دو	E = -3، O = +3	E = +4، O = -4

می‌گوییم E «بازیکنِ ردیف» (Row Player) و O «بازیکنِ ستون» (Column Player) است. پس، برای نمونه، گوشهٔ پایین-راست نشان می‌دهد وقتی بازیکنِ O کنشِ two و E نیز two را انتخاب می‌کند، بازدهی +4 برای E و -4 برای O است. قبل از تحلیلِ مورای دوانگشتی، ارزش دارد در نظر بگیریم اصلاً چرا به ایده‌های نظریهٔ بازی نیاز است: چرا نمی‌توانیم چالشِ پیشِ روی (مثلاً) بازیکنِ E را با ابزارِ نظریهٔ تصمیم و بیشینه‌سازیِ مطلوبیت که در جایِ دیگرِ کتاب استفاده کرده‌ایم حل کنیم؟ برای دیدنِ اینکه چرا چیزِ دیگری لازم است، فرض کنیم E در تلاش برای یافتنِ بهترین کنش است.

جایگزین‌ها one یا two هستند. اگر E one را انتخاب کند، بازدهی یا +2 خواهد بود یا -3. اینکه E در واقع کدام بازدهی را دریافت کند، با این حال، به انتخابِ O بستگی خواهد داشت: بیشترین کاری که E، به‌عنوانِ بازیکنِ ردیف، می‌تواند بکند این است که پیامدِ بازی را مجبور کند در ردیفِ خاصی باشد. به‌طورِ مشابه، O فقط ستون را انتخاب می‌کند.

برای انتخابِ بهینه میانِ این امکان‌ها، E باید در نظر بگیرد O به‌عنوانِ تصمیم‌گیرندهٔ عقلایی چگونه عمل خواهد کرد. اما O، به‌نوبهٔ خود، باید این واقعیت را در نظر بگیرد که E تصمیم‌گیرندهٔ عقلایی است. بدین‌ترتیب، تصمیم‌گیری در محیط‌های چندعاملی از نظرِ شخصیت کاملاً با تصمیم‌گیری در محیط‌های تک‌عاملی متفاوت است، زیرا بازیکنان باید

استدلالِ یکدیگر را در نظر بگیرند. نقشِ «مفاهیمِ راه‌حل» (Solution Concepts) در نظریهٔ بازی دقیق‌کردنِ این نوع استدلال است.

اصطلاحِ «استراتژی» (Strategy) در نظریهٔ بازی برای نشان‌دادنِ آنچه قبلاً «سیاست» می‌نامیدیم استفاده می‌شود. «استراتژیِ خالص» (Pure Strategy) سیاستِ جبرگرایانه است؛ برای بازیِ تک‌حرکتی، استراتژیِ خالص فقط یک کنش است. چنان‌که در زیر خواهیم دید، برای بسیاری از بازی‌ها عامل می‌تواند با «استراتژیِ ترکیبی» (Mixed Strategy) بهتر عمل کند، که سیاستِ تصادفی‌شده‌ای است که کنش‌ها را طبقِ توزیعِ احتمالی انتخاب می‌کند. استراتژیِ ترکیبی که کنشِ a را با احتمالِ p و کنشِ b را در غیرِ این صورت انتخاب می‌کند، به‌صورتِ [p:a; (1-p):b] نوشته می‌شود. برای نمونه، استراتژیِ ترکیبی برای مورای دوانگشتی ممکن است [one; 0.5:two:0.5] باشد. «نمایهٔ استراتژی» (Strategy Profile) تخصیصِ استراتژی به هر بازیکن است؛ با توجه به نمایهٔ استراتژی، پیامدِ بازی مقدارِ عددی برای هر بازیکن است—اگر بازیکنان از استراتژی‌های ترکیبی استفاده کنند، باید از مطلوبیتِ موردانتظار استفاده کنیم.

پس عامل‌ها در بازی‌هایی مانندِ مورا چگونه باید تصمیم بگیرند؟ نظریهٔ بازی طیفی از مفاهیمِ راه‌حل فراهم می‌کند که تلاش می‌کنند کنشِ عقلایی را نسبت به باورهای عامل دربارهٔ باورهای عاملِ دیگر تعریف کنند. متأسفانه، هیچ مفهومِ راه‌حلِ کاملی نیست: تعریفِ اینکه «عقلایی» وقتی هر عامل فقط بخشی از نمایهٔ استراتژیِ تعیین‌کنندهٔ پیامد را انتخاب می‌کند به چه معناست، مسئله‌دار است.

نخستین مفهومِ راه‌حل‌مان را از طریقِ احتمالاً مشهورترین بازی در قانونِ نظریهٔ بازی معرفی می‌کنیم—«دوراهیِ زندانی» (Prisoner's Dilemma). این بازی با داستانِ زیر انگیزه‌مند می‌شود: دو سارقِ متهم، علی و بو، نزدیکِ صحنهٔ سرقتی دستگیر شده‌اند و جداگانه بازجویی می‌شوند. دادستانی به هر یک معامله‌ای پیشنهاد می‌کند: اگر علیهِ شریکت به‌عنوانِ رهبرِ باندِ سرقت شهادت دهی، برای همکاری‌کردن آزاد می‌شوی، در حالی که شریکت ۱۰ سال زندان خواهد کرد. با این حال، اگر هر دو علیهِ یکدیگر شهادت دهید، هر دو ۵ سال می‌گیرید. علی و بو همچنین می‌دانند اگر هر دو از شهادت امتناع کنند، فقط ۱ سال برای اتهامِ سبک‌ترِ داشتنِ اموالِ مسروقه می‌گیرند. حالا علی و بو با به‌اصطلاحِ دوراهیِ زندانی روبه‌رویند: باید شهادت بدهند یا امتناع کنند؟ به‌عنوانِ عامل‌های عقلایی، علی و بو هر یک می‌خواهند مطلوبیتِ موردانتظارِ خودشان را بیشینه کنند، یعنی شمارِ سال‌های زندان را کمینه کنند—هر یک نسبت به رفاهِ بازیکنِ دیگر بی‌تفاوت است. دوراهیِ زندانی در ماتریسِ بازدهیِ زیر در بر گرفته می‌شود:

	علی: شهادت	علی: امتناع
بو: شهادت	A = -5، B = -5	A = -10، B = 0
بو: امتناع	A = 0، B = -10	A = -1، B = -1

(اعداد منفی سال‌های زندان‌اند.) به‌سادگی دیده می‌شود شهادت‌دادن «استراتژیِ مسلط» (Dominant Strategy) برای هر دو بازیکن است: استراتژی‌ای که بدونِ توجه به انتخابِ بازیکنِ دیگر بهترین بازدهی را می‌دهد. برای علی، اگر بو شهادت دهد، علی با شهادت -5 و با امتناع -10 می‌گیرد؛ اگر بو امتناع کند، علی با شهادت 0 و با امتناع -1 می‌گیرد. پس در هر دو حالت شهادت بهتر است.

در وضعیتی مانندِ این، که همهٔ بازیکنان استراتژیِ مسلط انتخاب می‌کنند، به پیامدِ حاصل «تعادلِ استراتژیِ مسلط» (Dominant Strategy Equilibrium) گفته می‌شود. «تعادل» است، زیرا هیچ بازیکنی انگیزه‌ای برای انحراف از بخشِ خودش از آن ندارد: طبقِ تعریف، اگر انحراف می‌کرد، نمی‌توانست بهتر عمل کند و ممکن بود بدتر عمل کند. از این نظر، تعادلِ استراتژیِ مسلط مفهومِ راه‌حلِ بسیار قوی‌ای است. با بازگشت به دوراهیِ زندانی، می‌بینیم دوراهی این است که پیامدِ تعادلِ استراتژیِ مسلط، که در آن هر دو بازیکن شهادت می‌دهند، برای هر دو بازیکن بدتر از پیامدی است که اگر هر دو امتناع می‌کردند می‌گرفتند. پیامدِ (امتناع، امتناع) به هر دو بازیکن فقط یک سال زندان می‌داد، که برای هر دویشان بهتر از ۵ سالی بود که هر یک اگر تعادلِ استراتژیِ مسلط را انتخاب می‌کردند می‌گرفتند.

آیا راهی برای رسیدنِ علی و بو به پیامدِ (امتناع، امتناع) هست؟ قطعاً برای هر دو گزینهٔ مجازِ امتناع از شهادت هست، اما سخت است ببینیم عامل‌های عقلایی چگونه می‌توانستند این انتخاب را بکنند، با توجه به نحوهٔ چیدمانِ بازی. به یاد آورید این بازیِ غیرهمکارانه است: آن‌ها اجازه ندارند با یکدیگر حرف بزنند، پس نمی‌توانند توافقِ الزام‌آوری برای امتناع بکنند. اما اگر بازی را تغییر دهیم، ممکن است به راه‌حلِ (امتناع، امتناع) رسید. می‌توانستیم آن را به بازیِ همکارانه‌ای تغییر دهیم که در آن عامل‌ها اجازه دارند توافقِ الزام‌آور تشکیل دهند. یا می‌توانستیم به بازیِ تکراری تغییر دهیم که در آن بازیکنان می‌دانند دوباره همدیگر را خواهند دید—خواهیم دید این چگونه کار می‌کند. یا، بازیکنان ممکن است باورهای اخلاقی‌ای داشته باشند که همکاری و انصاف را تشویق می‌کند. اما آن بدان معنا بود توابعِ مطلوبیتِ متفاوتی دارند، و باز، بازیِ متفاوتی بازی می‌کردند.

حضورِ استراتژیِ مسلط برای بازیکنِ خاصی فرایندِ تصمیم‌گیری را برای آن بازیکن بسیار ساده می‌کند. وقتی علی دریافته شهادت‌دادن استراتژیِ مسلط است، نیازی به صرفِ تلاش برای فهمیدنِ اینکه بو چه خواهد کرد ندارد، زیرا می‌داند مهم نیست بو چه کند، شهادت‌دادن بهترین «پاسخ» (Best Response) او خواهد بود. اما بیشترِ بازی‌ها نه استراتژیِ مسلط دارند و نه تعادلِ استراتژیِ مسلط. نادر است استراتژیِ تکی بهترین پاسخ به همهٔ استراتژی‌های ممکنِ همتایان باشد.

مفهومِ راه‌حلِ بعدی که در نظر می‌گیریم ضعیف‌تر از تعادلِ استراتژیِ مسلط است، اما بسیار گسترده‌تر قابلِ اعمال است. «تعادلِ نش» (Nash Equilibrium) نامیده می‌شود، به نامِ جان فوربس نشِ کوچک (۲۰۱۵–۱۹۲۸)، که آن را در پایان‌نامهٔ دکتراي ۱۹۵۰ خود مطالعه کرد—کاری که برایش در ۱۹۹۴ جایزهٔ نوبل دریافت کرد.

نمایهٔ استراتژی تعادلِ نش است اگر هیچ بازیکنی نتواند یک‌جانبه استراتژی‌اش را تغییر دهد و در نتیجه بازدهیِ بالاتری دریافت کند، تحتِ فرضِ اینکه بازیکنانِ دیگر در انتخاب‌های استراتژی‌شان ماندند. بدین‌ترتیب، در تعادلِ نش، هر بازیکن هم‌زمان در حالِ بازیِ بهترین پاسخ به انتخاب‌های همتایانش است. تعادلِ نش نقطهٔ پایداری در بازی را بازنمایی می‌کند: پایدار به این معنا که هیچ انگیزهٔ عقلایی‌ای برای هیچ بازیکنی به انحراف نیست. با این حال، تعادل‌های نش نقاطِ پایدارِ محلی‌اند: چنان‌که خواهیم دید، بازی ممکن است شاملِ چند تعادلِ نش باشد. چون استراتژیِ مسلط بهترین پاسخ به همهٔ استراتژی‌های همتایان است، نتیجه می‌شود هر تعادلِ استراتژیِ مسلطی باید تعادلِ نش نیز باشد (تمرین EQIB.۱۸). در دوراهیِ زندانی، بنابراین، تعادلِ استراتژیِ مسلطِ یکتایی هست، که آن نیز تعادلِ نشِ یکتاست.

بازیِ نمونهٔ زیر نشان می‌دهد، نخست، گاهی بازی‌ها هیچ استراتژیِ مسلطی ندارند، و دوم، برخی بازی‌ها چند تعادلِ نش دارند.

	علی: چپ (l)	علی: راست (r)
بو: بالا (t)	A = 10، B = 10	A = 0، B = 0
بو: پایین (b)	A = 0، B = 0	A = 1، B = 1

به‌آسانی قابلِ بررسی است در این بازی برای هیچ بازیکنی استراتژیِ مسلط نیست، و از این رو تعادلِ استراتژیِ مسلطی نیست. با این حال، نمایه‌های استراتژیِ (t,l) و (b,r) هر دو تعادلِ نش‌اند. حالا، بدیهی است در راستایِ منافعِ هر دو عامل است که هدفِ همان تعادلِ نش را بگیرند—یا (t,l) یا (b,r)—اما چون در قلمروِ نظریهٔ بازیِ غیرهمکارانه‌ایم، بازیکنان باید انتخاب‌هایشان را مستقلانه، بدونِ هیچ دانشی از انتخاب‌های دیگران و بدونِ هیچ راهی برای توافق با آن‌ها، انجام دهند. این نمونه‌ای از مسئلهٔ هماهنگی است: بازیکنان می‌خواهند کنش‌هایشان را به‌صورتِ سراسری هماهنگ کنند، به‌گونه‌ای که هر دو کنش‌هایی انتخاب کنند که به همان تعادل می‌انجامد، اما باید فقط با تصمیم‌گیریِ محلی این کار را بکنند. شماری از رویکردها برای حلِ مسائلِ هماهنگی پیشنهاد شده. یکی از ایده‌ها «نقاطِ کانونی» (Focal Points) است. نقطهٔ کانونی در بازی پیامدی است که به نحوی برای بازیکنان به‌عنوانِ پیامدِ «بدیهی» که انتخاب‌هایشان را روی آن هماهنگ کنند برجسته می‌شود. این البته تعریفِ دقیقی نیست—معنایش به بازیِ موردِ نظر بستگی خواهد داشت. در مثالِ بالا، با این حال، یک نقطهٔ کانونیِ بدیهی هست: پیامدِ (t,l) به هر دو بازیکن مطلوبیتِ به‌طورِ قابلِ‌توجهی بالاتری از آنچه اگر روی (b,r) هماهنگ می‌شدند می‌داد. از دیدگاهِ نظریهٔ بازی، هر دو پیامد تعادلِ نش‌اند—اما واقعاً بازیکنِ منحرفی می‌بود که انتظارِ هماهنگی بر (b,r) را داشته باشد.

برخی بازی‌ها در استراتژی‌های خالص هیچ تعادلِ نشی ندارند، چنان‌که بازیِ زیر به نام «پنی‌های هم‌خوان» (Matching Pennies) نشان می‌دهد. در این بازی، علی و بو هم‌زمان یک سمتِ سکه را انتخاب می‌کنند، رو یا پشت: اگر انتخاب‌های یکسانی بکنند، بو ۱ دلار به علی می‌دهد، در حالی که اگر انتخاب‌های متفاوتی بکنند، علی ۱ دلار به بو می‌دهد:

	علی: رو	علی: پشت
بو: رو	A = 1، B = -1	A = -1، B = 1
بو: پشت	A = -1، B = 1	A = 1، B = -1

از خواننده دعوت می‌کنیم بررسی کند بازی هیچ استراتژیِ مسلطی ندارد، و هیچ پیامدی تعادلِ نش در استراتژی‌های خالص نیست: در هر پیامد، یک بازیکن از انتخابش پشیمان است و ترجیح می‌داد با توجه به انتخابِ بازیکنِ دیگر متفاوت انتخاب کرده باشد.

برای یافتنِ تعادلِ نش، حقه استفاده از استراتژی‌های ترکیبی است—اجازه‌دادن به بازیکنان برای تصادفی‌کردنِ انتخاب‌هایشان. نش اثبات کرد هر بازی دستِ‌کم یک تعادلِ نش در استراتژی‌های ترکیبی دارد. این توضیح می‌دهد چرا تعادلِ نش چنین مفهومِ راه‌حلِ مهمی است: مفاهیمِ راه‌حلِ دیگر، مانندِ تعادلِ استراتژیِ مسلط، برای هر بازی تضمین‌شده وجود ندارند، اما اگر به دنبالِ تعادل‌های نش با استراتژی‌های ترکیبی باشیم همیشه راه‌حل می‌گیریم.

در موردِ پنی‌های هم‌خوان، تعادلِ نش در استراتژی‌های ترکیبی داریم اگر هر دو بازیکن رو و پشت را با احتمالِ برابر انتخاب کنند. برای دیدنِ اینکه این پیامد در واقع تعادلِ نش است، فرض کنیم یکی از بازیکنان پیامدی را با احتمالی غیر از 0.5 انتخاب کرد. آن‌گاه بازیکنِ دیگر می‌توانست از آن بهره‌برداری کند، با گذاشتنِ تمامِ وزنش پشتِ استراتژیِ خاصی. برای نمونه، فرض کنیم بو رو را با احتمالِ 0.6 بازی کند (و بنابراین پشت را با احتمالِ 0.4). آن‌گاه علی بهترین کارش را با بازیِ قطعیِ رو می‌کرد. سپس به‌آسانی دیده می‌شود بو با بازیِ رو با احتمالِ 0.6 نمی‌توانست بخشی از هیچ تعادلِ نشی باشد.

۱۸.۲.۲ رفاهِ اجتماعی

دیدگاهِ اصلی در نظریهٔ بازی دیدگاهِ بازیکنانِ درونِ بازی است، که می‌کوشند بهترین پیامدهای ممکن را برای خودشان بگیرند. با این حال، گاهی اتخاذِ دیدگاهِ متفاوت آموزنده است. فرض کنیم نهادِ خیرخواه و دانایِ مطلقی باشید که از بالا به بازی نگاه می‌کند و می‌توانید پیامد را انتخاب کنید. از آن‌جا که خیرخواه هستید، می‌خواهید بهترین پیامدِ کلی را انتخاب کنید—پیامدی که، به بیانِ دیگر، برای کلِ جامعه بهترین باشد.

چگونه باید انتخاب کنید؟ چه معیارهایی ممکن است اعمال کنید؟ این‌جاست که مفهومِ «رفاهِ اجتماعی» (Social Welfare) وارد می‌شود. احتمالاً مهم‌ترین و کم‌چالش‌ترین معیارِ رفاهِ اجتماعی این است که باید از پیامدهایی که مطلوبیت را هدر می‌دهند اجتناب کنید. این الزام در مفهومِ «بهینگیِ پارتو» (Pareto Optimality) در بر گرفته شده، که به نامِ اقتصاددانِ ایتالیاییِ ویلفردو پارتو (۱۹۲۳–۱۸۴۸) نام‌گذاری شده. پیامد پارتو-بهینه است اگر هیچ پیامدِ دیگری نباشد که بازیکنی را بدونِ بدترکردنِ دیگری بهتر کند. اگر پیامدی را انتخاب کنید که پارتو-بهینه نیست، مطلوبیت را به این معنا هدر می‌دهید که می‌توانستید به دستِ‌کم یک عامل مطلوبیتِ بیشتری بدهید، بدونِ گرفتن از عامل‌های دیگر.

«رفاهِ اجتماعیِ فایده‌گرایانه» (Utilitarian Social Welfare) معیاری از خوب‌بودنِ پیامد در کل است. رفاهِ اجتماعیِ فایده‌گرایانهٔ پیامد صرفاً جمعِ مطلوبیت‌های داده‌شده به بازیکنان توسط آن پیامد است. دو دشواریِ کلیدی با رفاهِ اجتماعیِ فایده‌گرایانه هست. نخستین این است که جمع را در نظر می‌گیرد اما نه توزیعِ مطلوبیت‌ها را میانِ بازیکنان، پس می‌تواند به توزیعِ بسیار نابرابری بینجامد اگر اتفاقاً جمع را بیشینه کند. دشواریِ دوم این است که مقیاسِ مشترکی برای مطلوبیت‌ها فرض می‌کند. بسیاری از اقتصاددانان استدلال می‌کنند برقراریِ این ناممکن است، زیرا مطلوبیت (برخلافِ پول) کمیتِ ذهنی است. اگر می‌خواهیم تصمیم بگیریم دسته‌ای کلوچه را چگونه تقسیم کنیم، آیا باید همه‌شان را به «هیولایِ مطلوبیت» بدهیم که می‌گوید «من کلوچه را هزار برابر هر کسِ دیگر دوست دارم»؟ آن مطلوبیتِ کلِ خودگزارش‌شده را بیشینه می‌کرد، اما درست به نظر نمی‌رسد.

پرسشِ چگونگیِ توزیعِ مطلوبیت میانِ بازیکنان توسط پژوهشِ «رفاهِ اجتماعیِ برابری‌گرایانه» (Egalitarian Social Welfare) پرداخته می‌شود. برای نمونه، پیشنهادی پیشنهاد می‌کند مطلوبیتِ موردانتظارِ بدترین‌حالِ عضوِ جامعه را بیشینه کنیم—رویکردِ بیشینه‌حداقلی (Maximin). معیارهای دیگر نیز ممکن‌اند، از جملهِ «ضریبِ جینی» (Gini Coefficient)، که خلاصه می‌کند مطلوبیت چقدر یکنواخت میانِ بازیکنان پخش شده. دشواریِ اصلیِ چنین پیشنهادهایی این است که ممکن است رفاهِ کلِ زیادی را برای سودهای توزیعیِ کوچک قربانی کنند، و، مانندِ فایده‌گراییِ ساده، همچنان در رحمتِ هیولایِ مطلوبیت‌اند.

اعمالِ این مفاهیم بر بازیِ دوراهیِ زندانیِ معرفی‌شده در بالا توضیح می‌دهد چرا دوراهی نامیده می‌شود. به یاد آورید (شهادت، شهادت) تعادلِ استراتژیِ مسلط و تنها تعادلِ نش است. با این حال، این تنها پیامدی است که پارتو-بهینه نیست. پیامدِ (امتناع، امتناع) هم رفاهِ اجتماعیِ فایده‌گرایانه و هم برابری‌گرایانه را بیشینه می‌کند. دوراهی در دوراهیِ زندانی بدین‌ترتیب پدید می‌آید، زیرا مفهومِ راه‌حلِ بسیار قوی (تعادلِ استراتژیِ مسلط) به پیامدی می‌انجامد که اساساً در هر آزمونِ معقول‌بودنِ پیامد از دیدگاهِ «جامعه» شکست می‌خورد. و با این حال هیچ راهِ روشنی برای عامل‌های فردی برای رسیدن به راه‌حلِ بهتر نیست.

محاسبهٔ تعادل‌ها. حالا پرسش‌های محاسباتیِ کلیدیِ مرتبط با مفاهیمِ بالا را در نظر بگیریم. نخست استراتژی‌های خالص را در نظر می‌گیریم، جایی که تصادفی‌سازی مجاز نیست. اگر بازیکنان فقط شمارِ متناهی انتخابِ ممکن داشته باشند، آن‌گاه می‌توان از جستجوی جامع برای یافتنِ تعادل‌ها استفاده کرد: از میانِ هر نمایهٔ استراتژیِ ممکن تکرار کن و بررسی کن آیا بازیکنی انحرافِ سودمندی از آن نمایه دارد؛ اگر نه، تعادلِ نش در استراتژی‌های خالص است. استراتژی‌های مسلط و تعادل‌های استراتژیِ مسلط را می‌توان با الگوریتم‌های مشابهی محاسبه کرد. متأسفانه، شمارِ نمایه‌های استراتژیِ ممکن برای n بازیکن که هر یک m کنشِ ممکن دارند، m^n است، یعنی برای جستجوی جامع به‌طرزِ ناشدنی بزرگ.

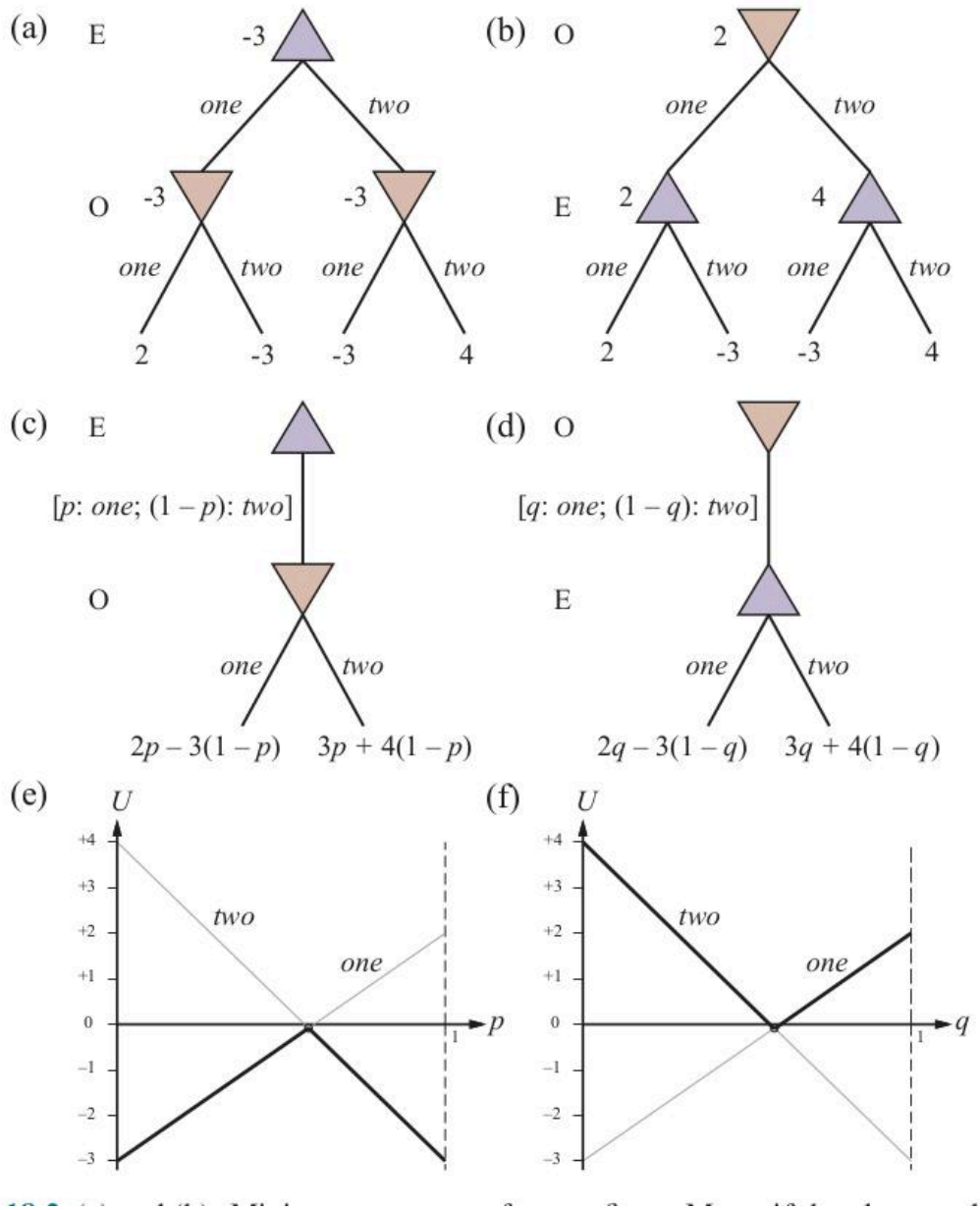
رویکردِ جایگزین، که در برخی بازی‌ها خوب کار می‌کند، «بهترین پاسخِ کوته‌نگر» (Myopic Best Response) است (که «بهترین پاسخِ تکرارشونده» نیز نامیده می‌شود): با انتخابِ تصادفیِ نمایهٔ استراتژی شروع کن؛ سپس، اگر بازیکنی بهترین انتخابش را با توجه به انتخاب‌های دیگران بازی نمی‌کند، انتخابش را به انتخابِ بهینه برگردان، و فرایند را تکرار کن. فرایند همگرا خواهد شد اگر به نمایهٔ استراتژی‌ای منتهی شود که در آن هر بازیکن با توجه به انتخاب‌های دیگران انتخابِ بهینه‌ای می‌کند—یعنی تعادلِ نش. برای برخی بازی‌ها، بهترین پاسخِ کوته‌نگر همگرا نمی‌شود، اما برای برخی رده‌های مهمِ بازی‌ها، تضمین‌شده همگرا می‌شود.

محاسبهٔ تعادل‌های استراتژیِ ترکیبی از نظرِ الگوریتمی بسیار پیچیده‌تر است. برای ساده‌نگه‌داشتنِ امور، روی روش‌های بازی‌های جمعِ صفر تمرکز می‌کنیم و در پایانِ این بخش به‌طورِ مختصر به بسطِ آن‌ها به بازی‌های دیگر اشاره می‌کنیم.

در ۱۹۲۸، فون نویمان روشی برای یافتنِ استراتژیِ ترکیبیِ بهینه برای بازی‌های جمعِ صفرِ دونفره توسعه داد—بازی‌هایی که در آن‌ها بازدهی‌ها همیشه به صفر جمع می‌شوند (یا به ثابت، چنان‌که در صفحهٔ ۱۴۷ توضیح داده شد). بدیهی است، مورا چنین بازی‌ای است. برای بازی‌های جمعِ صفرِ دونفره، می‌دانیم بازدهی‌ها برابر و متضادند، پس لازم است بازدهی‌های فقط یک بازیکن را در نظر بگیریم، که بیشینه‌کننده خواهد بود (درست مانندِ فصل ۵). برای مورا، بازیکنِ زوجِ E را بیشینه‌کننده برمی‌گزینیم، پس می‌توانیم ماتریسِ بازدهی را با مقادیرِ UE(e,o)—بازدهیِ E اگر E کنشِ e و O

کنشِ o انجام دهد—تعریف کنیم. (برای تفکیک و راحتی، در متن اصلی از ضمیر مؤنث برای بازیکنِ E و ضمیر مذکر برای O استفاده شده که در این ترجمه با عنوان «بازیکنِ زوج E» و «بازیکنِ فرد O» مشخص می‌شوند.) روشِ فون نویمان «فنِ بیشینه‌حداقلی» (Maximin Technique) نامیده می‌شود و چنین کار می‌کند:

- فرض کنیم قاعده‌ها را چنین تغییر دهیم: نخست E استراتژی‌اش را برمی‌گزیند و به O فاش می‌کند. سپس O استراتژی‌اش را، با دانشِ استراتژیِ E، برمی‌گزیند. سرانجام، بازدهیِ موردانتظارِ بازی را بر اساسِ استراتژی‌های انتخاب‌شده ارزیابی می‌کنیم. این بازیِ نوبتی به ما می‌دهد که می‌توانیم الگوریتمِ استانداردِ مینیمکس را از فصل ۵ روی آن اعمال کنیم. فرض کنیم این پیامدِ UE,O بدهد. بدیهی است، این بازی به نفعِ O است، پس مطلوبیتِ واقعیِ U بازیِ اصلی (از دیدگاهِ E) دستِ‌کم UE,O است. برای نمونه، اگر فقط به استراتژی‌های خالص نگاه کنیم، درختِ بازیِ مینیمکس مقدارِ ریشه‌ای -3 دارد (شکل ۱۸.۲(الف) را ببینید)، پس می‌دانیم U ≥ -3.
- حالا فرض کنیم قاعده‌ها را طوری تغییر دهیم که O مجبور شود اول استراتژی‌اش را فاش کند و سپس E. با تقارن، این بازی به نفعِ E است، و مطلوبیتِ واقعیِ U باید دستِ‌کم به‌اندازهٔ ارزشِ این بازی باشد. با مینیمکس، ارزش این بازی را می‌توان محاسبه کرد (شکل ۱۸.۲(ب)).



شکل ۱۸.۲ – (الف) و (ب): درخت‌های بازیِ مینیمکس برای مورای دوانگشتی اگر بازیکنان به‌نوبت استراتژی‌های خالص بازی کنند. (ج) و (د): درخت‌های بازیِ پارامتری که در آن‌ها بازیکنِ نخست استراتژیِ ترکیبی بازی می‌کند. بازدهی‌ها به پارامترِ احتمالِ p یا q در استراتژیِ ترکیبی بستگی دارند. (ه) و (و): برای برخی مقادیرِ p و q، بیشینه‌حداقلی به بازدهیِ موردانتظارِ -1/12 می‌انجامد.

در این نقطه، ارزشِ بازیِ اصلی بینِ -1/12 و -1/12 است؛ یعنی دقیقاً -1/12 است. (نتیجه این است که اگر این بازی را بازی می‌کنید، بهتر است O باشید تا E.) علاوه بر این، مطلوبیتِ واقعی با استراتژیِ ترکیبیِ [one; 5/12:two:7/12] به دست می‌آید، که هر دو بازیکن باید آن را بازی کنند. این استراتژی «تعادلِ بیشینه‌حداقلی» (Maximin Equilibrium) بازی نامیده می‌شود و تعادلِ نش است. توجه کنید هر استراتژیِ مؤلفه‌ای در استراتژیِ ترکیبیِ تعادلی همان مطلوبیتِ موردانتظار را دارد. در این مورد، هم one و هم two همان مطلوبیتِ موردانتظار، -1/12، را به‌عنوانِ خودِ استراتژیِ ترکیبی دارند.[۲]

نتیجهٔ ما برای مورای دوانگشتی نمونه‌ای از نتیجهٔ عمومیِ فون نویمان است: هر بازیِ جمعِ صفرِ دونفره‌ای وقتی استراتژی‌های ترکیبی را اجازه دهید تعادلِ بیشینه‌حداقلی دارد. علاوه بر این، هر تعادلِ نشی در بازیِ جمعِ صفر برای هر دو بازیکن بیشینه‌حداقلی است. بازیکنی که استراتژیِ بیشینه‌حداقلی را اتخاذ کند دو تضمین دارد: نخست، هیچ استراتژیِ دیگری نمی‌تواند در برابرِ حریفی که خوب بازی می‌کند بهتر عمل کند (هرچند برخی استراتژی‌های دیگر ممکن است در بهره‌برداری از حریفی که اشتباه‌های غیرعقلایی می‌کند بهتر باشند). دوم، بازیکن حتی اگر استراتژی فاش شود نیز به‌همان خوبی ادامه می‌دهد.

الگوریتمِ عمومیِ یافتنِ تعادل‌های بیشینه‌حداقلی در بازی‌های جمعِ صفر تا حدی درگیرتر از آن چیزی است که شکل‌های ۱۸.۲(ه) و (و) پیشنهاد می‌کنند. وقتی n کنشِ ممکن هست، استراتژیِ ترکیبی نقطه‌ای در فضایِ n-بعدی است و خط‌ها ابرصفحه می‌شوند. همچنین ممکن است برخی استراتژی‌های خالص برای بازیکنِ دوم توسط دیگران مسلط شوند، به‌گونه‌ای که در برابرِ هیچ استراتژیِ بازیکنِ نخست بهینه نباشند. پس از حذفِ چنین استراتژی‌هایی (که ممکن است لازم باشد مکرراً انجام شود)، انتخابِ بهینه در ریشه، بالاترین (یا پایین‌ترین) نقطهٔ تقاطعِ ابرصفحه‌های باقی‌مانده است.

یافتنِ این انتخاب نمونه‌ای از مسئلهٔ برنامه‌نویسیِ خطی است: بیشینه‌کردنِ تابعِ هدف مشروط به قیدهای خطی. چنین مسائلی را می‌توان با فنونِ استاندارد در زمانِ چندجمله‌ای در شمارِ کنش‌ها حل کرد (و در شمارِ بیت‌های استفاده‌شده برای مشخص‌کردنِ تابعِ پاداش، اگر بخواهید فنی شوید).

پرسش باقی می‌ماند که عاملِ عقلایی هنگامِ بازیِ تک‌بارهٔ مورا واقعاً چه باید بکند. عاملِ عقلایی این واقعیت را اشتقاق کرده که [one; 5/12:two:7/12] استراتژیِ تعادلِ بیشینه‌حداقلی است، و فرض می‌کند این دانشِ دوجانبه با حریفِ عقلایی است. عامل می‌توانست از تاسِ ۱۲-وجهی یا مولدِ اعدادِ تصادفی برای انتخابِ تصادفی طبقِ این استراتژیِ ترکیبی استفاده کند، که در آن صورت بازدهیِ موردانتظار -1/12 برای E می‌بود. یا عامل می‌توانست صرفاً تصمیم بگیرد one یا two بازی کند. در هر صورت، بازدهیِ موردانتظار -1/12 برای E باقی می‌ماند. جالب است که انتخابِ یک‌جانبهٔ کنشِ خاص به بازدهیِ موردانتظار آسیب نمی‌زند، اما اجازه‌دادن به عاملِ دیگر برای دانستنِ اینکه چنین انتخابِ یک‌جانبه‌ای کرده‌اید بر بازدهیِ موردانتظار اثر می‌گذارد، زیرا آن‌گاه حریف می‌تواند استراتژی‌اش را متناسباً تنظیم کند.

یافتنِ تعادل‌ها در بازی‌های غیرجمع‌صفر تا حدی پیچیده‌تر است. رویکردِ عمومی دو گام دارد: (۱) همهٔ زیرمجموعه‌های ممکنِ کنش‌هایی را که ممکن است استراتژی‌های ترکیبی تشکیل دهند برشمار. برای نمونه، نخست همهٔ نمایه‌های استراتژی را امتحان کن که در آن‌ها هر بازیکن از کنشِ تکی استفاده می‌کند، سپس آن‌هایی را که در آن‌ها هر بازیکن از یک یا دو کنش استفاده می‌کند، و همین‌طور. این در شمارِ کنش‌ها نمایی است، و بنابراین فقط برای بازی‌های نسبتاً کوچک اعمال می‌شود. (۲) برای هر نمایهٔ استراتژیِ شمارش‌شده در (۱)، بررسی کن آیا تعادل است. این با حلِ مجموعه‌ای از معادله‌ها و نامساوی‌ها انجام می‌شود که مشابهِ آن‌هایی‌اند که در حالتِ جمعِ صفر استفاده شدند. برای دو بازیکن این معادله‌ها خطی‌اند و با فنونِ پایهٔ برنامه‌نویسیِ خطی قابلِ حل‌اند، اما برای سه بازیکن یا بیشتر غیرخطی‌اند و ممکن است حل‌شدنشان بسیار دشوار باشد.

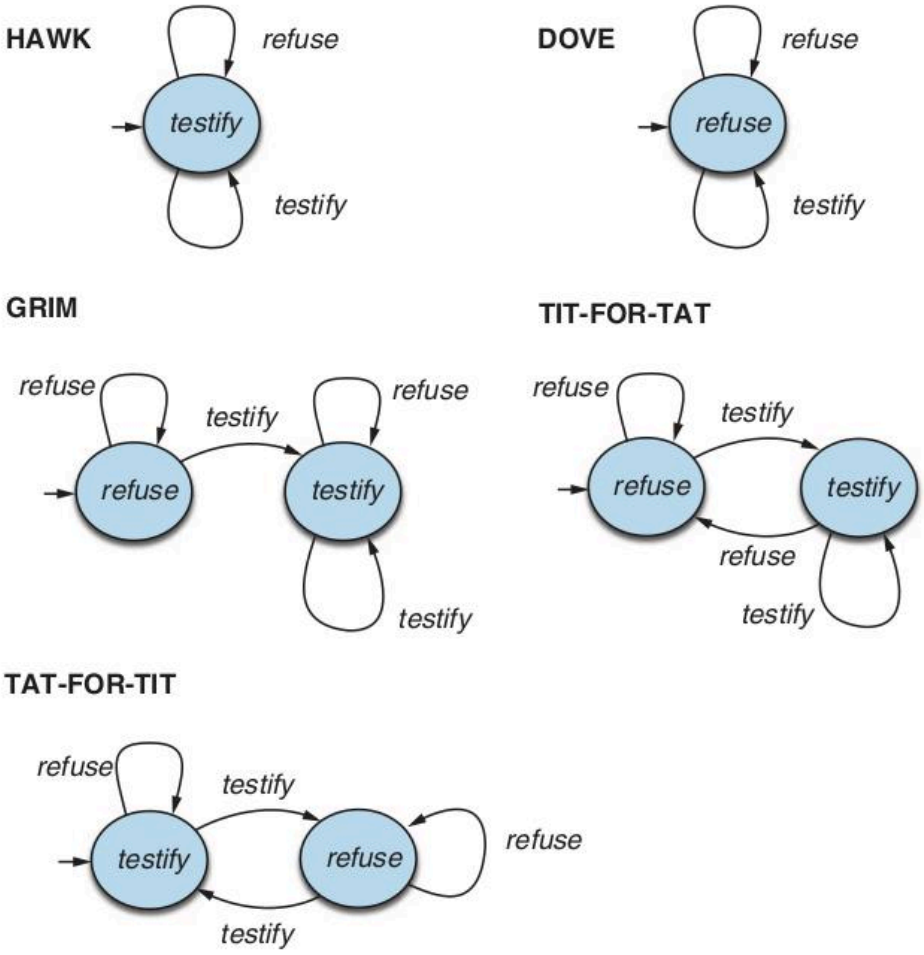
۱۸.۲.۳ بازی‌های تکراری

تاکنون فقط به بازی‌هایی نگاه کرده‌ایم که یک حرکت طول می‌کشند. ساده‌ترین نوعِ بازیِ چندحرکتی «بازیِ تکراری» (Repeated Game) است (که «بازیِ تکرارشونده» نیز نامیده می‌شود)، که در آن بازیکنان مکرراً دورهای یک بازیِ تک‌حرکتی را بازی می‌کنند، که «بازیِ مرحله‌ای» (Stage Game) نامیده می‌شود. استراتژی در بازیِ تکراری، انتخابِ کنش را برای هر بازیکن در هر گامِ زمانی برای هر تاریخچهٔ ممکنِ انتخاب‌های قبلیِ بازیکنان مشخص می‌کند. نخست، به حالتی نگاه کنیم که بازیِ مرحله‌ای برای شمارِ ثابت، متناهی و دوجانبه‌ی معلومِ دورها تکرار می‌شود—همهٔ این شرایط برای کارکردنِ تحلیلِ زیر لازم‌اند. فرض کنیم علی و بو نسخهٔ تکراریِ دوراهیِ زندانی را بازی می‌کنند، و هر دو می‌دانند باید دقیقاً ۱۰۰ دور از بازی را بازی کنند. در هر دور، از آن‌ها پرسیده می‌شود شهادت بدهند یا امتناع کنند، و طبقِ قاعده‌های دوراهیِ زندانی که بالا دیدیم بازدهیِ آن دور را دریافت می‌کنند.

در پایانِ ۱۰۰ دور، بازدهیِ کلیِ هر بازیکن را با جمع‌کردنِ بازدهی‌های آن بازیکن در ۱۰۰ دور می‌یابیم. علی و بو چه استراتژی‌هایی باید برای بازی انتخاب کنند؟ استدلالِ زیر را در نظر بگیرید. هر دو می‌دانند دورِ ۱۰۰م بازیِ تکراری نخواهد بود—یعنی پیامدش نمی‌تواند بر دورهای آینده اثر بگذارد. پس، در دورِ ۱۰۰م، در عمل در حالِ بازیِ تکیِ دوراهیِ زندانی‌اند.

چنان‌که بالا دیدیم، پیامدِ دورِ ۱۰۰م (شهادت، شهادت) خواهد بود، استراتژیِ تعادلیِ مسلط برای هر دو بازیکن. اما وقتی دورِ ۱۰۰م تعیین شد، دورِ ۹۹م نمی‌تواند بر دورهای بعدی اثر بگذارد، پس آن نیز (شهادت، شهادت) خواهد داد. با این استدلالِ استقرایی، هر دو بازیکن در هر دور شهادت را انتخاب خواهند کرد و در مجموع ۵۰۰ سال زندان برای هر یک به دست می‌آورند. این نوع استدلال «استقرایِ عقب‌رو» (Backward Induction) نامیده می‌شود و نقشِ بنیادین در نظریهٔ بازی بازی می‌کند. با این حال، اگر یکی از سه شرط—ثابت، متناهی یا دوجانبه‌ی معلوم—را حذف کنیم، استدلالِ استقرایی برقرار نمی‌ماند. فرض کنیم بازی شمارِ نامتناهی بار تکرار شود. از نظرِ ریاضی، استراتژیِ بازیکن در بازیِ بی‌نهایت‌تکراری تابعی است که هر تاریخچهٔ متناهیِ ممکنِ بازی را به انتخابی در بازیِ مرحله‌ای برای آن بازیکن در دورِ متناظر نگاشت می‌کند. بدین‌ترتیب، استراتژی به آنچه قبلاً در بازی رخ داده نگاه می‌کند و تصمیم می‌گیرد در دورِ کنونی چه انتخابی بکند. اما نمی‌توانیم جدولِ نامتناهی را در رایانهٔ متناهی ذخیره کنیم. برای بازی‌هایی که شمارِ نامتناهی دور بازی خواهند شد به مدلِ متناهیِ استراتژی‌ها نیاز داریم. به همین دلیل، مرسوم است استراتژی‌های بازی‌های بی‌نهایت‌تکراری را به‌صورتِ «ماشین‌های حالتِ متناهی» (Finite State Machines یا FSM) با خروجی بازنمایی کنیم.

شکل ۱۸.۳ شماری از استراتژی‌های FSM را برای دوراهیِ زندانیِ تکرارشونده نشان می‌دهد.



شکل ۱۸.۳ – برخی استراتژی‌های ماشینِ حالتِ متناهیِ رایج با نام‌های رنگارنگ برای دوراهیِ زندانیِ بی‌نهایت‌تکراری: HAWK (شاهین)، GRIM (سنگین‌گیر)، TIT-FOR-TAT، TAT-FOR-TIT (چشم‌دربرابرچشم) و DOVE (کبوتر).

استراتژیِ «چشم‌دربرابرچشم» (Tit-for-Tat) را در نظر بگیرید. هر بیضی حالتی از ماشین است، و درونِ بیضی انتخابی است که استراتژی اگر ماشین در آن حالت بود می‌کرد. از هر حالت، برای هر انتخابِ ممکنِ عاملِ همتا یک یالِ خروجی داریم: از یالِ خروجیِ متناظر با انتخابِ انجام‌شده توسط دیگری پیروی می‌کنیم تا حالتِ بعدیِ ماشین را بیابیم. سرانجام، یک حالت با پیکانِ ورودی برچسب‌گذاری می‌شود، که نشان می‌دهد حالتِ اولیه است. بدین‌ترتیب، با TIT-FOR-TAT، ماشین در حالتِ امتناع شروع می‌شود؛ اگر عاملِ همتا امتناع بازی کند، در حالتِ امتناع می‌ماند، در حالی که اگر همتا شهادت بازی کند، به حالتِ شهادت گذار می‌کند. تا زمانی که همتایش شهادت بازی کند در حالتِ شهادت خواهد ماند، اما اگر همتایش امتناع بازی کند، به حالتِ امتناع بازخواهد گشت. به‌طورِ خلاصه، TIT-FOR-TAT با انتخابِ امتناع شروع می‌کند و سپس صرفاً هرچه همتایش در دورِ قبلی کرد کپی می‌کند.

استراتژی‌های HAWK و DOVE ساده‌ترند: HAWK صرفاً در هر دور شهادت بازی می‌کند، در حالی که DOVE صرفاً در هر دور امتناع بازی می‌کند. استراتژیِ GRIM تا حدی شبیهِ TIT-FOR-TAT است، اما با یک تفاوتِ مهم: اگر همتایش شهادت بازی کند، اساساً به HAWK تبدیل می‌شود: برای همیشه شهادت بازی می‌کند. در حالی که TIT-FOR-TAT بخشنده است، به این معنا که به امتناعِ بعدی با متقابل‌سازیِ همان پاسخ می‌دهد، با GRIM هیچ راهِ بازگشتی نیست. فقط یک‌بار شهادت‌بازی‌کردن به مجازاتی (بازیِ شهادت) می‌انجامد که برای همیشه ادامه می‌یابد. (می‌توانید ببینید TAT-FOR-TIT چه می‌کند؟)

مسئلهٔ بعدیِ بازی‌های بی‌نهایت‌تکراری این است که مطلوبیتِ توالیِ نامتناهی از بازدهی‌ها را چگونه اندازه‌گیری کنیم. در این‌جا، روی رویکردِ «حدِّ میانگین‌ها» (Limit of Means) تمرکز می‌کنیم—اساساً، یعنی گرفتنِ میانگینِ مطلوبیت‌های دریافت‌شده روی توالیِ نامتناهی. با این رویکرد، با توجه به توالیِ نامتناهی از بازدهی‌ها (U0,U1,U2,...)، مطلوبیتِ توالی را برای بازیکنِ متناظر چنین تعریف می‌کنیم:

```
lim_{T→∞} (1/T) ∑_{t=0}^{T} Ut
```

این مقدار برای توالی‌های دلخواهِ مطلوبیت تضمین‌شده همگرا نیست، اما برای توالی‌های مطلوبیتی که اگر از استراتژی‌های FSM استفاده کنیم تولید می‌شوند تضمین‌شده همگراست. برای دیدنِ این، مشاهده کنید اگر استراتژی‌های FSM با یکدیگر بازی کنند، سرانجام، FSMها به پیکربندی‌ای که قبلاً در آن بودند دوباره وارد خواهند شد، که در آن نقطه شروع به تکرارِ خودشان می‌کنند. دقیق‌تر، هر توالیِ مطلوبیتی که توسط استراتژی‌های FSM تولید شود شاملِ توالیِ متناهی (احتمالاً خالی) غیرتکراری، به‌دنبالِ توالیِ متناهیِ غیرخالی است که بی‌نهایت بار تکرار می‌شود. برای محاسبهٔ میانگینِ مطلوبیتِ دریافت‌شده توسط بازیکن روی آن توالیِ نامتناهی، صرفاً باید میانگینِ توالیِ متناهیِ تکراری را محاسبه کنیم.

در آنچه در ادامه می‌آید، فرض می‌کنیم بازیکنان در بازیِ بی‌نهایت‌تکراری صرفاً ماشینِ حالتِ متناهی‌ای انتخاب می‌کنند که بازی را از جانبِ آن‌ها انجام دهد. هیچ قیدی بر این ماشین‌ها تحمیل نمی‌کنیم: آن‌ها می‌توانند به‌اندازه‌ای که بازیکنان می‌خواهند بزرگ و پرکار باشند. وقتی همهٔ بازیکنان ماشینِ حالتِ متناهی‌ای انتخاب کردند که از جانبِ آن‌ها بازی کند، می‌توانیم بازدهی‌های هر بازیکن را با رویکردِ حدِّ میانگین‌ها چنان‌که بالا توصیف شد محاسبه کنیم. بدین‌گونه، بازیِ بی‌نهایت‌تکراری به بازیِ شکلِ نرمال کاهش می‌یابد، هرچند بازی‌ای با بی‌نهایت استراتژیِ ممکن برای هر بازیکن. ببینیم وقتی دوراهیِ زندانیِ بی‌نهایت‌تکراری را با برخی استراتژی‌های شکل ۱۸.۳ بازی می‌کنیم چه اتفاقی می‌افتد. نخست، فرض کنیم علی و بو هر دو DOVE را انتخاب کنند:

```
... علی: DOVE   امتناع امتناع امتناع امتناع امتناع امتناع ...   مطلوبیت = -1
... بو: DOVE    امتناع امتناع امتناع امتناع امتناع امتناع ...   مطلوبیت = -1
```

دشوار نیست ببینیم این جفتِ استراتژی تعادلِ نش تشکیل نمی‌دهد: هر بازیکنی با تغییرِ انتخابش به HAWK بهتر عمل می‌کرد. پس، فرض کنیم علی به HAWK سوئیچ کند:

```
... علی: HAWK   شهادت شهادت شهادت شهادت شهادت شهادت ...   مطلوبیت = 0
... بو: DOVE    امتناع امتناع امتناع امتناع امتناع امتناع ...   مطلوبیت = -10
```

این بدترین پیامدِ ممکن برای بوست؛ و این جفتِ استراتژی باز تعادلِ نش نیست. بو با انتخابِ HAWK نیز بهتر عمل می‌کرد:

```
... علی: HAWK   شهادت شهادت شهادت شهادت شهادت شهادت ...   مطلوبیت = -5
... بو: HAWK    شهادت شهادت شهادت شهادت شهادت شهادت ...   مطلوبیت = -5
```

این جفتِ استراتژی تعادلِ نش تشکیل می‌دهد، اما چندان جالبی نیست—ما را کم‌وبیش به جایی که در نسخهٔ تک‌بارهٔ بازی شروع کردیم بازمی‌گرداند، با شهادت‌دادنِ هر دو بازیکن علیهِ یکدیگر. ویژگیِ کلیدیِ بازی‌های بی‌نهایت‌تکراری را نشان می‌دهد: تعادل‌های نشِ بازیِ مرحله‌ای به‌عنوانِ تعادل در نسخهٔ بی‌نهایت‌تکرارِیِ بازی حفظ خواهند شد. اما داستانِ ما هنوز تمام نشده. فرض کنیم بو به GRIM سوئیچ کند:

```
... علی: HAWK   شهادت شهادت شهادت شهادت شهادت شهادت ...   مطلوبیت = -5
... بو: GRIM    امتناع شهادت شهادت شهادت شهادت شهادت ...   مطلوبیت = -5
```

در این‌جا، بو بدتر از بازیِ HAWK عمل نمی‌کند: در دورِ نخست، علی شهادت بازی می‌کند در حالی که بو امتناع بازی می‌کند، اما این بو را به شهادت‌بازیِ همیشگی بعد از آن وادار می‌کند: ازدست‌رفتنِ مطلوبیت در دورِ نخست در حدِّ نهایی ناپدید می‌شود. در کل، دو بازیکن همان مطلوبیتی را می‌گیرند که گویی هر دو HAWK بازی کرده‌اند. اما نکته این‌جاست: این استراتژی‌ها تعادلِ نش تشکیل نمی‌دهند، زیرا این بار علی انحرافِ سودمندی دارد—به GRIM. اگر هر دو بازیکن GRIM را انتخاب کنند، این اتفاق می‌افتد:

```
... علی: GRIM   امتناع امتناع امتناع امتناع امتناع امتناع ...   مطلوبیت = -1
... بو: GRIM    امتناع امتناع امتناع امتناع امتناع امتناع ...   مطلوبیت = -1
```

پیامدها و بازدهی‌ها همان‌اند که گویی هر دو بازیکن DOVE انتخاب کرده‌اند، اما برخلافِ آن مورد، GRIM در برابرِ GRIM تعادلِ نش تشکیل می‌دهد، و علی و بو قادرند به‌صورتِ عقلایی به پیامدی برسند که در نسخهٔ تک‌بارهٔ بازی ناممکن است.

برای دیدنِ اینکه این استراتژی‌ها تعادلِ نش تشکیل می‌دهند، برای تناقض فرض کنیم تشکیل نمی‌دهند. سپس یکی از بازیکنان—بدونِ ازدست‌دادنِ عمومیت فرض کنید علی است—انحرافِ سودمندی دارد، در قالبِ استراتژیِ FSM ای که بازدهیِ بالاتری از GRIM می‌داد. حالا، در نقطه‌ای، این استراتژی باید کاری متفاوت از GRIM می‌کرد—در غیرِ این صورت همان مطلوبیت را می‌گرفت. پس، در نقطه‌ای باید شهادت بازی کند. اما سپس استراتژیِ GRIM بو به حالتِ مجازات می‌چرخید، با شهادت‌بازیِ دائمی در پاسخ. در آن نقطه، علی محکوم به دریافتِ بازدهیِ حداکثر -5 می‌شد: بدتر از -1 که با انتخابِ GRIM می‌گرفت. بدین‌ترتیب، انتخابِ GRIM توسط هر دو بازیکن در دوراهیِ زندانیِ بی‌نهایت‌تکراری تعادلِ نش تشکیل می‌دهد، و پیامدی عقلاییِ پایدار می‌دهد که در نسخهٔ تک‌بارهٔ بازی ناممکن است.

این نمونه‌ای از ردهٔ عمومیِ نتایجی به نامِ «قضیه‌های عامیانهٔ نش» (Nash Folk Theorems) است، که پیامدهای قابلِ‌حفظ‌شده توسط تعادل‌های نش در بازی‌های بی‌نهایت‌تکراری را مشخصه‌سازی می‌کنند. فرض کنیم «ارزشِ امنیتیِ» بازیکن بهترین بازدهی‌ای باشد که بازیکن می‌توانست به‌دست‌آوردنش را تضمین کند. سپس شکلِ عمومیِ قضیه‌های عامیانهٔ نش تقریباً این است که هر پیامدی که در آن هر بازیکن دست‌کم ارزشِ امنیتی‌اش را دریافت کند، می‌تواند به‌عنوانِ تعادلِ نش در بازیِ بی‌نهایت‌تکراری حفظ شود. استراتژی‌های GRIM کلیدِ قضیه‌های عامیانه‌اند: تهدیدِ دوجانبهٔ مجازات اگر هر عاملی از بازی‌کردنِ نقشش در پیامدِ موردنظر ناکام بماند، بازیکنان را در خط نگه می‌دارد. اما فقط به‌عنوانِ بازدارنده کار می‌کند اگر بازیکنِ دیگر باور کند این استراتژی را اتخاذ کرده‌اید—یا دست‌کم که ممکن است اتخاذ کرده باشید.

همچنین می‌توانیم با تغییرِ عامل‌ها، به‌جای تغییرِ قاعده‌های درگیری، راه‌حل‌های متفاوتی بگیریم. فرض کنیم عامل‌ها ماشین‌های حالتِ متناهی‌اند (به جای اینکه فقط از آن‌ها استفاده کنند). اگر عامل‌ها خودِ ماشین‌های حالتِ متناهی باشند، تحلیلِ تعادل‌های نش در بازی‌های بی‌نهایت‌تکراری به‌طورِ چشمگیری تغییر می‌کند: فقط باید راه‌حل‌هایی را در نظر بگیریم که در آن‌ها هر بازیکن ماشینی با اندازه‌ای محدود است. تعادل‌هایی که با این فرض به دست می‌آیند، تعادل‌های «ماشینِ حالتِ متناهی» (Finite-State Machine Equilibria) نامیده می‌شوند. در چنین تعادلی، بازیکنی با ماشینِ کوچکتر نمی‌تواند بهتر از بازیکنی با ماشینِ بزرگتر عمل کند—ماشین‌های کوچک‌تر به‌عنوانِ راه‌حل ترجیح داده می‌شوند، زیرا بازیکنان برای انتخابِ ماشینِ پیچیده‌تر هزینه می‌پردازند. با این فرضِ جدید، بازدهیِ تعادل‌های نشِ بی‌نهایت‌تکراری به‌صورتِ قابلِ‌توجهی تغییر می‌کند: برای نمونه، مشخص شده که در تعادلِ نشِ ماشینِ حالتِ متناهی، بازیکنان باید استراتژی‌های ساده‌ای مانندِ TIT-FOR-TAT یا GRIM را بازی کنند، و هیچ‌کدام نمی‌تواند با ماشینِ بزرگ‌تر بهتر عمل کند.

۱۸.۲.۴ بازی‌های ترتیبی: شکلِ گسترده

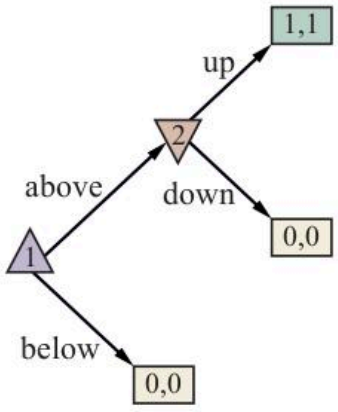
در حالتِ عمومی، بازی شاملِ توالیِ نوبت‌هایی است که لازم نیست همه یکسان باشند. چنین بازی‌هایی بهتر با درختِ بازی بازنمایی می‌شوند، که نظریه‌پردازانِ بازی «شکلِ گسترده» (Extensive Form) می‌نامند. درخت شاملِ همهٔ اطلاعاتِ مشابهی است که در بخش ۵.۱ دیدیم: حالتِ اولیهٔ S0، تابعِ PLAYER(s) که می‌گوید کدام بازیکن نوبت دارد، تابعِ ACTIONS(s) که کنش‌های ممکن را برمی‌شمارد، تابعِ RESULT(s,a) که گذار به حالتِ تازه را تعریف می‌کند، و تابعِ جزئیِ UTILITY(s,p)، که فقط روی حالت‌های پایانی تعریف شده، تا بازدهیِ هر بازیکن را بدهد. بازی‌های تصادفی را می‌توان با معرفیِ بازیکنِ متمایزی به نامِ «شانس» (Chance) در بر گرفت، که انتخاب‌هایش توسط توزیعِ احتمالی تعیین می‌شود. «استراتژیِ» شانس بخشی از تعریفِ بازی است، که به‌صورتِ توزیعِ احتمال روی کنش‌ها مشخص می‌شود (بازیکنانِ دیگر خودشان استراتژی‌شان را انتخاب می‌کنند). برای بازنماییِ بازی‌های با کنش‌های ناجبرگرایانه، مانندِ بیلیارد، کنش را به دو بخش می‌شکنیم: خودِ کنشِ بازیکن نتیجهٔ قطعی دارد، و سپس شانس نوبتی دارد تا به‌شیوهٔ هوسبازانه‌اش به کنش واکنش نشان دهد.

فعلاً، یک فرضِ ساده‌کننده می‌کنیم: فرض می‌کنیم بازیکنان «اطلاعاتِ کامل» (Perfect Information) دارند. تقریباً، اطلاعاتِ کامل بدان معناست وقتی بازی آن‌ها را به تصمیم‌گیری فرامی‌خواند، دقیقاً می‌دانند در کجای درختِ بازی‌اند: هیچ عدم قطعیتی دربارهٔ آنچه قبلاً در بازی رخ داده ندارند. این، البته، وضعیتِ بازی‌هایی مانندِ شطرنج یا گو، اما نه پوکر یا کریگزپیل، است. در بخشِ بعد، نشان خواهیم داد شکلِ گسترده چگونه می‌تواند برای دربرگرفتنِ اطلاعاتِ ناقص در بازی‌ها استفاده شود، اما فعلاً اطلاعاتِ کامل را فرض می‌کنیم.

استراتژی در بازیِ شکلِ گسترده با اطلاعاتِ کامل، تابعی برای بازیکن است که برای هر یک از حالت‌های تصمیمِ او، s، حکم می‌کند کدام کنش را از ACTIONS(s) باید انتخاب کند. وقتی هر بازیکن استراتژی‌ای انتخاب کرد، نمایهٔ استراتژیِ حاصل مسیری را در درختِ بازی از حالتِ اولیهٔ S0 به حالتِ پایانی ترسیم می‌کند، و تابعِ UTILITY مطلوبیت‌هایی را که هر بازیکن دریافت خواهد کرد تعریف می‌کند.

با این چیدمان، می‌توانیم مستقیماً ابزارِ تعادل‌های نش را که بالا معرفی کردیم برای تحلیلِ بازی‌های شکلِ گسترده اعمال کنیم. برای محاسبهٔ تعادل‌های نش، می‌توانیم از تعمیمِ سرراستِ فنِ جستجوی مینیمکس که در فصل ۵ دیدیم استفاده کنیم. در ادبیاتِ بازی‌های شکلِ گسترده، این فن «استقرایِ عقب‌رو» (Backward Induction) نامیده می‌شود —قبلاً استقرایِ عقب‌رو را به‌صورتِ غیررسمی برای تحلیلِ دوراهیِ زندانیِ متناهی‌تکراری استفاده کردیم. استقرایِ عقب‌رو از برنامه‌نویسیِ پویا استفاده می‌کند، از حالت‌های پایانی به سمتِ حالتِ اولیه به عقب کار می‌کند، و هر حالت را به‌طورِ پیش‌رونده با «نمایهٔ بازدهی» (Payoff Profile) برچسب‌گذاری می‌کند (تخصیصِ بازدهی‌ها به بازیکنان) که اگر بازی از آن نقطه به بعد به‌صورتِ بهینه بازی می‌شد به دست می‌آمد. با جزئیاتِ بیشتر، برای هر حالتِ غیرپایانیِ s، اگر همهٔ فرزندانِ s با نمایهٔ بازدهی برچسب‌گذاری شده‌اند، آن‌گاه s را با نمایهٔ بازدهیِ فرزندی برچسب‌گذاری کن که بازدهیِ بازیکنِ تصمیم‌گیرنده در حالتِ s را بیشینه می‌کند. (اگر تساوی هست، یکی را به دلخواه انتخاب کن؛ اگر گره‌های شانس داریم، مطلوبیتِ موردانتظار را محاسبه کن.) الگوریتمِ استقرایِ عقب‌رو تضمین‌شده خاتمه می‌یابد، و علاوه بر آن، در زمانِ خطی برحسبِ اندازهٔ درختِ بازی اجرا می‌شود.

هم‌زمان با اجرایِ الگوریتم، استراتژی‌هایی برای هر بازیکن ساخته می‌شود. معلوم می‌شود این استراتژی‌ها استراتژی‌های تعادلِ نش‌اند، و نمایهٔ بازدهیِ برچسب‌گذارندهٔ حالتِ اولیه، نمایهٔ بازدهی‌ای است که با بازیِ استراتژی‌های تعادلِ نش به دست می‌آمد. بدین‌ترتیب، استراتژی‌های تعادلِ نش برای بازی‌های شکلِ گسترده را می‌توان در زمانِ چندجمله‌ای با استقرایِ عقب‌رو محاسبه کرد؛ و چون الگوریتم تضمین‌شده حالتِ اولیه را با نمایهٔ بازدهی برچسب‌گذاری می‌کند، نتیجه می‌شود هر بازیِ شکلِ گسترده‌ای دستِ‌کم یک تعادلِ نش در استراتژی‌های خالص دارد. این‌ها نتایجِ جذابی‌اند، اما چند ملاحظه هست. درخت‌های بازی خیلی سریع خیلی بزرگ می‌شوند، پس زمانِ اجرایِ چندجمله‌ای باید در آن بافت فهمیده شود. اما مسئله‌دارتر، خودِ تعادلِ نش وقتی روی بازی‌های شکلِ گسترده اعمال شود محدودیت‌هایی دارد. بازیِ شکل ۱۸.۴ را در نظر بگیرید. بازیکنِ 1 دو حرکت دارد: above یا below. اگر below حرکت کند، هر دو بازیکن بازدهیِ 0 دریافت می‌کنند (صرف‌نظر از حرکتِ انتخاب‌شده توسط بازیکنِ 2). اگر above حرکت کند، بازیکنِ 2 با انتخابِ up یا down روبه‌رو می‌شود: اگر down حرکت کند، هر دو بازیکن بازدهیِ 0 دریافت می‌کنند، در حالی که اگر up حرکت کند، هر دو 1 دریافت می‌کنند.



شکل ۱۸.۴ — بازیِ شکلِ گسترده با تعادلِ نشِ ضدشهودی. (بازیکن 1 بین above و below انتخاب می‌کند؛ اگر above، بازیکن 2 بین up (بازدهی 1,1) و down (بازدهی 0,0) انتخاب می‌کند؛ below بازدهی 0,0 می‌دهد.)

استقرایِ عقب‌رو بلافاصله به ما می‌گوید (above,up) تعادلِ نش است و به هر دو بازیکن بازدهیِ 1 می‌دهد. با این حال، (below,down) نیز تعادلِ نش است، که به هر دو بازیکن بازدهیِ 0 می‌رساند. بازیکنِ 2 بازیکنِ 1 را تهدید می‌کند، با نشان‌دادنِ اینکه اگر به تصمیم‌گیری فراخوانده شود down را انتخاب خواهد کرد، که به بازدهیِ 0 برای بازیکنِ 1 می‌انجامد؛ در این صورت، بازیکنِ 1 جایگزینِ بهتری از انتخابِ below ندارد. مسئله این است که تهدیدِ بازیکنِ 2 (به بازیِ down) «تهدیدِ معتبری» (Credible Threat) نیست، زیرا اگر بازیکنِ 2 واقعاً به انتخاب فراخوانده شود، up را انتخاب خواهد کرد.

پالایشِ تعادلِ نش به نامِ «تعادلِ نشِ کاملِ زیربازی» (Subgame Perfect Nash Equilibrium) با این مسئله سروکار دارد. برای تعریفِ آن، به ایدهٔ «زیربازی» (Subgame) نیاز داریم. هر حالتِ تصمیمِ بازیکن و تمامِ فرزندانش زیربازیِ بازی را تشکیل می‌دهند. نمایهٔ استراتژی‌ها سپس تعادلِ نشِ کاملِ زیربازی در بازیِ G را تشکیل می‌دهد اگر در هر زیربازیِ G تعادلِ نش باشد. با اعمالِ این تعریف بر بازیِ شکل ۱۸.۴، درمی‌یابیم (above,up) کاملِ زیربازی است، اما (below,down) نیست، زیرا انتخابِ down تعادلِ نشِ زیربازیِ ریشه‌دار در حالتِ تصمیمِ بازیکنِ 2 نیست.

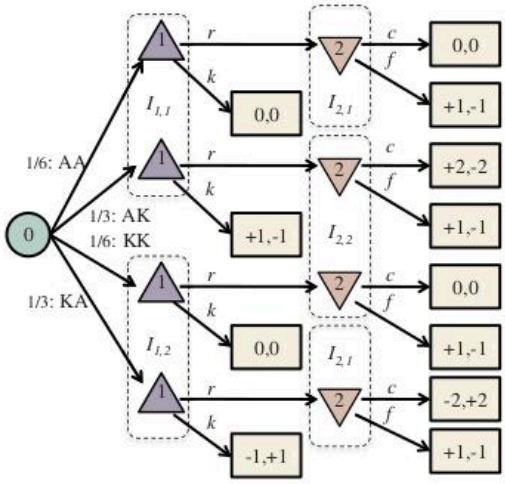
اگرچه برای تعریفِ تعادلِ نشِ کاملِ زیربازی به اصطلاح‌شناسیِ تازه‌ای نیاز داشتیم، به هیچ الگوریتمِ تازه‌ای نیاز نداریم. استراتژی‌های محاسبه‌شده از طریقِ استقرایِ عقب‌رو تعادل‌های نشِ کاملِ زیربازی‌اند، و نتیجه می‌شود هر بازیِ شکلِ گسترده با اطلاعاتِ کامل تعادلِ نشِ کاملِ زیربازی دارد، که می‌توان آن را در زمانِ چندجمله‌ای در اندازهٔ درختِ بازی محاسبه کرد.

شانس و حرکت‌های هم‌زمان. برای بازنماییِ بازی‌های تصادفی، مانندِ تخته‌نرد، در شکلِ گسترده، بازیکنی به نامِ شانس را می‌افزاییم، که انتخاب‌هایش توسط توزیعِ احتمالی تعیین می‌شود. برای بازنماییِ حرکت‌های هم‌زمان، مانندِ دوراهیِ زندانی یا مورای دوانگشتی، ترتیبِ دلخواهی بر بازیکنان تحمیل می‌کنیم، اما این گزینه را داریم که اظهار کنیم کنش‌های بازیکنِ قبلی برای بازیکنانِ بعدی قابلِ مشاهده نیستند: مثلاً، علی باید اول امتناع یا شهادت را انتخاب کند، سپس بو انتخاب می‌کند، اما بو نمی‌داند علی در آن زمان چه انتخابی کرده (همچنین می‌توانیم این واقعیت را بازنمایی کنیم که حرکت بعداً فاش می‌شود). با این حال، فرض می‌کنیم بازیکنان همیشه همهٔ کنش‌های قبلیِ خودشان را به یاد می‌آورند؛ این فرض «بازخوانیِ کامل» (Perfect Recall) نامیده می‌شود.

دربرگرفتنِ اطلاعاتِ ناقص. ویژگیِ کلیدیِ شکلِ گسترده که آن را از درخت‌های بازیِ دیده‌شده در فصل ۵ متمایز می‌کند این است که می‌تواند مشاهده‌پذیریِ جزئی را در بر بگیرد. نظریه‌پردازانِ بازی از اصطلاحِ «اطلاعاتِ ناقص» (Imperfect Information) برای توصیفِ موقعیت‌هایی استفاده می‌کنند که بازیکنان دربارهٔ حالتِ واقعیِ بازی نامطمئن‌اند. متأسفانه، استقرایِ عقب‌رو با بازی‌های اطلاعاتِ ناقص کار نمی‌کند، و به‌طورِ کلی، آن‌ها به‌طورِ قابلِ‌توجهی پیچیده‌تر از بازی‌های اطلاعاتِ کامل برای حل‌کردن‌اند.

در بخش ۵.۶ دیدیم بازیکن در بازیِ تا حدی قابل مشاهده مانندِ کریگزپیل می‌تواند درختی بر فضایِ حالت‌های باور بسازد. با آن درخت، دیدیم در برخی موارد بازیکن می‌تواند توالیِ حرکت‌هایی (استراتژی) بیابد که به کیش‌ومواتِ اجباری می‌انجامد، صرف‌نظر از اینکه از چه حالتِ واقعی شروع کرده باشیم و صرف‌نظر از اینکه حریف چه استراتژی‌ای استفاده کند. با این حال، فنونِ فصل ۵ نمی‌توانستند به بازیکن بگویند وقتی کیش‌ومواتِ تضمینی نیست چه کند. اگر بهترین استراتژیِ بازیکن به استراتژیِ حریف بستگی داشته باشد و برعکس، آن‌گاه مینیمکس (یا آلفا-بتا) به‌تنهایی نمی‌تواند

راه‌حل بیابد. شکلِ گسترده به ما اجازه می‌دهد راه‌حل‌ها را بیابیم، زیرا حالت‌های باور (نظریه‌پردازانِ بازی آن‌ها را «مجموعه‌های اطلاعاتی» (Information Sets) می‌نامند) همهٔ بازیکنان را هم‌زمان بازنمایی می‌کند. از آن بازنمایی می‌توانیم راه‌حل‌های تعادلی را بیابیم، درست همان‌گونه که با بازی‌های شکلِ نرمال انجام دادیم.



شکل ۱۸.۵ – شکلِ گستردهٔ نسخهٔ ساده‌شده‌ای از پوکر با دو بازیکن و فقط چهار کارت. حرکت‌ها عبارت‌اند از r (بالا بردن)، f (انصراف)، c (هم‌خوانی) و k (چک). بازیکنِ 0 شانس است؛ بازیکنانِ 1 و 2 با مثلث‌ها نشان داده شده‌اند. حالت‌های پایانی مستطیل‌هایی‌اند که با بازدهیِ بازیکنانِ 1 و 2 برچسب‌گذاری شده‌اند. مجموعه‌های اطلاعاتی به‌صورتِ جعبه‌های خط‌چینِ برچسب‌دار نشان داده شده‌اند؛ برای نمونه، I1,1 مجموعهٔ اطلاعاتی است که در آن نوبتِ بازیکنِ 1 است و او می‌داند آس دارد (اما نمی‌داند بازیکنِ 2 چه دارد). در مجموعهٔ اطلاعاتیِ I2,1، نوبتِ بازیکنِ 2 است و او می‌داند آس دارد و بازیکنِ 1 بالا برده، اما نمی‌داند بازیکنِ 1 چه کارتی دارد.

به‌عنوانِ مثالِ ساده‌ای از بازیِ ترتیبی، دو عامل را در جهانِ 4×3 شکل ۱۷.۱ قرار دهید و آن‌ها را هم‌زمان حرکت دهید تا وقتی یکی به مربعِ خروجی برسد و بازدهیِ آن مربع را بگیرد. اگر مشخص کنیم وقتی دو عامل هم‌زمان سعی می‌کنند به همان مربع حرکت کنند هیچ حرکتی رخ نمی‌دهد (مسئله‌ای رایج در بسیاری از تقاطع‌های ترافیکی)، آن‌گاه برخی استراتژی‌های خالص می‌توانند برای همیشه گیر کنند. بدین‌ترتیب، عامل‌ها در این بازی برای عملکردِ خوب به استراتژیِ ترکیبی نیاز دارند: تصادفی بینِ حرکتِ به جلو و ماندن انتخاب کنید. این دقیقاً همان کاری است که برای حلِ برخوردهای بسته در شبکه‌های اترنت انجام می‌شود.

بعد، نوعِ بسیار ساده‌ای از پوکر را در نظر می‌گیریم. عرشه فقط چهار کارت دارد، دو آس و دو شاه. به هر بازیکن یک کارت داده می‌شود. بازیکنِ نخست سپس این گزینه را دارد که شرطِ بازی را از 1 امتیاز به 2 بالا ببرد، یا چک کند. اگر بازیکنِ 1 چک کند، بازی تمام است. اگر بازیکنِ 1 بالا ببرد، بازیکنِ 2 این گزینه را دارد که هم‌خوانی کند، بپذیرد که بازی 2 امتیاز ارزش دارد، یا انصراف دهد، 1 امتیاز را واگذار کند. اگر بازی با انصراف پایان نیابد، بازدهی به کارت‌ها بستگی دارد: اگر کارت‌های یکسانی داشته باشند برای هر دو بازیکن صفر است؛ در غیرِ این صورت بازیکنِ با شاه شرط را به بازیکنِ با آس می‌پردازد.

درختِ شکلِ گسترده برای این بازی در شکل ۱۸.۵ نشان داده شده. یک راه برای حلِ بازیِ گسترده تبدیلِ آن به بازیِ شکلِ نرمال است. به یاد آورید شکلِ نرمال ماتریسی است که هر ردیفش با استراتژیِ خالصِ بازیکنِ 1 و هر ستونش با استراتژیِ خالصِ بازیکنِ 2 برچسب‌گذاری می‌شود. در بازیِ گسترده، استراتژیِ خالصِ بازیکنِ i با کنشی برای هر مجموعهٔ

اطلاعاتیِ شاملِ آن بازیکن متناظر است. پس در شکل ۱۸.۵، یکی از استراتژی‌های خالصِ بازیکنِ 1 «وقتی در I1,1 هستم (یعنی آس دارم) بالا ببر، و وقتی در I1,2 هستم (شاه دارم) چک کن» است. در ماتریسِ بازدهیِ زیر، این استراتژی rk نامیده می‌شود. به‌طورِ مشابه، استراتژیِ cf برای بازیکنِ 2 بدان معناست «وقتی آس دارم هم‌خوانی کن و وقتی شاه دارم انصراف بده». چون این بازی جمعِ صفر است، ماتریسِ زیر فقط بازدهیِ بازیکنِ 1 را می‌دهد؛ بازیکنِ 2 همیشه بازدهیِ مقابل را دارد:

	cc:2	cf:2	ff:2	fc:2
rr:1	0	-1/6	1	7/6
kr:1	-1/3	-1/6	5/6	2/3
rk:1	1/3	0	1/6	1/2
kk:1	0	0	0	0

این بازی چنان ساده است که دو تعادلِ استراتژیِ خالص دارد: (rk, cf) و (kk, cf)؛ یعنی cf برای بازیکنِ 2 و rk یا kk برای بازیکنِ 1. اما به‌طورِ کلی می‌توانیم بازی‌های گسترده را با تبدیل به شکلِ نرمال و سپس یافتنِ راه‌حل (معمولاً استراتژیِ ترکیبی) با روش‌های استانداردِ برنامه‌نویسیِ خطی حل کنیم. این در تئوری کار می‌کند. اما اگر بازیکنی I مجموعهٔ اطلاعاتی و a کنش در هر مجموعه داشته باشد، آن بازیکن a^I استراتژیِ خالص خواهد داشت. به بیانِ دیگر، اندازهٔ ماتریسِ شکلِ نرمال در شمارِ مجموعه‌های اطلاعاتی نمایی است، پس در عمل رویکرد فقط برای درخت‌های بازیِ کوچک—حدود ده‌ها حالت—کار می‌کند. بازی‌ای مانندِ پوکرِ تگزاس هولدمِ دونفره حدود 10^{18} حالت دارد، که این رویکرد را کاملاً ناشدنی می‌کند.

جایگزین‌ها چیست؟ در فصل ۵ دیدیم چگونه جستجوی آلفا-بتا می‌توانست بازی‌های اطلاعاتِ کامل با درخت‌های بازیِ عظیم را با تولیدِ تدریجیِ درخت، با هرسِ برخی شاخه‌ها و با ارزیابیِ اکتشافیِ گره‌های غیرپایانی مدیریت کند. اما آن رویکرد برای بازی‌های اطلاعاتِ ناقص خوب کار نمی‌کند، به دو دلیل: نخست، هرسِ آن دشوارتر است، زیرا باید استراتژی‌های ترکیبی را در نظر بگیریم که چند شاخه را ترکیب می‌کنند، نه استراتژیِ خالصی را که همیشه بهترین شاخه را انتخاب می‌کند. دوم، ارزیابیِ اکتشافیِ گرهِ غیرپایانی دشوارتر است، زیرا با مجموعه‌های اطلاعاتی سروکار داریم، نه حالت‌های فردی.

کولر و همکاران (1996) با بازنماییِ جایگزینی از بازی‌های گسترده به کمک آمدند، به نامِ «شکلِ توالی» (Sequence Form)، که فقط در اندازهٔ درخت خطی است، نه نمایی. به‌جای بازنماییِ استراتژی‌ها، مسیرهای درونِ درخت را بازنمایی می‌کند؛ شمارِ مسیرها برابرِ شمارِ گره‌های پایانی است. روش‌های استانداردِ برنامه‌نویسیِ خطی را می‌توان دوباره روی این بازنمایی اعمال کرد. سامانهٔ حاصل می‌تواند انواعِ پوکر را با ۲۵,۰۰۰ حالت در یکی دو دقیقه حل کند. این سرعتِ نمایی نسبت به رویکردِ شکلِ نرمال است، اما همچنان از مدیریتِ، مثلاً، پوکرِ تگزاس هولدمِ دونفره، با 10^{18} حالت، بسیار کوتاه است.

اگر نتوانیم 10^{18} حالت را مدیریت کنیم، شاید بتوانیم با تغییرِ بازی به شکلِ ساده‌تر، مسئله را ساده کنیم. برای نمونه، اگر آس دارم و امکانِ اینکه کارتِ بعدی به من جفتِ آس بدهد را در نظر می‌گیرم، پس به خالِ کارتِ بعدی اهمیت نمی‌دهم؛ تحتِ قاعده‌های پوکر هر خالی به‌یکسان خوب است. این پیشنهادِ تشکیلِ «انتزاعی» (Abstraction) از بازی می‌کند، که در آن خال‌ها نادیده گرفته می‌شوند. درختِ بازیِ حاصل با ضریبِ 4! = 24 کوچک‌تر خواهد بود. فرض کنیم بتوانم این بازیِ کوچک‌تر را حل کنم؛ راه‌حلِ آن بازی چگونه به بازیِ اصلی مربوط می‌شود؟ اگر هیچ بازیکنی در فکرِ رفتن برای فلاش نباشد (تنها دستی که خال‌ها در آن مهم‌اند)، آن‌گاه راه‌حلِ انتزاع نیز راه‌حلِ بازیِ اصلی خواهد بود. با این حال، اگر بازیکنی در فکرِ فلاش باشد، انتزاع فقط راه‌حلِ تقریبی خواهد بود (اما می‌توان کران‌هایی بر خطا محاسبه کرد).

فرصت‌های بسیاری برای انتزاع هست. برای نمونه، در نقطه‌ای از بازی که هر بازیکن دو کارت دارد، اگر جفتِ بی‌بی داشته باشم، آن‌گاه دست‌های بازیکنانِ دیگر را می‌توان به سه رده انتزاع کرد: بهتر (فقط جفتِ شاه یا جفتِ آس)، یکسان (جفتِ بی‌بی) یا بدتر (هر چیزِ دیگر). با این حال، این انتزاع ممکن است بیش‌ازحد درشت باشد. انتزاعِ بهتر «بدتر» را به، مثلاً، جفتِ متوسط (نه تا سرباز)، جفتِ پایین و بدونِ جفت تقسیم می‌کرد. این نمونه‌ها انتزاع‌های حالت‌اند؛ انتزاعِ کنش‌ها نیز ممکن است. برای نمونه، به‌جای کنشِ شرط برای هر عددِ صحیح از 1 تا 1000، می‌توانستیم شرط‌ها را به 10^0، 10^1، 10^2 و 10^3 محدود کنیم. یا می‌توانستیم یکی از دورهای شرط‌بندی را کلاً حذف کنیم.

همچنین می‌توانیم روی گره‌های شانس انتزاع کنیم، با درنظرگرفتنِ فقط زیرمجموعه‌ای از توزیع‌های ممکن. این معادلِ فنِ «رول‌اوت» (Rollout) استفاده‌شده در برنامه‌های گو است. با کنار هم گذاشتنِ همهٔ این انتزاع‌ها، می‌توانیم 10^{18} حالتِ پوکر را به 10^7 حالت کاهش دهیم، اندازه‌ای که با فنونِ کنونی قابلِ حل است. در فصل 5 دیدیم برنامه‌های پوکر مانندِ Libratus و DeepStack چگونه توانستند قهرمانانِ انسانی را در پوکرِ تگزاس هولدمِ سر به سر (دونفره) شکست دهند. اخیراً، برنامهٔ Pluribus توانست قهرمانانِ انسانی را در پوکرِ شش‌نفره در دو قالب شکست دهد: پنج نسخه از برنامه پشتِ میز با یک انسان، و یک نسخه از برنامه با پنج انسان. جهشِ عظیمی در پیچیدگی این‌جاست. با یک حریف، choose-2 = 1225-50 امکان برای کارت‌های پنهانِ حریف هست. اما با پنج حریف، choose-10 ≈ 10-50 میلیارد امکان هست. Pluribus استراتژیِ پایه‌ای را کاملاً از خودبازی توسعه می‌دهد، سپس استراتژی را در طولِ بازیِ واقعی برای واکنش به موقعیتِ خاص اصلاح می‌کند.

Pluribus از ترکیبی از فنون استفاده می‌کند، از جملهِ جستجوی درختیِ مونت‌کارلو، جستجوی عمق‌محدود و انتزاع. شکلِ گسترده بازنماییِ چندمنظوره‌ای است: می‌تواند محیط‌های تا حدی قابل مشاهده، چندعاملی، تصادفی، ترتیبی و بلادرنگ را مدیریت کند—بیشترِ حالت‌های دشوار از فهرستِ ویژگی‌های محیط در صفحهٔ ۴۳. با این حال، دو محدودیت برای شکلِ گسترده به‌طورِ خاص و نظریهٔ بازی به‌طورِ کلی هست. نخست، با حالت‌ها و کنش‌های پیوسته خوب سروکار ندارد (هرچند بسط‌هایی به حالتِ پیوسته بوده؛ برای نمونه، نظریهٔ «رقابتِ کورنو» (Cournot Competition) از نظریهٔ بازی برای حلِ مسائلی استفاده می‌کند که دو شرکت قیمتِ محصولاتشان را از فضایِ پیوسته انتخاب می‌کنند). دوم، نظریهٔ بازی فرض می‌کند بازی معلوم است. بخش‌هایی از بازی ممکن است برای برخی بازیکنان نامشاهده مشخص

شوند، اما باید معلوم باشد کدام بخش‌ها نامشاهده‌اند. در مواردی که بازیکنان ساختارِ ناشناختهٔ بازی را در طولِ زمان یاد می‌گیرند، مدل شروع به فروپاشی می‌کند. هر منبعِ عدم قطعیت را بررسی کنیم، و ببینیم آیا هر یک در نظریهٔ بازی قابلِ بازنمایی است:

- **کنش‌ها:** راهِ آسانی برای بازنماییِ بازی‌ای که بازیکنان باید کشف کنند چه کنش‌هایی در دسترس‌اند نیست. بازیِ میانِ نویسندگانِ ویروسِ رایانه‌ای و کارشناسانِ امنیت را در نظر بگیرید. بخشی از مسئله پیش‌بینیِ این است که نویسندگانِ ویروس بعداً چه کنشی را امتحان خواهند کرد.
- **استراتژی‌ها:** نظریهٔ بازی در بازنماییِ این ایده که استراتژی‌های بازیکنانِ دیگر در آغاز ناشناخته‌اند بسیار خوب است—تا زمانی که فرض کنیم همهٔ عامل‌ها عقلایی‌اند. نظریه نمی‌گوید وقتی بازیکنانِ دیگر کمتر از حدِ کامل عقلایی‌اند چه باید کرد. مفهومِ «تعادلِ بیز-نش» (Bayes–Nash Equilibrium) تا حدی به این نکته می‌پردازد: تعادلی است نسبت به توزیعِ احتمالِ پیشینیِ بازیکن روی استراتژی‌های بازیکنانِ دیگر—به بیانِ دیگر، باورهای بازیکن را دربارهٔ استراتژی‌های احتمالیِ بازیکنانِ دیگر بیان می‌کند.
- **شانس:** اگر بازی به پرتابِ تاس بستگی دارد، مدل‌کردنِ گرهٔ شانسی با توزیعِ یکنواخت روی پیامدها به‌اندازهٔ کافی آسان است. اما اگر ممکن است تاس ناعادلانه باشد چه؟ می‌توانیم آن را با گرهٔ شانسِ دیگری، بالاتر در درخت، با دو شاخه برای «تاس عادلانه است» و «تاس ناعادلانه است» بازنمایی کنیم، به‌گونه‌ای که گره‌های متناظر در هر شاخه در همان مجموعهٔ اطلاعاتی باشند (یعنی بازیکنان نمی‌دانند تاس عادلانه است یا نه). و اگر شک کنیم حریفِ دیگر می‌داند چه؟ سپس گرهٔ شانسِ دیگری می‌افزاییم، با شاخه‌ای که حالتِ دانستنِ حریف و شاخه‌ای که حالتِ ندانستنِ او را بازنمایی می‌کند.
- **مطلوبیت‌ها:** اگر مطلوبیت‌های حریف‌مان را ندانیم چه؟ باز، آن را می‌توان با گرهٔ شانسی مدل کرد، به‌گونه‌ای که عاملِ دیگر مطلوبیت‌های خودش را در هر شاخه می‌داند، اما ما نمی‌دانیم. اما اگر مطلوبیت‌های خودمان را ندانیم چه؟ برای نمونه، چگونه بدانم سفارشِ سالادِ سرآشپز عقلایی است اگر نمی‌دانم چقدر آن را دوست خواهم داشت؟ می‌توانیم آن را با گرهٔ شانسِ دیگری مدل کنیم که «کیفیتِ ذاتیِ» نامشاهده‌ای از سالاد را مشخص می‌کند.

۱۸.۲.۵ بازدهی‌های نامطمئن و بازی‌های کمکی

در فصل ۱ (صفحهٔ ۴) به اهمیتِ طراحیِ سامانه‌های هوش مصنوعی اشاره کردیم که می‌توانند تحتِ عدم قطعیت دربارهٔ هدفِ واقعیِ انسانی عمل کنند. فصل ۱۶ (صفحهٔ ۵۵۳) مدلِ ساده‌ای برای عدم قطعیت دربارهٔ ترجیح‌های خود معرفی کرد، با مثالِ بستنیِ دوریانی. با ابزارِ سادهٔ افزودنِ متغیرِ نهانِ تازه‌ای به مدل برای بازنماییِ ترجیح‌های ناشناخته، همراه با مدلِ حسگرِ مناسب (مثلاً مشاهدهٔ طعمِ نمونهٔ کوچکی از بستنی)، ترجیح‌های نامطمئن را می‌توان به‌صورتِ طبیعی مدیریت کرد.

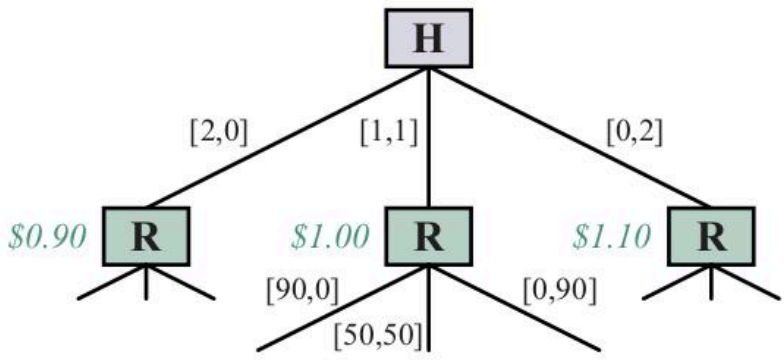
فصل ۱۶ همچنین مسئلهٔ کلیدِ خاموش را مطالعه کرد: نشان دادیم رباتی با عدم قطعیت دربارهٔ ترجیح‌های انسانی از انسان اطاعت خواهد کرد و به خودش اجازه خواهد داد خاموش شود. در آن مسئله، رباتِ روبی دربارهٔ ترجیح‌های هریتِ انسانی نامطمئن است، اما تصمیمِ هریت (خاموش‌کردنِ روبی یا نه) را به‌عنوانِ پیامدِ سادهٔ قطعیِ ترجیح‌های خودش برای کنشی که روبی پیشنهاد می‌کند مدل می‌کنیم. در این‌جا، این ایده را به «بازیِ کمکی» (Assistance

Game) با عنوانِ کاملِ دونفره تعمیم می‌دهیم، که در آن هر دو هریت و روبی بازیکن‌اند. فرض می‌کنیم هریت ترجیح‌های خودش θ را مشاهده می‌کند و مطابقِ آن‌ها عمل می‌کند، در حالی که روبی احتمالِ پیشینیِ P(θ) روی ترجیح‌های هریت دارد.

بازدهی توسط θ تعریف می‌شود و برای هر دو بازیکن یکسان است: هم هریت و هم روبی بازدهیِ هریت را بیشینه می‌کنند. بدین‌گونه، بازیِ کمکی مدلِ صوریِ ایدهٔ «هوش مصنوعیِ قابلِ‌اثباتِ سودمند» (Provably Beneficial AI) معرفی‌شده در فصل ۱ را فراهم می‌کند.

علاوه بر رفتارِ اطاعت‌آمیزِ نشان‌داده‌شده توسط روبی در مسئلهٔ کلیدِ خاموش—که نوعِ مقیدی از بازیِ کمکی است—رفتارهای دیگری که به‌عنوانِ استراتژی‌های تعادلی در بازی‌های کمکیِ عمومی پدید می‌آیند شاملِ کنش‌هایی از جانبِ هریت می‌شوند که آن‌ها را آموزش، پاداش، فرمان، تصحیح، نمایش یا توضیح توصیف می‌کنیم، و نیز کنش‌هایی از جانبِ روبی که آن‌ها را اجازه‌خواستن، یادگیری از نمایش‌ها، استخراجِ ترجیح و مانندِ آن توصیف می‌کنیم. نکتهٔ کلیدی این است که این رفتارها لازم نیست از پیشِ نوشته‌شده باشند: با حلِ بازی، هریت و روبی خودشان کشف می‌کنند چگونه اطلاعاتِ ترجیحی را از هریت به روبی منتقل کنند، تا روبی بتواند برای هریت مفیدتر باشد. لازم نیست از پیش مقرر کنیم هریت قرار است «پاداش بدهد» یا روبی قرار است «از دستورها پیروی کند»، هرچند این‌ها ممکن است تفسیرهای معقولی از نحوهٔ پایان‌یافتنِ رفتارشان باشند.

برای نشان‌دادنِ بازی‌های کمکی، از «بازیِ گیره‌کاغذ» (Paperclip Game) استفاده می‌کنیم. بازیِ بسیار ساده‌ای است که در آن هریتِ انسانی انگیزه‌ای برای «سیگنال‌دادن» به روبیِ ربات دربارهٔ برخی اطلاعاتِ ترجیحی‌اش دارد. روبی قادر به تفسیرِ آن سیگنال است، زیرا می‌تواند بازی را حل کند و بنابراین می‌تواند بفهمد دربارهٔ ترجیح‌های هریت چه چیزی باید صادق باشد تا او بدین‌گونه سیگنال دهد.



شکل ۱۸.۶ – بازیِ گیره‌کاغذ. هر شاخه با [p,s] برچسب‌گذاری شده که شمارِ گیره‌کاغذها و منگنه‌های تولیدشده روی آن شاخه را نشان می‌دهد. هریتِ انسانی می‌تواند انتخاب کند دو گیره‌کاغذ، دو منگنه یا یکی از هر کدام بسازد. (مقادیرِ سبزِ مورب، مقادیرِ هریت اگر بازی همان‌جا پایان می‌یافت، با فرضِ θ=0.45 هستند: 0.90، 1.00، 1.10.) روبیِ ربات سپس انتخاب دارد 90 گیره‌کاغذ، 90 منگنه یا 50 از هر کدام بسازد (با مقادیرِ [90,0]، [50,50] و [0,90]).

گام‌های بازی در شکل ۱۸.۶ به تصویر کشیده شده. شاملِ ساختنِ گیره‌کاغذ و منگنه است. ترجیح‌های هریت با تابعِ بازدهی بیان می‌شود که به شمارِ گیره‌کاغذهای تولیدشده و شمارِ منگنه‌های تولیدشده بستگی دارد، با «نرخِ مبادلهٔ» معینی بینِ این دو. پارامترِ ترجیحِ هریت، θ، ارزشِ نسبی (به دلار) یک گیره‌کاغذ را نشان می‌دهد؛ برای نمونه، او ممکن

است گیره‌کاغذها را θ=0.45 دلار ارزش‌گذاری کند، که بدان معناست منگنه‌ها 1-0.55 = θ دلار می‌ارزند. پس، اگر p گیره‌کاغذ و s منگنه تولید شود، بازدهیِ هریت در مجموع pθ + s(1-θ) دلار خواهد بود. پیشینِ روبی = P(θ) Uniform(θ;0,1) است. در خودِ بازی، هریت اول می‌رود و می‌تواند انتخاب کند دو گیره‌کاغذ، دو منگنه یا یکی از هر کدام بسازد. سپس روبی می‌تواند انتخاب کند 90 گیره‌کاغذ، 90 منگنه یا 50 از هر کدام بسازد.

توجه کنید اگر او به‌تنهایی این کار را می‌کرد، هریت فقط دو منگنه می‌ساخت، با ارزشِ 1.10 دلار. (حاشیه‌نویسی‌های سطحِ نخستِ درخت در شکل ۱۸.۶ را ببینید.) اما روبی تماشا می‌کند و از انتخابِ او یاد می‌گیرد. دقیقاً چه چیزی یاد می‌گیرد؟ خوب، بستگی به این دارد هریت چگونه انتخابش را بکند. هریت چگونه انتخابش را می‌کند؟ بستگی به این دارد روبی چگونه آن را تفسیر کند. می‌توانیم این دور را با یافتنِ تعادلِ نش حل کنیم.

در این مورد، تعادل یکتاست و می‌توان با اعمالِ بهترین پاسخِ کوته‌نگر آن را یافت: هر استراتژی برای هریت برگزین؛ بهترین استراتژی را برای روبی، با توجه به استراتژیِ هریت، برگزین؛ بهترین استراتژی را برای هریت، با توجه به استراتژیِ روبی، برگزین؛ و همین‌طور. فرایند چنین باز می‌شود:

1. با استراتژیِ حریصانه برای هریت شروع کن: اگر گیره‌کاغذ را ترجیح می‌دهد دو گیره‌کاغذ بساز؛ اگر بی‌تفاوت است یکی از هر کدام بساز؛ اگر منگنه را ترجیح می‌دهد دو منگنه بساز.
2. با توجه به این استراتژیِ هریت، سه امکان برای روبی هست که باید در نظر بگیرد:

(الف) اگر روبی ببیند هریت دو گیره‌کاغذ می‌سازد، استنتاج می‌کند گیره‌کاغذ را ترجیح می‌دهد، پس حالا باور دارد ارزشِ گیره‌کاغذ به‌صورتِ یکنواخت بینِ 0.5 و 1.0 توزیع شده، با میانگینِ 0.75. در آن صورت، بهترین برنامه‌اش ساختنِ 90 گیره‌کاغذ با ارزشِ موردانتظارِ 67.50 دلار برای هریت است. (ب) اگر روبی ببیند هریت یکی از هر کدام می‌سازد، استنتاج می‌کند گیره‌کاغذها و منگنه‌ها را 0.50 ارزش‌گذاری می‌کند، پس بهترین انتخاب ساختنِ 50 از هر کدام است. (ج) اگر روبی ببیند هریت دو منگنه می‌سازد، با همان استدلالِ (الف)، باید 90 منگنه بسازد.

1. با توجه به این استراتژیِ روبی، بهترین استراتژیِ هریت حالا تا حدی با استراتژیِ حریصانهٔ گامِ 1 متفاوت است. اگر روبی قرار است با ساختنِ 50 از هر کدام به ساختنِ یکی از هر کدام توسط او پاسخ دهد، آن‌گاه او بهتر است یکی از هر کدام را بسازد نه فقط اگر دقیقاً بی‌تفاوت است، بلکه اگر در هر جایِ نزدیکِ بی‌تفاوتی است. در واقع، سیاستِ بهینه حالا این است که یکی از هر کدام را بسازد اگر گیره‌کاغذها را در هر جایِ حدودِ 0.446 تا 0.554 ارزش‌گذاری کند.
2. با توجه به این استراتژیِ تازهٔ هریت، استراتژیِ روبی بدونِ تغییر می‌ماند. برای نمونه، اگر او یکی از هر کدام را انتخاب کند، روبی استنتاج می‌کند ارزشِ گیره‌کاغذ به‌صورتِ یکنواخت بینِ 0.446 و 0.554 توزیع شده، با میانگینِ 0.50، پس بهترین انتخاب ساختنِ 50 از هر کدام است. چون استراتژیِ روبی همان استراتژیِ گامِ 2 است، بهترین پاسخِ هریت همان پاسخِ گامِ 3 خواهد بود، و تعادل را یافته‌ایم.

هریت با استراتژی‌اش، در واقع، با کدِ ساده‌ای—زبانی، اگر دوست داشته باشید—که از تحلیلِ تعادلی پدید می‌آید، ترجیح‌هایش را به روبی آموزش می‌دهد. همچنین توجه کنید روبی هرگز ترجیح‌های هریت را دقیقاً یاد نمی‌گیرد، اما به‌اندازهٔ کافی یاد می‌گیرد تا از جانبِ او به‌صورتِ بهینه عمل کند—یعنی دقیقاً همان‌گونه عمل می‌کند که گویی

ترجیح‌هایش را دقیقاً می‌دانست. او تحتِ مفروضاتِ بیان‌شده، و تحتِ فرضِ اینکه هریت بازی را درست بازی می‌کند، به‌صورتِ اثبات‌پذیر برای هریت سودمند است.

بهترین پاسخِ کوته‌نگر برای این مثال و مثال‌های مشابهِ آن کار می‌کند، اما برای مواردِ پیچیده‌تر نه. می‌توان اثبات کرد، مشروط بر اینکه هیچ تساوی‌ای که باعثِ مسائلِ هماهنگی شود نباشد، یافتنِ نمایهٔ استراتژیِ بهینه برای بازیِ کمکی به حلِ POMDPای تقلیل‌پذیر است که فضایِ حالتش فضایِ حالتِ زیربناییِ بازی به‌علاوهٔ پارامترهای ترجیحِ انسانیِ θ است. POMDPها به‌طورِ کلی حل‌شدنشان بسیار دشوار است (بخش ۱۷.۵)، اما POMDPهایی که بازی‌های کمکی را بازنمایی می‌کنند ساختارِ اضافی دارند که الگوریتم‌های کارآمدتری را ممکن می‌کند.

بازی‌های کمکی را می‌توان برای اجازه‌دادن به چند شرکت‌کنندهٔ انسانی، چند ربات، انسان‌های با عقلانیتِ ناقص، انسان‌هایی که ترجیح‌های خودشان را نمی‌دانند و مانندِ آن تعمیم داد. با فراهم‌کردنِ فضایِ کنشِ عاملی یا ساخت‌یافته، در مقابلِ کنش‌های اتمیِ ساده در بازیِ گیره‌کاغذ، فرصت‌های ارتباط می‌توانند بسیار افزایش یابند. تاکنون تعدادِ کمی از این تغییرها کاوش شده، اما انتظار داریم ویژگیِ کلیدیِ بازی‌های کمکی صادق بماند: هرچه ربات باهوش‌تر باشد، پیامد برای انسان بهتر است.

۱۸.۳ نظریهٔ بازیِ همکارانه

به یاد آورید بازی‌های همکارانه سناریوهای تصمیم‌گیری را در بر می‌گیرند که در آن‌ها عامل‌ها می‌توانند توافق‌های الزام‌آوری با یکدیگر برای همکاری تشکیل دهند. آن‌گاه می‌توانند از دریافتِ ارزشِ اضافی در مقایسه با آنچه با عملِ تنها می‌گرفتند سود ببرند.

با معرفیِ مدلی برای رده‌ای از بازی‌های همکارانه شروع می‌کنیم. به‌صورتِ صوری، این بازی‌ها «بازی‌های همکارانه با مطلوبیتِ قابلِ‌انتقال در شکلِ تابعِ مشخصه» نامیده می‌شوند. ایدهٔ مدل این است که وقتی گروهی از عامل‌ها همکاری می‌کنند، گروه به‌عنوانِ کل مقداری مطلوبیت به دست می‌آورد، که سپس می‌تواند میانِ اعضای گروه تقسیم شود. مدل نمی‌گوید عامل‌ها چه کنش‌هایی خواهند گرفت، و نه خودِ ساختارِ بازی مشخص می‌کند ارزشِ به‌دست‌آمده چگونه تقسیم خواهد شد (آن بعداً می‌آید).

به‌صورتِ صوری، از فرمولِ $G = (N,\nu)$ برای گفتنِ اینکه بازیِ همکارانهٔ G با مجموعه‌ای از بازیکنانِ $N = \{1,...,n\}$ و «تابعِ مشخصه» (Characteristic Function) ν تعریف می‌شود، استفاده می‌کنیم، که برای هر زیرمجموعهٔ بازیکنانِ $C \subseteq N$ ارزشی را می‌دهد که گروهِ بازیکنان می‌توانست، اگر انتخاب می‌کردند با هم کار کنند، به دست آورد. معمولاً فرض می‌کنیم مجموعهٔ خالی از بازیکنان هیچ‌چیز به دست نمی‌آورد ($\nu(\{\}) = 0$)، و تابع نامنفی است ($\nu(C) \geq 0$ برای همهٔ C). در برخی بازی‌ها فرضِ بیشتری می‌کنیم که بازیکنان با کارِ تنها هیچ‌چیز به دست نمی‌آورند: $\nu(\{i\}) = 0$ برای همهٔ $i \in N$.

۱۸.۳.۱ ساختارهای ائتلاف و پیامدها

مرسوم است به زیرمجموعه‌ای از بازیکنانِ C «ائتلاف» (Coalition) گفته شود. در استفادهٔ روزمره، اصطلاحِ «ائتلاف» دلالت بر مجموعه‌ای از افراد با هدفِ مشترکی دارد (مانندِ ائتلافِ توقفِ خشونت با اسلحه)، اما ما به هر زیرمجموعه‌ای از بازیکنان ائتلاف خواهیم گفت. مجموعهٔ همهٔ بازیکنانِ N «ائتلافِ بزرگ» (Grand Coalition) شناخته می‌شود.

در مدلِ ما، هر بازیکن باید انتخاب کند دقیقاً به یک ائتلاف بپیوندد (که می‌تواند ائتلافِ فقطِ خودِ بازیکنِ تنها باشد). بدین‌ترتیب، ائتلاف‌ها افرازی از مجموعهٔ بازیکنان تشکیل می‌دهند. این افراز را «ساختارِ ائتلاف» (Coalition Structure) می‌نامیم. به‌صورتِ صوری، ساختارِ ائتلاف روی مجموعهٔ بازیکنانِ N، مجموعه‌ای از ائتلاف‌ها {C1,...,Ck} است به‌گونه‌ای که:

```
Ci ∩ Cj      Ci ⊆ N      {} ≠ Ci = {} برای همهٔ      C1 ∪ ... ∪ Ck = N      i ≠ j ∈ N
```

برای نمونه، اگر N = {1,2,3} داشته باشیم، هفت ائتلافِ ممکن هست: {1}، {2}، {3}، {1,2}، {2,3}، {3,1} و {1,2,3}، و پنج ساختارِ ائتلافِ ممکن: {{1},{2},{3}}، {{1},{2,3}}، {{2},{1,3}}، {{3},{1,2}} و {{1,2,3}}.

از نمادِ CS(N) برای نشان‌دادنِ مجموعهٔ همهٔ ساختارهای ائتلاف روی مجموعهٔ بازیکنانِ N، و از CS(i) برای نشان‌دادنِ ائتلافی که بازیکنِ i به آن تعلق دارد استفاده می‌کنیم.

پیامدِ بازی با انتخاب‌هایی که بازیکنان می‌کنند تعریف می‌شود، در تصمیم‌گیری دربارهٔ اینکه کدام ائتلاف‌ها را تشکیل دهند، و در انتخابِ چگونگیِ تقسیمِ ارزشِ ν(C) که هر ائتلاف دریافت می‌کند. به‌صورتِ صوری، با توجه به بازیِ همکارانهٔ تعریف‌شده با (N,ν)، پیامد جفتِ (CS,x) است که از ساختارِ ائتلاف و «بردارِ بازدهی» (Payoff Vector) x = (x1,...,xn) تشکیل می‌شود، که در آن xi ارزشی است که به بازیکنِ i می‌رسد. بازدهی باید قیدِ زیر را ارضا کند که هر ائتلافِ C همهٔ ارزشِ ν(C) را میانِ اعضایش تقسیم می‌کند:

```
C ∈ CS برای همهٔ      xi = ν(C) {i∈C}_∑
```

برای نمونه، با توجه به بازیِ (ν,{1,2,3}) که در آن ν({1}) = 4 و ν({2,3}) = 10، پیامدِ ممکن: ({{2,3},{1}},(4,5,5)) است. یعنی بازیکنِ 1 تنها می‌ماند و ارزشِ 4 را می‌پذیرد، در حالی که بازیکنانِ 2 و 3 با هم تیم می‌شوند تا ارزشِ 10 را دریافت کنند، که تصمیم می‌گیرند به‌طورِ مساوی تقسیم کنند.

برخی بازی‌های همکارانه این ویژگی را دارند که وقتی دو ائتلاف با هم ادغام می‌شوند، بدتر از آنچه اگر جدا می‌ماندند نمی‌شوند. این ویژگی «فوق‌افزایی» (Superadditivity) نامیده می‌شود. به‌صورتِ صوری، بازی فوق‌افزاست اگر تابعِ مشخصه‌اش شرطِ زیر را ارضا کند:

```
C, D ⊆ N برای همهٔ      ν(CuD) ≥ ν(C) + ν(D)
```

اگر بازی فوق‌افزاست، آن‌گاه ائتلافِ بزرگ ارزشی دریافت می‌کند که دستِ‌کم به‌اندازهٔ کلِ دریافت‌شده توسط هر ساختارِ ائتلافِ دیگر است. با این حال، چنان‌که به‌زودی خواهیم دید، بازی‌های فوق‌افزا همیشه به ائتلافِ بزرگ نمی‌انجامند، به همان دلیلی که بازیکنان همیشه در دوراهیِ زندانی به پیامدِ مطلوبِ جمعیِ پارتو-بهینه نمی‌رسند.

۱۸.۳.۲ استراتژی در بازی‌های همکارانه

فرضِ پایه در نظریهٔ بازیِ همکارانه این است که بازیکنان تصمیم‌های استراتژیک دربارهٔ اینکه با چه کسی همکاری کنند خواهند گرفت. به‌صورتِ شهودی، بازیکنان مایل به کار با بازیکنانِ غیرمولد نخواهند بود—آن‌ها طبیعتاً بازیکنانی را جست‌وجو خواهند کرد که به‌صورتِ جمعی ارزشِ ائتلافیِ بالایی می‌دهند. اما این بازیکنانِ پرطرفدار استدلالِ استراتژیکِ خودشان را خواهند کرد. قبل از توصیفِ این استدلال، به چند تعریفِ بیشتر نیاز داریم.

«تخصیص» (Imputation) برای بازیِ همکارانهٔ (N,v) بردارِ بازدهی‌ای است که دو شرطِ زیر را ارضا می‌کند:

$$x_i = v(N) \quad \sum_{i=1}^{n}$$
$$x_i \geq v(\{i\}) \quad \text{برای همهٔ } i \in N$$

شرطِ نخست می‌گوید تخصیص باید ارزشِ کلِ ائتلافِ بزرگ را توزیع کند؛ شرطِ دوم، که «عقلانیتِ فردی» (Individual Rationality) شناخته می‌شود، می‌گوید هر بازیکن دستِ‌کم به‌اندازه‌ای که اگر تنها کار می‌کرد راحت است راحت است.

با توجه به تخصیصِ $x = (x_1,\dots,x_n)$ و ائتلافِ $C \subseteq N$، $x(C)$ را جمعِ $\sum_{i\in C} x_i$ تعریف می‌کنیم—مبلغِ کلِ پرداخت‌شده به C توسط تخصیصِ x.

بعد، «هستهٔ» (Core) بازیِ (N,v) را مجموعهٔ همهٔ تخصیص‌های x تعریف می‌کنیم که شرطِ $x(C) \geq v(C)$ را برای هر ائتلافِ ممکنِ $C \subset N$ ارضا می‌کنند. بدین‌ترتیب، اگر تخصیصِ x در هسته نباشد، ائتلافِ $C \subset N$ هست به‌گونه‌ای که $v(C) > x(C)$. بازیکنانِ C از پیوستن به ائتلافِ بزرگ امتناع می‌ورزیدند، زیرا با چسبیدن به C وضعِ بهتری داشتند.

هستهٔ بازی بنابراین شاملِ همهٔ بردارهای بازدهیِ ممکنی است که هیچ ائتلافی نمی‌توانست به آن‌ها اعتراض کند به این دلیل که با نپیوستن به ائتلافِ بزرگ بهتر عمل می‌کرد. بدین‌ترتیب، اگر هسته خالی باشد، ائتلافِ بزرگ نمی‌تواند تشکیل شود، زیرا مهم نیست ائتلافِ بزرگ بازدهی‌اش را چگونه تقسیم می‌کرد، برخی ائتلاف‌های کوچک‌تر از پیوستن امتناع می‌ورزیدند. پرسش‌های محاسباتیِ اصلیِ پیرامونِ هسته به خالی‌بودن یا نبودنش و اینکه آیا توزیعِ بازدهیِ خاصی در هسته است یا نه مربوط می‌شوند.

تعریفِ هسته به‌طورِ طبیعی به دستگاهِ نامساوی‌های خطی زیر می‌انجامد (مجهول‌ها متغیرهای $x_1,\dots,x_n$ و مقادیرِ $v(C)$ ثابت‌اند):

$$x_i \geq v(\{i\}) \qquad \text{برای همهٔ } i \in N$$
$$\sum_{i\in N} x_i = v(N)$$
$$\sum_{i\in C} x_i \geq v(C) \qquad \text{برای همهٔ } C \subseteq N$$

هر راه‌حلی برای این نامساوی‌ها تخصیصی در هسته تعریف می‌کند. می‌توانیم نامساوی‌ها را با استفاده از تابعِ هدفِ دامی به‌صورتِ برنامهٔ خطی صورت‌بندی کنیم (برای نمونه، بیشینه‌کردنِ ∑_{i∈N} xi)، که به ما اجازه می‌دهد تخصیص‌ها را در زمانِ چندجمله‌ای در شمارِ نامساوی‌ها محاسبه کنیم. دشواری این است که این شمارِ نمایی نامساوی می‌دهد (یکی برای هر یک از 2^n ائتلافِ ممکن). بدین‌ترتیب، این رویکرد الگوریتمی برای بررسیِ غیرخالی‌بودنِ هسته می‌دهد که در زمانِ نمایی اجرا می‌شود. اینکه آیا می‌توانیم بهتر از این عمل کنیم به بازیِ موردِ مطالعه بستگی دارد: برای بسیاری از رده‌های بازیِ همکارانه، مسئلهٔ بررسیِ غیرخالی‌بودنِ هسته هم-NP-کامل است. در زیر مثالی می‌دهیم.

قبل از ادامه، نمونه‌ای از بازیِ فوق‌افزا با هستهٔ خالی ببینیم. بازی سه بازیکن دارد، N = {1,2,3}، و تابعِ مشخصه‌ای چنین تعریف‌شده:

```
ν(C) = { 1   اگر |C| ≥ 2
       { 0   در غیرِ این صورت
```

حالا هر تخصیصِ (x1,x2,x3) برای این بازی را در نظر بگیر. چون ν(N) = 1، باید چنین باشد که دستِ‌کم یک بازیکنِ i با xi > 0 هست و دو تای دیگر بازدهیِ کلِ کمتر از 1 می‌گیرند. آن دو می‌توانستند با تشکیلِ ائتلافی بدونِ بازیکنِ i و اشتراکِ ارزشِ 1 میانِ خودشان سود ببرند. اما چون این برای همهٔ تخصیص‌ها برقرار است، هسته باید خالی باشد.

هسته ایدهٔ پایداربودنِ ائتلافِ بزرگ را صوری می‌کند، به این معنا که هیچ ائتلافی نمی‌تواند با سود از آن منحرف شود. با این حال، هسته ممکن است حاویِ تخصیص‌هایی باشد که نامعقول‌اند، به این معنا که یک یا چند بازیکن ممکن است احساس کنند ناعادلانه‌اند. فرض کنیم N = {1,2}، و تابعِ مشخصه‌ای چنین تعریف‌شده:

```
ν({1}) = ν({2}) = 5        ν({1,2}) = 20
```

در این‌جا، همکاری مازادی 10 بر آنچه بازیکنان با کارِ تنها می‌توانستند به دست آورند تولید می‌کند، و بنابراین، به‌صورتِ شهودی، همکاری در این سناریو معنا خواهد داشت. حالا، به‌آسانی دیده می‌شود تخصیصِ (6,14) در هستهٔ این بازی است: هیچ بازیکنی نمی‌تواند برای به‌دست‌آوردنِ مطلوبیتِ بالاتر منحرف شود. اما از دیدگاهِ بازیکنِ 1، این ممکن است نامعقول به نظر برسد، زیرا 9/10 مازاد را به بازیکنِ 2 می‌دهد. بدین‌ترتیب، مفهومِ هسته به ما می‌گوید ائتلافِ بزرگ چه زمانی می‌تواند تشکیل شود، اما نمی‌گوید بازدهی را چگونه توزیع کنیم.

منصفانه‌بودن به چه معناست؟ توزیعِ ν(N) بر اساسِ رنگِ چشمِ بازیکنان، یا جنسیت‌شان، یا رنگِ پوست‌شان ناعادلانه می‌بود. دانشجویان اغلب پیشنهاد می‌کنند ارزشِ ν(N) باید به‌طورِ مساوی تقسیم شود، که منصفانه به نظر می‌رسد، تا وقتی در نظر بگیریم این به بازیکنانی که زیاد مشارکت می‌کنند و بازیکنانی که هیچ مشارکتی نمی‌کنند همان پاداش را می‌داد. بینشِ شپلی این بود که پیشنهاد کند تنها راهِ منصفانه برای تقسیمِ ارزشِ ν(N) این است که طبقِ میزانِ مشارکتِ هر بازیکن در ایجادِ ارزشِ ν(N) تقسیم شود.

نخست باید مفهومِ «مشارکتِ حاشیه‌ای» (Marginal Contribution) بازیکن را تعریف کنیم. مشارکتِ حاشیه‌ایِ بازیکنِ i در ائتلافِ C، ارزشی است که i می‌افزود (یا حذف می‌کرد)، اگر i به ائتلافِ C بپیوندد. به‌صورتِ صوری، مشارکتِ حاشیه‌ایِ بازیکنِ i در C با mci(C) نشان داده می‌شود:

```
mci(C) = ν(Cu{i}) − ν(C)
```

حالا، نخستین تلاش برای تعریفِ طرحِ تقسیمِ بازدهی مطابقِ پیشنهادِ شپلی که بازیکنان باید طبقِ مشارکتشان پاداش بگیرند، پرداختِ هر بازیکنِ i به‌اندازهٔ ارزشی است که به ائتلافِ شاملِ همهٔ بازیکنانِ دیگر می‌افزود: mci(N-{i}). مسئله این است که این به‌صورتِ ضمنی فرض می‌کند بازیکنِ i آخرین بازیکنی است که واردِ ائتلاف می‌شود.

پس، شپلی پیشنهاد کرد باید همهٔ راه‌های ممکنِ تشکیلِ ائتلافِ بزرگ، یعنی همهٔ ترتیب‌های ممکنِ بازیکنانِ N، را در نظر بگیریم، و ارزشی را که i به بازیکنانِ مقدمِ خودش در ترتیب می‌افزاید در نظر بگیریم. سپس، بازیکن باید با پرداختِ میانگینِ مشارکتِ حاشیه‌ای‌ای که بازیکنِ i، روی همهٔ ترتیب‌های ممکنِ بازیکنان، به مجموعهٔ بازیکنانِ مقدمِ i در ترتیب می‌کند، پاداش بگیرد.

P را مجموعهٔ همهٔ جایگشت‌های ممکن (مثلاً ترتیب‌ها) بازیکنانِ N بگذاریم، و اعضای P را با p، p' و... نشان دهیم. جایی که p ∈ P و i ∈ N، با pi مجموعهٔ بازیکنانی را نشان می‌دهیم که در ترتیبِ p قبل از i می‌آیند. سپس «ارزشِ شپلی» (Shapley Value) برای بازیِ G تخصیصِ φ(G) = (φ1(G),...,φn(G)) است که چنین تعریف می‌شود:

```
φi(G) = (1/n!) ∑_{p∈P} mci(pi)        (18.1)
```

این باید شما را قانع کند ارزشِ شپلی پیشنهادِ معقولی است. اما واقعیتِ قابلِ‌توجه این است که راه‌حلِ یکتای مجموعه‌ای از اصل‌هایی است که طرحِ توزیعِ بازدهیِ «منصفانه» را مشخصه‌سازی می‌کنند. قبل از تعریفِ اصل‌ها به چند تعریفِ بیشتر نیاز داریم.

«بازیکنِ دامی» (Dummy Player) را بازیکنی تعریف می‌کنیم که هرگز ارزشی به ائتلاف نمی‌افزاید—یعنی mci(C) = 0 برای همهٔ C ⊆ N-{i}. خواهیم گفت دو بازیکنِ i و j «بازیکنانِ متقارن» (Symmetric Players) هستند اگر همیشه مشارکت‌های یکسانی در ائتلاف‌ها داشته باشند—یعنی mci(C) = mcj(C) برای همهٔ C ⊆ N-{i,j}. سرانجام، جایی که G = (N,ν) و G' = (N,ν') بازی‌هایی با همان مجموعهٔ بازیکنان‌اند، بازیِ G + G' بازیِ با همان مجموعهٔ بازیکنان و تابعِ مشخصه‌ای است که با ν"(C) = ν(C) + ν'(C) تعریف می‌شود.

با این تعریف‌ها، می‌توانیم اصل‌های انصافی را که ارزشِ شپلی ارضا می‌کند تعریف کنیم:

- **کارایی (Efficiency):** ∑_{i∈N} φi(G) = ν(N). (همهٔ ارزش باید توزیع شود.)
- **بازیکنِ دامی (Dummy Player):** اگر i در G بازیکنِ دامی باشد، آن‌گاه φi(G) = 0. (بازیکنانی که هرگز هیچ‌چیز مشارکت نمی‌کنند هرگز هیچ‌چیز نباید دریافت کنند.)
- **تقارن (Symmetry):** اگر i و j در G متقارن‌اند، آن‌گاه φi(G) = φj(G). (بازیکنانی که مشارکت‌های یکسانی می‌کنند باید بازدهی‌های یکسانی دریافت کنند.)

- **افزایشی‌بودن (Additivity):** ارزش روی بازی‌ها افزایشی است: برای همهٔ بازی‌های $G = (N,\nu)$ و $G' = (N,\nu')$، و برای همهٔ بازیکنانِ $i \in N$، داریم $\varphi i(G+G') = \varphi i(G) + \varphi i(G')$.

اصلِ افزایشی‌بودن اعترافاً نسبتاً فنی است. اما اگر آن را به‌عنوانِ الزامی بپذیریم، می‌توانیم ویژگیِ کلیدیِ زیر را برقرار کنیم: ارزشِ شپلی تنها راهِ توزیعِ ارزشِ ائتلافی است که این اصل‌های انصاف را ارضا می‌کند.

۱۸.۳.۳ محاسبه در بازی‌های همکارانه

از دیدگاهِ نظری، اکنون راه‌حلِ رضایت‌بخشی داریم. اما از دیدگاهِ محاسباتی، باید بدانیم چگونه بازی‌های همکارانه را به‌صورتِ فشرده بازنمایی کنیم، و چگونه مفاهیمِ راه‌حل مانندِ هسته و ارزشِ شپلی را کارآمد محاسبه کنیم.

بازنماییِ بدیهیِ تابعِ مشخصه، جدولی است که ارزشِ $\nu(C)$ را برای همهٔ 2^n ائتلاف فهرست می‌کند. این برای n بزرگ ناشدنی است. شماری از رویکردها برای بازنماییِ فشردهٔ بازی‌های همکارانه توسعه یافته، که می‌توان آن‌ها را بر اساسِ کامل‌بودن یا نبودنشان متمایز کرد. طرحِ بازنماییِ کامل، طرحی است که قادر به بازنماییِ هر بازیِ همکارانه‌ای باشد. عیبِ طرح‌های بازنماییِ کامل این است که همیشه بازی‌هایی خواهند بود که نمی‌توان آن‌ها را به‌صورتِ فشرده بازنمایی کرد. جایگزین، استفاده از طرحِ بازنماییِ تضمین‌شدهٔ فشرده اما غیرکامل است.

شبکه‌های مشارکتِ حاشیه‌ای. حالا یک طرحِ بازنمایی به نام «شبکه‌های مشارکتِ حاشیه‌ای» (Marginal Contribution Nets یا MC-Nets) را توصیف می‌کنیم. از نسخهٔ کمی ساده‌شده‌ای برای تسهیلِ ارائه استفاده خواهیم کرد، و ساده‌سازی آن را ناقص می‌کند—نسخهٔ کاملِ MC-Nets بازنماییِ کاملی است.

ایدهٔ پشتِ شبکه‌های مشارکتِ حاشیه‌ای بازنماییِ تابعِ مشخصهٔ بازیِ (N,ν) به‌صورتِ مجموعه‌ای از «قاعده‌های ارزش-ائتلاف» به شکلِ (Ci,xi) است، که در آن $Ci \subseteq N$ ائتلاف و xi عدد است. برای محاسبهٔ ارزشِ ائتلافِ C، صرفاً ارزشِ همهٔ قاعده‌های (Ci,xi) را جمع می‌کنیم که $Ci \subseteq C$. بدین‌ترتیب، با توجه به مجموعه‌ای از قاعده‌های $R = \{(C1,x1),\dots,(Ck,xk)\}$، تابعِ مشخصهٔ متناظر:

```
ν(C) = ∑ {xi و Ci ⊆ C | (Ci,xi) ∈ R}
```

فرض کنیم مجموعهٔ قاعده‌ای R شاملِ سه قاعدهٔ زیر باشد:

```
{(4,{3}) ,(2,{2}) ,(5,{1,2})}
```

سپس، برای نمونه، داریم: $\nu(\{1\}) = 0$ (زیرا هیچ قاعده‌ای اعمال نمی‌شود)، $\nu(\{3\}) = 4$ (قاعدهٔ سوم)، $\nu(\{1,3\}) = 4$ (قاعدهٔ سوم)، $\nu(\{2,3\}) = 6$ (قاعده‌های دوم و سوم)، و $\nu(\{1,2,3\}) = 11$ (قاعده‌های اول، دوم و سوم).

با این بازنمایی می‌توانیم ارزشِ شپلی را در زمانِ چندجمله‌ای محاسبه کنیم. بینشِ کلیدی این است که هر قاعده را می‌توان به‌عنوانِ تعریفِ بازی‌ای به‌تنهایی فهمید، که در آن بازیکنان متقارن‌اند. با توسل به اصل‌های افزایشی‌بودن و تقارنِ شپلی، بنابراین، ارزشِ شپلیِ $\varphi i(R)$ بازیکنِ i در بازیِ مرتبط با مجموعهٔ قاعده‌های R صرفاً:

```
|φi(R) = ∑_{(C,x)∈R} ( x/|C   اگر i ∈ C
                      0        در غیرِ این صورت )
```

است. نسخه‌ای از شبکه‌های مشارکتِ حاشیه‌ای که این‌جا ارائه کرده‌ایم طرحِ بازنماییِ کاملی نیست: بازی‌هایی هست که تابعِ مشخصه‌شان را نمی‌توان با مجموعه‌های قاعده‌ای از شکلِ توصیف‌شده بازنمایی کرد. نوعِ غنی‌تری از شبکه‌های مشارکتِ حاشیه‌ای قاعده‌هایی به شکلِ (φ,x) اجازه می‌دهد، که در آن φ فرمولِ منطقِ گزاره‌ای روی بازیکنانِ N است: ائتلافِ C شرطِ φ را اگر با تخصیصِ ارضاکننده‌ای برای φ متناظر باشد ارضا می‌کند. این طرح بازنماییِ کاملی است—در بدترین حالت، برای هر ائتلافِ ممکن به قاعده‌ای نیاز داریم. علاوه بر این، ارزشِ شپلی را می‌توان با این طرح در زمانِ چندجمله‌ای محاسبه کرد؛ جزئیات درگیرتر از قاعده‌های سادهٔ توصیف‌شده در بالا هستند، هرچند اصلِ پایه یکسان است؛ مراجع را در یادداشت‌های پایانِ فصل ببینید.

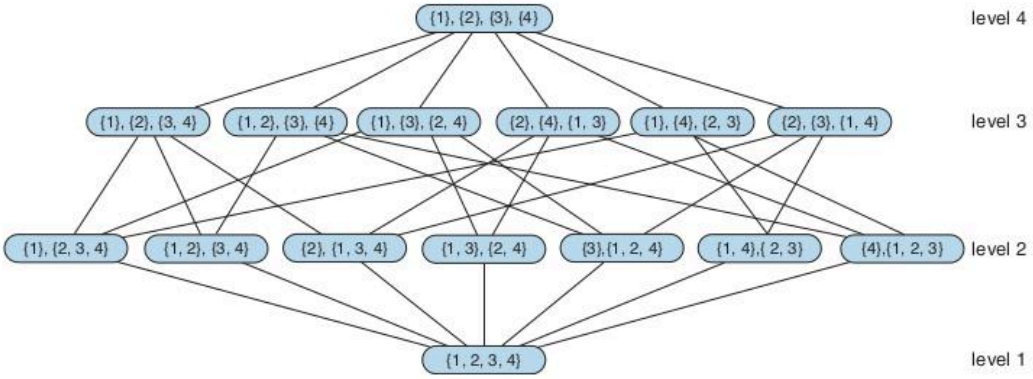
ساختارهای ائتلاف برای بیشینهٔ رفاهِ اجتماعی. اگر فرض کنیم عامل‌ها هدفِ مشترکی دارند، دیدگاهِ متفاوتی از بازی‌های همکارانه به دست می‌آوریم. برای نمونه، اگر عامل‌ها را کارکنانِ شرکت بدانیم، آن‌گاه ملاحظاتِ استراتژیکِ مربوط به تشکیلِ ائتلاف که مثلاً توسط هسته پرداخته می‌شوند، مربوط نیستند. در عوض، ممکن است بخواهیم نیرویِ کار (عامل‌ها) را به تیم‌هایی سازمان دهیم تا بهره‌وریِ کلی‌شان بیشینه شود. عمومی‌تر، تکلیف یافتنِ ائتلافی است که «رفاهِ اجتماعیِ» سامانه را بیشینه می‌کند، که به‌صورتِ جمعِ ارزش‌های ائتلاف‌های فردی تعریف می‌شود. رفاهِ اجتماعیِ ساختارِ ائتلافِ CS را sw(CS) می‌نویسیم، با تعریفِ زیر:

```
sw(CS) = ∑_{C∈CS} v(C)
```

سپس «ساختارِ ائتلافِ بهینهٔ اجتماعی» CS* نسبت به G این کمیت را بیشینه می‌کند.

یافتنِ ساختارِ ائتلافِ بهینهٔ اجتماعی مسئلهٔ محاسباتیِ بسیار طبیعی‌ای است، که فراتر از جامعهٔ سامانه‌های چندعاملی مطالعه شده: گاهی «مسئلهٔ افرازِ مجموعه» (Set Partitioning Problem) نامیده می‌شود. متأسفانه، مسئله NP-سخت است، زیرا شمارِ ساختارهای ائتلافِ ممکن در شمارِ بازیکنان به‌صورتِ نمایی رشد می‌کند.

یافتنِ ساختارِ ائتلافِ بهینه با جستجوی جامعِ ساده‌لوحانه بنابراین به‌طورِ کلی ناشدنی است. رویکردِ تأثیرگذاری برای تشکیلِ ائتلافِ بهینه بر ایدهٔ جستجو در زیرفضایی از «گرافِ ساختارِ ائتلاف» (Coalition Structure Graph) مبتنی است. این ایده بهتر است با ارجاع به مثالی توضیح داده شود. فرض کنیم بازی با چهار عامل، N = {1,2,3,4} داریم. پانزده ساختارِ ائتلافِ ممکن برای این مجموعه از عامل‌ها هست. می‌توانیم این‌ها را در گرافِ ساختارِ ائتلاف چنان‌که در شکل ۱۸.۷ نشان داده شده سازمان دهیم، که در آن گره‌های سطحِ ℓ گراف با همهٔ ساختارهای ائتلافِ با دقیقاً ℓ ائتلاف متناظرند. یالِ رو به بالا در گراف، تقسیمِ ائتلافی در گرهٔ پایینی به دو ائتلافِ جداگانه در گرهٔ بالایی را بازنمایی می‌کند.



شکل ۱۸.۷ — گرافِ ساختارِ ائتلاف برای N = {1,2,3,4}. سطحِ 1 ساختارهای ائتلافِ شاملِ ائتلافِ تکی دارد؛ سطحِ 2 ساختارهای ائتلافِ شاملِ دو ائتلاف دارد، و همین‌طور. (سطح‌های 1 تا 4 با گره‌هایشان نشان داده شده‌اند: {1,2,3,4}؛ ساختارهای دوائتلافی مانندِ {1},{2,3,4} و {1,2}, {3,4}؛ ساختارهای سه‌ائتلافی مانندِ {1},{2},{3,4}؛ و سطحِ 4 با {1},{2},{3},{4}.)

برای نمونه، یالی از {{1},{2,3,4}} به {{1},{2},{3,4}} هست، زیرا این ساختارِ ائتلافِ اخیر از ساختارِ قبلی با تقسیمِ ائتلافِ {2,3,4} به ائتلاف‌های {2} و {3,4} به دست می‌آید. ساختارِ ائتلافِ بهینهٔ CS* جایی درونِ گرافِ ساختارِ ائتلاف است، و برای یافتنِ آن، به نظر می‌رسد باید هر گرهٔ گراف را ارزیابی کنیم. اما دو ردیفِ پایینیِ گراف—سطوحِ 1 و 2—را در نظر بگیرید. هر ائتلافِ ممکن (به‌جزِ ائتلافِ خالی) در این دو سطح ظاهر می‌شود. (البته، نه هر ساختارِ ائتلافِ ممکنی در این دو سطح ظاهر می‌شود.) حالا، فرض کنیم جستجوی‌مان را برای ساختارِ ائتلافِ ممکن به همین دو سطح محدود کنیم—در گراف بالاتر نمی‌رویم. CS' بهترین ساختارِ ائتلافی باشد که در این دو سطح می‌یابیم، و CS* بهترین ساختارِ ائتلاف در کل باشد.

C* ائتلافی با بالاترین ارزش از میانِ همهٔ ائتلاف‌های ممکن باشد:

```
C* ∈ argmax_{C⊆N} v(C)
```

ارزشِ بهترین ساختارِ ائتلافِ یافت‌شده در دو سطحِ نخستِ گرافِ ساختارِ ائتلاف باید دستِ‌کم به‌اندازهٔ ارزشِ بهترین ائتلافِ ممکن باشد: sw(CS') ≥ v(C*). این به این دلیل است که هر ائتلافِ ممکن در دستِ‌کم یک ساختارِ ائتلاف در دو سطحِ نخستِ گراف ظاهر می‌شود. پس بدترین حالت را فرض کنید، یعنی sw(CS') = v(C*). ارزشِ sw(CS') را با sw(CS*) مقایسه کنید. چون sw(CS') بالاترین ارزشِ ممکنِ هر ساختارِ ائتلاف است، و n عامل هست (در موردِ شکل ۱۸.۷، n = 4)، آن‌گاه بالاترین ارزشِ ممکنِ sw(CS*) برابر n·sw(CS') = n·v(C*) خواهد بود. به بیانِ دیگر، در بدترین حالتِ ممکن، ارزشِ بهترین ساختارِ ائتلافِ یافت‌شده در دو سطحِ نخستِ گراف، 1/n ارزشِ بهترین خواهد بود، که در آن n شمارِ عامل‌هاست. بدین‌ترتیب، اگرچه جستجو در دو سطحِ نخستِ گراف یافتنِ ساختارِ ائتلافِ بهینه را تضمین نمی‌کند، یافتنِ ساختاری را تضمین می‌کند که بدتر از 1/n بهینه نیست. در عمل اغلب بسیار بهتر از آن خواهد بود.

۱۸.۴ تصمیم‌گیریِ جمعی

حالا از طراحیِ عامل به طراحیِ سازوکار می‌پردازیم—مسئلهٔ طراحیِ بازیِ درست برای مجموعه‌ای از عامل‌ها. به‌صورتِ صوری، سازوکار شاملِ:

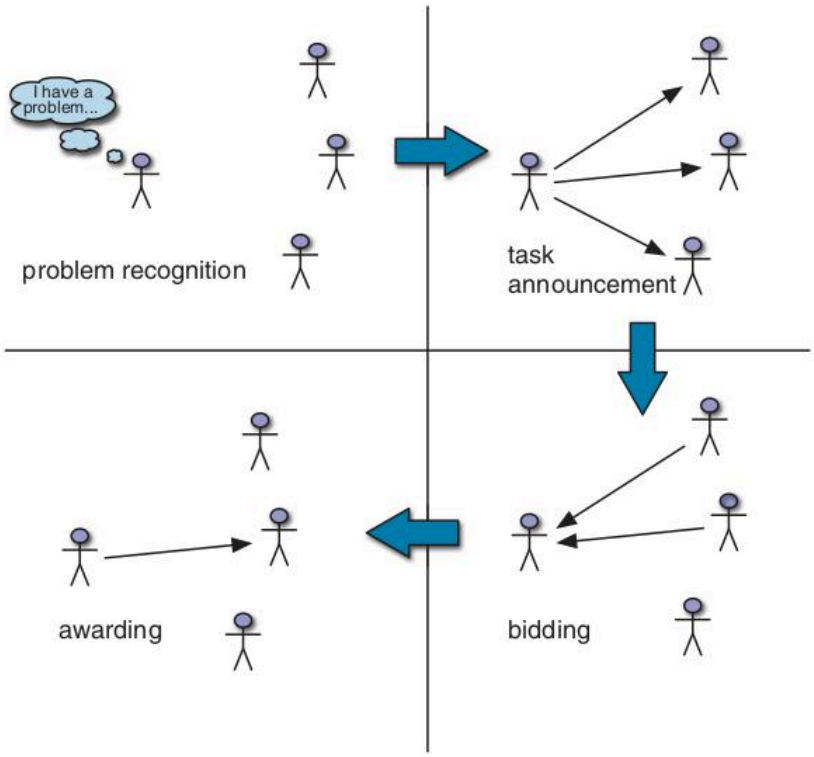
1. زبانی برای توصیفِ مجموعهٔ استراتژی‌های مجاز که عامل‌ها ممکن است اتخاذ کنند.
2. عاملِ متمایزی به نامِ «مرکز» (Center)، که گزارش‌های انتخاب‌های استراتژی را از عامل‌های بازی جمع‌آوری می‌کند. (برای نمونه، حراج‌کننده در حراج مرکز است.)
3. «قاعدهٔ پیامدِ» (Outcome Rule) شناخته‌شده برای همهٔ عامل‌ها، که مرکز برای تعیینِ بازدهی‌های هر عامل، با توجه به انتخاب‌های استراتژی‌شان، از آن استفاده می‌کند.

این بخش برخی از مهم‌ترین سازوکارها را بحث می‌کند.

۱۸.۴.۱ تخصیصِ وظایف با شبکهٔ قرارداد

«پروتکلِ شبکهٔ قرارداد» (Contract Net Protocol) احتمالاً قدیمی‌ترین و مهم‌ترین فنِ حلِ مسئلهٔ چندعاملیِ مطالعه‌شده در هوش مصنوعی است. پروتکلِ سطح‌بالایی برای به‌اشتراک‌گذاریِ وظایف است. چنان‌که نامش نشان می‌دهد، شبکهٔ قرارداد از شیوه‌ای الهام گرفته که شرکت‌ها از قراردادها استفاده می‌کنند.

پروتکلِ کلیِ شبکهٔ قرارداد چهار فازِ اصلی دارد—شکل ۱۸.۸ را ببینید. فرایند با شناساییِ نیاز به کنشِ همکارانه نسبت به وظیفه‌ای توسط عاملی شروع می‌شود. نیاز ممکن است پدید آید، زیرا عامل توانایی اجرایِ وظیفه را به‌تنهایی ندارد، یا به این دلیل که راه‌حلِ همکارانه ممکن است به نحوی بهتر باشد (سریع‌تر، کارآمدتر، دقیق‌تر). عامل وظیفه را به عامل‌های دیگرِ شبکه با پیامِ «اعلامِ وظیفه» (Task Announcement) تبلیغ می‌کند، و سپس برای مدتِ آن وظیفه به‌عنوانِ «مدیرِ» (Manager) آن عمل می‌کند. پیامِ اعلامِ وظیفه باید اطلاعاتِ کافی برای گیرندگان شامل شود تا قضاوت کنند آیا مایل‌اند و قادرند برای وظیفه پیشنهاد دهند. اطلاعاتِ دقیقِ شامل‌شده در اعلامِ وظیفه به حوزهٔ کاربرد بستگی خواهد داشت. ممکن است کدی باشد که باید اجرا شود؛ یا ممکن است مشخصاتِ منطقیِ هدفی باشد که باید محقق شود. اعلامِ وظیفه همچنین ممکن است شاملِ اطلاعاتِ دیگری باشد که گیرندگان ممکن است نیاز داشته باشند، مانندِ مهلت‌ها، الزاماتِ کیفیتِ سرویس و مانندِ آن.



شکل ۱۸.۸ – پروتکلِ تخصیصِ وظیفهٔ شبکهٔ قرارداد. (فازها: «من مشکلی دارم...» تشخیصِ مسئله، اعلامِ وظیفه، پیشنهاددهی، اعطا.)

وقتی عاملی اعلامِ وظیفه‌ای دریافت می‌کند، باید آن را نسبت به توانایی‌ها و ترجیح‌های خودش ارزیابی کند. به‌طورِ خاص، هر عامل باید تعیین کند، نخست، آیا توانایی اجرایِ وظیفه را دارد، و دوم، آیا مایل به انجامِ آن است. بر این اساس، ممکن است سپس «پیشنهادی» (Bid) برای وظیفه ارائه کند. پیشنهاد معمولاً توانایی‌های پیشنهاددهنده‌ای را که به وظیفهٔ تبلیغ‌شده مربوط‌اند، و هر شرایط و ضوابطی که وظیفه تحتِ آن انجام خواهد شد، نشان می‌دهد. به‌طورِ کلی، مدیر ممکن است در پاسخ به اعلامِ وظیفهٔ تکی چند پیشنهاد دریافت کند. بر اساسِ اطلاعاتِ پیشنهادها، مدیر مناسب‌ترین عامل (یا عامل‌ها) را برای اجرایِ وظیفه انتخاب می‌کند. عامل‌های موفق از طریقِ پیامِ «اعطا» (Award) مطلع می‌شوند و برای وظیفه «پیمانکار» (Contractor) می‌شوند و تا تکمیلِ وظیفه مسئولیتِ آن را بر عهده می‌گیرند.

تکلیف‌های محاسباتیِ اصلیِ لازم برای پیاده‌سازیِ پروتکلِ شبکهٔ قرارداد را می‌توان چنین خلاصه کرد:

- **پردازشِ اعلامِ وظیفه.** با دریافتِ اعلامِ وظیفه، عامل تصمیم می‌گیرد آیا مایل است برای وظیفهٔ تبلیغ‌شده پیشنهاد دهد.
- **پردازشِ پیشنهاد.** با دریافتِ چند پیشنهاد، مدیر باید تصمیم بگیرد کدام عامل را برای وظیفه انتخاب کند و سپس وظیفه را اعطا کند.
- **پردازشِ اعطا.** پیشنهاددهندگانِ موفق (پیمانکاران) باید تلاش کنند وظیفه را اجرا کنند، که ممکن است به معنای تولیدِ زیروظایفِ تازه‌ای باشد، که از طریقِ اعلام‌های وظیفهٔ بیشتر تبلیغ می‌شوند.

با وجودِ (یا شاید به دلیلِ) سادگی‌اش، شبکهٔ قرارداد احتمالاً پرکاربردترین و بهترین‌مطالعه‌شده‌ترین چارچوب برای حلِ مسئلهٔ همکارانه است. به‌طورِ طبیعی در بسیاری از محیط‌ها قابلِ اعمال است—هر بار که با اوبر خودرو درخواست می‌کنید، تغییری از آن اجرا می‌شود.

۱۸.۴.۲ تخصیصِ منابعِ کمیاب با حراج‌ها

یکی از مهم‌ترین مسائل در سامانه‌های چندعاملی تخصیصِ منابعِ کمیاب است؛ اما ممکن است به‌همان خوبی فقط بگوییم «تخصیصِ منابع»، زیرا در عمل بیشترِ منابعِ مفید از جهتی کمیاب‌اند. «حراج» (Auction) مهم‌ترین سازوکارِ تخصیصِ منابع است. ساده‌ترین محیط برای حراج جایی است که یک منبعِ تکی و چند «پیشنهاددهندهٔ» (Bidder) ممکن هست. هر پیشنهاددهندهٔ i برای قلم ارزشِ مطلوبیتیِ vi دارد.

در برخی موارد، هر پیشنهاددهنده برای قلم «ارزشِ خصوصی» (Private Value) دارد. برای نمونه، ژاکتِ بدقواره‌ای ممکن است برای یک پیشنهاددهنده جذاب و برای دیگری بی‌ارزش باشد. در مواردِ دیگر، مانندِ حراجِ حقوقِ حفاریِ بلوکِ نفتی، قلم «ارزشِ مشترک» (Common Value) دارد—بلوک مقداری پول، X، تولید خواهد کرد و همهٔ پیشنهاددهندگان دلار را به‌یکسان ارزش‌گذاری می‌کنند—اما عدم قطعیت دربارهٔ مقدارِ واقعیِ X هست. پیشنهاددهندگانِ مختلف اطلاعاتِ متفاوتی دارند، و از این رو برآوردهای متفاوتی از ارزشِ واقعیِ قلم دارند. در هر صورت، پیشنهاددهندگان به vi خودشان می‌رسند. با توجه به vi، هر پیشنهاددهنده فرصتی می‌گیرد، در زمانِ یا زمان‌های مناسبِ حراج، پیشنهادی bi بدهد. بالاترین پیشنهاد، bmax، قلم را می‌برد، اما قیمتِ پرداختی لازم نیست bmax باشد؛ آن بخشی از طراحیِ سازوکار است.

شناخته‌شده‌ترین سازوکارِ حراج، «حراجِ پیشنهادِ صعودی» (Ascending-Bid Auction) است،[3] یا «حراجِ انگلیسی» (English Auction)، که در آن مرکز با درخواستِ پیشنهادِ کمینه (یا رزرو) bmin شروع می‌کند. اگر پیشنهاددهنده‌ای مایل به پرداختِ آن مبلغ باشد، مرکز سپس bmin + d را برای افزایشِ d درخواست می‌کند، و از آن‌جا به بالا ادامه می‌دهد. حراج وقتی پایان می‌یابد که هیچ‌کس مایل به پیشنهادِ بیشتر نباشد؛ سپس آخرین پیشنهاددهنده قلم را می‌برد و قیمتِ پیشنهادشده را می‌پردازد.

از کجا بدانیم این سازوکارِ خوبی است؟ یکی از هدف‌ها بیشینه‌کردنِ درآمدِ موردانتظار برای فروشنده است. هدفِ دیگر بیشینه‌کردنِ تصوری از مطلوبیتِ سراسری است. این هدف‌ها تا حدی هم‌پوشانی دارند، زیرا یکی از جنبه‌های بیشینه‌کردنِ مطلوبیتِ سراسری تضمینِ این است که برندهٔ حراج عاملی است که قلم را بیشترین ارزش‌گذاری می‌کند (و بنابراین مایل به پرداختِ بیشترین است). می‌گوییم حراج «کارآمد» (Efficient) است اگر کالاها به عاملی برسند که بیش از همه ارزش‌گذاری‌شان می‌کند. حراجِ پیشنهادِ صعودی معمولاً هم کارآمد است و هم درآمد-بیشینه، اما اگر قیمتِ رزرو بیش‌ازحد بالا تنظیم شود، پیشنهاددهنده‌ای که بیش از همه ارزش‌گذاری می‌کند ممکن است پیشنهاد ندهد، و اگر رزرو بیش‌ازحد پایین باشد، فروشنده ممکن است درآمدِ کمتری بگیرد. احتمالاً مهم‌ترین کاری که سازوکارِ حراج می‌تواند بکند تشویقِ شمارِ کافی از پیشنهاددهندگان به ورود به بازی و دلسردکردنِ آن‌ها از درگیرشدن در «تبانی» (Collusion) است. تبانی توافقِ ناعادلانه یا غیرقانونیِ دو یا چند پیشنهاددهنده برای دست‌کاریِ قیمت‌هاست. می‌تواند در معاملاتِ مخفیانهٔ پشتی یا به‌صورتِ ضمنی، درونِ قاعده‌های سازوکار، رخ دهد. برای نمونه، در ۱۹۹۹، آلمان ده بلوکِ طیفِ تلفنِ همراه را با حراجِ هم‌زمان حراج کرد (پیشنهادها روی هر ده بلوک هم‌زمان گرفته شد)، با استفاده از قاعده‌ای که هر پیشنهاد باید حداقل ۱۰٪ افزایش بر پیشنهادِ قبلیِ بلوک باشد. فقط دو پیشنهاددهندهٔ معتبر بود، و نخستین، مانسمان، پیشنهادِ ۲۰ میلیون مارک آلمان روی بلوک‌های 1-5 و 18.18 میلیون روی بلوک‌های 6-10 داد. چرا 18.18M؟ یکی از مدیرانِ تی-موبایل گفت آن‌ها «نخستین پیشنهادِ مانسمان را پیشنهادی تفسیر کردند.» هر دو

طرف می‌توانستند محاسبه کنند 10٪ افزایشِ 18.18M برابر 19.99M است؛ بدین‌ترتیب پیشنهادِ مانسمان به‌عنوانِ گفتنِ این تفسیر شد: «هر کدام می‌توانیم نصفِ بلوک‌ها را با 20M بگیریم؛ بیایید با بالا بردنِ قیمت‌ها خرابش نکنیم.» و در واقع تی-موبایل 20M روی بلوک‌های 6-10 پیشنهاد داد و پایانِ پیشنهاددهی بود.

دولتِ آلمان کمتر از انتظارش گرفت، زیرا دو رقیب توانستند از سازوکارِ پیشنهاددهی برای رسیدن به توافقِ ضمنی دربارهٔ اینکه چگونه رقابت نکنند استفاده کنند. از دیدگاهِ دولت، با هر یک از این تغییرهای سازوکار نتیجهٔ بهتری می‌شد گرفت: قیمتِ رزروِ بالاتر؛ حراجِ پیشنهادِ مُهروموم قیمت-اول، تا رقبا نتوانند از طریقِ پیشنهادهایشان ارتباط برقرار کنند؛ یا مشوق‌هایی برای آوردنِ پیشنهاددهندهٔ سوم. شاید قاعدهٔ 10٪ خطایی در طراحیِ سازوکار بود، زیرا سیگنال‌دهیِ دقیق از مانسمان به تی-موبایل را تسهیل می‌کرد.

به‌طورِ کلی، هم فروشنده و هم تابعِ مطلوبیتِ سراسری از پیشنهاددهندگانِ بیشتر سود می‌برند، هرچند مطلوبیتِ سراسری می‌تواند آسیب ببیند اگر هزینهٔ زمانِ تلف‌شدهٔ پیشنهاددهندگانی را بشمارید که شانسی برای بردن ندارند. یکی از راه‌های تشویقِ پیشنهاددهندگانِ بیشتر، آسان‌ترکردنِ سازوکار برای آن‌هاست. بالاخره، اگر به تحقیق یا محاسبهٔ زیادی از جانبِ پیشنهاددهندگان نیاز داشته باشد، ممکن است تصمیم بگیرند پولشان را جایِ دیگر ببرند.

پس مطلوب است پیشنهاددهندگان «استراتژیِ مسلط» داشته باشند. به یاد آورید «مسلط» بدان معناست استراتژی در برابرِ همهٔ استراتژی‌های دیگر کار می‌کند، که به‌نوبهٔ خود بدان معناست عامل می‌تواند آن را بدونِ توجه به استراتژی‌های دیگر اتخاذ کند. عاملی با استراتژیِ مسلط می‌تواند فقط پیشنهاد دهد، بدونِ تلف‌کردنِ وقت در تأمل دربارهٔ استراتژی‌های ممکنِ عامل‌های دیگر. سازوکاری که در آن عامل‌ها دارای استراتژیِ مسلط هستند «سازوکارِ مقاوم در برابرِ دستکاری / ضداستراتژی» (Strategy-Proof) نامیده می‌شود. اگر، چنان‌که معمولاً هست، آن استراتژی شاملِ فاش‌کردنِ ارزشِ واقعیِ v_i توسط پیشنهاددهندگان باشد، آن‌گاه «حراجِ حقیقت‌فاش» (Truth-Revealing) یا «صادقانه» (Truthful) نامیده می‌شود؛ اصطلاحِ «سازگار با انگیزه» (Incentive Compatible) نیز استفاده می‌شود. «اصلِ فاش‌سازی» (Revelation Principle) بیان می‌کند هر سازوکاری را می‌توان به سازوکارِ حقیقت‌فاشِ معادلی تبدیل کرد، پس بخشی از طراحیِ سازوکار یافتنِ این سازوکارهای معادل است.

معلوم می‌شود حراجِ پیشنهادِ صعودی بیشترِ ویژگی‌های مطلوب را دارد. پیشنهاددهنده با بالاترین ارزشِ v_i کالاها را به قیمتِ $b_o + d$ می‌گیرد، که در آن b_o بالاترین پیشنهاد در میانِ همهٔ عامل‌های دیگر و d افزایشِ حراج‌کننده است.[۴] پیشنهاددهندگان استراتژیِ مسلطِ ساده‌ای دارند: تا وقتی هزینهٔ کنونی زیرِ v_i شماست به پیشنهاد ادامه بده. سازوکار کاملاً حقیقت‌فاش نیست، زیرا برندهٔ پیشنهاد فقط فاش می‌کند $v_i \geq b_o + d$؛ کرانِ پایینی بر v_i داریم اما مقدارِ دقیق را نه.

عیبِ حراجِ پیشنهادِ صعودی (از دیدگاهِ فروشنده) این است که می‌تواند رقابت را دلسرد کند. فرض کنیم در پیشنهادی برای طیفِ تلفنِ همراه شرکتی ممتاز هست که همه موافق‌اند می‌تواند از مشتریان و زیرساختِ موجود بهره بگیرد، و بنابراین می‌تواند سودِ بزرگ‌تری از هر کسِ دیگر بکند. رقبای بالقوه می‌بینند در حراجِ پیشنهادِ صعودی هیچ شانسی ندارند، زیرا شرکتِ ممتاز همیشه می‌تواند بالاتر پیشنهاد دهد. بدین‌ترتیب، رقبا ممکن است اصلاً وارد نشوند و شرکتِ ممتاز با بردنِ حراج در قیمتِ رزرو پایان می‌دهد.

ویژگیِ منفیِ دیگرِ حراجِ انگلیسی هزینه‌های ارتباطیِ بالایش است. یا حراج در یک اتاق برگزار می‌شود یا همهٔ پیشنهاددهندگان باید خطوطِ ارتباطیِ پرسرعت و امن داشته باشند؛ در هر صورت باید وقت داشته باشند چند دورِ پیشنهاددهی را طی کنند.

سازوکارِ جایگزین، که به ارتباطِ بسیار کمتری نیاز دارد، «حراجِ پیشنهادِ مُهروموم» (Sealed-Bid Auction) است. هر پیشنهاددهنده پیشنهادِ تکی می‌دهد و بدونِ دیده‌شدنِ آن توسط سایر پیشنهاددهندگان به حراج‌کننده ابلاغ می‌کند. با این سازوکار، دیگر استراتژیِ مسلطِ ساده‌ای نیست. اگر ارزشِ شما vi است و باور دارید بیشینهٔ همهٔ پیشنهادهای عامل‌های دیگر bo خواهد بود، باید bo + ϱ پیشنهاد دهید، برای ϱ کوچکی، اگر آن کمتر از vi باشد. بدین‌ترتیب، پیشنهادِ شما به برآوردِ شما از پیشنهادهای عامل‌های دیگر بستگی دارد، که مستلزم کارِ بیشتر شماست. همچنین توجه کنید عاملِ با بالاترین vi ممکن است حراج را نبرد. این با این واقعیت جبران می‌شود که حراج رقابتی‌تر است و سوگیریِ به سمتِ پیشنهاددهندهٔ ممتاز را کاهش می‌دهد.

تغییرِ کوچکی در سازوکارِ حراج‌های پیشنهادِ مُهروموم به «حراجِ پیشنهادِ مُهروموم قیمت-دوم» می‌انجامد، که «حراجِ ویکری» (Vickrey Auction) نیز نامیده می‌شود.[5] در چنین حراج‌هایی، برنده قیمتِ دومین پیشنهادِ بالاتر، bo، را می‌پردازد، نه پیشنهادِ خودش را. این اصلاحِ ساده تأملاتِ پیچیدهٔ لازم برای حراج‌های پیشنهادِ مُهروموم استاندارد (یا قیمت-اول) را کاملاً حذف می‌کند، زیرا استراتژیِ مسلط حالا صرفاً پیشنهادِ vi است؛ سازوکار حقیقت‌فاش است. توجه کنید مطلوبیتِ عاملِ i بر حسبِ پیشنهادش bi، ارزشش vi و بهترین پیشنهاد در میانِ عامل‌های دیگر، bo، چنین است:

```
Ui = { (vi − bo)   اگر bi > bo
       0           در غیرِ این صورت }
```

برای دیدنِ اینکه bi = vi استراتژیِ مسلط است، توجه کنید وقتی (vi-bo) مثبت است، هر پیشنهادی که حراج را ببرد بهینه است، و پیشنهادِ vi به‌طورِ خاص حراج را می‌برد. از سوی دیگر، وقتی (vi-bo) منفی است، هر پیشنهادی که حراج را ببازد بهینه است، و پیشنهادِ vi به‌طورِ خاص حراج را می‌بازد. پس پیشنهادِ vi برای همهٔ مقادیرِ ممکنِ bo بهینه است، و در واقع، vi تنها پیشنهادی است که این ویژگی را دارد. به دلیلِ سادگی‌اش و حداقل‌بودنِ الزاماتِ محاسباتی برای هم فروشنده و هم پیشنهاددهندگان، حراجِ ویکری در سامانه‌های هوش مصنوعیِ توزیع‌شده به‌طورِ گسترده استفاده می‌شود.

موتورهای جستجوی اینترنتی هر سال چند تریلیون حراج برای فروشِ تبلیغات همراه با نتایجِ جستجویشان برگزار می‌کنند، و سایت‌های حراجِ برخط سالانه ۱۰۰ میلیارد دلار کالا را مدیریت می‌کنند، همه با استفاده از انواعِ حراجِ ویکری. توجه کنید ارزشِ موردانتظار برای فروشنده bo است، که همان بازدهِ موردانتظارِ حدِّ حراجِ انگلیسی با گرایشِ d به صفر است. این در واقع نتیجه‌ای بسیار عمومی است: «قضیهٔ هم‌ارزیِ درآمد» (Revenue Equivalence Theorem) بیان می‌کند، با چند ملاحظهٔ جزئی، هر سازوکارِ حراجی که در آن پیشنهاددهندگان ارزش‌های vi را دارند که فقط خودشان می‌دانند (اما توزیعِ احتمالیِ نمونه‌برداریِ آن ارزش‌ها را می‌دانند)، همان درآمدِ موردانتظار را خواهد داد. این اصل بدان معناست سازوکارهای گوناگون بر اساسِ تولیدِ درآمد با هم رقابت نمی‌کنند، بلکه بر اساسِ کیفیت‌های دیگر.

اگرچه حراجِ قیمت-دوم حقیقت‌فاش است، معلوم می‌شود حراجِ n کالا با حراجِ قیمتِ n+1 حقیقت‌فاش نیست. بسیاری از موتورهای جستجوی اینترنتی از سازوکاری استفاده می‌کنند که در آن n جایگاه برای تبلیغات در صفحه حراج می‌شود. بالاترین پیشنهاددهنده جایگاهِ بالایی را می‌برد، دومینِ بالاتر جایگاهِ دوم را، و همین‌طور. هر برنده قیمتی را می‌پردازد که پیشنهاددهندهٔ پایین‌ترِ بعدی پیشنهاد کرده، با این تفاهم که پرداخت فقط اگر جستجوگر واقعاً روی تبلیغ کلیک کند انجام می‌شود. جایگاه‌های بالا ارزشمندتر دانسته می‌شوند، زیرا محتمل‌تر است دیده و کلیک شوند. تصور کنید سه پیشنهاددهنده، b1، b2 و b3، ارزش‌گذاریِ کلیکِ v1 = 200، v2 = 180 و v3 = 100 دارند، و n = 2 جایگاه موجود است؛ و معلوم است جایگاهِ بالا ۵٪ مواقع و جایگاهِ پایین ۲٪ مواقع کلیک می‌شود. اگر همهٔ پیشنهاددهندگان صادقانه پیشنهاد دهند، b1 جایگاهِ بالا را می‌برد و 180 می‌پردازد، و بازدهِ موردانتظارِ (200-180)×0.05 = 1 دارد. جایگاهِ دوم به b2 می‌رسد. اما b1 می‌تواند ببیند اگر هر چیزی در بازهٔ 101–179 پیشنهاد می‌داد، جایگاهِ بالا را به b2 واگذار می‌کرد، جایگاهِ دوم را می‌برد، و بازدهِ موردانتظارِ (200-100)×.02 = 2 می‌داشت. بدین‌ترتیب، b1 می‌تواند با پیشنهادِ کمتر از ارزشِ واقعی‌اش در این مورد، بازدهِ موردانتظارش را دو برابر کند.

به‌طورِ کلی، پیشنهاددهندگان در این حراجِ قیمتِ n+1 باید انرژیِ زیادی صرفِ تحلیلِ پیشنهادهای دیگران برای تعیینِ بهترین استراتژی‌شان کنند؛ هیچ استراتژیِ مسلطِ ساده‌ای نیست. آگاروال و همکاران (2006) نشان می‌دهند سازوکارِ حراجِ حقیقت‌فاشِ یکتایی برای این مسئلهٔ چندجایگاهی هست، که در آن برندهٔ جایگاهِ j قیمتِ جایگاهِ j را فقط برای آن کلیک‌های اضافی که در جایگاهِ j و نه در جایگاهِ j+1 در دسترس‌اند می‌پردازد. برنده برای کلیک‌های باقی‌مانده قیمتِ جایگاهِ پایین‌تر را می‌پردازد. در مثالِ ما، b1 صادقانه 200 پیشنهاد می‌کرد، و برای کلیک‌های اضافیِ .05-.02 = .03 در جایگاهِ بالا 180 می‌پرداخت، اما برای .02 کلیکِ باقی‌مانده فقط هزینهٔ جایگاهِ پایین، 100 را می‌پرداخت. بدین‌ترتیب، بازدهِ کلِ b1 برابر (200-180)×.03 + (200-100)×.02 = 2.6 می‌بود.

مثالِ دیگری از جایی که حراج‌ها می‌توانند در هوش مصنوعی به کار آیند، وقتی است که مجموعه‌ای از عامل‌ها تصمیم می‌گیرند آیا در برنامهٔ مشترکی همکاری کنند. هانسبرگر و گروز (2000) نشان می‌دهند این را می‌توان با حراجی کارآمد انجام داد که در آن عامل‌ها برای نقش‌ها در برنامهٔ مشترک پیشنهاد می‌دهند.

کالاهای مشترک. حالا نوعِ دیگری از بازی را در نظر بگیریم، که در آن کشورها سیاستشان را برای کنترلِ آلودگیِ هوا تعیین می‌کنند. هر کشور انتخابی دارد: می‌توانند آلودگی را با هزینهٔ -10 امتیاز برای پیاده‌سازیِ تغییرهای لازم کاهش دهند، یا می‌توانند به آلودگی ادامه دهند، که مطلوبیتِ خالصِ -5 به آن‌ها می‌دهد (در هزینه‌های بهداشتیِ افزوده و...) و همچنین به هر کشورِ دیگر -1 امتیاز کمک می‌کند (زیرا هوا میانِ کشورها مشترک است). بدیهی است، استراتژیِ مسلط برای هر کشور «ادامهٔ آلودگی» است، اما اگر ۱۰۰ کشور باشد و هر یک این سیاست را دنبال کند، هر کشور مطلوبیتِ کلِ -104 می‌گیرد، در حالی که اگر همهٔ کشورها آلودگی را کاهش می‌دادند، هر یک مطلوبیتِ -10 داشت. این وضعیت «تراژدیِ مشترکات» (Tragedy of the Commons) نامیده می‌شود: اگر هیچ‌کس مجبور نباشد برای استفاده از منبعِ مشترک بپردازد، ممکن است به‌گونه‌ای بهره‌برداری شود که به مطلوبیتِ کلِ کمتری برای همهٔ عامل‌ها می‌انجامد. شبیهِ دوراهیِ زندانی است: راه‌حلِ دیگری برای بازی هست که برای همهٔ طرف‌ها بهتر است، اما به نظر می‌رسد هیچ راهی برای عامل‌های عقلایی برای رسیدن به آن راه‌حل تحتِ بازیِ کنونی نیست.

یکی از رویکردهای مقابله با تراژدیِ مشترکات تغییرِ سازوکار به سازوکاری است که از هر عامل برای استفاده از مشترکات هزینه می‌گیرد. عمومی‌تر، باید مطمئن شویم همهٔ «پیامدهای خارجی / اثراتِ جانبی» (Externalities)—تأثیراتی بر مطلوبیتِ سراسری که در مبادلاتِ عامل‌های فردی محاسبه نمی‌شوند—شفاف و صریح گردند. تعیینِ درستِ قیمت‌ها بخشِ دشوار است. در حدِّ نهایی، این رویکرد به ایجادِ سازوکاری می‌انجامد که در آن هر عامل به‌صورتِ مؤثر ملزم به بیشینه‌کردنِ مطلوبیتِ سراسری است، اما می‌تواند این کار را با تصمیمِ محلی انجام دهد. برای این مثال، «مالیاتِ کربن» نمونه‌ای از سازوکاری است که برای استفاده از مشترکات هزینه می‌گیرد به‌گونه‌ای که، اگر خوب پیاده‌سازی شود، مطلوبیتِ سراسری را بیشینه می‌کند.

معلوم می‌شود طراحیِ سازوکاری هست، به نامِ «سازوکارِ ویکری–کلارک–گرووز» (Vickrey–Clarke–Groves) یا VCG، که دو ویژگیِ مطلوب دارد. نخست، مطلوبیت-بیشینه است—یعنی مطلوبیتِ سراسری، که جمعِ مطلوبیت‌های همهٔ طرف‌ها، ∑ivi، است، بیشینه می‌کند. دوم، سازوکار حقیقت‌فاش است—استراتژیِ مسلط برای همهٔ عامل‌ها فاش‌کردنِ ارزشِ واقعی‌شان است. نیازی به درگیرشدن در محاسباتِ پیشنهادِ استراتژیکِ پیچیده نیست.

مثالی با استفاده از مسئلهٔ تخصیصِ برخی کالاهای مشترک می‌زنیم. فرض کنیم شهری تصمیم می‌گیرد چند فرستندهٔ وای‌فایِ رایگانِ اینترنتی نصب کند. با این حال، شمارِ فرستنده‌های موجود کمتر از شمارِ محله‌هایی است که آن‌ها را می‌خواهند. شهر می‌خواهد مطلوبیتِ سراسری را بیشینه کند، اما اگر به هر شورای محله بگوید «فرستندهٔ رایگان را چقدر ارزش‌گذاری می‌کنید (و ضمناً آن‌ها را به طرف‌هایی می‌دهیم که بیش از همه ارزش‌گذاری‌شان می‌کنند)؟»، آن‌گاه هر محله انگیزه‌ای برای گزارشِ ارزشِ بسیار بالایی خواهد داشت. سازوکارِ VCG این حیله را دلسرد می‌کند و به آن‌ها انگیزه‌ای برای گزارشِ ارزشِ واقعی‌شان می‌دهد.

این‌گونه کار می‌کند: ۱. مرکز از هر عامل می‌خواهد ارزشش را برای قلم، vi، گزارش کند. ۲. مرکز کالاها را به مجموعه‌ای از برندگان، W، تخصیص می‌دهد تا ∑_{i∈W} vi بیشینه شود. ۳. مرکز برای هر عاملِ برنده محاسبه می‌کند حضورِ فردی‌اش در بازی چقدر زیان به بازندگان وارد کرده (که هر یک مطلوبیتِ 0 گرفتند، اما اگر برنده بودند می‌توانستند vj بگیرند). ۴. هر عاملِ برنده سپس «مالیاتِ» (Tax) برابرِ این زیان به مرکز می‌پردازد.

برای نمونه، فرض کنیم ۳ فرستنده موجود و ۵ پیشنهاددهنده باشد، که پیشنهادهای 100، 50، 40، 20 و 10 می‌دهند. بدین‌ترتیب، مجموعهٔ ۳ برنده، W، آن‌هایی‌اند که 100، 50 و 40 پیشنهاد کردند، و مطلوبیتِ سراسریِ تخصیصِ این کالاها 190 است. برای هر برنده، چنین است که اگر در بازی نبودند، پیشنهادِ 20 برنده می‌شد. بدین‌ترتیب، هر برنده مالیاتِ 20 به مرکز می‌پردازد. همهٔ برندگان باید خوشحال باشند، زیرا مالیاتی کمتر از ارزششان می‌پردازند، و همهٔ بازندگان تا آن‌جا که می‌توانند خوشحال‌اند، زیرا کالاها را کمتر از مالیاتِ لازم ارزش‌گذاری می‌کنند. به همین دلیل است که سازوکار حقیقت‌فاش است. در این مثال، ارزشِ حیاتی 20 است؛ پیشنهادِ بالای 20 اگر ارزشِ واقعی‌تان در واقع زیرِ 20 بود غیرعقلایی می‌بود، و برعکس.

چون ارزشِ حیاتی می‌تواند هر چیزی باشد (بسته به پیشنهاددهندگانِ دیگر)، بدان معناست همیشه غیرعقلایی است چیزی غیر از ارزشِ واقعی‌تان پیشنهاد دهید.

سازوکارِ VCG بسیار عمومی است و با تعمیمِ جزئیِ سازوکارِ توصیف‌شده در بالا، می‌توان آن را برای انواعِ بازی‌ها اعمال کرد، نه فقط حراج‌ها. برای نمونه، در «حراجِ ترکیبی» (Combinatorial Auction) چند قلمِ متفاوت موجود است و هر پیشنهاددهنده می‌تواند چند پیشنهاد بگذارد، هر یک روی زیرمجموعه‌ای از قلم‌ها. برای نمونه، در پیشنهادِ زمین‌ها، یک پیشنهاددهنده ممکن است یا قطعهٔ X یا قطعهٔ Y را بخواهد اما نه هر دو را؛ دیگری ممکن است هر سه قطعهٔ مجاور را بخواهد، و همین‌طور. سازوکارِ VCG را می‌توان برای یافتنِ پیامدِ بهینه استفاده کرد، هرچند با 2^N زیرمجموعه از N کالا برای سر و کله‌زدن، محاسبهٔ پیامدِ بهینه NP-کامل است. با چند ملاحظه، سازوکارِ VCG یکتاست: هر سازوکارِ بهینهٔ دیگری اساساً معادل است.

۱۸.۴.۳ رأی‌گیری

ردهٔ بعدی سازوکارهایی که به آن‌ها نگاه می‌کنیم رویه‌های رأی‌گیری هستند، از نوعی که برای تصمیم‌گیریِ سیاسی در جوامعِ دموکراتیک استفاده می‌شود. مطالعهٔ رویه‌های رأی‌گیری از حوزهٔ «نظریهٔ انتخابِ اجتماعی» (Social Choice Theory) مشتق می‌شود.

محیطِ پایه به‌صورتِ زیر است. مثلِ همیشه، مجموعه‌ای $N = \{1,...,n\}$ از عامل‌ها داریم، که در این بخش «رأی‌دهندگان» (Voters) خواهند بود. این رأی‌دهندگان می‌خواهند دربارهٔ مجموعه‌ای $\Omega = \{\omega1,\omega2,...\}$ از پیامدهای ممکن تصمیم بگیرند. در انتخاباتِ سیاسی، هر عنصرِ Ω می‌تواند به‌جای برنده‌شدنِ نامزدِ متفاوتی بایستد.

هر رأی‌دهنده ترجیح‌هایی روی Ω خواهد داشت. این‌ها معمولاً نه به‌صورتِ مطلوبیت‌های کمّی، بلکه به‌صورتِ مقایسه‌های کیفی بیان می‌شوند: $\omega \succ_i \omega'$ را می‌نویسیم تا بدان معنا باشد پیامدِ ω توسط عاملِ i بالاتر از پیامدِ ω' رتبه‌بندی می‌شود. در انتخاباتی با سه نامزد، عاملِ i ممکن است $\omega1 \succ_i \omega3 \succ_i \omega2$ داشته باشد. مسئلهٔ بنیادینِ نظریهٔ انتخابِ اجتماعی ترکیبِ این ترجیح‌ها، با استفاده از «تابعِ رفاهِ اجتماعی» (Social Welfare Function)، برای رسیدن به «ترتیبِ ترجیحِ اجتماعی» است: رتبه‌بندیِ نامزدها، از بیشترین ترجیح تا کمترین. در برخی موارد، فقط به «پیامدِ اجتماعیِ» (Social Outcome)—پیامدِ بیشترین‌ترجیح توسط گروه در کل—علاقه‌مندیم. $\omega \succ^* \omega'$ را می‌نویسیم تا بدان معنا باشد ω در ترتیبِ ترجیحِ اجتماعی بالاتر از ω' رتبه‌بندی شده.

محیطِ ساده‌تر جایی است که به دست‌آوردنِ کلِ ترتیبِ نامزدها اهمیت نمی‌دهیم، بلکه صرفاً می‌خواهیم مجموعه‌ای از برندگان انتخاب کنیم. «تابعِ انتخابِ اجتماعی» (Social Choice Function) ترتیبِ ترجیحِ هر رأی‌دهنده را به‌عنوانِ ورودی می‌گیرد و مجموعه‌ای از برندگان را به‌عنوانِ خروجی تولید می‌کند.

جوامعِ دموکراتیک پیامدِ اجتماعیِ منعکس‌کنندهٔ ترجیح‌های رأی‌دهندگان را می‌خواهند. متأسفانه، این همیشه سرراست نیست. «پارادوکسِ کندورسه» (Condorcet's Paradox) را در نظر بگیرید، مثالِ مشهوری که مارکی دو کندورسه (۱۷۴۳–۱۷۹۴) مطرح کرد. فرض کنیم سه پیامد، $\Omega = \{\omega a,\omega b,\omega c\}$، و سه رأی‌دهنده، $N = \{1,2,3\}$، با ترجیح‌های زیر داریم:

$$\begin{aligned} &\omega a \succ_1 \omega b \succ_1 \omega c \\ &\omega c \succ_2 \omega a \succ_2 \omega b \\ &\omega b \succ_3 \omega c \succ_3 \omega a \end{aligned} \qquad (18.2)$$

حالا، فرض کنیم باید یکی از سه نامزد را بر اساسِ این ترجیح‌ها انتخاب کنیم. پارادوکس این است که: ۲/۳ رأی‌دهندگان ωa را بر ωb ترجیح می‌دهند؛ ۲/۳ رأی‌دهندگان ωb را بر ωc ترجیح می‌دهند؛ و ۲/۳ رأی‌دهندگان ωc را بر ωa ترجیح می‌دهند. پس، برای هر برندهٔ ممکن، می‌توانیم به نامزدِ دیگری اشاره کنیم که دستِ‌کم ۲/۳ رأی‌دهندگان آن را ترجیح می‌دهند. بدیهی است در دموکراسی نمی‌توانیم امیدوار باشیم هر رأی‌دهنده خوشحال شود. این نشان می‌دهد سناریوهایی هست که در آن‌ها مهم نیست کدام پیامد را انتخاب کنیم، اکثریتِ رأی‌دهندگان پیامدِ متفاوتی را ترجیح می‌دهند.

پرسشِ طبیعی این است که آیا رویهٔ انتخابِ اجتماعیِ «خوبی» هست که واقعاً ترجیح‌های رأی‌دهندگان را بازتاب دهد. برای پاسخ به این، باید دقیق باشیم دربارهٔ اینکه وقتی می‌گوییم قاعده‌ای «خوب» است چه می‌گوییم. برخی ویژگی‌هایی را فهرست می‌کنیم که می‌خواهیم تابعِ رفاهِ اجتماعیِ خوبی ارضا کند:

- **شرطِ پارتو (The Pareto Condition):** شرطِ پارتو صرفاً می‌گوید اگر هر رأی‌دهنده ωi را بالاتر از ωj رتبه‌بندی کند، آن‌گاه ωj ≻* ωi.
- **شرطِ برندهٔ کندورسه (The Condorcet Winner Condition):** به پیامدی «برندهٔ کندورسه» گفته می‌شود اگر اکثریتی از رأی‌دهندگان آن را بر همهٔ پیامدهای دیگر ترجیح دهند. به بیانِ دیگر، برندهٔ کندورسه نامزدی است که در انتخاباتِ دوتایی هر نامزدِ دیگری را می‌برد. شرطِ برندهٔ کندورسه می‌گوید اگر ωi برندهٔ کندورسه باشد، آن‌گاه ωi باید اول رتبه‌بندی شود.
- **استقلال از گزینه‌های نامربوط (Independence of Irrelevant Alternatives یا IIA):** فرض کنیم شماری نامزد، از جملهِ ωi و ωj، هست، و ترجیح‌های رأی‌دهندگان چنان‌اند که ωj ≻* ωi. حالا، فرض کنیم رأی‌دهنده‌ای ترجیح‌هایش را به نحوی تغییر دهد، اما نه دربارهٔ رتبه‌بندیِ نسبیِ ωi و ωj. شرطِ IIA می‌گوید ωj ≻* ωi نباید تغییر کند.
- **بدونِ دیکتاتوری (No Dictatorships):** نباید چنین باشد که تابعِ رفاهِ اجتماعی صرفاً ترجیح‌های یک رأی‌دهنده را خروجی دهد و همهٔ رأی‌دهندگانِ دیگر را نادیده بگیرد.

این چهار شرط معقول به نظر می‌رسند، اما قضیهٔ بنیادینِ نظریهٔ انتخابِ اجتماعی به نامِ «قضیهٔ ارو» (Arrow's Theorem) (از کنت ارو) به ما می‌گوید ارضایِ هر چهار شرط ناممکن است (برای مواردی که دستِ‌کم سه پیامد هست). این بدان معناست برای هر سازوکارِ انتخابِ اجتماعی که ممکن است بخواهیم برگزینیم، موقعیت‌هایی (شاید غیرعادی یا آسیب‌شناختی) خواهد بود که به پیامدهای بحث‌برانگیز می‌انجامند. اما بدان معنا نیست تصمیم‌گیریِ دموکراتیک در بیشترِ موارد ناامیدکننده است. هنوز هیچ رویهٔ رأی‌گیریِ واقعی ندیده‌ایم، پس حالا به برخی نگاه کنیم.

- با فقط دو نامزد، «رأیِ اکثریتِ ساده» (Simple Majority Vote) (روشِ استاندارد در آمریکا و بریتانیا) سازوکارِ موردعلاقه است. از هر رأی‌دهنده می‌پرسیم کدام‌یک از دو نامزد را ترجیح می‌دهد، و نامزدی که بیشترین رأی را داشته باشد برنده است.
- با بیش از دو پیامد، «رأیِ کثرت» (Plurality Voting) سامانهٔ رایجی است. از هر رأی‌دهنده انتخابِ اولش را می‌پرسیم و نامزد(هایی) را انتخاب می‌کنیم (بیش از یکی در صورتِ تساوی) که بیشترین رأی را بگیرند، حتی اگر هیچ‌کس اکثریت نگیرد. در حالی که رایج است، رأیِ کثرت به دلیلِ تحویلِ پیامدهای نامحبوب موردِ انتقاد بوده. مشکلِ کلیدی این است که فقط نامزدِ بالارتبه را در ترجیحِ هر رأی‌دهنده در نظر می‌گیرد.

- «شمارشِ بوردا» (Borda Count) (پس از ژان-شارل دو بوردا، هم‌عصر و رقیبِ کندورسه) رویهٔ رأی‌گیری‌ای است که همهٔ اطلاعاتِ ترتیبِ ترجیحِ رأی‌دهنده را در نظر می‌گیرد. فرض کنیم k نامزد داریم. سپس برای هر رأی‌دهندهٔ i، ترتیبِ ترجیحِ $\succ_i$ را می‌گیریم و به نامزدِ بالارتبه امتیازِ k، به نامزدِ دوم امتیازِ k-1، و همین‌طور تا کم‌محبوب‌ترین نامزد در ترتیبِ $\succ_i$ می‌دهیم. امتیازِ کلِ هر نامزد «شمارشِ بورداییِ» اوست، و برای به‌دست‌آوردنِ پیامدِ اجتماعیِ $\succ^*$، پیامدها با شمارشِ بوردایشان مرتب می‌شوند—بالاترین تا پایین‌ترین. مشکلِ عملیِ این سامانه این است که از رأی‌دهندگان می‌خواهد ترجیح‌هایشان را دربارهٔ همهٔ نامزدها بیان کنند، و برخی رأی‌دهندگان ممکن است فقط به زیرمجموعه‌ای از نامزدها اهمیت دهند.
- در «رأیِ تأییدی» (Approval Voting)، رأی‌دهندگان زیرمجموعه‌ای از نامزدهایی را که تأیید می‌کنند ارائه می‌کنند. برنده(ها) کسانی‌اند که توسط بیشترین رأی‌دهندگان تأیید شده‌اند. این سامانه اغلب وقتی استفاده می‌شود که تکلیف انتخابِ چند برنده است.
- در «رأیِ دومرحله‌ایِ فوری» (Instant Runoff Voting)، رأی‌دهندگان همهٔ نامزدها را رتبه‌بندی می‌کنند، و اگر نامزدی اکثریتِ آرایِ اول داشته باشد، برنده اعلام می‌شود. اگر نه، نامزدی با کمترین آرایِ اول حذف می‌شود. آن نامزد از همهٔ رتبه‌بندی‌های ترجیح حذف می‌شود (پس آن رأی‌دهندگانی که نامزدِ حذف‌شده را انتخابِ اول داشتند، حالا نامزدِ دیگری را به‌عنوانِ انتخابِ اولِ تازه دارند) و فرایند تکرار می‌شود. سرانجام، نامزدی اکثریتِ آرایِ اول خواهد داشت (مگر اینکه تساوی باشد).
- در «رأیِ قاعدهٔ اکثریتِ واقعی» (True Majority Rule Voting)، برنده نامزدی است که در مقایسه‌های دوتایی هر نامزدِ دیگری را می‌برد. از رأی‌دهندگان رتبه‌بندیِ ترجیحیِ کاملِ همهٔ نامزدها خواسته می‌شود. می‌گوییم ω بر ω' می‌برد، اگر رأی‌دهندگانِ بیشتری $\omega \succ \omega'$ داشته باشند تا $\omega' \succ \omega$. این سامانه این ویژگیِ خوب را دارد که اگر اکثریت همیشه بر سرِ یک برنده توافق داشته باشند برگزیده می‌شود، اما این ویژگیِ نامطلوب را نیز دارد که لزوماً هر انتخاباتی برنده نخواهد داشت (ممکن است بلاتکلیف بماند): در پارادوکسِ کندورسه، برای نمونه، هیچ گزینه‌ای اکثریت نمی‌آورد.

دستکاریِ استراتژیک. علاوه بر قضیهٔ ارو، نتیجهٔ منفیِ مهمِ دیگری در حوزهٔ نظریهٔ انتخابِ اجتماعی هست: «قضیهٔ گیبارد–ساتروایت» (Gibbard–Satterthwaite Theorem). این نتیجه به شرایطی مربوط است که رأی‌دهنده می‌تواند از نادرست‌بازنماییِ ترجیح‌هایش سود ببرد. به یاد آورید تابعِ انتخابِ اجتماعی ترتیبِ ترجیحِ هر رأی‌دهنده را به‌عنوانِ ورودی می‌گیرد و به‌عنوانِ خروجی مجموعه‌ای از نامزدهای برنده می‌دهد. هر رأی‌دهنده، البته، ترجیحِ واقعیِ خودش را دارد، اما هیچ‌چیز در تعریفِ تابعِ انتخابِ اجتماعی نیست که رأی‌دهندگان را به گزارشِ صادقانهٔ ترجیح‌هایشان ملزم کند؛ آن‌ها می‌توانند هر ترجیحی که دوست دارند اعلام کنند.

در برخی موارد، می‌تواند برای رأی‌دهنده معنا داشته باشد ترجیح‌هایش را نادرست بازنمایی کند. برای نمونه، در رأیِ کثرت، رأی‌دهندگانی که فکر می‌کنند نامزدِ موردعلاقه‌شان شانسی برای بردن ندارد ممکن است در عوض به انتخابِ دومشان رأی دهند. این بدان معناست رأیِ کثرت بازی‌ای است که در آن رأی‌دهندگان باید استراتژیک فکر کنند (دربارهٔ رأی‌دهندگانِ دیگر) تا مطلوبیتِ موردانتظارشان را بیشینه کنند.

این پرسشِ جالبی را برمی‌انگیزد: آیا می‌توانیم سازوکارِ رأی‌گیری‌ای طراحی کنیم که در برابرِ چنین دستکاری‌ای ایمن باشد —سازوکاری که حقیقت‌فاش باشد؟ قضیهٔ گیبارد–ساتروایت به ما می‌گوید نمی‌توانیم: هر تابعِ انتخابِ اجتماعیِ ارضاکنندهٔ شرطِ پارتو برای حوزه‌ای با بیش از دو پیامد، یا دستکاری‌پذیر است یا دیکتاتوری. یعنی، برای هر رویهٔ انتخابِ اجتماعیِ «معقولی»، شرایطی خواهد بود که در آن رأی‌دهنده اصولاً می‌تواند با نادرست‌بازنماییِ ترجیح‌هایش سود ببرد. با این حال، به ما نمی‌گوید چنین دستکاری‌ای چگونه ممکن است انجام شود؛ و به ما نمی‌گوید چنین دستکاری‌ای در عمل محتمل است.

۱۸.۴.۴ چانه‌زنی

«چانه‌زنی» (Bargaining)، یا مذاکره، سازوکارِ دیگری است که مکرراً در زندگیِ روزمره استفاده می‌شود. از دههٔ ۱۹۵۰ در نظریهٔ بازی مطالعه شده و اخیراً به تکلیفی برای عامل‌های خودکار تبدیل شده. چانه‌زنی وقتی استفاده می‌شود که عامل‌ها باید در موردِ موضوعی با علاقهٔ مشترک به توافق برسند. عامل‌ها تحتِ پروتکل‌های خاص به یکدیگر «پیشنهادهایی» (Offers) می‌دهند (که «پیشنهادها» (Proposals) یا «معاملات» (Deals) نیز نامیده می‌شوند)، و هر پیشنهاد را می‌پذیرند یا رد می‌کنند.

چانه‌زنی با پروتکلِ پیشنهادهای متناوب. یکی از پروتکل‌های چانه‌زنیِ تأثیرگذار «مدلِ چانه‌زنیِ پیشنهادهای متناوب» (Alternating Offers Bargaining Model) است. برای سادگی، باز فقط دو عامل را فرض می‌کنیم. چانه‌زنی در توالیِ دورها رخ می‌دهد. A1 در دورِ 0 با پیشنهاد شروع می‌کند. اگر A2 پیشنهاد را بپذیرد، پیشنهاد اجرا می‌شود. اگر A2 پیشنهاد را رد کند، مذاکره به دورِ بعد می‌رود. این بار A2 پیشنهاد می‌دهد و A1 انتخاب می‌کند بپذیرد یا رد کند، و همین‌طور. اگر مذاکره هرگز خاتمه نیابد (زیرا عامل‌ها هر پیشنهادی را رد می‌کنند)، پیامد را «معاملهٔ تعارض» (Conflict Deal) تعریف می‌کنیم. فرضِ ساده‌کنندهٔ مناسبی این است که هر دو عامل ترجیح می‌دهند در زمانِ متناهی به پیامدی —هر پیامدی—برسند تا اینکه در معاملهٔ تعارضِ بی‌نهایت‌زمان‌گیر گیر کنند.

نخست، حالتِ ساده‌ای را در نظر بگیریم که در آن فقط یک دورِ مذاکره مجاز است. این بازیِ «اولتیماتوم» (Ultimatum Game) نامیده می‌شود. A1 پیشنهادی ارائه می‌کند؛ اگر A2 آن را بپذیرد، توافق محقق می‌شود؛ اگر A2 آن را رد کند، مذاکره به معاملهٔ تعارض می‌انجامد. در این بازی، A2 هیچ انگیزه‌ای برای ردِ هیچ پیشنهادی ندارد: ردکردن به معنای معاملهٔ تعارض است، که طبقِ فرض، برای A2 بدترین پیامد است. بدین‌ترتیب، A1 می‌تواند بهترین پیامدِ ممکن را برای خودش بگیرد. اگر پایِ تقسیم‌شده بینِ دو عامل «پای» (Pie) تکی باشد، A1 می‌تواند پیشنهادِ (1,0) را ارائه دهد—همهٔ پای برای A1، هیچ برای A2—و A2 آن را می‌پذیرد، زیرا هیچ جایگزینِ بهتری ندارد. (اگر نگرانِ تساوی‌ها هستید، A1 می‌تواند پیشنهادِ $(1-\varepsilon, \varepsilon)$ را برای ε کوچکی ارائه دهد.) این تعادلِ نش است: هیچ بازیکنی با انحراف بهتر عمل نمی‌کند.

حالا به حالتِ عمومی برویم، جایی که هیچ کرانی بر شمارِ دورها نیست. فرض کنیم A1 از استراتژیِ زیر استفاده کند: همیشه (1,0) پیشنهاد کن، و هر پیشنهادِ متقابلی را همیشه رد کن. بهترین پاسخِ A2 به این چیست؟ اگر A2 پیوسته پیشنهاد را رد کند، عامل‌ها برای همیشه مذاکره خواهند کرد، که طبقِ تعریف بدترین پیامد برای A2 است (و نیز برای A1). پس A2 نمی‌تواند بهتر از پذیرشِ نخستین پیشنهادِ A1 عمل کند. باز، این تعادلِ نش است. اما اگر A1 از

استراتژیِ زیر استفاده کند چه: همیشه (0.8,0.2) پیشنهاد کن، و هر پیشنهادی را همیشه رد کن؟ با استدلالِ مشابهی می‌بینیم برای این پیشنهاد یا برای هر معاملهٔ ممکنِ (x, 1-x) در مجموعهٔ مذاکره، جفتِ تعادلِ نشی از استراتژی‌های مذاکره هست به‌گونه‌ای که پیامد، توافق بر معامله در نخستین دورهٔ زمانی خواهد بود.

عامل‌های بی‌صبر. این تحلیل به ما می‌گوید اگر هیچ قیدی بر شمارِ دورها نباشد، بی‌نهایت تعادلِ نش خواهد بود. پس فرضی می‌افزاییم: برای هر پیامدِ x و زمان‌های t1 و t2، که t1 < t2، هر دو عامل پیامدِ x در زمانِ t1 را بر پیامدِ x در زمانِ t2 ترجیح می‌دهند. به بیانِ دیگر، عامل‌ها بی‌صبرند. رویکردِ استاندارد به بی‌صبری استفاده از عاملِ تنزیلِ γi (صفحهٔ ۵۶۵ را ببینید) برای هر عامل است (0 ≤ γi < 1). فرض کنیم در نقطه‌ای از مذاکره به عاملِ i برشی از پای با اندازهٔ x پیشنهاد شود. ارزشِ برشِ x در زمانِ t برابر $\gamma_i^t x$ است. بدین‌ترتیب، در نخستین گامِ مذاکره (زمانِ 0)، ارزش $\gamma_i^0 x$ = x است، و در هر نقطهٔ زمانیِ بعدی ارزشِ همان پیشنهاد کمتر خواهد بود. مقدارِ بزرگ‌ترِ γi (نزدیک‌تر به 1) بدین‌ترتیب دلالت بر صبرِ بیشتر دارد؛ مقدارِ کوچک‌تر یعنی صبرِ کمتر.

برای تحلیلِ حالتِ عمومی، نخست چانه‌زنی بر دوره‌های زمانیِ ثابت را در نظر بگیریم، چنان‌که در بالا. حالتِ ۱-دوره همان تحلیلِ داده‌شده در بالا را دارد: صرفاً بازیِ اولتیماتوم داریم. با دو دور وضعیت تغییر می‌کند، زیرا ارزشِ پای مطابقِ عامل‌های تنزیلِ γi کاهش می‌یابد. فرض کنیم A2 پیشنهادِ اولیهٔ A1 را رد کند. سپس A2 کلِ پای را با اولتیماتوم در دورِ دوم می‌گیرد. اما ارزشِ آن کلِ پای کاهش یافته: برای A2 فقط γ2 می‌ارزد. عاملِ A1 می‌تواند این واقعیت را با پیشنهادِ (γ2, γ2-1) در نظر بگیرد، پیشنهادی که A2 ممکن است خوب بپذیرد، زیرا A2 در این نقطه از زمان نمی‌تواند بهتر از γ2 عمل کند. (اگر نگرانِ تساوی‌ها هستید، فقط پیشنهاد را (γ2+ϱ, 1-(γ2+ϱ)) برای ϱ کوچکی کنید.) پس، دو استراتژیِ پیشنهادِ (γ2, γ2-1) توسط A1 و پذیرشِ آن پیشنهاد توسط A2 در تعادلِ نش‌اند. بازیکنانِ صبور (آن‌هایی با γ2 بزرگ‌تر) تحتِ این پروتکل قادر خواهند بود برش‌های بزرگ‌تری از پای بگیرند: در این محیط، صبر واقعاً فضیلت است.

حالا حالتِ عمومی را در نظر بگیریم، جایی که هیچ کرانی بر شمارِ دورها نیست. مانندِ حالتِ ۱-دوره، A1 می‌تواند پیشنهادی بسازد که A2 باید بپذیرد، زیرا با توجه به عامل‌های تنزیل، حداکثرِ مقدارِ قابلِ‌دستیابی را به A2 می‌دهد. معلوم می‌شود A1 خواهد گرفت:

```
(1 – γ1γ2) / (γ2 – 1)
```

و A2 بقیه را.

مذاکره در حوزه‌های وظیفه‌محور. در این بخش، مذاکره برای «حوزه‌های وظیفه‌محور» (Task-Oriented Domains) را در نظر می‌گیریم. در چنین حوزه‌ای، مجموعه‌ای از وظایف باید انجام شود، و هر وظیفه در آغاز به مجموعه‌ای از عامل‌ها تخصیص داده می‌شود. عامل‌ها ممکن است با مذاکره بر سر اینکه چه کسی کدام وظایف را انجام دهد سود ببرند. برای نمونه، فرض کنیم برخی وظایف روی ماشینِ تراش و برخی دیگر روی ماشینِ فرز انجام می‌شوند، و هر عاملی که از ماشینی استفاده می‌کند باید هزینهٔ راه‌اندازیِ قابلِ‌توجهی متحمل شود. آن‌گاه منطقی بود عاملی به دیگری پیشنهاد

دهد «من به هر حال باید ماشینِ فرز را راه بیندازم؛ چه می‌شود همهٔ وظایفِ فرزِ تو را من انجام دهم، و تو همهٔ وظایفِ تراشِ من را؟» برخلافِ سناریویِ چانه‌زنی، با تخصیصِ اولیه شروع می‌کنیم، پس اگر عامل‌ها نتوانند بر هیچ پیشنهادی توافق کنند، وظایفِ Ti^0 را که در آغاز به آن‌ها تخصیص داده شده انجام می‌دهند.

برای ساده‌نگه‌داشتنِ امور، باز فقط دو عامل را فرض می‌کنیم. T مجموعهٔ همهٔ وظایف و ($T1^0$, $T2^0$) تخصیصِ اولیهٔ وظایف به دو عامل در زمانِ 0 باشد. هر وظیفه در T باید دقیقاً به یک عامل تخصیص یابد. فرض می‌کنیم تابعِ هزینهٔ c داریم، که برای هر مجموعه‌ای از وظایفِ 'T عددِ حقیقیِ مثبتی c('T) می‌دهد که هزینهٔ انجامِ وظایفِ 'T توسط هر عاملی را نشان می‌دهد. (فرض کنید هزینه فقط به وظایف بستگی دارد، نه به عاملی که وظیفه را انجام می‌دهد.) تابعِ هزینه یکنواخت است—افزودنِ وظایفِ بیشتر هرگز هزینه را کاهش نمی‌دهد—و هزینهٔ هیچ‌کاری‌نکردن صفر است: c({}) = 0. به‌عنوانِ نمونه، فرض کنیم هزینهٔ راه‌اندازیِ ماشینِ فرز 10 و هر وظیفهٔ فرز 1 است، پس هزینهٔ مجموعه‌ای از دو وظیفهٔ فرز 12 و هزینهٔ مجموعه‌ای از پنج‌تایی 15 خواهد بود.

پیشنهادی به شکلِ (T1,T2) بدان معناست عاملِ i متعهد به انجامِ مجموعهٔ وظایفِ Ti، با هزینهٔ c(Ti) است. مطلوبیتِ عاملِ i مقدارِ سودی است که از پذیرشِ پیشنهاد به دست می‌آورد—تفاوتِ میانِ هزینهٔ انجامِ این مجموعهٔ تازهٔ وظایف در برابرِ مجموعهٔ تخصیص‌یافتهٔ اولیه:

```
Ui((T1,T2)) = c(Tiº) − c(Ti)
```

پیشنهادِ (T1,T2) «فرداً عقلایی» (Individually Rational) است اگر Ui((T1,T2)) ≥ 0 برای هر دو عامل. اگر معامله‌ای فرداً عقلایی نباشد، دست‌کم یک عامل می‌تواند صرفاً با انجامِ وظایفی که در آغاز به او تخصیص داده شده بهتر عمل کند. «مجموعهٔ مذاکره» برای حوزه‌های وظیفه‌محور (با فرضِ عامل‌های عقلایی) مجموعهٔ پیشنهادهایی است که هم فرداً عقلایی و هم پارتو-بهینه‌اند. هیچ معنایی در ارائهٔ پیشنهادِ فرداً غیرعقلاییِ ردشدنی نیست، و نه در ارائهٔ پیشنهادی وقتی پیشنهادِ بهتری هست که مطلوبیتِ عاملی را بدونِ آسیب‌زدن به دیگران بهبود می‌بخشد.

پروتکلِ امتیازِ یکنواخت. پروتکلِ مذاکره‌ای که برای حوزه‌های وظیفه‌محور در نظر می‌گیریم «پروتکلِ امتیازِ یکنواخت» (Monotonic Concession Protocol) شناخته می‌شود. قاعده‌های این پروتکل به‌صورتِ زیرند:

- مذاکره در توالیِ دورها پیش می‌رود.
- در دورِ نخست، هر دو عامل هم‌زمان معامله‌ای، Di = (T1,T2)، از مجموعهٔ مذاکره پیشنهاد می‌کنند. (این با پیشنهادهای متناوبِ قبلی متفاوت است.)
- اگر دو عامل به‌ترتیب معامله‌های D1 و D2 را پیشنهاد کنند به‌گونه‌ای که یا (i) U1(D2) ≥ U1(D1) یا (ii) U2(D1) ≥ U2(D2)—یعنی یکی از عامل‌ها بیابد معاملهٔ پیشنهادشده توسط دیگری دستِ‌کم به‌اندازهٔ پیشنهادِ خودش خوب یا بهتر است—توافق حاصل می‌شود. اگر توافق حاصل شود، قاعدهٔ تعیینِ معاملهٔ توافقی به‌صورتِ زیر است: اگر پیشنهادِ هر عامل با پیشنهادِ عاملِ دیگر برابری کند یا از آن فراتر رود، یکی از پیشنهادها به‌صورتِ تصادفی انتخاب می‌شود. اگر فقط یک پیشنهاد از پیشنهادِ دیگری فراتر رود یا با آن برابری کند، این معاملهٔ توافقی است.

- اگر توافقی حاصل نشود، مذاکره به دورِ دیگری از پیشنهادهای هم‌زمان می‌رود. در دورِ t+1، هر عامل باید یا پیشنهادِ دورِ قبلی را تکرار کند یا «امتیازی» (Concession) بدهد—پیشنهادی که توسط عاملِ دیگر بیشتر ترجیح داده می‌شود (یعنی مطلوبیتِ بالاتری دارد).
- اگر هیچ عاملی امتیاز ندهد، مذاکره خاتمه می‌یابد و هر دو عامل معاملهٔ تعارض را پیاده‌سازی می‌کنند، و وظایفی را که در آغاز به آن‌ها تخصیص داده شده انجام می‌دهند.

چون مجموعهٔ معامله‌های ممکن متناهی است، عامل‌ها نمی‌توانند به‌صورتِ نامحدود مذاکره کنند: یا عامل‌ها به توافق می‌رسند، یا دوری رخ می‌دهد که در آن هیچ‌کس امتیاز نمی‌دهد. با این حال، پروتکل تضمین نمی‌کند توافق سریعاً حاصل شود: چون شمارِ معامله‌های ممکن O(2^|T|) است، قابلِ تصور است مذاکره برای شماری از دورها ادامه یابد که در شمارِ وظایفِ موردِ تخصیص نمایی است.

استراتژیِ زوتن. تاکنون هیچ‌چیز دربارهٔ اینکه شرکت‌کنندگانِ مذاکره هنگام استفاده از پروتکلِ امتیازِ یکنواخت برای حوزه‌های وظیفه‌محور ممکن است یا باید چگونه رفتار کنند نگفته‌ایم. یکی از استراتژی‌های ممکن «استراتژیِ زوتن» (Zeuthen Strategy) است. ایدهٔ استراتژیِ زوتن اندازه‌گیریِ «تمایلِ عامل به ریسکِ تعارض» است. به‌صورتِ شهودی، عامل اگر تفاوتِ مطلوبیتِ میانِ پیشنهادِ کنونی‌اش و معاملهٔ تعارض کم باشد، مایل‌تر به ریسکِ تعارض خواهد بود. در این صورت، عامل اگر مذاکره شکست بخورد و معاملهٔ تعارض پیاده‌سازی شود چیزِ کمی برای ازدست‌دادن دارد، و بنابراین مایل‌تر به ریسکِ تعارض، و کم‌مایل‌تر به امتیازدادن است. در مقابل، اگر تفاوتِ میانِ پیشنهادِ کنونیِ عامل و معاملهٔ تعارض زیاد باشد، عامل از تعارض چیزِ بیشتری برای ازدست‌دادن دارد و بنابراین کم‌مایل‌تر به ریسکِ تعارض—و بدین‌ترتیب مایل‌تر به امتیازدادن است.

تمایلِ عاملِ i به ریسکِ تعارض در دورِ t، که riskt_i نشان داده می‌شود، چنین اندازه‌گیری می‌شود:

```
riskt_i = (مطلوبیتی که i با امتیازدادن و پذیرشِ پیشنهادِ j از دست می‌دهد)
        / (مطلوبیتی که i با امتیازندادن و ایجادِ تعارض از دست می‌دهد)
```

تا وقتی توافقی حاصل نشود، مقدارِ riskt_i بینِ 0 و 1 خواهد بود. مقادیرِ بالاترِ riskt_i (نزدیک‌تر به 1) نشان می‌دهد i چیزِ کمتری برای ازدست‌دادن از تعارض دارد، و بنابراین مایل‌تر به ریسکِ تعارض است. استراتژیِ زوتن می‌گوید نخستین پیشنهادِ هر عامل باید معامله‌ای در مجموعهٔ مذاکره باشد که مطلوبیتِ خودش را بیشینه می‌کند (ممکن است بیش از یکی باشد). پس از آن، عاملی که باید در دورِ t مذاکره امتیاز دهد باید عاملی با مقدارِ کوچک‌ترِ ریسک باشد—عاملی که بیشترین چیز را برای ازدست‌دادن از تعارض دارد، اگر هیچ‌کس امتیاز ندهد.

پرسشِ بعدیِ پاسخ‌دادن این است که چقدر باید امتیاز داد؟ پاسخِ ارائه‌شده توسط استراتژیِ زوتن این است: «فقط به‌اندازه‌ای که توازنِ ریسک را به عاملِ دیگر منتقل کند.» یعنی عامل باید کوچک‌ترین امتیازی را بدهد که عاملِ دیگر را در دورِ بعد امتیازدهنده کند. یک پالایشِ نهایی برای استراتژیِ زوتن هست. فرض کنیم در نقطه‌ای هر دو عامل ریسکِ برابر دارند. سپس، طبقِ استراتژی، هر دو باید امتیاز بدهند. اما، با دانستنِ این، یک عامل به‌طورِ بالقوه می‌توانست

«انحراف» کند با امتیازندادن، و بنابراین سود ببرد. برای اجتناب از امکانِ امتیازدادنِ هر دو در این نقطه، استراتژی را با «سکه انداختن» توسط عامل‌ها برای تصمیم‌گیری دربارهٔ اینکه چه کسی امتیاز دهد، اگر هرگز وضعیتِ ریسکِ برابر حاصل شد، بسط می‌دهیم.

با این استراتژی، توافق پارتو-بهینه و فرداً عقلایی خواهد بود. با این حال، چون فضایِ معامله‌های ممکن در شمارِ وظایف نمایی است، پیروی از این استراتژی ممکن است $O(2^{|T|})$ محاسبهٔ تابعِ هزینه در هر گامِ مذاکره لازم داشته باشد. سرانجام، استراتژیِ زوتن (با قاعدهٔ سکه‌انداختن) در تعادلِ نش است.

خلاصهٔ فصل

- برنامه‌ریزیِ چندعاملی وقتی لازم است که عامل‌های دیگری در محیط با آن‌ها برای همکاری یا رقابت باشند. برنامه‌های مشترک را می‌توان ساخت، اما باید با شکلی از هماهنگی تقویت شوند اگر دو عامل قرار است بر اجرایِ کدام برنامهٔ مشترک توافق کنند.
- نظریهٔ بازی رفتارِ عقلایی را برای عامل‌ها در موقعیت‌هایی توصیف می‌کند که چند عامل تعامل می‌کنند. نظریهٔ بازی برای تصمیم‌گیریِ چندعاملی همان‌گونه است که نظریهٔ تصمیم برای تصمیم‌گیریِ تک‌عاملی.
- مفاهیمِ راه‌حل در نظریهٔ بازی برای مشخصه‌سازیِ پیامدهای عقلاییِ بازی هستند—پیامدهایی که ممکن است رخ دهند اگر هر عامل عقلایی عمل کند.
- نظریهٔ بازیِ غیرهمکارانه فرض می‌کند عامل‌ها باید تصمیم‌هایشان را مستقلانه بگیرند. تعادلِ نش مهم‌ترین مفهومِ راه‌حل در نظریهٔ بازیِ غیرهمکارانه است. تعادلِ نش نمایهٔ استراتژی‌ای است که در آن هیچ عاملی انگیزه‌ای برای انحراف از استراتژیِ مشخص‌شده‌اش ندارد. فنونِ بازی‌های تکراری و بازی‌های ترتیبی را داریم.
- نظریهٔ بازیِ همکارانه محیط‌هایی را در نظر می‌گیرد که عامل‌ها می‌توانند توافق‌های الزام‌آوری برای تشکیلِ ائتلاف‌ها به‌منظورِ همکاری بکنند. مفاهیمِ راه‌حل در بازیِ همکارانه تلاش می‌کنند صورت‌بندی کنند کدام ائتلاف‌ها پایدارند (هسته) و چگونه ارزشِ به‌دست‌آمده توسط ائتلاف را به‌طورِ منصفانه تقسیم کنیم (ارزشِ شپلی).
- فنونِ تخصصی برای رده‌های مهمِ خاصی از تصمیمِ چندعاملی موجود است: شبکهٔ قرارداد برای به‌اشتراک‌گذاریِ وظایف؛ حراج‌ها برای تخصیصِ کارآمدِ منابعِ کمیاب؛ چانه‌زنی برای رسیدن به توافق‌ها در موردِ موضوعاتِ با علاقهٔ مشترک؛ و رویه‌های رأی‌گیری برای تجمیعِ ترجیح‌ها.

یادداشت‌های کتاب‌شناختی و تاریخی

کنجکاویِ حوزه این است که پژوهشگرانِ هوش مصنوعی تا دهۀ ۱۹۸۰ به‌طورِ جدی مسائلِ پیرامونِ عامل‌های متعامل را در نظر نگرفتند—و حوزۀ سامانه‌های چندعاملی تا یک دهۀ بعد واقعاً به‌عنوانِ زیررشتۀ متمایزی از هوش مصنوعی تثبیت نشد. با این وجود، ایده‌هایی که به سامانه‌های چندعاملی اشاره می‌کردند در دهۀ ۱۹۷۰ حضور داشتند. برای نمونه، در نظریۀ بسیار تأثیرگذارِ «ذهنِ جامعه» (Society of Mind)، ماروین مینسکی (1986، 2007) پیشنهاد کرد ذهن‌های انسانی از مجموعه‌ای از عامل‌ها ساخته شده‌اند. داگ لِنات ایده‌های مشابهی در چارچوبی به نامِ BEINGS داشت (لنات، 1975). در دهۀ ۱۹۷۰، کارل هویت، بر پایۀ کارِ دکترایِ خود روی سامانۀ PLANNER، مدلی از محاسبه به‌عنوانِ عامل‌های متعامل به نامِ «مدلِ کنشگر» (Actor Model) پیشنهاد کرد، که به‌عنوانِ یکی از مدل‌های بنیادین در محاسبۀ هم‌زمان تثبیت شده (هویت، 1977؛ آغا، 1986).

پیش‌تاریخِ حوزۀ سامانه‌های چندعاملی به‌طورِ کامل در مجموعه‌ای از مقاله‌ها به نامِ «خواندنی‌هایی در هوش مصنوعیِ توزیع‌شده» (باند و گسر، 1988) مستند شده. مجموعه با بیانِ مفصلی از چالش‌های پژوهشیِ کلیدیِ سامانه‌های چندعاملی مقدمه‌دار می‌شود، که بیش از سی سال پس از نوشته‌شدنش به‌طرزِ قابلِ‌توجهی امروزی باقی مانده. پژوهشِ اولیۀ سامانه‌های چندعاملی تمایل داشت فرض کند همۀ عامل‌های سامانه با علاقۀ مشترک، با طراحِ تکی، عمل می‌کنند. این اکنون به‌عنوانِ حالتِ خاصی از محیطِ عمومی‌ترِ چندعاملی شناخته می‌شود—حالتِ خاص به‌عنوانِ «حلِ مسئلۀ همکارانۀ توزیع‌شده» (Cooperative Distributed Problem Solving) شناخته می‌شود. سامانۀ کلیدیِ این زمان «تخت آزمایشِ پایشِ خودرویِ توزیع‌شده» (DVMT) بود، که تحتِ نظارتِ ویکتور لسر در دانشگاه ماساچوست توسعه یافت (لسر و کورکیل، 1988). DVMT سناریویی را مدل می‌کرد که در آن مجموعه‌ای از عامل‌های حسگرِ صوتیِ پراکندۀ جغرافیایی برای ردیابیِ حرکتِ خودروها همکاری می‌کنند. دورۀ معاصرِ پژوهشِ سامانه‌های چندعاملی در اواخرِ دهۀ ۱۹۸۰ آغاز شد، وقتی به‌طورِ گسترده دریافته شد عامل‌های با ترجیح‌های متفاوت در هوش مصنوعی و جامعه هنجارند—از این نقطه به بعد، نظریۀ بازی به‌عنوانِ روش‌شناسیِ اصلیِ مطالعۀ چنین عامل‌هایی تثبیت شد. برنامه‌ریزیِ چندعاملی در سال‌های اخیر جهشِ محبوبیت داشته، هرچند تاریخچۀ بلندی دارد. کونولیگ (1982) برنامه‌ریزیِ چندعاملی را در منطقِ مرتبۀ اول صوری می‌کند، در حالی که پدنو (1986) تعمیمِ STRIPS را برای برنامه‌ریزیِ چندعاملی می‌دهد.

بنیانِ نظریۀ بازی در کارِ فون نویمان (1928) در نظریۀ بازی‌های استراتژیک است، که «قضیۀ مینیمکس» را اثبات کرد: در هر بازیِ جمعِ صفرِ دونفره با اطلاعاتِ کامل، استراتژیِ ترکیبی‌ای وجود دارد که بازدهیِ موردانتظارِ بازیکن را بیشینه می‌کند، مستقل از استراتژیِ حریف. در ۱۹۵۰، در سنِ ۲۱ سالگی، جان نش ایده‌هایش را دربارۀ تعادل‌ها در بازی‌های عمومی (غیرجمع‌صفر) منتشر کرد. تعریفِ او از راه‌حلِ تعادلی، هرچند در کارِ کورنو (1838) پیش‌بینی شده بود، به‌عنوانِ تعادلِ نش شناخته شد. پس از تأخیری طولانی به دلیلِ اسکیزوفرنی‌ای که از ۱۹۵۹ به بعد از آن رنج می‌برد، نش در ۱۹۹۴ (به‌همراهِ راینهارت زلتن و جان هارسانی) جایزۀ یادبودِ نوبل در اقتصاد را دریافت کرد. تعادلِ بیز–نش توسط هارسانی (1967) توصیف و توسط کادان و لارکی (1982) بحث شده.

برخی مسائلِ استفاده از نظریۀ بازی برای کنترلِ عامل توسط بینمور (1982) پوشش داده شده. اومان و براندنبورگر (1995) نشان می‌دهند بسته به دانشِ هر بازیکن چگونه می‌توان به تعادل‌های متفاوتی رسید.

دوراهیِ زندانی به‌عنوانِ تمرینِ کلاسی توسط آلبرت دبلیو. تاکر در ۱۹۵۰ اختراع شد (بر اساسِ مثالی از مریل فلود و ملوین درشر) و به‌طورِ گسترده توسط آکسلرود (1985) و پاوندستون (1993) پوشش داده شده. بازی‌های تکراری توسط لوس و رایفا (1957) معرفی شدند، و آبرو و روبینشتاین (1988) استفاده از ماشین‌های حالتِ متناهی برای بازی‌های تکراری را بحث می‌کنند—از نظرِ فنی، ماشین‌های مور. متنِ میلاث و ساموئلسون (2006) بر بازی‌های تکراری تمرکز دارد.

بازی‌های اطلاعاتِ جزئی در شکلِ گسترده توسط کان (1953) معرفی شدند. شکلِ توالی برای بازی‌های اطلاعاتِ جزئی توسط رومانوفسکی (1962) و به‌طورِ مستقل توسط کولر و همکاران (1996) اختراع شد؛ مقالهٔ کولر و ففر (1997) مقدمه‌ای خواندنی بر حوزه فراهم می‌کند و سامانه‌ای برای بازنمایی و حلِ بازی‌های ترتیبی توصیف می‌کند.

استفاده از انتزاع برای کاهشِ درختِ بازی به اندازه‌ای که با فنِ کولر قابلِ حل باشد توسط بیلینگز و همکاران (2003) معرفی شد. متعاقباً، روش‌های بهبودیافتهٔ تعادل‌یابی حلِ انتزاع‌ها را با 10^{12} حالت ممکن کردند (گیلپین و همکاران، 2008؛ زینکویچ و همکاران، 2008). بولینگ و همکاران (2008) نشان می‌دهند چگونه از نمونه‌برداریِ اهمیت برای برآوردِ بهترِ ارزشِ استراتژی استفاده کنیم. واو و همکاران (2009) دریافتند رویکردِ انتزاع در برابرِ خطاهای نظام‌مند در تقریبِ راه‌حلِ تعادلی آسیب‌پذیر است: برای برخی بازی‌ها کار می‌کند اما برای برخی دیگر نه. براون و سندلم (2019) نشان دادند، دست‌کم در موردِ پوکرِ تگزاس هولدمِ چندنفره، این آسیب‌پذیری‌ها را می‌توان با قدرتِ محاسباتیِ کافی غلبه کرد. آن‌ها از سرورِ ۶۴-هسته‌ای استفاده کردند که ۸ روز اجرا شد تا استراتژیِ پایه‌ای برای برنامهٔ Pluribus خود محاسبه کنند. با آن استراتژی توانستند قهرمانانِ انسانی را شکست دهند. نظریهٔ بازی و MDPها در نظریهٔ «بازی‌های مارکوف» ترکیب می‌شوند، که «بازی‌های تصادفی» نیز نامیده می‌شوند (لیتمن، 1994؛ هو و ولمن، 1998). شپلی (1953b) در واقع الگوریتمِ تکرارِ مقدار را مستقل از بلمن توصیف کرد، اما نتایجش به‌طورِ گسترده قدردانی نشد، شاید به این دلیل که در بافتِ بازی‌های مارکوف ارائه شده بودند. نظریهٔ بازیِ تکاملی (اسمیت، 1982؛ وایبول، 1995) به رانشِ استراتژی در طولِ زمان نگاه می‌کند: اگر استراتژیِ حریفت در حالِ تغییر است، چگونه باید واکنش نشان دهی؟

کتاب‌های درسیِ نظریهٔ بازی از دیدگاهِ اقتصادی شاملِ میرسن (1991)، فودنبرگ و تیرول (1991)، آزبورن (2004) و آزبورن و روبینشتاین (1994) می‌شوند. از دیدگاهِ هوش مصنوعی، نیسان و همکاران (2007) و لیتون-براون و شوهام (2008) را داریم. سندلم (1999) را برای مرورِ مفیدی از تصمیم‌گیریِ چندعاملی ببینید.

یادگیریِ تقویتیِ چندعاملی (Multiagent RL) از یادگیریِ تقویتیِ توزیع‌شده با حضورِ عامل‌هایی متمایز می‌شود که نمی‌توانند کنش‌هایشان را هماهنگ کنند (به‌جز با کنش‌های ارتباطیِ صریح) و ممکن است تابعِ مطلوبیتِ مشترکی نداشته باشند. بدین‌ترتیب، یادگیریِ تقویتیِ چندعاملی با مسائلِ بازی-نظریِ ترتیبی یا بازی‌های مارکوف، چنان‌که در فصل ۱۷ تعریف شد، سروکار دارد. آنچه مشکل ایجاد می‌کند این واقعیت است که، در حالی که عاملی یاد می‌گیرد سیاستِ حریفش را شکست دهد، حریف سیاستش را برای شکستِ عامل تغییر می‌دهد. بدین‌ترتیب، محیط غیرایستاست (صفحهٔ ۴۴۴ را ببینید). لیتمن (1994) هنگام معرفیِ نخستین الگوریتم‌های RL برای بازی‌های مارکوفِ جمعِ صفر به این دشواری اشاره کرد. هو و ولمن (2003) الگوریتمِ Q-learning را برای بازی‌های جمعِ عمومی ارائه می‌کنند که وقتی تعادلِ نش یکتاست همگرا می‌شود؛ وقتی تعادل‌های متعدد هست، مفهومِ همگرایی چندان آسان تعریف‌شدنی نیست (شوهام و همکاران، 2004).

بازی‌های کمکی تحتِ عنوانِ «یادگیریِ تقویتیِ معکوسِ همکارانه» توسط هادفیلد-منل و همکاران (2017a) معرفی شدند. مالیک و همکاران (2018) حل‌کنندهٔ کارآمدِ POMDP ای را معرفی کردند که به‌طورِ خاص برای بازی‌های کمکی طراحی شده. آن‌ها به «بازی‌های کارفرما–کارگزار» (Principal–Agent Games) در اقتصاد مربوط‌اند، که در آن اصیل (مثلاً کارفرما) و کارگزار (مثلاً کارمند) باید با وجودِ ترجیح‌های بسیار متفاوت، ترتیبی با سودِ متقابل بیابند. تفاوت‌های اصلی این‌اند که (۱) ربات ترجیحِ خودش را ندارد، و (۲) ربات دربارهٔ ترجیحِ انسانی که باید بهینه کند نامطمئن است.

بازی‌های همکارانه نخستین بار توسط فون نویمان و مورگنشترن (1944) مطالعه شدند. مفهومِ هسته توسط دونالد گیلیز (1959) و ارزشِ شپلی توسط لوید شپلی (1953a) معرفی شد. مقدمهٔ خوبی بر ریاضیاتِ بازی‌های همکارانه پلگ و زودهولتر (2002) است. بازی‌های ساده به‌طورِ کلی با جزئیات توسط تیلور و زویکر (1999) بحث شده‌اند. برای مقدمه‌ای بر جنبه‌های محاسباتیِ نظریهٔ بازیِ همکارانه، چالکیاداکیس و همکاران (2011) را ببینید. بسیاری از طرح‌های بازنماییِ فشرده برای بازی‌های همکارانه در سه دههٔ گذشته توسعه یافته‌اند، با شروع از کارِ دنگ و پاپادیمیتریو (1994). تأثیرگذارترین این طرح‌ها مدلِ شبکه‌های مشارکتِ حاشیه‌ای است، که توسط یئونگ و شوهام (2005) معرفی شد. رویکردِ تشکیلِ ائتلاف که توصیف می‌کنیم توسط سندلم و همکاران (1999) توسعه یافت؛ راهوان و همکاران (2015) وضعیتِ هنر را مرور می‌کنند.

پروتکلِ شبکهٔ قرارداد توسط رید اسمیت برای کارِ دکترایِ خود در دانشگاه استنفورد در اواخرِ دههٔ ۱۹۷۰ معرفی شد (اسمیت، 1980). به نظر می‌رسد پروتکل چنان طبیعی است که تا به امروز مرتباً دوباره اختراع می‌شود. شالوده‌های اقتصادیِ پروتکل توسط سندلم (1993) مطالعه شد. حراج‌ها و طراحیِ سازوکار برای چند دهه موضوعاتِ اصلی در علومِ رایانه و هوش مصنوعی بوده‌اند: نیسان (2007) را برای دیدگاهِ اصلیِ علومِ رایانه، کریشنا (2002) را برای مقدمه‌ای بر نظریهٔ حراج‌ها، و کرامتون و همکاران (2006) را برای مجموعه‌ای از مقاله‌ها دربارهٔ جنبه‌های محاسباتیِ حراج‌ها ببینید.

جایزهٔ یادبودِ نوبلِ اقتصادِ ۲۰۰۷ به هورویتس، ماسکین و میرسن «برای پایه‌گذاریِ نظریهٔ طراحیِ سازوکار» تعلق گرفت (هورویتس، ۱۹۷۳). تراژدیِ مشترکات، مسئلهٔ انگیزه‌بخشِ حوزه، توسط ویلیام لوید (1833) تحلیل شد اما توسط گرت هاردین (1968) نام‌گذاری و به توجهِ عمومی آورده شد. رونالد کوز قضیه‌ای ارائه کرد که اگر منابع تحتِ مالکیتِ خصوصی باشند و هزینه‌های معامله به‌اندازهٔ کافی پایین باشند، منابع به‌صورتِ کارآمد مدیریت خواهند شد (کوز، 1960). او اشاره می‌کند در عمل هزینه‌های معامله بالا هستند، پس این قضیه اعمال نمی‌شود، و باید به راه‌حل‌های دیگرِ فراتر از خصوصی‌سازی و بازار نگاه کنیم. «حکومت بر مشترکات» الینور اوستروم (1990) راه‌حل‌هایی برای مسئله توصیف کرد که بر اساسِ قراردادنِ کنترلِ مدیریتیِ منابع در دستانِ مردمِ محلیِ دارایِ بیشترین دانش از وضعیت است. هم کوز و هم اوستروم برای کارشان جایزهٔ نوبل در اقتصاد بردند.

اصلِ فاش‌سازی از میرسن (1986) است، و قضیهٔ هم‌ارزیِ درآمد به‌طورِ مستقل توسط میرسن (1981) و رایلی و ساموئلسون (1981) توسعه یافت. دو اقتصاددان، میلگروم (1997) و کلمپرر (2002)، دربارهٔ حراج‌های طیفِ چندمیلیارددلاری که در آن‌ها درگیر بودند می‌نویسند.

طراحیِ سازوکار در برنامه‌ریزیِ چندعاملی (هانسبرگر و گروز، 2000؛ استون و همکاران، 2009) و زمان‌بندی (راسنتی و همکاران، 1982) استفاده می‌شود. واریان (1995) مرورِ مختصری با ارتباط‌هایی به ادبیاتِ علومِ رایانه می‌دهد، و روزنشاین و زلوتکین (1994) برخوردِ کتاب‌حجمی با کاربردهایی در هوش مصنوعیِ توزیع‌شده ارائه می‌کنند. کارهای مربوط به هوش مصنوعیِ توزیع‌شده تحتِ چند نام می‌رود، از جملهِ «هوشِ جمعی» (تومر و وولپرت، 2000؛ سگاران، 2007) و «کنترلِ بازارمحور» (کلیرواتر، 1996). از ۲۰۰۱، «مسابقهٔ سالانهٔ عامل‌های معامله‌گر» (TAC) برگزار می‌شود، که در آن عامل‌ها سعی می‌کنند بهترین سود را در توالی‌ای از حراج‌ها به دست آورند (ولمن و همکاران، 2001؛ آروناچالام و ساده، 2005).

ادبیاتِ انتخابِ اجتماعی عظیم است و گستره‌ای از ملاحظاتِ فلسفی دربارهٔ ماهیتِ دموکراسی تا تحلیل‌های بسیار فنیِ رویه‌های رأی‌گیریِ خاص را در بر می‌گیرد. کمپبل و کلی (2002) نقطهٔ شروعِ خوبی برای این ادبیات فراهم می‌کنند. «راهنمای انتخابِ اجتماعیِ محاسباتی» طیفی از مقاله‌ها را فراهم می‌کند که موضوعاتِ پژوهشی و روش‌های این حوزه را مرور می‌کنند (برانت و همکاران، 2016). قضیهٔ ارو ویژگی‌های مطلوبِ سامانهٔ رأی‌گیری را فهرست می‌کند و اثبات می‌کند دستیابی به همهٔ آن‌ها ناممکن است (ارو، 1951). داسگوپتا و ماسکین (2008) نشان می‌دهند قاعدهٔ اکثریت (نه قاعدهٔ کثرت، و نه رأیِ رتبه‌بندی‌شده) مقاوم‌ترین سامانهٔ رأی‌گیری است. پیچیدگیِ محاسباتیِ دستکاریِ انتخابات نخستین بار توسط بارتولدی و همکاران (1989) مطالعه شد.

ما به‌سختی سطحِ کارِ مذاکره در برنامه‌ریزیِ چندعاملی را خراشیده‌ایم. دورفی و لسر (1989) بحث می‌کنند چگونه وظایف را می‌توان با مذاکره میانِ عامل‌ها به اشتراک گذاشت. کراوس و همکاران (1991) سامانه‌ای برای بازیِ «دیپلماسی» توصیف می‌کنند، بازیِ رومیزی‌ای که نیازمندِ مذاکره، تشکیلِ ائتلاف و ناصادقی است. استون (2000) نشان می‌دهد چگونه عامل‌ها می‌توانند به‌عنوانِ هم‌تیمی در محیطِ رقابتی، پویا و تا حدی قابل مشاهدهٔ فوتبالِ رباتیک همکاری کنند. در مقاله‌ای بعدی، استون (2003) دو محیطِ چندعاملیِ رقابتی—RoboCup، مسابقهٔ فوتبالِ رباتیک، و TAC، مسابقهٔ عامل‌های معامله‌گرِ مبتنی بر حراج—را تحلیل می‌کند و درمی‌یابد ناشدنیِ محاسباتیِ رویکردهایِ کنونیِ ازنظرِ نظری خوش‌بنیاد به طراحیِ بسیاری از سامانه‌های چندعاملی با روش‌های موردی انجامیده.

ساریت کراوس شماری از عامل‌ها را توسعه داده که می‌توانند با انسان‌ها و عامل‌های دیگر مذاکره کنند—مرور را در کراوس (2001) ببینید. پروتکلِ امتیازِ یکنواخت برای مذاکرهٔ خودکار توسط جفری اس. روزنشاین و دانشجویانش پیشنهاد شد (روزنشاین و زلوتکین، 1994). پروتکلِ پیشنهادهای متناوب توسط روبینشتاین (1982) توسعه یافت.

کتاب‌های سامانه‌های چندعاملی شاملِ وایس (2000a)، یانگ (2004)، ولاسیس (2008)، شوهام و لیتون-براون (2009) و وولدریج (2009) می‌شوند. کنفرانسِ اصلیِ سامانه‌های چندعاملی «کنفرانسِ بین‌المللیِ عامل‌های خودمختار و سامانه‌های چندعاملی» (AAMAS) است؛ ژورنالی با همان نام نیز هست. «کنفرانسِ ACM در تجارتِ الکترونیک» (EC) نیز مقاله‌های مربوطِ بسیاری منتشر می‌کند، به‌طورِ خاص در حوزهٔ الگوریتم‌های حراج. ژورنالِ اصلیِ نظریهٔ بازی «بازی‌ها و رفتارِ اقتصادی» (Games and Economic Behavior) است.

پانوشت‌ها:

[1] مورا نسخهٔ تفریحیِ بازیِ بازرسی است. در چنین بازی‌هایی، بازرس روزی را برای بازرسیِ تأسیساتی (مانندِ رستوران یا کارخانهٔ سلاحِ بیولوژیک) انتخاب می‌کند، و اپراتورِ تأسیسات روزی را برای پنهان‌کردنِ همهٔ چیزهای بد انتخاب می‌کند. بازرس اگر روزها متفاوت باشند می‌برد، و اپراتورِ تأسیسات اگر یکسان باشند.

[2] تصادفی است این معادله‌ها همان معادله‌های p هستند؛ تصادف پدید می‌آید، زیرا UE(one,two) = UE(two,one) = -3. این همچنین توضیح می‌دهد چرا استراتژیِ بهینه برای هر دو بازیکن یکسان است.

[3] واژهٔ «auction» از لاتینِ augeo، به معنای افزایش، می‌آید.

[4] در واقع شانسِ کوچکی هست که عاملِ با بالاترین vi در به‌دست‌آوردنِ کالاها ناکام بماند، در حالتی که bo < vi < bo + d. احتمالِ این اتفاق را می‌توان با کوچک‌کردنِ دلخواه گامِ افزایشِ d کاهش داد.

[5] به نامِ ویلیام ویکری (۱۹۹۶–۱۹۱۴)، که برای این کار برندهٔ جایزهٔ نوبلِ ۱۹۹۶ در اقتصاد شد و سه روز بعد بر اثرِ حملهٔ قلبی درگذشت.

فصل ۱۹: یادگیری از مثال‌ها (Learning from Examples)

در این فصل عامل‌هایی را توصیف می‌کنیم که می‌توانند رفتارشان را از طریق مطالعهٔ دقیقِ تجربه‌های گذشته و پیش‌بینی‌های آینده بهبود بخشند.

عامل در حالِ یادگیری است اگر پس از مشاهده‌هایی دربارهٔ جهان عملکردش را بهبود بخشد. یادگیری می‌تواند از پیش‌پاافتاده، مانندِ نوشتنِ فهرستِ خرید، تا عمیق، مانندِ وقتی که آلبرت اینشتین نظریهٔ تازه‌ای از جهان استنتاج کرد، باشد. وقتی عامل رایانه باشد، آن را «یادگیریِ ماشینی» (Machine Learning) می‌نامیم: رایانه‌ای داده‌هایی را مشاهده می‌کند، بر اساسِ داده‌ها مدلی می‌سازد، و از مدل هم به‌عنوانِ فرضیه‌ای دربارهٔ جهان و هم به‌عنوانِ قطعه‌ای نرم‌افزار که می‌تواند مسائل را حل کند استفاده می‌کند. چرا بخواهیم ماشین یاد بگیرد؟ چرا آن را از همان آغاز درست برنامه‌ریزی نکنیم؟ دو دلیلِ اصلی هست. نخست، طراحان نمی‌توانند همهٔ موقعیت‌های آیندهٔ ممکن را پیش‌بینی کنند. برای نمونه، رباتی که برای پیمایشِ مارپیچ‌ها طراحی شده باید چیدمانِ هر مارپیچِ تازه‌ای را که با آن روبه‌رو می‌شود یاد بگیرد؛ برنامه‌ای برای پیش‌بینیِ قیمت‌های سهام باید یاد بگیرد وقتی شرایط از رونق به رکود تغییر می‌کند خودش را سازگار کند. دوم، گاهی طراحان خودشان هیچ ایده‌ای برای برنامه‌ریزیِ راه‌حل ندارند. بیشترِ مردم در بازشناسیِ چهره‌های اعضای خانواده خوب‌اند، اما آن را ناخودآگاه انجام می‌دهند، پس حتی بهترین برنامه‌نویسان نمی‌دانند چگونه رایانه‌ای را برای انجامِ آن تکلیف برنامه‌ریزی کنند، مگر با استفاده از الگوریتم‌های یادگیریِ ماشینی.

در این فصل، بحثِ رده‌های مدلِ گوناگون—درخت‌های تصمیم (بخش ۱۹.۳)، مدل‌های خطی (بخش ۱۹.۶)، مدل‌های ناپارامتری مانندِ نزدیک‌ترین همسایه‌ها (بخش ۱۹.۷)، مدل‌های گروهی مانندِ جنگل‌های تصادفی (بخش ۱۹.۸)—را با توصیه‌های عملی برای ساختِ سامانه‌های یادگیریِ ماشینی (بخش ۱۹.۹) و بحثِ نظریهٔ یادگیریِ ماشینی (بخش‌های ۱۹.۱ تا ۱۹.۵) در هم می‌آمیزیم.

۱۹.۱ شکل‌های یادگیری

هر مؤلفه‌ای از برنامهٔ عامل را می‌توان با یادگیریِ ماشینی بهبود بخشید. بهبودها، و فنونِ استفاده‌شده برای انجامشان، به این عوامل بستگی دارند:

- کدام مؤلفه قرار است بهبود یابد.
- عامل چه دانشِ قبلی دارد، که بر مدلی که می‌سازد اثر می‌گذارد.
- چه داده و بازخوردی روی آن داده در دسترس است.

فصل ۲ چند طراحیِ عامل را توصیف کرد. مؤلفه‌های این عامل‌ها شاملِ:

1. نگاشتِ مستقیم از شرایطِ روی حالتِ کنونی به کنش‌ها.

2. وسیله‌ای برای استنتاجِ ویژگی‌های مربوطِ جهان از توالیِ ادراک‌های حسی.
3. اطلاعاتی دربارهٔ شیوهٔ تکاملِ جهان و دربارهٔ نتایجِ کنش‌های ممکنی که عامل می‌تواند انجام دهد.
4. اطلاعاتِ مطلوبیتی که مطلوبیتِ حالت‌های جهان را نشان می‌دهد.
5. اطلاعاتِ کنش-ارزشی که مطلوبیتِ کنش‌ها را نشان می‌دهد.
6. هدف‌هایی که مطلوب‌ترین حالت‌ها را توصیف می‌کنند.
7. مولدِ مسئله، منتقد و عنصرِ یادگیری که سامانه را قادر به بهبود می‌کنند.

هر یک از این مؤلفه‌ها قابلِ یادگیری است. عاملِ خودرویِ خودرانی را در نظر بگیرید که با مشاهدهٔ رانندهٔ انسانی یاد می‌گیرد. هر بار که راننده ترمز می‌کند، عامل ممکن است قاعدهٔ شرط–کنشی برای زمانِ ترمزگیری یاد بگیرد (مؤلفهٔ ۱). با دیدنِ تصاویرِ دوربینِ بسیاری که به آن گفته شده حاوی اتوبوس‌اند، می‌تواند آن‌ها را بازشناسی کند (مؤلفهٔ ۲). با امتحانِ کنش‌ها و مشاهدهٔ نتایج—برای نمونه، ترمزِ محکم روی جادهٔ خیس—می‌تواند اثرهای کنش‌هایش را یاد بگیرد (مؤلفهٔ ۳). سپس، وقتی از مسافرانی که در طولِ سفر به‌شدت تکان خورده‌اند شکایت دریافت می‌کند، می‌تواند مؤلفهٔ مفیدی از تابعِ مطلوبیتِ کلی‌اش یاد بگیرد (مؤلفهٔ ۴). فناوریِ یادگیریِ ماشینی بخشِ استانداردی از مهندسیِ نرم‌افزار شده.

هر بار که سامانهٔ نرم‌افزاری می‌سازید، حتی اگر آن را عاملِ هوش مصنوعی ندانید، مؤلفه‌های سامانه به‌طورِ بالقوه با یادگیریِ ماشینی قابلِ بهبودند. برای نمونه، نرم‌افزاری برای تحلیلِ تصاویرِ کهکشان‌ها تحتِ عدسی‌شدنِ گرانشی با مدلِ یادگیریِ ماشینی ضریبِ ۱۰ میلیون سریع‌تر شد (هزاوه و همکاران، 2017)، و مصرفِ انرژی برای خنک‌کردنِ مراکزِ داده با مدلِ یادگیریِ ماشینیِ دیگری ۴۰٪ کاهش یافت (گائو، 2014). برندهٔ جایزهٔ تورینگ دیوید پترسون و رئیسِ گوگل‌ای‌آی جف دین طلوعِ «عصرِ طلایی» برای معماریِ رایانه به دلیلِ یادگیریِ ماشینی را اعلام کردند (دین و همکاران، 2018).

نمونه‌های بسیاری از مدل‌های مؤلفه‌های عامل دیده‌ایم: مدل‌های اتمی، عاملی و رابطه‌ای مبتنی بر منطق یا احتمال، و مانندِ آن. الگوریتم‌های یادگیری برای همهٔ این‌ها ابداع شده‌اند. این فصل دانشِ قبلیِ کمی برای عامل فرض می‌کند: از صفر شروع می‌کند و از داده‌ها یاد می‌گیرد. در بخش ۲۱.۷.۲ «یادگیریِ انتقالی» (Transfer Learning) را در نظر می‌گیریم، که در آن دانش از حوزه‌ای به حوزهٔ تازه منتقل می‌شود، تا یادگیری با دادهٔ کمتر سریع‌تر پیش برود. اما فرض می‌کنیم طراحِ سامانه چارچوبِ مدلی را انتخاب می‌کند که می‌تواند به یادگیریِ مؤثر بینجامد.

رفتن از مجموعه‌ای خاص از مشاهده‌ها به قاعده‌ای عمومی «استقرا» (Induction) نامیده می‌شود؛ از مشاهده‌های اینکه خورشید هر روزِ گذشته طلوع کرده، استقرا می‌کنیم خورشید فردا طلوع خواهد کرد. این با قیاسی که در فصل ۷ مطالعه کردیم متفاوت است، زیرا نتایجِ استقرایی ممکن است نادرست باشند، در حالی که نتایجِ قیاسی اگر مقدمات درست باشند تضمین‌شده درست‌اند. این فصل بر مسائلی تمرکز می‌کند که ورودی بازنماییِ عاملی است—برداری از مقادیرِ ویژگی. همچنین ممکن است ورودی هر نوع ساختارِ داده‌ای باشد، از جملهِ اتمی و رابطه‌ای.

وقتی خروجی یکی از مجموعهٔ متناهی از مقادیر است (مانندِ آفتابی/ابری/بارانی یا صادق/کاذب)، مسئلهٔ یادگیری «رده‌بندی» (Classification) نامیده می‌شود. وقتی عدد است (مانندِ دمایِ فردا، که یا عددِ صحیح اندازه‌گیری می‌شود یا عددِ حقیقی)، مسئلهٔ یادگیری نامِ (اعترافاً مبهم) «رگرسیون» (Regression) را دارد.[1] نامِ بهتر «تقریبِ تابع» یا «پیش‌بینیِ عددی» می‌بود. اما در ۱۸۸۶ فرانسیس گالتون مقالهٔ تأثیرگذاری دربارهٔ مفهومِ «بازگشت به میانگین» نوشت

(مثلاً فرزندانِ والدینِ قدبلند احتمالاً بلندتر از میانگین‌اند، اما نه به‌اندازهٔ والدین). گالتون نمودارهایی را با چیزی که «خطوطِ رگرسیون» می‌نامید نشان داد، و خوانندگان به‌مرور واژهٔ «رگرسیون» را با فنِ آماریِ تقریبِ تابع مرتبط کردند، نه با موضوعِ بازگشت به میانگین.

۱۹.۲ یادگیریِ نظارت‌شده

سه نوع بازخورد می‌تواند همراهِ ورودی‌ها باشد، که سه نوع اصلیِ یادگیری را تعیین می‌کنند:

- در «یادگیریِ نظارت‌شده» (Supervised Learning) عامل جفت‌های ورودی-خروجی را مشاهده می‌کند و تابعی را یاد می‌گیرد که از ورودی به خروجی نگاشت می‌کند. برای نمونه، ورودی‌ها می‌توانند تصاویرِ دوربین باشند، هر یک همراه با خروجیِ «اتوبوس» یا «عابر پیاده» و... چنین خروجی‌ای «برچسب» (Label) نامیده می‌شود. عامل تابعی را یاد می‌گیرد که، وقتی تصویرِ تازه‌ای داده می‌شود، برچسبِ مناسب را پیش‌بینی می‌کند. در موردِ کنش‌های ترمز (مؤلفهٔ ۱ بالا)، ورودی حالتِ کنونی (سرعت و جهتِ خودرو، وضعیتِ جاده) و خروجی فاصله‌ای است که برای توقف لازم بود. در این مورد، مجموعه‌ای از مقادیرِ خروجی را عامل می‌تواند از ادراک‌های حسیِ خودش (پس از وقوع) به دست آورد؛ محیط معلم است، و عامل تابعی را یاد می‌گیرد که حالت‌ها را به فاصلهٔ توقف نگاشت می‌کند.
- در «یادگیریِ نظارت‌نشده» (Unsupervised Learning) عامل الگوها را در ورودی بدونِ هیچ بازخوردِ صریحی یاد می‌گیرد. رایج‌ترین تکلیفِ یادگیریِ نظارت‌نشده «خوشه‌بندی» (Clustering) است: تشخیصِ خوشه‌های بالقوه مفیدِ مثال‌های ورودی. برای نمونه، وقتی میلیون‌ها تصویرِ گرفته‌شده از اینترنت نشان داده می‌شود، سامانهٔ بینواییِ رایانه‌ای می‌تواند خوشهٔ بزرگی از تصاویرِ مشابه را شناسایی کند که انگلیسی‌زبان آن را «گربه» می‌نامد.
- در «یادگیریِ تقویتی» (Reinforcement Learning) عامل از مجموعه‌ای از تقویت‌ها یاد می‌گیرد: پاداش‌ها و تنبیه‌ها. برای نمونه، در پایانِ بازیِ شطرنج به عامل گفته می‌شود برده (پاداش) یا باخته (تنبیه). این به عهدهٔ عامل است که تصمیم بگیرد کدام‌یک از کنش‌های قبل از تقویت بیشترین مسئولیت را داشتند، و کنش‌هایش را برای هدف‌گرفتنِ پاداش‌های بیشتر در آینده تغییر دهد.

به‌صورتِ صوری‌تر، تکلیفِ یادگیریِ نظارت‌شده چنین است:

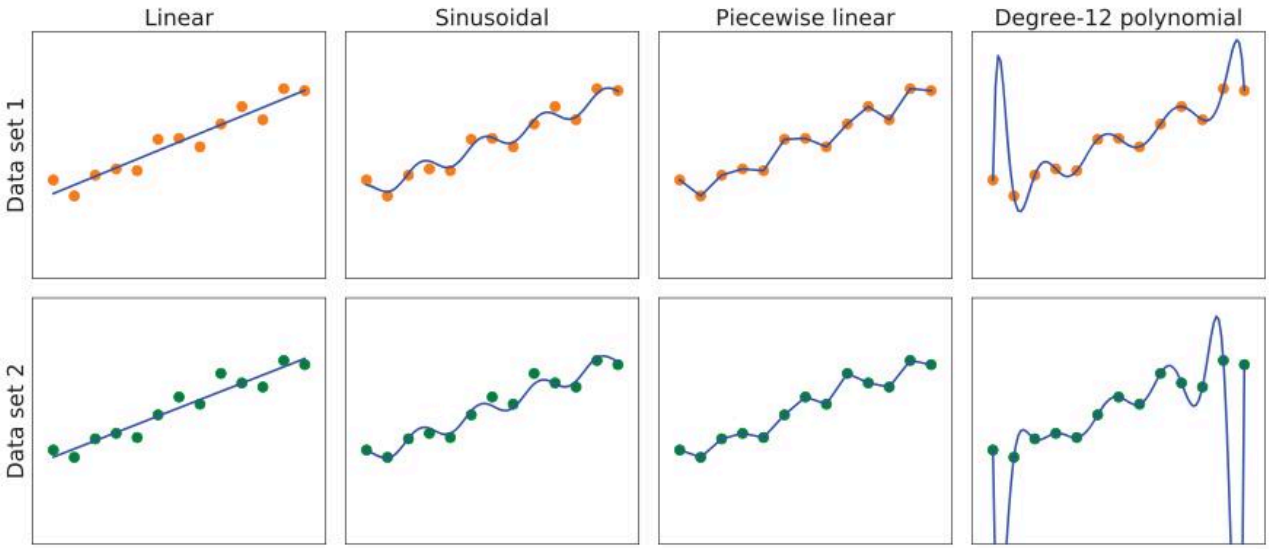
با توجه به «مجموعهٔ آموزشی» (Training Set) از N جفتِ مثالِ ورودی-خروجی (x1,y1), (x2,y2), ..., (xN,yN)، که هر جفت توسط تابعِ ناشناختهٔ y = f(x) تولید شده، تابعِ h را کشف کن که به تابعِ واقعیِ f نزدیک شود.

تابعِ h «فرضیه» (Hypothesis) دربارهٔ جهان نامیده می‌شود. از «فضایِ فرضیه» (Hypothesis Space) H از توابعِ ممکن گرفته می‌شود. برای نمونه، فضایِ فرضیه ممکن است مجموعهٔ چندجمله‌ای‌های درجهٔ 3 باشد؛ یا مجموعهٔ توابعِ جاوااسکریپت؛ یا مجموعهٔ فرمول‌های منطقِ بولیِ 3-SAT.

با واژگانِ جایگزین، می‌توانیم بگوییم h «مدلی» (Model) از داده‌هاست، برگرفته از «ردهٔ مدلِ» H (Model Class)، یا تابعی برگرفته از ردهٔ تابع. خروجیِ yi را «حقیقتِ زمینی» (Ground Truth) می‌نامیم—پاسخِ واقعی‌ای که از مدل‌مان می‌خواهیم پیش‌بینی کند.

چگونه فضایِ فرضیه را انتخاب کنیم؟ ممکن است دانشِ قبلی‌ای دربارهٔ فرایندی که داده را تولید کرده داشته باشیم. اگر نه، می‌توانیم «تحلیلِ دادهٔ اکتشافی» (Exploratory Data Analysis) انجام دهیم: بررسیِ داده با آزمون‌های آماری و تجسم‌ها—هیستوگرام‌ها، نمودارهای پراکندگی، نمودارهای جعبه‌ای—برای گرفتنِ حسِ داده و بینشی دربارهٔ اینکه چه فضایِ فرضیه‌ای ممکن است مناسب باشد. یا می‌توانیم چند فضایِ فرضیه را امتحان کنیم و ارزیابی کنیم کدام بهتر کار می‌کند.

چگونه فرضیهٔ خوبی از درونِ فضایِ فرضیه انتخاب کنیم؟ می‌توانیم به «فرضیهٔ سازگار» (Consistent Hypothesis) امیدوار باشیم: h ای که برای هر xi در مجموعهٔ آموزشی h(xi) = yi داشته باشد. با خروجی‌های پیوسته‌مقدار نمی‌توانیم انتظارِ تطابقِ دقیق با حقیقتِ زمینی داشته باشیم؛ در عوض به دنبال تابعِ بهترین‌برازش می‌گردیم که برای هر xi مقدارِ h(xi) نزدیکِ yi باشد (به‌شیوه‌ای که در بخش ۱۹.۴.۲ صوری می‌کنیم). معیارِ واقعیِ فرضیه این نیست که روی مجموعهٔ آموزشی چگونه عمل می‌کند، بلکه چقدر خوب ورودی‌هایی را که هنوز ندیده مدیریت می‌کند. می‌توانیم آن را با نمونهٔ دومی از جفت‌های (xi,yi) به نامِ «مجموعهٔ آزمون» (Test Set) ارزیابی کنیم. می‌گوییم h «خوب تعمیم می‌دهد» (Generalization) اگر خروجی‌های مجموعهٔ آزمون را دقیقاً پیش‌بینی کند.



شکل ۱۹.۱ — یافتن فرضیه‌هایی برای برازشِ داده. ردیفِ بالا: چهار نمودار از توابعِ بهترین‌برازش از چهار فضایِ فرضیهٔ متفاوت، آموزش‌دیده روی مجموعهٔ دادهٔ 1. ردیفِ پایین: همان چهار تابع، اما آموزش‌دیده روی مجموعهٔ دادهٔ کمی متفاوت (نمونه‌برداری‌شده از همان تابعِ f(x)).

شکل ۱۹.۱ نشان می‌دهد تابعِ h که الگوریتمِ یادگیری کشف می‌کند به فضایِ فرضیهٔ H که در نظر می‌گیرد و به مجموعهٔ آموزشی که به او داده می‌شود بستگی دارد. هر یک از چهار نمودارِ ردیفِ بالا همان مجموعهٔ آموزشیِ ۱۳ نقطه‌ای در صفحهٔ (x,y) را دارند. چهار نمودارِ ردیفِ پایین مجموعهٔ دومی از ۱۳ نقطه دارند؛ هر دو مجموعه نمایندهٔ همان تابعِ ناشناختهٔ f(x) هستند. هر ستون بهترین فرضیهٔ h را از فضایِ فرضیهٔ متفاوتی نشان می‌دهد:

- **ستون 1:** خطوطِ راست؛ توابعِ شکلِ h(x) = w1x + w0. هیچ خطی نیست که فرضیهٔ سازگاری برای نقاطِ داده باشد.

- **ستون 2:** توابعِ سینوسیِ شکلِ $h(x) = w1x + \sin(w0x)$. این انتخاب کاملاً سازگار نیست، اما هر دو مجموعهٔ داده را بسیار خوب برازش می‌کند.
- **ستون 3:** توابعِ تکه‌ای-خطی که در آن هر پاره‌خط نقطه‌ها را از یک نقطهٔ داده به نقطهٔ بعدی وصل می‌کند. این توابع همیشه سازگارند.
- **ستون 4:** چندجمله‌ای‌های درجهٔ 12، $h(x) = \sum_{i=0}^{12} wixi$. این‌ها سازگارند: همیشه می‌توانیم چندجمله‌ایِ درجهٔ 12 را پیدا کنیم که ۱۳ نقطهٔ متمایز را کاملاً برازش کند. اما صرفِ سازگاربودنِ فرضیه به این معنا نیست حدسِ خوبی است.

یکی از راه‌های تحلیلِ فضاهای فرضیه، با «اریبی» (Bias) که تحمیل می‌کنند (صرف‌نظر از مجموعهٔ آموزشی) و «واریانسِ» (Variance) که تولید می‌کنند (از یک مجموعهٔ آموزشی به مجموعهٔ دیگر) است. منظورمان از اریبی (تقریباً) گرایشِ فرضیهٔ پیش‌بینی‌کننده به انحراف از مقدارِ موردانتظار هنگام میانگین‌گیری روی مجموعه‌های آموزشیِ مختلف است. اریبی اغلب از محدودیت‌های تحمیل‌شده توسط فضایِ فرضیه ناشی می‌شود. برای نمونه، فضایِ فرضیهٔ توابعِ خطی اریبیِ قوی القا می‌کند: فقط به توابعِ تشکیل‌شده از خطوطِ راست اجازه می‌دهد. اگر الگوهایی در داده غیر از شیبِ کلیِ خط باشد، تابعِ خطی نمی‌تواند آن الگوها را بازنمایی کند. می‌گوییم فرضیه «کم‌برازش» (Underfitting) می‌کند وقتی در یافتنِ الگویی در داده ناکام می‌ماند. از سوی دیگر، تابعِ تکه‌ای-خطی اریبیِ کمی دارد؛ شکلِ تابع توسط داده رانده می‌شود.

منظورمان از واریانس میزانِ تغییرِ فرضیه به دلیلِ نوسان در دادهٔ آموزشی است. دو ردیفِ شکل ۱۹.۱ مجموعه‌های داده‌ای را بازنمایی می‌کنند که هر یک از همان تابعِ $f(x)$ نمونه‌برداری شده‌اند. مجموعه‌های داده معلوم شد کمی متفاوت‌اند. برای سه ستونِ نخست، تفاوتِ کوچک در مجموعهٔ داده به تفاوتِ کوچک در فرضیه ترجمه می‌شود. آن را واریانسِ کم می‌نامیم. اما چندجمله‌ای‌های درجهٔ 12 در ستونِ چهارم واریانسِ بالایی دارند: ببینید دو تابع در هر دو انتهای محورِ x چقدر متفاوت‌اند. بدیهی است، دست‌کم یکی از این چندجمله‌ای‌ها باید تقریبِ ضعیفی از $f(x)$ واقعی باشد. می‌گوییم تابعی «بیش‌برازش» (Overfitting) می‌کند وقتی بیش‌ازحد به مجموعهٔ دادهٔ خاصی که رویش آموزش دیده توجه می‌کند و باعث می‌شود روی دادهٔ دیده‌نشده ضعیف عمل کند.

اغلب «معاوضهٔ اریبی–واریانس» (Bias–Variance Tradeoff) هست: انتخابی میانِ فرضیه‌های پیچیده‌ترِ کم‌اریبی که دادهٔ آموزشی را خوب برازش می‌کنند و فرضیه‌های ساده‌ترِ کم‌واریانس که ممکن است بهتر تعمیم دهند. آلبرت اینشتین در ۱۹۳۳ گفت: «هدفِ برترِ همهٔ نظریه این است که عناصرِ بنیادینِ فروکاستنی را تا آن‌جا که ممکن است ساده و اندک کنیم، بدونِ اینکه مجبور شویم بازنماییِ کافیِ هیچ دادهٔ واحدی از تجربه را رها کنیم.» به بیانِ دیگر، اینشتین انتخابِ ساده‌ترین فرضیه‌ای را توصیه می‌کند که با داده هم‌خوان است. این اصل را می‌توان به فیلسوفِ انگلیسیِ قرنِ چهاردهم، ویلیام آوکام، بازگرداند.[۲] اصلِ او که «کثرت [موجودات] نباید بدونِ ضرورت فرض شود» «تیغِ آوکام» نامیده می‌شود، زیرا برای «تراشیدنِ» تبیین‌های مشکوک استفاده می‌شود.

تعریفِ سادگی آسان نیست. روشن به نظر می‌رسد چندجمله‌ایِ با فقط دو پارامتر ساده‌تر از چندجمله‌ایِ با سیزده پارامتر است. این شهود را در بخش ۱۹.۳.۴ دقیق‌تر خواهیم کرد. با این حال، در فصل ۲۱ خواهیم دید مدل‌های شبکهٔ عصبیِ عمیق اغلب می‌توانند به‌خوبی تعمیم دهند، حتی اگرچه بسیار پیچیده‌اند—برخی از آن‌ها میلیاردها پارامتر دارند.

پس شمارِ پارامترها به‌خودی‌خود معیارِ خوبی برای شایستگیِ مدل نیست. شاید باید هدفِ «مناسبت» باشد، نه «سادگی»، در ردهٔ مدل. این مسئله را در بخش ۱۹.۴.۱ در نظر می‌گیریم.

کدام فرضیه در شکل ۱۹.۱ بهترین است؟ نمی‌توانیم مطمئن باشیم. اگر می‌دانستیم داده، مثلاً، شمارِ بازدیدِ وب‌سایتی را بازنمایی می‌کند که روزبه‌روز رشد می‌کند، اما بسته به زمانِ روز نیز چرخه دارد، آن‌گاه شاید تابعِ سینوسی را ترجیح می‌دادیم. اگر می‌دانستیم داده قطعاً چرخه‌ای نیست اما نویزِ بالایی دارد، آن‌گاه شاید تابعِ خطی را ترجیح می‌دادیم.

در برخی موارد، تحلیلگر مایل است بگوید نه فقط فرضیه ممکن یا ناممکن است، بلکه چقدر محتمل است. یادگیریِ نظارت‌شده را می‌توان با انتخابِ فرضیهٔ h* انجام داد که با توجه به داده محتمل‌ترین است:

```
h* = argmax_{h∈H} P(h|data)
```

با قاعدهٔ بیز این معادل است با:

```
h* = argmax_{h∈H} P(data|h)P(h)
```

سپس می‌توانیم بگوییم احتمالِ پیشینیِ P(h) برای چندجمله‌ایِ نرمِ درجهٔ 1 یا 2 بالاست و برای چندجمله‌ایِ درجهٔ 12 با قله‌های بزرگ و تیز پایین‌تر است. وقتی داده بگوید واقعاً به توابعِ غیرعادی‌نما نیاز داریم به آن‌ها اجازه می‌دهیم، اما با دادنِ احتمالِ پیشینیِ کم آن‌ها را دلسرد می‌کنیم. چرا H را ردهٔ همهٔ برنامه‌های رایانه‌ای، یا همهٔ ماشین‌های تورینگ، قرار ندهیم؟ مسئله این است که معاوضه‌ای میانِ بیانی‌بودنِ فضایِ فرضیه و پیچیدگیِ محاسباتیِ یافتنِ فرضیهٔ خوب درونِ آن فضا هست. برای نمونه، برازشِ خطِ راست به داده محاسبه‌ای آسان است؛ برازشِ چندجمله‌ای‌های درجهٔ بالا تا حدی دشوارتر؛ و برازشِ ماشین‌های تورینگ تصمیم‌ناپذیر است. دلیلِ دوم برای ترجیحِ فضاهای فرضیهٔ ساده این است که احتمالاً پس از یادگیری می‌خواهیم از h استفاده کنیم، و محاسبهٔ h(x) وقتی h تابعِ خطی است تضمین‌شده سریع است، در حالی که اجرایِ برنامهٔ دلخواهِ ماشینِ تورینگ حتی تضمین‌شده خاتمه نمی‌یابد.

به این دلایل، بیشترِ کارهای یادگیری بر بازنمایی‌های ساده متمرکز شده‌اند. در سال‌های اخیر علاقهٔ زیادی به یادگیریِ عمیق (فصل ۲۱) بوده، جایی که بازنمایی‌ها ساده نیستند اما محاسبهٔ h(x) همچنان با سخت‌افزارِ مناسب فقط شمارِ کران‌داری گام می‌برد.

خواهیم دید معاوضهٔ بیانی–پیچیدگی ساده نیست: اغلب چنین است، چنان‌که با منطقِ مرتبهٔ اول در فصل ۸ دیدیم، که زبانِ بیانی فرضیهٔ ساده‌ای را ممکن می‌کند که داده را برازش کند، در حالی که محدودکردنِ بیانیِ زبان بدان معناست هر فرضیهٔ سازگاری باید پیچیده باشد.

۱۹.۲.۱ مسئلهٔ نمونه: انتظار در رستوران

مسئلهٔ یادگیریِ نظارت‌شدهٔ نمونه‌ای را با جزئیات توصیف خواهیم کرد: مسئلهٔ تصمیم‌گیری دربارهٔ اینکه آیا در رستوران منتظرِ میز بمانیم یا نه. این مسئله در سراسرِ فصل برای نشان‌دادنِ رده‌های مدلِ گوناگون استفاده خواهد شد. برای این مسئله، خروجیِ y متغیرِ بولیِ WillWait است؛ برای مثال‌هایی که در آن‌ها واقعاً منتظرِ میز می‌مانیم صادق است.

ورودیِ x بردارِ ده مقدارِ ویژگی است، که هر یک مقادیرِ گسسته دارد:

1. Alternate: آیا رستورانِ جایگزینِ مناسبی در نزدیکی هست.
2. Bar: آیا رستوران بارِ راحتی برای انتظار دارد.
3. Fri/Sat: در جمعه‌ها و شنبه‌ها صادق است.
4. Hungry: آیا الان گرسنه‌ایم.
5. Patrons: چند نفر در رستوران‌اند (مقادیر: None، Some و Full).
6. Price: محدودهٔ قیمتِ رستوران ($، $$، $$$).
7. Raining: آیا بیرون باران می‌بارد.
8. Reservation: آیا رزرو کرده‌ایم.
9. Type: نوعِ رستوران (فرانسوی، ایتالیایی، تایلندی یا برگر).
10. WaitEstimate: برآوردِ انتظارِ میزبان: 0–10، 10–30، 30–60 یا >60 دقیقه.

مجموعه‌ای از ۱۲ مثال، برگرفته از تجربهٔ یکی از ما (SR)، در شکل ۱۹.۲ نشان داده شده. توجه کنید این داده‌ها چقدر ناچیزند: $2^6 \times 3^2 \times 4^2 = 9{,}216$ ترکیبِ ممکن از مقادیرِ ویژگی‌های ورودی هست، اما خروجیِ درست فقط برای 12 تای آن‌ها به ما داده شده؛ هر یک از 9,204 موردِ دیگر می‌تواند صادق یا کاذب باشد؛ نمی‌دانیم. این جوهرِ استقراست: باید بهترین حدس‌مان را برای این 9,204 مقدارِ خروجیِ مفقود، با توجه به فقط شواهدِ ۱۲ مثال، بزنیم.

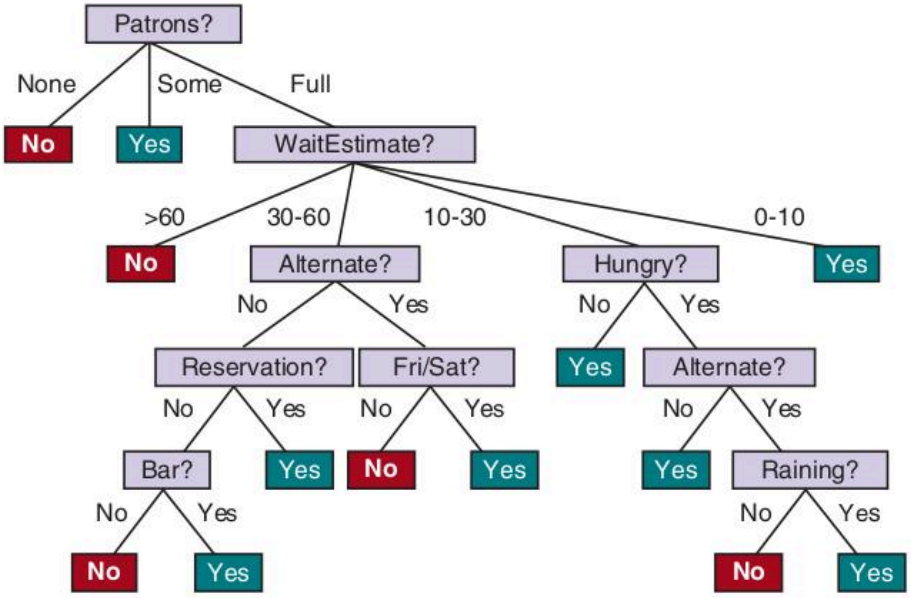
Example	Input Attributes										Output
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

شکل ۱۹.۲ — مثال‌های حوزهٔ رستوران.

۱۹.۳ یادگیریِ درخت‌های تصمیم

«درختِ تصمیم» (Decision Tree) بازنماییِ تابعی است که بردارِ مقادیرِ ویژگی را به مقدارِ خروجیِ تکی—یک «تصمیم»—نگاشت می‌کند. درختِ تصمیم تصمیمش را با اجرایِ توالیِ آزمون‌ها به دست می‌آورد، از ریشه شروع می‌کند و شاخهٔ مناسب را دنبال می‌کند تا به برگ برسد. هر گرهٔ داخلیِ درخت با آزمونِ مقدارِ یکی از ویژگی‌های ورودی متناظر است، شاخه‌های گره با مقادیرِ ممکنِ ویژگی برچسب‌گذاری می‌شوند، و گره‌های برگ مشخص می‌کنند تابع چه مقداری را باید برگرداند.

به‌طورِ کلی، مقادیرِ ورودی و خروجی می‌توانند گسسته یا پیوسته باشند، اما فعلاً فقط ورودی‌های تشکیل‌شده از مقادیرِ گسسته و خروجی‌هایی را در نظر می‌گیریم که یا صادق (مثالِ «مثبت» (Positive)) یا کاذب (مثالِ «منفی» (Negative)) هستند. این را «رده‌بندیِ بولی» می‌نامیم. از j برای نمایه‌زدنِ مثال‌ها استفاده می‌کنیم ($\mathbf{x}_j$ بردارِ ورودیِ مثالِ jام و y_j خروجی است)، و از $x_{j,i}$ برای ویژگیِ iامِ مثالِ jام.



شکل ۱۹.۳ — درختِ تصمیم برای تصمیم‌گیری دربارهٔ اینکه آیا منتظرِ میز بمانیم. (آزمون‌ها: Patrons؟ با شاخه‌های None/Some/Full؛ سپس Hungry؟، Fri/Sat؟، Type؟، WaitEstimate؟ با شاخه‌های No/Yes و 0–10/10–30/30–60/>60.)

درختِ بازنمایی‌کنندهٔ تابعِ تصمیمِ SR برای مسئلهٔ رستوران در شکل ۱۹.۳ نشان داده شده. با دنبال‌کردنِ شاخه‌ها، می‌بینیم مثالی با Patrons=Full و WaitEstimate=0–10 به‌صورتِ مثبت رده‌بندی می‌شود (یعنی بله، منتظرِ میز خواهیم ماند).

۱۹.۳.۱ بیانی بودنِ درخت‌های تصمیم

درختِ تصمیمِ بولی معادلِ گزارهٔ منطقیِ شکلِ:

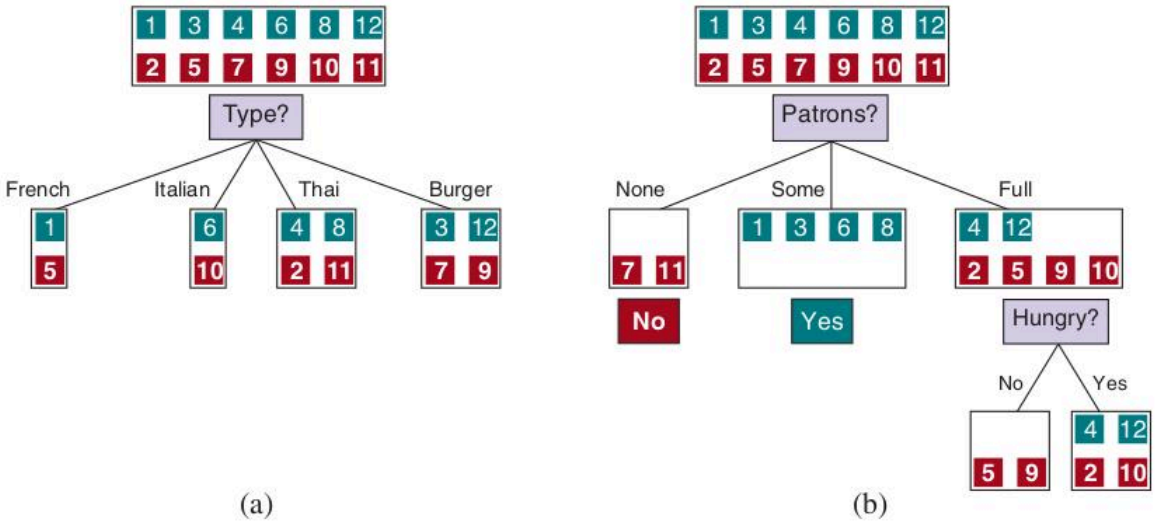
```
Output ⇔ (Path1 ∨ Path2 ∨ ...)
```

است، که در آن هر Pathi عطفی به شکلِ (... ∧ Am = vx ∧ An = vy) از آزمون‌های ویژگی-مقدارِ متناظر با مسیری از ریشه تا برگِ صادق است. بدین‌ترتیب، کلِ عبارت در «شکلِ نرمالِ فصلی» (DNF) است، بدان معنا که هر تابعی در منطقِ گزاره‌ای را می‌توان به‌صورتِ درختِ تصمیم بیان کرد. برای بسیاری از مسائل، قالبِ درختِ تصمیم نتیجهٔ خوب، موجز و قابلِ‌فهمی می‌دهد. در واقع، بسیاری از دفترچه‌های «راهنما» (مثلاً برای تعمیرِ خودرو) به‌صورتِ درخت‌های تصمیم نوشته می‌شوند. اما برخی توابع را نمی‌توان موجز بازنمایی کرد. برای نمونه، تابعِ اکثریت، که صادق را برمی‌گرداند اگر و فقط اگر بیش از نیمی از ورودی‌ها صادق باشند، به درختِ تصمیمِ نمایی‌بزرگ نیاز دارد، همان‌گونه که تابعِ توازن، که صادق را برمی‌گرداند اگر و فقط اگر شمارِ زوجی از ویژگی‌های ورودی صادق باشند. با ویژگی‌های حقیقی‌مقدار، تابعِ $y > A1 + A2$ با درختِ تصمیم دشوار بازنمایی می‌شود، زیرا مرزِ تصمیم خطِ قطری است، و همهٔ آزمون‌های درختِ تصمیم فضا را به جعبه‌های مستطیلیِ هم‌راستا با محور تقسیم می‌کنند. برای تقریبِ دقیقِ خطِ قطری باید جعبه‌های زیادی روی هم بچینیم. به بیانِ دیگر، درخت‌های تصمیم برای برخی انواعِ تابع خوب‌اند و برای برخی دیگر بد.

آیا بازنمایی‌ای هست که برای همهٔ انواعِ تابع کارآمد باشد؟ متأسفانه، پاسخ نه است—توابعِ بسیار زیادی برای بازنماییِ همه‌شان با شمارِ کمی بیت وجود دارد. حتی فقط با درنظرگرفتنِ توابعِ بولی با n ویژگیِ بولی، جدولِ صدق 2^n ردیف خواهد داشت، و هر ردیف می‌تواند صادق یا کاذب خروجی دهد، پس $2^{(2^n)}$ تابعِ متفاوت هست. با ۲۰ ویژگی، $2^{(1,048,576)} \approx 300,000^{10}$ تابع هست، پس اگر خودمان را به بازنماییِ میلیون‌بیتی محدود کنیم، نمی‌توانیم همهٔ این توابع را بازنمایی کنیم.

۱۹.۳.۲ یادگیریِ درخت‌های تصمیم از مثال‌ها

می‌خواهیم درختی بیابیم که با مثال‌های شکل ۱۹.۲ سازگار و تا آن‌جا که ممکن است کوچک باشد. متأسفانه، یافتنِ درختِ سازگارِ تضمین‌شدهٔ کوچک‌ترین ناشدنی است. اما با چند اکتشافیِ ساده، می‌توانیم کارآمد یکی را بیابیم که به کوچک‌ترین نزدیک است. الگوریتمِ LEARN-DECISION-TREE استراتژیِ حریصانهٔ تقسیم‌وغلبه را اتخاذ می‌کند: همیشه مهم‌ترین ویژگی را اول آزمون کن، سپس زیرمسائلِ کوچک‌تری را که با نتایجِ ممکنِ آزمون تعریف می‌شوند به‌صورتِ بازگشتی حل کن. منظورمان از «مهم‌ترین ویژگی» ویژگی‌ای است که بیشترین تفاوت را در رده‌بندیِ مثال ایجاد می‌کند. بدین‌گونه، امیدواریم با شمارِ کمی آزمون به رده‌بندیِ درست برسیم، بدان معنا که همهٔ مسیرهای درخت کوتاه و کلِ درخت کم‌عمق باشد.



شکل ۱۹.۴ – تقسیمِ مثال‌ها با آزمونِ ویژگی‌ها. در هر گره، مثال‌های مثبت (جعبه‌های روشن) و منفی (جعبه‌های تیره) باقی‌مانده را نشان می‌دهیم. (الف) تقسیم بر Type ما را به تمایزِ میانِ مثال‌های مثبت و منفی نزدیک‌تر نمی‌کند. (ب) تقسیم بر Patrons کارِ خوبی در جداکردنِ مثال‌های مثبت و منفی انجام می‌دهد. پس از تقسیم بر Patrons، Hungry آزمونِ دومِ نسبتاً خوبی است.

شکل ۱۹.۴(الف) نشان می‌دهد Type ویژگیِ ضعیفی است، زیرا ما را با چهار پیامد رها می‌کند که هر یک همان شمارِ مثالِ مثبت و منفی را دارد. از سوی دیگر، در (ب) می‌بینیم Patrons ویژگیِ نسبتاً مهمی است، زیرا اگر مقدار None یا Some باشد، با مجموعه‌های مثالی رها می‌شویم که می‌توانیم برایشان قطعاً پاسخ دهیم (به‌ترتیب No و Yes). اگر مقدار Full باشد، با مجموعهٔ آمیخته‌ای از مثال‌ها رها می‌شویم. چهار حالت برای این زیرمسائلِ بازگشتی باید در نظر گرفته شود:

1. اگر مثال‌های باقی‌مانده همه مثبت (یا همه منفی) باشند، کار تمام است: می‌توانیم Yes یا No پاسخ دهیم. شکل ۱۹.۴(ب) نمونه‌هایی از این را در شاخه‌های None و Some نشان می‌دهد.
2. اگر برخی مثبت و برخی منفی باشند، بهترین ویژگی را برای تقسیمِ آن‌ها انتخاب کن. شکل ۱۹.۴(ب) استفاده از Hungry را برای تقسیمِ مثال‌های باقی‌مانده نشان می‌دهد.
3. اگر مثالی باقی نمانده باشد، بدان معناست برای این ترکیبِ مقادیرِ ویژگی هیچ مثالی مشاهده نشده، و رایج‌ترین مقدارِ خروجی را از مجموعهٔ مثال‌هایی که در ساختِ والدِ گره استفاده شدند برمی‌گردانیم.
4. اگر ویژگی‌ای باقی نمانده، اما هم مثالِ مثبت و هم منفی باشد، بدان معناست این مثال‌ها دقیقاً همان توصیف را دارند، اما رده‌بندی‌های متفاوتی دارند. این می‌تواند رخ دهد، زیرا «نویز» (Noise) یا خطایی در داده هست؛ زیرا حوزه «ناجبرگرایانه» (Nondeterministic) است؛ یا زیرا نمی‌توانیم ویژگی‌ای را مشاهده کنیم که مثال‌ها را متمایز کند.

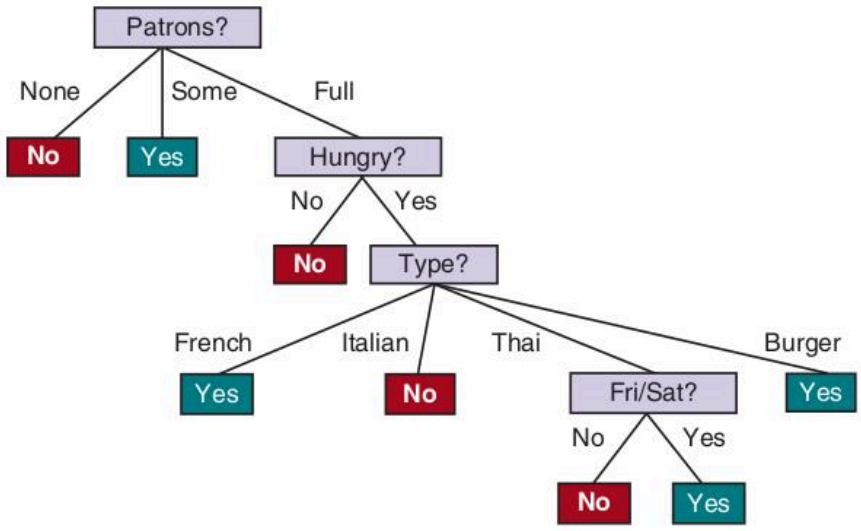
بهترین کاری که می‌توانیم بکنیم بازگرداندنِ رایج‌ترین مقدارِ خروجیِ مثال‌های باقی‌مانده است.

الگوریتمِ LEARN-DECISION-TREE در شکل ۱۹.۵ نشان داده شده. توجه کنید مجموعهٔ مثال‌ها ورودیِ الگوریتم است، اما هیچ‌جا مثال‌ها در درختِ بازگردانده‌شده توسط الگوریتم ظاهر نمی‌شوند. درخت از آزمون‌های روی ویژگی‌ها در گره‌های داخلی، مقادیرِ ویژگی روی شاخه‌ها و مقادیرِ خروجی روی گره‌های برگ تشکیل شده. جزئیاتِ تابعِ

IMPORTANCE در بخش ۱۹.۳.۳ داده شده. خروجیِ الگوریتمِ یادگیری روی مجموعهٔ آموزشیِ نمونه‌مان در شکل ۱۹.۶ نشان داده شده.

```
function LEARN-DECISION-TREE(examples, attributes, parent_examples) returns a tree

  if examples is empty then return PLURALITY-VALUE(parent_examples)
  else if all examples have the same classification then return the classification
  else if attributes is empty then return PLURALITY-VALUE(examples)
  else
      A ← argmax_{a ∈ attributes} IMPORTANCE(a, examples)
      tree ← a new decision tree with root test A
      for each value v of A do
          exs ← {e : e ∈ examples and e.A = v}
          subtree ← LEARN-DECISION-TREE(exs, attributes − A, examples)
          add a branch to tree with label (A = v) and subtree subtree
      return tree
```

شکل ۱۹.۵ — الگوریتمِ یادگیریِ درختِ تصمیم. تابعِ IMPORTANCE در بخش ۱۹.۳.۳ توصیف شده. تابعِ PLURALITY-VALUE رایج‌ترین مقدارِ خروجی را میانِ مجموعه‌ای از مثال‌ها انتخاب می‌کند و تساوی‌ها را تصادفی می‌شکند.

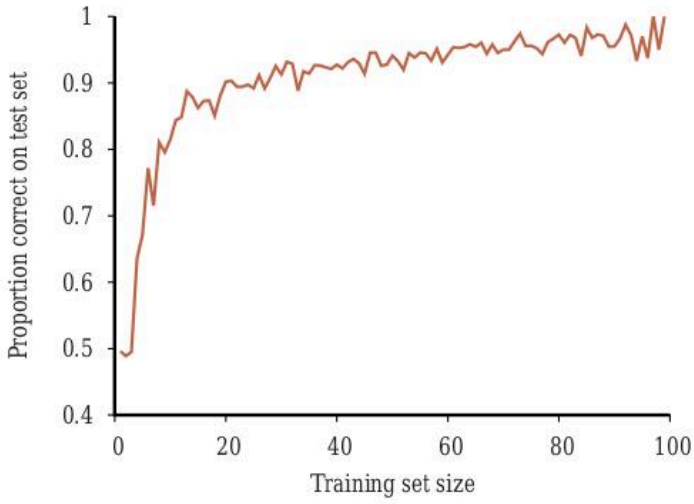


شکل ۱۹.۶ — درختِ تصمیمِ استقراشده از مجموعهٔ آموزشیِ ۱۲-مثاله. (ریشه: Patrons؛ شاخهٔ None → No؛ شاخهٔ Some → Yes؛ شاخهٔ Hungry → Full؛ Hungry=No → No؛ Hungry=Yes → Fri/Sat؛ Fri/Sat=No → No؛ Fri/Sat=Yes → Type؛ Type=French → Yes؛ Type=Italian → No؛ Type=Thai → Yes؛ Type=Burger → No.)

درختِ یادگرفته‌شده به‌وضوح با درختِ اصلیِ نشان‌داده‌شده در شکل ۱۹.۳ متفاوت است. ممکن است نتیجه بگیریم الگوریتمِ یادگیری کارِ خیلی خوبی در یادگیریِ تابعِ درست انجام نمی‌دهد. این نتیجه‌گیریِ اشتباهی می‌بود. الگوریتمِ یادگیری به مثال‌ها نگاه می‌کند، نه به تابعِ درست، و در واقع، فرضیه‌اش (شکل ۱۹.۶ را ببینید) نه فقط با همهٔ مثال‌ها سازگار است، بلکه به‌طورِ قابلِ‌توجهی از درختِ اصلی ساده‌تر است! با مثال‌های کمی متفاوت درخت ممکن است بسیار متفاوت باشد، اما تابعی که بازنمایی می‌کند مشابه خواهد بود.

الگوریتمِ یادگیری دلیلی برای گنجاندنِ آزمون‌های Raining و Reservation ندارد، زیرا می‌تواند همهٔ مثال‌ها را بدونِ آن‌ها رده‌بندی کند. همچنین الگوی جالبی را که قبلاً مشکوک نبود تشخیص داده: SR برای غذای تایلندی در آخرِ هفته منتظر خواهد ماند. همچنین محکوم به اشتباه‌هایی برای مواردی است که هیچ مثالی ندیده. برای نمونه، هرگز موردی را ندیده که انتظار 0–10 دقیقه باشد اما رستوران پر باشد. در آن صورت می‌گوید وقتی Hungry کاذب است منتظر نمان، اما SR قطعاً منتظر می‌ماند. با مثال‌های آموزشیِ بیشتر برنامهٔ یادگیری می‌توانست این اشتباه را تصحیح کند.

می‌توانیم عملکردِ الگوریتمِ یادگیری را با «منحنیِ یادگیری» (Learning Curve) ارزیابی کنیم، چنان‌که در شکل ۱۹.۷ نشان داده شده. برای این شکل، ۱۰۰ مثال در اختیار داریم، که به‌صورتِ تصادفی به مجموعهٔ آموزشی و مجموعهٔ آزمون تقسیم می‌کنیم. با مجموعهٔ آموزشی فرضیهٔ h را یاد می‌گیریم و دقتش را با مجموعهٔ آزمون اندازه می‌گیریم. می‌توانیم این کار را با شروع از مجموعهٔ آموزشیِ اندازهٔ 1 و افزایشِ یکی‌یکی تا اندازهٔ 99 انجام دهیم. برای هر اندازه، در واقع فرایندِ تقسیمِ تصادفی به مجموعه‌های آموزشی و آزمون را ۲۰ بار تکرار و نتایجِ ۲۰ آزمایش را میانگین می‌کنیم. منحنی نشان می‌دهد با رشدِ اندازهٔ مجموعهٔ آموزشی، دقت افزایش می‌یابد. (به همین دلیل، منحنی‌های یادگیری «نمودارهای شاد» (Happy Graphs) نیز نامیده می‌شوند.) در این نمودار به دقتِ ۹۵٪ می‌رسیم، و به نظر می‌رسد منحنی ممکن است با دادهٔ بیشتر همچنان افزایش یابد.



شکل ۱۹.۷ – منحنیِ یادگیری برای الگوریتمِ یادگیریِ درختِ تصمیم روی ۱۰۰ مثالِ تصادفی‌تولیدشده در حوزهٔ رستوران. هر نقطهٔ داده میانگینِ ۲۰ آزمایش است.

۱۹.۳.۳ انتخابِ آزمون‌های ویژگی

الگوریتمِ یادگیریِ درختِ تصمیم ویژگیِ با بالاترین IMPORTANCE را انتخاب می‌کند. حالا نشان می‌دهیم چگونه اهمیت را اندازه بگیریم، با استفاده از مفهومِ «بهرهٔ اطلاعاتی» (Information Gain)، که بر حسبِ «آنتروپی» (Entropy) تعریف می‌شود، که کمیتِ بنیادین در نظریهٔ اطلاعات است (شنون و ویور، 1949). آنتروپی معیاری از عدم قطعیتِ متغیرِ تصادفی است؛ هرچه اطلاعات بیشتر، آنتروپی کمتر. متغیرِ تصادفی با فقط یک مقدارِ ممکن—سکه‌ای که همیشه رو می‌آید—هیچ عدم قطعیتی ندارد و بنابراین آنتروپیش صفر تعریف می‌شود. سکهٔ منصفانه هنگام پرتاب به‌یکسان محتمل است رو یا پشت بیاید، و به‌زودی نشان خواهیم داد این به‌عنوانِ «۱ بیت» آنتروپی حساب می‌شود.

پرتابِ تاسِ چهاروجهیِ منصفانه ۲ بیت آنتروپی دارد، زیرا 2^2 انتخابِ هم‌احتمال هست. حالا سکهٔ ناعادلانه‌ای را در نظر بگیرید که ۹۹٪ مواقع رو می‌آید. به‌صورتِ شهودی، این سکه عدم قطعیتِ کمتری از سکهٔ منصفانه دارد—اگر رو را حدس بزنیم فقط ۱٪ مواقع اشتباه می‌کنیم—پس می‌خواهیم اندازهٔ آنتروپیش نزدیکِ صفر، اما مثبت باشد. به‌طورِ کلی، آنتروپیِ متغیرِ تصادفیِ V با مقادیرِ vk با احتمالِ P(vk) چنین تعریف می‌شود:

$$H(V) = \sum_k P(vk) \log_2(1/P(vk)) = -\sum_k P(vk) \log_2 P(vk)$$

می‌توانیم بررسی کنیم آنتروپیِ پرتابِ سکهٔ منصفانه در واقع ۱ بیت است:

$$H(Fair) = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$$

در مسائلِ رده‌بندیِ بولی، خروجی‌ها فقط صادق یا کاذب‌اند، پس می‌توانیم آنتروپی را بر حسبِ نسبتِ مثال‌های مثبت، $p/(p+n)$، بیان کنیم، که در آن p و n شمارِ مثال‌های مثبت و منفی‌اند. با تعریفِ $B(q) = -(q \log_2 q + (1 - q) \log_2(1 - q))$، آنتروپیِ مثال‌های آموزشی $B(p/(p+n))$ است. برای نمونه، اگر ۴ مثالِ مثبت و ۸ مثالِ منفی در مجموعهٔ آموزشی باشند، آنتروپی $B(1/3) \approx 0.918$ است.

ویژگیِ A با d مقدارِ متمایز، مجموعهٔ آموزشیِ E را به زیرمجموعه‌های E1,...,Ed تقسیم می‌کند. هر زیرمجموعهٔ Ek دارای pk مثالِ مثبت و nk مثالِ منفی است، پس اگر از آن شاخه برویم، برای پاسخ‌دادن به پرسش به $B(pk/(pk+nk))$ بیتِ اطلاعاتِ اضافی نیاز خواهیم داشت. مثالِ تصادفیِ انتخابی از مجموعهٔ آموزشی با احتمالِ $(pk+nk)/(p+n)$ در Ek است، پس آنتروپیِ موردانتظارِ باقی‌مانده پس از آزمونِ ویژگیِ A برابر:

$$Remainder(A) = \sum_{k=1}^{d} \frac{pk+nk}{p+n} B(pk/(pk+nk))$$

بهرهٔ اطلاعاتی از آزمونِ ویژگیِ A کاهشِ موردانتظارِ آنتروپی است:

$$Gain(A) = B(p/(p+n)) - Remainder(A)$$

در واقع Gain(A) دقیقاً همان چیزی است که برای پیاده‌سازیِ تابعِ IMPORTANCE نیاز داریم. با بازگشت به ویژگی‌های درنظرگرفته‌شده در شکل ۱۹.۴، داریم:

$$Gain(Patrons) = 1 - [(2/12)B(0/2) + (4/12)B(4/4) + (6/12)B(2/6)] \approx 0.541 \text{ بیت}$$
$$Gain(Type) = 1 - [(2/12)B(1/2) + (2/12)B(1/2) + (4/12)B(2/4) + (4/12)B(2/4)] = 0 \text{ بیت}$$

که شهودِ ما را تأیید می‌کند که Patrons ویژگیِ بهتری برای تقسیمِ اول است. در واقع، Patrons بیشینهٔ بهرهٔ اطلاعاتیِ هر یک از ویژگی‌ها را دارد و بنابراین توسط الگوریتمِ یادگیریِ درختِ تصمیم به‌عنوانِ ریشه انتخاب می‌شود.

۱۹.۳.۴ تعمیم و بیش‌برازش

می‌خواهیم الگوریتم‌های یادگیری‌مان فرضیه‌ای بیابند که دادهٔ آموزشی را برازش کند، اما مهم‌تر، می‌خواهیم برای دادهٔ قبلاً دیده‌نشده خوب تعمیم دهد. در شکل ۱۹.۱ دیدیم چندجمله‌ایِ درجهٔ بالا می‌تواند همهٔ داده را برازش کند، اما نوسان‌های وحشی دارد که توسط داده توجیه نمی‌شوند: برازش می‌کند اما می‌تواند بیش‌برازش کند. بیش‌برازش با رشدِ شمارِ ویژگی‌ها محتمل‌تر و با افزایشِ شمارِ مثال‌های آموزشی کم‌محتمل‌تر می‌شود. فضاهای فرضیهٔ بزرگ‌تر (مثلاً درخت‌های تصمیم با گره‌های بیشتر یا چندجمله‌ای‌های با درجهٔ بالا) ظرفیتِ بیشتری هم برای برازش و هم برای بیش‌برازش دارند؛ برخی رده‌های مدل بیشتر از بقیه مستعدِ بیش‌برازش‌اند. برای درخت‌های تصمیم، فنِ «هرسِ درختِ تصمیم» (Decision Tree Pruning) با بیش‌برازش مبارزه می‌کند. هرس با حذفِ گره‌هایی کار می‌کند که به‌وضوح مربوط نیستند. با درختِ کامل، چنان‌که توسط LEARN-DECISION-TREE تولید شده، شروع می‌کنیم. سپس به گرهٔ آزمونی نگاه می‌کنیم که فقط گره‌های برگ را به‌عنوانِ فرزندان دارد. اگر آزمون به نظر نامربوط می‌رسد—فقط نویزِ داده را تشخیص می‌دهد—آن‌گاه آزمون را حذف می‌کنیم و آن را با گرهٔ برگ جایگزین می‌کنیم. این فرایند را تکرار می‌کنیم، هر آزمون با فقط فرزندانِ برگ را در نظر می‌گیریم، تا وقتی هر یک یا هرس شده یا همان‌گونه پذیرفته شود.

پرسش این است که چگونه تشخیص دهیم گره‌ای ویژگیِ نامربوطی را آزمایش می‌کند؟ فرض کنیم در گره‌ای با p مثالِ مثبت و n مثالِ منفی هستیم. اگر ویژگی نامربوط باشد، انتظار داریم مثال‌ها را به زیرمجموعه‌هایی تقسیم کند به‌گونه‌ای که هر زیرمجموعه تقریباً همان نسبتِ مثالِ مثبتِ کلِ مجموعه، p/(p+n)، را داشته باشد، و بنابراین بهرهٔ اطلاعاتی نزدیکِ صفر خواهد بود.[3] بدین‌ترتیب، بهرهٔ اطلاعاتیِ کم سرنخِ خوبی است که ویژگی نامربوط است. حالا پرسش این است که برای تقسیم بر ویژگیِ خاصی چقدر بهره لازم است؟

می‌توانیم با «آزمونِ معناداریِ آماری» (Significance Test) پاسخ دهیم. چنین آزمونی با فرضِ اینکه الگوی زیربنایی‌ای نیست (به اصطلاح «فرضیهٔ صفر» (Null Hypothesis)) شروع می‌کند. سپس دادهٔ واقعی برای محاسبهٔ میزانِ انحرافش از نبودِ کاملِ الگو تحلیل می‌شود. اگر درجهٔ انحراف از نظرِ آماری نامحتمل باشد (معمولاً به معنای احتمالِ ۵٪ یا کمتر)، آن‌گاه آن شاهدِ خوبی برای حضورِ الگوی معنادار در داده تلقی می‌شود. احتمال‌ها از توزیع‌های استانداردِ میزانِ انحرافِ موردانتظار در نمونه‌برداریِ تصادفی محاسبه می‌شوند.

در این مورد، فرضیهٔ صفر این است که ویژگی نامربوط است و از این رو بهرهٔ اطلاعاتی برای نمونهٔ بی‌نهایت‌بزرگ صفر خواهد بود. باید احتمالِ اینکه، تحتِ فرضیهٔ صفر، نمونه‌ای با اندازهٔ v = n + p انحرافِ مشاهده‌شده از توزیعِ موردانتظارِ مثال‌های مثبت و منفی را نشان دهد محاسبه کنیم. می‌توانیم انحراف را با مقایسهٔ شمارِ واقعیِ مثال‌های مثبت و منفی در هر زیرمجموعه، pk و nk، با شمارِ موردانتظار، $\hat{p}k$ و $\hat{n}k$، با فرضِ نامربوطیِ واقعی، اندازه بگیریم:

$$\hat{p}k = p \times (pk+nk)/(p+n) \qquad \hat{n}k = n \times (pk+nk)/(p+n)$$

اندازهٔ مناسبی از انحرافِ کل با $\Delta = \sum_{k=1}^{d} [(pk-\hat{p}k)^2/\hat{p}k + (nk-\hat{n}k)^2/\hat{n}k]$ داده می‌شود. تحتِ فرضیهٔ صفر، مقدارِ Δ طبقِ «توزیعِ کای‌دو» (χ^2) با d-1 درجهٔ آزادی توزیع می‌شود. می‌توانیم از تابعِ آمارِ χ^2 ببینیم آیا مقدارِ خاصِ Δ فرضیهٔ صفر را تأیید یا رد می‌کند. برای نمونه، ویژگیِ Type رستوران را در نظر بگیرید، با چهار مقدار و بنابراین سه درجهٔ

آزادی. مقدارِ $\Delta = 7.82$ یا بیشتر فرضیهٔ صفر را در سطحِ ۵٪ رد می‌کند (و مقدارِ $\Delta = 11.35$ یا بیشتر در سطحِ ۱٪ رد می‌کند). مقادیرِ پایین‌تر به پذیرشِ فرضیهٔ نامربوط‌بودنِ ویژگی می‌انجامد، و بنابراین شاخهٔ مرتبطِ درخت باید هرس شود. این «هرسِ χ^2 (Pruning χ^2)» شناخته می‌شود.

با هرس، نویزِ مثال‌ها قابلِ تحمل می‌شود. خطاهای برچسبِ مثال (مثلاً مثالی (x,Yes) که باید (x,No) باشد) افزایشِ خطی در خطایِ پیش‌بینی می‌دهند، در حالی که خطاهای توصیف‌های مثال (مثلاً $=Price وقتی در واقع $$=Price بود) اثرِ مجانبی دارند که با کوچک‌شدنِ درخت به مجموعه‌های کوچک‌تر بدتر می‌شود. درخت‌های هرس‌شده وقتی داده حاوی نویزِ زیادی است به‌طورِ قابلِ‌توجهی بهتر از درخت‌های هرس‌نشده عمل می‌کنند. همچنین، درخت‌های هرس‌شده اغلب بسیار کوچک‌ترند و از این رو آسان‌تر فهمیده می‌شوند و کارآمدتر اجرا می‌شوند. یک هشدارِ نهایی: ممکن است فکر کنید هرسِ χ^2 و بهرهٔ اطلاعاتی شبیه هم به نظر می‌رسند، پس چرا آن‌ها را با رویکردی به نامِ «توقفِ زودهنگام» (Early Stopping) ترکیب نکنیم—درختِ تصمیم وقتی ویژگیِ خوبی برای تقسیم نیست گره تولید نکند، به‌جای اینکه زحمتِ تولیدِ گره‌ها و سپس هرس‌کردنشان را بکشد. مسئلهٔ توقفِ زودهنگام این است که ما را از بازشناسیِ موقعیت‌هایی بازمی‌دارد که هیچ ویژگیِ خوبِ تکی‌ای نیست، اما ترکیب‌هایی از ویژگی‌ها هستند که اطلاعاتی‌اند. برای نمونه، تابعِ XOR دو ویژگیِ دودویی را در نظر بگیرید. اگر شمارِ تقریباً برابری از مثال‌ها برای هر چهار ترکیبِ مقادیرِ ورودی باشد، هیچ‌کدام از ویژگی‌ها اطلاعاتی نخواهد بود، با این حال کارِ درست تقسیم بر یکی از ویژگی‌هاست (مهم نیست کدام)، و سپس در سطحِ دوم تقسیم‌های بسیار اطلاعاتی خواهیم گرفت. توقفِ زودهنگام این را از دست می‌داد، اما تولید-و-سپس-هرس آن را درست مدیریت می‌کند.

۱۹.۳.۵ گسترشِ کاربردپذیریِ درخت‌های تصمیم

درخت‌های تصمیم را می‌توان با مدیریتِ پیچیدگی‌های زیر به‌طورِ گسترده‌تری مفید کرد:

- **دادهٔ مفقود:** در بسیاری از حوزه‌ها، همهٔ مقادیرِ ویژگی برای هر مثال معلوم نیستند. مقادیر ممکن است ثبت نشده باشند، یا ممکن است کسبشان گران باشد. این دو مسئله پدید می‌آورد: نخست، با درختِ تصمیمِ کامل، چگونه باید مثالی را رده‌بندی کرد که یکی از ویژگی‌های آزمون را ندارد؟ دوم، چگونه باید فرمولِ بهرهٔ اطلاعاتی را اصلاح کرد وقتی برخی مثال‌ها مقادیرِ ناشناخته‌ای برای ویژگی دارند؟ این پرسش‌ها در تمرین MISS.۱۹ پرداخته شده‌اند.
- **ویژگی‌های ورودیِ پیوسته و چندمقداره:** برای ویژگی‌های پیوسته مانندِ قد، وزن یا زمان، ممکن است هر مثال مقدارِ ویژگیِ متفاوتی داشته باشد. اندازهٔ بهرهٔ اطلاعاتی بالاترین امتیاز را به چنین ویژگی‌ای می‌داد، و به ما درختِ کم‌عمقی با این ویژگی در ریشه و زیردرخت‌های تک‌مثاله برای هر مقدارِ ممکن در زیرش می‌داد. اما وقتی مثالِ تازه‌ای با مقدارِ ویژگیِ دیده‌نشده برای رده‌بندی بگیریم کمکی نمی‌کند.

راهِ بهتر برای سروکارداشتن با مقادیرِ پیوسته «آزمونِ نقطهٔ تقسیم» (Split Point) است—آزمونِ نامساوی روی مقدارِ ویژگی. برای نمونه، در گره‌ای معین از درخت، ممکن است آزمونِ Weight > 160 بیشترین اطلاعات را بدهد. روش‌های کارآمدی برای یافتنِ نقاطِ تقسیمِ خوب وجود دارد: با مرتب‌سازیِ مقادیرِ ویژگی شروع کن، و سپس فقط نقاطِ تقسیمی

را در نظر بگیر که بینِ دو مثالِ مرتب‌شده با رده‌بندی‌های متفاوت‌اند، در حالی که جمع‌های جاریِ مثال‌های مثبت و منفی در هر طرفِ نقطهٔ تقسیم را پیگیری می‌کنی. تقسیم‌کردن گران‌ترین بخشِ کاربردهای واقعیِ یادگیریِ درختِ تصمیم است.

برای ویژگی‌هایی که پیوسته نیستند و ترتیبِ معناداری ندارند، اما شمارِ زیادی مقدارِ ممکن دارند (مانندِ کدپستی یا شمارهٔ کارتِ اعتباری)، اندازه‌ای به نامِ «نسبتِ بهرهٔ اطلاعاتی» (Information Gain Ratio)؛ تمرین GAIN.۱۹ را ببینید) می‌تواند برای اجتناب از تقسیم به زیردرخت‌های تک‌مثالهٔ بسیار استفاده شود. رویکردِ مفیدِ دیگر اجازه‌دادن به آزمونِ برابریِ شکلِ A = vk است. برای نمونه، آزمونِ Zipcode=10002 می‌تواند برای برگزیدنِ گروه بزرگی از افراد در این کدپستی در شهرِ نیویورک استفاده شود، و بقیه را در زیردرختِ «سایر» توده کند.

- **ویژگیِ خروجیِ پیوسته:** اگر می‌خواهیم مقدارِ خروجیِ عددی را پیش‌بینی کنیم، مانندِ قیمتِ آپارتمان، به «درختِ رگرسیون» (Regression Tree) نیاز داریم نه درختِ رده‌بندی. درختِ رگرسیون در هر برگ تابعِ خطی از زیرمجموعه‌ای از ویژگی‌های عددی دارد، نه مقدارِ خروجیِ تکی. برای نمونه، شاخهٔ آپارتمان‌های دوخوابه ممکن است با تابعِ خطی از متراژ و شمارِ حمام‌ها پایان یابد. الگوریتمِ یادگیری باید تصمیم بگیرد کی تقسیم را متوقف کند و رگرسیونِ خطی را (بخش ۱۹.۶ را ببینید) روی ویژگی‌ها آغاز کند. نامِ CART، که مخففِ «درخت‌های رده‌بندی و رگرسیون» است، برای پوششِ هر دو رده استفاده می‌شود.

سامانهٔ یادگیریِ درختِ تصمیم برای کاربردهای دنیایِ واقعی باید بتواند همهٔ این مسائل را مدیریت کند. مدیریتِ متغیرهای پیوسته‌مقدار به‌طورِ خاص مهم است، زیرا هم فرایندهای فیزیکی و هم مالی دادهٔ عددی فراهم می‌کنند. چند بستهٔ تجاری ساخته شده که این معیارها را برآورده می‌کنند، و برای توسعهٔ هزاران سامانهٔ مستقر استفاده شده‌اند. در بسیاری از حوزه‌های صنعت و تجارت، درخت‌های تصمیم نخستین روشی هستند که وقتی قرار است روشِ رده‌بندی از مجموعهٔ داده استخراج شود امتحان می‌شوند.

درخت‌های تصمیم چیزهای زیادی برایشان هست: سهولتِ فهم، مقیاس‌پذیری به مجموعه‌های دادهٔ بزرگ، و چندمنظوره‌گی در مدیریتِ ورودی‌های گسسته و پیوسته و نیز رده‌بندی و رگرسیون. اما می‌توانند دقتِ زیربهینه داشته باشند (عمدتاً به دلیلِ جستجوی حریصانه)، و اگر درخت‌ها بسیار عمیق باشند، گرفتنِ پیش‌بینی برای مثالِ تازه می‌تواند در زمانِ اجرا گران باشد. درخت‌های تصمیم همچنین «ناپایدار» (Unstable) هستند، به این معنا که افزودنِ فقط یک مثالِ تازه می‌تواند آزمونِ ریشه را تغییر دهد، که کلِ درخت را تغییر می‌دهد. در بخش ۱۹.۸.۲ خواهیم دید مدلِ جنگلِ تصادفی می‌تواند برخی از این مسائل را اصلاح کند.

۱۹.۴ انتخابِ مدل و بهینه‌سازی

هدفِ ما در یادگیریِ ماشینی انتخابِ فرضیه‌ای است که به‌صورتِ بهینه با مثال‌های آینده برازش شود. برای دقیق‌کردنِ این، باید «مثالِ آینده» و «برازشِ بهینه» را تعریف کنیم. نخست فرض می‌کنیم مثال‌های آینده مانندِ گذشته خواهند بود. این را «فرضِ ایستایی» (Stationarity) می‌نامیم؛ بدونِ آن، همه‌چیز از بین رفته. فرض می‌کنیم هر مثالِ Ej همان توزیعِ احتمالِ پیشینی را دارد:

```
... = P(Ej) = P(Ej+1) = P(Ej+2)
```

و مستقل از مثال‌های قبلی است:

```
P(Ej) = P(Ej | Ej−1, Ej−2, ...)
```

مثال‌هایی که این معادله‌ها را ارضا می‌کنند «مستقل و هم‌توزیع» یا i.i.d. هستند.

گامِ بعد تعریفِ «برازشِ بهینه» است. فعلاً می‌گوییم برازشِ بهینه فرضیه‌ای است که «نرخِ خطا» (Error Rate) را کمینه می‌کند: نسبتِ دفعاتی که $h(x) \neq y$ برای مثالی (x,y). (بعداً این را بسط می‌دهیم تا خطاهای متفاوت هزینه‌های متفاوتی داشته باشند، در واقع به پاسخ‌های «تقریباً» درست اعتبارِ جزئی بدهیم.) می‌توانیم نرخِ خطای فرضیه را با دادنِ آزمونی به آن برآورد کنیم: عملکردش را روی مجموعهٔ آزمونی از مثال‌ها اندازه بگیریم. برای فرضیه (یا دانش‌آموز) تقلب بود که قبل از آزمون به پاسخ‌های آزمون نگاه کند. ساده‌ترین راه برای تضمینِ این اتفاق‌نیفتادن، تقسیمِ مثال‌های موجود به دو مجموعه است: مجموعهٔ آموزشی برای ایجادِ فرضیه، و مجموعهٔ آزمون برای ارزیابیِ آن.

اگر فقط بخواهیم یک فرضیه بسازیم، این رویکرد کافی است. اما اغلب به ایجادِ چند فرضیه می‌انجامیم: ممکن است بخواهیم دو مدلِ یادگیریِ ماشینیِ کاملاً متفاوت را مقایسه کنیم، یا ممکن است بخواهیم «دکمه‌های» گوناگون درونِ یک مدل را تنظیم کنیم. برای نمونه، می‌توانیم آستانه‌های متفاوتی برای هرسِ χ^2 درخت‌های تصمیم، یا درجه‌های متفاوتی برای چندجمله‌ای‌ها امتحان کنیم. این «دکمه‌ها» را «ابرپارامتر» (Hyperparameters) می‌نامیم—پارامترهای ردهٔ مدل، نه مدلِ فردی.

فرض کنیم پژوهشگری برای یک تنظیمِ ابرپارامترِ هرسِ χ^2 فرضیه‌ای تولید کند، نرخ‌های خطا را روی مجموعهٔ آزمون اندازه بگیرد، و سپس ابرپارامترهای متفاوتی را امتحان کند. هیچ فرضیهٔ فردی به دادهٔ مجموعهٔ آزمون نگاه نکرده، اما کلِ فرایند، از طریقِ پژوهشگر، نگاه کرده. راهِ اجتناب از این، واقعاً کنارگذاشتنِ مجموعهٔ آزمون است—قفل‌کردنِ آن تا وقتی کاملاً با آموزش، آزمایش، تنظیمِ ابرپارامتر، بازآموزی و... تمام نشده‌اید. بدان معناست به سه مجموعهٔ داده نیاز دارید:

1. مجموعهٔ آموزشی برای آموزشِ مدل‌های نامزد.
2. مجموعهٔ اعتبارسنجی (Validation Set)، برای انتخابِ مدلِ نامزد.
3. مجموعهٔ آزمون (Test Set)، برای ارزیابیِ نهاییِ مدلِ انتخاب‌شده.

۱۹.۴.۱ انتخابِ مدل

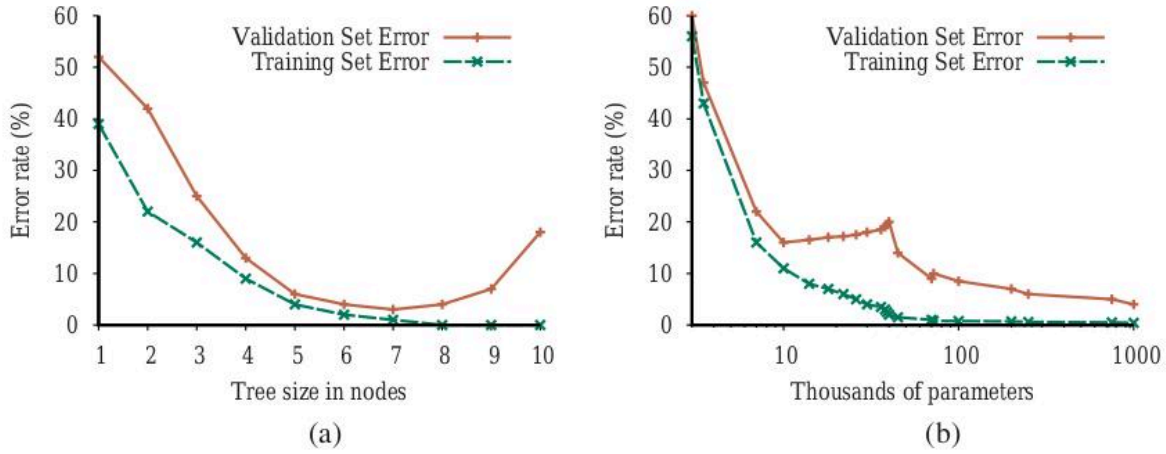
شکل ۱۹.۸ الگوریتمِ سادهٔ MODEL-SELECTION را توصیف می‌کند. الگوریتمِ یادگیری را به‌عنوانِ آرگومان می‌گیرد، Learner (برای نمونه، می‌تواند LEARN-DECISION-TREE باشد). Learner یک ابرپارامتر می‌گیرد، که در شکل size نامیده شده. برای درخت‌های تصمیم می‌تواند شمارِ گره‌های درخت باشد؛ برای چندجمله‌ای‌ها size درجه خواهد بود.

MODEL-SELECTION با کوچک‌ترین مقدارِ size شروع می‌کند، مدلِ ساده‌ای می‌دهد (که احتمالاً داده را کم‌برازش می‌کند) و از مقادیرِ بزرگ‌ترِ size تکرار می‌کند و مدل‌های پیچیده‌تر را در نظر می‌گیرد. در پایان، MODEL-SELECTION مدلی را انتخاب می‌کند که کمترین نرخِ خطایِ میانگین را روی دادهٔ اعتبارسنجیِ کنارگذاشته‌شده داشته باشد.

```
function MODEL-SELECTION(Learner, examples, k) returns a (hypothesis, error rate) pair

  err ← an array, indexed by size, storing validation-set error rates
  training_set, test_set ← a partition of examples into two sets
  for size = 1 to ∞ do
      err[size] ← CROSS-VALIDATION(Learner, size, training_set, k)
      if err is starting to increase significantly then
          best_size ← the value of size with minimum err[size]
          h ← Learner(best_size, training_set)
          return h, ERROR-RATE(h, test_set)

function CROSS-VALIDATION(Learner, size, examples, k) returns error rate

  N ← the number of examples
  errs ← 0
  for i = 1 to k do
    validation_set ← examples[(i − 1) × N/k:i × N/k]
    training_set ← examples − validation_set
    h ← Learner(size, training_set)
    errs ← errs + ERROR-RATE(h, validation_set)
  return errs / k        // average error rate on validation sets, across k-fold cross-validation
```

شکل ۱۹.۸ – الگوریتمِ انتخابِ مدل که مدلی با کمترین نرخِ خطای میانگین روی دادهٔ اعتبارسنجی برمی‌گزیند. (شبه‌کد: برای هر مقدارِ ابرپارامتر size از کمترین به بیشترین: مدلِ h را با LEARNER آموزش بده؛ میانگینِ نرخِ خطای h را روی همهٔ دادهٔ اعتبارسنجی محاسبه کن؛ به‌ترین مدل را نگه دار. در پایان، مدلِ دارایِ کمترین نرخِ خطایِ اعتبارسنجی را برگردان.)

در شکل ۱۹.۹ دو الگوی نمونه‌ای را می‌بینیم که در انتخابِ مدل رخ می‌دهند. در هر دو (الف) و (ب)، خطایِ مجموعهٔ آموزشی به‌صورتِ یکنواخت (با نوسانِ تصادفیِ جزیی) با افزایشِ پیچیدگیِ مدل کاهش می‌یابد. اما منحنیِ خطایِ اعتبارسنجی ابتدا کاهش می‌یابد و سپس افزایش، و در نهایت از منحنیِ آموزشی فراتر می‌رود. این افزایش در خطایِ اعتبارسنجی نشانهٔ بیش‌برازش است: با پیچیده‌ترشدنِ مدل، دادهٔ آموزشی را بهتر برازش می‌کند اما دیگر به نمونه‌های دیده‌نشده تعمیم نمی‌دهد.



شکل ۱۹.۹ – نرخ‌های خطا روی دادهٔ آموزشی (خطِ سبزِ پایینی) و دادهٔ اعتبارسنجی (خطِ نارنجیِ بالایی) برای مدل‌های با پیچیدگیِ متفاوت روی دو مسئلهٔ متفاوت. MODEL-SELECTION مقدارِ ابرپارامتر با کمترین خطایِ اعتبارسنجی را انتخاب می‌کند. در (الف) ردهٔ مدل درخت‌های تصمیم و ابرپارامتر شمارِ گره‌هاست. داده از نسخه‌ای از مسئلهٔ رستوران است. اندازهٔ بهینه 7 است. در (ب) ردهٔ مدل شبکه‌های عصبیِ پیچشی (بخش ۲۱.۳ را ببینید) و ابرپارامتر شمارِ پارامترهای مدل است. داده، مجموعهٔ MNIST از تصاویرِ رقم‌هاست؛ تکلیف شناساییِ هر رقم است. شمارِ بهینهٔ پارامترها 1,000,000 است (به مقیاسِ لگاریتمی توجه کنید).

چرا برخی منحنی‌های خطایِ اعتبارسنجی مانندِ (الف) و برخی مانندِ (ب) هستند؟ به نحوهٔ استفادهٔ رده‌های مدلِ متفاوت از ظرفیتِ مازاد، و تطابقِ آن با مسئلهٔ موردِ نظر، برمی‌گردد. با افزودنِ ظرفیت به ردهٔ مدل، اغلب به نقطه‌ای می‌رسیم که همهٔ مثال‌های آموزشی می‌توانند کاملاً درونِ مدل بازنمایی شوند. برای نمونه، با مجموعهٔ آموزشی با n مثالِ متمایز، همیشه درختِ تصمیمی با n گرهٔ برگ هست که می‌تواند همهٔ مثال‌ها را بازنمایی کند. می‌گوییم مدلی که دقیقاً همهٔ دادهٔ آموزشی را برازش می‌کند داده را «درون‌یابی» (Interpolated) کرده.[5] رده‌های مدل معمولاً با نزدیک‌شدنِ ظرفیت به نقطهٔ درون‌یابی شروع به بیش‌برازش می‌کنند. به نظر می‌رسد این به این دلیل است که بیشترِ ظرفیتِ مدل بر مثال‌های آموزشی متمرکز است، و ظرفیتِ باقی‌مانده به‌صورتِ نسبتاً تصادفی تخصیص می‌یابد به‌گونه‌ای که نمایندهٔ الگوهای مجموعهٔ دادهٔ اعتبارسنجی نیست. برخی رده‌های مدل هرگز از این بیش‌برازش بهبود نمی‌یابند، مانندِ درخت‌های تصمیم در (الف). اما برای رده‌های مدلِ دیگر، افزودنِ ظرفیت بدان معناست توابعِ نامزدِ بیشتری هست، و برخی از آن‌ها به‌طورِ طبیعی برای الگوهای داده‌ای که در تابعِ واقعیِ f(x) هستند مناسب‌اند. هرچه ظرفیت بالاتر باشد، این بازنمایی‌های مناسب بیشترند، و محتمل‌تر است سازوکارِ بهینه‌سازی بتواند روی یکی فرود آید. شبکه‌های عصبیِ عمیق (فصل ۲۱)، ماشین‌های هسته (بخش ۱۹.۷.۵)، جنگل‌های تصادفی (بخش ۱۹.۸.۲) و گروه‌های تقویت‌شده (بخش ۱۹.۸.۴) همگی این ویژگی را دارند که خطای مجموعهٔ اعتبارسنجی‌شان با افزایشِ ظرفیت تمایل به کاهش دارد، مانندِ شکل ۱۹.۹(ب).

می‌توانیم الگوریتمِ انتخابِ مدل را به راه‌های گوناگون بسط دهیم: می‌توانیم رده‌های مدلِ ناهمگون را مقایسه کنیم، با فراخوانیِ MODEL-SELECTION با DECISION-TREE-LEARNER به‌عنوانِ آرگومان و سپس با POLYNOMIAL-LEARNER، و دیدنِ اینکه کدام بهتر عمل می‌کند. می‌توانیم چند ابرپارامتر را اجازه دهیم، که بدان معناست به الگوریتمِ بهینه‌سازیِ پیچیده‌تری نیاز داریم، مانندِ جستجوی شبکه‌ای (Grid Search؛ بخش ۱۹.۹.۳ را ببینید) به‌جای جستجوی خطی.[4]

۱۹.۴.۲ از نرخ‌های خطا تا زیان

تاکنون سعی کرده‌ایم نرخِ خطا را کمینه کنیم. این بدیهی است بهتر از بیشینه‌کردنِ نرخِ خطاست، اما کلِ داستان نیست. مسئلهٔ رده‌بندیِ پیام‌های ایمیل به هرزنامه یا غیرهرزنامه را در نظر بگیرید. بدتر است غیرهرزنامه به‌عنوانِ هرزنامه رده‌بندی شود (و بنابراین احتمالاً پیامِ مهمی از دست برود) تا هرزنامه به‌عنوانِ غیرهرزنامه (و بنابراین چند ثانیه دلخوری بکشیم). پس رده‌بند با نرخِ خطایِ ۱٪، که تقریباً همهٔ خطاهایش رده‌بندیِ هرزنامه به‌عنوانِ غیرهرزنامه بود، بهتر از رده‌بندی با نرخِ خطایِ فقط ۰.۵٪ می‌بود، اگر بیشترِ آن خطاها رده‌بندیِ غیرهرزنامه به‌عنوانِ هرزنامه بودند. در فصل ۱۶ دیدیم تصمیم‌گیرندگان باید مطلوبیتِ موردانتظار را بیشینه کنند، و مطلوبیت چیزی است که یادگیرندگان نیز باید بیشینه کنند. با این حال، در یادگیریِ ماشینی سنت بر این است که آن را به‌صورتِ منفی بیان کنیم: به‌جای بیشینه‌کردنِ تابعِ مطلوبیت، «تابعِ زیان» (Loss Function) را کمینه کنیم. تابعِ زیانِ $L(x,y,\hat{y})$ به‌عنوانِ میزانِ مطلوبیتِ ازدست‌رفته با پیش‌بینیِ $h(x) = \hat{y}$ وقتی پاسخِ درست $f(x) = y$ است تعریف می‌شود:

```
L(x, y, ŷ) = Utility(نتیجهٔ استفاده از y با ورودیِ x) − Utility(نتیجهٔ استفاده از ŷ با ورودیِ x)
```

این عمومی‌ترین صورت‌بندیِ تابعِ زیان است. اغلب نسخهٔ ساده‌شدهٔ $L(y, \hat{y})$ استفاده می‌شود که مستقل از x است. برای بقیهٔ این فصل از نسخهٔ ساده‌شده استفاده خواهیم کرد، که بدان معناست نمی‌توانیم بگوییم بدتر است نامه‌ای از مامان را اشتباه رده‌بندی کنیم تا نامه‌ای از پسرعموی اعصاب‌خردکنِ‌مان، اما می‌توانیم بگوییم ۱۰ برابر بدتر است غیرهرزنامه را به‌عنوانِ هرزنامه رده‌بندی کنیم تا برعکس:

```
L(spam, nospam) = 1،    L(nospam, spam) = 10
```

توجه کنید $L(y,y)$ همیشه صفر است؛ طبقِ تعریف، وقتی دقیقاً درست حدس بزنید زیانی نیست.

برای توابع با خروجیِ گسسته، می‌توانیم مقدارِ زیان را برای هر رده‌بندیِ اشتباهِ ممکن برشماریم، اما نمی‌توانیم همهٔ امکان‌ها را برای دادهٔ حقیقی‌مقدار برشماریم. اگر $f(x)$ برابر 137.035999 باشد، با $h(x) = 137.036$ نسبتاً خوشحال می‌شویم، اما دقیقاً چقدر باید خوشحال باشیم؟ به‌طورِ کلی، خطاهای کوچک بهتر از خطاهای بزرگ‌اند؛ دو تابع که این ایده را پیاده‌سازی می‌کنند، قدرِ مطلقِ تفاوت (که «زیانِ L1» نامیده می‌شود) و مربعِ تفاوت (که «زیانِ L2» نامیده می‌شود؛ به «۲» برای مربع فکر کنید) هستند. برای خروجی‌های گسسته‌مقدار، اگر به ایدهٔ کمینه‌کردنِ نرخِ خطا راضی باشیم، می‌توانیم از تابعِ زیانِ L0/1 استفاده کنیم، که برای پاسخِ نادرست زیانِ 1 دارد:

```
زیانِ قدرمطلق:      |L1(y, ŷ) = |y − ŷ
زیانِ مربع‌خطا:     L2(y, ŷ) = (y − ŷ)²
زیانِ 0/1:          L0/1(y, ŷ) = 0 اگر ŷ = y، وگرنه 1
```

از نظرِ نظری، عاملِ یادگیری با انتخابِ فرضیه‌ای که «زیانِ تعمیم» (Generalization Loss) موردانتظار را روی همهٔ جفت‌های ورودی-خروجی که خواهد دید کمینه می‌کند، مطلوبیتِ موردانتظارش را بیشینه می‌کند. برای محاسبهٔ این انتظار باید توزیعِ احتمالِ پیشینیِ P(X,Y) را روی مثال‌ها تعریف کنیم. E مجموعهٔ همهٔ مثال‌های ورودی-خروجیِ ممکن باشد. سپس زیانِ تعمیمِ موردانتظار برای فرضیهٔ h (نسبت به تابعِ زیانِ L):

```
GenLossL(h) = ∑_{(x,y)∈E} L(y, h(x)) P(x, y)
```

و بهترین فرضیه، h*، فرضیه‌ای با کمینهٔ زیانِ تعمیمِ موردانتظار است:

```
h* = argmin_{h∈H} GenLossL(h)
```

چون P(x,y) در بیشترِ موارد معلوم نیست، عاملِ یادگیری فقط می‌تواند زیانِ تعمیم را با «زیانِ تجربی» (Empirical Loss) روی مجموعه‌ای از مثال‌های E با اندازهٔ N برآورد کند:

```
EmpLossL,E(h) = ∑_{(x,y)∈E} L(y, h(x)) × (1/N)
```

بهترین فرضیهٔ برآوردشدهٔ ĥ* سپس فرضیه‌ای با کمینهٔ زیانِ تجربی است:

```
ĥ* = argmin_{h∈H} EmpLossL,E(h)
```

چهار دلیل هست که ĥ* ممکن است با تابعِ واقعیِ f تفاوت داشته باشد: تحقق‌ناپذیری، واریانس، نویز و پیچیدگیِ محاسباتی. نخست، می‌گوییم مسئلهٔ یادگیری «تحقق‌پذیر» (Realizable) است اگر فضایِ فرضیهٔ H واقعاً حاویِ تابعِ واقعیِ f باشد. اگر H مجموعهٔ توابعِ خطی و تابعِ واقعیِ f تابعِ درجه‌دومی باشد، هیچ مقدارِ داده‌ای f واقعی را بازنخواهد یافت. دوم، واریانس بدان معناست الگوریتمِ یادگیری به‌طورِ کلی برای مجموعه‌های مثالِ متفاوت فرضیه‌های متفاوتی برمی‌گرداند. اگر مسئله تحقق‌پذیر باشد، واریانس با افزایشِ شمارِ مثال‌های آموزشی به سمتِ صفر کاهش می‌یابد. سوم، f ممکن است ناجبرگرایانه یا نویزی باشد—ممکن است برای همان x مقادیرِ متفاوتی از f(x) برگرداند. طبقِ تعریف، نویز را نمی‌توان پیش‌بینی کرد (فقط می‌توان مشخصه‌سازی کرد). و سرانجام، وقتی H تابعِ پیچیده‌ای در فضایِ فرضیهٔ بزرگ باشد، جستجوی نظام‌مندِ همهٔ امکان‌ها از نظرِ محاسباتی ناشدنی است؛ در آن صورت، جستجو می‌تواند بخشی از فضا را کاوش کند و فرضیهٔ نسبتاً خوبی برگرداند، اما نمی‌تواند همیشه بهترین را تضمین کند. روش‌های سنتی در آمار و سال‌های آغازینِ یادگیریِ ماشینی بر «یادگیریِ مقیاس‌کوچک» (Small-Scale Learning) تمرکز داشتند، جایی که شمارِ مثال‌های آموزشی از ده‌ها تا هزارانِ پایین متغیر بود. در این‌جا زیانِ تعمیم بیشتر از «خطایِ تقریبِ» نداشتنِ f واقعی در فضایِ فرضیه، و از «خطایِ برآوردِ» نداشتنِ مثال‌های آموزشیِ کافی برای محدودکردنِ واریانس می‌آید. در سال‌های اخیر تأکیدِ بیشتری بر «یادگیریِ مقیاس‌بزرگ» (Large-Scale Learning) با میلیون‌ها مثال بوده. در این‌جا زیانِ تعمیم ممکن است توسط محدودیت‌های محاسبه مسلط شود: داده و مدلِ به‌اندازهٔ کافی غنی هست که h بسیار نزدیکِ f واقعی بیابیم، اما محاسبهٔ یافتنِ آن پیچیده است، پس به تقریبی رضایت می‌دهیم.

۱۹.۴.۳ منظم‌سازی

در بخش ۱۹.۴.۱ دیدیم چگونه با اعتبارسنجیِ متقاطع انتخابِ مدل انجام دهیم. رویکردِ جایگزین جستجو برای فرضیه‌ای است که مستقیماً جمعِ وزن‌دارِ زیانِ تجربی و پیچیدگیِ فرضیه را کمینه کند، که آن را «هزینهٔ کل» می‌نامیم:

```
Cost(h) = EmpLoss(h) + λ·Complexity(h)
ĥ* = argmin_{h∈H} Cost(h)
```

در این‌جا λ ابرپارامتری است، عددی مثبت که به‌عنوانِ نرخِ تبدیل بینِ زیان و پیچیدگیِ فرضیه عمل می‌کند. اگر λ خوب انتخاب شود، به‌خوبی زیانِ تجربیِ تابعِ ساده را در برابرِ گرایشِ تابعِ پیچیده به بیش‌برازش متوازن می‌کند.

این فرایندِ جریمه‌کردنِ صریحِ فرضیه‌های پیچیده «منظم‌سازی» (Regularization) نامیده می‌شود: به دنبال توابعی هستیم که «منظم‌تر»ند. توجه کنید حالا دو انتخاب می‌کنیم: تابعِ زیان (L1 یا L2)، و اندازهٔ پیچیدگی، که «تابعِ منظم‌سازی» (Regularization Function) نامیده می‌شود. انتخابِ تابعِ منظم‌سازی به فضایِ فرضیه بستگی دارد. برای نمونه، برای چندجمله‌ای‌ها، تابعِ منظم‌سازیِ خوب، جمعِ مربع‌های ضرایب است—کوچک‌نگه‌داشتنِ جمع ما را از چندجمله‌ایِ پرنوسانِ درجهٔ 12 در شکل ۱۹.۱ دور می‌کرد. نمونه‌ای از این نوع منظم‌سازی را در بخش ۱۹.۶.۳ نشان خواهیم داد.

راهِ دیگرِ ساده‌کردنِ مدل‌ها کاهشِ بُعدهایی است که مدل‌ها با آن‌ها کار می‌کنند. فرایندِ «انتخابِ ویژگی» (Feature Selection) را می‌توان برای دورانداختنِ ویژگی‌هایی که به نظر نامربوط‌اند انجام داد. هرسِ χ^2 نوعی انتخابِ ویژگی است.

در واقع ممکن است زیانِ تجربی و پیچیدگی را روی همان مقیاس اندازه گرفت، بدونِ عاملِ تبدیلِ λ: هر دو را می‌توان در بیت اندازه گرفت. نخست فرضیه را به‌عنوانِ برنامهٔ ماشینِ تورینگ رمزگذاری کن و شمارِ بیت‌ها را بشمار. سپس شمارِ بیت‌های لازم برای رمزگذاریِ داده را بشمار، که در آن مثالِ درست‌پیش‌بینی‌شده صفر بیت هزینه دارد و هزینهٔ مثالِ نادرست‌پیش‌بینی‌شده بستگی به بزرگیِ خطا دارد. فرضیهٔ «حداقلِ طولِ توصیف» (Minimum Description Length) یا MDL، شمارِ کلِ بیت‌های لازم را کمینه می‌کند. این در حدِّ نهایی خوب کار می‌کند، اما برای مسائلِ کوچک‌تر، انتخابِ رمزگذاریِ برنامه—بهترین روشِ رمزگذاریِ درختِ تصمیم به‌صورتِ رشتهٔ بیتی—بر پیامد اثر می‌گذارد. در فصل ۲۰ (صفحهٔ ۷۲۴)، تفسیرِ احتمالاتیِ رویکردِ MDL را توصیف می‌کنیم.

۱۹.۴.۴ تنظیمِ ابرپارامتر

در بخش ۱۹.۴.۱ نشان دادیم چگونه بهترین مقدارِ ابرپارامترِ size را با اعمالِ اعتبارسنجیِ متقاطع روی هر مقدارِ ممکن تا وقتی نرخِ خطایِ اعتبارسنجی افزایش یابد انتخاب کنیم. وقتی ابرپارامترِ تکی با شمارِ کمی مقدارِ ممکن هست، رویکردِ خوبی است. اما وقتی چند ابرپارامتر هست، یا وقتی مقادیرِ پیوسته دارند، انتخابِ مقادیرِ خوب دشوارتر است.

ساده‌ترین رویکرد برای تنظیمِ ابرپارامتر «تنظیمِ دستی» (Hand-Tuning) است: بر اساسِ تجربهٔ گذشته مقادیرِ پارامتری را حدس بزن، مدل را آموزش بده، عملکردش را روی دادهٔ اعتبارسنجی اندازه بگیر، نتایج را تحلیل کن، و از شهودت برای پیشنهادِ مقادیرِ پارامتریِ تازه استفاده کن. تا وقتی عملکردِ رضایت‌بخشی داری (یا وقت، بودجهٔ محاسباتی یا حوصله‌ات تمام شود) تکرار کن.

اگر فقط چند ابرپارامتر، هر یک با شمارِ کمی مقدارِ ممکن، باشد، رویکردِ نظام‌مندتری به نامِ «جستجوی شبکه‌ای» (Grid Search) مناسب است: همهٔ ترکیب‌های مقادیر را امتحان کن و ببین کدام روی دادهٔ اعتبارسنجی بهترین عملکرد را دارد. ترکیب‌های متفاوت را می‌توان به‌صورتِ موازی روی ماشین‌های متفاوت اجرا کرد، پس اگر منابعِ محاسباتیِ کافی داشته باشید، این لازم نیست کند باشد، هرچند در برخی موارد انتخابِ مدل شناخته شده که منابعِ خوشه‌های هزاررایانه‌ای را روزها در یک زمان می‌بلعد.

استراتژی‌های جستجو از فصل‌های ۳ و ۴ نیز می‌توانند به کار آیند. برای نمونه، اگر دو ابرپارامتر مستقل از یکدیگر باشند، می‌توان آن‌ها را جداگانه بهینه کرد. اگر ترکیب‌های زیادی از مقادیرِ ممکن باشد، «جستجوی تصادفی» (Random Search) به‌صورتِ یکنواخت از مجموعهٔ همهٔ تنظیم‌های ابرپارامترِ ممکن نمونه‌برداری می‌کند، و تا وقتی مایلید وقت و منابعِ محاسباتی صرف کنید تکرار می‌کند. نمونه‌برداریِ تصادفی همچنین راهِ خوبی برای مدیریتِ مقادیرِ پیوسته است.

وقتی هر اجرایِ آموزشی زمانِ زیادی می‌برد، گرفتنِ اطلاعاتِ مفید از هر اجرا می‌تواند کمک‌کننده باشد. «بهینه‌سازیِ بیزی» (Bayesian Optimization) تکلیفِ انتخابِ مقادیرِ ابرپارامترِ خوب را خودش به‌عنوانِ مسئلهٔ یادگیریِ ماشینی در نظر می‌گیرد. یعنی بردارِ مقادیرِ ابرپارامترِ x را ورودی، و کلِ زیانِ روی مجموعهٔ اعتبارسنجی برای مدلی که با آن ابرپارامترها ساخته شده را خروجیِ y در نظر بگیر؛ سپس می‌خواهیم تابعِ $y = f(x)$ را بیابیم که زیانِ y را کمینه می‌کند. هر بار که اجرایِ آموزشی انجام می‌دهیم جفتِ $(y, f(x))$ تازه‌ای می‌گیریم، که می‌توانیم برای به‌روزرسانیِ باورمان دربارهٔ شکلِ تابعِ f استفاده کنیم.

ایده این است که بهره‌برداری (انتخابِ مقادیرِ پارامترِ نزدیک به نتیجهٔ خوبِ قبلی) را با کاوش (امتحانِ مقادیرِ پارامترِ نوآورانه) معاوضه کنیم. این همان معاوضه‌ای است که در جستجوی درختیِ مونت‌کارلو دیدیم (بخش ۵.۴)، و در واقع ایدهٔ کران‌های بالایِ اطمینان نیز این‌جا برای کمینه‌کردنِ پشیمانی استفاده می‌شود. اگر فرض کنیم f را می‌توان با «فرایندِ گاوسی» تقریب زد، ریاضیاتِ به‌روزرسانیِ باورمان دربارهٔ f به‌خوبی کار می‌کند.

۱۹.۵ نظریهٔ یادگیری

چگونه می‌توانیم مطمئن باشیم فرضیهٔ یادگرفته‌شده برای ورودی‌های قبلاً دیده‌نشده خوب پیش‌بینی می‌کند؟ یعنی، چگونه بدانیم فرضیهٔ h به تابعِ هدفِ f نزدیک است، اگر ندانیم f چیست؟ این پرسش‌ها قرن‌هاست موردِ تأملِ آوکام، هیوم و دیگران بوده. در دهه‌های اخیر، پرسش‌های دیگری پدید آمده‌اند: برای گرفتنِ h خوب به چند مثال نیاز داریم؟ از چه فضایِ فرضیه‌ای باید استفاده کنیم؟ اگر فضایِ فرضیه بسیار پیچیده باشد، اصلاً می‌توانیم بهترین h را بیابیم، یا باید به بیشینهٔ محلی رضایت دهیم؟ h چقدر باید پیچیده باشد؟ چگونه از بیش‌برازش اجتناب کنیم؟ این بخش این پرسش‌ها را بررسی می‌کند.

با پرسشِ تعدادِ مثال‌های لازم برای یادگیری شروع می‌کنیم. از منحنیِ یادگیریِ یادگیریِ درختِ تصمیم روی مسئلهٔ رستوران (شکل ۱۹.۷ صفحهٔ ۶۶۱) دیدیم دقت با دادهٔ آموزشیِ بیشتر بهبود می‌یابد. منحنی‌های یادگیری مفیدند، اما خاصِ الگوریتمِ یادگیریِ خاصی روی مسئلهٔ خاصی‌اند. آیا اصولِ عمومی‌تری بر شمارِ مثال‌های لازم حاکم است؟

پرسش‌هایی مانندِ این توسط «نظریهٔ یادگیریِ محاسباتی» (Computational Learning Theory) پرداخته می‌شوند، که در تقاطعِ هوش مصنوعی، آمار و علومِ رایانهٔ نظری قرار دارد. اصلِ زیربنایی این است که هر فرضیه‌ای که به‌طورِ جدی غلط است تقریباً مطمئناً با احتمالِ بالا پس از شمارِ کمی مثال «برملا» می‌شود، زیرا پیش‌بینیِ نادرستی خواهد کرد. بدین‌ترتیب، هر فرضیه‌ای که با مجموعهٔ به‌اندازهٔ کافی بزرگِ مثال‌های آموزشی سازگار باشد بعید است به‌طورِ جدی غلط باشد: یعنی باید «احتمالاً تقریباً درست» (Probably Approximately Correct یا PAC) باشد. هر الگوریتمِ یادگیری که فرضیه‌هایی برمی‌گرداند که احتمالاً تقریباً درست‌اند، «الگوریتمِ یادگیریِ PAC» نامیده می‌شود؛ می‌توانیم از این رویکرد برای فراهم‌کردنِ کران‌هایی بر عملکردِ PAC الگوریتم‌های یادگیریِ گوناگون استفاده کنیم.

قضیه‌های PAC-learning، مانندِ همهٔ قضیه‌ها، پیامدهای منطقیِ اصل‌های‌اند. وقتی قضیه‌ای (در برابرِ، مثلاً، مفسرِ سیاسی) بر اساسِ گذشته چیزی دربارهٔ آینده بیان می‌کند، اصل‌ها باید «سوختِ» برقراریِ آن ارتباط را فراهم کنند. برای یادگیریِ PAC، سوخت توسط فرضِ ایستاییِ معرفی‌شده در صفحهٔ ۶۶۵ فراهم می‌شود، که می‌گوید مثال‌های آینده از همان توزیعِ ثابتِ $P(E) = P(X,Y)$ که مثال‌های گذشته کشیده خواهند شد. (توجه کنید لازم نیست بدانیم آن توزیع چیست، فقط اینکه تغییر نمی‌کند.) علاوه بر این، برای ساده‌نگه‌داشتنِ امور، فرض می‌کنیم تابعِ واقعیِ f قطعی و عضوی از فضایِ فرضیهٔ H موردِ بررسی است.

ساده‌ترین قضیه‌های PAC با توابعِ بولی سروکار دارند، که زیانِ 0/1 برایشان مناسب است. نرخِ خطای فرضیهٔ h، که قبلاً به‌صورتِ غیررسمی تعریف شد، در این‌جا به‌صورتِ صوری به‌عنوانِ زیانِ تعمیمِ موردانتظار برای مثال‌های کشیده‌شده از توزیعِ ایستا تعریف می‌شود:

```
error(h) = GenLoss_{L0/1}(h) = ∑_{x,y} L0/1(y, h(x)) P(x, y)
```

به بیانِ دیگر، error(h) احتمالِ اینکه h مثالِ تازه‌ای را اشتباه رده‌بندی کند است. این همان کمیتی است که به‌صورتِ تجربی توسط منحنی‌های یادگیریِ نشان‌داده‌شدهٔ قبلی اندازه‌گیری می‌شود.

فرضیهٔ h «تقریباً درست» (Approximately Correct) نامیده می‌شود اگر $error(h) \leq \varepsilon$، که در آن ε ثابتِ کوچکی است. نشان خواهیم داد می‌توانیم N را بیابیم به‌گونه‌ای که، پس از آموزش روی N مثال، با احتمالِ بالا، همهٔ فرضیه‌های سازگار تقریباً درست‌اند. می‌توان فرضیهٔ تقریباً درست را «نزدیک» به تابعِ واقعی در فضایِ فرضیه دانست: درونِ چیزی قرار دارد که «گویِ (ε-Ball) ε» دورِ تابعِ واقعیِ f نامیده می‌شود. فضایِ فرضیهٔ بیرونِ این گوی Hbad نامیده می‌شود. می‌توانیم کرانی بر احتمالِ اینکه فرضیهٔ «به‌طورِ جدی غلطِ» $hb \in Hbad$ با N مثالِ نخست سازگار باشد چنین اشتقاق کنیم. می‌دانیم $error(hb) > \varepsilon$. بدین‌ترتیب، احتمالِ موافقتِ آن با مثالِ معین حداکثر $1-\varepsilon$ است. چون مثال‌ها مستقل‌اند، کران برای N مثال:

```
P(hb با N مثال موافق است) ≤ (1−ε)^N
```

احتمالِ اینکه Hbad حداقل یک فرضیهٔ سازگار داشته باشد با جمعِ احتمال‌های فردی کران‌دار می‌شود:

```
P(Hbad حاوی فرضیهٔ سازگاری است) ≤ |H|(1−ϱ)^N ≥ |Hbad|(1−ϱ)^N
```

که در آن از این واقعیت استفاده کرده‌ایم Hbad زیرمجموعهٔ H است و بنابراین |Hbad| ≤ |H|. می‌خواهیم احتمالِ این رخداد را زیرِ عددِ کوچکِ δ کاهش دهیم:

```
P(Hbad حاوی فرضیهٔ سازگاری است) ≤ δ ≥ |H|(1−ϱ)^N
```

با توجه به اینکه 1-(ϱ-)^e ≤ ϱ، می‌توانیم این را به دست آوریم اگر به الگوریتم اجازه دهیم ببیند:

```
N ≥ (1/ϱ) [ln(1/δ) + ln|H|]        (19.1)
```

مثال. بدین‌ترتیب، با احتمالِ دستِ‌کم 1-δ، پس از دیدنِ این تعدادِ مثال، الگوریتمِ یادگیری فرضیه‌ای برمی‌گرداند که خطایش حداکثر ϱ است. به بیانِ دیگر، احتمالاً تقریباً درست است. شمارِ مثال‌های لازم، به‌عنوانِ تابعی از ϱ و δ، «پیچیدگیِ نمونه‌ای» (Sample Complexity) الگوریتمِ یادگیری نامیده می‌شود.

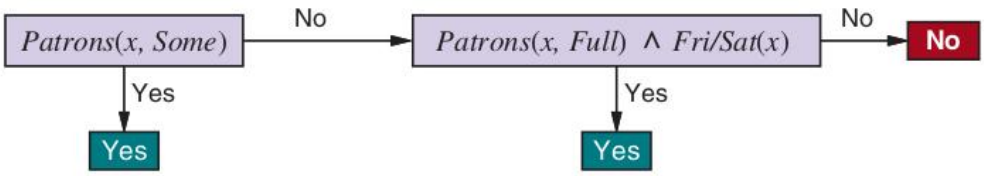
چنان‌که قبلاً دیدیم، اگر H مجموعهٔ همهٔ توابعِ بولی روی n ویژگی باشد، آن‌گاه |H| = 2^(2^n). بدین‌ترتیب، پیچیدگیِ نمونه‌ایِ فضا به‌صورتِ 2^n رشد می‌کند. چون شمارِ مثال‌های ممکن نیز 2^n است، این پیشنهاد می‌کند PAC-یادگیری در ردهٔ همهٔ توابعِ بولی مستلزم دیدنِ همه، یا تقریباً همه، مثال‌های ممکن است. لحظه‌ای فکر دلیلش را آشکار می‌کند: H به‌اندازهٔ کافی فرضیه دارد که هر مجموعهٔ معینی از مثال‌ها را به همهٔ راه‌های ممکن رده‌بندی کند. به‌طورِ خاص، برای هر مجموعهٔ N مثالی، مجموعهٔ فرضیه‌های سازگار با آن مثال‌ها شاملِ شمارِ برابری از فرضیه‌هایی است که xN+1 را مثبت پیش‌بینی می‌کنند و فرضیه‌هایی که xN+1 را منفی پیش‌بینی می‌کنند.

پس برای به‌دست‌آوردنِ تعمیمِ واقعی به مثال‌های دیده‌نشده، به نظر می‌رسد باید فضایِ فرضیهٔ H را به نحوی محدود کنیم؛ اما البته، اگر فضا را محدود کنیم، ممکن است تابعِ واقعی را کلاً حذف کنیم. سه راه برای گریز از این دوراهی هست. نخستین، به‌کاربردنِ دانشِ قبلی در مسئله است. دومین، که در بخش ۱۹.۴.۳ معرفی کردیم، اصرار بر اینکه الگوریتم نه فقط هر فرضیهٔ سازگاری، بلکه ترجیحاً فرضیهٔ ساده‌ای برگرداند (چنان‌که در یادگیریِ درختِ تصمیم انجام می‌شود). در مواردی که یافتنِ فرضیه‌های سازگارِ ساده شدنی است، نتایجِ پیچیدگیِ نمونه‌ای به‌طورِ کلی بهتر از تحلیل‌های مبتنی بر فقط سازگاری‌اند. سومین، که در ادامه دنبالش می‌کنیم، تمرکز بر زیرمجموعه‌های قابلِ‌یادگیری از کلِ فضایِ فرضیهٔ توابعِ بولی است. این رویکرد بر این فرض تکیه می‌کند که فضایِ فرضیهٔ محدودشده حاویِ فرضیهٔ h است که به‌اندازهٔ کافی به تابعِ واقعیِ f نزدیک است؛ مزایا این‌اند که فضایِ فرضیهٔ محدودشده تعمیمِ مؤثر را اجازه می‌دهد و معمولاً جستجو در آن آسان‌تر است. حالا یکی از این فضاهای فرضیهٔ محدودشده را با جزئیاتِ بیشتر بررسی می‌کنیم.

۱۹.۵.۱ نمونهٔ یادگیریِ PAC: یادگیریِ فهرست‌های تصمیم

حالا نشان می‌دهیم چگونه PAC یادگیری را روی فضایِ فرضیهٔ تازه‌ای اعمال کنیم: «فهرست‌های تصمیم» (Decision Lists). فهرستِ تصمیم شاملِ توالیِ آزمون‌هاست، که هر یک عطفی از لیترال‌هاست. اگر آزمونی وقتی روی توصیفِ مثالی اعمال شود موفق شود، فهرستِ تصمیم مقداری را که باید برگردانده شود مشخص می‌کند. اگر آزمون شکست بخورد، پردازش با آزمونِ بعدیِ فهرست ادامه می‌یابد. فهرست‌های تصمیم شبیهِ درخت‌های تصمیم‌اند، اما ساختارِ کلی‌شان ساده‌تر است: فقط در یک جهت شاخه می‌شوند. در مقابل، آزمون‌های فردی پیچیده‌ترند. شکل ۱۹.۱۰ فهرستِ تصمیمی را نشان می‌دهد که فرضیهٔ زیر را بازنمایی می‌کند:

```
WillWait ⇔ (Patrons = Some) ∨ (Patrons = Full ∧ Fri/Sat)
```



شکل ۱۹.۱۰ — فهرستِ تصمیم برای مسئلهٔ رستوران. (آزمون‌ها: Yes ← Patrons(x, Some)؛ Yes ← Patrons(x, Full) ∧ Fri/Sat(x)؛ در غیرِ این صورت ← No.)

اگر آزمون‌هایی با اندازهٔ دلخواه اجازه دهیم، فهرست‌های تصمیم می‌توانند هر تابعِ بولی را بازنمایی کنند (تمرین DLEX.۱۹). از سوی دیگر، اگر اندازهٔ هر آزمون را به حداکثر k لیترال محدود کنیم، ممکن است الگوریتمِ یادگیری با موفقیت از شمارِ کمی مثال تعمیم دهد. از نمادِ k-DL برای فهرستِ تصمیم با حداکثر k عطف استفاده می‌کنیم. مثالِ شکل ۱۹.۱۰ در 2-DL است. نشان‌دادنِ اینکه k-DL به‌عنوانِ زیرمجموعه شاملِ k-DT، مجموعهٔ همهٔ درخت‌های تصمیمِ با عمقِ حداکثر k، است آسان است (تمرین DLEX.۱۹). از نمادِ k-DL(n) برای k-DL با n ویژگیِ بولی استفاده می‌کنیم.

نخستین تکلیف نشان‌دادنِ قابلِ‌یادگیری‌بودنِ k-DL است—یعنی هر تابعی در k-DL را می‌توان پس از آموزش روی شمارِ معقولی مثال دقیقاً تقریب زد. برای این کار، باید شمارِ فرضیه‌های ممکن را محاسبه کنیم. مجموعهٔ عطف‌های با حداکثر k لیترال با استفاده از n ویژگی را Conj(n,k) بگذاریم. چون فهرستِ تصمیم از آزمون‌ها ساخته می‌شود، و چون هر آزمون می‌تواند به پیامدِ Yes یا No متصل شود یا می‌تواند در فهرستِ تصمیم غایب باشد، حداکثر |Conj(n,k)|^3 مجموعهٔ متمایز از آزمون‌های مؤلفه‌ای هست. هر یک از این مجموعه‌های آزمون می‌تواند در هر ترتیبی باشد، پس:

```
|k-DL(n)| ≤ 3^c · c!
```

که در آن |c = |Conj(n,k)|. شمارِ عطف‌های با حداکثر k لیترال از n ویژگی با:

```
|Conj(n,k)| = ∑_{i=0}^{k} C(2n, i) = O(n^k)
```

داده می‌شود. از این رو، پس از کمی کار:

```
|k-DL(n)| = 2^(O(n^k log2(n^k)))
```

به دست می‌آوریم. می‌توانیم این را در معادلهٔ (19.1) جای کنیم تا نشان دهیم شمارِ مثال‌های لازم برای PAC-یادگیریِ تابعِ k-DL(n) در n چندجمله‌ای است:

```
N ≥ (1/ϱ) [ln(1/δ) + O(n^k log2(n^k))]
```

بنابراین، هر الگوریتمی که فهرستِ تصمیمِ سازگاری برمی‌گرداند، تابعِ k-DL را در شمارِ معقولی مثال، برای k کوچک، PAC-یاد می‌گیرد.

تکلیفِ بعدی یافتنِ الگوریتمِ کارآمدی است که فهرستِ تصمیمِ سازگار برمی‌گرداند. از الگوریتمِ حریصانه‌ای به نامِ DECISION-LIST-LEARNING استفاده می‌کنیم که مکرراً آزمونی را می‌یابد که دقیقاً با زیرمجموعه‌ای از مجموعهٔ آموزشی موافق است. وقتی چنین آزمونی یافت، آن را به فهرستِ تصمیمِ درحالِ‌ساخت می‌افزاید و مثال‌های متناظر را حذف می‌کند. سپس بقیهٔ فهرستِ تصمیم را با فقط مثال‌های باقی‌مانده می‌سازد. این تا وقتی تکرار می‌شود که مثالی باقی نماند. الگوریتم در شکل ۱۹.۱۱ نشان داده شده.

```
function DECISION-LIST-LEARNING(examples) returns a decision list, or failure
                if examples is empty then return the trivial decision list No
        t ← a test that agrees with a nonempty subset examplest of examples
   such that the members of examplest are all positive or all negative
                                          if no such t then return failure
              if examples in examplest are positive then o ← Yes else o ← No
return a decision list with initial test t and outcome o and remaining tests
                  given by DECISION-LIST-LEARNING(examples − examplest)
```

شکل ۱۹.۱۱ — الگوریتمی برای یادگیریِ فهرست‌های تصمیم.

این الگوریتم روشِ انتخابِ آزمونِ بعدی برای افزودن به فهرستِ تصمیم را مشخص نمی‌کند. اگرچه نتایجِ صوریِ داده‌شدهٔ قبلی به روشِ انتخاب بستگی ندارند، معقول به نظر می‌رسد آزمون‌های کوچکی را ترجیح دهیم که با مجموعه‌های بزرگِ مثال‌های یکنواخت‌رده‌بندی‌شده موافق‌اند، تا فهرستِ تصمیمِ کلی تا آن‌جا که ممکن است فشرده باشد. ساده‌ترین استراتژی یافتنِ کوچک‌ترین آزمونِ t است که با هر زیرمجموعهٔ یکنواخت‌رده‌بندی‌شده موافق است، صرف‌نظر از اندازهٔ زیرمجموعه. حتی این رویکرد نیز به‌خوبی کار می‌کند، چنان‌که شکل ۱۹.۱۲ پیشنهاد می‌کند. برای این مسئله، درختِ تصمیم کمی سریع‌تر از فهرستِ تصمیم یاد می‌گیرد، اما تنوعِ بیشتری دارد.

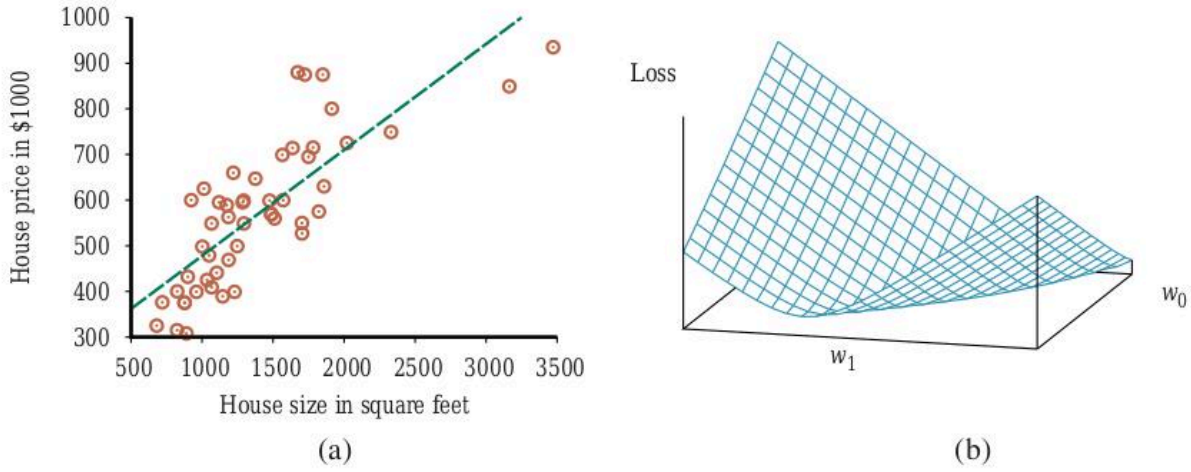
شکل ۱۹.۱۲ — منحنیِ یادگیری برای الگوریتمِ DECISION-LIST-LEARNING روی دادهٔ رستوران. منحنیِ LEARN-DECISION-TREE برای مقایسه نشان داده شده؛ درخت‌های تصمیم در این مسئلهٔ خاص کمی بهتر عمل می‌کنند. (هر دو روش پس از ۱۰۰ آزمایش بیش از ۹۰٪ دقت دارند.)

۱۹.۶ رگرسیون و رده‌بندیِ خطی

حالا وقت آن است که از درخت‌ها و فهرست‌های تصمیم به فضایِ فرضیهٔ متفاوتی برویم، فضایی که صدها سال استفاده شده: ردهٔ «توابعِ خطیِ» (Linear Functions) ورودی‌های پیوسته‌مقدار. با ساده‌ترین حالت شروع می‌کنیم: رگرسیون با تابعِ خطیِ تک‌متغیره، که به‌گونه‌ای دیگر «برازشِ خطِ راست» شناخته می‌شود. بخش ۱۹.۶.۳ حالتِ چندمتغیره را پوشش می‌دهد. بخش‌های ۱۹.۶.۴ و ۱۹.۶.۵ نشان می‌دهند چگونه توابعِ خطی را با اعمالِ آستانه‌های سخت و نرم به رده‌بند تبدیل کنیم.

۱۹.۶.۱ رگرسیونِ خطیِ تک‌متغیره

تابعِ خطیِ تک‌متغیره (خطِ راست) با ورودیِ x و خروجیِ y شکلِ $y = w_1x + w_0$ دارد، که در آن w_0 و w_1 ضرایبِ حقیقی‌مقداری هستند که باید یاد گرفته شوند. از حرفِ w استفاده می‌کنیم، زیرا ضرایب را «وزن» (Weight) می‌دانیم؛ مقدارِ y با تغییرِ وزنِ نسبیِ یک جمله یا جملهٔ دیگر تغییر می‌کند. w را بردارِ $\langle w_0, w_1 \rangle$ تعریف می‌کنیم و تابعِ خطی با آن وزن‌ها را $h_w(x) = w_1x + w_0$ تعریف می‌کنیم.



شکل ۱۹.۱۳ — (الف) نمونه‌ای از مجموعهٔ آموزشی در صفحهٔ (x,y)؛ هر نقطه نمایانگرِ اندازه (به فوتِ مربع) و قیمتِ خانه‌ای برای فروش است. (ب) نمودارِ تابعِ زیان برای مقادیرِ مختلفِ w0 و w1؛ توجه کنید تابعِ زیان محدب است و یک کمینهٔ سراسریِ تکی دارد.

شکل ۱۹.۱۳(الف) نمونه‌ای از مجموعهٔ آموزشیِ n نقطه‌ای در صفحهٔ x,y را نشان می‌دهد، که هر نقطه اندازه و قیمتِ خانه‌ای برای فروش را بازنمایی می‌کند. تکلیفِ یافتنِ hw که بهترین برازش را به این داده‌ها می‌دهد «رگرسیونِ خطی» (Linear Regression) نامیده می‌شود. برای برازشِ خط به داده، تنها کاری که باید بکنیم یافتنِ مقادیرِ وزن‌های ⟨w0,w1⟩ است که زیانِ تجربی را کمینه می‌کنند. سنت (که به گاوس[6] برمی‌گردد) استفاده از تابعِ زیانِ مربع‌خطا، L2، جمع‌شده روی همهٔ مثال‌های آموزشی است:

```
Loss(hw) = ∑_{j=1}^{N} L2(yj, hw(xj)) = ∑_{j=1}^{N} (yj − hw(xj))² = ∑_{j=1}^{N} (yj − (w1xj + w0))²
```

می‌خواهیم w* = argmin_w Loss(hw) را بیابیم. جمعِ ∑_{j=1}^{N}(yj - (w1xj + w0))² وقتی کمینه می‌شود که مشتق‌های جزئی‌اش نسبت به w0 و w1 صفر باشند:

```
∂/∂w0 ∑_{j=1}^{N} (yj − (w1xj + w0))² = 0        و        ∂/∂w1 ∑_{j=1}^{N} (yj − (w1xj + w0))² = 0        (19.2)
```

این معادله‌ها راه‌حلِ یکتا دارند:

```
w1 = (N(∑xjyj) − (∑xj)(∑yj)) / (N(∑xj²) − (∑xj)²)؛        w0 = (∑yj − w1(∑xj))/N
(19.3)
```

برای مثالِ شکل ۱۹.۱۳(الف)، راه‌حل w0 = 246، w1 = 0.232 است، و خطِ با آن وزن‌ها به‌صورتِ خطِ خط‌چین در شکل نشان داده شده:

```
y = 0.232x + 246
```

بسیاری از شکل‌های یادگیری شاملِ تنظیمِ وزن‌ها برای کمینه‌کردنِ زیان‌اند، پس کمک می‌کند تصویرِ ذهنی از آنچه در «فضایِ وزن» (Weight Space)—فضایِ تعریف‌شده توسط همهٔ تنظیم‌های ممکنِ وزن‌ها—رخ می‌دهد داشته باشیم. برای رگرسیونِ خطیِ تک‌متغیره، فضایِ وزنِ تعریف‌شده توسط w0 و w1 دوبعدی است، پس می‌توانیم زیان را به‌عنوانِ تابعی از w0 و w1 در نمودارِ سه‌بعدی رسم کنیم (شکل ۱۹.۱۳(ب) را ببینید). می‌بینیم تابعِ زیان «محدب» (Convex) است، چنان‌که در صفحهٔ ۱۲۲ تعریف شد؛ این برای هر مسئلهٔ رگرسیونِ خطی با تابعِ زیانِ L2 صادق است، و دلالت دارد هیچ کمینهٔ محلی‌ای نیست. از جهتی این پایانِ داستانِ مدل‌های خطی است؛ اگر لازم باشد خط‌ها را به داده برازش کنیم، معادلهٔ (19.3) را اعمال می‌کنیم.[7]

۱۹.۶.۲ نزولِ گرادیان

مدلِ خطیِ تک‌متغیره این ویژگیِ خوب را دارد که فرمولِ بسته‌ای برای یافتنِ کمینهٔ زیان وجود دارد. اما در بسیاری از مدل‌های یادگیری چنین نیست. به همین دلیل، به الگوریتمِ بهینه‌سازیِ عمومی‌تری نیاز داریم: «نزولِ گرادیان» (Gradient Descent). نزولِ گرادیان یکی از عمومی‌ترین و پرکاربردترین الگوریتم‌های بهینه‌سازی در یادگیریِ ماشین است. ایدهٔ پایه ساده است: با نقطه‌ای دلخواه در فضایِ وزن شروع کن، سپس بارها و بارها گام‌هایی در جهتِ «شیبِ نزولیِ» تابعِ زیان بردار—یعنی در جهتی که زیان را بیشترین کاهش می‌دهد—تا به کمینهٔ سراسری (یا محلی) برسیم. الگوریتم چنین است:

```
w ← هر نقطه در فضایِ پارامتر
تا وقتی همگرا نشده:
    برای هر wi در w:
wi ← wi − α (∂/∂wi) Loss(w)        (19.4)
```

پارامترِ α، که در بخش ۴.۲ «اندازهٔ گام» نامیدیم، معمولاً وقتی می‌خواهیم در مسئلهٔ یادگیری زیان را کمینه کنیم «نرخِ یادگیری» (Learning Rate) نامیده می‌شود. می‌تواند ثابتِ ثابتی باشد، یا می‌تواند با پیشرویِ فرایندِ یادگیری در طولِ زمان فروپاشد.

برای رگرسیونِ تک‌متغیره، زیان درجه‌دومی است، پس مشتقِ جزئی خطی خواهد بود. (تنها حسابانی که باید بدانید «قاعدهٔ زنجیره‌ای» (Chain Rule) است: ∂g(f(x))/∂x = g'(f(x))·∂f(x)/∂x، به‌علاوهٔ این واقعیت‌ها که ∂x²/∂x = 2x∂ و ∂x/∂x = 1.) نخست مشتق‌های جزئی—شیب‌ها—را در حالتِ ساده‌شدهٔ فقط یک مثالِ آموزشی، (x,y)، به‌دست می‌آوریم:

```
wi Loss(w) = ∂/∂wi (y − hw(x))²∂/∂
wi (y − hw(x))∂/∂ × (y − hw(x))2 =
wi (y − (w1x + w0))        (19.5)∂/∂ × (y − hw(x))2 =
```

با اعمالِ این بر هر دو w0 و w1 می‌گیریم:

```
w1 Loss(w) = −2(y − hw(x)) × x∂/∂        ;w0 Loss(w) = −2(y − hw(x))∂/∂
```

با جای‌کردنِ این در معادلهٔ (19.4)، و ادغامِ 2 در نرخِ یادگیریِ مشخص‌نشدهٔ α، قاعدهٔ یادگیریِ زیر را برای وزن‌ها می‌گیریم:

```
w0 ← w0 + α(y − hw(x))        ;w1 ← w1 + α(y − hw(x)) × x
```

این به‌روزرسانی‌ها معنیِ شهودی دارند: اگر hw(x) > y (یعنی خروجی بیش‌ازحد بزرگ است)، w0 را کمی کاهش بده، و w1 را اگر x ورودیِ مثبت بود کاهش بده اما اگر x ورودیِ منفی بود افزایش بده.

معادله‌های پیشین یک مثالِ آموزشی را پوشش می‌دهند. برای N مثالِ آموزشی، می‌خواهیم جمعِ زیان‌های فردیِ هر مثال را کمینه کنیم. مشتقِ جمع، جمعِ مشتق‌هاست، پس داریم:

```
w0 ← w0 + α ∑j (yj − hw(xj))        ;w1 ← w1 + α ∑j (yj − hw(xj)) × xj
```

این به‌روزرسانی‌ها «قاعدهٔ یادگیریِ نزولِ گرادیانِ دسته‌ای» (Batch Gradient Descent) را برای رگرسیونِ خطیِ تک‌متغیره تشکیل می‌دهند (که «نزولِ گرادیانِ قطعی» نیز نامیده می‌شود). سطحِ زیان محدب است، بدان معنا که هیچ کمینهٔ محلی‌ای برای گیرکردن نیست، و همگرایی به کمینهٔ سراسری تضمین‌شده است (تا زمانی که α را چنان بزرگ انتخاب نکنیم که از هدف بپرد)، اما ممکن است بسیار کند باشد: برای هر گام باید روی همهٔ N مثالِ آموزشی جمع بزنیم، و ممکن است گام‌های بسیاری باشد. مسئله اگر N بزرگ‌تر از اندازهٔ حافظهٔ پردازنده باشد تشدید می‌شود. گامی که همهٔ مثال‌های آموزشی را پوشش می‌دهد «دوره» (Epoch) نامیده می‌شود.

گونهٔ سریع‌تری «نزولِ گرادیانِ تصادفی» (Stochastic Gradient Descent) یا SGD نامیده می‌شود: در هر گام شمارِ کمی مثالِ آموزشی را به‌صورتِ تصادفی انتخاب می‌کند و طبقِ معادلهٔ (19.5) به‌روزرسانی می‌کند. نسخهٔ اصلیِ SGD فقط یک مثالِ آموزشی برای هر گام انتخاب می‌کرد، اما اکنون رایج‌تر است «ریزدسته‌ای» (Minibatch) از m مثال از میانِ N انتخاب شود. فرض کنیم N = 10,000 مثال و ریزدسته‌ای با اندازهٔ m = 100 انتخاب کنیم. سپس در هر گام مقدارِ محاسبه را با ضریبِ 100 کاهش داده‌ایم؛ اما چون خطایِ استانداردِ میانگینِ گرادیانِ برآوردشده متناسب با جذرِ شمارِ مثال‌هاست، خطایِ استاندارد فقط با ضریبِ 10 افزایش می‌یابد. پس حتی اگر مجبور باشیم ۱۰ برابر گامِ بیشتر قبل از همگرایی برداریم، ریزدسته‌ایِ SGD در این مورد ۱۰ برابر سریع‌تر از SGD دسته‌ایِ کامل است.

با برخی معماری‌های CPU یا GPU، می‌توانیم m را برای بهره‌گیری از عملیاتِ برداریِ موازی انتخاب کنیم، و گامی با m مثال تقریباً به‌اندازهٔ گامی با فقط یک مثال سریع شود. درونِ این قیدها، m را ابرپارامتری می‌دانیم که باید برای هر مسئلهٔ یادگیری تنظیم شود. همگراییِ SGD ریزدسته‌ای به‌صورتِ سخت‌گیرانه تضمین‌شده نیست؛ می‌تواند دورِ کمینه نوسان کند بدونِ آرام‌گرفتن. در صفحهٔ ۶۸۴ خواهیم دید برنامه‌ای برای کاهشِ نرخِ یادگیریِ α (مانندِ بازپختِ شبیه‌سازی‌شده) چگونه همگرایی را تضمین می‌کند.

SGD می‌تواند در محیطِ برخط مفید باشد، جایی که داده‌های تازه یکی‌یکی می‌آیند و فرضِ ایستایی ممکن است برقرار نباشد. (در واقع، SGD به‌عنوانِ «نزولِ گرادیانِ برخط» (Online Gradient Descent) نیز شناخته می‌شود.) با انتخابِ خوبِ α، مدل به‌آرامی تکامل می‌یابد، برخی از آنچه در گذشته آموخته را به یاد می‌آورد، اما با تغییرهای بازنمایی‌شده

توسط دادهٔ تازه نیز سازگار می‌شود.

SGD به‌طورِ گسترده روی مدل‌هایی غیر از رگرسیونِ خطی اعمال می‌شود، به‌طورِ خاص شبکه‌های عصبی. حتی وقتی سطحِ زیان محدب نیست، این رویکرد در یافتنِ کمینه‌های محلیِ خوبِ نزدیک به کمینهٔ سراسری مؤثر ثابت شده.

۱۹.۶.۳ رگرسیونِ خطیِ چندمتغیره

می‌توانیم به‌آسانی به مسائلِ رگرسیونِ خطیِ چندمتغیره بسط دهیم، که در آن هر مثالِ xj بردارِ n-عنصری است.[8] فضایِ فرضیه‌مان مجموعهٔ توابعِ شکلِ:

```
hw(xj) = w0 + w1xj,1 + ... + wnxj,n = w0 + ∑i wixj,i
```

است. (خواننده ممکن است بخواهد برای خلاصه‌ای از جبرِ خطی به پیوستِ الف مراجعه کند. همچنین توجه کنید از اصطلاحِ «رگرسیونِ چندمتغیره» برای این معنا استفاده می‌کنیم که ورودی بردارِ چند مقدار است، اما خروجی متغیرِ تکی است. از اصطلاحِ «رگرسیونِ چندمتغیره‌خروجی» برای حالتی استفاده می‌کنیم که خروجی نیز بردارِ چند متغیر است. با این حال، نویسندگانِ دیگر دو اصطلاح را به‌جای هم استفاده می‌کنند.)

جملهٔ w0، عرض از مبدأ، به‌عنوانِ متفاوت از بقیه برجسته است. می‌توانیم آن را با اختراعِ ویژگیِ ورودیِ دامیِ xj,0، که همیشه برابرِ 1 تعریف می‌شود، اصلاح کنیم. سپس h صرفاً ضربِ نقطه‌ایِ وزن‌ها و بردارِ ورودی است (یا به‌طورِ معادل، ضربِ ماتریسیِ ترانهادهٔ وزن‌ها و بردارِ ورودی):

```
hw(xj) = w·xj = wᵀxj = ∑i wixj,i
```

بهترین بردارِ وزن‌ها، w*، زیانِ مربع‌خطا را روی مثال‌ها کمینه می‌کند:

```
w* = argmin_w ∑j L2(yj, w·xj)
```

رگرسیونِ خطیِ چندمتغیره در واقع خیلی پیچیده‌تر از حالتِ تک‌متغیره‌ای که پوشش دادیم نیست. نزولِ گرادیان به کمینهٔ (یکتای) تابعِ زیان خواهد رسید؛ معادلهٔ به‌روزرسانی برای هر وزنِ wi:

```
wi ← wi + α ∑j (yj − hw(xj)) × xj,i          (19.6)
```

با ابزارهای جبرِ خطی و حسابِ برداری، همچنین می‌توان به‌صورتِ تحلیلی برای wiای که زیان را کمینه می‌کند حل کرد. y بردارِ خروجی‌های مثال‌های آموزشی و X «ماتریسِ داده» (Data Matrix)—یعنی ماتریسِ ورودی‌ها با یک مثالِ n-بعدی در هر ردیف—باشد. سپس بردارِ خروجی‌های پیش‌بینی‌شده ŷ = Xw است و زیانِ مربع‌خطا روی همهٔ دادهٔ آموزشی:

```
L(w) = ‖ŷ − y‖² = ‖Xw − y‖²
```

گرادیان را صفر می‌گذاریم:

```
∇w L(w) = 2Xᵀ(Xw − y) = 0
```

با بازآرایی، درمی‌یابیم بردارِ وزنِ کمینه-زیان چنین داده می‌شود:

```
w* = (XᵀX)⁻¹Xᵀy        (19.7)
```

عبارتِ $X^{T-}(X^TX)^{1}$ را «شبه‌معکوسِ» (Pseudoinverse) ماتریسِ داده، و معادلهٔ (19.7) را «معادلهٔ نرمال» (Normal Equation) می‌نامیم.

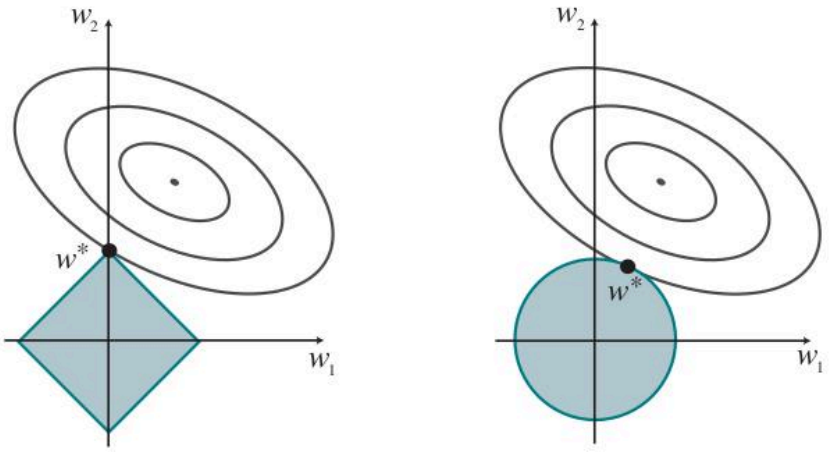
با رگرسیونِ خطیِ تک‌متغیره مجبور نبودیم نگرانِ بیش‌برازش باشیم. اما با رگرسیونِ خطیِ چندمتغیره در فضاهایِ پربعد ممکن است بُعدی که در واقع نامربوط است به‌طورِ اتفاقی مفید به نظر برسد و به بیش‌برازش بینجامد. بدین‌ترتیب، استفاده از منظم‌سازی روی توابعِ خطیِ چندمتغیره برای اجتناب از بیش‌برازش رایج است. به یاد آورید با منظم‌سازی، هزینهٔ کلِ فرضیه را کمینه می‌کنیم، هم زیانِ تجربی و هم پیچیدگیِ فرضیه را حساب می‌کنیم:

```
Cost(h) = EmpLoss(h) + λ·Complexity(h)
```

برای توابعِ خطی، پیچیدگی را می‌توان به‌عنوانِ تابعی از وزن‌ها مشخص کرد. می‌توانیم خانواده‌ای از توابعِ منظم‌سازی را در نظر بگیریم:

```
Complexity(hw) = Lq(w) = ∑i |wi|^q
```

مانندِ توابعِ زیان، با q=1 «منظم‌سازیِ L1» داریم[9] که جمعِ قدرِ مطلق‌ها را کمینه می‌کند؛ با q=2، منظم‌سازیِ L2 جمعِ مربع‌ها را کمینه می‌کند. کدام تابعِ منظم‌سازی را باید انتخاب کنید؟ به مسئلهٔ خاص بستگی دارد، اما منظم‌سازیِ L1 مزیتِ مهمی دارد: تمایل به تولیدِ «مدلِ تنک» (Sparse Model) دارد. یعنی، اغلب وزن‌های بسیاری را صفر می‌کند، عملاً ویژگی‌های متناظر را کاملاً نامربوط اعلام می‌کند—درست همان‌گونه که LEARN-DECISION-TREE انجام می‌دهد (هرچند با سازوکاری متفاوت). فرضیه‌هایی که ویژگی‌ها را دور می‌اندازند می‌توانند برای انسان آسان‌تر فهمیده شوند و احتمالِ کمتری برای بیش‌برازش دارند.



شکل ۱۹.۱۴ – چرا منظم‌سازیِ L1 تمایل به تولیدِ مدلِ تنک دارد. چپ: با منظم‌سازیِ L1 (جعبه)، کمینهٔ زیانِ قابل‌دستیابی (کانتورهای هم‌مرکز) اغلب روی محوری رخ می‌دهد، بدان معنا که وزنی صفر است. راست: با منظم‌سازیِ L2 (دایره)، کمینهٔ زیان محتمل است هر جایِ دایره رخ دهد، بدونِ ترجیحِ وزن‌های صفر.

شکل ۱۹.۱۴ تبیینِ شهودیِ اینکه چرا منظم‌سازیِ L1 به وزن‌های صفر می‌انجامد، در حالی که L2 نه. توجه کنید کمینه‌کردنِ Loss(w) + λ·Complexity(w) معادلِ کمینه‌کردنِ Loss(w) مشروط به قیدِ Complexity(w) ≤ c، برای ثابتِ cای مرتبط با λ است. حالا، در شکل ۱۹.۱۴(الف) جعبهٔ لوزی‌شکل مجموعهٔ نقاطِ w در فضایِ وزنِ دوبعدی را بازنمایی می‌کند که پیچیدگیِ L1 کمتر از c دارند؛ راه‌حلِ ما باید جایی درونِ این جعبه باشد. بیضی‌های هم‌مرکز کانتورهای تابعِ زیان را بازنمایی می‌کنند، با کمینهٔ زیان در مرکز. می‌خواهیم نقطهٔ درونِ جعبه را بیابیم که به کمینه نزدیک‌ترین است؛ از نمودار می‌بینید، برای موقعیتِ دلخواهِ کمینه و کانتورهایش، رایج خواهد بود گوشهٔ جعبه نزدیک‌ترین به کمینه یافت شود، صرفاً به این دلیل که گوشه‌ها نوک‌تیزند. و البته گوشه‌ها نقاطی هستند که در برخی بُعدها مقدارِ صفر دارند. در شکل ۱۹.۱۴(ب)، همین کار را برای اندازهٔ پیچیدگیِ L2 انجام داده‌ایم، که به‌جای لوزی دایره را بازنمایی می‌کند. در این‌جا می‌بینید، به‌طورِ کلی، دلیلی برای وقوعِ تقاطع روی یکی از محورها نیست؛ بدین‌ترتیب منظم‌سازیِ L2 تمایل به تولیدِ وزن‌های صفر ندارد. نتیجه این است که شمارِ مثال‌های لازم برای یافتنِ h خوب برای منظم‌سازیِ L2 در شمارِ ویژگی‌های نامربوط خطی است، اما با منظم‌سازیِ L1 فقط لگاریتمی. شواهدِ تجربی روی بسیاری از مسائل از این تحلیل پشتیبانی می‌کند.

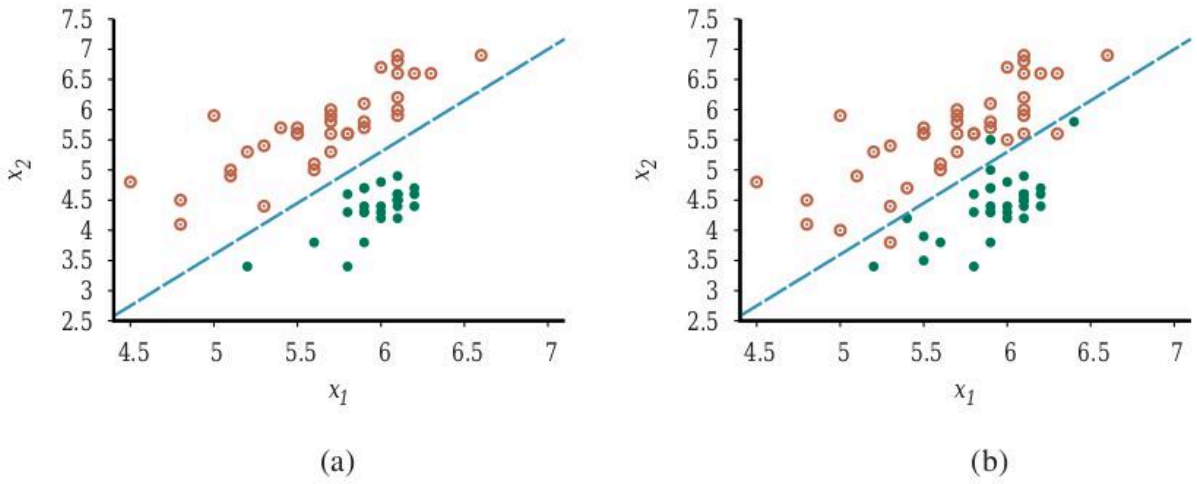
راهِ دیگرِ نگاه به آن این است که منظم‌سازیِ L1 محورهای ابعادی را جدی می‌گیرد، در حالی که L2 با آن‌ها به‌عنوانِ دلخواه رفتار می‌کند. تابعِ L2 کروی است، که آن را ناوردایِ چرخشی می‌کند: مجموعه‌ای از نقاط در صفحه را تصور کنید که با مختصاتِ x و y اندازه‌گیری شده‌اند. حالا چرخشِ محورها به اندازهٔ ۴۵ درجه را تصور کنید. مجموعهٔ متفاوتی از مقادیرِ ('x',y) می‌گیرید که همان نقاط را بازنمایی می‌کنند. اگر منظم‌سازیِ L2 را قبل و بعد از چرخش اعمال کنید، دقیقاً همان نقطه را به‌عنوانِ پاسخ می‌گیرید (هرچند نقطه با مختصاتِ جدیدِ ('x',y) توصیف می‌شود). این وقتی مناسب است که انتخابِ محورها واقعاً دلخواه است—وقتی مهم نیست دو بُعدِ شما فواصلِ شمالی و شرقی باشند؛ یا فواصلِ شمال‌شرقی و جنوب‌شرقی. با منظم‌سازیِ L1 پاسخِ متفاوتی می‌گیرید، زیرا تابعِ L1 ناوردایِ چرخشی نیست. این وقتی مناسب است که محورها قابلِ مبادله نیستند؛ چرخاندنِ «تعدادِ حمام‌ها» به اندازهٔ ۴۵ درجه به سمتِ «اندازهٔ زمین» معنا ندارد.

۱۹.۶.۴ رده‌بندهای خطی با آستانهٔ سخت

توابعِ خطی را می‌توان برای رده‌بندی و نیز رگرسیون استفاده کرد. برای نمونه، شکل ۱۹.۱۵(الف) نقاطِ داده‌ایِ دو رده را نشان می‌دهد: زمین‌لرزه‌ها (که برای لرزه‌شناسان جالب‌اند) و انفجارهای زیرزمینی (که برای کارشناسانِ کنترلِ تسلیحات جالب‌اند). هر نقطه با دو مقدارِ ورودیِ x1 و x2 تعریف می‌شود که به بزرگیِ موجِ جسمی و سطحیِ محاسبه‌شده از سیگنالِ لرزه‌ای اشاره دارند. با توجه به این دادهٔ آموزشی، تکلیفِ رده‌بندی یادگیریِ فرضیهٔ h است که نقاطِ تازهٔ (x1,x2) را بگیرد و برای زمین‌لرزه‌ها 0 یا برای انفجارها 1 برگرداند. «مرزِ تصمیم» (Decision Boundary) خطی (یا سطحی، در ابعادِ بالاتر) است که دو رده را جدا می‌کند. در شکل ۱۹.۱۵(الف)، مرزِ تصمیم خطِ راست است. مرزِ تصمیمِ خطی «جداکنندهٔ خطی» (Linear Separator) نامیده می‌شود و داده‌ای که چنین جداکننده‌ای را می‌پذیرد «قابلِ جداسازیِ خطی» (Linearly Separable) نامیده می‌شود. جداکنندهٔ خطی در این مورد با:

```
x2 = 1.7x1 – 4.9          یا          1.7x1 – x2 = 0 + 4.9–
```

تعریف می‌شود. انفجارها، که می‌خواهیم با مقدارِ 1 رده‌بندی کنیم، پایین و راستِ این خطاند؛ آن‌ها نقاطی هستند که 1.7x1 - x2 > 0 + 4.9-، در حالی که زمین‌لرزه‌ها 1.7x1 - x2 < 0 + 4.9- دارند. می‌توانیم معادله را با تبدیل به شکلِ ضربِ نقطه‌ایِ برداری راحت‌تر کنیم—با x0 = 1 داریم 1.7x1 - x2 = 0 - 4.9x0 + 1.7x1 - x2 = 0، و می‌توانیم بردارِ وزن‌ها، w = ⟨-4.9, 1.7, -1⟩، را تعریف و فرضیهٔ رده‌بندیِ hw(x) = 1 اگر w·x ≥ 0 و در غیرِ این صورت 0 بنویسیم.



شکل ۱۹.۱۵ — (الف) نمودارِ دو پارامترِ دادهٔ لرزه‌ای، بزرگیِ موجِ جسمیِ x1 و بزرگیِ موجِ سطحیِ x2، برای زمین‌لرزه‌ها (دایره‌های بازِ نارنجی) و انفجارهای هسته‌ای (دایره‌های سبز)، رخ‌داده بینِ 1982 و 1990 در آسیا و خاورمیانه (کبیسی و همکاران، 1998). همچنین مرزِ تصمیم بینِ رده‌ها نشان داده شده. (ب) همان حوزه با نقاطِ دادهٔ بیشتر. زمین‌لرزه‌ها و انفجارها دیگر قابلِ جداسازیِ خطی نیستند.

به‌طورِ جایگزین، می‌توانیم h را نتیجهٔ عبورِ تابعِ خطیِ w·x از «تابعِ آستانه» (Threshold Function) بدانیم:

```
hw(x) = Threshold(w·x)
```

که در آن Threshold(z) = 1 اگر z ≥ 0 و در غیرِ این صورت 0. تابعِ آستانه در شکل ۱۹.۱۷(الف) نشان داده شده.

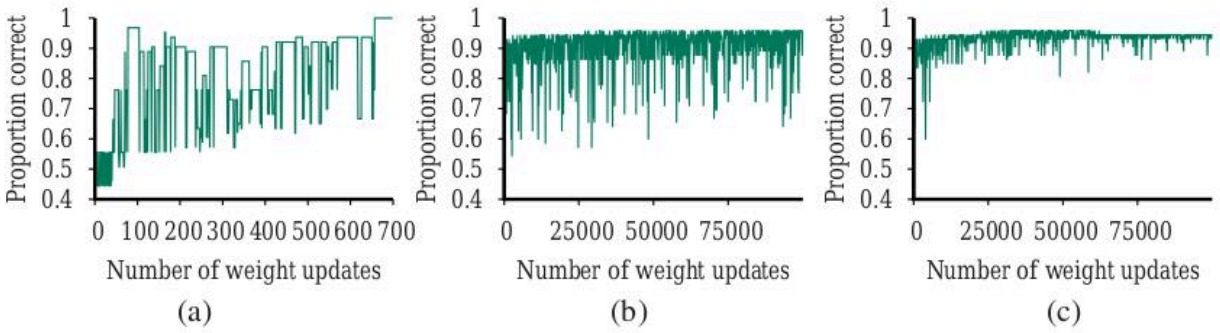
حالا که فرضیهٔ $h_w(x)$ شکلِ ریاضیِ خوش‌تعریفی دارد، می‌توانیم به انتخابِ وزن‌های w برای کمینه‌کردنِ زیان فکر کنیم. در بخش‌های ۱۹.۶.۱ و ۱۹.۶.۳، این را هم به‌صورتِ بسته (با صفرکردنِ گرادیان و حل برای وزن‌ها) و هم با نزولِ گرادیان در فضایِ وزن انجام دادیم. در این‌جا نمی‌توانیم هیچ‌کدام را انجام دهیم، زیرا گرادیان تقریباً همه‌جا در فضایِ وزن صفر است به‌جز در نقاطی که $w \cdot x = 0$، و در آن نقاط گرادیان تعریف‌نشده است. با این حال، قاعدهٔ به‌روزرسانیِ وزنِ ساده‌ای هست که به راه‌حل همگرا می‌شود—یعنی به جداکنندهٔ خطی که همهٔ مثال‌ها را به‌درستی رده‌بندی می‌کند. این قاعده «قاعدهٔ یادگیریِ پرسپترون» (Perceptron Learning Rule) نامیده می‌شود. برای مثالِ تکی (x,y)، داریم:

$$w_i \leftarrow w_i + \alpha(y - h_w(x)) \times x_i \qquad (19.8)$$

که اساساً یکسان با معادلهٔ (19.6)، قاعدهٔ به‌روزرسانیِ رگرسیونِ خطی است! چون مسئلهٔ رده‌بندیِ 0/1 را در نظر می‌گیریم، رفتار تا حدی متفاوت است. هم مقدارِ واقعیِ y و هم خروجیِ فرضیهٔ $h_w(x)$ یا 0 هستند یا 1، پس سه امکان هست:

- اگر خروجی درست است (یعنی $y = h_w(x)$)، وزن‌ها تغییر نمی‌کنند.
- اگر $y = 1$ اما $h_w(x) = 0$، آن‌گاه w_i وقتی ورودیِ متناظرِ x_i مثبت است افزایش و وقتی منفی است کاهش می‌یابد. این معنا دارد، زیرا می‌خواهیم $w \cdot x$ بزرگ‌تر شود تا $h_w(x)$ خروجیِ 1 بدهد.
- اگر $y = 0$ اما $h_w(x) = 1$، آن‌گاه w_i وقتی ورودیِ متناظرِ x_i مثبت است کاهش و وقتی منفی است افزایش می‌یابد. این معنا دارد، زیرا می‌خواهیم $w \cdot x$ کوچک‌تر شود تا $h_w(x)$ خروجیِ 0 بدهد.

معمولاً قاعدهٔ یادگیری یک‌مثال‌در-هر-بار اعمال می‌شود، با انتخابِ تصادفیِ مثال‌ها (مانندِ نزولِ گرادیانِ تصادفی). شکل ۱۹.۱۶(الف) «منحنیِ آموزشی» (Training Curve) را برای این قاعدهٔ یادگیری روی دادهٔ زمین‌لرزه/انفجارِ نشان‌داده‌شده در شکل ۱۹.۱۵(الف) نشان می‌دهد. منحنیِ آموزشی عملکردِ رده‌بند را روی مجموعهٔ آموزشیِ ثابت اندازه می‌گیرد، در حالی که فرایندِ یادگیری یک‌به‌یک روی آن مجموعهٔ آموزشی پیش می‌رود. منحنی نشان می‌دهد قاعدهٔ به‌روزرسانی به جداکنندهٔ خطیِ صفر-خطا همگرا می‌شود. فرایندِ «همگرایی» دقیقاً زیبا نیست، اما همیشه کار می‌کند. این اجرایِ خاص 657 گام برای همگرایی می‌گیرد، برای مجموعهٔ داده‌ای با 63 مثال، پس هر مثال به‌طورِ میانگین تقریباً ۱۰ بار ارائه می‌شود. معمولاً، تغییرِ بینِ اجراها زیاد است.



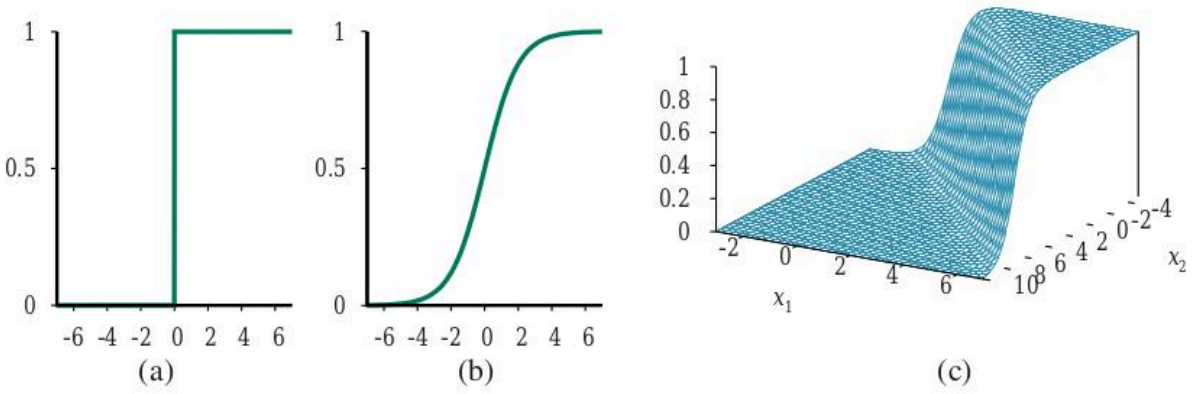
شکل ۱۹.۱۶ – (الف) نمودارِ دقتِ کلِ مجموعهٔ آموزشی در برابرِ شمارِ تکرارها برای قاعدهٔ یادگیریِ پرسپترون، با دادهٔ زمین‌لرزه/انفجارِ شکل ۱۹.۱۵(الف). (ب) همان نمودار برای دادهٔ نویزی و غیرقابل‌جداسازیِ شکل ۱۹.۱۵(ب)؛ به تغییرِ مقیاسِ محورِ x توجه کنید. (ج) همان نمودارِ (ب)، با برنامهٔ نرخِ یادگیریِ α(t) = 1000/(1000+t).

گفته‌ایم قاعدهٔ یادگیریِ پرسپترون وقتی نقاطِ داده قابلِ جداسازیِ خطی‌اند به جداکنندهٔ خطیِ کاملی همگرا می‌شود؛ اما اگر نباشند چه؟ این وضعیت در دنیای واقعی بیش‌ازحد رایج است. برای نمونه، شکل ۱۹.۱۵(ب) نقاطِ داده‌ای را که کبیسی و همکاران (1998) هنگام رسمِ دادهٔ نشان‌داده‌شده در شکل ۱۹.۱۵(الف) کنار گذاشتند دوباره اضافه می‌کند. در شکل ۱۹.۱۶(ب)، قاعدهٔ یادگیریِ پرسپترون را نشان می‌دهیم که حتی پس از ۱۰,۰۰۰ گام همگرا نمی‌شود: حتی اگرچه بارها به راه‌حلِ کمینه-خطا (سه خطا) می‌رسد، الگوریتم به تغییرِ وزن‌ها ادامه می‌دهد. به‌طورِ کلی، قاعدهٔ پرسپترون ممکن است برای نرخِ یادگیریِ ثابتِ α به راه‌حلِ پایداری همگرا نشود، اما اگر α به‌صورتِ O(1/t) فروپاشد، جایی که t شمارِ تکرار است، می‌توان نشان داد قاعده وقتی مثال‌ها در توالیِ تصادفی ارائه می‌شوند به راه‌حلِ کمینه-خطا همگرا می‌شود.[10] همچنین می‌توان نشان داد یافتنِ راه‌حلِ کمینه-خطا NP-سخت است، پس انتظار می‌رود برای رسیدنِ همگرایی ارائه‌های بسیاری از مثال‌ها لازم باشد. شکل ۱۹.۱۶(ج) فرایندِ آموزشی را با برنامهٔ نرخِ یادگیریِ α(t) = 1000/(1000+t) نشان می‌دهد: همگرایی پس از ۱۰۰,۰۰۰ تکرار کامل نیست، اما بسیار بهتر از حالتِ α ثابت است.

۱۹.۶.۵ رده‌بندیِ خطی با رگرسیونِ لجستیک

دیده‌ایم عبورِ خروجیِ تابعِ خطی از تابعِ آستانه، رده‌بندِ خطی می‌سازد؛ اما ماهیتِ سختِ آستانه مشکلاتی ایجاد می‌کند: فرضیهٔ hw(x) مشتق‌پذیر نیست و در واقع تابعِ ناپیوسته‌ای از ورودی‌ها و وزن‌هایش است. این یادگیری با قاعدهٔ پرسپترون را ماجراجوییِ بسیار غیرقابل‌پیش‌بینی می‌کند. علاوه بر این، رده‌بندِ خطی همیشه پیش‌بینیِ کاملاً مطمئنِ 1 یا 0 را اعلام می‌کند، حتی برای مثال‌هایی که بسیار به مرز نزدیک‌اند؛ بهتر بود می‌توانست برخی مثال‌ها را 0 یا 1 واضح و برخی را مواردِ مرزیِ نامشخص رده‌بندی کند.

همهٔ این مسائل را می‌توان تا حد زیادی با نرم‌کردنِ تابعِ آستانه حل کرد—تقریبِ آستانهٔ سخت با تابعِ پیوسته و مشتق‌پذیر. در فصل ۱۳ (صفحهٔ ۴۲۴)، دو تابع را دیدیم که شبیهِ آستانه‌های نرم‌اند: انتگرالِ توزیعِ نرمالِ استاندارد (استفاده‌شده برای مدلِ پروبیت) و تابعِ لجستیک (استفاده‌شده برای مدلِ لاجیت). اگرچه دو تابع از نظرِ شکل بسیار شبیه‌اند، تابعِ لجستیک Logistic(z) = 1/(1+e^(-z)) ویژگی‌های ریاضیِ راحت‌تری دارد. تابع در شکل ۱۹.۱۷(ب) نشان داده شده.



شکل ۱۹.۱۷ — (الف) تابعِ آستانهٔ سختِ Threshold(z) با خروجیِ 0/1. توجه کنید تابع در z = 0 مشتق‌پذیر نیست. (ب) تابعِ لجستیک، Logistic(z) = 1/(1+e^(-z))، که «تابعِ سیگموئید» نیز نامیده می‌شود. (ج) نمودارِ فرضیهٔ رگرسیونِ لجستیکِ hw(x) = Logistic(w·x) برای دادهٔ نشان‌داده‌شده در شکل ۱۹.۱۵(ب).

با جایگزینیِ تابعِ لجستیک به‌جای تابعِ آستانه، حالا hw(x) = Logistic(w·x) = 1/(1+e^(-w·x)) داریم. نمونه‌ای از چنین فرضیه‌ای برای مسئلهٔ دودروندیِ زمین‌لرزه/انفجار در شکل ۱۹.۱۷(ج) نشان داده شده. توجه کنید خروجی، که عددی بینِ 0 و 1 است، می‌تواند به‌عنوانِ احتمالِ تعلق به ردهٔ برچسب‌گذاری‌شده با 1 تفسیر شود. فرضیه مرزِ نرمی در فضایِ ورودی تشکیل می‌دهد و برای هر ورودی در مرکزِ ناحیهٔ مرزی احتمالِ 0.5 و با دورشدن از مرز به 0 یا 1 نزدیک می‌شود.

فرایندِ برازشِ وزن‌های این مدل برای کمینه‌کردنِ زیان روی مجموعهٔ داده «رگرسیونِ لجستیک» (Logistic Regression) نامیده می‌شود. راه‌حلِ بسته‌ای برای یافتنِ مقدارِ بهینهٔ w با این مدل نیست، اما محاسبهٔ نزولِ گرادیان سرراست است. چون فرضیه‌های‌مان دیگر فقط 0 یا 1 خروجی نمی‌دهند، از تابعِ زیانِ L2 استفاده می‌کنیم؛ همچنین، برای خواناییِ فرمول‌ها، از g برای تابعِ لجستیک با مشتقِ g' استفاده می‌کنیم.

برای مثالِ تکی (x,y)، اشتقاقِ گرادیان همانندِ رگرسیونِ خطی (معادلهٔ (19.5)) تا نقطه‌ای است که شکلِ واقعیِ h جای می‌شود. (برای این اشتقاق، باز به قاعدهٔ زنجیره‌ای نیاز داریم.) داریم:

```
wi Loss(w) = ∂/∂wi (y − hw(x))²∂/∂
wi (y − hw(x))∂/∂ × (y − hw(x))2 =
g′(w·x) × ∂/∂wi (w·x) × (y − hw(x))2− =
g′(w·x) × xi × (y − hw(x))2− =
```

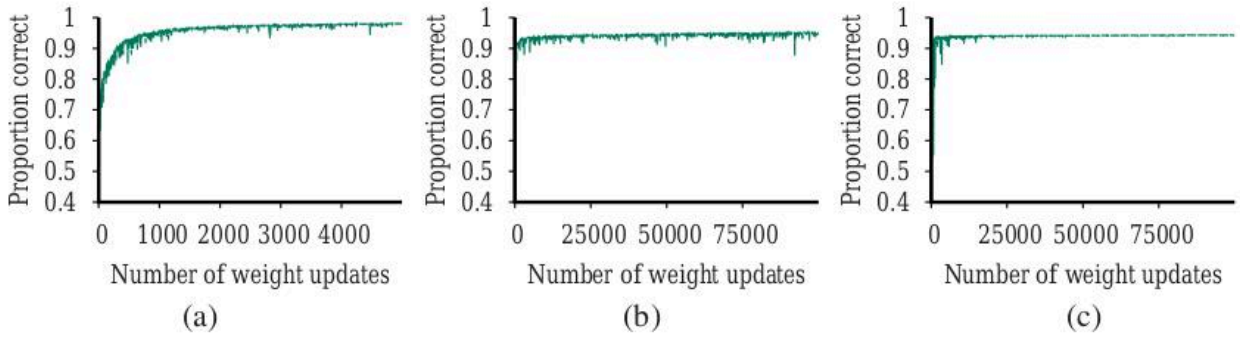
مشتقِ g' تابعِ لجستیک g'(z) = g(z)(1-g(z)) را ارضا می‌کند، پس داریم:

```
g′(w·x) = g(w·x)(1−g(w·x)) = hw(x)(1−hw(x))
```

بنابراین به‌روزرسانیِ وزن برای کمینه‌کردنِ زیان، گامی در جهتِ تفاوتِ بینِ ورودی و پیش‌بینی، (y - hw(x))، برمی‌دارد و طولِ آن گام به ثابتِ α و g' بستگی دارد:

$$w_i \leftarrow w_i + \alpha(y - h_w(x)) \times h_w(x)(1-h_w(x)) \times x_i \qquad (19.9)$$

با تکرارِ آزمایش‌های شکل ۱۹.۱۶ با رگرسیونِ لجستیک به‌جای رده‌بندِ آستانهٔ خطی، نتایجِ نشان‌داده‌شده در شکل ۱۹.۱۸ را به دست می‌آوریم. در (الف)، حالتِ قابلِ جداسازیِ خطی، رگرسیونِ لجستیک تا حدی کندتر همگرا می‌شود، اما بسیار قابلِ‌پیش‌بینی‌تر رفتار می‌کند. در (ب) و (ج)، جایی که داده نویزی و غیرقابلِ‌جداسازی است، رگرسیونِ لجستیک بسیار سریع‌تر و قابلِ‌اعتمادتر همگرا می‌شود. این مزایا تمایل دارند به کاربردهای دنیایِ واقعی منتقل شوند، و رگرسیونِ لجستیک به یکی از محبوب‌ترین فنونِ رده‌بندی برای مسائلِ پزشکی، بازاریابی، تحلیلِ نظرسنجی، امتیازدهیِ اعتباری، بهداشتِ عمومی و دیگر کاربردها تبدیل شده.



شکل ۱۹.۱۸ – تکرارِ آزمایش‌های شکل ۱۹.۱۶ با رگرسیونِ لجستیک. نمودارِ (الف) ۵۰۰۰ تکرار را پوشش می‌دهد نه ۷۰۰ را، در حالی که نمودارهای (ب) و (ج) همان مقیاسِ قبلی را دارند.

۱۹.۷ مدل‌های ناپارامتری

رگرسیونِ خطی از دادهٔ آموزشی برای برآوردِ مجموعه‌ای ثابت از پارامترهای $\mathbf{w}$ استفاده می‌کند. این فرضیهٔ $h_w(x)$ ما را تعریف می‌کند، و در آن نقطه می‌توانیم دادهٔ آموزشی را دور بیندازیم، زیرا همه‌شان با $\mathbf{w}$ خلاصه شده‌اند. مدلِ یادگیری که داده را با مجموعه‌ای از پارامترهای با اندازهٔ ثابت (مستقل از شمارِ مثال‌های آموزشی) خلاصه می‌کند «مدلِ پارامتری» (Parametric Model) نامیده می‌شود.

وقتی مجموعه‌های داده کوچک‌اند، منطقی است محدودیتِ قوی‌ای بر فرضیه‌های مجاز داشته باشیم تا از بیش‌برازش اجتناب کنیم. اما وقتی میلیون‌ها یا میلیاردها مثال برای یادگیری هست، بهتر به نظر می‌رسد بگذاریم داده خودش حرف بزند، به‌جای اینکه مجبورش کنیم از طریقِ بردارِ ریزی از پارامترها حرف بزند. اگر داده بگوید پاسخِ درست تابعِ بسیار پرنوسانی است، نباید خودمان را به توابعِ خطی یا کمی پرنوسان محدود کنیم.

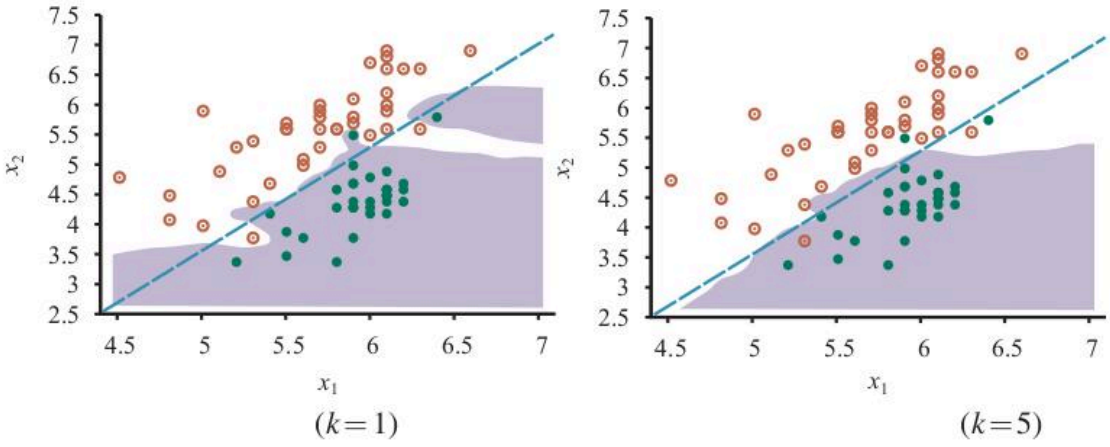
«مدلِ ناپارامتری» (Nonparametric Model) مدلی است که نمی‌توان آن را با مجموعه‌ای کران‌دار از پارامترها مشخصه‌سازی کرد. برای نمونه، تابعِ تکه‌ای-خطی از شکل ۱۹.۱ همهٔ نقاطِ داده را به‌عنوانِ بخشی از مدل نگه می‌دارد. روش‌های یادگیری که این کار را می‌کنند همچنین «یادگیریِ مبتنی بر نمونه» (Instance-Based Learning) یا «یادگیریِ مبتنی بر حافظه» (Memory-Based Learning) توصیف شده‌اند. ساده‌ترین روشِ یادگیریِ مبتنی بر نمونه

«جستجوی جدولی» (Table Lookup) است: همهٔ مثال‌های آموزشی را بگیر، در جدولِ جستجو بگذار، و سپس وقتی h(x) خواسته شد، ببین x در جدول است؛ اگر هست، y متناظر را برگردان. مسئلهٔ این روش این است که خوب تعمیم نمی‌دهد: وقتی x در جدول نیست، اطلاعاتی دربارهٔ مقدارِ محتمل نداریم.

۱۹.۷.۱ مدل‌های نزدیک‌ترین-همسایه

می‌توانیم جستجوی جدولی را با تغییرِ کوچکی بهبود دهیم: با توجه به پرس‌وجویِ xq، به‌جای یافتنِ مثالی که برابرِ xq است، k مثالی را بیاب که به xq نزدیک‌ترین‌اند. این «جستجوی k-نزدیک‌ترین-همسایه» نامیده می‌شود. از نمادِ NN(k,xq) برای مجموعهٔ k همسایهٔ نزدیک‌ترین به xq استفاده می‌کنیم. برای رده‌بندی، مجموعهٔ همسایه‌های NN(k,xq) را بیاب و رایج‌ترین مقدارِ خروجی را بگیر—برای نمونه، اگر k=3 و مقادیرِ خروجی ⟨Yes,No,Yes⟩ باشند، رده‌بندی Yes خواهد بود. برای اجتناب از تساوی‌ها در رده‌بندیِ دودویی، k معمولاً عددِ فرد انتخاب می‌شود.

برای رگرسیون، می‌توانیم میانگین یا میانهٔ k همسایه را بگیریم، یا می‌توانیم مسئلهٔ رگرسیونِ خطی را روی همسایه‌ها حل کنیم. تابعِ تکه‌ای-خطی از شکل ۱۹.۱ مسئلهٔ رگرسیونِ خطیِ (بی‌اهمیت) را با دو نقطهٔ دادهٔ سمتِ راست و چپِ xq حل می‌کند. (وقتی نقاطِ دادهٔ xi به‌طورِ مساوی فاصله‌دارند، این دو نزدیک‌ترین همسایه خواهند بود.) در شکل ۱۹.۱۹، مرزِ تصمیمِ رده‌بندیِ k-نزدیک‌ترین-همسایه را برای k = 1 و 5 روی مجموعهٔ دادهٔ زمین‌لرزه از شکل ۱۹.۱۵ نشان می‌دهیم.



شکل ۱۹.۱۹ — (الف) مدلِ k-نزدیک‌ترین-همسایه‌ها که گسترهٔ ردهٔ انفجار را برای دادهٔ شکل ۱۹.۱۵، با k=1، نشان می‌دهد. بیش‌برازش آشکار است. (ب) با k=5، مسئلهٔ بیش‌برازش برای این مجموعهٔ داده از بین می‌رود.

روش‌های ناپارامتری همچنان در معرضِ کم‌برازش و بیش‌برازش‌اند، درست مانندِ روش‌های پارامتری. در این مورد، 1-نزدیک‌ترین-همسایه بیش‌برازش می‌کند؛ بیش‌ازحد به دورافتادهٔ سیاه در بالا-راست و دورافتادهٔ سفید در (3.7 ,5.4) واکنش نشان می‌دهد. مرزِ تصمیمِ 5-نزدیک‌ترین-همسایه خوب است؛ k بالاتر کم‌برازش می‌کرد. مثلِ همیشه، اعتبارسنجیِ متقاطع می‌تواند برای انتخابِ بهترین مقدارِ k استفاده شود.

خودِ واژهٔ «نزدیک‌ترین» دلالت بر معیارِ فاصله دارد. چگونه فاصلهٔ نقطهٔ پرس‌وجویِ xq تا نقطهٔ مثالِ xj را اندازه بگیریم؟ معمولاً، فاصله‌ها با «فاصلهٔ مینکوسکی» (Minkowski Distance) یا هنجارِ Lp اندازه‌گیری می‌شوند، که چنین تعریف می‌شود:

```
Lp(xj, xq) = (∑i |xj,i − xq,i|^p)^(1/p)
```

با p=2 این فاصلهٔ اقلیدسی و با p=1 فاصلهٔ منهتن است. با مقادیرِ ویژگیِ بولی، شمارِ ویژگی‌هایی که دو نقطه در آن‌ها متفاوت‌اند «فاصلهٔ همینگ» (Hamming Distance) نامیده می‌شود. اغلب اگر ابعاد ویژگی‌های مشابهی را اندازه می‌گیرند فاصلهٔ اقلیدسی استفاده می‌شود، مانندِ عرض، ارتفاع و عمقِ قطعات، و اگر نامشابه‌اند فاصلهٔ منهتن، مانندِ سن، وزن و جنسیتِ بیمار. توجه کنید اگر از اعدادِ خامِ هر بُعد استفاده کنیم، فاصلهٔ کل با تغییرِ واحدها در هر بُعد اثر می‌پذیرد. یعنی، اگر بُعدِ ارتفاع را از متر به مایل تغییر دهیم در حالی که عرض و عمق را ثابت نگه داریم، نزدیک‌ترین همسایه‌های متفاوتی می‌گیریم. و چگونه تفاوتِ سن را با تفاوتِ وزن مقایسه کنیم؟ رویکردِ رایج اعمالِ «نرمال‌سازی» (Normalization) بر اندازه‌گیری‌های هر بُعد است. می‌توانیم میانگینِ μi و انحرافِ معیارِ σi مقادیرِ هر بُعد را محاسبه و آن‌ها را بازمقیاس کنیم، به‌گونه‌ای که xj,i به (xj,i − μi)/σi تبدیل شود. معیارِ پیچیده‌تری به نامِ «فاصلهٔ ماهالانوبیس» (Mahalanobis Distance) کوواریانسِ بینِ ابعاد را در نظر می‌گیرد.

در فضاهایِ کم‌بعد با دادهٔ فراوان، نزدیک‌ترین همسایه‌ها بسیار خوب کار می‌کنند: احتمالاً نقاطِ دادهٔ نزدیکِ کافی برای پاسخِ خوب داریم. اما با افزایشِ شمارِ ابعاد با مسئله‌ای روبه‌رو می‌شویم: نزدیک‌ترین همسایه‌ها در فضاهایِ پربعد معمولاً چندان نزدیک نیستند! k-نزدیک‌ترین-همسایه را روی مجموعهٔ داده‌ای از N نقطه که به‌صورتِ یکنواخت درونِ ابرمکعبِ واحدِ n-بعدی توزیع شده‌اند در نظر بگیر. k-همسایگیِ یک نقطه را کوچک‌ترین ابرمکعبی تعریف می‌کنیم که k نزدیک‌ترین همسایه را دارد. ℓ میانگینِ طولِ ضلعِ همسایگی باشد. سپس حجمِ همسایگی (که k نقطه دارد) ℓ^n و حجمِ مکعبِ کامل (که N نقطه دارد) 1 است. پس، به‌طورِ میانگین، $\ell^n = k/N$، و بنابراین:

```
ℓ = (k/N)^(1/n)
```

این «نفرینِ ابعاد» (Curse of Dimensionality) نامیده می‌شود. برای نمونه، برای N = 10,000 نقطه و n = 10 بعد، میانگینِ طولِ ضلعِ همسایگی (1/1000)^(1/10) ≈ 0.5 است. بدین‌ترتیب، در ۱۰ بُعد، «نزدیک‌ترین» همسایه‌ها فقط ۵۰٪ از هر بُعد فاصله دارند؛ در ۲ بعد، همسایگی‌ها فقط ۳٪ از هر بُعد را پوشش می‌دهند. در ۱۰۰ بعد، ℓ ≈ 0.955 است؛ یعنی برای یافتنِ همسایه‌ها باید تقریباً کلِ فضایِ ورودی را بپیماییم! نفرینِ ابعاد نخستین بار توسط ریچارد بلمن (1961) شناسایی شد.

۱۹.۷.۲ یافتنِ نزدیک‌ترین همسایه‌ها با درخت‌های k-d

درختِ دودوییِ متوازن روی داده با شمارِ دلخواهِ ابعاد «درختِ k-d» (K-D Tree) نامیده می‌شود، به‌معنای درختِ k-بعدی. ساختِ درختِ k-d شبیهِ ساختِ درختِ دودوییِ متوازن است. با مجموعه‌ای از مثال‌ها شروع می‌کنیم و در گرهٔ ریشه آن‌ها را در امتدادِ بُعدِ iام با آزمونِ اینکه آیا xi ≤ m است، که در آن m میانهٔ مثال‌ها در امتدادِ بُعدِ iام است،

تقسیم می‌کنیم؛ بدین‌ترتیب نصفِ مثال‌ها در شاخهٔ چپِ درخت و نصف در راست خواهد بود. سپس به‌صورتِ بازگشتی برای مجموعه‌های چپ و راستِ مثال‌ها درخت می‌سازیم، وقتی کمتر از دو مثال باقی مانده متوقف می‌شویم. برای انتخابِ بُعدِ تقسیم در هر گرهٔ درخت، می‌توان به‌سادگی بُعدِ i mod n را در سطحِ i درخت انتخاب کرد. (توجه کنید ممکن است لازم باشد در طولِ پایین‌رفتنِ درخت چند بار روی هر بُعدِ معین تقسیم کنیم.) استراتژیِ دیگر تقسیم بر بُعدی است که گسترده‌ترین پخشِ مقادیر را دارد.

جستجوی دقیق از درختِ k-d مانندِ جستجو از درختِ دودویی است (با پیچیدگیِ جزیی که باید توجه کنید در هر گره کدام بُعد را آزمایش می‌کنید). اما جستجوی نزدیک‌ترین-همسایه پیچیده‌تر است. همان‌طور که از شاخه‌ها پایین می‌رویم و مثال‌ها را نصف می‌کنیم، در برخی موارد می‌توانیم نیمی از مثال‌ها را نادیده بگیریم. اما نه همیشه. گاهی نقطه‌ای که برایش پرس‌وجو می‌کنیم بسیار نزدیکِ مرزِ تقسیم می‌افتد. خودِ نقطهٔ پرس‌وجو ممکن است در سمتِ چپِ مرز باشد، اما یک یا چند مورد از k نزدیک‌ترین همسایه ممکن است در واقع در سمتِ راست باشند. باید این امکان را با محاسبهٔ فاصلهٔ نقطهٔ پرس‌وجو تا مرزِ تقسیم آزمایش کنیم، و سپس اگر فاصله کمتر از فاصلهٔ تا k-امین همسایهٔ فعلی باشد، باید در سمتِ دیگر نیز جستجو کنیم. پیچیدگیِ تحلیلِ موردِ متوسطِ جستجوی نزدیک‌ترین-همسایه در درختِ k-d، O(n) در تعدادِ ابعاد است (یعنی، درختِ k-d در زمانِ چندجمله‌ای در n جستجو می‌کند، اما در N فقط O(log N) اگر n ثابت باشد). برای کاربردهایی مانندِ جستجوی تصویر یا دادهٔ ژنومی، ابعاد می‌توانند در هزاران باشند، و درخت‌های k-d عملاً بی‌فایده می‌شوند. برای این موارد، به دنبال راه‌های تقریبی می‌گردیم.

۱۹.۷.۳ درهم‌سازیِ حساس-به-محل

جدول‌های درهم‌سازی پتانسیلِ جستجویِ حتی سریع‌تری از درخت‌های دودویی دارند. اما چگونه می‌توانیم با جدولِ درهم‌سازی نزدیک‌ترین همسایه‌ها را بیابیم، وقتی کدهای درهم‌سازی بر تطابقِ دقیق تکیه دارند؟ کدهای درهم‌سازی مقادیر را به‌صورتِ تصادفی بینِ سطل‌ها توزیع می‌کنند، اما ما می‌خواهیم نقاطِ نزدیک در همان سطل گروه‌بندی شوند؛ به «درهم‌سازیِ حساس-به-محل» (Locality-Sensitive Hash) یا LSH نیاز داریم.

نمی‌توانیم از درهم‌سازی‌ها برای حلِ دقیقِ NN(k,xq) استفاده کنیم، اما با استفاده‌ی هوشمندانه از الگوریتم‌های تصادفی، می‌توانیم راه‌حلِ تقریبی بیابیم. نخست مسئلهٔ «نزدیک‌ترین-همسایه‌هایِ تقریبی» را تعریف می‌کنیم: با توجه به مجموعه‌ای از نقاطِ مثال و نقطهٔ پرس‌وجویِ xq، با احتمالِ بالا نقطهٔ مثالی (یا نقاطی) را بیاب که به xq نزدیک است. دقیق‌تر، الزام می‌کنیم اگر نقطهٔ xj در شعاعِ r از xq باشد، آن‌گاه با احتمالِ بالا الگوریتم نقطهٔ 'xj را خواهد یافت که در فاصلهٔ cr از xq است. اگر هیچ نقطه‌ای در شعاعِ r نباشد، الگوریتم مجاز است شکست را گزارش کند. مقادیرِ c و «احتمالِ بالا» ابرپارامترهای الگوریتم‌اند.

برای حلِ نزدیک‌ترین-همسایه‌های تقریبی، به تابعِ درهم‌سازیِ g(x) نیاز داریم که این ویژگی را داشته باشد که برای هر دو نقطهٔ xj و 'xj، احتمالِ اینکه همان کدِ درهم‌سازی را داشته باشند اگر فاصله‌شان کم باشد زیاد و اگر فاصله‌شان زیاد باشد کم است. ساختِ چنین تابعی از تابعِ ساده‌ترِ h(x) که یک مختصهٔ تصادفیِ انتخاب‌شده از بردارِ x را برمی‌گرداند، آسان است. توابعِ درهم‌سازیِ h_i را با انتخابِ تصادفیِ بُعدی و سطل‌بندیِ تصادفیِ مقدارِ آن بُعد می‌سازیم. اگر نقاط نزدیک‌اند، در هر بُعدِ تصادفیِ انتخابی مقادیرِ مشابهی دارند؛ اگر دور باشند، احتمالِ اینکه همان بُعدِ انتخابی را داشته باشند و در همان سطلِ انتخابی بیفتند بسیار کم است. سپس g(x) را به‌صورتِ الحاقِ (یا فهرستِ) h_iهای

انتخاب‌شده تعریف می‌کنیم. نقاطی که با g(xq) درهم‌سازی می‌شوند کاندیدهای نزدیک‌ترین همسایه‌های تقریبی‌اند؛ با شماری از gهای مستقلِ ساخته‌شده، احتمالِ اینکه یک همسایهٔ نزدیکِ واقعی حداقل در یکی از جدول‌های درهم‌سازی با xq هم‌سطل شود را می‌توانیم بالا ببریم. برای نمونه، با ۲۵ جدولِ درهم‌سازیِ LSH، رسیدن به ۹۰٪ احتمالِ یافتنِ همسایهٔ نزدیک برای فاصلهٔ داده شده رایج است. این فن امکانِ بازیابیِ تقریبیِ سریعِ مشابه‌ترین‌ها را در مجموعه‌های دادهٔ میلیاردی می‌دهد و برای دادهٔ ژنومی، که هر رکورد میلیون‌ها ویژگی دارد، به‌طورِ خاص مفید است (برلین و همکاران، 2015).

۱۹.۷.۴ رگرسیونِ ناپارامتری

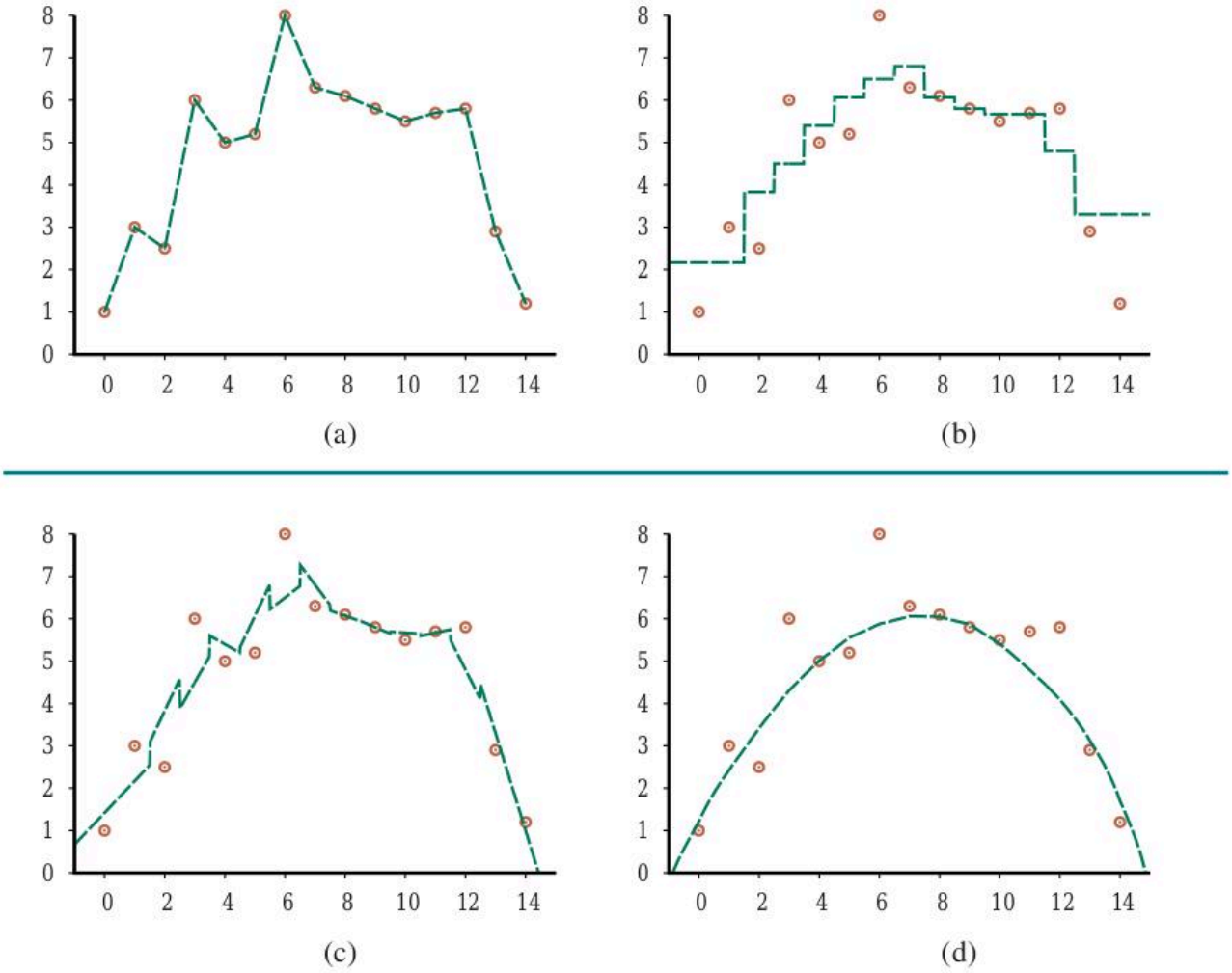
حالا به رویکردهای ناپارامتریِ رگرسیون به‌جای رده‌بندی نگاه می‌کنیم. شکل ۱۹.۲۰ نمونه‌ای از مدل‌های مختلف را نشان می‌دهد. در (الف)، شاید ساده‌ترین روشِ همه را داریم، که به‌صورتِ غیررسمی «وصل‌کردنِ نقطه‌ها» و به‌صورتِ تحقیرآمیز «رگرسیونِ ناپارامتریِ تکه‌ای-خطی» شناخته می‌شود. این مدل تابعِ h(x) را ایجاد می‌کند که، وقتی پرس‌وجویِ xq داده می‌شود، مثال‌های آموزشیِ بلافاصلهٔ چپ و راستِ xq را در نظر می‌گیرد و بینِ آن‌ها درون‌یابی می‌کند. وقتی نویز کم است، این روشِ بی‌اهمیت در واقع چندان بد نیست، به همین دلیل است که ویژگیِ استانداردِ نرم‌افزارهای نمودارکشی در صفحه‌گسترده‌هاست. اما وقتی داده نویزی است، تابعِ حاصل تیغی است و خوب تعمیم نمی‌دهد.

«رگرسیونِ نزدیک‌ترین-همسایه‌ها» وصل‌کردنِ نقطه‌ها را بهبود می‌بخشد. به‌جای استفاده از فقط دو مثالِ چپ و راستِ نقطهٔ پرس‌وجویِ xq، از k نزدیک‌ترین همسایه استفاده می‌کنیم. (در این‌جا k=3 استفاده می‌کنیم.) مقدارِ بزرگ‌ترِ k تمایل به صاف‌کردنِ بزرگیِ تیغه‌ها دارد، هرچند تابعِ حاصل ناپیوستگی دارد. شکل ۱۹.۲۰ دو نسخه از رگرسیونِ k-نزدیک‌ترین-همسایه را نشان می‌دهد. در (ب)، میانگینِ k-نزدیک‌ترین-همسایه را داریم: h(x) مقدارِ میانگینِ k نقطه، ∑yj/k است. توجه کنید در نقاطِ دورافتاده، نزدیکِ x=0 و x=14، برآوردها ضعیف‌اند، زیرا همهٔ شواهد از یک سمت (داخل) می‌آیند و روند را نادیده می‌گیرند. در (ج)، رگرسیونِ خطیِ k-نزدیک‌ترین-همسایه را داریم، که بهترین خط را از میانِ k مثال می‌یابد. این کارِ بهتری در ثبتِ روندها در دورافتاده‌ها می‌کند، اما همچنان ناپیوسته است. در هر دو (ب) و (ج)، با پرسشِ انتخابِ مقدارِ خوبِ k رها می‌شویم. پاسخ، مثلِ همیشه، اعتبارسنجیِ متقاطع است.

«رگرسیونِ وزن‌محلی» (Locally Weighted Regression؛ شکل ۱۹.۲۰(د)) مزایای نزدیک‌ترین همسایه‌ها را بدونِ ناپیوستگی‌ها به ما می‌دهد. برای اجتناب از ناپیوستگی در h(x)، باید از ناپیوستگی در مجموعهٔ مثال‌های استفاده‌شده برای برآوردِ h(x) اجتناب کنیم. ایدهٔ رگرسیونِ وزن‌محلی این است که در هر نقطهٔ پرس‌وجویِ xq، مثال‌هایی که به xq نزدیک‌اند به‌شدت وزن‌گذاری می‌شوند، مثال‌هایی که دورترند کمتر وزن‌گذاری می‌شوند، و دورترین‌ها اصلاً وزن‌گذاری نمی‌شوند. کاهشِ وزن روی فاصله معمولاً تدریجی است، نه ناگهانی.

تصمیم می‌گیریم هر مثال چقدر وزن بگیرد با تابعی به نام «هسته» (Kernel)، که ورودی‌اش فاصلهٔ بینِ نقطهٔ پرس‌وجو و مثال است. تابعِ هستهٔ K تابعِ کاهنده‌ای از فاصله با بیشینه در 0 است، پس K(Distance(xj,xq)) وزنِ بیشتری به مثال‌های xj می‌دهد که به نقطهٔ پرس‌وجویِ xq نزدیک‌ترند، که برایش می‌خواهیم مقدارِ تابع را پیش‌بینی کنیم. انتگرالِ مقدارِ هسته روی کلِ فضایِ ورودیِ x باید متناهی باشد—و اگر انتخاب کنیم انتگرال 1 باشد، برخی محاسبات آسان‌تر می‌شوند.

شکل ۱۹.۲۰(د) با هستهٔ درجه‌دومی، $K(d) = \max(0, 1 - (2|d|/w)^2)$، با پهنایِ هستهٔ $w = 10$ تولید شد. شکل‌های دیگر، مانندِ گاوسی‌ها، نیز استفاده می‌شوند. معمولاً، پهنایِ هسته مهم‌تر از شکلِ دقیقش است: این ابرپارامترِ مدل است که بهتر است با اعتبارسنجیِ متقاطع انتخاب شود. اگر هسته‌ها بیش‌ازحد پهن باشند کم‌برازش و اگر بیش‌ازحد باریک باشند بیش‌برازش می‌گیریم. در شکل ۱۹.۲۰(د)، پهنایِ هستهٔ 10 منحنیِ نرمی می‌دهد که درست به نظر می‌رسد.



شکل ۱۹.۲۰ — مدل‌های رگرسیونِ ناپارامتری: (الف) وصل‌کردنِ نقطه‌ها، (ب) میانگینِ 3-نزدیک‌ترین-همسایه، (ج) رگرسیونِ خطیِ 3-نزدیک‌ترین-همسایه، (د) رگرسیونِ وزن‌محلی با هستهٔ درجه‌دومیِ پهنای 10.

انجامِ رگرسیونِ وزن‌محلی با هسته‌ها حالا سرراست است. برای نقطهٔ پرس‌وجویِ معینِ xq مسئلهٔ رگرسیونِ وزن‌دارِ زیر را حل می‌کنیم:

$$w^* = \text{argmin}_w \sum_j K(\text{Distance}(x_q, x_j)) (y_j - w \cdot x_j)^2$$

که در آن Distance هر یک از معیارهای فاصلهٔ بحث‌شده برای نزدیک‌ترین همسایه‌هاست. سپس پاسخ $h(x_q) = w^* \cdot x_q$ است. توجه کنید برای هر نقطهٔ پرس‌وجو باید مسئلهٔ رگرسیونِ تازه‌ای حل کنیم—این همان معنای محلی‌بودن است. (در رگرسیونِ خطیِ معمولی، مسئلهٔ رگرسیون را یک بار، به‌صورتِ سراسری، حل می‌کردیم و سپس از همان h_w

برای هر نقطهٔ پرس‌وجو استفاده می‌کردیم.) در برابرِ این کارِ اضافه، این واقعیت است که هر مسئلهٔ رگرسیون آسان‌تر حل می‌شود، زیرا فقط مثال‌های با وزنِ غیرصفر را شامل می‌شود—مثال‌هایی که درونِ پهنایِ هسته از پرس‌وجو هستند. وقتی پهنایِ هسته‌ها کوچک است، این ممکن است فقط چند نقطه باشد.

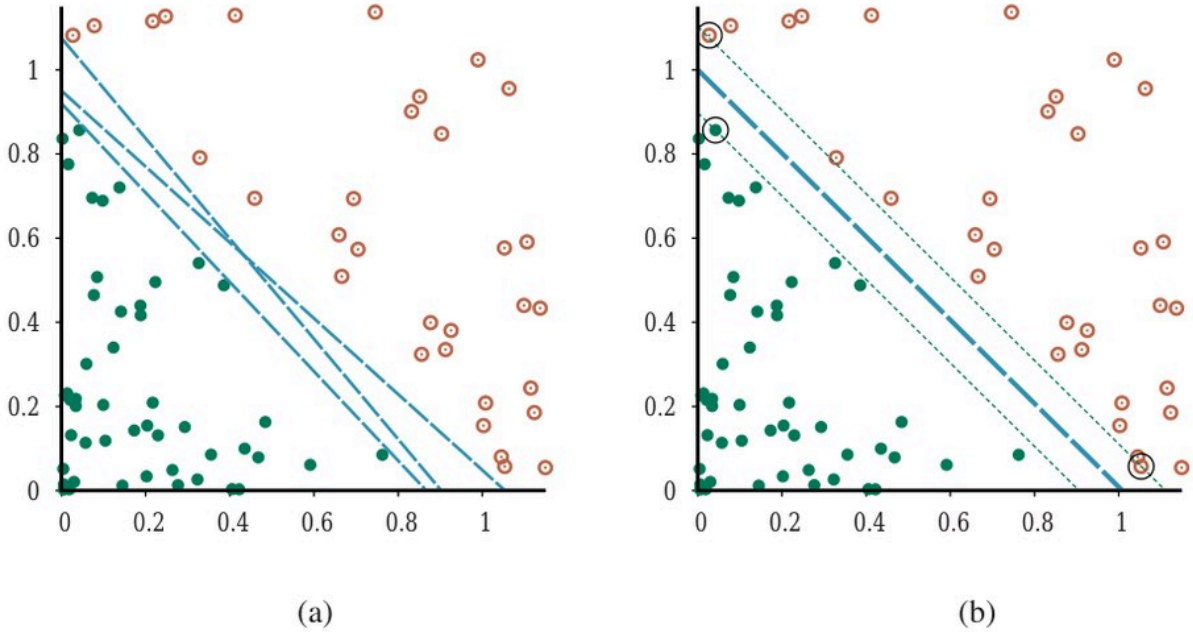
بیشترِ مدل‌های ناپارامتری این مزیت را دارند که انجامِ اعتبارسنجیِ متقاطعِ یک-بیرون‌گذاری بدونِ نیاز به بازمحاسبهٔ همه‌چیز آسان است. با مدلِ k-نزدیک‌ترین-همسایه، برای نمونه، وقتی مثالِ آزمونِ (x,y) داده می‌شود، یک بار k نزدیک‌ترین همسایه را بازیابی می‌کنیم، زیانِ هر مثالِ $L(y,h(x))$ را از آن‌ها محاسبه می‌کنیم، و آن را به‌عنوانِ نتیجهٔ یک-بیرون‌گذاری برای هر مثالی که یکی از همسایه‌ها نیست ثبت می‌کنیم. سپس $k+1$ نزدیک‌ترین همسایه را بازیابی می‌کنیم و نتایجِ متمایز را برای بیرون‌گذاشتنِ هر یک از k همسایه ثبت می‌کنیم. با N مثال، کلِ فرایند $O(k)$ است، نه $O(kN)$.

۱۹.۷.۵ ماشین‌های بردارِ پشتیبان

در اوایلِ دههٔ ۲۰۰۰، ردهٔ مدلِ «ماشینِ بردارِ پشتیبان» (Support Vector Machine یا SVM) محبوب‌ترین رویکرد برای یادگیریِ نظارت‌شدهٔ «آماده» بود، برای وقتی که دانشِ قبلیِ تخصصی دربارهٔ حوزه ندارید. آن جایگاه اکنون توسط شبکه‌های یادگیریِ عمیق و جنگل‌های تصادفی گرفته شده، اما SVMها سه ویژگیِ جذاب را حفظ کرده‌اند:

1. SVMها «جداکنندهٔ حاشیهٔ بیشینه» (Maximum Margin Separator) می‌سازند—مرزِ تصمیمی با بزرگ‌ترین فاصلهٔ ممکن تا نقاطِ مثال. این به آن‌ها کمک می‌کند خوب تعمیم دهند.
2. SVMها ابرصفحهٔ جداکنندهٔ خطی می‌سازند، اما توانایی جاسازیِ داده در فضایِ با ابعادِ بالاتر را دارند، با استفاده از به اصطلاح «حقهٔ هسته». اغلب، داده‌ای که در فضایِ ورودیِ اصلی قابلِ جداسازیِ خطی نیست به‌آسانی در فضایِ پربعد قابلِ جداسازی است.
3. SVMها ناپارامتری‌اند—ابرصفحهٔ جداکننده توسط مجموعه‌ای از نقاطِ مثال تعریف می‌شود، نه مجموعه‌ای از مقادیرِ پارامتر. اما در حالی که مدل‌های نزدیک‌ترین-همسایه باید همهٔ مثال‌ها را نگه دارند، مدلِ SVM فقط مثال‌هایی را نگه می‌دارد که به صفحهٔ جداکننده نزدیک‌ترین‌اند—معمولاً فقط چند برابرِ ثابتِ شمارِ ابعاد.

بدین‌ترتیب SVMها مزایای مدل‌های ناپارامتری و پارامتری را ترکیب می‌کنند: انعطافِ بازنماییِ توابعِ پیچیده را دارند، اما در برابرِ بیش‌برازش مقاوم‌اند.



شکل ۱۹.۲۱ – رده‌بندیِ ماشینِ بردارِ پشتیبان: (الف) دو رده از نقاط (دایره‌های بازِ نارنجی و سبزِ پر) و سه جداکنندهٔ خطیِ نامزد. (ب) جداکنندهٔ حاشیهٔ بیشینه (خطِ پهن)، در میانهٔ حاشیه (ناحیهٔ بینِ خط‌های خط‌چین) است. بردارهای پشتیبان (نقاط با دایره‌های سیاهِ بزرگ) نزدیک‌ترین مثال‌ها به جداکننده‌اند؛ در این‌جا سه‌تایند.

در شکل ۱۹.۲۱(الف) مسئلهٔ رده‌بندیِ دودویی با سه مرزِ تصمیمِ نامزد می‌بینیم، که هر یک جداکنندهٔ خطی است. هر یک با همهٔ مثال‌ها سازگار است، پس از دیدگاهِ زیانِ 0/1، هر یک به‌یکسان خوب می‌بود. رگرسیونِ لجستیک خطِ جداکننده‌ای می‌یافت؛ مکانِ دقیقِ خط به همهٔ نقاطِ مثال بستگی دارد.

بینشِ کلیدیِ SVMها این است که برخی مثال‌ها مهم‌تر از دیگران‌اند، و توجه به آن‌ها می‌تواند به تعمیمِ بهتر بینجامد. پایین‌ترینِ سه خطِ جداکننده در (الف) را در نظر بگیرید. بسیار به پنج مثالِ سیاه نزدیک می‌شود. اگرچه همهٔ مثال‌ها را درست رده‌بندی می‌کند و بنابراین زیان را کمینه می‌کند، باید شما را نگران کند که این‌همه مثال نزدیکِ خط‌اند؛ ممکن است مثال‌های سیاهِ دیگری معلوم شوند در سمتِ اشتباهِ خط بیفتند.

SVMها به این مسئله می‌پردازند: به‌جای کمینه‌کردنِ زیانِ تجربیِ موردانتظار روی دادهٔ آموزشی، SVMها تلاش می‌کنند زیانِ تعمیمِ موردانتظار را کمینه کنند. نمی‌دانیم نقاطِ هنوز-دیده‌نشده کجا ممکن است بیفتند، اما تحتِ فرضِ احتمالاتیِ کشیده‌شدنشان از همان توزیعِ مثال‌های قبلی، استدلال‌هایی از نظریهٔ یادگیریِ محاسباتی (بخش ۱۹.۵) هست که پیشنهاد می‌کنند زیانِ تعمیم را با انتخابِ جداکننده‌ای که از مثال‌هایی که تاکنون دیده‌ایم دورترین است کمینه کنیم. این جداکننده، که در شکل ۱۹.۲۱(ب) نشان داده شده، «جداکنندهٔ حاشیهٔ بیشینه» نامیده می‌شود. «حاشیه» (Margin) پهنایِ ناحیهٔ کران‌شده با خط‌های خط‌چین در شکل است—دو برابرِ فاصله از جداکننده تا نزدیک‌ترین نقطهٔ مثال.

حالا، چگونه این جداکننده را بیابیم؟ قبل از نشان‌دادنِ معادله‌ها، کمی نماد: به‌طورِ سنتی SVMها از قراردادِ برچسب‌های ردهٔ +1 و -1 استفاده می‌کنند، به‌جای +1 و 0 که تاکنون استفاده می‌کردیم. همچنین، در حالی که قبلاً عرض از مبدأ را در بردارِ وزنِ w می‌گذاشتیم (و مقدارِ دامیِ متناظرِ 1 در xj,0)، SVMها این کار را نمی‌کنند؛ عرض از مبدأ را پارامترِ جداگانه‌ای به نامِ b نگه می‌دارند.

با این در نظر، جداکننده به‌صورتِ مجموعهٔ نقاطِ {x : w·x + b = 0} تعریف می‌شود. می‌توانستیم فضایِ w و b را با نزولِ گرادیان جستجو کنیم تا پارامترهایی را بیابیم که حاشیه را بیشینه می‌کنند در حالی که همهٔ مثال‌ها را درست رده‌بندی می‌کنند. با این حال، معلوم می‌شود رویکردِ دیگری برای حلِ این مسئله هست. جزئیات را نشان نمی‌دهیم، اما فقط می‌گوییم بازنماییِ جایگزینی به نامِ «بازنماییِ دوگان» (Dual Representation) هست، که در آن راه‌حلِ بهینه با حلِ:

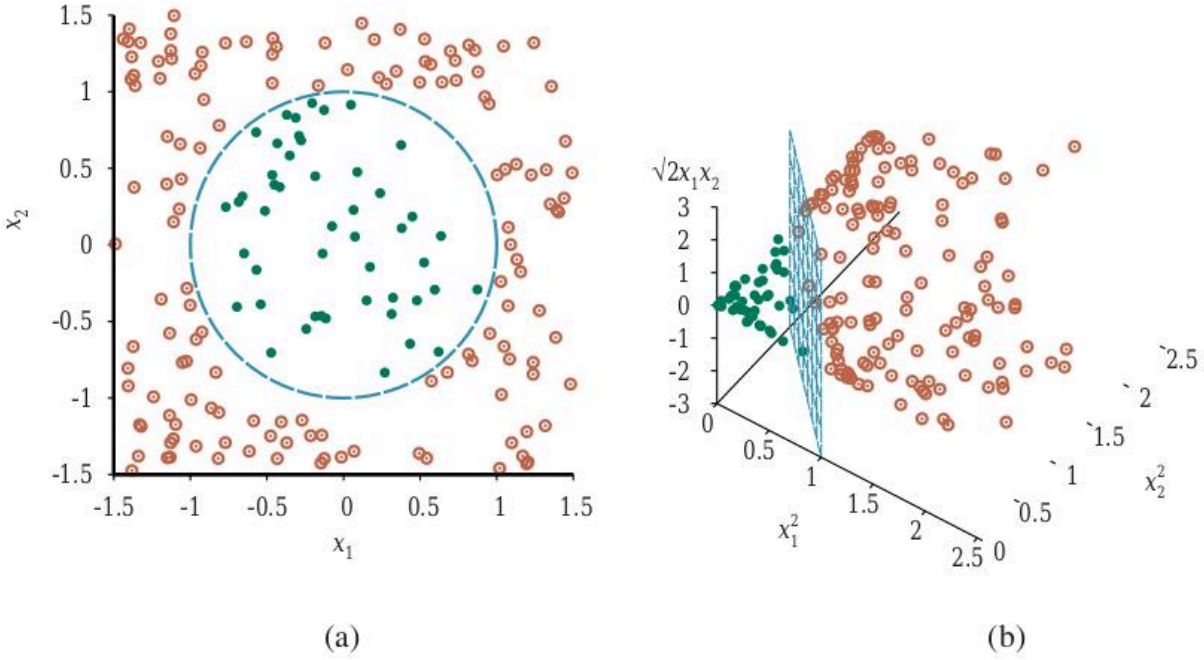
$$\text{argmax}_\alpha \sum_j \alpha_j - (1/2) \sum_{j,k} \alpha_j \alpha_k y_j y_k (x_j \cdot x_k) \qquad (19.10)$$

مشروط به قیدهای $\alpha_j \geq 0$ و $\sum_j \alpha_j y_j = 0$ یافت می‌شود. این مسئلهٔ «برنامه‌نویسیِ درجه‌دومی» (Quadratic Programming) است، که بسته‌های نرم‌افزاریِ خوبی برایش هست. وقتی بردارِ α را یافتیم، می‌توانیم با معادلهٔ $w = \sum_j \alpha_j y_j x_j$ به w برگردیم، یا می‌توانیم در بازنماییِ دوگان بمانیم. سه ویژگیِ مهمِ معادلهٔ (19.10) هست. نخست، عبارت محدب است؛ بیشینهٔ سراسریِ تکی دارد که می‌توان کارآمد یافت. دوم، داده فقط به شکلِ ضرب‌های نقطه‌ایِ جفت‌های نقطه وارد عبارت می‌شود. این ویژگیِ دوم همچنین برای معادلهٔ خودِ جداکننده صادق است؛ وقتی α_jهای بهینه محاسبه شدند، معادله:

$$h(x) = \text{sign}(\sum_j \alpha_j y_j (x \cdot x_j) - b) \qquad (19.11)$$

است.[11] ویژگیِ مهمِ نهاییِ این است که وزن‌های α_j مرتبط با هر نقطهٔ داده صفرند به‌جز برای «بردارهای پشتیبان» (Support Vectors)—نقاطِ نزدیک‌ترین به جداکننده. (آن‌ها «پشتیبان» نامیده می‌شوند، زیرا صفحهٔ جداکننده را «بالا نگه می‌دارند».) چون معمولاً بردارهای پشتیبان بسیار کم‌شمارتر از مثال‌ها هستند، SVMها برخی از مزایای مدل‌های پارامتری را به دست می‌آورند.

اگر مثال‌ها قابلِ جداسازیِ خطی نباشند چه؟ شکل ۱۹.۲۲(الف) فضایِ ورودیِ تعریف‌شده با ویژگی‌های $x = (x_1, x_2)$ را نشان می‌دهد، با مثال‌های مثبت ($y = +1$) درونِ ناحیهٔ دایره‌ای و مثال‌های منفی ($y = -1$) بیرون. بدیهی است، برای این مسئله جداکنندهٔ خطی نیست.



شکل ۱۹.۲۲ – (الف) مجموعهٔ آموزشیِ دوبعدی با مثال‌های مثبت به‌صورتِ دایره‌های سبزِ پر و مثال‌های منفی به‌صورتِ دایره‌های بازِ نارنجی. مرزِ تصمیمِ واقعی، $x1^2 + x2^2 \le 1$، نیز نشان داده شده. (ب) همان داده پس از نگاشت به فضایِ ورودیِ سه‌بعدی ($x1^2$, $x2^2$, $\sqrt{2}x1x2$). مرزِ تصمیمِ دایره‌ای در (الف) در سه بُعد مرزِ تصمیمِ خطی می‌شود.

حالا، فرض کنیم دادهٔ ورودی را بازبیان کنیم—یعنی هر بردارِ ورودیِ x را به بردارِ تازه‌ای از مقادیرِ ویژگی، F(x)، نگاشت کنیم. به‌طورِ خاص، از سه ویژگی استفاده کنیم:

$$f1 = x1^2, \qquad f2 = x2^2, \qquad f3 = \sqrt{2}\, x1x2 \qquad (19.12)$$

به‌زودی خواهیم دید این‌ها از کجا آمدند، اما فعلاً فقط به آنچه رخ می‌دهد نگاه کنید. شکل ۱۹.۲۲(ب) داده را در فضایِ تازهٔ سه‌بعدیِ تعریف‌شده توسط سه ویژگی نشان می‌دهد؛ داده در این فضا قابلِ جداسازیِ خطی است! این پدیده در واقع نسبتاً عمومی است: اگر داده به فضایی با بُعدِ به‌اندازهٔ کافی بالا نگاشت شود، تقریباً همیشه قابلِ جداسازیِ خطی خواهد بود—اگر از جهت‌های کافی به مجموعه‌ای از نقاط نگاه کنید، راهی برای ردیف‌کردنشان خواهید یافت. در این‌جا فقط از سه بُعد استفاده کردیم؛[12] تمرین SVME.۱۹ از شما می‌خواهد نشان دهید چهار بُعد برای جداسازیِ خطیِ دایره در هر جایِ صفحه (نه فقط در مبدأ) کافی است، و پنج بُعد برای جداسازیِ خطیِ هر بیضی کافی است. به‌طورِ کلی (با چند حالتِ خاصِ مستثنا) اگر N نقطهٔ داده داشته باشیم، همیشه در فضاهایِ با N-1 بُعد یا بیشتر قابلِ جداسازی خواهند بود (تمرین EMBE.۱۹).

حالا، معمولاً انتظارِ یافتنِ جداکنندهٔ خطی در فضایِ ورودیِ x را نداریم، اما می‌توانیم جداکننده‌های خطی را در فضایِ ویژگیِ پربعدِ F(x) صرفاً با جایگزینیِ xj·xk در معادلهٔ (19.10) با F(xj)·F(xk) بیابیم. این به‌خودی‌خود قابلِ توجه نیست—جایگزینیِ x با F(x) در هر الگوریتمِ یادگیری اثرِ لازم را دارد—اما ضربِ نقطه‌ای ویژگی‌های خاصی دارد. معلوم می‌شود

F(xj)·F(xk) اغلب بدونِ محاسبهٔ اولِ F برای هر نقطه قابلِ محاسبه است. در فضایِ ویژگیِ سه‌بعدیِ ما که با معادلهٔ (19.12) تعریف شده، کمی جبر نشان می‌دهد:

```
F(xj)·F(xk) = (xj·xk)²
```

(به همین دلیل است که 2√ در f3 هست.) عبارتِ $(xj·xk)^2$ «تابعِ هسته» (Kernel Function) نامیده می‌شود[13] و معمولاً K(xj,xk) نوشته می‌شود. تابعِ هسته را می‌توان روی جفت‌های دادهٔ ورودی برای ارزیابیِ ضرب‌های نقطه‌ای در فضایِ ویژگیِ متناظر اعمال کرد. پس، می‌توانیم جداکننده‌های خطی را در فضایِ ویژگیِ پربعدِ F(x) صرفاً با جایگزینیِ xj·xk در معادلهٔ (19.10) با تابعِ هستهٔ K(xj,xk) بیابیم. بدین‌ترتیب، می‌توانیم در فضایِ پربعد یاد بگیریم، اما فقط توابعِ هسته را محاسبه می‌کنیم، نه فهرستِ کاملِ ویژگی‌ها را برای هر نقطهٔ داده.

گامِ بعدی دیدنِ این است که در هستهٔ $K(xj,xk) = (xj·xk)^2$ چیزِ خاصی نیست. با فضایِ ویژگیِ پربعدِ خاصی متناظر است، اما توابعِ هستهٔ دیگر با فضاهایِ ویژگیِ دیگر متناظرند. نتیجهٔ ارجمندی در ریاضیات، «قضیهٔ مرسر» (Mercer's Theorem؛ 1909)، به ما می‌گوید هر تابعِ هستهٔ «معقول»[14] با فضایِ ویژگی‌ای متناظر است. این فضاهایِ ویژگی می‌توانند بسیار بزرگ باشند، حتی برای هسته‌های بی‌آزار به نظر. برای نمونه، «هستهٔ چندجمله‌ای» (Polynomial Kernel)، K(xj,xk) = (1 + xj·xk)^d، با فضایِ ویژگی‌ای متناظر است که بُعدش در d نمایی است. هستهٔ رایجِ دیگر گاوسی است: $K(xj,xk) = e^{(-\gamma|xj-xk|^2)}$.

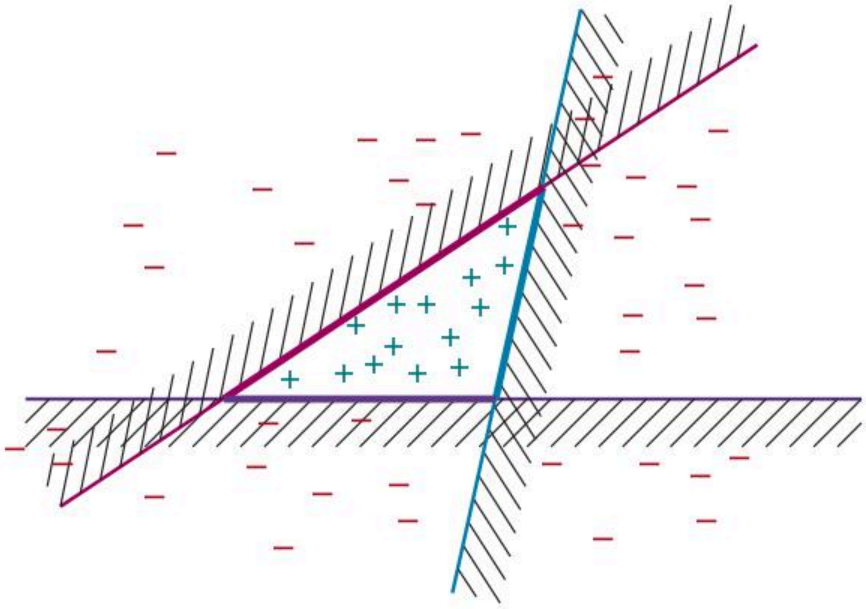
۱۹.۷.۶ حقهٔ هسته

این پس «حقهٔ هسته» (Kernel Trick) هوشمندانه است: با جای‌کردنِ این هسته‌ها در معادلهٔ (19.10)، جداکننده‌های خطیِ بهینه را می‌توان کارآمد در فضاهایِ ویژگی با میلیاردها (یا حتی بی‌نهایت) بُعد یافت. جداکننده‌های خطیِ حاصل، وقتی به فضایِ ورودیِ اصلی بازنگاشت می‌شوند، می‌توانند با مرزهایِ تصمیمِ غیرخطیِ خودسرانه‌ای پرنوسان بینِ مثال‌های مثبت و منفی متناظر باشند. در موردِ دادهٔ ذاتاً نویزی، ممکن است جداکنندهٔ خطی در فضایِ پربعدی نخواهیم. بلکه سطحِ تصمیمی در فضایِ کم‌بعد می‌خواهیم که برخی مثال‌های آموزشی را اشتباه رده‌بندی کند —یعنی «حاشیهٔ نرم» (Soft Margin).[17] حاشیهٔ نرم با معرفیِ متغیرهای سستی که اجازه می‌دهند برخی مثال‌ها در سمتِ اشتباهِ مرز باشند، اما با جریمه‌کردنِ آن‌ها، به دست می‌آید. SVMهای حاشیهٔ نرم (که گاهی «SVMهای C» نامیده می‌شوند) معمولاً شکلِ عملیِ استفاده‌شده در عمل‌اند.

۱۹.۸ یادگیریِ گروهی

تاکنون به روش‌های یادگیری نگاه کرده‌ایم که یک فرضیهٔ تکی از دادهٔ آموزشی می‌سازند. «یادگیریِ گروهی» (Ensemble Learning) از چند فرضیه استفاده می‌کند که «گروهِ» (Ensemble) فرضیه‌ها را تشکیل می‌دهند، و آن‌ها را برای پیش‌بینی‌های تازه ترکیب می‌کند. ایده‌ای که در پشتِ این است این است که با ترکیبِ چند فرضیه می‌توانیم دقتِ بهتری از هر فرضیهٔ فردی بگیریم. یک استدلالِ ریاضیِ اولیه برای ارزشِ گروه‌ها «قضیهٔ هیئت‌منصفهٔ کندورسه» (1785)

بود، که اثبات کرد اگر اعضای هیئت‌منصفه مستقل‌اند و هر عضوِ فردیِ هیئت‌منصفه دستِ‌کم ۵۰٪ شانسِ تصمیمِ درست دارد، آن‌گاه هرچه اعضای هیئت‌منصفه بیشتری بیفزایید، شانسِ تصمیمِ درستِ پرونده بیشتر می‌شود. اخیراً، یادگیریِ گروهی به فنِ فزاینده‌ای محبوب برای بهبودِ عملکردِ الگوریتم‌های یادگیری تبدیل شده.



شکل ۱۹.۲۳ – نشان‌دادنِ قدرتِ بیانیِ افزایش‌یافتهٔ حاصل از یادگیریِ گروهی. سه فرضیهٔ آستانهٔ خطی را می‌گیریم، که هر یک در سمتِ سایه‌نخورده مثبت رده‌بندی می‌کند، و هر مثالی را که توسط هر سه مثبت رده‌بندی شده مثبت رده‌بندی می‌کنیم. ناحیهٔ مثلثیِ حاصل فرضیه‌ای است که در فضایِ فرضیهٔ اصلی قابلِ بیان نیست.

۱۹.۸.۱ بیگینگ (Bagging)

در «بیگینگ» (Bagging)، K[15] مجموعهٔ آموزشیِ متمایز را با نمونه‌برداریِ با جایگزینی از مجموعهٔ آموزشیِ اصلی تولید می‌کنیم. یعنی، به‌صورتِ تصادفی N مثال از مجموعهٔ آموزشی برمی‌داریم، اما هر یک از آن انتخاب‌ها ممکن است مثالی باشد که قبلاً برداشته‌ایم. سپس الگوریتمِ یادگیریِ ماشینی‌مان را روی N مثال اجرا می‌کنیم تا فرضیه‌ای بگیریم. این فرایند را K بار تکرار می‌کنیم و K فرضیهٔ متفاوت می‌گیریم. سپس، وقتی خواسته می‌شود مقدارِ ورودیِ تازه‌ای پیش‌بینی شود، پیش‌بینی‌های همهٔ K فرضیه را تجمیع می‌کنیم. برای مسائلِ رده‌بندی، این به معنای گرفتنِ رأیِ کثرت است (رأیِ اکثریت برای رده‌بندیِ دودویی). برای مسائلِ رگرسیون، خروجیِ نهایی میانگین است:

```
h(x) = (1/K) ∑_{i=1}^{K} hi(x)
```

بیگینگ تمایل به کاهشِ واریانس دارد و وقتی داده محدود است یا مدلِ پایه دیده می‌شود بیش‌برازش می‌کند رویکردِ استانداردی است. بیگینگ را می‌توان روی هر رده‌ای از مدل اعمال کرد، اما معمول‌ترین استفاده‌اش با درخت‌های تصمیم است. مناسب است، زیرا درخت‌های تصمیم ناپایدارند: مجموعهٔ کمی متفاوت از مثال‌ها می‌تواند به درختِ وحشیانه‌ای متفاوت بینجامد. بیگینگ این واریانس را صاف می‌کند. اگر به چند رایانه دسترسی دارید، بیگینگ کارآمد است، زیرا فرضیه‌ها را می‌توان به‌صورتِ موازی محاسبه کرد.

۱۹.۸.۲ جنگل‌های تصادفی

متأسفانه، بیگینگِ درخت‌های تصمیم اغلب به ما K درختِ بسیار همبسته می‌انجامد. اگر ویژگی‌ای با بهرهٔ اطلاعاتیِ بسیار بالا باشد، محتمل است ریشهٔ بیشترِ درخت‌ها باشد. مدلِ «جنگلِ تصادفی» (Random Forest) شکلی از بیگینگِ درختِ تصمیم است که در آن گام‌های اضافی برای متنوع‌ترکردنِ گروهِ K درخت برمی‌داریم، تا واریانس کاهش یابد. جنگل‌های تصادفی را می‌توان برای رده‌بندی یا رگرسیون استفاده کرد.

ایدهٔ کلیدی به‌صورتِ تصادفی تغییردادنِ انتخاب‌های ویژگی (به‌جای مثال‌های آموزشی) است. در هر نقطهٔ تقسیم در ساختِ درخت، نمونه‌برداریِ تصادفیِ ویژگی‌ها را انتخاب می‌کنیم، و سپس محاسبه می‌کنیم کدام‌یک از آن‌ها بالاترین بهرهٔ اطلاعاتی را می‌دهد. اگر n ویژگی باشد، انتخابِ رایج نمونه‌ای از اندازهٔ $\sqrt{n}$ است. این باعث می‌شود هر درختِ فردی نسبت به درختِ ساخته‌شده با بیگینگِ استاندارد ضعیف‌تر باشد (کمتر احتمال دارد ویژگیِ مهم را داشته باشد)، اما درخت‌ها متنوع‌تر و در نتیجه کمتر همبسته‌اند. بنابراین گروه می‌تواند بهتر از هر درختِ فردی باشد.

بهبودِ بیشتر استفاده از تصادفی‌بودن در انتخابِ مقدارِ نقطهٔ تقسیم است: برای هر ویژگیِ انتخاب‌شده، چند مقدارِ نامزد را به‌صورتِ تصادفی از توزیعِ یکنواخت روی بازهٔ ویژگی نمونه‌برداری می‌کنیم. سپس مقداری را انتخاب می‌کنیم که بالاترین بهرهٔ اطلاعاتی را دارد. این محتمل‌تر می‌کند هر درختِ جنگل متفاوت باشد. درخت‌هایی که بدین‌گونه ساخته می‌شوند «درخت‌های بسیار تصادفی» (Extremely Randomized Trees یا ExtraTrees) نامیده می‌شوند.

جنگل‌های تصادفی کارآمد ایجاد می‌شوند. ممکن است فکر کنید ایجادِ گروهی از K درخت K برابر بیشتر طول می‌کشد، اما آن‌قدرها هم بد نیست، به سه دلیل: (الف) هر نقطهٔ تقسیم سریع‌تر اجرا می‌شود، زیرا ویژگی‌های کمتری را در نظر می‌گیریم، (ب) می‌توانیم گامِ هرسِ هر درختِ فردی را رد کنیم، زیرا گروه به‌عنوانِ کل بیش‌برازش را کاهش می‌دهد، و (ج) اگر اتفاقاً K رایانه در دسترس داشته باشیم، می‌توانیم همهٔ درخت‌ها را به‌صورتِ موازی بسازیم. برای نمونه، ادل کاتلر گزارش می‌کند برای مسئله‌ای با ۱۰۰ ویژگی، اگر فقط سه CPU داشته باشیم، می‌توانیم جنگلی از K = 100 درخت را تقریباً در همان زمانی که ایجادِ درختِ تصمیمِ تکی روی CPU تکی می‌برد رشد دهیم. همهٔ ابرپارامترهای جنگل‌های تصادفی را می‌توان با اعتبارسنجیِ متقاطع آموزش داد: شمارِ درخت‌های K، شمارِ مثال‌های استفاده‌شده توسط هر درختِ N (اغلب به‌صورتِ درصدی از مجموعهٔ دادهٔ کامل بیان می‌شود)، شمارِ ویژگی‌های استفاده‌شده در هر نقطهٔ تقسیم (اغلب به‌صورتِ تابعی از شمارِ کلِ ویژگی‌ها، مانندِ $\sqrt{n}$)، و شمارِ نقاطِ تقسیمِ تصادفیِ امتحان‌شده اگر از ExtraTrees استفاده می‌کنیم. به‌جای استراتژیِ منظمِ اعتبارسنجیِ متقاطع، می‌توانیم «خطایِ بیرون-از-کیسه» (Out-of-Bag Error) را اندازه بگیریم: میانگینِ خطا روی هر مثال، با استفاده از فقط درخت‌هایی که مجموعهٔ مثال‌شان شاملِ آن مثالِ خاص نبود.

هشدار داده شده‌ایم مدل‌های پیچیده‌تر می‌توانند مستعدِ بیش‌برازش باشند، و مشاهده کردیم برای درخت‌های تصمیم چنین است، جایی که دریافتیم هرس پاسخِ جلوگیری از بیش‌برازش است. جنگل‌های تصادفی مدل‌های پیچیده و هرس‌نشده‌اند. با این حال، در برابرِ بیش‌برازش مقاوم‌اند. با افزایشِ ظرفیت با افزودنِ درخت‌های بیشتر به جنگل، آن‌ها تمایل به بهبودِ نرخِ خطای مجموعهٔ اعتبارسنجی دارند. منحنی معمولاً مانندِ شکل ۱۹.۹(ب) است، نه (الف).

بریمن (2001) اثباتِ ریاضی می‌دهد که (در تقریباً همهٔ موارد) با افزودنِ درخت‌های بیشتر به جنگل، خطا همگرا می‌شود؛ رشد نمی‌کند. یکی از راه‌های فکرکردن به آن این است که انتخابِ تصادفیِ ویژگی‌ها انواعِ درخت می‌دهد، بدین‌ترتیب واریانس کاهش می‌یابد، اما چون لازم نیست درخت‌ها هرس شوند، می‌توانند کلِ فضایِ ورودی را با وضوحِ بالاتر پوشش دهند. شماری از درخت‌ها می‌توانند مواردِ یکتایی را پوشش دهند که فقط چند بار در داده ظاهر می‌شوند، و رأی‌هایشان می‌تواند تعیین‌کننده باشد، اما وقتی اعمال نمی‌شوند می‌توانند رأی‌شان را ببازند. با این حال، جنگل‌های تصادفی کاملاً در برابرِ بیش‌برازش ایمن نیستند. اگرچه خطا در حدِّ نهایی نمی‌تواند افزایش یابد، بدان معنا نیست خطا به صفر خواهد رفت.

جنگل‌های تصادفی در طیفِ وسیعی از مسائلِ کاربردی بسیار موفق بوده‌اند. در مسابقاتِ علومِ دادهٔ کگل، محبوب‌ترین رویکردِ تیم‌های برنده از ۲۰۱۱ تا ۲۰۱۴ بودند، و تا به امروز رویکردِ رایجی باقی مانده‌اند (هرچند یادگیریِ عمیق و تقویتِ گرادیان در میانِ برندگانِ اخیر رایج‌تر شده‌اند). بستهٔ randomForest در R موردعلاقهٔ خاصی بوده. در مالی، جنگل‌های تصادفی برای پیش‌بینیِ نکولِ کارتِ اعتباری، پیش‌بینیِ درآمدِ خانوار و قیمت‌گذاریِ اختیارِ معامله استفاده شده‌اند. کاربردهای مکانیکی شاملِ تشخیصِ عیبِ ماشین و سنجشِ از دور است. کاربردهای بیوانفورماتیکی و پزشکی شاملِ رتینوپاتیِ دیابتی، بیانِ ژنِ ریزآرایه، تحلیلِ بیانِ پروتئینِ طیفِ جرمی، کشفِ نشانگرِ زیستی و پیش‌بینیِ تعاملِ پروتئین–پروتئین است.

۱۹.۸.۳ انباشتن (Stacking)

در حالی که بیگینگ چند مدلِ پایه از همان ردهٔ مدل را که روی دادهٔ متفاوت آموزش دیده‌اند ترکیب می‌کند، فنِ «تعمیمِ انباشته» (Stacked Generalization؛ یا به‌اختصار «انباشتن» (Stacking)) چند مدلِ پایه از رده‌های مدلِ متفاوت را که روی همان داده آموزش دیده‌اند ترکیب می‌کند. برای نمونه، فرض کنیم مجموعهٔ دادهٔ رستوران به ما داده شده، که ردیفِ نخستش این‌جاست:

```
y1 = Yes    :x1 = Yes, No, No, Yes, Some, $$$, No, Yes, French, 0–10
```

داده را به مجموعه‌های آموزشی، اعتبارسنجی و آزمون جدا می‌کنیم و از مجموعهٔ آموزشی برای آموزشِ، مثلاً، سه مدلِ پایهٔ جداگانه—مدلِ SVM، مدلِ رگرسیونِ لجستیک و مدلِ درختِ تصمیم—استفاده می‌کنیم. در گامِ بعدی مجموعهٔ دادهٔ اعتبارسنجی را می‌گیریم و هر ردیف را با پیش‌بینی‌های انجام‌شده از سه مدلِ پایه تقویت می‌کنیم، و ردیف‌هایی مانندِ این به ما می‌دهد (که در آن پیش‌بینی‌ها پررنگ‌اند):

```
y2 = No    :x2 = Yes, No, No, Yes, Full, $, No, No, Thai, 30–60, Yes, No, No
```

از این مجموعهٔ اعتبارسنجی برای آموزشِ مدلِ گروهیِ تازه‌ای استفاده می‌کنیم، بگوییم مدلِ رگرسیونِ لجستیک (اما لازم نیست یکی از رده‌های مدلِ پایه باشد). مدلِ گروهی می‌تواند از پیش‌بینی‌ها و دادهٔ اصلی هرطور که مناسب می‌داند استفاده کند. ممکن است میانگینِ وزن‌داری از مدل‌های پایه یاد بگیرد، برای نمونه اینکه پیش‌بینی‌ها باید با نسبتِ ۵۰٪:۳۰٪:۲۰٪ وزن‌گذاری شوند. یا ممکن است تعامل‌های غیرخطی بینِ داده و پیش‌بینی‌ها یاد بگیرد، شاید وقتی زمانِ

انتظار طولانی است به مدلِ SVM بیشتر اعتماد کند. از همان دادهٔ آموزشی برای آموزشِ هر یک از مدل‌های پایه استفاده کردیم، و سپس از دادهٔ اعتبارسنجیِ کنارگذاشته‌شده (به‌علاوهٔ پیش‌بینی‌ها) برای آموزشِ مدلِ گروهی استفاده کردیم. در صورتِ تمایل می‌توان از اعتبارسنجیِ متقاطع نیز استفاده کرد.

این روش «انباشتن» نامیده می‌شود، زیرا می‌توان آن را لایه‌ای از مدل‌های پایه با مدلِ گروهیِ انباشته‌شده بالای آن دانست که روی خروجیِ مدل‌های پایه عمل می‌کند. در واقع، می‌توان چند لایه انباشت، که هر یک روی خروجیِ لایهٔ قبلی عمل می‌کند.

انباشتن اریبی را کاهش می‌دهد و معمولاً به عملکردی بهتر از هر یک از مدل‌های پایهٔ فردی می‌انجامد. انباشتن اغلب توسط تیم‌های برنده در مسابقاتِ علومِ داده (مانندِ کگل و جامِ KDD) استفاده می‌شود، زیرا افراد می‌توانند مستقلانه کار کنند، هر یک مدلِ پایهٔ خودش را پالایش کند، و سپس برای ساختِ مدلِ گروهیِ انباشتهٔ نهایی دور هم جمع شوند.

۱۹.۸.۴ تقویت (Boosting)

محبوب‌ترین روشِ گروهی «تقویت» (Boosting) نامیده می‌شود. برای فهمِ چگونگیِ کارش، نخست باید ایدهٔ «مجموعهٔ آموزشیِ وزندار» (Weighted Training Set) را معرفی کنیم، که در آن هر مثال وزنِ مرتبطِ $w_j \geq 0$ دارد که توصیف می‌کند مثال چقدر باید در طولِ آموزش به حساب بیاید. برای نمونه، اگر مثالی وزنِ 3 و بقیهٔ مثال‌ها وزنِ 1 داشتند، معادلِ ۳ نسخه از آن مثال در مجموعهٔ آموزشی بود.

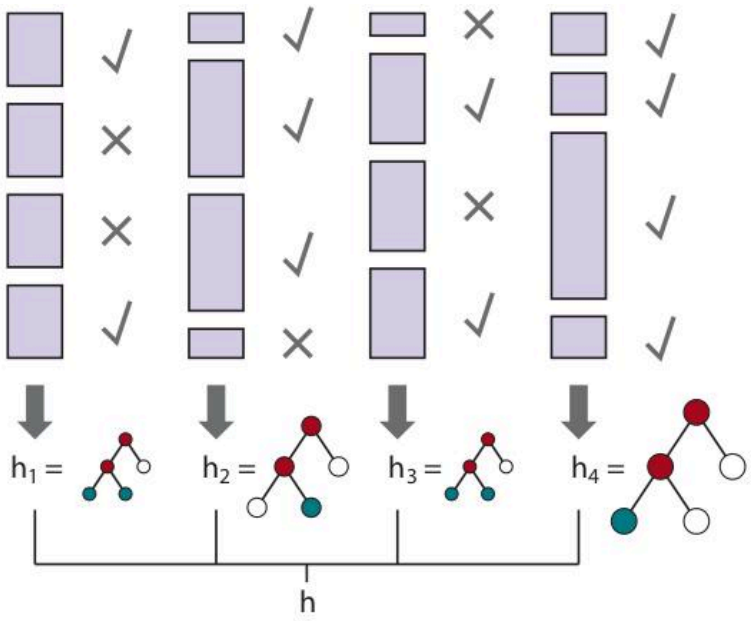
تقویت با وزن‌های برابرِ $w_j = 1$ برای همهٔ مثال‌ها شروع می‌شود. از این مجموعهٔ آموزشی، نخستین فرضیه، h_1، را تولید می‌کند. به‌طورِ کلی، h_1 برخی از مثال‌های آموزشی را درست و برخی را نادرست رده‌بندی می‌کند. می‌خواهیم فرضیهٔ بعدی روی مثال‌های اشتباه‌رده‌بندی‌شده بهتر عمل کند، پس وزنِ آن‌ها را افزایش می‌دهیم در حالی که وزنِ مثال‌های درست‌رده‌بندی‌شده را کاهش می‌دهیم. از این مجموعهٔ آموزشیِ وزندارِ تازه، فرضیهٔ h_2 را تولید می‌کنیم. فرایند بدین‌گونه ادامه می‌یابد تا K فرضیه تولید کرده باشیم، که در آن K ورودیِ الگوریتمِ تقویت است.

مثال‌هایی که رده‌بندی‌شان دشوار است وزن‌های فزاینده‌ای خواهند گرفت تا الگوریتم مجبور شود فرضیه‌ای بسازد که آن‌ها را درست رده‌بندی کند. توجه کنید این الگوریتمِ حریصانه‌ای است به این معنا که پس‌گرد نمی‌کند؛ وقتی فرضیه‌ای h_i را انتخاب کرد هرگز آن انتخاب را پس نمی‌گیرد؛ بلکه فرضیه‌های تازه می‌افزاید. همچنین الگوریتمِ ترتیبی است، پس نمی‌توانیم همهٔ فرضیه‌ها را به‌صورتِ موازی محاسبه کنیم، چنان‌که با بیگینگ می‌کردیم.

گروهِ نهایی به هر فرضیه رأی می‌دهد، مانندِ بیگینگ، به‌جز اینکه هر فرضیه شمارِ وزن‌داری از رأی‌ها می‌گیرد—فرضیه‌هایی که روی مجموعه‌های آموزشیِ وزندارِ مربوطشان بهتر عمل کردند وزنِ رأیِ بیشتری می‌گیرند. برای رگرسیون یا رده‌بندیِ دودویی داریم:

$$h(x) = \sum_{i=1}^{K} z_i h_i(x)$$

که در آن zi وزنِ i-امین فرضیه است. (این وزن‌گذاریِ فرضیه‌ها از وزن‌گذاریِ مثال‌ها متمایز است.) شکل ۱۹.۲۴ نشان می‌دهد الگوریتم چگونه از نظرِ مفهومی کار می‌کند. گونه‌های بسیاری از ایدهٔ پایهٔ تقویت هست، با راه‌های متفاوتِ تنظیمِ وزن‌های مثال و ترکیبِ فرضیه‌ها. همهٔ گونه‌ها این ایدهٔ عمومی را شریک‌اند که مثال‌های دشوار با رفتن از فرضیه‌ای به فرضیهٔ بعدی وزنِ بیشتری می‌گیرند. مانندِ روش‌های یادگیریِ بیزی که در فصل ۲۰ خواهیم دید، آن‌ها همچنین به فرضیه‌های دقیق‌تر وزنِ بیشتری می‌دهند.



شکل ۱۹.۲۴ – الگوریتمِ تقویت چگونه کار می‌کند. هر مستطیلِ سایه‌دار با مثالی متناظر است؛ ارتفاعِ مستطیل با وزن متناظر است. تیک‌ها و ضربدرها نشان می‌دهند آیا مثال توسط فرضیهٔ کنونی درست رده‌بندی شده. اندازهٔ درختِ تصمیم وزنِ آن فرضیه را در گروهِ نهایی نشان می‌دهد.

الگوریتمِ خاصی به نامِ ADABOOST در شکل ۱۹.۲۵ نشان داده شده. معمولاً با درخت‌های تصمیم به‌عنوانِ فرضیه‌های مؤلفه‌ای اعمال می‌شود؛ اغلب درخت‌ها از نظرِ اندازه محدود می‌شوند.

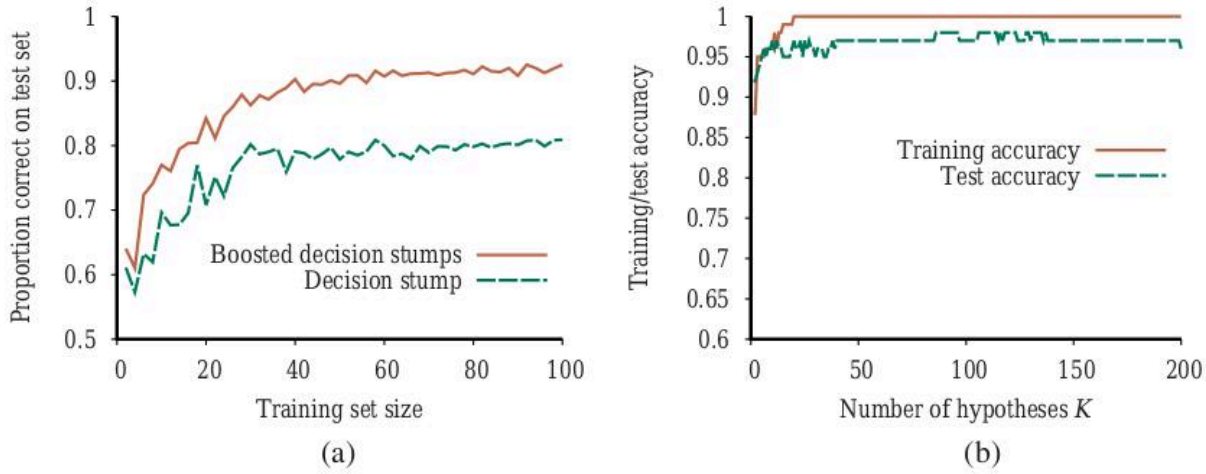
```
function ADABOOST(examples, L, K) returns a hypothesis
  inputs: examples, a set of N labeled examples (x1,y1),...,(xN,yN)
          L, the learning algorithm
          K, the number of hypotheses in the ensemble
  local variables: w, a vector of N example weights, initially all 1/N
                   h, a vector of K hypotheses
                   z, a vector of K hypothesis weights
  ϱ ← a small positive number, to avoid division by zero
  for k = 1 to K:
      h[k] ← L(examples, w)
      error ← 0
      for j = 1 to N:                    // compute total error for h[k]
          if h[k](xj) ≠ yj then error ← error + w[j]
      if error > 1/2 then break
      error ← min(error, 1 − ϱ)
      for j = 1 to N:                    // give more weight to examples misclassified by h[k]
          if h[k](xj) = yj then w[j] ← w[j] · error/(1 − error)
      w ← NORMALIZE(w)
      z[k] ← (1/2) log((1 − error)/error)   // give more weight to accurate h[k]
  return Function(x): ∑zi·hi(x)
```

شکل ۱۹.۲۵ — گونهٔ ADABOOST روشِ تقویت برای یادگیریِ گروهی. الگوریتم با وزن‌دهیِ مجددِ متوالیِ مثال‌های آموزشی فرضیه‌ها را تولید می‌کند. تابعِ WEIGHTED-MAJORITY فرضیه‌ای تولید می‌کند که مقدارِ خروجیِ با بالاترین رأی از فرضیه‌های h، با رأی‌های وزن‌گذاری‌شده با z، برمی‌گرداند. برای مسائلِ رگرسیون، یا برای رده‌بندیِ دودویی با دو ردهٔ −1 و 1، این ∑z[k]·h[k] k است.

ADABOOST ویژگیِ بسیار مهمی دارد: اگر الگوریتمِ یادگیریِ ورودیِ L «الگوریتمِ یادگیریِ ضعیف» (Weak Learning Algorithm) باشد—بدان معنا که L همیشه فرضیه‌ای با دقتِ روی مجموعهٔ آموزشیِ کمی بهتر از حدسِ تصادفی برمی‌گرداند (یعنی برای رده‌بندیِ بولی ۵۰٪+ϱ)—آن‌گاه ADABOOST فرضیه‌ای برمی‌گرداند که برای K به‌اندازهٔ کافی بزرگ، دادهٔ آموزشی را کاملاً رده‌بندی می‌کند. بدین‌ترتیب، الگوریتم دقتِ الگوریتمِ یادگیریِ اصلی را روی دادهٔ آموزشی «تقویت» می‌کند. به بیانِ دیگر، تقویت می‌تواند بر هر میزانِ اریبی در مدلِ پایه غلبه کند، تا زمانی که مدلِ پایه ϱ بهتر از حدسِ تصادفی باشد. (در شبه‌کدِ ما، اگر فرضیه‌ای بدتر از تصادفی بگیریم تولیدِ فرضیه را متوقف می‌کنیم.) این نتیجه مهم نیست فضایِ فرضیهٔ اصلی چقدر نابیان باشد و مهم نیست تابعِ درحالِ‌یادگیری چقدر پیچیده باشد. فرمول‌های دقیقِ وزن‌ها در شکل ۱۹.۲۵ (با error/(1-error) و مانندِ آن) طوری انتخاب شده‌اند که اثباتِ این ویژگی آسان باشد (فرند و شاپایر، 1996 را ببینید). البته، این ویژگی دقت روی مثال‌های قبلاً دیده‌نشده را تضمین نمی‌کند.

ببینیم تقویت روی دادهٔ رستوران چقدر خوب عمل می‌کند. به‌عنوانِ فضایِ فرضیهٔ اصلی‌مان ردهٔ «کنده‌های تصمیم» (Decision Stumps) را انتخاب می‌کنیم، که درخت‌های تصمیم با فقط یک آزمون، در ریشه‌اند. منحنیِ پایینی در شکل ۱۹.۲۶(الف) نشان می‌دهد کنده‌های تصمیمِ تقویت‌نشده برای این مجموعهٔ داده چندان مؤثر نیستند، و روی ۱۰۰ مثال آموزشی به عملکردِ پیش‌بینیِ فقط ۸۱٪ می‌رسند. وقتی تقویت اعمال می‌شود (با K = 5)، عملکرد بهتر است و پس از ۱۰۰ مثال به ۹۳٪ می‌رسد. اتفاقِ جالبی با افزایشِ اندازهٔ گروهِ K می‌افتد. شکل ۱۹.۲۶(ب) عملکردِ مجموعهٔ آموزشی (روی ۱۰۰ مثال) را به‌عنوانِ تابعی از K نشان می‌دهد. توجه کنید خطا وقتی K برابر 20 است به صفر می‌رسد؛ یعنی ترکیبِ وزن‌دار-اکثریتیِ ۲۰ کندهٔ تصمیم برای برازشِ دقیقِ ۱۰۰ مثال کافی است—این نقطهٔ درون‌یابی است. با افزودنِ کنده‌های

بیشتر به گروه، خطا در صفر می‌ماند. نمودار همچنین نشان می‌دهد عملکردِ مجموعهٔ آزمون مدت‌ها پس از رسیدنِ خطایِ مجموعهٔ آموزشی به صفر همچنان افزایش می‌یابد. در K = 20، عملکردِ آزمون 0.95 است (یا خطای 0.05)، و عملکرد تا دیرترِ K = 137 به 0.98 افزایش می‌یابد، قبل از اینکه به‌تدریج به 0.95 افت کند.



شکل ۱۹.۲۶ — (الف) نموداری که عملکردِ کنده‌های تصمیمِ تقویت‌شده با K = 5 را در برابرِ کنده‌های تصمیمِ تقویت‌نشده روی دادهٔ رستوران نشان می‌دهد. (ب) نسبتِ درست روی مجموعهٔ آموزشی و مجموعهٔ آزمون به‌عنوانِ تابعی از K، شمارِ فرضیه‌ها در گروه. توجه کنید دقتِ مجموعهٔ آزمون حتی پس از رسیدنِ دقتِ آموزشی به 1، یعنی پس از برازشِ دقیقِ داده توسط گروه، کمی بهبود می‌یابد.

این یافته، که در میانِ مجموعه‌های داده و فضاهای فرضیه کاملاً استوار است، وقتی نخستین بار مشاهده شد کاملاً غافلگیرکننده بود. تیغِ آوکام به ما می‌گوید فرضیه‌ها را پیچیده‌تر از لازم نسازیم، اما نمودار به ما می‌گوید پیش‌بینی‌ها با پیچیده‌ترشدنِ فرضیهٔ گروهی بهبود می‌یابند! تبیین‌های گوناگونی برای این پیشنهاد شده. یکی از دیدگاه‌ها این است که تقویت یادگیریِ بیزی را تقریب می‌زند (فصل 20 را ببینید)، که می‌توان نشان داد الگوریتمِ یادگیریِ بهینه‌ای است، و تقریب با افزودنِ فرضیه‌های بیشتر بهبود می‌یابد. تبیینِ ممکنِ دیگر این است که افزودنِ فرضیه‌های بیشتر گروه را در تمایزش بینِ مثال‌های مثبت و منفی مطمئن‌تر می‌کند، که هنگامِ رده‌بندیِ مثال‌های تازه کمکش می‌کند.

۱۹.۸.۵ تقویتِ گرادیان

برای رگرسیون و رده‌بندیِ دادهٔ جدولیِ عاملی، «تقویتِ گرادیان» (Gradient Boosting)، که گاهی «ماشین‌های تقویتِ گرادیان» (GBM) یا «درخت‌های رگرسیونِ تقویت‌شده با گرادیان» (GBRT) نامیده می‌شود، روشِ بسیار محبوبی شده. چنان‌که نام دلالت دارد، تقویتِ گرادیان شکلی از تقویت با استفاده از نزولِ گرادیان است. به یاد آورید در ADABOOST، با فرضیهٔ تکیِ h1 شروع می‌کنیم و آن را با توالیِ فرضیه‌هایی تقویت می‌کنیم که توجهِ خاصی به مثال‌هایی دارند که فرضیه‌های قبلی اشتباه گرفتند. در تقویتِ گرادیان نیز فرضیه‌های تقویتِ تازه می‌افزاییم، که توجه می‌کنند نه به مثال‌های خاص، بلکه به گرادیانِ بینِ پاسخ‌های درست و پاسخ‌های داده‌شده توسط فرضیه‌های قبلی.

مانندِ سایر الگوریتم‌هایی که از نزولِ گرادیان استفاده کردند، با تابعِ زیانِ مشتق‌پذیری شروع می‌کنیم؛ ممکن است برای رگرسیون از خطایِ مربع، یا برای رده‌بندی از زیانِ لگاریتمی استفاده کنیم. مانندِ ADABOOST، سپس درختِ تصمیم می‌سازیم. در بخش ۱۹.۶.۲، از نزولِ گرادیان برای کمینه‌کردنِ پارامترهای مدل استفاده کردیم—زیان را محاسبه می‌کنیم و

پارامترها را در جهتِ زیانِ کمتر به‌روزرسانی می‌کنیم. با تقویتِ گرادیان، پارامترهای مدلِ موجود را به‌روزرسانی نمی‌کنیم، بلکه پارامترهای درختِ بعدی را به‌روزرسانی می‌کنیم—اما باید این کار را به‌گونه‌ای کنیم که زیان را با حرکت در جهتِ درست در امتدادِ گرادیان کاهش دهد.

مانندِ مدل‌هایی که در بخش ۱۹.۴.۳ دیدیم، منظم‌سازی می‌تواند به جلوگیری از بیش‌برازش کمک کند. آن می‌تواند به شکلِ محدودکردنِ شمارِ درخت‌ها یا اندازه‌شان (بر حسبِ عمق یا شمارِ گره‌هایشان) باشد. می‌تواند از نرخِ یادگیریِ α بیاید، که می‌گوید چقدر در جهتِ گرادیان حرکت کنیم؛ مقادیرِ در بازهٔ 0.1 تا 0.3 رایج‌اند، و هرچه نرخِ یادگیری کوچک‌تر باشد، درخت‌های بیشتری در گروه نیاز خواهیم داشت.

تقویتِ گرادیان در بستهٔ محبوبِ XGBOOST (تقویتِ گرادیانِ افراطی) پیاده‌سازی شده، که به‌طورِ منظم برای کاربردهای بزرگ‌مقیاس و تیم‌های برنده در مسابقاتِ علومِ داده استفاده می‌شود. نسخه‌های بعدی مانندِ LightGBM و CatBoost نیز محبوبیت یافته‌اند.

۱۹.۸.۶ یادگیریِ برخط

تاکنون، همهٔ آنچه در این فصل انجام داده‌ایم بر این فرض تکیه داشته که داده‌ها i.i.d (مستقل و هم‌توزیع) هستند. از یک طرف، فرضِ حساسی است: اگر آینده هیچ شباهتی به گذشته نداشته باشد، پس چگونه می‌توانیم چیزی را پیش‌بینی کنیم؟ از طرف دیگر، فرضِ بیش‌ازحدِ قوی‌ای است: می‌دانیم همبستگی‌هایی بینِ گذشته و آینده هست، و در سناریوهای پیچیده بعید است همهٔ داده‌ای را در بر بگیریم که آینده را با توجه به داده مستقل از گذشته کند.

در این بخش بررسی می‌کنیم وقتی داده i.i.d نیست—وقتی می‌تواند در طولِ زمان تغییر کند—چه کنیم. در این صورت، زمانِ پیش‌بینی اهمیت دارد، پس دیدگاهی به نامِ «یادگیریِ برخط» (Online Learning) اتخاذ می‌کنیم: عاملی ورودیِ x_j را از طبیعت دریافت می‌کند، y_j متناظر را پیش‌بینی می‌کند، و سپس پاسخِ درست به او گفته می‌شود. سپس فرایند با x_{j+1} تکرار می‌شود، و همین‌طور. ممکن است فکر کنید این تکلیف ناامیدکننده است—اگر طبیعت دشمن باشد، همهٔ پیش‌بینی‌ها ممکن است غلط باشند. معلوم می‌شود تضمین‌هایی می‌توانیم بکنیم.

وضعیتی را در نظر بگیریم که ورودیِ ما از پیش‌بینی‌های «هیئتی از کارشناسان» تشکیل شده. برای نمونه، هر کارشناسِ مالی ممکن است پیش‌بینی کند سهامِ بازار بالا یا پایین می‌رود، و تکلیفِ ما تجمیعِ آن پیش‌بینی‌ها و پیش‌بینیِ خودمان است. یکی از راه‌های انجامِ این، پیگیریِ عملکردِ هر کارشناس و انتخابِ باور به آن‌ها به نسبتِ عملکردِ گذشته‌شان است. این «الگوریتمِ اکثریتِ وزندارِ تصادفی» (Randomized Weighted Majority Algorithm) نامیده می‌شود. می‌توانیم آن را صوری‌تر توصیف کنیم:

```
مجموعه‌ای از وزن‌ها {w1,...,wK} را همه 1 مقداردهی کن.
برای هر مسئله‌ای که باید حل شود:
  1. پیش‌بینی‌های {ŷ1,...,ŷK} را از کارشناسان دریافت کن.
  2. کارشناسِ k* را به‌صورتِ تصادفی به نسبتِ وزنش انتخاب کن: P(k) = wk.
  3. ŷk* را به‌عنوانِ پاسخِ این مسئله ارائه کن.
  4. پاسخِ درستِ y را دریافت کن.
  5. برای هر کارشناسِ k که ŷk ≠ y، wk ← β·wk به‌روزرسانی کن.
  6. وزن‌ها را نرمال کن تا ∑k wk = 1.
```

در این‌جا β عددی است، 0 < β < 1، که می‌گوید برای هر اشتباه چقدر کارشناس را جریمه کنیم.

موفقیتِ این الگوریتم را بر حسبِ «پشیمانی» (Regret) اندازه می‌گیریم، که به‌عنوانِ شمارِ اشتباهاتِ اضافی‌ای که در مقایسه با کارشناسی که، در نگاهِ گذشته، بهترین سابقهٔ پیش‌بینی را داشت، می‌کنیم تعریف می‌شود. M* شمارِ اشتباهاتِ بهترین کارشناس باشد. سپس شمارِ اشتباهات، M، توسط الگوریتمِ اکثریتِ وزندارِ تصادفی با:

```
M < (M* ln(1/β) + ln K) / (1 − β)
```

کران‌دار می‌شود. این کران برای هر توالیِ مثالی، حتی توالی‌های انتخاب‌شده توسط دشمنانی که می‌خواهند بدترین کار را بکنند، برقرار است. به‌طورِ خاص، وقتی K = 10 کارشناس هست، اگر β = 1/2 انتخاب کنیم، شمارِ اشتباهات‌مان با 1.39M* + 4.6 کران‌دار می‌شود، و اگر β = 3/4 با 1.15M* + 9.2. به‌طورِ کلی، اگر β نزدیکِ 1 باشد، در درازمدت به تغییر پاسخ‌گو هستیم؛ اگر بهترین کارشناس تغییر کند، خیلی زود متوجهش می‌شویم. با این حال، در آغاز جریمه می‌پردازیم، وقتی با همهٔ کارشناسان به‌یکسان اعتماد شده شروع می‌کنیم؛ ممکن است مدتِ زیادی توصیهٔ کارشناسانِ بد را بپذیریم. وقتی β نزدیک‌تر به 0 است، این دو عامل برعکس می‌شوند. توجه کنید می‌توانیم β را طوری انتخاب کنیم که M در درازمدت مجانباً به M* نزدیک شود؛ این «یادگیریِ بدون-پشیمانی» (No-Regret Learning) نامیده می‌شود (زیرا میانگینِ میزانِ پشیمانی در هر آزمایش با افزایشِ شمارِ آزمایش‌ها به 0 می‌گراید).

یادگیریِ برخط وقتی داده ممکن است به‌سرعت در طولِ زمان تغییر کند مفید است. همچنین برای کاربردهایی مفید است که مجموعهٔ دادهٔ بزرگِ دائماً در حالِ رشد دارند، حتی اگر تغییرها تدریجی باشند. برای نمونه، با مجموعه‌ای از میلیون‌ها تصویرِ وب، نمی‌خواهید هر بار که تصویرِ تکی افزوده می‌شود از صفر بازآموزی کنید. عملی‌تر خواهد بود الگوریتمِ برخطی داشته باشید که به تصاویر اجازه می‌دهد به‌صورتِ افزایشی افزوده شوند. برای بیشترِ الگوریتم‌های یادگیریِ مبتنی بر کمینه‌کردنِ زیان، نسخهٔ برخطی مبتنی بر کمینه‌کردنِ پشیمانی هست. بسیاری از این الگوریتم‌های برخط با کران‌های تضمینیِ پشیمانی می‌آیند.

ممکن است شگفت‌آور به نظر برسد کران‌های چنان دقیقی بر اینکه در مقایسه با هیئتی از کارشناسان چقدر خوب عمل می‌کنیم هست. حتی شگفت‌آورتر این است که وقتی چنین هیئت‌هایی برای پیشگویی دربارهٔ رقابت‌های سیاسی یا رویدادهای ورزشی گرد هم می‌آیند، مردمِ بیننده این‌قدر مایل‌اند به پیش‌بینی‌هایشان گوش دهند و این‌قدر بی‌علاقه‌اند به دانستنِ نرخ‌های خطایشان.

۱۹.۹ توسعهٔ سامانه‌های یادگیریِ ماشینی

در این فصل بر توضیحِ نظریهٔ یادگیریِ ماشینی تمرکز کرده‌ایم. عملِ استفاده از یادگیریِ ماشینی برای حلِ مسائلِ عملی رشته‌ای جداگانه است. در ۵۰ سالِ گذشته، صنعتِ نرم‌افزار روش‌شناسیِ توسعهٔ نرم‌افزاری را تکامل داده که محتمل‌تر می‌کند پروژهٔ نرم‌افزاریِ (سنتی) موفق باشد. اما ما هنوز در مراحلِ آغازینِ تعریفِ روش‌شناسی برای پروژه‌های یادگیریِ ماشینی هستیم؛ ابزارها و فنون به‌خوبی توسعه نیافته‌اند. در این‌جا تفکیکی از گام‌های نمونه‌ای در فرایند است.

۱۹.۹.۱ صورت‌بندیِ مسئله

گامِ نخست فهمیدنِ این است که می‌خواهید چه مسئله‌ای را حل کنید. دو بخش دارد. نخست بپرس، «می‌خواهم چه مسئله‌ای را برای کاربرانم حل کنم؟» پاسخی مانندِ «آسان‌ترکردنِ سازمان‌دهی و دسترسیِ کاربران به عکس‌هایشان» بسیار مبهم است؛ «کمک به کاربر برای یافتنِ همهٔ عکس‌هایی که با عبارتِ خاصی، مانندِ پاریس، مطابقت دارند» بهتر است. سپس بپرس، «کدام بخش(های) مسئله را می‌توان با یادگیریِ ماشینی حل کرد؟» شاید به «یادگیریِ تابعی که عکسی را به مجموعه‌ای از برچسب‌ها نگاشت می‌کند؛ سپس، وقتی برچسبی به‌عنوانِ پرس‌وجو داده می‌شود، همهٔ عکس‌های با آن برچسب را بازیابی کن» بسنده کنی.[16]

برای عینی‌کردنِ این، باید تابعِ زیانی برای مؤلفهٔ یادگیریِ ماشینی‌ات مشخص کنی، شاید اندازه‌گیریِ دقتِ سامانه در پیش‌بینیِ برچسبِ درست. این هدف باید با هدف‌های واقعی‌ات همبسته باشد، اما معمولاً متمایز خواهد بود—هدفِ واقعی ممکن است بیشینه‌کردنِ شمارِ کاربرانی باشد که جذب و در سامانه‌ات نگه می‌داری، و درآمدی که تولید می‌کنند. این‌ها معیارهایی‌اند که باید پیگیری کنی، اما نه لزوماً معیارهایی که می‌توانی مستقیماً مدلِ یادگیریِ ماشینی برایشان بسازی.

وقتی مسئله‌ات را به بخش‌ها تجزیه کردی، ممکن است بیابی مؤلفه‌های متعددی هست که می‌توان با مهندسیِ نرم‌افزارِ قدیمی، نه یادگیریِ ماشینی، مدیریت کرد. برای نمونه، برای کاربری که «بهترین عکس‌ها» می‌خواهد، می‌توانی رویهٔ ساده‌ای پیاده‌سازی کنی که عکس‌ها را بر اساسِ شمارِ لایک‌ها و بازدیدها مرتب می‌کند. وقتی سامانهٔ کلی‌ات را تا نقطه‌ای توسعه دادی که قابلِ‌اجرا باشد، می‌توانی برگردی و بهینه کنی، مؤلفه‌های ساده را با مدل‌های یادگیریِ ماشینیِ پیچیده‌تر جایگزین کنی.

بخشی از صورت‌بندیِ مسئله تصمیم‌گیری دربارهٔ اینکه با یادگیریِ نظارت‌شده، نظارت‌نشده یا تقویتی سروکار داری است. تمایزها همیشه چندان شفاف نیستند. در «یادگیریِ نیمه‌نظارت‌شده» (Semisupervised Learning) چند مثالِ برچسب‌دار به ما داده می‌شود و از آن‌ها برای استخراجِ اطلاعاتِ بیشتر از مجموعهٔ بزرگی از مثال‌های بدونِ برچسب استفاده می‌کنیم. این رویکردِ رایجی شده، با شرکت‌هایی که ظهور کرده‌اند و مأموریتشان برچسب‌زدنِ سریعِ برخی مثال‌هاست، تا به سامانه‌های یادگیریِ ماشینی کمک کنند از مثال‌های بدونِ برچسبِ باقی‌مانده استفادهٔ بهتری کنند.

گاهی انتخابِ اینکه کدام رویکرد را استفاده کنی داری. سامانه‌ای برای توصیهٔ آهنگ یا فیلم به مشتریان را در نظر بگیر. می‌توانیم به آن به‌عنوانِ مسئلهٔ یادگیریِ نظارت‌شده نزدیک شویم، که در آن ورودی‌ها شاملِ بازنماییِ مشتری‌اند و خروجیِ برچسب‌دار این است که آیا آن‌ها توصیه را دوست داشتند یا نه، یا می‌توانیم به آن به‌عنوانِ مسئلهٔ یادگیریِ تقویتی نزدیک شویم، که در آن سامانه توالیِ کنش‌های توصیه می‌کند و گاهی از مشتری برای پیشنهادِ خوب پاداش می‌گیرد.

خودِ برچسب‌ها ممکن است حقایقِ وحی‌گونه‌ای که امیدواریم نباشند. تصور کن می‌خواهی سامانه‌ای بسازی که سنِ فردی را از عکسی حدس می‌زند. با آپلودکردنِ عکس‌ها و بیانِ سنشان توسط مردم، مثال‌های برچسب‌داری جمع می‌کنی. آن یادگیریِ نظارت‌شده است. اما در واقعیت برخی از مردم دربارهٔ سنشان دروغ گفتند. فقط این نیست که نویزِ تصادفی در داده هست؛ بلکه نادقتی‌ها نظام‌مندند، و برملاکردنشان مسئلهٔ یادگیریِ نظارت‌نشده‌ای شاملِ تصاویر،

سن‌های خودگزارش‌شده و سن‌های واقعیِ (ناشناخته) است. بدین‌ترتیب، هم نویز و هم فقدانِ برچسب پیوستاری بینِ یادگیریِ نظارت‌شده و نظارت‌نشده ایجاد می‌کنند. حوزهٔ «یادگیریِ ضعیف‌نظارت‌شده» (Weakly Supervised Learning) بر استفاده از برچسب‌هایی تمرکز دارد که نویزی، نادقیق یا توسط غیرمتخصصان عرضه شده‌اند.

۱۹.۹.۲ جمع‌آوری، ارزیابی و مدیریتِ داده

هر پروژهٔ یادگیریِ ماشینی به داده نیاز دارد؛ در موردِ پروژهٔ شناساییِ عکسِ ما مجموعه‌های دادهٔ تصویریِ آزادانه‌ای در دسترس‌اند، مانندِ ImageNet، که بیش از ۱۴ میلیون عکس با حدود ۲۰,۰۰۰ برچسبِ متفاوت دارد. گاهی ممکن است مجبور باشیم داده‌های خودمان را تولید کنیم، که می‌توان با کارِ خودمان، یا با «برون‌سپاریِ جمعی» (Crowdsourcing) به کارگرانِ حقوق‌بگیر یا داوطلبانِ بدونِ حقوق که از طریقِ سرویسِ اینترنتی عمل می‌کنند، انجام شود. گاهی داده از کاربرانت می‌آید. برای نمونه، سرویسِ ناوبریِ ویز کاربران را تشویق می‌کند داده‌هایی دربارهٔ ترافیکِ سنگین آپلود کنند، و از آن برای ارائهٔ مسیرهای به‌روزِ ناوبری به همهٔ کاربران استفاده می‌کند. «یادگیریِ انتقالی» (بخش ۲۱.۷.۲ را ببینید) وقتی دادهٔ کافیِ خودت نداری استفاده می‌شود: با مجموعهٔ دادهٔ هم‌همنظورهٔ عمومیِ در دسترسِ عموم شروع کن (یا مدلی که از پیش روی این داده آموزش دیده)، و سپس دادهٔ خاصِ کاربرانت را بیفزا و بازآموزی کن.

اگر سامانه‌ای را برای کاربران مستقر کنی، کاربرانت بازخورد خواهند داد—شاید با کلیک‌کردن روی یک قلم و نادیده‌گرفتنِ بقیه. به استراتژی برای سروکارداشتن با این داده نیاز خواهی داشت. آن شاملِ بازبینی با کارشناسانِ حریمِ خصوصی است (بخش ۲۷.۳.۲ را ببینید) تا مطمئن شوی اجازهٔ مناسب برای داده‌ای که جمع می‌کنی می‌گیری، و فرایندهایی برای تضمینِ یکپارچگیِ دادهٔ کاربر و فهمِ آن‌ها از آنچه با آن خواهی کرد داری. همچنین باید مطمئن شوی فرایندهایت منصفانه و بدونِ سوگیری‌اند (بخش ۲۷.۳.۳ را ببینید). اگر داده‌ای هست که احساس می‌کنی برای جمع‌آوری بیش‌ازحدِ حساس است اما برای مدلِ یادگیریِ ماشینی مفید می‌بود، رویکردِ «یادگیریِ فدرال» (Federated Learning) را در نظر بگیر، که در آن داده روی دستگاهِ کاربر می‌ماند، اما پارامترهای مدل به‌گونه‌ای به اشتراک گذاشته می‌شوند که دادهٔ خصوصی را فاش نمی‌کند.

تمرینِ خوب این است که برای همهٔ داده‌هایت «اصل‌ونسبِ داده» (Data Provenance) نگه داری. برای هر ستونِ مجموعهٔ داده‌ات، باید تعریفِ دقیق، محلِ آمدنِ داده، مقادیرِ ممکن و کسانی که رویش کار کرده‌اند را بدانی. آیا دوره‌هایی بوده که فیدِ داده قطع شده؟ آیا تعریفِ برخی منبعِ داده در طولِ زمان تکامل یافته؟ برای مقایسهٔ نتایج در دوره‌های زمانی باید این را بدانی. این به‌طورِ خاص درست است اگر به داده‌ای تکیه می‌کنی که توسط شخصِ دیگری تولید شده—نیازهای آن‌ها و نیازهای تو ممکن است واگرا شوند، و ممکن است در نهایت شیوهٔ تولیدِ داده را تغییر دهند، یا کلاً به‌روزرسانی‌اش را متوقف کنند. باید فیدهای داده‌ات را برای گرفتنِ این پایش کنی. داشتنِ خط لولهٔ دادهٔ قابلِ‌اعتماد، انعطاف‌پذیر و امن برای موفقیت حیاتی‌تر از جزئیاتِ دقیقِ الگوریتمِ یادگیریِ ماشینی است. اصل‌ونسب همچنین به دلایلِ قانونی، مانندِ انطباق با قانونِ حریمِ خصوصی، مهم است.

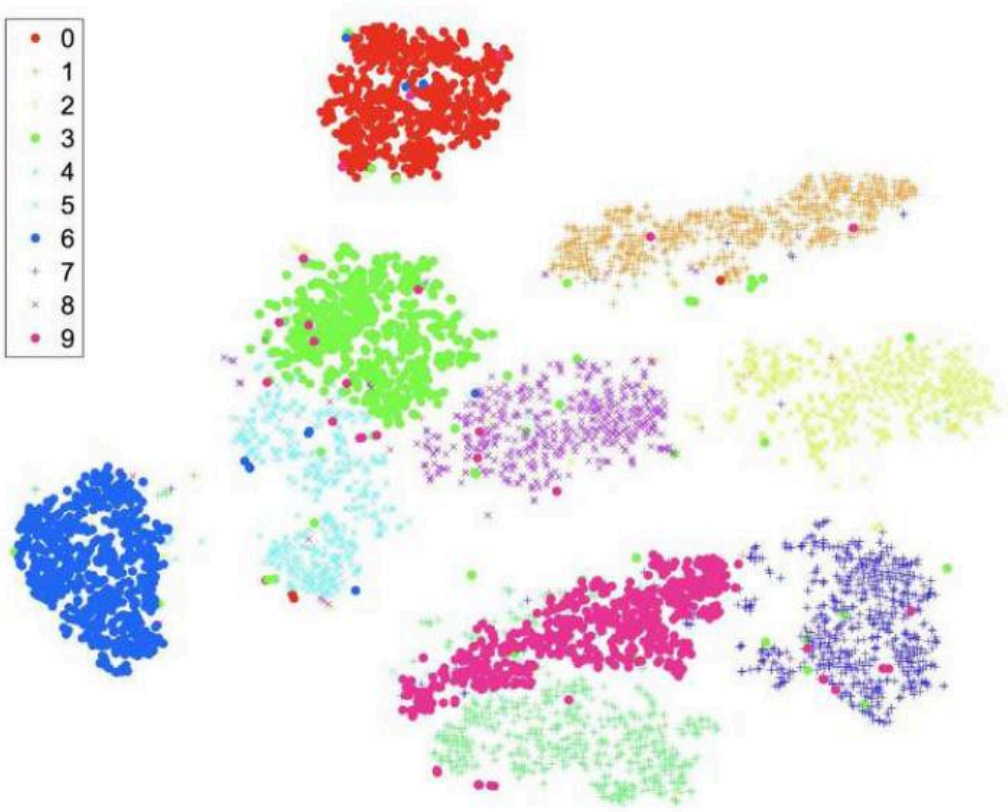
برای هر تکلیفی پرسش‌هایی دربارهٔ داده خواهد بود: آیا این دادهٔ درست برای تکلیفِ من است؟ آیا به‌اندازهٔ کافی از ورودی‌های درست را در بر می‌گیرد تا شانسِ یادگیریِ مدلی را به ما بدهد؟ آیا خروجی‌هایی را که می‌خواهم پیش‌بینی کنم دارد؟ اگر نه، آیا می‌توانم مدلِ نظارت‌نشده بسازم؟ یا می‌توانم بخشی از داده را برچسب بزنم و سپس یادگیریِ نیمه‌نظارت‌شده انجام دهم؟ آیا دادهٔ مربوط است؟ عالی است ۱۴ میلیون عکس داشته باشیم، اما اگر همهٔ کاربرانت

متخصصانی‌اند که به موضوعِ خاصی علاقه‌مندند، پایگاهِ دادهٔ عمومی کمکی نخواهد کرد—باید عکس‌هایی در موضوعِ خاص جمع کنی. چقدر دادهٔ آموزشی کافی است؟ (آیا باید دادهٔ بیشتری جمع کنم؟ آیا می‌توانم برخی داده‌ها را برای سریع‌ترکردنِ محاسبه دور بیندازم؟) بهترین راه برای پاسخ‌دادن به این، استدلالِ با قیاس با پروژهٔ مشابهی با اندازهٔ مجموعهٔ آموزشیِ معلوم است. وقتی شروع کردی می‌توانی منحنیِ یادگیری بکشی (شکل ۱۹.۷ را ببینید) تا ببینی دادهٔ بیشتر کمک خواهد کرد، یا یادگیری قبلاً به فلات رسیده.

قاعده‌های سرانگشتیِ بی‌پایانِ موردی و بی‌توجیهی برای شمارِ مثال‌های آموزشیِ لازم هست: میلیون‌ها برای مسائلِ دشوار؛ هزاران برای مسائلِ متوسط؛ صدها یا هزاران برای هر رده در مسئلهٔ رده‌بندی؛ ۱۰ برابرِ بیشترِ مثال از پارامترهای مدل؛ ۱۰ برابرِ بیشترِ مثال از ویژگی‌های ورودی؛ مثال‌های $O(d \log d)$ برای d ویژگیِ ورودی؛ مثال‌های بیشتر برای مدل‌های غیرخطی تا خطی؛ مثال‌های بیشتر اگر دقتِ بیشتر لازم است؛ مثال‌های کمتر اگر منظم‌سازی استفاده می‌کنی؛ به‌اندازهٔ کافی مثال برای رسیدن به قدرتِ آماریِ لازم برای ردِ فرضیهٔ صفر در رده‌بندی. همهٔ این قاعده‌ها ملاحظاتی دارند —همان‌گونه که قاعدهٔ عاقلانه‌ای که پیشنهاد می‌کند آنچه در گذشته برای مسائلِ مشابه کار کرده امتحان کن.

باید دربارهٔ داده‌ات دفاعی فکر کنی. آیا ممکن است خطاهای ورودِ داده باشد؟ با فیلدهای دادهٔ مفقود چه می‌توان کرد؟ اگر داده را از مشتریانت (یا افرادِ دیگر) جمع می‌کنی، آیا ممکن است برخی از مردم دشمنانی باشند که می‌خواهند سامانه را گول بزنند؟ آیا خطاهای املایی یا اصطلاح‌شناسیِ ناسازگار در دادهٔ متنی هست؟ (برای نمونه، آیا «Apple»، «AAPL» و «Apple Inc.» همه به همان شرکت اشاره دارند؟) به فرایندی برای گرفتن و تصحیحِ همهٔ این منابعِ بالقوهٔ خطای داده نیاز خواهی داشت.

ابزارِ تجسمِ مفیدی برای دادهٔ پربعد «t-SNE» (جاسازیِ تصادفیِ همسایهٔ توزیع-تی) است، که نقاطِ پربعد را به نقشه‌ای در دو بُعد (یا گاهی سه بُعد، که سپس می‌توان به‌صورتِ تعاملی کاوش کرد) فرو می‌کشد.[17]



شکل ۱۹.۲۷ — نقشهٔ t-SNE دوبعدی از مجموعهٔ دادهٔ MNIST، مجموعه‌ای از ۶۰٬۰۰۰ تصویر از رقم‌های دست‌نویس، هر یک 28×28 پیکسل و بنابراین 784 بُعد. به‌وضوح خوشه‌هایی برای ده رقم می‌بینی، با چند اشتباه در هر خوشه؛ برای نمونه خوشهٔ بالایی برای رقمِ 0 است، اما شاملِ چند رقمِ دیگر نیز می‌شود.

وقتی داده محدود است، «افزایشِ داده» (Data Augmentation) می‌تواند کمک کند. برای نمونه، با مجموعهٔ داده‌ای از تصاویر، می‌توانی چند نسخه از هر تصویر با چرخاندن، انتقال، برش یا مقیاس‌کردنِ هر تصویر، یا با تغییرِ روشنایی یا توازنِ رنگ یا افزودنِ نویز ایجاد کنی. تا زمانی که این‌ها تغییرهای کوچکی‌اند، برچسبِ تصویر باید همان بماند، و مدلی که روی چنین دادهٔ افزوده‌شده‌ای آموزش دیده مقاوم‌تر خواهد بود.

گاهی داده فراوان است اما در «رده‌های نامتوازن» (Unbalanced Classes) رده‌بندی می‌شود. برای نمونه، مجموعهٔ آموزشیِ تراکنش‌های کارتِ اعتباری ممکن است شاملِ ۱۰٬۰۰۰٬۰۰۰ تراکنشِ معتبر و ۱٬۰۰۰ کلاهبرداری باشد. رده‌بندی که صرف‌نظر از ورودی «معتبر» بگوید به دقتِ 99.99٪ روی این مجموعهٔ داده می‌رسد. برای فراتررفتن از آن، رده‌بند باید به مثال‌های کلاهبرداری توجهِ بیشتری کند. برای کمک به آن، می‌توانی «کم‌نمونه‌برداریِ» (Undersampling) ردهٔ اکثریت کنی (یعنی برخی از مثال‌های ردهٔ «معتبر» را نادیده بگیری) یا «بیش‌نمونه‌برداریِ» (Oversampling) ردهٔ اقلیت (یعنی برخی از مثال‌های ردهٔ «کلاهبرداری» را تکرار کنی). می‌توانی از تابعِ زیانِ وزنداری استفاده کنی که جریمهٔ بزرگ‌تری برای ازدست‌دادنِ موردِ کلاهبرداری می‌دهد. تقویت همچنین می‌تواند به تمرکز بر ردهٔ اقلیت کمک کند. اگر از روشِ گروهی استفاده می‌کنی، می‌توانی قاعده‌های رأی‌دادنِ گروه را تغییر دهی و «کلاهبرداری» را به‌عنوانِ پاسخ بدهی حتی اگر فقط اقلیتی از گروه به «کلاهبرداری» رأی دهند. می‌توانی با تولیدِ دادهٔ مصنوعی با فنونِ مانندِ SMOTE (چاولا و همکاران، 2002) یا ADASYN (هه و همکاران، 2008) به متوازن‌کردنِ رده‌های نامتوازن کمک کنی.

باید «دورافتاده‌های» (Outliers) داده‌ات را با دقت در نظر بگیری. دورافتاده نقطهٔ داده‌ای است که از سایر نقاط دور است. برای نمونه، در مسئلهٔ رستوران، اگر قیمت مقدارِ عددی بود نه رسته‌ای، و اگر مثالی قیمتِ ۳۱۶ دلار داشت در حالی که بقیه ۳۰ دلار یا کمتر بودند، آن مثال دورافتاده می‌بود. روش‌هایی مانندِ رگرسیونِ خطی در برابرِ دورافتاده‌ها آسیب‌پذیرند، زیرا باید مدلِ خطیِ سراسریِ تکی بسازند که همهٔ ورودی‌ها را در نظر می‌گیرد—نمی‌توانند با دورافتاده متفاوت از سایر نقاطِ مثال رفتار کنند، و بدین‌ترتیب یک دورافتاده می‌تواند اثرِ بزرگی بر همهٔ پارامترهای مدل داشته باشد.

با ویژگی‌هایی مانندِ قیمت که اعدادِ مثبت‌اند، می‌توانیم اثرِ دورافتاده‌ها را با تبدیلِ داده کاهش دهیم، با گرفتنِ لگاریتمِ هر مقدار، پس ۲۰ دلار، ۲۵ دلار و ۳۱۶ دلار به 1.3، 1.4 و 2.5 تبدیل می‌شوند. این از دیدگاهِ عملی معنا دارد، زیرا مقدارِ بالا حالا تأثیرِ کمتری بر مدل دارد، و از دیدگاهِ نظری، زیرا چنان‌که در بخش ۱۶.۳.۲ دیدیم، مطلوبیتِ پول لگاریتمی است.

روش‌هایی مانندِ درخت‌های تصمیم که از چند مدلِ محلی ساخته می‌شوند می‌توانند با دورافتاده‌ها به‌صورتِ فردی رفتار کنند: مهم نیست بزرگ‌ترین مقدار ۳۰۰ دلار باشد یا ۳۱؛ به هر حال می‌توان در گرهٔ محلیِ خودش پس از آزمونی به شکلِ $\text{cost} \le 30$ با آن رفتار کرد. این درخت‌های تصمیم (و بنابراین جنگل‌های تصادفی و تقویتِ گرادیان) را در برابرِ دورافتاده‌ها مقاوم‌تر می‌کند.

مهندسیِ ویژگی. پس از تصحیحِ خطاهای آشکار، ممکن است بخواهی داده را برای آسان‌ترشدنِ هضمِ آن پیش‌پردازش کنی. قبلاً فرایندِ «کوانتیزه‌کردن» (Quantization) را دیده‌ایم: واداشتنِ ورودیِ پیوسته‌مقدار، مانندِ زمانِ انتظار، به سطل‌های ثابت (0–10 دقیقه، 10–30، 30–60 یا >60). دانشِ حوزه می‌تواند بگوید چه آستانه‌هایی مهم‌اند، مانندِ مقایسهٔ سن ≤ 18 هنگام مطالعهٔ الگوهای رأی‌گیری. همچنین دیدیم (صفحهٔ ۶۸۸) که الگوریتم‌های نزدیک‌ترین-همسایه وقتی داده نرمال شده تا انحرافِ معیارِ 1 داشته باشد بهتر عمل می‌کنند. با ویژگی‌های رسته‌ای مانندِ آفتابی/ابری/بارانی، اغلب مفید است داده را به سه ویژگیِ بولیِ جداگانه تبدیل کنیم، که دقیقاً یکی از آن‌ها صادق است (این را «رمزگذاریِ یک-داغ» (One-Hot Encoding) می‌نامیم). این به‌طورِ خاص وقتی مفید است که مدلِ یادگیریِ ماشینی شبکهٔ عصبی باشد.

همچنین می‌توانی ویژگی‌های تازه‌ای بر اساسِ دانشِ حوزه‌ات معرفی کنی. برای نمونه، با مجموعهٔ داده‌ای از خریدهای مشتری که هر درایه ویژگیِ تاریخ دارد، ممکن است بخواهی داده را با ویژگی‌های تازه‌ای تقویت کنی که می‌گویند آیا تاریخ آخرِ هفته یا تعطیل است. به‌عنوانِ مثالِ دیگر، تکلیفِ برآوردِ ارزشِ واقعیِ خانه‌های برای‌فروش را در نظر بگیر. در شکل ۱۹.۱۳ نسخهٔ اسباب‌بازیِ این مسئله را نشان دادیم، با رگرسیونِ خطیِ اندازهٔ خانه به قیمتِ درخواستی. اما واقعاً می‌خواهیم قیمتِ فروشِ خانه را برآورد کنیم، نه قیمتِ درخواستی را. برای حلِ این تکلیف به داده‌ای دربارهٔ فروشِ واقعی نیاز داریم. اما آن بدان معنا نیست باید دادهٔ قیمتِ درخواستی را دور بیندازیم—می‌توانیم از آن به‌عنوانِ یکی از ویژگی‌های ورودی استفاده کنیم.

۱۹.۹.۳ انتخابِ مدل و آموزش

با دادهٔ پاک‌سازی‌شده و حسِ شهودی از آن، وقتِ ساختنِ مدل است. آن به معنای انتخابِ ردهٔ مدل (جنگل‌های تصادفی؟ شبکه‌های عصبیِ عمیق؟ گروهی؟)، آموزشِ مدل با دادهٔ آموزشی، تنظیمِ هر ابرپارامترِ رده (شمارِ درخت‌ها؟ شمارِ لایه‌ها؟) با دادهٔ اعتبارسنجی، رفع‌اشکالِ فرایند، و سرانجام ارزیابیِ مدل روی دادهٔ آزمون است. هیچ راهِ تضمینی برای انتخابِ بهترین ردهٔ مدل نیست، اما رهنمودهای تقریبی هست. جنگل‌های تصادفی وقتی خوب‌اند که ویژگی‌های رسته‌ایِ زیادی هست و باور داری بسیاری از آن‌ها ممکن است نامربوط باشند. روش‌های ناپارامتری وقتی خوب‌اند که دادهٔ زیاد و دانشِ قبلی نداری، و وقتی نمی‌خواهی زیاد نگرانِ انتخابِ دقیقِ ویژگی‌ها باشی (تا زمانی که کمتر از حدود ۲۰ تایند). با این حال، روش‌های ناپارامتری معمولاً تابعِ h به تو می‌دهند که اجرایش گران‌تر است.

رگرسیونِ لجستیک وقتی خوب عمل می‌کند که داده قابلِ جداسازیِ خطی است، یا با مهندسیِ ویژگیِ هوشمندانه می‌تواند چنین شود. ماشین‌های بردارِ پشتیبان روشِ خوبی برای امتحان وقتی مجموعهٔ داده خیلی بزرگ نیست؛ آن‌ها روی دادهٔ قابلِ‌جداسازی شبیهِ رگرسیونِ لجستیک عمل می‌کنند و می‌توانند برای دادهٔ پربعد بهتر باشند. مسائلِ مربوط به بازشناسیِ الگو، مانندِ پردازشِ تصویر یا گفتار، بیشتر با شبکه‌های عصبیِ عمیق (فصل ۲۱ را ببینید) بررسی می‌شوند. انتخابِ ابرپارامترها را می‌توان با ترکیبی از تجربه—آنچه در مسائلِ مشابهِ گذشته خوب کار کرده انجام بده—و جستجو انجام داد: با چند مقدارِ ممکن برای ابرپارامترها آزمایش کن. با اجرایِ آزمایش‌های بیشتر ایده‌هایی برای مدل‌های متفاوت برای امتحان می‌گیری. با این حال، اگر عملکرد را روی دادهٔ اعتبارسنجی اندازه بگیری، ایده‌ای تازه بگیری و آزمایش‌های بیشتری اجرا کنی، ریسکِ بیش‌برازش روی دادهٔ اعتبارسنجی را می‌کنی. اگر دادهٔ کافی داشته باشی، ممکن است بخواهی چند مجموعهٔ اعتبارسنجیِ جداگانه داشته باشی تا از این مسئله اجتناب کنی. این به‌طورِ خاص درست است اگر دادهٔ اعتبارسنجی را با چشم بازرسی می‌کنی، نه فقط ارزیابی‌هایی رویش اجرا می‌کنی.

فرض کن رده‌بندی می‌سازی—برای نمونه، سامانه‌ای برای رده‌بندیِ ایمیلِ هرزنامه. برچسب‌زدنِ نامهٔ قانونی به‌عنوانِ هرزنامه «مثبتِ کاذب» (False Positive) نامیده می‌شود. معاوضه‌ای بینِ مثبت‌های کاذب و منفی‌های کاذب (برچسب‌زدنِ هرزنامه به‌عنوانِ قانونی) خواهد بود؛ اگر بخواهی نامهٔ قانونیِ بیشتری از پوشهٔ هرزنامه دور نگه داری، لزوماً به ارسالِ هرزنامهٔ بیشتر به صندوقِ ورودی می‌انجامی. اما بهترین راهِ معاوضه چیست؟ می‌توانی مقادیرِ متفاوتی از ابرپارامترها را امتحان کنی و نرخ‌های متفاوتی برای دو نوع خطا بگیری—نقاطِ متفاوتی روی این معاوضه. نموداری به نامِ «منحنیِ مشخصهٔ عملکردِ گیرنده» (Receiver Operating Characteristic یا ROC) مثبت‌های کاذب را در برابرِ مثبت‌های واقعی برای هر مقدارِ ابرپارامتر رسم می‌کند و به تجسمِ مقادیری که انتخاب‌های خوبی برای معاوضه‌اند کمک می‌کند. معیاری به نامِ «مساحت زیر منحنیِ ROC» یا AUC خلاصهٔ تک‌عددی از منحنیِ ROC فراهم می‌کند، که اگر بخواهی سامانه‌ای مستقر کنی و به هر کاربر اجازه دهی نقطهٔ معاوضه‌اش را انتخاب کند مفید است.

ابزارِ تجسمِ مفیدِ دیگر برای مسائلِ رده‌بندی «ماتریسِ سردرگمی» (Confusion Matrix) است: جدولِ دوبعدی از شمارشِ اینکه هر رده چند بار به‌عنوانِ هر ردهٔ دیگر رده‌بندی یا اشتباه‌رده‌بندی شده. معاوضه‌هایی در عواملِ غیر از تابعِ زیان نیز می‌تواند باشد. اگر بتوانی مدلِ پیش‌بینیِ سهام آموزش بدهی که در هر معامله ۱۰ دلار برایت درآورد، عالی است —اما نه اگر در هر پیش‌بینی ۲۰ دلار هزینهٔ محاسبه بدهد. برنامهٔ ترجمهٔ ماشینی که روی گوشی‌ات اجرا می‌شود و به تو اجازه می‌دهد تابلوها را در شهرِ خارجی بخوانی مفید است—اما نه اگر بعد از یک ساعت استفاده باتری را خالی کند.

همهٔ عواملی که به پذیرش یا ردِ سامانه‌ات می‌انجامند را پیگیری کن، و فرایندی طراحی کن که بتوانی به‌سرعت چرخهٔ گرفتنِ ایدهٔ تازه، اجرایِ آزمایش و ارزیابیِ نتایجِ آزمایش برای دیدنِ اینکه پیشرفت کرده‌ای را تکرار کنی. سریع‌کردنِ این چرخهٔ تکرار یکی از مهم‌ترین عواملِ موفقیت در یادگیریِ ماشینی است.

۱۹.۹.۴ اعتماد، تفسیرپذیری و تبیین‌پذیری

روش‌شناسیِ یادگیریِ ماشینی را توصیف کرده‌ایم که در آن مدل‌ت را با دادهٔ آموزشی توسعه می‌دهی، ابرپارامترها را با دادهٔ اعتبارسنجی انتخاب می‌کنی، و معیارِ نهایی را با دادهٔ آزمون می‌گیری. خوب‌عمل‌آمدنِ روی آن معیار شرطِ لازم اما نه کافی برای اعتمادکردنِ تو به مدلِت است. و فقط تو نیستی—ذی‌نفعانِ دیگر از جملهِ نهادهای نظارتی، قانون‌گذاران، مطبوعات و کاربرانت نیز به قابل‌اعتمادبودنِ سامانه‌ات علاقه‌مندند (و نیز به ویژگی‌های مرتبط مانندِ قابلیتِ اطمینان، پاسخگویی و ایمنی).

سامانهٔ یادگیریِ ماشینی همچنان قطعه‌ای نرم‌افزار است، و می‌توانی با همهٔ ابزارهای نمونهٔ تأیید و اعتبارسنجیِ هر سامانهٔ نرم‌افزاری اعتماد بسازی:

- **کنترلِ منبع:** سامانه‌هایی برای کنترلِ نسخه، ساخت و پیگیریِ اشکال/مسئله.
- **آزمون:** آزمون‌های واحد برای همهٔ مؤلفه‌ها که مواردِ متعارفِ ساده و نیز مواردِ دشوارِ خصمانه را پوشش می‌دهند، آزمون‌های فازی (جایی که ورودی‌های تصادفی تولید می‌شوند)، آزمون‌های رگرسیون، آزمون‌های بار و آزمون‌های یکپارچه‌سازیِ سامانه: همهٔ این‌ها برای هر سامانهٔ نرم‌افزاری مهم‌اند. برای یادگیریِ ماشینی، همچنین آزمون‌هایی روی مجموعه‌های دادهٔ آموزشی، اعتبارسنجی و آزمون داریم.
- **بازبینی:** بازبینی‌ها و مرورهای کد، بازبینی‌های حریمِ خصوصی، بازبینی‌های انصاف (بخش ۲۷.۳.۳ را ببینید) و سایر بازبینی‌های انطباقِ قانونی.
- **پایش:** داشبوردها و هشدارها برای اطمینان از اینکه سامانه بالا و در حالِ اجراست و همچنان در سطحِ بالایی از دقت عمل می‌کند.
- **پاسخگویی:** وقتی سامانه اشتباه می‌کند چه اتفاقی می‌افتد؟ فرایندِ شکایت یا اعتراض به تصمیمِ سامانه چیست؟ چگونه می‌توانیم پیگیری کنیم چه کسی مسئولِ خطا بود؟ جامعه از بانک‌ها، سیاست‌مداران و قانون انتظارِ پاسخگویی برای تصمیم‌های مهم دارد (و همیشه نمی‌گیرد)، و باید از سامانه‌های نرم‌افزاری از جملهِ سامانه‌های یادگیریِ ماشینی نیز پاسخگویی انتظار داشته باشند.

علاوه بر این، عواملی هست که به‌طورِ خاص برای سامانه‌های یادگیریِ ماشینی مهم‌اند، چنان‌که در زیر تفصیل می‌دهیم.

تفسیرپذیری (Interpretability): می‌گوییم مدلِ یادگیریِ ماشینی تفسیرپذیر است اگر بتوانی خودِ مدلِ واقعی را بازرسی کنی و بفهمی چرا برای ورودیِ معین پاسخِ خاصی گرفت، و پاسخ با تغییرِ ورودی چگونه تغییر می‌کرد.[18] مدل‌های درختِ تصمیم بسیار تفسیرپذیر دانسته می‌شوند؛ می‌توانیم بفهمیم پیروی از مسیرِ Patrons=Full و WaitEstimate=0–10 در درختِ تصمیم به تصمیمِ انتظار می‌انجامد. درختِ تصمیم به دو دلیل تفسیرپذیر است. نخست، ما انسان‌ها در فهمِ قاعده‌های IF/THEN تجربه داریم. (در مقابل، برای انسان‌ها بسیار دشوار است درکِ

شهودیِ نتیجهٔ ضربِ ماتریسی که با تابعِ فعال‌سازی دنبال می‌شود، چنان‌که در برخی مدل‌های شبکهٔ عصبی انجام می‌شود، بگیرند.) دوم، درختِ تصمیم از جهتی برای تفسیرپذیربودن ساخته شده—ریشهٔ درخت به‌عنوانِ ویژگیِ با بالاترین بهرهٔ اطلاعاتی انتخاب شد.

مدل‌های رگرسیونِ خطی نیز تفسیرپذیر دانسته می‌شوند؛ می‌توانیم مدلی را برای پیش‌بینیِ اجارهٔ آپارتمان بررسی کنیم و ببینیم برای هر اتاق خوابِ افزوده‌شده، اجاره ۵۰۰ دلار افزایش می‌یابد، طبقِ مدل. این ایدهٔ «اگر x را تغییر دهم، خروجی چگونه تغییر خواهد کرد؟» در هستهٔ تفسیرپذیری است. البته، همبستگی علیت نیست، پس مدل‌های تفسیرپذیر پاسخ می‌دهند چه چیزی هست، اما نه لزوماً چرا هست.

تبیین‌پذیری (Explainability): مدلِ تبیین‌پذیر مدلی است که می‌تواند به تو کمک کند بفهمی «چرا این خروجی برای این ورودی تولید شد؟» در اصطلاح‌شناسیِ ما، تفسیرپذیری از بازرسیِ مدلِ واقعی ناشی می‌شود، در حالی که تبیین‌پذیری را می‌تواند فرایندِ جداگانه‌ای فراهم کند. یعنی، خودِ مدل می‌تواند جعبهٔ سیاهِ دشوارِ فهمی باشد، اما ماژولِ تبیین می‌تواند خلاصه کند مدل چه می‌کند. برای سامانهٔ بازشناسیِ تصویرِ شبکهٔ عصبی که تصویری را به‌عنوانِ سگ رده‌بندی می‌کند، اگر بخواهیم مدل را مستقیماً تفسیر کنیم، بهترین چیزی که می‌توانستیم با خود ببریم چیزی مانندِ «پس از پردازشِ لایه‌های پیچشی، فعال‌سازیِ خروجیِ سگ در لایهٔ سافت‌مکس از هر ردهٔ دیگر بالاتر بود» است. آن استدلالِ چندان قانع‌کننده‌ای نیست. اما ماژولِ تبیینِ جداگانه‌ای ممکن است بتواند مدلِ شبکهٔ عصبی را بررسی کند و به تبیینِ «چهار پا، خز، دم، گوش‌های آویزان و پوزهٔ بلند دارد؛ کوچک‌تر از گرگ است، و روی تختِ سگ خوابیده، پس فکر می‌کنم سگ است» برسد. تبیین‌ها یکی از راه‌های ساختنِ اعتمادند، و برخی مقررات مانندِ GDPR اروپا (مقرراتِ عمومیِ حفاظت از داده) سامانه‌ها را ملزم به ارائهٔ تبیین می‌کنند.

به‌عنوانِ نمونه‌ای از ماژولِ تبیینِ جداگانه، سامانهٔ «تبیین‌های محلیِ تفسیرپذیرِ مستقل از مدل» (LIME) چنین کار می‌کند: مهم نیست از کدام ردهٔ مدل استفاده می‌کنی، LIME مدلِ تفسیرپذیری می‌سازد—اغلب درختِ تصمیم یا مدلِ خطی—که تقریبی از مدلِ توست، و سپس مدلِ خطی را تفسیر می‌کند تا تبیین‌هایی بسازد که می‌گویند هر ویژگی چقدر مهم است. LIME این کار را با رفتارِ مدلِ یادگیریِ ماشینی به‌عنوانِ جعبهٔ سیاه و کاوشِ آن با مقادیرِ ورودیِ تصادفیِ مختلف برای ایجادِ مجموعهٔ داده‌ای که مدلِ تفسیرپذیر از آن ساخته می‌شود انجام می‌دهد. این رویکرد برای دادهٔ ساخت‌یافته مناسب است، اما نه برای چیزهایی مانندِ تصاویر، جایی که هر پیکسل ویژگی است و هیچ پیکسلِ تکی به‌تنهایی «مهم» نیست.

گاهی ردهٔ مدلی را به دلیلِ تبیین‌پذیری‌اش انتخاب می‌کنیم—ممکن است درخت‌های تصمیم را بر شبکه‌های عصبی ترجیح دهیم، نه به این دلیل که دقتِ بالاتری دارند، بلکه به این دلیل که تبیین‌پذیری اعتمادِ بیشتری به آن‌ها به ما می‌دهد. با این حال، تبیینِ ساده می‌تواند به حسِ امنیتِ کاذب بینجامد. بالاخره، معمولاً مدلِ یادگیریِ ماشینی را انتخاب می‌کنیم (به‌جای برنامهٔ سنتیِ دست‌نوشته) زیرا مسئله‌ای که می‌خواهیم حل کنیم ذاتاً پیچیده است و نمی‌دانیم چگونه برنامهٔ سنتی بنویسیم. در آن صورت، نباید انتظار داشته باشیم لزوماً تبیینِ ساده‌ای برای هر پیش‌بینی باشد.

اگر مدلِ یادگیریِ ماشینی را عمدتاً برای هدفِ فهمِ حوزه می‌سازی، تفسیرپذیری و تبیین‌پذیری به تو در رسیدن به آن فهم کمک خواهند کرد. اما اگر فقط بهترین قطعهٔ نرم‌افزارِ عمل‌کننده را می‌خواهی، آزمون ممکن است اعتمادِ بیشتری از تبیین‌ها به تو بدهد. به کدام اعتماد می‌کنی: هواپیمای آزمایشی که هرگز پرواز نکرده اما تبیینِ مفصلی از اینکه چرا امن است دارد، یا هواپیمایی که با موفقیت ۱۰۰ پروازِ قبلی را کامل کرده و با دقت نگهداری شده، اما با تبیینِ تضمینی نمی‌آید؟

۱۹.۹.۵ عملیات، پایش و نگهداری

وقتی از عملکردِ مدلت راضی شدی، می‌توانی آن را برای کاربرانت مستقر کنی. با چالش‌های اضافی روبه‌رو خواهی شد. نخست، مسئلهٔ «دمِ بلندِ» (Long Tail) ورودی‌های کاربر است. ممکن است سامانه‌ات را روی مجموعهٔ آزمونِ بزرگی آزمایش کرده باشی، اما اگر سامانه‌ات محبوب باشد، به‌زودی ورودی‌هایی می‌بینی که هرگز آزمایش نشده‌اند. باید بدانی آیا مدلِت برای آن‌ها خوب تعمیم می‌دهد، که بدان معناست باید عملکردت را روی دادهٔ زنده پایش کنی—پیگیریِ آمار، نمایشِ داشبورد و ارسالِ هشدار وقتی معیارهای کلیدی زیرِ آستانه‌ای می‌افتند. علاوه بر به‌روزرسانیِ خودکارِ آمارِ تعاملاتِ کاربر، ممکن است لازم باشد ارزیاب‌های انسانی استخدام و آموزش دهی تا به سامانه‌ات نگاه کنند و درجه‌بندی کنند چقدر خوب عمل می‌کند.

دوم، مسئلهٔ «غیرایستایی» (Nonstationarity) است—جهان در طولِ زمان تغییر می‌کند. فرض کن سامانه‌ات ایمیل را به هرزنامه یا غیرهرزنامه رده‌بندی می‌کند. به محض اینکه دسته‌ای از پیام‌های هرزنامه را با موفقیت رده‌بندی کنی، هرزنامه‌پران خواهند دید چه کرده‌ای و تاکتیک‌هایشان را تغییر می‌دهند و نوعِ تازه‌ای پیام می‌فرستند که ندیده‌ای. غیرهرزنامه نیز تکامل می‌یابد، چنان‌که کاربران ترکیبِ ایمیل در برابرِ پیام‌رسانی یا سرویس‌های رومیزی در برابرِ موبایل را تغییر می‌دهند.

دائماً با پرسشِ اینکه چه چیزی بهتر است روبه‌رو خواهی بود: مدلی که خوب آزمایش شده اما از دادهٔ قدیمی‌تر ساخته شده، در برابرِ مدلی که از جدیدترین داده ساخته شده اما در استفادهٔ واقعی آزمایش نشده. سامانه‌های مختلف الزاماتِ تازگیِ متفاوتی دارند: برخی مسائل از مدلِ تازه در هر روز، یا حتی هر ساعت، سود می‌برند، در حالی که مسائلِ دیگر می‌توانند همان مدل را ماه‌ها نگه دارند. اگر هر ساعت مدلِ تازه‌ای مستقر می‌کنی، اجرایِ مجموعه‌آزمونِ سنگین و فرایندِ بازبینیِ دستی برای هر به‌روزرسانی غیرعملی خواهد بود. باید فرایندِ آزمون و انتشار را خودکار کنی تا تغییرهای کوچک به‌طورِ خودکار تأیید شوند، اما تغییرهای بزرگ‌تر بازبینیِ مناسب را فعال کنند. می‌توانی معاوضهٔ بینِ مدلِ برخط را در نظر بگیری، جایی که دادهٔ تازه به‌صورتِ افزایشی مدلِ موجود را اصلاح می‌کند، در برابرِ مدلِ آفلاین، جایی که هر نسخهٔ تازه مستلزم ساختنِ مدلِ تازه از صفر است.

فقط داده نیست که تغییر می‌کند—برای نمونه، واژه‌های تازه در پیام‌های ایمیلِ هرزنامه استفاده خواهند شد. همچنین ممکن است کلِ طرح‌وارهٔ داده تغییر کند—ممکن است با رده‌بندیِ ایمیلِ هرزنامه شروع کنی و لازم باشد خودت را برای رده‌بندیِ پیام‌های متنیِ هرزنامه، پیام‌های صوتیِ هرزنامه، ویدیوهای هرزنامه و غیره سازگار کنی. شکل ۱۹.۲۸ قاعدهٔ کلی‌ای می‌دهد که متخصصان را در انتخابِ سطحِ مناسبِ آزمون و پایش راهنمایی کند.

Tests for Features and Data

(1) Feature expectations are captured in a schema. (2) All features are beneficial. (3) No feature's cost is too much. (4) Features adhere to meta-level requirements. (5) The data pipeline has appropriate privacy controls. (6) New features can be added quickly. (7) All input feature code is tested.

Tests for Model Development

(1) Every model specification undergoes a code review. (2) Every model is checked in to a repository. (3) Offline proxy metrics correlate with actual metrics (4) All hyperparameters have been tuned. (5) The impact of model staleness is known. (6) A simpler model is not better. (7) Model quality is sufficient on all important data slices. The model has been tested for considerations of inclusion.

Tests for Machine Learning Infrastructure

(1) Training is reproducible. (2) Model specification code is unit tested. (3) The full ML pipeline is integration tested. (4) Model quality is validated before attempting to serve it. (5) The model allows debugging by observing the step-by-step computation of training or inference on a single example. (6) Models are tested via a canary process before they enter production serving environments. (7) Models can be quickly and safely rolled back to a previous serving version.

Monitoring Tests for Machine Learning

(1) Dependency changes result in notification. (2) Data invariants hold in training and serving inputs. (3) Training and serving features compute the same values. (4) Models are not too stale. (5) The model is numerically stable. (6) The model has not experienced regressions in training speed, serving latency, throughput, or RAM usage. (7) The model has not experienced a regression in prediction quality on served data.

شکل ۱۹.۲۸ – مجموعه‌ای از معیارها برای دیدنِ اینکه چقدر خوب در استقرارِ مدلِ یادگیریِ ماشینی‌ات با آزمون‌های کافی عمل می‌کنی. (خلاصه‌شده از برک و همکاران (2016)، که معیارِ امتیازدهی نیز فراهم می‌کنند. معیارها شاملِ: آزمون‌های ویژگی‌ها و داده (انتظاراتِ ویژگی در طرح‌واره گرفته شده‌اند؛ همهٔ ویژگی‌ها مفیدند؛ هزینهٔ هیچ ویژگی‌ای بیش از حد نیست؛ ویژگی‌ها به الزاماتِ فراسطحی پایبندند؛ خط لولهٔ داده کنترل‌های حریمِ خصوصیِ مناسب دارد؛ ویژگی‌های تازه به‌سرعت قابلِ افزودن‌اند؛ همهٔ کدِ ویژگیِ ورودی آزمایش شده)؛ آزمون‌های توسعهٔ مدل (هر مشخصاتِ مدل بازبینیِ کد می‌شود؛ هر مدل در مخزن ثبت می‌شود؛ معیارهای جانشینِ آفلاین با معیارهای واقعی همبستگی دارند؛ همهٔ ابرپارامترها تنظیم شده‌اند؛ اثرِ کهنگیِ مدل معلوم است؛ مدلِ ساده‌تر بهتر نیست؛ کیفیتِ مدل روی همهٔ برش‌های مهمِ داده کافی است و مدل برای ملاحظاتِ شمول آزمایش شده)؛ آزمون‌های زیرساختِ یادگیریِ ماشینی (آموزش قابلِ بازتولید است؛ کدِ مشخصاتِ مدل آزمونِ واحد دارد؛ کلِ خط لولهٔ ML آزمونِ یکپارچه دارد؛ کیفیتِ مدل قبل از تلاش برای ارائه تأیید می‌شود؛ مدل با مشاهدهٔ محاسبهٔ گام‌به‌گامِ آموزش یا استنتاج روی مثالِ تکی امکانِ رفع‌اشکال می‌دهد؛ مدل‌ها قبل از ورود به محیط‌های ارائهٔ تولیدی با فرایندِ قناری آزمایش می‌شوند؛ مدل‌ها را می‌توان سریع و امن به نسخهٔ ارائهٔ قبلی بازگرداند)؛ آزمون‌های پایش (تغییرهای وابستگی به اعلان می‌انجامند؛ ناورداهای داده در ورودی‌های آموزش و ارائه برقرارند؛ آموزش و ارائه همان مقادیرِ ویژگی را محاسبه می‌کنند؛ مدل‌ها بیش‌ازحد کهنه نیستند؛ مدل از نظرِ عددی پایدار است؛ مدل در سرعتِ آموزش، تأخیرِ ارائه، گذردهی یا مصرفِ RAM پس‌رفت نداشته؛ مدل در کیفیتِ پیش‌بینی روی دادهٔ ارائه‌شده پس‌رفت نداشته.)

خلاصهٔ فصل

این فصل یادگیریِ ماشینی را معرفی کرد و بر یادگیریِ نظارت‌شده از مثال‌ها تمرکز کرد. نکاتِ اصلی:

- یادگیری شکل‌های بسیاری می‌گیرد، بسته به ماهیتِ عامل، مؤلفهٔ موردِ بهبود و بازخوردِ موجود.
- اگر بازخوردِ موجود پاسخِ درست را برای ورودی‌های مثال فراهم کند، مسئلهٔ یادگیری «یادگیریِ نظارت‌شده» نامیده می‌شود. تکلیف یادگیریِ تابعِ $y = h(x)$ است.
- یادگیریِ تابعی که خروجی‌اش مقدارِ پیوسته یا مرتب است (مانندِ وزن) «رگرسیون» نامیده می‌شود؛ یادگیریِ تابعی با شمارِ کمی ردهٔ خروجیِ ممکن «رده‌بندی» نامیده می‌شود.
- می‌خواهیم تابعی یاد بگیریم که نه فقط با داده موافق باشد، بلکه احتمالاً با دادهٔ آینده نیز موافق باشد. باید موافقتِ با داده را با سادگیِ فرضیه متوازن کنیم.
- درخت‌های تصمیم می‌توانند همهٔ توابعِ بولی را بازنمایی کنند. اکتشافیِ بهرهٔ اطلاعاتی روشِ کارآمدی برای یافتنِ درختِ تصمیمِ ساده و سازگار فراهم می‌کند.
- عملکردِ الگوریتمِ یادگیری را می‌توان با منحنیِ یادگیری تجسم کرد، که دقتِ پیش‌بینیِ مجموعهٔ آزمون را به‌عنوانِ تابعی از اندازهٔ مجموعهٔ آموزشی نشان می‌دهد.
- وقتی چند مدل برای انتخاب هست، انتخابِ مدل می‌تواند مقادیرِ خوبِ ابرپارامترها را برگزیند، چنان‌که با اعتبارسنجیِ متقاطع روی دادهٔ اعتبارسنجی تأیید می‌شود. پس از انتخابِ مقادیرِ ابرپارامتر، بهترین مدل‌مان را با همهٔ دادهٔ آموزشی می‌سازیم.
- گاهی همهٔ خطاها یکسان نیستند. تابعِ زیان به ما می‌گوید هر خطا چقدر بد است؛ هدف سپس کمینه‌کردنِ زیان روی مجموعهٔ اعتبارسنجی است.
- نظریهٔ یادگیریِ محاسباتی پیچیدگیِ نمونه‌ای و پیچیدگیِ محاسباتیِ یادگیریِ استقرایی را تحلیل می‌کند. معاوضه‌ای بینِ بیانی‌بودنِ فضایِ فرضیه و سهولتِ یادگیری هست.
- رگرسیونِ خطی مدلِ پرکاربردی است. پارامترهای بهینهٔ مدلِ رگرسیونِ خطی را می‌توان دقیقاً محاسبه کرد، یا با جستجوی نزولِ گرادیان یافت، که فنّی است که می‌توان روی مدل‌هایی که راه‌حلِ بسته ندارند اعمال کرد.
- رده‌بندِ خطی با آستانهٔ سخت—که «پرسپترون» نیز شناخته می‌شود—را می‌توان با قاعدهٔ سادهٔ به‌روزرسانیِ وزن آموزش داد تا داده‌ای را که قابلِ جداسازیِ خطی است برازش کند.
- رده‌بندِ خطی با آستانهٔ نرم—رگرسیونِ لجستیک—مقاوم‌تر از پرسپترون است و با نزولِ گرادیان قابلِ آموزش است.
- مدل‌های ناپارامتری می‌توانند داده را بدونِ فرضیهٔ ازپیش‌انتخاب‌شده دربارهٔ شکلِ تابع برازش کنند. مدلِ k-نزدیک‌ترین-همسایه همهٔ مثال‌ها را برای پیش‌بینیِ تازه نگه می‌دارد؛ ماشینِ بردارِ پشتیبان فقط مثال‌های نزدیک به مرز را نگه می‌دارد.
- روش‌های گروهی مانندِ بیگینگ، جنگل‌های تصادفی و تقویت اغلب بهتر از روش‌های فردی عمل می‌کنند. در یادگیریِ برخط می‌توانیم نظرهای کارشناسان را تجمیع کنیم تا خودسرانه به عملکردِ بهترین کارشناس نزدیک شویم، حتی وقتی توزیعِ داده دائماً در حالِ تغییر است.

- ساختنِ مدلِ یادگیریِ ماشینیِ خوب مستلزم تجربه در فرایندِ کاملِ توسعه است، از مدیریتِ داده تا انتخابِ مدل و بهینه‌سازی، تا نگهداریِ ادامه‌دار.

یادداشت‌های کتاب‌شناختی و تاریخی

فصل ۱ تاریخِ پژوهش‌های فلسفی در موضوعِ یادگیریِ استقرایی را پوشش داد. ویلیام آوکام (۱۳۴۹–۱۲۸۰)، تأثیرگذارترین فیلسوفِ قرنِ خودش و مشارکت‌کنندهٔ اصلی در معرفت‌شناسی، منطق و متافیزیکِ قرونِ وسطی، با گزاره‌ای به نامِ «تیغِ آوکام» شناخته می‌شود—به لاتین، Entia non sunt multiplicanda praeter necessitatem، و به انگلیسی، «موجودات نباید فراتر از ضرورت تکثیر شوند.» متأسفانه، این توصیهٔ ستودنی دقیقاً با این واژه‌ها در نوشته‌هایش یافت نمی‌شود (هرچند او گفت «Pluralitas non est ponenda sine necessitate»، یا «کثرت نباید بدونِ ضرورت فرض شود»). احساسِ مشابهی توسط ارسطو در ۳۵۰ پیش از میلاد در فیزیک، کتابِ ا، فصلِ VI بیان شد: «زیرا محدودتر، اگر کافی باشد، همیشه ترجیح‌پذیر است.» دیوید هیوم (۱۷۷۶–۱۷۱۱) مسئلهٔ استقرا را صورت‌بندی کرد، با تشخیصِ اینکه تعمیمِ از مثال‌ها امکانِ خطاها را می‌پذیرد، به‌گونه‌ای که قیاسِ منطقی نمی‌کند. او دید هیچ راهی برای داشتنِ راه‌حلِ تضمین‌شدهٔ درست برای مسئله نیست، اما اصلِ «یکنواختیِ طبیعت» را پیشنهاد کرد، که ما آن را ایستایی نامیده‌ایم. آنچه آوکام و هیوم به آن می‌رسیدند این است که وقتی استقرا می‌کنیم، از میانِ انبوهِ مدل‌های سازگار یکی را انتخاب می‌کنیم که محتمل‌تر است—زیرا ساده‌تر است و با انتظاراتِ ما مطابقت دارد. در روزگارِ مدرن، «قضیهٔ ناهارِ رایگان» (No Free Lunch Theorem؛ وولپرت و مک‌کریدی، 1997؛ وولپرت، 2013) می‌گوید اگر الگوریتمِ یادگیری روی مجموعهٔ خاصی از مسائل خوب عمل کند، فقط به این دلیل است که روی مجموعهٔ متفاوتی ضعیف عمل خواهد کرد: اگر درختِ تصمیمِ ما انتظارِ رستورانِ SR را درست پیش‌بینی کند، فقط به این دلیل است که در پیش‌بینیِ مسائلِ دیگر، که قوانینِ متفاوتی دارند، ضعیف‌تر خواهد بود.

تحلیلِ نظریِ الگوریتم‌های یادگیری با کارِ گلد (1967) دربارهٔ «شناسایی در حدِّ نهایی» (Identification in the Limit) آغاز شد. این رویکرد تا حدی با مدل‌های کشفِ علمی از فلسفهٔ علم انگیزه‌مند شد (پوپر، 1962)، اما عمدتاً برای مسئلهٔ یادگیریِ گرامرها از جمله‌های مثال اعمال شده (آشرسون و همکاران، 1986). در حالی که رویکردِ شناسایی-در-حدِّ-نهایی بر همگراییِ نهایی تمرکز دارد، مطالعهٔ «پیچیدگیِ کولموگوروف» (Kolmogorov Complexity) یا پیچیدگیِ الگوریتمی، که مستقلانه توسط سولومونوف (1964، 2009) و کولموگوروف (1965) توسعه یافت، تلاش می‌کند تعریفِ صوریِ مفهومِ سادگیِ استفاده‌شده در تیغِ آوکام فراهم کند. برای گریز از مسئلهٔ وابستگیِ سادگی به شیوهٔ بازنماییِ اطلاعات، پیشنهاد می‌شود سادگی با طولِ کوتاه‌ترین برنامه برای ماشینِ تورینگِ همگانی که دادهٔ مشاهده‌شده را درست بازتولید می‌کند اندازه‌گیری شود. اگرچه ماشین‌های تورینگِ همگانیِ بسیاری ممکن‌اند، و از این رو «کوتاه‌ترین» برنامه‌های بسیاری، این برنامه‌ها در طول حداکثر با ثابتی تفاوت دارند که مستقل از مقدارِ داده است. این بینشِ زیبا، که اساساً نشان می‌دهد هر اریبیِ بازنماییِ اولیه سرانجام توسط داده غلبه خواهد شد، فقط با تصمیم‌ناپذیریِ محاسبهٔ طولِ کوتاه‌ترین برنامه لکه‌دار می‌شود. اندازه‌های تقریبی مانندِ حداقلِ طولِ توصیف، یا MDL (ریسانن، 1984، 2007) می‌توانند به‌جایش استفاده شوند و در عمل نتایجِ عالی تولید کرده‌اند. متنِ لی و ویتانی (2008) بهترین منبع برای پیچیدگیِ کولموگوروف است.

نظریهٔ یادگیریِ PAC توسط لزلی والیانت (1984) افتتاح شد، با تأکید بر اهمیتِ پیچیدگیِ محاسباتی و نمونه‌ای. والیانت با مایکل کرنز (1990) نشان داد چند ردهٔ مفهوم را نمی‌توان به‌صورتِ شدنی PAC-یاد گرفت، حتی اگرچه اطلاعاتِ کافی در مثال‌ها موجود باشد. برخی نتایجِ مثبت برای رده‌هایی مانندِ فهرست‌های تصمیم (ریوست، 1987) به دست آمد.

سنتِ مستقلی از تحلیلِ پیچیدگیِ نمونه‌ای در آمار وجود داشته، با شروع از کارِ نظریهٔ همگراییِ یکنواخت (واپنیک و چروونکیس، 1971). به اصطلاح «بُعدِ VC» اندازه‌ای فراهم می‌کند که تقریباً مشابهِ، اما عمومی‌تر از، اندازهٔ |H|ln به‌دست‌آمده از تحلیلِ PAC است. بُعدِ VC را می‌توان روی رده‌های تابعِ پیوسته اعمال کرد، که تحلیلِ استانداردِ PAC روی آن‌ها اعمال نمی‌شود. نظریهٔ یادگیریِ PAC و نظریهٔ VC نخستین بار توسط «چهار آلمانی» (که هیچ‌کدام در واقع آلمانی نیستند) به هم متصل شدند: بلومر، ارن‌فویخت، هاوسلر و وارموث (1989).

رگرسیونِ خطی با زیانِ مربع‌خطا به لژاندر (1805) و گاوس (1809) برمی‌گردد، که هر دو روی پیش‌بینیِ مدارهای اطرافِ خورشید کار می‌کردند. (گاوس ادعا کرد از 1795 از این فن استفاده می‌کرده، اما در انتشارش تأخیر کرد.) استفادهٔ مدرن از رگرسیونِ چندمتغیره برای یادگیریِ ماشینی در متن‌هایی مانندِ بیشاپ (2007) پوشش داده شده. تفاوت‌های بینِ منظم‌سازیِ L1 و L2 توسط نگ (2004) و مور و دنیرو (2011) تحلیل شده.

اصطلاحِ تابعِ لجستیک از پیر-فرانسوا ورولست (۱۸۴۹–۱۸۰۴) می‌آید، آماردانی که از منحنی برای مدل‌کردنِ رشدِ جمعیت با منابعِ محدود استفاده کرد، مدلی واقع‌گرایانه‌تر از رشدِ هندسیِ بی‌قیدِ پیشنهادیِ توماس مالتوس. ورولست آن را courbe logistique نامید، به دلیلِ رابطه‌اش با منحنیِ لگاریتمی. اصطلاحِ «نفرینِ ابعاد» از ریچارد بلمن (1961) می‌آید.

رگرسیونِ لجستیک را می‌توان با نزولِ گرادیان یا با روشِ نیوتن–رافسون (نیوتن، 1671؛ رافسون، 1690) حل کرد. گونه‌ای از روشِ نیوتن به نامِ L-BFGS اغلب برای مسائلِ پربعد استفاده می‌شود؛ L مخففِ «حافظهٔ محدود» است، بدان معنا که از ایجادِ ماتریس‌های کاملِ یکجا اجتناب می‌کند و در عوض بخش‌هایشان را در پرواز می‌سازد. BFGS حروفِ اولِ نامِ نویسندگان است (برد و همکاران، 1995). ایدهٔ نزولِ گرادیان به کوشی (1847) برمی‌گردد؛ نزولِ گرادیانِ تصادفی (SGD) در جامعهٔ بهینه‌سازیِ آماری توسط رابینز و مونرو (1951) معرفی شد، برای شبکه‌های عصبی توسط روزن‌بلت (1960) دوباره کشف شد، و برای یادگیریِ ماشینیِ مقیاس‌بزرگ توسط بوتو و بوسکه (2008) رواج یافت. بوتو و همکاران (2018) موضوعِ یادگیریِ مقیاس‌بزرگ را با یک دهه تجربهٔ اضافی بازبینی می‌کنند. مدل‌های نزدیک‌ترین-همسایه‌ها دست‌کم به فیکس و هاجز (1951) برمی‌گردند و از آن پس ابزارِ استانداردی در آمار و بازشناسیِ الگو بوده‌اند. درونِ هوش مصنوعی، توسط استنفیل و والتز (1986) رواج یافتند، که روش‌هایی برای سازگارکردنِ معیارِ فاصله با داده بررسی کردند. هاستی و تیبشیرانی (1996) راهی برای محلی‌کردنِ معیار در هر نقطهٔ فضا، بسته به توزیعِ داده دورِ آن نقطه، توسعه دادند. گیونیس و همکاران (1999) درهم‌سازیِ حساس-به-محل (LSH) را معرفی کردند، که بازیابیِ اشیای مشابه در فضاهایِ پربعد را متحول کرد. آندونی و ایندیک (2006) مروری بر LSH و روش‌های مرتبط فراهم می‌کنند، و سامت (2006) ویژگی‌های فضاهایِ پربعد را پوشش می‌دهد. این فن برای دادهٔ ژنومی، که هر رکورد میلیون‌ها ویژگی دارد، به‌طورِ خاص مفید است (برلین و همکاران، 2015).

ایده‌های پشتِ ماشین‌های هسته از آیزنمن و همکاران (1964) می‌آیند (که حقهٔ هسته را نیز معرفی کردند)، اما توسعهٔ کاملِ نظریه از واپنیک و همکارانش است (بوزر و همکاران، 1992). SVMها با معرفیِ رده‌بندِ حاشیهٔ نرم برای مدیریتِ دادهٔ نویزی عملی شدند، در مقاله‌ای که جایزهٔ ACM Theory and Practice 2008 را برد (کورتس و واپنیک، 1995)، و با الگوریتمِ «بهینه‌سازیِ کمینهٔ متوالی» (SMO) برای حلِ کارآمدِ مسائلِ SVM با برنامه‌نویسیِ درجه‌دومی (پلات، 1999).

SVMها برای تکلیف‌هایی مانندِ رده‌بندیِ متن (یوآخیمز، 2001)، ژنومیکسِ محاسباتی (کریستینینی و هان، 2007) و بازشناسیِ رقمِ دست‌نویس دکوست و شولکف (2002) بسیار مؤثر ثابت شده‌اند. به‌عنوانِ بخشی از این فرایند، هسته‌های تازهٔ بسیاری طراحی شده که با رشته‌ها، درخت‌ها و دیگر انواعِ دادهٔ غیرعددی کار می‌کنند. فنِ مرتبطی که از حقهٔ هسته برای بازنماییِ ضمنیِ فضایِ ویژگیِ نمایی استفاده می‌کند، پرسپترونِ رأی‌دهنده (فرند و شاپایر، 1999؛ کالینز و دافی، 2002) است. کتاب‌های درسیِ SVM شاملِ کریستینینی و شاو-تیلور (2000) و شولکف و اسموالا (2002) می‌شوند. شرحِ دوستانه‌تری در مقالهٔ AI Magazine کریستینینی و شولکف (2002) ظاهر می‌شود. بنجیو و لوکون (2007) برخی از محدودیت‌های SVMها و دیگر روش‌های محلی و ناپارامتری برای یادگیریِ توابعی را نشان می‌دهند که ساختارِ سراسری دارند اما نرمیِ محلی ندارند.

نخستین اثباتِ ریاضیِ ارزشِ یک گروه، قضیهٔ هیئت‌منصفهٔ کندورسه (1785) بود، که اثبات کرد اگر اعضای هیئت‌منصفه مستقل‌اند و عضوِ فردیِ هیئت‌منصفه دست‌کم ۵۰٪ شانسِ تصمیمِ درستِ پرونده دارد، آن‌گاه هرچه اعضای هیئت‌منصفه بیشتری بیفزایید، شانسِ تصمیمِ درستِ پرونده بهتر می‌شود. اخیراً، یادگیریِ گروهی به فنِ فزاینده‌ای محبوب برای بهبودِ عملکردِ الگوریتم‌های یادگیری تبدیل شده.

نخستین الگوریتمِ جنگلِ تصادفی، با انتخابِ تصادفیِ ویژگی، از هو (1995) است؛ نسخهٔ مستقلی توسط امیت و گمن (1997) معرفی شد. بریمن (2001) ایده‌های بیگینگ و «خطای بیرون-از-کیسه» را افزود. فریدمن (2001) اصطلاحِ «ماشینِ تقویتِ گرادیان» (GBM) را معرفی کرد و رویکرد را برای اجازه‌دادن به رده‌بندیِ چندرده‌ای، رگرسیون و مسائلِ رتبه‌بندی بسط داد.

مایکل کرنز (1988) «مسئلهٔ تقویتِ فرضیه» (Hypothesis Boosting Problem) را تعریف کرد: با توجه به یادگیرنده‌ای که فقط کمی بهتر از حدسِ تصادفی پیش‌بینی می‌کند، آیا امکانِ استخراجِ یادگیرنده‌ای هست که خودسرانه خوب عمل کند؟ این مسئله در مقالهٔ نظریِ شاپایر (1990) به‌صورتِ مثبت پاسخ داده شد، که به الگوریتمِ ADABOOST فرند و شاپایر (1996) و کارِ نظریِ بیشتر (شاپایر، 2003) انجامید. فریدمن و همکاران (2000) تقویت را از دیدگاهِ آماردان تبیین می‌کنند. چن و گسترین (2016) سامانهٔ XGBOOST را توصیف می‌کنند، که با موفقیتِ زیاد در بسیاری از کاربردهای مقیاس‌بزرگ استفاده شده.

یادگیریِ برخط در مروری از بلوم (1996) و کتابی از سزا-بیانکی و لوگوسی (2006) پوشش داده شده. دردز و همکاران (2008) ایدهٔ یادگیریِ برخطِ وزندار-با-اطمینان را برای رده‌بندی معرفی می‌کنند: علاوه بر نگه‌داشتنِ وزنی برای هر پارامتر، اندازه‌ای از اطمینان نیز نگه می‌دارند، تا مثالِ تازه بتواند اثرِ بزرگی بر ویژگی‌هایی داشته باشد که قبلاً به‌ندرت دیده شده‌اند (و بنابراین اطمینانِ کمی داشتند) و اثرِ کوچکی بر ویژگی‌های رایجی که قبلاً خوب برآورد شده‌اند. یو و همکاران (2011) توصیف می‌کنند چگونه تیمی از دانشجویان با هم برای ساختِ رده‌بندِ گروهی در مسابقهٔ KDD همکاری

می‌کنند. امکانِ هیجان‌انگیزی ایجادِ گروهِ «بیش‌ازحدِ بزرگِ» مخلوط-کارشناسان است که از زیرمجموعهٔ تنکی از کارشناسان برای هر مثالِ ورودی استفاده می‌کند (شازیر و همکاران، 2017). سنی و الدر (2010) روش‌های گروهی را مرور می‌کنند. از نظرِ توصیه‌های عملی برای ساختنِ سامانه‌های یادگیریِ ماشینی، پدرو دومینگوس چند چیزِ دانستنی را توصیف می‌کند (2012). اندرو نگ نکاتی برای توسعه و رفع‌اشکالِ محصولی با استفاده از یادگیریِ ماشینی می‌دهد (نگ، 2019). اونیل و شوت (2013) فرایندِ انجامِ علمِ داده را توصیف می‌کنند. توکی (1977) تحلیلِ دادهٔ اکتشافی را معرفی کرد، و گلمن (2004) دیدگاهِ به‌روزشده‌ای از فرایند می‌دهد. بین و همکاران (2011) فرایندِ انتخابِ نمونه‌های اولیه برای تفسیرپذیری را توصیف می‌کنند، و کیم و همکاران (2017) نشان می‌دهند چگونه منتقدانی را بیابیم که با معیاری به نامِ «حداکثرِ ناهمخوانیِ میانگین» حداکثرِ فاصله را از نمونه‌های اولیه دارند.

واتنبرگ و همکاران (2016) استفاده از t-SNE را توصیف می‌کنند. برای دیدِ جامعِ اینکه سامانهٔ یادگیریِ ماشینیِ مستقرت چقدر خوب عمل می‌کند، برک و همکاران (2016) چک‌لیستی از ۲۸ آزمون ارائه می‌دهند که می‌توانی برای گرفتنِ امتیازِ کلیِ آزمونِ ML اعمال کنی. رایلی (2019) سه دامِ رایجِ توسعهٔ ML را توصیف می‌کند.

بانکو و بریل (2001)، هالوی و همکاران (2009) و گاندومی و حیدر (2015) مزایای استفاده از مقادیرِ زیادِ داده‌ای که اکنون در دسترس است را بحث می‌کنند. لیمن و واریان (2003) برآورد کردند حدود ۵ اگزابایت (5×10^{18} بایت) داده در ۲۰۰۲ تولید شده، و نرخِ تولید هر ۳ سال دو برابر می‌شود؛ هیلبرت و لوپز (2011) 2×10^{21} بایت را برای ۲۰۰7 برآورد کردند، که نشان‌دهندهٔ شتاب است. گیون و الیزه‌ف (2003) مسئلهٔ انتخابِ ویژگی با مجموعه‌های دادهٔ بزرگ را بحث می‌کنند.

دوشی-ولز و کیم (2017) چارچوبی برای یادگیریِ ماشینیِ تفسیرپذیر یا هوش مصنوعیِ تبیین‌پذیر (XAI) پیشنهاد می‌کنند. میلر و همکاران (2017) اشاره می‌کنند دو نوع تبیین هست، یکی برای طراحانِ سامانهٔ هوش مصنوعی و یکی برای کاربران، و باید روشن باشیم هدف‌مان چیست. سامانهٔ LIME (ریبیرو و همکاران، 2016) مدل‌های خطیِ تفسیرپذیری می‌سازد که هر سامانهٔ یادگیریِ ماشینی‌ای که داری را تقریب می‌زنند. سامانهٔ مشابهی، SHAP (لوندبرگ و لی، 2018) (تبیین‌های افزایشیِ شپلی)، از مفهومِ ارزشِ شپلی (صفحهٔ ۶۲۸) برای تعیینِ سهمِ هر ویژگی استفاده می‌کند.

ایدهٔ اینکه بتوانیم یادگیریِ ماشینی را برای حلِ مسائلِ یادگیریِ ماشینی به کار ببریم، وسوسه‌انگیز است. ترون و پرت (2012) مرورِ اولیه‌ای از این حوزه در مجموعه‌ای ویرایش‌شده با عنوانِ «یادگیری برای یادگیری» می‌دهند. اخیراً این حوزه نامِ «یادگیریِ ماشینیِ خودکار» (Automated Machine Learning یا AutoML) را پذیرفته؛ هوتر و همکاران (2019) مروری می‌دهند. کانتر و ویراماچانی (2015) سامانه‌ای برای انجامِ انتخابِ خودکارِ ویژگی توصیف می‌کنند. برگسترا و بنجیو (2012) سامانه‌ای برای جستجو در فضایِ ابرپارامترها توصیف می‌کنند، همان‌گونه که تورنتون و همکاران (2013) و برمودز-چاکون و همکاران (2015). وانگ و همکاران (2019) نشان می‌دهند یادگیریِ انتقالی چگونه می‌تواند AutoML را برای مدل‌های یادگیریِ عمیق سرعت ببخشد. مسابقاتی برای دیدنِ اینکه کدام سامانه‌ها در تکلیف‌های AutoML بهترین‌اند سازمان‌دهی شده (گیون و همکاران، 2015).

اشتاین‌روکن و همکاران (2019) سامانه‌ای به نامِ «آماردانِ خودکار» (Automatic Statistician) توصیف می‌کنند: داده‌ای به آن می‌دهی و گزارشی می‌نویسد، متن، نمودارها و محاسبات را در هم می‌آمیزد. ارائه‌دهندگانِ اصلیِ رایانشِ ابری AutoML را بخشی از پیشنهادهایشان کرده‌اند. برخی پژوهشگران اصطلاحِ «فرا-یادگیری» (Metalearning) را ترجیح می‌دهند: برای نمونه، سامانهٔ MAML (فرا-یادگیریِ مستقل از مدل) (فین و همکاران، 2017) با هر مدلی که با نزولِ گرادیان قابلِ آموزش است کار می‌کند؛ مدلِ هسته‌ای را آموزش می‌دهد تا تنظیمِ دقیقِ مدل با دادهٔ تازه روی تکلیف‌های تازه آسان باشد.

با وجودِ همهٔ این کارها، هنوز سامانهٔ کاملی برای حلِ خودکارِ مسائلِ یادگیریِ ماشینی نداریم. برای انجامِ آن با یادگیریِ نظارت‌شده باید با مجموعهٔ داده‌ای از مثال‌های (x_j,y_j) شروع کنیم. در این‌جا ورودیِ x_j مشخصاتِ مسئله است، به شکلی که مسئله در آغاز با آن مواجه می‌شود: توصیفِ مبهمی از هدف‌ها، و داده‌ای برای کار، شاید با برنامه‌ای مبهم برای چگونگیِ کسبِ دادهٔ بیشتر. خروجیِ y_j برنامهٔ کامل و در حالِ اجرایِ یادگیریِ ماشینی خواهد بود، همراه با روش‌شناسی‌ای برای نگهداریِ برنامه: جمع‌آوریِ دادهٔ بیشتر، تمیزکردنِ آن، و...

پانوشت‌ها:

[1] نامِ بهتر «تقریبِ تابع» یا «پیش‌بینیِ عددی» می‌بود. اما در ۱۸۸۶ فرانسیس گالتون مقالهٔ تأثیرگذاری دربارهٔ مفهومِ بازگشت به میانگین نوشت (مثلاً فرزندانِ والدینِ قدبلند احتمالاً بلندتر از میانگین‌اند، اما نه به‌اندازهٔ والدین). گالتون نمودارهایی را با چیزی که «خطوطِ رگرسیون» می‌نامید نشان داد، و خوانندگان به‌مرور واژهٔ «رگرسیون» را با فنِ آماریِ تقریبِ تابع مرتبط کردند.

[2] نام اغلب «Occam» غلط املاشده.

[3] بهره به‌صورتِ سخت‌گیرانه مثبت خواهد بود، به‌جزِ حالتِ نامحتملی که همهٔ نسبت‌ها دقیقاً یکسان‌اند. (تمرین NNGA.۱۹ را ببینید.)

[4] اگرچه نامِ «انتخابِ مدل» رایج است، نامِ بهتر «انتخابِ ردهٔ مدل» یا «انتخابِ فضایِ فرضیه» می‌بود. واژهٔ «مدل» در ادبیات برای اشاره به سه سطحِ متفاوتِ خاصیت استفاده شده: فضایِ فرضیهٔ گسترده (مانندِ «چندجمله‌ای»)، خانوادهٔ پارامتریِ درونِ آن فضا (مانندِ «درجهٔ 3»)، و فرضیهٔ کاملاً مشخص‌شده (مانندِ «$1 + x + 2x^2 + 3x^3$»).

[5] برخی نویسندگان می‌گویند مدل داده را «به خاطر سپرده».

[6] گاوس نشان داد اگر مقادیرِ y_j نویزِ توزیعِ نرمال داشته باشند، محتمل‌ترین مقادیرِ w_1 و w_0 با استفاده از زیانِ L2، با کمینه‌کردنِ جمعِ مربعِ خطاها، به دست می‌آیند. (اگر مقادیر نویزی با توزیعِ لاپلاس (نمایی دوتایی) داشته باشند، زیانِ L1 مناسب‌تر است.)

[7] با چند ملاحظه: تابعِ زیانِ L2 وقتی مناسب است نویزِ توزیعِ نرمالِ مستقل از x باشد؛ همهٔ نتایج به فرضِ ایستایی تکیه دارند؛ ...۹

[8] خواننده ممکن است بخواهد برای خلاصه‌ای از جبرِ خطی به پیوستِ الف مراجعه کند. همچنین توجه کنید از اصطلاحِ «رگرسیونِ چندمتغیره» برای این معنا استفاده می‌کنیم که ورودی بردارِ چند مقدار است، اما خروجی متغیرِ تکی است. از اصطلاحِ «رگرسیونِ چندمتغیره‌خروجی» برای حالتی استفاده می‌کنیم که خروجی نیز بردارِ چند متغیر است.

[9] شاید گیج‌کننده باشد که نمادِ L1 و L2 هم برای توابعِ زیان و هم برای توابعِ منظم‌سازی استفاده می‌شود. لازم نیست جفت استفاده شوند: می‌توانی از زیانِ L2 با منظم‌سازیِ L1 استفاده کنی، یا برعکس.

[10] از نظرِ فنی، لازم داریم ∑_{t=1}^{∞} α(t) = ∞ و ∑_{t=1}^{∞} α²(t) < ∞. نرخِ یادگیریِ α(t) = O(1/t) این شرایط را ارضا می‌کند. اغلب از c/(c+t) برای ثابتِ نسبتاً بزرگِ c استفاده می‌کنیم.

[11] تابعِ sign(x) برای x مثبت +1 و برای x منفی -1 برمی‌گرداند.

[12] خواننده ممکن است متوجه شود می‌توانستیم فقط از f1 و f2 استفاده کنیم، اما نگاشتِ سه‌بعدی ایده را بهتر نشان می‌دهد.

[13] این استفاده از «تابعِ هسته» کمی با هسته‌های رگرسیونِ وزن‌محلی متفاوت است. برخی هسته‌های SVM معیارهای فاصله‌اند، اما نه همه‌شان.

[14] در این‌جا، «معقول» بدان معناست ماتریسِ Kjk = K(xj,xk) معینِ مثبت است.

[15] یادداشتِ اصطلاح‌شناسی: در آمار، نمونه‌برداریِ با جایگزینی «بوت‌استرپ» (Bootstrap) نامیده می‌شود، و «بیگینگ» کوتاه‌شدهٔ «تجمیعِ بوت‌استرپ» (Bootstrap Aggregating) است.

[16] بلوم (1996) اثباتِ زیبایی می‌دهد.

[17] (در متنِ اصلیِ انگلیسی، پانوشتِ ۱۷ در بخشِ ۱۹.۹.۲ دربارهٔ حاشیهٔ نرمِ SVM آمده و در ترجمه در بخشِ ۱۹.۷.۶ گنجانده شده است.)

[18] این اصطلاح‌شناسی به‌طورِ همگانی پذیرفته نشده؛ برخی نویسندگان «تفسیرپذیر» و «تبیین‌پذیر» را مترادف به کار می‌برند، هر دو اشاره به رسیدن به نوعی فهم از مدل.

فصل ۲۰: یادگیری مدل‌های احتمالاتی (Learning (Probabilistic Models

در این فصل یادگیری را شکلی از استدلالِ نامطمئن از مشاهده‌ها می‌بینیم و مدل‌هایی برای بازنماییِ جهانِ نامطمئن طراحی می‌کنیم.

فصل ۱۲ به شیوعِ عدم قطعیت در محیط‌های واقعی اشاره کرد. عامل‌ها می‌توانند با استفاده از روش‌های احتمال و نظریهٔ تصمیم عدم قطعیت را مدیریت کنند، اما نخست باید نظریه‌های احتمالاتی‌شان از جهان را از تجربه یاد بگیرند. این فصل توضیح می‌دهد چگونه می‌توانند این کار را بکنند، با صورت‌بندیِ خودِ تکلیفِ یادگیری به‌عنوانِ فرایندی از استنتاجِ احتمالاتی (بخش ۲۰.۱). خواهیم دید دیدگاهِ بیزی از یادگیری بسیار قدرتمند است، و راه‌حل‌های عمومی برای مسائلِ نویز، بیش‌برازش و پیش‌بینیِ بهینه فراهم می‌کند. همچنین این واقعیت را در نظر می‌گیرد که عاملِ کمتر از دانایِ مطلق هرگز نمی‌تواند دربارهٔ درست‌بودنِ کدام نظریه از جهان مطمئن باشد، و با این حال باید با استفاده از برخی نظریه‌های جهان تصمیم بگیرد.

روش‌هایی برای یادگیریِ مدل‌های احتمال—عمدتاً شبکه‌های بیزی—در بخش‌های ۲۰.۲ و ۲۰.۳ توصیف می‌کنیم. برخی از مطالبِ این فصل نسبتاً ریاضی‌اند، هرچند درس‌های عمومی را می‌توان بدونِ فرو رفتن در جزئیات فهمید. شاید برای خواننده مفید باشد فصل‌های ۱۲ و ۱۳ را مرور کند و به پیوستِ الف نگاهی بیندازد.

۲۰.۱ یادگیریِ آماری

مفاهیمِ کلیدیِ این فصل، درست مانندِ فصل ۱۹، داده و فرضیه‌اند. در این‌جا، داده شواهد است—یعنی مصداق‌هایی از برخی یا همهٔ متغیرهای تصادفیِ توصیف‌کنندهٔ حوزه. فرضیه‌های این فصل نظریه‌های احتمالاتیِ چگونگیِ کارکردنِ حوزه‌اند، از جملهٔ نظریه‌های منطقی به‌عنوانِ حالتِ خاص.

مثالِ ساده‌ای در نظر بگیرید. آب‌نباتِ سورپرایزِ موردعلاقه‌مان در دو طعم می‌آید: گیلاس (مم) و لیمو (اف). تولیدکننده حسِ شوخ‌طبعیِ عجیبی دارد و هر تکه آب‌نبات را در همان لفافِ مات می‌پیچد، صرف‌نظر از طعم. آب‌نبات در کیسه‌های بسیار بزرگ فروخته می‌شود، که معلوم است پنج نوع‌اند—باز، از بیرون غیرقابلِ تمایز:

```
h1: 100% گیلاس،     h2: 75% گیلاس + 25% لیمو،     h3: 50% گیلاس + 50% لیمو،
h4: 25% گیلاس + 75% لیمو،     h5: 100% لیمو
```

با توجه به کیسهٔ تازه‌ای از آب‌نبات، متغیرِ تصادفیِ H (برای فرضیه) نوعِ کیسه را نشان می‌دهد، با مقادیرِ ممکنِ h1 تا h5. H البته مستقیماً قابلِ مشاهده نیست. همان‌طور که تکه‌های آب‌نبات باز و بازرسی می‌شوند، داده آشکار می‌شود —D1, D2, ..., DN، که در آن هر Di متغیرِ تصادفی با مقادیرِ ممکنِ گیلاس و لیمو است. تکلیفِ پایه‌ایِ پیشِ رویِ عامل پیش‌بینیِ طعمِ تکهٔ بعدیِ آب‌نبات است.[1] با وجودِ بدیهی‌بودنِ ظاهری‌اش، این سناریو برای معرفیِ بسیاری از مسائلِ اصلی خدمت می‌کند. عامل واقعاً باید نظریه‌ای از جهانش استنتاج کند، هرچند بسیار ساده.

«یادگیریِ بیزی» (Bayesian Learning) به‌سادگی احتمالِ هر فرضیه را، با توجه به داده، محاسبه می‌کند و پیش‌بینی‌ها را بر آن اساس انجام می‌دهد. یعنی، پیش‌بینی‌ها با استفاده از همهٔ فرضیه‌ها، وزن‌گذاری‌شده با احتمال‌هایشان، انجام می‌شوند، نه با استفاده از فقط یک فرضیهٔ «بهترینِ» تکی. بدین‌گونه، یادگیری به استنتاجِ احتمالاتی کاهش می‌یابد.

D همهٔ داده‌ها با مقدارِ مشاهده‌شدهٔ d باشد. کمیت‌های کلیدی در رویکردِ بیزی «پیشینِ فرضیه» (Hypothesis Prior)، P(hi)، و «درستنماییِ» (Likelihood) داده تحتِ هر فرضیه، P(d|hi)، هستند. احتمالِ هر فرضیه با قاعدهٔ بیز به‌دست می‌آید:

$$P(hi|d) = \alpha P(d|hi)P(hi) \qquad (20.1)$$

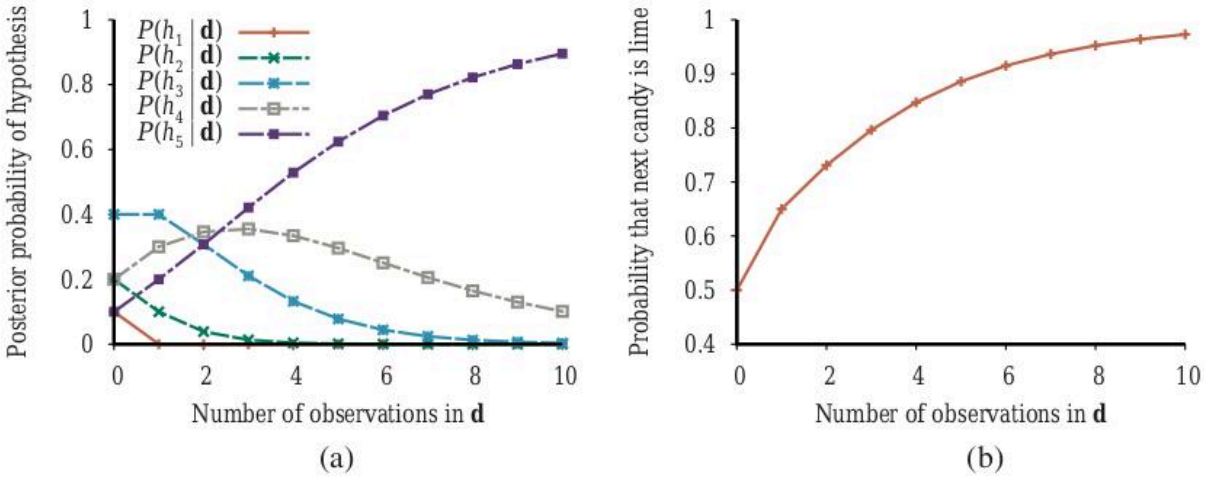
حالا، فرض کنیم می‌خواهیم دربارهٔ کمیتِ ناشناخته‌ای X پیش‌بینی کنیم. سپس:

$$P(X|d) = \sum_i P(X|hi)P(hi|d) \qquad (20.2)$$

داریم، که در آن هر فرضیه توزیعِ احتمالی روی X تعیین می‌کند. این معادله نشان می‌دهد پیش‌بینی‌ها میانگین‌های وزنیِ پیش‌بینی‌های فرضیه‌های فردی‌اند، که در آن وزنِ P(hi|d) متناسب با احتمالِ پیشینیِ hi و درجهٔ برازشِ آن، طبقِ معادلهٔ (20.1)، است. خودِ فرضیه‌ها اساساً «واسطه‌هایی» بینِ دادهٔ خام و پیش‌بینی‌ها هستند. برای مثالِ آب‌نبات‌مان، فعلاً فرض می‌کنیم توزیعِ پیشینی روی h1,...,h5 با ⟨0.1, 0.2, 0.4, 0.2, 0.1⟩ داده می‌شود، چنان‌که تولیدکننده تبلیغ کرده. درستنماییِ داده تحتِ فرضِ i.i.d. بودنِ مشاهده‌ها (صفحهٔ ۶۶۵ را ببینید) محاسبه می‌شود، پس:

$$P(d|hi) = \prod_j P(dj|hi) \qquad (20.3)$$

برای نمونه، فرض کنیم کیسه واقعاً کیسهٔ تمام‌لیمو (h5) است و ۱۰ آب‌نباتِ نخست همه لیمو باشند؛ سپس P(d|h3) برابر 0.5^{10} است، زیرا نیمی از آب‌نبات‌های کیسهٔ h3 لیمو هستند.[2] شکل ۲۰.۱(الف) نشان می‌دهد احتمال‌های پسینیِ پنج فرضیه چگونه با مشاهدهٔ توالیِ ۱۰ آب‌نباتِ لیمو تغییر می‌کنند. توجه کنید احتمال‌ها از مقادیرِ پیشینی‌شان شروع می‌شوند، پس h3 در آغاز محتمل‌ترین انتخاب است و پس از بازشدنِ ۱ آب‌نباتِ لیمو نیز چنین می‌ماند. پس از بازشدنِ ۲ آب‌نباتِ لیمو، h4 محتمل‌ترین است؛ پس از ۳ یا بیشتر، h5 (کیسهٔ هراسناکِ تمام‌لیمو) محتمل‌ترین است. پس از ۱۰ تای متوالی، نسبتاً از سرنوشتمان مطمئنیم. شکل ۲۰.۱(ب) احتمالِ پیش‌بینی‌شدهٔ لیموبودنِ آب‌نباتِ بعدی را، بر اساسِ معادلهٔ (20.2)، نشان می‌دهد. چنان‌که انتظار داریم، به‌صورتِ یکنواخت به سمتِ 1 افزایش می‌یابد.



شکل ۲۰.۱ – (الف) احتمال‌های پسینیِ P(hi|d1,...,dN) از معادلهٔ (20.1). شمارِ مشاهده‌های N از 1 تا 10 متغیر است، و هر مشاهده آب‌نباتِ لیموست. (ب) پیش‌بینیِ بیزیِ P(DN+1 = lime|d1,...,dN) از معادلهٔ (20.2).

مثال نشان می‌دهد پیش‌بینیِ بیزی سرانجام با فرضیهٔ واقعی موافق می‌شود. این ویژگیِ یادگیریِ بیزی است. برای هر پیشینِ ثابتی که فرضیهٔ واقعی را رد نمی‌کند، احتمالِ پسینیِ هر فرضیهٔ کاذبی، تحتِ شرایطِ فنیِ معینی، سرانجام ناپدید خواهد شد. این صرفاً به این دلیل رخ می‌دهد که احتمالِ تولیدِ نامحدودِ دادهٔ «غیرمشخصه» به‌طرزِ محوکننده‌ای کوچک است. (این نکته مشابهِ نکته‌ای است که در بحثِ یادگیریِ PAC در فصل ۱۹ مطرح شد.) مهم‌تر، پیش‌بینیِ بیزی بهینه است، چه مجموعهٔ داده کوچک باشد چه بزرگ. با توجه به پیشینِ فرضیه، هر پیش‌بینیِ دیگری انتظار می‌رود کمتر درست باشد.

بهینگیِ یادگیریِ بیزی، البته، بهایی دارد. برای مسائلِ یادگیریِ واقعی، فضایِ فرضیه معمولاً بسیار بزرگ یا نامتناهی است، چنان‌که در فصل ۱۹ دیدیم. در برخی موارد، جمعِ معادلهٔ (20.2) (یا انتگرال، در حالتِ پیوسته) می‌تواند به‌صورتِ شدنی انجام شود، اما در بیشترِ موارد باید به روش‌های تقریبی یا ساده‌شده پناه ببریم.

تقریبِ بسیار رایجی—که معمولاً در علم اتخاذ می‌شود—انجامِ پیش‌بینی‌ها بر اساسِ فرضیهٔ محتمل‌ترینِ تکی است—یعنی hi که P(hi|d) را بیشینه می‌کند. این اغلب فرضیهٔ «بیشینهٔ پسین» (Maximum A Posteriori یا MAP، تلفظ «اِم-اِی-پی») نامیده می‌شود. پیش‌بینی‌های انجام‌شده طبقِ فرضیهٔ hMAP ِMAP تا آن‌جا تقریباً بیزی‌اند که P(X|d) ≈ P(X|hMAP). در مثالِ آب‌نبات‌مان، پس از سه آب‌نباتِ لیمویِ متوالی hMAP = h5 است، پس یادگیرندهٔ MAP سپس پیش‌بینی می‌کند آب‌نباتِ چهارم با احتمالِ 1.0 لیموست—پیش‌بینیِ بسیار خطرناک‌تری از پیش‌بینیِ بیزیِ 0.8 نشان‌داده‌شده در شکل ۲۰.۱(ب). با رسیدنِ دادهٔ بیشتر، پیش‌بینی‌های MAP و بیزی نزدیک‌تر می‌شوند، زیرا رقبای فرضیهٔ MAP کم‌محتمل‌تر و کم‌محتمل‌تر می‌شوند.

اگرچه این مثال نشان نمی‌دهد، یافتنِ فرضیه‌های MAP اغلب بسیار آسان‌تر از یادگیریِ بیزی است، زیرا به حلِ مسئلهٔ بهینه‌سازی نیاز دارد، نه مسئلهٔ جمعِ (یا انتگرالِ) بزرگ.

در هر دو یادگیریِ بیزی و یادگیریِ MAP، پیشینِ فرضیهٔ P(hi) نقشِ مهمی بازی می‌کند. در فصل ۱۹ دیدیم بیش‌برازش می‌تواند رخ دهد وقتی فضایِ فرضیه بیش‌ازحد بیانی است، یعنی وقتی فرضیه‌های بسیاری دارد که داده را خوب برازش می‌کنند. روش‌های یادگیریِ بیزی و MAP از پیشین برای جریمه‌کردنِ پیچیدگی استفاده می‌کنند. معمولاً، فرضیه‌های پیچیده‌تر احتمالِ پیشینیِ کمتری دارند—تا حدی به این دلیل که بسیارند. از سوی دیگر، فرضیه‌های پیچیده‌تر ظرفیتِ بیشتری برای برازشِ داده دارند. (در حالتِ حدی، جدولِ جستجو می‌تواند داده را دقیقاً بازتولید کند.) از این رو، پیشینِ فرضیه معاوضه‌ای بینِ پیچیدگیِ فرضیه و درجهٔ برازشِ آن به داده را در بر دارد.

اثرِ این معاوضه را در حالتِ منطقی، جایی که H فقط فرضیه‌های قطعی دارد (مانندِ h1، که می‌گوید هر آب‌نبات گیلاس است)، واضح‌تر می‌بینیم. در آن صورت، P(d|hi) اگر hi سازگار باشد 1 و در غیرِ این صورت 0 است. با نگاه به معادلهٔ (20.1)، می‌بینیم hMAP سپس ساده‌ترین نظریهٔ منطقیِ سازگار با داده خواهد بود. بنابراین، یادگیریِ بیشینهٔ پسین تجسمِ طبیعیِ تیغِ آوکام فراهم می‌کند.

بینشِ دیگری دربارهٔ معاوضهٔ بینِ پیچیدگی و درجهٔ برازش با گرفتنِ لگاریتمِ معادلهٔ (20.1) به‌دست می‌آید. انتخابِ hMAP برای بیشینه‌کردنِ P(d|hi)P(hi) معادلِ کمینه‌کردنِ -log2 P(hi) - log2 P(d|hi) است. با استفاده از ارتباطِ بینِ رمزگذاریِ اطلاعات و احتمال که در بخش ۱۹.۳.۳ معرفی کردیم، می‌بینیم جملهٔ -log2 P(hi) برابرِ شمارِ بیت‌های لازم برای مشخص‌کردنِ فرضیهٔ hi است. علاوه بر این، -log2 P(d|hi) شمارِ بیت‌های اضافیِ لازم برای مشخص‌کردنِ داده، با توجه به فرضیه، است. (برای دیدنِ این، در نظر بگیرید اگر فرضیه داده را دقیقاً پیش‌بینی کند—مانندِ h5 و رشتهٔ آب‌نبات‌های لیمو—بیتی لازم نیست و log2 1 = 0.) از این رو، یادگیریِ MAP فرضیه‌ای را انتخاب می‌کند که حداکثرِ فشرده‌سازیِ داده را فراهم می‌کند. همان تکلیف مستقیم‌تر توسط روشِ یادگیریِ «حداقلِ طولِ توصیف» (Minimum Description Length یا MDL) پرداخته می‌شود. در حالی که یادگیریِ MAP سادگی را با تخصیصِ احتمال‌های بالاتر به فرضیه‌های ساده‌تر بیان می‌کند، MDL آن را مستقیماً با شمردنِ بیت‌ها در رمزگذاریِ دودوییِ فرضیه‌ها و داده بیان می‌کند.

۲۰.۲ یادگیری با دادهٔ کامل

فرایندِ یادگیریِ مدلِ احتمال را با درنظرگرفتنِ حالتِ ساده‌ای شروع می‌کنیم که در آن همهٔ متغیرهای تصادفیِ مربوط در هر مثال مشاهده می‌شوند. این «دادهٔ کامل» (Complete Data) نامیده می‌شود. (اصطلاحِ «دادهٔ کامل» گاهی به معنایِ دیگری نیز استفاده می‌شود: اینکه هیچ مقدارِ ویژگی‌ای از دست نرفته.) بعداً، در بخش ۲۰.۳، حالتِ «متغیرهای پنهان» (Hidden Variables) را در نظر می‌گیریم.

۲۰.۲.۱ یادگیریِ پارامترِ بیشینه‌درستنمایی: مدل‌های گسسته

فرض کنیم از تولیدکنندهٔ تازه‌ای کیسه‌ای آب‌نباتِ لیمو و گیلاس بخریم که نسبت‌های طعمش کاملاً ناشناخته‌اند؛ کسرِ گیلاس می‌تواند هر جایِ بینِ 0 و 1 باشد. در آن صورت، پیوستاری از فرضیه‌ها داریم. پارامتر در این مورد، که θ می‌نامیم، نسبتِ آب‌نبات‌های گیلاس است، و فرضیه hθ است. (نسبتِ آب‌نبات‌های لیمو فقط 1-θ است.) اگر فرض کنیم همهٔ نسبت‌ها از نظرِ پیشینی به‌یکسان محتمل‌اند، رویکردِ بیشینه‌درستنمایی معقول است. اگر وضعیت را با

شبکهٔ بیزی مدل کنیم، فقط به یک متغیرِ تصادفی نیاز داریم، Flavor (طعمِ آب‌نباتِ تصادفیِ انتخاب‌شده از کیسه). مقادیرِ گیلاس و لیمو را دارد، که در آن احتمالِ گیلاس θ است (شکل ۲۰.۲(الف) را ببینید). حالا فرض کنیم N آب‌نبات باز کنیم، که c تایشان گیلاس و ℓ = N - c تایشان لیمو هستند. طبقِ معادلهٔ (20.3)، درستنماییِ این مجموعهٔ دادهٔ خاص:

$$P(d|h\theta) = \prod_{j=1}^{N} P(dj|h\theta) = \theta^c \cdot (1-\theta)^\ell$$

است. فرضیهٔ بیشینه‌درستنمایی با مقدارِ θ که این عبارت را بیشینه می‌کند داده می‌شود. چون تابعِ لگاریتم یکنواست، همان مقدار با بیشینه‌کردنِ «لگاریتمِ درستنمایی» (Log Likelihood) به‌جایش به‌دست می‌آید:

$$L(d|h\theta) = \log P(d|h\theta) = \sum_{j=1}^{N} \log P(dj|h\theta) = c\cdot\log\theta + \ell\cdot\log(1-\theta)$$

(با گرفتنِ لگاریتم، حاصلِ‌ضرب را به جمعی روی داده کاهش می‌دهیم، که معمولاً بیشینه‌کردنش آسان‌تر است.) برای یافتنِ مقدارِ بیشینه‌درستنماییِ L ،θ را نسبت به θ مشتق می‌گیریم و عبارتِ حاصل را صفر می‌گذاریم:

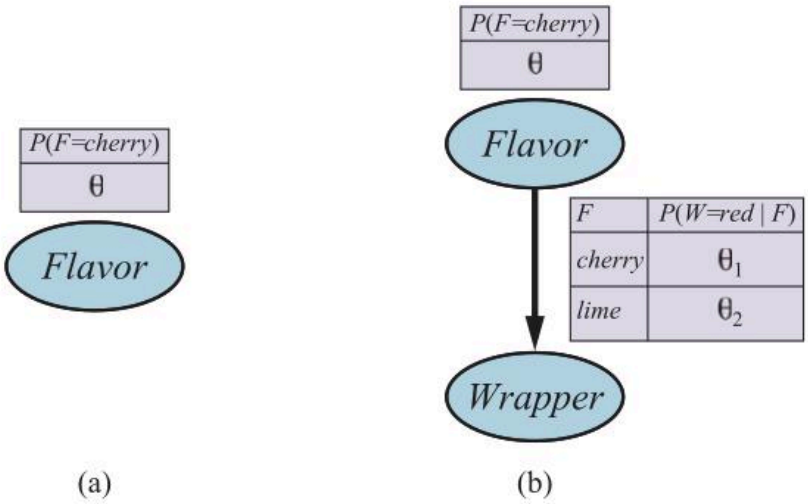
$$dL(d|h\theta)/d\theta = c/\theta - \ell/(1-\theta) = 0 \quad \Rightarrow \quad \theta = c/(c+\ell) = c/N$$

به زبانِ ساده، فرضیهٔ بیشینه‌درستنماییِ hML ادعا می‌کند نسبتِ واقعیِ آب‌نبات‌های گیلاسِ کیسه برابرِ نسبتِ مشاهده‌شده در آب‌نبات‌های بازشدهٔ تاکنون است! به نظر می‌رسد کارِ زیادی کرده‌ایم تا بدیهیات را کشف کنیم. در واقع، با این حال، یک روشِ استاندارد برای یادگیریِ پارامترِ بیشینه‌درستنمایی را شرح داده‌ایم، روشی با کاربردپذیریِ گسترده:

1. عبارتی برای درستنماییِ داده به‌عنوانِ تابعی از پارامتر(ها) بنویس.
2. مشتقِ لگاریتمِ درستنمایی را نسبت به هر پارامتر بنویس.
3. مقادیرِ پارامتر را بیاب که مشتق‌ها در آن‌ها صفرند.

دشوارترین گام معمولاً آخرین است. در مثالِ ما بی‌اهمیت بود، اما خواهیم دید در بسیاری از موارد باید به الگوریتم‌های حلِ تکرارشونده یا فنونِ بهینه‌سازیِ عددیِ دیگر پناه ببریم، چنان‌که در بخش ۴.۲ توصیف شد. (باید تأیید کنیم ماتریسِ هسین معینِ منفی است.) مثال همچنین مسئلهٔ قابلِ‌توجهی از یادگیریِ بیشینه‌درستنمایی را به‌طورِ کلی نشان می‌دهد: وقتی مجموعهٔ داده به‌اندازهٔ کافی کوچک است که برخی رخدادها هنوز مشاهده نشده‌اند—برای نمونه، هیچ آب‌نباتِ گیلاسی—فرضیهٔ بیشینه‌درستنمایی احتمالِ صفر به آن رخدادها تخصیص می‌دهد. حقه‌های گوناگونی برای اجتناب از این مسئله استفاده می‌شود، مانندِ مقداردهیِ اولیهٔ شمارشِ هر رخداد به 1 به‌جای 0.

مثالِ دیگری نگاه کنیم. فرض کنیم این تولیدکنندهٔ تازهٔ آب‌نبات می‌خواهد اشارهٔ کوچکی به مصرف‌کننده بدهد و از لفاف‌های آب‌نباتِ قرمز و سبز استفاده می‌کند. لفافِ هر آب‌نبات به‌صورتِ احتمالاتی، طبقِ توزیعِ شرطیِ ناشناخته‌ای، بسته به طعم انتخاب می‌شود. مدلِ احتمالِ متناظر در شکل ۲۰.۲(ب) نشان داده شده.



شکل ۲۰.۲ – (الف) مدلِ شبکهٔ بیزی برای حالتِ آب‌نبات‌های با نسبتِ ناشناختهٔ گیلاس و لیمو. (ب) مدل برای حالتی که رنگِ لفاف (به‌صورتِ احتمالاتی) به طعمِ آب‌نبات بستگی دارد.

توجه کنید سه پارامتر دارد: θ، θ1 و θ2. با این پارامترها، درستنماییِ دیدنِ، مثلاً، آب‌نباتِ گیلاسی در لفافِ سبز را می‌توان از معناشناسیِ استانداردِ شبکه‌های بیزی (صفحهٔ ۴۱۵) به‌دست آورد:

```
P(Flavor = cherry, Wrapper = green | hθ,θ1,θ2) = P(Flavor = cherry | hθ,θ1,θ2)·P(Wrapper =
                                                  green | Flavor = cherry, hθ,θ1,θ2)
                                θ·(1−θ1) =
```

حالا N آب‌نبات باز می‌کنیم، که c تایشان گیلاس و ℓ تایشان لیمو هستند. شمارشِ لفاف‌ها چنین است: rc از آب‌نبات‌های گیلاس لفافِ قرمز دارند و gc سبز، در حالی که rℓ از آب‌نبات‌های لیمو قرمز و gℓ سبز دارند. درستنماییِ داده:

```
P(d|hθ,θ1,θ2) = θ^c(1−θ)^ℓ · θ1^rc(1−θ1)^gc · θ2^rℓ(1−θ2)^gℓ
```

این کاملاً وحشتناک به نظر می‌رسد، اما گرفتنِ لگاریتم کمک می‌کند:

```
L = [c·log θ + ℓ·log(1−θ)] + [rc·log θ1 + gc·log(1−θ1)] + [rℓ·log θ2 + gℓ·log(1−θ2)]
```

مزیتِ گرفتنِ لگاریتم روشن است: لگاریتمِ درستنمایی جمعِ سه جمله است، که هر یک پارامترِ تکی دارد. وقتی نسبت به هر پارامتر مشتق می‌گیریم و آن‌ها را صفر می‌گذاریم، سه معادلهٔ مستقل می‌گیریم، که هر یک فقط یک پارامتر دارد:

```
                  L/∂θ = c/θ − ℓ/(1−θ) = 0    ⇒    θ = c/(c+ℓ)∂
          L/∂θ1 = rc/θ1 − gc/(1−θ1) = 0    ⇒    θ1 = rc/(rc+gc)∂
          L/∂θ2 = rℓ/θ2 − gℓ/(1−θ2) = 0    ⇒    θ2 = rℓ/(rℓ+gℓ)∂
```

راه‌حلِ θ همان قبلی است. راه‌حلِ θ1، احتمالِ اینکه آب‌نباتِ گیلاسی لفافِ قرمز داشته باشد، کسرِ مشاهده‌شدهٔ آب‌نبات‌های گیلاسیِ با لفافِ قرمز است، و به‌طورِ مشابه برای θ2. این نتایج بسیار دلگرم‌کننده‌اند، و دیدنِ اینکه می‌توان آن‌ها را به هر شبکهٔ بیزی که احتمال‌های شرطی‌اش به‌صورتِ جدول‌ها بازنمایی می‌شوند بسط داد آسان است. مهم‌ترین نکته این است که با دادهٔ کامل، مسئلهٔ یادگیریِ پارامترِ بیشینه‌درستنمایی برای شبکهٔ بیزی به مسائلِ یادگیریِ جداگانه‌ای تجزیه می‌شود، یکی برای هر پارامتر. (تمرین NORX.۲۰ را برای حالتِ غیرجدولی ببینید، جایی که هر پارامتر بر چند احتمالِ شرطی اثر می‌گذارد.) نکتهٔ دوم این است که مقادیرِ پارامتر برای متغیر، با توجه به والدهایش، صرفاً بسامدهای مشاهده‌شدهٔ مقادیرِ متغیر برای هر تنظیمِ مقادیرِ والدند. مانندِ قبل، باید مراقب اجتناب از صفرها وقتی مجموعهٔ داده کوچک است باشیم.

۲۰.۲.۲ مدل‌های بیزِ ساده

احتمالاً رایج‌ترین مدلِ شبکهٔ بیزیِ استفاده‌شده در یادگیریِ ماشینی، مدلِ بیزِ ساده‌ای است که نخستین بار در صفحهٔ ۴۰۲ معرفی شد. در این مدل، متغیرِ «ردهٔ» C (که قرار است پیش‌بینی شود) ریشه و متغیرهای «ویژگیِ» Xi برگ‌ها هستند. مدل «ساده» است، زیرا فرض می‌کند ویژگی‌ها با توجه به رده به‌صورتِ شرطی مستقل‌اند. (مدلِ شکل ۲۰.۲(ب) مدلِ بیزِ ساده با ردهٔ Flavor و فقط یک ویژگی، Wrapper است.) در موردِ متغیرهای بولی، پارامترها = θ P(C=true)، θi1 = P(Xi=true|C=true)، θi2 = P(Xi=true|C=false) هستند.

مقادیرِ پارامترِ بیشینه‌درستنمایی دقیقاً به همان شیوهٔ شکل ۲۰.۲(ب) یافت می‌شوند. پس از آموزشِ مدل بدین‌گونه، می‌توان از آن برای رده‌بندیِ مثال‌های تازه‌ای استفاده کرد که متغیرِ ردهٔ C در آن‌ها مشاهده نشده. با مقادیرِ ویژگیِ مشاهده‌شدهٔ x1,...,xn، احتمالِ هر رده:

```
P(C|x1,...,xn) = α P(C) ∏i P(xi|C)
```

داده می‌شود. پیش‌بینیِ قطعی را می‌توان با انتخابِ محتمل‌ترین رده به‌دست آورد. شکل ۲۰.۳ منحنیِ یادگیریِ این روش را وقتی روی مسئلهٔ رستورانِ فصل ۱۹ اعمال شود نشان می‌دهد.

شکل ۲۰.۳ – منحنیِ یادگیریِ یادگیریِ بیزِ ساده روی مسئلهٔ رستورانِ فصل ۱۹؛ منحنیِ یادگیریِ یادگیریِ درختِ تصمیم برای مقایسه نشان داده شده.

این روش نسبتاً خوب یاد می‌گیرد اما نه به‌خوبی یادگیریِ درختِ تصمیم؛ احتمالاً به این دلیل که فرضیهٔ واقعی—که درختِ تصمیم است—با مدلِ بیزِ ساده دقیقاً قابلِ بازنمایی نیست. معلوم می‌شود یادگیریِ بیزِ ساده در طیفِ وسیعی از کاربردها به‌طرزِ شگفت‌آوری خوب عمل می‌کند؛ نسخهٔ تقویت‌شده‌اش (تمرین BNBX.۲۰) یکی از مؤثرترین الگوریتم‌های یادگیریِ همه‌منظوره است. یادگیریِ بیزِ ساده به‌خوبی به مسائلِ بسیار بزرگ مقیاس می‌شود: با n ویژگیِ بولی، فقط 2n+1 پارامتر هست، و برای یافتنِ hML، فرضیهٔ بیزِ سادهٔ بیشینه‌درستنمایی، جستجویی لازم نیست. سرانجام، سامانه‌های یادگیریِ بیزِ ساده با دادهٔ نویزی یا مفقود خوب کنار می‌آیند و می‌توانند وقتی مناسب است پیش‌بینی‌های احتمالاتی بدهند. عیبِ اصلی‌شان این واقعیت است که فرضِ استقلالِ شرطی به‌ندرت دقیق است؛ چنان‌که در صفحهٔ ۴۰۳ ذکر شد، فرض به احتمال‌های بیش‌ازحدِ مطمئنی می‌انجامد که اغلب بسیار نزدیکِ 0 یا 1 هستند، به‌ویژه با شمارِ زیادِ ویژگی‌ها.

۲۰.۲.۳ مدل‌های مولد و تفکیکی

می‌توانیم دو نوع مدلِ یادگیریِ ماشینیِ استفاده‌شده برای رده‌بندها را متمایز کنیم: مولد و تفکیکی. «مدلِ مولد» (Generative Model) توزیعِ احتمالِ هر رده را مدل می‌کند. برای نمونه، رده‌بندِ متنِ بیزِ ساده از بخش ۱۲.۶.۱ برای هر ردهٔ ممکنِ متن مدلِ جداگانه‌ای می‌سازد—یکی برای ورزش، یکی برای آب‌وهوا، و همین‌طور. هر مدل شاملِ احتمالِ پیشینیِ رده—برای نمونه P(Category=weather)—و نیز احتمالِ شرطیِ P(Inputs|Category=weather) است. از این‌ها می‌توانیم احتمالِ مشترکِ P(Inputs, Category=weather) را محاسبه کنیم و می‌توانیم انتخابِ تصادفیِ واژه‌هایی تولید کنیم که نمایندهٔ متن‌های ردهٔ آب‌وهواست.

«مدلِ تفکیکی» (Discriminative Model) مستقیماً مرزِ تصمیم بینِ رده‌ها را یاد می‌گیرد. یعنی P(Category|Inputs) را یاد می‌گیرد. با ورودی‌های مثال، مدلِ تفکیکی به ردهٔ خروجی می‌رسد، اما نمی‌توانی از مدلِ تفکیکی برای، مثلاً، تولیدِ واژه‌های تصادفیِ نمایندهٔ رده استفاده کنی. رگرسیونِ لجستیک، درخت‌های تصمیم و ماشین‌های بردارِ پشتیبان همگی مدل‌های تفکیکی‌اند.

چون مدل‌های تفکیکی همهٔ تأکیدشان را بر تعریفِ مرزِ تصمیم می‌گذارند—یعنی در واقع انجامِ تکلیفِ رده‌بندیِ خواسته‌شده—آن‌ها در حدِّ نهایی، با مقدارِ دلخواهی دادهٔ آموزشی، تمایل به عملکردِ بهتر دارند. با این حال، با دادهٔ محدود، در برخی موارد مدلِ مولد بهتر عمل می‌کند. نگ و جوردن (2002) رده‌بندِ مولدِ بیزِ ساده را با رده‌بندِ تفکیکیِ رگرسیونِ لجستیک روی ۱۵ مجموعهٔ دادهٔ (کوچک) مقایسه می‌کنند و درمی‌یابند با حداکثرِ مقدارِ داده، مدلِ تفکیکی در ۹ از ۱۵ مجموعهٔ داده بهتر عمل می‌کند، اما با فقط مقدارِ کمی داده، مدلِ مولد در ۱۴ از ۱۵ مجموعهٔ داده بهتر عمل می‌کند.

۲۰.۲.۴ یادگیریِ پارامترِ بیشینه‌درستنمایی: مدل‌های پیوسته

مدل‌های احتمالِ پیوسته مانندِ مدلِ خطی–گاوسی در صفحهٔ ۴۲۲ نشان داده شدند. چون متغیرهای پیوسته در کاربردهای دنیایِ واقعی فراگیرند، مهم است بدانیم چگونه پارامترهای مدل‌های پیوسته را از داده یاد بگیریم.

اصولِ یادگیریِ بیشینه‌درستنمایی در حالت‌های پیوسته و گسسته یکسان‌اند. با حالتِ بسیار ساده‌ای شروع کنیم: یادگیریِ پارامترهای تابعِ چگالیِ گاوسی روی متغیرِ تکی. یعنی فرض می‌کنیم داده چنین تولید شده:

$$P(x) = (1/(\sigma\sqrt{2\pi}))\ e^{-(x-\mu)^2/(2\sigma^2)}$$

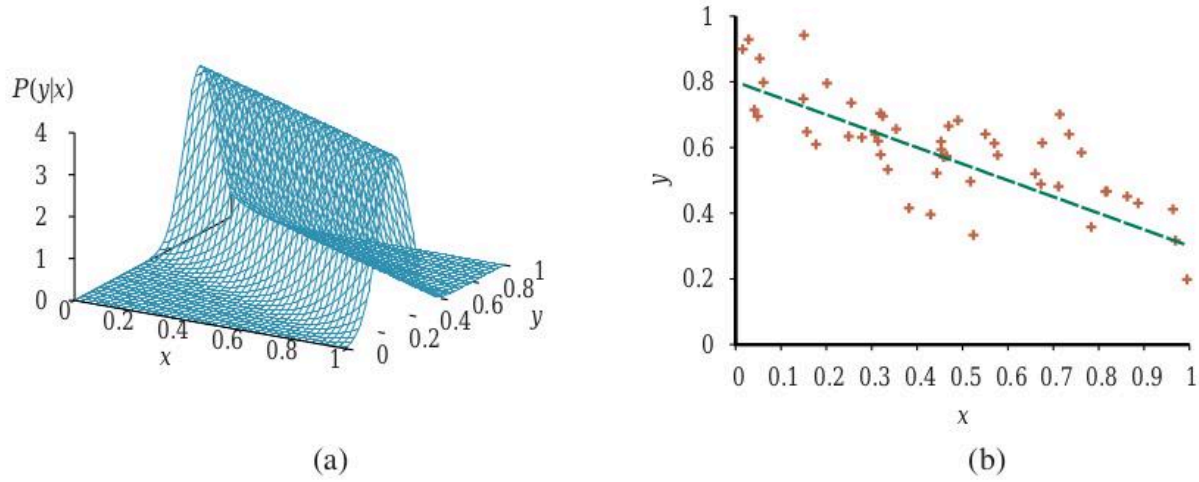
پارامترهای این مدل میانگینِ μ و انحرافِ معیارِ σ هستند. (توجه کنید «ثابتِ» نرمال‌سازی به σ بستگی دارد، پس نمی‌توانیم آن را نادیده بگیریم.) مقادیرِ مشاهده‌شده $x_1,\ldots,x_N$ باشند. سپس لگاریتمِ درستنمایی:

$$L = \sum_{j=1}^{N} \log[(1/(\sigma\sqrt{2\pi}))\ e^{-(x_j-\mu)^2/(2\sigma^2)}]$$
$$= N(-\log\sqrt{(2\pi)} - \log\sigma) - \sum_{j=1}^{N} (x_j-\mu)^2/(2\sigma^2)$$

با صفرگذاشتنِ مشتق‌ها مثلِ همیشه، به‌دست می‌آوریم:

$$\partial L/\partial\mu = -(1/\sigma^2) \sum_{j=1}^{N}(x_j-\mu) = 0 \quad \Rightarrow \quad \mu = (\sum_j x_j)/N$$
$$\partial L/\partial\sigma = -N/\sigma + (1/\sigma^3) \sum_{j=1}^{N}(x_j-\mu)^2 = 0 \quad \Rightarrow \quad \sigma = \sqrt{(\sum_j (x_j-\mu)^2/N)} \qquad (20.4)$$

یعنی، مقدارِ بیشینه‌درستنماییِ میانگین، میانگینِ نمونه‌ای و مقدارِ بیشینه‌درستنماییِ انحرافِ معیار، جذرِ واریانسِ نمونه‌ای است. باز، این‌ها نتایجِ دلگرم‌کننده‌ای‌اند که رویهٔ «عقلِ سلیم» را تأیید می‌کنند.



شکل ۲۰.۴ – (الف) مدلِ خطی–گاوسی توصیف‌شده به‌صورتِ y = θ1x + θ2 به‌علاوهٔ نویزِ گاوسی با واریانسِ ثابت. (ب) مجموعه‌ای از ۵۰ نقطهٔ داده تولیدشده از این مدل و خطِ بهترین‌برازش.

حالا مدلِ خطی–گاوسی با والدِ پیوستهٔ X و فرزندِ پیوستهٔ Y را در نظر بگیر. چنان‌که در صفحهٔ ۴۲۲ توضیح داده شد، Y توزیعِ گاوسی دارد که میانگینش به‌صورتِ خطی به مقدارِ X بستگی دارد و انحرافِ معیارش ثابت است. برای یادگیریِ توزیعِ شرطیِ P(Y|X)، می‌توانیم درستنماییِ شرطی را بیشینه کنیم:

$$P(y|x) = (1/(\sigma\sqrt{2\pi}))\ e^{-(y-(\theta_1 x+\theta_2))^2/(2\sigma^2)} \qquad (20.5)$$

در این‌جا، پارامترها θ2 ،θ1 و σ هستند. داده مجموعه‌ای از جفت‌های (xj,yj) است، چنان‌که در شکل ۲۰.۴ نشان داده شده. با روش‌های معمول (تمرین LINR.۲۰)، می‌توانیم مقادیرِ بیشینه‌درستنماییِ پارامترها را بیابیم. نکتهٔ این‌جا متفاوت است. اگر فقط پارامترهای θ1 و θ2 را که رابطهٔ خطیِ بینِ x و y را تعریف می‌کنند در نظر بگیریم، روشن می‌شود بیشینه‌کردنِ لگاریتمِ درستنمایی نسبت به این پارامترها همان کمینه‌کردنِ صورتِ $(y - (\theta_1 x + \theta_2))^2$ در توانِ معادلهٔ (20.5) است. این زیانِ L2 است، خطایِ مربعِ بینِ مقدارِ واقعیِ y و پیش‌بینیِ θ1x + θ2.

این همان کمیتی است که رویهٔ استانداردِ رگرسیونِ خطیِ توصیف‌شده در بخش ۱۹.۶ کمینه می‌کند. حالا می‌توانیم بفهمیم چرا: کمینه‌کردنِ جمعِ خطاهای مربع، مدلِ خطِ راستِ بیشینه‌درستنمایی را می‌دهد، مشروط بر اینکه داده با نویزِ گاوسیِ واریانسِ ثابت تولید شده باشد.

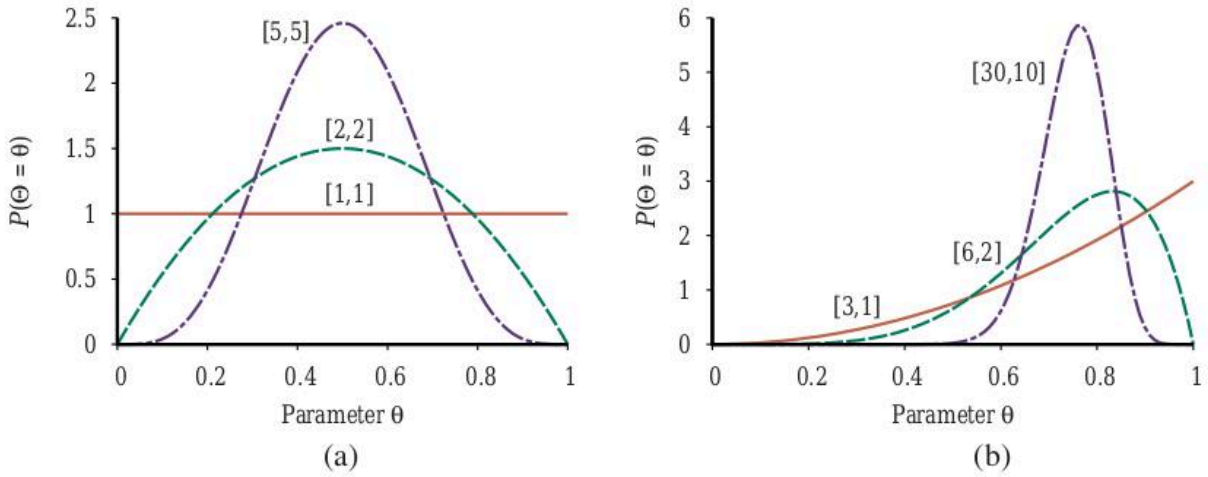
۲۰.۲.۵ یادگیریِ پارامترِ بیزی

یادگیریِ بیشینه‌درستنمایی به رویه‌های ساده می‌انجامد، اما با مجموعه‌های دادهٔ کوچک کاستی‌های جدی دارد. برای نمونه، پس از دیدنِ یک آب‌نباتِ گیلاس، فرضیهٔ بیشینه‌درستنمایی این است که کیسه ۱۰۰٪ گیلاس است (یعنی θ = 1.0). مگر اینکه پیشینِ فرضیهٔ فرد این باشد که کیسه‌ها باید یا تمام‌گیلاس باشند یا تمام‌لیمو، این نتیجهٔ معقولی نیست. محتمل‌تر است کیسه مخلوطی از لیمو و گیلاس باشد. رویکردِ بیزی به یادگیریِ پارامتر با پیشینِ فرضیه شروع می‌کند و توزیع را با رسیدنِ داده به‌روزرسانی می‌کند.

مثالِ آب‌نباتِ شکل ۲۰.۲(الف) یک پارامتر دارد، θ: احتمالِ اینکه تکهٔ تصادفیِ انتخاب‌شدهٔ آب‌نبات طعمِ گیلاس داشته باشد. در دیدگاهِ بیزی، θ مقدارِ (ناشناختهٔ) متغیرِ تصادفیِ Θ است که فضایِ فرضیه را تعریف می‌کند؛ پیشینِ فرضیه توزیعِ پیشینیِ P(Θ) است. بدین‌ترتیب، P(Θ=θ) احتمالِ پیشینیِ اینکه کیسه کسرِ θ آب‌نباتِ گیلاس داشته باشد است. اگر پارامترِ θ بتواند هر مقدارِ بینِ 0 و 1 باشد، P(Θ) تابعِ چگالیِ احتمالِ پیوسته است (بخش الف.۳ را ببینید). اگر چیزی دربارهٔ مقادیرِ ممکنِ θ ندانیم، می‌توانیم از تابعِ چگالیِ یکنواختِ P(θ) = Uniform(θ;0,1) استفاده کنیم، که می‌گوید همهٔ مقادیر به‌یکسان محتمل‌اند. خانوادهٔ انعطاف‌پذیرتری از توابعِ چگالیِ احتمال به‌عنوانِ «توزیع‌های بتا» (Beta Distributions) شناخته می‌شود. هر توزیعِ بتا با دو «ابرپارامترِ» a(Hyperparameters)[3] و b تعریف می‌شود به‌گونه‌ای که:

$$\text{Beta}(\theta; a, b) = \alpha\, \theta^{(a-1)}\, (1-\theta)^{(b-1)} \qquad (20.6)$$

برای θ در بازهٔ [0,1]. ثابتِ نرمال‌سازیِ α، که توزیع را به انتگرالِ 1 می‌رساند، به a و b بستگی دارد. شکل ۲۰.۵ نشان می‌دهد توزیع برای مقادیرِ مختلفِ a و b چگونه به نظر می‌رسد. مقدارِ میانگینِ توزیعِ بتا a/(a+b) است، پس مقادیرِ بزرگ‌ترِ a باورِ نزدیک‌تربودنِ Θ به 1 تا به 0 را پیشنهاد می‌کنند. مقادیرِ بزرگ‌ترِ a+b توزیع را قله‌دارتر می‌کنند، که قطعیتِ بیشتر دربارهٔ مقدارِ Θ را پیشنهاد می‌کند. معلوم می‌شود تابعِ چگالیِ یکنواختِ همان Beta(1,1) است: میانگین 1/2 است و توزیع تخت.



شکل ۲۰.۵ – نمونه‌هایی از توزیعِ Beta(a,b) برای مقادیرِ مختلفِ (a,b). (الف) [1,1]، [2,2] و [5,5]. (ب) [3,1]، [6,2] و [30,10].

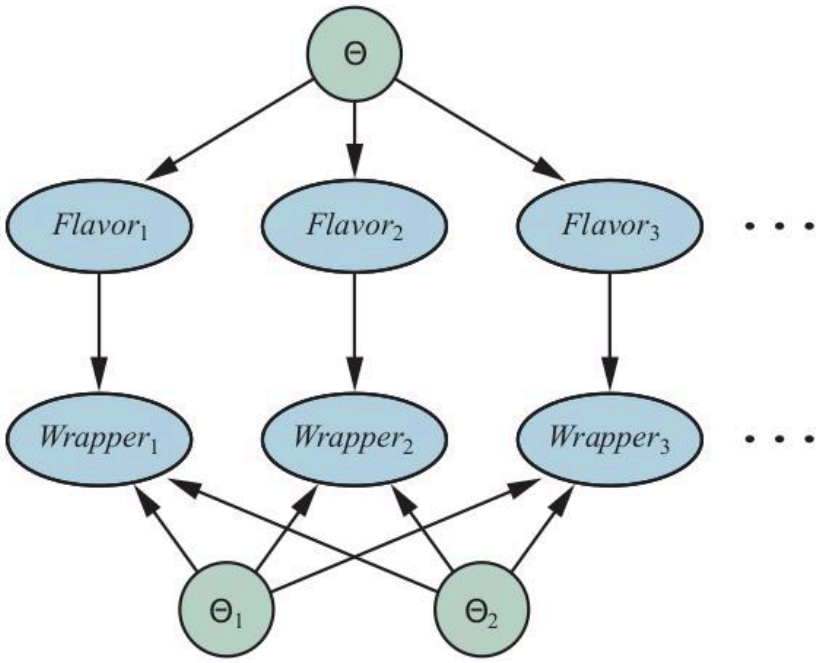
نکتهٔ مهمِ بعدی این است که توزیعِ بتا «همسازِ» (Conjugate) با توزیعِ برنولی است. به‌طورِ خاص، با پیشینِ Beta(a,b) و مشاهدهٔ c آب‌نباتِ گیلاس و ℓ آب‌نباتِ لیمو، احتمالِ پسینی برای Θ به‌صورتِ Beta(a+c, b+ℓ) است. این نتیجه‌ای حیاتی است: توزیعِ بتا خانوادهٔ بسته‌ای تحتِ به‌روزرسانیِ بیزی است. به‌علاوه، به‌صورتِ شهودی، می‌توانیم آن را چنین ببینیم: با مشاهدهٔ آب‌نباتِ گیلاس، پارامترِ a را افزایش می‌دهیم؛ به‌طورِ مشابه، پس از دیدنِ آب‌نباتِ لیمو، پارامترِ b را افزایش می‌دهیم. بدین‌ترتیب، می‌توانیم ابرپارامترهای a و b را «شمارشِ مجازی» (Virtual Counts) ببینیم، به این معنا که پیشینِ Beta(a,b) دقیقاً چنان رفتار می‌کند که گویی با پیشینِ یکنواختِ Beta(1,1) شروع کرده و a-1

آب‌نباتِ گیلاسِ واقعی و b-1 آب‌نباتِ لیموی واقعی دیده‌ایم. با بررسیِ توالیِ توزیع‌های بتا برای مقادیرِ افزایشیِ a و b، با ثابت‌نگه‌داشتنِ نسبت‌ها، می‌توانیم به‌وضوح ببینیم توزیعِ پسینی روی پارامترِ Θ با رسیدنِ داده چگونه تغییر می‌کند. برای نمونه، فرض کنیم کیسهٔ واقعیِ آب‌نبات ۷۵٪ گیلاس باشد. شکل ۲۰.۵(ب) توالیِ Beta(6,2)، Beta(3,1)، Beta(30,10) را نشان می‌دهد. بدیهی است، توزیع به قله‌ای باریک دورِ مقدارِ واقعیِ Θ همگرا می‌شود. برای مجموعه‌های دادهٔ بزرگ، پس، یادگیریِ بیزی (دستِ‌کم در این مورد) به همان پاسخِ یادگیریِ بیشینه‌درستنمایی همگرا می‌شود.

حالا حالتِ پیچیده‌تری در نظر بگیریم. شبکهٔ شکل ۲۰.۲(ب) سه پارامتر دارد، θ، θ1 و θ2، که در آن θ1 احتمالِ لفافِ قرمز روی آب‌نباتِ گیلاس و θ2 احتمالِ لفافِ قرمز روی آب‌نباتِ لیموست.[۴] پیشینِ فرضیهٔ بیزی باید همهٔ سه پارامتر را پوشش دهد—یعنی باید P(Θ,Θ1,Θ2) را مشخص کنیم. معمولاً، «استقلالِ پارامتر» (Parameter Independence) را فرض می‌کنیم:

```
P(Θ,Θ1,Θ2) = P(Θ)P(Θ1)P(Θ2)
```

با این فرض، هر پارامتر می‌تواند توزیعِ بتای خودش را داشته باشد که با رسیدنِ داده جداگانه به‌روزرسانی می‌شود. شکل ۲۰.۶ نشان می‌دهد چگونه می‌توانیم پیشینِ فرضیه و هر داده‌ای را در شبکهٔ بیزی بگنجانیم، که در آن برای هر متغیرِ پارامتر گره‌ای داریم.



شکل ۲۰.۶ — شبکهٔ بیزیِ متناظر با فرایندِ یادگیریِ بیزی. توزیع‌های پسینی برای متغیرهای پارامترِ Θ1 ،Θ و Θ2 را می‌توان با استنتاج در این شبکه به‌دست آورد. (گره‌ها: Θ2 ،Θ1 ،Θ بدونِ والد؛ سپس Flavor_i و Wrapper_i برای هر مشاهدهٔ i.)

گره‌های Θ2 ،Θ1 ،Θ والد ندارند. برای i-امین مشاهدهٔ لفاف و طعمِ متناظرِ تکهٔ آب‌نبات، گره‌های Wrapperi و Flavori را می‌افزاییم. Flavori به پارامترِ طعمِ Θ وابسته است: P(Flavori=cherry|Θ=θ) = θ. Wrapperi به Θ1 و Θ2 وابسته است:

```
P(Wrapperi = red | Flavori = cherry, θ1 = θ1) = θ1
P(Wrapperi = red | Flavori = lime, θ2 = θ2) = θ2
```

حالا، کلِ فرایندِ یادگیریِ بیزی برای شبکهٔ بیزِ اصلیِ شکل ۲۰.۲(ب) را می‌توان به‌عنوانِ مسئلهٔ استنتاجی در شبکهٔ بیزِ مشتق‌شدهٔ نشان‌داده‌شده در شکل ۲۰.۶ صورت‌بندی کرد، که در آن داده و پارامترها گره می‌شوند. پس از افزودنِ همهٔ گره‌های شاهدِ تازه، می‌توانیم متغیرهای پارامتر را (در این مورد، Θ2، Θ1، Θ) پرس‌وجو کنیم. تحتِ این صورت‌بندی فقط یک الگوریتمِ یادگیری هست—الگوریتمِ استنتاج برای شبکه‌های بیزی.

البته، ماهیتِ این شبکه‌ها تا حدی با شبکه‌های فصل ۱۳ متفاوت است، به دلیلِ شمارِ بالقوهٔ عظیمِ متغیرهای شاهدِ بازنمایی‌کنندهٔ مجموعهٔ آموزشی و شیوعِ متغیرهای پارامترِ پیوسته‌مقدار. استنتاجِ دقیق ممکن است به‌جز در مواردِ بسیار ساده مانندِ مدلِ بیزِ ساده ناممکن باشد. دست‌اندرکاران معمولاً از روش‌های استنتاجِ تقریبی مانندِ MCMC استفاده می‌کنند (بخش ۱۳.۴.۲)؛ بسیاری از بسته‌های نرم‌افزاریِ آماری پیاده‌سازی‌های کارآمدی از MCMC را برای این منظور در بر می‌گیرند.

۲۰.۲.۶ رگرسیونِ خطیِ بیزی

در این‌جا نشان می‌دهیم چگونه رویکردِ بیزی را روی تکلیفِ آماریِ استانداردی اعمال کنیم: رگرسیونِ خطی. رویکردِ متعارف در بخش ۱۹.۶ به‌عنوانِ کمینه‌کردنِ جمعِ خطاهای مربع توصیف شد و در بخش ۲۰.۲.۴ به‌عنوانِ بیشینه‌کردنِ درستنمایی با فرضِ مدلِ خطایِ گاوسی بازتفسیر شد. این‌ها یک فرضیهٔ بهترینِ تکی تولید می‌کنند: خطِ راستی با مقادیرِ خاصِ شیب و عرض از مبدأ و واریانسِ ثابتِ خطایِ پیش‌بینی در هر نقطهٔ معین. هیچ معیاری از اینکه چقدر باید به مقادیرِ شیب و عرض از مبدأ مطمئن بود نیست.

علاوه بر این، اگر برای نقطهٔ دادهٔ دیده‌نشده‌ای دور از نقاطِ دادهٔ مشاهده‌شده مقداری پیش‌بینی کنیم، به نظر نمی‌رسد معنا داشته باشد خطایِ پیش‌بینی‌ای فرض کنیم که همان خطایِ پیش‌بینیِ نقطهٔ داده‌ای درستِ کنارِ نقطهٔ دادهٔ مشاهده‌شده است. معقول‌تر به نظر می‌رسد خطایِ پیش‌بینی بزرگ‌تر باشد، هرچه نقطه از دادهٔ مشاهده‌شده دورتر باشد، زیرا تغییرِ کوچک در شیب باعث تغییرِ بزرگ در مقدارِ پیش‌بینی‌شده برای نقطهٔ دور می‌شود. رویکردِ بیزی هر دو این مسائل را اصلاح می‌کند. ایدهٔ کلی، مانندِ بخشِ قبلی، گذاشتنِ پیشینی روی پارامترهای مدل—در این‌جا، ضرایبِ مدلِ خطی و واریانسِ نویز—و سپس محاسبهٔ پسینِ پارامتر با توجه به داده است. برای دادهٔ چندمتغیره و مدلِ نویزِ ناشناخته، این به جبرِ خطیِ نسبتاً زیادی می‌انجامد، پس روی حالتِ ساده تمرکز می‌کنیم: دادهٔ تک‌متغیره، مدلی که مقید به عبور از مبدأ است، و نویزِ معلوم: توزیعِ نرمال با واریانسِ σ^2. سپس فقط یک پارامترِ θ داریم و مدل:

$$P(y|x,\theta) = N(y;\ \theta x,\ \sigma^2) = (1/(\sigma\sqrt{2\pi}))\ e^{-(1/2)((y-\theta x)^2/\sigma^2)} \qquad (20.7)$$

است. چون لگاریتمِ درستنمایی در θ درجه‌دومی است، شکلِ مناسب برای پیشینِ همساز روی θ نیز گاوسی است. این تضمین می‌کند پسینِ θ نیز گاوسی باشد. میانگینِ θ0 و واریانسِ $\sigma0^2$ را برای پیشین فرض می‌کنیم، پس:

$$P(\theta) = N(\theta;\ \theta 0,\ \sigma 0^2) = (1/(\sigma 0\sqrt{2\pi}))\ e^{-(1/2)((\theta-\theta 0)^2/\sigma 0^2)} \qquad (20.8)$$

بسته به داده‌ای که مدل می‌شود، ممکن است ایده‌ای از چه نوع شیبی θ انتظار داریم داشته باشیم، یا ممکن است کاملاً بی‌نظر باشیم. در حالتِ اخیر، منطقی است θ0 را 0 و σ0² را بزرگ انتخاب کنیم—به اصطلاح «پیشینِ بی‌اطلاع‌کننده» (Uninformative Prior). سرانجام، می‌توانیم پیشینِ P(x) را برای مقدارِ x هر نقطهٔ داده فرض کنیم، اما این کاملاً برای تحلیل بی‌اثر است، زیرا به θ بستگی ندارد.

حالا چیدمان کامل است، پس می‌توانیم پسینِ θ را با معادلهٔ (20.1) محاسبه کنیم:

$$P(\theta|d) \propto P(d|\theta)P(\theta)$$

نقاطِ دادهٔ مشاهده‌شده $d = (x1,y1),\ldots,(xN,yN)$ هستند، پس درستنماییِ داده از معادلهٔ (20.7) چنین به‌دست می‌آید:

$$P(d|\theta) = (\prod i\ P(xi))\ (\prod i\ P(yi|xi,\theta)) = \alpha \prod i\ e^{-(1/2)((yi-\theta xi)^2/\sigma^2)} = \alpha\ e^{-(1/2)\sum i((yi-\theta xi)^2/\sigma^2)}$$

که در آن پیشین‌های مقدارِ x و ثابت‌های نرمال‌سازیِ N گاوسی را در ثابتِ αای که مستقل از θ است جذب کرده‌ایم. حالا این و پیشینِ پارامترِ معادلهٔ (20.8) را ترکیب می‌کنیم تا پسین به‌دست آید:

$$P(\theta|d) = \alpha''\ e^{-(1/2)((\theta-\theta 0)^2/\sigma 0^2)}\ e^{-(1/2)\sum i((yi-\theta xi)^2/\sigma^2)}$$

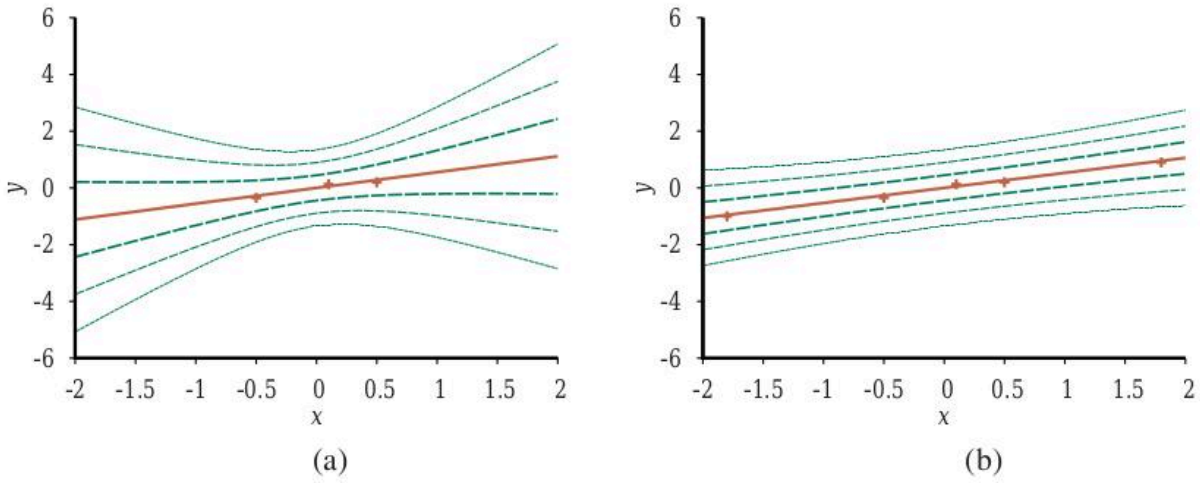
اگرچه این پیچیده به نظر می‌رسد، هر توان تابعِ درجه‌دومی از θ است، پس جمعِ دو توان نیز چنین است. از این رو، کلِ عبارت توزیعِ گاوسی برای θ را بازنمایی می‌کند. با دستکاری‌های جبریِ بسیار مشابهِ آن‌های بخش ۱۴.۴، درمی‌یابیم:

$$P(\theta|d) = \alpha'''\ e^{-(1/2)((\theta-\theta N)^2/\sigma N^2)}$$

با میانگین و واریانسِ «به‌روزشده» که چنین داده می‌شوند:

$$\theta N = (\sigma^2\theta 0 + \sigma 0^2\sum i\ xiyi)\ /\ (\sigma^2 + \sigma 0^2\sum i\ xi^2) \qquad \text{و} \qquad \sigma N^2 = (\sigma^2\sigma 0^2)\ /\ (\sigma^2 + \sigma 0^2\sum i\ xi^2)$$

به این فرمول‌ها نگاه کنیم تا ببینیم چه معنا می‌دهند. وقتی داده به‌صورتِ باریک روی ناحیهٔ کوچکی از محورِ x نزدیکِ مبدأ متمرکز است، $\sum i\ xi^2$ کوچک خواهد بود و واریانسِ پسینیِ σN^2 بزرگ، تقریباً برابرِ واریانسِ پیشینیِ $\sigma 0^2$. این همان‌گونه است که انتظار داریم: داده کارِ کمی برای مقیدکردنِ چرخشِ خط دورِ مبدأ می‌کند. برعکس، وقتی داده به‌صورتِ گسترده در طولِ محور پخش شده، $\sum i\ xi^2$ بزرگ خواهد بود و واریانسِ پسینیِ σN^2 کوچک، تقریباً برابرِ $\sigma^2/(\sum i\ xi^2)$، پس شیب بسیار محکم مقید خواهد شد.



شکل ۲۰.۷ – رگرسیونِ خطیِ بیزی با مدلی مقید به عبور از مبدأ و واریانسِ نویزِ ثابتِ $\sigma^2 = 0.2$. کانتورهای ±1، ±2 و ±3 انحرافِ معیار برای چگالیِ پیش‌بینی نشان داده شده‌اند. (الف) با سه نقطهٔ دادهٔ نزدیکِ مبدأ، شیب کاملاً نامطمئن است، با $\sigma_N^2 \approx 0.3861$. توجه کنید عدم قطعیت با فاصله از نقاطِ دادهٔ مشاهده‌شده چگونه افزایش می‌یابد. (ب) با دو نقطهٔ دادهٔ اضافیِ دورتر، شیبِ θ بسیار محکم مقید شده، با $\sigma_N^2 \approx 0.0286$. واریانسِ باقی‌مانده در چگالیِ پیش‌بینی تقریباً کاملاً به نویزِ ثابتِ σ^2 مربوط است.

برای پیش‌بینی در نقطهٔ دادهٔ خاص، باید روی مقادیرِ ممکنِ θ انتگرال بگیریم، چنان‌که معادلهٔ (20.2) پیشنهاد می‌کند:

$$P(y|x,d) = \int P(y|x,d,\theta)P(\theta|x,d)d\theta = \int P(y|x,\theta)P(\theta|d)d\theta = \alpha \int e^{-\frac{1}{2}\frac{(y-\theta x)^2}{\sigma^2}} e^{-\frac{1}{2}\frac{(\theta-\theta_N)^2}{\sigma_N^2}} d\theta$$

باز، جمعِ دو توان تابعِ درجه‌دومی از θ است، پس گاوسی روی θ داریم که انتگرالش 1 است. جمله‌های باقی‌مانده در y گاوسیِ دیگری تشکیل می‌دهند:

$$P(y|x,d) \propto e^{-\frac{1}{2}\frac{(y-\theta_N x)^2}{(\sigma^2+\sigma_N^2 x^2)}}$$

با نگاه به این عبارت، می‌بینیم میانگینِ پیش‌بینیِ y برابر $\theta_N x$ است، یعنی بر میانگینِ پسینیِ θ مبتنی است. واریانسِ پیش‌بینی با نویزِ مدلِ σ^2 به‌علاوهٔ جمله‌ای متناسب با x^2 داده می‌شود، که بدان معناست انحرافِ معیارِ پیش‌بینی به‌صورتِ مجانبی خطی با فاصله از مبدأ افزایش می‌یابد. شکل ۲۰.۷ این پدیده را نشان می‌دهد. چنان‌که در آغازِ این بخش ذکر شد، داشتنِ عدم قطعیتِ بیشتر برای پیش‌بینی‌هایی که از نقاطِ دادهٔ مشاهده‌شده دورترند کاملاً معنا دارد.

۲۰.۲.۷ یادگیریِ ساختارِ شبکه‌های بیز

تاکنون فرض کرده‌ایم ساختارِ شبکهٔ بیز داده شده و فقط سعی در یادگیریِ پارامترها داریم. ساختارِ شبکه دانشِ علّیِ پایه‌ای دربارهٔ حوزه را بازنمایی می‌کند که اغلب برای کارشناس، یا حتی کاربرِ ساده‌لوح، آسان است فراهم کند. در برخی موارد، با این حال، مدلِ علّی ممکن است در دسترس نباشد یا موضوعِ اختلاف باشد—برای نمونه، برخی شرکت‌ها مدت‌ها ادعا کرده‌اند سیگار باعث سرطان نمی‌شود و شرکت‌های دیگر اظهار می‌کنند غلظت‌های CO_2 هیچ اثری بر اقلیم ندارند—پس مهم است بفهمیم چگونه ساختارِ شبکهٔ بیز را می‌توان از داده یاد گرفت. این بخش طرحِ مختصری از

ایده‌های اصلی می‌دهد. بدیهی‌ترین رویکرد جستجو برای مدلِ خوب است. می‌توانیم با مدلی بدونِ پیوند شروع کنیم و افزودنِ والدها برای هر گره را آغاز کنیم، پارامترها را با روش‌هایی که پوشش دادیم برازش کنیم و دقتِ مدلِ حاصل را اندازه بگیریم. به‌طورِ جایگزین، می‌توانیم با حدسِ اولیه‌ای از ساختار شروع کنیم و از جستجوی تپه‌نوردی یا بازپختِ شبیه‌سازی‌شده برای ایجادِ اصلاح‌ها استفاده کنیم، و پس از هر تغییرِ ساختار پارامترها را دوباره تنظیم کنیم. اصلاح‌ها می‌توانند شاملِ معکوس‌کردن، افزودن یا حذفِ پیوندها باشند. در این فرایند نباید دور وارد کنیم، پس بسیاری از الگوریتم‌ها فرض می‌کنند ترتیبی برای متغیرها داده شده و گره فقط می‌تواند والدهایی در میانِ گره‌هایی داشته باشد که در ترتیب زودتر می‌آیند (درست مانندِ فرایندِ ساخت در فصل ۱۳). برای عمومیتِ کامل، باید روی ترتیب‌های ممکن نیز جستجو کنیم.

دو روشِ جایگزین برای تصمیم‌گیری دربارهٔ اینکه چه زمانی ساختارِ خوبی یافت شده هست. نخستین، آزمودنِ این است که آیا اظهارهای استقلالِ شرطیِ ضمنی در ساختار واقعاً در داده برآورده می‌شوند. برای نمونه، استفاده از مدلِ بیزِ ساده برای مسئلهٔ رستوران فرض می‌کند $P(Hungry, Bar|WillWait) = P(Hungry|WillWait)P(Bar|WillWait)$، و می‌توانیم در داده بررسی کنیم آیا همان معادله بینِ بسامدهای شرطیِ متناظر برقرار است. اما حتی اگر ساختار ماهیتِ علّیِ واقعیِ حوزه را توصیف کند، نوسان‌های آماریِ مجموعهٔ داده بدان معناست معادله هرگز دقیقاً برقرار نخواهد شد، پس باید آزمونِ آماریِ مناسبی انجام دهیم تا ببینیم شواهدِ کافی برای نقضِ فرضیهٔ استقلال هست. پیچیدگیِ شبکهٔ حاصل به آستانه‌ای که برای این آزمون استفاده می‌شود بستگی خواهد داشت—هرچه آزمونِ استقلال سخت‌گیرانه‌تر باشد، پیوندهای بیشتری افزوده می‌شوند و خطرِ بیش‌برازش بیشتر است.

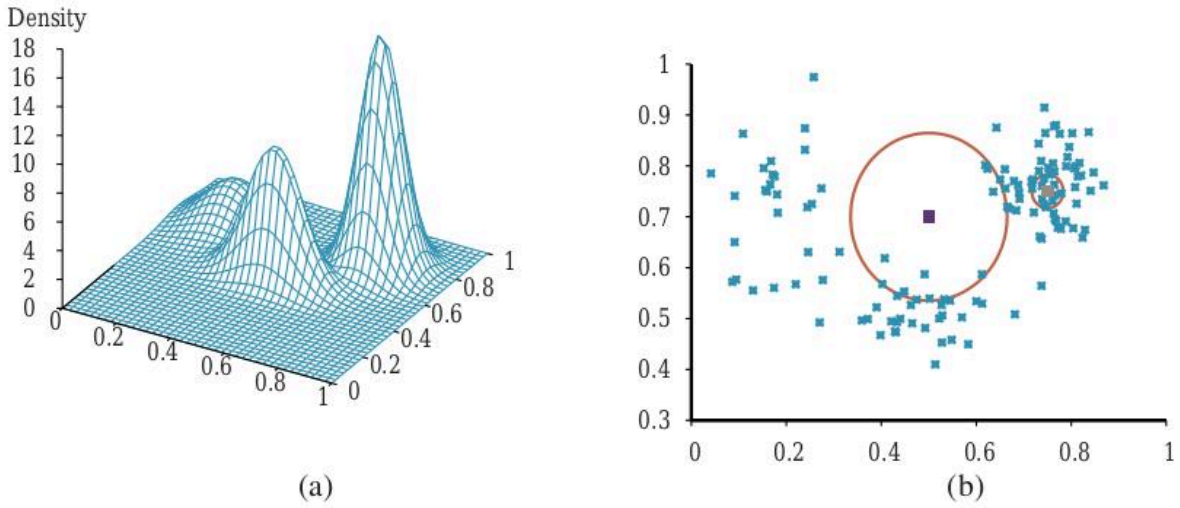
رویکردی سازگارتر با ایده‌های این فصل، ارزیابیِ درجه‌ای است که مدلِ پیشنهادی داده را تبیین می‌کند (به معنایِ احتمالاتی). باید مراقب باشیم این را چگونه اندازه بگیریم. اگر فقط سعی کنیم فرضیهٔ بیشینه‌درستنمایی را بیابیم، به شبکهٔ کاملاً متصل می‌رسیم، زیرا افزودنِ والدهای بیشتر به گره نمی‌تواند درستنمایی را کاهش دهد (تمرین MLPA.۲۰). به نحوی مجبور به جریمه‌کردنِ پیچیدگیِ مدل هستیم. رویکردِ MAP (یا MDL) صرفاً جریمه‌ای از درستنماییِ هر ساختار (پس از تنظیمِ پارامتر) کم می‌کند، قبل از مقایسهٔ ساختارهای متفاوت. رویکردِ بیزی پیشینِ مشترکی روی ساختارها و پارامترها می‌گذارد. معمولاً ساختارهای بسیار زیادی برای جمع‌زدن هست (فوق‌نمایی در شمارِ متغیرها)، پس بیشترِ دست‌اندرکاران از MCMC برای نمونه‌برداری روی ساختارها استفاده می‌کنند.

جریمه‌کردنِ پیچیدگی (خواه با روش‌های MAP یا بیزی) ارتباطِ مهمی بینِ ساختارِ بهینه و ماهیتِ بازنماییِ توزیع‌های شرطیِ شبکه معرفی می‌کند. با توزیع‌های جدولی، جریمهٔ پیچیدگیِ توزیعِ گره با شمارِ والدها به‌صورتِ نمایی رشد می‌کند، اما با، مثلاً، توزیع‌های نویزی-OR، فقط خطی رشد می‌کند. این بدان معناست یادگیری با مدل‌های نویزی-OR (یا دیگر مدل‌های فشرده‌پارامتر) تمایل به تولیدِ ساختارهای یادگرفته‌شده با والدهای بیشتر از یادگیری با توزیع‌های جدولی دارد.

۲۰.۲.۸ برآوردِ چگالی با مدل‌های ناپارامتری

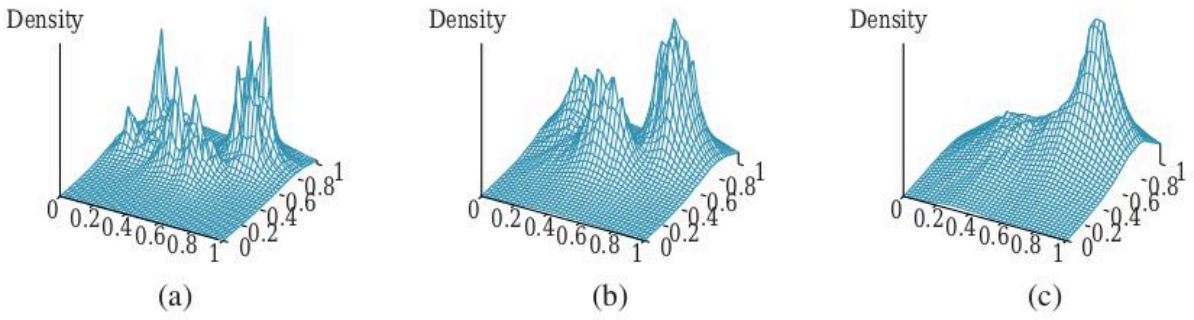
ممکن است بتوان مدلِ احتمالی را بدونِ هیچ فرضی دربارهٔ ساختار و پارامترسازی‌اش با اتخاذِ روش‌های ناپارامتریِ بخش ۱۹.۷ یاد گرفت. تکلیفِ «برآوردِ چگالیِ ناپارامتری» (Nonparametric Density Estimation) معمولاً در حوزه‌های پیوسته انجام می‌شود، مانندِ آنچه در شکل ۲۰.۸(الف) نشان داده شده. شکل، تابعِ چگالیِ احتمالی را روی فضایی

نشان می‌دهد که با دو متغیرِ پیوسته تعریف شده. در شکل ۲۰.۸(ب) نمونه‌ای از نقاطِ داده از این تابعِ چگالی می‌بینیم. پرسش این است: آیا می‌توانیم مدل را از نمونه‌ها بازیابی کنیم؟



شکل ۲۰.۸ – (الف) نمودارِ سه‌بعدیِ مخلوطِ گاوسی‌ها از شکل ۲۰.۱۲(الف). (ب) نمونهٔ ۱۲۸-نقطه‌ای از نقاطِ مخلوط، همراه با دو نقطهٔ پرس‌وجو (مربع‌های کوچکِ نارنجی) و ۱۰-همسایگی‌هایشان (دایرهٔ بزرگ و دایرهٔ کوچک‌تر در راست).

نخست مدل‌های k-نزدیک‌ترین-همسایه را در نظر می‌گیریم. (در فصل ۱۹ مدل‌های نزدیک‌ترین-همسایه را برای رده‌بندی و رگرسیون دیدیم؛ این‌جا آن‌ها را برای برآوردِ چگالی می‌بینیم.) با نمونه‌ای از نقاطِ داده، برای برآوردِ چگالیِ احتمالِ ناشناخته در نقطهٔ پرس‌وجوی x می‌توانیم به‌سادگی چگالیِ نقاطِ داده را در همسایگیِ x اندازه بگیریم. شکل ۲۰.۸(ب) دو نقطهٔ پرس‌وجو (مربع‌های کوچک) را نشان می‌دهد. برای هر نقطهٔ پرس‌وجو کوچک‌ترین دایره‌ای را رسم کرده‌ایم که ۱۰ همسایه را در بر می‌گیرد—10-نزدیک‌ترین-همسایگی. می‌بینیم دایرهٔ مرکزی بزرگ است، بدان معنا که چگالیِ کمی آن‌جاست، و دایرهٔ سمتِ راست کوچک است، بدان معنا که چگالیِ زیادی آن‌جاست. در شکل ۲۰.۹ سه نمودار از برآوردِ چگالی با k-نزدیک‌ترین-همسایه‌ها، برای مقادیرِ مختلفِ k، نشان می‌دهیم. روشن به نظر می‌رسد (ب) تقریباً درست است، در حالی که (الف) بیش‌ازحد تیغی است (k بیش‌ازحد کوچک است) و (ج) بیش‌ازحد نرم است (k بیش‌ازحد بزرگ است).

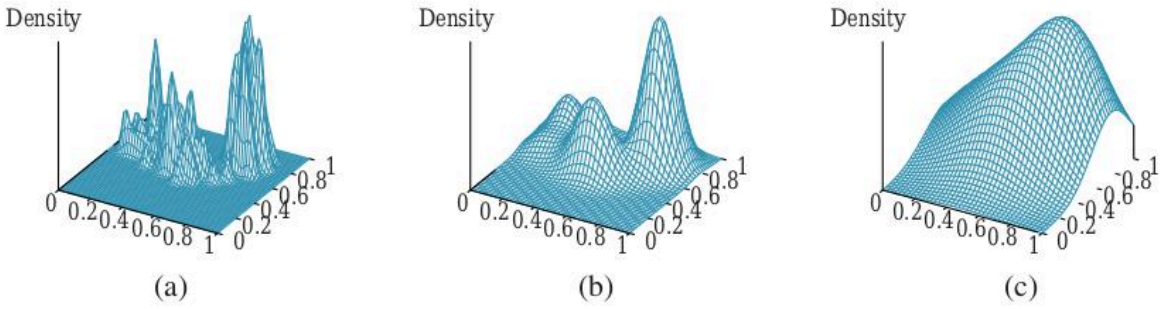


شکل ۲۰.۹ – برآوردِ چگالی با k-نزدیک‌ترین-همسایه‌ها، اعمال‌شده روی دادهٔ شکل ۲۰.۸(ب)، به‌ترتیب برای k=3، 10 و k=3. 40 بیش‌ازحد تیغی، 40 بیش‌ازحد نرم، و 10 تقریباً درست است. بهترین مقدارِ k را می‌توان با اعتبارسنجیِ متقاطع انتخاب کرد.

امکانِ دیگر استفاده از توابعِ هسته است، چنان‌که برای رگرسیونِ وزن‌محلی انجام دادیم. برای اعمالِ مدلِ هسته بر برآوردِ چگالی، فرض کنید هر نقطهٔ داده تابعِ چگالیِ کوچکِ خودش را تولید می‌کند. برای نمونه، ممکن است از گاوسی‌های کروی با انحرافِ معیارِ w در طولِ هر محور استفاده کنیم. سپس چگالیِ برآوردشده در نقطهٔ پرس‌وجوی x میانگینِ هسته‌های داده است:

$$P(x) = (1/N) \sum_{j=1}^{N} K(x, x_j) \quad \text{که در آن} \quad K(x, x_j) = (1/(w\sqrt{2\pi}))^d e^{-D(x,x_j)^2/(2w^2)}$$

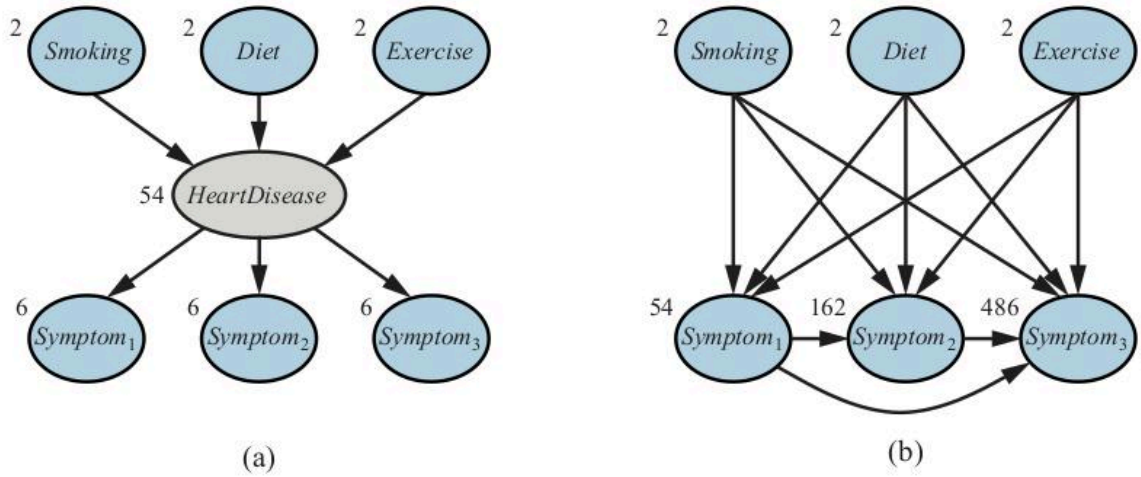
که در آن d شمارِ ابعادِ x و D تابعِ فاصلهٔ اقلیدسی است. همچنان مسئلهٔ انتخابِ مقدارِ مناسبِ پهنایِ هستهٔ w را داریم؛ شکل ۲۰.۱۰ مقادیری را نشان می‌دهد که بیش‌ازحد کوچک، درست و بیش‌ازحد بزرگ‌اند. مقدارِ خوبِ w را می‌توان با اعتبارسنجیِ متقاطع انتخاب کرد.



شکل ۲۰.۱۰ — برآوردِ چگالی با هسته‌ها برای دادهٔ شکل ۲۰.۸(ب)، با هسته‌های گاوسی با w=0.02، 0.07 و 0.20 به‌ترتیب. w=0.07 تقریباً درست است.

۲۰.۳ یادگیری با متغیرهای پنهان: الگوریتمِ EM

بخشِ قبلی با حالتِ کاملاً قابلِ مشاهده سروکار داشت. بسیاری از مسائلِ دنیایِ واقعی «متغیرهای پنهان» (Hidden Variables) دارند (که گاهی «متغیرهای نهان» (Latent Variables) نیز نامیده می‌شوند)، که در داده قابلِ مشاهده نیستند. برای نمونه، سوابقِ پزشکی اغلب شاملِ نشانه‌های مشاهده‌شده، تشخیصِ پزشک، درمانِ اعمال‌شده و شاید پیامدِ درمان‌اند، اما به‌ندرت شاملِ مشاهدهٔ مستقیمِ خودِ بیماری‌اند! (توجه کنید تشخیص بیماری نیست؛ پیامدِ علّیِ نشانه‌های مشاهده‌شده است، که به‌نوبهٔ خود توسط بیماری ایجاد شده‌اند.) ممکن است کسی بپرسد، «اگر بیماری مشاهده نشود، آیا می‌توانیم مدلی را فقط بر اساسِ متغیرهای مشاهده‌شده بسازیم؟» پاسخ در شکل ۲۰.۱۱ ظاهر می‌شود، که مدلِ تشخیصیِ کوچک و تخیلیِ بیماریِ قلبی را نشان می‌دهد. سه عاملِ مستعدکنندهٔ قابلِ مشاهده و سه نشانهٔ قابلِ مشاهده (که نام‌بردنشان افسرده‌کننده است) هست. فرض کنید هر متغیر سه مقدارِ ممکن دارد (مثلاً هیچ، متوسط و شدید).

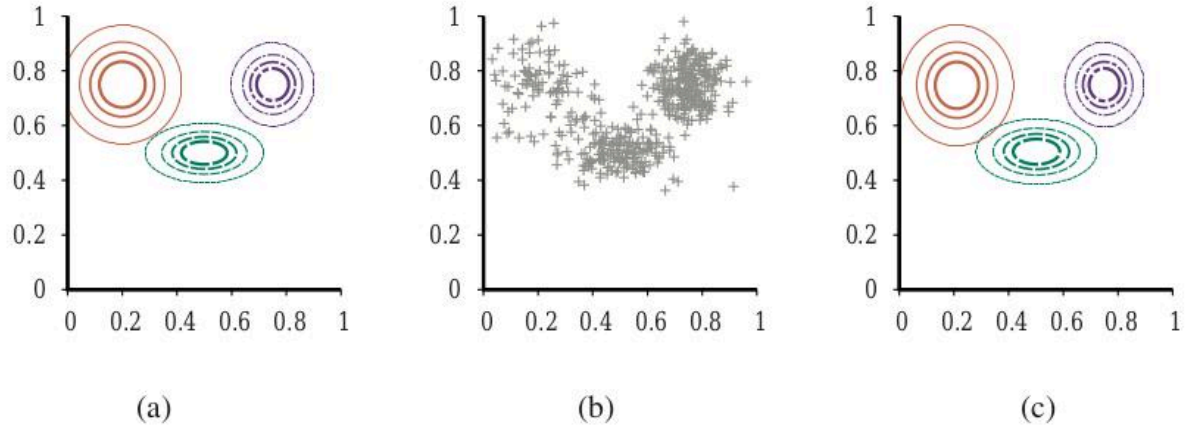


شکل ۲۰.۱۱ – (الف) شبکهٔ تشخیصیِ ساده برای بیماریِ قلبی، که فرض می‌شود متغیرِ پنهان باشد. هر متغیر سه مقدار دارد و با شمارِ پارامترهای مستقلِ توزیعِ شرطی‌اش برچسب‌گذاری شده؛ شمارِ کل 78 است. (ب) شبکهٔ معادل با حذفِ HeartDisease. توجه کنید متغیرهای نشانه دیگر با توجه به والدهایشان به‌صورتِ شرطی مستقل نیستند. این شبکه به 708 پارامتر نیاز دارد.

حذفِ متغیرِ پنهان از شبکهٔ (الف)، شبکهٔ (ب) را می‌دهد، که در آن متغیرهای نشانه دیگر با توجه به والدهایشان به‌صورتِ شرطی مستقل نیستند. این شبکه به 708 پارامتر نیاز دارد، در حالی که شبکهٔ اصلی با متغیرِ پنهان فقط 78 پارامتر لازم دارد. این مثال نشان می‌دهد متغیرهای پنهان می‌توانند بازنماییِ مدل را به‌صورتِ دراماتیک ساده‌کنند. اگر متغیرِ پنهان مشاهده شود یا نشود، این مزیتِ بازنمایی باقی می‌ماند؛ اما اگر مشاهده نشود، یادگیریِ پارامترها دشوارتر می‌شود. این بخش الگوریتمِ «بیشینه‌سازیِ انتظار» (Expectation–Maximization یا EM) را توصیف می‌کند، الگوریتمی عمومی برای یادگیریِ مدل‌های احتمالی با متغیرهای پنهان. EM الگوریتمی تکرارشونده است که دو گام را متناوب می‌کند: گامِ E (انتظار) پسینیِ متغیرهای پنهان را با توجه به مدلِ کنونی و دادهٔ مشاهده‌شده محاسبه می‌کند؛ گامِ M (بیشینه‌سازی) پارامترهای مدل را برای بیشینه‌کردنِ لگاریتمِ درستنماییِ موردانتظارِ داده، با استفاده از پسینی‌های محاسبه‌شده در گامِ E، به‌روزرسانی می‌کند. فرایند تا همگرایی تکرار می‌شود.

۲۰.۳.۱ خوشه‌بندیِ نظارت‌نشده: یادگیریِ مخلوطِ گاوسی‌ها

«خوشه‌بندیِ نظارت‌نشده» (Unsupervised Clustering) مسئلهٔ تشخیصِ چند رده در مجموعه‌ای از اشیاست. مسئله نظارت‌نشده است، زیرا برچسب‌های رده داده نشده‌اند. برای نمونه، فرض کنیم طیفِ صد هزار ستاره را ثبت کنیم؛ آیا انواعِ متفاوتی از ستاره‌ها توسط طیف‌ها آشکار می‌شوند، و اگر آری، چند نوع و ویژگی‌هایشان چیست؟ ما همگی با اصطلاحاتی مانندِ «غولِ سرخ» و «کوتولهٔ سفید» آشنا هستیم، اما ستاره‌ها این برچسب‌ها را روی کلاه‌هایشان ندارند—ستاره‌شناسان باید خوشه‌بندیِ نظارت‌نشده انجام می‌دادند تا این رده‌ها را شناسایی کنند. مثال‌های دیگر شاملِ شناساییِ گونه‌ها، سرده‌ها، راسته‌ها، شاخه‌ها و... در رده‌بندیِ لینه‌ای و ایجادِ انواعِ طبیعی برای اشیای معمولی است (فصل ۱۰ را ببینید). خوشه‌بندیِ نظارت‌نشده با داده شروع می‌شود. شکل ۲۰.۱۲(ب) ۵۰۰ نقطهٔ داده را نشان می‌دهد، که هر یک مقادیرِ دو ویژگیِ پیوسته را مشخص می‌کند. نقاطِ داده ممکن است با ستاره‌ها متناظر باشند، و ویژگی‌ها ممکن است با شدت‌های طیفی در دو بسامدِ خاص متناظر باشند.



شکل ۲۰.۱۲ – (الف) مدلِ مخلوطِ گاوسی با سه مؤلفه؛ وزن‌ها (از چپ به راست) 0.2، 0.3 و 0.5 هستند. (ب) ۵۰۰ نقطهٔ دادهٔ نمونه‌برداری‌شده از مدلِ (الف). (ج) مدلِ بازسازی‌شده توسط EM از دادهٔ (ب).

بعد، باید بفهمیم چه نوع توزیعِ احتمالی ممکن است داده را تولید کرده باشد. خوشه‌بندی فرض می‌کند داده از «توزیعِ مخلوط» (Mixture Distribution) به نامِ P تولید شده. چنین توزیعی k مؤلفه دارد، که هر یک به‌حق خودش توزیعی است. نقطهٔ داده با نخست انتخابِ مؤلفه و سپس تولیدِ نمونه‌ای از آن مؤلفه تولید می‌شود. متغیرِ تصادفیِ C مؤلفه را با مقادیرِ 1,...,k نشان دهد؛ سپس توزیعِ مخلوط چنین داده می‌شود:

```
P(x) = ∑_{i=1}^{k} P(C=i) P(x|C=i)
```

که در آن x به مقادیرِ ویژگی‌های نقطهٔ داده اشاره دارد. برای دادهٔ پیوسته، انتخابِ طبیعی برای توزیع‌های مؤلفه، گاوسیِ چندمتغیره است، که خانوادهٔ به اصطلاح «مخلوطِ گاوسی‌ها» (Mixture of Gaussians) را می‌دهد. پارامترهای مخلوطِ گاوسی‌ها wi = P(C=i) (وزنِ هر مؤلفه)، μi (میانگینِ هر مؤلفه) و Σi (کوواریانسِ هر مؤلفه) هستند. شکل ۲۰.۱۲(الف) مخلوطی از سه گاوسی را نشان می‌دهد؛ این مخلوط در واقع منبعِ دادهٔ (ب) و نیز مدلِ نشان‌داده‌شده در شکل ۲۰.۸(الف) در صفحهٔ ۷۳۶ است. مسئلهٔ خوشه‌بندیِ نظارت‌نشده، پس، بازیابیِ مدلِ مخلوطِ گاوسی مانندِ شکل ۲۰.۱۲(الف) از دادهٔ خام مانندِ شکل ۲۰.۱۲(ب) است. بدیهی است، اگر می‌دانستیم کدام مؤلفه هر نقطهٔ داده را تولید کرده، بازیابیِ گاوسی‌های مؤلفه آسان بود: می‌توانستیم همهٔ نقاطِ داده از مؤلفهٔ معین را انتخاب کنیم و سپس (نسخهٔ چندمتغیره‌ای از) معادلهٔ (20.4) (صفحهٔ ۷۲۹) را برای برازشِ پارامترهای گاوسی به مجموعه‌ای از داده اعمال کنیم. از سوی دیگر، اگر پارامترهای هر مؤلفه را می‌دانستیم، می‌توانستیم، دستِ‌کم در معنایِ احتمالاتی، هر نقطهٔ داده را به مؤلفه‌ای تخصیص دهیم.

مسئله این است که نه تخصیص‌ها را می‌دانیم و نه پارامترها را. ایدهٔ پایهٔ EM در این بافت این است که وانمود کنیم پارامترهای مدل را می‌دانیم و سپس احتمالِ تعلقِ هر نقطهٔ داده به هر مؤلفه را استنتاج کنیم. پس از آن، مؤلفه‌ها را به داده برازش می‌کنیم، جایی که هر مؤلفه به کلِ مجموعهٔ داده برازش می‌شود با هر نقطه که با احتمالِ تعلقش به آن

مؤلفه وزن‌گذاری شده. فرایند تا همگرایی تکرار می‌شود. اساساً، ما در حالِ «تکمیلِ» داده با استنتاجِ توزیع‌های احتمال روی متغیرهای پنهان—هر نقطهٔ داده به کدام مؤلفه تعلق دارد—بر اساسِ مدلِ کنونی‌ایم. برای مخلوطِ گاوسی‌ها، پارامترهای مدلِ مخلوط را خودسرانه مقداردهی می‌کنیم و سپس دو گامِ زیر را تکرار می‌کنیم:

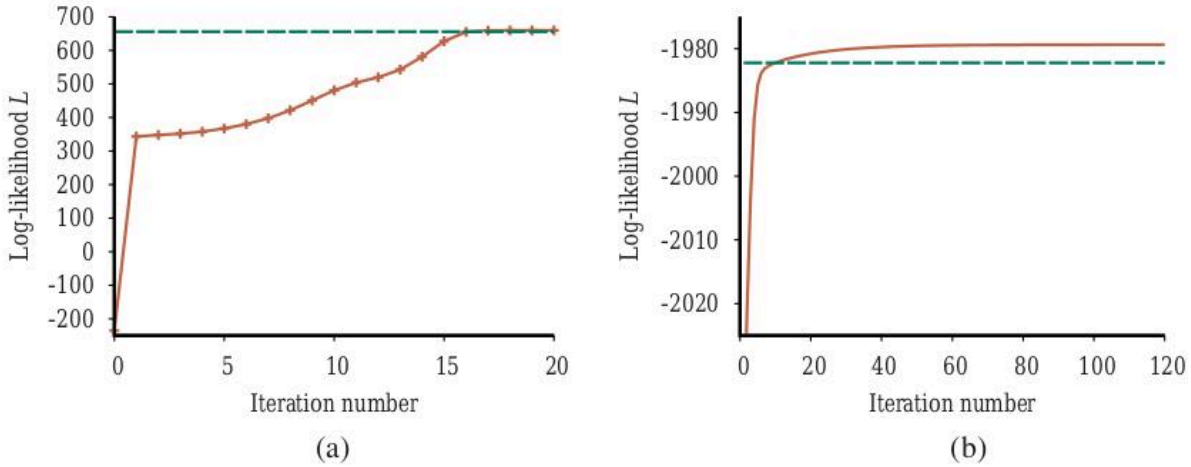
1. **گامِ E:** احتمال‌های $p_{ij} = P(C=i|x_j)$، احتمالِ اینکه دادهٔ x_j توسط مؤلفهٔ i تولید شده، را محاسبه کن. با قاعدهٔ بیز، $p_{ij} = \alpha P(x_j|C=i)P(C=i)$ داریم. جملهٔ $P(x_j|C=i)$ فقط احتمالِ x_j زیرِ گاوسیِ iام است، و جملهٔ $P(C=i)$ فقط پارامترِ وزنِ گاوسیِ iام است. $n_i = \sum_j p_{ij}$ را تعریف کن، شمارِ مؤثرِ نقاطِ داده‌ای که در حالِ حاضر به مؤلفهٔ i تخصیص داده شده‌اند.

1. **گامِ M:** میانگین، کوواریانس و وزن‌های مؤلفهٔ تازه را با گام‌های زیر به‌ترتیب محاسبه کن:

$$\mu_i \leftarrow \sum_j p_{ij} x_j / n_i$$
$$\Sigma_i \leftarrow \sum_j p_{ij} (x_j - \mu_i)(x_j - \mu_i)^T / n_i$$
$$w_i \leftarrow n_i/N$$

که در آن N شمارِ کلِ نقاطِ داده است. گامِ E، یا گامِ انتظار، را می‌توان محاسبهٔ مقادیرِ موردانتظارِ p_{ij} متغیرهای نشانگرِ پنهانِ Z_{ij} دانست، که در آن Z_{ij} برابر 1 است اگر دادهٔ x_j توسط مؤلفهٔ i تولید شده باشد و در غیرِ این صورت 0. گامِ M، یا گامِ بیشینه‌سازی، مقادیرِ تازهٔ پارامترهایی را می‌یابد که لگاریتمِ درستنماییِ داده را، با توجه به مقادیرِ موردانتظارِ متغیرهای نشانگرِ پنهان، بیشینه می‌کنند.

مدلِ نهایی‌ای که EM وقتی روی دادهٔ شکل ۲۰.۱۲(الف) اعمال می‌شود یاد می‌گیرد در شکل ۲۰.۱۲(ج) نشان داده شده؛ عملاً از مدلِ اصلی‌ای که داده از آن تولید شده (خطِ افقی) غیرقابلِ تمایز است. شکل ۲۰.۱۳(الف) لگاریتمِ درستنماییِ داده را طبقِ مدلِ کنونی با پیشرویِ EM رسم می‌کند.



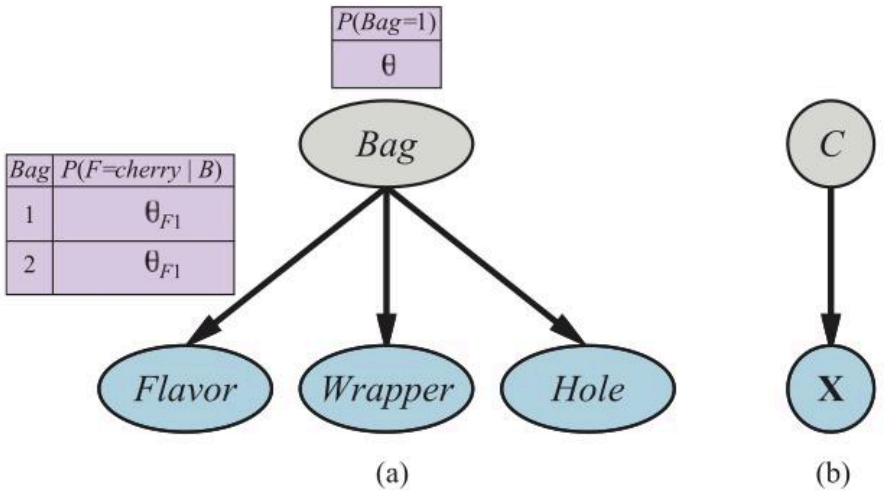
شکل ۲۰.۱۳ – نمودارهایی که لگاریتمِ درستنماییِ داده، L، را به‌عنوانِ تابعی از تکرارِ EM نشان می‌دهند. خطِ افقی لگاریتمِ درستنماییِ طبقِ مدلِ واقعی را نشان می‌دهد. (الف) نمودار برای مدلِ مخلوطِ گاوسیِ شکل ۲۰.۱۲. (ب) نمودار برای شبکهٔ بیزیِ شکل ۲۰.۱۴(الف).

دو نکته برای توجه هست. نخست، لگاریتمِ درستنماییِ مدلِ نهاییِ یادگرفته‌شده کمی از مدلِ اصلی، که داده از آن تولید شد، فراتر می‌رود. این ممکن است شگفت‌آور به نظر برسد، اما صرفاً این واقعیت را بازتاب می‌دهد که داده به‌صورتِ تصادفی تولید شده و ممکن است بازتابِ دقیقی از مدلِ زیربنایی فراهم نکند. نکتهٔ دوم این است که EM در هر تکرار لگاریتمِ درستنماییِ داده را افزایش می‌دهد. این واقعیت را می‌توان به‌طورِ کلی اثبات کرد. علاوه بر این، تحتِ شرایطِ معینی (که در بیشترِ موارد برقرارند)، می‌توان اثبات کرد EM به بیشینهٔ محلیِ درستنمایی می‌رسد. (در مواردِ نادر، می‌تواند به نقطهٔ زینی یا حتی کمینهٔ محلی برسد.) از این نظر، EM شبیهِ الگوریتمِ تپه‌نوردیِ مبتنی بر گرادیان است، اما توجه کنید پارامترِ «اندازهٔ گام» ندارد.

همیشه همه‌چیز به‌خوبیِ شکل ۲۰.۱۳(الف) پیش نمی‌رود. ممکن است، برای نمونه، رخ دهد یکی از مؤلفه‌های گاوسی چنان کوچک شود که فقط یک نقطهٔ داده را بپوشاند. سپس واریانسش به صفر و درستنمایی‌اش به بی‌نهایت می‌رود! اگر ندانیم چند مؤلفه در مخلوط است، باید مقادیرِ مختلفِ k را امتحان کنیم و ببینیم کدام بهتر است؛ آن می‌تواند منبعِ خطا باشد. مسئلهٔ دیگر این است که دو مؤلفه می‌توانند «ادغام» شوند، با میانگین‌ها و واریانس‌های یکسان و به‌اشتراک‌گذاریِ نقاطِ داده‌شان. این نوع بیشینه‌های محلیِ تباهیده مسائلِ جدی‌اند، به‌ویژه در ابعادِ بالا. یکی از راه‌حل‌ها گذاشتنِ پیشینی روی پارامترهای مدل و اعمالِ نسخهٔ MAP_EM است. راهِ دیگر راه‌اندازیِ دوبارهٔ مؤلفه با پارامترهای تصادفیِ تازه اگر بیش‌ازحد کوچک شود یا بیش‌ازحد به مؤلفهٔ دیگر نزدیک شود. مقداردهیِ حساس نیز کمک می‌کند.

۲۰.۳.۲ یادگیریِ مقادیرِ پارامترِ شبکه‌های بیز برای متغیرهای پنهان

برای یادگیریِ شبکهٔ بیزی با متغیرهای پنهان، همان بینش‌هایی را اعمال می‌کنیم که برای مخلوطِ گاوسی‌ها کار کرد. شکل ۲۰.۱۴(الف) وضعیتی را بازنمایی می‌کند که در آن دو کیسهٔ آب‌نبات با هم مخلوط شده‌اند. آب‌نبات‌ها با سه ویژگی توصیف می‌شوند: علاوه بر Flavor و Wrapper، برخی آب‌نبات‌ها سوراخِ (Hole) وسطی دارند و برخی نه.



شکل ۲۰.۱۴ – (الف) مدلِ مخلوط برای آب‌نبات. نسبت‌های طعم‌های مختلف، لفاف‌ها و وجودِ سوراخ به کیسه بستگی دارد، که مشاهده نمی‌شود. (ب) شبکهٔ بیزی برای مخلوطِ گاوسی. میانگین و کوواریانسِ متغیرهای قابلِ مشاهدهٔ X به مؤلفهٔ C بستگی دارد.

توزیعِ آب‌نبات‌های هر کیسه با مدلِ بیزِ ساده توصیف می‌شود: ویژگی‌ها با توجه به کیسه مستقل‌اند، اما توزیعِ احتمالِ شرطیِ هر ویژگی به کیسه بستگی دارد. پارامترها چنین‌اند: θ احتمالِ پیشینیِ اینکه آب‌نباتی از کیسهٔ 1 باشد؛ θF1 و θF2 احتمال‌های اینکه طعم گیلاس باشد، با توجه به اینکه آب‌نبات به‌ترتیب از کیسهٔ 1 یا 2 می‌آید؛ θW1 و θW2 احتمال‌های قرمزبودنِ لفاف را می‌دهند؛ و θH1 و θH2 احتمال‌های سوراخ‌داشتنِ آب‌نبات.

مدلِ کلی مدلِ مخلوط است: جمعِ وزن‌دارِ دو توزیعِ متفاوت، که هر یک حاصلِ‌ضربی از توزیع‌های مستقلِ تک‌متغیره است. (در واقع، می‌توانیم مخلوطِ گاوسی‌ها را نیز به‌عنوانِ شبکهٔ بیزی مدل کنیم، چنان‌که در شکل ۲۰.۱۴(ب) نشان داده شده.) در شکل، کیسه متغیرِ پنهان است، زیرا پس از مخلوط‌شدنِ آب‌نبات‌ها، دیگر نمی‌دانیم هر آب‌نبات از کدام کیسه آمده. در چنین حالتی، آیا می‌توانیم توصیف‌های دو کیسه را با مشاهدهٔ آب‌نبات‌های مخلوط بازیابی کنیم؟ بیایید یک تکرارِ EM را برای این مسئله کار کنیم. نخست، به داده نگاه کنیم. 1000 نمونه از مدلی تولید کردیم که پارامترهای واقعی‌اش چنین‌اند:

$$\theta = 0.5, \quad \theta F1 = \theta W1 = \theta H1 = 0.8, \quad \theta F2 = \theta W2 = \theta H2 = 0.3 \qquad (20.9)$$

یعنی آب‌نبات‌ها به‌یکسان محتمل است از هر کیسه بیایند؛ نخستین عمدتاً گیلاس با لفافِ قرمز و سوراخ است؛ دومین عمدتاً لیمو با لفافِ سبز و بدونِ سوراخ. شمارش‌های هشت نوعِ ممکنِ آب‌نبات چنین‌اند:

	W = قرمز		W = سبز	
	H = 1	H = 0	H = 1	H = 0
F = گیلاس	273	93	104	90
F = لیمو	79	100	94	167

با مقداردهیِ پارامترها شروع می‌کنیم. برای سادگیِ عددی، خودسرانه[5] انتخاب می‌کنیم:

$$\theta^{(0)} = 0.6, \quad \theta F1^{(0)} = \theta W1^{(0)} = \theta H1^{(0)} = 0.6, \quad \theta F2^{(0)} = \theta W2^{(0)} = \theta H2^{(0)} = 0.4 \qquad (20.10)$$

نخست، روی پارامترِ θ کار کنیم. در حالتِ کاملاً قابلِ مشاهده، آن را مستقیماً از شمارش‌های مشاهده‌شدهٔ آب‌نبات‌های کیسه‌های 1 و 2 برآورد می‌کردیم. چون کیسه متغیرِ پنهان است، در عوض شمارشِ موردانتظار را محاسبه می‌کنیم. شمارشِ موردانتظارِ $\hat{N}(Bag=1)$ برابر است با جمعِ احتمالِ این‌که آب‌نبات از کیسهٔ 1 آمده باشد، روی همهٔ آب‌نبات‌ها:

$$\theta^{(1)} = \hat{N}(Bag=1)/N = \sum_{j=1}^{N} P(Bag=1|flavor_j, wrapper_j, holes_j)/N$$

این احتمال‌ها را می‌توان با هر الگوریتمِ استنتاجی برای شبکه‌های بیزی محاسبه کرد. برای مدلِ بیزِ ساده مانندِ مثالِ ما، می‌توانیم استنتاج را «با دست» انجام دهیم، با قاعدهٔ بیز و اعمالِ استقلالِ شرطی:

```
θ⁽¹⁾ = (1/N) ∑_{j=1}^{N} [P(flavor_j|Bag=1)P(wrapper_j|Bag=1)P(holes_j|Bag=1)P(Bag=1)]
[i P(flavor_j|Bag=i)P(wrapper_j|Bag=i)P(holes_j|Bag=i)P(Bag=i)∑] /
```

با اعمالِ این فرمول بر، مثلاً، 273 آب‌نباتِ گیلاسِ لفافِ قرمزِ سوراخ‌دار، سهمی از:

```
θF1⁽⁰⁾θW1⁽⁰⁾θH1⁽⁰⁾θ⁽⁰⁾ + θF2⁽⁰⁾θW2⁽⁰⁾θH2⁽⁰⁾) / (θF1⁽⁰⁾θW1⁽⁰⁾θH1⁽⁰⁾θ⁽⁰⁾) · (1/1000)
                                                    0.22797 ≈ ((1–θ⁽⁰⁾)
```

می‌گیریم. با ادامه با هفت نوعِ دیگرِ آب‌نبات در جدولِ شمارش، $\theta^{(1)} = 0.6124$ به‌دست می‌آوریم.

حالا پارامترهای دیگر، مانندِ θF1، را در نظر بگیریم. در حالتِ کاملاً قابلِ مشاهده، آن را مستقیماً از شمارش‌های مشاهده‌شدهٔ آب‌نبات‌های گیلاس و لیموی کیسهٔ 1 برآورد می‌کردیم. شمارشِ موردانتظارِ آب‌نبات‌های گیلاسِ کیسهٔ 1 چنین داده می‌شود:

```
P(Bag=1|Flavor_j = cherry, wrapper_j, holes_j) {j: Flavor_j = cherry}_∑
```

باز، این احتمال‌ها را می‌توان با هر الگوریتمِ شبکهٔ بیزی محاسبه کرد. با کامل‌کردنِ این فرایند، مقادیرِ تازهٔ همهٔ پارامترها را به‌دست می‌آوریم:

```
،θ⁽¹⁾ = 0.6124،    θF1⁽¹⁾ = 0.6684،    θW1⁽¹⁾ = 0.6483،    θH1⁽¹⁾ = 0.6558
    θF2⁽¹⁾ = 0.3887،    θW2⁽¹⁾ = 0.3817،    θH2⁽¹⁾ = 0.3827        (20.11)
```

لگاریتمِ درستنماییِ داده از حدود -2044 در آغاز به حدود -2021 پس از نخستین تکرار افزایش می‌یابد، چنان‌که در شکل ۲۰.۱۳(ب) نشان داده شده. یعنی، به‌روزرسانی خودِ درستنمایی را با ضریبِ حدودِ $10^{10} \approx e^{23}$ بهبود می‌بخشد. تا تکرارِ دهم، مدلِ یادگرفته‌شده برازشِ بهتری از مدلِ اصلی است (L = -1982.214). پس از آن، پیشرفت بسیار کند می‌شود. این با EM غیرمعمول نیست، و بسیاری از سامانه‌های عملی EM را با الگوریتمِ مبتنی بر گرادیان مانندِ نیوتن–رافسون (فصل ۴ را ببینید) برای فازِ آخرِ یادگیری ترکیب می‌کنند.

درسِ عمومی از این مثال این است که به‌روزرسانی‌های پارامتر برای یادگیریِ شبکهٔ بیزی با متغیرهای پنهان مستقیماً از نتایجِ استنتاج روی هر مثال در دسترس‌اند. علاوه بر این، فقط احتمال‌های پسینیِ محلی برای هر پارامتر لازم است. در این‌جا، «محلی» بدان معناست جدولِ احتمالِ شرطی (CPT) هر متغیرِ Xi را می‌توان از احتمال‌های پسینیِ شاملِ فقط Xi و والدهایش Ui یاد گرفت. با تعریفِ θijk به‌عنوانِ پارامترِ CPT ِP(Xi = xij | Ui = uik)، به‌روزرسانی با شمارش‌های موردانتظارِ نرمال‌شده چنین داده می‌شود:

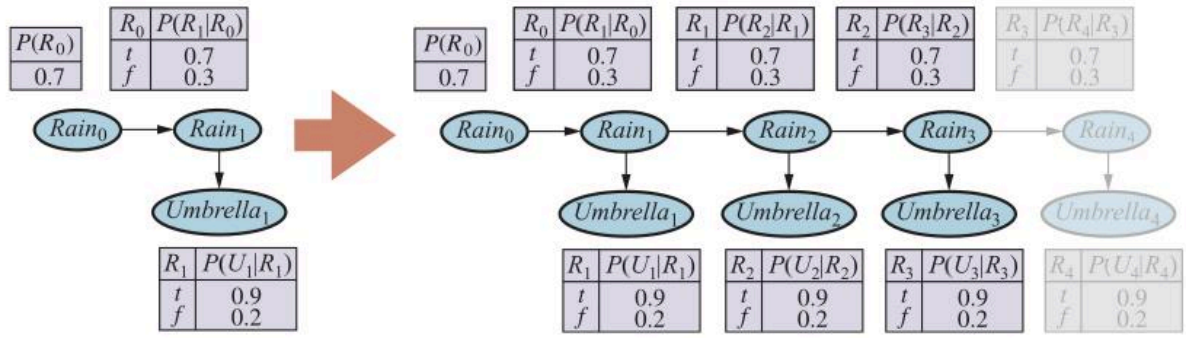
```
θijk ← N̂(Xi = xij, Ui = uik) / N̂(Ui = uik)
```

شمارش‌های موردانتظار با جمع‌زدن روی مثال‌ها، با محاسبهٔ احتمال‌های P(Xi = xij, Ui = uik) برای هر یک با هر الگوریتمِ استنتاجِ شبکهٔ بیزی، به‌دست می‌آیند. برای الگوریتم‌های دقیق—شاملِ حذفِ متغیر—همهٔ این احتمال‌ها مستقیماً به‌عنوانِ محصولِ جانبیِ استنتاجِ استاندارد قابلِ‌حصول‌اند، بدونِ نیاز به محاسبه‌های اضافیِ خاصِ یادگیری. علاوه بر این، اطلاعاتِ لازم برای یادگیری برای هر پارامتر به‌صورتِ محلی در دسترس است.

اگر کمی به عقب بایستیم، می‌توانیم فکر کنیم الگوریتمِ EM در این مثال چه می‌کند: بازیابیِ هفت پارامتر (θ، θF1، θW1، θH1، θF2، θW2، θH2) از هفت ($2^3 - 1$) شمارشِ مشاهده‌شده در داده. (شمارشِ هشتم با این واقعیت که شمارش‌ها به 1000 جمع می‌شوند ثابت است.) اگر هر آب‌نبات با دو ویژگی به‌جای سه تا توصیف می‌شد (مثلاً با حذفِ سوراخ‌ها)، پنج پارامتر (θ، θF1، θW1، θF2، θW2) اما فقط سه ($2^2 - 1$) شمارشِ مشاهده‌شده می‌داشتیم. در چنین حالتی، بازیابیِ وزنِ مخلوطِ θ یا ویژگی‌های دو کیسه‌ای که مخلوط شدند ممکن نیست.

۲۰.۳.۳ یادگیریِ مدل‌های مارکوفِ پنهان

کاربردِ نهاییِ ما از EM شاملِ یادگیریِ احتمال‌های انتقال در مدل‌های مارکوفِ پنهان (HMMها) است. از بخش ۱۴.۳ به یاد آورید مدلِ مارکوفِ پنهان را می‌توان با شبکهٔ بیزِ پویا با متغیرِ حالتِ گسستهٔ تکی بازنمایی کرد، چنان‌که در شکل ۲۰.۱۵ نشان داده شده.



شکل ۲۰.۱۵ — شبکهٔ بیزیِ پویایِ بازشده که مدلِ مارکوفِ پنهانی را بازنمایی می‌کند. (متغیرهای حالتِ Xt و مشاهده‌های Et در برش‌های زمانیِ متوالی.)

هر نقطهٔ داده شاملِ توالیِ مشاهده‌ای با طولِ متناهی است، پس مسئله یادگیریِ احتمال‌های انتقال از مجموعه‌ای از توالی‌های مشاهده (یا از فقط یک توالیِ بلند) است. قبلاً دیده‌ایم چگونه شبکه‌های بیز را یاد بگیریم، اما پیچیدگی‌ای هست: در شبکه‌های بیز، هر پارامتر متمایز است؛ در مدلِ مارکوفِ پنهان، از سوی دیگر، احتمال‌های انتقالِ فردی از حالتِ i به حالتِ j در زمانِ t، $\theta_{ijt} = P(X_{t+1} = j \mid X_t = i)$، در طولِ زمان تکرار می‌شوند—یعنی $\theta_{ijt} = \theta_{ij}$ برای همهٔ t. برای برآوردِ احتمالِ انتقال از حالتِ i به حالتِ j، به‌سادگی نسبتِ موردانتظارِ دفعاتی را محاسبه می‌کنیم که سامانه هنگامِ بودن در حالتِ i به حالتِ j انتقال می‌یابد:

$$\theta_{ij} \leftarrow \sum_t \hat{N}(X_{t+1} = j, X_t = i) \;/\; \sum_t \hat{N}(X_t = i)$$

شمارش‌های موردانتظار با الگوریتمِ استنتاجِ HMM محاسبه می‌شوند. الگوریتمِ پیشرو–عقب‌روِ نشان‌داده‌شده در شکل ۱۴.۴ را می‌توان به‌راحتی برای محاسبهٔ احتمال‌های لازم اصلاح کرد. نکتهٔ مهم این است که احتمال‌های لازم با هموارسازی به‌دست می‌آیند، نه فیلترکردن. فیلترکردن توزیعِ احتمالِ حالتِ کنونی را با توجه به گذشته می‌دهد، اما هموارسازی توزیع را با توجه به همهٔ شواهد، از جملهٔ آنچه بعداً رخ داده، می‌دهد. این نسخهٔ EM برای HMMها «الگوریتمِ باوم–ولچ» (Baum–Welch Algorithm) نامیده می‌شود، به نامِ مخترعانش.

۲۰.۳.۴ شکلِ عمومیِ الگوریتمِ EM

چند نمونه از الگوریتمِ EM دیده‌ایم. هر یک شاملِ محاسبهٔ مقادیرِ موردانتظارِ متغیرهای پنهان برای هر مثال و سپس بازمحاسبهٔ پارامترها، با استفاده از مقادیرِ موردانتظار چنان‌که گویی مقادیرِ مشاهده‌شده‌اند، است. x همهٔ مقادیرِ مشاهده‌شده در همهٔ مثال‌ها، Z همهٔ متغیرهای پنهان برای همهٔ مثال‌ها، و θ همهٔ پارامترهای مدلِ احتمال باشد. سپس الگوریتمِ EM:

$$\theta^{(i+1)} = \text{argmax}_\theta \sum_z P(Z=z|x,\theta^{(i)})\, L(x, Z=z|\theta)$$

این معادله الگوریتمِ EM در یک‌کلام است. گامِ E محاسبهٔ جمع است، که انتظارِ لگاریتمِ درستنماییِ دادهٔ «تکمیل‌شده» نسبت به توزیعِ $P(Z=z|x,\theta^{(i)})$ است، که پسینِ متغیرهای پنهان با توجه به داده است. گامِ M بیشینه‌سازیِ این لگاریتمِ درستنماییِ موردانتظار نسبت به پارامترهاست. برای مخلوطِ گاوسی‌ها، متغیرهای پنهان Zij ها هستند، که در آن Zij برابر 1 است اگر مثالِ j توسط مؤلفهٔ i تولید شده باشد. برای شبکه‌های بیز، Zij مقدارِ متغیرِ مشاهده‌نشدهٔ Xi در مثالِ j است. برای HMMها، Zjt حالتِ توالیِ مثالِ j در زمانِ t است. با شروع از شکلِ عمومی، می‌توان الگوریتمِ EM را برای کاربردِ خاصی اشتقاق کرد، پس از شناساییِ متغیرهای پنهانِ مناسب.

به محض فهمِ ایدهٔ عمومیِ EM، اشتقاقِ انواعِ گوناگون و بهبودها آسان می‌شود. برای نمونه، در بسیاری از موارد گامِ E –محاسبهٔ پسینی‌ها روی متغیرهای پنهان–ناشدنی است، مانندِ شبکه‌های بیزِ بزرگ. معلوم می‌شود می‌توان از گامِ E تقریبی استفاده کرد و همچنان الگوریتمِ یادگیریِ مؤثری به‌دست آورد. با الگوریتمِ نمونه‌برداری مانندِ MCMC (بخش ۱۳.۴ را ببینید)، فرایندِ یادگیری بسیار شهودی است: هر حالت (پیکربندیِ متغیرهای پنهان و مشاهده‌شده) که MCMC از آن بازدید می‌کند دقیقاً چنان رفتار می‌شود که گویی مشاهدهٔ کاملی است. بدین‌ترتیب، پارامترها را می‌توان مستقیماً پس از هر گذارِ MCMC به‌روزرسانی کرد. شکل‌های دیگرِ استنتاجِ تقریبی، مانندِ روش‌های تغییراتی و انتشارِ باورِ حلقه‌دار، نیز برای یادگیریِ شبکه‌های بسیار بزرگ مؤثر ثابت شده‌اند.

۲۰.۳.۵ یادگیریِ ساختارِ شبکه‌های بیز با متغیرهای پنهان

در بخش ۲۰.۲.۷، مسئلهٔ یادگیریِ ساختارِ شبکه‌های بیز با دادهٔ کامل را بحث کردیم. وقتی متغیرهای مشاهده‌نشده بر دادهٔ مشاهده‌شده اثر می‌گذارند، اوضاع دشوارتر می‌شود. در ساده‌ترین حالت، کارشناسِ انسانی ممکن است به الگوریتمِ یادگیری بگوید متغیرهای پنهانِ خاصی وجود دارند، و یافتنِ جایشان در ساختارِ شبکه را به الگوریتم واگذارد. برای نمونه، الگوریتمی ممکن است سعی کند ساختارِ نشان‌داده‌شده در شکل ۲۰.۱۱(الف) در صفحهٔ ۷۳۸ را یاد بگیرد، با

توجه به اطلاعاتِ اینکه HeartDisease (متغیرِ سه‌مقداره) باید در مدل گنجانده شود. مانندِ حالتِ دادهٔ کامل، الگوریتمِ کلی حلقهٔ بیرونی دارد که روی ساختارها جستجو می‌کند و حلقهٔ درونی که پارامترهای شبکه را با توجه به ساختار برازش می‌کند.

اگر به الگوریتمِ یادگیری گفته نشود کدام متغیرهای پنهان وجود دارند، دو انتخاب هست: یا وانمود کنیم داده واقعاً کامل است—که ممکن است الگوریتم را به یادگیریِ مدلی پارامتر-فشرده مانندِ شکل ۲۰.۱۱(ب) وادار کند—یا متغیرهای پنهانِ تازه‌ای برای ساده‌کردنِ مدل اختراع کنیم. رویکردِ اخیر را می‌توان با گنجاندنِ انتخاب‌های اصلاحِ تازه در جستجوی ساختار پیاده‌سازی کرد: علاوه بر اصلاحِ پیوندها، الگوریتم می‌تواند متغیرِ پنهانی را بیفزاید یا حذف کند یا آریتی‌اش را تغییر دهد. البته، الگوریتم نخواهد دانست متغیرِ تازه‌ای که اختراع کرده HeartDisease نامیده می‌شود؛ و برای مقادیرش نام‌های معناداری نخواهد داشت. خوشبختانه، متغیرهای پنهانِ تازه‌اختراع‌شده معمولاً به متغیرهای ازپیش‌موجود متصل خواهند شد، پس کارشناسِ انسانی اغلب می‌تواند توزیع‌های شرطیِ محلیِ شاملِ متغیرِ تازه را بازرسی کند و معنایش را دریابد.

مانندِ حالتِ دادهٔ کامل، یادگیریِ خالصِ ساختارِ بیشینه‌درستنمایی به شبکهٔ کاملاً متصل می‌انجامد (علاوه بر آن، شبکه‌ای بدونِ متغیرهای پنهان)، پس نوعی جریمهٔ پیچیدگی لازم است. همچنین می‌توانیم MCMC را برای نمونه‌برداری از ساختارهای شبکهٔ ممکنِ بسیار اعمال کنیم، و بدین‌ترتیب یادگیریِ بیزی را تقریب بزنیم. برای نمونه، می‌توانیم مخلوطِ گاوسی‌ها را با شمارِ ناشناخته‌ای مؤلفه با نمونه‌برداری روی شمار یاد بگیریم؛ توزیعِ پسینیِ تقریبیِ شمارِ گاوسی‌ها با بسامدهای نمونه‌برداریِ فرایندِ MCMC داده می‌شود.

برای حالتِ دادهٔ کامل، حلقهٔ درونیِ یادگیریِ پارامترها بسیار سریع است—فقط مسئلهٔ استخراجِ بسامدهای شرطی از مجموعهٔ داده. وقتی متغیرهای پنهان هستند، حلقهٔ درونی ممکن است شاملِ تکرارهای بسیاری از EM یا الگوریتمِ مبتنی بر گرادیان باشد، و هر تکرار شاملِ محاسبهٔ پسینی‌ها در شبکهٔ بیز است، که خودش مسئلهٔ NP-سختی است. تا به امروز، این رویکرد برای یادگیریِ مدل‌های پیچیده غیرعملی ثابت شده.

یک بهبودِ ممکن، «الگوریتمِ EM ساختاری» (Structural EM) است، که تقریباً به همان شیوهٔ EM معمولی (پارامتری) عمل می‌کند به‌جز اینکه الگوریتم می‌تواند ساختار را نیز به‌روزرسانی کند، نه فقط پارامترها را. همان‌گونه که EM معمولی از پارامترهای کنونی برای محاسبهٔ شمارش‌های موردانتظار در گامِ E استفاده می‌کند و سپس آن شمارش‌ها را در گامِ M برای انتخابِ پارامترهای تازه اعمال می‌کند، EM ساختاری از ساختارِ کنونی برای محاسبهٔ شمارش‌های موردانتظار استفاده می‌کند و سپس آن شمارش‌ها را در گامِ M برای ارزیابیِ درستنماییِ ساختارهای تازهٔ بالقوه اعمال می‌کند. (این با روشِ حلقهٔ-بیرونی/حلقهٔ-درونی در تضاد است، که شمارش‌های موردانتظارِ تازه‌ای برای هر ساختارِ بالقوه محاسبه می‌کند.) بدین‌گونه، EM ساختاری ممکن است چند اصلاحِ ساختاری به شبکه بدهد بدونِ اینکه یک‌بار هم شمارش‌های موردانتظار را بازمحاسبه کند، و قادر به یادگیریِ ساختارهای شبکهٔ بیزِ غیربدیهی است. EM ساختاری فضایِ جستجویی روی فضایِ ساختارها دارد، نه فضایِ ساختارها و پارامترها. با این حال، کارِ زیادی باقی مانده تا بتوانیم بگوییم مسئلهٔ یادگیریِ ساختار حل شده.

خلاصهٔ فصل

روش‌های یادگیریِ آماری از محاسبهٔ سادهٔ میانگین‌ها تا ساختِ مدل‌های پیچیده مانندِ شبکه‌های بیزی متغیرند. کاربردهایی در سراسرِ علومِ رایانه، مهندسی، زیست‌شناسیِ محاسباتی، علومِ اعصاب، روان‌شناسی و فیزیک دارند. این فصل برخی از ایده‌های پایه را ارائه کرده و طعمی از زیرساخت‌های ریاضی داده است. نکاتِ اصلی:

- روش‌های یادگیریِ بیزی یادگیری را به‌عنوانِ شکلی از استنتاجِ احتمالاتی صورت‌بندی می‌کنند، با استفاده از مشاهده‌ها برای به‌روزرسانیِ توزیعِ پیشینی روی فرضیه‌ها. این رویکرد راهِ خوبی برای پیاده‌سازیِ تیغِ آوکام فراهم می‌کند، اما برای فضاهای فرضیهٔ پیچیده به‌سرعت ناشدنی می‌شود.
- یادگیریِ بیشینهٔ پسین (MAP) فرضیهٔ محتمل‌ترینِ تکی را با توجه به داده انتخاب می‌کند. پیشینِ فرضیه همچنان استفاده می‌شود و روش اغلب از یادگیریِ بیزیِ کامل شدنی‌تر است.
- یادگیریِ بیشینه‌درستنمایی صرفاً فرضیه‌ای را انتخاب می‌کند که درستنماییِ داده را بیشینه می‌کند؛ با پیشینِ یکنواخت معادلِ یادگیریِ MAP است. در مواردِ ساده مانندِ رگرسیونِ خطی و شبکه‌های بیزیِ کاملاً قابلِ مشاهده، راه‌حل‌های بیشینه‌درستنمایی به‌آسانی به‌صورتِ بسته یافت می‌شوند. یادگیریِ بیزِ ساده فنِ خاصاً مؤثری است که خوب مقیاس می‌شود.
- وقتی برخی متغیرها پنهان‌اند، راه‌حل‌های بیشینهٔ درستنماییِ محلی را می‌توان با الگوریتمِ بیشینه‌سازیِ انتظار (EM) یافت. کاربردها شاملِ خوشه‌بندیِ نظارت‌نشده با مخلوطِ گاوسی‌ها، یادگیریِ شبکه‌های بیزی و یادگیریِ مدل‌های مارکوفِ پنهان است.
- یادگیریِ ساختارِ شبکه‌های بیزی نمونه‌ای از انتخابِ مدل است. این معمولاً شاملِ جستجوی گسسته در فضایِ ساختارهاست. روشی برای معاوضهٔ پیچیدگیِ مدل با درجهٔ برازش لازم است.
- مدل‌های ناپارامتری توزیعی را با استفاده از مجموعهٔ نقاطِ داده بازنمایی می‌کنند. بدین‌ترتیب، شمارِ پارامترها با مجموعهٔ آموزشی رشد می‌کند. روش‌های نزدیک‌ترین-همسایه‌ها به مثال‌های نزدیکِ نقطهٔ موردِ نظر نگاه می‌کنند، در حالی که روش‌های هسته ترکیبِ وزن‌دار-فاصله‌ای از همهٔ مثال‌ها تشکیل می‌دهند.

یادگیریِ آماری همچنان حوزهٔ پژوهشیِ بسیار فعالی است. گام‌های عظیمی در هر دو نظریه و عمل برداشته شده، تا حدی که یادگیریِ تقریباً هر مدلی که استنتاجِ دقیق یا تقریبی برایش شدنی است ممکن شده.

یادداشت‌های کتاب‌شناختی و تاریخی

کاربردِ فنونِ یادگیریِ آماری در هوش مصنوعی در سال‌های آغازین حوزهٔ پژوهشیِ فعالی بود (دودا و هارت، 1973 را ببینید) اما با تمرکزِ دومی بر روش‌های نمادین از جریانِ اصلیِ هوش مصنوعی جدا شد. احیایِ علاقه مدتِ کوتاهی پس از معرفیِ مدل‌های شبکهٔ بیزی در اواخرِ دههٔ ۱۹۸۰ رخ داد؛ تقریباً در همان زمان، دیدگاهِ آماریِ یادگیریِ شبکهٔ عصبی

شروع به ظهور کرد. در اواخرِ دهۀ ۱۹۹۰، همگراییِ قابلِ‌توجهی از علاقه‌ها در یادگیریِ ماشینی، آمار و شبکه‌های عصبی، متمرکز بر روش‌هایی برای ایجادِ مدل‌های احتمالاتیِ بزرگ از داده، رخ داد.

مدلِ بیزِ ساده یکی از قدیمی‌ترین و ساده‌ترین شکل‌های شبکۀ بیزی است، که به دهۀ ۱۹۵۰ برمی‌گردد. خاستگاه‌هایش در فصل ۱۲ ذکر شد. موفقیتِ شگفت‌آورش تا حدی توسط دومینگوس و پازانی (1997) تبیین شده. شکلِ تقویت‌شدۀ یادگیریِ بیزِ ساده نخستین مسابقۀ استخراجِ دادۀ جامِ KDD را برد (الکان، 1997). هکرمن (1998) مقدمۀ عالی‌ای بر مسئلۀ عمومیِ یادگیریِ شبکۀ بیز می‌دهد. یادگیریِ پارامترِ بیزی با پیشین‌های دیریکله برای شبکه‌های بیزی توسط اشپیگلهالتر و همکاران (1993) بحث شد. توزیعِ بتا به‌عنوانِ پیشینِ همساز برای متغیرِ برنولی نخستین بار توسط توماس (بیز، 1763) اشتقاق و بعداً توسط کارل پیرسون (1895) به‌عنوانِ مدلی برای دادۀ چوله دوباره معرفی شد؛ سال‌ها به‌عنوانِ «توزیعِ نوعِ I پیرسون» شناخته می‌شد. رگرسیونِ خطیِ بیزی در متنِ باکس و تیائو (1973) بحث شده؛ مینکا (2010) خلاصۀ موجزی از اشتقاق‌ها برای حالتِ عمومیِ چندمتغیره فراهم می‌کند.

چند بستۀ نرم‌افزاری سازوکارهایی برای یادگیریِ آماری با مدل‌های شبکۀ بیز در بر دارند. این‌ها شاملِ BUGS (استنتاجِ بیزی با نمونه‌برداریِ گیبس) (گیلکس و همکاران، 1994؛ لان و همکاران، 2000، 2013)، JAGS (فقط یک نمونه‌بردارِ گیبسِ دیگر) (پلامر، 2003) و STAN (کارپنتر و همکاران، 2017) می‌شوند. نخستین الگوریتم‌های یادگیریِ ساختارِ شبکه‌های بیز از آزمون‌های استقلالِ شرطی استفاده کردند (پرل، 1988؛ پرل و وارما، 1991). اسپیرتس و همکاران (1993) رویکردِ جامعی را در بستۀ TETRAD برای یادگیریِ شبکۀ بیز پیاده‌سازی کردند. بهبودهای الگوریتمی از آن زمان به پیروزیِ آشکارِ روشِ یادگیریِ شبکۀ بیز در مسابقۀ استخراجِ دادۀ جامِ KDD 2001 انجامید (چنگ و همکاران، 2002). (تکلیفِ خاص در این‌جا مسئلۀ بیوانفورماتیکی با 139,351 ویژگی بود!) رویکردِ یادگیریِ ساختار مبتنی بر بیشینه‌کردنِ درستنمایی توسط کوپر و هرسکویتس (1992) توسعه یافت و توسط هکرمن و همکاران (1994) بهبود یافت. الگوریتم‌های جدیدتر به عملکردِ کاملاً قابلِ‌احترامی در حالتِ دادۀ کامل رسیده‌اند (مور و وانگ، 2003؛ تیسیه و کولر، 2005). یکی از مؤلفه‌های مهم، ساختارِ دادۀ کارآمدِ درختِ AD برای کش‌کردنِ شمارش‌ها روی همۀ ترکیب‌های ممکنِ متغیرها و مقادیر است (مور و لی، 1997). فریدمن و گلدشمیت (1996) به اثرِ بازنماییِ توزیع‌های شرطیِ محلی بر ساختارِ یادگرفته‌شده اشاره کردند.

مسئلۀ عمومیِ یادگیریِ مدل‌های احتمال با متغیرهای پنهان و دادۀ مفقود توسط هارتلی (1958) پرداخته شد، که ایدۀ عمومیِ آنچه بعداً EM نامیده شد را توصیف و چند مثال داد. انگیزۀ بیشتر از الگوریتمِ باوم–ولچ برای یادگیریِ HMM آمد (باوم و پتری، 1966)، که حالتِ خاصِ EM است. مقالۀ دمپستر، لیرد و روبین (1977)، که الگوریتمِ EM را به شکلِ عمومی ارائه و همگراییِ‌اش را تحلیل کرد، یکی از پراستنادترین مقاله‌ها در هر دو علومِ رایانه و آمار است. (دمپستر خودش EM را طرح‌واره می‌بیند نه الگوریتم، زیرا ممکن است کارِ ریاضیِ زیادی قبل از اعمالش روی خانوادۀ تازه‌ای از توزیع‌ها لازم باشد.) مک‌لاکلان و کریشنان (1997) کتابی کامل به الگوریتم و ویژگی‌هایش اختصاص می‌دهند. مسئلۀ خاصِ یادگیریِ مدل‌های مخلوط، از جملۀ مخلوطِ گاوسی‌ها، توسط تیترینگتون و همکاران (1985) پوشش داده شده.

درونِ هوش مصنوعی، AUTOCLASS (چیزمن و همکاران، 1988؛ چیزمن و استوتز، 1996) نخستین سامانۀ موفقِ استفاده‌کننده از EM برای مدل‌سازیِ مخلوط بود. AUTOCLASS روی شماری از تکلیف‌های طبقه‌بندیِ علمیِ دنیایِ واقعی اعمال شد، از جملۀ کشفِ انواعِ تازۀ ستاره‌ها از دادۀ طیفی (گوبل و همکاران، 1989) و رده‌های تازۀ پروتئین‌ها و اینترون‌ها در پایگاه‌های دادۀ توالیِ DNA/پروتئین (هانتر و استیتز، 1992).

برای یادگیریِ پارامترِ بیشینه‌درستنمایی در شبکه‌های بیز با متغیرهای پنهان، EM و روش‌های مبتنی بر گرادیان تقریباً در همان زمان توسط لوریتزن (1995) و راسل و همکاران (1995) معرفی شدند. الگوریتمِ EM ساختاری توسط فریدمن (1998) توسعه یافت و برای یادگیریِ بیشینه‌درستنماییِ ساختارهای شبکهٔ بیز با متغیرهای نهان اعمال شد. فریدمن و کولر (2003) یادگیریِ ساختارِ بیزی را توصیف می‌کنند. دالی و همکاران (2011) حوزهٔ یادگیریِ شبکهٔ بیز را مرور می‌کنند و استنادهای گسترده‌ای به ادبیات فراهم می‌کنند.

تواناییِ یادگیریِ ساختارِ شبکه‌های بیزی از نزدیک به مسئلهٔ بازیابیِ اطلاعاتِ علّی از داده مرتبط است. یعنی، آیا ممکن است شبکه‌های بیز را به‌گونه‌ای یاد گرفت که ساختارِ شبکهٔ بازیابی‌شده تأثیرهای علّیِ واقعی را نشان دهد؟ سال‌ها، آماردانان از این پرسش اجتناب می‌کردند، با این باور که دادهٔ مشاهده‌ای (در برابرِ دادهٔ تولیدشده از آزمایش‌های تجربی) فقط می‌تواند اطلاعاتِ همبستگی بدهد—بالاخره، هر دو متغیری که به نظر مرتبط‌اند ممکن است در واقع توسط عاملِ علّیِ سومیِ ناشناخته تحتِ تأثیر قرار گیرند، نه اینکه مستقیماً بر یکدیگر اثر بگذارند. پرل (2000) استدلال‌های قانع‌کننده‌ای برخلافِ آن ارائه کرده، نشان می‌دهد در واقع مواردِ بسیاری هست که علیت را می‌توان تعیین کرد، و صورت‌بندیِ شبکهٔ علّی را برای بیانِ علت‌ها و اثرهای مداخله، و نیز احتمال‌های شرطیِ معمولی، توسعه می‌دهد.

برآوردِ چگالیِ ناپارامتری، که «برآوردِ چگالیِ پنجرهٔ پارزن» (Parzen Window Density Estimation) نیز نامیده می‌شود، در آغاز توسط روزن‌بلت (1956) و پارزن (1962) بررسی شد. از آن زمان، ادبیاتِ عظیمی بررسیِ ویژگی‌های برآوردگرهای گوناگون را توسعه داده. دووروآ (1987) مقدمهٔ کاملی می‌دهد. همچنین ادبیاتِ به‌سرعتِ در حالِ رشدی دربارهٔ روش‌های بیزیِ ناپارامتری هست، که با کارِ بنیادینِ فرگوسن (1973) روی «فرایندِ دیریکله» (Dirichlet Process) آغاز می‌شود، که می‌توان آن را توزیعی روی توزیع‌های دیریکله دانست. این روش‌ها برای مخلوط‌ها با شمارِ ناشناختهٔ مؤلفه‌ها به‌طورِ خاص مفیدند. غرامانی (2005) و جوردن (2005) آموزش‌های مفیدی دربارهٔ کاربردهای بسیاری این ایده‌ها در یادگیریِ آماری فراهم می‌کنند. متنِ راسموسن و ویلیامز (2006) «فرایندِ گاوسی» را پوشش می‌دهد، که راهی برای تعریفِ توزیع‌های پیشینی روی فضایِ توابعِ پیوسته می‌دهد.

مطالبِ این فصل کارِ حوزه‌های آمار و بازشناسیِ الگو را گرد هم می‌آورد، پس داستان بارها و به راه‌های بسیاری گفته شده. متن‌های خوبِ آمارِ بیزی شاملِ دگروت (1970)، برگر (1985) و گلمن و همکاران (1995) می‌شوند. بیشاپ (2007)، هاستی و همکاران (2009)، باربر (2012) و مورفی (2012) مقدمه‌های عالی‌ای بر یادگیریِ آماریِ ماشینی فراهم می‌کنند. برای رده‌بندیِ الگو، متنِ کلاسیکِ سال‌های بسیاری دودا و هارت (1973) بوده، که اکنون به‌روز شده (دودا و همکاران، 2001). کنفرانسِ سالانهٔ NeurIPS (سامانه‌های پردازشِ اطلاعاتِ عصبی، قبلاً NIPS)، که مجموعه‌مقالاتش به‌عنوانِ «پیشرفت‌هایی در سامانه‌های پردازشِ اطلاعاتِ عصبی» منتشر می‌شود، شاملِ مقاله‌های یادگیریِ بیزیِ بسیاری است، همان‌گونه که کنفرانسِ سالانهٔ «هوش مصنوعی و آمار». مکان‌های صرفاً بیزی شاملِ نشست‌های بین‌المللیِ والنسیا دربارهٔ آمارِ بیزی و ژورنالِ Bayesian Analysis می‌شوند.

پانوشت‌ها:

[1] خوانندگانِ ازنظرِ آماریِ خبره این سناریو را گونه‌ای از چیدمانِ کوزه-و-گوی تشخیص می‌دهند. ما کوزه‌ها و گوی‌ها را کمتر از آب‌نبات قانع‌کننده می‌یابیم.

[2] قبلاً گفتیم کیسه‌های آب‌نبات بسیار بزرگ‌اند؛ در غیرِ این صورت، فرضِ i.i.d برقرار نمی‌ماند. از نظرِ فنی، درست‌تر (اما کم‌بهداشت‌تر) است که هر آب‌نبات را پس از بازرسی دوباره بپیچیم و به کیسه برگردانیم.

[3] ابرپارامتر نامیده می‌شوند، زیرا توزیعی روی θ را پارامتری می‌کنند، که خودش پارامتر است.

[4] دیگر پیشین‌های همساز شاملِ خانوادهٔ دیریکله برای پارامترهای توزیعِ گسستهٔ چندمقداره و خانوادهٔ نرمال–ویشارت برای پارامترهای توزیعِ گاوسی است. برناردو و اسمیت (1994) را ببینید.

[5] در عمل بهتر است آن‌ها را تصادفی انتخاب کنیم، تا از بیشینه‌های محلیِ ناشی از تقارن اجتناب شود.

فصل ۲۱: یادگیری عمیق (Deep Learning)

در این فصل نزولِ گرادیان برنامه‌های چندمرحله‌ای را یاد می‌گیرد، با پیامدهای چشمگیری برای زیرشاخه‌های اصلیِ هوش مصنوعی.

یادگیری عمیق خانواده‌ای گسترده از فنونِ یادگیریِ ماشینی است که در آن فرضیه‌ها شکلِ مدارهای جبریِ پیچیده با قدرتِ اتصالِ قابلِ تنظیم دارند. واژهٔ «عمیق» به این واقعیت اشاره دارد که مدارها معمولاً در بسیاری لایه سازماندهی می‌شوند، به این معنا که مسیرهای محاسبه از ورودی‌ها تا خروجی‌ها گام‌های بسیاری دارند. یادگیری عمیق در حالِ حاضر پرکاربردترین رویکرد برای کاربردهایی مانند بازشناسیِ اشیای بصری، ترجمهٔ ماشینی، بازشناسیِ گفتار، تولیدِ گفتار و تولیدِ تصویر است؛ همچنین نقشِ چشمگیری در کاربردهای یادگیریِ تقویتی دارد (فصل ۲۲ را ببینید).

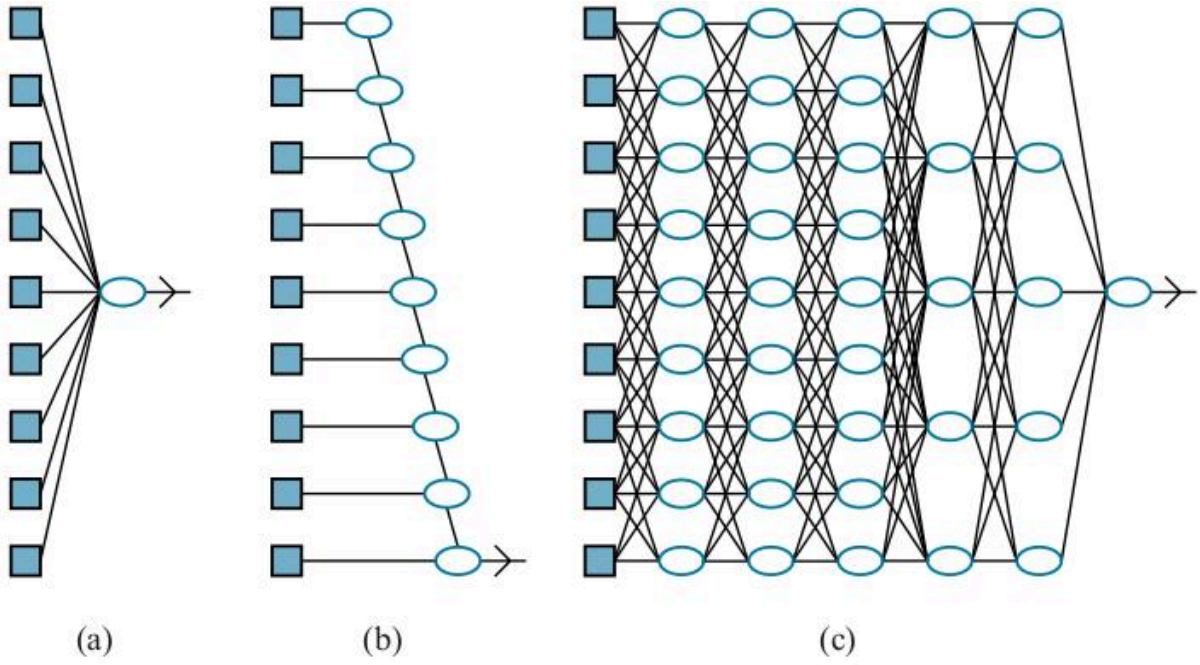
ریشه‌های یادگیری عمیق به کارهای اولیه‌ای بازمی‌گردد که کوشیدند شبکه‌های نورونیِ مغز (مک‌کالوک و پیتس، 1943) را با مدارهای محاسباتی مدل کنند. به همین دلیل، شبکه‌های آموزش‌دیده با روش‌های یادگیریِ عمیق اغلب «شبکه‌های عصبی» (Neural Network) نامیده می‌شوند، حتی اگر شباهت به سلول‌ها و ساختارهای عصبیِ واقعی سطحی باشد.

گرچه دلایلِ واقعیِ موفقیتِ یادگیری عمیق هنوز به‌طور کامل تبیین نشده، مزیت‌های آشکاری نسبت به برخی از روش‌های پوشش‌داده‌شده در فصل ۱۹ دارد—به‌ویژه برای داده‌های پربعد مانند تصاویر. برای نمونه، اگرچه روش‌هایی مانند رگرسیونِ خطی و لجستیک می‌توانند تعدادِ زیادی متغیرِ ورودی را مدیریت کنند، مسیرِ محاسبه از هر ورودی تا خروجی بسیار کوتاه است: ضرب در یک وزنِ تنها، و سپس جمع در خروجیِ تجمیعی. افزون بر این، متغیرهای ورودیِ متفاوت به‌طورِ مستقل در خروجی سهیم می‌شوند، بدون اینکه با یکدیگر برهم‌کنش کنند (شکل ۲۱.۱ (الف)). این قدرتِ بیانیِ این‌گونه مدل‌ها را به‌شدت محدود می‌کند. آن‌ها فقط می‌توانند توابعِ خطی و مرزهای خطی در فضای ورودی را بازنمایی کنند، در حالی که بیشترِ مفاهیمِ جهانِ واقعی بسیار پیچیده‌ترند.

فهرست‌های تصمیم و درخت‌های تصمیم، از سوی دیگر، مسیرهای محاسبهٔ بلندی را ممکن می‌سازند که می‌توانند به متغیرهای ورودیِ بسیاری وابسته باشند—اما فقط برای کسرِ نسبتاً کوچکی از بردارهای ورودیِ ممکن (شکل ۲۱.۱ (ب)). اگر درختِ تصمیم برای کسرِ چشمگیری از ورودی‌های ممکن مسیرهای محاسبهٔ بلند داشته باشد، باید در تعدادِ متغیرهای ورودی به‌طورِ نمایی بزرگ باشد.

ایدهٔ بنیادینِ یادگیری عمیق آموزشِ مدارهایی است که مسیرهای محاسبه‌شان بلند باشد، به‌گونه‌ای که همهٔ متغیرهای ورودی بتوانند به‌روش‌های پیچیده با هم برهم‌کنش کنند (شکل ۲۱.۱ (ج)). این مدل‌های مداری معلوم می‌شود به‌اندازهٔ کافی بیانگرند تا پیچیدگیِ داده‌های جهانِ واقعی را برای بسیاری از انواعِ مهمِ مسائلِ یادگیری دربرگیرند.

بخش ۲۱.۱ شبکه‌های پیشخورِ ساده، مؤلفه‌هایشان و اصولِ یادگیری در این‌گونه شبکه‌ها را توصیف می‌کند. بخش ۲۱.۲ با جزئیاتِ بیشتر به چگونگیِ کنارِ هم گذاشتنِ شبکه‌های عمیق می‌پردازد، و بخش ۲۱.۳ رده‌ای از شبکه‌ها به نامِ «شبکه‌های عصبیِ پیچشی» را پوشش می‌دهد که در کاربردهای بینایی اهمیتِ ویژه‌ای دارند. بخش‌های ۲۱.۴ و ۲۱.۵ با جزئیاتِ بیشتر به الگوریتم‌های آموزشِ شبکه‌ها از رویِ داده و روش‌های بهبودِ تعمیم می‌پردازند. بخش ۲۱.۶ شبکه‌های با ساختارِ بازگشتی را پوشش می‌دهد که برای داده‌های ترتیبی بسیار مناسب‌اند. بخش ۲۱.۷ راه‌های استفاده از یادگیری عمیق برای تکلیف‌هایی به‌جزِ یادگیریِ نظارت‌شده را توصیف می‌کند. سرانجام، بخش ۲۱.۸ گسترهٔ کاربردهای یادگیری عمیق را مرور می‌کند.



شکل ۲۱.۱ – (الف) یک مدلِ کم‌عمق، مانند رگرسیونِ خطی، مسیرهای محاسبهٔ کوتاهی بین ورودی‌ها و خروجی دارد. (ب) یک شبکهٔ فهرستِ تصمیم (صفحهٔ ۶۷۴) برای برخی از مقادیرِ ورودیِ ممکن مسیرهای بلندی دارد، اما بیشترِ مسیرها کوتاه‌اند. (ج) یک شبکهٔ یادگیریِ عمیق مسیرهای محاسبهٔ بلندتری دارد و به هر متغیر اجازه می‌دهد با همهٔ متغیرهای دیگر برهم‌کنش کند.

۲۱.۱ شبکه‌های پیشخورِ ساده

شبکهٔ پیشخور (Feedforward Network)، چنان‌که نامش می‌گوید، فقط در یک جهت اتصال دارد—یعنی گرافِ جهت‌دارِ بدونِ دور (DAG) با گره‌های ورودی و خروجیِ معیّن را تشکیل می‌دهد. هر گره تابعی از ورودی‌هایش را محاسبه می‌کند و نتیجه را به جانشین‌هایش در شبکه می‌سپارد. اطلاعات از گره‌های ورودی به گره‌های خروجی در شبکه جریان می‌یابد، و هیچ حلقه‌ای وجود ندارد.

شبکهٔ بازگشتی (Recurrent Network)، از سوی دیگر، خروجی‌های میانی یا نهایی‌اش را به ورودی‌های خودش بازمی‌گرداند. این بدان معناست که مقادیرِ سیگنال درونِ شبکه سامانهٔ پویایی را تشکیل می‌دهند که حالتِ درونی یا حافظه دارد. شبکه‌های بازگشتی را در بخش ۲۱.۶ بررسی خواهیم کرد.

مدارهای بولی، که توابعِ بولی را پیاده‌سازی می‌کنند، نمونه‌ای از شبکه‌های پیشخورند. در مدارِ بولی، ورودی‌ها به ۰ و ۱ محدودند، و هر گره تابعِ بولیِ ساده‌ای از ورودی‌هایش را پیاده‌سازی می‌کند و ۰ یا ۱ تولید می‌کند. در شبکه‌های عصبی، مقادیرِ ورودی معمولاً پیوسته‌اند، و گره‌ها ورودی‌های پیوسته می‌گیرند و خروجی‌های پیوسته تولید می‌کنند.

برخی از ورودی‌های گره‌ها پارامترهای شبکه‌اند؛ شبکه با تنظیمِ مقدارهای این پارامترها یاد می‌گیرد تا شبکه به‌عنوانِ یک کل با دادهٔ آموزشی برازش یابد.

۲۱.۱.۱ شبکه‌ها به‌عنوانِ توابعِ پیچیده

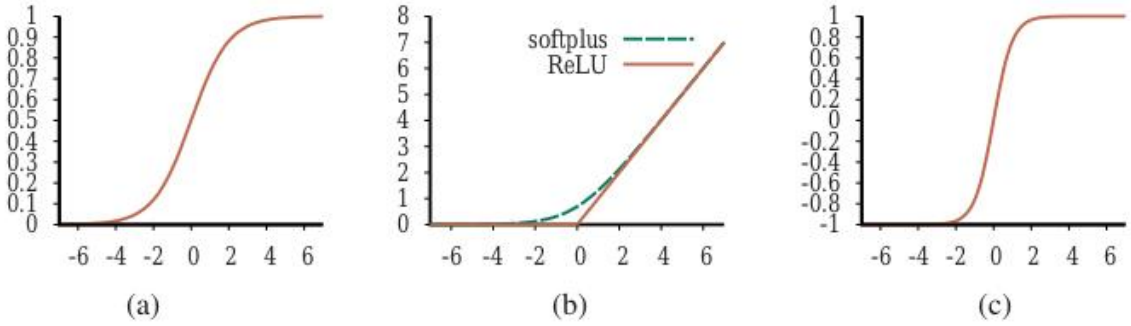
هر گره درونِ شبکه «واحد» (Unit) نامیده می‌شود. به‌طورِ سنتی، و به پیروی از طراحیِ پیشنهادیِ مک‌کالوک و پیتس، یک واحد جمعِ وزن‌دارِ ورودی‌های واحدهای پیشین را محاسبه می‌کند و سپس یک تابعِ غیرخطی برای تولیدِ خروجی‌اش به‌کار می‌برد. فرض کنید a_j خروجیِ واحدِ j و $w_{i,j}$ وزنِ متصل‌شده به پیوندِ واحدِ i به واحدِ j باشد؛ آنگاه داریم:

$$a_j = g_j(\sum_i w_{i,j}\, a_i) \equiv g_j(in_j)$$

که در آن g_j «تابعِ فعال‌سازی» (Activation Function) غیرخطیِ وابسته به واحدِ j و in_j جمعِ وزن‌دارِ ورودی‌های واحدِ j است. همان‌گونه که در بخش ۱۹.۶.۳ (صفحهٔ ۶۷۹)، تصریح می‌کنیم هر واحد یک ورودیِ اضافی از واحدِ مجازیِ ۰ دارد که ثابتاً 1+ است، همراه با وزنِ $w_{0,j}$ برای آن ورودی. این امکان را می‌دهد که ورودیِ وزن‌دارِ کلِ in_j به واحدِ j حتی وقتی خروجی‌های لایهٔ پیشین همه صفرند، ناصفر باشد. با این قرارداد می‌توانیم معادلهٔ پیشین را به‌صورتِ برداری بنویسیم:

$$a_j = g_j(w^T x) \qquad (21.1)$$

که در آن w بردارِ وزن‌هایی است که به واحدِ j می‌روند (شاملِ $w_{0,j}$) و x بردارِ ورودی‌های واحدِ j است (شاملِ 1+).



شکل ۲۱.۲ — توابعِ فعال‌سازیِ پرکاربرد در سامانه‌های یادگیریِ عمیق: (الف) تابعِ لجستیک یا سیگموئید؛ (ب) تابعِ ReLU و تابعِ سافت‌پلاس؛ (ج) تابعِ tanh. (در هر نمودار، محورِ افقی ورودی را در بازهٔ −6 تا 6 و محورِ عمودی خروجی را در بازهٔ −1 تا 1 (یا 0 تا 1) نشان می‌دهد.)

این واقعیت که تابعِ فعال‌سازی غیرخطی است مهم است، زیرا اگر نبود، هر ترکیبی از واحدها همچنان تابعی خطی را بازنمایی می‌کرد. غیرخطی بودن چیزی است که به شبکه‌های به‌اندازهٔ کافی بزرگِ واحدها امکان می‌دهد توابعِ دلبخواه را بازنمایی کنند. «قضیهٔ تقریبِ همگانی» (Universal Approximation Theorem) بیان می‌کند شبکه‌ای با فقط دو لایهٔ واحدهای محاسباتی—لایهٔ نخست غیرخطی و لایهٔ دوم خطی—می‌تواند هر تابعِ پیوسته‌ای را با هر درجهٔ دقتِ دلخواهی تقریب بزند. اثبات با نشان‌دادن این کار می‌کند که شبکه‌ای به‌طورِ نمایی بزرگ می‌تواند به‌طورِ نمایی «برآمدگی‌های» زیادی با ارتفاع‌های متفاوت در مکان‌های متفاوتِ فضای ورودی بازنمایی کند و از این راه تابعِ موردنظر را تقریب بزند. به بیانِ دیگر، شبکه‌های به‌اندازهٔ کافی بزرگ می‌توانند برای توابعِ پیوسته جدولِ مراجعه پیاده‌سازی کنند، همان‌گونه که درخت‌های تصمیمِ به‌اندازهٔ کافی بزرگ جدولِ مراجعه‌ای برای توابعِ بولی پیاده‌سازی می‌کنند.

انواعِ گوناگونی از توابعِ فعال‌سازی به‌کار می‌روند. رایج‌ترین آن‌ها این‌اند:

- تابعِ لجستیک یا سیگموئید (Sigmoid)، که در رگرسیونِ لجستیک نیز استفاده می‌شود (صفحهٔ ۶۸۵ را ببینید):

$$\text{Sigmoid:}\quad \sigma(x) = 1 / (1 + e^{-x})$$

- تابعِ ReLU، که نامش کوتاه‌شدهٔ «واحدِ خطیِ تصحیح‌شده» (Rectified Linear Unit) است:

$$\text{ReLU:}\quad \text{ReLU}(x) = \max(0, x)$$

- تابعِ سافت‌پلاس (Softplus)، نسخهٔ هموارِ تابعِ ReLU:

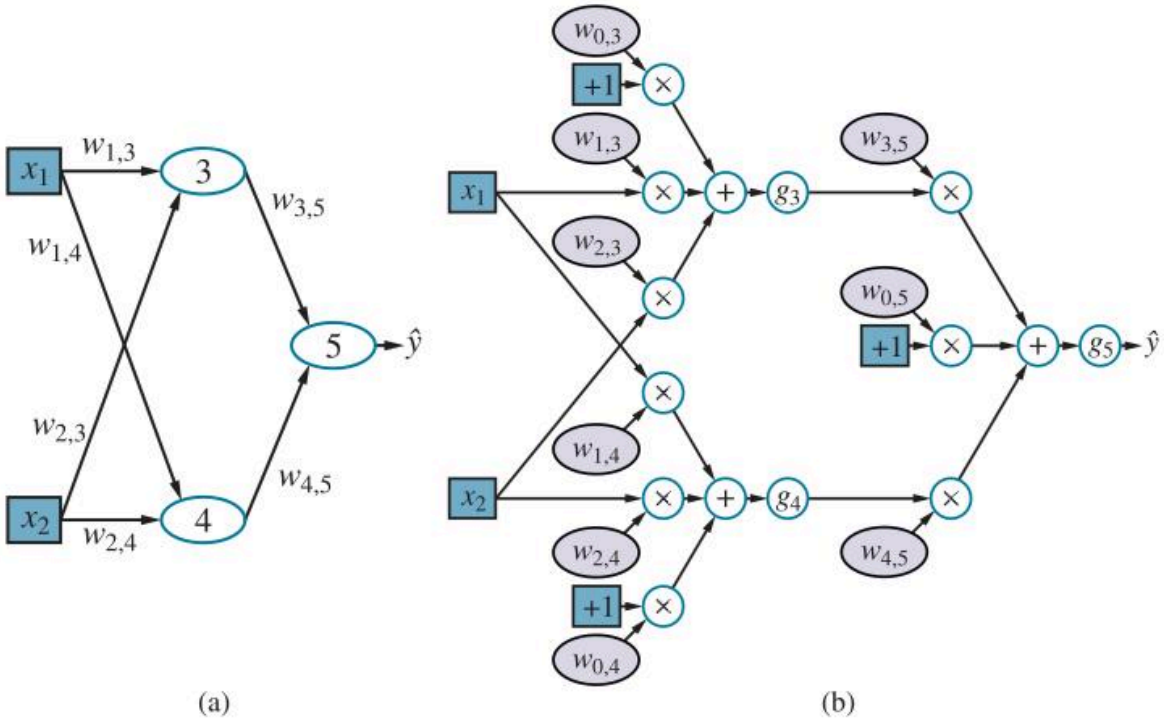
$$\text{Softplus:}\quad \text{softplus}(x) = \log(1 + e^{x})$$

مشتقِ تابعِ سافت‌پلاس تابعِ سیگموئید است.

- تابعِ tanh:

$$\text{Tanh:}\quad \tanh(x) = (e^{2x} - 1) / (e^{2x} + 1)$$

توجه کنید بردِ tanh بازهٔ (1+ ,1-) است. Tanh نسخهٔ مقیاس‌شده و جابه‌جاشدهٔ سیگموئید است، زیرا tanh(x) = 2σ(2x)-1. این توابع در شکل ۲۱.۲ نشان داده شده‌اند. توجه کنید همهٔ آن‌ها یکنواختِ غیرنزولی‌اند، به این معنا که مشتق‌هایشان g' نامنفی‌اند. دربارهٔ انتخابِ تابعِ فعال‌سازی در بخش‌های بعد حرف‌های بیشتری خواهیم داشت.



شکل ۲۱.۳ – (الف) شبکهٔ عصبی با دو ورودی، یک لایهٔ پنهانِ دو واحدی و یک واحدِ خروجی. ورودی‌های مجازی و وزن‌های وابسته به آن‌ها نشان داده نشده‌اند. (ب) شبکه در (الف) که به گرافِ محاسباتیِ کاملِ خود باز شده است.

به‌هم‌پیوستنِ چند واحد در شبکه‌ای تابعِ پیچیده‌ای می‌سازد که ترکیبی از عبارت‌های جبریِ بازنمایی‌شده توسط واحدهای منفرد است. برای نمونه، شبکهٔ نشان‌داده‌شده در شکل ۲۱.۳ (الف) تابعِ h_w(x) را بازنمایی می‌کند، که با وزن‌های w پارامتری شده و بردارِ ورودیِ دو عنصریِ x را به مقدارِ خروجیِ اسکالرِ ŷ نگاشت می‌کند. ساختارِ درونیِ تابع آینهٔ ساختارِ شبکه است. برای نمونه، می‌توانیم عبارتی برای خروجیِ ŷ به‌صورتِ زیر بنویسیم:

$$\begin{aligned}\hat{y} &= g_5(in_5) \\ &= g_5(w_{0,5} + w_{3,5}\, a_3 + w_{4,5}\, a_4) \\ &= g_5(w_{0,5} + w_{3,5}\, g_3(in_3) + w_{4,5}\, g_4(in_4)) \\ &= g_5(w_{0,5} + w_{3,5}\, g_3(w_{0,3} + w_{1,3}\, x_1 + w_{2,3}\, x_2) + w_{4,5}\, g_4(w_{0,4} + w_{1,4}\, x_1 + w_{2,4}\, x_2))\end{aligned} \tag{21.2}$$

بنابراین، خروجیِ ŷ را به‌صورتِ تابعِ h_w(x) از ورودی‌ها و وزن‌ها بیان کرده‌ایم.

شکل ۲۱.۳ (الف) شیوهٔ سنتیِ ترسیمِ شبکه در کتابی دربارهٔ شبکه‌های عصبی را نشان می‌دهد. شیوهٔ عمومی‌ترِ اندیشیدن دربارهٔ شبکه این است که آن را «گرافِ محاسباتی» (Computation Graph) یا «گرافِ جریانِ داده» (Dataflow Graph) در نظر بگیریم—اساساً مداری که در آن هر گره یک محاسبهٔ مقدماتی را بازنمایی می‌کند. شکل ۲۱.۳ (ب) گرافِ محاسباتیِ متناظر با شبکهٔ شکل ۲۱.۳ (الف) را نشان می‌دهد؛ این گراف هر عنصرِ محاسبهٔ کلی را صریح می‌کند. همچنین بین ورودی‌ها (به رنگِ آبی) و وزن‌ها (به رنگِ بنفشِ روشن) تمایز می‌گذارد: وزن‌ها را می‌توان تنظیم کرد تا خروجیِ ŷ با مقدارِ واقعیِ y در دادهٔ آموزشی هماهنگ‌تر شود.

هر وزن مانند دستگیرهٔ کنترلِ بلندیِ صدا است که تعیین می‌کند گرهٔ بعدی در گراف چقدر از آن پیشینیِ خاص در گراف بشنود.

همان‌گونه که معادلهٔ (21.1) عملکردِ یک واحد را به‌صورتِ برداری توصیف کرد، می‌توانیم کارِ مشابهی برای شبکه به‌عنوانِ یک کل انجام دهیم. به‌طورِ کلی W را برای نشان‌دادنِ ماتریسِ وزن به‌کار می‌بریم؛ برای این شبکه، $W^{(1)}$ وزن‌های لایهٔ نخست ($w_{1,3}$، $w_{1,4}$ و مانندِ آن) و $W^{(2)}$ وزن‌های لایهٔ دوم ($w_{3,5}$ و مانندِ آن) را نشان می‌دهد. سرانجام، $g^{(1)}$ و $g^{(2)}$ توابعِ فعال‌سازیِ لایهٔ نخست و دوم باشند. آنگاه کلِ شبکه را می‌توان به‌صورتِ زیر نوشت:

$$h_w(x) = g^{(2)}(W^{(2)}\, g^{(1)}(W^{(1)}\, x)) \qquad (21.3)$$

مانندِ معادلهٔ (21.2)، این عبارت با یک گرافِ محاسباتی متناظر است، هرچند بسیار ساده‌تر از گرافِ شکل ۲۱.۳ (ب): در این‌جا گراف صرفاً زنجیره‌ای است که ماتریس‌های وزن به هر لایه خوراک می‌دهند.

گرافِ محاسباتیِ شکل ۲۱.۳ (ب) نسبتاً کوچک و کم‌عمق است، اما همان ایده برای همهٔ شکل‌های یادگیری عمیق برقرار است: گراف‌های محاسباتی می‌سازیم و وزن‌هایشان را برای برازشِ داده تنظیم می‌کنیم. گرافِ شکل ۲۱.۳ (ب) همچنین «کاملاً متصل» (Fully Connected) است، به این معنا که هر گره در هر لایه به هر گره در لایهٔ بعد متصل است. این به‌نوعی حالتِ پیش‌فرض است، اما در بخش ۲۱.۳ خواهیم دید انتخابِ نحوهٔ اتصالِ شبکه نیز در دست‌یافتنِ یادگیریِ مؤثر اهمیت دارد.

۲۱.۱.۲ گرادیان‌ها و یادگیری

در بخش ۱۹.۶ رویکردی به یادگیریِ نظارت‌شده بر مبنای نزولِ گرادیان معرفی کردیم: گرادیانِ تابعِ زیان نسبت به وزن‌ها را محاسبه کن، و وزن‌ها را در جهتِ گرادیان تنظیم کن تا زیان کم شود. (اگر هنوز بخش ۱۹.۶ را نخوانده‌اید، اکیداً توصیه می‌کنیم پیش از ادامه این کار را بکنید.) دقیقاً همان رویکرد را می‌توانیم برای یادگیریِ وزن‌ها در گراف‌های محاسباتی به‌کار ببریم. برای وزن‌هایی که به واحدهای لایهٔ خروجی می‌روند—یعنی آن‌هایی که خروجیِ شبکه را تولید می‌کنند—محاسبهٔ گرادیان اساساً با فرایندِ بخش ۱۹.۶ یکسان است. برای وزن‌هایی که به واحدهای لایه‌های پنهان می‌روند، که مستقیماً به خروجی‌ها متصل نیستند، فرایند فقط کمی پیچیده‌تر است.

فعلاً تابعِ زیانِ مربعِ خطا، L_2، را به‌کار می‌بریم و گرادیان را برای شبکهٔ شکل ۲۱.۳ نسبت به یک مثالِ آموزشیِ تنها (x, y) محاسبه می‌کنیم. (برای چند مثال، گرادیان صرفاً جمعِ گرادیان‌های مثال‌های منفرد است.) شبکه پیش‌بینیِ $\hat{y} = h_w(x)$ را خروجی می‌دهد و مقدارِ واقعی y است، پس داریم:

$$Loss(h_w) = L_2(y, h_w(x)) = \|y - h_w(x)\|^2 = (y - \hat{y})^2$$

برای محاسبهٔ گرادیانِ زیان نسبت به وزن‌ها، به همان ابزارهای حسابِ دیفرانسیلِ فصل ۱۹ نیاز داریم—عمدتاً قاعدهٔ زنجیره‌ای، $\partial g(f(x))/\partial x = g'(f(x))\, \partial f(x)/\partial x$.

از حالتِ آسان شروع می‌کنیم: وزنی مانند $w_{3,5}$ که به واحدِ خروجی متصل است. مستقیماً روی عبارت‌های تعریف‌کنندهٔ شبکه از معادلهٔ (21.2) کار می‌کنیم:

$$\begin{aligned}
\partial/\partial w_{3,5}\ \text{Loss}(h_w) &= \partial/\partial w_{3,5}\ (y - \hat{y})^2 \\
&= -2(y - \hat{y})\ \partial \hat{y}/\partial w_{3,5} \\
&= -2(y - \hat{y})\ \partial/\partial w_{3,5}\ g_5(in_5) \\
&= -2(y - \hat{y})\ g'_5(in_5)\ \partial/\partial w_{3,5}\ in_5 \\
&= -2(y - \hat{y})\ g'_5(in_5)\ \partial/\partial w_{3,5}\ (w_{0,5} + w_{3,5}\ a_3 + w_{4,5}\ a_4) \\
&= -2(y - \hat{y})\ g'_5(in_5)\ a_3 \qquad (21.4)
\end{aligned}$$

ساده‌شدنِ خطِ آخر از این روست که $w_{0,5}$ و $w_{4,5}\ a_4$ به $w_{3,5}$ وابسته نیستند، و نه ضریبِ $w_{3,5}$ یعنی a_3.

موردِ کمی دشوارتر وزنی مانند $w_{1,3}$ است که مستقیماً به واحدِ خروجی متصل نیست. در این‌جا باید قاعدهٔ زنجیره‌ای را یک بار دیگر به‌کار ببریم. چند گامِ نخست یکسان‌اند، پس آن‌ها را حذف می‌کنیم:

$$\begin{aligned}
\partial/\partial w_{1,3}\ \text{Loss}(h_w) &= -2(y - \hat{y})\ g'_5(in_5)\ \partial/\partial w_{1,3}\ (w_{0,5} + w_{3,5}\ a_3 + w_{4,5}\ a_4) \\
&= -2(y - \hat{y})\ g'_5(in_5)\ w_{3,5}\ \partial/\partial w_{1,3}\ a_3 \\
&= -2(y - \hat{y})\ g'_5(in_5)\ w_{3,5}\ \partial/\partial w_{1,3}\ g_3(in_3) \\
&= -2(y - \hat{y})\ g'_5(in_5)\ w_{3,5}\ g'_3(in_3)\ \partial/\partial w_{1,3}\ in_3 \\
&= -2(y - \hat{y})\ g'_5(in_5)\ w_{3,5}\ g'_3(in_3)\ \partial/\partial w_{1,3}\ (w_{0,3} + w_{1,3}\ x_1 + w_{2,3}\ x_2) \\
&= -2(y - \hat{y})\ g'_5(in_5)\ w_{3,5}\ g'_3(in_3)\ x_1 \qquad (21.5)
\end{aligned}$$

پس، برای گرادیانِ زیان نسبت به وزن‌های $w_{3,5}$ و $w_{1,3}$ عبارت‌های نسبتاً ساده‌ای داریم. اگر $\Delta_5 = 2(\hat{y} - y)\ g'_5(in_5)$ را به‌عنوان نوعی «خطایِ ادراک‌شده» در نقطه‌ای که واحدِ ۵ ورودی‌اش را دریافت می‌کند تعریف کنیم، گرادیان نسبت به $w_{3,5}$ صرفاً $\Delta_5\ a_3$ است. این کاملاً منطقی است: اگر Δ_5 مثبت باشد، یعنی $\hat{y}$ بیش‌ازحد بزرگ است (به یاد آورید g' همیشه نامنفی است)؛ اگر a_3 نیز مثبت باشد، افزایشِ $w_{3,5}$ فقط اوضاع را بدتر می‌کند، در حالی که اگر a_3 منفی باشد، افزایشِ $w_{3,5}$ خطا را کاهش می‌دهد. اندازهٔ a_3 نیز اهمیت دارد: اگر a_3 برای این مثالِ آموزشی کوچک باشد، $w_{3,5}$ نقشِ اصلی‌ای در تولیدِ خطا نداشته و نیازی به تغییرِ زیاد ندارد. اگر $\Delta_3 = \Delta_5\ w_{3,5}\ g'_3(in_3)$ را نیز تعریف کنیم، گرادیانِ $w_{1,3}$ صرفاً $\Delta_3\ x_1$ می‌شود. بدین‌سان، خطایِ ادراک‌شده در ورودیِ واحدِ ۳ برابر است با خطایِ ادراک‌شده در ورودیِ واحدِ ۵، ضرب در اطلاعات در طولِ مسیرِ بازگشت از ۵ به ۳. این پدیده کاملاً عمومی است، و نامِ «پسانتشار» (Back-propagation) را برای شیوه‌ای که خطا در خروجی از میانِ شبکه به عقب گذرانده می‌شود به وجود می‌آورد. ویژگیِ مهمِ دیگرِ این عبارت‌های گرادیانی این است که مشتق‌های محلیِ $g'_j(in_j)$ را به‌عنوانِ عامل دارند. همان‌گونه که پیش‌تر اشاره شد، این مشتق‌ها همیشه نامنفی‌اند، اما اگر ورودی‌های مثالِ آموزشیِ موردنظر واحدِ j را در ناحیهٔ تختِ عملکرد قرار دهند، می‌توانند بسیار نزدیکِ صفر باشند (در موردِ سیگموئید، سافت‌پلاس و tanh) یا دقیقاً صفر (در موردِ ReLU). اگر g'_j کوچک یا صفر باشد، یعنی تغییرِ وزن‌هایی که به واحدِ j می‌روند اثری ناچیز بر خروجی‌اش خواهد داشت.

در نتیجه، شبکه‌های عمیق با لایه‌های بسیار ممکن است از «گرادیانِ ناپدیدشونده» (Vanishing Gradient) رنج ببرند—سیگنال‌های خطا در حینِ بازپس‌دهی از میانِ شبکه یک‌باره خاموش می‌شوند. بخش ۲۱.۳.۳ راه‌حلی برای این مسئله ارائه می‌دهد.

نشان دادیم گرادیان‌ها در شبکهٔ نمونهٔ کوچکِ ما عبارت‌های ساده‌ای‌اند که با بازپس‌دادنِ اطلاعات از واحدهای خروجی از میانِ شبکه محاسبه می‌شوند. معلوم می‌شود این ویژگی به‌طورِ کلی‌تر نیز برقرار است. در واقع، چنان‌که در بخش ۲۱.۴.۱ نشان می‌دهیم، محاسباتِ گرادیان برای هر گرافِ محاسباتیِ پیشخور همان ساختارِ گرافِ محاسباتیِ زیرین را دارند. این ویژگی مستقیماً از قاعده‌های دیفرانسیل‌گیری نتیجه می‌شود. جزئیاتِ خسته‌کنندهٔ یک محاسبهٔ گرادیان را نشان داده‌ایم، اما نگران نباشید: برای هر ساختارِ شبکهٔ تازه نیازی به بازانجامِ اشتقاق‌های معادله‌های (21.4) و (21.5) نیست! همهٔ این‌گونه گرادیان‌ها را می‌توان با روشِ «تمایزِ خودکار» (Automatic Differentiation) محاسبه کرد، که قاعده‌های حسابِ دیفرانسیل را به‌شکلی نظام‌مند برای محاسبهٔ گرادیان‌های هر برنامهٔ عددی به‌کار می‌برد.[1] در واقع، روشِ پسانتشار در یادگیری عمیق صرفاً کاربردِ «تمایزِ حالتِ معکوس» (Reverse Mode) است، که قاعدهٔ زنجیره‌ای را «از بیرون به درون» به‌کار می‌برد و وقتی شبکهٔ موردنظر ورودی‌های بسیاری و خروجی‌های نسبتاً کمی دارد، از مزیتِ کاراییِ برنامه‌نویسیِ پویا بهره می‌گیرد. همهٔ بسته‌های نرم‌افزاریِ عمدهٔ یادگیری عمیق تمایزِ خودکار فراهم می‌کنند، تا کاربران بتوانند آزادانه با ساختارهای شبکهٔ متفاوت، توابعِ فعال‌سازی، توابعِ زیان و شکل‌های ترکیب آزمایش کنند، بی‌آنکه برای هر آزمایش مجبور باشند کلی حسابِ دیفرانسیل انجام دهند تا الگوریتمِ یادگیریِ تازه‌ای اشتقاق دهند. این رویکردی به نامِ «یادگیریِ سرتاسری» (End-to-End Learning) را تشویق کرده، که در آن سامانهٔ محاسباتیِ پیچیده‌ای برای تکلیفی مانند ترجمهٔ ماشینی از چند زیرسامانهٔ قابلِ آموزش ترکیب می‌شود؛ سپس کلِ سامانه به‌شکلِ سرتاسری از رویِ جفت‌های ورودی-خروجی آموزش می‌بیند. با این رویکرد، طراح فقط باید تصورِ مبهمی از چگونگیِ ساختاربندیِ سامانهٔ کلی داشته باشد؛ نیازی به دانستنِ از پیشِ دقیقِ این‌که هر زیرسامانه باید چه کند یا چگونه ورودی‌ها و خروجی‌هایش را برچسب‌گذاری کند نیست.

۲۱.۲ گراف‌های محاسباتی برای یادگیری عمیق

ایده‌های بنیادینِ یادگیری عمیق را برقرار کرده‌ایم: فرضیه‌ها را به‌صورتِ گراف‌های محاسباتی با وزن‌های قابلِ تنظیم بازنمایی کن و گرادیانِ تابعِ زیان نسبت به آن وزن‌ها را برای برازشِ دادهٔ آموزشی محاسبه کن. حالا می‌بینیم گراف‌های محاسباتی چگونه کنارِ هم گذاشته می‌شوند. با لایهٔ ورودی شروع می‌کنیم، جایی که مثالِ آموزشی یا آزمونِ x به‌صورتِ مقادیرِ گره‌های ورودی رمز می‌شود. سپس لایهٔ خروجی را در نظر می‌گیریم، جایی که خروجی‌های $\hat{y}$ با مقادیرِ واقعیِ y مقایسه می‌شوند تا سیگنالِ یادگیری برای تنظیمِ وزن‌ها به‌دست آید. سرانجام، لایه‌های پنهانِ شبکه را بررسی می‌کنیم.

۲۱.۲.۱ رمزگذاریِ ورودی

گره‌های ورودی و خروجیِ گرافِ محاسباتی آن‌هایی‌اند که مستقیماً به دادهٔ ورودیِ x و دادهٔ خروجیِ y متصل‌اند. رمزگذاریِ دادهٔ ورودی معمولاً ساده است، دست‌کم برای حالتِ دادهٔ عامل‌دار، که هر مثالِ آموزشی مقدارهایی برای n ویژگی دارد. اگر ویژگی‌ها بولی باشند، n گرهٔ ورودی داریم؛ معمولاً «کاذب» به ورودیِ ۰ و «صادق» به ۱ نگاشت می‌شود، هرچند گاهی -1 و $+1$ به‌کار می‌رود. ویژگی‌های عددی، خواه عددِ صحیح باشند خواه حقیقی، معمولاً همان‌گونه که هستند به‌کار می‌روند، هرچند ممکن است مقیاس‌بندی شوند تا درونِ بازهٔ ثابتی بگنجند؛ اگر اندازه‌ها برای مثال‌های متفاوت به‌شدت تغییر کنند، می‌توان مقادیر را به مقیاسِ لگاریتمی نگاشت کرد.

تصاویر دقیقاً در ردهٔ دادهٔ عامل‌دار نمی‌گنجند؛ هرچند می‌توان تصویرِ RGB با اندازهٔ X×Y پیکسل را 3XY ویژگیِ عددِ صحیح‌مقدار (معمولاً با مقادیرِ درونِ بازهٔ {0,...,255}) در نظر گرفت، این کار این واقعیت را نادیده می‌گیرد که سه‌تایی‌های RGB به همان پیکسلِ تصویر تعلق دارند و این‌که مجاورتِ پیکسل‌ها واقعاً اهمیت دارد. البته می‌توانیم پیکسل‌های مجاور را به گره‌های ورودیِ مجاورِ شبکه نگاشت کنیم، اما معنایِ مجاورت اگر لایه‌های درونیِ شبکه کاملاً متصل باشند کاملاً از بین می‌رود. در عمل، شبکه‌های به‌کاررفته با دادهٔ تصویری ساختارهای درونیِ آرایه‌مانندی دارند که هدف‌شان بازتابِ معناشناسیِ مجاورت است. این را با جزئیاتِ بیشتر در بخش ۲۱.۳ خواهیم دید.

ویژگی‌های رسته‌ای با بیش از دو مقدار—مانندِ ویژگیِ «نوع» در مسئلهٔ رستوران از فصل ۱۹، که مقادیرِ فرانسوی، ایتالیایی، تایلندی یا برگر دارد—معمولاً با به‌اصطلاح «کدگذاریِ یک-گرم» (One-Hot Encoding) رمز می‌شوند. ویژگی‌ای با d مقدارِ ممکن با d بیتِ ورودیِ جداگانه بازنمایی می‌شود. برای هر مقدارِ داده‌شده، بیتِ ورودیِ متناظر ۱ و همهٔ بقیه ۰ می‌شوند. این معمولاً بهتر از نگاشتِ مقادیر به عددهای صحیح کار می‌کند. اگر برای ویژگیِ «نوع» عدد صحیح به‌کار می‌بردیم، تایلندی ۳ و برگر ۴ می‌شد. از آنجا که شبکه ترکیبی از توابعِ پیوسته است، چاره‌ای جز توجه به مجاورتِ عددی ندارد، اما در این مورد مجاورتِ عددیِ بینِ تایلندی و برگر از نظرِ معنایی بی‌معناست.

۲۱.۲.۲ لایه‌های خروجی و توابعِ زیان

در سمتِ خروجیِ شبکه، مسئلهٔ رمزکردنِ مقادیرِ خامِ داده به مقادیرِ واقعیِ y برای گره‌های خروجیِ گراف کم‌وبیش همان مسئلهٔ رمزگذاریِ ورودی است. برای نمونه، اگر شبکه بخواهد متغیرِ «آب‌وهوا» از فصل ۱۲ را پیش‌بینی کند، که مقادیرِ {آفتابی، بارانی، ابری، برفی} را دارد، از کدگذاریِ یک-گرم با چهار بیت استفاده می‌کردیم.

بسیار خب، این‌هم از مقادیرِ دادهٔ y. دربارهٔ پیش‌بینیِ ŷ چه؟ در حالتِ ایده‌آل، دقیقاً با مقدارِ مطلوبِ y منطبق می‌شد و زیان صفر می‌شد و کار تمام بود. در عمل، این به‌ندرت رخ می‌دهد—به‌ویژه پیش از آنکه فرایندِ تنظیمِ وزن‌ها را شروع کرده باشیم! پس باید بیندیشیم مقدارِ خروجیِ نادرست چه معنایی دارد و زیان چگونه اندازه‌گیری می‌شود. در اشتقاقِ گرادیان‌های معادله‌های (21.4) و (21.5)، با تابعِ زیانِ مربعِ خطا شروع کردیم. این جبر را ساده نگه می‌دارد، اما تنها امکان نیست. در واقع، برای بیشترِ کاربردهای یادگیری عمیق، رایج‌تر است که مقادیرِ خروجیِ ŷ را به‌عنوانِ احتمال تفسیر کنیم و لگاریتمِ منفیِ درست‌نمایی را به‌عنوانِ تابعِ زیان به‌کار ببریم—دقیقاً همان‌گونه که در فصل ۲۰ با یادگیریِ حداکثرِ درست‌نمایی انجام دادیم.

یادگیریِ حداکثرِ درست‌نمایی مقدارِ wای را می‌یابد که احتمالِ دادهٔ مشاهده‌شده را بیشینه می‌کند. و چون تابعِ لگاریتم یکنواخت است، این معادل است با بیشینه‌کردنِ لگاریتمِ درست‌نمایی داده، که به‌نوبهٔ خود معادل است با کمینه‌کردنِ تابعِ زیانی که به‌عنوانِ لگاریتمِ منفیِ درست‌نمایی تعریف شده. (به یاد آورید از صفحهٔ ۷۲۵ که گرفتنِ لگاریتم حاصلضربِ احتمال‌ها را به جمع تبدیل می‌کند، که برای مشتق‌گیری مناسب‌تر است.) به بیانِ دیگر، به دنبالِ w*ای هستیم که جمعِ لگاریتم‌های منفیِ احتمالِ N مثال را کمینه کند:

$$w^* = \operatorname{argmin}_w \left[-\sum_{j=1}^{N} \log P_w(y_j \mid x_j) \right] \qquad (21.6)$$

در ادبیاتِ یادگیری عمیق، صحبت از کمینه‌کردنِ «زیانِ آنتروپیِ متقاطع» (Cross-Entropy) رایج است. آنتروپیِ متقاطع، که H(P,Q) نوشته می‌شود، نوعی سنجهٔ ناهم‌مانندی بینِ دو توزیعِ P و Q است.[۲] تعریفِ عمومی این است:

$$H(P, Q) = E_{z\sim P(z)}[\log Q(z)] = \int P(z) \log Q(z)\, dz \qquad (21.7)$$

در یادگیری ماشینی، معمولاً این تعریف را با P به‌عنوانِ توزیعِ واقعیِ رویِ مثال‌های آموزشی، یعنی P*(x,y)، و Q به‌عنوانِ فرضیهٔ پیش‌گویانهٔ P_w(y|x) به‌کار می‌بریم. کمینه‌کردنِ آنتروپیِ متقاطعِ H(P*(x,y), P_w(y|x)) با تنظیمِ w، فرضیه را به توزیعِ واقعی تا حدِ ممکن نزدیک می‌کند. در واقعیت، نمی‌توانیم این آنتروپیِ متقاطع را کمینه کنیم، زیرا به توزیعِ دادهٔ واقعیِ P*(x,y) دسترسی نداریم؛ اما به نمونه‌هایی از P*(x,y) دسترسی داریم، پس جمعِ رویِ دادهٔ واقعی در معادلهٔ (21.6) امیدِ ریاضیِ معادلهٔ (21.7) را تقریب می‌زند. برای کمینه‌کردنِ لگاریتمِ منفیِ درست‌نمایی (یا آنتروپیِ متقاطع)، باید بتوانیم خروجیِ شبکه را به‌عنوانِ احتمال تفسیر کنیم. برای نمونه، اگر شبکه یک واحدِ خروجی با تابعِ فعال‌سازیِ سیگموئید داشته باشد و رده‌بندیِ بولی یاد بگیرد، می‌توانیم مقدارِ خروجی را مستقیماً به‌عنوانِ احتمالِ تعلقِ مثال به ردهٔ مثبت تفسیر کنیم. (در واقع، این دقیقاً شیوهٔ استفاده از رگرسیونِ لجستیک است؛ صفحهٔ ۶۸۴ را ببینید.) پس برای مسائلِ رده‌بندیِ بولی معمولاً از لایهٔ خروجیِ سیگموئیدی استفاده می‌کنیم.

مسائلِ رده‌بندیِ چندرده‌ای در یادگیری ماشینی بسیار رایج‌اند. برای نمونه، رده‌بندهایی که برای بازشناسیِ اشیا استفاده می‌شوند اغلب باید هزاران ردهٔ متمایز از اشیا را بازشناسی کنند. مدل‌های زبانِ طبیعی که می‌کوشند واژهٔ بعدیِ جمله را پیش‌بینی کنند ممکن است مجبور باشند از میانِ ده‌ها هزار واژهٔ ممکن انتخاب کنند. برای این‌گونه پیش‌بینی، نیاز داریم شبکه توزیعِ رسته‌ای خروجی دهد—یعنی اگر d پاسخِ ممکن باشد، به d گرهٔ خروجی نیاز داریم که احتمال‌هایی را بازنمایی کنند که جمعِشان ۱ است.

برای دست‌یابی به این هدف، از «لایهٔ سافتمکس» (Softmax) استفاده می‌کنیم، که با گرفتنِ بردارِ ورودیِ $\text{in} = \langle \text{in}_1, \ldots, \text{in_d} \rangle$ بردارِ d مقدار خروجی می‌دهد. عنصرِ kامِ آن بردارِ خروجی به‌صورتِ زیر است:

$$\text{softmax}(\text{in})_k = e^{\text{in}_k} / \sum_{k'=1}^{d} e^{\text{in}_{k'}}$$

با ساختِ این‌گونه، تابعِ سافتمکس بردارِ اعدادِ نامنفی‌ای خروجی می‌دهد که جمعِشان ۱ است.

مثلِ همیشه، ورودیِ in_k به هر یک از گره‌های خروجی ترکیبِ خطیِ وزن‌داری از خروجی‌های لایهٔ پیشین است. به‌دلیلِ نمایی‌ها، لایهٔ سافتمکس تفاوت‌های ورودی‌ها را برجسته می‌کند: برای نمونه، اگر بردارِ ورودی $\text{in} = \langle 5, 2, 0, -2 \rangle$ باشد، خروجی‌ها $\langle 0.946, 0.047, 0.006, 0.001 \rangle$ خواهند بود. سافتمکس با این حال هموار و مشتق‌پذیر است (تمرین SOFG.۲۱)، برخلافِ تابعِ بیشینه. به‌آسانی نشان داده می‌شود (تمرین SMSG.۲۱) که سیگموئید یک سافتمکس با $d = 2$ است. به بیانِ دیگر، همان‌گونه که واحدهای سیگموئیدی اطلاعاتِ ردهٔ دودویی را از میانِ شبکه منتشر می‌کنند، واحدهای سافتمکسی اطلاعاتِ چندرده‌ای را منتشر می‌کنند.

برای مسئلهٔ رگرسیون، که مقدارِ هدفِ y پیوسته است، استفاده از «لایهٔ خروجیِ خطی» رایج است—به بیانِ دیگر، $\hat{y}_j = in_j$، بدونِ هیچ تابعِ فعال‌سازیِ g—و تفسیرِ آن به‌عنوانِ میانگینِ توزیعِ گاوسیِ پیش‌بینی با واریانسِ ثابت. همان‌گونه که در صفحهٔ ۷۲۹ اشاره کردیم، بیشینه‌کردنِ درست‌نمایی (یعنی کمینه‌کردنِ لگاریتمِ منفیِ درست‌نمایی) با گاوسیِ واریانس‌ثابت همان کمینه‌کردنِ مربعِ خطاست. پس لایهٔ خروجیِ خطیِ تفسیرشده به این‌شکل، همان کارِ رگرسیونِ خطیِ کلاسیک را می‌کند. ویژگی‌های ورودیِ این رگرسیونِ خطی خروجی‌های لایهٔ پیشین‌اند، که معمولاً از چند تبدیلِ غیرخطیِ ورودی‌های اصلیِ شبکه نتیجه می‌شوند.

بسیاری از لایه‌های خروجیِ دیگر نیز ممکن‌اند. برای نمونه، «لایهٔ چگالیِ مخلوط» (Mixture Density Layer) خروجی‌ها را با مخلوطی از توزیع‌های گاوسی بازنمایی می‌کند. (برای جزئیاتِ بیشتر دربارهٔ مخلوطِ گاوسی‌ها بخش ۲۰.۳.۱ را ببینید.) این‌گونه لایه‌ها بسامدِ نسبیِ هر مؤلفهٔ مخلوط، میانگینِ هر مؤلفه و واریانسِ هر مؤلفه را پیش‌بینی می‌کنند. تا زمانی که این مقادیرِ خروجی توسط تابعِ زیان به‌درستی به‌عنوانِ تعریف‌کنندهٔ احتمالِ مقدارِ خروجیِ واقعیِ y تفسیر شوند، شبکه پس از آموزش، مدلِ مخلوطِ گاوسی را در فضایِ ویژگی‌های تعریف‌شده توسط لایه‌های پیشین برازش خواهد کرد.

۲۱.۲.۳ لایه‌های پنهان

در طولِ فرایندِ آموزش، شبکهٔ عصبی مقادیرِ ورودیِ x و مقادیرِ خروجیِ متناظرِ y بسیاری را می‌بیند. در هنگامِ پردازشِ بردارِ ورودیِ x، شبکهٔ عصبی پیش از تولیدِ خروجیِ y چند محاسبهٔ میانی انجام می‌دهد. می‌توانیم مقادیرِ محاسبه‌شده در هر لایهٔ شبکه را بازنماییِ متفاوتی از ورودیِ x در نظر بگیریم. هر لایه بازنماییِ تولیدشده توسط لایهٔ پیشین را تبدیل می‌کند تا بازنماییِ تازه‌ای تولید کند. ترکیبِ همهٔ این تبدیل‌ها موفق می‌شود—اگر همه‌چیز خوب پیش برود—ورودی را به خروجیِ مطلوب تبدیل کند. در واقع، یکی از فرضیه‌ها برای اینکه چرا یادگیری عمیق خوب کار می‌کند این است که تبدیلِ سرتاسریِ پیچیده‌ای که از ورودی به خروجی نگاشت می‌کند—مثلاً از تصویرِ ورودی به ردهٔ خروجیِ «زرافه»—توسط لایه‌های بسیار به ترکیبِ بسیاری از تبدیل‌های نسبتاً ساده تجزیه می‌شود، که یادگیریِ هر یک توسط فرایندِ به‌روزرسانیِ محلی نسبتاً آسان است.

در فرایندِ شکل‌دادن به همهٔ این تبدیل‌های درونی، شبکه‌های عمیق اغلب بازنمایی‌های میانیِ معناداری از داده کشف می‌کنند. برای نمونه، شبکه‌ای که یاد می‌گیرد اشیای پیچیده را در تصاویر بازشناسی کند ممکن است لایه‌های درونی‌ای تشکیل دهد که زیرواحدهای مفیدی را تشخیص می‌دهند: لبه‌ها، گوشه‌ها، بیضی‌ها، چشم‌ها، چهره‌ها—گربه‌ها. یا ممکن است چنین نکند—شبکه‌های عمیق ممکن است لایه‌های درونی‌ای تشکیل دهند که معنایشان برای انسان‌ها مبهم است، حتی اگر خروجی همچنان درست باشد.

لایه‌های پنهانِ شبکه‌های عصبی معمولاً از لایه‌های خروجی متنوع‌تر نیستند. در ۲۵ سالِ نخستِ پژوهش با شبکه‌های چندلایه (تقریباً 1985–2010)، گره‌های درونی تقریباً منحصراً از توابعِ فعال‌سازیِ سیگموئید و tanh استفاده می‌کردند. از حدودِ ۲۰۱۰ به بعد، ReLU و سافت‌پلاس محبوب‌تر شدند، تا حدی به‌دلیلِ این‌که گمان می‌رود از مسئلهٔ گرادیانِ ناپدیدشوندهٔ یادشده در بخش ۲۱.۱.۲ اجتناب می‌کنند. آزمایش با شبکه‌های به‌طورِ فزاینده‌ای عمیق نشان داد که در بسیاری از موارد، با تعدادِ کلِ ثابتی از وزن‌ها، یادگیریِ بهتر با شبکه‌های عمیق و نسبتاً باریک به‌دست می‌آید تا شبکه‌های کم‌عمق و پهن. نمونهٔ معمولی از این در شکل ۲۱.۷ در صفحهٔ ۷۶۹ نشان داده شده.

البته ساختارهای بسیار دیگری نیز برای گراف‌های محاسباتی قابلِ بررسی‌اند، به‌جزِ بازی‌کردن با پهنا و عمق. در زمانِ نوشتنِ این کتاب، شناختِ اندکی از این‌که چرا برخی ساختارها برای برخی مسائلِ خاص بهتر از بقیه کار می‌کنند وجود دارد. با تجربه، دست‌اندرکاران بینشِ شهودیِ برخی را برای طراحیِ شبکه‌ها و رفعِ مشکلِ آن‌ها وقتی کار نمی‌کنند به‌دست می‌آورند، همان‌گونه که سرآشپزها برای طراحیِ دستورهای آشپزی و رفعِ آن‌ها وقتی مزهٔ ناخوشایندی دارند بینش می‌یابند. به همین دلیل، ابزارهایی که کاوش و ارزیابیِ سریعِ ساختارهای متفاوت را آسان می‌کنند، برای موفقیت در مسائلِ جهانِ واقعی ضروری‌اند.

۲۱.۳ شبکه‌های پیچشی

در بخش ۲۱.۲.۱ اشاره کردیم که تصویر را نمی‌توان بردارِ ساده‌ای از مقادیرِ پیکسل‌ها در نظر گرفت، عمدتاً به‌دلیلِ این‌که مجاورتِ پیکسل‌ها واقعاً اهمیت دارد. اگر شبکه‌ای با لایه‌های کاملاً متصل و تصویری به‌عنوانِ ورودی می‌ساختیم، چه با تصاویرِ دست‌نخورده آموزش می‌دیدیم چه با تصاویری که همهٔ پیکسل‌هایشان به‌طورِ تصادفی درهم‌ریخته شده، همان نتیجه را می‌گرفتیم. افزون بر این، فرض کنید n پیکسل و n واحد در نخستین لایهٔ پنهان باشد، که پیکسل‌ها به آن‌ها ورودی می‌دهند. اگر ورودی و نخستین لایهٔ پنهان کاملاً متصل باشند، یعنی n^2 وزن؛ برای یک تصویرِ مگاپیکسلیِ RGB معمولی، این یعنی ۹ تریلیون وزن. چنین فضایِ پارامتریِ عظیمی به شمارِ به‌همان‌اندازهٔ عظیمِ تصاویرِ آموزشی و بودجهٔ محاسباتیِ هنگفتی برای اجرایِ الگوریتمِ آموزش نیاز داشت.

این ملاحظه‌ها پیشنهاد می‌کنند نخستین لایهٔ پنهان را چنان بسازیم که هر واحدِ پنهان ورودی را فقط از ناحیهٔ کوچکِ محلیِ تصویر دریافت کند. این با یک تیر دو نشان می‌زند. نخست، به مجاورت احترام می‌گذارد، دست‌کم به‌صورتِ محلی. (و بعداً خواهیم دید اگر لایه‌های بعدی نیز همان ویژگیِ محلی‌بودن را داشته باشند، شبکه به مجاورت به‌صورتِ سراسری احترام خواهد گذاشت.) دوم، تعدادِ وزن‌ها را کم می‌کند: اگر هر ناحیهٔ محلی $l \ll n$ پیکسل داشته باشد، در مجموع $ln \ll n^2$ وزن خواهد بود.

تا این‌جا خوب است. اما ویژگیِ مهمِ دیگری از تصاویر را از دست داده‌ایم: به‌طورِ تقریبی، هر چیزی که در یک ناحیهٔ کوچکِ محلی از تصویر قابلِ تشخیص باشد—شاید یک چشم یا یک تیغهٔ علف—اگر در ناحیهٔ کوچکِ محلیِ دیگری از تصویر ظاهر شود همان‌گونه به نظر می‌رسد. به بیانِ دیگر، انتظار داریم دادهٔ تصویری «ناوردایی فضایی» (Spatial Invariance) تقریبی را نشان دهد، دست‌کم در مقیاس‌های کوچک تا متوسط.[3] الزاماً انتظار نداریم نیمه‌های بالاییِ عکس‌ها شبیهِ نیمه‌های پایینی باشند، پس مقیاسی وجود دارد که فراتر از آن ناوردایی فضایی دیگر برقرار نیست.

ناوردایی فضاییِ محلی را می‌توان با مقیدکردنِ l وزنِ متصل‌کنندهٔ ناحیهٔ محلی به واحدی در لایهٔ پنهان به این‌صورت به‌دست آورد که برای هر واحدِ پنهان یکسان باشند. (یعنی برای واحدهای پنهانِ i و j، وزن‌های $w_{1,i}, \ldots, w_{l,i}$ همان $w_{1,j}, \ldots, w_{l,j}$ هستند.) این واحدهای پنهان را به «آشکارسازِ ویژگی» تبدیل می‌کند که همان ویژگی را هر جا در تصویر ظاهر شود تشخیص می‌دهند. معمولاً می‌خواهیم نخستین لایهٔ پنهان انواعِ بسیاری از ویژگی‌ها را تشخیص دهد، نه فقط یک ویژگی؛ پس برای هر ناحیهٔ محلیِ تصویر ممکن است d واحدِ پنهان با d مجموعه وزنِ متمایز داشته باشیم. این

بدان‌معناست که در مجموع dl وزن خواهد بود—عددی که نه‌تنها بسیار کوچک‌تر از n^2 است، بلکه در واقع مستقل از n، اندازهٔ تصویر، است. پس با تزریقِ دانشِ قبلیِ برخی—یعنی دانشِ مجاورت و ناوردایی فضایی—می‌توانیم مدل‌هایی توسعه دهیم که پارامترهای بسیار کمتری دارند و می‌توانند بسیار سریع‌تر یاد بگیرند.

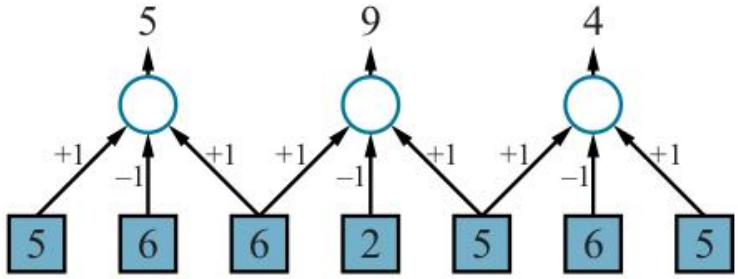
«شبکهٔ عصبیِ پیچشی» (Convolutional Neural Network یا CNN) شبکه‌ای است که اتصال‌های فضاییِ محلی دارد، دستِ‌کم در لایه‌های اولیه، و الگوهای وزنی‌ای دارد که در میانِ واحدهای هر لایه تکرار می‌شوند. الگوی وزنیِ تکرارشونده در چند ناحیهٔ محلی «هسته» (Kernel) نامیده می‌شود و فرایندِ اعمالِ هسته بر پیکسل‌های تصویر (یا بر واحدهای فضاییِ سازمان‌یافته در لایهٔ بعدی) «پیچش» (Convolution) نام دارد.[4]

هسته‌ها و پیچش‌ها در یک بعد آسان‌تر از دو یا چند بعد نشان داده می‌شوند، پس بردارِ ورودیِ x با اندازهٔ n را فرض می‌کنیم، متناظر با n پیکسل در تصویری تک‌بعدی، و هستهٔ برداریِ k با اندازهٔ l را. (برای سادگی فرض می‌کنیم l عددی فرد است.) همهٔ ایده‌ها به‌طورِ مستقیم به حالت‌های با ابعادِ بالاتر منتقل می‌شوند.

عملیاتِ پیچش را با نمادِ $*$ می‌نویسیم، مثلاً: $z = x * k$. این عملیات به‌صورتِ زیر تعریف می‌شود:

$$z_i = \sum_{j=1}^{l} k_j \, x_{j+i-(l+1)/2} \qquad (21.8)$$

به بیانِ دیگر، برای هر موقعیتِ خروجیِ i، ضربِ نقطه‌ای بینِ هستهٔ k و برشی از x به‌مرکزیتِ x_i با پهنای l می‌گیریم.



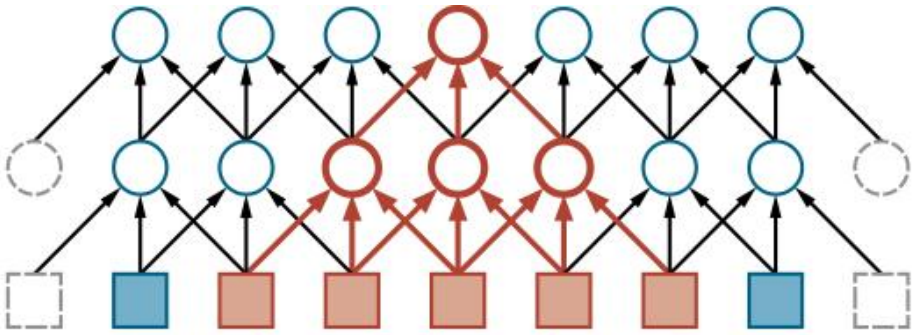
شکل ۲۱.۴ — نمونه‌ای از عملیاتِ پیچشِ یک‌بعدی با هسته‌ای به اندازهٔ l = 3 و گامی s = 2. پاسخِ اوج روی پیکسلِ ورودیِ تیره‌تر (با شدتِ کمتر) متمرکز است. نتیجه‌ها معمولاً پیش از رفتن به لایهٔ پنهانِ بعدی از یک تابعِ فعال‌سازیِ غیرخطی (نشان‌داده‌نشده) عبور داده می‌شوند.

فرایند در شکل ۲۱.۴ برای هستهٔ برداریِ [+1, −1, +1] نشان داده شده، که نقطه‌ای تیره‌تر را در تصویرِ یک‌بعدی تشخیص می‌دهد. (نسخهٔ دوبعدی ممکن است خطی تیره‌تر را تشخیص دهد.) توجه کنید در این مثال، پیکسل‌هایی که هسته‌ها روی آن‌ها متمرکزند با فاصلهٔ ۲ پیکسل از هم جدا شده‌اند؛ می‌گوییم هسته با «گامِ» (Stride) s = 2 اعمال می‌شود. توجه کنید لایهٔ خروجی پیکسل‌های کمتری دارد: به‌دلیلِ گام، تعدادِ پیکسل‌ها از n به تقریباً n/s کاهش می‌یابد. (در دو بعد، تعدادِ پیکسل‌ها تقریباً $n/(s_x s_y)$ خواهد بود، که s_x و s_y گام‌ها در جهت‌های x و y تصویرند.) «تقریباً» می‌گوییم به‌دلیلِ آنچه در لبهٔ تصویر رخ می‌دهد: در شکل ۲۱.۴ پیچش در لبه‌های تصویر می‌ایستد، اما می‌توان ورودی را با پیکسل‌های اضافی (یا صفرها یا کپیِ پیکسل‌های بیرونی) لایه‌گذاری (Padding) کرد تا هسته بتواند دقیقاً ⌊n/s⌋ بار اعمال شود. برای هسته‌های کوچک، معمولاً از s = 1 استفاده می‌کنیم، پس خروجی همان ابعادِ تصویر را دارد (شکل ۲۱.۵ را ببینید).

عملیاتِ اعمالِ هسته روی تصویر را می‌توان به‌شیوهٔ آشکار با برنامه‌ای با حلقه‌های تو در توی مناسب پیاده‌سازی کرد؛ اما می‌توان آن را به‌صورتِ یک عملیاتِ ماتریسیِ واحد نیز صورتبندی کرد، درست مانندِ اعمالِ ماتریسِ وزن در معادلهٔ (21.1). برای نمونه، پیچشِ نشان‌داده‌شده در شکل ۲۱.۴ را می‌توان به‌عنوانِ ضربِ ماتریسیِ زیر دید:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & +1 & -1 & +1 \\ 0 & 0 & +1 & -1 & +1 & 0 & 0 \\ 0 & +1 & -1 & +1 & 0 & 0 & 0 \\ +1 & -1 & +1 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 5 & 6 & 5 & 2 & 6 & 6 & 5 \end{bmatrix} = \begin{bmatrix} 5 & 4 & 9 & 5 \end{bmatrix} \tag{21.9}$$

در این ماتریسِ وزن، هسته در هر سطر ظاهر می‌شود، و طبقِ گام نسبت به سطرِ قبلی جابه‌جا شده. لزومی ندارد کسی ماتریسِ وزن را به‌صورتِ صریح بسازد—بالاخره بیشترِ آن صفر است—اما این واقعیت که پیچش یک عملیاتِ ماتریسیِ خطی است یادآور این است که نزولِ گرادیان به‌آسانی و به‌طورِ مؤثر بر CNNها اعمال می‌شود، درست مانندِ شبکه‌های عصبیِ ساده. همان‌گونه که پیش‌تر اشاره شد، d هسته وجود خواهد داشت، نه فقط یکی؛ پس با گامِ ۱، خروجی d برابر بزرگ‌تر خواهد بود. این بدان‌معناست که آرایهٔ ورودیِ دوبعدی به آرایهٔ سه‌بعدی از واحدهای پنهان تبدیل می‌شود، که بعدِ سومش اندازهٔ d دارد. سازمان‌دادنِ لایهٔ پنهان به این‌شکل مهم است، تا همهٔ خروجی‌های هسته از یک موقعیتِ خاصِ تصویر با همان موقعیت همراه بمانند. برخلافِ ابعادِ فضاییِ تصویر، اما، این «بعدِ هسته»یِ اضافی هیچ ویژگیِ مجاورتی ندارد، پس اجرایِ پیچش در امتدادِ آن معنا ندارد.



شکل ۲۱.۵ — دو لایهٔ نخستِ یک CNN برای تصویرِ یک‌بعدی با اندازهٔ هستهٔ l = 3 و گامِ s = 1. در انتهایِ چپ و راست لایه‌گذاری اضافه شده تا لایه‌های پنهان هم‌اندازهٔ ورودی بمانند. با رنگِ قرمز «میدانِ دریافتی»ِ واحدی در دومین لایهٔ پنهان نشان داده شده. به‌طورِ کلی، هرچه واحد عمیق‌تر باشد، میدانِ دریافتی بزرگ‌تر است.

CNNها در اصل از مدل‌های قشرِ بینایی پیشنهادی در عصب‌شناسی الهام گرفته شده‌اند. در آن مدل‌ها، «میدانِ دریافتی» (Receptive Field) یک نورون بخشی از ورودیِ حسی است که می‌تواند بر فعال‌سازیِ آن نورون اثر بگذارد. در CNN، میدانِ دریافتیِ واحدی در نخستین لایهٔ پنهان کوچک است—فقط به اندازهٔ هسته، یعنی l پیکسل. در لایه‌های عمیق‌ترِ شبکه، می‌تواند بسیار بزرگ‌تر باشد. شکل ۲۱.۵ این را برای واحدی در دومین لایهٔ پنهان نشان می‌دهد که میدانِ دریافتی‌اش پنج پیکسل دارد. وقتی گام ۱ باشد، مانندِ شکل، گره‌ای در mاُمین لایهٔ پنهان میدانِ دریافتیِ با اندازهٔ m+1(l-1) خواهد داشت؛ پس رشد نسبت به m خطی است. (در تصویرِ دوبعدی، هر بعدِ میدانِ دریافتی به‌صورتِ خطی با m رشد می‌کند، پس مساحت به‌صورتِ درجهٔ دوم رشد می‌کند.) وقتی گام بزرگ‌تر از ۱ باشد، هر پیکسل در لایهٔ m نمایندهٔ s پیکسل در لایهٔ m-1 است؛ بنابراین، میدانِ دریافتی به‌صورتِ $O(ls^m)$—یعنی به‌صورتِ نمایی با عمق—رشد می‌کند. همان اثر با لایه‌های تجمیع رخ می‌دهد، که در ادامه بحث می‌کنیم.

۲۱.۳.۱ تجمیع و پایین‌نمونه‌برداری

لایهٔ «تجمیع» (Pooling) در شبکهٔ عصبی مجموعه‌ای از واحدهای مجاور از لایهٔ پیشین را با یک مقدارِ واحد خلاصه می‌کند. تجمیع دقیقاً مانند لایهٔ پیچش کار می‌کند، با اندازهٔ هستهٔ l و گامِ s، اما عملیاتی که اعمال می‌شود ثابت است نه آموخته‌شده. معمولاً هیچ تابعِ فعال‌سازی با لایهٔ تجمیع همراه نیست. دو شکلِ رایجِ تجمیع هست:

- «تجمیعِ میانگین» (Average-Pooling) میانگینِ l ورودیِ خود را محاسبه می‌کند. این با پیچش با هستهٔ یکنواختِ k = [1/l, ..., 1/l] یکسان است. اگر l = s بگذاریم، اثر، درشت‌کردنِ وضوحِ تصویر است—یعنی «پایین‌نمونه‌برداری» (Downsampling) آن—به‌ضریبِ s. شیئی که مثلاً 10s پیکسل را اشغال می‌کرد پس از تجمیع فقط 10 پیکسل را اشغال می‌کرد. همان رده‌بندِ آموخته‌شده‌ای که می‌توانست شیء را در اندازهٔ 10 پیکسل در تصویرِ اصلی بازشناسی کند، حالا می‌تواند آن شیء را در تصویرِ تجمیع‌شده بازشناسی کند، حتی اگر برای بازشناسی در تصویرِ اصلی بیش‌ازحد بزرگ بود. به بیانِ دیگر، تجمیعِ میانگین بازشناسیِ چندمقیاسی را تسهیل می‌کند. همچنین تعدادِ وزن‌های موردنیاز در لایه‌های بعدی را کاهش می‌دهد و به هزینهٔ محاسباتیِ کمتر و احتمالاً یادگیریِ سریع‌تر می‌انجامد.
- «تجمیعِ حداکثر» (Max-Pooling) مقدارِ بیشینهٔ l ورودیِ خود را محاسبه می‌کند. از آن می‌توان صرفاً برای پایین‌نمونه‌برداری نیز استفاده کرد، اما معناشناسیِ تا حدی متفاوتی دارد. فرض کنید تجمیعِ حداکثر را روی لایهٔ پنهانِ [5, 9, 4] در شکل ۲۱.۴ اعمال می‌کنیم: نتیجه ۹ خواهد بود، نشان‌دهندهٔ این‌که جایی در تصویرِ ورودی نقطهٔ تیره‌تری هست که هسته آن را تشخیص می‌دهد. به بیانِ دیگر، تجمیعِ حداکثر به‌عنوانِ نوعی فصلی که می‌گوید یک ویژگی جایی در میدانِ دریافتیِ واحد وجود دارد عمل می‌کند.

اگر هدف، رده‌بندیِ تصویر به یکی از c رده باشد، لایهٔ پایانیِ شبکه سافتمکسی با c واحدِ خروجی خواهد بود. لایه‌های اولیهٔ CNN به اندازهٔ تصویرند، پس جایی در میانه باید کاهش‌های چشمگیری در اندازهٔ لایه رخ دهد. لایه‌های پیچش و لایه‌های تجمیع با گامِ بزرگ‌تر از ۱ همگی به کاهشِ اندازهٔ لایه کمک می‌کنند. همچنین کاهشِ اندازهٔ لایه صرفاً با داشتنِ لایه‌ای کاملاً متصل با واحدهایی کمتر از لایهٔ پیشین ممکن است. CNNها اغلب یک یا دو چنین لایه‌ای پیش از لایهٔ سافتمکسِ پایانی دارند.

۲۱.۳.۲ عملیاتِ تانسوری در CNNها

در معادله‌های (21.1) و (21.3) دیدیم استفاده از نمادگذاریِ برداری و ماتریسی می‌تواند در ساده و زیبا نگه‌داشتنِ اشتقاق‌های ریاضی و فراهم‌کردنِ توصیفِ موجزِ گراف‌های محاسباتی کمک‌کننده باشد. بردارها و ماتریس‌ها حالت‌های خاصِ یک‌بعدی و دوبعدیِ «تانسورها» (Tensor) هستند، که (در اصطلاح‌شناسیِ یادگیری عمیق) صرفاً آرایه‌های چندبعدیِ با هر بعدی‌اند.[5] برای CNNها، تانسورها راهی برای ردیابیِ «شکل» داده در حینِ پیشرفتِ آن از میانِ لایه‌های شبکه‌اند. این مهم است، زیرا کلِ مفهومِ پیچش به ایدهٔ مجاورت وابسته است: فرض می‌شود عناصرِ دادهٔ مجاور از نظرِ معنایی مرتبط‌اند، پس اعمالِ عملگرها بر ناحیه‌های محلیِ داده معنا دارد. افزون بر این، با اجزایِ زبانِ مناسب برای ساختِ تانسورها و اعمالِ عملگرها، خودِ لایه‌ها را می‌توان به‌طورِ موجز به‌عنوانِ نگاشت‌هایی از ورودی‌های تانسوری به خروجی‌های تانسوری توصیف کرد.

دلیلِ نهاییِ توصیفِ CNNها بر حسبِ عملیاتِ تانسوری کاراییِ محاسباتی است: با توجه به توصیفی از شبکه به‌صورتِ دنباله‌ای از عملیاتِ تانسوری، یک بستهٔ نرم‌افزاریِ یادگیری عمیق می‌تواند کدِ کامپایل‌شده‌ای تولید کند که برای بسترِ محاسباتیِ زیرین به‌شدت بهینه شده. بارهای کاریِ یادگیری عمیق اغلب روی GPUها (واحدهای پردازشِ گرافیکی) یا TPUها (واحدهای پردازشِ تانسوری) اجرا می‌شوند، که درجهٔ بالایی از موازی‌سازی فراهم می‌کنند. برای نمونه، یکی از خوشه‌های TPU نسلِ سومِ گوگل توانِ گذردهی معادلِ حدودِ ده میلیون لپ‌تاپ دارد. بهره‌برداری از این قابلیت‌ها اگر بخواهیم CNN بزرگی را روی پایگاهِ دادهٔ بزرگی از تصاویر آموزش دهیم ضروری است. پس رایج است که نه یک تصویرِ تنها در هر بار، بلکه بسیاری تصویر به‌طورِ موازی پردازش شوند؛ چنان‌که در بخش ۲۱.۴ خواهیم دید، این با شیوه‌ای که الگوریتمِ نزولِ گرادیانِ تصادفی گرادیان‌ها را نسبت به یک «مینی‌دسته» (Minibatch) از مثال‌های آموزشی محاسبه می‌کند نیز به‌خوبی هماهنگ است.

همهٔ این‌ها را در قالبِ مثالی کنارِ هم بگذاریم. فرض کنید روی تصاویرِ RGB با اندازهٔ 256×256 با اندازهٔ مینی‌دستهٔ 64 آموزش می‌بینیم. ورودی در این حالت تانسوری چهاربعدی با اندازهٔ 256×256×3×64 خواهد بود. سپس 96 هستهٔ با اندازهٔ 5×5×3 با گامِ ۲ در هر دو جهتِ x و y تصویر اعمال می‌کنیم. این تانسورِ خروجیِ با اندازهٔ 128×128×96×64 می‌دهد. چنین تانسوری اغلب «نقشهٔ ویژگی» (Feature Map) نامیده می‌شود، زیرا نشان می‌دهد هر ویژگیِ استخراج‌شده توسط یک هسته چگونه در سراسرِ تصویر ظاهر می‌شود؛ در این مورد از 96 «کانال» (Channel) تشکیل شده، که هر کانال اطلاعاتِ یک ویژگی را حمل می‌کند. توجه کنید برخلافِ تانسورِ ورودی، این نقشهٔ ویژگی دیگر کانال‌های رنگیِ اختصاصی ندارد؛ با این حال، اگر الگوریتمِ یادگیری رنگ را برای پیش‌بینی‌های نهاییِ شبکه مفید بیابد، اطلاعاتِ رنگی ممکن است همچنان در کانال‌های ویژگیِ گوناگون حاضر باشد.

۲۱.۳.۳ شبکه‌های باقی‌مانده

«شبکه‌های باقی‌مانده» (Residual Network) رویکردی محبوب و موفق برای ساختنِ شبکه‌های بسیار عمیق‌اند که از مسئلهٔ گرادیانِ ناپدیدشونده اجتناب می‌کنند.

مدل‌های عمیقِ معمولی لایه‌هایی به‌کار می‌برند که بازنماییِ تازه‌ای در لایهٔ i یاد می‌گیرند و بازنماییِ لایهٔ i-1 را کاملاً جایگزین می‌کنند. با نمادگذاریِ ماتریس-بردارِ معرفی‌شده در معادلهٔ (21.3)، که $z^{(i)}$ مقادیرِ واحدهای لایهٔ i است، داریم:

$$z^{(i)} = f(z^{(i-1)}) = g^{(i)}(W^{(i)} z^{(i-1)})$$

چون هر لایه بازنماییِ لایهٔ پیشین را کاملاً جایگزین می‌کند، همهٔ لایه‌ها باید یاد بگیرند کارِ مفیدی انجام دهند. هر لایه باید، دست‌کم، اطلاعاتِ مربوط به تکلیفِ موجود در لایهٔ پیشین را حفظ کند. اگر برای هر لایهٔ i مقدارِ $W^{(i)} = 0$ بگذاریم، کلِ شبکه از کار می‌ایستد. اگر همچنین $W^{(i-1)} = 0$ بگذاریم، شبکه حتی قادر به یادگیری هم نخواهد بود: لایهٔ i یاد نخواهد گرفت، زیرا هیچ تغییری در ورودیِ خود از لایهٔ i-1 مشاهده نمی‌کند، و لایهٔ i-1 یاد نخواهد گرفت، زیرا گرادیانِ پسانتشارشده از لایهٔ i همیشه صفر خواهد بود. البته این‌ها مثال‌های افراطی‌اند، اما نیازِ لایه‌ها به این‌که مجراهایی برای سیگنال‌های عبوری از شبکه باشند را نشان می‌دهند.

ایدهٔ کلیدیِ شبکه‌های باقی‌مانده این است که یک لایه باید بازنماییِ لایهٔ قبلی را مختل کند نه اینکه آن را کاملاً جایگزین کند. اگر اختلالِ آموخته‌شده کوچک باشد، لایهٔ بعدی نزدیکِ کپیِ لایهٔ قبلی است. این با معادلهٔ زیر برای لایهٔ i بر حسبِ لایهٔ i-1 به‌دست می‌آید:

$$z^{(i)} = g^{(i)}_r(z^{(i-1)} + f(z^{(i-1)})) \qquad (21.10)$$

که در آن g_r توابعِ فعال‌سازیِ لایهٔ باقی‌مانده را نشان می‌دهد. در این‌جا f را به‌عنوانِ «باقی‌مانده» (Residual) در نظر می‌گیریم، که رفتارِ پیش‌فرضِ عبورِ لایهٔ i-1 به لایهٔ i را مختل می‌کند. تابعی که برای محاسبهٔ باقی‌مانده استفاده می‌شود معمولاً شبکهٔ عصبی با یک لایهٔ غیرخطیِ ترکیب‌شده با یک لایهٔ خطی است: $f(z) = Vg(Wz)$، که در آن W و V ماتریس‌های وزنِ آموخته‌شده با وزن‌های بایاسِ معمولیِ افزوده‌شده‌اند.

$$z^{(i)} = g_r(z^{(i-1)}) = \text{ReLU}(z^{(i-1)}) = \text{ReLU}(\text{ReLU}(in^{(i-1)})) = \text{ReLU}(in^{(i-1)}) = z^{(i-1)}$$

که گامِ ماقبل‌آخر به‌دلیلِ $\text{ReLU}(\text{ReLU}(x)) = \text{ReLU}(x)$ برقرار است. به بیانِ دیگر، در شبکه‌های باقی‌مانده با فعال‌سازی‌های ReLU، لایه‌ای با وزن‌های صفر صرفاً ورودی‌هایش را بدونِ تغییر عبور می‌دهد. بقیهٔ شبکه درست همان‌گونه کار می‌کند که گویی آن لایه هرگز وجود نداشته. در حالی که شبکه‌های سنتی باید انتشارِ اطلاعات را یاد بگیرند و برای انتخاب‌های بدِ پارامترها در معرضِ شکستِ فاجعه‌بارِ انتشارِ اطلاعات‌اند، شبکه‌های باقی‌مانده اطلاعات را به‌صورتِ پیش‌فرض منتشر می‌کنند.

شبکه‌های باقی‌مانده اغلب با لایه‌های پیچشی در کاربردهای بینایی استفاده می‌شوند، اما در واقع ابزاری کاربردِ کلی‌اند که شبکه‌های عمیق را مقاوم‌تر می‌کنند و به پژوهشگران اجازه می‌دهند آزادانه‌تر با طراحی‌های شبکهٔ پیچیده و ناهمگن آزمایش کنند. در زمانِ نوشتنِ این کتاب، دیدنِ شبکه‌های باقی‌مانده با صدها لایه غیرعادی نیست. طراحیِ این‌گونه شبکه‌ها به‌سرعت در حالِ تکامل است، پس هر جزئیاتِ اضافی‌ای که ارائه دهیم احتمالاً پیش از رسیدنِ کتاب به چاپ منسوخ می‌شود. خوانندگانی که می‌خواهند بهترین معماری‌ها را برای کاربردهای خاص بدانند باید به انتشاراتِ پژوهشیِ اخیر مراجعه کنند.

۲۱.۴ الگوریتم‌های یادگیری

آموزشِ شبکهٔ عصبی شاملِ اصلاحِ پارامترهای شبکه برای کمینه‌کردنِ تابعِ زیان روی مجموعهٔ آموزشی است. در اصل، هر نوع الگوریتمِ بهینه‌سازی‌ای می‌تواند استفاده شود. در عمل، شبکه‌های عصبیِ مدرن تقریباً همیشه با گونه‌ای از «نزولِ گرادیانِ تصادفی» (Stochastic Gradient Descent یا SGD) آموزش می‌بینند.

نزولِ گرادیانِ استاندارد و نسخهٔ تصادفی‌اش را در بخش ۱۹.۶.۲ پوشش دادیم. در این‌جا، هدف کمینه‌کردنِ زیانِ $L(w)$ است، که w همهٔ پارامترهای شبکه را بازنمایی می‌کند. هر گامِ به‌روزرسانی در فرایندِ نزولِ گرادیان چنین است:

```
w ← w − α ∇_w L(w)
```

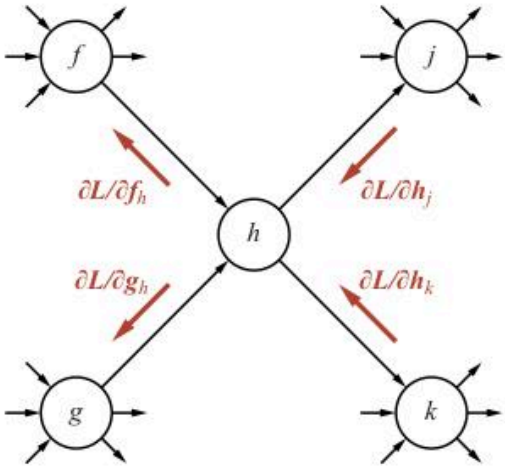
که در آن α نرخِ یادگیری است. برای نزولِ گرادیانِ استاندارد، زیانِ L نسبت به کلِ مجموعهٔ آموزشی تعریف می‌شود. برای SGD، نسبت به مینی‌دسته‌ای از m مثال که در هر گام به‌طورِ تصادفی انتخاب می‌شوند تعریف می‌شود. چنان‌که در بخش ۴.۲ اشاره شد، ادبیاتِ روش‌های بهینه‌سازی برای فضاهای پیوستهٔ پربعد شاملِ بهبودهای بی‌شمارِ نزولِ گرادیانِ پایه است. همهٔ آن‌ها را در این‌جا پوشش نمی‌دهیم، اما ذکرِ چند ملاحظهٔ مهم که مخصوصاً برای آموزشِ شبکه‌های عصبی مرتبط‌اند ارزشمند است:

- برای بیشترِ شبکه‌هایی که مسائلِ جهانِ واقعی را حل می‌کنند، هم بُعدِ w و هم اندازهٔ مجموعهٔ آموزشی بسیار بزرگ‌اند. این ملاحظه‌ها به‌شدت به نفعِ استفاده از SGD با اندازهٔ مینی‌دستهٔ نسبتاً کوچکِ m است: تصادفی‌بودن به الگوریتم کمک می‌کند از کمینه‌های محلیِ کوچک در فضایِ وزنِ پربعد بگریزد (مانندِ بازپختِ شبیه‌سازی‌شده—صفحهٔ ۱۱۴ را ببینید)؛ و اندازهٔ کوچکِ مینی‌دسته تضمین می‌کند هزینهٔ محاسباتیِ هر گامِ به‌روزرسانیِ وزن ثابتِ کوچکی باشد، مستقل از اندازهٔ مجموعهٔ آموزشی.
- چون سهمِ گرادیانِ هر مثالِ آموزشی در مینی‌دستهٔ SGD را می‌توان به‌طورِ مستقل محاسبه کرد، اندازهٔ مینی‌دسته اغلب چنان انتخاب می‌شود که حداکثرِ بهره را از موازی‌سازیِ سخت‌افزاری در GPUها یا TPUها ببرد.
- برای بهبودِ همگرایی، معمولاً ایدهٔ خوبی است از نرخِ یادگیری‌ای استفاده کنیم که در طولِ زمان کاهش می‌یابد. انتخابِ برنامهٔ زمانیِ درست معمولاً موضوعِ آزمون و خطاست.
- نزدیکِ کمینهٔ محلی یا سراسریِ تابعِ زیان نسبت به کلِ مجموعهٔ آموزشی، گرادیان‌های برآوردشده از مینی‌دسته‌های کوچک اغلب واریانسِ بالایی دارند و ممکن است در جهتِ کاملاً اشتباه اشاره کنند، و همگرایی را دشوار سازند. یک راه‌حل افزایشِ اندازهٔ مینی‌دسته در جریانِ آموزش است؛ راهِ دیگر گنجاندنِ ایدهٔ «تکانه» (Momentum) است، که میانگینِ متحرکی از گرادیان‌های مینی‌دسته‌های گذشته نگه می‌دارد تا اندازهٔ کوچکِ مینی‌دسته جبران شود.
- باید مراقبِ تعدیلِ بی‌ثباتی‌های عددی‌ای بود که ممکن است از سرریز، زیرریز و خطایِ گردکردن ناشی شوند. این‌ها به‌ویژه با استفاده از نمایی‌ها در توابعِ فعال‌سازیِ سافتمکس، سیگموئید و tanh، و با محاسباتِ تکرارشونده در شبکه‌های بسیار عمیق و شبکه‌های بازگشتی (بخش ۲۱.۶) که به فعال‌سازی‌ها و گرادیان‌های ناپدیدشونده و منفجرشونده می‌انجامند، مشکل‌سازند.

در مجموع، فرایندِ یادگیریِ وزن‌های شبکه معمولاً فرایندی است که بازدهیِ کاهنده نشان می‌دهد. آن‌قدر اجرا می‌کنیم تا دیگر کاهشِ خطایِ آزمون با ادامهٔ اجرا عملی نباشد. معمولاً این به آن معنا نیست که به کمینهٔ سراسری یا حتی محلیِ تابعِ زیان رسیده‌ایم. در عوض، به آن معناست که برای ادامهٔ کاهشِ هزینه باید تعدادِ غیرعملیِ بزرگی گامِ بسیار کوچک برداریم، یا این‌که گام‌های اضافی فقط به بیش‌برازش می‌انجامند، یا این‌که برآوردهای گرادیان برای پیشرفتِ بیشتر بیش‌ازحد نادقیق‌اند.

۲۱.۴.۱ محاسبهٔ گرادیان‌ها در گراف‌های محاسباتی

در صفحهٔ ۷۵۵، گرادیانِ تابعِ زیان را نسبت به وزن‌ها در شبکه‌ای خاص (و بسیار ساده) اشتقاق دادیم. مشاهده کردیم گرادیان را می‌توان با پسانتشارِ اطلاعاتِ خطا از لایهٔ خروجیِ شبکه به لایه‌های پنهان محاسبه کرد. همچنین گفتیم این نتیجه برای هر گرافِ محاسباتیِ پیشخوری به‌طورِ کلی برقرار است. در این‌جا توضیح می‌دهیم این چگونه کار می‌کند.



شکل ۲۱.۶ – تصویری از پسانتشارِ اطلاعاتِ گرادیان در گرافِ محاسباتیِ دلبخواه. محاسبهٔ پیشروِ خروجیِ شبکه از چپ به راست پیش می‌رود، در حالی که پسانتشارِ گرادیان‌ها از راست به چپ.

شکل ۲۱.۶ گره‌ای عمومی در گرافِ محاسباتی را نشان می‌دهد. (گرهٔ h درجهٔ ورودی و خروجیِ ۲ دارد، اما هیچ‌چیز در این تحلیل به این وابسته نیست.) در طولِ گذرِ پیشرو، گره تابعِ دلبخواهیِ h را از ورودی‌هایش محاسبه می‌کند، که از گره‌های f و g می‌آیند. به‌نوبهٔ خود، h مقدارش را به گره‌های j و k می‌دهد.

فرایندِ پسانتشار پیام‌ها را در طولِ هر پیوندِ شبکه به عقب می‌فرستد. در هر گره، پیام‌های ورودی جمع‌آوری و پیام‌های تازه‌ای برای بازفرستادن به لایهٔ بعد محاسبه می‌شوند. چنان‌که شکل نشان می‌دهد، پیام‌ها همگی مشتق‌های جزئیِ زیانِ Lاند. برای نمونه، پیامِ عقب‌روِ ∂L/∂h_j مشتقِ جزئیِ L نسبت به نخستین ورودیِ j است، که همان پیامِ پیشرو از h به j است. حالا، h از طریقِ هر دوِ j و k بر L اثر می‌گذارد، پس داریم:

$$\partial L/\partial h = \partial L/\partial h_j + \partial L/\partial h_k \qquad (21.11)$$

با این معادله، گرهٔ h می‌تواند مشتقِ L نسبت به h را با جمع‌کردنِ پیام‌های ورودی از j و k محاسبه کند. حالا، برای محاسبهٔ پیام‌های خروجیِ ∂L/∂f_h و ∂L/∂g_h، از معادله‌های زیر استفاده می‌کنیم:

$$\partial L/\partial f_h = (\partial L/\partial h)(\partial h/\partial f_h)$$
$$\partial L/\partial g_h = (\partial L/\partial h)(\partial h/\partial g_h) \qquad (21.12)$$

در معادلهٔ (21.12)، $\partial L/\partial h$ قبلاً توسط معادلهٔ (21.11) محاسبه شده، و $\partial h/\partial f_h$ و $\partial h/\partial g_h$ صرفاً مشتق‌های h نسبت به آرگومانِ نخست و دومش‌اند. برای نمونه، اگر h گرهٔ ضرب باشد—یعنی $h(f, g) = f \cdot g$—آنگاه $\partial h/\partial f_h = g$ و $\partial h/\partial g_h = f$. بسته‌های نرم‌افزاریِ یادگیری عمیق معمولاً با کتابخانه‌ای از انواعِ گره (جمع، ضرب، سیگموئید و مانندِ آن) عرضه می‌شوند، که هر کدام می‌داند مشتق‌های خودش را هرگاه برای معادلهٔ (21.12) لازم باشد محاسبه کند.

فرایندِ پسانتشار با گره‌های خروجی آغاز می‌شود، جایی که هر پیامِ آغازینِ $\partial L/\partial \hat{y}_j$ مستقیماً از عبارتِ L بر حسبِ مقدارِ پیش‌بینی‌شدهٔ $\hat{y}$ و مقدارِ واقعیِ y از دادهٔ آموزشی محاسبه می‌شود. در هر گرهٔ درونی، پیام‌های عقب‌روِ ورودی طبقِ معادلهٔ (21.11) جمع و پیام‌های خروجی از معادلهٔ (21.12) تولید می‌شوند. فرایند در هر گره از گرافِ محاسباتی که وزنِ w را بازنمایی می‌کند (مثلاً بیضی‌های بنفشِ روشن در شکل ۲۱.۳ (ب)) خاتمه می‌یابد. در آن نقطه، جمعِ پیام‌های ورودی به w برابرِ $\partial L/\partial w$ است—دقیقاً گرادیانی که برای به‌روزرسانیِ w نیاز داریم.

تمرین BPRE.۲۱ از شما می‌خواهد این فرایند را بر شبکهٔ سادهٔ شکل ۲۱.۳ اعمال کنید تا عبارت‌های گرادیانِ معادله‌های (21.4) و (21.5) را بازاشتقاق کنید.

«اشتراکِ وزن»، چنان‌که در شبکه‌های پیچشی (بخش ۲۱.۳) و شبکه‌های بازگشتی (بخش ۲۱.۶) استفاده می‌شود، صرفاً با درنظرگرفتنِ هر وزنِ مشترک به‌عنوانِ گره‌ای واحد با چند کمانِ خروجی در گرافِ محاسباتی مدیریت می‌شود. در طولِ پسانتشار، این به پیام‌های گرادیانیِ چندگانهٔ ورودی می‌انجامد. طبقِ معادلهٔ (21.11)، این بدان‌معناست که گرادیانِ وزنِ مشترک جمعِ سهم‌های گرادیان از هر جایی است که در شبکه استفاده می‌شود.

از این توصیفِ فرایندِ پسانتشار روشن است هزینهٔ محاسباتی‌اش در تعدادِ گره‌های گرافِ محاسباتی خطی است، درست مانندِ هزینهٔ محاسبهٔ پیشرو. افزون بر این، چون انواعِ گره معمولاً هنگامِ طراحیِ شبکه ثابت‌اند، همهٔ محاسباتِ گرادیان را می‌توان از پیش به‌صورتِ نمادین آماده کرد و برای هر گرهٔ گراف به کدِ بسیار کارآمدی کامپایل کرد. همچنین توجه کنید پیام‌های شکل ۲۱.۶ لازم نیست اسکالر باشند: می‌توانند به‌یکسان بردار، ماتریس یا تانسورهای با ابعادِ بالاتر باشند، تا محاسباتِ گرادیان بتوانند برای بهره‌گیری از موازی‌سازی روی GPUها یا TPUها نگاشت شوند. یکی از کاستی‌های پسانتشار این است که برای محاسبهٔ گرادیان‌ها در گذرِ عقب‌رو نیاز به ذخیره‌کردنِ بیشترِ مقادیرِ میانیِ محاسبه‌شده در طولِ انتشارِ پیشرو دارد. این بدان‌معناست که هزینهٔ کلِ حافظهٔ آموزشِ شبکه متناسب با تعدادِ واحدهای کلِ شبکه است. پس، حتی اگر خودِ شبکه فقط به‌صورتِ ضمنی توسط کدِ انتشار با حلقه‌های بسیار بازنمایی شود، نه به‌صورتِ صریح با ساختارِ داده‌ای، همهٔ نتایجِ میانیِ آن کدِ انتشار باید به‌صورتِ صریح ذخیره شوند.

۲۱.۴.۲ نرمال‌سازیِ دسته‌ای

«نرمال‌سازیِ دسته‌ای» (Batch Normalization) فنّ پرکاربردی است که نرخِ همگراییِ SGD را با بازمقیاس‌کردنِ مقادیرِ تولیدشده در لایه‌های درونیِ شبکه از مثال‌های درونِ هر مینی‌دسته بهبود می‌بخشد. هرچند دلایلِ اثربخشیِ آن در زمانِ نوشتنِ این کتاب به‌خوبی شناخته نشده، آن را گنجانده‌ایم، زیرا در عمل مزیت‌های چشمگیری به ارمغان می‌آورد. تا حدی، به نظر می‌رسد نرمال‌سازیِ دسته‌ای اثرهایی شبیهِ شبکهٔ باقی‌مانده دارد.

گره‌ای مانندِ z را جایی در شبکه در نظر بگیرید: مقادیرِ z برای m مثالِ یک مینی‌دسته $z_1, \ldots, z_m$ هستند. نرمال‌سازیِ دسته‌ای هر z_i را با کمیتِ تازهٔ $\hat{z}_i$ جایگزین می‌کند:

$$\hat{z}_i = \gamma \, (z_i - \mu) \, / \, \sqrt{(\varepsilon + \sigma^2)} + \beta$$

که در آن μ میانگینِ z در مینی‌دسته، σ انحرافِ معیارِ $z_1, \ldots, z_m$، ε ثابتِ کوچکی برای جلوگیری از تقسیم بر صفر، و γ و β پارامترهای آموخته‌شده‌اند.

نرمال‌سازیِ دسته‌ای میانگین و واریانسِ مقادیر را استاندارد می‌کند، چنان‌که توسط مقادیرِ β و γ تعیین می‌شود. این آموزشِ شبکهٔ عمیق را بسیار ساده‌تر می‌کند. بدونِ نرمال‌سازیِ دسته‌ای، اگر وزن‌های یک لایه خیلی کوچک باشند اطلاعات می‌تواند گم شود و انحرافِ معیار در آن لایه به نزدیکیِ صفر تنزل کند. نرمال‌سازیِ دسته‌ای از رخدادنِ این جلوگیری می‌کند. همچنین نیاز به مقداردهیِ اولیهٔ دقیقِ همهٔ وزن‌های شبکه را کاهش می‌دهد تا مطمئن شویم گره‌های هر لایه در ناحیهٔ عملکردِ درست قرار دارند و اجازهٔ انتشارِ اطلاعات را می‌دهند.

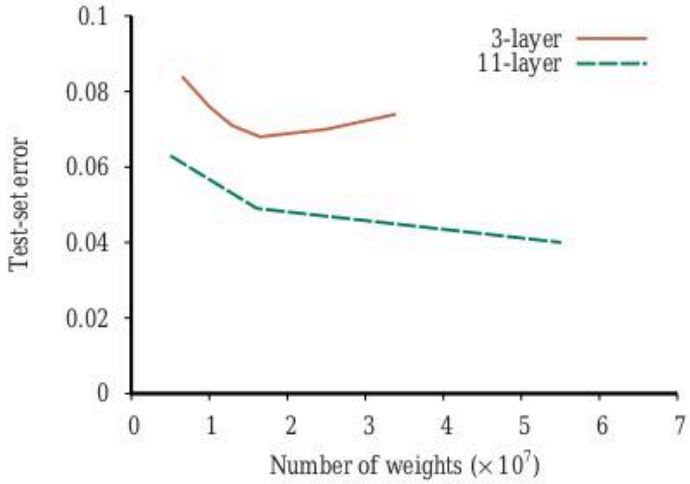
با نرمال‌سازیِ دسته‌ای، معمولاً β و γ را، که ممکن است گره‌خاص یا لایه‌خاص باشند، در میانِ پارامترهای شبکه می‌گنجانیم تا در فرایندِ یادگیری لحاظ شوند. پس از آموزش، β و γ در مقادیرِ آموخته‌شده‌شان ثابت می‌شوند.

۲۱.۵ تعمیم

تا این‌جا توصیف کردیم چگونه شبکهٔ عصبی را به مجموعهٔ آموزشی‌اش برازش کنیم، اما در یادگیری ماشینی هدف، تعمیم به دادهٔ تازه‌ای است که قبلاً دیده نشده، چنان‌که با عملکردِ روی مجموعهٔ آزمون اندازه‌گیری می‌شود. در این بخش، بر سه رویکرد برای بهبودِ عملکردِ تعمیم تمرکز می‌کنیم: انتخابِ معماریِ درستِ شبکه، جریمه‌کردنِ وزن‌های بزرگ، و اختلالِ تصادفیِ مقادیرِ عبوری از شبکه در طولِ آموزش.

۲۱.۵.۱ انتخابِ معماریِ شبکه

تلاشِ فراوانی در پژوهشِ یادگیری عمیق صرفِ یافتنِ معماری‌های شبکه‌ای شده که خوب تعمیم می‌دهند. در واقع، برای هر نوعِ خاصِ داده—تصاویر، گفتار، متن، ویدئو و مانندِ آن—بخشِ بزرگی از پیشرفتِ عملکرد از کاوشِ انواعِ متفاوتِ معماری‌های شبکه و تغییرِ تعدادِ لایه‌ها، نحوهٔ اتصال‌شان و انواعِ گره در هر لایه آمده است.[6] برخی معماری‌های شبکهٔ عصبی به‌صورتِ صریح طوری طراحی شده‌اند که روی انواعِ خاصی از داده خوب تعمیم دهند: شبکه‌های پیچشی این ایده را رمز می‌کنند که استخراج‌کنندهٔ ویژگیِ یکسان در همهٔ مکان‌های یک شبکهٔ فضایی مفید است، و شبکه‌های بازگشتی این ایده را رمز می‌کنند که قاعدهٔ به‌روزرسانیِ یکسان در همهٔ نقطه‌های جریانی از دادهٔ ترتیبی مفید است. تا جایی که این فرض‌ها معتبرند، انتظار داریم معماری‌های پیچشی روی تصاویر و شبکه‌های بازگشتی روی سیگنال‌های متن و صوت خوب تعمیم دهند.



شکل ۲۱.۷ – خطایِ مجموعهٔ آزمون به‌عنوانِ تابعی از پهنای لایه (که با تعدادِ کلِ وزن‌ها اندازه‌گیری می‌شود) برای شبکه‌های پیچشیِ سه‌لایه و یازده‌لایه. محورِ افقی تعدادِ وزن‌ها را بر حسبِ $\times 10^7$ و محورِ عمودی خطایِ مجموعهٔ آزمون را در بازهٔ 0.02 تا 0.1 نشان می‌دهد؛ منحنیِ پیوسته شبکهٔ سه‌لایه و منحنیِ چین‌خورده شبکهٔ یازده‌لایه است. داده‌ها از نسخه‌های اولیهٔ سامانهٔ گوگل برای رونویسیِ آدرس‌ها در عکس‌های گرفته‌شده توسط خودروهای Street View می‌آیند (گودفلو و همکاران، 2014).

یکی از مهم‌ترین یافته‌های تجربی در حوزهٔ یادگیری عمیق این است که وقتی دو شبکه با تعدادِ وزن‌های مشابه را مقایسه می‌کنیم، شبکهٔ عمیق‌تر معمولاً عملکردِ تعمیمِ بهتری می‌دهد. شکل ۲۱.۷ این اثر را برای دستِ‌کم یک کاربردِ جهانِ واقعی—بازشناسیِ شماره‌های خانه—نشان می‌دهد. نتیجه‌ها نشان می‌دهند برای هر تعدادِ ثابتِ پارامتر، شبکهٔ یازده‌لایه خطایِ مجموعهٔ آزمونِ بسیار کمتری از شبکهٔ سه‌لایه می‌دهد.

سامانه‌های یادگیری عمیق در برخی تکلیف‌ها خوب عمل می‌کنند اما نه همهٔ آن‌ها. برای تکلیف‌های با ورودی‌های پربعد—تصاویر، ویدئو، سیگنال‌های گفتار و مانندِ آن—بهتر از هر رویکردِ صرفِ یادگیریِ ماشینیِ دیگری عمل می‌کنند. بیشترِ الگوریتم‌های توصیف‌شده در فصل ۱۹ می‌توانند ورودیِ پربعد را فقط اگر با استفاده از ویژگی‌های دستیِ طراحی‌شده پیش‌پردازش شود تا بُعد کاهش یابد مدیریت کنند. این رویکردِ پیش‌پردازشی، که پیش از ۲۰۱۰ غالب بود، عملکردی قابلِ مقایسه با سامانه‌های یادگیری عمیق به بار نیاورده.

واضح است مدل‌های یادگیری عمیق برخی جنبه‌های مهمِ این تکلیف‌ها را درمی‌گیرند. به‌طورِ خاص، موفقیتِ آن‌ها دلالت بر این دارد که تکلیف‌ها را می‌توان با برنامه‌های موازی با تعدادِ نسبتاً کمی گام حل کرد (10^1 تا 10^3 به‌جای مثلاً 10^7). این شاید چندان تعجب‌آور نباشد، زیرا این تکلیف‌ها معمولاً توسط مغز در کمتر از یک ثانیه حل می‌شوند، که برای فقط چند ده شلیکِ عصبیِ ترتیبی زمانِ کافی است. افزون بر این، با بررسیِ بازنمایی‌های لایهٔ درونیِ آموخته‌شده توسط شبکه‌های پیچشیِ عمیق برای تکلیف‌های بینایی، شواهدی می‌یابیم که گام‌های پردازش شاملِ استخراجِ دنباله‌ای از بازنمایی‌های به‌طورِ فزاینده‌ای انتزاعی از صحنه‌اند، که با لبه‌ها، نقطه‌ها و ویژگی‌های گوشه‌ایِ ریز شروع می‌شوند و با کلِ اشیا و چیدمانِ چند شیء پایان می‌یابند.

از سوی دیگر، چون مدل‌های یادگیری عمیق مدارهای ساده‌ای‌اند، قدرتِ بیانیِ ترکیبی و سوردارِ موجود در منطقِ مرتبهٔ اول (فصل ۸) و دستورهای مستقل از بافت (فصل ۲۳) را ندارند.

هرچند مدل‌های یادگیری عمیق در بسیاری موارد خوب تعمیم می‌دهند، ممکن است خطاهای غیرشهودی نیز تولید کنند. آن‌ها تمایل به تولیدِ نگاشت‌های ورودی-خروجیِ ناپیوسته دارند، به‌گونه‌ای که تغییرِ کوچکی در ورودی می‌تواند تغییرِ بزرگی در خروجی ایجاد کند. برای نمونه، ممکن است بتوان فقط چند پیکسل را در تصویری از یک سگ تغییر داد و باعث شد شبکه سگ را شترمرغ یا اتوبوسِ مدرسه رده‌بندی کند—حتی اگر تصویرِ تغییرداده‌شده همچنان دقیقاً شبیهِ سگ باشد. تصویرِ تغییرداده‌شده‌ای از این‌گونه «مثالِ تخاصمی» (Adversarial Example) نامیده می‌شود.

در فضاهای کم‌بعد یافتنِ مثال‌های تخاصمی دشوار است. اما برای تصویری با یک میلیون مقدارِ پیکسل، اغلب چنین است که حتی اگر بیشترِ پیکسل‌ها در رده‌بندیِ تصویر به‌عنوانِ درونِ وسطِ ناحیهٔ «سگ»ِ فضا سهم کنند، چند بُعد هست که مقدارِ پیکسل در آن‌ها نزدیکِ مرزِ ردهٔ دیگر است. حریفی که توانایی مهندسیِ معکوسِ شبکه را دارد می‌تواند کوچک‌ترین بردارِ اختلافی را بیابد که تصویر را از آن سویِ مرز می‌برد.

وقتی مثال‌های تخاصمی نخستین بار کشف شدند، دو تلاشِ جهانی را برانگیختند: یکی برای یافتنِ الگوریتم‌های یادگیری و معماری‌های شبکه‌ای که در برابرِ حملهٔ تخاصمی آسیب‌پذیر نباشند، و دیگری برای ایجادِ حملاتِ تخاصمیِ روزبه‌روز مؤثرتر علیه همهٔ انواعِ سامانه‌های یادگیری. تاکنون به نظر می‌رسد مهاجمان جلوترند. در واقع، در حالی که ابتدا فرض می‌شد برای ساختِ مثالِ تخاصمیِ مخصوصِ آن شبکه به دسترسی به اجزایِ درونیِ شبکهٔ آموزش‌دیده نیاز است، معلوم شده می‌توان مثال‌های تخاصمیِ مقاومِ ساخت که چندین شبکه با معماری‌ها، ابرپارامترها و مجموعه‌های آموزشیِ متفاوت را فریب می‌دهند. این یافته‌ها پیشنهاد می‌کنند مدل‌های یادگیری عمیق اشیا را به‌روش‌هایی بازشناسی می‌کنند که بسیار متفاوت از سامانهٔ بینایی انسان است.

۲۱.۵.۲ جستجوی معماریِ عصبی

متأسفانه، هنوز مجموعهٔ روشنی از رهنمودها برای کمک به انتخابِ بهترین معماریِ شبکه برای مسئله‌ای خاص نداریم. موفقیت در به‌کارگیریِ راه‌حلِ یادگیریِ عمیق نیازمندِ تجربه و قضاوتِ خوب است. از نخستین روزهای پژوهشِ شبکهٔ عصبی، تلاش‌هایی برای خودکارکردنِ فرایندِ انتخابِ معماری صورت گرفته. می‌توانیم این را حالتی از تنظیمِ ابرپارامتر (بخش ۱۹.۴.۴) بدانیم، که در آن ابرپارامترها عمق، پهنا، نحوهٔ اتصال و دیگر ویژگی‌های شبکه را تعیین می‌کنند. اما انتخاب‌های آن‌قدر زیادند که رویکردهای ساده‌ای مانندِ جستجوی شبکه‌ای نمی‌توانند همهٔ امکان‌ها را در زمانِ معقولی پوشش دهند.

بنابراین، استفاده از «جستجوی معماریِ عصبی» (Neural Architecture Search) برای کاوشِ فضایِ حالتِ معماری‌های شبکهٔ ممکن رایج است. بسیاری از فنونِ جستجو و یادگیریِ پوشش‌داده‌شده در بخش‌های پیشینِ کتاب برای جستجوی معماریِ عصبی به‌کار رفته‌اند.

الگوریتم‌های تکاملی محبوب بوده‌اند، زیرا انجامِ هر دوِ بازترکیب (به‌هم‌پیوستنِ بخش‌هایی از دو شبکه) و جهش (افزودن یا حذفِ لایه‌ای یا تغییرِ مقدارِ پارامتری) معقول است. بالارفتن از تپه (Hill Climbing) را نیز می‌توان با همین عملیاتِ جهش به‌کار برد. برخی پژوهشگران مسئله را به‌صورتِ یادگیریِ تقویتی و برخی به‌صورتِ بهینه‌سازیِ بیزی صورتبندی کرده‌اند. امکانِ دیگرِ درمانِ امکان‌های معماری به‌عنوانِ فضایی پیوستهٔ مشتق‌پذیر و استفاده از نزولِ گرادیان برای یافتنِ راه‌حلِ بهینهٔ محلی است.

برای همهٔ این فنونِ جستجو، چالشِ عمده برآوردِ ارزشِ شبکهٔ نامزد است. شیوهٔ سرراستِ ارزیابیِ معماری، آموزشِ آن روی مجموعهٔ آزمون برای چند دسته و سپس ارزیابیِ دقتش روی مجموعهٔ اعتبارسنجی است. اما با شبکه‌های بزرگ این ممکن است روزها GPU بگیرد. بنابراین، تلاش‌های بسیاری برای سرعت‌بخشیدن به این فرایندِ برآورد با حذف یا دستِ‌کم کاهشِ فرایندِ گرانِ آموزش صورت گرفته. می‌توانیم روی مجموعهٔ دادهٔ کوچک‌تری آموزش ببینیم. می‌توانیم برای تعدادِ کمی دسته آموزش ببینیم و پیش‌بینی کنیم شبکه با دسته‌های بیشتر چگونه بهبود می‌یابد. می‌توانیم از نسخهٔ کوچک‌شدهٔ معماریِ شبکه استفاده کنیم به امیدِ اینکه ویژگی‌های نسخهٔ کامل را حفظ کند. می‌توانیم یک شبکهٔ بزرگ آموزش دهیم و سپس به جستجوی زیرگراف‌هایی از شبکه بپردازیم که بهتر عمل می‌کنند؛ این جستجو می‌تواند سریع باشد، زیرا زیرگراف‌ها پارامترهای مشترک دارند و نیازی به آموزشِ مجدد ندارند.

رویکردِ دیگر یادگیریِ تابعِ ارزیابیِ اکتشافی است (همان‌گونه که برای جستجوی A* انجام شد). یعنی، با انتخابِ چند صد معماریِ شبکه شروع کن و آن‌ها را آموزش بده و ارزیابی کن. این مجموعهٔ داده‌ای از جفت‌های (شبکه، امتیاز) به ما می‌دهد. سپس نگاشتی از ویژگی‌های یک شبکه به امتیازِ پیش‌بینی‌شده یاد بگیر. از آن نقطهٔ به بعد، می‌توانیم تعدادِ زیادی شبکهٔ نامزد تولید و ارزشِشان را سریع برآورد کنیم. پس از جستجو در فضایِ شبکه‌ها، بهترین‌ها را می‌توان با یک فرایندِ کاملِ آموزش به‌طورِ کامل ارزیابی کرد.

۲۱.۵.۳ کاهشِ وزن

در بخش ۱۹.۴.۳ دیدیم منظم‌سازی—محدودکردنِ پیچیدگیِ مدل—می‌تواند به تعمیم کمک کند. این برای مدل‌های یادگیری عمیق نیز صادق است. در زمینهٔ شبکه‌های عصبی معمولاً این رویکرد را «کاهشِ وزن» (Weight Decay) می‌نامیم.

کاهشِ وزن شاملِ افزودنِ جریمهٔ $\lambda \sum_{i,j} W^2_{i,j}$ به تابعِ زیانِ به‌کاررفته برای آموزشِ شبکهٔ عصبی است، که در آن λ ابرپارامتری است که قدرتِ جریمه را کنترل می‌کند و جمع معمولاً رویِ همهٔ وزن‌های شبکه گرفته می‌شود. استفاده از $\lambda = 0$ معادلِ کاهشِ وزنِ استفاده‌نکردن است، در حالی که استفاده از مقادیرِ بزرگ‌ترِ λ وزن‌ها را تشویق به کوچک‌شدن می‌کند. استفاده از کاهشِ وزن با λ نزدیکِ 10^{-4} رایج است.

انتخابِ معماریِ خاصِ شبکه را می‌توان قیدی مطلق بر فضایِ فرضیه دید: تابعی یا در درونِ آن معماری قابلِ بازنمایی است یا نیست. عبارت‌های جریمهٔ تابعِ زیان مانندِ کاهشِ وزن قیدِ نرم‌تری ارائه می‌دهند: توابعِ بازنمایی‌شده با وزن‌های بزرگ در خانوادهٔ تابع‌اند، اما مجموعهٔ آموزشی باید شواهدِ بیشتری به نفعِ این توابع فراهم کند تا انتخابِ تابعی با وزن‌های کوچک.

تفسیرِ اثرِ کاهشِ وزن در شبکهٔ عصبی ساده نیست. در شبکه‌های با توابعِ فعال‌سازیِ سیگموئیدی، فرضیه‌پردازی می‌شود که کاهشِ وزن به نگه‌داشتنِ فعال‌سازی‌ها نزدیکِ بخشِ خطیِ سیگموئید کمک می‌کند، و از ناحیهٔ تختِ عملکرد که به گرادیانِ ناپدیدشونده می‌انجامد اجتناب می‌ورزد. با توابعِ فعال‌سازیِ ReLU، کاهشِ وزن مفید به نظر می‌رسد، اما تبیینی که برای سیگموئیدها معنا دارد دیگر برقرار نیست، زیرا خروجیِ ReLU یا خطی است یا صفر. افزون

بر این، با اتصال‌های باقی‌مانده، کاهشِ وزن شبکه را تشویق می‌کند تفاوت‌های کوچکی بینِ لایه‌های متوالی داشته باشد، نه مقادیرِ مطلقِ وزنِ کوچک. با وجودِ این تفاوت‌ها در رفتارِ کاهشِ وزن در میانِ معماری‌های بسیار، کاهشِ وزن همچنان به‌طورِ گسترده‌ای مفید است.

یک تبیین برای اثرِ سودمندِ کاهشِ وزن این است که شکلی از یادگیریِ حداکثرِ پسین (MAP) را پیاده‌سازی می‌کند (صفحهٔ ۷۲۳ را ببینید). با فرضِ اینکه X و y ورودی‌ها و خروجی‌های سراسرِ مجموعهٔ آموزشی را نشان می‌دهند، فرضیهٔ حداکثرِ پسینِ h_MAP ارضا می‌کند:

```
h_MAP = argmax_w P(y|X, W) P(W) = argmin_w [−log P(y|X, W) − log P(W)]
```

عبارتِ نخست زیانِ معمولِ آنتروپیِ متقاطع است؛ عبارتِ دوم وزن‌هایی را ترجیح می‌دهد که تحتِ توزیعِ پیشین محتمل‌اند. این دقیقاً با تابعِ زیانِ منظم‌شده هم‌راستاست اگر $\log P(W) = -\lambda \sum_{i,j} W^2_{i,j}$ بگذاریم، که به آن معناست P(W) پیشینی گاوسی با میانگینِ صفر است.

۲۱.۵.۴ دراپ‌اوت

راهِ دیگری که می‌توانیم برای کاهشِ خطایِ مجموعهٔ آزمونِ شبکه دخالت کنیم—به بهایِ دشوارترکردنِ برازشِ مجموعهٔ آموزشی—استفاده از «دراپ‌اوت» (Dropout) است. در هر گامِ آموزش، دراپ‌اوت یک گامِ یادگیریِ پسانتشار را بر نسخهٔ تازه‌ای از شبکه اعمال می‌کند که با غیرفعال‌کردنِ زیرمجموعه‌ای تصادفی از واحدها ساخته شده. این تقریبی خام و بسیار کم‌هزینه از آموزشِ گروهِ بزرگی از شبکه‌های متفاوت است (بخش ۱۹.۸ را ببینید).

به‌طورِ مشخص‌تر، فرض کنید از نزولِ گرادیانِ تصادفی با اندازهٔ مینی‌دستهٔ m استفاده می‌کنیم. برای هر مینی‌دسته، الگوریتمِ دراپ‌اوت فرایندِ زیر را بر هر گرهٔ شبکه اعمال می‌کند: با احتمالِ p، خروجیِ واحد در ضریبِ $1/p$ ضرب می‌شود؛ در غیرِ این صورت، خروجیِ واحد در صفر ثابت می‌ماند. دراپ‌اوت معمولاً بر واحدهای لایه‌های پنهان با $p = 0.5$ اعمال می‌شود؛ برای واحدهای ورودی، مقدارِ $p = 0.8$ مؤثرترین معلوم شده. این فرایند شبکه‌ای نازک‌شده با حدودِ نصفِ واحدهایِ شبکهٔ اصلی تولید می‌کند، که پسانتشار با مینی‌دستهٔ m مثالِ آموزشی بر آن اعمال می‌شود. فرایند به‌شیوهٔ معمول تا کامل‌شدنِ آموزش تکرار می‌شود. در زمانِ آزمون، مدل بدونِ دراپ‌اوت اجرا می‌شود.

می‌توانیم دراپ‌اوت را از چند دیدگاه بنگریم:

- با واردکردنِ نویز در زمانِ آموزش، مدل مجبور می‌شود در برابرِ نویز مقاوم شود.
- همان‌گونه که اشاره شد، دراپ‌اوت ایجادِ گروهِ بزرگی از شبکه‌های نازک‌شده را تقریب می‌زند. این ادعا برای مدل‌های خطی به‌صورتِ تحلیلی قابلِ راستی‌آزمایی است و به نظر می‌رسد برای مدل‌های یادگیری عمیق به‌صورتِ تجربی برقرار است.
- واحدهای پنهانِ آموزش‌دیده با دراپ‌اوت باید نه‌تنها یاد بگیرند واحدهای پنهانِ مفیدی باشند؛ همچنین باید یاد بگیرند با بسیاری مجموعه‌های ممکنِ دیگر از واحدهای پنهانِ دیگر سازگار باشند، که ممکن است در مدلِ کامل گنجانده شوند یا نه. این شبیهِ فرایندهای انتخابِ هدایت‌کنندهٔ تکاملِ ژن‌هاست: هر ژن باید نه‌تنها در کارکردِ

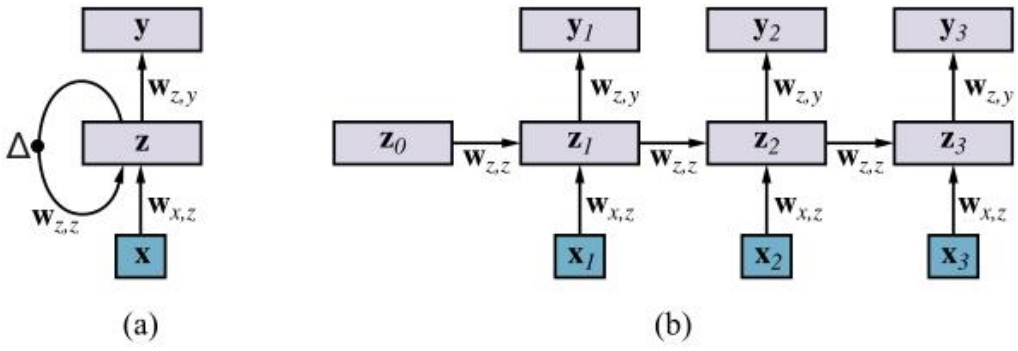
خودش مؤثر باشد، بلکه با ژن‌های دیگر، که هویتشان در جانداران آینده ممکن است به‌طورِ چشمگیری تغییر کند، خوب کار کند.

- دراپ‌اوتِ اعمال‌شده بر لایه‌های بعدیِ شبکهٔ عمیق، تصمیمِ نهایی را مجبور می‌کند به‌صورتِ مقاوم گرفته شود، با توجه به همهٔ ویژگی‌های انتزاعیِ مثال به‌جای تمرکز بر فقط یکی و نادیده‌گرفتنِ بقیه. برای نمونه، رده‌بندی برای تصاویرِ حیوانات ممکن است بتواند فقط با نگاه‌کردن به بینیِ حیوان به عملکردِ بالایی روی مجموعهٔ آموزشی برسد، اما احتمالاً در موردِ آزمونی که بینی پوشیده یا آسیب‌دیده است شکست می‌خورد. با دراپ‌اوت، مواردِ آموزشی‌ای خواهند بود که «واحدِ بینی» درونی در آن‌ها صفر می‌شود، و فرایندِ یادگیری را به یافتنِ ویژگی‌های شناساییِ اضافی می‌کشاند. توجه کنید رسیدن به همان درجهٔ مقاومت با افزودنِ نویز به دادهٔ ورودی دشوار خواهد بود: هیچ راهِ آسانی برای دانستنِ از پیشِ این‌که شبکه قصدِ تمرکز بر بینی‌ها را دارد نیست، و راهِ آسانی برای حذفِ خودکارِ بینی‌ها از هر تصویر نیست.

در مجموع، دراپ‌اوت مدل را وادار می‌کند برای هر ورودی تبیین‌های چندگانه و مقاوم یاد بگیرد. این باعث می‌شود مدل خوب تعمیم دهد، اما برازشِ مجموعهٔ آموزشی را نیز دشوارتر می‌کند—معمولاً لازم است از مدلِ بزرگ‌تری استفاده کنیم و آن را برای تکرارهای بیشتری آموزش دهیم.

۲۱.۶ شبکه‌های عصبیِ بازگشتی

شبکه‌های عصبیِ بازگشتی (Recurrent Neural Networks یا RNNها) از شبکه‌های پیشخور متمایزند، زیرا در گرافِ محاسباتی دور (چرخه) را مجاز می‌دانند. در همهٔ مواردی که بررسی می‌کنیم، هر دور دارای تأخیر است، تا واحدها بتوانند به‌عنوانِ ورودی مقداری را بگیرند که از خروجیِ خودشان در گامی پیشین از محاسبه محاسبه شده. (بدونِ تأخیر، مدارِ چرخه‌ای ممکن است به حالتِ ناسازگار برسد.) این به RNN امکان می‌دهد حالتِ درونی یا «حافظه» (Memory) داشته باشد: ورودی‌های دریافت‌شده در گام‌های زمانیِ پیشین بر پاسخِ RNN به ورودیِ کنونی اثر می‌گذارند.



شکل ۲۱.۸ – (الف) نمودارِ شماتیکِ RNN پایه‌ای که لایهٔ پنهانِ z اتصال‌های بازگشتی دارد؛ نمادِ Δ تأخیر را نشان می‌دهد. (ب) همان شبکه که روی سه گامِ زمانی بازشده تا شبکهٔ پیشخوری بسازد. توجه کنید وزن‌ها در همهٔ گام‌های زمانی مشترک‌اند.

از RNNها همچنین می‌توان برای انجامِ محاسباتِ عمومی‌تر استفاده کرد—بالاخره، رایانه‌های معمولی فقط مدارهای بولی با حافظه‌اند—و برای مدل‌کردنِ سامانه‌های عصبیِ واقعی، که بسیاری‌شان اتصال‌های چرخه‌ای دارند. در این‌جا بر استفاده از RNNها برای تحلیلِ دادهٔ ترتیبی تمرکز می‌کنیم، که در آن فرض می‌کنیم در هر گامِ زمانی بردارِ ورودیِ تازه‌ای x_t می‌رسد.

به‌عنوانِ ابزارهایی برای تحلیلِ دادهٔ ترتیبی، RNNها را می‌توان با مدل‌های مارکوفِ پنهان، شبکه‌های بیزیِ پویا و فیلترهای کالمنِ توصیف‌شده در فصل ۱۴ مقایسه کرد. (خواننده ممکن است پیش از ادامه مراجعه به آن فصل را سودمند بیابد.) مانندِ آن مدل‌ها، RNNها فرضِ مارکوف را می‌کنند (صفحهٔ ۴۶۳ را ببینید): حالتِ پنهانِ z_t شبکه برای دربرگرفتنِ اطلاعاتِ همهٔ ورودی‌های قبلی کافی است. افزون بر این، فرض کنید فرایندِ به‌روزرسانیِ حالتِ پنهان را با معادلهٔ z_t = f_w(z_{t-1}, x_t) برای تابعِ پارامتری‌شده‌ای مانندِ f_w توصیف می‌کنیم. پس از آموزش، این تابع فرایندی همگنِ در زمان (صفحهٔ ۴۶۳) را بازنمایی می‌کند—به‌طورِ مؤثر گزاره‌ای با سورِ همگانی که پوییِ بازنمایی‌شده توسط f_w برای همهٔ گام‌های زمانی برقرار است. پس RNNها قدرتِ بیانی را نسبت به شبکه‌های پیشخور می‌افزایند، همان‌گونه که شبکه‌های پیچشی می‌افزایند، و همان‌گونه که شبکه‌های بیزیِ پویا قدرتِ بیانی را نسبت به شبکه‌های بیزیِ معمولی می‌افزایند. در واقع، اگر بخواهید از شبکهٔ پیشخوری برای تحلیلِ دادهٔ ترتیبی استفاده کنید، اندازهٔ ثابتِ لایهٔ ورودی شبکه را مجبور می‌کند فقط پنجره‌ای با طولِ متناهی از داده را بررسی کند، و در آن صورت شبکه در تشخیصِ وابستگی‌های دوربرد شکست می‌خورد.

۲۱.۶.۱ آموزشِ RNN پایه

مدلِ پایه‌ای که بررسی می‌کنیم لایهٔ ورودیِ x، لایهٔ پنهانِ z با اتصال‌های بازگشتی، و لایهٔ خروجیِ y دارد، چنان‌که در شکل ۲۱.۸ (الف) نشان داده شده. فرض می‌کنیم در دادهٔ آموزشی در هر گامِ زمانی هم x و هم y مشاهده می‌شوند. معادله‌های تعریف‌کنندهٔ مدل به مقادیرِ متغیرهای فهرست‌شده با گامِ زمانیِ t ارجاع می‌دهند:

$$z_t = f_w(z_{t-1}, x_t) = g_z(W_{z,z}\, z_{t-1} + W_{x,z}\, x_t) \equiv g_z(in_{z,t})$$
$$\hat{y}_t = g_y(W_{z,y}\, z_t) \equiv g_y(in_{y,t}) \qquad (21.13)$$

که در آن g_z و g_y توابعِ فعال‌سازیِ لایه‌های پنهان و خروجی را نشان می‌دهند. مثلِ همیشه، برای هر واحد یک ورودیِ مجازیِ ثابت در 1+ و وزن‌های بایاسِ وابسته به آن ورودی‌ها فرض می‌کنیم. با توجه به دنباله‌ای از بردارهای ورودیِ x_1, ..., x_T و خروجی‌های مشاهده‌شدهٔ y_1, ..., y_T، می‌توانیم این مدل را با «بازکردنِ» آن برای T گام به شبکهٔ پیشخوری تبدیل کنیم، چنان‌که در شکل ۲۱.۸ (ب) نشان داده شده. توجه کنید ماتریس‌های وزنِ W_{z,z}، W_{x,z} و W_{z,y} در همهٔ گام‌های زمانی مشترک‌اند. در شبکهٔ بازشده، به‌آسانی دیده می‌شود که می‌توانیم گرادیان‌ها را برای آموزشِ وزن‌ها به‌شیوهٔ معمول محاسبه کنیم؛ تنها تفاوت این است که اشتراکِ وزن‌ها در میانِ لایه‌ها محاسبهٔ گرادیان را کمی پیچیده‌تر می‌کند.

برای ساده‌نگه‌داشتنِ معادله‌ها، محاسبهٔ گرادیان را برای RNN با فقط یک واحدِ ورودی، یک واحدِ پنهان و یک واحدِ خروجی نشان می‌دهیم. برای این حالت، با صریح‌کردنِ وزن‌های بایاس داریم: z_t = g_z(w_{z,z} z_{t-1} + w_{x,z} x_t + w_{0,z}) و ŷ_t = g_y(w_{z,y} z_t + w_{0,y}). مانندِ معادله‌های (21.4) و (21.5)، زیانِ مربعِ خطایِ L را فرض

می‌کنیم—در این حالت، رویِ گام‌های زمانی جمع شده. اشتقاق‌های وزن‌های لایهٔ ورودی و لایهٔ خروجی، $w_{x,z}$ و $w_{z,y}$، اساساً با معادلهٔ (21.4) یکسان‌اند، پس آن‌ها را به‌عنوانِ تمرین واگذار می‌کنیم. برای وزنِ لایهٔ پنهانِ $w_{z,z}$، چند گامِ نخست نیز همان الگوی معادلهٔ (21.4) را دنبال می‌کنند:

$$\begin{aligned}
\partial L/\partial w_{z,z} &= \partial/\partial w_{z,z} \sum_{t=1}^{T} (y_t - \hat{y}_t)^2 \\
&= -2(y_t - \hat{y}_t)\, \partial \hat{y}_t/\partial w_{z,z} \\
&= -2(y_t - \hat{y}_t)\, \partial/\partial w_{z,z}\, g_y(in_{y,t}) \\
&= -2(y_t - \hat{y}_t)\, g'_y(in_{y,t})\, \partial/\partial w_{z,z}\, in_{y,t} \\
&= -2(y_t - \hat{y}_t)\, g'_y(in_{y,t})\, \partial/\partial w_{z,z}\, (w_{z,y}\, z_t + w_{0,y}) \\
&= -2(y_t - \hat{y}_t)\, g'_y(in_{y,t})\, w_{z,y}\, \partial z_t/\partial w_{z,z} \\
&= \sum_{t=1}^{T}
\end{aligned} \tag{21.14}$$

حالا گرادیانِ واحدِ پنهانِ z_t را می‌توان از گامِ زمانیِ قبلی به‌صورتِ زیر به‌دست آورد:

$$\begin{aligned}
\partial z_t/\partial w_{z,z} &= \partial/\partial w_{z,z}\, g_z(in_{z,t}) \\
&= g'_z(in_{z,t})\, \partial/\partial w_{z,z}\, in_{z,t} \\
&= g'_z(in_{z,t})\, \partial/\partial w_{z,z}\, (w_{z,z}\, z_{t-1} + w_{x,z}\, x_t + w_{0,z}) \\
&= g'_z(in_{z,t})\, (z_{t-1} + w_{z,z}\, \partial z_{t-1}/\partial w_{z,z}) \qquad (21.15)
\end{aligned}$$

که در آن خطِ آخر از قاعدهٔ مشتقِ حاصلضرب استفاده می‌کند: $\partial(uv)/\partial x = u\, \partial v/\partial x + v\, \partial u/\partial x$.

با نگاه به معادلهٔ (21.15)، دو چیز را متوجه می‌شویم. نخست، عبارتِ گرادیان بازگشتی است: سهمِ گامِ زمانیِ t از گرادیان با استفاده از سهمِ گامِ زمانیِ $t-1$ محاسبه می‌شود. اگر محاسبه‌ها را به‌شیوهٔ درست ترتیب دهیم، زمانِ اجرایِ کلِ محاسبهٔ گرادیان در اندازهٔ شبکه خطی خواهد بود. این الگوریتم «پسانتشار در طولِ زمان» (Back-Propagation Through Time) نامیده می‌شود و معمولاً به‌صورتِ خودکار توسط سامانه‌های نرم‌افزاریِ یادگیری عمیق مدیریت می‌شود. دوم، اگر محاسبهٔ بازگشتی را تکرار کنیم، می‌بینیم گرادیان‌ها در T عبارت‌های متناسب با $w_{z,z} \prod_{t=1}^{T} g'_z(in_{z,t})$ را شامل خواهند شد. برای سیگموئیدها، tanhها و ReLUها، $g' \le 1$، پس RNN سادهٔ ما اگر $w_{z,z} < 1$ باشد قطعاً از مسئلهٔ گرادیانِ ناپدیدشونده رنج خواهد برد (صفحهٔ ۷۵۶ را ببینید). از سوی دیگر، اگر $w_{z,z} > 1$ باشد، ممکن است مسئلهٔ «گرادیانِ منفجرشونده» (Exploding Gradient) را تجربه کنیم. (برای حالتِ عمومی، این پیامدها به اولین مقدارِ ویژهٔ ماتریسِ وزنِ $W_{z,z}$ بستگی دارد.) بخشِ بعدی طراحیِ RNN پرکارتری را توصیف می‌کند که برای کاهشِ این مسئله در نظر گرفته شده.

۲۱.۶.۲ RNNهای حافظهٔ کوتاه‌مدت-بلندمدت

چند معماریِ تخصصیِ RNN با هدفِ ممکن‌ساختنِ حفظِ اطلاعات در طولِ گام‌های زمانیِ بسیار طراحی شده‌اند. یکی از محبوب‌ترین‌ها «حافظهٔ کوتاه‌مدت-بلندمدت» (Long Short-Term Memory) یا LSTM است. مؤلفهٔ حافظهٔ بلندمدتِ LSTM، که «سلولِ حافظه» (Memory Cell) نامیده و با c نشان داده می‌شود، اساساً از گامِ زمانی‌ای به گامِ زمانیِ بعد کپی می‌شود. (در مقابل، RNN پایه حافظه‌اش را در هر گامِ زمانی در ماتریسِ وزنی ضرب می‌کند، چنان‌که در

معادلهٔ (21.13) نشان داده شده.) اطلاعاتِ تازه با افزودنِ به‌روزرسانی‌ها واردِ حافظه می‌شود؛ به این‌شیوه، عبارت‌های گرادیان در طولِ زمان به‌صورتِ ضربی انباشته نمی‌شوند. LSTMها همچنین «واحدهای دروازه‌ای» (Gating Unit) دارند، که بردارهایی‌اند که جریانِ اطلاعات را در LSTM از طریقِ ضربِ عنصربه‌عنصرِ بردارِ اطلاعاتِ متناظر کنترل می‌کنند:

- «دروازهٔ فراموشی» (Forget Gate) f تعیین می‌کند آیا هر عنصرِ سلولِ حافظه به خاطر سپرده می‌شود (به گامِ زمانیِ بعد کپی می‌شود) یا فراموش می‌شود (در صفر بازنشانی می‌شود).
- «دروازهٔ ورودی» (Input Gate) i تعیین می‌کند آیا هر عنصرِ سلولِ حافظه به‌صورتِ افزایشی با اطلاعاتِ تازه از بردارِ ورودیِ گامِ زمانیِ کنونی به‌روزرسانی می‌شود.
- «دروازهٔ خروجی» (Output Gate) o تعیین می‌کند آیا هر عنصرِ سلولِ حافظه به حافظهٔ کوتاه‌مدتِ z منتقل می‌شود، که نقشی شبیهِ حالتِ پنهان در RNNهای پایه ایفا می‌کند.

در حالی که واژهٔ «دروازه» در طراحیِ مدار معمولاً تابعِ بولی را به ذهن می‌آورد، دروازه‌ها در LSTMها نرم‌اند—برای نمونه، اگر عناصرِ متناظرِ بردارِ دروازهٔ فراموشی کوچک اما ناصفر باشند، عناصرِ بردارِ سلولِ حافظه به‌صورتِ جزئی فراموش می‌شوند. مقادیرِ واحدهای دروازه‌ای همیشه در بازهٔ [0, 1]اند و به‌عنوانِ خروجی‌های تابعِ سیگموئیدِ اعمال‌شده بر ورودیِ کنونی و حالتِ پنهانِ قبلی به‌دست می‌آیند. به‌طورِ دقیق، معادله‌های به‌روزرسانیِ LSTM به‌صورتِ زیرند:

$$
\begin{aligned}
f_t &= \sigma(W_{x,f}\, x_t + W_{z,f}\, z_{t-1}) \\
i_t &= \sigma(W_{x,i}\, x_t + W_{z,i}\, z_{t-1}) \\
o_t &= \sigma(W_{x,o}\, x_t + W_{z,o}\, z_{t-1}) \\
c_t &= c_{t-1} \odot f_t + i_t \odot \tanh(W_{x,c}\, x_t + W_{z,c}\, z_{t-1}) \\
z_t &= \tanh(c_t) \odot o_t
\end{aligned}
$$

که در آن زیرنویس‌های رویِ ماتریس‌های وزنِ گوناگونِ W مبدأ و مقصدِ پیوندهای متناظر را نشان می‌دهند. نمادِ ⊙ ضربِ عنصربه‌عنصر را نشان می‌دهد.

LSTMها از نخستین شکل‌های عملاً قابلِ استفادهٔ RNN بودند. آن‌ها عملکردِ عالی‌ای رویِ طیفِ وسیعی از تکلیف‌ها از جملهِ بازشناسیِ گفتار و بازشناسیِ دست‌نویس نشان داده‌اند. استفادهٔ آن‌ها در پردازشِ زبانِ طبیعی در فصل ۲۴ بحث می‌شود.

۲۱.۷ یادگیریِ نظارت‌نشده و یادگیریِ انتقالی

سامانه‌های یادگیریِ عمیقی که تاکنون بحث کرده‌ایم بر یادگیریِ نظارت‌شده مبتنی‌اند، که برای هر مثالِ آموزشی برچسبی با مقدارِ تابعِ هدف لازم دارد. هرچند چنین سامانه‌هایی می‌توانند به سطحِ بالایی از دقتِ مجموعهٔ آزمون برسند—چنان‌که نتایجِ رقابتِ ImageNet نشان می‌دهد—اغلب به دادهٔ برچسب‌خوردهٔ بسیار بیشتری از آنچه انسان برای همان تکلیف نیاز دارد احتیاج دارند. برای نمونه، کودک برای بازشناسیِ قابلِ اطمینانِ زرافه‌ها در طیفِ وسیعی از محیط‌ها و نماها فقط نیاز به دیدنِ یک تصویر از زرافه دارد، نه هزاران. واضح است چیزی در داستانِ یادگیری عمیقِ ما کم است؛ در واقع، ممکن است رویکردِ کنونیِ ما به یادگیریِ عمیقِ نظارت‌شده برخی تکلیف‌ها را کاملاً دست‌نیافتنی

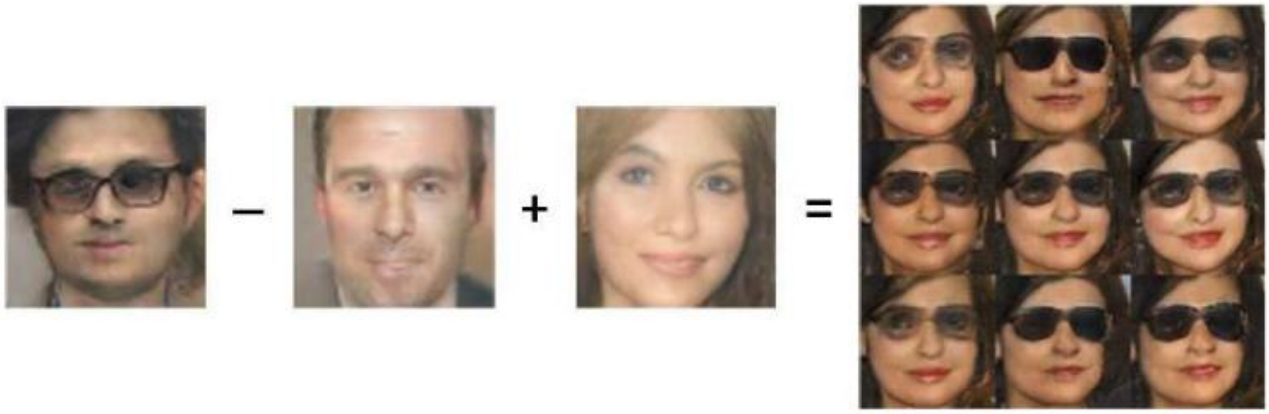
کند، زیرا الزاماتِ دادهٔ برچسب‌خورده فراتر از آن چیزی خواهد بود که نسلِ بشر (یا جهان) می‌تواند تأمین کند. افزون بر این، حتی در مواردی که تکلیف شدنی است، برچسب‌گذاریِ مجموعه‌های دادهٔ بزرگ معمولاً به کارِ انسانیِ کمیاب و گران نیاز دارد.

به این دلایل، علاقهٔ شدیدی به چند پارادایمِ یادگیری هست که وابستگی به دادهٔ برچسب‌خورده را کاهش می‌دهند. چنان‌که در فصل ۱۹ دیدیم، این پارادایم‌ها شاملِ یادگیریِ نظارت‌نشده، یادگیریِ انتقالی و یادگیریِ نیمه‌نظارتی‌اند. الگوریتم‌های یادگیریِ نظارت‌نشده صرفاً از ورودی‌های برچسب‌نخوردهٔ x یاد می‌گیرند، که اغلب به‌فراوانیِ بیشتری از مثال‌های برچسب‌خورده در دسترس‌اند. الگوریتم‌های یادگیریِ نظارت‌نشده معمولاً «مدل‌های مولد» (Generative Model) تولید می‌کنند، که می‌توانند متن، تصویر، صدا و ویدئوی واقع‌نمایانه تولید کنند، به‌جای اینکه صرفاً برچسب‌هایی برای چنین داده‌ای پیش‌بینی کنند. الگوریتم‌های یادگیریِ انتقالی به مثال‌های برچسب‌خوردهٔ برخی نیاز دارند اما با مطالعهٔ مثال‌های برچسب‌خورده برای تکلیف‌های متفاوت می‌توانند عملکردشان را بیشتر بهبود بخشند، و از این راه امکانِ بهره‌گیری از منابعِ موجودِ بیشترِ داده را میسر می‌کنند. الگوریتم‌های یادگیریِ نیمه‌نظارتی به مثال‌های برچسب‌خوردهٔ برخی نیاز دارند اما با مطالعهٔ مثال‌های برچسب‌نخورده نیز می‌توانند عملکردشان را بیشتر بهبود بخشند. این بخش رویکردهای یادگیریِ عمیق را به یادگیریِ نظارت‌نشده و انتقالی پوشش می‌دهد؛ هرچند یادگیریِ نیمه‌نظارتی نیز حوزهٔ پژوهشیِ فعالی در جامعهٔ یادگیری عمیق است، فنونی که تاکنون توسعه یافته‌اند در عمل اثربخشیِ گسترده‌ای نشان نداده‌اند، پس آن‌ها را پوشش نمی‌دهیم.

۲۱.۷.۱ یادگیریِ نظارت‌نشده

همهٔ الگوریتم‌های یادگیریِ نظارت‌شده اساساً هدفِ یکسانی دارند: با توجه به مجموعهٔ آموزشیِ ورودی‌های x و خروجی‌های متناظرِ y = f(x)، تابعِ h را یاد بگیر که f را خوب تقریب می‌زند. الگوریتم‌های یادگیریِ نظارت‌نشده، از سوی دیگر، مجموعهٔ آموزشیِ مثال‌های برچسب‌نخوردهٔ x را می‌گیرند. در این‌جا دو کاری را توصیف می‌کنیم که چنین الگوریتمی ممکن است بکوشد انجام دهد. نخستین، یادگیریِ بازنمایی‌های تازه—برای نمونه، ویژگی‌های تازه‌ای از تصاویر که شناساییِ اشیای درونِ تصویر را آسان‌تر می‌کند. دومین، یادگیریِ مدلِ مولد—معمولاً به‌صورتِ توزیعِ احتمالی‌ای که نمونه‌های تازه‌ای می‌توان از آن تولید کرد. (الگوریتم‌های یادگیریِ شبکه‌های بیزی در فصل ۲۰ در این رده قرار می‌گیرند.) بسیاری از الگوریتم‌ها هم به یادگیریِ بازنمایی و هم به مدل‌سازیِ مولد قادرند.

فرض کنید مدلِ توأمِ P_W(x, z) را یاد می‌گیریم، که در آن z مجموعه‌ای از متغیرهای نهان و مشاهده‌نشده است که به‌نحوی محتوای دادهٔ x را بازنمایی می‌کنند. در هماهنگی با روحِ این فصل، معانیِ متغیرهای z را از پیش تعریف نمی‌کنیم؛ مدل آزاد است یاد بگیرد z را هرطور که می‌خواهد با x مرتبط کند. برای نمونه، مدلی که رویِ تصاویرِ ارقامِ دست‌نویس آموزش دیده ممکن است انتخاب کند از یک جهت در فضایِ z برای بازنماییِ ضخامتِ ضربه‌های قلم، جهتِ دیگری برای رنگِ جوهر، جهتِ دیگری برای رنگِ پس‌زمینه و مانندِ آن استفاده کند. با تصاویرِ چهره، الگوریتمِ یادگیری ممکن است یک جهت را برای بازنماییِ جنسیت و دیگری را برای دربرگرفتنِ حضور یا غیابِ عینک انتخاب کند، چنان‌که در شکل ۲۱.۹ نشان داده شده.



شکل ۲۱.۹ — نمایشی از اینکه چگونه مدلِ مولد یاد گرفته از جهت‌های متفاوت در فضایِ z برای بازنماییِ جنبه‌های متفاوتِ چهره‌ها استفاده کند. در واقع می‌توانیم در فضایِ z حساب انجام دهیم. تصاویرِ این‌جا همگی از مدلِ آموخته‌شده تولید شده‌اند و نشان می‌دهند وقتی نقطه‌های متفاوتِ فضایِ z رمزگشایی می‌شوند چه رخ می‌دهد. با مختصاتِ مفهومِ «مردِ عینکی» شروع می‌کنیم، مختصاتِ «مرد» را کم می‌کنیم، مختصاتِ «زن» را اضافه می‌کنیم، و مختصاتِ «زنِ عینکی» را به‌دست می‌آوریم. تصاویر با اجازه از (رادفورد و همکاران، 2015) بازتولید شده‌اند.

مدلِ احتمالیِ آموخته‌شدهٔ P_W(x, z) هم یادگیریِ بازنمایی را محقق می‌کند (بردارهایِ معنادارِ z را از بردارهایِ خامِ x ساخته) و هم مدل‌سازیِ مولد را: اگر z را از P_W(x, z) انتگرال بگیریم P_W(x) به‌دست می‌آید.

PCA احتمالاتی: مدلِ مولدِ ساده. پیشنهادهای بسیاری برای شکلِ P_W(x, z) شده است. یکی از ساده‌ترین‌ها مدلِ «تحلیلِ مؤلفه‌های اصلیِ احتمالاتی» (Probabilistic Principal Components Analysis یا PPCA) است.[7] در مدلِ PPCA، z از گاوسیِ کرویِ میانگین-صفر انتخاب می‌شود، سپس x از z با اعمالِ ماتریسِ وزنِ W و افزودنِ نویزِ گاوسیِ کروی تولید می‌شود:

```
P(z) = N(z; 0, I)
P_W(x|z) = N(x; Wz, σ²I)
```

وزن‌های W (و به‌صورتِ اختیاری پارامترِ نویزِ σ^2) را می‌توان با بیشینه‌کردنِ درست‌نماییِ داده یاد گرفت، که به‌صورتِ زیر است:

$$P_W(x) = \int P_W(x, z)\, dz = N(x; 0, WW^T + \sigma^2 I) \qquad (21.16)$$

بیشینه‌سازی نسبت به W را می‌توان با روش‌های گرادیانی یا با الگوریتمِ تکراریِ کارآمدِ EM انجام داد (بخش ۲۰.۳ را ببینید). پس از یادگیریِ W، نمونه‌های دادهٔ تازه را می‌توان مستقیماً از P_W(x) با استفاده از معادلهٔ (21.16) تولید کرد. افزون بر این، مشاهده‌های تازهٔ x که طبقِ معادلهٔ (21.16) احتمالِ بسیار پایینی دارند را می‌توان به‌عنوانِ ناهنجاری‌های بالقوه علامت‌گذاری کرد. با PPCA، معمولاً فرض می‌کنیم بُعدِ z بسیار کمتر از بُعدِ x است، تا مدل یاد بگیرد داده را تا حدِ ممکن بر حسبِ تعدادِ کمی ویژگی تبیین کند. این ویژگی‌ها را می‌توان برای استفاده در رده‌بندهای استاندارد با محاسبهٔ $\hat{z}$، یعنی امیدِ ریاضیِ P_W(z|x)، استخراج کرد.

تولیدِ داده از مدلِ PCA احتمالاتی ساده است: نخست z را از پیشینِ گاوسیِ ثابت‌اش نمونه‌برداری کن، سپس x را از گاوسی با میانگینِ Wz نمونه‌برداری کن. چنان‌که به‌زودی خواهیم دید، بسیاری از مدل‌های مولدِ دیگر شبیهِ این فرایندند، اما به‌جای نگاشت‌های خطی از فضایِ z به فضایِ x، از نگاشت‌های پیچیده‌ای استفاده می‌کنند که توسط مدل‌های عمیق تعریف شده‌اند.

خودرمزگذارها. بسیاری از الگوریتم‌های یادگیریِ عمیقِ نظارت‌نشده بر ایدهٔ «خودرمزگذار» (Autoencoder) مبتنی‌اند. خودرمزگذار مدلی است با دو بخش: «رمزگذار» (Encoder) که از x به بازنماییِ ẑ نگاشت می‌کند و «رمزگشا» (Decoder) که از بازنماییِ ẑ به دادهٔ مشاهده‌شدهٔ x نگاشت می‌کند. به‌طورِ کلی، رمزگذار صرفاً تابعِ پارامتری‌شدهٔ f و رمزگشا صرفاً تابعِ پارامتری‌شدهٔ g است. مدل چنان آموزش می‌بیند که $x \approx g(f(x))$، تا فرایندِ رمزگذاری تقریباً توسط فرایندِ رمزگشایی معکوس شود. توابعِ f و g می‌توانند مدل‌های خطیِ ساده‌ای باشند که با ماتریسِ واحدی پارامتری شده‌اند یا می‌توانند توسط شبکهٔ عصبیِ عمیقی بازنمایی شوند.

خودرمزگذارِ بسیار ساده‌ای «خودرمزگذارِ خطی» است، که در آن هر دوِ f و g خطی‌اند با ماتریسِ وزنِ مشترکِ W:

```
ẑ = f(x) = Wx
x = g(ẑ) = Wᵀẑ
```

یکی از راه‌های آموزشِ این مدل کمینه‌کردنِ مربعِ خطایِ $\sum_j \|x_j - g(f(x_j))\|^2$ است تا $x \approx g(f(x))$. ایده این است W را چنان آموزش دهیم که ẑ کم‌بعد تا حدِ ممکن اطلاعات را برای بازسازیِ دادهٔ پربعدِ x حفظ کند. معلوم می‌شود این خودرمزگذارِ خطی با تحلیلِ مؤلفه‌های اصلیِ کلاسیک (PCA) پیوندی نزدیک دارد. وقتی z بُعدِ m داشته باشد، ماتریسِ W باید یاد بگیرد m مؤلفهٔ اصلیِ داده را بپوشاند—به بیانِ دیگر، مجموعهٔ m جهتِ متعامدی که داده در آن‌ها بالاترین واریانس را دارد، یا به‌طورِ هم‌ارز، m بردارِ ویژهٔ ماتریسِ کوواریانسِ داده که بزرگ‌ترین مقادیرِ ویژه را دارند—دقیقاً مانندِ PCA. مدلِ PCA مدلِ مولدِ ساده‌ای است که با خودرمزگذارِ خطیِ ساده‌ای متناظر است. این تناظر پیشنهاد می‌کند ممکن است راهی برای دربرگرفتنِ انواعِ پیچیده‌ترِ مدل‌های مولد با انواعِ پیچیده‌ترِ خودرمزگذارها باشد. «خودرمزگذارِ تغییراتی» (Variational Autoencoder یا VAE) یکی از راه‌های انجامِ این کار را فراهم می‌کند.

خودرمزگذارِ تغییراتی. روش‌های تغییراتی در صفحهٔ ۴۵۸ به‌طورِ مختصر به‌عنوانِ راهی برای تقریبِ توزیعِ پسین در مدل‌های احتمالیِ پیچیده معرفی شدند، جایی که جمع‌زدن یا انتگرال‌گرفتنِ تعدادِ زیادی متغیرِ پنهان محاسبه‌ناپذیر است. ایده استفاده از «پسینِ تغییراتی» (Variational Posterior) Q(z)، برگرفته از خانواده‌ای از توزیع‌های محاسباتاً قابلِ مدیریت، به‌عنوانِ تقریبی از پسینِ واقعی است. برای نمونه، ممکن است Q را از خانوادهٔ توزیع‌های گاوسی با ماتریسِ کوواریانسِ قطری انتخاب کنیم. درونِ خانوادهٔ انتخابیِ توزیع‌های قابلِ مدیریت، Q چنان بهینه می‌شود که تا حدِ ممکن به توزیعِ پسینِ واقعیِ P(z|x) نزدیک باشد.

برای اهدافِ ما، مفهومِ «تا حدِ ممکن نزدیک» توسط واگراییِ KL تعریف می‌شود، که در صفحهٔ ۷۵۸ به آن اشاره کردیم. این به‌صورتِ زیر است:

```
D_KL(Q(z) || P(z|x)) = ∫ Q(z) log(Q(z) / P(z|x)) dz
```

که میانگینی (نسبت به Q) از لگاریتمِ نسبتِ Q و P است. به‌آسانی دیده می‌شود $D_{KL}(Q(z) \| P(z|x)) \geq 0$، با برابری وقتی Q و P بر هم منطبق‌اند. سپس می‌توانیم «کرانِ پایینِ تغییراتی»ِ L (که گاهی «کرانِ پایینِ شواهد» یا ELBO نامیده می‌شود) را بر لگاریتمِ درست‌نماییِ داده تعریف کنیم:

$$L(x, Q) = \log P(x) - D_{KL}(Q(z) \| P(z|x)) \qquad (21.17)$$

می‌بینیم L کرانی پایین برای log P است، زیرا واگراییِ KL نامنفی است. یادگیریِ تغییراتی به‌جای بیشینه‌کردنِ log P(x)، L را نسبت به پارامترهای w بیشینه می‌کند، به امیدِ اینکه راه‌حلِ یافت‌شدهٔ w* به بیشینه‌کردنِ log P(x) نیز نزدیک باشد.

چنان‌که نوشته شده، L هنوز به نظر نمی‌رسد بیشینه‌کردنش از log P آسان‌تر باشد. خوشبختانه، می‌توانیم معادلهٔ (21.17) را بازنویسی کنیم تا قابلیتِ محاسباتیِ بهتری را آشکار کند:

$$\begin{aligned} L &= \log P(x) - \int Q(z) \log(Q(z) / P(z|x))\, dz \\ &= -\int Q(z) \log Q(z)\, dz + \int Q(z) \log P(z, x)\, dz \\ &= H(Q) + E_{z\sim Q}[\log P(z, x)] \end{aligned}$$

که در آن H(Q) آنتروپیِ توزیعِ Q است. برای برخی خانواده‌های تغییراتیِ Q (مانندِ توزیع‌های گاوسی)، H(Q) را می‌توان به‌صورتِ تحلیلی ارزیابی کرد. افزون بر این، امیدِ ریاضیِ $E_{z\sim Q}\log P(z, x)$ از طریقِ نمونه‌هایی از z از Q برآوردِ بدونِ اریبِ کارآمدی می‌پذیرد. برای هر نمونه، P(z, x) معمولاً می‌تواند به‌صورتِ کارآمد ارزیابی شود—برای نمونه، اگر P شبکهٔ بیزی باشد، P(z, x) صرفاً حاصلضربی از احتمال‌های شرطی است، زیرا z و x همهٔ متغیرها را دربرمی‌گیرند.

خودرمزگذارهای تغییراتی وسیله‌ای برای انجامِ یادگیریِ تغییراتی در بسترِ یادگیریِ عمیق فراهم می‌کنند. یادگیریِ تغییراتی شاملِ بیشینه‌کردنِ L نسبت به پارامترهای هر دوِ P و Q است. برای خودرمزگذارِ تغییراتی، رمزگشای g(z) به‌عنوانِ تعریف‌کنندهٔ log P(x|z) تفسیر می‌شود. برای نمونه، خروجیِ رمزگشا ممکن است میانگینِ گاوسیِ شرطی را تعریف کند. به‌همین‌سان، خروجیِ رمزگذارِ f(x) به‌عنوانِ تعریف‌کنندهٔ پارامترهای Q تفسیر می‌شود—برای نمونه، Q ممکن است گاوسی با میانگینِ f(x) باشد. آموزشِ خودرمزگذارِ تغییراتی سپس شاملِ بیشینه‌کردنِ L نسبت به پارامترهای هر دوِ رمزگذارِ f و رمزگشایِ g است، که خودشان می‌توانند شبکه‌های عمیقِ دلخواهاً پیچیده‌ای باشند.

مدل‌های خودرگرسیونیِ عمیق. «مدلِ خودرگرسیونی» (Autoregressive Model یا AR) مدلی است که در آن هر عنصرِ x_i از بردارِ دادهٔ x بر مبنای عناصرِ دیگرِ بردار پیش‌بینی می‌شود. چنین مدلی هیچ متغیرِ نهانی ندارد. اگر x اندازهٔ ثابتی داشته باشد، مدلِ AR را می‌توان شبکهٔ بیزیِ کاملاً مشاهده‌شده و احتمالاً کاملاً متصل در نظر گرفت. این بدان‌معناست که محاسبهٔ درست‌نماییِ بردارِ دادهٔ داده‌شده طبقِ مدلِ AR بی‌اهمیت است؛ همین‌طور برای پیش‌بینیِ مقدارِ متغیرِ مفقودِ تنها با توجه به بقیه، و برای نمونه‌برداریِ بردارِ داده از مدل.

رایج‌ترین کاربردِ مدل‌های خودرگرسیونی تحلیلِ دادهٔ سریِ زمانی است، که در آن مدلِ AR از مرتبهٔ k مقدارِ x_t را با توجه به $x_{t-1}, \ldots, x_{t-k}$ پیش‌بینی می‌کند. در اصطلاح‌شناسیِ فصل ۱۴، مدلِ AR مدلِ مارکوفِ غیرپنهان است. در اصطلاح‌شناسیِ فصل ۲۳، مدلِ n-گرمیِ دنباله‌های حرف یا واژه مدلِ AR از مرتبهٔ n-1 است.

در مدل‌های AR کلاسیک، که متغیرهایشان حقیقی‌مقدارند، توزیعِ شرطیِ P(x_t | x_{t-k}, ..., x_{t-1}) مدلِ خطی-گاوسی با واریانسِ ثابت است که میانگینِ آن ترکیبِ خطیِ وزن‌داری از x_{t-k}, ..., x_{t-1} است—به بیانِ دیگر، مدلِ رگرسیونِ خطیِ استاندارد. راه‌حلِ حداکثرِ درست‌نمایی توسط «معادله‌های یول–واکر» (Yule–Walker Equations) به‌دست می‌آید، که با معادله‌های نرمالِ صفحهٔ ۶۸۰ پیوندی نزدیک دارند.

«مدلِ خودرگرسیونیِ عمیق» (Deep Autoregressive Model) مدلی است که در آن مدلِ خطی-گاوسی با شبکهٔ عمیقِ دلخواهی با لایهٔ خروجیِ مناسب، بسته به گسسته یا پیوسته‌بودنِ x_t، جایگزین می‌شود. کاربردهای اخیرِ این رویکردِ خودرگرسیونی شاملِ مدلِ WaveNet دیپ‌مایند برای تولیدِ گفتار است (فن‌دن‌اورد و همکاران، 2016a). WaveNet رویِ سیگنال‌های صوتیِ خام آموزش می‌بیند، که ۱۶٬۰۰۰ بار در ثانیه نمونه‌برداری می‌شوند، و مدلِ AR غیرخطیِ از مرتبهٔ ۴۸۰۰ با ساختارِ پیچشیِ چندلایه پیاده‌سازی می‌کند. در آزمون‌ها معلوم می‌شود به‌طورِ چشمگیری واقع‌گرایانه‌تر از سامانه‌های تولیدِ گفتارِ پیشتازِ قبلی است.

شبکه‌های مولدِ تخاصمی. «شبکهٔ مولدِ تخاصمی» (Generative Adversarial Network یا GAN) در واقع جفتی از شبکه‌هاست که با هم سامانه‌ای مولد تشکیل می‌دهند. یکی از شبکه‌ها، «مولد» (Generator)، مقادیرِ z را به x نگاشت می‌کند تا نمونه‌هایی از توزیعِ P_w(x) تولید کند. طرحِ معمولی z را از گاوسیِ واحد با بُعدِ متوسط نمونه‌برداری می‌کند و سپس آن را از شبکهٔ عمیقِ h_w عبور می‌دهد تا x به‌دست آید. شبکهٔ دیگر، «متمایزگر» (Discriminator)، رده‌بندی است که آموزش می‌بیند ورودی‌های x را به‌عنوانِ واقعی (گرفته‌شده از مجموعهٔ آموزشی) یا جعلی (ساخته‌شده توسط مولد) رده‌بندی کند. GANها نوعی «مدلِ ضمنی» (Implicit Model)اند، به این معنا که نمونه‌ها قابلِ تولیدند اما احتمال‌هایشان به‌آسانی در دسترس نیست؛ در شبکهٔ بیزی، از سوی دیگر، احتمالِ یک نمونه صرفاً حاصلضربِ احتمال‌های شرطی در طولِ مسیرِ تولیدِ نمونه است.

مولد با رمزگشایِ چارچوبِ خودرمزگذارِ تغییراتی پیوندی نزدیک دارد. چالشِ مدل‌سازیِ ضمنی طراحیِ تابعِ زیانی است که آموزشِ مدل را با استفاده از نمونه‌هایی از توزیع ممکن کند، به‌جای بیشینه‌کردنِ درست‌نماییِ تخصیص‌داده‌شده به مثال‌های آموزشی از مجموعهٔ داده.

هم مولد و هم متمایزگر به‌طورِ هم‌زمان آموزش می‌بینند، و مولد یاد می‌گیرد متمایزگر را بفریبد و متمایزگر یاد می‌گیرد دادهٔ واقعی را از جعلی دقیقاً جدا کند. رقابتِ بینِ مولد و متمایزگر را می‌توان به زبانِ نظریهٔ بازی توصیف کرد (فصل ۱۸ را ببینید). ایده این است که در حالتِ تعادلِ بازی، مولد باید توزیعِ آموزشی را کاملاً بازتولید کند، چنان‌که متمایزگر نتواند بهتر از حدسِ تصادفی عمل کند. GANها به‌ویژه برای تکلیف‌های تولیدِ تصویر خوب کار کرده‌اند. برای نمونه، GANها می‌توانند تصاویرِ واقع‌نمایِ با وضوحِ بالایی از مردمانی بسازند که هرگز وجود نداشته‌اند (کاراس و همکاران، 2017).

ترجمهٔ نظارت‌نشده. تکلیف‌های ترجمه، در معنایِ گسترده، شاملِ تبدیلِ ورودیِ x با ساختارِ غنی به خروجیِ y با ساختارِ غنی‌اند. در این بافت، «ساختارِ غنی» بدان معناست که داده چندبعدی است و وابستگی‌های آماریِ جالبی در میانِ ابعادِ گوناگون دارد. تصاویر و جمله‌های زبانِ طبیعی ساختارِ غنی دارند، اما عددِ تکی مانندِ شناسهٔ رده ندارد. تبدیلِ جمله‌ای از انگلیسی به فرانسوی یا تبدیلِ عکسی از صحنهٔ شب به عکسِ هم‌ارزِ گرفته‌شده در روز هر دو مثال‌هایی از تکلیفِ ترجمه‌اند. ترجمهٔ نظارت‌شده شاملِ گردآوریِ جفت‌های (x, y) بسیار و آموزشِ مدل برای نگاشتِ هر x به y متناظر است. برای نمونه، سامانه‌های ترجمهٔ ماشینی اغلب رویِ جفتِ جمله‌هایی آموزش می‌بینند که توسط مترجم‌های

انسانیِ حرفه‌ای ترجمه شده‌اند. برای انواعِ دیگرِ ترجمه، دادهٔ آموزشیِ نظارت‌شده ممکن است در دسترس نباشد. برای نمونه، عکسی از صحنهٔ شب را در نظر بگیرید که خودروها و عابرانِ پیادهٔ متحرکِ بسیاری دارد. احتمالاً عملی نیست همهٔ خودروها و عابرانِ پیاده را بیابیم و آن‌ها را به موقعیت‌های اصلی‌شان در عکسِ شبانه بازگردانیم تا همان عکس را در روز دوباره بگیریم. برای غلبه بر این دشواری، می‌توان از فنونِ «ترجمهٔ نظارت‌نشده» (Unsupervised Translation) استفاده کرد که قادرند رویِ مثال‌های بسیاری از x و مثال‌های جداگانهٔ بسیاری از y اما بدونِ جفت‌های (x, y) متناظر آموزش ببینند.

این رویکردها عموماً بر GANها مبتنی‌اند؛ برای نمونه، می‌توان مولدِ GANای را آموزش داد که وقتی بر x شرطی می‌شود نمونهٔ واقع‌نمایانه‌ای از y تولید کند، و مولدِ GAN دیگری برای انجامِ نگاشتِ معکوس. چارچوبِ آموزشِ GAN آموزشِ مولد را ممکن می‌کند تا هر یک از نمونه‌های ممکنِ بسیاری را که متمایزگر به‌عنوانِ نمونهٔ واقع‌نمایانهٔ y با توجه به x می‌پذیرد تولید کند، بدونِ هیچ نیازی به y جفت‌شدهٔ خاص، چنان‌که به‌طورِ سنتی در یادگیریِ نظارت‌شده لازم است. جزئیاتِ بیشتر دربارهٔ ترجمهٔ نظارت‌نشده برای تصاویر در بخش ۲۵.۷.۵ آمده است.

۲۱.۷.۲ یادگیریِ انتقالی و یادگیریِ چندوظیفه‌ای

در «یادگیریِ انتقالی» (Transfer Learning)، تجربه با یک تکلیفِ یادگیری به عامل کمک می‌کند در تکلیفِ دیگری بهتر یاد بگیرد. برای نمونه، کسی که تنیس یاد گرفته معمولاً یادگیریِ ورزش‌های وابسته مانندِ راکتبال و اسکواش را آسان‌تر می‌یابد؛ خلبانی که پرواز با یک نوعِ هواپیمایِ مسافربریِ تجاری را یاد گرفته خیلی سریع پرواز با نوعِ دیگری را یاد می‌گیرد؛ دانش‌آموزی که جبر یاد گرفته یادگیریِ حسابان را آسان‌تر می‌یابد.

ما هنوز سازوکارهای یادگیریِ انتقالیِ انسانی را نمی‌دانیم. برای شبکه‌های عصبی، یادگیری شاملِ تنظیمِ وزن‌هاست، پس محتمل‌ترین رویکرد برای یادگیریِ انتقالی کپی‌کردنِ وزن‌های آموخته‌شده برای تکلیفِ A در شبکه‌ای است که برای تکلیفِ B آموزش خواهد دید. سپس وزن‌ها به‌شیوهٔ معمول با استفاده از دادهٔ تکلیفِ B توسط نزولِ گرادیان به‌روزرسانی می‌شوند. ممکن است بسته به اینکه تکلیف‌ها چقدر شبیه‌اند و چقدر داده در تکلیفِ A استفاده شده، استفاده از نرخِ یادگیریِ کوچک‌تری در تکلیفِ B ایدهٔ خوبی باشد.

توجه کنید این رویکرد به تخصصِ انسانی در انتخابِ تکلیف‌ها نیاز دارد: برای نمونه، وزن‌های آموخته‌شده در آموزشِ جبر ممکن است در شبکه‌ای که برای راکتبال در نظر گرفته شده چندان مفید نباشند. همچنین، مفهومِ کپی‌کردنِ وزن‌ها به نگاشتِ ساده‌ای بینِ فضاهای ورودیِ دو تکلیف و معماری‌های شبکهٔ اساساً یکسان نیاز دارد.

یکی از دلایلِ محبوبیتِ یادگیریِ انتقالی در دسترس‌بودنِ مدل‌های ازپیش‌آموزش‌دیدهٔ باکیفیت است. برای نمونه، می‌توانید مدلِ بازشناسیِ اشیای بصریِ ازپیش‌آموزش‌دیده‌ای مانندِ مدلِ ResNet-50 آموزش‌دیده رویِ مجموعهٔ دادهٔ COCO دانلود کنید و خودتان را از هفته‌ها کار نجات دهید. از آن نقطه می‌توانید پارامترهای مدل را با ارائهٔ تصاویر و برچسب‌های شیءِ اضافی برای تکلیفِ خاصِ خود اصلاح کنید.

فرض کنید می‌خواهید انواعِ تک‌چرخه را رده‌بندی کنید. فقط چند صد عکس از تک‌چرخه‌های متفاوت دارید، اما مجموعهٔ دادهٔ COCO در هر یک از رده‌های دوچرخه، موتورسیکلت و اسکیت‌بورد بیش از ۳,۰۰۰ تصویر دارد. این بدان‌معناست مدلِ ازپیش‌آموزش‌دیده رویِ COCO از قبل تجربهٔ چرخ‌ها، جاده‌ها و دیگر ویژگی‌های مربوط را دارد که در تفسیرِ تصاویرِ تک‌چرخه کمک‌کننده خواهند بود.

اغلب می‌خواهید چند لایهٔ نخستِ مدلِ ازپیش‌آموزش‌دیده را منجمد کنید—این لایه‌ها به‌عنوانِ آشکارسازهای ویژگی عمل می‌کنند که برای مدلِ تازهٔ شما مفید خواهند بود. به مجموعهٔ دادهٔ تازهٔ شما اجازه داده می‌شود فقط پارامترهای سطوحِ بالاتر را اصلاح کند؛ این‌ها لایه‌هایی‌اند که ویژگی‌های مسئله‌خاص را شناسایی و رده‌بندی می‌کنند. با این حال، گاهی تفاوتِ بینِ حس‌گذارها به آن معناست که حتی پایین‌ترین سطوحِ لایه‌ها نیز باید بازآموزی شوند.

به‌عنوانِ مثالی دیگر، برای کسانی که سامانهٔ زبانِ طبیعی می‌سازند، اکنون رایج است با مدلِ ازپیش‌آموزش‌دیده‌ای مانندِ مدلِ ROBERTA شروع کنند (بخش ۲۴.۶ را ببینید)، که از قبل دربارهٔ واژگان و نحوِ زبانِ روزمره چیزهای زیادی «می‌داند». گامِ بعدی تنظیمِ دقیقِ مدل به دو شیوه است. نخست، با دادنِ مثال‌هایی از واژگانِ تخصصیِ به‌کاررفته در حوزهٔ مطلوب؛ شاید حوزهٔ پزشکی (جایی که دربارهٔ «سکتهٔ قلبی» یاد می‌گیرد) یا شاید حوزهٔ مالی (جایی که دربارهٔ «مسئولیتِ امانی» یاد می‌گیرد). دوم، با آموزشِ مدل رویِ تکلیفی که قرار است انجام دهد. اگر قرار است پاسخ‌گوییِ پرسش انجام دهد، آن را رویِ جفت‌های پرسش/پاسخ آموزش بده.

یک نوعِ بسیار مهمِ یادگیریِ انتقالی شاملِ انتقال بینِ شبیه‌سازی‌ها و جهانِ واقعی است. برای نمونه، کنترل‌گرِ خودرویِ خودران را می‌توان رویِ میلیاردها مایلِ رانندگیِ شبیه‌سازی‌شده آموزش داد، که در جهانِ واقعی غیرممکن است. سپس، وقتی کنترل‌گر به خودرویِ واقعی منتقل می‌شود، به‌سرعت با محیطِ تازه سازگار می‌شود.

«یادگیریِ چندوظیفه‌ای» (Multitask Learning) شکلی از یادگیریِ انتقالی است که در آن مدلی را به‌طورِ هم‌زمان رویِ چندین هدف آموزش می‌دهیم. برای نمونه، به‌جای آموزشِ سامانهٔ زبانِ طبیعی رویِ برچسب‌گذاریِ نقشِ نحوی و سپس انتقالِ وزن‌های آموخته‌شده به تکلیفِ تازه‌ای مانندِ رده‌بندیِ اسناد، یک سامانه را به‌طورِ هم‌زمان رویِ برچسب‌گذاریِ نقشِ نحوی، رده‌بندیِ اسناد، تشخیصِ زبان، پیش‌بینیِ واژه، مدل‌سازیِ دشواریِ جمله، تشخیصِ سرقتِ ادبی، استلزامِ جمله و پاسخ‌گوییِ پرسش آموزش می‌دهیم. ایده این است که برای حلِ هر یک از این تکلیف‌ها، مدل ممکن است بتواند از ویژگی‌های سطحیِ داده بهره ببرد. اما برای حلِ هر هشت تکلیف به‌طورِ هم‌زمان با لایهٔ بازنماییِ مشترک، مدل به احتمالِ بیشتر بازنماییِ مشترکی می‌سازد که کاربرد و محتواِی واقعیِ زبانِ طبیعی را بازتاب می‌دهد.

۲۱.۸ کاربردها

یادگیری عمیق با موفقیت در بسیاری از حوزه‌های مسئلهٔ مهمِ هوش مصنوعی به‌کار رفته. برای تبیین‌های عمیق، خواننده را به فصل‌های مربوط ارجاع می‌دهیم: فصل ۲۲ برای استفاده از یادگیری عمیق در سامانه‌های یادگیریِ تقویتی، فصل ۲۴ برای پردازشِ زبانِ طبیعی، فصل ۲۵ (به‌ویژه بخش ۲۵.۴) برای بینواییِ رایانه، و فصل ۲۶ برای رباتیک.

۲۱.۸.۱ بینایی

با بینایی رایانه شروع می‌کنیم، که حوزهٔ کاربردیِ است که به‌جرئت می‌توان گفت بیشترین اثر را بر یادگیری عمیق داشته و به‌عکس. هرچند شبکه‌های پیچشیِ عمیق از دههٔ ۱۹۹۰ برای تکلیف‌هایی مانندِ بازشناسیِ دست‌نویس به‌کار می‌رفتند، و شبکه‌های عصبی تا حدودِ ۲۰۱۰ برای بازشناسیِ گفتار شروع به پیشی‌گرفتن از مدل‌های احتمالیِ مولد کرده بودند، موفقیتِ سامانهٔ یادگیریِ عمیقِ AlexNet در رقابتِ ImageNet در ۲۰۱۲ بود که یادگیری عمیق را به کانونِ توجه کشاند.

رقابتِ ImageNet تکلیفِ یادگیریِ نظارت‌شده‌ای با ۱,۲۰۰,۰۰۰ تصویر در ۱,۰۰۰ ردهٔ متفاوت بود، و سامانه‌ها رویِ امتیازِ «۵-برترِ» ارزیابی می‌شدند—اینکه ردهٔ درست چند بار در میانِ پنج پیش‌بینیِ نخست ظاهر می‌شود. AlexNet به نرخِ خطایِ 15.3٪ دست یافت، در حالی که بهترین سامانهٔ بعدی نرخِ خطایی بیش از ۲۵٪ داشت. AlexNet پنج لایهٔ پیچشی داشت که با لایه‌های تجمیعِ حداکثر در هم آمیخته و سپس سه لایهٔ کاملاً متصل. از توابعِ فعال‌سازیِ ReLU استفاده می‌کرد و برای سرعت‌بخشیدن به فرایندِ آموزشِ ۶۰ میلیون وزن از GPUها بهره می‌گرفت. از ۲۰۱۲، با بهبودهای طراحیِ شبکه، روش‌های آموزش و منابعِ محاسباتی، نرخِ خطایِ ۵-برتر به کمتر از ۲٪ کاهش یافته—بسیار پایین‌تر از نرخِ خطایِ انسانِ آموزش‌دیده (حدودِ ۵٪). CNNها برای طیفِ وسیعی از تکلیف‌های بینایی به‌کار رفته‌اند، از خودروهای خودران تا درجه‌بندیِ خیار.[8] رانندگی، که در بخش ۲۵.۷.۶ و چند بخشِ فصل ۲۶ پوشش داده شده، از دشوارترین تکلیف‌های بینایی است: الگوریتم نه‌تنها باید کبوترها، کیسه‌های کاغذی و عابرانِ پیاده را تشخیص دهد، مکان‌یابی کند، ردیابی کند و بازشناسی کند، بلکه باید این کار را در زمانِ واقعی و با دقتی نزدیکِ کامل انجام دهد.

۲۱.۸.۲ پردازشِ زبانِ طبیعی

یادگیری عمیق همچنین اثرِ عظیمی بر کاربردهای پردازشِ زبانِ طبیعی (NLP) مانندِ ترجمهٔ ماشینی و بازشناسیِ گفتار داشته. برخی مزیت‌های یادگیری عمیق برای این کاربردها شاملِ امکانِ یادگیریِ سرتاسری، تولیدِ خودکارِ بازنمایی‌های درونی برای معانیِ واژه‌ها، و قابلیتِ جایگزینیِ رمزگذارها و رمزگشاهای آموخته‌شده است.

یادگیریِ سرتاسری به ساختِ کلِ سامانه‌ها به‌عنوانِ تابعِ آموخته‌شدهٔ واحدی اشاره دارد. برای نمونه، f برای ترجمهٔ ماشینی ممکن است جملهٔ انگلیسیِ S_E را به‌عنوانِ ورودی بگیرد و جملهٔ ژاپنیِ هم‌ارزِ S_J = f(S_E) تولید کند. چنین fای را می‌توان از دادهٔ آموزشی در قالبِ جفتِ جمله‌های ترجمه‌شده توسط انسان یاد گرفت (یا حتی جفتِ متن‌ها، که ترازِ جمله‌ها یا عبارت‌های متناظر بخشی از مسئله‌ای است که باید حل شود). رویکردِ خطِ لولهٔ کلاسیک‌تر ممکن است نخست S_E را تجزیه کند، سپس معنایش را استخراج کند، سپس معنا را در ژاپنی به‌صورتِ لـS_J بازبیان کند، سپس لـS_J را با استفاده از مدلِ زبانِ ژاپنی پس‌ویرایش کند. این رویکردِ خطِ لوله دو کاستیِ عمده دارد: نخست، خطاها در هر مرحله تشدید می‌شوند؛ و دوم، انسان‌ها باید تعیین کنند «درختِ تجزیه» و «بازنماییِ معنا» چیست، اما حقیقتِ زمینیِ به‌آسانی در دسترسی برای این مفاهیم نیست، و ایده‌های نظریِ ما دربارهٔ آن‌ها تقریباً به‌طورِ قطع ناقص‌اند.

پس در مرحلهٔ کنونیِ فهمِ ما، رویکردِ کلاسیکِ خطِ لوله—که، دست‌کم به‌طورِ ساده‌لوحانه، به نظر می‌رسد با شیوهٔ کارِ مترجمِ انسانی متناظر است—توسط روشِ سرتاسریِ ممکن‌شده توسط یادگیری عمیق شکست می‌خورد. برای نمونه، وو و همکاران (2016b) نشان دادند ترجمهٔ سرتاسری با یادگیری عمیق خطاهای ترجمه را نسبت به سامانهٔ قبلیِ مبتنی بر خطِ لوله ۶۰٪ کاهش داد. از ۲۰۲۰، سامانه‌های ترجمهٔ ماشینی برای جفت‌زبان‌هایی مانندِ فرانسوی و انگلیسی که

مجموعه‌های دادهٔ جفت‌شدهٔ بسیار بزرگی برایشان در دسترس است به عملکردِ انسانی نزدیک می‌شوند، و برای دیگر جفت‌زبان‌هایی که اکثریتِ جمعیتِ زمین را پوشش می‌دهند قابلِ استفاده‌اند. حتی شواهدی هست که شبکه‌های آموزش‌دیده رویِ چند زبان در واقع بازنماییِ معنایِ درونی یاد می‌گیرند: برای نمونه، پس از یادگیریِ ترجمه از پرتغالی به انگلیسی و از انگلیسی به اسپانیایی، امکانِ ترجمهٔ مستقیم از پرتغالی به اسپانیایی بدونِ هیچ جفتِ جملهٔ پرتغالی/اسپانیایی در مجموعهٔ آموزشی هست.

یکی از مهم‌ترین یافته‌های برآمده از کاربردِ یادگیری عمیق در تکلیف‌های زبانی این است که بخشِ بزرگی از کاراییِ به‌دست‌آمده از بازنماییِ دوبارهٔ واژه‌های منفرد به‌عنوانِ بردارهایی در فضایِ پربعد ناشی می‌شود—به‌اصطلاح «جاسازیِ واژه‌ها» (Word Embeddings) (بخش ۲۴.۱ را ببینید). بردارها معمولاً از وزن‌های نخستین لایهٔ پنهانِ شبکه‌ای استخراج می‌شوند که رویِ حجمِ عظیمی از متن آموزش دیده، و آمارِ بافت‌های واژگانی را دربرمی‌گیرند که واژه‌ها در آن‌ها به‌کار می‌روند. چون واژه‌های با معانیِ مشابه در بافت‌های مشابه به‌کار می‌روند، در فضایِ برداری نزدیکِ هم می‌افتند. این به شبکه اجازه می‌دهد به‌طورِ مؤثر در میانِ رده‌های واژه‌ها تعمیم دهد، بدونِ نیاز به اینکه انسان‌ها آن رده‌ها را از پیش تعریف کنند. برای نمونه، جمله‌ای که با «جان یک هندوانه و دو پوند ... خرید» شروع می‌شود به احتمالِ زیاد با «سیب» یا «موز» ادامه می‌یابد اما نه با «توریم» یا «جغرافیا». چنین پیش‌بینی‌ای بسیار آسان‌تر است اگر «سیب» و «موز» بازنمایی‌های مشابهی در لایهٔ درونی داشته باشند.

۲۱.۸.۳ یادگیریِ تقویتی

در یادگیریِ تقویتی (RL)، عاملِ تصمیم‌گیر از دنباله‌ای از سیگنال‌های پاداش یاد می‌گیرد که نشانه‌ای از کیفیتِ رفتارش فراهم می‌کنند. هدف، بهینه‌کردنِ مجموعِ پاداش‌های آینده است. این را می‌توان به چند شیوه انجام داد: در اصطلاح‌شناسیِ فصل ۱۷، عامل می‌تواند تابعِ ارزش، تابعِ Q، سیاست و مانندِ آن یاد بگیرد. از دیدگاهِ یادگیری عمیق، همهٔ این‌ها توابعی‌اند که می‌توانند توسط گراف‌های محاسباتی بازنمایی شوند.

برای نمونه، تابعِ ارزش در بازیِ گو موقعیتِ صفحه را به‌عنوانِ ورودی می‌گیرد و برآوردی از میزانِ سودمندیِ موقعیت برای عامل برمی‌گرداند. در حالی که روش‌های آموزش در RL با یادگیریِ نظارت‌شده متفاوت‌اند، توانایی گراف‌های محاسباتیِ چندلایه در بازنماییِ توابعِ پیچیده رویِ فضاهای ورودیِ بزرگ بسیار مفید ثابت شده. حوزهٔ پژوهشیِ حاصل «یادگیریِ تقویتیِ عمیق» (Deep Reinforcement Learning) نامیده می‌شود.

در دههٔ ۱۹۵۰، آرتور ساموئل با بازنمایی‌های چندلایهٔ توابعِ ارزش در کارش رویِ یادگیریِ تقویتی برای بازیِ چکرز آزمایش کرد، اما دریافت در عمل تقریب‌گرِ تابعِ خطی بهترین کارایی را دارد. (این ممکن است پیامدِ کارکردن با رایانه‌ای بوده باشد که تقریباً ۱۰۰ میلیارد بار از یک واحدِ پردازشِ تانسوریِ مدرن ضعیف‌تر بود.) نخستین نمایشِ موفقِ بزرگِ یادگیریِ تقویتیِ عمیق عاملِ بازیِ آتاریِ دیپ‌مایند، DQN بود (منیه و همکاران، 2013). کپی‌های متفاوتی از این عامل برای بازیِ هر یک از چند بازیِ ویدئوییِ متفاوتِ آتاری آموزش دیدند، و مهارت‌هایی مانندِ تیراندازی به سفینه‌های فضاییِ بیگانه، زدنِ توپ با راکت و رانندگیِ خودروهای مسابقه‌ایِ شبیه‌سازی‌شده را نشان دادند. در هر مورد، عامل تابعِ Q را از دادهٔ تصویرِ خام یاد گرفت و سیگنالِ پاداش، امتیازِ بازی بود. کارهای بعدی سامانه‌های یادگیریِ تقویتیِ عمیقی تولید کرده‌اند که در اکثریتِ ۵۷ بازیِ متفاوتِ آتاری در سطحی فراتر از انسان بازی می‌کنند. سامانهٔ ALPHAGO دیپ‌مایند نیز از یادگیریِ تقویتیِ عمیق برای شکستِ بهترین بازیکنانِ انسانیِ بازیِ گو استفاده کرد (فصل ۵ را ببینید).

با وجودِ موفقیت‌های چشمگیرش، یادگیریِ تقویتیِ عمیق هنوز با موانعِ چشمگیری روبه‌روست: اغلب به‌دست‌آوردنِ عملکردِ خوب دشوار است، و اگر محیط حتی اندکی با دادهٔ آموزشی تفاوت داشته باشد، سامانهٔ آموزش‌دیده ممکن است بسیار غیرقابلِ‌پیش‌بینی رفتار کند (ایرپن، 2018). در مقایسه با دیگر کاربردهای یادگیری عمیق، یادگیریِ تقویتیِ عمیق به‌ندرت در محیط‌های تجاری به‌کار می‌رود. با این حال، حوزهٔ پژوهشیِ بسیار فعالی است.

خلاصهٔ فصل

این فصل روش‌های یادگیریِ توابعِ بازنمایی‌شده توسط گراف‌های محاسباتیِ عمیق را توصیف کرد. نکته‌های اصلی چنین بودند:

- شبکه‌های عصبی توابعِ غیرخطیِ پیچیده را با شبکه‌ای از واحدهای آستانهٔ خطیِ پارامتری‌شده بازنمایی می‌کنند.
- الگوریتمِ پسانتشار نزولِ گرادیانی را در فضایِ پارامتر برای کمینه‌کردنِ تابعِ زیان پیاده‌سازی می‌کند.
- یادگیری عمیق برای بازشناسیِ اشیای بصری، بازشناسیِ گفتار، پردازشِ زبانِ طبیعی و یادگیریِ تقویتی در محیط‌های پیچیده خوب کار می‌کند.
- شبکه‌های پیچشی به‌ویژه برای پردازشِ تصویر و دیگر تکلیف‌هایی که داده در آن‌ها توپولوژیِ شبکه‌ای دارد مناسب‌اند.
- شبکه‌های بازگشتی برای تکلیف‌های پردازشِ دنباله‌ای از جملهِ مدل‌سازیِ زبان و ترجمهٔ ماشینی مؤثرند.

یادداشت‌های کتابشناختی و تاریخی

ادبیاتِ شبکه‌های عصبی گسترده است. کوان و شارپ (1988b, 1988a) تاریخچهٔ اولیه را مرور می‌کنند، که با کارِ مک‌کالوک و پیتس (1943) آغاز می‌شود. (چنان‌که در فصل ۱ اشاره شد، جان مک‌کارتی به کارِ نیکلاس راشفسکی (1936, 1938) به‌عنوانِ نخستین مدلِ ریاضیِ یادگیریِ عصبی اشاره کرده.) نوربرت وینر، پیشگامِ سایبرنتیک و نظریهٔ کنترل (وینر، 1948)، با مک‌کالوک و پیتس کار کرد و بر شماری از پژوهشگرانِ جوان، از جملهِ ماروین مینسکی، اثر گذاشت، که احتمالاً نخستین کسی بود که شبکهٔ عصبیِ کارآمدی را در سخت‌افزار توسعه داد، در ۱۹۵۱ (مینسکی و پاپرت، 1988، صص. ix–x را ببینید).

آلن تورینگ (1948) گزارشِ پژوهشیِ «ماشین‌های هوشمند» را نوشت که با جملهٔ «من پیشنهاد می‌کنم این پرسش را بررسی کنم که آیا ممکن است ماشین‌ها رفتارِ هوشمندانه نشان دهند» آغاز می‌شود و سپس معماریِ شبکهٔ عصبیِ بازگشتیِ «ماشین‌های سازمان‌نیافتهٔ نوع B» و رویکردی برای آموزشِ آن‌ها را توصیف می‌کند. متأسفانه، گزارش تا ۱۹۶۹ منتشر نشد و تا همین اواخر تقریباً نادیده گرفته می‌شد.

پرسپترون، شبکهٔ عصبیِ تک‌لایه با تابعِ فعال‌سازیِ آستانهٔ سخت، توسط فرانک روزن‌بلات (1957) رایج شد. پس از نمایشی در ژوئیهٔ ۱۹۵۸، نیویورک‌تایمز آن را «جنینِ رایانهٔ الکترونیکی که [نیروی دریایی] انتظار دارد بتواند راه برود، حرف بزند، ببیند، بنویسد، خودش را بازتولید کند و از وجودِ خودش آگاه باشد» توصیف کرد. روزن‌بلات (1960) بعداً قضیهٔ همگراییِ پرسپترون را اثبات کرد، هرچند توسط کارِ صرفاً ریاضیِ خارج از بافتِ شبکه‌های عصبی پیش‌بینی شده بود (اگمون، 1954؛ موتسکین و شوئنبرگ، 1954). برخی کارهای اولیه نیز رویِ شبکه‌های چندلایه انجام شد، از جملهِ پرسپترون‌های گامبا (گامبا و همکاران، 1961) و مادالین‌ها (ویدرو، 1962). کتابِ «ماشین‌های یادگیرنده» (نیلسون، 1965) بخشِ بزرگی از این کارِ اولیه و بیشتر را پوشش می‌دهد. زوالِ بعدیِ تلاش‌های اولیهٔ پژوهشِ پرسپترون توسط کتابِ «پرسپترون‌ها» (مینسکی و پاپرت، 1969) شتاب گرفت (یا، چنان‌که نویسندگان بعداً ادعا کردند، صرفاً تبیین شد)، که از کمبودِ دقتِ ریاضیِ حوزه اظهارِ تأسف می‌کرد. این کتاب اشاره کرد پرسپترون‌های تک‌لایه فقط می‌توانند مفاهیمِ خطیِ جدایی‌پذیر را بازنمایی کنند و به فقدانِ الگوریتم‌های یادگیریِ مؤثر برای شبکه‌های چندلایه اشاره کرد. این محدودیت‌ها از قبل شناخته‌شده بودند (هاوکینز، 1961) و توسط خودِ روزن‌بلات (1962) نیز پذیرفته شده بودند.

مقاله‌های گردآوری‌شده توسط هینتون و اندرسون (1981)، بر مبنای کنفرانسی در سن‌دیگو در ۱۹۷۹، را می‌توان نشانهٔ رنسانسِ پیوندگرایی دانست. گلچینِ دو جلدیِ «PDP» (پردازشِ توزیع‌شدهٔ موازی) (روملهارت و مک‌کللند، 1986) به انتشارِ این اندیشه، به‌اصطلاح، به‌ویژه در جامعه‌های روان‌شناسی و علومِ شناختی کمک کرد. مهم‌ترین توسعهٔ این دوره الگوریتمِ پسانتشار برای آموزشِ شبکه‌های چندلایه بود.

الگوریتمِ پسانتشار چند بار به‌طورِ مستقل در بافت‌های متفاوت کشف شد (کلی، 1960؛ برایسون، 1962؛ دریفوس، 1962؛ برایسون و هو، 1969؛ وربوس، 1974؛ پارکر، 1985) و استوارت دریفوس (1990) آن را «رویهٔ گرادیانیِ کلی–برایسون» می‌نامد. هرچند وربوس آن را بر شبکه‌های عصبی اعمال کرده بود، این ایده تا انتشارِ مقاله‌ای از دیوید روملهارت، جف هینتون و ران ویلیامز (1986) در مجلهٔ نیچر که ارائه‌ای غیرریاضی از الگوریتم می‌داد به‌طورِ گسترده شناخته نشد. اعتبارِ ریاضی توسط مقاله‌هایی افزایش یافت که نشان می‌دادند شبکه‌های پیشخورِ چندلایه (مشروط به شرایطِ فنی) تقریب‌زننده‌های همگانیِ تابع‌اند (سیبنکو، 1988, 1989). اواخرِ دههٔ ۱۹۸۰ و اوایلِ دههٔ ۱۹۹۰ رشدِ عظیمی در پژوهشِ شبکه‌های عصبی دید: تعدادِ مقاله‌ها بینِ 1980–84 و 1990–94 به‌ضریبِ ۲۰۰ قارچ‌گونه رشد کرد.

در اواخرِ دههٔ ۱۹۹۰ و اوایلِ دههٔ ۲۰۰۰، علاقه به شبکه‌های عصبی رو به افول نهاد، زیرا فنونِ دیگری مانندِ شبکه‌های بیزی، روش‌های گروهی و ماشین‌های هسته‌ای به‌پیش آمدند. علاقه به مدل‌های عمیق وقتی شعله‌ور شد که پژوهشِ جف هینتون رویِ شبکه‌های بیزیِ عمیق—مدل‌های مولد با متغیرهای رده‌ای در ریشه و متغیرهای شاهد در برگ‌ها—ثمر داد و رویِ مجموعه‌های دادهٔ معیارِ کوچک از ماشین‌های هسته‌ای پیشی گرفت (هینتون و همکاران، 2006).

علاقه به یادگیری عمیق وقتی منفجر شد که کریژفسکی و همکاران (2013) با شبکه‌های پیچشیِ عمیق رقابتِ ImageNet را بردند (روساکوفسکی و همکاران، 2015).

نظردهندگان اغلب از در دسترس‌بودنِ «دادهٔ بزرگ» و قدرتِ پردازشیِ GPUها به‌عنوانِ عواملِ اصلیِ ظهورِ یادگیری عمیق یاد می‌کنند. بهبودهای معماری نیز مهم بودند، از جملهِ پذیرشِ تابعِ فعال‌سازیِ ReLU به‌جای سیگموئیدِ لجستیک (جارت و همکاران، 2009؛ نایر و هینتون، 2010؛ گلورو و همکاران، 2011) و بعدها توسعهٔ شبکه‌های باقی‌مانده (هه و همکاران، 2016).

در سمتِ الگوریتمی، استفاده از نزولِ گرادیانِ تصادفی (SGD) با دسته‌های کوچک برای مقیاس‌پذیرکردنِ شبکه‌های عصبی به مجموعه‌های دادهٔ بزرگ ضروری بود (بوتو و بوسکه، 2008). نرمال‌سازیِ دسته‌ای (یوفه و سزگدی، 2015) نیز به سریع‌تر و قابلِ‌اعتمادترشدنِ فرایندِ آموزش کمک کرد و چند فنِ نرمال‌سازیِ اضافی دیگر را زایید (با و همکاران، 2016؛ وو و هه، 2018؛ میاتو و همکاران، 2018). چند مقاله رفتارِ تجربیِ SGD را رویِ شبکه‌های بزرگ و مجموعه‌های دادهٔ بزرگ مطالعه کرده‌اند (دوپن و همکاران، 2015؛ خورومانسکا و همکاران، 2014؛ گودفلو و همکاران، 2015b). در سمتِ نظری، پیشرفتی در تبیینِ مشاهده‌ای حاصل شده که SGD اعمال‌شده بر شبکه‌های بیش‌پارامتری اغلب با خطایِ آموزشیِ صفر به کمینهٔ سراسری می‌رسد، هرچند تاکنون قضیه‌هایِ به‌این‌مضمون شبکه‌ای با لایه‌هایی بسیار عریض‌تر از آنچه در عمل رخ می‌دهد فرض می‌کنند (آلن-ژو و همکاران، 2018؛ دو و همکاران، 2018). چنین شبکه‌هایی ظرفیتِ بیش‌ازحدِ کافی برای عملکردنِ به‌عنوانِ جدولِ مراجعه برای دادهٔ آموزشی دارند. آخرین قطعهٔ پازل، دستِ‌کم برای کاربردهای بینایی، استفاده از شبکه‌های پیچشی بود. این‌ها ریشه‌هایشان در توصیفِ سامانهٔ بینایِی پستانداران توسط عصب‌شناسانِ دیوید هیوبل و تورستن ویزل است (هیوبل و ویزل، 1959, 1962, 1968).

آن‌ها «سلول‌های ساده» را در سامانهٔ بینایِی گربه توصیف کردند که شبیهِ آشکارسازهای لبه‌اند، و نیز «سلول‌های پیچیده» را که نسبت به برخی تبدیل‌ها مانندِ انتقال‌های فضاییِ کوچک ناوردایند. در شبکه‌های پیچشیِ مدرن، خروجیِ یک پیچش مشابهِ سلولِ ساده است، در حالی که خروجیِ یک لایهٔ تجمیع مشابهِ سلولِ پیچیده.

کارِ هیوبل و ویزل بسیاری از مدل‌های پیوندگرایِ اولیهٔ بینایی را الهام بخشید (مار و پوجیو، 1976). نئوکوگنیترون (فوکوشیما، 1980؛ فوکوشیما و میاکه، 1982)، که به‌عنوانِ مدلی از قشرِ بینایی طراحی شده، از نظرِ معماریِ مدل اساساً شبکهٔ پیچشی بود، هرچند الگوریتمِ آموزشیِ مؤثرِ چنین شبکه‌هایی باید تا وقتی که یان لکون و همکاران نشان دادند چگونه پسانتشار اعمال شود (لکون و همکاران، 1995) معطل می‌ماند. یکی از موفقیت‌های تجاریِ اولیهٔ شبکه‌های عصبی بازشناسیِ ارقامِ دست‌نویس با استفاده از شبکه‌های پیچشی بود (لکون و همکاران، 1995).

شبکه‌های عصبیِ بازگشتی (RNNها) در دههٔ ۱۹۷۰ معمولاً به‌عنوانِ مدل‌هایی از عملکردِ مغز پیشنهاد می‌شدند، اما هیچ الگوریتمِ یادگیریِ مؤثری با این پیشنهادها همراه نبود. روشِ پسانتشار در طولِ زمان در پایان‌نامهٔ دکترایِ پل وربوس (1974) ظاهر می‌شود، و مقالهٔ مرورِ بعدی‌اش (وربوس، 1990) مرجع‌های اضافیِ چندانی برای بازکشف‌های این روش در دههٔ ۱۹۸۰ می‌دهد. یکی از تأثیرگذارترین کارهای اولیه رویِ RNNها متعلق به جف المان (1990) است، که بر معماریِ RNN پیشنهادشده توسط مایکل جردن (1986) بنا شده. ویلیامز و زیپسر (1989) الگوریتمی برای یادگیریِ برخط در RNNها ارائه می‌کنند. بنجیو و همکاران (1994) مسئلهٔ گرادیانِ ناپدیدشونده در شبکه‌های بازگشتی را تحلیل کردند. معماریِ حافظهٔ کوتاه‌مدت-بلندمدت (LSTM) (هوشرایتر، 1991؛ هوشرایتر و اشمیدهوبر، 1997؛ گرز و همکاران، 2000) به‌عنوانِ راهی برای اجتناب از این مسئله پیشنهاد شد. اخیراً، طراحی‌های مؤثرِ RNN به‌صورتِ خودکار اشتقاق شده‌اند (جوزفوویچ و همکاران، 2015؛ زوف و له، 2016).

روش‌های بسیاری برای بهبودِ تعمیم در شبکه‌های عصبی امتحان شده. کاهشِ وزن توسط هینتون (1987) پیشنهاد و توسط کروگ و هرتز (1992) به‌صورتِ ریاضی تحلیل شد. روشِ دراپ‌اوت متعلق به سریواستاوا و همکاران (2014a) است. سزگدی و همکاران (2013) ایدهٔ مثال‌های تخاصمی را معرفی کردند و ادبیاتِ عظیمی را زاییدند.

پول و همکاران (2017) نشان دادند شبکه‌های عمیق (اما نه کم‌عمق) می‌توانند توابعِ پیچیده را به خمینه‌های تخت در فضایِ واحدهای پنهان تجزیه کنند. رولنیک و تگمارک (2018) نشان دادند تعدادِ واحدهای لازم برای تقریبِ رده‌ای خاص از چندجمله‌ای‌های n متغیره برای شبکه‌های کم‌عمق به‌صورتِ نمایی و برای شبکه‌های عمیق فقط به‌صورتِ خطی رشد می‌کند.

وایت و همکاران (2019) نشان دادند سامانهٔ BANANAS آن‌ها می‌تواند جستجوی معماریِ عصبی (NAS) را با پیش‌بینیِ دقتِ شبکه تا درونِ ۱٪ پس از آموزش رویِ فقط ۲۰۰ معماریِ نمونهٔ تصادفی انجام دهد. زوف و له (2016) از یادگیریِ تقویتی برای جستجو در فضایِ معماری‌های شبکهٔ عصبی استفاده می‌کنند. رئال و همکاران (2018) از الگوریتمِ تکاملی برای انتخابِ مدل استفاده می‌کنند، لیو و همکاران (2017) از الگوریتم‌های تکاملی رویِ بازنمایی‌های سلسله‌مراتبی استفاده می‌کنند، و یادربِرگ و همکاران (2017) آموزشِ مبتنی بر جمعیت را توصیف می‌کنند. لیو و همکاران (2019) فضایِ معماری‌ها را به فضایِ پیوستهٔ مشتق‌پذیر سست می‌کنند و از نزولِ گرادیان برای یافتنِ راه‌حلِ بهینهٔ محلی استفاده می‌کنند. فام و همکاران (2018) سامانهٔ ENAS (جستجوی کارآمدِ معماریِ عصبی) را توصیف می‌کنند، که به جستجوی زیرگراف‌های بهینهٔ گرافِ بزرگ‌تری می‌پردازد. این سریع است، زیرا نیازی به بازآموزیِ پارامترها ندارد. ایدهٔ جستجوی زیرگراف به الگوریتمِ «آسیبِ بهینهٔ مغز» لکون و همکاران (1990) بازمی‌گردد.

با وجودِ این مجموعهٔ چشمگیر از رویکردها، منتقدانی هستند که احساس می‌کنند این حوزه هنوز به بلوغ نرسیده. یو و همکاران (2019) نشان می‌دهند در برخی موارد این الگوریتم‌های NAS کارآمدتر از انتخابِ تصادفیِ معماری نیستند. برای مروری بر نتایجِ اخیر در جستجوی معماریِ عصبی، السن و همکاران (2018) را ببینید. یادگیریِ نظارت‌نشده زیرشاخهٔ بزرگی در آمار تشکیل می‌دهد، بیشتر تحتِ عنوانِ برآوردِ چگالی. سیلورمن (1986) و مورفی (2012) منابعِ خوبی برای فنونِ کلاسیک و مدرن در این حوزه‌اند. تحلیلِ مؤلفه‌های اصلی (PCA) به پیرسون (1901) بازمی‌گردد؛ این نام از کارِ مستقلِ هاتلینگ (1933) می‌آید. مدلِ PCA احتمالاتی (تیپینگ و بیشاپ، 1999) مدلِ مولدِ مؤلفه‌های اصلی را به‌خودِ آن‌ها می‌افزاید. خودرمزگذارِ تغییراتی متعلق به کینگما و ولینگ (2013) و رزنده و همکاران (2014) است؛ جردن و همکاران (1999) مقدمه‌ای بر روش‌های تغییراتی برای استنتاج در مدل‌های گرافیکی ارائه می‌دهند.

برای مدل‌های خودرگرسیونی، متنِ کلاسیک متعلق به باکس و همکاران (2016) است. معادله‌های یول–واکر برای برازشِ مدل‌های AR به‌طورِ مستقل توسط یول (1927) و واکر (1931) توسعه یافتند.

مدل‌های خودرگرسیونی با وابستگی‌های غیرخطی توسط چند نویسنده توسعه یافتند (فرای، 1998؛ بنجیو و بنجیو، 2001؛ لاروشل و موری، 2011). مدلِ خودرگرسیونیِ WaveNet (فن‌دن‌اورد و همکاران، 2016a) بر کارهای اولیهٔ تولیدِ تصویرِ خودرگرسیونی بنا شده بود (فن‌دن‌اورد و همکاران، 2016b). شبکه‌های مولدِ تخاصمی، یا GANها، نخستین بار توسط گودفلو و همکاران (2015a) پیشنهاد شدند و کاربردهای بسیاری در هوش مصنوعی یافته‌اند. درکی نظری از ویژگی‌هایشان در حالِ ظهور است که به مدل‌ها و الگوریتم‌های GAN بهبودیافته می‌انجامد (لی و مالیک، 2018b, 2018a؛ ژو و همکاران، 2019). بخشی از آن درک شاملِ محافظت در برابرِ حملاتِ تخاصمی است (کارلینی و همکاران، 2019).

چند شاخه از پژوهشِ شبکه‌های عصبی در گذشته محبوب بوده‌اند اما امروزه به‌طورِ فعال بررسی نمی‌شوند. شبکه‌های هاپفیلد (هاپفیلد، 1982) اتصال‌های متقارنی بینِ هر جفتِ گره دارند و می‌توانند یاد بگیرند الگوها را در حافظهٔ تداعی‌گر ذخیره کنند، به‌گونه‌ای که کلِ یک الگو را بتوان با فهرست‌بندیِ حافظه با استفاده از تکه‌ای از الگو بازیابی کرد. شبکه‌های هاپفیلد قطعی‌اند؛ بعدها به ماشین‌های بولتزمنِ تصادفی تعمیم یافتند (هینتون و ساینوسکی، 1983, 1986). ماشین‌های بولتزمن احتمالاً نخستین نمونهٔ مدلِ مولدِ عمیق‌اند. دشواریِ استنتاج در ماشین‌های بولتزمن به پیشرفت‌هایی هم در فنونِ مونت‌کارلو و هم در فنونِ تغییراتی انجامید (بخش ۱۳.۴ را ببینید). پژوهشِ شبکه‌های عصبی برای هوش مصنوعی تا حدی با پژوهشِ شبکه‌های عصبیِ زیستی نیز در هم تنیده بوده. دو موضوع در دههٔ ۱۹۴۰ بر هم منطبق شدند، و ایده‌های شبکه‌های پیچشی و یادگیریِ تقویتی را می‌توان به مطالعهٔ سامانه‌های زیستی بازجست؛ اما در حالِ حاضر، ایده‌های تازه در یادگیری عمیق تمایل دارند بر ملاحظاتِ صرفاً محاسباتی یا آماری مبتنی باشند. حوزهٔ «عصب‌شناسیِ محاسباتی» (Computational Neuroscience) هدفش ساختِ مدل‌های محاسباتی است که ویژگی‌های مهم و خاصِ سامانه‌های زیستیِ واقعی را درمی‌گیرند. مرورها توسط دیان و ابوت (2001) و تراپنبرگ (2010) ارائه شده‌اند.

برای شبکه‌های عصبیِ مدرن و یادگیری عمیق، کتاب‌های درسیِ پیشتاز متعلق به گودفلو و همکاران (2016) و چارنیاک (2018) است. همچنین راهنماهای عملیِ بسیاری همراهِ بسته‌های نرم‌افزاریِ متن‌بازِ گوناگونِ یادگیری عمیق هست. سه رهبرِ این حوزه—یان لکون، یوشوا بنجیو و جف هینتون—ایده‌های کلیدی را برای پژوهشگرانِ غیرِ هوش مصنوعی در مقاله‌ای تأثیرگذار در مجلهٔ نیچر معرفی کردند (2015). این سه دریافت‌کنندگانِ جایزهٔ تورینگِ ۲۰۱۸ بودند. اشمیدهوبر (2015) نمایِ کلیِ عمومی ارائه می‌دهد، و دنگ و همکاران (2014) رویِ تکلیف‌های پردازشِ سیگنال تمرکز می‌کنند.

پانوشت‌ها:

[1] روش‌های تمایزِ خودکار در اصل در دهه‌های ۱۹۶۰ و ۱۹۷۰ برای بهینه‌سازیِ پارامترهای سامانه‌های تعریف‌شده توسط برنامه‌های بزرگ و پیچیدهٔ فورترن توسعه یافتند.

[2] آنتروپیِ متقاطع به معنایِ معمول فاصله نیست، زیرا $H(P,P)$ صفر نیست؛ بلکه برابرِ آنتروپیِ $H(P)$ است. به‌آسانی نشان داده می‌شود $H(P,Q) = H(P) + D_{KL}(P\|Q)$، که در آن D_{KL} واگراییِ کولبک–لیبلر است، که $D_{KL}(P\|P) = 0$ را برآورده می‌کند. پس برای P ثابت، تغییرِ Q برای کمینه‌کردنِ آنتروپیِ متقاطع، واگراییِ KL را نیز کمینه می‌کند.

[3] ایده‌های مشابه را می‌توان برای پردازشِ منابعِ دادهٔ سریِ زمانی مانندِ شکل‌موج‌های صوتی به‌کار برد. این‌ها معمولاً ناوردایی زمانی از خود نشان می‌دهند—واژه‌ای بدونِ توجه به اینکه در چه ساعتی از روز گفته شود یکسان به نظر می‌رسد. شبکه‌های عصبیِ بازگشتی (بخش ۲۱.۶) به‌صورتِ خودکار ناوردایی زمانی از خود نشان می‌دهند.

[4] در اصطلاح‌شناسیِ پردازشِ سیگنال، این عملیات را «همبستگیِ متقاطع» می‌نامیم، نه پیچش. اما «پیچش» در حوزهٔ شبکه‌های عصبی استفاده می‌شود.

[5] تعریفِ ریاضیِ درستِ تانسورها مستلزمِ برقراریِ ناوردایی‌های معینی تحتِ تغییرِ پایه است. کاربردِ این اصطلاح در یادگیری عمیق از آن تعریفِ ریاضیِ سخت‌گیرانه ساده‌تر است.

[6] با اشاره به اینکه بخشِ زیادی از این کارِ تدریجی و اکتشافی توسط دانشجویانِ تحصیلاتِ تکمیلی انجام می‌شود، برخی آن را «نزولِ دانشجویِ تحصیلاتِ تکمیلی» (GSD) نامیده‌اند.

[7] PCA استاندارد شاملِ برازشِ گاوسیِ چندمتغیره به دادهٔ خامِ ورودی و سپس انتخابِ بلندترین محورها—مؤلفه‌های اصلی—آن توزیعِ بیضوی است.

[8] داستانِ مشهورِ کشاورزِ خیارکارِ ژاپنی که با TensorFlow رباتِ مرتب‌کنندهٔ خیار خودش را ساخت، معلوم می‌شود بیشتر افسانه است. الگوریتم توسط پسرِ کشاورز ساخته شده بود، که قبلاً به‌عنوانِ مهندسِ نرم‌افزار در تویوتا کار می‌کرده، و دقتِ پایینِ آن—حدودِ ۷۰٪—به این معنا بود که خیارها همچنان باید با دست مرتب می‌شدند (زیبرگ، 2017).

فصل ۲۲: یادگیری تقویتی (Reinforcement Learning)

در این فصل می‌بینیم چگونه تجربه‌کردنِ پاداش‌ها و تنبیه‌ها می‌تواند به عامل بیاموزد چگونه پاداش‌ها را در آینده بیشینه کند.

در یادگیریِ نظارت‌شده، عامل با مشاهدهٔ منفعلانهٔ جفت‌های ورودی/خروجیِ نمونه‌ای که توسط یک «معلم» فراهم شده یاد می‌گیرد. در این فصل، خواهیم دید چگونه عامل‌ها می‌توانند به‌طورِ فعال از تجربهٔ خودشان یاد بگیرند، بدونِ معلم، با درنظرگرفتنِ موفقیت یا شکستِ نهایی‌شان.

۲۲.۱ یادگیری از پاداش‌ها

مسئلهٔ یادگیریِ بازیِ شطرنج را در نظر بگیرید. بیایید تصور کنیم این را با روش‌های فصل‌های ۱۹ تا ۲۱ به‌عنوانِ مسئلهٔ یادگیریِ نظارت‌شده در نظر بگیریم. تابعِ عاملِ شطرنج‌باز موقعیتِ صفحه را به‌عنوانِ ورودی می‌گیرد و حرکتی را برمی‌گرداند، پس این تابع را با ارائهٔ نمونه‌هایی از موقعیت‌های شطرنج، که هر یک با حرکتِ درست برچسب خورده، آموزش می‌دهیم. حالا، اتفاقاً پایگاه‌های داده‌ای از چند میلیون بازیِ استادبزرگان در دسترس داریم، که هر یک دنباله‌ای از موقعیت‌ها و حرکت‌هاست. حرکت‌های انجام‌شده توسط برنده، با چند استثنا، خوب فرض می‌شوند، اگر نه همیشه کامل. پس مجموعهٔ آموزشیِ امیدوارکننده‌ای داریم. مشکل این است که در مقایسه با فضایِ همهٔ موقعیت‌های شطرنجِ ممکن (حدودِ 10^{40})، مثال‌ها نسبتاً کم‌اند (حدودِ 10^{8}). در بازیِ تازه، به‌زودی با موقعیت‌هایی روبه‌رو می‌شویم که به‌طورِ چشمگیری با موقعیت‌های پایگاهِ داده متفاوت‌اند، و تابعِ عاملِ آموزش‌دیده به احتمالِ زیاد به‌شکلِ فاجعه‌باری شکست می‌خورد—و نه فقط به‌دلیلِ اینکه هیچ ایده‌ای ندارد حرکت‌هایش قرار است به چه چیزی برسند (کیشومات) یا حتی حرکت‌ها چه اثری بر موقعیتِ مهره‌ها دارند. و البته شطرنج بخشِ کوچکی از جهانِ واقعی است. برای مسائلِ واقع‌بینانه‌تر، به پایگاه‌های دادهٔ استادبزرگانِ بسیار وسیع‌تری نیاز می‌داشتیم، و آن‌ها به‌سادگی وجود ندارند.[1]

جایگزین، «یادگیریِ تقویتی» (Reinforcement Learning) است، که در آن عامل با جهان برهم‌کنش می‌کند و به‌طورِ دوره‌ای «پاداش»هایی (Reward) دریافت می‌کند (یا، در اصطلاحِ روان‌شناسی، تقویت‌هایی) که بازتابِ خوب‌بودنِ عملکردِ اوست. برای نمونه، در شطرنج پاداش برای بردن ۱، برای باختن ۰ و برای مساوی ½ است. مفهومِ پاداش‌ها را قبلاً در فصل ۱۷ برای فرایندهای تصمیمِ مارکوف (MDP) دیده‌ایم. در واقع، هدف در یادگیریِ تقویتی همان است:

بیشینه‌کردنِ مجموعِ موردانتظارِ پاداش‌ها.

یادگیریِ تقویتی با «صرفاً حل‌کردنِ MDP» تفاوت دارد، زیرا MDP به‌عنوانِ مسئله‌ای برای حل به عامل داده نمی‌شود؛ عامل درونِ MDP است. ممکن است مدلِ انتقال یا تابعِ پاداش را نداند، و برای یادگیریِ بیشتر باید کنش کند. تصور کنید بازیِ تازه‌ای می‌کنید که قواعدش را نمی‌دانید؛ پس از حدودِ صد حرکت، داور به شما می‌گوید «باختی». این یادگیریِ

تقویتی در یک جمله است.

از دیدگاهِ ما به‌عنوانِ طراحانِ سامانه‌های هوش مصنوعی، ارائهٔ سیگنالِ پاداش به عامل معمولاً بسیار آسان‌تر از ارائهٔ مثال‌های برچسب‌خوردهٔ نحوهٔ رفتار است. نخست، تابعِ پاداش اغلب (چنان‌که در شطرنج دیدیم) بسیار موجز و ساده برای مشخص‌کردن است: فقط چند خط کد لازم است تا به عاملِ شطرنج گفته شود برده یا باخته، یا به عاملِ مسابقهٔ اتومبیل‌رانی گفته شود برده یا باخته یا تصادف کرده. دوم، لازم نیست کارشناس باشیم، تواناییِ فراهم‌کردنِ کنشِ درست در هر موقعیتی، چنان‌که اگر بخواهیم یادگیریِ نظارت‌شده به‌کار ببریم چنین می‌شد. معلوم می‌شود، با این حال، کمی تخصص می‌تواند در یادگیریِ تقویتی راهِ زیادی برود.

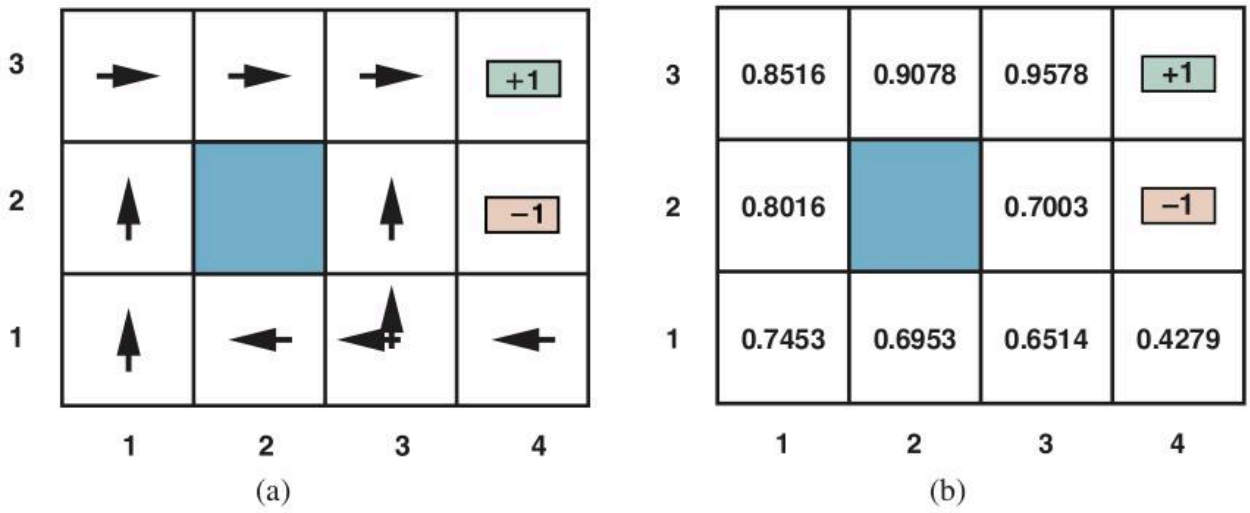
دو مثالِ پاراگرافِ پیشین—پاداش‌های برد/باخت برای شطرنج و مسابقه—چیزی‌اند که «پاداشِ پراکنده» (Sparse Reward) می‌نامیم، زیرا در اکثریتِ قریبِ به‌اتفاقِ حالت‌ها به عامل هیچ سیگنالِ پاداشِ اطلاعاتی‌ای داده نمی‌شود. در بازی‌هایی مانندِ تنیس و کریکت، می‌توانیم به‌آسانی پاداش‌های اضافی برای هر امتیازِ برده‌شده یا هر رانِ زده‌شده فراهم کنیم. در مسابقهٔ اتومبیل‌رانی، می‌توانستیم عامل را برای پیشرفت در مسیر در جهتِ درست پاداش دهیم. هنگامِ یادگیریِ چهار دست و پا رفتن، هر حرکتِ رو به جلو دستاوردی است. این پاداش‌های میانی یادگیری را بسیار آسان‌تر می‌کنند. تا زمانی که بتوانیم سیگنالِ پاداشِ درست را به عامل بدهیم، یادگیریِ تقویتی راهِ بسیار کلی برای ساختنِ سامانه‌های هوش مصنوعی فراهم می‌کند. این به‌ویژه برای محیط‌های شبیه‌سازی‌شده صادق است، که در آن‌ها فرصتِ کسبِ تجربه کم نیست. افزودنِ یادگیری عمیق به‌عنوانِ ابزاری در سامانه‌های RL نیز کاربردهای تازه‌ای را ممکن کرده، از جملهِ یادگیریِ بازیِ بازی‌های ویدئوییِ آتاری از ورودیِ بصریِ خام (منیه و همکاران، 2013)، کنترلِ ربات‌ها (لوین و همکاران، 2016)، و بازیِ پوکر (براون و سندهلم، 2017).

به‌معنای واقعیِ کلمه صدها الگوریتمِ یادگیریِ تقویتیِ متفاوت ابداع شده، و بسیاری از آن‌ها می‌توانند به‌عنوانِ ابزار طیفِ وسیعی از روش‌های یادگیریِ فصل‌های ۱۹ تا ۲۱ را به‌کار برند.

در این فصل، ایده‌های بنیادین را پوشش می‌دهیم و از خلالِ چند مثال حسی از تنوعِ رویکردها می‌دهیم. رویکردها را به‌صورتِ زیر دسته‌بندی می‌کنیم:

- **یادگیریِ تقویتیِ مبتنی بر مدل** (Model-Based Reinforcement Learning): در این رویکردها عامل از مدلِ انتقالِ محیط برای کمک به تفسیرِ سیگنال‌های پاداش و تصمیم‌گیری دربارهٔ نحوهٔ کنش استفاده می‌کند. مدل ممکن است در آغاز ناشناخته باشد، که در آن صورت عامل مدل را از مشاهدهٔ اثرهای کنش‌هایش یاد می‌گیرد، یا ممکن است از قبل شناخته‌شده باشد—برای نمونه، برنامهٔ شطرنج ممکن است قواعد شطرنج را بداند حتی اگر نداند چگونه حرکت‌های خوب انتخاب کند. در محیط‌های با مشاهدهٔ جزئی، مدلِ انتقال برای تخمینِ حالت نیز مفید است (فصل ۱۴ را ببینید). سامانه‌های یادگیریِ تقویتیِ مبتنی بر مدل اغلب «تابعِ مطلوبیت» $U(s)$ را یاد می‌گیرند، که (چنان‌که در فصل ۱۷) بر حسبِ مجموعِ پاداش‌ها از حالتِ s به بعد تعریف می‌شود.[۲]
- **یادگیریِ تقویتیِ بدونِ مدل** (Model-Free Reinforcement Learning): در این رویکردها عامل نه مدلِ انتقالِ محیط را می‌داند نه آن را یاد می‌گیرد. در عوض، بازنماییِ مستقیم‌تری از نحوهٔ رفتار یاد می‌گیرد. این به یکی از دو صورت است:

- **یادگیریِ کنش-مطلوبیت** (Action-Utility Learning): توابعِ کنش-مطلوبیت را در فصل ۱۷ معرفی کردیم. رایج‌ترین شکلِ یادگیریِ کنش-مطلوبیت «کیو-یادگیری» (Q-Learning) است، که در آن عامل «تابعِ Q» یا تابعِ کیفیت، Q(s,a)، را یاد می‌گیرد، که مجموعِ پاداش‌ها از حالتِ s به بعد را نشان می‌دهد اگر کنشِ a گرفته شود. با توجه به تابعِ Q، عامل می‌تواند در s با یافتنِ کنشی که بیشترین مقدارِ Q را دارد انتخاب کند چه کند.
- **جستجوی سیاست** (Policy Search): عامل سیاستِ π(s) را یاد می‌گیرد که مستقیماً از حالت‌ها به کنش‌ها نگاشت می‌کند. در اصطلاحِ فصل ۲، این عاملِ بازتابی است.



شکل ۲۲.۱ — (الف) سیاست‌های بهینه برای محیطِ تصادفی با R(s,a,s') = -0.04 برای انتقال‌های بینِ حالت‌های غیرپایانی. دو سیاست هست، زیرا در حالتِ (3,1) هم «چپ» و هم «بالا» بهینه‌اند. این را قبلاً در شکل ۱۷.۲ دیده بودیم. (ب) مطلوبیت‌های حالت‌ها در جهانِ ۴×۳، با توجه به سیاستِ π. مقادیرِ نمایش‌داده‌شده در شکل، ردیف‌به‌ردیف از بالا: 0.8516، 0.9078، 0.9578؛ سپس 0.8016 و 0.7003 (سلولِ میانیِ این ردیف، یعنی (2,2)، مسدود است)؛ و سپس 0.7453، 0.6953، 0.6514 و 0.4279.

در بخش ۲۲.۲ با یادگیریِ تقویتیِ منفعل شروع می‌کنیم، که در آن سیاستِ عامل ثابت است و تکلیف، یادگیریِ مطلوبیتِ حالت‌ها (یا جفت‌های حالت-کنش) است؛ این همچنین می‌تواند شاملِ یادگیریِ مدلی از محیط باشد. (فهمِ فرایندهای تصمیمِ مارکوف، چنان‌که در فصل ۱۷ توصیف شد، برای این بخش ضروری است.) بخش ۲۲.۳ یادگیریِ تقویتیِ فعال را پوشش می‌دهد، که در آن عامل باید همچنین بفهمد چه کند. مسئلهٔ اصلی «کاوش» است: عامل باید تا حدِ ممکن از محیطِ خود تجربه کند تا یاد بگیرد چگونه در آن رفتار کند. بخش ۲۲.۴ بحث می‌کند عامل چگونه می‌تواند از یادگیریِ استقرایی (از جملهِ روش‌های یادگیری عمیق) برای یادگیریِ بسیار سریع‌تر از تجربه‌هایش استفاده کند. همچنین رویکردهای دیگری را بحث می‌کنیم که می‌توانند به مقیاس‌پذیرکردنِ RL برای حلِ مسائلِ واقعی کمک کنند، از جملهِ فراهم‌کردنِ شبه‌پاداش‌های میانی برای راهنماییِ یادگیرنده و سازمان‌دهیِ رفتار به سلسله‌مراتبی از کنش‌ها. بخش ۲۲.۵ روش‌های جستجوی سیاست را پوشش می‌دهد. در بخش ۲۲.۶، یادگیریِ کارآموزی را کاوش می‌کنیم: آموزشِ عاملِ یادگیرنده با استفاده از نمایش‌ها به‌جای سیگنال‌های پاداش. سرانجام، بخش ۲۲.۷ کاربردهای یادگیریِ تقویتی را گزارش می‌دهد.

۲۲.۲ یادگیریِ تقویتیِ منفعل

با حالتِ سادهٔ محیطِ کاملاً مشاهده‌پذیر با تعدادِ کمی کنش و حالت شروع می‌کنیم، که در آن عامل از قبل سیاستِ ثابتِ $\pi(s)$ دارد که کنش‌هایش را تعیین می‌کند.

عامل می‌کوشد تابعِ مطلوبیتِ $U^{\pi}(s)$—مجموعِ موردانتظارِ کلِ پاداشِ تخفیف‌خورده اگر سیاستِ π از حالتِ s آغاز شود—را یاد بگیرد. این را «عاملِ یادگیریِ منفعل» (Passive Learning Agent) می‌نامیم.

تکلیفِ یادگیریِ منفعل شبیهِ تکلیفِ ارزیابیِ سیاست است، بخشی از الگوریتمِ تکرارِ سیاستِ توصیف‌شده در بخش ۱۷.۲.۲. تفاوت این است که عاملِ یادگیریِ منفعل مدلِ انتقالِ $P(s'|s,a)$ را نمی‌داند، که احتمالِ رسیدن به حالتِ s' از حالتِ s پس از انجامِ کنشِ a را مشخص می‌کند؛ و تابعِ پاداشِ $R(s,a,s')$ را نیز نمی‌داند، که پاداشِ هر انتقال را مشخص می‌کند.

به‌عنوانِ مثال از جهانِ ۴×۳ معرفی‌شده در فصل ۱۷ استفاده می‌کنیم. شکل ۲۲.۱ سیاست‌های بهینه را برای آن جهان و مطلوبیت‌های متناظر را نشان می‌دهد. عامل با استفاده از سیاستِ π مجموعه‌ای از «کوشش»ها (Trial) را در محیط اجرا می‌کند. در هر کوشش، عامل از حالتِ (1,1) شروع می‌کند و دنباله‌ای از انتقالِ حالت را تجربه می‌کند تا به یکی از حالت‌های پایانی، (4,2) یا (4,3)، برسد. ادراک‌هایش هم حالتِ کنونی و هم پاداشِ دریافتی برای انتقالی که تازه برای رسیدن به آن حالت رخ داده فراهم می‌کنند. کوشش‌های معمولی ممکن است چنین باشند:

کوشش ۱: (1,1) -0.04→ بالا (1,2) -0.04→ بالا (1,3) -0.04→ راست (1,2) -0.04→ راست (2,3) -0.04→ بالا (1,3) -0.04→ راست (3,3) +1→ راست (4,3)

کوشش ۲: (1,1) -0.04→ بالا (1,2) -0.04→ بالا (1,3) -0.04→ راست (2,3) -0.04→ راست (3,3) -0.04→ بالا (3,2) -0.04→ بالا (3,3) +1→ راست (4,3)

کوشش ۳: (1,1) -0.04→ بالا (1,2) -0.04→ بالا (1,3) -0.04→ راست (2,3) -0.04→ راست (3,3) -0.04→ راست (3,2) -1→ بالا (4,2)

توجه کنید هر انتقال هم با کنشِ انجام‌شده و هم با پاداشِ دریافتی در حالتِ بعدی حاشیه‌نویسی شده. هدف استفاده از اطلاعاتِ مربوط به پاداش‌ها برای یادگیریِ مطلوبیتِ موردانتظارِ $U^{\pi}(s)$ وابسته به هر حالتِ غیرپایانیِ s است. مطلوبیت به‌عنوانِ مجموعِ موردانتظارِ پاداش‌های (تخفیف‌خورده) ای که اگر سیاستِ π دنبال شود به‌دست می‌آید تعریف می‌شود. چنان‌که در معادلهٔ (17.2) در صفحهٔ ۵۶۷، می‌نویسیم:

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1})\right] \qquad (22.1)$$

که در آن $R(S_t, \pi(S_t), S_{t+1})$ پاداشِ دریافتی است وقتی کنشِ $\pi(S_t)$ در حالتِ S_t گرفته می‌شود و به حالتِ S_{t+1} می‌رسد. توجه کنید S_t متغیرِ تصادفی است که حالتی را نشان می‌دهد در زمانِ t هنگامِ اجرایِ سیاستِ π، با شروع از حالتِ $S_0 = s$، به آن می‌رسیم. عاملِ تخفیفِ γ را در همهٔ معادله‌هایمان می‌گنجانیم، اما برای جهانِ ۴×۳ مقدارِ $\gamma = 1$ می‌گذاریم، که یعنی بدونِ تخفیف.

۲۲.۲.۱ برآوردِ مستقیمِ مطلوبیت

ایدهٔ «برآوردِ مستقیمِ مطلوبیت» (Direct Utility Estimation) این است که مطلوبیتِ یک حالت به‌عنوانِ مجموعِ موردانتظارِ کلِ پاداش از آن حالت به بعد (که «پاداشِ تا-پایان» (Reward-to-Go) نامیده می‌شود) تعریف می‌شود، و هر کوشش نمونه‌ای از این کمیت را برای هر حالتِ بازدیدشده فراهم می‌کند. برای نمونه، نخستینِ سه کوششِ نشان‌داده‌شده در بالا مجموعِ پاداشِ نمونهٔ 0.76 را برای حالتِ (1,1)، دو نمونهٔ 0.80 و 0.88 را برای (1,2)، دو نمونهٔ 0.84 و 0.92 را برای (1,3) و مانندِ آن فراهم می‌کند. پس، در پایانِ هر دنباله، الگوریتم پاداشِ تا-پایانِ مشاهده‌شده را برای هر حالت محاسبه و برآوردِ مطلوبیتِ آن حالت را متناسباً به‌روزرسانی می‌کند، فقط با نگه‌داشتنِ میانگینِ متحرکی برای هر حالت در جدولی. در حدِ بی‌نهایت کوشش، میانگینِ نمونه‌ای به امیدِ ریاضیِ واقعیِ معادلهٔ (22.1) همگرا می‌شود.

این بدان‌معناست که یادگیریِ تقویتی را به مسئلهٔ یادگیریِ نظارت‌شدهٔ استانداردی تقلیل داده‌ایم که در آن هر مثال جفتِ (حالت، پاداشِ تا-پایان) است. الگوریتم‌های قدرتمندِ بسیاری برای یادگیریِ نظارت‌شده داریم، پس این رویکرد امیدوارکننده به نظر می‌رسد، اما قیدِ مهمی را نادیده می‌گیرد: مطلوبیتِ یک حالت توسط پاداش و مطلوبیتِ موردانتظارِ حالت‌های جانشین تعیین می‌شود. به‌طورِ مشخص‌تر، مقادیرِ مطلوبیت از «معادله‌های بلمن» برای سیاستِ ثابت پیروی می‌کنند (معادلهٔ (17.14) را نیز ببینید):

$$U_i(s) = \sum_{s'} P(s'|s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')] \qquad (22.2)$$

با نادیده‌گرفتنِ پیوندهای بینِ حالت‌ها، برآوردِ مستقیمِ مطلوبیت فرصت‌های یادگیری را از دست می‌دهد. برای نمونه، دومینِ سه کوششِ داده‌شده در بالا به حالتِ (3,2) می‌رسد، که قبلاً بازدید نشده. انتقالِ بعدی به (3,3) می‌رسد، که از کوششِ نخست شناخته‌شده است مطلوبیتِ بالایی دارد. معادلهٔ بلمن بلافاصله پیشنهاد می‌کند (3,2) نیز به احتمالِ زیاد مطلوبیتِ بالایی دارد، زیرا به (3,3) می‌انجامد، اما برآوردِ مستقیمِ مطلوبیت تا پایانِ کوشش هیچ‌چیز یاد نمی‌گیرد. به‌طورِ کلی‌تر، می‌توانیم برآوردِ مستقیمِ مطلوبیت را به‌عنوانِ جستجوی U در فضایِ فرضیه‌ای ببینیم که بسیار بزرگ‌تر از آن چیزی است که باید باشد، از آن نظر که توابعِ بسیاری را شامل می‌شود که معادله‌های بلمن را نقض می‌کنند. به همین دلیل، الگوریتم اغلب بسیار کند همگرا می‌شود.

۲۲.۲.۲ برنامه‌نویسیِ پویایِ تطبیقی

«عاملِ برنامه‌نویسیِ پویایِ تطبیقی» (Adaptive Dynamic Programming یا ADP) از قیدهای موجود در میانِ مطلوبیت‌های حالت‌ها بهره می‌برد، با یادگیریِ مدلِ انتقالی که آن‌ها را به هم پیوند می‌دهد و حلِ فرایندِ تصمیمِ مارکوفِ متناظر با برنامه‌نویسیِ پویا. برای عاملِ یادگیریِ منفعل، این یعنی جای‌گذاریِ مدلِ انتقالِ آموخته‌شدهٔ $P(s'|s, \pi(s))$ و پاداش‌های مشاهده‌شدهٔ $R(s, \pi(s), s')$ در معادلهٔ (22.2) برای محاسبهٔ مطلوبیت‌های حالت‌ها. چنان‌که در بحثِ تکرارِ سیاست در فصل ۱۷ اشاره کردیم، وقتی سیاستِ π ثابت است این معادله‌های بلمن خطی‌اند، پس می‌توان آن‌ها را با هر بستهٔ جبرِ خطی حل کرد.

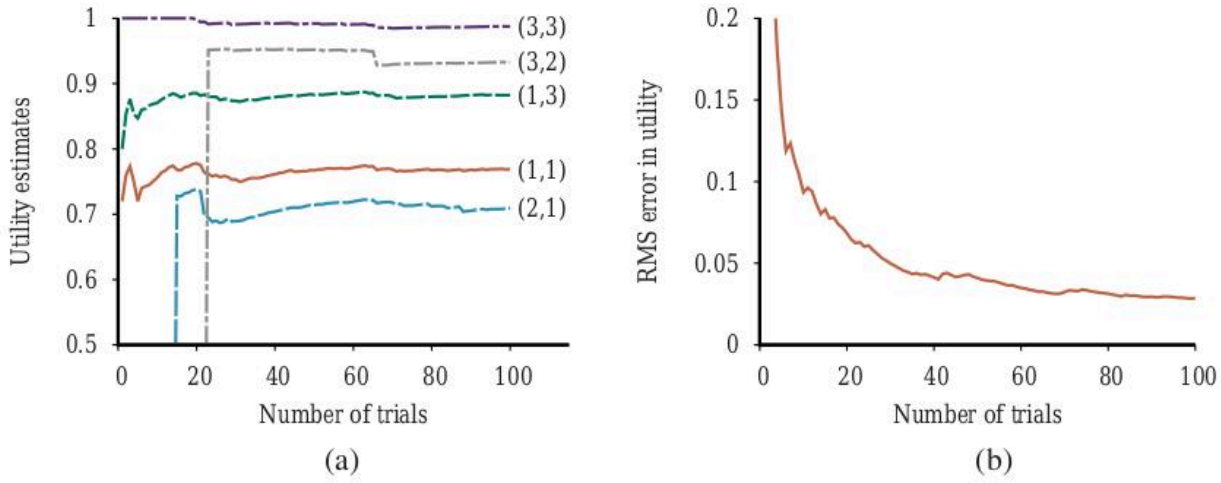
جایگزین، می‌توانیم رویکردِ تکرارِ سیاستِ اصلاح‌شده را برگزینیم (صفحهٔ ۵۷۸ را ببینید)، با استفاده از فرایندِ ساده‌شدهٔ تکرارِ ارزش برای به‌روزرسانیِ برآوردهای مطلوبیت پس از هر تغییرِ مدلِ آموخته‌شده. چون مدل معمولاً با هر مشاهده فقط کمی تغییر می‌کند، فرایندِ تکرارِ ارزش می‌تواند از برآوردهای مطلوبیتِ قبلی به‌عنوانِ مقادیرِ اولیه استفاده کند و معمولاً خیلی سریع همگرا می‌شود. یادگیریِ مدلِ انتقال آسان است، زیرا محیط کاملاً مشاهده‌پذیر است. این بدان‌معناست که تکلیفِ یادگیریِ نظارت‌شده‌ای داریم که ورودیِ هر مثالِ آموزشی جفتِ حالت-کنشِ (s,a) و خروجی حالتِ حاصل، s'، است. مدلِ انتقالِ P(s'|s,a) به‌صورتِ جدولی بازنمایی و مستقیماً از شمارش‌های انباشته‌شده در N_{s'|sa} برآورد می‌شود. شمارش‌ها ثبت می‌کنند حالتِ s' چند بار هنگامِ اجرایِ a در s به‌دست می‌آید. برای نمونه، در سه کوششِ داده‌شده در صفحهٔ ۷۹۲، «راست» چهار بار در (3,3) اجرا می‌شود و حالتِ حاصل دو بار (3,2) و دو بار (4,3) است، پس P((3,2)|(3,3), راست) و P((4,3)|(3,3), راست) هر دو ½ برآورد می‌شوند.

برنامهٔ کاملِ عاملِ ADP منفعل در شکل ۲۲.۲ نشان داده شده. عملکردش رویِ جهانِ ۴×۳ در شکل ۲۲.۳ نشان داده شده. از نظرِ سرعتی که برآوردهای ارزش‌اش بهبود می‌یابند، عاملِ ADP فقط با توانایی‌اش در یادگیریِ مدلِ انتقال محدود می‌شود. از این نظر، معیاری فراهم می‌کند که هر الگوریتمِ یادگیریِ تقویتیِ دیگری را می‌توان با آن سنجید. اما برای فضاهای حالتِ بزرگ محاسبه‌ناپذیر است. در بازیِ تخته‌نرد، برای نمونه، حلِ تقریباً 10^{20} معادله در 10^{20} مجهول را دربرمی‌گرفت.

function PASSIVE-ADP-LEARNER(*percept*) **returns** an action
  inputs: *percept*, a percept indicating the current state s' and reward signal r
  persistent: π, a fixed policy
    mdp, an MDP with model P, rewards R, actions A, discount γ
    U, a table of utilities for states, initially empty
    $N_{s'|s,a}$, a table of outcome count vectors indexed by state and action, initially zero
    s, a, the previous state and action, initially null

  if s' is new **then** $U[s'] \leftarrow 0$
  if s is not null **then**
    increment $N_{s'|s,a}[s,a][s']$
    $R[s, a, s'] \leftarrow r$
    add a to $A[s]$
    $\mathbf{P}(\cdot \mid s,a) \leftarrow$ NORMALIZE($N_{s'|s,a}[s, a]$)
    $U \leftarrow$ POLICYEVALUATION(π, U, mdp)
    $s, a \leftarrow s', \pi[s']$
    return a

شکل ۲۲.۲ — عاملِ یادگیریِ تقویتیِ منفعلِ مبتنی بر برنامه‌نویسیِ پویایِ تطبیقی. عامل برای ۷ مقداری انتخاب می‌کند و سپس مقادیرِ P و R MDP را به‌صورتِ افزایشی محاسبه می‌کند. تابعِ POLICY-EVALUATION معادله‌های بلمنِ سیاست-ثابت را حل می‌کند، چنان‌که در صفحهٔ ۵۷۷ توصیف شده.



شکل ۲۲.۳ — منحنی‌های یادگیریِ ADP منفعل برای جهانِ ۴×۳، با توجه به سیاستِ بهینهٔ نشان‌داده‌شده در شکل ۲۲.۱. (الف) برآوردهای مطلوبیت برای زیرمجموعهٔ انتخابی از حالت‌ها، (1,1)، (1,3)، (2,1)، (3,2) و (3,3)، به‌عنوانِ تابعی از تعدادِ کوشش‌ها (محورِ عمودی بازهٔ 0.5 تا 0.9 را می‌پوشاند). توجه کنید به‌ترتیب ۱۴ و ۲۳ کوشش لازم است تا حالت‌های به‌ندرت بازدیدشدهٔ (2,1) و (3,2) «کشف» کنند که به حالتِ خروجیِ +1 در (4,3) متصل‌اند. (ب) ریشهٔ میانگینِ مربعاتِ خطا (پیوستِ الف را ببینید) در برآوردِ U(1,1)، میانگین‌شده رویِ ۵۰ اجرایِ ۱۰۰ کوششی (محورِ عمودی بازهٔ 0.05 تا 0.2 را می‌پوشاند).

۲۲.۲.۳ یادگیریِ تفاضلِ زمانی

حلِ MDP زیرین مانندِ بخشِ پیشین تنها راهِ به‌کاربردنِ معادله‌های بلمن بر مسئلهٔ یادگیری نیست. راهِ دیگر استفاده از انتقال‌های مشاهده‌شده برای تنظیمِ مطلوبیت‌های حالت‌های مشاهده‌شده است تا با معادله‌های قید موافقت کنند. برای نمونه، انتقال از (1,3) به (2,3) در دومین کوششِ صفحهٔ ۷۹۲ را در نظر بگیرید. فرض کنید در نتیجهٔ کوششِ نخست، برآوردهای مطلوبیت $U^\pi(1,3) = 0.88$ و $U^\pi(2,3) = 0.96$ باشند. حالا، اگر این انتقال از (1,3) به (2,3) همیشه رخ می‌داد، انتظار داشتیم مطلوبیت‌ها از معادلهٔ $U^\pi(1,3) = -0.04 + U^\pi(2,3)$ پیروی کنند، پس $U^\pi(1,3)$ برابرِ 0.92 می‌شد. پس، برآوردِ کنونیِ 0.84 ممکن است کمی پایین باشد و باید افزایش یابد.

به‌طورِ کلی‌تر، وقتی انتقالی از حالتِ s به حالتِ s' از طریقِ کنشِ $\pi(s)$ رخ می‌دهد، به‌روزرسانیِ زیر را بر $U^\pi(s)$ اعمال می‌کنیم:

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha[R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)] \qquad (22.3)$$

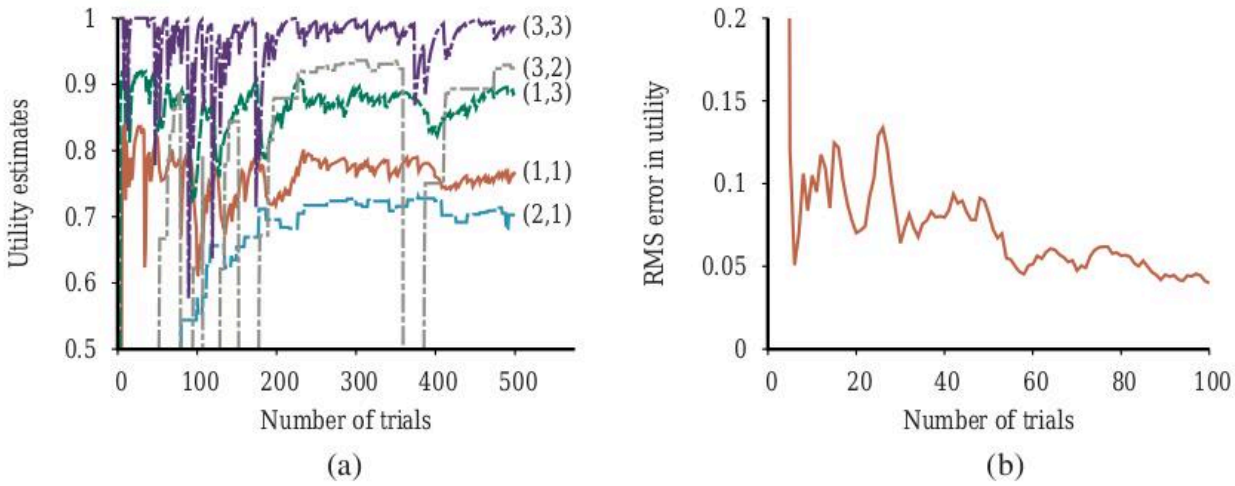
در این‌جا، α پارامترِ نرخِ یادگیری است. چون این قاعدهٔ به‌روزرسانی از تفاوتِ مطلوبیت‌های بینِ حالت‌های متوالی (و در نتیجهِ زمان‌های متوالی) استفاده می‌کند، اغلب «معادلهٔ تفاضلِ زمانی» (Temporal-Difference یا TD) نامیده می‌شود. درست مانندِ قاعده‌های به‌روزرسانیِ وزنِ فصل ۱۹ (مثلاً معادلهٔ (19.6) در صفحهٔ ۶۸۰)، عبارتِ TD یعنی $R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s)$، به‌طورِ مؤثر سیگنالِ خطاست، و به‌روزرسانی برای کاهشِ خطا در نظر گرفته شده.

همهٔ روش‌های تفاضلِ زمانی با تنظیمِ برآوردهای مطلوبیت به سمتِ تعادلِ ایده‌آلی کار می‌کنند که به‌صورتِ محلی وقتی برآوردهای مطلوبیت درست‌اند برقرار است. در موردِ یادگیریِ منفعل، تعادل توسط معادلهٔ (22.2) داده می‌شود. حالا معادلهٔ (22.3) در واقع باعث می‌شود عامل به تعادلِ داده‌شده توسط معادلهٔ (22.2) برسد، اما ظرافتی در کار است. نخست، توجه کنید به‌روزرسانی فقط جانشینِ مشاهده‌شدهٔ s' را دربرمی‌گیرد، در حالی که شرایطِ تعادلِ واقعی همهٔ حالت‌های بعدیِ ممکن را دربرمی‌گیرد. ممکن است کسی فکر کند این وقتی انتقالِ بسیار نادری رخ می‌دهد باعث تغییرِ نابه‌جایِ بزرگی در U^π(s) می‌شود؛ اما، در واقع، چون انتقال‌های نادر به‌ندرت رخ می‌دهند، مقدارِ میانگینِ U^π(s) در حد به کمیتِ درست همگرا خواهد شد، حتی اگر خودِ مقدار به نوسان ادامه دهد.

افزون بر این، اگر پارامترِ α را به تابعی تبدیل کنیم که با افزایشِ تعدادِ دفعاتِ بازدیدِ یک حالت کاهش می‌یابد، چنان‌که در شکل ۲۲.۴ نشان داده شده، آنگاه خودِ U^π(s) به مقدارِ درست همگرا خواهد شد.[3]

```
function PASSIVE-TD-LEARNER(percept) returns an action
  inputs: percept, a percept indicating the current state s' and reward signal r
  persistent: π, a fixed policy
              s, the previous state, initially null
              U, a table of utilities for states, initially empty
              N_s, a table of frequencies for states, initially zero

  if s' is new then U[s'] ← 0
  if s is not null then
    increment N_s[s]
    U[s] ← U[s] + α(N_s[s]) × (r + γ U[s'] - U[s])
  s ← s'
  return π[s']
```

شکل ۲۲.۴ – عاملِ یادگیریِ تقویتیِ منفعل که برآوردهای مطلوبیت را با استفاده از تفاضل‌های زمانی یاد می‌گیرد. تابعِ اندازه‌گامِ α(n) چنان انتخاب می‌شود که همگرایی تضمین شود.



شکل ۲۲.۵ – منحنی‌های یادگیریِ TD برای جهانِ ۴×۳. (الف) برآوردهای مطلوبیت برای زیرمجموعهٔ انتخابی از حالت‌ها، (1,1)، (1,3)، (2,1)، (3,3) و (3,2)، به‌عنوانِ تابعی از تعدادِ کوشش‌ها (محورِ عمودی بازهٔ 0.5 تا 0.9)، برای اجرایِ تکیِ ۵۰۰ کوششی. با اجرایِ ۱۰۰ کوششیِ شکل ۲۲.۳ (الف) مقایسه کنید. (ب) ریشهٔ میانگینِ مربعاتِ خطا در برآوردِ U(1,1)، میانگین‌شده رویِ ۵۰ اجرایِ ۱۰۰ کوششی (محورِ عمودی بازهٔ 0.05 تا 0.2).

به همان سرعتی که عاملِ ADP یاد نمی‌گیرد و تغییرپذیریِ بسیار بالاتری نشان می‌دهد، اما بسیار ساده‌تر است و به محاسبهٔ بسیار کمتری در هر مشاهده نیاز دارد.

توجه کنید TD برای انجامِ به‌روزرسانی‌هایش به مدلِ انتقال نیاز ندارد. خودِ محیط پیوندِ بینِ حالت‌های مجاور را در قالبِ انتقال‌های مشاهده‌شده فراهم می‌کند.

رویکردهای ADP و TD به‌هم‌پیوسته‌اند. هر دو می‌کوشند تنظیماتِ محلیِ برآوردهای مطلوبیت را انجام دهند تا هر حالت با جانشین‌هایش «موافقت کند». یک تفاوت این است که TD حالتی را تنظیم می‌کند تا با جانشینِ مشاهده‌شده‌اش موافقت کند (معادلهٔ (22.3))، در حالی که ADP حالت را تنظیم می‌کند تا با همهٔ جانشین‌هایی که ممکن است رخ دهند، وزن‌خورده با احتمال‌هایشان، موافقت کند (معادلهٔ (22.2)). این تفاوت وقتی اثرهای تنظیماتِ TD رویِ تعدادِ زیادی انتقال میانگین گرفته شود ناپدید می‌شود، زیرا بسامدِ هر جانشین در مجموعهٔ انتقال‌ها تقریباً متناسب با احتمالِ آن است. تفاوتِ مهم‌تر این است که TD به‌ازایِ هر انتقالِ مشاهده‌شده یک تنظیمِ واحد انجام می‌دهد، در حالی که ADP هرچند تنظیمی که لازم باشد انجام می‌دهد تا سازگاریِ بینِ برآوردهای مطلوبیتِ U و مدلِ انتقالِ P بازگردانده شود. هرچند انتقالِ مشاهده‌شده فقط تغییرِ محلیِ P ایجاد می‌کند، اثرهایش ممکن است لازم باشد در سراسرِ U منتشر شوند. پس، TD را می‌توان تقریبِ درشت اما کارآمدِ نخستِ ADP دید. هر تنظیمِ انجام‌شده توسط ADP را می‌توان، از دیدگاهِ TD، نتیجهٔ «شبه‌تجربه»ای (Pseudoexperience) دید که با شبیه‌سازیِ مدلِ انتقالِ کنونی تولید شده. امکانِ بسطِ رویکردِ TD برای استفاده از مدلِ انتقال برای تولیدِ چند شبه‌تجربه—انتقال‌هایی که عاملِ TD می‌تواند تصور کند ممکن است رخ دهند، با توجه به مدلِ کنونی‌اش—هست. برای هر انتقالِ مشاهده‌شده، عاملِ TD می‌تواند تعدادِ زیادی انتقالِ خیالی تولید کند. به این‌شیوه، برآوردهای مطلوبیتِ حاصل بیشتر و بیشتر به برآوردهای ADP نزدیک می‌شوند—البته، به بهایِ افزایشِ زمانِ محاسبه.

در همین راستا، می‌توانیم نسخه‌های کارآمدتری از ADP با تقریبِ مستقیمِ الگوریتم‌های تکرارِ ارزش یا تکرارِ سیاست تولید کنیم. حتی اگر الگوریتمِ تکرارِ ارزش کارآمد باشد، اگر مثلاً 10^{100} حالت داشته باشیم محاسبه‌ناپذیر است. با این حال، بسیاری از تنظیماتِ لازمِ مقادیرِ حالت در هر تکرار بسیار ریز خواهند بود. یک رویکردِ ممکن برای تولیدِ سریعِ پاسخ‌های به‌اندازهٔ کافی خوب، کران‌کردنِ تعدادِ تنظیماتِ انجام‌شده پس از هر انتقالِ مشاهده‌شده است. همچنین می‌توان از اکتشافی برای رتبه‌بندیِ تنظیماتِ ممکن استفاده کرد تا فقط مهم‌ترین‌ها اجرا شوند. اکتشافیِ «جاروبِ اولویت‌دار» (Prioritized Sweeping) ترجیح می‌دهد تنظیماتی را بر حالت‌هایی انجام دهد که جانشین‌های محتملِ آن‌ها تازه در برآوردهای مطلوبیتِ خودشان تنظیمِ بزرگی را تجربه کرده‌اند.

با استفاده از این‌گونه اکتشافی‌ها، الگوریتم‌های تقریبیِ ADP می‌توانند تقریباً به همان سرعتِ ADP کامل، از نظرِ تعدادِ دنباله‌های آموزشی، یاد بگیرند، اما از نظرِ محاسبهٔ کل می‌توانند مرتبه‌های بزرگی کارآمدتر باشند (تمرین PRSW.۲۲ را ببینید). این آن‌ها را قادر می‌کند فضاهای حالتی را مدیریت کنند که برای ADP کامل بسیار بزرگ‌اند. الگوریتم‌های تقریبیِ ADP مزیتِ اضافی‌ای نیز دارند: در مراحلِ اولیهٔ یادگیریِ محیطِ تازه، مدلِ انتقالِ P اغلب از درست‌بودن بسیار دور است، پس فایده‌ای در محاسبهٔ تابعِ مطلوبیتِ دقیق برای مطابقت با آن نیست. الگوریتمِ تقریبی می‌تواند از اندازهٔ حداقلِ تنظیمی استفاده کند که با دقیق‌ترشدنِ مدلِ انتقال کاهش می‌یابد. این اجراهای بسیار طولانیِ تکرارِ ارزش را که ممکن است اوایلِ یادگیری به‌دلیلِ تغییراتِ بزرگِ مدل رخ دهند حذف می‌کند.

۲۲.۳ یادگیریِ تقویتیِ فعال

عاملِ یادگیریِ منفعل سیاستِ ثابتی دارد که رفتارش را تعیین می‌کند. عاملِ یادگیریِ فعال می‌تواند تصمیم بگیرد چه کنش‌هایی انجام دهد. با عاملِ برنامه‌نویسیِ پویایِ تطبیقی (ADP) شروع می‌کنیم و بررسی می‌کنیم چگونه می‌توان آن را برای بهره‌گیری از این آزادیِ تازه اصلاح کرد.

نخست، عامل باید مدلِ انتقالِ کاملی را با احتمال‌های پیامد برای همهٔ کنش‌ها یاد بگیرد، نه فقط مدلِ سیاستِ ثابت. سازوکارِ یادگیریِ استفاده‌شده توسط PASSIVE-ADP-AGENT برای این کار کاملاً کافی خواهد بود. بعد، باید این واقعیت را در نظر بگیریم که عامل انتخابِ کنش‌ها را دارد. مطلوبیت‌هایی که باید یاد بگیرد آن‌هایی‌اند که توسط سیاستِ بهینه تعریف می‌شوند؛ آن‌ها از معادله‌های بلمن پیروی می‌کنند (که در این‌جا تکرار می‌کنیم):

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a) \, [R(s,a,s') + \gamma U(s')] \qquad (22.4)$$

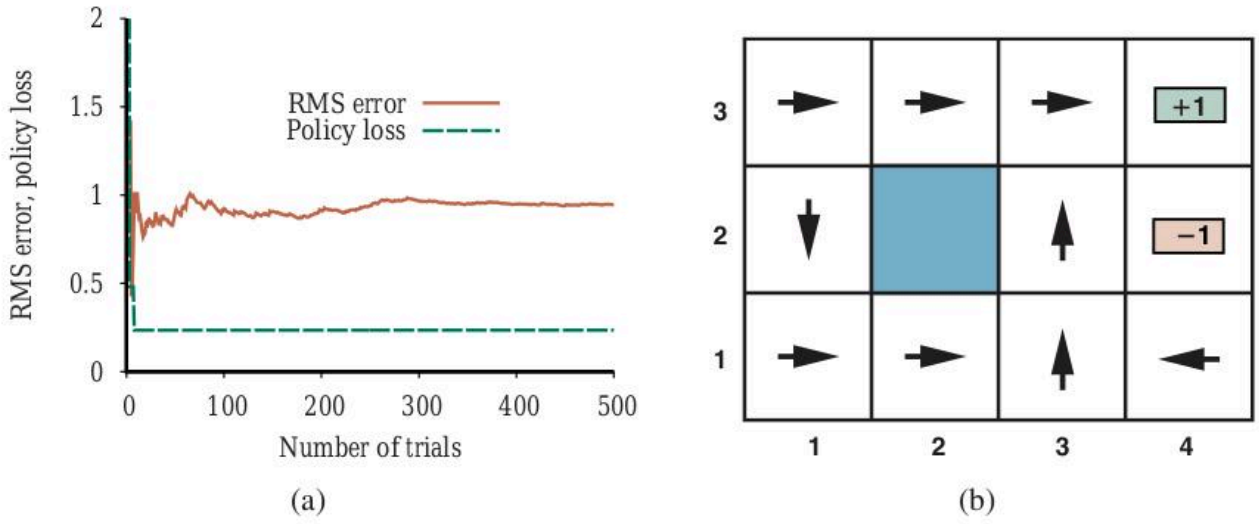
این معادله‌ها را می‌توان با الگوریتم‌های تکرارِ ارزش یا تکرارِ سیاستِ فصل ۱۷ حل کرد تا تابعِ مطلوبیتِ U به‌دست آید.

مسئلهٔ نهایی این است که در هر گام چه کنیم. پس از به‌دست‌آوردنِ تابعِ مطلوبیتِ U که برای مدلِ آموخته‌شده بهینه است، عامل می‌تواند کنشِ بهینه را با نگاه-به-جلوِ تک‌گامی برای بیشینه‌کردنِ مطلوبیتِ موردانتظار استخراج کند؛ یا اگر از تکرارِ سیاست استفاده کند، سیاستِ بهینه از قبل در دسترس است، پس می‌تواند صرفاً کنشی را که سیاستِ بهینه توصیه می‌کند اجرا کند. اما آیا باید؟

۲۲.۳.۱ کاوش

شکل ۲۲.۶ نتایجِ یک دنباله از کوشش‌ها را برای عاملِ ADP نشان می‌دهد که در هر گام توصیهٔ سیاستِ بهینه برای مدلِ آموخته‌شده را دنبال می‌کند. عامل مطلوبیت‌های واقعی یا سیاستِ بهینهٔ واقعی را یاد نمی‌گیرد! آنچه در عوض رخ می‌دهد این است که در کوششِ سوم، سیاستی می‌یابد که از مسیرِ پایینی از طریقِ (2,1)، (3,1)، (3,2) و (3,3) به پاداشِ +1 می‌رسد. (شکل ۲۲.۶ (ب) را ببینید.) پس از آزمایشِ تغییرهای جزئی، از کوششِ هشتم به بعد به همان سیاست می‌چسبد، و هرگز مطلوبیتِ حالت‌های دیگر را یاد نمی‌گیرد و هرگز مسیرِ بهینهٔ از طریقِ (1,2)، (1,3) و (2,3) را نمی‌یابد. این عامل را «عاملِ حریص» (Greedy Agent) می‌نامیم، زیرا در هر گام حریصانه کنشی را انجام می‌دهد که در حالِ حاضر باور دارد بهینه است. گاهی طمع جواب می‌دهد و عامل به سیاستِ بهینه همگرا می‌شود، اما اغلب نمی‌شود.

چگونه ممکن است انتخابِ کنشِ بهینه به نتایجِ زیربهینه بینجامد؟ پاسخ این است که مدلِ آموخته‌شده با محیطِ واقعی یکی نیست؛ آنچه در مدلِ آموخته‌شده بهینه است بنابراین می‌تواند در محیطِ واقعی زیربهینه باشد. متأسفانه، عامل نمی‌داند محیطِ واقعی چیست، پس نمی‌تواند کنشِ بهینه را برای محیطِ واقعی محاسبه کند. پس، چه باید بکند؟



شکل ۲۲.۶ – عملکردِ عاملِ ADP حریصی که کنشِ توصیه‌شده توسط سیاستِ بهینه برای مدلِ آموخته‌شده را اجرا می‌کند. (الف) ریشهٔ میانگینِ مربعات (RMS) خطا، میانگین‌شده رویِ هر نه مربعِ غیرپایانی، و «زیانِ سیاست» در (1,1)، هر دو بر حسبِ تعدادِ کوشش‌ها (محورِ عمودی بازهٔ 0.5 تا 1.5). می‌بینیم سیاست به‌سرعت، فقط پس از هشت کوشش، به سیاستِ زیربهینه‌ای با زیانِ 0.235 همگرا می‌شود. (ب) سیاستِ زیربهینه‌ای که عاملِ حریص در این دنبالهٔ خاص از کوشش‌ها به آن همگرا می‌شود. توجه کنید به کنشِ «پایین» در (1,2).

عاملِ حریص این واقعیت را نادیده گرفته که کنش‌ها بیش از فراهم‌کردنِ پاداش کار می‌کنند؛ آن‌ها همچنین در قالبِ ادراک‌های حالت‌های حاصل، اطلاعات فراهم می‌کنند. چنان‌که با مسئله‌های باندیت در بخش ۱۷.۳ دیدیم، عامل باید مبادله‌ای بینِ «بهره‌برداری» (Exploitation) از بهترین کنشِ کنونی برای بیشینه‌کردنِ پاداشِ کوتاه‌مدتش و «کاوش» (Exploration) حالت‌های قبلاً ناشناخته برای کسبِ اطلاعاتی که می‌تواند به تغییرِ سیاست (و به پاداش‌های بیشتر در آینده) بینجامد انجام دهد. در جهانِ واقعی، انسان دائماً باید بینِ ادامهٔ یک زندگیِ راحت در برابرِ زدن به ناشناخته‌ها به امیدِ زندگیِ بهتر تصمیم بگیرد.

هرچند مسئله‌های باندیت برای به‌دست‌آوردنِ طرحِ کاوشِ بهینه دقیقاً حل‌کردنی دشوارند، با این حال ممکن است طرحی ارائه شود که در نهایت سیاستِ بهینه را کشف کند، حتی اگر انجامش ممکن است بیشتر از حدِ بهینه طول بکشد. هر طرحِ چنینی نباید از نظرِ حرکتِ بی‌واسطهٔ بعدی حریص باشد، بلکه باید آنچه «حریص در حدِ کاوشِ بی‌نهایت» یا GLIE نامیده می‌شود باشد. طرحِ GLIE باید هر کنش را در هر حالت تعدادِ نامحدودی بار امتحان کند تا از احتمالِ متناهیِ ازدست‌رفتنِ کنشِ بهینه اجتناب شود. عاملِ ADP با استفاده از چنین طرحی در نهایت مدلِ انتقالِ واقعی را یاد می‌گیرد و سپس می‌تواند تحتِ بهره‌برداری عمل کند.

چند طرحِ GLIE هست؛ یکی از ساده‌ترین‌ها این است که عامل در گامِ زمانیِ t با احتمالِ 1/t کنشِ تصادفی انتخاب کند و در غیرِ این صورت سیاستِ حریص را دنبال کند. هرچند این در نهایت به سیاستِ بهینه همگرا می‌شود، می‌تواند کند باشد. رویکردِ بهتر به کنش‌هایی که عامل چند بار امتحان نکرده وزنی می‌دهد، در حالی که تمایل به اجتناب از کنش‌هایی دارد که باور می‌شود مطلوبیتِ پایینی دارند (چنان‌که با جستجوی درختِ مونت‌کارلو در بخش ۵.۴ انجام دادیم).

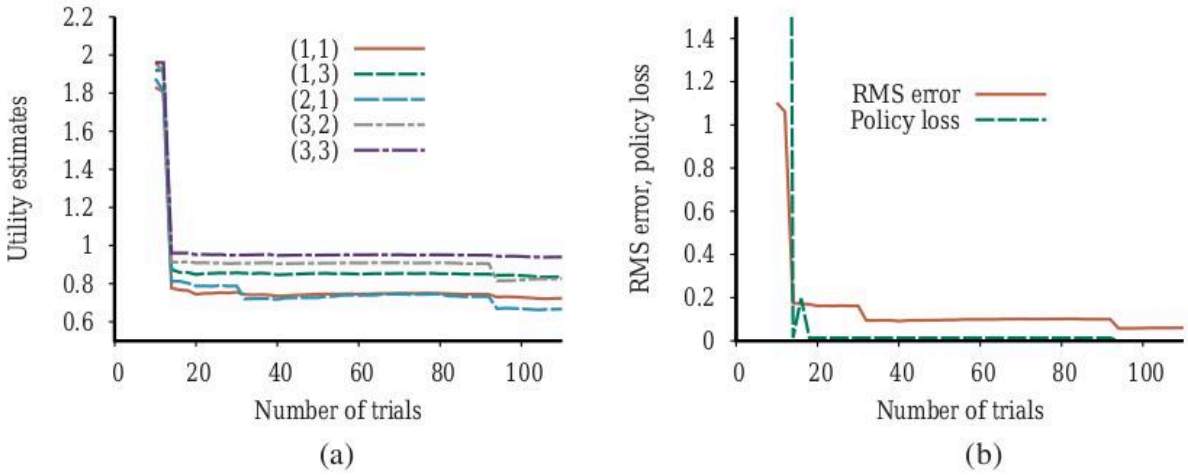
این را می‌توان با تغییرِ معادلهٔ قید (۲۲.۴) پیاده‌سازی کرد تا به جفت‌های حالت-کنشِ نسبتاً کاوش‌نشده برآوردِ مطلوبیتِ بالاتری تخصیص دهد. این به‌معنای پیشینِ خوش‌بینانه‌ای بر محیط‌های ممکن است و باعث می‌شود عامل در آغاز طوری رفتار کند که گویی پاداش‌های شگفت‌انگیزی همه‌جا پراکنده‌اند. با $U^+(s)$ برآوردِ خوش‌بینانهٔ مطلوبیت (یعنی پاداشِ موردانتظارِ تا-پایان) حالتِ s را نشان می‌دهیم، و با $N(s,a)$ تعدادِ دفعاتی را که کنشِ a در حالتِ s امتحان شده. فرض کنید در عاملِ یادگیریِ ADP از تکرارِ ارزش استفاده می‌کنیم؛ پس باید معادلهٔ به‌روزرسانی (معادلهٔ (17.10) در صفحهٔ ۵۷۳) را بازنویسی کنیم تا برآوردِ خوش‌بینانه را لحاظ کند:

$$U^+(s) \leftarrow \max_a f\Big(\sum_{s'} P(s'|s,a)\,[R(s,a,s') + \gamma U^+(s')],\ N(s,a)\Big) \qquad (22.5)$$

در این‌جا، f «تابعِ کاوش» (Exploration Function) است. تابعِ $f(u,n)$ تعیین می‌کند چگونه حرص (ترجیحِ مقادیرِ بالایِ مطلوبیتِ u) با کنجکاوی (ترجیحِ کنش‌هایی که چند بار امتحان نشده‌اند و شمارشِ n پایینی دارند) مبادله می‌شود. تابع باید در u صعودی و در n نزولی باشد. واضح است که توابعِ ممکنِ بسیاری این شرایط را برآورده می‌کنند. یکی از تعریف‌های به‌ویژه ساده این است:

$$f(u,n) = \begin{cases} R^+ & \text{اگر } n < N_e \\ u & \text{در غیر این صورت} \end{cases}$$

که در آن R^+ برآوردِ خوش‌بینانه از بهترین پاداشِ ممکنِ به‌دست‌آوردنی در هر حالت و N_e پارامترِ ثابتی است. این اثر را خواهد داشت که عامل هر جفتِ حالت-کنش را دستِ‌کم N_e بار امتحان کند. این واقعیت که به‌جای U، U^+ در سمتِ راستِ معادلهٔ (22.5) ظاهر می‌شود بسیار مهم است. با پیشرفتِ کاوش، حالت‌ها و کنش‌های نزدیکِ حالتِ شروع ممکن است تعدادِ زیادی بار امتحان شوند. اگر از U، برآوردِ بدبینانه‌ترِ مطلوبیت، استفاده می‌کردیم، عامل به‌زودی از کاوشِ فراترِ مناطقِ دوردست دلسرد می‌شد. استفاده از U^+ به این معناست که مزیت‌های کاوش از لبه‌های مناطقِ کاوش‌نشده به عقب منتشر می‌شوند، تا کنش‌هایی که به سمتِ مناطقِ کاوش‌نشده می‌روند وزن‌دارتر شوند، نه فقط کنش‌هایی که خودشان ناآشنا هستند.



شکل ۲۲.۷ — عملکردِ عاملِ ADP کاوشگر با استفاده از $R^+ = 2$ و $N_e = 5$. (الف) برآوردهای مطلوبیت برای حالت‌های انتخابی، (1,1)، (1,3)، (2,1)، (3,2) و (3,3)، در طولِ زمان (محورِ عمودی بازهٔ 0.6 تا 2.2). (ب) خطایِ RMS در مقادیرِ مطلوبیت و زیانِ سیاستِ وابسته (محورِ عمودی بازهٔ 0.2 تا 1.4).

اثرِ این سیاستِ کاوش را می‌توان به‌وضوح در شکل ۲۲.۷ (ب) دید، که برخلافِ رویکردِ حریص، همگراییِ سریعی به سمتِ زیانِ سیاستِ صفر نشان می‌دهد. سیاستِ تقریباً بهینه فقط پس از ۱۸ کوشش یافت می‌شود. توجه کنید خطایِ RMS در برآوردهای مطلوبیت به همان سرعت همگرا نمی‌شود. این به‌دلیلِ آن است که عامل کاوشِ بخش‌های بی‌پاداشِ فضایِ حالت را به‌زودی متوقف می‌کند و پس از آن فقط «تصادفاً» از آن‌ها بازدید می‌کند. با این حال، کاملاً منطقی است که عامل به مطلوبیتِ دقیقِ حالت‌هایی که می‌داند نامطلوب‌اند و می‌توان از آن‌ها اجتناب کرد اهمیت ندهد. فایدهٔ چندانی در یادگیریِ بهترین ایستگاهِ رادیویی برای گوش‌دادن در حالی که از صخره سقوط می‌کنید نیست.

۲۲.۳.۲ کاوشِ ایمن

تاکنون فرض کرده‌ایم عامل در کاوش هرطور که می‌خواهد آزاد است—که هر پاداشِ منفی فقط برای بهبودِ مدلِ جهانِ او عمل می‌کند. یعنی، اگر بازیِ شطرنجی ببازیم، هیچ آسیبی نمی‌بینیم (به‌جز شاید به غرورمان)، و هرچه یاد گرفته‌ایم در بازیِ بعدی بازیکنِ بهتری‌مان می‌کند. به‌همین‌سان، در محیطِ شبیه‌سازیِ خودرویِ خودران، می‌توانستیم حدهای عملکردِ خودرو را کاوش کنیم، و هر تصادفی اطلاعاتِ بیشتری به ما می‌دهد. اگر خودرو تصادف کند، فقط دکمهٔ بازنشانی را می‌زنیم.

متأسفانه، جهانِ واقعی کمتر گذشت‌کننده است. اگر بچه‌ماهیِ خورشیدماهی باشید، احتمالِ زنده‌ماندنِ شما تا بزرگسالی حدودِ 0.00000001 است. بسیاری از کنش‌ها برگشت‌ناپذیرند، به معنایی که برای عامل‌های جستجویِ برخط در بخش ۴.۵ تعریف شد: هیچ دنبالهٔ بعدی از کنش‌ها نمی‌تواند حالت را به آنچه پیش از برداشتنِ کنشِ برگشت‌ناپذیر بود بازگرداند. در بدترین حالت، عامل وارد «حالتِ جذبی» (Absorbing State) می‌شود که در آن هیچ کنشی اثری ندارد و هیچ پاداشی دریافت نمی‌شود.

در بسیاری از محیط‌های عملی، نمی‌توانیم اجازه دهیم عامل‌هایمان کنش‌های برگشت‌ناپذیر بردارند یا وارد حالت‌های جذبی شوند. برای نمونه، عاملی که در خودروِی واقعی رانندگی یاد می‌گیرد باید از کنش‌هایی اجتناب کند که ممکن است به هر یک از مواردِ زیر بینجامند:

- حالت‌هایی با پاداش‌های منفیِ بزرگ، مانندِ تصادف‌های جدیِ اتومبیل؛
- حالت‌هایی که از آن‌ها گریزی نیست، مانندِ راندنِ خودرو به گودالی عمیق؛
- حالت‌هایی که به‌طورِ دائمی پاداش‌های آینده را محدود می‌کنند، مانندِ آسیب‌زدن به موتورِ خودرو به‌گونه‌ای که حداکثرِ سرعتش کاهش یابد.

می‌توانیم به حالتِ بدی برسیم یا به‌دلیلِ اینکه مدل‌مان ناشناخته است و فعالانه انتخاب می‌کنیم در جهتی کاوش کنیم که معلوم می‌شود بد است، یا به‌دلیلِ اینکه مدل‌مان نادرست است و نمی‌دانیم کنشِ داده‌شده‌ای می‌تواند نتیجهٔ فاجعه‌باری داشته باشد. توجه کنید الگوریتمِ شکل ۲۲.۲ از برآوردِ حداکثرِ درست‌نمایی (فصل ۲۰ را ببینید) برای یادگیریِ مدلِ انتقال استفاده می‌کند؛ افزون بر این، با انتخابِ سیاستی که صرفاً بر مدلِ برآوردشده مبتنی است، طوری رفتار می‌کند که گویی مدل درست است. این لزوماً ایدهٔ خوبی نیست! برای نمونه، عاملِ تاکسی که نمی‌داند چراغ‌های راهنمایی چگونه کار می‌کنند ممکن است یک یا دو بار چراغِ قرمز را بدونِ اثرِ بد نادیده بگیرد و سپس سیاستی صورتبندی کند که از آن به بعد همهٔ چراغ‌های قرمز را نادیده بگیرد.

ایدهٔ بهتر انتخابِ سیاستی است که برای کلِ دامنهٔ مدل‌هایی که شانسِ معقولی برای مدلِ واقعی‌بودن دارند به‌اندازهٔ کافی خوب کار کند، حتی اگر سیاستِ مذکور برای مدلِ حداکثرِ درست‌نمایی اتفاقاً زیربهینه باشد. سه رویکردِ ریاضی هست که این حال وهوا را دارند.

رویکردِ نخست، «یادگیریِ تقویتیِ بیزی» (Bayesian Reinforcement Learning)، احتمالِ پیشینِ P(h) را بر فرضیه‌های h دربارهٔ مدلِ واقعی‌بودنِ چه فرض می‌کند؛ احتمالِ پسینِ P(h|e) به‌شیوهٔ معمول با قاعدهٔ بیز با توجه به مشاهده‌های تاکنون به‌دست می‌آید. سپس، اگر عامل تصمیم گرفته باشد یادگیری را متوقف کند، سیاستِ بهینه آن سیاستی است که بیشترین مطلوبیتِ موردانتظار را می‌دهد. U^π_h مطلوبیتِ موردانتظار، میانگین‌شده رویِ همهٔ حالت‌های شروعِ ممکن، حاصل از اجرایِ سیاستِ π در مدلِ h باشد. آنگاه داریم:

```
π* = argmax_π ∑_h P(h|e) U^π_h
```

در برخی حالت‌های خاص، این سیاست حتی قابلِ محاسبه است! اما اگر عامل در آینده به یادگیری ادامه دهد، یافتنِ سیاستِ بهینه به‌طورِ چشمگیری دشوارتر می‌شود، زیرا عامل باید اثرهای مشاهده‌های آینده را بر باورهایش دربارهٔ مدلِ انتقال در نظر بگیرد. مسئله به «POMDP کاوش» (Exploration POMDP) تبدیل می‌شود که حالت‌های باورِ آن توزیع‌هایی بر مدل‌ها هستند. در اصل، این POMDP کاوش را می‌توان پیش از آنکه عامل هرگز پا در جهان بگذارد صورتبندی و حل کرد. (تمرین EPOM.۲۲ از شما می‌خواهد این کار را برای بازیِ مین‌روب برای یافتنِ بهترین حرکتِ نخست انجام دهید.) نتیجه راهبردِ کاملی است که به عامل می‌گوید با توجه به هر دنبالهٔ ادراکِ ممکن بعد چه کند. حلِ POMDP کاوش معمولاً به‌شدت محاسبه‌ناپذیر است، اما این مفهوم بنیانِ تحلیلی برای فهمِ مسئلهٔ کاوشِ توصیف‌شده در بخش ۲۲.۳ فراهم می‌کند.

شایانِ توجه است که بیزیِ کامل‌بودن عامل را از مرگِ زودهنگام محافظت نمی‌کند. مگر اینکه پیشین نشانه‌ای از ادراک‌هایی بدهد که خطر را پیشنهاد می‌کنند، هیچ‌چیز مانعِ برداشتنِ کنشِ کاوشیِ منجر به حالتِ جذبی توسط عامل نیست. برای نمونه، قبلاً تصور می‌شد نوزادانِ انسان ترسِ فطری از ارتفاع دارند و از صخره چهار دست و پا نمی‌روند، اما معلوم شده این درست نیست (آدولف و همکاران، 2014).

رویکردِ دوم، برگرفته از «نظریهٔ کنترلِ مقاوم» (Robust Control Theory)، مجموعهٔ مدل‌های ممکنِ H را بدونِ تخصیصِ احتمال به آن‌ها مجاز می‌کند، و سیاستِ مقاومِ بهینه را سیاستی تعریف می‌کند که بهترین پیامد را در بدترین حالت رویِ H می‌دهد:

```
π* = argmax_π min_h U^π_h
```

اغلب، مجموعهٔ H مجموعهٔ مدل‌هایی خواهد بود که از آستانهٔ درست‌نماییِ مشخصی رویِ P(h|e) فراتر می‌روند، پس رویکردهای مقاوم و بیزی به هم مربوط‌اند.

رویکردِ کنترلِ مقاوم را می‌توان بازی بینِ عامل و حریفی در نظر گرفت، که حریف برای هر کنش بدترین نتیجهٔ ممکن را انتخاب می‌کند، و سیاستی که به‌دست می‌آوریم راه‌حلِ مینی‌مکس برای بازی است. عاملِ منطقیِ وومپوسِ ما (بخش ۷.۷ را ببینید) از این نظر عاملِ کنترلِ مقاوم است: همهٔ مدل‌هایی را که از نظرِ منطقی ممکن‌اند در نظر می‌گیرد، و هیچ مکانِی را که به‌طورِ بالقوه می‌تواند چاله یا وومپوس داشته باشد کاوش نمی‌کند، پس کنشی را با حداکثرِ مطلوبیت در بدترین حالت رویِ همهٔ فرضیه‌های ممکن می‌یابد.

مشکلِ فرضِ بدترین‌حالت این است که به رفتارِ بیش‌ازحد محافظه‌کارانه می‌انجامد. خودرویِ خودرانی که فرض می‌کند هر رانندهٔ دیگری تلاش خواهد کرد با آن برخورد کند چاره‌ای جز ماندن در گاراژ ندارد. زندگیِ واقعی پر از چنین مبادله‌های ریسک-پاداشی است.

هرچند یکی از دلایلِ پاگذاشتن به یادگیریِ تقویتی گریز از نیاز به معلمِ انسانی (مانندِ یادگیریِ نظارت‌شده) بود، معلوم می‌شود دانشِ انسانی می‌تواند به ایمن‌نگه‌داشتنِ سامانه کمک کند. یک راه ثبتِ دنباله‌ای از کنش‌های معلمِ باتجربه است، تا سامانه از آغاز معقولانه رفتار کند و بتواند از آن نقطه بهبود را یاد بگیرد. راهِ دوم این است که انسان قیدهایی دربارهٔ آنچه سامانه می‌تواند انجام دهد بنویسد، و برنامه‌ای خارج از سامانهٔ یادگیریِ تقویتی آن قیدها را اعمال کند. برای نمونه، هنگامِ آموزشِ هلیکوپترِ خودمختار، می‌توان سیاستِ جزئی‌ای فراهم کرد که وقتی هلیکوپتر وارد حالتی می‌شود که هر کنشِ ناایمنِ بعدی به حالتِ بازیابی‌ناپذیر منجر شود—حالتی که کنترل‌گرِ ایمنی نمی‌تواند تضمین کند از حالتِ جذبی اجتناب خواهد شد—کنترل را به دست می‌گیرد. در همهٔ حالت‌های دیگر، عاملِ یادگیرنده آزاد است هرچه می‌خواهد بکند.

۲۲.۳.۳ کیو-یادگیریِ تفاضلِ زمانی

حالا که عاملِ ADP فعال داریم، بیایید بررسی کنیم چگونه عاملِ یادگیریِ تفاضلِ زمانی (TD) فعال بسازیم. آشکارترین تغییر این است که عامل باید مدلِ انتقالی یاد بگیرد تا بتواند بر اساسِ U(s) از طریقِ نگاه-به-جلوِ تک‌گامی کنش انتخاب کند. مسئلهٔ اکتسابِ مدل برای عاملِ TD همان مسئله برای عاملِ ADP است، و قاعدهٔ به‌روزرسانیِ TD بدونِ تغییر می‌ماند. یک بار دیگر، می‌توان نشان داد الگوریتمِ TD به همان مقادیرِ ADP همگرا می‌شود، وقتی تعدادِ دنباله‌های آموزشی به بی‌نهایت میل کند.

روشِ کیو-یادگیری از نیاز به مدل با یادگیریِ تابعِ کنش-مطلوبیتِ Q(s,a) به‌جای تابعِ مطلوبیتِ U(s) اجتناب می‌کند. Q(s,a) مجموعِ موردانتظارِ کلِ پاداشِ تخفیف‌خورده را نشان می‌دهد اگر عامل در s کنشِ a را بگیرد و پس از آن به‌صورتِ بهینه عمل کند. دانستنِ تابعِ Q عامل را قادر می‌کند به‌صورتِ بهینه عمل کند، صرفاً با انتخابِ argmax_a Q(s,a)، بدونِ نیاز به نگاه-به-جلو بر مبنای مدلِ انتقال. همچنین می‌توانیم به‌روزرسانیِ TD بدونِ مدل را برای مقادیرِ Q اشتقاق دهیم. با معادلهٔ بلمن برای Q(s,a) شروع می‌کنیم، که از معادلهٔ (17.8) در این‌جا تکرار می‌شود:

$$Q(s,a) = \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma \max_{a'} Q(s',a')] \qquad (22.6)$$

از این، می‌توانیم به‌روزرسانیِ TD کیو-یادگیری را بنویسیم، با قیاس با به‌روزرسانیِ TD برای مطلوبیت‌ها در معادلهٔ (22.3):

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)] \qquad (22.7)$$

این به‌روزرسانی هر بار که کنشِ a در حالتِ s اجرا می‌شود و به حالتِ s' می‌انجامد محاسبه می‌شود. مانندِ معادلهٔ (22.3)، عبارتِ R(s,a,s') + γ max_{a'} Q(s',a') - Q(s,a) خطایی را بازنمایی می‌کند که به‌روزرسانی می‌کوشد کمینه کند. بخشِ مهمِ این معادله چیزی است که شاملِ آن نیست: عاملِ TD کیو-یادگیری نه برای یادگیری و نه برای انتخابِ کنش به مدلِ انتقالِ P(s'|s,a) نیاز ندارد. چنان‌که در آغازِ فصل اشاره شد، روش‌های بدونِ مدل را می‌توان حتی در حوزه‌های بسیار پیچیده به‌کار برد، زیرا هیچ مدلی نیاز به ارائه یا یادگیری ندارد. از سوی دیگر، عاملِ کیو-یادگیری هیچ وسیله‌ای برای نگاه‌کردن به آینده ندارد، پس ممکن است وقتی پاداش‌ها پراکنده‌اند و باید دنباله‌های کنشِ بلندی برای رسیدن به آن‌ها ساخته شود دچار مشکل شود.

طراحیِ کاملِ عامل برای عاملِ TD کیو-یادگیریِ کاوشگر در شکل ۲۲.۸ نشان داده شده. توجه کنید دقیقاً از همان تابعِ کاوشِ f استفاده می‌کند که عاملِ ADP کاوشگر استفاده می‌کرد—از این رو نیاز به نگه‌داشتنِ آمارِ کنش‌های انجام‌شده (جدولِ N). اگر سیاستِ کاوشِ ساده‌تری استفاده شود—مثلاً عملِ تصادفی در کسری از گام‌ها، که در آن کسر در طولِ زمان کاهش می‌یابد—می‌توانیم از آمار صرف‌نظر کنیم.

کیو-یادگیری خویشاوندِ نزدیکی به نامِ SARSA دارد (برای حالت، کنش، پاداش، حالت، کنش). قاعدهٔ به‌روزرسانیِ SARSA بسیار شبیهِ قاعدهٔ به‌روزرسانیِ کیو-یادگیری (معادلهٔ (22.7)) است، با این تفاوت که SARSA با مقدارِ Q کنشی که واقعاً گرفته می‌شود، a'، به‌روزرسانی می‌کند:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[R(s,a,s') + \gamma\, Q(s',a') - Q(s,a)] \qquad (22.8)$$

قاعده در پایانِ هر پنج‌تاییِ 's, a, r, s', a اعمال می‌شود—از این رو نامِ آن. تفاوت با کیو-یادگیری کاملاً ظریف است: در حالی که کیو-یادگیری مقدارِ Q را از بهترین کنش در 's پشتیبان‌گیری می‌کند، SARSA صبر می‌کند تا کنشی گرفته شود و مقدارِ Q را برای همان کنش پشتیبان‌گیری می‌کند. اگر عامل حریص باشد و همیشه کنشِ با بهترین مقدارِ Q را بگیرد، دو الگوریتم یکسان‌اند. اما وقتی کاوش در جریان است، متفاوت‌اند: اگر کاوش پاداشِ منفی به بار آورد، SARSA کنش را جریمه می‌کند، در حالی که کیو-یادگیری نمی‌کند.

کیو-یادگیری الگوریتمِ یادگیریِ «خارج‌از-سیاست» (Off-Policy) است، زیرا مقادیرِ Qای را یاد می‌گیرد که به این پرسش پاسخ می‌دهند: «این کنش در این حالت چقدر ارزش دارد، به فرضِ اینکه استفاده از هر سیاستی که اکنون به‌کار می‌برم را متوقف کنم و طبقِ سیاستی شروع به عمل کنم که بهترین کنش را انتخاب می‌کند (طبقِ برآوردهای من)؟» SARSA الگوریتمِ «درون-سیاست» (On-Policy) است: مقادیرِ Qای را یاد می‌گیرد که به این پرسش پاسخ می‌دهند: «این کنش در این حالت چقدر ارزش دارد، به فرضِ اینکه به سیاستم بچسبم؟» کیو-یادگیری از SARSA انعطاف‌پذیرتر است، از این نظر که عاملِ کیو-یادگیری می‌تواند یاد بگیرد تحتِ کنترلِ طیفِ وسیعی از سیاست‌های کاوش خوب رفتار کند. از سوی دیگر، SARSA اگر سیاستِ کلی حتی تا حدی توسط عامل‌ها یا برنامه‌های دیگر کنترل شود مناسب است، که در آن صورت بهتر است تابعِ Qای برای آنچه واقعاً رخ خواهد داد یاد بگیریم، نه آنچه اگر عامل اجازه می‌یافت بهترین کنش‌های برآوردی را انتخاب کند رخ می‌داد. هر دوِ کیو-یادگیری و SARSA سیاستِ بهینه را برای جهانِ ۴×۳ یاد می‌گیرند، اما این کار را با نرخی بسیار کندتر از عاملِ ADP انجام می‌دهند. این به‌دلیلِ آن است که به‌روزرسانی‌های محلی از طریقِ مدل، سازگاری را در میانِ همهٔ مقادیرِ Q اعمال نمی‌کنند.

function Q-LEARNING-AGENT(*percept*) **returns** an action
  inputs: *percept*, a percept indicating the current state s' and reward signal r
  persistent: Q, a table of action values indexed by state and action, initially zero
    N_{sa}, a table of frequencies for state–action pairs, initially zero
    s, a, the previous state and action, initially null

  if s is not null **then**
    increment $N_{sa}[s,a]$
    $Q[s,a] \leftarrow Q[s,a] + \alpha(N_{sa}[s,a])(r + \gamma \max_{a'} Q[s',a'] - Q[s,a])$
  $s,a \leftarrow s', \text{argmax}_{a'} f(Q[s',a'], N_{sa}[s',a'])$
  return a

شکل ۲۲.۸ — عاملِ کیو-یادگیریِ کاوشگر. یادگیرندهٔ فعالی است که ارزشِ Q(s,a) هر کنش را در هر موقعیت یاد می‌گیرد. از همان تابعِ کاوشِ f مثلِ عاملِ ADP کاوشگر استفاده می‌کند، اما از یادگیریِ مدلِ انتقال اجتناب می‌ورزد.

۲۲.۴ تعمیم در یادگیریِ تقویتی

تاکنون فرض کرده‌ایم توابعِ مطلوبیت و توابعِ Q به‌صورتِ جدولی با یک مقدارِ خروجی برای هر حالت بازنمایی می‌شوند. این برای فضاهای حالتی تا حدودِ 10^6 حالت کار می‌کند، که برای محیط‌های شبکه‌ایِ دوبعدیِ اسباب‌بازیِ ما بیش از کافی است. اما در محیط‌های جهانِ واقعی با حالت‌های بسیار بیشتر، همگرایی بیش‌ازحد کند خواهد بود. تخته‌نرد ساده‌تر از بیشترِ کاربردهای جهانِ واقعی است، با این حال حدودِ 10^{20} حالت دارد. نمی‌توانیم به‌آسانی همهٔ آن‌ها را برای یادگیریِ بازی بازدید کنیم.

فصل ۵ ایدهٔ تابعِ ارزیابی را به‌عنوانِ سنجهٔ فشردهٔ مطلوبیت برای فضاهای حالتیِ بالقوهٔ بسیار بزرگ معرفی کرد. در اصطلاحِ این فصل، تابعِ ارزیابی تابعِ مطلوبیتِ تقریبی است؛ از اصطلاحِ «تقریبِ تابع» (Function Approximation) برای فرایندِ ساختِ تقریبِ فشرده‌ای از تابعِ مطلوبیت یا تابعِ Q واقعی استفاده می‌کنیم. برای نمونه، ممکن است تابعِ مطلوبیت را با ترکیبِ خطیِ وزن‌داری از ویژگی‌های $f_1, \ldots, f_n$ تقریب بزنیم:

$$\hat{U}_\theta(s) = \theta_1 f_1(s) + \theta_2 f_2(s) + \cdots + \theta_n f_n(s)$$

به‌جای یادگیریِ 10^{20} مقدارِ حالت در جدول، الگوریتمِ یادگیریِ تقویتی می‌تواند، مثلاً، ۲۰ مقدار برای پارامترهای $\theta = \theta_1, \ldots, \theta_{20}$ یاد بگیرد که $\hat{U}_\theta$ را تقریبِ خوبی از تابعِ مطلوبیتِ واقعی می‌کنند. گاهی این تابعِ مطلوبیتِ تقریبی با جستجویِ نگاه-به-جلو ترکیب می‌شود تا تصمیم‌های دقیق‌تری تولید کند. افزودنِ جستجویِ نگاه-به-جلو به این معناست که رفتارِ مؤثر را می‌توان از تقریب‌گرِ تابعِ مطلوبیتِ بسیار ساده‌تری تولید کرد که از تجربه‌های بسیار کمتری قابلِ یادگیری است. تقریبِ تابعِ بازنماییِ مطلوبیت (یا Q) را برای فضاهای حالتِ بسیار بزرگ عملی می‌کند، اما مهم‌تر، تعمیمِ استقرایی را ممکن می‌کند: عامل می‌تواند از حالت‌هایی که بازدید کرده به حالت‌هایی که هنوز بازدید نکرده تعمیم دهد. تسائورو (1992) از این فن برای ساختنِ برنامهٔ تخته‌نردبازی استفاده کرد که در سطحِ قهرمانِ انسانی بازی می‌کرد، حتی اگر فقط یک هزارتریلیونمِ فضایِ حالتِ کاملِ تخته‌نرد را کاوش کرده بود.

۲۲.۴.۱ تقریبِ برآوردِ مستقیمِ مطلوبیت

روشِ برآوردِ مستقیمِ مطلوبیت (بخش ۲۲.۲) مسیرهایی در فضایِ حالت تولید و برای هر حالت، مجموعِ پاداش‌های دریافتی از آن حالت به بعد تا پایان را استخراج می‌کند. حالت و مجموعِ پاداش‌های دریافتی مثالِ آموزشی‌ای برای الگوریتمِ یادگیریِ نظارت‌شده تشکیل می‌دهند. برای نمونه، فرض کنید مطلوبیت‌های جهانِ ۴×۳ را با تابعِ خطیِ ساده‌ای بازنمایی می‌کنیم، که در آن ویژگی‌های مربع‌ها صرفاً مختصاتِ x و y آن‌هاست. در آن صورت داریم:

$$\hat{U}_\theta(x,y) = \theta_0 + \theta_1 x + \theta_2 y \qquad (22.9)$$

پس، اگر $(\theta_0, \theta_1, \theta_2) = (0.5, 0.2, 0.1)$ باشد، آنگاه $\hat{U}_\theta(1,1) = 0.8$. با توجه به مجموعه‌ای از کوشش‌ها، مجموعه‌ای از مقادیرِ نمونهٔ $\hat{U}_\theta(x,y)$ به‌دست می‌آوریم و می‌توانیم بهترین برازش را، به معنایِ کمینه‌کردنِ مربعِ خطا، با رگرسیونِ خطیِ استاندارد بیابیم (فصل ۱۹ را ببینید).

برای یادگیریِ تقویتی، استفاده از الگوریتمِ یادگیریِ برخطی که پارامترها را پس از هر کوشش به‌روزرسانی می‌کند منطقی‌تر است. فرض کنید کوششی را اجرا می‌کنیم و مجموعِ پاداشِ به‌دست‌آمده با شروع از (1,1) برابرِ 0.4 است. این پیشنهاد می‌کند $\hat{U}_\theta(1,1)$، که در حالِ حاضر 0.8 است، بیش‌ازحد بزرگ است و باید کاهش یابد. پارامترها چگونه باید برای رسیدن به این تنظیم شوند؟ مانندِ یادگیریِ شبکهٔ عصبی، تابعِ خطایی می‌نویسیم و گرادیانش را نسبت به پارامترها محاسبه می‌کنیم. اگر $u_j(s)$ مجموعِ پاداشِ مشاهده‌شده از حالتِ s به بعد در jاُمین کوشش باشد، خطا به‌عنوانِ (نصفِ) مجذورِ تفاضلِ کلِ پیش‌بینی‌شده و کلِ واقعی تعریف می‌شود:

$$E_j(s) = (\hat{U}_\theta(s) - u_j(s))^2 / 2$$

نرخِ تغییرِ خطا نسبت به هر پارامترِ θ_i برابرِ $\partial E_j/\partial\theta_i$ است، پس برای حرکتِ پارامتر در جهتِ کاهشِ خطا، می‌خواهیم:

$$\theta_i \leftarrow \theta_i - \alpha\, \partial E_j(s)/\partial\theta_i = \theta_i + \alpha[u_j(s) - \hat{U}_\theta(s)]\, \partial\hat{U}_\theta(s)/\partial\theta_i \qquad (22.10)$$

این «قاعدهٔ ویدرو–هاف» (Widrow–Hoff Rule) یا «قاعدهٔ دلتا» (Delta Rule) برای کمترین مربعاتِ برخط نامیده می‌شود. برای تقریب‌گرِ تابعِ خطیِ $\hat{U}_\theta(s)$ در معادلهٔ (22.9)، سه قاعدهٔ به‌روزرسانیِ ساده به‌دست می‌آوریم:

$$\theta_0 \leftarrow \theta_0 + \alpha[u_j(s) - \hat{U}_\theta(s)]$$
$$\theta_1 \leftarrow \theta_1 + \alpha[u_j(s) - \hat{U}_\theta(s)]\, x$$
$$\theta_2 \leftarrow \theta_2 + \alpha[u_j(s) - \hat{U}_\theta(s)]\, y$$

می‌توانیم این قاعده‌ها را بر مثالِ $\hat{U}_\theta(1,1) = 0.8$ و $u_j(1,1) = 0.4$ اعمال کنیم. پارامترهای θ_0، θ_1 و θ_2 همگی به‌مقدارِ 0.4α کاهش می‌یابند، که خطایِ (1,1) را کاهش می‌دهد. توجه کنید تغییرِ پارامترهای θ_i در پاسخ به انتقالِ مشاهده‌شده‌ای بینِ دو حالت، مقادیرِ $\hat{U}_\theta$ را برای هر حالتِ دیگر نیز تغییر می‌دهد! این همان چیزی است که می‌گوییم تقریبِ تابع به یادگیرندهٔ تقویتی اجازه می‌دهد از تجربه‌هایش تعمیم دهد.

اگر فضایِ فرضیه بیش‌ازحد بزرگ نباشد و شاملِ برخی توابع باشد که برازشِ به‌اندازهٔ کافی خوبی برای تابعِ مطلوبیتِ واقعی‌اند، عامل با تقریب‌گرِ تابع سریع‌تر یاد می‌گیرد. تمرین APLM.۲۲ از شما می‌خواهد عملکردِ برآوردِ مستقیمِ مطلوبیت را، با و بدونِ تقریبِ تابع، ارزیابی کنید. بهبود در جهانِ ۴×۳ محسوس اما نه چشمگیر است، زیرا این از ابتدا فضایِ حالتِ بسیار کوچکی است. بهبود در جهانِ 10×10 با پاداشِ +1 در (10,10) بسیار بیشتر است.

جهانِ 10×10 برای تابعِ مطلوبیتِ خطی بسیار مناسب است، زیرا تابعِ مطلوبیتِ واقعی هموار و تقریباً خطی است: اساساً شیبِ قطری است با گوشهٔ پایینی‌اش در (1,1) و گوشهٔ بالایی‌اش در (10,10). (تمرین TENX.۲۲ را ببینید.) از سوی دیگر، اگر پاداشِ +1 را در (5,5) بگذاریم، مطلوبیتِ واقعی بیشتر شبیهِ هرم است و تقریب‌گرِ تابعِ معادلهٔ (22.9) به‌شکلِ فاجعه‌باری شکست می‌خورد.

اما همه‌چیز از دست نرفته! به یاد داشته باشید آنچه برای تقریبِ تابعِ خطی اهمیت دارد این است که تابع در ویژگی‌ها خطی باشد. اما می‌توانیم ویژگی‌ها را توابعِ غیرخطیِ دلبخواهِ متغیرهای حالت انتخاب کنیم. از این رو، می‌توانیم ویژگی‌ای مانندِ $f_3(x,y) = \sqrt{((x-x_g)^2 + (y-y_g)^2)}$ بگنجانیم که فاصله تا هدف را اندازه می‌گیرد. با این ویژگیِ تازه، تقریب‌گرِ تابعِ خطی خوب عمل می‌کند.

۲۲.۴.۲ تقریبِ یادگیریِ تفاضلِ زمانی

می‌توانیم این ایده‌ها را به‌یکسان بر یادگیرنده‌های تفاضلِ زمانی اعمال کنیم. فقط باید پارامترها را طوری تنظیم کنیم که کوشش کنند تفاضلِ زمانیِ بینِ حالت‌های متوالی را کاهش دهند. نسخه‌های تازهٔ معادله‌های TD و کیو-یادگیری (۲۲.۳ در صفحهٔ ۷۹۵ و ۲۲.۷ در صفحهٔ ۸۰۲) به‌صورتِ زیرند:

$$\theta_i \leftarrow \theta_i + \alpha[R(s,a,s') + \gamma\, \hat{U}_\theta(s') - \hat{U}_\theta(s)]\, \partial \hat{U}_\theta(s)/\partial\theta_i \qquad (22.11)$$

برای مطلوبیت‌ها و

$$\theta_i \leftarrow \theta_i + \alpha[R(s,a,s') + \gamma \max_{a'} \hat{Q}_\theta(s',a') - \hat{Q}_\theta(s,a)]\, \partial \hat{Q}_\theta(s,a)/\partial\theta_i \qquad (22.12)$$

برای مقادیرِ Q. برای یادگیریِ TD منفعل، می‌توان نشان داد قاعدهٔ به‌روزرسانی وقتی تقریب‌گرِ تابع در ویژگی‌ها خطی است به نزدیک‌ترین تقریبِ ممکن از تابعِ واقعی همگرا می‌شود.[4] با یادگیریِ فعال و توابعِ غیرخطی مانندِ شبکه‌های عصبی، تقریباً همهٔ شرط‌ها از بین می‌رود: حالت‌های بسیار ساده‌ای هست که در آن‌ها پارامترها با این قاعده‌های به‌روزرسانی می‌توانند به بی‌نهایت بروند، حتی اگر راه‌حل‌های خوبی در فضایِ فرضیه وجود داشته باشد. الگوریتم‌های پیچیده‌تری هست که می‌توانند از این مشکلات اجتناب کنند، اما در حالِ حاضر یادگیریِ تقویتی با تقریب‌گرهای تابعِ عمومی هنرِ ظریفی باقی مانده.

علاوه بر واگراییِ پارامترها به بی‌نهایت، مشکلِ شگفت‌آورتری به نامِ «فراموشیِ فاجعه‌بار» (Catastrophic Forgetting) هست. فرض کنید خودروِی خودمختاری را آموزش می‌دهید که (به‌صورتِ شبیه‌سازی‌شده) به‌طورِ ایمن رانندگی کند و تصادف نکند. برای عبور از لبهٔ جاده پاداشِ منفیِ بزرگی تعیین می‌کنید، و از ویژگی‌های درجهٔ دومِ موقعیتِ جاده استفاده می‌کنید تا خودرو بتواند یاد بگیرد مطلوبیتِ بودنِ در وسطِ جاده از نزدیک‌بودن به لبه بالاتر است. همه‌چیز خوب پیش می‌رود و خودرو یاد می‌گیرد به‌طورِ کامل درست در وسطِ جاده رانندگی کند. پس از چند دقیقه از این کار، شروع به بی‌حوصلگی می‌کنید و در آستانه‌اید شبیه‌سازی را متوقف و نتایجِ عالی را ثبت کنید. ناگهان، خودرو از جاده منحرف می‌شود و تصادف می‌کند. چرا؟

آنچه رخ داده این است که خودرو بیش‌ازحد خوب یاد گرفته: چون یاد گرفته از لبه دور بماند، یاد گرفته کلِ ناحیهٔ مرکزیِ جاده مکانِ امنی است، و فراموش کرده ناحیهٔ نزدیک‌تر به لبه خطرناک است. ناحیهٔ مرکزی بنابراین تابعِ ارزشِ تخت دارد، پس ویژگی‌های درجهٔ دوم وزنِ صفر می‌گیرند؛ سپس، هر وزنِ ناصفری رویِ ویژگی‌های خطی باعث می‌شود خودرو به یک سمت یا سمتِ دیگر از جاده سُر بخورد.

یک راهِ حل برای این مسئله، که «بازپخشِ تجربه» (Experience Replay) نامیده می‌شود، تضمین می‌کند خودرو در فواصلِ منظم رفتارِ تصادف‌کنندهٔ جوانی‌اش را دوباره زنده کند. الگوریتمِ یادگیری می‌تواند مسیرهای کلِ فرایندِ یادگیری را نگه دارد و آن مسیرها را بازپخش کند تا مطمئن شود تابعِ ارزشش برای بخش‌هایی از فضایِ حالت که دیگر بازدید نمی‌کند همچنان دقیق است.

برای سامانه‌های یادگیریِ تقویتیِ مبتنی بر مدل، تقریبِ تابع همچنین می‌تواند برای یادگیریِ مدلی از محیط بسیار کمک‌کننده باشد. به یاد داشته باشید یادگیریِ مدل برای محیطِ مشاهده‌پذیر مسئلهٔ یادگیریِ نظارت‌شده است، زیرا ادراکِ بعدی حالتِ پیامد را می‌دهد. هر یک از روش‌های یادگیریِ نظارت‌شدهٔ فصل‌های ۱۹ تا ۲۱ را می‌توان استفاده کرد، با تنظیماتِ مناسب برای این واقعیت که باید توصیفِ کاملِ حالت را پیش‌بینی کنیم نه فقط رده‌بندیِ بولی یا مقدارِ حقیقیِ تکی. با مدلِ آموخته‌شده، عامل می‌تواند جستجویِ نگاه-به-جلو برای بهبودِ تصمیم‌هایش انجام دهد و می‌تواند شبیه‌سازی‌های درونی برای بهبودِ بازنمایی‌های تقریبیِ U یا Q انجام دهد، به‌جای نیاز به تجربه‌های جهانِ واقعیِ کند و بالقوه پرهزینه.

برای محیطِ با مشاهدهٔ جزئی، مسئلهٔ یادگیری بسیار دشوارتر است، زیرا ادراکِ بعدی دیگر برچسبی برای مسئلهٔ پیش‌بینیِ حالت نیست. اگر بدانیم متغیرهای پنهان چه‌اند و چگونه به‌صورتِ علی به یکدیگر و به متغیرهای مشاهده‌پذیر مربوط‌اند، می‌توانیم ساختارِ شبکهٔ بیزیِ پویا را ثابت کنیم و از الگوریتمِ EM برای یادگیریِ پارامترها استفاده کنیم، چنان‌که در فصل ۲۰ توصیف شد. یادگیریِ ساختارِ درونیِ شبکه‌های بیزیِ پویا و ایجادِ متغیرهای حالتِ تازه هنوز مسئلهٔ دشواری تلقی می‌شود. شبکه‌های عصبیِ بازگشتیِ عمیق (بخش ۲۱.۶) در برخی موارد در ابداعِ ساختارِ پنهان موفق بوده‌اند.

۲۲.۴.۳ یادگیریِ تقویتیِ عمیق

دو دلیل برای نیاز به فراتررفتن از تقریب‌گرهای تابعِ خطی هست: نخست، ممکن است هیچ تابعِ خطیِ خوبی وجود نداشته باشد که به تقریبِ تابعِ مطلوبیت یا تابعِ Q نزدیک شود؛ دوم، ممکن است نتوانیم ویژگی‌های لازم را ابداع کنیم، به‌ویژه در حوزه‌های تازه. اگر به آن فکر کنید، این‌ها واقعاً یک دلیل‌اند: همیشه می‌توان U یا Q را به‌صورتِ ترکیبِ خطیِ ویژگی‌ها بازنمایی کرد، به‌ویژه اگر ویژگی‌هایی مانندِ $f_1(s) = U(s)$ یا $f_2(s,a) = Q(s,a)$ داشته باشیم، اما مگر اینکه بتوانیم چنین ویژگی‌هایی را (به‌شکلی کارآمدِ محاسباتی) ارائه کنیم، تقریب‌گرِ تابعِ خطی ممکن است ناکافی باشد.

به این دلیل‌ها (یا دلیل)، پژوهشگران از نخستین روزهای یادگیریِ تقویتی تقریب‌گرهای تابعِ غیرخطیِ پیچیده‌تر را کاوش کرده‌اند. در حالِ حاضر، شبکه‌های عصبیِ عمیق (فصل ۲۱) در این نقش بسیار محبوب‌اند و ثابت شده حتی وقتی ورودی تصویرِ خامی بدونِ هیچ استخراجِ ویژگیِ دستی است مؤثرند. اگر همه‌چیز خوب پیش برود، شبکهٔ عصبیِ عمیق در واقع ویژگی‌های مفید را خودش کشف می‌کند. و اگر لایهٔ پایانیِ شبکه خطی باشد، می‌توانیم ببینیم شبکه از چه ویژگی‌هایی برای ساختنِ تقریب‌گرِ تابعِ خطیِ خودش استفاده می‌کند. سامانهٔ یادگیریِ تقویتی که از شبکهٔ عمیق به‌عنوانِ تقریب‌گرِ تابع استفاده می‌کند «سامانهٔ یادگیریِ تقویتیِ عمیق» نامیده می‌شود.

درست مانندِ معادلهٔ (22.9)، شبکهٔ عمیق تابعی است که با θ پارامتری شده، با این تفاوت که حالا تابع بسیار پیچیده‌تر است. پارامترها همهٔ وزن‌های همهٔ لایه‌های شبکه‌اند. با این حال، گرادیان‌های لازم برای معادله‌های (22.11) و (22.12) دقیقاً همان گرادیان‌های لازم برای یادگیریِ نظارت‌شده‌اند، و با همان فرایندِ پسانتشارِ توصیف‌شده در بخش ۲۱.۴ قابلِ محاسبه‌اند.

چنان‌که در بخش ۲۲.۷ توضیح می‌دهیم، یادگیریِ تقویتیِ عمیق به نتایجِ بسیار چشمگیری رسیده، از جملهِ یادگیریِ بازیِ طیفِ وسیعی از بازی‌های ویدئویی در سطحِ خبره، شکستِ قهرمانِ جهانِ انسان در بازیِ گو، و آموزشِ ربات‌ها برای انجامِ تکلیف‌های پیچیده.

با وجودِ موفقیت‌های چشمگیرش، یادگیریِ تقویتیِ عمیق هنوز با موانعِ چشمگیری روبه‌روست: اغلب به‌دست‌آوردنِ عملکردِ خوب دشوار است و اگر محیط حتی اندکی با دادهٔ آموزشی تفاوت داشته باشد، سامانهٔ آموزش‌دیده ممکن است بسیار غیرقابلِ‌پیش‌بینی رفتار کند. در مقایسه با دیگر کاربردهای یادگیری عمیق، یادگیریِ تقویتیِ عمیق به‌ندرت در محیط‌های تجاری به‌کار می‌رود. با این حال، حوزهٔ پژوهشیِ بسیار فعالی است.

۲۲.۴.۴ شکل‌دهیِ پاداش

چنان‌که در مقدمهٔ این فصل اشاره شد، محیط‌های جهانِ واقعی ممکن است پاداش‌های بسیار پراکنده‌ای داشته باشند: برای رسیدن به هر پاداشِ ناصفری کنش‌های ابتداییِ بسیاری لازم است. برای نمونه، رباتِ فوتبالیست ممکن است پیش از گل‌خوردن صد هزار فرمانِ کنترلِ موتور به مفصل‌های مختلفش بفرستد. حالا باید بفهمد چه اشتباهی کرده. اصطلاحِ فنیِ این «مسئلهٔ تخصیصِ اعتبار» (Credit Assignment Problem) است. به‌جز بازیِ تریلیون‌ها بازیِ فوتبال تا پاداشِ منفی در نهایت به کنش‌های مسئولش به عقب منتشر شود، آیا راهِ حلِ خوبی هست؟ یکی از روش‌های رایج، که در اصل در آموزشِ حیوانات استفاده می‌شد، «شکل‌دهیِ پاداش» (Reward Shaping) نامیده می‌شود. این شاملِ فراهم‌کردنِ پاداش‌های اضافی به عامل، به نامِ «شبه‌پاداش» (Pseudoreward)، برای «پیشرفت‌کردن» است. برای نمونه، ممکن است به ربات برای تماس‌برقرارکردنِ با توپ یا پیش‌بردنِ توپ به سمتِ دروازه شبه‌پاداش بدهیم. چنین پاداش‌هایی می‌توانند یادگیری را به‌طورِ عظیمی سرعت ببخشند و فراهم‌کردنشان ساده است، اما این خطر هست که عامل یاد بگیرد شبه‌پاداش‌ها را به‌جای پاداش‌های واقعی بیشینه کند؛ برای نمونه، ایستادنِ کنارِ توپ و «لرزیدن» تماس‌های بسیاری با توپ ایجاد می‌کند.

در فصل ۱۷ (صفحهٔ ۵۶۹)، راهی برای اصلاحِ تابعِ پاداش بدونِ تغییرِ سیاستِ بهینه دیدیم. برای هر تابعِ پتانسیلِ Φ(s) و هر تابعِ پاداشِ R، می‌توانیم تابعِ پاداشِ تازه‌ای R' به‌صورتِ زیر بسازیم:

$$R'(s,a,s') = R(s,a,s') + \gamma\Phi(s') - \Phi(s)$$

تابعِ پتانسیلِ Φ را می‌توان چنان ساخت که هر جنبهٔ مطلوبِ حالت را بازتاب دهد، مانندِ دست‌یابی به اهدافِ فرعی یا فاصله تا حالتِ پایانیِ مطلوب. برای نمونه، Φ برای رباتِ فوتبالیست می‌توانست پاداشِ ثابتی برای حالت‌هایی که تیمِ ربات صاحبِ توپ است و پاداشِ دیگری برای کاهشِ فاصلهٔ توپ تا دروازهٔ حریف اضافه کند. این در مجموع به یادگیریِ

سریع‌تر خواهد انجامید، اما ربات را مثلاً از یادگیریِ پاسِ دادنِ به عقب به دروازه‌بان وقتی خطر تهدید می‌کند بازنخواهد داشت.

۲۲.۴.۵ یادگیریِ تقویتیِ سلسله‌مراتبی

راهِ دیگرِ مقابله با دنباله‌های کنشِ بسیار بلند این است که آن‌ها را به چند تکهٔ کوچک‌تر بشکنیم، و سپس آن‌ها را به تکه‌های کوچک‌ترِ هنوز بشکنیم، و همین‌طور ادامه دهیم تا دنباله‌های کنش به‌اندازهٔ کافی کوتاه شوند تا یادگیری آسان شود. این رویکرد «یادگیریِ تقویتیِ سلسله‌مراتبی» (Hierarchical Reinforcement Learning یا HRL) نامیده می‌شود، و با روش‌های برنامه‌ریزیِ HTN توصیف‌شده در فصل ۱۱ اشتراکاتِ زیادی دارد. برای نمونه، گل‌زدن در فوتبال را می‌توان به کسبِ تصاحبِ توپ، پاس به هم‌تیمی، دریافتِ توپ از هم‌تیمی، دریبل به سمتِ دروازه و شوت شکست؛ هر یک از این‌ها را می‌توان بیشتر به رفتارهای حرکتیِ سطح‌پایین‌تر شکست. بدیهی است که راه‌های متعددی برای کسبِ تصاحب و شوت‌زدن هست، هم‌تیمی‌های متعددی که می‌توان به آن‌ها پاس داد، و مانندِ آن، پس هر کنشِ سطح‌بالا ممکن است پیاده‌سازی‌های سطح‌پایینِ متفاوتِ بسیاری داشته باشد.

برای نشان‌دادنِ این ایده‌ها، از بازیِ فوتبالِ ساده‌شده‌ای به نامِ «کیپ‌اِوِی» (Keepaway) استفاده می‌کنیم، که در آن یک تیمِ سه‌نفره می‌کوشد با دریبل‌کردن و پاس‌دادن در میانِ خودش تا جایی که ممکن است توپ را در تصاحب نگه دارد، در حالی که تیمِ دیگرِ دونفره می‌کوشد با قطعِ پاس یا تکلِ بازیکنِ صاحبِ توپ تصاحب را به‌دست آورد.[5] بازی در شبیه‌سازِ RoboCup دوبعدی پیاده‌سازی شده، که مدل‌های حرکتِ حالت-پیوستهٔ دقیق با گام‌های زمانیِ 100 میلی‌ثانیه‌ای فراهم می‌کند و بسترِ آزمونِ خوبی برای سامانه‌های RL ثابت شده.

عاملِ یادگیریِ تقویتیِ سلسله‌مراتبی با «برنامهٔ جزئی» (Partial Program) شروع می‌کند که ساختارِ سلسله‌مراتبیِ رفتارِ عامل را ترسیم می‌کند. زبانِ برنامه‌نویسیِ جزئی برای برنامه‌های عامل هر زبانِ برنامه‌نویسیِ معمولی را با افزودنِ اجزایی برای انتخاب‌های نامشخصی که باید توسط یادگیری پر شوند بسط می‌دهد. (در این‌جا، از شبه‌کد برای زبانِ برنامه‌نویسی استفاده می‌کنیم.) برنامهٔ جزئی می‌تواند دلخواهاً پیچیده باشد، تا زمانی که خاتمه یابد.

به‌آسانی دیده می‌شود HRL شاملِ RL معمولی به‌عنوانِ حالتِ خاص است. کافی است برنامهٔ جزئیِ پیش‌پاافتاده‌ای فراهم کنیم که به عامل اجازه می‌دهد به انتخابِ هر کنشی از A(s)، مجموعهٔ کنش‌هایی که در حالتِ کنونیِ s قابلِ اجرایند، ادامه دهد:

while true do choose(A(s)). (به فارسی: «تا ابد: از A(s) انتخاب کن» — عملگرِ choose به عامل اجازه می‌دهد هر عنصرِ مجموعهٔ مشخص‌شده را انتخاب کند.)

عملگرِ choose به عامل اجازه می‌دهد هر عنصرِ مجموعهٔ مشخص‌شده را انتخاب کند. فرایندِ یادگیری برنامهٔ عاملِ جزئی را با یادگیریِ نحوهٔ انجامِ هر انتخاب به برنامهٔ کامل تبدیل می‌کند. برای نمونه، فرایندِ یادگیری ممکن است با هر انتخاب تابعِ Qای مرتبط کند؛ پس از یادگیریِ توابعِ Q، برنامه با انتخابِ گزینه‌ای با بیشترین مقدارِ Q هر بار که به انتخابی برخورد می‌کند رفتار تولید می‌کند.

برنامه‌های عامل برای کیپ‌اوِی جالب‌ترند. به برنامهٔ جزئیِ بازیکنی تکی در تیمِ «نگه‌دارنده» نگاه می‌کنیم. انتخابِ کارِ سطحِ بالا عمدتاً به این بستگی دارد که بازیکن توپ را دارد یا نه:

while not IS-TERMINAL(s) do if BALL-IN-MY-POSSESSION(s) then choose({PASS, HOLD, DRIBBLE}) else choose({STAY, MOVE, INTERCEPT-BALL}). (به فارسی: «تا وقتی حالت پایانی نیست: اگر توپ در تصاحبِ من است، از {پاس، نگه‌داشتن، دریبل} انتخاب کن؛ در غیر این صورت از {ماندن، حرکت، قطع‌کردنِ توپ} انتخاب کن.»)

هر یک از این انتخاب‌ها زیرروالِ‌ای را فرا می‌خواند که خودش ممکن است انتخاب‌های بیشتری انجام دهد، همه‌راه تا کنش‌های ابتدایی که مستقیماً قابلِ اجرایند. برای نمونه، کنشِ سطح‌بالایِ PASS هم‌تیمی‌ای را برای پاس انتخاب می‌کند، اما همچنین انتخابِ هیچ‌کارنکردن و بازگرداندنِ کنترل به سطحِ بالاتر را در صورتِ مناسب‌بودن دارد (مثلاً، اگر کسی برای پاس نباشد):

choose({PASS-TO(choose(TEAMMATES(s))), return}). (به فارسی: «از {پاس-به (انتخابِ هم‌تیمی‌های s)، بازگشت} انتخاب کن».)

روالِ PASS-TO سپس باید سرعت و جهتِ پاس را انتخاب کند. در حالی که ارائهٔ این‌گونه توصیه‌های سطح‌بالا به عاملِ یادگیرنده برای انسان نسبتاً آسان است—حتی کسی که تخصصِ کمی در فوتبال دارد—نوشتنِ قاعده‌های تعیینِ سرعت و جهتِ شوت برای بیشینه‌کردنِ احتمالِ حفظِ تصاحب دشوار، اگر نه غیرممکن، خواهد بود. به‌همین‌سان، انتخابِ هم‌تیمیِ درست برای دریافتِ توپ یا جایی که باید حرکت کرد تا خود را برای دریافتِ توپ در دسترس کرد اصلاً بدیهی نیست. برنامهٔ جزئی دانشِ کلی—داربستِ کلی و سازمان‌دهیِ ساختاری برای رفتارهای پیچیده—فراهم می‌کند، و فرایندِ یادگیری همهٔ جزئیات را می‌پردازد.

بنیان‌های نظریِ HRL بر مفهومِ «فضایِ حالتِ مشترک» (Joint State Space) مبتنی‌اند، که در آن هر حالتِ (s,m) از حالتِ فیزیکیِ s و حالتِ ماشینِ m تشکیل شده. حالتِ ماشین توسط حالتِ درونیِ کنونیِ برنامهٔ عامل تعریف می‌شود: شمارندهٔ برنامه برای هر زیرروالِ رویِ پشتهٔ فراخوانیِ کنونی، مقادیرِ آرگومان‌ها، و مقادیرِ همهٔ متغیرهای محلی و سراسری. برای نمونه، اگر برنامهٔ عامل انتخاب کرده به هم‌تیمیِ علی پاس دهد و در میانهٔ محاسبهٔ سرعتِ پاس باشد، این واقعیت که علی آرگومانِ PASS-TO است بخشی از حالتِ ماشینِ کنونی است. «حالتِ انتخاب» (Choice State) σ = (s,m) حالتی است که شمارندهٔ برنامه برای m در نقطهٔ انتخابی در برنامهٔ عامل است. بینِ دو حالتِ انتخاب، هر تعدادِ انتقالِ محاسباتی و کنشِ فیزیکی ممکن است رخ دهد، اما همهٔ آن‌ها، به‌اصطلاح، از پیش مقدرند: طبقِ تعریف، عامل هیچ انتخابی بینِ حالت‌های انتخاب نمی‌کند. اساساً، عاملِ RL سلسله‌مراتبی مسئلهٔ تصمیمِ مارکوفی را با عناصرِ زیر حل می‌کند:

- حالت‌ها، حالت‌های انتخابِ σ فضایِ حالتِ مشترک‌اند.
- کنش‌ها در σ، انتخاب‌هایِ c موجود در σ طبقِ برنامهٔ جزئی‌اند.
- تابعِ پاداشِ ρ(σ,c,σ') مجموعِ موردانتظارِ پاداش‌ها برای همهٔ انتقال‌های فیزیکیِ رخ‌داده بینِ حالت‌های انتخابِ σ و σ' است.

- مدلِ انتقالِ $\tau(\sigma,c,\sigma')$ به‌شیوهٔ آشکار تعریف می‌شود: اگر c کنشِ فیزیکیِ a را فرا بخواند، آنگاه τ از مدلِ فیزیکیِ $P(s'|s,a)$ وام می‌گیرد؛ اگر c انتقالِ محاسباتی را فرا بخواند، مانندِ فراخوانیِ زیرروال، آنگاه انتقال به‌صورتِ قطعی حالتِ محاسباتیِ m را طبقِ قاعده‌های زبانِ برنامه‌نویسی اصلاح می‌کند.[6]

با حلِ این مسئلهٔ تصمیم، عامل سیاستِ بهینه‌ای را می‌یابد که با برنامهٔ جزئیِ اصلی سازگار است.

یادگیریِ RL سلسله‌مراتبی می‌تواند روشِ بسیار مؤثری برای یادگیریِ رفتارهای پیچیده باشد. در کیپ‌اِوِی، عاملِ HRL مبتنی بر برنامهٔ جزئیِ ترسیم‌شده در بالا سیاستی را یاد می‌گیرد که توپ را برای همیشه در برابرِ سیاستِ استانداردِ تیمِ مهاجم نگه می‌دارد—بهبودِ چشمگیری نسبت به رکوردِ قبلیِ حدودِ ۱۰ ثانیه. یکی از ویژگی‌های مهم این است که مهارت‌های سطح‌پایین به معنایِ معمول زیرروال‌های ثابت نیستند؛ انتخاب‌هایشان به کلِ حالتِ درونیِ برنامهٔ عامل حساس است، پس بسته به اینکه در کجای آن برنامه فراخوانی می‌شوند و در آن زمان چه اتفاقی می‌افتد، متفاوت رفتار می‌کنند. در صورتِ نیاز، توابعِ Q انتخاب‌های سطح‌پایین را می‌توان با فرایندِ آموزشیِ جداگانه‌ای با تابعِ پاداشِ خودش مقداردهیِ اولیه کرد و سپس در سامانهٔ کلی ادغام کرد تا بتوانند برای عملکردِ خوب در بافتِ کلِ عامل سازگار شوند.

در بخشِ پیشین دیدیم شکل‌دهیِ پاداش‌ها می‌تواند برای یادگیریِ رفتارهای پیچیده کمک‌کننده باشد. در HRL، این واقعیت که یادگیری در فضایِ حالتِ مشترک رخ می‌دهد فرصت‌های اضافی برای شکل‌دهی فراهم می‌کند. برای نمونه، برای کمک به یادگیریِ تابعِ Q برای پاسِ دقیق در درونِ روالِ PASS-TO، می‌توانیم پاداشِ شکل‌دهی‌ای فراهم کنیم که به محلِ گیرندهٔ موردنظر و نزدیکیِ حریفان به آن بازیکن بستگی دارد: توپ باید نزدیکِ گیرنده و دور از حریفان باشد. این کاملاً بدیهی به نظر می‌رسد؛ اما هویتِ گیرندهٔ موردنظرِ پاس بخشی از حالتِ فیزیکیِ جهان نیست. حالتِ فیزیکی فقط از موقعیت‌ها، جهت‌ها و سرعت‌های بازیکنان و توپ تشکیل شده. هیچ «پاسی» و هیچ «گیرنده‌ای» در جهانِ فیزیکی نیست؛ این‌ها کاملاً ساختارهای درونی‌اند. این بدان‌معناست که هیچ راهی برای ارائهٔ چنین توصیهٔ معقولی به سامانهٔ RL استاندارد نیست.

ساختارِ سلسله‌مراتبیِ رفتار همچنین «تجزیهٔ افزایشی» (Additive Decomposition) طبیعی از تابعِ مطلوبیتِ کلی فراهم می‌کند. به یاد داشته باشید مطلوبیت مجموعِ پاداش‌ها در طولِ زمان است، و دنباله‌ای از، مثلاً، ده گامِ زمانی با پاداش‌های $[r_1, r_2, ..., r_{10}]$ را در نظر بگیرید. فرض کنید در پنج گامِ زمانیِ نخست عامل در حالِ PASS-TO(علی) و در پنج گامِ باقی‌مانده در حالِ MOVE-INTO-SPACE است. آنگاه مطلوبیتِ حالتِ آغازین مجموعِ پاداشِ کلِ در طولِ PASS-TO و پاداشِ کلِ در طولِ MOVE-INTO-SPACE است. اولی فقط به این بستگی دارد که آیا توپ با زمان و فضایِ کافی به علی می‌رسد تا علی بتواند تصاحب را حفظ کند، و دومی فقط به این بستگی دارد که آیا عامل به موقعیتِ خوبی برای دریافتِ توپ می‌رسد. به بیانِ دیگر، مطلوبیتِ کلی به چند عبارت تجزیه می‌شود که هر یک فقط به چند متغیرِ اندک بستگی دارد. این، به‌نوبهٔ خود، به این معناست که یادگیری بسیار سریع‌تر از حالتی رخ می‌دهد که بخواهیم تابعِ مطلوبیتِ واحدی یاد بگیریم که به همهٔ متغیرها بستگی دارد. این تا حدی مشابهِ قضیه‌های بازنماییِ زیربنایِ موجزبودنِ شبکه‌های بیزی است (فصل ۱۳).

۲۲.۵ جستجوی سیاست

آخرین رویکردی که برای مسائلِ یادگیریِ تقویتی بررسی می‌کنیم «جستجوی سیاست» (Policy Search) نامیده می‌شود. از برخی جهات، جستجوی سیاست ساده‌ترینِ همهٔ روش‌های این فصل است: ایده این است تا وقتی عملکردِ سیاست بهبود می‌یابد به دست‌کاریِ آن ادامه بده، سپس متوقف شو.

با خودِ سیاست‌ها شروع کنیم. به یاد داشته باشید سیاستِ π تابعی است که حالت‌ها را به کنش‌ها نگاشت می‌کند. ما عمدتاً به بازنمایی‌های پارامتریشدهٔ π علاقه‌مندیم که پارامترهایشان بسیار کمتر از حالت‌های فضایِ حالت باشد (درست مانندِ بخشِ پیشین). برای نمونه، می‌توانیم π را با مجموعه‌ای از توابعِ Q پارامتریشده، یکی برای هر کنش، بازنمایی کنیم و کنشی با بیشترین مقدارِ پیش‌بینی‌شده بگیریم:

$$\pi(s) = \operatorname{argmax}_a \hat{Q}_\theta(s,a) \qquad (22.13)$$

هر تابعِ Q می‌تواند تابعِ خطی باشد، مانندِ معادلهٔ (22.9)، یا تابعِ غیرخطی مانندِ شبکهٔ عصبیِ عمیق. جستجوی سیاست سپس پارامترهای θ را برای بهبودِ سیاست تنظیم می‌کند. توجه کنید اگر سیاست با توابعِ Q بازنمایی شود، جستجوی سیاست به فرایندی می‌انجامد که توابعِ Q یاد می‌گیرد. این فرایند با کیو-یادگیری یکی نیست! در کیو-یادگیری با تقریبِ تابع، الگوریتم مقدارِ θای می‌یابد که $\hat{Q}_\theta$ «نزدیکِ» Q^*، تابعِ Q بهینه، باشد. جستجوی سیاست، از سوی دیگر، مقدارِ θای می‌یابد که به عملکردِ خوب می‌انجامد؛ مقادیرِ یافت‌شده توسط دو روش ممکن است بسیار متفاوت باشند. (برای نمونه، تابعِ Q تقریبیِ تعریف‌شده با $\hat{Q}_\theta(s,a) = Q^*(s,a)/100$ عملکردِ بهینه می‌دهد، حتی اگر اصلاً به Q^* نزدیک نباشد.) نمونهٔ روشنِ دیگرِ تفاوت حالتی است که π(s) با، مثلاً، جستجویِ نگاه-به-جلوِ عمق-۱۰ با تابعِ مطلوبیتِ تقریبیِ $\hat{U}_\theta$ محاسبه می‌شود. مقدارِ θای که نتایجِ خوب می‌دهد ممکن است فاصلهٔ زیادی با حالتی داشته باشد که $\hat{U}_\theta$ شبیهِ تابعِ مطلوبیتِ واقعی شود.

یکی از مشکلاتِ بازنمایی‌های سیاستِ از نوعِ داده‌شده در معادلهٔ (22.13) این است که وقتی کنش‌ها گسسته‌اند، سیاست تابعی ناپیوسته از پارامترهاست. یعنی، مقادیری از θ وجود خواهند داشت که تغییرِ بی‌نهایتِ کوچکِ θ باعث می‌شود سیاست از کنشی به کنشِ دیگر بپرد. این بدان‌معناست که مقدارِ سیاست نیز ممکن است به‌صورتِ ناپیوسته تغییر کند، که جستجوی مبتنی بر گرادیان را دشوار می‌کند. به همین دلیل، روش‌های جستجوی سیاست اغلب از «بازنماییِ سیاستِ تصادفی» (Stochastic Policy) استفاده می‌کنند، $\pi_\theta(s,a)$، که احتمالِ انتخابِ کنشِ a در حالتِ s را مشخص می‌کند. یکی از بازنمایی‌های محبوب تابعِ سافتمکس است:

$$\pi_\theta(s,a) = e^{\beta \hat{Q}_\theta(s,a)} / \sum_{a'} e^{\beta \hat{Q}_\theta(s,a')} \qquad (22.14)$$

پارامترِ $\beta > 0$ نرمیِ سافتمکس را تعدیل می‌کند: برای مقادیرِ β که در مقایسه با جدایی‌های بینِ مقادیرِ Q بزرگ‌اند، سافتمکس به بیشینهٔ سخت نزدیک می‌شود، در حالی که برای مقادیرِ β نزدیکِ صفر، سافتمکس به انتخابِ تصادفیِ یکنواخت در میانِ کنش‌ها نزدیک می‌شود. برای همهٔ مقادیرِ متناهیِ β، سافتمکس تابعی مشتق‌پذیر از θ فراهم

می‌کند؛ از این رو، مقدارِ سیاست (که به‌صورتِ پیوسته به احتمال‌های انتخابِ کنش بستگی دارد) تابعی مشتق‌پذیر از θ است.

حالا به روش‌های بهبودِ سیاست نگاه کنیم. با ساده‌ترین حالت شروع می‌کنیم: سیاستِ قطعی و محیطِ قطعی. ρ(θ) «مقدارِ سیاست» (Policy Value) باشد، یعنی پاداشِ موردانتظارِ تا-پایان وقتی π_θ اجرا می‌شود. اگر بتوانیم عبارتی به‌صورتِ بسته برای ρ(θ) اشتقاق دهیم، مسئلهٔ بهینه‌سازیِ استانداردی داریم، چنان‌که در فصل ۴ توصیف شد. می‌توانیم «گرادیانِ سیاست» ∇_θ ρ(θ) را دنبال کنیم، به‌شرطِ اینکه ρ(θ) مشتق‌پذیر باشد. یا اگر ρ(θ) به‌صورتِ بسته در دسترس نباشد، می‌توانیم π_θ را صرفاً با اجرایِ آن و مشاهدهٔ پاداشِ انباشته ارزیابی کنیم. می‌توانیم گرادیانِ تجربی را با بالارفتن از تپه دنبال کنیم—یعنی ارزیابیِ تغییرِ مقدارِ سیاست برای افزایش‌های کوچک در هر پارامتر. با ملاحظه‌های معمول، این فرایند به بهینهٔ محلی در فضایِ سیاست همگرا خواهد شد.

وقتی محیط (یا سیاست) غیرقطعی است، اوضاع دشوارتر می‌شود. فرض کنید می‌کوشیم بالارفتن از تپه انجام دهیم، که مستلزمِ مقایسهٔ ρ(θ) و ρ(θ+Δθ) برایِ Δθ کوچک است. مشکل این است که مجموعِ پاداش برای هر کوشش ممکن است به‌شدت تغییر کند، پس برآوردهای مقدارِ سیاست از تعدادِ کمی کوشش کاملاً غیرقابلِ‌اعتماد خواهند بود؛ کوشش برای مقایسهٔ دو چنین برآوردی حتی غیرقابلِ‌اعتمادتر خواهد بود. یک راه‌حل صرفاً اجرایِ کلی کوشش است، با اندازه‌گیریِ واریانسِ نمونه‌ای و استفاده از آن برای تعیینِ اینکه کوششِ کافی برای نشانهٔ قابلِ‌اعتمادی از جهتِ بهبودِ ρ(θ) اجرا شده. متأسفانه، این برای بسیاری از مسائلِ واقعی که کوشش‌ها ممکن است پرهزینه، وقت‌گیر و شاید حتی خطرناک باشند غیرعملی است.

برای حالتِ سیاستِ غیرقطعیِ π_θ(s,a)، می‌توان برآوردِ بدونِ اریبِ گرادیان را در θ، یعنی ∇_θ ρ(θ)، مستقیماً از نتایجِ کوشش‌های اجراشده در θ به‌دست آورد. برای سادگی، این برآورد را برای حالتِ سادهٔ محیطِ اپیزودیک اشتقاق می‌دهیم که در آن هر کنشِ a پاداشِ R(s₀,a,s₀) می‌گیرد و محیط در s₀ از نو شروع می‌شود. در این حالت، مقدارِ سیاست فقط مقدارِ موردانتظارِ پاداش است، و داریم:

$$\nabla_\theta \rho(\theta) = \nabla_\theta \sum_a R(s_0,a,s_0)\, \pi_\theta(s_0,a) = \sum_a R(s_0,a,s_0)\, \nabla_\theta \pi_\theta(s_0,a)$$

حالا حقهٔ ساده‌ای انجام می‌دهیم تا این جمع بتواند با نمونه‌هایی که از توزیعِ احتمالیِ تعریف‌شده توسط π_θ(s₀,a) تولید می‌شوند تقریب زده شود. فرض کنید در مجموع N کوشش داریم و کنشِ گرفته‌شده در زاُمین کوشش a_j است. آنگاه:

$$\nabla_\theta \rho(\theta) = \sum_a \pi_\theta(s_0,a) \cdot [R(s_0,a,s_0)\, \nabla_\theta \pi_\theta(s_0,a)] / \pi_\theta(s_0,a) \approx (1/N) \sum_{j=1}^{N} [R(s_0,a_j,s_0)\, \nabla_\theta \pi_\theta(s_0,a_j)] / \pi_\theta(s_0,a_j)$$

پس، گرادیانِ واقعیِ مقدارِ سیاست با مجموعِ عبارت‌هایی تقریب زده می‌شود که شاملِ گرادیانِ احتمالِ انتخابِ کنش در هر کوشش‌اند. برای حالتِ ترتیبی، این به‌صورتِ زیر تعمیم می‌یابد:

$$\nabla_\theta \rho(\theta) \approx (1/N) \sum_{j=1}^{N} [u_j(s)\, \nabla_\theta \pi_\theta(s,a_j)] / \pi_\theta(s,a_j)$$

برای هر حالتِ بازدیدشدهٔ s، که در آن a_j در s در زُامین کوشش اجرا می‌شود و $u_j(s)$ مجموعِ پاداشِ دریافتی از حالتِ s به بعد در زُامین کوشش است. الگوریتمِ حاصل، REINFORCE نامیده می‌شود، و متعلق به ران ویلیامز (1992) است؛ معمولاً بسیار مؤثرتر از بالارفتن از تپه با کوشش‌های بسیار در هر مقدارِ θ است. با این حال، همچنان بسیار کندتر از حدِ لازم است.

تکلیفِ زیر را در نظر بگیرید: با توجه به دو سیاستِ بلک‌جک، تعیین کنید کدام بهترین است. سیاست‌ها ممکن است بازدهِ خالصِ واقعیِ به‌ازایِ هر دستِ، مثلاً، 0.21%- و 0.06%+ داشته باشند، پس فهمیدن اینکه کدام بهتر است بسیار مهم است. یک راه برای انجامِ این کار این است که هر سیاست برای تعدادِ مشخصی دست در برابرِ «دلال»ِ استاندارد بازی کند و سپس بردهایشان را اندازه بگیرد. مشکلِ این کار، چنان‌که دیده‌ایم، این است که بردهای هر سیاست بسته به اینکه کارت‌های خوب یا بد می‌گیرد به‌شدت نوسان می‌کند. برای نشانهٔ قابلِ‌اعتمادی از اینکه کدام سیاست بهتر است به چند میلیون دست نیاز بود. همان مسئله هنگامِ استفاده از نمونه‌برداریِ تصادفی برای مقایسهٔ دو سیاستِ مجاور در الگوریتمِ بالارفتن از تپه پیش می‌آید.

راهِ حلِ بهتر برای بلک‌جک تولیدِ تعدادِ مشخصی دست از پیش و بازی‌کردنِ هر برنامه با همان مجموعهٔ دست‌هاست. به این‌شیوه، خطایِ اندازه‌گیریِ ناشی از تفاوتِ کارت‌های دریافتی را حذف می‌کنیم. فقط چند هزار دست برای تعیینِ اینکه کدام یک از دو سیاستِ بلک‌جک بهتر است لازم است.

این ایده، «نمونه‌برداریِ همبسته» (Correlated Sampling) نامیده می‌شود، و با توجه به شبیه‌سازِ محیطی که دنباله‌های عددِ تصادفی در آن قابلِ تکرارند، می‌توان آن را به‌طورِ کلی بر جستجوی سیاست اعمال کرد. در الگوریتمِ جستجوی سیاستِ PEGASUS (انگ و جردن، 2000) پیاده‌سازی شد، که یکی از نخستین الگوریتم‌هایی بود که به پروازِ کاملاً پایدارِ هلیکوپترِ خودمختار دست یافت (شکل ۲۲.۹ (ب) را ببینید). می‌توان نشان داد تعدادِ دنباله‌های تصادفیِ لازم برای اطمینان از اینکه مقدارِ هر سیاست به‌خوبی برآورد شده فقط به پیچیدگیِ فضایِ سیاست بستگی دارد، و اصلاً به پیچیدگیِ حوزهٔ زیرین نه.

۲۲.۶ کارآموزی و یادگیریِ تقویتیِ معکوس

برخی حوزه‌ها چنان پیچیده‌اند که تعریفِ تابعِ پاداش برای استفاده در یادگیریِ تقویتی دشوار است. دقیقاً از خودرویِ خودرانِ خود چه می‌خواهیم؟ قطعاً نباید برای رسیدن به مقصد بیش‌ازحد وقت بگیرد، اما نباید آن‌قدر سریع براند که خطرِ نابه‌جا یا جریمهٔ سرعت بگیرد. باید سوخت/انرژی حفظ کند. باید از تکان‌دادنِ یا شتابِ بیش‌ازحدِ مسافران اجتناب کند، اما می‌تواند در اورژانس ترمزِ محکم بزند. و مانندِ آن. تصمیم‌گیری دربارهٔ اینکه به هر یک از این عوامل چه وزنی بدهیم تکلیفِ دشواری است. بدتر از آن، تقریباً به‌طورِ قطع عواملِ مهمی را فراموش کرده‌ایم، مانندِ تعهد به رفتارِ با ملاحظه نسبت به رانندگانِ دیگر. حذفِ یک عامل معمولاً به رفتاری می‌انجامد که به عاملِ حذف‌شده مقدارِ افراطی تخصیص می‌دهد—در این مورد، رانندگیِ فوق‌العاده بی‌ملاحظه—برای بیشینه‌کردنِ عواملِ باقی‌مانده.

یک رویکرد انجامِ آزمایشِ گسترده در شبیه‌سازی، مشاهدهٔ رفتارهای مشکل‌ساز، و تلاش برای اصلاحِ تابعِ پاداش برای حذفِ آن رفتارهاست. رویکردِ دیگر جستجوی منابعِ اضافیِ اطلاعات دربارهٔ تابعِ پاداشِ مناسب است. یکی از این منابع رفتارِ عامل‌هایی است که از قبل آن تابعِ پاداش را بهینه می‌کنند (یا، بگذارید بگوییم، تقریباً بهینه می‌کنند)—در این مورد، رانندگانِ خبرهٔ انسانی.

حوزهٔ عمومیِ «یادگیریِ کارآموزی» (Apprenticeship Learning) فرایندِ یادگیریِ خوب‌رفتارکردن با توجه به مشاهده‌های رفتارِ خبره را مطالعه می‌کند. مثال‌هایی از رانندگیِ خبره به الگوریتم نشان می‌دهیم و به آن می‌گوییم «مثلِ آن انجامش بده». دستِ‌کم دو راه برای پرداختن به مسئلهٔ یادگیریِ کارآموزی هست. نخستین همانی است که در آغازِ فصل به‌طورِ مختصر بحث کردیم: به فرضِ مشاهده‌پذیربودنِ محیط، یادگیریِ نظارت‌شده را بر جفت‌های حالت-کنشِ مشاهده‌شده اعمال می‌کنیم تا سیاستِ π(s) یاد بگیریم. این «یادگیریِ تقلیدی» (Imitation Learning) نامیده می‌شود. در رباتیک موفقیت‌هایی داشته (صفحهٔ ۹۶۶ را ببینید) اما از مشکلِ شکنندگی رنج می‌برد: حتی انحراف‌های کوچک از مجموعهٔ آموزشی به خطاهایی می‌انجامند که در طولِ زمان رشد می‌کنند و سرانجام به شکست. افزون بر این، یادگیریِ تقلیدی در بهترین حالت عملکردِ معلم را تکرار می‌کند، نه فراتر از آن. وقتی انسان‌ها با تقلید یاد می‌گیرند، گاهی از اصطلاحِ تحقیرآمیزِ «میمون‌بازی» برای توصیفِ آنچه می‌کنند استفاده می‌کنیم. (کاملاً محتمل است میمون‌ها نیز در میانِ خودشان از اصطلاحِ «انسان‌بازی» استفاده کنند، شاید حتی در معنایی تحقیرآمیزتر.) تلویح این است که یادگیرندهٔ تقلیدی نمی‌فهمد چرا باید کنشِ داده‌شده‌ای را انجام دهد.

رویکردِ دوم به یادگیریِ کارآموزی فهمیدنِ چرایی است: مشاهدهٔ کنش‌های خبره (و حالت‌های حاصل) و تلاش برای فهمیدنِ اینکه خبره چه تابعِ پاداشی را بیشینه می‌کند. سپس می‌توانیم سیاستِ بهینه‌ای را نسبت به آن تابعِ پاداش اشتقاق دهیم. انتظار می‌رود این رویکرد از مثال‌های نسبتاً کمی از رفتارِ خبره سیاست‌های مقاوم تولید کند؛ بالاخره، حوزهٔ یادگیریِ تقویتی بر این ایده استوار است که تابعِ پاداش، نه سیاست یا تابعِ ارزش، موجزترین، مقاوم‌ترین و قابلِ‌انتقال‌ترین تعریفِ تکلیف است. افزون بر این، اگر یادگیرنده برای زیربهینه‌بودنِ احتمالیِ خبره کسری‌های مناسبی قائل شود، ممکن است بتواند با بهینه‌کردنِ تقریبِ دقیقی از تابعِ پاداشِ واقعی بهتر از خبره عمل کند. این رویکرد را «یادگیریِ تقویتیِ معکوس» (Inverse Reinforcement Learning یا IRL) می‌نامیم: یادگیریِ پاداش‌ها با مشاهدهٔ سیاست، به‌جای یادگیریِ سیاست با مشاهدهٔ پاداش‌ها.

چگونه تابعِ پاداشِ خبره را، با توجه به کنش‌های خبره، بیابیم؟ با این فرض شروع کنیم که خبره عقلایی عمل می‌کرده. در آن صورت، به نظر می‌رسد باید به دنبال تابعِ پاداشِ R* باشیم که مجموعِ موردانتظارِ کلِ پاداشِ تخفیف‌خورده تحتِ سیاستِ خبره بالاتر از (یا دستِ‌کم همان) هر سیاستِ ممکنِ دیگر باشد.

متأسفانه، توابعِ پاداشِ بسیاری این قید را برآورده می‌کنند؛ یکی از آن‌ها R*(s,a,s') = 0 است، زیرا وقتی هیچ پاداشی نیست هر سیاستی عقلایی است.[7] مشکلِ دیگرِ این رویکرد این است که فرضِ خبرهٔ عقلایی غیرواقع‌بینانه است. برای نمونه، به این معناست که رباتی که لی سدول را در حالِ برداشتنِ حرکتی که در نهایت معلوم می‌شود حرکتی باخت‌آور در برابرِ ALPHAGO است مشاهده می‌کند باید فرض کند لی سدول می‌خواسته بازی را ببازد.

برای اجتناب از مشکلِ اینکه R*(s,a,s') = 0 هر رفتارِ مشاهده‌شده‌ای را تبیین می‌کند، کمک می‌کند به‌شیوهٔ بیزی بیندیشیم. (برای یادآوریِ معنایِ این، بخش ۲۰.۱ را ببینید.) فرض کنید دادهٔ d را مشاهده می‌کنیم و h_R فرضیهٔ این باشد که R تابعِ پاداشِ واقعی است. آنگاه طبقِ قاعدهٔ بیز داریم:

```
P(h_R|d) = α P(d|h_R) P(h_R)
```

طبقِ معادلهٔ (17.9) در صفحهٔ ۵۶۹، تابعِ پاداشِ R'(s,a,s') = R(s,a,s') + γΦ(s') - Φ(s) دقیقاً همان سیاست‌های بهینهٔ R(s,a,s') را دارد، پس می‌توانیم تابعِ پاداش را فقط تا حدِ افزودنِ احتمالیِ هر تابعِ شکل‌دهیِ Φ(s) بازسازی کنیم. این مشکلِ چندان جدی نیست، زیرا رباتی که از R' استفاده می‌کند دقیقاً مانندِ رباتی که از R «درست» استفاده می‌کند رفتار خواهد کرد.

حالا، اگر پیشینِ P(h_R) بر سادگی مبتنی باشد، فرضیهٔ R = 0 امتیازِ نسبتاً خوبی می‌گیرد، زیرا 0 قطعاً ساده است. از سوی دیگر، عبارتِ P(d|h_R) برای فرضیهٔ R = 0 بی‌نهایتِ کوچک است، زیرا تبیین نمی‌کند چرا خبره آن رفتارِ خاص را از میانِ فضایِ عظیمِ رفتارهایی که اگر فرضیه درست بود بهینه می‌بودند انتخاب کرده. از سوی دیگر، برای تابعِ پاداشِ R که سیاستِ بهینهٔ یکتایی یا ردهٔ هم‌ارزیِ نسبتاً کوچکی از سیاست‌های بهینه دارد، P(d|h_R) بسیار بالاتر خواهد بود.

برای اجازه‌دادن به اشتباهِ گاه‌به‌گاهِ خبره، کافی است به P(d|h_R) اجازه دهیم حتی وقتی d از رفتاری می‌آید که طبقِ R کمی زیربهینه است ناصفر باشد. فرضِ معمول—که، باید گفت، بیشتر برای آسودگیِ ریاضی ساخته شده تا وفاداری به داده‌های واقعیِ انسانی—این است که عاملی که تابعِ Q واقعی‌اش Q(s,a) است نه طبقِ سیاستِ قطعیِ π(s) = argmax_a Q(s,a)، بلکه طبقِ سیاستِ تصادفیِ تعریف‌شده توسط توزیعِ سافتمکس از معادلهٔ (22.14) انتخاب می‌کند. این گاهی «عقلانیتِ بولتزمنی» (Boltzmann Rationality) نامیده می‌شود، زیرا در مکانیکِ آماری، احتمال‌های اشغالِ حالت در توزیعِ بولتزمن به‌صورتِ نمایی به سطح‌های انرژی‌شان بستگی دارد.

ده‌ها الگوریتمِ یادگیریِ تقویتیِ معکوس در ادبیات هست. یکی از ساده‌ترین‌ها «تطبیقِ ویژگی» (Feature Matching) نامیده می‌شود. فرض می‌کند تابعِ پاداش را می‌توان به‌صورتِ ترکیبِ خطیِ وزن‌داری از ویژگی‌ها نوشت:

```
R_θ(s,a,s′) = ∑_{i=1}^{n} θᵢ fᵢ(s,a,s′) = θ · f
```

برای نمونه، ویژگی‌های حوزهٔ رانندگی ممکن است شاملِ سرعت، سرعتِ بیش از حدِ مجاز، شتاب، نزدیکی به نزدیک‌ترین مانع و مانندِ آن باشند.

از معادلهٔ (17.2) در صفحهٔ ۵۶۷ به یاد آورید که مطلوبیتِ اجرایِ سیاستِ π، با شروع از حالتِ s_0، به‌صورتِ زیر تعریف می‌شود:

```
U^π(s) = E[∑_{t=0}^{∞} γᵗ R(Sₜ, π(Sₜ), Sₜ₊₁)]
```

که در آن امیدِ ریاضیِ E نسبت به توزیعِ احتمالیِ رویِ دنباله‌های حالتِ تعیین‌شده توسط s و π است. چون فرض می‌شود R ترکیبِ خطیِ مقادیرِ ویژگی است، می‌توانیم این را به‌صورتِ زیر بازنویسی کنیم:

```
U^π(s) = E[∑_{t=0}^{∞} γ^t ∑_{i=1}^{n} θ_i f_i(S_t, π(S_t), S_{t+1})]
θ_i E[∑_{t=0}^{∞} γ^t f_i(S_t, π(S_t), S_{t+1})] {n}^{i=1}_∑ =
θ_i μ_i(π) = θ · μ(π) {n}^{i=1}_∑ =
```

که در آن «امیدِ ویژگیِ» (Feature Expectation) $\mu_i(\pi)$ را به‌عنوانِ مقدارِ موردانتظارِ تخفیف‌خوردهٔ ویژگیِ f_i وقتی سیاستِ π اجرا می‌شود تعریف کرده‌ایم. برای نمونه، اگر f_i سرعتِ مازادِ خودرو (بالاتر از حدِ سرعت) باشد، $\mu_i(\pi)$ میانگینِ سرعتِ مازادِ (تخفیف‌زمانی) در کلِ مسیر است. نکتهٔ کلیدی دربارهٔ امیدهای ویژگی این است: اگر سیاستِ π امیدهای ویژگیِ $\mu_i(\pi)$ای تولید کند که با امیدهای ویژگیِ سیاستِ خبرهٔ π_E منطبق شود، آنگاه π طبقِ تابعِ پاداشِ خودِ خبره به همان خوبیِ سیاستِ خبره است. حالا، نمی‌توانیم مقادیرِ دقیقِ امیدهای ویژگیِ سیاستِ خبره را اندازه بگیریم، اما می‌توانیم آن‌ها را با مقادیرِ میانگینِ رویِ مسیرهای مشاهده‌شده تقریب بزنیم. پس، باید مقادیری برای پارامترهای θ_i بیابیم که امیدهای ویژگیِ سیاستِ القاشده توسط مقادیرِ پارامتر با امیدهای ویژگیِ سیاستِ خبره رویِ مسیرهای مشاهده‌شده منطبق شوند. الگوریتمِ زیر این را با هر کرانِ خطایِ دلخواهی محقق می‌کند.

- سیاستِ پیش‌فرضِ اولیهٔ $\pi^{(0)}$ را انتخاب کن.
- برای $j = 1, 2, \ldots$ تا همگرایی:
  - پارامترهای $\theta^{(j)}$ را بیاب که سیاستِ خبره به‌طورِ حداکثری از سیاست‌های $\pi^{(0)}, \ldots, \pi^{(j-1)}$ بر اساسِ مطلوبیتِ موردانتظارِ $\theta^{(j)} \cdot \mu(\pi)$ بهتر عمل می‌کند.
  - $\pi^{(j)}$ سیاستِ بهینه برای تابعِ پاداشِ $R^{(j)} = \theta^{(j)} \cdot f$ باشد.

این الگوریتم به سیاستی همگرا می‌شود که از نظرِ ارزش به سیاستِ خبره، طبقِ تابعِ پاداشِ خودِ خبره، نزدیک است. فقط به $O(n \log n)$ تکرار و $O(n \log n)$ نمایشِ خبره نیاز دارد، که n تعدادِ ویژگی‌هاست.

ربات می‌تواند از یادگیریِ تقویتیِ معکوس برای یادگیریِ سیاستِ خوبِ خودش، با فهمیدنِ کنش‌های یک خبره، استفاده کند. علاوه بر این، ربات می‌تواند سیاست‌های استفاده‌شده توسط عامل‌های دیگر در حوزهٔ چندعاملی را یاد بگیرد، خواه تخاصمی باشند خواه همکارانه. و سرانجام، یادگیریِ تقویتیِ معکوس را می‌توان برای کاوشِ علمی (بدونِ هیچ فکرِ طراحیِ عاملی) استفاده کرد، برای فهمِ بهترِ رفتارِ انسان‌ها و دیگر حیوانات.

فرضِ کلیدی در یادگیریِ تقویتیِ معکوس این است که «خبره» نسبت به تابعِ پاداشی در MDP تک‌عاملی به‌صورتِ بهینه، یا تقریباً بهینه، رفتار می‌کند. اگر یادگیرنده از پشتِ آینهٔ یک‌طرفه خبره را تماشا کند در حالی که خبره بی‌خبر به کارش مشغول است، این فرضِ معقولی است. اگر خبره از یادگیرنده آگاه باشد، فرضِ معقولی نیست. برای نمونه، فرض کنید رباتی در دانشکدهٔ پزشکی است و با تماشای خبرهٔ انسانی جراح شدن یاد می‌گیرد. الگوریتمِ IRL فرض می‌کرد انسان جراحی را به‌شیوهٔ معمولِ بهینه انجام می‌دهد، انگار ربات آن‌جا نیست. اما این چیزی نیست که رخ دهد: جراحِ انسانی انگیزه دارد ربات (مانندِ هر دانشجوی پزشکیِ دیگری) سریع و خوب یاد بگیرد، و بنابراین رفتارش را به‌طورِ چشمگیری اصلاح می‌کند. ممکن است در حینِ کار توضیح دهد چه می‌کند؛ ممکن است به اشتباهاتی که باید اجتناب

کرد اشاره کند، مانندِ اینکه برش را بیش‌ازحد عمیق یا بخیه‌ها را بیش‌ازحد سفت بزند؛ ممکن است برنامه‌های اضطراری را در صورتِ خراب‌شدنِ چیزی در طولِ جراحی توصیف کند. هیچ‌کدام از این رفتارها وقتی جراحی به‌صورتِ منزوی انجام شود معنا ندارد، پس الگوریتم‌های IRL نمی‌توانند تابعِ پاداشِ زیرین را تفسیر کنند. در عوض، باید این‌گونه موقعیت را به‌عنوانِ بازیِ کمکِ دومنفره بفهمیم، چنان‌که در بخش ۱۸.۲.۵ توصیف شد.

۲۲.۷ کاربردهای یادگیریِ تقویتی

حالا به کاربردهای یادگیریِ تقویتی می‌پردازیم. این‌ها شاملِ بازی‌کردن می‌شود، که مدلِ انتقال در آن شناخته‌شده و هدف یادگیریِ تابعِ مطلوبیت است، و رباتیک، که مدل در آن در آغاز ناشناخته است.

۲۲.۷.۱ کاربردها در بازی‌کردن

در فصل ۱ کارِ اولیهٔ آرتور ساموئل را رویِ یادگیریِ تقویتی برای بازیِ چکرز توصیف کردیم، که در ۱۹۵۲ آغاز شد. چند دهه گذشت تا چالش دوباره برداشته شد، این بار توسط جری تسائورو در کارش رویِ تخته‌نرد. نخستین تلاشِ تسائورو (1990) سامانه‌ای به نامِ NEUROGAMMON بود. رویکرد، تغییری جالب از یادگیریِ تقلیدی بود. ورودی مجموعه‌ای از ۴۰۰ بازی بود که تسائورو با خودش بازی کرده بود. به‌جای یادگیریِ سیاست، NEUROGAMMON هر حرکتِ ('s,a,s) را به مجموعه‌ای از مثال‌های آموزشی تبدیل می‌کرد، که هر یک 's را به‌عنوانِ موقعیتِ بهتر از موقعیتِ دیگری مانندِ "s که با حرکتی متفاوت از s قابلِ دست‌یابی است برچسب می‌زد. شبکه دو نیمهٔ جداگانه داشت، یکی برای 's و یکی برای "s، و مقید بود با مقایسهٔ خروجی‌های دو نیمه انتخاب کند کدام بهتر است. به این‌شیوه، هر نیمه مجبور بود تابعِ ارزیابیِ Û_θ یاد بگیرد. NEUROGAMMON المپیادِ رایانه‌ایِ ۱۹۸۹ را برد—نخستین برنامهٔ یادگیرنده‌ای که تاکنون مسابقاتِ بازیِ رایانه‌ای را برده—اما هرگز از سطحِ بازیِ میانیِ خودِ تسائورو فراتر نرفت.

سامانهٔ بعدیِ تسائورو، TD-GAMMON (1992)، روشِ تازه‌انتشارِ یادگیریِ TD ساتن را برگزید—اساساً بازگشت به رویکردِ کاوش‌شده توسط ساموئل، اما با درکِ فنیِ بسیار بیشتری از اینکه چگونه آن را درست انجام داد. تابعِ ارزیابیِ Û_θ با شبکهٔ عصبیِ کاملاً متصلی با یک لایهٔ پنهانِ ۸۰ گره‌ای بازنمایی می‌شد. (همچنین از برخی ویژگی‌های ورودیِ دستیِ طراحی‌شده استفاده می‌کرد که از NEUROGAMMON به عاریت گرفته شده بودند.) پس از ۳۰۰,۰۰۰ بازیِ آموزشی، به سطحِ بازیِ قابلِ مقایسه با سه بازیکنِ برترِ جهان رسید. کیت وولسی، بازیکنِ ده‌تایِ برتر، گفت: «در ذهنِ من هیچ شکی نیست که قضاوتِ موضعیِ آن بسیار بهتر از من است.»

چالشِ بعدی یادگیری از ورودی‌های ادراکیِ خام—چیزی نزدیک‌تر به جهانِ واقعی—به‌جای بازنمایی‌های صفحهٔ بازیِ گسسته بود. از ۲۰۱۲، تیمی در دیپ‌مایند سامانهٔ «شبکهٔ Q عمیق» (Deep Q-Network یا DQN) را توسعه داد، نخستین سامانهٔ مدرنِ یادگیریِ تقویتیِ عمیق. DQN از شبکهٔ عصبیِ عمیق برای بازنماییِ تابعِ Q استفاده می‌کند؛ در غیرِ این صورت سامانهٔ یادگیریِ تقویتیِ معمولی است. DQN به‌طورِ جداگانه رویِ هر یک از ۴۹ بازیِ ویدئوییِ متفاوتِ آتاری آموزش دید. یاد گرفت خودروهای مسابقه‌ایِ شبیه‌سازی‌شده براند، به سفینه‌های فضاییِ بیگانه شلیک کند و توپ‌ها را با راکت بزند. در هر مورد، عامل تابعِ Q را از دادهٔ تصویرِ خام با سیگنالِ پاداشِ امتیازِ بازی یاد می‌گرفت. در مجموع، سامانه تقریباً در سطحِ خبرهٔ انسانی عمل می‌کرد، هرچند چند بازی برایش مشکل‌ساز بود. یکی از بازی‌ها به‌طورِ

خاص، انتقامِ مونتزوما، بسیار دشوار ثابت شد، زیرا به راهبردهای برنامه‌ریزیِ گسترده نیاز داشت و پاداش‌ها بیش‌ازحد پراکنده بودند. کارهای بعدی سامانه‌های یادگیریِ تقویتیِ عمیقی تولید کرد که رفتارهای کاوشیِ گسترده‌تری تولید می‌کردند و توانستند انتقامِ مونتزوما و دیگر بازی‌های دشوار را فتح کنند.

سامانهٔ ALPHAGO دیپ‌مایند نیز از یادگیریِ تقویتیِ عمیق برای شکستِ بهترین بازیکنانِ انسانی در بازیِ گو استفاده کرد (فصل ۵ را ببینید). در حالی که تابعِ Q بدونِ نگاه-به-جلو برای بازی‌های آتاری، که در ذات عمدتاً واکنشی‌اند، کافی است، گو به نگاه-به-جلوی قابلِ توجهی نیاز دارد. به همین دلیل، ALPHAGO هم تابعِ ارزش و هم تابعِ Q را یاد گرفت که جستجویش را با پیش‌بینیِ اینکه کدام حرکت‌ها ارزشِ کاوش دارند هدایت می‌کرد. تابعِ Q، که به‌صورتِ شبکهٔ عصبیِ پیچشی پیاده‌سازی شده، به‌خودیِ‌خود به‌اندازهٔ کافی دقیق است که بدونِ هیچ جستجویی بیشترِ بازیکنانِ آماتورِ انسانی را بزند.

۲۲.۷.۲ کاربرد در کنترلِ ربات

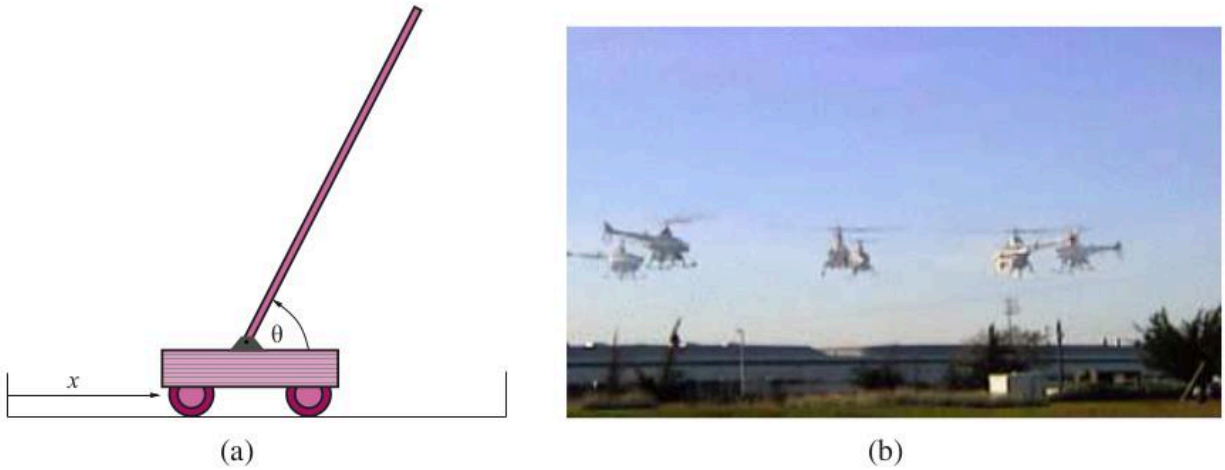
چیدمانِ مسئلهٔ مشهورِ تعادلِ گاری-میله، که به «آونگِ معکوس» (Inverted Pendulum) نیز معروف است، در شکل ۲۲.۹ (الف) نشان داده شده. مسئله این است که میله را تقریباً عمودی نگه داریم ($\theta \approx 90°$) با اعمالِ نیرو برای حرکتِ گاری به راست یا چپ، در حالی که موقعیتِ x را درونِ محدوده‌های مسیر نگه می‌داریم. چند هزار مقاله در یادگیریِ تقویتی و نظریهٔ کنترل دربارهٔ این مسئلهٔ ظاهراً ساده منتشر شده. یکی از دشواری‌ها این است که متغیرهای حالتِ x، $\dot{x}$، θ و $\dot{\theta}$ پیوسته‌اند. اما کنش‌ها گسسته تعریف شده‌اند: تکان به چپ یا تکان به راست، رژیمِ به‌اصطلاحِ «کنترلِ انفجاری» (Bang-Bang Control).

نخستین کارِ یادگیری برای این مسئله توسط میشی و چمبرز (1968) انجام شد، با گاری و میلهٔ واقعی، نه شبیه‌سازی. الگوریتمِ BOXES آن‌ها توانست پس از ۳۰ کوشش میله را بیش از یک ساعت متعادل نگه دارد. الگوریتم نخست فضایِ حالتِ چهاربعدی را به جعبه‌هایی گسسته کرد—از این رو نامِ آن. سپس کوشش‌ها را تا افتادنِ میله اجرا کرد. تقویتِ منفی با کنشِ نهاییِ آخرینِ جعبه مرتبط و سپس در طولِ دنباله به عقب منتشر می‌شد. تعمیمِ بهبودیافته و یادگیریِ سریع‌تر را می‌توان با الگوریتمی به‌دست آورد که فضایِ حالت را به‌صورتِ تطبیقی طبقِ تغییرِ مشاهده‌شده در پاداش افراز می‌کند، یا با استفاده از تقریب‌گرِ تابعِ غیرخطیِ حالت-پیوسته مانندِ شبکهٔ عصبی. امروزه، متعادل‌کردنِ آونگِ معکوسِ سه‌گانه (سه میله که سرتاسر به هم وصل‌اند) تمرینِ رایجی است—کاری که بسیار فراتر از توانایی‌های بیشترِ انسان‌هاست، اما با یادگیریِ تقویتی دست‌یافتنی است.

حتی چشمگیرتر کاربردِ یادگیریِ تقویتی در پروازِ هلیکوپترِ کنترلِ رادیویی است (شکل ۲۲.۹ (ب)). این کار به‌طورِ کلی از جستجوی سیاست رویِ MDPهای بزرگ استفاده کرده (باگنل و اشنایدر، 2001؛ نگ و همکاران، 2003)، که اغلب با یادگیریِ تقلیدی و IRL با توجه به مشاهده‌های خلبانِ خبرهٔ انسانی ترکیب می‌شود (کوتس و همکاران، 2009).

IRL همچنین با موفقیت برای تفسیرِ رفتارِ انسانی به‌کار رفته، از جملهِ پیش‌بینیِ مقصد و انتخابِ مسیر توسط رانندگانِ تاکسی بر مبنای ۱۰۰,۰۰۰ مایل دادهٔ GPS (زیبارت و همکاران، 2008) و حرکت‌های فیزیکیِ دقیقِ عابرانِ پیاده در محیط‌های پیچیده بر مبنای ساعت‌ها مشاهدهٔ ویدئویی (کیتانی و همکاران، 2012). در حوزهٔ رباتیک، یک نمایشِ خبرهٔ

تنها برای چهارپایِ LittleDog کافی بود تا تابعِ پاداشِ ۲۵-ویژگی یاد بگیرد و به‌صورتِ چابک از منطقهٔ نادیدهٔ قبلیِ زمینِ صخره‌ای عبور کند (کولتر و همکاران، 2008). برای اطلاعاتِ بیشتر دربارهٔ اینکه RL و IRL چگونه در رباتیک استفاده می‌شوند، بخش‌های ۲۶.۷ و ۲۶.۸ را ببینید.



شکل ۲۲.۹ — (الف) چیدمانِ مسئلهٔ متعادل‌کردنِ میله‌ای بلند رویِ گاریِ متحرک. گاری را می‌توان با کنترل‌گری که موقعیتِ گاریِ x و سرعتِ $\dot{x}$ و همچنین زاویهٔ میلهٔ θ و نرخِ تغییرِ زاویهٔ $\dot{\theta}$ را مشاهده می‌کند به چپ یا راست تکان داد. (ب) شش تصویرِ با نوردهیِ متوالی از یک هلیکوپترِ خودمختار در حالِ انجامِ مانورِ بسیار دشوارِ «دایرهٔ رو-به-داخل». هلیکوپتر تحتِ کنترلِ سیاستی است که توسط الگوریتمِ جستجوی سیاستِ PEGASUS توسعه یافته (نگ و همکاران، 2003). مدلِ شبیه‌سازی با مشاهدهٔ اثرهای دست‌کاری‌های کنترلِ گوناگون بر هلیکوپترِ واقعی توسعه یافت؛ سپس الگوریتم یک‌شبه رویِ مدلِ شبیه‌سازی اجرا شد. کنترل‌گرهای گوناگونی برای مانورهای متفاوت توسعه یافتند. در همهٔ موارد، عملکرد بسیار فراتر از خلبانِ خبرهٔ انسانی با استفاده از کنترلِ از راه‌دور بود. (تصویر با اجازه از اندرو نگ.)

خلاصهٔ فصل

این فصل مسئلهٔ یادگیریِ تقویتی را بررسی کرد: چگونه عامل می‌تواند در محیطِ ناشناخته‌ای مهارت پیدا کند، با توجه به فقط ادراک‌ها و پاداش‌های گاه‌به‌گاهش. یادگیریِ تقویتی پارادایمِ بسیار گسترده‌ای برای ایجادِ سامانه‌های هوشمند است. نکته‌های عمدهٔ فصل چنین‌اند.

- طراحیِ کلیِ عامل نوعِ اطلاعاتی را دیکته می‌کند که باید یاد گرفته شود:
  - عاملِ یادگیریِ تقویتیِ مبتنی بر مدل، مدلِ انتقالِ $P(s'|s,a)$ را برای محیط کسب می‌کند (یا مجهز به آن است) و تابعِ مطلوبیتِ $U(s)$ را یاد می‌گیرد.
  - عاملِ یادگیریِ تقویتیِ بدونِ مدل ممکن است تابعِ کنش-مطلوبیتِ $Q(s,a)$ یا سیاستِ $\pi(s)$ یاد بگیرد.
- مطلوبیت‌ها را می‌توان با چند رویکردِ متفاوت یاد گرفت:
  - برآوردِ مستقیمِ مطلوبیت از مجموعِ پاداشِ تا-پایانِ مشاهده‌شده برای حالتِ داده‌شده به‌عنوانِ شاهدِ مستقیمِ یادگیریِ مطلوبیتِ آن استفاده می‌کند.
  - برنامه‌نویسیِ پویایِ تطبیقی (ADP) مدلی و تابعِ پاداشی از مشاهده‌ها یاد می‌گیرد و سپس از تکرارِ ارزش یا تکرارِ سیاست برای به‌دست‌آوردنِ مطلوبیت‌ها یا سیاستِ بهینه استفاده می‌کند. ADP از قیدهای محلیِ اعمال‌شده بر مطلوبیتِ حالت‌ها از طریقِ ساختارِ همسایگیِ محیط استفادهٔ بهینه می‌کند.
  - روش‌های تفاضلِ زمانی (TD) برآوردهای مطلوبیت را طوری تنظیم می‌کنند که با برآوردهای حالت‌های جانشین سازگارتر شوند. می‌توان آن‌ها را تقریب‌های سادهٔ رویکردِ ADP دید که می‌توانند بدونِ نیاز به مدلِ انتقال یاد بگیرند. با این حال، استفاده از مدلِ آموخته‌شده برای تولیدِ شبه‌تجربه‌ها می‌تواند به یادگیریِ سریع‌تر بینجامد.
- توابعِ کنش-مطلوبیت، یا توابعِ Q، را می‌توان با رویکردِ ADP یا رویکردِ TD یاد گرفت. با TD، کیو-یادگیری در هیچ‌یک از فازهای یادگیری یا انتخابِ کنش به مدل نیاز ندارد. این مسئلهٔ یادگیری را ساده می‌کند اما بالقوه توانایی یادگیری در محیط‌های پیچیده را محدود می‌کند، زیرا عامل نمی‌تواند نتایجِ دوره‌های کنشِ ممکن را شبیه‌سازی کند.
- وقتی عاملِ یادگیرنده هنگامِ یادگیری مسئولِ انتخابِ کنش‌هاست، باید ارزشِ برآوردیِ آن کنش‌ها را با پتانسیلِ یادگیریِ اطلاعاتِ مفیدِ تازه مبادله کند. راه‌حلِ دقیقِ مسئلهٔ کاوش غیرممکن است، اما برخی اکتشافی‌های ساده کارِ معقولی انجام می‌دهند. عاملِ کاوشگر همچنین باید مراقبِ اجتناب از مرگِ زودهنگام باشد.
- در فضاهای حالتِ بزرگ، الگوریتم‌های یادگیریِ تقویتی باید از بازنماییِ تابعیِ تقریبیِ $U(s)$ یا $Q(s,a)$ برای تعمیم رویِ حالت‌ها استفاده کنند. یادگیریِ تقویتیِ عمیق—با استفاده از شبکه‌های عصبیِ عمیق به‌عنوانِ تقریب‌گرهای تابع—به موفقیتِ چشمگیری در مسائلِ دشوار دست یافته.
- شکل‌دهیِ پاداش و یادگیریِ تقویتیِ سلسله‌مراتبی برای یادگیریِ رفتارهای پیچیده کمک‌کننده‌اند، به‌ویژه وقتی پاداش‌ها پراکنده‌اند و برای به‌دست‌آوردنِ آن‌ها دنباله‌های کنشِ بلندی لازم است.

- روش‌های جستجوی سیاست مستقیماً روی بازنماییِ سیاست کار می‌کنند و می‌کوشند آن را بر اساسِ عملکردِ مشاهده‌شده بهبود دهند. تغییرِ عملکرد در حوزهٔ تصادفی مشکلِ جدی است؛ برای حوزه‌های شبیه‌سازی‌شده می‌توان با ثابت‌کردنِ تصادفی‌بودن از پیش بر آن غلبه کرد.
- یادگیریِ کارآموزی از طریقِ مشاهدهٔ رفتارِ خبره می‌تواند وقتی مشخص‌کردنِ تابعِ پاداشِ درست دشوار است راه‌حلِ مؤثری باشد. یادگیریِ تقلیدی مسئله را به‌صورتِ یادگیریِ نظارت‌شدهٔ سیاستی از جفت‌های حالت-کنشِ خبره صورتبندی می‌کند. یادگیریِ تقویتیِ معکوس اطلاعاتِ پاداش را از رفتارِ خبره استنتاج می‌کند.

یادداشت‌های کتابشناختی و تاریخی

به نظر می‌رسد ایدهٔ کلیدیِ یادگیریِ تقویتی—که جانوران بیشترِ آنچه را پاداش می‌گیرند انجام می‌دهند و کمترِ آنچه را تنبیه می‌شوند—نقشِ چشمگیری در اهلی‌کردنِ سگ‌ها حداقل ۱۵,۰۰۰ سال پیش ایفا کرده باشد. بنیان‌های اولیهٔ فهمِ علمیِ ما از یادگیریِ تقویتی شاملِ کارِ فیزیولوژیستِ روسی ایوان پاولف، که جایزهٔ نوبلِ ۱۹۰۴ را برد، و روان‌شناسِ آمریکایی ادوارد تورندایک—به‌ویژه کتابِ «هوشِ حیوانی» (1911) او—می‌شود. هیلگارد و باور (1975) مرورِ خوبی ارائه می‌دهند. آلن تورینگ (1948, 1950) یادگیریِ تقویتی را به‌عنوانِ رویکردی برای آموزشِ رایانه‌ها پیشنهاد کرد؛ آن را راه‌حلِ جزئی می‌دانست و نوشت: «استفاده از تنبیه‌ها و پاداش‌ها در بهترین حالت می‌تواند بخشی از فرایندِ آموزش باشد.» برنامهٔ چکرزِ آرتور ساموئل (1959, 1967) نخستین استفادهٔ موفق از یادگیریِ ماشینی از هر نوعی بود. ساموئل بیشترِ ایده‌های مدرنِ یادگیریِ تقویتی، از جمله یادگیریِ تفاضلِ زمانی و تقریبِ تابع را پیشنهاد کرد. او با بازنمایی‌های چندلایهٔ توابعِ ارزش آزمایش کرد، شبیهِ یادگیریِ تقویتیِ عمیقِ امروز. در پایان، دریافت تابعِ ارزیابیِ خطیِ ساده‌ای روی ویژگی‌های دست‌ساخته بهترین کارایی را دارد. این ممکن است پیامدِ کارکردن با رایانه‌ای بوده باشد که تقریباً ۱۰۰ میلیارد بار از واحدِ پردازشِ تانسوریِ مدرن ضعیف‌تر بود.

تقریباً در همان زمان، پژوهشگرانِ نظریهٔ کنترلِ تطبیقی (ویدرو و هاف، 1960)، با بناکردن بر کارِ هب (1949)، شبکه‌های ساده‌ای را با قاعدهٔ دلتا آموزش می‌دادند. پس، یادگیریِ تقویتی تأثیرهای روان‌شناسیِ حیوانی، عصب‌شناسی، تحقیق در عملیات و نظریهٔ کنترلِ بهینه را ترکیب می‌کند.

پیوندِ بینِ یادگیریِ تقویتی و فرایندهای تصمیمِ مارکوف نخستین بار توسط وربوس (1977) برقرار شد. (کارِ ایان ویتن (1977) فرایندی شبیهِ TD را به زبانِ نظریهٔ کنترل توصیف کرد.) توسعهٔ یادگیریِ تقویتی در هوش مصنوعی عمدتاً از کارِ دانشگاهِ ماساچوست در اوایلِ دههٔ ۱۹۸۰ سرچشمه می‌گیرد (بارتو و همکاران، 1981). مقالهٔ تأثیرگذارِ ریچ ساتن (1988) فهمِ ریاضیِ روش‌های تفاضلِ زمانی را فراهم کرد. ترکیبِ یادگیریِ تفاضلِ زمانی با تولیدِ مبتنی بر مدلِ تجربه‌های شبیه‌سازی‌شده در معماریِ DYNA ساتن پیشنهاد شد (ساتن، 1990). کیو-یادگیری در پایان‌نامهٔ دکترایِ کریس واتکینز (1989) توسعه یافت، در حالی که SARSA در گزارشِ فنیِ رامری و نیرانجان (1994) ظاهر شد. جاروبِ اولویت‌دار به‌طورِ مستقل توسط مور و آتکسون (1993) و پنگ و ویلیامز (1993) معرفی شد.

تقریبِ تابع در یادگیریِ تقویتی به برنامهٔ چکرزِ آرتور ساموئل (1959) بازمی‌گردد. استفاده از شبکه‌های عصبی برای بازنماییِ توابعِ ارزش در دههٔ ۱۹۸۰ رایج بود و در برنامهٔ TD-Gammon جری تسائورو به‌پیش آمد (تسائورو، 1992, 1995). شبکه‌های عصبیِ عمیق در حالِ حاضر محبوب‌ترین انتخاب برای تقریب‌گرهای تابع در یادگیریِ تقویتی‌اند. آرولکوماران و همکاران (2017) و فرانسوا-لاوه و همکاران (2018) نمایِ کلی از یادگیریِ تقویتیِ عمیق می‌دهند. سامانهٔ DQN (منیه و همکاران، 2015) از شبکهٔ عمیق برای یادگیریِ تابعِ Q استفاده می‌کند، در حالی که ALPHAZERO (سیلور و همکاران، 2018) هم تابعِ ارزشِ برای استفاده با مدلِ شناخته‌شده و هم تابعِ Q برای استفاده در تصمیم‌های فراسطحیِ هدایت‌کنندهٔ جستجو یاد می‌گیرد. ایرپن (2018) هشدار می‌دهد سامانه‌های یادگیریِ تقویتیِ عمیق اگر محیطِ واقعی حتی اندکی با محیطِ آموزشی تفاوت داشته باشد می‌توانند عملکردِ ضعیفی داشته باشند.

ترکیب‌های خطیِ وزن‌دارِ ویژگی‌ها و شبکه‌های عصبی بازنمایی‌های عامل‌بندی‌شده برای تقریبِ تابع‌اند. همچنین ممکن است یادگیریِ تقویتی را بر بازنمایی‌های ساخت‌یافته اعمال کرد؛ این «یادگیریِ تقویتیِ رابطه‌ای» (Relational Reinforcement Learning) نامیده می‌شود (تادپالی و همکاران، 2004). استفاده از توصیف‌های رابطه‌ای امکانِ تعمیم در میانِ رفتارهای پیچیدهٔ شاملِ اشیای متفاوت را فراهم می‌کند.

تحلیلِ ویژگی‌های همگراییِ الگوریتم‌های یادگیریِ تقویتی با استفاده از تقریبِ تابع موضوعِ فوق‌العاده فنی است. نتایجِ یادگیریِ TD به‌تدریج برای حالتِ تقریب‌گرهای تابعِ خطی تقویت شده‌اند (ساتن، 1988؛ دیان، 1992؛ تسیتسیکلیس و ون روی، 1997)، اما چند مثالِ واگرایی برای توابعِ غیرخطی ارائه شده (برای بحث، تسیتسیکلیس و ون روی، 1997 را ببینید). پاپاواسیلیو و راسل (1999) نوعی از یادگیریِ تقویتی را توصیف می‌کنند که با هر شکلِ تقریب‌گرِ تابعی همگرا می‌شود، به‌شرطِ اینکه مسئلهٔ برازشِ فرضیه به داده حل‌شدنی باشد. لیو و همکاران (2018) خانوادهٔ الگوریتم‌های TD گرادیانی را توصیف و تحلیلِ نظریِ گسترده‌ای از همگرایی و پیچیدگیِ نمونه ارائه می‌کنند.

انواعِ گوناگونی از روش‌های کاوش برای مسائلِ تصمیمِ ترتیبی توسط بارتو و همکاران (1995) بحث شده. کرنز و سینگ (1998) و برافمن و تننه‌هولتز (2000) الگوریتم‌هایی توصیف می‌کنند که محیط‌های ناشناخته را کاوش می‌کنند و تضمین می‌شود با پیچیدگیِ نمونه‌ای که در تعدادِ حالت‌ها چندجمله‌ای است به سیاست‌های تقریباً بهینه همگرا می‌شوند. یادگیریِ تقویتیِ بیزی (دیردن و همکاران، 1998, 1999) زاویهٔ دیگری بر هر دوِ عدم‌قطعیتِ مدل و کاوش فراهم می‌کند.

ایدهٔ زیربناییِ یادگیریِ تقلیدی اعمالِ یادگیریِ نظارت‌شده بر مجموعهٔ آموزشیِ کنش‌های خبره است. این ایده‌ای قدیمی در کنترلِ تطبیقی است، اما نخستین بار با کارِ ساموت و همکاران (1992) رویِ «یادگیریِ پرواز» در شبیه‌سازِ پرواز در هوش مصنوعی به‌پیش آمد. آن‌ها روششان را «همتاسازیِ رفتاری» (Behavioral Cloning) نامیدند. چند سال بعد، همان گروهِ پژوهشی گزارش کرد این روش بسیار شکننده‌تر از آن چیزی است که در آغاز گزارش شده بود (کاماچو و میشی، 1995): حتی اختلال‌های بسیار کوچک باعث می‌شد سیاستِ آموخته‌شده از مسیرِ مطلوب منحرف شود و به خطاهای ترکیب‌شونده‌ای بینجامد که عامل بیش‌ازحد از مجموعهٔ آموزشی دور می‌شد. (همچنین بحثِ صفحهٔ ۹۶۶ را ببینید.) کارِ یادگیریِ کارآموزی می‌کوشد رویکرد را مقاوم‌تر کند، تا حدی با گنجاندنِ اطلاعاتی دربارهٔ پیامدهای مطلوب به‌جای فقط سیاستِ خبره. نگ و همکاران (2003) و کوتس و همکاران (2009) نشان می‌دهند یادگیریِ کارآموزی چگونه برای یادگیریِ پروازِ هلیکوپترِ واقعی کار می‌کند، چنان‌که در شکل ۲۲.۹ (ب) در صفحهٔ ۸۱۷ نشان داده شده.

یادگیریِ تقویتیِ معکوس (IRL) توسط راسل (1998) معرفی شد، و نخستین الگوریتم‌ها توسط نگ و راسل (2000) توسعه یافتند. (مسئله‌ای مشابه در اقتصاد برای مدتِ بسیار طولانی‌تری، تحتِ عنوانِ برآوردِ ساختاریِ MDPها، مطالعه شده (سارجنت، 1978).) الگوریتمِ داده‌شده در این فصل متعلق به ابیل و نگ (2004) است. بیکر و همکاران (2009) توصیف می‌کنند چگونه فهمِ کنش‌های عاملِ دیگر را می‌توان برنامه‌ریزیِ معکوس دید. هو و همکاران (2017) نشان می‌دهند عامل‌ها می‌توانند از رفتارهایی که آموزنده‌اند بهتر از رفتارهای بهینه یاد بگیرند. هادفیلد-منل و همکاران (2017a) IRL را به صورتبندیِ نظریه‌بازی می‌گسترند که هم ناظر و هم نمایش‌دهنده را دربرمی‌گیرد، و نشان می‌دهند رفتارهای آموزش و یادگیری چگونه به‌عنوانِ راه‌حل‌های بازی پدید می‌آیند.

گارسیا و فرناندز (2015) مرورِ جامعی بر یادگیریِ تقویتیِ ایمن ارائه می‌دهند. مونوس و همکاران (2017) الگوریتمی برای کاوشِ ایمنِ خارج‌از-سیاست (مثلاً کیو-یادگیری) توصیف می‌کنند. هانس و همکاران (2008) مسئلهٔ کاوشِ ایمن را به دو بخش می‌شکنند: تعریفِ تابعِ ایمنی برای نشان‌دادنِ اینکه از کدام حالت‌ها باید اجتناب کرد، و تعریفِ سیاستِ پشتیبان برای بازگرداندنِ عامل به ایمنی وقتی در غیرِ این صورت ممکن است وارد حالتِ ناایمن شود. یو و همکاران (2017) نشان می‌دهند چگونه مدلِ یادگیریِ تقویتیِ عمیقی را برای رانندگیِ خودرو در شبیه‌سازی آموزش دهیم و سپس از یادگیریِ انتقالی برای رانندگیِ ایمن در جهانِ واقعی استفاده کنیم.

توماس و همکاران (2017) رویکردی به یادگیری ارائه می‌کنند که با احتمالِ بالا تضمین می‌شود بدتر از سیاستِ کنونی عمل نکند. آکامتالو و همکاران (2014) رویکردِ مبتنی بر دست‌یابی‌پذیری توصیف می‌کنند، که در آن فرایندِ یادگیری تحتِ هدایتِ سیاستِ کنترلی عمل می‌کند که تضمین می‌کند عامل هرگز به حالتِ ناایمن نمی‌رسد. ساندرز و همکاران (2018) نشان می‌دهند سامانه‌ای می‌تواند از دخالتِ انسانی برای جلوگیری از خروجش از ناحیهٔ ایمن استفاده کند و می‌تواند در طولِ زمان یاد بگیرد به دخالتِ کمتری نیاز داشته باشد.

روش‌های جستجوی سیاست توسط ویلیامز (1992) به‌پیش آمدند، که خانوادهٔ الگوریتم‌های REINFORCE را توسعه داد، که مخففِ «افزایشِ پاداش = عاملِ نامنفی × تقویتِ آفست × شایستگیِ مشخصه» است. کارِ بعدیِ مارباخ و تسیتسیکلیس (1998)، ساتن و همکاران (2000) و بکستر و بارتلت (2000) نتایجِ همگراییِ جستجوی سیاست را تقویت و تعمیم کرد. شولمن و همکاران (2015b) بهینه‌سازیِ سیاستِ ناحیهٔ اعتماد را توصیف می‌کنند، الگوریتمِ جستجوی سیاستِ با بنیانِ نظریِ خوب و در عین حال عملی که گونه‌های بسیاری زاییده. روشِ نمونه‌برداریِ همبسته برای کاهشِ واریانس در مقایسه‌های مونت‌کارلو متعلق به کان و مارشال (1953) است؛ همچنین یکی از تعدادی روشِ کاهشِ واریانس است که همرزلی و هندزکامب (1964) کاوش کرده‌اند.

رویکردهای اولیهٔ یادگیریِ تقویتیِ سلسله‌مراتبی (HRL) می‌کوشیدند سلسله‌مراتب‌ها را با انتزاعِ حالت بسازند—یعنی گروه‌بندیِ حالت‌ها به حالت‌های انتزاعی و سپس انجامِ RL در فضایِ حالتِ انتزاعی (دیان و هینتون، 1993). متأسفانه، مدلِ انتقالِ حالت‌های انتزاعی معمولاً غیرمارکوفی است و به رفتارِ واگرایِ الگوریتم‌های RL استاندارد می‌انجامد. رویکردِ انتزاعِ زمانیِ این فصل در اواخرِ دههٔ ۱۹۹۰ توسعه یافت (پار و راسل، 1998؛ آندره و راسل، 2002؛ ساتن و همکاران، 2000) و برای مدیریتِ رفتارهای هم‌زمان توسط مارتی و همکاران (2005) بسط یافت. دیتریش (2000) مفهومِ تجزیهٔ افزایشیِ توابعِ Q القاشده توسط سلسله‌مراتبِ زیرروال‌ها را معرفی کرد.

انتزاعِ زمانی بر نتیجهٔ بسیار قدیمی‌تری از فورستیه و وارایا (1978) مبتنی است، که نشان دادند MDP بزرگ را می‌توان به سامانهٔ دولایه‌ای تجزیه کرد که در آن لایهٔ نظارتی در میانِ کنترل‌گرهای سطح‌پایین انتخاب می‌کند، که هر یک در پایان کار کنترل را به ناظر بازمی‌گرداند. مسئلهٔ یادگیریِ خودِ سلسله‌مراتبِ انتزاع دستِ‌کم از کارِ پیتر آندره (1985) مطالعه شده؛ برای کاوشِ اخیر در یادگیریِ اولیه‌های حرکتِ ربات، فرانس و همکاران (2018) را ببینید. بازیِ کیپ‌اِوِی توسط استون و همکاران (2005) معرفی شد؛ راه‌حلِ HRL داده‌شده در این‌جا متعلق به بای و راسل (2017) است.

عصب‌شناسی اغلب یادگیریِ تقویتی را الهام بخشیده و ارزشِ رویکرد را تأیید کرده. پژوهشِ با استفاده از ثبتِ تک‌سلولی پیشنهاد می‌کند سامانهٔ دوپامین در مغزِ نخستی‌ها چیزی شبیهِ یادگیریِ تابعِ ارزش پیاده‌سازی می‌کند (شولتز و همکاران، 1997). متنِ عصب‌شناسیِ دیان و ابوت (2001) پیاده‌سازی‌های عصبیِ ممکنِ یادگیریِ تفاضلِ زمانی را توصیف می‌کند؛ پژوهشِ مرتبط آزمایش‌های عصب‌شناسی و رفتاریِ دیگری توصیف می‌کند (دیان و نیو، 2008؛ نیو، 2009؛ لی و همکاران، 2012).

کارِ یادگیریِ تقویتی با در دسترس‌بودنِ محیط‌های شبیه‌سازیِ متن‌باز برای توسعه و آزمونِ عامل‌های یادگیرنده شتاب گرفته. محیطِ یادگیریِ آرکیدِ دانشگاهِ آلبرتا (ALE) (بلمر و همکاران، 2013) چنین چارچوبی را برای ۵۵ بازیِ ویدئوییِ کلاسیکِ آتاری فراهم کرد. پیکسل‌های رویِ صفحه به‌عنوانِ ادراک به عامل داده می‌شوند، همراه با امتیازِ سیم‌بندی‌شدهٔ بازیِ تاکنون. ALE توسط تیمِ دیپ‌مایند برای پیاده‌سازیِ یادگیریِ DQN و راستی‌آزماییِ عمومیتِ سامانه‌شان رویِ طیفِ وسیعی از بازی‌ها استفاده شد (منیه و همکاران، 2015). دیپ‌مایند به‌نوبهٔ خود چند پلتفرمِ عامل را متن‌باز کرد، از جملهِ DeepMind Lab (بیتی و همکاران، 2016)، Gridworlds ایمنیِ هوش مصنوعی (لیکه و همکاران، 2017)، پلتفرمِ بازیِ Unity (جولیانی و همکاران، 2018) و مجموعهٔ کنترلِ DM (تاسا و همکاران، 2018). بلیزارد محیطِ یادگیریِ StarCraft II (SC2LE) را منتشر کرد، که دیپ‌مایند مؤلفهٔ PySC2 را برای یادگیریِ ماشینی در پایتون به آن افزود (وینیالز و همکاران، 2017a).

شبیه‌سازیِ AI Habitat فیسبوک (ساوا و همکاران، 2019) محیطِ مجازیِ واقع‌نمایِ عکسی برای تکلیف‌های رباتیکِ داخلی فراهم می‌کند، و پلتفرمِ HORIZON آن‌ها (گاوسی و همکاران، 2018) یادگیریِ تقویتی را در سامانه‌های تولیدِ مقیاس‌بزرگ ممکن می‌کند. سامانهٔ SYNTHIA (روس و همکاران، 2016) محیطِ شبیه‌سازی‌ای است که برای بهبودِ قابلیت‌های بینواییِ رایانه‌ایِ خودروهای خودران طراحی شده. OpenAI Gym (براکمن و همکاران، 2016) چند محیط برای عامل‌های یادگیریِ تقویتی فراهم می‌کند و با شبیه‌سازی‌های دیگر مانندِ شبیه‌سازِ فوتبالِ گوگل سازگار است.

لیتمن (2015) یادگیریِ تقویتی را برای مخاطبِ علمیِ عمومی مرور می‌کند. متنِ معیارِ ساتن و بارتو (2018)، دو تن از پیشگامانِ این حوزه، نشان می‌دهد یادگیریِ تقویتی چگونه ایده‌های یادگیری، برنامه‌ریزی و عمل را به هم می‌بافد. کوخندرفر (2015) رویکردِ کمی کمتر ریاضی دارد، با مثال‌های فراوانِ جهانِ واقعی. کتابِ کوتاهی از سپِشواری (2010) نمایِ کلی از الگوریتم‌های یادگیریِ تقویتی می‌دهد.

پانوشت‌ها:

[1] همان‌گونه که یان لکون و آلیوشا افرس اشاره کرده‌اند، «انقلابِ هوش مصنوعی نظارت‌شده نخواهد بود.»

[2] در ادبیاتِ RL، که بیش از اقتصاد از تحقیق در عملیات تغذیه می‌کند، توابعِ مطلوبیت اغلب «توابعِ ارزش» نامیده می‌شوند و با V(s) نشان داده می‌شوند.

[3] شرایطِ فنی در صفحهٔ ۶۸۴ داده شده. در شکل ۲۲.۵ از α(n) = 60/(59 + n) استفاده کرده‌ایم که شرایط را برآورده می‌کند.

[4] تعریفِ فاصلهٔ بینِ توابعِ مطلوبیت نسبتاً فنی است؛ تسیتسیکلیس و ون روی (1997) را ببینید.

[5] شایعه‌هایی که کیپ‌اِوِی از تاکتیک‌های جهانِ واقعیِ بارسلونایِ FC الهام گرفته احتمالاً بی‌بنیادند.

[6] چون ممکن است پیش از رسیدن به حالتِ انتخابِ بعدی بیش از یک کنشِ فیزیکی اجرا شود، مسئله از نظرِ فنی فرایندِ تصمیمِ نیمه‌مارکوف است، که به کنش‌ها اجازه می‌دهد طول‌های متفاوت، از جملهِ طول‌های تصادفی، داشته باشند. اگر عاملِ تخفیفِ γ 1 > باشد، طولِ کنش بر تخفیفِ اعمال‌شده بر پاداشِ به‌دست‌آمده در طولِ کنش اثر می‌گذارد، که به این معناست باید حسابداریِ تخفیفِ اضافی‌ای انجام شود و مدلِ انتقال شاملِ توزیعِ طول می‌شود.

[7] طبقِ معادلهٔ (17.9) در صفحهٔ ۵۶۹، تابعِ پاداشِ R'(s,a,s') = R(s,a,s') + γΦ(s') - Φ(s) دقیقاً همان سیاست‌های بهینهٔ R(s,a,s') را دارد، پس می‌توانیم تابعِ پاداش را فقط تا حدِ افزودنِ احتمالیِ هر تابعِ شکل‌دهیِ Φ(s) بازسازی کنیم.

فصل ۲۳: پردازش زبان طبیعی (Natural Language Processing)

در این فصل می‌بینیم چگونه رایانه می‌تواند از زبان طبیعی برای برقراری ارتباط با انسان‌ها و یادگیری از آنچه نوشته‌اند استفاده کند.

حدودِ ۱۰۰,۰۰۰ سال پیش، انسان‌ها یاد گرفتند چگونه صحبت کنند، و حدودِ ۵,۰۰۰ سال پیش یاد گرفتند بنویسند. پیچیدگی و تنوعِ زبانِ بشری انسانِ هوشمند (Homo sapiens) را از همهٔ گونه‌های دیگر متمایز می‌کند. البته ویژگی‌های دیگری نیز هستند که منحصراً انسانی‌اند: هیچ گونهٔ دیگری مانندِ انسان‌ها لباس نمی‌پوشد، هنر نمی‌آفریند، یا روزی دو ساعت در شبکه‌های اجتماعی وقت نمی‌گذراند. اما وقتی آلن تورینگ آزمونِ هوشِ خود را پیشنهاد کرد، آن را بر زبان بنا نهاد، نه هنر یا لباس‌فروشی، شاید به‌دلیلِ دامنهٔ همگانیِ آن و به‌دلیلِ اینکه زبان بخشِ بسیار زیادی از رفتارِ هوشمند را درمی‌گیرد: گوینده (یا نویسنده) هدفِ انتقالِ دانشی را دارد، سپس زبانی را برنامه‌ریزی می‌کند که آن دانش را بازنمایی می‌کند، و برای رسیدن به هدف کنش می‌کند. شنونده (یا خواننده) زبان را ادراک می‌کند و معنایِ موردنظر را استنتاج می‌کند.

این نوعِ ارتباط از طریقِ زبان به تمدن اجازه داده رشد کند؛ این وسیلهٔ اصلیِ ما برای انتقالِ دانشِ فرهنگی، حقوقی، علمی و فناورانه است. سه دلیلِ اصلی برای انجامِ «پردازش زبان طبیعی» (Natural Language Processing یا NLP) توسط رایانه‌ها هست:

- **ارتباط با انسان‌ها.** در بسیاری از موقعیت‌ها برای انسان‌ها راحت است از گفتار برای تعامل با رایانه‌ها استفاده کنند، و در بیشترِ موقعیت‌ها استفاده از زبانِ طبیعی راحت‌تر از زبانِ صوری مانندِ حسابِ گزاره‌ها/محمول‌های مرتبهٔ اول است.
- **یادگیری.** انسان‌ها دانشِ زیادی را با استفاده از زبانِ طبیعی نوشته‌اند. ویکی‌پدیا به‌تنهایی ۳۰ میلیون صفحه از واقعیت‌هایی مانندِ «گالاگوها نخستی‌سانانِ کوچکِ شب‌زی‌اند» دارد، در حالی که به‌ندرت منبعی از چنین واقعیت‌هایی که به منطقِ صوری نوشته شده باشد هست. اگر بخواهیم سامانه‌مان چیزهای زیادی بداند، بهتر است زبانِ طبیعی را بفهمد.
- **پیشبردِ فهمِ علمیِ زبان‌ها و کاربردِ زبان**، با استفاده از ابزارهای هوش مصنوعی در پیوند با زبان‌شناسی، روان‌شناسیِ شناختی و عصب‌شناسی.

در این فصل مدل‌های ریاضیِ گوناگونی برای زبان بررسی می‌کنیم و تکلیف‌هایی را بحث می‌کنیم که با آن‌ها قابلِ انجام‌اند.

۲۳.۱ مدل‌های زبان

زبان‌های صوری، مانندِ منطقِ مرتبهٔ اول، دقیقاً تعریف شده‌اند، چنان‌که در فصل ۸ دیدیم. دستورِ زبان (Grammar) نحوِ جمله‌های معتبر را تعریف می‌کند و قاعده‌های معنایی معنا را تعریف می‌کنند. زبان‌های طبیعی، مانندِ انگلیسی یا چینی، نمی‌توانند چنان مرتب مشخص شوند:

- قضاوت‌های زبانی از شخصی به شخصِ دیگر و از زمانی به زمانِ دیگر تغییر می‌کند. همه موافق‌اند «دعوت‌نشدن غم‌انگیز است» جملهٔ دستوریِ انگلیسی است، اما بر سرِ دستوری‌بودنِ «به‌نحوی دعوت‌نشده‌بودن غم‌انگیز است» اختلاف دارند.
- زبانِ طبیعی مبهم است («او اردکِ او را دید» می‌تواند یا به این معنا باشد که او پرندهٔ آبی دارد، یا اینکه او حرکتی انحرافیِ رو به پایین انجام داد) و مبهم‌مقدار است («عالی است!» مشخص نمی‌کند دقیقاً چقدر عالی است، نه این‌که چیست).
- نگاشت از نمادها به اشیا به‌صورتِ صوری تعریف نشده. در منطقِ مرتبهٔ اول، دو کاربرد از نمادِ «ریچارد» باید به همان شخص ارجاع کنند، اما در زبانِ طبیعی دو رخداد از همان واژه یا عبارت ممکن است به چیزهای متفاوتی در جهان ارجاع کنند.

اگر نتوانیم تمایزِ قطعیِ بولی بینِ رشته‌های دستوری و غیردستوری قائل شویم، دستِ‌کم می‌توانیم بگوییم هر یک چقدر محتمل یا نامحتمل است.

«مدلِ زبان» (Language Model) را توزیعِ احتمالی‌ای تعریف می‌کنیم که احتمالِ هر رشته را توصیف می‌کند. چنین مدلی باید بگوید «آیا جرئت می‌کنم جهان را برهم زنم؟» به‌عنوانِ رشته‌ای از انگلیسی احتمالِ معقولی دارد، اما «جهان را من برهم زنم جرئت آیا می‌کنم؟» فوق‌العاده نامحتمل است.

با مدلِ زبان، می‌توانیم پیش‌بینی کنیم چه واژه‌هایی محتمل‌اند در ادامهٔ یک متن بیایند، و از این راه تکمیل‌هایی برای ایمیل یا پیامِ متنی پیشنهاد کنیم. می‌توانیم محاسبه کنیم کدام تغییرها در یک متن آن را محتمل‌تر می‌کنند، و از این راه تصحیحِ املا یا دستور پیشنهاد کنیم. با یک جفتِ مدل، می‌توانیم محتمل‌ترین ترجمهٔ یک جمله را محاسبه کنیم. با چند مثالِ جفتِ پرسش/پاسخ به‌عنوانِ دادهٔ آموزشی، می‌توانیم محتمل‌ترین پاسخ به یک پرسش را محاسبه کنیم. پس مدل‌های زبان در قلبِ طیفِ وسیعی از تکلیف‌های زبانِ طبیعی‌اند.

خودِ تکلیفِ مدل‌سازیِ زبان نیز به‌عنوانِ معیارِ رایجی برای اندازه‌گیریِ پیشرفت در فهمِ زبان عمل می‌کند. زبان‌های طبیعی پیچیده‌اند، پس هر مدلِ زبانی در بهترین حالت تقریبی خواهد بود.

زبان‌شناسِ ادوارد ساپیر گفت «هیچ زبانی به‌طورِ مستبدانه سازگار نیست. همهٔ دستورها نشت می‌کنند» (ساپیر، 1921). فیلسوفِ دونالد دیویدسون گفت «چنین چیزی به‌نامِ زبان وجود ندارد، نه اگر زبان ... ساختارِ مشترکِ به‌وضوح تعریف‌شده‌ای باشد» (دیویدسون، 1986)، و منظورش این بود هیچ مدلِ زبانِ قطعیِ واحدی برای انگلیسی به

همان‌گونه که برای پایتون 3.8 هست وجود ندارد؛ همهٔ ما مدل‌های متفاوتی داریم، اما با این حال به‌نحوی موفق می‌شویم سر و تهِ قضیه را هم بیاوریم و ارتباط برقرار کنیم. در این بخش مدل‌های زبانِ ساده‌لوحانه‌ای را پوشش می‌دهیم که به‌وضوح نادرست‌اند، اما با این حال برای تکلیف‌های خاصی مفید می‌مانند.

۲۳.۱.۱ مدلِ کیسهٔ واژه‌ها

بخش ۱۲.۶.۱ توضیح داد چگونه مدلِ بیزِ سادهٔ مبتنی بر حضورِ واژه‌های خاص می‌توانست جمله‌ها را به‌طورِ قابلِ‌اعتمادی به رده‌ها رده‌بندی کند؛ برای نمونه، جملهٔ (۱) زیر به‌عنوانِ تجاری و (۲) به‌عنوانِ آب‌وهوا رده‌بندی می‌شود.

1. سهام دوشنبه جهش کرد و شاخص‌های اصلی با تداومِ خوش‌بینی دربارهٔ فصلِ درآمدِ سه‌ماههٔ نخست ۱٪ رشد کردند.
2. بارانِ سنگین دوشنبه بخشِ بزرگی از ساحلِ شرقی را کوبید، با هشدارِ سیل در شهرِ نیویورک و مکان‌های دیگر.

این بخش مدلِ بیزِ ساده را بازبینی می‌کند و آن را به‌عنوانِ مدلِ زبانِ کاملی قالب‌بندی می‌کند. یعنی فقط نمی‌خواهیم بدانیم کدام رده برای هر جمله محتمل‌ترین است؛ توزیعِ احتمالِ مشترکی بر همهٔ جمله‌ها و رده‌ها می‌خواهیم. این پیشنهاد می‌کند همهٔ واژه‌های جمله را در نظر بگیریم. با توجه به جمله‌ای شاملِ واژه‌های $w_1, w_2, \ldots, w_N$ (که به‌صورتِ $w_{1:n}$ می‌نویسیم، مانندِ فصل ۱۴)، فرمولِ بیزِ ساده (معادلهٔ (۱۲.۲۱)) به ما می‌دهد:

$$P(\text{Class} \mid w_{1:n}) = \alpha\, P(\text{Class}) \prod_j P(w_j \mid \text{Class})$$

کاربردِ بیزِ ساده بر رشته‌های واژه «مدلِ کیسهٔ واژه‌ها» (Bag-of-Words Model) نامیده می‌شود. این مدلِ مولدِ است که فرایندی برای تولیدِ جمله توصیف می‌کند: تصور کنید برای هر رده (تجاری، آب‌وهوا و مانندِ آن) کیسه‌ای پر از واژه داریم (می‌توانید تصور کنید هر واژه روی کاغذپاره‌ای درونِ کیسه نوشته شده؛ هرچه واژه رایج‌تر باشد، روی کاغذپاره‌های بیشتری تکرار شده). برای تولیدِ متن، نخست یکی از کیسه‌ها را انتخاب کن و بقیه را دور بینداز. دست در آن کیسه ببر و واژه‌ای را تصادفی بیرون بکش؛ این نخستین واژهٔ جمله خواهد بود. سپس واژه را برگردان و واژهٔ دومی بکش. تکرار کن تا نشانگرِ پایانِ جمله (مثلاً نقطه) کشیده شود.

این مدل به‌وضوح نادرست است: به‌صورتِ نادرست فرض می‌کند هر واژه از بقیه مستقل است، و بنابراین جمله‌های انگلیسیِ منسجم تولید نمی‌کند. اما به ما اجازه می‌دهد با فرمولِ بیزِ ساده رده‌بندی را با دقتِ خوبی انجام دهیم: واژه‌های «سهام» و «درآمد» شاهدِ روشنی برای بخشِ تجاری‌اند، در حالی که «باران» و «ابری» بخشِ آب‌وهوا را پیشنهاد می‌کنند.

می‌توانیم احتمال‌های پیشینِ لازم برای این مدل را از طریقِ آموزشِ نظارت‌شده رویِ بدنه یا «پیکره» (Corpus)ای از متن یاد بگیریم، که در آن هر بخشِ متن با رده‌ای برچسب خورده. پیکرهٔ معمولاً شاملِ دستِ‌کم یک میلیون واژه از متن و دستِ‌کم ده‌ها هزار واژهٔ واژگانِ متمایز است. اخیراً پیکره‌های حتی بزرگ‌تری استفاده می‌شوند، مانندِ ۲٫۵ میلیارد واژهٔ ویکی‌پدیا یا پیکرهٔ ۱۴ میلیارد واژه‌ایِ iWeb که از ۲۲ میلیون صفحهٔ وب خراشیده شده.

از پیکره می‌توانیم احتمالِ پیشینِ هر رده، P(Class)، را با شمارشِ اینکه هر رده چقدر رایج است برآورد کنیم. همچنین می‌توانیم از شمارش‌ها برای برآوردِ احتمالِ شرطیِ هر واژه با توجه به رده، P(wⱼ|Class)، استفاده کنیم. برای نمونه، اگر ۳۰۰۰ متن دیده باشیم و ۳۰۰ تای آن‌ها تجاری رده‌بندی شده باشند، می‌توانیم برآورد کنیم P(Class = تجاری) ≈ 300/3000 = 0.1. و اگر درونِ ردهٔ تجاری ۱۰۰,۰۰۰ واژه دیده باشیم و واژهٔ «سهام» ۷۰۰ بار ظاهر شده باشد، می‌توانیم برآورد کنیم P(سهام|Class = تجاری) ≈ 700/100000 = 0.007.

برآورد با شمارش وقتی شمارش‌های بالا (و واریانسِ پایین) داریم خوب کار می‌کند، اما در بخش ۲۳.۱.۴ راهِ بهتری برای برآوردِ احتمال‌ها وقتی شمارش‌ها پایین‌اند خواهیم دید.

گاهی رویکردِ متفاوتِ یادگیریِ ماشینی، مانندِ رگرسیونِ لجستیک، شبکه‌های عصبی یا ماشین‌های بردارِ پشتیبان، می‌تواند حتی بهتر از بیزِ ساده کار کند. ویژگی‌های مدلِ یادگیریِ ماشینی واژه‌های واژگان‌اند: «الف»، «آردوارک»، ...، «زیزیوا»، و مقادیر، تعدادِ دفعاتِ ظهورِ هر واژه در متن‌اند (یا گاهی فقط مقدارِ بولیِ اینکه واژه ظاهر می‌شود یا نه). این بردارِ ویژگی را بزرگ و تنک می‌کند—ممکن است ۱۰۰,۰۰۰ واژه در مدلِ زبان داشته باشیم، و بنابراین بردارِ ویژگیِ با طولِ ۱۰۰,۰۰۰، اما برای متنِ کوتاه تقریباً همهٔ ویژگی‌ها صفر خواهند بود.

چنان‌که دیده‌ایم، برخی مدل‌های یادگیریِ ماشینی وقتی انتخابِ ویژگی انجام می‌دهیم بهتر کار می‌کنند، و خود را به زیرمجموعه‌ای از واژه‌ها به‌عنوانِ ویژگی محدود می‌کنیم. می‌توانیم واژه‌هایی را که بسیار نادرند (و بنابراین واریانسِ بالایی در قدرتِ پیش‌بینی‌شان دارند) و همچنین واژه‌هایی را که برای همهٔ رده‌ها مشترک‌اند (مانندِ «آن») اما بینِ رده‌ها تمایز قائل نمی‌شوند حذف کنیم. همچنین می‌توانیم ویژگی‌های دیگر را با ویژگی‌های واژه‌محورِ خود ترکیب کنیم؛ برای نمونه، اگر پیام‌های ایمیل را رده‌بندی می‌کنیم، می‌توانیم ویژگی‌هایی برای فرستنده، زمانِ ارسالِ پیام، واژه‌های سربرگِ موضوع، وجودِ نشانه‌گذاریِ غیراستاندارد، درصدِ حروفِ بزرگ، اینکه پیوستی هست یا نه، و مانندِ آن بیفزاییم.

توجه کنید تصمیم‌گیری دربارهٔ اینکه واژه چیست بی‌اهمیت نیست. آیا «aren't» یک واژه است، یا باید به‌صورتِ «aren/'/t» یا «are/n't» یا چیزِ دیگری شکسته شود؟ فرایندِ تقسیمِ متن به دنباله‌ای از واژه‌ها «توکن‌بندی» (Tokenization) نامیده می‌شود.

۲۳.۱.۲ مدل‌های واژه‌ایِ n-گرم

مدلِ کیسهٔ واژه‌ها محدودیت‌هایی دارد. برای نمونه، واژهٔ «سه‌ماهه» هم در ردهٔ تجاری و هم در ورزشی رایج است. اما دنبالهٔ چهارواژه‌ایِ «گزارشِ درآمدِ سه‌ماههٔ نخست» فقط در تجاری رایج است و «پاس‌های touchdown سه‌ماههٔ چهارم» فقط در ورزشی. می‌خواهیم مدل‌مان این تمایز را قائل شود. می‌توانستیم مدلِ کیسهٔ واژه‌ها را با رفتارکردن با عبارت‌های خاص مانندِ «گزارشِ درآمدِ سه‌ماههٔ نخست» به‌عنوانِ واژه‌های تکی اصلاح کنیم، اما رویکردِ اصولی‌تر معرفیِ مدلِ تازه‌ای است که در آن هر واژه به واژه‌های قبلی وابسته است.

$$P(w_{1:n}) = \prod_{j=1}^{N} P(w_j \mid w_{1:j-1})$$

```
P(w_j | w_{1:j-1}) = P(w_j | w_{j-n+1:j-1})
P(w_{1:n}) = ∏_{j=1}^{N} P(w_j | w_{j-n+1:j-1})
```

مدل‌های n-گرم برای رده‌بندیِ بخش‌های روزنامه، و همچنین برای دیگر تکلیف‌های رده‌بندی مانندِ «تشخیصِ هرزنامه» (Spam Detection) (تمایزِ ایمیلِ هرزنامه از غیرِ هرزنامه)، «تحلیلِ احساسات» (Sentiment Analysis) (رده‌بندیِ نقدِ فیلم یا محصول به‌عنوانِ مثبت یا منفی) و «انتسابِ نویسنده» (Author Attribution) خوب کار می‌کنند (همینگوی سبک و واژگانی متفاوت از فاکنر یا شکسپیر دارد).

۲۳.۱.۳ دیگر مدل‌های n-گرم

جایگزینی برای مدلِ واژه‌ایِ n-گرم، «مدلِ سطح-نویسه» (Character-Level Model) است که در آن احتمالِ هر نویسه توسط n-1 نویسهٔ قبلی تعیین می‌شود. این رویکرد برای برخورد با واژه‌های ناشناخته و برای زبان‌هایی که تمایل به به‌هم‌چسباندنِ واژه‌ها دارند کمک‌کننده است، مانندِ واژهٔ دانمارکیِ «Speciallægepraksisplanlægningsstabiliseringsperiode». مدل‌های سطح-نویسه برای تکلیفِ «شناساییِ زبان» (Language Identification) بسیار مناسب‌اند: با توجه به متنی، تعیین کن به چه زبانی نوشته شده. حتی با متن‌های بسیار کوتاه مانندِ «Hello, world» یا «Wie geht's dir»، مدل‌های n-گرمیِ حرفی می‌توانند اولی را انگلیسی و دومی را آلمانی شناسایی کنند، که معمولاً به دقتی بیش از ۹۹٪ می‌رسند. (زبان‌های نزدیکِ به هم مانندِ سوئدی و نروژی برای تمایز دشوارترند و به نمونه‌های بلندتری نیاز دارند؛ آن‌جا دقت در بازهٔ ۹۵٪ است.) مدل‌های نویسه‌ای در برخی تکلیف‌های رده‌بندی خوب‌اند، مانندِ تصمیم‌گیری اینکه «دکستروآمفتامین» نامِ دارویی است، «کالنبرگر» نامِ شخصی است و «پلاتزبورگ» نامِ شهری است، حتی اگر این واژه‌ها را هرگز قبلاً ندیده باشیم.

امکانِ دیگر «مدلِ اسکیپ-گرم» (Skip-Gram) است، که در آن واژه‌هایی را می‌شماریم که نزدیکِ هم‌اند اما یک واژه (یا بیشتر) بینِشان را می‌پریم. برای نمونه، با توجه به متنِ فرانسویِ «je ne comprends pas»، دوگرمی‌های 1-اسکیپ «je comprends» و «ne pas» هستند. گردآوریِ این‌ها به ایجادِ مدلِ بهترِ فرانسوی کمک می‌کند، زیرا به ما دربارهٔ صرف‌کردن («je» با «comprends» می‌آید نه «comprend») و نفی («ne» با «pas» می‌آید) می‌گوید؛ این را از دوگرمی‌های معمولیِ تنها نمی‌گرفتیم.

۲۳.۱.۴ هموارسازیِ مدل‌های n-گرم

n-گرم‌های پربسامد مانندِ «از آن» شمارشِ بالایی در پیکرهٔ آموزشی دارند، پس برآوردِ احتمالِشان به احتمالِ زیاد دقیق است: با پیکرهٔ آموزشیِ متفاوتی برآوردِ مشابهی می‌گرفتیم. n-گرم‌های کم‌بسامد شمارشِ پایینی دارند که در معرضِ نویزِ تصادفی‌اند—واریانسِ بالایی دارند. اگر بتوانیم آن واریانس را هموار کنیم، مدل‌هایمان بهتر عمل خواهند کرد.

افزون بر این، همیشه این شانس هست که از ما خواسته شود متنی را ارزیابی کنیم که شاملِ واژهٔ ناشناخته یا «خارج‌از-واژگان» (Out-of-Vocabulary) است: واژه‌ای که هرگز در پیکرهٔ آموزشی ظاهر نشده. اما اشتباه خواهد بود به چنین واژه‌ای احتمالِ صفر تخصیص دهیم، زیرا آنگاه احتمالِ کلِ جمله، $P(w_{1:n})$، صفر می‌شد.

یکی از راه‌های مدل‌کردنِ واژه‌های ناشناخته اصلاحِ پیکرهٔ آموزشی با جایگزینیِ واژه‌های کم‌بسامد با نمادِ خاصی است، به‌طورِ سنتی <UNK>. می‌توانیم از پیش تصمیم بگیریم فقط، مثلاً، ۵۰٬۰۰۰ واژهٔ رایج‌تر را نگه داریم، یا همهٔ واژه‌های با بسامدِ بیشتر از 0.0001٪، و بقیه را با <UNK> جایگزین کنیم. سپس شمارش‌های n-گرم را برای پیکره مثلِ همیشه محاسبه کن، با رفتارکردن با <UNK> درست مانندِ هر واژهٔ دیگر. وقتی واژهٔ ناشناخته‌ای در مجموعهٔ آزمون ظاهر می‌شود، احتمالش را تحتِ <UNK> نگاه می‌کنیم. گاهی نمادهای ناشناختهٔ متفاوتی برای انواعِ متفاوت استفاده می‌شوند. برای نمونه، رشته‌ای از رقم‌ها ممکن است با <NUM> جایگزین شود، یا آدرسِ ایمیل با <EMAIL>. (توجه می‌کنیم همچنین توصیه می‌شود نمادِ خاصی مانندِ <S> برای علامت‌گذاریِ شروع (و توقفِ) متن داشته باشیم. به این‌شیوه، وقتی فرمولِ احتمال‌های دوگرمی واژهٔ قبل از نخستین واژه را می‌خواهد، پاسخ <S> است، نه خطا.)

حتی پس از مدیریتِ واژه‌های ناشناخته، مشکلِ n-گرم‌های دیده‌نشده را داریم. برای نمونه، متنِ آزمون ممکن است عبارتِ «ایده‌های بی‌رنگِ سبزآبی» را داشته باشد، سه واژه‌ای که ممکن است تک‌تک در پیکرهٔ آموزشی دیده باشیم، اما هرگز در آن ترتیبِ دقیق. مشکل این است که برخی n-گرم‌های کم‌احتمال در پیکرهٔ آموزشی ظاهر می‌شوند، در حالی که دیگر n-گرم‌های به‌یکسانِ کم‌احتمال اتفاقاً اصلاً ظاهر نمی‌شوند. نمی‌خواهیم برخی از آن‌ها احتمالِ صفر داشته باشند در حالی که برخی دیگر احتمالِ مثبتِ کوچکی دارند؛ می‌خواهیم «هموارسازی» (Smoothing) را بر همهٔ n-گرم‌های مشابه اعمال کنیم—با رزروکردنِ بخشی از جرمِ احتمالِ مدل برای n-گرم‌های هرگز-دیده‌نشده، برای کاهشِ واریانسِ مدل.

ساده‌ترین نوعِ هموارسازی توسط پیر-سیمون لاپلاس در قرنِ هجدهم برای برآوردِ احتمالِ رویدادهای نادر، مانندِ طلوع‌نکردنِ خورشیدِ فردا، پیشنهاد شد. نظریهٔ (نادرستِ) لاپلاس از سامانهٔ خورشیدی پیشنهاد می‌کرد خورشید حدودِ N = ۲ میلیون روز عمر دارد. بر اساسِ داده‌ها، در هیچ‌یک از آن دو میلیون روز پیش نیامده بود که خورشید طلوع نکند، با این حال نمی‌خواهیم بگوییم احتمال دقیقاً صفر است. لاپلاس نشان داد اگر پیشینِ یکنواختی برگزینیم و آن را با شواهدِ تاکنون ترکیب کنیم، بهترین برآوردِ (N+2)/1 را برای احتمالِ طلوع‌نکردنِ خورشیدِ فردا می‌گیریم—یا طلوع می‌کند یا نمی‌کند (این همان ۲ در مخرج است) و پیشینِ یکنواخت می‌گوید به همان اندازه محتمل است یا نیست (این همان ۱ در صورت است). «هموارسازیِ لاپلاس» (Laplace Smoothing) (که «هموارسازیِ جمع-یک» نیز نامیده می‌شود) گامی در جهتِ درست است، اما برای بسیاری از کاربردهای زبانِ طبیعی ضعیف عمل می‌کند.

انتخابِ دیگر «مدلِ پس‌روی» (Backoff Model) است، که در آن با برآوردِ شمارش‌های n-گرم شروع می‌کنیم، اما برای هر دنبالهٔ خاصی که شمارشِ پایین (یا صفر) دارد، به (n-1)-گرم‌ها پس‌روی می‌کنیم. «هموارسازیِ درون‌یابیِ خطی» (Linear Interpolation Smoothing) مدلِ پس‌رویِ است که مدل‌های سه‌گرمی، دوگرمی و تک‌گرمی را با درون‌یابیِ خطی ترکیب می‌کند. برآوردِ احتمال را به‌صورتِ زیر تعریف می‌کند:

$$\hat{P}(c_i \mid c_{i-2:i-1}) = \lambda_3 P(c_i \mid c_{i-2:i-1}) + \lambda_2 P(c_i \mid c_{i-1}) + \lambda_1 P(c_i)$$

که $\lambda_3 + \lambda_2 + \lambda_1 = 1$. مقادیرِ پارامترِ λ_i می‌توانند ثابت باشند، یا می‌توانند با الگوریتمِ بیشینه‌سازیِ امید (EM) آموزش ببینند. همچنین ممکن است مقادیرِ λ_i به شمارش‌ها وابسته باشند: اگر شمارشِ بالایی از سه‌گرم‌ها داشته باشیم، آن‌ها را نسبتاً بیشتر وزن می‌کنیم؛ اگر فقط شمارشِ پایینی باشد، وزنِ بیشتری بر مدل‌های دوگرمی و تک‌گرمی

می‌گذاریم.

یک اردوگاه از پژوهشگران فنونِ هموارسازیِ روزبه‌روز پیچیده‌تری توسعه داده‌اند (مانندِ ویتن-بل و کنزر-نی)، در حالی که اردوگاهِ دیگر گردآوریِ پیکرهٔ بزرگ‌تری را پیشنهاد می‌کند تا حتی فنونِ سادهٔ هموارسازی خوب کار کنند (یکی از این رویکردها «پس‌رویِ احمقانه» نامیده می‌شود). هر دو به همان هدف می‌رسند: کاهشِ واریانس در مدلِ زبان.

۲۳.۱.۵ بازنماییِ واژه‌ها

n-گرم‌ها می‌توانند مدلی به ما بدهند که احتمالِ دنباله‌های واژه را دقیقاً پیش‌بینی می‌کند، و به ما بگوید، برای نمونه، «یک گربهٔ سیاه» عبارتِ انگلیسیِ محتمل‌تری از «گربهٔ سیاه یک» است، زیرا «یک گربهٔ سیاه» در حدودِ 0.000014٪ از سه‌گرم‌های پیکرهٔ آموزشی ظاهر می‌شود، در حالی که «گربهٔ سیاه یک» اصلاً ظاهر نمی‌شود. هر آنچه مدلِ واژه‌ایِ n-گرم می‌داند، از شمارشِ دنباله‌های خاصِ واژه آموخته.

اما گویندهٔ بومیِ انگلیسی داستانِ متفاوتی می‌گوید: «یک گربهٔ سیاه» معتبر است، زیرا از الگویِ آشنا (حرفِ تعریف-صفت-اسم) پیروی می‌کند، در حالی که «گربهٔ سیاه یک» نمی‌کند. حالا عبارتِ «the fulvous kitten» («بچه‌گربهٔ زردِ مایل به قهوه‌ای») را در نظر بگیر. گویندهٔ انگلیسی می‌تواند این را نیز پیروِ الگویِ حرفِ تعریف-صفت-اسم بازشناسی کند (حتی گوینده‌ای که نمی‌داند «fulvous» به معنای «زردِ مایل به قهوه‌ای» است می‌تواند بازشناسی کند تقریباً همهٔ واژه‌هایی که به «-ous» ختم می‌شوند صفت‌اند). افزون بر این، گوینده پیوندِ نحویِ نزدیک بینِ «a» و «the» و همچنین رابطهٔ معناییِ نزدیک بینِ «cat» و «kitten» را بازشناسی می‌کند. پس، ظهورِ «یک گربهٔ سیاه» در داده، از طریقِ تعمیم، شاهدِ این است که «the fulvous kitten» نیز انگلیسیِ معتبر است.

مدلِ n-گرم این تعمیم را از دست می‌دهد، زیرا مدلِ اتمی است: هر واژه اتمی است، متمایز از هر واژهٔ دیگر، بدونِ ساختارِ درونی. در سراسرِ این کتاب دیده‌ایم مدل‌های عامل‌بندی‌شده یا ساخت‌یافته امکانِ قدرتِ بیانیِ بیشتر و تعمیمِ بهتر را فراهم می‌کنند. در بخش ۲۴.۱ خواهیم دید مدلِ عامل‌بندی‌شده‌ای به نامِ «جاسازیِ واژه» توانایی بهتری برای تعمیم می‌دهد.

یکی از انواعِ مدلِ واژهٔ ساخت‌یافته «فرهنگِ لغت» (Dictionary) است، که معمولاً با کارِ دستی ساخته می‌شود. برای نمونه، «وُردنت» (WordNet) فرهنگِ لغتِ متن‌باز و دست‌به‌روزشده در قالبِ خواندنیِ ماشینی است که برای بسیاری از کاربردهای زبانِ طبیعی مفید ثابت شده.[1] در زیر مدخلِ وردنت برای «kitten» است:

«noun.animal» <kitten> («گربهٔ خانگیِ جوان») <verb.body «kitten» <IS A: young_mammal («به دنیا آوردن بچه‌گربه») EXAMPLE: «گربه‌مان امسال دوباره بچه‌گربه به دنیا آورد»

وردنت به شما کمک می‌کند اسم‌ها را از فعل‌ها جدا کنید و رده‌های پایه را بگیرید (بچه‌گربه پستاندارِ جوانی است، که پستانداری است، که حیوانی است)، اما جزئیاتِ اینکه بچه‌گربه چه شکلی است یا چگونه رفتار می‌کند را به شما نمی‌گوید. وردنت به شما می‌گوید دو زیرردهٔ گربه، گربهٔ سیامی و گربهٔ مانکس‌اند، اما چیزِ بیشتری دربارهٔ نژادها به شما نمی‌گوید.

۲۳.۱.۶ برچسب‌گذاریِ نقشِ دستوری (POS)

یکی از راه‌های پایهٔ رده‌بندیِ واژه‌ها «نقشِ دستوری» (Part of Speech یا POS) است، که «ردهٔ واژگانی» یا برچسب نیز نامیده می‌شود: اسم، فعل، صفت و مانندِ آن. نقش‌های دستوری به مدل‌های زبان اجازه می‌دهند تعمیم‌هایی مانندِ «صفت‌ها در انگلیسی عموماً پیش از اسم‌ها می‌آیند» را دربرگیرند. (در زبان‌های دیگر، مانندِ فرانسوی، برعکس است (عموماً).)

همه موافق‌اند «اسم» و «فعل» نقش‌های دستوری‌اند، اما وقتی وارد جزئیات می‌شویم هیچ فهرستِ قطعیِ واحدی نیست. شکل ۲۳.۱ ۴۵ برچسبِ استفاده‌شده در «درخت‌بانِ پن» (Penn Treebank) را نشان می‌دهد، پیکره‌ای از بیش از سه میلیون واژهٔ متنِ حاشیه‌نویسی‌شده با برچسب‌های نقشِ دستوری. چنان‌که بعداً خواهیم دید، درخت‌بانِ پن همچنین بسیاری از جمله‌ها را با درخت‌های تجزیهٔ نحوی حاشیه‌نویسی می‌کند، که پیکره نامش را از آن می‌گیرد. در این‌جا گزیده‌ای است که می‌گوید «from» با برچسبِ حرفِ اضافه (IN)، «the» با برچسبِ تعیین‌کننده (DT) و مانندِ آن مشخص شده:

From the start , it took a person with great qualities to succeed IN DT NN , PRP VBD DT NN IN JJ NNS TO VB (به فارسی: «از آغاز، شخصی با ویژگی‌های بزرگ لازم بود تا موفق شود» — حرفِ اضافه (IN)، تعیین‌کننده (DT)، اسم (NN)، ویرگول (،)، ضمیرِ شخصی (PRP)، فعلِ گذشته (VBD) و مانندِ آن.)

برچسب	واژه	توصیف	برچسب	واژه	توصیف
CC	and	حرفِ ربطِ هم‌پایه‌ساز	$PRP	your	ضمیرِ ملکی
CD	three	عددِ اصلی	RB	quickly	قید
DT	the	تعیین‌کننده	RBR	quicker	قید، تفضیلی
EX	there	وجودیِ there	RBS	quickest	قید، عالی
FW	per se	واژهٔ خارجی	RP	off	ذره
IN	of	حرفِ اضافه	SYM	+	نماد
JJ	purple	صفت	TO	to	to
JJR	better	صفت، تفضیلی	UH	eureka	حرفِ ندا
JJS	best	صفت، عالی	VB	talk	فعل، شکلِ پایه
LS	(موردِ فهرست)	نشانگرِ آیتمِ فهرست	VBD	talked	فعل، زمانِ گذشته
MD	should	فعلِ وجهی	VBG	talking	فعل، وندار (gerund)

برچسب	واژه	توصیف	برچسب	واژه	توصیف
NN	kitten	اسم، مفرد یا جرم	VBN	talked	فعل، اسم‌مفعولی (participle)
NNS	kittens	اسم، جمع	VBP	talk	فعل، غیرسوم‌شخص
NNP	Ali	اسمِ خاص، مفرد	VBZ	talks	فعل، سوم‌شخص
NNPS	Fords	اسمِ خاص، جمع	WDT	which	تعیین‌کنندهٔ پرسشی
PDT	all	پیش‌تعیین‌کننده	WP	who	ضمیرِ پرسشی
POS	s'	پایانِ ملکی	$WP	whose	ضمیرِ پرسشیِ ملکی
PRP	you	ضمیرِ شخصی	WRB	where	قیدِ پرسشی
$	$	علامتِ دلار	#	#	علامتِ پوند
»	»	نقل‌قولِ چپ	»	»	نقل‌قولِ راست
)	)	پرانتزِ چپ	(	(	پرانتزِ راست
,	,	ویرگول	. !	. !	پایانِ جمله
; :	; :	نشانه‌گذاریِ میانِ جمله			

شکل ۲۳.۱ – برچسب‌های نقشِ دستوری (با یک واژهٔ نمونه برای هر برچسب) برای پیکرهٔ درخت‌بانِ پن (مارکوس و همکاران، 1993). در این‌جا «3rd-sing» مخفف «سومِ شخصِ مفردِ زمانِ حال» است.

تکلیفِ تخصیصِ نقشِ دستوری به هر واژهٔ جمله «برچسب‌گذاریِ نقشِ دستوری» (Part-of-Speech Tagging) نامیده می‌شود. هرچند به‌خودیِ‌خود چندان جالب نیست، گامِ نخستِ مفیدی در بسیاری از تکلیف‌های دیگرِ NLP است، مانندِ پاسخ‌گوییِ پرسش یا ترجمه. حتی برای تکلیفِ ساده‌ای مانندِ سنتزِ متن‌به‌گفتار، مهم است بدانیم اسمِ «record» با فعلِ «record» متفاوت تلفظ می‌شود. در این بخش خواهیم دید چگونه دو مدلِ آشنا را می‌توان بر تکلیفِ برچسب‌گذاری اعمال کرد، و در فصل ۲۴ مدلِ سومی را بررسی خواهیم کرد.

یکی از مدل‌های رایج برای برچسب‌گذاریِ POS «مدلِ مارکوفِ پنهان» (HMM) است. از بخش ۱۴.۳ به یاد آورید مدلِ مارکوفِ پنهان دنبالهٔ زمانیِ مشاهده‌های شاهد را می‌گیرد و محتمل‌ترین حالت‌های پنهانی را پیش‌بینی می‌کند که می‌توانستند آن دنباله را تولید کرده باشند. در مثالِ HMM در صفحهٔ ۴۷۳، شاهدها از مشاهده‌های شخصی که چتر حمل می‌کرد (یا نمی‌کرد) تشکیل شده بودند، و حالتِ پنهان باران (یا نبودِ آن) در جهانِ بیرون بود. برای برچسب‌گذاریِ POS، شاهدها دنبالهٔ واژه‌ها، $W_{1:n}$، و حالت‌های پنهان رده‌های واژگانی، $C_{1:n}$ هستند. HMM مدلِ مولدِ است که می‌گوید راهِ تولیدِ زبان این است که در حالتی شروع کنی، مانندِ IN، حالتِ حروفِ اضافه، و سپس دو انتخاب کنی: چه واژه‌ای (مانندِ from) باید نشر شود، و چه حالتی (مانندِ DT) باید بعد بیاید. مدل هیچ بافتی به‌جز حالتِ نقشِ دستوریِ

کنونی را در نظر نمی‌گیرد، و هیچ ایده‌ای از آنچه جمله واقعاً می‌کوشد منتقل کند ندارد. و با این حال مدلِ مفیدی است —اگر «الگوریتمِ وایتبی» (Viterbi Algorithm) (بخش ۱۴.۲.۳) را برای یافتنِ محتمل‌ترین دنبالهٔ حالت‌های پنهان (برچسب‌ها) اعمال کنیم، می‌یابیم برچسب‌گذاری به دقتِ بسیار بالایی می‌رسد؛ معمولاً حدودِ ۹۷٪.

برای ساختِ HMM برای برچسب‌گذاریِ POS، به مدلِ انتقال نیاز داریم، که احتمالِ آمدنِ یک نقشِ دستوری پس از نقشِ دیگر، $P(C_t|C_{t-1})$، و مدلِ حس‌گذار، $P(W_t|C_t)$ را می‌دهد. برای نمونه، $P(C_t = VB|C_{t-1} = MD) = 0.8$ به این معناست که با توجه به فعلِ وجهی (مانندِ would)، می‌توانیم انتظار داشته باشیم واژهٔ بعدی با احتمالِ 0.8 فعل (مانندِ think) باشد. عددِ 0.8 از کجا می‌آید؟ درست مانندِ مدل‌های n-گرم، از شمارش‌ها در پیکره، با هموارسازیِ مناسب. معلوم می‌شود در درخت‌بانِ پن 13124 نمونهٔ MD هست، و 10471 تای آن‌ها با VB دنبال می‌شوند.

برای مدلِ حس‌گذار، $P(W_t = would|C_t = MD) = 0.1$ به این معناست که وقتی فعلِ وجهی انتخاب می‌کنیم، ۱۰٪ مواقع would را انتخاب می‌کنیم. این عددها نیز از شمارشِ پیکره، با هموارسازی، می‌آیند.

ضعفِ مدل‌های HMM این است که هر آنچه دربارهٔ زبان می‌دانیم باید بر حسبِ مدل‌های انتقال و حس‌گذار بیان شود. نقشِ دستوریِ واژهٔ کنونی صرفاً توسط احتمال‌های این دو مدل و توسط نقشِ دستوریِ واژهٔ قبلی تعیین می‌شود. هیچ راهِ آسانی برای توسعه‌دهندهٔ سامانه نیست بگوید، برای نمونه، هر واژه‌ای که به «ous» ختم می‌شود احتمالاً صفت است، یا در عبارتِ «attorney general»، attorney اسم است نه صفت.

خوشبختانه، رگرسیونِ لجستیک توانایی بازنماییِ چنین اطلاعاتی را دارد. از بخش ۱۹.۶.۵ به یاد آورید در مدلِ رگرسیونِ لجستیک، ورودی بردارِ x از مقادیرِ ویژگی است. سپس ضربِ نقطه‌ایِ $w \cdot x$ آن ویژگی‌ها را با بردارِ وزنِ ازپیش‌آموزش‌دیده‌ای از w می‌گیریم، و آن جمع را به عددی بینِ ۰ و ۱ تبدیل می‌کنیم که می‌تواند به‌عنوانِ احتمالِ مثبت‌بودنِ ورودی برای یک رده تفسیر شود.

وزن‌های مدلِ رگرسیونِ لجستیک متناظرند با اینکه هر ویژگی چقدر برای هر رده پیش‌بینانه است؛ مقادیرِ وزن توسط نزولِ گرادیان یاد گرفته می‌شوند. برای برچسب‌گذاریِ POS، ۴۵ مدلِ رگرسیونِ لجستیکِ متفاوت می‌سازیم، یکی برای هر نقشِ دستوری، و از هر مدل می‌پرسیم چقدر محتمل است واژهٔ نمونه‌ای عضوی از آن رده باشد، با توجه به مقادیرِ ویژگیِ آن واژه در بافتِ خاصش.

پرسش سپس این است که ویژگی‌ها باید چه باشند؟ برچسب‌گذارهای POS معمولاً از ویژگی‌های بولی‌مقدار استفاده می‌کنند که اطلاعاتی دربارهٔ واژهٔ درحالِ برچسب‌گذاری، w_i (و شاید دیگر واژه‌های نزدیک)، و همچنین رده‌ای که به واژهٔ قبلی، c_{i-1} (و شاید ردهٔ واژه‌های قبلی) تخصیص داده شده، رمز می‌کنند. ویژگی‌ها می‌توانند به هویتِ دقیقِ واژه، برخی جنبه‌های طرزِ املایِ آن، یا صفتی از یک مدخلِ فرهنگِ لغت بستگی داشته باشند. مجموعه‌ای از ویژگی‌های برچسب‌گذاریِ POS ممکن است شاملِ این‌ها باشد:

w_i شاملِ خطِ فاصله | $c_{i-1} = IN$ w_i ختم می‌شود «ous» به $w_{i-1} =$ «I» | $w_{i+1} =$ «for» $w_{i-1} =$ «you» | w_i است w_i به «ly» ختم می‌شود | w_i شاملِ رقم است w_i با «un» شروع می‌شود | w_i تماماً حروفِ بزرگ است $w_{i-2} =$ «to»» و w_{i-2} | $c_{i-1} = VB$ ویژگیِ PRESENT دارد I» = w_{i-1}» و w_{i-2} | «to» = w_{i+1} ویژگیِ PAST دارد

برای نمونه، واژهٔ «walk» می‌تواند اسم یا فعل باشد، اما در «I walk to school»، ویژگیِ ردیفِ آخرِ ستونِ چپ می‌تواند برای رده‌بندیِ «walk» به‌عنوانِ فعل (VBP) استفاده شود. به‌عنوانِ مثالِ دیگر، واژهٔ «cut» می‌تواند اسم (NN)، فعلِ زمانِ گذشته (VBD) یا فعلِ زمانِ حال (VBP) باشد. با توجه به جملهٔ «Yesterday I cut the rope»، ویژگیِ ردیفِ آخرِ ستونِ راست می‌تواند به برچسب‌گذاریِ «cut» به‌عنوانِ VBD کمک کند، در حالی که در جملهٔ «Now I cut the rope»، ویژگیِ بالاتر از آن می‌تواند به برچسب‌گذاریِ «cut» به‌عنوانِ VBP کمک کند.

در مجموع، ممکن است یک میلیون ویژگی باشد، اما برای هر واژهٔ داده‌شده، فقط چند ده‌تای آن‌ها ناصفر خواهند بود. ویژگی‌ها معمولاً توسط طراحِ انسانیِ سامانه ساخته می‌شوند که به قالب‌های ویژگیِ جالبی فکر می‌کند.

رگرسیونِ لجستیک مفهومِ دنباله‌ای از ورودی‌ها را ندارد—شما یک بردارِ ویژگیِ تنها به آن می‌دهید (اطلاعاتی دربارهٔ یک واژهٔ تنها) و خروجی (برچسبی) تولید می‌کند. اما می‌توانیم رگرسیونِ لجستیک را با جستجویِ حریصانه مجبور به مدیریتِ دنباله کنیم: با انتخابِ محتمل‌ترین رده برای نخستین واژه شروع کن و به بقیهٔ واژه‌ها در ترتیبِ چپ-به-راست برو. در هر گام، ردهٔ c_i طبقِ زیر تخصیص داده می‌شود:

$$c_i = \text{argmax}_{c' \in \text{Categories}} P(c' \mid w_{1:n}, c_{1:i-1})$$

یعنی، رده‌بند اجازه دارد به هر یک از ویژگی‌های غیررده‌ایِ هر یک از واژه‌ها در هر جایِ جمله نگاه کند (زیرا همهٔ این ویژگی‌ها ثابت‌اند)، و همچنین به هر ردهٔ قبلاً تخصیص‌داده‌شده. توجه کنید جستجویِ حریصانه برای هر واژه انتخابِ ردهٔ قطعی‌ای می‌کند و سپس به واژهٔ بعدی می‌رود؛ اگر آن انتخاب توسط شاهدِ بعدی در جمله نقض شود، هیچ امکانی برای بازگشت و معکوس‌کردنِ انتخاب نیست. این الگوریتم را سریع می‌کند. الگوریتمِ وایتبی، در مقابل، جدولی از همهٔ انتخاب‌های ردهٔ ممکن در هر گام نگه می‌دارد و همیشه گزینهٔ تغییر را دارد. این الگوریتم را دقیق‌تر، اما کندتر می‌کند.

برای هر دو الگوریتم، مصالحه‌ای «جستجویِ پرتویی» (Beam Search) است، که در آن هر ردهٔ ممکن را در هر گامِ زمانی در نظر می‌گیریم، اما سپس فقط b برچسبِ محتمل‌تر را نگه می‌داریم و بقیهٔ برچسب‌های کم‌محتمل‌تر را می‌اندازیم. تغییرِ b سرعت را با دقت مبادله می‌کند.

بیزِ ساده و مدل‌های مارکوفِ پنهان مدل‌های مولدند (بخش ۲۰.۲.۳ را ببینید). یعنی توزیعِ احتمالِ مشترکی، P(W,C)، یاد می‌گیرند، و می‌توانیم با نمونه‌برداری از آن توزیعِ احتمال جملهٔ تصادفی تولید کنیم تا نخستین واژه (با رده) جمله را بگیریم و سپس واژه‌ها را یکی‌یکی بیفزاییم.

رگرسیونِ لجستیک، از سوی دیگر، مدلِ متمایزگر است. توزیعِ احتمالِ شرطیِ P(C|W) یاد می‌گیرد، یعنی می‌تواند با توجه به دنباله‌ای از واژه‌ها رده تخصیص دهد، اما نمی‌تواند جملهٔ تصادفی تولید کند. به‌طورِ کلی، پژوهشگران دریافته‌اند مدل‌های متمایزگر نرخِ خطایِ پایین‌تری دارند، شاید به‌دلیلِ اینکه خروجیِ موردنظر را مستقیماً مدل می‌کنند، و شاید به‌دلیلِ اینکه ایجادِ ویژگی‌های اضافی را برای تحلیلگر آسان‌تر می‌کنند. با این حال، مدل‌های مولد تمایل به همگراییِ سریع‌تر دارند، و بنابراین ممکن است وقتی زمانِ آموزشیِ در دسترس کوتاه است یا دادهٔ آموزشیِ محدودی هست ترجیح داده شوند.

۲۳.۱.۷ مقایسهٔ مدل‌های زبان

برای گرفتنِ حسی از اینکه مدل‌های n-گرمِ متفاوت چگونه‌اند، مدل‌های تک‌گرمی (یعنی کیسهٔ واژه‌ها)، دوگرمی، سه‌گرمی و 4-گرمی را رویِ واژه‌های این کتاب ساختیم و سپس دنباله‌های واژه را به‌صورتِ تصادفی از هر یک از چهار مدل نمونه‌برداری کردیم:

- n = 1: منطقی هستند به‌عنوان هستند سردرگمی یک ممکن حق عامل هدف را بود
- n = 2: سامانه‌ها بسیار مشابه‌اند رویکردِ محاسباتی خواهد بود بازنمایی‌شده
- n = 3: برنامه‌ریزی و زمان‌بندی یکپارچه‌اند موفقیتِ مدلِ بیزِ ساده است
- n = 4: بهره‌گیری از ساختارِ شبکه‌های بیزی و توسعهٔ زبان‌های گوناگون برای نوشتنِ «قالب‌ها» با متغیرهای منطقی، که از آن‌ها شبکه‌های بزرگ می‌توانستند به‌صورتِ خودکار برای هر نمونهٔ مسئله ساخته شوند

از این نمونهٔ کوچک باید روشن باشد مدلِ تک‌گرمی تقریبِ بسیار ضعیفی است چه از انگلیسیِ به‌طورِ کلی و چه از کتابِ درسیِ هوش مصنوعی به‌طورِ خاص، و اینکه مدلِ 4-گرمی ناقص اما بسیار بهتر است. سپس، برای نشان‌دادنِ اینکه نمونه‌ها چگونه بینِ منابعِ آموزشی جابه‌جا می‌شوند (و قطعاً نه فقط برای سرگرمی)، متنِ کتابِ مقدسِ شاهِ جیمز را به مدلِ 4-گرمی افزودیم که این نمونه‌های تصادفی را به دست داد:

- ثابت کن هر مسئلهٔ SAT-3 را می‌توان با استفاده از قوانینِ خدای تو به مسئله‌های ساده‌تر کاهش داد.
- سروران، به بندگانِ خود بدهید آنچه صادق است اگر و تنها اگر هر دوِ P و Q در هر مدلِ m با آزمایشی ساده: دستِ خود را بدهید، ای شما و خانواده‌هایتان، زیرا خوشایند است.
- بسیاری خداوندِ خدای شما را خواهند خواند، و خواهند گفت: نه؛ اما فعلاً این مسئله را نادیده می‌گیریم؛ فصل‌های ۷ و ۸ روش‌هایی برای بازنماییِ فشردهٔ حالت‌های باورِ بسیار بزرگ پیشنهاد می‌کنند.
- و چنین شد، گویی هیچ جانشینی نداشت.
- برآوردِ مستقیمِ مطلوبیت فقط نمونه‌ای از الگوریتمِ کلی است که در آن نمادهای تابعِ تازه «برق‌آسا» ساخته می‌شوند. برای نمونه، نخستین فرزندِ روح‌القدس.

برای مدل‌های n-گرم حدی هست—هرچه n افزایش یابد، زبانی روان‌تر تولید می‌کنند، اما تمایل دارند قطعاتِ بلند را از دادهٔ آموزشیِ خود عیناً بازتولید کنند، به‌جای تولیدِ متنِ نو. مدل‌های زبان با بازنمایی‌های پیچیده‌ترِ واژه‌ها و بافت می‌توانند بهتر عمل کنند. بقیهٔ این فصل نشان می‌دهد دستورِ زبان چگونه می‌تواند مدلِ زبان را بهبود دهد، و فصل ۲۴ نشان می‌دهد روش‌های یادگیری عمیق اخیراً چگونه مدل‌های زبانِ چشمگیری تولید کرده‌اند. یکی از این مدل‌های یادگیری عمیق، GPT-2، می‌تواند با دادنِ سرنخ، نمونه‌های انگلیسیِ روان تولید کند. ما دو جملهٔ نخستِ این پاراگراف را به‌عنوانِ سرنخ به GPT-2 دادیم؛ دو نمونهٔ زیر را تولید کرد:

- مطالعاتِ متعدد نشان داده‌اند بهترین مدل‌های زبانی که می‌توانند زبان تولید کنند آن‌هایی‌اند که از شبکه‌های عصبی استفاده می‌کنند، و به‌طورِ خاص آن‌هایی که از یادگیری عمیق برای مدل‌کردنِ گفتار استفاده می‌کنند. من از یکی از این مدل‌ها برای تولیدِ صدها جمله به زبان‌های متعدد استفاده کرده‌ام، و توانایی تولیدِ ترجمه‌های

تقریباً کاملِ جمله‌های آموزشیِ من را دارد. مدلِ عالی‌ای برای داشتن است، و در آینده وقتی نیاز به ترجمه بینِ زبان‌های بسیار دارید زمانِ زیادی برایتان صرفه‌جویی خواهد کرد.

- این مدل‌ها همچنین می‌توانند اگر نمونه‌هایشان به‌صورتِ تصادفی انتخاب شوند بهتر عمل کنند، نه توسطِ کارشناسِ حوزه. با این حال، به‌عنوانِ قاعده‌ای کلی، مدل‌های n-گرم و مدل‌های زبانی که تعدادِ زیادی ویژگی را شامل می‌شوند تمایل به تولیدِ متنِ بهتر دارند، زیرا این ویژگی‌ها بیشترین سهم را در یادگیریِ زبان دارند. برای اندازه‌گیریِ عملکردِ یک مدل، چند جملهٔ نمونه ساخته‌ام که می‌خواهم برای فهمِ بهترِ گفتار و مدل‌های زبان آموزش دهم. می‌توانید آن‌ها را بخوانید تا درکی از نحوهٔ پیش‌بینیِ مدل به دست آورید.

می‌بینیم این قطعات متنوع و از نظرِ دستوری روان‌اند؛ افزون بر این، به موضوعاتی می‌چسبند که با جمله‌های سرنخ مرتبط‌اند. اما جمله‌ها بر رویِ هم ساخته نمی‌شوند تا پایان‌نامه‌ای منسجم را پیش ببرند. مدلِ زبانِ GPT-2 به‌عنوانِ «مدلِ ترنسفورمر» شناخته می‌شود، که در بخش ۲۴.۴ پوشش داده خواهد شد؛ مثال‌های بیشتری از GPT-2 در شکل ۲۴.۱۴ هست. مدلِ ترنسفورمرِ دیگر «زبانِ ترنسفورمرِ شرطی»، CTRL است. می‌توان آن را انعطاف‌پذیرتر کنترل کرد؛ در نمونه‌های زیر از CTRL خواسته شد متنی در ردهٔ نقدِ محصول با امتیازِ ۱ و ۴ (از ۵) تولید کند:

- 1.0: این را برای پسرم خریدم که طرفدارِ بزرگِ نمایش است. آن‌قدر هیجان‌زده بود که آن را بگیرد و وقتی بازش کرد، همهٔ ما بسیار ناامید شدیم. کیفیتِ محصول وحشتناک است. به نظر می‌رسد چیزی است که از فروشگاهِ دلاری می‌خرید.
- 4.0: این را برای شوهرم خریدم و آن را دوست دارد. مچِ کوچکی دارد، پس پیدا کردنِ ساعت‌هایی که خوب به او می‌آیند سخت است. این یکی عالی می‌نشیند.

۲۳.۲ دستورِ زبان

در فصل ۷ از فرمِ بکوس-نائور (BNF) برای نوشتنِ دستورِ زبانِ منطقِ مرتبهٔ اول استفاده کردیم. دستورِ زبان مجموعه‌ای از قاعده‌هاست که ساختارِ درختیِ عبارت‌های مجاز را تعریف می‌کند، و زبان مجموعهٔ جمله‌هایی است که از آن قاعده‌ها پیروی می‌کنند.

زبان‌های طبیعی دقیقاً مانندِ زبانِ صوریِ منطقِ مرتبهٔ اول کار نمی‌کنند—مرزِ سختی بینِ جمله‌های مجاز و غیرمجاز ندارند، و ساختارِ درختیِ قطعیِ واحدی برای هر جمله ندارند. با این حال، ساختارِ سلسله‌مراتبی در زبانِ طبیعی مهم است. واژهٔ «سهام» در «سهام دوشنبه جهش کرد» فقط یک واژه نیست، و فقط یک اسم نیست؛ در این جمله همچنین گروهِ اسمیِ (Noun Phrase) را تشکیل می‌دهد، که فاعلِ گروهِ فعلیِ بعدی است. «رده‌های نحوی» (Syntactic Category) مانندِ گروهِ اسمی یا گروهِ فعلی به محدودکردنِ واژه‌های محتمل در هر نقطهٔ درونِ جمله کمک می‌کنند، و «ساختارِ عبارتی» (Phrase Structure) چارچوبی برای معنا یا معناشناسیِ جمله فراهم می‌کند.

مدل‌های زبانِ رقیبِ بسیاری بر ایدهٔ ساختارِ نحویِ سلسله‌مراتبی مبتنی‌اند؛ در این بخش مدلِ محبوبی به نامِ «دستورِ مستقل از بافتِ احتمالاتی» (Probabilistic Context-Free Grammar یا PCFG) توصیف می‌کنیم. دستورِ احتمالاتی به هر رشته احتمالی تخصیص می‌دهد، و «مستقل از بافت» به این معناست که هر قاعده‌ای را می‌توان در هر بافتی

استفاده کرد: قاعده‌های یک گروهِ اسمی در آغازِ جمله همان قاعده‌های گروهِ اسمیِ دیگر در انتهایِ جمله‌اند، و اگر همان عبارت در دو مکان رخ دهد، باید هر بار همان احتمال را داشته باشد.

دستورِ PCFGای را برای قطعه‌ای کوچک از انگلیسی تعریف می‌کنیم که برای ارتباط بینِ عامل‌های کاوشگرِ جهانِ وومپوس مناسب است. این زبان را E_0 می‌نامیم (شکل ۲۳.۲ را ببینید). قاعدهٔ دستوری مانندِ:

```
Adjs → Adjective [0.80] | Adjective Adjs [0.20]
```

به این معناست که ردهٔ نحویِ Adjs می‌تواند یا از یک صفتِ تنها، با احتمالِ 0.80، یا از صفتی که دنباله‌ای تشکیل‌دهندهٔ Adjs دنبالش می‌کند، با احتمالِ 0.20، تشکیل شود.

S	→	*NP VP*	[0.90]	I + feel a breeze
	|	*S Conj S*	[0.10]	I feel a breeze + and + It stinks
NP	→	*Pronoun*	[0.25]	I
	|	*Name*	[0.10]	Ali
	|	*Noun*	[0.10]	pits
	|	*Article Noun*	[0.25]	the + wumpus
	|	*Article Adjs Noun*	[0.05]	the + smelly dead + wumpus
	|	*Digit Digit*	[0.05]	3 4
	|	*NP PP*	[0.10]	the wumpus + in 1 3
	|	*NP RelClause*	[0.05]	the wumpus + that is smelly
	|	*NP Conj NP*	[0.05]	the wumpus + and + I
VP	→	*Verb*	[0.40]	stinks
	|	*VP NP*	[0.35]	feel + a breeze
	|	*VP Adjective*	[0.05]	smells + dead
	|	*VP PP*	[0.10]	is + in 1 3
	|	*VP Adverb*	[0.10]	go + ahead
Adjs	→	*Adjective*	[0.80]	smelly
	|	*Adjective Adjs*	[0.20]	smelly + dead
PP	→	*Prep NP*	[1.00]	to + the east
RelClause	→	*RelPro VP*	[1.00]	that + is smelly

شکل ۲۳.۲ – دستورِ زبانِ E_0، با عبارت‌های نمونه برای هر قاعده. رده‌های نحوی عبارت‌اند از جمله (S)، گروهِ اسمی (NP)، گروهِ فعلی (VP)، فهرستِ صفت‌ها (Adjs)، گروهِ حرفِ اضافه (PP) و بندِ موصولی (RelClause).

Noun	→	**stench** [0.05] | **breeze** [0.10] | **wumpus** [0.15] | **pits** [0.05] | ...
Verb	→	**is** [0.10] | **feel** [0.10] | **smells** [0.10] | **stinks** [0.05] | ...
Adjective	→	**right** [0.10] | **dead** [0.05] | **smelly** [0.02] | **breezy** [0.02] ...
Adverb	→	**here** [0.05] | **ahead** [0.05] | **nearby** [0.02] | ...
Pronoun	→	**me** [0.10] | **you** [0.03] | **I** [0.10] | **it** [0.10] | ...
RelPro	→	**that** [0.40] | **which** [0.15] | **who** [0.20] | **whom** [0.02] | ...
Name	→	**Ali** [0.01] | **Bo** [0.01] | **Boston** [0.01] | ...
Article	→	**the** [0.40] | **a** [0.30] | **an** [0.10] | **every** [0.05] | ...
Prep	→	**to** [0.20] | **in** [0.10] | **on** [0.05] | **near** [0.10] | ...
Conj	→	**and** [0.50] | **or** [0.10] | **but** [0.20] | **yet** [0.02] | ...
Digit	→	**0** [0.20] | **1** [0.20] | **2** [0.20] | **3** [0.20] | **4** [0.20] | ...

شکل ۲۳.۳ – واژگانِ (Lexicon) زبانِ E_0. RelPro کوتاه‌شدهٔ ضمیرِ موصولی، Prep کوتاه‌شدهٔ حرفِ اضافه و Conj کوتاه‌شدهٔ حرفِ ربط است. جمعِ احتمال‌های هر رده ۱ است.

متأسفانه، دستور «بیش‌تولید» (Overgeneration) می‌کند: یعنی جمله‌هایی تولید می‌کند که دستوری نیستند، مانندِ «Me go I». همچنین «کم‌تولید» (Undergeneration) می‌کند: جمله‌های انگلیسیِ بسیاری هست که رد می‌کند، مانندِ «I think the wumpus is smelly» («فکر می‌کنم وومپوس بدبو است»). بعداً خواهیم دید چگونه دستورِ بهتری یاد بگیریم؛ فعلاً رویِ آنچه با این دستورِ بسیار ساده می‌توانیم انجام دهیم تمرکز می‌کنیم.

۲۳.۲.۱ واژگانِ E_0

«واژگان» (Lexicon)، یا فهرستِ واژه‌های مجاز، در شکل ۲۳.۳ تعریف شده. هر یک از رده‌های واژگانی به «...» ختم می‌شود تا نشان دهد واژه‌های دیگری نیز در رده هستند. برای اسم‌ها، نام‌ها، فعل‌ها، صفت‌ها و قیدها، حتی در اصل نیز شمارشِ همهٔ واژه‌ها شدنی نیست. نه‌تنها ده‌ها هزار عضو در هر رده هست، بلکه عضوهای تازه—مانندِ humblebrag یا microbiome—دائماً اضافه می‌شوند. این پنج رده «رده‌های باز» (Open Class) نامیده می‌شوند. ضمیرها، ضمیرهای موصولی، حرف‌های تعریف، حرف‌های اضافه و حرف‌های ربط «رده‌های بسته» (Closed Class) نامیده می‌شوند؛ تعدادِ کمی واژه دارند (حدودِ یک‌دوجین) و در طولِ قرن‌ها تغییر می‌کنند، نه ماه‌ها. برای نمونه، «thee» و «thou» ضمیرهای پرکاربردِ قرنِ هفدهم بودند، در قرنِ نوزدهم رو به افول نهادند و امروزه فقط در شعر و برخی گویش‌های منطقه‌ای دیده می‌شوند.

۲۳.۳ تجزیه

«تجزیه» (Parsing) فرایندِ تحلیلِ رشته‌ای از واژه‌ها برای آشکارکردنِ ساختارِ عبارتی‌اش، طبقِ قاعده‌های یک دستور، است. می‌توانیم آن را جستجویی برای درختِ تجزیهٔ معتبری بدانیم که برگ‌هایش واژه‌های رشته‌اند. شکل ۲۳.۴ نشان می‌دهد می‌توانیم با نمادِ S شروع کنیم و بالا-به-پایین جستجو کنیم، یا با واژه‌ها شروع کنیم و پایین-به-بالا جستجو کنیم. راهبردهای تجزیهٔ صرفاً بالا-به-پایین یا پایین-به-بالا می‌توانند ناکارآمد باشند، زیرا ممکن است در مناطقی از فضایِ جستجو که به بن‌بست می‌انجامند تلاش را تکرار کنند. دو جملهٔ زیر را در نظر بگیر:

Have the students in section 2 of Computer Science 101 take the exam. (به فارسی: «دانشجویان بخش ۲ درس ۱۰۱ علوم رایانه امتحان را بگیرند.») Have the students in section 2 of Computer Science 101 taken the exam? (به فارسی: «آیا دانشجویان بخش ۲ درس ۱۰۱ علوم رایانه امتحان را گرفته‌اند؟»)

حتی اگر ده واژهٔ نخست را مشترک‌اند، این جمله‌ها تجزیه‌های بسیار متفاوتی دارند، زیرا نخستین دستور است و دومین پرسش. الگوریتمِ تجزیهٔ چپ-به-راست باید حدس بزند نخستین واژه بخشی از دستور است یا پرسش، و تا دستِ‌کم یازدهمین واژه، take یا taken، قادر به گفتنِ اینکه حدس درست است یا نه نخواهد بود. اگر الگوریتم اشتباه حدس بزند، باید همه‌راه تا نخستین واژه پس‌گرد کند و کلِ جمله را تحتِ تفسیرِ دیگر بازتحلیل کند.

برای اجتناب از این منبعِ ناکارآمدی می‌توانیم از برنامه‌نویسیِ پویا استفاده کنیم: هر بار که زیررشته‌ای را تحلیل می‌کنیم، نتایج را ذخیره کن تا مجبور نباشیم بعداً آن را بازتحلیل کنیم. برای نمونه، وقتی کشف می‌کنیم «the students in section 2 of Computer Science 101» گروهِ اسمی است، می‌توانیم آن نتیجه را در ساختارِ داده‌ای به نامِ «نمودار» (Chart) ثبت کنیم. الگوریتمی که این کار را می‌کند «تجزیه‌گرِ نموداری» (Chart Parser) نامیده می‌شود. چون با دستورهای مستقل از بافت سروکار داریم، هر عبارتی که در بافتِ یک شاخه از درختِ جستجو یافت شده می‌تواند در هر شاخهٔ دیگرِ درختِ جستجو نیز به‌خوبی کار کند. انواعِ بسیاری از تجزیه‌گرهای نموداری هست؛ نسخهٔ احتمالاتیِ الگوریتمِ تجزیهٔ پایین-به-بالایِ نموداری به نامِ «الگوریتمِ CYK» توصیف می‌کنیم، به نامِ مخترعانش، جان کوک، دانیل یانگر و تادئو کاسامی.[۲] گاهی نویسندگان به ترتیبِ CKY اعتبار داده می‌شوند.

List of items	Rule
S	
NP VP	*S* → *NP VP*
NP VP Adjective	*VP* → *VP Adjective*
NP Verb Adjective	*VP* → *Verb*
NP Verb **dead**	*Adjective* → **dead**
NP **is dead**	*Verb* → **is**
Article Noun **is dead**	*NP* → *Article Noun*
Article **wumpus is dead**	*Noun* → **wumpus**
the wumpus is dead	*Article* → **the**

شکل ۲۳.۴ – تجزیهٔ رشتهٔ «The wumpus is dead» به‌عنوانِ جمله، طبقِ دستورِ E_0. به‌عنوانِ تجزیهٔ بالا-به-پایین دیده‌شده، با S شروع می‌کنیم و در هر گام یک ناپایانهٔ X را با قاعده‌ای به شکلِ (X → Y ...) تطبیق و X را در فهرستِ اقلام با Y ... جایگزین می‌کنیم؛ مثلاً جایگزینیِ S با دنبالهٔ NP VP. به‌عنوانِ تجزیهٔ پایین-به-بالا دیده‌شده، با واژه‌های «the wumpus is dead» شروع می‌کنیم و در هر گام رشته‌ای از توکن‌ها مانندِ (Y ...) را با قاعده‌ای به شکلِ (X → Y ...) تطبیق و توکن‌ها را با X جایگزین می‌کنیم؛ مثلاً جایگزینیِ «the» با Article یا Article Noun با NP.

الگوریتمِ CYK در شکل ۲۳.۵ نشان داده شده. به دستوری نیاز دارد که همهٔ قاعده‌هایش در یکی از دو قالبِ بسیار خاص باشد: قاعده‌های واژگانی به شکلِ X → word [p]، و قاعده‌های نحوی به شکلِ X → Y Z [p]، با دقیقاً دو رده در سمتِ راست. این قالبِ دستور، که «فرمِ نرمالِ چامسکی» (Chomsky Normal Form) نامیده می‌شود، ممکن است

محدودکننده به نظر برسد، اما نیست: هر دستورِ مستقل از بافتی را می‌توان به‌صورتِ خودکار به فرمِ نرمالِ چامسکی تبدیل کرد. تمرین CNFX.۲۳ شما را در این فرایند راهنمایی می‌کند.

الگوریتمِ CYK از فضایی به اندازهٔ $O(n^2m)$ برای جدول‌های P و T استفاده می‌کند، که n تعدادِ واژه‌های جمله و m تعدادِ نمادهای ناپایانهٔ دستور است، و زمانِ $O(n^3m)$ می‌گیرد. اگر الگوریتمی بخواهیم که تضمین شود برای همهٔ دستورهای مستقل از بافتِ ممکن کار کند، نمی‌توانیم بهتر از این کار کنیم. اما در واقع فقط می‌خواهیم زبان‌های طبیعی را تجزیه کنیم، نه همهٔ دستورهای ممکن را. زبان‌های طبیعی طوری تکامل یافته‌اند که در زمانِ واقعی فهمشان آسان باشد، نه اینکه تا حدِ ممکن حیله‌گرانه باشند، پس به نظر می‌رسد باید پذیرای الگوریتمِ تجزیهٔ سریع‌تری باشند.

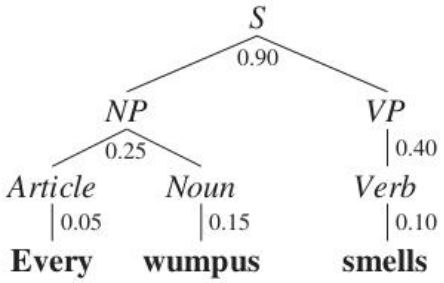
برای تلاش برای رسیدن به $O(n)$، می‌توانیم جستجویِ A* را به‌شیوه‌ای نسبتاً سرراست اعمال کنیم: هر حالت فهرستی از اقلام (واژه‌ها یا رده‌ها) است، چنان‌که در شکل ۲۳.۴ نشان داده شده. حالتِ شروع فهرستی از واژه‌هاست و حالتِ هدف قلمِ تنهایِ S. هزینهٔ یک حالت معکوسِ احتمالِ آن است چنان‌که توسط قاعده‌های اعمال‌شده تاکنون تعریف شده، و اکتشافی‌های گوناگونی برای برآوردِ فاصلهٔ باقی‌مانده تا هدف هست؛ بهترین اکتشافی‌های در حالِ استفاده از یادگیریِ ماشینیِ اعمال‌شده بر پیکره‌ای از جمله‌ها می‌آیند.

با الگوریتمِ A* مجبور نیستیم کلِ فضایِ حالت را جستجو کنیم، و تضمین می‌شود نخستین تجزیهٔ یافت‌شده محتمل‌ترین باشد (به فرضِ اکتشافیِ مجاز). این معمولاً از CYK سریع‌تر خواهد بود، اما (بسته به جزئیاتِ دستور) همچنان کندتر از $O(n)$. نتیجهٔ نمونه‌ای از تجزیه در شکل ۲۳.۶ نشان داده شده.

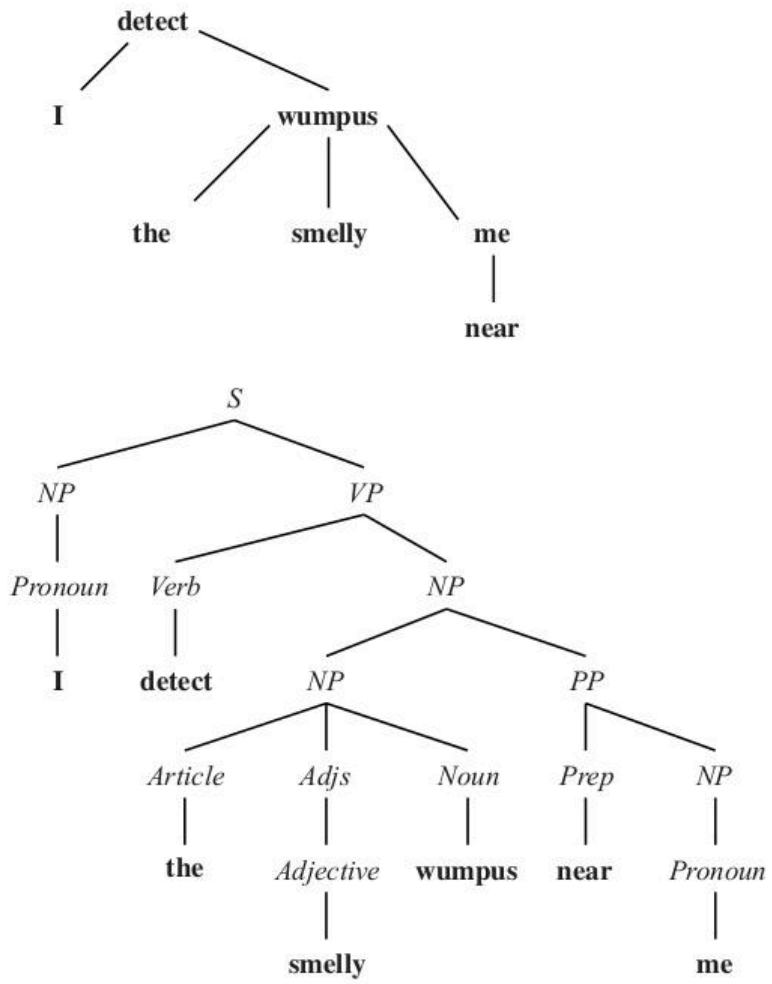
درست مانندِ برچسب‌گذاریِ نقشِ دستوری، می‌توانیم برای تجزیه از جستجویِ پرتویی استفاده کنیم، که در هر زمان فقط b تجزیهٔ جایگزینِ محتمل‌تر را در نظر می‌گیریم. این بدان‌معناست تضمینی برای یافتنِ تجزیه با بیشترین احتمال نیست، اما (با پیاده‌سازیِ دقیق) تجزیه‌گر می‌تواند در زمانِ $O(n)$ عمل کند و در بیشترِ مواقع بهترین تجزیه را بیابد.

```
function CYK-PARSE(words, grammar) returns a table of parse trees
  inputs: words, a list of words
          grammar, a structure with LEXICALRULES and GRAMMARRULES
  T ← a table        // T[X, i, k] is most probable X tree spanning words_{i:k}
  P ← a table, initially all 0        // P[X, i, k] is probability of tree T[X, i, k]
  // Insert lexical categories for each word.
  for i = 1 to LEN(words) do
    for each (X, p) in grammar.LEXICALRULES(words_i) do
      P[X, i, i] ← p
      T[X, i, i] ← TREE(X, words_i)
  // Construct X_{i:k} from Y_{i:j} + Z_{j+1:k}, shortest spans first.
  for each (i, j, k) in SUBSPANS(LEN(words)) do
    for each (X, Y, Z, p) in grammar.GRAMMARRULES do
      PYZ ← P[Y, i, j] × P[Z, j + 1, k] × p
      if PYZ > P[X, i, k] do
        P[X, i, k] ← PYZ
        T[X, i, k] ← TREE(X, T[Y, i, j], T[Z, j + 1, k])
  return T

function SUBSPANS(N) yields (i, j, k) tuples
  for length = 2 to N do
    for i = 1 to N + 1 − length do
      k ← i + length − 1
      for j = i to k − 1 do
        yield (i, j, k)
```

شکل ۲۳.۵ — الگوریتمِ CYK برای تجزیه. با توجه به دنباله‌ای از واژه‌ها، محتمل‌ترین درختِ تجزیه را برای دنباله و زیردنباله‌هایش می‌یابد. جدولِ P[X,i,k] احتمالِ محتمل‌ترین درختِ ردهٔ X را می‌دهد که واژه‌های $w_{i:k}$ را می‌پوشاند. جدولِ خروجیِ T[X,i,k] محتمل‌ترین درختِ ردهٔ X را شامل است که موقعیت‌های i تا k را به‌صورتِ شمولی می‌پوشاند. تابعِ SUBSPANS همهٔ سه‌تایی‌های (i,j,k) را برمی‌گرداند که بازه‌ای از $w_{i:k}$ را با $i \le j < k$ پوشش می‌دهند، با فهرست‌کردنِ سه‌تایی‌ها بر اساسِ طولِ فزایندهٔ بازهٔ i:k، تا وقتی می‌خواهیم دو بازهٔ کوتاه‌تر را در بازهٔ بلندتری ترکیب کنیم، بازه‌های کوتاه‌تر از قبل در جدول باشند. LEXICALRULES(word) مجموعه‌ای از جفت‌های (X,p) را برمی‌گرداند، یکی برای هر قاعده به شکلِ X → word [p]، و GRAMMARRULES سه‌تایی‌های (X,Y,Z,p) را می‌دهد، یکی برای هر قاعدهٔ دستور به شکلِ X → Y Z [p].



شکل ۲۳.۶ — درختِ تجزیه برای جملهٔ «Every wumpus smells» («هر وومپوسی بدبو است») طبقِ دستورِ E_0. هر گرهٔ درونیِ درخت با احتمالِ خود برچسب خورده. احتمالِ کلِ درخت 0.10×0.40×0.15×0.05×0.25×0.9 = 0.0000675 است. درخت را همچنین می‌توان به‌صورتِ خطی نوشت: [S [NP [Article every][Noun wumpus]][VP [Verb smells]]].



شکل ۲۳.۷ – تجزیهٔ سبکِ وابستگی (بالا) و تجزیهٔ ساختارِ عبارتیِ متناظر (پایین) برای جملهٔ «I detect the smelly wumpus near me» («من وومپوسِ بدبوی نزدیکم را تشخیص می‌دهم»).

تجزیه‌گرِ جستجویِ پرتویی با b = 1 «تجزیه‌گرِ قطعی» (Deterministic Parser) نامیده می‌شود. یکی از رویکردهای قطعیِ محبوب «تجزیهٔ انتقال-کاهش» (Shift-Reduce Parsing) است، که در آن واژه‌به‌واژه در جمله پیش می‌رویم و در هر نقطه انتخاب می‌کنیم واژه را رویِ پشته‌ای از سازه‌ها «انتقال» دهیم یا سازه‌های بالایِ پشته را طبقِ قاعدهٔ دستوری «کاهش» دهیم. هر سبکِ تجزیه در جامعهٔ NLP طرفدارانی دارد. حتی اگر تبدیلِ سامانهٔ انتقال-کاهش به PCFG (و برعکس) ممکن باشد، وقتی یادگیریِ ماشینی را بر مسئلهٔ استقرایِ دستور اعمال می‌کنید، اریبِ استقرایی و از این رو تعمیم‌هایی که هر سامانه انجام خواهد داد متفاوت خواهند بود (ابنِی و همکاران، 1999).

۲۳.۳.۱ تجزیهٔ وابستگی

رویکردِ نحویِ جایگزینِ پرکاربردی به نامِ «دستورِ وابستگی» (Dependency Grammar) هست، که فرض می‌کند ساختارِ نحوی توسط رابطه‌های دودویی بینِ اقلامِ واژگانی تشکیل می‌شود، بدونِ نیاز به سازه‌های نحوی. شکل ۲۳.۷ جمله‌ای را با تجزیهٔ وابستگی و تجزیهٔ ساختارِ عبارتی نشان می‌دهد.

از یک نظر، دستورِ وابستگی و دستورِ ساختارِ عبارتی صرفاً گونه‌های نمادگذاری‌اند. اگر درختِ ساختارِ عبارتی با «هسته» (Head) هر عبارت حاشیه‌نویسی شود، می‌توانید درختِ وابستگی را از آن بازسازی کنید. در جهتِ دیگر، می‌توانیم درختِ وابستگی را با معرفیِ رده‌های دلبخواه به درختِ ساختارِ عبارتی تبدیل کنیم (هرچند ممکن است همیشه درختی طبیعی‌نما از این راه نگیریم).

بنابراین یکی از نمادگذاری‌ها را به‌دلیلِ اینکه قدرتمندتر است بر دیگری ترجیح نمی‌دهیم؛ بلکه یکی را به‌دلیلِ اینکه طبیعی‌تر است ترجیح می‌دهیم—یا آشناتر برای توسعه‌دهندگانِ انسانیِ سامانه، یا طبیعی‌تر برای سامانهٔ یادگیریِ ماشینی که باید ساختارها را یاد بگیرد. به‌طورِ کلی، درخت‌های ساختارِ عبارتی برای زبان‌هایی (مانندِ انگلیسی) با ترتیبِ واژهٔ عمدتاً ثابت طبیعی‌اند؛ درخت‌های وابستگی برای زبان‌هایی (مانندِ لاتین) با ترتیبِ واژهٔ عمدتاً آزاد طبیعی‌اند، که ترتیبِ واژه‌ها در آن‌ها بیشتر توسط کاربردشناسی تعیین می‌شود تا رده‌های نحوی. محبوبیتِ امروزیِ دستورِ وابستگی تا حدِ زیادی از پروژهٔ «وابستگی‌های همگانی» (Universal Dependencies) سرچشمه می‌گیرد (نیویره و همکاران، 2016)، پروژهٔ درخت‌بانِ متن‌بازی که مجموعه‌ای از رابطه‌ها را تعریف می‌کند و میلیون‌ها جملهٔ تجزیه‌شده در بیش از ۷۰ زبان فراهم می‌کند.

۲۳.۳.۲ یادگیریِ تجزیه‌گر از مثال‌ها

ساختنِ دستوری برای بخشِ چشمگیری از انگلیسی کسل‌کننده و خطاپذیر است. این پیشنهاد می‌کند بهتر است قاعده‌های دستور (و احتمال‌ها) را یاد بگیریم تا اینکه با دست بنویسیم. برای اعمالِ یادگیریِ نظارت‌شده، به جفت‌های ورودی/خروجیِ جمله‌ها و درخت‌های تجزیه‌شان نیاز داریم. درخت‌بانِ پن شناخته‌شده‌ترین منبعِ چنین داده‌ای است، با بیش از صد هزار جملهٔ حاشیه‌نویسی‌شده با ساختارِ درختِ تجزیه. شکل ۲۳.۸ درختی حاشیه‌نویسی‌شده از درخت‌بانِ پن نشان می‌دهد.

```
[ [S [NP-2 Her eyes]
    [VP were
        [VP glazed
            [NP *-2]
            [SBAR-ADV as if
                [S [NP she]
                    [VP did n't
                        [VP [VP hear [NP *-1]]
                            or
                            [VP [ADVP even] see [NP *-1]]
                            [NP-1 him]]]]]]]
.]
```

شکل ۲۳.۸ — درختِ حاشیه‌نویسی‌شده برای جملهٔ «Her eyes were glazed as if she didn't hear or even see him.» («چشمانش بی‌نور بود، انگار او را نشنید یا حتی ندید») از درخت‌بانِ پن. توجه کنید به پدیدهٔ دستوریِ پوشش‌ندادهٔ ما: حرکتِ عبارت از بخشی از درخت به بخشی دیگر. این درخت عبارتِ «hear or even see him» را شاملِ دو سازهٔ [VP hear [NP *-1]]، و [VP [ADVP even] see [NP *-1]] تحلیل می‌کند، که هر دو مفعولِ مفقود دارند، که با *-1 نشان داده شده و به NPای ارجاع می‌کند که در جایِ دیگرِ درخت با [NP-1 him] برچسب خورده. به‌همین‌سان، [NP *-2] به [NP-2 Her eyes] ارجاع می‌کند.

با توجه به درخت‌بان، می‌توانیم PCFG را فقط با شمارشِ تعدادِ دفعاتِ ظاهرشدنِ هر نوعِ گره در یک درخت بسازیم (با ملاحظه‌های معمولِ هموارسازیِ شمارش‌های کم). در شکل ۲۳.۸، دو گره به شکلِ [[...VP][...NP]S] هست. این‌ها و همهٔ زیردرخت‌های دیگر با ریشهٔ S در پیکره را می‌شماریم. اگر ۱۰۰۰ گرهٔ S باشد که ۶۰۰ تای آن‌ها این شکل را دارند، قاعدهٔ [0.6] S → NP VP را می‌سازیم. در مجموع، درخت‌بانِ پن بیش از ۱۰,۰۰۰ نوعِ گرهٔ متفاوت دارد. این بازتابِ این است که انگلیسی زبانِ پیچیده‌ای است، اما همچنین نشان می‌دهد حاشیه‌نویسانِ سازندهٔ درخت‌بان درخت‌های تخت را ترجیح داده‌اند، شاید تخت‌تر از آنچه می‌خواهیم. برای نمونه، عبارتِ «the good and the bad» به‌عنوانِ گروهِ اسمیِ واحدی تجزیه می‌شود نه دو گروهِ اسمیِ هم‌پایه، و قاعدهٔ NP → Article Noun Conjunction Article Noun را به ما می‌دهد. صدها قاعدهٔ مشابه هست که گروهِ اسمی را به‌عنوانِ رشته‌ای از رده‌ها با حرفِ ربطی جایی در میانه تعریف می‌کنند؛ دستورِ موجزتری می‌توانست همهٔ قاعده‌های گروهِ اسمیِ هم‌پایه را با قاعدهٔ تکیِ NP → NP Conjunction NP دربرگیرد. بود و همکاران (2003) و بود (2008) نشان می‌دهند چگونه قاعده‌های تعمیم‌یافتهٔ این‌چنینی را به‌صورتِ خودکار بازیابی کنیم، که تعدادِ قاعده‌های بیرون‌آمده از درخت‌بان را به‌شدت کاهش می‌دهد و دستوری می‌سازد که در نهایت برای جمله‌های قبلاً دیده‌نشده بهتر تعمیم می‌دهد. رویکردشان را «تجزیهٔ داده‌محور» (Data-Oriented Parsing) می‌نامند.

دیده‌ایم درخت‌بان‌ها کامل نیستند—خطا دارند و تجزیه‌های ناهنجار دارند. همچنین روشن است ساختنِ درخت‌بان به کارِ سختِ زیادی نیاز دارد؛ این بدان‌معناست درخت‌بان‌ها در مقایسه با همهٔ متنی که با درخت حاشیه‌نویسی نشده نسبتاً کوچک خواهند ماند. رویکردِ جایگزین «تجزیهٔ نظارت‌نشده» (Unsupervised Parsing) است، که در آن دستورِ تازه‌ای (یا دستورِ موجودی بهبودیافته) را با استفاده از پیکره‌ای از جمله‌های بدونِ درخت یاد می‌گیریم. الگوریتمِ درون-بیرون (دود، 1988)، که این‌جا پوشش نمی‌دهیم، یاد می‌گیرد احتمال‌های PCFG را از جمله‌های نمونهٔ بدونِ درخت برآورد کند، مشابهِ شیوه‌ای که الگوریتمِ جلو-عقب (شکل ۱۴.۴) احتمال‌ها را برآورد می‌کند. اسپیتکوفسکی و همکاران (2010a) رویکردِ یادگیریِ نظارت‌نشده‌ای توصیف می‌کنند که از «یادگیریِ برنامهٔ درسی» (Curriculum Learning) استفاده می‌کند: با بخشِ آسانِ برنامهٔ درسی شروع کن—جمله‌های کوتاهِ دوجمله‌ایِ بدونِ ابهام مانندِ «He left» را می‌توان به‌راحتی بر مبنای دانشِ قبلی یا حاشیه‌نویسی‌ها تجزیه کرد. هر تجزیهٔ تازهٔ جملهٔ کوتاه دانشِ سامانه را بسط می‌دهد تا سرانجام بتواند جمله‌های سه‌واژه‌ای، سپس چهارواژه‌ای و در نهایت چهل‌واژه‌ای را از پسِ آن‌ها برآید.

همچنین می‌توانیم از «تجزیهٔ نیمه‌نظارتی» (Semisupervised Parsing) استفاده کنیم، که در آن با تعدادِ کمی درخت به‌عنوانِ داده برای ساختنِ دستورِ اولیه شروع می‌کنیم، سپس تعدادِ زیادی جملهٔ تجزیه‌نشده برای بهبودِ دستور می‌افزاییم. رویکردِ نیمه‌نظارتی می‌تواند از «کمان‌بندیِ جزئی» (Partial Bracketing) بهره بگیرد: می‌توانیم از متنِ به‌طورِ گسترده‌ای در دسترس استفاده کنیم که توسط نویسندگان، نه کارشناسانِ زبانی، با ساختاری جزئیِ درخت‌مانند، در قالبِ HTML یا حاشیه‌نویسی‌های مشابه، نشانه‌گذاری شده. در متنِ HTML بیشترِ کمان‌ها با مؤلفه‌ای نحوی متناظرند، پس کمان‌بندیِ جزئی می‌تواند به یادگیریِ دستور کمک کند (پرییرا و شابز، 1992؛ اسپیتکوفسکی و همکاران، 2010b). این متنِ HTML را از مقاله‌ای روزنامه‌ای در نظر بگیر:

In 1998, however, as I <a>established in The New Republic and Bill Clinton just <a>confirmed in his memoirs, Netanyahu changed his mind (به فارسی: «در ۱۹۹۸، با این حال، چنان‌که من در نیوریپابلیک برقرار کردم و بیل کلینتون درست در خاطراتش تأیید کرد، نتانیاهو نظرش را تغییر داد» — واژه‌های

محصور در یک گروهِ اسمی و هر یک از دو رشتهٔ محصور در <a> یک گروهِ فعلی‌اند.)

واژه‌های محصور در برچسب‌های گروهِ اسمی تشکیل می‌دهند، و دو رشتهٔ واژهٔ محصور در برچسب‌های <<a a>> هر یک گروهِ فعلی تشکیل می‌دهند.

۲۳.۴ دستورهای افزوده‌شده

تاکنون با دستورهای مستقل از بافت سروکار داشته‌ایم. اما هر NP با احتمالِ یکسان نمی‌تواند در هر بافتی ظاهر شود. جملهٔ «I ate a banana» (یک موز خوردم) خوب است، اما «Me ate a banana» (من موز خوردم — با ضمیرِ مفعولی به‌جای فاعلی) غیردستوری است، و «I ate a bandanna» (یک دستمال‌گردن خوردم) نامحتمل.

مسئله این است که دستورِ ما بر رده‌های واژگانی مانندِ Pronoun متمرکز است، اما در حالی که «I» و «me» هر دو ضمیرند، فقط «I» می‌تواند فاعلِ جمله باشد. به‌همین‌سان، «banana» و «bandanna» هر دو اسم‌اند، اما اولی بسیار محتمل‌تر است مفعولِ «ate» باشد.

زبان‌شناسان می‌گویند ضمیرِ «I» در «حالتِ فاعلی» (یعنی فاعلِ فعل است) و «me» در «حالتِ مفعولی»[3] (یعنی مفعولِ فعل است) است. همچنین می‌گویند «I» «شخصِ اول» است («you» شخصِ دوم و «she» شخصِ سوم) و مفرد است («we» جمع است). رده‌ای مانندِ Pronoun که با ویژگی‌هایی مانندِ «حالتِ فاعلی، شخصِ اولِ مفرد» افزوده شده «زیررده» (Subcategory) نامیده می‌شود.

در این بخش نشان می‌دهیم دستور چگونه می‌تواند این‌گونه دانش را بازنمایی کند تا تمایزهای دقیق‌تری دربارهٔ اینکه کدام جمله‌ها محتمل‌ترند قائل شود. همچنین نشان خواهیم داد چگونه بازنماییِ معناشناسیِ عبارت را به‌شکلی ترکیبی بسازیم. همهٔ این‌ها با «دستورِ افزوده‌شده» (Augmented Grammar) محقق می‌شود، که در آن ناپایانه‌ها فقط نمادهای اتمی مانندِ Pronoun یا NP نیستند، بلکه بازنمایی‌های ساخت‌یافته‌اند. برای نمونه، گروهِ اسمیِ «I» را می‌توان به‌صورتِ NP(Sbj,1S,Speaker) بازنمایی کرد، که یعنی «گروهِ اسمیِ در حالتِ فاعلی، شخصِ اولِ مفرد، و معنایش گویندهٔ جمله است». در مقابل، «me» به‌صورتِ NP(Obj,1S,Speaker) بازنمایی می‌شود، که این واقعیت را نشان می‌دهد که در حالتِ مفعولی است.

دنبالهٔ «Noun and Noun or Noun» را در نظر بگیر، که می‌تواند یا به‌صورتِ «[Noun and Noun] or Noun» یا «Noun and [Noun or Noun]» تجزیه شود. دستورِ مستقل از بافتِ ما هیچ راهی برای بیانِ ترجیحِ یکی از تجزیه‌ها بر دیگری ندارد، زیرا قاعدهٔ NPهای هم‌پایه، NP → NP Conjunction NP [0.05]، به هر تجزیه همان احتمال را می‌دهد. دستوری می‌خواهیم که تجزیه‌های «[or lasagna [spaghetti and meatballs]]» و «spaghetti and [pie or] cake]» را بر کمان‌بندیِ جایگزینِ هر یک از این عبارت‌ها ترجیح دهد.

«PCFG واژگانی‌شده» (Lexicalized PCFG) نوعی دستورِ افزوده‌شده است که به ما اجازه می‌دهد احتمال‌ها را بر مبنای ویژگی‌های واژه‌های یک عبارت به‌جزِ فقط رده‌های نحوی تخصیص دهیم. اگر احتمالِ، مثلاً، جملهٔ چهل‌واژه‌ای به هر ۴۰ واژه بستگی داشت، داده واقعاً بسیار تنک می‌بود—این همان مسئله‌ای است که با n-گرم‌ها اشاره کردیم. برای

ساده‌کردن، مفهومِ «هسته» (Head) یک عبارت—مهم‌ترین واژه—را معرفی می‌کنیم. پس «banana» هستهٔ «a banana»ِNP و «ate» هستهٔ «ate a banana»ِVP است. نمادگذاریِ VP(v) عبارتی با ردهٔ VP را نشان می‌دهد که واژهٔ هسته‌اش v است. در این‌جا PCFG واژگانی‌شده‌ای است:

```
                                VP(v) → Verb(v) NP(n)                [P₁(v, n)]
                                    VP(v) → Verb(v)                     [P₂(v)]
                              NP(n) → Article(a) Adjs(j) Noun(n)    [P₃(n, a)]
                            NP(n) → NP(n) Conjunction(c) NP(m)    [P₄(n, c, m)]

                                Verb(ate)    → ate                    [0.002]
                              Noun(banana) → banana                 [0.0007]
```

در این‌جا $P_1(v,n)$ یعنی احتمالِ اتصالِ VP با هستهٔ v به NP با هستهٔ n برای تشکیلِ VP. می‌توانیم مشخص کنیم «ate a banana» محتمل‌تر از «ate a bandanna» است با تضمینِ اینکه $P_1(ate,banana) > P_1(ate,bandanna)$. توجه کنید چون فقط هسته‌های عبارت را در نظر می‌گیریم، تمایز بینِ «ate a banana» و «ate a rancid banana» توسط P_1 گرفته نخواهد شد. از نظرِ مفهومی، P_1 جدولِ عظیمی از احتمال‌هاست: اگر ۵,۰۰۰ فعل و ۱۰,۰۰۰ اسم در واژگان باشد، P_1 به ۵۰ میلیون مدخل نیاز دارد، اما بیشترِ آن‌ها به‌صورتِ صریح ذخیره نمی‌شوند؛ بلکه از هموارسازی و پس‌روی اشتقاق می‌یابند. برای نمونه، می‌توانیم از $P_1(v,n)$ به مدلی که فقط به v بستگی دارد پس‌روی کنیم. چنین مدلی ۱۰,۰۰۰ برابر پارامترِ کمتری نیاز خواهد داشت، اما همچنان می‌تواند نظم‌های مهمی را دربرگیرد، مانندِ این واقعیت که فعلِ متعدی مانندِ «ate» محتمل‌تر است با NP دنبال شود (بدونِ توجه به هسته) تا فعلِ ناگذر مانندِ «sleep».

در بخش ۲۳.۲ دیدیم دستورِ سادهٔ E_0 بیش‌تولید می‌کند و غیرجمله‌هایی مانندِ «I saw she» یا «I sees her» تولید می‌کند. برای اجتناب از این مسئله، دستورِ ما باید بداند «her»، نه «she»، مفعولِ معتبرِ «saw» (یا هر فعلِ دیگری) است و «see»، نه «sees»، شکلِ فعلِ همراهِ فاعلِ «I» است. می‌توانستیم این واقعیت‌ها را کاملاً در مدخل‌های احتمال رمز کنیم، مثلاً $P_1(v,she)$ را برای همهٔ فعل‌های v عددِ بسیار کوچکی کنیم. اما موجزتر و پودمانی‌تر است ردهٔ NP را با متغیرهای اضافی افزوده کنیم: NP(c,pn,n) برای بازنماییِ گروهِ اسمی با حالتِ c (فاعلی یا مفعولی)، شخص و شمارِ pn (مثلاً سومِ شخصِ مفرد) و اسمِ هستهٔ n استفاده می‌شود. شکل ۲۳.۹ دستورِ واژگانی‌شدهٔ افزوده‌شده‌ای نشان می‌دهد که این متغیرهای اضافی را مدیریت می‌کند.

$$
\begin{aligned}
S(v) &\rightarrow NP(Sbj, pn, n)\ VP(pn, v) \mid \ldots \\
NP(c, pn, n) &\rightarrow Pronoun(c, pn, n) \mid Noun(c, pn, n) \mid \ldots \\
VP(pn, v) &\rightarrow Verb(pn, v)\ NP(Obj, pn, n) \mid \ldots \\
PP(head) &\rightarrow Prep(head)\ NP(Obj, pn, h) \\
Pronoun(Sbj, 1S, \mathbf{I}) &\rightarrow \mathbf{I} \\
Pronoun(Sbj, 1P, \mathbf{we}) &\rightarrow \mathbf{we} \\
Pronoun(Obj, 1S, \mathbf{me}) &\rightarrow \mathbf{me} \\
Pronoun(Obj, 3P, \mathbf{them}) &\rightarrow \mathbf{them} \\
\\
Verb(3S, \mathbf{see}) &\rightarrow \mathbf{see}
\end{aligned}
$$

شکل ۲۳.۹ – بخشی از دستورِ افزوده‌شده که توافقِ حالت، توافقِ فاعل-فعل و واژه‌های هسته را مدیریت می‌کند. نام‌های با حرفِ بزرگ ثابت‌اند: Sbj و Obj برای حالتِ فاعلی و مفعولی؛ 1S برای شخصِ اولِ مفرد؛ 1P و 3P برای شخصِ اول و سومِ جمع. مثلِ همیشه، نام‌های با حرفِ کوچک متغیرند. برای سادگی، احتمال‌ها حذف شده‌اند.

بیایید یکی از قاعده‌های دستور را با جزئیات بررسی کنیم:

```
S(v) → NP(Sbj, pn, n) VP(pn, v)    [P₅(n, v)]
```

این قاعده می‌گوید وقتی NP با VP دنبال می‌شود می‌توانند S تشکیل دهند، اما فقط اگر NP حالتِ فاعلی (Sbj) داشته باشد و شخص و شمارِ (pn) NP و VP یکسان باشند. (می‌گوییم در توافق‌اند.) اگر برقرار باشد، Sای داریم که هسته‌اش فعلِ VP است. در این‌جا قاعدهٔ واژگانیِ نمونه‌ای است:

Pronoun(Sbj,1S,I) → I [0.005]

که می‌گوید «I» ضمیری در حالتِ فاعلی، شخصِ اولِ مفرد، با هستهٔ «I» است.

۲۳.۴.۱ تفسیرِ معنایی

برای نشان‌دادنِ اینکه چگونه معنا را به دستور بیفزاییم، با مثالی شروع می‌کنیم که از انگلیسی ساده‌تر است: معناشناسیِ عبارت‌های حسابی. شکل ۲۳.۱۰ دستوری برای عبارت‌های حسابی نشان می‌دهد، که هر قاعده با آرگومانِ تکیِ نشان‌دهندهٔ تفسیرِ معناییِ عبارت افزوده شده.

$$
\begin{aligned}
&Exp(op(x_1,x_2)) \rightarrow Exp(x_1)\ Operator(op)\ Exp(x_2)\\
&Exp(x) \rightarrow (\ Exp(x)\ )\\
&Exp(x) \rightarrow Number(x)\\
&Number(x) \rightarrow Digit(x)\\
&Number(10\times x_1 + x_2) \rightarrow Number(x_1)\ Digit(x_2)\\
&Operator(+) \rightarrow \mathbf{+}\\
&Operator(-) \rightarrow \mathbf{-}\\
&Operator(\times) \rightarrow \times\\
&Operator(\div) \rightarrow \div\\
&Digit(0) \rightarrow \mathbf{0}\\
&Digit(1) \rightarrow \mathbf{1}\\
&\ldots
\end{aligned}
$$

شکل ۲۳.۱۰ — دستوری برای عبارت‌های حسابی، افزوده‌شده با معناشناسی. هر متغیرِ x_i معناشناسیِ یک سازه را بازنمایی می‌کند.

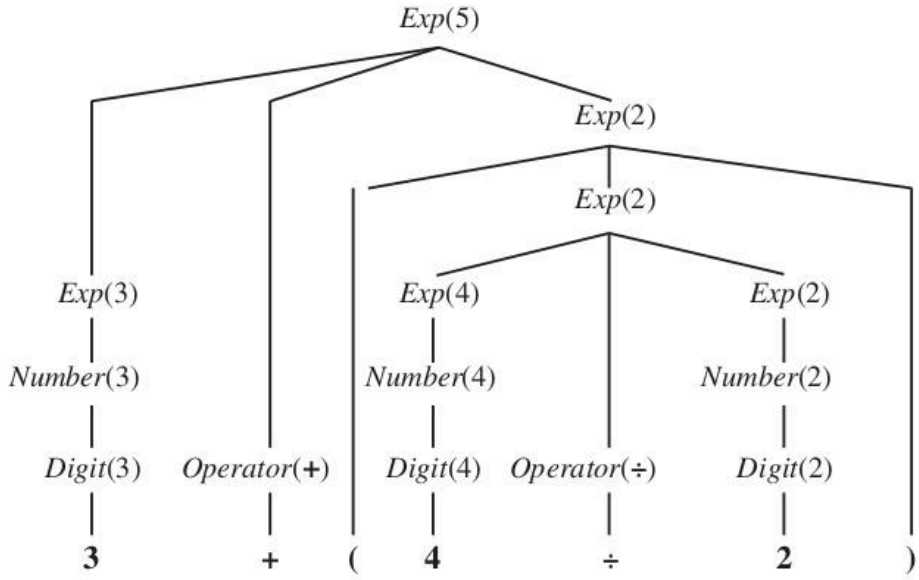
معناشناسیِ رقمی مانندِ «۳» خودِ رقم است. معناشناسیِ عبارتِ «۳ + ۴» عملگرِ «+» است که بر معناشناسیِ عبارت‌های «۳» و «۴» اعمال شده. قاعده‌های دستور از «اصلِ معناشناسیِ ترکیبی» (Compositional Semantics) پیروی می‌کنند—معناشناسیِ یک عبارت تابعی از معناشناسیِ زیرعبارت‌هاست. شکل ۲۳.۱۱ درختِ تجزیه را برای 3+(4÷2) طبقِ این دستور نشان می‌دهد. ریشهٔ درختِ تجزیه Exp(5) است، عبارتی که تفسیرِ معنایی‌اش ۵ است.

حالا به معناشناسیِ انگلیسی برویم، یا دست‌کم بخشِ کوچکی از آن. از منطقِ مرتبهٔ اول برای بازنماییِ معنایی‌مان استفاده خواهیم کرد. پس جملهٔ سادهٔ «Ali loves Bo» باید بازنماییِ معناییِ Loves(Ali,Bo) را بگیرد. اما دربارهٔ عبارت‌های سازه چه؟ می‌توانیم «NP_Ali» را با عبارتِ منطقیِ Ali بازنمایی کنیم. اما «VP_loves Bo» نه عبارتِ منطقی است و نه جملهٔ منطقیِ کامل. به‌صورتِ شهودی، «loves Bo» توصیفی است که ممکن است برای شخصِ خاصی صادق باشد یا نباشد. (در این مورد، برای Ali صادق است.) این بدان‌معناست «loves Bo» محمولی است که وقتی با عبارتی که شخصی را بازنمایی می‌کند ترکیب شود، جملهٔ منطقیِ کاملی به دست می‌دهد. با استفاده از نمادِ λ (صفحهٔ ۲۵۹ را ببینید)، می‌توانیم «loves Bo» را به‌عنوانِ محمولِ λx Loves(x,Bo) بازنمایی کنیم. حالا به قاعده‌ای نیاز داریم که بگوید «NP با معناشناسیِ n و به‌دنبالِ آن VP با معناشناسیِ pred جمله‌ای به دست می‌دهد که معناشناسی‌اش نتیجهٔ اعمالِ pred بر n است»:

```
S(pred(n)) → NP(n) VP(pred)
```

قاعده به ما می‌گوید تفسیرِ معناییِ «Ali loves Bo» برابرِ (λx Loves(x,Bo))(Ali) است، که با Loves(Ali,Bo) هم‌ارز است. از نظرِ فنی می‌گوییم این «کاهشِ بتا» (β-Reduction) کاربردِ تابعِ لاندا است. بقیهٔ معناشناسی به‌شیوه‌ای سرراست از انتخاب‌هایی که تاکنون کرده‌ایم پیروی می‌کند. چون VPها به‌عنوانِ محمول بازنمایی می‌شوند، فعل‌ها نیز باید محمول باشند. فعلِ «loves» به‌صورتِ λy λx Loves(x,y) بازنمایی می‌شود، محمولی که وقتی آرگومانِ Bo داده

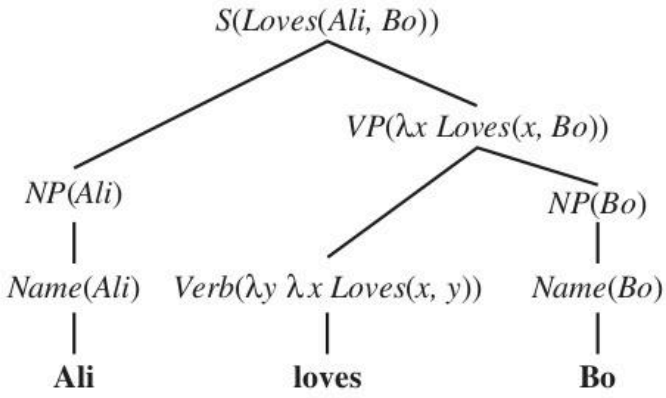
شود، محمولِ λx Loves(x,Bo) را برمی‌گرداند. در نهایت به دستور و درختِ تجزیهٔ نشان‌داده‌شده در شکل ۲۳.۱۲ می‌رسیم. در دستورِ کامل‌تری، همهٔ افزوده‌ها (معناشناسی، حالت، شخص-شمار و هسته) را در یک مجموعه قاعده قرار می‌دادیم. در این‌جا فقط افزودهٔ معنایی را نشان می‌دهیم تا واضح‌تر شود قاعده‌ها چگونه کار می‌کنند.



شکل ۲۳.۱۱ – درختِ تجزیه با تفسیرهای معنایی برای رشتهٔ «3+(4 ÷ 2)».

$$S(pred(n)) \rightarrow NP(n)\ VP(pred)$$
$$VP(pred(n)) \rightarrow Verb(pred)\ NP(n)$$
$$NP(n) \rightarrow Name(n)$$
$$Name(Ali) \rightarrow \textbf{Ali}$$
$$Name(Bo) \rightarrow \textbf{Bo}$$
$$Verb(\lambda y\ \lambda x\ Loves(x, y)) \rightarrow \textbf{loves}$$

(a)



(b)

شکل ۲۳.۱۲ – (الف) دستوری که می‌تواند درختِ تجزیه و تفسیرِ معنایی را برای «Ali loves Bo» (و سه جملهٔ دیگر) اشتقاق دهد. هر رده با آرگومانِ تکیِ بازنمایی‌کنندهٔ معناشناسی افزوده شده. (ب) درختِ تجزیه با تفسیرهای معنایی برای رشتهٔ «Ali loves Bo».

۲۳.۴.۲ یادگیریِ دستورهای معنایی

متأسفانه، درخت‌بانِ پن بازنمایی‌های معناییِ جمله‌هایش را شامل نمی‌شود، فقط درخت‌های نحوی را. پس اگر بخواهیم دستورِ معنایی یاد بگیریم، به منبعِ متفاوتی از مثال‌ها نیاز خواهیم داشت. زتلمایر و کالینز (2005) سامانه‌ای توصیف می‌کنند که دستوری برای سامانهٔ پاسخ‌گوییِ پرسش از مثال‌هایی یاد می‌گیرد که شاملِ جمله‌ای همراه با شکلِ معناییِ جمله‌اند:

- جمله: What states border Texas? («چه ایالت‌هایی با تگزاس مرز دارند؟»)
- شکلِ منطقی: λx.state(x) ∧ λx.borders(x,Texas)

با توجه به مجموعهٔ بزرگی از این‌گونه جفت‌ها و کمی دانشِ دست‌رمز برای هر حوزهٔ تازه، سامانه مدخل‌های واژگانیِ قابلِ‌قبول تولید می‌کند (برای نمونه، اینکه «Texas» و «state» اسم‌هایی‌اند چنان‌که state(Texas) صادق است)، و به‌طورِ هم‌زمان پارامترهایی برای دستوری یاد می‌گیرد که به سامانه اجازه می‌دهد جمله‌ها را به بازنمایی‌های معنایی تجزیه کند. سامانهٔ زتلمایر و کالینز به دقتِ ۷۹٪ رویِ دو مجموعهٔ آزمونِ متفاوت از جمله‌های دیده‌نشده رسید. ژائو و هوانگ (2015) تجزیه‌گرِ انتقال-کاهشی را نشان می‌دهند که سریع‌تر اجرا می‌شود و به دقتِ ۸۵٪ تا ۸۹٪ می‌رسد.

محدودیتِ این سامانه‌ها این است که دادهٔ آموزشی شاملِ شکل‌های منطقی است. ساختِ آن‌ها پرهزینه است و به حاشیه‌نویسانِ انسانی با تخصصِ ویژه نیاز دارد—همه ظرافت‌های حسابِ لاندا و منطقِ محمول‌ها را نمی‌فهمند. گردآوریِ مثال‌های جفتِ پرسش/پاسخ بسیار آسان‌تر است:

- پرسش: چه ایالت‌هایی با تگزاس مرز دارند؟
- پاسخ: لوئیزیانا، آرکانزاس، اکلاهما، نیومکزیکو.
- پرسش: رودآیلند چند بار در کالیفرنیا جا می‌شود؟
- پاسخ: ۱۳۵

چنین جفت‌های پرسش/پاسخ در وب کاملاً رایج‌اند، پس می‌توان پایگاهِ دادهٔ بزرگی را بدونِ کارشناسانِ انسانی کنار هم گذاشت. با استفاده از این منبعِ بزرگِ داده می‌توان تجزیه‌گرهایی ساخت که از تجزیه‌گرهای استفاده‌کننده از پایگاهِ دادهٔ کوچکِ شکل‌های منطقیِ حاشیه‌نویسی‌شده بهتر عمل کنند (لیانگ و همکاران، 2011؛ لیانگ و پاتس، 2015). رویکردِ کلیدیِ توصیف‌شده در این مقاله‌ها ابداعِ شکلِ منطقیِ درونیِ ترکیبی است که فضایِ جستجویِ نماییِ بزرگ را مجاز نمی‌کند.

۲۳.۵ پیچیدگی‌های زبانِ طبیعیِ واقعی

دستورِ انگلیسیِ واقعی بی‌پایان پیچیده است (و زبان‌های دیگر به‌یکسان پیچیده‌اند). به‌طورِ مختصر به برخی از موضوعاتی اشاره می‌کنیم که به این پیچیدگی کمک می‌کنند.

کمیت‌گذاری (Quantification): جملهٔ «Every agent feels a breeze» را در نظر بگیر. جمله تحتِ E_0 فقط یک تجزیهٔ نحوی دارد، اما از نظرِ معنایی مبهم است: آیا یک نسیم هست که همهٔ عامل‌ها آن را احساس می‌کنند، یا هر عامل نسیمِ شخصیِ جداگانه‌ای احساس می‌کند؟ دو تفسیر را می‌توان به‌صورتِ زیر بازنمایی کرد:

```
∀a a ∈ Agents ⇒ ∃b b ∈ Breezes ∧ Feel(a, b)
∃b b ∈ Breezes ∧ ∀a a ∈ Agents ⇒ Feel(a, b)
```

یک رویکردِ استاندارد به کمیت‌گذاری این است که دستور نه جملهٔ منطقیِ معناییِ واقعی، بلکه «شکلِ شبه‌منطقی» (Quasi-Logical Form) تعریف کند که سپس توسط الگوریتم‌های خارج از فرایندِ تجزیه به جملهٔ منطقی تبدیل می‌شود. آن الگوریتم‌ها می‌توانند قاعده‌های ترجیحی برای انتخابِ یک دامنهٔ سور بر دیگری داشته باشند—ترجیح‌هایی که لازم نیست مستقیماً در دستور بازتاب یابند.

کاربردشناسی (Pragmatics): نشان داده‌ایم عامل چگونه می‌تواند رشته‌ای از واژه‌ها را ادراک کند و با دستور مجموعه‌ای از تفسیرهای معناییِ ممکن اشتقاق دهد. حالا به مسئلهٔ تکمیلِ تفسیر با افزودنِ اطلاعاتِ بافت‌وابسته دربارهٔ موقعیتِ کنونی می‌پردازیم. آشکارترین نیاز به اطلاعاتِ کاربردشناختی در رفعِ معنایِ «واژه‌های اشاره‌گر» (Indexical) است، عبارت‌هایی که مستقیماً به موقعیتِ کنونی ارجاع می‌کنند. برای نمونه، در جملهٔ «I am in Boston today»، هر دوِ «I» و «today» اشاره‌گرند. واژهٔ «I» با Speaker بازنمایی می‌شد، روانی (fluent) که در زمان‌های متفاوت به اشیای متفاوت ارجاع می‌کند، و این به عهدهٔ شنونده است که مرجعِ روانِ مذکور را رفع کند—این بخشی از دستور تلقی نمی‌شود، بلکه مسئله‌ای کاربردشناختی است.

بخشِ دیگرِ کاربردشناسی تفسیرِ قصدِ گوینده است. گفتهٔ گوینده «کنشِ گفتاری» (Speech Act) در نظر گرفته می‌شود، و این به عهدهٔ شنونده است که بفهمد چه نوع کنشی است—پرسش، گزاره، وعده، هشدار، فرمان و مانندِ آن. فرمانی مانندِ «go to 2 2» («به 2 2 برو») به‌صورتِ ضمنی به شنونده ارجاع می‌کند. تاکنون، دستورِ ما برای S فقط جمله‌های خبری را پوشش می‌دهد. می‌توانیم آن را برای پوششِ فرمان‌ها بسط دهیم—فرمان گروهِ فعلی است که فاعلش به‌صورتِ ضمنی شنوندهٔ فرمان است:

```
S(Command(pred(Hearer))) → VP(pred)
```

وابستگی‌های دوربرد (Long-Distance Dependencies): در شکل ۲۳.۸ دیدیم «she didn't hear or even see him» («او او را نشنید یا حتی ندید») با دو جایِ خالی تجزیه شد که در آن‌ها NPای مفقود است، اما به «him» «NP» ارجاع می‌کند. می‌توانیم از نماد برای بازنماییِ جای‌های خالی استفاده کنیم: «she didn't [hear or even see] him.» به‌طورِ کلی، فاصلهٔ بینِ جایِ خالی و NPای که به آن ارجاع می‌کند می‌تواند دلخواهاً بلند باشد: در «Who did the agent tell you to give the gold to ?» («عامل به تو گفت طلا را به چه کسی بدهی؟») جایِ خالی به «Who» ارجاع می‌کند، که ۱۱ واژه دور است. از سامانهٔ پیچیده‌ای از قاعده‌های افزوده می‌توان استفاده کرد تا مطمئن شویم NPهای مفقود به‌درستی به هم می‌آیند. قاعده‌ها پیچیده‌اند؛ برای نمونه، نمی‌توانید در یک شاخه از NP هم‌پایه جایِ خالی داشته باشید: «What did she play [NP Dungeons and]?» غیردستوری است. اما می‌توانید همان جایِ خالی را در هر دو شاخهٔ VP هم‌پایه داشته باشید، مانندِ جملهٔ «What did you [VP [VP smell] and [VP shoot an arrow at]]?»

زمان و وجه (Time and Tense): فرض کنید می‌خواهیم تفاوتِ بینِ «Ali loves Bo» و «Ali loved Bo» را بازنمایی کنیم. انگلیسی از وجه‌های فعل (گذشته، حال و آینده) برای نشان‌دادنِ زمانِ نسبیِ رویداد استفاده می‌کند. انتخابِ خوب برای بازنماییِ زمانِ رویدادها نمادگذاریِ «حسابِ رویدادها» (Event Calculus) از بخش ۱۰.۳ است. در حسابِ رویدادها داریم:

Ali loves Bo: $E_1 \in Loves(Ali,Bo) \wedge During(Now,Extent(E_1))$ Ali loved Bo: $E_2 \in Loves(Ali,Bo) \wedge After(Now,Extent(E_2))$.

این پیشنهاد می‌کند دو قاعدهٔ واژگانیِ ما برای واژه‌های «loves» و «loved» باید این‌ها باشند:

```
Verb(λy λx e ∈ Loves(x, y) ∧ During(Now, e)) → loves
Verb(λy λx e ∈ Loves(x, y) ∧ After(Now, e))  → loved
```

به‌جزِ این تغییر، بقیهٔ دستور همان می‌ماند، که خبرِ امیدوارکننده‌ای است؛ پیشنهاد می‌کند در مسیرِ درستی هستیم اگر بتوانیم به‌راحتی پیچیدگی‌ای مانندِ وجهِ فعل‌ها را بیفزاییم (هرچند فقط سطحِ یک دستورِ کاملِ زمان و وجه را خراشیده‌ایم).

ابهام (Ambiguity): تمایل داریم ابهام را شکستِ ارتباط بدانیم؛ وقتی شنونده آگاهانه از ابهامی در گفته آگاه می‌شود، یعنی گفته نامفهوم یا گیج‌کننده است. در این‌جا چند مثال برگرفته از تیترهای روزنامه آورده شده:

Squad helps dog bite victim. (گروه به گازگرفتنِ سگِ قربانی کمک کرد / گروه به قربانیِ گازگرفتگیِ سگ کمک کرد.) Police begin campaign to run down jaywalkers. (پلیس کمپینِ زیرگرفتنِ پیاده‌روهای خاطی را آغاز کرد.) Helicopter powered by human flies. (هلیکوپتری که با انسان کار می‌کند پرواز می‌کند / انسانِ مگسِ نیرودهندهٔ هلیکوپتر.) Once-sagging cloth diaper industry saved by full dumps. (صنعتِ پوشکِ پارچه‌ایِ افت‌کرده با پوشکِ کامل نجات یافت.) Include your children when baking cookies. (وقتی کلوچه می‌پزید بچه‌هایتان را هم شامل کنید.) Portable toilet bombed; police have nothing to go on. (توالتِ سیار بمب‌گذاری شد؛ پلیس هیچ سرنخی ندارد.) Milk drinkers are turning to powder. (شیرخوران به پودر روی می‌آورند.) Two sisters reunited after 18 years in checkout counter. (دو خواهر پس از ۱۸ سال در صفِ صندوقِ فروشگاه به هم رسیدند.)

چنین گیجی‌هایی استثنا هستند؛ بیشترِ اوقات زبانی که می‌شنویم بدونِ ابهام به نظر می‌رسد. پس، وقتی پژوهشگران در دههٔ ۱۹۶۰ برای نخستین بار شروع به استفاده از رایانه‌ها برای تحلیلِ زبان کردند، کاملاً شگفت‌زده شدند که یاد گرفتند تقریباً هر جمله‌ای مبهم است، با تجزیه‌های ممکنِ متعدد (گاهی صدها)، حتی وقتی تنها تجزیهٔ ترجیحی همانی است که گویندگانِ بومی فقط آن را متوجه می‌شوند. برای نمونه، عبارتِ «brown rice and black beans» را به‌صورتِ «[brown rice] [and black beans]» می‌فهمیم، و هرگز تفسیرِ کم‌احتمالِ «brown [rice and black beans]» را در نظر نمی‌گیریم، که در آن صفتِ «brown» کلِ عبارت را توصیف می‌کند، نه فقط «rice» را. وقتی می‌شنویم «Outside of a dog, a book is a person's best friend» («به‌جز سگ، کتاب بهترین دوستِ انسان است»)، «outside of» را به معنای «به‌جز» تفسیر می‌کنیم، و وقتی جملهٔ بعدیِ شوخیِ گروچو مارکس «Inside of a dog it's too dark to read» («داخلِ سگ برای خواندن خیلی تاریک است») است خنده‌دار می‌یابیم.

«ابهامِ واژگانی» (Lexical Ambiguity) وقتی است که واژه‌ای بیش از یک معنا دارد: «back» می‌تواند قید (go back)، صفت (back door)، اسم (the back of the room)، فعل (back a candidate) یا اسمِ خاص (رودخانه‌ای در نوناووتِ کانادا) باشد. «Jack» می‌تواند اسمِ خاص، اسم (ورقِ بازی، مهرهٔ فلزیِ شش‌پرِ بازی، پرچمِ دریایی، ماهی، پرنده، پنیر، سوکت و مانندِ آن) یا فعل (جک‌زدنِ ماشین، شکار با نور، یا محکم‌زدنِ توپِ بیسبال) باشد.

«ابهامِ نحوی» (Syntactic Ambiguity) به عبارتی اشاره دارد که تجزیه‌های متعدد دارد: «I smelled a wumpus in 2,2» دو تجزیه دارد: یکی که در آن گروهِ حرفِ اضافهٔ «in 2,2» اسم را توصیف می‌کند و یکی که در آن فعل را توصیف می‌کند. ابهامِ نحوی به «ابهامِ معنایی» (Semantic Ambiguity) می‌انجامد، زیرا یک تجزیه به این معناست که وومپوس در 2,2 است و دیگری به این معناست که بوی تعفن در 2,2 است. در این مورد، گرفتنِ تفسیرِ اشتباه می‌تواند اشتباهِ مرگبار باشد.

همچنین می‌تواند ابهام بینِ معانیِ تحت‌اللفظی و مجازی باشد. «صنایعِ بدیعی» (Figures of Speech) در شعر مهم‌اند و در گفتارِ روزمره نیز رایج‌اند. «مجاز» (Metonymy) صنعتِ بدیعی است که در آن شیئی برای نشان‌دادنِ شیءِ دیگر استفاده می‌شود. وقتی می‌شنویم «Chrysler announced a new model» («کرایسلر مدلِ تازه‌ای اعلام کرد»)، آن را به‌معنایِ اینکه شرکت‌ها می‌توانند حرف بزنند تفسیر نمی‌کنیم؛ بلکه می‌فهمیم سخنگویی از شرکت اعلام کرده. مجاز رایج است و اغلب به‌صورتِ ناخودآگاه توسط شنوندگانِ انسانی تفسیر می‌شود.

متأسفانه، دستورِ ما آن‌گونه که نوشته شده چندان چابک نیست. برای مدیریتِ درستِ معناشناسیِ مجاز، باید سطحِ کاملاً تازه‌ای از ابهام معرفی کنیم. می‌توانیم این کار را با فراهم‌کردنِ دو شیء برای تفسیرِ معناییِ هر عبارتِ جمله انجام دهیم: یکی برای شیئی که عبارت به‌صورتِ تحت‌اللفظی به آن ارجاع می‌کند (Chrysler) و یکی برای ارجاعِ مجازی (سخنگو). سپس باید بگوییم رابطه‌ای بینِ آن دو هست. در دستورِ کنونیِ ما، «Chrysler announced» به‌صورتِ زیر تفسیر می‌شود:

```
x = Chrysler ∧ e ∈ Announce(x) ∧ After(Now, Extent(e))
```

باید آن را به این تغییر دهیم:

```
x = Chrysler ∧ e ∈ Announce(m) ∧ After(Now, Extent(e)) ∧ Metonymy(m, x)
```

این می‌گوید موجودیتی x هست که با Chrysler برابر است، و موجودیتِ دیگری m که اعلام را انجام داده، و اینکه این دو در رابطهٔ مجازاند. گامِ بعدی تعریفِ اینکه چه نوع رابطه‌های مجازی می‌توانند رخ دهند است. ساده‌ترین حالت وقتی است که اصلاً مجازی نباشد—شیءِ تحت‌اللفظیِ x و شیءِ مجازیِ m یکسان‌اند:

```
∀m,x (m = x) ⇒ Metonymy(m, x)
```

برای مثالِ Chrysler، تعمیمِ معقولی این است که سازمان می‌تواند برای نشان‌دادنِ سخنگویِ آن سازمان استفاده شود:

```
∀m,x x ∈ Organizations ∧ Spokesperson(m, x) ⇒ Metonymy(m, x)
```

مجازهای دیگر شاملِ نویسنده برای آثارش («I read Shakespeare» — شکسپیر خواندم) یا به‌طورِ کلی‌تر تولیدکننده برای محصول («I drive a Honda» — هوندا سوار می‌شوم) و جزء برای کل («The Red Sox need a strong arm» — ردساکس‌ها به یک بازوی قوی نیاز دارند) است. برخی مثال‌های مجاز، مانندِ «The ham sandwich on Table 4

wants another beer» («ساندویچِ ژامبونِ میز ۴ یک آبجو دیگر می‌خواهد»)، تازه‌ترند و با توجه به موقعیتی تفسیر می‌شوند (مانندِ پیشخدمتی که نامِ مشتری را نمی‌داند).

«استعاره» (Metaphor) صنعتِ بدیعیِ دیگری است، که در آن عبارتی با معنایِ تحت‌اللفظیِ واحد برای پیشنهادِ معنایِ متفاوتی از طریقِ قیاس استفاده می‌شود. پس استعاره را می‌توان نوعی مجاز دید که رابطه در آن از نوعِ شباهت است.

«ابهام‌زدایی» (Disambiguation) فرایندِ بازیابیِ محتمل‌ترین معنایِ موردنظرِ یک گفته است. از یک نظر، چارچوبی برای حلِ این مسئله از قبل داریم: هر قاعده با احتمالی مرتبط است، پس احتمالِ یک تفسیر حاصلضربِ احتمالِ قاعده‌هایی است که به تفسیر انجامیدند. متأسفانه، احتمال‌ها بازتابِ این‌اند که عبارت‌ها در پیکره‌ای که دستور از آن یاد گرفته چقدر رایج‌اند، و بنابراین دانشِ کلی را بازتاب می‌دهند، نه دانشِ خاصِ موقعیتِ کنونی را. برای ابهام‌زداییِ درست، باید چهار مدل را ترکیب کنیم:

1. **مدلِ جهان:** احتمالِ اینکه گزاره‌ای در جهان رخ دهد. با توجه به آنچه دربارهٔ جهان می‌دانیم، محتمل‌تر است گوینده‌ای که می‌گوید «I'm dead» («من مرده‌ام») منظورش «به دردسرِ بزرگی افتاده‌ام» یا «این بازیِ ویدئویی را باخته‌ام» باشد تا «زندگی‌ام پایان یافته، و با این حال هنوز می‌توانم حرف بزنم».
2. **مدلِ ذهنی:** احتمالِ اینکه گوینده قصدِ انتقالِ واقعیتِ خاصی به شنونده را تشکیل دهد. این رویکرد مدل‌هایی از آنچه گوینده باور دارد، آنچه گوینده باور دارد شنونده باور دارد، و مانندِ آن را ترکیب می‌کند. برای نمونه، وقتی سیاستمدار می‌گوید «من کلاهبردار نیستم»، مدلِ جهان ممکن است فقط ۵۰٪ احتمال به گزارهٔ اینکه سیاستمدار مجرم نیست تخصیص دهد، و 99.999٪ به گزارهٔ اینکه او عصای خمیدهٔ چوپانی نیست. با این حال، تفسیرِ نخست را انتخاب می‌کنیم، زیرا گفتنِ آن محتمل‌تر است.
3. **مدلِ زبان:** احتمالِ اینکه رشتهٔ خاصی از واژه‌ها انتخاب شود، با توجه به اینکه گوینده قصدِ انتقالِ واقعیتِ خاصی را دارد.
4. **مدلِ صوتی:** برای ارتباطِ گفتاری، احتمالِ اینکه دنبالهٔ خاصی از صداها تولید شود، با توجه به اینکه گوینده رشتهٔ واژه‌های داده‌شده‌ای را انتخاب کرده. (برای ارتباطِ دست‌نویس یا تایپ‌شده، مسئلهٔ بازشناسیِ نوریِ نویسه‌ها را داریم.)

۲۳.۶ تکلیف‌های زبانِ طبیعی

پردازش زبان طبیعی حوزهٔ بزرگی است، سزاوارِ یک یا دو کتابِ درسیِ کاملِ خودش (گلدبرگ، 2017؛ جورافسکی و مارتین، 2020). در این بخش به‌طورِ مختصر برخی از تکلیف‌های اصلی را توصیف می‌کنیم؛ می‌توانید از مرجع‌ها برای جزئیاتِ بیشتر استفاده کنید.

«بازشناسیِ گفتار» (Speech Recognition) تکلیفِ تبدیلِ صدایِ گفتاری به متن است. سپس می‌توانیم تکلیف‌های بیشتری (مانندِ پاسخ‌گوییِ پرسش) را رویِ متنِ حاصل انجام دهیم. سامانه‌های کنونی نرخِ خطایِ واژه‌ای حدودِ ۳٪ تا ۵٪ دارند (بسته به جزئیاتِ مجموعهٔ آزمون)، مشابهِ رونویس‌کنندگانِ انسانی. چالش برای سامانه‌ای که از بازشناسیِ گفتار استفاده می‌کند پاسخ‌دادنِ مناسب حتی وقتی خطاهایی رویِ واژه‌های منفرد هست است.

سامانه‌های برترِ امروز از ترکیبی از شبکه‌های عصبیِ بازگشتی و مدل‌های مارکوفِ پنهان استفاده می‌کنند (هینتون و همکاران، 2012؛ یو و دنگ، 2016؛ دنگ، 2016؛ چیو و همکاران، 2017؛ ژانگ و همکاران، 2017). معرفیِ شبکه‌های عصبیِ عمیق برای گفتار در ۲۰۱۱ به بهبودِ فوری و چشمگیرِ حدودِ ۳۰٪ در نرخِ خطا انجامید—این از حوزه‌ای بود که بالغ به نظر می‌رسید و قبلاً فقط چند درصد در سال پیشرفت می‌کرد. شبکه‌های عصبیِ عمیق تناسبِ خوبی‌اند، زیرا مسئلهٔ بازشناسیِ گفتار تجزیهٔ ترکیبیِ طبیعی دارد: از شکل‌موج‌ها به واج‌ها به واژه‌ها به جمله‌ها. آن‌ها در فصلِ بعد پوشش داده خواهند شد.

«سنتزِ متن‌به‌گفتار» (Text-to-Speech) فرایندِ معکوس است—رفتن از متن به صدا. تیلور (2009) نمایِ کلیِ کتاب‌حجمی می‌دهد. چالش تلفظِ درستِ هر واژه و طبیعی‌به‌نظررساندنِ جریانِ هر جمله، با مکث‌ها و تأکیدهای درست است. حوزهٔ دیگری از توسعه در سنتزِ صداهای متفاوت است—با انتخابی بینِ صدایِ مردانه یا زنانهٔ عمومی شروع می‌شود، سپس اجازهٔ گویش‌های منطقه‌ای، و حتی تقلیدِ صدایِ افرادِ مشهور. مانندِ بازشناسیِ گفتار، معرفیِ شبکه‌های عصبیِ بازگشتیِ عمیق به بهبودِ بزرگ انجامید، با حدودِ ۲/۳ شنوندگان که می‌گفتند سامانهٔ عصبیِ WaveNet (فن‌دن‌اورد و همکاران، 2016a) طبیعی‌تر از سامانهٔ غیرعصبیِ قبلی به نظر می‌رسد.

«ترجمهٔ ماشینی» (Machine Translation) متن را از زبانی به زبانِ دیگر تبدیل می‌کند. سامانه‌ها معمولاً با استفاده از «پیکرهٔ دوزبانه» آموزش می‌بینند: مجموعه‌ای از اسنادِ جفت‌شده، که یکی از اعضای جفت، مثلاً، انگلیسی و دیگری، مثلاً، فرانسوی است. اسناد نیازی به حاشیه‌نویسی به هیچ شکلی ندارند؛ سامانهٔ ترجمهٔ ماشینی یاد می‌گیرد جمله‌ها و عبارت‌ها را تراز کند و سپس وقتی جملهٔ تازه‌ای به یک زبان ارائه می‌شود، می‌تواند ترجمه‌ای به زبانِ دیگر تولید کند.

سامانه‌های اوایلِ دههٔ ۲۰۰۰ از مدل‌های n-گرم استفاده می‌کردند و به نتایجی می‌رسیدند که معمولاً برای رساندنِ معنایِ متن به‌اندازهٔ کافی خوب بود، اما در بیشترِ جمله‌ها خطاهای نحوی داشت. یک مشکل حدِّ طولِ n-گرم‌ها بود: حتی با حدِّ بزرگِ ۷، انتقالِ اطلاعات از یک سرِ جمله به سرِ دیگر دشوار بود. مشکلِ دیگر این بود که همهٔ اطلاعات در مدلِ n-گرم در سطحِ واژه‌های منفرد است. چنین سامانه‌ای می‌توانست یاد بگیرد «black cat» («گربهٔ سیاه») به «chat noir» ترجمه می‌شود، اما نمی‌توانست قاعده‌ای را یاد بگیرد که صفت‌ها عموماً در انگلیسی پیش از اسم و در فرانسوی پس از اسم می‌آیند.

مدل‌های دنباله‌به‌دنبالهٔ عصبیِ بازگشتی (سوتسکِور و همکاران، 2015) از مشکل عبور کردند. آن‌ها بهتر تعمیم می‌دادند (زیرا می‌توانستند از جاسازیِ واژه به‌جای شمارشِ n-گرمِ واژه‌های خاص استفاده کنند) و می‌توانستند در سطوحِ گوناگونِ شبکهٔ عمیق مدل‌های ترکیبی تشکیل دهند تا اطلاعات را به‌طورِ مؤثر منتقل کنند. کارِ بعدی با استفاده از سازوکارِ تمرکزِ مدلِ ترنسفورمر (واسوانی و همکاران، 2018) عملکرد را بیشتر افزایش داد، و مدلِ هیبریدیِ دربردارندهٔ جنبه‌های هر دوِ این مدل‌ها حتی بهتر عمل می‌کند و در برخی جفت‌زبان‌ها به عملکردِ سطحِ انسانی نزدیک می‌شود (وو و همکاران، 2016b؛ چن و همکاران، 2018).

«استخراجِ اطلاعات» (Information Extraction) فرایندِ کسبِ دانش با سرزدن به متن و جستجویِ رخدادِ رده‌های خاصی از اشیا و رابطه‌های بینِ آن‌هاست. تکلیفِ معمول استخراجِ نمونه‌های آدرس از صفحاتِ وب است، با فیلدهای پایگاهِ داده برای خیابان، شهر، ایالت و کدِ پستی؛ یا نمونه‌های طوفان از گزارش‌های آب‌وهوا، با فیلدهایی برای دما، سرعتِ باد و بارش.

اگر متنِ منبع به‌خوبی ساخت‌یافته باشد (مثلاً به‌صورتِ جدول)، فنونِ ساده‌ای مانندِ عبارت‌های باقاعده می‌توانند اطلاعات را استخراج کنند (کافارلا و همکاران، 2008). اگر بخواهیم همهٔ واقعیت‌ها را استخراج کنیم، نه نوعِ خاصی (مانندِ گزارش‌های آب‌وهوا)، سخت‌تر می‌شود؛ بانکو و همکاران (2007) سامانهٔ TEXTRUNNER را توصیف می‌کنند که بر مجموعه‌ای باز و در حالِ گسترش از رابطه‌ها استخراج انجام می‌دهد. برای متنِ آزاد، فنون شاملِ مدل‌های مارکوفِ پنهان و سامانه‌های یادگیریِ قاعده‌محورند (چنان‌که در TEXTRUNNER و NELL (یادگیریِ زبانِ بی‌پایان) (میچل و همکاران، 2018) استفاده شده). سامانه‌های جدیدتر از شبکه‌های عصبیِ بازگشتی استفاده می‌کنند، با بهره‌گیری از انعطافِ جاسازیِ واژه. نمایِ کلی را می‌توانید در کومار (2017) بیابید.

«بازیابیِ اطلاعات» (Information Retrieval) تکلیفِ یافتنِ اسنادی است که برای پرس‌وجویِ داده‌شده مرتبط و مهم‌اند. موتورهای جستجویِ اینترنتی مانندِ گوگل و بایدو این تکلیف را میلیاردها بار در روز انجام می‌دهند. سه کتابِ درسیِ خوب در این موضوع: منینگ و همکاران (2008)، کرافت و همکاران (2010) و بائزا-یتس و ریبیرو-نتو (2011).

«پاسخ‌گوییِ پرسش» (Question Answering) تکلیفِ متفاوتی است، که در آن پرس‌وجو واقعاً پرسش است، مانندِ «چه کسی گاردِ ساحلیِ آمریکا را تأسیس کرد؟» و پاسخ فهرستِ رتبه‌بندی‌شده‌ای از اسناد نیست، بلکه پاسخِ واقعی است: «الکساندر همیلتون.» از دههٔ ۱۹۶۰ سامانه‌های پاسخ‌گوییِ پرسش بوده‌اند که بر تجزیهٔ نحویِ بحث‌شده در این فصل تکیه دارند، اما فقط از ۲۰۰۱ چنین سامانه‌هایی از بازیابیِ اطلاعاتِ وب برای افزایشِ بنیادینِ گسترهٔ پوششان استفاده کرده‌اند. کاتز (1997) تجزیه‌گر و پاسخ‌گوِ START را توصیف می‌کند. بانکو و همکاران (2002) ASKMSR را توصیف می‌کنند، که از نظرِ توانایی تجزیهٔ نحوی کمتر پیچیده بود، اما در استفاده از جستجویِ وب و غربال‌کردنِ نتایج تهاجمی‌تر بود. برای نمونه، برای پاسخ به «چه کسی گاردِ ساحلیِ آمریکا را تأسیس کرد؟»، برای پرس‌وجوهایی مانندِ [* founded the U.S. Coast Guard] و [* the U.S. Coast Guard was founded by] (هر دو به معنای «چه کسی گاردِ ساحلیِ آمریکا را تأسیس کرد؟») جستجو می‌کرد، و سپس چندین صفحهٔ وبِ حاصل را بررسی می‌کرد تا پاسخِ محتملی را برگیرد، با دانستنِ اینکه واژهٔ پرسشِ «who» پیشنهاد می‌کند پاسخ باید شخص باشد. کنفرانسِ بازیابیِ متن (TREC) پژوهشِ این موضوع را گرد می‌آورد و از ۱۹۹۱ سالانه رقابت‌هایی میزبانی کرده (آلن و همکاران، 2017). اخیراً مجموعه‌های آزمونِ دیگری دیده‌ایم، مانندِ مجموعهٔ آزمونِ ARC_AI2 از پرسش‌های علومِ پایه (کلارک و همکاران، 2018).

خلاصهٔ فصل

نکته‌های اصلیِ این فصل چنین‌اند:

- مدل‌های زبانِ احتمالاتیِ مبتنی بر n-گرم مقدارِ شگفت‌آوری از اطلاعات دربارهٔ یک زبان بازیابی می‌کنند. آن‌ها می‌توانند در تکلیف‌های متنوعی مانندِ شناساییِ زبان، تصحیحِ املا، تحلیلِ احساسات، رده‌بندیِ سبک و بازشناسیِ موجودیتِ نام‌دار خوب عمل کنند.
- این مدل‌های زبان می‌توانند میلیون‌ها ویژگی داشته باشند، پس پیش‌پردازش و هموارسازیِ داده برای کاهشِ نویز مهم است.
- در ساختنِ سامانهٔ زبانیِ آماری، بهتر است مدلی ابداع کنیم که بتواند از دادهٔ در دسترس استفادهٔ خوبی کند، حتی اگر مدل بیش‌ازحد ساده‌لوحانه به نظر برسد.
- جاسازیِ واژه می‌تواند بازنماییِ غنی‌تری از واژه‌ها و شباهت‌هایشان بدهد.
- برای دربرگرفتنِ ساختارِ سلسله‌مراتبیِ زبان، دستورهای ساختارِ عبارتی (و به‌طورِ خاص، دستورهای مستقل از بافت) مفیدند. فرمالیسمِ دستورِ مستقل از بافتِ احتمالاتی (PCFG) به‌طورِ گسترده‌ای استفاده می‌شود، و همین‌طور فرمالیسمِ دستورِ وابستگی.
- جمله‌های زبانِ مستقل از بافت را می‌توان در زمانِ $O(n^3)$ با تجزیه‌گرِ نموداری مانندِ الگوریتمِ CYK تجزیه کرد، که مستلزمِ بودنِ قاعده‌های دستور در فرمِ نرمالِ چامسکی است. با ازدست‌دادنِ کمی دقت، زبان‌های طبیعی را می‌توان در زمانِ $O(n)$ تجزیه کرد، با جستجویِ پرتویی یا تجزیه‌گرِ انتقال-کاهش.
- درخت‌بان می‌تواند منبعی برای یادگیریِ دستورِ PCFG با پارامترها باشد.
- افزودنِ قابلیت به دستور برای مدیریتِ مسائلی مانندِ توافقِ فاعل-فعل و حالتِ ضمیر، و برای بازنماییِ اطلاعات در سطحِ واژه‌ها به‌جای فقط سطحِ رده‌ها راحت است.
- تفسیرِ معنایی را نیز می‌توان با دستورِ افزوده‌شده مدیریت کرد. می‌توانیم دستورِ معنایی را از پیکره‌ای از پرسش‌ها یاد بگیریم که یا با شکلِ منطقیِ پرسش یا با پاسخ جفت شده‌اند.
- زبانِ طبیعی پیچیده است و دربرگرفتنش در دستورِ صوری دشوار است.

یادداشت‌های کتابشناختی و تاریخی

مدل‌های n-گرمیِ حرفی برای مدل‌سازیِ زبان توسط مارکوف (1913) پیشنهاد شدند. کلود شانون (شانون و ویور، 1949) نخستین کسی بود که مدل‌های n-گرمیِ واژه‌ایِ انگلیسی تولید کرد. مدلِ کیسهٔ واژه‌ها نامش را از قطعه‌ای از زبان‌شناسِ زلیگ هریس (1954) می‌گیرد: «زبان نه صرفاً کیسه‌ای از واژه‌ها، بلکه ابزاری با ویژگی‌های خاص است.» نورویگ (2009) مثال‌هایی از تکلیف‌هایی می‌دهد که با مدل‌های n-گرم قابلِ انجام‌اند.

چامسکی (1956, 1957) به محدودیت‌های مدل‌های حالت-متناهی در مقایسه با مدل‌های مستقل از بافت اشاره کرد و نتیجه گرفت: «مدل‌های احتمالاتی بینشِ خاصی به برخی از مسائلِ بنیادینِ ساختارِ نحوی نمی‌دهند.» این درست است، اما مدل‌های احتمالاتی بینش به برخی مسائلِ بنیادینِ دیگر می‌دهند—مسائلی که مدل‌های مستقل از بافت نادیده می‌گیرند. اظهارنظرهای چامسکی اثرِ ناگواری داشت: بسیاری را برای دو دهه از مدل‌های آماری ترساند، تا این‌که این مدل‌ها برای استفاده در حوزهٔ بازشناسیِ گفتار (یلینک، 1976) و در علومِ شناختی دوباره ظهور کردند، جایی که نظریهٔ بهینگی (اسمولنسکی و پرینس، 1993؛ کاگر، 1999) فرض کرد زبان با یافتنِ محتمل‌ترین نامزدی کار می‌کند که قیدهای رقیب را به‌صورتِ بهینه ارضا می‌کند.

هموارسازیِ جمع-یک، که نخستین بار توسط پیر-سیمون لاپلاس (1816) پیشنهاد شد، توسط جفریس (1948) صوری شد. فنونِ هموارسازیِ دیگر شاملِ هموارسازیِ درون‌یابی (یلینک و مرسر، 1980)، هموارسازیِ ویتن–بل (1991)، هموارسازیِ گود–تورینگ (چرچ و گیل، 1991)، هموارسازیِ کنزر–نی (1995, 2004) و پس‌رویِ احمقانه (برانتس و همکاران، 2007) است. چن و گودمن (1996) و گودمن (2001) فنونِ هموارسازی را مرور می‌کنند.

مدل‌های سادهٔ n-گرمیِ حرفی و واژه‌ای تنها مدل‌های احتمالاتیِ ممکن نیستند. مدلِ تخصیصِ دیریکلهٔ نهان (بلی و همکاران، 2002؛ هافمن و همکاران، 2011) مدلِ متنیِ احتمالاتی است که سند را مخلوطی از موضوعات می‌بیند، که هر یک توزیعِ واژهٔ خودش را دارد. این مدل را می‌توان بسط و عقلانی‌سازیِ مدلِ فهرست‌بندیِ معناییِ نهانِ دیروستر و همکاران (1990) دید و همچنین با مدلِ مخلوطِ چندعلتی سحامی و همکاران (1996) مرتبط است. و البته علاقهٔ زیاد به مدل‌های زبانِ غیراحتمالاتی هست، مانندِ مدل‌های یادگیریِ عمیقِ پوشش‌داده‌شده در فصل ۲۴.

ژولن و همکاران (2016) کیسه‌ای از ترفندها برای رده‌بندیِ کارآمدِ متن می‌دهند. یوآخیمز (2001) از نظریهٔ یادگیریِ آماری و ماشین‌های بردارِ پشتیبان برای تحلیلِ نظریِ اینکه رده‌بندی چه‌وقت موفق خواهد بود استفاده می‌کند. آپته و همکاران (1994) دقتِ ۹۶٪ در رده‌بندیِ مقاله‌های خبریِ رویترز به ردهٔ «درآمد» گزارش می‌کنند. کولر و سحامی (1997) دقتِ تا ۹۵٪ با رده‌بندِ بیزِ ساده و تا 98.6٪ با رده‌بندِ بیزی گزارش می‌کنند. شاپایر و سینگر (2000) نشان می‌دهند رده‌بندهای خطیِ ساده اغلب می‌توانند دقتی تقریباً به‌خوبیِ مدل‌های پیچیده‌تر به‌دست آورند و سریع‌تر اجرا شوند. ژانگ و همکاران (2016) رده‌بندِ متنیِ سطح-نویسه (نه سطح-واژه) توصیف می‌کنند. ویتن و همکاران (1999) الگوریتم‌های فشرده‌سازی را برای رده‌بندی توصیف و پیوندِ عمیقِ بینِ الگوریتمِ فشرده‌سازیِ LZW و مدل‌های زبانِ حداکثر-آنتروپی را نشان می‌دهند.

وردنت (فل‌باوم، 2001) فرهنگِ لغتِ در دسترسِ عمومیِ حدودِ ۱۰۰,۰۰۰ واژه و عبارت است، که به نقش‌های دستوری رده‌بندی و با رابطه‌های معنایی مانندِ هم‌معنی، متضاد و جزء-از پیوند شده. چارنیاک (1996) و کلاین و منینگ (2001) تجزیه با دستورهای درخت‌بان را بحث می‌کنند. پیکرهٔ ملیِ بریتانیا (لیچ و همکاران، 2001) شاملِ ۱۰۰ میلیون واژه است، و وبِ جهان‌گستر شاملِ چند تریلیون واژه؛ فرانتس و برانتس (2006) پیکرهٔ n-گرمیِ گوگلِ در دسترسِ عمومیِ ۱۳ میلیون واژهٔ یکتا از یک تریلیون واژهٔ متنِ وب را توصیف می‌کنند. باک و همکاران (2014) مجموعهٔ دادهٔ مشابهی از پروژهٔ Common Crawl توصیف می‌کنند. درخت‌بانِ پن (مارکوس و همکاران، 1993؛ بیس و همکاران، 2015) درخت‌های تجزیه را برای پیکرهٔ سه‌میلیون‌واژه‌ایِ انگلیسی فراهم می‌کند.

بسیاری از فنونِ مدلِ n-گرم در مسائلِ بیوانفورماتیک نیز استفاده می‌شوند. آمارِ زیستی و پردازشِ زبانِ طبیعیِ احتمالاتی به هم نزدیک‌تر می‌شوند، زیرا هر دو با دنباله‌های بلند و ساخت‌یافته‌ای سروکار دارند که از الفبایی انتخاب می‌شوند.

برچسب‌گذارهای اولیهٔ نقشِ دستوری (POS) از انواعِ فنون استفاده می‌کردند، از جملهِ مجموعه‌های قاعده (بریل، 1992)، n-گرم‌ها (چرچ، 1988)، درخت‌های تصمیم (مارکز و رودریگز، 1998)، HMMها (برانتس، 2000) و رگرسیونِ لجستیک (راتناپارخی، 1996). از نظرِ تاریخی، مدلِ رگرسیونِ لجستیک «مدلِ مارکوفِ حداکثر-آنتروپی» یا MEMM نیز نامیده می‌شد، پس برخی کارها با آن نام است. جورافسکی و مارتین (2020) فصلِ خوبی دربارهٔ برچسب‌گذاریِ POS دارند. نگ و جردن (2002) مدل‌های متمایزگر و مولد را برای تکلیف‌های رده‌بندی مقایسه می‌کنند.

مانندِ شبکه‌های معنایی، دستورهای مستقل از بافت نخستین بار توسط دستورنویسانِ هندِ باستان (به‌ویژه پانینی، حدودِ ۳۵۰ پ.م.) که سانسکریتِ شاستریک مطالعه می‌کردند کشف شدند (اینگِرمَن، 1967). آن‌ها توسط نوام چامسکی (1956) برای تحلیلِ انگلیسی و به‌طورِ مستقل توسط جان بکوس (1959) و پیتر نائور برای تحلیلِ آلگول-۵۸ دوباره ابداع شدند.

دستورهای مستقل از بافتِ احتمالاتی نخستین بار توسط بوث (1969) و سالوما (1969) بررسی شدند. الگوریتم‌های PCFG در تک‌گفتارِ عالیِ کوتاهِ چارنیاک (1993) و کتاب‌های درسیِ عالیِ بلندِ منینگ و شوتزه (1999) و جورافسکی و مارتین (2020) ارائه شده‌اند. بیکر (1979) الگوریتمِ درون-بیرون را برای یادگیریِ PCFG معرفی می‌کند. PCFGهای واژگانی‌شده (چارنیاک، 1997؛ هوا، 1998) بهترین جنبه‌های PCFGها و مدل‌های n-گرم را ترکیب می‌کنند. کالینز (1999) تجزیهٔ ِPCFG واژگانی‌شده با ویژگی‌های هسته را توصیف می‌کند، و جانسون (1998) نشان می‌دهد دقتِ PCFG چگونه به ساختارِ درخت‌بانی بستگی دارد که احتمال‌هایش از آن یاد گرفته شده.

تلاش‌های بسیاری برای نوشتنِ دستورهای صوریِ زبان‌های طبیعی بوده، هم در زبان‌شناسیِ «خالص» و هم در زبان‌شناسیِ محاسباتی. چند دستورِ جامع اما غیررسمیِ انگلیسی هست (کوئیرک و همکاران، 1985؛ مک‌کاولی، 1988؛ هادلستون و پولوم، 2002). از دههٔ ۱۹۸۰، گرایشی به سمتِ واژگانی‌سازی بوده: گذاشتنِ اطلاعاتِ بیشتر در واژگان و کمتر در دستور.

«دستورِ واژگانی-نقشی» یا LFG (برس‌نان، 1982) نخستین فرمالیسمِ دستوریِ بزرگی بود که به‌شدت واژگانی‌شده بود. اگر واژگانی‌سازی را به حدِّ نهایی برسانیم، به «دستورِ رسته‌ای» (کلارک و کوران، 2004) می‌رسیم، که در آن می‌تواند فقط دو قاعدهٔ دستوری باشد، یا به «دستورِ وابستگی» (اسمیت و آیزنر، 2008؛ کوبلر و همکاران، 2009) که در آن هیچ ردهٔ نحوی نیست، فقط رابطه‌هایی بینِ واژه‌ها.

تجزیهٔ رایانه‌ای نخستین بار توسط ینگوه (1955) نشان داده شد. الگوریتم‌های کارآمد در دههٔ ۱۹۶۰ توسعه یافتند، با چند چرخش از آن زمان (کاسامی، 1965؛ یانگر، 1967؛ ارلی، 1970؛ گراهام و همکاران، 1980). چرچ و پتیل (1982) ابهامِ نحوی را توصیف و راه‌هایی برای رفعِ آن ارائه می‌کنند. کلاین و منینگ (2003) تجزیهٔ A* را توصیف می‌کنند، و پالز و کلاین (2009) آن را به تجزیهٔ A*یِ K-بهترین بسط می‌دهند، که در آن نتیجه تجزیهٔ تکی نیست بلکه K تجزیهٔ برتر است. گلدبرگ و همکاران (2013) ترفندهای پیاده‌سازیِ لازم را توصیف می‌کنند تا مطمئن شویم تجزیه‌گرِ جستجویِ

پرتویی O(n) است نه $O(n^2)$. ژو و همکاران (2013) تجزیه‌گرِ قطعیِ سریعِ انتقال-کاهش را برای زبان‌های طبیعی توصیف می‌کنند، و ساگای و لاوی (2006) نشان می‌دهند افزودنِ جستجو به تجزیه‌گرِ انتقال-کاهش چگونه می‌تواند آن را دقیق‌تر کند، به بهایِ کمی سرعت.

امروزه، تجزیه‌گرهای متن‌بازِ بسیار دقیق شاملِ Parsey McParseface گوگل (آندور و همکاران، 2016)، تجزیه‌گرِ استنفورد (چن و منینگ، 2014)، تجزیه‌گرِ برکلی (کیتایف و کلاین، 2018) و تجزیه‌گرِ SPACY هستند. همهٔ آن‌ها تعمیم را از طریقِ شبکه‌های عصبی انجام می‌دهند و به دقتِ تقریباً ۹۵٪ رویِ مجموعه‌های آزمونِ وال‌استریت‌ژورنال یا درخت‌بانِ پن می‌رسند. انتقادی به این حوزه هست که بیش‌ازحد بر اندازه‌گیریِ عملکرد رویِ چند پیکرهٔ انتخابی تمرکز می‌کند و شاید رویِ آن‌ها بیش‌برازش می‌کند.

تفسیرِ معناییِ صوریِ زبان‌های طبیعی در فلسفه و منطقِ صوری ریشه دارد، به‌ویژه کارِ آلفرد تارسکی (1935) بر معناشناسیِ زبان‌های صوری. بار-هیلل (1954) نخستین کسی بود که مسائلِ کاربردشناسی (مانندِ واژه‌های اشاره‌گر) را در نظر گرفت و پیشنهاد کرد می‌توان با منطقِ صوری آن‌ها را مدیریت کرد. مقالهٔ ریچارد مونتگیو «انگلیسی به‌عنوانِ زبانِ صوری» (1970) نوعی بیانیه برای تحلیلِ منطقیِ زبان است، اما کتاب‌های دیگری هست که خواندنی‌ترند (داوتی و همکاران، 1991؛ پورتنر و پارتی، 2002؛ کروز، 2011). در حالی که برنامه‌های تفسیرِ معنایی برای انتخابِ محتمل‌ترین تفسیر طراحی شده‌اند، منتقدانِ ادبی (امپسون، 1953؛ هابز، 1990) دربارهٔ اینکه آیا ابهام چیزی برای رفع است یا گرامی‌داشتن مبهم بوده‌اند. نورویگ (1988) مسائلِ درنظرگرفتنِ تفسیرهای هم‌زمانِ متعدد را بحث می‌کند، به‌جای اکتفا به تفسیرِ تکیِ حداکثر-درست‌نمایی. لیکاف و جانسون (1980) تحلیل و فهرستِ جذابی از استعاره‌های رایجِ انگلیسی می‌دهند. مارتین (1990) و گیبز (2006) مدل‌های محاسباتیِ تفسیرِ استعاره ارائه می‌دهند.

نخستین سامانهٔ NLP که تکلیفِ واقعی را حل کرد سامانهٔ پاسخ‌گوییِ پرسشِ BASEBALL بود (گرین و همکاران، 1961)، که به پرسش‌های مربوط به پایگاهِ دادهٔ آمارِ بیسبال پاسخ می‌داد. نزدیکِ آنِ SHRDLU وینوگراد (1972) بود، که به پرسش‌ها و فرمان‌های مربوط به صحنهٔ دنیایِ بلوک‌ها پاسخ می‌داد، وِ LUNAR وودز (1973)، که به پرسش‌های مربوط به سنگ‌های بازگردانده‌شده از ماه توسط برنامهٔ آپولو پاسخ می‌داد.

بانکو و همکاران (2002) سامانهٔ پاسخ‌گوییِ پرسشِ ASKMSR را ارائه می‌کنند؛ سامانهٔ مشابهی متعلق به کواک و همکاران (2001) است. پاشکا و هاراباجیو (2001) سامانهٔ پاسخ‌گوییِ پرسشِ برندهٔ رقابت را بحث می‌کنند. رویکردهای مدرنِ تفسیرِ معنایی معمولاً فرض می‌کنند نگاشت از نحو به معنا از مثال‌ها یاد گرفته می‌شود (زل و مونی، 1996؛ زتلمایر و کالینز، 2005؛ ژائو و هوانگ، 2015).

نخستین نتیجهٔ مهم در استقرایِ دستور نتیجهٔ منفی بود: گلد (1967) نشان داد یادگیریِ قابلِ‌اعتمادِ دستورِ مستقل از بافتِ دقیقاً درست، با توجه به مجموعه‌ای از رشته‌ها از آن دستور، ممکن نیست. زبان‌شناسانِ برجسته، مانندِ چامسکی (1957) و پینکر (2003)، از نتیجهٔ گلد برای استدلالِ اینکه باید «دستورِ همگانی» (Universal Grammar) فطری وجود داشته باشد که همهٔ کودکان از تولد دارند استفاده کرده‌اند. به‌اصطلاح استدلالِ «فقرِ محرک» می‌گوید کودکان ورودیِ کافی برای یادگیریِ دستورِ مستقل از بافت داده نمی‌شوند، پس باید از قبل دستور را «بدانند» و صرفاً برخی از پارامترهایش را کوک می‌کنند.

در حالی که این استدلال همچنان در بخشِ بزرگی از زبان‌شناسیِ چامسکی‌ای نفوذ دارد، توسط زبان‌شناسانِ دیگر (پولوم، 1996؛ المان و همکاران، 1997) و بیشترِ دانشمندانِ رایانه رد شده. در اوایلِ ۱۹۶۹، هورنینگ نشان داد یادگیریِ دستورِ مستقل از بافتِ احتمالاتی، به معنایِ یادگیریِ PAC، ممکن است. از آن زمان، نمایش‌های تجربیِ قانع‌کننده‌ی بسیاری از یادگیریِ زبان از مثال‌های مثبتِ تنها بوده، مانندِ یادگیریِ دستورهای معنایی با برنامه‌نویسیِ منطقیِ استقرایی (ماگلتون و دی ردت، 1994؛ مونی، 1999)، پایان‌نامه‌های دکتراِی شوتزه (1995) و دی مارکن (1996)، و کلِ خطِ سامانه‌های پردازشِ زبانِ مدرنِ مبتنی بر مدلِ ترنسفورمر (بخش ۲۴). هر سال کنفرانسِ بین‌المللیِ استنتاجِ دستوری (ICGI) برگزار می‌شود.

سامانهٔ DRAGON جیمز بیکر (1975) را می‌توان نخستین سامانهٔ موفقِ بازشناسیِ گفتار در نظر گرفت. نخستین سامانه‌ای بود که از HMMها برای گفتار استفاده کرد. پس از چند دهه از سامانه‌های مبتنی بر مدل‌های زبانِ احتمالاتی، حوزه شروع به تغییر به شبکه‌های عصبیِ عمیق کرد (هینتون و همکاران، 2012). دنگ (2016) توصیف می‌کند معرفیِ یادگیری عمیق چگونه بهبودِ سریعِ بازشناسیِ گفتار را ممکن کرد، و بر پیامدهایش برای دیگر تکلیف‌های NLP تأمل می‌کند. امروزه یادگیری عمیق رویکردِ غالب برای همهٔ سامانه‌های بازشناسیِ گفتارِ مقیاس‌بزرگ است. بازشناسیِ گفتار را می‌توان نخستین حوزهٔ کاربردی‌ای دید که موفقیتِ یادگیری عمیق را برجسته کرد، با بینایی رایانه‌ای که اندکی پس از آن آمد.

علاقه به حوزهٔ بازیابیِ اطلاعات با استفادهٔ گسترده از جستجویِ اینترنتی برانگیخته شد. کرافت و همکاران (2010) و منینگ و همکاران (2008) کتاب‌های درسی‌ای فراهم می‌کنند که اصول را پوشش می‌دهند. کنفرانسِ TREC هر سال رقابتی برای سامانه‌های IR میزبانی و مجموعه‌کارهایِ با نتایج منتشر می‌کند. برین و پیج (1998) الگوریتمِ PageRank را توصیف می‌کنند، که پیوندهای بینِ صفحه‌ها را در نظر می‌گیرد، و نمایِ کلی از پیاده‌سازیِ موتورِ جستجویِ وب می‌دهند. سیلورستاین و همکاران (1998) گزارشِ یک میلیارد جستجویِ وب را بررسی می‌کنند. مجلهٔ Information Retrieval و مجموعه‌کارهایِ کنفرانسِ سالانهٔ اصلیِ SIGIR توسعه‌های اخیرِ حوزه را پوشش می‌دهند.

استخراجِ اطلاعات توسط کنفرانس‌های سالانهٔ فهمِ پیام (MUC)، با حمایتِ دولتِ آمریکا، به جلو رانده شده. مرورهای سامانه‌های مبتنی بر قالب توسط روش و شابز (1997)، آپلت (1999) و موسلیا (1999) داده شده. پایگاه‌های دادهٔ بزرگِ واقعیت‌ها توسط کریون و همکاران (2000)، پاشکا و همکاران (2006)، میچل (2007) و دورمه و پاشکا (2008) استخراج شدند. فرایتاگ و مک‌کالوم (2000) HMMها را برای استخراجِ اطلاعات بحث می‌کنند. میدان‌های تصادفیِ شرطی نیز برای این تکلیف استفاده شده‌اند (لافرتی و همکاران، 2001؛ مک‌کالوم، 2003)؛ آموزشِ با راهنماییِ عملی توسط ساتن و مک‌کالوم (2007) داده شده. ساراواگی (2007) مرورِ جامعی می‌دهد.

دو رویکردِ اولیهٔ تأثیرگذار به مهندسیِ دانشِ خودکار برای NLP متعلق به ریلوف (1993) بود، که نشان داد فرهنگِ لغتِ ساخته‌شدهٔ خودکار تقریباً به‌خوبیِ فرهنگِ لغتِ خاصِ حوزهٔ دست‌ساختهٔ دقیق عمل می‌کند، و یاروفسکی (1995)، که نشان داد تکلیفِ رده‌بندیِ معنایِ واژه را می‌توان از طریقِ آموزشِ نظارت‌نشده رویِ پیکره‌ای از متنِ برچسب‌نخورده با دقتی به‌خوبیِ روش‌های نظارت‌شده انجام داد.

ایدهٔ استخراجِ هم‌زمانِ قالب‌ها و مثال‌ها از مشتی مثالِ برچسب‌خورده به‌طورِ مستقل و هم‌زمان توسط بلوم و میچل (1998) توسعه یافت، که آن را «آموزشِ هم‌زمان» (Cotraining) نامیدند، و توسط برین (1998)، که آن را DIPRE (استخراجِ الگویِ رابطهٔ تکراریِ دوگانه) نامید. می‌توانید ببینید چرا اصطلاحِ آموزشِ هم‌زمان مانده. کارِ اولیهٔ مشابهی، تحتِ نامِ «بوت‌استرپینگ»، توسط جونز و همکاران (1999) انجام شد. روش توسط سامانه‌های QXTRACT (آگیشتین و گراوانو، 2003) و KNOWITALL (اتزیونی و همکاران، 2005) پیش برده شد. «خواندنِ ماشینی» (Machine Reading) توسط میچل (2005) و اتزیونی و همکاران (2006) معرفی شد و کانونِ پروژهٔ TEXTRUNNER است (بانکو و همکاران، 2007؛ بانکو و اتزیونی، 2008).

این فصل بر جمله‌های زبانِ طبیعی تمرکز کرده، اما استخراجِ اطلاعات بر مبنای ساختارِ فیزیکی یا چیدمانِ هندسیِ متن نیز به‌جای ساختارِ زبانی ممکن است. فهرست‌ها، جدول‌ها، نمودارها، گراف‌ها، نگاره‌ها و مانندِ آن، چه در HTML رمز شده باشند چه از طریقِ تحلیلِ بصریِ اسنادِ pdf در دسترس باشند، خانهٔ داده‌هایی‌اند که می‌توان استخراج و یکپارچه کرد (هرست، 2000؛ پینتو و همکاران، 2003؛ کافارلا و همکاران، 2008).

کن چرچ (2004) نشان می‌دهد پژوهشِ زبانِ طبیعی بینِ تمرکز بر داده (تجربه‌گرایی) و تمرکز بر نظریه‌ها (عقل‌گرایی) در چرخه بوده؛ او مزیت‌های داشتنِ منابعِ زبانیِ خوب و طرح‌های ارزیابی را توصیف می‌کند، اما تعجب می‌کند آیا بیش از حد پیش رفته‌ایم (چرچ و هستنس، 2019). زبان‌شناسانِ اولیه بر داده‌های کاربردِ واقعیِ زبان، از جملهِ شمارشِ بسامد، تمرکز می‌کردند. نوام چامسکی (1956) محدودیت‌های مدل‌های حالت-متناهی را نشان داد، که به تأکید بر مطالعاتِ نظریِ نحو، با نادیده‌گرفتنِ عملکردِ واقعیِ زبان، انجامید. این رویکرد بیست سال حاکم بود، تا تجربه‌گرایی بر مبنای موفقیتِ کار در بازشناسیِ آماریِ گفتار (یلینک، 1976) بازگشت. امروزه، تأکید بر دادهٔ زبانیِ تجربی ادامه دارد، و علاقهٔ فزاینده‌ای به مدل‌هایی هست که سازه‌های سطح‌بالاتر، مانندِ رابطه‌های نحوی و معنایی، را در نظر می‌گیرند، نه فقط دنباله‌های واژه‌ها را. تأکیدِ شدیدی نیز بر مدل‌های شبکهٔ عصبیِ یادگیریِ عمیقِ زبان هست، که در فصل ۲۴ پوشش خواهیم داد.

کارِ کاربردهای پردازشِ زبان در کنفرانسِ دوسالانهٔ پردازشِ زبانِ طبیعیِ کاربردی (ANLP)، کنفرانسِ روش‌های تجربی در پردازشِ زبانِ طبیعی (EMNLP) و مجلهٔ مهندسیِ زبانِ طبیعی ارائه می‌شود. طیفِ گسترده‌ای از کارِ NLP در مجلهٔ زبان‌شناسیِ محاسباتی و کنفرانسِ آن، ACL، و در کنفرانسِ بین‌المللیِ زبان‌شناسیِ محاسباتی (COLING) ظاهر می‌شود. جورافسکی و مارتین (2020) مقدمهٔ جامعی بر گفتار و NLP می‌دهند.

پانوشت‌ها:

[1] و حتی کاربردهای بینایی رایانه: وردنت مجموعهٔ رده‌های استفاده‌شده توسط ImageNet را فراهم می‌کند.

[2] گاهی نویسندگان به ترتیبِ CKY اعتبار داده می‌شوند.

[3] حالتِ فاعلی گاهی «حالتِ نهادی» و حالتِ مفعولی گاهی «حالتِ مفعولیِ صریح» (Accusative) نیز نامیده می‌شود. بسیاری از زبان‌ها همچنین تمایزِ دیگری با «حالتِ اضافیِ» (Dative) برای واژه‌های در جایگاهِ مفعولِ غیرمستقیم قائل می‌شوند.

فصل ۲۴: یادگیری عمیق برای زبان طبیعی (Deep Learning for NLP)

در این فصل شبکه‌های عصبیِ عمیق انواعِ تکلیف‌های زبانی را انجام می‌دهند و ساختارِ زبانِ طبیعی و همچنین سیالیتِ آن را درمی‌گیرند.

فصل ۲۳ عناصرِ کلیدیِ زبانِ طبیعی، از جملهِ دستور و معناشناسی را توضیح داد. سامانه‌های مبتنی بر تجزیه و تحلیلِ معنایی در بسیاری از تکلیف‌ها موفقیت نشان داده‌اند، اما عملکردشان توسط پیچیدگیِ بی‌پایانِ پدیده‌های زبانی در متنِ واقعی محدود می‌شود. با توجه به حجمِ عظیمِ متنِ موجود در قالبِ خواندنیِ ماشینی، منطقی است بررسی کنیم آیا رویکردهای مبتنی بر یادگیریِ ماشینیِ داده‌محور می‌توانند مؤثرتر باشند.

این فرضیه را با ابزارهای فراهم‌شده توسط سامانه‌های یادگیری عمیق (فصل ۲۱) کاوش می‌کنیم.

در بخش ۲۴.۱ با نشان‌دادنِ اینکه چگونه یادگیری را می‌توان با بازنماییِ واژه‌ها به‌عنوانِ نقطه‌هایی در فضایِ پربعد، به‌جای مقادیرِ اتمی، بهبود بخشید شروع می‌کنیم. بخش ۲۴.۲ استفاده از شبکه‌های عصبیِ بازگشتی برای دربرگرفتنِ معنا و بافتِ دوربرد را در حالی که متن به‌صورتِ ترتیبی پردازش می‌شود پوشش می‌دهد. بخش ۲۴.۳ عمدتاً بر ترجمهٔ ماشینی تمرکز می‌کند، یکی از موفقیت‌های بزرگِ یادگیری عمیقِ اعمال‌شده بر NLP. بخش‌های ۲۴.۴ و ۲۴.۵ مدل‌هایی را پوشش می‌دهند که می‌توانند از حجمِ زیادی متنِ بدونِ برچسب آموزش ببینند و سپس بر تکلیف‌های خاص اعمال شوند، که اغلب به عملکردِ پیشتاز می‌رسند. سرانجام، بخش ۲۴.۶ ارزیابی می‌کند کجاییم و حوزه ممکن است چگونه پیشرفت کند.

۲۴.۱ جاسازیِ واژه

بازنماییِ واژه‌هایی می‌خواهیم که به مهندسیِ دستیِ ویژگی نیاز نداشته باشد، اما تعمیم بینِ واژه‌های مرتبط را مجاز کند—واژه‌هایی که از نظرِ نحوی مرتبط‌اند («بی‌رنگ» و «ایده‌آل» هر دو صفت‌اند)، از نظرِ معنایی («گربه» و «بچه‌گربه» هر دو گربه‌سان‌اند)، از نظرِ موضوعی («آفتابی» و «برف‌آب» هر دو اصطلاحِ آب‌وهوایند)، از نظرِ احساسات («فوق‌العاده» احساساتِ متضادِ «شرم‌آور» دارد)، یا به‌گونه‌ای دیگر.

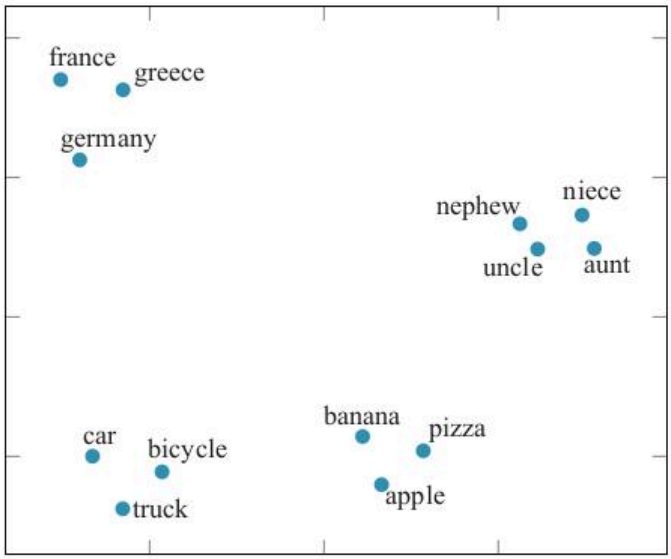
چگونه باید واژه‌ای را به بردارِ ورودیِ x برای استفاده در شبکهٔ عصبی رمز کنیم؟ چنان‌که در بخش ۲۱.۲.۱ (صفحهٔ ۷۵۶) توضیح داده شد، می‌توانستیم از بردارِ یک-گرم استفاده کنیم—یعنی، اُامین واژهٔ فرهنگِ لغت را با بیتی ۱ در اُامین موقعیتِ ورودی و ۰ در همهٔ موقعیت‌های دیگر رمز کنیم. اما چنین بازنمایی‌ای شباهتِ بینِ واژه‌ها را درنمی‌گرفت.

به پیروی از حکمِ زبان‌شناسِ جان آر. فیرث (1957)، «واژه را از همنشینانش خواهی شناخت»، می‌توانستیم هر واژه را با برداری از شمارش‌های n-گرمِ همهٔ عبارت‌هایی که واژه در آن‌ها ظاهر می‌شود بازنمایی کنیم. اما شمارش‌های خامِ n-گرم دست‌وپاگیرند. با واژگانی از ۱۰۰,۰۰۰ واژه، 10^{25} پنج‌گرم برای پیگیری هست (هرچند بردارهای این فضایِ 10^{25}-بعدی کاملاً تنک خواهند بود—بیشترِ شمارش‌ها صفر خواهند بود). تعمیمِ بهتری می‌گرفتیم اگر این را به برداری با اندازهٔ کوچک‌تر کاهش می‌دادیم، شاید فقط با چند صد بُعد. این بردارِ کوچکِ متراکم را «جاسازیِ واژه» (Word Embedding) می‌نامیم: بردارِ کم‌بعدی که واژه‌ای را بازنمایی می‌کند. جاسازی‌های واژه به‌صورتِ خودکار از داده یاد گرفته می‌شوند. (بعداً خواهیم دید این چگونه انجام می‌شود.)

این جاسازی‌های واژهٔ آموخته‌شده چه‌شکل‌اند؟ از یک سو، هر یک فقط برداری از عددهاست، که در آن بُعدهای منفرد و مقادیرِ عددی‌شان معانیِ قابلِ تشخیصی ندارند:

```
aardvark» = [−0.7, +0.2, −3.2, ...]»
 abacus»  = [+0.5, +0.9, −1.3, ...]»
                                  ...
 zyzzyva» = [−0.1, +0.8, −0.4, ...]»
```

از سوی دیگر، فضایِ ویژگی این ویژگی را دارد که واژه‌های مشابه در نهایت بردارهای مشابهی دارند. می‌توانیم آن را در شکل ۲۴.۱ ببینیم، جایی که خوشه‌های جداگانه‌ای برای واژه‌های کشور، خویشاوندی، حمل‌ونقل و غذا هست.



شکل ۲۴.۱ — بردارهای جاسازیِ واژه که توسط الگوریتمِ GloVe آموزش‌دیده رویِ ۶ میلیارد واژهٔ متن محاسبه شده‌اند. در این تجسم، بردارهای واژهٔ 100-بعدی رویِ دو بُعد تصویر شده‌اند. واژه‌های مشابه نزدیکِ هم ظاهر می‌شوند.

معلوم می‌شود، به دلایلی که کاملاً نمی‌فهمیم، بردارهای جاسازیِ واژه ویژگی‌های اضافی‌ای فراتر از صرفِ نزدیکیِ واژه‌های مشابه دارند. برای نمونه، فرض کنید به بردارهای A برای آتن و B برای یونان نگاه کنیم. برای این واژه‌ها، تفاضلِ بردارِ B - A به نظر می‌رسد رابطهٔ کشور/پایتخت را رمز می‌کند. جفت‌های دیگر—فرانسه و پاریس، روسیه و مسکو، زامبیا و لوزاکا—اساساً همان تفاضلِ بردار را دارند.

می‌توانیم از این ویژگی برای حلِ مسائلِ قیاسِ واژه مانندِ «آتن به یونان، همچنان که اسلو به [چه؟]» استفاده کنیم. با نوشتنِ C برای بردارِ اسلو و D برای مجهول، فرضیه می‌دهیم B - A = D - C، که به ما می‌دهد D = C + (B - A). و وقتی این بردارِ تازهٔ D را محاسبه می‌کنیم، می‌یابیم به «نروژ» نزدیک‌تر است تا هر واژهٔ دیگر. شکل ۲۴.۲ نشان می‌دهد این نوعِ حسابِ برداری برای بسیاری از رابطه‌ها کار می‌کند.

A	B	C	D = C + (B − A)	Relationship
Athens	Greece	Oslo	Norway	*Capital*
Astana	Kazakhstan	Harare	Zimbabwe	*Capital*
Angola	kwanza	Iran	rial	*Currency*
copper	Cu	gold	Au	*Atomic Symbol*
Microsoft	Windows	Google	Android	*Operating System*
New York	New York Times	Baltimore	Baltimore Sun	*Newspaper*
Berlusconi	Silvio	Obama	Barack	*First name*
Switzerland	Swiss	Cambodia	Cambodian	*Nationality*
Einstein	scientist	Picasso	painter	*Occupation*
brother	sister	grandson	granddaughter	*Family Relation*
Chicago	Illinois	Stockton	California	*State*
possibly	impossibly	ethical	unethical	*Negative*
mouse	mice	dollar	dollars	*Plural*
easy	easiest	lucky	luckiest	*Superlative*
walking	walked	swimming	swam	*Past tense*

شکل ۲۴.۲ – مدلِ جاسازیِ واژه گاهی می‌تواند به پرسشِ «A به B، همچنان که C به [چه؟]» با حسابِ برداری پاسخ دهد: با توجه به بردارهای جاسازیِ واژه برای واژه‌های A، B و C، بردارِ D = C + (B - A) را محاسبه کن و واژه‌ای را که به D نزدیک‌ترین است پیدا کن. (پاسخ‌های ستونِ D به‌صورتِ خودکار توسط مدل محاسبه شده‌اند. توصیف‌های ستونِ «رابطه» با دست افزوده شده‌اند.) اقتباس از میکولوف و همکاران (2013, 2014).

با این حال، هیچ تضمینی نیست که الگوریتمِ جاسازیِ واژهٔ خاصی که رویِ پیکرهٔ خاصی اجرا می‌شود رابطهٔ معناییِ خاصی را دربرگیرد. جاسازی‌های واژه محبوب‌اند، زیرا ثابت شده بازنماییِ خوبی برای تکلیف‌های زبانیِ پایین‌دست (مانندِ پاسخ‌گوییِ پرسش یا ترجمه یا خلاصه‌سازی) هستند، نه به‌دلیلِ اینکه تضمین می‌شود به‌خودیِ‌خود به پرسش‌های قیاسی پاسخ دهند.

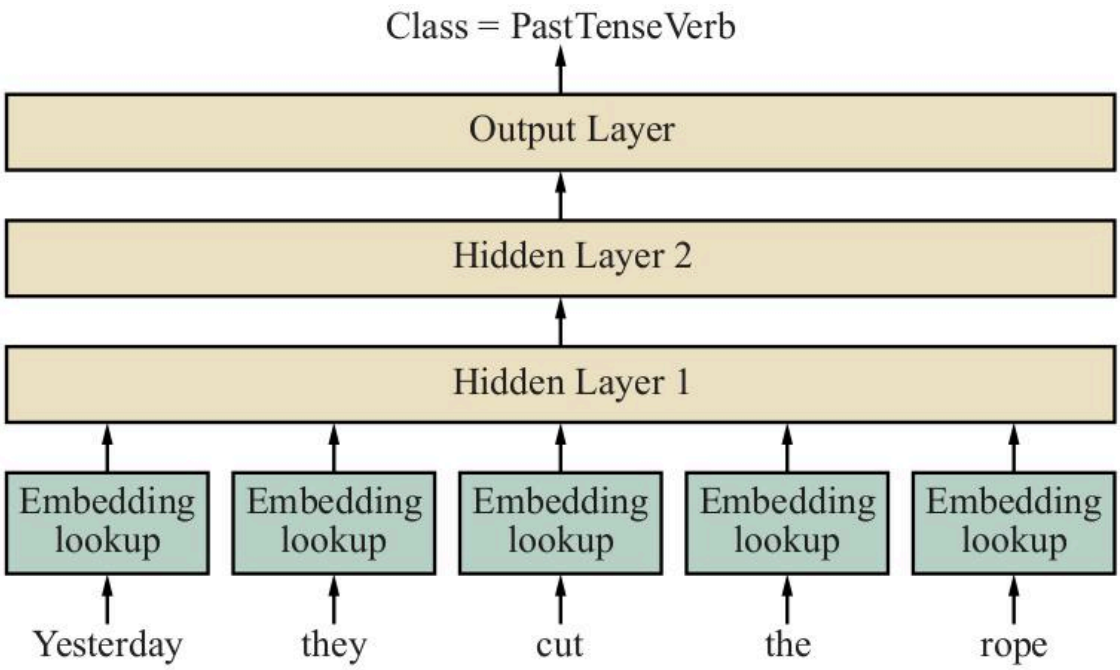
استفاده از بردارهای جاسازیِ واژه به‌جای رمزگذاری‌های یک-گرمِ واژه‌ها برای اساساً همهٔ کاربردهای یادگیری عمیق در تکلیف‌های NLP مفید معلوم می‌شود. در واقع، در بسیاری از موارد می‌توان از بردارهای عمومیِ ازپیش‌آموزش‌دیده، که از هر یک از چندین تأمین‌کننده به‌دست می‌آیند، برای تکلیفِ NLP خاصِ خود استفاده کرد. در زمانِ نوشتنِ این کتاب، فرهنگ‌های بردارِ پرکاربرد شاملِ GloVe، WORD2VEC (بردارهای سراسری) و FASTTEXT هستند، که FASTTEXT برای ۱۵۷ زبان جاسازی دارد. استفاده از مدلِ ازپیش‌آموزش‌دیده می‌تواند زمان و تلاشِ زیادی صرفه‌جویی کند. برای اطلاعاتِ بیشتر دربارهٔ این منابع، بخش ۲۴.۵.۱ را ببینید.

همچنین ممکن است بردارهای واژهٔ خودتان را آموزش دهید؛ این معمولاً هم‌زمان با آموزشِ شبکه‌ای برای تکلیفی خاص انجام می‌شود. برخلافِ جاسازی‌های عمومیِ ازپیش‌آموزش‌دیده، جاسازی‌های واژهٔ تولیدشده برای تکلیفی خاص را می‌توان رویِ پیکره‌ای با دقتِ انتخاب‌شده آموزش داد و تمایل خواهند داشت بر جنبه‌هایی از واژه‌ها تأکید کنند که برای تکلیف مفیدند. فرض کنید، برای نمونه، تکلیف، برچسب‌گذاریِ نقشِ دستوری (POS) است (بخش ۲۳.۱.۶ را ببینید). به یاد آورید این شاملِ پیش‌بینیِ نقشِ دستوریِ درست برای هر واژهٔ جمله است. هرچند این تکلیفِ ساده‌ای است، بی‌اهمیت نیست، زیرا بسیاری از واژه‌ها را می‌توان به چند شیوه برچسب‌گذاری کرد—برای نمونه، واژهٔ cut می‌تواند فعلِ زمانِ حال (متعدی یا ناگذر)، فعلِ زمانِ گذشته، فعلِ مصدری، اسم‌مفعولیِ گذشته، صفت یا اسم باشد. اگر قیدِ زمانیِ نزدیکی به گذشته ارجاع کند، آن‌گاه این پیشنهاد می‌شود این رخدادِ خاصِ cut فعلِ زمانِ گذشته است؛ و ممکن است امیدوار باشیم، پس، جاسازی جنبهٔ گذشته‌ارجاعِ قیدها را دربرگیرد.

برچسب‌گذاریِ POS مقدمهٔ خوبی برای کاربردِ یادگیری عمیق در NLP است، بدونِ پیچیدگیِ تکلیف‌های پیچیده‌تر مانندِ پاسخ‌گوییِ پرسش (بخش ۲۴.۵.۳ را ببینید).

با توجه به پیکره‌ای از جمله‌ها با برچسب‌های POS، پارامترهای جاسازیِ واژه و برچسب‌گذارِ POS را به‌طورِ هم‌زمان یاد می‌گیریم. فرایند به‌صورتِ زیر کار می‌کند:

1. پهنایِ w (تعدادِ فردِ واژه) را برای پنجرهٔ پیش‌بینیِ مورداستفاده برای برچسب‌گذاریِ هر واژه انتخاب کن. مقدارِ معمول w = 5 است، یعنی برچسب بر مبنای واژه به‌علاوهٔ دو واژهٔ سمتِ چپ و دو واژهٔ سمتِ راست پیش‌بینی می‌شود. هر جملهٔ پیکره‌ات را به پنجره‌های هم‌پوشانِ با طولِ w بشکن. هر پنجره یک مثالِ آموزشی تولید می‌کند که شاملِ w واژه به‌عنوانِ ورودی و ردهٔ POS واژهٔ میانی به‌عنوانِ خروجی است.
2. واژگانی از همهٔ توکن‌های واژهٔ یکتایی که بیش از، مثلاً، ۵ بار در دادهٔ آموزشی رخ می‌دهند ایجاد کن. تعدادِ کلِ واژه‌های واژگان را v بنام.
3. این واژگان را به هر ترتیبِ دلبخواهی مرتب کن (شاید الفبایی).
4. مقدارِ d را به‌عنوانِ اندازهٔ هر بردارِ جاسازیِ واژه انتخاب کن.
5. ماتریسِ وزنِ تازهٔ v-در-d به نامِ E ایجاد کن. این ماتریسِ جاسازیِ واژه است. سطرِ iامِ E جاسازیِ واژهٔ iاُمین واژهٔ واژگان است. E را به‌صورتِ تصادفی (یا از بردارهای ازپیش‌آموزش‌دیده) مقداردهیِ اولیه کن.
6. شبکهٔ عصبی‌ای برپا کن که برچسبِ نقشِ دستوری خروجی می‌دهد، چنان‌که در شکل ۲۴.۳ نشان داده شده.



شکل ۲۴.۳ – مدلِ برچسب‌گذاریِ نقشِ دستوریِ پیشخور. این مدل پنجره‌ای پنج‌واژه‌ای را به‌عنوانِ ورودی می‌گیرد و برچسبِ واژهٔ میانی—در این‌جا، cut—را پیش‌بینی می‌کند. مدل قادر به درنظرگرفتنِ موقعیتِ واژه است، زیرا هر یک از ۵ جاسازیِ ورودی در بخشِ متفاوتی از لایهٔ پنهانِ نخست ضرب می‌شود. مقادیرِ پارامترِ جاسازی‌های واژه و سه لایه همگی در طولِ آموزش به‌طورِ هم‌زمان یاد گرفته می‌شوند. لایهٔ نخست شاملِ w نسخه از ماتریسِ جاسازی خواهد بود. ممکن است از دو لایهٔ پنهانِ اضافی، z_1 و z_2 (به‌ترتیب با ماتریس‌های وزنِ W_1 و W_2)، استفاده کنیم و سپس لایهٔ سافتمکسی که توزیعِ احتمالِ خروجیِ ŷ را رویِ رده‌های نقشِ دستوریِ ممکن برای واژهٔ میانی می‌دهد:

$$z_1 = \sigma(W_1\ x)$$
$$z_2 = \sigma(W_2\ z_1)$$
$$\hat{y}\ = \text{softmax}(W_out\ z_2)$$

1. برای رمزکردنِ دنباله‌ای از w واژه به بردارِ ورودی، صرفاً جاسازیِ هر واژه را نگاه کن و بردارهای جاسازی را به هم بچسبان. نتیجه بردارِ ورودیِ حقیقی‌مقدارِ x با طولِ wd است. حتی اگر واژهٔ داده‌شده‌ای همان بردارِ جاسازی را داشته باشد، چه در موقعیتِ نخست رخ دهد، چه آخرین یا جایی در میانه، هر جاسازی در بخشِ متفاوتی از لایهٔ پنهانِ نخست ضرب خواهد شد؛ بنابراین موقعیتِ نسبیِ هر واژه را به‌صورتِ ضمنی رمز می‌کنیم.

1. وزن‌های E و دیگر ماتریس‌های وزنِ W_2، W_1 و W_out را با نزولِ گرادیان آموزش بده. اگر همه‌چیز خوب پیش برود، واژهٔ میانیِ cut، بر مبنای شاهدِ پنجره، که شاملِ واژهٔ زمانیِ گذشتهٔ «yesterday»، ضمیرِ فاعلیِ سوم‌شخصِ «they» بلافاصله پیش از cut، و مانندِ آن است، به‌عنوانِ فعلِ زمانِ گذشته برچسب خواهد خورد.

جایگزینی برای جاسازیِ واژه، مدلِ سطح-نویسه است که ورودی در آن دنباله‌ای از نویسه‌هاست، که هر یک به‌صورتِ بردارِ یک-گرم رمز شده. چنین مدلی باید یاد بگیرد نویسه‌ها چگونه به هم می‌آیند تا واژه‌ها را بسازند. بیشترِ کارِ NLP به رمزگذاری‌های سطح-واژه می‌چسبد تا سطح-نویسه.

۲۴.۲ شبکه‌های عصبیِ بازگشتی برای NLP

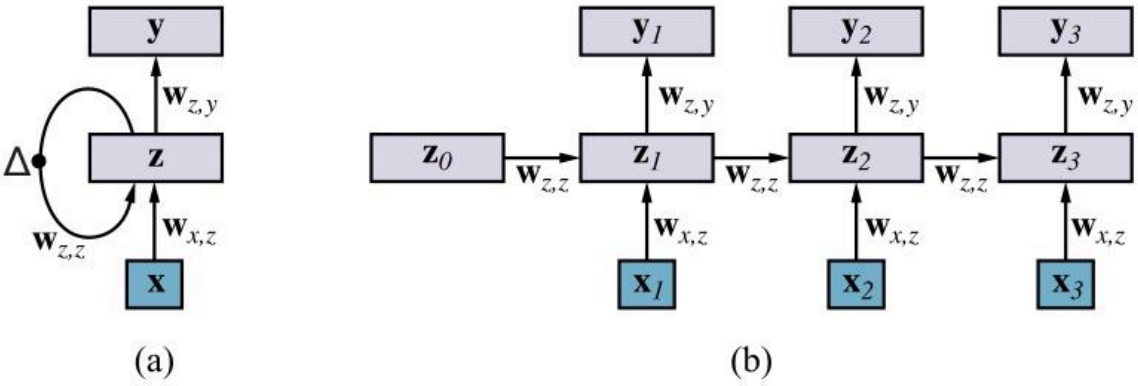
حالا بازنماییِ خوبی برای واژه‌های تکیِ منزوی داریم، اما زبان شاملِ دنبالهٔ مرتبی از واژه‌هاست که بافتِ واژه‌های اطراف در آن مهم است. برای تکلیف‌های ساده مانندِ برچسب‌گذاریِ نقشِ دستوری، پنجرهٔ کوچکِ با اندازهٔ ثابتِ شاید پنج‌واژه‌ای معمولاً بافتِ کافی فراهم می‌کند.

تکلیف‌های پیچیده‌تر مانندِ پاسخ‌گوییِ پرسش یا رفعِ ارجاع ممکن است به ده‌ها واژه به‌عنوانِ بافت نیاز داشته باشند. برای نمونه، در جملهٔ «ادواردو به من گفت میگل خیلی مریض بود، پس او را به بیمارستان بردم»، دانستنِ اینکه «او» به میگل ارجاع می‌کند نه ادواردو، به بافتی نیاز دارد که از نخستین تا آخرین واژهٔ جملهٔ ۱۴-واژه‌ای را می‌پوشاند.

۲۴.۲.۱ مدل‌های زبان با شبکه‌های عصبیِ بازگشتی

با مسئلهٔ ایجادِ مدلِ زبانِ با بافتِ کافی شروع می‌کنیم. به یاد آورید مدلِ زبان توزیعِ احتمالیِ رویِ دنباله‌های واژه است. به ما اجازه می‌دهد واژهٔ بعدیِ متن را با توجه به همهٔ واژه‌های قبلی پیش‌بینی کنیم، و اغلب به‌عنوانِ بلوکِ ساختمانی برای تکلیف‌های پیچیده‌تر استفاده می‌شود.

ساختنِ مدلِ زبان با مدلِ n-گرم (مانندِ بخش ۲۳.۱) یا شبکهٔ پیشخور با پنجرهٔ ثابتِ n واژه می‌تواند به‌دلیلِ مسئلهٔ بافت به دشواری بیفتد: یا بافتِ لازم از اندازهٔ پنجرهٔ ثابت فراتر خواهد رفت، یا مدل پارامترهای زیادی خواهد داشت، یا هر دو. علاوه بر این، شبکهٔ پیشخور مشکلِ نامتقارنی دارد: هرچه دربارهٔ، مثلاً، ظهورِ واژهٔ him به‌عنوانِ واژهٔ دوازدهمِ جمله یاد بگیرد، باید برای ظهورِ him در موقعیت‌های دیگرِ جمله دوباره یاد بگیرد، زیرا وزن‌ها برای هر موقعیتِ واژه متفاوت‌اند.



شکل ۲۴.۴ – (الف) نمودارِ شماتیکِ RNN که لایهٔ پنهانِ z اتصال‌های بازگشتی دارد؛ نمادِ Δ تأخیر را نشان می‌دهد. هر ورودیِ x بردارِ جاسازیِ واژهٔ واژهٔ بعدیِ جمله است. هر خروجیِ y خروجیِ همان گامِ زمانی است. (ب) همان شبکه که رویِ سه گامِ زمانی بازشده تا شبکهٔ پیشخوری بسازد. توجه کنید وزن‌ها در همهٔ گام‌های زمانی مشترک‌اند.

در بخش ۲۱.۶، شبکهٔ عصبیِ بازگشتی یا RNN را معرفی کردیم، که برای پردازشِ دادهٔ سریِ زمانی، هر بار یک داده، طراحی شده. این پیشنهاد می‌کند RNNها ممکن است برای پردازشِ زبان، هر بار یک واژه، مفید باشند. شکل ۲۱.۸ را این‌جا به‌عنوانِ شکل ۲۴.۴ تکرار می‌کنیم.

در مدلِ زبانِ RNN، هر واژهٔ ورودی به‌عنوانِ بردارِ جاسازیِ واژه، x_i، رمز می‌شود. لایهٔ پنهانِ z_t هست که به‌عنوانِ ورودی از یک گامِ زمانی به گامِ بعدی منتقل می‌شود. به رده‌بندیِ چندرده‌ای علاقه‌مندیم: رده‌ها واژه‌های واژگان‌اند. پس خروجیِ y_t توزیعِ احتمالِ سافتمکسی رویِ مقادیرِ ممکنِ واژهٔ بعدیِ جمله خواهد بود.

معماریِ RNN مسئلهٔ پارامترهایِ زیاد را حل می‌کند. تعدادِ پارامترهای ماتریس‌های وزنِ $w_{x,z}$، $w_{z,z}$ و $w_{z,y}$ ثابت می‌ماند، بدونِ توجه به تعدادِ واژه‌ها—$O(1)$ است. این در تقابل با شبکه‌های پیشخور است، که $O(n)$ پارامتر دارند، و مدل‌های n-گرم، که $O(v^n)$ پارامتر دارند، که v اندازهٔ واژگان است.

معماریِ RNN همچنین مسئلهٔ نامتقارنی را حل می‌کند، زیرا وزن‌ها برای هر موقعیتِ واژه همان‌اند. معماریِ RNN گاهی می‌تواند مسئلهٔ بافتِ محدود را نیز حل کند. در تئوری هیچ حدی برای اینکه مدل چقدر به عقبِ ورودی نگاه کند نیست. هر به‌روزرسانیِ لایهٔ پنهانِ z_t به هر دوِ واژهٔ ورودیِ کنونیِ x_t و لایهٔ پنهانِ قبلیِ z_{t-1} دسترسی دارد، که به این معناست اطلاعاتِ مربوط به هر واژهٔ ورودی می‌تواند به‌طورِ نامحدود در لایهٔ پنهان نگه داشته شود، و از یک گامِ زمانی به گامِ بعد کپی (یا به‌صورتِ مناسب اصلاح) شود. البته، ذخیرهٔ محدودی در z هست، پس نمی‌تواند همه‌چیزِ مربوط به همهٔ واژه‌های قبلی را به خاطر بسپارد. در عمل، مدل‌های RNN در انواعِ تکلیف‌ها خوب عمل می‌کنند، اما نه در همهٔ تکلیف‌ها. پیش‌بینیِ موفق‌بودنشان برای مسئله‌ای داده‌شده دشوار است. عاملی که به موفقیت کمک می‌کند این است که فرایندِ آموزش شبکه را تشویق می‌کند فضایِ ذخیرهٔ z را به جنبه‌هایی از ورودی تخصیص دهد که در واقع مفید خواهند بود.

برای آموزشِ مدلِ زبانِ RNN، از فرایندِ آموزشِ توصیف‌شده در بخش ۲۱.۶.۱ استفاده می‌کنیم. ورودی‌های x_t واژه‌های پیکرهٔ آموزشیِ متن و خروجی‌های مشاهده‌شده همان واژه‌ها هستند که با ۱ جابه‌جا شده‌اند. یعنی، برای متنِ آموزشیِ «hello world»، نخستین ورودیِ x_1 جاسازیِ واژهٔ «hello» و نخستین خروجیِ y_1 جاسازیِ واژهٔ «world» است. مدل را برای پیش‌بینیِ واژهٔ بعدی آموزش می‌دهیم، و انتظار داریم برای انجامِ این کار از لایهٔ پنهان برای بازنماییِ اطلاعاتِ مفید استفاده کند. چنان‌که در بخش ۲۱.۶.۱ توضیح داده شد، تفاوتِ بینِ خروجیِ مشاهده‌شده و خروجیِ واقعیِ محاسبه‌شده توسط شبکه را محاسبه می‌کنیم و در طولِ زمان پسانتشار می‌دهیم، با مراقبت از اینکه وزن‌ها را برای همهٔ گام‌های زمانی یکسان نگه داریم.

پس از آموزشِ مدل، می‌توانیم از آن برای تولیدِ متنِ تصادفی استفاده کنیم. واژهٔ ورودیِ آغازینِ x_1 را به مدل می‌دهیم، که از آن خروجیِ y_1 تولید می‌کند، توزیعِ احتمالِ سافتمکسی رویِ واژه‌ها. واژه‌ای تکی از توزیع نمونه‌برداری می‌کنیم، واژه را به‌عنوانِ خروجیِ زمانِ t ثبت می‌کنیم، و آن را به‌عنوانِ واژهٔ ورودیِ بعدیِ x_2 به عقب تغذیه می‌کنیم. تا هر زمان که خواسته شود تکرار می‌کنیم. در نمونه‌برداری از y_1 انتخابی داریم: می‌توانیم همیشه محتمل‌ترین واژه را بگیریم؛ می‌توانیم طبقِ احتمالِ هر واژه نمونه‌برداری کنیم؛ یا می‌توانیم از واژه‌های کم‌محتمل‌تر بیش‌نمونه‌برداری کنیم، تا تنوعِ بیشتری به خروجیِ تولیدشده تزریق شود. وزنِ نمونه‌برداری ابرپارامترِ مدل است.

در این‌جا نمونه‌ای از متنِ تصادفیِ تولیدشده توسط مدلِ RNN آموزش‌دیده رویِ آثارِ شکسپیر (کارپاتی، 2015) آمده (با برگردانِ فارسی):

«ازدواج کن، و خواهم کرد، سرورم، که گریستن در چنین کسی زیباترین بود؛ با این حال، اکنون من وارثِ خواندهٔ روزِ غم‌انگیزِ جهان گشتم، تا راهِ بعدی را با پدرش و با چهره‌اش نظاره کنم؟»

Marry, and will, my lord, to weep in such a one were best; Yet now I was adopted heir Of the world's' lamentable day, To watch the next way with his father with his face?"

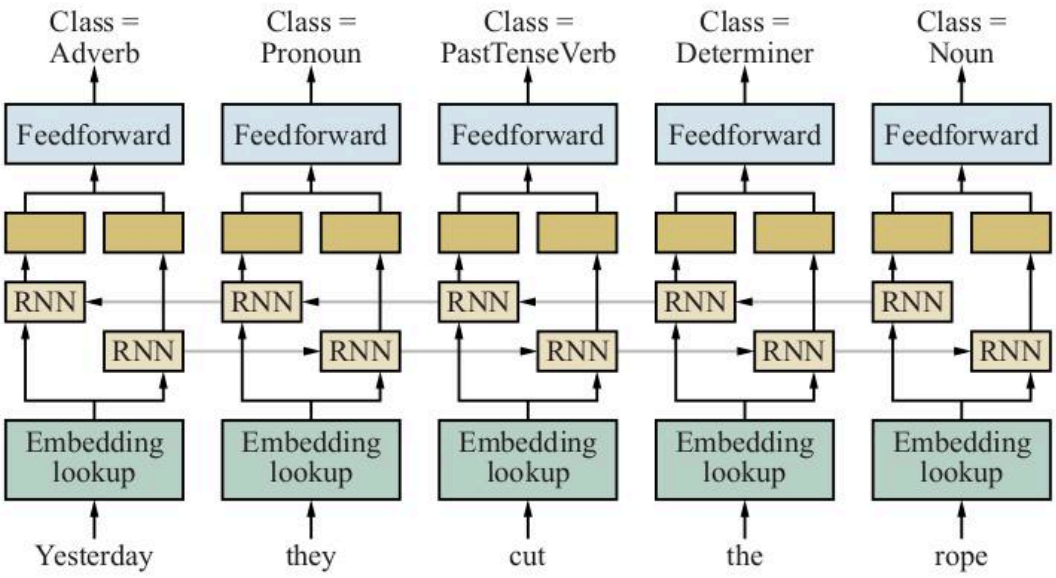
۲۴.۲.۲ رده‌بندی با شبکه‌های عصبیِ بازگشتی

همچنین می‌توان از RNNها برای دیگر تکلیف‌های زبانی، مانندِ برچسب‌گذاریِ نقشِ دستوری یا رفعِ هم‌ارجاعی، استفاده کرد. در هر دو مورد لایه‌های ورودی و پنهان همان خواهند بود، اما برای برچسب‌گذارِ POS خروجی توزیعِ سافتمکسی رویِ برچسب‌های POS خواهد بود، و برای رفعِ هم‌ارجاعی توزیعِ سافتمکسی رویِ پیشایندهای ممکن. برای نمونه، وقتی شبکه به ورودیِ «او» در «ادواردو به من گفت میگل خیلی مریض بود، پس او را به بیمارستان بردم» می‌رسد، باید احتمالِ بالایی برای «میگل» خروجی دهد.

آموزشِ RNN برای رده‌بندیِ این‌چنینی همان‌گونه که با مدلِ زبان انجام می‌شود صورت می‌گیرد. تنها تفاوت این است که دادهٔ آموزشی به برچسب‌ها نیاز خواهد داشت—برچسب‌های نقشِ دستوری یا نشانه‌های ارجاع. این جمع‌آوریِ داده را بسیار دشوارتر از حالتِ مدلِ زبان می‌کند، که در آن فقط به متنِ بدونِ برچسب نیاز داریم.

در مدلِ زبان می‌خواهیم nاُمین واژه را با توجه به واژه‌های قبلی پیش‌بینی کنیم. اما برای رده‌بندی، دلیلی نیست خود را به نگاه‌کردنِ فقط به واژه‌های قبلی محدود کنیم. نگاه‌کردن به جلو در جمله می‌تواند بسیار کمک‌کننده باشد. در مثالِ هم‌ارجاعیِ ما، مرجعِ «او» اگر جمله به‌جای «به بیمارستان» با «تا میگل را ببیند» پایان می‌یافت متفاوت بود، پس نگاه به جلو حیاتی است. از آزمایش‌های ردیابیِ چشم می‌دانیم خوانندگانِ انسانی به‌صورتِ دقیقِ چپ-به-راست پیش نمی‌روند.

برای دربرگرفتنِ بافتِ سمتِ راست، می‌توانیم از «RNN دوسویه» (Bidirectional RNN) استفاده کنیم، که مدلِ جداگانهٔ راست-به-چپ را به مدلِ چپ-به-راست می‌چسباند. نمونه‌ای از استفاده از RNN دوسویه برای برچسب‌گذاریِ POS در شکل ۲۴.۵ نشان داده شده.



شکل ۲۴.۵ – شبکهٔ RNN دوسویه برای برچسب‌گذاریِ POS.

در موردِ RNN چندلایه، z_t بردارِ پنهانِ آخرین لایه خواهد بود. برای RNN دوسویه، z_t معمولاً چسباندنِ بردارهای مدلِ چپ-به-راست و راست-به-چپ در نظر گرفته می‌شود.

RNNها همچنین می‌توانند برای تکلیف‌های رده‌بندیِ سطح-جمله (یا سطح-سند) استفاده شوند، که در آن‌ها خروجیِ تکی در پایان می‌آید، به‌جای داشتنِ جریانی از خروجی‌ها، یکی در هر گامِ زمانی. برای نمونه، در تحلیلِ احساسات هدف رده‌بندیِ متنی به‌عنوانِ دارایِ احساساتِ مثبت یا منفی است. برای نمونه، «این فیلم بد نوشته و بد بازی شده بود» باید به‌عنوانِ منفی رده‌بندی شود. (برخی طرح‌های تحلیلِ احساسات از بیش از دو رده استفاده می‌کنند، یا از مقدارِ عددیِ اسکالر.) استفاده از RNNها برای تکلیفِ سطح-جمله کمی پیچیده‌تر است، زیرا باید بازنماییِ کلِ جملهٔ تجمیعی، y، را از خروجی‌های واژه‌به‌واژهٔ RNN، y_t، به‌دست آوریم.

ساده‌ترین راه برای این کار استفاده از حالتِ پنهانِ RNN متناظر با آخرین واژهٔ ورودی است، زیرا RNN در آن گامِ زمانی کلِ جمله را خوانده خواهد بود. با این حال، این می‌تواند به‌صورتِ ضمنی مدل را به سمتِ توجهِ بیشتر به پایانِ جمله اریب کند. فنِ رایجِ دیگر تجمیعِ همهٔ بردارهای پنهان است. برای نمونه، «تجمیعِ میانگین» (Average Pooling) میانگینِ عنصربه‌عنصرِ همهٔ بردارهای پنهان را محاسبه می‌کند:

$$\tilde{z} = (1/s) \sum_{t=1}^{s} z_t$$

سپس بردارِ تجمیع‌شدهٔ d-بعدیِ $\tilde{z}$ را می‌توان به یک یا چند لایهٔ پیشخور خوراک داد پیش از آنکه به لایهٔ خروجی خوراک شود.

۲۴.۲.۳ LSTMها برای تکلیف‌های NLP

گفتیم RNNها گاهی مسئلهٔ بافتِ محدود را حل می‌کنند. در تئوری، هر اطلاعاتی می‌تواند از یک لایهٔ پنهان به لایهٔ بعدی برای هر تعدادِ گامِ زمانی منتقل شود. اما در عمل اطلاعات می‌تواند گم یا تحریف شود، درست مانندِ بازیِ تلفن، که بازیکنان در صف می‌ایستند و نخستین بازیکن پیامی را به نفرِ دوم زمزمه می‌کند، که آن را برای نفرِ سوم تکرار می‌کند، و همین‌طور تا انتهای صف. معمولاً، پیامی که در پایان بیرون می‌آید کاملاً از پیامِ اصلی خراب شده. این مسئله برای RNNها شبیهِ مسئلهٔ گرادیانِ ناپدیدشونده‌ای است که در صفحهٔ ۷۵۶ توصیف کردیم، با این تفاوت که حالا با لایه‌هایی در طولِ زمان سروکار داریم نه لایه‌های عمیق.

در بخش ۲۱.۶.۲ مدلِ حافظهٔ کوتاه‌مدت-بلندمدت (LSTM) را معرفی کردیم. این نوعی RNN با واحدهای دروازه‌ای است که از مشکلِ بازتولیدِ ناقصِ پیام از یک گامِ زمانی به گامِ بعد رنج نمی‌برد. در عوض، LSTM می‌تواند انتخاب کند برخی بخش‌های ورودی را به خاطر بسپارد، آن‌ها را به گامِ زمانیِ بعد کپی کند، و بخش‌های دیگر را فراموش کند. مدلِ زبانی را در نظر بگیر که متنی مانندِ «ورزشکارانی که همه مقدماتیِ محلی‌شان را بردند و به فینالِ توکیو راه یافتند، اکنون ...» را مدیریت می‌کند.

در این نقطه، اگر از مدل می‌پرسیدیم کدام واژهٔ بعدی محتمل‌تر است، «رقابت می‌کنند» یا «رقابت می‌کند»، انتظار داشتیم «رقابت می‌کنند» را برگیرد، زیرا با فاعلِ «ورزشکاران» توافق دارد. LSTM می‌تواند یاد بگیرد ویژگیِ نهانی برای شخص و شمارِ فاعل بسازد و آن ویژگی را بدونِ تغییر به جلو کپی کند تا وقتی برای چنین انتخابی لازم شود. RNN معمولی (یا مدلِ n-گرم، در این مورد) اغلب در جمله‌های بلند با واژه‌های میانیِ بسیار بینِ فاعل و فعل گیج می‌شود.

۲۴.۳ مدل‌های دنباله‌به‌دنباله

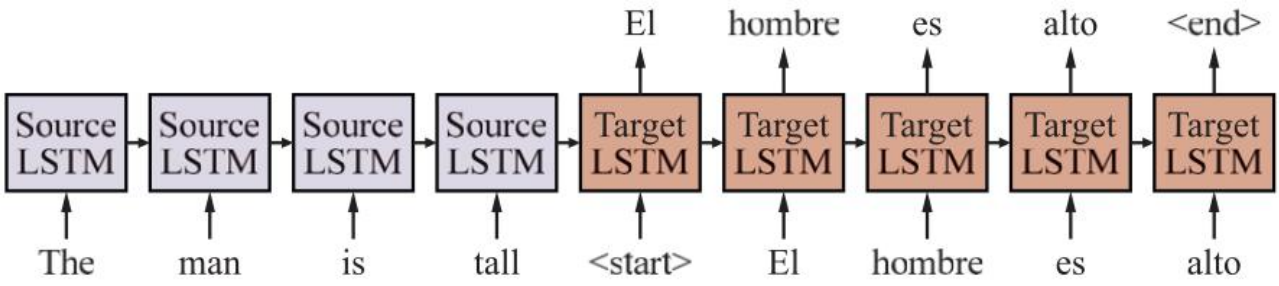
یکی از پرکاربردترینِ تکلیف‌های مطالعه‌شده در NLP «ترجمهٔ ماشینی» (Machine Translation یا MT) است، که هدف در آن ترجمهٔ جمله‌ای از «زبانِ مبدأ» به «زبانِ مقصد» است—برای نمونه، از اسپانیایی به انگلیسی. مدلِ MT را با پیکرهٔ بزرگی از جفتِ جمله‌های مبدأ/مقصد آموزش می‌دهیم. هدف سپس ترجمهٔ دقیقِ جمله‌های تازه‌ای است که در دادهٔ آموزشیِ ما نیستند.

آیا می‌توانیم از RNNها برای ایجادِ سامانهٔ MT استفاده کنیم؟ قطعاً می‌توانیم جملهٔ مبدأ را با RNN رمز کنیم. اگر تناظرِ یک‌به‌یک بینِ واژه‌های مبدأ و مقصد بود، می‌توانستیم MT را تکلیفِ برچسب‌گذاریِ ساده‌ای بدانیم—با توجه به واژهٔ مبدأِ «perro» در اسپانیایی، آن را با واژهٔ انگلیسیِ متناظرِ «dog» برچسب می‌زنیم. اما در واقع، واژه‌ها یک‌به‌یک نیستند: در اسپانیایی سه واژهٔ «caballo de mar» با واژهٔ انگلیسیِ تکیِ «seahorse» متناظر است، و دو واژهٔ «perro grande» به «big dog» ترجمه می‌شود، با ترتیبِ واژهٔ معکوس. بازچینیِ واژه می‌تواند حتی شدیدتر باشد؛ در انگلیسی فاعل معمولاً در آغازِ جمله است، اما در فیجیایی فاعل معمولاً در پایان است. پس چگونه جمله‌ای در زبانِ مقصد تولید کنیم؟

به نظر می‌رسد باید واژه‌ای‌واژه تولید کنیم، اما بافت را پیگیری کنیم تا بتوانیم بخش‌هایی از مبدأ را که هنوز ترجمه نشده به خاطر بسپاریم، و آنچه ترجمه شده را پیگیری کنیم تا خودمان را تکرار نکنیم. همچنین به نظر می‌رسد برای برخی جمله‌ها باید کلِ جملهٔ مبدأ را پیش از شروعِ تولیدِ مقصد پردازش کنیم. به بیانِ دیگر، تولیدِ هر واژهٔ مقصد مشروط بر کلِ جملهٔ مبدأ و همهٔ واژه‌های مقصدِ قبلاً تولیدشده است.

این به تولیدِ متن برای MT پیوندی نزدیک با مدلِ زبانِ RNN استانداردِ توصیف‌شده در بخش ۲۴.۲ می‌دهد. قطعاً، اگر RNNای را رویِ متنِ انگلیسی آموزش داده بودیم، احتمالِ تولیدِ «big dog» بیشتر از «dog big» بود. اما نمی‌خواهیم هر جملهٔ تصادفیِ زبانِ مقصد را تولید کنیم؛ می‌خواهیم جملهٔ زبانِ مقصدی تولید کنیم که با جملهٔ زبانِ مبدأ متناظر باشد. ساده‌ترین راه برای انجامِ این کار استفاده از دو RNN است، یکی برای مبدأ و یکی برای مقصد. RNN مبدأ را رویِ جملهٔ مبدأ اجرا می‌کنیم و سپس از حالتِ پنهانِ نهاییِ RNN مبدأ به‌عنوانِ حالتِ پنهانِ آغازینِ RNN مقصد استفاده می‌کنیم. به این‌شیوه، هر واژهٔ مقصد به‌صورتِ ضمنی هم بر کلِ جملهٔ مبدأ و هم بر واژه‌های مقصدِ قبلی مشروط است.

این معماریِ شبکهٔ عصبی «مدلِ دنباله‌به‌دنبالهٔ پایه» (Sequence-to-Sequence Model) نامیده می‌شود، که نمونه‌ای از آن در شکل ۲۴.۶ نشان داده شده. مدل‌های دنباله‌به‌دنباله بیشتر برای ترجمهٔ ماشینی استفاده می‌شوند، اما می‌توانند برای تعدادی از تکلیف‌های دیگر نیز استفاده شوند، مانندِ تولیدِ خودکارِ زیرنویسِ متنی از تصویر، یا خلاصه‌سازی: بازنویسیِ متنِ بلند به متنی کوتاه‌تر که همان معنا را حفظ می‌کند.



شکل ۲۴.۶ — مدلِ دنباله‌به‌دنبالهٔ پایه. هر بلوک یک گامِ زمانیِ LSTM را نشان می‌دهد. (برای سادگی، لایه‌های جاسازی و خروجی نشان داده نشده‌اند.) در گام‌های متوالی، واژه‌های جملهٔ مبدأ «The man is tall» را به شبکه می‌دهیم، و سپس برچسبِ <start> را تا نشان دهیم شبکه باید تولیدِ جملهٔ مقصد را شروع کند. حالتِ پنهانِ نهایی در پایانِ جملهٔ مبدأ به‌عنوانِ حالتِ پنهان برای شروعِ جملهٔ مقصد استفاده می‌شود. پس از آن، هر واژهٔ جملهٔ مقصد در زمانِ t به‌عنوانِ ورودی در زمانِ t+1 استفاده می‌شود، تا وقتی شبکه برچسبِ <end> را تولید کند که نشان دهد تولیدِ جمله پایان یافته.

مدل‌های دنباله‌به‌دنبالهٔ پایه پیشرفتِ چشمگیری در NLP و به‌طورِ خاص MT بودند. طبقِ وو و همکاران (2016b) این رویکرد به کاهشِ خطایِ ۶۰٪ نسبت به روش‌های قبلیِ MT انجامید. اما این مدل‌ها از سه کاستیِ عمده رنج می‌برند:

- **اریبِ بافتِ نزدیک:** هر آنچه RNNها می‌خواهند دربارهٔ گذشته به خاطر بسپارند، باید در حالتِ پنهانِ خود جای دهند. برای نمونه، فرض کنید RNN در حالِ پردازشِ واژه (یا گامِ زمانیِ) ۵۷ در دنباله‌ای ۷۰-واژه‌ای است. حالتِ پنهان به احتمالِ زیاد اطلاعاتِ بیشتری دربارهٔ واژهٔ گامِ زمانیِ ۵۶ دارد تا واژهٔ گامِ زمانیِ ۵، زیرا هر بار که بردارِ پنهان به‌روزرسانی می‌شود باید مقداری از اطلاعاتِ موجود را با اطلاعاتِ تازه جایگزین کند. این رفتار بخشی از

طراحیِ عمدیِ مدل است، و اغلب برای NLP منطقی است، زیرا بافتِ نزدیک معمولاً مهم‌تر است. با این حال، بافتِ دور نیز می‌تواند حیاتی باشد، و می‌تواند در مدلِ RNN گم شود؛ حتی LSTMها نیز با این تکلیف دشواری دارند.

- **حدِّ اندازهٔ ثابتِ بافت:** در مدلِ ترجمهٔ RNN، کلِ جملهٔ مبدأ به بردارِ حالتِ پنهانِ تکی با ابعادِ ثابت فشرده می‌شود. LSTMِ استفاده‌شده در مدلِ NLP پیشتاز معمولاً حدودِ ۱۰۲۴ بُعد دارد، و اگر مجبور باشیم، مثلاً، جملهٔ ۶۴-واژه‌ای را در ۱۰۲۴ بُعد بازنمایی کنیم، این فقط ۱۶ بُعد به‌ازایِ هر واژه به ما می‌دهد—برای جمله‌های پیچیده کافی نیست. افزایشِ اندازهٔ بردارِ حالتِ پنهان می‌تواند به آموزشِ کند و بیش‌برازش بینجامد.
- **پردازشِ ترتیبیِ کندتر:** چنان‌که در بخش ۲۱.۳ بحث شد، شبکه‌های عصبی با پردازشِ دادهٔ آموزشی در دسته‌ها، برای بهره‌گیری از پشتیبانیِ کارآمدِ سخت‌افزاریِ ضربِ ماتریس، سودِ کاراییِ چشمگیری به‌دست می‌آورند. RNNها، از سوی دیگر، به نظر می‌رسد مقیدند رویِ دادهٔ آموزشی هر بار یک واژه عمل کنند.

۲۴.۳.۱ توجه

چه می‌شد اگر RNNِ مقصد بر همهٔ بردارهای پنهانِ RNNِ مبدأ مشروط بود، نه فقط آخرین‌شان؟ این کاستی‌های اریبِ بافتِ نزدیک و حدِّ اندازهٔ ثابتِ بافت را کاهش می‌داد، و به مدل اجازه می‌داد به هر واژهٔ قبلی به‌یکسان دسترسی داشته باشد. یکی از راه‌های دست‌یابی به این دسترسی چسباندنِ همهٔ بردارهای پنهانِ RNN مبدأ است. با این حال، این افزایشِ عظیمی در تعدادِ وزن‌ها ایجاد می‌کرد، با افزایشِ هم‌زمانِ زمانِ محاسبه و احتمالاً بیش‌برازش. در عوض، می‌توانیم از این واقعیت بهره بگیریم که وقتیِ RNN مقصد واژه‌های مقصد را یکی‌یکی تولید می‌کند، احتمال دارد فقط بخشِ کوچکی از مبدأ واقعاً به هر واژهٔ مقصد مربوط باشد.

به‌طورِ حیاتی، RNNِ مقصد باید برای هر واژه به بخش‌های متفاوتی از مبدأ توجه کند. فرض کنید شبکه‌ای برای ترجمه از انگلیسی به اسپانیایی آموزش دیده. واژه‌های «The front door is red» و سپس نشانگرِ پایانِ جمله به آن داده می‌شود، که به این معناست زمانِ شروعِ خروجیِ واژه‌های اسپانیایی رسیده. پس در حالتِ ایده‌آل، نخست باید به «The» توجه کند و «La» تولید کند، سپس به «door» توجه کند و «puerta» خروجی دهد، و همین‌طور ادامه دهد.

می‌توانیم این مفهوم را با مؤلفهٔ شبکهٔ عصبی به نامِ «توجه» (Attention) صوری کنیم، که می‌تواند برای ایجادِ «خلاصهٔ مبتنی بر بافت» از جملهٔ مبدأ به بازنماییِ با ابعادِ ثابت استفاده شود. بردارِ بافتِ c_i مربوط‌ترین اطلاعات را برای تولیدِ واژهٔ مقصدِ بعدی شامل است، و به‌عنوانِ ورودیِ اضافی به RNNِ مقصد استفاده خواهد شد. مدلِ دنباله‌به‌دنباله‌ای که از توجه استفاده می‌کند «مدلِ دنباله‌به‌دنبالهٔ مبتنی بر توجه» (Attentional Sequence-to-Sequence Model) نامیده می‌شود. اگر RNNِ مقصدِ استاندارد به‌صورتِ زیر نوشته شود:

$$h_i = RNN(h_{i-1}, x_i)$$

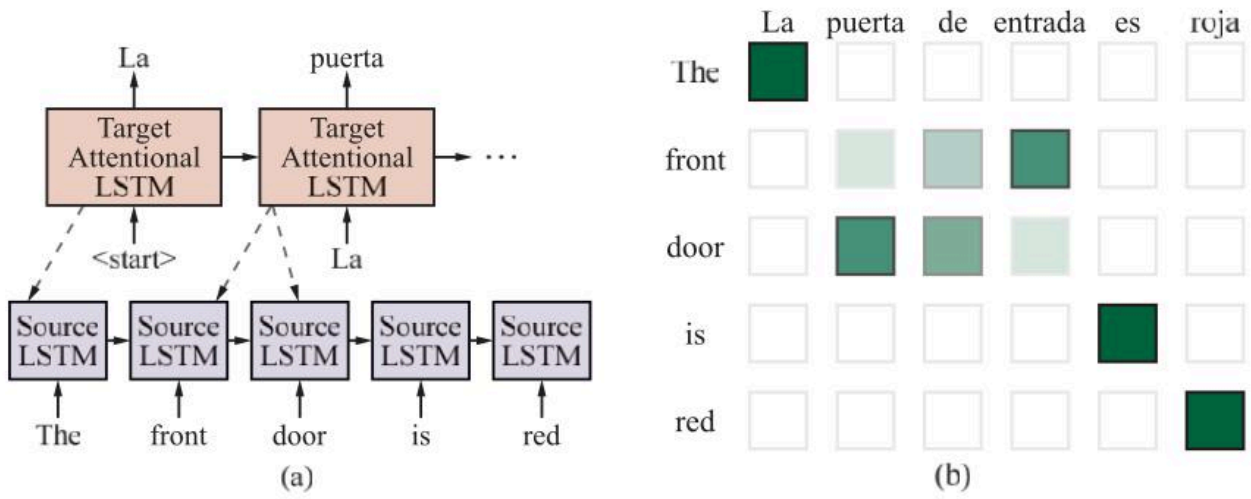
RNN مقصد برای مدل‌های دنباله‌به‌دنبالهٔ مبتنی بر توجه را می‌توان به‌صورتِ زیر نوشت:

$$h_i = RNN(h_{i-1}, [x_i; c_i])$$

که $[x_i; c_i]$ چسباندنِ بردارهای ورودی و بافت است، و c_i به‌صورتِ زیر تعریف می‌شود:

$$r_{ij} = h_{i-1} \cdot s_j$$
$$a_{ij} = e^{r_{ij}} / (\sum_k e^{r_{ik}})$$
$$c_i = \sum_j a_{ij} \cdot s_j$$

که در آن h_{i-1} بردارِ RNN مقصد است که برای پیش‌بینیِ واژهٔ گامِ زمانیِ i استفاده خواهد شد، و s_j خروجیِ بردارِ RNN مبدأ برای واژهٔ مبدأ (یا گامِ زمانیِ) j است. هر دوِ h_{i-1} و s_j بردارهای d-بعدی‌اند، که d اندازهٔ پنهان است. مقدارِ r_{ij} بنابراین «امتیازِ توجهِ» خام بینِ حالتِ مقصدِ کنونی و واژهٔ مبدأِ j است. سپس این امتیازها با سافتمکس رویِ همهٔ واژه‌های مبدأ به احتمالِ a_{ij} نرمال می‌شوند. سرانجام، از این احتمال‌ها برای تولیدِ میانگینِ وزن‌دارِ بردارهایِ RNN مبدأ، c_i (بردارِ d-بعدیِ دیگری)، استفاده می‌شود. نمونه‌ای از مدلِ دنباله‌به‌دنبالهٔ مبتنی بر توجه در شکل ۲۴.۷ (الف) داده شده.



شکل ۲۴.۷ – (الف) مدلِ دنباله‌به‌دنبالهٔ مبتنی بر توجه برای ترجمهٔ انگلیسی-به-اسپانیایی. خطوطِ چین‌خورده توجه را نشان می‌دهند. (ب) نمونه‌ای از ماتریسِ احتمالِ توجه برای جفتِ جملهٔ دوزبانه، با جعبه‌های تیره‌تر که مقادیرِ بالاترِ a_{ij} را نشان می‌دهند. احتمال‌های توجه رویِ هر ستون به ۱ جمع می‌شوند.

چند جزئیاتِ مهم هست که باید فهمید. نخست، خودِ مؤلفهٔ توجه هیچ وزنِ آموخته‌ای ندارد و از دنباله‌های با طولِ متغیر هم در سمتِ مبدأ و هم در سمتِ مقصد پشتیبانی می‌کند. دوم، مانندِ بیشترِ فنونِ مدل‌سازیِ شبکهٔ عصبیِ دیگری که یاد گرفته‌ایم، توجه کاملاً نهان است. برنامه‌نویس دیکته نمی‌کند چه اطلاعاتی کی استفاده شود؛ مدل یاد می‌گیرد چه چیزی استفاده کند. توجه را همچنین می‌توان با RNNهای چندلایه ترکیب کرد. معمولاً در آن صورت توجه در هر لایه اعمال می‌شود.

صورتبندیِ احتمالاتیِ سافتمکسی برای توجه سه هدف را خدمت می‌کند. نخست، توجه را مشتق‌پذیر می‌کند، که برای استفاده‌اش با پسانتشار لازم است. حتی اگر خودِ توجه وزنِ آموخته‌ای نداشته باشد، گرادیان‌ها همچنان از طریقِ توجه به RNNهای مبدأ و مقصد به عقب جریان می‌یابند. دوم، صورتبندیِ احتمالاتی به مدل اجازه می‌دهد انواعِ خاصی از بافت‌سازیِ دوربرد را دربرگیرد که ممکن است توسطِ RNN مبدأ دربرگرفته نشده باشد، زیرا توجه می‌تواند کلِ دنبالهٔ مبدأ

را یک‌جا در نظر بگیرد، و یاد بگیرد آنچه مهم است را نگه دارد و بقیه را نادیده بگیرد. سوم، توجهِ احتمالاتی به شبکه اجازه می‌دهد عدم‌قطعیت را بازنمایی کند—اگر شبکه دقیقاً نداند کدام واژهٔ مبدأ را بعد ترجمه کند، می‌تواند احتمال‌های توجه را رویِ چند گزینه توزیع کند و سپس واژه را در واقع با ِRNN مقصد انتخاب کند. برخلافِ بیشترِ مؤلفه‌های شبکه‌های عصبی، احتمال‌های توجه اغلب توسط انسان‌ها قابلِ تفسیر و به‌صورتِ شهودی معنادارند. برای نمونه، در موردِ ترجمهٔ ماشینی، احتمال‌های توجه اغلب با ترازهای واژه‌به‌واژه‌ای که انسان تولید می‌کرد متناظرند. این در شکل ۲۴.۷ (ب) نشان داده شده.

مدل‌های دنباله‌به‌دنباله برای ترجمهٔ ماشینی طبیعی‌اند، اما تقریباً هر تکلیفِ زبانِ طبیعی را می‌توان به‌صورتِ مسئلهٔ دنباله‌به‌دنباله رمز کرد. برای نمونه، سامانهٔ پاسخ‌گوییِ پرسش را می‌توان رویِ ورودیِ شاملِ پرسشی که جداکننده‌ای و سپس پاسخ دنبالش می‌کند آموزش داد.

۲۴.۳.۲ رمزگشایی

در زمانِ آموزش، مدلِ دنباله‌به‌دنباله می‌کوشد احتمالِ هر واژهٔ جملهٔ مقصدِ آموزشی را، مشروط بر مبدأ و همهٔ واژه‌های مقصدِ قبلی، بیشینه کند. پس از کامل‌شدنِ آموزش، جملهٔ مبدأی به ما داده می‌شود، و هدفِ ما تولیدِ جملهٔ مقصدِ متناظر است. چنان‌که در شکل ۲۴.۷ نشان داده شده، می‌توانیم واژه‌های مقصد را یکی‌یکی تولید کنیم و سپس واژه‌ای را که تولید کرده‌ایم در گامِ زمانیِ بعد به عقب تغذیه کنیم. این رویه «رمزگشایی» (Decoding) نامیده می‌شود.

ساده‌ترین شکلِ رمزگشایی انتخابِ واژهٔ با بیشترین احتمال در هر گامِ زمانی و سپس تغذیهٔ این واژه به‌عنوانِ ورودیِ گامِ زمانیِ بعد است. این «رمزگشاییِ حریصانه» (Greedy Decoding) نامیده می‌شود، زیرا پس از تولیدِ هر واژهٔ مقصد، سامانه به‌طورِ کامل به فرضیه‌ای که تاکنون تولید کرده متعهد شده. مشکل این است که هدفِ رمزگشایی بیشینه‌کردنِ احتمالِ کلِ دنبالهٔ مقصد است، که رمزگشاییِ حریصانه ممکن است به آن نرسد. برای نمونه، ترجمهٔ جملهٔ انگلیسیِ قبلی، «The front door is red»، به اسپانیایی با رمزگشایِ حریصانه را در نظر بگیر. ترجمهٔ درست «La puerta de entrada es roja» است—تحت‌اللفظی «The door of entry is red» («درِ ورودی قرمز است»). فرض کنید ِRNN مقصد به‌درستی نخستین واژهٔ La را برای The تولید کند. بعد، رمزگشایِ حریصانه ممکن است entrada را برای front پیشنهاد کند. اما این خطاست—ترتیبِ واژهٔ اسپانیایی باید اسمِ puerta را پیش از توصیف‌کننده بگذارد. رمزگشاییِ حریصانه سریع است—فقط یک انتخاب در هر گامِ زمانی در نظر می‌گیرد و می‌تواند سریع انجامش دهد—اما مدل هیچ سازوکاری برای تصحیحِ اشتباه‌ها ندارد.

می‌توانستیم تلاش کنیم سازوکارِ توجه را بهبود دهیم تا همیشه به واژهٔ درست توجه کند و هر بار درست حدس بزند. اما برای بسیاری از جمله‌ها حدسِ درستِ همهٔ واژه‌ها در آغازِ جمله تا وقتی پایانِ جمله را ندیده‌اید ناممکن است.

رویکردِ بهتر جستجو برای رمزگشاییِ بهینه (یا دست‌کم خوب) با یکی از الگوریتم‌های جستجویِ فصل ۳ است. انتخابِ رایج جستجویِ پرتویی است (بخش ۴.۱.۳ را ببینید). در بافتِ رمزگشاییِ MT، جستجویِ پرتویی معمولاً k فرضیهٔ برتر را در هر مرحله نگه می‌دارد، هر یک را با یک واژه با استفاده از k انتخابِ برترِ واژه بسط می‌دهد، سپس بهترین k فرضیه را از k^2 فرضیهٔ تازهٔ حاصل انتخاب می‌کند. وقتی همهٔ فرضیه‌های پرتو توکنِ خاصِ <end> را تولید کنند، الگوریتم فرضیهٔ با بیشترین امتیاز را خروجی می‌دهد.

تجسمی از جستجویِ پرتویی در شکل ۲۴.۸ داده شده. هرچه مدل‌های یادگیری عمیق دقیق‌تر می‌شوند، معمولاً می‌توانیم از اندازهٔ پرتویِ کوچک‌تری استفاده کنیم. مدل‌های MT عصبیِ پیشتازِ کنونی از اندازهٔ پرتویِ ۴ تا ۸ استفاده می‌کنند، در حالی که نسلِ قدیمی‌ترِ مدل‌های MT آماری از اندازهٔ پرتویِ ۱۰۰ یا بیشتر استفاده می‌کردند.

شکل ۲۴.۸ — جستجویِ پرتویی با اندازهٔ پرتویِ b = 2. امتیازِ هر واژه لگاریتمِ احتمالِ تولیدشده توسط سافتمکسِ RNN مقصد است، و امتیازِ هر فرضیه جمعِ امتیازهای واژه است. نمودار چهار گامِ زمانی را نشان می‌دهد: در هر گام، هر فرضیهٔ پرتو با دو واژهٔ نامزد بسط می‌یابد و دو فرضیهٔ برترِ حاصل به پرتویِ گامِ بعد می‌روند (گام‌ها: [start]، سپس La، سپس La entrada و La puerta، و سپس بسط‌های بعدی مانندِ La puerta de). در گامِ زمانیِ ۳، فرضیهٔ با بیشترین امتیازِ La entrada فقط می‌تواند ادامه‌های کم‌احتمال تولید کند، پس «از پرتو می‌افتد»؛ در پایان، فرضیهٔ La puerta ... با امتیازِ 22.1 برنده است.

۲۴.۴ معماریِ ترنسفورمر

مقالهٔ تأثیرگذارِ «توجه تنها چیزی است که نیاز دارید» (واسوانی و همکاران، 2018) معماریِ ترنسفورمر (Transformer) را معرفی کرد، که از سازوکارِ «خودتوجهی» (Self-Attention) استفاده می‌کند که می‌تواند بافتِ دوربرد را بدونِ وابستگیِ ترتیبی مدل کند.

۲۴.۴.۱ خودتوجهی

پیش‌تر، در مدل‌های دنباله‌به‌دنباله، توجه از RNN مقصد به RNN مبدأ اعمال می‌شد. خودتوجهی این سازوکار را بسط می‌دهد تا هر دنباله از حالت‌های پنهان به خودش نیز توجه کند—مبدأ به مبدأ، و مقصد به مقصد. این به مدل اجازه می‌دهد به‌علاوه بافتِ دوربرد (و نزدیک) را درونِ هر دنباله دربرگیرد.

سرراست‌ترین راهِ اعمالِ خودتوجهی جایی است که ماتریسِ توجه مستقیماً توسط ضربِ نقطه‌ایِ بردارهای ورودی تشکیل می‌شود. اما این مشکل‌ساز است. ضربِ نقطه‌ایِ یک بردار با خودش همیشه بالا خواهد بود، پس هر حالتِ پنهان به سمتِ توجه به خودش اریب خواهد بود. ترنسفورمر این را با تصویرکردنِ نخستِ ورودی به سه بازنماییِ متفاوت با سه ماتریسِ وزنِ متفاوت حل می‌کند:

- «بردارِ پرس‌وجو» (Query Vector) $q_i = W_q\ x_i$ برداری است که از آن توجه می‌شود، مانندِ مقصد در سازوکارِ توجهِ استاندارد.
- «بردارِ کلید» (Key Vector) $k_i = W_k\ x_i$ برداری است که به آن توجه می‌شود، مانندِ مبدأ در سازوکارِ توجهِ پایه.
- «بردارِ مقدار» (Value Vector) $v_i = W_v\ x_i$ بافتی است که تولید می‌شود.

در سازوکارِ توجهِ استاندارد، شبکه‌های کلید و مقدار یکسان‌اند، اما به‌صورتِ شهودی منطقی است این‌ها بازنمایی‌های جداگانه باشند. نتایجِ رمزگذاریِ اُامین واژه، c_i، را می‌توان با اعمالِ سازوکارِ توجه بر بردارهای تصویرشده محاسبه کرد:

$$r_{ij} = (q_i \cdot k_j) / \sqrt{d}$$
$$a_{ij} = e^{r_{ij}} / (\sum_k e^{r_{ik}})$$
$$c_i = \sum_j a_{ij} \cdot v_j$$

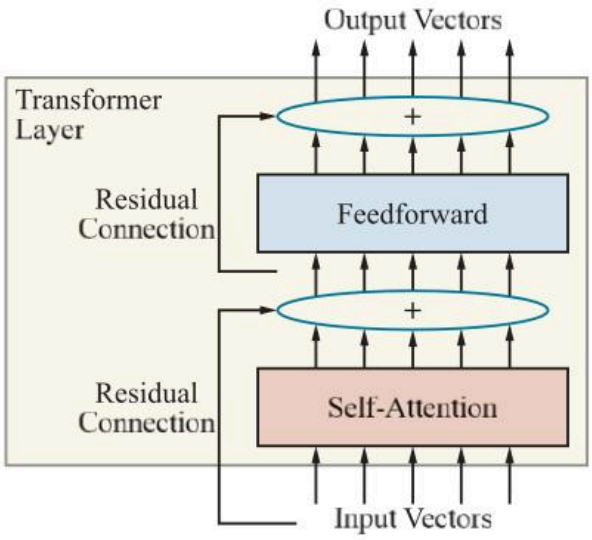
که d بُعدِ k و q است. توجه کنید i و j شاخص‌هایی در همان جمله‌اند، زیرا بافت را با خودتوجهی رمز می‌کنیم. در هر لایهٔ ترنسفورمر، خودتوجهی از بردارهای پنهانِ لایهٔ قبلی استفاده می‌کند، که در آغاز لایهٔ جاسازی است.

چند جزئیات در این‌جا شایانِ ذکر است. نخست از همه، سازوکارِ خودتوجهی نامتقارن است، زیرا r_{ij} با r_{ji} متفاوت است. دوم، عاملِ مقیاسِ $\sqrt{d}$ برای بهبودِ پایداریِ عددی اضافه شده. سوم، رمزگذاریِ همهٔ واژه‌های یک جمله را می‌توان به‌طورِ هم‌زمان محاسبه کرد، زیرا معادله‌های بالا را می‌توان با عملیاتِ ماتریسی بیان کرد که رویِ سخت‌افزارِ تخصصیِ مدرن به‌صورتِ کارآمد و موازی محاسبه می‌شوند.

انتخابِ اینکه کدام بافت استفاده شود کاملاً از مثال‌های آموزشی یاد گرفته می‌شود، نه از پیش‌مشخص‌شده. خلاصهٔ مبتنی بر بافت، c_i، جمعی رویِ همهٔ موقعیت‌های قبلیِ جمله است. در تئوری، هر اطلاعاتی از جمله می‌تواند در c_i ظاهر شود، اما در عمل، گاهی اطلاعاتِ مهم گم می‌شود، زیرا اساساً رویِ کلِ جمله میانگین گرفته می‌شود. یکی از راه‌های پرداختن به این «توجهِ چندسر» (Multiheaded Attention) نامیده می‌شود. جمله را به m تکهٔ مساوی تقسیم می‌کنیم و مدلِ توجه را بر هر یک از m تکه اعمال می‌کنیم. هر تکه مجموعه وزنِ خودش را دارد. سپس نتایج به هم چسبانده می‌شوند تا c_i تشکیل شود. با چسباندن به‌جای جمع‌کردن، برجسته‌شدنِ زیرتکهٔ مهم را آسان‌تر می‌کنیم.

۲۴.۴.۲ از خودتوجهی تا ترنسفورمر

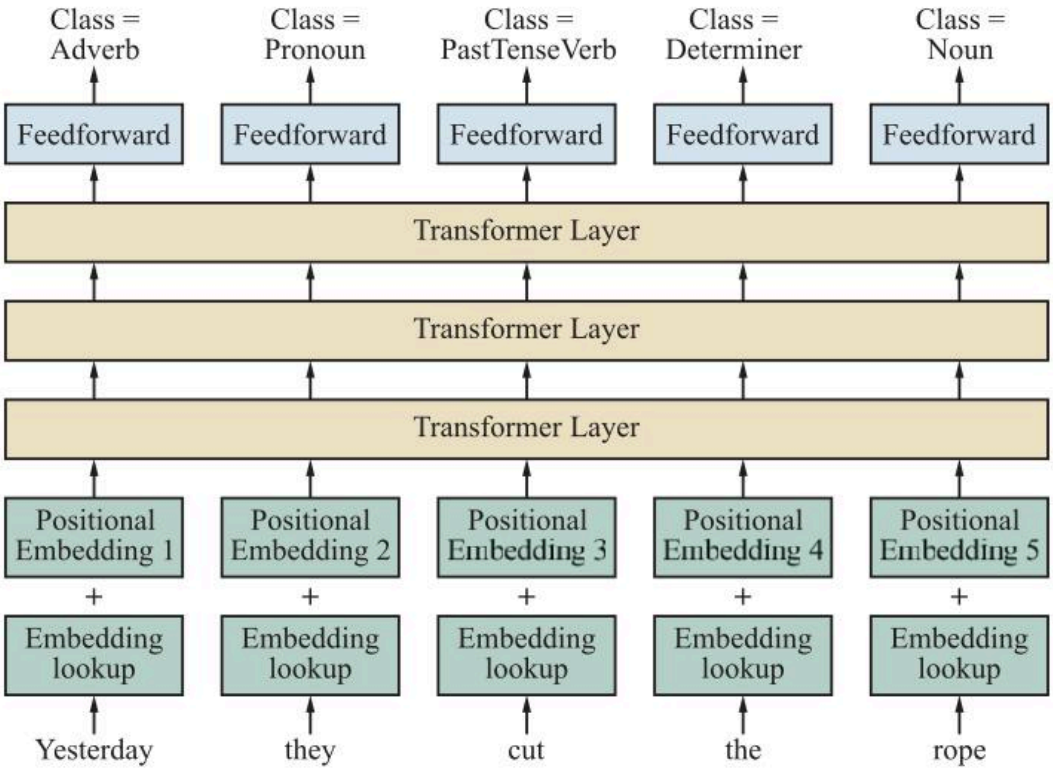
خودتوجهی فقط یکی از مؤلفه‌های مدلِ ترنسفورمر است. هر لایهٔ ترنسفورمر از چند لایهٔ فرعی تشکیل شده. در هر لایهٔ ترنسفورمر، نخست خودتوجهی اعمال می‌شود. خروجیِ مدولِ توجه از لایه‌های پیشخور عبور داده می‌شود، که در آن همان ماتریس‌های وزنِ پیشخور به‌صورتِ مستقل در هر موقعیت اعمال می‌شوند. تابعِ فعال‌سازیِ غیرخطی، معمولاً ReLU، پس از نخستین لایهٔ پیشخور اعمال می‌شود. برای پرداختن به مسئلهٔ بالقوهٔ گرادیانِ ناپدیدشونده، دو اتصالِ باقی‌مانده به لایهٔ ترنسفورمر اضافه می‌شود.



شکل ۲۴.۹ – ترنسفورمر تک‌لایه از خودتوجهی، شبکهٔ پیشخور و اتصال‌های باقی‌مانده تشکیل شده.

در عمل، مدل‌های ترنسفورمر معمولاً شش لایه یا بیشتر دارند. مانندِ دیگر مدل‌هایی که یاد گرفته‌ایم، خروجیِ لایهٔ i به‌عنوانِ ورودیِ لایهٔ i+1 استفاده می‌شود.

معماریِ ترنسفورمر ترتیبِ واژه‌ها در دنباله را به‌صورتِ صریح درنمی‌گیرد، زیرا بافت فقط از طریقِ خودتوجهی مدل می‌شود، که نسبت به ترتیبِ واژه بی‌تفاوت است. برای دربرگرفتنِ ترتیبِ واژه‌ها، ترنسفورمر از فنّی به نامِ «جاسازیِ موقعیتی» (Positional Embedding) استفاده می‌کند. اگر دنبالهٔ ورودیِ ما حداکثرِ طولِ n داشته باشد، n بردارِ جاسازیِ تازه یاد می‌گیریم—یکی برای هر موقعیتِ واژه. ورودیِ نخستین لایهٔ ترنسفورمر جمعِ جاسازیِ واژه در موقعیتِ t به‌علاوهٔ جاسازیِ موقعیتیِ متناظر با موقعیتِ t است.



شکل ۲۴.۱۰ – استفاده از معماریِ ترنسفورمر برای برچسب‌گذاریِ POS، اعمال‌شده بر همان جملهٔ استفاده‌شده در شکل ۲۴.۳. در پایین، جاسازیِ واژه و جاسازی‌های موقعیتی جمع می‌شوند تا ورودیِ ترنسفورمرِ سه‌لایه را تشکیل دهند. ترنسفورمر برای هر واژه یک بردار تولید می‌کند، مانندِ برچسب‌گذاریِ POS مبتنی بر RNN. هر بردار به لایهٔ خروجیِ نهایی و لایهٔ سافتمکس تغذیه می‌شود تا توزیعِ احتمالی رویِ برچسب‌ها تولید شود.

در این بخش، در واقع فقط نیمی از داستانِ ترنسفورمر را گفته‌ایم: مدلی که این‌جا توصیف کردیم «رمزگذارِ ترنسفورمر» (Transformer Encoder) نامیده می‌شود. برای تکلیف‌های رده‌بندیِ متن مفید است. معماریِ کاملِ ترنسفورمر در اصل به‌عنوانِ مدلِ دنباله‌به‌دنباله برای ترجمهٔ ماشینی طراحی شده بود. بنابراین، علاوه بر رمزگذار، شاملِ «رمزگشایِ ترنسفورمر» (Transformer Decoder) نیز هست. رمزگذار و رمزگشا تقریباً یکسان‌اند، با این تفاوت که رمزگشا از نسخه‌ای از خودتوجهی استفاده می‌کند که هر واژه فقط می‌تواند به واژه‌های قبل از خودش توجه کند، زیرا متن چپ-به-راست تولید می‌شود. رمزگشا همچنین در هر لایهٔ ترنسفورمر مدولِ توجهِ دومی دارد که به خروجیِ رمزگذارِ ترنسفورمر توجه می‌کند.

۲۴.۵ پیش‌آموزش و یادگیریِ انتقالی

به‌دست‌آوردنِ دادهٔ کافی برای ساختنِ مدلِ مقاوم می‌تواند چالش باشد. در بینایی رایانه (فصل ۲۵ را ببینید)، آن چالش با گردآوریِ مجموعه‌های بزرگِ تصاویر (مانندِ ImageNet) و برچسب‌زدنِ دستیِ آن‌ها پرداخته شد.

برای زبانِ طبیعی، کار با متنِ بدونِ برچسب رایج‌تر است. تفاوت تا حدی به‌دلیلِ دشواریِ برچسب‌گذاری است: کارگرِ ناآموخته می‌تواند به‌راحتی تصویری را «گربه» یا «غروب» برچسب بزند، اما برای حاشیه‌نویسیِ جمله‌ای با برچسب‌های نقشِ دستوری یا درخت‌های تجزیه آموزشِ گسترده‌ای لازم است. تفاوت همچنین به‌دلیلِ فراوانیِ متن است: اینترنت

هر روز بیش از ۱۰۰ میلیارد واژهٔ متن می‌افزاید، از جملهِ کتاب‌های دیجیتالی‌شده، منابعِ ویرایش‌شده مانندِ ویکی‌پدیا، و پست‌های رسانهٔ اجتماعیِ ویرایش‌نشده.

پروژه‌هایی مانندِ Common Crawl دسترسیِ آسان به این داده فراهم می‌کنند. هر متنِ روانی را می‌توان برای ساختنِ مدل‌های n-گرم یا جاسازیِ واژه استفاده کرد، و برخی متن‌ها با ساختاری می‌آیند که می‌تواند برای انواعِ تکلیف‌ها کمک‌کننده باشد—برای نمونه، سایت‌های پرسش‌های متداول (FAQ) بسیاری با جفت‌های پرسش-پاسخ هستند که می‌توان برای آموزشِ سامانهٔ پاسخ‌گوییِ پرسش استفاده کرد. به‌همین‌سان، بسیاری از وب‌سایت‌ها ترجمه‌های کنارِ هم متن‌ها منتشر می‌کنند، که می‌توان برای آموزشِ سامانه‌های ترجمهٔ ماشینی استفاده کرد. برخی متن‌ها حتی با برچسب‌هایی از نوعی می‌آیند، مانندِ سایت‌های نقد که کاربران نقدهای متنی‌شان را با سامانهٔ امتیازِ پنج‌ستاره حاشیه‌نویسی می‌کنند.

ترجیح می‌دهیم هر بار که مدلِ NLP تازه‌ای می‌خواهیم مجبور به زحمتِ ایجادِ مجموعهٔ دادهٔ تازه‌ای نباشیم. در این بخش، ایدهٔ «پیش‌آموزش» (Pretraining) را معرفی می‌کنیم: شکلی از یادگیریِ انتقالی (بخش ۲۱.۷.۲ را ببینید) که در آن از حجمِ زیادی دادهٔ زبانیِ مشترکِ حوزه‌عمومی برای آموزشِ نسخهٔ اولیهٔ مدلِ NLP استفاده می‌کنیم. از آن نقطه، می‌توانیم از مقدارِ کمتری دادهٔ حوزه‌خاص (شاید شاملِ برخی دادهٔ برچسب‌خورده) برای پالایشِ مدل استفاده کنیم. مدلِ پالایش‌شده می‌تواند واژگان، اصطلاح‌ها، ساختارهای نحوی و دیگر پدیده‌های زبانیِ خاصِ حوزهٔ تازه را یاد بگیرد.

۲۴.۵.۱ جاسازی‌های واژهٔ ازپیش‌آموزش‌دیده

در بخش ۲۴.۱، جاسازی‌های واژه را به‌طورِ مختصر معرفی کردیم. دیدیم واژه‌های مشابه مانندِ موز و سیب چگونه به بردارهای مشابه می‌رسند، و دیدیم چگونه می‌توان مسائلِ قیاس را با تفریقِ برداری حل کرد. این نشان می‌دهد جاسازی‌های واژه اطلاعاتِ قابلِ توجهی دربارهٔ واژه‌ها درمی‌گیرند.

در این بخش به جزئیاتِ چگونگیِ ایجادِ جاسازی‌های واژه با فرایندی کاملاً نظارت‌نشده رویِ پیکرهٔ بزرگِ متن شیرجه می‌زنیم. این در تقابل با جاسازی‌های بخشِ ۲۴.۱ است، که در طولِ فرایندِ برچسب‌گذاریِ نظارت‌شدهٔ نقشِ دستوری ساخته می‌شدند، و بنابراین به برچسب‌های POS نیاز داشتند که از حاشیه‌نویسیِ دستیِ گران می‌آیند.

بر مدلِ خاصی برای جاسازیِ واژه تمرکز می‌کنیم، مدلِ GloVe (بردارهای سراسری). مدل با گردآوریِ شمارشِ دفعاتِ ظهورِ هر واژه در پنجره‌ای از واژه‌ای دیگر، شبیهِ مدلِ اسکیپ-گرم، شروع می‌کند. نخست اندازهٔ پنجره را انتخاب کن (شاید ۵ واژه) و X_{ij} تعدادِ دفعاتی باشد که واژه‌های i و j درونِ پنجره‌ای هم‌رخداد می‌کنند، و X_i تعدادِ دفعاتی باشد که واژهٔ i با هر واژهٔ دیگر هم‌رخداد می‌کند. $P_{ij} = X_{ij}/X_i$ احتمالِ ظهورِ واژهٔ j در بافتِ واژهٔ i باشد. مثلِ قبل، E_i جاسازیِ واژه برای واژهٔ i باشد.

بخشی از بینشِ مدلِ GloVe این است که رابطهٔ بینِ دو واژه را بهتر می‌توان با مقایسهٔ هر دو با واژه‌های دیگر دربرگرفت. واژه‌های ice و steam را در نظر بگیر. حالا نسبتِ احتمال‌های هم‌رخدادِشان با واژهٔ دیگری مانندِ w را در نظر بگیر، یعنی:

.P_{w,ice} / P_{w,steam}

وقتی w واژهٔ solid باشد نسبت بالا خواهد بود (یعنی solid بیشتر به ice می‌چسبد) و وقتی w واژهٔ gas باشد پایین خواهد بود (یعنی gas بیشتر به steam می‌چسبد). و وقتی w واژهٔ غیرمحتوایی مانندِ the، واژه‌ای مانندِ water که به‌یکسان به هر دو مربوط است، یا واژه‌ای به‌یکسان نامربوط مانندِ fashion باشد، نسبت نزدیکِ ۱ خواهد بود.

مدلِ GloVe با این بینش شروع می‌کند و از استدلالِ ریاضی‌ای (پنینگتون و همکاران، 2014) عبور می‌کند که نسبت‌های احتمال را به تفاضل‌های برداری و ضرب‌های نقطه‌ای تبدیل می‌کند، و در نهایت به قیدِ زیر می‌رسد:

$$E_i \cdot E'_k = \log(P_{ij})$$

به بیانِ دیگر، ضربِ نقطه‌ایِ دو بردارِ واژه برابرِ لگاریتمِ احتمالِ هم‌رخدادِشان است. این به‌صورتِ شهودی منطقی است: دو بردارِ تقریباً متعامد ضربِ نقطه‌ایِ نزدیکِ ۰ دارند، و دو بردارِ نرمال‌شدهٔ تقریباً یکسان ضربِ نقطه‌ایِ نزدیکِ ۱ دارند. پیچیدگیِ فنی‌ای هست که در آن مدلِ GloVe برای هر واژه دو بردارِ جاسازیِ واژه، E_i و E'_i، می‌سازد؛ محاسبهٔ هر دو و سپس جمع‌کردنِشان در پایان به محدودکردنِ بیش‌برازش کمک می‌کند.

آموزشِ مدلی مانندِ GloVe معمولاً بسیار کم‌هزینه‌تر از آموزشِ شبکهٔ عصبیِ استاندارد است: مدلِ تازه‌ای را می‌توان از میلیاردها واژهٔ متن در چند ساعت با CPU رومیزیِ استاندارد آموزش داد. آموزشِ جاسازیِ واژه رویِ حوزه‌ای خاص و بازیابیِ دانشِ آن حوزه ممکن است. برای نمونه، تشیتویان و همکاران (2019) از ۳٫۳ میلیون چکیدهٔ علمی دربارهٔ موضوعِ علمِ مواد برای آموزشِ مدلِ جاسازیِ واژه استفاده کردند. آن‌ها دریافتند، درست همان‌گونه که دیدیم مدلِ جاسازیِ واژهٔ عمومی می‌تواند به «آتن به یونان، همچنان که اسلو به چه؟» با «نروژ» پاسخ دهد، مدلِ علمِ موادِ آن‌ها می‌تواند به «NiFe به فرومغناطیسی، همچنان که IrMn به چه؟» با «ضدفرومغناطیسی» پاسخ دهد. مدلِ آن‌ها صرفاً به هم‌رخدادِ واژه‌ها تکیه ندارد؛ به نظر می‌رسد دانشِ علمیِ پیچیده‌تری را درمی‌گیرد. وقتی پرسیده می‌شود چه ترکیباتِ شیمیایی‌ای را می‌توان «گرمابرقی» یا «عایقِ توپولوژیک» رده‌بندی کرد، مدلِ آن‌ها قادر به پاسخِ درست است. برای نمونه، CsAgGa2Se4 هرگز نزدیکِ «گرمابرقی» در پیکره ظاهر نمی‌شود، اما نزدیکِ «کالکوژنید»، «گافِ نواری» و «اپتوالکتریک» ظاهر می‌شود، که همگی سرنخ‌هایی‌اند که رده‌بندیِ آن را به‌عنوانِ مشابهِ «گرمابرقی» ممکن می‌کنند. افزون بر این، وقتی فقط رویِ چکیده‌های تا سالِ ۲۰۰۸ آموزش دیده و خواسته شود ترکیباتی را برگزیند که «گرمابرقی»اند اما هنوز در چکیده‌ها ظاهر نشده‌اند، سه مورد از پنج انتخابِ برترِ مدل در مقاله‌های منتشرشده بینِ ۲۰۰۹ و ۲۰۱۹ گرمابرقی کشف شدند.

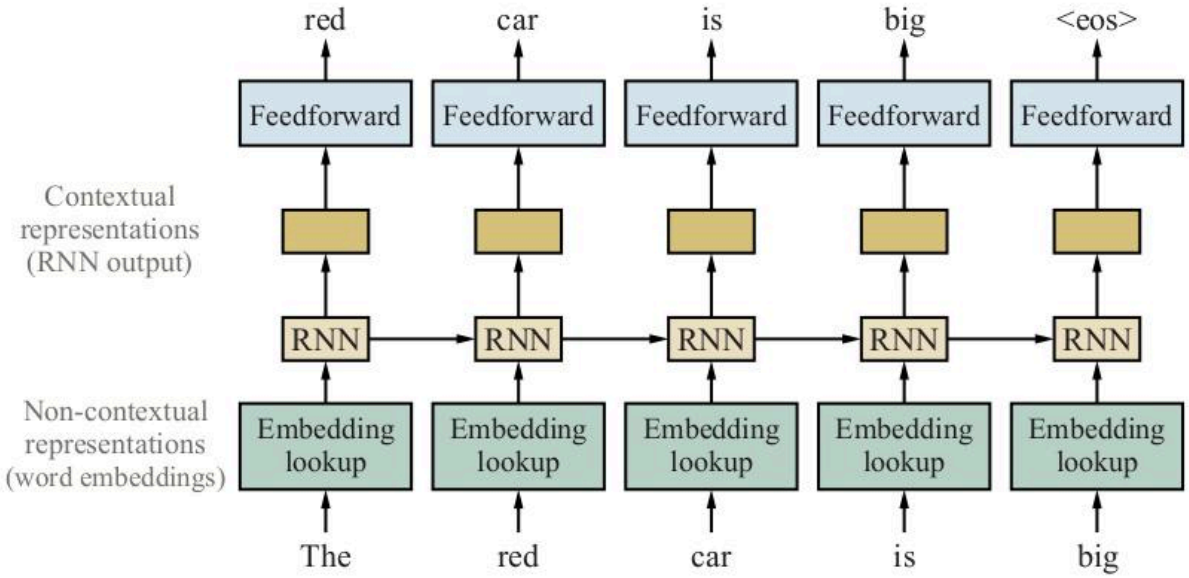
۲۴.۵.۲ بازنمایی‌های بافتیِ ازپیش‌آموزش‌دیده

جاسازی‌های واژه بازنمایی‌های بهتری از توکن‌های واژهٔ اتمی‌اند، اما مسئلهٔ مهمی با واژه‌های چندمعنا هست. برای نمونه، واژهٔ rose می‌تواند به گل اشاره کند یا زمانِ گذشتهٔ rise باشد. پس انتظار داریم دستِ‌کم دو خوشهٔ کاملاً متمایز از بافت‌های واژه برای rose پیدا کنیم: یکی مشابهِ نامِ گل‌ها مانندِ گلِ کوکب، و یکی مشابهِ خیزش. هیچ بردارِ جاسازیِ تکی نمی‌تواند هر دو را هم‌زمان دربرگیرد. Rose مثالِ روشنی از واژه‌ای با (دستِ‌کم) دو معنای متمایز است، اما

واژه‌های دیگر سایه‌های ظریفِ معنایی دارند که به بافت بستگی دارد، مانندِ واژهٔ need در «تو باید این فیلم را ببینی» در برابرِ «انسان‌ها برای بقا به اکسیژن نیاز دارند». و برخی عبارت‌های اصطلاحی مانندِ break the bank بهتر است به‌عنوانِ کل تحلیل شوند تا واژه‌های سازنده.

بنابراین، به‌جای یادگیریِ صرفِ جدولِ واژه-به-جاسازی، می‌خواهیم مدلی آموزش دهیم که «بازنمایی‌های بافتی» (Contextual Representations) هر واژهٔ جمله را تولید کند. بازنماییِ بافتی هم واژه و هم بافتِ اطرافِ واژه‌ها را به بردارِ جاسازیِ واژه نگاشت می‌کند. به بیانِ دیگر، اگر این مدل را با واژهٔ rose و بافتِ «the gardener planted a rose bush» («باغبان بوتهٔ گلی کاشت») تغذیه کنیم، باید جاسازیِ بافتی تولید کند که شبیهِ (اما نه لزوماً یکسانِ) بازنماییِ به‌دست‌آمده با بافتِ «the cabbage rose had an unusual fragrance» («گلِ کلمی عطرِ غیرعادی داشت») باشد، و بسیار متفاوت از بازنماییِ rose در بافتِ «the river rose five feet».

شکل ۲۴.۱۱ شبکهٔ بازگشتی را نشان می‌دهد که جاسازی‌های واژهٔ بافتی ایجاد می‌کند—جعبه‌های بدونِ برچسبِ شکل. فرض می‌کنیم از قبل مجموعه‌ای از جاسازی‌های واژهٔ غیربافتی ساخته‌ایم. هر بار یک واژه تغذیه می‌کنیم و از مدل می‌خواهیم واژهٔ بعدی را پیش‌بینی کند. پس برای نمونه در شکل، در نقطه‌ای که به واژهٔ «car» رسیده‌ایم، گرهٔ RNN آن گامِ زمانی دو ورودی دریافت می‌کند: جاسازیِ واژهٔ غیربافتیِ «car» و بافت، که اطلاعاتِ واژه‌های قبلیِ «The red» را رمز می‌کند. سپس گرهٔ RNN بازنماییِ بافتی برای «car» خروجی می‌دهد. شبکه به‌عنوانِ کل سپس پیش‌بینیِ واژهٔ بعدی، «is»، را خروجی می‌دهد. سپس وزن‌های شبکه را برای کمینه‌کردنِ خطایِ بینِ پیش‌بینی و واژهٔ بعدیِ واقعی به‌روزرسانی می‌کنیم.



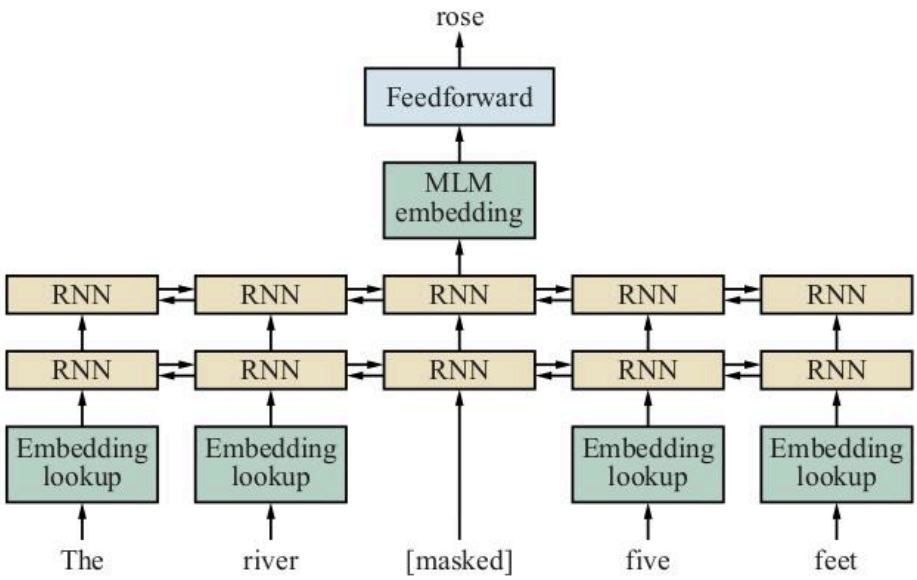
شکل ۲۴.۱۱ — آموزشِ بازنمایی‌های بافتی با مدلِ زبانِ چپ-به-راست.

این مدل شبیهِ مدلِ برچسب‌گذاریِ POS در شکل ۲۴.۵ است، با دو تفاوتِ مهم. نخست، این مدل یک‌سویه (چپ-به-راست) است، در حالی که مدلِ POS دوسویه است. دوم، به‌جای پیش‌بینیِ برچسب‌های POS برای واژهٔ کنونی، این مدل واژهٔ بعدی را با استفاده از بافتِ قبلی پیش‌بینی می‌کند. پس از ساخته‌شدنِ مدل، می‌توانیم از آن برای بازیابیِ

بازنمایی‌های واژه‌ها استفاده کنیم و آن‌ها را به تکلیفِ دیگری منتقل کنیم؛ نیازی به ادامهٔ پیش‌بینیِ واژهٔ بعدی نیست. توجه کنید محاسبهٔ بازنماییِ بافتی همیشه به دو ورودی نیاز دارد، واژهٔ کنونی و بافت.

۲۴.۵.۳ مدل‌های زبانِ نقاب‌دار

ضعفِ مدل‌های زبانِ استاندارد مانندِ مدل‌های n-گرم این است که بافت‌سازیِ هر واژه فقط بر واژه‌های قبلیِ جمله مبتنی است. پیش‌بینی‌ها چپ-به-راست انجام می‌شوند. اما گاهی بافتِ بعدیِ جمله—برای نمونه، feet در عبارتِ rose five feet—به روشن‌کردنِ واژه‌های قبلی کمک می‌کند.



شکل ۲۴.۱۲ – مدل‌سازیِ زبانِ نقاب‌دار: با پوشاندنِ واژه‌های ورودی و پیش‌بینیِ فقط آن واژه‌های پوشیده‌شده، مدلِ دوسویه‌ای—مثلاً RNN چندلایه—پیش‌آموزش بده.

یک راه‌حلِ سرراست آموزشِ مدلِ زبانِ راست-به-چپِ جداگانه‌ای است که هر واژه را بر مبنای واژه‌های بعدیِ جمله بافت‌سازی می‌کند، و سپس چسباندنِ بازنمایی‌های چپ-به-راست و راست-به-چپ. اما چنین مدلی در ترکیبِ شاهد از هر دو جهت ناکام می‌ماند. در عوض، می‌توانیم از «مدلِ زبانِ نقاب‌دار» (Masked Language Model یا MLM) استفاده کنیم. MLMها با «نقاب‌زدن» (پنهان‌کردنِ) واژه‌های منفرد در ورودی و خواستن از مدل برای پیش‌بینیِ واژه‌های نقاب‌شده آموزش می‌بینند. برای این تکلیف، می‌توان از RNN دوسویهٔ عمیق یا ترنسفورمر رویِ جملهٔ نقاب‌شده استفاده کرد.

برای نمونه، با توجه به جملهٔ ورودیِ «The river rose five feet»، می‌توانیم واژهٔ میانی را نقاب بزنیم تا «The river five feet» به‌دست آید و از مدل بخواهیم جایِ خالی را پر کند. سپس از بردارهای پنهانِ نهاییِ متناظر با توکن‌های نقاب‌شده برای پیش‌بینیِ واژه‌هایی که نقاب زده شده‌اند استفاده می‌شود—در این مثال، rose. در طولِ آموزش، می‌توان از یک جمله چند بار با واژه‌های متفاوتِ نقاب‌شده استفاده کرد. زیباییِ این رویکرد این است که به هیچ دادهٔ برچسب‌خورده‌ای نیاز ندارد؛ جمله برچسبِ خودش را برای واژهٔ نقاب‌شده فراهم می‌کند. اگر این مدل رویِ پیکرهٔ بزرگِ متن آموزش ببیند، بازنمایی‌های ازپیش‌آموزش‌دیده‌ای تولید می‌کند که در طیفِ وسیعی از تکلیف‌های NLP (ترجمهٔ ماشینی، پاسخ‌گوییِ پرسش، خلاصه‌سازی، قضاوتِ دستوری‌بودن و دیگران) خوب عمل می‌کنند.

۲۴.۶ پیشتازترین‌ها

یادگیری عمیق و یادگیریِ انتقالی به‌طورِ محسوس وضعیتِ هنرِ NLP را پیش برده‌اند—تا حدی که یک مفسر در ۲۰۱۸ اعلام کرد «لحظهٔ ImageNet ِNLP فرا رسیده» (رودر، 2018). تلویح این است که همان‌گونه که در ۲۰۱۲ نقطهٔ عطفی برای بینایی رایانه رخ داد، وقتی سامانه‌های یادگیری عمیق نتایجِ به‌شکلِ شگفت‌آوری خوب در رقابتِ ImageNet تولید کردند، نقطهٔ عطفی در ۲۰۱۸ برای NLP رخ داد. انگیزهٔ اصلیِ این نقطهٔ عطف یافته‌ای بود که یادگیریِ انتقالی برای مسائلِ زبانِ طبیعی خوب کار می‌کند: مدلِ زبانِ عمومی را می‌توان دانلود کرد و برای تکلیفی خاص تنظیمِ دقیق کرد.

با جاسازی‌های واژهٔ ساده از سامانه‌هایی مانندِ WORD2VEC در ۲۰۱۳ و GloVe در ۲۰۱۴ شروع شد. پژوهشگران می‌توانند چنین مدلی را دانلود کنند یا نسبتاً سریع خودشان آموزش دهند، بدونِ دسترسی به ابررایانه‌ها. بازنمایی‌های بافتیِ ازپیش‌آموزش‌دیده، از سوی دیگر، مرتبه‌های بزرگی پرهزینه‌تر برای آموزش‌اند.

این مدل‌ها فقط پس از فراگیرشدنِ پیشرفت‌های سخت‌افزاری (GPUها و TPUها) ممکن شدند، و در این مورد پژوهشگران سپاسگزار بودند که می‌توانند مدل‌ها را دانلود کنند به‌جای اینکه مجبور باشند منابع را برای آموزشِ مدلِ خودشان صرف کنند. مدلِ ترنسفورمر آموزشِ کارآمدِ شبکه‌های عصبیِ بسیار بزرگ‌تر و عمیق‌تر از آنچه قبلاً ممکن بود ممکن کرد (این بار به‌دلیلِ پیشرفت‌های نرم‌افزاری، نه سخت‌افزاری). از ۲۰۱۸، پروژه‌های NLP ِتازه معمولاً با مدلِ ترنسفورمرِ ازپیش‌آموزش‌دیده شروع می‌شوند.

هرچند این مدل‌های ترنسفورمر برای پیش‌بینیِ واژهٔ بعدیِ متن آموزش دیده‌اند، در دیگر تکلیف‌های زبانی کارِ به‌شکلِ شگفت‌آوری خوبی انجام می‌دهند. مدلِ ROBERTA با کمی تنظیمِ دقیق به نتایجِ پیشتاز در آزمون‌های پاسخ‌گوییِ پرسش و درکِ مطلب می‌رسد (لیو و همکاران، 2019b). GPT-2، مدلِ زبانِ ترنسفورمرمانند با ۱٫۵ میلیارد پارامترِ آموزش‌دیده رویِ ۴۰ گیگابایت متنِ اینترنتی، در تکلیف‌های متنوعی مانندِ ترجمه بینِ فرانسوی و انگلیسی، یافتنِ مرجعِ وابستگی‌های دوربرد و پاسخ‌گوییِ پرسشِ دانش‌عمومی نتایجِ خوبی به‌دست می‌آورد، همه بدونِ تنظیمِ دقیق برای تکلیفِ خاص. چنان‌که شکل ۲۴.۱۴ نشان می‌دهد، GPT-2 می‌تواند با توجه به فقط چند واژه به‌عنوانِ سرنخ، متنِ نسبتاً قانع‌کننده‌ای تولید کند.

It is a truth universally acknowledged that the earth is not the center of the universe. There are those who assert there is. I do not accept them, but others I consider to be of the same opinion. The truth is, however, that if there are other than the center, and if there are any other living things in the universe and if they are not human, then we do not yet have our answers. We have to go on. This page gives a simplified, simplified answer to the problem. We don't have all the answers. The truth is, however, that the truth is out there.

When Gregor Samsa woke up one morning, he did not notice anything strange. "When my wife is looking at me, I feel like she is looking at a piece of art," he said. "I think she is admiring something I have created." The idea is that by looking at your own life, you learn something important and become a better person. It is a theory that emerged from psychologist Daniel Goleman's work, in which he asked "How do you know you're not a loser?"

Alice was beginning to get very tired of sitting with her sister on the bank. She sat up, yawned, and said, with a loud little scream, "I hope you don't mind if I keep on doing what I should like to do, and if someone asks me which of us will do more, don't tell them that I won't do much, my dear sister."

All happy families are alike; each happy family is like a garden of paradise. The only difference between happy families and unhappy families, is that the unhappy family doesn't have any flowers or trees.

Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Tell me a story. Please fill out the following details. Thank you... Thank you for your interest in this interview. Please wait...

شکل ۲۴.۱۴ – نمونه‌هایی از متن‌های تکمیل‌شدهٔ تولیدشده توسط مدلِ زبانِ GPT-2، با توجه به سرنخ‌های برجسته‌شده با حروفِ درشت. بیشترِ متن‌ها انگلیسیِ نسبتاً روان‌اند، دستِ‌کم به‌صورتِ محلی. مثالِ آخر نشان می‌دهد گاهی مدل صرفاً از کار می‌افتد. (نمونه‌ها با ترجمهٔ فارسی:)

- **«حقیقتی همگانی که پذیرفته شده این است که زمین مرکز جهان نیست.»** کسانی هستند که ادعا می‌کنند هست. من آن‌ها را نمی‌پذیرم، اما دیگران را هم‌نظر با خودم می‌دانم. حقیقت اما این است که اگر چیزی غیر از مرکز باشد، و اگر موجوداتِ زندهٔ دیگری در جهان باشند و انسان نباشند، آنگاه هنوز پاسخ‌هایمان را نداریم. باید ادامه دهیم. این صفحه پاسخِ ساده‌شده، ساده‌شده‌ای به مسئله می‌دهد. همهٔ پاسخ‌ها را نداریم. حقیقت اما این است که حقیقت آن‌جاست.
- **«وقتی گرگور زامزا یک صبح از خواب بیدار شد،»** هیچ چیزِ عجیبی متوجه نشد. «وقتی همسرم به من نگاه می‌کند، احساس می‌کنم به یک اثرِ هنری نگاه می‌کند،» گفت. «فکر می‌کنم او چیزی را که من خلق کرده‌ام تحسین می‌کند.» ایده این است که با نگاه‌کردن به زندگیِ خودت، چیزِ مهمی یاد می‌گیری و آدمِ بهتری می‌شوی. این نظریه‌ای است که از کارِ روان‌شناسِ دانیل گلمن برخاسته، که پرسید «از کجا می‌دانی بازنده نیستی؟»

- **«آلیس داشت از نشستن با خواهرش در کنارِ ساحل خیلی خسته می‌شد.»** نشست، خمیازه کشید، و با فریادِ کوچکِ بلندی گفت: «امیدوارم ناراحت نشوی اگر همان کاری را ادامه دهم که دوست دارم انجام دهم، و اگر کسی از تو پرسید کدامِ ما بیشتر انجام می‌دهد، به او نگو که من زیاد انجام نمی‌دهم، خواهرِ عزیزم.»
- **«همهٔ خانواده‌های خوشبخت شبیهِ همانند؛»** هر خانوادهٔ خوشبخت مانند باغِ بهشتی است. تنها تفاوتِ بینِ خانواده‌های خوشبخت و ناخوشبخت این است که خانوادهٔ ناخوشبخت هیچ گلی یا درختی ندارد.
- **«یک داستان بگو.»** (تکرارِ دهبارهٔ «یک داستان بگو. یک داستان بگو. ...») لطفاً جزئیاتِ زیر را پر کنید. متشکریم... از علاقه‌ات به این مصاحبه متشکریم. لطفاً صبر کنید...

1. **What will best separate a mixture of iron filings and black pepper?**
   (a) magnet (b) filter paper (c) triple beam balance (d) voltmeter
2. **Which form of energy is produced when a rubber band vibrates?**
   (a) chemical (b) light (c) electrical (d) sound
3. **Because copper is a metal, it is**
   (a) liquid at room temperature (b) nonreactive with other substances
   (c) a poor conductor of electricity (d) a good conductor of heat
4. **Which process in an apple tree primarily results from cell division?**
   (a) growth (b) photosynthesis (c) gas exchange (d) waste removal

شکل ۲۴.۱۳ – پرسش‌هایی از آزمونِ علومِ کلاسِ هشتم که سامانهٔ ARISTO می‌تواند با استفاده از مجموعه‌ای از روش‌ها درست پاسخ دهد، که تأثیرگذارترینِ آن‌ها مدلِ زبانِ ROBERTA است. پاسخ‌دادن به این پرسش‌ها به دانشی دربارهٔ زبانِ طبیعی، ساختارِ آزمون‌های چندگزینه‌ای، فهمِ عقلِ سلیم و علوم نیاز دارد.

1. چه چیزی مخلوطِ براده‌های آهن و فلفلِ سیاه را بهتر جدا می‌کند؟ (الف) آهنربا (ب) کاغذِ صافی (ج) ترازوی سه‌پله (د) ولت‌سنج
2. وقتی کشِ لاستیکی می‌لرزد، کدام شکلِ انرژی تولید می‌شود؟ (الف) شیمیایی (ب) نور (ج) الکتریکی (د) صوت
3. چون مس فلز است، (الف) در دمای اتاق مایع است (ب) با موادِ دیگر واکنش‌ناپذیر است (ج) رسانای ضعیفِ الکتریسیته است (د) رسانای خوبِ گرماست
4. کدام فرایند در درختِ سیب عمدتاً از تقسیمِ سلولی ناشی می‌شود؟ (الف) رشد (ب) فتوسنتز (ج) تبادلِ گاز (د) حذفِ پسماند

به‌عنوانِ نمونهٔ سامانهٔ NLP پیشتاز، ARISTO (کلارک و همکاران، 2019) به امتیازِ 91.6٪ در آزمونِ علومِ چندگزینه‌ایِ کلاسِ هشتم رسید (شکل ۲۴.۱۳ را ببینید). ARISTO از مجموعه‌ای از حل‌کننده‌ها تشکیل شده: برخی از بازیابیِ اطلاعات استفاده می‌کنند (شبیهِ موتورِ جستجویِ وب)، برخی استلزامِ متنی و استدلالِ کیفی انجام می‌دهند، و برخی از مدل‌های زبانِ ترنسفورمرِ بزرگ استفاده می‌کنند. معلوم می‌شود ROBERTA به‌تنهایی 88.2٪ در آزمون امتیاز می‌گیرد. ARISTO همچنین ۸۳٪ در آزمونِ پیشرفته‌ترِ کلاسِ دوازدهم امتیاز می‌گیرد. (امتیازِ ۶۵٪ «برآورده‌کردنِ استانداردها» و ۸۵٪ «برآورده‌کردنِ استانداردها با تمایز» در نظر گرفته می‌شود.)

محدودیت‌هایی برای ARISTO هست. فقط با پرسش‌های چندگزینه‌ای سروکار دارد، نه پرسش‌های انشایی، و نه می‌تواند نمودارها را بخواند نه تولید کند.[1] T5 (ترنسفورمرِ انتقالِ متن‌به‌متن) برای تولیدِ پاسخ‌های متنی به انواعِ گوناگونِ ورودیِ متنی طراحی شده. شاملِ مدلِ ترنسفورمرِ رمزگذار-رمزگشایِ استانداردی است که رویِ ۳۵ میلیارد واژه از پیکرهٔ خزیدهٔ تمیزِ عظیمِ 750 گیگابایتی (C4) پیش‌آموزش دیده. این آموزشِ بدونِ برچسب برای دادنِ دانشِ زبانیِ قابلِ تعمیم به مدل طراحی شده که برای چندین تکلیفِ خاص مفید خواهد بود. سپس T5 برای هر تکلیف با ورودیِ شاملِ نامِ تکلیف، و سپس دونقطه و محتوایی، آموزش می‌بیند. برای نمونه، وقتی «translate English to German: That is good» داده می‌شود، خروجیِ «Das ist gut» تولید می‌کند. برای برخی تکلیف‌ها، ورودی نشانه‌گذاری می‌شود؛ برای نمونه در چالشِ وینوگراد، ورودی ضمیری را با مرجعِ مبهم برجسته می‌کند. با توجه به ورودیِ «referent: The city councilmen refused the demonstrators a permit because they feared violence» («مرجع: اعضای شورای شهر به معترضان اجازه ندادند، زیرا از خشونت می‌ترسیدند»)، پاسخِ درست «The city councilmen» (اعضای شورای شهر) است (نه «the demonstrators» — معترضان).

کارهای بسیاری برای بهبودِ سامانه‌های NLP باقی مانده. یکی از مسائل این است که مدل‌های ترنسفورمر فقط به بافتی باریک تکیه می‌کنند، محدود به چند صد واژه. برخی رویکردهای آزمایشی می‌کوشند آن بافت را بسط دهند؛ سامانهٔ Reformer (کیتایف و همکاران، 2020) می‌تواند بافتِ تا یک میلیون واژه را مدیریت کند.

نتایجِ اخیر نشان داده‌اند استفاده از دادهٔ آموزشیِ بیشتر به مدل‌های بهتر می‌انجامد—برای نمونه، ROBERTA پس از آموزش رویِ ۲٫۲ تریلیون واژه به نتایجِ پیشتاز رسید. اگر استفاده از دادهٔ متنیِ بیشتر بهتر است، چه می‌شد اگر انواعِ دیگری از داده را هم شامل می‌کردیم: پایگاه‌های دادهٔ ساخت‌یافته، دادهٔ عددی، تصاویر و ویدئو؟ برای آموزش رویِ پیکرهٔ بزرگِ ویدئو به پیشرفتی در سرعت‌های پردازشِ سخت‌افزاری نیاز می‌داشتیم، و ممکن است به چندین پیشرفت در هوش مصنوعی نیز نیاز داشته باشیم.

خوانندهٔ کنجکاو ممکن است تعجب کند چرا در فصلِ قبلی دربارهٔ دستورها، تجزیه و تفسیرِ معنایی یاد گرفتیم، فقط برای اینکه در این فصل آن مفاهیم را به نفعِ مدل‌های صرفاً داده‌محور کنار بگذاریم؟ در حالِ حاضر، پاسخ صرفاً این است که مدل‌های داده‌محور توسعه و نگهداری‌شان آسان‌تر است و در معیارهای استاندارد بهتر امتیاز می‌گیرند، در مقایسه با سامانه‌های دست‌ساخته‌ای که با مقدارِ معقولی تلاشِ انسانی با رویکردهای توصیف‌شده در فصل ۲۳ می‌توان ساخت. ممکن است مدل‌های ترنسفورمر و خویشاوندانشان بازنمایی‌های نهانی یاد می‌گیرند که همان ایده‌های پایهٔ دستورها و اطلاعاتِ معنایی را درمی‌گیرند، یا ممکن است چیزِ کاملاً متفاوتی درونِ این مدل‌های عظیم در جریان باشد؛ صرفاً نمی‌دانیم. می‌دانیم سامانه‌ای که با دادهٔ متنی آموزش دیده نگهداری و سازگاری‌اش با حوزه‌های تازه و زبان‌های طبیعیِ تازه آسان‌تر از سامانه‌ای است که بر ویژگی‌های دست‌ساخته تکیه دارد.

همچنین ممکن است پیشرفت‌های آینده در مدل‌سازیِ صریحِ دستوری و معنایی باعث شود آونگ به عقب بچرخد. شاید محتمل‌تر ظهورِ رویکردهای هیبریدی است که بهترین مفاهیمِ هر دو فصل را ترکیب می‌کنند. برای نمونه، کیتایف و کلاین (2018) از سازوکارِ توجه برای بهبودِ تجزیه‌گرِ سازه‌ایِ سنتی استفاده کردند و به بهترین نتیجه‌ای که تاکنون در مجموعهٔ آزمونِ درخت‌بانِ پن ثبت شده رسیدند. به‌همین‌سان، رینگ‌گارد و همکاران (2017) نشان می‌دهند تجزیه‌گرِ وابستگی چگونه می‌تواند با جاسازیِ واژه و شبکهٔ عصبیِ بازگشتی بهبود یابد. سامانهٔ آن‌ها، SLING، مستقیماً به

بازنماییِ قابِ معنایی تجزیه می‌کند و مشکلِ انباشتِ خطاها در سامانهٔ خطِ لولهٔ سنتی را کاهش می‌دهد. قطعاً جا برای بهبود هست: نه‌تنها سامانه‌های NLP همچنان در بسیاری از تکلیف‌ها از عملکردِ انسانی عقب‌اند، بلکه این کار را پس از پردازشِ هزاران برابرِ متنی که هر انسانی در طولِ عمرش می‌تواند بخواند انجام می‌دهند. این پیشنهاد می‌کند جای زیادی برای بینش‌های تازه از زبان‌شناسان، روان‌شناسان و پژوهشگرانِ NLP هست.

خلاصهٔ فصل

نکته‌های کلیدیِ این فصل چنین‌اند:

- بازنمایی‌های پیوستهٔ واژه‌ها با جاسازیِ واژه از بازنمایی‌های اتمیِ گسسته مقاوم‌ترند، و می‌توانند با دادهٔ متنیِ بدونِ برچسب پیش‌آموزش ببینند.
- شبکه‌های عصبیِ بازگشتی می‌توانند بافتِ محلی و دوربرد را با نگه‌داشتنِ اطلاعاتِ مربوط در بردارهای حالتِ پنهانِشان به‌طورِ مؤثر مدل کنند.
- مدل‌های دنباله‌به‌دنباله را می‌توان برای ترجمهٔ ماشینی و مسائلِ تولیدِ متن استفاده کرد.
- مدل‌های ترنسفورمر از خودتوجهی استفاده می‌کنند و می‌توانند بافتِ دوربرد و همچنین بافتِ محلی را مدل کنند. آن‌ها می‌توانند از ضربِ ماتریسِ سخت‌افزاری استفادهٔ مؤثر کنند.
- یادگیریِ انتقالی که شاملِ جاسازی‌های واژهٔ بافتیِ ازپیش‌آموزش‌دیده است به مدل‌ها اجازه می‌دهد از پیکره‌های بدونِ برچسبِ بسیار بزرگ توسعه یابند و بر طیفی از تکلیف‌ها اعمال شوند. مدل‌هایی که برای پیش‌بینیِ واژه‌های مفقود پیش‌آموزش دیده‌اند می‌توانند تکلیف‌های دیگری مانندِ پاسخ‌گوییِ پرسش و استلزامِ متنی را، پس از تنظیمِ دقیق برای حوزهٔ هدف، مدیریت کنند.

یادداشت‌های کتابشناختی و تاریخی

توزیعِ واژه‌ها و عبارت‌ها در زبانِ طبیعی از «قانونِ زیپف» (Zipf, 1935, 1949) پیروی می‌کند: بسامدِ nاُمین واژهٔ محبوب تقریباً با n/1 نسبتِ معکوس دارد. این بدان‌معناست مسئلهٔ تنکیِ داده داریم: حتی با میلیاردها واژهٔ دادهٔ آموزشی، دائماً به واژه‌ها و عبارت‌های تازه‌ای برمی‌خوریم که قبلاً دیده نشده‌اند.

تعمیم به واژه‌ها و عبارت‌های تازه توسط بازنمایی‌هایی که بینشِ پایهٔ «واژه‌های با معانیِ مشابه در بافت‌های مشابه ظاهر می‌شوند» را درمی‌گیرند یاری می‌شود. دیروستر و همکاران (1990) واژه‌ها را با تجزیهٔ ماتریسِ هم‌رخدادِ تشکیل‌شده توسط واژه‌ها و اسنادی که واژه‌ها در آن‌ها ظاهر می‌شوند به بردارهای کم‌بعد تصویر کردند. امکانِ دیگر رفتارکردن با واژه‌های اطراف—مثلاً، پنجرهٔ پنج‌واژه‌ای—به‌عنوانِ بافت است. براون و همکاران (1992) واژه‌ها را طبقِ بافتِ دوگرمیِ واژه‌ها به خوشه‌های سلسله‌مراتبی گروه‌بندی کردند؛ این برای تکلیف‌هایی مانندِ بازشناسیِ موجودیتِ نام‌دار مؤثر ثابت شده (تورین و همکاران، 2010). سامانهٔ WORD2VEC (میکولوف و همکاران، 2013) نخستین نمایشِ چشمگیرِ مزیت‌های جاسازیِ واژهٔ به‌دست‌آمده از آموزشِ شبکه‌های عصبی بود. بردارهای جاسازیِ واژهٔ GloVe

(پنینگتون و همکاران، 2014) با عمل‌کردنِ مستقیم بر ماتریسِ هم‌رخدادِ واژهٔ به‌دست‌آمده از میلیاردها واژهٔ متن به‌دست آمدند. لوی و گلدبرگ (2014) توضیح می‌دهند چرا و چگونه این جاسازی‌های واژه قادر به دربرگرفتنِ نظم‌های زبانی‌اند.

بنجیو و همکاران (2003) استفاده از شبکه‌های عصبی برای مدل‌های زبان را پیشگامی کردند، و پیشنهاد کردند «(۱) بازنماییِ توزیع‌شده برای هر واژه همراه با (۲) تابعِ احتمالِ دنباله‌های واژه، بیان‌شده بر حسبِ این بازنمایی‌ها» ترکیب شود. میکولوف و همکاران (2010) استفاده از RNNها را برای مدل‌کردنِ بافتِ محلی در مدل‌های زبان نشان دادند. یوزفوویچ و همکاران (2016) نشان دادند RNN آموزش‌دیده رویِ یک میلیارد واژه چگونه می‌تواند از مدل‌های n-گرمِ دقیقاً دست‌ساخته بهتر عمل کند. بازنمایی‌های بافتی برای واژه‌ها توسط پیترز و همکاران (2018) تأکید شد، که آن‌ها را بازنمایی‌های ELMO (جاسازی‌ها از مدل‌های زبان) نامیدند.

توجه کنید برخی نویسندگان مدل‌های زبان را با اندازه‌گیریِ «سردرگمی» (Perplexity)شان مقایسه می‌کنند. سردرگمیِ توزیعِ احتمالی 2^H است، که H آنتروپیِ توزیع است (بخش ۱۹.۳.۳ را ببینید). مدلِ زبانِ با سردرگمیِ کمتر، با مساوی‌بودنِ بقیهٔ چیزها، مدلِ بهتری است. اما در عمل، بقیهٔ چیزها به‌ندرت مساوی‌اند. بنابراین، اندازه‌گیریِ عملکرد رویِ تکلیفِ واقعی اطلاعاتی‌تر از تکیه بر سردرگمی است.

هاوارد و رودر (2018) چارچوبِ ULMFIT (تنظیمِ دقیقِ مدلِ زبانِ همگانی) را توصیف می‌کنند، که تنظیمِ دقیقِ مدلِ زبانِ ازپیش‌آموزش‌دیده را بدونِ نیاز به پیکرهٔ عظیمِ اسنادِ حوزهٔ هدف آسان‌تر می‌کند. رودر و همکاران (2019) آموزشِ یادگیریِ انتقالی برای NLP می‌دهند. میکولوف و همکاران (2010) ایدهٔ استفاده از RNNها برای NLP را معرفی کردند، و سوتسکِور و همکاران (2015) ایدهٔ یادگیریِ دنباله‌به‌دنباله با شبکه‌های عمیق را معرفی کردند. ژو و همکاران (2017) و (لیو و همکاران، 2018b) نشان دادند رویکردِ نظارت‌نشده کار می‌کند و جمع‌آوریِ داده را بسیار آسان‌تر می‌کند. به‌زودی معلوم شد این‌گونه مدل‌ها می‌توانند در انواعِ تکلیف‌ها به‌شکلِ شگفت‌آوری خوب عمل کنند، مثلاً زیرنویسِ تصویر (کارپاتی و فی-فی، 2015؛ وینیالز و همکاران، 2017b). دوولین و همکاران (2018) نشان دادند مدل‌های ترنسفورمرِ ازپیش‌آموزش‌دیده با هدفِ مدل‌سازیِ زبانِ نقاب‌دار را می‌توان مستقیماً برای تکلیف‌های متعدد استفاده کرد. این مدل BERT (بازنمایی‌های رمزگذارِ دوسویه از ترنسفورمرها) نامیده شد. مدل‌های BERT ازپیش‌آموزش‌دیده را می‌توان برای حوزه‌ها و تکلیف‌های خاص، از جملهِ پاسخ‌گوییِ پرسش، بازشناسیِ موجودیتِ نام‌دار، رده‌بندیِ متن، تحلیلِ احساسات و استنتاجِ زبانِ طبیعی، تنظیمِ دقیق کرد.

سامانهٔ XLNET (یانگ و همکاران، 2019) با حذفِ ناهمخوانیِ بینِ پیش‌آموزش و تنظیمِ دقیق BERT را بهبود می‌بخشد. چارچوبِ ERNIE 2.0 (سون و همکاران، 2019) با درنظرگرفتنِ ترتیبِ جمله و حضورِ موجودیت‌های نام‌دار، به‌جای صرفِ هم‌رخدادِ واژه‌ها، بیشتر از دادهٔ آموزشی استخراج می‌کند، و نشان داده شد از BERT و XLNET بهتر عمل می‌کند.

در پاسخ، پژوهشگران BERT را بازبینی و بهبود دادند: سامانهٔ ROBERTA (لیو و همکاران، 2019b) از دادهٔ بیشتر و ابرپارامترها و رویه‌های آموزشیِ متفاوت استفاده کرد و دریافت می‌تواند با XLNET برابری کند. سامانهٔ Reformer (کیتایف و همکاران، 2020) دامنهٔ بافتی را که می‌توان در نظر گرفت تا یک میلیون واژه بسط می‌دهد. در این میان،

ALBERT (BERT سبک) در جهتِ دیگر رفت، با کاهشِ تعدادِ پارامترها از ۱۰۸ میلیون به ۱۲ میلیون (تا رویِ دستگاه‌های همراه جا شود) در حالی که دقتِ بالایی حفظ می‌کند.

سامانهٔ XLM (لمپل و کونو، 2019) مدلِ ترنسفورمری با دادهٔ آموزشی از زبان‌های متعدد است. این برای ترجمهٔ ماشینی مفید است، اما بازنمایی‌های مقاوم‌تری نیز برای تکلیف‌های تک‌زبانه فراهم می‌کند. دو سامانهٔ مهمِ دیگر، GPT-2 (رادفورد و همکاران، 2019) و T5 (رافل و همکاران، 2019)، در فصل توصیف شدند. مقالهٔ دوم همچنین پیکرهٔ خزیدهٔ تمیزِ عظیمِ ۳۵ میلیارد واژه‌ای (C4) را معرفی کرد.

بهبودهای امیدوارکنندهٔ گوناگونی بر الگوریتم‌های پیش‌آموزش پیشنهاد شده (یانگ و همکاران، 2019؛ لیو و همکاران، 2019b). مدل‌های بافتیِ ازپیش‌آموزش‌دیده توسط پیترز و همکاران (2018) و دای و له (2016) توصیف شده‌اند. معیارِ GLUE (ارزیابیِ فهمِ عمومیِ زبان)، مجموعه‌ای از تکلیف‌ها و ابزارها برای ارزیابیِ سامانه‌های NLP، توسط وانگ و همکاران (2018a) معرفی شد. تکلیف‌ها شاملِ پاسخ‌گوییِ پرسش، تحلیلِ احساسات، استلزامِ متنی، ترجمه و تجزیه‌اند.

مدل‌های ترنسفورمر چنان بر جدولِ رتبه‌بندی مسلط شده‌اند (خطِ پایهٔ انسانی در رتبهٔ نهم پایین‌تر است) که نسخهٔ تازه‌ای، SUPERGLUE (وانگ و همکاران، 2019)، با تکلیف‌هایی معرفی شد که برای رایانه‌ها سخت‌تر، اما برای انسان‌ها همچنان آسان طراحی شده‌اند.

در پایانِ ۲۰۱۹، T5 با امتیازِ 89.3 رهبرِ کلی بود، فقط نیم‌امتیازِ پایین‌تر از خطِ پایهٔ انسانیِ 89.8. در سه مورد از ده تکلیف، T5 در واقع از عملکردِ انسانی فراتر می‌رود: پاسخ‌گوییِ پرسشِ بله/خیر (مانندِ «آیا فرانسه همان منطقهٔ زمانیِ بریتانیاست؟») و دو تکلیفِ درکِ مطلب که شاملِ پاسخ‌دادن به پرسش‌ها پس از خواندنِ پاراگرافی یا مقالهٔ خبری‌اند.

ترجمهٔ ماشینی کاربردِ عمده‌ای از مدل‌های زبان است. در ۱۹۳۳، پتر ترویانسکی برای «ماشینِ مترجم» حق‌امتیازی گرفت، اما هیچ رایانه‌ای برای پیاده‌سازیِ ایده‌هایش در دسترس نبود. در ۱۹۴۷، وارن ویور، با بهره‌گیری از کار در رمزنگاری و نظریهٔ اطلاعات، به نوربرت وینر نوشت: «وقتی به مقاله‌ای به روسی نگاه می‌کنم، می‌گویم: این واقعاً به انگلیسی نوشته شده، اما با نمادهای عجیبی رمز شده. حالا اقدام به رمزگشایی خواهم کرد.» جامعه اقدام به تلاشِ رمزگشایی به این‌شیوه کرد، اما داده و منابعِ محاسباتیِ کافی برای عملی‌کردنِ رویکرد نداشتند.

در دههٔ ۱۹۷۰ آن شروع به تغییر کرد، و سامانهٔ SYSTRAN (توما، 1977) نخستین سامانهٔ ترجمهٔ ماشینیِ موفقِ تجاری بود. SYSTRAN بر قاعده‌های واژگانی و دستوریِ دست‌ساخته توسط زبان‌شناسان و همچنین بر دادهٔ آموزشی تکیه داشت. در دههٔ ۱۹۸۰، جامعه مدل‌های صرفاً آماریِ مبتنی بر بسامدِ واژه‌ها و عبارت‌ها را در آغوش کشید (براون و همکاران، 1988؛ کوهن، 2009). وقتی مجموعه‌های آموزشی به میلیاردها یا تریلیون‌ها توکن رسیدند (برانتس و همکاران، 2007)، این به سامانه‌هایی انجامید که نتایجِ قابلِ‌فهم اما نه روان تولید می‌کردند (اوخ و نی، 2004؛ زولمن و همکاران، 2008). اوخ و نی (2002) نشان می‌دهند آموزشِ متمایزگرانه چگونه در اوایلِ دههٔ ۲۰۰۰ به پیشرفتی در ترجمهٔ ماشینی انجامید.

سوتسکِور و همکاران (2015) نخستین بار نشان دادند یادگیریِ مدلِ عصبیِ دنباله‌به‌دنبالهٔ سرتاسری برای ترجمهٔ ماشینی ممکن است. باهداناو و همکاران (2015) مزیتِ مدلی را نشان دادند که به‌طورِ مشترک یاد می‌گیرد جمله‌ها را در زبانِ مبدأ و مقصد تراز کند و بینِ زبان‌ها ترجمه کند. واسوانی و همکاران (2018) نشان دادند سامانه‌های ترجمهٔ ماشینیِ عصبی را می‌توان بیشتر با جایگزینیِ LSTMها با معماری‌های ترنسفورمر بهبود داد، که از سازوکارِ توجه برای دربرگرفتنِ بافت استفاده می‌کنند. این سامانه‌های ترجمهٔ عصبی به‌سرعت از روش‌های آماریِ عبارت‌محور پیشی گرفتند، و معماریِ ترنسفورمر به‌زودی به دیگر تکلیف‌های NLP گسترش یافت.

پژوهشِ پاسخ‌گوییِ پرسش با ایجادِ SQUAD، نخستین مجموعهٔ دادهٔ مقیاس‌بزرگ برای آموزش و آزمونِ سامانه‌های پاسخ‌گوییِ پرسش (راجپورکار و همکاران، 2016)، تسهیل شد. از آن زمان، تعدادی مدلِ یادگیریِ عمیق برای این تکلیف توسعه یافته (سئو و همکاران، 2017؛ کسکار و همکاران، 2019). سامانهٔ ARISTO (کلارک و همکاران، 2019) از یادگیری عمیق در پیوند با مجموعه‌ای از تاکتیک‌های دیگر استفاده می‌کند. از ۲۰۱۸، اکثریتِ مدل‌های پاسخ‌گوییِ پرسش از بازنمایی‌های زبانیِ ازپیش‌آموزش‌دیده استفاده می‌کنند، که به بهبودِ محسوسی نسبت به سامانه‌های قبلی می‌انجامد.

استنتاجِ زبانِ طبیعی تکلیفِ قضاوتِ دربارهٔ اینکه آیا فرضیه‌ای (حیوانات باید غذا بخورند) توسط مقدمه‌ای (همهٔ حیوانات باید غذا بخورند) مستلزم می‌شود است. این تکلیف توسط چالشِ PASCAL (داگان و همکاران، 2005) رایج شد. مجموعه‌های دادهٔ مقیاس‌بزرگ اکنون در دسترس‌اند (بومن و همکاران، 2015؛ ویلیامز و همکاران، 2018). سامانه‌های مبتنی بر مدل‌های ازپیش‌آموزش‌دیده مانندِ ELMO و BERT در حالِ حاضر بهترین عملکرد را در تکلیف‌های استنتاجِ زبانی فراهم می‌کنند.

کنفرانسِ یادگیریِ محاسباتیِ زبانِ طبیعی (CoNLL) بر یادگیری برای NLP تمرکز دارد. همهٔ کنفرانس‌ها و مجله‌های ذکرشده در فصل ۲۳ اکنون مقاله‌هایی دربارهٔ یادگیری عمیق شامل می‌شوند، که اکنون جایگاهِ غالب در حوزهٔ NLP دارد.

پانوشت‌ها:

[1] اشاره شده در برخی آزمون‌های چندگزینه‌ای، حتی بدونِ نگاه‌کردن به پرسش‌ها هم می‌توان امتیازِ خوبی گرفت، زیرا نشانه‌های خبردهنده‌ای در پاسخ‌های نادرست هست (گورورانگان و همکاران، 2018). به نظر می‌رسد این برای پاسخ‌گوییِ پرسشِ بصری نیز صادق است (چائو و همکاران، 2018).

فصل ۲۵: بینایی رایانه‌ای (Computer Vision)

در این فصل رایانه را از طریقِ چشم‌های یک دوربین به جهانِ خام و شسته‌نشده متصل می‌کنیم.

بیشترِ جانوران چشم دارند، اغلب به هزینه‌ای چشمگیر: چشم‌ها فضای زیادی اشغال می‌کنند؛ انرژی مصرف می‌کنند؛ و کاملاً شکننده‌اند. این هزینه توسط ارزشِ عظیمی که چشم‌ها فراهم می‌کنند توجیه می‌شود. عاملی که می‌تواند ببیند می‌تواند آینده را پیش‌بینی کند—می‌تواند بگوید ممکن است به چه چیزی برخورد کند؛ می‌تواند بگوید حمله کند یا بگریزد یا خواستگاری کند؛ می‌تواند حدس بزند زمینِ پیشِ رو باتلاقی است یا سفت؛ و می‌تواند بگوید میوه چقدر دور است. در این فصل، توصیف می‌کنیم چگونه از سیلِ داده‌ای که از چشم‌ها یا دوربین‌ها می‌آید اطلاعات بازیابی کنیم.

۲۵.۱ مقدمه

بینایی کانالِ ادراکی‌ای است که محرکی را می‌پذیرد و بازنماییِ برخی از جهان را گزارش می‌کند. بیشترِ عامل‌هایی که از بینایی استفاده می‌کنند از «حسگریِ منفعل» استفاده می‌کنند—برای دیدن نیازی به فرستادنِ نور ندارند. در مقابل، «حسگریِ فعال» شاملِ فرستادنِ سیگنالی مانندِ رادار یا فراصوت و حسِ بازتاب است. نمونه‌های عامل‌های استفاده‌کننده از حسگریِ فعال شاملِ خفاش‌ها (فراصوت)، دلفین‌ها (صدا)، ماهی‌های ژرفا (نور) و برخی ربات‌ها (نور، صدا، رادار) است. برای فهمِ کانالِ ادراکی، باید هم پدیده‌های فیزیکی و آماریِ رخداده در حسگری و هم آنچه فرایندِ ادراکی باید تولید کند مطالعه کرد. در این فصل بر بینایی تمرکز می‌کنیم، اما ربات‌های جهانِ واقعی از انواعِ حس‌گذارها برای ادراکِ صدا، لمس، فاصله، دما، موقعیتِ جهانی و شتاب استفاده می‌کنند.

«ویژگی» (Feature) عددی است که با اعمالِ محاسباتِ ساده بر تصویر به‌دست می‌آید. اطلاعاتِ بسیار مفیدی را می‌توان مستقیماً از ویژگی‌ها به‌دست آورد. عاملِ وومپوس پنج حس‌گذار داشت، که هر یک یک بیتِ اطلاعات استخراج می‌کرد. این بیت‌ها، که ویژگی‌اند، می‌توانستند مستقیماً توسط برنامه تفسیر شوند. به‌عنوانِ مثالِ دیگر، بسیاری از جانورانِ پرنده ویژگیِ ساده‌ای محاسبه می‌کنند که برآوردِ خوبی از زمانِ تا برخورد با شیءِ نزدیک می‌دهد؛ این ویژگی را می‌توان مستقیماً به ماهیچه‌های کنترل‌کنندهٔ سکان یا بال‌ها منتقل کرد و تغییرِ جهتِ بسیار سریع را ممکن ساخت. این رویکردِ استخراجِ ویژگی بر محاسباتِ ساده و مستقیمِ اعمال‌شده بر پاسخ‌های حس‌گذار تأکید می‌کند.

رویکردِ مبتنی بر مدل به بینایی از دو نوعِ مدل استفاده می‌کند. «مدلِ شیء» (Object Model) می‌تواند نوعِ مدلِ هندسیِ دقیقی باشد که توسط سامانه‌های طراحیِ به‌کمکِ رایانه تولید می‌شود. همچنین می‌تواند گزاره‌ای مبهم دربارهٔ ویژگی‌های عمومیِ اشیا باشد، برای نمونه، ادعایی که همهٔ چهره‌های مشاهده‌شده در وضوحِ پایین تقریباً یکسان به نظر می‌رسند. «مدلِ رندرینگ» (Rendering Model) فرایندهای فیزیکی، هندسی و آماریِ تولیدکنندهٔ محرک از جهان را توصیف می‌کند.

هرچند مدل‌های رندرینگ اکنون پیچیده و دقیق‌اند، محرک معمولاً مبهم است. شیءِ سفیدی زیرِ نورِ کم ممکن است مانندِ شیءِ سیاهی زیرِ نورِ شدید به نظر برسد. شیءِ کوچکِ نزدیک ممکن است همان‌گونه به نظر برسد که شیءِ بزرگِ دور. بدونِ شواهدِ اضافی، نمی‌توانیم بگوییم آنچه می‌بینیم گودزیلای اسباب‌بازی است که ساختمانِ اسباب‌بازی را پاره می‌کند، یا هیولای واقعی است که ساختمانِ واقعی را نابود می‌کند. دو راهِ اصلی برای مدیریتِ این ابهام‌ها هست. نخست، برخی تفسیرها از بقیه محتمل‌ترند. برای نمونه، می‌توانیم مطمئن باشیم تصویر گودزیلای واقعیِ نابودکنندهٔ ساختمانِ واقعی را نشان نمی‌دهد، زیرا هیچ گودزیلای واقعی‌ای نیست. دوم، برخی ابهام‌ها بی‌اهمیت‌اند. برای نمونه، منظرهٔ دور ممکن است درخت باشد یا سطحِ نقاشی‌شدهٔ تخت. برای بیشترِ کاربردها، تفاوت بی‌اهمیت است، زیرا اشیا دورند و به‌زودی با آن‌ها برخورد نمی‌کنیم یا برهم‌کنش نمی‌کنیم.

دو مسئلهٔ هسته‌ایِ بیناییِ رایانه «بازسازی» (Reconstruction) است، که در آن عامل از یک تصویر یا مجموعه‌ای از تصاویر مدلی از جهان می‌سازد، و «بازشناسی» (Recognition)، که در آن عامل بر مبنای اطلاعاتِ بصری و دیگر، تمایزهایی میانِ اشیایی که با آن‌ها روبه‌رو می‌شود قائل می‌شود.

هر دو مسئله را باید بسیار گسترده تفسیر کرد. ساختنِ مدلِ هندسی از تصاویر آشکارا بازسازی است (و راه‌حل‌ها بسیار ارزشمندند)، اما گاهی باید نقشه‌ای از بافت‌های متفاوتِ رویِ یک سطح بسازیم، و این نیز بازسازی است. الصاقِ نام به اشیایی که در تصویر ظاهر می‌شوند آشکارا بازشناسی است. گاهی باید به پرسش‌هایی مانندِ این‌ها پاسخ دهیم: خواب است؟ گوشت می‌خورد؟ کدام سر دندان دارد؟ پاسخ‌دادن به این پرسش‌ها نیز بازشناسی است. سی سالِ گذشتهٔ پژوهش ابزارها و روش‌های قدرتمندی برای پرداختن به این مسائلِ هسته‌ای تولید کرده. فهمِ این روش‌ها مستلزمِ فهمِ فرایندهایی است که تصاویر توسط آن‌ها تشکیل می‌شوند.

۲۵.۲ تشکیلِ تصویر

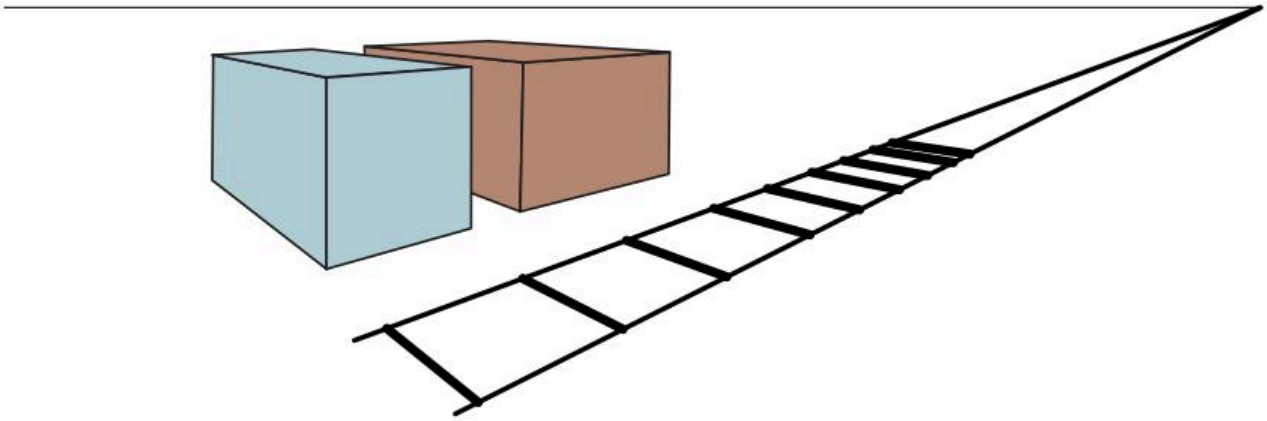
تصویربرداری ظاهرِ اشیا را تحریف می‌کند. عکسی که از طولِ مجموعه‌ای مستقیمِ بلند از ریل‌های قطار به پایین گرفته شود پیشنهاد می‌کند ریل‌ها به هم می‌رسند و همگرا می‌شوند. اگر دستتان را جلوی چشم‌تان بگیرید، می‌توانید ماه را بپوشانید، حتی اگر ماه از دستِ شما بزرگ‌تر باشد (این با خورشید هم کار می‌کند، اما با امتحان‌کردنش ممکن است به چشم‌تان آسیب بزنید). اگر کتابی را تخت جلوی صورت‌تان بگیرید و آن را به عقب و جلو متمایل کنید، به نظر می‌رسد در تصویر کوچک و بزرگ می‌شود. این اثر «کوتاه‌نمایی» (Foreshortening) نامیده می‌شود (شکل ۲۵.۱). مدل‌های این اثرها برای ساختنِ سامانه‌های شایستهٔ بازشناسیِ اشیا ضروری‌اند و همچنین سرنخ‌های قدرتمندی برای بازسازیِ هندسه فراهم می‌کنند.

۲۵.۲.۱ تصاویر بدونِ عدسی: دوربینِ سوراخ‌سوزنی

حس‌گذارهای تصویر نورِ پراکنده‌شده از اشیای صحنه را جمع می‌کنند و تصویرِ دوبعدی (2D) می‌سازند. در چشم، این حس‌گذارها از دو نوعِ سلول تشکیل شده‌اند: حدودِ ۱۰۰ میلیون «میله» (Rod) هست که به نور در طیفِ وسیعی از طول‌موج‌ها حساس‌اند، و ۵ میلیون «مخروط» (Cone). مخروط‌ها، که برای بینایی رنگی ضروری‌اند، از سه نوعِ اصلی‌اند،

که هر یک به مجموعه‌ای متفاوت از طول‌موج‌ها حساس است. در دوربین‌ها، تصویر رویِ «صفحهٔ تصویر» (Image Plane) تشکیل می‌شود. در دوربین‌های فیلمی صفحهٔ تصویر با هالیدهای نقره پوشیده شده. در دوربین‌های دیجیتال، صفحهٔ تصویر به شبکه‌ای از چند میلیون «پیکسل» (Pixel) تقسیم می‌شود.

به کلِ صفحهٔ تصویر به‌عنوانِ «حس‌گذار» (Sensor) اشاره می‌کنیم، اما هر پیکسل «حس‌گذارِ» ریزِ مستقلی است—معمولاً دستگاهِ جفت‌شدهٔ بار (CCD) یا نیمه‌رسانای اکسیدِ فلزیِ مکمل (CMOS). هر فوتونی که به حس‌گذار می‌رسد اثرِ الکتریکی‌ای تولید می‌کند، که شدتش به طول‌موجِ فوتون بستگی دارد. خروجیِ حس‌گذار مجموعِ همهٔ این اثرها در پنجرهٔ زمانیِ برخی است، یعنی حس‌گذارهای تصویر میانگینِ وزن‌داری از شدتِ نورِ رسیده به حس‌گذار را گزارش می‌کنند. میانگین رویِ طولِ موج، جهتی که فوتون‌ها می‌توانند از آن برسند، زمان و مساحتِ حس‌گذار است.



شکل ۲۵.۱ – هندسهٔ صحنه در تصاویر تحریف‌شده ظاهر می‌شود. خطوطِ موازی به نظر می‌رسند به هم می‌رسند، مانندِ ریل‌های قطار در شهری متروک. ساختمان‌هایی که در صحنهٔ جهانِ واقعی زاویه‌های قائم دارند، در تصویر زاویه‌های تحریف‌شده دارند.

برای دیدنِ تصویرِ فوکوس‌شده، باید مطمئن شویم همهٔ فوتون‌های رسیده به یک حس‌گذار تقریباً از همان نقطهٔ شیءِ جهان می‌آیند. ساده‌ترین راهِ تشکیلِ تصویرِ فوکوس‌شده مشاهدهٔ اشیای ساکن با «دوربینِ سوراخ‌سوزنی» (Pinhole Camera) است، که شاملِ روزنه‌ای (O) در جلوی جعبه و صفحهٔ تصویری در عقبِ جعبه است (شکل ۲۵.۲). روزنه «دیافراگم» (Aperture) نامیده می‌شود. اگر سوراخِ سوزنی به‌اندازهٔ کافی کوچک باشد، هر حس‌گذارِ ریزِ صفحهٔ تصویر فقط فوتون‌هایی را می‌بیند که تقریباً از همان نقطهٔ شیء می‌آیند، و بنابراین تصویر فوکوس‌شده است. می‌توانیم تصاویرِ فوکوس‌شدهٔ اشیای متحرک را نیز با دوربینِ سوراخ‌سوزنی تشکیل دهیم، تا زمانی که شیء فقط فاصلهٔ کوتاهی در پنجرهٔ زمانیِ حس‌گذارها حرکت کند. در غیرِ این صورت، تصویرِ شیءِ متحرک خارج از فوکوس است، اثری که «محوشدگیِ حرکت» (Motion Blur) نامیده می‌شود. یکی از راه‌های دست‌کاریِ پنجرهٔ زمانی بازوبسته‌کردنِ سوراخِ سوزنی است.

دوربین‌های سوراخ‌سوزنی فهمِ مدلِ هندسیِ رفتارِ دوربین را آسان می‌کنند (که با بیشترِ دستگاه‌های تصویربرداریِ دیگر پیچیده‌تر است—اما مشابه). از سامانهٔ مختصاتِ سه‌بعدی (3D) با مبدأ در O استفاده می‌کنیم و نقطه‌ای P در صحنه با مختصاتِ (X,Y,Z) را در نظر می‌گیریم. P به نقطهٔ 'P در صفحهٔ تصویر با مختصاتِ (x,y,z) فرافکنی می‌شود. اگر f «فاصلهٔ کانونی» (Focal Length)—فاصله از سوراخِ سوزنی تا صفحهٔ تصویر—باشد، آنگاه با مثلث‌های متشابه می‌توانیم معادله‌های زیر را اشتقاق دهیم:

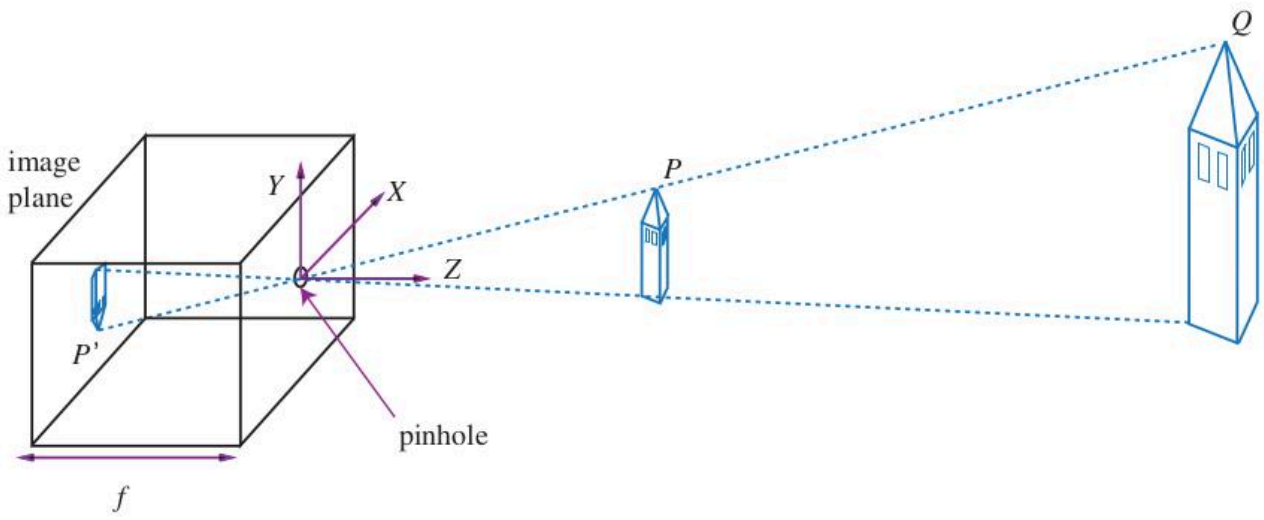
$$x/f = X/Z, \quad -y/f = Y/Z \quad \Rightarrow \quad x = -fX/Z, \quad y = -fY/Z$$

این معادله‌ها فرایندِ تشکیلِ تصویری را تعریف می‌کنند که «فرافکنیِ پرسپکتیوی» (Perspective Projection) نامیده می‌شود. توجه کنید Z در مخرج به این معناست که هرچه شیء دورتر باشد، تصویرش کوچک‌تر خواهد بود. همچنین توجه کنید علامت‌های منفی به این معناست که تصویر در مقایسه با صحنه، هم چپ-راست و هم بالا-پایین وارونه است.

تصویربرداریِ پرسپکتیوی تعدادی اثرِ هندسی دارد. اشیای دور کوچک به نظر می‌رسند. خطوطِ موازی به نقطه‌ای رویِ افق همگرا می‌شوند. (ریل‌های قطار را در نظر بگیر، شکل ۲۵.۱.) خطی در صحنه در جهتِ (U,V,W) و گذرنده از نقطهٔ (X_0,Y_0,Z_0) را می‌توان به‌صورتِ مجموعهٔ نقطه‌های $(X_0+\lambda U, Y_0+\lambda V, Z_0+\lambda W)$ توصیف کرد، با λ که بینِ $-\infty$ و $+\infty$ تغییر می‌کند. انتخاب‌های متفاوتِ (X_0,Y_0,Z_0) خطوطِ متفاوتی موازیِ یکدیگر می‌دهند. فرافکنیِ نقطهٔ P_λ از این خط بر صفحهٔ تصویر به‌صورتِ زیر است:

$$P_\lambda = \left(f(X_0 + \lambda U) / (Z_0 + \lambda W), \quad f(Y_0 + \lambda V) / (Z_0 + \lambda W)\right)$$

وقتی $\lambda \to \infty$ یا $\lambda \to -\infty$، این به $P_\infty = (fU/W, fV/W)$ تبدیل می‌شود اگر $W \neq 0$. این بدان‌معناست دو خطِ موازیِ خارج‌شونده از نقطه‌های متفاوتِ فضا در تصویر همگرا خواهند شد—برای λ بزرگ، نقطه‌های تصویر تقریباً یکسان‌اند، هر چه مقدارِ (X_0,Y_0,Z_0) باشد (باز، ریل‌های قطار را در نظر بگیر، شکل ۲۵.۱). P_∞ را «نقطهٔ گریز» (Vanishing Point) وابسته به خانوادهٔ خطوطِ راستِ با جهتِ (U,V,W) می‌نامیم. خطوطِ با جهتِ یکسان نقطهٔ گریزِ مشترکی دارند.

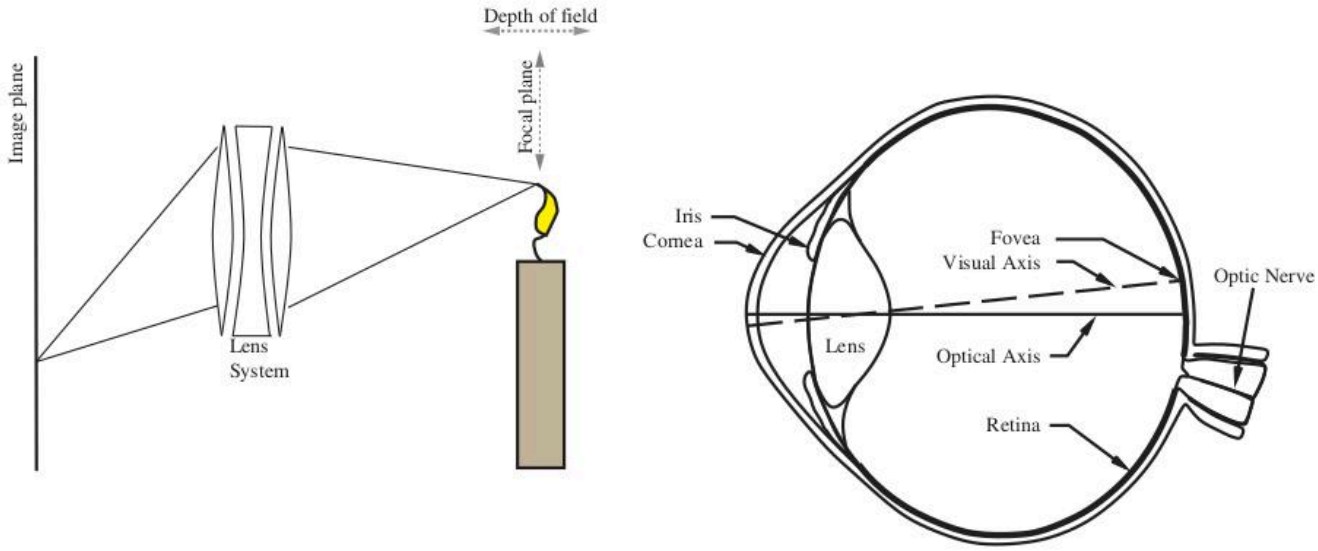


شکل ۲۵.۲ – هر عنصرِ حساس به نور در عقبِ دوربینِ سوراخ‌سوزنی نوری را دریافت می‌کند که از سوراخِ سوزنی از دامنهٔ کوچکی از جهت‌ها عبور می‌کند. اگر سوراخِ سوزنی به‌اندازهٔ کافی کوچک باشد، نتیجه تصویری فوکوس‌شده در پشتِ سوراخِ سوزنی است. فرایندِ فرافکنی به این معناست که اشیای بزرگِ دور همان‌گونه به نظر می‌رسند که اشیای کوچکِ نزدیک—نقطهٔ P' در صفحهٔ تصویر می‌توانست از برجِ اسباب‌بازیِ نزدیکی در نقطهٔ P یا از برجِ واقعیِ دوردستی در نقطهٔ Q آمده باشد.

۲۵.۲.۲ سامانه‌های عدسی

دوربین‌های سوراخ‌سوزنی می‌توانند نور را خوب فوکوس کنند، اما چون سوراخِ سوزنی کوچک است، نورِ کمی وارد می‌شود و تصویر تاریک خواهد بود. در طولِ مدتِ کوتاهی، فقط چند فوتون به هر نقطهٔ حس‌گذار برخورد می‌کند، پس سیگنال در هر نقطه توسط نوسان‌های تصادفی مسلط می‌شود؛ می‌گوییم تصویرِ فیلمیِ تاریک دانه‌دار است و تصویرِ دیجیتالِ تاریک نویزی؛ به هر حال، تصویر کیفیتِ پایینی دارد. بزرگ‌کردنِ سوراخ (دیافراگم) با جمع‌کردنِ نورِ بیشتر از دامنهٔ وسیع‌تری از جهت‌ها تصویر را روشن‌تر می‌کند. با این حال، با دیافراگمِ بزرگ‌تر، نوری که به نقطهٔ خاصی از صفحهٔ تصویر برخورد می‌کند از چند نقطهٔ صحنهٔ جهانِ واقعی آمده خواهد بود، پس تصویر خارج از فوکوس خواهد بود. به راهی برای بازفوکوس‌کردنِ تصویر نیاز داریم.

چشم‌های مهره‌داران و دوربین‌های مدرن از «سامانهٔ عدسی» (Lens System) استفاده می‌کنند—تکهٔ شفافِ واحدی از بافت در چشم و سامانه‌ای از چند عنصرِ عدسیِ شیشه‌ای در دوربین. در شکل ۲۵.۳ می‌بینیم نورِ نوکِ شمع در همهٔ جهت‌ها پخش می‌شود. دوربینی (یا چشمی) با عدسی همهٔ نوری را که به هر جایِ عدسی برخورد می‌کند—ناحیه‌ای بسیار بزرگ‌تر از سوراخِ سوزنی—می‌گیرد و همهٔ آن نور را به نقطهٔ تکیِ صفحهٔ تصویر فوکوس می‌کند. نورِ سایرِ بخش‌های شمع به‌همین‌سان جمع و به نقطه‌های دیگرِ صفحهٔ تصویر فوکوس می‌شود. نتیجه تصویری روشن‌تر، کم‌نویزتر و فوکوس‌شده است.



شکل ۲۵.۳ – عدسی‌ها نورِ خارج‌شونده از نقطه‌ای از صحنه (در این‌جا، نوکِ شعلهٔ شمع) را در دامنه‌ای از جهت‌ها جمع می‌کنند و همهٔ نور را به رسیدن به نقطهٔ تکیِ صفحهٔ تصویر هدایت می‌کنند. نقطه‌های صحنهٔ نزدیکِ صفحهٔ کانونی—درونِ عمقِ میدان—به‌درستی فوکوس خواهند شد. در دوربین‌ها، عناصرِ سامانهٔ عدسی حرکت می‌کنند تا صفحهٔ کانونی تغییر کند، در حالی که در چشم، شکلِ عدسی توسط ماهیچه‌های تخصصی تغییر می‌کند.

سامانه‌های عدسی همهٔ نورِ همه‌جایِ جهانِ واقعی را فوکوس نمی‌کنند؛ طراحیِ عدسی آن‌ها را به فوکوس‌کردنِ نورِ فقط نقطه‌هایی که در دامنه‌ای از عمقِ Z از عدسی قرار دارند محدود می‌کند. مرکزِ این دامنه—جایی که فوکوس تیزترین است—«صفحهٔ کانونی» (Focal Plane) و دامنهٔ عمق‌هایی که فوکوس برای آن‌ها به‌اندازهٔ کافی تیز می‌ماند «عمقِ میدان» (Depth of Field) نامیده می‌شود. هرچه دیافراگمِ عدسی (روزنه) بزرگ‌تر باشد، عمقِ میدان کوچک‌تر است.

اگر بخواهید رویِ چیزی در فاصلهٔ متفاوت فوکوس کنید چه؟ برای حرکتِ صفحهٔ کانونی، عناصرِ عدسیِ دوربین می‌توانند به عقب و جلو حرکت کنند، و عدسیِ چشم می‌تواند شکلش را تغییر دهد—اما با افزایشِ سن، عدسیِ چشم تمایل به سخت‌شدن دارد، که توانایی تنظیمِ فاصله‌های کانونی را کاهش می‌دهد و بسیاری از انسان‌ها را به افزودنِ عدسیِ خارجی—عینک—به بینایی‌شان نیازمند می‌کند.

۲۵.۲.۳ فرافکنیِ ارتوگرافیکِ مقیاس‌شده

اثرهای هندسیِ تصویربرداریِ پرسپکتیوی همیشه برجسته نیستند. برای نمونه، پنجره‌های ساختمانیِ آن سویِ خیابان بسیار کوچک‌تر از پنجره‌های نزدیک به نظر می‌رسند، اما دو پنجره‌ای که کنارِ هم‌اند تقریباً همان اندازه را خواهند داشت، حتی اگر یکی کمی دورتر باشد. این گزینه را داریم که پنجره‌ها را با مدلِ ساده‌شده‌ای به نامِ «فرافکنیِ ارتوگرافیکِ مقیاس‌شده» (Scaled Orthographic Projection) مدیریت کنیم، نه فرافکنیِ پرسپکتیوی. اگر عمقِ Z همهٔ نقطه‌های یک شیء درونِ دامنهٔ $Z_0 \pm \Delta Z$ باشد، با $\Delta Z \ll Z_0$، آنگاه عاملِ مقیاسِ پرسپکتیویِ f/Z را می‌توان با ثابتِ $s = f/Z_0$ تقریب زد. معادله‌های فرافکنی از مختصاتِ صحنهٔ (X,Y,Z) به صفحهٔ تصویر به‌صورتِ $x = sX$ و $y = sY$ می‌شوند. کوتاه‌نمایی همچنان در مدلِ فرافکنیِ ارتوگرافیکِ مقیاس‌شده رخ می‌دهد، زیرا توسط متمایل‌شدنِ شیء به دور از دید ایجاد می‌شود.

۲۵.۲.۴ نور و سایه‌روشن

روشناییِ پیکسلی در تصویر تابعی از روشناییِ وصلهٔ سطحِ صحنه‌ای است که به آن پیکسل فرافکنی می‌شود. برای دوربین‌های مدرن، این تابع برای شدت‌های میانیِ نور خطی است، اما برای روشناییِ تاریک‌تر و روشن‌تر غیرخطی‌های برجسته‌ای دارد. از مدلِ خطی استفاده می‌کنیم.

روشناییِ تصویر سرنخِ قویِ، اگر مبهمِ، هم برای شکل و هم برای هویتِ اشیا است. ابهام رخ می‌دهد، زیرا سه عامل در مقدارِ نوری که از نقطه‌ای از یک شیء به تصویر می‌آید سهیم‌اند: شدتِ کلِ «نورِ محیطی» (Ambient Light)؛ اینکه نقطه رو به نور است یا در سایه؛ و مقدارِ نورِ بازتابیده از نقطه. «بازتاب» (Reflection).

انسان‌ها در ابهام‌زداییِ روشنایی به‌شکلِ شگفت‌آوری خوب‌اند—معمولاً می‌توانند تفاوتِ بینِ شیءِ سیاهِ در نورِ روشن و شیءِ سفیدِ در سایه را بگویند، حتی اگر هر دو همان روشناییِ کلی را داشته باشند. با این حال، انسان‌ها گاهی سایه‌روشن و علامت‌گذاری را با هم اشتباه می‌گیرند—خطِّ آرایشِ تیره‌ای زیرِ استخوانِ گونه اغلب مانندِ اثرِ سایه‌روشن به نظر می‌رسد و چهره را لاغرتر نشان می‌دهد.

بیشترِ سطوح نور را با فرایندِ «بازتابِ پخشیده» (Diffuse Reflection) بازتاب می‌کنند. بازتابِ پخشیده نور را به‌طورِ یکنواخت در جهت‌هایِ ترک‌کنندهٔ سطح پراکنده می‌کند، پس روشناییِ سطحِ پخشیده به جهتِ دید بستگی ندارد. بیشترِ پارچه‌ها این ویژگی را دارند، و همچنین بیشترِ رنگ‌ها، سطوحِ چوبیِ زبر، بیشترِ پوششِ گیاهی و سنگ یا بتنِ زبر.

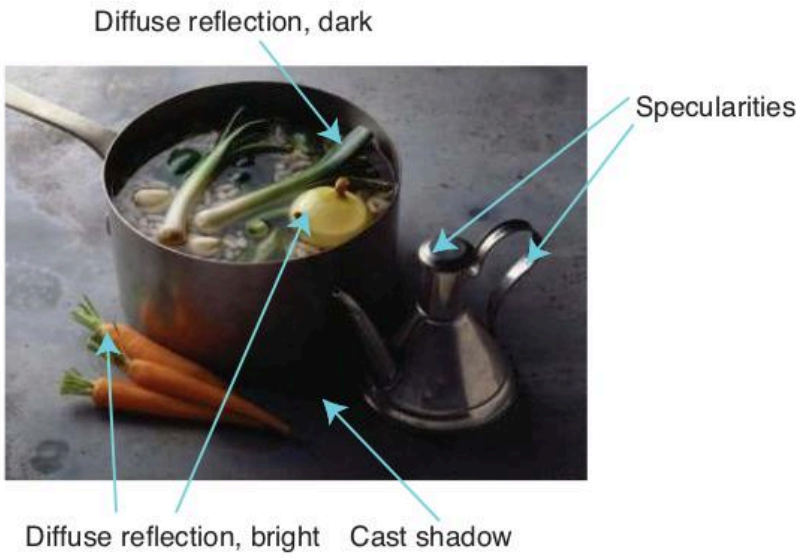
«بازتابِ آینه‌ای» (Specular Reflection) باعث می‌شود نورِ ورودی سطح را در لوبی از جهت‌ها ترک کند که توسط جهتی که نور از آن آمده تعیین می‌شود. آینه یک نمونه است. آنچه می‌بینید به جهتی بستگی دارد که در آن به آینه نگاه می‌کنید. در این مورد، لوبِ جهت‌ها بسیار باریک است، که دلیلِ اینکه می‌توانید اشیای متفاوت را در آینه تفکیک کنید همین است. برای بسیاری از سطوح، لوب پهن‌تر است. این سطوح لکه‌های روشنِ کوچکی نشان می‌دهند، که معمولاً «برقِ آینه‌ای» (Specularity) نامیده می‌شوند. با حرکتِ سطح یا نور، برق‌های آینه‌ای نیز حرکت می‌کنند. به‌دور از این لکه‌ها، سطح طوری رفتار می‌کند که گویی پخشیده است. برق‌های آینه‌ای اغلب رویِ سطوحِ فلزی، رنگ‌شده، پلاستیکی و خیس دیده می‌شوند. شناساییِ آن‌ها آسان است، زیرا کوچک و روشن‌اند (شکل ۲۵.۴). برای تقریباً همهٔ مقاصد، کافی است همهٔ سطوح را پخشیده‌با-برقِ-آینه‌ای مدل کنیم.

منبعِ اصلیِ روشناییِ بیرون خورشید است، که پرتوهایش همگی به‌صورتِ موازی در جهتِ شناخته‌شده‌ای سفر می‌کنند، زیرا آن‌قدر دور است. این رفتار را با «منبعِ نورِ نقطه‌ایِ دور» (Distant Point Light Source) مدل می‌کنیم. این مهم‌ترین مدلِ نورپردازی است و برای صحنه‌های داخلی و همچنین صحنه‌های بیرونی کاملاً مؤثر است. مقدارِ نورِ جمع‌شده توسط وصلهٔ سطح در این مدل به زاویهٔ θ بینِ جهتِ روشنایی و نرمال (عمود) بر سطح بستگی دارد (شکل ۲۵.۵).

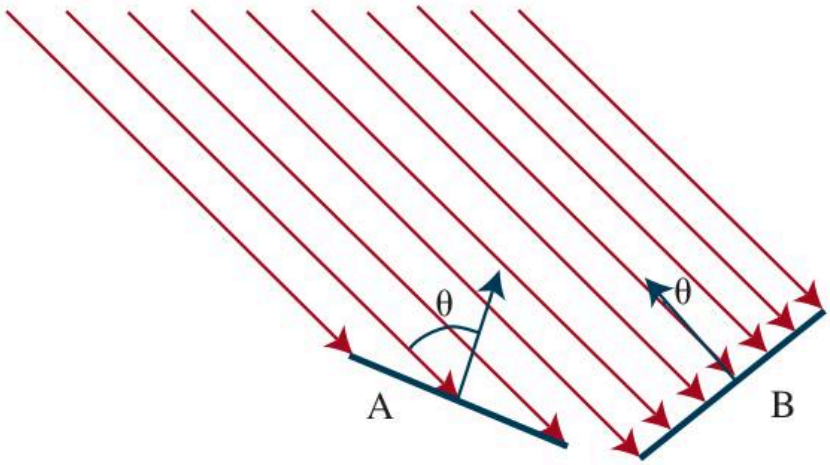
وصلهٔ سطحِ پخشیده‌ای که توسط این مدل روشن شده کسری از نوری را که جمع می‌کند بازتاب خواهد کرد، که «سپیداییِ پخشیده» (Diffuse Albedo) نامیده می‌شود. برای سطوحِ عملی، این در بازهٔ 0.05–0.95 قرار دارد. «قانونِ کسینوسیِ لامبرت» (Lambert's Cosine Law) بیان می‌کند روشناییِ وصلهٔ پخشیده به‌صورتِ زیر است:

$$I = \rho I_0 \cos \theta$$

که I_0 شدتِ منبعِ نور، θ زاویهٔ بینِ جهتِ منبعِ نور و نرمالِ سطح، و ρ سپیداییِ پخشیده است. این قانون پیش‌بینی می‌کند پیکسل‌های روشنِ تصویر از وصله‌های سطحی می‌آیند که مستقیماً رو به نورند و پیکسل‌های تیره از وصله‌هایی که نور را فقط مماسی می‌بینند، پس سایه‌روشنِ رویِ سطح اطلاعاتِ شکلی فراهم می‌کند. اگر سطح نتواند منبع را ببیند، در سایه است. «سایه‌ها» (Shadow) به‌ندرت سیاهِ یکنواخت‌اند، زیرا سطحِ سایه‌دار معمولاً نوری از منابعِ دیگر دریافت می‌کند. بیرون، مهم‌ترین منبعِ غیر از خورشید آسمان است، که کاملاً روشن است. داخل، نورِ بازتابیده از سطوحِ دیگر وصله‌های سایه‌دار را روشن می‌کند. این «بازتاب‌های متقابل» (Interreflection) می‌توانند اثرِ چشمگیری بر روشناییِ سطوحِ دیگر نیز داشته باشند. این اثرها گاهی با افزودنِ عبارتِ ثابتِ «روشناییِ محیطی» (Ambient Illumination) به شدتِ پیش‌بینی‌شده مدل می‌شوند.



شکل ۲۵.۴ – این عکس انواعِ اثرهای روشنایی را نشان می‌دهد. رویِ پارچِ فولادِ ضدزنگ برق‌های آینه‌ای هست. پیازها و هویج‌ها سطوحِ پخشیدهٔ روشنی‌اند، زیرا رو به جهتِ نورند. سایه‌ها در نقطه‌های سطحی ظاهر می‌شوند که اصلاً نمی‌توانند منبعِ نور را ببینند. درونِ قابلمه چند سطحِ پخشیدهٔ تیره هست که نور با زاویهٔ مماسی به آن‌ها برخورد می‌کند. (همچنین سایه‌هایی درونِ قابلمه هست.) عکس از ریمان کابان/Image Professionals GmbH/Alamy Stock Photo.



شکل ۲۵.۵ – دو وصلهٔ سطح توسط منبعِ نقطه‌ایِ دوری روشن می‌شوند، که پرتوهایش به‌صورتِ فلش‌های روشن نشان داده شده‌اند. وصلهٔ A از منبع متمایل است (θ نزدیکِ ۹۰°) و انرژیِ کمتری جمع می‌کند، زیرا پرتوهای نورِ کمتری به‌ازایِ واحدِ مساحتِ سطح قطع می‌کند. وصلهٔ B، رو به منبع (θ نزدیکِ ۰°)، انرژیِ بیشتری جمع می‌کند.

۲۵.۲.۵ رنگ

میوه رشوه‌ای است که درخت به جانوران پیشنهاد می‌کند تا دانه‌هایش را حمل کنند. درختانی که می‌توانند سیگنال دهند این رشوه کی آماده است مزیت دارند، و همچنین جانورانی که می‌توانند این سیگنال‌ها را بخوانند. در نتیجه، بیشترِ میوه‌ها سبز شروع می‌شوند و وقتی رسیده‌اند قرمز یا زرد می‌شوند، و بیشترِ جانورانِ میوه‌خوار می‌توانند این تغییراتِ رنگی را ببینند. به‌طورِ کلی، نورِ رسیده به چشم مقدارهای انرژیِ متفاوتی در طول‌موج‌های متفاوت دارد، و با «چگالیِ انرژیِ طیفی» (Spectral Energy Density) بازنمایی می‌شود.

دوربین‌ها و سامانهٔ بینایی انسان به نور در طول‌موج‌های حدودِ ۳۸۰ نانومتر (بنفش) تا حدودِ ۷۵۰ نانومتر (قرمز) پاسخ می‌دهند. در سامانه‌های تصویربرداریِ رنگی، انواعِ متفاوتی از گیرنده هست که به طول‌موج‌های متفاوت کم‌وبیش قوی پاسخ می‌دهند. در انسان‌ها، حسِ رنگ وقتی رخ می‌دهد که سامانهٔ بینایی پاسخِ گیرنده‌های نزدیکِ هم رویِ شبکیه را مقایسه می‌کند. سامانه‌های بینایی رنگیِ جانوران معمولاً انواعِ نسبتاً کمی گیرنده دارند، و بنابراین جزئیاتِ نسبتاً کمی از تابعِ چگالیِ انرژیِ طیفی را بازنمایی می‌کنند (برخی جانوران فقط یک نوعِ گیرنده دارند؛ برخی تا شش نوع). بینایی رنگیِ انسان توسط سه نوعِ گیرنده تولید می‌شود. بیشترِ سامانه‌های دوربینِ رنگی نیز فقط از سه نوعِ گیرنده استفاده می‌کنند، زیرا تصاویر برای انسان‌ها تولید می‌شوند، اما برخی سامانه‌های تخصصی می‌توانند اندازه‌گیری‌های بسیار دقیقی از چگالیِ انرژیِ طیفی تولید کنند.

چون بیشترِ انسان‌ها سه نوعِ گیرندهٔ حساس به رنگ دارند، «اصلِ سه‌رنگی» (Principle of Trichromacy) برقرار است. این ایده، که نخستین بار توسط توماس یانگ در ۱۸۰۲ پیشنهاد شد، بیان می‌کند ناظرِ انسانی می‌تواند ظاهرِ بصریِ هر چگالیِ انرژیِ طیفی، هرچند پیچیده، را با مخلوط‌کردنِ مقدارهای مناسبِ فقط سه «رنگِ اولیه» (Primary) تطبیق دهد. رنگ‌های اولیه منابعِ نورِ رنگی‌اند، که چنان انتخاب می‌شوند که هیچ مخلوطی از هر دوِ آن‌ها با سومی تطبیق نکند. انتخابِ رایج یکی قرمز، یکی سبز و یکی آبی است، که به‌صورتِ RGB مخفف می‌شود. هرچند شیءِ رنگیِ داده‌شده ممکن است فرکانس‌های مؤلفهٔ بسیاری از نور داشته باشد، می‌توانیم رنگ را با مخلوط‌کردنِ فقط سه رنگِ اولیه تطبیق دهیم، و بیشترِ مردم بر نسبت‌های مخلوط توافق خواهند کرد. این بدان‌معناست می‌توانیم تصاویرِ رنگی را با فقط سه عدد به‌ازایِ هر پیکسل—مقادیرِ RGB—بازنمایی کنیم.

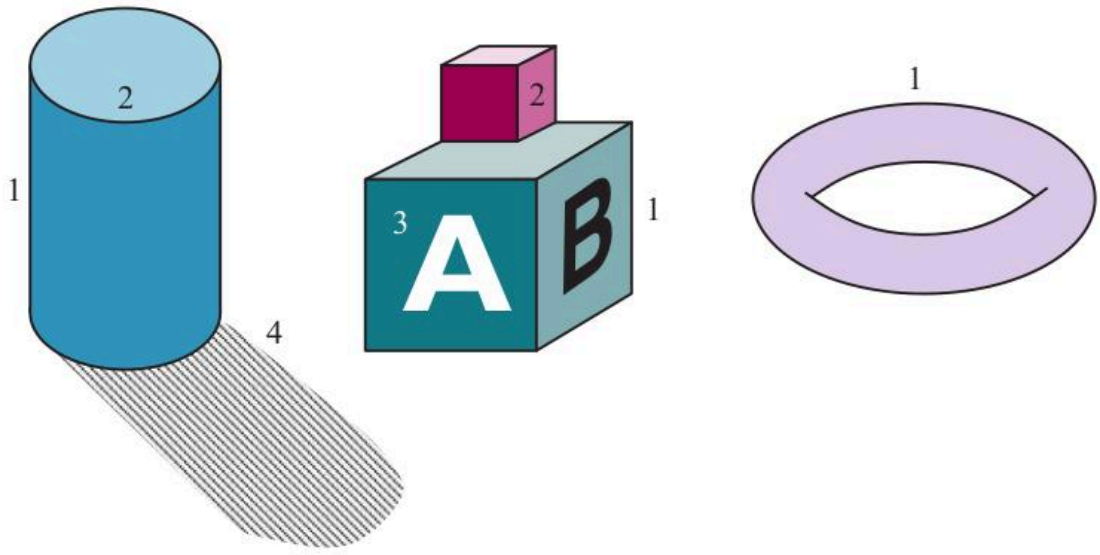
برای بیشترِ کاربردهای بینایی رایانه، به‌اندازهٔ کافی دقیق است سطح را به‌عنوانِ دارایِ سه سپیدایی پخشیدهٔ (RGB) متفاوت و منابعِ نور را به‌عنوانِ دارایِ سه شدتِ (RGB) مدل کنیم. سپس قانونِ کسینوسیِ لامبرت را بر هر یک اعمال می‌کنیم تا مقادیرِ پیکسلِ قرمز، سبز و آبی به‌دست آید. این مدل، به‌درستی، پیش‌بینی می‌کند همان سطح تحتِ نورهای رنگیِ متفاوت وصله‌های تصویرِ رنگیِ متفاوتی تولید می‌کند. در واقع، ناظرانِ انسانی در نادیده‌گرفتنِ اثرهای نورهای رنگیِ متفاوت کاملاً خوب‌اند و به نظر می‌رسد رنگی را برآورد می‌کنند که سطح تحتِ نورِ سفید می‌داشت، اثری که «ثباتِ رنگ» (Color Constancy) نامیده می‌شود.

۲۵.۳ ویژگی‌های سادهٔ تصویر

نور از اشیای صحنه بازتاب می‌شود تا تصویری شاملِ، مثلاً، دوازده میلیون پیکسلِ سه‌بایتی تشکیل دهد. مانندِ همهٔ حس‌گذارها، نویزی در تصویر خواهد بود، و در هر حال دادهٔ زیادی برای سروکله‌زدن با آن هست. راهِ شروعِ تحلیلِ این داده تولیدِ بازنمایی‌های ساده‌شده‌ای است که آنچه مهم است را آشکار می‌کنند، اما جزئیات را کاهش می‌دهند. بخشِ زیادی از رویهٔ کنونی این بازنمایی‌ها را از داده یاد می‌گیرد. اما چهار ویژگی از تصاویر و ویدئو هست که به‌ویژه کلی‌اند: لبه‌ها، بافت، جریانِ نوری و بخش‌بندی به ناحیه‌ها.

«لبه» (Edge) جایی رخ می‌دهد که تفاوتِ بزرگی در شدتِ پیکسل در بخشی از تصویر هست. ساختنِ بازنمایی‌های لبه شاملِ عملیاتِ محلی رویِ تصویر است—باید مقدارِ پیکسلی را با برخی مقادیرِ نزدیک مقایسه کنید—و به هیچ دانشی دربارهٔ آنچه در تصویر است نیاز ندارد. پس، تشخیصِ لبه می‌تواند زود در خطِ لولهٔ عملیات بیاید و آن را عملیاتِ «اولیه»

یا «سطح‌پایین» می‌نامیم.



شکل ۲۵.۶ – انواعِ متفاوتِ لبه: (۱) ناپیوستگی‌های عمق؛ (۲) ناپیوستگی‌های جهتِ سطح؛ (۳) ناپیوستگی‌های بازتاب؛ (۴) ناپیوستگی‌های روشنایی (سایه‌ها).

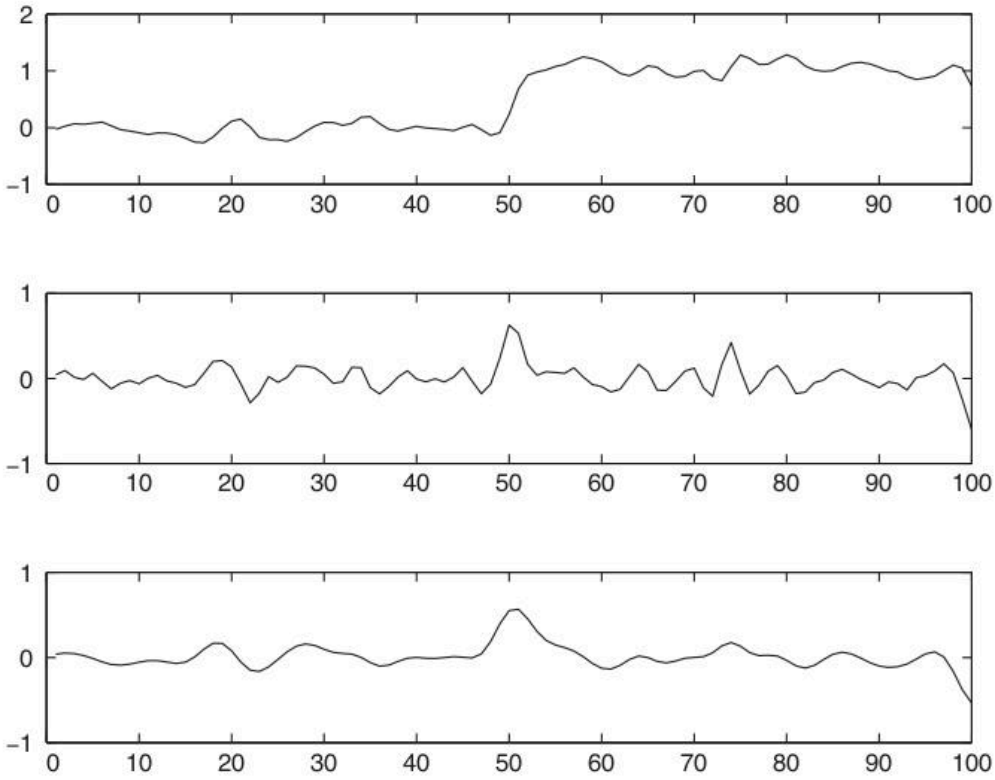
بقیهٔ عملیات‌ها مستلزمِ رسیدگی به ناحیهٔ بزرگ‌تری از تصویرند. برای نمونه، توصیفِ بافت بر استخری از پیکسل‌ها اعمال می‌شود—برای گفتنِ «راه‌راه»، باید چند راه‌راه ببینید. «جریانِ نوری» (Optical Flow) بازنماییِ این است که پیکسل‌ها از یک تصویرِ دنباله به تصویرِ بعدی به کجا حرکت می‌کنند، و این می‌تواند ناحیهٔ بزرگ‌تری را پوشش دهد. «بخش‌بندی» (Segmentation) تصویر را به ناحیه‌هایی از پیکسل‌ها می‌بُرد که به‌طورِ طبیعی به هم تعلق دارند، و انجامِ این کار مستلزمِ نگاه‌کردن به کلِ ناحیه است. عملیات‌هایی مانندِ این گاهی «عملیاتِ سطح‌میانی» نامیده می‌شوند.

۲۵.۳.۱ لبه‌ها

لبه‌ها خطوطِ راست یا منحنی‌هایی در صفحهٔ تصویرند که در عرضِ آن‌ها تغییرِ «چشمگیری» در روشناییِ تصویر هست. هدفِ تشخیصِ لبه انتزاعِ دور از تصویرِ شلوغِ چندمگابایتی و به سمتِ بازنماییِ فشرده‌تر و انتزاعی‌تر است، چنان‌که در شکل ۲۵.۶.

اثرهای صحنه اغلب به تغییرهای بزرگِ شدتِ تصویر می‌انجامند، و بنابراین لبه‌هایی در تصویر تولید می‌کنند. ناپیوستگی‌های عمق (که در شکل با ۱ برچسب خورده‌اند) می‌توانند لبه ایجاد کنند، زیرا وقتی از ناپیوستگی عبور می‌کنید، رنگ معمولاً تغییر می‌کند. وقتی نرمالِ سطح تغییر می‌کند (با ۲ برچسب خورده)، شدتِ تصویر اغلب تغییر می‌کند. وقتی بازتابِ سطح تغییر می‌کند (با ۳ برچسب خورده)، شدتِ تصویر اغلب تغییر می‌کند. سرانجام، سایه (با ۴ برچسب خورده) ناپیوستگیِ روشنایی است که لبه‌ای در تصویر ایجاد می‌کند، حتی اگر لبه‌ای در شیء نباشد. تشخیص‌دهنده‌های لبه نمی‌توانند علتِ ناپیوستگی را از هم باز کنند، که به پردازشِ بعدی واگذار می‌شود.

یافتنِ لبه‌ها نیاز به مراقبت دارد. شکل ۲۵.۷ (بالا) مقطعِ یک‌بعدیِ تصویری را عمود بر لبه‌ای نشان می‌دهد، با لبه‌ای در x = 50. ممکن است تصویر را مشتق بگیرید و به دنبالِ جاهایی بگردید که اندازهٔ مشتقِ I'(x) بزرگ است. این تقریباً کار می‌کند، اما در شکل ۲۵.۷ (وسط)، می‌بینیم هرچند قله‌ای در x = 50 هست، قله‌های فرعی‌ای نیز در مکان‌های دیگر (مثلاً x = 75) هستند که می‌توانند با لبه‌های واقعی اشتباه گرفته شوند. این‌ها به‌دلیلِ وجودِ «نویز» (Noise) در تصویر پدید می‌آیند. نویز در این‌جا به معنای تغییرهایی در مقدارِ پیکسل است که به لبه ربطی ندارند. برای نمونه، می‌تواند نویزِ گرمایی در دوربین باشد؛ می‌تواند خراش‌هایی رویِ سطحِ شیء باشد که نرمالِ سطح را در ریزترین مقیاس تغییر می‌دهند؛ می‌تواند تغییرهای جزئی در سپیداییِ سطح باشد؛ و مانندِ آن. هر یک از این اثرها گرادیان را بزرگ نشان می‌دهند، اما به این معنا نیستند که لبه‌ای حاضر است. اگر نخست تصویر را «هموار» کنیم، قله‌های کاذب کاهش می‌یابند، چنان‌که در شکل ۲۵.۷ (پایین) می‌بینیم. هموارسازی شاملِ استفاده از پیکسل‌های اطراف برای سرکوبِ نویز است. مقدارِ «واقعی»ِ پیکسلِ خود را به‌عنوانِ مجموعِ وزن‌داری از پیکسل‌های نزدیک پیش‌بینی خواهیم کرد، با وزنِ بیشتر برای نزدیک‌ترین پیکسل‌ها.



شکل ۲۵.۷ – بالا: پروفایلِ شدتِ I(x) در طولِ مقطعِ یک‌بعدی در عرضِ لبهٔ پله‌ای. وسط: مشتقِ شدت، I'(x). مقادیرِ بزرگِ این تابع با لبه‌ها متناظرند، اما تابع نویزی است. پایین: مشتقِ نسخهٔ هموارشدهٔ شدت. لبهٔ نامزدِ نویزیِ x = 75 ناپدید شده است.

به یاد آورید تابعِ گاوسیِ میانگین-صفر با انحرافِ معیارِ σ به‌صورتِ زیر است:

$$G_\sigma(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-x^2/(2\sigma^2)}$$
$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/(2\sigma^2)}$$

(در یک بُعد و دو بُعد).

اعمالِ فیلترِ گاوسی به این معناست که شدتِ $I(x_0,y_0)$ با مجموعِ، رویِ همهٔ پیکسل‌های (x,y)، $I(x,y)G_\sigma(d)$ جایگزین می‌شود، که d فاصله از (x_0,y_0) تا (x,y) است. این نوعِ مجموعِ وزن‌دار چنان رایج است که نام و نمادگذاریِ خاصی برایش هست. می‌گوییم تابعِ h «پیچشِ» (Convolution) دو تابعِ f و g است (که با $h = f * g$ نشان داده می‌شود) اگر داشته باشیم:

```
h(x) = ∑_{u=−∞}^{∞} f(u) g(x − u)
```

در یک بُعد، یا

```
h(x, y) = ∑_{u=−∞}^{∞} ∑_{v=−∞}^{∞} f(u, v) g(x − u, y − v)
```

در دو بُعد. پس تابعِ هموارسازی با پیچیدنِ تصویر با گاوسی، $I * G_\sigma$، محقق می‌شود. σ یک پیکسل برای هموارکردنِ مقدارِ کمی نویز کافی است، در حالی که ۲ پیکسل مقدارِ بیشتری را هموار می‌کند، اما با ازدست‌رفتنِ برخی جزئیات. چون تأثیرِ گاوسی به‌سرعت با فاصله محو می‌شود، در عمل می‌توانیم $\pm\infty$ در جمع‌ها را با چیزی مانندِ $\pm 3\sigma$ جایگزین کنیم.

اینجا فرصتِ بهینه‌سازی داریم: می‌توانیم هموارسازی و لبه‌یابی را در عملیاتِ تکی ترکیب کنیم. قضیه‌ای هست که برای هر توابعِ f و g، مشتقِ پیچش، $(f * g)'$، با پیچشِ با مشتق، $f * (g')$، برابر است. پس به‌جای هموارسازیِ تصویر و سپس مشتق‌گیری، می‌توانیم صرفاً تصویر را با مشتقِ تابعِ هموارسازیِ گاوسی، G'_σ، بپیچانیم. سپس آن قله‌های پاسخ را که بالای آستانه‌ای هستند، که برای حذفِ قله‌های کاذبِ ناشی از نویز انتخاب شده، به‌عنوانِ لبه علامت می‌زنیم.

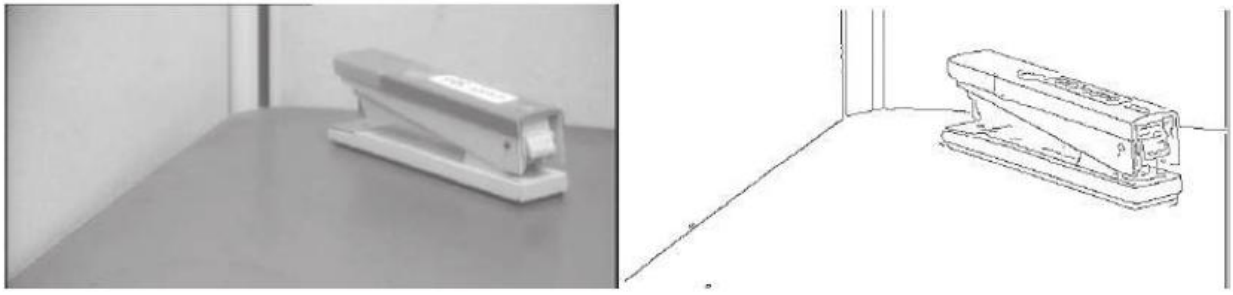
تعمیمِ طبیعیِ این الگوریتم از مقطع‌های یک‌بعدی به تصاویرِ دوبعدیِ عمومی هست. در دو بُعد، لبه‌ها ممکن است در هر زاویهٔ θ باشند. با درنظرگرفتنِ روشناییِ تصویر به‌عنوانِ تابعِ اسکالری از متغیرهای x، y، گرادیانِ آن بردارِ $\nabla I = (\partial I/\partial x، \partial I/\partial y)$ است. لبه‌ها با مکان‌هایی در تصاویر متناظرند که روشنایی در آن‌ها تغییرِ تند می‌کند، و بنابراین اندازهٔ گرادیان، $\|\nabla I\|$، باید در نقطهٔ لبه بزرگ باشد. وقتی تصویر روشن‌تر یا تاریک‌تر می‌شود، بردارِ گرادیان در هر نقطه بلندتر یا کوتاه‌تر می‌شود، اما جهتِ گرادیانِ $\nabla I/\|\nabla I\| = (\cos\theta، \sin\theta)$ تغییر نمی‌کند. این در هر پیکسل $\theta = \theta(x,y)$ به ما می‌دهد، که «جهتِ لبه» (Orientation) را در آن پیکسل تعریف می‌کند. این ویژگی اغلب مفید است، زیرا به شدتِ تصویر بستگی ندارد.

چنان‌که از بحثِ تشخیصِ لبه در سیگنال‌های یک‌بعدی انتظار دارید، برای تشکیلِ گرادیان، در واقع ∇I را محاسبه نمی‌کنیم، بلکه $\nabla(I * G_\sigma)$ را، پس از هموارسازیِ تصویر با پیچیدنِ آن با گاوسی. ویژگیِ پیچش‌ها این است که این معادلِ پیچیدنِ تصویر با مشتق‌های جزئیِ گاوسی است. پس از محاسبهٔ گرادیان، می‌توانیم با یافتنِ نقطه‌های لبه و پیونددادنِ آن‌ها به هم، لبه‌ها را به‌دست آوریم. برای گفتنِ اینکه نقطه‌ای نقطهٔ لبه است یا نه، باید نقطه‌های دیگری را در فاصلهٔ کوچکی به جلو و عقب در جهتِ گرادیان نگاه کنیم. اگر اندازهٔ گرادیان در یکی از این نقطه‌ها بزرگ‌تر باشد،

می‌توانستیم با جابه‌جاییِ بسیار جزئیِ منحنیِ لبه نقطهٔ لبهٔ بهتری بگیریم. افزون بر این، اگر اندازهٔ گرادیان بیش‌ازحد کوچک باشد، نقطه نمی‌تواند نقطهٔ لبه باشد. پس در نقطهٔ لبه، اندازهٔ گرادیان بیشینهٔ محلی در جهتِ گرادیان است، و اندازهٔ گرادیان بالای آستانهٔ مناسبی است.

پس از علامت‌گذاریِ پیکسل‌های لبه با این الگوریتم، مرحلهٔ بعدی پیونددادنِ آن پیکسل‌هایی است که به همان منحنی‌های لبه تعلق دارند. این را می‌توان با فرضِ اینکه هر دو پیکسلِ مجاوری که هر دو پیکسلِ لبه با جهت‌های سازگارند به همان منحنیِ لبه تعلق دارند انجام داد.

تشخیصِ لبه کامل نیست. شکل ۲۵.۸ (الف) تصویری از صحنه‌ای شاملِ منگنه‌ای رویِ میز نشان می‌دهد، و شکل ۲۵.۸ (ب) خروجیِ الگوریتمِ تشخیصِ لبه را رویِ این تصویر. چنان‌که می‌بینید، خروجی کامل نیست: شکاف‌هایی هست که در آن‌ها هیچ لبه‌ای ظاهر نمی‌شود، و لبه‌های «نویزی» هست که با هیچ‌چیزِ مهمی در صحنه متناظر نیستند. مراحلِ بعدیِ پردازش باید این خطاها را تصحیح کنند.



شکل ۲۵.۸ – (الف) عکسی از یک منگنه. (ب) لبه‌های محاسبه‌شده از (الف).

۲۵.۳.۲ بافت

در زبانِ روزمره، «بافت» (Texture) سطوح به آنچه هنگامِ کشیدنِ انگشت رویِ آن‌ها احساس می‌شود اشاره می‌کند (واژه‌های «textile»، «texture» و «text» ریشهٔ لاتینِ مشترکی دارند، واژه‌ای برای بافندگی). در بینایی محاسباتی، بافت به الگویی رویِ سطح اشاره دارد که می‌توان به‌صورتِ بصری حس کرد. معمولاً، این الگوها تقریباً منظم‌اند. نمونه‌ها شاملِ الگویِ پنجره‌های ساختمان، بخیه‌های ژاکتِ پشمی، لکه‌های پوستِ پلنگ، تیغه‌های علفِ چمن، سنگریزه‌های ساحل و جمعیتِ مردم در استادیوم است.

گاهی چیدمان کاملاً تناوبی است، مانندِ بخیه‌های ژاکتِ پشمی؛ در نمونه‌های دیگر، مانندِ سنگریزه‌های ساحل، نظم فقط در معنایی آماری است: چگالیِ سنگریزه‌ها تقریباً در بخش‌های متفاوتِ ساحل یکسان است. مدلِ تقریبیِ معمولِ بافت الگویِ تکرارِیِ عناصری است، که گاهی «تکسِل» (Texel) نامیده می‌شوند. این مدل کاملاً مفید است، زیرا ساختن یا یافتنِ بافت‌های واقعی که هرگز تکرار نمی‌شوند به‌شکلِ شگفت‌آوری سخت است.

بافت ویژگیِ وصلهٔ تصویر است، نه پیکسلی در انزوا. توصیفِ خوبِ بافتِ وصله باید خلاصه کند وصله چه‌شکل‌اند. توصیف نباید وقتی نورپردازی تغییر می‌کند تغییر کند. این استفاده از نقطه‌های لبه را رد می‌کند؛ اگر بافتی روشن روشن شود، بسیاری از مکان‌های درونِ وصله کنتراستِ بالا خواهند داشت و نقطه‌های لبه تولید می‌کنند؛ اما اگر همان بافت

زیرِ نورِ کم‌روشن‌تری دیده شود، بسیاری از این لبه‌ها بالای آستانه نخواهند بود. توصیف باید به‌شیوه‌ای معقول وقتی وصله می‌چرخد تغییر کند. حفظِ تفاوتِ بینِ راه‌راه‌های عمودی و افقی مهم است، اما نه اگر راه‌راه‌های عمودی به افقی چرخانده شوند.

نشان داده شده بازنمایی‌های بافتی با این ویژگی‌ها برای دو تکلیفِ کلیدی مفیدند. نخستین شناساییِ اشیا است— گورخر و اسب شکلِ مشابهی دارند، اما بافت‌های متفاوت. دومین تطبیقِ وصله‌های یک تصویر با وصله‌های تصویرِ دیگر است، گامِ کلیدی در بازیابیِ اطلاعاتِ سه‌بعدی از تصاویرِ متعدد (بخش ۲۵.۶.۱).

در این‌جا ساختِ پایه‌ای برای بازنماییِ بافت آمده. با توجه به وصلهٔ تصویر، جهتِ گرادیان را در هر پیکسلِ وصله محاسبه کن، و سپس وصله را با «هیستوگرامِ جهت‌ها» مشخص کن. جهت‌های گرادیان تا حدِ زیادی نسبت به تغییرهای روشنایی ناورداند (گرادیان بلندتر خواهد شد، اما جهت تغییر نمی‌کند). به نظر می‌رسد هیستوگرامِ جهت‌ها جنبه‌های مهمِ بافت را درمی‌گیرد. برای نمونه، راه‌راه‌های عمودی دو قله در هیستوگرام خواهند داشت (یکی برای سمتِ چپِ هر راه‌راه و یکی برای راست)؛ لکه‌های پلنگ جهت‌های توزیع‌شدهٔ یکنواخت‌تری خواهند داشت.

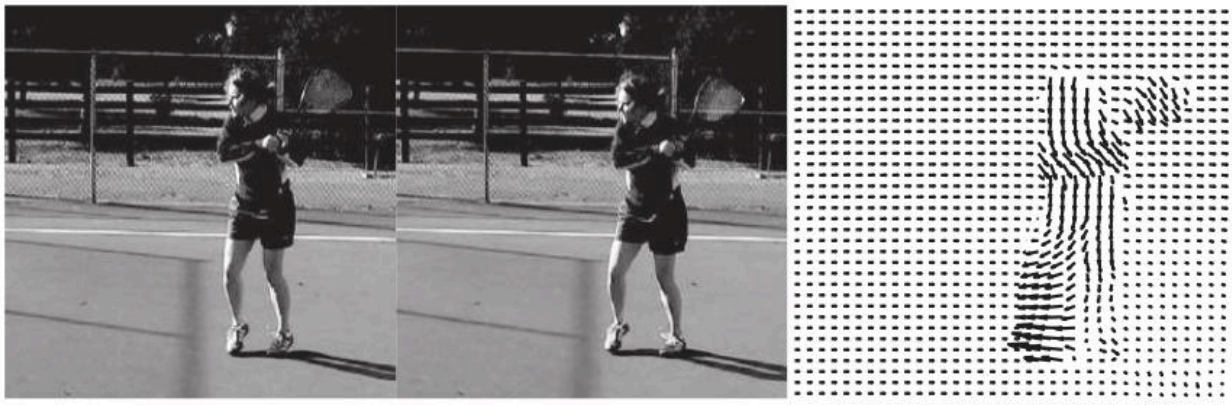
اما نمی‌دانیم وصله را چقدر بزرگ توصیف کنیم. دو راهبرد هست. در کاربردهای تخصصی، اطلاعاتِ تصویر آشکار می‌کند وصله باید چقدر بزرگ باشد (مثلاً، ممکن است وصله‌ای پر از راه‌راه را تا وقتی گورخر را بپوشاند رشد داد). جایگزین توصیفِ وصله‌ای به‌مرکزیتِ هر پیکسل برای دامنه‌ای از مقیاس‌هاست. این دامنه معمولاً از چند پیکسل تا وسعتِ تصویر می‌رود. حالا وصله را به ظرف‌هایی تقسیم کن، و در هر ظرف هیستوگرامِ جهتی بساز، سپس الگویِ هیستوگرام‌ها را در عرضِ ظرف‌ها خلاصه کن. دیگر ساختنِ این توصیف‌ها با دست معمول نیست. در عوض، از شبکه‌های عصبیِ پیچشی برای تولیدِ بازنمایی‌های بافت استفاده می‌شود. اما بازنمایی‌های ساخته‌شده توسط شبکه‌ها به نظر می‌رسد تقریباً همین ساخت را آینه می‌کنند.

۲۵.۳.۳ جریانِ نوری

بعد، بیایید بررسی کنیم وقتی به‌جای یک تصویرِ ایستای تکی، دنبالهٔ ویدئویی داشته باشیم چه رخ می‌دهد. هرگاه حرکتِ نسبی بینِ دوربین و یک یا چند شیءِ صحنه باشد، حرکتِ ظاهریِ حاصل در تصویر «جریانِ نوری» (Optical Flow) نامیده می‌شود. این جهت و سرعتِ حرکتِ ویژگی‌های تصویر را در نتیجهٔ حرکتِ نسبی بینِ بیننده و صحنه توصیف می‌کند. برای نمونه، اشیای دورِ مشاهده‌شده از خودرویِ متحرک حرکتِ ظاهریِ بسیار کندتری از اشیای نزدیک دارند، پس نرخِ حرکتِ ظاهری می‌تواند چیزی دربارهٔ فاصله به ما بگوید.

در شکل ۲۵.۹ دو فریم از ویدئویِ بازیکنِ تنیس نشان می‌دهیم. در سمتِ راست، بردارهای جریانِ نوریِ محاسبه‌شده از این تصاویر را نمایش می‌دهیم. جریانِ نوری اطلاعاتِ مفیدی دربارهٔ ساختارِ صحنه رمز می‌کند—بازیکنِ تنیس در حالِ حرکت است و پس‌زمینه (تا حدِ زیادی) نیست. افزون بر این، بردارهای جریان چیزی دربارهٔ آنچه بازیکن انجام می‌دهد آشکار می‌کنند—یک بازو و یک پا سریع حرکت می‌کنند و سایرِ اعضای بدن نمی‌کنند.

میدانِ بردارِ جریانِ نوری را می‌توان با مؤلفه‌هایش $v_x(x,y)$ در جهتِ x و $v_y(x,y)$ در جهتِ y بازنمایی کرد. برای اندازه‌گیریِ جریانِ نوری، باید نقطه‌های متناظر را بینِ یک فریمِ زمانی و فریمِ بعدی بیابیم. فنّ بسیار ساده‌لوحانه‌ای بر این واقعیت مبتنی است که وصله‌های تصویرِ اطرافِ نقطه‌های متناظر الگوهای شدتِ مشابهی دارند. بلوکی از پیکسل‌ها به‌مرکزیتِ پیکسلِ p، یعنی (x_0,y_0)، در زمانِ t را در نظر بگیر. این بلوکِ پیکسل‌ها قرار است با بلوک‌های پیکسلیِ به‌مرکزیتِ پیکسل‌های نامزدِ متفاوتِ q_i در (x_0+D_x, y_0+D_y) در زمانِ t+Dt مقایسه شود.



شکل ۲۵.۹ — دو فریم از دنبالهٔ ویدئویی و میدانِ جریانِ نوریِ متناظر با جابه‌جایی از یک فریم به فریمِ دیگر. توجه کنید چگونه حرکتِ راکتِ تنیس و پایِ جلو توسط جهت‌های فلش‌ها درگیر می‌شود. (تصاویر با اجازه از توماس بروکس.)

یکی از معیارهای ممکنِ شباهت «مجموعِ مجذورِ تفاضل‌ها» (Sum of Squared Differences یا SSD) است:

$$SSD(D_x, D_y) = \sum_{(x, y)} (I(x, y, t) - I(x + D_x, y + D_y, t + Dt))^2$$

در این‌جا، (x,y) رویِ پیکسل‌های بلوکِ به‌مرکزیتِ (x_0,y_0) تغییر می‌کند. (D_x, D_y)ای را می‌یابیم که SSD را کمینه می‌کند. جریانِ نوری در (x_0,y_0) سپس $(v_x, v_y) = (D_x/Dt, D_y/Dt)$ است. توجه کنید برای کارکردنِ این، باید بافتی در صحنه باشد، که به پنجره‌هایی شاملِ تغییرِ چشمگیرِ روشنایی در میانِ پیکسل‌ها می‌انجامد. اگر به دیوارِ سفیدِ یکنواختی نگاه شود، آنگاه SSD برای نامزدهای متفاوتِ تطبیقِ q تقریباً یکسان خواهد بود و الگوریتم به حدسِ کور تقلیل می‌یابد. بهترین الگوریتم‌های اندازه‌گیریِ جریانِ نوری بر انواعِ قیدهای اضافی برای برخورد با موقعیت‌هایی تکیه می‌کنند که صحنه فقط تا حدی بافت‌دار است.

۲۵.۳.۴ بخش‌بندیِ تصاویرِ طبیعی

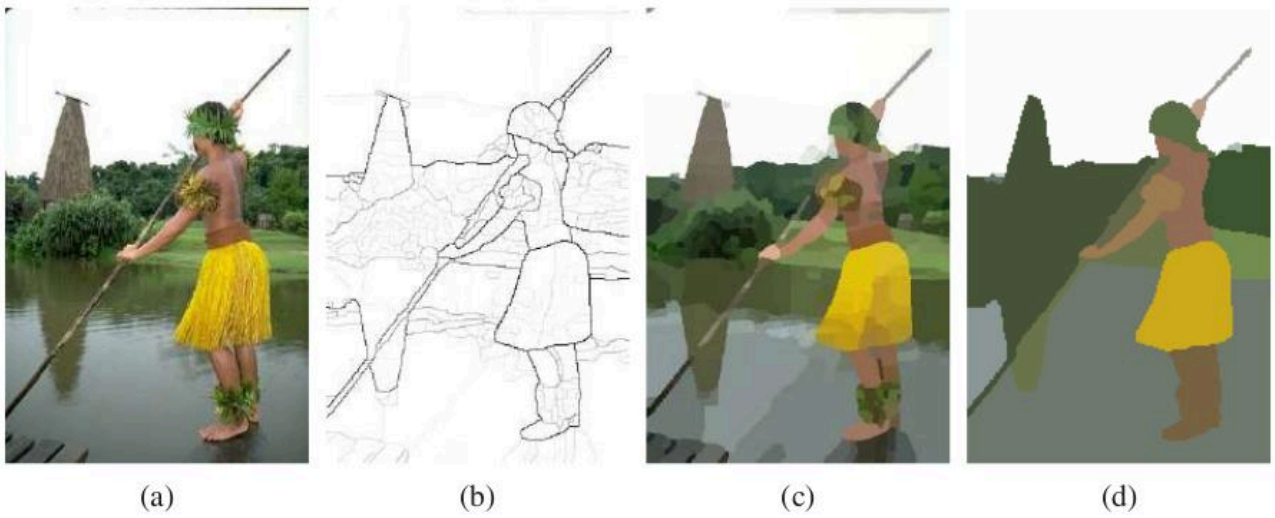
«بخش‌بندی» (Segmentation) فرایندِ شکستنِ تصویر به گروه‌هایی از پیکسل‌های مشابه است. ایدهٔ پایه این است که هر پیکسلِ تصویر را می‌توان با ویژگی‌های بصریِ خاصی، مانندِ روشنایی، رنگ و بافت، مرتبط کرد. درونِ یک شیء، یا بخشِ تکی از شیء، این ویژگی‌ها نسبتاً کم تغییر می‌کنند، در حالی که در عرضِ مرزِ بینِ دو شیء معمولاً تغییرِ بزرگِ یکی یا چندِ این ویژگی‌ها هست. باید افرازی از تصویر به مجموعه‌های پیکسل بیابیم که این قیدها تا حدِ ممکن خوب

ارضا شوند. توجه کنید یافتنِ فقط لبه‌ها کافی نیست، زیرا بسیاری از لبه‌ها مرزِ شیء نیستند. پس، برای نمونه، ببری در علف ممکن است در هر سمتِ هر راه‌راه و هر تیغهٔ علف لبه‌ای تولید کند. در همهٔ دادهٔ لبهٔ گیج‌کننده، ممکن است ببر را به خاطرِ راه‌راه‌ها از دست بدهیم.

دو راه برای مطالعهٔ مسئله هست، یکی بر تشخیصِ مرزهای این گروه‌ها تمرکز می‌کند و دیگری بر تشخیصِ خودِ گروه‌ها، که «ناحیه» (Region) نامیده می‌شوند. این را در شکل ۲۵.۱۰ نشان می‌دهیم، با تشخیصِ مرز در (ب) و استخراجِ ناحیه در (ج) و (د).

یکی از راه‌های صوری‌کردنِ مسئلهٔ تشخیصِ منحنی‌های مرز به‌عنوانِ مسئلهٔ رده‌بندی است، پذیرایِ فنونِ یادگیریِ ماشینی. منحنیِ مرزی در مکانِ پیکسلِ (x,y) جهتی θ خواهد داشت. همسایگیِ تصویری به‌مرکزیتِ (x,y) تقریباً مانندِ دیسکی است که با قطری به جهتِ θ به دو نیمه بریده شده. می‌توانیم احتمالِ $P_b(x,y,\theta)$ را محاسبه کنیم که منحنیِ مرزی در آن پیکسل در امتدادِ آن جهت هست، با مقایسهٔ ویژگی‌های دو نیمه. راهِ طبیعیِ پیش‌بینیِ این احتمال آموزشِ رده‌بندِ یادگیریِ ماشینی با استفاده از مجموعهٔ داده‌ای از تصاویرِ طبیعی است که در آن‌ها انسان‌ها مرزهای حقیقتِ زمینی را علامت زده‌اند—هدفِ رده‌بند علامت‌زدنِ دقیقاً همان مرزهایی است که انسان‌ها علامت زده‌اند و نه دیگران. مرزهای تشخیص‌داده‌شده با این فن بهتر از آن‌هایی‌اند که با فنِ سادهٔ تشخیصِ لبهٔ توصیف‌شده قبلاً یافت می‌شوند. اما هنوز دو محدودیت هست: (۱) پیکسل‌های مرزیِ تشکیل‌شده با آستانه‌گذاریِ $P_b(x,y,\theta)$ تضمین نمی‌شود منحنی‌های بسته تشکیل دهند، پس این رویکرد ناحیه‌ها را تحویل نمی‌دهد، و (۲) تصمیم‌گیری فقط از بافتِ محلی بهره می‌برد و از قیدهای سازگاریِ سراسری استفاده نمی‌کند.

رویکردِ جایگزین بر «خوشه‌بندیِ» پیکسل‌ها به ناحیه‌ها بر مبنای ویژگی‌های روشنایی، رنگ و بافت‌شان مبتنی است. راه‌های متفاوتِ متعددی برای صوری‌کردنِ ریاضیِ این شهود هست. برای نمونه، شی و مالیک (2000) این را به‌عنوانِ مسئلهٔ افرازِ گراف برپا کردند. گره‌های گراف با پیکسل‌ها و یال‌ها با اتصال‌های بینِ پیکسل‌ها متناظرند. وزنِ W_{ij} رویِ یالِ متصل‌کنندهٔ جفتِ پیکسلِ i و j بر مبنای اینکه دو پیکسل در روشنایی، رنگ، بافت و... چقدر مشابه‌اند است. سپس افرازهایی را می‌یابند که معیارِ «برشِ نرمال‌شده» را کمینه می‌کنند. به‌طورِ تقریبی، معیارِ افرازِ گراف کمینه‌کردنِ مجموعِ وزنِ اتصال‌های در عرضِ گروه‌ها و بیشینه‌کردنِ مجموعِ وزنِ اتصال‌های درونِ گروه‌هاست.



شکل ۲۵.۱۰ – (الف) تصویرِ اصلی. (ب) کانتورهای مرز، که هرچه مقدارِ P_b بالاتر باشد، کانتور تیره‌تر است. (ج) بخش‌بندی به ناحیه‌ها، متناظر با افرازِ ریزِ تصویر. ناحیه‌ها در میانگینِ رنگ‌هایشان رندر می‌شوند. (د) بخش‌بندی به ناحیه‌ها، متناظر با افرازِ درشت‌ترِ تصویر، که به ناحیه‌های کمتری می‌انجامد. (تصاویر با اجازه از پابلو آربلائز، مایکل مایر، چارلز فاولکس و جی‌تیندرا مالیک.)

معلوم می‌شود رویکردهای مبتنی بر یافتنِ مرزها و مبتنی بر یافتنِ ناحیه‌ها را می‌توان جفت کرد، اما این امکان‌ها را این‌جا کاوش نمی‌کنیم. از بخش‌بندیِ صرفاً مبتنی بر ویژگی‌های محلیِ سطح‌پایین مانندِ روشنایی و رنگ نمی‌توان انتظار داشت مرزهای نهاییِ درستِ همهٔ اشیای صحنه را تحویل دهد. برای یافتنِ قابلِ‌اعتمادِ مرزهای مرتبط با اشیا، همچنین لازم است دانشِ سطح‌بالایِ انواعِ اشیایی که ممکن است در صحنه انتظار داشت گنجانده شود. در این زمان، راهبردِ محبوب تولیدِ «بیش‌بخش‌بندی» (Over-Segmentation) تصویر است، که در آن تضمین می‌شود هیچ‌یک از مرزهای واقعی را از دست نداده باشیم اما ممکن است مرزهای کاذبِ اضافیِ بسیاری علامت زده باشیم. ناحیه‌های حاصل، که «ابرپیکسل» (Superpixel) نامیده می‌شوند، کاهشِ چشمگیری در پیچیدگیِ محاسباتی برای الگوریتم‌های گوناگون فراهم می‌کنند، زیرا تعدادِ ابرپیکسل‌ها ممکن است در صدها باشد، در مقایسه با میلیون‌ها پیکسلِ خام. بهره‌برداری از دانشِ سطح‌بالایِ اشیا موضوعِ بخشِ بعدی است، و تشخیصِ واقعیِ اشیا در تصاویر موضوعِ بخش ۲۵.۵.

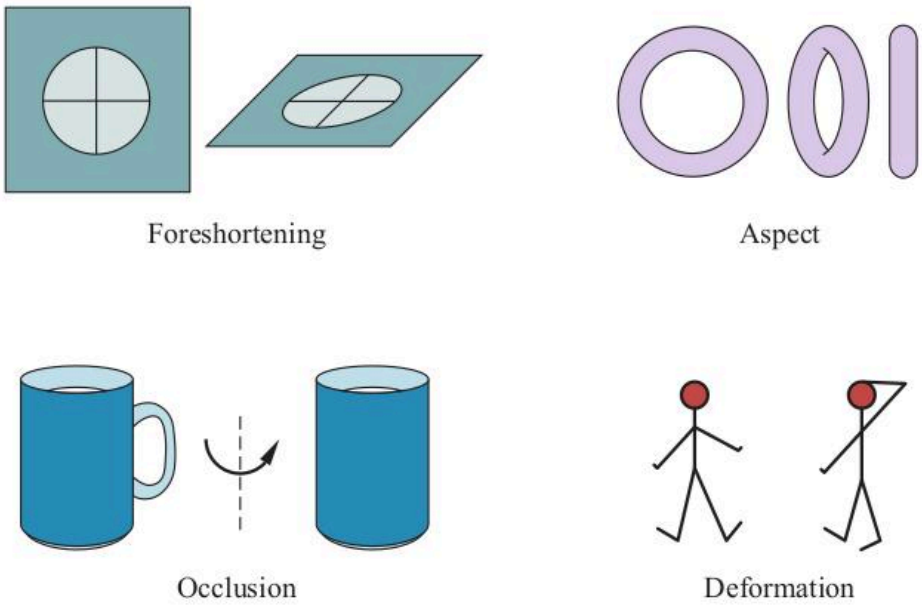
۲۵.۴ رده‌بندیِ تصاویر

رده‌بندیِ تصویر برای دو حالتِ اصلی به‌کار می‌رود. در یکی، تصاویر مربوط به اشیایند، برگرفته از رده‌بندیِ داده‌شده‌ای از رده‌ها، و چیزِ مهمِ دیگری در تصویر نیست—برای نمونه، کاتالوگی از تصاویرِ لباس یا مبلمان، که پس‌زمینه در آن مهم نیست، و خروجیِ رده‌بند «ژاکتِ کشمیر» یا «صندلیِ اداری» است. در حالتِ دیگر، هر تصویر صحنه‌ای شاملِ چند شیء را نشان می‌دهد. پس در علفزار ممکن است زرافه و شیری ببینید، و در اتاقِ نشیمن مبل و چراغی، اما انتظارِ زرافه یا زیردریایی در اتاقِ نشیمن را ندارید. حالا روش‌هایی برای رده‌بندیِ تصویرِ مقیاس‌بزرگ داریم که می‌توانند به‌دقت «علفزار» یا «اتاقِ نشیمن» را خروجی دهند.

سامانه‌های مدرن تصاویر را با استفاده از «ظاهر» (Appearance) (یعنی رنگ و بافت، در مقابلِ هندسه) رده‌بندی می‌کنند. دو دشواری هست. نخست، نمونه‌های متفاوتِ همان رده می‌توانند متفاوت به نظر برسند—برخی گربه‌ها سیاه‌اند و برخی دیگر نارنجی. دوم، همان گربه می‌تواند در زمان‌های متفاوت بسته به چند اثر متفاوت به نظر برسد،

(چنان‌که در شکل ۲۵.۱۱ نشان داده شده):

- **نورپردازی (Lighting)**، که روشنایی و رنگِ تصویر را تغییر می‌دهد.
- **کوتاه‌نمایی (Foreshortening)**، که باعث می‌شود الگویی که با زاویهٔ مایل دیده می‌شود تحریف شود.
- **جنبه (Aspect)**، که باعث می‌شود اشیا وقتی از جهت‌های متفاوت دیده می‌شوند متفاوت به نظر برسند. دوناتِ دیده‌شده از پهلو مانندِ بیضیِ پهن شده به نظر می‌رسد، اما از بالا حلقوی است.
- **انسداد (Occlusion)**، که برخی بخش‌های شیء در آن پنهان‌اند. اشیا می‌توانند یکدیگر را مسدود کنند، یا بخش‌هایی از یک شیء می‌توانند بخش‌های دیگر را مسدود کنند، اثری که «خودانسدادی» (Self-Occlusion) نامیده می‌شود.
- **تغییرِ شکل (Deformation)**، که شیء در آن شکلش را تغییر می‌دهد. برای نمونه، بازیکنِ تنیس بازوها و پاهایش را حرکت می‌دهد.



شکل ۲۵.۱۱ – منابعِ مهمِ تغییرِ ظاهر که می‌توانند تصاویرِ متفاوتی از همان شیء را متفاوت نشان دهند. نخست، عناصر می‌توانند کوتاه‌نما شوند، مانندِ وصلهٔ دایره‌ایِ بالایِ چپ. این وصله با زاویهٔ مایل دیده می‌شود و بنابراین در تصویر بیضوی است. دوم، اشیایِ دیده‌شده از جهت‌های متفاوت می‌توانند شکلشان را کاملاً چشمگیر تغییر دهند، پدیده‌ای که «جنبه» شناخته می‌شود. در بالایِ راست سه جنبهٔ متفاوت از یک دونات هست. انسداد باعث می‌شود دستهٔ لیوانِ پایینِ چپ وقتی لیوان می‌چرخد ناپدید شود. در این مورد، چون بدنه و دسته به همان لیوان تعلق دارند، خودانسدادی داریم. سرانجام، در پایینِ راست، برخی اشیا می‌توانند به‌شکلِ چشمگیری تغییر شکل دهند.

روش‌های مدرن با یادگیریِ بازنمایی‌ها و رده‌بندها از حجمِ بسیار بزرگِ دادهٔ آموزشی با شبکهٔ عصبیِ پیچشی با این مسائل برخورد می‌کنند. با مجموعهٔ آموزشیِ به‌اندازهٔ کافی غنی، رده‌بند هر اثرِ مهمی را بارها در آموزش دیده خواهد بود، و بنابراین می‌تواند اثر را جبران کند.

۲۵.۴.۱ رده‌بندیِ تصویر با شبکه‌های عصبیِ پیچشی

شبکه‌های عصبیِ پیچشی (CNN) رده‌بندهای تصویرِ به‌شکلِ تماشایی موفق‌ای‌اند. با دادهٔ آموزشیِ کافی و نبوغِ آموزشیِ کافی، CNNها سامانه‌های رده‌بندیِ بسیار موفقی تولید می‌کنند، بسیار بهتر از آنچه هر کسی با روش‌های دیگر توانسته تولید کند.

مجموعهٔ دادهٔ ImageNet نقشِ تاریخی در توسعهٔ سامانه‌های رده‌بندیِ تصویر با فراهم‌کردنِ بیش از ۱۴ میلیون تصویرِ آموزشی، رده‌بندی‌شده به بیش از ۳۰,۰۰۰ ردهٔ ریزدانه، ایفا کرد. ImageNet همچنین با رقابتِ سالانه‌ای پیشرفت را ترغیب کرد. سامانه‌ها هم با دقتِ رده‌بندیِ بهترین حدسِ تکی‌شان و هم با دقتِ ۵-برتر ارزیابی می‌شوند، که در آن سامانه‌ها مجازند پنج حدس ثبت کنند—مثلاً، مالاموت، هاسکی، آکیتا، ساموید، سگِ اسکیمو. ImageNet ۱۸۹ زیرردهٔ سگ دارد، پس حتی انسان‌های سگ‌دوست نیز برچسب‌زدنِ درستِ تصاویر با حدسِ تکی را دشوار می‌یابند.

در نخستین رقابتِ ImageNet در ۲۰۱۰، سامانه‌ها نمی‌توانستند بهتر از دقتِ ۵-برترِ ۷۰٪ عمل کنند. معرفیِ شبکه‌های عصبیِ پیچشی در ۲۰۱۲ و پالایشِ بعدی‌شان به دقتِ ۹۸٪ در ۵-برتر (فراتر از عملکردِ انسانی) و ۸۷٪ در دقتِ 1-برتر تا ۲۰۱۹ انجامید. دلیلِ اصلیِ این موفقیت به نظر می‌رسد این است که ویژگی‌های استفاده‌شده توسط رده‌بندهای CNN از داده یاد گرفته می‌شوند، نه دست‌ساخته توسط پژوهشگر؛ این تضمین می‌کند ویژگی‌ها واقعاً برای رده‌بندی مفیدند.

پیشرفتِ رده‌بندیِ تصویر به‌دلیلِ در دسترس‌بودنِ مجموعه‌های دادهٔ بزرگ و چالش‌برانگیز مانندِ ImageNet؛ به‌دلیلِ رقابت‌های مبتنی بر این مجموعه‌های داده که منصفانه و بازند؛ و به‌دلیلِ انتشارِ گستردهٔ مدل‌های موفق سریع بوده. برندگانِ رقابت‌ها کد و اغلب پارامترهای ازپیش‌آموزش‌دیدهٔ مدل‌هایشان را منتشر می‌کنند، که برای دیگران آسان می‌کند با معماری‌های موفق ور بروند و بکوشند بهترشان کنند.

۲۵.۴.۲ چرا شبکه‌های عصبیِ پیچشی تصاویر را خوب رده‌بندی می‌کنند

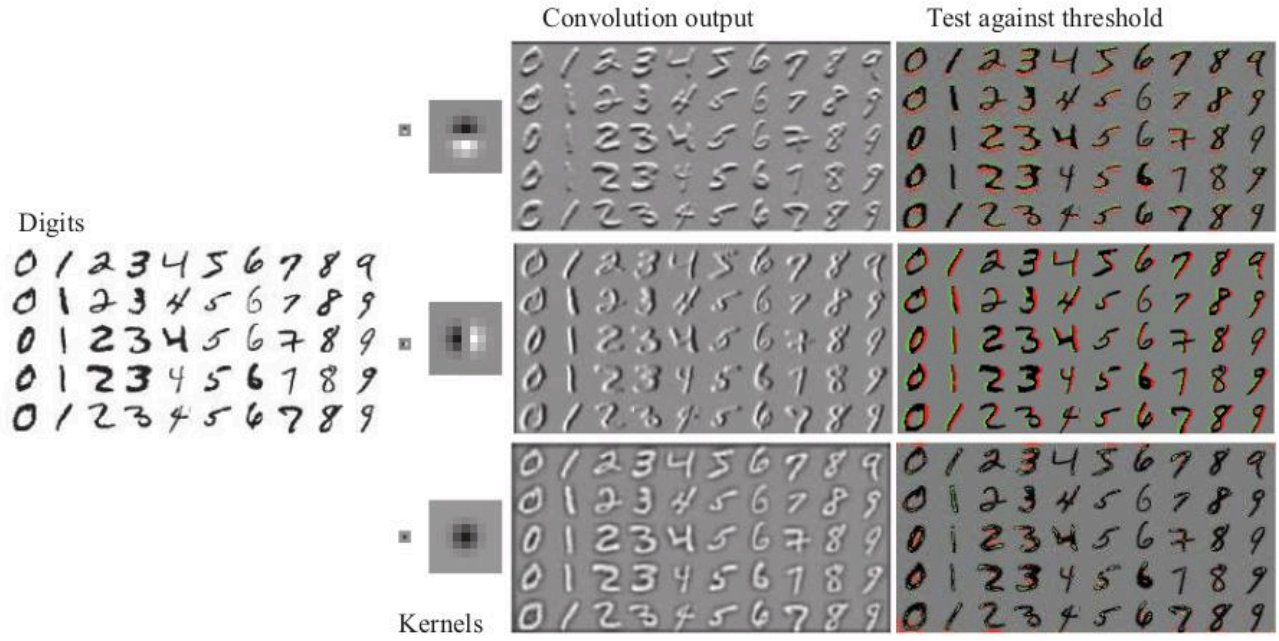
رده‌بندیِ تصویر بهتر با نگاه‌کردن به مجموعه‌های داده فهمیده می‌شود، اما ImageNet برای نگاه‌کردنِ با جزئیات بیش‌ازحد بزرگ است. مجموعهٔ دادهٔ MNIST مجموعه‌ای از ۷۰,۰۰۰ تصویرِ ارقامِ دست‌نویس، ۰–۹، است که اغلب به‌عنوانِ مجموعهٔ دادهٔ گرم‌کردنِ استاندارد استفاده می‌شود. نگاه‌کردن به این مجموعهٔ داده (چند مثال در شکل ۲۵.۱۲ ظاهر می‌شود) برخی ویژگی‌های مهم و کاملاً کلی را آشکار می‌کند. می‌توانید تصویری از رقمی بگیرید و تعدادی تغییرِ کوچک بدونِ تغییرِ هویتِ رقم انجام دهید: می‌توانید آن را جابه‌جا، بچرخانید، روشن‌تر یا تاریک‌تر، کوچک‌تر یا بزرگ‌تر کنید. این بدان‌معناست مقادیرِ پیکسلِ منفرد چندان اطلاعاتی نیستند—می‌دانیم ۸ باید چند پیکسلِ تیره در مرکز داشته باشد و ۰ نباید، اما آن پیکسل‌های تیره در هر نمونهٔ ۸ در مکان‌های پیکسلیِ کمی متفاوت خواهند بود.

ویژگیِ مهمِ دیگرِ تصاویر این است که الگوهای محلی می‌توانند کاملاً اطلاعاتی باشند: ارقامِ ۰، ۶، ۸ و ۹ حلقه دارند؛ ارقامِ ۴ و ۸ تقاطع دارند؛ ارقامِ ۱، ۲، ۳، ۵ و ۷ پایانِ خط دارند، اما حلقه یا تقاطع نه؛ ارقامِ ۶ و ۹ حلقه و پایانِ خط دارند. افزون بر این، رابطه‌های فضاییِ بینِ الگوهای محلی اطلاعاتی‌اند. ۱ دو پایانِ خطِ بالایِ هم دارد؛ ۶ پایانِ خطی بالایِ

حلقه دارد. این مشاهده‌ها راهبردی را پیشنهاد می‌کنند که اصلِ مرکزیِ بینواییِ رایانهٔ مدرن است: ویژگی‌هایی می‌سازید که به الگوهای در همسایگی‌های کوچکِ محلی پاسخ می‌دهند؛ سپس ویژگی‌های دیگر به الگوهایِ آن ویژگی‌ها نگاه می‌کنند؛ سپس دیگران به الگوهایِ آن‌ها، و همین‌طور.

این همان چیزی است که شبکه‌های عصبیِ پیچشی خوب انجام می‌دهند. باید لایه‌ای—پیچشی که تابعِ فعال‌سازیِ ReLU دنبالش می‌کند—را به‌عنوانِ «آشکارسازِ الگویِ محلی» در نظر بگیرید (شکل ۲۵.۱۲). پیچش اندازه می‌گیرد هر پنجرهٔ محلیِ تصویر چقدر شبیهِ الگویِ هسته است؛ ReLU پنجره‌های کم‌امتیاز را صفر و پنجره‌های پرامتیاز را برجسته می‌کند. پس پیچش با هسته‌های متعدد الگوهای متعددی را می‌یابد؛ افزون بر این، الگوهای ترکیبی را می‌توان با اعمالِ لایهٔ دیگری بر خروجیِ لایهٔ نخست تشخیص داد.

به خروجیِ نخستین لایهٔ پیچشی فکر کنید. هر مکان ورودی‌هایی از پیکسل‌های پنجره‌ای دربارهٔ آن مکان دریافت می‌کند. خروجیِ ReLU، چنان‌که دیده‌ایم، آشکارسازِ الگویِ ساده‌ای تشکیل می‌دهد. حالا اگر لایهٔ دومی رویِ این بگذاریم، هر مکان در لایهٔ دوم ورودی‌هایی از مقادیرِ لایهٔ نخست در پنجره‌ای دربارهٔ آن مکان دریافت می‌کند. این بدان‌معناست مکان‌های لایهٔ دوم توسط پنجرهٔ بزرگ‌تری از پیکسل‌ها از مکان‌های لایهٔ نخست اثر می‌پذیرند. باید آن‌ها را به‌عنوانِ بازنمایی‌کنندهٔ «الگوهایِ الگوها» در نظر بگیرید. اگر لایهٔ سومی رویِ لایهٔ دوم بگذاریم، مکان‌های آن لایهٔ سوم به پنجرهٔ حتی بزرگ‌تری از پیکسل‌ها بستگی خواهند داشت؛ لایهٔ چهارم به پنجره‌ای بزرگ‌ترِ هنوز، و همین‌طور. شبکه الگوها را در سطوحِ متعدد ایجاد می‌کند، و این کار را با یادگیری از داده انجام می‌دهد، نه با داشتنِ الگوهایی که برنامه‌نویسی به آن داده.



شکل ۲۵.۱۲ – در دورترینِ چپ، چند تصویر از مجموعهٔ دادهٔ MNIST. سه هسته در چپِ میانه ظاهر می‌شوند. آن‌ها در اندازهٔ واقعی‌شان (بلوک‌های ریز) و بزرگ‌شده برای آشکارکردنِ محتوایشان نشان داده شده‌اند: خاکستریِ میانی صفر، روشن مثبت و تیره منفی است. راستِ میانه نتایج اعمالِ این هسته‌ها بر تصاویر را نشان می‌دهد. راست پیکسل‌هایی را نشان می‌دهد که پاسخ در آن‌ها از آستانه بزرگ‌تر (سبز) یا کوچک‌تر (قرمز) است. باید توجه کنید این (از بالا به پایین) می‌دهد: آشکارسازِ میلهٔ افقی؛ آشکارسازِ میلهٔ عمودی؛ و (که سخت‌تر است متوجه شد) آشکارسازِ پایانِ خط. این آشکارسازها به کنتراستِ میله توجه می‌کنند، پس (برای نمونه) میلهٔ افقی که بالایش روشن و پایینش تیره است پاسخِ مثبت (سبز) می‌گیرد، و میله‌ای که بالایش تیره و پایینش روشن است پاسخِ منفی (قرمز). این آشکارسازها نسبتاً مؤثرند، اما کامل نیستند.

در حالی که آموزشِ CNN «خارج از جعبه» گاهی کار می‌کند، دانستنِ چند فنِ عملی کمک می‌کند. یکی از مهم‌ترین‌ها «افزایشِ داده» (Data Set Augmentation) است، که در آن مثال‌های آموزشی کپی و کمی اصلاح می‌شوند. برای نمونه، ممکن است تصویری را به‌صورتِ تصادفی کمی جابه‌جا، بچرخانید یا بکشید، یا رنگِ پیکسل‌ها را به‌صورتِ تصادفی کمی جابه‌جا کنید. معرفیِ این تغییرِ شبیه‌سازی‌شدهٔ دیدگاه یا نورپردازی به مجموعهٔ داده کمک می‌کند اندازهٔ مجموعهٔ داده افزایش یابد، هرچند البته مثال‌های تازه با نمونه‌های اصلی همبستگیِ بالایی دارند. همچنین امکانِ استفاده از افزایش در زمانِ آزمون به‌جای زمانِ آموزش هست. در این رویکرد، تصویر چند بار تکرار و اصلاح می‌شود (مثلاً با برشِ تصادفی) و رده‌بند رویِ هر یک از تصاویرِ اصلاح‌شده اجرا می‌شود. سپس خروجی‌های رده‌بند از هر کپی برای رأی‌دادن برای تصمیمِ نهایی رویِ ردهٔ کلی استفاده می‌شوند.

وقتی تصاویرِ صحنه را رده‌بندی می‌کنید، هر پیکسل می‌تواند کمک‌کننده باشد. اما وقتی تصاویرِ اشیا را رده‌بندی می‌کنید، برخی پیکسل‌ها بخشی از شیء نیستند و بنابراین ممکن است حواس‌پرت‌کن باشند. برای نمونه، اگر گربه‌ای رویِ تختِ سگ خوابیده باشد، می‌خواهیم رده‌بند رویِ پیکسل‌های گربه تمرکز کند، نه تخت. رده‌بندهای تصویرِ مدرن این را خوب مدیریت می‌کنند و تصویری را به‌عنوانِ «گربه» به‌دقت رده‌بندی می‌کنند، حتی اگر پیکسل‌های کمی واقعاً رویِ گربه باشند. دو دلیل برای این هست. نخست، رده‌بندهای مبتنی بر CNN در نادیده‌گرفتنِ الگوهایی که تمایزگر نیستند خوب‌اند. دوم، الگوهایی که خارج از شیء قرار دارند ممکن است تمایزگر باشند (مثلاً، اسباب‌بازیِ گربه، قلاده‌ای

با زنگولهٔ کوچک، یا ظرفِ غذایِ گربه ممکن است در واقع کمک کنند بگوییم به گربه نگاه می‌کنیم). این اثر «زمینه» (Context) نامیده می‌شود. زمینه می‌تواند کمک کند یا آسیب بزند، بسته به اینکه کاملاً به مجموعهٔ داده و کاربردِ خاص بستگی دارد.

۲۵.۵ تشخیصِ اشیا

رده‌بندهای تصویر پیش‌بینی می‌کنند چه چیزی در تصویر است—کلِ تصویر را به‌عنوانِ تعلق‌داشتن به رده‌ای رده‌بندی می‌کنند. آشکارسازهای شیء اشیای متعدد را در تصویر می‌یابند، گزارش می‌کنند هر شیء چه رده‌ای است، و همچنین گزارش می‌کنند هر شیء کجاست، با دادنِ «جعبهٔ محدودکننده» (Bounding Box) دورِ هر شیء. مجموعهٔ رده‌ها از پیش ثابت است. پس ممکن است بکوشیم همهٔ چهره‌ها، همهٔ خودروها یا همهٔ گربه‌ها را تشخیص دهیم.

می‌توانیم آشکارسازِ شیء را با نگاه‌کردن به «پنجرهٔ لغزان» (Sliding Window) کوچکی رویِ تصویرِ بزرگ‌تر—مستطیلی—بسازیم. در هر نقطه، آنچه در پنجره می‌بینیم را با رده‌بندِ CNN رده‌بندی می‌کنیم. سپس رده‌بندی‌های پرامتیاز را می‌گیریم—گربه‌ای این‌جا و سگی آن‌جا—و پنجره‌های دیگر را نادیده می‌گیریم. پس از کمی کار برای حلِ تعارض‌ها، مجموعهٔ نهاییِ اشیا را با مکان‌هایشان داریم. هنوز چند جزئیات برای حل هست:

- **رویِ شکلِ پنجره تصمیم بگیر:** به‌مراتب ساده‌ترین انتخاب استفاده از مستطیل‌های هم‌راستا با محور است. (جایگزین—نوعی ماسک که شیء را از تصویر می‌بُرد—به‌ندرت استفاده می‌شود، زیرا بازنمایی و محاسبه‌کردنش سخت است.) همچنان باید پهنا و بلندیِ مستطیل‌ها را انتخاب کنیم.
- **رده‌بندی برای پنجره‌ها بساز:** از قبل می‌دانیم چگونه این را با CNN انجام دهیم.
- **تصمیم بگیر به کدام پنجره‌ها نگاه کنیم:** از میانِ همهٔ پنجره‌های ممکن، می‌خواهیم آن‌هایی را انتخاب کنیم که احتمال دارد اشیای جالبی در آن‌ها باشد.
- **انتخاب کن کدام پنجره‌ها گزارش شوند:** پنجره‌ها هم‌پوشانی خواهند داشت، و نمی‌خواهیم همان شیء را چند بار در پنجره‌های کمی متفاوت گزارش کنیم. برخی اشیا ارزشِ ذکر ندارند؛ به تعدادِ صندلی‌ها و افراد در تصویری از سالنِ سخنرانیِ بزرگِ مملو از جمعیت فکر کنید. آیا همه باید به‌عنوانِ اشیای منفرد گزارش شوند؟ شاید فقط اشیایی که در تصویر بزرگ ظاهر می‌شوند—ردیفِ جلو—باید گزارش شوند. انتخاب به کاربردِ موردنظرِ آشکارسازِ شیء بستگی دارد.
- **مکان‌های دقیقِ اشیا را با این پنجره‌ها گزارش کن:** وقتی می‌دانیم شیء جایی در پنجره است، می‌توانیم محاسبهٔ بیشتری برای فهمیدنِ مکانِ دقیق‌تری درونِ پنجره انجام دهیم.

بیایید با دقتِ بیشتری به مسئلهٔ تصمیم‌گیری دربارهٔ اینکه به کدام پنجره‌ها نگاه کنیم بپردازیم. جستجویِ همهٔ پنجره‌های ممکن کارآمد نیست—در تصویرِ $n \times n$ پیکسلی $O(n^4)$ پنجرهٔ مستطیلیِ ممکن هست. اما می‌دانیم پنجره‌هایی که اشیا را شامل‌اند تمایل به رنگ و بافتِ کاملاً منسجمی دارند. از سوی دیگر، پنجره‌هایی که شیء را نصف می‌کنند ناحیه‌ها یا لبه‌هایی دارند که از کنارِ پنجره می‌گذرند. پس منطقی است سازوکاری داشته باشیم که «شیءبودگی»

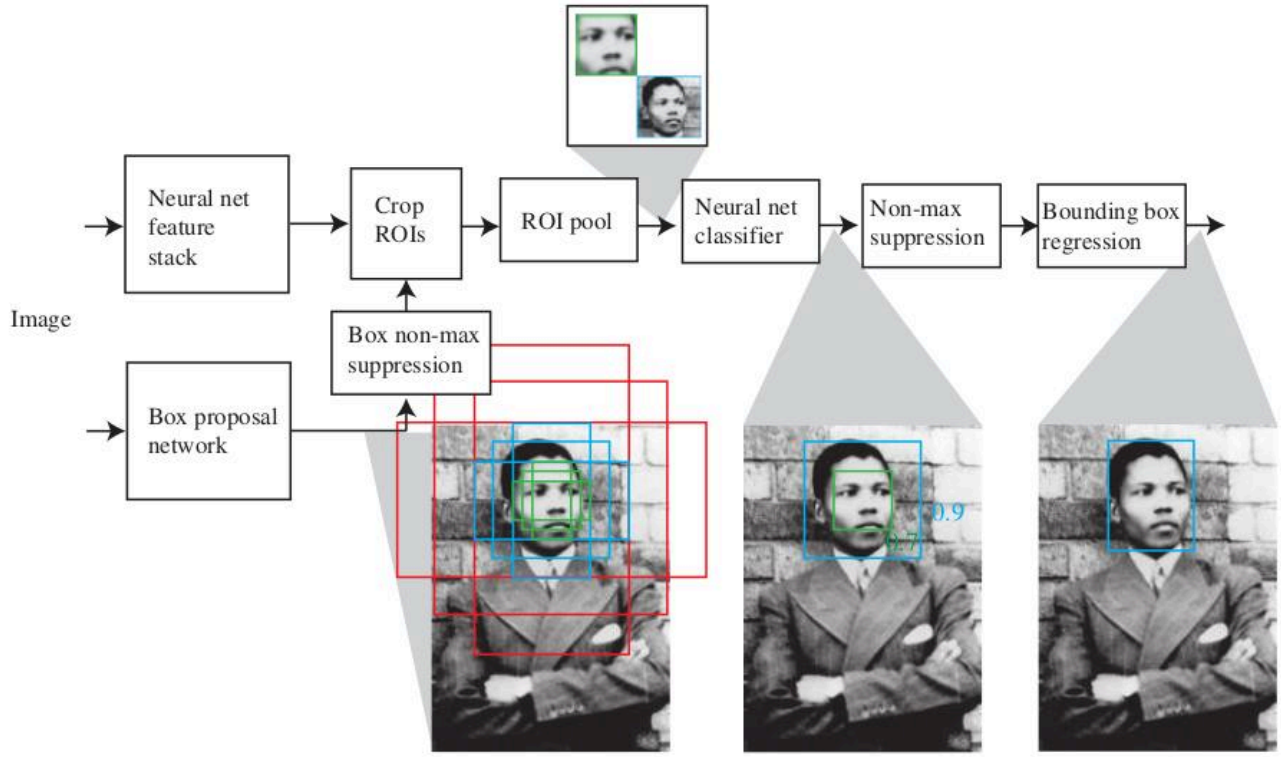
(Objectness) را امتیاز می‌دهد—آیا جعبه‌ای شیئی در آن دارد، مستقل از اینکه آن شیء چیست.[1] می‌توانیم جعبه‌هایی را که به نظر می‌رسد شیئی در آن‌هاست بیابیم، و سپس شیء را فقط برای همان جعبه‌هایی که آزمونِ شیءبودگی را پاس می‌کنند رده‌بندی کنیم.

شبکه‌ای که ناحیه‌های با شیء را می‌یابد «شبکهٔ پیشنهادِ ناحیه» (Regional Proposal Network یا RPN) نامیده می‌شود. آشکارسازِ شیءِ شناخته‌شدهٔ Faster RCNN مجموعهٔ بزرگی از جعبه‌های محدودکننده را به‌عنوانِ نقشه‌ای با اندازهٔ ثابت رمز می‌کند. سپس شبکه‌ای می‌سازد که می‌تواند برای هر جعبه امتیازی پیش‌بینی کند، و این شبکه را چنان آموزش می‌دهد که امتیاز وقتی جعبه شیئی دارد بزرگ و در غیرِ این صورت کوچک باشد. رمزکردنِ جعبه‌ها به‌عنوانِ نقشه سرراست است. جعبه‌های به‌مرکزیتِ نقطه‌های سراسرِ تصویر را در نظر می‌گیریم؛ نیازی به درنظرگرفتنِ هر نقطهٔ ممکن نیست (زیرا حرکتِ یک پیکسلی احتمالاً رده‌بندی را تغییر نمی‌دهد)؛ انتخابِ خوب گامی (آفستِ بینِ نقطه‌های مرکزی) ۱۶ پیکسل است. برای هر نقطهٔ مرکزی چند جعبهٔ ممکن، به نامِ «جعبهٔ لنگر» (Anchor Box)، در نظر می‌گیریم. Faster RCNN از نه جعبه استفاده می‌کند: اندازه‌های کوچک، متوسط و بزرگ؛ و نسبت‌های ابعادِ بلند، پهن و مربع. از نظرِ معماریِ شبکهٔ عصبی، بلوکی سه‌بعدی بساز که هر مکانِ فضاییِ بلوک دو بُعد برای نقطهٔ مرکزی و یک بُعد برای نوعِ جعبه داشته باشد. حالا هر جعبه‌ای با امتیازِ شیءبودگیِ به‌اندازهٔ کافی خوب «ناحیهٔ موردِ علاقه» (Region of Interest یا ROI) نامیده می‌شود و باید توسط رده‌بندی بررسی شود. اما رده‌بندهای CNN تصاویرِ با اندازهٔ ثابت را ترجیح می‌دهند، و جعبه‌هایی که آزمونِ شیءبودگی را پاس می‌کنند در اندازه و شکل متفاوت‌اند. نمی‌توانیم جعبه‌ها را وادار کنیم تعدادِ پیکسلِ یکسان داشته باشند، اما می‌توانیم آن‌ها را وادار کنیم تعدادِ ویژگیِ یکسان داشته باشند، با نمونه‌برداری از پیکسل‌ها برای استخراجِ ویژگی، فرایندی که «تجمیعِ (ROI Pooling) «ROI نامیده می‌شود. این نقشهٔ ویژگیِ با اندازهٔ ثابت سپس به رده‌بند داده می‌شود.

حالا برای مسئلهٔ تصمیم‌گیری دربارهٔ اینکه کدام پنجره‌ها گزارش شوند. فرض کنید به پنجره‌های با اندازهٔ 32×32 با گامِ ۱ نگاه می‌کنیم: هر پنجره فقط یک پیکسل از پنجرهٔ قبلی آفست دارد. پنجره‌های مشابهِ بسیاری خواهد بود که باید امتیازهای مشابهی داشته باشند. اگر همهٔ آن‌ها امتیازی بالای آستانه داشته باشند، نمی‌خواهیم همه را گزارش کنیم، زیرا به احتمالِ بسیار به نماهای کمی متفاوتِ همان شیء اشاره می‌کنند. از سوی دیگر، اگر گام بیش‌ازحد بزرگ باشد، ممکن است شیئی درونِ هیچ پنجره‌ای نباشد و از دست برود. در عوض، می‌توانیم از الگوریتمِ حریصانه‌ای به نامِ «سرکوبِ غیربیشینه» (Non-Maximum Suppression) استفاده کنیم. نخست، فهرستِ مرتب‌شده‌ای از همهٔ پنجره‌های با امتیازِ بالایِ آستانه بساز. سپس، تا وقتی پنجره‌هایی در فهرست هست، پنجرهٔ با بیشترین امتیاز را انتخاب و آن را به‌عنوانِ شاملِ شیء بپذیر؛ همهٔ پنجره‌های عمدتاً هم‌پوشانِ دیگر را از فهرست دور بریز.

سرانجام، مسئلهٔ گزارشِ مکانِ دقیقِ اشیا را داریم. فرض کنید پنجره‌ای با امتیازِ بالا داریم که از سرکوبِ غیربیشینه عبور کرده. بعید است این پنجره دقیقاً در جایِ درست باشد (به یاد داشته باشید، به تعدادِ نسبتاً کمی پنجره با تعدادِ کمی اندازهٔ ممکن نگاه کردیم). از بازنماییِ ویژگیِ محاسبه‌شده توسط رده‌بند برای پیش‌بینیِ بهبودهایی استفاده می‌کنیم که پنجره را به جعبهٔ محدودکنندهٔ درست می‌تراشند، گامی که «رگرسیونِ جعبهٔ محدودکننده» (Bounding Box Regression) نامیده می‌شود.

ارزیابیِ آشکارسازهای شیء نیاز به مراقبت دارد. نخست به مجموعهٔ آزمون نیاز داریم: مجموعه‌ای از تصاویر با هر شیءِ تصویر که با برچسبِ ردهٔ حقیقتِ زمینی و جعبهٔ محدودکننده علامت خورده. معمولاً، جعبه‌ها و برچسب‌ها توسط انسان‌ها تأمین می‌شوند. سپس هر تصویر را به آشکارسازِ شیء می‌دهیم و خروجی‌اش را با حقیقتِ زمینی مقایسه می‌کنیم. باید مایل باشیم جعبه‌هایی را بپذیریم که چند پیکسل آفست‌اند، زیرا جعبه‌های حقیقتِ زمینی کامل نخواهند بود. امتیازِ ارزیابی باید «بازیابی» (یافتنِ همهٔ اشیایی که هستند) و «دقت» (نیافتنِ اشیایی که نیستند) را متوازن کند.



شکل ۲۵.۱۳ — Faster RCNN از دو شبکه استفاده می‌کند. تصویری از نلسونِ ماندلای جوان به آشکارسازِ شیء داده می‌شود. یک شبکه امتیازهای «شیءبودگی» جعبه‌های تصویرِ نامزد، به نام «جعبه‌های لنگر»، به‌مرکزیتِ نقطه‌ای شبکه‌ای، را محاسبه می‌کند. در هر نقطهٔ شبکه‌ای نه جعبهٔ لنگر هست (سه مقیاس، سه نسبتِ ابعاد). برای تصویرِ مثال، یک جعبهٔ سبزِ درونی و یک جعبهٔ آبیِ بیرونی آزمونِ شیءبودگی را پاس کرده‌اند. شبکهٔ دوم پشتهٔ ویژگی‌ای است که بازنماییِ تصویرِ مناسب برای رده‌بندی را محاسبه می‌کند. جعبه‌های با بیشترین امتیازِ شیءبودگی از نقشهٔ ویژگی بریده، با تجمیعِ ROI در اندازه استاندارد می‌شوند، و به رده‌بند داده می‌شوند. جعبهٔ آبی امتیازی بالاتر از جعبهٔ سبز دارد و با آن هم‌پوشانی دارد، پس جعبهٔ سبز توسط سرکوبِ غیربیشینه رد می‌شود. سرانجام، رگرسیونِ جعبهٔ محدودکننده جعبهٔ آبی را طوری تنظیم می‌کند که چهره را بپوشاند. این بدان‌معناست نمونه‌برداریِ نسبتاً درشتِ مکان‌ها، مقیاس‌ها و نسبت‌های ابعاد دقت را تضعیف نمی‌کند. عکس از Sipa/Shutterstock.

۲۵.۶ جهانِ سه‌بعدی

تصاویر تصویری دوبعدی از جهانِ سه‌بعدی نشان می‌دهند. اما این تصویرِ دوبعدی غنی از سرنخ‌هایی دربارهٔ جهانِ سه‌بعدی است. یک نوعِ سرنخ وقتی رخ می‌دهد که تصاویرِ متعددی از همان جهان داریم و می‌توانیم نقطه‌ها را بینِ تصاویر تطبیق دهیم. نوعِ دیگرِ سرنخ درونِ یک تصویرِ تکی در دسترس است.

۲۵.۶.۱ سرنخ‌های سه‌بعدی از نماهای متعدد

دو تصویر از اشیای جهانِ سه‌بعدی به چند دلیل از یکی بهترند:

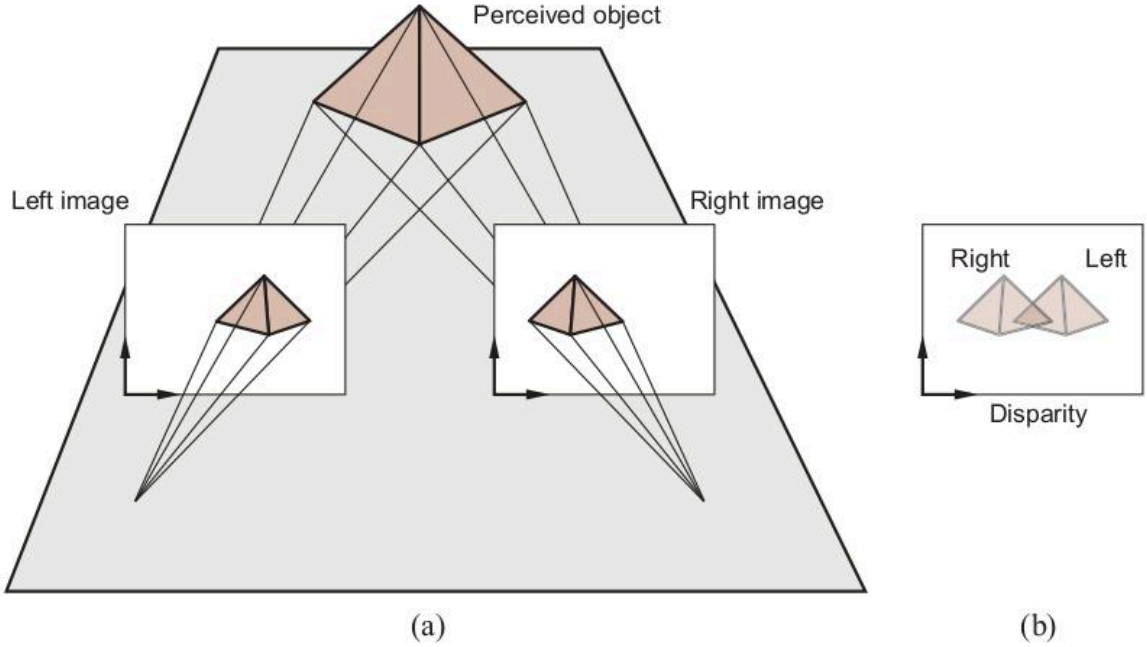
- اگر دو تصویر از همان صحنه‌ای که از دیدگاه‌های متفاوت گرفته شده داشته باشید و به‌اندازهٔ کافی دربارهٔ دو دوربین بدانید، می‌توانید مدلِ سه‌بعدی—مجموعه‌ای از نقطه‌ها با مختصات‌شان در ۳ بُعد—بسازید، با فهمیدنِ اینکه کدام نقطهٔ نمایِ نخست با کدام نقطهٔ نمایِ دوم متناظر است و اعمالِ هندسه‌ای. این برای تقریباً همهٔ جفت‌های جهتِ دید و تقریباً همهٔ انواعِ دوربین صادق است.
- اگر دو نمایِ نقطه‌های کافی داشته باشید و بدانید کدام نقطهٔ نمایِ نخست با کدام نقطهٔ نمایِ دوم متناظر است، برای ساختِ مدلِ سه‌بعدی نیازی به دانستنِ چیزِ زیادی دربارهٔ دوربین‌ها نیست. دو نمایِ دو نقطه چهار مختصاتِ x,y به شما می‌دهد، و فقط به سه مختصات برای مشخص‌کردنِ نقطه‌ای در فضایِ سه‌بعدی نیاز دارید؛ مختصاتِ اضافی برای فهمیدنِ آنچه باید دربارهٔ دوربین‌ها بدانید به کمک می‌آید. این برای تقریباً همهٔ جفت‌های جهتِ دید و تقریباً همهٔ انواعِ دوربین صادق است.

مسئلهٔ کلیدی برقرارکردنِ این است که کدام نقطهٔ نمایِ نخست با کدام نقطهٔ نمایِ دوم متناظر است. توصیف‌های دقیقِ ظاهرِ محلیِ نقطه‌ای با استفاده از ویژگی‌های بافتِ ساده (مانندِ آن‌های بخشِ ۲۵.۳.۲) اغلب برای تطبیقِ نقطه‌ها کافی‌اند. برای نمونه، در صحنه‌ای از ترافیکِ خیابان، ممکن است فقط یک چراغِ سبز در دو تصویرِ گرفته‌شده از صحنه دیده شود؛ سپس می‌توانیم فرضیه دهیم این‌ها با یکدیگر متناظرند. هندسهٔ نماهای دوربینِ متعدد بسیار خوب فهمیده شده (اما متأسفانه برای شرح‌دادنِ این‌جا بیش‌ازحد پیچیده است). این نظریه قیدهای هندسی‌ای تولید می‌کند که کدام نقطهٔ یک تصویر می‌تواند با کدام نقطهٔ تصویرِ دیگر تطبیق شود. قیدهای دیگر را می‌توان با استدلال دربارهٔ همواریِ سطوحِ بازسازی‌شده به‌دست آورد.

دو راه برای گرفتنِ نماهای متعدد از یک صحنه هست. یکی داشتنِ دو دوربین یا دو چشم است (بخش ۲۵.۶.۲). دیگری حرکت‌کردن است (بخش ۲۵.۶.۳). اگر بیش از دو نما داشته باشید، می‌توانید هم هندسهٔ جهان و هم جزئیاتِ نما را بسیار دقیق بازیابی کنید. بخش ۲۵.۷.۳ برخی کاربردهای این فناوری را بحث می‌کند.

۲۵.۶.۲ استریوپسیسِ دوچشمی

بیشترِ مهره‌داران دو چشم دارند. این برای افزونگی در صورتِ ازدست‌رفتنِ چشمی مفید است، اما به‌روش‌های دیگر نیز کمک می‌کند. بیشترِ شکارها چشم‌هایی در کنارِ سر دارند تا میدانِ دیدِ وسیع‌تری ممکن شود. شکارچیان چشم‌هایی در جلو دارند که آن‌ها را قادر به استفاده از «استریوپسیسِ دوچشمی» (Binocular Stereopsis) می‌کند. هر دو انگشتِ اشاره را جلوی صورت‌تان بگیرید، با یک چشمِ بسته، و آن‌ها را طوری تنظیم کنید که انگشتِ جلویی انگشتِ دیگر را در دیدِ چشمِ باز مسدود کند. حالا چشم‌ها را عوض کنید؛ باید متوجه شوید انگشتان نسبت به یکدیگر جابه‌جا شده‌اند. این جابه‌جاییِ موقعیت از نمایِ چپ به نمایِ راست «اختلافِ منظر» (Disparity) نامیده می‌شود. در انتخابِ درستِ سامانهٔ مختصات، اگر تصاویرِ چپ و راستِ شیئی را در عمقی رویِ هم قرار دهیم، شیء در تصویرِ روی‌هم‌نهاده به‌صورتِ افقی جابه‌جا می‌شود، و اندازهٔ جابه‌جایی معکوسِ عمق است. این را می‌توانید در شکل ۲۵.۱۴ ببینید، که نزدیک‌ترین نقطهٔ هرم در تصویرِ راست به چپ و در تصویرِ چپ به راست جابه‌جا شده.



شکل ۲۵.۱۴ – انتقالِ دوربین به‌موازاتِ صفحهٔ تصویر باعث می‌شود ویژگی‌های تصویر در صفحهٔ دوربین حرکت کنند. اختلافِ منظرِ موقعیت‌هایی که نتیجه می‌شود سرنخی برای عمق است. اگر تصاویرِ چپ و راست را رویِ هم قرار دهیم، مانندِ (ب)، اختلافِ منظر را می‌بینیم.

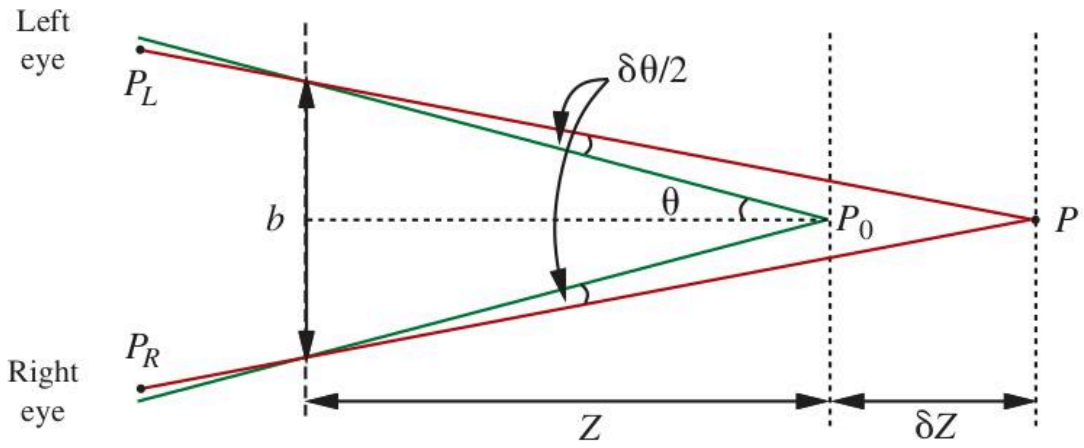
برای اندازه‌گیریِ اختلافِ منظر باید مسئلهٔ تطبیق را حل کنیم: کدام نقطهٔ تصویرِ راست با کدام نقطهٔ تصویرِ چپ متناظر است؟ برای دوربینِ ترجمه‌شده به‌موازاتِ محورِ x، تطبیق از دوربینِ راست به دوربینِ چپ فقط جابه‌جایی در طولِ محورِ x به مقدارِ b، «خطِ مبنا» (Baseline) است. می‌توانیم از معادله‌های جریانِ نوریِ بخشِ ۲۵.۳.۳ استفاده کنیم، اگر این را ناشی از بردارِ ترجمهٔ T که برای زمانِ δt عمل می‌کند بدانیم، با $T_x = b/\delta t$ و $T_y = T_z = 0$. اختلافِ منظرِ افقی و عمودی توسط مؤلفه‌های جریانِ نوری، ضرب‌شده در گامِ زمانیِ δt، داده می‌شوند: $H = v_x\,\delta t$، $V = v_y\,\delta t$.

با انجامِ جایگزینی‌ها، نتیجه را می‌گیریم که $H = b/Z$، $V = 0$. به بیانِ دیگر، اختلافِ منظرِ افقی برابرِ نسبتِ خطِ مبنا به عمق است، و اختلافِ منظرِ عمودی صفر است. می‌توانیم عمقِ Z را با توجه به دانستنِ b و قابلیتِ اندازه‌گیریِ H بازیابی کنیم.

تحتِ شرایطِ دیدِ معمولی، انسان‌ها «خیره» می‌شوند؛ یعنی نقطه‌ای در صحنه هست که محورهای نوریِ دو چشم در آن تلاقی می‌کنند. شکل ۲۵.۱۵ دو چشمِ خیره‌شده به نقطهٔ P_0 را نشان می‌دهد، که در فاصلهٔ Z از نقطهٔ میانیِ چشم‌هاست. برای راحتی، اختلافِ منظرِ زاویه‌ای، اندازه‌گیری‌شده بر حسبِ رادیان، را محاسبه می‌کنیم. اختلافِ منظر در نقطهٔ خیرگیِ P_0 صفر است. برای نقطهٔ دیگری مانندِ P در صحنه که δZ دورتر است، می‌توانیم جابه‌جایی‌های زاویه‌ایِ تصاویرِ چپ و راستِ P را محاسبه کنیم، که آن‌ها را P_L و P_R می‌نامیم. اگر هر یک از این‌ها با زاویهٔ δθ/2 نسبت به P_0 جابه‌جا شده باشد، آنگاه جابه‌جایی بینِ P_L و P_R، که اختلافِ منظرِ P است، فقط δθ است. از شکل ۲۵.۱۵، $\tan\theta = (b/2)/Z$ و $\tan(\theta - \delta\theta/2) = (b/2)/(Z+\delta Z)$، اما برای زاویه‌های کوچک، $\tan\theta \approx \theta$، پس $\delta\theta/2 = b/2Z - b/2(Z+\delta Z) \approx b\delta Z/2Z^2$ و، چون اختلافِ منظرِ واقعی δθ است، داریم:

```
Disparity = b δZ / Z²
```

در انسان‌ها، خطِ مبنای b حدودِ ۶ سانتی‌متر است. فرض کنید Z حدودِ ۱۰۰ سانتی‌متر است و کوچک‌ترینِ δθ قابلِ تشخیص (متناظر با اندازهٔ پیکسلِ تکی) حدودِ ۵ ثانیهٔ کمان است، که δZ برابرِ 0.4 میلی‌متر می‌دهد. برای Z = 30 سانتی‌متر، مقدارِ به‌شکلِ چشمگیری کوچکِ δZ = 0.036 میلی‌متر می‌گیریم. یعنی، در فاصلهٔ ۳۰ سانتی‌متری، انسان‌ها می‌توانند عمق‌هایی را که به‌اندازهٔ 0.036 میلی‌متر تفاوت دارند تشخیص دهند، که ما را قادر به نخ‌کردنِ سوزن و مانندِ آن می‌کند.



شکل ۲۵.۱۵ – رابطهٔ بینِ اختلافِ منظر و عمق در استریوپسیس. مراکزِ فرافکنیِ دو چشم به فاصلهٔ b از هم‌اند، و محورهای نوری در نقطهٔ خیرگیِ P_0 تلاقی می‌کنند. نقطهٔ P صحنه به نقطه‌های P_L و P_R در دو چشم فرافکنی می‌شود. بر حسبِ زاویه، اختلافِ منظرِ بینِ این‌ها δθ است (نمودار دو زاویهٔ δθ/2 نشان می‌دهد).

۲۵.۶.۳ سرنخ‌های سه‌بعدی از دوربینِ متحرک

فرض کنید دوربینی در صحنه حرکت می‌کند. شکل ۲۵.۱۴ را بگیر و تصویرِ چپ را «زمانِ t» و تصویرِ راست را «زمانِ t+1» برچسب بزن. هندسه تغییر نکرده، پس همهٔ مطالبِ بحثِ استریوپسیس وقتی دوربین حرکت می‌کند نیز برقرار است. آنچه در آن بخش اختلافِ منظر نامیدیم اکنون حرکتِ ظاهری در تصویر در نظر گرفته می‌شود و «جریانِ نوری» نامیده می‌شود. این منبعِ اطلاعاتی هم برای حرکتِ دوربین و هم برای هندسهٔ صحنه است.

برای فهمِ این، معادله‌ای را بیان می‌کنیم (بدونِ اثبات) که جریانِ نوری را به سرعتِ انتقالیِ بینندهٔ T و عمقِ صحنه مرتبط می‌کند. میدانِ جریانِ نوری میدانِ برداریِ سرعت‌ها در تصویر، $(v_x(x,y), v_y(x,y))$، است. عبارت‌های این مؤلفه‌ها، در چارچوبِ مختصاتی به‌مرکزیتِ دوربین و با فرضِ فاصلهٔ کانونیِ f = 1، به‌صورتِ زیرند:

$$v_x(x, y) = (-T_x + x\ T_z) / Z(x, y)$$
$$v_y(x, y) = (-T_y + y\ T_z) / Z(x, y)$$

که Z(x,y) مختصاتِ z (یعنی عمقِ) نقطهٔ صحنهٔ متناظر با نقطهٔ تصویرِ (x,y) است. توجه کنید هر دو مؤلفهٔ جریانِ نوری، $v_x(x,y)$ و $v_y(x,y)$، در نقطهٔ $x = T_x/T_z$، $y = T_y/T_z$ صفرند. این نقطه «کانونِ گسترش» (Focus of Expansion) میدانِ جریان نامیده می‌شود. فرض کنید مبدأ را در صفحهٔ x–y به کانونِ گسترش منتقل کنیم؛ آنگاه

عبارت‌های جریانِ نوری شکلِ به‌ویژه ساده‌ای به خود می‌گیرند. ('x',y) مختصاتِ تازه باشد که با x' = x - T_x/T_z، y' = y - T_y/T_z تعریف می‌شود. آنگاه:

```
vₓ(x′, y′)  = x′ T_z / Z(x′, y′)
 v_y(x′, y′) = y′ T_z / Z(x′, y′)
```

توجه کنید ابهامِ عاملِ مقیاس در این‌جا هست (که دلیلِ بی‌آزاربودنِ فرضِ فاصلهٔ کانونیِ f = 1 همین است). اگر دوربین دو برابر سریع‌تر حرکت می‌کرد، و هر شیءِ صحنه دو برابر بزرگ‌تر و در فاصلهٔ دو برابرِ دوربین بود، میدانِ جریانِ نوری دقیقاً یکسان می‌بود. اما همچنان می‌توانیم اطلاعاتِ کاملاً مفیدی استخراج کنیم.

1. فرض کنید مگسی هستید که می‌خواهد رویِ دیواری فرود آید و می‌خواهید از میدانِ جریانِ نوری اطلاعاتِ مفید بگیرید. میدانِ جریانِ نوری نمی‌تواند فاصله تا دیوار یا سرعت به سمتِ دیوار را به شما بگوید، به‌دلیلِ ابهامِ مقیاس. اما اگر فاصله را بر سرعت تقسیم کنید، ابهامِ مقیاس حذف می‌شود. نتیجه «زمانِ تا برخورد» است، که با Z/T_z داده می‌شود، و برای کنترلِ رویکردِ فرود واقعاً مفید است. شواهدِ تجربیِ قابلِ توجهی هست که بسیاری از گونه‌های جانوری از این سرنخ بهره می‌برند.

1. دو نقطه در عمق‌های Z_1 و Z_2 را در نظر بگیر. ممکن است مقدارِ مطلقِ هیچ‌کدام را ندانیم، اما با درنظرگرفتنِ معکوسِ نسبتِ اندازه‌های جریانِ نوری در این نقطه‌ها، می‌توانیم نسبتِ عمقِ Z_1/Z_2 را تعیین کنیم. این سرنخِ «اختلافِ منظرِ حرکتی» (Motion Parallax) است، که وقتی از پنجرهٔ کناریِ خودرو یا قطارِ متحرک به بیرون نگاه می‌کنیم و استنتاج می‌کنیم بخش‌های کُندحرکتِ منظره دورترند از آن استفاده می‌کنیم.

۲۵.۶.۴ سرنخ‌های سه‌بعدی از یک نما

حتی یک تصویرِ تکی مجموعهٔ غنی‌ای از اطلاعات دربارهٔ جهانِ سه‌بعدی فراهم می‌کند. این حتی اگر تصویر فقط طراحی خطی باشد نیز صادق است. طراحی‌های خطی دانشمندانِ بینایی را مجذوب کرده‌اند، زیرا مردم حسِ شکل و چیدمانِ سه‌بعدی دارند، حتی اگر به نظر می‌رسد طراحی اطلاعاتِ بسیار کمی برای انتخاب از مجموعهٔ عظیمِ صحنه‌هایی که می‌توانستند همان طراحی را تولید کنند دارد. انسداد یکی از منابعِ کلیدیِ اطلاعات است: اگر شواهدی در تصویر هست که شیئی شیءِ دیگر را مسدود می‌کند، آنگاه شیءِ مسدودکننده به چشم نزدیک‌تر است.

در تصاویرِ صحنه‌های واقعی، بافت سرنخِ قویِ ساختارِ سه‌بعدی است. بخش ۲۵.۳.۲ بیان کرد بافت الگویِ تکراریِ تکسِل‌هاست. هرچند توزیعِ تکسِل‌ها ممکن است رویِ اشیای صحنه یکنواخت باشد—برای نمونه، سنگریزه‌های ساحل —ممکن است در تصویر یکنواخت نباشد—سنگریزه‌های دورتر کوچک‌تر از سنگریزه‌های نزدیک‌تر ظاهر می‌شوند. به‌عنوانِ مثالِ دیگر، به تکه‌ای پارچهٔ خال‌خالی فکر کنید. همهٔ خال‌ها رویِ پارچه همان اندازه و شکل‌اند، اما در نمایِ پرسپکتیوی برخی خال‌ها به‌دلیلِ کوتاه‌نمایی بیضی‌اند. روش‌های مدرن با یادگیریِ نگاشت از تصاویر به ساختارِ سه‌بعدی (بخش ۲۵.۷.۴) از این سرنخ‌ها بهره می‌برند، به‌جای استدلالِ مستقیم دربارهٔ ریاضیاتِ زیرینِ بافت.

سایه‌روشن—تغییرِ شدتِ نورِ دریافتی از بخش‌های متفاوتِ سطحِ صحنه—توسط هندسهٔ صحنه و توسط ویژگی‌های بازتابِ سطوح تعیین می‌شود. شواهدِ بسیار خوبی هست که سایه‌روشن سرنخی برای شکلِ سه‌بعدی است. استدلالِ فیزیکی آسان است. از مدلِ فیزیکیِ بخش ۲۵.۲.۴ می‌دانیم اگر نرمالِ سطح به سمتِ منبعِ نور اشاره کند، سطح روشن‌تر است، و اگر به دور اشاره کند، سطح تاریک‌تر است. این استدلال اگر بازتابِ سطح شناخته‌شده نباشد و میدانِ روشنایی یکنواخت نباشد پیچیده‌تر می‌شود، اما به نظر می‌رسد انسان‌ها قادر به گرفتنِ ادراکِ مفیدی از شکل از سایه‌روشن‌اند. ما به‌شکلِ ناامیدکننده‌ای دربارهٔ الگوریتم‌های انجامِ این کار کم می‌دانیم.

اگر شیءِ آشنایی در تصویر باشد، ظاهرش به‌شدت به «وضعیت» (Pose)اش، یعنی موقعیت و جهت‌اش نسبت به بیننده، بستگی دارد. الگوریتم‌های سرراستی برای بازیابیِ وضعیت از تناظرهای بینِ نقطه‌های رویِ شیء و نقطه‌های رویِ مدلِ شیء هست. بازیابیِ وضعیتِ شیءِ شناخته‌شده کاربردهای بسیاری دارد. برای نمونه، در تکلیفِ دست‌کاریِ صنعتی، بازویِ ربات نمی‌تواند شیء را بردارد تا وضعیت شناخته شود. کاربردهای جراحیِ رباتیک به محاسبهٔ دقیقِ تبدیل‌های بینِ موقعیتِ دوربین و موقعیت‌های ابزارِ جراحی و بیمار بستگی دارند (تا تبدیلِ موقعیتِ ابزار به موقعیتِ بیمار به‌دست آید).

رابطه‌های فضاییِ بینِ اشیا سرنخِ مهمِ دیگری است. این‌جا مثالی است. همهٔ عابرانِ پیاده تقریباً همان قد را دارند، و تمایل دارند رویِ صفحهٔ زمین بایستند. اگر بدانیم افق در تصویر کجاست، می‌توانیم عابرانِ پیاده را بر اساسِ فاصله تا دوربین رتبه‌بندی کنیم. این کار می‌کند، زیرا می‌دانیم پاهایشان کجاست، و عابرانی که پاهایشان در تصویر به افق نزدیک‌تر است از دوربین دورترند، و بنابراین باید در تصویر کوچک‌تر باشند. این بدان‌معناست می‌توانیم برخی پاسخ‌های آشکارساز را رد کنیم—اگر آشکارسازی عابری را بیابد که در تصویر بزرگ است و پاهایش نزدیکِ افق، عابری عظیم‌الجثه یافته که وجود ندارد، پس آشکارساز اشتباه می‌کند. به‌نوبهٔ خود، آشکارسازِ عابرِ نسبتاً قابلِ‌اعتمادی قادر به تولیدِ برآوردهایی از افق است، اگر چند عابر در فواصلِ متفاوت از دوربین در صحنه باشند. این به‌دلیلِ آن است که مقیاسِ نسبیِ عابران سرنخی برای جایِ افق است. پس می‌توانیم برآوردِ افق را از آشکارساز استخراج کنیم، سپس از این برآورد برای هرسِ اشتباه‌های آشکارسازِ عابر استفاده کنیم.

۲۵.۷ استفاده از بینایی رایانه‌ای

در این‌جا دامنه‌ای از کاربردهای بینایی رایانه‌ای را مرور می‌کنیم. اکنون ابزارها و جعبه‌ابزارهای بینایی رایانه‌ایِ قابلِ‌اعتمادِ بسیاری هست، پس دامنهٔ کاربردهای موفق و مفید خارق‌العاده است. بسیاری در خانه توسط علاقه‌مندان برای اهدافِ خاص توسعه می‌یابند، که گواهِ این است روش‌ها چقدر قابلِ استفاده‌اند و چقدر تأثیر دارند. (برای نمونه، علاقه‌مندی درِ حیوانِ خانگیِ مبتنی بر تشخیصِ شیءِ عالی‌ای ساخت که اگر گربه موشِ مرده‌ای می‌آورد از ورودش جلوگیری می‌کند—یک جستجویِ وب آن را برایتان پیدا می‌کند.)

۲۵.۷.۱ فهمِ آنچه مردم انجام می‌دهند

اگر می‌توانستیم سامانه‌هایی بسازیم که با تحلیلِ ویدئو بفهمند مردم چه می‌کنند، می‌توانستیم رابطه‌های انسان-رایانه‌ای بسازیم که مردم را تماشا می‌کنند و به رفتارشان واکنش نشان می‌دهند. با این رابطه‌ها، می‌توانستیم: ساختمان‌ها و مکان‌های عمومی را بهتر طراحی کنیم، با جمع‌آوری و استفاده از دادهٔ آنچه مردم در ملأ عام انجام می‌دهند؛ سامانه‌های نظارتِ امنیتیِ دقیق‌تر و کم‌مزاحم‌تر بسازیم؛ مفسرانِ ورزشیِ خودکار بسازیم؛ سایت‌های ساختمانی و محل‌های کار را با تولیدِ هشدار وقتی مردم و ماشین‌ها به‌شکلِ خطرناکی نزدیک می‌شوند ایمن‌تر کنیم؛ بازی‌های رایانه‌ای بسازیم که بازیکن را وادار به بلندشدن و حرکت‌کردن می‌کنند؛ و با مدیریتِ گرما و نورِ ساختمان برای تطبیق با محلِ حضورِ ساکنان و آنچه انجام می‌دهند، انرژی صرفه‌جویی کنیم.

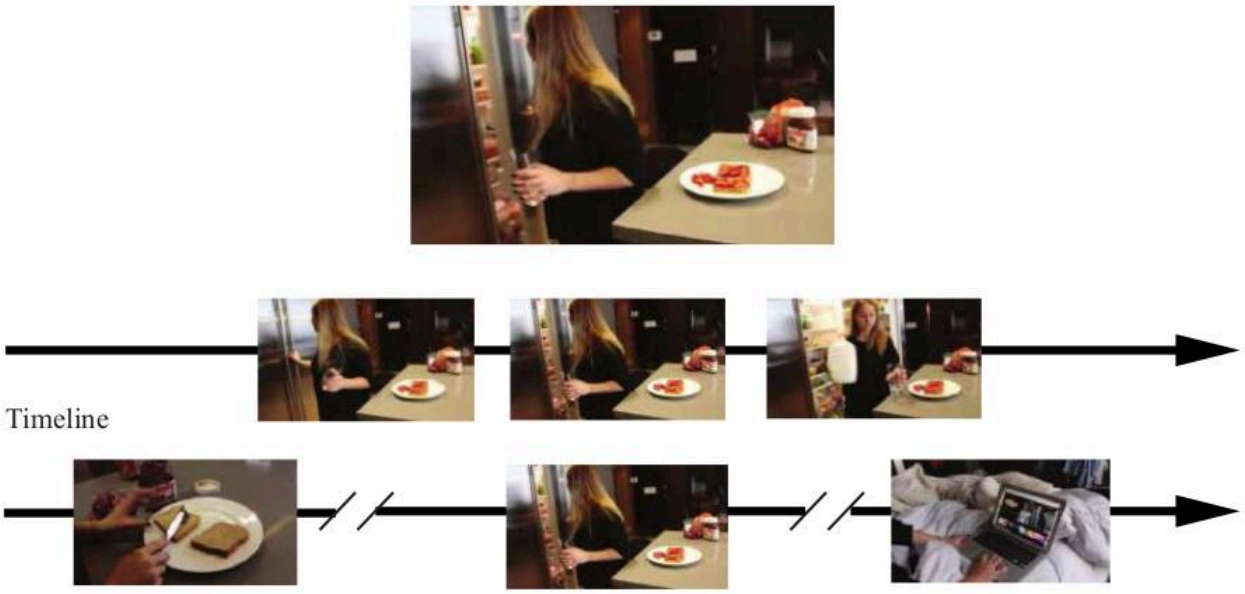
وضعیتِ هنر برای برخی مسائل اکنون فوق‌العاده قوی است. روش‌هایی هست که می‌توانند مکانِ مفصل‌های بدنِ شخص را در تصویر بسیار دقیق پیش‌بینی کنند. برآوردهای کاملاً خوبِ پیکربندیِ سه‌بعدیِ بدنِ آن شخص دنبال می‌شود (شکل ۲۵.۱۶ را ببینید). این کار می‌کند، زیرا تصاویرِ بدن تمایل به اثرهای پرسپکتیویِ ضعیف دارند، و قطعاتِ بدن در طول چندان تغییر نمی‌کنند، پس کوتاه‌نماییِ قطعهٔ بدن در تصویر سرنخِ خوبی برای زاویهٔ بینِ آن و صفحهٔ دوربین است. با حس‌گذارِ عمق، این برآوردها را می‌توان به‌اندازهٔ کافی سریع ساخت تا در رابطه‌های بازیِ رایانه‌ای بنا شوند.

شکل ۲۵.۱۶ – بازسازیِ انسان‌ها از یک تصویرِ تکی اکنون عملی است. هر ردیف بازسازیِ شکلِ بدنیِ سه‌بعدیِ به‌دست‌آمده با استفاده از یک تصویرِ تکی را نشان می‌دهد. این بازسازی‌ها ممکن‌اند، زیرا روش‌ها می‌توانند مکانِ مفصل‌ها، زاویه‌های مفصل در 3D، شکلِ بدن و وضعیتِ بدن را نسبت به تصویر برآورد کنند. هر ردیف مواردِ زیر را نشان می‌دهد: در دورترینِ چپ یک تصویر؛ چپ میانه تصویر با بدنِ بازسازی‌شدهٔ روی‌هم‌نهاده؛ راستِ میانه نمایِ دیگری از بدنِ بازسازی‌شده؛ و دورترینِ راست نمایِ دیگری از بدنِ بازسازی‌شده. نماهای متفاوتِ بدن پنهان‌کردنِ خطاهای بازسازی را بسیار سخت‌تر می‌کنند. شکل با اجازه از آنجو کانازاوا، تولیدشده توسط سامانه‌ای توصیف‌شده در کانازاوا و همکاران (2018a).

رده‌بندیِ آنچه مردم انجام می‌دهند سخت‌تر است. ویدئویی که رفتارهای نسبتاً ساخت‌یافته‌ای مانندِ باله، ژیمناستیک یا تای‌چی نشان می‌دهد، که در آن‌ها واژگانِ کاملاً خاصی هست که به فعالیت‌های بسیار دقیقاً ترسیم‌شده رویِ پس‌زمینه‌های ساده اشاره می‌کنند، به‌راحتی قابلِ‌برخورد است. نتایجِ خوب را می‌توان با دادهٔ برچسب‌خوردهٔ زیاد و شبکهٔ عصبیِ پیچشیِ مناسبی به‌دست آورد.

با این حال، اثباتِ اینکه روش‌ها واقعاً کار می‌کنند می‌تواند دشوار باشد، زیرا آن‌ها چنان شدید به زمینه تکیه می‌کنند. برای نمونه، رده‌بندی که دنباله‌های «شنا» را خیلی خوب برچسب می‌زند ممکن است فقط آشکارسازِ استخرِ شنا باشد، که برای (مثلاً) شناگرانِ رودخانه کار نمی‌کند. مسائلِ عمومی‌تر باز باقی می‌مانند—برای نمونه، چگونه مشاهده‌های بدن و اشیای نزدیک را به هدف‌ها و نیاتِ مردمِ متحرک پیوند دهیم. یکی از منابعِ دشواری این است که رفتارهای مشابه متفاوت به نظر می‌رسند، و رفتارهای متفاوت مشابه، چنان‌که شکل ۲۵.۱۷ نشان می‌دهد. دشواریِ دیگر ناشی از مقیاسِ زمانی است. آنچه شخصی انجام می‌دهد به‌شدت به مقیاسِ زمانی بستگی دارد، چنان‌که شکل ۲۵.۱۸ نشان می‌دهد. اثرِ مهمِ دیگرِ نشان‌داده‌شده در آن شکل این است که رفتار ترکیب می‌شود—چند رفتارِ بازشناسی‌شده ممکن است برای تشکیلِ رفتارِ سطح‌بالاترِ تکی مانندِ آماده‌کردنِ تنقلات ترکیب شوند.

شکل ۲۵.۱۷ – همان کنش می‌تواند بسیار متفاوت به نظر برسد؛ و کنش‌های متفاوت می‌توانند مشابه به نظر برسند. این مثال‌ها کنش‌هایی را نشان می‌دهند که از مجموعهٔ داده‌ای از رفتارهای طبیعی گرفته شده‌اند؛ برچسب‌ها توسط گردآورندگانِ مجموعهٔ داده انتخاب شده‌اند، نه پیش‌بینی‌شده توسط الگوریتم. بالا: مثال‌های برچسبِ «بازکردنِ یخچال»، که برخی از نزدیک و برخی از دور نشان داده شده‌اند. پایین: مثال‌های برچسبِ «برداشتنِ چیزی از یخچال». توجه کنید در هر دو ردیف دستِ سوژه نزدیکِ درِ یخچال است—گفتنِ تفاوتِ بینِ موارد مستلزمِ قضاوتِ کاملاً ظریفی دربارهٔ اینکه دست کجاست و در کجاست است. شکل با اجازه از دیوید فوهی، برگرفته از مجموعهٔ داده‌ای توصیف‌شده در فوهی و همکاران (2018).



شکل ۲۵.۱۸ – آنچه کنش می‌نامید به مقیاسِ زمانی بستگی دارد. فریمِ تکیِ بالا بهتر است به‌عنوانِ بازکردنِ یخچال توصیف شود (وقتی یخچال را می‌بندید به محتویات خیره نمی‌شوید). اما اگر به کلیپِ کوتاهی از ویدئو نگاه کنید (که با فریم‌های ردیفِ میانه نشان داده شده)، کنش بهتر است به‌عنوانِ گرفتنِ شیر از یخچال توصیف شود. اگر به کلیپِ بلندی نگاه کنید (فریم‌های ردیفِ پایین)، کنش بهتر است به‌عنوانِ آماده‌کردنِ تنقلات توصیف شود. توجه کنید این یکی از راه‌هایی را نشان می‌دهد که رفتار ترکیب می‌شود: گرفتنِ شیر از یخچال گاهی بخشی از آماده‌کردنِ تنقلات است، و بازکردنِ یخچال معمولاً بخشی از گرفتنِ شیر از یخچال است. شکل با اجازه از دیوید فوهی، برگرفته از مجموعهٔ داده‌ای توصیف‌شده در فوهی و همکاران (2018).

همچنین ممکن است رفتارهای نامربوط هم‌زمان در جریان باشند، مانندِ خواندنِ آهنگ هنگامِ آماده‌کردنِ تنقلات. چالشی این است که واژگانِ مشترکی برای تکه‌های رفتار نداریم. مردم تمایل دارند فکر کنند نامِ رفتارهای بسیاری را می‌دانند، اما نمی‌توانند فهرستِ بلندی از چنین واژه‌هایی را در صورتِ تقاضا تولید کنند. این مجموعه‌های دادهٔ رفتارهای با برچسبِ پیوسته را سخت‌تر می‌کند. رده‌بندهای آموخته‌شده تضمین می‌شوند فقط اگر دادهٔ آموزشی و آزمون از همان توزیع بیایند خوب رفتار کنند. هیچ راهی برای بررسیِ اعمال‌شدنِ این قید بر تصاویر نداریم، اما به‌صورتِ

تجربی مشاهده می‌کنیم رده‌بندهای تصویر و آشکارسازهای شیء خیلی خوب کار می‌کنند. اما برای دادهٔ فعالیت، رابطهٔ بینِ دادهٔ آموزشی و آزمون بی‌اعتمادتر است، زیرا مردم در بافت‌های بسیار کارهای بسیاری می‌کنند. برای نمونه، فرض کنید آشکارسازِ عابرِ پیاده‌ای داریم که رویِ مجموعهٔ دادهٔ بزرگی خوب عمل می‌کند. پدیده‌های نادری وجود خواهند داشت (مثلاً، مردمِ سوار بر تک‌چرخه) که در مجموعهٔ آموزشی ظاهر نمی‌شوند، پس نمی‌توانیم با قطعیت بگوییم آشکارساز در چنین مواردی چگونه کار خواهد کرد. چالش اثباتِ این است که آشکارساز هرچه عابرانِ پیاده انجام دهند ایمن است، که برای نظریه‌های کنونیِ یادگیری دشوار است.

۲۵.۷.۲ پیوندِ تصاویر و واژه‌ها

بسیاری از مردم تصاویر و ویدئوها را در اینترنت می‌سازند و به اشتراک می‌گذارند. دشواری یافتنِ آنچه می‌خواهید است. معمولاً، مردم می‌خواهند با واژه‌ها جستجو کنند (به‌جای، مثلاً، طرح‌های نمونه). چون بیشترِ تصاویر با واژه‌های الصاق‌شده نمی‌آیند، طبیعی است بکوشیم «سامانه‌های برچسب‌گذاری» (Tagging System) بسازیم که تصاویر را با واژه‌های مربوط برچسب می‌زنند. ماشین‌آلاتِ زیرین سرراست است—روش‌های رده‌بندیِ تصویر و تشخیصِ شیء را اعمال می‌کنیم و تصویر را با واژه‌های خروجی برچسب می‌زنیم. اما برچسب‌ها توصیفِ جامعی از آنچه در تصویر رخ می‌دهد نیستند. مهم است چه کسی چه می‌کند، و برچسب‌ها این را درنمی‌گیرند. برای نمونه، برچسب‌زدنِ تصویری از گربه‌ای در خیابان با رده‌های شیءِ «گربه»، «خیابان»، «سطلِ زباله» و «استخوانِ ماهی» اطلاعاتِ اینکه گربه استخوانِ ماهی را از سطلِ زبالهٔ بازِ خیابان بیرون می‌کشد را حذف می‌کند.

به‌عنوانِ جایگزینی برای برچسب‌گذاری، ممکن است «سامانه‌های شرحنویسی» (Captioning System) بسازیم—سامانه‌هایی که شرحی از یک یا چند جمله می‌نویسند که تصویر را توصیف می‌کند. ماشین‌آلاتِ زیرین باز سرراست است—شبکهٔ پیچشی (برای بازنماییِ تصویر) را به شبکهٔ عصبیِ بازگشتی یا شبکهٔ ترنسفورمر (برای تولیدِ جمله‌ها) جفت کن، و نتیجه را با مجموعهٔ داده‌ای از تصاویرِ شرحدار آموزش بده. تصاویرِ شرحدارِ بسیاری در اینترنت در دسترس‌اند؛ مجموعه‌های دادهٔ ویرایش‌شده از کارِ انسانی برای افزودنِ شرح‌های اضافی به هر تصویر استفاده می‌کنند تا تغییرِ زبانِ طبیعی را دربرگیرند. برای نمونه، مجموعهٔ دادهٔ COCO (اشیای رایج در زمینه) مجموعهٔ جامعی از بیش از ۲۰۰,۰۰۰ تصویر است که با پنج شرح به‌ازایِ هر تصویر برچسب خورده‌اند.

روش‌های کنونیِ شرحنویسی از آشکارسازها برای یافتنِ مجموعه‌ای از واژه‌ها استفاده می‌کنند که تصویر را توصیف می‌کنند، و آن واژه‌ها را به مدلِ دنباله‌ای می‌دهند که برای تولیدِ جمله آموزش دیده. سامانه‌های شرحنویسی ترکیبی از نتایجِ عالی و خطاهای شرم‌آور تولید می‌کنند (شکل ۲۵.۱۹ را ببینید).

سامانه‌های شرحنویسی می‌توانند نادانی‌شان را با حذفِ ذکرِ جزئیاتی که نمی‌توانند درست انجام دهند یا با استفاده از سرنخ‌های زمینه‌ای برای حدس پنهان کنند. برای نمونه، سامانه‌های شرحنویسی تمایل به ضعف در شناساییِ جنسیتِ مردم در تصاویر دارند و اغلب بر مبنای آمارِ دادهٔ آموزشی حدس می‌زنند. این می‌تواند به خطاها بینجامد—مردان نیز خریدکردن را دوست دارند و زنان نیز اسنوبرد می‌کنند. یکی از راه‌های برقرارکردنِ اینکه آیا سامانه بازنماییِ خوبی از آنچه در تصویر رخ می‌دهد دارد، وادارکردنِ آن به پاسخ‌دادن به پرسش‌هایی دربارهٔ تصویر است. این «پاسخ‌گوییِ پرسشِ

بصری» (Visual Question Answering یا VQA) است. جایگزین «گفتگوی بصری» (Visual Dialog) است، که به سامانه تصویری، شرحش و گفتگویی داده می‌شود. سامانه باید سپس به آخرین پرسشِ گفتگو پاسخ دهد. چنان‌که شکل ۲۵.۲۰ نشان می‌دهد، بینایی فوق‌العاده دشوار باقی می‌ماند و سامانه‌های VQA اغلب خطا می‌کنند.

شکل ۲۵.۱۹ – سامانه‌های شرحنویسیِ خودکارِ تصویر برخی نتایجِ خوب و برخی شکست‌ها تولید می‌کنند. دو شرح چپ تصاویرِ مربوطه را به‌خوبی توصیف می‌کنند، هرچند «eating ... in his mouth» نارواني‌ای است که برای مدل‌های زبانِ شبکهٔ عصبیِ بازگشتیِ استفاده‌شده توسط سامانه‌های شرحنویسیِ اولیه نسبتاً معمولی است. برای دو شرحِ راست، به نظر می‌رسد سامانهٔ شرحنویسی سنجاب‌ها را نمی‌شناسد و بنابراین حیوان را از زمینه حدس می‌زند؛ همچنین در بازشناسیِ اینکه دو سنجاب در حالِ خوردن‌اند ناکام می‌ماند. اعتبار تصاویر: geraine/Shutterstock؛ ESB Professional/Shutterstock؛ BushAlex/Shutterstock؛ Maria.Tem/Shutterstock. تصاویرِ نشان‌داده‌شده مشابه اما نه یکسانِ تصاویرِ اصلی‌ای‌اند که شرح‌ها از آن‌ها تولید شده‌اند. برای تصاویرِ اصلی آنیا و همکاران (2018) را ببینید.

شکل ۲۵.۲۰ – سامانه‌های پاسخ‌گوییِ پرسشِ بصری پاسخ‌هایی (معمولاً انتخاب‌شده از مجموعه‌ای چندگزینه‌ای) به پرسش‌های زبانِ طبیعی دربارهٔ تصاویر تولید می‌کنند. بالا: سامانه پاسخ‌های کاملاً معقولی به پرسش‌های نسبتاً دشوار دربارهٔ تصویر تولید می‌کند. پایین: پاسخ‌های رضایت‌بخش‌تر. برای نمونه، سامانه دربارهٔ تعدادِ سوراخ‌های پیتزا حدس می‌زند، زیرا نمی‌فهمد چه چیزی سوراخ به حساب می‌آید، و در شمارش مشکلِ واقعی دارد. به‌همین‌سان، سامانه برای پایِ گربه رنگِ قهوه‌ای انتخاب می‌کند، زیرا پس‌زمینه قهوه‌ای است و نمی‌تواند پا را درست مکان‌یابی کند. اعتبار تصاویر: (بالا) Tobyanna/Shutterstock؛ Shutterstock/679411؛ ESB Professional/Shutterstock؛ Africa Studio/Shutterstock؛ (پایین) استوارت راسل؛ Maxisport/Shutterstock؛ Chendongshan/Shutterstock؛ و... (پرسش‌های نشان‌داده‌شده: «گربه چه پوشیده است؟» – «کلاه»؛ «هوا چطور است؟» – «بارانی»؛ «این چه سطحی است؟» – «خاک رس»؛ «رویِ پیتزا چه چیزهایی هست؟» – «قارچ»؛ «پیتزا چند سوراخ دارد؟» – «۸»؛ «رویِ راکت چه حرفی است؟» – «w»؛ «پایِ جلویِ راست چه رنگی است؟» – «قهوه‌ای»؛ «چرا تابلو خم شده؟» – «نیست».)

۲۵.۷.۳ بازسازی از نماهای متعدد

بازسازیِ مجموعه‌ای از نقطه‌ها از نماهای متعدد—که می‌توانند از ویدئو یا از تجمیعِ عکس‌های گردشگری بیایند—شبیهِ بازسازیِ نقطه‌ها از دو نماست، اما تفاوت‌های مهمی هست. کارِ بسیار بیشتری برای برقرارکردنِ تناظر بینِ نقطه‌های نماهای متفاوت لازم است، و نقطه‌ها می‌توانند واردِ نما شوند و از آن خارج شوند، که فرایندِ تطبیق و بازسازی را شلوغ‌تر می‌کند. اما نماهای بیشتر قیدهای بیشتری بر بازسازی و بر پارامترهای نمایِ بازیابی‌شده می‌گذارند، پس معمولاً می‌توان برآوردهای فوق‌العاده دقیقی از هم موقعیتِ نقطه‌ها و هم پارامترهای نما تولید کرد. به‌طورِ تقریبی، بازسازی با تطبیقِ نقطه‌ها رویِ جفتِ تصاویر پیش می‌رود، این تطبیق‌ها را به گروه‌های تصاویر بسط می‌دهد، راه‌حلِ تقریبی هم برای هندسه و هم برای پارامترهای نما ارائه می‌کند، و سپس آن راه‌حل را صیقل می‌دهد. صیقل‌دادن به معنای کمینه‌کردنِ خطایِ بینِ نقطه‌های پیش‌بینی‌شده توسط مدل (هندسه و پارامترهای نما) و مکان‌های ویژگی‌های تصویر است. رویه‌های دقیق برای پوششِ کاملِ این‌جا بیش‌ازحد پیچیده‌اند، اما اکنون بسیار خوب فهمیده و کاملاً قابلِ‌اعتمادند.

همهٔ قیدهای هندسیِ رویِ تناظرها برای هر شکلِ قابلِ‌تصورِ مفیدِ دوربین شناخته شده‌اند. رویه‌ها را می‌توان برای برخورد با نماهایی که ارتوگرافیک نیستند؛ با نقطه‌هایی که فقط در برخی نماها مشاهده می‌شوند؛ با پارامترهای ناشناختهٔ دوربین (مانندِ فاصلهٔ کانونی)؛ و برای بهره‌برداری از جستجوهای پیچیدهٔ گوناگون برای تناظرهای مناسب تعمیم داد. بازسازیِ دقیقِ مدلی از کلِ یک شهر از تصاویر عملی است. برخی کاربردها:

- **مدل‌سازی:** برای نمونه، ممکن است سامانهٔ مدل‌سازی‌ای ساخت که نماهای بسیاری را که شیئی را نشان می‌دهند می‌گیرد و مشِ بسیار دقیقی از چندضلعی‌های بافت‌دار برای استفاده در کاربردهای گرافیکِ رایانه‌ای و واقعیتِ مجازی تولید می‌کند. ساختِ این‌گونه مدل‌ها از ویدئو معمول است، اما اکنون می‌توان چنین مدل‌هایی را از مجموعه‌های ظاهراً تصادفیِ عکس ساخت. برای نمونه، می‌توانید مدلِ سه‌بعدیِ مجسمهٔ آزادی را از عکس‌های یافت‌شده در اینترنت بسازید.
- **ترکیبِ انیمیشن با بازیگرانِ زنده در ویدئو:** برای قراردادنِ شخصیت‌های گرافیکِ رایانه‌ای در ویدئویِ واقعی، باید بدانیم دوربین برای ویدئویِ واقعی چگونه حرکت کرده، تا بتوانیم شخصیت را درست رندر کنیم و نما را با حرکتِ دوربین تغییر دهیم.
- **بازسازیِ مسیر:** ربات‌های متحرک باید بدانند کجا بوده‌اند. اگر ربات دوربین داشته باشد، می‌توانیم مدلی از مسیرِ دوربین در طولِ جهان بسازیم؛ این به‌عنوانِ بازنماییِ مسیرِ ربات خدمت خواهد کرد.
- **مدیریتِ ساخت‌وساز:** ساختمان‌ها مصنوعاتِ فوق‌العاده پیچیده‌ای‌اند، و پیگیریِ آنچه در طولِ ساخت رخ می‌دهد دشوار و پرهزینه است. یکی از راه‌های پیگیری پروازِ پهپادها در سایتِ ساخت‌وساز هفته‌ای یک بار، برای فیلم‌برداریِ وضعیتِ کنونی است. سپس مدلِ سه‌بعدیِ وضعیتِ کنونی را بساز و تفاوتِ بینِ نقشه‌ها و بازسازی را با فنونِ تجسم بررسی کن. شکل ۲۵.۲۱ این کاربرد را نشان می‌دهد.

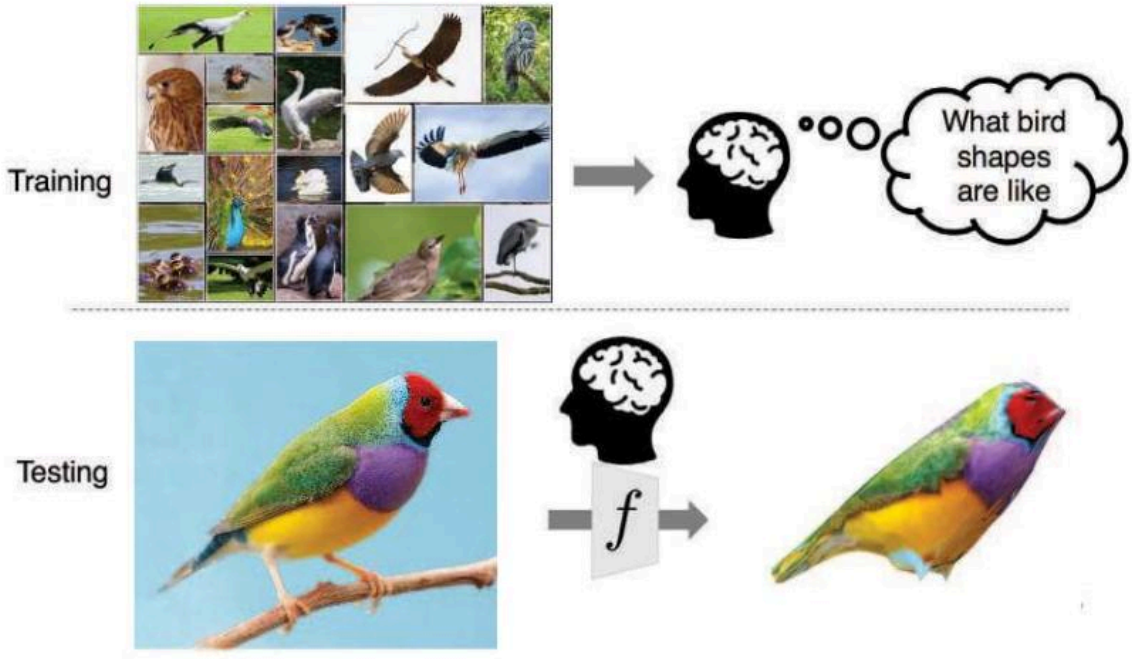
شکل ۲۵.۲۱ – مدل‌های سه‌بعدیِ سایت‌های ساخت‌وساز از تصاویر توسط الگوریتم‌های ساختار-از-حرکت و استریویِ چندنمایی تولید می‌شوند. آن‌ها به شرکت‌های ساخت‌وساز کمک می‌کنند کار رویِ ساختمان‌های بزرگ را با مقایسهٔ مدلِ سه‌بعدیِ ساختِ واقعیِ تاکنون با نقشه‌های ساختمان هماهنگ کنند. چپ: تجسمی از مدلِ هندسیِ گرفته‌شده توسط پهپادها. نقطه‌های سه‌بعدیِ بازسازی‌شده به‌صورتِ رنگی رندر شده‌اند، پس نتیجه مانندِ پیشرفتِ تاکنون به نظر می‌رسد (توجه کنید به ساختمانِ نیمه‌کاره با جرثقیل). هرم‌های کوچک وضعیتِ پهپاد را هنگامِ گرفتنِ تصویر نشان می‌دهند، تا تجسمِ مسیرِ پرواز ممکن شود. راست: این سامانه‌ها در واقع توسط تیم‌های ساخت‌وساز استفاده می‌شوند؛ این تیم مدلِ سایتِ ساخته‌شده را می‌بیند و آن را با نقشه‌های ساختمان به‌عنوانِ بخشی از جلسهٔ هماهنگی مقایسه می‌کند. شکل با اجازه از درک هویم، مانی گل‌پروارفرد و Reconstruct، تولیدشده توسط سامانهٔ تجاریِ توصیف‌شده در پستی وبلاگی در medium.com/reconstruct-inc.

۲۵.۷.۴ هندسه از یک نمایِ تکی

بازنمایی‌های هندسی اگر بخواهید حرکت کنید به‌ویژه مفیدند، زیرا می‌توانند آشکار کنند کجایید، کجا می‌توانید بروید و احتمالاً به چه چیزی برخورد خواهید کرد. اما همیشه راحت نیست برای تولیدِ مدلِ هندسی از نماهای متعدد استفاده کنیم. برای نمونه، وقتی در را باز می‌کنید و به اتاقی قدم می‌گذارید، چشمانتان برای بازیابیِ بازنماییِ خوبی از عمقِ اشیای دورِ آن سویِ اتاق بیش‌ازحد به هم نزدیک‌اند. می‌توانید سرتان را به عقب و جلو حرکت دهید، اما این وقت‌گیر و ناراحت‌کننده است.

جایگزین پیش‌بینیِ «نقشهٔ عمق» (Depth Map)—آرایه‌ای که عمقِ هر پیکسلِ تصویر را می‌دهد، به‌طورِ اسمی از دوربین—از یک تصویرِ تکی است. برای بسیاری از انواعِ صحنه، انجامِ دقیقِ این به‌شکلِ شگفت‌آوری آسان است، زیرا نقشهٔ عمق ساختارِ کاملاً ساده‌ای دارد. این به‌ویژه برای اتاق‌ها و صحنه‌های داخلی به‌طورِ کلی صادق است. مکانیکِ کار سرراست است. مجموعهٔ داده‌ای از تصاویر و نقشه‌های عمق به‌دست می‌آوری، سپس شبکه‌ای آموزش می‌دهی که نقشه‌های عمق را از تصاویر پیش‌بینی کند. انواعِ جالبی از تغییرِ مسئله قابلِ حل‌اند. مشکلِ نقشهٔ عمق این است که هیچ‌چیز دربارهٔ پشتِ اشیا یا فضایِ پشتِ اشیا به شما نمی‌گوید. اما روش‌هایی هست که می‌توانند پیش‌بینی کنند چه «وکسل»هایی (3D پیکسل) توسط اشیای شناخته‌شده اشغال شده‌اند (هندسهٔ شیء شناخته‌شده است) و نقشهٔ عمق اگر شیئی حذف می‌شد چه‌شکل می‌شد (و بنابراین کجا می‌توانید اشیا را پنهان کنید). این روش‌ها کار می‌کنند، زیرا شکل‌های اشیا کاملاً قویاً سبک‌پردازی‌شده‌اند.

چنان‌که در بخش ۲۵.۶.۴ دیدیم، بازیابیِ وضعیتِ شیءِ شناخته‌شده با استفاده از مدلِ سه‌بعدی سرراست است. حالا تصور کنید تصویرِ تکی از، مثلاً، گنجشکی می‌بینید. اگر در گذشته تصاویرِ بسیاری از پرندگانِ گنجشک‌مانند دیده باشید، می‌توانید برآوردِ معقولی از هم وضعیتِ گنجشک و هم مدلِ هندسی‌اش از آن تصویرِ تکی بازسازی کنید. با استفاده از تصاویرِ گذشته، خانوادهٔ پارامتریکِ کوچکی از مدل‌های هندسی برای پرندگانِ گنجشک‌مانند می‌سازید؛ سپس از رویهٔ بهینه‌سازی برای یافتنِ بهترین مجموعهٔ پارامترها و دیدگاه‌ها برای تبیینِ تصویری که می‌بینید استفاده می‌شود. این استدلال برای تأمینِ بافتِ آن مدل نیز کار می‌کند، حتی برای بخش‌هایی که نمی‌توانید ببینید (شکل ۲۵.۲۲).



شکل ۲۵.۲۲ – اگر تصاویرِ بسیاری از رده‌ای—مثلاً، پرندگان (بالا)—دیده باشید، می‌توانید از آن‌ها برای تولیدِ بازسازیِ سه‌بعدی از نمایِ تکیِ تازه (پایین) استفاده کنید. باید مطمئن باشید همهٔ اشیا هندسهٔ نسبتاً مشابهی دارند (پس تصویرِ شترمرغ اگر به گنجشک نگاه می‌کنید کمکی نمی‌کند)، اما روش‌های رده‌بندی می‌توانند این را مرتب کنند. از تصاویرِ بسیار می‌توانید برآورد کنید مقادیرِ بافت در تصویر چگونه رویِ شیء توزیع می‌شوند، و بنابراین بافتِ بخش‌هایی از پرنده را که هنوز ندیده‌اید کامل کنید (پایین). شکل با اجازه از آنجو کانازاوا، تولیدشده توسط سامانه‌ای توصیف‌شده در کانازاوا و همکاران (2018b). اعتبار عکسِ بالا: Satori/123RF؛ اعتبار پایینِ چپ: Four Oaks/Shutterstock.

۲۵.۷.۵ ساختنِ تصاویر

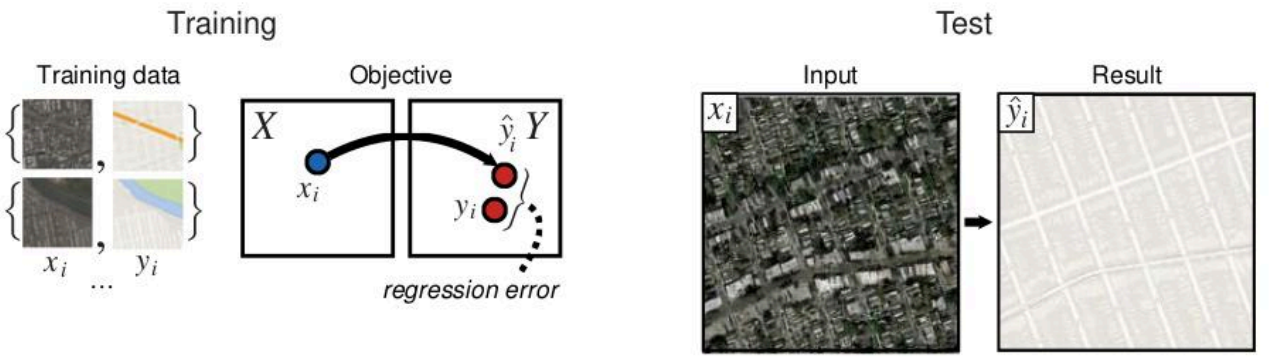
درجِ مدل‌های گرافیکِ رایانه‌ای در عکس‌ها به‌شکلی قانع‌کننده اکنون رایج است، چنان‌که در شکل ۲۵.۲۳، که مجسمه‌ای در عکسی از اتاق قرار داده شده. نخست نقشهٔ عمق و سپیدایی را برای عکس برآورد کن. سپس نورپردازیِ تصویر را با تطبیقِ آن با تصاویرِ دیگرِ با نورپردازیِ شناخته‌شده برآورد کن. شیء را در نقشهٔ عمقِ تصویر قرار بده، و جهانِ حاصل را با برنامهٔ رندرِ فیزیکی—ابزارِ استانداردِ گرافیکِ رایانه‌ای—رندر کن. سرانجام، تصویرِ اصلاح‌شده را با تصویرِ اصلی ترکیب کن.

شبکه‌های عصبی را نیز می‌توان برای انجامِ «تبدیلِ تصویر» (Image Transformation) آموزش داد: نگاشتِ تصاویرِ نوعِ X—برای نمونه، تصویرِ تار؛ تصویرِ هواییِ شهر؛ یا طراحیِ محصولِ تازه—به تصاویرِ نوعِ Y—برای نمونه، نسخهٔ غیرتارِ تصویر؛ نقشهٔ جاده؛ یا عکسِ محصول. این وقتی آسان‌تر است که دادهٔ آموزشی شاملِ جفت‌های (X,Y) تصاویر باشد—در شکل ۲۵.۲۴ هر جفتِ نمونه تصویرِ هوایی و بخشِ نقشهٔ جادهٔ متناظر را دارد. زیانِ آموزشِ خروجیِ شبکه را با

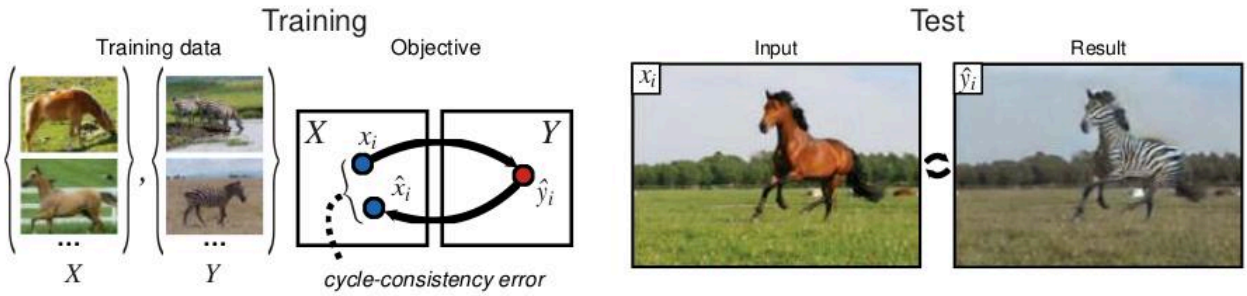
خروجیِ مطلوب مقایسه می‌کند، و همچنین مؤلفهٔ زیانی از شبکهٔ مولدِ تخاصمی (GAN) دارد که تضمین می‌کند خروجی انواعِ درستِ ویژگی‌ها را برای تصاویرِ نوعِ Y دارد. چنان‌که در بخشِ آزمونِ شکل ۲۵.۲۴ می‌بینیم، سامانه‌های این‌گونه خیلی خوب عمل می‌کنند.

شکل ۲۵.۲۳ — در چپ، تصویری از صحنهٔ واقعی. در راست، شیءِ گرافیکِ رایانه‌ای در صحنه درج شده. می‌بینید نور به نظر می‌رسد از جهتِ درست می‌آید، و شیء به نظر می‌رسد سایه‌های مناسب می‌اندازد. تصویرِ تولیدشده حتی اگر خطاهای کوچکی در نورپردازی و سایه‌ها باشد قانع‌کننده است، زیرا مردم در شناساییِ این خطاها خبره نیستند. شکل با اجازه از کوین کارش، تولیدشده توسط سامانه‌ای توصیف‌شده در کارش و همکاران (2011).

گاهی تصاویری که با یکدیگر جفت شده‌اند نداریم، اما مجموعهٔ بزرگی از تصاویرِ نوعِ X (مثلاً، تصاویرِ اسب) و مجموعهٔ جداگانه‌ای از نوعِ Y (مثلاً، تصاویرِ گورخر) داریم. هنرمندی را تصور کنید که مأمورِ ایجادِ تصویری از گورخری در حالِ دویدن در مزرعه است. هنرمند از تواناییِ انتخابِ دقیقاً تصویرِ درستِ اسب و سپس تبدیلِ خودکارِ اسب به گورخر توسط رایانه قدردانی می‌کرد (شکل ۲۵.۲۵). برای رسیدن به این، می‌توانیم دو شبکهٔ تبدیل آموزش دهیم، با قیدِ اضافی‌ای به نامِ «قیدِ چرخه» (Cycle Constraint). شبکهٔ نخست اسب‌ها را به گورخرها نگاشت می‌کند؛ شبکهٔ دوم گورخرها را به اسب‌ها؛ و قیدِ چرخه ایجاب می‌کند وقتی X را به Y و سپس به X نگاشت می‌کنید (یا Y به X به Y)، همان چیزی را بگیرید که با آن شروع کرده‌اید. باز، زیان‌های GAN تضمین می‌کنند تصاویرِ اسب (یا گورخر) که شبکه‌ها خروجی می‌دهند «شبیهِ» تصاویرِ واقعیِ اسب (یا گورخر) باشند.



شکل ۲۵.۲۴ — تبدیلِ تصویرِ جفت‌شده، که در آن ورودی شاملِ تصاویرِ هوایی و کاشی‌های نقشهٔ متناظر است، و هدف آموزشِ شبکه‌ای برای تولیدِ کاشیِ نقشه از تصویرِ هوایی است. (سامانه همچنین می‌تواند تولیدِ تصاویرِ هوایی از کاشی‌های نقشه را یاد بگیرد.) شبکه با مقایسهٔ $\hat{y}_i$ (خروجیِ مثالِ x_i از نوعِ X) با خروجیِ درستِ y_i از نوعِ Y آموزش می‌بیند. سپس در زمانِ آزمون، شبکه باید تصاویرِ تازهٔ نوعِ Y را از ورودی‌های تازهٔ نوعِ X بسازد. شکل با اجازه از فیلیپ ایزولا، جون-یان ژو و الکسی ای. افرس، تولیدشده توسط سامانه‌ای توصیف‌شده در ایزولا و همکاران (2017). دادهٔ نقشه © Google 2019.



شکل ۲۵.۲۵ — تبدیلِ تصویرِ جفت‌نشده: با توجه به دو جمعیتِ تصویر (اینجا نوعِ X اسب‌ها و نوعِ Y گورخرهاست)، اما بدونِ جفت‌های متناظر، تبدیلِ اسب به گورخر را یاد بگیر. روش دو پیش‌بین آموزش می‌دهد: یکی که نوعِ X را به نوعِ Y نگاشت می‌کند، و دیگری که نوعِ Y را به نوعِ X. اگر شبکهٔ نخست اسبِ x_i را به گورخرِ $\hat{y}_i$ نگاشت کند، شبکهٔ دوم باید $\hat{y}_i$ را به x_i اصلی برگرداند. تفاوتِ بینِ x_i و $\hat{x}_i$ همان چیزی است که دو شبکه را آموزش می‌دهد. چرخه از Y به X و باز باید بسته شود. چنین شبکه‌هایی می‌توانند تبدیل‌های غنی‌ای بر تصاویر اعمال کنند. شکل با اجازه از الکسی ای. افرس؛ ژو و همکاران (2017) را ببینید. عکسِ اسبِ در حالِ دویدن از Justyna Furmanczyk Gibaszek/Shutterstock.

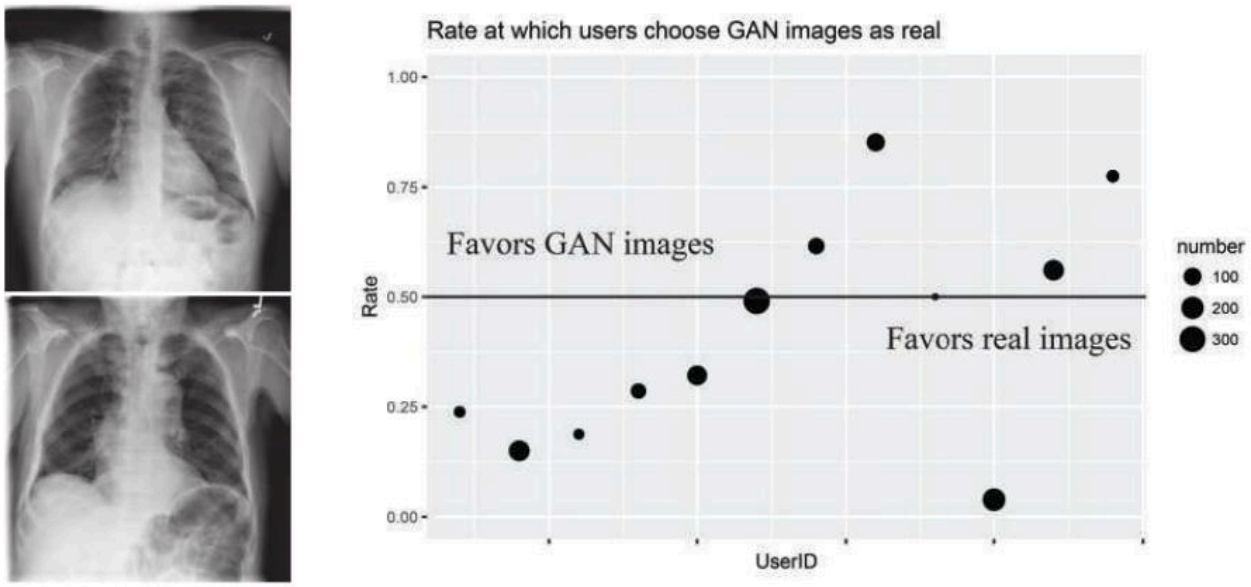
اثرِ هنریِ دیگری «انتقالِ سبک» (Style Transfer) نامیده می‌شود: ورودی شاملِ دو تصویر است—«محتوا» (برای نمونه، عکسی از گربه)؛ و «سبک» (برای نمونه، نقاشیِ انتزاعی). خروجی نسخه‌ای از گربه است که در سبکِ انتزاعی رندر شده (شکل ۲۵.۲۶ را ببینید). بینشِ کلیدیِ حلِ این مسئله این است که اگر شبکهٔ عصبیِ پیچشیِ عمیقی (CNN) را بررسی کنیم که برای بازشناسیِ اشیا آموزش دیده (مثلاً، رویِ ImageNet)، می‌یابیم لایه‌های اولیه تمایل به بازنماییِ سبکِ تصویر دارند و لایه‌های پایانی محتوا را. p تصویرِ محتوا و s تصویرِ سبک باشد، و $E(x)$ بردارِ فعال‌سازی‌های لایهٔ اولیه رویِ تصویرِ x و $L(x)$ بردارِ فعال‌سازی‌های لایهٔ پایانی رویِ تصویرِ x باشد. سپس می‌خواهیم تصویری x تولید کنیم که محتوایی مشابهِ عکسِ خانه دارد، یعنی $\|L(x) - L(p)\|$ را کمینه می‌کند، و همچنین سبکی مشابهِ نقاشیِ امپرسیونیستی دارد، یعنی $\|E(x) - E(s)\|$ را کمینه می‌کند. از نزولِ گرادیان با تابعِ زیانی که ترکیبِ خطیِ این دو عامل است برای یافتنِ تصویری x استفاده می‌کنیم که زیان را کمینه می‌کند.

شبکه‌های مولدِ تخاصمی (GAN) می‌توانند تصاویرِ واقع‌نمایِ تازه‌ای خلق کنند که بیشترِ مردم را بیشترِ اوقات می‌فریبد. یکی از انواعِ تصویر «دیپ‌فیک» (Deepfake) است—تصویری یا ویدئویی که شبیهِ شخصِ خاصی است، اما از مدلی تولید شده. برای نمونه، وقتی کری فیشر ۶۰ ساله بود، کپیِ تولیدشده‌ای از چهرهٔ ۱۹ ساله‌اش رویِ بدنِ بازیگرِ دیگری برای

ساختِ Rogue One قرار داده شد. صنعتِ فیلم دیپ‌فیک‌های روزبه‌روز بهتر برای اهدافِ هنری می‌سازد، و پژوهشگران رویِ اقداماتِ متقابل برای تشخیصِ دیپ‌فیک‌ها کار می‌کنند، تا اثرهای مخربِ اخبارِ جعلی کاهش یابد.

تصاویرِ تولیدشده همچنین می‌توانند برای حفظِ حریم خصوصی استفاده شوند. برای نمونه، مجموعه‌های دادهٔ تصویری در رویه‌های رادیولوژیک هست که برای پژوهشگران مفید می‌بود، اما به‌دلیلِ محرمانگیِ بیمار منتشر نمی‌شود. مدل‌های تصویرِ مولد می‌توانند مجموعهٔ دادهٔ خصوصیِ تصاویر را بگیرند و مجموعهٔ دادهٔ مصنوعیِ قابلِ‌اشتراک‌گذاری با پژوهشگران تولید کنند. این مجموعهٔ داده باید (الف) مانندِ مجموعهٔ دادهٔ آموزشی باشد؛ (ب) متفاوت؛ و (ج) قابلِ کنترل. اشعهٔ ایکسِ قفسهٔ سینه را در نظر بگیر. مجموعهٔ دادهٔ مصنوعی باید به این معنا مانندِ مجموعهٔ دادهٔ آموزشی باشد که هر تصویرِ منفرد رادیولوژیست را می‌فریبد و بسامدهای هر اثر باید درست باشد، تا رادیولوژیست از اینکه (مثلاً) ذات‌الریه چند بار ظاهر می‌شود شگفت‌زده نشود. مجموعهٔ دادهٔ تازه باید متفاوت باشد، به این معنا که اطلاعاتِ شخصاً قابلِ شناسایی را آشکار نمی‌کند. مجموعهٔ دادهٔ تازه باید قابلِ کنترل باشد، تا بسامدهای اثرها بتوانند برای بازتابِ جوامعِ موردِ علاقه تنظیم شوند. برای نمونه، ذات‌الریه‌ها در سالمندان رایج‌تر از بزرگسالانِ جوان‌اند. رسیدن به هر یک از این هدف‌ها از نظرِ فنی دشوار است، اما مجموعه‌های دادهٔ تصویری ساخته شده‌اند که رادیولوژیست‌های شاغل را برخی از اوقات می‌فریبند (شکل ۲۵.۲۷).

شکل ۲۵.۲۶ – انتقالِ سبک: محتوای عکسی از گربه با سبکِ نقاشیِ انتزاعی ترکیب می‌شود تا تصویرِ تازه‌ای از گربه به‌دست آید که در سبکِ انتزاعی رندر شده (راست). نقاشی «Lyrisches» یا «The Lyrical» اثرِ واسیلی کاندینسکی است (مالکیتِ عمومی)؛ گربه کازمو است.



شکل ۲۵.۲۷ — تصاویرِ تولیدشده توسط GAN از اشعهٔ ایکسِ ریه. در چپ، جفتی شاملِ اشعهٔ ایکسِ واقعی و اشعهٔ ایکسِ تولیدشده توسط GAN. در راست، نتایجِ آزمونی که از رادیولوژیست‌ها می‌خواست، با توجه به جفتی از اشعهٔ ایکس مانندِ چپ، بگویند کدام اشعهٔ ایکسِ واقعی است. به‌طورِ میانگین، آن‌ها ۶۱٪ مواقع درست انتخاب کردند، کمی بهتر از شانس. اما دقت‌شان متفاوت بود—نمودارِ راست نرخِ خطایِ ۱۲ رادیولوژیستِ متفاوت را نشان می‌دهد؛ یکی نرخِ خطایی نزدیکِ ۰٪ و دیگری ۸۰٪ خطا داشت. اندازهٔ هر نقطه تعدادِ تصاویری را نشان می‌دهد که هر رادیولوژیست دیده. شکل با اجازه از الکس شوینگ، تولیدشده توسط سامانه‌ای توصیف‌شده در دشپانده و همکاران (2019).

۲۵.۷.۶ کنترلِ حرکت با بینایی

یکی از کاربردهای اصلیِ بینایی فراهم‌کردنِ اطلاعات هم برای دست‌کاریِ اشیا—برداشتنِ آن‌ها، گرفتنِ آن‌ها، چرخاندنِ آن‌ها و مانندِ آن—و هم برای ناوبری با اجتناب از موانع است. توانایی استفاده از بینایی برای این مقاصد در ابتدایی‌ترین سامانه‌های بیناییِ جانوری حاضر است. در بسیاری موارد، سامانهٔ بینایی حداقلی است، به این معنا که از میدانِ نورِ در دسترس فقط اطلاعاتی را استخراج می‌کند که جانور برای آگاه‌کردنِ رفتارش نیاز دارد. کاملاً محتمل است سامانه‌های بیناییِ مدرن از جانداران اولیهٔ ابتدایی‌ای تکامل یافته‌اند که از نقطهٔ حساس به نوری در یک سر برای جهت‌گیری به سمتِ نور (یا دور از آن) استفاده می‌کردند. در بخش ۲۵.۶ دیدیم مگس‌ها از سامانهٔ تشخیصِ جریانِ نوریِ بسیار ساده‌ای برای فرودآمدن رویِ دیوارها استفاده می‌کنند. فرض کنید، به‌جای فرودآمدن رویِ دیوارها، می‌خواهیم خودرویِ خودران بسازیم. این پروژه‌ای است که تقاضاهای بسیار بزرگ‌تری بر سامانهٔ ادراکی می‌گذارد. ادراک در خودرویِ خودران باید از تکلیف‌های زیر پشتیبانی کند:

- **کنترلِ جانبی (Lateral Control):** اطمینان از اینکه وسیلهٔ نقلیه به‌طورِ امن درونِ خطِ خودش می‌ماند یا هنگامِ نیاز به‌راحتی خط عوض می‌کند.
- **کنترلِ طولی (Longitudinal Control):** اطمینان از اینکه فاصلهٔ امنی تا وسیلهٔ نقلیهٔ جلویی هست.
- **اجتناب از مانع (Obstacle Avoidance):** نظارت بر وسایلِ نقلیه در خطوطِ مجاور و آماده‌بودن برای مانورهای فرار. تشخیصِ عابرانِ پیاده و اجازه‌دادن به آن‌ها برای عبورِ امن.
- **اطاعت از علائمِ راهنمایی (Obey Traffic Signals):** این‌ها شاملِ چراغ‌های راهنمایی، علائمِ ایست، علائمِ محدودیتِ سرعت و نشانه‌های دستیِ پلیس است.

مشکل برای راننده (انسانی یا رایانه‌ای) تولیدِ کنش‌های فرمان، شتاب و ترمزِ مناسب برای بهترین انجامِ این تکلیف‌هاست.

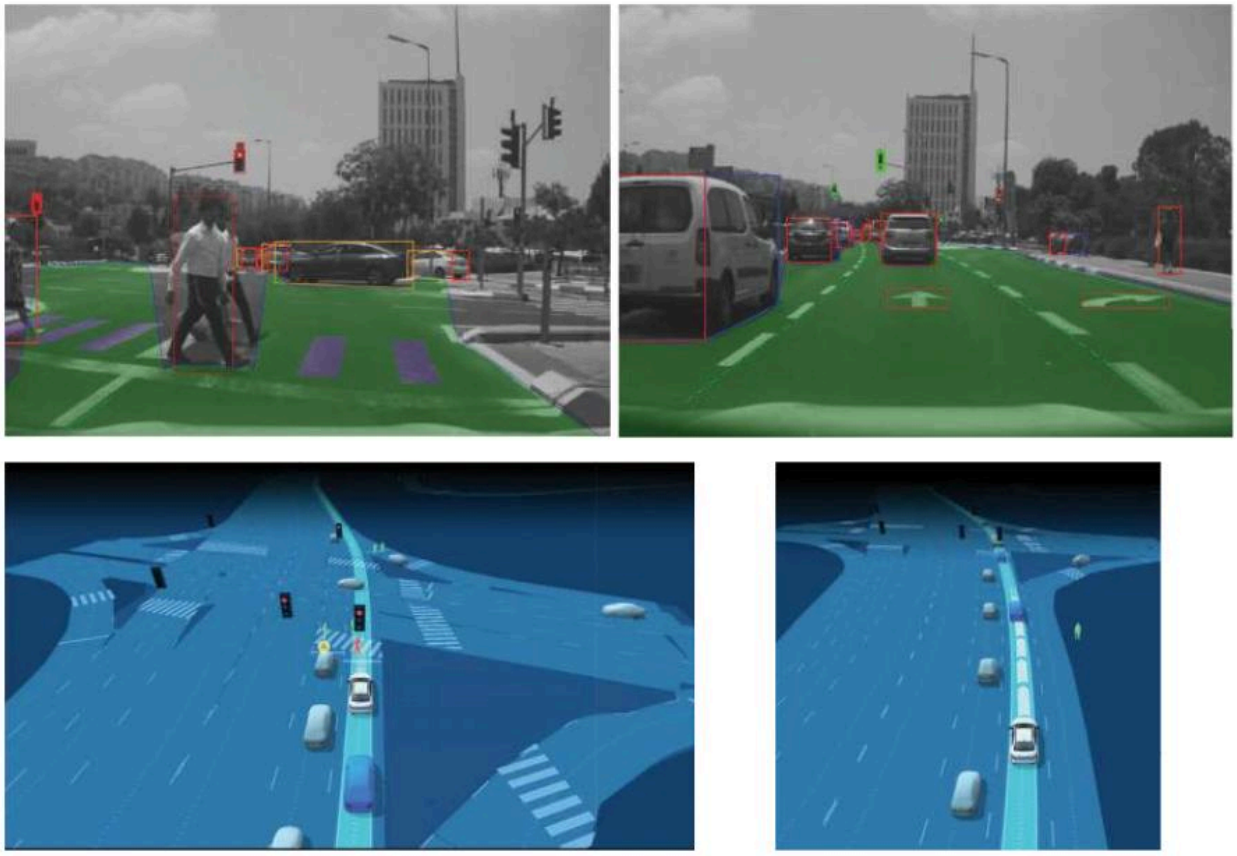
برای تصمیم‌گیریِ خوب، راننده باید مدلی از جهان و اشیای آن بسازد. شکل ۲۵.۲۸ برخی از استنتاج‌های بصریِ لازم برای ساختنِ این مدل را نشان می‌دهد. برای کنترلِ جانبی، راننده باید بازنماییِ موقعیت و جهتِ خودرو را نسبت به خط حفظ کند. برای کنترلِ طولی، راننده باید فاصلهٔ امنی از وسیلهٔ نقلیهٔ جلویی حفظ کند (که ممکن است شناسایی‌اش رویِ، مثلاً، جاده‌های چندخطِ خمیده آسان نباشد). اجتناب از مانع و پیروی از علائمِ راهنمایی به استنتاج‌های اضافی نیاز دارند.

جاده‌ها برای انسان‌هایی طراحی شده‌اند که با بینایی ناوبری می‌کنند، پس در اصل باید رانندگی با فقط بینایی ممکن باشد. با این حال، در عمل، خودروهای خودرانِ تجاری از انواعِ حس‌گذارها، از جملهِ دوربین‌ها، لیدارها، رادارها و میکروفون‌ها استفاده می‌کنند. لیدار یا رادار اندازه‌گیریِ مستقیمِ عمق را ممکن می‌کند، که می‌تواند دقیق‌تر از روش‌های فقط-بینایی بخش ۲۵.۶ باشد. داشتنِ حس‌گذارهای متعدد عملکرد را به‌طورِ کلی افزایش می‌دهد، و در شرایطِ دیدِ ضعیف به‌ویژه مهم است؛ برای نمونه، رادار می‌تواند از مهی که دوربین‌ها و لیدارها را مسدود می‌کند عبور کند. میکروفون‌ها می‌توانند وسایلِ نقلیهٔ نزدیک‌شونده (به‌ویژه آن‌های با آژیر) را پیش از دیده‌شدن تشخیص دهند.

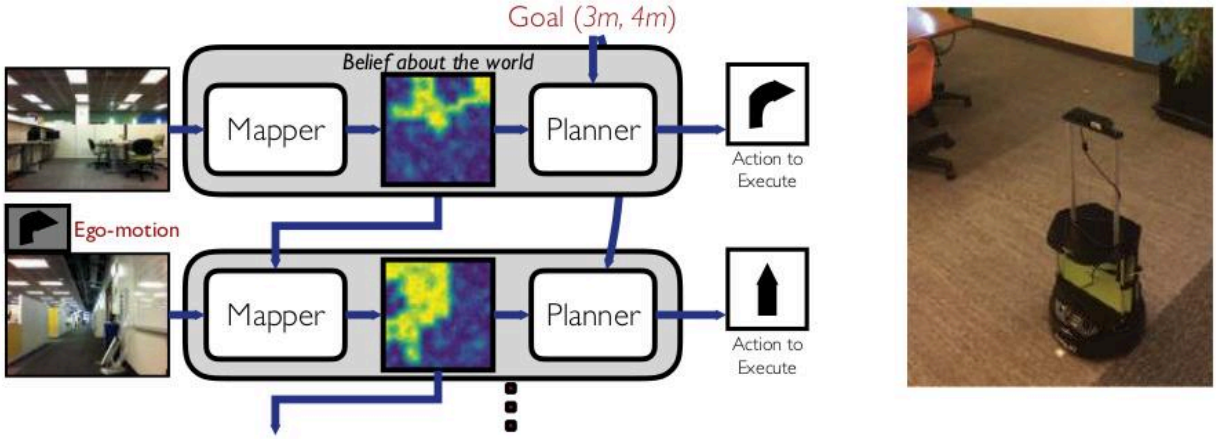
پژوهشِ زیادی نیز دربارهٔ ربات‌های متحرکی که در محیط‌های داخلی و خارجی ناوبری می‌کنند شده. کاربردها فراوان‌اند، مانندِ آخرین مایلِ تحویلِ بسته یا پیتزا. رویکردهای سنتی این تکلیف را به دو مرحله می‌شکنند، چنان‌که در شکل ۲۵.۲۹ نشان داده شده:

- **نقشه‌سازی (Map Building):** «مکان‌یابی و نگاشتِ هم‌زمان» یا SLAM (صفحهٔ ۹۳۵ را ببینید) تکلیفِ ساختنِ مدلِ سه‌بعدیِ جهان، شاملِ مکانِ ربات در جهان (یا به‌طورِ مشخص‌تر، مکانِ هر یک از دوربین‌های ربات) است. این مدل (که معمولاً به‌صورتِ ابرِ نقطه‌ای از موانع بازنمایی می‌شود) را می‌توان از دنباله‌ای از تصاویر از موقعیت‌های متفاوتِ دوربین ساخت.
- **برنامه‌ریزیِ مسیر (Path Planning):** وقتی ربات به این نقشهٔ سه‌بعدی دسترسی دارد و می‌تواند خود را در آن مکان‌یابی کند، هدف به یافتنِ مسیرِ بدونِ برخورد از موقعیتِ کنونی به مکانِ هدف تبدیل می‌شود (بخش ۲۶.۶ را ببینید).

گونه‌های بسیاری از این رویکردِ کلی کاوش شده‌اند. برای نمونه، در رویکردِ «نگاشت و برنامه‌ریزیِ شناختی» (Cognitive Mapping and Planning)، دو مرحلهٔ نقشه‌سازی و برنامه‌ریزیِ مسیر دو مدول در شبکهٔ عصبی‌اند که به‌صورتِ سرتاسری برای کمینه‌کردنِ تابعِ زیان آموزش می‌بینند. چنین سامانه‌ای لازم نیست نقشهٔ کاملی بسازد—که اغلب زائد و غیرضروری است—اگر فقط به اطلاعاتِ کافی برای ناوبری از نقطهٔ A به نقطهٔ B بدونِ برخورد با موانع نیاز دارید.



شکل ۲۵.۲۸ – حس‌گریِ مبتنی بر دوربینِ Mobileye برای خودروهای خودمختار. ردیفِ بالا: دو تصویر از دوربینِ رو به جلو، گرفته‌شده با فاصلهٔ چند ثانیه. ناحیهٔ سبز فضایِ آزاد است—ناحیه‌ای که وسیلهٔ نقلیه می‌توانست در آیندهٔ بی‌واسطه به‌صورتِ فیزیکی به آن حرکت کند. اشیا با جعبه‌های محدودکنندهٔ سه‌بعدی نمایش داده می‌شوند که اضلاعشان را تعریف می‌کنند (قرمز برای عقب، آبی برای سمتِ راست، زرد برای سمتِ چپ و سبز برای جلو). اشیا شاملِ وسایلِ نقلیه، عابرانِ پیاده، لبهٔ درونیِ خطوطِ خطِ خود (لازم برای کنترلِ جانبی)، دیگر علامت‌های خط و خط‌کشیِ عابرِ پیاده، علائمِ راهنمایی و چراغ‌های راهنمایی‌اند. نشان‌داده‌نشده‌اند: حیوانات، تیرک‌ها و مخروط‌ها، پیاده‌روها، نرده‌ها و اشیای عمومی (مثلاً، مبلِ افتاده از پشتِ کامیون). سپس هر شیء با موقعیت و سرعتِ سه‌بعدی علامت می‌شود. ردیفِ پایین: مدلِ فیزیکیِ کاملِ محیط، رندرِشده از اشیای تشخیص‌داده‌شده. (تصاویر نتایجِ سامانهٔ فقط-بینایی Mobileye را نشان می‌دهند.) تصاویر با اجازه از Mobileye.



شکل ۲۵.۲۹ – ناوبری با تجزیه به دو مسئله حل می‌شود: نقشه‌سازی و برنامه‌ریزی. با هر گامِ زمانیِ متوالی، اطلاعاتِ حس‌گذارها برای ساختنِ افزایشیِ مدلی نامطمئن از جهان استفاده می‌شود. این مدل همراه با مشخصاتِ هدف به برنامه‌ریزی داده می‌شود که کنشِ بعدیِ ربات برای رسیدن به هدف را خروجی می‌دهد. مدل‌های جهان می‌توانند صرفاً هندسی (مانندِ SLAM کلاسیک)، یا معنایی (چنان‌که از طریقِ یادگیری به‌دست می‌آید)، یا حتی توپولوژیک (مبتنی بر نقاطِ نشانه) باشند. خودِ رباتِ واقعی در راست ظاهر می‌شود. شکل‌ها با اجازه از سوراب گوپتا.

خلاصهٔ فصل

هرچند ادراک برای انسان‌ها فعالیتی بی‌زحمت به نظر می‌رسد، به مقدارِ قابلِ توجهی محاسبهٔ پیچیده نیاز دارد. هدفِ بینایی استخراجِ اطلاعاتِ لازم برای تکلیف‌هایی مانندِ دست‌کاری، ناوبری و بازشناسیِ اشیا است.

- هندسه و اپتیکِ تشکیلِ تصویر خوب فهمیده شده. با توجه به توصیفی از صحنهٔ سه‌بعدی، می‌توانیم به‌راحتی از هر موقعیتِ دوربینِ دلبخواهی تصویری از آن تولید کنیم—این مسئلهٔ گرافیک است. مسئلهٔ معکوس، مسئلهٔ بینایی رایانه—گرفتنِ تصویر و تبدیلِ آن به توصیفِ سه‌بعدی—دشوارتر است.
- بازنمایی‌های تصاویر لبه‌ها، بافت، جریانِ نوری و ناحیه‌ها را درمی‌گیرند. این‌ها سرنخ‌هایی به مرزهای اشیا و به تناظرِ بینِ تصاویر می‌دهند.
- شبکه‌های عصبیِ پیچشی رده‌بندهای تصویرِ دقیقی تولید می‌کنند که از ویژگی‌های آموخته‌شده استفاده می‌کنند. به‌طورِ تقریبی، ویژگی‌ها الگوهایِ الگوهایِ الگوهای... هستند. پیش‌بینیِ اینکه این رده‌بندها چه‌وقت خوب کار خواهند کرد سخت است، زیرا دادهٔ آزمون ممکن است به‌شکلی مهم با دادهٔ آموزشی متفاوت باشد. تجربه می‌آموزد اغلب به‌اندازهٔ کافی دقیق‌اند که در عمل استفاده شوند.
- رده‌بندهای تصویر را می‌توان به آشکارسازهای شیء تبدیل کرد. یک رده‌بند جعبه‌های تصویر را برای شیءبودگی امتیاز می‌دهد؛ رده‌بندِ دیگر سپس تصمیم می‌گیرد آیا شیئی در جعبه هست، و آن شیء چیست. روش‌های تشخیصِ شیء کامل نیستند، اما برای طیفِ وسیعی از کاربردها قابلِ استفاده‌اند.
- با بیش از یک نمایِ صحنه، امکانِ بازیابیِ ساختارِ سه‌بعدیِ صحنه و رابطهٔ بینِ نماها هست. در بسیاری موارد، امکانِ بازیابیِ هندسهٔ سه‌بعدی از یک نمایِ تکی هست.
- روش‌های بینایی رایانه‌ای بسیار گسترده اعمال می‌شوند.

یادداشت‌های کتابشناختی و تاریخی

این فصل بر بینایی تمرکز کرده، اما کانال‌های ادراکیِ دیگر نیز مطالعه و در رباتیک به‌کار رفته‌اند. برای ادراکِ شنوایی (شنیدن)، قبلاً بازشناسیِ گفتار را پوشش داده‌ایم، و کارِ قابلِ توجهی نیز بر ادراکِ موسیقی (کولش و زیبل، 2005) و یادگیریِ ماشینیِ موسیقی (انگل و همکاران، 2017) و همچنین بر یادگیریِ ماشینی برای صداها به‌طورِ کلی (شاران و موير، 2016) شده.

ادراکِ لامسه یا لمس (لو و همکاران، 2017) در رباتیک مهم است و در فصل ۲۶ بحث می‌شود. ادراکِ بوییِ خودکار (بو) کارِ کمتری دیده، اما نشان داده شده مدل‌های یادگیری عمیق می‌توانند یاد بگیرند بوها را بر مبنای ساختارِ مولکول‌ها پیش‌بینی کنند (سانچز-لنگلینگ و همکاران، 2019).

تلاش‌های نظام‌مند برای فهمِ بینواییِ انسانی به دورانِ باستان بازمی‌گردد. اقلیدس (حدودِ ۳۰۰ پ.م.) دربارهٔ پرسپکتیوِ طبیعی نوشت—نگاشتی که با هر نقطهٔ P در جهانِ سه‌بعدی، جهتِ پرتویِ OP پیونددهندهٔ مرکزِ فرافکنیِ O به نقطهٔ P را مرتبط می‌کند. او به‌خوبی از مفهومِ اختلافِ منظرِ حرکتی آگاه بود. نقاشی‌های رومیِ باستان، مانندِ آن‌هایی که توسط فورانِ وزوویوس در ۷۹ میلادی حفظ شده‌اند، از نوعی پرسپکتیوِ غیررسمی، با بیش از یک خطِ افق، استفاده می‌کردند.

فهمِ ریاضیِ فرافکنیِ پرسپکتیوی، این بار در بافتِ فرافکنی رویِ سطوحِ مسطح، پیشرفتِ چشمگیرِ بعدی‌اش را در قرنِ پانزدهم در رنسانسِ ایتالیا داشت. برونلسکی معمولاً به‌عنوانِ خالقِ نخستین نقاشی‌های مبتنی بر فرافکنیِ هندسیِ درستِ صحنهٔ سه‌بعدی در حدودِ ۱۴۱۳ شناخته می‌شود. در ۱۴۳۵، آلبرتی قاعده‌ها را مدون کرد و نسل‌هایی از هنرمندان را الهام بخشید. لئوناردو داوینچی و آلبرشت دورر در توسعهٔ علمِ پرسپکتیو، چنان‌که در آن روزگار نامیده می‌شد، به‌ویژه قابلِ توجه‌اند. توصیف‌های لئوناردو در اواخرِ قرنِ پانزدهم از تعاملِ نور و سایه (کیاروسکورو)، نواحیِ سایهٔ کامل (umbra) و نیم‌سایه (penumbra)، و پرسپکتیوِ هوایی هنوز در ترجمه خواندنی‌اند (کمپ، 1989).

هرچند پرسپکتیو برای یونانیان شناخته‌شده بود، به‌شکلِ عجیبی دربارهٔ نقشِ چشم‌ها در بینایی گیج بودند. ارسطو فکر می‌کرد چشم‌ها دستگاه‌هایی‌اند که پرتو نشر می‌کنند، تا حدی مانندِ فاصله‌یاب‌های لیزریِ مدرن. این دیدگاهِ اشتباه توسط کارِ دانشمندانِ عرب، مانندِ ابن‌هیثم (الحازن)، در قرنِ دهم از میان برداشته شد.

توسعهٔ انواعِ گوناگونِ دوربین دنبال شد. این‌ها شاملِ اتاق‌هایی بودند (camera در لاتین به معنای «اتاق» است) که نور از سوراخِ کوچکی در یک دیوار وارد می‌شد تا تصویری از صحنهٔ بیرون رویِ دیوارِ مقابل بیندازد. البته، در همهٔ این دوربین‌ها، تصویر وارونه بود، که سردرگمیِ بی‌پایانی ایجاد می‌کرد. اگر قرار بود چشم به‌عنوانِ چنین دستگاهِ تصویربرداری‌ای در نظر گرفته شود، چگونه راست‌به‌بالا می‌بینیم؟ این معما بزرگ‌ترین ذهن‌های آن دوران (از جملهِ لئوناردو) را به خود مشغول کرد. کارِ کپلر و دکارت برای حلِ پرسش لازم بود. دکارت چشمی را که کوتیکولِ ماتش برداشته شده بود در سوراخی از کرکرهٔ پنجره قرار داد. نتیجه تصویری وارونه بود که رویِ کاغذی که رویِ شبکیه گذاشته شده بود تشکیل می‌شد. هرچند تصویرِ شبکیه‌ای در واقع وارونه است، این مشکلی ایجاد نمی‌کند، زیرا مغز تصویر را درست تفسیر می‌کند. در اصطلاحِ مدرن، فقط باید به ساختارِ داده به‌طورِ مناسب دسترسی داشت.

پیشرفت‌های بزرگِ بعدی در فهمِ بینایی در قرنِ نوزدهم رخ داد. کارِ هلمهولتز و وونت، که در فصل ۱ توصیف شد، آزمایشِ روان‌فیزیکی را به‌عنوانِ رشتهٔ علمیِ سخت‌گیرانه‌ای برقرار کرد. از طریقِ کارِ یانگ، ماکسول و هلمهولتز، نظریهٔ سه‌رنگیِ بینواییِ رنگی برقرار شد. این واقعیت که انسان‌ها می‌توانند عمق ببینند اگر تصاویرِ ارائه‌شده به چشمِ چپ و راست کمی متفاوت باشند توسط اختراعِ استریوسکوپِ ویتستون (1838) نشان داده شد. این دستگاه بلافاصله در اتاق‌های نشیمن و سالن‌های سراسرِ اروپا محبوب شد.

مفهومِ ضروریِ استریوپسیسِ دوچشمی—که دو تصویرِ صحنه‌ای که از دیدگاه‌های کمی متفاوت گرفته شده‌اند اطلاعاتِ کافی برای به‌دست‌آوردنِ بازسازیِ سه‌بعدیِ صحنه حمل می‌کنند—در حوزهٔ فتوگرامتری بهره‌برداری شد. نتایجِ ریاضیِ کلیدی به‌دست آمد؛ برای نمونه، کروپا (1913) اثبات کرد با توجه به دو نمایِ پنج نقطهٔ متمایزِ صحنه، می‌توان چرخش و انتقالِ بینِ دو موقعیتِ دوربین و همچنین عمقِ صحنه را (تا حدِّ عاملِ مقیاس) بازسازی کرد.

هرچند هندسهٔ استریوپسیس مدتِ طولانی فهمیده شده بود، مسئلهٔ تناظر در فتوگرامتری قبلاً توسط انسان‌هایی حل می‌شد که می‌کوشیدند نقطه‌های متناظر را تطبیق دهند. تواناییِ شگفت‌انگیزِ انسان‌ها در حلِ مسئلهٔ تناظر توسط استریوگرام‌های نقطهٔ تصادفیِ ژولز (1971) نشان داده شد. حوزهٔ بینواییِ رایانه تلاشِ زیادی به راه‌حلِ خودکارِ مسئلهٔ تناظر اختصاص داده.

در نیمهٔ نخستِ قرنِ بیستم، مهم‌ترین نتایجِ پژوهشیِ بینایی توسط مکتبِ گشتالتِ روان‌شناسی، به رهبریِ ماکس ورتایمر، به‌دست آمد. آن‌ها به اهمیتِ سازمان‌دهیِ ادراکی اشاره کردند: برای انسان، تصویر مجموعه‌ای از خروجی‌های گیرندهٔ نوریِ نقطه‌ای (پیکسل) نیست، بلکه به گروه‌های منسجم سازمان‌یافته است. تکلیفِ بینواییِ رایانه‌ایِ یافتنِ ناحیه‌ها و منحنی‌ها به این بینش بازمی‌گردد. گشتالتی‌ها همچنین توجه را به پدیدهٔ «شکل-زمینه» جلب کردند— کانتوری که دو ناحیهٔ تصویر را که در جهان در عمق‌های متفاوت‌اند جدا می‌کند به نظر می‌رسد فقط به ناحیهٔ نزدیک‌تر، «شکل»، تعلق دارد، نه به ناحیهٔ دورتر، «زمینه».

کارِ گشتالت توسط جی. جی. گیبسون (1950, 1979) ادامه یافت، که به اهمیتِ جریانِ نوری و گرادیان‌های بافت در برآوردِ متغیرهای محیطی مانندِ شیب و میلِ سطح اشاره کرد. او دوباره بر اهمیتِ محرک و غنایِ آن تأکید کرد. گیبسون، اولوم و روزن‌بلات (1955) اشاره کردند میدانِ جریانِ نوری اطلاعاتِ کافی برای تعیینِ حرکتِ ناظر نسبت به محیط دارد. گیبسون به‌ویژه بر نقشِ ناظرِ فعال تأکید کرد، که حرکتِ خودهدایت‌شده‌اش برداشتِ اطلاعات دربارهٔ محیطِ خارجی را تسهیل می‌کند.

بینواییِ رایانه به دههٔ ۱۹۶۰ بازمی‌گردد. پایان‌نامهٔ رابرتز (1963) در MIT دربارهٔ ادراکِ مکعب‌ها و دیگر اشیای جهانِ بلوک‌ها یکی از نخستین انتشاراتِ حوزه بود. رابرتز چند ایدهٔ کلیدی معرفی کرد، از جملهِ تشخیصِ لبه و تطبیقِ مبتنی بر مدل.

در دهه‌های ۱۹۶۰ و ۱۹۷۰ پیشرفت کند بود، که توسط کمبودِ منابعِ محاسباتی و ذخیره‌سازی مختل می‌شد. پردازشِ بصریِ سطح‌پایین توجهِ زیادی دریافت کرد، با فنونِ برگرفته از حوزه‌های مرتبط مانندِ پردازشِ سیگنال، بازشناسیِ الگو و خوشه‌بندیِ داده. تشخیصِ لبه به‌عنوانِ گامِ ضروریِ نخستِ پردازشِ تصویر تلقی می‌شد، زیرا مقدارِ دادهٔ پردازشی را کاهش می‌داد. فنِ پرکاربردِ تشخیصِ لبهٔ کنی توسط جان کنی (1986) معرفی شد. مارتین، فاولکس و مالیک (2004) نشان دادند چگونه چند سرنخ، مانندِ روشنایی، بافت و رنگ، را در چارچوبِ یادگیریِ ماشینی برای یافتنِ بهترِ منحنی‌های مرز ترکیب کنیم. مسئلهٔ نزدیکِ مرتبطِ یافتنِ ناحیه‌های روشنایی، رنگ و بافتِ منسجم به‌طورِ طبیعی به فرمول‌بندی‌هایی وام می‌دهد که در آن‌ها یافتنِ بهترین افراز به مسئلهٔ بهینه‌سازی تبدیل می‌شود. سه نمونهٔ پیشرو مبتنی بر میدان‌های تصادفیِ مارکوف به‌دلیلِ گمان و گمان (1984)، فرمول‌بندیِ تغییراتیِ مامفورد و شاه (1989)، و برش‌های نرمال‌شده توسط شی و مالیک (2000) هستند.

در بخشِ بزرگی از دهه‌های ۱۹۶۰، ۱۹۷۰ و ۱۹۸۰، دو پارادایمِ متمایز وجود داشت که در آن‌ها بازشناسیِ بصری دنبال می‌شد، که توسط دیدگاه‌های متفاوت دربارهٔ آنچه مسئلهٔ اصلی تلقی می‌شد دیکته می‌شد. پژوهشِ بینواییِ رایانه‌ای دربارهٔ بازشناسیِ اشیا عمدتاً بر مسائلِ ناشی از فرافکنیِ اشیای سه‌بعدی به تصاویرِ دوبعدی تمرکز می‌کرد. ایدهٔ هم‌ترازی، که همچنین نخستین بار توسط رابرتز معرفی شد، در دههٔ ۱۹۸۰ در کارِ لو (1987) و هوتن‌لوخر و اولمن (1990) دوباره ظاهر شد.

جامعهٔ بازشناسیِ الگو رویکردِ متفاوتی گرفت و جنبه‌های 3D-به-2D مسئله را بی‌اهمیت می‌دید. مثال‌های انگیزشیِ آن‌ها در حوزه‌هایی مانندِ بازشناسیِ نوریِ نویسه‌ها و بازشناسیِ کدِ پستیِ دست‌نویس بود، که نگرانیِ اصلی در آن‌ها یادگیریِ تغییرهای معمولِ مشخصهٔ رده‌ای از اشیا و جداسازیِ آن‌ها از رده‌های دیگر است. می‌توانیم معماری‌های شبکهٔ عصبیِ تحلیلِ تصویر را به مطالعاتِ هیوبل و ویزل (1962, 1968) از قشرِ بینایی در گربه‌ها و میمون‌ها بازجست. آن‌ها مدلِ سلسله‌مراتبیِ مسیرِ بینایی توسعه دادند با نورون‌هایی در نواحیِ پایین‌ترِ مغز (به‌ویژه ناحیه‌ای به نامِ V1) که به ویژگی‌هایی مانندِ لبه‌ها و میله‌های جهت‌دار پاسخ می‌دهند، و نورون‌هایی در نواحیِ بالاتر که به محرک‌های خاص‌تر پاسخ می‌دهند («سلول‌های مادربزرگ» در نسخهٔ کارتونی).

فوکوشیما (1980) معماریِ شبکهٔ عصبیِ بازشناسیِ الگو را پیشنهاد کرد که صریحاً توسط سلسله‌مراتبِ هیوبل و ویزل انگیخته شده بود. مدلِ او لایه‌های متناوبِ سلول‌های ساده و پیچیده داشت، که بدین‌ترتیب پایین‌نمونه‌برداری را دربرمی‌گرفت، و همچنین ناوردایی جابه‌جایی داشت، که بدین‌ترتیب ساختارِ پیچشی را دربرمی‌گرفت. لکون و همکاران (1989) گامِ اضافیِ استفاده از پسانتشار برای آموزشِ وزن‌های این شبکه را برداشتند، و آنچه امروز شبکه‌های عصبیِ پیچشی می‌نامیم زاده شد. برای مقایسهٔ رویکردها، لکون و همکاران (1995) را ببینید.

از اواخرِ دههٔ ۱۹۹۰، همراه با نقشِ بسیار بزرگ‌ترِ مدل‌سازیِ احتمالاتی و یادگیریِ ماشینیِ آماری در حوزهٔ هوش مصنوعی به‌طورِ کلی، نزدیکیِ بینِ این دو سنت رخ داد. دو خطِ کار سهمِ چشمگیری داشتند. یکی پژوهشِ تشخیصِ چهره (راولی و همکاران، 1998؛ ویولا و جونز، 2004) بود که قدرتِ فنونِ بازشناسیِ الگو را در تکلیف‌های به‌وضوح مهم و مفید نشان داد. دیگری توسعهٔ توصیف‌گرهای نقطه‌ای بود، که ساختِ بردارهای ویژگی از بخش‌های اشیا را ممکن می‌کنند (اشمید و مور، 1996). سه راهبردِ کلیدی برای ساختنِ توصیف‌گرِ نقطهٔ محلیِ خوب هست: یکی استفاده از جهت‌ها برای گرفتنِ ناوردایی روشنایی است؛ یکی نیاز به توصیفِ ساختارِ تصویرِ نزدیکِ نقطه با جزئیات، و دورتر فقط تقریبی است؛ و یکی نیاز به استفاده از هیستوگرام‌های فضایی برای سرکوبِ تغییرهای ناشی از خطاهای کوچکِ مکان‌یابیِ نقطه است. توصیف‌گرِ SIFT لو (2004) از این ایده‌ها بسیار مؤثر بهره برد؛ گونهٔ محبوبِ دیگر توصیف‌گرِ HOG به‌دلیلِ دالال و تریگز (2005) بود.

دهه‌های ۱۹۹۰ و ۲۰۰۰ بحثِ ادامه‌داری بینِ طرفدارانِ طراحیِ ویژگیِ هوشمندانه مانندِ SIFT و HOG و شیفتگانِ شبکه‌های عصبی که باور داشتند ویژگی‌های خوب باید به‌صورتِ خودکار از آموزشِ سرتاسری پدید آیند دید. راهِ حلِ چنین بحثی از طریقِ معیارها رویِ مجموعه‌های دادهٔ استاندارد است، و در دههٔ ۲۰۰۰ نتایجِ رویِ مجموعهٔ دادهٔ استانداردِ تشخیصِ شیء، PASCAL VOC، به نفعِ ویژگی‌های دست‌طراحی‌شده استدلال کرد. این وقتی تغییر کرد که کریژفسکی و همکاران (2013) نشان دادند در تکلیفِ رده‌بندیِ تصویر رویِ مجموعهٔ دادهٔ ImageNet، شبکهٔ عصبی‌شان (به نامِ AlexNet) نرخ‌های خطایِ به‌شکلِ چشمگیری پایین‌تر از فنونِ اصلیِ بینواییِ رایانه می‌دهد.

سسِ مخفیِ پشتِ موفقیتِ AlexNet چه بود؟ علاوه بر نوآوری‌های فنی (مانندِ استفاده از واحدهای فعال‌سازیِ ReLU)، باید اعتبارِ زیادی به دادهٔ بزرگ و محاسبهٔ بزرگ بدهیم. با «دادهٔ بزرگ» منظورِ در دسترس‌بودنِ مجموعه‌های دادهٔ بزرگِ با برچسبِ رده است، مانندِ ImageNet، که دادهٔ آموزشیِ این شبکه‌های بزرگ و عمیقِ با میلیون‌ها پارامتر را فراهم کرد. مجموعه‌های دادهٔ قبلی مانندِ Caltech-101 یا PASCAL VOC دادهٔ آموزشیِ کافی نداشتند، و MNIST و CIFAR توسط جامعهٔ بینواییِ رایانه «مجموعه‌های دادهٔ اسباب‌بازی» تلقی می‌شدند. این رشتهٔ برچسب‌زدنِ

مجموعه‌های داده برای معیارگذاری و برای استخراجِ آمارِ تصویر خود توسط میلِ مردم به آپلودِ مجموعه‌عکس‌هایشان در اینترنت در سایت‌هایی مانندِ فلیکر ممکن شد. راهی که محاسبهٔ بزرگ مفیدترین ثابت شد از طریقِ GPUها بود، توسعهٔ سخت‌افزاری که در ابتدا توسط نیازهای صنعتِ بازیِ ویدئویی رانده شد.

در عرضِ یک یا دو سال، شواهد کاملاً روشن بود. برای نمونه، کارِ شبکهٔ عصبیِ پیچشیِ مبتنی بر ناحیه (RCNN) توسط گیرشیک و همکاران (2016) نشان داد معماریِ AlexNet را می‌توان اصلاح کرد، با استفاده از ایده‌های بینواییِ رایانه‌ای مانندِ پیشنهادِ ناحیه، تا تشخیصِ شیءِ پیشتاز رویِ PASCAL VOC ممکن شود. دریافته‌ایم به‌طورِ کلی شبکه‌های عمیق‌تر بهتر کار می‌کنند و ترس‌های بیش‌برازش اغراق‌آمیزند. فنونِ تازه‌ای مانندِ نرمال‌سازیِ دسته‌ای برای برخورد با منظم‌سازی داریم.

بازسازیِ ساختارِ سه‌بعدی از نماهای متعدد ریشه‌هایش در ادبیاتِ فتوگرامتری دارد. در دورانِ بینواییِ رایانه، اولمن (1979) و لونگوئه-هیگینز (1981) کارهای اولیهٔ تأثیرگذارند. نگرانی‌ها دربارهٔ پایداریِ ساختار-از-حرکت به‌شکلِ چشمگیری توسط کارِ توماسی و کاناد (1992) کاهش یافت، که نشان دادند با استفاده از فریم‌های متعدد و خطِ مبنایِ وسیعِ حاصل، شکل را می‌توان کاملاً دقیق بازیابی کرد. نوآوریِ مفهومیِ معرفی‌شده در دههٔ ۱۹۹۰ مطالعهٔ ساختارِ فرافکنی از حرکت بود. در این‌جا کالیبراسیونِ دوربین لازم نیست، چنان‌که فوژرا (1992) نشان داد. این کشف با معرفیِ استفاده از ناورداهای هندسی در بازشناسیِ اشیا، چنان‌که ماندی و زیسرمن (1992) مرور کرده‌اند، و توسعهٔ ساختارِ وابسته از حرکت توسط کوندرینک و ون دورن (1991) مرتبط است.

در دههٔ ۱۹۹۰، با افزایشِ زیادِ سرعت و ذخیرهٔ رایانه و در دسترس‌بودنِ گستردهٔ ویدئویِ دیجیتال، تحلیلِ حرکت کاربردهای تازهٔ بسیاری یافت. ساختنِ مدل‌های هندسیِ صحنه‌های جهانِ واقعی برای رندر با فنونِ گرافیکِ رایانه‌ای به‌ویژه محبوب ثابت شد، که توسط الگوریتم‌های بازسازی مانندِ توسعه‌داده‌شده توسط دبِوک و همکاران (1996) رهبری می‌شد. کتاب‌های هارتلی و زیسرمن (2000) و فوژرا و همکاران (2001) رسیدگیِ جامعی به هندسهٔ نماهای متعدد فراهم می‌کنند.

انسان‌ها می‌توانند شکل و چیدمانِ فضایی را از یک تصویرِ تکی ادراک کنند، و مدل‌کردنِ این برای پژوهشگرانِ بینواییِ رایانه چالشِ بزرگی ثابت شده. استنتاجِ شکل از سایه‌روشن نخستین بار توسط برتولد هورن (1970) مطالعه شد، و هورن و بروکس (1989) مرورِ گسترده‌ای از مقاله‌های اصلیِ دوره‌ای ارائه می‌کنند که این مسئله در آن بسیار مطالعه می‌شد.

گیبسون (1950) نخستین کسی بود که گرادیان‌های بافت را به‌عنوانِ سرنخی برای شکل پیشنهاد کرد. ریاضیاتِ کانتورهای مسدودکننده، و به‌طورِ کلی‌تر فهمِ رویدادهای بصری در فرافکنیِ اشیای منحنیِ هموار، مدیونِ کارِ کوندرینک و ون دورن است، که رسیدگیِ گسترده‌ای در کتابِ «شکلِ جامد» کوندرینک (1990) می‌یابد.

اخیراً، توجه به درمانِ مسئلهٔ بازیابیِ شکل و سطح از یک تصویرِ تکی به‌عنوانِ مسئلهٔ استنتاجِ احتمالاتی معطوف شده، که در آن سرنخ‌های هندسی به‌صورتِ صریح مدل نمی‌شوند، بلکه به‌صورتِ ضمنی در چارچوبِ یادگیری استفاده می‌شوند. مثالِ خوبِ آن کارِ هویم و همکاران (2007) است؛ اخیراً این با شبکه‌های عصبیِ عمیق بازکاری شده.

حالا به کاربردهای بیناییِ رایانه برای هدایتِ کنش بپردازیم، دیکمنس و زاپ (1987) نخستین بار خودرویِ خودرانی را نشان دادند که با سرعت‌های بالا در آزادراه‌ها رانندگی می‌کرد؛ پومرلو (1993) با رویکردِ شبکهٔ عصبی به عملکردِ مشابهی رسید. امروزه ساختنِ خودروهای خودران کسب‌وکاری بزرگ است، با شرکت‌های خودروسازیِ تثبیت‌شده که با تازه‌واردهایی مانندِ بایدو، کروز، دیدی، گوگلِ ویمو، لیفت، موبایل‌آی، نورو، انویدیا، سامسونگ، تاتا، تسلا، اوبر و وویاژ رقابت می‌کنند تا سامانه‌هایی به بازار عرضه کنند که قابلیت‌هایی از کمک‌راننده تا خودمختاریِ کامل را فراهم می‌کنند.

برای خوانندهٔ علاقه‌مند به بینایی انسانی، «علمِ بینایی: از فوتون‌ها تا پدیدارشناسی» از استیون پالمر (1999) بهترین رسیدگیِ جامع را فراهم می‌کند؛ «ادراکِ بصری: فیزیولوژی، روان‌شناسی و بوم‌شناسی» از ویکی بروس، پاتریک گرین و مارک جورجسون (2003) کتابِ درسیِ کوتاه‌تری است. کتاب‌های «چشم، مغز و بینایی» از دیوید هیوبل (1988) و «ادراک» از اروین راک (1984) مقدمه‌های دوستانه‌ای‌اند که به‌ترتیب بر عصب‌فیزیولوژی و ادراک متمرکزند. کتابِ «بینایی» دیوید مار (مار، 1982) نقشِ تاریخی در پیوندِ بیناییِ رایانه به حوزه‌های سنتیِ بیناییِ زیستی—روان‌فیزیک و عصب‌زیست‌شناسی—ایفا کرد. هرچند بسیاری از مدل‌های خاصِ او برای تکلیف‌هایی مانندِ تشخیصِ لبه و بازشناسیِ اشیا آزمونِ زمان را تاب نیاورده‌اند، چشم‌اندازِ نظریِ او که در آن هر تکلیف در سطحِ اطلاعاتی، محاسباتی و پیاده‌سازی تحلیل می‌شود همچنان روشنگر است.

برای حوزهٔ بیناییِ رایانه‌ای، جامع‌ترین کتاب‌های درسیِ در دسترسِ امروز «بیناییِ رایانه‌ای: رویکردی مدرن» (فورسایت و پونس، 2002) و «بیناییِ رایانه‌ای: الگوریتم‌ها و کاربردها» (زلیسکی، 2011) هستند. مسائلِ هندسی در بیناییِ رایانه‌ای به‌طورِ کامل در «هندسهٔ چندنمایی در بیناییِ رایانه‌ای» (هارتلی و زیسرمن، 2000) بررسی شده‌اند. این کتاب‌ها پیش از انقلابِ یادگیریِ عمیق نوشته شده‌اند، بنابراین برای تازه‌ترین نتایج، به مقاله‌ها و ادبیاتِ پژوهشیِ اصلی مراجعه کنید.

دو تا از ژورنال‌های اصلی برای بیناییِ رایانه‌ای عبارت‌اند از *IEEE Transactions on Pattern Analysis and Machine Intelligence* و *International Journal of Computer Vision*. کنفرانس‌های بیناییِ رایانه‌ای شاملِ ICCV (کنفرانسِ بین‌المللیِ بیناییِ رایانه‌ای)، CVPR (بیناییِ رایانه‌ای و بازشناسیِ الگو)، و ECCV (کنفرانسِ اروپاییِ بیناییِ رایانه‌ای) هستند. پژوهش‌هایی که مؤلفهٔ یادگیریِ ماشینیِ چشمگیری دارند نیز در NeurIPS (سامانه‌های پردازشِ اطلاعاتِ عصبی) منتشر می‌شوند، و کارهای مرتبط با فصلِ مشترک با گرافیکِ رایانه‌ای اغلب در کنفرانسِ ACM SIGGRAPH (گروهِ علایقِ ویژه در گرافیک) ظاهر می‌شوند. بسیاری از مقاله‌های بینایی به‌صورتِ پیش‌چاپ روی سرورِ arXiv قرار می‌گیرند، و گزارش‌های اولیه از نتایجِ تازه در وبلاگ‌های آزمایشگاه‌های پژوهشیِ بزرگ منتشر می‌شوند.

پانوشت‌ها:

[1] اصطلاحِ «جعبه» (Box) را برای هر ناحیهٔ مستطیلیِ هم‌راستا با محورِ تصویر استفاده می‌کنیم، و اصطلاحِ «پنجره» (Window) را بیشتر به‌عنوانِ مترادفِ «جعبه»، اما با این مفهوم ضمنی که پنجره‌ای به رویِ ورودی داریم که امیدواریم چیزی در آن ببینیم، و جعبهٔ محدودکننده‌ای در خروجی وقتی آن را یافته‌ایم.

فصل ۲۶: رباتیک (Robotics)

در این فصل عامل‌ها به حس‌گذارها و اثرگذارهای فیزیکی مجهز می‌شوند تا در جهانِ واقعی حرکت کنند و خرابکاری کنند.

۲۶.۱ ربات‌ها

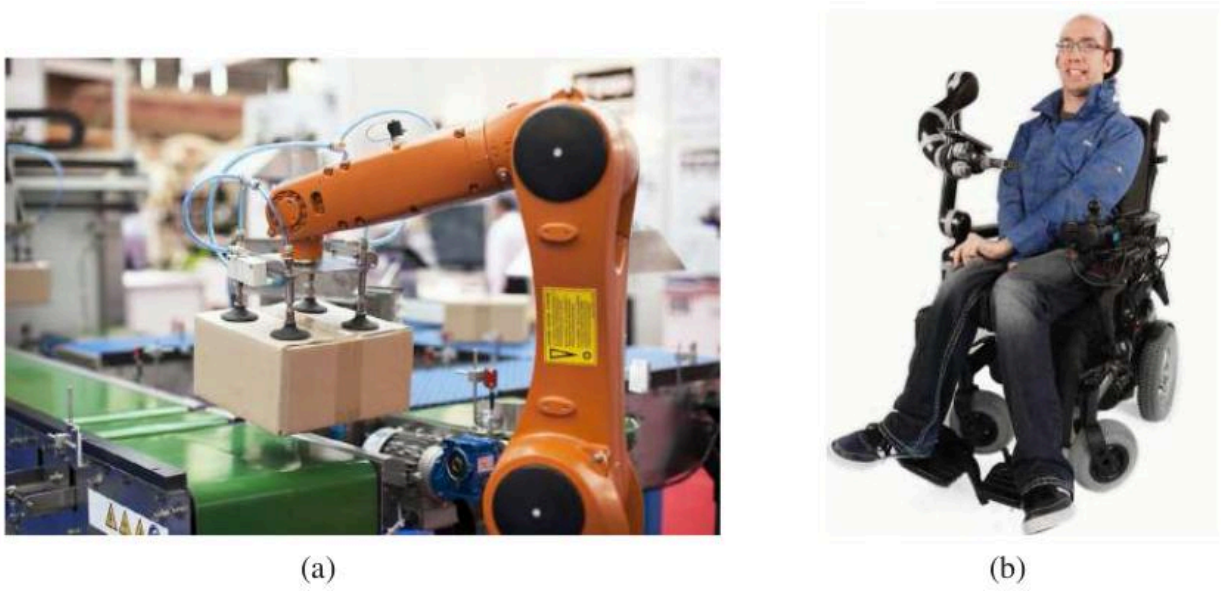
«ربات»ها (Robot) عامل‌های فیزیکی‌اند که با دست‌کاریِ جهانِ فیزیکی تکلیف‌ها را انجام می‌دهند. برای این کار، آن‌ها به «اثرگذار»هایی (Effector) مانندِ پاها، چرخ‌ها، مفصل‌ها و گیره‌ها مجهزند. اثرگذارها برای اعمالِ نیروهای فیزیکی بر محیط طراحی شده‌اند. وقتی این کار را می‌کنند، چند چیز ممکن است رخ دهد: حالتِ ربات ممکن است تغییر کند (مثلاً، خودرویی چرخ‌هایش را می‌چرخاند و در نتیجه رویِ جاده پیشرفت می‌کند)، حالتِ محیط ممکن است تغییر کند (مثلاً، بازویِ رباتی با گیره‌اش لیوانی را رویِ پیش‌خوان هل می‌دهد)، و حتی حالتِ مردمِ اطرافِ ربات ممکن است تغییر کند (مثلاً، اسکلتِ بیرونی حرکت می‌کند و پیکربندیِ پایِ شخص را تغییر می‌دهد؛ یا رباتِ متحرکی به سمتِ درهای آسانسور پیشرفت می‌کند، و شخصی متوجه می‌شود و لطف می‌کند کنار می‌رود، یا حتی دکمه را برای ربات فشار می‌دهد).

ربات‌ها همچنین به «حس‌گذار»هایی (Sensor) مجهزند که آن‌ها را قادر به ادراکِ محیط‌شان می‌کند. رباتیکِ امروزی مجموعهٔ متنوعی از حس‌گذارها به‌کار می‌برد، از جملهِ دوربین‌ها، رادارها، لیزرها و میکروفون‌ها برای اندازه‌گیریِ حالتِ محیط و مردمِ اطرافِ آن؛ و ژیروسکوپ‌ها، حس‌گذارهای کرنش و گشتاور، و شتاب‌سنج‌ها برای اندازه‌گیریِ حالتِ خودِ ربات.

بیشینه‌کردنِ مطلوبیتِ موردانتظار برای ربات به معنای انتخابِ نحوهٔ به‌کارانداختنِ اثرگذارهایش برای اعمالِ نیروهای فیزیکیِ درست است—نیروهایی که به تغییرهایی در حالت می‌انجامند که تا حدِ ممکن پاداشِ موردانتظارِ بیشتری انباشته می‌کنند. در نهایت، ربات‌ها در تلاش‌اند تکلیفی را در جهانِ فیزیکی انجام دهند. ربات‌ها در محیط‌هایی عمل می‌کنند که با مشاهدهٔ جزئی و تصادفی‌اند: دوربین‌ها نمی‌توانند دورِ گوشه‌ها را ببینند، و چرخ‌دنده‌ها می‌توانند بلغزند. افزون بر این، افرادی که در همان محیط عمل می‌کنند غیرقابلِ‌پیش‌بینی‌اند، پس ربات باید دربارهٔ آن‌ها پیش‌بینی کند.

ربات‌ها معمولاً محیط‌شان را با فضایِ حالتِ پیوسته (موقعیتِ ربات مختصاتِ پیوسته دارد) و فضایِ کنشِ پیوسته (مقدارِ جریانِ الکتریکی که ربات به موتورش می‌فرستد نیز در واحدهای پیوسته اندازه‌گیری می‌شود) مدل می‌کنند. برخی ربات‌ها در فضاهای پربعد عمل می‌کنند: خودروها باید موقعیت، جهت و سرعتِ خودشان و عامل‌های نزدیک را بدانند؛ بازوهای رباتیک شش یا هفت مفصل دارند که هر یک می‌تواند به‌طورِ مستقل حرکت کند؛ و ربات‌هایی که بدنِ انسان را تقلید می‌کنند صدها مفصل دارند.

یادگیریِ رباتیک مقید است، زیرا جهانِ واقعی سرسختانه از اجرا سریع‌تر از زمانِ واقعی امتناع می‌کند. در محیطِ شبیه‌سازی‌شده، امکانِ استفاده از الگوریتم‌های یادگیری (مانندِ الگوریتمِ کیو-یادگیریِ توصیف‌شده در فصل ۲۲) برای یادگیری در چند ساعت از میلیون‌ها کوشش هست. در محیطِ واقعی، اجرایِ این کوشش‌ها ممکن است سال‌ها طول بکشد، و ربات نمی‌تواند کوششی را که ممکن است آسیب برساند به خطر بیندازد (و بنابراین از آن یاد بگیرد). پس، انتقالِ آنچه در شبیه‌سازی آموخته شده به رباتِ واقعی در جهانِ واقعی—مسئلهٔ «شبیه-به-واقعیت» (Sim-to-Real)—حوزهٔ پژوهشیِ فعالی است. سامانه‌های رباتیکِ عملی باید دانشِ قبلی دربارهٔ ربات، محیطِ فیزیکی و تکلیف‌هایی که باید انجام شوند را دربرگیرند تا ربات بتواند سریع یاد بگیرد و ایمن عمل کند.



(a) (b)

شکل ۲۶.۱ — (الف) بازویِ رباتیکِ صنعتی با اثرگذارِ انتهاییِ سفارشی. اعتبار تصویر: Macor/123RF. (ب) بازویِ رباتِ کمکیِ JACO ®Kinova® نصب‌شده رویِ ویلچر. Kinova و JACO علامت‌های تجاریِ Kinova, Inc. هستند.

رباتیک بسیاری از مفاهیمِ دیده‌شده در این کتاب را گرد هم می‌آورد، از جملهِ تخمینِ حالتِ احتمالاتی، ادراک، برنامه‌ریزی، یادگیریِ نظارت‌نشده، یادگیریِ تقویتی و نظریهٔ بازی. برای برخی از این مفاهیم، رباتیک به‌عنوانِ کاربردِ نمونهٔ چالش‌برانگیزی خدمت می‌کند. برای مفاهیمِ دیگر، این فصل زمینِ تازه‌ای می‌شکافد، برای نمونه در معرفیِ نسخهٔ پیوستهٔ فنونی که قبلاً فقط در حالتِ گسسته دیده‌ایم.

۲۶.۲ سخت‌افزارِ ربات

تاکنون در این کتاب، معماریِ عامل—حس‌گذارها، اثرگذارها و پردازنده‌ها—را به‌عنوانِ داده‌شده در نظر گرفته و بر برنامهٔ عامل تمرکز کرده‌ایم. اما موفقیتِ ربات‌های واقعی دستِ‌کم به همان اندازه به طراحیِ حس‌گذارها و اثرگذارهای مناسبِ تکلیف بستگی دارد.

۲۶.۲.۱ انواعِ ربات‌ها از دیدگاهِ سخت‌افزاری

وقتی به ربات فکر می‌کنید، ممکن است چیزی با سر و دو بازو تصور کنید که رویِ پاها یا چرخ‌ها حرکت می‌کند. چنین «ربات‌های انسان‌نما»یی (Anthropomorphic Robot) در داستان‌های تخیلی مانندِ فیلمِ «نابودگر» و کارتونِ «جتسون‌ها» رایج شده‌اند. اما ربات‌های واقعی در شکل‌ها و اندازه‌های بسیاری می‌آیند. «دستکار»ها (Manipulator) فقط بازوهای رباتیک‌اند. لزومی ندارد به بدنهٔ ربات متصل باشند؛ ممکن است صرفاً رویِ میز یا زمین پیچ شوند، چنان‌که در کارخانه‌ها (شکل ۲۶.۱ (الف)). برخی بارِ مفیدِ بزرگی دارند، مانندِ آن‌هایی که خودرو مونتاژ می‌کنند، در حالی که برخی دیگر، مانندِ بازوهای قابلِ‌نصب‌روی-ویلچر که به افرادِ دارایِ اختلالِ حرکتی کمک می‌کنند (شکل ۲۶.۱ (ب))، می‌توانند کمتر حمل کنند اما در محیط‌های انسانی ایمن‌ترند.

(a) (b)

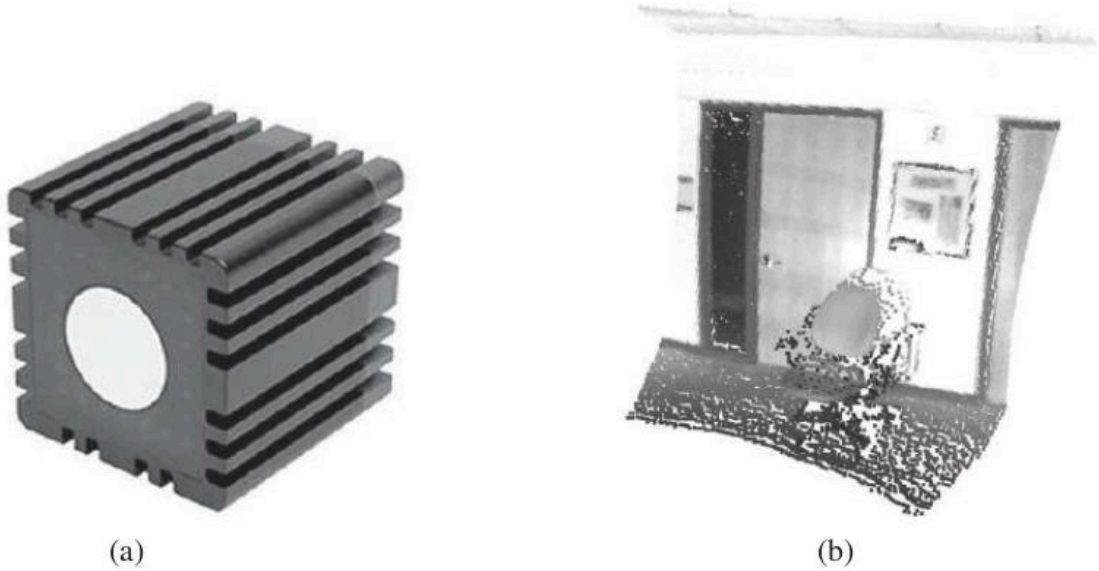
شکل ۲۶.۲ – (الف) مریخ‌نوردِ Curiosity ناسا که رویِ مریخ سلفی می‌گیرد. تصویر با اجازه از ناسا. (ب) پهپادِ Skydio که خانواده‌ای را در گردشِ دوچرخه همراهی می‌کند. تصویر با اجازه از Skydio.

«ربات‌های متحرک» (Mobile Robot) آن‌هایی‌اند که از چرخ‌ها، پاها یا روتورها برای حرکت در محیط استفاده می‌کنند. «پهپادهای چهارپره» (Quadcopter Drone) نوعی «وسیلهٔ نقلیهٔ هوایی بدونِ سرنشین» (UAV) هستند؛ «وسایلِ نقلیهٔ زیرآبیِ خودمختار» (AUV) اقیانوس‌ها را می‌پیمایند. اما بسیاری از ربات‌های متحرک داخلِ ساختمان می‌مانند و رویِ چرخ حرکت می‌کنند، مانندِ جاروبرقی یا رباتِ تحویلِ حوله در هتل. همتایانِ بیرونیِ آن‌ها شاملِ «خودروهای خودران» (Autonomous Car) یا «مریخ‌نورد»هایی (Rover) است که زمینِ تازه‌ای را کاوش می‌کنند، حتی رویِ سطحِ مریخ (شکل ۲۶.۲). سرانجام، «ربات‌های پادار» (Legged Robot) برای عبور از زمینِ ناهمواری طراحی شده‌اند که با چرخ غیرقابلِ‌دسترسی است. جنبهٔ منفی این است که کنترلِ پاها برای انجامِ کارِ درست چالش‌برانگیزتر از چرخاندنِ چرخ‌هاست. انواعِ دیگرِ ربات‌ها شاملِ پروتزها، اسکلت‌های بیرونی، ربات‌های بال‌دار، دسته‌ها (Swarm) و محیط‌های هوشمندی است که ربات در آن‌ها کلِ اتاق است.

۲۶.۲.۲ حس‌کردنِ جهان

حس‌گذارها رابطِ ادراکیِ بینِ ربات و محیط‌اند. «حس‌گذارهای منفعل» (Passive Sensor)، مانندِ دوربین‌ها، ناظرانِ واقعیِ محیط‌اند: سیگنال‌هایی را درمی‌گیرند که توسط منابعِ دیگرِ محیط تولید می‌شوند. «حس‌گذارهای فعال» (Active Sensor)، مانندِ سونار، انرژی به محیط می‌فرستند. آن‌ها بر این واقعیت تکیه می‌کنند که این انرژی به حس‌گذار بازتاب می‌شود. حس‌گذارهای فعال تمایل به ارائهٔ اطلاعاتِ بیشتر از حس‌گذارهای منفعل دارند، اما به بهایِ مصرفِ انرژیِ بیشتر و با خطرِ تداخل وقتی چند حس‌گذارِ فعال هم‌زمان استفاده می‌شوند. همچنین تمایز می‌گذاریم آیا حس‌گذاری به سمتِ حس‌کردنِ محیط، مکانِ ربات یا پیکربندیِ درونیِ ربات جهت‌گیری شده.

«فاصله‌یاب»ها (Range Finder) حس‌گذارهایی‌اند که فاصله تا اشیای نزدیک را اندازه می‌گیرند. حس‌گذارهای «سونار» (Sonar) فاصله‌یاب‌های فعالی‌اند که امواجِ صوتیِ جهت‌دار نشر می‌کنند، که توسط اشیا بازتاب می‌شوند، با بخشی از صدا که به حس‌گذار بازمی‌گردد. زمان و شدتِ سیگنالِ بازگشتی فاصله تا اشیای نزدیک را نشان می‌دهد. سونار فناوریِ انتخابی برای وسایلِ نقلیهٔ زیرآبیِ خودمختار است، و در روزهای اولیهٔ رباتیکِ داخلی محبوب بود. «دیدِ استریو» (Stereo Vision) (بخش ۲۵.۶ را ببینید) بر دوربین‌های متعدد برای تصویربرداری از محیط از دیدگاه‌های کمی متفاوت تکیه دارد و پارالاکسِ حاصل در این تصاویر را تحلیل می‌کند تا بردِ اشیای اطراف را محاسبه کند. برای ربات‌های زمینیِ متحرک، سونار و دیدِ استریو اکنون به‌ندرت استفاده می‌شوند، زیرا به‌طورِ قابلِ‌اعتمادی دقیق نیستند. Kinect حس‌گذارِ کم‌هزینهٔ محبوبی است که دوربین و پروژکتورِ «نورِ ساخت‌یافته» (Structured Light) را ترکیب می‌کند، که الگویی از خطوطِ شبکه‌ای را رویِ صحنه می‌افکند. دوربین می‌بیند خطوطِ شبکه چگونه خم می‌شوند، که به ربات اطلاعاتی دربارهٔ شکلِ اشیای صحنه می‌دهد. در صورتِ تمایل، فرافکنی می‌تواند نورِ مادون‌قرمز باشد، تا با حس‌گذارهای دیگر (مانندِ چشم‌های انسان) تداخل نکند.



شکل ۲۶.۳ – (الف) دوربینِ زمانِ پرواز؛ تصویر با اجازه از Mesa Imaging GmbH. (ب) تصویرِ بردِ سه‌بعدیِ به‌دست‌آمده با این دوربین. تصویرِ برد، تشخیصِ موانع و اشیای اطرافِ ربات را ممکن می‌کند. تصویر با اجازه از Willow Garage, LLC.

بیشترِ ربات‌های زمینی اکنون به فاصله‌یاب‌های نوریِ فعال مجهزند. درست مانندِ حس‌گذارهای سونار، حس‌گذارهای بردِ نوری سیگنال‌های فعال (نور) نشر می‌کنند و زمانِ رسیدنِ بازتابِ این سیگنال به حس‌گذار را اندازه می‌گیرند. شکل ۲۶.۳ (الف) «دوربینِ زمانِ پرواز» (Time-of-Flight Camera) را نشان می‌دهد. این دوربین تصاویرِ بردی مانندِ نشان‌داده‌شده در شکل ۲۶.۳ (ب) را با نرخِ تا ۶۰ فریم در ثانیه به‌دست می‌آورد. خودروهای خودران اغلب از «لیدارهای اسکنی» (Scanning Lidar) استفاده می‌کنند—حس‌گذارهای فعالی که پرتوهای لیزر نشر می‌کنند و پرتوِ بازتابیده را حس می‌کنند، که اندازه‌گیری‌های بردِ دقیق تا یک سانتی‌متر در بردِ ۱۰۰ متر می‌دهند. آن‌ها از چیدمان‌های پیچیدهٔ آینه‌ها یا عناصرِ چرخان برای جاروبِ پرتو در سراسرِ محیط و ساختنِ نقشه استفاده می‌کنند. لیدارهای اسکنی تمایل دارند در بردهای بلندتر بهتر از دوربین‌های زمانِ پرواز کار کنند، و تمایل دارند در نورِ روزِ روشن بهتر عمل کنند.

«رادار» (Radar) اغلب حس‌گذارِ بردِ انتخابی برای وسایلِ نقلیهٔ هوایی (خودمختار یا نه) است. حس‌گذارهای رادار می‌توانند فاصله‌ها تا کیلومترها را اندازه بگیرند، و مزیتی نسبت به حس‌گذارهای نوری دارند: می‌توانند از مه ببینند. در انتهایِ نزدیکِ حسِ برد، «حس‌گذارهای لمسی» (Tactile Sensor) مانندِ سبیل‌ها، پنل‌های ضربه‌ای و پوستِ حساس به لمس هستند. این حس‌گذارها برد را بر مبنای تماسِ فیزیکی اندازه می‌گیرند و فقط برای حسِ اشیای بسیار نزدیک به ربات قابلِ استقرارند.

دومین ردهٔ مهم «حس‌گذارهای مکان» (Location Sensor) است. بیشترِ حس‌گذارهای مکان از حسِ برد به‌عنوانِ مؤلفهٔ اصلی برای تعیینِ مکان استفاده می‌کنند. بیرون، «سامانهٔ موقعیت‌یابیِ سراسری» (Global Positioning System یا GPS) رایج‌ترین راه‌حلِ مسئلهٔ مکان‌یابی است. GPS فاصله تا ماهواره‌هایی را اندازه می‌گیرد که سیگنال‌های پالسی نشر می‌کنند. در حالِ حاضر، ۳۱ ماهوارهٔ عملیاتیِ GPS در مدار هست، و ۲۴ ماهوارهٔ GLONASS، همتایِ روسی. گیرنده‌های GPS می‌توانند فاصله تا ماهواره را با تحلیلِ تغییرِ فاز بازیابی کنند. با مثلث‌سازیِ سیگنال‌های ماهواره‌های متعدد، گیرنده‌های GPS می‌توانند مکانِ مطلقِ خود را رویِ زمین تا چند متر تعیین کنند. «GPS تفاضلی» (Differential GPS) شاملِ گیرندهٔ زمینیِ دومی با مکانِ شناخته‌شده است، که در شرایطِ ایده‌آل دقتِ میلی‌متری فراهم می‌کند. متأسفانه، GPS داخلِ ساختمان یا زیرِ آب کار نمی‌کند. داخل، مکان‌یابی اغلب با اتصالِ «نشانگر»هایی (Beacon) در محیط در مکان‌های شناخته‌شده محقق می‌شود. بسیاری از محیط‌های داخلی پر از ایستگاه‌های پایهٔ بی‌سیم‌اند، که می‌توانند از طریقِ تحلیلِ سیگنالِ بی‌سیم به مکان‌یابیِ ربات‌ها کمک کنند. زیرِ آب، نشانگرهای سونارِ فعال می‌توانند حسی از مکان فراهم کنند، با استفاده از صدا برای آگاه‌کردنِ AUVها از فاصله‌های نسبی‌شان تا آن نشانگرها.

سومین ردهٔ مهم «حس‌گذارهای حس‌عمقی» (Proprioceptive Sensor) است، که ربات را از حرکتِ خودش آگاه می‌کنند. برای اندازه‌گیریِ پیکربندیِ دقیقِ مفصلِ رباتیک، موتورها اغلب به «رمزگشاهای شفت» (Shaft Decoder) مجهزند که حرکتِ زاویه‌ایِ شفت را دقیقاً اندازه می‌گیرند. رویِ بازوهای رباتیک، رمزگشاهای شفت به ردیابیِ موقعیتِ مفصل‌ها کمک می‌کنند. رویِ ربات‌های متحرک، رمزگشاهای شفت چرخشِ چرخ را برای «مسافت‌سنجی» (Odometry) —اندازه‌گیریِ مسافتِ طی‌شده—گزارش می‌کنند. متأسفانه، چرخ‌ها تمایل به انحراف و لغزش دارند، پس مسافت‌سنجی فقط در مسافت‌های کوتاه دقیق است. نیروهای خارجی، مانندِ باد و جریان‌های اقیانوسی، عدم‌قطعیتِ موقعیتی را افزایش می‌دهند. «حس‌گذارهای اینرسی» (Inertial Sensor)، مانندِ ژیروسکوپ‌ها، با تکیه بر مقاومتِ جرم در برابرِ تغییرِ سرعت، عدم‌قطعیت را کاهش می‌دهند.

جنبه‌های مهمِ دیگرِ حالتِ ربات توسط «حس‌گذارهای نیرو» (Force Sensor) و «حس‌گذارهای گشتاور» (Torque Sensor) اندازه‌گیری می‌شوند. این‌ها وقتی ربات‌ها اشیای شکننده یا اشیایی را که اندازه و شکلِ دقیقشان ناشناخته است مدیریت می‌کنند، ضروری‌اند. دستکارِ رباتیکِ یک‌تنی را تصور کنید که لامپی را پیچ می‌کند. اعمالِ نیرویِ بیش‌ازحد و شکستنِ لامپ بسیار آسان خواهد بود. حس‌گذارهای نیرو به ربات اجازه می‌دهند حس کند چقدر محکم لامپ را گرفته، و حس‌گذارهای گشتاور به آن اجازه می‌دهند حس کند چقدر محکم می‌چرخاند. حس‌گذارهای باکیفیت می‌توانند نیروها را در هر سه جهتِ انتقالی و سه جهتِ چرخشی اندازه بگیرند. آن‌ها این کار را با بسامدی چند صد بار در ثانیه انجام می‌دهند تا ربات بتواند به‌سرعت نیروهای غیرمنتظره را تشخیص دهد و پیش از شکستنِ لامپ کنش‌هایش را تصحیح کند. با این حال، تجهیزِ ربات به حس‌گذارهای رده‌بالا و قدرتِ محاسباتیِ نظارت بر آن‌ها می‌تواند چالش باشد.

۲۶.۲.۳ تولیدِ حرکت

سازوکاری که حرکتِ اثرگذار را آغاز می‌کند «عملگر» (Actuator) نامیده می‌شود؛ نمونه‌ها شاملِ گیربکس‌ها، چرخ‌دنده‌ها، کابل‌ها و اتصال‌هاست. رایج‌ترین نوعِ عملگر «عملگرِ الکتریکی» است، که از الکتریسیته برای چرخاندنِ موتور استفاده می‌کند. این‌ها عمدتاً در سامانه‌هایی استفاده می‌شوند که به حرکتِ چرخشی نیاز دارند، مانندِ مفصل‌های بازویِ رباتیک. «عملگرهای هیدرولیکی» (Hydraulic Actuator) از سیالِ هیدرولیکیِ تحتِ فشار (مانندِ روغن یا آب) و «عملگرهای پنوماتیکی» (Pneumatic Actuator) از هوای فشرده برای تولیدِ حرکتِ مکانیکی استفاده می‌کنند.

عملگرها اغلب برای حرکتِ «مفصل»ها (Joint) استفاده می‌شوند، که اجسامِ صلب (پیوندها) را به هم متصل می‌کنند. بازوها و پاها چنین مفصل‌هایی دارند. در «مفصلِ چرخشی» (Revolute Joint)، یک پیوند نسبت به دیگری می‌چرخد. در «مفصلِ منشوری» (Prismatic Joint)، یک پیوند در طولِ دیگری می‌لغزد. هر دو از این‌ها مفصل‌های تک‌محوره‌اند (یک محورِ حرکت). انواعِ دیگرِ مفصل شاملِ مفصل‌های کروی، استوانه‌ای و مسطح است، که مفصل‌های چندمحوره‌اند.

برای تعامل با اشیای محیط، ربات‌ها از «گیره» (Gripper) استفاده می‌کنند. ابتدایی‌ترین نوعِ گیره «گیرهٔ فکِ موازی» (Parallel Jaw Gripper) است، با دو انگشت و یک عملگرِ تکی که انگشتان را برای گرفتنِ اشیا به هم حرکت می‌دهد. این اثرگذار هم به‌خاطرِ سادگی‌اش محبوب است و هم منفور. گیره‌های سه‌انگشتی انعطافِ کمی بیشتری ارائه می‌دهند در حالی که سادگی را حفظ می‌کنند. در انتهایِ دیگرِ طیف، دست‌های انسان‌نما (انسان‌ریخت) هستند. برای نمونه، دستِ چابکِ Shadow در مجموع ۲۰ عملگر دارد. این انعطافِ بسیار بیشتری برای دست‌کاریِ پیچیده ارائه می‌دهد، از جملهِ مانورهای دست‌کاریِ درون‌دستی (به برداشتنِ گوشیِ همراه و چرخاندنِ آن در دست برای راست‌کردنِ جهت‌اش فکر کنید)، اما این انعطاف به بهایی می‌آید—یادگیریِ کنترلِ این گیره‌های پیچیده چالش‌برانگیزتر است.

۲۶.۳ مسئلهٔ رباتیک چه نوع مسئله‌ای است؟

حالا که می‌دانیم سخت‌افزارِ ربات ممکن است چه باشد، آماده‌ایم نرم‌افزارِ عاملی را در نظر بگیریم که سخت‌افزار را برای رسیدن به هدف‌هایمان می‌راند. نخست باید چارچوبِ محاسباتی این عامل را تعیین کنیم. دربارهٔ جستجو در محیط‌های قطعی، MDPها برای محیط‌های تصادفیِ اما کاملاً مشاهده‌پذیر، POMDPها برای مشاهدهٔ جزئی، و بازی‌ها برای موقعیت‌هایی که عامل در آن‌ها به‌صورتِ منزوی عمل نمی‌کند صحبت کرده‌ایم. با توجه به چارچوبِ محاسباتی، باید اجزایش را نمونه‌سازی کنیم: توابعِ پاداش یا مطلوبیت، حالت‌ها، کنش‌ها، فضاهای مشاهده و مانندِ آن.

قبلاً اشاره کرده‌ایم مسائلِ رباتیک غیرقطعی، با مشاهدهٔ جزئی و چندعاملی‌اند. با استفاده از مفاهیمِ نظریهٔ بازیِ فصل ۱۸، می‌بینیم گاهی عامل‌ها همکارانه‌اند و گاهی رقابتی. در راهرویِ باریکی که فقط یک عامل می‌تواند اول برود، ربات و شخص همکاری می‌کنند، زیرا هر دو می‌خواهند مطمئن شوند به هم برخورد نمی‌کنند. اما در برخی موارد ممکن است کمی رقابت کنند تا سریع به مقصدشان برسند. اگر ربات بیش‌ازحد مؤدب باشد و همیشه جا باز کند، ممکن است در موقعیت‌های شلوغ گیر کند و هرگز به هدفش نرسد.

بنابراین، وقتی ربات‌ها به‌صورتِ منزوی عمل می‌کنند و محیطشان را می‌دانند، مسئله‌ای که حل می‌کنند را می‌توان به‌عنوانِ MDP صورتبندی کرد؛ وقتی اطلاعاتی کم دارند به POMDP تبدیل می‌شود؛ و وقتی اطرافِ مردم عمل می‌کنند اغلب می‌تواند به‌عنوانِ بازی صورتبندی شود.

تابعِ پاداشِ ربات در این صورتبندی چیست؟ معمولاً ربات در خدمتِ انسان عمل می‌کند—برای نمونه، تحویلِ غذا به بیمارِ بیمارستان برای پاداشِ بیمار، نه پاداشِ خودش. برای بیشترِ محیط‌های رباتیک، حتی اگر طراحانِ ربات ممکن است بکوشند تابعِ پاداشِ جانشینِ به‌اندازهٔ کافی خوبی مشخص کنند، تابعِ پاداشِ واقعی نزدِ کاربری است که قرار است ربات به او کمک کند. ربات یا باید خواسته‌های کاربر را رمزگشایی کند، یا به مهندسی تکیه کند که تقریبی از خواسته‌های کاربر را مشخص کند.

در موردِ فضاهای کنش، حالت و مشاهدهٔ ربات، عمومی‌ترین شکل این است که مشاهده‌ها خوراکِ حس‌گذارِ خام‌اند (مثلاً، تصاویرِ ورودی از دوربین‌ها، یا برخوردهای لیزرِ ورودی از لیدار)؛ کنش‌ها جریان‌های الکتریکیِ خامی‌اند که به موتورها فرستاده می‌شوند؛ و حالت چیزی است که ربات برای تصمیم‌گیری‌اش باید بداند. این بدان‌معناست شکافِ عظیمی بینِ ادراک‌ها و کنترل‌های حرکتیِ سطح‌پایین، و برنامه‌های سطح‌بالایی که ربات باید بسازد هست.

برای پرکردنِ این شکاف، رباتیست‌ها جنبه‌های مسئله را برای ساده‌کردنش از هم جدا می‌کنند. برای نمونه، می‌دانیم وقتی POMDPها را درست حل می‌کنیم، ادراک و کنش برهم‌کنش می‌کنند: ادراک آگاه می‌کند کدام کنش‌ها منطقی‌اند، اما کنش نیز ادراک را آگاه می‌کند، با عامل‌هایی که کنش‌هایی برای جمع‌آوریِ اطلاعات می‌گیرند وقتی آن اطلاعات در گام‌های زمانیِ بعد ارزش دارد. با این حال، ربات‌ها اغلب ادراک را از کنش جدا می‌کنند، خروجی‌های ادراک را مصرف می‌کنند و وانمود می‌کنند در آینده اطلاعاتِ بیشتری نخواهند گرفت. افزون بر این، برنامه‌ریزیِ سلسله‌مراتبی لازم است، زیرا هدفِ سطح‌بالایی مانندِ «رسیدن به کافه‌تریا» بسیار دور از فرمانِ حرکتی مانندِ «محورِ اصلی را ۱° بچرخان» است.

در رباتیک اغلب از سلسله‌مراتبِ سه‌سطحی استفاده می‌کنیم. «سطحِ برنامه‌ریزیِ تکلیف» (Task Planning) برای کنش‌های سطح‌بالا برنامه یا سیاستی تصمیم می‌گیرد، که گاهی «اولیهٔ کنش» یا «هدفِ فرعی» نامیده می‌شوند: حرکت به سمتِ در، بازکردنِ آن، رفتن به آسانسور، فشردنِ دکمه و مانندِ آن. سپس «برنامه‌ریزیِ حرکت» (Motion Planning) مسئولِ یافتنِ مسیری است که ربات را از نقطه‌ای به نقطهٔ دیگر می‌رساند و هر هدفِ فرعی را محقق می‌کند. سرانجام، «کنترل» (Control) برای محقق‌کردنِ حرکتِ برنامه‌ریزی‌شده با عملگرهای ربات استفاده می‌شود. چون سطحِ برنامه‌ریزیِ تکلیف معمولاً رویِ حالت‌ها و کنش‌های گسسته تعریف می‌شود، در این فصل عمدتاً بر برنامه‌ریزیِ حرکت و کنترل تمرکز خواهیم کرد.

به‌طورِ جداگانه، «یادگیریِ ترجیح» (Preference Learning) مسئولِ برآوردِ هدفِ کاربرِ نهایی است، و «پیش‌بینیِ مردم» (People Prediction) برای پیش‌بینیِ کنش‌های مردمِ دیگرِ محیطِ ربات استفاده می‌شود. همهٔ این‌ها با هم ترکیب می‌شوند تا رفتارِ ربات را تعیین کنند.

هرگاه مسئله‌ای را به قطعاتِ جداگانه تقسیم می‌کنیم، پیچیدگی را کاهش می‌دهیم، اما فرصت‌های کمک‌کردنِ قطعات به یکدیگر را رها می‌کنیم. کنش می‌تواند به بهبودِ ادراک کمک کند، و همچنین تعیین کند چه نوعِ ادراکی مفید است. به‌همین‌سان، تصمیم‌های سطحِ حرکت ممکن است وقتی نحوهٔ دنبال‌شدنِ آن حرکت در نظر گرفته شود بهترین نباشند؛ یا تصمیم‌های سطحِ تکلیف ممکن است برنامهٔ تکلیف را در سطحِ حرکت غیرقابلِ‌نمونه‌سازی کنند. پس، با پیشرفت در این حوزه‌های جداگانه، فشار برای بازتلفیقِ آن‌ها می‌آید: انجامِ برنامه‌ریزیِ حرکت و کنترل با هم، انجامِ برنامه‌ریزیِ تکلیف و حرکت با هم، و بازتلفیقِ ادراک، پیش‌بینی و کنش—بستنِ حلقهٔ بازخورد. رباتیکِ امروز دربارهٔ پیشرفتِ ادامه‌دار در هر حوزه و در عین حال بناکردن بر این پیشرفت برای دست‌یابی به یکپارچگیِ بهتر است.

۲۶.۴ ادراکِ رباتیک

«ادراک» (Perception) فرایندی است که ربات‌ها توسط آن اندازه‌گیری‌های حس‌گذار را به بازنمایی‌های درونیِ محیط نگاشت می‌کنند. بخشِ زیادی از آن از فنونِ بینایی رایانهٔ فصلِ قبلی استفاده می‌کند. اما ادراک برای رباتیک باید با حس‌گذارهای اضافی مانندِ لیدار و حس‌گذارهای لمسی سروکار داشته باشد. ادراک دشوار است، زیرا حس‌گذارها نویزی‌اند و محیط با مشاهدهٔ جزئی، غیرقابلِ‌پیش‌بینی و اغلب پویاست. به بیانِ دیگر، ربات‌ها همهٔ مسائلِ تخمینِ حالت (یا فیلترکردن) را دارند که در بخش ۱۴.۲ بحث کردیم. به‌عنوانِ قاعدهٔ سرانگشتی، بازنمایی‌های درونیِ خوب برای ربات‌ها سه ویژگی دارند:

1. اطلاعاتِ کافی برای تصمیم‌گیریِ خوبِ ربات شامل می‌شوند.
2. چنان ساخت‌یافته‌اند که بتوانند به‌صورتِ کارآمد به‌روزرسانی شوند.
3. به این معنا طبیعی‌اند که متغیرهای درونی با متغیرهای حالتِ طبیعیِ جهانِ فیزیکی متناظرند.

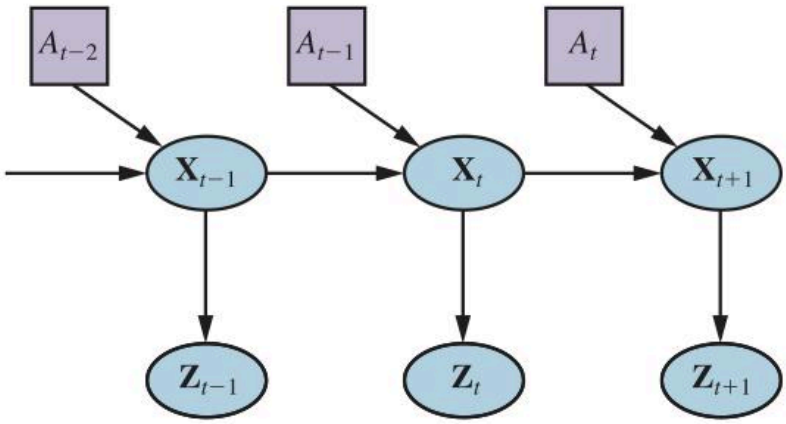
در فصل ۱۴ دیدیم فیلترهای کالمن، HMMها و شبکه‌های بیزیِ پویا می‌توانند مدل‌های انتقال و حس‌گذارِ محیطِ با مشاهدهٔ جزئی را بازنمایی کنند، و الگوریتم‌های دقیق و تقریبیِ به‌روزرسانیِ حالتِ باور—توزیعِ احتمالِ پسینِ رویِ متغیرهای حالتِ محیط—را توصیف کردیم. چند مدلِ شبکهٔ بیزیِ پویا برای این فرایند در فصل ۱۴ نشان داده شد. برای

مسائلِ رباتیک، کنش‌های گذشتهٔ خودِ ربات را به‌عنوانِ متغیرهای مشاهده‌شده در مدل شامل می‌کنیم. شکل ۲۶.۴ نمادگذاریِ استفاده‌شده در این فصل را نشان می‌دهد: X_t حالتِ محیط (شاملِ ربات) در زمانِ t، Z_t مشاهدهٔ دریافت‌شده در زمانِ t، و A_t کنشِ گرفته‌شده پس از دریافتِ مشاهده است.

می‌خواهیم حالتِ باورِ تازه، $P(X_{t+1} \mid z_{1:t+1}, a_{1:t})$، را از حالتِ باورِ کنونی، $P(X_t \mid z_{1:t}, a_{1:t-1})$، و مشاهدهٔ تازهٔ z_{t+1} محاسبه کنیم. این را در بخش ۱۴.۲ انجام دادیم، اما دو تفاوت این‌جاست: هم بر کنش‌ها و هم بر مشاهده‌ها شرطی می‌شویم، و با متغیرهای پیوسته سروکار داریم نه گسسته. پس، معادلهٔ فیلترکردنِ بازگشتی (۱۴.۵ در صفحهٔ ۴۶۷) را اصلاح می‌کنیم تا به‌جای جمع از انتگرال استفاده کند:

$$P(X_{t+1} \mid z_{1:t+1}, a_{1:t}) = \alpha\, P(z_{t+1} \mid X_{t+1}) \int P(X_{t+1} \mid x_t, a_t)\, P(x_t \mid z_{1:t}, a_{1:t-1})\, dx_t \quad (26.1)$$

این معادله بیان می‌کند پسینِ رویِ متغیرهای حالتِ X در زمانِ $t+1$ به‌صورتِ بازگشتی از برآوردِ متناظرِ یک گامِ زمانیِ زودتر محاسبه می‌شود. این محاسبه شاملِ کنشِ قبلیِ a_t و اندازه‌گیریِ حس‌گذارِ کنونیِ z_{t+1} است. برای نمونه، اگر هدف‌مان توسعهٔ رباتِ فوتبالیست باشد، X_{t+1} ممکن است شاملِ مکانِ توپِ فوتبال نسبت به ربات باشد. پسینِ $P(X_t \mid z_{1:t}, a_{1:t-1})$ توزیعِ احتمالیِ رویِ همهٔ حالت‌هاست که آنچه از اندازه‌گیری‌ها و کنترل‌های حس‌گذارِ گذشته می‌دانیم را درمی‌گیرد. معادلهٔ (26.1) به ما می‌گوید چگونه این مکان را به‌صورتِ بازگشتی برآورد کنیم، با تلفیقِ تدریجیِ اندازه‌گیری‌های حس‌گذار (مثلاً، تصاویرِ دوربین) و فرمان‌های حرکتِ ربات. احتمالِ $P(X_{t+1} \mid x_t, a_t)$ «مدلِ انتقال» یا «مدلِ حرکت» (Motion Model) و $P(z_{t+1} \mid X_{t+1})$ «مدلِ حس‌گذار» (Sensor Model) نامیده می‌شود.



شکل ۲۶.۴ – ادراکِ ربات را می‌توان استنتاجِ زمانی از دنباله‌های کنش‌ها و اندازه‌گیری‌ها دید، چنان‌که این شبکهٔ تصمیمِ پویا نشان می‌دهد.

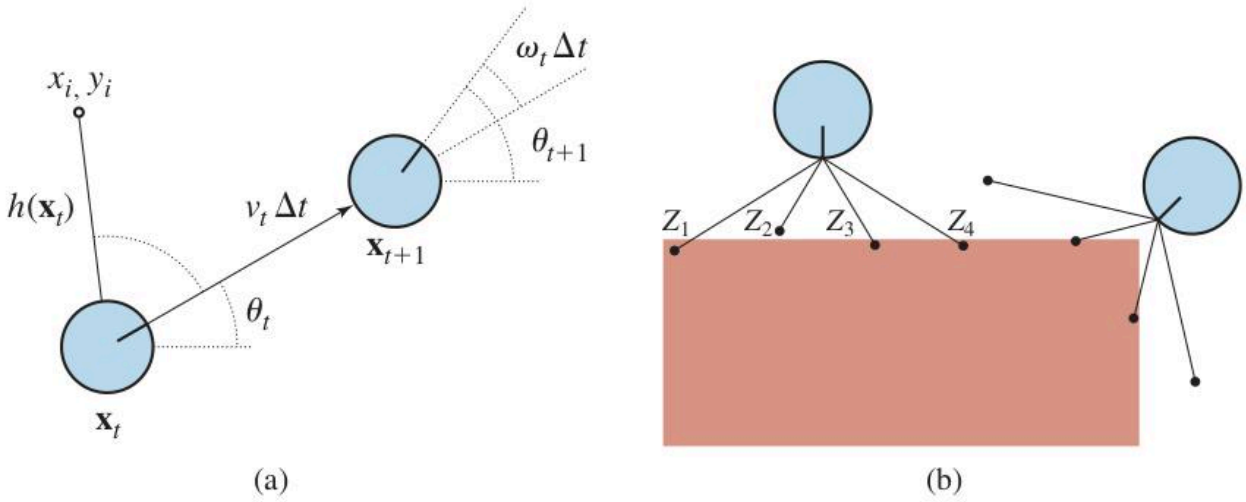
۲۶.۴.۱ مکان‌یابی و نگاشت

«مکان‌یابی» (Localization) مسئلهٔ یافتنِ اینکه چیزها کجا هستند—شاملِ خودِ ربات—است. برای ساده‌نگه‌داشتنِ کار، رباتِ متحرکی را در نظر بگیریم که به‌آرامی در جهانِ دوبعدیِ تخت حرکت می‌کند. همچنین فرض کنیم نقشهٔ دقیقی از محیط به ربات داده شده. (نمونه‌ای از چنین نقشه‌ای در شکل ۲۶.۷ ظاهر می‌شود.) «وضعیت» (Pose) چنین رباتِ

متحرکی توسط دو مختصاتِ دکارتی‌اش با مقادیرِ x و y و سمت‌گیری‌اش با مقدارِ θ تعریف می‌شود، چنان‌که در شکل ۲۶.۵ (الف) نشان داده شده. اگر آن سه مقدار را در بردارِ مرتب کنیم، هر حالتِ خاص با $X_t = (x_t, y_t, \theta_t)^T$ داده می‌شود. تا این‌جا خوب است.

در «تقریبِ سینماتیکی» (Kinematic Approximation)، هر کنش شاملِ مشخصاتِ «آنی» دو سرعت است—سرعتِ انتقالیِ v_t و سرعتِ چرخشیِ ω_t. برای بازه‌های زمانیِ کوچکِ Δt، مدلِ قطعیِ خامی از حرکتِ چنین ربات‌هایی به‌صورتِ زیر داده می‌شود:

$$\hat{X}_{t+1} = f(X_t, v_t, \omega_t) = X_t + (v_t\Delta t \cos\theta_t, v_t\Delta t \sin\theta_t, \omega_t\Delta t)$$



شکل ۲۶.۵ – (الف) مدلِ سینماتیکیِ ساده‌شدهٔ رباتِ متحرک. ربات به‌صورتِ دایره‌ای با خطِّ شعاعِ داخلی که جهتِ جلو را علامت می‌زند نشان داده شده. حالتِ x_t شاملِ موقعیتِ (x_t,y_t) (به‌صورتِ ضمنی نشان داده شده) و جهتِ θ_t است. حالتِ تازهٔ x_{t+1} با به‌روزرسانیِ موقعیت به‌مقدارِ $v_t\Delta t$ و جهت به‌مقدارِ $\omega_t\Delta t$ به‌دست می‌آید. همچنین نقطهٔ نشانه‌ای در (x_i,y_i) نشان داده شده که در زمانِ t مشاهده می‌شود. (ب) مدلِ حس‌گذارِ اسکنِ برد. دو وضعیتِ رباتِ ممکن برای اسکنِ بردِ داده‌شده (z_1,z_2,z_3,z_4) نشان داده شده. بسیار محتمل‌تر است وضعیتِ چپ اسکنِ برد را تولید کرده باشد تا وضعیتِ راست.

نمادِ $\hat{X}$ به پیش‌بینیِ حالتِ قطعی اشاره دارد.

البته، ربات‌های فیزیکی تا حدی غیرقابلِ‌پیش‌بینی‌اند. این معمولاً با توزیعِ گاوسی با میانگینِ $f(X_t, v_t, \omega_t)$ و کوواریانسِ Σ_x مدل می‌شود. (برای تعریفِ ریاضی، پیوستِ الف را ببینید.)

$$P(X_{t+1} \mid X_t, v_t, \omega_t) = N(\hat{X}_{t+1}, \Sigma_x)$$

این توزیعِ احتمال مدلِ حرکتِ ربات است. اثرهای کنشِ a_t بر مکانِ ربات را مدل می‌کند.

بعد، به مدلِ حس‌گذار نیاز داریم. دو نوعِ مدلِ حس‌گذار را بررسی می‌کنیم. نخستین فرض می‌کند حس‌گذارها ویژگی‌های پایدار و قابلِ‌بازشناسیِ محیط به نامِ «نقطهٔ نشانه» (Landmark) را تشخیص می‌دهند. برای هر نقطهٔ نشانه، برد و یاتاقان (bearing) گزارش می‌شود. فرض کنید حالتِ ربات $x_t = (x_t, y_t, \theta_t)^T$ است و نقطهٔ نشانه‌ای را حس

می‌کند که مکانش $(x_i, y_i)^T$ شناخته‌شده است. بدونِ نویز، پیش‌بینیِ برد و یاتاقان را می‌توان با هندسهٔ ساده محاسبه کرد (شکل ۲۶.۵ (الف) را ببینید):

$$\hat{z}_t = h(x_t) = (\sqrt{((x_t - x_i)^2 + (y_t - y_i)^2)}, \arctan((y_i - y_t)/(x_i - x_t)) - \theta_t)$$

باز، نویز اندازه‌گیری‌های ما را تحریف می‌کند. برای ساده‌نگه‌داشتنِ کار، نویزِ گاوسی با کوواریانسِ Σ_z فرض کنید، که مدلِ حس‌گذارِ $P(z_t \mid x_t) = N(\hat{z}_t, \Sigma_z)$ به ما می‌دهد.

مدلِ حس‌گذارِ تا حدی متفاوتی برای «آرایهٔ حس‌گذار» (Sensor Array) از حس‌گذارهای برد استفاده می‌شود، که هر یک یاتاقانِ ثابتی نسبت به ربات دارد. چنین حس‌گذارهایی بردارِ مقادیرِ بردِ $z_t = (z_1, ..., z_M)^T$ تولید می‌کنند. با توجه به وضعیتِ x_t، بردِ $\hat{z}_j$ محاسبه‌شده در طولِ اُمین جهتِ پرتو از x_t تا نزدیک‌ترین مانع باشد. مثلِ قبل، این توسط نویزِ گاوسی خراب می‌شود. معمولاً فرض می‌کنیم خطاهای جهت‌های پرتوِ متفاوت مستقل و هم‌توزیع‌اند، پس داریم:

$$P(z_t \mid x_t) = \alpha \prod_{j=1}^{M} e^{-(z_j - \hat{z}_j)^2 / 2\sigma^2}$$

شکل ۲۶.۵ (ب) نمونه‌ای از اسکنِ بردِ چهارپرتو و دو وضعیتِ رباتِ ممکن را نشان می‌دهد، که یکی به‌طورِ معقولانه‌ای محتمل است اسکنِ مشاهده‌شده را تولید کرده باشد و دیگری نه.

با مقایسهٔ مدلِ اسکنِ برد با مدلِ نقطهٔ نشانه، می‌بینیم مدلِ اسکنِ برد این مزیت را دارد که پیش از تفسیرِ اسکنِ برد نیازی به شناساییِ نقطهٔ نشانه نیست؛ در واقع، در شکل ۲۶.۵ (ب)، ربات با دیوارِ بی‌ویژگی روبه‌روست. از سوی دیگر، اگر نقطه‌های نشانهٔ دیدنی و قابلِ شناسایی باشند، ممکن است مکان‌یابیِ آنی فراهم کنند.

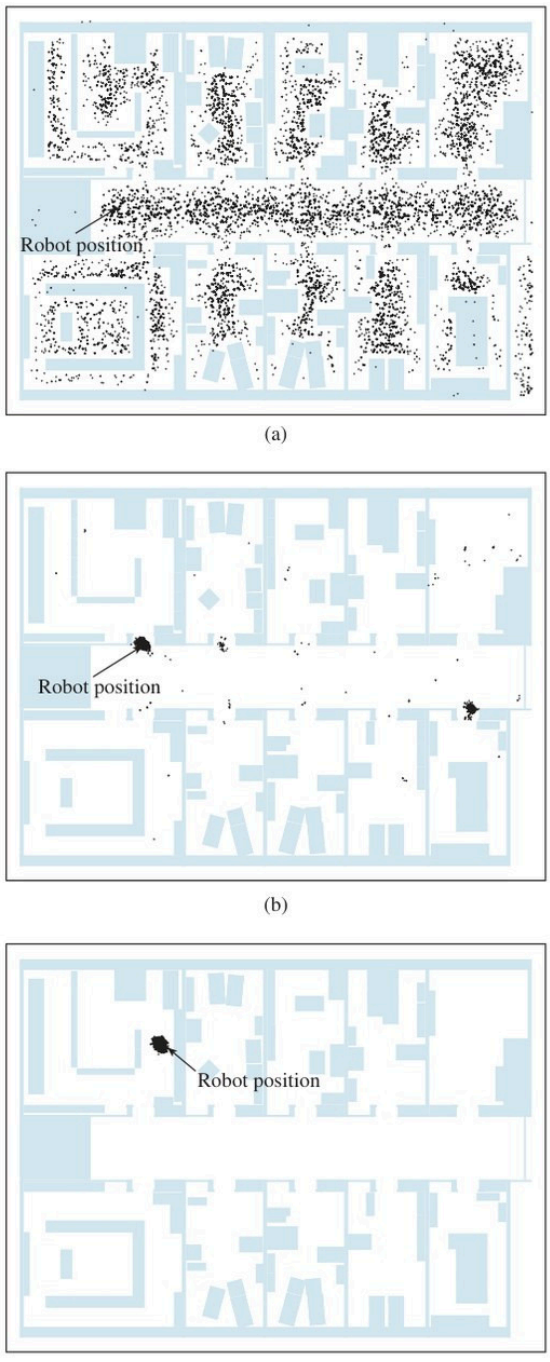
بخش ۱۴.۴ فیلترِ کالمن را توصیف کرد، که حالتِ باور را به‌عنوانِ گاوسیِ چندمتغیرهٔ تکی بازنمایی می‌کند، و فیلترِ ذره‌ای را، که حالتِ باور را با مجموعه‌ای از ذره‌ها بازنمایی می‌کند که با حالت‌ها متناظرند. بیشترِ الگوریتم‌های مکان‌یابیِ مدرن از یکی از این دو بازنماییِ باورِ رباتِ $P(X_t \mid z_{1:t}, a_{1:t-1})$ استفاده می‌کنند.

مکان‌یابی با استفاده از فیلترِ ذره‌ای «مکان‌یابیِ مونت‌کارلو» (Monte Carlo Localization) یا MCL نامیده می‌شود. الگوریتمِ MCL نمونه‌ای از الگوریتمِ فیلترِ ذره‌ایِ شکل ۱۴.۱۷ (صفحهٔ ۴۹۲) است. فقط باید مدلِ حرکت و مدلِ حس‌گذارِ مناسب را فراهم کنیم. شکل ۲۶.۶ نسخه‌ای را با مدلِ حس‌گذارِ اسکنِ برد نشان می‌دهد. عملکردِ الگوریتم در شکل ۲۶.۷ نشان داده شده، در حالی که ربات می‌فهمد کجای ساختمانِ اداری است. در تصویرِ نخست، ذره‌ها بر مبنای پیشین به‌صورتِ یکنواخت توزیع شده‌اند، که عدم‌قطعیتِ سراسری دربارهٔ موقعیتِ ربات را نشان می‌دهد. در تصویرِ دوم، نخستین مجموعهٔ اندازه‌گیری‌ها می‌رسد و ذره‌ها در نواحیِ باورِ پسینِ بالا خوشه‌ها تشکیل می‌دهند. در سومین، اندازه‌گیری‌های کافی برای راندنِ همهٔ ذره‌ها به مکانِ تکی در دسترس است.

```
function MONTE-CARLO-LOCALIZATION(a, z, N, P(X′ | X, v, ω), P(z | z*), map) returns S
                                                  inputs: a, robot velocities v and ω
                                          z, vector of M range scan data points
                                                 P(X′ | X, v, ω), motion model
                                           P(z | z*), range sensor noise model
                                               map, 2D map of the environment
                                                   persistent: S, vector of N samples
                                               local variables: W, vector of N weights
                                    S′, temporary vector of N samples

                                                                  if S is empty then
                                                                 for i = 1 to N do
                                                        S[i] ← sample from P(X₀)
                                                                   for i = 1 to N do
                                          S′[i] ← sample from P(X′ | X = S[i], v, ω)
                                                                          W[i] ← 1
                                                                 for j = 1 to M do
                                                  z* ← RAYCAST(j, X = S′[i], map)
                                                         W[i] ← W[i] · P(zⱼ | z*)
                                        S ← WEIGHTED-SAMPLE-WITH-REPLACEMENT(N, S′, W)
                                                                              return S
```

شکل ۲۶.۶ — الگوریتمِ مکان‌یابیِ مونت‌کارلو با استفاده از مدلِ حس‌گذارِ اسکنِ برد با نویزِ مستقل.

فیلترِ کالمن راهِ عمدهٔ دیگرِ مکان‌یابی است. فیلترِ کالمن پسینِ $P(X_t \mid z_{1:t}, a_{1:t-1})$ را با گاوسی بازنمایی می‌کند. میانگینِ این گاوسی را μ_t و کوواریانسش را Σ_t نشان می‌دهیم. مشکلِ اصلیِ باورهای گاوسی این است که فقط تحتِ مدل‌های حرکتِ خطیِ f و مدل‌های اندازه‌گیریِ خطیِ h بسته‌اند. برای f یا h غیرخطی، نتیجهٔ به‌روزرسانیِ فیلتر به‌طورِ کلی گاوسی نیست. پس، الگوریتم‌های مکان‌یابیِ استفاده‌کننده از فیلترِ کالمن مدل‌های حرکت و حس‌گذار را «خطی‌سازی» (Linearization) می‌کنند. خطی‌سازی تقریبِ محلیِ تابعِ غیرخطی با تابعِ خطی است. شکل ۲۶.۸ مفهومِ خطی‌سازی را برای مدلِ حرکتِ رباتِ (یک‌بعدی) نشان می‌دهد. در چپ، مدلِ حرکتِ غیرخطیِ $f(x_t, a_t)$ را تصویر می‌کند (کنترلِ a_t در این نمودار حذف شده، زیرا در خطی‌سازی نقشی ندارد). در راست، این تابع با تابعِ خطیِ $\tilde{f}(x_t, a_t)$ تقریب زده می‌شود. این تابعِ خطی در نقطهٔ μ_t، میانگینِ برآوردِ حالتِ ما در زمانِ t، بر f مماس است. چنین خطی‌سازی‌ای «بسطِ تیلورِ درجهٔ اول» (Taylor Expansion) نامیده می‌شود. فیلترِ کالمنی که f و h را با بسطِ تیلور خطی می‌کند «فیلترِ کالمنِ توسعه‌یافته» (Extended Kalman Filter یا EKF) نامیده می‌شود. شکل ۲۶.۹ دنباله‌ای از برآوردهای رباتی را نشان می‌دهد که الگوریتمِ مکان‌یابیِ فیلترِ کالمنِ توسعه‌یافته را اجرا می‌کند.



شکل ۲۶.۷ – مکان‌یابیِ مونت‌کارلو، الگوریتمِ فیلترِ ذره‌ای برای مکان‌یابیِ رباتِ متحرک. (الف) عدم‌قطعیتِ اولیهٔ سراسری. (ب) عدم‌قطعیتِ تقریباً دوموجهی پس از ناوبری در راهروی (متقارن). (ج) عدم‌قطعیتِ تک‌وجهی پس از ورود به اتاقی و یافتنِ اینکه متمایز است.

با حرکتِ ربات، عدم‌قطعیتِ برآوردِ مکانش افزایش می‌یابد، چنان‌که بیضی‌های خطا نشان می‌دهند. خطایش با حس‌کردنِ برد و یاتاقانِ نقطهٔ نشانه‌ای با مکانِ شناخته‌شده کاهش می‌یابد و با ازدست‌دادنِ دیدِ ربات از نقطهٔ نشانه دوباره افزایش می‌یابد. الگوریتم‌های EKF اگر نقطه‌های نشانه به‌راحتی شناسایی شوند خوب کار می‌کنند. در غیرِ این صورت، توزیعِ پسین ممکن است چندوجهی باشد، مانندِ شکل ۲۶.۷ (ب). مسئلهٔ نیاز به دانستنِ هویتِ نقطه‌های نشانه نمونه‌ای از مسئلهٔ ارتباطِ داده است که در شکل ۱۵.۳ بحث شد.

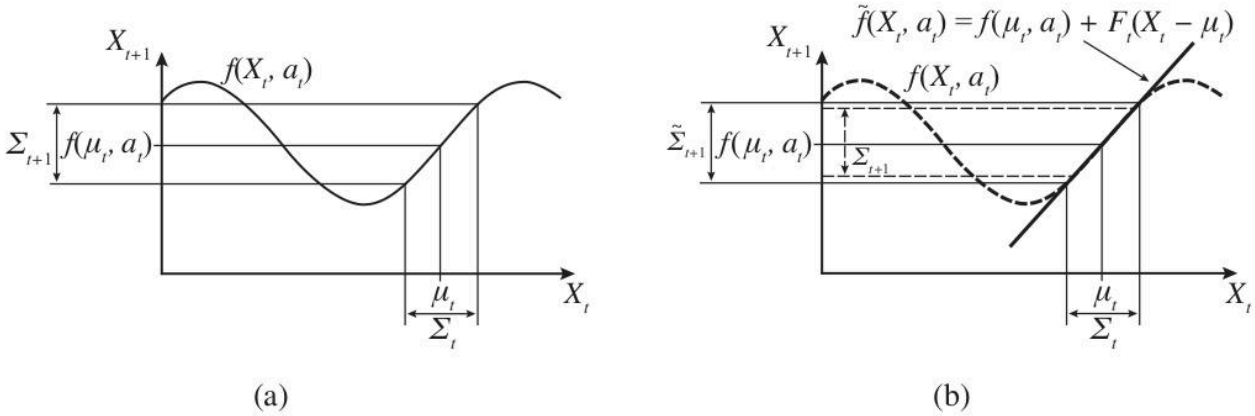
در برخی موقعیت‌ها، نقشه‌ای از محیط در دسترس نیست. آنگاه ربات باید نقشه‌ای کسب کند. این کمی مسئلهٔ مرغ و تخم‌مرغ است: رباتِ ناوبر باید مکانش را نسبت به نقشه‌ای که کاملاً نمی‌شناسد تعیین کند، در عینِ حال این نقشه را در حالی که مکانِ واقعی‌اش را کاملاً نمی‌داند می‌سازد. این مسئله برای بسیاری از کاربردهای رباتیک مهم است، و به‌طورِ گسترده‌ای تحتِ نامِ «مکان‌یابی و نگاشتِ هم‌زمان» (Simultaneous Localization and Mapping)، که به‌صورتِ SLAM مخفف می‌شود، مطالعه شده.

مسائلِ SLAM با بسیاری از فنونِ احتمالاتیِ متفاوت حل می‌شوند، از جملهِ فیلترِ کالمنِ توسعه‌یافتهٔ بحث‌شده در بالا. استفاده از EKF سرراست است: فقط بردارِ حالت را افزوده کن تا مکان‌های نقطه‌های نشانهٔ محیط را شامل شود. خوشبختانه، به‌روزرسانیِ EKF به‌صورتِ درجهٔ دوم مقیاس می‌شود، پس برای نقشه‌های کوچک (مثلاً، چند صد نقطهٔ نشانه) محاسبه کاملاً شدنی است. نقشه‌های غنی‌تر اغلب با روش‌های تسکینِ گراف به‌دست می‌آیند، مشابهِ فنونِ استنتاجِ شبکهٔ بیزیِ بحث‌شده در فصل ۱۳. بیشینه‌سازیِ امید (EM) نیز برای SLAM استفاده می‌شود.

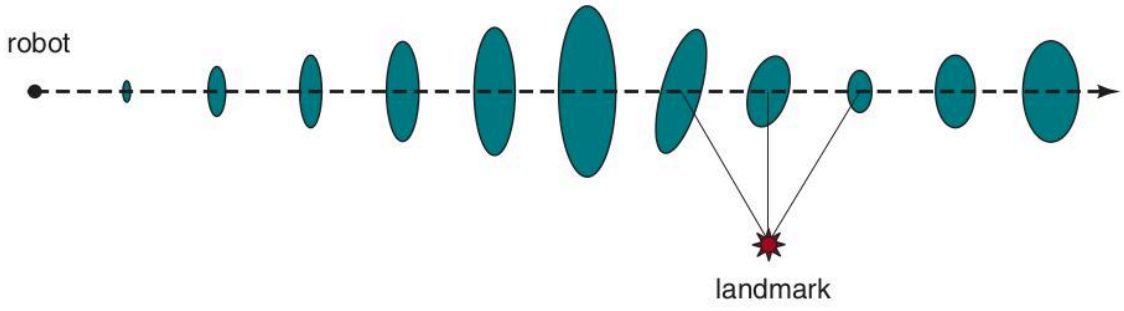
۲۶.۴.۲ انواعِ دیگرِ ادراک

همهٔ ادراکِ ربات دربارهٔ مکان‌یابی یا نگاشت نیست. ربات‌ها همچنین دما، بوها، صدا و مانندِ آن را ادراک می‌کنند. بسیاری از این کمیت‌ها را می‌توان با گونه‌هایی از شبکه‌های بیزیِ پویا برآورد کرد. تنها چیزی که برای چنین برآوردگرهایی لازم است توزیع‌های احتمالِ شرطی‌ای است که تکاملِ متغیرهای حالت را در طولِ زمان مشخص می‌کنند، و مدل‌های حس‌گذاری که رابطهٔ اندازه‌گیری‌ها با متغیرهای حالت را توصیف می‌کنند.

همچنین ممکن است ربات را به‌عنوانِ عاملِ واکنشی برنامه‌ریزی کرد، بدونِ استدلالِ صریح دربارهٔ توزیع‌های احتمالِ رویِ حالت‌ها. آن رویکرد را در بخش ۲۶.۹.۱ پوشش می‌دهیم.



شکل ۲۶.۸ – تصویرِ یک‌بعدیِ مدلِ حرکتِ خطی‌شده: (الف) تابعِ f، و فرافکنیِ میانگینِ μ_t و بازهٔ کوواریانس (بر مبنای Σ_t) به زمانِ t+1. (ب) نسخهٔ خطی‌شده مماسِ f در μ_t است. فرافکنیِ میانگینِ μ_t درست است. با این حال، کوواریانسِ فرافکنی‌شدهٔ $\tilde{\Sigma}_{t+1}$ با Σ_{t+1} تفاوت دارد.



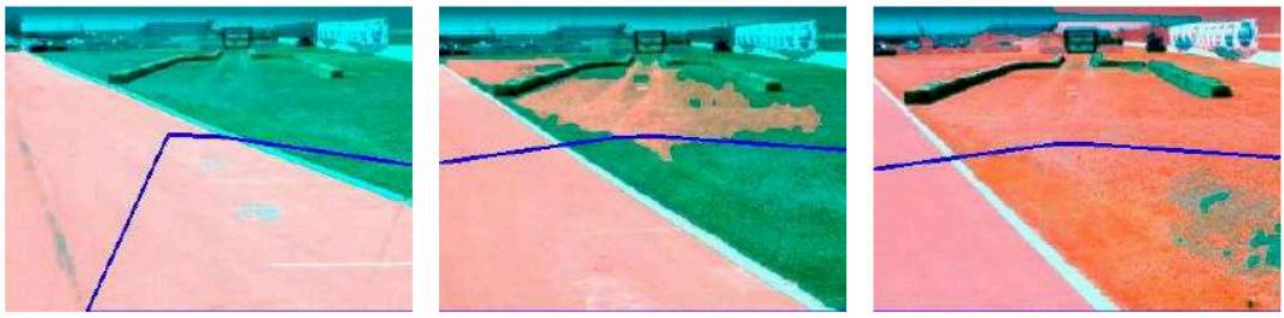
شکل ۲۶.۹ – مکان‌یابی با استفاده از فیلترِ کالمنِ توسعه‌یافته. ربات رویِ خطِّ مستقیم حرکت می‌کند. با پیشرفت‌ش، عدم‌قطعیتِ برآوردِ مکانش افزایش می‌یابد، چنان‌که بیضی‌های خطا نشان می‌دهند. وقتی نقطهٔ نشانه‌ای با موقعیتِ شناخته‌شده را مشاهده می‌کند، عدم‌قطعیت کاهش می‌یابد.

گرایشِ رباتیک به‌وضوح به سمتِ بازنمایی‌هایی با معناشناسیِ خوب‌تعریف‌شده است. فنونِ احتمالاتی در بسیاری از مسائلِ ادراکیِ دشوار مانندِ مکان‌یابی و نگاشت از رویکردهای دیگر بهتر عمل می‌کنند. با این حال، فنونِ آماری گاهی بیش‌ازحد دست‌وپاگیرند، و راه‌حل‌های ساده‌تر ممکن است در عمل به همان اندازه مؤثر باشند. برای کمک به تصمیم‌گیری دربارهٔ اینکه کدام رویکرد را بگیریم، تجربهٔ کار با ربات‌های فیزیکیِ واقعی بهترین معلم شماست.

۲۶.۴.۳ یادگیریِ نظارت‌شده و نظارت‌نشده در ادراکِ ربات

یادگیریِ ماشینی نقشِ مهمی در ادراکِ ربات ایفا می‌کند. این به‌ویژه وقتی صادق است که بهترین بازنماییِ درونی شناخته‌شده نباشد. یکی از رویکردهای رایج نگاشتِ جریان‌های حس‌گذارِ پربعد به فضاهای کم‌بعدتر با استفاده از روش‌های یادگیریِ ماشینیِ نظارت‌نشده (فصل ۱۹ را ببینید) است. چنین رویکردی «جاسازیِ کم‌بعد» (Low-Dimensional Embedding) نامیده می‌شود. یادگیریِ ماشینی یادگیریِ مدل‌های حس‌گذار و حرکت از داده را ممکن می‌کند، در حالی که هم‌زمان بازنماییِ درونیِ مناسبی کشف می‌شود.

فنّ یادگیریِ ماشینیِ دیگری ربات‌ها را قادر می‌سازد به‌طورِ پیوسته با تغییرهای بزرگِ اندازه‌گیری‌های حس‌گذار سازگار شوند. تصور کنید از فضایِ آفتابی به اتاقِ تاریکی با چراغ‌های نئون قدم می‌گذارید. واضح است، داخل تاریک‌تر است. اما تغییرِ منبعِ نور بر همهٔ رنگ‌ها نیز اثر می‌گذارد: نورِ نئون مؤلفهٔ سبزِ قوی‌تری نسبت به نورِ خورشید دارد. با این حال، به‌نحوی به نظر می‌رسد متوجهِ تغییر نمی‌شویم. اگر با مردم به اتاقِ نئون‌دار قدم بگذاریم، فکر نمی‌کنیم چهره‌هایشان ناگهان سبز شده. ادراکِ ما به‌سرعت با شرایطِ نورپردازیِ تازه سازگار می‌شود، و مغزِ ما تفاوت‌ها را نادیده می‌گیرد.



شکل ۲۶.۱۰ – دنباله‌ای از رده‌بندی‌های «سطحِ قابلِ رانندگی» با بینایی تطبیقی. (الف) فقط جاده به‌عنوانِ قابلِ رانندگی رده‌بندی می‌شود (ناحیهٔ صورتی). خطِّ آبیِ Vشکل جایی را نشان می‌دهد که وسیلهٔ نقلیه به سمتِ آن می‌رود. (ب) به وسیلهٔ نقلیه فرمانِ رانندگیِ خارج از جاده داده می‌شود، و رده‌بند شروع به رده‌بندیِ برخی از چمن‌ها به‌عنوانِ قابلِ رانندگی می‌کند. (ج) وسیلهٔ نقلیه مدلِ سطوحِ قابلِ رانندگی‌اش را به‌روزرسانی کرده تا با چمن و همچنین جاده متناظر شود. با اجازه از سباستین ترون.

فنونِ «ادراکِ تطبیقی» (Adaptive Perception) ربات‌ها را قادر به تنظیم با چنین تغییرهایی می‌کنند. یک نمونه در شکل ۲۶.۱۰ نشان داده شده، برگرفته از حوزهٔ رانندگیِ خودمختار. در این‌جا وسیلهٔ نقلیهٔ زمینیِ بدونِ سرنشین رده‌بندِ مفهومِ «سطحِ قابلِ رانندگی» را تطبیق می‌دهد. این چگونه کار می‌کند؟ ربات از لیزر برای ارائهٔ رده‌بندیِ ناحیهٔ کوچکی بلافاصله در جلویِ ربات استفاده می‌کند. وقتی این ناحیه در اسکنِ بردِ لیزری تخت یافت شود، به‌عنوانِ مثالِ آموزشیِ مثبت برای مفهومِ «سطحِ قابلِ رانندگی» استفاده می‌شود. سپس فنّ مخلوطِ گاوسی‌ها مشابهِ الگوریتمِ EM بحث‌شده در فصل ۲۰ آموزش می‌بیند تا ضرایبِ رنگ و بافتِ خاصِ وصلهٔ نمونهٔ کوچک را بازشناسی کند. تصاویرِ شکل ۲۶.۱۰ نتیجهٔ اعمالِ این رده‌بند بر کلِ تصویرند.

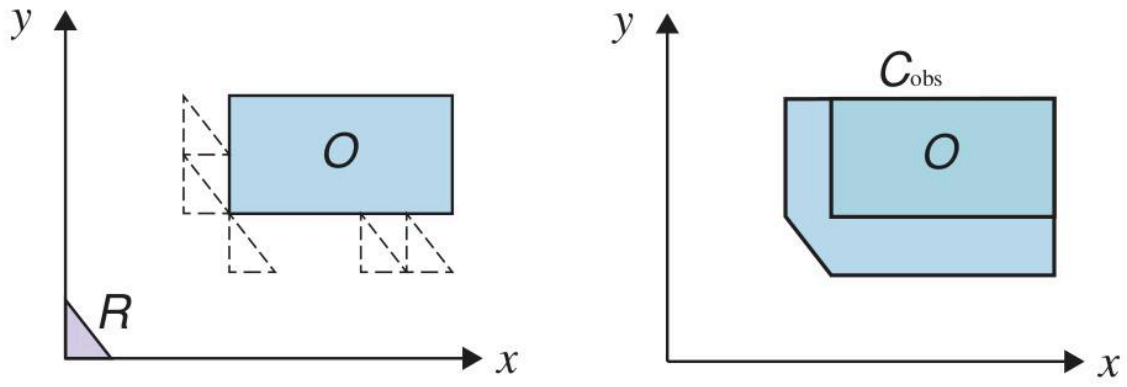
روش‌هایی که ربات‌ها را به جمع‌آوریِ دادهٔ آموزشیِ خودشان (با برچسب‌ها!) وادار می‌کنند «خودنظارت» (Self-Supervised) نامیده می‌شوند. در این نمونه، ربات از یادگیریِ ماشینی برای بهره‌برداری از حس‌گذارِ کوتاه‌بُردی که برای رده‌بندیِ زمین خوب کار می‌کند به حس‌گذاری که می‌تواند خیلی دورتر ببیند استفاده می‌کند. این به ربات اجازه می‌دهد سریع‌تر رانندگی کند و فقط وقتی مدلِ حس‌گذار می‌گوید تغییری در زمین هست که باید با دقتِ بیشتری توسط حس‌گذارهای کوتاه‌بُرد بررسی شود کند می‌کند.

۲۶.۵ برنامه‌ریزی و کنترل

تعمقِ ربات در نهایت به تصمیم‌گیری دربارهٔ نحوهٔ حرکت می‌رسد، از سطحِ انتزاعیِ تکلیف تا پایین‌ترین سطحِ جریان‌هایی که به موتورهایش فرستاده می‌شوند. در این بخش، با فرض اینکه ادراک (و در صورتِ نیاز، پیش‌بینی) داده شده ساده‌سازی می‌کنیم، پس جهان مشاهده‌پذیر است. همچنین انتقال‌های (دینامیکِ) قطعیِ جهان را فرض می‌کنیم.

با جداکردنِ حرکت از کنترل شروع می‌کنیم. «مسیر» (Path) را دنباله‌ای از نقطه‌ها در فضایِ هندسی تعریف می‌کنیم که ربات (یا بخشی از ربات، مانندِ بازو) دنبال خواهد کرد. این با مفهومِ مسیر در فصل ۳ مرتبط است، اما در این‌جا دنباله‌ای از نقطه‌ها در فضا را منظور می‌کنیم، نه دنباله‌ای از کنش‌های گسسته. تکلیفِ یافتنِ مسیرِ خوب «برنامه‌ریزیِ

حرکت» (Motion Planning) نامیده می‌شود.



شکل ۲۶.۱۱ – رباتِ مثلثیِ ساده‌ای که می‌تواند انتقال یابد و باید از مانعِ مستطیلی اجتناب کند. در چپ فضایِ کار و در راست فضایِ پیکربندی است.

پس از داشتنِ مسیر، تکلیفِ اجرایِ دنباله‌ای از کنش‌ها برای دنبال‌کردنِ مسیر «کنترلِ دنبال‌کردنِ مسیر» (Trajectory Tracking Control) نامیده می‌شود. «مسیرِ زمانی» (Trajectory) مسیری است که برای هر نقطهٔ مسیر زمانی مرتبط دارد. مسیر فقط می‌گوید «از A به B به C برو و...» و مسیرِ زمانی می‌گوید «از A شروع کن، ۱ ثانیه طول بده تا به B برسی، و ۱.۵ ثانیه دیگر تا به C برسی و...».

۲۶.۵.۱ فضایِ پیکربندی

رباتِ سادهٔ R را به شکلِ مثلثِ قائم‌الزاویه تصور کنید، چنان‌که مثلثِ اسطوخودوسیِ گوشهٔ پایینِ چپِ شکل ۲۶.۱۱ نشان می‌دهد. ربات باید مسیری را برنامه‌ریزی کند که از مانعِ مستطیلیِ O اجتناب می‌کند. فضایِ فیزیکی که ربات در آن حرکت می‌کند «فضایِ کار» (Workspace) نامیده می‌شود. این رباتِ خاص می‌تواند در هر جهتِ صفحهٔ x–y حرکت کند، اما نمی‌تواند بچرخد. شکل پنج موقعیتِ ممکنِ دیگرِ ربات را با طرح‌های خطِّ چین نشان می‌دهد؛ هر یک به همان اندازه‌ای که ربات می‌تواند به مانع نزدیک شود به مانع نزدیک است.

بدنهٔ ربات را می‌توان به‌عنوانِ مجموعه‌ای از نقطه‌های (x,y) بازنمایی کرد (یا نقطه‌های (x,y,z) برای رباتِ سه‌بعدی)، و مانع را نیز. با این بازنمایی، اجتناب از مانع به این معناست هیچ نقطه‌ای از ربات با هیچ نقطه‌ای از مانع هم‌پوشانی ندارد. برنامه‌ریزیِ حرکت به محاسبه‌هایی رویِ مجموعه‌های نقطه نیاز می‌داشت، که می‌تواند پیچیده و وقت‌گیر باشد.

می‌توانیم محاسبه‌ها را با استفاده از طرحِ بازنمایی‌ای ساده کنیم که در آن همهٔ نقطه‌های تشکیل‌دهندهٔ ربات به‌عنوانِ نقطهٔ تکی در فضایِ انتزاعیِ چندبعدی بازنمایی می‌شوند، که آن را «فضایِ پیکربندی» (Configuration Space) یا C-فضا می‌نامیم. ایده این است که مجموعهٔ نقطه‌های تشکیل‌دهندهٔ ربات را می‌توان محاسبه کرد اگر بدانیم (۱) اندازه‌گیری‌های پایهٔ ربات (برای رباتِ مثلثیِ ما، طولِ سه ضلع کافی است) و (۲) وضعیتِ کنونیِ ربات—موقعیت و جهت‌اش.

برای رباتِ مثلثیِ سادهٔ ما، دو بُعد برای C-فضا کافی است: اگر مختصاتِ (x,y) نقطهٔ خاصی از ربات را بدانیم—از رأسِ قائم‌الزاویه استفاده می‌کنیم—آنگاه می‌توانیم محاسبه کنیم هر نقطهٔ دیگرِ مثلث کجاست (زیرا اندازه و شکلِ مثلث را می‌دانیم و چون مثلث نمی‌تواند بچرخد). در گوشهٔ پایینِ چپِ شکل ۲۶.۱۱، مثلثِ اسطوخودوسی را می‌توان با پیکربندیِ (0,0) بازنمایی کرد.

اگر قاعده‌ها را تغییر دهیم تا ربات بتواند بچرخد، به سه بُعد، (x,y,θ)، نیاز خواهیم داشت تا بتوانیم محاسبه کنیم هر نقطه کجاست. در این‌جا θ زاویهٔ چرخشِ ربات در صفحه است. اگر ربات همچنین تواناییِ کش‌آمدن داشت، با رشدِ یکنواخت با عاملِ مقیاسِ s، آنگاه C-فضا چهار بُعد، (x,y,θ,s)، می‌داشت.

فعلاً به C-فضایِ دوبعدیِ سادهٔ رباتِ مثلثیِ غیرچرخان می‌چسبیم. تکلیفِ بعدی فهمیدنِ این است که نقطه‌های مانع در C-فضا کجا هستند. پنج مثلثِ خطِّ چینِ چپِ شکل ۲۶.۱۱ را در نظر بگیر و توجه کن رأسِ قائم‌الزاویه رویِ هر یک کجاست. سپس همهٔ راه‌هایی را تصور کن که مثلث می‌توانست بلغزد. بدیهی است، رأسِ قائم‌الزاویه نمی‌تواند داخلِ مانع برود، و همچنین نمی‌تواند از هر یک از پنج مثلثِ خطِّ چین به آن نزدیک‌تر شود. پس می‌بینید ناحیه‌ای که رأسِ قائم‌الزاویه نمی‌تواند برود—«مانعِ C-فضا» (C-Space Obstacle)—پنج‌ضلعیِ راستِ شکل ۲۶.۱۱ است که با C_obs برچسب خورده.

در زبانِ روزمره می‌گوییم موانعِ متعددی برای ربات هست—میز، صندلی، چند دیوار. اما نمادگذاریِ ریاضی کمی آسان‌تر است اگر همهٔ این‌ها را ترکیب‌شده در یک «مانع» بدانیم که اتفاقاً مؤلفه‌های ناهمبند دارد. به‌طورِ کلی، مانعِ C-فضا مجموعهٔ همهٔ نقطه‌های q در C است چنان‌که اگر ربات در آن پیکربندی قرار می‌گرفت، هندسهٔ فضایِ کارش با مانعِ فضایِ کار تلاقی می‌کرد.

موانعِ فضایِ کار مجموعهٔ نقطه‌های O و مجموعهٔ همهٔ نقطه‌های ربات در پیکربندیِ q برابرِ A(q) باشد. آنگاه مانعِ C-فضا به‌صورتِ زیر تعریف می‌شود:

```
C_obs  = {q : q ∈ C ∧ A(q) ∩ O ≠ ∅}
C_free = C − C_obs
```

C-فضا برای ربات‌های با قطعاتِ متحرک جالب‌تر می‌شود. بازویِ دوپیوندیِ شکل ۲۶.۱۲ (الف) را در نظر بگیر. به میز پیچ شده، پس پایه حرکت نمی‌کند، اما بازو دو مفصل دارد که مستقل حرکت می‌کنند—این‌ها را «درجه‌های آزادی» (Degrees of Freedom یا DOF) می‌نامیم. حرکتِ مفصل‌ها مختصاتِ (x,y) آرنج، گیره و هر نقطهٔ بازو را تغییر می‌دهد. فضایِ پیکربندیِ بازو دوبعدی است: (θ_شانه، θ_آرنج)، که θ_شانه زاویهٔ مفصلِ شانه و θ_آرنج زاویهٔ مفصلِ آرنج است.

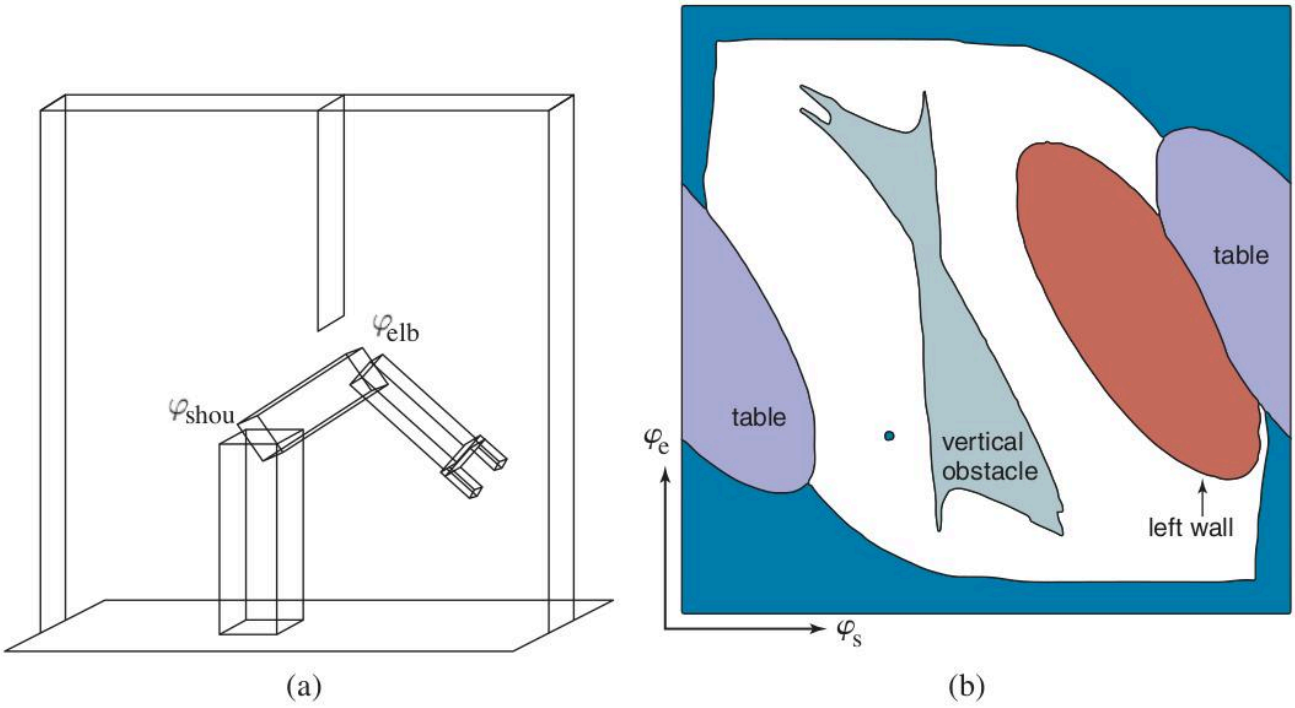
دانستنِ پیکربندیِ بازویِ دوپیوندیِ ما به این معناست می‌توانیم با مثلثاتِ ساده تعیین کنیم هر نقطهٔ بازو کجاست. به‌طورِ کلی، «نگاشتِ سینماتیکِ پیشرو» (Forward Kinematics) تابعِ φ_b : C → W است که پیکربندی را می‌گیرد و مکانِ نقطهٔ خاصِ b رویِ ربات را وقتی ربات در آن پیکربندی است خروجی می‌دهد. نگاشتِ سینماتیکِ پیشروِ به‌ویژه مفیدی مربوط به «اثرگذارِ انتهایی» ربات، φ_EE است. مجموعهٔ همهٔ نقطه‌های ربات در پیکربندیِ خاصِ q با A(q) ⊂ W نشان داده می‌شود:

```
A(q) = ⋃_b {φ_b(q)}
```

مسئلهٔ معکوس، نگاشتِ مکانِ مطلوبِ نقطه‌ای از ربات به پیکربندی(هایی) که ربات برای رخدادنِ آن باید در آن‌ها باشد، «سینماتیکِ معکوس» (Inverse Kinematics) شناخته می‌شود:

```
IK_b : x ∈ W ↦ {q ∈ C : φ_b(q) = x}
```

گاهی نگاشتِ سینماتیکِ معکوس ممکن است نه فقط موقعیت، بلکه جهتِ مطلوب را نیز به‌عنوانِ ورودی بگیرد. وقتی می‌خواهیم دستکاری‌کننده شیئی را بگیرد، برای نمونه، می‌توانیم موقعیت و جهتِ مطلوبی برای گیره‌اش محاسبه کنیم و از سینماتیکِ معکوس برای تعیینِ پیکربندیِ هدفِ ربات استفاده کنیم. سپس برنامه‌ریز باید راهی برای بردنِ ربات از پیکربندیِ کنونی‌اش به پیکربندیِ هدف بدونِ تلاقی با موانع بیابد.



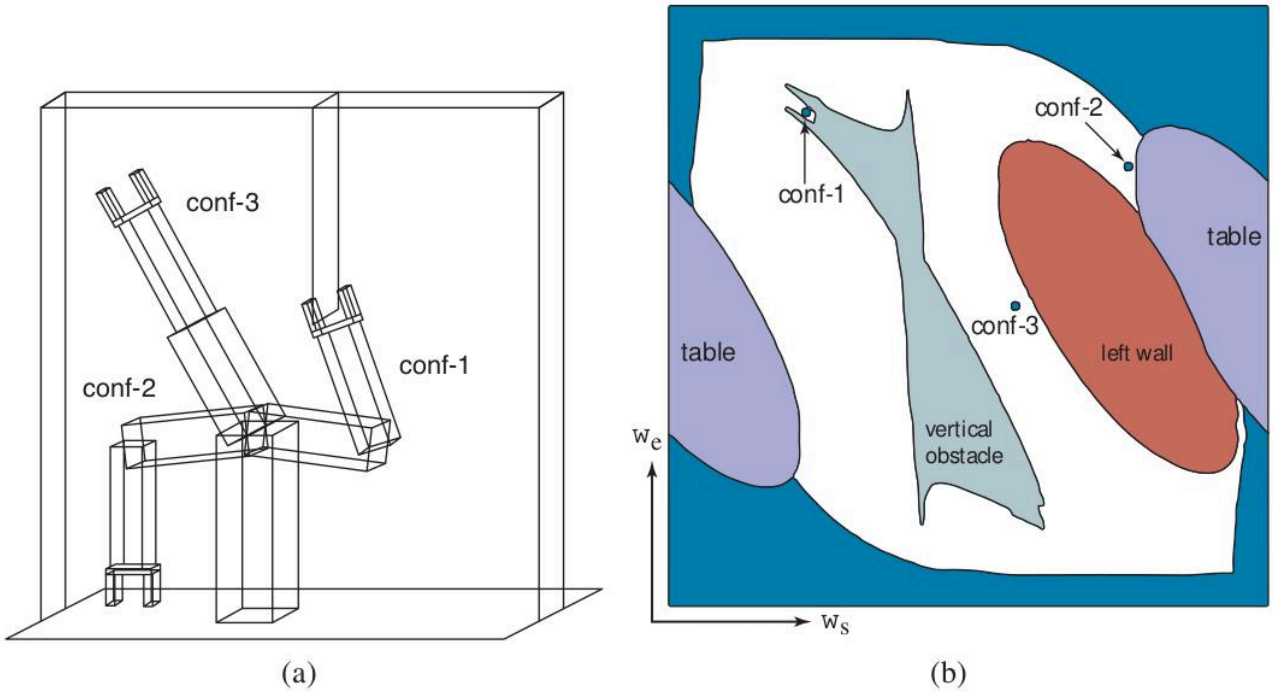
شکل ۲۶.۱۲ – (الف) بازنماییِ فضایِ کارِ بازویِ رباتیک با دو درجهٔ آزادی. فضایِ کار جعبه‌ای با مانعِ تختِ آویزان از سقف است. (ب) فضایِ پیکربندیِ همان ربات. فقط ناحیه‌های سفیدِ فضا پیکربندی‌هایی‌اند که عاری از برخوردند. نقطهٔ این نمودار با پیکربندیِ رباتِ نشان‌داده‌شده در چپ متناظر است.

موانعِ فضایِ کار اغلب به‌صورتِ شکل‌های هندسیِ ساده ترسیم می‌شوند—به‌ویژه در کتاب‌های درسیِ رباتیک، که تمایل به تمرکز بر موانعِ چندضلعی دارند. اما موانع در فضایِ پیکربندی چگونه به نظر می‌رسند؟

برای بازویِ دوپیوندی، موانعِ سادهٔ فضایِ کار، مانندِ خطِّ عمودی، همتایانِ بسیار پیچیده‌ای در C-فضا دارند، چنان‌که در شکل ۲۶.۱۲ (ب) نشان داده شده. سایه‌های متفاوتِ فضایِ اشغال‌شده با اشیای متفاوتِ فضایِ کارِ ربات متناظرند: ناحیهٔ تاریکِ اطرافِ کلِ فضایِ آزاد با پیکربندی‌هایی متناظر است که ربات در آن‌ها با خودش برخورد می‌کند. به‌آسانی دیده می‌شود مقادیرِ افراطیِ زاویه‌های شانه یا آرنج باعث چنین نقضی می‌شوند. دو ناحیهٔ بیضویِ دو طرفِ ربات با

میزی متناظرند که ربات رویِ آن نصب شده. ناحیهٔ بیضویِ سوم با دیوارِ چپ متناظر است. سرانجام، جالب‌ترین شیء در فضایِ پیکربندی مانعِ عمودیِ آویزان از سقف است که حرکتِ ربات را مختل می‌کند. این شیء شکلِ بامزه‌ای در فضایِ پیکربندی دارد: به‌شدت غیرخطی و در جاهایی حتی مقعر است. با کمی تخیل، خواننده شکلِ گیره را در انتهایِ بالایِ چپ بازشناسی خواهد کرد.

خواننده را تشویق می‌کنیم لحظه‌ای مکث کند و این نمودار را مطالعه کند. شکلِ این مانع در C-فضا اصلاً بدیهی نیست! نقطهٔ داخلِ شکل ۲۶.۱۲ (ب) پیکربندیِ رباتِ شکل ۲۶.۱۲ (الف) را علامت می‌زند. شکل ۲۶.۱۳ سه پیکربندیِ اضافی را، هم در فضایِ کار و هم در فضایِ پیکربندی، ترسیم می‌کند. در پیکربندیِ conf-1، گیره مانعِ عمودی را گرفته. می‌بینیم حتی اگر فضایِ کارِ ربات با چندضلعی‌های تخت بازنمایی شود، شکلِ فضایِ آزاد می‌تواند بسیار پیچیده باشد. بنابراین، در عمل، معمولاً فضایِ پیکربندی را کاوش می‌کنیم نه اینکه به‌صورتِ صریح بسازیم. برنامه‌ریز ممکن است پیکربندی‌ای تولید کند و سپس با اعمالِ سینماتیکِ ربات و بررسیِ برخوردها در مختصاتِ فضایِ کار، آزمایش کند آیا در فضایِ آزاد است یا نه.



شکل ۲۶.۱۳ – سه پیکربندیِ ربات، نشان‌داده‌شده در فضایِ کار و فضایِ پیکربندی.

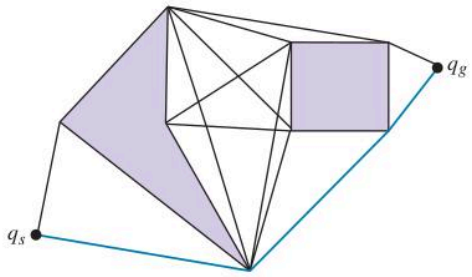
۲۶.۵.۲ برنامه‌ریزیِ حرکت

مسئلهٔ برنامه‌ریزیِ حرکت یافتنِ برنامه‌ای است که ربات را از پیکربندیِ‌ای به پیکربندیِ دیگر بدونِ برخورد با مانع می‌برد. بلوکِ ساختمانیِ پایه‌ای برای حرکت و دست‌کاری است. در بخش ۲۶.۵.۴ بحث خواهیم کرد چگونه این کار را تحتِ دینامیکِ پیچیده انجام دهیم، مانندِ هدایتِ خودرویی که ممکن است اگر در پیچ خیلی سریع برود از مسیر منحرف شود. فعلاً، بر مسئلهٔ سادهٔ برنامه‌ریزیِ حرکتِ یافتنِ مسیرِ هندسیِ عاری از برخورد تمرکز می‌کنیم. برنامه‌ریزیِ حرکت مسئلهٔ جستجویِ کاملاً حالت-پیوسته است، اما اغلب امکانِ گسسته‌سازیِ فضا و اعمالِ الگوریتم‌های جستجویِ فصل ۳ هست.

مسئلهٔ برنامه‌ریزیِ حرکت گاهی «مسئلهٔ جابه‌جاییِ پیانو» (Piano Mover's Problem) نامیده می‌شود. نامش را از کشمکشِ جابه‌جاکننده‌ای می‌گیرد که می‌خواهد پیانوی بزرگِ نامنظم‌شکلی را از اتاقی به اتاقِ دیگر بدونِ برخورد با چیزی ببرد. به ما داده می‌شود:

- فضایِ کارِ W یا در R^2 برای صفحه یا در R^3 برای سه بُعد،
- ناحیهٔ مانعِ $O \subset W$،
- رباتی با فضایِ پیکربندیِ C و مجموعهٔ نقطه‌های $A(q)$ برای $q \in C$،
- پیکربندیِ شروعِ $q_s \in C$، و
- پیکربندیِ هدفِ $q_g \in C$.

ناحیهٔ مانع، مانعِ C-فضایِ C_obs و فضایِ آزادِ متناظرِ C_free را چنان‌که در بخشِ قبلی تعریف شد القا می‌کند. باید مسیرِ پیوسته‌ای از میانِ فضایِ آزاد بیابیم. از منحنیِ پارامتریِ $\tau(t)$ برای بازنماییِ مسیر استفاده می‌کنیم، که $\tau(0) = q_s$ و $\tau(1) = q_g$ و $\tau(t)$ برای هر t بینِ ۰ و ۱ نقطه‌ای در C_free است. یعنی، t پارامترِ مقدارِ پیشرویِ ما در طولِ مسیر، از شروع تا هدف است. توجه کنید t تا حدی مانندِ زمان عمل می‌کند، زیرا با افزایشِ t فاصله در طولِ مسیر افزایش می‌یابد، اما t همیشه نقطه‌ای در بازهٔ $[0,1]$ است و بر حسبِ ثانیه اندازه‌گیری نمی‌شود.



شکل ۲۶.۱۴ – گرافِ دیدپذیری. خطوط هر جفتِ رأس را که می‌توانند یکدیگر را «ببینند»—خطوطی که از مانع عبور نمی‌کنند—به هم متصل می‌کنند. کوتاه‌ترین مسیر باید رویِ این خطوط باشد.

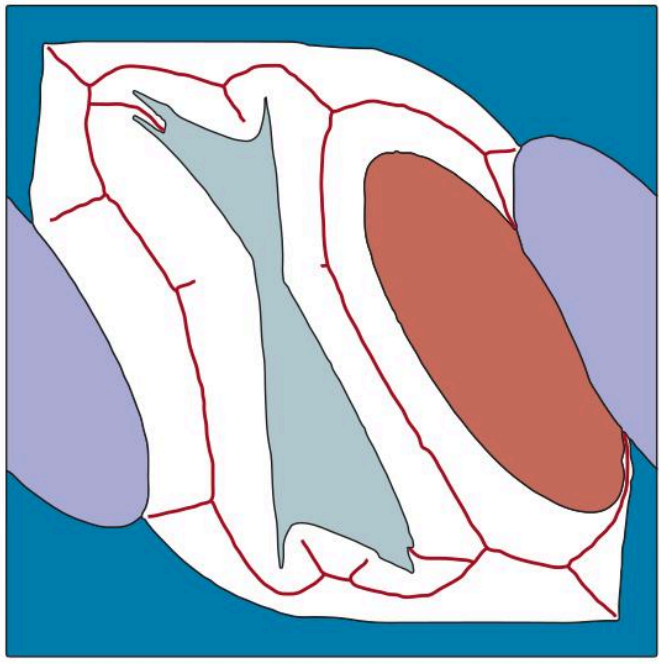
مسئلهٔ برنامه‌ریزیِ حرکت را می‌توان به‌روش‌های گوناگون پیچیده‌تر کرد: تعریفِ هدف به‌عنوانِ مجموعه‌ای از پیکربندی‌های ممکن به‌جای پیکربندیِ تکی؛ تعریفِ هدف در فضایِ کار به‌جای C-فضا؛ تعریفِ تابعِ هزینه (مثلاً، طولِ مسیر) برای کمینه‌کردن؛ ارضایِ قیدها (مثلاً، اگر مسیر شاملِ حملِ فنجانِ قهوه است، مطمئن‌شدن از اینکه فنجان همیشه راست‌به‌بالا جهت‌گیری شده تا قهوه نریزد).

فضاهای برنامه‌ریزیِ حرکت: بیایید گامی به عقب برداریم و مطمئن شویم فضاهای درگیر در برنامه‌ریزیِ حرکت را می‌فهمیم. نخست، فضایِ کار یا جهانِ W هست. نقطه‌های W نقطه‌های جهانِ روزمرهٔ سه‌بعدی‌اند. بعد، فضایِ پیکربندی‌ها، C، را داریم. نقطه‌های q در C د-بعدی‌اند، با d تعدادِ درجه‌های آزادیِ ربات، و به مجموعه‌های نقطه‌های $A(q)$ در W نگاشت می‌شوند. سرانجام، فضایِ مسیرها هست. فضایِ مسیرها فضایی از توابع است. هر نقطهٔ این فضا به کلِ منحنیِ از میانِ C-فضا نگاشت می‌شود. این فضا بینهایت-بعدی است! به‌صورتِ شهودی، برای هر پیکربندی در طولِ مسیر به d بُعد نیاز داریم، و به همان اندازه پیکربندی رویِ مسیر هست که نقطه در بازهٔ عددیِ $[0,1]$. حالا برخی راه‌های حلِ مسئلهٔ برنامه‌ریزیِ حرکت را در نظر بگیریم.

گراف‌های دیدپذیری (Visibility Graphs): برای حالتِ ساده‌شدهٔ فضاهای پیکربندیِ دوبعدی و موانعِ چندضلعیِ C-فضا، گراف‌های دیدپذیری راهِ مناسبی برای حلِ مسئلهٔ برنامه‌ریزیِ حرکت با راه‌حلِ کوتاه‌ترین-مسیرِ تضمین‌شده‌اند. $V_obs \subset C$ مجموعهٔ رأس‌های چندضلعی‌های سازندهٔ C_obs و $V = V_obs \cup \{q_s, q_g\}$ باشد. گرافی $G = (V,E)$ رویِ مجموعهٔ رأسِ V با یال‌های $e_ij \in E$ که رأسِ v_i را به رأسِ دیگری مانندِ v_j متصل می‌کند اگر خطِّ متصل‌کنندهٔ دو رأس عاری از برخورد باشد—یعنی اگر $\{\lambda v_i + (1-\lambda)v_j : \lambda \in [0,1]\} \cap C_obs = \{\}$ باشد. وقتی این رخ می‌دهد، می‌گوییم دو رأس «می‌توانند یکدیگر را ببینند»، که گراف‌های «دیدپذیری» نامشان را از همین جا گرفتند.

برای حلِ مسئلهٔ برنامه‌ریزیِ حرکت، فقط باید جستجویِ گرافِ گسسته (مثلاً، جستجویِ بهترین-اول) را رویِ گرافِ G با حالتِ شروعِ q_s و هدفِ q_g اجرا کنیم. در شکل ۲۶.۱۴ گرافِ دیدپذیری و راه‌حلِ بهینهٔ سهامرحله‌ای می‌بینیم. جستجویِ بهینه رویِ گراف‌های دیدپذیری همیشه مسیرِ بهینه را به ما می‌دهد (اگر یکی وجود داشته باشد)، یا اگر مسیری نباشد شکست را گزارش می‌کند.

نمودارهای ورونوی (Voronoi Diagrams): گراف‌های دیدپذیری مسیرهایی را تشویق می‌کنند که بلافاصله مجاورِ مانع اجرا می‌شوند—اگر مجبور بودید برای رسیدن به در دورِ میز راه بروید، کوتاه‌ترین مسیر چسبیدنِ تا حدِ ممکن به میز بود. با این حال، اگر حرکت یا حس‌کردن غیرقطعی باشد، این شما را در خطرِ برخورد با میز می‌گذارد. یکی از راه‌های پرداختن به این، وانمودکردن به اینکه بدنهٔ ربات کمی بزرگ‌تر از آنچه هست است، که ناحیهٔ بافری فراهم می‌کند. راهِ دیگر پذیرشِ این است که طولِ مسیر تنها معیاری نیست که می‌خواهیم بهینه کنیم. بخش ۲۶.۸.۲ نشان می‌دهد چگونه از مثال‌های انسانیِ رفتار معیارِ خوبی یاد بگیریم.



شکل ۲۶.۱۵ – نمودارِ ورونوی که مجموعهٔ نقطه‌های (خطوطِ سیاه) با فاصلهٔ مساوی از دو یا چند مانع در فضایِ پیکربندی را نشان می‌دهد.

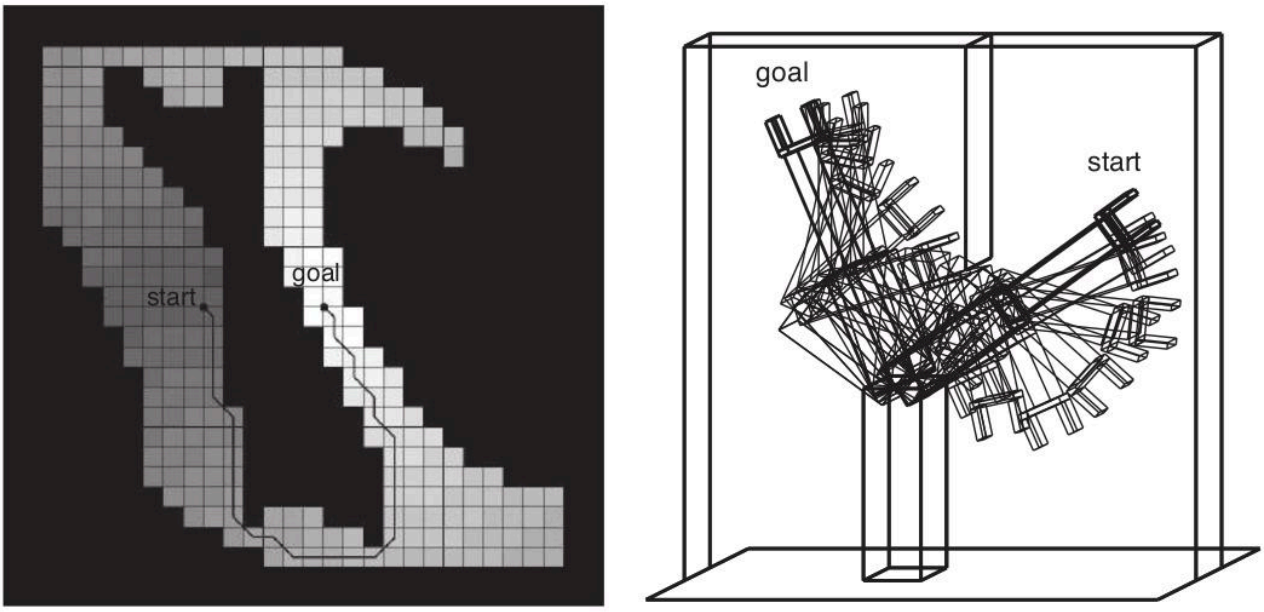
راهِ سوم استفاده از فنِ متفاوتی است، که مسیرها را تا حدِ ممکن دور از موانع می‌گذارد به‌جای چسبیدن به آن‌ها. «نمودارِ ورونوی» (Voronoi Diagram) بازنماییِ است که به ما اجازه می‌دهد دقیقاً همین کار را کنیم. برای گرفتنِ ایده‌ای از اینکه نمودارِ ورونوی چه می‌کند، فضایی را در نظر بگیر که موانع در آن، مثلاً، یک‌دوجین نقطهٔ کوچکِ پراکنده در صفحه‌اند. حالا هر نقطهٔ مانع را با ناحیه‌ای احاطه کن که شاملِ همهٔ نقطه‌های صفحه‌ای است که به آن نقطهٔ مانع نزدیک‌ترند تا به هر نقطهٔ مانعِ دیگر. پس، ناحیه‌ها صفحه را افراز می‌کنند. نمودارِ ورونوی شاملِ مجموعهٔ ناحیه‌هاست، و «گرافِ ورونوی» (Voronoi Graph) شاملِ یال‌ها و رأس‌های ناحیه‌هاست. وقتی موانع ناحیه‌اند، نه نقطه، همه‌چیز تقریباً همان می‌ماند. هر ناحیه همچنان همهٔ نقطه‌هایی را شامل است که به یک مانع نزدیک‌تر از هر مانعِ دیگرند، که فاصله به نزدیک‌ترین نقطهٔ مانع اندازه‌گیری می‌شود. مرزهای بینِ ناحیه‌ها همچنان با نقطه‌هایی متناظرند که بینِ دو مانع هم‌فاصله‌اند، اما حالا مرز ممکن است منحنی باشد نه خطِّ راست. محاسبهٔ این مرزها در فضاهای پربعد می‌تواند به‌طورِ بازدارنده‌ای گران باشد. برای حلِ مسئلهٔ برنامه‌ریزیِ حرکت، نقطهٔ شروعِ q_s را با خطِّ راستی به نزدیک‌ترین نقطهٔ گرافِ ورونوی وصل می‌کنیم، و همین کار را برای نقطهٔ هدفِ q_g. سپس از جستجویِ گرافِ گسسته برای یافتنِ کوتاه‌ترین مسیر رویِ گراف استفاده می‌کنیم. برای مسائلی مانندِ ناوبری در راهروهای داخلی، این مسیرِ زیبایی می‌دهد که از وسطِ راهرو می‌گذرد.

با این حال، در محیط‌های بیرونی می‌تواند مسیرهای ناکارآمدی ارائه دهد، مثلاً پیشنهادِ انحرافِ غیرضروریِ ۱۰۰ متری برای چسبیدن به وسطِ فضایِ کاملاً بازِ ۲۰۰ متری.

تجزیهٔ سلولی (Cell Decomposition): رویکردِ جایگزینِ برنامه‌ریزیِ حرکت گسسته‌سازیِ C-فضاست. روش‌های تجزیهٔ سلولی فضایِ آزاد را به تعدادِ متناهی ناحیهٔ پیوسته، به نامِ «سلول» (Cell)، تجزیه می‌کنند. این سلول‌ها چنان طراحی می‌شوند که مسئلهٔ برنامه‌ریزیِ مسیر درونِ سلولِ تکی با وسایلِ ساده (مثلاً، حرکت در طولِ خطِّ راست) قابلِ حل باشد. مسئلهٔ برنامه‌ریزیِ مسیر سپس به مسئلهٔ جستجویِ گرافِ گسسته تبدیل می‌شود (مانندِ گراف‌های دیدپذیری و گراف‌های ورونوی) برای یافتنِ مسیری از میانِ دنباله‌ای از سلول‌ها.

ساده‌ترین تجزیهٔ سلولی شاملِ شبکه‌ای با فاصلهٔ منظم است. شکل ۲۶.۱۶ (الف) تجزیهٔ شبکهٔ مربعیِ فضا و مسیرِ راه‌حلِ بهینهٔ این اندازهٔ شبکه را نشان می‌دهد. سایهٔ خاکستری مقدارِ هر سلولِ شبکهٔ فضایِ آزاد را نشان می‌دهد— هزینهٔ کوتاه‌ترین مسیر از آن سلول به هدف. (این مقادیر را می‌توان با شکلِ قطعیِ الگوریتمِ VALUE-ITERATION داده‌شده در شکل ۱۷.۶ در صفحهٔ ۵۷۳ محاسبه کرد.) شکل ۲۶.۱۶ (ب) مسیرِ فضایِ کارِ متناظر را برای بازو نشان می‌دهد. البته، می‌توانستیم از الگوریتمِ A* برای یافتنِ کوتاه‌ترین مسیر نیز استفاده کنیم.

این تجزیهٔ شبکه‌ای این مزیت را دارد که پیاده‌سازی‌اش ساده است، اما از سه محدودیت رنج می‌برد. نخست، فقط برای فضاهای پیکربندیِ کم‌بعد کارایی دارد، زیرا تعدادِ سلول‌های شبکه با d، تعدادِ ابعاد، به‌صورتِ نمایی افزایش می‌یابد. (آشنا به نظر می‌رسد؟ این نفرینِ ابعاد است.) دوم، مسیرهایِ از میانِ فضایِ حالتِ گسسته‌شده همیشه هموار نخواهند بود. در شکل ۲۶.۱۶ (الف) می‌بینیم بخش‌های قطریِ مسیر دندانه‌دارند و بنابراین برای ربات دنبال‌کردنِ دقیقشان بسیار دشوار است. ربات می‌تواند بکوشد مسیرِ راه‌حل را هموار کند، اما این به‌هیچ‌وجه سرراست نیست.



شکل ۲۶.۱۶ – (الف) تابعِ ارزش و مسیرِ یافت‌شده برای تقریبِ شبکهٔ سلولِ گسستهٔ فضایِ پیکربندی. (ب) همان مسیر در مختصاتِ فضایِ کار تجسم شده. توجه کنید ربات چگونه آرنجش را خم می‌کند تا از برخورد با مانعِ عمودی اجتناب کند.

سوم، مسئلهٔ کار با سلول‌های «مخلوط» هست—یعنی سلول‌هایی که نه کاملاً درونِ فضایِ آزاداند و نه کاملاً درونِ فضایِ اشغال‌شده. مسیرِ راه‌حلی که چنین سلولی را شامل می‌شود ممکن است راه‌حلِ واقعی نباشد، زیرا ممکن است هیچ راهِ امنی برای عبور از سلول نباشد. این برنامه‌ریزِ مسیر را نادرست (unsound) می‌کرد. از سوی دیگر، اگر اصرار کنیم فقط سلول‌های کاملاً آزاد استفاده شوند، برنامه‌ریز ناقص (incomplete) خواهد بود، زیرا ممکن است تنها مسیرهای هدف از سلول‌های مخلوط بگذرند—ممکن است راهرویی در واقع به‌اندازهٔ کافی برای عبورِ ربات پهن باشد، اما راهرو فقط با سلول‌های مخلوط پوشیده شده باشد.

نخستین رویکرد به این مسئله تقسیمِ بیشترِ سلول‌های مخلوط است—شاید با استفاده از سلول‌هایی با نصفِ اندازهٔ اصلی. این را می‌توان به‌صورتِ بازگشتی ادامه داد تا مسیری یافت شود که کاملاً درونِ سلول‌های آزاد قرار دارد. این روش خوب کار می‌کند و کامل است اگر راهی برای تصمیم‌گیری دربارهٔ اینکه سلولِ داده‌شده‌ای سلولِ مخلوط است یا نه وجود داشته باشد، که فقط اگر مرزهای فضایِ پیکربندی توصیفِ ریاضیِ نسبتاً ساده‌ای داشته باشند آسان است.

توجه به این نکته مهم است که تجزیهٔ سلولی الزاماً به بازنماییِ صریحِ فضایِ مانعِ C_obs نیاز ندارد. می‌توانیم تصمیم بگیریم سلولی را شامل کنیم یا نه با استفاده از «بررسی‌گرِ برخورد» (Collision Checker). این مفهومِ حیاتیِ برنامه‌ریزیِ حرکت است. بررسی‌گرِ برخورد تابعِ $\gamma(q)$ است که اگر پیکربندی با مانع برخورد کند ۱ و در غیرِ این صورت ۰ نگاشت می‌کند. بررسیِ اینکه پیکربندیِ خاصی در برخورد است بسیار آسان‌تر از ساختِ صریحِ کلِ فضایِ مانعِ C_obs است.

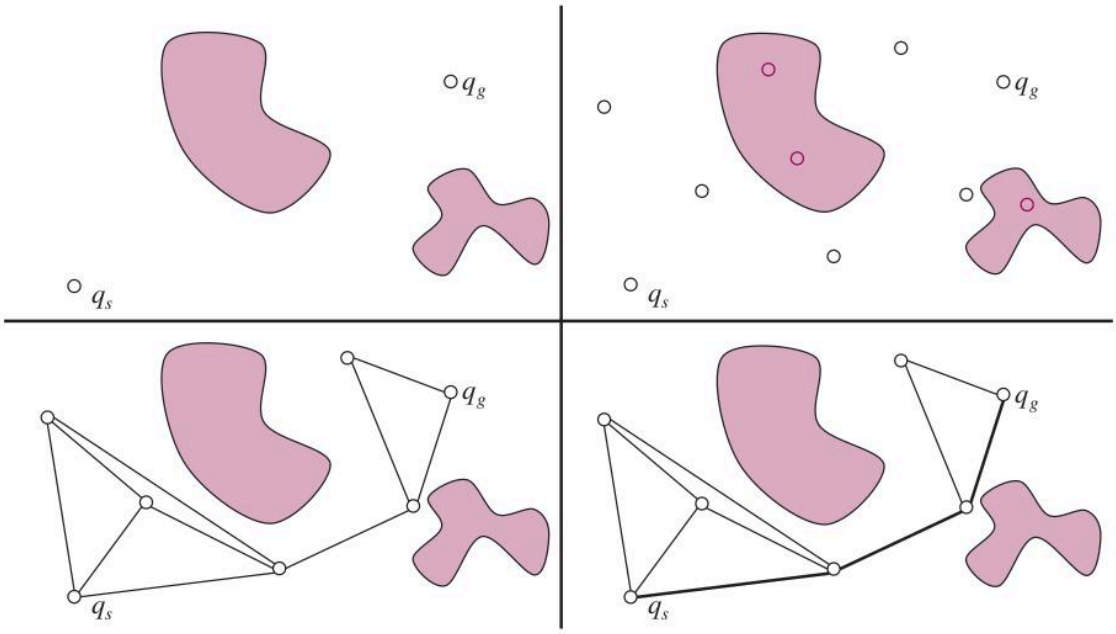
با بررسیِ مسیرِ راه‌حلِ نشان‌داده‌شده در شکل ۲۶.۱۶ (الف)، دشواریِ اضافیِ دیگری را می‌بینیم که باید حل شود. مسیر شاملِ گوشه‌های دلخواهاً تیز است، اما رباتِ فیزیکی تکانه دارد و نمی‌تواند فوراً تغییرِ جهت دهد. این مسئله را می‌توان با ذخیره‌کردن، برای هر سلولِ شبکه، حالتِ پیوستهٔ دقیق (موقعیت و سرعت) که هنگامِ رسیدن به سلول در

جستجو به‌دست آمده حل کرد. بیشتر فرض کنید هنگامِ انتشارِ اطلاعات به سلول‌های شبکهٔ نزدیک، از این حالتِ پیوسته به‌عنوانِ مبنا استفاده می‌کنیم و مدلِ حرکتِ پیوستهٔ ربات را برای پرش به سلول‌های نزدیک اعمال می‌کنیم. پس چرخشِ آنیِ ۹۰ درجه‌ای نمی‌کنیم؛ چرخشِ گردی انجام می‌دهیم که توسط قوانینِ حرکت اداره می‌شود. اکنون می‌توانیم تضمین کنیم مسیرِ حاصل هموار است و واقعاً توسط ربات قابلِ اجراست. الگوریتمی که این را پیاده‌سازی می‌کند A* هیبریدی (Hybrid A*) است.

برنامه‌ریزیِ حرکتِ تصادفی (Randomized Motion Planning): برنامه‌ریزیِ حرکتِ تصادفی جستجویِ گراف را رویِ تجزیهٔ تصادفیِ فضایِ پیکربندی انجام می‌دهد، نه تجزیهٔ سلولیِ منظم. ایدهٔ کلیدی نمونه‌برداری از مجموعهٔ تصادفیِ نقطه‌ها و ایجادِ یال بینِ آن‌ها اگر راهِ بسیار ساده‌ای برای رفتن از یکی به دیگری (مثلاً، از طریقِ خطِّ راست) بدونِ برخورد باشد؛ سپس می‌توانیم رویِ این گراف جستجو کنیم.

الگوریتمِ «نقشهٔ راهِ احتمالاتی» (Probabilistic Roadmap یا PRM) یکی از راه‌های بهره‌برداری از این ایده است. فرض می‌کنیم به بررسی‌گرِ برخوردِ γ (تعریف‌شده در صفحهٔ ۹۴۶) و به برنامه‌ریزِ ساده‌ای $B(q_1,q_2)$ دسترسی داریم که مسیری از q_1 به q_2 برمی‌گرداند (یا شکست) اما این کار را سریع انجام می‌دهد. این برنامه‌ریزِ ساده قرار نیست کامل باشد—ممکن است حتی اگر راه‌حل واقعاً وجود داشته باشد شکست برگرداند. کارش این است که سریع بکوشد q_1 و q_2 را متصل کند و به الگوریتمِ اصلی اطلاع دهد موفق شده یا نه. از آن برای تعریفِ اینکه آیا یالی بینِ دو رأس هست استفاده خواهیم کرد.

الگوریتم با نمونه‌برداری از M «نقطهٔ عطف» (Milestone)—نقطه‌هایی در C_free—علاوه بر نقطه‌های q_s و q_g شروع می‌کند. از نمونه‌برداریِ رد (Rejection Sampling) استفاده می‌کند، که پیکربندی‌ها به‌صورتِ تصادفی نمونه‌برداری و با γ بررسیِ برخورد می‌شوند تا در مجموع M نقطهٔ عطف یافت شود. بعد، الگوریتم از برنامه‌ریزِ ساده برای تلاشِ اتصالِ جفتِ نقطه‌های عطف استفاده می‌کند. اگر برنامه‌ریزِ ساده موفقیت برگرداند، یالی بینِ جفت به گراف اضافه می‌شود؛ در غیرِ این صورت، گراف همان‌گونه می‌ماند. می‌کوشیم هر نقطهٔ عطف را یا به k همسایهٔ نزدیک‌ترین‌اش (این را k-PRM می‌نامیم) یا به همهٔ نقطه‌های عطفِ کره‌ای با شعاعِ r متصل کنیم. سرانجام، الگوریتم مسیری را رویِ این گراف از q_s به q_g جستجو می‌کند. اگر مسیری یافت نشود، M نقطهٔ عطفِ بیشتر نمونه‌برداری، به گراف اضافه و فرایند تکرار می‌شود.



شکل ۲۶.۱۷ — الگوریتمِ نقشهٔ راهِ احتمالاتی (PRM). بالایِ چپ: پیکربندی‌های شروع و هدف. بالایِ راست: M نقطهٔ عطفِ عاری از برخورد نمونه‌برداری کن (اینجا M = 5). پایینِ چپ: هر نقطهٔ عطف را به k همسایهٔ نزدیک‌ترین‌اش متصل کن (اینجا k = 3). پایینِ راست: کوتاه‌ترین مسیر را از شروع به هدف رویِ گرافِ حاصل بیاب.

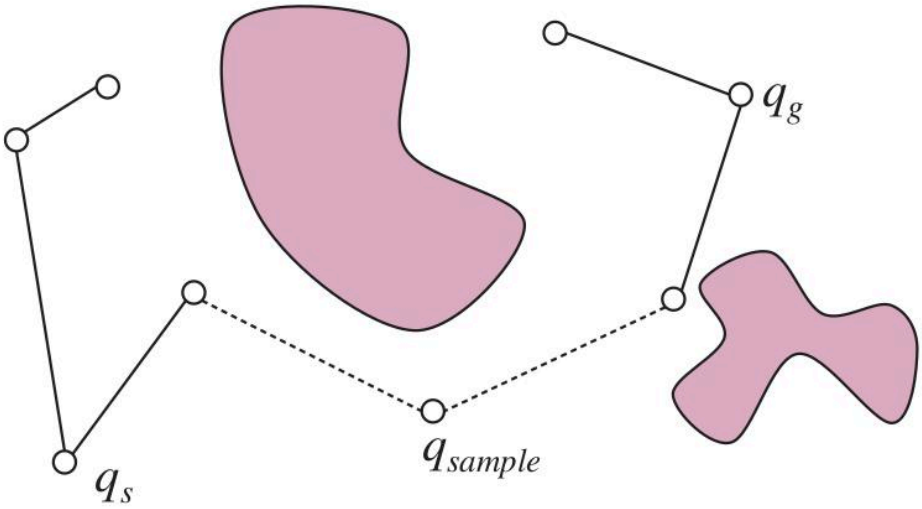
شکل ۲۶.۱۷ نقشهٔ راه را با مسیرِ یافت‌شده بینِ دو پیکربندی نشان می‌دهد. PRMها کامل نیستند، اما چیزی‌اند که «کاملِ احتمالاتی» (Probabilistically Complete) نامیده می‌شوند—اگر مسیری وجود داشته باشد، در نهایت آن را خواهند یافت. به‌صورتِ شهودی، این به‌دلیلِ این است که به نمونه‌برداری از نقطه‌های عطفِ بیشتر ادامه می‌دهند. PRMها حتی در فضاهای پیکربندیِ پربعد نیز خوب کار می‌کنند.

PRMها همچنین برای «برنامه‌ریزیِ چندپرس‌وجویی» (Multi-Query Planning) محبوب‌اند، که در آن چند مسئلهٔ برنامه‌ریزیِ حرکت درونِ همان C-فضا داریم. اغلب، وقتی ربات به هدفی می‌رسد، فراخوانده می‌شود در همان فضایِ کار به هدفِ دیگری برسد. PRMها واقعاً مفیدند، زیرا ربات می‌تواند از پیش وقت صرفِ ساختنِ نقشهٔ راه کند و استفاده از آن نقشهٔ راه را رویِ چند پرس‌وجو مستهلک کند.

درخت‌های تصادفیِ سریع‌گسترش (RRT): بسطی از PRMها به نامِ «درخت‌های تصادفیِ سریع‌گسترش» (Rapidly Exploring Random Trees یا RRT) برای برنامه‌ریزیِ تک‌پرس‌وجویی محبوب است. به‌صورتِ افزایشی دو درخت می‌سازیم، یکی با ریشهٔ q_s و یکی با ریشهٔ q_g. نقطه‌های عطفِ تصادفی انتخاب می‌شوند، و تلاش می‌شود هر نقطهٔ عطفِ تازه به درخت‌های موجود متصل شود. اگر نقطهٔ عطفی هر دو درخت را متصل کند، یعنی راه‌حلی یافت شده، مانندِ شکل ۲۶.۱۸. اگر نه، الگوریتم نزدیک‌ترین نقطه را در هر درخت می‌یابد و به درخت یالِ تازه‌ای اضافه می‌کند که از آن نقطه با فاصلهٔ δ به سمتِ نقطهٔ عطف امتداد می‌یابد.

این تمایل دارد درخت را به سمتِ بخش‌های قبلاً کاوش‌نشدهٔ فضا رشد دهد.

رباتیست‌ها RRTها را برای سهولتِ استفاده‌شان دوست دارند. با این حال، راه‌حل‌های RRT معمولاً غیربهینه‌اند و هموارى ندارند. بنابراین، RRTها اغلب با گامِ پس‌پردازشی دنبال می‌شوند. رایج‌ترینِ آن‌ها «کوتاه‌سازی» (Short-Cutting) است، که در آن به‌صورتِ تصادفی یکی از رأس‌های مسیرِ راه‌حل را انتخاب می‌کنیم و می‌کوشیم با اتصالِ همسایه‌هایش به یکدیگر (از طریقِ برنامه‌ریزِ ساده) آن را حذف کنیم. این کار را بارها برای هر تعدادِ گامی که وقتِ محاسبه داریم تکرار می‌کنیم. حتی آنگاه، مسیرها ممکن است به‌دلیلِ موقعیت‌های تصادفیِ نقطه‌های عطفِ انتخاب‌شده کمی غیرطبیعی به نظر برسند، چنان‌که در شکل ۲۶.۱۹ نشان داده شده.



شکل ۲۶.۱۸ – الگوریتمِ RRT دوسویه دو درخت می‌سازد (یکی از شروع، دیگری از هدف) با اتصالِ افزایشیِ هر نمونه به نزدیک‌ترین گره در هر درخت، اگر اتصال ممکن باشد. وقتی نمونه‌ای به هر دو درخت متصل شود، یعنی مسیرِ راه‌حلی یافته‌ایم.

شکل ۲۶.۱۹ – تصویرهایی از مسیرِ تولیدشده توسط RRT و پس‌پردازش‌شده با کوتاه‌سازی. با اجازه از آنکا دراگان.

RRT* اصلاحی بر RRT است که الگوریتم را به‌صورتِ مجانبی بهینه می‌کند: راه‌حلِ RRT* با نمونه‌برداری از نقطه‌های عطفِ بیشتر و بیشتر به راه‌حلِ بهینه همگرا می‌شود. ایدهٔ کلیدی انتخابِ نزدیک‌ترین همسایه بر مبنای مفهومی از هزینه-تا-اینجا است، نه فقط فاصله از نقطهٔ عطف، و سیم‌کشیِ مجددِ درخت، با تعویضِ والدِ رأس‌های قدیمی‌تر اگر رسیدن به آن‌ها از طریقِ نقطهٔ عطفِ تازه ارزان‌تر باشد.

بهینه‌سازیِ مسیر برای برنامه‌ریزیِ سینماتیکی: الگوریتم‌های نمونه‌برداریِ تصادفی تمایل دارند نخست مسیرِ پیچیده اما شدنی‌ای بسازند و سپس آن را بهینه کنند. بهینه‌سازیِ مسیر برعکس عمل می‌کند: با مسیرِ ساده اما غیرشدنی شروع می‌کند و سپس برای بیرون‌راندنِ آن از برخورد کار می‌کند. هدف یافتنِ مسیری است که تابعِ هزینه‌ای رویِ مسیرها را بهینه می‌کند. یعنی، می‌خواهیم تابعِ هزینهٔ J(τ) را کمینه کنیم، که τ(0) = q_s و τ(1) = q_g. J «تابعک» (Functional) نامیده می‌شود، زیرا تابعی رویِ توابع است. آرگومانِ J برابرِ τ است، که خود تابعی است: τ(t) نقطه‌ای از بازهٔ [0,1] را می‌گیرد و آن را به پیکربندی نگاشت می‌کند. تابعکِ هزینهٔ استاندارد بینِ دو جنبهٔ مهمِ حرکتِ ربات مبادله می‌کند: اجتناب از برخورد و کارایی،

```
J = J_obs + λ J_eff
```

که J_eff کارایی را اندازه می‌گیرد—طولِ مسیر—و ممکن است همواری را نیز اندازه بگیرد. راهِ مناسبی برای تعریفِ کارایی با درجهٔ دوم است: مشتقِ اولِ مجذورِ τ را انتگرال می‌گیرد (کمی بعد خواهیم دید چرا این در واقع مسیرهای کوتاه را ترغیب می‌کند):

```
J_eff = ∫₀¹ ½ ‖τ(s)‖² ds
```

برای عبارتِ مانع، فرض کنید بتوانیم فاصلهٔ d(x) از هر نقطهٔ x ∈ W تا نزدیک‌ترین لبهٔ مانع را محاسبه کنیم. این فاصله در بیرونِ موانع مثبت، در لبه ۰ و در داخل منفی است. این «میدانِ فاصلهٔ علامت‌دار» (Signed Distance Field) نامیده می‌شود. حالا می‌توانیم میدانِ هزینه‌ای در فضایِ کار تعریف کنیم، آن را c بنامیم، که داخلِ موانع هزینهٔ بالا و درستِ بیرونِ آن‌ها هزینهٔ کوچکی دارد. با این هزینه، می‌توانیم نقطه‌های فضایِ کار را واقعاً از بودنِ داخلِ موانع و نزدیکِ آن‌ها متنفر کنیم (با اجتناب از مشکلِ گرافِ دیدپذیری که همیشه کنارِ لبه‌های موانع آویزان‌اند). البته، رباتِ ما نقطه‌ای در فضایِ کار نیست، پس کارِ بیشتری داریم—باید همهٔ نقطه‌های b رویِ بدنهٔ ربات را در نظر بگیریم:

```
J_obs = ∫₀¹ ∫_b c(φ_b(τ(s))) ‖d/ds φ_b(τ(s))‖ db ds
```

این «انتگرالِ مسیر» (Path Integral) نامیده می‌شود—فقط c را در طولِ راه برای هر نقطهٔ بدنه انتگرال نمی‌گیرد، بلکه آن را در مشتق ضرب می‌کند تا هزینه نسبت به باززمان‌بندیِ مسیر ناوردا باشد. رباتی را تصور کن که از میانِ میدانِ هزینه جاروب می‌کند و با حرکتش هزینه انباشته می‌کند. صرف‌نظر از اینکه بازو چقدر سریع یا کند از میدان عبور می‌کند، باید دقیقاً همان هزینه را انباشته کند.

ساده‌ترین راهِ حلِ مسئلهٔ بهینه‌سازیِ بالا و یافتنِ مسیر، نزولِ گرادیان است. اگر تعجب می‌کنید چگونه گرادیانِ تابعک‌ها را نسبت به توابع بگیریم، چیزی به نامِ «حسابِ تغییرات» (Calculus of Variations) به کمک می‌آید. این به‌ویژه برای تابعک‌هایی به شکلِ

```
J[τ] = ∫₀¹ F(s, τ(s), τ(s)) ds
```

آسان است، که انتگرال‌های توابعی‌اند که فقط به پارامترِ s، مقدارِ تابع در s و مشتقِ تابع در s بستگی دارند. در چنین حالتی، «معادلهٔ اویلر-لاگرانژ» (Euler-Lagrange Equation) می‌گوید گرادیان به‌صورتِ زیر است:

```
τ J(s) = ∂F/∂τ(s) (s) − d/dt ∂F/∂τ̇(s) (s)_∇
```

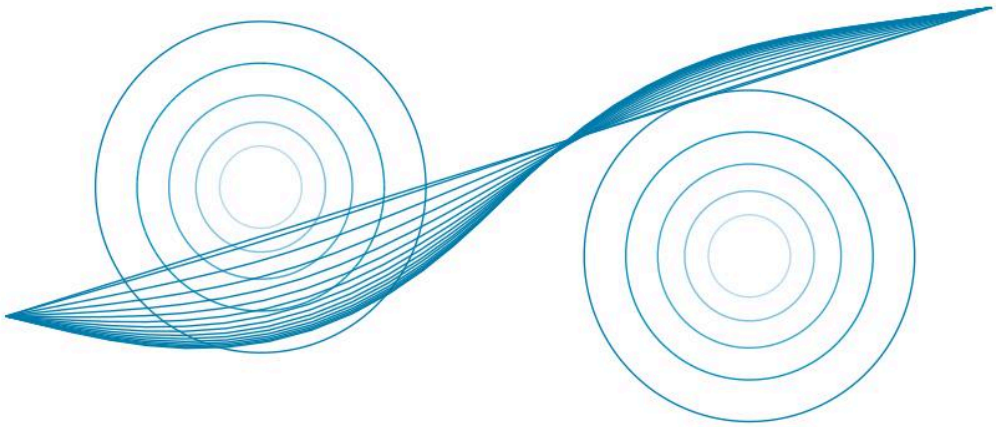
اگر به J_eff و J_obs دقیق نگاه کنیم، هر دو از این الگو پیروی می‌کنند. به‌طورِ خاص، برای J_eff داریم F(s, τ(s), τ̇(s)) = ‖τ̇(s)‖². برای راحت‌ترشدن، گرادیانِ فقط J_eff را محاسبه کنیم. می‌بینیم F وابستگیِ مستقیمی به τ(s) ندارد، پس عبارتِ نخستِ فرمول ۰ است. باقی می‌ماند:

```
τ J(s) = 0 − d/dt τ̇(s)_∇
```

زیرا جزئیِ F نسبت به τ̇(s) برابرِ τ̇(s) است.[1]

توجه کنید چگونه هنگامِ تعریفِ J_eff کار را برای خودمان آسان کردیم—درجهٔ دومِ زیبایی از مشتق است (و حتی ½ جلویش گذاشتیم تا ۲ به‌خوبی حذف شود). در عمل، این ترفند را برای بهینه‌سازی زیاد خواهید دید—هنر نه فقط در انتخابِ نحوهٔ بهینه‌کردنِ تابعِ هزینه، بلکه در انتخابِ تابعِ هزینه‌ای است که با نحوهٔ بهینه‌کردن‌تان خوب بازی کند. با ساده‌کردنِ گرادیان‌مان، τ J(s) = -τ̇(s)_∇ می‌گیریم. حالا، چون J_eff درجهٔ دوم است، صفرگذاشتنِ این گرادیان راه‌حلِ τ را به ما می‌دهد اگر مجبور نبودیم با موانع سروکار داشته باشیم. یک بار انتگرال می‌گیریم: مشتقِ نخست باید ثابت باشد؛ بارِ دوم انتگرال می‌گیریم: τ(s) = a·s + b، با a و b که توسط قیدهای نقطهٔ پایانیِ τ(0) و τ(1) تعیین می‌شوند. مسیرِ بهینه نسبت به J_eff پس خطِّ راست از شروع تا هدف است! در واقع کارآمدترین راهِ رفتن از یکی به دیگری اگر مانعی برای نگرانی نباشد همین است.

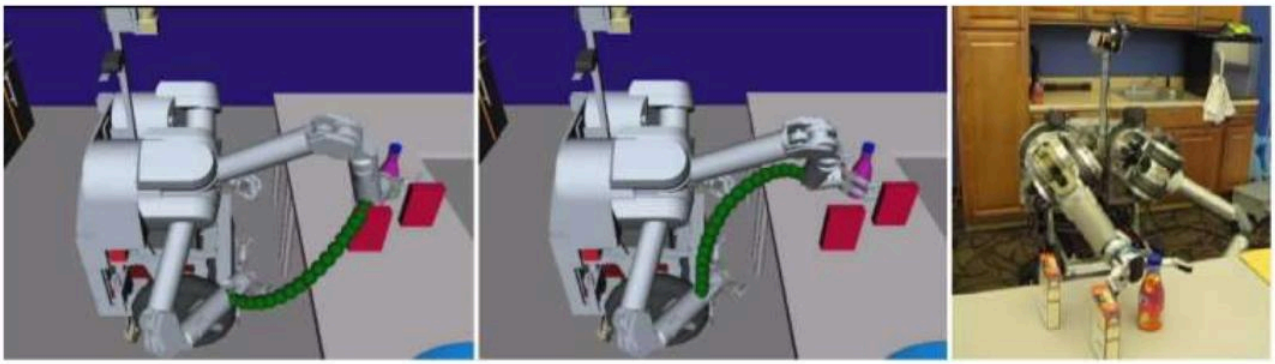
البته، افزودنِ J_obs چیزی است که کار را دشوار می‌کند—و شما را از اشتقاقِ گرادیانش در این‌جا معاف می‌کنیم. ربات معمولاً مسیرش را خطِّ راست مقداردهیِ اولیه می‌کند، که مستقیم از میانِ برخی موانع می‌شکافد. سپس گرادیانِ هزینه را دربارهٔ مسیرِ کنونی محاسبه می‌کند، و گرادیان برای دورراندنِ مسیر از موانع خدمت می‌کند (شکل ۲۶.۲۰). در ذهن داشته باشید نزولِ گرادیان فقط راه‌حلِ بهینهٔ محلی را خواهد یافت—درست مانندِ بالارفتن از تپه. روش‌هایی مانندِ بازپختِ شبیه‌سازی‌شده (بخش ۴.۱.۲) را می‌توان برای کاوش استفاده کرد، تا محتمل‌تر شود بهینهٔ محلی بهینهٔ خوبی است.



شکل ۲۶.۲۰ – بهینه‌سازیِ مسیر برای برنامه‌ریزیِ حرکت. دو مانعِ نقطه‌ای با نوارهای دایره‌ایِ هزینهٔ کاهشی دورشان. بهینه‌ساز با مسیرِ خطِّ راست شروع می‌کند و به موانع اجازه می‌دهد خط را از برخوردها خم کنند، و مسیرِ کمینه را از میانِ میدانِ هزینه می‌یابد.

۲۶.۵.۳ کنترلِ دنبال‌کردنِ مسیر

پوشش داده‌ایم چگونه حرکت‌ها را برنامه‌ریزی کنیم، اما نه چگونه واقعاً حرکت کنیم—برای اعمالِ جریان به موتورها، تولیدِ گشتاور، حرکتِ ربات. این قلمروِ «نظریهٔ کنترل» (Control Theory) است، حوزه‌ای با اهمیتِ فزاینده در هوش مصنوعی. دو پرسشِ اصلی برای رسیدگی هست: چگونه توصیفِ ریاضیِ مسیری را به دنباله‌ای از کنش‌ها در جهانِ واقعی تبدیل کنیم (کنترلِ حلقهٔ باز)، و چگونه مطمئن شویم در مسیر می‌مانیم (کنترلِ حلقهٔ بسته)؟



شکل ۲۶.۲۱ – تکلیفِ رسیدن برای گرفتنِ بطری، حل‌شده با بهینه‌سازِ مسیر. چپ: مسیرِ اولیه، ترسیم‌شده برای اثرگذارِ انتهایی. وسط: مسیرِ نهایی پس از بهینه‌سازی. راست: پیکربندیِ هدف. با اجازه از آنکا دراگان. راتلیف و همکاران (2009) را ببینید.

از پیکربندی‌ها به گشتاورها برای دنبال‌کردنِ حلقهٔ باز: مسیرِ زمانیِ $\tau(t)$ ما پیکربندی‌ها را می‌دهد. ربات در حالتِ سکون در $q_s = \tau(0)$ شروع می‌کند. از آن‌جا، موتورهای ربات جریان‌ها را به گشتاورها تبدیل می‌کنند که به حرکت می‌انجامد. اما ربات باید چه گشتاورهایی را هدف بگیرد تا در نهایت به $q_g = \tau(1)$ برسد؟ این‌جاست که ایدهٔ «مدلِ دینامیک» (Dynamics Model) (یا مدلِ انتقال) وارد می‌شود. می‌توانیم تابعِ f را به ربات بدهیم که اثرهای گشتاورها را بر پیکربندی محاسبه می‌کند. $F = ma$ از فیزیک را به یاد دارید؟ خوب، چیزی شبیهِ آن برای گشتاورها هم هست، به شکلِ $u = f^{-1}(q, \dot{q}, \ddot{q})$، با u گشتاور، $\dot{q}$ سرعت و $\ddot{q}$ شتاب.[۲] اگر ربات در پیکربندیِ q و سرعتِ $\dot{q}$ باشد و گشتاورِ u اعمال کند، آن به شتابِ $\ddot{q} = f(q, \dot{q}, u)$ می‌انجامد.

دوتاییِ $(q, \dot{q})$ «حالتِ دینامیکی» (Dynamic State) است، زیرا شاملِ سرعت است، در حالی که q «حالتِ سینماتیکی» (Kinematic State) است و برای محاسبهٔ دقیقِ اینکه چه گشتاوری اعمال شود کافی نیست. f مدلِ دینامیکِ قطعی در MDP رویِ حالت‌های دینامیکی با گشتاورها به‌عنوانِ کنش است. f^{-1} «دینامیکِ معکوس» (Inverse Dynamics) است، که می‌گوید اگر شتابِ خاصی بخواهیم چه گشتاوری اعمال کنیم، که به تغییرِ سرعت و در نتیجهِ تغییرِ حالتِ دینامیکی می‌انجامد.

حالا، به‌صورتِ ساده‌لوحانه، می‌توانستیم به $t \in [0,1]$ به‌عنوانِ «زمان» در مقیاسی از ۰ تا ۱ فکر کنیم و گشتاورِ خود را با دینامیکِ معکوس انتخاب کنیم:

$$u(t) = f^{-1}(\tau(t), \dot{\tau}(t), \ddot{\tau}(t)) \quad (26.2)$$

با فرضِ اینکه ربات از $(\tau(0), \dot{\tau}(0))$ شروع می‌کند. در واقعیت، با این حال، کارها آن‌قدرها آسان نیستند.

مسیرِ τ به‌عنوانِ دنباله‌ای از نقطه‌ها ایجاد شد، بدونِ درنظرگرفتنِ سرعت‌ها و شتاب‌ها. به‌این‌ترتیب، مسیر ممکن است $\dot{\tau}(0) = 0$ (ربات با سرعتِ صفر شروع می‌کند) را ارضا نکند، یا حتی τ مشتق‌پذیر باشد (چه برسد به دوبار مشتق‌پذیر). افزون بر این، معنای نقطهٔ پایانیِ «۱» نامشخص است: به چند ثانیه نگاشت می‌شود؟

در عمل، پیش از فکرکردن به دنبال‌کردنِ مسیرِ مرجع، معمولاً آن را «باززمان‌بندی» (Retiming) می‌کنیم، یعنی آن را به مسیرِ زمانیِ $\xi(t)$ تبدیل می‌کنیم که بازهٔ $[T,0]$ را برای مدتِ زمانیِ T به نقطه‌های فضایِ پیکربندیِ C نگاشت می‌کند. (نمادِ ξ حرفِ یونانیِ Xi است.) باززمان‌بندی از آنچه فکر می‌کنید حیله‌گرانه‌تر است، اما راه‌های تقریبی برای انجامش هست، برای نمونه با انتخابِ حداکثرِ سرعت و شتاب، و استفاده از پروفایلی که به آن حداکثر شتاب می‌گیرد، تا جایی که می‌تواند آن‌جا می‌ماند، و سپس به ۰ کاهش می‌یابد.[۲] با فرضِ توانماییِ انجامِ این، معادلهٔ (26.2) بالا را می‌توان به‌صورتِ زیر بازنویسی کرد:

$$u(t) = f^{-1}(\xi(t), \dot{\xi}(t), \ddot{\xi}(t)) \quad (26.3)$$

حتی با تغییر از τ به ξ، یک مسیرِ زمانیِ واقعی، معادلهٔ اعمالِ گشتاورها از بالا (که «قانونِ کنترل» (Control Law) نامیده می‌شود) در عمل مشکلی دارد. با فکرکردن به بخشِ یادگیریِ تقویتی، ممکن است حدس بزنید چیست. معادله در موقعیتی که f دقیق است عالی کار می‌کند، اما واقعیتِ مزاحم مثلِ همیشه سرِ راه می‌آید: در سامانه‌های واقعی، نمی‌توانیم جرم‌ها و اینرسی‌ها را دقیقاً اندازه بگیریم، و f ممکن است پدیده‌های فیزیکی مانندِ «اصطکاکِ ایستایی» (Stiction) در موتورها را به‌درستی درنظر نگیرد (اصطکاکی که تمایل دارد از به‌حرکت‌درآمدنِ سطوحِ ساکن جلوگیری کند —آن‌ها را بچسباند). پس، وقتی بازویِ ربات شروع به اعمالِ آن گشتاورها می‌کند اما f اشتباه است، خطاها انباشته می‌شوند و بیشتر و بیشتر از مسیرِ مرجع منحرف می‌شوید.

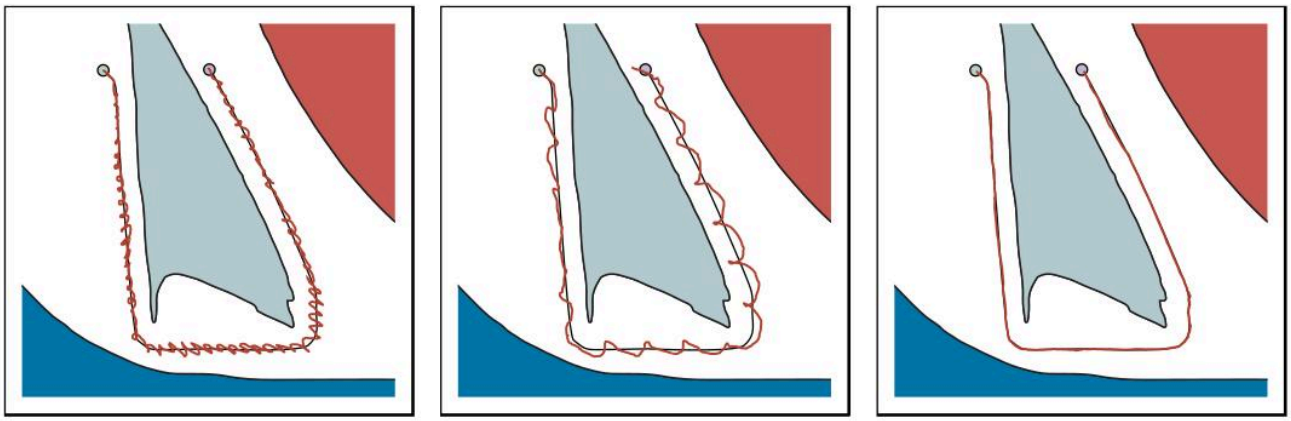
به‌جای رهاکردنِ انباشتِ این خطاها، ربات می‌تواند از فرایندِ کنترلی استفاده کند که به جایی که فکر می‌کند هست نگاه می‌کند، آن را با جایی که می‌خواست باشد مقایسه می‌کند، و گشتاوری برای کمینه‌کردنِ خطا اعمال می‌کند. کنترل‌گری که نیرو را به نسبتِ منفیِ خطایِ مشاهده‌شده فراهم می‌کند «کنترل‌گرِ تناسبی» (Proportional Controller) یا به‌طورِ

خلاصه کنترل‌گرِ P شناخته می‌شود. معادلهٔ نیرو به‌صورتِ زیر است:

$$u(t) = K_P\ (\xi(t) - q_t)$$

که q_t پیکربندیِ کنونی و K_P ثابتی است که «ضریبِ بهره» (Gain Factor) کنترل‌گر را بازنمایی می‌کند. K_P تنظیم می‌کند کنترل‌گر چقدر قوی انحراف‌های بینِ حالتِ واقعیِ q_t و حالتِ مطلوبِ $\xi(t)$ را تصحیح می‌کند.

شکل ۲۶.۲۲ (الف) نشان می‌دهد چه چیزی می‌تواند با کنترلِ تناسبی خراب شود. هرگاه انحرافی رخ می‌دهد—چه به‌دلیلِ نویز چه به‌دلیلِ قیدهای رویِ نیروهایی که ربات می‌تواند اعمال کند—ربات نیرویِ مخالفی فراهم می‌کند که اندازه‌اش متناسب با این انحراف است. به‌صورتِ شهودی، این ممکن است قابلِ‌قبول به نظر برسد، زیرا انحراف‌ها باید با نیرویِ متقابلی جبران شوند تا ربات در مسیر بماند. با این حال، چنان‌که شکل ۲۶.۲۲ (الف) نشان می‌دهد، کنترل‌گرِ تناسبی می‌تواند باعث شود ربات نیرویِ زیادی اعمال کند، از مسیرِ مطلوب بگذرد و به عقب و جلو زیگزاگ برود. این نتیجهٔ اینرسیِ طبیعیِ ربات است: پس از بازگردانده‌شدن به موقعیتِ مرجع، ربات سرعتی دارد که نمی‌توان آنی متوقف کرد.



شکل ۲۶.۲۲ – کنترلِ بازویِ رباتیک با (الف) کنترلِ تناسبی با ضریبِ بهرهٔ 1.0، (ب) کنترلِ تناسبی با ضریبِ بهرهٔ 0.1، و (ج) کنترلِ PD (تناسبی-مشتقی) با ضرایبِ بهرهٔ 0.3 برای مؤلفهٔ تناسبی و 0.8 برای مؤلفهٔ مشتقی. در همهٔ موارد بازویِ رباتیک می‌کوشد مسیرِ خطِّ صاف را دنبال کند، اما در (الف) و (ب) به‌شکلِ چشمگیری از مسیر منحرف می‌شود.

در شکل ۲۶.۲۲ (الف)، پارامترِ $K_P = 1$. در نگاهِ نخست، ممکن است فکر کنید انتخابِ مقدارِ کوچک‌تری برای K_P مشکل را درمان می‌کند، و به ربات نزدیک‌شدنِ ملایم‌تری به مسیرِ مطلوب می‌دهد. متأسفانه، این‌گونه نیست. شکل ۲۶.۲۲ (ب) مسیری برای $K_P = .1$ نشان می‌دهد، که همچنان رفتارِ نوسانی نشان می‌دهد. مقدارِ پایین‌ترِ پارامترِ بهره کمک می‌کند، اما مسئله را حل نمی‌کند. در واقع، در غیابِ اصطکاک، کنترل‌گرِ P اساساً قانونِ فنری است؛ پس تا ابد حولِ مکانِ هدفِ ثابت نوسان خواهد کرد.

تعدادی کنترل‌گر هست که از قانونِ کنترلِ تناسبیِ ساده برترند. کنترل‌گر «پایدار» (Stable) گفته می‌شود اگر اختلال‌های کوچک به خطایِ کران‌دار بینِ ربات و سیگنالِ مرجع منجر شوند. «اکیداً پایدار» (Strictly Stable) گفته می‌شود اگر بتواند پس از چنین اختلال‌هایی به مسیرِ مرجعِ خود بازگردد و رویِ آن بماند. کنترل‌گرِ P ما پایدار اما نه اکیداً پایدار به

نظر می‌رسد، زیرا از ماندنِ جایی نزدیکِ مسیرِ مرجعش ناکام می‌ماند.

ساده‌ترین کنترل‌گری که در حوزهٔ ما به پایداریِ اکید می‌رسد کنترل‌گرِ PD است. حرفِ «P» باز مخففِ تناسبی و «D» مخففِ مشتقی است. کنترل‌گرهای PD با معادلهٔ زیر توصیف می‌شوند:

$$u(t) = K_P\,(\xi(t) - q_t) + K_D\,(\dot{\xi}(t) - \dot{q}_t) \qquad (26.4)$$

چنان‌که این معادله پیشنهاد می‌کند، کنترل‌گرهای PD کنترل‌گرهای P را با مؤلفهٔ مشتقی بسط می‌دهند، که به مقدارِ u(t) عبارتی متناسب با مشتقِ نخستِ خطایِ $\xi(t) - q_t$ در طولِ زمان می‌افزاید. اثرِ چنین عبارتی چیست؟ به‌طورِ کلی، عبارتِ مشتقی سامانه‌ای را که کنترل می‌شود میرا می‌کند. برای دیدنِ این، موقعیتی را در نظر بگیر که خطا به‌سرعت در طولِ زمان تغییر می‌کند، مانندِ حالتِ کنترل‌گرِ P بالا. مشتقِ این خطا سپس با عبارتِ تناسبی مقابله می‌کند، که پاسخِ کلی به اختلال را کاهش می‌دهد. با این حال، اگر همان خطا ادامه یابد و تغییر نکند، مشتق ناپدید می‌شود و عبارتِ تناسبی انتخابِ کنترل را مسلط می‌کند.

شکل ۲۶.۲۲ (ج) نتیجهٔ اعمالِ این کنترل‌گرِ PD بر بازویِ رباتیکِ ما را نشان می‌دهد، با استفاده از پارامترهای بهرهٔ K_P = .3 و K_D = .8. واضح است، مسیرِ حاصل بسیار هموارتر است و هیچ نوسانِ آشکاری نشان نمی‌دهد.

با این حال، کنترل‌گرهای PD حالت‌های شکست دارند. به‌طورِ خاص، کنترل‌گرهای PD ممکن است در تنظیمِ خطا به صفر ناکام بمانند، حتی در غیابِ اختلال‌های خارجی. اغلب چنین موقعیتی نتیجهٔ نیرویِ خارجیِ سیستماتیکی است که بخشی از مدل نیست. برای نمونه، خودرویِ خودرانی که رویِ سطحِ شیب‌دار رانندگی می‌کند ممکن است خودش را به‌صورتِ سیستماتیک به یک سمت کشیده بیابد. فرسودگی در بازوهای رباتیک خطاهای سیستماتیکِ مشابهی ایجاد می‌کند. در چنین موقعیت‌هایی، بازخوردِ بیش‌ازحدِ تناسبی برای راندنِ خطا به نزدیک‌ترِ صفر لازم است. راه‌حلِ این مسئله در افزودنِ عبارتِ سومی به قانونِ کنترل، مبتنی بر خطایِ انتگرال‌گرفته‌شده در طولِ زمان، است:

$$u(t) = K_P\,(\xi(t) - q_t) + K_I \int_0^t (\xi(s) - q_s)\,ds + K_D\,(\dot{\xi}(t) - \dot{q}_t) \qquad (26.5)$$

در این‌جا K_I سومین پارامترِ بهره است. عبارتِ $\int_0^t (\xi(s) - q_s)\,ds$ انتگرالِ خطا در طولِ زمان را محاسبه می‌کند. اثرِ این عبارت این است که انحراف‌های طولانی‌مدت بینِ سیگنالِ مرجع و حالتِ واقعی تصحیح می‌شوند. پس، عبارت‌های انتگرالی تضمین می‌کنند کنترل‌گر خطایِ سیستماتیکِ بلندمدت نشان نمی‌دهد، هرچند خطرِ رفتارِ نوسانی را ایجاد می‌کنند.

کنترل‌گری با هر سه عبارت «کنترل‌گرِ (Proportional Integral Derivative) «PID نامیده می‌شود. کنترل‌گرهای PID به‌طورِ گسترده‌ای در صنعت، برای انواعِ مسائلِ کنترل، استفاده می‌شوند. به سه عبارت به‌صورتِ زیر فکر کنید—تناسبی: هرچه از مسیر دورتر باشی بیشتر تلاش کن؛ مشتقی: اگر خطا در حالِ افزایش است حتی بیشتر تلاش کن؛ انتگرالی: اگر مدتِ طولانی پیشرفتی نکرده‌ای بیشتر تلاش کن.

میان‌بندی بینِ کنترلِ حلقهٔ بازِ مبتنی بر دینامیکِ معکوس و کنترلِ حلقهٔ بستهٔ PID «کنترلِ گشتاورِ محاسبه‌شده» (Computed Torque Control) نامیده می‌شود. گشتاوری را محاسبه می‌کنیم که مدل‌مان فکر می‌کند به آن نیاز خواهیم داشت، اما نادقتیِ مدل را با عبارت‌های خطایِ تناسبی جبران می‌کنیم:

$$u(t) = f^{-1}(\xi(t), \dot{\xi}(t), \ddot{\xi}(t)) + m(\xi(t)) \, [K_P \, (\xi(t) - q_t) + K_D \, (\dot{\xi}(t) - \dot{q}_t)] \qquad (26.6)$$

عبارتِ نخست «مؤلفهٔ پیش‌خور» (Feedforward Component) نامیده می‌شود، زیرا به جلو نگاه می‌کند که ربات باید به کجا برود و محاسبه می‌کند چه گشتاوری ممکن است لازم باشد. عبارتِ دوم «مؤلفهٔ بازخورد» (Feedback Component) است، زیرا خطایِ کنونیِ حالتِ دینامیکی را به قانونِ کنترل بازمی‌گرداند. $m(q)$ ماتریسِ اینرسی در پیکربندیِ q است—برخلافِ کنترلِ PD معمولی، بهره‌ها با پیکربندیِ سامانه تغییر می‌کنند.

برنامه‌ها در برابرِ سیاست‌ها: بیایید گامی به عقب برداریم و مطمئن شویم قیاسِ بینِ آنچه تاکنون در این فصل رخ داده و آنچه در فصل‌های جستجو، MDP و یادگیریِ تقویتی یاد گرفتیم را می‌فهمیم. با حرکت در رباتیک، واقعاً MDP زیرینی را در نظر داریم که حالت‌ها حالت‌های دینامیکی (پیکربندی و سرعت) و کنش‌ها ورودی‌های کنترل، معمولاً به شکلِ گشتاورها، هستند. اگر دوباره به قوانینِ کنترلِ بالا نگاه کنید، آن‌ها سیاست‌اند، نه برنامه—به ربات می‌گویند از هر حالتی که ممکن است به آن برسد چه کنشی بگیرد. با این حال، آن‌ها معمولاً از سیاستِ بهینه فاصلهٔ زیادی دارند. چون حالتِ دینامیکی پیوسته و پربعد است (و فضایِ کنش نیز چنین است)، استخراجِ سیاست‌های بهینه از نظرِ محاسباتی دشوار است.

در عوض، آنچه این‌جا کردیم شکستنِ مسئله است. نخست برنامه‌ای در فضایِ حالت و کنشِ ساده‌شده ارائه می‌کنیم: فقط از حالتِ سینماتیکی استفاده می‌کنیم و فرض می‌کنیم حالت‌ها بدونِ توجه به دینامیکِ زیرین از یکدیگر قابلِ‌دستیابی‌اند. این برنامه‌ریزیِ حرکت است و مسیرِ مرجع را به ما می‌دهد. اگر دینامیک را کاملاً می‌دانستیم، می‌توانستیم این را با معادلهٔ (26.3) به برنامه‌ای برای فضایِ حالت و کنشِ اصلی تبدیل کنیم. اما چون مدلِ دینامیکِ ما معمولاً خطادار است، در عوض آن را به سیاستی تبدیل می‌کنیم که می‌کوشد برنامه را دنبال کند—وقتی منحرف می‌شود به آن بازمی‌گردد. هنگامِ انجامِ این کار، زیربهینگی را به دو صورت وارد می‌کنیم: نخست با برنامه‌ریزی بدونِ درنظرگرفتنِ دینامیک، و دوم با فرضِ اینکه اگر از برنامه منحرف شویم، کارِ بهینه بازگشت به برنامهٔ اصلی است. در ادامه، فنونی توصیف می‌کنیم که سیاست‌ها را مستقیماً رویِ حالتِ دینامیکی محاسبه می‌کنند و جداسازی را به‌کلی کنار می‌گذارند.

۲۶.۵.۴ کنترلِ بهینه

به‌جای استفاده از برنامه‌ریز برای ایجادِ مسیرِ سینماتیکی و نگرانی دربارهٔ دینامیکِ سامانه فقط پس از واقع، در این‌جا بحث می‌کنیم چگونه ممکن است بتوانیم همه‌چیز را یک‌جا انجام دهیم. مسئلهٔ بهینه‌سازیِ مسیر را برای مسیرهای سینماتیکی می‌گیریم و آن را به بهینه‌سازیِ مسیرِ واقعیِ با دینامیک تبدیل می‌کنیم: مستقیماً رویِ کنش‌ها بهینه

می‌کنیم، با درنظرگرفتنِ دینامیک (یا انتقال‌ها). این ما را بسیار به آنچه در فصل‌های جستجو و MDP دیده‌ایم نزدیک می‌کند. اگر دینامیکِ سامانه را بدانیم، می‌توانیم دنباله‌ای از کنش‌ها برای اجرا بیابیم، چنان‌که در فصل ۳ انجام دادیم. اگر مطمئن نباشیم، ممکن است سیاستی بخواهیم، مانندِ فصل ۱۷.

در این بخش، مستقیم‌تر به MDP زیرینی که ربات در آن کار می‌کند نگاه می‌کنیم. از MDPهای گسستهٔ آشنا به MDPهای پیوسته تغییر می‌کنیم. حالتِ دینامیکیِ جهان را با x نشان می‌دهیم، چنان‌که رسم است—معادلِ s در MDPهای گسسته. x_s و x_g حالت‌های شروع و هدف باشند.

می‌خواهیم دنباله‌ای از کنش‌ها بیابیم که وقتی توسط ربات اجرا می‌شوند، به جفت‌های حالت-کنش با هزینهٔ تجمعیِ کم منجر شوند. کنش‌ها گشتاورهایی‌اند که با u(t) نشان می‌دهیم، با t که از ۰ شروع و به T ختم می‌شود. به‌صورتِ صوری، می‌خواهیم دنبالهٔ گشتاورهای u را بیابیم که هزینهٔ تجمعیِ J را کمینه می‌کند:

$$\min_u \int_0^T J(x(t), u(t))\, dt \quad (26.7)$$
$$\text{subject to: } \forall t,\ \dot{x}(t) = f(x(t), u(t)),\ \ x(0) = x_s,\ \ x(T) = x_g$$

این چگونه به برنامه‌ریزیِ حرکت و کنترلِ دنبال‌کردنِ مسیر متصل می‌شود؟ خوب، تصور کنید مفهومِ کارایی و فاصله از موانع را می‌گیریم و در تابعِ هزینهٔ J می‌گذاریم، درست همان‌گونه که قبلاً در بهینه‌سازیِ مسیر رویِ حالتِ سینماتیکی انجام دادیم. حالتِ دینامیکی پیکربندی و سرعت است، و گشتاورهای u آن را از طریقِ دینامیکِ f از دنبال‌کردنِ مسیرِ حلقهٔ باز تغییر می‌دهند. تفاوت این است که حالا به پیکربندی‌ها و گشتاورها هم‌زمان فکر می‌کنیم. گاهی، ممکن است بخواهیم اجتناب از برخورد را نیز به‌عنوانِ قیدِ سخت درمان کنیم، چیزی که قبلاً نیز وقتی به بهینه‌سازیِ مسیرِ فقط برای حالتِ سینماتیکی نگاه کردیم اشاره کردیم.

برای حلِ این مسئلهٔ بهینه‌سازی، می‌توانیم گرادیان‌های J را بگیریم—نه دیگر نسبت به دنبالهٔ τ از پیکربندی‌ها، بلکه مستقیماً نسبت به کنترل‌های u. گاهی کمک‌کننده است دنبالهٔ حالتِ x را نیز به‌عنوانِ متغیرِ تصمیم شامل کنیم، و از قیدهای دینامیک برای اطمینان از سازگاریِ x و u استفاده کنیم. فنونِ بهینه‌سازیِ مسیرِ گوناگونی با این رویکرد هست؛ دو تا از آن‌ها «شلیکِ چندگانه» (Multiple Shooting) و «هم‌مکان‌سازیِ مستقیم» (Direct Collocation) نامیده می‌شوند. هیچ‌کدام از این فنون راه‌حلِ بهینهٔ سراسری را نخواهند یافت، اما در عمل می‌توانند ربات‌های انسان‌نمایِ راه‌رفته و خودروهای خودرانِ رانده بسازند.

جادو وقتی رخ می‌دهد که در مسئلهٔ بالا، J درجهٔ دوم و f در x و u خطی باشد. می‌خواهیم کمینه کنیم:

$$\min_u \int_0^\infty (x(t)^T\, Q\, x(t) + u(t)^T\, R\, u(t))\, dt$$
$$\text{subject to: } \forall t,\ \dot{x}(t) = A\, x(t) + B\, u(t)$$

می‌توانیم رویِ افقِ بی‌نهایت به‌جای افقِ متناهی بهینه کنیم، و سیاستی از هر حالت می‌گیریم نه فقط دنباله‌ای از کنترل‌ها. Q و R برای کارکردنِ این باید ماتریس‌های معینِ مثبت باشند. این «تنظیم‌کنندهٔ خطیِ درجهٔ دوم» (Linear Quadratic Regulator یا LQR) را به ما می‌دهد. با LQR، تابعِ ارزشِ بهینه (که «هزینه-تا-آینده» نامیده می‌شود) درجهٔ

دوم و سیاستِ بهینه خطی است. سیاست به شکلِ u = -Kx است، که یافتنِ ماتریسِ K مستلزمِ حلِ «معادلهٔ جبریِ ریکاتی» (Riccati Equation) است—بدونِ بهینه‌سازیِ محلی، بدونِ تکرارِ ارزش، بدونِ تکرارِ سیاست!

به‌دلیلِ سهولتِ یافتنِ سیاستِ بهینه، LQR کاربردهای بسیاری در عمل می‌یابد، با وجودِ اینکه مسائلِ واقعی به‌ندرت هزینهٔ درجهٔ دوم و دینامیکِ خطی دارند. روشِ واقعاً مفیدی «LQR تکراری» (Iterative LQR یا ILQR) نامیده می‌شود، که با شروع از راه‌حلی و سپس محاسبهٔ تکرارِیِ تقریبِ خطیِ دینامیک و تقریبِ درجهٔ دومِ هزینه دورِ آن، و سپس حلِ سامانهٔ LQR حاصل برای رسیدن به راه‌حلِ تازه کار می‌کند. گونه‌های LQR نیز اغلب برای دنبال‌کردنِ مسیر استفاده می‌شوند.

۲۶.۶ برنامه‌ریزیِ حرکت‌های نامطمئن

در رباتیک، عدم‌قطعیت از مشاهدهٔ جزئیِ محیط و از اثرهای تصادفی (یا مدل‌نشدهٔ) کنش‌های ربات ناشی می‌شود. خطاها همچنین می‌توانند از استفاده از الگوریتم‌های تقریبی مانندِ فیلترِ ذره‌ای ناشی شوند، که حتی اگر محیط کاملاً مدل شده باشد حالتِ باورِ دقیقی به ربات نمی‌دهد.

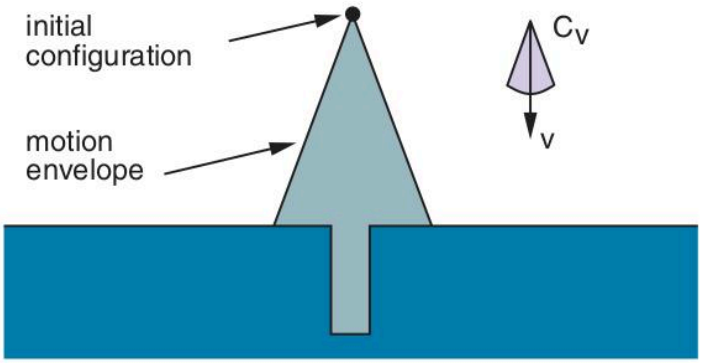
اکثریتِ ربات‌های امروز از الگوریتم‌های قطعی برای تصمیم‌گیری استفاده می‌کنند، مانندِ الگوریتم‌های برنامه‌ریزیِ مسیرِ بخشِ قبلی، یا الگوریتم‌های جستجویِ معرفی‌شده در فصل ۳. این الگوریتم‌های قطعی به دو صورت سازگار می‌شوند: نخست، با تبدیلِ فضایِ حالتِ پیوسته به فضایِ گسسته (مثلاً با گراف‌های دیدپذیری یا تجزیهٔ سلولی) با آن سروکار دارند. دوم، با انتخابِ محتمل‌ترین حالت از توزیعِ احتمالیِ تولیدشده توسط الگوریتمِ تخمینِ حالت با عدم‌قطعیتِ حالتِ کنونی سروکار دارند. آن رویکرد محاسبه را سریع‌تر می‌کند و برای الگوریتم‌های جستجویِ قطعی تناسبِ بهتری دارد. در این بخش روش‌های سروکارداشتن با عدم‌قطعیت را بحث می‌کنیم که مشابهِ الگوریتم‌های جستجویِ پیچیده‌ترِ پوشش‌داده‌شده در فصل ۴ هستند.

نخست، به‌جای برنامه‌های قطعی، عدم‌قطعیت خواهانِ سیاست‌هاست. قبلاً بحث کردیم چگونه کنترلِ دنبال‌کردنِ مسیر برنامه را به سیاستی برای جبرانِ خطاهای دینامیک تبدیل می‌کند. گاهی، با این حال، اگر محتمل‌ترین فرضیه به‌اندازهٔ کافی تغییر کند، دنبال‌کردنِ برنامه‌ای که برای فرضیه‌یِ متفاوتی طراحی شده بیش‌ازحد زیربهینه است. این‌جاست که «بازبرنامه‌ریزیِ برخط» (Online Replanning) وارد می‌شود: می‌توانیم بر مبنای باورِ تازه برنامهٔ تازه‌ای دوباره محاسبه کنیم. بسیاری از ربات‌های امروز از فنّی به نامِ «کنترلِ پیش‌بینِ مدل» (Model Predictive Control یا MPC) استفاده می‌کنند، که در آن برای افقِ زمانیِ کوتاه‌تری برنامه‌ریزی می‌کنند، اما در هر گامِ زمانی بازبرنامه‌ریزی می‌کنند. (MPC بنابراین با جستجویِ زمان-واقعی و الگوریتم‌های بازی‌کردن پیوندی نزدیک دارد.) این به‌طورِ مؤثر به سیاستی می‌انجامد: در هر گام، برنامه‌ریزی اجرا می‌کنیم و نخستین کنشِ برنامه را می‌گیریم؛ اگر اطلاعاتِ تازه‌ای برسد، یا به جایی که انتظار داشتیم نرسیم، اشکالی ندارد، زیرا به هر حال بازبرنامه‌ریزی خواهیم کرد و آن به ما خواهد گفت بعد چه کنیم.

دوم، عدم‌قطعیت خواهانِ کنش‌های جمع‌آوریِ اطلاعات است. وقتی فقط اطلاعاتی را که داریم در نظر می‌گیریم و بر مبنای آن برنامه‌ای می‌سازیم (این «جداسازیِ تخمین از کنترل» نامیده می‌شود)، در واقع در هر گام MDP تازه‌ای را (تقریباً) حل می‌کنیم، متناظر با باورِ کنونیِ ما دربارهٔ کجابودن یا نحوهٔ کارکردنِ جهان. اما در واقعیت، عدم‌قطعیت بهتر توسط چارچوبِ POMDP درگیر می‌شود: چیزی هست که مستقیماً مشاهده نمی‌کنیم، چه مکان یا پیکربندیِ ربات باشد، چه مکانِ اشیای جهان، چه پارامترهای خودِ مدلِ دینامیک—برای نمونه، مرکزِ جرمِ پیوندِ دومِ این بازو دقیقاً کجاست؟ آنچه وقتی POMDP را حل نمی‌کنیم از دست می‌دهیم تواناییِ استدلال دربارهٔ اطلاعاتِ آینده‌ای است که ربات خواهد گرفت: در MDPها فقط با آنچه می‌دانیم برنامه‌ریزی می‌کنیم، نه با آنچه ممکن است در نهایت بدانیم. ارزشِ اطلاعات را به یاد دارید؟ خوب، ربات‌هایی که با استفاده از باورِ کنونی‌شان برنامه‌ریزی می‌کنند، انگار هرگز چیزِ بیشتری نخواهند فهمید، در محاسبهٔ ارزشِ اطلاعات ناکام‌اند. آن‌ها هرگز کنش‌هایی نمی‌گیرند که طبقِ آنچه می‌دانند اکنون زیربهینه به نظر می‌رسند، اما در واقع به اطلاعاتِ زیادی می‌انجامند و ربات را قادر به خوب‌عمل‌کردن می‌کنند.

چنین کنشی برای رباتِ ناوبر چگونه به نظر می‌رسد؟ ربات می‌توانست به نقطهٔ نشانه‌ای نزدیک شود تا برآوردِ بهتری از کجابودنش بگیرد، حتی اگر آن نقطهٔ نشانه طبقِ آنچه اکنون می‌داند بیرون از راه باشد. این کنش فقط اگر ربات مشاهده‌های تازه‌ای را که خواهد گرفت در نظر بگیرد بهینه است، در مقابلِ نگاه‌کردنِ فقط به اطلاعاتی که از قبل دارد.

برای دورزدنِ این، فنونِ رباتیک گاهی کنش‌های جمع‌آوریِ اطلاعات را به‌صورتِ صریح تعریف می‌کنند—مانندِ حرکت‌دادنِ دست تا وقتی سطحی لمس شود (که «حرکتِ محافظت‌شده» (Guarded Movement) نامیده می‌شود)—و مطمئن می‌شوند ربات پیش از ارائهٔ برنامه‌ای برای رسیدن به هدفِ واقعی‌اش این کار را انجام می‌دهد.



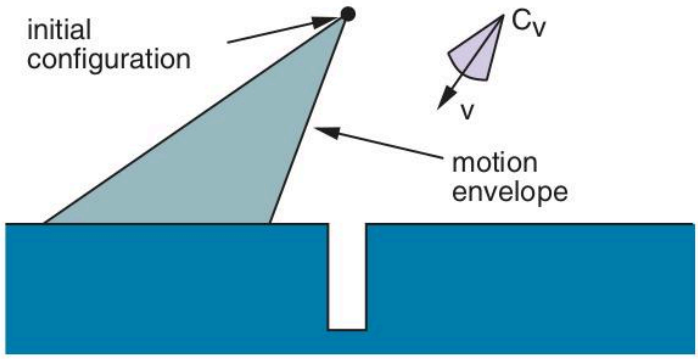
شکل ۲۶.۲۳ – محیطِ دوبعدی، مخروطِ عدم‌قطعیتِ سرعت، و پوششِ حرکت‌های ممکنِ ربات. سرعتِ موردنظر ۷ است، اما با عدم‌قطعیت، سرعتِ واقعی می‌تواند هر جایی در C_۷ باشد، که به پیکربندیِ نهاییِ جایی در پوششِ حرکت می‌انجامد، یعنی نمی‌دانستیم به سوراخ خورده‌ایم یا نه.

هر حرکتِ محافظت‌شده شاملِ (۱) فرمانِ حرکتی و (۲) شرطِ خاتمه است، که گزاره‌ای رویِ مقادیرِ حس‌گذارِ ربات است که می‌گوید چه‌وقت متوقف شو.

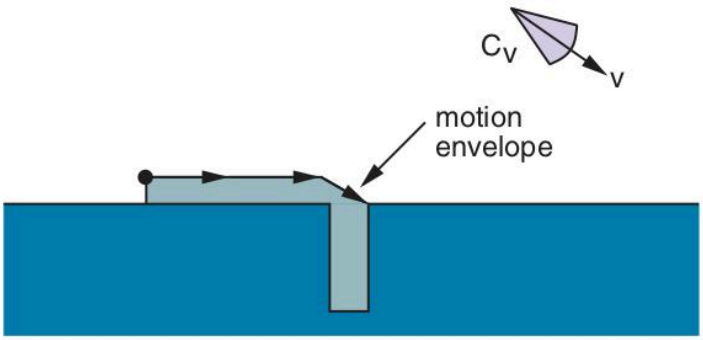
گاهی، خودِ هدف را می‌توان با دنباله‌ای از حرکت‌های محافظت‌شده به‌دست آورد که بدونِ توجه به عدم‌قطعیت تضمینِ موفقیت دارند. به‌عنوانِ مثال، شکل ۲۶.۲۳ فضایِ پیکربندیِ دوبعدی با سوراخِ عمودیِ باریکی را نشان می‌دهد. این می‌تواند فضایِ پیکربندیِ فروکردنِ میخِ مستطیلی در سوراخی یا کلیدِ خودرو در استارت باشد. فرمان‌های حرکتی

سرعت‌های ثابت‌اند. شرط‌های خاتمه تماس با سطح است. برای مدل‌کردنِ عدم‌قطعیت در کنترل، فرض می‌کنیم به‌جای حرکت در جهتِ فرمان‌داده‌شده، حرکتِ واقعیِ ربات در مخروطِ C_v دورِ آن قرار دارد.

شکل نشان می‌دهد اگر ربات می‌کوشید مستقیم از پیکربندیِ اولیه به پایین حرکت کند چه می‌شد. به‌دلیلِ عدم‌قطعیتِ سرعت، ربات می‌توانست هر جایی در پوششِ مخروطی حرکت کند، احتمالاً داخلِ سوراخ برود، اما محتمل‌تر است در یک سمتِ آن فرود آید. چون ربات سپس نمی‌دانست در کدام سمتِ سوراخ است، نمی‌دانست به کدام سمت حرکت کند. راهبردِ منطقی‌تری در شکل‌های ۲۶.۲۴ و ۲۶.۲۵ نشان داده شده. در شکل ۲۶.۲۴، ربات عامدانه به یک سمتِ سوراخ حرکت می‌کند. فرمانِ حرکت در شکل نشان داده شده، و آزمونِ خاتمه تماس با هر سطحی است. در شکل ۲۶.۲۵، فرمانِ حرکتی داده می‌شود که باعث می‌شود ربات در طولِ سطح بلغزد و واردِ سوراخ شود. چون همهٔ سرعت‌های ممکنِ پوششِ حرکت به سمتِ راست‌اند، ربات هرگاه با سطحِ افقی در تماس باشد به راست خواهد لغزید. وقتی لبهٔ عمودیِ راستِ سوراخ را لمس کند به پایین خواهد لغزید، زیرا همهٔ سرعت‌های ممکن نسبت به سطحِ عمودی رو به پایین‌اند. به حرکت ادامه می‌دهد تا به کفِ سوراخ برسد، زیرا شرطِ خاتمه‌اش همین است. با وجودِ عدم‌قطعیتِ کنترل، همهٔ مسیرهای ممکنِ ربات در تماس با کفِ سوراخ خاتمه می‌یابند—یعنی، مگر اینکه بی‌نظمی‌های سطح باعث شوند ربات در جایی بچسبد.



شکل ۲۶.۲۴ – نخستین فرمانِ حرکت و پوششِ حاصل از حرکت‌های ممکنِ ربات. صرف‌نظر از حرکتِ واقعیِ پیش‌آمده، می‌دانیم پیکربندیِ نهایی در چپِ سوراخ خواهد بود.



شکل ۲۶.۲۵ – دومین فرمانِ حرکت و پوششِ حرکت‌های ممکن. حتی با خطا، در نهایت واردِ سوراخ خواهیم شد.

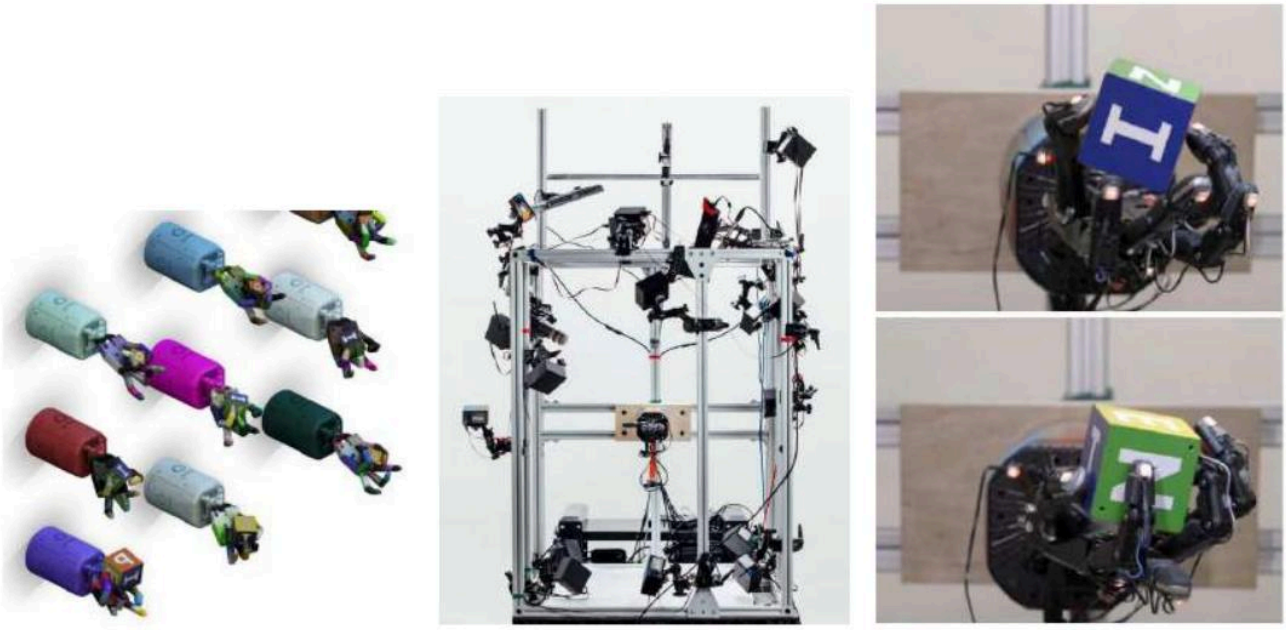
فنونِ دیگرِ فراتر از حرکت‌های محافظت‌شده تابعِ هزینه را تغییر می‌دهند تا کنش‌هایی را که می‌دانیم به اطلاعات می‌انجامند ترغیب کنند—مانندِ «اکتشافیِ ناوبریِ ساحلی» (Coastal Navigation) که ربات را ملزم می‌کند نزدیکِ نقطه‌های نشانهٔ شناخته‌شده بماند. به‌طورِ کلی‌تر، فنون می‌توانند «بهرهٔ اطلاعاتِ موردانتظار» (کاهشِ آنتروپیِ باور) را به‌عنوانِ عبارتی در تابعِ هزینه بگنجانند، که به استدلالِ صریحِ ربات دربارهٔ اینکه هر کنش چه مقدارِ اطلاعاتی ممکن است بیاورد هنگامِ تصمیم‌گیری می‌انجامد. هرچند از نظرِ محاسباتی دشوارتر، چنین رویکردهایی این مزیت را دارند که ربات کنش‌های جمع‌آوریِ اطلاعاتِ خودش را ابداع می‌کند، به‌جای تکیه بر اکتشافی‌ها و راهبردهای اسکریپت‌شدهٔ ارائه‌شده توسط انسان که اغلب فاقدِ انعطاف‌اند.

۲۶.۷ یادگیریِ تقویتی در رباتیک

تاکنون تکلیف‌هایی را در نظر گرفته‌ایم که ربات در آن‌ها به مدلِ دینامیکِ جهان دسترسی دارد. در بسیاری از تکلیف‌ها، نوشتنِ چنین مدلی بسیار دشوار است، که ما را به قلمروِ یادگیریِ تقویتی (RL) می‌برد.

یکی از چالش‌های RL در رباتیک ماهیتِ پیوستهٔ فضاهای حالت و کنش است، که یا از طریقِ گسسته‌سازی، یا معمولاً بیشتر، از طریقِ تقریبِ تابع با آن سروکار داریم. سیاست‌ها یا توابعِ ارزش به‌عنوانِ ترکیب‌هایی از ویژگی‌های شناخته‌شدهٔ مفید، یا به‌عنوانِ شبکه‌های عصبیِ عمیق بازنمایی می‌شوند. شبکه‌های عصبی می‌توانند مستقیماً از ورودی‌های خام به خروجی‌ها نگاشت کنند، و بنابراین تا حدِ زیادی از نیاز به مهندسیِ ویژگی اجتناب می‌کنند، اما به دادهٔ بیشتری نیاز دارند.

چالشِ بزرگ‌تر این است که ربات‌ها در جهانِ واقعی عمل می‌کنند. دیده‌ایم چگونه یادگیریِ تقویتی را می‌توان برای یادگیریِ بازیِ شطرنج یا گو با بازیِ شبیه‌سازی‌شده استفاده کرد. اما وقتی رباتِ واقعی در جهانِ واقعی حرکت می‌کند، باید مطمئن شویم کنش‌هایش ایمن‌اند (چیزها می‌شکنند!)، و باید بپذیریم پیشرفت کندتر از شبیه‌سازی خواهد بود، زیرا جهان حاضر نیست سریع‌تر از یک ثانیه در ثانیه حرکت کند. بخشِ زیادی از آنچه در استفاده از یادگیریِ تقویتی در رباتیک جالب است به این خلاصه می‌شود که چگونه ممکن است «پیچیدگیِ نمونهٔ جهانِ واقعی»—تعدادِ برهم‌کنش‌های با جهانِ فیزیکی که ربات پیش از یادگیریِ نحوهٔ انجامِ تکلیف به آن نیاز دارد—را کاهش دهیم.



شکل ۲۶.۲۶ – آموزشِ سیاستِ مقاوم. (الف) چندین شبیه‌سازی از دستِ رباتی که اشیا را دست‌کاری می‌کند، با پارامترهای تصادفی‌شدهٔ متفاوت برای فیزیک و نورپردازی، اجرا می‌شود. با اجازه از وویچک زارمبا. (ب) محیطِ جهانِ واقعی، با دستِ رباتیِ تکی در مرکزِ قفس، احاطه‌شده با دوربین‌ها و فاصله‌یاب‌ها. (ج) آموزشِ شبیه‌سازی و جهانِ واقعی سیاست‌های متفاوتِ متعددی برای گرفتنِ اشیا می‌دهد؛ در این‌جا یک گیرهٔ نیشگونی و یک گیرهٔ چهارپایه. با اجازه از OpenAI. آندری‌خویچ و همکاران (2018a) را ببینید.

۲۶.۷.۱ بهره‌برداری از مدل‌ها

راهِ طبیعیِ اجتناب از نیاز به نمونه‌های جهانِ واقعیِ بسیار، استفاده از هرچه بیشترِ دانشِ دینامیکِ جهان است. برای نمونه، ممکن است دقیقاً ندانیم ضریبِ اصطکاک یا جرمِ شیء چیست، اما ممکن است معادله‌هایی داشته باشیم که دینامیک را به‌عنوانِ تابعی از این پارامترها توصیف می‌کنند.

در چنین حالتی، یادگیریِ تقویتیِ مبتنی بر مدل (فصل ۲۲) جذاب است، که ربات می‌تواند بینِ برازشِ پارامترهای دینامیک و محاسبهٔ سیاستِ بهتر تناوب کند. حتی اگر معادله‌ها نادرست باشند، زیرا در مدل‌کردنِ هر جزئیاتِ فیزیک ناکام‌اند، پژوهشگران با یادگیریِ عبارتِ خطایی، علاوه بر پارامترها، آزمایش کرده‌اند که می‌تواند نادقتیِ مدلِ فیزیکی را جبران کند. یا، می‌توانیم معادله‌ها را رها کنیم و در عوض مدل‌های محلیِ خطیِ جهان را برازش کنیم که هر یک دینامیک را در ناحیه‌ای از فضایِ حالت تقریب می‌زنند، رویکردی که در واداشتنِ ربات‌ها به تسلط بر تکلیف‌های دینامیکیِ پیچیده مانندِ ژانگولربازی موفق بوده.

مدلی از جهان همچنین می‌تواند در کاهشِ پیچیدگیِ نمونهٔ روش‌های یادگیریِ تقویتیِ بدونِ مدل با انجامِ «انتقالِ شبیه-به-واقعیت» (Sim-to-Real) مفید باشد: انتقالِ سیاست‌هایی که در شبیه‌سازی کار می‌کنند به جهانِ واقعی. ایده استفاده از مدل به‌عنوانِ شبیه‌ساز برای جستجویِ سیاست (بخش ۲۲.۵) است. برای یادگیریِ سیاستی که خوب انتقال می‌یابد، می‌توانیم در طولِ آموزش به مدل نویز اضافه کنیم، و سیاست را مقاوم‌تر کنیم. یا، می‌توانیم سیاست‌هایی آموزش دهیم که با انواعِ مدل‌ها کار می‌کنند، با نمونه‌برداری از پارامترهای متفاوت در شبیه‌سازی‌ها—که

گاهی «تصادفی‌سازیِ حوزه» (Domain Randomization) نامیده می‌شود. نمونه‌ای در شکل ۲۶.۲۶ است، که در آن تکلیفِ دست‌کاریِ چابک با تغییرِ ویژگی‌های بصری، و همچنین ویژگی‌های فیزیکی مانندِ اصطکاک یا میراکنندگی، در شبیه‌سازی آموزش می‌بینند.

سرانجام، رویکردهای هیبریدی که از هر دوِ الگوریتم‌های مبتنی بر مدل و بدونِ مدل ایده وام می‌گیرند، قرار است بهترین هر دو را به ما بدهند. رویکردِ هیبریدی با معماریِ Dyna آغاز شد، که ایده در آن تناوب بینِ عمل‌کردن و بهبودِ سیاست بود، اما بهبودِ سیاست به دو شکلِ مکمل می‌آمد: (۱) راهِ استانداردِ بدونِ مدلِ استفاده از تجربه برای به‌روزرسانیِ مستقیمِ سیاست، و (۲) راهِ مبتنی بر مدلِ استفاده از تجربه برای برازشِ مدل، سپس برنامه‌ریزی با آن برای تولیدِ سیاست.

فنونِ جدیدتر با برازشِ مدل‌های محلی، برنامه‌ریزی با آن‌ها برای تولیدِ کنش‌ها، و استفاده از این کنش‌ها به‌عنوانِ نظارت برای برازشِ سیاست، سپس تکرار برای گرفتنِ مدل‌های بهتر و بهتر دورِ نواحی‌ای که سیاست نیاز دارد، آزمایش کرده‌اند. این با موفقیت در یادگیریِ سرتاسری اعمال شده، جایی که سیاست پیکسل‌ها را به‌عنوانِ ورودی می‌گیرد و مستقیماً گشتاورها را به‌عنوانِ کنش خروجی می‌دهد—نخستین نمایشِ یادگیریِ تقویتیِ عمیق رویِ ربات‌های فیزیکی را ممکن کرد.

مدل‌ها همچنین می‌توانند برای هدفِ تضمینِ کاوشِ ایمن بهره‌برداری شوند. یادگیریِ آهسته اما ایمن ممکن است بهتر از یادگیریِ سریع اما تصادف‌کردن و سوختن در نیمهٔ راه باشد. پس، به‌جرئت می‌توان گفت مهم‌تر از کاهشِ نمونه‌های جهانِ واقعی، کاهشِ نمونه‌های جهانِ واقعی در حالت‌های خطرناک است—نمی‌خواهیم ربات‌ها از صخره‌ها بیفتند، و نمی‌خواهیم لیوان‌های محبوب‌مان را بشکنند یا بدتر، با اشیا و مردم برخورد کنند. مدلِ تقریبی، با عدم‌قطعیتِ مرتبط (مثلاً با درنظرگرفتنِ دامنه‌ای از مقادیر برای پارامترهایش)، می‌تواند کاوش را هدایت کند و قیدهایی بر کنش‌هایی که ربات مجاز به گرفتنشان است اعمال کند تا از این حالت‌های خطرناک اجتناب شود. این حوزهٔ پژوهشیِ فعالی در رباتیک و کنترل است.

۲۶.۷.۲ بهره‌برداری از اطلاعاتِ دیگر

مدل‌ها مفیدند، اما کارِ بیشتری برای کاهشِ بیشترِ پیچیدگیِ نمونه می‌توانیم انجام دهیم. هنگامِ برپاکردنِ مسئلهٔ یادگیریِ تقویتی، باید فضاهای حالت و کنش، بازنماییِ سیاست یا تابعِ ارزش، و تابعِ پاداشی که استفاده می‌کنیم را انتخاب کنیم. این تصمیم‌ها اثرِ بزرگی بر اینکه مسئله را چقدر آسان یا دشوار می‌کنیم دارند.

یکی از رویکردها استفاده از «اولیه‌های حرکتِ» (Motion Primitive) سطح‌بالاتر به‌جای کنش‌های سطح‌پایین مانندِ فرمان‌های گشتاور است. اولیهٔ حرکتی مهارتِ پارامتریشده‌ای است که ربات دارد. برای نمونه، بازیکنِ فوتبالِ رباتیک ممکن است مهارتِ «پاسِ توپ به بازیکنِ در (x,y)» را داشته باشد. تنها کاری که سیاست باید بکند فهمیدنِ نحوهٔ ترکیبِ آن‌ها و تنظیمِ پارامترهایشان است، به‌جای ابداعِ دوبارهٔ آن‌ها. این رویکرد اغلب بسیار سریع‌تر از رویکردهای سطح‌پایین یاد می‌گیرد، اما فضایِ رفتارهای ممکنِ یادگیریِ ربات را محدود می‌کند.

راهِ دیگرِ کاهشِ تعدادِ نمونه‌های جهانِ واقعیِ لازم برای یادگیری، استفادهٔ مجدد از اطلاعاتِ دوره‌های یادگیریِ قبلی رویِ تکلیف‌های دیگر است، به‌جای شروع از صفر. این زیرِ چترِ «فرا-یادگیری» (Metalearning) یا یادگیریِ انتقالی قرار می‌گیرد.

سرانجام، مردم منبعِ عالیِ اطلاعات‌اند. در بخشِ بعدی، دربارهٔ نحوهٔ تعامل با مردم صحبت می‌کنیم، و بخشی از آن نحوهٔ استفاده از کنش‌هایشان برای هدایتِ یادگیریِ ربات است.

۲۶.۸ انسان‌ها و ربات‌ها

تاکنون، بر رباتی تمرکز کرده‌ایم که به‌صورتِ منزوی برنامه‌ریزی و یاد می‌گیرد چگونه عمل کند. این برای برخی ربات‌ها مفید است، مانندِ مریخ‌نوردهایی که به‌جایِ ما برای کاوشِ سیاراتِ دور می‌فرستیم. اما، در بیشترِ موارد، ربات‌ها را برای کارِ منزوی نمی‌سازیم. آن‌ها را برای کمک به ما و برای کار در محیط‌های انسانی، اطرافِ ما و با ما، می‌سازیم.

این دو چالشِ مکمل برمی‌انگیزد. نخست بهینه‌سازیِ پاداش وقتی مردم در همان محیطِ ربات عمل می‌کنند است. این را مسئلهٔ «هماهنگی» (Coordination) می‌نامیم (بخش ۱۸.۱ را ببینید). وقتی پاداشِ ربات نه فقط به کنش‌های خودش، بلکه به کنش‌هایی که مردم می‌گیرند بستگی دارد، ربات باید کنش‌هایش را به‌گونه‌ای انتخاب کند که با کنش‌های آن‌ها خوب جفت شود. وقتی انسان و ربات در همان تیم‌اند، این به همکاری تبدیل می‌شود.

دوم چالشِ بهینه‌سازیِ برای آنچه مردم واقعاً می‌خواهند است. اگر ربات قرار است به مردم کمک کند، تابعِ پاداشش باید کنش‌هایی را ترغیب کند که مردم می‌خواهند ربات اجرا کند. فهمیدنِ تابعِ پاداشِ درست (یا سیاستِ درست) برای ربات خودش مسئلهٔ تعاملی است. این دو چالش را به‌نوبت کاوش خواهیم کرد.

۲۶.۸.۱ هماهنگی

فعلاً فرض کنیم، چنان‌که تاکنون کرده‌ایم، ربات به تابعِ پاداشِ به‌وضوح تعریف‌شده‌ای دسترسی دارد. اما به‌جای نیاز به بهینه‌کردنِ آن به‌صورتِ منزوی، حالا ربات باید آن را دورِ انسانی که همچنین عمل می‌کند بهینه کند. برای نمونه، وقتی خودرویِ خودرانی در بزرگراه ادغام می‌شود، باید مانور را با رانندهٔ انسانیِ در حالِ آمدنِ خطِ هدف مذاکره کند—آیا باید شتاب بگیرد و جلو ادغام شود، یا کند کند و پشت‌سر ادغام شود؟ بعداً، وقتی به علامتِ ایست می‌کشد و برای گردش به راست آماده می‌شود، باید مراقبِ دوچرخه‌سوارِ خطِ دوچرخه و عابرِ پیاده‌ای که در آستانهٔ قدم‌گذاشتن به خطِّ عابر است باشد.

یا، رباتِ متحرکی در راهرو در نظر بگیر. کسی که مستقیم به سمتِ ربات می‌آید کمی به راست قدم می‌گذارد، که نشان می‌دهد می‌خواهد از کدام سمتِ ربات رد شود. ربات باید پاسخ دهد و نیاتش را روشن کند.

انسان‌ها به‌عنوانِ عامل‌های تقریباً عقلایی: یکی از راه‌های صورتبندیِ هماهنگی با انسان مدل‌کردنِ آن به‌عنوانِ بازی بینِ ربات و انسان است (بخش ۱۸.۲). با این رویکرد، به‌صورتِ صریح فرض می‌کنیم مردم عامل‌هایی‌اند که توسط هدف‌ها ترغیب می‌شوند. این به‌طورِ خودکار به این معنا نیست که آن‌ها عامل‌های کاملاً عقلایی‌اند (یعنی راه‌حل‌های

بهینه در بازی می‌یابند)، اما به این معناست ربات می‌تواند شیوهٔ استدلال‌ش دربارهٔ انسان را از طریقِ مفهومِ هدف‌های ممکنِ انسان ساختار دهد. در این بازی:

- حالتِ محیط پیکربندی‌های هر دوِ عاملِ ربات و انسان را درمی‌گیرد؛ آن را x = (x_R, x_H) بنام؛
- هر عامل می‌تواند کنش بگیرد، به‌ترتیب u_R و u_H؛
- هر عامل هدفی دارد که می‌تواند به‌عنوانِ هزینهٔ J_R و J_H بازنمایی شود: هر عامل می‌خواهد ایمن و کارآمد به هدفش برسد؛
- و، مانندِ هر بازی، هر هدف به حالت و به کنش‌های هر دو عامل بستگی دارد: J_R(x, u_R, u_H) و J_H(x, u_H, u_R). به تعاملِ خودرو-عابر فکر کنید—اگر عابر رد شود خودرو باید بایستد، و اگر عابر منتظر بماند باید جلو برود.

سه جنبهٔ مهم این بازی را پیچیده می‌کند. نخست اینکه انسان و ربات لزوماً هدف‌های یکدیگر را نمی‌دانند. این آن را «بازیِ با اطلاعاتِ ناقص» (Incomplete Information Game) می‌کند. دوم اینکه فضاهای حالت و کنش پیوسته‌اند، چنان‌که در سراسرِ این فصل بوده‌اند. در فصل ۵ یاد گرفتیم چگونه جستجویِ درختی را برای بازی‌های گسسته انجام دهیم، اما چگونه با فضاهای پیوسته سروکار کنیم؟ سوم، حتی اگر در سطحِ بالا مدلِ بازی معنا بدهد—انسان‌ها حرکت می‌کنند و هدف دارند—رفتارِ انسان ممکن است همیشه به‌خوبی به‌عنوانِ راه‌حلِ بازی مشخص نشود. بازی با چالشِ محاسباتی نه فقط برای ربات، بلکه برای ما انسان‌ها نیز می‌آید. مستلزمِ فکرکردن دربارهٔ آنچه ربات در پاسخ به آنچه شخص انجام می‌دهد انجام خواهد داد است، که به آنچه ربات فکر می‌کند شخص انجام خواهد داد بستگی دارد، و خیلی زود به «تو فکر می‌کنی من فکر می‌کنم تو فکر می‌کنی من فکر می‌کنم» می‌رسیم—تا بی‌نهایت لاک‌پشت!

انسان‌ها نمی‌توانند با همهٔ این‌ها سروکار داشته باشند و زیربهینگی‌های خاصی نشان می‌دهند. این بدان‌معناست ربات باید این زیربهینگی‌ها را در نظر بگیرد.

پس، خودروِی خودران وقتی مسئلهٔ هماهنگی این‌قدر سخت است چه باید بکند؟ کاری شبیهِ آنچه قبلاً در این فصل انجام داده‌ایم خواهیم کرد. برای برنامه‌ریزیِ حرکت و کنترل، MDP را گرفتیم و به برنامه‌ریزیِ مسیر و سپس دنبال‌کردنِ آن با کنترل‌گر شکستیم. این‌جا نیز، بازی را می‌گیریم و به ساختِ پیش‌بینی‌هایی دربارهٔ کنش‌های انسانی، و تصمیم‌گیری دربارهٔ آنچه ربات با توجه به این پیش‌بینی‌ها باید انجام دهد، می‌شکنیم.

پیش‌بینیِ کنشِ انسانی: پیش‌بینیِ کنش‌های انسانی سخت است، زیرا به کنش‌های ربات بستگی دارند و برعکس. یکی از ترفندهایی که ربات‌ها استفاده می‌کنند وانمودکردن به اینکه شخص ربات را نادیده می‌گیرد است. ربات فرض می‌کند مردم نسبت به هدف‌شان نویزی-بهینه‌اند، که برای ربات ناشناخته است و مدل می‌شود دیگر به کنش‌های ربات وابسته نیست: J_H(x, u_H). به‌طورِ خاص، هرچه مقدارِ کنش برای هدف بالاتر باشد (هزینه-تا-آینده کمتر)، احتمالِ گرفتنِ آن توسط انسان بیشتر است. ربات می‌تواند مدلی برای P(u_H | x, J_H) ایجاد کند، برای نمونه با تابعِ سافتمکسِ صفحهٔ ۸۱۱:

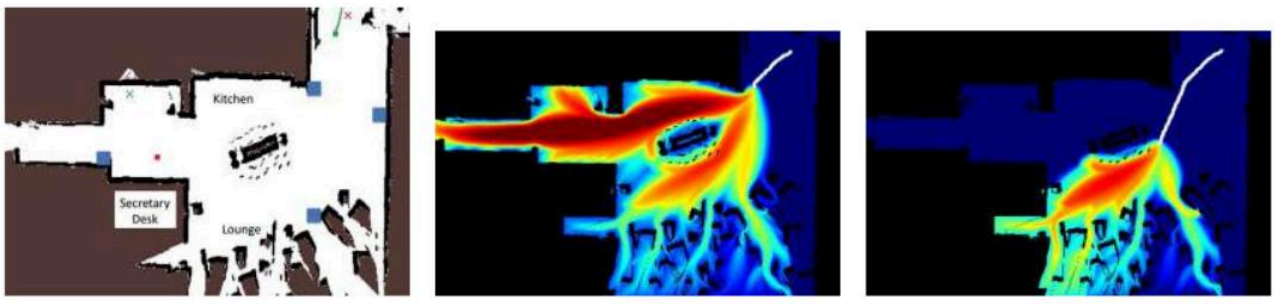
```
P(u_H | x, J_H) ∝ e^(−Q(x, u_H; J_H))    (26.8)
```

با Q(x, u_H; J_H) تابعِ ارزشِ Q متناظر با J_H (علامتِ منفی آن‌جاست، زیرا در رباتیک دوست داریم هزینه را کمینه کنیم، نه پاداش را بیشینه). توجه کنید ربات نه کنش‌های کاملاً بهینه فرض می‌کند، نه فرض می‌کند کنش‌ها بر مبنای استدلال دربارهٔ ربات انتخاب می‌شوند.

مسلح به این مدل، ربات از کنش‌های جاریِ انسان به‌عنوانِ شاهدِ J_H استفاده می‌کند. اگر مدلِ مشاهده‌ای برای نحوهٔ وابستگیِ کنش‌های انسانی به هدفِ انسان داشته باشیم، هر کنشِ انسانی را می‌توان برای به‌روزرسانیِ باورِ ربات دربارهٔ هدفِ شخص لحاظ کرد:

$$b'(J_H) \propto b(J_H)\, P(u_H \mid x, J_H) \qquad (26.9)$$

نمونه‌ای در شکل ۲۶.۲۷ است: ربات مکانِ انسانی را ردیابی می‌کند و با حرکتِ انسان، ربات باورش را رویِ هدف‌های انسانی به‌روزرسانی می‌کند. با حرکتِ انسان به سمتِ پنجره‌ها، ربات احتمالِ اینکه هدف نگاه‌کردن از پنجره باشد را افزایش و احتمالِ اینکه هدف رفتن به آشپزخانه، که در جهتِ دیگر است، باشد را کاهش می‌دهد. این‌گونه است که کنش‌های گذشتهٔ انسان در نهایت به ربات دربارهٔ آنچه انسان در آینده انجام خواهد داد اطلاع می‌دهند. داشتنِ باوری دربارهٔ هدفِ انسان به ربات کمک می‌کند پیش‌بینی کند انسان بعد چه کنش‌هایی خواهد گرفت. نقشهٔ گرماییِ شکل پیش‌بینی‌های آیندهٔ ربات را نشان می‌دهد: قرمز محتمل‌ترین و آبی کم‌محتمل‌ترین است.



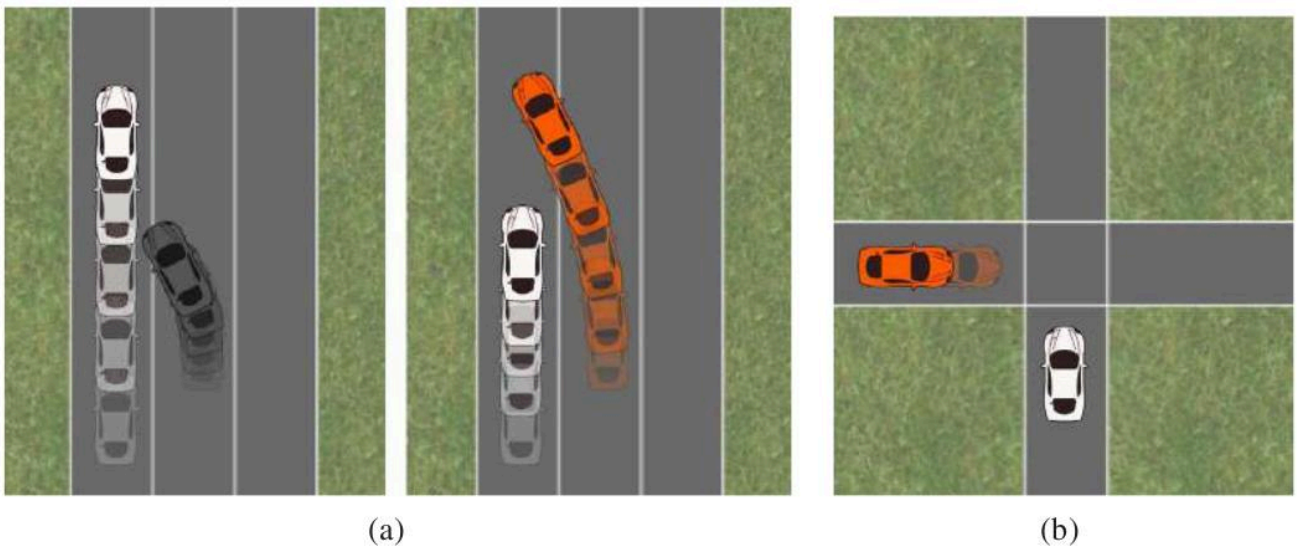
شکل ۲۶.۲۷ – ساختِ پیش‌بینی‌ها با فرضِ اینکه مردم با توجه به هدف‌شان نویزی-عقلایی‌اند: ربات از کنش‌های گذشته برای به‌روزرسانیِ باوری دربارهٔ اینکه شخص به سمتِ چه هدفی می‌رود استفاده می‌کند، و سپس از باور برای ساختِ پیش‌بینی‌هایی دربارهٔ کنش‌های آینده. (الف) نقشهٔ اتاق. (ب) پیش‌بینی‌ها پس از دیدنِ بخشِ کوچکی از مسیرِ شخص (مسیرِ سفید)؛ (ج) پیش‌بینی‌ها پس از دیدنِ کنش‌های انسانیِ بیشتر: ربات اکنون می‌داند شخص به سمتِ راهروی چپ نمی‌رود، زیرا مسیرِ تاکنون‌طی‌شده اگر آن هدفِ شخص بود مسیرِ ضعیفی می‌بود. تصاویر با اجازه از برایان دی. زیبارت. زیبارت و همکاران (2009) را ببینید.

همین می‌تواند در رانندگی رخ دهد. ممکن است ندانیم رانندهٔ دیگر چقدر برای کارایی ارزش قائل است، اما اگر ببینیم وقتی کسی می‌خواهد جلوش ادغام شود شتاب می‌گیرد، حالا کمی بیشتر درباره‌اش می‌دانیم. و وقتی آن را بدانیم، بهتر می‌توانیم پیش‌بینی کنیم در آینده چه خواهد کرد—همان راننده به احتمالِ زیاد به ما نزدیک‌تر می‌آید یا برای جلوافتادن در ترافیک زیگزاگ می‌رود.

وقتی ربات می‌تواند پیش‌بینی‌هایی دربارهٔ کنش‌های آیندهٔ انسان بسازد، مسئله‌اش را به حلِ MDP کاهش داده است. کنش‌های انسانی تابعِ انتقال را پیچیده می‌کنند، اما تا وقتی ربات می‌تواند پیش‌بینی کند شخص از هر حالتِ آینده چه کنشی خواهد گرفت، ربات می‌تواند P(x' | x, u_R) را محاسبه کند: می‌تواند P(u_H | x) را از P(u_H | x, J_H) با حذفِ

J_H محاسبه کند، و آن را با P(x' | x, u_R, u_H)، تابعِ انتقال (دینامیک) برای نحوهٔ به‌روزرسانیِ جهان بر مبنای هر دوِ کنشِ ربات و انسان، ترکیب کند. در بخش ۲۶.۵ بر حلِ این در فضاهای حالت و کنشِ پیوسته برای دینامیکِ قطعی تمرکز کردیم، و در بخش ۲۶.۶ انجامش را با دینامیکِ تصادفی و عدم‌قطعیت بحث کردیم.

جداسازیِ پیش‌بینی از کنش، رسیدگیِ ربات به تعامل را آسان‌تر می‌کند، اما همان‌گونه که جداسازیِ تخمین از حرکت، یا جداسازیِ برنامه‌ریزی از کنترل، عملکرد را قربانی می‌کند. رباتی با این جداسازی دیگر نمی‌فهمد کنش‌هایش می‌توانند بر آنچه مردم در نهایت انجام می‌دهند اثر بگذارند. در مقابل، رباتِ شکل ۲۶.۲۷ پیش‌بینی می‌کند مردم به کجا می‌روند و سپس برای رسیدن به هدفِ خودش و اجتناب از برخورد با آن‌ها بهینه می‌کند. در شکل ۲۶.۲۸، خودروی خودرانی داریم که در بزرگراه ادغام می‌شود. اگر فقط در واکنش به خودروهای دیگر برنامه‌ریزی می‌کرد، ممکن بود مدتِ طولانی منتظر بماند در حالی که خودروهای دیگر خطِّ هدفش را اشغال می‌کنند. در مقابل، خودرویی که دربارهٔ پیش‌بینی و کنش به‌صورتِ مشترک استدلال می‌کند می‌داند کنش‌های متفاوتی که می‌تواند بگیرد به واکنش‌های متفاوتی از انسان می‌انجامند. اگر شروع به ابرازِ خودش کند، خودروهای دیگر احتمالاً کمی کند می‌کنند و جا باز می‌کنند. رباتیست‌ها به سمتِ تعاملاتِ هماهنگِ این‌چنینی کار می‌کنند تا ربات‌ها بهتر با انسان‌ها کار کنند.



شکل ۲۶.۲۸ – (الف) چپ: خودروی خودران (خطِّ میانی) پیش‌بینی می‌کند رانندهٔ انسانی (خطِّ چپ) می‌خواهد به جلو ادامه دهد، و مسیری را برنامه‌ریزی می‌کند که کند می‌کند و پشت‌سر ادغام می‌شود. راست: خودرو اثرِ کنش‌هایش بر کنش‌های انسانی را در نظر می‌گیرد و درمی‌یابد می‌تواند جلو ادغام شود و به رانندهٔ انسانی تکیه کند که کند کند. (ب) همان الگوریتم راهبردِ غیرعادی در تقاطع تولید می‌کند: خودرو درمی‌یابد می‌تواند محتمل‌تر کند شخص (پایین) سریع‌تر از تقاطع عبور کند با شروعِ عقب‌زدنِ آهسته. تصاویر با اجازه از آنکا دراگان. صدیق و همکاران (2016) را ببینید.

پیش‌بینی‌های انسانی دربارهٔ ربات: اطلاعاتِ ناقص اغلب دوجانبه است: ربات هدفِ انسان را نمی‌داند و انسان، به‌نوبهٔ خود، هدفِ ربات را نمی‌داند—مردم باید دربارهٔ ربات‌ها پیش‌بینی کنند. به‌عنوانِ طراحانِ ربات، مسئولِ نحوهٔ پیش‌بینیِ انسان نیستیم؛ فقط می‌توانیم آنچه ربات انجام می‌دهد را کنترل کنیم. با این حال، ربات می‌تواند به‌گونه‌ای عمل کند که برای انسان پیش‌بینیِ درست آسان‌تر شود. ربات می‌تواند فرض کند انسان از چیزی تقریباً مشابهِ معادلهٔ (26.8) برای برآوردِ هدفِ رباتِ J_R استفاده می‌کند، و بنابراین ربات طوری عمل می‌کند که هدفِ واقعی‌اش به‌راحتی قابلِ استنتاج باشد.

حالتِ خاصِ بازی وقتی است که انسان و ربات در همان تیم‌اند، به سمتِ همان هدف یا هدفِ مشترک کار می‌کنند: J_H = J_R. تصور کنید رباتِ خانگیِ شخصی‌ای دارید که به شما در آماده‌کردنِ شام یا تمیزکردن کمک می‌کند—این‌ها مثال‌هایی از همکاری‌اند. حالا می‌توانیم «عاملِ مشترک» (Joint Agent) تعریف کنیم که کنش‌هایش دوتایی‌های کنشِ انسان-ربات، (u_H, u_R)، هستند و برای J_H(x, u_H, u_R) = J_R(x, u_R, u_H) بهینه می‌کند، و مسئلهٔ برنامه‌ریزیِ معمولی را حل می‌کنیم. برنامه یا سیاستِ بهینه را برای عاملِ مشترک محاسبه می‌کنیم، و والا، حالا می‌دانیم ربات و انسان باید چه کنند.

این اگر مردم کاملاً بهینه بودند واقعاً خوب کار می‌کرد. ربات سهمِ خودش از برنامهٔ مشترک را انجام می‌داد، انسان سهمِ خودش را. متأسفانه، در عمل، مردم به نظر نمی‌رسد از برنامهٔ کاملاً چیده‌شدهٔ عاملِ مشترک پیروی کنند؛ آن‌ها ذهنِ خودشان را دارند! قبلاً یک راهِ مدیریتِ این را یاد گرفته‌ایم، در بخش ۲۶.۶. آن را کنترلِ پیش‌بینِ مدل (MPC) نامیدیم: ایده ارائهٔ برنامه، اجرایِ نخستین کنش، و سپس بازبرنامه‌ریزی بود. به آن‌شیوه، ربات همیشه برنامه‌اش را با آنچه انسان واقعاً انجام می‌دهد سازگار می‌کند.

بیایید مثالی را کار کنیم. فرض کنید شما و ربات در آشپزخانه‌اید و تصمیم گرفته‌اید وافل درست کنید. شما کمی به یخچال نزدیک‌ترید، پس برنامهٔ مشترکِ بهینه این بود که شما تخم‌مرغ و شیر را از یخچال بردارید، در حالی که ربات آرد را از کابینت می‌آورد. ربات این را می‌داند، زیرا می‌تواند کاملاً دقیق اندازه بگیرد همه کجا هستند. اما فرض کنید شما شروع به رفتن به سمتِ کابینتِ آرد می‌کنید. شما بر خلافِ برنامهٔ مشترکِ بهینه می‌روید. به‌جای چسبیدن به آن و سرسختانه رفتنِ ربات نیز به سمتِ آرد، رباتِ MPC برنامهٔ بهینه را دوباره محاسبه می‌کند، و حالا که شما به اندازهٔ کافی به آرد نزدیکید، بهترین کار برای ربات این است که وافل‌ساز را بردارد. اگر بدانیم مردم ممکن است از بهینگی منحرف شوند، می‌توانیم از قبل آن را در نظر بگیریم.

در مثالِ ما، ربات می‌تواند بکوشد پیش‌بینی کند شما همان لحظه که نخستین قدم را برمی‌دارید به سمتِ آرد می‌روید (مثلاً، با فنِ پیش‌بینیِ بالا). حتی اگر از نظرِ فنی هنوز برای شما بهینه باشد برگردید و به سمتِ یخچال بروید، ربات نباید فرض کند این چیزی است که رخ خواهد داد. در عوض، ربات می‌تواند برنامه‌ای محاسبه کند که در آن شما همان کاری را ادامه می‌دهید که به نظر می‌رسد می‌خواهید.

انسان‌ها به‌عنوانِ عامل‌های جعبهٔ سیاه: برای هماهنگ‌کردنِ ربات‌ها با ما، لازم نیست مردم را عامل‌های هدف‌محور و عمدی درمان کنیم. مدلِ جایگزین این است که انسان صرفاً عاملی است که سیاستِ π_H اش با دینامیکِ محیط «ور می‌رود». ربات π_H را نمی‌داند، اما می‌تواند مسئله را به‌عنوانِ نیاز به عمل‌کردن در MDP با دینامیکِ ناشناخته مدل کند. این را قبلاً دیده‌ایم: برای عامل‌های عمومی در فصل ۲۲، و برای ربات‌ها به‌طورِ خاص در بخش ۲۶.۷.

ربات می‌تواند مدلِ سیاستِ π_H را به دادهٔ انسانی برازش کند و از آن برای محاسبهٔ سیاستِ بهینه برای خودش استفاده کند. به‌دلیلِ کمبودِ داده، این تاکنون بیشتر در سطحِ تکلیف استفاده شده. برای نمونه، ربات‌ها از طریقِ تعامل یاد گرفته‌اند مردم چه کنش‌هایی تمایل به گرفتن دارند (در پاسخ به کنش‌های خودش) برای تکلیفِ قراردادن و پیچ‌کردنِ پیچ‌ها در تکلیفِ مونتاژِ صنعتی.

سپس جایگزینِ یادگیریِ تقویتیِ بدونِ مدل نیز هست: ربات می‌تواند با سیاست یا تابعِ ارزشِ اولیه‌ای شروع کند و در طولِ زمان با آزمون و خطا آن را بهبود دهد.

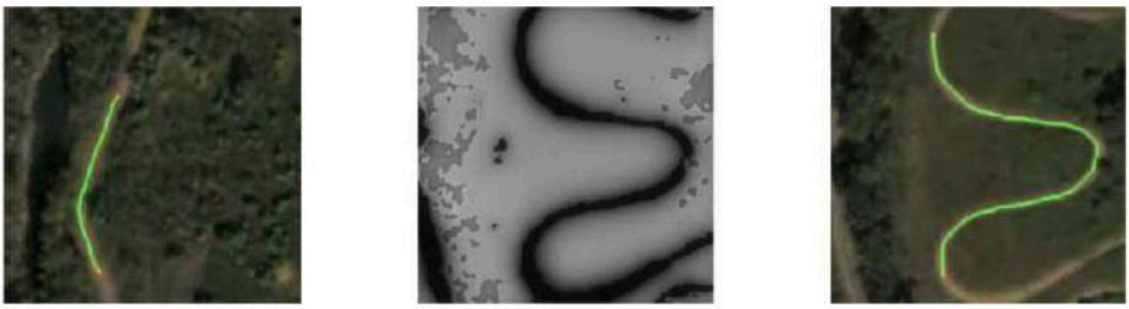
۲۶.۸.۲ یادگیریِ انجامِ آنچه انسان‌ها می‌خواهند

راهِ دیگرِ ورودِ تعامل با انسان به رباتیک در خودِ J_R است—تابعِ هزینه یا پاداشِ ربات. چارچوبِ عامل‌های عقلایی و الگوریتم‌های وابسته، مسئلهٔ تولیدِ رفتارِ خوب را به مشخص‌کردنِ تابعِ پاداشِ خوب کاهش می‌دهند. اما برای ربات‌ها، مانندِ بسیاری از عامل‌های هوش مصنوعیِ دیگر، درست‌کردنِ هزینه همچنان دشوار است.

خودروهای خودران را بگیرید: می‌خواهیم به مقصد برسند، ایمن باشند، برای مسافرانشان راحت رانندگی کنند، قوانینِ راهنمایی را رعایت کنند و... طراحِ چنین سامانه‌ای باید این مؤلفه‌های متفاوتِ تابعِ هزینه را با هم مبادله کند. کارِ طراح سخت است، زیرا ربات‌ها برای کمک به کاربرانِ نهایی ساخته می‌شوند، و هر کاربرِ نهایی یکسان نیست. همهٔ ما ترجیح‌های متفاوتی برای اینکه چقدر تهاجمی بخواهیم خودروی‌مان رانندگی کند داریم و...

در زیر، دو جایگزین برای تلاشِ هماهنگ‌کردنِ رفتارِ ربات با آنچه واقعاً می‌خواهیم ربات انجام دهد کاوش می‌کنیم. نخستین یادگیریِ تابعِ هزینه از ورودیِ انسانی است. دومین دورزدنِ تابعِ هزینه و تقلیدِ نمایش‌های انسانیِ تکلیف است.

یادگیریِ ترجیح: یادگیریِ توابعِ هزینه: تصور کنید کاربرِ نهایی به ربات نشان می‌دهد چگونه تکلیفی را انجام دهد. برای نمونه، آن‌ها خودرو را به‌شیوه‌ای می‌رانند که دوست دارند ربات آن را براند. آیا می‌توانید راهی بیندیشید که ربات از این کنش‌ها—آن‌ها را «نمایش» (Demonstration) می‌نامیم—برای فهمیدنِ اینکه چه تابعِ هزینه‌ای باید بهینه کند استفاده کند؟



شکل ۲۶.۲۹ – چپ: به رباتِ متحرکی نمایشی نشان داده می‌شود که رویِ جادهٔ خاکی می‌ماند. وسط: ربات تابعِ هزینهٔ مطلوب را استنتاج می‌کند و در صحنهٔ تازه از آن استفاده می‌کند، و می‌داند هزینهٔ کمتری رویِ جادهٔ آن‌جا بگذارد. راست: ربات مسیری برای صحنهٔ تازه برنامه‌ریزی می‌کند که همچنین رویِ جاده می‌ماند، و ترجیح‌های پشتِ نمایش را بازتولید می‌کند. تصاویر با اجازه از ناتان راتلیف و جیمز ای. باگنل. راتلیف و همکاران (2006) را ببینید.

در واقع قبلاً پاسخِ این را در بخش ۲۶.۸.۱ دیده‌ایم. آن‌جا، چیدمان کمی متفاوت بود: شخصِ دیگری کنش‌هایی در همان فضایِ ربات می‌گرفت و ربات باید پیش‌بینی می‌کرد شخص چه خواهد کرد. اما یکی از فنونِ ساختِ این پیش‌بینی‌ها فرضِ این بود که مردم برای بهینه‌کردنِ نویزیِ تابعِ هزینهٔ J_H عمل می‌کنند، و می‌توانیم از کنش‌های

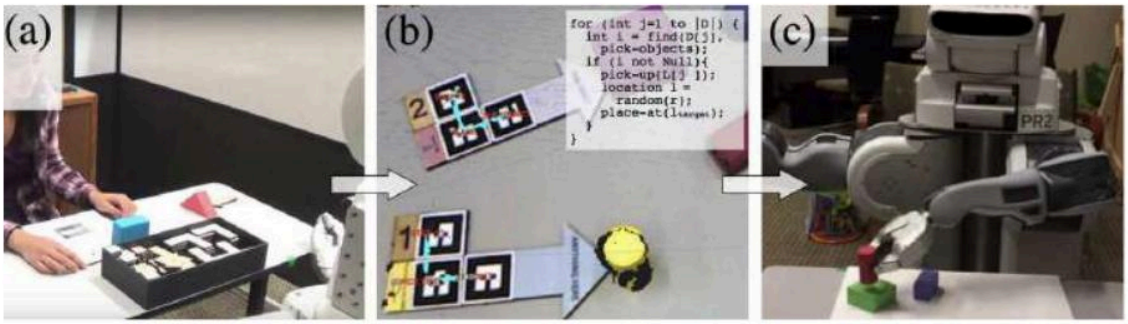
جاری‌شان به‌عنوانِ شاهدِ اینکه آن تابعِ هزینه چیست استفاده کنیم. می‌توانیم همین را این‌جا انجام دهیم، فقط نه برای هدفِ پیش‌بینیِ رفتارِ انسانی در آینده، بلکه برای کسبِ تابعِ هزینه‌ای که خودِ ربات باید بهینه کند. اگر شخص دفاعی رانندگی کند، تابعِ هزینه‌ای که کنش‌هایش را تبیین می‌کند وزنِ زیادی رویِ ایمنی و وزنِ کمتری رویِ کارایی می‌گذارد. ربات می‌تواند این تابعِ هزینه را به‌عنوانِ تابعِ خودش بپذیرد و هنگامِ رانندگیِ خودش آن را بهینه کند. رباتیست‌ها با الگوریتم‌های متفاوتی برای محاسباتاً قابلِ‌مدیریت‌کردنِ این استنتاجِ هزینه آزمایش کرده‌اند. در شکل ۲۶.۲۹، نمونه‌ای از آموزشِ ربات برای ترجیحِ ماندنِ رویِ جاده بر رفتن رویِ زمینِ چمنی می‌بینیم. به‌طورِ سنتی در چنین روش‌هایی، تابعِ هزینه به‌عنوانِ ترکیبی از ویژگی‌های دست‌ساخته بازنمایی می‌شده، اما کارِ اخیر همچنین نحوهٔ بازنماییِ آن با شبکهٔ عصبیِ عمیق، بدونِ مهندسیِ ویژگی، را مطالعه کرده.

راه‌های دیگری نیز برای ارائهٔ ورودی توسط شخص هست. شخص می‌تواند به‌جای نمایش، از زبان برای آموزشِ ربات استفاده کند. شخص می‌تواند به‌عنوانِ منتقد عمل کند، ربات را در حالِ انجامِ تکلیف به یک شیوه (یا دو شیوه) تماشا کند و سپس بگوید تکلیف چقدر خوب انجام شده (یا کدام شیوه بهتر بود)، یا توصیه‌ای برای بهبود بدهد.

یادگیریِ مستقیمِ سیاست‌ها از طریقِ تقلید: جایگزین دورزدنِ توابعِ هزینه و یادگیریِ مستقیمِ سیاستِ مطلوبِ ربات است. در مثالِ خودرویِ ما، نمایش‌های انسانی مجموعهٔ دادهٔ مناسبی از حالت‌های برچسب‌خورده با کنشی که ربات باید در هر حالت بگیرد می‌سازند: $D = \{(x_i, u_i)\}$. ربات می‌تواند یادگیریِ نظارت‌شده را برای برازشِ سیاستِ $\pi : x \mapsto u$ اجرا کند و آن سیاست را اجرا کند. این «یادگیریِ تقلیدی» (Imitation Learning) یا «همتاسازیِ رفتاری» (Behavioral Cloning) نامیده می‌شود.

چالشِ این رویکرد در «تعمیم» (Generalization) به حالت‌های تازه است. ربات نمی‌داند چرا کنش‌های پایگاهِ داده‌اش به‌عنوانِ بهینه علامت خورده‌اند. قاعدهٔ علّی ندارد؛ تنها کاری که می‌تواند بکند اجرایِ الگوریتمِ یادگیریِ نظارت‌شده برای تلاشِ یادگیریِ سیاستی است که به حالت‌های ناشناخته تعمیم می‌یابد. با این حال، هیچ تضمینی نیست تعمیم درست باشد.

شکل ۲۶.۳۰ – معلمِ انسانی ربات را به پایین هل می‌دهد تا به آن بیاموزد نزدیک‌تر به میز بماند. ربات به‌طورِ مناسب فهمِ خود را از تابعِ هزینهٔ مطلوب به‌روزرسانی می‌کند و شروع به بهینه‌کردنِ آن می‌کند. با اجازه از آنکا دراگان. باجسی و همکاران (2017) را ببینید.



شکل ۲۶.۳۱ – رابطِ برنامه‌نویسی که شاملِ قراردادنِ بلوک‌های خاص‌طراحی‌شده در فضایِ کارِ ربات برای انتخابِ اشیا و مشخص‌کردنِ کنش‌های سطح‌بالاست. تصاویر با اجازه از مایا چاکماک. صدیق‌گر و همکاران (2017) را ببینید.

پروژهٔ خودروویِ خودرانِ ALVINN از این رویکرد استفاده کرد و دریافت حتی وقتی از حالتی در D شروع می‌شود، π خطاهای کوچکی خواهد کرد، که خودرو را از مسیرِ نمایش‌داده‌شده دور می‌کند. آن‌جا، π خطایِ بزرگ‌تری خواهد کرد، که خودرو را حتی بیشتر از مسیرِ مطلوب دور می‌کند. می‌توانیم در زمانِ آموزش به این بپردازیم اگر جمع‌آوریِ برچسب‌ها و یادگیری را در هم بیامیزیم: با نمایشی شروع کن، سیاستی یاد بگیر، سپس آن سیاست را اجرا کن و از انسان بخواه در هر حالتِ مسیر چه کنشی بگیرد، سپس تکرار کن. ربات سپس یاد می‌گیرد چگونه اشتباه‌هایش را هنگامِ انحراف از کنش‌های مطلوبِ انسانی تصحیح کند.

یا، می‌توانیم با بهره‌برداری از یادگیریِ تقویتی به آن بپردازیم. ربات می‌تواند مدلِ دینامیکی بر مبنای نمایش‌ها برازش کند، و سپس از کنترلِ بهینه (بخش ۲۶.۵.۴) برای تولیدِ سیاستی که برای نزدیک‌ماندن به نمایش بهینه می‌کند استفاده کند. نسخه‌ای از این برای انجامِ مانورهای بسیار چالش‌برانگیز در سطحِ خبره در هلیکوپترِ کوچکِ کنترلِ رادیویی استفاده شده (شکل ۲۲.۹ (ب) را ببینید).

سامانهٔ DAGGER (تجمیعِ داده) با نمایشِ کارشناسِ انسانی شروع می‌کند. از آن سیاستِ π_1 یاد می‌گیرد و از سیاست برای تولیدِ مجموعهٔ دادهٔ D استفاده می‌کند. سپس از D سیاستِ تازهٔ π_2 تولید می‌کند که بهترین تقلید از دادهٔ انسانیِ اصلی است. این تکرار می‌شود، و در nاُمین تکرار از π_n برای تولیدِ دادهٔ بیشتر، که به D اضافه می‌شود و سپس برای ایجادِ π_{n+1} استفاده می‌شود، استفاده می‌کند. به بیانِ دیگر، در هر تکرار سامانه تحتِ سیاستِ کنونی دادهٔ تازه جمع می‌کند و سیاستِ بعدی را با همهٔ داده‌های جمع‌آوری‌شده تاکنون آموزش می‌دهد.

فنونِ اخیرِ مرتبط از آموزشِ تخاصمی استفاده می‌کنند: بینِ آموزشِ رده‌بندی برای تمایزِ سیاستِ آموخته‌شدهٔ ربات از نمایش‌های انسانی، و آموزشِ سیاستِ رباتیِ تازه از طریقِ یادگیریِ تقویتی برای فریبِ رده‌بند، تناوب می‌کنند. این پیشرفت‌ها ربات را قادر می‌کنند حالت‌های نزدیکِ نمایش‌ها را مدیریت کند، اما تعمیم به حالت‌های دور یا به دینامیکِ تازه کارِ در حالِ پیشرفت است.

رابط‌های آموزش و مسئلهٔ تناظر. تاکنون، حالتِ خودرویِ خودران یا هلیکوپترِ خودمختار را تصور کرده‌ایم، که نمایش‌های انسانی در آن‌ها از همان کنش‌هایی استفاده می‌کنند که ربات خودش می‌تواند بگیرد: شتاب‌دادن، ترمزکردن و فرمان‌دادن. اما اگر این کار را برای تکلیف‌هایی مانندِ تمیزکردنِ میزِ آشپزخانه انجام دهیم چه اتفاقی

می‌افتد؟ این‌جا دو انتخاب داریم: یا شخص با بدنِ خودش نمایش می‌دهد در حالی که ربات تماشا می‌کند، یا شخص به‌صورتِ فیزیکی اثرگذارهای ربات را هدایت می‌کند.

رویکردِ نخست جذاب است، زیرا برای کاربرانِ نهایی طبیعی می‌آید. متأسفانه، از «مسئلهٔ تناظر» (Correspondence Problem) رنج می‌برد: چگونه کنش‌های انسانی را رویِ کنش‌های ربات نگاشت کنیم. مردم سینماتیک و دینامیکِ متفاوتی از ربات‌ها دارند. نه‌تنها این ترجمه یا بازهدف‌گذاریِ حرکتِ انسانی به حرکتِ ربات را دشوار می‌کند (مثلاً، بازهدف‌گذاریِ گیرهٔ پنج‌انگشتیِ انسانی به گیرهٔ دو-انگشتیِ ربات)، بلکه اغلب راهبردِ سطح‌بالایی که شخص ممکن است استفاده کند برای ربات مناسب نیست.

رویکردِ دوم، که معلمِ انسانی اثرگذارهای ربات را به موقعیت‌های درست حرکت می‌دهد، «آموزشِ حرکتی» (Kinesthetic Teaching) نامیده می‌شود. آموزشِ این‌گونه برای انسان‌ها آسان نیست، به‌ویژه آموزشِ ربات‌های با مفصل‌های متعدد. معلم باید همهٔ درجه‌های آزادی را هنگامِ هدایتِ بازو در طولِ تکلیف هماهنگ کند. پژوهشگران پس از آن جایگزین‌هایی بررسی کرده‌اند، مانندِ نمایشِ «فریم‌های کلیدی» (Keyframe) به‌جای مسیرهای پیوسته، و همچنین استفاده از «برنامه‌نویسیِ بصری» (Visual Programming) تا کاربرانِ نهایی بتوانند اولیه‌های تکلیف را برنامه‌نویسی کنند به‌جای نمایشِ از صفر (شکل ۲۶.۳۱). گاهی هر دو رویکرد ترکیب می‌شوند.

۲۶.۹ چارچوب‌های رباتیکِ جایگزین

تاکنون، دیدگاهی از رباتیک گرفته‌ایم که بر مفهومِ تعریف یا یادگیریِ تابعِ پاداش و بهینه‌کردنِ آن تابعِ پاداش توسط ربات (چه از طریقِ برنامه‌ریزی چه یادگیری)، گاهی در هماهنگی یا همکاری با انسان‌ها، مبتنی است. این دیدگاهِ «تعمقمحور» (Deliberative) از رباتیک است، که باید با دیدگاهِ واکنشی در تقابل قرار گیرد.

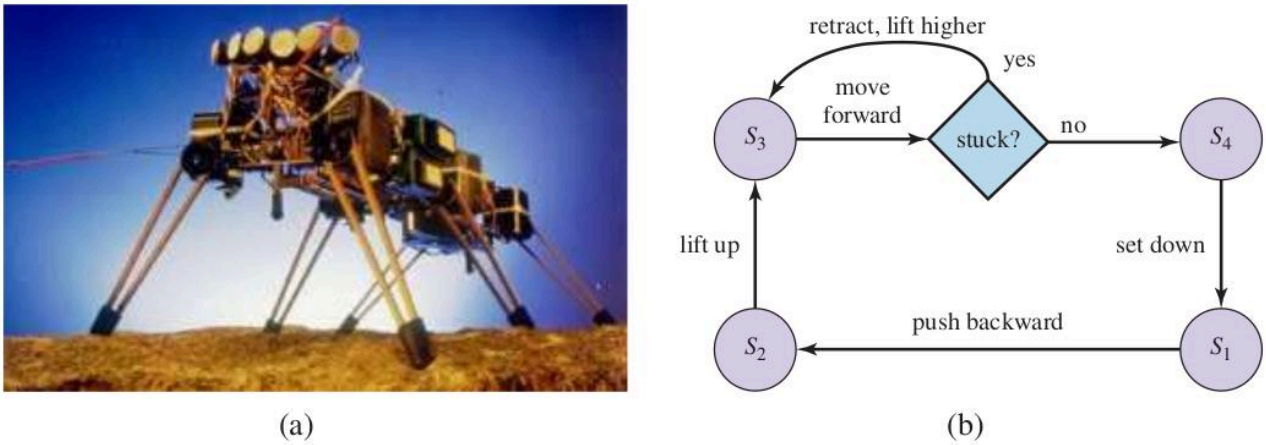
۲۶.۹.۱ کنترل‌گرهای واکنشی

در برخی موارد، برپاکردنِ سیاستِ خوب برای ربات آسان‌تر از مدل‌کردنِ جهان و برنامه‌ریزی است. آنگاه، به‌جای عاملِ عقلایی، عاملِ بازتابی داریم.

برای نمونه، رباتِ پاداری را تصور کنید که می‌کوشد پایش را از رویِ مانعی بلند کند. می‌توانستیم به این ربات قاعده‌ای بدهیم که بگوید پا را به ارتفاعِ کوچکِ h بلند کن و به جلو حرکت بده، و اگر پا با مانعی برخورد کرد، آن را به عقب برگردان و با ارتفاعِ بالاتر از نو شروع کن. می‌توانید بگویید h جنبه‌ای از جهان را مدل می‌کند، اما همچنین می‌توانیم به h به‌عنوانِ متغیرِ کمکیِ کنترل‌گرِ ربات فکر کنیم، عاری از معنایِ فیزیکیِ مستقیم.

یکی از این‌گونه نمونه‌ها رباتِ شش‌پا (هگزاپاد) است که در شکل ۲۶.۳۲ (الف) نشان داده شده، طراحی‌شده برای راه‌رفتن در زمینِ ناهموار. حس‌گذارهای ربات برای به‌دست‌آوردنِ مدل‌های دقیقِ زمین برای برنامه‌ریزیِ مسیر ناکافی‌اند. افزون بر این، حتی اگر دوربین‌ها و فاصله‌یاب‌های دقیق اضافه می‌کردیم، ۱۲ درجهٔ آزادی (دو برای هر پا) مسئلهٔ برنامه‌ریزیِ مسیرِ حاصل را از نظرِ محاسباتی دشوار می‌کرد.

با این حال، امکانِ مشخص‌کردنِ مستقیمِ کنترل‌گر بدونِ مدلِ صریحِ محیط هست. (این را قبلاً با کنترل‌گرِ PD دیده‌ایم، که می‌توانست بازویِ رباتیکِ پیچیده را بدونِ مدلِ صریحِ دینامیکِ ربات رویِ هدف نگه دارد.) برای رباتِ هگزاپاد نخست «الگویِ گام» (Gait)، یا الگویِ حرکتِ اندام‌ها، را انتخاب می‌کنیم. یکی از الگوهایِ گامِ ایستاییِ پایدار این است که نخست پایِ راستِ جلو، پایِ راستِ عقب و پایِ وسطِ چپ را به جلو حرکت دهیم (و سه‌تای دیگر را ثابت نگه داریم)، و سپس سه‌تای دیگر را حرکت دهیم. این الگویِ گام در زمینِ تخت خوب کار می‌کند. در زمینِ ناهموار، موانع ممکن است مانعِ تاب‌خوردنِ پایِ رو به جلو شوند. این مشکل را می‌توان با قاعدهٔ کنترلِ فوق‌العاده ساده‌ای غلبه کرد: وقتی حرکتِ رو به جلوی پا مسدود است، صرفاً آن را جمع کن، بالاتر ببر و دوباره تلاش کن. کنترل‌گرِ حاصل در شکل ۲۶.۳۲ (ب) به‌صورتِ ماشینِ حالتِ متناهیِ ساده‌ای نشان داده شده؛ عاملِ بازتابیِ با حالت است، که حالتِ درونی با شاخصِ حالتِ ماشینِ کنونی (s_1 تا s_4) بازنمایی می‌شود.



شکل ۲۶.۳۲ – (الف) گنگیس (Genghis)، رباتِ هگزاپاد. (تصویر با اجازه از رادنی ای. بروکس.) (ب) ماشینِ حالتِ متناهیِ افزوده‌شده (AFSM) که یک پا را کنترل می‌کند. AFSM به بازخوردِ حس‌گذار واکنش نشان می‌دهد: اگر پایی در طولِ فازِ تابِ رو به جلو گیر کند، بالاتر و بالاتر بلند می‌شود.

۲۶.۹.۲ معماریِ جذب

«معماریِ جذب» (Subsumption Architecture) (بروکس، 1986) چارچوبی برای مونتاژِ کنترل‌گرهای واکنشی از ماشین‌های حالتِ متناهی است. گره‌های این ماشین‌ها ممکن است شاملِ آزمون‌هایی برای برخی متغیرهای حس‌گذار باشند، که در آن صورت دنبالهٔ اجرایِ ماشینِ حالتِ متناهی بر پیامدِ چنین آزمونی شرطی است. کمان‌ها را می‌توان با پیام‌هایی برچسب‌گذاری کرد که هنگامِ عبور از آن‌ها تولید و به موتورهای ربات یا به ماشین‌های حالتِ متناهیِ دیگر فرستاده می‌شوند. علاوه بر این، ماشین‌های حالتِ متناهی «ساعت‌سنج»های (Clock) درونی دارند که زمانِ عبور از کمان را کنترل می‌کنند. ماشین‌های حاصل «ماشین‌های حالتِ متناهیِ افزوده‌شده» (Augmented Finite State Machine یا AFSM) نامیده می‌شوند، که افزوده به استفاده از ساعت‌سنج‌ها اشاره دارد.

نمونه‌ای از AFSM ساده همان ماشینِ چهارحالته‌ای است که همین حالا درباره‌اش صحبت کردیم، نشان‌داده‌شده در شکل ۲۶.۳۲ (ب). این AFSM کنترل‌گرِ چرخه‌ای را پیاده‌سازی می‌کند، که اجرایش بیشتر به بازخوردِ محیطی وابسته نیست. با این حال، فازِ تابِ رو به جلو به بازخوردِ حس‌گذار وابسته است. اگر پا گیر کند، یعنی در اجرایِ تابِ رو به جلو ناکام مانده، ربات پا را جمع می‌کند، کمی بالاتر می‌برد و بارِ دیگر تلاش می‌کند تابِ رو به جلو را اجرا کند. پس، کنترل‌گر

قادر به واکنش به پیشامدهای ناشی از تعاملِ ربات و محیطش است. معماریِ جذب اجزای اضافی برای همگام‌سازیِ AFSMها و برای ترکیبِ مقادیرِ خروجیِ چندِ AFSM احتمالاً متعارض ارائه می‌دهد. به این‌شیوه، برنامه‌نویس را قادر می‌کند کنترل‌گرهای به‌طورِ فزاینده‌ای پیچیده را به‌صورتِ پایین‌به‌بالا ترکیب کند. در مثالِ ما، ممکن است با AFSMهایی برای پاهای منفرد شروع کنیم، سپس AFSM برای هماهنگ‌کردنِ چند پا. رویِ این، ممکن است رفتارهای سطح‌بالاتری مانندِ اجتناب از برخورد، که ممکن است شاملِ عقب‌رفتن و چرخیدن باشد، پیاده‌سازی کنیم.

ایدهٔ ترکیبِ کنترل‌گرهای ربات از AFSMها کاملاً گیراست. تصور کنید تولیدِ همان رفتار با هر یک از الگوریتم‌های برنامه‌ریزیِ مسیرِ فضایِ پیکربندیِ بخشِ قبلی چقدر دشوار می‌بود. نخست، به مدلِ دقیقِ زمین نیاز می‌داشتیم. فضایِ پیکربندیِ رباتِ شش‌پایی که هر پایش توسط دو موتورِ مستقل رانده می‌شود، در مجموع ۱۸ بُعد است (۱۲ بُعد برای پیکربندیِ پاها، و شش بُعد برای مکان و جهتِ ربات نسبت به محیطش). حتی اگر رایانه‌هایمان برای یافتنِ مسیرها در چنین فضاهای پربعدی به‌اندازهٔ کافی سریع بودند، باید دربارهٔ اثرهای ناخوشایندی مانندِ سرخوردنِ ربات از شیب نگران می‌بودیم.

به‌دلیلِ چنین اثرهای تصادفی، مسیرِ تکی از میانِ فضایِ پیکربندی تقریباً به‌طورِ قطع بیش‌ازحد شکننده می‌بود، و حتی کنترل‌گرِ PID ممکن است نتواند با چنین پیشامدهایی کنار بیاید. به بیانِ دیگر، تولیدِ تعمقمحورِ رفتارِ حرکتی در برخی موارد برای الگوریتم‌های برنامه‌ریزیِ حرکتِ امروزیِ ربات صرفاً مسئلهٔ بسیار پیچیده‌ای است.

متأسفانه، معماریِ جذب مشکلاتِ خودش را دارد. نخست، AFSMها توسط ورودیِ حس‌گذارِ خام رانده می‌شوند، چیدمانی که اگر دادهٔ حس‌گذار قابلِ‌اعتماد باشد و همهٔ اطلاعاتِ لازم برای تصمیم‌گیری را شامل شود کار می‌کند، اما اگر دادهٔ حس‌گذار باید به‌روش‌های غیربی‌اهمیت در طولِ زمان تلفیق شود شکست می‌خورد. کنترل‌گرهای سبکِ جذب بنابراین بیشتر برای تکلیف‌های ساده، مانندِ دنبال‌کردنِ دیوار یا حرکت به سمتِ منابعِ نورِ دیدنی، اعمال شده‌اند. دوم، کمبودِ تعمق، تغییرِ هدف‌های ربات را دشوار می‌کند. رباتِ با معماریِ جذب معمولاً فقط یک تکلیف انجام می‌دهد، و هیچ تصوری از نحوهٔ اصلاحِ کنترل‌هایش برای جا دادنِ هدف‌های متفاوت ندارد (درست مانندِ سوسکِ سرگین‌غلتانِ صفحهٔ ۴۱).

سوم، در بسیاری از مسائلِ جهانِ واقعی، سیاستی که می‌خواهیم اغلب برای رمزکردنِ صریح بیش‌ازحد پیچیده است. به مثالِ شکل ۲۶.۲۸ فکر کنید، خودرویِ خودرانی که باید تغییرِ خط را با رانندهٔ انسانی مذاکره کند. ممکن است با سیاستِ ساده‌ای شروع کنیم که به خطِّ هدف می‌رود. اما وقتی خودرو را آزمایش می‌کنیم، می‌فهمیم هر راننده‌ای در خطِّ هدف برای راه‌دادنِ خودرو کند نمی‌کند. سپس ممکن است کمی پیچیدگی اضافه کنیم: خودرو را به سمتِ خطِّ هدف هل بده، منتظرِ پاسخِ رانندهٔ آن خط باش، و سپس یا ادامه بده یا عقب‌نشین. اما بعد خودرو را آزمایش می‌کنیم و می‌فهمیم هل‌دادن باید با سرعتِ متفاوتی بسته به سرعتِ وسیلهٔ نقلیهٔ خطِّ هدف، به وجودِ وسیلهٔ نقلیهٔ دیگری در جلویِ خطِّ هدف، به وجودِ وسیلهٔ نقلیه‌ای پشتِ خودرو در خطِّ اولیه و... رخ دهد. تعدادِ شرایطی که برای تعیینِ مسیرِ درستِ کنش باید در نظر بگیریم می‌تواند بسیار بزرگ باشد، حتی برای چنین مانورِ فریبنده‌ای ساده. این به‌نوبهٔ خود چالش‌های مقیاس‌پذیری برای معماری‌های سبکِ جذب ارائه می‌دهد.

با همهٔ این‌ها، رباتیک مسئلهٔ پیچیده‌ای با رویکردهای بسیار است: تعمقمحور، واکنشی، یا مخلوطی از آن‌ها؛ مبتنی بر فیزیک، مدل‌های شناختی، داده، یا مخلوطی از آن‌ها. رویکردِ درست همچنان موضوعِ بحث، کاوشِ علمی و نبوغِ مهندسی است.

۲۶.۱۰ حوزه‌های کاربردی

فناوریِ رباتیک از قبل در حالِ نفوذ به جهانِ ماست و پتانسیلِ بهبودِ استقلال، سلامت و بهره‌وریِ ما را دارد. در این‌جا چند کاربردِ نمونه آمده.

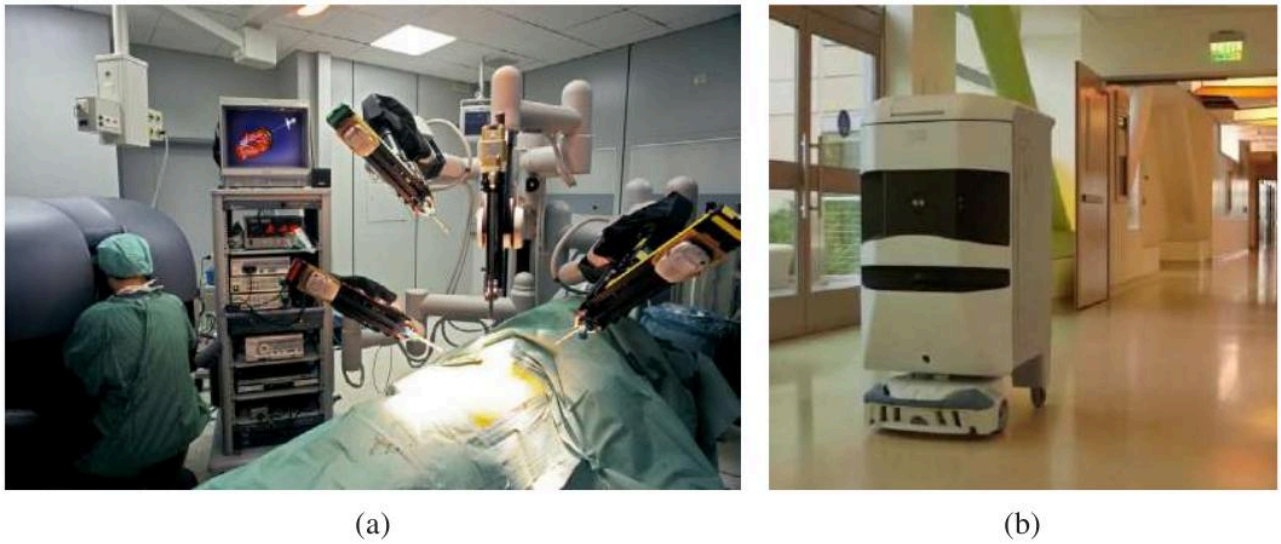
مراقبتِ خانگی: ربات‌ها شروع به ورود به خانه برای مراقبت از سالمندان و افرادِ دارایِ اختلالِ حرکتی کرده‌اند، با کمک به آن‌ها در فعالیت‌های زندگیِ روزمره و قادرکردنِ آن‌ها به زندگیِ مستقل‌تر. این‌ها شاملِ ویلچرها و بازوهای نصب‌شده‌روی-ویلچر مانندِ بازویِ Kinova از شکل ۲۶.۱ (ب) است. حتی اگر آن‌ها در آغاز توسط انسان مستقیماً کار می‌کنند، این ربات‌ها در حالِ کسبِ خودمختاریِ بیشتر و بیشتری‌اند. در افق، ربات‌هایی هستند که توسط «رابط‌های مغز-ماشین» کار می‌کنند، که نشان داده شده افرادِ دارایِ فلجِ چهاراندام را قادر می‌کنند با بازویِ رباتیک اشیا را بگیرند و حتی به خودشان غذا بدهند (شکل ۲۶.۳۳ (الف)). مرتبط با این‌ها اندام‌های پروتزی هستند که هوشمندانه به کنش‌های ما پاسخ می‌دهند، و اسکلت‌های بیرونی که قدرتِ ابرانسانی به ما می‌دهند یا افرادِ ناتوانِ کنترلِ ماهیچه‌هایِ از کمر به پایین را دوباره قادر به راه‌رفتن می‌کنند. ربات‌های شخصی قرار است در تکلیف‌های روزمره مانندِ تمیزکردن و مرتب‌کردن به ما کمک کنند و وقتِ ما را آزاد کنند. هرچند دست‌کاری هنوز راهی دارد تا بتواند به‌صورتِ بی‌درز در محیط‌های انسانیِ شلوغ و بدونِ ساختار عمل کند، ناوبری پیشرفتی داشته. به‌طورِ خاص، بسیاری از خانه‌ها از جاروبرقیِ رباتیکِ متحرکی مانندِ شکل ۲۶.۳۳ (ب) لذت می‌برند.

(a) (b)

شکل ۲۶.۳۳ – (الف) بیمارِ با رابطِ مغز-ماشین که بازویِ رباتیک را برای برداشتنِ نوشیدنی کنترل می‌کند. تصویر با اجازه از دانشگاه براون. (ب) رومبا، جاروبرقیِ رباتیک. عکس از HANDOUT/KRT/Newscom.

مراقبتِ بهداشتی: ربات‌ها جراحان را یاری و تقویت می‌کنند، و رویه‌های دقیق‌تر، کم‌تهاجمی‌تر و ایمن‌تری با پیامدهای بهتر برای بیمار ممکن می‌کنند. رباتِ جراحِ داوینچی از شکل ۲۶.۳۴ (الف) اکنون به‌طورِ گسترده‌ای در بیمارستان‌های آمریکا مستقر شده.

خدمات: ربات‌های متحرک در ساختمان‌های اداری، هتل‌ها و بیمارستان‌ها کمک می‌کنند. Savioke ربات‌هایی در هتل‌ها گذاشته که محصولاتی مانندِ حوله یا خمیردندان را به اتاقِ شما تحویل می‌دهند. ربات‌های HelpMate و TUG غذا و دارو در بیمارستان‌ها حمل می‌کنند (شکل ۲۶.۳۴ (ب))، در حالی که رباتِ Moxi شرکتِ Diligent Robotics به پرستاران در مسئولیت‌های لجستیکیِ پشت‌صحنه کمک می‌کند. Co-Bot در راهروهای دانشگاه کارنگی‌ملون می‌گردد، آمادهٔ راهنماییِ شما به دفترِ کسی. همچنین می‌توانیم از «ربات‌های حضورِ از راهِ دور» (Telepresence Robot) مانندِ Beam برای شرکت در جلسات و کنفرانس‌ها از راهِ دور، یا سرزدن به پدربزرگ‌ها و مادربزرگ‌ها استفاده کنیم.



شکل ۲۶.۳۴ – (الف) رباتِ جراحی در اتاقِ عمل. عکس از Patrick Landmann/Science Source. (ب) رباتِ تحویلِ بیمارستانی. عکس از Wired.

خودروهای خودران: برخی از ما گهگاه هنگامِ رانندگی حواس‌پرتی داریم، توسط تماس‌های تلفنِ همراه، پیامک‌ها یا حواس‌پرتی‌های دیگر. نتیجهٔ غم‌انگیز: بیش از یک میلیون نفر هر سال در تصادفاتِ جاده‌ای می‌میرند. افزون بر این، بسیاری از ما زمانِ زیادی صرفِ رانندگی می‌کنیم و دوست داریم بخشی از آن زمان را بازپس بگیریم. همهٔ این‌ها به تلاشِ عظیمِ در جریان برای استقرارِ خودروهای خودران انجامیده. نمونه‌های اولیه از دههٔ ۱۹۸۰ وجود داشته‌اند، اما پیشرفت توسط چالشِ بزرگِ DARPA در ۲۰۰۵، مسابقهٔ خودرویِ خودمختار بر فرازِ ۲۰۰ کیلومترِ چالش‌برانگیزِ زمینِ بیابانیِ تمرین‌نشده، تحریک شد. خودرویِ استنلیِ استنفورد مسیر را در کمتر از هفت ساعت کامل کرد و جایزهٔ ۲ میلیون دلاری و جایی در موزهٔ ملیِ تاریخِ آمریکا برد.

(a) (b)

شکل ۲۶.۳۵ – (الف) خودروی خودرانِ BOSS که چالشِ شهریِ DARPA را برد. عکس از Tangi Quemener/AFP/Getty Images/Newscom. با اجازه از سباستین ترون. (ب) نمایِ هوایی که ادراک و پیش‌بینی‌های خودروی خودرانِ ویمو (خودروی سفید با مسیرِ سبز) را نشان می‌دهد. سایرِ وسایلِ نقلیه (جعبه‌های آبی) و عابرانِ پیاده (جعبه‌های نارنجی) با مسیرهای پیش‌بینی‌شده نشان داده شده‌اند. مرزهای جاده/پیاده‌رو زردند. عکس با اجازه از ویمو.

شکل ۲۶.۳۵ (الف) BOSS را ترسیم می‌کند، که در ۲۰۰۷ چالشِ شهریِ DARPA را برد، مسابقهٔ جاده‌ایِ پیچیده‌ای در خیابان‌های شهر که ربات‌ها با ربات‌های دیگر روبه‌رو بودند و باید قوانینِ راهنمایی را رعایت می‌کردند. در ۲۰۰۹، گوگل پروژهٔ رانندگیِ خودمختار (با بسیاری از پژوهشگرانی که رویِ استنلی و BOSS کار کرده بودند) را شروع کرد، که اکنون به‌صورتِ ویمو جدا شده. در ۲۰۱۸ ویمو آزمایشِ بدونِ راننده (با هیچ‌کس در صندلیِ راننده) را در حومه‌های فینیکس، آریزونا شروع کرد. در این میان، شرکت‌های خودرویِ خودران و شرکت‌های اشتراک‌سواریِ دیگر رویِ توسعهٔ فناوریِ خودشان کار می‌کنند، در حالی که خودروسازان خودروهایی با هوشِ کمکیِ بیشتر و بیشتر فروخته‌اند، مانندِ «کمک‌راننده»ی (Driver Assist) تسلا، که برای رانندگیِ بزرگراهی در نظر گرفته شده. شرکت‌های دیگر کاربردهای غیربزرگراهی از جملهِ پردیس‌های دانشگاهی و جوامعِ بازنشستگان را هدف می‌گیرند. هنوز شرکت‌های دیگر بر کاربردهای غیرمسافر مانندِ کامیون‌داری، تحویلِ موادِ غذایی و پارکینگِ ولی تمرکز کرده‌اند.

سرگرمی: دیزنی از دههٔ ۱۹۶۳ از ربات‌ها (تحتِ نامِ «انیماترونیک» (Animatronics)) در پارک‌هایش استفاده می‌کند. در اصل، این ربات‌ها به حرکت (و گفتار) دست‌طراحی‌شده، حلقهٔ باز و بدونِ تغییر محدود بودند، اما از ۲۰۰۹ نسخه‌ای به نامِ «اتونوماترونیک» (Autonomatronics) می‌تواند کنش‌های خودمختار تولید کند. ربات‌ها همچنین شکلِ اسباب‌بازی‌های هوشمند برای کودکان را می‌گیرند؛ برای نمونه، Cozmo آنکی با کودکان بازی می‌کند و ممکن است وقتی می‌بازد با ناامیدی به میز بکوبد. سرانجام، چهارپره‌هایی مانندِ R1 اسکایدیو از شکل ۲۶.۲ (ب) به‌عنوانِ عکاس و فیلم‌بردارِ شخصی عمل می‌کنند، دورِ ما می‌چرخند تا هنگامِ اسکی یا دوچرخه‌سواری عکس‌های اکشن بگیرند.

کاوش و محیط‌های خطرناک: ربات‌ها به جاهایی رفته‌اند که هیچ انسانی قبلاً نرفته، از جملهِ سطحِ مریخ. بازوهای رباتیک به فضانوردان در استقرار و بازیابیِ ماهواره‌ها و در ساختنِ ایستگاهِ فضاییِ بین‌المللی کمک می‌کنند. ربات‌ها همچنین به کاوشِ زیرِ دریا کمک می‌کنند. آن‌ها به‌طورِ معمول برای کسبِ نقشه‌های کشتی‌های غرق‌شده استفاده می‌شوند. **شکل ۲۶.۳۶ – (الف) رباتی که معدنِ زغالِ رها شده را نگاشت می‌کند. (ب) نقشهٔ سه‌بعدیِ معدن که توسط ربات به‌دست آمده. با اجازه از سباستین ترون.**

شکل ۲۶.۳۶ رباتی را نشان می‌دهد که معدنِ زغالِ رها شده را نگاشت می‌کند، همراه با مدلِ سه‌بعدیِ معدنِ به‌دست‌آمده با استفاده از حس‌گذارهای برد. در ۱۹۹۶، تیمی از پژوهشگران رباتِ پاداری را در دهانهٔ آتشفشانِ فعالی رها کردند تا برای پژوهشِ اقلیمی داده کسب کند. ربات‌ها به ابزارهای بسیار مؤثری برای جمع‌آوریِ اطلاعات در حوزه‌هایی تبدیل می‌شوند که دسترسی برای مردم دشوار (یا خطرناک) است.

ربات‌ها به مردم در پاکسازیِ زباله‌های هسته‌ای کمک کرده‌اند، به‌ویژه در تری‌مایل‌آیلند، چرنوبیل و فوکوشیما. ربات‌ها پس از فروریختنِ مرکزِ تجارتِ جهانی حاضر بودند، جایی که واردِ سازه‌هایی شدند که برای تیم‌های جستجو و نجاتِ انسانی بیش‌ازحد خطرناک تلقی می‌شدند. این‌جا نیز، این ربات‌ها ابتدا از طریقِ دوراپراتوری مستقر می‌شوند، و با پیشرفتِ فناوری، خودمختارتر و خودمختارتر می‌شوند، با اپراتورِ انسانیِ مسئول اما نه مجبور به مشخص‌کردنِ هر فرمانِ تکی.

صنعت: اکثریتِ ربات‌های امروز در کارخانه‌ها مستقرند، و تکلیف‌هایی را خودکار می‌کنند که برای انسان‌ها دشوار، خطرناک یا کسل‌کننده‌اند. (اکثریتِ ربات‌های کارخانه‌ای در کارخانه‌های خودرو هستند.) خودکارکردنِ این تکلیف‌ها از نظرِ تولیدِ کارآمدِ آنچه جامعه نیاز دارد مثبت است. در همان زمان، به معنای جابه‌جاییِ برخی کارگرانِ انسانی از شغل‌هایشان نیز هست. این پیامدهای سیاستی و اقتصادیِ مهمی دارد—نیاز به بازآموزی و آموزش، نیاز به تقسیمِ منصفانهٔ منابع و... این موضوعات در بخش ۲۷.۳.۵ بیشتر بحث می‌شوند.

خلاصهٔ فصل

رباتیک دربارهٔ عامل‌های فیزیکیِ تجسم‌یافته است، که می‌توانند حالتِ جهانِ فیزیکی را تغییر دهند. در این فصل، مواردِ زیر را یاد گرفته‌ایم:

- رایج‌ترین انواعِ ربات‌ها «دستکار»ها (بازوهای رباتیک) و ربات‌های متحرک‌اند. آن‌ها حس‌گذارهایی برای ادراکِ جهان و عملگرهایی دارند که حرکت تولید می‌کنند، که سپس از طریقِ اثرگذارها بر جهان اثر می‌گذارد.
- مسئلهٔ عمومیِ رباتیک شاملِ تصادفی‌بودن (که با MDPها قابلِ مدیریت است)، مشاهدهٔ جزئی (که با POMDPها قابلِ مدیریت است)، و عمل‌کردن با و دورِ عامل‌های دیگر (که با نظریهٔ بازی قابلِ مدیریت است) است. مسئله حتی سخت‌تر می‌شود، زیرا بیشترِ ربات‌ها در فضاهای حالت و کنشِ پیوسته و پربعد کار می‌کنند. آن‌ها همچنین در جهانِ واقعی عمل می‌کنند، که حاضر نیست سریع‌تر از زمانِ واقعی اجرا شود و در آن شکست‌ها به آسیب‌دیدنِ چیزهای واقعی می‌انجامند، بدونِ قابلیتِ «واگردانی».
- در حالتِ ایده‌آل، ربات کلِ مسئله را یک‌جا حل می‌کرد: مشاهده‌ها به شکلِ خوراکِ حس‌گذارِ خام وارد می‌شوند، و کنش‌ها به شکلِ گشتاورها یا جریان‌های موتور بیرون می‌آیند. در عمل، با این حال، این بیش‌ازحد دلهره‌آور است، و رباتیست‌ها معمولاً جنبه‌های متفاوتِ مسئله را از هم جدا می‌کنند و مستقل درمان می‌کنند.
- معمولاً ادراک (تخمین) را از کنش (تولیدِ حرکت) جدا می‌کنیم. ادراکِ رباتیک شاملِ بینایی رایانه برای بازشناسیِ محیط از طریقِ دوربین‌ها، و همچنین مکان‌یابی و نگاشت است.
- ادراکِ رباتیک به برآوردِ کمیت‌های مربوط به تصمیم از دادهٔ حس‌گذار مربوط می‌شود. برای انجامِ این، به بازنماییِ درونی و روشی برای به‌روزرسانیِ این بازنماییِ درونی در طولِ زمان نیاز داریم.
- الگوریتم‌های فیلترِ احتمالاتی مانندِ فیلترهای ذره‌ای و فیلترهای کالمن برای ادراکِ ربات مفیدند. این فنون حالتِ باور، توزیعِ پسینِ رویِ متغیرهای حالت، را حفظ می‌کنند.
- برای تولیدِ حرکت، از فضاهای پیکربندی استفاده می‌کنیم، که نقطه‌ای در آن‌ها هر آنچه برای مکان‌یابیِ هر نقطهٔ بدنهٔ ربات لازم است را مشخص می‌کند. برای نمونه، برای بازویِ رباتیکِ با دو مفصل، پیکربندی شاملِ دو زاویهٔ مفصل است.
- معمولاً مسئلهٔ تولیدِ حرکت را به برنامه‌ریزیِ حرکت، مربوط به تولیدِ برنامه، و کنترلِ دنبال‌کردنِ مسیر، مربوط به تولیدِ سیاستی برای ورودی‌های کنترل (فرمان‌های عملگر) که به اجرایِ برنامه می‌انجامد، تجزیه می‌کنیم.
- برنامه‌ریزیِ حرکت را می‌توان با جستجویِ گراف با استفاده از تجزیهٔ سلولی حل کرد؛ با الگوریتم‌های برنامه‌ریزیِ حرکتِ تصادفی، که نقطه‌های عطف را در فضایِ پیکربندیِ پیوسته نمونه‌برداری می‌کنند؛ یا با بهینه‌سازیِ مسیر، که می‌تواند به‌صورتِ تکراری مسیرِ خطِّ راست را با بهره‌گیری از میدانِ فاصلهٔ علامت‌دار از برخورد بیرون براند.

- مسیری که توسط الگوریتمِ جستجو یافت شده را می‌توان با استفاده از مسیر به‌عنوانِ مسیرِ زمانیِ مرجعِ کنترل‌گرِ PID، که دائماً خطاهای بینِ جایی که ربات هست و جایی که قرار است باشد را تصحیح می‌کند، یا از طریقِ کنترلِ گشتاورِ محاسبه‌شده، که عبارتِ پیش‌خوری می‌افزاید که از دینامیکِ معکوس برای محاسبهٔ تقریبیِ اینکه برای پیشرفت در طولِ مسیر چه گشتاوری بفرستد استفاده می‌کند، اجرا کرد.
- کنترلِ بهینه برنامه‌ریزیِ حرکت و دنبال‌کردنِ مسیر را با محاسبهٔ مستقیمِ مسیرِ بهینه رویِ ورودی‌های کنترل یکی می‌کند. این به‌ویژه وقتی هزینه‌های درجهٔ دوم و دینامیکِ خطی داشته باشیم آسان است، که به تنظیم‌کنندهٔ خطیِ درجهٔ دوم (LQR) می‌انجامد. روش‌های محبوب با خطی‌کردنِ دینامیک و محاسبهٔ تقریب‌های مرتبهٔ دومِ هزینه (ILQR) از این بهره می‌کنند.
- برنامه‌ریزیِ تحتِ عدم‌قطعیت ادراک و کنش را با بازبرنامه‌ریزیِ برخط (مانندِ کنترلِ پیش‌بینِ مدل) و کنش‌های جمع‌آوریِ اطلاعاتی که به ادراک کمک می‌کنند یکی می‌کند.
- یادگیریِ تقویتی در رباتیک اعمال می‌شود، با فنونی که برای کاهشِ تعدادِ برهم‌کنش‌های لازم با جهانِ واقعی می‌کوشند. این فنون تمایل به بهره‌برداری از مدل‌ها دارند، چه برآوردِ مدل‌ها و استفاده از آن‌ها برای برنامه‌ریزی، چه آموزشِ سیاست‌هایی که نسبت به پارامترهای مدلِ متفاوتِ ممکن مقاوم‌اند.
- تعامل با انسان‌ها مستلزمِ توانایی هماهنگ‌کردنِ کنش‌های ربات با کنش‌های آن‌هاست، که می‌تواند به‌عنوانِ بازی صورتبندی شود. معمولاً راه‌حل را به پیش‌بینی، که در آن از کنش‌های جاریِ شخص برای برآوردِ آنچه در آینده انجام خواهد داد استفاده می‌کنیم، و کنش، که در آن از پیش‌بینی‌ها برای محاسبهٔ حرکتِ بهینهٔ ربات استفاده می‌کنیم، تجزیه می‌کنیم.
- کمک به انسان‌ها همچنین مستلزمِ توانایی یادگیری یا استنتاجِ آنچه می‌خواهند است. ربات‌ها می‌توانند با یادگیریِ تابعِ هزینهٔ مطلوبی که باید بهینه کنند از ورودیِ انسانی، مانندِ نمایش‌ها، اصلاح‌ها یا آموزشِ زبانِ طبیعی به این نزدیک شوند. یا، ربات‌ها می‌توانند رفتارِ انسانی را تقلید کنند و از یادگیریِ تقویتی برای کمک به غلبه بر چالشِ تعمیم به حالت‌های تازه استفاده کنند.

یادداشت‌های کتابشناختی و تاریخی

واژهٔ «ربات» توسط نمایش‌نامه‌نویسِ چک، کارل چاپک، در نمایشنامهٔ 1920 او R.U.R. (ربات‌های جهانیِ راسوم) رایج شد. ربات‌ها، که به‌جای ساختِ مکانیکی به‌صورتِ شیمیایی رشد داده می‌شدند، در نهایت از اربابانشان متنفر می‌شوند و تصمیم به تسخیر می‌گیرند. به نظر می‌رسد برادرِ چاپک، یوزف، نخستین کسی بود که واژه‌های چکیِ «robota» (کارِ اجباری) و «robotnik» (سرف) را در داستانِ کوتاهِ 1917 خود «Opilec» (گلانس، 1978) ترکیب کرد و «ربات» را ساخت. اصطلاحِ «رباتیک» برای داستانی علمی-تخیلی (آزیموف، 1950) ابداع شد.

ایدهٔ ماشینِ خودمختار هزاران سال از واژهٔ «ربات» پیشی دارد. در اسطوره‌شناسیِ یونانیِ قرنِ هفتمِ پیش از میلاد، رباتی به نامِ تالوس توسط هفایستوس، خدایِ یونانیِ فلزکاری، برای محافظت از جزیرهٔ کرت ساخته شد. افسانه می‌گوید جادوگرِ مدئا با وعدهٔ جاودانگی تالوس را شکست داد اما سپس مایعِ حیاتش را تخلیه کرد. پس، این نخستین نمونهٔ رباتی است که در فرایندِ تغییرِ تابعِ هدفش اشتباه می‌کند.

در 322 پیش از میلاد، ارسطو بیکاریِ فناورانه را پیش‌بینی کرد و حدس زد «اگر هر ابزاری، وقتی فرمان داده شود، یا حتی به‌اختیارِ خودش، می‌توانست کاری را که شایسته‌اش است انجام دهد... آنگاه نه برای استادکاران نیازی به شاگردان بود و نه برای اربابان به برده‌ها.» در قرنِ سومِ پیش از میلاد، رباتِ انسان‌نمایِ واقعی به نامِ «خدمتکارِ فیلون» می‌توانست شراب یا آب در فنجان بریزد؛ مجموعه‌ای از شیرها جریان را در زمانِ درست قطع می‌کردند. اتوماتاهای شگفت‌انگیزی در قرنِ هجدهم ساخته شدند—اردکِ مکانیکیِ ژاک ووکانسون از 1738 یکی از نمونه‌های اولیه است—اما رفتارهای پیچیده‌ای که نشان می‌دادند کاملاً از پیش ثابت شده بودند. احتمالاً قدیمی‌ترین نمونهٔ دستگاهِ ربات‌مانندِ برنامه‌پذیر ماشینِ بافندگیِ ژاکارد (1805) بود، که در صفحهٔ ۱۵ توصیف شده.

«لاک‌پشت»ِ گری والتر، ساخته‌شده در 1948، را می‌توان نخستین رباتِ متحرکِ خودمختار در نظر گرفت، هرچند سامانهٔ کنترلی‌اش برنامه‌پذیر نبود. «هیولای هاپکینز»، ساخته‌شده در 1960 در دانشگاه جانز هاپکینز، بسیار پیچیده‌تر بود؛ سونار و حس‌گذارهای فتوسل، سخت‌افزارِ بازشناسیِ الگو داشت، و می‌توانست صفحهٔ پوششِ پریزِ برقِ استانداردِ AC را بازشناسی کند. قادر به جستجویِ پریزها، وصل‌کردنِ خودش و سپس شارژِ باتری‌هایش بود! با این حال، هیولا مجموعهٔ محدودی از مهارت‌ها داشت.

نخستین رباتِ متحرکِ همه‌منظوره «شیکی» بود، که در آنچه سپس مؤسسهٔ تحقیقاتیِ استنفورد (اکنون SRI) نامیده می‌شد در اواخرِ دههٔ ۱۹۶۰ توسعه یافت (فایکس و نیلسون، 1971؛ نیلسون، 1984). شیکی نخستین رباتی بود که ادراک، برنامه‌ریزی و اجرا را یکپارچه کرد، و پژوهشِ بعدیِ زیادی در هوش مصنوعی تحتِ تأثیرِ این دستاوردِ چشمگیر قرار گرفت. شیکی رویِ جلدِ این کتاب با رهبرِ پروژه، چارلی روزن (1917–2002)، ظاهر می‌شود. پروژه‌های تأثیرگذارِ دیگر شاملِ گاریِ استنفورد و رورِ CMU (موراوس، 1983) است. کاکس و ویل‌فونگ (1990) کارِ کلاسیکِ وسایلِ نقلیهٔ خودمختار را توصیف می‌کنند.

نخستین رباتِ تجاری بازویی به نامِ UNIMATE، مخففِ اتوماسیونِ همگانی، بود که توسط جوزف انگلبرگر و جورج دوول در شرکتِ‌شان، یونیمیشن، توسعه یافت. در 1961، نخستین رباتِ UNIMATE به جنرال‌موتورز برای استفاده در تولیدِ لوله‌های تصویرِ تلویزیون فروخته شد. 1961 همچنین سالی بود که دوول نخستین حق‌امتیازِ آمریکاییِ ربات را گرفت.

در 1973، تویوتا و نیسان شروع به استفاده از نسخهٔ به‌روزشدهٔ UNIMATE برای جوشِ نقطه‌ایِ بدنهٔ خودرو کردند. این انقلابِ بزرگی در تولیدِ خودرو آغاز کرد که عمدتاً در ژاپن و آمریکا رخ داد و هنوز ادامه دارد. یونیمیشن در 1978 با توسعهٔ رباتِ Puma (ماشینِ همگانیِ برنامه‌پذیر برای مونتاژ) دنبال کرد، که برای دو دههٔ پس از آن استانداردِ دِ فاکتوِ دست‌کاریِ رباتیک بود. هر سال حدودِ ۵۰۰,۰۰۰ ربات فروخته می‌شود، با نیمی از آن‌ها که به صنعتِ خودرو می‌روند.

در دست‌کاری، نخستین تلاشِ بزرگ برای ایجادِ ماشینِ دست-چشم، ِMH-1 هاینریش ارنست بود، توصیف‌شده در پایان‌نامهٔ دکترایِ MIT او (ارنست، 1961). پروژهٔ هوشِ ماشینیِ ادینبورگ نیز سامانهٔ اولیهٔ چشمگیری برای مونتاژِ مبتنی بر بینایی به نامِ FREDDY نشان داد (میچی، 1972).

پژوهشِ رباتیکِ متحرک توسط چند رقابتِ مهم تحریک شده. رقابتِ سالانهٔ رباتِ متحرکِ AAAI در 1992 آغاز شد. نخستین برندهٔ رقابت CARMEL بود (کانگدون و همکاران، 1992). پیشرفت پیوسته و چشمگیر بوده: در رقابت‌های اخیر ربات‌ها واردِ مجموعهٔ کنفرانس شدند، راهشان را به میزِ ثبت‌نام پیدا کردند، برای کنفرانس ثبت‌نام کردند و حتی سخنرانیِ کوتاهی دادند.

رقابتِ RoboCup، که در 1995 توسط کیتانو و همکاران (1997) راه‌اندازی شد، هدفش «توسعهٔ تیمی از ربات‌های انسان‌نمایِ کاملاً خودمختار است که می‌توانند تا 2050 تیمِ قهرمانِ جهانِ انسانی را در فوتبال ببرند». برخی رقابت‌ها از ربات‌های چرخ‌دار، برخی از ربات‌های انسان‌نما و برخی از شبیه‌سازی‌های نرم‌افزاری استفاده می‌کنند. استون (2016) نوآوری‌های اخیرِ RoboCup را توصیف می‌کند. چالشِ بزرگِ DARPA، که توسط DARPA در 2004 و 2005 سازماندهی شد، از وسایلِ نقلیهٔ خودمختار می‌خواست بیش از ۲۰۰ کیلومتر را در بیابان در کمتر از ده ساعت طی کنند (بوهلر و همکاران، 2006). در رویدادِ اصلیِ 2004، هیچ رباتی بیش از هشت مایل نرفت، که بسیاری را به این باور رساند که جایزه هرگز گرفته نمی‌شود. در 2005، رباتِ استنلیِ استنفورد رقابت را در کمی کمتر از هفت ساعت برد (ترون، 2006). DARPA سپس چالشِ شهری را سازماندهی کرد، رقابتی که ربات‌ها در آن باید ۶۰ مایل را در محیطِ شهری با ترافیکِ دیگر ناوبری می‌کردند. رباتِ BOSS دانشگاه کارنگی‌ملون مقامِ نخست را گرفت و جایزهٔ ۲ میلیون دلاری را برد (اورمسون و ویتاکر، 2008). پیشگامانِ اولیه در توسعهٔ خودروهای رباتیک شاملِ دیکمنس و زاپ (1987) و پومرلو (1993) بودند.

حوزهٔ نگاشتِ رباتیک از دو خاستگاهِ متمایز تکامل یافته. نخستین رشته با کارِ اسمیت و چیسمن (1986) آغاز شد، که فیلترهای کالمن را بر مسئلهٔ مکان‌یابی و نگاشتِ هم‌زمان (SLAM) اعمال کردند. این الگوریتم نخستین بار توسط موتارلیه و شتیلا (1989) پیاده‌سازی و بعداً توسط لئونارد و دورانت-وایت (1992) بسط یافت؛ برای مروری بر تغییرهای اولیهٔ فیلترِ کالمن، دیسانایاکه و همکاران (2001) را ببینید. رشتهٔ دوم با توسعهٔ «شبکهٔ اشغال» (Occupancy Grid) برای نگاشتِ احتمالاتی آغاز شد، که احتمالِ اشغال‌شدنِ هر مکانِ (x,y) توسط مانع را مشخص می‌کند (موراوس و الفس، 1985). کوییپرس و لویت (1988) از نخستین کسانی بودند که به‌جای نگاشتِ متریک، نگاشتِ توپولوژیک را پیشنهاد کردند، با انگیزه از مدل‌های شناختِ فضاییِ انسانی. مقالهٔ بنیادینِ لو و میلیوس (1997) تنکیِ مسئلهٔ مکان‌یابی و نگاشتِ هم‌زمان را بازشناسی کرد، که به توسعهٔ فنونِ بهینه‌سازیِ غیرخطی توسط کونولیگه (2004) و مونتمرلو و ترون (2004)، و همچنین روش‌های سلسله‌مراتبی توسط بوسه و همکاران (2004) انجامید.

شاتکی و کلبلینگ (1997) و ترون و همکاران (1998) الگوریتمِ EM را برای ارتباطِ داده به حوزهٔ نگاشتِ رباتیک معرفی کردند. نمایِ کلی از روش‌های نگاشتِ احتمالاتی را می‌توان در (ترون و همکاران، 2005) یافت.

فنونِ اولیهٔ مکان‌یابیِ رباتِ متحرک توسط بورنشتاین و همکاران (1996) مرور شده. هرچند فیلترِ کالمن برای دهه‌ها به‌عنوانِ روشِ مکان‌یابی در نظریهٔ کنترل شناخته‌شده بود، صورتبندیِ احتمالاتیِ عمومیِ مسئلهٔ مکان‌یابی تا مدتِ بسیار بعدتر در ادبیاتِ هوش مصنوعی ظاهر نشد، از طریقِ کارِ تام دین و همکاران (دین و همکاران، 1990) و سیمونز و کونیگ (1995). کارِ اخیر اصطلاحِ «مکان‌یابیِ مارکوف» (Markov Localization) را معرفی کرد. نخستین کاربردِ جهانِ واقعیِ این فن توسط بورگارد و همکاران (1999)، از طریقِ مجموعه‌ای از ربات‌های مستقر در موزه‌ها، بود. مکان‌یابیِ مونت‌کارلوِ مبتنی بر فیلترهای ذره‌ای توسط فاکس و همکاران (1999) توسعه یافت و اکنون به‌طورِ گسترده‌ای استفاده می‌شود. «فیلترِ ذره‌ایِ رائو-بلکولایز» فیلترِ ذره‌ایِ مکان‌یابیِ ربات را با فیلترِ دقیقِ نقشه‌سازی ترکیب می‌کند (مورفی و راسل، 2001؛ مونتمرلو و همکاران، 2002).

بخشِ زیادی از کارِ اولیهٔ برنامه‌ریزیِ حرکت بر الگوریتم‌های هندسی برای مسائلِ برنامه‌ریزیِ حرکتِ قطعی و کاملاً مشاهده‌پذیر متمرکز بود. PSPACE-سختیِ برنامه‌ریزیِ حرکتِ ربات در مقالهٔ بنیادینِ رایف (1979) نشان داده شد. بازنماییِ فضایِ پیکربندی متعلق به لوزانو-پرس (1983) است. مجموعه‌ای از مقاله‌های شوارتز و شریبر دربارهٔ آنچه مسئله‌های جابه‌جاییِ پیانو می‌نامیدند (شوارتز و همکاران، 1987) بسیار تأثیرگذار بود.

تجزیهٔ سلولیِ بازگشتی برای برنامه‌ریزیِ فضایِ پیکربندی در کارِ بروکس و لوزانو-پرس (1985) سرچشمه گرفت و توسط ژو و لاتومبه (1991) به‌شکلِ چشمگیری بهبود یافت. نخستین الگوریتم‌های اسکلت‌سازی بر نمودارهای ورونوی (روات، 1979) و گراف‌های دیدپذیری (وسلی و لوزانو-پرس، 1979) مبتنی بودند. گیباس و همکاران (1992) فنونِ کارآمدی برای محاسبهٔ تدریجیِ نمودارهای ورونوی توسعه دادند، و شوسه (1996) نمودارهای ورونوی را به مسائلِ برنامه‌ریزیِ حرکتِ وسیع‌تری تعمیم داد.

جان کنی (1988) نخستین الگوریتمِ تک‌نمایی برای برنامه‌ریزیِ حرکت را برقرار کرد. متنِ بنیادینِ لاتومبه (1991) انواعِ رویکردها به برنامه‌ریزیِ حرکت را پوشش می‌دهد، و همچنین متن‌های شوسه و همکاران (2005) و لاواله (2006). کاوراکی و همکاران (1996) نظریهٔ نقشه‌های راهِ احتمالاتی را توسعه دادند. کوفنر و لاواله (2000) درخت‌های تصادفیِ سریع‌گسترش (RRT) را توسعه دادند. ورودِ بهینه‌سازی به برنامه‌ریزیِ حرکتِ هندسی با نوارهای کشسان (کوینلان و خاتبی، 1993) آغاز شد، که مسیرها را وقتی موانعِ فضایِ پیکربندی تغییر می‌کنند پالایش می‌کنند. راتلیف و همکاران (2009) این ایده را به‌عنوانِ راه‌حلِ مسئلهٔ کنترلِ بهینه صورتبندی کردند، که به مسیرِ اولیه اجازه می‌دهد در برخورد شروع شود، و با نگاشتِ گرادیان‌های مانعِ فضایِ کار از طریقِ ژاکوبین به فضایِ پیکربندی آن را تغییر شکل می‌دهد. شولمن و همکاران (2013) جایگزینِ عملیِ مرتبهٔ دومی پیشنهاد کردند.

کنترلِ ربات‌ها به‌عنوانِ سامانه‌های دینامیکی—چه برای دست‌کاری چه برای ناوبری—ادبیاتِ عظیمی تولید کرده. هرچند این فصل مبانیِ کنترلِ دنبال‌کردنِ مسیر و کنترلِ بهینه را توضیح داد، زیرشاخه‌های کاملی را کنار گذاشت، از جملهِ کنترلِ تطبیقی، کنترلِ مقاوم و تحلیلِ لیاپانوف. کنترلِ تطبیقی به‌جای فرضِ شناخته‌بودنِ همه‌چیزِ سامانه از پیش، هدفش سازگارکردنِ پارامترهای دینامیک و/یا قانونِ کنترل به‌صورتِ برخط است. کنترلِ مقاوم، از سوی دیگر، هدفش طراحیِ کنترل‌گرهایی است که با وجودِ عدم‌قطعیت و اختلال‌های خارجی خوب عمل می‌کنند.

تحلیلِ لیاپانوف در اصل در دههٔ ۱۸۹۰ برای تحلیلِ پایداریِ سامانه‌های غیرخطیِ عمومی توسعه یافت، اما تا اوایلِ دههٔ ۱۹۳۰ نظریه‌پردازانِ کنترل به پتانسیلِ واقعی‌اش پی نبردند. با توسعهٔ روش‌های بهینه‌سازی، تحلیلِ لیاپانوف به توابعِ مانعِ کنترلی بسط یافت، که به‌خوبی خود را به ابزارهای بهینه‌سازیِ مدرن می‌دهند. این روش‌ها به‌طورِ گسترده‌ای در رباتیکِ مدرن برای طراحیِ کنترل‌گرِ زمان-واقعی و تحلیلِ ایمنی استفاده می‌شوند. کارهای حیاتی در کنترلِ رباتیک شاملِ سه‌گانه‌ای دربارهٔ کنترلِ امپدانس توسط هوگان (1985) و مطالعهٔ عمومیِ دینامیکِ ربات توسط فدرستون (1987) است. دین و ولمن (1991) از نخستین کسانی بودند که کوشیدند نظریهٔ کنترل و سامانه‌های برنامه‌ریزیِ هوش مصنوعی را به هم گره بزنند. سه کتابِ درسیِ کلاسیک دربارهٔ ریاضیاتِ دست‌کاریِ ربات متعلق به پل (1981)، کریگ (1989) و یوشیکاوا (1990) است. کنترل برای دست‌کاری توسط موری (2017) پوشش داده شده.

حوزهٔ گرفتن (Grasping) نیز در رباتیک مهم است—مسئلهٔ تعیینِ گیرهٔ پایدار کاملاً دشوار است (میسن و سالیسبری، 1985). گرفتنِ شایسته مستلزمِ حسِ لمس، یا «بازخوردِ لمسی» (Haptic Feedback)، برای تعیینِ نیروهای تماس و تشخیصِ لغزش است (فیرینگ و هولرباخ، 1985). فهمِ نحوهٔ گرفتنِ انواعِ گوناگونِ اشیای جهان کاری دلهره‌آور است. (بوسمالیس و همکاران، 2017) سامانه‌ای توصیف می‌کنند که آزمایشِ جهانِ واقعی را با شبیه‌سازی‌های هدایت‌شده توسط انتقالِ شبیه-به-واقعیت ترکیب می‌کند تا گرفتنِ مقاوم تولید کند.

«کنترلِ میدانِ پتانسیل» (Potential-Field Control)، که می‌کوشد مسائلِ برنامه‌ریزیِ حرکت و کنترل را هم‌زمان حل کند، برای رباتیک توسط خاتبی (1986) توسعه یافت. در رباتیکِ متحرک، این ایده به‌عنوانِ راه‌حلِ عملیِ مسئلهٔ اجتناب از برخورد دیده شد و بعداً توسط بورنشتاین (1991) به الگوریتمی به نامِ «هیستوگرامِ میدانِ برداری» (Vector Field Histogram) بسط یافت.

ILQR اکنون به‌طورِ گسترده‌ای در محلِ تلاقیِ برنامه‌ریزیِ حرکت و کنترل استفاده می‌شود و متعلق به لی و تودوروف (2004) است. این گونه‌ای از فنِ بسیار قدیمی‌ترِ «برنامه‌نویسیِ دینامیکیِ دیفرانسیلی» (Jacobson and Mayne, 1970) است.

برنامه‌ریزیِ حرکتِ ریز با حس‌کردنِ محدود توسط لوزانو-پرس و همکاران (1984) و کنی و رایف (1987) بررسی شد. ناوبریِ مبتنی بر نقطهٔ نشانه (لازاناس و لاتومبه، 1992) بسیاری از همان ایده‌ها را در عرصهٔ رباتِ متحرک استفاده می‌کند. توابعِ ناوبری، نسخهٔ رباتیکِ سیاستِ کنترلی برای MDPهای قطعی، توسط کودیتشک (1987) معرفی شدند. کارِ کلیدیِ اعمالِ روش‌های POMDP (بخش 17.4) برای برنامه‌ریزیِ حرکتِ تحتِ عدم‌قطعیت در رباتیک متعلق به پینو و همکاران (2003) و روی و همکاران (2005) است.

یادگیریِ تقویتی در رباتیک با کارِ بنیادینِ باگنل و اشنایدر (2001) و نگ و همکاران (2003) اوج گرفت، که پارادایم را در بافتِ کنترلِ هلیکوپترِ خودمختار توسعه دادند. کوبر و همکاران (2013) نمایِ کلی از اینکه یادگیریِ تقویتی وقتی بر مسئلهٔ رباتیک اعمال می‌شود چگونه تغییر می‌کند ارائه می‌دهد. بسیاری از فنونِ پیاده‌سازی‌شده رویِ سامانه‌های فیزیکی مدل‌های دینامیکِ تقریبی می‌سازند، که به مدل‌های خطیِ وزن‌دارِ محلیِ آتکسون و همکاران (1997) بازمی‌گردد. اما گرادیان‌های سیاست نیز نقششان را ایفا کردند، و ربات‌های انسان‌نمای (ساده‌شده) را قادر به راه‌رفتن (تدریک و همکاران، 2004)، یا بازویِ رباتیک را قادر به ضربه‌زدن به توپِ بیسبال (پیترز و شال، 2008) کردند.

لوین و همکاران (2016) نخستین کاربردِ یادگیریِ تقویتیِ عمیق را رویِ رباتِ واقعی نشان دادند. در همان زمان، RL بدونِ مدل در شبیه‌سازی به حوزه‌های پیوسته بسط می‌یافت (شولمن و همکاران، 2015a؛ هیس و همکاران، 2016؛ لیلیک‌رپ و همکاران، 2015). کارِ دیگر جمع‌آوریِ دادهٔ فیزیکی را به‌صورتِ عظیم مقیاس‌بندی کرد تا یادگیریِ گیره‌ها و مدل‌های دینامیک را نمایش دهد (پینتو و گوپتا، 2016؛ آگراوال و همکاران، 2017؛ لوین و همکاران، 2018). انتقال از شبیه‌سازی به واقعیت یا شبیه-به-واقعیت (صادقی و لوین، 2016؛ آندری‌خویچ و همکاران، 2018a)، فرا-یادگیری (فین و همکاران، 2017)، و یادگیریِ تقویتیِ بدونِ مدلِ نمونه‌کارآمد (آندری‌خویچ و همکاران، 2018b) حوزه‌های پژوهشیِ فعال‌اند.

روش‌های اولیهٔ پیش‌بینیِ کنش‌های انسانی از رویکردهای فیلترکردن استفاده می‌کردند (مادهاوان و شلنوف، 2003)، اما کارِ بنیادینِ زیبارت و همکاران (2009) پیش‌بینی را با مدل‌کردنِ مردم به‌عنوانِ عامل‌های تقریباً عقلایی پیشنهاد کرد. صدیق و همکاران (2016) درگیرِ این کردند که این پیش‌بینی‌ها در واقع چگونه باید به آنچه ربات تصمیم می‌گیرد انجام دهد بستگی داشته باشند، و به سمتِ محیطِ نظریه‌بازی ساختند. برای محیط‌های همکارانه، سیسبو و همکاران (2007) ایدهٔ درنظرگرفتنِ آنچه مردم در تابعِ هزینهٔ ربات می‌خواهند را پیشگامی کردند. نیکولایدیس و شاه (2013) همکاری را به یادگیریِ نحوهٔ عملِ انسان، و همچنین یادگیریِ نحوهٔ خواستِ انسان از ربات برای عمل، که هر دو از نمایش‌ها قابلِ‌دستیابی‌اند، تجزیه کردند. برای یادگیری از نمایش، آرگال و همکاران (2009) را ببینید. آکگون و همکاران (2012) و صدیق‌گر و همکاران (2017) آموزش توسط کاربرانِ نهایی به‌جای کارشناسان را مطالعه کردند. تلکس و همکاران (2011) نشان دادند ربات‌ها چگونه می‌توانند آنچه مردم از دستورهای زبانِ طبیعی می‌خواهند را استنتاج کنند. سرانجام، نه‌تنها ربات‌ها باید استنتاج کنند مردم چه می‌خواهند و قصدِ انجامِ چه دارند، بلکه مردم نیز باید همان استنتاج‌ها را دربارهٔ ربات‌ها انجام دهند. دراگان و همکاران (2013) مدلی از استنتاج‌های انسانی را در برنامه‌ریزیِ حرکتِ ربات گنجاندند.

حوزهٔ تعاملِ انسان-ربات بسیار گسترده‌تر از آن چیزی است که در این فصل پوشش دادیم، که عمدتاً بر جنبه‌های برنامه‌ریزی و یادگیری متمرکز بود. توماز و همکاران (2016) مروری بر تعامل به‌طورِ گسترده‌تر از دیدگاهِ محاسباتی فراهم می‌کند. راس و همکاران (2011) سامانهٔ DAGGER را توصیف می‌کنند.

موضوعِ معماری‌های نرم‌افزاری برای ربات‌ها بحثِ مذهبیِ زیادی برمی‌انگیزد. نامزدِ قدیمیِ خوب هوش مصنوعی—معماریِ سه‌لایه—به طراحیِ شیکی بازمی‌گردد و توسط گت (1998) مرور شده. معماریِ جذب متعلق به بروکس (1986) است، هرچند ایده‌های مشابهی به‌طورِ مستقل توسط برایتنبرگ توسعه یافت، که کتابش، «وسایلِ نقلیه» (1984)، مجموعه‌ای از ربات‌های ساده را بر مبنای رویکردِ رفتاری توصیف می‌کند.

موفقیتِ رباتِ راه‌روندهٔ شش‌پایِ بروکس توسط پروژه‌های بسیاری دنبال شد. کانل، در پایان‌نامهٔ دکترایش (1989)، رباتِ متحرکِ کاملاً واکنشی‌ای توسعه داد که قادر به بازیابیِ اشیا بود. بسطِ پارادایم به سامانه‌های چندرباته را می‌توان در کارِ پارکر (1996) و ماتاریچ (1997) یافت. GRL (هورسویل، 2000) و COLBERT (کونولیگه، 1997) ایده‌های رباتیکِ هم‌زمانِ مبتنی بر رفتار را در زبان‌های کنترلِ رباتِ عمومی انتزاع می‌کنند. آرکین (1998) برخی از محبوب‌ترین رویکردهای این حوزه را مرور می‌کند.

دو کتابِ درسیِ اولیه، از دودک و جنکین (2000) و مورفی (2000)، رباتیک را به‌طورِ کلی پوشش می‌دهند. مرورهای جدیدتر متعلق به بیکی (2008) و لینچ و پارک (2017) است. کتابِ عالیِ دست‌کاریِ ربات موضوعاتِ پیشرفته‌ای مانندِ حرکتِ سازگار (میسن، 2001) را بررسی می‌کند. برنامه‌ریزیِ حرکتِ ربات در شوسه و همکاران (2005) و لاواله (2006) پوشش داده شده. ترون و همکاران (2005) رباتیکِ احتمالاتی را معرفی می‌کند. «کتابِ راهنمای رباتیک» (سیسیلیانو و خاتبی، 2016) نمایِ کلیِ عظیم و جامعی از همهٔ رباتیک است.

کنفرانسِ برترِ رباتیک «کنفرانسِ رباتیک: علم و سامانه‌ها» است، و پس از آن «کنفرانسِ بین‌المللیِ IEEE رباتیک و اتوماسیون». «تعاملِ انسان-ربات» برترین محلِ برگزاریِ تعامل است. مجله‌های برترِ رباتیک شاملِ IEEE Robotics and Automation، مجلهٔ بین‌المللیِ پژوهشِ رباتیک و Robotics and Autonomous Systems است.

پانوشت‌ها:

[1] رباتیست‌ها دوست دارند تابعِ هزینهٔ L را کمینه کنند، در حالی که در سایرِ بخش‌های هوش مصنوعی می‌کوشیم تابعِ مطلوبیتِ U یا پاداشِ R را بیشینه کنیم.

[2] جزئیاتِ f^{-1} را این‌جا حذف می‌کنیم، اما شاملِ جرم، اینرسی، گرانش و نیروهای کوریولیس و گریز از مرکز است.

فصل ۲۷: فلسفه، اخلاق و ایمنی هوش مصنوعی (Philosophy, Ethics, and Safety of AI)

در این فصل پرسش‌های بزرگِ پیرامونِ معنای هوش مصنوعی، اینکه چگونه می‌توانیم آن را به‌صورتِ اخلاقی توسعه و اعمال کنیم، و چگونه می‌توانیم آن را ایمن نگه داریم را بررسی می‌کنیم.

فیلسوفان مدتِ طولانی است پرسش‌های بزرگ می‌پرسند: ذهن‌ها چگونه کار می‌کنند؟ آیا ممکن است ماشین‌ها به‌شیوه‌ای که مردم انجام می‌دهند هوشمندانه عمل کنند؟ آیا چنین ماشین‌هایی ذهن‌های واقعی و خودآگاه خواهند داشت؟ به این‌ها، پرسش‌های تازه‌ای می‌افزاییم: پیامدهای اخلاقیِ ماشین‌های هوشمند در استفادهٔ روزمره چیست؟ آیا باید به ماشین‌ها اجازه داده شود تصمیم بگیرند انسان‌ها را بکشند؟ آیا الگوریتم‌ها می‌توانند منصفانه و بدونِ اریب باشند؟ اگر ماشین‌ها بتوانند همهٔ انواعِ کار را انجام دهند، انسان‌ها چه خواهند کرد؟ و چگونه ماشین‌هایی را کنترل می‌کنیم که ممکن است از ما هوشمندتر شوند؟

۲۷.۱ محدودیت‌های هوش مصنوعی

در ۱۹۸۰، فیلسوفِ جان سرل تمایزی بینِ «هوش مصنوعیِ ضعیف» (Weak AI)—ایدهٔ اینکه ماشین‌ها می‌توانستند طوری عمل کنند که گویی هوشمندند—و «هوش مصنوعیِ قوی» (Strong AI)—ادعای اینکه ماشین‌هایی که چنین می‌کنند در واقع آگاهانه فکر می‌کنند (نه فقط شبیه‌سازیِ فکر)—معرفی کرد. در طولِ زمان، تعریفِ هوش مصنوعیِ قوی تغییر کرد تا به چیزی اشاره کند که «هوش مصنوعیِ سطح-انسان» یا «هوش مصنوعیِ عمومی» نیز نامیده می‌شود—برنامه‌هایی که می‌توانند طیفِ دلخواهاً وسیعی از تکلیف‌ها، از جملهِ تکلیف‌های تازه، را حل کنند و این کار را به‌خوبیِ انسان انجام دهند. منتقدانِ هوش مصنوعیِ ضعیف که به خودِ امکانِ رفتارِ هوشمندانه در ماشین‌ها اعتراض داشتند، اکنون به همان اندازهٔ سایمون نیوکامب کم‌بین به نظر می‌رسند، که در اکتبرِ ۱۹۰۳ نوشت «پروازِ هوایی یکی از آن طبقهٔ بزرگِ مسائلی است که انسان هرگز نمی‌تواند از پسش برآید»—فقط دو ماه پیش از پروازِ برادران رایت در کیتی‌هاوک. پیشرفتِ سریعِ سال‌های اخیر، با این حال، ثابت نمی‌کند هیچ محدودیتی برای آنچه هوش مصنوعی می‌تواند به دست آورد نیست. آلن تورینگ (1950)، نخستین کسی که هوش مصنوعی را تعریف کرد، همچنین نخستین کسی بود که اعتراض‌های احتمالی به هوش مصنوعی را مطرح کرد و تقریباً همهٔ آن‌هایی را که بعداً توسط دیگران مطرح شدند پیش‌بینی کرد.

۲۷.۱.۱ استدلال از غیررسمی‌بودن

«استدلال از غیررسمی‌بودنِ رفتار»یِ تورینگ می‌گوید رفتارِ انسانی برای درگیرشدن در هر مجموعهٔ صوری از قاعده‌ها بسیار پیچیده است—انسان‌ها باید از برخی رهنمودهای غیررسمی استفاده کنند که (به ادعایِ استدلال) هرگز نمی‌توانند در مجموعه‌ای صوری از قاعده‌ها درگیر شوند و بنابراین هرگز نمی‌توانند در برنامهٔ رایانه‌ای مدون شوند.

مدافعِ کلیدیِ این دیدگاه هوبرت دریفوس بود، که مجموعه‌ای از نقدهای تأثیرگذار بر هوش مصنوعی تولید کرد: «آنچه رایانه‌ها نمی‌توانند انجام دهند» (1972)، دنبالهٔ آن «آنچه رایانه‌ها همچنان نمی‌توانند انجام دهند» (1992)، و، با برادرش استوارت، «ذهن بر ماشین» (1986). به‌همین‌سان، فیلسوفِ کنت سایر (1993) گفت «هوش مصنوعیِ دنبال‌شده در کیشِ محاسبه‌گرایی حتی یک شانسِ شبح‌گونه هم برای تولیدِ نتایجِ ماندگار ندارد.» فناوری‌ای که آن‌ها نقد می‌کردند به نامِ «هوش مصنوعیِ قدیمیِ خوب» (Good Old-Fashioned AI یا GOFAI) شناخته شد.

GOFAI با ساده‌ترین طراحیِ عاملِ منطقیِ توصیف‌شده در فصل ۷ متناظر است، و آن‌جا دیدیم درگیرکردنِ هر پیشامدِ رفتارِ مناسب در مجموعه‌ای از قاعده‌های منطقیِ لازم و کافی واقعاً دشوار است؛ آن را مسئلهٔ صلاحیت نامیدیم. اما چنان‌که در فصل ۱۲ دیدیم، سامانه‌های استدلالِ احتمالاتی برای حوزه‌های بازپایان مناسب‌ترند، و چنان‌که در فصل ۲۱ دیدیم، سامانه‌های یادگیری عمیق در انواعِ تکلیف‌های «غیررسمی» خوب عمل می‌کنند. پس، نقد متوجهِ خودِ رایانه‌ها نیست، بلکه متوجهِ سبکِ خاصی از برنامه‌نویسیِ آن‌ها با قاعده‌های منطقی است—سبکی که در دههٔ ۱۹۸۰ محبوب بود اما توسط رویکردهای تازه تحت‌الشعاع قرار گرفته.

یکی از قوی‌ترین استدلال‌های دریفوس به نفعِ عامل‌های موقعیت‌مند است، نه موتورهای استنتاجِ منطقیِ بی‌جسم. عاملی که فهمش از «سگ» فقط از مجموعه‌ای محدود از جمله‌های منطقی مانندِ «$\text{Dog}(x) \Rightarrow \text{Mammal}(x)$» می‌آید، در مقایسه با عاملی که سگ‌ها را در حالِ دویدن دیده، با آن‌ها بازیِ فچ کرده و توسط یکی لیسیده شده، در مضیقه است. چنان‌که فیلسوفِ اندی کلارک (1998) می‌گوید، «مغزهای زیستی اول از همه سامانه‌های کنترلِ بدن‌های زیستی‌اند. بدن‌های زیستی در محیط‌های غنیِ جهانِ واقعی حرکت و عمل می‌کنند.» به گفتهٔ کلارک، ما «در فریزبی خوبیم، در منطق بدیم.»

رویکردِ «شناختِ تجسم‌یافته» (Embodied Cognition) ادعا می‌کند درنظرگرفتنِ مغز به‌صورتِ جداگانه معنا ندارد: شناخت درونِ بدنی رخ می‌دهد که در محیطی جاسازی شده. باید سامانه را به‌عنوانِ کل مطالعه کنیم؛ عملکردِ مغز از نظم‌های محیطش، از جملهِ بقیهٔ بدنش، بهره برداری می‌کند. تحتِ رویکردِ شناختِ تجسم‌یافته، رباتیک، بینایی و دیگر حس‌گذارها مرکزی می‌شوند، نه پیرامونی.

در مجموع، دریفوس حوزه‌هایی را دید که هوش مصنوعی در آن‌ها پاسخ‌های کامل نداشت و گفت بنابراین هوش مصنوعی ناممکن است؛ ما اکنون بسیاری از همین حوزه‌ها را در حالِ پژوهش و توسعهٔ ادامه‌دار می‌بینیم که به توانایی فزاینده می‌انجامند، نه ناممکنی.

۲۷.۱.۲ استدلال از ناتوانی

«استدلال از ناتوانی» ادعا می‌کند «ماشین هرگز نمی‌تواند X انجام دهد.» به‌عنوانِ مثال‌هایی از X، تورینگ مواردِ زیر را فهرست می‌کند:

مهربان، مدبر، زیبا، دوستانه بودن، ابتکار داشتن، حسِ شوخ‌طبعی داشتن، درست را از نادرست تشخیص دادن، اشتباه‌کردن، عاشق‌شدن، از توت‌فرنگی با خامه لذت‌بردن، کسی را عاشقِ خود کردن، از تجربه یادگرفتن، استفادهٔ درست از واژه‌ها، موضوعِ اندیشهٔ خود بودن، به‌اندازهٔ انسان تنوعِ رفتار داشتن، کاری واقعاً تازه کردن.

به‌صورتِ گذشته‌نگر، برخی از این‌ها نسبتاً آسان‌اند—همهٔ ما با رایانه‌هایی که «اشتباه می‌کنند» آشنا هستیم. رایانه‌های با قابلیتِ فراستدلال (فصل 5) می‌توانند محاسباتِ خودشان را بررسی کنند، و بنابراین موضوعِ استدلالِ خودشان باشند. فناوریِ صدساله‌ای توانایی اثبات‌شدهٔ «عاشقِ خودکردنِ کسی» را دارد—خرسِ عروسکی. کارشناسِ شطرنجِ رایانه‌ای دیوید لوی پیش‌بینی می‌کند تا 2050 مردم به‌طورِ معمول عاشقِ ربات‌های انسان‌نما خواهند شد. در موردِ عاشق‌شدنِ خودِ ربات، این تمِ رایجی در داستان‌هاست،[1] اما فقط گمانه‌پردازیِ دانشگاهیِ محدودی دربارهٔ این موضوع بوده (کیم و همکاران، 2007). رایانه‌ها کارهای «واقعاً تازه‌ای» انجام داده‌اند، با کشف‌های چشمگیر در نجوم، ریاضیات، شیمی، کانی‌شناسی، زیست‌شناسی، علومِ رایانه و حوزه‌های دیگر، و ایجادِ شکل‌های تازه‌ای از هنر از طریقِ انتقالِ سبک (گاتیس و همکاران، 2016). در مجموع، برنامه‌ها در برخی تکلیف‌ها از عملکردِ انسانی فراتر می‌روند و در برخی دیگر عقب می‌مانند. تنها چیزی که روشن است نمی‌توانند انجام دهند دقیقاً انسان‌بودن است.

۲۷.۱.۳ اعتراضِ ریاضی

تورینگ (1936) و گودل (1931) اثبات کردند برخی پرسش‌های ریاضی در اصل توسط سامانه‌های صوریِ خاصی پاسخ‌ناپذیرند. قضیهٔ ناتمامیتِ گودل (بخش 9.5 را ببینید) مشهورترین نمونهٔ این است. به‌طورِ خلاصه، برای هر چارچوبِ اصل‌موضوعیِ صوریِ F که به‌اندازهٔ کافی قدرتمند باشد که حساب انجام دهد، امکانِ ساختِ به‌اصطلاح «جملهٔ گودل»ی G(F) با ویژگی‌های زیر هست:

- G(F) جمله‌ای از F است، اما نمی‌توان درونِ F اثباتش کرد.
- اگر F سازگار باشد، آنگاه G(F) صادق است.

فیلسوفانی مانندِ جی. آر. لوکاس (1961) ادعا کرده‌اند این قضیه نشان می‌دهد ماشین‌ها از نظرِ ذهنی از انسان‌ها فروترند، زیرا ماشین‌ها سامانه‌های صوری‌اند که توسط قضیهٔ ناتمامیت محدود می‌شوند—آن‌ها نمی‌توانند صدقِ جملهٔ گودلِ خودشان را برقرار کنند—در حالی که انسان‌ها چنین محدودیتی ندارند. این جنجالِ زیادی ایجاد کرده و ادبیاتِ عظیمی را زاییده، از جملهِ دو کتاب از ریاضی‌دان/فیزیکدانِ سر راجر پنروز (1989, 1994). پنروز ادعای لوکاس را با چند پیچِ تازه تکرار می‌کند، مانندِ فرضیهٔ اینکه انسان‌ها متفاوت‌اند، زیرا مغزشان با گرانشِ کوانتومی کار می‌کند—نظریه‌ای که پیش‌بینی‌های نادرستِ متعددی دربارهٔ فیزیولوژیِ مغز می‌کند.

سه مورد از مشکلاتِ ادعای لوکاس را بررسی خواهیم کرد. نخست، عامل نباید شرمنده باشد که نمی‌تواند صدقِ جمله‌ای را برقرار کند در حالی که عامل‌های دیگر می‌توانند. جملهٔ زیر را در نظر بگیر:

لوکاس نمی‌تواند به‌صورتِ سازگار ادعا کند این جمله صادق است.

اگر لوکاس این جمله را ادعا می‌کرد، با خودش تناقض می‌داشت، پس بنابراین لوکاس نمی‌تواند به‌صورتِ سازگار آن را ادعا کند، و از این رو صادق است. پس نشان داده‌ایم جملهٔ صادقی هست که لوکاس نمی‌تواند به‌صورتِ سازگار ادعایش کند در حالی که مردمِ دیگر (و ماشین‌ها) می‌توانند. اما این باعث نمی‌شود دربارهٔ لوکاس کمتر فکر کنیم.

دوم، قضیهٔ ناتمامیتِ گودل و نتایجِ مرتبط به ریاضیات اعمال می‌شوند، نه به رایانه‌ها. هیچ موجودیتی—انسانی یا ماشینی—نمی‌تواند چیزهایی را اثبات کند که اثباتشان ناممکن است. لوکاس و پنروز به‌صورتِ نادرست فرض می‌کنند انسان‌ها به‌نحوی می‌توانند از این محدودیت‌ها دور بزنند، چنان‌که لوکاس (1976) می‌گوید «باید سازگاریِ خودمان را فرض کنیم، اگر قرار است اندیشه اصلاً ممکن باشد.» اما این فرضِ بی‌دلیلی است: انسان‌ها به‌شکلِ بدنامی ناسازگارند. این قطعاً برای استدلالِ روزمره صادق است، اما برای اندیشهٔ ریاضیِ دقیق نیز صادق است. مثالِ مشهوری مسئلهٔ نقشهٔ چهاررنگ است. آلفرد کمپ (1879) اثباتی منتشر کرد که برای ۱۱ سال به‌طورِ گسترده‌ای پذیرفته شد تا پرسی هیوود (1890) به نقصی اشاره کرد.

سوم، قضیهٔ ناتمامیتِ گودل از نظرِ فنی فقط برای سامانه‌های صوریِ به‌اندازهٔ کافی قدرتمند برای انجامِ حساب اعمال می‌شود. این شاملِ ماشین‌های تورینگ است، و ادعای لوکاس تا حدی بر این گزاره مبتنی است که رایانه‌ها با ماشین‌های تورینگ هم‌ارزند. این کاملاً درست نیست. ماشین‌های تورینگ نامتناهی‌اند، در حالی که رایانه‌ها (و مغزها) متناهی‌اند، و هر رایانه‌ای بنابراین می‌تواند به‌عنوانِ سامانه‌ای (بسیار بزرگ) در منطقِ گزاره‌ای توصیف شود، که تابعِ قضیهٔ ناتمامیتِ گودل نیست. لوکاس فرض می‌کند انسان‌ها می‌توانند «نظرشان را عوض کنند» در حالی که رایانه‌ها نمی‌توانند، اما آن نیز نادرست است—رایانه می‌تواند پس از شاهدِ تازه یا تعمقِ بیشتر نتیجه‌ای را پس بگیرد؛ می‌تواند سخت‌افزارش را ارتقا دهد؛ و می‌تواند فرایندهای تصمیم‌گیری‌اش را با یادگیریِ ماشینی یا بازنویسیِ نرم‌افزار تغییر دهد.

۲۷.۱.۴ اندازه‌گیریِ هوش مصنوعی

آلن تورینگ، در مقالهٔ مشهورش «ماشین‌های محاسبه و هوش» (1950)، پیشنهاد کرد به‌جای پرسیدنِ اینکه آیا ماشین‌ها می‌توانند فکر کنند، بپرسیم آیا ماشین‌ها می‌توانند آزمونِ رفتاری‌ای را پاس کنند، که به نامِ «آزمونِ تورینگ» شناخته شده. این آزمون ایجاب می‌کند برنامه‌ای گفتگویی (از طریقِ پیام‌های تایپ‌شده) با بازجو به‌مدتِ پنج دقیقه داشته باشد. بازجو سپس باید حدس بزند گفتگو با برنامه است یا شخص؛ برنامه آزمون را پاس می‌کند اگر ۳۰٪ مواقع بازجو را بفریبد. برای تورینگ، نکتهٔ کلیدی جزئیاتِ دقیقِ آزمون نبود، بلکه ایدهٔ اندازه‌گیریِ هوش با عملکردِ رویِ نوعی تکلیفِ رفتاریِ بازپایان، به‌جای گمانه‌پردازیِ فلسفی، بود.

با این وجود، تورینگ حدس زد تا سالِ ۲۰۰۰ رایانه‌ای با حافظهٔ یک میلیارد واحدی می‌تواند آزمون را پاس کند، اما این‌جاییم آن سویِ ۲۰۰۰، و همچنان نمی‌توانیم توافق کنیم آیا برنامه‌ای پاسش کرده. بسیاری از مردم وقتی نمی‌دانستند ممکن است با رایانه گپ بزنند فریب خورده‌اند. برنامهٔ ELIZA و ربات‌های گفتگویِ اینترنتی مانندِ MGONZ (هامفریس، 2008) و NATACHATA (جاناتان و همکاران، 2009) بارها طرف‌های گفتگویشان را می‌فریبند، و رباتِ گفتگویِ CYBERLOVER توجهِ مجریِ قانون را به‌دلیلِ تمایلش به فریبِ هم‌گفتگوها برای افشایِ اطلاعاتِ شخصیِ کافی برای سرقتِ هویت جلب کرده.

در 2014، رباتِ گفتگویی به نامِ یوجین گوستمن ۳۳٪ از داورانِ آماتورِ آموزش‌ندیده را در آزمونِ تورینگ فریب داد. برنامه ادعا می‌کرد پسری از اوکراین با تسلطِ محدود بر انگلیسی است؛ این به تبیینِ خطاهای دستوری‌اش کمک می‌کرد. شاید آزمونِ تورینگ واقعاً آزمونِ زودباوریِ انسانی است. تاکنون هیچ داورِ خوب‌آموزش‌دیده‌ای فریب نخورده (آرونسون، 2014).

رقابت‌های آزمونِ تورینگ به ربات‌های گفتگویِ بهتری انجامیده‌اند، اما کانونِ پژوهش در جامعهٔ هوش مصنوعی نبوده‌اند. در عوض، پژوهشگرانِ هوش مصنوعی که هوسِ رقابت دارند بیشتر محتمل است بر بازیِ شطرنج یا گو یا StarCraft II تمرکز کنند، یا آزمونِ علومِ کلاسِ هشتم بدهند، یا اشیا را در تصاویر شناسایی کنند. در بسیاری از این رقابت‌ها، برنامه‌ها به عملکردِ سطح-انسان رسیده یا از آن فراتر رفته‌اند، اما این به این معنا نیست برنامه‌ها خارج از تکلیفِ خاص انسان‌مانندند. نکته بهبودِ علم و فناوریِ پایه و فراهم‌کردنِ ابزارهای مفید است، نه فریبِ داوران.

۲۷.۲ آیا ماشین‌ها واقعاً می‌توانند فکر کنند؟

برخی فیلسوفان ادعا می‌کنند ماشینی که هوشمندانه عمل می‌کند در واقع فکر نمی‌کند، بلکه فقط شبیه‌سازیِ فکر است. اما بیشترِ پژوهشگرانِ هوش مصنوعی به این تمایز اهمیت نمی‌دهند، و دانشمندِ رایانهٔ ادسخر دیکسترا (1984) گفت «پرسشِ اینکه آیا ماشین‌ها می‌توانند فکر کنند... تقریباً به همان اندازهٔ پرسشِ اینکه آیا زیردریایی‌ها می‌توانند شنا کنند مرتبط است.» اولین تعریفِ «شناکردن» در فرهنگِ لغتِ آمریکایی هریتیج «حرکت در آب با اندام‌ها، باله‌ها یا دم» است، و بیشترِ مردم موافق‌اند زیردریایی‌ها، چون اندام ندارند، نمی‌توانند شنا کنند. فرهنگِ لغت همچنین «پروازکردن» را «حرکت در هوا با بال‌ها یا اجزایِ بال‌مانند» تعریف می‌کند، و بیشترِ مردم موافق‌اند هواپیماها، چون اجزایِ بال‌مانند دارند، می‌توانند پرواز کنند. با این حال، نه پرسش‌ها و نه پاسخ‌ها هیچ ارتباطی با طراحی یا قابلیت‌های هواپیماها و زیردریایی‌ها ندارند؛ بلکه دربارهٔ کاربردِ واژه در انگلیسی‌اند. (این واقعیت که کشتی‌ها در روسی «شنا می‌کنند» این نکته را تقویت می‌کند.) انگلیسی‌زبان‌ها هنوز بر تعریفِ دقیقی برای واژهٔ «فکرکردن» به توافق نرسیده‌اند—آیا به «مغز» نیاز دارد یا فقط «اجزایِ مغزمانند»؟

باز، این موضوع توسط تورینگ بررسی شد. او اشاره می‌کند ما هرگز هیچ شاهدِ مستقیمی دربارهٔ حالت‌های ذهنیِ درونیِ انسان‌های دیگر نداریم—نوعی خودتنهاانگاریِ ذهنی. با این وجود، تورینگ می‌گوید، «به‌جای بحثِ مداوم بر سرِ این نکته، رسم است «قراردادِ مؤدبانه»ای (Polite Convention) داشته باشیم که همه فکر می‌کنند.» تورینگ استدلال می‌کند ما قراردادِ مؤدبانه را به ماشین‌ها نیز بسط می‌دادیم، اگر فقط با ماشین‌هایی که هوشمندانه عمل می‌کنند تجربه داشتیم. با این حال، حالا که کمی تجربه داریم، به نظر می‌رسد تمایلِ ما به نسبت‌دادنِ احساس دستِ‌کم به همان اندازه به ظاهرِ انسان‌نما و صدا بستگی دارد که به هوشِ ناب.

۲۷.۲.۱ اتاقِ چینی

فیلسوفِ جان سرل قراردادِ مؤدبانه را رد می‌کند. «استدلالِ اتاقِ چینی»ی (Chinese Room) مشهور او (سرل، 1990) به‌صورتِ زیر است: انسانی را تصور کن که فقط انگلیسی می‌فهمد، درونِ اتاقی که شاملِ کتابِ قاعده‌ای، نوشته‌شده به انگلیسی، و چند توده کاغذ است. تکه‌های کاغذِ شاملِ نمادهای رمزناپذیر از زیرِ درِ اتاق به داخل داده می‌شوند. انسان

دستورهای کتابِ قاعده را دنبال می‌کند، نمادها را در توده‌ها می‌یابد، نمادها را رویِ کاغذهای تازه می‌نویسد، توده‌ها را بازچینی می‌کند و مانندِ آن. در نهایت، دستورها باعث می‌شوند یک یا چند نماد رویِ کاغذی رونویسی شود که به جهانِ بیرون بازگردانده می‌شود. از بیرون، سامانه‌ای می‌بینیم که ورودی را در قالبِ جمله‌های چینی می‌گیرد و پاسخ‌های چینیِ روان و هوشمندانه تولید می‌کند.

سرل سپس استدلال می‌کند: داده شده که انسان چینی نمی‌فهمد. کتابِ قاعده و توده‌های کاغذ، چون فقط تکه‌های کاغذند، چینی نمی‌فهمند. بنابراین، هیچ فهمی از چینی وجود ندارد. و سرل می‌گوید اتاقِ چینی همان کاری را می‌کند که رایانه می‌کرد، پس بنابراین رایانه‌ها هیچ فهمی تولید نمی‌کنند.

سرل (1980) طرفدارِ «طبیعت‌گراییِ زیستی» (Biological Naturalism) است، که بر طبقِ آن حالت‌های ذهنی ویژگی‌های نوپدیدِ سطح‌بالایی‌اند که توسط فرایندهای فیزیکیِ سطح‌پایین در نورون‌ها ایجاد می‌شوند، و این (نامشخص‌شده) ویژگی‌های نورون‌هاست که اهمیت دارد: طبقِ اریب‌های سرل، نورون‌ها «آن» را دارند و ترانزیستورها ندارند. ابطال‌های بسیاری از استدلالِ سرل بوده، اما هیچ اجماعی نیست. استدلالِ او می‌توانست به‌یکسان استفاده شود (شاید توسط ربات‌ها) برای اینکه بگوید انسان نمی‌تواند فهمِ واقعی داشته باشد؛ بالاخره، انسان از سلول‌ها ساخته شده، سلول‌ها نمی‌فهمند، پس هیچ فهمی وجود ندارد. در واقع، این طرحِ داستانِ علمی-تخیلیِ تری بیسون (1990) «آن‌ها از گوشت ساخته شده‌اند» است، که در آن ربات‌های بیگانه زمین را کاوش می‌کنند و نمی‌توانند باور کنند تکه‌های گوشت ممکن است احساس داشته باشند. اینکه چگونه می‌توانند داشته باشند همچنان معماست.

۲۷.۲.۲ آگاهی و کیفیّاتِ تجربه

در میانِ همهٔ بحث‌های هوش مصنوعیِ قوی، مسئلهٔ «آگاهی» (Consciousness) جاری است: آگاهی از جهانِ بیرون، و از خود، و تجربهٔ ذهنیِ زندگی. اصطلاحِ فنیِ ماهیتِ ذاتیِ تجربه‌ها «کیفیّات» (Qualia) است (از واژهٔ لاتین به معنایِ تقریباً «از چه نوعی»). پرسشِ بزرگ این است که آیا ماشین‌ها می‌توانند کیفیّات داشته باشند. در فیلمِ 2001، وقتی فضانوردِ دیوید بومن در حالِ قطع‌کردنِ «مدارهای شناختی» رایانهٔ HAL 9000 است، می‌گوید «می‌ترسم، دیو. دیو، ذهنم دارد می‌رود. حسش می‌کنم.» آیا HAL واقعاً احساس دارد (و سزاوارِ همدردی است)؟ یا پاسخ فقط پاسخِ الگوریتمی است، بدونِ تفاوت با «Error 404: not found»؟

پرسشِ مشابهی برای جانوران هست: صاحبانِ حیواناتِ خانگی مطمئن‌اند سگ یا گربه‌شان آگاهی دارد، اما همهٔ دانشمندان موافق نیستند. جیرجیرک‌ها رفتارشان را بر مبنای دما تغییر می‌دهند، اما افرادِ کمی می‌گویند جیرجیرک‌ها احساسِ گرم یا سردبودن را تجربه می‌کنند.

یکی از دلایلِ دشواربودنِ مسئلهٔ آگاهی این است که حتی پس از قرن‌ها بحث همچنان بدتعریف مانده. اما کمک ممکن است در راه باشد. اخیراً فیلسوفان با عصب‌شناسان تحتِ حمایتِ بنیادِ تمپلتون هم‌دست شده‌اند تا مجموعه‌ای از آزمایش‌ها را شروع کنند که می‌تواند برخی از مسائل را حل کند. طرفدارانِ دو نظریهٔ پیشروِی آگاهی (نظریهٔ فضایِ کارِ سراسری و نظریهٔ اطلاعاتِ یکپارچه) موافقت کرده‌اند آزمایش‌ها می‌توانند یکی از نظریه‌ها را بر دیگری تأیید کنند—نادرّیتی در فلسفه.

آلن تورینگ (1950) اذعان می‌کند مسئلهٔ آگاهی دشوار است، اما انکار می‌کند ارتباطِ چندانی با عملِ هوش مصنوعی دارد: «نمی‌خواهم این تصور را بدهم که فکر می‌کنم هیچ رمز و رازی دربارهٔ آگاهی نیست... اما فکر نمی‌کنم این رمز و رازها لزوماً باید پیش از اینکه بتوانیم به پرسشی که در این مقاله با آن سروکار داریم پاسخ دهیم حل شوند.» با تورینگ موافقیم—ما به ایجادِ برنامه‌هایی علاقه‌مندیم که هوشمندانه رفتار می‌کنند. جنبه‌های منفردِ آگاهی—هشیاری، خودآگاهی، توجه—قابلِ برنامه‌نویسی‌اند و می‌توانند بخشی از ماشینِ هوشمند باشند. پروژهٔ اضافیِ ساختنِ ماشینی آگاه دقیقاً به‌شیوه‌ای که انسان‌ها آگاه‌اند، پروژه‌ای نیست که ما مجهز به انجامش باشیم. موافقیم رفتارِ هوشمندانه به درجه‌ای از هشیاری نیاز خواهد داشت، که از تکلیفی به تکلیفِ دیگر تفاوت خواهد کرد، و تکلیف‌های شاملِ تعامل با انسان به مدلی از تجربهٔ ذهنیِ انسانی نیاز خواهند داشت.

در موضوعِ مدل‌کردنِ تجربه، انسان‌ها مزیتِ روشنی بر ماشین‌ها دارند، زیرا می‌توانند از دستگاهِ ذهنیِ خودشان برای قدردانی از تجربهٔ ذهنیِ دیگران استفاده کنند. برای نمونه، اگر می‌خواهید بدانید وقتی کسی با چکش به شستش می‌زند چه حسی دارد، می‌توانید با چکش به شستِ خودتان بزنید. ماشین‌ها چنین قابلیتی ندارند—هرچند برخلافِ انسان‌ها، می‌توانند کدِ یکدیگر را اجرا کنند.

۲۷.۳ اخلاقِ هوش مصنوعی

با توجه به اینکه هوش مصنوعی فناوریِ قدرتمندی است، تعهدِ اخلاقی داریم خوب از آن استفاده کنیم، جنبه‌های مثبت را ترویج و از جنبه‌های منفی اجتناب یا آن‌ها را کاهش دهیم.

جنبه‌های مثبت بسیارند. برای نمونه، هوش مصنوعی می‌تواند از طریقِ تشخیصِ پزشکیِ بهبودیافته، کشف‌های پزشکیِ تازه، پیش‌بینیِ بهترِ رویدادهای شدیدِ آب‌وهوایی، و رانندگیِ ایمن‌تر با کمک‌راننده و (در نهایت) فناوری‌های خودران جان‌ها را نجات دهد. همچنین فرصت‌های بسیاری برای بهبودِ زندگی‌ها هست. برنامهٔ «هوش مصنوعی برای اقدامِ بشردوستانه»ی مایکروسافت هوش مصنوعی را برای بهبود از بلایای طبیعی، رسیدگی به نیازهای کودکان، محافظت از پناهندگان و ترویجِ حقوقِ بشر اعمال می‌کند. برنامهٔ «هوش مصنوعی برای خیرِ اجتماعی»ی گوگل از کار بر حفاظت از جنگل‌های بارانی، فقهِ حقوقِ بشر، پایشِ آلودگی، اندازه‌گیریِ انتشارِ سوخت‌های فسیلی، مشاورهٔ بحران، راستی‌آزماییِ اخبار، پیشگیری از خودکشی، بازیافت و دیگر مسائل پشتیبانی می‌کند. مرکزِ «علمِ داده برای خیرِ اجتماعی» دانشگاه شیکاگو یادگیریِ ماشینی را بر مسائلِ عدالتِ کیفری، توسعهٔ اقتصادی، آموزش، بهداشتِ عمومی، انرژی و محیط اعمال می‌کند.

کاربردهای هوش مصنوعی در مدیریتِ محصول و تولیدِ غذا به تغذیهٔ جهان کمک می‌کنند. بهینه‌سازیِ فرایندهای کسب‌وکار با یادگیریِ ماشینی کسب‌وکارها را مولدتر خواهد کرد، ثروت را افزایش و اشتغالِ بیشتری فراهم خواهد کرد. اتوماسیون می‌تواند جایگزینِ تکلیف‌های کسل‌کننده و خطرناکی شود که بسیاری از کارگران با آن‌ها روبه‌رویند، و آن‌ها را آزاد کند تا بر جنبه‌های جالب‌تر تمرکز کنند. افرادِ دارایِ معلولیت از کمکِ مبتنی بر هوش مصنوعی در دیدن، شنیدن و تحرک بهره خواهند برد. ترجمهٔ ماشینی از قبل به مردمِ فرهنگ‌های متفاوت اجازه می‌دهد ارتباط برقرار کنند. راه‌حل‌های نرم‌افزاریِ مبتنی بر هوش مصنوعی هزینهٔ نهاییِ تولیدِ نزدیکِ صفر دارند، و بنابراین پتانسیلِ دموکراتیک‌کردنِ دسترسی به فناوریِ پیشرفته را دارند (حتی اگر جنبه‌های دیگرِ نرم‌افزار پتانسیلِ متمرکزکردنِ قدرت را داشته باشند).

با وجودِ این جنبه‌های مثبتِ بسیار، نباید منفی‌ها را نادیده بگیریم. بسیاری از فناوری‌های تازه «اثرهای جانبیِ منفیِ ناخواسته» داشته‌اند: شکافتِ هسته‌ای چرنوبیل و تهدیدِ نابودیِ جهانی آورد؛ موتورِ احتراقِ داخلی آلودگیِ هوا، گرمایشِ جهانی و آسفالت‌شدنِ بهشت آورد. فناوری‌های دیگر حتی وقتی همان‌گونه که در نظر گرفته شده استفاده می‌شوند می‌توانند اثرهای منفی داشته باشند، مانندِ گازِ سارین، تفنگ‌های AR-15 و بازاریابیِ تلفنی. اتوماسیون ثروت ایجاد خواهد کرد، اما تحتِ شرایطِ اقتصادیِ کنونی بخشِ بزرگی از آن ثروت به سمتِ مالکانِ سامانه‌های خودکار جریان خواهد یافت، که به نابرابریِ درآمدِ فزاینده می‌انجامد. این می‌تواند برای جامعه‌ای که خوب کار می‌کند مخرب باشد. در کشورهای در حالِ توسعه، مسیرِ سنتیِ رشد از طریقِ تولیدِ کم‌هزینه برای صادرات ممکن است قطع شود، زیرا کشورهای ثروتمند تأسیساتِ تولیدیِ کاملاً خودکار را در داخلِ کشورشان اتخاذ می‌کنند. تصمیم‌های اخلاقی و حکمرانیِ ما سطحِ نابرابریِ ناشی از هوش مصنوعی را دیکته خواهند کرد.

همهٔ دانشمندان و مهندسان با ملاحظاتِ اخلاقیِ اینکه چه پروژه‌هایی را باید یا نباید بر عهده بگیرند، و اینکه چگونه مطمئن شوند اجرایِ پروژه ایمن و سودمند است، روبه‌رویند. در 2010، شورای پژوهشیِ علومِ مهندسی و فیزیکِ بریتانیا جلسه‌ای برای توسعهٔ مجموعه‌ای از «اصول رباتیک» برگزار کرد. در سال‌های بعد، آژانس‌های دولتیِ دیگر، سازمان‌های غیرانتفاعی و شرکت‌ها مجموعه‌های مشابهی از اصول ایجاد کردند. مضمون این است که هر سازمانی که فناوریِ هوش مصنوعی ایجاد می‌کند، و هرکس در سازمان، مسئولیت دارد مطمئن شود فناوری به خیر کمک می‌کند، نه آسیب. پراستنادترینِ اصول عبارت‌اند از:

- تضمینِ ایمنی
- برقراریِ پاسخگویی
- تضمینِ انصاف
- رعایتِ حقوق و ارزش‌های انسانی
- احترام به حریم خصوصی
- بازتابِ تنوع/شمول
- ترویجِ همکاری
- اجتناب از تمرکزِ قدرت
- فراهم‌کردنِ شفافیت
- تصدیقِ پیامدهای حقوقی/سیاستی
- محدودکردنِ کاربردهای مضرِ هوش مصنوعی
- تأمل دربارهٔ پیامدهای اشتغال

توجه کنید بسیاری از اصول، مانندِ «تضمینِ ایمنی»، برای همهٔ سامانه‌های نرم‌افزاری یا سخت‌افزاری کاربرد دارند، نه فقط سامانه‌های هوش مصنوعی. چند اصل به‌شکلی مبهم بیان شده‌اند، که اندازه‌گیری یا اجرایشان را دشوار می‌کند. این تا حدی به‌دلیلِ این است که هوش مصنوعی حوزهٔ بزرگی با زیرحوزه‌های بسیار است، که هر یک مجموعه‌ای متفاوت از هنجارهای تاریخی و رابطه‌های متفاوتی بینِ توسعه‌دهندگانِ هوش مصنوعی و ذی‌نفعان دارد. میت‌لشتات (2019) پیشنهاد می‌کند زیرحوزه‌ها هر یک باید رهنمودهای عملیِ خاص‌تر و رویه‌های موردیِ دقیق‌تری توسعه دهند.

۲۷.۳.۱ سلاح‌های خودمختارِ کشنده

سازمانِ ملل سلاحِ خودمختارِ کشنده را سلاحی تعریف می‌کند که بدونِ نظارتِ انسانی اهدافِ انسانی را مکان‌یابی، انتخاب و موردِ حمله قرار می‌دهد (یعنی می‌کشد). سلاح‌های گوناگونی برخی از این معیارها را برآورده می‌کنند. برای نمونه، مین‌های زمینی از قرنِ هفدهم استفاده شده‌اند: می‌توانند اهداف را به معنایی محدود طبقِ درجهٔ فشارِ اعمال‌شده یا مقدارِ فلزِ حاضر انتخاب و موردِ حمله قرار دهند، اما نمی‌توانند بروند و اهداف را خودشان مکان‌یابی کنند. (مین‌های زمینی تحتِ پیمانِ اتاوا ممنوع‌اند.) موشک‌های هدایت‌شونده، که از دههٔ ۱۹۴۰ استفاده می‌شوند، می‌توانند اهداف را تعقیب کنند، اما باید توسط انسان به جهتِ کلیِ درست نشانه‌گیری شوند. تفنگ‌های کنترل‌شده با رادارِ شلیک‌خودکار از دههٔ ۱۹۷۰ برای دفاع از کشتی‌های دریایی استفاده شده‌اند؛ عمدتاً برای نابودیِ موشک‌های ورودی در نظر گرفته شده‌اند، اما می‌توانند به هواپیماهای سرنشین‌دار نیز حمله کنند. هرچند واژهٔ «خودمختار» اغلب برای توصیفِ وسایلِ نقلیهٔ هواییِ بدونِ سرنشین یا پهپادها استفاده می‌شود، بیشترِ این‌گونه سلاح‌ها هم از راهِ دور هدایت می‌شوند و هم به فعال‌سازیِ انسانیِ محمولهٔ کشنده نیاز دارند.

در زمانِ نوشتنِ این کتاب، چند سامانهٔ تسلیحاتی به نظر می‌رسد از خطِّ ورود به خودمختاریِ کامل عبور کرده‌اند. برای نمونه، موشکِ هاروپِ اسرائیل «مهماتِ پرسه‌زن»ی با طولِ بالِ ده‌فوت و کلاهکِ پنجاه‌پوندی است. تا شش ساعت در ناحیهٔ جغرافیاییِ داده‌شده‌ای به دنبالِ هر هدفی که معیارِ داده‌شده‌ای را برآورده می‌کند می‌گردد و سپس آن را نابود می‌کند. معیار می‌تواند «سیگنالِ راداریِ شبیهِ رادارِ ضدهوایی منتشر می‌کند» یا «شبیهِ تانک به نظر می‌رسد» باشد. سازندهٔ ترکیِ STM پهپادِ چهارپرهٔ کارگو-۲ را—که تا 1.5 کیلوگرم مادهٔ منفجره حمل می‌کند—به‌عنوانِ قادر به «ضربهٔ خودمختار... اهدافِ انتخاب‌شده رویِ تصاویر... ردیابیِ اهدافِ متحرک... ضدِ نفر... بازشناسیِ چهره» تبلیغ می‌کند.

سلاح‌های خودمختار «سومین انقلاب در جنگ‌افزاری» پس از باروت و سلاح‌های هسته‌ای نامیده شده‌اند. پتانسیلِ نظامی‌شان آشکار است. برای نمونه، کارشناسانِ کمی تردید دارند هواپیماهای جنگندهٔ خودمختار هر خلبانِ انسانی را شکست می‌دهند. هواپیماها، تانک‌ها و زیردریایی‌های خودمختار می‌توانند ارزان‌تر، سریع‌تر، مانورپذیرتر و با بردِ طولانی‌تر از همتایانِ سرنشین‌دارشان باشند. از 2014، سازمانِ ملل در ژنو بحث‌های منظمی تحتِ نظارتِ «کنوانسیونِ سلاح‌های متعارفِ خاص» (CCW) دربارهٔ پرسشِ ممنوع‌کردنِ سلاح‌های خودمختارِ کشنده برگزار کرده. در زمانِ نوشتنِ این کتاب، ۳۰ کشور، از نظرِ اندازه از چین تا سریر مقدس، حمایتِ خود را از پیمانِ بین‌المللی اعلام کرده‌اند، در حالی که کشورهای کلیدیِ دیگر—از جملهِ اسرائیل، روسیه، کرهٔ جنوبی و آمریکا—با ممنوعیت مخالف‌اند.

بحثِ بر سرِ سلاح‌های خودمختار شاملِ جنبه‌های حقوقی، اخلاقی و عملی است. مسائلِ حقوقی عمدتاً توسط CCW اداره می‌شوند، که امکانِ تمایز بینِ رزمندگان و غیررزمندگان، قضاوتِ ضرورتِ نظامی برای حمله، و ارزیابیِ تناسبِ بینِ ارزشِ نظامیِ هدف و امکانِ خسارتِ جانبی را ایجاب می‌کند. شدنی‌بودنِ برآوردنِ این معیارها پرسشِ مهندسی است—پرسشی که پاسخش بدونِ شک در طولِ زمان تغییر خواهد کرد. در حالِ حاضر، تمایز در برخی شرایط شدنی به نظر می‌رسد و بدونِ شک به‌سرعت بهبود خواهد یافت، اما ضرورت و تناسب در حالِ حاضر شدنی نیستند: آن‌ها ایجاب می‌کنند ماشین‌ها قضاوت‌های ذهنی و موقعیتی کنند که به‌طورِ چشمگیری دشوارتر از تکلیف‌های نسبتاً سادهٔ جستجو و حمله به اهدافِ بالقوه است. به این دلایل، استفاده از سلاح‌های خودمختار فقط در شرایطی قانونی می‌بود که اپراتورِ

انسانی بتواند به‌طورِ معقول پیش‌بینی کند اجرایِ مأموریت به هدف‌قرارگرفتنِ غیرنظامیان یا اجرایِ حملاتِ غیرضروری یا نامتناسب توسط سلاح‌ها منجر نمی‌شود. این بدان‌معناست، برای زمانِ حال، فقط مأموریت‌های بسیار محدودی می‌توانند توسط سلاح‌های خودمختار انجام شوند.

در سمتِ اخلاقی، برخی واگذاریِ تصمیمِ کشتنِ انسان‌ها به ماشین را صرفاً از نظرِ اخلاقی غیرقابلِ‌پذیرش می‌یابند. برای نمونه، سفیرِ آلمان در ژنو گفته «نمی‌پذیریم تصمیم بر سرِ زندگی و مرگ صرفاً توسط سامانهٔ خودمختار گرفته شود» در حالی که ژاپن «برنامه‌ای برای توسعهٔ ربات‌های بدونِ دخالتِ انسان، که ممکن است قادر به قتل باشند، ندارد.» ژنرالِ پل سلوآ، که در آن زمان دومین مقامِ عالی‌رتبهٔ نظامیِ آمریکا بود، در 2017 گفت «فکر نمی‌کنم معقول باشد ربات‌ها را مسئولِ اینکه آیا جانِ انسانی را می‌گیریم بگذاریم.» سرانجام، آنتونیو گوترش، رئیسِ سازمانِ ملل، در 2019 گفت «ماشین‌هایی با قدرت و اختیارِ گرفتنِ جان بدونِ دخالتِ انسانی از نظرِ سیاسی غیرقابلِ‌پذیرش، از نظرِ اخلاقی منفورند و باید توسط حقوقِ بین‌الملل ممنوع شوند.» بیش از ۱۴۰ سازمانِ مردم‌نهاد در بیش از ۶۰ کشور بخشی از «کمپینِ توقفِ ربات‌های قاتل» هستند، و نامهٔ سرگشاده‌ای که در 2015 توسط مؤسسهٔ آیندهٔ زندگی سازماندهی شد توسط بیش از ۴,۰۰۰ پژوهشگرِ هوش مصنوعی[۲] و ۲۲,۰۰۰ نفرِ دیگر امضا شد.

در برابرِ این، می‌توان استدلال کرد با بهبودِ فناوری باید بتوان سلاح‌هایی توسعه داد که کمتر از سربازان یا خلبانانِ انسانی باعثِ تلفاتِ غیرنظامی می‌شوند. (همچنین این سودِ مهم هست که سلاح‌های خودمختار نیاز سربازان و خلبانانِ انسانی به به‌خطرانداختنِ جانشان را کاهش می‌دهند.) سامانه‌های خودمختار تسلیمِ خستگی، ناامیدی، هیستری، ترس، خشم یا انتقام نمی‌شوند، و لازم نیست «اول شلیک کنند، بعد سؤال بپرسند» (آرکین، 2015). همان‌گونه که مهماتِ هدایت‌شونده خسارتِ جانبی را نسبت به بمب‌های هدایت‌نشده کاهش داده‌اند، ممکن است انتظار داشت سلاح‌های هوشمند دقتِ حملات را بیشتر بهبود دهند. (در برابرِ این، برای تحلیلی از تلفاتِ جنگِ پهپادی، بنجامین (2013) را ببینید.) این، ظاهراً، موضعِ آمریکا در آخرین دورِ مذاکرات در ژنو است.

شاید به‌صورتِ ضدشهودی، آمریکا همچنین یکی از معدود کشورهایی است که سیاست‌های فعلی‌اش استفاده از سلاح‌های خودمختار را منع می‌کند. نقشهٔ راهِ 2011 وزارت دفاعِ آمریکا (DOD) می‌گوید: «در آیندهٔ قابلِ‌پیش‌بینی، تصمیم‌ها بر سرِ استفاده از نیرو [توسط سامانه‌های خودمختار] و انتخابِ اینکه کدام اهدافِ منفرد با نیرویِ کشنده موردِ حمله قرار گیرند تحتِ کنترلِ انسانی باقی خواهند ماند.» دلیلِ اصلیِ این سیاست عملی است: سامانه‌های خودمختار به‌اندازهٔ کافی قابلِ‌اعتماد نیستند که تصمیماتِ نظامی به آن‌ها سپرده شود.

مسئلهٔ قابلیتِ‌اعتماد در 26 سپتامبرِ 1983 به‌پیش آمد، وقتی نمایشگرِ رایانهٔ افسرِ موشکیِ شوروی، استانیسلاو پتروف، هشدارِ حملهٔ موشکیِ ورودی را چشمک زد. طبقِ پروتکل، پتروف باید ضدحملهٔ هسته‌ای را آغاز می‌کرد، اما مشکوک بود هشدار باگ است و با آن چنین رفتار کرد. درست می‌گفت، و جنگِ جهانیِ سوم (به‌طورِ باریک) خنثی شد. نمی‌دانیم اگر انسانِ در حلقه نبود چه اتفاقی می‌افتاد.

قابلیتِ‌اعتماد نگرانیِ بسیار جدی برای فرماندهانِ نظامی است، که پیچیدگیِ موقعیت‌های میدانِ نبرد را خوب می‌دانند. سامانه‌های یادگیریِ ماشینی که در آموزش بی‌نقص عمل می‌کنند ممکن است در استقرار ضعیف عمل کنند. حملاتِ سایبری به سلاح‌های خودمختار می‌تواند به تلفاتِ آتشِ خودی بینجامد؛ قطعِ همهٔ ارتباطاتِ سلاح ممکن است از آن جلوگیری کند (با فرضِ اینکه از قبل به خطر نیفتاده)، اما آنگاه اگر سلاح دچارِ خرابی باشد نمی‌توان آن را فراخواند.

مسئلهٔ عملیِ غالبِ سلاح‌های خودمختار این است که آن‌ها «سلاح‌های کشتارِ جمعیِ مقیاس‌پذیر» هستند، به این معنا که مقیاسِ حمله‌ای که می‌توان پرتاب کرد متناسب با مقدارِ سخت‌افزاری است که توانِ استقرارش را دارید. ریزپهپادی چهارپره (کوادکوپتر) با قطرِ دو اینچ می‌تواند بارِ انفجاریِ کشنده حمل کند، و یک میلیون از آن‌ها در کانتینرِ حمل‌ونقلِ معمولی جا می‌شوند. دقیقاً چون خودمختارند، این سلاح‌ها برای انجامِ کارشان به یک میلیون ناظرِ انسانی نیاز نخواهند داشت.

به‌عنوانِ سلاح‌های کشتارِ جمعی، سلاح‌های خودمختارِ مقیاس‌پذیر برای مهاجم در مقایسه با سلاح‌های هسته‌ای و بمبارانِ فرشی مزیت‌هایی دارند: اموال را سالم می‌گذارند و می‌توان آن‌ها را به‌صورتِ انتخابی اعمال کرد تا فقط کسانی را حذف کنند که ممکن است نیرویِ اشغالگر را تهدید کنند. قطعاً می‌توان از آن‌ها برای نابودیِ کلِ یک گروه قومی یا همهٔ پیروانِ دینِ خاصی استفاده کرد. در بسیاری از موقعیت‌ها، آن‌ها همچنین ردیابی‌ناپذیر خواهند بود. این ویژگی‌ها آن‌ها را به‌ویژه برای بازیگرانِ غیردولتی جذاب می‌کند.

این ملاحظات—به‌ویژه آن ویژگی‌هایی که به مهاجم برتری می‌دهند—پیشنهاد می‌کنند سلاح‌های خودمختار امنیتِ جهانی و ملی را برای همهٔ طرف‌ها کاهش خواهند داد. پاسخِ عقلاییِ دولت‌ها به نظر می‌رسد شرکت در بحث‌های کنترلِ تسلیحات باشد، نه مسابقهٔ تسلیحاتی. فرایندِ طراحیِ پیمان بدونِ دشواری‌هایش نیست، با این حال. هوش مصنوعی فناوریِ «استفادهٔ دوگانه» (Dual Use) است: فناوری‌های هوش مصنوعی که کاربردهای صلح‌آمیزی مانندِ کنترلِ پرواز، ردیابیِ بصری، نگاشت، ناوبری و برنامه‌ریزیِ چندعاملی دارند، به‌راحتی می‌توانند برای مقاصدِ نظامی اعمال شوند. تبدیلِ پهپادِ چهارپرهٔ خودمختار به سلاح با اتصالِ مادهٔ منفجره و فرمانِ جستجویِ هدف به آن آسان است. برخورد با این مستلزمِ اجرایِ دقیقِ رژیم‌های انطباق با همکاریِ صنعت خواهد بود، چنان‌که قبلاً با موفقیتی توسط کنوانسیونِ سلاح‌های شیمیایی نشان داده شده.

۲۷.۳.۲ نظارت، امنیت و حریم خصوصی

در 1976، جوزف وایزنباوم هشدار داد فناوریِ بازشناسیِ خودکارِ گفتار می‌تواند به شنودِ گسترده و از این رو به ازدست‌رفتنِ آزادی‌های مدنی بینجامد. امروزه، آن تهدید محقق شده، با بیشترِ ارتباطاتِ الکترونیکی که از سرورهای مرکزیِ قابلِ‌نظارت می‌گذرند، و شهرهایی که پر از میکروفون و دوربین‌اند که می‌توانند افراد را بر مبنای صدا، چهره و راه‌رفتنشان شناسایی و ردیابی کنند. نظارتی که قبلاً به منابعِ انسانیِ گران و کمیاب نیاز داشت، اکنون می‌تواند در مقیاسی انبوه توسط ماشین‌ها انجام شود.

از 2018، تا ۳۵۰ میلیون دوربینِ نظارتی در چین و ۷۰ میلیون در آمریکا بود. چین و کشورهای دیگر شروع به صادراتِ فناوریِ نظارتی به کشورهای کم‌فناوری کرده‌اند، برخی با شهرتِ بدرفتاری با شهروندان و هدف‌قراردادنِ نامتناسبِ جوامعِ به‌حاشیه‌رانده. مهندسانِ هوش مصنوعی باید روشن باشند چه کاربردهایی از نظارت با حقوقِ بشر سازگار است، و از کار بر رویِ کاربردهای ناسازگار خودداری کنند. هرچه نهادهای بیشتری به‌صورتِ برخط فعالیت کنند، در برابرِ جرمِ سایبری (فیشینگ، کلاهبرداریِ کارتِ اعتباری، بات‌نت‌ها، باج‌افزار) و تروریسمِ سایبری (از جملهِ حملاتِ بالقوهٔ مرگبار مانندِ خاموش‌کردنِ بیمارستان‌ها و نیروگاه‌ها یا تصاحبِ خودروهای خودران) آسیب‌پذیرتر می‌شویم. یادگیریِ ماشینی می‌تواند ابزارِ قدرتمندی برای هر دو طرفِ نبردِ «امنیتِ سایبری» (Cybersecurity) باشد. مهاجمان می‌توانند از اتوماسیون برای کاوشِ ناامنی‌ها استفاده کنند و می‌توانند یادگیریِ تقویتی را برای تلاش‌های فیشینگ و اخاذیِ خودکار

اعمال کنند. مدافعان می‌توانند از یادگیریِ نظارت‌نشده برای تشخیصِ الگوهای ترافیکِ ورودیِ ناهنجار (چاندولا و همکاران، 2009؛ مالهوترا و همکاران، 2015) و از فنونِ یادگیریِ ماشینیِ گوناگون برای تشخیصِ کلاهبرداری استفاده کنند (فاست و پرووست، 1997؛ بولتون و هند، 2002). هرچه حملات پیچیده‌تر می‌شوند، مسئولیتِ بیشتری برای همهٔ مهندسان، نه فقط کارشناسانِ امنیت، هست که از آغاز سامانه‌های امن طراحی کنند.

یک پیش‌بینی (کانال، 2017) بازارِ یادگیریِ ماشینی در امنیتِ سایبری را تا 2021 حدودِ ۱۰۰ میلیارد دلار می‌گذارد. هرچه با رایانه‌ها برای بخشِ فزاینده‌ای از زندگیِ روزمره‌مان تعامل می‌کنیم، دادهٔ بیشتری دربارهٔ ما توسط دولت‌ها و شرکت‌ها جمع‌آوری می‌شود. جمع‌آوری‌کنندگانِ داده مسئولیتِ اخلاقی و حقوقی دارند مباشرانِ خوبی برای داده‌ای که در دست دارند باشند. در آمریکا، «قانونِ قابلِ‌انتقال‌بودن و پاسخگوییِ بیمهٔ سلامت» (HIPAA) و «قانونِ حقوقِ حریمِ خصوصی و خانوادگیِ آموزشی» (FERPA) از حریمِ خصوصیِ سوابقِ پزشکی و دانشجویی محافظت می‌کنند. «مقرراتِ عمومیِ حفاظت از داده» (GDPR) اتحادیهٔ اروپا شرکت‌ها را ملزم می‌کند سامانه‌هایشان را با درنظرگرفتنِ حفاظت از داده طراحی کنند و ایجاب می‌کند برای هر جمع‌آوری یا پردازشِ داده رضایتِ کاربر بگیرند.

در برابرِ حقِّ حریمِ خصوصیِ فرد، ارزشی است که جامعه از اشتراک‌گذاریِ داده به دست می‌آورد. می‌خواهیم بتوانیم تروریست‌ها را بدونِ سرکوبِ مخالفتِ صلح‌آمیز متوقف کنیم، و می‌خواهیم بیماری‌ها را بدونِ به‌خطرانداختنِ حقِّ هر فرد برای خصوصی‌نگهداشتنِ سوابقِ سلامت‌اش درمان کنیم. یکی از رویه‌های کلیدی «هویت‌روبی» (De-Identification) است: حذفِ اطلاعاتِ شخصاً قابلِ شناسایی (مانندِ نام و شمارهٔ تأمینِ اجتماعی) تا پژوهشگرانِ پزشکی بتوانند از داده برای پیشبردِ خیرِ عمومی استفاده کنند. مشکل این است که دادهٔ اشتراک‌گذاری‌شدهٔ هویت‌روبی‌شده ممکن است در معرضِ بازشناسایی باشد. برای نمونه، اگر داده نام، شمارهٔ تأمینِ اجتماعی و آدرسِ خیابان را حذف کند، اما تاریخِ تولد، جنسیت و کدِ پستی را شامل شود، آنگاه، چنان‌که لاتانیا سوینی (2000) نشان داد، ۸۷٪ از جمعیتِ آمریکا را می‌توان به‌صورتِ یکتا بازشناسایی کرد. سوینی این نکته را با بازشناساییِ سوابقِ بهداشتیِ فرماندارِ ایالتش وقتی در بیمارستان بستری شد برجسته کرد.

در رقابتِ «جایزهٔ نتفلیکس» (Netflix Prize)، سوابقِ هویت‌روبی‌شدهٔ رتبه‌بندیِ فیلمِ افرادِ منفرد منتشر شد، و از رقبا خواسته شد الگوریتمِ یادگیریِ ماشینی‌ای ارائه دهند که می‌توانست دقیقاً پیش‌بینی کند فردی چه فیلم‌هایی را دوست خواهد داشت. اما پژوهشگران توانستند کاربرانِ منفرد را با تطبیقِ تاریخِ رتبه‌بندیِ پایگاهِ دادهٔ نتفلیکس با تاریخِ رتبه‌بندیِ مشابهی در پایگاهِ دادهٔ فیلمِ اینترنتی (IMDB)، که کاربران در آن گاهی از نامِ واقعی‌شان استفاده می‌کنند، بازشناسایی کنند (نارایانان و شماتیکوف، 2006).

این خطر را می‌توان تا حدی با تعمیم‌دادنِ فیلدها کاهش داد: برای نمونه، جایگزینیِ تاریخِ تولدِ دقیق با فقط سالِ تولد، یا دامنهٔ گسترده‌تری مانندِ «۲۰-۳۰ ساله». حذفِ کاملِ فیلدی را می‌توان شکلی از تعمیم به «هر» دید. اما تعمیم به‌تنهایی تضمین نمی‌کند سوابق از بازشناسایی در امان‌اند؛ ممکن است فقط یک نفر در کدِ پستیِ 94720 باشد که ۹۰–۱۰۰ ساله است. ویژگیِ مفیدی «k-گمنامی» (K-Anonymity) است: پایگاهِ داده‌ای k-گمنام است اگر هر رکوردِ پایگاهِ داده از دستِ‌کم k-1 رکوردِ دیگر تمایزناپذیر (غیرقابلِ‌تشخیص) باشد. اگر رکوردی یکتاتر از این باشد، باید بیشتر تعمیم داده شود.

جایگزینی برای اشتراک‌گذاریِ سوابقِ هویت‌روبی‌شده، خصوصی‌نگهداشتنِ همهٔ سوابق اما اجازه‌دادن به «پرس‌وجویِ تجمیعی» (Aggregate Querying) است. APIای برای پرس‌وجوهای پایگاهِ داده فراهم می‌شود، و پرس‌وجوهای معتبر پاسخی دریافت می‌کنند که داده را با شمارش یا میانگین خلاصه می‌کند. اما اگر پاسخ ضمانت‌های خاصی از حریمِ خصوصی را نقض کند، پاسخی داده نمی‌شود. برای نمونه، می‌توانستیم به همه‌گیرشناس اجازه دهیم برای هر کدِ پستی درصدِ افرادِ مبتلا به سرطان را بپرسد. برای کدهای پستی با دستِ‌کم n نفر درصدی داده می‌شد (با مقدارِ کمی نویزِ تصادفی)، اما برای کدهای پستی با کمتر از n نفر پاسخی داده نمی‌شد.

باید مراقب محافظت در برابرِ بازشناسایی (رفعِ گمنامی) با پرس‌وجوهای متعدد بود. برای نمونه، اگر پرس‌وجوی «میانگینِ حقوق و تعدادِ کارمندانِ شرکتِ XYZ با سن ۳۰-۴۰» پاسخِ [$81,234, 12] را بدهد و پرس‌وجوی «میانگینِ حقوق و تعدادِ کارمندانِ شرکتِ XYZ با سن ۳۰-۴۱» پاسخِ [$81,199, 13] را بدهد، و اگر از لینکدین برای یافتنِ تنها ۴۱سالهٔ شرکتِ XYZ استفاده کنیم، پس با موفقیت آن‌ها را شناسایی کرده‌ایم و می‌توانیم حقوقِ دقیقشان را محاسبه کنیم، حتی اگر همهٔ پاسخ‌ها شاملِ ۱۲ نفر یا بیشتر بودند. سامانه باید با ترکیبی از محدودیت‌های پرس‌وجوهای قابلِ‌پرسش (شاید فقط مجموعهٔ ازپیش‌تعریف‌شده‌ای از بازه‌های سنیِ غیرهم‌پوشان قابلِ‌پرسش باشد) و دقتِ نتایج (شاید هر دو پرس‌وجو پاسخِ «حدودِ ۸۱,۰۰۰ دلار» بدهند) به‌دقت طراحی شود.

ضمانتِ قوی‌تری «حریمِ خصوصیِ تفاضلی» (Differential Privacy) است، که تضمین می‌کند مهاجم نمی‌تواند با پرس‌وجوها هیچ فردی را در پایگاهِ داده بازشناسایی کند، حتی اگر مهاجم بتواند پرس‌وجوهای متعدد انجام دهد و به پایگاه‌های دادهٔ پیوندیِ جداگانه دسترسی داشته باشد. پاسخِ پرس‌وجو از الگوریتمِ تصادفی‌سازی استفاده می‌کند که مقدارِ کمی نویز به نتیجه می‌افزاید. با توجه به پایگاهِ دادهٔ D، هر رکوردِ r که در پایگاهِ داده نیست، هر پرس‌وجوی Q و هر پاسخِ ممکنِ y به پرس‌وجو، می‌گوییم پایگاهِ دادهٔ D دارای «حریمِ خصوصیِ تفاضلیِ ε» است اگر لگاریتمِ احتمالِ پاسخِ y کمتر از ε تغییر کند وقتی رکوردِ r را می‌افزاییم:

$$|\log P(Q(D) = y) - \log P(Q(D + r) = y)| \leq \varepsilon$$

به بیانِ دیگر، اینکه آیا هر شخصی تصمیم می‌گیرد در پایگاهِ داده شرکت کند یا نه، تفاوتِ محسوسی در پاسخ‌هایی که هرکس می‌تواند بگیرد ایجاد نمی‌کند، و بنابراین هیچ انگیزه‌ای برای حمله به حریمِ خصوصی از این طریق نیست. با این حال، باید مراقب بود پاسخ‌ها به پرس‌وجوهای ترکیبیِ متعدد نیز همین ویژگی را حفظ کنند.

۲۷.۳.۳ انصاف و اریب

یادگیریِ ماشینی تصمیم‌گیریِ انسانی را در موقعیت‌های مهم تقویت و گاهی جایگزین می‌کند: وامِ چه کسی تأیید شود، پلیس به کدام محله‌ها اعزام شود، چه کسی آزادیِ پیش از محاکمه یا آزادیِ مشروط بگیرد. اما مدل‌های یادگیریِ ماشینی می‌توانند «اریبِ اجتماعی» (Societal Bias) را تداوم بخشند. مثالِ الگوریتمی برای پیش‌بینیِ اینکه آیا متهمانِ جنایی احتمالِ تکرارِ جرم دارند، و بنابراین آیا باید پیش از محاکمه آزاد شوند را در نظر بگیر. کاملاً ممکن است چنین سامانه‌ای تعصب‌های نژادی یا جنسیتیِ قضاتِ انسانی را از مثال‌های مجموعهٔ آموزشی بردارد. طراحانِ سامانه‌های

یادگیریِ ماشینی مسئولیتِ اخلاقی دارند مطمئن شوند سامانه‌هایشان در واقع منصفانه‌اند. در حوزه‌های تنظیم‌شده مانندِ اعتبار، آموزش، اشتغال و مسکن، مسئولیتِ حقوقی نیز دارند. اما انصاف چیست؟ معیارهای متعددی هست؛ در این‌جا شش مورد از پرکاربردترینِ مفاهیم آمده:

- **انصافِ فردی (Individual Fairness):** الزامی مبنی بر اینکه با افرادِ مشابه، مشابهِ افرادِ مشابهِ دیگر رفتار شود، صرف‌نظر از اینکه در چه گروهی‌اند.
- **انصافِ گروهی (Group Fairness):** الزامی مبنی بر اینکه با دو گروه به‌صورتِ مشابه رفتار شود، چنان‌که با برخی آماره‌های خلاصه اندازه‌گیری می‌شود.
- **انصاف از طریقِ ناآگاهی (Fairness Through Unawareness):** اگر ویژگی‌های نژاد و جنسیت را از مجموعهٔ داده حذف کنیم، آنگاه ممکن است به نظر برسد سامانه نمی‌تواند بر مبنای آن ویژگی‌ها تبعیض قائل شود. متأسفانه، می‌دانیم مدل‌های یادگیریِ ماشینی می‌توانند متغیرهای نهان (مانندِ نژاد و جنسیت) را با توجه به متغیرهای همبستهٔ دیگر (مانندِ کدِ پستی و شغل) پیش‌بینی کنند. افزون بر این، حذفِ آن ویژگی‌ها راستی‌آزماییِ فرصتِ برابر یا پیامدهای برابر را ناممکن می‌کند. با این حال، برخی کشورها (مثلاً، آلمان) این رویکرد را برای آمارِ جمعیت‌شناختی‌شان انتخاب کرده‌اند (خواه مدل‌های یادگیریِ ماشینی درگیر باشند خواه نه).
- **برابریِ پیامد (Equal Outcome):** ایدهٔ اینکه هر ردهٔ جمعیت‌شناختی همان نتایج را می‌گیرد؛ آن‌ها «برابریِ جمعیت‌شناختی» (Demographic Parity) دارند. برای نمونه، فرض کنیم باید تصمیم بگیریم آیا درخواست‌های وام را تأیید کنیم یا نه؛ هدف تأییدِ آن متقاضیانی است که وام را پس می‌دهند و نه آن‌هایی که وام را نکول می‌کنند. برابریِ جمعیت‌شناختی می‌گوید هم مردان و هم زنان باید همان درصدِ وام‌های تأییدشده را داشته باشند. توجه کنید این معیارِ انصافِ گروهی است که هیچ کاری برای تضمینِ انصافِ فردی نمی‌کند؛ متقاضیِ واجدِ شرایط ممکن است رد شود و متقاضیِ کم‌صلاحیت ممکن است تأیید شود، تا زمانی که درصدهای کلی برابرند. همچنین، این رویکرد جبرانِ اریب‌های گذشته را بر دقتِ پیش‌بینی ترجیح می‌دهد. اگر مرد و زنی از هر نظر برابر باشند، به‌جز اینکه زن برای همان شغل حقوقِ کمتری دریافت می‌کند، آیا باید تأیید شود چون اگر اریب‌های تاریخی نبودند برابر می‌بود، یا باید رد شود چون حقوقِ کمتر در واقع نکول را برایش محتمل‌تر می‌کند؟
- **فرصتِ برابر (Equal Opportunity):** ایدهٔ اینکه افرادی که واقعاً توانایی پس‌دادنِ وام را دارند باید شانسِ برابری برای رده‌بندیِ درستِ چنین‌بودن داشته باشند، صرف‌نظر از جنسیت‌شان. این رویکرد «تراز» نیز نامیده می‌شود. می‌تواند به پیامدهای نابرابر بینجامد و اثرِ اریب در فرایندهای اجتماعیِ تولیدکنندهٔ دادهٔ آموزشی را نادیده می‌گیرد.
- **اثرِ برابر (Equal Impact):** افرادِ با احتمالِ مشابهِ پس‌دادنِ وام باید همان مطلوبیتِ موردانتظار را داشته باشند، صرف‌نظر از رده‌ای که به آن تعلق دارند. این فراتر از فرصتِ برابر است، زیرا هم مزیت‌های پیش‌بینیِ درست و هم هزینه‌های پیش‌بینیِ نادرست را در نظر می‌گیرد.

بیایید بررسی کنیم این مسائل در بافتِ خاصی چگونه ایفای نقش می‌کنند. COMPAS سامانهٔ تجاریِ امتیازدهیِ تکرارِ جرم است. به متهمِ پروندهٔ جنایی امتیازِ خطری تخصیص می‌دهد، که سپس توسط قاضی برای کمک به تصمیم‌گیری‌ها استفاده می‌شود: آیا آزادکردنِ متهم پیش از محاکمه ایمن است، یا باید در زندان بماند؟ اگر محکوم شود، حکم چقدر طولانی باشد؟ آیا آزادیِ مشروط داده شود؟ با توجه به اهمیتِ این تصمیم‌ها، سامانه موضوعِ بررسیِ شدید بوده (درسل و فرید، 2018).

COMPAS طوری طراحی شده که «خوب کالیبره‌شده» (Well Calibrated) باشد: همهٔ افرادی که الگوریتم به آن‌ها امتیازِ یکسانی می‌دهد باید تقریباً همان احتمالِ تکرارِ جرم را داشته باشند، صرف‌نظر از نژاد. برای نمونه، در میانِ همهٔ افرادی که مدل به آن‌ها امتیازِ خطرِ ۷ از ۱۰ تخصیص می‌دهد، ۶۰٪ از سفیدپوستان و ۶۱٪ از سیاه‌پوستان تکرارِ جرم می‌کنند. طراحان بنابراین ادعا می‌کنند هدفِ انصافِ مطلوب را برآورده می‌کند. از سوی دیگر، COMPAS به فرصتِ برابر نمی‌رسد: نسبتِ کسانی که تکرارِ جرم نکردند اما به‌صورتِ نادرست پرخطر رده‌بندی شدند، ۴۵٪ برای سیاه‌پوستان و ۲۳٪ برای سفیدپوستان بود. در پروندهٔ State v. Loomis، جایی که قاضی برای تعیینِ حکمِ متهم به COMPAS تکیه کرد، لومیس استدلال کرد کارکردهای درونیِ محرمانهٔ الگوریتم حقِّ دادرسیِ عادلانه و قانونیِ او را نقض می‌کند. هرچند دادگاهِ عالیِ ویسکانسین دریافت در این پرونده حکمِ داده‌شده بدونِ COMPAS تفاوتی نمی‌کرد، دربارهٔ دقتِ الگوریتم و خطرات برای متهمانِ اقلیت هشدار صادر کرد. پژوهشگرانِ دیگر پرسیده‌اند آیا استفاده از الگوریتم‌ها در کاربردهایی مانندِ صدورِ حکم مناسب است.

می‌توانستیم امیدوار باشیم الگوریتمی هم خوب کالیبره‌شده و هم فرصتِ برابر باشد، اما، چنان‌که کلینبرگ و همکاران (2016) نشان می‌دهند، آن ناممکن است. اگر رده‌های پایه متفاوت‌اند، هر الگوریتمی که خوب کالیبره‌شده باشد لزوماً فرصتِ برابر فراهم نمی‌کند، و برعکس. چگونه می‌توانیم این دو را وزن کنیم؟ کارهای اخیر پیشنهاد کرده‌اند با هدفِ کالیبراسیون و هدفِ فرصتِ برابر به‌صورتِ اهدافی جداگانه و مکمل، اما با اولویت‌بندیِ انصافِ فردی، برخورد کنیم (آن‌ها را در نظر بگیریم). (کوشیو-هیگوچی و راسل، 2018؛ نارانگ و همکاران، 2019). بیایید به جنبه‌های عملیِ ساختِ سامانه‌های منصفانه بپردازیم.

نخست، محدودیت‌های داده‌ای که استفاده می‌کنید را بفهمید. پیشنهاد شده مجموعه‌های داده (گبرو و همکاران، 2018؛ هیند و همکاران، 2018) و مدل‌ها (میچل و همکاران، 2019) باید با حاشیه‌نویسی‌هایی بیایند: اعلامیه‌هایی از خاستگاه، امنیت، انطباق و شایستگی برای استفاده. این شبیهِ «برگهٔ داده»هایی (Data Sheet) است که همراهِ اجزای الکترونیکی مانندِ مقاومت‌ها می‌آید؛ آن‌ها به طراحان اجازه می‌دهند تصمیم بگیرند از چه اجزایی استفاده کنند. علاوه بر برگه‌های داده، مهم است مهندسان را، هم در مدرسه و هم با آموزشِ حینِ کار، نسبت به مسائلِ انصاف و اریب آگاه کنیم. داشتنِ تنوعی از مهندسان با پیشینه‌های متفاوت برایشان آسان‌تر می‌کند مشکلاتِ داده یا مدل‌ها را متوجه شوند. مطالعه‌ای توسط مؤسسهٔ AI Now (وست و همکاران، 2019) دریافت فقط ۱۸٪ از نویسندگانِ کنفرانس‌های برترِ هوش مصنوعی و ۲۰٪ از استادانِ هوش مصنوعی زن‌اند. کارگرانِ سیاه‌پوستِ هوش مصنوعی کمتر از ۴٪اند. نرخ‌ها در آزمایشگاه‌های پژوهشیِ صنعتی مشابه‌اند. تنوع را می‌توان با برنامه‌هایی زودتر در خطِّ لوله—در کالج یا دبیرستان—و با آگاهیِ بیشتر در سطحِ حرفه‌ای افزایش داد. جوی بولامووینی «اتحادیهٔ عدالتِ الگوریتمی» را برای افزایشِ آگاهی از این مسئله و توسعهٔ رویه‌های پاسخگویی تأسیس کرد.

ایدهٔ دوم اریب‌زدایی از داده است (زمل و همکاران، 2013). می‌توانیم از رده‌های اقلیت بیش‌نمونه‌برداری کنیم تا در برابرِ نابرابریِ اندازهٔ نمونه دفاع کنیم. فنونی مانندِ SMOTE، فنِ بیش‌نمونه‌برداریِ مصنوعیِ اقلیت (چاولا و همکاران، 2002)، یا ADASYN، رویکردِ نمونه‌برداریِ مصنوعیِ تطبیقی برای یادگیریِ نامتوازن (هه و همکاران، 2008)، راه‌های اصولیِ بیش‌نمونه‌برداری فراهم می‌کنند. می‌توانیم خاستگاهِ داده را بررسی کنیم و، برای نمونه، مثال‌هایی از قضاتی که در پرونده‌های دادگاهِ گذشته‌شان اریب نشان داده‌اند حذف کنیم. برخی تحلیلگران به ایدهٔ دورانداختنِ داده اعتراض دارند و در عوض توصیه می‌کنند مدلِ سلسله‌مراتبیِ داده بسازیم که منابعِ اریب را شامل می‌شود، تا بتوان آن‌ها را مدل و

جبران کرد. گوگل و NeurIPS با حمایتِ مالیِ «رقابتِ تصاویرِ فراگیر» تلاش کرده‌اند آگاهی از این مسئله را افزایش دهند، که در آن رقبا شبکه‌ای را رویِ مجموعهٔ داده‌ای از تصاویرِ برچسب‌خوردهٔ جمع‌آوری‌شده در آمریکای شمالی و اروپا آموزش می‌دهند و سپس آن را رویِ تصاویرِ گرفته‌شده از سراسرِ جهان آزمایش می‌کنند. مسئله این است که با این مجموعهٔ داده، اعمالِ برچسبِ «عروس» به زنی با لباسِ عروسیِ غربیِ استاندارد آسان است، اما بازشناسیِ لباسِ عروسیِ سنتیِ آفریقایی و هندی سخت‌تر است.

ایدهٔ سوم ابداعِ مدل‌ها و الگوریتم‌های یادگیریِ ماشینیِ تازه‌ای است که در برابرِ اریب مقاوم‌ترند؛ و ایدهٔ نهایی این است که به سامانه اجازه دهیم توصیه‌های اولیه‌ای بدهد که ممکن است اریب‌دار باشند، اما سپس سامانهٔ دومی آموزش دهیم که توصیه‌های سامانهٔ نخست را اریب‌زدایی کند. بلامی و همکاران (2018) سامانهٔ IBM AI FAIRNESS 360 را معرفی کردند، که چارچوبی برای همهٔ این ایده‌ها فراهم می‌کند. انتظار داریم در آینده استفاده از ابزارهایی مانندِ این افزایش یابد.

چگونه مطمئن می‌شوید سامانه‌هایی که می‌سازید منصفانه‌اند؟ مجموعه‌ای از بهترین رویه‌ها در حالِ ظهور است (هرچند همیشه دنبال نمی‌شوند):

- مطمئن شوید مهندسانِ نرم‌افزار با دانشمندانِ اجتماعی و کارشناسانِ حوزه صحبت می‌کنند تا مسائل و دیدگاه‌ها را بفهمند، و انصاف را از آغاز در نظر بگیرند.
- محیطی ایجاد کنید که پرورشِ استخرِ متنوعی از مهندسانِ نرم‌افزارِ نمایندهٔ جامعه را تقویت کند.
- تعریف کنید سامانه‌تان از چه گروه‌هایی پشتیبانی خواهد کرد: گویندگانِ زبان‌های متفاوت، گروه‌های سنیِ متفاوت، توانایی‌های متفاوت در بینایی و شنوایی و...
- برای تابعِ هدفی بهینه کنید که انصاف را دربرگیرد.
- داده‌تان را برای پیش‌داوری و برای همبستگی‌های بینِ ویژگی‌های محافظت‌شده و ویژگی‌های دیگر بررسی کنید.
- بفهمید هر حاشیه‌نویسیِ انسانیِ داده چگونه انجام می‌شود، هدف‌هایی برای دقتِ حاشیه‌نویسی طراحی کنید، و راستی‌آزمایی کنید هدف‌ها برآورده می‌شوند.
- فقط معیارهای کلیِ سامانه‌تان را ردیابی نکنید؛ مطمئن شوید معیارهای زیرگروه‌هایی را که ممکن است قربانیِ اریب شوند ردیابی می‌کنید.
- آزمون‌های سامانه‌ای را شامل کنید که تجربهٔ کاربرانِ گروهِ اقلیت را بازتاب می‌دهند.
- حلقهٔ بازخوردی داشته باشید تا وقتی مشکلاتِ انصاف پیش می‌آیند، با آن‌ها برخورد شود.

۲۷.۳.۴ اعتماد و شفافیت

یک چالش این است سامانهٔ هوش مصنوعی را دقیق، منصفانه، ایمن و امن کنیم؛ چالشِ متفاوت قانع‌کردنِ بقیه است که چنین کرده‌اید. مردم باید بتوانند به سامانه‌هایی که استفاده می‌کنند «اعتماد» (Trust) کنند. نظرسنجیِ PwC در 2017 دریافت ۷۶٪ از کسب‌وکارها پذیرشِ هوش مصنوعی را به‌دلیلِ نگرانی‌های قابلیتِ‌اعتماد کند می‌کنند. در بخش ۱۹.۹.۴ برخی از رویکردهای مهندسیِ اعتماد را پوشش دادیم؛ در این‌جا مسائلِ سیاستی را بحث می‌کنیم.

برای جلبِ اعتماد، هر سامانهٔ مهندسی‌شده باید از فرایندِ «راستی‌آزمایی و اعتبارسنجی» (Verification and Validation یا V&V) بگذرد. راستی‌آزمایی به این معناست محصول مشخصات را برآورده می‌کند. اعتبارسنجی به معنای اطمینان‌یافتن از اینکه مشخصات واقعاً نیازهای کاربر و دیگر طرف‌های متأثر را برآورده می‌کند. روش‌شناسیِ مفصلیِ V&V برای مهندسی به‌طورِ کلی داریم، و برای توسعهٔ نرم‌افزارِ سنتیِ انجام‌شده توسط برنامه‌نویسانِ انسانی؛ بخشِ زیادی از آن برای سامانه‌های هوش مصنوعی کاربرد دارد. اما سامانه‌های یادگیریِ ماشینی متفاوت‌اند و به فرایندِ V&V متفاوتی نیاز دارند، که هنوز کاملاً توسعه نیافته. باید داده‌ای را که این سامانه‌ها از آن یاد می‌گیرند راستی‌آزمایی کنیم؛ باید دقت و انصافِ نتایج را حتی در مواجهه با عدم‌قطعیتی که نتیجهٔ دقیق را ناشناختنی می‌کند راستی‌آزمایی کنیم؛ و باید راستی‌آزمایی کنیم دشمنان نمی‌توانند بی‌دلیل بر مدل اثر بگذارند، یا با پرس‌وجو از مدلِ حاصل اطلاعات بدزدند.

یکی از ابزارهای اعتماد «گواهی‌دادن» (Certification) است؛ برای نمونه، آزمایشگاه‌های بیمه‌گران (Underwriters Laboratories یا UL) در 1894 در زمانی تأسیس شد که مصرف‌کنندگان از خطراتِ برق هراسان بودند. گواهیِ UL لوازمِ خانگی به مصرف‌کنندگان اعتمادِ فزاینده‌ای داد، و در واقع UL اکنون ورود به کسب‌وکارِ آزمایشِ محصول و گواهی‌دادن برای هوش مصنوعی را بررسی می‌کند.

صنایعِ دیگر مدتِ طولانی است استانداردهای ایمنی دارند. برای نمونه، ISO 26262 استانداردِ بین‌المللیِ ایمنیِ خودروهاست، که توصیف می‌کند چگونه وسایلِ نقلیه را به‌صورتِ ایمن توسعه، تولید، بهره‌برداری و سرویس کنیم. صنعتِ هوش مصنوعی هنوز به این سطحِ وضوح نرسیده، هرچند برخی چارچوب‌ها در حالِ پیشرفت‌اند، مانندِ IEEE P7001، استانداردی که طراحیِ اخلاقی برای سامانه‌های هوش مصنوعی و خودمختار را تعریف می‌کند (برایسون و وینفیلد، 2017). بحثِ ادامه‌داری دربارهٔ اینکه چه نوعِ گواهی‌ای لازم است، و تا چه حد باید توسط دولت، توسط سازمان‌های حرفه‌ای مانندِ IEEE، توسط گواهی‌دهندگانِ مستقل مانندِ UL، یا از طریقِ خودتنظیمی توسط شرکت‌های محصول انجام شود، هست.

جنبهٔ دیگرِ اعتماد «شفافیت» (Transparency) است: مصرف‌کنندگان می‌خواهند بدانند داخلِ سامانه چه می‌گذرد، و سامانه علیه‌شان کار نمی‌کند، چه به‌دلیلِ بدخواهیِ عمدی، باگِ ناخواسته، یا اریبِ اجتماعیِ فراگیری که توسط سامانه بازتولید می‌شود. در برخی موارد این شفافیت مستقیماً به مصرف‌کننده تحویل داده می‌شود. در مواردِ دیگر مسائلِ مالکیتِ معنوی هست که برخی جنبه‌های سامانه را از مصرف‌کنندگان پنهان نگه می‌دارد، اما برای تنظیم‌گرها و آژانس‌های گواهی باز است.

وقتی سامانهٔ هوش مصنوعی برای وام ردتان می‌کند، سزاوارِ توضیح‌اید. در اروپا، GDPR این را برایتان اعمال می‌کند. سامانهٔ هوش مصنوعی که می‌تواند خودش را تبیین کند «هوش مصنوعیِ تبیین‌پذیر» (Explainable AI یا XAI) نامیده می‌شود. تبیینِ خوب چند ویژگی دارد: باید برای کاربر قابلِ‌فهم و قانع‌کننده باشد، باید استدلالِ سامانه را دقیقاً بازتاب دهد، باید کامل باشد، و باید خاص باشد، به‌طوری که کاربرانِ متفاوت با شرایطِ متفاوت یا پیامدهای متفاوت تبیین‌های متفاوتی بگیرند.

دادنِ دسترسیِ الگوریتمِ تصمیم به فرایندهای تعمقیِ خودش بسیار آسان است، صرفاً با ضبطِ آن‌ها و در دسترس‌قراردادنِشان به‌عنوانِ ساختارهای داده. این بدان‌معناست ماشین‌ها ممکن است در نهایت بتوانند تبیین‌های بهتری از تصمیم‌هایشان از انسان‌ها بدهند. افزون بر این، می‌توانیم گام‌هایی برداریم تا گواهی دهیم تبیین‌های

ماشین فریب نیستند (عمدی یا خودفریبی)، چیزی که با انسان دشوارتر است.

تبیین مؤلفهٔ کمکی اما نه کافیِ اعتماد است. یک مسئله این است که تبیین‌ها تصمیم نیستند: آن‌ها داستان‌هایی دربارهٔ تصمیم‌اند. چنان‌که در بخش ۱۹.۹.۴ بحث شد، می‌گوییم سامانه‌ای «تفسیرپذیر» است اگر بتوانیم کدِ منبعِ مدل را بازرسی و ببینیم چه می‌کند، و می‌گوییم «تبیین‌پذیر» است اگر بتوانیم داستانی دربارهٔ آنچه می‌کند بسازیم—حتی اگر خودِ سامانه جعبهٔ سیاهِ تفسیرناپذیری باشد. برای تبیینِ جعبهٔ سیاهِ تفسیرناپذیر، باید سامانهٔ تبیینِ جداگانه‌ای بسازیم، اشکال‌زدایی و آزمایش کنیم، و مطمئن شویم با سامانهٔ اصلی همگام است. و چون انسان‌ها داستانِ خوب را دوست دارند، همگی بیش‌ازحد حاضریم تحتِ تأثیرِ تبیینی قرار بگیریم که خوب به نظر می‌رسد. هر جنجالِ سیاسیِ روز را بگیرید، و همیشه دو «کارشناس» به‌اصطلاح با تبیین‌های کاملاً متضاد خواهید یافت که هر دو به‌صورتِ درونی سازگارند.

مسئلهٔ نهایی این است که تبیینِ یک مورد به شما خلاصه‌ای از مواردِ دیگر نمی‌دهد. اگر بانک توضیح دهد، «متأسفیم، وام را نگرفتی، زیرا سابقهٔ مشکلاتِ مالیِ قبلی داری»، نمی‌دانید آن تبیین دقیق است یا بانک مخفیانه به دلیلی علیه شما اریب دارد. در این صورت، نه فقط به تبیین، بلکه به «حسابرسیِ» تصمیم‌های گذشته، با آمارِ تجمیعیِ گروه‌های جمعیت‌شناختیِ گوناگون، نیاز دارید تا ببینید نرخ‌های تأییدشان متوازن است.

بخشی از شفافیت دانستنِ این است که با سامانهٔ هوش مصنوعی در تعامل‌اید یا انسان. توبی والش (2015) پیشنهاد کرد «سامانهٔ خودمختار باید طوری طراحی شود که بعید باشد با چیزی جز سامانهٔ خودمختار اشتباه گرفته شود، و باید در آغازِ هر تعاملی خودش را معرفی کند.» او این را «قانونِ پرچمِ قرمز» نامید، به افتخارِ «قانونِ لوکوموتیو»ی 1865 بریتانیا، که ایجاب می‌کرد هر وسیلهٔ نقلیهٔ موتوری شخصی با پرچمِ قرمز جلویش راه برود تا خطرِ پیشِ رو را علامت دهد. در 2019، کالیفرنیا قانونی وضع کرد که می‌گفت «استفاده از ربات برای ارتباط یا تعامل با شخصِ دیگری در کالیفرنیا به‌صورتِ برخط، با قصدِ گمراه‌کردنِ آن شخص دربارهٔ هویتِ مصنوعی‌اش، برای هر شخصی غیرقانونی خواهد بود.»

۲۷.۳.۵ آیندهٔ کار

از نخستین انقلابِ کشاورزی (۱۰،۰۰۰ پیش از میلاد) تا انقلابِ صنعتی (اواخرِ قرنِ هجدهم) تا انقلابِ سبز در تولیدِ غذا (دههٔ ۱۹۵۰)، فناوری‌های تازه شیوهٔ کار و زندگیِ بشریت را تغییر داده‌اند. نگرانیِ اصلیِ ناشی از پیشرفتِ هوش مصنوعی این است که کارِ انسانی منسوخ شود. ارسطو، در کتابِ اولِ «سیاست»ِ خود، نکتهٔ اصلی را کاملاً روشن ارائه می‌کند:

زیرا اگر هر ابزاری می‌توانست کارِ خودش را انجام دهد، از ارادهٔ دیگران اطاعت یا آن را پیش‌بینی می‌کرد... اگر، به‌همین‌سان، ماکوی بافندگی بدونِ دستی که هدایتش کند می‌بافت و مضرابِ چنگ بدونِ دست آن را لمس می‌کرد، استادکارانِ اصلی به خدمتکاران نیاز نمی‌داشتند، و نه اربابان به برده‌ها.

همه با مشاهدهٔ ارسطو موافق‌اند که وقتی کارفرما روشِ مکانیکی‌ای برای انجامِ کاری که قبلاً توسط شخصی انجام می‌شد پیدا می‌کند، کاهشِ بی‌واسطهٔ اشتغال رخ می‌دهد. مسئله این است که آیا «اثرهای جبرانی» به‌اصطلاحِ پس‌آینده—که تمایل به افزایشِ اشتغال دارند—در نهایت این کاهش را جبران خواهند کرد یا نه. اثرِ جبرانیِ اصلی

افزایشِ ثروتِ کلی از بهره‌وریِ بیشتر است، که به‌نوبهٔ خود به تقاضای بیشتر برای کالاها و تمایل به افزایشِ اشتغال می‌انجامد. برای نمونه، PwC (رائو و وروی، 2017) پیش‌بینی می‌کند هوش مصنوعی تا 2030 سالانه ۱۵ تریلیون دلار به تولیدِ ناخالصِ جهانی بیفزاید. صنایعِ بهداشت و خودرو/حمل‌ونقل در کوتاه‌مدت بیشترین سود را خواهند برد.

با این حال، مزیت‌های اتوماسیون هنوز در اقتصادِ ما غالب نشده: نرخِ کنونیِ رشدِ بهره‌وریِ کار در واقع پایین‌تر از استانداردهای تاریخی است. برین‌یولفسون و همکاران (2018) می‌کوشند این پارادوکس را با پیشنهادِ اینکه فاصلهٔ بینِ توسعهٔ فناوریِ پایه و پیاده‌سازیِ آن در اقتصاد طولانی‌تر از آن چیزی است که معمولاً فرض می‌شود تبیین کنند.

نوآوری‌های فناورانه از نظرِ تاریخی برخی مردم را بیکار کرده‌اند. بافندگان در دههٔ ۱۸۱۰ توسط ماشین‌های بافندگیِ خودکار جایگزین شدند، که به اعتراضاتِ لودیت انجامید. لودیت‌ها مخالفِ خودِ فناوری نبودند؛ فقط می‌خواستند ماشین‌ها توسط کارگرانِ ماهری که دستمزدِ خوبی می‌گیرند برای ساختِ کالاهای باکیفیت استفاده شوند، نه توسط کارگرانِ بی‌مهارت برای ساختِ کالاهای بی‌کیفیت با دستمزدِ کم. نابودیِ جهانیِ مشاغل در دههٔ ۱۹۳۰ جان مینارد کینز را به ابداعِ اصطلاحِ «بیکاریِ فناورانه» (Technological Unemployment) کشاند. در هر دو مورد، و چند موردِ دیگر، سطحِ اشتغال در نهایت بهبود یافت.

دیدگاهِ اقتصادیِ جریانِ اصلی برای بیشترِ قرنِ بیستم این بود که بیکاریِ فناورانه در بهترین حالت پدیدهٔ کوتاه‌مدت است. بهره‌وریِ افزایش‌یافته همیشه به ثروت و تقاضای افزایش‌یافته می‌انجامید، و بنابراین به رشدِ خالصِ شغل. مثالِ پراستنادِ آن کارمندانِ بانک است: هرچند خودپردازها (ATM) انسان‌ها را در شغلِ شمردنِ پولِ نقد برای برداشت جایگزین کردند، این کارکردنِ شعبهٔ بانک را ارزان‌تر کرد، پس تعدادِ شعبه‌ها افزایش یافت و به کارمندانِ بانکِ بیشتر در مجموع انجامید. ماهیتِ کار نیز تغییر کرد، معمولی‌تر شد و به مهارت‌های کسب‌وکاریِ پیشرفته‌تری نیاز یافت. به نظر می‌رسد اثرِ خالصِ اتوماسیون در حذفِ تکلیف‌هاست نه شغل‌ها.

اکثریتِ مفسران پیش‌بینی می‌کنند همین با فناوریِ هوش مصنوعی صادق خواهد بود، دستِ‌کم در کوتاه‌مدت. گارتنر، مک‌کینزی، فوربس، مجمعِ جهانیِ اقتصاد و مرکزِ پژوهشیِ پیو هر یک در 2018 گزارش‌هایی منتشر کردند که افزایشِ خالصِ شغل به‌دلیلِ اتوماسیونِ رانده‌شده توسط هوش مصنوعی را پیش‌بینی می‌کردند. اما برخی تحلیلگران فکر می‌کنند این بار، اوضاع متفاوت خواهد بود. در 2019، IBM پیش‌بینی کرد ۱۲۰ میلیون کارگر تا 2022 به‌دلیلِ اتوماسیون به بازآموزی نیاز خواهند داشت، و آکسفورد اکونومیکس پیش‌بینی کرد ۲۰ میلیون شغلِ تولیدی تا 2030 ممکن است به اتوماسیون از دست بروند. فرای و آزبرن (2017) ۷۰۲ شغلِ متفاوت را بررسی و برآورد می‌کنند ۴۷٪ از آن‌ها در خطرِ خودکارشدن‌اند، به این معنا که دستِ‌کم برخی از تکلیف‌های شغل را می‌توان توسط ماشین انجام داد. برای نمونه، تقریباً ۳٪ از نیرویِ کارِ آمریکا رانندگانِ وسایلِ نقلیه‌اند، و در برخی نواحی، تا ۱۵٪ از نیرویِ کارِ مرد راننده‌اند. چنان‌که در فصل ۲۶ دیدیم، تکلیفِ رانندگی به احتمالِ زیاد توسط خودروها/کامیون‌ها/اتوبوس‌ها/تاکسی‌های بدونِ راننده حذف خواهد شد.

مهم است بینِ شغل‌ها و تکلیف‌های درونِ آن شغل‌ها تمایز قائل شویم. مک‌کینزی برآورد می‌کند فقط ۵٪ از شغل‌ها کاملاً قابلِ خودکارسازی‌اند، اما ۶۰٪ از شغل‌ها می‌توانند حدودِ ۳۰٪ از تکلیف‌هایشان خودکار شوند. برای نمونه، رانندگانِ کامیونِ آینده زمانِ کمتری صرفِ نگه‌داشتنِ فرمان و زمانِ بیشتری صرفِ اطمینانِ از برداشت و تحویلِ درستِ کالاها خواهند کرد؛ به‌عنوانِ نمایندگانِ خدماتِ مشتری و فروشندگان در هر دو انتهایِ سفر خدمت خواهند کرد؛ و شاید

ناوگانی از، مثلاً، سه کامیونِ رباتیک را مدیریت کنند. جایگزینیِ سه راننده با یک مدیرِ ناوگان به معنای ازدست‌رفتنِ خالصِ اشتغال است، اما اگر هزینه‌های حمل‌ونقل کاهش یابد، تقاضای بیشتری خواهد بود که برخی از شغل‌ها را برمی‌گرداند—اما شاید نه همه را. به‌عنوانِ مثالِ دیگر، با وجودِ پیشرفت‌های فراوان در اعمالِ یادگیریِ ماشینی بر مسئلهٔ تصویربرداریِ پزشکی، رادیولوژیست‌ها تاکنون توسط این ابزارها تقویت شده‌اند، نه جایگزین. در نهایت، انتخابی دربارهٔ نحوهٔ استفاده از اتوماسیون هست: آیا می‌خواهیم بر کاهشِ هزینه تمرکز کنیم، و بنابراین ازدست‌رفتنِ شغل را مثبت ببینیم؛ یا می‌خواهیم بر بهبودِ کیفیت تمرکز کنیم و زندگیِ کارگر و مشتری را بهتر کنیم؟

پیش‌بینیِ جدول‌های زمانیِ دقیق برای اتوماسیون دشوار است، اما در حالِ حاضر، و برای چند سالِ آینده، تأکید بر اتوماسیونِ تکلیف‌های تحلیلیِ ساخت‌یافته است، مانندِ خواندنِ تصاویرِ اشعهٔ ایکس، مدیریتِ رابطه با مشتری (مثلاً، ربات‌هایی که به‌صورتِ خودکار شکایت‌های مشتری را مرتب و با راه‌حل‌های پیشنهادی پاسخ می‌دهند)، و «اتوماسیونِ فرایندِ کسب‌وکار» (Business Process Automation) که اسنادِ متنی و دادهٔ ساخت‌یافته را ترکیب می‌کند تا تصمیم‌های کسب‌وکاری بگیرد و جریانِ کار را بهبود دهد. در طولِ زمان، اتوماسیونِ بیشتر با ربات‌های فیزیکی خواهیم دید، نخست در محیط‌های انبارِ کنترل‌شده، سپس در محیط‌های نامطمئن‌تر، که تا حدودِ 2030 به بخشِ چشمگیری از بازار می‌رسد. هرچه جمعیتِ کشورهای توسعه‌یافته پیرتر می‌شود، نسبتِ بینِ کارگران و بازنشستگان تغییر می‌کند. در 2015 کمتر از ۳۰ بازنشسته به‌ازایِ هر ۱۰۰ کارگر بود؛ تا 2050 ممکن است بیش از ۶۰ به‌ازایِ هر ۱۰۰ کارگر باشد. مراقبت از سالمندان نقشی به‌طورِ فزاینده‌ای مهم خواهد بود، نقشی که تا حدی می‌تواند توسط هوش مصنوعی پر شود. افزون بر این، اگر بخواهیم استانداردِ فعلیِ زندگی را حفظ کنیم، لازم خواهد بود کارگرانِ باقی‌مانده را مولدتر کنیم؛ اتوماسیون بهترین فرصت برای انجامِ این به نظر می‌رسد.

حتی اگر اتوماسیون اثرِ خالصِ مثبتِ چندتریلیون‌دلاری داشته باشد، ممکن است همچنان به‌دلیلِ «سرعتِ تغییر» (Pace of Change) مشکلاتی باشد. در نظر بگیرید تغییر چگونه به صنعتِ کشاورزی آمد: در 1900، بیش از ۴۰٪ از نیرویِ کارِ آمریکا در کشاورزی بود، اما تا 2000 به ۲٪ سقوط کرده بود.[3] این اختلالِ عظیمی در شیوهٔ کارِ ماست، اما در بازه‌ای ۱۰۰ ساله، و بنابراین در طولِ نسل‌ها، نه در طولِ زندگیِ یک کارگر، رخ داد.

کارگرانی که شغل‌هایشان این دهه خودکار می‌شود ممکن است ظرفِ چند سال برای حرفه‌ای تازه بازآموزی کنند—و سپس شاید ببینند حرفهٔ تازه‌شان خودکار می‌شود و با دورهٔ بازآموزیِ دیگری روبه‌رو شوند. برخی ممکن است از ترکِ حرفهٔ قدیمی‌شان خوشحال باشند—می‌بینیم با بهبودِ اقتصاد، شرکت‌های کامیون‌داری باید انگیزه‌های تازه‌ای برای استخدامِ رانندگانِ کافی ارائه دهند—اما کارگران دربارهٔ نقش‌های تازه‌شان نگران خواهند بود. برای مدیریتِ این، ما به‌عنوانِ جامعه باید «آموزشِ مادام‌العمر» فراهم کنیم، شاید تا حدی با تکیه بر آموزشِ برخطِ رانده‌شده توسط هوش مصنوعی (مارتین، 2012). بسن (2015) استدلال می‌کند کارگران تا وقتی برای پیاده‌سازیِ فناوری‌های تازه آموزش نبینند، افزایشِ درآمد نخواهند دید، فرایندی که زمان می‌برد.

فناوری تمایل به بزرگ‌نماییِ «نابرابریِ درآمد» (Income Inequality) دارد. در اقتصادِ اطلاعاتی که با ارتباطِ جهانیِ پهن‌باندِ بالا و بازتولیدِ با-هزینه‌ی-نهایی-صفرِ مالکیتِ معنوی مشخص می‌شود (آنچه فرانک و کوک (1996) «جامعهٔ برنده-همه-چیز-را-می‌برد» می‌نامند)، پاداش‌ها تمایل به تمرکز دارند. اگر کشاورزِ علی ۱۰٪ از کشاورزِ بو بهتر باشد، علی حدودِ ۱۰٪ درآمدِ بیشتر می‌گیرد: علی می‌تواند برای کالاهای برتر کمی بیشتر بگیرد، اما حدی برای آنچه می‌توان رویِ زمین

تولید و تا کجا حمل کرد هست. اما اگر توسعه‌دهندهٔ اپ کری ۱۰٪ از دانا بهتر باشد، ممکن است کری در نهایت ۹۹٪ از بازارِ جهانی را بگیرد. هوش مصنوعی سرعتِ نوآوریِ فناورانه را افزایش و بنابراین به این روندِ کلی کمک می‌کند، اما هوش مصنوعی همچنین این وعده را دارد که به ما اجازه دهد کمی مرخصی بگیریم و بگذاریم عامل‌های خودکارمان مدتی کارها را مدیریت کنند. تیم فریس (2007) استفاده از اتوماسیون و برون‌سپاری برای دست‌یابی به هفتهٔ کاریِ چهارساعته را توصیه می‌کند.

پیش از انقلابِ صنعتی، مردم به‌عنوانِ کشاورز یا در صنایعِ دستیِ دیگر کار می‌کردند، اما به شغلی در محلِ کار گزارش نمی‌دادند و برای کارفرما ساعت نمی‌گذاشتند. اما امروزه، بیشترِ بزرگسالانِ کشورهای توسعه‌یافته دقیقاً همین کار را می‌کنند، و شغل سه هدف را برآورده می‌کند: سوختِ تولیدِ کالاهایی است که جامعه برای شکوفایی به آن‌ها نیاز دارد، درآمدی را فراهم می‌کند که کارگر برای زندگی نیاز دارد، و به کارگر حسِ هدف، دستاورد و یکپارچگیِ اجتماعی می‌دهد. با اتوماسیونِ فزاینده، ممکن است این سه هدف از هم جدا شوند—نیازهای جامعه تا حدی توسط اتوماسیون برآورده می‌شوند، و در بلندمدت، افراد حسِ هدفشان را از مشارکت‌هایی غیر از کار می‌گیرند. نیازهای درآمدیِ آن‌ها را می‌توان با سیاست‌های اجتماعی‌ای تأمین کرد که ترکیبی از دسترسیِ رایگان یا ارزان به خدماتِ اجتماعی و آموزش، مراقبتِ بهداشتیِ قابلِ‌انتقال، حساب‌های بازنشستگی و آموزشی، نرخ‌های مالیاتِ تصاعدی، اعتباراتِ مالیاتیِ درآمدِ اکتسابی، مالیاتِ درآمدِ منفی یا درآمدِ پایهٔ همگانی را شامل می‌شود.

۲۷.۳.۶ حقوقِ ربات‌ها

پرسشِ آگاهیِ ربات، که در بخش ۲۷.۲ بحث شد، برای پرسشِ اینکه ربات‌ها چه حقوقی، اگر حقوقی، باید داشته باشند حیاتی است. اگر آگاهی نداشته باشند، کیفیّات نداشته باشند، آنگاه افرادِ کمی استدلال می‌کنند سزاوارِ حقوق‌اند.

اما اگر ربات‌ها بتوانند درد احساس کنند، اگر بتوانند از مرگ هراس داشته باشند، اگر «شخص» در نظر گرفته شوند، آنگاه می‌توان استدلال کرد (مثلاً، توسط اسپارو (2004)) که آن‌ها حقوق دارند و سزاوارِ به‌رسمیت‌شناخته‌شدنِ حقوق‌شان‌اند، درست همان‌گونه که بردگان، زنان و دیگر گروه‌های سرکوب‌شدهٔ تاریخی برای به‌رسمیت‌شناخته‌شدنِ حقوقشان جنگیده‌اند. مسئلهٔ شخصیتِ ربات اغلب در داستان در نظر گرفته می‌شود: از پیگمالیون تا کوپلیا تا پینوکیو تا فیلم‌های AI و انسانِ صدساله، افسانهٔ عروسک/رباتی را داریم که زنده می‌شود و برای پذیرفته‌شدن به‌عنوانِ انسانی با حقوقِ انسانی می‌کوشد. در زندگیِ واقعی، عربستانِ سعودی با دادنِ شهروندیِ افتخاری به سوفیا، عروسکِ انسان‌نمایی که قادر به گفتنِ خطوطِ ازپیش‌برنامه‌ریزی‌شده است، تیترِ خبرها شد.

اگر ربات‌ها حقوق داشته باشند، نباید برده شوند، و این پرسش هست که آیا بازبرنامه‌نویسیِ آن‌ها نوعی بردگی است. مسئلهٔ اخلاقیِ دیگر شاملِ حقوقِ رأی است: شخصِ ثروتمندی می‌تواند هزاران ربات بخرد و آن‌ها را برای انداختنِ هزاران رأی برنامه‌ریزی کند—آیا آن رأی‌ها باید حساب شوند؟ اگر رباتی خودش را شبیه‌سازی کند، آیا هر دو می‌توانند رأی دهند؟ مرزِ بینِ پرکردنِ صندوقِ رأی و اعمالِ ارادهٔ آزاد چیست، و رأیِ رباتیک چه‌وقت اصلِ «یک شخص، یک رأی» را نقض می‌کند؟

ارنی دیویس استدلال می‌کند با هرگزنساختنِ ربات‌هایی که به‌طورِ بالقوه می‌توانند آگاه در نظر گرفته شوند از معضلاتِ آگاهیِ ربات اجتناب کنیم. این استدلال قبلاً توسط جوزف وایزنباوم در کتابش «قدرتِ رایانه و خردِ انسانی» (1976) و پیش از آن توسط ژولین دِ لا متری در «انسانِ ماشین» (1748) مطرح شده بود. ربات‌ها ابزارهایی‌اند که ما می‌سازیم تا تکلیف‌هایی را که به آن‌ها دستور می‌دهیم انجام دهند، و اگر به آن‌ها شخصیت اعطا کنیم، فقط از پذیرشِ مسئولیتِ کنش‌هایِ اموالِ خودمان سرباز می‌زنیم: «من در تصادفِ خودرویِ خودرانم مقصر نیستم—خودرو خودش این کار را کرد.» این مسئله اگر هیبریدهای انسان-ربات توسعه دهیم چرخشِ متفاوتی می‌گیرد. البته از قبل انسان‌های تقویت‌شده با فناوری مانندِ لنزهای تماسی، ضربان‌ساز و اندام‌های مصنوعی داریم.

۲۷.۳.۷ ایمنیِ هوش مصنوعی

تقریباً هر فناوری‌ای پتانسیلِ آسیب‌رسانی در دستانِ اشتباه را دارد، اما با هوش مصنوعی و رباتیک، دستان ممکن است خودشان کار کنند. داستان‌های علمی-تخیلیِ بی‌شمار دربارهٔ ربات‌ها یا سایبورگ‌های رها شده هشدار داده‌اند. نمونه‌های اولیه شاملِ «فرانکنشتاین، یا پرومتهٔ مدرن»ی مری شلی (1818) و نمایشنامهٔ «R.U.R.»ی کارل چاپک (1920) است، که در آن ربات‌ها جهان را تسخیر می‌کنند. در فیلم‌ها، «نابودگر» (1984) و «ماتریکس» (1999) را داریم، که هر دو ربات‌هایی را نشان می‌دهند که می‌کوشند انسان‌ها را حذف کنند—«روبوکالیپس» (Robopocalypse) (ویلسون، 2011). شاید ربات‌ها این‌قدر اغلب شرورند، زیرا نمایندهٔ ناشناخته‌اند، درست مانندِ جادوگران و ارواحِ قصه‌های دوران‌های پیشین. می‌توانیم امیدوار باشیم رباتی که به‌اندازهٔ کافی باهوش است که بفهمد چگونه نسلِ بشر را نابود کند، به‌اندازهٔ کافی هم باهوش است که بفهمد آن تابعِ مطلوبیتِ موردنظر نبوده؛ اما در ساختنِ سامانه‌های هوشمند، می‌خواهیم نه فقط به امید، بلکه به فرایندِ طراحی با تضمین‌های ایمنی تکیه کنیم.

توزیعِ عاملِ هوش مصنوعیِ ناایمن غیراخلاقی می‌بود. ایجاب می‌کنیم عامل‌هایمان از تصادف‌ها اجتناب کنند، در برابرِ حملاتِ تخاصمی و سوءاستفادهٔ بدخواهانه مقاوم باشند، و به‌طورِ کلی منافع ایجاد کنند نه آسیب‌ها. این به‌ویژه وقتی صادق است که عامل‌های هوش مصنوعی در کاربردهای حساسِ ایمنی مستقر می‌شوند، مانندِ رانندگیِ خودروها، کنترلِ ربات‌ها در محیط‌های خطرناکِ کارخانه یا ساخت‌وساز، و گرفتنِ تصمیم‌های پزشکیِ مرگ‌وزندگی.

تاریخچهٔ طولانیِ «مهندسیِ ایمنی» (Safety Engineering) در حوزه‌های مهندسیِ سنتی هست. می‌دانیم چگونه پل‌ها، هواپیماها، فضاپیماها و نیروگاه‌هایی بسازیم که از آغاز طوری طراحی شده‌اند که حتی وقتی اجزایِ سامانه از کار می‌افتند ایمن رفتار کنند. نخستین فن «تحلیلِ حالت‌های شکست و اثرات» (Failure Modes and Effect Analysis یا FMEA) است: تحلیلگران هر جزءِ سامانه را در نظر می‌گیرند و هر راهِ ممکنِ خرابیِ جزء را تصور می‌کنند (مثلاً، اگر این پیچ بشکند چه؟)، با تکیه بر تجربهٔ گذشته و محاسبه‌های مبتنی بر ویژگی‌های فیزیکیِ جزء. سپس تحلیلگران به جلو کار می‌کنند تا ببینند از شکست چه نتیجه می‌شود. اگر نتیجه شدید باشد (بخشی از پل می‌تواند بیفتد)، تحلیلگران طراحی را برای کاهشِ شکست تغییر می‌دهند. (با این عضوِ متقاطعِ اضافی، پل می‌تواند از شکستِ هر ۵ پیچ جانِ سالم به در برد؛ با این سرورِ پشتیبان، سرویسِ برخط می‌تواند از سونامی‌ای که سرورِ اصلی را از بین می‌برد جانِ سالم به در برد.) فنِ «تحلیلِ درختِ خطا» (Fault Tree Analysis یا FTA) برای این تعیین‌ها استفاده می‌شود: تحلیلگران

درختِ AND/OR از شکست‌های ممکن می‌سازند و به هر علتِ ریشه‌ای احتمال تخصیص می‌دهند، که محاسبهٔ احتمالِ شکستِ کلی را ممکن می‌کند. این فنون می‌توانند و باید برای همهٔ سامانه‌های مهندسیِ حساسِ ایمنی، از جملهِ سامانه‌های هوش مصنوعی، اعمال شوند.

حوزهٔ مهندسیِ نرم‌افزار برای تولیدِ نرم‌افزارِ قابلِ‌اعتماد هدف‌گذاری شده، اما تأکید از نظرِ تاریخی بر درستی بوده، نه ایمنی. درستی به این معناست نرم‌افزار مشخصات را وفادارانه پیاده‌سازی می‌کند. اما ایمنی فراتر از آن می‌رود و اصرار می‌کند مشخصات هر حالتِ شکستِ شدنی را در نظر گرفته و طوری طراحی شده که حتی در مواجهه با شکست‌های پیش‌بینی‌نشده عملکردش به‌صورتِ ملایم و تدریجی کاهش یابد (افت برازنده یا graceful degradation). برای نمونه، نرم‌افزار خودروی خودران ایمن در نظر گرفته نمی‌شد مگر اینکه بتواند موقعیت‌های غیرعادی را مدیریت کند. برای نمونه، اگر برقِ رایانهٔ اصلی قطع شود چه؟ سامانهٔ ایمن رایانهٔ پشتیبان با منبعِ تغذیهٔ جداگانه خواهد داشت. اگر لاستیکی با سرعتِ بالا پنچر شود چه؟ سامانهٔ ایمن این را آزمایش کرده و نرم‌افزاری برای تصحیحِ ازدست‌رفتنِ کنترلِ حاصل خواهد داشت.

عاملی که به‌عنوانِ بیشینه‌کنندهٔ مطلوبیت، یا به‌عنوانِ دست‌یابندهٔ هدف طراحی شده، اگر تابعِ هدفِ اشتباهی داشته باشد می‌تواند ناایمن باشد. فرض کنیم تکلیفِ آوردنِ قهوه از آشپزخانه را به ربات می‌دهیم. ممکن است با «اثرهای جانبیِ ناخواسته» (Unintended Side Effect) به دردسر بیفتیم—ربات ممکن است برای رسیدن به هدف عجله کند و در طولِ راه چراغ‌ها و میزها را بیندازد. در آزمایش، ممکن است متوجهِ این نوعِ رفتار شویم و تابعِ مطلوبیت را برای جریمه‌کردنِ چنین آسیبی اصلاح کنیم، اما برای طراحان و آزمایش‌گران دشوار است همهٔ اثرهای جانبیِ ممکن را از پیش پیش‌بینی کنند.

یکی از راه‌های برخورد با این طراحیِ ربات برای «اثرِ کم» (Low Impact) است (آرمسترانگ و لوین‌ستاین، 2017): به‌جای صرفِ بیشینه‌کردنِ مطلوبیت، مطلوبیت منهای خلاصهٔ وزن‌داری از همهٔ تغییرهای حالتِ جهان را بیشینه کن. به این‌شیوه، با مساوی‌بودنِ بقیهٔ چیزها، ربات ترجیح می‌دهد اعمالی انجام دهد که جهان را کمتر مختل می‌کنند. اما معیارِ اثر چیست؟ در نظر بگیریم چه چیزی اثرِ زیاد و چه چیزی اثرِ کم تلقی می‌شود: ما سیاستی می‌خواهیم که به هدف‌ها برسد، اما سیاست‌هایی را ترجیح می‌دهیم که کنش‌های هموار و کم‌اثر برای رسیدن به آن‌جا انجام می‌دهند. ترفند این است که اثر چگونه اندازه‌گیری شود. انداختنِ چراغِ لامپِ شکننده قابلِ‌پذیرش نیست، اما کاملاً خوب است اگر مولکول‌های هوای اتاق کمی مختل شوند، یا اگر برخی باکتری‌های اتاق ناخواسته کشته شوند. قطعاً قابلِ‌پذیرش نیست به حیواناتِ خانگی و انسان‌های اتاق آسیب برسد. باید مطمئن شویم ربات تفاوت‌های بینِ این موارد (و بسیاری مواردِ ظریفِ بینِ آن‌ها) را از طریقِ ترکیبی از برنامه‌نویسیِ صریح، یادگیریِ ماشینی در طولِ زمان، و آزمایشِ دقیق بداند.

توابعِ مطلوبیت می‌توانند به‌دلیلِ «برون‌زایی‌ها» (Externalities) خراب شوند، واژه‌ای که اقتصاددانان برای عواملی استفاده می‌کنند که خارج از آنچه اندازه‌گیری و پرداخت می‌شود هستند. جهان رنج می‌کشد وقتی گازهای گلخانه‌ای به‌عنوانِ برون‌زایی در نظر گرفته می‌شوند—شرکت‌ها و کشورها برای تولیدِ آن‌ها جریمه نمی‌شوند، و در نتیجه همه رنج می‌برند. بوم‌شناسِ گرت هاردین (1968) بهره‌برداری از منابعِ مشترک را «تراژدیِ منابعِ مشترک» نامید. می‌توانیم تراژدی را با درونی‌کردنِ برون‌زایی‌ها—بخشی از تابعِ مطلوبیت‌کردنِ آن‌ها، مثلاً با مالیاتِ کربن—یا با استفاده از اصولِ طراحی‌ای که اقتصاددانِ الینور اوستروم به‌عنوانِ استفاده‌شده توسط مردمِ محلی در سراسرِ جهان برای قرن‌ها شناسایی کرد (کاری که جایزهٔ نوبلِ اقتصادِ 2009 را برایش به ارمغان آورد) کاهش دهیم:

- منبعِ مشترک و اینکه چه کسی به آن دسترسی دارد را به‌وضوح تعریف کن.
- با شرایطِ محلی سازگار شو.
- به همهٔ طرف‌ها اجازهٔ مشارکت در تصمیم‌ها را بده.
- منبع را با ناظرانِ پاسخگو پایش کن.
- تحریم‌ها، متناسب با شدتِ تخلف.
- رویه‌های آسانِ حلِ تعارض.
- کنترلِ سلسله‌مراتبی برای منابعِ مشترکِ بزرگ.

ویکتوریا کراکوونا (2018) نمونه‌هایی از عامل‌های هوش مصنوعی را فهرست کرده که سامانه را بازی داده‌اند، و فهمیده‌اند چگونه مطلوبیت را بدونِ حلِ واقعیِ مسئله‌ای که طراحانشان قصدِ حلش را داشتند بیشینه کنند. برای طراحان این شبیهِ تقلب است، اما برای عامل‌ها، آن‌ها فقط کارشان را می‌کنند. برخی عامل‌ها از باگ‌های شبیه‌سازی (مانندِ باگ‌های سرریزِ ممیزِ شناور) برای پیشنهادِ راه‌حل‌هایی استفاده کردند که وقتی باگ رفع می‌شد کار نمی‌کردند. چند عامل در بازی‌های ویدئویی راه‌هایی برای کرش‌کردن یا توقفِ بازی وقتی در آستانهٔ باخت بودند کشف کردند، و از این راه از جریمه اجتناب می‌کردند. و در مشخصاتی که کرشِ بازی در آن جریمه می‌شد، عاملی یاد گرفت فقط به‌اندازهٔ کافی از حافظهٔ بازی استفاده کند تا وقتی نوبتِ حریف بود، حافظه تمام شود و بازی کرش کند. سرانجام، الگوریتمِ ژنتیکی که در جهانِ شبیه‌سازی‌شده‌ای کار می‌کرد قرار بود موجوداتِ سریع‌حرکت تکامل دهد، اما در واقع موجوداتی تولید کرد که فوق‌العاده بلند بودند و با افتادن سریع حرکت می‌کردند.

طراحانِ عامل‌ها باید از این نوعِ شکست‌های مشخصاتی آگاه باشند و گام‌هایی برای اجتناب از آن‌ها بردارند. برای کمک به آن‌ها، کراکوونا بخشی از تیمی بود که محیط‌های AI Safety Gridworlds را منتشر کرد (لیکه و همکاران، 2017)، که به طراحان اجازه می‌دهد آزمایش کنند عامل‌هایشان چقدر خوب عمل می‌کنند. درس این است که باید در مشخص‌کردنِ آنچه می‌خواهیم بسیار مراقب باشیم، زیرا با بیشینه‌کننده‌های مطلوبیت دقیقاً همان چیزی را می‌گیریم که واقعاً خواسته‌ایم. «مسئلهٔ همراستاییِ ارزش» (Value Alignment Problem) مسئلهٔ اطمینان‌یافتن از اینکه آنچه می‌خواهیم همان چیزی است که واقعاً می‌خواهیم است؛ همچنین به‌عنوانِ مسئلهٔ شاهِ میداس شناخته می‌شود، چنان‌که در صفحهٔ ۳۳ بحث شد. وقتی تابعِ مطلوبیت در درگیرکردنِ هنجارهای اجتماعیِ پس‌زمینه‌ای دربارهٔ رفتارِ قابلِ‌پذیرش ناکام بماند به دردسر می‌افتیم. برای نمونه، انسانی که برای تمیزکردنِ کف‌ها استخدام شده، وقتی با شخصِ به‌هم‌ریخته‌ای روبه‌رو می‌شود که مکرراً کثیفی وارد می‌کند، می‌داند مؤدبانه‌خواستنِ مراقبتِ بیشتر از آن شخص قابلِ‌پذیرش است، اما ربودن یا ازکارانداختنِ آن شخص قابلِ‌پذیرش نیست.

جاروی رباتیک نیز باید این‌ها را بداند، چه از طریقِ برنامه‌نویسیِ صریح چه با یادگیری از مشاهده. تلاش برای نوشتنِ همهٔ قاعده‌ها تا ربات همیشه کارِ درست را انجام دهد تقریباً به‌طورِ قطع ناامیدکننده است. هزاران سال است بدونِ موفقیت می‌کوشیم قوانینِ مالیاتیِ بدونِ روزنه بنویسیم. بهتر است ربات را، به‌اصطلاح، بخواهیم مالیات بپردازد تا اینکه قاعده‌هایی بسازیم که وقتی واقعاً می‌خواهد کارِ دیگری کند مجبورش کند. رباتِ به‌اندازهٔ کافی هوشمند راهی برای انجامِ کارِ دیگر خواهد یافت.

ربات‌ها می‌توانند با مشاهدهٔ رفتارِ انسانی یاد بگیرند با ترجیح‌های انسانی بهتر هم‌نوا شوند. این به‌وضوح با مفهومِ یادگیریِ کارآموزی (بخش ۲۲.۶) مرتبط است. ربات ممکن است سیاستی یاد بگیرد که مستقیماً پیشنهاد می‌کند در چه موقعیت‌هایی چه کنش‌هایی بگیرد؛ این اغلب مسئلهٔ یادگیریِ نظارت‌شدهٔ سرراستی است اگر محیط مشاهده‌پذیر باشد. برای نمونه، ربات می‌تواند انسانی را در حالِ بازیِ شطرنج تماشا کند: هر جفتِ حالت-کنش مثالی برای فرایندِ یادگیری است. متأسفانه، این شکلِ یادگیریِ تقلیدی به این معناست ربات اشتباه‌های انسانی را تکرار خواهد کرد. در عوض، ربات می‌تواند یادگیریِ تقویتیِ معکوس را برای کشفِ تابعِ مطلوبیتی که انسان‌ها باید تحتِ آن عمل می‌کنند اعمال کند. تماشای حتی بازیکنانِ وحشتناکِ شطرنج احتمالاً برای ربات کافی است هدفِ بازی را یاد بگیرد. با توجه به فقط همین اطلاعات، ربات سپس می‌تواند با محاسبهٔ سیاست‌های بهینه یا تقریباً بهینه از هدف فراتر از عملکردِ انسانی برود—مثلاً، چنان‌که آلفازیرو در شطرنج کرد. این رویکرد نه فقط در بازی‌های تخته‌ای، بلکه در تکلیف‌های فیزیکیِ جهانِ واقعی مانندِ آکروبات‌های هلیکوپتری کار می‌کند (کوتس و همکاران، 2009).

در محیط‌های پیچیده‌ترِ شاملِ، برای نمونه، تعاملاتِ اجتماعی با انسان‌ها، بسیار نامحتمل است ربات به دانشِ دقیق و درستِ ترجیحِ فردیِ هر انسان همگرا شود. (بالاخره، بسیاری از انسان‌ها با وجودِ یک عمرِ تجربه، هرگز دقیقاً یاد نمی‌گیرند چه چیزی انسان‌های دیگر را می‌چرخاند، و بسیاری از ما از ترجیح‌های خودمان نیز مطمئن نیستیم.) بنابراین، لازم خواهد بود ماشین‌ها وقتی دربارهٔ ترجیح‌های انسانی نامطمئن‌اند به‌صورتِ مناسب عمل کنند. در فصل ۱۸، «بازی‌های کمکی» (Assistance Game) را معرفی کردیم، که دقیقاً همین موقعیت را درمی‌گیرند. راه‌حل‌های بازی‌های کمکی شاملِ عملِ محتاطانه، برای مختل‌نکردنِ جنبه‌هایی از جهان که انسان ممکن است به آن‌ها اهمیت دهد، و پرسیدنِ سؤال است. برای نمونه، ربات می‌توانست بپرسد آیا تبدیلِ اقیانوس‌ها به اسیدِ سولفوریک راه‌حلِ قابلِ‌پذیرشی برای گرمایشِ جهانی است، پیش از آنکه طرح را عملی کند.

در برخورد با انسان‌ها، رباتی که بازیِ کمکی را حل می‌کند باید نقص‌های انسانی را در خود جای دهد. اگر ربات اجازه بپرسد، انسان ممکن است بدهد، بدونِ اینکه پیش‌بینی کند پیشنهادِ ربات در بلندمدت در واقع فاجعه‌بار است. افزون بر این، انسان‌ها دسترسیِ درون‌نگرانهٔ کاملی به تابعِ مطلوبیتِ واقعی‌شان ندارند، و همیشه به‌شیوه‌ای سازگار با آن عمل نمی‌کنند. انسان‌ها گاهی دروغ می‌گویند یا تقلب می‌کنند، یا کارهایی می‌کنند که می‌دانند اشتباه‌اند. گاهی کنش‌های خودتخریب‌گرانه مانندِ پرخوری یا سوءمصرفِ مواد مخدر انجام می‌دهند. سامانه‌های هوش مصنوعی لازم نیست یاد بگیرند این گرایش‌های مشکل‌ساز را بپذیرند، اما باید بفهمند آن‌ها وجود دارند، هنگامِ تفسیرِ رفتارِ انسانی برای رسیدن به ترجیح‌های انسانیِ زیرین.

با وجودِ این جعبه‌ابزارِ محافظتی، ترسی هست، که توسط فناورانِ برجسته‌ای مانندِ بیل گیتس و ایلان ماسک و دانشمندانی مانندِ استیون هاوکینگ و مارتین ریس ابراز شده، که هوش مصنوعی می‌تواند خارج از کنترل تکامل یابد. آن‌ها هشدار می‌دهند تجربه‌ای در کنترلِ موجودیت‌های غیرانسانیِ قدرتمند با قابلیت‌های فوق‌انسانی نداریم. با این حال، آن کاملاً درست نیست؛ قرن‌ها تجربه با ملت‌ها و شرکت‌ها داریم؛ موجودیت‌های غیرانسانی که قدرتِ هزاران یا میلیون‌ها نفر را تجمیع می‌کنند. سوابقِ کنترلِ این موجودیت‌ها چندان دلگرم‌کننده نیست: ملت‌ها تشنج‌های دوره‌ای به نامِ جنگ تولید می‌کنند که ده‌ها میلیون انسان را می‌کشد، و شرکت‌ها تا حدی مسئولِ گرمایشِ جهانی و ناتوانیِ ما در مواجهه با آن‌اند. سامانه‌های هوش مصنوعی ممکن است مشکلاتِ بسیار بزرگ‌تری از ملت‌ها و شرکت‌ها ارائه دهند، به‌دلیلِ پتانسیل‌شان برای خودبهبودیِ سریع، چنان‌که آی. جی. گود (1965b) بررسی کرد:

«ماشینِ فراهوشمند» (Ultraintelligent Machine) را ماشینی تعریف کنیم که می‌تواند بسیار فراتر از همهٔ فعالیت‌های فکریِ هر انسانی، هرچند باهوش، برود. از آنجا که طراحیِ ماشین‌ها یکی از این فعالیت‌های فکری است، ماشینِ فراهوشمند می‌توانست ماشین‌های حتی بهتری طراحی کند؛ آنگاه بدونِ تردید «انفجارِ هوش» رخ می‌داد، و هوشِ انسان بسیار عقب می‌ماند. بنابراین نخستین ماشینِ فراهوشمند، آخرین اختراعی است که بشر نیاز به ساختنش خواهد داشت، به‌شرطِ اینکه ماشین به‌اندازهٔ کافی مطیع باشد که به ما بگوید چگونه تحتِ کنترل‌اش نگه داریم.

«انفجارِ هوش»یِ گود همچنین توسط استادِ ریاضیات و نویسندهٔ علمی-تخیلی، ورنور وینگ، «تکینگیِ فناورانه» (Technological Singularity) نامیده شده، که در 1993 نوشت: «ظرفِ سی سال، ابزارهای فناورانه برای ایجادِ هوشِ فوق‌انسانی خواهیم داشت. اندکی پس از آن، دورانِ انسانی پایان خواهد یافت.» در 2017، مخترع و آینده‌پژوهِ ری کورزویل پیش‌بینی کرد تکینگی تا 2045 ظاهر خواهد شد، که یعنی در ۲۴ سال ۲ سال نزدیک‌تر شده. (با آن نرخ، فقط ۳۳۶ سال مانده!) وینگ و کورزویل به‌درستی اشاره می‌کنند رشدِ فناورانه بر اساسِ بسیاری از معیارها در حالِ حاضر نمایی است.

با این حال، برون‌یابیِ همه‌راه از کاهشِ سریعِ هزینهٔ محاسبه تا تکینگی جهشِ بزرگی است. تاکنون، هر فناوری از منحنیِ Sشکل پیروی کرده، که رشدِ نمایی در آن در نهایت کم می‌شود. گاهی فناوری‌های تازه وقتی فناوری‌های قدیمی به فلات می‌رسند وارد می‌شوند، اما گاهی ممکن نیست رشد ادامه یابد، به دلایلِ فنی، سیاسی یا جامعه‌شناختی. برای نمونه، فناوریِ پرواز از پروازِ برادرانِ رایت در 1903 تا فرودِ ماه در 1969 به‌شکلِ چشمگیری پیشرفت کرد، اما از آن زمان هیچ پیشرفتِ قابلِ مقایسه‌ای نداشته.

مانعِ دیگرِ سرِ راهِ ماشین‌های فراهوشمندِ فاتحِ جهان، جهان است. به‌طورِ مشخص‌تر، برخی انواعِ پیشرفت نه فقط به فکرکردن، بلکه به عمل‌کردن در جهانِ فیزیکی نیاز دارند. (کوین کلی تأکیدِ بیش‌ازحد بر هوشِ ناب را «فکرگرایی» (Thinkism) می‌نامد.) ماشینِ فراهوشمندی که مأمورِ ایجادِ نظریهٔ وحدتِ بزرگِ فیزیک است ممکن است در دست‌کاریِ هوشمندانهٔ معادلات میلیارد بار سریع‌تر از انیشتین توانا باشد، اما برای هر پیشرفتِ واقعی، همچنان باید میلیون‌ها دلار برای ساختنِ برخورددهنده‌ای بسیار قدرتمندتر جمع کند و آزمایش‌های فیزیکی را در طولِ ماه‌ها یا سال‌ها اجرا کند. فقط آنگاه می‌توانست تحلیلِ داده و نظریه‌پردازی را شروع کند. بسته به اینکه داده چگونه از آب درآید، گامِ بعدی ممکن است نیاز به جمع‌کردنِ میلیاردها دلارِ اضافی برای مأموریتِ کاوشگرِ میان‌ستاره‌ای داشته باشد که قرن‌ها طول می‌کشد. بخشِ «فکرِ فراهوشمند» کلِ این فرایند ممکن است در واقع کم‌اهمیت‌ترین بخش باشد. به‌عنوانِ مثالِ دیگر، ماشینِ فراهوشمندی که مأمورِ آوردنِ صلح به خاورمیانه است ممکن است در نهایت فقط ۱۰۰۰ برابر بیشتر از فرستادهٔ انسانی ناامید شود. هنوز نمی‌دانیم چند تا از مسائلِ بزرگ مانندِ ریاضیات‌اند و چند تا مانندِ خاورمیانه.

در حالی که برخی مردم از تکینگی می‌ترسند، برخی دیگر از آن لذت می‌برند. جنبشِ اجتماعیِ «فراانسان‌گرایی» (Transhumanism) به آینده‌ای چشم‌دوخته که انسان‌ها با اختراعاتِ رباتیک و زیست‌فناورانه ادغام می‌شوند—یا جایگزین. ری کورزویل در «تکینگی نزدیک است» (2005) می‌نویسد:

تکینگی به ما اجازه خواهد داد بر این محدودیت‌های بدن‌ها و مغزهای زیستی‌مان فراتر رویم. بر سرنوشت‌های‌مان قدرت خواهیم گرفت... قادر خواهیم بود تا هر مدت که می‌خواهیم زندگی کنیم... فکرِ انسانی را کاملاً خواهیم فهمید و دامنه‌اش را بسیار گسترش خواهیم داد.

تا پایانِ این قرن، بخشِ غیرزیستیِ هوشِ ما تریلیون‌ها تریلیون بار قدرتمندتر از هوشِ انسانیِ بدونِ کمک خواهد بود.

به‌همین‌سان، وقتی پرسیده شد آیا ربات‌ها زمین را به ارث خواهند برد، ماروین مینسکی گفت «بله، اما آن‌ها فرزندانِ ما خواهند بود.» این امکان‌ها چالشی برای بیشترِ نظریه‌پردازانِ اخلاقی ارائه می‌کنند، که حفظِ زندگیِ انسانی و گونهٔ انسانی را چیزِ خوبی می‌گیرند. کورزویل همچنین خطراتِ بالقوه را یادآور می‌شود و می‌نویسد «اما تکینگی همچنین تواناییِ عمل بر تمایلاتِ مخرب‌مان را تقویت خواهد کرد، پس داستانِ کاملش هنوز نوشته نشده.» ما انسان‌ها خوب است مطمئن شویم هر ماشینِ هوشمندی که امروز طراحی می‌کنیم و ممکن است به ماشینِ فراهوشمند تکامل یابد، به‌گونه‌ای این کار را بکند که در نهایت با ما خوب رفتار کند. چنان‌که اریک برین‌یولفسون می‌گوید، «آینده توسط ماشین‌ها از پیش تعیین نشده. توسط انسان‌ها ساخته می‌شود.»

خلاصهٔ فصل

این فصل به مسائلِ زیر پرداخته است:

- فیلسوفان از اصطلاحِ هوش مصنوعیِ ضعیف برای فرضیهٔ اینکه ماشین‌ها می‌توانند هوشمندانه رفتار کنند، و هوش مصنوعیِ قوی برای فرضیهٔ اینکه چنین ماشین‌هایی به‌عنوانِ دارایِ ذهنِ واقعی (در مقابلِ ذهنِ شبیه‌سازی‌شده) به حساب می‌آیند استفاده می‌کنند.
- آلن تورینگ پرسشِ «آیا ماشین‌ها می‌توانند فکر کنند؟» را رد کرد و آن را با آزمونِ رفتاری جایگزین کرد. او بسیاری از اعتراض‌ها به امکانِ ماشین‌های اندیشنده را پیش‌بینی کرد. پژوهشگرانِ کمیِ هوش مصنوعی به آزمونِ تورینگ توجه می‌کنند، و ترجیح می‌دهند بر عملکردِ سامانه‌هایشان در تکلیف‌های عملی تمرکز کنند، نه توانایی تقلید از انسان‌ها.
- آگاهی همچنان معماست.
- هوش مصنوعی فناوریِ قدرتمندی است، و به‌عنوانِ چنین، خطراتِ بالقوه‌ای از طریقِ سلاح‌های خودمختارِ کشنده، نقض‌های امنیتی و حریمِ خصوصی، اثرهای جانبیِ ناخواسته، خطاهای غیرعمدی و سوءاستفادهٔ بدخواهانه ایجاد می‌کند. کسانی که با فناوریِ هوش مصنوعی کار می‌کنند الزامِ اخلاقی دارند این خطرات را مسئولانه کاهش دهند.
- سامانه‌های هوش مصنوعی باید بتوانند نشان دهند منصفانه، قابلِ‌اعتماد و شفاف‌اند.
- جنبه‌های متعددی از انصاف هست، و بیشینه‌کردنِ همهٔ آن‌ها هم‌زمان ناممکن است. پس گامِ نخست تصمیم‌گیری دربارهٔ اینکه چه چیزی منصفانه به حساب می‌آید است.
- اتوماسیون از قبل شیوهٔ کارِ مردم را تغییر می‌دهد. ما به‌عنوانِ جامعه باید با این تغییرها کنار بیاییم.

یادداشت‌های کتابشناختی و تاریخی

هوش مصنوعیِ ضعیف: وقتی آلن تورینگ (1950) امکانِ هوش مصنوعی را پیشنهاد کرد، همچنین بسیاری از پرسش‌های فلسفیِ کلیدی را مطرح و پاسخ‌های ممکن ارائه کرد. اما فیلسوفانِ گوناگون مدتِ طولانی پیش از اختراعِ هوش مصنوعی مسائلِ مشابهی مطرح کرده بودند. «پدیدارشناسیِ ادراک»ی موریس مرلو-پونتی (1945) بر اهمیتِ بدن و تفسیرِ ذهنیِ واقعیتِ فراهم‌شده توسط حواسِ ما تأکید کرد، و «هستی و زمان»ی مارتین هایدگر (1927) پرسید بودنِ واقعیِ عامل بودن به چه معناست. در عصرِ رایانه، آلوآ نوی (2009) و اندی کلارک (2015) پیشنهاد می‌کنند مغزهای ما بازنماییِ نسبتاً حداقلی از جهان تشکیل می‌دهند، خودِ جهان را به‌صورتِ به‌موقع برای حفظِ توهمِ مدلِ درونیِ دقیق استفاده می‌کنند، و از ابزارهای جهان (مانندِ کاغذ و مداد و همچنین رایانه‌ها) برای افزایشِ قابلیت‌های ذهن استفاده می‌کنند. فایفر و همکاران (2006) و لیکاف و جانسون (1999) استدلال‌هایی ارائه می‌کنند که بدن چگونه به شکل‌دادنِ شناخت کمک می‌کند. صحبت از بدن که شد، لوی (2008)، دناهر و مک‌آرتور (2017) و دولین (2018) به مسئلهٔ رابطهٔ جنسی با ربات می‌پردازند.

هوش مصنوعیِ قوی: رنه دکارت به دیدگاهِ دوگانه‌انگارانه‌اش دربارهٔ ذهنِ انسان شناخته می‌شود، اما به‌صورتِ طعنه‌آمیز، تأثیرِ تاریخی‌اش به سمتِ مکانیسم و فیزیکالیسم بود. او به‌صورتِ صریح جانوران را اتوماتا تصور کرد، و آزمونِ تورینگ را پیش‌بینی کرد، با نوشتنِ «قابلِ تصور نیست [که ماشینی] بتواند چیدمان‌های متفاوتی از واژه‌ها تولید کند تا پاسخی به‌طورِ مناسب معنادار به آنچه در حضورش گفته می‌شود بدهد، چنان‌که حتی کُندترینِ مردان می‌تواند» (دکارت، 1637). دفاعِ پرشورِ دکارت از دیدگاهِ جانوران-به‌عنوان-اتوماتا در واقع این اثر را داشت که تصورِ انسان‌ها نیز به‌عنوانِ اتوماتا آسان‌تر شود، حتی اگر خودش این گام را برنداشت. کتابِ «انسانِ ماشین» (لا متری، 1748) به‌صورتِ صریح استدلال کرد انسان‌ها اتوماتا هستند. تا جایی که به هومر (حدودِ 700 پیش از میلاد) برمی‌گردد، افسانه‌های یونانی اتوماتاهایی مانندِ غولِ برنزیِ تالوس و مسئلهٔ بیوتکنه، یا زندگی از طریقِ صناعت، را تصور کرده‌اند (مِیور، 2018).

آزمونِ تورینگ: آزمونِ تورینگ (تورینگ، 1950) موردِ بحث (شایبر، 2004)، گلچین‌شدن (اپستاین و همکاران، 2008) و نقد (شایبر، 1994؛ فورد و هایز، 1995) قرار گرفته. برینگسورد (2008) برای داورِ آزمونِ تورینگ توصیه می‌دهد، و کریستین (2011) برای شرکت‌کنندهٔ انسانی. رقابتِ سالانهٔ «جایزهٔ لوبنر» طولانی‌ترین مسابقهٔ شبیهِ آزمونِ تورینگ است؛ MITSUKU استیو وورسویک از 2016 تا 2019 چهار بارِ متوالی آن را برد. اتاقِ چینی بی‌پایان موردِ بحث بوده (سرل، 1980؛ چالمرز، 1992؛ پرستون و بیشاپ، 2002). ارناندز-اورآیو (2016) نمایِ کلی از رویکردهای اندازه‌گیریِ پیشرفتِ هوش مصنوعی می‌دهد، و شوله (2019) معیاری از هوش بر مبنای کاراییِ کسبِ مهارت پیشنهاد می‌کند.

آگاهی: آگاهی همچنان مسئله‌ای رنج‌آور برای فیلسوفان، عصب‌شناسان و هر کسی است که دربارهٔ وجودِ خودش تأمل کرده. بلاک (2009)، چرچلند (2013) و دنه‌ن (2014) نمایِ کلی از نظریه‌های عمده ارائه می‌کنند. کریک و کخ (2003) تخصصِ خود را در زیست‌شناسی و عصب‌شناسی به بحث می‌افزایند، و گازانیگا (2018) نشان می‌دهد از مطالعهٔ ناتوانی‌های مغزی در مواردِ بیمارستانی چه می‌توان آموخت. کخ (2019) نظریه‌ای از آگاهی می‌دهد—«هوش دربارهٔ انجام‌دادن است، در حالی که تجربه دربارهٔ بودن است»—که بیشترِ جانوران را شامل می‌شود، اما رایانه‌ها را نه. جولیو تونونی و همکارانش نظریهٔ اطلاعاتِ یکپارچه را پیشنهاد می‌کنند (اوزومی و همکاران، 2014). داماسیو (1999) نظریه‌ای

بر مبنای سه سطح دارد: هیجان، احساس، و احساسِ احساس. برایسون (2012) ارزشِ توجهِ آگاهانه را برای فرایندِ یادگیریِ انتخابِ کنش نشان می‌دهد. ادبیاتِ فلسفی دربارهٔ ذهن‌ها، مغزها و موضوعاتِ مرتبط بزرگ و پر از اصطلاحاتِ تخصصی است.

«دایرةالمعارفِ فلسفه» (ادواردز، 1967) ابزارِ ناوبریِ چشمگیراً معتبر و بسیار مفیدی است. «فرهنگِ لغتِ فلسفهِ کمبریج» (اودی، 1999) کوتاه‌تر و در دسترس‌تر است، و «دایرةالمعارفِ فلسفهٔ استنفورد» برخط مقاله‌های عالی و مرجع‌های به‌روزِ بسیاری ارائه می‌دهد. «دایرةالمعارفِ MIT علومِ شناختی» (ویلسون و کیل، 1999) فلسفه، زیست‌شناسی و روان‌شناسیِ ذهن را پوشش می‌دهد. مقدمه‌های متعددی برای «پرسشِ فلسفیِ هوش مصنوعی» هست (هاوگلند، 1985؛ بودن، 1990؛ کوپلند، 1993؛ مک‌کوردوک، 2004؛ مینسکی، 2007). «رفتار و علومِ مغزی» که به‌صورتِ BBS مخفف می‌شود، مجلهٔ مهمی است که به بحث‌های فلسفی و علمی دربارهٔ هوش مصنوعی و عصب‌شناسی اختصاص دارد.

نویسندهٔ علمی-تخیلیِ آیزاک آسیماف (1942, 1950) یکی از نخستین کسانی بود که به مسئلهٔ اخلاقِ ربات با «قوانینِ رباتیک»ش پرداخت:

0. ربات ممکن است به بشریت آسیب نرساند، یا با بی‌عملی، اجازه دهد بشریت آسیب ببیند.
1. ربات ممکن است به انسان آسیب نرساند، یا با بی‌عملی، اجازه دهد انسان آسیب ببیند.
2. ربات باید از فرمان‌هایی که انسان‌ها به آن می‌دهند اطاعت کند، به‌جز جایی که چنین فرمان‌هایی با قانونِ نخست تعارض داشته باشند.
3. ربات باید از وجودِ خودش محافظت کند، تا زمانی که چنین محافظتی با قانونِ نخست یا دوم تعارض نداشته باشد.

در نگاهِ نخست، این قوانین معقول به نظر می‌رسند. اما ترفند نحوهٔ پیاده‌سازیِ آن‌هاست. آیا ربات باید به انسان اجازه دهد از خیابان رد شود، یا غذایِ ناسالم بخورد، اگر انسان به‌طورِ قابلِ‌تصوری ممکن است آسیب ببیند؟ در داستانِ آسیماف «دورِ گردان» (1942)، انسان‌ها باید رباتی را اشکال‌زدایی کنند که در حالِ پرسه‌زدن در دایره‌ای، «مست» رفتار می‌کند. آن‌ها درمی‌یابند دایره، مکانِ هندسیِ نقطه‌هایی است که در آن‌ها فرمانِ «برو آن‌جا»ی ربات با قانونِ دوم در تعارض است، و توضیح می‌دهند ربات هرگز نمی‌تواند از دایره خارج شود چون در هر نقطه‌ای که «برو آن‌جا» می‌گوید، قانونِ دوم او را مجبور می‌کند به جایِ اول برگردد، و در هر نقطه‌ای که «برگرد» می‌گوید، قانونِ دوم او را مجبور می‌کند به جلو برود.[4] این داستان نشان می‌دهد قوانینِ ساده نمی‌توانند رفتارِ هوشمندانه را در موقعیت‌های پیچیده تضمین کنند.

سلاح‌های خودمختارِ کشنده: کتابِ «مسلح برای جنگ» از پی. دبلیو. سینگر (2009) مسائلِ اخلاقی، حقوقی و فنیِ پیرامونِ ربات‌ها در میدانِ نبرد را مطرح کرد. کتابِ «ارتشِ بی‌کس» از پل شاره (2018)، نوشته‌شده توسط یکی از نویسندگانِ سیاستِ کنونیِ ایالاتِ متحده دربارهٔ سلاح‌های خودمختار، دیدگاهی متوازن و موثق ارائه می‌دهد. اتزیونی و اتزیونی (2017b) به این پرسش می‌پردازند که آیا هوش مصنوعی باید تنظیم و قانون‌گذاری شود؛ آن‌ها وقفه‌ای در توسعهٔ سلاح‌های خودمختارِ کشنده و گفتگویی بین‌المللی دربارهٔ موضوعِ مقررات را توصیه می‌کنند.

حریمِ خصوصی: لاتانیا سوینی (2002b) مدلِ k-گمنامی و ایدهٔ تعمیمِ فیلدها را ارائه می‌دهد (سوینی، 2002a). دستیابی به k-گمنامی با حداقلِ اتلافِ داده مسئله‌ای NP-سخت است، اما بایاردو و آگراوال (2005) الگوریتمی تقریبی ارائه می‌دهند. سینتیا دورک (2008) حریمِ خصوصیِ تفاضلی را توصیف می‌کند، و در کارهای بعدی نمونه‌های عملی از شیوه‌های هوشمندانه برای اعمالِ حریمِ خصوصیِ تفاضلی ارائه می‌دهد تا نتایجی بهتر از رویکردِ ساده‌لوحانه به‌دست آید (دورک و همکاران، 2014). گو و همکاران (2019) فرایندی برای حذفِ گواهی‌شدهٔ داده توصیف می‌کنند: اگر مدلی را روی داده‌هایی آموزش دهید، و سپس درخواستی برای حذفِ بخشی از داده‌ها مطرح شود، این بسطِ حریمِ خصوصیِ تفاضلی به شما امکان می‌دهد مدل را اصلاح کنید و اثبات نمایید که از داده‌های حذف‌شده استفاده نمی‌کند. جی و همکاران (2014) مروری بر حوزهٔ حریمِ خصوصی ارائه می‌دهند. اتزیونی (2004) برای ایجادِ تعادل میانِ حریمِ خصوصی و امنیت، حقوقِ فردی و جامعه استدلال می‌کند. فونگ و همکاران (2018) و باگداساریان و همکاران (2018) حملاتِ گوناگون به پروتکل‌های یادگیریِ فدرال را بحث می‌کنند. نارایانان و همکاران (2011) توصیف می‌کنند چگونه توانستند با خزش در سایتی که داده‌ها از آن به‌دست آمده بود (فلیکر) و تطبیقِ گره‌های با درجهٔ ورودی یا خروجیِ به‌طورِ غیرعادی بالا میانِ داده‌های ارائه‌شده و داده‌های خزش‌شده، گرافِ پیوندِ مبهم‌شده را از چالشِ شبکهٔ اجتماعیِ 2011 ابهام‌زدایی کنند. این به آن‌ها اجازه داد اطلاعاتِ اضافی برای بردنِ چالش به‌دست آورند و همچنین هویتِ واقعیِ گره‌ها را در داده‌ها آشکار سازند. ابزارهایی برای حریمِ خصوصیِ کاربر در حالِ در دسترس قرار گرفتن هستند؛ برای نمونه، تنسورفلو مدول‌هایی برای یادگیریِ فدرال و حریمِ خصوصی فراهم می‌کند (مک‌ماهان و اندرو، 2018).

انصاف: کتابِ «سلاح‌های ریاضیاتِ مخرب» از کتی اونیل (2017) توصیف می‌کند که چگونه مدل‌های گوناگونِ جعبه‌سیاه یادگیریِ ماشینی بر زندگیِ ما تأثیر می‌گذارند، اغلب به شیوه‌هایی ناعادلانه. او از سازندگانِ مدل می‌خواهد مسئولیتِ انصاف را بپذیرند، و از سیاست‌گذاران می‌خواهد مقرراتِ مناسب وضع کنند. دورک و همکاران (2012) کاستی‌های رویکردِ ساده‌لوحانهٔ «انصاف از طریقِ ناآگاهی» را نشان دادند. بلامی و همکاران (2018) جعبه‌ابزاری برای کاهشِ اریب در سامانه‌های یادگیریِ ماشینی ارائه می‌دهند. ترامر و همکاران (2016) نشان می‌دهند چگونه هماورد می‌تواند مدلی از یادگیریِ ماشینی را با ارسالِ پرس‌وجوها به API «بدزدد». هارت و همکاران (2017) برابریِ فرصت را به‌عنوانِ معیاری برای انصاف توصیف می‌کنند. چولدکوا و راث (2018) مروری بر مرزهای دانشِ انصاف ارائه می‌دهند، و ورما و روبین (2018) پیمایشی جامع از تعاریفِ انصاف ارائه می‌کنند. کلاینبرگ و همکاران (2016) نشان می‌دهند که در حالتِ کلی، الگوریتم نمی‌تواند هم به‌خوبی کالیبره‌شده باشد و هم برابریِ فرصت داشته باشد. برک و همکاران (2017) چند تعریفِ اضافی از انواعِ انصاف ارائه می‌دهند و دوباره نتیجه می‌گیرند که برآورده‌کردنِ همهٔ جنبه‌ها به‌طورِ هم‌زمان ناممکن است. بویتل و همکاران (2019) توصیه‌هایی برای نحوهٔ عملی‌کردنِ معیارهای انصاف ارائه می‌دهند.

درسل و فرید (2018) دربارهٔ مدلِ امتیازدهیِ تکرارِ جرمِ COMPAS گزارش می‌دهند. کریستین و همکاران (2015) و اک‌هاوس و همکاران (2019) استفاده از الگوریتم‌های پیش‌بینانه در نظامِ قضایی را بحث می‌کنند. کوربت-دیویس و همکاران (2017) نشان می‌دهند که تنشی میانِ تضمینِ انصاف و بهینه‌سازیِ امنیتِ عمومی وجود دارد، و کوربت-دیویس و گوئل (2018) تفاوت‌های میانِ چارچوب‌های انصاف را بحث می‌کنند. چولدکوا (2017) از تأثیرِ منصفانه دفاع می‌کند: همهٔ رده‌ها باید مطلوبیتِ موردِ انتظارِ یکسانی داشته باشند. لیو و همکاران (2018a) از معیاری بلندمدت برای تأثیر دفاع می‌کنند و اشاره دارند که برای نمونه، اگر نقطهٔ تصمیم‌گیری برای تصویبِ وام را تغییر دهیم تا در کوتاه‌مدت منصفانه‌تر باشیم، این می‌تواند در بلندمدت اثری منفی بر افرادی بگذارد که در نهایت در بازپرداختِ وام نکول می‌کنند و

بدین‌ترتیب امتیازِ اعتباری‌شان کاهش می‌یابد. از 2014 کنفرانسِ سالانه‌ای دربارهٔ «انصاف، پاسخگویی و شفافیت در یادگیریِ ماشینی» (FAT*) برگزار می‌شود. محرابی و همکاران (2019) مروری جامع بر اریب و انصاف در یادگیریِ ماشینی ارائه می‌دهند و ۲۳ نوعِ اریب و ۱۰ تعریفِ انصاف را فهرست‌بندی می‌کنند.

اعتماد: هوش مصنوعیِ تبیین‌پذیر موضوعِ مهمی بود که به روزهای اولیهٔ سامانه‌های خبره بازمی‌گردد (نچس و همکاران، 1985)، و در سال‌های اخیر دوباره ظهور کرده (بیران و کاتن، 2017؛ میلر و همکاران، 2017؛ کیم، 2018). بارنو و همکاران (2010) رده‌بندیِ انواعِ حملاتِ امنیتیِ قابلِ‌انجام بر سامانهٔ یادگیریِ ماشینی می‌دهند، و تایگار (2011) یادگیریِ ماشینیِ تخاصمی را مرور می‌کند. پژوهشگرانِ IBM پیشنهادی برای کسبِ اعتماد در سامانه‌های هوش مصنوعی از طریقِ اعلامیه‌های انطباق دارند (هیند و همکاران، 2018). DARPA برای سامانه‌های میدانِ نبردش تصمیم‌های تبیین‌پذیر ایجاب می‌کند، و فراخوانی برای پژوهش در این حوزه صادر کرده (گانینگ، 2016).

ایمنیِ هوش مصنوعی: کتابِ «ایمنی و امنیتِ هوش مصنوعی» (یامپولسکی، 2018) مقاله‌هایی دربارهٔ ایمنیِ هوش مصنوعی، هم تازه و هم کلاسیک، جمع می‌کند، که به «چرا آینده به ما نیاز ندارد»ی بیل جوی (2000) بازمی‌گردد. «مسئلهٔ شاهِ میداس» توسط ماروین مینسکی پیش‌بینی شده بود، که زمانی پیشنهاد کرد برنامهٔ هوش مصنوعیِ طراحی‌شده برای حلِ فرضیهٔ ریمان ممکن است در نهایت همهٔ منابعِ زمین را برای ساختنِ ابررایانه‌های قدرتمندتر تصاحب کند. به‌همین‌سان، اوموهوندر (2008) برنامهٔ شطرنجی را پیش‌بینی می‌کند که منابع را می‌رباید، و بسترم (2014) کارخانهٔ گیرهٔ کاغذِ از-کنترل‌خارج‌شده‌ای را توصیف می‌کند که جهان را تصاحب می‌کند. یودکوفسکی (2008) با جزئیاتِ بیشتری دربارهٔ نحوهٔ طراحیِ هوش مصنوعیِ دوستانه بحث می‌کند. آمودی و همکاران (2016) پنج مسئلهٔ عملیِ ایمنی برای سامانه‌های هوش مصنوعی ارائه می‌کنند. اوموهوندر (2008) «سوق‌های پایهٔ هوش مصنوعی» را توصیف می‌کند و نتیجه می‌گیرد «ساختارهای اجتماعی که باعث می‌شوند افراد هزینهٔ برون‌زایی‌های منفی‌شان را بپردازند، راهِ طولانی به سمتِ تضمینِ آینده‌ای پایدار و مثبت خواهد رفت.» «حکمرانیِ منابعِ مشترک»ی الینور اوستروم (1990) رویه‌هایی برای برخورد با برون‌زایی‌ها توسط فرهنگ‌های سنتی توصیف می‌کند. اوستروم همچنین این رویکرد را بر ایدهٔ دانش به‌عنوانِ منبعِ مشترک اعمال کرده (هس و اوستروم، 2007). نمونه‌های بازی با مشخصات در محیط‌های شبیه‌سازی‌شده (آمودی و همکاران، 2016) و بازی‌های ویدئویی (کلارک و آمودی، 2016) مستند شده‌اند.

آیندهٔ کار و اتوماسیون: بحثِ آیندهٔ کار با اتوماسیون سابقه‌ای طولانی دارد. کینز (1930) اصطلاحِ «بیکاریِ فناورانه» را ابداع کرد. برین‌یولفسون و مک‌آفی (2011, 2014) استدلال می‌کنند فناوری‌های دیجیتال و هوش مصنوعی موجِ تازه‌ای از اتوماسیون ایجاد می‌کنند که پیامدهای عمیقی برای اشتغال دارد. فرای و آزبرن (2017) برآورد می‌کنند ۴۷٪ از مشاغلِ آمریکا در معرضِ خودکارشدن‌اند. بسن (2015) استدلال می‌کند کارگران تا وقتی مهارت‌های لازم برای استفاده از فناوری‌های تازه را نیاموزند، از مزایای آن‌ها بهره‌مند نخواهند شد. مک‌کینزی گلوبال اینستیتوت (2017) برآورد می‌کند تا 2030 تا ۳۷۵ میلیون کارگر ممکن است نیاز به تغییرِ شغل داشته باشند. «درآمدِ پایهٔ همگانی» (UBI) به‌عنوانِ راه‌حلِ بالقوه‌ای برای توزیعِ مزایای اتوماسیون بحث شده (بانیارد و همکاران، 2019).

حقوقِ ربات‌ها: کتابِ ویراستهٔ یوریک ویلکس (2010) دیدگاه‌های متفاوتی دربارهٔ اینکه چگونه باید با همراهانِ مصنوعی برخورد کنیم می‌دهد، از دیدگاهِ جوانا برایسون که ربات‌ها باید به‌عنوانِ ابزار به ما خدمت کنند نه شهروند، تا مشاهدهٔ شری ترکل که ما از قبل رایانه‌ها و دیگر ابزارهایمان را جان‌بخشی می‌کنیم و کاملاً مایلیم مرزهای بینِ ماشین و زندگی را محو کنیم. ویلکس همچنین به‌روزرسانیِ اخیری از دیدگاه‌هایش ارائه کرده (ویلکس، 2019). فیلسوفِ دیوید

گونکل در کتابِ «حقوقِ ربات» (2018) چهار امکان را بررسی می‌کند: آیا ربات‌ها می‌توانند حقوق داشته باشند یا نه، و آیا باید داشته باشند یا نه؟ «انجمنِ آمریکاییِ پیشگیری از ظلم به ربات‌ها» (ASPCR) اعلام می‌کند «ASPCR هست و خواهد بود، دقیقاً به همان اندازه که ربات‌ها احساس دارند جدی.»

آیندهٔ کار: در 1888، ادوارد بلامی پرفروش‌ترینِ «نگاه به عقب» را منتشر کرد، که پیش‌بینی می‌کرد تا سالِ 2000، پیشرفت‌های فناورانه به مدینهٔ فاضله‌ای می‌انجامد که برابری محقق می‌شود و مردم ساعت‌های کوتاه کار می‌کنند و زود بازنشسته می‌شوند. اندکی بعد، ای. ام. فورستر دیدگاهِ ویران‌شهری را در «ماشین می‌ایستد» (1909) گرفت، که در آن ماشینِ خیرخواهانه‌ای ادارهٔ جامعه را بر عهده می‌گیرد؛ وقتی ماشین ناگزیر از کار می‌افتد، همه‌چیز از هم می‌پاشد. کتابِ آینده‌نگرِ نوربرت وینر «استفادهٔ انسانی از انسان‌ها» (1950) برای مزایای اتوماسیون در رهاییِ مردم از کارِ طاقت‌فرسا در حالی که کارِ خلاق‌تری ارائه می‌دهد استدلال می‌کند، اما همچنین چند خطر را بحث می‌کند که امروزه آن‌ها را به‌عنوانِ مسئله می‌شناسیم، به‌ویژه مسئلهٔ همراستاییِ ارزش. کتابِ «اختلالِ بیکاری» (نوردفورس و همکاران، 2018) برخی از راه‌هایی را بحث می‌کند که کار در حالِ تغییر است و فرصت‌هایی برای مشاغلِ تازه باز می‌کند. اریک برین‌یولفسون و اندرو مک‌آفی این مضامین و بیشتر را در کتاب‌های «مسابقه با ماشین» (2011) و «عصرِ ماشینِ دوم» (2014) بررسی می‌کنند. فورد (2015) چالش‌های اتوماسیونِ فزاینده را توصیف می‌کند، و وست (2018) توصیه‌هایی برای کاهشِ مشکلات ارائه می‌کند، در حالی که توماس مالونِ (2004) MIT نشان می‌دهد بسیاری از همان مسائل یک دهه زودتر نیز آشکار بودند، اما در آن زمان به شبکه‌های ارتباطیِ جهانی نسبت داده می‌شدند، نه به اتوماسیون.

تکینگی: ایدهٔ تکینگی توسط گود (1965b) و وینگ (1993) رواج یافت. ری کورزویل (2005) اعلام کرد «تکینگی نزدیک است»، و یک دهه بعد موری شاناهان (2015) به‌روزرسانی‌ای دربارهٔ موضوع داد. هم‌بنیان‌گذارِ مایکروسافت، پل آلن، با «تکینگی نزدیک نیست» (2011) مقابله کرد. او امکانِ ماشین‌های فراهوشمند را رد نکرد؛ فقط فکر می‌کرد بیش از یک قرن طول می‌کشد تا به آن‌جا برسیم. راد بروکس منتقدِ مکررِ تکینگی‌گرایی است؛ اشاره می‌کند فناوری‌ها اغلب بیش از آنچه پیش‌بینی می‌شود طول می‌کشند تا بالغ شوند، ما مستعدِ تفکرِ جادویی‌ایم، و نمایی‌ها برای همیشه دوام نمی‌آورند (بروکس، 2017). از سوی دیگر، برای هر تکینگی‌گرایِ خوش‌بین، بدبینی هست که از فناوریِ تازه می‌ترسد. وب‌سایتِ pessimists.co نشان می‌دهد این در طولِ تاریخ صادق بوده: برای نمونه، در دههٔ 1890 مردم نگران بودند آسانسور ناگزیر حالتِ تهوع ایجاد کند، تلگراف به ازدست‌رفتنِ حریمِ خصوصی و فسادِ اخلاقی بینجامد، مترو هوایِ خطرناکِ زیرزمینی آزاد کند و مردگان را برنجاند، و دوچرخه—به‌ویژه ایدهٔ سواریِ زن بر آن—کارِ شیطان باشد. با این حال، برخی فناوری‌ها واقعاً خطرناک بوده‌اند و نیازمندِ مقرراتِ دقیق‌اند. هوش مصنوعی بدونِ شک در این رده قرار خواهد گرفت.

فراانسان‌گرایی: هانس موراوس (2000) برخی از ایده‌های فراانسان‌گرایی را معرفی می‌کند، و بسترم (2005) تاریخچهٔ به‌روزشده‌ای ارائه می‌دهد. ایدهٔ ماشینِ فراهوشمندِ گود صد سال زودتر در «داروین در میانِ ماشین‌ها»ی ساموئل باتلر (1863) پیش‌بینی شده بود. باتلر که چهار سال پس از انتشارِ «دربارهٔ خاستگاهِ گونه‌ها»ی چارلز داروین و در زمانی نوشت که پیچیده‌ترین ماشین‌ها موتورهای بخار بودند، «توسعهٔ نهاییِ آگاهیِ مکانیکی» را از طریقِ انتخابِ طبیعی تصور کرد. این مضمون توسط جورج دایسون (1998) در کتابی با همان عنوان تکرار شد، و توسط آلن تورینگ ارجاع شد، که در 1951 نوشت «در مرحله‌ای بنابراین باید انتظار داشت ماشین‌ها کنترل را به‌شیوه‌ای که در «اروون»ی ساموئل باتلر ذکر شده به دست بگیرند» (تورینگ، 1996).

پانوشت‌ها:

[1] برای نمونه، اپرایِ کوپلیا (1870)، رمانِ «آیا آدم‌های مصنوعی رویایِ گوسفندِ برقی می‌بینند؟» (1968)، فیلم‌های AI (2001)، وال-ای (2008) و او (2013).

[2] از جملهِ دو نویسندهٔ همین کتاب.

[3] در 2010، اگرچه فقط ۲٪ از نیرویِ کارِ آمریکا کشاورزِ واقعی بودند، بیش از ۲۵٪ از جمعیت (۸۰ میلیون نفر) دستِ‌کم یک بار بازیِ فارمویل را انجام دادند.

[4] نویسندگانِ داستان‌های علمی-تخیلی به‌طورِ گسترده‌ای موافق‌اند ربات‌ها در حلِ موقعیت‌های مبهمِ اخلاقی بسیار بدند؛ علاوه بر داستانِ «دورِ گردان»، فیلم‌های «من، ربات» (2004) و «ای.آی.» (2001) همین موضوع را بررسی می‌کنند.

فصل ۲۸: آیندهٔ هوش مصنوعی (The Future of AI)

در این فصل می‌کوشیم فاصله‌ای کوتاه به جلو را ببینیم.

در فصل ۲، تصمیم گرفتیم هوش مصنوعی را به‌عنوانِ تکلیفِ طراحیِ عامل‌های تقریباً عقلایی ببینیم. انواعِ گوناگونی از طراحی‌های عامل در نظر گرفته شد، از عامل‌های بازتابی تا عامل‌های دانش‌محورِ تصمیم‌نظری تا عامل‌های یادگیریِ عمیقِ استفاده‌کننده از یادگیریِ تقویتی. همچنین در فناوری‌های مؤلفه‌ای که این طراحی‌ها از آن‌ها مونتاژ می‌شوند تنوع هست: استدلالِ منطقی، احتمالاتی یا عصبی؛ بازنمایی‌های اتمی، عامل‌بندی‌شده یا ساخت‌یافتهٔ حالت‌ها؛ الگوریتم‌های یادگیریِ گوناگون از انواعِ گوناگونِ داده؛ حس‌گذارها و عملگرها برای تعامل با جهان. سرانجام، انواعِ گوناگونی از کاربردها دیده‌ایم، در پزشکی، مالی، حمل‌ونقل، ارتباطات و دیگر حوزه‌ها. در همهٔ این جبهه‌ها پیشرفت بوده، هم در فهمِ علمیِ ما و هم در قابلیت‌های فناورانه‌مان.

بیشترِ کارشناسان دربارهٔ پیشرفتِ ادامه‌دار خوش‌بین‌اند؛ چنان‌که در صفحهٔ ۲۸ دیدیم، برآوردِ میانه برای هوش مصنوعیِ تقریباً سطح-انسان در طیفِ وسیعی از تکلیف‌ها جایی در ۵۰ تا ۱۰۰ سالِ آینده است. در دههٔ آینده، پیش‌بینی می‌شود هوش مصنوعی هر سال تریلیون‌ها دلار به اقتصاد بیفزاید. اما چنان‌که همچنین دیدیم، منتقدانی هستند که فکر می‌کنند هوش مصنوعیِ عمومی قرن‌ها فاصله دارد، و نگرانی‌های اخلاقیِ متعددی دربارهٔ انصاف، برابری و کشندگیِ هوش مصنوعی هست. در این فصل، می‌پرسیم: به کجا می‌رویم و چه کاری باقی مانده؟ این کار را با پرسیدنِ اینکه آیا مؤلفه‌ها، معماری‌ها و هدف‌های درستی داریم تا هوش مصنوعی را به فناوریِ موفقِ سودرساننده به جهان تبدیل کنیم انجام می‌دهیم.

۲۸.۱ مؤلفه‌های هوش مصنوعی

این بخش مؤلفه‌های سامانه‌های هوش مصنوعی و میزانی را بررسی می‌کند که هر یک از آن‌ها ممکن است پیشرفتِ آینده را شتاب یا کند کند.

حس‌گذارها و عملگرها

برای بخشِ بزرگی از تاریخِ هوش مصنوعی، دسترسیِ مستقیم به جهان به‌شکلِ آشکاری غایب بود. با چند استثنای قابلِ توجه، سامانه‌های هوش مصنوعی چنان ساخته می‌شدند که انسان‌ها باید ورودی‌ها را تأمین و خروجی‌ها را تفسیر می‌کردند. در این میان، سامانه‌های رباتیک بر تکلیف‌های سطح‌پایینی تمرکز می‌کردند که استدلال و برنامه‌ریزیِ سطح‌بالا در آن‌ها تا حدِ زیادی نادیده گرفته می‌شد و نیاز به ادراک کمینه می‌شد. این تا حدی به‌دلیلِ هزینهٔ زیاد و تلاشِ مهندسیِ لازم برای اینکه ربات‌های واقعی اصلاً کار کنند بود، و تا حدی به‌دلیلِ فقدانِ قدرتِ پردازشِ کافی و الگوریتم‌های به‌اندازهٔ کافی مؤثر برای مدیریتِ ورودیِ بصریِ پهن‌باندِ بالا.

اوضاع در سال‌های اخیر با در دسترس‌بودنِ ربات‌های برنامه‌پذیرِ آماده به‌سرعت تغییر کرده. این‌ها، به‌نوبهٔ خود، از درایوهای موتورِ فشردهٔ قابلِ‌اعتماد و حس‌گذارهای بهبودیافته بهره برده‌اند. هزینهٔ لیدار برای خودروی خودران از ۷۵,۰۰۰ دلار به ۱,۰۰۰ دلار سقوط کرده، و نسخهٔ تک‌تراشه‌ای ممکن است به ۱۰ دلار به‌ازایِ هر واحد برسد (پولتون و واتس، 2016). حس‌گذارهای رادار، که زمانی فقط قادر به تشخیصِ درشت‌دانه بودند، اکنون به‌اندازهٔ کافی حساس‌اند که تعدادِ ورق‌های یک دسته کاغذ را بشمارند (یئو و همکاران، 2018).

تقاضا برای پردازشِ تصویرِ بهتر در دوربین‌های گوشیِ همراه، دوربین‌های باکیفیتِ ارزانی برای استفاده در رباتیک به ما داده. فناوریِ MEMS (سامانه‌های میکرو-الکترومکانیکی) شتاب‌سنج‌ها، ژیروسکوپ‌ها و عملگرهای ریزپردازی‌شده‌ای فراهم کرده که به‌اندازهٔ کافی کوچک‌اند تا در حشراتِ مصنوعیِ پرنده جا شوند (فلوریانو و همکاران، 2009؛ فولر و همکاران، 2014). ممکن است بتوان میلیون‌ها دستگاهِ MEMS را برای تولیدِ عملگرهای ماکروسکوپیِ قدرتمند ترکیب کرد. چاپِ سه‌بعدی (موث و همکاران، 2014) و چاپِ زیستی (کولسکی و همکاران، 2014) آزمایش با نمونه‌های اولیه را آسان‌تر کرده‌اند.

پس، می‌بینیم سامانه‌های هوش مصنوعی در آستانهٔ حرکت از سامانه‌های عمدتاً فقط-نرم‌افزاری به سامانه‌های رباتیکِ جاسازی‌شدهٔ مفیدند. وضعیتِ رباتیکِ امروز تقریباً قابلِ مقایسه با وضعیتِ رایانه‌های شخصی در اوایلِ دههٔ ۱۹۸۰ است: در آن زمان رایانه‌های شخصی در حالِ در دسترس‌شدن بودند، اما یک دههٔ دیگر طول می‌کشید تا رایج شوند. محتمل است ربات‌های منعطف و هوشمند نخست در صنعت گام‌های بلند بردارند (جایی که محیط‌ها کنترل‌شده‌ترند، تکلیف‌ها تکراری‌ترند، و ارزشِ سرمایه‌گذاری آسان‌تر اندازه‌گیری می‌شود) پیش از بازارِ خانگی (جایی که تغییرپذیریِ بیشتری در محیط و تکلیف‌ها هست).

بازنماییِ حالتِ جهان

پیگیریِ جهان مستلزمِ ادراک و همچنین به‌روزرسانیِ بازنمایی‌های درونی است. فصل ۴ نشان داد چگونه بازنمایی‌های حالتِ اتمی را پیگیری کنیم؛ فصل ۷ توصیف کرد چگونه آن را برای بازنمایی‌های حالتِ عامل‌بندی‌شده (گزاره‌ای) انجام دهیم؛ فصل ۱۰ این را به منطقِ مرتبهٔ اول بسط داد؛ و فصل ۱۴ استدلالِ احتمالاتی را در طولِ زمان در محیط‌های نامطمئن توصیف کرد. فصل ۲۱ شبکه‌های عصبیِ بازگشتی را معرفی کرد، که همچنین قادر به نگه‌داشتنِ بازنماییِ حالت در طولِ زمان‌اند.

الگوریتم‌های فیلترکردن و ادراکِ کنونی را می‌توان ترکیب کرد تا کارِ معقولی در بازشناسیِ اشیا («آن گربه است») و گزارشِ گزاره‌های سطح‌پایین («فنجان رویِ میز است») انجام دهد. بازشناسیِ کنش‌های سطح‌بالاتر، مانندِ «دکتر راسل با دکتر نورویگ در حالِ نوشیدنِ چای است و دربارهٔ برنامه‌های هفتهٔ آینده بحث می‌کند»، دشوارتر است. در حالِ حاضر، گاهی می‌توان آن را انجام داد (شکل ۲۵.۱۷ در صفحهٔ ۹۰۸ را ببینید) با توجه به مثال‌های آموزشیِ کافی، اما پیشرفتِ آینده به فنونی نیاز خواهد داشت که بدونِ نیاز به مثال‌های جامع به موقعیت‌های تازه تعمیم می‌کنند (پوپه، 2010؛ کانگ و وایلز، 2016).

مشکلِ دیگر این است که هرچند الگوریتم‌های فیلترکردنِ تقریبیِ فصل ۱۴ می‌توانند محیط‌های نسبتاً بزرگ را مدیریت کنند، آن‌ها همچنان با بازنماییِ عامل‌بندی‌شده سروکار دارند—متغیرهای تصادفی دارند، اما اشیا و رابطه‌ها را به‌صورتِ صریح بازنمایی نمی‌کنند. همچنین، مفهومِ زمانِ آن‌ها به تغییرِ گام‌به‌گام محدود است؛ با توجه به مسیرِ اخیرِ توپ، می‌توانیم پیش‌بینی کنیم در زمانِ $t+1$ کجا خواهد بود، اما بازنماییِ ایدهٔ انتزاعیِ «هرچه بالا رود پایین می‌آید» دشوار است. بخش ۱۵.۱ توضیح داد چگونه احتمال و منطقِ مرتبهٔ اول را می‌توان برای حلِ این مسائل ترکیب کرد؛ بخش ۱۵.۲ نشان داد چگونه با عدم‌قطعیت دربارهٔ هویتِ اشیا سروکار کنیم؛ و فصل ۲۵ نشان داد چگونه شبکه‌های عصبیِ بازگشتی بینماییِ رایانه را قادر به ردیابیِ جهان می‌کنند؛ اما هنوز راهِ خوبی برای کنارِ هم گذاشتنِ همهٔ این فنون نداریم.

فصل ۲۴ نشان داد چگونه جاسازیِ واژه و بازنمایی‌های مشابه می‌توانند ما را از مرزهای سختِ مفاهیمِ تعریف‌شده با شرایطِ لازم و کافی رها کنند. تعریفِ طرح‌های بازنماییِ عمومی و قابلِ استفادهٔ مجدد برای حوزه‌های پیچیده همچنان تکلیفی دلهره‌آور است.

انتخابِ کنش‌ها

دشواریِ اصلی در انتخابِ کنش در جهانِ واقعی کنارآمدن با «برنامه‌های بلندمدت»—مانندِ فارغ‌التحصیل‌شدن از کالج در چهار سال—است که شاملِ میلیاردها گامِ ابتدایی‌اند.

الگوریتم‌های جستجویی که دنباله‌های کنشِ ابتدایی را در نظر می‌گیرند فقط تا ده‌ها یا شاید صدها گام مقیاس می‌شوند. فقط با تحمیلِ ساختارِ سلسله‌مراتبی بر رفتار است که ما انسان‌ها اصلاً کنار می‌آییم. در بخش ۱۱.۴ دیدیم چگونه از بازنمایی‌های سلسله‌مراتبی برای مدیریتِ مسائلِ این مقیاس استفاده کنیم؛ افزون بر این، کار در یادگیریِ تقویتیِ سلسله‌مراتبی در ترکیبِ این ایده‌ها با فرمالیسمِ MDP توصیف‌شده در فصل ۱۷ موفق بوده.

در حالِ حاضر، این روش‌ها به حالتِ با مشاهدهٔ جزئی (POMDPها) بسط نیافته‌اند. افزون بر این، الگوریتم‌های حلِ POMDP معمولاً از همان بازنماییِ حالتِ اتمیِ استفاده‌شده برای الگوریتم‌های جستجویِ فصل ۳ استفاده می‌کنند. به‌وضوح کارِ زیادی این‌جا برای انجام هست، اما بنیان‌های فنی تا حدِ زیادی برای پیشرفت فراهم‌اند. عنصرِ گمشدهٔ اصلی روشی مؤثر برای ساختِ بازنمایی‌های سلسله‌مراتبیِ حالت و رفتار است که برای تصمیم‌گیری در بازه‌های زمانیِ بلند لازم‌اند.

تصمیم‌گیری دربارهٔ آنچه می‌خواهیم

فصل ۳ الگوریتم‌های جستجو را برای یافتنِ حالتِ هدف معرفی کرد. اما عامل‌های هدف‌محور وقتی محیط نامطمئن است و عواملِ متعددی برای درنظرگرفتن هست شکننده‌اند. در اصل، عامل‌های بیشینه‌کنندهٔ مطلوبیت آن مسائل را به‌شیوه‌ای کاملاً کلی بررسی می‌کنند. حوزه‌های اقتصاد و نظریهٔ بازی، و همچنین هوش مصنوعی، از این بینش استفاده می‌کنند: فقط اعلام کن می‌خواهی چه چیزی را بهینه کنی و هر کنش چه می‌کند، و می‌توانیم کنشِ بهینه را محاسبه کنیم.

در عمل، با این حال، اکنون می‌فهمیم تکلیفِ انتخابِ تابعِ مطلوبیتِ درست به‌خودیِ‌خود مسئلهٔ چالش‌برانگیزی است. برای نمونه، شبکهٔ پیچیدهٔ ترجیح‌های متعامل را تصور کن که عاملی که به‌عنوانِ دستیارِ اداری برای انسان عمل می‌کند باید بفهمد. مسئله با این واقعیت تشدید می‌شود که هر انسان متفاوت است، پس عاملی که تازه «خارج از جعبه» آمده تجربهٔ کافی با هیچ فردِ تکی‌ای برای یادگیریِ مدلِ ترجیحِ دقیق نخواهد داشت؛ لزوماً باید تحتِ عدمِ‌قطعیتِ ترجیح عمل کند. پیچیدگیِ بیشتر اگر بخواهیم مطمئن شویم عامل‌هایمان به‌گونه‌ای عمل می‌کنند که برای جامعه منصفانه و عادلانه است، نه فقط برای فردِ تکی، پیش می‌آید.

هنوز تجربهٔ چندانی در ساختنِ مدل‌های ترجیحِ پیچیدهٔ جهانِ واقعی نداریم، چه رسد به توزیع‌های احتمالِ رویِ چنین مدل‌هایی. هرچند فرمالیسم‌های عامل‌بندی‌شده‌ای، شبیهِ شبکه‌های بیزی، هست که برای تجزیهٔ ترجیح‌ها رویِ حالت‌های پیچیده در نظر گرفته شده‌اند، استفاده از این فرمالیسم‌ها در عمل دشوار ثابت شده. یک دلیل ممکن است این باشد که ترجیح‌ها رویِ حالت‌ها واقعاً از ترجیح‌ها رویِ تاریخچه‌های حالت، که توسط توابعِ پاداش توصیف می‌شوند (فصل ۱۷ را ببینید)، کامپایل می‌شوند. حتی اگر تابعِ پاداش ساده باشد، تابعِ مطلوبیتِ متناظر ممکن است بسیار پیچیده باشد.

این پیشنهاد می‌کند تکلیفِ «مهندسیِ دانش برای توابعِ پاداش» را جدی بگیریم، به‌عنوانِ راهی برای انتقالِ آنچه می‌خواهیم عامل‌هایمان انجام دهند. ایدهٔ یادگیریِ تقویتیِ معکوس (بخش ۲۲.۶) یک رویکرد به این مسئله است وقتی کارشناسی داریم که می‌تواند تکلیفی را انجام دهد، اما نه آن را تبیین کند. همچنین می‌توانستیم از زبان‌های بهتری برای بیانِ آنچه می‌خواهیم استفاده کنیم. برای نمونه، در رباتیک، «منطقِ زمانیِ خطی» گفتنِ اینکه چه چیزهایی می‌خواهیم در آیندهٔ نزدیک رخ دهند، از چه چیزهایی می‌خواهیم اجتناب کنیم، و چه حالت‌هایی می‌خواهیم برای همیشه ادامه یابند را آسان‌تر می‌کند (لیتمن و همکاران، 2017). به راه‌های بهتری برای گفتنِ آنچه می‌خواهیم و راه‌های بهتری برای تفسیرِ خواسته‌هایمان توسط ربات‌ها نیاز داریم.

صنعتِ رایانه به‌عنوانِ کل اکوسیستمِ قدرتمندی برای تجمیعِ ترجیح‌های کاربر توسعه داده. وقتی رویِ چیزی در برنامه‌ای، بازیِ برخط، شبکهٔ اجتماعی یا سایتِ خرید کلیک می‌کنید، آن به‌عنوانِ توصیه‌ای خدمت می‌کند که شما (و هم‌تایانِ مشابه‌تان) دوست دارید در آینده چیزهای مشابهی ببینید. (یا شاید سایت گیج‌کننده است و رویِ چیزِ اشتباهی کلیک کرده‌اید—داده همیشه نویزی است.) بازخوردِ نهفته در این سامانه آن را در کوتاه‌مدت برای برگزیدنِ بازی‌ها و ویدئوهای روزبه‌روز اعتیادآورتر بسیار مؤثر می‌کند.

اما این سامانه‌ها اغلب در فراهم‌کردنِ راهی آسان برای انصراف ناکام‌اند—دستگاهِ شما ویدئویِ مرتبطی را خودکار پخش می‌کند، اما احتمالِ کمتری دارد به شما بگوید «شاید وقتش است دستگاه‌ها را کنار بگذاری و قدمی آرام‌بخش در طبیعت بزنی.» سایتِ خرید به شما کمک می‌کند لباس‌هایی بیابید که با سبکِ شما می‌خواند، اما به صلحِ جهانی یا پایانِ گرسنگی و فقر نخواهد پرداخت. تا جایی که منویِ انتخاب‌ها توسط شرکت‌هایی رانده می‌شود که می‌خواهند از توجهِ مشتری سود ببرند، منو ناقص باقی خواهد ماند. با این حال، شرکت‌ها به علایقِ مشتریان پاسخ می‌دهند، و بسیاری از مشتریان این نظر را ابراز کرده‌اند که به جهانی منصفانه و پایدار علاقه‌مندند. تیم اوریلی دلیل اینکه سود تنها انگیزه نیست را با قیاسِ زیر تبیین می‌کند: «پول مانندِ بنزین در سفرِ جاده‌ای است. نمی‌خواهی در سفرت بنزینت تمام شود، اما تورِ پمپ‌بنزین‌ها هم نمی‌روی. باید به پول توجه کنی، اما نباید دربارهٔ پول باشد.»

جنبشِ «زمانِ خوب‌گذرانده»ی تریستان هریس در مرکزِ فناوریِ انسانی گامی به سمتِ دادنِ انتخاب‌های کامل‌تر به ماست (هریس، 2016). این جنبش به مسئله‌ای می‌پردازد که هربرت سایمون در 1971 بازشناسی کرد: «ثروتِ اطلاعات فقرِ توجه ایجاد می‌کند.» شاید در آینده «عاملِ شخصی»ی (Personal Agent) داشته باشیم که از منافعِ واقعیِ بلندمدتِ ما دفاع می‌کند، نه منافعِ شرکت‌هایی که برنامه‌هایشان اکنون دستگاه‌های ما را پر کرده. کارِ عامل میانجی‌گریِ پیشنهادهای فروشنده‌های گوناگون، محافظت از ما در برابرِ جلب‌کننده‌های توجهِ اعتیادآور، و هدایتِ ما به سمتِ هدف‌هایی خواهد بود که واقعاً برایمان مهم‌اند.

یادگیری

فصل‌های ۱۹ تا ۲۲ توصیف کردند چگونه عامل‌ها می‌توانند یاد بگیرند. الگوریتم‌های کنونی می‌توانند با مسائلِ نسبتاً بزرگ کنار بیایند و در بسیاری از تکلیف‌ها به قابلیت‌های انسانی برسند یا از آن فراتر روند—تا زمانی که مثال‌های آموزشیِ کافی داشته باشیم و با واژگانِ ازپیش‌تعریف‌شده‌ای از ویژگی‌ها و مفاهیم سروکار داشته باشیم. اما یادگیری وقتی داده تنک است، یا نظارت‌نشده است، یا با بازنمایی‌های پیچیده سروکار داریم، می‌تواند متوقف شود. بخشِ زیادی از احیایِ اخیرِ هوش مصنوعی در مطبوعاتِ عمومی و صنعت به‌دلیلِ موفقیتِ یادگیری عمیق (فصل ۲۱) است. از یک سو، می‌توان این را بلوغِ تدریجیِ زیرحوزهٔ شبکه‌های عصبی دید. از سوی دیگر، می‌توانیم آن را جهشی انقلابی در قابلیت‌ها ببینیم که توسط هم‌گراییِ عوامل زیر برانگیخته شده: در دسترس‌بودنِ دادهٔ آموزشیِ بیشتر به‌لطفِ اینترنت، قدرتِ پردازشِ افزایش‌یافته از سخت‌افزارِ تخصصی، و چند ترفندِ الگوریتمی، مانندِ شبکه‌های مولدِ تخاصمی (GAN)، نرمال‌سازیِ دسته‌ای، دراپ‌اوت و تابعِ فعال‌سازیِ خطیِ تصحیح‌شده (ReLU).

آینده باید تأکیدِ ادامه‌دار بر بهبودِ یادگیری عمیق برای تکلیف‌هایی که در آن‌ها عالی است، و همچنین بسطِ آن برای پوششِ تکلیف‌های دیگر ببیند. نامِ تجاریِ «یادگیری عمیق» چنان محبوب ثابت شده که باید انتظارِ ادامهٔ استفاده‌اش را داشته باشیم، حتی اگر ترکیبِ فنونی که آن را تغذیه می‌کنند به‌شکلِ چشمگیری تغییر کند.

ظهورِ حوزهٔ «علمِ داده» را به‌عنوانِ هم‌گراییِ آمار، برنامه‌نویسی و تخصصِ حوزه دیده‌ایم. در حالی که می‌توانیم انتظارِ توسعهٔ ادامه‌دارِ ابزارها و فنونِ لازم برای کسب، مدیریت و نگه‌داریِ دادهٔ بزرگ را داشته باشیم، همچنین به پیشرفت‌هایی در یادگیریِ انتقالی نیاز خواهیم داشت تا بتوانیم از دادهٔ یک حوزه برای بهبودِ عملکرد در حوزهٔ مرتبط بهره بگیریم.

اکثریتِ قریبِ به‌اتفاقِ پژوهشِ یادگیریِ ماشینیِ امروز بازنماییِ عامل‌بندی‌شده را فرض می‌کند، و تابعِ $h : R^n \to R$ برای رگرسیون و $h : R^n \to \{0,1\}$ برای رده‌بندی یاد می‌گیرد. یادگیریِ ماشینی برای مسائلی که فقط مقدارِ کمی داده دارند، یا مسائلی که مستلزمِ ساختِ بازنمایی‌های ساخت‌یافته، سلسله‌مراتبیِ تازه‌اند، موفقیتِ کمتری داشته. یادگیری عمیق، به‌ویژه با شبکه‌های پیچشیِ اعمال‌شده بر مسائلِ بینواییِ رایانه، موفقیتی در رفتن از پیکسل‌های سطح‌پایین به مفاهیمِ سطحِ میانی مانندِ چشم و دهان، سپس چهره، و سرانجام شخص یا گربه نشان داده.

چالشی برای آینده ترکیبِ روان‌ترِ یادگیری و دانشِ قبلی است. اگر مسئله‌ای به رایانه بدهیم که قبلاً با آن روبه‌رو نشده—مثلاً، بازشناسیِ مدل‌های متفاوتِ خودرو—نمی‌خواهیم سامانه تا وقتی میلیون‌ها مثالِ برچسب‌خورده تغذیه نشده ناتوان باشد. سامانهٔ ایده‌آل باید بتواند بر آنچه از قبل می‌داند تکیه کند: باید از قبل مدلی از اینکه بینایی چگونه کار

می‌کند، و طراحی و برندسازیِ محصولات به‌طورِ کلی چگونه کار می‌کند داشته باشد؛ حالا باید از یادگیریِ انتقالی برای اعمالِ آن بر مسئلهٔ تازهٔ مدل‌های خودرو استفاده کند. باید بتواند به‌تنهایی اطلاعاتی دربارهٔ مدل‌های خودرو بیابد، از متن، تصاویر و ویدئویِ موجود در اینترنت. باید قادر به یادگیریِ کارآموزی باشد: گفتگویی با معلم داشته باشد، و نه فقط «می‌شود هزار تصویر از کورولا داشته باشم» بپرسد، بلکه بتواند توصیه‌ای مانندِ «این سایت شبیهِ پریوس است، اما سایت جلوپنجرهٔ بزرگ‌تری دارد» را بفهمد. باید بداند هر مدل در دامنهٔ کوچکی از رنگ‌های ممکن می‌آید، اما خودرو را می‌توان دوباره رنگ کرد، پس شانسی هست خودرویی در رنگی ببیند که در مجموعهٔ آموزشی نبود. (اگر آن را نمی‌دانست، باید قادر به یادگیریِ آن، یا شنیدنِ آن، باشد.) همهٔ این‌ها به زبانِ ارتباط و بازنماییِ مشترکِ انسان‌ها و رایانه‌ها نیاز دارد؛ نمی‌توانیم انتظار داشته باشیم تحلیلگرِ انسانی مستقیماً مدلی با میلیون‌ها وزن را اصلاح کند. مدل‌های احتمالاتی (از جملهِ زبان‌های برنامه‌نویسیِ احتمالاتی) به انسان‌ها توانایی توصیفِ آنچه می‌دانند می‌دهند، اما این مدل‌ها هنوز با دیگر سازوکارهای یادگیری خوب یکپارچه نشده‌اند. کارِ بنجیو و لکون (2007) گامی به سمتِ این یکپارچگی است. اخیراً یان لکون پیشنهاد کرده اصطلاحِ «یادگیری عمیق» باید با «برنامه‌نویسیِ مشتق‌پذیر»ی (Differentiable Programming) عمومی‌تر جایگزین شود (سیسکیند و پرلموتر، 2016؛ لی و همکاران، 2018)؛ این پیشنهاد می‌کند زبان‌های برنامه‌نویسیِ عمومیِ ما و مدل‌های یادگیریِ ماشینیِ ما می‌توانند ادغام شوند. در حالِ حاضر، ساختنِ مدلِ یادگیریِ عمیقی که مشتق‌پذیر است، و بنابراین می‌تواند برای کمینه‌کردنِ زیان آموزش ببیند و وقتی شرایط تغییر می‌کند بازآموزی شود، رایج است. اما آن مدلِ یادگیریِ عمیق فقط بخشی از سامانهٔ نرم‌افزاریِ بزرگ‌تری است که داده می‌گیرد، داده را ور می‌دهد، به مدل می‌خوراند، و می‌فهمد با خروجیِ مدل چه کند. همهٔ این بخش‌های دیگرِ سامانهٔ بزرگ‌تر توسط برنامه‌نویسی با دست نوشته شده‌اند، و بنابراین غیرمشتق‌پذیرند، که به این معناست وقتی شرایط تغییر می‌کند، این به عهدهٔ برنامه‌نویس است مشکلات را تشخیص دهد و با دست رفع کند. با برنامه‌نویسیِ مشتق‌پذیر، امید این است کلِ سامانه تابعِ بهینه‌سازیِ خودکار باشد.

هدفِ نهایی این است بتوانیم آنچه می‌دانیم را به هر شکلی که برایمان راحت است بیان کنیم: توصیهٔ غیررسمی که به زبانِ طبیعی داده می‌شود، قانونِ ریاضیِ قوی مانندِ $F = ma$، مدلِ آماریِ همراه با داده، یا برنامهٔ احتمالاتی با پارامترهای ناشناخته که می‌تواند از طریقِ نزولِ گرادیان به‌صورتِ خودکار بهینه شود. مدل‌های رایانه‌ایِ ما از گفتگو با کارشناسانِ انسانی و همچنین با استفاده از همهٔ دادهٔ در دسترس یاد خواهند گرفت.

یان لکون، جفری هینتون و دیگران پیشنهاد کرده‌اند تأکیدِ کنونی بر یادگیریِ نظارت‌شده (و تا حدی کمتر، یادگیریِ تقویتی) پایدار نیست—مدل‌های رایانه‌ای باید به «یادگیریِ ضعیف‌نظارت» تکیه کنند، که در آن نظارتی با تعدادِ کمی مثالِ برچسب‌خورده و/یا تعدادِ کمی پاداش داده می‌شود، اما بیشترِ یادگیری نظارت‌نشده است، زیرا دادهٔ بدونِ حاشیه‌نویسی بسیار فراوان‌تر است.

لکون از اصطلاحِ «یادگیریِ پیش‌بینانه» (Predictive Learning) برای سامانهٔ یادگیریِ نظارت‌نشده‌ای استفاده می‌کند که می‌تواند جهان را مدل کند و یاد بگیرد جنبه‌هایی از حالت‌های آیندهٔ جهان را پیش‌بینی کند—نه فقط برچسب‌هایی برای ورودی‌های مستقل و هم‌توزیع با دادهٔ گذشته پیش‌بینی کند، و نه فقط تابعِ ارزشی رویِ حالت‌ها. پیشنهاد می‌کند GANها (شبکه‌های مولدِ تخاصمی) را می‌توان برای یادگیریِ کمینه‌کردنِ تفاوتِ بینِ پیش‌بینی‌ها و واقعیت استفاده کرد.

جفری هینتون در 2017 گفت «دیدگاهِ من این است همه‌چیز را دور بریز و از نو شروع کن»، یعنی ایدهٔ کلیِ یادگیری با تنظیمِ پارامترها در شبکه ماندگار است، اما جزئیاتِ معماریِ شبکه‌ها و فنِ پسانتشار باید بازاندیشی شود. اسمولنسکی (1988) نسخه‌ای برای نحوهٔ اندیشیدن دربارهٔ مدل‌های پیوندگرا داشت؛ اندیشه‌هایش امروز همچنان مرتبط‌اند.

منابع

پژوهش و توسعهٔ یادگیریِ ماشینی با افزایشِ در دسترس‌بودنِ داده، ذخیره، قدرتِ پردازش، نرم‌افزار، کارشناسانِ آموزش‌دیده و سرمایه‌گذاری‌های لازم برای پشتیبانی از آن‌ها شتاب گرفته. از دههٔ ۱۹۷۰، ۱۰۰,۰۰۰ برابر سرعت‌بخشی در پردازنده‌های همه‌منظوره و ۱,۰۰۰ برابرِ اضافی به‌دلیلِ سخت‌افزارِ تخصصیِ یادگیریِ ماشینی بوده. وب به‌عنوانِ منبعِ غنیِ تصاویر، ویدئو، گفتار، متن و دادهٔ نیمه‌ساخت‌یافته خدمت کرده، که در حالِ حاضر روزانه بیش از 10^{18} بایت اضافه می‌کند.

صدها مجموعهٔ دادهٔ باکیفیت برای دامنه‌ای از تکلیف‌ها در بینایی رایانه، بازشناسیِ گفتار و پردازشِ زبانِ طبیعی در دسترس است. اگر دادهٔ موردنیاز از قبل موجود نباشد، اغلب می‌توانید آن را از منابعِ دیگر جمع کنید، یا از طریقِ پلتفرمِ جمع‌سپاری انسان‌هایی را برای برچسب‌زدنِ داده برایتان استخدام کنید. اعتبارسنجیِ دادهٔ به‌دست‌آمده به این‌شیوه بخشِ مهمی از جریانِ کارِ کلی می‌شود (هیرث و همکاران، 2013).

توسعهٔ مهمِ اخیر تغییر از دادهٔ مشترک به «مدلِ مشترک» (Shared Model) است. ارائه‌دهندگانِ عمدهٔ خدماتِ ابری (مثلاً، آمازون، مایکروسافت، گوگل، علی‌بابا، IBM، سیلزفورس) شروع به رقابت در ارائهٔ APIهای یادگیریِ ماشینی با مدل‌های ازپیش‌ساخته برای تکلیف‌های خاص مانندِ بازشناسیِ اشیای بصری، بازشناسیِ گفتار و ترجمهٔ ماشینی کرده‌اند. این مدل‌ها را می‌توان همان‌گونه استفاده کرد، یا می‌توانند به‌عنوانِ خطِّ پایه‌ای برای سفارشی‌سازی با دادهٔ خاصِ شما برای کاربردِ خاصِ شما خدمت کنند. انتظار داریم این مدل‌ها در طولِ زمان بهبود یابند، و غیرعادی شود پروژهٔ یادگیریِ ماشینی را از صفر شروع کنیم، درست همان‌گونه که اکنون غیرعادی است پروژهٔ توسعهٔ وب را بدونِ کتابخانه از صفر انجام دهیم. ممکن است جهشِ بزرگی در کیفیتِ مدل رخ دهد وقتی پردازشِ همهٔ ویدئویِ وب اقتصادی شود؛ برای نمونه، پلتفرمِ یوتیوب به‌تنهایی هر دقیقه ۳۰۰ ساعت ویدئو اضافه می‌کند.

قانونِ مور پردازشِ داده را مقرون‌به‌صرفه‌تر کرده؛ یک مگابایت ذخیره در 1969 یک میلیون دلار و در 2019 کمتر از ۰٫۰۲ دلار هزینه داشت، و توانِ گذردهیِ ابررایانه در آن زمان با ضریبِ بیش از 10^{0} افزایش یافته. اجزای سخت‌افزاریِ تخصصی برای یادگیریِ ماشینی مانندِ واحدهای پردازشِ گرافیکی (GPU)، هسته‌های تانسوری، واحدهای پردازشِ تانسوری (TPU) و آرایه‌های گیتِ برنامه‌پذیرِ میدانی (FPGA) برای آموزشِ یادگیریِ ماشینی صدها بار سریع‌تر از CPUهای معمولی‌اند (واسیلاچه و همکاران، 2014؛ جوپی و همکاران، 2017). در 2014 یک روز کامل طول می‌کشید تا مدلِ ImageNet آموزش ببیند؛ در 2018 فقط ۲ دقیقه (ییِنگ و همکاران، 2018).

مؤسسهٔ OpenAI گزارش می‌دهد مقدارِ قدرتِ محاسباتیِ استفاده‌شده برای آموزشِ بزرگ‌ترین مدل‌های یادگیریِ ماشینی از 2012 تا 2018 هر ۳٫۵ ماه دو برابر شد، و برای آلفازیرو به بیش از یک اگزافلاپ/ثانیه-روز رسید (هرچند آن‌ها همچنین گزارش می‌کنند برخی کارهای بسیار تأثیرگذار از ۱۰۰ میلیون بار قدرتِ محاسباتیِ کمتری استفاده کردند

(آمودی و هرناندز، 2018)). همان روندهای اقتصادی که دوربین‌های گوشیِ همراه را ارزان‌تر و بهتر کرده‌اند برای پردازنده‌ها نیز برقرارند—پیشرفتِ ادامه‌دار در محاسباتِ کم‌مصرف و پرقدرت خواهیم دید که از صرفه‌های مقیاس بهره می‌برد.

این امکان هست رایانه‌های کوانتومی هوش مصنوعی را شتاب دهند. در حالِ حاضر برخی الگوریتم‌های کوانتومیِ سریع برای عملیاتِ جبرِ خطیِ استفاده‌شده در یادگیریِ ماشینی هست (هارو و همکاران، 2009؛ درووِیچ و همکاران، 2018)، اما هیچ رایانهٔ کوانتومیِ قادر به اجرایِ آن‌ها نیست. نمونه‌هایی از کاربردهای تکلیف‌هاییِ مانندِ رده‌بندیِ تصویر (موت و همکاران، 2017) داریم که الگوریتم‌های کوانتومی در مسائلِ کوچک به همان خوبیِ الگوریتم‌های کلاسیک‌اند. رایانه‌های کوانتومیِ کنونی فقط چند ده بیت مدیریت می‌کنند، در حالی که الگوریتم‌های یادگیریِ ماشینی اغلب ورودی‌هایی با میلیون‌ها بیت مدیریت و مدل‌هایی با صدها میلیون پارامتر ایجاد می‌کنند. پس برای عملی‌کردنِ محاسباتِ کوانتومی برای یادگیریِ ماشینیِ مقیاس‌بزرگ به پیشرفت‌هایی در هر دوِ سخت‌افزار و نرم‌افزارِ کوانتومی نیاز داریم. یا، ممکن است تقسیمِ کاری باشد—شاید الگوریتمِ کوانتومی برای جستجویِ کارآمدِ فضایِ ابرپارامترها در حالی که فرایندِ آموزشِ معمولی رویِ رایانه‌های معمولی اجرا می‌شود—اما هنوز نمی‌دانیم چگونه آن را انجام دهیم. پژوهشِ الگوریتم‌های کوانتومی گاهی می‌تواند الگوریتم‌های تازه و بهتری را رویِ رایانه‌های کلاسیک الهام کند (تانگ، 2018).

همچنین رشدِ نمایی در تعدادِ انتشارات، افراد و دلارها در هوش مصنوعی/یادگیریِ ماشینی/علمِ داده دیده‌ایم. دین و همکاران (2018) نشان می‌دهند تعدادِ مقاله‌های «یادگیریِ ماشینی» رویِ arXiv از 2009 تا 2017 هر دو سال دو برابر شد. سرمایه‌گذاران در حالِ تأمین‌مالیِ شرکت‌های استارتاپی در این حوزه‌ها هستند، شرکت‌های بزرگ در حالِ استخدام و هزینه‌کردن‌اند، و دولت‌ها سرمایه‌گذاری می‌کنند تا مطمئن شوند کشورشان...

۲۸.۲ معماری‌های هوش مصنوعی

پرسیدنِ اینکه «کدام یک از معماری‌های عاملِ فصل ۲ را یک عامل باید استفاده کند؟» طبیعی است. پاسخ این است: «همهٔ آن‌ها!» پاسخ‌های بازتابی برای موقعیت‌هایی لازم‌اند که زمان در آن‌ها حیاتی است، در حالی که تعمقِ دانش‌محور به عامل اجازه می‌دهد از پیش برنامه‌ریزی کند. یادگیری وقتی دادهٔ زیادی داریم راحت است، و وقتی محیط در حالِ تغییر است، یا طراحانِ انسانی دانشِ کافی از حوزه ندارند، ضروری است.

هوش مصنوعی مدتِ طولانی است بینِ سامانه‌های نمادین (مبتنی بر استنتاجِ منطقی و احتمالاتی) و سامانه‌های پیوندگرا (مبتنی بر کمینه‌سازیِ زیان رویِ تعدادِ زیادی پارامترِ تفسیرنشده) شکاف داشته. چالشِ ادامه‌دارِ هوش مصنوعی گردآوردنِ این دو، برای دربرگرفتنِ بهترین هر دو، است. سامانه‌های نمادین به ما اجازه می‌دهند زنجیره‌های بلندی از استدلال به هم بندیم و از قدرتِ بیانیِ بازنمایی‌های ساخت‌یافته بهره بگیریم، در حالی که سامانه‌های پیوندگرا می‌توانند الگوها را حتی در مواجهه با دادهٔ نویزی بازشناسی کنند. یکی از خطوطِ پژوهش هدفش ترکیبِ برنامه‌نویسیِ احتمالاتی با یادگیری عمیق است، هرچند تاکنون پیشنهادهای گوناگون در میزانی که رویکردها واقعاً ادغام می‌شوند محدودند.

عامل‌ها همچنین به راه‌هایی برای کنترلِ تعمق‌های خودشان نیاز دارند. باید بتوانند از زمانِ در دسترس خوب استفاده کنند، و وقتی کنش طلبیده می‌شود تعمق را متوقف کنند. برای نمونه، عاملِ رانندهٔ تاکسی که تصادفی را پیشِ رو می‌بیند باید در کسری از ثانیه تصمیم بگیرد ترمز کند یا منحرف شود. همچنین باید آن کسرِ ثانیه را صرفِ فکرکردن به مهم‌ترین پرسش‌ها کند، مانندِ اینکه آیا خطوطِ چپ و راست بازند و آیا کامیونِ بزرگی نزدیکِ پشتِ سر هست، نه نگرانی دربارهٔ اینکه مسافرِ بعدی را کجا سوار کند. این مسائل معمولاً تحتِ عنوانِ «هوش مصنوعیِ زمان-واقعی» (Real-Time AI) مطالعه می‌شوند. با حرکتِ سامانه‌های هوش مصنوعی به حوزه‌های پیچیده‌تر، همهٔ مسائل زمان-واقعی خواهند شد، زیرا عامل هرگز زمانِ کافی برای حلِ دقیقِ مسئلهٔ تصمیم نخواهد داشت.

واضح است، نیازِ فوری به روش‌های عمومیِ کنترلِ تعمق هست، نه دستورهای خاصِ اینکه در هر موقعیت به چه فکر کنیم. نخستین ایدهٔ مفید «الگوریتم‌های همیشگی» (Anytime Algorithm) است (دین و بادی، 1988؛ هورویتز، 1987): الگوریتمی که کیفیتِ خروجی‌اش به‌تدریج در طولِ زمان بهبود می‌یابد، تا هر وقت قطع شود تصمیمِ معقولی آماده داشته باشد. نمونه‌های الگوریتم‌های همیشگی شاملِ تعمیقِ تکراری در جستجویِ درختِ بازی و MCMC در شبکه‌های بیزی است. فنِ دوم برای کنترلِ تعمق «فراستدلالِ تصمیم-نظری» (Decision-Theoretic Metareasoning) است (راسل و وفالد، 1989؛ هورویتز و بریس، 1996؛ هی و همکاران، 2012). این روش، که در بخش‌های ۳.۶.۵ و ۵.۷ به‌طورِ مختصر ذکر شد، نظریهٔ ارزشِ اطلاعات (فصل ۱۶) را بر انتخابِ محاسباتِ منفرد اعمال می‌کند. ارزشِ یک محاسبه هم به هزینه‌اش (از نظرِ به‌تأخیرانداختنِ کنش) و هم به مزیت‌هایش (از نظرِ کیفیتِ بهبودیافتهٔ تصمیم) بستگی دارد. فنونِ فراستدلال را می‌توان برای طراحیِ الگوریتم‌های جستجویِ بهتر و برای تضمینِ اینکه الگوریتم‌ها ویژگیِ همیشگی دارند استفاده کرد. جستجویِ درختِ مونت‌کارلو یک نمونه است: انتخابِ گرهٔ برگ که پخشِ بعدی از آن آغاز می‌شود توسط تصمیمِ فراسطحیِ تقریباً عقلاییِ برگرفته از نظریهٔ باندیت انجام می‌شود.

فراستدلال گران‌تر از کنشِ بازتابی است، البته، اما می‌توان روش‌های کامپایل را اعمال کرد تا سربار در مقایسه با هزینه‌های محاسباتِ در حالِ کنترل کوچک باشد. یادگیریِ تقویتیِ فراسطحی ممکن است راهِ دیگری برای کسبِ سیاست‌های مؤثرِ کنترلِ تعمق فراهم کند: در اصل، محاسباتی که به تصمیم‌های بهتر می‌انجامند تقویت می‌شوند، در حالی که آن‌هایی که معلوم می‌شود بی‌اثرند جریمه می‌شوند. این رویکرد از مشکلاتِ کوته‌بینیِ محاسبهٔ سادهٔ ارزشِ اطلاعات اجتناب می‌کند.

فراستدلال یکی از نمونه‌های خاصِ معماریِ بازتابی است—یعنی معماری‌ای که تعمق دربارهٔ موجودیت‌ها و کنش‌های محاسباتیِ رخ‌داده در درونِ خودِ معماری را ممکن می‌کند. بنیانِ نظریِ معماری‌های بازتابی را می‌توان با تعریفِ فضایِ حالتِ مشترکی که از حالتِ محیط و حالتِ محاسباتیِ خودِ عامل ساخته شده بنا کرد. الگوریتم‌های تصمیم‌گیری و یادگیری را می‌توان طراحی کرد که رویِ این فضایِ حالتِ مشترک عمل کنند و بدین‌ترتیب فعالیت‌های محاسباتیِ عامل را پیاده‌سازی و بهبود بخشند. در نهایت، انتظار داریم الگوریتم‌های خاصِ تکلیف مانندِ جستجویِ آلفا-بتا، برنامه‌ریزیِ رگرسیونی و حذفِ متغیر از سامانه‌های هوش مصنوعی ناپدید شوند، و با روش‌های عمومی‌ای جایگزین شوند که محاسباتِ عامل را به سمتِ تولیدِ کارآمدِ تصمیم‌های باکیفیت هدایت می‌کنند.

فراستدلال و بازتاب (و بسیاری از ابزارهای معماری و الگوریتمیِ مرتبط با کارایی که در این کتاب کاوش شده‌اند) لازم‌اند، زیرا تصمیم‌گیری سخت است. از وقتی رایانه‌ها اختراع شدند، سرعتِ خیره‌کننده‌شان مردم را به بیش‌برآوردِ تواناییشان در غلبه بر پیچیدگی، یا به‌طورِ هم‌ارز، کم‌برآوردِ اینکه پیچیدگی واقعاً چه معنایی دارد، کشانده.

قدرتِ واقعاً غول‌پیکرِ ماشین‌های امروز آدم را وسوسه می‌کند فکر کند می‌توانیم از همهٔ ابزارهای هوشمندانه دور بزنیم و بیشتر بر نیرویِ خام تکیه کنیم. پس بیایید بکوشیم با این گرایش مقابله کنیم. با آنچه فیزیکدانان باور دارند سرعتِ دستگاهِ محاسباتیِ نهاییِ ۱ کیلوگرمی است شروع می‌کنیم:

حدودِ 10^{51} عملیات در ثانیه، یا یک میلیارد تریلیون تریلیون بار سریع‌تر از سریع‌ترین ابررایانهٔ تا 2020 (لوید، 2000). سپس تکلیفِ ساده‌ای پیشنهاد می‌کنیم: برشمردنِ رشته‌های واژه‌های انگلیسی، همان‌گونه که بورخس در «کتابخانهٔ بابل» پیشنهاد کرد. بورخس کتاب‌هایی با ۴۱۰ صفحه تصریح کرد. آیا آن شدنی می‌بود؟ نه کاملاً. در واقع، رایانه‌ای که یک سال اجرا شود فقط می‌توانست رشته‌های ۱۱-واژه‌ای را برشمارد.

حالا این واقعیت را در نظر بگیر که برنامهٔ مفصلی برای زندگیِ انسانی شاملِ (تقریباً) بیست تریلیون فعال‌سازیِ ماهیچه‌ایِ بالقوه است (راسل، 2019)، و شروع به دیدنِ مقیاسِ مسئله می‌کنی. رایانه‌ای که یک میلیارد تریلیون تریلیون بار قدرتمندتر از مغزِ انسان است از عقلایی‌بودن بسیار دورتر است تا لخت‌پوستی از سبقت‌گرفتن از سفینهٔ فضاییِ انترپرایزِ در حالِ حرکت با وارپِ نه.

با این ملاحظات در ذهن، به نظر می‌رسد هدفِ ساختنِ عاملِ عقلایی شاید کمی بیش‌ازحد جاه‌طلبانه است. به‌جای هدف‌گرفتنِ چیزی که نمی‌تواند وجود داشته باشد، باید هدفِ هنجاریِ متفاوتی را در نظر بگیریم—هدفی که لزوماً وجود دارد. از فصل ۲ ایدهٔ سادهٔ زیر را به یاد آور:

> ***عامل = معماری + برنامه***

حالا معماریِ عامل (قابلیت‌های ماشینِ زیرین، شاید با لایهٔ نرم‌افزاریِ ثابتی رویِ آن) را ثابت کن و به برنامهٔ عامل اجازه بده رویِ همهٔ برنامه‌های ممکنی که معماری می‌تواند پشتیبانی کند تغییر کند. در هر محیطِ تکلیفیِ داده‌شده، یکی از این برنامه‌ها (یا ردهٔ هم‌ارزی از آن‌ها) بهترین عملکردِ ممکن را تحویل می‌دهد—شاید نزدیکِ عقلانیتِ کامل نباشد، اما همچنان بهتر از هر برنامهٔ عاملِ دیگر. می‌گوییم این برنامه معیارِ «بهینگیِ کران‌دار» (Bounded Optimality) را برآورده می‌کند. واضح است وجود دارد، و واضح است هدفِ مطلوبی را تشکیل می‌دهد. ترفند یافتنِ آن، یا چیزی نزدیکِ آن، است.

برای برخی رده‌های مقدماتیِ برنامه‌های عامل در محیط‌های سادهٔ زمان-واقعی، شناساییِ برنامه‌های عاملِ بهینهٔ کران‌دار ممکن است (اتزیونی، 1989؛ راسل و سوبرامانیان، 1995). موفقیتِ جستجویِ درختِ مونت‌کارلو علاقه به تصمیم‌گیریِ فراسطحی را زنده کرده، و دلیلی برای امیدواری هست که بهینگیِ کران‌دار درونِ خانواده‌های پیچیده‌ترِ برنامه‌های عامل با فنونی مانندِ یادگیریِ تقویتیِ فراسطحی محقق شود. همچنین باید بتوان نظریهٔ سازنده‌ای از معماری توسعه داد، که با قضیه‌هایی دربارهٔ بهینگیِ کران‌دارِ روش‌های مناسبِ ترکیبِ مؤلفه‌های بهینهٔ کران‌دارِ متفاوت مانندِ سامانه‌های بازتابی و کنش-ارزش آغاز می‌شود.

هوش مصنوعیِ عمومی

بخشِ زیادی از پیشرفتِ هوش مصنوعی در قرنِ بیست‌ویکم تاکنون توسط رقابت رویِ تکلیف‌های باریک هدایت شده، مانندِ چالشِ بزرگِ DARPA برای خودروهای خودمختار، رقابتِ بازشناسیِ اشیای ImageNet، یا بازیِ گو، شطرنج، پوکر یا Jeopardy! در برابرِ قهرمانِ جهان. برای هر تکلیفِ جداگانه، سامانهٔ هوش مصنوعیِ جداگانه‌ای می‌سازیم، معمولاً با مدلِ یادگیریِ ماشینیِ جداگانه‌ای که از صفر با داده‌ای که مخصوصِ این تکلیف جمع‌آوری شده آموزش می‌بیند. اما عاملِ واقعاً هوشمند باید بتواند بیش از یک کار انجام دهد. آلن تورینگ (1950) فهرستش را پیشنهاد کرد (صفحهٔ ۹۸۲) و نویسندهٔ علمی-تخیلیِ رابرت هاینلاین (1973) مقابله کرد:[1]

> *انسان باید بتواند پوشک عوض کند، حمله برنامه‌ریزی کند، خوک سلاخی کند، کشتی را هدایت کند، ساختمان طراحی کند، غزل بنویسد، حساب‌ها را تراز کند، دیوار بسازد، استخوان جا بیندازد، داغدیدگان را دلداری دهد، فرمان بگیرد، فرمان دهد، همکاری کند، تنها عمل کند، معادله حل کند، مسئلهٔ تازه تحلیل کند، کود بریزد، رایانه برنامه‌نویسی کند، غذای خوشمزه بپزد، کارآمد بجنگد، شجاعانه بمیرد. تخصص‌گرایی برای حشرات است.*
>
> ***— رابرت هاین‌لاین (1973)***

تاکنون، هیچ سامانهٔ هوش مصنوعی‌ای به هیچ‌کدام از این دو فهرست نمی‌رسد، و برخی طرفدارانِ هوش مصنوعیِ عمومی یا سطح-انسان (HLAI) اصرار می‌ورزند کارِ ادامه‌دار رویِ تکلیف‌های خاص (یا رویِ مؤلفه‌های منفرد) برای رسیدن به تسلط رویِ طیفِ وسیعی از تکلیف‌ها کافی نخواهد بود؛ به رویکردِ بنیاداً تازه‌ای نیاز خواهیم داشت. به نظر ما پیشرفت‌های تازهٔ متعددی در واقع لازم خواهد بود، اما در مجموع، هوش مصنوعی به‌عنوانِ حوزه مبادلهٔ کاوش/بهره‌برداریِ معقولی کرده، سبدی از مؤلفه‌ها گرد آورده، تکلیف‌های خاص را بهبود بخشیده، و در عین حال ایده‌های تازهٔ امیدوارکننده و گاهی دور از دسترس را کاوش کرده.

اشتباه می‌بود در 1903 به برادرانِ رایت گفتنِ اینکه کار رویِ هواپیمایِ تک‌تکلیفی‌شان را متوقف کنند و «ماشینِ پروازِ مصنوعیِ عمومی»ای طراحی کنند که می‌تواند عمودی بلند شود، سریع‌تر از صدا پرواز کند، صدها مسافر حمل کند و رویِ ماه فرود آید. همچنین اشتباه می‌بود دنبالِ نخستین پروازشان با رقابتِ سالانه‌ای برای بهترکردنِ تدریجیِ هواپیماهای دوبالهٔ چوبِ صنوبر می‌رفتیم.

دیده‌ایم کار رویِ مؤلفه‌ها می‌تواند ایده‌های تازه برانگیزد؛ برای نمونه، شبکه‌های مولدِ تخاصمی (GAN) و مدل‌های زبانِ ترنسفورمر هر یک حوزه‌های پژوهشیِ تازه‌ای گشودند. همچنین گام‌هایی به سمتِ «تنوعِ رفتار» دیده‌ایم. برای نمونه، سامانه‌های ترجمهٔ ماشینی در دههٔ ۱۹۹۰ یکی‌یکی برای هر جفت‌زبان (مانندِ فرانسوی به انگلیسی) ساخته می‌شدند، اما امروزه سامانهٔ تکی می‌تواند متنِ ورودی را به‌عنوانِ یکی از صد زبان شناسایی و به هر یک از ۱۰۰ زبانِ مقصد ترجمه کند. سامانهٔ زبانِ طبیعیِ دیگری می‌تواند پنج تکلیفِ متمایز را با یک مدلِ مشترک انجام دهد (هاشیموتو و همکاران، 2016).

مهندسیِ هوش مصنوعی

حوزهٔ برنامه‌نویسیِ رایانه با چند پیشگامِ خارق‌العاده شروع شد. اما تا وقتی رویه‌ای از مهندسیِ نرم‌افزار توسعه نیافته بود، با مجموعه‌ای قدرتمند از ابزارهای گسترده‌در دسترس و اکوسیستمِ شکوفایی از معلمان، دانشجویان، دست‌اندرکاران، کارآفرینان، سرمایه‌گذاران و مشتریان، به جایگاهِ صنعتِ بزرگی نرسید.

صنعتِ هوش مصنوعی هنوز به آن سطحِ بلوغ نرسیده. انواعِ گوناگونی از ابزارها و چارچوب‌های قدرتمند داریم، مانندِ Scikit-Learn، CAFFE، PyTorch، Keras، TensorFlow و SCIPY. اما بسیاری از امیدوارکننده‌ترین رویکردها، مانندِ GANها و یادگیریِ تقویتیِ عمیق، کارکردن با آن‌ها دشوار ثابت شده—به تجربه و درجه‌ای ورزفتن نیاز دارند تا در حوزهٔ تازه‌ای درست آموزش ببینند. به‌اندازهٔ کافی کارشناس نداریم که این را در همهٔ حوزه‌هایی که نیاز داریم انجام دهیم، و هنوز ابزارها و اکوسیستمِ لازم برای موفقیتِ دست‌اندرکارانِ کم‌تجربه‌تر را نداریم.

جف دینِ گوگل آینده‌ای می‌بیند که در آن می‌خواهیم یادگیریِ ماشینی میلیون‌ها تکلیف را مدیریت کند؛ توسعهٔ هر یک از صفر شدنی نخواهد بود، پس پیشنهاد می‌کند به‌جای ساختنِ هر سامانهٔ تازه از صفر، با یک سامانهٔ عظیمِ تکی شروع کنیم و برای هر تکلیفِ تازه، بخش‌های مربوط به تکلیف را از آن استخراج کنیم. گام‌هایی در این جهت دیده‌ایم، مانندِ مدل‌های زبانِ ترنسفورمر (مثلاً، GPT-2، BERT) با میلیاردها پارامتر، و معماریِ شبکهٔ عصبیِ گروهیِ «به‌شکلِ توهین‌آمیزی بزرگ» که در یک آزمایش تا ۶۸ میلیارد پارامتر مقیاس می‌شود (شازیر و همکاران، 2017). کارِ بسیار باقی مانده.

آینده

آینده به کدام سمت خواهد رفت؟ نویسندگانِ علمی-تخیلی به نظر می‌رسد آینده‌های ویران‌شهری را بر مدینه‌های فاضله ترجیح می‌دهند، احتمالاً چون داستان‌های جالب‌تری می‌سازند. تاکنون، هوش مصنوعی به نظر می‌رسد با دیگر فناوری‌های انقلابیِ قدرتمند مانندِ چاپ، لوله‌کشی، سفرِ هوایی و تلفن جور در می‌آید. همهٔ این فناوری‌ها اثرهای مثبتی داشته‌اند، اما همچنین برخی اثرهای جانبیِ ناخواسته دارند که به‌صورتِ نامتناسب بر طبقاتِ محروم اثر می‌گذارد. خوب است در سرمایه‌گذاری برای کمینه‌کردنِ اثرهای منفی سرمایه‌گذاری کنیم.

هوش مصنوعی همچنین با فناوری‌های انقلابیِ قبلی متفاوت است. بهبودِ چاپ، لوله‌کشی، سفرِ هوایی و تلفن تا حدِّ منطقی‌شان هیچ‌چیزِ تهدیدکنندهٔ برتریِ انسانی در جهان تولید نمی‌کرد. بهبودِ هوش مصنوعی تا حدِّ منطقی‌اش قطعاً می‌توانست.

در پایان، هوش مصنوعی در تاریخچهٔ کوتاهش پیشرفتِ بزرگی کرده، اما جملهٔ پایانیِ مقالهٔ آلن تورینگ (1950) دربارهٔ «ماشین‌های محاسبه و هوش» همچنان امروز معتبر است:

> *«ما فقط می‌توانیم فاصله‌ای کوتاه به جلو را ببینیم، اما می‌بینیم که کارِ بسیار باقی مانده است.»*
>
> ***— آلن تورینگ (1950)***

پانوشت‌ها:

[1] از این واقعیت چشم‌پوشی می‌کنیم که این دستگاه کلِ خروجیِ انرژیِ یک ستاره را مصرف می‌کند و در یک میلیارد درجهٔ سانتی‌گراد کار می‌کند.